



Developer Guide for version 2.x

AWS SDK for Java 2.x



AWS SDK for Java 2.x: Developer Guide for version 2.x

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

What is the AWS SDK for Java 2.x	1
Get started with the SDK	1
Develop mobile applications	1
Maintenance and support for SDK major versions	1
Additional resources	2
Contribute to the SDK	2
Getting started	3
Setting up	4
Setup overview	4
Install Java development prerequisites	5
Set up an Apache Maven project	6
Set up a Gradle project	12
Set up a GraalVM Native Image project	18
Authenticating with AWS	20
Set up for authentication	20
Additional authentication options	22
Creating a simple application	22
Step 1: Set up for this tutorial	22
Step 2: Create the project	23
Step 3: Write the code	28
Step 4: Build and run the application	32
Next steps	33
Configuring service clients	34
Client configuration externally	35
Configuration provider chain for client configuration	35
Create a service client configured using external settings	36
SDK for Java 2.x environment variables and JVM system properties	37
Client configuration in code	38
Basic configuration in code	38
Advanced configuration in code	38
Configuration options unavailable in code	40
Singleton service clients	41
Benefits of singleton service clients	41
Create and use singleton service clients	42

Important considerations	42
AWS Region	43
Explicitly configure an AWS Region	43
Determine Region from environment	44
Check for service availability	45
Choose a specific endpoint	46
Credentials providers	46
Interactive development work	47
Default credentials provider chain	50
Credentials caching	55
Specify a specific credentials provider	58
Use shared configuration profiles	59
Use an external process	62
Supply credentials in code	67
Read IAM role credentials on Amazon EC2	71
Retries	72
Retry strategies	73
Specify a strategy	77
Customize a strategy	78
Migrating from RetryPolicy to RetryStrategy	80
Timeouts	80
Service client timeouts	81
HTTP client timeouts	83
Timeout interactions and hierarchy	84
Use smart configuration defaults	85
Summary	86
Observability	86
Metrics	87
Monitoring	120
Logging	120
Endpoints	131
Endpoint configuration options	132
In-code endpoint configuration	132
Request-level endpoint configuration	133
External endpoint configuration	134
Configuration precedence	135

Service-specific endpoint configuration	135
Best practices	137
Configure HTTP clients	137
Available clients	137
Client recommendations	139
Smart defaults	141
Configure the Apache-based HTTP client	143
Configure the URLConnection-based HTTP client	148
Configure the Netty-based HTTP client	155
Configure AWS CRT-based HTTP clients	162
Configure HTTP proxies	175
Configure the Apache 5.x based HTTP client	180
Interceptors	183
Lifecycle	183
Register interceptors	184
Example	184
Best practices	192
Context objects	192
Using the SDK	193
Making AWS service requests	196
Using service clients to make requests	196
Make requests	198
Handle responses	200
Asynchronous programming	201
Use asynchronous client APIs	201
Handle streaming in asynchronous methods	204
Configure advanced asynchronous options	208
Best practices	209
Prevent hanging requests	209
Reuse clients	210
Release resources	211
Prevent connection issues	211
HTTP optimization	211
SSL optimization	211
Monitor performance	212
Handling errors	212

Why unchecked exceptions?	212
AwsServiceException (and subclasses)	213
SdkClientException	214
Exceptions and retry behavior	214
Pagination	214
Synchronous pagination	215
Asynchronous pagination	217
Waiters	223
Prerequisites	223
Using waiters	224
Configure waiters	224
Code examples	225
Troubleshooting	226
Connection reset	226
Connection timeout	226
Read timed out	227
Connection pool timeout	227
Classpath errors	231
Signature errors	232
Connection pool shut down	233
Credentials provider chain error	235
Reduce SDK startup time for AWS Lambda	238
Use an AWS CRT-based HTTP client	238
Remove unused HTTP client dependencies	238
Configure service clients to shortcut lookups	239
Initialize the SDK client outside of the Lambda function handler	240
Minimize dependency injection	241
Use a Maven Archetype targeting AWS Lambda	241
Lambda SnapStart	241
Version 2.x changes that affect startup time	242
Additional resources	242
Implement ContentStreamProvider	242
Use mark() and reset()	243
Use buffering if mark() and reset() are not available	243
Create new streams	244
Set the JVM TTL for DNS name lookups	244

How to set the JVM TTL	244
Work with HTTP/2	246
Calling AWS services	247
CloudWatch	247
Get metrics from CloudWatch	248
Publish custom metric data to CloudWatch	250
Work with CloudWatch alarms	252
Use Amazon CloudWatch Events	256
AWS database services	259
Amazon DynamoDB	260
Amazon RDS	260
Amazon Redshift	261
Amazon Aurora Serverless v1	261
Amazon DocumentDB	261
DynamoDB	262
Choose your DynamoDB client	262
Configure DynamoDB clients	262
What's covered in this topic	263
Work with tables in DynamoDB	264
Work with items in DynamoDB	273
Map objects to DynamoDB items	280
Amazon EC2	419
Manage Amazon EC2 instances	419
Use AWS Regions and Availability Zones	426
Work with security groups in Amazon EC2	433
Work with Amazon EC2 instance metadata	438
IAM	444
Manage IAM access keys	444
Manage IAM Users	450
Create IAM policies	455
Work with IAM policies	463
Work with IAM server certificates	469
Kinesis	474
Subscribe to Amazon Kinesis Data Streams	474
Lambda	485
Invoke a Lambda function	485

List Lambda functions	486
Delete a Lambda function	487
Amazon S3	488
S3 clients in the SDK	488
Uploading streams to S3	492
Pre-signed URLs	498
Cross-Region access	507
Data integrity protection with checksums	508
Use a performant S3 client	516
Configure parallel transfer support	520
Transfer files and directories	523
S3 Event Notifications	533
Amazon SNS	543
Create a topic	543
List your Amazon SNS topics	544
Subscribe an endpoint to a topic	545
Publish a message to a topic	546
Unsubscribe an endpoint from a topic	547
Delete a topic	548
Amazon SQS	549
Use automatic request batching	549
Queue operations	555
Message operations	558
Amazon Transcribe	562
Set up the microphone	562
Create a publisher	563
Create the client and start the stream	565
More information	561
Code examples	568
ACM	571
Actions	571
API Gateway	589
Actions	571
Scenarios	593
AWS community contributions	594
Application Auto Scaling	595

Actions	571
Application Recovery Controller	603
Actions	571
Aurora	606
Get started	606
Basics	608
Actions	571
Scenarios	593
Auto Scaling	642
Get started	606
Basics	608
Actions	571
Scenarios	593
AWS Batch	706
Get started	606
Basics	608
Actions	571
Amazon Bedrock	755
Actions	571
Amazon Bedrock Runtime	760
Scenarios	593
Amazon Nova	778
Amazon Nova Canvas	798
Amazon Titan Image Generator	801
Amazon Titan Text Embeddings	804
Anthropic Claude	807
Cohere Command	826
Meta Llama	837
Mistral AI	848
Stable Diffusion	858
CloudFront	861
Actions	571
Scenarios	593
CloudWatch	891
Get started	606
Basics	608

Actions	571
Scenarios	593
CloudWatch Events	966
Actions	571
CloudWatch Logs	972
Actions	571
Scenarios	593
Amazon Cognito Identity	991
Actions	571
Amazon Cognito Identity Provider	998
Get started	606
Actions	571
Scenarios	593
Amazon Comprehend	1025
Actions	571
Scenarios	593
AWS Control Tower	1037
Get started	606
Basics	608
Actions	571
Firehose	1098
Actions	571
Scenarios	593
Amazon DocumentDB	1108
Serverless examples	1108
DynamoDB	1110
Get started	606
Basics	608
Actions	571
Scenarios	593
Serverless examples	1108
AWS community contributions	594
Amazon EC2	1370
Get started	606
Basics	608
Actions	571

Scenarios	593
Amazon ECR	1467
Get started	606
Basics	608
Actions	571
Amazon ECS	1507
Actions	571
Elastic Load Balancing - Version 2	1521
Get started	606
Actions	571
Scenarios	593
MediaStore	1565
Actions	571
AWS Entity Resolution	1580
Get started	606
Basics	608
Actions	571
OpenSearch Service	1628
Get started	606
Basics	608
Actions	571
EventBridge	1657
Get started	606
Basics	608
Actions	571
Scenarios	593
EventBridge Scheduler	1692
Get started	606
Actions	571
Scenarios	593
Forecast	1717
Actions	571
Amazon Glacier	1730
Actions	571
AWS Glue	1746
Get started	606

Basics	608
Actions	571
HealthImaging	1778
Actions	571
Scenarios	593
IAM	1808
Get started	606
Basics	608
Actions	571
Scenarios	593
Amazon Inspector	1892
Get started	606
Basics	608
Actions	571
AWS IoT	1940
Get started	606
Basics	608
Actions	571
AWS IoT data	1979
Actions	571
AWS IoT SiteWise	1982
Get started	606
Basics	608
Actions	571
Amazon Keyspaces	2021
Get started	606
Basics	608
Actions	571
Kinesis	2047
Actions	571
Serverless examples	1108
AWS KMS	2060
Get started	606
Basics	608
Actions	571
Lambda	2116

Get started	606
Basics	608
Actions	571
Scenarios	593
Serverless examples	1108
AWS community contributions	594
Amazon Lex	2153
Scenarios	593
Amazon Location	2153
Get started	606
Basics	608
Actions	571
Location Service Places	2201
Actions	571
AWS Marketplace Catalog API	2206
AMI products	2207
Channel partner offers	2231
Container products	2248
Entities	2254
Offers	2259
Products	2319
Resale authorization	2324
SaaS products	2365
Utilities	2391
AWS Marketplace Agreement API	2395
Agreements	2395
MediaConvert	2448
Actions	571
Migration Hub	2469
Actions	571
Amazon MSK	2482
Serverless examples	1108
Neptune	2484
Get started	606
Basics	608
Actions	571

Scenarios	593
Partner Central	2526
Actions	571
Scenarios	593
Amazon Personalize	2561
Actions	571
Amazon Personalize Events	2591
Actions	571
Amazon Personalize Runtime	2594
Actions	571
Amazon Pinpoint	2599
Actions	571
Amazon Pinpoint SMS and Voice API	2643
Actions	571
Amazon Polly	2646
Actions	571
Scenarios	593
Amazon RDS	2653
Get started	606
Basics	608
Actions	571
Scenarios	593
Serverless examples	1108
Amazon RDS Data Service	2695
Scenarios	593
Amazon Redshift	2697
Get started	606
Basics	608
Actions	571
Scenarios	593
Amazon Rekognition	2735
Actions	571
Scenarios	593
Route 53 domain registration	2807
Get started	606
Basics	608

Actions	571
Amazon S3	2829
Get started	606
Basics	608
Actions	571
Scenarios	593
Serverless examples	1108
Amazon S3 Control	3016
Get started	606
Basics	608
Actions	571
S3 Directory Buckets	3059
Get started	606
Basics	608
Actions	571
Scenarios	593
SageMaker AI	3141
Get started	606
Actions	571
Scenarios	593
Secrets Manager	3170
Actions	571
Amazon SES	3172
Actions	571
Scenarios	593
Amazon SES API v2	3188
Actions	571
Scenarios	593
Amazon SNS	3208
Get started	606
Actions	571
Scenarios	593
Serverless examples	1108
Amazon SQS	3277
Get started	606
Actions	571

Scenarios	593
Serverless examples	1108
Step Functions	3376
Get started	606
Basics	608
Actions	571
Scenarios	593
AWS STS	3400
Actions	571
Support	3403
Get started	606
Basics	608
Actions	571
Systems Manager	3426
Get started	606
Basics	608
Actions	571
Amazon Textract	3469
Actions	571
Scenarios	593
Amazon Transcribe	3480
Actions	571
Scenarios	593
Amazon Transcribe Streaming	3491
Actions	571
Scenarios	593
Amazon Translate	3510
Scenarios	593
X-Ray	3513
Actions	571
Migrate to version 2	3526
What's new in version 2	3526
Find applications using 1.x clients	3527
Use CloudWatch Logs Insights to find applications with 1.x clients	3527
Search application warning-level logs for SDK deprecation	3529
Search source code and dependencies	3529

Inspect deployable Java artifacts	3531
Use CloudTrail Lake to find applications with 1.x clients	3533
How to migrate	3534
Migration tool	3535
Step-by-step instructions	3545
What's different between 1.x and 2.x	3563
Package name change	3563
Adding version 2.x to your project	3564
Immutable POJOs	3564
Setter and getter methods	3565
Model class names	3566
Libraries and utilities	3566
Client changes	3568
Credentials provider changes	3613
Region changes	3622
Operations, requests and responses changes	3624
Exception changes	3633
Service-specific changes	3634
Working with Amazon S3	3640
Profile file changes	3689
External configuration	3690
Waiters	3693
EC2 metadata utility	3698
CloudFront presigning	3705
IAM Policy Builder API	3709
Working with DynamoDB	3715
SQS automatic request batching	3753
Use the SDK for Java 1.x and 2.x side-by-side	3761
Security	3763
Data protection	3763
Transport Layer Security (TLS)	3764
Check TLS versions	3765
Enforce TLS versions	3765
Migrate to TLS 1.2	3766
Identity and Access Management	3766
Audience	3766

Authenticating with identities	3767
Managing access using policies	3768
How AWS services work with IAM	3770
Troubleshooting AWS identity and access	3770
Compliance Validation	3772
Resilience	3772
Infrastructure Security	3773
OpenPGP key	3774
Current key	3774
Previous keys	3780
Document history	3787

What is the AWS SDK for Java 2.x

The AWS SDK for Java provides a Java API for AWS services. Using the SDK, you can build Java applications that work with Amazon S3, Amazon EC2, DynamoDB, and more.

The AWS SDK for Java 2.x is a major rewrite of the version 1.x code base. It's built on top of Java 8+ and adds several frequently requested features. These include support for non-blocking I/O and the ability to plug in a different HTTP implementation at runtime.

We regularly add support for new services to the AWS SDK for Java. For a list of changes and features in a particular version, view the [change log](#).

Get started with the SDK

If you're ready to get hands-on with the SDK, follow the [Getting started](#) tutorial.

To set up your development environment, see [the section called "Setting up"](#).

If you're currently using version 1.x of the SDK for Java, see [Migrate to version 2](#) for specific guidance.

For information on making requests to Amazon S3, DynamoDB, Amazon EC2 and other AWS services, see [Use the SDK for Java](#) and [Work with AWS services](#).

Develop mobile applications

If you're a mobile app developer, Amazon Web Services provides the [AWS Amplify](#) framework.

Maintenance and support for SDK major versions

For information about maintenance and support for SDK major versions and their underlying dependencies, see the following topics in the [AWS SDKs and Tools Reference Guide](#):

- [AWS SDKs and Tools Maintenance Policy](#)
- [AWS SDKs and Tools Version Support Matrix](#)

Additional resources

In addition to this guide, the following are valuable online resources for AWS SDK for Java developers:

- [AWS SDK for Java 2.x API Reference](#)
- [Java developer blog](#)
- [Java development topic in AWS re:Post](#)
- [SDK source](#) on GitHub
- [AWS SDK Code Examples library](#)
- [@awsforjava](#) (Twitter)

Contribute to the SDK

Developers can also contribute feedback through the following channels:

- [Submit SDK issues](#) on GitHub
- Join an informal chat about the SDK on the AWS SDK for Java 2.x [gitter channel](#)

Getting started with the AWS SDK for Java 2.x

The sections in this topic walk you through the essential steps to begin building Java applications that connect to AWS service. The sections cover setting up your development environment with Java and build tools like Maven or Gradle, configuring secure authentication to AWS, and creating your first working application through a hands-on tutorial. This beginner-friendly topic serves as your entry point to AWS development with Java, providing the foundation you'll need before exploring more advanced features.

Contents

- [Setting up the AWS SDK for Java 2.x](#)
 - [Setup overview](#)
 - [Install Java and a build tool to work with the AWS SDK for Java 2.x](#)
 - [Set up an Apache Maven project that uses the AWS SDK for Java 2.x](#)
 - [Prerequisites](#)
 - [Create a Maven project](#)
 - [Configure the Java compiler for Maven](#)
 - [Declare the SDK as a dependency](#)
 - [Set dependencies for SDK modules](#)
 - [Build the entire SDK into your project](#)
 - [Build your project](#)
 - [Set up a Gradle project that uses the AWS SDK for Java 2.x](#)
 - [Set up a GraalVM Native Image project that uses the AWS SDK for Java 2.x](#)
 - [Prerequisites](#)
 - [Create a project using the archetype](#)
 - [Build a native image](#)
- [Authenticating with AWS using the AWS SDK for Java 2.x](#)
 - [Set up for authentication](#)
 - [1. Local development using short-term credentials](#)
 - [2. Setup for single sign-on access for the SDK](#)
 - [3. Sign in using the AWS CLI](#)
 - [Additional authentication options](#)

- [Creating a simple application using the AWS SDK for Java 2.x](#)
 - [Step 1: Set up for this tutorial](#)
 - [Step 2: Create the project](#)
 - [Step 3: Write the code](#)
 - [Step 4: Build and run the application](#)
 - [Success](#)
 - [Cleanup](#)
 - [Next steps](#)

Setting up the AWS SDK for Java 2.x

This section provides information about how to set up your development environment and projects to use the AWS SDK for Java 2.x.

Setup overview

To successfully develop applications that access AWS services using the AWS SDK for Java, the following conditions are required:

- The Java SDK must have access to credentials to [authenticate requests](#) on your behalf.
- The [permissions of the IAM role](#) configured for the SDK must allow access to the AWS services that your application requires. The permissions associated with the **PowerUserAccess** AWS managed policy are sufficient for most development needs.
- A development environment with the following elements:
 - [Shared configuration files](#) that are set up in at least one of the following ways:
 - The config file contains [IAM Identity Center single sign-on settings](#) so that the SDK can get AWS credentials.
 - The credentials file contains temporary credentials.
 - An [installation of Java 8](#) or later.
 - A [build automation tool](#) such as [Maven](#) or [Gradle](#).
 - A text editor to work with code.
 - (Optional, but recommended) An IDE (integrated development environment) such as [IntelliJ IDEA](#), [Eclipse](#), or [NetBeans](#).

If you use IntelliJ IDEA, you can also add the [AWS Toolkit for IntelliJ IDEA](#) to integrate AWS services directly into the IDE to help you streamline development.

- An active AWS access portal session when you are ready to run your application. You use the AWS Command Line Interface to [initiate the sign-in process](#) to IAM Identity Center's AWS access portal.

Important

The instructions in this setup section assume that you or organization uses IAM Identity Center. If your organization uses an external identity provider that works independently of IAM Identity Center, find out how you can get temporary credentials for the SDK for Java to use. Follow [these instructions](#) to add temporary credentials to the `~/.aws/credentials` file.

If your identity provider adds temporary credentials automatically to the `~/.aws/credentials` file, make sure that the profile name is `[default]` so that you do not need to provide a profile name to the SDK or AWS CLI.

Install Java and a build tool to work with the AWS SDK for Java 2.x

You need the following Java development environment requirements to work with SDK for Java 2.x:

- Java 8 or later. The AWS SDK for Java works with the [Oracle Java SE Development Kit](#) and with distributions of Open Java Development Kit (OpenJDK) such as [Amazon Corretto](#), [Red Hat OpenJDK](#), and [Adoptium](#).
- A Maven-compatible build tools such as Apache Maven, Apache Ant with Ivy, Gradle, or IntelliJ.
 - For information about how to install and use Maven, see <https://maven.apache.org/>.
 - For information about how to install and use Apache Ivy, see <https://ant.apache.org/ivy/>.
 - For information about how to install and use Gradle, see <https://gradle.org/>.
 - For information about how to install and use IntelliJ IDEA, see <https://www.jetbrains.com/idea/>.

Set up an Apache Maven project that uses the AWS SDK for Java 2.x

You can use [Apache Maven](#) to set up and build AWS SDK for Java 2.x projects, or to [build the SDK itself](#).

Prerequisites

To use the SDK for Java 2.x with Maven, you need the following:

- **Java 8.0 or later.** You can download the latest Java SE Development Kit software from <http://www.oracle.com/technetwork/java/javase/downloads/>. The SDK for Java 2.x also works with [OpenJDK](#) and Amazon Corretto, a distribution of the Open Java Development Kit (OpenJDK). Download the latest OpenJDK version from <https://openjdk.java.net/install/index.html>. Download the latest Amazon Corretto 8 or Amazon Corretto 11 version from [the Corretto page](#).
- **Apache Maven.** If you need to install Maven, go to <http://maven.apache.org/> to download and install it.

Create a Maven project

To create a Maven project from the command line, run the following command from a terminal or command prompt window.

```
mvn -B archetype:generate \
  -DarchetypeGroupId=software.amazon.awssdk \
  -DarchetypeArtifactId=archetype-lambda -Dservice=s3 -Dregion=US_WEST_2 \
  -DarchetypeVersion=2.X.X \
  -DgroupId=com.example.myapp \
  -DartifactId=myapp
```

Note

Replace *com.example.myapp* with the full package namespace of your application. Also replace *myapp* with your project name. This becomes the name of the directory for your project.

To use the latest version of the archetype, replace *2.X.X* with the [latest from Maven central](#).

This command creates a Maven project using the archetype templating toolkit. The archetype generates the scaffolding for an AWS Lambda function handler project. This project archetype is preconfigured to compile with Java SE 8 and includes a dependency to the version of the SDK for Java 2.x specified with `-DarchetypeVersion`.

For more information about creating and configuring Maven projects, see the [Maven Getting Started Guide](#).

Configure the Java compiler for Maven

If you created your project using the AWS Lambda project archetype as described previously, the configuration of the Java compiler is already done for you.

To verify that this configuration is present, start by opening the `pom.xml` file from the project folder you created (for example, `myapp`) when you executed the previous command. Look on lines 11 and 12 to see the Java compiler version setting for this Maven project, and the required inclusion of the Maven compiler plugin on lines 71-75.

```
<project>
  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>${maven.compiler.plugin.version}</version>
      </plugin>
    </plugins>
  </build>
</project>
```

If you create your project with a different archetype or by using another method, you must ensure that the Maven compiler plugin is part of the build and that its source and target properties are both set to **1.8** in the `pom.xml` file.

See the previous snippet for one way to configure these required settings.

Alternatively, you can configure the compiler configuration inline with the plugin declaration, as follows.

```
<project>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Declare the SDK as a dependency

To use the AWS SDK for Java in your project, you need to declare it as a dependency in your project's `pom.xml` file.

If you created your project using the project archetype as described previously, the latest version of the SDK is already configured as a dependency in your project.

The archetype generates a BOM (bill of materials) artifact dependency for the `software.amazon.awssdk` group id. With a BOM, you do not have to specify the maven version for individual artifact dependencies that share the same group id.

If you created your Maven project in a different way, configure the latest version of the SDK for your project by ensuring that the `pom.xml` file contains the following.

```
<project>
  <properties>
    <aws.java.sdk.version>2.X.X</aws.java.sdk.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.java.sdk.version}</version>
        <type>pom</type>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

```
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
</project>
```

Note

Replace **2.X.X** in the `pom.xml` file with the [latest version of the AWS SDK for Java 2.x](#).

Set dependencies for SDK modules

Now that you have configured the SDK, you can add dependencies for one or more of the AWS SDK for Java modules to use in your project.

Although you can specify the version number for each component, you don't need to because you already declared the SDK version in the `dependencyManagement` section using the bill of materials artifact. To load a different version of a given module, specify a version number for its dependency.

If you created your project using the project archetype as described previously, your project is already configured with multiple dependencies. These include dependences for AWS Lambda function handlers and Amazon S3, as follows.

```
<project>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>s3</artifactId>
      <exclusions>
        <exclusion>
          <groupId>software.amazon.awssdk</groupId>
          <artifactId>netty-nio-client</artifactId>
        </exclusion>
        <exclusion>
          <groupId>software.amazon.awssdk</groupId>
          <artifactId>apache-client</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
  </dependencies>
</project>
```

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>url-connection-client</artifactId>
</dependency>

<dependency>
  <groupId>com.amazonaws</groupId>
  <artifactId>aws-lambda-java-core</artifactId>
  <version>${aws.lambda.java.version}</version>
</dependency>
</dependencies>
</project>
```

Note

In the `pom.xml` example above, the dependencies are from different `groupId`s. The `s3` dependency is from `software.amazon.awssdk`, whereas the `aws-lambda-java-core` dependency is from `com.amazonaws`. The BOM dependency management configuration affects artifacts for `software.amazon.awssdk`, so a version is needed for the `aws-lambda-java-core` artifact.

For the development of *Lambda function handlers* using the SDK for Java 2.x, `aws-lambda-java-core` is the correct dependency. However, if your application needs to manage Lambda resources, using operations such as `listFunctions`, `deleteFunction`, `invokeFunction`, and `createFunction`, your application requires the following dependency.

```
<groupId>software.amazon.awssdk</groupId>
<artifactId>lambda</artifactId>
```

Note

The `s3` dependency excludes the `netty-nio-client` and `apache-client` transitive dependencies. In place of either of those HTTP clients, the archetype includes the `url-connection-client` dependency, which helps [reduce the startup latency for AWS Lambda functions](#).

Add the modules to your project for the AWS service and features you need for your project. The modules (dependencies) that are managed by the AWS SDK for Java BOM are listed on the [Maven central repository](#).

Note

You can look at the `pom.xml` file from a code example to determine which dependencies you need for your project. For example, if you're interested in the dependencies for the DynamoDB service, see [this example](#) from the [AWS Code Examples Repository](#) on GitHub. (Look for the `pom.xml` file under [/javav2/example_code/dynamodb.](#))

Build the entire SDK into your project

To optimize your application, we strongly recommend that you pull in only the components you need instead of the entire SDK. However, to build the entire AWS SDK for Java into your project, declare it in your `pom.xml` file, as follows.

```
<project>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>aws-sdk-java</artifactId>
      <version>2.X.X</version>
    </dependency>
  </dependencies>
</project>
```

Build your project

After you configure the `pom.xml` file, you can use Maven to build your project.

To build your Maven project from the command line, open a terminal or command prompt window, navigate to your project directory (for example, `myapp`), enter or paste the following command, then press Enter or Return.

```
mvn package
```

This creates a single `.jar` file (JAR) in the `target` directory (for example, `myapp/target`). This JAR contains all of the SDK modules you specified as dependencies in your `pom.xml` file.

Set up a Gradle project that uses the AWS SDK for Java 2.x

You can use [Gradle](#) to set up and build AWS SDK for Java 2.x projects.

The initial steps in the following example come from [Gradle's Getting Started guide](#) for version 8.4. If you use a different version, your results may differ slightly.

To create a Java application with Gradle (command line)

1. Create a directory to hold your project. In this example, demo is the directory name.
2. Inside the demo directory, execute the `gradle init` command and supply the values highlighted in red as shown in the following command line output. For the walk through, we choose Kotlin as the build script DSL language, but a complete example for Groovy is also shown at the end of this topic.

```
> gradle init
Starting a Gradle Daemon (subsequent builds will be faster)

Select type of project to generate:
1: basic
2: application
3: library
4: Gradle plugin
Enter selection (default: basic) [1..4] 2

Select implementation language:
1: C++
2: Groovy
3: Java
4: Kotlin
5: Scala
6: Swift
Enter selection (default: Java) [1..6] 3

Generate multiple subprojects for application? (default: no) [yes, no] no
Select build script DSL:
1: Kotlin
2: Groovy
Enter selection (default: Kotlin) [1..2] <Enter>

Select test framework:
1: JUnit 4
```

```
2: TestNG
3: Spock
4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 4

Project name (default: demo): <Enter>
Source package (default: demo): <Enter>
Enter target version of Java (min. 7) (default: 11): <Enter>
Generate build using new APIs and behavior (some features may change in the next
  minor release)? (default: no) [yes, no] <Enter>

> Task :init
To learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.4/samples/sample\_building\_java\_applications.html

BUILD SUCCESSFUL in 3m 43s
2 actionable tasks: 2 executed
```

3. After the `init` task completes, the `demo` directory contains the following tree structure. We take a closer look at the main build file, `build.gradle.kts` (highlighted in red), in the next section.

```
### app
#   ### build.gradle.kts
#   ### src
#       ### main
#           #   ### java
#           #       #   ### demo
#           #           ### App.java
#           #   ### resources
#       ### test
#           ### java
#           #   ### demo
#           #       ### AppTest.java
#           ### resources
### gradle
#   ### wrapper
#       ### gradle-wrapper.jar
#       ### gradle-wrapper.properties
### gradlew
### gradlew.bat
### settings.gradle.kts
```

The `build.gradle.kts` file contains the following scaffolded content.

```
/*
 * This file was generated by the Gradle 'init' task.
 *
 * This generated file contains a sample Java application project to get you
 * started.
 * For more details on building Java & JVM projects, please refer to https://docs.gradle.org/8.4/userguide/building\_java\_projects.html in the Gradle
 * documentation.
 */

plugins {
    // Apply the application plugin to add support for building a CLI application
    // in Java.
    application
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation("org.junit.jupiter:junit-jupiter:5.9.3")

    testRuntimeOnly("org.junit.platform:junit-platform-launcher")

    // This dependency is used by the application.
    implementation("com.google.guava:guava:33.3.0-jre")
}

// Apply a specific Java toolchain to ease working on different environments.
java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(11))
    }
}

application {
    // Define the main class for the application.
    mainClass.set("demo.App")
}
```

```
}

tasks.named<Test>("test") {
    // Use JUnit Platform for unit tests.
    useJUnitPlatform()
}
```

4. Use the scaffolded Gradle build file as the basis for your AWS project.
 - a. To manage SDK dependencies for your Gradle project, add the Maven bill of materials (BOM) for the AWS SDK for Java 2.x to the dependencies section of the `build.gradle.kts` file.

```
...
dependencies {
    implementation(platform("software.amazon.awssdk:bom:2.27.21"))
    // With the bom declared, you specify individual SDK dependencies without a
    // version.
    ...
}
...
```

 Note

In this example build file, replace `2.27.21` with the latest version of the SDK for Java 2.x. Find the latest version available in [Maven central repository](#).

- b. Specify the SDK modules your application needs in the dependencies section. As an example, the following adds a dependency on Amazon Simple Storage Service.

```
...
dependencies {
    implementation(platform("software.amazon.awssdk:bom:2.27.21"))
    implementation("software.amazon.awssdk:s3")
    ...
}
...
```

Gradle automatically resolves the correct version of declared dependencies by using the information from the BOM.

The following examples show complete Gradle build files in both the Kotlin and Groovy DSLs. The build file contains dependencies for Amazon S3, authentication, logging, and testing. The source and target version of Java is version 11.

Kotlin DSL (build.gradle.kts)

```
/*
 * This file was generated by the Gradle 'init' task.
 *
 * This generated file contains a sample Java application project to get you
 * started.
 * For more details on building Java & JVM projects, please refer to https://
 * docs.gradle.org/8.4/userguide/building_java_projects.html in the Gradle
 * documentation.
 */

plugins {
    // Apply the application plugin to add support for building a CLI application in
    // Java.
    application
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    implementation(platform("software.amazon.awssdk:bom:2.20.56"))
    implementation("software.amazon.awssdk:s3")
    implementation("software.amazon.awssdk:sso")
    implementation("software.amazon.awssdk:ssoidc")
    implementation(platform("org.apache.logging.log4j:log4j-bom:2.20.0"))
    implementation("org.apache.logging.log4j:log4j-slf4j2-impl")
    implementation("org.apache.logging.log4j:log4j-1.2-api")
    testImplementation(platform("org.junit:junit-bom:5.10.0"))
    testImplementation("org.junit.jupiter:junit-jupiter")
}

// Apply a specific Java toolchain to ease working on different environments.
java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(11))
    }
}
```

```
    }  
}  
  
application {  
    // Define the main class for the application.  
    mainClass.set("demo.App")  
}  
  
tasks.named<Test>("test") {  
    // Use JUnit Platform for unit tests.  
    useJUnitPlatform()  
}
```

Groovy DSL (build.gradle)

```
/*  
 * This file was generated by the Gradle 'init' task.  
 *  
 * This generated file contains a sample Java application project to get you  
 * started.  
 * For more details on building Java & JVM projects, please refer to https://docs.gradle.org/8.4/userguide/building\_java\_projects.html in the Gradle  
 * documentation.  
 */  
  
plugins {  
    // Apply the application plugin to add support for building a CLI application in  
    // Java.  
    id 'application'  
}  
  
repositories {  
    // Use Maven Central for resolving dependencies.  
    mavenCentral()  
}  
  
dependencies {  
    implementation platform('software.amazon.awssdk:bom:2.27.21')  
    implementation 'software.amazon.awssdk:s3'  
    implementation 'software.amazon.awssdk:sso'  
    implementation 'software.amazon.awssdk:ssoidc'  
    implementation platform('org.apache.logging.log4j:log4j-bom:2.20.0')  
    implementation 'org.apache.logging.log4j:log4j-slf4j2-impl'
```

```
implementation 'org.apache.logging.log4j:log4j-1.2-api'
testImplementation platform('org.junit:junit-bom:5.10.0')
testImplementation 'org.junit.jupiter:junit-jupiter'
}

// Apply a specific Java toolchain to ease working on different environments.
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(11)
    }
}

application {
    // Define the main class for the application.
    mainClass = 'demo_groovy.App'
}

tasks.named('test') {
    // Use JUnit Platform for unit tests.
    useJUnitPlatform()
}
```

For next steps, see the Getting Started guide on the Gradle website for instructions on how to [build and run a Gradle application](#).

Set up a GraalVM Native Image project that uses the AWS SDK for Java 2.x

With versions 2.16.1 and later, the AWS SDK for Java 2.x provides out-of-the-box support for GraalVM Native Image applications. Use the archetype-app-quickstart Maven archetype to set up a project with built-in native image support.

Prerequisites

- Complete the steps in [Setting up the AWS SDK for Java 2.x](#).
- Install [GraalVM Native Image](#).

Create a project using the archetype

To create a Maven project with built-in native image support, in a terminal or command prompt window, use the following command.

Note

Replace `com.example.mynativeimageapp` with the full package namespace of your application. Also replace `mynativeimageapp` with your project name. This becomes the name of the directory for your project.

```
mvn archetype:generate \  
  -DarchetypeGroupId=software.amazon.awssdk \  
  -DarchetypeArtifactId=archetype-app-quickstart \  
  -DarchetypeVersion=2.27.21\  
  -DnativeImage=true \  
  -DhttpClient=apache-client \  
  -Dservice=s3 \  
  -DgroupId=com.example.mynativeimageapp \  
  -DartifactId=mynativeimageapp \  
  -DinteractiveMode=false
```

This command creates a Maven project configured with dependencies for the AWS SDK for Java, Amazon S3, and the `ApacheHttpClient` HTTP client. It also includes a dependency for the [GraalVM Native Image Maven plugin](#), so that you can build native images using Maven.

To include dependencies for a different Amazon Web Services, set the value of the `-Dservice` parameter to the artifact ID of that service. Examples include `dynamodb`, `comprehend`, and `pinpoint`. For a complete list of artifact IDs, see the list of managed dependencies for [software.amazon.awssdk on Maven Central](#).

To use an asynchronous HTTP client, set the `-DhttpClient` parameter to `netty-nio-client`. To use `URLConnectionHttpClient` as the synchronous HTTP client instead of `apache-client`, set the `-DhttpClient` parameter to `url-connection-client`.

Build a native image

After you create the project, run the following command from your project directory, for example, `mynativeimageapp`:

```
mvn package -P native-image
```

This creates a native image application in the target directory, for example, target/mynativeimageapp.

Authenticating with AWS using the AWS SDK for Java 2.x

You must establish how the AWS SDK for Java 2.x authenticates with AWS when you develop with AWS services. The SDK manages credential discovery, signature creation, and credential refreshing completely behind the scenes, letting you focus on your application logic.

Set up for authentication

The [Authentication and access](#) topic in the AWS SDKs and Tools Reference Guide describes the different authentication approaches.

For local development the recommended approach is to use short-term credentials by logging into the AWS CLI with your console login credentials. Recommended if you use root, IAM users, or federation with IAM for AWS account access. Follow the instructions at [Access credentials for interactive development work using AWS SDK for Java 2.x](#).

After following the instructions in AWS SDKs and Tools Reference Guide, your system should be set up to allow the SDK to sign requests:

1. Local development using short-term credentials

For local development the recommended approach is to use short-term credentials by logging into the AWS CLI with your console login credentials. Recommended if you use root, IAM users, or federation with IAM for AWS account access.

For instructions, see [Using credentials providers in the AWS SDK for Java 2.x](#).

2. Setup for single sign-on access for the SDK

After you complete Step 2 in the [programmatic access section](#) so that the SDK can use IAM Identity Center authentication, your system should contain the following elements.

- The AWS CLI, which you use to start an [AWS access portal session](#) before you run your application.

- An `~/.aws/config` file that contains a [default profile](#). The SDK for Java uses the profile's SSO token provider configuration to acquire credentials before sending requests to AWS. The `sso_role_name` value, which is an IAM role connected to an IAM Identity Center permission set, should allow access to the AWS services used in your application.

The following sample config file shows a default profile set up with SSO token provider configuration. The profile's `sso_session` setting refers to the named `sso-session` section. The `sso-session` section contains settings to initiate an AWS access portal session.

```
[default]
sso_session = my-sso
sso_account_id = 111122223333
sso_role_name = SampleRole
region = us-east-1
output = json

[sso-session my-sso]
sso_region = us-east-1
sso_start_url = https://provided-domain.awsapps.com/start
sso_registration_scopes = sso:account:access
```

For more details about the settings used in the SSO token provider configuration, see [SSO token provider configuration](#) in the AWS SDKs and Tools Reference Guide.

If your development environment is not set up for programmatic access as previously shown, follow [Step 2 in the SDKs Reference Guide](#).

3. Sign in using the AWS CLI

Before running an application that accesses AWS services, you need an active AWS access portal session in order for the SDK to use IAM Identity Center authentication to resolve credentials. Run the following command in the AWS CLI to sign in to the AWS access portal.

```
aws sso login
```

Since you have a default profile setup, you do not need to call the command with a `--profile` option. If your SSO token provider configuration is using a named profile, the command is `aws sso login --profile named-profile`.

To test if you already have an active session, run the following AWS CLI command.

```
aws sts get-caller-identity
```

The response to this command should report the IAM Identity Center account and permission set configured in the shared config file.

 Note

If you already have an active AWS access portal session and run `aws sso login`, you will not be required to provide credentials.

However, you will see a dialog that requests permission for `botocore` to access your information. `botocore` is the foundation for the AWS CLI .

Select **Allow** to authorize access to your information for the AWS CLI and SDK for Java.

Additional authentication options

For more options on authentication for the SDK, such as the use of profiles and environment variables, see the [configuration](#) chapter in the AWS SDKs and Tools Reference Guide.

Creating a simple application using the AWS SDK for Java 2.x

This tutorial shows you how to use [Apache Maven](#) to define dependencies for the SDK for Java 2.x and then write code that connects to Amazon S3 to upload a file.

Follow these steps to complete this tutorial:

- [Step 1: Set up for this tutorial](#)
- [Step 2: Create the project](#)
- [Step 3: Write the code](#)
- [Step 4: Build and run the application](#)

Step 1: Set up for this tutorial

Before you begin this tutorial, you need the following:

- Permission to access Amazon S3

- A Java development environment that is configured to access AWS services using single sign-on to the AWS IAM Identity Center

Use the instructions in [???](#) to get set up for this tutorial. After you have [configured your development environment with single sign-on access](#) for the Java SDK and you have an [active AWS access portal session](#), continue with Step 2 of this tutorial.

Step 2: Create the project

To create the project for this tutorial, you run a Maven command that prompts you for input on how to configure the project. After all input is entered and confirmed, Maven finishes building out the project by creating a `pom.xml` and creates stub Java files.

1. Open a terminal or command prompt window and navigate to a directory of your choice, for example, your Desktop or Home folder.
2. Enter the following command at the terminal and press Enter.

```
mvn archetype:generate \  
-DarchetypeGroupId=software.amazon.awssdk \  
-DarchetypeArtifactId=archetype-app-quickstart \  
-DarchetypeVersion=2.27.21
```

3. Enter the value listed in the second column for each prompt.

Prompt	Value to enter
Define value for property 'service':	s3
Define value for property 'httpClient' :	apache-client
Define value for property 'nativeImage' :	false
Define value for property 'credentialProvider'	identity-center

Prompt	Value to enter
Define value for property 'groupId':	org.example
Define value for property 'artifactId':	getstarted
Define value for property 'version' 1.0-SNAPSHOT:	<Enter>
Define value for property 'package' org.example:	<Enter>

- After the last value is entered, Maven lists the choices you made. Confirm by entering *Y* or re-enter values by entering *N*.

Maven creates the project folder named `getstarted` based on the `artifactId` value that you entered. Inside the `getstarted` folder, find a `README.md` file that you can review, a `pom.xml` file, and a `src` directory.

Maven builds the following directory tree.

```
getstarted
### README.md
### pom.xml
### src
    ### main
    #   ### java
    #   #   ### org
    #   #       ### example
    #   #           ### App.java
    #   #           ### DependencyFactory.java
    #   #           ### Handler.java
    #   ### resources
    #       ### simplelogger.properties
    ### test
    ### java
    ### org
    ### example
    ### HandlerTest.java
```

10 directories, 7 files

The following shows the contents of the `pom.xml` project file.

pom.xml

The `dependencyManagement` section contains a dependency to the AWS SDK for Java 2.x and the `dependencies` section has a dependency for Amazon S3. The project uses Java 1.8 because of the 1.8 value in the `maven.compiler.source` and `maven.compiler.target` properties.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>getstarted</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <maven.shade.plugin.version>3.2.1</maven.shade.plugin.version>
    <maven.compiler.plugin.version>3.6.1</maven.compiler.plugin.version>
    <exec-maven-plugin.version>1.6.0</exec-maven-plugin.version>
    <aws.java.sdk.version>2.27.21</aws.java.sdk.version> <----- SDK version
picked up from archetype version.
    <slf4j.version>1.7.28</slf4j.version>
    <junit5.version>5.8.1</junit5.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.java.sdk.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
```

```

    </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId> <----- S3 dependency
    <exclusions>
      <exclusion>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>netty-nio-client</artifactId>
      </exclusion>
      <exclusion>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>apache-client</artifactId>
      </exclusion>
    </exclusions>
  </dependency>

  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>sso</artifactId> <----- Required for identity center
authentication.
  </dependency>

  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>ssoidc</artifactId> <----- Required for identity center
authentication.
  </dependency>

  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>apache-client</artifactId> <----- HTTP client specified.
    <exclusions>
      <exclusion>
        <groupId>commons-logging</groupId>
        <artifactId>commons-logging</artifactId>
      </exclusion>
    </exclusions>
  </dependency>

  <dependency>
    <groupId>org.slf4j</groupId>

```



```
        <artifactId>slf4j-api</artifactId>
        <version>${slf4j.version}</version>
    </dependency>

    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>${slf4j.version}</version>
    </dependency>

    <!-- Needed to adapt Apache Commons Logging used by Apache HTTP Client to Slf4j
to avoid
    ClassNotFoundException: org.apache.commons.logging.impl.LogFactoryImpl during
runtime -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>jcl-over-slf4j</artifactId>
        <version>${slf4j.version}</version>
    </dependency>

    <!-- Test Dependencies -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit5.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>${maven.compiler.plugin.version}</version>
        </plugin>
    </plugins>
</build>

</project>
```

Step 3: Write the code

The following code shows the App class created by Maven. The main method is the entry point into the application, which creates an instance of the Handler class and then calls its `sendRequest` method.

App class

```
package org.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class App {
    private static final Logger logger = LoggerFactory.getLogger(App.class);

    public static void main(String... args) {
        logger.info("Application starts");

        Handler handler = new Handler();
        handler.sendRequest();

        logger.info("Application ends");
    }
}
```

The `DependencyFactory` class created by Maven contains the `s3Client` factory method that builds and returns an [S3Client](#) instance. The `S3Client` instance uses an instance of the Apache-based HTTP client. This is because you specified `apache-client` when Maven prompted you for which HTTP client to use.

The `DependencyFactory` is shown in the following code.

DependencyFactory class

```
package org.example;

import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;

/**
 * The module containing all dependencies required by the {@link Handler}.
 */
```

```
public class DependencyFactory {

    private DependencyFactory() {}

    /**
     * @return an instance of S3Client
     */
    public static S3Client s3Client() {
        return S3Client.builder()
            .httpClientBuilder(ApacheHttpClient.builder())
            .build();
    }
}
```

The `Handler` class contains the main logic of your program. When an instance of `Handler` is created in the `App` class, the `DependencyFactory` furnishes the `S3Client` service client. Your code uses the `S3Client` instance to call the Amazon S3 service.

Maven generates the following `Handler` class with a *TODO* comment. The next step in the tutorial replaces the *TODO* with code.

Handler class, Maven-generated

```
package org.example;

import software.amazon.awssdk.services.s3.S3Client;

public class Handler {
    private final S3Client s3Client;

    public Handler() {
        s3Client = DependencyFactory.s3Client();
    }

    public void sendRequest() {
        // TODO: invoking the api calls using s3Client.
    }
}
```

To fill in the logic, replace the entire contents of the `Handler` class with the following code. The `sendRequest` method is filled in and the necessary imports are added.

Handler class, implemented

The code first creates a new S3 bucket with the last part of the name generated using `System.currentTimeMillis()` in order to make the bucket name unique.

After creating the bucket in the `createBucket()` method, the program uploads an object using the [putObject](#) method of `S3Client`. The contents of the object is a simple string created with the `RequestBody.fromString` method.

Finally, the program deletes the object followed by the bucket in the `cleanUp` method.

```
package org.example;

import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadBucketRequest;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

public class Handler {
    private final S3Client s3Client;

    public Handler() {
        s3Client = DependencyFactory.s3Client();
    }

    public void sendRequest() {
        String bucket = "bucket" + System.currentTimeMillis();
        String key = "key";

        createBucket(s3Client, bucket);

        System.out.println("Uploading object...");

        s3Client.putObject(PutObjectRequest.builder().bucket(bucket).key(key)
            .build(),
            RequestBody.fromString("Testing with the {sdk-java}"));

        System.out.println("Upload complete");
    }
}
```

```

        System.out.printf("%n");

        cleanUp(s3Client, bucket, key);

        System.out.println("Closing the connection to {S3}");
        s3Client.close();
        System.out.println("Connection closed");
        System.out.println("Exiting...");
    }

    public static void createBucket(S3Client s3Client, String bucketName) {
        try {
            s3Client.createBucket(CreateBucketRequest
                .builder()
                .bucket(bucketName)
                .build());
            System.out.println("Creating bucket: " + bucketName);
            s3Client.waitFor().waitUntilBucketExists(HeadBucketRequest.builder()
                .bucket(bucketName)
                .build());
            System.out.println(bucketName + " is ready.");
            System.out.printf("%n");
        } catch (S3Exception e) {
            System.err.println(e.awsErrorDetails().errorMessage());
            System.exit(1);
        }
    }

    public static void cleanUp(S3Client s3Client, String bucketName, String keyName) {
        System.out.println("Cleaning up...");
        try {
            System.out.println("Deleting object: " + keyName);
            DeleteObjectRequest deleteObjectRequest =
DeleteObjectRequest.builder().bucket(bucketName).key(keyName).build();
            s3Client.deleteObject(deleteObjectRequest);
            System.out.println(keyName + " has been deleted.");
            System.out.println("Deleting bucket: " + bucketName);
            DeleteBucketRequest deleteBucketRequest =
DeleteBucketRequest.builder().bucket(bucketName).build();
            s3Client.deleteBucket(deleteBucketRequest);
            System.out.println(bucketName + " has been deleted.");
            System.out.printf("%n");
        } catch (S3Exception e) {
            System.err.println(e.awsErrorDetails().errorMessage());

```

```
        System.exit(1);
    }
    System.out.println("Cleanup complete");
    System.out.printf("%n");
}
}
```

Step 4: Build and run the application

After the project is created and contains the complete `Handler` class, build and run the application.

1. Make sure that you have an active IAM Identity Center session. To do so, run the AWS Command Line Interface command `aws sts get-caller-identity` and check the response. If you don't have an active session, see [this section](#) for instructions.
2. Open a terminal or command prompt window and navigate to your project directory `getstarted`.
3. Use the following command to build your project:

```
mvn clean package
```

4. Use the following command to run the application.

```
mvn exec:java -Dexec.mainClass="org.example.App"
```

To view the new bucket and object that the program creates, perform the following steps.

1. In `Handler.java`, comment out the line `cleanUp(s3Client, bucket, key)` in the `sendRequest` method and save the file.
2. Rebuild the project by running `mvn clean package`.
3. Rerun `mvn exec:java -Dexec.mainClass="org.example.App"` to upload the text object once more.
4. Sign in to [the S3 console](#) to view the new object in the newly created bucket.

After you view the file, delete the object, and then delete the bucket.

Success

If your Maven project built and ran without error, then congratulations! You have successfully built your first Java application using the SDK for Java 2.x.

Cleanup

To clean up the resources you created during this tutorial, do the following:

- If you haven't done so already, in [the S3 console](#), delete any objects and any buckets created when you ran the application.
- Delete the project folder (getstarted).

Next steps

Now that you have the basics down, you can learn about the following:

- [Working with Amazon S3](#)
- [Working with other Amazon Web Services](#), such as [DynamoDB](#), [Amazon EC2](#), and [various database services](#)
- [Use the SDK](#)
- [Security for the AWS SDK for Java](#)

Configuring service clients in the AWS SDK for Java 2.x

To programmatically access AWS services, the SDK for Java 2.x uses a client object for each AWS service. For example, if your application needs to access Amazon EC2, you create an Amazon EC2 client object—an instance of the [Ec2Client](#) class—to interface with that service. You then use the service client to make requests to that AWS service. For most applications, you can use a [singleton instance of a service client](#).

There are many ways to configure SDK behavior, but ultimately everything has to do with the behavior of service clients. Any configuration has no effect until your code creates a service client that uses the configuration.

Examples of the configuration that you provide are:

- How your code authenticates with AWS when you call a service
- The AWS Region you want a service client to use
- Retry and timeout settings for service calls
- HTTP proxy configuration

Refer to the [AWS SDKs and Tools Reference Guide](#) for settings, features, and other foundational concepts common to many of the AWS SDKs.

Topics

- [Configuring service clients for the AWS SDK for Java 2.x externally](#)
- [Configuring service clients in code for the AWS SDK for Java 2.x](#)
- [Use singleton service client instances with the AWS SDK for Java 2.x](#)
- [Setting the AWS Region for the AWS SDK for Java 2.x](#)
- [Using credentials providers in the AWS SDK for Java 2.x](#)
- [Configure retry behavior in the AWS SDK for Java 2.x](#)
- [Configure timeouts in AWS SDK for Java 2.x](#)
- [Configuring observability features in the AWS SDK for Java 2.x](#)
- [Configuring client endpoints in the AWS SDK for Java 2.x](#)
- [Configure HTTP clients in the AWS SDK for Java 2.x](#)
- [Use execution interceptors in the AWS SDK for Java 2.x](#)

Configuring service clients for the AWS SDK for Java 2.x externally

Many configuration settings can be handled outside of your code. When you handle configuration externally, it can apply to all your applications in the same Java process. Most configuration settings can be set as either environment variables, JVM system properties, or in a separate shared AWS config file. The shared config file can maintain separate sets of settings, called profiles, to provide different configurations for different environments or tests.

Most environment variables and shared config file settings are standardized and shared across AWS SDKs and tools to support consistent functionality across different programming languages and applications. In most cases, the JVM system properties that the SDK for Java can use mirror the environment variables.

See the [AWS SDKs and Tools Reference Guide](#) to learn about configuring your application through these methods, plus details on each cross-sdk setting. To see all the all settings that the SDK can resolve from the environment variables, JVM system properties, or configuration files, see the [Settings reference](#) in the *AWS SDKs and Tools Reference Guide*.

Configuration provider chain for client configuration

The SDK checks several places (or sources) to find configuration values.

1. Any explicit setting set in the code or on a service client itself takes precedence over anything else.
2. JVM system properties
 - For details on setting JVM system properties, see [How to set JVM system properties](#) in the *AWS SDKs and Tools Reference Guide*.
3. Environment variables
 - For details on setting environment variables, see [environment variables](#) in the *AWS SDKs and Tools Reference Guide*.
 - Note that you can configure environment variables for a shell at different levels of scope: system-wide, user-wide, and for a specific terminal session.
4. Shared config and credentials files
 - For details on setting up these files, see the [Shared config and credentials files](#) in the *AWS SDKs and Tools Reference Guide*.

5. Any default value provided by the SDK source code itself is used last.

- Some properties, such as Region, don't have a default. You must specify them either explicitly in code, in an environment setting, or in the shared config file. If the SDK can't resolve required configuration, API requests can fail at runtime.

Besides this general configuration chain, the SDK for Java 2.x also uses specialized provider chains including the [credentials provider chain](#) and the [AWS Region provider chain](#). These specialized chains add additional providers that take into account the environment that the SDK is running in. For example, in a container or an EC2 instance.

Create a service client configured using external settings

You need to create a service client in your application to talk to an AWS service. Service clients are your essential connection to AWS services, handling all the complex communication details so you don't have to worry about them. They take care of important tasks like security, error handling, and retries automatically, letting you focus on building your application rather than dealing with technical complications.

Use the `create()` method

If all configuration settings that you need are coming from external sources, you can create a service client with a simple method:

```
S3Client s3Client = S3Client.create();
```

The previous code snippet creates an `S3Client` instance. During creation, the SDK looks through the configuration provider chain for settings. Once the SDK finds a setting value, the value will be used even if configuration exists that is later in the chain.

For example, assume a users sets the JVM setting for the AWS Region by setting the system property `-Daws.region=us-west-2`. If the `AWS_REGION` environment variable is also set, its value is ignored.

The default region provider chain and the default credentials provider chain will also be used in the creation process. Somewhere in the chain, the SDK must resolve the AWS Region to use and find settings that enable it to retrieve credentials for signing requests. If the SDKs files to find those values, the client creation fails.

Although you can create a client by using this empty builder pattern, you generally use this pattern when you want to [add configuration in code](#).

SDK for Java 2.x environment variables and JVM system properties

Beyond the [cross-sdk settings](#) supported by most AWS SDKs, the SDK for Java 2.x provides the following settings.

Note

These environment variables and JVM system properties are primarily intended for advanced use cases, testing, or specific deployment scenarios. In most application code, it's preferable to use the programmatic configuration options provided by the SDK's client builders for better type safety and IDE support.

Container Credential Provider Environment Variables

In addition to the standard container credential environment variables documented in the reference guide, the SDK also supports:

AWS_CONTAINER_SERVICE_ENDPOINT—This environment variable specifies the endpoint for the container metadata service when using the container credentials provider.

Java system property: `aws.containerServiceEndpoint`

Default value: `http://169.254.170.2`

HTTP Client Implementation Environment Variables

SYNC_HTTP_SERVICE_IMPL—Explicitly identifies the default [synchronous HTTP implementation](#) the SDK will use. This is useful when there are multiple implementations on the classpath or as a performance optimization since implementation discovery requires classpath scanning.

Java system property: `software.amazon.awssdk.http.service.impl`

ASYNC_HTTP_SERVICE_IMPL—Explicitly identifies the default [asynchronous HTTP implementation](#) the SDK will use. This is useful when there are multiple implementations on the classpath or as a performance optimization since implementation discovery requires classpath scanning.

Java system property: `software.amazon.awssdk.http.async.service.impl`

Configuring service clients in code for the AWS SDK for Java 2.x

As an alternative to—or in addition to—[configuring service clients externally](#), you can configure them programmatically in code.

By configuring service clients in code, you gain fine-grained control of the many options available to you. Most of the configuration that you can set externally are also available for you to set in code.

Basic configuration in code

For example, the following snippet sets the AWS Region to `EU_SOUTH_2` for an Amazon S3 service client in code:

```
S3Client s3Client = S3Client.builder()
    .region(Region.EU_SOUTH_2)
    .build();
```

The previous snippet shows the static factory method, `builder()`. The `builder()` method returns a `builder` object that allows you to customize the service client. The fluent setter methods return the `builder` object—in this case, an [S3ClientBuilder](#) instance—so that you can chain the method calls for convenience and for more readable code. After you configure the properties you want, call the `build()` method to create the client.

Advanced configuration in code

The following snippet shows additional configuration options:

```
ClientOverrideConfiguration clientOverrideConfiguration =
    ClientOverrideConfiguration.builder()
        .apiCallAttemptTimeout(Duration.ofSeconds(1))
        .addMetricPublisher(CloudWatchMetricPublisher.create())
        .build();

S3Client s3Client = S3Client.builder()
    .region(Region.EU_SOUTH_2)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
    .overrideConfiguration(clientOverrideConfiguration)
```

```

        .httpClientBuilder(
            ApacheHttpClient.builder()
                .maxConnections(100)
                .connectionTimeout(Duration.ofSeconds(5))
                .proxyConfiguration(ProxyConfiguration.builder()
                    .endpoint(URI.create("http://proxy:8080"))
                    .build())
        ).build();

```

In the previous snippet, you can see several entry points to configure a service client:

- [A `ClientOverrideConfiguration.Builder` object](#) that provides configuration options common across all service clients. These settings are AWS-specific behaviors independent of any HTTP implementation.
- **HTTP client configuration through a separate HTTP client builder implementation.**
The `ApacheHttpClient.Builder` is an example. The service client provides the `httpClientBuilder()` method to associate the configured HTTP client to the service client.
- **Methods on the [client builder](#) itself**, such as `region()` and `credentialsProvider()`

Same configuration using configuration blocks

Instead of creating separate objects and then passing them to service client methods, the AWS SDK for Java 2.x provides methods that accept lambda expressions to build these objects inline. The configuration methods on the builder are named the same, but have different signatures. For example:

- [`overrideConfiguration\(ClientOverrideConfiguration overrideConfiguration\)`](#)
- [`overrideConfiguration\(Consumer<ClientOverrideConfiguration.Builder> overrideConfiguration\)`](#)

The configuration of the S3 client shown earlier using this approach can be done in one block of code:

```

S3Client s3Client = S3Client.builder()
    .region(Region.EU_SOUTH_2)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
    .overrideConfiguration(b -> b
        .apiCallAttemptTimeout(Duration.ofSeconds(1))
        .addMetricPublisher(CloudWatchMetricPublisher.create()))

```

```
.httpClientBuilder(ApacheHttpClient.builder()
    .maxConnections(100)
    .connectionTimeout(Duration.ofSeconds(5))
    .proxyConfiguration(ProxyConfiguration.builder()
        .endpoint(URI.create("http://proxy:8080"))
        .build()))
    .build();
```

Configuration options unavailable in code

Because the following settings affect fundamental initialization processes in the SDK, you can set the following configuration settings only externally and not in code:

File Location Settings

These settings control the location of the shared config and credentials files and cannot be overridden programmatically after the SDK has loaded them:

- **AWS_CONFIG_FILE** (environment variable) / **aws.configFile**(JVM system property)
- **AWS_SHARED_CREDENTIALS_FILE** (environment variable) / **aws.sharedCredentialsFile** (JVM system property)

These settings must be set before the SDK loads the configuration files, as they determine where the SDK looks for configuration. Once the SDK has initialized, changing these values has no effect.

Instance Metadata Service Disablement

- **AWS_EC2_METADATA_DISABLED** (environment variable) / **aws.disableEc2Metadata** (JVM system property)

This setting controls whether the SDK attempts to use the EC2 Instance Metadata Service at all. Once the SDK has initialized, you can't change this setting programmatically.

Profile Selection

- **AWS_PROFILE** (environment variable) / **aws.profile** (JVM system property)

This setting tells the SDK which profile to load from the shared config and credentials files. Once loaded, changing this value has no effect.

Container Credential Paths

- **AWS_CONTAINER_CREDENTIALS_RELATIVE_URI**
- **AWS_CONTAINER_CREDENTIALS_FULL_URI**
- **AWS_CONTAINER_AUTHORIZATION_TOKEN**
- **AWS_CONTAINER_AUTHORIZATION_TOKEN_FILE**

You use these environment variables to tell the SDK how to fetch credentials from container services. After the credential provider chain has been established during service client initialization, you can't change these settings.

Default HTTP Implementation Selection

- **SYNC_HTTP_SERVICE_IMPL** (environment variable) / **software.amazon.awssdk.http.service.impl** (JVM system property)
- **ASYNC_HTTP_SERVICE_IMPL** (environment variable) / **software.amazon.awssdk.http.async.service.impl** (JVM system property)

These global settings determine which HTTP client implementation the SDK uses for all service clients unless overridden in code for individual service clients. You must set these before the SDK initializes its HTTP clients and cannot be changed afterward.

Use singleton service client instances with the AWS SDK for Java 2.x

Service clients in the AWS SDK for Java 2.x are thread-safe. You can create one instance of each service client and reuse it throughout your application. This approach improves performance and manages resources more efficiently.

Benefits of singleton service clients

Connection pooling

Service clients maintain internal HTTP connection pools. Creating and destroying these pools is expensive. When you reuse clients, these pools are shared efficiently across requests.

Reduced initialization overhead

Creating a client involves loading configuration, establishing credentials, and initializing internal components. Singleton instances eliminate this overhead.

Better resource utilization

Singleton clients prevent resource exhaustion that can occur when you create many client instances.

Create and use singleton service clients

The following example shows how to create and use singleton service clients:

```
// Create one instance and use it throughout the application.
public class ServiceClientSource {
    private static final S3Client s3Client = S3Client.create();

    public static S3Client getS3Client() {
        return s3Client;
    }
}
```

Don't create new clients for each operation:

```
// This approach creates unnecessary overhead.
public void badExample() {
    try (S3Client s3 = S3Client.create()) {
        s3.listBuckets();
    }
}
```

Important considerations

- Service clients are thread-safe. You can safely share them across multiple threads.
- Close clients only when your application shuts down or if you no longer need the client. Use `client.close()` or try-with-resources at the application level.
- If you need different configurations such as regions or credentials, create separate singleton instances for each configuration.

If you use dependency injection frameworks like Spring, configure service clients as singleton beans. This ensures proper lifecycle management.

Setting the AWS Region for the AWS SDK for Java 2.x

SDK clients connect to an AWS service in a specific AWS Region that you specify when you create the client. This configuration allows your application to interact with AWS resources in that geographical area. When you create a service client without explicitly setting a Region, the SDK uses the default Region from your external configuration.

Explicitly configure an AWS Region

To explicitly set a Region, we recommend that you use the constants defined in the [Region](#) class. This is an enumeration of all publicly available regions.

To create a client with an enumerated Region from the class, use the client builder's `region` method.

```
Ec2Client ec2 = Ec2Client.builder()
    .region(Region.US_WEST_2)
    .build();
```

If the Region you want to use isn't one of the enumerations in the `Region` class, you can create a new Region by using the static `of` method. This method allows you access to new Regions without upgrading the SDK.

```
Region newRegion = Region.of("us-east-42");
Ec2Client ec2 = Ec2Client.builder()
    .region(newRegion)
    .build();
```

Note

After you build a client with the builder, it's *immutable* and the AWS Region *cannot be changed*. If you need to work with multiple AWS Regions for the same service, you should create multiple clients—one per Region.

Let the SDK automatically determine the default AWS Region from the environment

When your code runs on Amazon EC2 or AWS Lambda, you might want to configure clients to use the same AWS Region that your code is running on. This decouples your code from the environment it's running in and makes it easier to deploy your application to multiple AWS Regions for lower latency or redundancy.

To use the default AWS Region provider chain to determine the Region from the environment, use the client builder's `create` method.

```
Ec2Client ec2 = Ec2Client.create();
```

You can also configure the client in other ways, but not set the Region. The SDK picks up the AWS Region by using the default region provider chain:

```
Ec2Client ec2Client = Ec2Client.builder()
    .credentialsProvider(ProfileCredentialsProvider.builder()
        .profileName("my-profile")
        .build())
    .build();
```

If you don't explicitly set an AWS Region by using the `region` method, the SDK consults the default region provider chain to determine the Region to use.

Understanding the default AWS Region provider chain

The SDK takes the following steps to look for an AWS Region:

1. Any explicit Region set by using the `region` method on the builder itself takes precedence over anything else.
2. The SDK looks for the JVM system property `aws.region` and uses its value if found.
3. The `AWS_REGION` environment variable is checked. If it's set, that Region is used to configure the client.

Note

The Lambda container sets this environment variable.

4. The SDK checks the active profile in the [AWS shared config and credentials files](#). If the `region` property is present, the SDK uses it.

The default profile is the active profile unless overridden by `AWS_PROFILE` environment variable or `aws.profile` JVM system property. If the SDK finds the `region` property in both files for the same profile (including the default profile), the SDK uses the value in the shared credentials file.

5. The SDK attempts to use the Amazon EC2 instance metadata service (IMDS) to determine the Region of the currently running Amazon EC2 instance.
 - For greater security, you should disable the SDK from attempting to use version 1 of IMDS. You use the same setting to disable version 1 that are described in the [the section called “Securely”](#) section.
6. If the SDK still hasn’t found a Region by this point, client creation fails with an exception.

When developing AWS applications, a common approach is to use the *shared configuration file* to set the Region for local development, and rely on the default region provider chain to determine the Region when the application runs on AWS infrastructure. This greatly simplifies client creation and keeps your application portable.

Check to see if a service is available in a Region

To see if a particular AWS service is available in a Region, use the static `serviceMetadata` method on a service client:

```
DynamoDbClient.serviceMetadata().regions().forEach(System.out::println);
```

The previous snippet prints out a long list of AWS Region codes that have the DynamoDB service:

```
af-south-1
ap-east-1
ap-northeast-1
ap-northeast-2
ap-northeast-3
ap-south-1
ap-south-2
ap-southeast-1
...
```

You can use a code to look up the [Region class enumeration](#) for the Region you need your service client to use.

For example, if you want to work with DynamoDB in the Region with the code `ap-northeast-2`, create your DynamoDB client with at least the following configuration:

```
DynamoDbClient ddb = DynamoDbClient.builder()
    .region(Region.AP_NORTHEAST_2)
    .build();
```

Choose a specific endpoint

In certain situations—such as to test preview features of a service before the features graduate to general availability—you may need to specify a specific endpoint in a Region. In these situations, service clients can be configured by calling the `endpointOverride` method.

For example, to configure an Amazon EC2 client to use the Europe (Ireland) Region with a specific endpoint, use the following code.

```
Ec2Client ec2 = Ec2Client.builder()
    .region(Region.EU_WEST_1)
    .endpointOverride(URI.create("https://ec2.eu-west-1.amazonaws.com"))
    .build();
```

See [Regions and Endpoints](#) for the current list of regions and their corresponding endpoints for all AWS services.

Using credentials providers in the AWS SDK for Java 2.x

The role of a credentials provider in the AWS SDK for Java 2.x is to source and supply credentials to the SDK's AWS service clients. The SDK uses the credentials it sources to authenticate with the service by cryptographically signing each request. Credentials usually consist of access keys—an access key ID and a secret access key together.

When you use temporary credentials, which are used when you setup [SSO token provider configuration](#) or configure your runtime to [assume an IAM \(AWS Identity and Access Management\) role](#) as examples, a session token is added to the access keys, providing time-limited access to AWS resources.

This topic discusses several ways that you enable the SDK to access credentials.

Topics

- [Access credentials for interactive development work using AWS SDK for Java 2.x](#)
- [Default credentials provider chain in the AWS SDK for Java 2.x](#)
- [Credentials caching in the AWS SDK for Java 2.x](#)
- [Specify a specific credentials provider in the AWS SDK for Java 2.x](#)
- [Use AWS shared configuration profiles in the AWS SDK for Java 2.x](#)
- [Load credentials from an external process using the AWS SDK for Java 2.x](#)
- [Supply credentials in code using the AWS SDK for Java 2.x](#)
- [Read IAM role credentials on Amazon EC2 using the SDK for Java 2.x](#)

Access credentials for interactive development work using AWS SDK for Java 2.x

For increased security, AWS recommends that you configure the SDK for Java to [use temporary credentials](#) instead of long-lived credentials. Temporary credentials consist of access keys (access key id and secret access key) and a session token.

Several approaches are available to you to work with temporary credentials. The approach you use, and therefore the configuration that you provide to the SDK, depends on your use case.

When you do interactive development work with the Java SDK, we recommend that you use AWS Console login credentials.

Using console login credentials

You can use your existing AWS Management Console sign-in credentials for programmatic access to AWS services. After a browser-based authentication flow, AWS generates temporary credentials that work across local development tools like the AWS CLI and the SDK for Java 2.x.

With this process, you can authenticate using root credentials created during initial account set up, an IAM user, or a federated identity from your identity provider, and the AWS CLI automatically manages the temporary credentials for you. This approach enhances security by eliminating the need to store long-term credentials locally.

When you run the `aws login` command, you can select from your active console sessions, or sign in through the browser-based authentication flow and this will automatically generate temporary credentials. The SDK for Java 2.x will automatically refresh these credentials for up to 12 hours.

Important

In addition to the configuration that you set in the shared config file that works for all projects, each individual Java project requires the following dependency in the Maven `pom.xml` file:

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>signin</artifactId>
</dependency>
```

The `signin` dependency provides the code that enables the SDK for Java 2.x to access and use console login credentials.

For more information on prerequisites, logging in and signing out, see [Login for AWS local development using console credentials](#) in the *AWS SDK and Tools Reference Guide*.

Single-sign-on approach

When you do interactive development work with the Java SDK, you can alternatively use the single sign-on approach. This approach requires the following setup:

- [Set up through the IAM Identity Center](#)
- [Configure a profile in the AWS shared config file](#)
- using the AWS CLI and [running a command](#) to login and creating an active session

IAM Identity Center configuration

When you configure the SDK to use IAM Identity Center single sign-on access as described in [???](#) in this guide, the SDK uses temporary credentials.

The SDK uses the IAM Identity Center access token to gain access to the IAM role that is configured with the `sso_role_name` setting in your config file. The SDK assumes this IAM role and retrieves temporary credentials to sign AWS service requests.

For more details about how the SDK gets temporary credentials from the configuration, see the [Understanding IAM Identity Center authentication](#) section of the AWS SDKs and Tools Reference Guide.

Important

In addition to the configuration that you set in the shared config file that works for all projects, each individual Java project requires the following dependencies in the Maven `pom.xml` file:

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>sso</artifactId>
</dependency>
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>ssooidc</artifactId>
</dependency>
```

The `sso` and `ssooidc` dependencies provide the code that enables the SDK for Java 2.x to access temporary credentials.

Retrieve temporary credentials from the AWS access portal

As an alternative to IAM Identity Center single sign-on configuration, you can copy and use temporary credentials available in the AWS access portal. You can use the temporary credentials in a profile or use them as values for system properties and environment variables.

Set up a local credentials file for temporary credentials

1. [Create a shared credentials file](#)
2. In the credentials file, paste the following placeholder text until you paste in working temporary credentials.

```
[default]
aws_access_key_id=<value from AWS access portal>
aws_secret_access_key=<value from AWS access portal>
aws_session_token=<value from AWS access portal>
```

3. Save the file. The file `~/.aws/credentials` should now exist on your local development system. This file contains the [\[default\] profile](#) that the SDK for Java uses if a specific named profile is not specified.
4. [Sign in to the AWS access portal](#)


```
// Any external Region configuration is overridden.  
// The SDK uses the default credentials provider chain because no specific credentials  
// provider is specified.  
Region region = Region.US_WEST_2;  
DynamoDbClient ddb =  
    DynamoDbClient.builder()  
        .region(region)  
        .build();
```

Credential settings retrieval order

The default credentials provider chain of the SDK for Java 2.x searches for configuration in your environment using a predefined sequence.

1. Java system properties

- The SDK uses the [SystemPropertyCredentialsProvider](#) class to load temporary credentials from the `aws.accessKeyId`, `aws.secretAccessKey`, and `aws.sessionToken` Java system properties.

Note

For information on how to set Java system properties, see the [System Properties](#) tutorial on the official *Java Tutorials* website.

2. Environment variables

- The SDK uses the [EnvironmentVariableCredentialsProvider](#) class to load temporary credentials from the `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` environment variables.

3. Web identity token and IAM role ARN

- The SDK uses the [WebIdentityTokenFileCredentialsProvider](#) class to load credentials by assuming a role using a web identity token.
- The credentials provider looks for the following environment variables or JVM system properties:
 - `AWS_WEB_IDENTITY_TOKEN_FILE` or `aws.webIdentityTokenFile`
 - `AWS_ROLE_ARN` or `aws.roleArn`
 - `AWS_ROLE_SESSION_NAME` or `aws.roleSessionName` (optional)

- After the SDK acquires the values, it calls the AWS Security Token Service (STS) and uses the temporary credentials it returns to sign requests.
- Runtime environments such as Amazon Elastic Kubernetes Service (EKS) automatically make web identity tokens available to AWS SDKs, enabling applications to obtain temporary AWS credentials.

4. The shared credentials and config files

- The SDK uses the [ProfileCredentialsProvider](#) to load IAM Identity Center single sign-on settings or temporary credentials from the [default] profile in the shared credentials and config files.

The AWS SDKs and Tools Reference Guide has [detailed information](#) about how the SDK for Java works with the IAM Identity Center single sign-on token to get temporary credentials that the SDK uses to call AWS services.

Note

The credentials and config files are shared by various AWS SDKs and Tools. For more information, see [The .aws/credentials and .aws/config files](#) in the AWS SDKs and Tools Reference Guide.

- Because a profile in the shared credentials and config files can contain many different sets of settings, the ProfileCredentialsProvider delegates to a series of other providers to look for settings under the [default] profile:
 - **Web identity token credentials** (class WebIdentityTokenCredentialsProvider): When the profile contains role_arn and web_identity_token_file.
 - **SSO credentials** (class SsoCredentialsProvider): When the profile contains SSO-related properties such as sso_role_name, sso_account_id.
 - **Role-based credentials with source profile** (class StsAssumeRoleCredentialsProvider): When the profile contains role_arn and source_profile.
 - **Role-based credentials with credential source** (class StsAssumeRoleWithSourceCredentialsProvider): When the profile contains role_arn and credential_source.

- When `credential_source = Environment`: It uses a chain of `SystemPropertyCredentialsProvider` and `EnvironmentVariableCredentialsProvider`
- When `credential_source = Ec2InstanceMetadata`: It uses `InstanceProfileCredentialsProvider`
- When `credential_source = EcsContainer`: It uses `ContainerCredentialsProvider`
- **Console login credentials** (class `LoginCredentialsProvider`): When the profile contains `login_session`
- **Process credentials** (class `ProcessCredentialsProvider`): When the profile contains `credential_process`.
- **Session credentials** (class `StaticSessionCredentialsProvider`): When the profile contains `aws_access_key_id`, `aws_secret_access_key`, and `aws_session_token`.
- **Basic credentials** (class `StaticCredentialsProvider`): When the profile contains `aws_access_key_id` and `aws_secret_access_key`.

5. Amazon ECS container credentials

- The SDK uses the [ContainerCredentialsProvider](#) class to load temporary credentials using the following environment variables:
 - a. `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` or `AWS_CONTAINER_CREDENTIALS_FULL_URI`
 - b. `AWS_CONTAINER_AUTHORIZATION_TOKEN_FILE` or `AWS_CONTAINER_AUTHORIZATION_TOKEN`

The ECS container agent automatically sets the `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` environment variable, which points to the ECS credentials endpoint. The other environment variables are typically set in specific scenarios where the standard ECS credential endpoint isn't used.

6. Amazon EC2 instance IAM role-provided credentials

- The SDK uses the [InstanceProfileCredentialsProvider](#) class to load temporary credentials from the Amazon EC2 metadata service.

7. If the SDK can't find the necessary configuration settings through all the steps listed above, it throws an exception with output similar to the following:

```
software.amazon.awssdk.core.exception.SdkClientException: Unable to load credentials
  from any of the providers
in the chain
  AwsCredentialsProviderChain(credentialsProviders=[SystemPropertyCredentialsProvider(),
  EnvironmentVariableCredentialsProvider(), WebIdentityTokenCredentialsProvider(),
  ProfileCredentialsProvider(),
  ContainerCredentialsProvider(), InstanceProfileCredentialsProvider()])
```

Use the DefaultCredentialsProvider in code

You can explicitly use the default credentials provider chain in your code. This is functionally equivalent to you not specifying a credentials provider at all, since the SDK uses `DefaultCredentialsProvider` by default. However, explicitly using it can make your code more readable and self-documenting. It clearly shows your intention to use the default credentials chain.

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;

public class ExplicitDefaultCredentialsExample {
    public static void main(String[] args) {
        // Explicitly create the DefaultCredentialsProvider.
        DefaultCredentialsProvider defaultCredentialsProvider =
        DefaultCredentialsProvider

                                                                    .builder().build();

        // Use it with any service client.
        S3Client s3Client = S3Client.builder()
            .region(Region.US_WEST_2)
            .credentialsProvider(defaultCredentialsProvider)
            .build();

        // Now you can use the client with the default credentials chain.
        s3Client.listBuckets();
    }
}
```

When you build the default credentials provider you can provide more configuration:

```
DefaultCredentialsProvider customizedProvider = DefaultCredentialsProvider.builder()
```

```
.profileName("custom-profile") // Use a specific profile if the chain gets to the
`ProfileCredentialsProvider` stage.
.asyncCredentialUpdateEnabled(true) // Enable async credential updates.
.build();
```

This approach gives you more control while still providing the convenience of the default credentials chain.

Credentials caching in the AWS SDK for Java 2.x

The AWS SDK for Java 2.x implements credential caching to improve performance and reduce calls to credentials sources. This section explains how credentials caching works and how you can configure it for your applications.

Understanding credential provider caching

Credential providers in the SDK for Java 2.x use different caching strategies:

- **Internal credential caching:** Many providers cache credentials that they retrieve.
- **Automatic refresh:** Providers with cached credentials implement refresh mechanisms.

Providers with internal credentials caching

The following credentials providers cache credentials internally, even when you create new instances:

- **Instance profile credentials provider:** Caches credentials from the Amazon EC2 metadata service.
- **Container credentials provider:** Caches credentials from the container metadata endpoint.
- **STS-based providers:** Cache temporary credentials from AWS Security Token Service (STS).
- **Web identity token providers:** Cache credentials obtained from web identity tokens.
- **Process credentials provider:** Caches credentials from external processes.

Providers without internal caching

The following providers don't implement internal caching:

- **Environment Variable Credentials Provider**
- **System Property Credentials Provider**

- **Static Credentials Provider**

Configuring credential caching

You can customize caching behavior when building credential providers:

Stale Time

Controls when credentials are considered stale and need refreshing:

```
.staleTime(Duration.ofMinutes(2)) // Consider stale 2 minutes before expiration.
```

Prefetch Time

Determines when to start refreshing credentials before they expire:

```
.prefetchTime(Duration.ofMinutes(10)) // Start refresh 10 minutes before expiration.
```

Asynchronous Updates

Enables background credential refreshing:

```
.asyncCredentialUpdateEnabled(true) // Refresh credentials in background thread.
```

Session Duration

For STS-based providers, controls how long temporary credentials remain valid:

```
.refreshRequest(r -> r.durationSeconds(3600)) // 1 hour session.
```

Caching credentials configuration example

As an example of configuring caching for a credentials provider implementation, you might want to have the SDK use a background thread to pre-fetch (retrieve in advance) credentials before they expire. That way you can avoid the blocking call that retrieves fresh credentials.

The following shows an example that creates an [StsAssumeRoleCredentialsProvider](#) that uses a background thread to pre-fetch credentials by setting the [asyncCredentialUpdateEnabled](#) property to true on the builder:

```
StsAssumeRoleCredentialsProvider provider = StsAssumeRoleCredentialsProvider.builder()
```

```
.refreshRequest(r -> r
    .roleArn("arn:aws:iam::111122223333:role/example-role")
    .roleSessionName("example-session")
    .durationSeconds(3600)) // 1 hour session
.staleTime(Duration.ofMinutes(5)) // Consider stale 5 minutes before expiration
.prefetchTime(Duration.ofMinutes(10)) // Start refresh 10 minutes before
expiration
.asyncCredentialUpdateEnabled(true) // Refresh in background
.build();

S3Client s3 = S3Client.builder()
    .credentialsProvider(provider)
    .build();
```

When you invoke an operation on `s3Client` for the first time, an [AssumeRoleRequest](#) is sent to the AWS Security Token Service (STS). STS returns temporary credentials that are valid for 15 minutes (900 seconds). The `s3Client` instance uses the cached credentials until it's time to refresh them before the 15 minutes elapse. By default, the SDK attempts to retrieve new credentials for a new session between 5 minutes and 1 minute before the expiration time of the current session. The pre-fetch window is configurable by using the [prefetchTime](#) and [staleTime](#) properties.

You can configure the following session-based credentials providers similarly:

- `StsWebIdentityTokenFileCredentialsProvider`
- `StsGetSessionTokenCredentialsProvider`
- `StsGetFederationTokenCredentialsProvider`
- `StsAssumeRoleWithWebIdentityCredentialsProvider`
- `StsAssumeRoleWithSamlCredentialsProvider`
- `StsAssumeRoleCredentialsProvider`
- `DefaultCredentialsProvider` (when it delegates to credentials provider that uses sessions)
- `ProcessCredentialsProvider`
- `WebIdentityTokenFileCredentialsProvider`
- `ContainerCredentialsProvider`
- `InstanceProfileCredentialsProvider`

Understanding credential caching helps you optimize your application's performance and reliability when working with AWS services.

Specify a specific credentials provider in the AWS SDK for Java 2.x

While the default credentials provider chain is convenient for many scenarios, explicitly specifying credentials providers gives you greater control over authentication behavior, performance, and security.

Reasons that you might want to specify a credentials provider might include:

- The default provider chain checks multiple sources sequentially, which can add latency:

```
// The default provider chain checks might check multiple sources until it finds
// sufficient configuration.
S3Client s3Client = S3Client.builder().build();

// You can specify exactly where to look.
S3Client optimizedClient = S3Client.builder()
    .credentialsProvider(InstanceProfileCredentialsProvider.create())
    .build();
```

- You need to use non-standard locations for accessing credentials configuration:

```
// Use configuration from a custom file location.
S3Client s3Client = S3Client.builder()
    .credentialsProvider(ProfileCredentialsProvider.builder()
        .profileFile(ProfileFile.builder()
            .content(Paths.get("/custom/path/to/configuration/file"))
            .type(ProfileFile.Type.CONFIGURATION) // Expects all non-default
profiles to be prefixed with "profile".
            .build())
        .profileName("custom")
        .build())
    .build();
```

- Use different credentials for different service clients. For example, if your application needs to access multiple AWS accounts or use different permissions for different services:

```
// S3 client using one set of credentials.
S3Client s3Client = S3Client.builder()
    .credentialsProvider(ProfileCredentialsProvider.create("s3-readonly"))
    .build();

// DynamoDB client using different credentials.
```



```
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .credentialsProvider(ProfileCredentialsProvider.create("dynamodb-admin"))
    .build();
```

- Control credential refresh behavior:

```
// Create a provider with custom refresh behavior.
StsAssumeRoleCredentialsProvider customRefreshProvider =
    StsAssumeRoleCredentialsProvider.builder()
        .refreshRequest(AssumeRoleRequest.builder()
            .roleArn("arn:aws:iam::123456789012:role/my-role")
            .roleSessionName("custom-session")
            .build())
        .stsClient(StsClient.create())
        .asyncCredentialUpdateEnabled(true) // Use a background thread to prefetch
credentials.
        .build();

S3Client s3Client = S3Client.builder()
    .credentialsProvider(customRefreshProvider)
    .build();
```

Use AWS shared configuration profiles in the AWS SDK for Java 2.x

Using the shared config and credentials file, you can set up several profiles. This enables your application to use multiple sets of credentials configuration. The [default] profile was mentioned previously. The SDK uses the [ProfileCredentialsProvider](#) class to load settings from profiles defined in the shared credentials file.

The following code snippet demonstrates how to build a service client that uses the settings defined as part of the profile named my_profile.

```
Region region = Region.US_WEST_2;
DynamoDbClient ddb = DynamoDbClient.builder()
    .region(region)
    .credentialsProvider(ProfileCredentialsProvider.create("my_profile"))
    .build();
```

Set a different profile as the default

To set a profile other than the [default] profile as the default for your application, set the `AWS_PROFILE` environment variable to the name of your custom profile.

To set this variable on Linux, macOS, or Unix, use `export`:

```
export AWS_PROFILE="other_profile"
```

To set these variables on Windows, use `set`:

```
set AWS_PROFILE="other_profile"
```

Alternatively, set the `aws.profile` Java system property to the name of the profile.

Reload profile credentials

You can configure any credentials provider that has a `profileFile()` method on its builder to reload profile credentials. These credentials profile classes are: `ProfileCredentialsProvider`, `DefaultCredentialsProvider`, `InstanceProfileCredentialsProvider`, and `ProfileTokenProvider`.

Note

Profile credential reloading works only with the following settings in the profile file : `aws_access_key_id`, `aws_secret_access_key`, and `aws_session_token`. Settings such as `region`, `sso_session`, `sso_account_id`, and `source_profile` are ignored.

To configure a supported credentials provider to reload profile settings, provide an instance of [ProfileFileSupplier](#) to the `profileFile()` builder method. The following code example demonstrates a `ProfileCredentialsProvider` that reloads credential settings from the [default] profile.

```
ProfileCredentialsProvider provider = ProfileCredentialsProvider
    .builder()
    .profileFile(ProfileFileSupplier.defaultSupplier())
    .build();
```

```
// Set up a service client with the provider instance.
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .region(Region.US_EAST_1)
    .credentialsProvider(provider)
    .build();

/*
    Before dynamoDbClient makes a request, it reloads the credentials settings
    by calling provider.resolveCredentials().
*/
```

When `ProfileCredentialsProvider.resolveCredentials()` is called, the SDK for Java reloads the settings. `ProfileFileSupplier.defaultSupplier()` is one of [several convenience implementations](#) of `ProfileFileSupplier` provided by the SDK. If your use case requires, you can provide your own implementation.

The following example shows the use of the `ProfileFileSupplier.reloadWhenModified()` convenience method. `reloadWhenModified()` takes a `Path` parameter, which gives you flexibility in designating the source file for the configuration rather than the standard `~/.aws/credentials` (or `config`) location.

The settings will be reloaded when `resolveCredentials()` is called only if SDK determines the file's content has been modified.

```
Path credentialsFilePath = ...

ProfileCredentialsProvider provider = ProfileCredentialsProvider
    .builder()
    .profileFile(ProfileFileSupplier.reloadWhenModified(credentialsFilePath,
        ProfileFile.Type.CREDENTIALS))
    .profileName("my-profile")
    .build();

/*
    A service client configured with the provider instance calls
    provider.resolveCredential()
    before each request.
*/
```

The `ProfileFileSupplier.aggregate()` method merges the contents of multiple configuration files. You decide whether a file is reloaded per call to `resolveCredentials()` or a file's settings are fixed at the time it was first read.

The following example shows a `DefaultCredentialsProvider` that merges the settings of two files that contain profile settings. The SDK reloads the settings in the file pointed to by the `credentialsFilePath` variable each time `resolveCredentials()` is called and the settings have changed. The settings from the `profileFile` object remain the same.

```
Path credentialsFilePath = ...;
ProfileFile profileFile = ...;

DefaultCredentialsProvider provider = DefaultCredentialsProvider
    .builder()
    .profileFile(ProfileFileSupplier.aggregate(
        ProfileFileSupplier.reloadWhenModified(credentialsFilePath,
        ProfileFile.Type.CREDENTIALS),
        ProfileFileSupplier.fixedProfileFile(profileFile)))
    .profileName("my-profile")
    .build();

/*
   A service client configured with the provider instance calls
   provider.resolveCredential()
   before each request.
*/
```

Load credentials from an external process using the AWS SDK for Java 2.x

Warning

The following describes a method of sourcing temporary credentials from an external process. This can potentially be dangerous, so proceed with caution. We recommend that you use other credential providers if at all possible. If using this option, we recommend that you make sure the config file is as locked down as possible using security best practices for your operating system.

Make sure that your custom credentials tool does not write any secret information to `StdErr`. SDKs and AWS CLI can capture and log such information, potentially exposing it to unauthorized users.

With the SDK for Java 2.x, you can acquire temporary credentials from an external process for custom use cases. There are two ways to configure this functionality.

Use the `credential_process` setting

If you have a method that provides temporary credentials, you can integrate it by adding the `credential_process` setting as part of a profile definition in the config file. The value you specify must use the full path to the command file. If the file path contains any spaces, you must surround it with quotation marks.

The SDK calls the command exactly as given and then reads JSON data from stdout.

The following examples show the use of this setting for file paths without spaces and file paths with spaces.

Linux/macOS

No spaces in file path

```
[profile process-credential-profile]
credential_process = /path/to/credential/file/credential_file.sh --custom-command
custom_parameter
```

Spaces in file path

```
[profile process-credential-profile]
credential_process = "/path/with/space to/credential/file/credential_file.sh" --
custom-command custom_parameter
```

Windows

No spaces in file path

```
[profile process-credential-profile]
credential_process = C:\Path\To\credentials.cmd --custom_command custom_parameter
```

Spaces in file path

```
[profile process-credential-profile]
credential_process = "C:\Path\With Space To\credentials.cmd" --custom_command
custom_parameter
```

The following code snippet demonstrates how to build a service client that uses the temporary credentials defined as part of the profile named `process-credential-profile`.

```
Region region = Region.US_WEST_2;
S3Client s3Client = S3Client.builder()
    .region(region)
    .credentialsProvider(ProfileCredentialsProvider.create("process-credential-
profile"))
    .build();
```

For detailed information about using an external process as a source of temporary credentials, refer to the [process credentials section](#) in the AWS SDKs and Tools Reference Guide.

Use a ProcessCredentialsProvider

As an alternative to using settings in the config file, you can use the SDK's [ProcessCredentialsProvider](#) to load temporary credentials using Java.

The following examples show various versions of how to specify an external process using the `ProcessCredentialsProvider` and configuring a service client that uses the temporary credentials.

Linux/macOS

No spaces in file path

```
ProcessCredentialsProvider credentials =
    ProcessCredentialsProvider
        .builder()
        .command("/path/to/credentials.sh optional_param1 optional_param2")
        .build();

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .credentialsProvider(credentials)
    .build();
```

Spaces in file path

```
ProcessCredentialsProvider credentials =
    ProcessCredentialsProvider
```

```
        .builder()
        .command("/path\\ with\\ spaces\\ to/credentials.sh optional_param1
optional_param2")
        .build();

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .credentialsProvider(credentials)
    .build();
```

Windows

No spaces in file path

```
ProcessCredentialsProvider credentials =
    ProcessCredentialsProvider
        .builder()
        .command("C:\\Path\\To\\credentials.exe optional_param1 optional_param2")
        .build();

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .credentialsProvider(credentials)
    .build();
```

Spaces in file path

```
ProcessCredentialsProvider credentials =
    ProcessCredentialsProvider
        .builder()
        .command("\"C:\\Path\\With Spaces To\\credentials.exe\" optional_param1
optional_param2")
        .build();

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .credentialsProvider(credentials)
    .build();
```

Use IAM Roles Anywhere for authentication

[IAM Roles Anywhere](#) is an AWS service that allows you to obtain temporary AWS credentials for workloads running outside of AWS. It enables secure access to AWS resources from on-premises or other cloud environments.

Before you can authenticate requests with IAM Roles Anywhere, you first need to gather the required information and download the [credential helper tool](#). By following the [Getting started](#) instructions in the IAM Roles Anywhere User Guide, you can create the necessary artifacts.

The SDK for Java doesn't have a dedicated credentials provider to retrieve temporary credentials from IAM Roles Anywhere, but you can use the credential helper tool along with one of the options to [retrieve credentials from an external process](#).

Use the `credential_process` setting in a profile

The following snippet in the shared AWS config file shows a profile named `roles_anywhere` that uses the `credential_process` setting:

```
[profile roles_anywhere]
credential_process = ./aws_signing_helper credential-process \
  --certificate /path/to/certificate \
  --private-key /path/to/private-key \
  --trust-anchor-arn arn:aws:rolesanywhere:region:account:trust-anchor/TA_ID \
  --profile-arn arn:aws:rolesanywhere:region:account:profile/PROFILE_ID \
  --role-arn arn:aws:iam:account:role/role-name-with-path
```

You need to replace the text shown in red with your values after you have assembled all the artifacts. The first element in the setting, `aws_signing_helper`, is the executable of the credential helper tool and `credential-process` is the command.

When you configure a service client to use the `roles_anywhere` profile—as shown in the following code—the SDK caches the temporary credentials and refreshes them before they expire:

```
S3Client s3Client = S3Client.builder()
    .credentialsProvider(ProfileCredentialsProvider.builder()
        .profileName("roles_anywhere").build())
    .build();
```


Configure a ProcessCredentialsProvider

As shown next, you can use a code-only approach with the `ProcessCredentialsProvider` instead of using profile settings:

```
ProcessCredentialsProvider processCredentialsProvider =
    ProcessCredentialsProvider.builder()
        .command("""
            ./aws_signing_helper credential-process \
            --certificate /path/to/certificate \
            --private-key /path/to/private-key \
            --trust-anchor-arn arn:aws:rolesanywhere:region:account:trust-anchor/TA_ID \
            --profile-arn arn:aws:rolesanywhere:region:account:profile/PROFILE_ID \
            --role-arn arn:aws:iam::account:role/role-name-with-path
        """).build();

S3Client s3Client = S3Client.builder()
    .credentialsProvider(processCredentialsProvider)
    .build();
```

Replace the text shown in red with your values after you have assembled all the artifacts.

Supply credentials in code using the AWS SDK for Java 2.x

If the default credential chain or a specific or custom provider or provider chain doesn't work for your application, you can supply temporary credentials directly in code. These can be [IAM role credentials](#) as [described above](#) or temporary credentials retrieved from AWS Security Token Service (AWS STS). If you retrieved temporary credentials using AWS STS, provide them to an AWS service client as shown in the following code example.

1. Assume a role by calling `StsClient.assumeRole()`.
2. Create a [StaticCredentialsProvider](#) object and supply it with the `AwsSessionCredentials` object.
3. Configure the service client builder with the `StaticCredentialsProvider` and build the client.

The following example creates an Amazon S3 service client using temporary credentials returned by AWS STS for an IAM assumed role.

```

// The AWS IAM Identity Center identity (user) who executes this method does not
have permission to list buckets.
// The identity is configured in the [default] profile.
public static void assumeRole(String roleArn, String roleSessionName) {
    // The IAM role represented by the 'roleArn' parameter can be assumed by
    identities in two different accounts
    // and the role permits the user to only list buckets.

    // The SDK's default credentials provider chain will find the single sign-on
    settings in the [default] profile.
    // The identity configured with the [default] profile needs permission to call
    AssumeRole on the STS service.
    try {
        Credentials tempRoleCredentials;
        try (StsClient stsClient = StsClient.create()) {
            AssumeRoleRequest roleRequest = AssumeRoleRequest.builder()
                .roleArn(roleArn)
                .roleSessionName(roleSessionName)
                .build();

            AssumeRoleResponse roleResponse = stsClient.assumeRole(roleRequest);
            tempRoleCredentials = roleResponse.credentials();
        }
        // Use the following temporary credential items for the S3 client.
        String key = tempRoleCredentials.accessKeyId();
        String secKey = tempRoleCredentials.secretAccessKey();
        String secToken = tempRoleCredentials.sessionToken();

        // List all buckets in the account associated with the assumed role
        // by using the temporary credentials retrieved by invoking
        stsClient.assumeRole().
        StaticCredentialsProvider staticCredentialsProvider =
        StaticCredentialsProvider.create(
            AwsSessionCredentials.create(key, secKey, secToken));
        try (S3Client s3 = S3Client.builder()
            .credentialsProvider(staticCredentialsProvider)
            .build()) {
            List<Bucket> buckets = s3.listBuckets().buckets();
            for (Bucket bucket : buckets) {
                System.out.println("bucket name: " + bucket.name());
            }
        }
    } catch (StsException | S3Exception e) {

```

```
        logger.error(e.getMessage());
        System.exit(1);
    }
}
```

Permission set

The following permission set defined in AWS IAM Identity Center allows the identity (user) to perform the following two operations

1. The `GetObject` operation of the Amazon Simple Storage Service.
2. The `AssumeRole` operation of the AWS Security Token Service.

Without assuming the role, the `s3.listBuckets()` method shown in the example would fail.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "sts:AssumeRole"
      ],
      "Resource": [
        "*"
      ]
    }
  ]
}
```

Assumed role

Assumed role permissions policy

The following permissions policy is attached to the role that is assume in the previous example. This permissions policy permits the ability to list all buckets in the same account as the role.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListAllMyBuckets"
      ],
      "Resource": [
        "*"
      ]
    }
  ]
}
```

Assumed role trust policy

The following trust policy is attached to the role that is assumed in the previous example. The policy allows the role to be assumed by identities (users) in two accounts.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::111122223333:root",
          "arn:aws:iam::555555555555:root"
        ]
      },
      "Action": "sts:AssumeRole",
      "Condition": {}
    }
  ]
}
```

Read IAM role credentials on Amazon EC2 using the SDK for Java 2.x

You can use an IAM role to manage temporary credentials for applications that are running on an EC2 instance and making AWS CLI or AWS API requests. This is preferable to storing access keys within the EC2 instance. To assign an AWS role to an EC2 instance and make it available to all of its applications, you create an instance profile that is attached to the instance. An instance profile contains the role and enables programs that are running on the EC2 instance to get temporary credentials. For more information, see [Use an IAM role to grant permissions to applications running on Amazon EC2 instances](#) in the *IAM User Guide*.

This topic provides information on how to set up your Java application to run on an EC2 instance and enable the AWS SDK for Java 2.x to acquire IAM role credentials.

Acquire IAM role credentials from the environment

If your application creates an AWS service client by using the `create` method (or `builder().build()` methods), the SDK for Java uses the *default credentials provider chain*. The default credentials provider chain searches the execution environment for configuration elements that the SDK can trade for temporary credentials. The [the section called “Default credentials provider chain”](#) section describes the full search process.

The final step in the default provider chain is available only when your application runs on an Amazon EC2 instance. In this step, the SDK uses an `InstanceProfileCredentialsProvider` to read the IAM role defined in the EC2 instance profile. The SDK then acquires temporary credentials for that IAM role.

Although these credentials are temporary and would eventually expire, an `InstanceProfileCredentialsProvider` periodically refreshes them for you so that they continue to allow access to AWS.

Acquire IAM role credentials programmatically

As an alternative to the default credentials provider chain that eventually uses an `InstanceProfileCredentialsProvider` on EC2, you can configure a service client explicitly with an `InstanceProfileCredentialsProvider`. This approach is shown in the following snippet.

```
S3Client s3 = S3Client.builder()
```

```
.credentialsProvider(InstanceProfileCredentialsProvider.create())  
.build();
```

Securely acquire IAM role credentials

By default, EC2 instances run [IMDS](#) (Instance Metadata Service) that allows the SDK's `InstanceProfileCredentialsProvider` to access information such as the IAM role that has been configured. EC2 instances run two versions of IMDS by default:

- Instance Metadata Service Version 1 (IMDSv1) – a request/response method
- Instance Metadata Service Version 2 (IMDSv2) – a session-oriented method

[IMDSv2 is a more secure approach](#) than IMDSv1.

By default, the Java SDK first tries IMDSv2 to get the IAM role, but if that fails, it tries IMDSv1. However, since IMDSv1 is less secure, AWS recommends the use of IMDSv2 only and to disable the SDK from trying IMDSv1.

To use the more secure approach, disable the SDK from using IMDSv1 by providing one of the following settings with a value of `true`.

- Environment variable: `AWS_EC2_METADATA_V1_DISABLED`
- JVM system property: `aws.disableEc2MetadataV1`
- Shared config file setting: `ec2_metadata_v1_disabled`

With one of these settings set to `true`, the SDK does not load IMDS role credentials by using IMDSv1 if the initial IMDSv2 call fails.

Configure retry behavior in the AWS SDK for Java 2.x

Calls to AWS services can fail occasionally for unexpected reasons. Certain errors, such as throttling (rate exceeded) or transient errors, might succeed if the call is retried. The AWS SDK for Java 2.x has a built-in mechanism to detect such errors and automatically retry the call that is enabled by default for all clients.

This page describes how this works, how to configure the distinct modes, and tailor the retry behavior.

Retry strategies

A retry strategy is a mechanism used in the SDK to implement retries. Each SDK client has a retry strategy created at build time that cannot be modified after the client is built.

The retry strategy has the following responsibilities.

- Classify exceptions as retryable or not.
- Compute the suggested delay to wait before the next attempt.
- Maintain a [token bucket](#) that provides a mechanism to stop retries when a large percentage of requests are failing and retries are unsuccessful.

Note

Before the release of *retry strategies* with version 2.26.0 of the SDK, *retry policies* provided the retry mechanism in the SDK. The *retry policy* API is made up of the core [RetryPolicy](#) class in the `software.amazon.awssdk.core.retry` package, whereas the [software.amazon.awssdk.retries](#) package contains the *retry strategy* API elements.

The retry strategy API was introduced as part of the AWS-wide effort to unify the interfaces and behavior of the core components of the SDKs.

The SDK for Java 2.x has three built-in retry strategies: standard, legacy, and adaptive. All three retry strategies are preconfigured to retry on a set of retryable exceptions. Examples of retryable errors are socket timeouts, service-side throttling, concurrency or optimistic lock failures, and transient service errors.

Standard retry strategy

The [standard retry strategy](#) is the recommended `RetryStrategy` implementation for normal use cases. Unlike the `AdaptiveRetryStrategy`, the standard strategy is generally useful across all retry use cases.

By default, the standard retry strategy does the following.

- Retries on the conditions that are configured at build time. You can adjust this with [StandardRetryStrategy.Builder#retryOnException](#).

- Retries 2 times for a total of 3 attempts. You can adjust this with `StandardRetryStrategy.Builder#maxAttempts(int)`.
- For non-throttling exceptions, it uses the [BackoffStrategy#exponentialDelay](#) backoff strategy, with a base delay of 100 milliseconds and a max delay of 20 seconds. You can adjust this with `StandardRetryStrategy.Builder#backoffStrategy`.
- For throttling exceptions, it uses the `BackoffStrategy#exponentialDelay` backoff strategy, with a base delay of 1 second and a max delay of 20 seconds. You can adjust this with `StandardRetryStrategy.Builder#throttlingBackoffStrategy`.
- Performs circuit breaking (disabling retries) in the event of high downstream failures. The first attempt is always executed, only retries are disabled. Adjust with `StandardRetryStrategy.Builder#circuitBreakerEnabled`.

Legacy retry strategy

The [legacy retry strategy](#) is a `RetryStrategy` for normal use cases, however, it is deprecated in favor of the `StandardRetryStrategy`. This is the default retry strategy used by clients when you don't specify another strategy.

It is characterized by treating throttling and non-throttling exceptions differently, for throttling exceptions the base delay for the backoff is larger (500ms) than the base delay for non-throttling exceptions (100ms), and throttling exceptions do not affect the token bucket state.

Experience using this strategy at scale inside AWS has shown that is not particularly better than the standard retry strategy. Moreover, it fails to protect downstream services from retry storms and can lead to resource starvation on the client side.

By default, the legacy retry strategy does the following.

- Retries on the conditions that are configured at build time. You can adjust this with [LegacyRetryStrategy.Builder#retryOnException](#).
- Retries 3 times for a total of 4 attempts. You can adjust this with `LegacyRetryStrategy.Builder#maxAttempts(int)`.
- For non-throttling exceptions, it uses the `BackoffStrategy#exponentialDelay` backoff strategy, with a base delay of 100 milliseconds and a max delay of 20 seconds. You can adjust this with `LegacyRetryStrategy.Builder#backoffStrategy`.

- For throttling exceptions, it uses the `BackoffStrategy#exponentialDelay` backoff strategy, with a base delay of 500 milliseconds and a max delay of 20 seconds. You can adjust this with `LegacyRetryStrategy.Builder#throttlingBackoffStrategy`.
- Performs circuit breaking (disabling retries) in the event of high downstream failures. Circuit breaking never prevents a successful first attempt. You can adjust this behavior with `LegacyRetryStrategy.Builder#circuitBreakerEnabled`.
- The state of the circuit breaker is not affected by throttling exceptions.

Adaptive retry strategy

The [adaptive retry strategy](#) is a `RetryStrategy` for use cases with a high level of resource constraints.

The adaptive retry strategy includes all the features of the standard strategy and adds a client-side rate limiter that measures the rate of throttled requests compared to non-throttled requests. The strategy uses this measurement to slow down the requests in an attempt to stay within a safe bandwidth, ideally causing zero throttling errors.

By default, the adaptive retry strategy does the following.

- Retries on the conditions that are configured at build time. You can adjust this with [AdaptiveRetryStrategy.Builder#retryOnException](#).
- Retries 2 times for a total of 3 attempts. You can adjust this with `AdaptiveRetryStrategy.Builder#maxAttempts(int)`.
- Uses a dynamic backoff delay that is based on the current load against the downstream resource.
- Performs circuit breaking (disabling retries) when there are high number of downstream failures. Circuit breaking may prevent a second attempt in outage scenarios to protect the downstream service.

Warning

The adaptive retry strategy assumes that the client works against a single resource (for example, one DynamoDB table or one Amazon S3 bucket).

If you use a single client for multiple resources, throttling or outages associated with one resource result in increased latency and failures when the client accesses all other

resources. When you use the adaptive retry strategy, we recommend that you use a single client for each resource.

We also recommend that you use this strategy in situations where all clients use the adaptive retry strategy against the resource.

Important

The release of retry strategies with 2.26.0 of the Java SDK includes the new [RetryMode.ADAPTIVE_V2](#) enumeration value. The ADAPTIVE_V2 mode corrects an error that failed to delay the first attempt when throttling errors were detected previously. With the 2.26.0 release, users automatically get the ADAPTIVE_V2 mode behavior by setting the mode as adaptive with an environment variable, system property, or profile setting. There is no adaptive_v2 value for these settings. See the following [the section called “Specify a strategy”](#) section for how to set the mode.

Users can get the previous behavior by setting the mode in code using `RetryMode.ADAPTIVE`.

Summary: Comparison of retry strategy default values

The following table shows the default values for the properties of each retry strategy.

Strategy	Maximum attempts	Base delay for non-throttling errors	Base delay for throttling errors	Token bucket size	Token cost per non-throttling retry	Token cost per throttling retry
Standard	3	100 ms	1000 ms	500	5	5
Legacy	4	100 ms	500 ms	500	5	0
Adaptive	3	100 ms	100 ms	500	5	5

Note

DynamoDB clients use a default maximum retry count of 8 for all retry strategies, which is higher than the values shown in the table above for other AWS service clients.

Specify a strategy

You have four ways to specify a strategy for your service client.

In code

When you build a client, you can configure a lambda expression with a retry strategy. The following snippet configures a standard retry strategy that uses default values on a DynamoDB service client.

```
DynamoDbClient client = DynamoDbClient.builder()
    .overrideConfiguration(o -> o.retryStrategy(RetryMode.STANDARD))
    .build();
```

You can specify `RetryMode.LEGACY` or `RetryMode.ADAPTIVE` in place of `RetryMode.STANDARD`.

As a profile setting

Include `retry_mode` as profile setting in the [shared AWS config file](#). Specify `standard`, `legacy`, or `adaptive` as a value. When set as a profile setting, all service clients that are created while the profile is active use the specified retry strategy with default values. You can override this setting by configuring a retry strategy in code as shown previously.

With the following profile, all service clients use the standard retry strategy.

```
[profile dev]
region = us-east-2
retry_mode = standard
```

As a JVM system property

You can configure a retry strategy for all service clients, unless overridden in code, by using the system property `aws.retryMode`. Specify `standard`, `legacy`, or `adaptive` as a value.

Use the `-D` switch when you invoke Java as shown in the following command.

```
java -Daws.retryMode=standard ...
```

Alternatively, set the system property in the code *before* creating any client as shown in the following snippet.

```
public void main(String[] args) {  
    // Set the property BEFORE any AWS service clients are created.  
    System.setProperty("aws.retryMode", "standard");  
    ...  
}
```

With an environment variable

You can also use the `AWS_RETRY_MODE` environment variable with a value of `standard`, `legacy`, or `adaptive`. As with a profile setting or JVM system property, the environment variable configures all service clients with the specified retry mode unless you configure a client in code.

The following command sets the retry mode to `standard` for the current shell session.

```
export AWS_RETRY_MODE=standard
```

Customize a strategy

You can customize any retry strategy by setting the maximum attempts, backoff strategy, and exceptions that are retryable. You can customize when you build a retry strategy or when you build a client by using an override builder that allows further refinements of the configured strategy.

Customize maximum attempts

You can configure the maximum number of attempts during client construction as shown in the following statement. The following statement customizes the default retry strategy for the client to a maximum of 5 attempt--a first attempt plus 4 retries.

```
DynamoDbClient client = DynamoDbClient.builder()  
    .overrideConfiguration(o -> o.retryStrategy(b -> b.maxAttempts(5)))  
    .build();
```

Alternatively, you can build the strategy and provide it to the client as in the following code example. The following code replaces the standard 3 maximum attempts with 10 and configures a DynamoDB client with the customized strategy.

```
StandardRetryStrategy strategy = AwsRetryStrategy.standardRetryStrategy()  
    .toBuilder()  
    .maxAttempts(10)  
    .build();  
DynamoDbClient client = DynamoDbClient.builder()  
    .overrideConfiguration(o -> o.retryStrategy(strategy))  
    .build();
```

Warning

We recommended that you configure each client with a unique `RetryStrategy` instance. If a `RetryStrategy` instance is shared, failures in one client might affect the retry behavior in the other.

You can also set the maximum number of attempts for all clients by using [external settings](#) instead of code. You configure this setting as describe in the [the section called “Specify a strategy”](#) section.

Customize retryable exceptions

You can configure additional exceptions that trigger retries during client construction. This customization is provided for edge cases where exceptions are thrown that are not included in the default set of retryable exceptions.

The `retryOnException` and `retryOnExceptionOrCause` methods **add** new exception types to the existing set of retryable exceptions; they do not replace the default set. This allows you to extend the retry behavior while maintaining the SDK's default retry capabilities.

The `retryOnExceptionOrCause` method adds a retryable exception if the SDK throws the direct exception or if the exception is wrapped as a cause in another exception.

The following code snippet shows the methods you use to customize the retry exceptions--`retryOnException` and `retryOnExceptionOrCause`. The `retryOnExceptionOrCause` methods adds a retryable exception if the SDK throws the direct exception or if the exception is wrapped.

```
DynamoDbClient client = DynamoDbClient.builder()
    .overrideConfiguration(o -> o.retryStrategy(
        b -> b.retryOnException(EdgeCaseException.class)
            .retryOnExceptionOrCause(WrappedEdgeCaseException.class)))
    .build();
```

Important

Retries are disabled for the Kinesis asynchronous client when you call a [subscribeToShard](#) method, regardless of the retry strategy configuration.

Customize the backoff strategy

You can build the backoff strategy and supply it to the client.

The following code builds a `BackoffStrategy` that replaces the default, standard strategy's exponential delay backoff strategy.

```
BackoffStrategy backoffStrategy =
    BackoffStrategy.exponentialDelay(Duration.ofMillis(150), // The base delay.
                                   Duration.ofSeconds(15)); // The maximum delay.
DynamoDbClient client = DynamoDbClient.builder()
    .overrideConfiguration(o -> o.retryStrategy(
        b -> b.backoffStrategy(backoffStrategy)))
    .build();
```

Migrating from `RetryPolicy` to `RetryStrategy`

`RetryPolicy` (the retry policy API) will be supported for the foreseeable future. If you currently use an instance of `RetryPolicy` to configure your client, everything is going to work as before. Behind the scenes the Java SDK adapts it to a `RetryStrategy`. The new retry strategy interfaces provide the same functionality as a `RetryPolicy` but are created and configured differently.

Configure timeouts in AWS SDK for Java 2.x

The AWS SDK for Java 2.x provides multiple layers of timeout configuration to help you build resilient applications. The SDK offers different types of timeouts that work together to optimize your application's performance and reliability.

There are two main categories of timeouts in the SDK:

- **Service Client Timeouts** - High-level timeouts that control API operations
- **HTTP Client Timeouts** - Low-level timeouts that control network communication

Service client timeouts

Service client timeouts operate at the API level and control the overall behavior of service operations, including retries and multiple attempts.

API call timeout

The API call timeout sets the maximum amount of time for an entire API operation, including all retry attempts. This timeout provides a hard limit on how long your application waits for a complete operation to finish.

```
S3Client s3Client = S3Client.builder()
    .overrideConfiguration(ClientOverrideConfiguration.builder()
        .apiCallTimeout(Duration.ofMinutes(2)) // Total time for entire operation,
        such as when you call the getObject method.
        .build())
    .build();
```

Key characteristics:

- Includes all retry attempts.
- Includes time spent waiting between retries.
- Provides absolute maximum wait time.
- Prevents operations from running indefinitely.

API call attempt timeout

The API call attempt timeout sets the maximum time for a single attempt of an API operation. If this timeout is exceeded, the SDK retries the operation (if retries are configured) rather than failing the entire call.

```
S3Client s3Client = S3Client.builder()
    .overrideConfiguration(ClientOverrideConfiguration.builder()
        .apiCallAttemptTimeout(Duration.ofSeconds(30)) // Time for single attempt.
```

```
.build())  
.build();
```

Key characteristics:

- Applies to individual attempts only.
- Enables fast failure and retry for slow requests.
- Must be shorter than API call timeout.
- Helps identify and recover from transient issues.

Configure service client timeouts

You can configure service client timeouts globally for all operations or per-request:

Global Configuration:

```
S3Client s3Client = S3Client.builder()  
    .overrideConfiguration(b -> b  
        .apiCallTimeout(Duration.ofSeconds(105L))  
        .apiCallAttemptTimeout(Duration.ofSeconds(25L)))  
    .build();  
// When you use the s3Client for an API operation, the SDK uses the configured timeout  
// values.
```

Per-Request Configuration:

```
S3Client basicS3Client = S3Client.create();  
  
// The following configuration uses the same settings as shown before, but these  
// settings  
// apply to only the `putObject` call. When you use `basicS3Client` in another API call  
// without  
// supplying the override configuration, there are no API timeout limits. No timeout  
// limits is the default for the SDK.  
AwsRequestOverrideConfiguration overrideConfiguration =  
    AwsRequestOverrideConfiguration.builder()  
        .apiCallTimeout(Duration.ofSeconds(105L))  
        .apiCallAttemptTimeout(Duration.ofSeconds(25L))  
        .build();  
  
basicS3Client.putObject(b -> b
```



```
.bucket("amzn-s3-demo-bucket")  
.key("example-key")  
.overrideConfiguration(overrideConfiguration),  
RequestBody.fromString("test"));
```

Best practices for API timeouts

The SDK for Java 2.x doesn't set API call timeouts or individual API call attempt timeouts by default. Set timeouts for both individual attempts and the entire request. This helps your application fail fast when transient issues cause request attempts to take longer or when fatal network issues occur.

HTTP client timeouts

HTTP client timeouts operate at the network level and control various aspects of HTTP communication. These timeouts vary depending on which HTTP client implementation you use.

Connection timeout

The connection timeout controls how long to wait when establishing a new connection to the AWS service endpoint.

```
// Available with all HTTP clients.  
ApacheHttpClient.builder()  
    .connectionTimeout(Duration.ofSeconds(5L))  
    .build();
```

Purpose:

- Prevents hanging on network connectivity issues.
- Fails fast when services are unreachable.
- Essential for applications that need responsive error handling.

Socket timeout (Apache and URLConnection clients)

The socket timeout controls how long to wait for data on an established connection.

```
ApacheHttpClient.builder()  
    .socketTimeout(Duration.ofSeconds(30L)) // Time to wait for response data.  
    .build();
```

Read and write timeouts (Netty client)

The Netty client provides separate timeouts for read and write operations:

```
NettyNioAsyncHttpClient.builder()
    .readTimeout(Duration.ofSeconds(30L))    // Reading response data.
    .writeTimeout(Duration.ofSeconds(30L))    // Writing request data.
    .build();
```

TLS negotiation timeout (Netty client)

Controls the time allowed for TLS/SSL handshake:

```
NettyNioAsyncHttpClient.builder()
    .tlsNegotiationTimeout(Duration.ofSeconds(3L))
    .build();
```

Connection pool timeouts

Some HTTP clients provide timeouts for connection pool operations:

```
ApacheHttpClient.builder()
    .connectionAcquisitionTimeout(Duration.ofSeconds(10L)) // Wait for pool
    connection.
    .connectionTimeToLive(Duration.ofMinutes(5L))           // Maximum connection age.
    .connectionMaxIdleTime(Duration.ofSeconds(60L))         // Maximum idle time.
    .build()
```

The [Configure HTTP clients](#) contains more information on HTTP clients in the AWS SDK for Java 2.x

Timeout interactions and hierarchy

Understanding how different timeouts interact is crucial for proper configuration:

Timeout hierarchy

```
API Call Timeout (2 minutes)
### Retry Attempt 1
#   ### API Call Attempt Timeout (45 seconds)
#   ### HTTP Client Timeouts
#       ### Connection Timeout (5 seconds)
#       ### TLS Negotiation Timeout (3 seconds)
```

```
#      ### Read/Write Timeout (30 seconds)
### Retry Attempt 2
#      ### [Same structure as Attempt 1]
### Retry Attempt 3
      ### [Same structure as Attempt 1]
```

Configuration rules

API call timeout $\geq$ API call attempt timeout

```
// Correct configuration.
.apiCallTimeout(Duration.ofMinutes(2))           // 120 seconds.
.apiCallAttemptTimeout(Duration.ofSeconds(30)) // 30 seconds.
```

API call attempt timeout $\geq$ HTTP client timeouts

```
// HTTP client timeouts must be less than attempt timeout.
.apiCallAttemptTimeout(Duration.ofSeconds(30L)) // 30 seconds.
// HTTP client configuration.
.connectionTimeout(Duration.ofSeconds(5L))      // 5 seconds.
.readTimeout(Duration.ofSeconds(25L))           // 25 seconds (< 30).
```

Account for multiple attempts

```
// If you have 3 retry attempts, each taking up to 30 seconds
// API call timeout must be at least 90 seconds plus overhead.
.apiCallTimeout(Duration.ofMinutes(2L))           // 120 seconds.
.apiCallAttemptTimeout(Duration.ofSeconds(30)) // 30 seconds per attempt.
```

Use smart configuration defaults

The SDK provides smart defaults that automatically configure appropriate timeout values:

```
// Enable smart defaults.
S3Client client = S3Client.builder()
    .defaultsMode(DefaultsMode.AUTO) // Automatically choose appropriate defaults.
    .build();

// Available modes:
// - STANDARD: Balanced defaults
// - IN_REGION: Optimized for same-region calls
```

```
// - CROSS_REGION: Optimized for cross-region calls
// - MOBILE: Optimized for mobile applications
// - AUTO: Automatically detect and choose appropriate mode
// - LEGACY: Provides settings that were used before smart defaults existed.
```

Smart defaults automatically configure:

- Connection timeout values.
- TLS negotiation timeout values.
- Other client settings.

Summary

Effective timeout configuration in AWS SDK for Java 2.x requires understanding the interaction between service client timeouts and HTTP client timeouts:

1. **Service client timeouts** control high-level API behavior.
2. **HTTP client timeouts** control low-level network behavior.
3. **Proper hierarchy** ensures timeouts work together effectively.
4. **Smart defaults** provide good starting points for most applications.

Configure timeouts appropriately for your use case to build applications that are both resilient to network issues and responsive to users.

Configuring observability features in the AWS SDK for Java 2.x

Telemetry is the automated collection and transmission of data from remote sources that enable you to monitor and analyze your system's behavior.

Observability in the SDK for Java 2.x provides developers with comprehensive insight into how their Java applications interact with AWS services through rich telemetry data that captures the complete request lifecycle. These capabilities enable you to collect, analyze, and visualize telemetry signals such as metrics, logs, and traces, allowing you to monitor request patterns, identify bottlenecks, and optimize your application's AWS interactions for improved reliability and performance.

Topics

- [Publish SDK metrics from the AWS SDK for Java 2.x](#)
- [Monitoring AWS SDK for Java 2.x applications](#)
- [Logging with the SDK for Java 2.x](#)

Publish SDK metrics from the AWS SDK for Java 2.x

With the AWS SDK for Java 2.x you can collect metrics about the service clients and requests in your application, analyze the output in Amazon CloudWatch Logs, and then act on it.

By default, metrics collection is disabled in the SDK. This topic helps you to enable and configure it.

Getting started with SDK metrics

To enable metrics collection in your application, choose the appropriate implementation of the [MetricPublisher](#) interface based on your use case and follow the detailed setup instructions:

For long-running applications:

- Use [CloudWatchMetricPublisher](#)
- See [Publish SDK metrics from long-running applications](#) for complete setup instructions, code examples, and configuration options.

For AWS Lambda functions:

- Use [EmfMetricLoggingPublisher](#)
- See [Publish SDK metrics for AWS Lambda functions](#) for complete setup instructions, dependencies, and Lambda-specific configuration.

For troubleshooting and console output:

- Use [LoggingMetricPublisher](#)
- See [Output SDK metrics to console for development and debugging](#) for setup instructions, formatting options, and examples for local development and troubleshooting.

Quick implementation preview

Here's what enabling metrics looks like for each use case:

Long-running applications:

```
MetricPublisher metricsPub = CloudWatchMetricPublisher.create();
DynamoDbClient ddb = DynamoDbClient.builder()
    .overrideConfiguration(c -> c.addMetricPublisher(metricsPub))
    .build();
```

Lambda functions:

```
EmfMetricLoggingPublisher emfPublisher = EmfMetricLoggingPublisher.builder()
    .namespace("MyApp")
    .build();
DynamoDbClient dynamoDb = DynamoDbClient.builder()
    .overrideConfiguration(c -> c.addMetricPublisher(emfPublisher))
    .build();
```

Development and debugging:

```
MetricPublisher loggingPublisher = LoggingMetricPublisher.create();
S3Client s3 = S3Client.builder()
    .overrideConfiguration(c -> c.addMetricPublisher(loggingPublisher))
    .build();
```

Metrics limitation of the AWS CRT-based S3 client

The [AWS CRT-based S3 client](#) does not currently support SDK metrics collection. The builder for an AWS CRT-based S3 client instance, [S3CrtAsyncClientBuilder](#), does not provide methods to configure metrics publishers.

When are metrics available?

Metrics are generally available within 5-10 minutes after the SDK for Java emits them. For accurate and up-to-date metrics, check Cloudwatch at least 10 minutes after emitting the metrics from your Java applications.

What information is collected?

Metrics collection includes the following:

- Number of API requests, including whether they succeed or fail
- Information about the AWS services you call in your API requests, including exceptions returned

- The duration for various operations such as Marshalling, Signing, and HTTP requests
- HTTP client metrics, such as the number of open connections, the number of pending requests, and the name of the HTTP client used

Note

The metrics available vary by HTTP client.

For a complete list, see [Service client metrics](#).

How can I use this information?

You can use the metrics the SDK collects to monitor the service clients in your application. You can look at overall usage trends, identify anomalies, review service client exceptions returned, or to dig in to understand a particular issue. Using Amazon CloudWatch Logs, you can also create alarms to notify you as soon as your application reaches a condition that you define.

For more information, see [Using Amazon CloudWatch Logs Metrics](#) and [Using Amazon CloudWatch Logs Alarms](#) in the [Amazon CloudWatch Logs User Guide](#).

Publish SDK metrics from long-running applications using the AWS SDK for Java 2.x

Because the [CloudWatchMetricPublisher](#) implementation aggregates and periodically uploads metrics to Amazon CloudWatch with a delay, its use is best suited for long-running applications.

The default settings of the metrics publisher are meant to minimize memory usage and CloudWatch cost, while still providing a useful amount of insight into the metric data.

Set-up

Before you can enable and use metrics by using `CloudWatchMetricPublisher`, complete the following steps.

Step 1: Add required dependency

Configure your project dependencies (for example, in your `pom.xml` or `build.gradle` file) to use version `2.14.0` or later of the AWS SDK for Java.

Include the artifactId `cloudwatch-metric-publisher` with the version number `2.14.0` or later in your project's dependencies.

For example:

```
<project>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>2.30.11</version>  <!-- Navigate the link to see the latest version.
-->
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>cloudwatch-metric-publisher</artifactId>
    </dependency>
  </dependencies>
</project>
```

Step 2: Configure required permissions

Enable `cloudwatch:PutMetricData` permissions for the IAM identity used by the metrics publisher to allow the SDK for Java to write metrics.

Enable metrics for a specific request

The following class shows how to enable the CloudWatch metrics publisher for a request to Amazon DynamoDB. It uses the default metrics publisher configuration.

```
import software.amazon.awssdk.metrics.MetricPublisher;
import software.amazon.awssdk.metrics.publishers.cloudwatch.CloudWatchMetricPublisher;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;

public class DefaultConfigForRequest {
```



```
// Use one MetricPublisher for your application. It can be used with requests or
// service clients.
static MetricPublisher metricsPub = CloudWatchMetricPublisher.create();

public static void main(String[] args) {
    DynamoDbClient ddb = DynamoDbClient.create();
    // Publish metrics the for ListTables operation.
    ddb.listTables(ListTablesRequest.builder()
        .overrideConfiguration(c -> c.addMetricPublisher(metricsPub))
        .build());

    // Perform more work in your application.

    // A MetricsPublisher has its own lifecycle independent of any service client
    // or request that uses it.
    // If you no longer need the publisher, close it to free up resources.
    metricsPub.close(); // All metrics stored in memory are flushed to CloudWatch.

    // Perform more work with the DynamoDbClient instance without publishing
    // metrics.
    // Close the service client when you no longer need it.
    ddb.close();
}
}
```

Important

Make sure your application calls `close` on the [MetricPublisher](#) instance when the service client is no longer in use. Failure to do so results in possible thread or file descriptor leaks.

Enable summary metrics for a specific service client

The following code snippet shows how to enable a CloudWatch metrics publisher with default settings for a service client.

```
MetricPublisher metricsPub = CloudWatchMetricPublisher.create();

DynamoDbClient ddb = DynamoDbClient.builder()
    .overrideConfiguration(c -> c.addMetricPublisher(metricsPub))
    .build();
```

Customize a CloudWatch metrics publisher

The following class demonstrates how to set up a custom configuration for the metrics publisher for a specific service client. The customizations include loading a specific profile, specifying a AWS Region where the metrics publisher sends requests, and customizing how often the publisher sends metrics to CloudWatch.

```
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.core.metrics.CoreMetric;
import software.amazon.awssdk.metrics.MetricPublisher;
import software.amazon.awssdk.metrics.publishers.cloudwatch.CloudWatchMetricPublisher;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchAsyncClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;

import java.time.Duration;

public class CustomConfigForDDBClient {
    // Use one MetricPublisher for your application. It can be used with requests or
    // service clients.
    static MetricPublisher metricsPub = CloudWatchMetricPublisher.builder()
        .cloudWatchClient(CloudWatchAsyncClient.builder()
            .region(Region.US_WEST_2)
            .credentialsProvider(ProfileCredentialsProvider.create("cloudwatch"))
            .build())
        .uploadFrequency(Duration.ofMinutes(5))
        .maximumCallsPerUpload(100)
        .namespace("ExampleSDKV2Metrics")
        .detailedMetrics(CoreMetric.API_CALL_DURATION)
        .build();

    public static void main(String[] args) {
        DynamoDbClient ddb = DynamoDbClient.builder()
            .overrideConfiguration(c -> c.addMetricPublisher(metricsPub))
            .build();
        // Publish metrics for DynamoDB operations.
        ddb.listTables();
        ddb.describeEndpoints();
        ddb.describeLimits();
        // Perform more work in your application.

        // A MetricsPublisher has its own lifecycle independent of any service client
        // or request that uses it.
    }
}
```

```
// If you no longer need the publisher, close it to free up resources.
metricsPub.close(); // All metrics stored in memory are flushed to CloudWatch.

// Perform more work with the DynamoDbClient instance without publishing
metrics.
// Close the service client when you no longer need it.
ddb.close();
}
}
```

The customizations shown in the previous snippet have the following effects.

- The `cloudWatchClient` method lets you customize the CloudWatch client used to send metrics. In this example, we use a different region from the default of *us-east-1* where the client sends metrics. We also use a different named profile, *cloudwatch*, whose credentials will be used to authenticate requests to CloudWatch. Those credentials must have permissions to `cloudwatch:PutMetricData`.
- The `uploadFrequency` method allows you to specify how frequently the metrics publisher uploads metrics to CloudWatch. The default is once a minute.
- The `maximumCallsPerUpload` method limits the number of calls made per upload. The default is unlimited.
- By default, the SDK for Java 2.x publishes metrics under the namespace `AwsSdk/JavaSdk2`. You can use the `namespace` method to specify a different value.
- By default, the SDK publishes summary metrics. Summary metrics consist of average, minimum, maximum, sum, and sample count. By specifying one or more SDK metrics in the `detailedMetrics` method, the SDK publishes additional data for each metric. This additional data enables percentile statistics like p90 and p99 that you can query in CloudWatch. The detailed metrics are especially useful for latency metrics like `APICallDuration`, which measures the end-to-end latency for SDK client requests. You can use fields of the [CoreMetric](#) class to specify other common SDK metrics.

Next steps: If you're also working with Lambda functions, see [Publish SDK metrics for AWS Lambda functions](#) for EMF-based metrics publishing.

Publish SDK metrics for AWS Lambda functions using the AWS SDK for Java 2.x

Because Lambda functions typically execute for milliseconds to minutes, any delay in sending the metrics, which happens with the `CloudWatchMetricPublisher`, risks the loss of data.

[EmfMetricLoggingPublisher](#) provides a more suitable approach by immediately writing metrics as structured log entries in [CloudWatch Embedded Metric Format \(EMF\)](#).

`EmfMetricLoggingPublisher` works in execution environments that have built-in integration with Amazon CloudWatch Logs such as AWS Lambda and Amazon Elastic Container Service.

Set-up

Before you can enable and use metrics by using `EmfMetricLoggingPublisher`, complete the following steps.

Step 1: Add required dependency

Configure your project dependencies (for example, in your `pom.xml` or `build.gradle` file) to use version `2.30.3` or later of the AWS SDK for Java.

Include the artifactId `emf-metric-logging-publisher` with the version number `2.30.3` or later in your project's dependencies.

For example:

```
<project>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>2.30.11</version>  <!-- Navigate the link to see the latest version.
-->
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>emf-metric-logging-publisher</artifactId>
    </dependency>
```

```
</dependencies>
</project>
```

Step 2: Configure required permissions

Enable `logs:PutLogEvents` permissions for the IAM identity used by the metrics publisher to allow the SDK for Java to write EMF-formatted logs.

Step 3: Setup logging

To ensure proper metric collection, configure your logging to output to the console at the INFO level or lower (such as DEBUG). In your `log4j2.xml` file:

```
<Loggers>
  <Root level="WARN">
    <AppenderRef ref="ConsoleAppender"/>
  </Root>
  <Logger
    name="software.amazon.awssdk.metrics.publishers.emf.EmfMetricLoggingPublisher"
    level="INFO" />
</Loggers>
```

See the [logging topic](#) in this guide for more information on how to set up a `log4j2.xml` file.

Configure and use `EmfMetricLoggingPublisher`

The following Lambda function class first creates and configures an `EmfMetricLoggingPublisher` instance and then uses it with a Amazon DynamoDB service client:

```
public class GameIdHandler implements RequestHandler<Map<String, String>, String> {
    private final EmfMetricLoggingPublisher emfPublisher;
    private final DynamoDbClient dynamoDb;

    public GameIdHandler() {
        // Build the publisher.
        this.emfPublisher = EmfMetricLoggingPublisher.builder()
            .namespace("namespace")
            .dimensions(CoreMetric.SERVICE_ID,
                CoreMetric.OPERATION_NAME)
            .build();
        // Add the publisher to the client.
```

```
        this.dynamoDb = DynamoDbClient.builder()
            .overrideConfiguration(c -> c.addMetricPublisher(emfPublisher))
            .region(Region.of(System.getenv("AWS_REGION")))
            .build();
    }

    @Override
    public String handleRequest(Map<String, String> event, Context context) {
        Map<String, AttributeValue> gameItem = new HashMap<>();

        gameItem.put("gameId", AttributeValue.builder().s(event.get("id")).build());

        PutItemRequest putItemRequest = PutItemRequest.builder()
            .tableName("games")
            .item(gameItem)
            .build();

        dynamoDb.putItem(putItemRequest);

        return "Request handled";
    }
}
```

When the DynamoDB client executes the `putItem` method, it automatically publishes metrics to a CloudWatch log stream in EMF format.

Example of an EMF log event

For example, if you send the following event to the GameHandler Lambda function with logging configured as shown previously:

```
{
  "id": "23456"
}
```

After the function processes the event, you find two log events that look similar to the following example. The JSON object in the second event contains the Java SDK metric data for the `PutItem` operation to DynamoDB.

When CloudWatch receives a log event in EMF format, it automatically parses the structured JSON to extract metric data. CloudWatch then creates corresponding metrics while storing the original log entry in CloudWatch Logs.

```
2025-07-11 15:58:30 [main] INFO org.example.GameIdHandler:39 - Received map:
{id=23456}
```

```
2025-07-11 15:58:34 [main] INFO
software.amazon.awssdk.metrics.publishers.emf.EmfMetricLoggingPublisher:43 -
{
```

```
  "_aws": {
    "Timestamp": 1752249513975,
    "LogGroupName": "/aws/lambda/GameId",
    "CloudWatchMetrics": [
      {
        "Namespace": "namespace",
        "Dimensions": [
          "OperationName",
          "ServiceId"
        ],
      },
      "Metrics": [
        {
          "Name": "AvailableConcurrency"
        },
        {
          "Name": "PendingConcurrencyAcquires"
        },
        {
          "Name": "ServiceCallDuration",
          "Unit": "Milliseconds"
        },
        {
          "Name": "EndpointResolveDuration",
          "Unit": "Milliseconds"
        },
        {
          "Name": "MaxConcurrency"
        },
        {
          "Name": "BackoffDelayDuration",
          "Unit": "Milliseconds"
        },
        {
          "Name": "MarshallingDuration",
          "Unit": "Milliseconds"
        }
      ]
    }
  }
```

```

        },
        {
            "Name": "LeasedConcurrency"
        },
        {
            "Name": "SigningDuration",
            "Unit": "Milliseconds"
        },
        {
            "Name": "ConcurrencyAcquireDuration",
            "Unit": "Milliseconds"
        },
        {
            "Name": "ApiCallSuccessful"
        },
        {
            "Name": "RetryCount"
        },
        {
            "Name": "UnmarshallingDuration",
            "Unit": "Milliseconds"
        },
        {
            "Name": "ApiCallDuration",
            "Unit": "Milliseconds"
        },
        {
            "Name": "CredentialsFetchDuration",
            "Unit": "Milliseconds"
        }
    ]
}

]
},
"AvailableConcurrency": 0,
"PendingConcurrencyAcquires": 0,
"OperationName": "PutItem",
"ServiceCallDuration": 1339,
"EndpointResolveDuration": 81,
"MaxConcurrency": 50,
"BackoffDelayDuration": 0,
"ServiceId": "DynamoDB",
"MarshallingDuration": 181,
"LeasedConcurrency": 1,

```



```
"SigningDuration": 184,  
"ConcurrencyAcquireDuration": 83,  
"ApiCallSuccessful": 1,  
"RetryCount": 0,  
"UnmarshallingDuration": 85,  
"ApiCallDuration": 1880,  
"CredentialsFetchDuration": 138  
}
```

The [API documentation](#) for `EmfMetricLoggingPublisher.Builder` shows the configuration options that you can use.

You can also enable EMF metric logging for a single request as [shown for the CloudWatchMetricPublisher](#).

Next steps: For long-running applications, see [Publish SDK metrics from long-running applications](#) for CloudWatch-based metrics publishing.

Output SDK metrics to the console using the AWS SDK for Java 2.x

The [LoggingMetricPublisher](#) implementation outputs metrics directly to your application's console or log files. This approach is ideal for development, debugging, and understanding what metrics the SDK collects without requiring external services like Amazon CloudWatch.

Unlike `CloudWatchMetricPublisher` and `EmfMetricLoggingPublisher`, `LoggingMetricPublisher` provides immediate output with no delays or external dependencies. This makes it perfect for local development and troubleshooting scenarios.

When to use LoggingMetricPublisher

Use `LoggingMetricPublisher` when you need to:

- Debug metric collection during development
- Understand what metrics the SDK collects for your operations
- Troubleshoot performance issues locally
- Test metric collection without external service dependencies
- View metrics immediately in your console or log files

Note

LoggingMetricPublisher is not recommended for production environments where you need persistent metric storage and analysis capabilities.

Set up console logging for metrics

To see LoggingMetricPublisher output, configure your logging framework to display INFO level messages. The following log4j2.xml configuration ensures metrics appear in your console:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg
%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="INFO">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
    <!-- Ensure LoggingMetricPublisher output appears. -->
    <Logger name="software.amazon.awssdk.metrics.LoggingMetricPublisher"
level="INFO" />
  </Loggers>
</Configuration>
```

This configuration directs the SDK to output metrics to your console at the INFO level. The LoggingMetricPublisher logger configuration ensures that metric output appears even if your root logger uses a higher level like WARN or ERROR.

Enable console metrics for a service client

The following example shows how to create a LoggingMetricPublisher and use it with an Amazon Simple Storage Service client:

```
import software.amazon.awssdk.metrics.LoggingMetricPublisher;
import software.amazon.awssdk.metrics.MetricPublisher;
import software.amazon.awssdk.regions.Region;
```

```
import software.amazon.awssdk.services.s3.S3Client;

// Create a LoggingMetricPublisher with default settings.
MetricPublisher metricPublisher = LoggingMetricPublisher.create();

// Add the publisher to your service client.
S3Client s3Client = S3Client.builder()
    .region(Region.US_EAST_1)
    .overrideConfiguration(config -> config.addMetricPublisher(metricPublisher))
    .build();

// Make requests - metrics will appear in your console.
s3Client.listBuckets();

// Clean up resources.
metricPublisher.close();
s3Client.close();
```

Choose metric output format

LoggingMetricPublisher supports two output formats:

- **PLAIN format (default):** Outputs metrics as compact, single-line entries
- **PRETTY format:** Outputs metrics in a multi-line, human-readable format

The following example shows how to use the PRETTY format for easier reading during development:

```
import org.slf4j.event.Level;
import software.amazon.awssdk.metrics.LoggingMetricPublisher;

// Create a LoggingMetricPublisher with PRETTY format.
MetricPublisher prettyMetricPublisher = LoggingMetricPublisher.create(
    Level.INFO,
    LoggingMetricPublisher.Format.PRETTY
);

// Use with your service client.
S3Client s3Client = S3Client.builder()
    .region(Region.US_EAST_1)
    .overrideConfiguration(config -> config.addMetricPublisher(prettyMetricPublisher))
    .build();
```

Complete example

The following example demonstrates using `LoggingMetricPublisher` in two ways:

- At the service client level
- For a single request

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.event.Level;
import software.amazon.awssdk.metrics.LoggingMetricPublisher;
import software.amazon.awssdk.metrics.MetricPublisher;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListBucketsRequest;
import software.amazon.awssdk.services.s3.model.ListBucketsResponse;

/**
 * Demonstrates how to use LoggingMetricPublisher with AWS S3 SDK for Java 2.x.
 * <p>
 * This demo focuses on the S3 listBuckets operation to show how metrics are collected
 * and logged to the console for development and debugging purposes.
 * <p>
 * LoggingMetricPublisher is ideal for:
 * - Development and debugging
 * - Console output for troubleshooting
 * - Understanding what metrics are being collected
 * - Testing metric collection without external dependencies
 */
public class S3LoggingMetricPublisherDemo {

    private static final Logger logger =
        LoggerFactory.getLogger(S3LoggingMetricPublisherDemo.class);

    public static void main(String[] args) {
        S3LoggingMetricPublisherDemo demo = new S3LoggingMetricPublisherDemo();
        demo.demonstrateUsage();
    }

    /**
     * Demonstrates basic usage with S3Client and metrics enabled at the client level.
     */
}
```

```
private void demonstrateUsage() {

    // Create a LoggingMetricPublisher with default settings. The SDK logs metrics
    as text in a single line.
    // The default settings are equivalent to using
    `LoggingMetricPublisher.Format.PLAIN`.

    MetricPublisher metricPublisher = LoggingMetricPublisher.create();

    // Create an S3 client with metrics enabled.
    try (S3Client s3Client = S3Client.builder()
        .region(Region.US_EAST_1)
        .overrideConfiguration(config ->
            config.addMetricPublisher(metricPublisher))
        .build()) {

        // Make the listBuckets request - metrics will be logged to console.
        ListBucketsResponse response =
            s3Client.listBuckets(ListBucketsRequest.builder().build());

        // The next block shows the using a different LoggingMetricPublisher with a
        `PRETTY` format.
        // Since the metric publisher is added to the request using the
        `overrideConfiguration`, this formatting
        // applies only to the one request.
        try {
            s3Client.listBuckets(ListBucketsRequest.builder()
                .overrideConfiguration(config -> config
                    .addMetricPublisher(LoggingMetricPublisher.create(
                        Level.INFO,
                        LoggingMetricPublisher.Format.PRETTY)))
                .build());
        } catch (Exception e) {
            logger.info("Request failed with metrics logged: {}", e.getMessage());
        }
        logger.info("Found {} buckets in your AWS account.",
            response.buckets().size());

    } catch (Exception e) {
        logger.error("Error during S3 operation: {}", e.getMessage());
        logger.info("Note: This is expected if AWS credentials are not
            configured.");
    }
}
```

```

        // Close the metric publisher to flush any remaining metrics.
        metricPublisher.close();
    }
}

```

The code logs the following to the console:

```

INFO LoggingMetricPublisher - Metrics published: MetricCollection(name=ApiCall,
metrics=[MetricRecord(metric=MarshallingDuration, value=PT0.005409792S),
MetricRecord(metric=RetryCount, value=0), MetricRecord(metric=ApiCallSuccessful,
value=true), MetricRecord(metric=OperationName, value=ListBuckets),
MetricRecord(metric=EndpointResolveDuration, value=PT0.000068S),
MetricRecord(metric=ApiCallDuration, value=PT0.163802958S),
MetricRecord(metric=CredentialsFetchDuration, value=PT0.145686542S),
MetricRecord(metric=ServiceEndpoint, value=https://s3.amazonaws.com),
MetricRecord(metric=ServiceId, value=S3)],
children=[MetricCollection(name=ApiCallAttempt,
metrics=[MetricRecord(metric=TimeToFirstByte, value=PT0.138816S),
MetricRecord(metric=SigningDuration, value=PT0.007803459S),
MetricRecord(metric=ReadThroughput, value=165153.96002660287),
MetricRecord(metric=ServiceCallDuration, value=PT0.138816S),
MetricRecord(metric=AwsExtendedRequestId, value=e13Swj3uwn0qP10z
+m7II50Gq7jf8xxT8H18iDfRBCQmDg+gU4ek91Xrsl8XxRLR0lIzCAPQtsQF0DAAW0b8ntuKCzX2AJdj),
MetricRecord(metric=HttpStatusCode, value=200),
MetricRecord(metric=BackoffDelayDuration, value=PT0S),
MetricRecord(metric=TimeToLastByte, value=PT0.148915667S),
MetricRecord(metric=AwsRequestId, value=78AW9BM7SWR6YMGB)],
children=[MetricCollection(name=HttpClient,
metrics=[MetricRecord(metric=MaxConcurrency, value=50),
MetricRecord(metric=AvailableConcurrency, value=0),
MetricRecord(metric=LeasedConcurrency, value=1),
MetricRecord(metric=ConcurrencyAcquireDuration, value=PT0.002623S),
MetricRecord(metric=PendingConcurrencyAcquires, value=0),
MetricRecord(metric=HttpClientName, value=Apache)], children=[])]])
INFO LoggingMetricPublisher - [4e6f2bb5] ApiCall
INFO LoggingMetricPublisher - [4e6f2bb5] #####
INFO LoggingMetricPublisher - [4e6f2bb5] # MarshallingDuration=PT0.000063S #
INFO LoggingMetricPublisher - [4e6f2bb5] # RetryCount=0 #
INFO LoggingMetricPublisher - [4e6f2bb5] # ApiCallSuccessful=true #
INFO LoggingMetricPublisher - [4e6f2bb5] # OperationName=ListBuckets #
INFO LoggingMetricPublisher - [4e6f2bb5] # EndpointResolveDuration=PT0.000024375S #
INFO LoggingMetricPublisher - [4e6f2bb5] # ApiCallDuration=PT0.018463083S #
INFO LoggingMetricPublisher - [4e6f2bb5] # CredentialsFetchDuration=PT0.000022334S #

```

```

INFO LoggingMetricPublisher - [4e6f2bb5] # ServiceEndpoint=https://s3.amazonaws.com #
INFO LoggingMetricPublisher - [4e6f2bb5] # ServiceId=S3 #
INFO LoggingMetricPublisher - [4e6f2bb5] #####
INFO LoggingMetricPublisher - [4e6f2bb5] ApiCallAttempt
INFO LoggingMetricPublisher - [4e6f2bb5]
#####
INFO LoggingMetricPublisher - [4e6f2bb5] # TimeToFirstByte=PT0.0165575S
#
INFO LoggingMetricPublisher - [4e6f2bb5] # SigningDuration=PT0.000301125S
#
INFO LoggingMetricPublisher - [4e6f2bb5] # ReadThroughput=1195591.792850103
#
INFO LoggingMetricPublisher - [4e6f2bb5] # ServiceCallDuration=PT0.0165575S
#
INFO LoggingMetricPublisher - [4e6f2bb5] #
  AwsExtendedRequestId=3QI1eenRuokdszWqZBmBMDUmko6F1SmHkM
+CUMNMelOr7gJml4D4lv6QXUZ1zWoTgG+tHbr6yo2vHdz4h1P8PDovvtMFRCEB #
INFO LoggingMetricPublisher - [4e6f2bb5] # HttpStatusCode=200
#
INFO LoggingMetricPublisher - [4e6f2bb5] # BackoffDelayDuration=PT0S
#
INFO LoggingMetricPublisher - [4e6f2bb5] # TimeToLastByte=PT0.017952625S
#
INFO LoggingMetricPublisher - [4e6f2bb5] # AwsRequestId=78AVFAF795AAWAXH
#
INFO LoggingMetricPublisher - [4e6f2bb5]
#####
INFO LoggingMetricPublisher - [4e6f2bb5] HttpClient
INFO LoggingMetricPublisher - [4e6f2bb5]
#####
INFO LoggingMetricPublisher - [4e6f2bb5] # MaxConcurrency=50
#
INFO LoggingMetricPublisher - [4e6f2bb5] # AvailableConcurrency=0
#
INFO LoggingMetricPublisher - [4e6f2bb5] # LeasedConcurrency=1
#
INFO LoggingMetricPublisher - [4e6f2bb5] #
  ConcurrencyAcquireDuration=PT0.00004S #
INFO LoggingMetricPublisher - [4e6f2bb5] # PendingConcurrencyAcquires=0
#
INFO LoggingMetricPublisher - [4e6f2bb5] # HttpClientName=Apache
#
INFO LoggingMetricPublisher - [4e6f2bb5]
#####

```

INFO S3LoggingMetricPublisherDemo - Found 6 buckets in your AWS account.

Additional artifacts for the example

Maven pom.xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>s3-logging-metric-publisher-demo</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>AWS S3 LoggingMetricPublisher Demo</name>
  <description>Demonstrates how to use LoggingMetricPublisher with AWS S3 SDK for
    Java 2.x</description>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <aws.java.sdk.version>2.31.66</aws.java.sdk.version>
    <log4j.version>2.24.3</log4j.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <!-- AWS SDK BOM for dependency management -->
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.java.sdk.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>

      <!-- Log4j BOM for logging dependency management -->
      <dependency>
        <groupId>org.apache.logging.log4j</groupId>
```



```
        <artifactId>log4j-bom</artifactId>
        <version>${log4j.version}</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>

<dependencies>
    <!-- AWS S3 SDK for demonstration -->
    <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>s3</artifactId>
    </dependency>

    <!-- Log4j2 SLF4J implementation -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-slf4j2-impl</artifactId>
    </dependency>

    <!-- Log4j2 Core -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <source>17</source>
                <target>17</target>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>
```

Log4j2.xml configuration file

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg
%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="INFO">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
    <!-- Ensure LoggingMetricPublisher output appears. -->
    <Logger name="software.amazon.awssdk.metrics.LoggingMetricPublisher"
level="INFO"/>
  </Loggers>
</Configuration>
```

The metrics include timing information, service details, operation names, and HTTP status codes that help you understand your application's AWS API usage patterns.

Next steps

After using `LoggingMetricPublisher` for development and debugging, consider these options for production environments:

- For long-running applications, use [CloudWatchMetricPublisher](#) to send metrics to Amazon CloudWatch for analysis and alerting
- For AWS Lambda functions, use [EmfMetricLoggingPublisher](#) to publish metrics in CloudWatch Embedded Metric Format

AWS SDK for Java 2.x: Comprehensive Metrics Reference

With the AWS SDK for Java 2.x, you can collect metrics from the service clients in your application and then publish (output) those metrics to [Amazon CloudWatch](#).

These tables list the metrics that you can collect and any HTTP client usage requirement.

For more information about enabling and configuring metrics for the SDK, see [Enabling SDK metrics](#).

Metrics collected with each request

Metric name	Description	Type
ApiCallDuration	The duration of the API call. This includes all call attempts made.	Duration*
ApiCallSuccessful	True if the API call succeeded, false otherwise.	Boolean
CredentialsFetchDuration	The duration of time to fetch signing credentials for the API call.	Duration*
EndpointResolveDuration	The duration of time to resolve the endpoint used for the API call.	Duration*
MarshallingDuration	The duration of time to marshall the SDK request to an HTTP request.	Duration*
OperationName	The name of the service operation being invoked.	String
RetryCount	The number of retries that the SDK performed in the execution of the request. 0 implies that the request worked the first time and that no retries were attempted. For more information about configuring retry behavior, see Retry strategies.	Integer

Metric name	Description	Type
ServiceId	The unique ID for the service.	String
ServiceEndpoint	The endpoint for the service.	URI
TokenFetchDuration	The duration of time to fetch signing credentials for the API call.	Duration*

*[java.time.Duration](#).

Metrics collected for each request attempt

Each API call might require multiple attempts before a response is received. These metrics are collected for each attempt.

Core metrics

Metric name	Description	Type
AwsExtendedRequestId	The extended request ID of the service request.	String
AwsRequestId	The request ID of the service request.	String
BackoffDelayDuration	The duration of time that the SDK has waited before this API call attempt. The value is based on the BackoffStrategy set on the client. See the Retry strategies section in this guide for more information.	Duration*
ErrorType	The type of error that occurred for a call attempt.	String

Metric name	Description	Type
	The following are possible values: • <code>Throttling</code> : The service responded with a throttling error. • <code>ServerError</code> : The service responded with an error other than throttling. • <code>ConfiguredTimeout</code> : A client timeout occurred, either at the API call level, or API call attempt level. • <code>IO</code>: An I/O error occurred. • <code>Other</code>: Catch-all for other errors that don't fall into one of categories list above.	

Metric name	Description	Type
ReadThroughput	The read throughput of the client, defined as $\text{NumberOfResponseBytesRead} / (\text{TTLB} - \text{TTFB})$. This value is in bytes per second. Note that this metric only measures the bytes read from within the <code>ResponseTransformer</code> or <code>AsyncResponseTransformer</code>. Data that is read outside the transformer—for example when the response stream is returned as the result of the transformer—is not included in the calculation.	Double

Metric name	Description	Type
WriteThroughput	The write throughput of the client, defined as $\text{RequestBytesWritten} / (\text{LastByteWrittenTime} - \text{FirstByteWrittenTime})$. This value is in bytes per second. This metric measures the rate at which the SDK provides the request body to the HTTP client. It excludes connection setup, TLS handshake time, and server processing time. This metric is only reported for requests that have a streaming body such as S3 PutObject. Note that this metric does not account for buffering in the HTTP client layer. The actual network transmission rate may be lower if the HTTP client buffers data before sending. This metric represents an upper bound of the network throughput.	Double

Metric name	Description	Type
ServiceCallDuration	The duration of time to connect to the service (or acquire a connection from the connection pool), send the serialized request and receive the initial response (for example HTTP status code and headers). This DOES NOT include the time to read the entire response from the service.	Duration*
SigningDuration	The duration of time to sign the HTTP request.	Duration*
TimeToFirstByte	The duration of time from sending the HTTP request (including acquiring a connection) to the service, and receiving the first byte of the headers in the response.	Duration*

Metric name	Description	Type
TimeToLastByte	The duration of time from sending the HTTP request (including acquiring a connection) to the service, and receiving the last byte of the response. Note that for APIs that return streaming responses, this metric spans the time until the <code>ResponseTransformer</code> or <code>AsyncResponseTransformer</code> completes.	Duration*
UnmarshallingDuration	The duration of time to unmarshall the HTTP response to an SDK response. Note: For streaming operations, this does not include the time to read the response payload.	Duration*

*[java.time.Duration](#).

HTTP Metrics

Metric name	Description	Type	HTTP client required*
Available Concurrency	The number of additional concurrent requests that the HTTP client supports without establishing new connections to the target server.	Integer	Apache, Netty, CRT

Metric name	Description	Type	HTTP client required*
	For HTTP/1 operations, this equals the number of idle TCP connections established with the service. For HTTP/2 operations, this equals the number of idle streams. Note: This value varies by HTTP client implementation: • Apache client: Value applies to the entire HTTP client • Netty client: Value applies per endpoint • AWS CRT-based client: Value applies per endpoint The value is scoped to an individual HTTP client instance and excludes concurrency from other HTTP clients in the same JVM.		
ConcurrencyAcquireDuration	The duration of time to acquire a channel from the connection pool. For HTTP/1 operations, a channel equals a TCP connection. For HTTP/2 operations, a channel equals an HTTP/2 stream channel. Acquiring a new channel may include time for: 1. Awaiting a concurrency permit, as restricted by the client's max concurrency configuration. 2. Establishing a new connection, if no existing connection is available in the pool. 3. Performing the TLS handshake and negotiation, if TLS is enabled.	Duration	Apache, Netty, CRT
HttpClientName	The name of the HTTP used for the request.	String	Apache, Netty, CRT

Metric name	Description	Type	HTTP client required*
HttpStatusCode	The status code of the HTTP response.	Integer	Any
LeasedConcurrency	The number of requests that the HTTP client currently executes. For HTTP/1 operations, this equals the number of active TCP connections with the service (excluding idle connections). For HTTP/2 operations, this equals the number of active HTTP streams with the service (excluding idle stream capacity). Note: This value varies by HTTP client implementation: • Apache client: Value applies to the entire HTTP client • Netty client: Value applies per endpoint • AWS CRT-based client: Value applies per endpoint The value is scoped to an individual HTTP client instance and excludes concurrency from other HTTP clients in the same JVM.	Integer	Apache, Netty, CRT
LocalStreamWindowSize	The local HTTP/2 window size in bytes for the stream that executes this request.	Integer	Netty

Metric name	Description	Type	HTTP client required*
MaxConcurrency	The maximum number of concurrent requests that the HTTP client supports. For HTTP/1 operations, this equals the maximum number of TCP connections that the HTTP client can pool. For HTTP/2 operations, this equals the maximum number of streams that the HTTP client can pool. Note: This value varies by HTTP client implementation: • Apache client: Value applies to the entire HTTP client • Netty client: Value applies per endpoint • AWS CRT-based client: Value applies per endpoint The value is scoped to an individual HTTP client instance and excludes concurrency from other HTTP clients in the same JVM.	Integer	Apache, Netty, CRT

Metric name	Description	Type	HTTP client required*
PendingConcurrencyAcquires	The number of requests that wait for concurrency from the HTTP client. For HTTP/1 operations, this equals the number of requests waiting for a TCP connection to establish or return from the connection pool. For HTTP/2 operations, this equals the number of requests waiting for a new stream (and possibly a new HTTP/2 connection) from the connection pool. Note: This value varies by HTTP client implementation: • Apache client: Value applies to the entire HTTP client • Netty client: Value applies per endpoint • AWS CRT-based client: Value applies per endpoint The value is scoped to an individual HTTP client instance and excludes concurrency from other HTTP clients in the same JVM.	Integer	Apache, Netty, CRT
RemoteStreamWindowSize	The remote HTTP/2 window size in bytes for the stream that executes this request.	Integer	Netty

*[java.time.Duration](#).

The terms used in the column mean:

- Apache: the Apache-based HTTP client ([ApacheHttpClient](#))
- Netty: the Netty-based HTTP client ([NettyNioAsyncHttpClient](#))
- CRT: the AWS CRT-based HTTP client ([AwsCrtAsyncHttpClient](#))

- Any: the collection of metric data does not depend on the HTTP client; this includes the URLConnection-based HTTP client ([URLConnectionHttpClient](#))

Monitoring AWS SDK for Java 2.x applications

Monitoring is an important part of maintaining the reliability, availability, and performance of applications using the AWS SDK for Java 2.x. AWS provides the following monitoring tools to watch SDK for Java 2.x, report when something is wrong, and take automatic actions when appropriate:

- *Amazon CloudWatch* monitors your AWS resources and the applications you run on AWS in real time. You can collect and track metrics, create customized dashboards, and set alarms that notify you or take actions when a specified metric reaches a threshold that you specify. For example, you can have CloudWatch track CPU usage or other metrics of your Amazon EC2 instances and automatically launch new instances when needed. For more information, see the [Amazon CloudWatch User Guide](#).
- *Amazon CloudWatch Logs* enables you to monitor, store, and access your log files from Amazon EC2 instances, CloudTrail, and other sources. CloudWatch Logs can monitor information in the log files and notify you when certain thresholds are met. You can also archive your log data in highly durable storage. For more information, see the [Amazon CloudWatch Logs User Guide](#).
- *AWS CloudTrail* captures API calls and related events made by or on behalf of your AWS account and delivers the log files to an Amazon S3 bucket that you specify. You can identify which users and accounts called AWS, the source IP address from which the calls were made, and when the calls occurred. For more information, see the [AWS CloudTrail User Guide](#).

Logging with the SDK for Java 2.x

The AWS SDK for Java 2.x uses [SLF4J](#), which is an abstraction layer that enables the use of any one of several logging systems at runtime.

Supported logging systems include the Java Logging Framework and Apache [Log4j 2](#), among others. This topic shows you how to use Log4j 2 as the logging system for working with the SDK.

Log4j 2 configuration file

You typically use a configuration file, named `log4j2.xml` with Log4j 2. Example configuration files are shown below. To learn more about the values used in the configuration file, see the [manual for Log4j configuration](#).

The `log4j2.xml` file needs to be on the classpath when your application starts up. For a Maven project, put the file in the `<project-dir>/src/main/resources` directory.

The `log4j2.xml` configuration file specifies properties such as [logging level](#), where logging output is sent (for example, [to a file or to the console](#)), and the [format of the output](#). The logging level specifies the level of detail that Log4j 2 outputs. Log4j 2 supports the concept of multiple logging [hierarchies](#). The logging level is set independently for each hierarchy. The main logging hierarchy that you use with the AWS SDK for Java 2.x is `software.amazon.awssdk`.

Add logging dependency

To configure the Log4j 2 binding for SLF4J in your build file, use the following.

Maven

Add the following elements to your `pom.xml` file.

```
...
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-slf4j2-impl</artifactId>
  <version>VERSION</version>
</dependency>
...
```

Gradle–Kotlin DSL

Add the following to your `build.gradle.kts` file.

```
...
dependencies {
  ...
  implementation("org.apache.logging.log4j:log4j-slf4j2-impl:VERSION")
  ...
}
...
```

Use `2.20.0` for the minimum version of the `log4j-slf4j2-impl` artifact. For the latest version, use the version published to [Maven central](#). Replace `VERSION` with version you'll use.

SDK-specific errors and warnings

We recommend that you always leave the "software.amazon.awssdk" logger hierarchy set to "WARN" to catch any important messages from the SDK's client libraries. For example, if the Amazon S3 client detects that your application hasn't properly closed an `InputStream` and could be leaking resources, the S3 client reports it through a warning message to the logs. This also ensures that messages are logged if the client has any problems handling requests or responses.

The following `log4j2.xml` file sets the `rootLogger` to "WARN", which causes warning and error-level messages from all loggers in the application to be output, *including* those in the "software.amazon.awssdk" hierarchy. Alternatively, you can explicitly set the "software.amazon.awssdk" logger hierarchy to "WARN" if `<Root level="ERROR">` is used.

Example Log4j2.xml configuration file

This configuration will log messages at the "ERROR" and "WARN" levels to the console for all logger hierarchies.

```
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{YYYY-MM-dd HH:mm:ss} [%t] %-5p %c:%L - %m%n" />
    </Console>
  </Appenders>

  <Loggers>
    <Root level="WARN">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
  </Loggers>
</Configuration>
```

Request/response summary logging

Every request to an AWS service generates a unique AWS request ID that is useful if you run into an issue with how an AWS service is handling a request. AWS request IDs are accessible programmatically through [SdkServiceException](#) objects in the SDK for any failed service call, and can also be reported through the "DEBUG" log level of the "software.amazon.awssdk.request" logger.

The following `log4j2.xml` file enables a summary of requests and responses.


```
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{YYYY-MM-dd HH:mm:ss} [%t] %-5p %c:%L - %m%n" />
    </Console>
  </Appenders>

  <Loggers>
    <Root level="ERROR">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
    <Logger name="software.amazon.awssdk" level="WARN" />
    <Logger name="software.amazon.awssdk.request" level="DEBUG" />
  </Loggers>
</Configuration>
```

Here is an example of the log output:

```
2022-09-23 16:02:08 [main] DEBUG software.amazon.awssdk.request:85 - Sending Request:
DefaultSdkHttpRequest(httpMethod=POST, protocol=https, host=dynamodb.us-
east-1.amazonaws.com, encodedPath=/, headers=[amz-sdk-invocation-id, Content-Length,
Content-Type, User-Agent, X-Amz-Target], queryParameters=[])
2022-09-23 16:02:08 [main] DEBUG software.amazon.awssdk.request:85 - Received
successful response: 200, Request ID:
QS9DUMME2NHEDH8TGT9N5V530JVV4KQNS05AEMVJF66Q9ASUAAJG, Extended Request ID: not
available
```

If you are interested in only the request ID use `<Logger name="software.amazon.awssdk.requestId" level="DEBUG" />`.

Debug-level SDK logging

If you need more detail about what the SDK is doing, you can set the logging level of the `software.amazon.awssdk` logger to `DEBUG`. At this level, the SDK outputs a large amount of detail, so we recommend that you set this level to resolve errors using integration tests.

At this logging level, the SDK logs information about configuration, credentials resolution, execution interceptors, high-level TLS activity, request signing, and much more.

The following is a sampling of statements that are output by the SDK at `DEBUG` level for a `S3Client#listBuckets()` call.

```

DEBUG s.a.a.r.p.AwsRegionProviderChain:57 - Unable to load region from
software.amazon.awssdk.regions.providers.SystemSettingsRegionProvider@324dcd31:Unable
to load region from system settings. Region must be specified either via environment
variable (AWS_REGION) or system property (aws.region).
DEBUG s.a.a.c.i.h.l.ClasspathSdkHttpServiceProvider:85 - The HTTP implementation loaded
is software.amazon.awssdk.http.apache.ApacheSdkHttpService@a23a01d
DEBUG s.a.a.c.i.ExecutionInterceptorChain:85 - Creating an interceptor
chain that will apply interceptors in the following order:
[software.amazon.awssdk.core.internal.interceptor.HttpChecksumValidationInterceptor@69b2f8e5,
software.amazon.awssdk.awscore.interceptor.HelpfulUnknownHostExceptionInterceptor@6331250e,
software.amazon.awssdk.awscore.eventstream.EventStreamInitialRequestInterceptor@a10c1b5,
software.amazon.awssdk.awscore.interceptor.TraceIdExecutionInterceptor@644abb8f,
software.amazon.awssdk.services.s3.auth.scheme.internal.S3AuthSchemeInterceptor@1a411233,
software.amazon.awssdk.services.s3.endpoints.internal.S3ResolveEndpointInterceptor@70325d20,
software.amazon.awssdk.services.s3.endpoints.internal.S3RequestSetEndpointInterceptor@7c2327fa,
software.amazon.awssdk.services.s3.internal.handlers.StreamingRequestInterceptor@4d847d32,
software.amazon.awssdk.services.s3.internal.handlers.CreateBucketInterceptor@5f462e3b,
software.amazon.awssdk.services.s3.internal.handlers.CreateMultipartUploadRequestInterceptor@3,
software.amazon.awssdk.services.s3.internal.handlers.DecodeUrlEncodedResponseInterceptor@58065,
software.amazon.awssdk.services.s3.internal.handlers.GetBucketPolicyInterceptor@3605c4d3,
software.amazon.awssdk.services.s3.internal.handlers.S3ExpressChecksumInterceptor@585c13de,
software.amazon.awssdk.services.s3.internal.handlers.AsyncChecksumValidationInterceptor@187eb9,
software.amazon.awssdk.services.s3.internal.handlers.SyncChecksumValidationInterceptor@726a6b9,
software.amazon.awssdk.services.s3.internal.handlers.EnableTrailingChecksumInterceptor@6ad11a5,
software.amazon.awssdk.services.s3.internal.handlers.ExceptionTranslationInterceptor@522b2631,
software.amazon.awssdk.services.s3.internal.handlers.GetObjectInterceptor@3ff57625,
software.amazon.awssdk.services.s3.internal.handlers.CopySourceInterceptor@1ee29c84,
software.amazon.awssdk.services.s3.internal.handlers.ObjectMetadataInterceptor@7c8326a4]
DEBUG s.a.a.u.c.CachedSupplier:85 - (Sso0idcTokenProvider()) Cached value is stale and
will be refreshed.
...
DEBUG s.a.a.c.i.ExecutionInterceptorChain:85 - Creating an interceptor
chain that will apply interceptors in the following order:
[software.amazon.awssdk.core.internal.interceptor.HttpChecksumValidationInterceptor@51351f28,
software.amazon.awssdk.awscore.interceptor.HelpfulUnknownHostExceptionInterceptor@21618fa7,
software.amazon.awssdk.awscore.eventstream.EventStreamInitialRequestInterceptor@15f2eda3,
software.amazon.awssdk.awscore.interceptor.TraceIdExecutionInterceptor@34cf294c,
software.amazon.awssdk.services.sso.auth.scheme.internal.SsoAuthSchemeInterceptor@4d7aaca2,
software.amazon.awssdk.services.sso.endpoints.internal.SsoResolveEndpointInterceptor@604b1e1d,
software.amazon.awssdk.services.sso.endpoints.internal.SsoRequestSetEndpointInterceptor@625668,
...
DEBUG s.a.a.request:85 - Sending Request: DefaultSdkHttpFullRequest(httpMethod=GET,
protocol=https, host=portal.sso.us-east-1.amazonaws.com, encodedPath=/federation/

```

```

credentials, headers=[amz-sdk-invocation-id, User-Agent, x-amz-ss0_bearer_token],
  queryParameters=[role_name, account_id])
DEBUG s.a.a.c.i.h.p.s.SigningStage:85 - Using SelectedAuthScheme: smithy.api#noAuth
DEBUG s.a.a.h.a.i.c.SdkTlsSocketFactory:366 - Connecting socket to portal.sso.us-
east-1.amazonaws.com/18.235.195.183:443 with timeout 2000
...
DEBUG s.a.a.requestId:85 - Received successful response: 200, Request ID: bb4f40f4-
e920-4b5c-8648-58f26e7e08cd, Extended Request ID: not available
DEBUG s.a.a.request:85 - Received successful response: 200, Request ID: bb4f40f4-
e920-4b5c-8648-58f26e7e08cd, Extended Request ID: not available
DEBUG s.a.a.u.c.CachedSupplier:85 -
  (software.amazon.awssdk.services.sso.auth.SsoCredentialsProvider@b965857) Successfully
  refreshed cached value. Next Prefetch Time: 2024-04-25T22:03:10.097Z. Next Stale Time:
  2024-04-25T22:05:30Z
DEBUG s.a.a.c.i.ExecutionInterceptorChain:85 - Interceptor
  'software.amazon.awssdk.services.s3.endpoints.internal.S3RequestSetEndpointInterceptor@7c2327f
  modified the message with its modifyHttpRequest method.
...
DEBUG s.a.a.c.i.h.p.s.SigningStage:85 - Using SelectedAuthScheme: aws.auth#sigv4
...
DEBUG s.a.a.a.s.Aws4Signer:85 - AWS4 Canonical Request: GET
...
DEBUG s.a.a.h.a.a.i.s.DefaultV4RequestSigner:85 - AWS4 String to sign: AWS4-HMAC-SHA256
20240425T210631Z
20240425/us-east-1/s3/aws4_request
aafb7784627fa7a49584256cb746279751c48c2076f813259ef767ecce304d64
DEBUG s.a.a.h.a.i.c.SdkTlsSocketFactory:366 - Connecting socket to s3.us-
east-1.amazonaws.com/52.217.41.86:443 with timeout 2000
...

```

The following `log4j2.xml` file configures the previous output.

```

<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%-5p %c{1.}:%L - %m%n" />
    </Console>
  </Appenders>

  <Loggers>
    <Root level="WARN">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
  </Loggers>
</Configuration>

```

```
<Logger name="software.amazon.awssdk" level="DEBUG" />
</Loggers>
</Configuration>
```

Enable wire logging

It can be useful to see the exact requests and responses that the SDK for Java 2.x sends and receives. If you need access to this information, you can temporarily enable it by adding the necessary configuration depending on the HTTP client the service client uses.

By default, synchronous service clients, such as the [S3Client](#), use an underlying Apache HttpClient, and asynchronous service clients, such as the [S3AsyncClient](#), use a Netty non-blocking HTTP client.

Here is a breakdown of HTTP clients you can use for the two categories of service clients:

Synchronous HTTP Clients	Asynchronous HTTP Clients
ApacheHttpClient (default)	NettyNioAsyncHttpClient (default)
URLConnectionHttpClient	AwsCrtAsyncHttpClient
AwsCrtHttpClient	
Apache5HttpClient	

Consult the appropriate tab below for configuration settings you need to add depending on the underlying HTTP client.

Warning

We recommend you only use wire logging for debugging purposes. Disable it in your production environments because it can log sensitive data. It logs the full request or response without encryption, even for an HTTPS call. For large requests (e.g., to upload a file to Amazon S3) or responses, verbose wire logging can also significantly impact your application's performance.

ApacheHttpClient

Add the "org.apache.http.wire" logger to the `log4j2.xml` configuration file and set the level to "DEBUG".

The following `log4j2.xml` file turns on full wire logging for the Apache HttpClient.

```
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{YYYY-MM-dd HH:mm:ss} [%t] %-5p %c:%L - %m%n" />
    </Console>
  </Appenders>

  <Loggers>
    <Root level="WARN">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
    <Logger name="software.amazon.awssdk" level="WARN" />
    <Logger name="software.amazon.awssdk.request" level="DEBUG" />
    <Logger name="org.apache.http.wire" level="DEBUG" />
  </Loggers>
</Configuration>
```

An additional Maven dependency on the `log4j-1.2-api` artifact is required for wire logging with Apache since it uses 1.2 under the hood.

The full set of Maven dependencies for log4j 2, including wire logging for the Apache HTTP client are shown in the following build file snippets.

Maven

```
...
<dependencyManagement>
  ...
  <dependencies>
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-bom</artifactId>
      <version>VERSION</version>
      <type>pom</type>
      <scope>import</scope>
```

```

        </dependency>
    </dependencies>
</dependencyManagement>
...
<!-- The following is needed for Log4j2 with SLF4J -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-slf4j2-impl</artifactId>
</dependency>

<!-- The following is needed for Apache HttpClient wire logging -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-1.2-api</artifactId>
</dependency>
...

```

Gradle–Kotlin DSL

```

...
dependencies {
    ...
    implementation(platform("org.apache.logging.log4j:log4j-bom:VERSION"))
    implementation("org.apache.logging.log4j:log4j-slf4j2-impl")
    implementation("org.apache.logging.log4j:log4j-1.2-api")
}
...

```

Use 2.20.0 for the minimum version of the log4j-bom artifact. For the latest version, use the version published to [Maven central](#). Replace *VERSION* with version you'll use.

Apache5HttpClient

Add the "org.apache.hc.client5.http.wire" logger to the log4j2.xml configuration file and set the level to "DEBUG".

The following log4j2.xml file turns on full wire logging for the Apache5 HttpClient.

```

<Configuration status="WARN">
    <Appenders>
        <Console name="ConsoleAppender" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{YYYY-MM-dd HH:mm:ss} [%t] %-5p %c:%L - %m%n" />
        </Console>
    </Appenders>

```

```

</Appenders>

<Loggers>
  <Root level="WARN">
    <AppenderRef ref="ConsoleAppender"/>
  </Root>
  <Logger name="software.amazon.awssdk" level="WARN" />
  <Logger name="software.amazon.awssdk.request" level="DEBUG" />
  <Logger name="org.apache.http.client5.http.wire" level="DEBUG" />
</Loggers>
</Configuration>

```

URLConnectionHttpClient

To log details for service clients that use the `URLConnectionHttpClient`, first create a `logging.properties` file with the following contents:

```

handlers=java.util.logging.ConsoleHandler
java.util.logging.ConsoleHandler.level=FINEST
sun.net.www.protocol.http.HttpURLConnection.level=ALL

```

Set the following JVM system property with the full path of the `logging.properties`:

```
-Djava.util.logging.config.file=/full/path/to/logging.properties
```

This configuration will log the only the headers of the request and response, for example:

```

<Request> FINE: sun.net.www.MessageHeader@35a9782c11 pairs: {GET /fileuploadtest
HTTP/1.1: null}{amz-sdk-invocation-id: 5f7e707e-4ac5-bef5-ba62-00d71034ffdc}
{amz-sdk-request: attempt=1; max=4}{Authorization: AWS4-HMAC-SHA256
Credential=<deleted>/20220927/us-east-1/s3/aws4_request, SignedHeaders=amz-sdk-
invocation-id;amz-sdk-request;host;x-amz-content-sha256;x-amz-date;x-amz-te,
Signature=e367fa0bc217a6a65675bb743e1280cf12f8e8d566196a816d948fdf0b42ca1a}{User-
Agent: aws-sdk-java/2.17.230 Mac_OS_X/12.5 OpenJDK_64-Bit_Server_VM/25.332-b08
Java/1.8.0_332 vendor/Amazon.com_Inc. io/sync http/URLConnection cfg/retry-mode/
legacy}{x-amz-content-sha256: UNSIGNED-PAYLOAD}{X-Amz-Date: 20220927T133955Z}{x-amz-
te: append-md5}{Host: tkhill-test1.s3.amazonaws.com}{Accept: text/html, image/gif,
image/jpeg, */*; q=.2, */*; q=.2}{Connection: keep-alive}
<Response> FINE: sun.net.www.MessageHeader@70a36a6611 pairs: {null: HTTP/1.1
200 OK}{x-amz-id-2: sAFeZD0KdUMsBbkdyDZw7P0oocb4C9KbiuzfJ6TWKQsGXHM/
dFu0vr2tUb7Y1wEHGdJ3DSIxq0=}{x-amz-request-id: P9QW9SMZ97FKZ9X7}{Date: Tue,
27 Sep 2022 13:39:57 GMT}{Last-Modified: Tue, 13 Sep 2022 14:38:12 GMT}{ETag:

```

```
"2cbe5ad4a064cedec33b452bebf48032"){x-amz-transfer-encoding: append-md5}{Accept-
Ranges: bytes}{Content-Type: text/plain}{Server: AmazonS3}{Content-Length: 67}
```

To see the request/response bodies, add `-Djavax.net.debug=all` to the JVM properties. This additional property logs a great deal of information, including all SSL information.

Within the log console or log file, search for "GET" or "POST" to quickly go to the section of the log containing actual requests and responses. Search for "Plaintext before ENCRYPTION" for requests and "Plaintext after DECRYPTION" for responses to see the full text of the headers and bodies.

NettyNioAsyncHttpClient

If your asynchronous service client uses the default `NettyNioAsyncHttpClient`, add two additional loggers to your `log4j2.xml` file to log HTTP headers and request/response bodies.

```
<Logger name="io.netty.handler.logging" level="DEBUG" />
<Logger name="io.netty.handler.codec.http2.Http2FrameLogger" level="DEBUG" />
```

Here is a complete `log4j2.xml` example:

```
<Configuration status="WARN">
  <Appenders>
    <Console name="ConsoleAppender" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{YYYY-MM-dd HH:mm:ss} [%t] %-5p %c:%L - %m
%n" />
    </Console>
  </Appenders>

  <Loggers>
    <Root level="WARN">
      <AppenderRef ref="ConsoleAppender"/>
    </Root>
    <Logger name="software.amazon.awssdk" level="WARN" />
    <Logger name="software.amazon.awssdk.request" level="DEBUG" />
    <Logger name="io.netty.handler.logging" level="DEBUG" />
    <Logger name="io.netty.handler.codec.http2.Http2FrameLogger" level="DEBUG" /
  >
  </Loggers>
</Configuration>
```

These settings log all header details and request/response bodies.

AwsCrtAsyncHttpClient/AwsCrtHttpClient

If you have configured your service client to use an instance of an AWS CRT-based HTTP client, you can log details by setting JVM system properties or programmatically.

Log to a file at "Debug" level

Using system properties:

```
-Daws.crt.log.level=Trace  
-Daws.crt.log.destination=File  
-Daws.crt.log.filename=<path to file>
```

Programmatically:

```
import software.amazon.awssdk.crt.Log;  
  
// Execute this statement before constructing the  
// SDK service client.  
Log.initLoggingToFile(Log.LogLevel.Trace,  
    "<path to file>");
```

Log to the console at "Debug" level

Using system properties:

```
-Daws.crt.log.level=Trace  
-Daws.crt.log.destination=Stdout
```

Programmatically:

```
import software.amazon.awssdk.crt.Log;  
  
// Execute this statement before constructing the  
// SDK service client.  
Log.initLoggingToStdout(Log.LogLevel.Trace);
```

For security reasons, at the "Trace" level the AWS CRT-based HTTP clients log only response headers. Request headers, request bodies, and response bodies are not logged.

Configuring client endpoints in the AWS SDK for Java 2.x

The SDK for Java 2.x provides multiple ways to configure service endpoints. An endpoint is the URL that the SDK uses to make API calls to AWS services. By default, the SDK automatically determines the appropriate endpoint for each service based on the AWS Region you've configured. However, there are scenarios where you might need to customize or override these endpoints:

- Working with local or third-party service implementations (such as LocalStack)
- Connecting to AWS services through a proxy or VPC endpoint

- Testing against beta or pre-release service endpoints

Endpoint configuration options

The AWS SDK for Java 2.x provides several ways to configure endpoints:

- In-code configuration by using the service client builder
- External configuration with environment variables
- External configuration with JVM system properties
- External configuration with shared AWS config file

In-code endpoint configuration

Using `endpointOverride`

The most direct way to configure an endpoint is by using the `endpointOverride` method on the service client builder. This method accepts a `URI` object representing the custom endpoint URL.

Example Setting a custom endpoint for an Amazon S3 client

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import java.net.URI;

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .endpointOverride(URI.create("https://my-custom-s3-endpoint.example.com"))
    .build();
```

When using `endpointOverride`, you must still specify a region for the client, even though the endpoint is being explicitly set. The region is used for signing requests.

Endpoint discovery

Some AWS services support endpoint discovery, where the SDK can automatically discover the optimal endpoint to use. You can enable or disable this feature using the `endpointDiscoveryEnabled` method on the service client builder.

Example Enabling endpoint discovery for a DynamoDB client

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;

DynamoDbClient dynamoDb = DynamoDbClient.builder()
    .region(Region.US_WEST_2)
    .endpointDiscoveryEnabled(true)
    .build();
```

Request-level endpoint configuration

In some cases, you might need to override the endpoint for a specific request while using the same client for other requests with the default endpoint. The AWS SDK for Java 2.x supports this through request overrides.

Example Overriding the endpoint for a specific request

```
import software.amazon.awssdk.core.SdkRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.http.SdkHttpRequest;

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .build();

// Create a request
GetObjectRequest getObjectRequest = GetObjectRequest.builder()
    .bucket("amzn-s3-demo-bucket")
    .key("my-key")
    .overrideConfiguration(c -> c.putHeader("Host", "custom-endpoint.example.com"))
    .build();

// Execute the request with the custom endpoint
s3.getObject(getObjectRequest);
```

Note that request-level endpoint overrides are limited and may not work for all services or scenarios. For most cases, it's recommended to use client-level endpoint configuration.

External endpoint configuration

Using environment variables

You can configure endpoints using environment variables. The SDK supports service-specific endpoint configuration through environment variables in the format `AWS_ENDPOINT_URL_[SERVICE]`, where `[SERVICE]` is the uppercase service identifier.

Example Setting an S3 endpoint using environment variables

```
# For Linux/macOS
export AWS_ENDPOINT_URL_S3=https://my-custom-s3-endpoint.example.com

# For Windows
set AWS_ENDPOINT_URL_S3=https://my-custom-s3-endpoint.example.com
```

You can also set a global endpoint URL prefix or suffix using the following environment variables:

- `AWS_ENDPOINT_URL` - Sets a global endpoint for all services
- `AWS_ENDPOINT_URL_PREFIX` - Adds a prefix to all service endpoints
- `AWS_ENDPOINT_URL_SUFFIX` - Adds a suffix to all service endpoints

Using JVM system properties

You can also configure endpoints using JVM system properties. The format is similar to environment variables but uses a different naming convention.

Example Setting an S3 endpoint using JVM system properties

```
java -Daws.endpointUrl.s3=https://my-custom-s3-endpoint.example.com -jar your-
application.jar
```

Global endpoint configuration is also available through system properties:

- `aws.endpointUrl` - Sets a global endpoint for all services
- `aws.endpointUrl.prefix` - Adds a prefix to all service endpoints
- `aws.endpointUrl.suffix` - Adds a suffix to all service endpoints

Using the shared AWS config file

The AWS SDK for Java 2.x also supports endpoint configuration through the shared AWS config file, typically located at `~/.aws/config` (Linux/macOS) or `%USERPROFILE%\.aws\config` (Windows). See the [AWS SDKs and Tools Reference Guide](#) for information and examples.

Configuration precedence

When multiple endpoint configurations are present, the SDK follows this order of precedence (from highest to lowest):

1. Request-level overrides (when applicable)
2. Client-level configuration via `endpointOverride`
3. Environment variables
4. JVM system properties
5. Shared AWS config file
6. Default endpoints based on the configured AWS Region

Service-specific endpoint configuration

Some AWS services have additional endpoint configuration options specific to that service. Here are a few examples:

Amazon S3 Endpoint Configuration

Amazon S3 supports several endpoint configurations through the `S3Configuration` class:

- `dualstackEnabled` - Enables IPv6 support
- `accelerateModeEnabled` - Enables S3 Transfer Acceleration
- `pathStyleAccessEnabled` - Uses path-style access instead of virtual-hosted style
- `useArnRegionEnabled` - Uses the region from an ARN for cross-region requests
- `fipsModeEnabled` - Routes requests to FIPS-compliant endpoints

Example Configuring S3-specific endpoint options

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.S3Configuration;

S3Client s3 = S3Client.builder()
    .region(Region.US_WEST_2)
    .serviceConfiguration(S3Configuration.builder()
        .accelerateModeEnabled(true)
        .dualstackEnabled(true)
        .pathStyleAccessEnabled(false)
        .fipsModeEnabled(true)
        .build())
    .build();
```

DynamoDB endpoint configuration

For DynamoDB, you might want to use endpoint discovery or connect to [DynamoDB local](#) for testing:

Example Connecting to DynamoDB local

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import java.net.URI;

DynamoDbClient dynamoDb = DynamoDbClient.builder()
    .endpointOverride(URI.create("http://localhost:8000"))
    // The region is meaningless for DynamoDB local but required for the client
    builder()
    .region(Region.US_WEST_2)
    .build();
```

DynamoDB also supports the use of [account-based endpoints](#), which you can configure in code or using external settings. The following example shows how to disable the use of account-based endpoints in code when you create the client (the default settings is *preferred*):

```
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .region(Region.US_EAST_1)
    .accountIdEndpointMode(AccountIdEndpointMode.DISABLED)
    .build();
```

Best practices

When configuring endpoints in the AWS SDK for Java 2.x, consider these best practices:

- *Use external configuration for environment-specific endpoints*—Use environment variables, system properties, or the AWS config file for endpoints that vary between environments (development, testing, production).
- *Use in-code configuration for application-specific endpoints*— Use the client builder's `endpointOverride` method for endpoints that are specific to your application's design.
- *Always specify a region*—Even when overriding endpoints, always specify a region as it's used for request signing.
- *Be cautious with global endpoint overrides*—Using global endpoint overrides can affect all services, which might not be what you intend.
- *Consider security implications*—When using custom endpoints, ensure they have appropriate security measures, especially for production workloads.

Configure HTTP clients in the AWS SDK for Java 2.x

You can change the HTTP client to use for your service client as well as change the default configuration for HTTP clients with the AWS SDK for Java 2.x. This section discusses HTTP clients and settings for the SDK.

HTTP clients available in the SDK for Java

Synchronous clients

Synchronous HTTP clients in the SDK for Java implement the [SdkHttpClient](#) interface. A synchronous service client, such as the `S3Client` or the `DynamoDbClient`, requires the use of a synchronous HTTP client. The AWS SDK for Java offers three synchronous HTTP clients.

ApacheHttpClient (default)

[ApacheHttpClient](#) is the default HTTP client for synchronous service clients. For information about configuring the `ApacheHttpClient`, see [Configure the Apache-based HTTP client](#).

AwsCrtHttpClient

[AwsCrtHttpClient](#) provides high throughput and non-blocking IO. It is built on the AWS Common Runtime (CRT) Http Client. For information about configuring the `AwsCrtHttpClient` and using it with service clients, see [the section called “Configure AWS CRT-based HTTP clients”](#).

URLConnectionHttpClient

To minimize the number of jars and third-party libraries your application uses, you can use the [URLConnectionHttpClient](#). For information about configuring the `URLConnectionHttpClient`, see [Configure the URLConnection-based HTTP client](#).

Apache5HttpClient

[Apache5HttpClient](#) is an updated version of `ApacheHttpClient` that is built on the *Apache 5.x HttpClient*. *Version 5.x* is the version actively maintained by Apache, and improves on the the previous version used by `ApacheHttpClient` by bringing modern Java ecosystem compatibility including virtual thread support for Java 21, and enhanced logging flexibility through SLF4J. `Apache5HttpClient` will replace `ApacheHttpClient` as the default synchronous client in a future version of the AWS SDK for Java 2.x. It has an identical API and features, making it ideal as a drop-in replacement. For information about configuring the `Apache5HttpClient`, see [Configure the Apache 5.x based HTTP client](#).

Asynchronous clients

Asynchronous HTTP clients in the SDK for Java implement the [SdkAsyncHttpClient](#) interface. An asynchronous service client, such as the `S3AsyncClient` or the `DynamoDbAsyncClient`, requires the use of an asynchronous HTTP client. The AWS SDK for Java offers two asynchronous HTTP clients.

NettyNioAsyncHttpClient (default)

[NettyNioAsyncHttpClient](#) is the default HTTP client used by asynchronous clients. For information about configuring the `NettyNioAsyncHttpClient`, see [the section called “Configure the Netty-based HTTP client”](#).

AwsCrtAsyncHttpClient

The [AwsCrtAsyncHttpClient](#) is based on the AWS Common Runtime (CRT) HTTP Client. For information about configuring the `AwsCrtAsyncHttpClient`, see [the section called “Configure AWS CRT-based HTTP clients”](#).

HTTP client recommendations

Several factors come into play when you choose an HTTP client implementation. Use the following information to help you decide.

HTTP client comparison

The following table provides detailed information for each HTTP client.

HTTP client	Sync or async	When to use	Limitation/ drawback
Apache-based HTTP client <i>(default sync HTTP client)</i>	Sync	Use it if you prefer low latency over high throughput	Slower startup time compared to other HTTP clients
URLConnection-based HTTP client	Sync	Use it if you have a hard requirement for limiting third-party dependencies	• Does not support the HTTP PATCH method, required by some APIs like Amazon APIGateway Update operations • Does not support a configurable

HTTP client	Sync or async	When to use	Limitation/ drawback
			idle connection timeout. See Configure the URLConnection-based HTTP client for details.
AWS CRT-based sync HTTP client ¹	Sync	• Use it if your application is running in AWS Lambda • Use it if you prefer high throughput over low latency • Use it if you prefer sync SDK clients	The following Java system properties are not supported: • <code>javax.net.ssl.keyStore</code> • <code>javax.net.ssl.keyStorePassword</code> • <code>javax.net.ssl.trustStore</code> • <code>javax.net.ssl.trustStorePassword</code>
Netty-based HTTP client <i>(default async HTTP client)</i>	Async	• Use it if your application relies on Java system properties for TLS configuration (such as <code>javax.net.ssl.keyStore</code>)	Slower startup time compared to other HTTP clients

HTTP client	Sync or async	When to use	Limitation/ drawback
AWS CRT-based async HTTP client ¹	Async	• Use it if your application is running in AWS Lambda • Use it if you prefer high throughput over low latency • Use it if you prefer async SDK clients	The following Java system properties are not supported: • javax.net.ssl.keyStore • javax.net.ssl.keyStorePassword • javax.net.ssl.trustStore • javax.net.ssl.trustStorePassword

¹Because of their added benefits, we recommend that you use the AWS CRT-based HTTP clients if possible.

Smart configuration defaults

The AWS SDK for Java 2.x (version 2.17.102 or later) offers a smart configuration defaults feature. This feature optimizes two HTTP client properties along with other properties that don't affect the HTTP client.

The smart configuration defaults set sensible values for the `connectTimeoutInMillis` and `tlsNegotiationTimeoutInMillis` properties based on a defaults mode value that you provide. You choose the defaults mode value based on your application's characteristics.

For more information about smart configuration defaults and how to choose the defaults mode value that is best suited for your applications, see the [AWS SDKs and Tools Reference Guide](#).

Following are four ways to set the defaults mode for your application.

Service client

Use the service client builder to configure the defaults mode directly on the service client. The following example sets the defaults mode to auto for the `DynamoDbClient`.

```
DynamoDbClient ddbClient = DynamoDbClient.builder()
    .defaultsMode(DefaultsMode.AUTO)
    .build();
```

System property

You can use the `aws.defaultsMode` system property to specify the defaults mode. If you set the system property in Java, you need to set the property before initializing any service client.

The following example shows you how to set the defaults mode to auto using a system property set in Java.

```
System.setProperty("aws.defaultsMode", "auto");
```

The following example demonstrates how you set the defaults mode to auto using a `-D` option of the `java` command.

```
java -Daws.defaultsMode=auto
```

Environment variable

Set a value for environment variable `AWS_DEFAULTS_MODE` to select the defaults mode for your application.

The following information shows the command to run to set the value for the defaults mode to auto using an environment variable.

Operating system	Command to set environment variables
Linux, macOS, or Unix	<code>export AWS_DEFAULTS_MODE=auto</code>
Windows	<code>set AWS_DEFAULTS_MODE=auto</code>

AWS config file

You can add a `defaults_mode` configuration property to the shared AWS config file as the following example shows.

```
[default]
defaults_mode = auto
```

If you set the defaults mode globally with the system property, environment variable, or AWS config file, you can override the settings when you build an HTTP client.

When you build an HTTP client with the `httpClientBuilder()` method, settings apply only to the instance that you are building. An example of this is shown [here](#). The Netty-based HTTP client in this example overrides any default mode values set globally for `connectTimeoutInMillis` and `tlsNegotiationTimeoutInMillis`.

Configure the Apache-based HTTP client

Synchronous service clients in the AWS SDK for Java 2.x use an Apache-based HTTP client, [ApacheHttpClient](#) by default. The SDK's `ApacheHttpClient` is based on the Apache [HttpClient](#).

The SDK also offers the [URLConnectionHttpClient](#), which loads more quickly, but has fewer features. For information about configuring the `URLConnectionHttpClient`, see [the section called "Configure the URLConnection-based HTTP client"](#).

To see the full set of configuration options available to you for the `ApacheHttpClient`, see [ApacheHttpClient.Builder](#) and [ProxyConfiguration.Builder](#).

Access the ApacheHttpClient

In most situations, you use the `ApacheHttpClient` without any explicit configuration. You declare your service clients and the SDK will configure the `ApacheHttpClient` with standard values for you.

If you want to explicitly configure the `ApacheHttpClient` or use it with multiple service clients, you need to make it available for configuration.

No configuration needed

When you declare a dependency on a service client in Maven, the SDK adds a *runtime* dependency on the `apache-client` artifact. This makes the `ApacheHttpClient` class available to your code

at runtime, but not at compile time. If you are not configuring the Apache-based HTTP client, you do not need to specify a dependency for it.

In the following XML snippet of a Maven `pom.xml` file, the dependency declared with `<artifactId>s3</artifactId>` automatically brings in the Apache-based HTTP client. You don't need to declare a dependency specifically for it.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
      <version>2.27.21</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
<dependencies>
  <!-- The s3 dependency automatically adds a runtime dependency on the
  ApacheHttpClient-->
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
  </dependency>
</dependencies>
```

With these dependencies, you cannot make any explicit HTTP configuration changes, because the `ApacheHttpClient` library is only on the runtime classpath.

Configuration needed

To configure the `ApacheHttpClient`, you need to add a dependency on the `apache-client` library at *compile* time.

Refer to the following example of a Maven `pom.xml` file to configure the `ApacheHttpClient`.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
```

```
<version>2.27.21</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
<dependencies>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
  </dependency>
  <!-- By adding the apache-client dependency, ApacheHttpClient will be added to
       the compile classpath so you can configure it. -->
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>apache-client</artifactId>
  </dependency>
</dependencies>
```

Use and configure the `ApacheHttpClient`

You can configure an instance of `ApacheHttpClient` along with building a service client, or you can configure a single instance to share across multiple service clients.

With either approach, you use the [ApacheHttpClient.Builder](#) to configure the properties for the Apache-based HTTP client.

Best practice: dedicate an `ApacheHttpClient` instance to a service client

If you need to configure an instance of the `ApacheHttpClient`, we recommend that you build the dedicated `ApacheHttpClient` instance. You can do so by using the `httpClientBuilder` method of the service client's builder. This way, the lifecycle of the HTTP client is managed by the SDK, which helps avoid potential memory leaks if the `ApacheHttpClient` instance is not closed down when it's no longer needed.

The following example creates an `S3Client` and configures the embedded instance of `ApacheHttpClient` with `maxConnections` and `connectionTimeout` values. The HTTP instance is created using the `httpClientBuilder` method of `S3Client.Builder`.

Imports

```
import software.amazon.awssdk.http.apache.ApacheHttpClient;
```

```
import software.amazon.awssdk.services.s3.S3Client;
import java.time.Duration;
```

Code

```
S3Client s3Client = S3Client // Singleton: Use the s3Client for all requests.
    .builder()
    .httpClientBuilder(ApacheHttpClient.builder()
        .maxConnections(100)
        .connectionTimeout(Duration.ofSeconds(5))
    ).build();

// Perform work with the s3Client.

s3Client.close(); // Requests completed: Close all service clients.
```

Alternative approach: share an `ApacheHttpClient` instance

To help keep resource and memory usage lower for your application, you can configure an `ApacheHttpClient` and share it across multiple service clients. The HTTP connection pool will be shared, which lowers resource usage.

Note

When an `ApacheHttpClient` instance is shared, you must close it when it is ready to be disposed. The SDK will not close the instance when the service client is closed.

The following example configures an Apache-based HTTP client that is used by two service clients. The configured `ApacheHttpClient` instance is passed to the `httpClient` method of each builder. When the service clients and the HTTP client are no longer needed, the code explicitly closes them. The code closes the HTTP client last.

Imports

```
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.s3.S3Client;
```

Code


```
SdkHttpClient apacheHttpClient = ApacheHttpClient.builder()
    .maxConnections(100).build();

// Singletons: Use the s3Client and dynamoDbClient for all requests.
S3Client s3Client =
    S3Client.builder()
        .httpClient(apacheHttpClient).build();

DynamoDbClient dynamoDbClient =
    DynamoDbClient.builder()
        .httpClient(apacheHttpClient).build();

// Perform work with the s3Client and dynamoDbClient.

// Requests completed: Close all service clients.
s3Client.close();
dynamoDbClient.close();
apacheHttpClient.close(); // Explicitly close apacheHttpClient.
```

Proxy configuration example

The following code snippet uses the [proxy configuration builder for the Apache HTTP client](#).

```
SdkHttpClient apacheHttpClient = ApacheHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .endpoint(URI.create("http://example.com:1234"))
        .username("username")
        .password("password")
        .addNonProxyHost("localhost")
        .addNonProxyHost("host.example.com")
        .build())
    .build();
```

The equivalent Java system properties for the proxy configuration are shown in the following command line snippet.

```
$ java -Dhttp.proxyHost=example.com -Dhttp.proxyPort=1234 -Dhttp.proxyUser=username \
-Dhttp.proxyPassword=password -Dhttp.nonProxyHosts=localhost|host.example.com -cp ...
App
```

The equivalent setup that uses environment variables is:

```
// Set the following environment variables.
// $ export HTTP_PROXY="http://username:password@example.com:1234"
// $ export NO_PROXY="localhost|host.example.com"

// Set the 'useSystemPropertyValues' to false on the proxy configuration.
SdkHttpClient apacheHttpClient = ApacheHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .useSystemPropertyValues(Boolean.FALSE)
        .build())
    .build();

// Run the application.
// $ java -cp ... App
```

Note

The Apache HTTP client does not currently support HTTPS proxy system properties or the `HTTPS_PROXY` environment variable.

Configure the `URLConnection`-based HTTP client

The AWS SDK for Java 2.x offers a lighter-weight [URLConnectionHttpClient](#) HTTP client in comparison to the default `ApacheHttpClient`. The `URLConnectionHttpClient` is based on Java's [URLConnection](#).

The `URLConnectionHttpClient` loads more quickly than the Apache-based HTTP client, but has fewer features. Because it loads more quickly, it is a [good solution](#) for Java AWS Lambda functions.

The `URLConnectionHttpClient` has several [configurable options](#) that you can access.

Note

The `URLConnectionHttpClient` does not support the HTTP PATCH method. A handful of AWS API operations require PATCH requests. Those operation names usually start with `Update*`. The following are several examples.

- [Several Update* operations](#) in the AWS Security Hub CSPM API and also the [BatchUpdateFindings](#) operation
- All Amazon API Gateway API [Update* operations](#)

If you might use the `URLConnectionHttpClient`, first refer to the API Reference for the AWS service that you're using. Check to see if the operations you need use the PATCH operation.

Note

The `URLConnectionHttpClient` does not support a configurable idle connection timeout.

Connection pooling is managed internally by the JVM. To configure the idle timeout, set the JVM system property `http.keepAlive.time.server` to the desired value in seconds:

```
$ java -Dhttp.keepAlive.time.server=60 -cp ... App
```

This is a JVM-global setting that affects all `HttpURLConnection` usage in the process, not just SDK clients. If you need per-client control over idle connection timeout, use the `ApacheHttpClient` with the `connectionMaxIdleTime` setting.

Access the `URLConnectionHttpClient`

To configure and use the `URLConnectionHttpClient`, you declare a dependency on the `url-connection-client` Maven artifact in your `pom.xml` file.

Unlike the `ApacheHttpClient`, the `URLConnectionHttpClient` is not automatically added to your project, so you must specifically declare it.

The following example of a `pom.xml` file shows the dependencies required to use and configure the HTTP client.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
      <version>2.27.21</version>
      <type>pom</type>
```

```
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>

<!-- other dependencies such as s3 or dynamodb -->

<dependencies>
    <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>url-connection-client</artifactId>
    </dependency>
</dependencies>
```

Use and configure the `URLConnectionHttpClient`

You can configure an instance of `URLConnectionHttpClient` along with building a service client, or you can configure a single instance to share across multiple service clients.

With either approach, you use the [URLConnectionHttpClient.Builder](#) to configure the properties for the `URLConnection`-based HTTP client.

Best practice: dedicate an `URLConnectionHttpClient` instance to a service client

If you need to configure an instance of the `URLConnectionHttpClient`, we recommend that you build the dedicated `URLConnectionHttpClient` instance. You can do so by using the `httpClientBuilder` method of the service client's builder. This way, the lifecycle of the HTTP client is managed by the SDK, which helps avoid potential memory leaks if the `URLConnectionHttpClient` instance is not closed down when it's no longer needed.

The following example creates an `S3Client` and configures the embedded instance of `URLConnectionHttpClient` with `socketTimeout` and `proxyConfiguration` values. The `proxyConfiguration` method takes a Java lambda expression of type `Consumer` <[ProxyConfiguration.Builder](#)>.

Imports

```
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.urlconnection.URLConnectionHttpClient;
import java.net.URI;
import java.time.Duration;
```

Code

```
// Singleton: Use the s3Client for all requests.
S3Client s3Client =
    S3Client.builder()
        .httpClientBuilder(URLConnectionHttpClient.builder()
            .socketTimeout(Duration.ofMinutes(5))
            .proxyConfiguration(proxy -> proxy.endpoint(URI.create("http://
proxy.mydomain.net:8888"))))
        .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
        .build();

// Perform work with the s3Client.

s3Client.close(); // Requests completed: Close the s3client.
```

Alternative approach: share an `URLConnectionHttpClient` instance

To help keep resource and memory usage lower for your application, you can configure an `URLConnectionHttpClient` and share it across multiple service clients. The HTTP connection pool will be shared, which lowers resource usage.

Note

When an `URLConnectionHttpClient` instance is shared, you must close it when it is ready to be disposed. The SDK will not close the instance when the service client is closed.

The following example configures an `URLConnection`-based HTTP client that is used by two service clients. The configured `URLConnectionHttpClient` instance is passed to the `httpClient` method of each builder. When the service clients and the HTTP client are no longer needed, the code explicitly closes them. The code closes the HTTP client last.

Imports

```
import software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.awscore.defaultsmode.DefaultsMode;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.urlconnection.ProxyConfiguration;
import software.amazon.awssdk.http.urlconnection.UrlConnectionHttpClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
```

```
import software.amazon.awssdk.services.s3.S3Client;
import java.net.URI;
import java.time.Duration;
```

Code

```
SdkHttpClient urlHttpClient = UrlConnectionHttpClient.create();

// Singletons: Use the s3Client and dynamoDbClient for all requests.
S3Client s3Client =
    S3Client.builder()
        .httpClient(urlHttpClient)
        .defaultsMode(DefaultsMode.IN_REGION)
        .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
        .build();

DynamoDbClient dynamoDbClient =
    DynamoDbClient.builder()
        .httpClient(urlHttpClient)
        .defaultsMode(DefaultsMode.IN_REGION)
        .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
        .build();

// Perform work with the s3Client and dynamoDbClient.

// Requests completed: Close all service clients.
s3Client.close();
dynamoDbClient.close();
urlHttpClient.close();
```

Use `UrlConnectionHttpClient` and `ApacheHttpClient` together

When you use the `UrlConnectionHttpClient` in your application, you must supply each service client with either a `UrlConnectionHttpClient` instance or a `ApacheHttpClient` instance using the service client builder's `httpClientBuilder` method.

An exception occurs if your program uses multiple service clients and both of the following are true:

- One service client is configured to use a `UrlConnectionHttpClient` instance
- Another service client uses the default `ApacheHttpClient` without explicitly building it with the `httpClient()` or `httpClientBuilder()` methods

The exception will state that multiple HTTP implementations were found on the classpath.

The following example code snippet leads to an exception.

```
// The dynamoDbClient uses the UrlConnectionHttpClient
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .httpClient(UrlConnectionHttpClient.create())
    .build();

// The s3Client below uses the ApacheHttpClient at runtime, without specifying it.
// An SdkClientException is thrown with the message that multiple HTTP implementations
// were found on the classpath.
S3Client s3Client = S3Client.create();

// Perform work with the s3Client and dynamoDbClient.

dynamoDbClient.close();
s3Client.close();
```

Avoid the exception by explicitly configuring the S3Client with an ApacheHttpClient.

```
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .httpClient(UrlConnectionHttpClient.create())
    .build();

S3Client s3Client = S3Client.builder()
    .httpClient(ApacheHttpClient.create())    // Explicitly build the
    ApacheHttpClient.
    .build();

// Perform work with the s3Client and dynamoDbClient.

dynamoDbClient.close();
s3Client.close();
```

Note

To explicitly create the ApacheHttpClient, you must [add a dependency](#) on the apache-client artifact in your Maven project file.

Proxy configuration example

The following code snippet uses the [proxy configuration builder for the URL connection HTTP client](#).

```
SdkHttpClient urlHttpClient = UrlConnectionHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .endpoint(URI.create("http://example.com:1234"))
        .username("username")
        .password("password")
        .addNonProxyHost("localhost")
        .addNonProxyHost("host.example.com")
        .build())
    .build();
```

The equivalent Java system properties for the proxy configuration are shown in the following command line snippet.

```
$ java -Dhttp.proxyHost=example.com -Dhttp.proxyPort=1234 -Dhttp.proxyUser=username \
-Dhttp.proxyPassword=password -Dhttp.nonProxyHosts=localhost|host.example.com -cp ...
App
```

The equivalent setup that uses environment variables is:

```
// Set the following environment variables.
// $ export HTTP_PROXY="http://username:password@example.com:1234"
// $ export NO_PROXY="localhost|host.example.com"

// Set the 'useSystemPropertyValues' to false on the proxy configuration.
SdkHttpClient apacheHttpClient = UrlConnectionHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .useSystemPropertyValues(Boolean.FALSE)
        .build())
    .build();

// Run the application.
// $ java -cp ... App
```


Note

The `URLConnection`-based HTTP client does not currently support HTTPS proxy system properties or the `HTTPS_PROXY` environment variable.

Configure the Netty-based HTTP client

The default HTTP client for asynchronous operations in the AWS SDK for Java 2.x is the Netty-based [NettyNioAsyncHttpClient](#). The Netty-based client is based on the asynchronous event-driven network framework of the [Netty project](#).

As an alternative HTTP client, you can use the new [AWS CRT-based HTTP client](#). This topic shows you how to configure the `NettyNioAsyncHttpClient`.

Access the `NettyNioAsyncHttpClient`

In most situations, you use the `NettyNioAsyncHttpClient` without any explicit configuration in asynchronous programs. You declare your asynchronous service clients and the SDK will configure the `NettyNioAsyncHttpClient` with standard values for you.

If you want to explicitly configure the `NettyNioAsyncHttpClient` or use it with multiple service clients, you need to make it available for configuration.

No configuration needed

When you declare a dependency on a service client in Maven, the SDK adds a *runtime* dependency on the `netty-nio-client` artifact. This makes the `NettyNioAsyncHttpClient` class available to your code at runtime, but not at compile time. If you are not configuring the Netty-based HTTP client, you don't need to specify a dependency for it.

In the following XML snippet of a Maven `pom.xml` file, the dependency declared with `<artifactId>dynamodb-enhanced</artifactId>` transitively brings in the Netty-based HTTP client. You don't need to declare a dependency specifically for it.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
```

```

        <version>2.27.21</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>
<dependencies>
    <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>dynamodb-enhanced</artifactId>
    </dependency>
</dependencies>

```

With these dependencies, you cannot make any HTTP configuration changes, since the `NettyNioAsyncHttpClient` library is only on the runtime classpath.

Configuration needed

To configure the `NettyNioAsyncHttpClient`, you need to add a dependency on the `netty-nio-client` artifact at *compile* time.

Refer to the following example of a Maven `pom.xml` file to configure the `NettyNioAsyncHttpClient`.

```

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>software.amazon.awssdk</groupId>
            <artifactId>bom</artifactId>
            <version>2.27.21</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
<dependencies>
    <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>dynamodb-enhanced</artifactId>
    </dependency>
    <!-- By adding the netty-nio-client dependency, NettyNioAsyncHttpClient will
be
        added to the compile classpath so you can configure it. -->

```

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>netty-nio-client</artifactId>
</dependency>
</dependencies>
```

Use and configure the NettyNioAsyncHttpClient

You can configure an instance of `NettyNioAsyncHttpClient` along with building a service client, or you can configure a single instance to share across multiple service clients.

With either approach, you use the [NettyNioAsyncHttpClient.Builder](#) to configure the properties for the Netty-based HTTP client instance.

Best practice: dedicate a NettyNioAsyncHttpClient instance to a service client

If you need to configure an instance of the `NettyNioAsyncHttpClient`, we recommend that you build a dedicated `NettyNioAsyncHttpClient` instance. You can do so by using the `httpClientBuilder` method of the service client's builder. This way, the lifecycle of the HTTP client is managed by the SDK, which helps avoid potential memory leaks if the `NettyNioAsyncHttpClient` instance is not closed down when it's no longer needed.

The following example creates a `DynamoDbAsyncClient` instance that is used by a `DynamoDbEnhancedAsyncClient` instance. The `DynamoDbAsyncClient` instance contains the `NettyNioAsyncHttpClient` instance with `connectionTimeout` and `maxConcurrency` values. The HTTP instance is created using `httpClientBuilder` method of `DynamoDbAsyncClient.Builder`.

Imports

```
import software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.awscore.defaultsmode.DefaultsMode;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedAsyncClient;
import
  software.amazon.awssdk.enhanced.dynamodb.extensions.AutoGeneratedTimestampRecordExtension;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import java.time.Duration;
```

Code

```
// DynamoDbAsyncClient is the lower-level client used by the enhanced client.
DynamoDbAsyncClient dynamoDbAsyncClient =
    DynamoDbAsyncClient
        .builder()
            .httpClientBuilder(NettyNioAsyncHttpClient.builder()
                .connectionTimeout(Duration.ofMillis(5_000))
                .maxConcurrency(100)
                .tlsNegotiationTimeout(Duration.ofMillis(3_500)))
            .defaultsMode(DefaultsMode.IN_REGION)
            .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
        .build();

// Singleton: Use dynamoDbAsyncClient and enhancedClient for all requests.
DynamoDbEnhancedAsyncClient enhancedClient =
    DynamoDbEnhancedAsyncClient
        .builder()
            .dynamoDbClient(dynamoDbAsyncClient)
            .extensions(AutoGeneratedTimestampRecordExtension.create())
        .build();

// Perform work with the dynamoDbAsyncClient and enhancedClient.

// Requests completed: Close dynamoDbAsyncClient.
dynamoDbAsyncClient.close();
```

Alternative approach: share a NettyNioAsyncHttpClient instance

To help keep resource and memory usage lower for your application, you can configure a `NettyNioAsyncHttpClient` and share it across multiple service clients. The HTTP connection pool will be shared, which lowers resource usage.

Note

When a `NettyNioAsyncHttpClient` instance is shared, you must close it when it is ready to be disposed. The SDK will not close the instance when the service client is closed.

The following example configures a Netty-based HTTP client that is used by two service clients. The configured `NettyNioAsyncHttpClient` instance is passed to the `httpClient` method of each builder. When the service clients and the HTTP client are no longer needed, the code explicitly closes them. The code closes the HTTP client last.

Imports

```
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.s3.S3Client;
```

Code

```
// Create a NettyNioAsyncHttpClient shared instance.
SdkAsyncHttpClient nettyHttpClient =
    NettyNioAsyncHttpClient.builder().maxConcurrency(100).build();

// Singletons: Use the s3AsyncClient, dbAsyncClient, and enhancedAsyncClient for all
// requests.
S3AsyncClient s3AsyncClient =
    S3AsyncClient.builder()
        .httpClient(nettyHttpClient)
        .build();

DynamoDbAsyncClient dbAsyncClient =
    DynamoDbAsyncClient.builder()
        .httpClient(nettyHttpClient)
        .defaultsMode(DefaultsMode.IN_REGION)

        .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
        .build();

DynamoDbEnhancedAsyncClient enhancedAsyncClient =
    DynamoDbEnhancedAsyncClient.builder()
        .dynamoDbClient(dbAsyncClient)

        .extensions(AutoGeneratedTimestampRecordExtension.create())
        .build();

// Perform work with s3AsyncClient, dbAsyncClient, and enhancedAsyncClient.

// Requests completed: Close all service clients.
s3AsyncClient.close();
dbAsyncClient.close();
nettyHttpClient.close(); // Explicitly close nettyHttpClient.
```

Configure ALPN protocol negotiation

ALPN (Application-Layer Protocol Negotiation) is a TLS extension that allows the application layer to negotiate which protocol should be performed over a secure connection in a manner that avoids additional round trips and provides better performance.

To enable the Netty-based HTTP client to use ALPN, call the builder methods as shown in the following snippet:

```
import software.amazon.awssdk.http.Protocol;
import software.amazon.awssdk.http.ProtocolNegotiation;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import
    software.amazon.awssdk.services.transcribestreaming.TranscribeStreamingAsyncClient;

// Configure the Netty-based HTTP client to use the ALPN protocol.
SdkAsyncHttpClient nettyClient = NettyNioAsyncHttpClient.builder()
    .protocol(Protocol.HTTP2)

    .protocolNegotiation(ProtocolNegotiation.ALPN)
    .build();

// Use the Netty-based HTTP client with a service client.
TranscribeStreamingAsyncClient transcribeClient =
    TranscribeStreamingAsyncClient.builder()

    .httpClient(nettyClient)

    .build();
```

ALPN protocol negotiation currently works with only with the HTTP/2 protocol as shown in the previous snippet.

Proxy configuration example

The following code snippet uses the [proxy configuration builder for the Netty HTTP client](#).

```
SdkAsyncHttpClient nettyHttpClient = NettyNioAsyncHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .scheme("https")
```

```

        .host("myproxy")
        .port(1234)
        .username("username")
        .password("password")
        .nonProxyHosts(Set.of("localhost", "host.example.com"))
        .build())
    .build();

```

The equivalent Java system properties for the proxy configuration are shown in the following command line snippet.

```

$ java -Dhttps.proxyHost=myproxy -Dhttps.proxyPort=1234 -Dhttps.proxyUser=username \
-Dhttps.proxyPassword=password -Dhttp.nonProxyHosts=localhost|host.example.com -cp ...
App

```

Important

To use any of the HTTPS proxy system properties, the scheme property must be set in code to `https`. If the scheme property is not set in code, the scheme defaults to `HTTP` and the SDK looks only for `http.*` system properties.

The equivalent setup that uses environment variables is:

```

// Set the following environment variables.
// $ export HTTPS_PROXY="https://username:password@myproxy:1234"
// $ export NO_PROXY="localhost|host.example.com"

// Set the 'useSystemPropertyValues' to false on the proxy configuration.
SdkAsyncHttpClient nettyHttpClient = NettyNioAsyncHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .useSystemPropertyValues(Boolean.FALSE)
        .build())
    .build();

// Run the application.
// $ java -cp ... App

```

Configure AWS CRT-based HTTP clients

The AWS CRT-based HTTP clients include the synchronous [AwsCrtHttpClient](#) and asynchronous [AwsCrtAsyncHttpClient](#). The AWS CRT-based HTTP clients provide the following HTTP client benefits:

- Faster SDK startup time
- Smaller memory footprint
- Reduced latency time
- Connection health management
- DNS load balancing

AWS CRT-based components in the SDK

The AWS CRT-based *HTTP* clients, described in this topic, and the AWS CRT-based *S3* client are different components in the SDK.

The synchronous and asynchronous **AWS CRT-based HTTP clients** are implementations SDK HTTP client interfaces and are used for general HTTP communication. They are alternatives to the other synchronous or asynchronous HTTP clients in the SDK with additional benefits.

The **AWS CRT-based S3 client** is an implementation of the [S3AsyncClient](#) interface and is used for working with the Amazon S3 service. It is an alternative to the Java-based implementation of the [S3AsyncClient](#) interface and offers several advantages.

Although both components use libraries from the [AWS Common Runtime](#), the AWS CRT-based HTTP clients do not use the [aws-c-s3 library](#) and do not support the [S3 multipart upload API](#) features. The AWS CRT-based S3 client, by contrast, was purpose-built to support the S3 multipart upload API features.

Access the AWS CRT-based HTTP clients

Before you can use the AWS CRT-based HTTP clients, add the `aws-crt-client` artifact with a minimum version of 2.22.0 to your project's dependencies.

Use one of the following options to set up your Maven `pom.xml` file.

Note

You might choose to use the *Platform-specific jar option* if you need to keep the size of the runtime dependencies smaller, for example if your application runs in an AWS Lambda function.

Uber-jar option

By default, the `aws-crt-client` uses an uber-jar of AWS CRT artifacts that contains binaries for several platforms, including Linux, Windows, and macOS.

```
<project>
  <properties>
    <aws.sdk.java.version>2.29.10*</aws.sdk.java.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.sdk.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>aws-crt-client</artifactId>
    </dependency>
  </dependencies>
</project>
```

*Replace the version shown in red with the version of the Java SDK that you want to use. Find the latest on [Maven Central](#).

Platform-specific jar option

To restrict the Java runtime to platform-specific version of the AWS CRT library, make the following changes to the *Uber-jar option*.

- Add an exclusions element to the SDK's `aws-crt-client` artifact. This exclusion prevents the SDK from transitively using the AWS CRT uber-jar.
- Add a dependency element for the specific AWS CRT platform version you need. See the **Steps to determine the AWS CRT artifact version** below for how you can determine the correct version.

```
<project>
  <properties>
    <aws.sdk.java.version>2.29.101</aws.sdk.java.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.sdk.java.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>aws-crt-client</artifactId>
      <exclusions>
        <exclusion>
          <groupId>software.amazon.awssdk.crt</groupId>
          <artifactId>aws-crt</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
    <dependency>
      <groupId>software.amazon.awssdk.crt</groupId>
      <artifactId>aws-crt</artifactId>
      <version>0.31.32</version>
      <classifier>linux-x86_643</classifier>
    </dependency>
  </dependencies>
```

¹Replace the version shown in red with the version of the Java SDK that you want to use. Find the latest on [Maven Central](#).

²Replace the version of `software.amazon.awssdk.crt:aws-crt` that would be provided by the *Uber-jar option*. See the following **Steps to determine the AWS CRT artifact version**.

³Replace the `classifier` value with one for your platform. Refer to the AWS CRT for Java GitHub page for a [listing of available values](#).

Steps to determine the AWS CRT artifact version

Use the following steps to determine the AWS CRT artifact version that is compatible with the version of the SDK for Java that you are using.

1. Set up your `pom.xml` file as shown in the *Uber-jar option*. This setup allows you to see what version of `software.amazon.awssdk.crt:aws-crt` the SDK brings in by default.
2. At the root of the project (in the same directory as the `pom.xml` file), run the following Maven command:

```
mvn dependency:tree -Dincludes=software.amazon.awssdk.crt:aws-crt
```

Maven might perform other actions, but at the end you should see console output of the `software.amazon.awssdk.crt:aws-crt` dependency that the SDK transitively uses. The following snippet shows sample output based on an SDK version of 2.29.10:

```
[INFO] org.example:yourProject:jar:1.0-SNAPSHOT
[INFO] \- software.amazon.awssdk:aws-crt-client:jar:2.29.10:compile
[INFO]    \- software.amazon.awssdk.crt:aws-crt:jar:0.31.3:compile
```

3. Use the version that the console shows for the `software.amazon.awssdk.crt:aws-crt` artifact. In this case, add 0.31.3 to your `pom.xml` file.

Use and configure an AWS CRT-based HTTP client

You can configure an AWS CRT-based HTTP client along with building a service client, or you can configure a single instance to share across multiple service clients.

With either approach, you use a builder to [configure the properties](#) for the AWS CRT-based HTTP client instance.

Best practice: dedicate an instance to a service client

If you need to configure an instance of an AWS CRT-based HTTP client, we recommend that you dedicate the instance by building it along with the service client. You can do so by using the `httpClientBuilder` method of the service client's builder. This way, the lifecycle of the HTTP client is managed by the SDK, which helps avoid potential memory leaks if the AWS CRT-based HTTP client instance is not closed down when it's no longer needed.

The following example creates an S3 service client and configures an AWS CRT-based HTTP client with `connectionTimeout` and `maxConcurrency` values.

Synchronous client

Imports

```
import software.amazon.awssdk.http.crt.AwsCrtHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import java.time.Duration;
```

Code

```
// Singleton: Use s3Client for all requests.
S3Client s3Client = S3Client.builder()
    .httpClientBuilder(AwsCrtHttpClient
        .builder()
        .connectionTimeout(Duration.ofSeconds(3))
        .maxConcurrency(100))
    .build();

// Perform work with the s3Client.

// Requests completed: Close the s3Client.
s3Client.close();
```

Asynchronous client

Imports

```
import software.amazon.awssdk.http.crt.AwsCrtAsyncHttpClient;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import java.time.Duration;
```

Code

```
// Singleton: Use s3AsyncClient for all requests.
S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
    .httpClientBuilder(AwsCrtAsyncHttpClient
        .builder()
        .connectionTimeout(Duration.ofSeconds(3))
        .maxConcurrency(100))
    .build();

// Perform work with the s3AsyncClient.

// Requests completed: Close the s3AsyncClient.
s3AsyncClient.close();
```

Alternative approach: share an instance

To help keep resource and memory usage lower for your application, you can configure an AWS CRT-based HTTP client and share it across multiple service clients. The HTTP connection pool will be shared, which lowers resource usage.

Note

When an AWS CRT-based HTTP client instance is shared, you must close it when it is ready to be disposed. The SDK will not close the instance when the service client is closed.

The following example configures an AWS CRT-based HTTP client instance with `connectionTimeout` and `maxConcurrency` values. The configured instance is passed to the `httpClient` method of each service client's builder. When the service clients and the HTTP client are no longer needed, they are explicitly closed. The HTTP client is closed last.

Synchronous client

Imports

```
import
    software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.awscore.defaultsmode.DefaultsMode;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.crt.AwsCrtHttpClient;
```

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.s3.S3Client;
import java.time.Duration;
```

Code

```
// Create an AwsCrtHttpClient shared instance.
SdkHttpClient crtHttpClient = AwsCrtHttpClient.builder()
    .connectionTimeout(Duration.ofSeconds(3))
    .maxConcurrency(100)
    .build();

// Singletons: Use the s3Client and dynamoDbClient for all requests.
S3Client s3Client = S3Client.builder()
    .httpClient(crtHttpClient)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.crea
    .defaultsMode(DefaultsMode.IN_REGION)
    .region(Region.US_EAST_1)
    .build();

DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .httpClient(crtHttpClient)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.crea
    .defaultsMode(DefaultsMode.IN_REGION)
    .region(Region.US_EAST_1)
    .build();

// Requests completed: Close all service clients.
s3Client.close();
dynamoDbClient.close();
crtHttpClient.close(); // Explicitly close crtHttpClient.
```

Asynchronous client

Imports

```
import
    software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.awscore.defaultsmode.DefaultsMode;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.crt.AwsCrtAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
```

```
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import java.time.Duration;
```

Code

```
// Create an AwsCrtAsyncHttpClient shared instance.
SdkAsyncHttpClient crtAsyncHttpClient = AwsCrtAsyncHttpClient.builder()
    .connectionTimeout(Duration.ofSeconds(3))
    .maxConcurrency(100)
    .build();

// Singletons: Use the s3AsyncClient and dynamoDbAsyncClient for all requests.
S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
    .httpClient(crtAsyncHttpClient)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
    .defaultsMode(DefaultsMode.IN_REGION)
    .region(Region.US_EAST_1)
    .build();

DynamoDbAsyncClient dynamoDbAsyncClient = DynamoDbAsyncClient.builder()
    .httpClient(crtAsyncHttpClient)
    .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
    .defaultsMode(DefaultsMode.IN_REGION)
    .region(Region.US_EAST_1)
    .build();

// Requests completed: Close all service clients.
s3AsyncClient.close();
dynamoDbAsyncClient.close();
crtAsyncHttpClient.close(); // Explicitly close crtAsyncHttpClient.
```

Set an AWS CRT-based HTTP client as the default

You can setup your Maven build file to have the SDK use an AWS CRT-based HTTP client as the default HTTP client for service clients.

You do this by adding an `exclusions` element with the default HTTP client dependencies to each service client artifact.

In the following `pom.xml` example, the SDK uses an AWS CRT-based HTTP client for S3 services. If the service client in your code is an `S3AsyncClient`, the SDK uses `AwsCrtAsyncHttpClient`.

If the service client is an `S3Client`, the SDK uses `AwsCrtHttpClient`. With this setup the default Netty-based asynchronous HTTP client and the default Apache-based synchronous HTTP are not available.

```
<project>
  <properties>
    <aws.sdk.version>VERSION</aws.sdk.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>s3</artifactId>
      <version>${aws.sdk.version}</version>
      <exclusions>
        <exclusion>
          <groupId>software.amazon.awssdk</groupId>
          <artifactId>netty-nio-client</artifactId>
        </exclusion>
        <exclusion>
          <groupId>software.amazon.awssdk</groupId>
          <artifactId>apache-client</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>aws-crt-client</artifactId>
    </dependency>
  </dependencies>
</project>
```

Visit the Maven central repository for the latest [VERSION](#) value.

 Note

If multiple service clients are declared in a `pom.xml` file, all require the `exclusions` XML element.

Use a Java system property

To use the AWS CRT-based HTTP clients as the default HTTP for your application, you can set the Java system property `software.amazon.awssdk.http.async.service.impl` to a value of `software.amazon.awssdk.http.crt.AwsCrtSdkHttpClient`.

To set during application startup, run a command similar to the following.

```
java app.jar -Dsoftware.amazon.awssdk.http.async.service.impl=\
software.amazon.awssdk.http.crt.AwsCrtSdkHttpClient
```

Use the following code snippet to set the system property in your application code.

```
System.setProperty("software.amazon.awssdk.http.async.service.impl",
"software.amazon.awssdk.http.crt.AwsCrtSdkHttpClient");
```

Note

You need to add a dependency on the `aws-crt-client` artifact in your `pom.xml` file when you use a system property to configure the use of the AWS CRT-based HTTP clients.

Advanced configuration of AWS CRT-based HTTP clients

You can use various configuration settings of the AWS CRT-based HTTP clients, including connection health configuration and maximum idle time. You can review the configuration [options available](#) for the `AwsCrtAsyncHttpClient`. You can configure the same options for the `AwsCrtHttpClient`.

Connection health configuration

You can configure connection health configuration for the AWS CRT-based HTTP clients by using the `connectionHealthConfiguration` method on the HTTP client builder.

The following example creates an S3 service client that uses a AWS CRT-based HTTP client instance configured with connection health configuration and a maximum idle time for connections.

Synchronous client

Imports

```
import software.amazon.awssdk.http.crt.AwsCrtHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import java.time.Duration;
```

Code

```
// Singleton: Use the s3Client for all requests.
S3Client s3Client = S3Client.builder()
    .httpClientBuilder(AwsCrtHttpClient
        .builder()
        .connectionHealthConfiguration(builder -> builder
            .minimumThroughputInBps(32000L)
            .minimumThroughputTimeout(Duration.ofSeconds(3)))
        .connectionMaxIdleTime(Duration.ofSeconds(5)))
    .build();

// Perform work with s3Client.

// Requests complete: Close the service client.
s3Client.close();
```

Asynchronous client

Imports

```
import software.amazon.awssdk.http.crt.AwsCrtAsyncHttpClient;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import java.time.Duration;
```

Code

```
// Singleton: Use the s3AsyncClient for all requests.
S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
    .httpClientBuilder(AwsCrtAsyncHttpClient
        .builder()
        .connectionHealthConfiguration(builder -> builder
            .minimumThroughputInBps(32000L)
            .minimumThroughputTimeout(Duration.ofSeconds(3)))
        .connectionMaxIdleTime(Duration.ofSeconds(5)))
    .build();

// Perform work with s3AsyncClient.
```

```
// Requests complete: Close the service client.  
s3AsyncClient.close();
```

Proxy configuration example

The following code snippet shows the use of the [ProxyConfiguration.Builder](#) that you use to configure proxy setting in code.

Synchronous client

Imports

```
import software.amazon.awssdk.http.SdkHttpClient;  
import software.amazon.awssdk.http.crt.AwsCrtHttpClient;  
import software.amazon.awssdk.http.crt.ProxyConfiguration;
```

Code

```
SdkHttpClient crtHttpClient = AwsCrtHttpClient.builder()  
    .proxyConfiguration(ProxyConfiguration.builder()  
        .scheme("https")  
        .host("myproxy")  
        .port(1234)  
        .username("username")  
        .password("password")  
        .nonProxyHosts(Set.of("localhost", "host.example.com"))  
        .build())  
    .build();
```

Asynchronous client

Imports

```
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;  
import software.amazon.awssdk.http.crt.AwsCrtAsyncHttpClient;  
import software.amazon.awssdk.http.crt.ProxyConfiguration;
```

Code

```
SdkAsyncHttpClient crtAsyncHttpClient = AwsCrtAsyncHttpClient.builder()
```

```
.proxyConfiguration(ProxyConfiguration.builder()
    .scheme("https")
    .host("myproxy")
    .port(1234)
    .username("username")
    .password("password")
    .nonProxyHosts(Set.of("localhost", "host.example.com"))
    .build())
.build();
```

The equivalent Java system properties for the proxy configuration are shown in the following command line snippet.

```
$ java -Dhttps.proxyHost=myproxy -Dhttps.proxyPort=1234 -Dhttps.proxyUser=username \
-Dhttps.proxyPassword=password -Dhttp.nonProxyHosts=localhost|host.example.com -cp ...
App
```

Important

To use any of the HTTPS proxy system properties, the scheme property must be set in code to https. If the scheme property is not set in code, the scheme defaults to HTTP and the SDK looks only for http.* system properties.

The equivalent setup that uses environment variables is:

```
// Set the following environment variables.
// $ export HTTPS_PROXY="https://username:password@myproxy:1234"
// $ export NO_PROXY="localhost|host.example.com"

// Set the 'useSystemPropertyValues' to false on the proxy configuration.
SdkAsyncHttpClient crtAsyncHttpClient = AwsCrtAsyncHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .scheme("https")
        .useSystemPropertyValues(Boolean.FALSE)
        .build())
    .build();

// Run the application.
// $ java -cp ... App
```

Configure HTTP proxies

You can configure HTTP proxies by using code, by setting Java system properties, or by setting environment variables.

Configure in code

You configure proxies in code with a client-specific `ProxyConfiguration` builder when you build the service client. The following code shows an example proxy configuration for an Apache-based HTTP client that is used by an Amazon S3 service client.

```
SdkHttpClient httpClient1 = ApacheHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .endpoint(URI.create("http://proxy.example.com"))
        .username("username")
        .password("password")
        .addNonProxyHost("localhost")
        .build())
    .build();

S3Client s3Client = S3Client.builder()
    .httpClient(httpClient)
    .build();
```

The section for each HTTP client in this topic shows a proxy configuration example.

- [Apache HTTP client](#)
- [URLConnection-based HTTP client](#)
- [Netty-based HTTP client](#)
- [AWS CRT-based HTTP client](#)

Configure HTTP proxies with external settings

Even if you don't explicitly use a `ProxyConfiguration` builder in code, the SDK looks for external settings to configure a default proxy configuration.

By default, the SDK first searches for JVM system properties. If even one property is found, the SDK uses the value and any other system property values. If no system properties are available, the SDK looks for proxy environment variables.

The SDK can use the following Java system properties and environment variables.

Java system properties

System property	Description	HTTP client support
http.proxyHost	Host name of the HTTP proxy server	All
http.proxyPort	Port number of the HTTP proxy server	All
http.proxyUser	Username for HTTP proxy authentication	All
http.proxyPassword	Password for HTTP proxy authentication	All
http.nonProxyHosts	List of hosts that should be reached directly, bypassing the proxy. This list is also valid when HTTPS is used.	All
https.proxyHost	Host name of the HTTPS proxy server	Netty, CRT
https.proxyPort	Port number of the HTTPS proxy server	Netty, CRT
https.proxyUser	Username for HTTPS proxy authentication	Netty, CRT
https.proxyPassword	Password for HTTPS proxy authentication	Netty, CRT

Environment variables

Environment variable	Description	HTTP client support
HTTP_PROXY ¹	A valid URL with a scheme of HTTP	All
HTTPS_PROXY ¹	A valid URL with a scheme of HTTPS	Netty, CRT
NO_PROXY ²	List of hosts that should be reached directly, bypassing the proxy. The list is valid for both HTTP and HTTPS.	All

View key and footnotes

All - All HTTP clients offered by the SDK—`URLConnectionHttpClient`, `ApacheHttpClient`, `NettyNioAsyncHttpClient`, `AwsCrtAsyncHttpClient`.

Netty - The Netty-based HTTP client (`NettyNioAsyncHttpClient`).

CRT - The AWS CRT-based HTTP clients, (`AwsCrtHttpClient` and `AwsCrtAsyncHttpClient`).

¹The environment variable queried, whether `HTTP_PROXY` or `HTTPS_PROXY`, depends on the scheme setting in the client's `ProxyConfiguration`. The default scheme is HTTP. The following snippet shows how to change the scheme to HTTPS used for environment variable resolution.

```
SdkHttpClient httpClient = ApacheHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .scheme("https")
        .build())
    .build();
```

²The `NO_PROXY` environment variable supports a mix of "|" and "," separators between host names. Host names may include the "*" wildcard.

Use a combination of settings

You can use a combination of HTTP proxy settings in code, system properties, and environment variables.

Example– configuration provided by a system property and by code

```
// Command line with the proxy password set as a system property.
$ java -Dhttp.proxyPassword=SYS_PROP_password -cp ... App

// Since the 'useSystemPropertyValues' setting is 'true' (the default), the SDK will
// supplement
// the proxy configuration in code with the 'http.proxyPassword' value from the system
// property.
SdkHttpClient apacheHttpClient = ApacheHttpClient.builder()
    .proxyConfiguration(ProxyConfiguration.builder()
        .endpoint(URI.create("http://localhost:1234"))
        .username("username")
        .build())
    .build();

// Use the apache HTTP client with proxy configuration.
DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
    .httpClient(apacheHttpClient)
    .build();
```

The SDK resolves the following proxy settings.

```
Host = localhost
Port = 1234
Password = SYS_PROP_password
UserName = username
Non ProxyHost = null
```

Example– both system properties and environment variables are available

Each HTTP client's `ProxyConfiguration` builder offers settings named `useSystemPropertyValues` and `useEnvironmentVariablesValues`. By default, both settings are true. When true, the SDK automatically uses values from system properties or environment variables for options that are not provided by the `ProxyConfiguration` builder.

Important

System properties take precedence over environment variables. If an HTTP proxy system property is found, the SDK retrieves *all* values from system properties and none from

environment variables. If you want to prioritize environment variables over system properties, set `useSystemPropertyValues` to `false`.

For this example, the following settings are available a runtime:

```
// System properties
http.proxyHost=SYS_PROP_HOST.com
http.proxyPort=2222
http.password=SYS_PROP_PASSWORD
http.user=SYS_PROP_USER

// Environment variables
HTTP_PROXY="http://EnvironmentUser:EnvironmentPassword@ENV_VAR_HOST:3333"
NO_PROXY="environmentnonproxy.host,environmentnonproxy2.host:1234"
```

The service client is created with one of the following statements. None of the statements explicitly set a proxy setting.

```
DynamoDbClient client = DynamoDbClient.create();
DynamoDbClient client = DynamoDbClient.builder().build();
DynamoDbClient client = DynamoDbClient.builder()
    .httpClient(ApacheHttpClient.builder()
        .proxyConfiguration(ProxyConfiguration.builder()
            .build())
        .build())
    .build();
```

The following proxy settings are resolved by the SDK:

```
Host = SYS_PROP_HOST.com
Port = 2222
Password = SYS_PROP_PASSWORD
UserName = SYS_PROP_USER
Non ProxyHost = null
```

Because the service client has default proxy settings, the SDK searches for system properties and then environment variables. Since system properties settings take precedence over environment variables, the SDK uses only system properties.

If the use of system properties is changed to `false` as shown in the following code, the SDK resolves only the environment variables.

```
DynamoDbClient client = DynamoDbClient.builder()
    .httpClient(ApacheHttpClient.builder()
        .proxyConfiguration(ProxyConfiguration.builder()
            .useSystemPropertyValues(Boolean.FALSE)
            .build())
        .build())
    .build();
```

The resolved proxy settings using HTTP are:

```
Host = ENV_VAR_HOST
Port = 3333
Password = EnvironmentPassword
UserName = EnvironmentUser
Non ProxyHost = environmentnonproxy.host, environmentnonproxy2.host:1234
```

Configure the Apache 5.x based HTTP client

Access the Apache5HttpClient

In order to use the `Apache5HttpClient` you must add a dependency on **apache5-client** and explicitly configure `Apache5HttpClient` on your service clients.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
      <version>2.41.0*</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
```

```
<artifactId>s3</artifactId>
</dependency>

<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>apache5-client</artifactId>
</dependency>
</dependencies>
```

*Replace the version shown in red with the version of the Java SDK that you want to use. Find the latest on [Maven Central](#).

Use and configure the `Apache5HttpClient`

You can configure an instance of `Apache5HttpClient` along with building a service client, or you can configure a single instance to share across multiple service clients.

With either approach, you use the [Apache5HttpClient.Builder](#) to configure the properties for the Apache 5 based HTTP client.

Best practice: dedicate an `Apache5HttpClient` instance to a service client

If you need to configure an instance of the `Apache5HttpClient`, we recommend that you build the dedicated `Apache5HttpClient` instance. You can do so by using the `httpClientBuilder` method of the service client's builder. This way, the lifecycle of the HTTP client is managed by the SDK, which helps avoid potential memory leaks if the `Apache5HttpClient` instance is not closed down when it's no longer needed.

The following example creates an `S3Client` and configures the embedded instance of `Apache5HttpClient` with `maxConnections` and `connectionTimeout` values. The HTTP instance is created using the `httpClientBuilder` method of `S3Client.Builder`.

Imports

```
import software.amazon.awssdk.http.apache5.Apache5HttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import java.time.Duration;
```

Code

```
S3Client s3Client = S3Client // Singleton: Use the s3Client for all requests.
```

```
.builder()
.httpClientBuilder(Apache5HttpClient.builder()
    .maxConnections(100)
    .connectionTimeout(Duration.ofSeconds(5))
)
.build();

// Perform work with the s3Client.

s3Client.close(); // Requests completed: Close all service clients.
```

Alternative approach: share an `Apache5HttpClient` instance

To help keep resource and memory usage lower for your application, you can configure an `Apache5HttpClient` and share it across multiple service clients. The HTTP connection pool will be shared, which lowers resource usage.

Note

When an `Apache5HttpClient` instance is shared, you must close it when it is ready to be disposed. The SDK will not close the instance when the service client is closed.

The following example configures an Apache-based HTTP client that is used by two service clients. The configured `ApacheHttpClient` instance is passed to the `httpClient` method of each builder. When the service clients and the HTTP client are no longer needed, the code explicitly closes them. The code closes the HTTP client last.

Imports

```
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.apache5.Apache5HttpClient;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.s3.S3Client;
```

Code

```
SdkHttpClient apache5HttpClient = Apache5HttpClient.builder()
    .maxConnections(100).build();

// Singletons: Use the s3Client and dynamoDbClient for all requests.
```

```
S3Client s3Client =
    S3Client.builder()
        .httpClient(apache5HttpClient).build();

DynamoDbClient dynamoDbClient =
    DynamoDbClient.builder()
        .httpClient(apache5HttpClient).build();

// Perform work with the s3Client and dynamoDbClient.

// Requests completed: Close all service clients.
s3Client.close();
dynamoDbClient.close();
apache5HttpClient.close(); // Explicitly close apache5HttpClient.
```

Use execution interceptors in the AWS SDK for Java 2.x

Execution interceptors in the AWS SDK for Java 2.x hook into the request and response lifecycle to perform custom logic at various stages of API call execution. Use interceptors to implement cross-cutting concerns such as logging, metrics collection, request modification, debugging, and error handling.

Interceptors implement the [ExecutionInterceptor](#) interface, which provides hooks for different phases of request execution.

Interceptor lifecycle

The `ExecutionInterceptor` interface provides methods that are called at specific points during request execution:

- `beforeExecution` - Called before the request executes
- `modifyRequest` - Modifies the SDK request object
- `beforeMarshalling` - Called before the request marshals to HTTP
- `afterMarshalling` - Called after the request marshals to HTTP
- `modifyHttpRequest` - Modifies the HTTP request
- `beforeTransmission` - Called before the HTTP request sends
- `afterTransmission` - Called after the HTTP response is received
- `modifyHttpResponse` - Modifies the HTTP response

- `beforeUnmarshalling` - Called before the HTTP response unmarshals
- `afterUnmarshalling` - Called after the HTTP response unmarshals
- `modifyResponse` - Modifies the SDK response object
- `afterExecution` - Called after successful request execution
- `onExecutionFailure` - Called when request execution fails

Register interceptors

Register interceptors when you build a service client using the `overrideConfiguration` method. You can register multiple interceptors, and they execute in the order you register them.

```
// Register a single interceptor.
SqsClient client = SqsClient.builder()
    .overrideConfiguration(c -> c.addExecutionInterceptor(new LoggingInterceptor()))
    .build();

// Register multiple interceptors.
S3Client s3Client = S3Client.builder()
    .overrideConfiguration(config -> config
        .addExecutionInterceptor(new TimingInterceptor())
        .addExecutionInterceptor(new LoggingInterceptor())
        .addExecutionInterceptor(new RequestModificationInterceptor()))
    .build();
```

Interceptor example

The following class demonstrates how to use execution interceptors to add cross-cutting concerns like logging, performance monitoring, and request modification to your S3 operations without changing your core business logic.

This example shows you how to register multiple interceptors on an S3 client and see them in action during real AWS API calls.

Imports

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.awscore.exception.AwsServiceException;
import software.amazon.awssdk.core.ApiName;
```

```
import software.amazon.awssdk.core.SdkRequest;
import software.amazon.awssdk.core.interceptor.Context;
import software.amazon.awssdk.core.interceptor.ExecutionAttributes;
import software.amazon.awssdk.core.interceptor.ExecutionInterceptor;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.SdkHttpResponse;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.HeadBucketRequest;
import software.amazon.awssdk.services.s3.model.ListBucketsResponse;

import java.time.Duration;
import java.time.Instant;
```

```
public class S3InterceptorsDemo {
    private static final Logger logger =
        LoggerFactory.getLogger(S3InterceptorsDemo.class);

    public static void main(String[] args) {
        logger.info("=== AWS SDK for Java v2 - S3 Interceptors Demo ===");

        // Create an S3 client with multiple interceptors.
        S3Client s3Client = S3Client.builder()
            .overrideConfiguration(config -> config
                .addExecutionInterceptor(new TimingInterceptor())
                .addExecutionInterceptor(new LoggingInterceptor())
                .addExecutionInterceptor(new RequestModificationInterceptor()))
            .build();

        try {
            logger.info("Starting S3 operations with interceptors...");

            // Operation 1: List buckets.
            logger.info("Operation 1: Listing S3 buckets");
            logger.info("-----");
            ListBucketsResponse listBucketsResponse = s3Client.listBuckets();
            logger.info("Found {} buckets", listBucketsResponse.buckets().size());

            // Operation 2: Try to access a bucket that likely doesn't exist.
            logger.info("Operation 2: Checking non-existent bucket (demonstrating error interceptor)");
            logger.info("-----");
            try {
                s3Client.headBucket(HeadBucketRequest.builder()
```

```

        .bucket("amzn-s3-demo-bucket-that-does-not-exist-1234")
        .build());
    } catch (Exception e) {
        logger.info("Expected error occurred (interceptor should have logged
it)");
    }

    } catch (Exception e) {
        logger.error(" Error during S3 operations: {}", e.getMessage(), e);
    } finally {
        s3Client.close();
        logger.info("Demo completed - S3 client closed");
    }
}

// Logging interceptor.
private static class LoggingInterceptor implements ExecutionInterceptor {
    private static final Logger logger =
LoggerFactory.getLogger(LoggingInterceptor.class);

    @Override
    public void beforeExecution(Context.BeforeExecution context,
ExecutionAttributes executionAttributes) {
        logger.info("[LOGGING] Starting request: {}",
context.request().getClass().getSimpleName());
    }

    @Override
    public void afterExecution(Context.AfterExecution context, ExecutionAttributes
executionAttributes) {
        logger.info("[LOGGING] Completed request: {}",
context.request().getClass().getSimpleName());
    }

    @Override
    public void onExecutionFailure(Context.FailedExecution context,
ExecutionAttributes executionAttributes) {
        logger.error("[LOGGING] Request failed: {}",
context.request().getClass().getSimpleName());
        if (context.exception() instanceof AwsServiceException) {
            AwsServiceException ase = (AwsServiceException) context.exception();
            if (ase.awsErrorDetails().errorCode() != null) {
                SdkHttpResponse httpResponse =
ase.awsErrorDetails().sdkHttpResponse();

```



```

        logger.error("    HTTP Status: {}", httpResponse.statusCode());
        logger.error("    Error Code: {}",
ase.awsErrorDetails().errorCode());
        logger.error("    Error Message: {}",
ase.awsErrorDetails().errorMessage());
    }
}

// Performance timing interceptor.
private static class TimingInterceptor implements ExecutionInterceptor {
    private static final Logger logger =
LoggerFactory.getLogger(TimingInterceptor.class);
    private Instant startTime;

    @Override
    public void beforeExecution(Context.BeforeExecution context,
ExecutionAttributes executionAttributes) {
        startTime = Instant.now();
        logger.info("## [TIMING] Request started at: {}", startTime);
    }

    @Override
    public void afterExecution(Context.AfterExecution context, ExecutionAttributes
executionAttributes) {
        if (startTime != null) {
            Duration duration = Duration.between(startTime, Instant.now());
            logger.info("## [TIMING] Request completed in: {}ms",
duration.toMillis());
        }
    }

    @Override
    public void onExecutionFailure(Context.FailedExecution context,
ExecutionAttributes executionAttributes) {
        if (startTime != null) {
            Duration duration = Duration.between(startTime, Instant.now());
            logger.warn("## [TIMING] Request failed after: {}ms",
duration.toMillis());
        }
    }
}

```

```

// Request modification interceptor
private static class RequestModificationInterceptor implements ExecutionInterceptor
{
    private static final Logger logger =
LoggerFactory.getLogger(RequestModificationInterceptor.class);

    @Override
    public SdkRequest modifyRequest(Context.ModifyRequest context,
ExecutionAttributes executionAttributes) {
        SdkRequest originalRequest = context.request();
        logger.info("[MODIFY] Modifying request: {}",
originalRequest.getClass().getSimpleName());

        // For ListBucketsRequest, we can't modify much since it has no settable
properties
        // For HeadBucketRequest, we can demonstrate modifying the request
        if (originalRequest instanceof HeadBucketRequest) {
            HeadBucketRequest headRequest = (HeadBucketRequest) originalRequest;

            // Create a new request with an API name added.
            return HeadBucketRequest.builder()
                .bucket(headRequest.bucket())
                .overrideConfiguration(b -> b.addApiName(ApiName.builder()
                    .name("My-API")
                    .version("1.0")
                    .build()))
                .build();
        }
        logger.info("Not a HeadBucketRequest, returning original request");
        return originalRequest;
    }

    @Override
    public SdkHttpRequest modifyHttpRequest(Context.ModifyHttpRequest context,
ExecutionAttributes executionAttributes) {
        logger.info("[MODIFY] Adding custom HTTP headers");
        return context.httpRequest().toBuilder()
            .putHeader("X-Custom-Header", "S3InterceptorDemo")
            .putHeader("X-Request-ID", java.util.UUID.randomUUID().toString())
            .build();
    }
}

```

Maven pom file

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>interceptors-examples</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>interceptors-examples</name>
  <description>Demonstration of execution interceptors in AWS SDK for Java v2</
description>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <aws.java.sdk.version>2.31.62</aws.java.sdk.version>
    <exec.mainClass>org.example.S3InterceptorsDemo</exec.mainClass>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>software.amazon.awssdk</groupId>
        <artifactId>bom</artifactId>
        <version>${aws.java.sdk.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-bom</artifactId>
        <version>2.23.1</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
```

```
<dependencies>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
  </dependency>

  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.13</version>
  </dependency>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-slf4j2-impl</artifactId>
  </dependency>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-1.2-api</artifactId>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- Compiler Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

```

<!-- Surefire Plugin for running tests -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.2</version>
</plugin>

<!-- Exec Plugin for running the main class -->
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.1.0</version>
  <configuration>
    <mainClass>${exec.mainClass}</mainClass>
  </configuration>
</plugin>

<!-- Shade Plugin to create executable JAR -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.4.1</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals>
        <goal>shade</goal>
      </goals>
      <configuration>
        <transformers>
          <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>${exec.mainClass}</mainClass>
          </transformer>
        </transformers>
        <createDependencyReducedPom>>false</
createDependencyReducedPom>
      </configuration>
    </execution>
  </executions>
</plugin>
</plugins>
</build>

```

```
</project>
```

Best practices

- **Keep interceptors lightweight** - Interceptors execute for every request, so avoid heavy computations or blocking operations that could impact performance.
- **Handle exceptions gracefully** - If your interceptor throws an exception, it causes the entire request to fail. Always use try-catch blocks for potentially failing operations.
- **Order matters** - Interceptors execute in the order you register them. Consider the dependencies between your interceptors when you register them.
- **Use `ExecutionAttributes` for state** - If you need to pass data between different interceptor methods, use `ExecutionAttributes` rather than instance variables to ensure thread safety.
- **Be mindful of sensitive data** - When you log requests and responses, be careful not to log sensitive information such as credentials or personal data.

Context objects

Each interceptor method receives a context object that provides access to request and response information at different stages of execution:

- `Context.BeforeExecution` - Provides access to the original SDK request
- `Context.ModifyRequest` - Modifies the SDK request
- `Context.ModifyHttpRequest` - Modifies the HTTP request
- `Context.AfterExecution` - Provides access to both request and response
- `Context.FailedExecution` - Provides access to the request and the exception that occurred

Using the AWS SDK for Java 2.x

This chapter shows you how to use the AWS SDK for Java 2.x effectively. Learn to create service clients, make requests, handle responses, and manage errors. The chapter covers synchronous and asynchronous programming, paginated results, waiters for resource monitoring, and performance optimization.

You'll also find best practices for client reuse, troubleshooting guidance, Lambda startup optimization, HTTP/2 support, and DNS configuration.

Contents

- [Making AWS service requests using the AWS SDK for Java 2.x](#)
 - [Using service clients to make requests](#)
 - [Create a service client](#)
 - [Default client configuration](#)
 - [Configure service clients](#)
 - [Close the service client](#)
 - [Make requests](#)
 - [Use requests to override client configuration](#)
 - [Handle responses](#)
- [Programming asynchronously using the AWS SDK for Java 2.x](#)
 - [Use asynchronous client APIs](#)
 - [Handle streaming in asynchronous methods](#)
 - [Configure advanced asynchronous options](#)
- [Best practices for using the AWS SDK for Java 2.x](#)
 - [Prevent hanging requests by configuring API timeouts](#)
 - [Improve performance by reusing service clients](#)
 - [Prevent resource leaks by closing unused service clients](#)
 - [Prevent connection pool exhaustion by closing input streams](#)
 - [Optimize HTTP performance for your application workload](#)
 - [Improve SSL performance with OpenSSL for async clients](#)
 - [Monitor application performance with SDK metrics](#)

- [Handling errors in the AWS SDK for Java 2.x](#)
 - [Why unchecked exceptions?](#)
 - [AwsServiceException \(and subclasses\)](#)
 - [SdkClientException](#)
 - [Exceptions and retry behavior](#)
- [Using paginated results in the AWS SDK for Java 2.x](#)
 - [Synchronous pagination](#)
 - [Iterate over pages](#)
 - [Iterate over objects](#)
 - [Use a stream](#)
 - [Use a for-each loop](#)
 - [Manual pagination](#)
 - [Asynchronous pagination](#)
 - [Iterate over pages of table names](#)
 - [Use a Subscriber](#)
 - [Use a Consumer](#)
 - [Iterate over table names](#)
 - [Use a Subscriber](#)
 - [Use a Consumer](#)
 - [Use third-party library](#)
- [Using waiters in the AWS SDK for Java 2.x](#)
 - [Prerequisites](#)
 - [Using waiters](#)
 - [Synchronous programming](#)
 - [Asynchronous programming](#)
 - [Configure waiters](#)
 - [Configure a waiter](#)
 - [Override configuration for a specific request](#)
 - [Code examples](#)
- [Troubleshooting FAQs](#)

- [How do I fix "java.net.SocketException: Connection reset" or "server failed to complete the response" error?](#)
- [How do I fix "connection timeout"?](#)
- [How do I fix "java.net.SocketTimeoutException: Read timed out"?](#)
- [How do I fix "Unable to execute HTTP request: Timeout waiting for connection from pool" error?](#)
- [How do I fix a NoClassDefFoundError, NoSuchMethodError or NoSuchFieldError?](#)
- [How do I fix a "SignatureDoesNotMatch" error or "The request signature we calculated does not match the signature you provided" error?](#)
- [How do I fix "java.lang.IllegalStateException: Connection pool shut down" error?](#)
- [How do I fix "Unable to load credentials from any of the providers in the chain AwsCredentialsProviderChain"?](#)
 - [Common causes and solutions](#)
 - [Review your credential configuration](#)
 - [For Amazon EC2 instances](#)
 - [For container environments](#)
 - [For local development](#)
 - [For web identity federation](#)
 - [Network or proxy connectivity issues](#)
- [Reduce SDK startup time for AWS Lambda](#)
 - [Use an AWS CRT-based HTTP client](#)
 - [Remove unused HTTP client dependencies](#)
 - [Configure service clients to shortcut lookups](#)
 - [Initialize the SDK client outside of the Lambda function handler](#)
 - [Minimize dependency injection](#)
 - [Use a Maven Archetype targeting AWS Lambda](#)
 - [Consider Lambda SnapStart for Java](#)
 - [Version 2.x changes that affect startup time](#)
 - [Additional resources](#)
- [Implement ContentStreamProvider in the AWS SDK for Java 2.x](#)
 - [Use mark\(\) and reset\(\)](#)

- [Use buffering if mark\(\) and reset\(\) are not available](#)
- [Create new streams](#)
- [Set the JVM TTL for DNS name lookups](#)
- [How to set the JVM TTL](#)
 - [Option 1: Set it programmatically in your application](#)
 - [Option 2: Set it in the java.security file](#)
 - [Option 3: Use the JDK system properties fallback \(command-line\)](#)
- [Work with HTTP/2 in the AWS SDK for Java](#)

Making AWS service requests using the AWS SDK for Java 2.x

Using service clients to make requests

After completing the steps in [Setting up the SDK](#) and understanding how to [configure service clients](#), you are ready to make requests to AWS services such as Amazon S3, Amazon DynamoDB, AWS Identity and Access Management, Amazon EC2, and more.

Create a service client

To make a request to an AWS service, you must first instantiate a service client for that service by using the static factory method, `builder()`. The `builder()` method returns a builder object that allows you to customize the service client. The fluent setter methods return the builder object, so that you can chain the method calls for convenience and for more readable code. After you configure the properties you want, call the `build()` method to create the client.

As an example, the following code snippet instantiates an `Ec2Client` object as a service client for Amazon EC2.

```
Region region = Region.US_WEST_2;
Ec2Client ec2Client = Ec2Client.builder()
    .region(region)
    .build();
```

Note

Service clients in the SDK are thread-safe. For best performance, treat them as long-lived objects. Each client has its own connection pool resource that is released when the client is garbage collected.

A service client object is immutable, so you must create a new client for each service to which you make requests, or if you want to use a different configuration for making requests to the same service.

Specifying the Region in the service client builder is not required for all AWS services; however, it is a best practice to set the Region for the API calls you make in your applications. See [AWS region selection](#) for more information.

Default client configuration

The client builders have another factory method named `create()`. This method creates a service client with the default configuration. It uses the [default provider chain](#) to load credentials and the [default AWS Region provider chain](#). If credentials or the Region can't be determined from the environment that the application is running in, the call to `create` fails. See [Using credentials](#) and [Region selection](#) for more information about how the SDK determines the credentials and Region to use.

For example, the following code snippet instantiates a `DynamoDbClient` object as a service client for Amazon DynamoDB:

```
DynamoDbClient dynamoDbClient = DynamoDbClient.create();
```

Configure service clients

For details about how to configure service clients, see [Client configuration externally](#) and [Client configuration in code](#).

Close the service client

As a best practice, you should use a service clients for multiple API service calls during the life of an application. However, if you need a service client for a one-time use or no longer need the service client, close it.

Call the `close()` method when the service client is no longer needed to free up resources.

```
ec2Client.close();
```

If you need a service client for one-time use, you can instantiate the service client as a resource in a try-with-resources statement. Service clients implement the [Autoclosable](#) interface, so the JDK automatically calls the `close()` method at the end of the statement.

The following example demonstrates how to use a service client for a one-off call. The `StsClient` that calls the AWS Security Token Service is closed after it returns the account ID.

```
import software.amazon.awssdk.services.sts.StsClient;

String getAccountID() {
    try (StsClient stsClient = StsClient.create()) {
        return stsClient.getCallerIdentity().account();
    }
}
```

Make requests

Use the service client to make requests to the corresponding AWS service.

For example, this code snippet shows how to create a `RunInstancesRequest` object to create a new Amazon EC2 instance:

```
// Create the request by using the fluid setter methods of the request builder.
RunInstancesRequest runInstancesRequest = RunInstancesRequest.builder()
    .imageId(amiId)
    .instanceType(InstanceType.T1_MICRO)
    .maxCount(1)
    .minCount(1)
    .build();

// Use the configured request with the service client.
RunInstancesResponse response = ec2Client.runInstances(runInstancesRequest);
```

Rather than create a request and pass in the instance, the SDK provides a fluent API that you can use to create a request. With the fluent API you can use a Java lambda expressions to create the request 'in-line'.

The following example rewrites the previous example by using the version of the `runInstances` [method that uses a builder](#) to create the request.

```
// Create the request by using a lambda expression.
RunInstancesResponse response = ec2.runInstances(r -> r
    .imageId(amiId)
    .instanceType(InstanceType.T1_MICRO)
    .maxCount(1)
    .minCount(1));
```

Use requests to override client configuration

Although a service client is immutable, you can override many of its settings at the request level. When you build a request, you can provide an [AwsRequestOverrideConfiguration](#) instance to provide the overridden settings. Some of the methods you can use to override client settings are:

- `apiCallAttemptTimeout`
- `apiCallTimeout`
- `credentialProvider`
- `compressionConfiguration`
- `putHeader`

For an example of overriding a client setting with a request, assume that you have the following S3 client that uses default settings.

```
S3Client s3Client = S3Client.create();
```

You want to download a large file and want to be sure the request doesn't timeout before the download finishes. To accomplish this, increase the timeout values for only a single `GetObject` request as shown in the following code.

Standard API

```
AwsRequestOverrideConfiguration overrideConfiguration =
    AwsRequestOverrideConfiguration.builder()
        .apiCallTimeout(Duration.ofSeconds(100L))
        .apiCallAttemptTimeout(Duration.ofSeconds(25L))
        .build();

GetObjectRequest request = GetObjectRequest.builder()
    .bucket("amzn-s3-demo-bucket")
```

```
.key("demo-key")
.overrideConfiguration(overrideConfiguration)
.build();

s3Client.getObject(request, myPath);
```

Fluent API

```
s3Client.getObject(b -> b
    .bucket("amzn-s3-demo-bucket")
    .key("demo-key")
    .overrideConfiguration(c -> c
        .apiCallTimeout(Duration.ofSeconds(100L))
        .apiCallAttemptTimeout(Duration.ofSeconds(25L))),
    myPath);
```

Handle responses

The SDK returns a response object for most service operations. Your code can process the information in the response object according to your needs.

For example, the following code snippet prints out the first instance id returned with the [RunInstancesResponse](#) object from the previous request.

```
RunInstancesResponse runInstancesResponse =
    ec2Client.runInstances(runInstancesRequest);
System.out.println(runInstancesResponse.instances().get(0).instanceId());
```

Not all operations return a response object with service-specific data, however. In these situations, you can query the HTTP response status to learn if the operation was successful.

For example, the code in the following snippet checks the HTTP response to see if the [DeleteContactList](#) operation of Amazon Simple Email Service was successful.

```
SesV2Client sesv2Client = SesV2Client.create();

DeleteContactListRequest request = DeleteContactListRequest.builder()
    .contactListName("ExampleContactListName")
    .build();
```

```
DeleteContactListResponse response = sesv2Client.deleteContactList(request);
if (response.sdkHttpResponse().isSuccessful()) {
    System.out.println("Contact list deleted successfully");
} else {
    System.out.println("Failed to delete contact list. Status code: " +
        response.sdkHttpResponse().statusCode());
}
```

Programming asynchronously using the AWS SDK for Java 2.x

The AWS SDK for Java 2.x features asynchronous clients with non-blocking I/O support that implement high concurrency across a few threads. However, total non-blocking I/O is not guaranteed. Asynchronous client may perform blocking calls in some cases such as credential retrieval, request signing using [AWS Signature Version 4 \(SigV4\)](#), or endpoint discovery.

Synchronous methods block your thread's execution until the client receives a response from the service. Asynchronous methods return immediately, giving control back to the calling thread without waiting for a response.

Because an asynchronous method returns before a response is available, you need a way to get the response when it's ready. The methods for asynchronous client in 2.x of the AWS SDK for Java return *CompletableFuture* objects that allow you to access the response when it's ready.

Use asynchronous client APIs

The signatures of asynchronous client methods are the same their synchronous counterpart, but the asynchronous methods return a [CompletableFuture](#) object that contains the results of the asynchronous operation *in the future*. If an error is thrown while the SDK's asynchronous method executes, the error is thrown as *CompletionException*.

One approach you can use to get the result is to chain a `whenComplete()` method onto the *CompletableFuture* returned by the SDK method call. The `whenComplete()` method receives the result or a *Throwable* object of type *CompletionException* depending on how the asynchronous call completed. You provide an action to `whenComplete()` to process or check the results before it is returned to the calling code.

If you want to return something other than the object returned by the SDK method, use the `handle()` method instead. The `handle()` method takes the same parameters as `whenComplete()`, but you can process the result and return an object.

To wait for the asynchronous chain to complete and retrieve the completion results, you can call the `join()` method. If the `Throwable` object was not handled in the chain, the `join()` method throws an unchecked `CompletionException` that wraps the original exception. You access the original exception with `CompletionException#getCause()`. You can also call the `CompletableFuture#get()` method to get the completion results. The `get()` method, however, can throw checked exceptions.

The following example shows two variations of how you can work with the `listTables()` method of the DynamoDB asynchronous client. The action passed to `whenComplete()` simply logs a successful response, whereas the `handle()` version extracts the list of table names and returns the list. In both cases if an error is generated in the asynchronous chain, the error is rethrown so the client code has a chance to handle it.

Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;
import software.amazon.awssdk.services.dynamodb.model.ListTablesResponse;

import java.util.List;
import java.util.concurrent.CompletableFuture;
```

Code

`whenComplete()` variation

```
public class DynamoDbAsyncListTables {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        DynamoDbAsyncClient dynamoDbAsyncClient =
        DynamoDbAsyncClient.builder().region(region).build();
        try {
            ListTablesResponse listTablesResponse =
            listTablesWhenComplete(dynamoDbAsyncClient).join(); // The join() method may throw
            a CompletionException.
            if (listTablesResponse.hasTableNames()){
                System.out.println("Table exist in this region: " + region.id());
            }
        } catch (RuntimeException e) {
```



```

        // Handle as needed. Here we simply print out the class names.
        System.out.println(e.getClass()); // Prints 'class
java.util.concurrent.CompletionException'.
        System.out.println(e.getCause().getClass()); // Prints 'class
software.amazon.awssdk.services.dynamodb.model.DynamoDbException'.
    }
}

public static CompletableFuture<ListTablesResponse>
listTablesWhenComplete(DynamoDbAsyncClient client) {
    return client.listTables(ListTablesRequest.builder().build())
        .whenComplete((listTablesResponse, throwable) -> {
            if (listTablesResponse != null) { // Consume the response.
                System.out.println("The SDK's listTables method completed
successfully.");
            } else {
                RuntimeException cause = (RuntimeException)
throwable.getCause(); // If an error was thrown during the SDK's listTables method
it is wrapped in a CompletionException.

                // The SDK throws only RuntimeExceptions, so this is a safe cast.
                System.out.println(cause.getMessage()); // Log error here, but
rethrow so the calling code can handle as needed.
                throw cause;
            }
        });
}

```

handle() variation

```

public class DynamoDbAsyncListTables {

    public static void main(String[] args) {
        Region region = Region.US_EAST_1;
        DynamoDbAsyncClient dynamoDbAsyncClient =
DynamoDbAsyncClient.builder().region(region).build();
        try {
            List<String> tableNames =
listTablesHandle(dynamoDbAsyncClient).join(); // The join() method may throw a
CompletionException.
            tableNames.forEach(System.out::println);
        } catch (RuntimeException e) {
            // Handle as needed. Here we simply print out the class names.

```

```

        System.out.println(e.getClass()); // Prints 'class
java.util.concurrent.CompletionException'.
        System.out.println(e.getCause().getClass()); // Prints 'class
software.amazon.awssdk.services.dynamodb.model.DynamoDbException'.
    }
}

public static CompletableFuture<List<String>>
listTablesHandle(DynamoDbAsyncClient client) {
    return client.listTables(ListTablesRequest.builder().build())
        .handle((listTablesResponse, throwable) -> {
            if (listTablesResponse != null) {
                return listTablesResponse.tableNames(); // Return the list of
table names.
            } else {
                RuntimeException cause = (RuntimeException)
throwable.getCause(); // If an error was thrown during the SDK's listTables method
it is wrapped in a CompletionException.

                // The SDK throws only RuntimeExceptions, so this is a safe cast.
                System.out.println(cause.getMessage()); // Log error here, but
rethrow so the calling code can handle as needed.
                throw cause;
            }
        });
}
}

```

Handle streaming in asynchronous methods

For asynchronous methods with streaming content, you must provide an [AsyncRequestBody](#) to provide the content incrementally, or an [AsyncResponseTransformer](#) to receive and process the response.

The following example uploads a file to Amazon S3 asynchronously by using the asynchronous form of the PutObject operation.

Imports

```

import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3AsyncClient;

```

```
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
import java.nio.file.Paths;
import java.util.concurrent.CompletableFuture;
```

Code

```
/**
 * To run this AWS code example, ensure that you have setup your development
 * environment, including your AWS credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class S3AsyncOps {

    public static void main(String[] args) {

        final String USAGE = "\n" +
            "Usage:\n" +
            "    S3AsyncOps <bucketName> <key> <path>\n\n" +
            "Where:\n" +
            "    bucketName - the name of the Amazon S3 bucket (for example,
bucket1). \n\n" +
            "    key - the name of the object (for example, book.pdf). \n" +
            "    path - the local path to the file (for example, C:/AWS/book.pdf).
\n" ;

        if (args.length != 3) {
            System.out.println(USAGE);
            System.exit(1);
        }

        String bucketName = args[0];
        String key = args[1];
        String path = args[2];

        Region region = Region.US_WEST_2;
        S3AsyncClient client = S3AsyncClient.builder()
            .region(region)
            .build();
```

```
PutObjectRequest objectRequest = PutObjectRequest.builder()
    .bucket(bucketName)
    .key(key)
    .build();

// Put the object into the bucket
CompletableFuture<PutObjectResponse> future = client.putObject(objectRequest,
    AsyncRequestBody.fromFile(Paths.get(path))
);
future.whenComplete((resp, err) -> {
    try {
        if (resp != null) {
            System.out.println("Object uploaded. Details: " + resp);
        } else {
            // Handle error
            err.printStackTrace();
        }
    } finally {
        // Only close the client when you are completely done with it
        client.close();
    }
});

future.join();
}
```

The following example gets a file from Amazon S3 by using the asynchronous form of the `GetObject` operation.

Imports

```
import software.amazon.awssdk.core.async.AsyncResponseTransformer;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import java.nio.file.Paths;
import java.util.concurrent.CompletableFuture;
```

Code

```
/**
 * To run this AWS code example, ensure that you have setup your development
 * environment, including your AWS credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class S3AsyncStreamOps {

    public static void main(String[] args) {

        final String USAGE = "\n" +
            "Usage:\n" +
            "    S3AsyncStreamOps <bucketName> <objectKey> <path>\n\n" +
            "Where:\n" +
            "    bucketName - the name of the Amazon S3 bucket (for example, bucket1). \n\n" +
            "    objectKey - the name of the object (for example, book.pdf). \n" +
            "    path - the local path to the file (for example, C:/AWS/book.pdf). \n" ;

        if (args.length != 3) {
            System.out.println(USAGE);
            System.exit(1);
        }

        String bucketName = args[0];
        String objectKey = args[1];
        String path = args[2];

        Region region = Region.US_WEST_2;
        S3AsyncClient client = S3AsyncClient.builder()
            .region(region)
            .build();

        GetObjectRequest objectRequest = GetObjectRequest.builder()
            .bucket(bucketName)
            .key(objectKey)
            .build();
```

```

        CompletableFuture<GetObjectResponse> futureGet =
client.getObject(objectRequest,
        AsyncResponseTransformer.toFile(Paths.get(path)));

        futureGet.whenComplete((resp, err) -> {
            try {
                if (resp != null) {
                    System.out.println("Object downloaded. Details: "+resp);
                } else {
                    err.printStackTrace();
                }
            } finally {
                // Only close the client when you are completely done with it
                client.close();
            }
        });
        futureGet.join();
    }
}

```

Configure advanced asynchronous options

The AWS SDK for Java 2.x uses [Netty](#), an asynchronous event-driven network application framework, to handle I/O threads. The AWS SDK for Java 2.x creates an `ExecutorService` behind Netty, to complete the futures returned from the HTTP client request through to the Netty client. This abstraction reduces the risk of an application breaking the async process if developers choose to stop or sleep threads. By default, each asynchronous client creates a threadpool based on the number of processors and manages the tasks in a queue within the `ExecutorService`.

You can specify a specific JDK implementation of `ExecutorService` when you build your asynchronous client. The following snippet create an `ExecutorService` with a fixed number of threads.

Code

```

S3AsyncClient clientThread = S3AsyncClient.builder()
    .asyncConfiguration(
        b -> b.advancedOption(SdkAdvancedAsyncClientOption
            .FUTURE_COMPLETION_EXECUTOR,
            Executors.newFixedThreadPool(10))
    )
)

```

```
.build();
```

To optimize performance, you can manage your own thread pool executor, and include it when you configure your client.

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(50, 50,
    10, TimeUnit.SECONDS,
    new LinkedBlockingQueue<>(<custom_value>),
    new ThreadFactoryBuilder()
        .threadNamePrefix("sdk-async-response").build());

// Allow idle core threads to time out
executor.allowCoreThreadTimeOut(true);

S3AsyncClient clientThread = S3AsyncClient.builder()
    .asyncConfiguration(
        b -> b.advancedOption(SdkAdvancedAsyncClientOption
            .FUTURE_COMPLETION_EXECUTOR,
            executor
        )
    )
    .build();
```

Best practices for using the AWS SDK for Java 2.x

Prevent hanging requests by configuring API timeouts

The SDK provides [default values](#) for some timeout options, such as connection timeout and socket timeouts, but not for API call timeouts or individual API call attempt timeouts. It is a good practice to set timeouts for both the individual attempts and the entire request. This will ensure your application fails fast in an optimal way when there are transient issues that could cause request attempts to take longer to complete or fatal network issues.

You can configure timeouts for all requests made by a service clients using [ClientOverrideConfiguration#apiCallAttemptTimeout](#) and [ClientOverrideConfiguration#apiCallTimeout](#).

The following example shows the configuration of an Amazon S3 client with custom timeout values.

```
S3Client.builder()  
    .overrideConfiguration(  
        b -> b.apiCallTimeout(Duration.ofSeconds(<custom value>))  
            .apiCallAttemptTimeout(Duration.ofMillis(<custom value>)))  
    .build();
```

apiCallAttemptTimeout

This setting sets the amount of time for a single HTTP attempt, after which the API call can be retried.

apiCallTimeout

The value for this property configures the amount of time for the entire execution, including all retry attempts.

As an alternative to setting these timeout values on the service client, you can use [RequestOverrideConfiguration#apiCallTimeout\(\)](#) and [RequestOverrideConfiguration#apiCallAttemptTimeout\(\)](#) to configure a single request .

The following example configures a single `listBuckets` request with custom timeout values.

```
s3Client.listBuckets(lbr -> lbr.overrideConfiguration(  
    b -> b.apiCallTimeout(Duration.ofSeconds(<custom value>))  
        .apiCallAttemptTimeout(Duration.ofMillis(<custom value>))));
```

When you use these properties together, you set a hard limit on the total time spent on all attempts across retries. You also set an individual HTTP request to fail fast on a slow request.

Improve performance by reusing service clients

Each [service client](#) maintains its own HTTP connection pool. A connection that already exists in the pool can be reused by a new request to cut down the time to establish a new connection. We recommend sharing a single instance of the client to avoid the overhead of having too many connection pools that aren't used effectively. All service clients are thread safe.

If you don't want to share a client instance, call `close()` on the instance to release the resources when the client is not needed.

Prevent resource leaks by closing unused service clients

Close a [service client](#) to release resources, such as threads, if it is no longer needed. If you don't want to share a client instance, call `close()` on the instance to release the resources when the client is not needed.

Prevent connection pool exhaustion by closing input streams

For streaming operations such as [S3Client#getObject](#), if you are working with [ResponseInputStream](#) directly, we recommend that you do the following:

- Read all the data from the input stream as soon as possible.
- Close the input stream as soon as possible.

We make these recommendations because the input stream is a direct stream of data from the HTTP connection and the underlying HTTP connection can't be reused until all data from the stream has been read and the stream is closed. If these rules are not followed, the client can run out of resources by allocating too many open, but unused, HTTP connections.

Optimize HTTP performance for your application workload

The SDK provides a set of [default http configurations](#) that apply to general use cases. We recommend that customers tune HTTP configurations for their applications based on their use cases.

As a good starting point, the SDK offers a [smart configuration defaults](#) feature. This feature is available starting with version 2.17.102. You choose a mode depending on your use case, which provides sensible configuration values.

Improve SSL performance with OpenSSL for async clients

By default, the SDK's [NettyNioAsyncHttpClient](#) uses the JDK's default SSL implementation as the `SslProvider`. Our testing found that OpenSSL performs better than JDK's default implementation. The Netty community also [recommends using OpenSSL](#).

To use OpenSSL, add `netty-tcnative` to your dependencies. For configuration details, see the [Netty project documentation](#).

After you have `netty-tcnative` configured for your project, the `NettyNioAsyncHttpClient` instance will automatically select OpenSSL. Alternatively, you can set the `SslProvider` explicitly using the `NettyNioAsyncHttpClient` builder as shown in the following snippet.

```
NettyNioAsyncHttpClient.builder()  
    .sslProvider(SslProvider.OPENSSL)  
    .build();
```

Monitor application performance with SDK metrics

The SDK for Java can [collect metrics](#) for the service clients in your application. You can use these metrics to identify performance issues, review overall usage trends, review service client exceptions returned, or to dig in to understand a particular issue.

We recommend that you collect metrics, then analyze the Amazon CloudWatch Logs, in order to gain a deeper understanding of your application's performance.

Handling errors in the AWS SDK for Java 2.x

Understanding how and when the AWS SDK for Java 2.x throws exceptions is important to building high-quality applications using the SDK. The following sections describe the different cases of exceptions that are thrown by the SDK and how to handle them appropriately.

Why unchecked exceptions?

The AWS SDK for Java uses runtime (or unchecked) exceptions instead of checked exceptions for these reasons:

- To allow developers fine-grained control over the errors they want to handle without forcing them to handle exceptional cases they aren't concerned about (and making their code overly verbose)
- To prevent scalability issues inherent with checked exceptions in large applications

In general, checked exceptions work well on small scales, but can become troublesome as applications grow and become more complex.

AwsServiceException (and subclasses)

[AwsServiceException](#) is the most common exception that you'll experience when using the AWS SDK for Java. `AwsServiceException` is a subclass of the more general [SdkServiceException](#). `AwsServiceExceptions` represent an error response from an AWS service. For example, if you try to terminate an Amazon EC2 instance that doesn't exist, Amazon EC2 will return an error response and all the details of that error response will be included in the `AwsServiceException` that's thrown.

When you encounter an `AwsServiceException`, you know that your request was successfully sent to the AWS service but couldn't be successfully processed. This can be because of errors in the request's parameters or because of issues on the service side.

`AwsServiceException` provides you with information such as:

- Returned HTTP status code
- Returned AWS error code
- Detailed error message from the service in the [AwsErrorDetails](#) class
- AWS request ID for the failed request

In most cases, a service-specific subclass of `AwsServiceException` is thrown to allow developers fine-grained control over handling error cases through catch blocks. The Java SDK API reference for [AwsServiceException](#) displays the large number of `AwsServiceException` subclasses. Use the subclass links to drill down to see the granular exceptions thrown by a service.

For example, the following links to the SDK API reference show the exception hierarchies for a few common AWS services. The list of subclasses shown on each page shows the specific exceptions that your code can catch.

- [Amazon S3](#)
- [DynamoDB](#)
- [Amazon SQS](#)

To learn more about an exception, inspect the `errorCode` on the [AwsErrorDetails](#) object. You can use the `errorCode` value to look up information in the service guide API. For example if an `S3Exception` is caught and the `AwsErrorDetails#errorCode()` value is `InvalidRequest`, use the [list of error codes](#) in the Amazon S3 API Reference to see more details.

SdkClientException

[SdkClientException](#) indicates that a problem occurred inside the Java client code, either while trying to send a request to AWS or while trying to parse a response from AWS. An `SdkClientException` is generally more severe than an `SdkServiceException`, and indicates a major problem that is preventing the client from making service calls to AWS services. For example, the AWS SDK for Java throws an `SdkClientException` if no network connection is available when you try to call an operation on one of the clients.

Exceptions and retry behavior

The SDK for Java retries requests for several [client-side exceptions](#) and for [HTTP status codes](#) that it receives from AWS service responses. These errors are handled as part of the legacy `RetryMode` that service clients use by default. The Java API reference for [RetryMode](#) describes the various ways that you can configure the mode.

To customize the exceptions and HTTP status codes that trigger automatic retries, configure your service client with a [RetryPolicy](#) that adds [RetryOnExceptionsCondition](#) and [RetryOnStatusCodeCondition](#) instances.

Using paginated results in the AWS SDK for Java 2.x

Many AWS operations return paginated results when the response object is too large to return in a single response. In the AWS SDK for Java 1.0, the response contains a token you use to retrieve the next page of results. In contrast, the AWS SDK for Java 2.x has autopagination methods that make multiple service calls to get the next page of results for you automatically. You only have to write code that processes the results. Autopagination is available for both synchronous and asynchronous clients.

Note

These code snippets assume that you understand [the basics of using the SDK](#), and have configured your environment with [single sign-on access](#).

Synchronous pagination

The following examples demonstrate synchronous pagination methods to list objects in an Amazon S3 bucket.

Iterate over pages

The first example demonstrates the use of a `listRes` paginator object, a [ListObjectsV2Iterable](#) instance, to iterate through all the response pages with the `stream` method. The code streams over the response pages, converts the response stream to a stream of [S3Object](#) content, and then processes the content of the Amazon S3 object.

The following imports apply to all examples in this synchronous pagination section.

Imports

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.util.Random;

import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.paginators.ListObjectsV2Iterable;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Response;
import software.amazon.awssdk.services.s3.model.S3Object;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadRequest;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.CompletedMultipartUpload;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.CreateBucketConfiguration;
import software.amazon.awssdk.services.s3.model.UploadPartRequest;
import software.amazon.awssdk.services.s3.model.CompleteMultipartUploadRequest;
import software.amazon.awssdk.services.s3.waiters.S3Waiter;
```

```
import software.amazon.awssdk.services.s3.model.HeadBucketRequest;
import software.amazon.awssdk.services.s3.model.HeadBucketResponse;
```

```
ListObjectsV2Request listReq = ListObjectsV2Request.builder()
    .bucket(bucketName)
    .maxKeys(1)
    .build();

ListObjectsV2Iterable listRes = s3.listObjectsV2Paginator(listReq);
// Process response pages
listRes.stream()
    .flatMap(r -> r.contents().stream())
    .forEach(content -> System.out
        .println(" Key: " + content.key() + " size = " + content.size()));
```

See the [complete example](#) on GitHub.

Iterate over objects

The following examples show ways to iterate over the objects returned in the response instead of the pages of the response. The `contents` method of `ListObjectsV2Iterable` class returns an [SdkIterable](#) that provides several methods to process the underlying content elements.

Use a stream

The following snippet uses the `stream` method on the response content to iterate over the paginated item collection.

```
// Helper method to work with paginated collection of items directly.
listRes.contents().stream()
    .forEach(content -> System.out
        .println(" Key: " + content.key() + " size = " + content.size()));
```

See the [complete example](#) on GitHub.

Use a for-each loop

Since `SdkIterable` extends the `Iterable` interface, you can process the contents like any `Iterable`. The following snippet uses standard `for`-each loop to iterate through the contents of the response.

```
for (S3Object content : listRes.contents()) {
    System.out.println(" Key: " + content.key() + " size = " + content.size());
}
```

See the [complete example](#) on GitHub.

Manual pagination

If your use case requires it, manual pagination is still available. Use the next token in the response object for the subsequent requests. The following example uses a while loop.

```
ListObjectsV2Request listObjectsReqManual = ListObjectsV2Request.builder()
    .bucket(bucketName)
    .maxKeys(1)
    .build();

boolean done = false;
while (!done) {
    ListObjectsV2Response listObjResponse =
s3.listObjectsV2(listObjectsReqManual);
    for (S3Object content : listObjResponse.contents()) {
        System.out.println(content.key());
    }

    if (listObjResponse.nextContinuationToken() == null) {
        done = true;
    }

    listObjectsReqManual = listObjectsReqManual.toBuilder()
        .continuationToken(listObjResponse.nextContinuationToken())
        .build();
}
```

See the [complete example](#) on GitHub.

Asynchronous pagination

The following examples demonstrate asynchronous pagination methods to list DynamoDB tables.

Iterate over pages of table names

The following two examples use an asynchronous DynamoDB client that call the `listTablesPaginator` method with a request to get a [ListTablesPublisher](#). `ListTablesPublisher` implements two interfaces, which provides many options to process responses. We'll look at methods of each interface.

Use a Subscriber

The following code example demonstrates how to process paginated results by using the `org.reactivestreams.Publisher` interface implemented by `ListTablesPublisher`. To learn more about the reactive streams model, see the [Reactive Streams GitHub repo](#).

The following imports apply to all examples in this asynchronous pagination section.

Imports

```
import io.reactivex.rxjava3.core.Flowable;
import org.reactivestreams.Subscriber;
import org.reactivestreams.Subscription;
import reactor.core.publisher.Flux;
import software.amazon.awssdk.core.async.SdkPublisher;
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;
import software.amazon.awssdk.services.dynamodb.model.ListTablesResponse;
import software.amazon.awssdk.services.dynamodb.paginators.ListTablesPublisher;

import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
```

The following code acquires a `ListTablesPublisher` instance.

```
// Creates a default client with credentials and region loaded from the
// environment.
final DynamoDbAsyncClient asyncClient = DynamoDbAsyncClient.create();

ListTablesRequest listTablesRequest =
ListTablesRequest.builder().limit(3).build();
ListTablesPublisher publisher =
asyncClient.listTablesPaginator(listTablesRequest);
```


The following code uses an anonymous implementation of `org.reactivestreams.Subscriber` to process the results for each page.

The `onSubscribe` method calls the `Subscription.request` method to initiate requests for data from the publisher. This method must be called to start getting data from the publisher.

The subscriber's `onNext` method processes a response page by accessing all the table names and printing out each one. After the page is processed, another page is requested from the publisher. This method that is called repeatedly until all pages are retrieved.

The `onError` method is triggered if an error occurs while retrieving data. Finally, the `onComplete` method is called when all pages have been requested.

```

        // A Subscription represents a one-to-one life-cycle of a Subscriber
        subscribing
        // to a Publisher.
        publisher.subscribe(new Subscriber<ListTablesResponse>() {
            // Maintain a reference to the subscription object, which is required to
            request
            // data from the publisher.
            private Subscription subscription;

            @Override
            public void onSubscribe(Subscription s) {
                subscription = s;
                // Request method should be called to demand data. Here we request a
                single
                // page.
                subscription.request(1);
            }

            @Override
            public void onNext(ListTablesResponse response) {
                response.tableNames().forEach(System.out::println);
                // After you process the current page, call the request method to
                signal that
                // you are ready for next page.
                subscription.request(1);
            }

            @Override
            public void onError(Throwable t) {
                // Called when an error has occurred while processing the requests.

```

```
    }

    @Override
    public void onComplete() {
        // This indicates all the results are delivered and there are no more
pages
        // left.
    }
});
```

See the [complete example](#) on GitHub.

Use a Consumer

The `SdkPublisher` interface that `ListTablesPublisher` implements has a `subscribe` method that takes a `Consumer` and returns a `CompletableFuture<Void>`.

The `subscribe` method from this interface can be used for simple use cases when an `org.reactivestreams.Subscriber` might be too much overhead. As the code below consumes each page, it calls the `tableNames` method on each. The `tableNames` method returns a `java.util.List` of DynamoDB table names that are processed with the `forEach` method.

```
// Use a Consumer for simple use cases.
CompletableFuture<Void> future = publisher.subscribe(
    response -> response.tableNames()
        .forEach(System.out::println));
```

See the [complete example](#) on GitHub.

Iterate over table names

The following examples show ways to iterate over the objects returned in the response instead of the pages of the response. Similar to the synchronous Amazon S3 example previously shown with its `contents` method, the DynamoDB asynchronous result class, `ListTablesPublisher` has the `tableNames` convenience method to interact with the underlying item collection. The return type of the `tableNames` method is an [SdkPublisher](#) that can be used to request items across all pages.

Use a Subscriber

The following code acquires an `SdkPublisher` of the underlying collection of table names.

```
// Create a default client with credentials and region loaded from the
// environment.
final DynamoDbAsyncClient asyncClient = DynamoDbAsyncClient.create();

ListTablesRequest listTablesRequest =
ListTablesRequest.builder().limit(3).build();
ListTablesPublisher listTablesPublisher =
asyncClient.listTablesPaginator(listTablesRequest);
SdkPublisher<String> publisher = listTablesPublisher.tableNames();
```

The following code uses an anonymous implementation of `org.reactivestreams.Subscriber` to process the results for each page.

The subscriber's `onNext` method processes an individual element of the collection. In this case, it's a table name. After the table name is processed, another table name is requested from the publisher. This method that is called repeatedly until all table names are retrieved.

```
// Use a Subscriber.
publisher.subscribe(new Subscriber<String>() {
    private Subscription subscription;

    @Override
    public void onSubscribe(Subscription s) {
        subscription = s;
        subscription.request(1);
    }

    @Override
    public void onNext(String tableName) {
        System.out.println(tableName);
        subscription.request(1);
    }

    @Override
    public void onError(Throwable t) {
    }

    @Override
    public void onComplete() {
    }
});
```

See the [complete example](#) on GitHub.

Use a Consumer

The following example uses the `subscribe` method of `SdkPublisher` that takes a `Consumer` to process each item.

```
// Use a Consumer.  
CompletableFuture<Void> future = publisher.subscribe(System.out::println);  
future.get();
```

See the [complete example](#) on GitHub.

Use third-party library

You can use other third party libraries instead of implementing a custom subscriber. This example demonstrates the use of RxJava, but any library that implements the reactive stream interfaces can be used. See the [RxJava wiki page on GitHub](#) for more information on that library.

To use the library, add it as a dependency. If using Maven, the example shows the POM snippet to use.

POM Entry

```
<dependency>  
  <groupId>io.reactivex.rxjava3</groupId>  
  <artifactId>rxjava</artifactId>  
  <version>3.1.6</version>  
</dependency>
```

Code

```
DynamoDbAsyncClient asyncClient = DynamoDbAsyncClient.create();  
ListTablesPublisher publisher =  
    asyncClient.listTablesPaginator(ListTablesRequest.builder()  
        .build());  
  
// The Flowable class has many helper methods that work with  
// an implementation of an org.reactivestreams.Publisher.  
List<String> tables = Flowable.fromPublisher(publisher)  
    .flatMapIterable(ListTablesResponse::tableNames)
```

```
        .toList()  
        .blockingGet();  
System.out.println(tables);
```

See the [complete example](#) on GitHub.

Using waiters in the AWS SDK for Java 2.x

The waiters utility of the AWS SDK for Java 2.x enables you to validate that AWS resources are in a specified state before performing operations on those resources.

A *waiter* is an abstraction used to poll AWS resources, such as DynamoDB tables or Amazon S3 buckets, until a desired state is reached (or until a determination is made that the resource won't ever reach the desired state). Instead of writing logic to continuously poll your AWS resources, which can be cumbersome and error-prone, you can use waiters to poll a resource and have your code continue to run after the resource is ready.

Prerequisites

Before you can use waiters in a project with the AWS SDK for Java, you must complete the steps in [Setting up the AWS SDK for Java 2.x](#).

You must also configure your project dependencies (for example, in your `pom.xml` or `build.gradle` file) to use version 2.15.0 or later of the AWS SDK for Java.

For example:

```
<project>  
  <dependencyManagement>  
    <dependencies>  
      <dependency>  
        <groupId>software.amazon.awssdk</groupId>  
        <artifactId>bom</artifactId>  
        <version>2.27.21</version>  
        <type>pom</type>  
        <scope>import</scope>  
      </dependency>  
    </dependencies>  
  </dependencyManagement>  
</project>
```

Using waiters

To instantiate a waiters object, first create a service client. Set the service client's `waiter()` method as the value of the waiter object. Once the waiter instance exists, set its response options to execute the appropriate code.

Synchronous programming

The following code snippet shows how to wait for a DynamoDB table to exist and be in an **ACTIVE** state.

```
DynamoDbClient dynamo = DynamoDbClient.create();
DynamoDbWaiter waiter = dynamo.waiter();

WaiterResponse<DescribeTableResponse> waiterResponse =
    waiter.waitUntilTableExists(r -> r.tableName("myTable"));

// print out the matched response with a tableStatus of ACTIVE
waiterResponse.matched().response().ifPresent(System.out::println);
```

Asynchronous programming

The following code snippet shows how to wait for a DynamoDB table to no longer exist.

```
DynamoDbAsyncClient asyncDynamo = DynamoDbAsyncClient.create();
DynamoDbAsyncWaiter asyncWaiter = asyncDynamo.waiter();

CompletableFuture<WaiterResponse<DescribeTableResponse>> waiterResponse =
    asyncWaiter.waitUntilTableNotExists(r -> r.tableName("myTable"));

waiterResponse.whenComplete((r, t) -> {
    if (t == null) {
        // print out the matched ResourceNotFoundException
        r.matched().exception().ifPresent(System.out::println);
    }
}).join();
```

Configure waiters

You can customize the configuration for a waiter by using the `overrideConfiguration()` on its builder. For some operations, you can apply a custom configuration when you make the request.

Configure a waiter

The following code snippet shows how to override the configuration on a waiter.

```
// sync
DynamoDbWaiter waiter =
    DynamoDbWaiter.builder()
        .overrideConfiguration(b -> b.maxAttempts(10))
        .client(dynamoDbClient)
        .build();

// async
DynamoDbAsyncWaiter asyncWaiter =
    DynamoDbAsyncWaiter.builder()
        .client(dynamoDbAsyncClient)
        .overrideConfiguration(o -> o.backoffStrategy(
            FixedDelayBackoffStrategy.create(Duration.ofSeconds(2))))
        .scheduledExecutorService(Executors.newScheduledThreadPool(3))
        .build();
```

Override configuration for a specific request

The following code snippet shows how to override the configuration for a waiter on a per-request basis. Note that only some operations have customizable configurations.

```
waiter.waitUntilTableNotExists(b -> b.tableName("myTable"),
    o -> o.maxAttempts(10));

asyncWaiter.waitUntilTableExists(b -> b.tableName("myTable"),
    o -> o.waitTimeout(Duration.ofMinutes(1)));
```

Code examples

For a complete example using waiters with DynamoDB, see [CreateTable.java](#) in the AWS Code Examples Repository.

For a complete example using waiters with Amazon S3, see [S3BucketOps.java](#) in the AWS Code Examples Repository.

Troubleshooting FAQs

As you use the AWS SDK for Java 2.x in your applications, you might encounter the runtime errors listed in this topic. Use the suggestions here to help you uncover the root cause and resolve the error.

How do I fix "`java.net.SocketException: Connection reset`" or "`server failed to complete the response`" error?

A connection reset error indicates that your host, the AWS service, or any intermediary party (for example, a NAT gateway, a proxy, a load balancer) closed the connection before the request was complete. Because there are many causes, finding a solution requires that you know why the connection is being closed. The following items commonly cause a connection to be closed.

- **The connection is inactive.** This is common for streaming operations, where data is not being written to or from the wire for a period of time, so an intermediary party detects the connection as dead and closes it. To prevent this, be sure your application actively downloads or uploads data.
- **You've closed the HTTP or SDK client.** Be sure not to close resources while they are in use.
- **A misconfigured proxy.** Try to bypass any proxies that you've configured to see if it fixes the problem. If this fixes the issue, the proxy is closing your connection for some reason. Research your specific proxy to determine why it's closing the connection.

If you cannot identify the problem, try running a TCP dump for an affected connection at the client edge of your network (for example, after any proxies that you control).

If you see that the AWS endpoint is sending a TCP RST (reset), [contact the affected service](#) to see if they can determine why the reset is occurring. Be prepared to provide request IDs and timestamps of when the issue occurred. The AWS support team might also benefit from [wire logs](#) that show exactly what bytes your application is sending and receiving and when.

How do I fix "connection timeout"?

A connection timeout error indicates that your host, the AWS service, or any intermediary party (for example, a NAT gateway, a proxy, a load balancer) failed to establish a new connection with the server within the configured connection timeout. The following items describe common causes of this issue.

- **The configured connection timeout is too low.** By default, the connection timeout is 2 seconds in the AWS SDK for Java 2.x. If you set the connection timeout too low, you may get this error. The recommended connection timeout is 1 second if you make only in-region calls and 3 seconds if you make cross-region requests.
- **A misconfigured proxy.** Try to bypass any proxies that you configured to see if it fixes the problem. If this fixes the issue, the proxy is the reason for the connection timeout. Research your specific proxy to determine why that is happening

If you cannot identify the problem, try running a TCP dump for an affected connection at the client edge of your network (for example, after any proxies that you control) to investigate any network issue.

How do I fix "java.net.SocketTimeoutException: Read timed out"?

A read timed out error indicates that the JVM attempted to read data from the underlying operating system, but data was not returned within the time configured via the SDK. This error can occur if the operating system, the AWS service, or any intermediary party (for example, a NAT gateway, a proxy, a load balancers) fails to send data within the time expected by the JVM. Because there are many causes, finding a solution requires that you know why the data is not being returned.

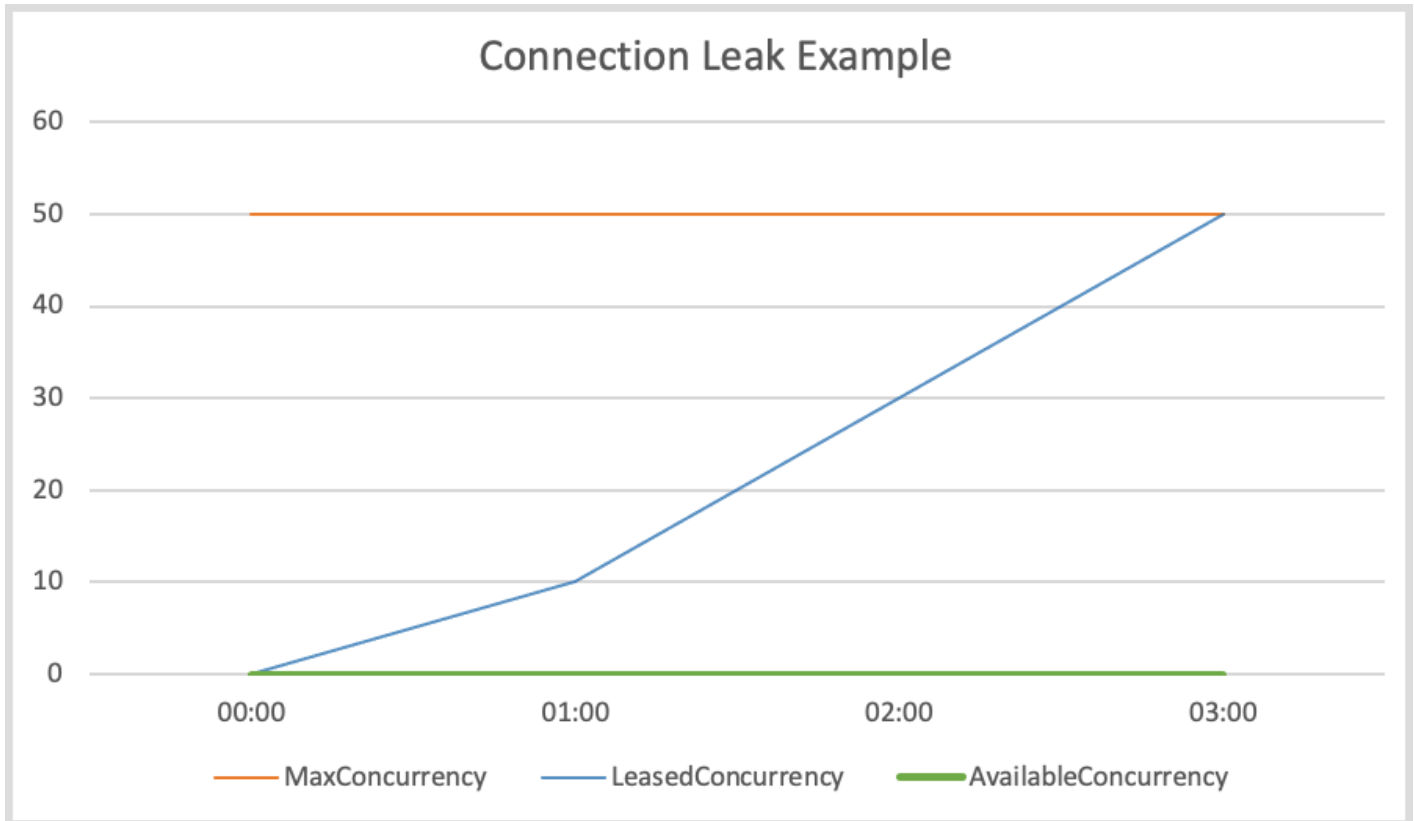
Try running a TCP dump for an affected connection at the client edge of your network (for example, after any proxies that you control).

If you see that the AWS endpoint is sending a TCP RST (reset), [contact the affected service](#). Be prepared to provide request IDs and timestamps of when the issue occurred. The AWS support team might also benefit from [wire logs](#) that show exactly what bytes your application is sending and receiving and when.

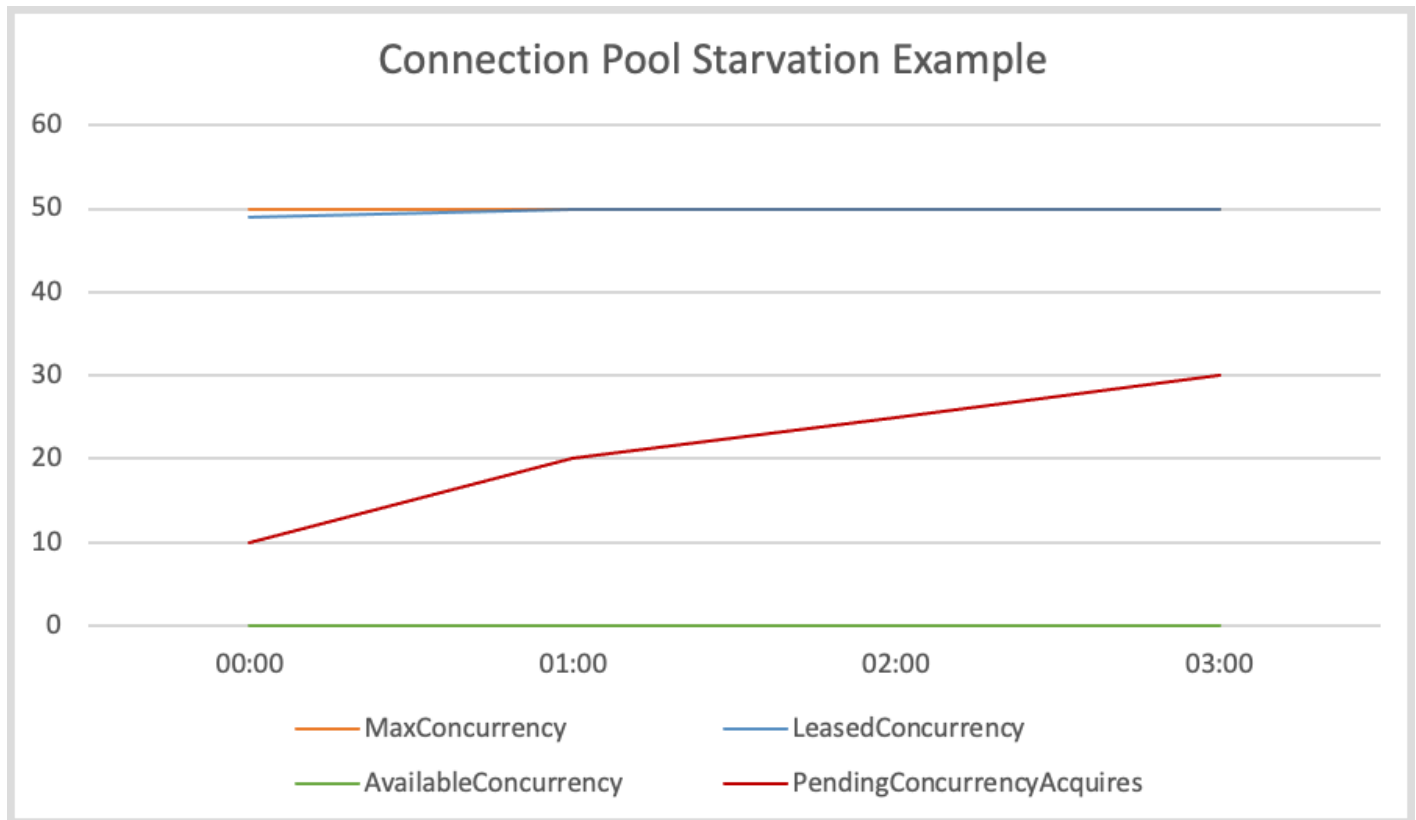
How do I fix "Unable to execute HTTP request: Timeout waiting for connection from pool" error?

This error indicates that a request cannot get a connection from the pool within the specified maximum time. To troubleshoot the issue, we recommend that you [enable SDK client-side metrics](#) to publish metrics to Amazon CloudWatch. The HTTP metrics can help narrow down the root cause. The following items describe common causes of this error.

- **Connection leak.** You can investigate this by checking `LeasedConcurrency`, `AvailableConcurrency`, and `MaxConcurrency` metrics. If `LeasedConcurrency` increases until it reaches `MaxConcurrency` but never decreases, there may be a connection leak. A common cause of a leak is because a streaming operation—such as a `S3 getObject` method—is not closed. We recommend that your application read all data from the input stream as soon as possible and [close the input stream afterwards](#). The following chart shows what SDK metrics might look like for connection leak.



- **Connection pool starvation.** This can happen if your request rate is too high and the connection pool size that has been configured cannot meet the request demand. The default connection pool size is 50, and when the connections in the pool reach the maximum, the HTTP client queues incoming requests until connections become available. The following chart shows what SDK metrics might look like for connection pool starvation.



To mitigate this issue, consider taking any of the following actions.

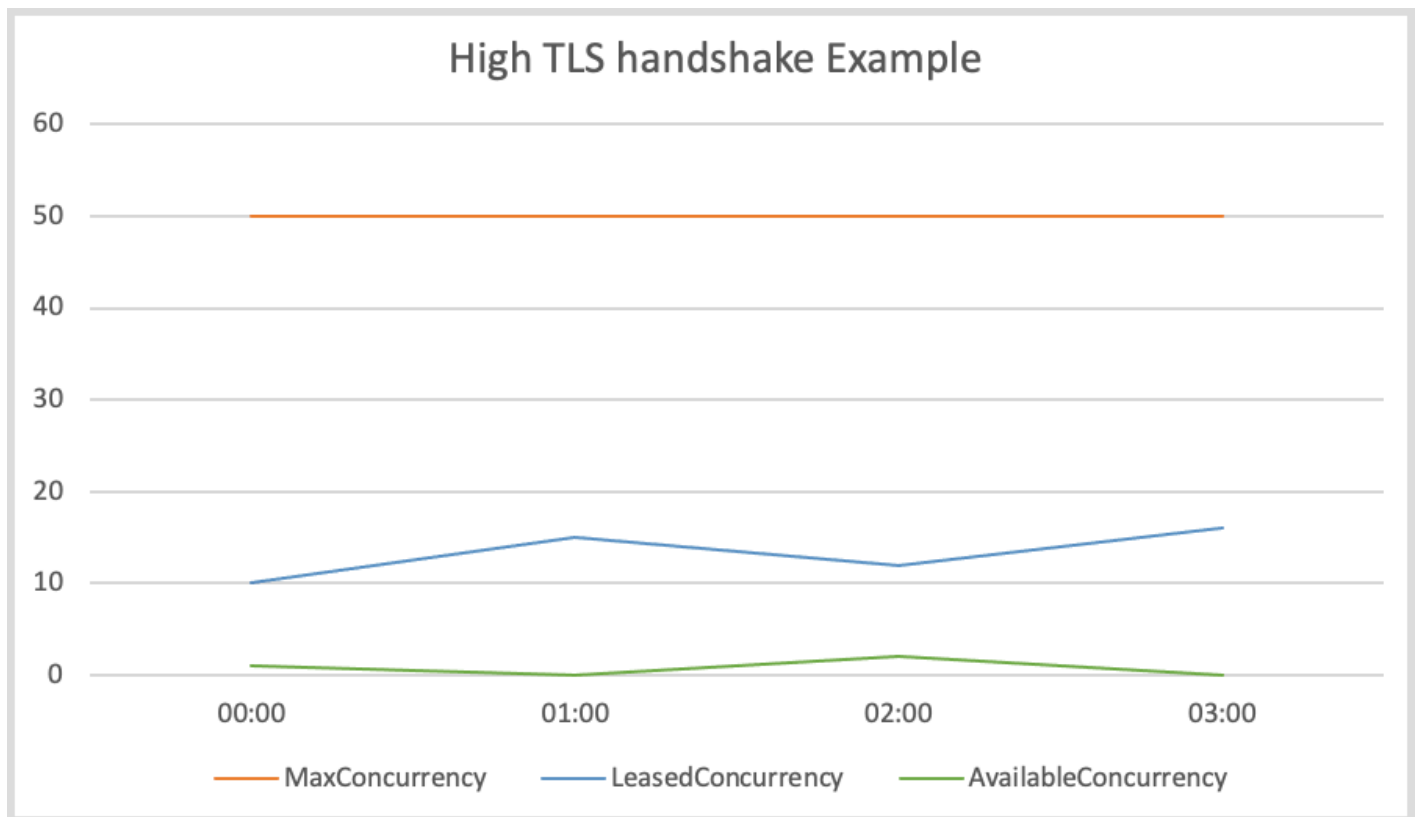
- Increase the connection pool size,
- Increase acquire timeout.
- Slow the request rate.

By increasing the maximum number of connections, client throughput can increase (unless the network interface is already fully utilized). However, you can eventually hit operation system limitations on the number of file descriptors used by the process. If you already fully use your network interface or cannot further increase your connection count, try increasing the acquire timeout. With the increase, you gain extra time for requests to acquire a connection before timing out. If the connections don't free up, the subsequent requests will still timeout.

If you are unable to fix the issue by using the first two mechanisms, slow the request rate by trying the following options.

- Smooth out your requests so that large traffic bursts don't overload the client.
- Be more efficient with calls to AWS services.
- Increase the number of hosts sending requests.

- **I/O Threads are too busy.** This only applies if you are using an asynchronous SDK client with [NettyNioAsyncHttpClient](#). If the `AvailableConcurrency` metric is not low—indicating that connections are available in the pool—but `ConcurrencyAcquireDuration` is high, it might be because I/O threads are not able to handle the requests. Be sure you are not passing `Runnable::run` as a [future completion executor](#) and performing time-consuming task in the response future completion chain since this can block an I/O thread. If that is not the case, consider increasing the number of I/O threads by using the [eventLoopGroupBuilder](#) method. For reference, the default number of I/O threads for a `NettyNioAsyncHttpClient` instance is twice the number of CPU cores of the host.
- **High TLS handshake latency.** If your `AvailableConcurrency` metric is near 0 and `LeasedConcurrency` is lower than `MaxConcurrency`, it might be because the TLS handshake latency is high. The following chart shows what SDK metrics might look like for high TLS handshake latency.



For HTTP clients offered by the Java SDK that are not based on CRT, try enabling [TLS logs](#) to troubleshoot TLS issues. For the AWS CRT-based HTTP client, try enabling [AWS CRT logs](#). If you see that the AWS endpoint seems to take a long time to perform a TLS handshake, you should [contact the affected service](#).

How do I fix a `NoClassDefFoundError`, `NoSuchMethodError` or `NoSuchFieldError`?

A `NoClassDefFoundError` indicates that a class could not be loaded at runtime. The two most common causes for this error are:

- the class does not exist in the classpath because the JAR is missing or the wrong version of the JAR is on the classpath.
- the class failed to load because its static initializer threw an exception.

Similarly, `NoSuchMethodErrors` and `NoSuchFieldErrors` typically result from a mismatched JAR version. We recommend that you perform the following steps.

1. **Check your dependencies** to make sure that you're using the *same version of all SDK jars*. The most common reason that a class, method, or field cannot be found is when you upgrade to a new client version but you continue to use an old 'shared' SDK dependency version. The new client version might attempt to use classes that exist only in newer 'shared' SDK dependencies. Try running `mvn dependency:tree` or `gradle dependencies` (for Gradle) to verify that the SDK library versions all match. To avoid this issue completely in the future, we recommend using [BOM \(Bill of Materials\)](#) to manage SDK module versions.

The following example shows you an example of mixed SDK versions.

```
[INFO] +- software.amazon.awssdk:dynamodb:jar:2.20.00:compile
[INFO] |   +- software.amazon.awssdk:aws-core:jar:2.13.19:compile
[INFO] +- software.amazon.awssdk:netty-nio-client:jar:2.20.00:compile
```

The version of `dynamodb` is 2.20.00 and the version of `aws-core` is 2.13.19. The `aws-core` artifact version should also be 2.20.00.

2. **Check statements early in your logs** to see if a class is failing to load because of a static initialization failure. The first time the class fails to load, it may throw a different, more useful exception that specifies *why* the class cannot be loaded. This potentially useful exception occurs only once, so later log statements will only report that the class is not found.
3. **Check your deployment process** to make sure that it actually deploys required JAR files along with your application. It's possible that you're building with the correct version, but the process that creates the classpath for your application is excluding a required dependency.

How do I fix a "SignatureDoesNotMatch" error or "The request signature we calculated does not match the signature you provided" error?

A `SignatureDoesNotMatch` error indicates that the signature generated by the AWS SDK for Java and the signature generated by the AWS service do not match. The following items describe potential causes.

- A proxy or intermediary party modifies the request. For example, a proxy or load balancer might modify a header, path or query string that was signed by the SDK.
- The service and SDK differ in the way they encode the request when each generates the string to sign.

To debug this issue, we recommend that you [enable debug logging](#) for the SDK. Try to reproduce the error and find the canonical request that the SDK generated. In the log, the canonical request is labeled with `AWS4 Canonical Request: ...` and the string to sign is labeled `AWS4 String to sign: ....`

If you cannot enable debugging—for example, because it's only reproducible in production—add logic to your application that logs information about the request when the error occurs. You can then use that information to try to replicate the error outside of production in an integration test with debug logging enabled.

After you have collected the canonical request and string to sign, compare them against the [AWS Signature Version 4 specification](#) to determine if there are any issues in the way the SDK generated the string to sign. If something seems wrong, you can create a [GitHub bug report](#) to the AWS SDK for Java.

If nothing appears wrong, you can compare the SDK's string to sign with the string to sign that some AWS services return as part of the failure response (Amazon S3, for example) . If this isn't available, you should [contact the affected service](#) to see what canonical request and string to sign they generated for comparison. These comparisons can help to identify intermediary parties that might have modified the request or encoding differences between the service and client.

For more background information about signing requests, see [Signing AWS API requests](#) in the AWS Identity and Access Management User Guide.

Example of a canonical request

```
PUT
/Example-Bucket/Example-Object
partNumber=19&uploadId=string
amz-sdk-invocation-id:f8c2799d-367c-f024-e8fa-6ad6d0a1afb9
amz-sdk-request:attempt=1; max=4
content-encoding:aws-chunked
content-length:51
content-type:application/octet-stream
host:xxxxx
x-amz-content-sha256:STREAMING-UNSIGNED-PAYLOAD-TRAILER
x-amz-date:20240308T034733Z
x-amz-decoded-content-length:10
x-amz-sdk-checksum-algorithm:CRC32
x-amz-trailer:x-amz-checksum-crc32
```

Example of a string to sign

```
AWS4-HMAC-SHA256
20240308T034435Z
20240308/us-east-1/s3/aws4_request
5f20a7604b1ef65dd89c333fd66736fdef9578d11a4f5d22d289597c387dc713
```

How do I fix "java.lang.IllegalStateException: Connection pool shut down" error?

This error indicates the underlying Apache HTTP connection pool was closed. The following items describe potential causes.

- **The SDK client was closed prematurely.** The SDK only closes the connection pool when the associated client is closed. Be sure not to close resources while they are in use.
- **A java.lang.Error was thrown.** Errors such as `OutOfMemoryError` cause an Apache HTTP connection pool to [shut down](#). Examine your logs for error stack traces. Also review your code for places where it catches `Throwables` or `Errors` but swallows the output that prevents the error from surfacing. If your code does not report errors, rewrite the code so information is logged. The logged information helps determine the root cause of the error.
- **You attempted to use the credentials provider returned from `DefaultCredentialsProvider#create()` after it was closed.**

[`DefaultCredentialsProvider#create`](#) returns a singleton instance, so if it's closed and your code calls the `resolveCredentials` method, the exception is thrown after cached credentials (or token) expire.

Check your code for places where the `DefaultCredentialsProvider` is closed, as shown in the following examples.

- The singleton instance is closed by calling `DefaultCredentialsProvider#close()`.

```
DefaultCredentialsProvider defaultCredentialsProvider =
    DefaultCredentialsProvider.create(); // Singleton instance returned.
AwsCredentials credentials = defaultCredentialsProvider.resolveCredentials();

// Make calls to AWS services.

defaultCredentialsProvider.close(); // Explicit close.

// Make calls to AWS services.

// After the credentials expire, either of the following calls eventually results
// in a "Connection pool shut down" exception.
credentials = defaultCredentialsProvider.resolveCredentials();
// Or
credentials = DefaultCredentialsProvider.create().resolveCredentials();
```

- Invoke `DefaultCredentialsProvider#create()` in a try-with-resources block.

```
try (DefaultCredentialsProvider defaultCredentialsProvider =
    DefaultCredentialsProvider.create()) {
    AwsCredentials credentials = defaultCredentialsProvider.resolveCredentials();

    // Make calls to AWS services.
} // After the try-with-resources block exits, the singleton
// DefaultCredentialsProvider is closed.

// Make calls to AWS services.

DefaultCredentialsProvider defaultCredentialsProvider =
    DefaultCredentialsProvider.create(); // The closed singleton instance is returned.
// If the credentials (or token) has expired, the following call results in the
// error.
AwsCredentials credentials = defaultCredentialsProvider.resolveCredentials();
```


Create a new, non-singleton instance by calling `DefaultCredentialsProvider.builder().build()` if your code has closed the singleton instance and you need to resolve credentials by using a `DefaultCredentialsProvider`.

How do I fix "Unable to load credentials from any of the providers in the chain `AwsCredentialsProviderChain`"?

This error indicates that the AWS SDK for Java 2.x could not find valid AWS credentials through any of the credential providers in the default credential provider chain. The SDK automatically searches for credentials in a specific order, and this error occurs when all providers in the chain fail to provide valid credentials.

The full error message typically looks like this (line endings and indents added to improve readability):

```
Unable to load credentials from any of the providers in the chain
AwsCredentialsProviderChain(
  credentialsProviders=[
    SystemPropertyCredentialsProvider(),
    EnvironmentVariableCredentialsProvider(),
    WebIdentityTokenCredentialsProvider(),
    ProfileCredentialsProvider(profileName=default,
profileFile=ProfileFile(sections=[])),
    ContainerCredentialsProvider(),
    InstanceProfileCredentialsProvider()
  ]) : [
  SystemPropertyCredentialsProvider(): Unable to load credentials from system
settings.
  Access key must be specified either via environment variable
(AWS_ACCESS_KEY_ID)
  or system property (aws.accessKeyId).,

  EnvironmentVariableCredentialsProvider(): Unable to load credentials from
system settings.
  Access key must be specified either via environment variable
(AWS_ACCESS_KEY_ID)
  or system property (aws.accessKeyId).,

  WebIdentityTokenCredentialsProvider(): To use web identity tokens, the 'sts'
service module
```

```
must be on the class path.,

ProfileCredentialsProvider(profileName=default,
profileFile=ProfileFile(sections=[])):
  Profile file contained no credentials for profile 'default':
ProfileFile(sections=[]),

  ContainerCredentialsProvider(): Cannot fetch credentials from container -
neither
  AWS_CONTAINER_CREDENTIALS_FULL_URI or AWS_CONTAINER_CREDENTIALS_RELATIVE_URI
environment
  variables are set.,

  InstanceProfileCredentialsProvider(): Failed to load credentials from IMDS.]
```

Common causes and solutions

Review your credential configuration

When you use the default credentials provider (by calling `ServiceClient.create()` without explicitly configuring credentials), the SDK searches for credentials in a specific order. Review [how the default credential provider chain works](#) to understand which credential sources the SDK checks and in what order.

Ensure that the credential configuration method you intend to use is properly set up in your environment:

For Amazon EC2 instances

- **Check IAM role:** Verify that an IAM role is attached to your instance.
- **Intermittent IMDS failures:** If you're experiencing intermittent failures (typically lasting a few hundred milliseconds), this usually indicates transient network issues reaching the Instance Metadata Service (IMDS).

Solutions:

- Enable [debug logging](#) to analyze the timing and frequency of failures
- Consider implementing retry logic in your application for credential-related failures
- Check for network connectivity issues between your instance and the IMDS endpoint

For container environments

Confirm that task roles (Amazon ECS) or service accounts (Amazon EKS) are configured and that required environment variables are set.

For local development

Check that environment variables, credentials files, or IAM Identity Center configuration is in place.

For web identity federation

- **Verify configuration:** Verify that the web identity token file exists and that the required environment variables are configured.
- **Missing STS module dependency:** If you see the error `To use web identity tokens, the 'sts' service module must be on the class path`, you need to add the STS module as a dependency. This is common when using Amazon EKS Pod Identity or other web identity token authentication.

Solution: Add the STS module to your project dependencies:

- ```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>sts</artifactId>
</dependency>
```

For some services, you might also need the `aws-query-protocol` dependency:

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>aws-query-protocol</artifactId>
</dependency>
```

## Network or proxy connectivity issues

If you see `Connection refused` errors in the credential provider chain, this typically indicates network connectivity problems when the SDK tries to reach AWS endpoints.

### Solutions:

- Verify proxy configuration if you're using a proxy server
- Check that your network allows outbound HTTPS connections to AWS endpoints

- Enable [debug logging](#) to see detailed connection attempts
- Test connectivity using tools like `curl` to verify network access to AWS endpoints

## Reduce SDK startup time for AWS Lambda

One of the goals of the AWS SDK for Java 2.x is to reduce the startup latency for AWS Lambda functions. The SDK contains changes that reduce startup time, which are discussed at the end of this topic.

First, this topic focuses on changes that you can make to reduce cold start times. These include making changes in your code structure and in the configuration of service clients.

### Use an AWS CRT-based HTTP client

For working with AWS Lambda, we recommend the [AwsCrtHttpClient](#) for synchronous scenarios and the [AwsCrtAsyncHttpClient](#) for asynchronous scenarios.

The [the section called “Configure AWS CRT-based HTTP clients”](#) topic in this guide describes the benefits of using the HTTP clients, how to add the dependency, and how configure their use by service clients.

### Remove unused HTTP client dependencies

Along with the explicit use of an AWS CRT-based client, you can remove other HTTP clients that the SDK brings in by default. Lambda startup time is reduced when fewer libraries need to be loaded, so you should remove any unused artifacts that the JVM needs to load.

The following snippet of a Maven `pom.xml` file shows the exclusion of the Apache-based HTTP client and the Netty-based HTTP client. (These clients aren't needed when you use an AWS CRT-based client.) This example excludes the HTTP client artifacts from the S3 client dependency and adds the `aws-crt-client` artifact to allow access to the AWS CRT-based HTTP clients.

```
<project>
 <properties>
 <aws.java.sdk.version>2.27.21</aws.java.sdk.version>
 </properties>
 <dependencyManagement>
 <dependencies>
 <dependency>
```

```

 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>${aws.java.sdk.version}</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
</dependencies>
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>aws-crt-client</artifactId>
 </dependency>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3</artifactId>
 <exclusions>
 <exclusion>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>netty-nio-client</artifactId>
 </exclusion>
 <exclusion>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>apache-client</artifactId>
 </exclusion>
 </exclusions>
 </dependency>
</dependencies>
</project>

```

### Note

Add the `<exclusions>` element to all service client dependencies in your `pom.xml` file.

## Configure service clients to shortcut lookups

### Specify a region

When you create a service client, call the `region` method on the service client builder. This shortcuts the SDK's default [Region lookup process](#) that checks several places for the AWS Region information.

To keep the Lambda code independent of the region, use the following code inside the `region` method. This code accesses the `AWS_REGION` environment variable set by the Lambda container.

```
Region.of(System.getenv(SdkSystemSetting.AWS_REGION.environmentVariable()))
```

## Use the `EnvironmentVariableCredentialsProvider`

Much like the default lookup behavior for the Region information, the SDK looks in several places for credentials. By specifying the [EnvironmentVariableCredentialsProvider](#) when you build a service client, you save time in the SDK's lookup process for credentials.

### Note

Using this credentials provider enables the code to be used in Lambda functions, but might not work on Amazon EC2 or other systems.

If you intend to use [Lambda SnapStart for Java](#) at some point, you should rely on the default credentials provider chain to lookup credentials. If you specify the `EnvironmentVariableCredentialsProvider`, the initial credentials lookup works, but when SnapStart is activated, [the Java runtime sets container credentials environment variables](#). On activation, the environment variables used by the `EnvironmentVariableCredentialsProvider`—access key environment variables—are not available to the Java SDK.

The following code snippet shows an S3 service client appropriately configured for use in a Lambda environment.

```
S3Client s3Client = S3Client.builder()

 .region(Region.of(System.getenv(SdkSystemSetting.AWS_REGION.environmentVariable())))
 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .httpClient(AwsCrtHttpClient.builder().build())
 .build();
```

## Initialize the SDK client outside of the Lambda function handler

We recommend initializing an SDK client outside of the Lambda handler method. This way, if the execution context is reused, the initialization of the service client can be skipped. By reusing the

client instance and its connections, subsequent invocations of the handler method occur more quickly.

In the following example, the `S3Client` instance is initialized in the constructor using a static factory method. If the container that is managed by the Lambda environment is reused, the initialized `S3Client` instance is reused.

```
public class App implements RequestHandler<Object, Object> {
 private final S3Client s3Client;

 public App() {
 s3Client = DependencyFactory.s3Client();
 }

 @Override
 public Object handle Request(final Object input, final Context context) {
 ListBucketResponse response = s3Client.listBuckets();
 // Process the response.
 }
}
```

## Minimize dependency injection

Dependency injection (DI) frameworks might take additional time to complete the setup process. They might also require additional dependencies, which take time to load.

If a DI framework is needed, we recommend using lightweight DI frameworks such as [Dagger](#).

## Use a Maven Archetype targeting AWS Lambda

The AWS Java SDK team has developed a [Maven Archetype](#) template to bootstrap a Lambda project with minimal startup time. You can build out a Maven project from the archetype and know that the dependencies are configured suitably for the Lambda environment.

To learn more about the archetype and work through an example deployment, see this [blog post](#).

## Consider Lambda SnapStart for Java

If your runtime requirements are compatible, AWS offers [Lambda SnapStart for Java](#). Lambda SnapStart is an infrastructure-based solution that improves startup performance for Java

functions. When you publish a new version of a function, Lambda SnapStart initializes it and takes an immutable, encrypted snapshot of the memory and disk state. SnapStart then caches the snapshot for reuse.

## Version 2.x changes that affect startup time

In addition to changes that you make to your code, version 2.x of the SDK for Java includes three primary changes that reduce startup time:

- Use of [jackson-jr](#), which is a serialization library that improves initialization time
- Use of the [java.time](#) libraries for date and time objects, which is part of the JDK
- Use of [Slf4j](#) for a logging facade

## Additional resources

The AWS Lambda Developer Guide contains a [section on best practices](#) for developing Lambda functions that is not Java specific.

For an example of building a cloud-native application in Java that uses AWS Lambda, see this [workshop content](#). The workshop discusses performance optimization and other best practices.

You can consider using static images that are compiled ahead of time to reduce startup latency. For example, you can use the SDK for Java 2.x and Maven to [build a GraalVM native image](#).

## Implement `ContentStreamProvider` in the AWS SDK for Java 2.x

`ContentStreamProvider` is an abstraction used in the AWS SDK for Java 2.x to allow multiple reads of input data. This topic explains how to implement a `ContentStreamProvider` correctly for your applications.

The SDK for Java 2.x uses the `ContentStreamProvider#newStream()` method each time it needs to read an entire stream. For this to work for the entire stream, the returned stream must always be at the start of the content and it must contain the same data.

In the following sections, we provide three approaches for how to implement this behavior correctly.



## Use `mark()` and `reset()`

In the example below, we use `mark(int)` in the constructor before reading begins to ensure that we can reset the stream back to the beginning. For each invocation of `newStream()` we reset the stream:

```
public class MyContentStreamProvider implements ContentStreamProvider {
 private InputStream contentStream;

 public MyContentStreamProvider(InputStream contentStream) {
 this.contentStream = contentStream;
 this.contentStream.mark(MAX_LEN);
 }

 @Override
 public InputStream newStream() {
 contentStream.reset();
 return contentStream;
 }
}
```

## Use buffering if `mark()` and `reset()` are not available

If your stream doesn't support `mark()` and `reset()` directly, you can still use the solution shown previously by first wrapping the stream in a `BufferedInputStream`:

```
public class MyContentStreamProvider implements ContentStreamProvider {
 private BufferedInputStream contentStream;

 public MyContentStreamProvider(InputStream contentStream) {
 this.contentStream = new BufferedInputStream(contentStream);
 this.contentStream.mark(MAX_LEN);
 }

 @Override
 public InputStream newStream() {
 contentStream.reset();
 return contentStream;
 }
}
```

## Create new streams

A simpler approach is to simply obtain a new stream to your data on each invocation and close the previous one:

```
public class MyContentStreamProvider implements ContentStreamProvider {
 private InputStream contentStream;

 @Override
 public InputStream newStream() {
 if (contentStream != null) {
 contentStream.close();
 }
 contentStream = openStream();
 return contentStream;
 }
}
```

## Set the JVM TTL for DNS name lookups

The Java virtual machine (JVM) caches DNS name lookups. When the JVM resolves a hostname to an IP address, it caches the IP address for a specified period of time, known as the *time-to-live* (TTL).

Because AWS resources use DNS name entries that occasionally change, we recommend that you configure your JVM with a TTL value of 5 seconds. This ensures that when a resource's IP address changes, your application will be able to receive and use the resource's new IP address by requerying the DNS.

On some Java configurations, the JVM default TTL is set so that it will *never* refresh DNS entries until the JVM is restarted. Thus, if the IP address for an AWS resource changes while your application is still running, it won't be able to use that resource until you *manually restart* the JVM and the cached IP information is refreshed. In this case, it's crucial to set the JVM's TTL so that it will periodically refresh its cached IP information.

## How to set the JVM TTL

To modify the JVM's TTL, set the [networkaddress.cache.ttl](#) security property value. Note that `networkaddress.cache.ttl` is a *security property*, not a system property, i.e., it cannot be set with the `-D` command-line flag.

## Option 1: Set it programmatically in your application

Call `java.security.Security.setProperty()` early in your application startup, before any AWS SDK clients are created and before any network requests are made:

```
import java.security.Security;

public class MyApplication {
 public static void main(String[] args) {
 Security.setProperty("networkaddress.cache.ttl", "5");

 // ... create SDK clients and run application
 }
}
```

## Option 2: Set it in the java.security file

Set the `networkaddress.cache.ttl` property in the `$JAVA_HOME/jre/lib/security/java.security` file for Java 8 or `$JAVA_HOME/conf/security/java.security` file for Java 11 or higher.

The following is a snippet from a `java.security` file that shows the TTL cache set to 5 seconds.

```
#
The Java-level namelookup cache policy for successful lookups:
#
any negative value: caching forever
any positive value: the number of seconds to cache an address for
zero: do not cache
#
...
networkaddress.cache.ttl=5
...
```

All applications that run on the JVM represented by the `$JAVA_HOME` environment variable use this setting.

## Option 3: Use the JDK system properties fallback (command-line)

If you cannot modify the security configuration or code, you can use JDK system properties. These act as fallbacks if no security property is defined.

- `sun.net.inetaddr.ttl` – Controls successful lookups (positive TTL)
- `sun.net.inetaddr.negative.ttl` – Controls failed lookups (negative TTL)

```
java -Dsun.net.inetaddr.ttl=5 -Dsun.net.inetaddr.negative.ttl=1 -jar myapp.jar
```

### Note

These are JDK-internal properties documented in the [Oracle Java 8 Networking Properties](#) reference as private properties that "may not be supported in future releases". Use Options 1-2 when possible.

## Work with HTTP/2 in the AWS SDK for Java

HTTP/2 is a major revision of the HTTP protocol. This new version has several enhancements to improve performance:

- Binary data encoding provides more efficient data transfer.
- Header compression reduces the overhead bytes downloaded by the client, helping get the content to the client sooner. This is especially useful for mobile clients that are already constrained on bandwidth.
- Bidirectional asynchronous communication (multiplexing) allows multiple requests and response messages between the client and AWS to be in flight at the same time over a single connection, instead of over multiple connections, which improves performance.

Developers upgrading to the latest SDKs will automatically use HTTP/2 when it's supported by the service they're working with. New programming interfaces seamlessly take advantage of HTTP/2 features and provide new ways to build applications.

The AWS SDK for Java 2.x features new APIs for event streaming that implement the HTTP/2 protocol. For examples of how to use these new APIs, see [Working with Kinesis](#).

# Calling AWS services from the AWS SDK for Java 2.x

This section provides short tutorials and guidance for how to work with select AWS services. For a complete set of examples, see the [Code Examples section](#).

## Topics

- [Work with CloudWatch](#)
- [AWS database services and AWS SDK for Java 2.x](#)
- [Work with DynamoDB](#)
- [Work with Amazon EC2](#)
- [Work with IAM](#)
- [Work with Kinesis](#)
- [Invoke, list, and delete AWS Lambda functions](#)
- [Work with Amazon S3](#)
- [Work with Amazon Simple Notification Service](#)
- [Work with Amazon Simple Queue Service](#)
- [Work with Amazon Transcribe](#)

## Work with CloudWatch

This section provides examples of programming [Amazon CloudWatch](#) by using the AWS SDK for Java 2.x.

Amazon CloudWatch monitors your Amazon Web Services (AWS) resources and the applications you run on AWS in real time. You can use CloudWatch to collect and track metrics, which are variables you can measure for your resources and applications. CloudWatch alarms send notifications or automatically make changes to the resources you are monitoring based on rules that you define.

The following examples include only the code needed to demonstrate each technique. The [complete example code is available on GitHub](#). From there, you can download a single source file or clone the repository locally to get all the examples to build and run.

## Topics

- [Get metrics from CloudWatch](#)
- [Publish custom metric data to CloudWatch](#)
- [Work with CloudWatch alarms](#)
- [Use Amazon CloudWatch Events](#)

## Get metrics from CloudWatch

### Listing metrics

To list CloudWatch metrics, create a [ListMetricsRequest](#) and call the `CloudWatchClient`'s `listMetrics` method. You can use the `ListMetricsRequest` to filter the returned metrics by namespace, metric name, or dimensions.

#### Note

A list of metrics and dimensions that are posted by AWS services can be found within the [Amazon CloudWatch Metrics and Dimensions Reference](#) in the Amazon CloudWatch User Guide.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.ListMetricsRequest;
import software.amazon.awssdk.services.cloudwatch.model.ListMetricsResponse;
import software.amazon.awssdk.services.cloudwatch.model.Metric;
```

### Code

```
public static void listMets(CloudWatchClient cw, String namespace) {

 boolean done = false;
 String nextToken = null;

 try {
 while(!done) {
```

```
ListMetricsResponse response;

if (nextToken == null) {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .build();

 response = cw.listMetrics(request);
} else {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .nextToken(nextToken)
 .build();

 response = cw.listMetrics(request);
}

for (Metric metric : response.metrics()) {
 System.out.printf(
 "Retrieved metric %s", metric.metricName());
 System.out.println();
}

if(response.nextToken() == null) {
 done = true;
} else {
 nextToken = response.nextToken();
}
}

} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

The metrics are returned in a [ListMetricsResponse](#) by calling its `getMetrics` method.

The results may be *paged*. To retrieve the next batch of results, call `nextToken` on the response object and use the token value to build a new request object. Then call the `listMetrics` method again with the new request.

See the [complete example](#) on GitHub.

## More information

- [ListMetrics](#) in the Amazon CloudWatch API Reference

## Publish custom metric data to CloudWatch

A number of AWS services publish [their own metrics](#) in namespaces beginning with " AWS ". You can also publish custom metric data using your own namespace (as long as it doesn't begin with " AWS ").

### Publish custom metric data

To publish your own metric data, call the CloudWatchClient's putMetricData method with a [PutMetricDataRequest](#). The PutMetricDataRequest must include the custom namespace to use for the data, and information about the data point itself in a [MetricDatum](#) object.

#### Note

You cannot specify a namespace that begins with " AWS ". Namespaces that begin with " AWS " are reserved for use by Amazon Web Services products.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.Dimension;
import software.amazon.awssdk.services.cloudwatch.model.MetricDatum;
import software.amazon.awssdk.services.cloudwatch.model.StandardUnit;
import software.amazon.awssdk.services.cloudwatch.model.PutMetricDataRequest;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import java.time.Instant;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
```

### Code

```
public static void putMetData(CloudWatchClient cw, Double dataPoint) {
```



```
try {
 Dimension dimension = Dimension.builder()
 .name("UNIQUE_PAGES")
 .value("URLS")
 .build();

 // Set an Instant object
 String time =
 ZonedDateTime.now(ZoneOffset.UTC).format(DateTimeFormatter.ISO_INSTANT);
 Instant instant = Instant.parse(time);

 MetricDatum datum = MetricDatum.builder()
 .metricName("PAGES_VISITED")
 .unit(StandardUnit.NONE)
 .value(dataPoint)
 .timestamp(instant)
 .dimensions(dimension).build();

 PutMetricDataRequest request = PutMetricDataRequest.builder()
 .namespace("SITE/TRAFFIC")
 .metricData(datum).build();

 cw.putMetricData(request);

} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
System.out.printf("Successfully put data point %f", dataPoint);
}
```

See the [complete example](#) on GitHub.

## More information

- [Use Amazon CloudWatch Metrics](#) in the Amazon CloudWatch User Guide.
- [AWS Namespaces](#) in the Amazon CloudWatch User Guide.
- [PutMetricData](#) in the Amazon CloudWatch API Reference.

# Work with CloudWatch alarms

## Create an alarm

To create an alarm based on a CloudWatch metric, call the `CloudWatchClient`'s `putMetricAlarm` method with a [PutMetricAlarmRequest](#) filled with the alarm conditions.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.Dimension;
import software.amazon.awssdk.services.cloudwatch.model.PutMetricAlarmRequest;
import software.amazon.awssdk.services.cloudwatch.model.ComparisonOperator;
import software.amazon.awssdk.services.cloudwatch.model.Statistic;
import software.amazon.awssdk.services.cloudwatch.model.StandardUnit;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
```

### Code

```
public static void putMetricAlarm(CloudWatchClient cw, String alarmName, String
instanceId) {

 try {
 Dimension dimension = Dimension.builder()
 .name("InstanceId")
 .value(instanceId).build();

 PutMetricAlarmRequest request = PutMetricAlarmRequest.builder()
 .alarmName(alarmName)
 .comparisonOperator(
 ComparisonOperator.GREATER_THAN_THRESHOLD)
 .evaluationPeriods(1)
 .metricName("CPUUtilization")
 .namespace("AWS/EC2")
 .period(60)
 .statistic(Statistic.AVERAGE)
 .threshold(70.0)
 .actionsEnabled(false)
 .alarmDescription(
 "Alarm when server CPU utilization exceeds 70%")
 .unit(StandardUnit.SECONDS)
 .dimensions(dimension)
```

```
 .build();

 cw.putMetricAlarm(request);
 System.out.printf(
 "Successfully created alarm with name %s", alarmName);

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## List alarms

To list the CloudWatch alarms that you have created, call the `CloudWatchClient`'s `describeAlarms` method with a [DescribeAlarmsRequest](#) that you can use to set options for the result.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.DescribeAlarmsRequest;
import software.amazon.awssdk.services.cloudwatch.model.DescribeAlarmsResponse;
import software.amazon.awssdk.services.cloudwatch.model.MetricAlarm;
```

### Code

```
public static void desCWAAlarms(CloudWatchClient cw) {

 try {

 boolean done = false;
 String newToken = null;

 while(!done) {
 DescribeAlarmsResponse response;

 if (newToken == null) {
```

```
 DescribeAlarmsRequest request =
DescribeAlarmsRequest.builder().build();
 response = cw.describeAlarms(request);
 } else {
 DescribeAlarmsRequest request = DescribeAlarmsRequest.builder()
 .nextToken(newToken)
 .build();
 response = cw.describeAlarms(request);
 }

 for(MetricAlarm alarm : response.metricAlarms()) {
 System.out.printf("\n Retrieved alarm %s", alarm.alarmName());
 }

 if(response.nextToken() == null) {
 done = true;
 } else {
 newToken = response.nextToken();
 }
}

} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
System.out.printf("Done");
}
```

The list of alarms can be obtained by calling `MetricAlarms` on the [DescribeAlarmsResponse](#) that is returned by `describeAlarms`.

The results may be *paged*. To retrieve the next batch of results, call `nextToken` on the response object and use the token value to build a new request object. Then call the `describeAlarms` method again with the new request.

#### Note

You can also retrieve alarms for a specific metric by using the `CloudWatchClient`'s `describeAlarmsForMetric` method. Its use is similar to `describeAlarms`.

See the [complete example](#) on GitHub.

## Delete alarms

To delete CloudWatch alarms, call the `CloudWatchClient`'s `deleteAlarms` method with a [DeleteAlarmsRequest](#) containing one or more names of alarms that you want to delete.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.DeleteAlarmsRequest;
```

### Code

```
public static void deleteCWAlarm(CloudWatchClient cw, String alarmName) {

 try {
 DeleteAlarmsRequest request = DeleteAlarmsRequest.builder()
 .alarmNames(alarmName)
 .build();

 cw.deleteAlarms(request);
 System.out.printf("Successfully deleted alarm %s", alarmName);

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [Using Amazon CloudWatch alarms](#) in the Amazon CloudWatch User Guide
- [PutMetricAlarm](#) in the Amazon CloudWatch API Reference
- [DescribeAlarms](#) in the Amazon CloudWatch API Reference
- [DeleteAlarms](#) in the Amazon CloudWatch API Reference

## Use Amazon CloudWatch Events

CloudWatch Events delivers a near real-time stream of system events that describe changes in AWS resources to Amazon EC2 instances, Lambda functions, Kinesis streams, Amazon ECS tasks, Step Functions state machines, Amazon SNS topics, Amazon SQS queues, or built-in targets. You can match events and route them to one or more target functions or streams by using simple rules.

Amazon EventBridge is the [evolution](#) of CloudWatch Events. Both services use the same API, so you can continue using the [CloudWatch Events client](#) provided by the SDK or migrate to the SDK for Java's [EventBridge client](#) for CloudWatch Events functionality. CloudWatch Events [User Guide documentation](#) and [API reference](#) are now available through the EventBridge documentation sites.

### Add events

To add custom CloudWatch events, call the `CloudWatchEventsClient`'s `putEvents` method with a [PutEventsRequest](#) object that contains one or more [PutEventsRequestEntry](#) objects that provide details about each event. You can specify several parameters for the entry such as the source and type of the event, resources associated with the event, and so on.

#### Note

You can specify a maximum of 10 events per call to `putEvents`.

### Imports

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutEventsRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.PutEventsRequestEntry;
```

### Code

```
public static void putCWEvents(CloudWatchEventsClient cwe, String resourceArn) {

 try {

 final String EVENT_DETAILS =
 "{ \"key1\": \"value1\", \"key2\": \"value2\" }";

 PutEventsRequestEntry requestEntry = PutEventsRequestEntry.builder()
```

```

 .detail(EVENT_DETAILS)
 .detailType("sampleSubmitted")
 .resources(resourceArn)
 .source("aws-sdk-java-cloudwatch-example")
 .build();

 PutEventsRequest request = PutEventsRequest.builder()
 .entries(requestEntry)
 .build();

 cwe.putEvents(request);
 System.out.println("Successfully put CloudWatch event");

} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}

```

See the [complete example](#) on GitHub.

## Add rules

To create or update a rule, call the `CloudWatchEventsClient`'s `putRule` method with a [PutRuleRequest](#) with the name of the rule and optional parameters such as the [event pattern](#), IAM role to associate with the rule, and a [scheduling expression](#) that describes how often the rule is run.

## Imports

```

import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutRuleRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.PutRuleResponse;
import software.amazon.awssdk.services.cloudwatchevents.model.RuleState;

```

## Code

```

 public static void putCWRule(CloudWatchEventsClient cwe, String ruleName, String
 roleArn) {

 try {
 PutRuleRequest request = PutRuleRequest.builder()

```

```
 .name(ruleName)
 .roleArn(roleArn)
 .scheduleExpression("rate(5 minutes)")
 .state(RuleState.ENABLED)
 .build();

PutRuleResponse response = cwe.putRule(request);
System.out.printf(
 "Successfully created CloudWatch events rule %s with arn %s",
 roleArn, response.ruleArn());
} catch (
 CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

See the [complete example](#) on GitHub.

## Add targets

Targets are the resources that are invoked when a rule is triggered. Example targets include Amazon EC2 instances, Lambda functions, Kinesis streams, Amazon ECS tasks, Step Functions state machines, and built-in targets.

To add a target to a rule, call the `CloudWatchEventsClient`'s `putTargets` method with a [PutTargetsRequest](#) containing the rule to update and a list of targets to add to the rule.

## Imports

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutTargetsRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.PutTargetsResponse;
import software.amazon.awssdk.services.cloudwatchevents.model.Target;
```

## Code

```
public static void putCWTargets(CloudWatchEventsClient cwe, String ruleName, String
functionArn, String targetId) {

 try {
 Target target = Target.builder()
```



```
 .arn(functionArn)
 .id(targetId)
 .build();

 PutTargetsRequest request = PutTargetsRequest.builder()
 .targets(target)
 .rule(ruleName)
 .build();

 PutTargetsResponse response = cwe.putTargets(request);
 System.out.printf(
 "Successfully created CloudWatch events target for rule %s",
 ruleName);
} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

See the [complete example](#) on GitHub.

## More information

- [Adding Events with PutEvents](#) in the Amazon EventBridge User Guide
- [Schedule Expressions for Rules](#) in the Amazon EventBridge User Guide
- [Event Types for CloudWatch Events](#) in the Amazon EventBridge User Guide
- [Event Patterns](#) in the Amazon EventBridge User Guide
- [PutEvents](#) in the Amazon EventBridge API Reference
- [PutTargets](#) in the Amazon EventBridge API Reference
- [PutRule](#) in the Amazon EventBridge API Reference

## AWS database services and AWS SDK for Java 2.x

AWS offers several database types: relational, key-value, in-memory, document, and [several others](#). The SDK for Java 2.x support varies depending the nature of the database service in AWS.

Some database services, for example [Amazon DynamoDB](#) service, have web service APIs to manage the AWS resource (database) as well as web service APIs to interact with the data. In the SDK for Java 2.x these types of services have dedicated service clients, for example [DynamoDBClient](#).

Other database services have web service APIs that interact with the resource, such the [Amazon DocumentDB](#) API (for cluster, instance and resource management), but do not have a web service API for working with the data. The SDK for Java 2.x has a corresponding [DocDbClient](#) interface for working with the resource. However, you need another Java API, such as [MongoDB for Java](#) to work with the data.

Use the examples below to learn how you use the SDK for Java 2.x service clients with the different types of databases.

## Amazon DynamoDB examples

### Working with the data

SDK service client: [DynamoDbClient](#)

Example: [React/Spring REST application using DynamoDB](#)

Examples: [Several DynamoDB examples](#)

SDK service client: [DynamoDbEnhancedClient](#)

Example: [React/Spring REST application using DynamoDB](#)

Examples: [Several DynamoDB examples](#)  
(names starting with 'Enhanced')

### Working with the database

SDK service client: [DynamoDbClient](#)

Examples: [CreateTable](#), [ListTables](#), [DeleteTable](#)

See [additional DynamoDB examples](#) in the guided code examples section of this guide.

## Amazon RDS examples

Working with the data	Working with the database
Non-SDK API: JDBC, database-specific SQL flavor; your code manages database connections or a connection pool.	SDK service client: <a href="#">RdsClient</a>

Working with the data	Working with the database
Example: <a href="#">React/Spring REST application using MySQL</a>	Examples: <a href="#">Several RdsClient examples</a>

## Amazon Redshift examples

Working with the data	Working with the database
SDK service client: <a href="#">RedshiftDataClient</a>	SDK service client: <a href="#">RedshiftClient</a>
Examples: <a href="#">Several RedshiftDataClient examples</a>	Examples: <a href="#">Several RedshiftClient examples</a>
Example: <a href="#">React/Spring REST application using RedshiftDataClient</a>	

## Amazon Aurora Serverless v2 examples

Working with the data	Working with the database
SDK service client: <a href="#">RdsDataClient</a>	SDK service client: <a href="#">RdsClient</a>
Example: <a href="#">React/Spring REST application using RdsDataClient</a>	Examples: <a href="#">Several RdsClient examples</a>

## Amazon DocumentDB examples

Working with the data	Working with the database
Non-SDK API: MongoDB-specific Java library (for example <a href="#">MongoDB for Java</a> ); your code manages database connections or a connection pool.	SDK service client: <a href="#">DocDbClient</a>

Working with the data	Working with the database
Examples: <a href="#">DocumentDB (Mongo) Developer Guide</a> (select 'Java' tab)	

## Work with DynamoDB

[DynamoDB](#) is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. This section shows you how to work with DynamoDB using the AWS SDK for Java 2.x.

### Choose your DynamoDB client

The SDK provides two main approaches for working with DynamoDB:

#### Low-level client (`DynamoDbClient`)

Provides direct access to DynamoDB operations with full control over requests and responses. Use this client when you need fine-grained control or work with dynamic schemas.

#### Enhanced client (`DynamoDbEnhancedClient`)

Offers object-oriented programming with automatic mapping between Java objects and DynamoDB items. Also provides document-oriented capabilities for working with JSON-like data that doesn't follow a fixed schema. Use this client when working with well-defined data models or document-type data.

### Configure DynamoDB clients

Before working with DynamoDB, configure your client for optimal performance and reliability.

### Understand DynamoDB retry behavior

DynamoDB clients use a default maximum retry count of 8, which is higher than other AWS service clients. This higher retry count helps handle DynamoDB's distributed nature and temporary capacity limitations. For more information about retry strategies, see [the section called "Retries"](#).

## Optimize performance with account-based endpoints

DynamoDB offers [AWS account-based endpoints](#) that improve performance by using your AWS account ID to streamline request routing.

To use this feature, you need version 2.28.4 or greater of the AWS SDK for Java 2.x. You can find the latest version in the [Maven central repository](#). Supported SDK versions automatically use the new endpoints.

To opt out of account-based routing, choose one of these options:

- Configure a DynamoDB service client with `AccountIdEndpointMode` set to `DISABLED`.
- Set an environment variable.
- Set a JVM system property.
- Update the shared AWS config file setting.

The following example shows how to disable account-based routing by configuring a DynamoDB service client:

```
DynamoDbClient.builder()
 .accountIdEndpointMode(AccountIdEndpointMode.DISABLED)
 .build();
```

For more information about the other configuration options, see [Account-based endpoints](#) in the AWS SDKs and Tools Reference Guide.

## What's covered in this topic

The following sections show you how to work with DynamoDB:

- [the section called “Work with tables in DynamoDB”](#) - Create, describe, update, and delete tables
- [the section called “Work with items in DynamoDB”](#) - Add, retrieve, and update individual items
- [the section called “ Map objects to DynamoDB items”](#) - Use object mapping and document-oriented data with the Enhanced Client

For additional DynamoDB code examples, see [DynamoDB code examples](#) in the AWS Code Examples Library.

## Work with tables in DynamoDB

Tables are the containers for all items in a DynamoDB database. Before you can add or remove data from DynamoDB, you must create a table.

For each table, you must define:

- A table *name* that is unique for your account and Region.
- A *primary key* for which every value must be unique; no two items in your table can have the same primary key value.

A primary key can be *simple*, consisting of a single partition (HASH) key, or *composite*, consisting of a partition and a sort (RANGE) key.

Each key value has an associated *data type*, enumerated by the [ScalarAttributeType](#) class. The key value can be binary (B), numeric (N), or a string (S). For more information, see [Naming Rules and Data Types](#) in the Amazon DynamoDB Developer Guide.

- *Provisioned throughput* are values that define the number of reserved read/write capacity units for the table.

### Note

[Amazon DynamoDB pricing](#) is based on the provisioned throughput values that you set on your tables, so reserve only as much capacity as you think you'll need for your table.

Provisioned throughput for a table can be modified at any time, so you can adjust capacity as your needs change.

## Create a table

Use the `DynamoDbClient`'s `createTable` method to create a new DynamoDB table. You need to construct table attributes and a table schema, both of which are used to identify the primary key of your table. You must also supply initial provisioned throughput values and a table name.

### Note

If a table with the name you chose already exists, a [DynamoDbException](#) is thrown.

## Create a table with a simple primary key

This code creates a table with one attribute that is the table's simple primary key. The example uses [AttributeDefinition](#) and [KeySchemaElement](#) objects for the [CreateTableRequest](#).

### Imports

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.CreateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.CreateTableResponse;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.waiters.DynamoDbWaiter;
```

### Code

```
public static String createTable(DynamoDbClient ddb, String tableName, String key)
{
 DynamoDbWaiter dbWaiter = ddb.waiter();
 CreateTableRequest request = CreateTableRequest.builder()
 .attributeDefinitions(AttributeDefinition.builder()
 .attributeName(key)
 .attributeType(ScalarAttributeType.S)
 .build())
 .keySchema(KeySchemaElement.builder()
 .attributeName(key)
 .keyType(KeyType.HASH)
 .build())
 .provisionedThroughput(ProvisionedThroughput.builder()
 .readCapacityUnits(new Long(10))
 .writeCapacityUnits(new Long(10))
 .build())
 .tableName(tableName)
```

```
 .build();

String newTable = "";
try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created
 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);

 newTable = response.tableDescription().tableName();
 return newTable;

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
return "";
}
```

See the [complete example](#) on GitHub.

## Create a table with a composite primary key

The following example creates a table with two attributes. Both attributes are used for the composite primary key.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.CreateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.CreateTableResponse;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
```



## Code

```

public static String createTableComKey(DynamoDbClient ddb, String tableName) {
 CreateTableRequest request = CreateTableRequest.builder()
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("Language")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("Greeting")
 .attributeType(ScalarAttributeType.S)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("Language")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("Greeting")
 .keyType(KeyType.RANGE)
 .build())
 .provisionedThroughput(
 ProvisionedThroughput.builder()
 .readCapacityUnits(new Long(10))
 .writeCapacityUnits(new Long(10)).build())
 .tableName(tableName)
 .build();

 String tableId = "";

 try {
 CreateTableResponse result = ddb.createTable(request);
 tableId = result.tableDescription().tableId();
 return tableId;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}

```

See the [complete example](#) on GitHub.

## List tables

You can list the tables in a particular Region by calling the `DynamoDbClient`'s `listTables` method.

### Note

If the named table doesn't exist for your account and Region, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ListTablesResponse;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import java.util.List;
```

## Code

```
public static void listAllTables(DynamoDbClient ddb){

 boolean moreTables = true;
 String lastName = null;

 while(moreTables) {
 try {
 ListTablesResponse response = null;
 if (lastName == null) {
 ListTablesRequest request = ListTablesRequest.builder().build();
 response = ddb.listTables(request);
 } else {
 ListTablesRequest request = ListTablesRequest.builder()
 .exclusiveStartTableName(lastName).build();
 response = ddb.listTables(request);
 }

 List<String> tableNames = response.tableNames();

 if (tableNames.size() > 0) {
```

```

 for (String curName : tableNames) {
 System.out.format("* %s\n", curName);
 }
 } else {
 System.out.println("No tables found!");
 System.exit(0);
 }

 lastName = response.lastEvaluatedTableName();
 if (lastName == null) {
 moreTables = false;
 }
} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
}
System.out.println("\nDone!");
}

```

By default, up to 100 tables are returned per call—use `lastEvaluatedTableName` on the returned [ListTablesResponse](#) object to get the last table that was evaluated. You can use this value to start the listing after the last returned value of the previous listing.

See the [complete example](#) on GitHub.

## Describe (get information about) a table

Use the `DynamoDbClient`'s `describeTable` method to get information about a table.

### Note

If the named table doesn't exist for your account and Region, a [ResourceNotFoundException](#) is thrown.

## Imports

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;

```

```
import software.amazon.awssdk.services.dynamodb.model.DescribeTableRequest;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughputDescription;
import software.amazon.awssdk.services.dynamodb.model.TableDescription;
import java.util.List;
```

## Code

```
public static void describeDynamoDBTable(DynamoDbClient ddb,String tableName) {

 DescribeTableRequest request = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 TableDescription tableInfo =
 ddb.describeTable(request).table();

 if (tableInfo != null) {
 System.out.format("Table name : %s\n",
 tableInfo.tableName());
 System.out.format("Table ARN : %s\n",
 tableInfo.tableArn());
 System.out.format("Status : %s\n",
 tableInfo.tableStatus());
 System.out.format("Item count : %d\n",
 tableInfo.itemCount().longValue());
 System.out.format("Size (bytes): %d\n",
 tableInfo.tableSizeBytes().longValue());

 ProvisionedThroughputDescription throughputInfo =
 tableInfo.provisionedThroughput();
 System.out.println("Throughput");
 System.out.format(" Read Capacity : %d\n",
 throughputInfo.readCapacityUnits().longValue());
 System.out.format(" Write Capacity: %d\n",
 throughputInfo.writeCapacityUnits().longValue());

 List<AttributeDefinition> attributes =
 tableInfo.attributeDefinitions();
 System.out.println("Attributes");

 for (AttributeDefinition a : attributes) {
 System.out.format(" %s (%s)\n",
```

```
 a.attributeName(), a.attributeType());
 }
 }
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("\nDone!");
}
```

See the [complete example](#) on GitHub.

## Modify (update) a table

You can modify your table's provisioned throughput values at any time by calling the `DynamoDbClient`'s `updateTable` method.

### Note

If the named table doesn't exist for your account and Region, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.UpdateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
```

## Code

```
public static void updateDynamoDBTable(DynamoDbClient ddb,
 String tableName,
 Long readCapacity,
 Long writeCapacity) {

 System.out.format(
 "Updating %s with new provisioned throughput values\n",
 tableName);
}
```

```
System.out.format("Read capacity : %d\n", readCapacity);
System.out.format("Write capacity : %d\n", writeCapacity);

ProvisionedThroughput tableThroughput = ProvisionedThroughput.builder()
 .readCapacityUnits(readCapacity)
 .writeCapacityUnits(writeCapacity)
 .build();

UpdateTableRequest request = UpdateTableRequest.builder()
 .provisionedThroughput(tableThroughput)
 .tableName(tableName)
 .build();

try {
 ddb.updateTable(request);
} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

System.out.println("Done!");
}
```

See the [complete example](#) on GitHub.

## Delete a table

To delete a table, call `DynamoDbClient`'s `deleteTable` method and provide the table's name.

### Note

If the named table doesn't exist for your account and Region, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DeleteTableRequest;
```

## Code

```
public static void deleteDynamoDBTable(DynamoDbClient ddb, String tableName) {

 DeleteTableRequest request = DeleteTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 ddb.deleteTable(request);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println(tableName + " was successfully deleted!");
}
```

See the [complete example](#) on GitHub.

## More information

- [Guidelines for Working with Tables](#) in the Amazon DynamoDB Developer Guide
- [Working with Tables in DynamoDB](#) in the Amazon DynamoDB Developer Guide

## Work with items in DynamoDB

In DynamoDB, an item is a collection of *attributes*, each of which has a *name* and a *value*. An attribute value can be a scalar, set, or document type. For more information, see [Naming Rules and Data Types](#) in the Amazon DynamoDB Developer Guide.

### Retrieve (get) an item from a table

Call the `DynamoDbClient`'s `getItem` method and pass it a [GetItemRequest](#) object with the table name and primary key value of the item you want. It returns a [GetItemResponse](#) object with all of the attributes for that item. You can specify one or more [projection expressions](#) in the `GetItemRequest` to retrieve specific attributes.

You can use the returned `GetItemResponse` object's `item()` method to retrieve a [Map](#) of key (String) and value ([AttributeValue](#)) pairs that are associated with the item.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;
```

## Code

```
public static void getDynamoDBItem(DynamoDbClient ddb,String tableName,String
key,String keyVal) {

 HashMap<String,AttributeValue> keyToGet = new HashMap<String,AttributeValue>();

 keyToGet.put(key, AttributeValue.builder()
 .s(keyVal).build());

 GetItemRequest request = GetItemRequest.builder()
 .key(keyToGet)
 .tableName(tableName)
 .build();

 try {
 Map<String,AttributeValue> returnedItem = ddb.getItem(request).item();

 if (returnedItem != null) {
 Set<String> keys = returnedItem.keySet();
 System.out.println("Amazon DynamoDB table attributes: \n");

 for (String key1 : keys) {
 System.out.format("%s: %s\n", key1,
returnedItem.get(key1).toString());
 }
 } else {
 System.out.format("No item found with the key %s!\n", key);
 }
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```



```
}
}
```

See the [complete example](#) on GitHub.

## Retrieve (get) an item from a table using the asynchronous client

Invoke the `getItem` method of the `DynamoDbAsyncClient` and pass it a [GetItemRequest](#) object with the table name and primary key value of the item you want.

You can return a [Collection](#) instance with all of the attributes for that item (refer to the following example).

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
```

### Code

```
public static void getItem(DynamoDbAsyncClient client, String tableName, String
key, String keyVal) {

 HashMap<String, AttributeValue> keyToGet =
 new HashMap<String, AttributeValue>();

 keyToGet.put(key, AttributeValue.builder()
 .s(keyVal).build());

 try {

 // Create a GetItemRequest instance
 GetItemRequest request = GetItemRequest.builder()
 .key(keyToGet)
 .tableName(tableName)
 .build();
```

```
// Invoke the DynamoDbAsyncClient object's getItem
java.util.Collection<AttributeValue> returnedItem =
client.getItem(request).join().item().values();

// Convert Set to Map
Map<String, AttributeValue> map =
returnedItem.stream().collect(Collectors.toMap(AttributeValue::s, s->s));
Set<String> keys = map.keySet();
for (String sinKey : keys) {
 System.out.format("%s: %s\n", sinKey, map.get(sinKey).toString());
}

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
```

See the [complete example](#) on GitHub.

## Add a new item to a table

Create a [Map](#) of key-value pairs that represent the item's attributes. These must include values for the table's primary key fields. If the item identified by the primary key already exists, its fields are *updated* by the request.

### Note

If the named table doesn't exist for your account and region, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import java.util.HashMap;
```

## Code

```
public static void putItemInTable(DynamoDbClient ddb,
 String tableName,
 String key,
 String keyVal,
 String albumTitle,
 String albumTitleValue,
 String awards,
 String awardVal,
 String songTitle,
 String songTitleVal){

 HashMap<String,AttributeValue> itemValues = new
HashMap<String,AttributeValue>();

 // Add all content to the table
 itemValues.put(key, AttributeValue.builder().s(keyVal).build());
 itemValues.put(songTitle, AttributeValue.builder().s(songTitleVal).build());
 itemValues.put(albumTitle,
AttributeValue.builder().s(albumTitleValue).build());
 itemValues.put(awards, AttributeValue.builder().s(awardVal).build());

 PutItemRequest request = PutItemRequest.builder()
 .tableName(tableName)
 .item(itemValues)
 .build();

 try {
 ddb.putItem(request);
 System.out.println(tableName + " was successfully updated");

 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be found.
\n", tableName);
 System.err.println("Be sure that it exists and that you've typed its name
correctly!");
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Update an existing item in a table

You can update an attribute for an item that already exists in a table by using the `DynamoDBClient`'s `updateItem` method, providing a table name, primary key value, and a map of fields to update.

### Note

If the named table doesn't exist for your account and region, or if the item identified by the primary key you passed in doesn't exist, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.AttributeAction;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.AttributeValueUpdate;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import java.util.HashMap;
```

## Code

```
public static void updateTableItem(DynamoDbClient ddb,
 String tableName,
 String key,
 String keyVal,
 String name,
 String updateVal){

 HashMap<String,AttributeValue> itemKey = new HashMap<String,AttributeValue>();

 itemKey.put(key, AttributeValue.builder().s(keyVal).build());

 HashMap<String,AttributeValueUpdate> updatedValues =
 new HashMap<String,AttributeValueUpdate>();
```

```
// Update the column specified by name with updatedVal
updatedValues.put(name, AttributeValueUpdate.builder()
 .value(AttributeValue.builder().s(updateVal).build())
 .action(AttributeAction.PUT)
 .build());

UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(itemKey)
 .attributeUpdates(updatedValues)
 .build();

try {
 ddb.updateItem(request);
} catch (ResourceNotFoundException e) {
 System.err.println(e.getMessage());
 System.exit(1);
} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

System.out.println("Done!");
}
```

See the [complete example](#) on GitHub.

## Delete an existing item in a table

You can delete an item that exists in a table by using the `DynamoDbClient`'s `deleteItem` method and providing a table name as well as the primary key value.

### Note

If the named table doesn't exist for your account and region, or if the item identified by the primary key you passed in doesn't exist, a [ResourceNotFoundException](#) is thrown.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
```

```
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DeleteItemRequest;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import java.util.HashMap;
```

## Code

```
public static void deleteDynamoDBItem(DynamoDbClient ddb, String tableName, String
key, String keyVal) {

 HashMap<String,AttributeValue> keyToGet =
 new HashMap<String,AttributeValue>();

 keyToGet.put(key, AttributeValue.builder()
 .s(keyVal)
 .build());

 DeleteItemRequest deleteReq = DeleteItemRequest.builder()
 .tableName(tableName)
 .key(keyToGet)
 .build();

 try {
 ddb.deleteItem(deleteReq);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [Guidelines for Working with Items](#) in the Amazon DynamoDB Developer Guide
- [Working with Items in DynamoDB](#) in the Amazon DynamoDB Developer Guide

## Map Java objects to DynamoDB items with the AWS SDK for Java 2.x

The [DynamoDB Enhanced Client API](#) is a high-level library that is the successor to the `DynamoDBMapper` class of in the SDK for Java v1.x. It offers a straightforward way to map

client-side classes to DynamoDB tables. You define the relationships between tables and their corresponding data classes in your code. After you define those relationships, you can intuitively perform various create, read, update, or delete (CRUD) operations on tables or items in DynamoDB.

The DynamoDB Enhanced Client API also includes the [Enhanced Document API](#) that enables you to work with document-type items that do not follow a defined schema.

**The DynamoDB Enhanced Client API is discussed in the following topics.**

- [Get Started using the DynamoDB Enhanced Client API](#)
- [Learn the basics of the DynamoDB Enhanced Client API](#)
- [Use advanced mapping features](#)
- [Work with JSON documents with the Enhanced Document API for DynamoDB](#)
- [Use extensions to customize DynamoDB Enhanced Client operations](#)
- [Use the DynamoDB Enhanced Client API asynchronously](#)
- [Data class annotations](#)

## Get Started using the DynamoDB Enhanced Client API

The following tutorial introduces you to fundamentals that you need to work with the DynamoDB Enhanced Client API.

### Add dependencies

To begin working with the DynamoDB Enhanced Client API in your project, add a dependency on the dynamodb-enhanced Maven artifact. This is shown in the following examples.

#### Maven

```
<project>
 <dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version><VERSION></version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
```

```
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>dynamodb-enhanced</artifactId>
 </dependency>
</dependencies>
...
</project>
```

Perform a search of the Maven central repository for the [latest version](#) and replace `<VERSION>` with this value.

## Gradle

```
repositories {
 mavenCentral()
}
dependencies {
 implementation(platform("software.amazon.awssdk:bom:<VERSION>"))
 implementation("software.amazon.awssdk:dynamodb-enhanced")
 ...
}
```

Perform a search of the Maven central repository for the [latest version](#) and replace `<VERSION>` with this value.

## Generate a TableSchema from a data class

A [TableSchema](#) enables the enhanced client to map DynamoDB attribute values to and from your client-side classes. In this tutorial, you learn about TableSchemas derived from a static data class and generated from code by using a builder.

### Use an annotated data class

The SDK for Java 2.x includes a [set of annotations](#) that you can use with a data class to quickly generate a TableSchema for mapping your classes to tables.

Start by creating a data class that conforms to the [JavaBean specification](#). The specification requires that a class has a no-argument public constructor and has getters and setters for each attribute in the class. Include a class-level annotation to indicate that the data class is a



DynamoDBBean. Also, at a minimum, include a `DynamoDbPartitionKey` annotation on the getter or setter for the primary key attribute.

You can apply [attribute-level annotations](#) to getters or setters, but not both.

 **Note**

The term property is normally used for a value encapsulated in a JavaBean. However, this guide uses the term attribute instead, to be consistent with the terminology used by DynamoDB.

The following `Customer` class uses a composite primary key that consists of a partition key (`id`) and a sort key (`email`). A DynamoDB primary key is either a single partition key attribute or a partition key combined with a sort key. Use the `@DynamoDbPartitionKey` and `@DynamoDbSortKey` annotations to identify the key attributes.

### Customer class

```
package org.example.tests.model;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

import java.time.Instant;

@DynamoDbBean
public class Customer {

 private String id;
 private String name;
 private String email;
 private Instant regDate;

 @DynamoDbPartitionKey
 public String getId() { return this.id; }

 public void setId(String id) { this.id = id; }

 public String getCustName() { return this.name; }
```

```
public void setCustName(String name) { this.name = name; }

@DynamoDbSortKey
public String getEmail() { return this.email; }

public void setEmail(String email) { this.email = email; }

public Instant getRegistrationDate() { return this.regDate; }

public void setRegistrationDate(Instant registrationDate) { this.regDate =
registrationDate; }

@Override
public String toString() {
 return "Customer [id=" + id + ", name=" + name + ", email=" + email
 + ", regDate=" + regDate + "]";
}
}
```

After you have created an annotated data class, use it to create the `TableSchema`, as shown in the following snippet.

```
static final TableSchema<Customer> customerTableSchema =
 TableSchema.fromBean(Customer.class);
```

A `TableSchema` is designed to be static and immutable. You can usually instantiate it at class-load time.

The static `TableSchema.fromBean()` factory method introspects the bean to generate the mapping of data class attributes (properties) to and from DynamoDB attributes.

For an example of working with a data model made up of several data classes, see the `Person` class in the [??? section](#).

## Use a builder

You can skip the cost of bean introspection if you define the table schema in code. If you code the schema, your class does not need to follow JavaBean naming standards nor does it need to be annotated. The following example uses a builder and is equivalent to the `Customer` class example that uses annotations.

```
static final TableSchema<Customer> customerTableSchema =
 TableSchema.builder(Customer.class)
 .newItemSupplier(Customer::new)
 .addAttribute(String.class, a -> a.name("id")
 .getter(Customer::getId)
 .setter(Customer::setId)
 .tags(StaticAttributeTags.primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("email")
 .getter(Customer::getEmail)
 .setter(Customer::setEmail)
 .tags(StaticAttributeTags.primarySortKey()))
 .addAttribute(String.class, a -> a.name("name")
 .getter(Customer::getCustName)
 .setter(Customer::setCustName))
 .addAttribute(Instant.class, a -> a.name("registrationDate")
 .getter(Customer::getRegistrationDate)
 .setter(Customer::setRegistrationDate))
 .build();
```

## Create an enhanced client and DynamoDbTable

### Create an enhanced client

The [DynamoDbEnhancedClient](#) class or its asynchronous counterpart, [DynamoDbEnhancedAsyncClient](#), is the entry point to working with the DynamoDB Enhanced Client API.

The enhanced client requires a standard [DynamoDbClient](#) to perform work. The API offers two ways to create a `DynamoDbEnhancedClient` instance. The first option, shown in the following snippet, creates a standard `DynamoDbClient` with default settings picked up from configuration settings.

```
DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.create();
```

If you want to configure the underlying standard client, you can supply it to the enhanced client's builder method as shown in the following snippet.

```
// Configure an instance of the standard DynamoDbClient.
DynamoDbClient standardClient = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(ProfileCredentialsProvider.create())
```

```
.build();

// Use the configured standard client with the enhanced client.
DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(standardClient)
 .build();
```

## Create a `DynamoDbTable` instance

Think of a [DynamoDbTable](#) as the client-side representation of a DynamoDB table that uses the mapping functionality provided by a `TableSchema`. The `DynamoDbTable` class provides methods for CRUD operations that let you interact with a single DynamoDB table.

`DynamoDbTable<T>` is a generic class that takes a single type argument, whether it is a custom class or an `EnhancedDocument` when working with document-type items. This argument type establishes the relationship between the class that you use and the single DynamoDB table.

Use the `table()` factory method of the `DynamoDbEnhancedClient` to create a `DynamoDbTable` instance as shown in the following snippet.

```
static final DynamoDbTable<Customer> customerTable =
 enhancedClient.table("Customer", TableSchema.fromBean(Customer.class));
```

`DynamoDbTable` instances are candidates for singletons because they are immutable and can be used throughout your application.

Your code now has an in-memory representation of a DynamoDB table that can work with `Customer` instances. The actual DynamoDB table might or might not exist. If the table named `Customer` already exists, you can begin performing CRUD operations against it. If it doesn't exist, use the `DynamoDbTable` instance to create the table as discussed in the next section.

## Create a DynamoDB table if needed

After you have created a `DynamoDbTable` instance, use it to perform a *one-time* creation of a table in DynamoDB.

## Create table example code

The following example creates a DynamoDB table based on the `Customer` data class.

This example creates a DynamoDB table with the name `Customer`—identical to the class name—but the table name can be something else. Whatever you name the table, you must use

this name in additional applications to work with the table. Supply this name to the `table()` method anytime you create another `DynamoDbTable` object in order to work with the underlying DynamoDB table.

The Java lambda parameter, `builder`, passed to the `createTable` method lets you [customize the table](#). In this example, [provisioned throughput](#) is configured. If you want to use default settings when you create a table, skip the builder as shown in the following snippet.

```
customerTable.createTable();
```

When default settings are used, values for provisioned throughput are not set. Instead, the billing mode for the table is set to [on-demand](#).

The example also uses a [DynamoDbWaiter](#) before attempting to print out the table name received in the response. The creation of a table takes some time. Therefore, using a waiter means you don't have to write logic that polls the DynamoDB service to see if the table exists before using the table.

## Imports

```
import com.example.dynamodb.Customer;
import software.amazon.awssdk.core.internal.waiters.ResponseOrException;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
import software.amazon.awssdk.enhanced.dynamodb.model.CreateTableEnhancedRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableResponse;
import software.amazon.awssdk.services.dynamodb.waiters.DynamoDbWaiter;
```

## Code

```
public static void createCustomerTable(DynamoDbTable<Customer> customerTable,
DynamoDbClient standardClient) {
 // Create the DynamoDB table using the 'customerTable' DynamoDbTable instance.
 customerTable.createTable(builder -> builder
 .provisionedThroughput(b -> b
 .readCapacityUnits(10L)
 .writeCapacityUnits(10L)
 .build())
);
 // The DynamoDbClient instance (named 'standardClient') passed to the builder for
 the DynamoDbWaiter is the same instance
```

```

 // that was passed to the builder of the DynamoDbEnhancedClient instance that we
 // created previously.
 // By using the same instance, it ensures that the same Region that was configured
 // on the standard DynamoDbClient
 // instance is used for other service clients that accept a DynamoDbClient during
 // construction.
 try (DynamoDbWaiter waiter =
 DynamoDbWaiter.builder().client(standardClient).build()) { // DynamoDbWaiter is
 Autocloseable
 ResponseOrException<DescribeTableResponse> response = waiter
 .waitUntilTableExists(builder ->
 builder.tableName("Customer").build())
 .matched();
 DescribeTableResponse tableDescription = response.response().orElseThrow(
 () -> new RuntimeException("Customer table was not created."));
 // The actual error can be inspected in response.exception()
 logger.info("Customer table was created.");
 }
 }
}

```

### Note

A DynamoDB table's attribute names begin with a lowercase letter when the table is generated from a data class. If you want the table's attribute name to begin with an uppercase letter, use the [@DynamoDbAttribute\(\*NAME\*\) annotation](#) and provide the name you want as a parameter.

## Perform operations

After the table is created, use the `DynamoDbTable` instance to perform operations against the DynamoDB table.

In the following example, a singleton `DynamoDbTable<Customer>` is passed as a parameter along with a [Customer data class](#) instance to add a new item to the table.

```

public static void putItemExample(DynamoDbTable<Customer> customerTable, Customer
customer){
 logger.info(customer.toString());
 customerTable.putItem(customer);
}

```

## Customer object

```
Customer customer = new Customer();
customer.setId("1");
customer.setCustName("Customer Name");
customer.setEmail("customer@example.com");
customer.setRegistrationDate(Instant.parse("2023-07-03T10:15:30.00Z"));
```

Before sending the customer object to the DynamoDB service, log the output of the object's `toString()` method to compare it to what the enhanced client sends.

```
Customer [id=1, name=Customer Name, email=customer@example.com,
regDate=2023-07-03T10:15:30Z]
```

Wire-level logging shows the payload of the generated request. The enhanced client generated the low-level representation from the data class. The `regDate` attribute, which is an `Instant` type in Java, is represented as a DynamoDB string.

```
{
 "TableName": "Customer",
 "Item": {
 "registrationDate": {
 "S": "2023-07-03T10:15:30Z"
 },
 "id": {
 "S": "1"
 },
 "custName": {
 "S": "Customer Name"
 },
 "email": {
 "S": "customer@example.com"
 }
 }
}
```

## Work with an existing table

The previous section showed how to create a DynamoDB table starting with a Java data class. If you already have an existing table and want to use the features of the enhanced client, you can

create a Java data class to work with the table. You need to examine the DynamoDB table and add the necessary annotations to the data class.

Before you work with an existing table, call the `DynamoDbEnhanced.table()` method. This was done in the previous example with the following statement.

```
DynamoDbTable<Customer> customerTable = enhancedClient.table("Customer",
 TableSchema.fromBean(Customer.class));
```

After the `DynamoDbTable` instance is returned, you can begin working right away with the underlying table. You do not need to recreate the table by calling the `DynamoDbTable.createTable()` method.

The following example demonstrates this by immediately retrieving a `Customer` instance from the DynamoDB table.

```
DynamoDbTable<Customer> customerTable = enhancedClient.table("Customer",
 TableSchema.fromBean(Customer.class));
// The Customer table exists already and has an item with a primary key value of "1"
// and a sort key value of "customer@example.com".
customerTable.getItem(
 Key.builder()
 .partitionValue("1")
 .sortValue("customer@example.com").build());
```

### Important

The table name used in the `table()` method must match the existing DynamoDB table name.

## Learn the basics of the DynamoDB Enhanced Client API

This topic discusses the basic features of the DynamoDB Enhanced Client API and compares it to the [standard DynamoDB client API](#).

If you are new to the DynamoDB Enhanced Client API, we recommend that you go through the [introductory tutorial](#) to familiarize yourself with fundamental classes.



## DynamoDB items in Java

DynamoDB tables store items. Depending on your use case, items on the Java side can take the form of statically structured data or structures created dynamically.

If your use case calls for items with a consistent set of attributes, use [annotated classes](#) or use a [builder](#) to generate the appropriate statically-typed `TableSchema`.

Alternatively, if you need to store items that consist of varying structures, create a `DocumentTableSchema`. `DocumentTableSchema` is part of the [Enhanced Document API](#) and requires only a statically-typed primary key and works with `EnhancedDocument` instances to hold the data elements. The Enhanced Document API is covered in another [topic](#).

### Attribute types for data model classes

Although DynamoDB supports [a small number of attribute types](#) compared to the rich type system of Java, the DynamoDB Enhanced Client API provides mechanisms to convert members of a Java class to and from DynamoDB attribute types.

The attribute types (properties) of your Java data classes should be objects types, not primitives. For example, always use `Long` and `Integer` object data types, not `long` and `int` primitives.

By default, the DynamoDB Enhanced Client API supports attribute converters for a large number of types, such as [Integer](#), [String](#), [BigDecimal](#), and [Instant](#). The list appears in the [known implementing classes of the AttributeConverter interface](#). The list includes many types and collections such as maps, lists, and sets.

To store the data for an attribute type that isn't supported by default or doesn't conform to the JavaBean convention, you can write a custom `AttributeConverter` implementation to do the conversion. See the attribute conversion section for an [example](#).

To store the data for an attribute type whose class conforms to the Java beans specification (or an [immutable data class](#)), you can take two approaches.

- If you have access to the source file, you can annotate the class with `@DynamoDbBean` (or `@DynamoDbImmutable`). The section that discusses nested attributes shows [examples](#) of using annotated classes.
- If do not have access to the source file of the JavaBean data class for the attribute (or you don't want to annotate the source file of a class that you do have access to), then you can use the builder approach. This creates a table schema without defining the keys. Then, you can nest this

table schema inside another table schema to perform the mapping. The nested attribute section has an [example](#) showing use of nested schemas.

## Null values

When you use the `putItem` method, the enhanced client does not include null-valued attributes of a mapped data object in the request to DynamoDB.

The SDK's default behavior for `updateItem` requests removes attributes from the item in DynamoDB that are set to null in the object that you submit in the `updateItem` method. If you intend to update some attribute values and keep the others unchanged, you have two options.

- Retrieve the item (by using `getItem`) before you make changes to values. By using this approach, the SDK submits all updated and old values to DynamoDB.
- Use either the [IgnoreNullsMode](#).`SCALAR_ONLY` or `IgnoreNullsMode`.`MAPS_ONLY` when you build the request to update the item. Both modes ignore null-valued properties in the object that represent scalar attributes in DynamoDB. The [the section called "Update items that contain complex types"](#) topic in this guide contains more information about the `IgnoreNullsMode` values and how to work with complex types.

The following example demonstrates `ignoreNullsMode()` for the `updateItem()` method.

```
public static void updateItemNullsExample() {
 Customer customer = new Customer();
 customer.setCustName("CustomerName");
 customer.setEmail("email");
 customer.setId("1");
 customer.setRegistrationDate(Instant.now());

 logger.info("Original customer: {}", customer);

 // Put item with values for all attributes.
 try {
 customerAsyncDynamoDbTable.putItem(customer).join();
 } catch (RuntimeException rte) {
 logger.error("A exception occurred during putItem: {}",
 rte.getCause().getMessage(), rte);
 return;
 }
}
```

```

 // Create a Customer instance with the same 'id' and 'email' values, but a
 different 'name' value.
 // Do not set the 'registrationDate' attribute.
 Customer customerForUpdate = new Customer();
 customerForUpdate.setCustName("NewName");
 customerForUpdate.setEmail("email");
 customerForUpdate.setId("1");

 // Update item without setting the 'registrationDate' property and set
 IgnoreNullsMode to SCALAR_ONLY.
 try {
 Customer updatedWithNullsIgnored = customerAsyncDynamoDbTable.updateItem(b
-> b
 .item(customerForUpdate)
 .ignoreNullsMode(IgnoreNullsMode.SCALAR_ONLY))
 .join();
 logger.info("Customer updated with nulls ignored: {}",
updatedWithNullsIgnored.toString());
 } catch (RuntimeException rte) {
 logger.error("An exception occurred during updateItem: {}",
rte.getCause().getMessage(), rte);
 return;
 }

 // Update item without setting the registrationDate attribute and not setting
 ignoreNulls to true.
 try {
 Customer updatedWithNullsUsed =
customerAsyncDynamoDbTable.updateItem(customerForUpdate)
 .join();
 logger.info("Customer updated with nulls used: {}",
updatedWithNullsUsed.toString());
 } catch (RuntimeException rte) {
 logger.error("An exception occurred during updateItem: {}",
rte.getCause().getMessage(), rte);
 }
 }

 // Logged lines.
 Original customer: Customer [id=1, name=CustomerName, email=email,
 regDate=2024-10-11T14:12:30.222858Z]
 Customer updated with nulls ignored: Customer [id=1, name=NewName, email=email,
 regDate=2024-10-11T14:12:30.222858Z]

```

```
Customer updated with nulls used: Customer [id=1, name=NewName, email=email,
regDate=null]
```

## DynamoDB Enhanced Client basic methods

Basic methods of the enhanced client map to the DynamoDB service operations that they're named after. The following examples show the simplest variation of each method. You can customize each method by passing in an enhanced request object. Enhanced request objects offer most of the features available in the standard DynamoDB client. They are fully documented in the AWS SDK for Java 2.x API Reference.

The example uses the [the section called “Customer class”](#) shown previously.

```
// CreateTable
customerTable.createTable();

// GetItem
Customer customer =
 customerTable.getItem(Key.builder().partitionValue("a123").build());

// UpdateItem
Customer updatedCustomer = customerTable.updateItem(customer);

// PutItem
customerTable.putItem(customer);

// DeleteItem
Customer deletedCustomer =
 customerTable.deleteItem(Key.builder().partitionValue("a123").sortValue(456).build());

// Query
PageIterable<Customer> customers = customerTable.query(keyEqualTo(k ->
 k.partitionValue("a123")));

// Scan
PageIterable<Customer> customers = customerTable.scan();

// BatchGetItem
BatchGetResultPageIterable batchResults =
 enhancedClient.batchGetItem(r -> r.addReadBatch(ReadBatch.builder(Customer.class)
 .mappedTableResource(customerTable)
 .addItem(key1)
 .addItem(key2))
```

```

 .addGetItem(key3)
 .build()));

// BatchWriteItem
batchResults = enhancedClient.batchWriteItem(r ->
 r.addWriteBatch(WriteBatch.builder(Customer.class)
 .mappedTableResource(customerTable)
 .addPutItem(customer)
 .addDeleteItem(key1)
 .addDeleteItem(key1)
 .build()));

// TransactGetItems
transactResults = enhancedClient.transactGetItems(r -> r.addGetItem(customerTable,
 key1)
 .addGetItem(customerTable,
 key2));

// TransactWriteItems
enhancedClient.transactWriteItems(r -> r.addConditionCheck(customerTable,
 i -> i.key(orderKey)

 .conditionExpression(conditionExpression))
 .addUpdateItem(customerTable, customer)
 .addDeleteItem(customerTable, key));

```

## Compare DynamoDB Enhanced Client to standard DynamoDB client

Both DynamoDB client APIs—[standard](#) and [enhanced](#)—let you work with DynamoDB tables to perform CRUD (create, read, update and delete) data-level operations. The difference between the client APIs is how that is accomplished. Using the standard client, you work directly with low-level data attributes. The enhanced client API uses familiar Java classes and maps to the low-level API behind the scenes.

While both client APIs support data-level operations, the standard DynamoDB client also supports resource-level operations. Resource-level operations manage the database, such as creating backups, listing tables, and updating tables. The enhanced client API supports a select number of resource-level operations such as creating, describing, and deleting tables.

To illustrate the different approaches used by the two client APIs, the following code examples show the creation of the same `ProductCatalog` table using the standard client and the enhanced client.

## Compare: Create a table using the standard DynamoDB client

```

DependencyFactory.dynamoDbClient().createTable(builder -> builder
 .tableName(TABLE_NAME)
 .attributeDefinitions(
 b -> b.attributeName("id").attributeType(ScalarAttributeType.N),
 b -> b.attributeName("title").attributeType(ScalarAttributeType.S),
 b -> b.attributeName("isbn").attributeType(ScalarAttributeType.S)
)
 .keySchema(
 builder1 -> builder1.attributeName("id").keyType(KeyType.HASH),
 builder2 -> builder2.attributeName("title").keyType(KeyType.RANGE)
)
 .globalSecondaryIndexes(builder3 -> builder3
 .indexName("products_by_isbn")
 .keySchema(builder2 -> builder2
 .attributeName("isbn").keyType(KeyType.HASH))
 .projection(builder2 -> builder2
 .projectionType(ProjectionType.INCLUDE)
 .nonKeyAttributes("price", "authors"))
 .provisionedThroughput(builder4 -> builder4
 .writeCapacityUnits(5L).readCapacityUnits(5L))
)
 .provisionedThroughput(builder1 -> builder1
 .readCapacityUnits(5L).writeCapacityUnits(5L))
);

```

## Compare: Create a table using the DynamoDB Enhanced Client

```

DynamoDbEnhancedClient enhancedClient = DependencyFactory.enhancedClient();
productCatalog = enhancedClient.table(TABLE_NAME,
 TableSchema.fromImmutableClass(ProductCatalog.class));
productCatalog.createTable(b -> b
 .provisionedThroughput(b1 -> b1.readCapacityUnits(5L).writeCapacityUnits(5L))
 .globalSecondaryIndices(b2 -> b2.indexName("products_by_isbn")
 .projection(b4 -> b4
 .projectionType(ProjectionType.INCLUDE)
 .nonKeyAttributes("price", "authors"))
 .provisionedThroughput(b3 ->
 b3.writeCapacityUnits(5L).readCapacityUnits(5L))
)
);

```

The enhanced client uses the following annotated data class. The DynamoDB Enhanced Client maps Java data types to DynamoDB data types for less verbose code that is easier to follow. `ProductCatalog` is an example of using an immutable class with the DynamoDB Enhanced Client. The use of Immutable classes for mapped data classes is [discussed later](#) in this topic.

## ProductCatalog class

```
package org.example.tests.model;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbIgnore;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbImmutable;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

import java.math.BigDecimal;
import java.util.Objects;
import java.util.Set;

@DynamoDbImmutable(builder = ProductCatalog.Builder.class)
public class ProductCatalog implements Comparable<ProductCatalog> {
 private Integer id;
 private String title;
 private String isbn;
 private Set<String> authors;
 private BigDecimal price;

 private ProductCatalog(Builder builder){
 this.authors = builder.authors;
 this.id = builder.id;
 this.isbn = builder.isbn;
 this.price = builder.price;
 this.title = builder.title;
 }

 public static Builder builder(){ return new Builder(); }

 @DynamoDbPartitionKey
 public Integer id() { return id; }
```

```

@DynamoDbSortKey
public String title() { return title; }

@DynamoDbSecondaryPartitionKey(indexNames = "products_by_isbn")
public String isbn() { return isbn; }
public Set<String> authors() { return authors; }
public BigDecimal price() { return price; }

public static final class Builder {
 private Integer id;
 private String title;
 private String isbn;
 private Set<String> authors;
 private BigDecimal price;
 private Builder(){}

 public Builder id(Integer id) { this.id = id; return this; }
 public Builder title(String title) { this.title = title; return this; }
 public Builder isbn(String ISBN) { this.isbn = ISBN; return this; }
 public Builder authors(Set<String> authors) { this.authors = authors; return
this; }
 public Builder price(BigDecimal price) { this.price = price; return this; }
 public ProductCatalog build() { return new ProductCatalog(this); }
}

@Override
public String toString() {
 final StringBuffer sb = new StringBuffer("ProductCatalog{");
 sb.append("id=").append(id);
 sb.append(", title=").append(title).append('\n');
 sb.append(", isbn=").append(isbn).append('\n');
 sb.append(", authors=").append(authors);
 sb.append(", price=").append(price);
 sb.append('}');
 return sb.toString();
}

@Override
public boolean equals(Object o) {
 if (this == o) return true;
 if (o == null || getClass() != o.getClass()) return false;
 ProductCatalog that = (ProductCatalog) o;

```



```

 return id.equals(that.id) && title.equals(that.title) && Objects.equals(isbn,
that.isbn) && Objects.equals(authors, that.authors) && Objects.equals(price,
that.price);
 }

 @Override
 public int hashCode() {
 return Objects.hash(id, title, isbn, authors, price);
 }

 @Override
 @DynamoDbIgnore
 public int compareTo(ProductCatalog other) {
 if (this.id.compareTo(other.id) != 0){
 return this.id.compareTo(other.id);
 } else {
 return this.title.compareTo(other.title);
 }
 }
}

```

The following two code examples of a batch write illustrate the verbosity and lack of type safety when using the standard client as opposed to the enhanced client.

### Compare: Batch write using the standard DynamoDB client

```

public static void batchWriteStandard(DynamoDbClient dynamoDbClient, String
tableName) {

 Map<String, AttributeValue> catalogItem = Map.of(
 "authors", AttributeValue.builder().ss("a", "b").build(),
 "id", AttributeValue.builder().n("1").build(),
 "isbn", AttributeValue.builder().s("1-565-85698").build(),
 "title", AttributeValue.builder().s("Title 1").build(),
 "price", AttributeValue.builder().n("52.13").build());

 Map<String, AttributeValue> catalogItem2 = Map.of(
 "authors", AttributeValue.builder().ss("a", "b", "c").build(),
 "id", AttributeValue.builder().n("2").build(),
 "isbn", AttributeValue.builder().s("1-208-98073").build(),
 "title", AttributeValue.builder().s("Title 2").build(),
 "price", AttributeValue.builder().n("21.99").build());
}

```

```

Map<String, AttributeValue> catalogItem3 = Map.of(
 "authors", AttributeValue.builder().ss("g", "k", "c").build(),
 "id", AttributeValue.builder().n("3").build(),
 "isbn", AttributeValue.builder().s("7-236-98618").build(),
 "title", AttributeValue.builder().s("Title 3").build(),
 "price", AttributeValue.builder().n("42.00").build());

Set<WriteRequest> writeRequests = Set.of(
 WriteRequest.builder().putRequest(b -> b.item(catalogItem)).build(),
 WriteRequest.builder().putRequest(b -> b.item(catalogItem2)).build(),
 WriteRequest.builder().putRequest(b -> b.item(catalogItem3)).build());

Map<String, Set<WriteRequest>> productCatalogItems = Map.of(
 "ProductCatalog", writeRequests);

BatchWriteItemResponse response = dynamoDbClient.batchWriteItem(b ->
b.requestItems(productCatalogItems));

logger.info("Unprocessed items: " + response.unprocessedItems().size());
}

```

## Compare: Batch write using the DynamoDB Enhanced Client

```

public static void batchWriteEnhanced(DynamoDbTable<ProductCatalog> productCatalog)
{
 ProductCatalog prod = ProductCatalog.builder()
 .id(1)
 .isbn("1-565-85698")
 .authors(new HashSet<>(Arrays.asList("a", "b")))
 .price(BigDecimal.valueOf(52.13))
 .title("Title 1")
 .build();
 ProductCatalog prod2 = ProductCatalog.builder()
 .id(2)
 .isbn("1-208-98073")
 .authors(new HashSet<>(Arrays.asList("a", "b", "c")))
 .price(BigDecimal.valueOf(21.99))
 .title("Title 2")
 .build();
 ProductCatalog prod3 = ProductCatalog.builder()
 .id(3)
 .isbn("7-236-98618")
 .authors(new HashSet<>(Arrays.asList("g", "k", "c")))

```

```

 .price(BigDecimal.valueOf(42.00))
 .title("Title 3")
 .build();

BatchWriteResult batchWriteResult = DependencyFactory.enhancedClient()
 .batchWriteItem(b -> b.writeBatches(
 WriteBatch.builder(ProductCatalog.class)
 .mappedTableResource(productCatalog)
 .addPutItem(prod).addPutItem(prod2).addPutItem(prod3)
 .build()
));
logger.info("Unprocessed items: " +
batchWriteResult.unprocessedPutItemsForTable(productCatalog).size());
}

```

## Work with immutable data classes

The mapping feature of the DynamoDB Enhanced Client API works with immutable data classes. An immutable class has only getters and requires a builder class that the SDK uses to create instances of the class. Instead of using the `@DynamoDbBean` annotation as shown in the [Customer class](#), immutable classes use the `@DynamoDbImmutable` annotation, which takes a parameter that indicates the builder class to use.

The following class is an immutable version of Customer.

```

package org.example.tests.model.immutable;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbImmutable;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondarySortKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

import java.time.Instant;

@DynamoDbImmutable(builder = CustomerImmutable.Builder.class)
public class CustomerImmutable {
 private final String id;
 private final String name;
 private final String email;
}

```

```

private final Instant regDate;

private CustomerImmutable(Builder b) {
 this.id = b.id;
 this.email = b.email;
 this.name = b.name;
 this.regDate = b.regDate;
}

// This method will be automatically discovered and used by the TableSchema.
public static Builder builder() { return new Builder(); }

@DynamoDbPartitionKey
public String id() { return this.id; }

@DynamoDbSortKey
public String email() { return this.email; }

@DynamoDbSecondaryPartitionKey(indexNames = "customers_by_name")
public String name() { return this.name; }

@DynamoDbSecondarySortKey(indexNames = {"customers_by_date", "customers_by_name"})
public Instant regDate() { return this.regDate; }

public static final class Builder {
 private String id;
 private String email;
 private String name;
 private Instant regDate;

 // The private Builder constructor is visible to the enclosing
 CustomerImmutable class.
 private Builder() {}

 public Builder id(String id) { this.id = id; return this; }
 public Builder email(String email) { this.email = email; return this; }
 public Builder name(String name) { this.name = name; return this; }
 public Builder regDate(Instant regDate) { this.regDate = regDate; return
this; }

 // This method will be automatically discovered and used by the TableSchema.
 public CustomerImmutable build() { return new CustomerImmutable(this); }
}

```

```
}
```

You must meet the following requirements when you annotate a data class with `@DynamoDbImmutable`.

1. Every method that is both not an override of `Object.class` and has not been annotated with `@DynamoDbIgnore` must be a getter for an attribute of the DynamoDB table.
2. Every getter must have a corresponding case-sensitive setter on the builder class.
3. Only one of the following construction conditions must be met.
  - The builder class must have a public default constructor.
  - The data class must have a public static method named `builder()` that takes no parameters and returns an instance of the builder class. This option is shown in the immutable `Customer` class.
4. The builder class must have a public method named `build()` that takes no parameters and returns an instance of the immutable class.

To create a `TableSchema` for your immutable class, use the `fromImmutableClass()` method on `TableSchema` as shown in the following snippet.

```
static final TableSchema<CustomerImmutable> customerImmutableTableSchema =
 TableSchema.fromImmutableClass(CustomerImmutable.class);
```

Just as you can create a DynamoDB table from a mutable class, you can create one from an immutable class with a *one-time* call to `createTable()` of `DynamoDbTable` as shown in the following snippet example.

```
static void createTableFromImmutable(DynamoDbEnhancedClient enhancedClient, String
 tableName, DynamoDbWaiter waiter){
 // First, create an in-memory representation of the table using the 'table()'
 method of the DynamoDb Enhanced Client.
 // 'table()' accepts a name for the table and a TableSchema instance that you
 created previously.
 DynamoDbTable<CustomerImmutable> customerDynamoDbTable = enhancedClient
 .table(tableName, TableSchema.fromImmutableClass(CustomerImmutable.class));

 // Second, call the 'createTable()' method on the DynamoDbTable instance.
 customerDynamoDbTable.createTable();
 waiter.waitUntilTableExists(b -> b.tableName(tableName));
```

```
}
```

## Use third-party libraries, such as Lombok

Third-party libraries, such as [Project Lombok](#), help generate boilerplate code associated with immutable objects. The DynamoDB Enhanced Client API works with these libraries as long as the data classes follow the conventions detailed in this section.

The following example shows the immutable `CustomerImmutable` class with Lombok annotations. Note how Lombok's `onMethod` feature copies attribute-based DynamoDB annotations, such as `@DynamoDbPartitionKey`, onto the generated code.

```
@Value
@Builder
@dynamoDbImmutable(builder = Customer.CustomerBuilder.class)
public class Customer {
 @Getter(onMethod_=@DynamoDbPartitionKey)
 private String id;

 @Getter(onMethod_=@DynamoDbSortKey)
 private String email;

 @Getter(onMethod_=@DynamoDbSecondaryPartitionKey(indexNames = "customers_by_name"))
 private String name;

 @Getter(onMethod_=@DynamoDbSecondarySortKey(indexNames = {"customers_by_date",
"customers_by_name"}))
 private Instant createdAt;
}
```

## Use expressions and conditions

Expressions in the DynamoDB Enhanced Client API are Java representations of [DynamoDB expressions](#).

The DynamoDB Enhanced Client API uses three types of expressions:

### [Expression](#)

The `Expression` class is used when you define conditions and filters.

## [QueryConditional](#)

This type of expression represents [key conditions](#) for query operations.

## [UpdateExpression](#)

This class helps you write DynamoDB [update expressions](#) and is currently used in the extension framework when you update an item.

### Expression anatomy

An expression is made up of the following:

- A string expression (required). The string contains a DynamoDB logic expression with placeholder names for attribute names and attribute values.
- A map of expression values (usually required).
- A map of expression names (optional).

Use a builder to generate an `Expression` object that takes the following general form.

```
Expression expression = Expression.builder()
 .expression(<String>)
 .expressionNames(<Map>)
 .expressionValues(<Map>)
 .build()
```

Expressions usually require a map of expression values. The map provides the values for the placeholders in the string expression. The map key consists of the placeholder name preceded with a colon (:) and the map value is an instance of [AttributeValue](#). The [AttributeValues](#) class has convenience methods to generate an `AttributeValue` instance from a literal. Alternatively, you can use the `AttributeValue.Builder` to generate an `AttributeValue` instance.

The following snippet shows a map with two entries after comment line 2. The string passed to the `expression()` method, shown after comment line 1, contains the placeholders that DynamoDB resolves before performing the operation. This snippet doesn't contain a map of expression names, because *price* is a permissible attribute name.

```
public static void scanAsync(DynamoDbAsyncTable productCatalog) {
 ScanEnhancedRequest request = ScanEnhancedRequest.builder()
 .consistentRead(true)
```

```

 .attributesToProject("id", "title", "authors", "price")
 .filterExpression(Expression.builder()
 // 1. :min_value and :max_value are placeholders for the values
provided by the map
 .expression("price >= :min_value AND price <= :max_value")
 // 2. Two values are needed for the expression and each is
supplied as a map entry.
 .expressionValues(
 Map.of(":min_value", numberValue(8.00),
 ":max_value", numberValue(400_000.00)))
 .build())
 .build();

```

If an attribute name in the DynamoDB table is a reserved word, begins with a number, or contains a space, a map of expression names is required for the Expression.

For example, if the attribute name was *1price* instead of *price* in the previous code example, the example would need to be modified as shown in the following example.

```

ScanEnhancedRequest request = ScanEnhancedRequest.builder()
 .filterExpression(Expression.builder()
 .expression("#price >= :min_value AND #price <= :max_value")
 .expressionNames(Map.of("#price", "1price"))
 .expressionValues(
 Map.of(":min_value", numberValue(8.00),
 ":max_value", numberValue(400_000.00)))
 .build())
 .build();

```

A placeholder for an expression name begins with the pound sign (#). An entry for the map of expression names uses the placeholder as the key and the attribute name as the value. The map is added to the expression builder with the `expressionNames()` method. DynamoDB resolves the attribute name before it performs the operation.

Expression values are not required if a function is used in the string expression. An example of an expression function is `attribute_exists(<attribute_name>)`.

The following example builds an Expression that uses a [DynamoDB function](#). The expression string in this example uses no placeholders. This expression could be used on a `putItem` operation to check if an item already exists in the database with a `movie` attribute's value equal to the data object's `movie` attribute.



```
Expression exp = Expression.builder().expression("attribute_not_exists
(movie)").build();
```

The DynamoDB Developer Guide contains complete information on the [low-level expressions](#) that are used with DynamoDB.

## Condition expressions and conditionals

When you use the `putItem()`, `updateItem()`, and `deleteItem()` methods, and also when you use transaction and batch operations, you use [Expression](#) objects to specify conditions that DynamoDB must meet to proceed with the operation. These expressions are named condition expressions. For an example, see the condition expression used in the `addDeleteItem()` method (after comment line 1) of [transaction example](#) shown in this guide.

When you work with the `query()` methods, a condition is expressed as a [QueryConditional](#). The `QueryConditional` class has several static convenience methods that help you write the criteria that determine which items to read from DynamoDB.

For examples of `QueryConditionals`, see the first code example of the [the section called "Query method examples"](#) section of this guide.

## Filter expressions

Filter expressions are used in scan and query operations to filter the items that are returned.

A filter expression is applied after all the data is read from the database, so the read cost is the same as if there were no filter. The *Amazon DynamoDB Developer Guide* has more information about using filter expressions for both [query](#) and [scan](#) operations.

The following example shows a filter expression added to a scan request. The criteria restricts the items returned to items with a price between 8.00 and 80.00 inclusive.

```
Map<String, AttributeValue> expressionValues = Map.of(
 ":min_value", numberValue(8.00),
 ":max_value", numberValue(80.00));

ScanEnhancedRequest request = ScanEnhancedRequest.builder()
 .consistentRead(true)
 // 1. the 'attributesToProject()' method allows you to specify which
 values you want returned.
```

```
 .attributesToProject("id", "title", "authors", "price")
 // 2. Filter expression limits the items returned that match the
 provided criteria.
 .filterExpression(Expression.builder()
 .expression("price >= :min_value AND price <= :max_value")
 .expressionValues(expressionValues)
 .build())
 .build();
```

## Update expressions

The DynamoDB Enhanced Client's `updateItem()` method provides a standard way to update items in DynamoDB. However, when you require more functionality, [UpdateExpressions](#) provide a type-safe representation of DynamoDB [update expression syntax](#). For example, you can use `UpdateExpressions` to increase values without first reading items from DynamoDB, or add individual members to a list. Update expressions are currently available in custom extensions for the `updateItem()` method.

For an example that uses update expressions, see the [custom extension example](#) in this guide.

More information about update expressions is available in the [Amazon DynamoDB Developer Guide](#).

## Work with paginated results: scans and queries

The scan, query and batch methods of the DynamoDB Enhanced Client API return responses with one or more *pages*. A page contains one or more items. Your code can process the response on per-page basis or it can process individual items.

A paginated response returned by the synchronous `DynamoDbEnhancedClient` client returns a [PagerIterable](#) object, whereas a response returned by the asynchronous `DynamoDbEnhancedAsyncClient` returns a [PagePublisher](#) object.

This section looks at processing paginated results and provides examples that use the scan and query APIs.

### Scan a table

The SDK's [scan](#) method corresponds to the [DynamoDB operation](#) of the same name. The DynamoDB Enhanced Client API offers the same options but it uses a familiar object model and handles the pagination for you.

First, we explore the `PageIterable` interface by looking at the `scan` method of the synchronous mapping class, [DynamoDbTable](#).

## Use the synchronous API

The following example shows the `scan` method that uses an [expression](#) to filter the items that are returned. The [ProductCatalog](#) is the model object that was shown earlier.

The filtering expression shown after comment line 2 limits the `ProductCatalog` items that are returned to those with a price value between 8.00 and 80.00 inclusively.

This example also excludes the `isbn` values by using the `attributesToProject` method shown after comment line 1.

After comment line 3, the `PageIterable` object, `pagedResults`, is returned by the `scan` method. The `stream` method of `PageIterable` returns a [java.util.Stream](#) object, which you can use to process the pages. In this example, the number of pages is counted and logged.

Starting with comment line 4, the example shows two variations of accessing the `ProductCatalog` items. The version after comment line 4a streams through each page and sorts and logs the items on each page. The version after comment line 4b skips the page iteration and accesses the items directly.

The `PageIterable` interface offers multiple ways to process results because of its two parent interfaces—[java.lang.Iterable](#) and [SdkIterable](#). `Iterable` brings the `forEach`, `iterator` and `splititerator` methods, and `SdkIterable` brings the `stream` method.

```
public static void scanSync(DynamoDbTable<ProductCatalog> productCatalog) {

 Map<String, AttributeValue> expressionValues = Map.of(
 ":min_value", numberValue(8.00),
 ":max_value", numberValue(80.00));

 ScanEnhancedRequest request = ScanEnhancedRequest.builder()
 .consistentRead(true)
 // 1. the 'attributesToProject()' method allows you to specify which
 values you want returned.
 .attributesToProject("id", "title", "authors", "price")
 // 2. Filter expression limits the items returned that match the
 provided criteria.
 .filterExpression(Expression.builder()
```

```

 .expression("price >= :min_value AND price <= :max_value")
 .expressionValues(expressionValues)
 .build())
 .build();

// 3. A PageIterable object is returned by the scan method.
PageIterable<ProductCatalog> pagedResults = productCatalog.scan(request);
logger.info("page count: {}", pagedResults.stream().count());

// 4. Log the returned ProductCatalog items using two variations.
// 4a. This version sorts and logs the items of each page.
pagedResults.stream().forEach(p -> p.items().stream()
 .sorted(Comparator.comparing(ProductCatalog::price))
 .forEach(
 item -> logger.info(item.toString())
));
// 4b. This version sorts and logs all items for all pages.
pagedResults.items().stream()
 .sorted(Comparator.comparing(ProductCatalog::price))
 .forEach(
 item -> logger.info(item.toString())
);
}

```

## Use the asynchronous API

The asynchronous scan method returns results as a `PagePublisher` object. The `PagePublisher` interface has two subscribe methods that you can use to process response pages. One subscribe method comes from the `org.reactivestreams.Publisher` parent interface. To process pages using this first option, pass a [Subscriber](#) instance to the subscribe method. The first example that follows shows the use of subscribe method.

The second subscribe method comes from the [SdkPublisher](#) interface. This version of subscribe accepts a [Consumer](#) rather than a `Subscriber`. This subscribe method variation is shown in the second example that follows.

The following example shows the asynchronous version of the scan method that uses the same filter expression shown in the previous example.

After comment line 3, `DynamoDbAsyncTable.scan` returns a `PagePublisher` object. On the next line, the code creates an instance of the `org.reactivestreams.Subscriber` interface, `ProductCatalogSubscriber`, which subscribes to the `PagePublisher` after comment line 4.

The `Subscriber` object collects the `ProductCatalog` items from each page in the `onNext` method after comment line 8 in the `ProductCatalogSubscriber` class example. The items are stored in the private `List` variable and are accessed in the calling code with the `ProductCatalogSubscriber.getSubscribedItems()` method. This is called after comment line 5.

After the list is retrieved, the code sorts all `ProductCatalog` items by price and logs each item.

The [CountDownLatch](#) in the `ProductCatalogSubscriber` class blocks the calling thread until all items have been added to the list before continuing after comment line 5.

```
public static void scanAsync(DynamoDbAsyncTable productCatalog) {
 ScanEnhancedRequest request = ScanEnhancedRequest.builder()
 .consistentRead(true)
 .attributesToProject("id", "title", "authors", "price")
 .filterExpression(Expression.builder()
 // 1. :min_value and :max_value are placeholders for the values
provided by the map
 .expression("price >= :min_value AND price <= :max_value")
 // 2. Two values are needed for the expression and each is
supplied as a map entry.
 .expressionValues(
 Map.of(":min_value", numberValue(8.00),
 ":max_value", numberValue(400_000.00)))
 .build())
 .build();

 // 3. A PagePublisher object is returned by the scan method.
 PagePublisher<ProductCatalog> pagePublisher = productCatalog.scan(request);
 ProductCatalogSubscriber subscriber = new ProductCatalogSubscriber();
 // 4. Subscribe the ProductCatalogSubscriber to the PagePublisher.
 pagePublisher.subscribe(subscriber);
 // 5. Retrieve all collected ProductCatalog items accumulated by the
subscriber.
 subscriber.getSubscribedItems().stream()
 .sorted(Comparator.comparing(ProductCatalog::price))
 .forEach(item ->
 logger.info(item.toString()));
 // 6. Use a Consumer to work through each page.
 pagePublisher.subscribe(page -> page
 .items().stream()
 .sorted(Comparator.comparing(ProductCatalog::price))
 .forEach(item ->
```

```

 logger.info(item.toString()))
 .join(); // If needed, blocks the subscribe() method thread until it is
finished processing.
// 7. Use a Consumer to work through each ProductCatalog item.
pagePublisher.items()
 .subscribe(product -> logger.info(product.toString()))
 .exceptionally(failure -> {
 logger.error("ERROR - ", failure);
 return null;
 })
 .join(); // If needed, blocks the subscribe() method thread until it is
finished processing.
}

```

```

private static class ProductCatalogSubscriber implements
Subscriber<Page<ProductCatalog>> {
 private CountDownLatch latch = new CountDownLatch(1);
 private Subscription subscription;
 private List<ProductCatalog> itemsFromAllPages = new ArrayList<>();

 @Override
 public void onSubscribe(Subscription sub) {
 subscription = sub;
 subscription.request(1L);
 try {
 latch.await(); // Called by main thread blocking it until latch is
released.
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 }

 @Override
 public void onNext(Page<ProductCatalog> productCatalogPage) {
 // 8. Collect all the ProductCatalog instances in the page, then ask the
publisher for one more page.
 itemsFromAllPages.addAll(productCatalogPage.items());
 subscription.request(1L);
 }

 @Override
 public void onError(Throwable throwable) {
 }
}

```

```

@Override
public void onComplete() {
 latch.countDown(); // Call by subscription thread; latch releases.
}

List<ProductCatalog> getSubscribedItems() {
 return this.itemsFromAllPages;
}
}

```

The following snippet example uses the version of the `PagePublisher.subscribe` method that accepts a `Consumer` after comment line 6. The Java lambda parameter consumes pages, which further process each item. In this example, each page is processed and the items on each page are sorted and then logged.

```

// 6. Use a Consumer to work through each page.
pagePublisher.subscribe(page -> page
 .items().stream()
 .sorted(Comparator.comparing(ProductCatalog::price))
 .forEach(item ->
 logger.info(item.toString())))
 .join(); // If needed, blocks the subscribe() method thread until it is
finished processing.

```

The `items` method of `PagePublisher` unwraps the model instances so that your code can process the items directly. This approach is shown in the following snippet.

```

// 7. Use a Consumer to work through each ProductCatalog item.
pagePublisher.items()
 .subscribe(product -> logger.info(product.toString()))
 .exceptionally(failure -> {
 logger.error("ERROR - ", failure);
 return null;
 })
 .join(); // If needed, blocks the subscribe() method thread until it is
finished processing.

```

## Query a table

You can use the DynamoDB Enhanced Client to query your table and retrieve multiple items that match specific criteria. The [query\(\)](#) method finds items based on primary key values using the

`@DynamoDbPartitionKey` and optional `@DynamoDbSortKey` annotations defined on your data class.

The `query()` method requires a partition key value and optionally accepts sort key conditions to further refine results. Like the scan API, queries return a `PageIterable` for synchronous calls and a `PagePublisher` for asynchronous calls.

## Query method examples

The `query()` method code example that follows uses the `MovieActor` class. The data class defines a composite primary key that is made up of the **movie** attribute as the partition key and the **actor** attribute as the sort key.

### MovieActor class

```
package org.example.tests.model;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbAttribute;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondarySortKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

import java.util.Objects;

@DynamoDbBean
public class MovieActor implements Comparable<MovieActor> {

 private String movieName;
 private String actorName;
 private String actingAward;
 private Integer actingYear;
 private String actingSchoolName;

 @DynamoDbPartitionKey
 @DynamoDbAttribute("movie")
 public String getMovieName() {
 return movieName;
 }
}
```



```
public void setMovieName(String movieName) {
 this.movieName = movieName;
}

@DynamoDbSortKey
@DynamoDbAttribute("actor")
public String getActorName() {
 return actorName;
}

public void setActorName(String actorName) {
 this.actorName = actorName;
}

@DynamoDbSecondaryPartitionKey(indexNames = "acting_award_year")
@DynamoDbAttribute("actingaward")
public String getActingAward() {
 return actingAward;
}

public void setActingAward(String actingAward) {
 this.actingAward = actingAward;
}

@DynamoDbSecondarySortKey(indexNames = {"acting_award_year", "movie_year"})
@DynamoDbAttribute("actingyear")
public Integer getActingYear() {
 return actingYear;
}

public void setActingYear(Integer actingYear) {
 this.actingYear = actingYear;
}

@DynamoDbAttribute("actingschoolname")
public String getActingSchoolName() {
 return actingSchoolName;
}

public void setActingSchoolName(String actingSchoolName) {
 this.actingSchoolName = actingSchoolName;
}
```

```

@Override
public String toString() {
 final StringBuffer sb = new StringBuffer("MovieActor{");
 sb.append("movieName=").append(movieName).append('\n');
 sb.append(", actorName=").append(actorName).append('\n');
 sb.append(", actingAward=").append(actingAward).append('\n');
 sb.append(", actingYear=").append(actingYear);
 sb.append(", actingSchoolName=").append(actingSchoolName).append('\n');
 sb.append('}');
 return sb.toString();
}

@Override
public boolean equals(Object o) {
 if (this == o) return true;
 if (o == null || getClass() != o.getClass()) return false;
 MovieActor that = (MovieActor) o;
 return Objects.equals(movieName, that.movieName) && Objects.equals(actorName,
that.actorName) && Objects.equals(actingAward, that.actingAward) &&
Objects.equals(actingYear, that.actingYear) && Objects.equals(actingSchoolName,
that.actingSchoolName);
}

@Override
public int hashCode() {
 return Objects.hash(movieName, actorName, actingAward, actingYear,
actingSchoolName);
}

@Override
public int compareTo(MovieActor o) {
 if (this.movieName.compareTo(o.movieName) != 0){
 return this.movieName.compareTo(o.movieName);
 } else {
 return this.actorName.compareTo(o.actorName);
 }
}
}

```

The code examples that follow query against the following items.

## Items in the **MovieActor** table

```
MovieActor{movieName='movie01', actorName='actor0', actingAward='actingaward0',
 actingYear=2001, actingSchoolName='null'}
MovieActor{movieName='movie01', actorName='actor1', actingAward='actingaward1',
 actingYear=2001, actingSchoolName='actingschool1'}
MovieActor{movieName='movie01', actorName='actor2', actingAward='actingaward2',
 actingYear=2001, actingSchoolName='actingschool2'}
MovieActor{movieName='movie01', actorName='actor3', actingAward='actingaward3',
 actingYear=2001, actingSchoolName='null'}
MovieActor{movieName='movie01', actorName='actor4', actingAward='actingaward4',
 actingYear=2001, actingSchoolName='actingschool4'}
MovieActor{movieName='movie02', actorName='actor0', actingAward='actingaward0',
 actingYear=2002, actingSchoolName='null'}
MovieActor{movieName='movie02', actorName='actor1', actingAward='actingaward1',
 actingYear=2002, actingSchoolName='actingschool1'}
MovieActor{movieName='movie02', actorName='actor2', actingAward='actingaward2',
 actingYear=2002, actingSchoolName='actingschool2'}
MovieActor{movieName='movie02', actorName='actor3', actingAward='actingaward3',
 actingYear=2002, actingSchoolName='null'}
MovieActor{movieName='movie02', actorName='actor4', actingAward='actingaward4',
 actingYear=2002, actingSchoolName='actingschool4'}
MovieActor{movieName='movie03', actorName='actor0', actingAward='actingaward0',
 actingYear=2003, actingSchoolName='null'}
MovieActor{movieName='movie03', actorName='actor1', actingAward='actingaward1',
 actingYear=2003, actingSchoolName='actingschool1'}
MovieActor{movieName='movie03', actorName='actor2', actingAward='actingaward2',
 actingYear=2003, actingSchoolName='actingschool2'}
MovieActor{movieName='movie03', actorName='actor3', actingAward='actingaward3',
 actingYear=2003, actingSchoolName='null'}
MovieActor{movieName='movie03', actorName='actor4', actingAward='actingaward4',
 actingYear=2003, actingSchoolName='actingschool4'}
```

The following code defines two `QueryConditional` instances: `keyEqual` (after comment line 1) and `sortGreaterThanOrEqualTo` (after comment line 1a).

### Query items by partition key

The `keyEqual` instance matches items with a partition key value of **movie01**.

This example also defines a filter expression after comment line 2 that filters out any item that does not have an **actingschoolname** value.

The `QueryEnhancedRequest` combines the key condition and filter expression for the query.

```
public static void query(DynamoDbTable movieActorTable) {

 // 1. Define a QueryConditional instance to return items matching a partition
 value.
 QueryConditional keyEqual = QueryConditional.keyEqualTo(b ->
b.partitionValue("movie01"));
 // 1a. Define a QueryConditional that adds a sort key criteria to the partition
 value criteria.
 QueryConditional sortGreaterThanOrEqualTo =
QueryConditional.sortGreaterThanOrEqualTo(b ->
b.partitionValue("movie01").sortValue("actor2"));
 // 2. Define a filter expression that filters out items whose attribute value
 is null.
 final Expression filterOutNoActingschoolname =
Expression.builder().expression("attribute_exists(actingschoolname)").build();

 // 3. Build the query request.
 QueryEnhancedRequest tableQuery = QueryEnhancedRequest.builder()
 .queryConditional(keyEqual)
 .filterExpression(filterOutNoActingschoolname)
 .build();
 // 4. Perform the query using the "keyEqual" conditional and filter expression.
 PageIterable<MovieActor> pagedResults = movieActorTable.query(tableQuery);
 logger.info("page count: {}", pagedResults.stream().count()); // Log number of
 pages.

 pagedResults.items().stream()
 .sorted()
 .forEach(
 item -> logger.info(item.toString()) // Log the sorted list of
 items.
);
}
```

### Example– Output using the `keyEqual` query conditional

The following is the output from running the method. The output displays items with a `movieName` value of **movie01** and displays no items with `actingSchoolName` equal to **null**.

```
2023-03-05 13:11:05 [main] INFO org.example.tests.QueryDemo:46 - page count: 1
```

```

2023-03-05 13:11:05 [main] INFO org.example.tests.QueryDemo:51 -
 MovieActor{movieName='movie01', actorName='actor1', actingAward='actingaward1',
 actingYear=2001, actingSchoolName='actingschool1'}
2023-03-05 13:11:05 [main] INFO org.example.tests.QueryDemo:51 -
 MovieActor{movieName='movie01', actorName='actor2', actingAward='actingaward2',
 actingYear=2001, actingSchoolName='actingschool2'}
2023-03-05 13:11:05 [main] INFO org.example.tests.QueryDemo:51 -
 MovieActor{movieName='movie01', actorName='actor4', actingAward='actingaward4',
 actingYear=2001, actingSchoolName='actingschool4'}

```

## Query items by partition key and sort key

The `sortGreaterThanOrEqualTo` `QueryConditional` refines a partition key match (**movie01**) by adding a sort key condition for values greater than or equal to **actor2**.

[QueryConditional methods](#) that begin with `sort` require a partition key value to match and further refine the query by a comparison based on the sort key value. `Sort` in the method name does not mean the results are sorted, but that a sort key value will be used for comparison.

In the following snippet, we change the query request shown previously after comment line 3. This snippet replaces the `"keyEqual"` query conditional with the `"sortGreaterThanOrEqualTo"` query conditional that was defined after comment line 1a. The following code also removes the filter expression.

```

QueryEnhancedRequest tableQuery = QueryEnhancedRequest.builder()
 .queryConditional(sortGreaterThanOrEqualTo).build();

```

## Example– Output using the `sortGreaterThanOrEqualTo` query conditional

The following output displays the results from the query. The query returns items that have a `movieName` value equal to **movie01** and only items that have an `actorName` value that is greater than or equal to **actor2**. Because we remove the filter, the query returns items that have no value for the `actingSchoolName` attribute.

```

2023-03-05 13:15:00 [main] INFO org.example.tests.QueryDemo:46 - page count: 1
2023-03-05 13:15:00 [main] INFO org.example.tests.QueryDemo:51 -
 MovieActor{movieName='movie01', actorName='actor2', actingAward='actingaward2',
 actingYear=2001, actingSchoolName='actingschool2'}
2023-03-05 13:15:00 [main] INFO org.example.tests.QueryDemo:51 -
 MovieActor{movieName='movie01', actorName='actor3', actingAward='actingaward3',
 actingYear=2001, actingSchoolName='null'}

```

```
2023-03-05 13:15:00 [main] INFO org.example.tests.QueryDemo:51 -
MovieActor{movieName='movie01', actorName='actor4', actingAward='actingaward4',
actingYear=2001, actingSchoolName='actingschool4'}
```

## Perform batch operations

The DynamoDB Enhanced Client API offers two batch methods, [batchGetItem\(\)](#) and [batchWriteItem\(\)](#).

### `batchGetItem()` example

With the [DynamoDbEnhancedClient.batchGetItem\(\)](#) method, you can retrieve up to 100 individual items across multiple tables in one overall request. The following example uses the [Customer](#) and [MovieActor](#) data classes shown previously.

In the example after lines 1 and 2, you build [ReadBatch](#) objects that you later add as parameters to the `batchGetItem()` method after comment line 3.

The code after comment line 1 builds the batch to read from the `Customer` table. The code after comment line 1a shows the use of a [GetItemEnhancedRequest](#) builder that takes a primary key value and a sort key value to specify the item to read. If the data class has composite key, you must provide both the partition key value and the sort key value.

In contrast to specifying key values to request an item, you can use a data class to request an item as shown after comment line 1b. The SDK extracts the key values behind the scenes before submitting the request.

When you specify the item using the key-based approach as shown in the two statements after 2a, you can also specify that DynamoDB should perform a [strongly consistent read](#). When the `consistentRead()` method is used, it must be used on all requested items for the same table.

To retrieve the items that DynamoDB found, use the [resultsForTable\(\)](#) method that is shown after comment line 4. Call the method for each table that was read in the request. `resultsForTable()` returns a list of found items that you can process using any `java.util.List` method. This example logs each item.

To discover items that DynamoDB did not process, use the approach after comment line 5. The `BatchGetResultPage` class has the [unprocessedKeysForTable\(\)](#) method that gives you access to each key that was unprocessed. The [BatchGetItem API reference](#) has more information about situations that result in unprocessed items.

```

public static void batchGetItemExample(DynamoDbEnhancedClient enhancedClient,
 DynamoDbTable<Customer> customerTable,
 DynamoDbTable<MovieActor> movieActorTable) {

 Customer customer2 = new Customer();
 customer2.setId("2");
 customer2.setEmail("cust2@example.org");

 // 1. Build a batch to read from the Customer table.
 ReadBatch customerBatch = ReadBatch.builder(Customer.class)
 .mappedTableResource(customerTable)
 // 1a. Specify the primary key value and sort key value for the item.
 .addGetItem(b -> b.key(k ->
k.partitionValue("1").sortValue("cust1@orgname.org")))
 // 1b. Alternatively, supply a data class instances to provide the
primary key values.
 .addGetItem(customer2)
 .build();

 // 2. Build a batch to read from the MovieActor table.
 ReadBatch moveActorBatch = ReadBatch.builder(MovieActor.class)
 .mappedTableResource(movieActorTable)
 // 2a. Call consistentRead(Boolean.TRUE) for each item for the same
table.
 .addGetItem(b -> b.key(k ->
k.partitionValue("movie01").sortValue("actor1")).consistentRead(Boolean.TRUE))
 .addGetItem(b -> b.key(k ->
k.partitionValue("movie01").sortValue("actor4")).consistentRead(Boolean.TRUE))
 .build();

 // 3. Add ReadBatch objects to the request.
 BatchGetResultPageIterable resultPages = enhancedClient.batchGetItem(b ->
b.readBatches(customerBatch, moveActorBatch));

 // 4. Retrieve the successfully requested items from each table.
 resultPages.resultsForTable(customerTable).forEach(item ->
logger.info(item.toString()));
 resultPages.resultsForTable(movieActorTable).forEach(item ->
logger.info(item.toString()));

 // 5. Retrieve the keys of the items requested but not processed by the
service.
 resultPages.forEach((BatchGetResultPage pageResult) -> {

```

```

 pageResult.unprocessedKeysForTable(customerTable).forEach(key ->
logger.info("Unprocessed item key: " + key.toString()));
 pageResult.unprocessedKeysForTable(movieActorTable).forEach(key ->
logger.info("Unprocessed item key: " + key.toString()));
 });
}

```

Assume that the following items are in the two tables before running the example code.

## Items in tables

```

Customer [id=1, name=CustName1, email=cust1@example.org,
 regDate=2023-03-31T15:46:27.688Z]
Customer [id=2, name=CustName2, email=cust2@example.org,
 regDate=2023-03-31T15:46:28.688Z]
Customer [id=3, name=CustName3, email=cust3@example.org,
 regDate=2023-03-31T15:46:29.688Z]
Customer [id=4, name=CustName4, email=cust4@example.org,
 regDate=2023-03-31T15:46:30.688Z]
Customer [id=5, name=CustName5, email=cust5@example.org,
 regDate=2023-03-31T15:46:31.689Z]
MovieActor{movieName='movie01', actorName='actor0', actingAward='actingaward0',
 actingYear=2001, actingSchoolName='null'}
MovieActor{movieName='movie01', actorName='actor1', actingAward='actingaward1',
 actingYear=2001, actingSchoolName='actingschool1'}
MovieActor{movieName='movie01', actorName='actor2', actingAward='actingaward2',
 actingYear=2001, actingSchoolName='actingschool2'}
MovieActor{movieName='movie01', actorName='actor3', actingAward='actingaward3',
 actingYear=2001, actingSchoolName='null'}
MovieActor{movieName='movie01', actorName='actor4', actingAward='actingaward4',
 actingYear=2001, actingSchoolName='actingschool4'}

```

The following output shows the items returned and logged after comment line 4.

```

Customer [id=1, name=CustName1, email=cust1@example.org,
 regDate=2023-03-31T15:46:27.688Z]
Customer [id=2, name=CustName2, email=cust2@example.org,
 regDate=2023-03-31T15:46:28.688Z]
MovieActor{movieName='movie01', actorName='actor4', actingAward='actingaward4',
 actingYear=2001, actingSchoolName='actingschool4'}
MovieActor{movieName='movie01', actorName='actor1', actingAward='actingaward1',
 actingYear=2001, actingSchoolName='actingschool1'}

```



## batchWriteItem() example

The `batchWriteItem()` method puts or deletes multiple items in one or more tables. You can specify up to 25 individual put or delete operations in the request. The following example uses the [ProductCatalog](#) and [MovieActor](#) model classes shown previously.

`WriteBatch` objects are built after comment lines 1 and 2. For the `ProductCatalog` table, the code puts one item and deletes one item. For the `MovieActor` table after comment line 2, the code puts two items and deletes one.

The `batchWriteItem` method is called after comment line 3. The [builder](#) parameter provides the batch requests for each table.

The returned [BatchWriteResult](#) object provides separate methods for each operation to view unprocessed requests. The code after comment line 4a provides the keys for unprocessed delete requests and the code after comment line 4b provides the unprocessed put items.

```
public static void batchWriteItemExample(DynamoDbEnhancedClient enhancedClient,
 DynamoDbTable<ProductCatalog>
catalogTable,
 DynamoDbTable<MovieActor> movieActorTable)
{

 // 1. Build a batch to write to the ProductCatalog table.
 WriteBatch products = WriteBatch.builder(ProductCatalog.class)
 .mappedTableResource(catalogTable)
 .addPutItem(b -> b.item(getProductCatItem1()))
 .addDeleteItem(b -> b.key(k -> k
 .partitionValue(getProductCatItem2().id())
 .sortValue(getProductCatItem2().title())))
 .build();

 // 2. Build a batch to write to the MovieActor table.
 WriteBatch movies = WriteBatch.builder(MovieActor.class)
 .mappedTableResource(movieActorTable)
 .addPutItem(getMovieActorYeoh())
 .addPutItem(getMovieActorBlanchettPartial())
 .addDeleteItem(b -> b.key(k -> k
 .partitionValue(getMovieActorStreep().getMovieName())
 .sortValue(getMovieActorStreep().getActorName())))
 .build();

 // 3. Add WriteBatch objects to the request.
```

```

 BatchWriteResult batchWriteResult = enhancedClient.batchWriteItem(b ->
b.writeBatches(products, movies));
 // 4. Retrieve keys for items the service did not process.
 // 4a. 'unprocessedDeleteItemsForTable()' returns keys for delete requests that
did not process.
 if (batchWriteResult.unprocessedDeleteItemsForTable(movieActorTable).size() >
0) {

batchWriteResult.unprocessedDeleteItemsForTable(movieActorTable).forEach(key ->
 logger.info(key.toString()));
 }
 // 4b. 'unprocessedPutItemsForTable()' returns keys for put requests that did
not process.
 if (batchWriteResult.unprocessedPutItemsForTable(catalogTable).size() > 0) {
 batchWriteResult.unprocessedPutItemsForTable(catalogTable).forEach(key ->
 logger.info(key.toString()));
 }
 }
}

```

The following helper methods provide the model objects for the put and delete operations.

## Helper methods

```

public static ProductCatalog getProductCatItem1() {
 return ProductCatalog.builder()
 .id(2)
 .isbn("1-565-85698")
 .authors(new HashSet<>(Arrays.asList("a", "b")))
 .price(BigDecimal.valueOf(30.22))
 .title("Title 55")
 .build();
}

public static ProductCatalog getProductCatItem2() {
 return ProductCatalog.builder()
 .id(4)
 .price(BigDecimal.valueOf(40.00))
 .title("Title 1")
 .build();
}

public static MovieActor getMovieActorBlanchettPartial() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Cate Blanchett");
}

```

```

 movieActor.setMovieName("Blue Jasmine");
 movieActor.setActingYear(2023);
 movieActor.setActingAward("Best Actress");
 return movieActor;
 }

 public static MovieActor getMovieActorStreep() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Meryl Streep");
 movieActor.setMovieName("Sophie's Choice");
 movieActor.setActingYear(1982);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("Yale School of Drama");
 return movieActor;
 }

 public static MovieActor getMovieActorYeoh(){
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Michelle Yeoh");
 movieActor.setMovieName("Everything Everywhere All at Once");
 movieActor.setActingYear(2023);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("Royal Academy of Dance");
 return movieActor;
 }
}

```

Assume that the tables contain the following items before you run the example code.

```

MovieActor{movieName='Blue Jasmine', actorName='Cate Blanchett', actingAward='Best Actress', actingYear=2013, actingSchoolName='National Institute of Dramatic Art'}
MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}
ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}

```

After the example code finishes, the tables contain the following items.

```

MovieActor{movieName='Blue Jasmine', actorName='Cate Blanchett', actingAward='Best Actress', actingYear=2013, actingSchoolName='null'}
MovieActor{movieName='Everything Everywhere All at Once', actorName='Michelle Yeoh', actingAward='Best Actress', actingYear=2023, actingSchoolName='Royal Academy of Dance'}
ProductCatalog{id=2, title='Title 55', isbn='1-565-85698', authors=[a, b], price=30.22}

```

Notice in the `MovieActor` table that the `Blue Jasmine` movie item has been replaced with the item used in the `put` request acquired through the `getMovieActorBlanchettPartial()` helper method. If a data bean attribute value was not provided, the value in the database is removed. This is why the resulting `actingSchoolName` is null for the `Blue Jasmine` movie item.

### Note

Although the API documentation suggests that condition expressions can be used and that consumed capacity and collection metrics can be returned with individual [put](#) and [delete](#) requests, this is not the case in a batch write scenario. To improve performance for batch operations, these individual options are ignored.

## Perform transaction operations

The DynamoDB Enhanced Client API provides the `transactGetItems()` and the `transactWriteItems()` methods. The transaction methods of the SDK for Java provide atomicity, consistency, isolation, and durability (ACID) in DynamoDB tables, helping you to maintain data correctness in your applications.

### `transactGetItems()` example

The [transactGetItems\(\)](#) method accepts up to 100 individual requests for items. All items are read in a single atomic transaction. The *Amazon DynamoDB Developer Guide* has information about the [conditions that cause a transactGetItems\(\) method to fail](#), and also about the isolation level used when you call [transactGetItem\(\)](#).

After comment line 1 in the following example, the code calls the `transactGetItems()` method with a [builder](#) parameter. The builder's [addGetItem\(\)](#) is invoked three times with a data object that contains the key values that the SDK will use to generate the final request.

The request returns a list of [Document](#) objects after comment line 2. The list of documents that is returned contains non-null [Document](#) instances of item data in the same order as requested. The [Document.getItem\(MappedTableResource<T> mappedTableResource\)](#) method converts an untyped `Document` object into a typed Java object if item data was returned, otherwise the method returns null.

```
public static void transactGetItemsExample(DynamoDbEnhancedClient enhancedClient,
 DynamoDbTable<ProductCatalog>
catalogTable,
```

```

DynamoDbTable<MovieActor>
movieActorTable) {

 // 1. Request three items from two tables using a builder.
 final List<Document> documents = enhancedClient.transactGetItems(b -> b
 .addGetItem(catalogTable,
Key.builder().partitionValue(2).sortValue("Title 55").build())
 .addGetItem(movieActorTable, Key.builder().partitionValue("Sophie's
Choice").sortValue("Meryl Streep").build())
 .addGetItem(movieActorTable, Key.builder().partitionValue("Blue
Jasmine").sortValue("Cate Blanchett").build())
 .build());

 // 2. A list of Document objects is returned in the same order as requested.
 ProductCatalog title55 = documents.get(0).getItem(catalogTable);
 if (title55 != null) {
 logger.info(title55.toString());
 }

 MovieActor sophiesChoice = documents.get(1).getItem(movieActorTable);
 if (sophiesChoice != null) {
 logger.info(sophiesChoice.toString());
 }

 // 3. The getItem() method returns null if the Document object contains no item
 from DynamoDB.
 MovieActor blueJasmine = documents.get(2).getItem(movieActorTable);
 if (blueJasmine != null) {
 logger.info(blueJasmine.toString());
 }
}

```

The DynamoDB tables contain the following items before the code example runs.

```

ProductCatalog{id=2, title='Title 55', isbn='orig_isbn', authors=[b, g], price=10}
MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best
Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}

```

The following output is logged. If an item is requested but not found, it not returned as is the case for the request for the movie named Blue Jasmine.

```

ProductCatalog{id=2, title='Title 55', isbn='orig_isbn', authors=[b, g], price=10}

```

```
MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}
```

## transactWriteItems() examples

The [transactWriteItems\(\)](#) accepts up to 100 put, update, or delete actions in a single atomic transaction across multiple tables. The *Amazon DynamoDB Developer Guide* contains details about restrictions and failure conditions of the [underlying DynamoDB service operation](#).

### Basic example

In the following example, four operations are requested for two tables. The corresponding model classes [ProductCatalog](#) and [MovieActor](#) were shown previously.

Each of the three possible operations—put, update, and delete—uses a dedicated request parameter to specify the details.

The code after comment line 1 shows the simple variation of the `addPutItem()` method. The method accepts a [MappedTableResource](#) object and the data object instance to put. The statement after comment line 2 shows the variation that accepts a [TransactPutItemEnhancedRequest](#) instance. This variation lets you add more options in the request, such as a condition expression. A subsequent [example](#) shows a condition expression for an individual operation.

An update operation is requested after comment line 3.

[TransactUpdateItemEnhancedRequest](#) has an `ignoreNulls()` method that lets you configure what the SDK does with null values on the model object. If the `ignoreNulls()` method returns true, the SDK does not remove the table's attribute values for data object attributes that are null. If the `ignoreNulls()` method returns false, the SDK requests the DynamoDB service to remove the attributes from the item in the table. The default value for `ignoreNulls` is false.

The statement after comment line 4 shows the variation of a delete request that takes a data object. The enhanced client extracts the key values before dispatching the final request.

```
public static void transactWriteItems(DynamoDbEnhancedClient enhancedClient,
 DynamoDbTable<ProductCatalog> catalogTable,
 DynamoDbTable<MovieActor> movieActorTable) {

 enhancedClient.transactWriteItems(b -> b
```

```

 // 1. Simplest variation of put item request.
 .addPutItem(catalogTable, getProductCatId2())
 // 2. Put item request variation that accommodates condition
expressions.
 .addPutItem(movieActorTable,
TransactPutItemEnhancedRequest.builder(MovieActor.class)
 .item(getMovieActorStreep())

 .conditionExpression(Expression.builder().expression("attribute_not_exists
(movie)").build())
 .build())
 // 3. Update request that does not remove attribute values on the table
if the data object's value is null.
 .addUpdateItem(catalogTable,
TransactUpdateItemEnhancedRequest.builder(ProductCatalog.class)
 .item(getProductCatId4ForUpdate())
 .ignoreNulls(Boolean.TRUE)
 .build())
 // 4. Variation of delete request that accepts a data object. The key
values are extracted for the request.
 .addDeleteItem(movieActorTable, getMovieActorBlanchett())
);
}

```

The following helper methods provide the data objects for the add\*Item parameters.

## Helper methods

```

public static ProductCatalog getProductCatId2() {
 return ProductCatalog.builder()
 .id(2)
 .isbn("1-565-85698")
 .authors(new HashSet<>(Arrays.asList("a", "b")))
 .price(BigDecimal.valueOf(30.22))
 .title("Title 55")
 .build();
}

public static ProductCatalog getProductCatId4ForUpdate() {
 return ProductCatalog.builder()
 .id(4)
 .price(BigDecimal.valueOf(40.00))
 .title("Title 1")
 .build();
}

```

```
}

public static MovieActor getMovieActorBlanchett() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Cate Blanchett");
 movieActor.setMovieName("Tar");
 movieActor.setActingYear(2022);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("National Institute of Dramatic Art");
 return movieActor;
}

public static MovieActor getMovieActorStreep() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Meryl Streep");
 movieActor.setMovieName("Sophie's Choice");
 movieActor.setActingYear(1982);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("Yale School of Drama");
 return movieActor;
}
```

The DynamoDB tables contain the following items before the code example runs.

```
1 | ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}
2 | MovieActor{movieName='Tar', actorName='Cate Blanchett', actingAward='Best Actress',
 actingYear=2022, actingSchoolName='National Institute of Dramatic Art'}
```

The following items are in the tables after the code finishes running.

```
3 | ProductCatalog{id=2, title='Title 55', isbn='1-565-85698', authors=[a, b],
 price=30.22}
4 | ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=40.0}
5 | MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best
 Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}
```

The item on line 2 has been deleted and lines 3 and 5 show the items that were put. Line 4 shows the update of line 1. The price value is the only value that changed on the item. If `ignoreNulls()` had returned false, line 4 would look like the following line.

```
ProductCatalog{id=4, title='Title 1', isbn='null', authors=null, price=40.0}
```



## Condition check example

The following example shows the use of a condition check. A condition check is used to check that an item exists or to check the condition of specific attributes of an item in the database. The item checked in the condition check cannot be used in another operation in the transaction.

### Note

You can't target the same item with multiple operations within the same transaction. For example, you can't perform a condition check and also attempt to update the same item in the same transaction.

The example shows one of each type of operation in a transactional write items request. After comment line 2, the `addConditionCheck()` method supplies the condition that fails the transaction if the `conditionExpression` parameter evaluates to false. The condition expression that is returned from the method shown in the Helper methods block checks if the award year for the movie Sophie's Choice is not equal to 1982. If it is, the expression evaluates to false and the transaction fails.

This guide discusses [expressions](#) in depth in another topic.

```
public static void conditionCheckFailExample(DynamoDbEnhancedClient enhancedClient,
 DynamoDbTable<ProductCatalog>
catalogTable,
 DynamoDbTable<MovieActor>
movieActorTable) {
 try {
 enhancedClient.transactWriteItems(b -> b
 // 1. Perform one of each type of operation with the next three
methods.
 .addPutItem(catalogTable,
TransactPutItemEnhancedRequest.builder(ProductCatalog.class)
 .item(getProductCatId2()).build())
 .addUpdateItem(catalogTable,
TransactUpdateItemEnhancedRequest.builder(ProductCatalog.class)
 .item(getProductCatId4ForUpdate())
 .ignoreNulls(Boolean.TRUE).build())
 .addDeleteItem(movieActorTable,
TransactDeleteItemEnhancedRequest.builder()
```

```

 .key(b1 -> b1

.partitionValue(getMovieActorBlanchett().getMovieName())

.sortValue(getMovieActorBlanchett().getActorName()).build())
 // 2. Add a condition check on a table item that is not involved in
another operation in this request.
 .addConditionCheck(movieActorTable, ConditionCheck.builder()
 .conditionExpression(buildConditionCheckExpression())
 .key(k -> k
 .partitionValue("Sophie's Choice")
 .sortValue("Meryl Streep"))
 // 3. Specify the request to return existing values from
the item if the condition evaluates to true.

.returnValuesOnConditionCheckFailure(ReturnValuesOnConditionCheckFailure.ALL_OLD)
 .build())
 .build());
 // 4. Catch the exception if the transaction fails and log the information.
} catch (TransactionCanceledException ex) {
 ex.cancellationReasons().stream().forEach(cancellationReason -> {
 logger.info(cancellationReason.toString());
 });
}
}

```

The following helper methods are used in the previous code example.

## Helper methods

```

private static Expression buildConditionCheckExpression() {
 Map<String, AttributeValue> expressionValue = Map.of(
 ":year", numberValue(1982));

 return Expression.builder()
 .expression("actingyear <> :year")
 .expressionValues(expressionValue)
 .build();
}

public static ProductCatalog getProductCatId2() {
 return ProductCatalog.builder()
 .id(2)
 .isbn("1-565-85698")

```

```

 .authors(new HashSet<>(Arrays.asList("a", "b")))
 .price(BigDecimal.valueOf(30.22))
 .title("Title 55")
 .build();
 }

 public static ProductCatalog getProductCatId4ForUpdate() {
 return ProductCatalog.builder()
 .id(4)
 .price(BigDecimal.valueOf(40.00))
 .title("Title 1")
 .build();
 }

 public static MovieActor getMovieActorBlanchett() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Cate Blanchett");
 movieActor.setMovieName("Blue Jasmine");
 movieActor.setActingYear(2013);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("National Institute of Dramatic Art");
 return movieActor;
 }
}

```

The DynamoDB tables contain the following items before the code example runs.

```

1 | ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}
2 | MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}
3 | MovieActor{movieName='Tar', actorName='Cate Blanchett', actingAward='Best Actress', actingYear=2022, actingSchoolName='National Institute of Dramatic Art'}

```

The following items are in the tables after the code finishes running.

```

ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}
MovieActor{movieName='Sophie's Choice', actorName='Meryl Streep', actingAward='Best Actress', actingYear=1982, actingSchoolName='Yale School of Drama'}
MovieActor{movieName='Tar', actorName='Cate Blanchett', actingAward='Best Actress', actingYear=2022, actingSchoolName='National Institute of Dramatic Art'}

```

Items remain unchanged in the tables because the transaction failed. The `actingYear` value for the movie `Sophie's Choice` is 1982, as shown on line 2 of the items in the table before the `transactWriteItem()` method is called.

To capture the cancellation information for the transaction, enclose the `transactWriteItems()` method call in a try block and catch the [TransactionCanceledException](#). After comment line 4 of the example, the code logs each [CancellationReason](#) object. Because the code following comment line 3 of the example specifies that values should be returned for the item that caused the transaction to fail, the log displays the raw database values for the `Sophie's Choice` movie item.

```
CancellationReason(Code=None)
CancellationReason(Code=None)
CancellationReason(Code=None)
CancellationReason(Item={actor=AttributeValue(S=Meryl Streep),
 movie=AttributeValue(S=Sophie's Choice), actingaward=AttributeValue(S=Best Actress),
 actingyear=AttributeValue(N=1982), actingschoolname=AttributeValue(S=Yale School of
 Drama)}, ~
 Code=ConditionalCheckFailed, Message=The conditional request failed.)
```

## Single operation condition example

The following example shows the use of a condition on a single operation in a transaction request. The delete operation after comment line 1 contains a condition that checks the value of the target item of the operation against the database. In this example, the condition expression created with the helper method after comment line 2 specifies that the item should be deleted from the database if the acting year of the movie is not equal to 2013.

[Expressions](#) are discussed later in this guide.

```
public static void singleOperationConditionFailExample(DynamoDbEnhancedClient
enhancedClient,

DynamoDbTable<ProductCatalog> catalogTable,

DynamoDbTable<MovieActor>
movieActorTable) {
 try {
 enhancedClient.transactWriteItems(b -> b
 .addPutItem(catalogTable,
TransactPutItemEnhancedRequest.builder(ProductCatalog.class)
 .item(getProductCatId2()))
```

```

 .build())
 .addUpdateItem(catalogTable,
TransactUpdateItemEnhancedRequest.builder(ProductCatalog.class)
 .item(getProductCatId4ForUpdate())
 .ignoreNulls(Boolean.TRUE).build())
 // 1. Delete operation that contains a condition expression
 .addDeleteItem(movieActorTable,
TransactDeleteItemEnhancedRequest.builder()
 .key((Key.Builder k) -> {
 MovieActor blanchett = getMovieActorBlanchett();
 k.partitionValue(blanchett.getMovieName())
 .sortValue(blanchett.getActorName());
 })
 .conditionExpression(buildDeleteItemExpression()))

 .returnValuesOnConditionCheckFailure(ReturnValuesOnConditionCheckFailure.ALL_OLD)
 .build())
 .build());
 } catch (TransactionCanceledException ex) {
 ex.cancellationReasons().forEach(cancellationReason ->
logger.info(cancellationReason.toString()));
 }
}

// 2. Provide condition expression to check if 'actingyear' is not equal to 2013.
private static Expression buildDeleteItemExpression() {
 Map<String, AttributeValue> expressionValue = Map.of(
 ":year", numberValue(2013));

 return Expression.builder()
 .expression("actingyear <> :year")
 .expressionValues(expressionValue)
 .build();
}

```

The following helper methods are used in the previous code example.

## Helper methods

```

public static ProductCatalog getProductCatId2() {
 return ProductCatalog.builder()
 .id(2)
 .isbn("1-565-85698")
 .authors(new HashSet<>(Arrays.asList("a", "b")))

```

```

 .price(BigDecimal.valueOf(30.22))
 .title("Title 55")
 .build();
 }

 public static ProductCatalog getProductCatId4ForUpdate() {
 return ProductCatalog.builder()
 .id(4)
 .price(BigDecimal.valueOf(40.00))
 .title("Title 1")
 .build();
 }

 public static MovieActor getMovieActorBlanchett() {
 MovieActor movieActor = new MovieActor();
 movieActor.setActorName("Cate Blanchett");
 movieActor.setMovieName("Blue Jasmine");
 movieActor.setActingYear(2013);
 movieActor.setActingAward("Best Actress");
 movieActor.setActingSchoolName("National Institute of Dramatic Art");
 return movieActor;
 }
}

```

The DynamoDB tables contain the following items before the code example runs.

```

1 | ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}
2 | MovieActor{movieName='Blue Jasmine', actorName='Cate Blanchett', actingAward='Best
 Actress', actingYear=2013, actingSchoolName='National Institute of Dramatic Art'}

```

The following items are in the tables after the code finishes running.

```

ProductCatalog{id=4, title='Title 1', isbn='orig_isbn', authors=[b, g], price=10}
2023-03-15 11:29:07 [main] INFO org.example.tests.TransactDemoTest:168 -
 MovieActor{movieName='Blue Jasmine', actorName='Cate Blanchett', actingAward='Best
 Actress', actingYear=2013, actingSchoolName='National Institute of Dramatic Art'}

```

Items remain unchanged in the tables because the transaction failed. The actingYear value for the movie Blue Jasmine is 2013 as shown on line 2 in the list of items before the code example runs.

The following lines are logged to the console.

```

CancellationReason(Code=None)

```

```

CancellationReason(Code=None)
CancellationReason(Item={actor=AttributeValue(S=Cate Blanchett),
 movie=AttributeValue(S=Blue Jasmine), actingaward=AttributeValue(S=Best Actress),
 actingyear=AttributeValue(N=2013), actingschoolname=AttributeValue(S=National
 Institute of Dramatic Art)},
 Code=ConditionalCheckFailed, Message=The conditional request failed)

```

## Use secondary indices

The [primary key](#) of a table defines the primary index, which determines how DynamoDB stores and retrieves items by default.

Secondary indexes provide alternative keys that you use in query and scan operations. Global secondary indexes (GSI) have a partition key and an optional sort key that can differ from those on the base table. In contrast, local secondary indexes (LSI) share the partition key of the primary index but define a different sort key.

## Define keys for global and local secondary indexes

Attributes that participate in secondary indices require either the `@DynamoDbSecondaryPartitionKey` or `@DynamoDbSecondarySortKey` annotation.

The following class shows annotations for two indices. The GSI named *SubjectLastPostedDateIndex* uses the `Subject` attribute for the partition key and the `LastPostedDateTime` for the sort key. The LSI named *ForumLastPostedDateIndex* uses the `ForumName` as its partition key and `LastPostedDateTime` as its sort key.

Note that the `Subject` attribute serves a dual role. It is the primary key's sort key and the partition key of the GSI named *SubjectLastPostedDateIndex*.

## MessageThread class

The `MessageThread` class is suitable to use as a data class for the [example Thread table](#) in the *Amazon DynamoDB Developer Guide*.

## Imports

```

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;

```

```
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondarySortKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSortKey;

import java.util.List;
```

```
@DynamoDbBean
public class MessageThread {
 private String ForumName;
 private String Subject;
 private String Message;
 private String LastPostedBy;
 private String LastPostedDateTime;
 private Integer Views;
 private Integer Replies;
 private Integer Answered;
 private List<String> Tags;

 @DynamoDbPartitionKey
 public String getForumName() {
 return ForumName;
 }

 public void setForumName(String forumName) {
 ForumName = forumName;
 }

 // Sort key for primary index and partition key for GSI
 "SubjectLastPostedDateIndex".
 @DynamoDbSortKey
 @DynamoDbSecondaryPartitionKey(indexNames = "SubjectLastPostedDateIndex")
 public String getSubject() {
 return Subject;
 }

 public void setSubject(String subject) {
 Subject = subject;
 }

 // Sort key for GSI "SubjectLastPostedDateIndex" and sort key for LSI
 "ForumLastPostedDateIndex".
 @DynamoDbSecondarySortKey(indexNames = {"SubjectLastPostedDateIndex",
 "ForumLastPostedDateIndex"})
```



```
public String getLastPostedDateTime() {
 return LastPostedDateTime;
}

public void setLastPostedDateTime(String lastPostedDateTime) {
 LastPostedDateTime = lastPostedDateTime;
}

public String getMessage() {
 return Message;
}

public void setMessage(String message) {
 Message = message;
}

public String getLastPostedBy() {
 return LastPostedBy;
}

public void setLastPostedBy(String lastPostedBy) {
 LastPostedBy = lastPostedBy;
}

public Integer getViews() {
 return Views;
}

public void setViews(Integer views) {
 Views = views;
}

@DynamoDbSecondaryPartitionKey(indexNames = "ForumRepliesIndex")
public Integer getReplies() {
 return Replies;
}

public void setReplies(Integer replies) {
 Replies = replies;
}

public Integer getAnswered() {
 return Answered;
}
```

```

 public void setAnswered(Integer answered) {
 Answered = answered;
 }

 public List<String> getTags() {
 return Tags;
 }

 public void setTags(List<String> tags) {
 Tags = tags;
 }

 public MessageThread() {
 this.Answered = 0;
 this.LastPostedBy = "";
 this.ForumName = "";
 this.Message = "";
 this.LastPostedDateTime = "";
 this.Replies = 0;
 this.Views = 0;
 this.Subject = "";
 }

 @Override
 public String toString() {
 return "MessageThread{" +
 "ForumName='" + ForumName + '\'' +
 ", Subject='" + Subject + '\'' +
 ", Message='" + Message + '\'' +
 ", LastPostedBy='" + LastPostedBy + '\'' +
 ", LastPostedDateTime='" + LastPostedDateTime + '\'' +
 ", Views=" + Views +
 ", Replies=" + Replies +
 ", Answered=" + Answered +
 ", Tags=" + Tags +
 '}';
 }
}

```

## Create the index

Beginning with version 2.20.86 of the SDK for Java, the `createTable()` method automatically generates secondary indexes from data class annotations. By default, all attributes from the base

table are copied to an index and the provisioned throughput values are 20 read capacity units and 20 write capacity units.

However, if you use an SDK version prior to 2.20.86, you need to build the index along with the table as shown in the following example. This example builds the two indexes for the Thread table. The [builder](#) parameter has methods to configure both types of indexes as shown after comment lines 1 and 2. You use the index builder's `indexName()` method to associate the index names specified in the data class annotations with the intended type of index.

This code configures all of the table attributes to end up in both indexes after comment lines 3 and 4. More information about [attribute projections](#) is available in the *Amazon DynamoDB Developer Guide*.

```
public static void createMessageThreadTable(DynamoDbTable<MessageThread>
messageThreadDynamoDbTable, DynamoDbClient dynamoDbClient) {
 messageThreadDynamoDbTable.createTable(b -> b
 // 1. Generate the GSI.
 .globalSecondaryIndices(gsi ->
gsi.indexName("SubjectLastPostedDateIndex")
 // 3. Populate the GSI with all attributes.
 .projection(p -> p
 .projectionType(ProjectionType.ALL))
)
 // 2. Generate the LSI.
 .localSecondaryIndices(lsi -> lsi.indexName("ForumLastPostedDateIndex")
 // 4. Populate the LSI with all attributes.
 .projection(p -> p
 .projectionType(ProjectionType.ALL))
)
);
}
```

## Query by using an index

The following example queries the local secondary index *ForumLastPostedDateIndex*.

Following comment line 2, you create a [QueryConditional](#) object that is required when calling the [DynamoDbIndex.query\(\)](#) method.

You get a reference to the index you want to query after comment line 3 by passing in the name of the index. Following comment line 4, you call the `query()` method on the index passing in the `QueryConditional` object.

You also configure the query to return three attribute values as shown after comment line 5. If `attributesToProject()` is not called, the query returns all attribute values. Notice that the specified attribute names begin with lowercase letters. These attribute names match those used in the table, not necessarily the attribute names of the data class.

Following comment line 6, iterate through the results and log each item returned by the query and also store it in the list to return to the caller.

```
public class IndexScanExamples {
 private static Logger logger = LoggerFactory.getLogger(IndexScanExamples.class);

 public static List<MessageThread> queryUsingSecondaryIndices(String lastPostedDate,
DynamoDbTable<MessageThread> threadTable) {
 // 1. Log the parameter value.
 logger.info("lastPostedDate value: {}", lastPostedDate);

 // 2. Create a QueryConditional whose sort key value must be greater than or
 equal to the parameter value.
 QueryConditional queryConditional =
 QueryConditional.sortGreaterThanOrEqualTo(qc ->
 qc.partitionValue("Forum02").sortValue(lastPostedDate));

 // 3. Specify the index name to query.
 final DynamoDbIndex<MessageThread> forumLastPostedDateIndex =
 threadTable.index("ForumLastPostedDateIndex");

 // 4. Perform the query using the QueryConditional object.
 final SdkIterable<Page<MessageThread>> pagedResult =
 forumLastPostedDateIndex.query(q -> q
 .queryConditional(queryConditional)
 // 5. Request three attribute in the results.
 .attributesToProject("forumName", "subject", "lastPostedDateTime"));

 List<MessageThread> collectedItems = new ArrayList<>();
 // 6. Iterate through pages response and sort the items.
 pagedResult.stream().forEach(page -> page.items().stream()
 .sorted(Comparator.comparing(MessageThread::getLastPostedDateTime))
 .forEach(mt -> {
 // 7. Log the returned items and add the collection to return to
 the caller.
 logger.info(mt.toString());
 collectedItems.add(mt);
 }
));
 }
}
```

```

 }));
 return collectedItems;
}

```

The following items exist in the database before the query is run.

```

MessageThread{ForumName='Forum01', Subject='Subject01', Message='Message01',
 LastPostedBy='', LastPostedDateTime='2023.03.28', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum02', Subject='Subject02', Message='Message02',
 LastPostedBy='', LastPostedDateTime='2023.03.29', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum02', Subject='Subject04', Message='Message04',
 LastPostedBy='', LastPostedDateTime='2023.03.31', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum02', Subject='Subject08', Message='Message08',
 LastPostedBy='', LastPostedDateTime='2023.04.04', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum02', Subject='Subject10', Message='Message10',
 LastPostedBy='', LastPostedDateTime='2023.04.06', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum03', Subject='Subject03', Message='Message03',
 LastPostedBy='', LastPostedDateTime='2023.03.30', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum03', Subject='Subject06', Message='Message06',
 LastPostedBy='', LastPostedDateTime='2023.04.02', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum03', Subject='Subject09', Message='Message09',
 LastPostedBy='', LastPostedDateTime='2023.04.05', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum05', Subject='Subject05', Message='Message05',
 LastPostedBy='', LastPostedDateTime='2023.04.01', Views=0, Replies=0, Answered=0,
 Tags=null}
MessageThread{ForumName='Forum07', Subject='Subject07', Message='Message07',
 LastPostedBy='', LastPostedDateTime='2023.04.03', Views=0, Replies=0, Answered=0,
 Tags=null}

```

The logging statements at lines 1 and 6 result in the following console output.

```

lastPostedDate value: 2023.03.31
MessageThread{ForumName='Forum02', Subject='Subject04', Message='', LastPostedBy='',
 LastPostedDateTime='2023.03.31', Views=0, Replies=0, Answered=0, Tags=null}

```

```
MessageThread{ForumName='Forum02', Subject='Subject08', Message='', LastPostedBy='',
 LastPostedDateTime='2023.04.04', Views=0, Replies=0, Answered=0, Tags=null}
MessageThread{ForumName='Forum02', Subject='Subject10', Message='', LastPostedBy='',
 LastPostedDateTime='2023.04.06', Views=0, Replies=0, Answered=0, Tags=null}
```

The query returned items with a `forumName` value of *Forum02* and a `lastPostedDateTime` value greater than or equal to *2023.03.31*. The results show message values with an empty string although the message attributes have values in the index. This is because the message attribute was not projected by the code after comment line 5.

## Use composite keys for global secondary indexes (GSI)

The SDK for Java 2.x DynamoDB Enhanced Client supports composite keys for global secondary indexes. You can define up to four partition key attributes and four sort key attributes for a single GSI. This removes the need to concatenate multiple attributes into a single string key on the client side.

Use the `order` parameter on the `@DynamoDbSecondaryPartitionKey` and `@DynamoDbSecondarySortKey` annotations to specify the position of each attribute in the composite key. The `order` parameter accepts values from the [Order](#) enum (part of the Enhanced DynamoDB Client): `FIRST`, `SECOND`, `THIRD`, and `FOURTH`.

## Annotate data class with composite key annotations

The following example models an order table. The base table uses `orderId` as the partition key. A GSI named *OrdersByStatusDateAmount* uses a composite partition key of `customerId` and `status`, and a composite sort key of `orderDate` and `amount`. This design lets you query a customer's orders filtered by status, date, and amount threshold.

## Imports

```
import software.amazon.awssdk.enhanced.dynamodb.mapper.Order;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondaryPartitionKey;
import
 software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbSecondarySortKey;
```

```
@DynamoDbBean
```

```
public class OrderItem {
 private String orderId;
 private String customerId;
 private String status;
 private String orderDate;
 private Integer amount;

 @DynamoDbPartitionKey
 public String getOrderId() {
 return orderId;
 }
 public void setOrderId(String orderId) {
 this.orderId = orderId;
 }

 // First partition key attribute for the GSI.
 @DynamoDbSecondaryPartitionKey(indexNames = "OrdersByStatusDateAmount", order =
Order.FIRST)
 public String getCustomerId() {
 return customerId;
 }
 public void setCustomerId(String customerId) {
 this.customerId = customerId;
 }

 // Second partition key attribute for the GSI.
 @DynamoDbSecondaryPartitionKey(indexNames = "OrdersByStatusDateAmount", order =
Order.SECOND)
 public String getStatus() {
 return status;
 }
 public void setStatus(String status) {
 this.status = status;
 }

 // First sort key attribute for the GSI.
 @DynamoDbSecondarySortKey(indexNames = "OrdersByStatusDateAmount", order =
Order.FIRST)
 public String getOrderDate() {
 return orderDate;
 }
 public void setOrderDate(String orderDate) {
 this.orderDate = orderDate;
 }
}
```

```

 // Second sort key attribute for the GSI.
 @DynamoDbSecondarySortKey(indexNames = "OrdersByStatusDateAmount", order =
Order.SECOND)
 public Integer getAmount() {
 return amount;
 }
 public void setAmount(Integer amount) {
 this.amount = amount;
 }
}

```

## Query by using a composite key index

When you query a composite key index, use the `addPartitionValue()` and `addSortValue()` methods on the [Key](#) builder. Add values in the same order as the `Order` values defined in the annotations. All partition keys are required. Sort keys are optional and can be provided from left to right.

You create a [QueryConditional](#) object to specify the condition for the query. Use `keyEqualTo` for exact matches, or a sort key condition such as `sortGreaterThan`, `sortLessThan`, `sortBetween`, or `sortBeginsWith`. The condition applies to the last sort key you provide.

The following examples query the *OrdersByStatusDateAmount* GSI, progressively adding sort keys to narrow the results.

```

DynamoDbIndex<OrderItem> index = orderTable.index("OrdersByStatusDateAmount");

// All PENDING orders for customer-123.
SdkIterable<Page<OrderItem>> allPending = index.query(q -> q
 .queryConditional(QueryConditional.keyEqualTo(k -> k
 .addPartitionValue("customer-123")
 .addPartitionValue("PENDING"))));

// PENDING orders on a specific date.
SdkIterable<Page<OrderItem>> pendingOnDate = index.query(q -> q
 .queryConditional(QueryConditional.keyEqualTo(k -> k
 .addPartitionValue("customer-123")
 .addPartitionValue("PENDING")
 .addSortValue("2025-11-04"))));

// PENDING orders on a specific date with amount greater than 100.

```



```
SdkIterable<Page<OrderItem>> filtered = index.query(q -> q
 .queryConditional(QueryConditional.sortGreaterThan(k -> k
 .addPartitionValue("customer-123")
 .addPartitionValue("PENDING")
 .addSortValue("2025-11-04")
 .addSortValue(100)))));
```

For more information, see [Multi-key support for Global Secondary Index in DynamoDB](#).

## Use advanced mapping features

Learn about advanced table schema features in the DynamoDB Enhanced Client API.

### Understand table schema types

[TableSchema](#) is the interface to the mapping functionality of the DynamoDB Enhanced Client API. It can map a data object to and from a map of [AttributeValues](#). A TableSchema object needs to know about the structure of the table it is mapping. This structure information is stored in a [TableMetadata](#) object.

The enhanced client API has several implementations of TableSchema, which follow.

### Table schema generated from annotated classes

It is a moderately expensive operation to build a TableSchema from annotated classes, so we recommend doing this once, at application startup.

#### [BeanTableSchema](#)

This implementation is built based on attributes and annotations of a bean class. An example of this approach is demonstrated in the [Get started section](#).

#### Note

If a BeanTableSchema is not behaving as you expect, enable debug logging for `software.amazon.awssdk.enhanced.dynamodb.beans`.

#### [ImmutableTableSchema](#)

This implementation is built from an immutable data class. This approach is described in the [??? section](#).

## Table schema generated with a builder

The following `TableSchemas` are built from code by using a builder. This approach is less costly than the approach that uses annotated data classes. The builder approach avoids the use of annotations and doesn't require JavaBean naming standards.

### [StaticTableSchema](#)

This implementation is built for mutable data classes. The getting started section of this guide demonstrated how to [generate a `StaticTableSchema` using a builder](#).

### [StaticImmutableTableSchema](#)

Similarly to how you build a `StaticTableSchema`, you generate an implementation of this type of `TableSchema` using a [builder](#) for use with immutable data classes.

## Table schema for data without a fixed schema

### [DocumentTableSchema](#)

Unlike other implementations of `TableSchema`, you don't define attributes for a `DocumentTableSchema` instance. Usually, you specify only primary keys and attribute converter providers. An `EnhancedDocument` instance provides the attributes that you build from individual elements or from a JSON string.

## Explicitly include or exclude attributes

The DynamoDB Enhanced Client API offers annotations to exclude data class attributes from becoming attributes on a table. With the API, you can also use an attribute name that's different from the data class attribute name.

### Exclude attributes

To ignore attributes that should not be mapped to a DynamoDB table, mark the attribute with the `@DynamoDbIgnore` annotation.

```
private String internalKey;

@DynamoDbIgnore
public String getInternalKey() { return this.internalKey; }
public void setInternalKey(String internalKey) { this.internalKey = internalKey;}
```

## Include attributes

To change the name of an attribute used in the DynamoDB table, mark it with the `@DynamoDbAttribute` annotation and supply a different name.

```
private String internalKey;

@dynamoDbAttribute("renamedInternalKey")
public String getInternalKey() { return this.internalKey; }
public void setInternalKey(String internalKey) { this.internalKey = internalKey;}
```

## Control attribute conversion

By default, a table schema provides converters for many common Java types through a default implementation of the [AttributeConverterProvider](#) interface. You can change the overall default behavior with a custom `AttributeConverterProvider` implementation. You can also change the converter for a single attribute.

For a list of available converters, see the [AttributeConverter](#) interface Java doc.

## Provide custom attribute converter providers

You can provide a single `AttributeConverterProvider` or a chain of ordered `AttributeConverterProviders` through the `@DynamoDbBean (converterProviders = {...})` annotation. Any custom `AttributeConverterProvider` must extend the `AttributeConverterProvider` interface.

Note that if you supply your own chain of attribute converter providers, you will override the default converter provider, `DefaultAttributeConverterProvider`. If you want to use the functionality of the `DefaultAttributeConverterProvider`, you must include it in the chain.

It's also possible to annotate the bean with an empty array `{}`. This disables the use of any attribute converter providers, including the default. In this case all attributes that are to be mapped must have their own attribute converter.

The following snippet shows a single converter provider.

```
@DynamoDbBean(converterProviders = ConverterProvider1.class)
public class Customer {
```

```
}
```

The following snippet shows the use of a chain of converter providers. Since the SDK default is provided last, it has the lowest priority.

```
@DynamoDbBean(converterProviders = {
 ConverterProvider1.class,
 ConverterProvider2.class,
 DefaultAttributeConverterProvider.class})
public class Customer {

}
```

The static table schema builders have an `attributeConverterProviders()` method that works the same way. This is shown in the following snippet.

```
private static final StaticTableSchema<Customer> CUSTOMER_TABLE_SCHEMA =
 StaticTableSchema.builder(Customer.class)
 .newItemSupplier(Customer::new)
 .addAttribute(String.class, a -> a.name("name")
 a.getter(Customer::getName)
 a.setter(Customer::setName))
 .attributeConverterProviders(converterProvider1, converterProvider2)
 .build();
```

## Override the mapping of a single attribute

To override the way a single attribute is mapped, supply an `AttributeConverter` for the attribute. This addition overrides any converters provided by `AttributeConverterProviders` in the table schema. This adds a custom converter for only that attribute. Other attributes, even those of the same type, won't use that converter unless it is explicitly specified for those other attributes.

The `@DynamoDbConvertedBy` annotation is used to specify the custom `AttributeConverter` class as shown in the following snippet.

```
@DynamoDbBean
public class Customer {
 private String name;

 @DynamoDbConvertedBy(CustomAttributeConverter.class)
```

```
public String getName() { return this.name; }
public void setName(String name) { this.name = name;}
}
```

The builders for static schemas have an equivalent attribute builder `attributeConverter()` method. This method takes an instance of an `AttributeConverter` as the following shows.

```
private static final StaticTableSchema<Customer> CUSTOMER_TABLE_SCHEMA =
 StaticTableSchema.builder(Customer.class)
 .newItemSupplier(Customer::new)
 .addAttribute(String.class, a -> a.name("name")
 a.getter(Customer::getName)
 a.setter(Customer::setName)
 a.attributeConverter(customAttributeConverter))
 .build();
```

## Example

This example shows an `AttributeConverterProvider` implementation that provides an attribute converter for [java.net.HttpCookie](http://java.net/HttpCookie) objects.

The following `SimpleUser` class contains an attribute named `lastUsedCookie` that is an instance of `HttpCookie`.

The parameter to the `@DynamoDbBean` annotations lists the two `AttributeConverterProvider` classes that provide converters.

## Class with annotations

```
@DynamoDbBean(converterProviders = {CookieConverterProvider.class,
DefaultAttributeConverterProvider.class})
public static final class SimpleUser {
 private String name;
 private HttpCookie lastUsedCookie;

 @DynamoDbPartitionKey
 public String getName() {
 return name;
 }

 public void setName(String name) {
 this.name = name;
 }
}
```

```

 }

 public HttpCookie getLastUsedCookie() {
 return lastUsedCookie;
 }

 public void setLastUsedCookie(HttpCookie lastUsedCookie) {
 this.lastUsedCookie = lastUsedCookie;
 }

```

## Static table schema

```

private static final TableSchema<SimpleUser> SIMPLE_USER_TABLE_SCHEMA =
 TableSchema.builder(SimpleUser.class)
 .newItemSupplier(SimpleUser::new)
 .attributeConverterProviders(CookieConverterProvider.create(),
AttributeConverterProvider.defaultProvider())
 .addAttribute(String.class, a -> a.name("name")
 .setter(SimpleUser::setName)
 .getter(SimpleUser::getName)
 .tags(StaticAttributeTags.primaryPartitionKey()))
 .addAttribute(HttpCookie.class, a -> a.name("lastUsedCookie")
 .setter(SimpleUser::setLastUsedCookie)
 .getter(SimpleUser::getLastUsedCookie))
 .build();

```

The `CookieConverterProvider` in the following example provides an instance of an `HttpCookieConverter`.

```

public static final class CookieConverterProvider implements
AttributeConverterProvider {
 private final Map<EnhancedType<?>, AttributeConverter<?>> converterCache =
ImmutableMap.of(
 // 1. Add HttpCookieConverter to the internal cache.
 EnhancedType.of(HttpCookie.class), new HttpCookieConverter());

 public static CookieConverterProvider create() {
 return new CookieConverterProvider();
 }

 // The SDK calls this method to find out if the provider contains a
 AttributeConverter instance

```

```

 // for the EnhancedType<T> argument.
 @SuppressWarnings("unchecked")
 @Override
 public <T> AttributeConverter<T> converterFor(EnhancedType<T> enhancedType) {
 return (AttributeConverter<T>) converterCache.get(enhancedType);
 }
 }
}

```

## Conversion code

In the `transformFrom()` method of the following `HttpCookieConverter` class, the code receives an `HttpCookie` instance and transforms it into a DynamoDB map that is stored as an attribute.

The `transformTo()` method receives a DynamoDB map parameter, then invokes the `HttpCookie` constructor that requires a name and a value.

```

public static final class HttpCookieConverter implements
AttributeConverter<HttpCookie> {

 @Override
 public AttributeValue transformFrom(HttpCookie httpCookie) {

 return AttributeValue.fromM(
 Map.of ("cookieName", AttributeValue.fromS(httpCookie.getName()),
 "cookieValue", AttributeValue.fromS(httpCookie.getValue()))
);
 }

 @Override
 public HttpCookie transformTo(AttributeValue attributeValue) {
 Map<String, AttributeValue> map = attributeValue.m();
 return new HttpCookie(
 map.get("cookieName").s(),
 map.get("cookieValue").s());
 }

 @Override
 public EnhancedType<HttpCookie> type() {
 return EnhancedType.of(HttpCookie.class);
 }

 @Override

```

```

 public AttributeValueType attributeValueType() {
 return AttributeValueType.M;
 }
 }
}

```

## Change update behavior of attributes

You can customize the update behavior of individual attributes when you perform an *update* operation. Some examples of update operations in the DynamoDB Enhanced Client API are [updateItem\(\)](#) and [transactWriteItems\(\)](#).

For example, imagine that you want to store a *created on* timestamp on your record. However, you want its value to be written only if there's no existing value for the attribute already in the database. In this case, you use the [WRITE\\_IF\\_NOT\\_EXISTS](#) update behavior.

The following example shows the annotation that adds the behavior to the `createdOn` attribute.

```

@DynamoDbBean
public class Customer extends GenericRecord {
 private String id;
 private Instant createdOn;

 @DynamoDbPartitionKey
 public String getId() { return this.id; }
 public void setId(String id) { this.name = id; }

 @DynamoDbUpdateBehavior(UpdateBehavior.WRITE_IF_NOT_EXISTS)
 public Instant getCreatedOn() { return this.createdOn; }
 public void setCreatedOn(Instant createdOn) { this.createdOn = createdOn; }
}

```

You can declare the same update behavior when you build a static table schema as shown in the following example after comment line 1.

```

static final TableSchema<Customer> CUSTOMER_TABLE_SCHEMA =
 TableSchema.builder(Customer.class)
 .newItemSupplier(Customer::new)
 .addAttribute(String.class, a -> a.name("id")
 .getter(Customer::getId)
 .setter(Customer::setId))

 .tags(StaticAttributeTags.primaryPartitionKey()))

```



```

 .addAttribute(Instant.class, a -> a.name("createdOn")
 .getter(Customer::getCreatedOn)
 .setter(Customer::setCreatedOn)
 // 1. Add an UpdateBehavior.

 .tags(StaticAttributeTags.updateBehavior(UpdateBehavior.WRITE_IF_NOT_EXISTS)))
 .build();

```

## Flatten attributes from other classes

If the attributes for your table are spread across several different Java classes, either through inheritance or composition, the DynamoDB Enhanced Client API provides support to flatten the attributes into one class.

### Use inheritance

If your classes use inheritance, use the following approaches to flatten the hierarchy.

### Use annotated beans

For the annotation approach, both classes must carry the `@DynamoDbBean` annotation and a class must carry one or more primary key annotations.

The following shows examples of data classes that have an inheritance relationship.

#### Standard data class

```

@DynamoDbBean
public class Customer extends GenericRecord {
 private String name;

 public String getName() { return name; }
 public void setName(String name) { this.name = name; }
}

@DynamoDbBean
public abstract class GenericRecord {
 private String id;
 private String createdAt;

 @DynamoDbPartitionKey
 public String getId() { return id; }
 public void setId(String id) { this.id = id; }
}

```

```

 public String getCreatedDate() { return createdDate; }
 public void setCreatedDate(String createdDate) { this.createdDate =
createdDate; }
}

```

## Lombok

Lombok's [onMethod option](#) copies attribute-based DynamoDB annotations, such as `@DynamoDbPartitionKey`, onto the generated code.

```

@DynamoDbBean
@Data
@ToString(callSuper = true)
public class Customer extends GenericRecord {
 private String name;
}

@Data
@DynamoDbBean
public abstract class GenericRecord {
 @Getter(onMethod_=@DynamoDbPartitionKey)
 private String id;
 private String createdDate;
}

```

## Use static schemas

For the static schema approach, use the `extend()` method of the builder to collapse the attributes of the parent class onto the child class. This is shown after comment line 1 in the following example.

```

 StaticTableSchema<org.example.tests.model.inheritance.stat.GenericRecord>
GENERIC_RECORD_SCHEMA =

StaticTableSchema.builder(org.example.tests.model.inheritance.stat.GenericRecord.class)
 // The partition key will be inherited by the top level mapper.
 .addAttribute(String.class, a -> a.name("id"))

 .getter(org.example.tests.model.inheritance.stat.GenericRecord::getId)

 .setter(org.example.tests.model.inheritance.stat.GenericRecord::setId)

```

```

 .tags(primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("created_date"))

getter(org.example.tests.model.inheritance.stat.GenericRecord::getCreatedDate)

setter(org.example.tests.model.inheritance.stat.GenericRecord::setCreatedDate))
 .build();

StaticTableSchema<org.example.tests.model.inheritance.stat.Customer>
CUSTOMER_SCHEMA =

StaticTableSchema.builder(org.example.tests.model.inheritance.stat.Customer.class)

.newItemSupplier(org.example.tests.model.inheritance.stat.Customer::new)
 .addAttribute(String.class, a -> a.name("name"))

getter(org.example.tests.model.inheritance.stat.Customer::getName)

setter(org.example.tests.model.inheritance.stat.Customer::setName))
 // 1. Use the extend() method to collapse the parent attributes
onto the child class.
 .extend(GENERIC_RECORD_SCHEMA) // All the attributes of the
GenericRecord schema are added to Customer.
 .build();

```

The previous static schema example uses the following data classes. Because the mapping is defined when you build the static table schema, the data classes don't require annotations.

## Data classes

### Standard data class

```

public class Customer extends GenericRecord {
 private String name;

 public String getName() { return name; }
 public void setName(String name) { this.name = name; }
}

public abstract class GenericRecord {
 private String id;
 private String createdDate;

```

```

public String getId() { return id; }
public void setId(String id) { this.id = id; }

public String getCreatedDate() { return createdDate; }
public void setCreatedDate(String createdDate) { this.createdDate =
createdDate; }

```

## Lombok

```

@Data
@ToString(callSuper = true)
public class Customer extends GenericRecord{
 private String name;
}

@Data
public abstract class GenericRecord {
 private String id;
 private String createdDate;
}

```

## Use composition

If your classes use composition, use the following approaches to flatten the hierarchy.

### Use annotated beans

The `@DynamoDbFlatten` annotation flattens the contained class.

The following data class examples use the `@DynamoDbFlatten` annotation to effectively add all attributes of the contained `GenericRecord` class to the `Customer` class.

#### Standard data class

```

@DynamoDbBean
public class Customer {
 private String name;
 private GenericRecord record;

 public String getName() { return this.name; }
}

```

```

 public void setName(String name) { this.name = name; }

 @DynamoDbFlatten
 public GenericRecord getRecord() { return this.record; }
 public void setRecord(GenericRecord record) { this.record = record; }

 @DynamoDbBean
 public class GenericRecord {
 private String id;
 private String createdAt;

 @DynamoDbPartitionKey
 public String getId() { return this.id; }
 public void setId(String id) { this.id = id; }

 public String getCreatedAt() { return this.createdAt; }
 public void setCreatedAt(String createdAt) { this.createdAt =
 createdAt; }
 }

```

## Lombok

```

@Data
@DynamoDbBean
public class Customer {
 private String name;
 @Getter(onMethod_=@DynamoDbFlatten)
 private GenericRecord record;
}

@Data
@DynamoDbBean
public class GenericRecord {
 @Getter(onMethod_=@DynamoDbPartitionKey)
 private String id;
 private String createdAt;
}

```

You can use the flatten annotation to flatten as many different eligible classes as you need to. The following constraints apply:

- All attribute names must be unique after they are flattened.

- There must never be more than one partition key, sort key, or table name.

## Use static schemas

When you build a static table schema, use the `flatten()` method of the builder. You also supply the getter and setter methods that identify the contained class.

```
StaticTableSchema<GenericRecord> GENERIC_RECORD_SCHEMA =
 StaticTableSchema.builder(GenericRecord.class)
 .newItemSupplier(GenericRecord::new)
 .addAttribute(String.class, a -> a.name("id")
 .getter(GenericRecord::getId)
 .setter(GenericRecord::setId)
 .tags(primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("created_date")
 .getter(GenericRecord::getCreatedDate)
 .setter(GenericRecord::setCreatedDate))
 .build();

StaticTableSchema<Customer> CUSTOMER_SCHEMA =
 StaticTableSchema.builder(Customer.class)
 .newItemSupplier(Customer::new)
 .addAttribute(String.class, a -> a.name("name")
 .getter(Customer::getName)
 .setter(Customer::setName))
 // Because we are flattening a component object, we supply a
getter and setter so the
 // mapper knows how to access it.
 .flatten(GENERIC_RECORD_SCHEMA, Customer::getRecord,
Customer::setRecord)
 .build();
```

The previous static schema example uses the following data classes.

## Data classes

### Standard data class

```
public class Customer {
 private String name;
 private GenericRecord record;
```

```

 public String getName() { return this.name; }
 public void setName(String name) { this.name = name; }

 public GenericRecord getRecord() { return this.record; }
 public void setRecord(GenericRecord record) { this.record = record; }

 public class GenericRecord {
 private String id;
 private String createdAt;

 public String getId() { return this.id; }
 public void setId(String id) { this.id = id; }

 public String getCreatedAt() { return this.createdAt; }
 public void setCreatedAt(String createdAt) { this.createdAt =
 createdAt; }
 }

```

## Lombok

```

@Data
public class Customer {
 private String name;
 private GenericRecord record;
}

@Data
public class GenericRecord {
 private String id;
 private String createdAt;
}

```

You can use the builder pattern to flatten as many different eligible classes as you need to.

## Implications for other code

When you use the `@DynamoDbFlatten` attribute (or `flatten()` builder method), the item in DynamoDB contains an attribute for each attribute of the composed object. It also includes the attributes of the composing object.

In contrast, if you annotate a data class with a composed class and don't use `@DynamoDbFlatten`, the item is saved with the composed object as a single attribute.

For example, compare the `Customer` class shown in the [flattening with composition example](#) with and without flattening of the `record` attribute. You can visualize the difference with JSON as shown in the following table.

With flattening	Without flattening
3 attributes	2 attributes
<pre>{   "id": "1",   "createdDate": "today",   "name": "my name" }</pre>	<pre>{   "id": "1",   "record": {     "createdDate": "today",     "name": "my name"   } }</pre>

The difference becomes important if you have other code accessing the DynamoDB table that expects to find certain attributes.

### Work with attributes that are beans, maps, lists and sets

A bean definition, such as the `Person` class shown below, might define properties (or attributes) that refer to types with additional attributes. For instance, in the `Person` class, `mainAddress` is a property that refers to an `Address` bean that defines additional value attributes. `addresses` refers to a Java Map, whose elements refer to `Address` beans. These complex types can be thought of as containers of simple attributes that you use for their data value in the context of DynamoDB.

DynamoDB refers to the value properties of nested elements, such as maps, lists, or beans, as *nested attributes*. The [Amazon DynamoDB Developer Guide](#) refers to the saved form of a Java map, list or bean as a *document type*. Simple attributes that you use for their data value in Java are referred to as *scalar types* in DynamoDB. Sets, which contains multiple scalar elements of the same type, and referred to as *set types*.

It is important to know that the DynamoDB Enhanced Client API converts a property that is bean to a DynamoDB map document type when it is save.



## Person class

```
@DynamoDbBean
public class Person {
 private Integer id;
 private String firstName;
 private String lastName;
 private Integer age;
 private Address mainAddress;
 private Map<String, Address> addresses;
 private List<PhoneNumber> phoneNumbers;
 private Set<String> hobbies;

 @DynamoDbPartitionKey
 public Integer getId() {
 return id;
 }

 public void setId(Integer id) {
 this.id = id;
 }

 public String getFirstName() {
 return firstName;
 }

 public void setFirstName(String firstName) {
 this.firstName = firstName;
 }

 public String getLastName() {
 return lastName;
 }

 public void setLastName(String lastName) {
 this.lastName = lastName;
 }

 public Integer getAge() {
 return age;
 }

 public void setAge(Integer age) {
 this.age = age;
 }
}
```

```
}

public Address getMainAddress() {
 return mainAddress;
}

public void setMainAddress(Address mainAddress) {
 this.mainAddress = mainAddress;
}

public Map<String, Address> getAddresses() {
 return addresses;
}

public void setAddresses(Map<String, Address> addresses) {
 this.addresses = addresses;
}

public List<PhoneNumber> getPhoneNumbers() {
 return phoneNumbers;
}

public void setPhoneNumbers(List<PhoneNumber> phoneNumbers) {
 this.phoneNumbers = phoneNumbers;
}

public Set<String> getHobbies() {
 return hobbies;
}

public void setHobbies(Set<String> hobbies) {
 this.hobbies = hobbies;
}

@Override
public String toString() {
 return "Person{" +
 "addresses=" + addresses +
 ", id=" + id +
 ", firstName='" + firstName + '\'' +
 ", lastName='" + lastName + '\'' +
 ", age=" + age +
 ", mainAddress=" + mainAddress +
 ", phoneNumbers=" + phoneNumbers +
 }
}
```

```
 ", hobbies=" + hobbies +
 '}';
 }
}
```

## Address class

```
@DynamoDbBean
public class Address {
 private String street;
 private String city;
 private String state;
 private String zipCode;

 public Address() {
 }

 public String getStreet() {
 return this.street;
 }

 public String getCity() {
 return this.city;
 }

 public String getState() {
 return this.state;
 }

 public String getZipCode() {
 return this.zipCode;
 }

 public void setStreet(String street) {
 this.street = street;
 }

 public void setCity(String city) {
 this.city = city;
 }

 public void setState(String state) {
 this.state = state;
 }
}
```

```

 }

 public void setZipCode(String zipCode) {
 this.zipCode = zipCode;
 }

 @Override
 public boolean equals(Object o) {
 if (this == o) return true;
 if (o == null || getClass() != o.getClass()) return false;
 Address address = (Address) o;
 return Objects.equals(street, address.street) && Objects.equals(city,
address.city) && Objects.equals(state, address.state) && Objects.equals(zipCode,
address.zipCode);
 }

 @Override
 public int hashCode() {
 return Objects.hash(street, city, state, zipCode);
 }

 @Override
 public String toString() {
 return "Address{" +
 "street='" + street + '\'' +
 ", city='" + city + '\'' +
 ", state='" + state + '\'' +
 ", zipCode='" + zipCode + '\'' +
 '}';
 }
}

```

## PhoneNumber class

```

@DynamoDbBean
public class PhoneNumber {
 String type;
 String number;

 public String getType() {
 return type;
 }
}

```

```
public void setType(String type) {
 this.type = type;
}

public String getNumber() {
 return number;
}

public void setNumber(String number) {
 this.number = number;
}

@Override
public String toString() {
 return "PhoneNumber{" +
 "type='" + type + '\'' +
 ", number='" + number + '\'' +
 '}';
}
}
```

## Save complex types

### Use annotated data classes

You save nested attributes for custom classes by simply annotating them. The `Address` class and `PhoneNumber` class shown previously are annotated with only the `@DynamoDbBean` annotation. When the DynamoDB Enhanced Client API builds the table schema for the `Person` class with the following snippet, the API discovers the use of the `Address` and `PhoneNumber` classes and builds the corresponding mappings to work with DynamoDB.

```
TableSchema<Person> personTableSchema = TableSchema.fromBean(Person.class);
```

### Use abstract schemas with builders

The alternative approach is to use static table schema builders for each nested bean class as shown in the following code.

The table schemas for the `Address` and `PhoneNumber` classes are abstract in the sense that they cannot be used with a DynamoDB table. This is because they lack definitions for the primary key. They are used, however, as nested schemas in the table schema for the `Person` class.

After comment lines 1 and 2 in the definition of `PERSON_TABLE_SCHEMA`, you see the code that uses the abstract table schemas. The use of `documentOf` in the `EnhanceType.documentOf(...)` method does not indicate that the method returns an `EnhancedDocument` type of the Enhanced Document API. The `documentOf(...)` method in this context returns an object that knows how to map its class argument to and from DynamoDB table attributes by using the table schema argument.

## Static schema code

```
// Abstract table schema that cannot be used to work with a DynamoDB table,
// but can be used as a nested schema.
public static final TableSchema<Address> TABLE_SCHEMA_ADDRESS =
TableSchema.builder(Address.class)
 .newItemSupplier(Address::new)
 .addAttribute(String.class, a -> a.name("street")
 .getter(Address::getStreet)
 .setter(Address::setStreet))
 .addAttribute(String.class, a -> a.name("city")
 .getter(Address::getCity)
 .setter(Address::setCity))
 .addAttribute(String.class, a -> a.name("zipcode")
 .getter(Address::getZipCode)
 .setter(Address::setZipCode))
 .addAttribute(String.class, a -> a.name("state")
 .getter(Address::getState)
 .setter(Address::setState))
 .build();

// Abstract table schema that cannot be used to work with a DynamoDB table,
// but can be used as a nested schema.
public static final TableSchema<PhoneNumber> TABLE_SCHEMA_PHONENUMBER =
TableSchema.builder(PhoneNumber.class)
 .newItemSupplier(PhoneNumber::new)
 .addAttribute(String.class, a -> a.name("type")
 .getter(PhoneNumber::getType)
 .setter(PhoneNumber::setType))
 .addAttribute(String.class, a -> a.name("number")
 .getter(PhoneNumber::getNumber)
 .setter(PhoneNumber::setNumber))
 .build();

// A static table schema that can be used with a DynamoDB table.
```

```

// The table schema contains two nested schemas that are used to perform mapping
to/from DynamoDB.
public static final TableSchema<Person> PERSON_TABLE_SCHEMA =
 TableSchema.builder(Person.class)
 .newItemSupplier(Person::new)
 .addAttribute(Integer.class, a -> a.name("id")
 .getter(Person::getId)
 .setter(Person::setId)
 .addTag(StaticAttributeTags.primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("firstName")
 .getter(Person::getFirstName)
 .setter(Person::setFirstName))
 .addAttribute(String.class, a -> a.name("lastName")
 .getter(Person::getLastName)
 .setter(Person::setLastName))
 .addAttribute(Integer.class, a -> a.name("age")
 .getter(Person::getAge)
 .setter(Person::setAge))
 .addAttribute(EnhancedType.documentOf(Address.class, TABLE_SCHEMA_ADDRESS),
a -> a.name("mainAddress")
 .getter(Person::getMainAddress)
 .setter(Person::setMainAddress))
 .addAttribute(EnhancedType.listOf(String.class), a -> a.name("hobbies")
 .getter(Person::getHobbies)
 .setter(Person::setHobbies))
 .addAttribute(EnhancedType.mapOf(
 EnhancedType.of(String.class),
 // 1. Use mapping functionality of the Address table schema.
 EnhancedType.documentOf(Address.class, TABLE_SCHEMA_ADDRESS)), a ->
a.name("addresses")
 .getter(Person::getAddresses)
 .setter(Person::setAddresses))
 .addAttribute(EnhancedType.listOf(
 // 2. Use mapping functionality of the PhoneNumber table schema.
 EnhancedType.documentOf(PhoneNumber.class, TABLE_SCHEMA_PHONENUMBER)),
a -> a.name("phoneNumbers")
 .getter(Person::getPhoneNumbers)
 .setter(Person::setPhoneNumbers))
 .build();

```

## Project attributes of complex types

For `query()` and `scan()` methods, you can specify which attributes you want to be returned in the results by using method calls such as `addNestedAttributeToProject()` and `attributesToProject()`. The DynamoDB Enhanced Client API converts the Java method call parameters into [projection expressions](#) before the request is sent.

The following example populates the `Person` table with two items, then performs three scan operations.

The first scan accesses all items in the table in order to compare the results to the other scan operations.

The second scan uses the [addNestedAttributeToProject\(\)](#) builder method to return only the street attribute value.

The third scan operation uses the [attributesToProject\(\)](#) builder method to return the data for the first-level attribute, hobbies. The attribute type of hobbies is a list. To access individual list items, perform a `get()` operation on the list.

```

 personDynamoDbTable = getDynamoDbEnhancedClient().table("Person",
PERSON_TABLE_SCHEMA);
 PersonUtils.createPersonTable(personDynamoDbTable, getDynamoDbClient());
 // Use a utility class to add items to the Person table.
 List<Person> personList = PersonUtils.getItemsForCount(2);
 // This utility method performs a put against DynamoDB to save the instances in
the list argument.
 PersonUtils.putCollection(getDynamoDbEnhancedClient(), personList,
personDynamoDbTable);

 // The first scan logs all items in the table to compare to the results of the
subsequent scans.
 final PageIterable<Person> allItems = personDynamoDbTable.scan();
 allItems.items().forEach(p ->
 // 1. Log what is in the table.
 logger.info(p.toString()));

 // Scan for nested attributes.
 PageIterable<Person> streetScanResult = personDynamoDbTable.scan(b -> b
 // Use the 'addNestedAttributeToProject()' or
'addNestedAttributesToProject()' to access data nested in maps in DynamoDB.
 .addNestedAttributeToProject(

```



```

 NestedAttributeName.create("addresses", "work", "street")
));

 streetScanResult.items().forEach(p ->
 //2. Log the results of requesting nested attributes.
 logger.info(p.toString()));

 // Scan for a top-level list attribute.
 PageIterable<Person> hobbiesScanResult = personDynamoDbTable.scan(b -> b
 // Use the 'attributesToProject()' method to access first-level
attributes.

 .attributesToProject("hobbies"));

 hobbiesScanResult.items().forEach((p) -> {
 // 3. Log the results of the request for the 'hobbies' attribute.
 logger.info(p.toString());
 // To access an item in a list, first get the parent attribute, 'hobbies',
then access items in the list.
 String hobby = p.getHobbies().get(1);
 // 4. Log an item in the list.
 logger.info(hobby);
 });

```

// Logged results from comment line 1.

```

Person{id=2, firstName='first name 2', lastName='last name 2', age=11,
 addresses={work=Address{street='street 21', city='city 21', state='state 21',
zipCode='33333'}, home=Address{street='street 2', city='city 2', state='state 2',
zipCode='22222'}}, phoneNumbers=[PhoneNumber{type='home', number='222-222-2222'},
PhoneNumber{type='work', number='333-333-3333'}], hobbies=[hobby 2, hobby 21]}
Person{id=1, firstName='first name 1', lastName='last name 1', age=11,
 addresses={work=Address{street='street 11', city='city 11', state='state 11',
zipCode='22222'}, home=Address{street='street 1', city='city 1', state='state 1',
zipCode='11111'}}, phoneNumbers=[PhoneNumber{type='home', number='111-111-1111'},
PhoneNumber{type='work', number='222-222-2222'}], hobbies=[hobby 1, hobby 11]}

```

// Logged results from comment line 2.

```

Person{id=null, firstName='null', lastName='null', age=null,
 addresses={work=Address{street='street 21', city='null', state='null',
zipCode='null'}}}, phoneNumbers=null, hobbies=null}
Person{id=null, firstName='null', lastName='null', age=null,
 addresses={work=Address{street='street 11', city='null', state='null',
zipCode='null'}}}, phoneNumbers=null, hobbies=null}

```

```
// Logged results from comment lines 3 and 4.
Person{id=null, firstName='null', lastName='null', age=null, addresses=null,
 phoneNumbers=null, hobbies=[hobby 2, hobby 21]}
hobby 21
Person{id=null, firstName='null', lastName='null', age=null, addresses=null,
 phoneNumbers=null, hobbies=[hobby 1, hobby 11]}
hobby 11
```

### Note

If the `attributesToProject()` method follows any other builder method that adds attributes that you want to project, the list of attribute names supplied to the `attributesToProject()` replaces all other attribute names.

A scan performed with the `ScanEnhancedRequest` instance in the following snippet returns only hobby data.

```
ScanEnhancedRequest lastOverwrites = ScanEnhancedRequest.builder()
 .addNestedAttributeToProject(
 NestedAttributeName.create("addresses", "work", "street"))
 .addAttributeToProject("firstName")
 // If the 'attributesToProject()' method follows other builder methods
 that add attributes for projection,
 // its list of attributes replace all previous attributes.
 .attributesToProject("hobbies")
 .build();

PageIterable<Person> hobbiesOnlyResult =
 personDynamoDbTable.scan(lastOverwrites);
hobbiesOnlyResult.items().forEach(p ->
 logger.info(p.toString()));

// Logged results.
Person{id=null, firstName='null', lastName='null', age=null, addresses=null,
 phoneNumbers=null, hobbies=[hobby 2, hobby 21]}
Person{id=null, firstName='null', lastName='null', age=null, addresses=null,
 phoneNumbers=null, hobbies=[hobby 1, hobby 11]}
```

The following code snippet uses the `attributesToProject()` method first. This ordering preserves all other requested attributes.

```
ScanEnhancedRequest attributesPreserved = ScanEnhancedRequest.builder()
```

```

 // Use 'attributesToProject()' first so that the method call does not
 // replace all other attributes
 // that you want to project.
 .attributesToProject("firstName")
 .addNestedAttributeToProject(
 NestedAttributeName.create("addresses", "work", "street"))
 .addAttributeToProject("hobbies")
 .build();
PageIterable<Person> allAttributesResult =
 personDynamoDbTable.scan(attributesPreserved);
allAttributesResult.items().forEach(p ->
 logger.info(p.toString()));

// Logged results.
Person{id=null, firstName='first name 2', lastName='null', age=null,
 addresses={work=Address{street='street 21', city='null', state='null',
 zipCode='null'}}}, phoneNumbers=null, hobbies=[hobby 2, hobby 21]}
Person{id=null, firstName='first name 1', lastName='null', age=null,
 addresses={work=Address{street='street 11', city='null', state='null',
 zipCode='null'}}}, phoneNumbers=null, hobbies=[hobby 1, hobby 11]}

```

## Use complex types in expressions

You can use complex types in expressions, such as filter expressions and condition expressions, by using dereferencing operators to navigate the structure of the complex type. For objects and maps, use the `.` (dot) and for list elements use `[n]` (square brackets around the sequence number of the element). You can't refer to individual elements of a set, but you can use the [contains function](#).

The following example show two filter expressions that are used in scan operations. The filter expressions specify the match conditions for items you want in the results. The example uses `Person`, `Address`, and `PhoneNumber` classes shown previously.

```

public void scanUsingFilterOfNestedAttr() {
 // The following is a filter expression for an attribute that is a map of
 // Address objects.
 // By using this filter expression, the SDK returns Person objects that have an
 // address
 // with 'mailing' as a key and 'MS2' for a state value.
 Expression addressFilter = Expression.builder()
 .expression("addresses.#type.#field = :value")

```

```

 .putExpressionName("#type", "mailing")
 .putExpressionName("#field", "state")
 .putExpressionValue(":value",
AttributeValue.builder().s("MS2").build())
 .build();

 PageIterable<Person> addressFilterResults = personDynamoDbTable.scan(rb -> rb.
 filterExpression(addressFilter));
 addressFilterResults.items().stream().forEach(p -> logger.info("Person: {}",
p));

 assert addressFilterResults.items().stream().count() == 1;

 // The following is a filter expression for an attribute that is a list of
phone numbers.
 // By using this filter expression, the SDK returns Person objects whose second
phone number
 // in the list has a type equal to 'cell'.
 Expression phoneFilter = Expression.builder()
 .expression("phoneNumbers[1].#type = :type")
 .putExpressionName("#type", "type")
 .putExpressionValue(":type",
AttributeValue.builder().s("cell").build())
 .build();

 PageIterable<Person> phoneFilterResults = personDynamoDbTable.scan(rb -> rb
 .filterExpression(phoneFilter)
 .attributesToProject("id", "firstName", "lastName", "phoneNumbers")
);

 phoneFilterResults.items().stream().forEach(p -> logger.info("Person: {}", p));

 assert phoneFilterResults.items().stream().count() == 1;
 assert
phoneFilterResults.items().stream().findFirst().get().getPhoneNumbers().get(1).getType().equal
}

```

## Helper method that populates the table

```

public static void populateDatabase() {
 Person person1 = new Person();
 person1.setId(1);
}

```

```
person1.setFirstName("FirstName1");
person1.setLastName("LastName1");

Address billingAddr1 = new Address();
billingAddr1.setState("BS1");
billingAddr1.setCity("BillingTown1");

Address mailing1 = new Address();
mailing1.setState("MS1");
mailing1.setCity("MailingTown1");

person1.setAddresses(Map.of("billing", billingAddr1, "mailing", mailing1));

PhoneNumber pn1_1 = new PhoneNumber();
pn1_1.setType("work");
pn1_1.setNumber("111-111-1111");

PhoneNumber pn1_2 = new PhoneNumber();
pn1_2.setType("home");
pn1_2.setNumber("222-222-2222");

List<PhoneNumber> phoneNumbers1 = List.of(pn1_1, pn1_2);
person1.setPhoneNumbers(phoneNumbers1);

personDynamoDbTable.putItem(person1);

Person person2 = person1;
person2.setId(2);
person2.setFirstName("FirstName2");
person2.setLastName("LastName2");

Address billingAddress2 = billingAddr1;
billingAddress2.setCity("BillingTown2");
billingAddress2.setState("BS2");

Address mailing2 = mailing1;
mailing2.setCity("MailingTown2");
mailing2.setState("MS2");

person2.setAddresses(Map.of("billing", billingAddress2, "mailing", mailing2));

PhoneNumber pn2_1 = new PhoneNumber();
pn2_1.setType("work");
pn2_1.setNumber("333-333-3333");
```

```
 PhoneNumber pn2_2 = new PhoneNumber();
 pn2_2.setType("cell");
 pn2_2.setNumber("444-444-4444");

 List<PhoneNumber> phoneNumbers2 = List.of(pn2_1, pn2_2);
 person2.setPhoneNumbers(phoneNumbers2);

 personDynamoDbTable.putItem(person2);
}
```

## JSON representation of items in the database

```
{
 "id": 1,
 "addresses": {
 "billing": {
 "city": "BillingTown1",
 "state": "BS1",
 "street": null,
 "zipCode": null
 },
 "mailing": {
 "city": "MailingTown1",
 "state": "MS1",
 "street": null,
 "zipCode": null
 }
 },
 "firstName": "FirstName1",
 "lastName": "LastName1",
 "phoneNumbers": [
 {
 "number": "111-111-1111",
 "type": "work"
 },
 {
 "number": "222-222-2222",
 "type": "home"
 }
]
}
```

```
{
 "id": 2,
 "addresses": {
 "billing": {
 "city": "BillingTown2",
 "state": "BS2",
 "street": null,
 "zipCode": null
 },
 "mailing": {
 "city": "MailingTown2",
 "state": "MS2",
 "street": null,
 "zipCode": null
 }
 },
 "firstName": "FirstName2",
 "lastName": "LastName2",
 "phoneNumbers": [
 {
 "number": "333-333-3333",
 "type": "work"
 },
 {
 "number": "444-444-4444",
 "type": "cell"
 }
]
}
```

## Update items that contain complex types

To update an item that contains complex types, you have two basic approaches:

- Approach 1: First retrieve the item (by using `getItem`), update the object, then call `DynamoDbTable#updateItem`.
- Approach 2: Don't retrieve the item, but construct a new instance, set the properties you want to update, and submit the instance to `DynamoDbTable#updateItem` by setting the appropriate value of [IgnoreNullsMode](#). This approach does not require that you fetch the item before updating it.

The examples shown in this section use the `Person`, `Address`, and `PhoneNumber` classes shown previously.

### Update approach 1: retrieve, then update

By using this approach, you ensure that no data is lost on update. The DynamoDB Enhanced Client API recreates the bean with the attributes from the item saved in DynamoDB including values of complex types. You then need to use the getters and setters to update the bean. The downside of this approach is the cost you incur retrieving the item first.

The following example demonstrates that no data is lost if you first retrieve the item before updating it.

```
public void retrieveThenUpdateExample() {
 // Assume that we ran this code yesterday.
 Person person = new Person();
 person.setId(1);
 person.setFirstName("FirstName");
 person.setLastName("LastName");

 Address mainAddress = new Address();
 mainAddress.setStreet("123 MyStreet");
 mainAddress.setCity("MyCity");
 mainAddress.setState("MyState");
 mainAddress.setZipCode("MyZipCode");
 person.setMainAddress(mainAddress);

 PhoneNumber homePhone = new PhoneNumber();
 homePhone.setNumber("1111111");
 homePhone.setType("HOME");
 person.setPhoneNumbers(List.of(homePhone));

 personDynamoDbTable.putItem(person);

 // Assume that we are running this code now.
 // First, retrieve the item
 Person retrievedPerson =
personDynamoDbTable.getItem(Key.builder().partitionValue(1).build());

 // Make any updates.
 retrievedPerson.getMainAddress().setCity("YourCity");
}
```



```

 // Save the updated bean. 'updateItem' returns the bean as it appears after the
 update.
 Person updatedPerson = personDynamoDbTable.updateItem(retrievedPerson);

 // Verify for this example.
 Address updatedMainAddress = updatedPerson.getMainAddress();
 assert updatedMainAddress.getCity().equals("YourCity");
 assert updatedMainAddress.getState().equals("MyState"); // Unchanged.
 // The list of phone numbers remains; it was not set to null;
 assert updatedPerson.getPhoneNumbers().size() == 1;
}

```

## Update approach 2: Use an IgnoreNullsMode enum without retrieving the item first

To update an item in DynamoDB, you can provide a new object that has only the properties that you want updated and leave the other values as null. With this approach, you need to be aware of how null values in the object are treated by the SDK and how you can control the behavior.

To specify which null-valued properties you want the SDK to ignore, provide an IgnoreNullsMode enum when you build the [UpdateItemEnhancedRequest](#). As an example of using one of the enumerated values, the following snippet uses the IgnoreNullsMode.SCALAR\_ONLY mode.

```

// Create a new Person object to update the existing item in DynamoDB.
Person personForUpdate = new Person();
personForUpdate.setId(1);
personForUpdate.setFirstName("updatedFirstName"); // 'firstName' is a top scalar
property.

Address addressForUpdate = new Address();
addressForUpdate.setCity("updatedCity");
personForUpdate.setMainAddress(addressForUpdate);

personDynamoDbTable.updateItem(r -> r
 .item(personForUpdate)
 .ignoreNullsMode(IgnoreNullsMode.SCALAR_ONLY));

/* With IgnoreNullsMode.SCALAR_ONLY provided, The SDK ignores all null properties. The
 SDK adds or replaces
 the 'firstName' property with the provided value, "updatedFirstName". The SDK updates
 the 'city' value of
 'mainAddress', as long as the 'mainAddress' attribute already exists in DynamoDB.

```

In the background, the SDK generates an update expression that it sends in the request to DynamoDB.

The following JSON object is a simplified version of what it sends. Notice that the SDK includes the paths

to 'mainAddress.city' and 'firstName' in the SET clause of the update expression. No null values in

'personForUpdate' are included.

```
{
 "TableName": "PersonTable",
 "Key": {
 "id": {
 "N": "1"
 }
 },
 "ReturnValues": "ALL_NEW",
 "UpdateExpression": "SET #mainAddress.#city = :mainAddress_city, #firstName
= :firstName",
 "ExpressionAttributeNames": {
 "#city": "city",
 "#firstName": "firstName",
 "#mainAddress": "mainAddress"
 },
 "ExpressionAttributeValues": {
 ":firstName": {
 "S": "updatedFirstName"
 },
 ":mainAddress_city": {
 "S": "updatedCity"
 }
 }
}
```

Had we chosen 'IgnoreNullsMode.DEFAULT' instead of 'IgnoreNullsMode.SCALAR\_ONLY', the SDK would have included

null values in the "ExpressionAttributeValues" section of the request as shown in the following snippet.

```
"ExpressionAttributeValues": {
 ":mainAddress": {
 "M": {
 "zipCode": {
 "NULL": true
 }
 },
```

```

 "city": {
 "S": "updatedCity"
 },
 "street": {
 "NULL": true
 },
 "state": {
 "NULL": true
 }
 },
 ":firstName": {
 "S": "updatedFirstName"
 }
}
*/

```

The Amazon DynamoDB Developer Guide contains more information about [update expressions](#).

### Descriptions of the `IgnoreNullsMode` options

- `IgnoreNullsMode.SCALAR_ONLY` - Use this setting to update scalar attributes at any level. The SDK constructs an update statement that sends only non-null, scalar attributes to DynamoDB. The SDK ignores null-valued, scalar attributes of a bean or map, retaining the saved value in DynamoDB.

When you update a scalar attribute of map or bean, the map must already exist in DynamoDB. If you add a map or a bean to the object that does not already exist for the object in DynamoDB, you get a `DynamoDbException` with the message *The document path provided in the update expression is invalid for update*. You must use `MAPS_ONLY` mode to add a bean or map to DynamoDB before you update any of its attributes.

- `IgnoreNullsMode.MAPS_ONLY` - Use this setting to add or replace properties that are a bean or a map. The SDK replaces or adds any map or bean provided in the object. Any beans or maps that are null in the object are ignored, retaining the map that exists in DynamoDB.
- `IgnoreNullsMode.DEFAULT` - With this setting, the SDK never ignores null values. Scalar attributes at any level that are null are updated to null. The SDK updates any null-valued bean, map, list, or set property in the object to null in DynamoDB. When you use this mode—or don't provide a mode since it's the default mode—you should retrieve the item first so that values in DynamoDB are not set to null that are provided in the object for updating, unless your intention is to set the values to null.

In all modes, if you provide an object to `updateItem` that has a non-null list or set, the list or set is saved to DynamoDB.

## Why the modes?

When you provide an object with a bean or map to the `updateItem` method, the SDK can't tell if it should use the property values in the bean (or entry values in the map) to update the item, or if the entire bean/map should replace what's been saved to DynamoDB.

Working from our previous example that shows the retrieval of the item first, let's attempt to update the `city` attribute of `mainAddress` without the retrieval.

```
/* The retrieval example saved the Person object with a 'mainAddress' property whose
 'city' property value is "MyCity".
 /* Note that we create a new Person with only the necessary information to update the
 city value
 of the mainAddress. */
Person personForUpdate = new Person();
personForUpdate.setId(1);
// The update we want to make changes the city.
Address mainAddressForUpdate = new Address();
mainAddressForUpdate.setCity("YourCity");
personForUpdate.setMainAddress(mainAddressForUpdate);
```

```
// Lets' try the following:
```

```
Person updatedPerson = personDynamoDbTable.updateItem(personForUpdate);
```

```
/*
```

Since we haven't retrieved the item, we don't know if the `'mainAddress'` property already exists, so what update expression should the SDK generate?

A) Should it replace or add the `'mainAddress'` with the provided object (setting all attributes to null other than `city`)  
as shown in the following simplified JSON?

```
{
 "TableName": "PersonTable",
 "Key": {
 "id": {
 "N": "1"
 }
 },
 "ReturnValues": "ALL_NEW",
 "UpdateExpression": "SET #mainAddress = :mainAddress",
```

```

 "ExpressionAttributeNames": {
 "#mainAddress": "mainAddress"
 },
 "ExpressionAttributeValues": {
 ":mainAddress": {
 "M": {
 "zipCode": {
 "NULL": true
 },
 "city": {
 "S": "YourCity"
 },
 "street": {
 "NULL": true
 },
 "state": {
 "NULL": true
 }
 }
 }
 }
}

```

B) Or should it update only the 'city' attribute of an existing 'mainAddress' as shown in the following simplified JSON?

```

{
 "TableName": "PersonTable",
 "Key": {
 "id": {
 "N": "1"
 }
 },
 "ReturnValues": "ALL_NEW",
 "UpdateExpression": "SET #mainAddress.#city = :mainAddress_city",
 "ExpressionAttributeNames": {
 "#city": "city",
 "#mainAddress": "mainAddress"
 },
 "ExpressionAttributeValues": {
 ":mainAddress_city": {
 "S": "YourCity"
 }
 }
}

```

```
}
```

However, assume that we don't know if the 'mainAddress' already exists. If it doesn't exist, the SDK would try to update an attribute of a non-existent map, which results in an exception.

In this particular case, we would likely select option B (SCALAR\_ONLY) to retain the other values of the 'mainAddress'.

```
*/
```

The following two examples show uses of the MAPS\_ONLY and SCALAR\_ONLY enumerated values. MAPS\_ONLY adds a map and SCALAR\_ONLY updates a map.

### IgnoreNullsMode.MAPS\_ONLY example

```
public void mapsOnlyModeExample() {
 // Assume that we ran this code yesterday.
 Person person = new Person();
 person.setId(1);
 person.setFirstName("FirstName");

 personDynamoDbTable.putItem(person);

 // Assume that we are running this code now.

 /* Note that we create a new Person with only the necessary information to
update the city value
of the mainAddress. */
 Person personForUpdate = new Person();
 personForUpdate.setId(1);
 // The update we want to make changes the city.
 Address mainAddressForUpdate = new Address();
 mainAddressForUpdate.setCity("YourCity");
 personForUpdate.setMainAddress(mainAddressForUpdate);

 Person updatedPerson = personDynamoDbTable.updateItem(r -> r
 .item(personForUpdate)
 .ignoreNullsMode(IgnoreNullsMode.MAPS_ONLY)); // Since the mainAddress
property does not exist, use MAPS_ONLY mode.
 assert updatedPerson.getMainAddress().getCity().equals("YourCity");
 assert updatedPerson.getMainAddress().getState() == null;
}
```

## IgnoreNullsMode.SCALAR\_ONLY example

```
public void scalarOnlyExample() {
 // Assume that we ran this code yesterday.
 Person person = new Person();
 person.setId(1);
 Address mainAddress = new Address();
 mainAddress.setCity("MyCity");
 mainAddress.setState("MyState");
 person.setMainAddress(mainAddress);

 personDynamoDbTable.putItem(person);

 // Assume that we are running this code now.

 /* Note that we create a new Person with only the necessary information to
update the city value
of the mainAddress. */
 Person personForUpdate = new Person();
 personForUpdate.setId(1);
 // The update we want to make changes the city.
 Address mainAddressForUpdate = new Address();
 mainAddressForUpdate.setCity("YourCity");
 personForUpdate.setMainAddress(mainAddressForUpdate);

 Person updatedPerson = personDynamoDbTable.updateItem(r -> r
 .item(personForUpdate)
 .ignoreNullsMode(IgnoreNullsMode.SCALAR_ONLY)); // SCALAR_ONLY mode
ignores null properties in the in mainAddress.
 assert updatedPerson.getMainAddress().getCity().equals("YourCity");
 assert updatedPerson.getMainAddress().getState().equals("MyState"); // The
state property remains the same.
}
```

Refer to the following table to see which null values are ignored for each mode. You can often work with either SCALAR\_ONLY and MAPS\_ONLY except when you work with beans or maps.

**Which null-valued properties in the object submitted to `updateItem` does the SDK ignore for each mode?**

Type of property	in SCALAR_ONLY mode	in MAPS_ONLY mode	in DEFAULT mode
Top scalar	Yes	Yes	No
Bean or map	Yes	Yes	No
Scalar value of a bean or map entry	Yes <sup>1</sup>	No <sup>2</sup>	No
List or set	Yes	Yes	No

<sup>1</sup>This assumes the map already exists in DynamoDB. Any scalar value—null or not null—of the bean or map that you provide in the object for update requires that a path to the value exists in DynamoDB. The SDK constructs a path to the attribute by using the `.` (dot) dereference operator before it submits the request.

<sup>2</sup>Since you use MAPS\_ONLY mode to fully replace or to add a bean or map, all null values in the bean or map are retained in the map saved to DynamoDB.

### Preserve empty objects with `@DynamoDbPreserveEmptyObject`

If you save a bean to Amazon DynamoDB with empty objects and you want the SDK to recreate the empty objects upon retrieval, annotate the getter of the inner bean with `@DynamoDbPreserveEmptyObject`.

To illustrate how the annotation works, the code example uses the following two beans.

### Example beans

The following data class contains two `InnerBean` fields. The getter method, `getInnerBeanWithoutAnno()`, is not annotated with `@DynamoDbPreserveEmptyObject`. The `getInnerBeanWithAnno()` method is annotated.

```
@DynamoDbBean
public class MyBean {
```



```

private String id;
private String name;
private InnerBean innerBeanWithoutAnno;
private InnerBean innerBeanWithAnno;

@DynamoDbPartitionKey
public String getId() { return id; }
public void setId(String id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public InnerBean getInnerBeanWithoutAnno() { return innerBeanWithoutAnno; }
public void setInnerBeanWithoutAnno(InnerBean innerBeanWithoutAnno)
{ this.innerBeanWithoutAnno = innerBeanWithoutAnno; }

@DynamoDbPreserveEmptyObject
public InnerBean getInnerBeanWithAnno() { return innerBeanWithAnno; }
public void setInnerBeanWithAnno(InnerBean innerBeanWithAnno)
{ this.innerBeanWithAnno = innerBeanWithAnno; }

@Override
public String toString() {
 return new StringJoiner(", ", MyBean.class.getSimpleName() + "[", "]")
 .add("innerBeanWithoutAnno=" + innerBeanWithoutAnno)
 .add("innerBeanWithAnno=" + innerBeanWithAnno)
 .add("id='" + id + "'")
 .add("name='" + name + "'")
 .toString();
}
}

```

Instances of the following InnerBean class are fields of MyBean and are initialized as empty objects in the example code.

```

@DynamoDbBean
public class InnerBean {

 private String innerBeanField;

 public String getInnerBeanField() {
 return innerBeanField;
 }
}

```

```

 public void setInnerBeanField(String innerBeanField) {
 this.innerBeanField = innerBeanField;
 }

 @Override
 public String toString() {
 return "InnerBean{" +
 "innerBeanField='" + innerBeanField + '\'' +
 '}';
 }
}

```

The following code example saves a `MyBean` object with initialized inner beans to DynamoDB and then retrieves the item. The logged output shows that the `innerBeanWithoutAnno` is not initialized, but `innerBeanWithAnno` has been created.

```

 public MyBean preserveEmptyObjectAnnoUsingGetItemExample(DynamoDbTable<MyBean>
myBeanTable) {
 // Save an item to DynamoDB.
 MyBean bean = new MyBean();
 bean.setId("1");
 bean.setInnerBeanWithoutAnno(new InnerBean()); // Instantiate the inner bean.
 bean.setInnerBeanWithAnno(new InnerBean()); // Instantiate the inner bean.
 myBeanTable.putItem(bean);

 GetItemEnhancedRequest request = GetItemEnhancedRequest.builder()
 .key(Key.builder().partitionValue("1").build())
 .build();
 MyBean myBean = myBeanTable.getItem(request);

 logger.info(myBean.toString());
 // Output 'MyBean[innerBeanWithoutAnno=null,
innerBeanWithAnno=InnerBean{innerBeanField='null'}, id='1', name='null']'.

 return myBean;
 }
}

```

## Alternative static schema

You can use the following `StaticTableSchema` version of the table schemas in place of the annotations on the beans.

```

public static TableSchema<MyBean> buildStaticSchemas() {

 StaticTableSchema<InnerBean> innerBeanStaticTableSchema =
 StaticTableSchema.builder(InnerBean.class)
 .newItemSupplier(InnerBean::new)
 .addAttribute(String.class, a -> a.name("innerBeanField")
 .getter(InnerBean::getInnerBeanField)
 .setter(InnerBean::setInnerBeanField))
 .build();

 return StaticTableSchema.builder(MyBean.class)
 .newItemSupplier(MyBean::new)
 .addAttribute(String.class, a -> a.name("id")
 .getter(MyBean::getId)
 .setter(MyBean::setId)
 .addTag(primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("name")
 .getter(MyBean::getName)
 .setter(MyBean::setName))
 .addAttribute(EnhancedType.documentOf(InnerBean.class,
 innerBeanStaticTableSchema),
 a -> a.name("innerBean1")
 .getter(MyBean::getInnerBeanWithoutAnno)
 .setter(MyBean::setInnerBeanWithoutAnno))
 .addAttribute(EnhancedType.documentOf(InnerBean.class,
 innerBeanStaticTableSchema,
 b -> b.preserveEmptyObject(true)),
 a -> a.name("innerBean2")
 .getter(MyBean::getInnerBeanWithAnno)
 .setter(MyBean::setInnerBeanWithAnno))
 .build();
}

```

## Avoid saving null attributes of nested objects

You can skip null attributes of nested objects when saving a data class object to DynamoDB by applying the `@DynamoDbIgnoreNulls` annotation. By contrast, top-level attributes with null values are never saved to the database.

To illustrate how the annotation works, the code example uses the following two beans.

## Example beans

The following data class contains two `InnerBean` fields. The getter method, `getInnerBeanWithoutAnno()`, is not annotated. The `getInnerBeanWithIgnoreNullsAnno()` method is annotated with `@DynamoDbIgnoreNulls`.

```
@DynamoDbBean
public class MyBean {

 private String id;
 private String name;
 private InnerBean innerBeanWithoutAnno;
 private InnerBean innerBeanWithIgnoreNullsAnno;

 @DynamoDbPartitionKey
 public String getId() { return id; }
 public void setId(String id) { this.id = id; }

 public String getName() { return name; }
 public void setName(String name) { this.name = name; }

 public InnerBean getInnerBeanWithoutAnno() { return innerBeanWithoutAnno; }
 public void setInnerBeanWithoutAnno(InnerBean innerBeanWithoutAnno)
 { this.innerBeanWithoutAnno = innerBeanWithoutAnno; }

 @DynamoDbIgnoreNulls
 public InnerBean getInnerBeanWithIgnoreNullsAnno() { return
innerBeanWithIgnoreNullsAnno; }
 public void setInnerBeanWithIgnoreNullsAnno(InnerBean innerBeanWithAnno)
 { this.innerBeanWithIgnoreNullsAnno = innerBeanWithAnno; }

 @Override
 public String toString() {
 return new StringJoiner(", ", MyBean.class.getSimpleName() + "[", "]")
 .add("innerBeanWithoutAnno=" + innerBeanWithoutAnno)
 .add("innerBeanWithIgnoreNullsAnno=" + innerBeanWithIgnoreNullsAnno)
 .add("id='" + id + "'")
 .add("name='" + name + "'")
 .toString();
 }
}
```

Instances of the following `InnerBean` class are fields of `MyBean` and are used in the following example code.

```
@DynamoDbBean
public class InnerBean {

 private String innerBeanFieldString;
 private Integer innerBeanFieldInteger;

 public String getInnerBeanFieldString() { return innerBeanFieldString; }
 public void setInnerBeanFieldString(String innerBeanFieldString)
 { this.innerBeanFieldString = innerBeanFieldString; }

 public Integer getInnerBeanFieldInteger() { return innerBeanFieldInteger; }
 public void setInnerBeanFieldInteger(Integer innerBeanFieldInteger)
 { this.innerBeanFieldInteger = innerBeanFieldInteger; }

 @Override
 public String toString() {
 return new StringJoiner(", ", InnerBean.class.getSimpleName() + "[", "]")
 .add("innerBeanFieldString='" + innerBeanFieldString + "'")
 .add("innerBeanFieldInteger=" + innerBeanFieldInteger)
 .toString();
 }
}
```

The following code example creates an `InnerBean` object and sets only one of its two attributes with a value.

```
public void ignoreNullsAnnoUsingPutItemExample(DynamoDbTable<MyBean> myBeanTable) {
 // Create an InnerBean object and give only one attribute a value.
 InnerBean innerBeanOneAttributeSet = new InnerBean();
 innerBeanOneAttributeSet.setInnerBeanFieldInteger(200);

 // Create a MyBean instance and use the same InnerBean instance both for
 attributes.
 MyBean bean = new MyBean();
 bean.setId("1");
 bean.setInnerBeanWithoutAnno(innerBeanOneAttributeSet);
 bean.setInnerBeanWithIgnoreNullsAnno(innerBeanOneAttributeSet);

 Map<String, AttributeValue> itemMap = myBeanTable.tableSchema().itemToMap(bean,
 true);
}
```

```

 logger.info(itemMap.toString());
 // Log the map that is sent to the database.
 //
 {innerBeanWithIgnoreNullsAnno=AttributeValue(M={innerBeanFieldInteger=AttributeValue(N=200)}),
 id=AttributeValue(S=1),
 innerBeanWithoutAnno=AttributeValue(M={innerBeanFieldInteger=AttributeValue(N=200),
 innerBeanFieldString=AttributeValue(NUL=true)}}}

 // Save the MyBean object to the table.
 myBeanTable.putItem(bean);
 }

```

To visualize the low-level data that is sent to DynamoDB, the code logs the attribute map before saving the MyBean object.

The logged output shows that the `innerBeanWithIgnoreNullsAnno` outputs one attribute,

```
innerBeanWithIgnoreNullsAnno=AttributeValue(M={innerBeanFieldInteger=AttributeValue(N=200)})
```

The `innerBeanWithoutAnno` instance outputs two attributes. One attribute has a value of 200 and the other is a null-valued attribute.

```
innerBeanWithoutAnno=AttributeValue(M={innerBeanFieldInteger=AttributeValue(N=200),
innerBeanFieldString=AttributeValue(NUL=true)})
```

## JSON representation of the attribute map

The following JSON representation makes it easier to see the data that is saved to DynamoDB.

```

{
 "id": {
 "S": "1"
 },
 "innerBeanWithIgnoreNullsAnno": {
 "M": {
 "innerBeanFieldInteger": {
 "N": "200"
 }
 }
 },
 "innerBeanWithoutAnno": {
 "M": {

```

```

 "innerBeanFieldInteger": {
 "N": "200"
 },
 "innerBeanFieldString": {
 "NULL": true
 }
}
}
}

```

## Alternative static schema

You can use the following `StaticTableSchema` version of the table schemas in place data class annotations.

```

public static TableSchema<MyBean> buildStaticSchemas() {

 StaticTableSchema<InnerBean> innerBeanStaticTableSchema =
 StaticTableSchema.builder(InnerBean.class)
 .newItemSupplier(InnerBean::new)
 .addAttribute(String.class, a -> a.name("innerBeanFieldString")
 .getter(InnerBean::getInnerBeanFieldString)
 .setter(InnerBean::setInnerBeanFieldString))
 .addAttribute(Integer.class, a -> a.name("innerBeanFieldInteger")
 .getter(InnerBean::getInnerBeanFieldInteger)
 .setter(InnerBean::setInnerBeanFieldInteger))
 .build();

 return StaticTableSchema.builder(MyBean.class)
 .newItemSupplier(MyBean::new)
 .addAttribute(String.class, a -> a.name("id")
 .getter(MyBean::getId)
 .setter(MyBean::setId)
 .addTag(primaryPartitionKey()))
 .addAttribute(String.class, a -> a.name("name")
 .getter(MyBean::getName)
 .setter(MyBean::setName))
 .addAttribute(EnhancedType.documentOf(InnerBean.class,
 innerBeanStaticTableSchema),
 a -> a.name("innerBeanWithoutAnno")
 .getter(MyBean::getInnerBeanWithoutAnno)
 .setter(MyBean::setInnerBeanWithoutAnno))
 .addAttribute(EnhancedType.documentOf(InnerBean.class,

```

```
 innerBeanStaticTableSchema,
 b -> b.ignoreNulls(true)),
 a -> a.name("innerBeanWithIgnoreNullsAnno")
 .getter(MyBean::getInnerBeanWithIgnoreNullsAnno)
 .setter(MyBean::setInnerBeanWithIgnoreNullsAnno))
 .build();
}
```

## Work with JSON documents with the Enhanced Document API for DynamoDB

The [Enhanced Document API](#) for AWS SDK for Java 2.x is designed to work with document-oriented data that has no fixed schema. However, it also lets you use custom classes to map individual attributes.

The Enhanced Document API is the successor to the [Document API](#) of the AWS SDK for Java v1.x.

### Contents

- [Get started using the Enhanced Document API](#)
  - [Create a DocumentTableSchema and a DynamoDbTable](#)
- [Build enhanced documents](#)
  - [Build from a JSON string](#)
  - [Build from individual elements](#)
- [Perform CRUD operations](#)
- [Access enhanced document attributes as custom objects](#)
- [Use an EnhancedDocument without DynamoDB](#)

### Get started using the Enhanced Document API

The Enhanced Document API requires the same [dependencies](#) that are needed for the DynamoDB Enhanced Client API. It also requires a [DynamoDbEnhancedClient instance](#) as shown at the start of this topic.

Because the Enhanced Document API was released with version 2.20.3 of the AWS SDK for Java 2.x, you need that version or greater.

### Create a DocumentTableSchema and a DynamoDbTable

To invoke commands against a DynamoDB table using the Enhanced Document API, associate the table with a client-side [DynamoDbTable<EnhancedDocument>](#) resource object.



The enhanced client's `table()` method creates a `DynamoDbTable<EnhancedDocument>` instance and requires parameters for the DynamoDB table name and a `DocumentTableSchema`.

The builder for a [DocumentTableSchema](#) requires a primary index key and one or more attribute converter providers. The `AttributeConverterProvider.defaultProvider()` method provides converters for [default types](#). It should be specified even if you provide a custom attribute converter provider. You can add an optional secondary index key to the builder.

The following code snippet shows the code that generates the client-side representation of a DynamoDB person table that stores schemaless `EnhancedDocument` objects.

```
DynamoDbTable<EnhancedDocument> documentDynamoDbTable =
 enhancedClient.table("person",
 TableSchema.documentSchemaBuilder()
 // Specify the primary key attributes.

 .addIndexPartitionKey(TableMetadata.primaryIndexName(),"id", AttributeValueType.S)
 .addIndexSortKey(TableMetadata.primaryIndexName(),
 "lastName", AttributeValueType.S)
 // Specify attribute converter providers. Minimally add the
 default one.

 .attributeConverterProviders(AttributeConverterProvider.defaultProvider())
 .build());

// Call documentTable.createTable() if "person" does not exist in DynamoDB.
// createTable() should be called only one time.
```

The following shows the JSON representation of a person object that is used throughout this section.

### JSON person object

```
{
 "id": 1,
 "firstName": "Richard",
 "lastName": "Roe",
 "age": 25,
 "addresses":
 {
 "home": {
```

```
 "zipCode": "00000",
 "city": "Any Town",
 "state": "FL",
 "street": "123 Any Street"
 },
 "work": {
 "zipCode": "00001",
 "city": "Anywhere",
 "state": "FL",
 "street": "100 Main Street"
 }
},
"hobbies": [
 "Hobby 1",
 "Hobby 2"
],
"phoneNumbers": [
 {
 "type": "Home",
 "number": "555-0100"
 },
 {
 "type": "Work",
 "number": "555-0119"
 }
]
}
```

## Build enhanced documents

An [EnhancedDocument](#) represents a document-type object that has complex structure with nested attributes. An `EnhancedDocument` requires top-level attributes that match the primary key attributes specified for the `DocumentTableSchema`. The remaining content is arbitrary and can consist of top-level attributes and also deeply nested attributes.

You create an `EnhancedDocument` instance by using a builder that provides several ways to add elements.

### Build from a JSON string

With a JSON string, you can build an `EnhancedDocument` in one method call. The following snippet creates an `EnhancedDocument` from a JSON string returned by the `jsonPerson()` helper

method. The `jsonPerson()` method returns the JSON string version of the [person object](#) shown previously.

```
EnhancedDocument document =
 EnhancedDocument.builder()
 .json(jsonPerson())
 .build();
```

## Build from individual elements

Alternatively, you can build an `EnhancedDocument` instance from individual components using type-safe methods of the builder.

The following example builds a person enhanced document similar to the enhanced document that is built from the JSON string in the previous example.

```
/* Define the shape of an address map whose JSON representation looks like the
following.
 Use 'addressMapEnhancedType' in the following EnhancedDocument.builder() to
simplify the code.
 "home": {
 "zipCode": "00000",
 "city": "Any Town",
 "state": "FL",
 "street": "123 Any Street"
 }*/
EnhancedType<Map<String, String>> addressMapEnhancedType =
 EnhancedType.mapOf(EnhancedType.of(String.class),
EnhancedType.of(String.class));

// Use the builder's typesafe methods to add elements to the enhanced
document.
EnhancedDocument personDocument = EnhancedDocument.builder()
 .putNumber("id", 50)
 .putString("firstName", "Shirley")
 .putString("lastName", "Rodriguez")
 .putNumber("age", 53)
 .putNull("nullAttribute")
 .putJson("phoneNumbers", phoneNumbersJSONString())
 /* Add the map of addresses whose JSON representation looks like the
following.
 {
```

```

 "home": {
 "zipCode": "00000",
 "city": "Any Town",
 "state": "FL",
 "street": "123 Any Street"
 }
 } */
 .putMap("addresses", getAddresses(), EnhancedType.of(String.class),
addressMapEnhancedType)
 .putList("hobbies", List.of("Theater", "Golf"),
EnhancedType.of(String.class))
 .build();

```

## Helper methods

```

private static String phoneNumbersJSONString() {
 return " [" +
 " {" +
 " \"type\": \"Home\", " +
 " \"number\": \"555-0140\"" +
 " }, " +
 " {" +
 " \"type\": \"Work\", " +
 " \"number\": \"555-0155\"" +
 " }" +
 "]";
}

private static Map<String, Map<String, String>> getAddresses() {
 return Map.of(
 "home", Map.of(
 "zipCode", "00002",
 "city", "Any Town",
 "state", "ME",
 "street", "123 Any Street"));
}

```

## Perform CRUD operations

After you define an `EnhancedDocument` instance, you can save it to a DynamoDB table. The following code snippet uses the [personDocument](#) that was created from individual elements.

```
documentDynamoDbTable.putItem(personDocument);
```

After you read an enhanced document instance from DynamoDB, you can extract the individual attribute values using getters as shown in the following code snippet that access the data saved from the `personDocument`. Alternatively, you can extract the complete content to a JSON string as shown in the last part of the example code.

```
// Read the item.
EnhancedDocument personDocFromDb =
documentDynamoDbTable.getItem(Key.builder().partitionValue(50).build());

// Access top-level attributes.
logger.info("Name: {} {}", personDocFromDb.getString("firstName"),
personDocFromDb.getString("lastName"));
// Name: Shirley Rodriguez

// Typesafe access of a deeply nested attribute. The addressMapEnhancedType
shown previously defines the shape of an addresses map.
Map<String, Map<String, String>> addresses =
personDocFromDb.getMap("addresses", EnhancedType.of(String.class),
addressMapEnhancedType);
addresses.keySet().forEach(k -> logger.info(addresses.get(k).toString()));
// {zipCode=00002, city=Any Town, street=123 Any Street, state=ME}

// Alternatively, work with AttributeValue types checking along the way for
deeply nested attributes.
Map<String, AttributeValue> addressesMap =
personDocFromDb.getMapOfUnknownType("addresses");
addressesMap.keySet().forEach((String k) -> {
 logger.info("Looking at data for [{}] address", k);
 // Looking at data for [home] address
 AttributeValue value = addressesMap.get(k);
 AttributeValue cityValue = value.m().get("city");
 if (cityValue != null) {
 logger.info(cityValue.s());
 // Any Town
 }
});

List<AttributeValue> phoneNumbers =
personDocFromDb.getListOfUnknownType("phoneNumbers");
phoneNumbers.forEach((AttributeValue av) -> {
```

```

 if (av.hasM()) {
 AttributeValue type = av.m().get("type");
 if (type.s() != null) {
 logger.info("Type of phone: {}", type.s());
 // Type of phone: Home
 // Type of phone: Work
 }
 }
 });

 String jsonPerson = personDocFromDb.toJson();
 logger.info(jsonPerson);
 // {"firstName":"Shirley","lastName":"Rodriguez","addresses":
{"home":{"zipCode":"00002","city":"Any Town","street":"123 Any
Street","state":"ME"}}, "hobbies":["Theater","Golf"],
 // "id":50,"nullAttribute":null,"age":53,"phoneNumbers":
[{"number":"555-0140","type":"Home"}, {"number":"555-0155","type":"Work"}]}

```

EnhancedDocument instances can be used with any method of [DynamoDbTable](#) or [DynamoDbEnhancedClient](#) in place of mapped data classes.

### Access enhanced document attributes as custom objects

In addition to providing an API to read and write attributes with schemaless structures, the Enhanced Document API lets you convert attributes to and from instances of custom classes.

The Enhanced Document API uses `AttributeConverterProviders` and `AttributeConverters` that were shown in the [control attribute conversion](#) section as part of the DynamoDB Enhanced Client API.

In the following example, we use a `CustomAttributeConverterProvider` with its nested `AddressConverter` class to convert `Address` objects.

This example shows that you can mix data from classes and also data from structures that are built as needed. This example also shows that custom classes can be used at any level of a nested structure. The `Address` objects in this example are values used in a map.

```

 public static void attributeToAddressClassMappingExample(DynamoDbEnhancedClient
enhancedClient, DynamoDbClient standardClient) {
 String tableName = "customer";

 // Define the DynamoDbTable for an enhanced document.

```

```

 // The schema builder provides methods for attribute converter providers and
 keys.
 DynamoDbTable<EnhancedDocument> documentDynamoDbTable =
enhancedClient.table(tableName,
 DocumentTableSchema.builder()
 // Add the CustomAttributeConverterProvider along with the
default when you build the table schema.
 .attributeConverterProviders(
 List.of(
 new CustomAttributeConverterProvider(),
 AttributeConverterProvider.defaultProvider()))
 .addIndexPartitionKey(TableMetadata.primaryIndexName(), "id",
AttributeValueType.N)
 .addIndexSortKey(TableMetadata.primaryIndexName(), "lastName",
AttributeValueType.S)
 .build());
 // Create the DynamoDB table if needed.
 documentDynamoDbTable.createTable();
 waitForTableCreation(tableName, standardClient);

 // The getAddressessForCustomMappingExample() helper method that provides
'addresses' shows the use of a custom Address class
 // rather than using a Map<String, Map<String, String> to hold the address
data.
 Map<String, Address> addresses = getAddressessForCustomMappingExample();

 // Build an EnhancedDocument instance to save an item with a mix of structures
defined as needed and static classes.
 EnhancedDocument personDocument = EnhancedDocument.builder()
 .putNumber("id", 50)
 .putString("firstName", "Shirley")
 .putString("lastName", "Rodriguez")
 .putNumber("age", 53)
 .putNull("nullAttribute")
 .putJson("phoneNumbers", phoneNumbersJSONString())
 // Note the use of 'EnhancedType.of(Address.class)' instead of the more
generic
 // 'EnhancedType.mapOf(EnhancedType.of(String.class),
EnhancedType.of(String.class))' that was used in a previous example.
 .putMap("addresses", addresses, EnhancedType.of(String.class),
EnhancedType.of(Address.class))
 .putList("hobbies", List.of("Hobby 1", "Hobby 2"),
EnhancedType.of(String.class))

```

```

 .build();
 // Save the item to DynamoDB.
 documentDynamoDbTable.putItem(personDocument);

 // Retrieve the item just saved.
 EnhancedDocument srPerson =
documentDynamoDbTable.getItem(Key.builder().partitionValue(50).sortValue("Rodriguez").build());

 // Access the addresses attribute.
 Map<String, Address> srAddresses = srPerson.get("addresses",
 EnhancedType.mapOf(EnhancedType.of(String.class),
EnhancedType.of(Address.class)));

 srAddresses.keySet().forEach(k -> logger.info(addresses.get(k).toString()));

 documentDynamoDbTable.deleteTable();

// The content logged to the console shows that the saved maps were converted to
Address instances.
Address{street='123 Main Street', city='Any Town', state='NC', zipCode='00000'}
Address{street='100 Any Street', city='Any Town', state='NC', zipCode='00000'}

```

## CustomAttributeConverterProvider code

```

public class CustomAttributeConverterProvider implements AttributeConverterProvider {

 private final Map<EnhancedType<?>, AttributeConverter<?>> converterCache =
ImmutableMap.of(
 // 1. Add AddressConverter to the internal cache.
 EnhancedType.of(Address.class), new AddressConverter());

 public static CustomAttributeConverterProvider create() {
 return new CustomAttributeConverterProvider();
 }

 // 2. The enhanced client queries the provider for attribute converters if it
 // encounters a type that it does not know how to convert.
 @SuppressWarnings("unchecked")
 @Override
 public <T> AttributeConverter<T> converterFor(EnhancedType<T> enhancedType) {
 return (AttributeConverter<T>) converterCache.get(enhancedType);
 }
}

```



```

// 3. Custom attribute converter
private class AddressConverter implements AttributeConverter<Address> {
 // 4. Transform an Address object into a DynamoDB map.
 @Override
 public AttributeValue transformFrom(Address address) {

 Map<String, AttributeValue> attributeValueMap = Map.of(
 "street", AttributeValue.fromS(address.getStreet()),
 "city", AttributeValue.fromS(address.getCity()),
 "state", AttributeValue.fromS(address.getState()),
 "zipCode", AttributeValue.fromS(address.getZipCode()));

 return AttributeValue.fromM(attributeValueMap);
 }

 // 5. Transform the DynamoDB map attribute to an Address object.
 @Override
 public Address transformTo(AttributeValue attributeValue) {
 Map<String, AttributeValue> m = attributeValue.m();
 Address address = new Address();
 address.setStreet(m.get("street").s());
 address.setCity(m.get("city").s());
 address.setState(m.get("state").s());
 address.setZipCode(m.get("zipCode").s());

 return address;
 }

 @Override
 public EnhancedType<Address> type() {
 return EnhancedType.of(Address.class);
 }

 @Override
 public AttributeValueType attributeValueType() {
 return AttributeValueType.M;
 }
}

```

## Address class

```
public class Address {
```

```
private String street;
private String city;
private String state;
private String zipCode;

public Address() {
}

public String getStreet() {
return this.street;
}

public String getCity() {
return this.city;
}

public String getState() {
return this.state;
}

public String getZipCode() {
return this.zipCode;
}

public void setStreet(String street) {
this.street = street;
}

public void setCity(String city) {
this.city = city;
}

public void setState(String state) {
this.state = state;
}

public void setZipCode(String zipCode) {
this.zipCode = zipCode;
}
}
```

## Helper method that provides addresses

The following helper method provides the map that use custom `Address` instances for values rather than generic `Map<String, String>` instances for values.

```
private static Map<String, Address> getAddressesForCustomMappingExample() {
 Address homeAddress = new Address();
 homeAddress.setStreet("100 Any Street");
 homeAddress.setCity("Any Town");
 homeAddress.setState("NC");
 homeAddress.setZipCode("00000");

 Address workAddress = new Address();
 workAddress.setStreet("123 Main Street");
 workAddress.setCity("Any Town");
 workAddress.setState("NC");
 workAddress.setZipCode("00000");

 return Map.of("home", homeAddress,
 "work", workAddress);
}
```

## Use an EnhancedDocument without DynamoDB

Although you usually use an instance of an `EnhancedDocument` to read and write document-type DynamoDB items, it can also be used independently of DynamoDB.

You can use `EnhancedDocuments` for their ability to convert between JSON strings or custom objects to low-level maps of `AttributeValues` as shown in the following example.

```
public static void conversionWithoutDynamoDbExample() {
 Address address = new Address();
 address.setCity("my city");
 address.setState("my state");
 address.setStreet("my street");
 address.setZipCode("00000");

 // Build an EnhancedDocument instance for its conversion functionality alone.
 EnhancedDocument addressEnhancedDoc = EnhancedDocument.builder()
 // Important: You must specify attribute converter providers when you
 // build an EnhancedDocument instance not used with a DynamoDB table.
 .attributeConverterProviders(new CustomAttributeConverterProvider(),
 DefaultAttributeConverterProvider.create())
 .build();
}
```

```

 .put("addressDoc", address, Address.class)
 .build();

 // Convert address to a low-level item representation.
 final Map<String, AttributeValue> addressAsAttributeMap =
addressEnhancedDoc.getMapOfUnknownType("addressDoc");
 logger.info("addressAsAttributeMap: {}", addressAsAttributeMap.toString());

 // Convert address to a JSON string.
 String addressAsJsonString = addressEnhancedDoc.getJson("addressDoc");
 logger.info("addressAsJsonString: {}", addressAsJsonString);
 // Convert addressEnhancedDoc back to an Address instance.
 Address addressConverted = addressEnhancedDoc.get("addressDoc",
Address.class);
 logger.info("addressConverted: {}", addressConverted.toString());
}

/* Console output:
 addressAsAttributeMap: {zipCode=AttributeValue(S=000000),
state=AttributeValue(S=my state), street=AttributeValue(S=my street),
city=AttributeValue(S=my city)}
 addressAsJsonString: {"zipCode":"000000","state":"my state","street":"my
street","city":"my city"}
 addressConverted: Address{street='my street', city='my city', state='my
state', zipCode='000000'}
*/

```

### Note

When you use an enhanced document independent of a DynamoDB table, make sure you explicitly set attribute converter providers on the builder.

In contrast, the document table schema supplies the converter providers when an enhanced document is used with a DynamoDB table.

## Use extensions to customize DynamoDB Enhanced Client operations

The DynamoDB Enhanced Client API supports plugin extensions that provide functionality beyond mapping operations. Extensions use two hook methods to modify data during read and write operations:

- `beforeWrite()` - Modifies a write operation before it happens

- `afterRead()` - Modifies the results of a read operation after it happens

Some operations (such as item updates) perform both a write and then a read, so both hook methods are called.

## How extensions are loaded

Extensions are loaded in the order that you specify in the enhanced client builder. The load order can be important because one extension can act on values that have been transformed by a previous extension.

By default, the enhanced client loads two extensions:

- [VersionedRecordExtension](#) - Provides optimistic locking
- [AtomicCounterExtension](#) - Automatically increments counter attributes

You can override the default behavior with the enhanced client builder and load any extension. You can also specify none if you don't want the default extensions.

### Important

If you load your own extensions, the enhanced client doesn't load any default extensions. If you want the behavior provided by either default extension, you need to explicitly add it to the list of extensions.

The following example shows how to load a custom extension named `verifyChecksumExtension` after the `VersionedRecordExtension`. The `AtomicCounterExtension` is not loaded in this example.

```
DynamoDbEnhancedClientExtension versionedRecordExtension =
 VersionedRecordExtension.builder().build();

DynamoDbEnhancedClient enhancedClient =
 DynamoDbEnhancedClient.builder()
 .dynamoDbClient(dynamoDbClient)
 .extensions(versionedRecordExtension,
 verifyChecksumExtension)
 .build();
```

## Available extension details and configuration

The following sections provide detailed information about each available extension in the SDK.

### Implement optimistic locking with the `VersionedRecordExtension`

The `VersionedRecordExtension` extension provides optimistic locking by incrementing and tracking an item version number as items are written to the database. A condition is added to every write that causes the write to fail if the version number of the actual persisted item doesn't match the value that the application last read.

#### Configuration

To specify which attribute to use to track the item version number, tag a numeric attribute in the table schema.

The following snippet specifies that the `version` attribute should hold the item version number.

```
@DynamoDbVersionAttribute
public Integer getVersion() {...};
public void setVersion(Integer version) {...};
```

The equivalent static table schema approach is shown in the following snippet.

```
.addAttribute(Integer.class, a -> a.name("version")
 .getter(Customer::getVersion)
 .setter(Customer::setVersion)
 // Apply the 'version' tag to the attribute.

.tags(VersionedRecordExtension.AttributeTags.versionAttribute())
```

#### How it works

Optimistic locking with the `VersionedRecordExtension` has the following impact on these `DynamoDbEnhancedClient` and `DynamoDbTable` methods:

#### `putItem`

New items are assigned a initial version value of 0. This can be configured with `@DynamoDbVersionAttribute(startAt = X)`.

## updateItem

If you retrieve an item, update one or more of its properties, and attempt to save the changes, the operation succeeds only if the version number on the client side and the server side match.

If successful, the version number is automatically incremented by 1. This can be configured with `@DynamoDbVersionAttribute(incrementBy = X)`.

## deleteItem

The `DynamoDbVersionAttribute` annotation has no effect. You must add a condition expressions manually when deleting an item.

The following example adds a conditional expression to ensure that the item deleted is the item that was read. In the following example `recordVersion` is the bean's attribute annotated with `@DynamoDbVersionAttribute`.

```
// 1. Read the item and get its current version.
Customer item =
 customerTable.getItem(Key.builder().partitionValue("someId").build());
// `recordVersion` is the bean's attribute that is annotated with
// `@DynamoDbVersionAttribute`.
AttributeValue currentVersion = item.getRecordVersion();

// 2. Create conditional delete with the `currentVersion` value.
DeleteItemEnhancedRequest deleteItemRequest =
 DeleteItemEnhancedRequest.builder()
 .key(KEY)
 .conditionExpression(Expression.builder()
 .expression("recordVersion = :current_version_value")
 .putExpressionValue(":current_version_value", currentVersion)
 .build()).build();

customerTable.deleteItem(deleteItemRequest);
```

## transactWriteItems

- `addPutItem`: This method has the same behavior as `putItem`.
- `addUpdateItem`: This method has the same behavior as `updateItem`.
- `addDeleteItem`: This method has the same behavior as `deleteItem`.

## batchWriteItem

- `addPutItem`: This method has the same behavior as `putItem`.

- `addDeleteItem`: This method has the same behavior as `deleteItem`.

### Note

DynamoDB global tables use a ['last writer wins' reconciliation](#) between concurrent updates, where DynamoDB makes a best effort to determine the last writer. If you use global tables, this 'last writer wins' policy means that locking strategies may not work as expected, because all replicas will eventually converge based on the last write determined by DynamoDB.

## How to disable

To disable optimistic locking, do not use the `@DynamoDbVersionAttribute` annotation.

## Implement counters with the `AtomicCounterExtension`

The `AtomicCounterExtension` extension increments a tagged numerical attribute each time a record is written to the database. You can specify start and increment values. If no values are specified, the start value is set to 0 and the attribute's value increments by 1.

## Configuration

To specify which attribute is a counter, tag an attribute of type `Long` in the table schema.

The following snippet shows the use of the default start and increment values for the counter attribute.

```
@DynamoDbAtomicCounter
public Long getCounter() {...};
public void setCounter(Long counter) {...};
```

The static table schema approach is shown in the following snippet. The atomic counter extension uses a start value of 10 and increments the value by 5 each time the record is written.

```
.addAttribute(Integer.class, a -> a.name("counter")
 .getter(Customer::getCounter)
 .setter(Customer::setCounter)
 // Apply the 'atomicCounter' tag to the
attribute with start and increment values.
```



```

 .tags(StaticAttributeTags.atomicCounter(10L,
5L))

```

## Add timestamps with the `AutoGeneratedTimestampRecordExtension`

The `AutoGeneratedTimestampRecordExtension` extension automatically updates tagged attributes of type [Instant](#) with a current timestamp every time the item is successfully written to the database. This extension is not loaded by default.

### Configuration

To specify which attribute to update with the current timestamp, tag the `Instant` attribute in the table schema.

The `lastUpdate` attribute is the target of the extension's behavior in the following snippet. Note the requirement that the attribute must be an `Instant` type.

```

@DynamoDbAutoGeneratedTimestampAttribute
public Instant getLastUpdate() {...}
public void setLastUpdate(Instant lastUpdate) {...}

```

The equivalent static table schema approach is shown in the following snippet.

```

 .addAttribute(Instant.class, a -> a.name("lastUpdate")
 .getter(Customer::getLastUpdate)
 .setter(Customer::setLastUpdate)
 // Applying the 'autoGeneratedTimestamp' tag to
the attribute.

 .tags(AutoGeneratedTimestampRecordExtension.AttributeTags.autoGeneratedTimestampAttribute())

```

## Generate a UUID with the `AutoGeneratedUuidExtension`

The `AutoGeneratedUuidExtension` extension generates a unique UUID (Universally Unique Identifier) for an attribute when a new record is written to the database. Uses the Java JDK [UUID.randomUUID\(\)](#) method and applies to attributes of type `java.lang.String`. This extension is not loaded by default.

### Configuration

The `uniqueId` attribute is the target of the extension's behavior in the following snippet.

```
@AutoGeneratedUuidExtension
public String getUniqueId() {...}
public void setUniqueId(String uniqueId) {...}
```

The equivalent static table schema approach is shown in the following snippet.

```
.addAttribute(String.class, a -> a.name("uniqueId")
 .getter(Customer::getUniqueId)
 .setter(Customer::setUniqueId)
 // Applying the 'autoGeneratedUuid' tag to the
attribute.

.tags(AutoGeneratedUuidExtension.AttributeTags.autoGeneratedUuidAttribute())
```

If you want the extension to populate the UUID only for putItem methods and not for updateItem methods, add the [update behavior](#) annotation as shown in the following snippet.

```
@AutoGeneratedUuidExtension
@DynamoDbUpdateBehavior(UpdateBehavior.WRITE_IF_NOT_EXISTS)
public String getUniqueId() {...}
public void setUniqueId(String uniqueId) {...}
```

If you use the static table schema approach, use the following equivalent code.

```
.addAttribute(String.class, a -> a.name("uniqueId")
 .getter(Customer::getUniqueId)
 .setter(Customer::setUniqueId)
 // Applying the 'autoGeneratedUuid' tag to the
attribute.

.tags(AutoGeneratedUuidExtension.AttributeTags.autoGeneratedUuidAttribute(),
StaticAttributeTags.updateBehavior(UpdateBehavior.WRITE_IF_NOT_EXISTS))
```

## Custom extension example

You can create custom extensions by implementing the `DynamoDbEnhancedClientExtension` interface. The following custom extension class shows a `beforeWrite()` method that uses an update expression to set a `registrationDate` attribute if the item in the database doesn't already have one.

```

public final class CustomExtension implements DynamoDbEnhancedClientExtension {

 // 1. In a custom extension, use an UpdateExpression to define what action to take
 // before
 // an item is updated.
 @Override
 public WriteModification beforeWrite(DynamoDbExtensionContext.BeforeWrite context)
 {
 if (context.operationContext().tableName().equals("Customer")
 && context.operationName().equals(OperationName.UPDATE_ITEM)) {
 return WriteModification.builder()
 .updateExpression(createUpdateExpression())
 .build();
 }
 return WriteModification.builder().build(); // Return an "empty"
 WriteModification instance if the extension should not be applied.
 // In this case, if the code is
 not updating an item on the Customer table.
 }

 private static UpdateExpression createUpdateExpression() {

 // 2. Use a SetAction, a subclass of UpdateAction, to provide the values in the
 // update.
 SetAction setAction =
 SetAction.builder()
 .path("registrationDate")
 .value("if_not_exists(registrationDate, :regValue)")
 .putExpressionValue(":regValue",
AttributeValue.fromS(Instant.now().toString()))
 .build();
 // 3. Build the UpdateExpression with one or more UpdateAction.
 return UpdateExpression.builder()
 .addAction(setAction)
 .build();
 }
}

```

## Use the DynamoDB Enhanced Client API asynchronously

If your application requires non-blocking, asynchronous calls to DynamoDB, you can use the [DynamoDbEnhancedAsyncClient](#). It's similar to the synchronous implementation but with the following key differences:

1. When you build the `DynamoDbEnhancedAsyncClient`, you must provide the asynchronous version of the standard client, `DynamoDbAsyncClient`, as shown in the following snippet.

```
DynamoDbEnhancedAsyncClient enhancedClient =
 DynamoDbEnhancedAsyncClient.builder()
 .dynamoDbClient(dynamoDbAsyncClient)
 .build();
```

2. Methods that return a single data object return a `CompletableFuture` of the result instead of only the result. Your application can then do other work without having to block on the result. The following snippet shows the asynchronous `getItem()` method.

```
CompletableFuture<Customer> result = customerDynamoDbTable.getItem(customer);
// Perform other work here.
return result.join(); // Now block and wait for the result.
```

3. Methods that return paginated lists of results return an [SdkPublisher](#) instead of an [SdkIterable](#) that the synchronous `DynamoDbEnhancedClient` returns for the same methods. Your application can then subscribe a handler to that publisher to deal with the results asynchronously without having to block.

```
PagePublisher<Customer> results = customerDynamoDbTable.query(r ->
 r.queryConditional(keyEqualTo(k -> k.partitionValue("Smith"))));
results.subscribe(myCustomerResultsProcessor);
// Perform other work and let the processor handle the results asynchronously.
```

For a more complete example of working with the `SdkPublisher` API, see [the example](#) in the section that discusses the asynchronous `scan()` method of this guide.

## Data class annotations

The following table lists the annotations that can be used on data classes and provides links to information and examples in this guide. The table is sorted in ascending alphabetical order by annotation name.

### Data class annotations used in this guide

Annotation name	Annotation applies to <sup>1</sup>	What it does	Where it is shown in this guide
DynamoDbAtomicCounter	attribute <sup>2</sup>	Increments a tagged numerical attribute each time a record is written to the database.	<a href="#">Introduction and discussion.</a>
DynamoDbAttribute	attribute	Defines or renames a bean property that is mapped to a DynamoDB table attribute.	• <a href="#">Initial discussion.</a>     • <a href="#">Get started section—see Note.</a>     • <a href="#">In MovieActor class In Query method examples.</a>
DynamoDbAutoGeneratedTimestampAttribute	attribute	Updates a tagged attribute with a current timestamp every time the item is successfully written to the database	<a href="#">Introduction and discussion.</a>
DynamoDbAutoGeneratedUuid	attribute	Generate a unique UUID (Universally Unique Identifier) for an attribute when a new record is written to the database.	<a href="#">Introduction and discussion.</a>

Annotation name	Annotation applies to <sup>1</sup>	What it does	Where it is shown in this guide
DynamoDbBean	class	Marks a data class as mappable to a table schema.	First use on the <a href="#">Customer class</a> in the Get started section. Several usages appear throughout the guide.
DynamoDbConvertedBy	attribute	Associates a custom Attribute Converter with the annotated attribute.	<a href="#">Initial discussion and example.</a>
DynamoDbFlatten	attribute	Flattens all the attributes of a separate DynamoDB data class and adds them as top-level attributes to the record that is read and written to the database.	• <a href="#">Initial discussion.</a>     • <a href="#">Implications for other code.</a>
DynamoDbIgnore	attribute	Results in the attribute remaining unmapped.	• <a href="#">Initial discussion.</a>     • <a href="#">Use in the ProductCatalog class.</a>
DynamoDbIgnoreNulls	attribute	Prevents saving null attributes of nested DynamoDb objects.	<a href="#">Discussion and examples.</a>

Annotation name	Annotation applies to <sup>1</sup>	What it does	Where it is shown in this guide
DynamoDbImmutable	class	Marks an immutable data class as mappable to a table schema.	• <a href="#">Introduction to the annotation.</a>     • <a href="#">Use in the ProductCatalog class.</a>     • <a href="#">Use with Lombok.</a>
DynamoDbPartitionKey	attribute	Marks an attribute as the primary partition key (hash key) of the DynamoDb table.	• <a href="#">Initial usage on the Customer class in the Get started section.</a>     • <a href="#">With Lombok.</a>
DynamoDbPreserveEmptyObject	attribute	Specifies that if no data is present for the object mapped to the annotated attribute, the object should be initialized with all null fields.	<a href="#">Discussion and examples.</a>
DynamoDbSecondaryPartitionKey	attribute	Marks an attribute as a partition key for a global secondary index.	• <a href="#">Use in secondary indices and example.</a>     • <a href="#">In Query method examples.</a>     • <a href="#">In Lombok example</a>     • <a href="#">With immutable classes.</a>

Annotation name	Annotation applies to <sup>1</sup>	What it does	Where it is shown in this guide
DynamoDbSecondarySortKey	attribute	Marks an attribute as an optional sort key for a global or local secondary index.	• <a href="#">Use in secondary indices and example.</a>     • <a href="#">In Query method examples.</a>     • <a href="#">In Lombok example.</a>     • <a href="#">With immutable classes.</a>
DynamoDbSortKey	attribute	Marks an attribute as the optional primary sort key (range key).	• <a href="#">Get started section on Customer class.</a>     • <a href="#">With immutable classes.</a>     • <a href="#">In Lombok example.</a>     • <a href="#">In Query method examples.</a>
DynamoDbUpdateBehavior	attribute	Specifies the behavior when this attribute is updated as part of an 'update' operation such as UpdateItem.	<a href="#">Introduction and example.</a>
DynamoDbVersionAttribute	attribute	Increments an item version number.	<a href="#">Introduction and discussion.</a>

<sup>1</sup>You can apply an attribute-level annotations to the getter or setter, but not both. This guide shows annotations on getters.



<sup>2</sup>The term `property` is normally used for a value encapsulated in a JavaBean data class. However, this guide uses the term `attribute` instead, to be consistent with the terminology used by DynamoDB.

## Work with Amazon EC2

This section provides examples of programming [Amazon EC2](#) that use the AWS SDK for Java 2.x.

### Topics

- [Manage Amazon EC2 instances](#)
- [Use AWS Regions and Availability Zones](#)
- [Work with security groups in Amazon EC2](#)
- [Work with Amazon EC2 instance metadata](#)

## Manage Amazon EC2 instances

### Create an instance

Create a new Amazon EC2 instance by calling the [Ec2Client](#)'s [runInstances](#) method, providing it with a [RunInstancesRequest](#) containing the [Amazon Machine Image \(AMI\)](#) to use and an [instance type](#).

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.InstanceType;
import software.amazon.awssdk.services.ec2.model.RunInstancesRequest;
import software.amazon.awssdk.services.ec2.model.RunInstancesResponse;
import software.amazon.awssdk.services.ec2.model.Tag;
import software.amazon.awssdk.services.ec2.model.CreateTagsRequest;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
```

### Code

```
public static String createEC2Instance(Ec2Client ec2, String name, String amiId) {

 RunInstancesRequest runRequest = RunInstancesRequest.builder()
```

```

 .imageId(amiId)
 .instanceType(InstanceType.T1_MICRO)
 .maxCount(1)
 .minCount(1)
 .build();

RunInstancesResponse response = ec2.runInstances(runRequest);
String instanceId = response.instances().get(0).instanceId();

Tag tag = Tag.builder()
 .key("Name")
 .value(name)
 .build();

CreateTagsRequest tagRequest = CreateTagsRequest.builder()
 .resources(instanceId)
 .tags(tag)
 .build();

try {
 ec2.createTags(tagRequest);
 System.out.printf(
 "Successfully started EC2 Instance %s based on AMI %s",
 instanceId, amiId);

 return instanceId;

} catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

return "";
}

```

See the [complete example](#) on GitHub.

## Start an instance

To start an Amazon EC2 instance, call the `Ec2Client`'s [startInstances](#) method, providing it with a [StartInstancesRequest](#) containing the ID of the instance to start.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.StartInstancesRequest;
import software.amazon.awssdk.services.ec2.model.StopInstancesRequest;
```

## Code

```
public static void startInstance(Ec2Client ec2, String instanceId) {

 StartInstancesRequest request = StartInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 ec2.startInstances(request);
 System.out.printf("Successfully started instance %s", instanceId);
}
```

See the [complete example](#) on GitHub.

## Stop an instance

To stop an Amazon EC2 instance, call the `Ec2Client`'s [stopInstances](#) method, providing it with a [StopInstancesRequest](#) containing the ID of the instance to stop.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.StartInstancesRequest;
import software.amazon.awssdk.services.ec2.model.StopInstancesRequest;
```

## Code

```
public static void stopInstance(Ec2Client ec2, String instanceId) {

 StopInstancesRequest request = StopInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 ec2.stopInstances(request);
 System.out.printf("Successfully stopped instance %s", instanceId);
}
```

```
}
```

See the [complete example](#) on GitHub.

## Reboot an instance

To reboot an Amazon EC2 instance, call the `Ec2Client`'s [rebootInstances](#) method, providing it with a [RebootInstancesRequest](#) containing the ID of the instance to reboot.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.RebootInstancesRequest;
```

### Code

```
public static void rebootEC2Instance(Ec2Client ec2, String instanceId) {

 try {
 RebootInstancesRequest request = RebootInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 ec2.rebootInstances(request);
 System.out.printf(
 "Successfully rebooted instance %s", instanceId);
 } catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Describe instances

To list your instances, create a [DescribeInstancesRequest](#) and call the `Ec2Client`'s [describeInstances](#) method. It will return a [DescribeInstancesResponse](#) object that you can use to list the Amazon EC2 instances for your account and region.

Instances are grouped by *reservation*. Each reservation corresponds to the call to `startInstances` that launched the instance. To list your instances, you must first call the `DescribeInstancesResponse` class' `reservations` method, and then call `instances` on each returned [Reservation](#) object.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesResponse;
import software.amazon.awssdk.services.ec2.model.Instance;
import software.amazon.awssdk.services.ec2.model.Reservation;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
```

## Code

```
public static void describeEC2Instances(Ec2Client ec2){

 String nextToken = null;

 try {

 do {
 DescribeInstancesRequest request =
DescribeInstancesRequest.builder().maxResults(6).nextToken(nextToken).build();
 DescribeInstancesResponse response = ec2.describeInstances(request);

 for (Reservation reservation : response.reservations()) {
 for (Instance instance : reservation.instances()) {
 System.out.println("Instance Id is " + instance.instanceId());
 System.out.println("Image id is "+ instance.imageId());
 System.out.println("Instance type is "+
instance.instanceType());
 System.out.println("Instance state name is "+
instance.state().name());
 System.out.println("monitoring information is "+
instance.monitoring().state());

 }
 }

 nextToken = response.nextToken();
 } while (nextToken != null);
 }
```

```
 } catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

Results are paged; you can get further results by passing the value returned from the result object's `nextToken` method to a new request object's `nextToken` method, then using the new request object in your next call to `describeInstances`.

See the [complete example](#) on GitHub.

## Monitor an instance

You can monitor various aspects of your Amazon EC2 instances, such as CPU and network utilization, available memory, and disk space remaining. To learn more about instance monitoring, see [Monitoring Amazon EC2](#) in the Amazon EC2 User Guide for Linux Instances.

To start monitoring an instance, you must create a [MonitorInstancesRequest](#) with the ID of the instance to monitor, and pass it to the `Ec2Client`'s [monitorInstances](#) method.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.MonitorInstancesRequest;
import software.amazon.awssdk.services.ec2.model.UnmonitorInstancesRequest;
```

## Code

```
public static void monitorInstance(Ec2Client ec2, String instanceId) {

 MonitorInstancesRequest request = MonitorInstancesRequest.builder()
 .instanceIds(instanceId).build();

 ec2.monitorInstances(request);
 System.out.printf(
 "Successfully enabled monitoring for instance %s",
 instanceId);
}
```

See the [complete example](#) on GitHub.

## Stop instance monitoring

To stop monitoring an instance, create an [UnmonitorInstancesRequest](#) with the ID of the instance to stop monitoring, and pass it to the `Ec2Client`'s [unmonitorInstances](#) method.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.MonitorInstancesRequest;
import software.amazon.awssdk.services.ec2.model.UnmonitorInstancesRequest;
```

### Code

```
public static void unmonitorInstance(Ec2Client ec2, String instanceId) {
 UnmonitorInstancesRequest request = UnmonitorInstancesRequest.builder()
 .instanceIds(instanceId).build();

 ec2.unmonitorInstances(request);

 System.out.printf(
 "Successfully disabled monitoring for instance %s",
 instanceId);
}
```

See the [complete example](#) on GitHub.

## More information

- [RunInstances](#) in the Amazon EC2 API Reference
- [DescribeInstances](#) in the Amazon EC2 API Reference
- [StartInstances](#) in the Amazon EC2 API Reference
- [StopInstances](#) in the Amazon EC2 API Reference
- [RebootInstances](#) in the Amazon EC2 API Reference
- [MonitorInstances](#) in the Amazon EC2 API Reference
- [UnmonitorInstances](#) in the Amazon EC2 API Reference

# Use AWS Regions and Availability Zones

## Describe Regions

To list the Regions available to your account, call the `Ec2Client`'s `describeRegions` method. It returns a [DescribeRegionsResponse](#). Call the returned object's `regions` method to get a list of [Region](#) objects that represent each Region.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeRegionsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesResponse;
import java.util.concurrent.CompletableFuture;
```

## Code

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeRegionsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesResponse;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeRegionsAndZones {
 public static void main(String[] args) {
 Ec2AsyncClient ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 try {
 CompletableFuture<Void> future =
 describeEC2RegionsAndZonesAsync(ec2AsyncClient);
 future.join(); // Wait for both async operations to complete.
 } catch (RuntimeException rte) {
```



```

 System.err.println("An exception occurred: " + (rte.getCause() != null ?
rte.getCause().getMessage() : rte.getMessage()));
 }
}

/**
 * Asynchronously describes the EC2 regions and availability zones.
 *
 * @param ec2AsyncClient the EC2 async client used to make the API calls
 * @return a {@link CompletableFuture} that completes when both the region and
availability zone descriptions are complete
 */
public static CompletableFuture<Void>
describeEC2RegionsAndZonesAsync(Ec2AsyncClient ec2AsyncClient) {
 // Initiate the asynchronous request to describe regions
 CompletableFuture<DescribeRegionsResponse> regionsResponse =
ec2AsyncClient.describeRegions();

 // Handle the response or exception for regions
 CompletableFuture<DescribeRegionsResponse> regionsFuture =
regionsResponse.whenComplete((regionsResp, ex) -> {
 if (ex != null) {
 // Handle the exception by throwing a RuntimeException
 throw new RuntimeException("Failed to describe EC2 regions.", ex);
 } else if (regionsResp == null || regionsResp.regions().isEmpty()) {
 // Throw an exception if the response is null or the result is empty
 throw new RuntimeException("No EC2 regions found.");
 } else {
 // Process the response if no exception occurred and the result is not
empty
 regionsResp.regions().forEach(region -> {
 System.out.printf(
 "Found Region %s with endpoint %s%n",
 region.regionName(),
 region.endpoint());
 });
 }
 });

 CompletableFuture<DescribeAvailabilityZonesResponse> zonesResponse =
ec2AsyncClient.describeAvailabilityZones();
 CompletableFuture<DescribeAvailabilityZonesResponse> zonesFuture =
zonesResponse.whenComplete((zonesResp, ex) -> {
 if (ex != null) {

```

```

 throw new RuntimeException("Failed to describe EC2 availability
zones.", ex);
 } else if (zonesResp == null || zonesResp.availabilityZones().isEmpty()) {
 throw new RuntimeException("No EC2 availability zones found.");
 } else {
 zonesResp.availabilityZones().forEach(zone -> {
 System.out.printf(
 "Found Availability Zone %s with status %s in region %s\n",
 zone.zoneName(),
 zone.state(),
 zone.regionName()
);
 });
 }
});

return CompletableFuture.allOf(regionsFuture, zonesFuture);
}
}

```

See the [complete example](#) on GitHub.

## Describe availability zones

To list each Availability Zone available to your account, call the `Ec2Client`'s `describeAvailabilityZones` method. It returns a [DescribeAvailabilityZonesResponse](#). Call its `availabilityZones` method to get a list of [AvailabilityZone](#) objects that represent each Availability Zone.

## Imports

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeRegionsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesResponse;
import java.util.concurrent.CompletableFuture;

```

## Code

Create the `Ec2Client`.

```

Ec2AsyncClient ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)

```

```
.build();
```

Then call `describeAvailabilityZones()` and retrieve results.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeRegionsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesResponse;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeRegionsAndZones {
 public static void main(String[] args) {
 Ec2AsyncClient ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 try {
 CompletableFuture<Void> future =
describeEC2RegionsAndZonesAsync(ec2AsyncClient);
 future.join(); // Wait for both async operations to complete.
 } catch (RuntimeException rte) {
 System.err.println("An exception occurred: " + (rte.getCause() != null ?
rte.getCause().getMessage() : rte.getMessage()));
 }
 }

 /**
 * Asynchronously describes the EC2 regions and availability zones.
 *
 * @param ec2AsyncClient the EC2 async client used to make the API calls
 * @return a {@link CompletableFuture} that completes when both the region and
availability zone descriptions are complete
 */
 public static CompletableFuture<Void>
describeEC2RegionsAndZonesAsync(Ec2AsyncClient ec2AsyncClient) {
```

```

 // Initiate the asynchronous request to describe regions
 CompletableFuture<DescribeRegionsResponse> regionsResponse =
ec2AsyncClient.describeRegions();

 // Handle the response or exception for regions
 CompletableFuture<DescribeRegionsResponse> regionsFuture =
regionsResponse.whenComplete((regionsResp, ex) -> {
 if (ex != null) {
 // Handle the exception by throwing a RuntimeException
 throw new RuntimeException("Failed to describe EC2 regions.", ex);
 } else if (regionsResp == null || regionsResp.regions().isEmpty()) {
 // Throw an exception if the response is null or the result is empty
 throw new RuntimeException("No EC2 regions found.");
 } else {
 // Process the response if no exception occurred and the result is not
empty
 regionsResp.regions().forEach(region -> {
 System.out.printf(
 "Found Region %s with endpoint %s\n",
 region.regionName(),
 region.endpoint());
 });
 }
 });

 CompletableFuture<DescribeAvailabilityZonesResponse> zonesResponse =
ec2AsyncClient.describeAvailabilityZones();
 CompletableFuture<DescribeAvailabilityZonesResponse> zonesFuture =
zonesResponse.whenComplete((zonesResp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to describe EC2 availability
zones.", ex);
 } else if (zonesResp == null || zonesResp.availabilityZones().isEmpty()) {
 throw new RuntimeException("No EC2 availability zones found.");
 } else {
 zonesResp.availabilityZones().forEach(zone -> {
 System.out.printf(
 "Found Availability Zone %s with status %s in region %s\n",
 zone.zoneName(),
 zone.state(),
 zone.regionName()
);
 });
 }
 });
 }
}

```

```
 });

 return CompletableFuture.allOf(regionsFuture, zonesFuture);
}
}
```

See the [complete example](#) on GitHub.

## Describe accounts

To list EC2-related information about your account, call the `Ec2Client`'s `describeAccountAttributes` method. This method returns a [DescribeAccountAttributesResponse](#) object. Invoke this object's `accountAttributes` method to get a list of [AccountAttribute](#) objects. You can iterate through the list to retrieve an `AccountAttribute` object.

You can get your account's attribute values by invoking the `AccountAttribute` object's `attributeValues` method. This method returns a list of [AccountAttributeValue](#) objects. You can iterate through this second list to display the value of attributes (see the following code example).

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeAccountAttributesResponse;
import java.util.concurrent.CompletableFuture;
```

## Code

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeAccountAttributesResponse;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
```

```

public class DescribeAccount {
 public static void main(String[] args) {
 Ec2AsyncClient ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 try {
 CompletableFuture<DescribeAccountAttributesResponse> future =
describeEC2AccountAsync(ec2AsyncClient);
 future.join();
 System.out.println("EC2 Account attributes described successfully.");
 } catch (RuntimeException rte) {
 System.err.println("An exception occurred: " + (rte.getCause() != null ?
rte.getCause().getMessage() : rte.getMessage()));
 }
 }

 /**
 * Describes the EC2 account attributes asynchronously.
 *
 * @param ec2AsyncClient the EC2 asynchronous client to use for the operation
 * @return a {@link CompletableFuture} containing the {@link
DescribeAccountAttributesResponse} with the account attributes
 */
 public static CompletableFuture<DescribeAccountAttributesResponse>
describeEC2AccountAsync(Ec2AsyncClient ec2AsyncClient) {
 CompletableFuture<DescribeAccountAttributesResponse> response =
ec2AsyncClient.describeAccountAttributes();
 return response.whenComplete((accountResults, ex) -> {
 if (ex != null) {
 // Handle the exception by throwing a RuntimeException.
 throw new RuntimeException("Failed to describe EC2 account
attributes.", ex);
 } else if (accountResults == null ||
accountResults.accountAttributes().isEmpty()) {
 // Throw an exception if the response is null or no account attributes
are found.
 throw new RuntimeException("No account attributes found.");
 } else {
 // Process the response if no exception occurred.
 accountResults.accountAttributes().forEach(attribute -> {
 System.out.println("\nThe name of the attribute is " +
attribute.attributeName());
 attribute.attributeValues().forEach(

```

```
 myValue -> System.out.println("The value of the attribute is "
+ myValue.attributeValue()));
 });
}
});
}
```

See the [complete example](#) on GitHub.

## More information

- [Regions and Availability Zones](#) in the Amazon EC2 User Guide for Linux Instances
- [DescribeRegions](#) in the Amazon EC2 API Reference
- [DescribeAvailabilityZones](#) in the Amazon EC2 API Reference

## Work with security groups in Amazon EC2

### Create a security group

To create a security group, call the `Ec2Client`'s `createSecurityGroup` method with a [CreateSecurityGroupRequest](#) that contains the key's name.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.CreateSecurityGroupRequest;
import software.amazon.awssdk.services.ec2.model.AuthorizeSecurityGroupIngressRequest;
import software.amazon.awssdk.services.ec2.model.AuthorizeSecurityGroupIngressResponse;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.IpPermission;
import software.amazon.awssdk.services.ec2.model.CreateSecurityGroupResponse;
import software.amazon.awssdk.services.ec2.model.IpRange;
```

### Code

```
CreateSecurityGroupRequest createRequest =
CreateSecurityGroupRequest.builder()
 .groupName(groupName)
 .description(groupDesc)
```

```
 .vpcId(vpcId)
 .build();

CreateSecurityGroupResponse resp= ec2.createSecurityGroup(createRequest);
```

See the [complete example](#) on GitHub.

## Configure a security group

A security group can control both inbound (ingress) and outbound (egress) traffic to your Amazon EC2 instances.

To add ingress rules to your security group, use the `Ec2Client`'s `authorizeSecurityGroupIngress` method, providing the name of the security group and the access rules ([IpPermission](#)) you want to assign to it within an [AuthorizeSecurityGroupIngressRequest](#) object. The following example shows how to add IP permissions to a security group.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.CreateSecurityGroupRequest;
import software.amazon.awssdk.services.ec2.model.AuthorizeSecurityGroupIngressRequest;
import software.amazon.awssdk.services.ec2.model.AuthorizeSecurityGroupIngressResponse;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.IpPermission;
import software.amazon.awssdk.services.ec2.model.CreateSecurityGroupResponse;
import software.amazon.awssdk.services.ec2.model.IpRange;
```

### Code

First, create an `Ec2Client`

```
Region region = Region.US_WEST_2;
Ec2Client ec2 = Ec2Client.builder()
 .region(region)
 .build();
```

Then use the `Ec2Client`'s `authorizeSecurityGroupIngress` method,

```
IpRange ipRange = IpRange.builder()
```



```

 .cidrIp("0.0.0.0/0").build();

 IpPermission ipPerm = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(80)
 .fromPort(80)
 .ipRanges(ipRange)
 .build();

 IpPermission ipPerm2 = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(22)
 .fromPort(22)
 .ipRanges(ipRange)
 .build();

 AuthorizeSecurityGroupIngressRequest authRequest =
 AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(groupName)
 .ipPermissions(ipPerm, ipPerm2)
 .build();

 AuthorizeSecurityGroupIngressResponse authResponse =
 ec2.authorizeSecurityGroupIngress(authRequest);

 System.out.printf(
 "Successfully added ingress policy to Security Group %s",
 groupName);

 return resp.groupId();

} catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "";
}

```

To add an egress rule to the security group, provide similar data in an [AuthorizeSecurityGroupEgressRequest](#) to the `Ec2Client`'s `authorizeSecurityGroupEgress` method.

See the [complete example](#) on GitHub.

## Describe security groups

To describe your security groups or get information about them, call the `Ec2Client`'s `describeSecurityGroups` method. It returns a [DescribeSecurityGroupsResponse](#) that you can use to access the list of security groups by calling its `securityGroups` method, which returns a list of [SecurityGroup](#) objects.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsRequest;
import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsResponse;
import software.amazon.awssdk.services.ec2.model.SecurityGroup;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
```

### Code

```
public static void describeEC2SecurityGroups(Ec2Client ec2, String groupId) {

 try {
 DescribeSecurityGroupsRequest request =
 DescribeSecurityGroupsRequest.builder()
 .groupIds(groupId).build();

 DescribeSecurityGroupsResponse response =
 ec2.describeSecurityGroups(request);

 for(SecurityGroup group : response.securityGroups()) {
 System.out.printf(
 "Found Security Group with id %s, " +
 "vpc id %s " +
 "and description %s",
 group.groupId(),
 group.vpcId(),
 group.description());
 }
 } catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Delete a security group

To delete a security group, call the `Ec2Client`'s `deleteSecurityGroup` method, passing it a [DeleteSecurityGroupRequest](#) that contains the ID of the security group to delete.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.DeleteSecurityGroupRequest;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
```

### Code

```
public static void deleteEC2SecGroup(Ec2Client ec2,String groupId) {

 try {
 DeleteSecurityGroupRequest request = DeleteSecurityGroupRequest.builder()
 .groupId(groupId)
 .build();

 ec2.deleteSecurityGroup(request);
 System.out.printf(
 "Successfully deleted Security Group with id %s", groupId);

 } catch (Ec2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [Amazon EC2 Security Groups](#) in the Amazon EC2 User Guide for Linux Instances
- [Authorize inbound traffic for Your Linux Instances](#) in the Amazon EC2 User Guide for Linux Instances
- [CreateSecurityGroup](#) in the Amazon EC2 API Reference

- [DescribeSecurityGroups](#) in the Amazon EC2 API Reference
- [DeleteSecurityGroup](#) in the Amazon EC2 API Reference
- [AuthorizeSecurityGroupIngress](#) in the Amazon EC2 API Reference

## Work with Amazon EC2 instance metadata

A Java SDK client for the Amazon EC2 Instance Metadata Service (metadata client) allows your applications to access metadata on their local EC2 instance. The metadata client works with the local instance of [IMDSv2](#) (Instance Metadata Service v2) and uses session-oriented requests.

Two client classes are available in the SDK. The synchronous [Ec2MetadataClient](#) is for blocking operations, and the [Ec2MetadataAsyncClient](#) is for asynchronous, non-blocking use cases.

### Get started

To use the metadata client, add the `imds` Maven artifact to your project. You also need classes for an [SdkHttpClient](#) (or an [SdkAsyncHttpClient](#) for the asynchronous variant) on the classpath.

The following Maven XML shows dependency snippets for using the synchronous [URLConnectionHttpClient](#) along with the dependency for metadata clients.

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>VERSION</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>

<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>imds</artifactId>
 </dependency>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
```

```
<artifactId>url-connection-client</artifactId>
</dependency>
<!-- other dependencies -->
</dependencies>
```

Search the [Maven central repository](#) for the latest version of the bom artifact.

To use an asynchronous HTTP client, replace the dependency snippet for the `url-connection-client` artifact. For example, the following snippet brings in the [NettyNioAsyncHttpClient](#) implementation.

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>netty-nio-client</artifactId>
</dependency>
```

## Use the metadata client

### Instantiate a metadata client

You can instantiate an instance of a synchronous `Ec2MetadataClient` when only one implementation of the `SdkHttpClient` interface is present on the classpath. To do so, call the static `Ec2MetadataClient#create()` method as shown in the following snippet.

```
Ec2MetadataClient client = Ec2MetadataClient.create(); //
'Ec2MetadataAsyncClient#create' is the asynchronous version.
```

If your application has multiple implementations of the `SdkHttpClient` or `SdkHttpAsyncClient` interface, you must specify an implementation for the metadata client to use as shown in the [the section called "Configurable HTTP client"](#) section.

#### Note

For most service clients, such as Amazon S3, the SDK for Java automatically adds implementations of the `SdkHttpClient` or `SdkHttpAsyncClient` interface. If your metadata client uses the same implementation, then `Ec2MetadataClient#create()` will work. If you require a different implementation, you must specify it when you create the metadata client.

## Send requests

To retrieve instance metadata, instantiate the `Ec2MetadataClient` class and call the `get` method with a path parameter that specifies the [instance metadata category](#).

The following example prints the value associated with the `ami-id` key to the console.

```
Ec2MetadataClient client = Ec2MetadataClient.create();
Ec2MetadataResponse response = client.get("/latest/meta-data/ami-id");
System.out.println(response.asString());
client.close(); // Closes the internal resources used by the Ec2MetadataClient class.
```

If the path isn't valid, the `get` method throws an exception.

Reuse the same client instance for multiple requests, but call `close` on the client when it is no longer needed to release resources. After the `close` method is called, the client instance can't be used anymore.

## Parse responses

EC2 instance metadata can be output in different formats. Plain text and JSON are the most commonly used formats. The metadata clients offer ways to work with those formats.

As the following example shows, use the `asString` method to get the data as a Java string. You can also use the `asList` method to separate a plain text response that returns multiple lines.

```
Ec2MetadataClient client = Ec2MetadataClient.create();
Ec2MetadataResponse response = client.get("/latest/meta-data/");
String fullResponse = response.asString();
List<String> splits = response.asList();
```

If the response is in JSON, use the `Ec2MetadataResponse#asDocument` method to parse the JSON response into a [Document](#) instance as shown in the following code snippet.

```
Document fullResponse = response.asDocument();
```

An exception will be thrown if the format of the metadata is not in JSON. If the response is successfully parsed, you can use the [document API](#) to inspect the response in more detail. Consult the instance [metadata category chart](#) to learn which metadata categories deliver JSON-formatted responses.

## Configure a metadata client

### Retries

You can configure a metadata client with a retry mechanism. If you do, then the client can automatically retry requests that fail for unexpected reasons. By default, the client retries three times on a failed request with an exponential backoff time between attempts.

If your use case requires a different retry mechanism, you can customize the client using the `retryPolicy` method on its builder. For example, the following example shows a synchronous client configured with a fixed delay of two seconds between attempts and five retry attempts.

```
BackoffStrategy fixedBackoffStrategy =
 FixedDelayBackoffStrategy.create(Duration.ofSeconds(2));
Ec2MetadataClient client =
 Ec2MetadataClient.builder()
 .retryPolicy(retryPolicyBuilder ->
 retryPolicyBuilder.numRetries(5)

 .backoffStrategy(fixedBackoffStrategy))
 .build();
```

There are several [BackoffStrategies](#) that you can use with a metadata client.

You can also disable the retry mechanism entirely, as the following snippet shows.

```
Ec2MetadataClient client =
 Ec2MetadataClient.builder()
 .retryPolicy(Ec2MetadataRetryPolicy.none())
 .build();
```

Using `Ec2MetadataRetryPolicy#none()` disables the default retry policy so that the metadata client attempts no retries.

### IP version

By default, a metadata client uses the IPV4 endpoint at `http://169.254.169.254`. To change the client to use the IPV6 version, use either the `endpointMode` or the `endpoint` method of the builder. An exception results if both methods are called on the builder.

The following examples show both IPV6 options.

```
Ec2MetadataClient client =
 Ec2MetadataClient.builder()
 .endpointMode(EndpointMode.IPV6)
 .build();
```

```
Ec2MetadataClient client =
 Ec2MetadataClient.builder()
 .endpoint(URI.create("http://[fd00:ec2::254]"))
 .build();
```

## Key features

### Asynchronous client

To use the non-blocking version of the client, instantiate an instance of the `Ec2MetadataAsyncClient` class. The code in the following example creates an asynchronous client with default settings and uses the `get` method to retrieve the value for the `ami-id` key.

```
Ec2MetadataAsyncClient asyncClient = Ec2MetadataAsyncClient.create();
CompletableFuture<Ec2MetadataResponse> response = asyncClient.get("/latest/meta-data/
ami-id");
```

The `java.util.concurrent.CompletableFuture` returned by the `get` method completes when the response returns. The following example prints the `ami-id` metadata to the console.

```
response.thenAccept(metadata -> System.out.println(metadata.asString()));
```

### Configurable HTTP client

The builder for each metadata client has a `httpClient` method that you can use to supply a customized HTTP client.

The following example shows code for a custom `URLConnectionHttpClient` instance.

```
SdkHttpClient httpClient =
 UrlConnectionHttpClient.builder()
 .socketTimeout(Duration.ofMinutes(5))
 .proxyConfiguration(proxy ->
 proxy.endpoint(URI.create("http://proxy.example.net:8888"))))
```



```
 .build();
Ec2MetadataClient metaDataClient =
 Ec2MetadataClient.builder()
 .httpClient(httpClient)
 .build();
// Use the metaDataClient instance.
metaDataClient.close(); // Close the instance when no longer needed.
```

The following example shows code for a custom `NettyNioAsyncHttpClient` instance with an asynchronous metadata client.

```
SdkAsyncHttpClient httpAsyncClient =
 NettyNioAsyncHttpClient.builder()
 .connectionTimeout(Duration.ofMinutes(5))
 .maxConcurrency(100)
 .build();
Ec2MetadataAsyncClient asyncMetaDataClient =
 Ec2MetadataAsyncClient.builder()
 .httpClient(httpAsyncClient)
 .build();
// Use the asyncMetaDataClient instance.
asyncMetaDataClient.close(); // Close the instance when no longer needed.
```

The [the section called “Configure HTTP clients”](#) topic in this guide provides details on how to configure the HTTP clients that are available in the SDK for Java.

## Token caching

Because the metadata clients use IMDSv2, all requests are associated with a session. A session is defined by a token that has an expiration, which the metadata client manages for you. Every metadata request automatically reuses the token until it expires.

By default, a token lasts for six hours (21,600 seconds). We recommend that you keep the default time-to-live value, unless your specific use case requires advanced configuration.

If needed, configure the duration by using the `tokenTtl` builder method. For example, the code in the following snippet creates a client with a session duration of five minutes.

```
Ec2MetadataClient client =
 Ec2MetadataClient.builder()
 .tokenTtl(Duration.ofMinutes(5))
```

```
.build();
```

If you omit calling the `tokenTtl` method on the builder, the default duration of 21,600 is used instead.

## Work with IAM

This section provides examples of programming AWS Identity and Access Management (IAM) by using the AWS SDK for Java 2.x.

AWS Identity and Access Management (IAM) enables you to securely control access to AWS services and resources for your users. Using IAM, you can create and manage AWS users and groups, and use permissions to allow and deny their access to AWS resources. For a complete guide to IAM, visit the [IAM User Guide](#).

The following examples include only the code needed to demonstrate each technique. The [complete example code is available on GitHub](#). From there, you can download a single source file or clone the repository locally to get all the examples to build and run.

### Topics

- [Manage IAM access keys](#)
- [Manage IAM Users](#)
- [Create IAM policies with the AWS SDK for Java 2.x](#)
- [Work with IAM policies](#)
- [Work with IAM server certificates](#)

## Manage IAM access keys

### Create an access key

To create an IAM access key, call the `IamClient`'s `createAccessKey` method with a [CreateAccessKeyRequest](#) object.

#### Note

You must set the region to **AWS\_GLOBAL** for `IamClient` calls to work because IAM is a global service.

## Imports

```
import software.amazon.awssdk.services.iam.model.CreateAccessKeyRequest;
import software.amazon.awssdk.services.iam.model.CreateAccessKeyResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

## Code

```
public static String createIAMAccessKey(IamClient iam,String user) {

 try {
 CreateAccessKeyRequest request = CreateAccessKeyRequest.builder()
 .userName(user).build();

 CreateAccessKeyResponse response = iam.createAccessKey(request);
 String keyId = response.accessKey().accessKeyId();
 return keyId;

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

See the [complete example](#) on GitHub.

## List access keys

To list the access keys for a given user, create a [ListAccessKeysRequest](#) object that contains the user name to list keys for, and pass it to the `IamClient`'s `listAccessKeys` method.

### Note

If you do not supply a user name to `listAccessKeys`, it will attempt to list access keys associated with the AWS account that signed the request.

## Imports

```
import software.amazon.awssdk.services.iam.model.AccessKeyMetadata;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListAccessKeysRequest;
import software.amazon.awssdk.services.iam.model.ListAccessKeysResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
```

## Code

```
public static void listKeys(IamClient iam,String userName){

 try {
 boolean done = false;
 String newMarker = null;

 while (!done) {
 ListAccessKeysResponse response;

 if(newMarker == null) {
 ListAccessKeysRequest request = ListAccessKeysRequest.builder()
 .userName(userName).build();
 response = iam.listAccessKeys(request);
 } else {
 ListAccessKeysRequest request = ListAccessKeysRequest.builder()
 .userName(userName)
 .marker(newMarker).build();
 response = iam.listAccessKeys(request);
 }

 for (AccessKeyMetadata metadata :
 response.accessKeyMetadata()) {
 System.out.format("Retrieved access key %s",
 metadata.accessKeyId());
 }

 if (!response.isTruncated()) {
 done = true;
 } else {
 newMarker = response.marker();
 }
 }

 } catch (IamException e) {
```

```
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

The results of `listAccessKeys` are paged (with a default maximum of 100 records per call). You can call `isTruncated` on the returned [ListAccessKeysResponse](#) object to see if the query returned fewer results than are available. If so, then call `marker` on the `ListAccessKeysResponse` and use it when creating a new request. Use that new request in the next invocation of `listAccessKeys`.

See the [complete example](#) on GitHub.

## Retrieve an access key's last used time

To get the time an access key was last used, call the `IamClient`'s `getAccessKeyLastUsed` method with the access key's ID (which can be passed in using a [GetAccessKeyLastUsedRequest](#) object).

You can then use the returned [GetAccessKeyLastUsedResponse](#) object to retrieve the key's last used time.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.GetAccessKeyLastUsedRequest;
import software.amazon.awssdk.services.iam.model.GetAccessKeyLastUsedResponse;
import software.amazon.awssdk.services.iam.model.IamException;
```

## Code

```
public static void getAccessKeyLastUsed(IamClient iam, String accessId){

 try {
 GetAccessKeyLastUsedRequest request = GetAccessKeyLastUsedRequest.builder()
 .accessKeyId(accessId).build();

 GetAccessKeyLastUsedResponse response = iam.getAccessKeyLastUsed(request);
```

```
 System.out.println("Access key was last used at: " +
 response.accessKeyLastUsed().lastUsedDate());

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
}
```

See the [complete example](#) on GitHub.

## Activate or deactivate access keys

You can activate or deactivate an access key by creating an [UpdateAccessKeyRequest](#) object, providing the access key ID, optionally the user name, and the desired [status](#), then passing the request object to the `IamClient`'s `updateAccessKey` method.

### Imports

```
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.StatusType;
import software.amazon.awssdk.services.iam.model.UpdateAccessKeyRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
```

### Code

```
public static void updateKey(IamClient iam, String username, String accessId,
 String status) {

 try {
 if (status.toLowerCase().equalsIgnoreCase("active")) {
 statusType = StatusType.ACTIVE;
 } else if (status.toLowerCase().equalsIgnoreCase("inactive")) {
 statusType = StatusType.INACTIVE;
 } else {
 statusType = StatusType.UNKNOWN_TO_SDK_VERSION;
 }
 UpdateAccessKeyRequest request = UpdateAccessKeyRequest.builder()
 .accessKeyId(accessId)
 .userName(username)
```

```
 .status(statusType)
 .build();

iam.updateAccessKey(request);

System.out.printf(
 "Successfully updated the status of access key %s to" +
 "status %s for user %s", accessId, status, username);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Delete an access key

To permanently delete an access key, call the `IamClient`'s `deleteKey` method, providing it with a [DeleteAccessKeyRequest](#) containing the access key's ID and username.

### Note

Once deleted, a key can no longer be retrieved or used. To temporarily deactivate a key so that it can be activated again later, use [updateAccessKey](#) method instead.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.DeleteAccessKeyRequest;
import software.amazon.awssdk.services.iam.model.IamException;
```

## Code

```
public static void deleteKey(IamClient iam ,String username, String accessKey) {

 try {
 DeleteAccessKeyRequest request = DeleteAccessKeyRequest.builder()
```

```
 .accessKeyId(accessKey)
 .userName(username)
 .build();

 iam.deleteAccessKey(request);
 System.out.println("Successfully deleted access key " + accessKey +
 " from user " + username);

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

See the [complete example](#) on GitHub.

## More information

- [CreateAccessKey](#) in the IAM API Reference
- [ListAccessKeys](#) in the IAM API Reference
- [GetAccessKeyLastUsed](#) in the IAM API Reference
- [UpdateAccessKey](#) in the IAM API Reference
- [DeleteAccessKey](#) in the IAM API Reference

## Manage IAM Users

### Create a User

Create a new IAM user by providing the user name to the `IamClient`'s `createUser` method using a [CreateUserRequest](#) object containing the user name.

### Imports

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.services.iam.model.CreateUserRequest;
import software.amazon.awssdk.services.iam.model.CreateUserResponse;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
```



```
import software.amazon.awssdk.services.iam.waiters.IamWaiter;
import software.amazon.awssdk.services.iam.model.GetUserRequest;
import software.amazon.awssdk.services.iam.model.GetUserResponse;
```

## Code

```
public static String createIAMUser(IamClient iam, String username) {

 try {
 // Create an IamWaiter object
 IamWaiter iamWaiter = iam.waiter();

 CreateUserRequest request = CreateUserRequest.builder()
 .userName(username)
 .build();

 CreateUserResponse response = iam.createUser(request);

 // Wait until the user is created
 GetUserRequest userRequest = GetUserRequest.builder()
 .userName(response.user().userName())
 .build();

 WaiterResponse<GetUserResponse> waitUntilUserExists =
iamWaiter.waitUntilUserExists(userRequest);
 waitUntilUserExists.matched().response().ifPresent(System.out::println);
 return response.user().userName();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

See the [complete example](#) on GitHub.

## List Users

To list the IAM users for your account, create a new [ListUsersRequest](#) and pass it to the `IamClient`'s `listUsers` method. You can retrieve the list of users by calling `users` on the returned [ListUsersResponse](#) object.

The list of users returned by `listUsers` is paged. You can check to see there are more results to retrieve by calling the response object's `isTruncated` method. If it returns `true`, then call the response object's `marker()` method. Use the marker value to create a new request object. Then call the `listUsers` method again with the new request.

## Imports

```
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListUsersRequest;
import software.amazon.awssdk.services.iam.model.ListUsersResponse;
import software.amazon.awssdk.services.iam.model.User;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
```

## Code

```
public static void listAllUsers(IamClient iam) {

 try {

 boolean done = false;
 String newMarker = null;

 while(!done) {
 ListUsersResponse response;

 if (newMarker == null) {
 ListUsersRequest request = ListUsersRequest.builder().build();
 response = iam.listUsers(request);
 } else {
 ListUsersRequest request = ListUsersRequest.builder()
 .marker(newMarker).build();
 response = iam.listUsers(request);
 }

 for(User user : response.users()) {
 System.out.format("\n Retrieved user %s", user.userName());
 }

 if(!response.isTruncated()) {
 done = true;
 } else {
 newMarker = response.marker();
 }
 }
 }
}
```

```
 }
 }
 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Update a User

To update a user, call the `IamClient` object's `updateUser` method, which takes a [UpdateUserRequest](#) object that you can use to change the user's *name* or *path*.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.UpdateUserRequest;
```

### Code

```
public static void updateIAMUser(IamClient iam, String curName, String newName) {

 try {
 UpdateUserRequest request = UpdateUserRequest.builder()
 .userName(curName)
 .newUserName(newName)
 .build();

 iam.updateUser(request);
 System.out.printf("Successfully updated user to username %s",
 newName);
 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Delete a User

To delete a user, call the `IamClient`'s `deleteUser` request with a [UpdateUserRequest](#) object set with the user name to delete.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.DeleteUserRequest;
import software.amazon.awssdk.services.iam.model.IamException;
```

### Code

```
public static void deleteIAMUser(IamClient iam, String userName) {

 try {
 DeleteUserRequest request = DeleteUserRequest.builder()
 .userName(userName)
 .build();

 iam.deleteUser(request);
 System.out.println("Successfully deleted IAM user " + userName);
 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More Information

- [IAM Users](#) in the IAM User Guide
- [Managing IAM Users](#) in the IAM User Guide
- [CreateUser](#) in the IAM API Reference
- [ListUsers](#) in the IAM API Reference
- [UpdateUser](#) in the IAM API Reference
- [DeleteUser](#) in the IAM API Reference

## Create IAM policies with the AWS SDK for Java 2.x

The [IAM Policy Builder API](#) is a library that you can use to build [IAM policies](#) in Java and upload them to AWS Identity and Access Management (IAM).

Instead of building an IAM policy by manually assembling a JSON string or by reading a file, the API provides a client-side, object-oriented approach to generate the JSON string. When you read an existing IAM policy in JSON format, the API converts it to an [IamPolicy](#) instance for handling.

The IAM Policy Builder API became available with version 2.20.105 of the SDK, so use that version or a later one in your Maven build file. The latest version number of the SDK is [listed on Maven central](#).

The following snippet shows an example dependency block for a Maven `pom.xml` file. This allows you to use the IAM Policy Builder API in your project.

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>iam-policy-builder</artifactId>
 <version>2.27.21</version>
</dependency>
```

### Create an IamPolicy

This section shows several examples of how to build policies by using the IAM Policy Builder API.

In each of the following examples, start with the [IamPolicy.Builder](#) and add one or more statements by using the `addStatement` method. Following this pattern, the [IamStatement.Builder](#) has methods to add the effect, actions, resources, and conditions to the statement.

#### Example: Create a time-based policy

The following example creates an identity-based policy that permits the Amazon DynamoDB `GetItem` action between two points in time.

```
public String timeBasedPolicyExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:GetItem")
```

```

 .addResource(IamResource.ALL)
 .addCondition(b1 -> b1
 .operator(IamConditionOperator.DATE_GREATER_THAN)
 .key("aws:CurrentTime")
 .value("2020-04-01T00:00:00Z"))
 .addCondition(b1 -> b1
 .operator(IamConditionOperator.DATE_LESS_THAN)
 .key("aws:CurrentTime")
 .value("2020-06-30T23:59:59Z")))
 .build();

 // Use an IamPolicyWriter to write out the JSON string to a more readable
 format.
 return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true)
 .build());
}

```

## JSON output

The last statement in the previous example returns the following JSON string.

Read more about this [example](#) in the *AWS Identity and Access Management User Guide*.

## JSON

```

{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Action": "dynamodb:GetItem",
 "Resource": "*",
 "Condition": {
 "DateGreaterThan": {
 "aws:CurrentTime": "2020-04-01T00:00:00Z"
 },
 "DateLessThan": {
 "aws:CurrentTime": "2020-06-30T23:59:59Z"
 }
 }
 }
]
}

```

## Example: Specify multiple conditions

The following example shows how you can create an identity-based policy that allows access to specific DynamoDB attributes. The policy contains two conditions.

```
public String multipleConditionsExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:GetItem")
 .addAction("dynamodb:BatchGetItem")
 .addAction("dynamodb:Query")
 .addAction("dynamodb:PutItem")
 .addAction("dynamodb:UpdateItem")
 .addAction("dynamodb>DeleteItem")
 .addAction("dynamodb:BatchWriteItem")
 .addResource("arn:aws:dynamodb:*:*:table/table-name")

 .addConditions(IamConditionOperator.STRING_EQUALS.addPrefix("ForAllValues:"),
 "dynamodb:Attributes",
 List.of("column-name1", "column-name2", "column-
name3"))

 .addCondition(b1 ->
 b1.operator(IamConditionOperator.STRING_EQUALS.addSuffix("IfExists"))
 .key("dynamodb:Select")
 .value("SPECIFIC_ATTRIBUTES"))

 .build();

 return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true).build());
}
```

## JSON output

The last statement in the previous example returns the following JSON string.

Read more about this [example](#) in the *AWS Identity and Access Management User Guide*.

JSON

```
{
 "Version": "2012-10-17",
```

```

 "Statement": {
 "Effect": "Allow",
 "Action": [
 "dynamodb:GetItem",
 "dynamodb:BatchGetItem",
 "dynamodb:Query",
 "dynamodb:PutItem",
 "dynamodb:UpdateItem",
 "dynamodb>DeleteItem",
 "dynamodb:BatchWriteItem"
],
 "Resource": "arn:aws:dynamodb:*:*:table/table-name",
 "Condition": {
 "ForAllValues:StringEquals": {
 "dynamodb:Attributes": [
 "column-name1",
 "column-name2",
 "column-name3"
]
 },
 "StringEqualsIfExists": {
 "dynamodb:Select": "SPECIFIC_ATTRIBUTES"
 }
 }
 }
 }
}

```

## Example: Specify principals

The following example shows how to create a resource-based policy that denies access to a bucket for all principals except for those specified in the condition.

```

public String specifyPrincipalsExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.DENY)
 .addAction("s3:*")
 .addPrincipal(IamPrincipal.ALL)
 .addResource("arn:aws:s3:::BUCKETNAME/*")
 .addResource("arn:aws:s3:::BUCKETNAME")
 .addCondition(b1 -> b1
 .operator(IamConditionOperator.ARN_NOT_EQUALS)

```



```

 .key("aws:PrincipalArn")
 .value("arn:aws:iam::444455556666:user/user-name")))
 .build();
return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true).build());
}

```

## JSON output

The last statement in the previous example returns the following JSON string.

Read more about this [example](#) in the *AWS Identity and Access Management User Guide*.

## JSON

```

{
 "Version": "2012-10-17",
 "Statement" : {
 "Effect" : "Deny",
 "Principal" : "*",
 "Action" : "s3:*",
 "Resource" : ["arn:aws:s3:::BUCKETNAME/*", "arn:aws:s3:::BUCKETNAME"],
 "Condition" : {
 "ArnNotEquals" : {
 "aws:PrincipalArn" : "arn:aws:iam::444455556666:user/user-name"
 }
 }
 }
}

```

## Example: Allow cross-account access

The following example shows how to allow another AWS account to upload objects to your bucket while retaining full owner control of the uploaded objects.

```

public String allowCrossAccountAccessExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addPrincipal(IamPrincipalType.AWS, "111122223333")
 .addAction("s3:PutObject")

```

```
 .addResource("arn:aws:s3:::amzn-s3-demo-bucket/*")
 .addCondition(b1 -> b1
 .operator(IamConditionOperator.STRING_EQUALS)
 .key("s3:x-amz-acl")
 .value("bucket-owner-full-control")))
 .build();
return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true).build());
}
```

## JSON output

The last statement in the previous example returns the following JSON string.

Read more about this [example](#) in the *Amazon Simple Storage Service User Guide*.

JSON

```
{
 "Version": "2012-10-17",
 "Statement" : {
 "Effect" : "Allow",
 "Principal" : {
 "AWS" : "111122223333"
 },
 "Action" : "s3:PutObject",
 "Resource" : "arn:aws:s3:::amzn-s3-demo-bucket/*",
 "Condition" : {
 "StringEquals" : {
 "s3:x-amz-acl" : "bucket-owner-full-control"
 }
 }
 }
}
```

## Use an IamPolicy with IAM

After you have created a `IamPolicy` instance, you use an [IamClient](#) to work with the IAM service.

The following example builds a policy that allows an [IAM identity](#) to write items to a DynamoDB table in the account that is specified with the `accountID` parameter. The policy is then uploaded to IAM as a JSON string.

```
public String createAndUploadPolicyExample(IamClient iam, String accountID, String
policyName) {
 // Build the policy.
 IamPolicy policy =
 IamPolicy.builder() // 'version' defaults to "2012-10-17".
 .addStatement(IamStatement.builder()
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:PutItem")
 .addResource("arn:aws:dynamodb:us-east-1:" + accountID
+ ":table/exampleTableName")
 .build())
 .build();
 // Upload the policy.
 iam.createPolicy(r ->
r.policyName(policyName).policyDocument(policy.toJson()));
 return policy.toJson(IamPolicyWriter.builder().prettyPrint(true).build());
}
```

The next example builds on the previous example. The code downloads the policy and uses it as the basis for a new policy by copying and altering the statement. The new policy is then uploaded.

```
public String createNewBasedOnExistingPolicyExample(IamClient iam, String
accountID, String policyName, String newPolicyName) {

 String policyArn = "arn:aws:iam::" + accountID + ":policy/" + policyName;
 GetPolicyResponse getPolicyResponse = iam.getPolicy(r ->
r.policyArn(policyArn));

 String policyVersion = getPolicyResponse.policy().defaultVersionId();
 GetPolicyVersionResponse getPolicyVersionResponse =
 iam.getPolicyVersion(r ->
r.policyArn(policyArn).versionId(policyVersion));

 // Create an IamPolicy instance from the JSON string returned from IAM.
 String decodedPolicy =
URLDecoder.decode(getPolicyVersionResponse.policyVersion().document(),
StandardCharsets.UTF_8);
 IamPolicy policy = IamPolicy.fromJson(decodedPolicy);
```

```

 /**
 * All IamPolicy components are immutable, so use the copy method that
 * creates a new instance that
 * can be altered in the same method call.
 *
 * Add the ability to get an item from DynamoDB as an additional action.
 */
 IamStatement newStatement = policy.statements().get(0).copy(s ->
s.addAction("dynamodb:GetItem"));

 // Create a new statement that replaces the original statement.
 IamPolicy newPolicy = policy.copy(p ->
p.statements(Arrays.asList(newStatement)));

 // Upload the new policy. IAM now has both policies.
 iam.createPolicy(r -> r.policyName(newPolicyName)
 .policyDocument(newPolicy.toJson()));

 return newPolicy.toJson(IamPolicyWriter.builder().prettyPrint(true).build());
}

```

## IamClient

The previous examples use an `IamClient` argument that is created as shown in the following snippet.

```
IamClient iam = IamClient.builder().region(Region.AWS_GLOBAL).build();
```

## Policies in JSON

The examples return the following JSON strings.

```

First example
{
 "Version": "2012-10-17",
 "Statement" : {
 "Effect" : "Allow",
 "Action" : "dynamodb:PutItem",
 "Resource" : "arn:aws:dynamodb:us-east-1:111122223333:table/exampleTableName"
 }
}

```

```
Second example
{
 "Version": "2012-10-17",
 "Statement" : {
 "Effect" : "Allow",
 "Action" : ["dynamodb:PutItem", "dynamodb:GetItem"],
 "Resource" : "arn:aws:dynamodb:us-east-1:111122223333:table/exampleTableName"
 }
}
```

## Work with IAM policies

### Create a policy

To create a new policy, provide the policy's name and a JSON-formatted policy document in a [CreatePolicyRequest](#) to the `IamClient`'s `createPolicy` method.

### Imports

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.services.iam.model.CreatePolicyRequest;
import software.amazon.awssdk.services.iam.model.CreatePolicyResponse;
import software.amazon.awssdk.services.iam.model.GetPolicyRequest;
import software.amazon.awssdk.services.iam.model.GetPolicyResponse;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.waiters.IamWaiter;
```

### Code

```
public static String createIAMPolicy(IamClient iam, String policyName) {

 try {
 // Create an IamWaiter object
 IamWaiter iamWaiter = iam.waiter();

 CreatePolicyRequest request = CreatePolicyRequest.builder()
 .policyName(policyName)
 .policyDocument(PolicyDocument).build();

 CreatePolicyResponse response = iam.createPolicy(request);
```

```
// Wait until the policy is created
GetPolicyRequest polRequest = GetPolicyRequest.builder()
 .policyArn(response.policy().arn())
 .build();

WaiterResponse<GetPolicyResponse> waitUntilPolicyExists =
iamWaiter.waitUntilPolicyExists(polRequest);
waitUntilPolicyExists.matched().response().ifPresent(System.out::println);
return response.policy().arn();

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "" ;
}
```

See the [complete example](#) on GitHub.

## Get a policy

To retrieve an existing policy, call the `IamClient`'s `getPolicy` method, providing the policy's ARN within a [GetPolicyRequest](#) object.

### Imports

```
import software.amazon.awssdk.services.iam.model.GetPolicyRequest;
import software.amazon.awssdk.services.iam.model.GetPolicyResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

### Code

```
public static void getIAMPolicy(IamClient iam, String policyArn) {

 try {
 GetPolicyRequest request = GetPolicyRequest.builder()
 .policyArn(policyArn).build();

 GetPolicyResponse response = iam.getPolicy(request);
```

```
 System.out.format("Successfully retrieved policy %s",
 response.policy().policyName());
 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Attach a role policy

You can attach a policy to an IAM [role](#) by calling the `IamClient`'s `attachRolePolicy` method, providing it with the role name and policy ARN in an [AttachRolePolicyRequest](#).

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.AttachRolePolicyRequest;
import software.amazon.awssdk.services.iam.model.AttachedPolicy;
import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesRequest;
import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesResponse;
import java.util.List;
```

### Code

```
public static void attachIAMRolePolicy(IamClient iam, String roleName, String
policyArn) {

 try {

 ListAttachedRolePoliciesRequest request =
ListAttachedRolePoliciesRequest.builder()
 .roleName(roleName)
 .build();

 ListAttachedRolePoliciesResponse response =
iam.listAttachedRolePolicies(request);
 List<AttachedPolicy> attachedPolicies = response.attachedPolicies();
```

```

// Ensure that the policy is not attached to this role
String polArn = "";
for (AttachedPolicy policy: attachedPolicies) {
 polArn = policy.policyArn();
 if (polArn.compareTo(policyArn)==0) {
 System.out.println(roleName +
 " policy is already attached to this role.");
 return;
 }
}

AttachRolePolicyRequest attachRequest =
 AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();

iam.attachRolePolicy(attachRequest);

System.out.println("Successfully attached policy " + policyArn +
 " to role " + roleName);

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

System.out.println("Done");
}

```

See the [complete example](#) on GitHub.

## List attached role policies

List attached policies on a role by calling the `IamClient`'s `listAttachedRolePolicies` method. It takes a [ListAttachedRolePoliciesRequest](#) object that contains the role name to list the policies for.

Call `getAttachedPolicies` on the returned [ListAttachedRolePoliciesResponse](#) object to get the list of attached policies. Results may be truncated; if the `ListAttachedRolePoliciesResponse` object's `isTruncated` method returns `true`, call the `ListAttachedRolePoliciesResponse` object's `marker` method. Use the marker returned to create a new request and use it to call `listAttachedRolePolicies` again to get the next batch of results.



## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.AttachRolePolicyRequest;
import software.amazon.awssdk.services.iam.model.AttachedPolicy;
import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesRequest;
import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesResponse;
import java.util.List;
```

## Code

```
public static void attachIAMRolePolicy(IamClient iam, String roleName, String
policyArn) {

 try {

 ListAttachedRolePoliciesRequest request =
ListAttachedRolePoliciesRequest.builder()
 .roleName(roleName)
 .build();

 ListAttachedRolePoliciesResponse response =
iam.listAttachedRolePolicies(request);
 List<AttachedPolicy> attachedPolicies = response.attachedPolicies();

 // Ensure that the policy is not attached to this role
 String polArn = "";
 for (AttachedPolicy policy: attachedPolicies) {
 polArn = policy.policyArn();
 if (polArn.compareTo(policyArn)==0) {
 System.out.println(roleName +
 " policy is already attached to this role.");
 return;
 }
 }

 AttachRolePolicyRequest attachRequest =
 AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();
```

```
iam.attachRolePolicy(attachRequest);

System.out.println("Successfully attached policy " + policyArn +
 " to role " + roleName);

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

System.out.println("Done");
}
```

See the [complete example](#) on GitHub.

## Detach a role policy

To detach a policy from a role, call the `IamClient`'s `detachRolePolicy` method, providing it with the role name and policy ARN in a [DetachRolePolicyRequest](#).

### Imports

```
import software.amazon.awssdk.services.iam.model.DetachRolePolicyRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

### Code

```
public static void detachPolicy(IamClient iam, String roleName, String policyArn)
{
 try {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();

 iam.detachRolePolicy(request);
 System.out.println("Successfully detached policy " + policyArn +
 " from role " + roleName);
 }
}
```

```
 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [Overview of IAM Policies](#) in the IAM User Guide.
- [AWS IAM Policy Reference](#) in the IAM User Guide.
- [CreatePolicy](#) in the IAM API Reference
- [GetPolicy](#) in the IAM API Reference
- [AttachRolePolicy](#) in the IAM API Reference
- [ListAttachedRolePolicies](#) in the IAM API Reference
- [DetachRolePolicy](#) in the IAM API Reference

## Work with IAM server certificates

To enable HTTPS connections to your website or application on AWS, you need an SSL/TLS *server certificate*. You can use a server certificate provided by AWS Certificate Manager or one that you obtained from an external provider.

We recommend that you use ACM to provision, manage, and deploy your server certificates. With ACM you can request a certificate, deploy it to your AWS resources, and let ACM handle certificate renewals for you. Certificates provided by ACM are free. For more information about ACM, see the [AWS Certificate Manager User Guide](#).

### Get a server certificate

You can retrieve a server certificate by calling the `IamClient`'s `getServerCertificate` method, passing it a [GetServerCertificateRequest](#) with the certificate's name.

### Imports

```
import software.amazon.awssdk.services.iam.model.GetServerCertificateRequest;
```

```
import software.amazon.awssdk.services.iam.model.GetServerCertificateResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

## Code

```
public static void getCertificate(IamClient iam,String certName) {

 try {
 GetServerCertificateRequest request = GetServerCertificateRequest.builder()
 .serverCertificateName(certName)
 .build();

 GetServerCertificateResponse response = iam.getServerCertificate(request);
 System.out.format("Successfully retrieved certificate with body %s",
 response.serverCertificate().certificateBody());

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## List server certificates

To list your server certificates, call the `IamClient`'s `listServerCertificates` method with a [ListServerCertificatesRequest](#). It returns a [ListServerCertificatesResponse](#).

Call the returned `ListServerCertificateResponse` object's `serverCertificateMetadataList` method to get a list of [ServerCertificateMetadata](#) objects that you can use to get information about each certificate.

Results may be truncated; if the `ListServerCertificateResponse` object's `isTruncated` method returns `true`, call the `ListServerCertificatesResponse` object's `marker` method and use the marker to create a new request. Use the new request to call `listServerCertificates` again to get the next batch of results.

## Imports

```
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListServerCertificatesRequest;
import software.amazon.awssdk.services.iam.model.ListServerCertificatesResponse;
import software.amazon.awssdk.services.iam.model.ServerCertificateMetadata;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
```

## Code

```
public static void listCertificates(IamClient iam) {

 try {
 boolean done = false;
 String newMarker = null;

 while(!done) {
 ListServerCertificatesResponse response;

 if (newMarker == null) {
 ListServerCertificatesRequest request =
 ListServerCertificatesRequest.builder().build();
 response = iam.listServerCertificates(request);
 } else {
 ListServerCertificatesRequest request =
 ListServerCertificatesRequest.builder()
 .marker(newMarker).build();
 response = iam.listServerCertificates(request);
 }

 for(ServerCertificateMetadata metadata :
 response.serverCertificateMetadataList()) {
 System.out.printf("Retrieved server certificate %s",
 metadata.serverCertificateName());
 }

 if(!response.isTruncated()) {
 done = true;
 } else {
 newMarker = response.marker();
 }
 }

 } catch (IamException e) {
```

```
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Update a server certificate

You can update a server certificate's name or path by calling the `IamClient`'s `updateServerCertificate` method. It takes a [UpdateServerCertificateRequest](#) object set with the server certificate's current name and either a new name or new path to use.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.UpdateServerCertificateRequest;
import software.amazon.awssdk.services.iam.model.UpdateServerCertificateResponse;
```

### Code

```
public static void updateCertificate(IamClient iam, String curName, String newName)
{
 try {
 UpdateServerCertificateRequest request =
 UpdateServerCertificateRequest.builder()
 .serverCertificateName(curName)
 .newServerCertificateName(newName)
 .build();

 UpdateServerCertificateResponse response =
 iam.updateServerCertificate(request);

 System.out.printf("Successfully updated server certificate to name %s",
 newName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

```
}
}
```

See the [complete example](#) on GitHub.

## Delete a server certificate

To delete a server certificate, call the `IamClient`'s `deleteServerCertificate` method with a [DeleteServerCertificateRequest](#) containing the certificate's name.

### Imports

```
import software.amazon.awssdk.services.iam.model.DeleteServerCertificateRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

### Code

```
public static void deleteCert(IamClient iam,String certName) {

 try {
 DeleteServerCertificateRequest request =
 DeleteServerCertificateRequest.builder()
 .serverCertificateName(certName)
 .build();

 iam.deleteServerCertificate(request);
 System.out.println("Successfully deleted server certificate " +
 certName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [Working with Server Certificates](#) in the IAM User Guide

- [GetServerCertificate](#) in the IAM API Reference
- [ListServerCertificates](#) in the IAM API Reference
- [UpdateServerCertificate](#) in the IAM API Reference
- [DeleteServerCertificate](#) in the IAM API Reference
- [AWS Certificate Manager User Guide](#)

## Work with Kinesis

This section provides examples of programming [Amazon Kinesis](#) using the AWS SDK for Java 2.x.

For more information about Kinesis, see the [Amazon Kinesis Developer Guide](#).

The following examples include only the code needed to demonstrate each technique. The [complete example code is available on GitHub](#). From there, you can download a single source file or clone the repository locally to get all the examples to build and run.

### Topics

- [Subscribe to Amazon Kinesis Data Streams](#)

## Subscribe to Amazon Kinesis Data Streams

The following examples show you how to retrieve and process data from Amazon Kinesis Data Streams using the `subscribeToShard` method. Kinesis Data Streams now employs the enhanced fanout feature and a low-latency HTTP/2 data retrieval API, making it easier for developers to run multiple low-latency, high-performance applications on the same Kinesis Data Stream.

### Set up

First, create an asynchronous Kinesis client and a [SubscribeToShardRequest](#) object. These objects are used in each of the following examples to subscribe to Kinesis events.

### Imports

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.atomic.AtomicInteger;
import org.reactivestreams.Subscriber;
import org.reactivestreams.Subscription;
import software.amazon.awssdk.core.async.SdkPublisher;
```



```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisAsyncClient;
import software.amazon.awssdk.services.kinesis.model.ShardIteratorType;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardEvent;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardEventStream;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardRequest;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardResponse;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardResponseHandler;
```

## Code

```
Region region = Region.US_EAST_1;
KinesisAsyncClient client = KinesisAsyncClient.builder()
 .region(region)
 .build();

SubscribeToShardRequest request = SubscribeToShardRequest.builder()
 .consumerARN(CONSUMER_ARN)
 .shardId("arn:aws:kinesis:us-east-1:111122223333:stream/
StockTradeStream")
 .startingPosition(s -> s.type(ShardIteratorType.LATEST)).build();
```

## Use the builder interface

You can use the builder method to simplify the creation of the [SubscribeToShardResponseHandler](#).

Using the builder, you can set each lifecycle callback with a method call instead of implementing the full interface.

## Code

```
private static CompletableFuture<Void> responseHandlerBuilder(KinesisAsyncClient
client, SubscribeToShardRequest request) {
 SubscribeToShardResponseHandler responseHandler =
SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
t.getMessage()))
 .onComplete(() -> System.out.println("All records stream
successfully"))
 // Must supply some type of subscriber
 .subscriber(e -> System.out.println("Received event - " + e))
```

```
 .build();
 return client.subscribeToShard(request, responseHandler);
}
```

For more control of the publisher, you can use the `publisherTransformer` method to customize the publisher.

## Code

```
private static CompletableFuture<Void>
responseHandlerBuilderPublisherTransformer(KinesisAsyncClient client,
SubscribeToShardRequest request) {
 SubscribeToShardResponseHandler responseHandler =
SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
t.getMessage()))
 .publisherTransformer(p -> p.filter(e -> e instanceof
SubscribeToShardEvent).limit(100))
 .subscriber(e -> System.out.println("Received event - " + e))
 .build();
 return client.subscribeToShard(request, responseHandler);
}
```

See the [complete example](#) on GitHub.

## Use a custom response handler

For full control of the subscriber and publisher, implement the `SubscribeToShardResponseHandler` interface.

In this example, you implement the `onEventStream` method, which allows you full access to the publisher. This demonstrates how to transform the publisher to event records for printing by the subscriber.

## Code

```
private static CompletableFuture<Void>
responseHandlerBuilderClassic(KinesisAsyncClient client, SubscribeToShardRequest
request) {
 SubscribeToShardResponseHandler responseHandler = new
SubscribeToShardResponseHandler() {
```

```

 @Override
 public void responseReceived(SubscribeToShardResponse response) {
 System.out.println("Receieved initial response");
 }

 @Override
 public void onEventStream(SdkPublisher<SubscribeToShardEventStream>
publisher) {
 publisher
 // Filter to only SubscribeToShardEvents
 .filter(SubscribeToShardEvent.class)
 // Flat map into a publisher of just records
 .flatMapIterable(SubscribeToShardEvent::records)
 // Limit to 1000 total records
 .limit(1000)
 // Batch records into lists of 25
 .buffer(25)
 // Print out each record batch
 .subscribe(batch -> System.out.println("Record Batch - " +
batch));
 }

 @Override
 public void complete() {
 System.out.println("All records stream successfully");
 }

 @Override
 public void exceptionOccurred(Throwable throwable) {
 System.err.println("Error during stream - " + throwable.getMessage());
 }
 };
 return client.subscribeToShard(request, responseHandler);
}

```

See the [complete example](#) on GitHub.

## Use the visitor interface

You can use a [Visitor](#) object to subscribe to specific events you're interested in watching.

### Code

```

 private static CompletableFuture<Void>
 responseHandlerBuilderVisitorBuilder(KinesisAsyncClient client,
 SubscribeToShardRequest request) {
 SubscribeToShardResponseHandler.Visitor visitor =
 SubscribeToShardResponseHandler.Visitor
 .builder()
 .onSubscribeToShardEvent(e -> System.out.println("Received subscribe to
 shard event " + e))
 .build();
 SubscribeToShardResponseHandler responseHandler =
 SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
 t.getMessage()))
 .subscriber(visitor)
 .build();
 return client.subscribeToShard(request, responseHandler);
 }

```

See the [complete example](#) on GitHub.

## Use a custom subscriber

You can also implement your own custom subscriber to subscribe to the stream.

This code snippet shows an example subscriber.

### Code

```

 private static class MySubscriber implements
 Subscriber<SubscribeToShardEventStream> {

 private Subscription subscription;
 private AtomicInteger eventCount = new AtomicInteger(0);

 @Override
 public void onSubscribe(Subscription subscription) {
 this.subscription = subscription;
 this.subscription.request(1);
 }

 @Override

```

```

 public void onNext(SubscribeToShardEventStream shardSubscriptionEventStream) {
 System.out.println("Received event " + shardSubscriptionEventStream);
 if (eventCount.incrementAndGet() >= 100) {
 // You can cancel the subscription at any time if you wish to stop
 receiving events.
 subscription.cancel();
 }
 subscription.request(1);
 }

 @Override
 public void onError(Throwable throwable) {
 System.err.println("Error occurred while stream - " +
 throwable.getMessage());
 }

 @Override
 public void onComplete() {
 System.out.println("Finished streaming all events");
 }
 }

```

You can pass the custom subscriber to the subscribe method as shown in the following code snippet.

## Code

```

 private static CompletableFuture<Void>
 responseHandlerBuilderSubscriber(KinesisAsyncClient client, SubscribeToShardRequest
 request) {
 SubscribeToShardResponseHandler responseHandler =
 SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
 t.getMessage()))
 .subscriber(MySubscriber::new)
 .build();
 return client.subscribeToShard(request, responseHandler);
 }

```

See the [complete example](#) on GitHub.

## Write data records into a Kinesis data stream

You can use the [KinesisClient](#) object to write data records into a Kinesis data stream by using the `putRecords` method. To successfully invoke this method, create a [PutRecordsRequest](#) object. You pass the name of the data stream to the `streamName` method. Also you must pass the data by using the `putRecords` method (as shown in the following code example).

### Imports

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.PutRecordRequest;
import software.amazon.awssdk.services.kinesis.model.KinesisException;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamRequest;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamResponse;
```

In the following Java code example, notice that **StockTrade** object is used as the data to write to the Kinesis data stream. Before running this example, ensure that you have created the data stream.

### Code

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.PutRecordRequest;
import software.amazon.awssdk.services.kinesis.model.KinesisException;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamRequest;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class StockTradesWriter {
 public static void main(String[] args) {
 final String usage = ""
```

Usage:  
     <streamName>

Where:

    streamName - The Amazon Kinesis data stream to which records are written (for example, StockTradeStream)

""";

```

if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
}

String streamName = args[0];
Region region = Region.US_EAST_1;
KinesisClient kinesisClient = KinesisClient.builder()
 .region(region)
 .build();

// Ensure that the Kinesis Stream is valid.
validateStream(kinesisClient, streamName);
setStockData(kinesisClient, streamName);
kinesisClient.close();
}

public static void setStockData(KinesisClient kinesisClient, String streamName) {
 try {
 // Repeatedly send stock trades with a 100 milliseconds wait in between.
 StockTradeGenerator stockTradeGenerator = new StockTradeGenerator();

 // Put in 50 Records for this example.
 int index = 50;
 for (int x = 0; x < index; x++) {
 StockTrade trade = stockTradeGenerator.getRandomTrade();
 sendStockTrade(trade, kinesisClient, streamName);
 Thread.sleep(100);
 }

 } catch (KinesisException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("Done");
}

```

```

 }

 private static void sendStockTrade(StockTrade trade, KinesisClient kinesisClient,
 String streamName) {
 byte[] bytes = trade.toJsonAsBytes();

 // The bytes could be null if there is an issue with the JSON serialization by
 // the Jackson JSON library.
 if (bytes == null) {
 System.out.println("Could not get JSON bytes for stock trade");
 return;
 }

 System.out.println("Putting trade: " + trade);
 PutRecordRequest request = PutRecordRequest.builder()
 .partitionKey(trade.getTickerSymbol()) // We use the ticker symbol as
the partition key, explained in
 // the Supplemental Information
section below.
 .streamName(streamName)
 .data(SdkBytes.fromByteArray(bytes))
 .build();

 try {
 kinesisClient.putRecord(request);
 } catch (KinesisException e) {
 System.err.println(e.getMessage());
 }
 }

 private static void validateStream(KinesisClient kinesisClient, String streamName)
 {
 try {
 DescribeStreamRequest describeStreamRequest =
DescribeStreamRequest.builder()
 .streamName(streamName)
 .build();

 DescribeStreamResponse describeStreamResponse =
kinesisClient.describeStream(describeStreamRequest);

 if (!
describeStreamResponse.streamDescription().streamStatus().toString().equals("ACTIVE"))
 {

```



```
 System.err.println("Stream " + streamName + " is not active. Please
wait a few moments and try again.");
 System.exit(1);
 }

 } catch (KinesisException e) {
 System.err.println("Error found while describing the stream " +
streamName);
 System.err.println(e);
 System.exit(1);
 }
}
}
```

See the [complete example](#) on GitHub.

## Use a third-party library

You can use other third-party libraries instead of implementing a custom subscriber. This example demonstrates using the RxJava implementation, but you can use any library that implements the Reactive Streams interfaces. See the [RxJava wiki page on Github](#) for more information on that library.

To use the library, add it as a dependency. If you're using Maven, the example shows the POM snippet to use.

### POM Entry

```
<dependency>
 <groupId>io.reactivex.rxjava2</groupId>
 <artifactId>rxjava</artifactId>
 <version>2.2.21</version>
</dependency>
```

### Imports

```
import java.net.URI;
import java.util.concurrent.CompletableFuture;

import io.reactivex.Flowable;
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.core.async.SdkPublisher;
import software.amazon.awssdk.http.Protocol;
```

```
import software.amazon.awssdk.http.SdkHttpConfigurationOption;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisAsyncClient;
import software.amazon.awssdk.services.kinesis.model.ShardIteratorType;
import software.amazon.awssdk.services.kinesis.model.StartingPosition;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardEvent;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardRequest;
import software.amazon.awssdk.services.kinesis.model.SubscribeToShardResponseHandler;
import software.amazon.awssdk.utils.AttributeMap;
```

This example uses RxJava in the `onEventStream` lifecycle method. This gives you full access to the publisher, which can be used to create an Rx Flowable.

## Code

```
SubscribeToShardResponseHandler responseHandler =
SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
t.getMessage()))
 .onEventStream(p -> Flowable.fromPublisher(p)
 .ofType(SubscribeToShardEvent.class)

.flatMapIterable(SubscribeToShardEvent::records)
 .limit(1000)
 .buffer(25)
 .subscribe(e -> System.out.println("Record
batch = " + e)))
 .build();
```

You can also use the `publisherTransformer` method with the Flowable publisher. You must adapt the Flowable publisher to an *SdkPublisher*, as shown in the following example.

## Code

```
SubscribeToShardResponseHandler responseHandler =
SubscribeToShardResponseHandler
 .builder()
 .onError(t -> System.err.println("Error during stream - " +
t.getMessage()))
 .publisherTransformer(p ->
SdkPublisher.adapt(Flowable.fromPublisher(p).limit(100)))
```

```
.build();
```

See the [complete example](#) on GitHub.

## More information

- [SubscribeToShardEvent](#) in the Amazon Kinesis API Reference
- [SubscribeToShard](#) in the Amazon Kinesis API Reference

## Invoke, list, and delete AWS Lambda functions

This section provides examples of programming with the Lambda service client by using the AWS SDK for Java 2.x.

### Topics

- [Invoke a Lambda function](#)
- [List Lambda functions](#)
- [Delete a Lambda function](#)

## Invoke a Lambda function

You can invoke a Lambda function by creating a [LambdaClient](#) object and invoking its `invoke` method. Create an [InvokeRequest](#) object to specify additional information such as the function name and the payload to pass to the Lambda function. Function names appear as *arn:aws:lambda:us-east-1:123456789012:function:HelloFunction*. You can retrieve the value by looking at the function in the AWS Management Console.

To pass payload data to a function, create a [SdkBytes](#) object that contains information. For example, in the following code example, notice the JSON data passed to the Lambda function.

### Imports

```
import software.amazon.awssdk.services.lambda.LambdaClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.lambda.model.InvokeRequest;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.services.lambda.model.InvokeResponse;
import software.amazon.awssdk.services.lambda.model.LambdaException;
```

## Code

The following code example demonstrates how to invoke a Lambda function.

```
public static void invokeFunction(LambdaClient awsLambda, String functionName) {

 InvokeResponse res = null ;
 try {
 //Need a SdkBytes instance for the payload
 String json = "{\"Hello \": \"Paris\"}";
 SdkBytes payload = SdkBytes.fromUtf8String(json) ;

 //Setup an InvokeRequest
 InvokeRequest request = InvokeRequest.builder()
 .functionName(functionName)
 .payload(payload)
 .build();

 res = awsLambda.invoke(request);
 String value = res.payload().asUtf8String() ;
 System.out.println(value);

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## List Lambda functions

Build a [LambdaClient](#) object and invoke its `listFunctions` method. This method returns a [ListFunctionsResponse](#) object. You can invoke this object's `functions` method to return a list of [FunctionConfiguration](#) objects. You can iterate through the list to retrieve information about the functions. For example, the following Java code example shows how to get each function name.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.lambda.LambdaClient;
import software.amazon.awssdk.services.lambda.model.LambdaException;
```

```
import software.amazon.awssdk.services.lambda.model.ListFunctionsResponse;
import software.amazon.awssdk.services.lambda.model.FunctionConfiguration;
import java.util.List;
```

## Code

The following Java code example demonstrates how to retrieve a list of function names.

```
public static void listFunctions(LambdaClient awsLambda) {

 try {
 ListFunctionsResponse functionResult = awsLambda.listFunctions();
 List<FunctionConfiguration> list = functionResult.functions();

 for (FunctionConfiguration config: list) {
 System.out.println("The function name is "+config.functionName());
 }

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Delete a Lambda function

Build a [LambdaClient](#) object and invoke its `deleteFunction` method. Create a [DeleteFunctionRequest](#) object and pass it to the `deleteFunction` method. This object contains information such as the name of the function to delete. Function names appear as *arn:aws:lambda:us-east-1:123456789012:function:HelloFunction*. You can retrieve the value by looking at the function in the AWS Management Console.

## Imports

```
import software.amazon.awssdk.services.lambda.LambdaClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.lambda.model.DeleteFunctionRequest;
import software.amazon.awssdk.services.lambda.model.LambdaException;
```

## Code

The following Java code demonstrates how to delete a Lambda function.

```
public static void deleteLambdaFunction(LambdaClient awsLambda, String
functionName) {
 try {
 DeleteFunctionRequest request = DeleteFunctionRequest.builder()
 .functionName(functionName)
 .build();

 awsLambda.deleteFunction(request);
 System.out.println("The " + functionName + " function was deleted");

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Work with Amazon S3

This section provides background information for working with Amazon S3 by using the AWS SDK for Java 2.x. This section compliments the [Amazon S3 Java v2 examples](#) presented in the *Code examples* section of this guide.

### S3 clients in the AWS SDK for Java 2.x

The AWS SDK for Java 2.x provides different types of S3 clients. The following table shows the differences and can help you decide what is best for your use cases.

#### Different flavors of Amazon S3 clients

S3 Client	Short description	When to use	Limitation/drawback
<b>AWS CRT-based S3 client</b>  Interface: <a href="#">S3AsyncClient</a>	Provides the same asynchronous API operations as the Java-based S3 async client	Your application transfers large objects (&gt; 8MB) and you	Supports <a href="#">fewer configuration settings</a> than the Java-based S3 clients.

S3 Client	Short description	When to use	Limitation/drawback
Builder: <a href="#">S3CrtAsyn</a> <a href="#">cClientBuilder</a>	but with greater performance.      - Requires the <code>aws-crt</code> dependency.    - Supports automatic parallel transfers (multipart).       See <a href="#">the section called "Use a performant S3 client"</a>.	want maximized performance.      - You want to upload objects with unknown content length.    - You want enhanced connection pooling and DNS load balancing, which improves throughput and performance.    - You want improved transfer reliability in the event of a network failure. Individual failed parts are retried without restarting the transfer from the beginning.	Requires an additional dependency.

S3 Client	Short description	When to use	Limitation/drawback
<b>Java-based S3 async client <i>with</i> multipart enabled</b>  Interface: <a href="#">S3AsyncClient</a>  Builder: <a href="#">S3AsyncClientBuilder</a>	- Provides an asynchronous API.    - Supports automatic parallel transfers (multipart) when you enable multipart at creation time.       See <a href="#">the section called “Configure parallel transfer support”</a>.	- Your application transfers large objects and you want improved performance.    - you want to upload object with unknown content length.    - You want improved transfer reliability in the event of a network failure. Individual failed parts are retried without restarting the transfer from the beginning.    - You need <a href="#">configuration options</a> that are not available with the AWS CRT-based S3 client.	Less performant than the AWS CRT-based S3 client.
<b>Java-based S3 async client <i>without</i> multipart enabled</b>  Interface: <a href="#">S3AsyncClient</a>  Builder: <a href="#">S3AsyncClientBuilder</a>	Provides an asynchronous API.	- You are transferring objects that are less than 8MB.    - You want an asynchronous API.	No performance optimization.



S3 Client	Short description	When to use	Limitation/drawback
<b>Java-based S3 sync client</b>  Interface: <a href="#">S3Client</a>  Builder: <a href="#">S3ClientBuilder</a>	Provides a synchronous API.	- You are transferring objects that are less than 8MB.    - You want a synchronous API.	No performance optimization.

### Note

From version 2.18.x and onward, the AWS SDK for Java 2.x uses [virtual hosted-style addressing](#) when including an endpoint override. This applies as long as the bucket name is a valid DNS label.

Call the [forcePathStyle](#) method with `true` in your client builder to force the client to use path-style addressing for buckets.

The following example shows a service client configured with an endpoint override and using path-style addressing.

```
S3Client client = S3Client.builder()
 .region(Region.US_WEST_2)
 .endpointOverride(URI.create("https://s3.us-
west-2.amazonaws.com"))
 .forcePathStyle(true)
 .build();
```

## Topics

- [Uploading streams to Amazon S3 using the AWS SDK for Java 2.x](#)
- [Work with Amazon S3 pre-signed URLs](#)
- [Cross-Region access for Amazon S3](#)
- [Data integrity protection with checksums](#)
- [Use a performant S3 client: AWS CRT-based S3 client](#)
- [Configure the Java-based S3 async client to use parallel transfers](#)
- [Transfer files and directories with the Amazon S3 Transfer Manager](#)

- [Work with S3 Event Notifications](#)

## Uploading streams to Amazon S3 using the AWS SDK for Java 2.x

When you use a stream to upload content to S3 using [putObject](#) or [uploadPart](#), you use a `RequestBody` factory class for the synchronous API to supply the stream. For the asynchronous API, the `AsyncRequestBody` is the equivalent factory class.

### Which methods upload streams?

For the synchronous API, you can use the following factory methods of `RequestBody` to supply the stream:

- [fromInputStream](#)(`InputStream inputStream`, `long contentLength`)
- [fromContentProvider](#)(`ContentStreamProvider provider`, `long contentLength`, `String mimeType`)
  - The `ContentStreamProvider` has the `fromInputStream(InputStream inputStream)` factory method
- [fromContentProvider](#)(`ContentStreamProvider provider`, `String mimeType`)

For the asynchronous API, you can use the following factory methods of `AsyncRequestBody`:

- [fromInputStream](#)(`InputStream inputStream`, `Long contentLength`, `ExecutorService executor`)
- [fromInputStream](#)(`AsyncRequestBodyFromInputStreamConfiguration configuration`)
  - You use the `AsyncRequestBodyFromInputStreamConfiguration.Builder` to supply the stream
- [fromInputStream](#)(`Consumer<AsyncRequestBodyFromInputStreamConfiguration.Builder> configuration`)
- [forBlockingInputStream](#)(`Long contentLength`)
  - The resulting [BlockingInputStreamAsyncRequestBody](#) contains the method `writeInputStream(InputStream inputStream)` that you can use to provide the stream

## Performing the upload

### If you know the length of the stream

As you can see from the signature of the methods shown previously, most methods accept a content length parameter.

If you know the content length in bytes, provide the exact value:

```
// Always provide the exact content length when it's available.
long contentLength = 1024; // Exact size in bytes.
s3Client.putObject(req -> req
 .bucket("amzn-s3-demo-bucket")
 .key("my-key"),
 RequestBody.fromInputStream(inputStream, contentLength));
```

#### Warning

When you upload from an input stream, if your specified content length doesn't match the actual byte count, you might experience:

- Truncated objects if your specified length is too small
- Failed uploads or hanging connections if your specified length is too large

### If you don't know the length of the stream

#### Using the synchronous API

Use the `fromContentProvider(ContentStreamProvider provider, String mimeType)`:

```
public PutObjectResponse syncClient_stream_unknown_size(String bucketName, String key,
 InputStream inputStream) {

 S3Client s3Client = S3Client.create();

 RequestBody body =
 RequestBody.fromContentProvider(ContentStreamProvider.fromInputStream(inputStream),
 "text/plain");
 PutObjectResponse putObjectResponse = s3Client.putObject(b ->
 b.bucket(BUCKET_NAME).key(KEY_NAME), body);
 return putObjectResponse;
```

```
}
```

Because the SDK buffers the entire stream in memory to calculate the content length, you can run into memory issues with large streams. If you need to upload large streams with the synchronous client, consider using the multipart API:

### Upload a stream using the synchronous client API and multipart API

```
public static void uploadStreamToS3(String bucketName, String key, InputStream
 inputStream) {
 // Create S3 client
 S3Client s3Client = S3Client.create();
 try {
 // Step 1: Initiate the multipart upload
 CreateMultipartUploadRequest createMultipartUploadRequest =
 CreateMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 CreateMultipartUploadResponse createResponse =
 s3Client.createMultipartUpload(createMultipartUploadRequest);
 String uploadId = createResponse.uploadId();
 System.out.println("Started multipart upload with ID: " + uploadId);

 // Step 2: Upload parts
 List<CompletedPart> completedParts = new ArrayList<>();
 int partNumber = 1;
 byte[] buffer = new byte[PART_SIZE];
 int bytesRead;

 try {
 while ((bytesRead = readFullyOrToEnd(inputStream, buffer)) > 0) {
 // Create request to upload a part
 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .partNumber(partNumber)
 .build();

 // If we didn't read a full buffer, create a properly sized byte array
 RequestBody requestBody;
```

```

 if (bytesRead < PART_SIZE) {
 byte[] lastPartBuffer = new byte[bytesRead];
 System.arraycopy(buffer, 0, lastPartBuffer, 0, bytesRead);
 requestBody = RequestBody.fromBytes(lastPartBuffer);
 } else {
 requestBody = RequestBody.fromBytes(buffer);
 }

 // Upload the part and save the response's ETag
 UploadPartResponse uploadPartResponse =
s3Client.uploadPart(uploadPartRequest, requestBody);
 CompletedPart part = CompletedPart.builder()
 .partNumber(partNumber)
 .eTag(uploadPartResponse.eTag())
 .build();
 completedParts.add(part);

 System.out.println("Uploaded part " + partNumber + " with size " +
bytesRead + " bytes");
 partNumber++;
 }

 // Step 3: Complete the multipart upload
 CompletedMultipartUpload completedMultipartUpload =
CompletedMultipartUpload.builder()
 .parts(completedParts)
 .build();

 CompleteMultipartUploadRequest completeRequest =
CompleteMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .multipartUpload(completedMultipartUpload)
 .build();

 CompleteMultipartUploadResponse completeResponse =
s3Client.completeMultipartUpload(completeRequest);
 System.out.println("Multipart upload completed. Object URL: " +
completeResponse.location());

} catch (Exception e) {
 // If an error occurs, abort the multipart upload
 System.err.println("Error during multipart upload: " + e.getMessage());
}

```

```

 AbortMultipartUploadRequest abortRequest =
AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .build();
 s3Client.abortMultipartUpload(abortRequest);
 System.err.println("Multipart upload aborted");
 } finally {
 try {
 inputStream.close();
 } catch (IOException e) {
 System.err.println("Error closing input stream: " + e.getMessage());
 }
 }
} finally {
 s3Client.close();
}
}

/**
 * Reads from the input stream into the buffer, attempting to fill the buffer
 * completely
 * or until the end of the stream is reached.
 *
 * @param inputStream the input stream to read from
 * @param buffer the buffer to fill
 * @return the number of bytes read, or -1 if the end of the stream is reached before
 * any bytes are read
 * @throws IOException if an I/O error occurs
 */
private static int readFullyOrToEnd(InputStream inputStream, byte[] buffer) throws
IOException {
 int totalBytesRead = 0;
 int bytesRead;

 while (totalBytesRead < buffer.length) {
 bytesRead = inputStream.read(buffer, totalBytesRead, buffer.length -
totalBytesRead);
 if (bytesRead == -1) {
 break; // End of stream
 }
 totalBytesRead += bytesRead;
 }
}

```

```
 return totalBytesRead > 0 ? totalBytesRead : -1;
}
```

### Note

For most use cases, we recommend using the asynchronous client API for streams of unknown size. This approach enables parallel transfers and offers a simpler programming interface, because the SDK handles the stream segmentation into multipart chunks if the stream is large.

Both the standard S3 asynchronous client with multipart enabled and the AWS CRT-based S3 client implement this approach. We show examples of this approach in the following section.

## Using the asynchronous API

You can provide `null` for the `contentLength` argument to the `fromInputStream(InputStream inputStream, Long contentLength, ExecutorService executor)`

### Example using the AWS CRT-based asynchronous client:

```
public PutObjectResponse crtClient_stream_unknown_size(String bucketName, String key,
 InputStream inputStream) {

 S3AsyncClient s3AsyncClient = S3AsyncClient.crtCreate();
 ExecutorService executor = Executors.newSingleThreadExecutor();
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
 executor); // 'null' indicates that the

 // content length is unknown.
 CompletableFuture<PutObjectResponse> responseFuture =
 s3AsyncClient.putObject(r -> r.bucket(bucketName).key(key), body)
 .exceptionally(e -> {
 if (e != null){
 logger.error(e.getMessage(), e);
 }
 return null;
 });
}
```

```
PutObjectResponse response = responseFuture.join(); // Wait for the response.
executor.shutdown();
return response;
}
```

### Example using the standard asynchronous client with multipart enabled:

```
public PutObjectResponse asyncClient_multipart_stream_unknown_size(String bucketName,
 String key, InputStream inputStream) {

 S3AsyncClient s3AsyncClient =
 S3AsyncClient.builder().multipartEnabled(true).build();
 ExecutorService executor = Executors.newSingleThreadExecutor();
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
 executor); // 'null' indicates that the

 // content length is unknown.
 CompletableFuture<PutObjectResponse> responseFuture =
 s3AsyncClient.putObject(r -> r.bucket(bucketName).key(key), body)
 .exceptionally(e -> {
 if (e != null) {
 logger.error(e.getMessage(), e);
 }
 return null;
 });

 PutObjectResponse response = responseFuture.join(); // Wait for the response.
 executor.shutdown();
 return response;
}
```

## Work with Amazon S3 pre-signed URLs

Pre-signed URLs provide temporary access to private S3 objects without requiring users to have AWS credentials or permissions.

For example, assume Alice has access to an S3 object, and she wants to temporarily share access to that object with Bob. Alice can generate a pre-signed GET request to share with Bob so that he can download the object without requiring access to Alice's credentials. You can generate pre-signed URLs for HTTP GET and for HTTP PUT requests.



## Generate a pre-signed URL for an object, then download it (GET request)

The following example consists of two parts.

- Part 1: Alice generates the pre-signed URL for an object.
- Part 2: Bob downloads the object by using the pre-signed URL.

### Part 1: Generate the URL

Alice already has an object in an S3 bucket. She uses the following code to generate a URL string that Bob can use in a subsequent GET request.

#### Imports

```
import com.example.s3.util.PresignUrlUtils;
import org.slf4j.Logger;
import software.amazon.awssdk.http.HttpExecuteRequest;
import software.amazon.awssdk.http.HttpExecuteResponse;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.SdkHttpMethod;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.GetObjectPresignRequest;
import software.amazon.awssdk.services.s3.presigner.model.PresignedGetObjectRequest;
import software.amazon.awssdk.utils.IoUtils;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.URISyntaxException;
import java.net.URL;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.file.Paths;
import java.time.Duration;
```

```
import java.util.UUID;
```

```
/* Create a pre-signed URL to download an object in a subsequent GET request. */
public String createPresignedGetUrl(String bucketName, String keyName) {
 try (S3Presigner presigner = S3Presigner.create()) {

 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();

 GetObjectPresignRequest presignRequest = GetObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // The URL will expire
in 10 minutes.
 .getObjectRequest(objectRequest)
 .build();

 PresignedGetObjectRequest presignedRequest =
presigner.presignGetObject(presignRequest);
 logger.info("Presigned URL: [{}]", presignedRequest.url().toString());
 logger.info("HTTP method: [{}]", presignedRequest.httpRequest().method());

 return presignedRequest.url().toExternalForm();
 }
}
```

## Part 2: Download the object

Bob uses one of the following three code options to download the object. Alternatively, he could use a browser to perform the GET request.

### Use JDK `HttpURLConnection` (since v1.1)

```
/* Use the JDK HttpURLConnection (since v1.1) class to do the download. */
public byte[] useHttpURLConnectionToGet(String presignedUrlString) {
 ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream(); //
Capture the response body to a byte array.

 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpURLConnection connection = (HttpURLConnection)
presignedUrl.openConnection();
 connection.setRequestMethod("GET");
```

```

 // Download the result of executing the request.
 try (InputStream content = connection.getInputStream()) {
 IoUtils.copy(content, byteArrayOutputStream);
 }
 logger.info("HTTP response code is " + connection.getResponseCode());

 } catch (S3Exception | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
}

```

## Use JDK HttpClient (since v11)

```

/* Use the JDK HttpClient (since v11) class to do the download. */
public byte[] useHttpClientToGet(String presignedUrlString) {
 ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream(); //
 Capture the response body to a byte array.

 HttpRequest.Builder requestBuilder = HttpRequest.newBuilder();
 HttpClient httpClient = HttpClient.newHttpClient();
 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpResponse<InputStream> response = httpClient.send(requestBuilder
 .uri(presignedUrl.toURI())
 .GET()
 .build(),
 HttpResponse.BodyHandlers.ofInputStream());

 IoUtils.copy(response.body(), byteArrayOutputStream);

 logger.info("HTTP response code is " + response.statusCode());

 } catch (URISyntaxException | InterruptedException | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
}

```

## Use SdkHttpClient from the SDK for Java

```

/* Use the AWS SDK for Java SdkHttpClient class to do the download. */
public byte[] useSdkHttpClientToGet(String presignedUrlString) {

```

```

 ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream(); //
 Capture the response body to a byte array.
 try {
 URL presignedUrl = new URL(presignedUrlString);
 SdkHttpRequest request = SdkHttpRequest.builder()
 .method(SdkHttpMethod.GET)
 .uri(presignedUrl.toURI())
 .build();

 HttpExecuteRequest executeRequest = HttpExecuteRequest.builder()
 .request(request)
 .build();

 try (SdkHttpClient sdkHttpClient = ApacheHttpClient.create()) {
 HttpExecuteResponse response =
 sdkHttpClient.prepareRequest(executeRequest).call();
 response.responseBody().ifPresentOrElse(
 abortableInputStream -> {
 try {
 IoUtils.copy(abortableInputStream,
byteArrayOutputStream);
 } catch (IOException e) {
 throw new RuntimeException(e);
 }
 },
 () -> logger.error("No response body."));

 logger.info("HTTP Response code is {}",
 response.httpResponse().statusCode());
 } catch (URISyntaxException | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
 }
 }

```

See the [complete example](#) and [test](#) on GitHub.

## Generate a pre-signed URL for an upload, then upload a file (PUT request)

The following example consists of two parts.

- Part 1: Alice generates the pre-signed URL to upload an object.

- Part 2: Bob uploads a file by using the pre-signed URL.

## Part 1: Generate the URL

Alice already has an S3 bucket. She uses the following code to generate a URL string that Bob can use in a subsequent PUT request.

### Imports

```
import com.example.s3.util.PresignUrlUtils;
import org.slf4j.Logger;
import software.amazon.awssdk.core.internal.sync.FileContentStreamProvider;
import software.amazon.awssdk.http.HttpExecuteRequest;
import software.amazon.awssdk.http.HttpExecuteResponse;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.SdkHttpMethod;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.PresignedPutObjectRequest;
import software.amazon.awssdk.services.s3.presigner.model.PutObjectPresignRequest;

import java.io.File;
import java.io.IOException;
import java.io.OutputStream;
import java.io.RandomAccessFile;
import java.net.HttpURLConnection;
import java.net.URISyntaxException;
import java.net.URL;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Duration;
import java.util.Map;
import java.util.UUID;
```

```

/* Create a presigned URL to use in a subsequent PUT request */
public String createPresignedUrl(String bucketName, String keyName, Map<String,
String> metadata) {
 try (S3Presigner presigner = S3Presigner.create()) {

 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .metadata(metadata)
 .build();

 PutObjectPresignRequest presignRequest = PutObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // The URL expires in
10 minutes.

 .putObjectRequest(objectRequest)
 .build();

 PresignedPutObjectRequest presignedRequest =
presigner.presignPutObject(presignRequest);
 String myURL = presignedRequest.url().toString();
 logger.info("Presigned URL to upload a file to: [{}]", myURL);
 logger.info("HTTP method: [{}]", presignedRequest.httpRequest().method());

 return presignedRequest.url().toExternalForm();
 }
}

```

## Part 2: Upload a file object

Bob uses one of the following three code options to upload a file.

### Use JDK `HttpURLConnection` (since v1.1)

```

/* Use the JDK HttpURLConnection (since v1.1) class to do the upload. */
public void useHttpURLConnectionToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());
 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpURLConnection connection = (HttpURLConnection)
presignedUrl.openConnection();
 connection.setDoOutput(true);

```

```

 metadata.forEach((k, v) -> connection.setRequestProperty("x-amz-meta-" + k,
v));

 connection.setRequestMethod("PUT");
 OutputStream out = connection.getOutputStream();

 try (RandomAccessFile file = new RandomAccessFile(fileToPut, "r");
 FileChannel inChannel = file.getChannel()) {
 ByteBuffer buffer = ByteBuffer.allocate(8192); //Buffer size is 8k

 while (inChannel.read(buffer) > 0) {
 buffer.flip();
 for (int i = 0; i < buffer.limit(); i++) {
 out.write(buffer.get());
 }
 buffer.clear();
 }
 } catch (IOException e) {
 logger.error(e.getMessage(), e);
 }

 out.close();
 connection.getResponseCode();
 logger.info("HTTP response code is " + connection.getResponseCode());

 } catch (S3Exception | IOException e) {
 logger.error(e.getMessage(), e);
 }
}

```

## Use JDK HttpClient (since v11)

```

/* Use the JDK HttpClient (since v11) class to do the upload. */
public void useHttpClientToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());

 HttpRequest.Builder requestBuilder = HttpRequest.newBuilder();
 metadata.forEach((k, v) -> requestBuilder.header("x-amz-meta-" + k, v));

 HttpClient httpClient = HttpClient.newHttpClient();
 try {
 final HttpResponse<Void> response = httpClient.send(requestBuilder
 .uri(new URL(presignedUrlString).toURI())

```

```

.PUT(HttpRequest.BodyPublishers.ofFile(Path.of(fileToPut.toURI()))
 .build(),
 HttpResponse.BodyHandlers.discarding());

 logger.info("HTTP response code is " + response.statusCode());

} catch (URISyntaxException | InterruptedException | IOException e) {
 logger.error(e.getMessage(), e);
}
}

```

## Use SdkHttpClient from the SDK for Java

```

/* Use the AWS SDK for Java V2 SdkHttpClient class to do the upload. */
public void useSdkHttpClientToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());

 try {
 URL presignedUrl = new URL(presignedUrlString);

 SdkHttpRequest.Builder requestBuilder = SdkHttpRequest.builder()
 .method(SdkHttpMethod.PUT)
 .uri(presignedUrl.toURI());
 // Add headers
 metadata.forEach((k, v) -> requestBuilder.putHeader("x-amz-meta-" + k, v));
 // Finish building the request.
 SdkHttpRequest request = requestBuilder.build();

 HttpExecuteRequest executeRequest = HttpExecuteRequest.builder()
 .request(request)
 .contentStreamProvider(new
FileContentStreamProvider(fileToPut.toPath()))
 .build();

 try (SdkHttpClient sdkHttpClient = ApacheHttpClient.create()) {
 HttpExecuteResponse response =
sdkHttpClient.prepareRequest(executeRequest).call();
 logger.info("Response code: {}", response.httpResponse().statusCode());
 }
 } catch (URISyntaxException | IOException e) {
 logger.error(e.getMessage(), e);
 }
}

```



```
}
}
```

See the [complete example](#) and [test](#) on GitHub.

## Cross-Region access for Amazon S3

When you work with Amazon Simple Storage Service (Amazon S3) buckets, you usually know the AWS Region for the bucket. The Region you work with is determined when you create the S3 client.

However, sometimes you might need to work with a specific bucket, but you don't know if it's located in the same Region that's set for the S3 client.

Instead of making more calls to determine the bucket Region, you can use the SDK to enable access to S3 buckets across different Regions.

### Setup

Support for cross-Region access became available with version 2.20.111 of the SDK. Use this version or a later one in your Maven build file for the `s3` dependency as shown in the following snippet.

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3</artifactId>
 <version>2.27.21</version>
</dependency>
```

Next, when you create your S3 client, enable cross-Region access as shown in the snippet. By default, access is not enabled.

```
S3AsyncClient client = S3AsyncClient.builder()
 .crossRegionAccessEnabled(true)
 .build();
```

### How the SDK provides cross-Region access

When you reference an existing bucket in a request, such as when you use the `putObject` method, the SDK initiates a request to the Region configured for the client.

If the bucket does not exist in that specific Region, the error response includes the actual Region where the bucket resides. The SDK then uses the correct Region in a second request.

To optimize future requests to the same bucket, the SDK caches this Region mapping in the client.

## Considerations

When you enable cross-Region bucket access, be aware that the first API call might result in increased latency if the bucket isn't in the client's configured Region. However, subsequent calls benefit from cached Region information, resulting in improved performance.

When you enable cross-Region access, access to the bucket is not affected. The user must be authorized to access the bucket in whatever Region it resides.

## Data integrity protection with checksums

Amazon Simple Storage Service (Amazon S3) provides the ability to specify a checksum when you upload an object. When you specify a checksum, it is stored with the object and can be validated when the object is downloaded.

Checksums provide an additional layer of data integrity when you transfer files. With checksums, you can verify data consistency by confirming that the received file matches the original file. For more information about checksums with Amazon S3, see the [Amazon Simple Storage Service User Guide](#) including the [supported algorithms](#).

You have the flexibility to choose the algorithm that best fits your needs and let the SDK calculate the checksum. Alternatively, you can provide a pre-computed checksum value by using one of the supported algorithms.

### Note

Beginning with version 2.30.0 of the AWS SDK for Java 2.x, the SDK provides default integrity protections by automatically calculating a CRC32 checksum for uploads. The SDK calculates this checksum if you don't provide a precalculated checksum value or if you don't specify an algorithm that the SDK should use to calculate a checksum.

The SDK also provides global settings for data integrity protections that you can set externally, which you can read about in the [AWS SDKs and Tools Reference Guide](#).

We discuss checksums in two request phases: uploading an object and downloading an object.

## Upload an object

When you upload an object with the `putObject` method and provide a checksum algorithm, the SDK computes the checksum for the specified algorithm.

The following code snippet shows a request to upload an object with a SHA256 checksum. When the SDK sends the request, it calculates the SHA256 checksum and uploads the object. Amazon S3 validates the integrity of the content by calculating the checksum and comparing it to the checksum provided by the SDK. Amazon S3 then stores the checksum with the object.

```
public void putObjectWithChecksum() {
 s3Client.putObject(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(ChecksumAlgorithm.SHA256),
 RequestBody.fromString("This is a test"));
}
```

If you don't provide a checksum algorithm with the request, the checksum behavior varies depending on the version of the SDK that you use as shown in the following table.

### Checksum behavior when no checksum algorithm is provided

Java SDK version	Checksum behavior
earlier than 2.30.0	The SDK doesn't automatically calculate a CRC-based checksum and provide it in the request.
2.30.0 or later	The SDK uses the CRC32 algorithm to calculate the checksum and provides it in the request. Amazon S3 validates the integrity of the transfer by computing its own CRC32 checksum and compares it to the checksum provided by the SDK. If the checksums match, the checksum is saved with the object.

## Use a pre-calculated checksum value

A pre-calculated checksum value provided with the request disables automatic computation by the SDK and uses the provided value instead.

The following example shows a request with a pre-calculated SHA256 checksum.

```
public void putObjectWithPrecalculatedChecksum(String bucketName, String key,
String filePath) {
 String checksum = calculateChecksum(filePath, "SHA-256");

 s3Client.putObject((b -> b
 .bucket(bucketName)
 .key(key)
 .checksumSHA256(checksum)),
 RequestBody.fromFile(Paths.get(filePath)));
}
```

If Amazon S3 determines the checksum value is incorrect for the specified algorithm, the service returns an error response.

## Multipart uploads

You can also use checksums with multipart uploads.

The SDK for Java 2.x provides two options to use checksums with multipart uploads. The first option uses the `S3TransferManager`.

The following transfer manager example specifies the SHA1 algorithm for the upload.

```
public void multipartUploadWithChecksumTm(String bucketName, String key, String
filePath) {
 S3TransferManager transferManager = S3TransferManager.create();
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(ChecksumAlgorithm.SHA1))
 .source(Paths.get(filePath))
 .build();
 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);
 fileUpload.completionFuture().join();
 transferManager.close();
}
```

```
}
```

If you don't provide a checksum algorithm when using the transfer manager for uploads, the SDK automatically calculates and checksum based on the CRC32 algorithm. The SDK performs this calculation for all versions of the SDK.

The second option uses the [S3Client API](#) (or the [S3AsyncClient API](#)) to perform the multipart upload. If you specify a checksum with this approach, you must specify the algorithm to use on the initiation of the upload. You must also specify the algorithm for each part request and provide the checksum calculated for each part after it is uploaded.

```
public void multipartUploadWithChecksumS3Client(String bucketName, String key,
String filePath) {
 ChecksumAlgorithm algorithm = ChecksumAlgorithm.CRC32;

 // Initiate the multipart upload.
 CreateMultipartUploadResponse createMultipartUploadResponse =
s3Client.createMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(algorithm)); // Checksum specified on initiation.
 String uploadId = createMultipartUploadResponse.uploadId();

 // Upload the parts of the file.
 int partNumber = 1;
 List<CompletedPart> completedParts = new ArrayList<>();
 ByteBuffer bb = ByteBuffer.allocate(1024 * 1024 * 5); // 5 MB byte buffer

 try (RandomAccessFile file = new RandomAccessFile(filePath, "r")) {
 long fileSize = file.length();
 long position = 0;
 while (position < fileSize) {
 file.seek(position);
 long read = file.getChannel().read(bb);

 bb.flip(); // Swap position and limit before reading from the buffer.
 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .checksumAlgorithm(algorithm) // Checksum specified on each part.
 .partNumber(partNumber)
 .build();
```

```

 UploadPartResponse partResponse = s3Client.uploadPart(
 uploadPartRequest,
 RequestBody.fromByteBuffer(bb));

 CompletedPart part = CompletedPart.builder()
 .partNumber(partNumber)
 .checksumCRC32(partResponse.checksumCRC32()) // Provide the
calculated checksum.
 .eTag(partResponse.eTag())
 .build();
 completedParts.add(part);

 bb.clear();
 position += read;
 partNumber++;
 }
} catch (IOException e) {
 System.err.println(e.getMessage());
}

// Complete the multipart upload.
s3Client.completeMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)

 .multipartUpload(CompletedMultipartUpload.builder().parts(completedParts).build()));
}

```

[Code for the complete examples](#) and [tests](#) are in the GitHub code examples repository.

## Download an object

When you use the [getObject](#) method to download an object, the SDK automatically validates the checksum when the `checksumMode` method of the builder for the `GetObjectRequest` is set to `ChecksumMode.ENABLED`.

The request in the following snippet directs the SDK to validate the checksum in the response by calculating the checksum and comparing the values.

```

public GetObjectResponse getObjectWithChecksum() {
 return s3Client.getObject(b -> b

```

```
 .bucket(bucketName)
 .key(key)
 .checksumMode(ChecksumMode.ENABLED))
 .response();
}
```

### Note

If the object wasn't uploaded with a checksum, no validation takes place.

## Other checksum calculation options

### Note

To verify the data integrity of transmitted data and to identify any transmission errors, we encourage users to keep the SDK default settings for the checksum calculation options. By default, the SDK adds this important check for many S3 operations including `PutObject` and `GetObject`.

If your use of Amazon S3 requires minimal checksum validation, however, you can disable many checks by changing the default configuration settings.

### Disable automatic checksum calculation unless it's required

You can disable automatic checksum calculation by the SDK for operations that support it, for example `PutObject` and `GetObject`. Some S3 operations, however, require a checksum calculation; you cannot disable checksum calculation for these operations.

The SDK provides separate settings for the calculation of a checksum for the payload of a request and for the payload of a response.

The following list describes the settings you can use to minimize checksum calculations at the different scopes.

- **All applications scope**—By changing the settings in environment variables or in a profile in the shared AWS config and `credentials` files, all applications can use these settings. These settings affect all service clients in all AWS SDK applications unless overridden at the application or service client scope.

- Add the settings in a profile:

```
[default]
request_checksum_calculation = WHEN_REQUIRED
response_checksum_validation = WHEN_REQUIRED
```

- Add environment variables:

```
AWS_REQUEST_CHECKSUM_CALCULATION=WHEN_REQUIRED
AWS_RESPONSE_CHECKSUM_VALIDATION=WHEN_REQUIRED
```

- **Current application scope**—You can set the Java system property `aws.requestChecksumCalculation` to `WHEN_REQUIRED` to limit checksum calculation. The corresponding system property for responses is `aws.responseChecksumValidation`.

These settings affect all SDK service clients in the application unless overridden during service client creation.

Set the system property at the start of your application:

```
import software.amazon.awssdk.core.SdkSystemSetting;
import software.amazon.awssdk.core.checksums.RequestChecksumCalculation;
import software.amazon.awssdk.core.checksums.ResponseChecksumValidation;
import software.amazon.awssdk.services.s3.S3Client;

class DemoClass {
 public static void main(String[] args) {

 System.setProperty(SdkSystemSetting.AWS_REQUEST_CHECKSUM_CALCULATION.property(), //
 Resolves to "aws.requestChecksumCalculation".
 "WHEN_REQUIRED");

 System.setProperty(SdkSystemSetting.AWS_RESPONSE_CHECKSUM_VALIDATION.property(), //
 Resolves to "aws.responseChecksumValidation".
 "WHEN_REQUIRED");

 S3Client s3Client = S3Client.builder().build();

 // Use s3Client.
 }
}
```



- **Single S3 service client scope**—You can configure a single S3 service client to calculate the minimum amount of checksums using builder methods:

```
import software.amazon.awssdk.core.checksums.RequestChecksumCalculation;
import software.amazon.awssdk.services.s3.S3Client;

public class RequiredChecksums {
 public static void main(String[] args) {
 S3Client s3 = S3Client.builder()
 .requestChecksumCalculation(RequestChecksumCalculation.WHEN_REQUIRED)
 .responseChecksumValidation(ResponseChecksumValidation.WHEN_REQUIRED)
 .build();

 // Use s3Client.
 }
 // ...
}
```

### Use the LegacyMd5Plugin for simplified MD5 compatibility

Along with the release of CRC32 checksum behavior with version 2.30.0, the SDK stopped calculating MD5 checksums on required operations.

If you need legacy MD5 checksum behavior for S3 operations, you can use the `LegacyMd5Plugin`, which was released with version 2.31.32 of the SDK.

The `LegacyMd5Plugin` is particularly useful when you need to maintain compatibility with applications that depend on the legacy MD5 checksum behavior, especially when working with third-party S3-compatible storage providers like those used with S3A filesystem connectors (Apache Spark, Iceberg).

To use the `LegacyMd5Plugin`, add it to your S3 client builder:

```
// For synchronous S3 client.
S3Client s3Client = S3Client.builder()
 .addPlugin(LegacyMd5Plugin.create())
 .build();

// For asynchronous S3 client.
S3AsyncClient asyncClient = S3AsyncClient.builder()
 .addPlugin(LegacyMd5Plugin.create())
```

```
.build();
```

If you want to add MD5 checksums to the operations that require checksums and want to skip adding SDK default checksums for operations that support checksums but are not required, you can enable the `ClientBuilder` options `requestChecksumCalculation` and `responseChecksumValidation` as `WHEN_REQUIRED`. This will add SDK default checksums only to operations that require checksums:

```
// Use the `LegacyMd5Plugin` with `requestChecksumCalculation` and
// `responseChecksumValidation` set to WHEN_REQUIRED.
S3AsyncClient asyncClient = S3AsyncClient.builder()
 .addPlugin(LegacyMd5Plugin.create())

 .requestChecksumCalculation(RequestChecksumCalculation.WHEN_REQUIRED)

 .responseChecksumValidation(ResponseChecksumValidation.WHEN_REQUIRED)
 .build();
```

This configuration is particularly useful when working with third-party S3-compatible storage systems that may not fully support the newer checksum algorithms but still require MD5 checksums for certain operations.

## Use a performant S3 client: AWS CRT-based S3 client

The AWS CRT-based S3 client—built on top of the [AWS Common Runtime \(CRT\)](#)—is an alternative S3 asynchronous client. It transfers objects to and from Amazon Simple Storage Service (Amazon S3) with enhanced performance and reliability by automatically using Amazon S3's [multipart upload API](#) and [byte-range fetches](#).

The AWS CRT-based S3 client improves transfer reliability in case there is a network failure. Reliability is improved by retrying individual failed parts of a file transfer without restarting the transfer from the beginning.

In addition, the AWS CRT-based S3 client offers enhanced connection pooling and Domain Name System (DNS) load balancing, which also improves throughput.

You can use the AWS CRT-based S3 client in place of the SDK's standard S3 asynchronous client and take advantage of its improved throughput right away.

**⚠ Important**

The AWS CRT-based S3 client does not currently support [SDK metrics collection](#) at the client level nor at the request level.

## AWS CRT-based components in the SDK

The AWS CRT-based S3 client, described in this topic, and the AWS CRT-based *HTTP* client are different components in the SDK.

The **AWS CRT-based S3 client** is an implementation of the [S3AsyncClient](#) interface and is used for working with the Amazon S3 service. It is an alternative to the Java-based implementation of the `S3AsyncClient` interface and offers several benefits.

The [AWS CRT-based HTTP client](#) is an implementation of the [SdkAsyncHttpClient](#) interface and is used for general HTTP communication. It is an alternative to the Netty implementation of the `SdkAsyncHttpClient` interface and offers several advantages.

Although both components use libraries from the [AWS Common Runtime](#), the AWS CRT-based S3 client uses the [aws-c-s3 library](#) and supports the [S3 multipart upload API](#) features. Since the AWS CRT-based HTTP client is meant for general purpose use, it does not support the S3 multipart upload API features.

## Add dependencies to use the AWS CRT-based S3 client

To use the AWS CRT-based S3 client, add the following two dependencies to your Maven project file. The example shows the minimum versions to use. Search the Maven central repository for the most recent versions of the [s3](#) and [aws-crt](#) artifacts.

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3</artifactId>
 <version>2.27.21</version>
</dependency>
<dependency>
 <groupId>software.amazon.awssdk.crt</groupId>
 <artifactId>aws-crt</artifactId>
 <version>0.30.11</version>
</dependency>
```

## Create an instance of the AWS CRT-based S3 client

Create an instance of the AWS CRT-based S3 client with default settings as shown in the following code snippet.

```
S3AsyncClient s3AsyncClient = S3AsyncClient.crtCreate();
```

To configure the client, use the AWS CRT client builder. You can switch from the standard S3 asynchronous client to AWS CRT-based client by changing the builder method.

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3AsyncClient;

S3AsyncClient s3AsyncClient =
 S3AsyncClient.crtBuilder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_WEST_2)
 .targetThroughputInGbps(20.0)
 .minimumPartSizeInBytes(8 * 1025 * 1024L)
 .build();
```

### Note

Some of the settings in the standard builder might not be currently supported in the AWS CRT client builder. Get the standard builder by calling `S3AsyncClient#builder()`.

## Use the AWS CRT-based S3 client

Use the AWS CRT-based S3 client to call Amazon S3 API operations. The following example demonstrates the [PutObject](#) and [GetObject](#) operations available through the AWS SDK for Java.

```
import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.core.async.AsyncResponseTransformer;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
```

```

S3AsyncClient s3Client = S3AsyncClient.crtCreate();

// Upload a local file to Amazon S3.
PutObjectResponse putObjectResponse =
 s3Client.putObject(req -> req.bucket(<BUCKET_NAME>)
 .key(<KEY_NAME>),
 AsyncRequestBody.fromFile(Paths.get(<FILE_NAME>)))
 .join();

// Download an object from Amazon S3 to a local file.
GetObjectResponse getObjectResponse =
 s3Client.getObject(req -> req.bucket(<BUCKET_NAME>)
 .key(<KEY_NAME>),
 AsyncResponseTransformer.toFile(Paths.get(<FILE_NAME>)))
 .join();

```

## Uploading streams of unknown size

One significant advantage of the AWS AWS CRT-based S3 client is its ability to handle input streams of unknown size efficiently. This is particularly useful when you need to upload data from a source where the total size cannot be determined in advance.

```

public PutObjectResponse crtClient_stream_unknown_size(String bucketName, String key,
 InputStream inputStream) {

 S3AsyncClient s3AsyncClient = S3AsyncClient.crtCreate();
 ExecutorService executor = Executors.newSingleThreadExecutor();
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
 executor); // 'null' indicates that the

 // content length is unknown.
 CompletableFuture<PutObjectResponse> responseFuture =
 s3AsyncClient.putObject(r -> r.bucket(bucketName).key(key), body)
 .exceptionally(e -> {
 if (e != null){
 logger.error(e.getMessage(), e);
 }
 return null;
 });
}

```

```
PutObjectResponse response = responseFuture.join(); // Wait for the response.
executor.shutdown();
return response;
}
```

This capability helps avoid common issues with traditional uploads where an incorrect content length specification can lead to truncated objects or failed uploads.

## Configuration limitations

The AWS CRT-based S3 client and Java-based S3 async client [provide comparable features](#), with the AWS CRT-based S3 client offering a performance edge. However, the AWS CRT-based S3 client lacks configuration settings that the Java-based S3 async client has. These settings include:

- *Client-level configuration*: API call attempt timeout, compression execution interceptors, metric publishers, custom execution attributes, custom advanced options, custom scheduled executor service, custom headers
- *Request-level configuration*: custom signers, API call attempt timeout

For a full listing of the configuration differences, see the API reference.

Java-based S3 async client	AWS CRT-based S3 client
Client-level configurations	Client-level configurations
• <a href="#">ClientOverrideConfiguration.Builder</a>	• <a href="#">S3CrtAsyncClientBuilder</a>
Request-level configurations	Request-level configurations
• <a href="#">RequestOverrideConfiguration.Builder</a>	• <a href="#">RequestOverrideConfiguration.Builder</a>
• <a href="#">AwsRequestOverrideConfiguration.Builder</a>	• <a href="#">AwsRequestOverrideConfiguration.Builder</a>

## Configure the Java-based S3 async client to use parallel transfers

Since version 2.27.5, the standard Java-based S3 async client supports automatic parallel transfers (multipart uploads and downloads). You configure support for parallel transfers when you create the Java-based S3 async client.

This section shows how to enable parallel transfers and how to customize the configuration.

## Create an instance of `S3AsyncClient`

When you create an `S3AsyncClient` instance without calling any of the `multipart*` methods on the [builder](#), parallel transfers are not enabled. Each of following statements create a Java-based S3 async client without support for multipart uploads and downloads.

### Create *without* multipart support

#### Example

```
import software.amazon.awssdk.auth.credentials.ProcessCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3AsyncClient;

S3AsyncClient s3Client = S3AsyncClient.create();

S3AsyncClient s3Client2 = S3AsyncClient.builder().build();

S3AsyncClient s3Client3 = S3AsyncClient.builder()
 .credentialsProvider(ProcessCredentialsProvider.builder().build())
 .region(Region.EU_NORTH_1)
 .build();
```

### Create *with* multipart support

To enable parallel transfers with default settings, call the `multipartEnabled` on the builder and pass in `true` as shown in the following example.

#### Example

```
S3AsyncClient s3AsyncClient2 = S3AsyncClient.builder()
 .multipartEnabled(true)
 .build();
```

The default value is 8 MiB for `thresholdInBytes` and `minimumPartSizeInBytes` settings.

If you customize the multipart settings, parallel transfers are automatically enabled as shown in the following.

## Example

```
import software.amazon.awssdk.services.s3.S3AsyncClient;
import static software.amazon.awssdk.transfer.s3.SizeConstant.MB;

S3AsyncClient s3AsyncClient2 = S3AsyncClient.builder()
 .multipartConfiguration(b -> b
 .thresholdInBytes(16 * MB)
 .minimumPartSizeInBytes(10 * MB))
 .build();
```

## Uploading streams of unknown size

The Java-based S3 asynchronous client with multipart enabled can efficiently handle input streams where the total size is not known in advance:

```
public PutObjectResponse asyncClient_multipart_stream_unknown_size(String bucketName,
 String key, InputStream inputStream) {

 S3AsyncClient s3AsyncClient =
 S3AsyncClient.builder().multipartEnabled(true).build();
 ExecutorService executor = Executors.newSingleThreadExecutor();
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
 executor); // 'null' indicates that the

 // content length is unknown.
 CompletableFuture<PutObjectResponse> responseFuture =
 s3AsyncClient.putObject(r -> r.bucket(bucketName).key(key), body)
 .exceptionally(e -> {
 if (e != null) {
 logger.error(e.getMessage(), e);
 }
 return null;
 });

 PutObjectResponse response = responseFuture.join(); // Wait for the response.
 executor.shutdown();
 return response;
}
```



This approach prevents issues that can occur when manually specifying an incorrect content length, such as truncated objects or failed uploads.

## Transfer files and directories with the Amazon S3 Transfer Manager

The Amazon S3 Transfer Manager is an open source, high level file transfer utility for the AWS SDK for Java 2.x. Use it to transfer files and directories to and from Amazon Simple Storage Service (Amazon S3).

When built on top of the [AWS CRT-based S3 client](#) or the [standard Java-based S3 async client with multipart enabled](#), the S3 Transfer Manager can take advantage of performance improvements such as the [multipart upload API](#) and [byte-range fetches](#).

With the S3 Transfer Manager, you can also monitor a transfer's progress in real time and pause the transfer for later execution.

### Get started

#### Add dependencies to your build file

To use the S3 Transfer Manager with enhanced multipart performance, configure your build file with necessary dependencies.

Use the AWS CRT-based S3 client

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.27.211</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3-transfer-manager</artifactId>
 </dependency>
 <dependency>
 <groupId>software.amazon.awssdk.crt</groupId>
```

```

 <artifactId>aws-crt</artifactId>
 <version>0.29.1432</version>
 </dependency>
</dependencies>

```

<sup>1</sup> [Latest version.](#)   <sup>2</sup> [Latest version.](#)

## Use the Java-based S3 async client

```

<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.27.211</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3-transfer-manager</artifactId>
 </dependency>
</dependencies>

```

<sup>1</sup> [Latest version.](#)

## Create an instance of the S3 Transfer Manager

To enable parallel transfer, you must pass in a AWS CRT-based S3 client OR a Java-based S3 async client with multipart enabled. The following examples shows how to configure a S3 Transfer Manager with custom settings.

### Use the AWS CRT-based S3 client

```

S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .multipartEnabled(true)
 .multipartConfiguration(c -> c.minimumPartSizeInBytes(8 * MB))

```

```
 .build();

 S3TransferManager transferManager = S3TransferManager.builder()
 .s3Client(s3AsyncClient)
 .build();
```

## Use the Java-based S3 async client

If the `aws-crt` dependency is not included in the build file, the S3 Transfer Manager is built on top of the standard Java-based S3 async client used in the SDK for Java 2.x.

### Custom configuration of S3 client - requires multipart enabled

```
S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
 .multipartEnabled(true)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

S3TransferManager transferManager = S3TransferManager.builder()
 .s3Client(s3AsyncClient)
 .build();
```

### No configuration of S3 client - multipart support automatically enabled

```
S3TransferManager transferManager = S3TransferManager.create();
```

## Upload a file to an S3 bucket

The following example shows a file upload example along with the optional use of a [LoggingTransferListener](#), which logs the progress of the upload.

To upload a file to Amazon S3 using the S3 Transfer Manager, pass an [UploadFileRequest](#) object to the S3TransferManager's [uploadFile](#) method.

The [FileUpload](#) object returned from the `uploadFile` method represents the upload process. After the request finishes, the [CompletedFileUpload](#) object contains information about the upload.

```
public void trackUploadFile(S3TransferManager transferManager, String bucketName,
 String key, URI filePathURI) {
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
```

```

 .putObjectRequest(b -> b.bucket(bucketName).key(key))
 .addTransferListener(LoggingTransferListener.create()) // Add
listener.

 .source(Paths.get(filePathURI))
 .build();

```

```
FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);
```

```
fileUpload.completionFuture().join();
/*
```

The SDK provides a `LoggingTransferListener` implementation of the `TransferListener` interface.

You can also implement the interface to provide your own logic.

Configure log4j2 with settings such as the following.

```

<Configuration status="WARN">
 <Appenders>
 <Console name="AlignedConsoleAppender" target="SYSTEM_OUT">
 <PatternLayout pattern="%m%n"/>
 </Console>
 </Appenders>

 <Loggers>
 <logger
name="software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener"
level="INFO" additivity="false">
 <AppenderRef ref="AlignedConsoleAppender"/>
 </logger>
 </Loggers>
</Configuration>

```

Log4j2 logs the progress. The following is example output for a 21.3 MB file upload.

```

Transfer initiated...
| | 0.0%
|==== | 21.1%
|===== | 60.5%
|===== | 100.0%
Transfer complete!

```

```
*/
```

```
}
```

## Imports

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedFileUpload;
import software.amazon.awssdk.transfer.s3.model.FileUpload;
import software.amazon.awssdk.transfer.s3.model.UploadFileRequest;
import software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener;
import java.net.URI;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Paths;
import java.util.UUID;
```

## Download a file from an S3 bucket

The following example shows a download example along with the optional use of a [LoggingTransferListener](#), which logs the progress of the download.

To download an object from an S3 bucket using the S3 Transfer Manager, build a [DownloadFileRequest](#) object and pass it to the [downloadFile](#) method.

The [FileDownload](#) object returned by the S3TransferManager's downloadFile method represents the file transfer. After the download completes, the [CompletedFileDownload](#) contains access to information about the download.

```
public void trackDownloadFile(S3TransferManager transferManager, String bucketName,
 String key, String downloadedFilePath) {
 DownloadFileRequest downloadFileRequest = DownloadFileRequest.builder()
 .getObjectRequest(b -> b.bucket(bucketName).key(key))
 .addTransferListener(LoggingTransferListener.create()) // Add
listener.
 .destination(Paths.get(downloadedFilePath))
 .build();

 FileDownload downloadFile = transferManager.downloadFile(downloadFileRequest);

 CompletedFileDownload downloadResult = downloadFile.completionFuture().join();
 /*
 The SDK provides a LoggingTransferListener implementation of the
 TransferListener interface.
```

You can also implement the interface to provide your own logic.

Configure log4J2 with settings such as the following.

```
<Configuration status="WARN">
 <Appenders>
 <Console name="AlignedConsoleAppender" target="SYSTEM_OUT">
 <PatternLayout pattern="%m%n"/>
 </Console>
 </Appenders>

 <Loggers>
 <logger
name="software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener"
level="INFO" additivity="false">
 <AppenderRef ref="AlignedConsoleAppender"/>
 </logger>
 </Loggers>
</Configuration>
```

Log4J2 logs the progress. The following is example output for a 21.3 MB file download.

```
Transfer initiated...
|=====| 39.4%
|=====| 78.8%
|=====| 100.0%
Transfer complete!

 */
}
```

## Imports

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedFileDownload;
import software.amazon.awssdk.transfer.s3.model.DownloadFileRequest;
import software.amazon.awssdk.transfer.s3.model.FileDownload;
import software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener;

import java.io.IOException;
```

```
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.UUID;
```

## Copy an Amazon S3 object to another bucket

The following example shows how to copy an object with the S3 Transfer Manager.

To begin the copy of an object from an S3 bucket to another bucket, create a basic [CopyObjectRequest](#) instance.

Next, wrap the basic CopyObjectRequest in a [CopyRequest](#) that can be used by the S3 Transfer Manager.

The Copy object returned by the S3TransferManager's copy method represents the copy process. After the copy process completes, the [CompletedCopy](#) object contains details about the response.

```
public String copyObject(S3TransferManager transferManager, String bucketName,
 String key, String destinationBucket, String destinationKey) {
 CopyObjectRequest copyObjectRequest = CopyObjectRequest.builder()
 .sourceBucket(bucketName)
 .sourceKey(key)
 .destinationBucket(destinationBucket)
 .destinationKey(destinationKey)
 .build();

 CopyRequest copyRequest = CopyRequest.builder()
 .copyObjectRequest(copyObjectRequest)
 .build();

 Copy copy = transferManager.copy(copyRequest);

 CompletedCopy completedCopy = copy.completionFuture().join();
 return completedCopy.response().copyObjectResult().eTag();
}
```

**Note**

To perform a cross-Region copy with the S3 Transfer Manager, enable `crossRegionAccessEnabled` on the AWS CRT-based S3 client builder as shown in the following snippet.

```
S3AsyncClient s3AsyncClient = S3AsyncClient.crtBuilder()
 .crossRegionAccessEnabled(true)
 .build();

S3TransferManager transferManager = S3TransferManager.builder()
 .s3Client(s3AsyncClient)
 .build();
```

**Imports**

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.model.CopyObjectRequest;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedCopy;
import software.amazon.awssdk.transfer.s3.model.Copy;
import software.amazon.awssdk.transfer.s3.model.CopyRequest;

import java.util.UUID;
```

**Upload a local directory to an S3 bucket**

The following example demonstrates how you can upload a local directory to S3.

Start by calling the [uploadDirectory](#) method of the `S3TransferManager` instance, passing in an [UploadDirectoryRequest](#).

The [DirectoryUpload](#) object represents the upload process, which generates a [CompletedDirectoryUpload](#) when the request completes. The `CompletedDirectoryUpload` object contains information about the results of the transfer, including which files failed to transfer.

```
public Integer uploadDirectory(S3TransferManager transferManager,
```



```

 URI sourceDirectory, String bucketName) {
 DirectoryUpload directoryUpload =
transferManager.uploadDirectory(UploadDirectoryRequest.builder()
 .source(Paths.get(sourceDirectory))
 .bucket(bucketName)
 .build());

 CompletedDirectoryUpload completedDirectoryUpload =
directoryUpload.completionFuture().join();
 completedDirectoryUpload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryUpload.failedTransfers().size();
}

```

## Imports

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedDirectoryUpload;
import software.amazon.awssdk.transfer.s3.model.DirectoryUpload;
import software.amazon.awssdk.transfer.s3.model.UploadDirectoryRequest;

import java.net.URI;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Paths;

```

## Download S3 bucket objects to a local directory

You can download the objects in an S3 bucket to a local directory as shown in the following example.

To download the objects in an S3 bucket to a local directory, begin by calling the [downloadDirectory](#) method of the Transfer Manager, passing in a [DownloadDirectoryRequest](#).

The [DirectoryDownload](#) object represents the download process, which generates a [CompletedDirectoryDownload](#) when the request completes. The `CompletedDirectoryDownload` object contains information about the results of the transfer, including which files failed to transfer.

```
public Integer downloadObjectsToDirectory(S3TransferManager transferManager,
 URI destinationPathURI, String bucketName) {
 DirectoryDownload directoryDownload =
transferManager.downloadDirectory(DownloadDirectoryRequest.builder()
 .destination(Paths.get(destinationPathURI))
 .bucket(bucketName)
 .build());
 CompletedDirectoryDownload completedDirectoryDownload =
directoryDownload.completionFuture().join();

 completedDirectoryDownload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryDownload.failedTransfers().size();
}
```

## Imports

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedDirectoryDownload;
import software.amazon.awssdk.transfer.s3.model.DirectoryDownload;
import software.amazon.awssdk.transfer.s3.model.DownloadDirectoryRequest;
import java.io.IOException;
import java.net.URI;
import java.net.URISyntaxException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.HashSet;
import java.util.Set;
import java.util.UUID;
import java.util.stream.Collectors;
```

## See complete examples

[GitHub contains the complete](#) code for all examples on this page.

## Work with S3 Event Notifications

To help you monitor activity in your buckets, Amazon S3 can send notifications when certain events happen. The Amazon S3 User Guide provides information on the [notifications that a bucket can send out](#).

You can set up a bucket to send events to four possible destinations using the SDK for Java:

- Amazon Simple Notification Service topics
- Amazon Simple Queue Service queues
- AWS Lambda functions
- Amazon EventBridge

When you setup up a bucket to send events to EventBridge, you have the ability to configure an EventBridge rule to fanout the same event to multiple destinations. When you configure your bucket to send directly to one of the first three destinations, only one destination type can be specified for each event.

In the next section, you'll see how to configure a bucket using the SDK for Java to send S3 Event Notifications in two ways: directly to an Amazon SQS queue and to EventBridge.

The last section shows you how to use the S3 Event Notifications API to work with notifications in an object-oriented way.

### Configure a bucket to send directly to a destination

The following example configures a bucket to send notifications when *object create* events or *object tagging* events occur against a bucket.

```
static void processS3Events(String bucketName, String queueArn) {
 // Configure the bucket to send Object Created and Object Tagging notifications to
 // an existing SQS queue.
 s3Client.putBucketNotificationConfiguration(b -> b
 .notificationConfiguration(ncb -> ncb
 .queueConfigurations(qcb -> qcb
 .events(Event.S3_OBJECT_CREATED, Event.S3_OBJECT_TAGGING)
 .queueArn(queueArn)))
 .bucket(bucketName)
);
}
```

The code shown above sets up one queue to receive two types of events. Conveniently, the `queueConfigurations` method allows you to set multiple queue destinations if needed. Also, in the `notificationConfiguration` method you can set additional destinations, such as one or more Amazon SNS topics or one or more Lambda functions. The following snippet shows an example with two queues and three types of destinations.

```
s3Client.putBucketNotificationConfiguration(b -> b
 .notificationConfiguration(ncb -> ncb
 .queueConfigurations(qcb -> qcb
 .events(Event.S3_OBJECT_CREATED,
 Event.S3_OBJECT_TAGGING)
 .queueArn(queueArn),
 qcb2 -> qcb2.<...>)
 .topicConfigurations(tcb -> tcb.<...>)
 .lambdaFunctionConfigurations(lfcb -> lfcb.<...>))
 .bucket(bucketName)
);
```

The Code Examples GitHub repository contains the [complete example](#) to send S3 event notifications directly to a queue.

## Configure a bucket to send to EventBridge

The following example configures a bucket to send notifications to EventBridge.

```
public static String setBucketNotificationToEventBridge(String bucketName) {
 // Enable bucket to emit S3 Event notifications to EventBridge.
 s3Client.putBucketNotificationConfiguration(b -> b
 .bucket(bucketName)
 .notificationConfiguration(b1 -> b1
 .eventBridgeConfiguration(SdkBuilder::build))
 .build());
}
```

When you configure a bucket to send events to EventBridge, you simply indicate the EventBridge destination, not the types of events nor the ultimate destination that EventBridge will dispatch to. You configure the ultimate targets and event types by using the Java SDK's EventBridge client.

The following code shows how to configure EventBridge to fan out *object created* events to a topic and a queue.

```
public static String configureEventBridge(String topicArn, String queueArn) {
```

```

try {
 // Create an EventBridge rule to route Object Created notifications.
 PutRuleRequest putRuleRequest = PutRuleRequest.builder()
 .name(RULE_NAME)
 .eventPattern("""
 {
 "source": ["aws.s3"],
 "detail-type": ["Object Created"],
 "detail": {
 "bucket": {
 "name": ["%s"]
 }
 }
 }
 """).formatted(bucketName))
 .build();

 // Add the rule to the default event bus.
 PutRuleResponse putRuleResponse = eventBridgeClient.putRule(putRuleRequest)
 .whenComplete((r, t) -> {
 if (t != null) {
 logger.error("Error creating event bus rule: " +
 t.getMessage(), t);
 throw new RuntimeException(t.getCause().getMessage(), t);
 }
 logger.info("Event bus rule creation request sent successfully.
ARN is: {}", r.ruleArn());
 }).join();

 // Add the existing SNS topic and SQS queue as targets to the rule.
 eventBridgeClient.putTargets(b -> b
 .eventBusName("default")
 .rule(RULE_NAME)
 .targets(List.of (
 Target.builder()
 .arn(queueArn)
 .id("Queue")
 .build(),
 Target.builder()
 .arn(topicArn)
 .id("Topic")
 .build()
)
).join());

```

```
 return putRuleResponse.ruleArn();
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}
```

To work with EventBridge in your Java code, add a dependency on the eventbridge artifact to your Maven pom.xml file.

```
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>eventbridge</artifactId>
</dependency>
```

The Code Examples GitHub repository contains the [complete example](#) to send S3 event notifications to EventBridge and then to a topic and queue.

## Use the S3 Event Notifications API to process events

After a destination receives S3 notification events, you can process them in an object-oriented way by using the S3 Event Notifications API. You can use the S3 Event Notifications API to work with event notifications that are dispatched directly to a target (as shown in the [first example](#)), but not with notifications routed through EventBridge. S3 event notifications sent by buckets to EventBridge contain a [different structure](#) that the S3 Event Notifications API does not currently handle.

### Add dependency

The S3 Event Notifications API was released with version 2.25.11 of the SDK for Java 2.x.

To use the S3 Event Notifications API, add the required dependency element to your Maven pom.xml as shown in the following snippet.

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.X.X1</version>
 <type>pom</type>
```

```

 <scope>import</scope>
 </dependency>
</dependencies>
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>s3-event-notifications</artifactId>
 </dependency>
</dependencies>

```

<sup>1</sup> [Latest version.](#)

## Use the `S3EventNotification` class

### Create an `S3EventNotification` instance from a JSON string

To convert a JSON string into an `S3EventNotification` object, use the static methods of the `S3EventNotification` class as shown in the following example.

```

import software.amazon.awssdk.eventnotifications.s3.model.S3EventNotification
import software.amazon.awssdk.eventnotifications.s3.model.S3EventNotificationRecord
import software.amazon.awssdk.services.sqs.model.Message;

public class S3EventNotificationExample {
 ...

 void receiveMessage(Message message) {
 // Message received from SQSClient.
 String sqsEventBody = message.body();
 S3EventNotification s3EventNotification =
 S3EventNotification.fromJson(sqsEventBody);

 // Use getRecords() to access all the records in the notification.

 List<S3EventNotificationRecord> records = s3EventNotification.getRecords();

 S3EventNotificationRecord record = records.stream().findFirst();
 // Use getters on the record to access individual attributes.
 String awsRegion = record.getAwsRegion();
 String eventName = record.getEventName();
 String eventSource = record.getEventSource();
 }
}

```

```
}
}
```

In this example, the `fromJson` method converts the JSON string into an `S3EventNotification` object. Missing fields in the JSON string will result in `null` values in the corresponding Java object fields and any extra fields in the JSON will be ignored.

Other APIs for an event notification record can be found in API reference for [S3EventNotificationRecord](#).

### Convert an `S3EventNotification` instance to a JSON string

Use the `toJson` (or `toJsonPretty`) method to convert an `S3EventNotification` object into a JSON string as shown in the following example.

```
import software.amazon.awssdk.eventnotifications.s3.model.S3EventNotification

public class S3EventNotificationExample {
 ...

 void toJsonString(S3EventNotification event) {

 String json = event.toJson();
 String jsonPretty = event.toJsonPretty();

 System.out.println("JSON: " + json);
 System.out.println("Pretty JSON: " + jsonPretty);
 }
}
```

Fields for `GlacierEventData`, `ReplicationEventData`, `IntelligentTieringEventData`, and `LifecycleEventData` are excluded from the JSON if they are `null`. Other `null` fields will be serialized as `null`.

The following shows example output of the `toJsonPretty` method for an S3 object tagging event.

```
{
 "Records" : [{
 "eventVersion" : "2.3",
 "eventSource" : "aws:s3",
```



```

 "awsRegion" : "us-east-1",
 "eventTime" : "2024-07-19T20:09:18.551Z",
 "eventName" : "ObjectTagging:Put",
 "userIdentity" : {
 "principalId" : "AWS:XXXXXXXXXXXX"
 },
 "requestParameters" : {
 "sourceIPAddress" : "XXX.XX.XX.XX"
 },
 "responseElements" : {
 "x-amz-request-id" : "XXXXXXXXXXXX",
 "x-amz-id-2" : "XXXXXXXXXXXX"
 },
 "s3" : {
 "s3SchemaVersion" : "1.0",
 "configurationId" : "XXXXXXXXXXXX",
 "bucket" : {
 "name" : "amzn-s3-demo-bucket",
 "ownerIdentity" : {
 "principalId" : "XXXXXXXXXXXX"
 },
 "arn" : "arn:aws:s3:::XXXXXXXXXXXX"
 },
 "object" : {
 "key" : "akey",
 "size" : null,
 "eTag" : "XXXXXXXXXX",
 "versionId" : null,
 "sequencer" : null
 }
 }
}]
}

```

A [complete example](#) is available in GitHub that shows how to use the API to work with notifications received by an Amazon SQS queue.

## Process S3 Events in Lambda with Java Libraries: AWS SDK for Java 2.x and `aws-lambda-java-events`

Instead of using the SDK for Java 2.x to process Amazon S3 event notifications in a Lambda function, you can use the [aws-lambda-java-events](#) library at version 3.x.x. AWS maintains the `aws-lambda-java-events` library independently, and it has its own dependency requirements.

The `aws-lambda-java-events` library works only with S3 events in Lambda functions, whereas the SDK for Java 2.x works with S3 events in Lambda functions, Amazon SNS, and Amazon SQS.

Both approaches model the JSON event notification payload in an object-oriented way with similar APIs. The following table shows the notable differences between using the two approaches.

	AWS SDK for Java	aws-lambda-java-events library
Package naming	<code>software.amazon.awssdk.eventnotifications.s3.model.S3EventNotification</code>	<code>com.amazonaws.services.lambda.runtime.events.models.s3.S3EventNotification</code>
RequestHandler parameter	Write your Lambda function's RequestHandler implementation to receive a JSON String:  <pre>import com.amazonaws.services.lambda.runtime.Context; import com.amazonaws.services.lambda.runtime.RequestHandler; import software.amazon.awssdk.eventnotifications.s3.model.S3EventNotification;  public class Handler implements RequestHandler&lt;String, String&gt; {      @Override     public String handleRequest(String</pre>	Write your Lambda function's RequestHandler implementation to receive an S3Event object:  <pre>import com.amazonaws.services.lambda.runtime.Context; import com.amazonaws.services.lambda.runtime.RequestHandler; import com.amazonaws.services.lambda.runtime.events.S3Event;  public class Handler implements RequestHandler&lt;S3Event, String&gt; {      @Override     public String handleRequest(S3Ev</pre>

	AWS SDK for Java	aws-lambda-java-events library
	<pre>jsonS3Event, Context context) {     S3EventNo tification s3Event = S3EventNotification      .fromJson(jsonS3Ev ent);     // Work with the s3Event object.      ... } }</pre>	<pre>ent s3event, Context context) {     // Work with the s3Event object.      ... } }</pre>

	AWS SDK for Java	aws-lambda-java-events library
Maven dependencies	<pre> &lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.X.X&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifactId&gt;s3- event-notifications&lt;/ artifactId&gt;   &lt;/dependency&gt;   &lt;!-- Add other SDK dependencies that you need. --&gt; &lt;/dependencies&gt; </pre>	<pre> &lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.X.X&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;!-- The following two dependencies are for the       aws-lambda- java-events library. -- &gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-lambda-java- core&lt;/artifactId&gt;       &lt;version&gt; 1.2.3&lt;/version&gt;     &lt;/dependency&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt; </pre>

	AWS SDK for Java	aws-lambda-java-events library
		<pre> &lt;artifact Id&gt;aws-lambda-java- events&lt;/artifactId&gt; &lt;version&gt; 3.15.0&lt;/version&gt; &lt;/dependency&gt; &lt;!-- Add other SDK dependencies that you need. --&gt; &lt;/dependencies&gt; </pre>

## Work with Amazon Simple Notification Service

With Amazon Simple Notification Service, you can easily push real-time notification messages from your applications to subscribers over multiple communication channels. This topic describes how to perform some of the basic functions of Amazon SNS.

### Create a topic

A **topic** is a logical grouping of communication channels that defines which systems to send a message to, for example, fanning out a message to AWS Lambda and an HTTP webhook. You send messages to Amazon SNS, then they're distributed to the channels defined in the topic. This makes the messages available to subscribers.

To create a topic, first build a [CreateTopicRequest](#) object, with the name of the topic set using the `name()` method in the builder. Then, send the request object to Amazon SNS by using the `createTopic()` method of the [SnsClient](#). You can capture the result of this request as a [CreateTopicResponse](#) object, as demonstrated in the following code snippet.

### Imports

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreateTopicRequest;
import software.amazon.awssdk.services.sns.model.CreateTopicResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

```

## Code

```
public static String createSNSTopic(SnsClient snsClient, String topicName) {

 CreateTopicResponse result = null;
 try {
 CreateTopicRequest request = CreateTopicRequest.builder()
 .name(topicName)
 .build();

 result = snsClient.createTopic(request);
 return result.topicArn();
 } catch (SnsException e) {

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

See the [complete example](#) on GitHub.

## List your Amazon SNS topics

To retrieve a list of your existing Amazon SNS topics, build a [ListTopicsRequest](#) object. Then, send the request object to Amazon SNS by using the `listTopics()` method of the `SnsClient`. You can capture the result of this request as a [ListTopicsResponse](#) object.

The following code snippet prints out the HTTP status code of the request and a list of Amazon Resource Names (ARNs) for your Amazon SNS topics.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.ListTopicsRequest;
import software.amazon.awssdk.services.sns.model.ListTopicsResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
```

## Code

```
public static void listSNSTopics(SnsClient snsClient) {
```

```
try {
 ListTopicsRequest request = ListTopicsRequest.builder()
 .build();

 ListTopicsResponse result = snsClient.listTopics(request);
 System.out.println("Status was " + result.sdkHttpResponse().statusCode() +
 "\n\nTopics\n\n" + result.topics());

} catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

See the [complete example](#) on GitHub.

## Subscribe an endpoint to a topic

After you create a topic, you can configure which communication channels will be endpoints for that topic. Messages are distributed to these endpoints after Amazon SNS receives them.

To configure a communication channel as an endpoint for a topic, subscribe that endpoint to the topic. To start, build a [SubscribeRequest](#) object. Specify the communication channel (for example, lambda or email) as the `protocol()`. Set the `endpoint()` to the relevant output location (for example, the ARN of a Lambda function or an email address), and then set the ARN of the topic to which you want to subscribe as the `topicArn()`. Send the request object to Amazon SNS by using the `subscribe()` method of the `SnsClient`. You can capture the result of this request as a [SubscribeResponse](#) object.

The following code snippet shows how to subscribe an email address to a topic.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;
```

### Code

```
public static void subEmail(SnsClient snsClient, String topicArn, String email) {

 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("email")
 .endpoint(email)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("Subscription ARN: " + result.subscriptionArn() + "\n\n"
 + "Status is " + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## Publish a message to a topic

After you have a topic and one or more endpoints configured for it, you can publish a message to it. To start, build a [PublishRequest](#) object. Specify the message ( ) to send, and the ARN of the topic (topicArn( )) to send it to. Then, send the request object to Amazon SNS by using the publish( ) method of the SnsClient. You can capture the result of this request as a [PublishResponse](#) object.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
```

### Code

```
public static void pubTopic(SnsClient snsClient, String message, String topicArn) {
```



```
try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .topicArn(topicArn)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out.println(result.messageId() + " Message sent. Status is " +
 result.sdkHttpResponse().statusCode());

} catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

See the [complete example](#) on GitHub.

## Unsubscribe an endpoint from a topic

You can remove the communication channels configured as endpoints for a topic. After doing that, the topic itself continues to exist and distribute messages to any other endpoints configured for that topic.

To remove a communication channel as an endpoint for a topic, unsubscribe that endpoint from the topic. To start, build an [UnsubscribeRequest](#) object and set the ARN of the topic you want to unsubscribe from as the `subscriptionArn()`. Then send the request object to SNS by using the `unsubscribe()` method of the `SnsClient`. You can capture the result of this request as an [UnsubscribeResponse](#) object.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.UnsubscribeRequest;
import software.amazon.awssdk.services.sns.model.UnsubscribeResponse;
```

### Code

```
public static void unSub(SnsClient snsClient, String subscriptionArn) {
```

```
try {
 UnsubscribeRequest request = UnsubscribeRequest.builder()
 .subscriptionArn(subscriptionArn)
 .build();

 UnsubscribeResponse result = snsClient.unsubscribe(request);

 System.out.println("\n\nStatus was " +
result.sdkHttpResponse().statusCode()
 + "\n\nSubscription was removed for " + request.subscriptionArn());

} catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

See the [complete example](#) on GitHub.

## Delete a topic

To delete an Amazon SNS topic, first build a [DeleteTopicRequest](#) object with the ARN of the topic set as the `topicArn()` method in the builder. Then send the request object to Amazon SNS by using the `deleteTopic()` method of the `SnsClient`. You can capture the result of this request as a [DeleteTopicResponse](#) object, as demonstrated in the following code snippet.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.DeleteTopicRequest;
import software.amazon.awssdk.services.sns.model.DeleteTopicResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
```

### Code

```
public static void deleteSNSTopic(SnsClient snsClient, String topicArn) {

 try {
 DeleteTopicRequest request = DeleteTopicRequest.builder()
 .topicArn(topicArn)
 .build();
```

```
 DeleteTopicResponse result = snsClient.deleteTopic(request);
 System.out.println("\n\nStatus was " +
 result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

See the [complete example](#) on GitHub.

For more information, see the [Amazon Simple Notification Service Developer Guide](#).

## Work with Amazon Simple Queue Service

This section provides examples of programming [Amazon Simple Queue Service](#) using the AWS SDK for Java 2.x.

The following examples include only the code needed to demonstrate each technique. The [complete example code is available on GitHub](#). From there, you can download a single source file or clone the repository locally to get all the examples to build and run.

### Topics

- [Use automatic request batching for Amazon SQS with the AWS SDK for Java 2.x](#)
- [Work with Amazon Simple Queue Service message queues](#)
- [Send, receive, and delete Amazon Simple Queue Service messages](#)

## Use automatic request batching for Amazon SQS with the AWS SDK for Java 2.x

The Automatic Request Batching API for Amazon SQS is a high-level library that provides an efficient way to batch and buffer requests for SQS operations. By using the batching API, you reduce the number of requests to SQS, which improves throughput and minimizes costs.

Because the batch API methods match the [SqsAsyncClient](#) methods—`sendMessage`, `changeMessageVisibility`, `deleteMessage`, `receiveMessage`—you can use the batch API as a drop-in replacement with minimal changes.

This topic gives you an overview of how to configure and work with the Automatic Request Batching API for Amazon SQS.

## Check prerequisites

You need to use version 2.28.0 or later of the SDK for Java 2.x to have access to the batching API. Your Maven `pom.xml` should at least contain the following elements.

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.28.231</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>
<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>sqs</artifactId>
 </dependency>
</dependencies>
```

<sup>1</sup> [Latest version](#)

## Create a batch manager

The automatic request batching API is implemented by the [SqsAsyncBatchManager](#) interface. You can create an instance of the manager a couple ways.

### Default configuration by using `SqsAsyncClient`

The simplest way you can create a batch manager is to call the `batchManager` factory method on an existing [SqsAsyncClient](#) instance. The simple approach is shown in the following snippet.

```
SqsAsyncClient asyncClient = SqsAsyncClient.create();
SqsAsyncBatchManager sqsAsyncBatchManager = asyncClient.batchManager();
```

When you use this approach, the `SqsAsyncBatchManager` instance uses the default values that are shown in the table in the [the section called “Configuration settings”](#) section. Additionally, the `SqsAsyncBatchManager` instance uses the `ExecutorService` of the `SqsAsyncClient` instance that it was created from.

## Custom configuration by using `SqsAsyncBatchManager.Builder`

For more advanced use cases, you can customize the batch manager using the [SqsAsyncBatchManager.Builder](#). By using this approach to create a `SqsAsyncBatchManager` instance, you can fine tune the batching behavior. The following snippet shows an example of how to use the builder to customize batching behavior.

```
SqsAsyncBatchManager batchManager = SqsAsyncBatchManager.builder()
 .client(SqsAsyncClient.create())
 .scheduledExecutor(Executors.newScheduledThreadPool(5))
 .overrideConfiguration(b -> b
 .receiveMessageMinWaitDuration(Duration.ofSeconds(10))
 .receiveMessageVisibilityTimeout(Duration.ofSeconds(1))
 .receiveMessageAttributeNames(Collections.singletonList("*"))

 .receiveMessageSystemAttributeNames(Collections.singletonList(MessageSystemAttributeName.ALL))
 .build();
```

When you use this approach, you can adjust the settings on the `BatchOverrideConfiguration` object that are shown in the table in the [the section called “Configuration settings”](#) section. You can also provide a custom [ScheduledExecutorService](#) for the batch manager by using this approach.

## Send messages

To send messages with the batch manager, use the [SqsAsyncBatchManager#sendMessage](#) method. The SDK buffers requests and sends them as a batch when the `maxBatchSize` or `sendRequestFrequency` values are reached.

The following example show a `sendMessage` request immediately following by another request. In this case, the SDK sends both messages in a single batch.

```
// Sending the first message
CompletableFuture<SendMessageResponse> futureOne =
 sqsAsyncBatchManager.sendMessage(r -> r.messageBody("One").queueUrl("queue"));
```

```
// Sending the second message
CompletableFuture<SendMessageResponse> futureTwo =
 sqsAsyncBatchManager.sendMessage(r -> r.messageBody("Two").queueUrl("queue"));

// Waiting for both futures to complete and retrieving the responses
SendMessageResponse messageOne = futureOne.join();
SendMessageResponse messageTwo = futureTwo.join();
```

## Change the message visibility timeout

You can change the visibility timeout of messages in a batch by using the [SqsAsyncBatchManager#changeMessageVisibility](#) method. The SDK buffers requests and sends them as a batch when the `maxBatchSize` or `sendRequestFrequency` values are reached.

The following example shows how to call the `changeMessageVisibility` method.

```
CompletableFuture<ChangeMessageVisibilityResponse> futureOne =
 sqsAsyncBatchManager.changeMessageVisibility(r ->
 r.receiptHandle("receiptHandle")
 .queueUrl("queue"));
ChangeMessageVisibilityResponse response = futureOne.join();
```

## Delete messages

You can delete messages in a batch using the [SqsAsyncBatchManager#deleteMessage](#) method. The SDK buffers requests and sends them as a batch when the `maxBatchSize` or `sendRequestFrequency` values are reached.

The following example shows how you can call the `deleteMessage` method.

```
CompletableFuture<DeleteMessageResponse> futureOne =
 sqsAsyncBatchManager.deleteMessage(r ->
 r.receiptHandle("receiptHandle")
 .queueUrl("queue"));
DeleteMessageResponse response = futureOne.join();
```

## Receive messages

### Use default settings

When you poll the [SqsAsyncBatchManager#receiveMessage](#) method in your application, the batch manager fetches messages from its internal buffer, which the SDK automatically updates in the background.

The following example shows how to call the `receiveMessage` method.

```
CompletableFuture<ReceiveMessageResponse> responseFuture =
 sqsAsyncBatchManager.receiveMessage(r -> r.queueUrl("queueUrl"));
```

### Use custom settings

If you want to customize the request further, for example by setting custom wait times and specifying the number of messages to retrieve, you can customize the request as shown in the following example.

```
CompletableFuture<ReceiveMessageResponse> response =
 sqsAsyncBatchManager.receiveMessage(r ->
 r.queueUrl("queueUrl")
 .waitTimeSeconds(5)
 .visibilityTimeout(20));
```

#### Note

If you call `receiveMessage` with a [ReceiveMessageRequest](#) that includes any of the following parameters, the SDK bypasses the batch manager and sends a regular asynchronous `receiveMessage` request:

- `messageAttributeNames`
- `messageSystemAttributeNames`
- `messageSystemAttributeNamesWithStrings`
- `overrideConfiguration`

## Override configuration settings for SqsAsyncBatchManager

You can adjust the following settings when you create an `SqsAsyncBatchManager` instance. The following list of settings are available on the [BatchOverrideConfiguration.Builder](#).

Setting	Description	Default value
<code>maxBatchSize</code>	Maximum number of request per batch for each <code>SendMessageBatchRequest</code> , <code>ChangeMessageVisibilityBatchRequest</code> , or <code>DeleteMessageBatchRequest</code> . The maximum value is 10.	10
<code>sendRequestFrequency</code>	Time before sending a batch, unless <code>maxBatchSize</code> is reached earlier. Higher values may reduce requests but increase latency.	200ms
<code>receiveMessageVisibilityTimeout</code>	Visibility timeout for messages. If unset, the queue's default is used.	Queue's default
<code>receiveMessageMinWaitDuration</code>	Minimum wait time for <code>receiveMessage</code> requests. Avoid setting to 0 to prevent CPU waste.	50ms
<code>receiveMessageSystemAttributeNames</code>	List of <a href="#">system attribute names</a> to request for <code>receiveMessage</code> calls.	None
<code>receiveMessageAttributeNames</code>	List of <a href="#">attribute names</a> to request for <code>receiveMessage</code> calls.	None



## Work with Amazon Simple Queue Service message queues

A *message queue* is the logical container used for sending messages reliably in Amazon Simple Queue Service. There are two types of queues: *standard* and *first-in, first-out* (FIFO). To learn more about queues and the differences between these types, see the [Amazon Simple Queue Service Developer Guide](#).

This topic describes how to create, list, delete, and get the URL of an Amazon Simple Queue Service queue by using the AWS SDK for Java.

The `sqsClient` variable that is used in the following examples can be created from the following snippet.

```
SqsClient sqsClient = SqsClient.create();
```

When you create an `SqsClient` by using the static `create()` method, the SDK configures the Region by using the [default region provider chain](#) and the credentials by using the [default credentials provider chain](#).

### Create a queue

Use the `SqsClient`'s `createQueue` method, and provide a [CreateQueueRequest](#) object that describes the queue parameters as shown in the following code snippet.

#### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

#### Code

```
CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();

sqsClient.createQueue(createQueueRequest);
```

See the [complete sample](#) on GitHub.

## List queues

To list the Amazon Simple Queue Service queues for your account, call the `SqsClient`'s `listQueues` method with a [ListQueuesRequest](#) object.

When you use the form of the [listQueues](#) method that takes no parameters, the service returns *all queues*—up to 1,000 queues.

You can supply a queue name prefix to the [ListQueuesRequest](#) object to limit the results to queues that match that prefix as shown in the following code.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
String prefix = "que";

try {
 ListQueuesRequest listQueuesRequest =
ListQueuesRequest.builder().queueNamePrefix(prefix).build();
 ListQueuesResponse listQueuesResponse =
sqsClient.listQueues(listQueuesRequest);

 for (String url : listQueuesResponse.queueUrls()) {
 System.out.println(url);
 }

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

See the [complete sample](#) on GitHub.

## Get the URL for a queue

The following code shows how to get the URL for a queue by calling the `SqsClient`'s `getQueueUrl` method with a [GetQueueUrlRequest](#) object.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
GetQueueUrlResponse getQueueUrlResponse =

sqsClient.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
String queueUrl = getQueueUrlResponse.queueUrl();
return queueUrl;
```

See the [complete sample](#) on GitHub.

## Delete a queue

Provide the queue's [URL](#) to the [DeleteQueueRequest](#) object. Then call the `SqsClient`'s `deleteQueue` method to delete a queue as shown in the following code.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
public static void deleteSQSQueue(SqsClient sqsClient, String queueName) {

 try {

 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
```

```
 .queueName(queueName)
 .build();

String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();

DeleteQueueRequest deleteQueueRequest = DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();

sqsClient.deleteQueue(deleteQueueRequest);

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

See the [complete sample](#) on GitHub.

## More information

- [CreateQueue](#) in the Amazon Simple Queue Service API Reference
- [GetQueueUrl](#) in the Amazon Simple Queue Service API Reference
- [ListQueues](#) in the Amazon Simple Queue Service API Reference
- [DeleteQueue](#) in the Amazon Simple Queue Service API Reference

## Send, receive, and delete Amazon Simple Queue Service messages

A message is a piece of data that can be sent and received by distributed components. Messages are always delivered using an [SQS Queue](#).

The `sqsClient` variable that is used in the following examples can be created from the following snippet.

```
SqsClient sqsClient = SqsClient.create();
```

When you create an `SqsClient` by using the static `create()` method, the SDK configures the Region by using the [default region provider chain](#) and the credentials by using the [default credentials provider chain](#).

## Send a message

Add a single message to an Amazon Simple Queue Service queue by calling the `SqsClient` client `sendMessage` method. Provide a [SendMessageRequest](#) object that contains the queue's [URL](#), message body, and optional delay value (in seconds).

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
sqsClient.sendMessage(SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody("Hello world!")
 .delaySeconds(10)
 .build());

sqsClient.sendMessage(sendMsgRequest);
```

## Send multiple messages in a request

Send more than one message in a single request by using the `SqsClient` `sendMessageBatch` method. This method takes a [SendMessageBatchRequest](#) that contains the queue URL and a list of messages to send. (Each message is a [SendMessageBatchRequestEntry](#).) You can also delay sending a specific message by setting a delay value on the message.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
SendMessageBatchRequest sendMessageBatchRequest =
 SendMessageBatchRequest.builder()
```

```
 .queueUrl(queueUrl)

 .entries(SendMessageBatchRequestEntry.builder().id("id1").messageBody("Hello from msg
1").build(),

SendMessageBatchRequestEntry.builder().id("id2").messageBody("msg
2").delaySeconds(10).build())
 .build();
sqsClient.sendMessageBatch(sendMessageBatchRequest);
```

See the [complete sample](#) on GitHub.

## Retrieve Messages

Retrieve any messages that are currently in the queue by calling the `SqsClient receiveMessage` method. This method takes a [ReceiveMessageRequest](#) that contains the queue URL. You can also specify the maximum number of messages to return. Messages are returned as a list of [Message](#) objects.

## Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

## Code

```
try {
 ReceiveMessageRequest receiveMessageRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .maxNumberOfMessages(5)
 .build();
 List<Message> messages =
sqsClient.receiveMessage(receiveMessageRequest).messages();
 return messages;
} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

```
return null;
```

See the [complete sample](#) on GitHub.

## Delete a message after receipt

After receiving a message and processing its contents, delete the message from the queue by sending the message's receipt handle and queue URL to the `SqsClient`'s [deleteMessage](#) method.

### Imports

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.*;
import java.util.List;
```

### Code

```
try {
 for (Message message : messages) {
 DeleteMessageRequest deleteMessageRequest =
 DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(message.receiptHandle())
 .build();
 sqsClient.deleteMessage(deleteMessageRequest);
 }
}
```

See the [complete sample](#) on GitHub.

## More Info

- [How Amazon Simple Queue Service Queues Work](#) in the Amazon Simple Queue Service Developer Guide
- [SendMessage](#) in the Amazon Simple Queue Service API Reference
- [SendMessageBatch](#) in the Amazon Simple Queue Service API Reference
- [ReceiveMessage](#) in the Amazon Simple Queue Service API Reference
- [DeleteMessage](#) in the Amazon Simple Queue Service API Reference

# Work with Amazon Transcribe

The following example shows how bidirectional streaming works using Amazon Transcribe. Bidirectional streaming implies that there's both a stream of data going to the service and being received back in real time. The example uses Amazon Transcribe streaming transcription to send an audio stream and receive a stream of transcribed text back in real time.

See [Streaming Transcription](#) in the Amazon Transcribe Developer Guide to learn more about this feature.

See [Getting Started](#) in the Amazon Transcribe Developer Guide to get started using Amazon Transcribe.

## Set up the microphone

This code uses the `javax.sound.sampled` package to stream audio from an input device.

### Code

```
import javax.sound.sampled.AudioFormat;
import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.DataLine;
import javax.sound.sampled.TargetDataLine;

public class Microphone {

 public static TargetDataLine get() throws Exception {
 AudioFormat format = new AudioFormat(16000, 16, 1, true, false);
 DataLine.Info datalineInfo = new DataLine.Info(TargetDataLine.class, format);

 TargetDataLine dataLine = (TargetDataLine) AudioSystem.getLine(datalineInfo);
 dataLine.open(format);

 return dataLine;
 }
}
```

See the [complete example](#) on GitHub.



## Create a publisher

This code implements a publisher that publishes audio data from the Amazon Transcribe audio stream.

### Code

```
package com.amazonaws.transcribe;

import java.io.IOException;
import java.io.InputStream;
import java.io.UncheckedIOException;
import java.nio.ByteBuffer;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicLong;
import org.reactivestreams.Publisher;
import org.reactivestreams.Subscriber;
import org.reactivestreams.Subscription;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.services.transcribestreaming.model.AudioEvent;
import software.amazon.awssdk.services.transcribestreaming.model.AudioStream;
import
 software.amazon.awssdk.services.transcribestreaming.model.TranscribeStreamingException;

public class AudioStreamPublisher implements Publisher<AudioStream> {
 private final InputStream inputStream;

 public AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {
 s.onSubscribe(new SubscriptionImpl(s, inputStream));
 }

 private class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024 * 1;
 private ExecutorService executor = Executors.newFixedThreadPool(1);
 private AtomicLong demand = new AtomicLong(0);
 }
}
```

```

 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;

 private SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream
inputStream) {
 this.subscriber = s;
 this.inputStream = inputStream;
 }

 @Override
 public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent = audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (TranscribeStreamingException e) {
 subscriber.onError(e);
 }
 });
 }

 @Override
 public void cancel() {

 }

 private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

```

```
int len = 0;
try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
} catch (IOException e) {
 throw new UncheckedIOException(e);
}

return audioBuffer;
}

private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
}
}
```

See the [complete example](#) on GitHub.

## Create the client and start the stream

In the main method, create a request object, start the audio input stream and instantiate the publisher with the audio input.

You must also create a [StartStreamTranscriptionResponseHandler](#) to specify how to handle the response from Amazon Transcribe.

Then, use the `TranscribeStreamingAsyncClient`'s `startStreamTranscription` method to start the bidirectional streaming.

### Imports

```
import javax.sound.sampled.AudioFormat;
import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.DataLine;
```

```

import javax.sound.sampled.TargetDataLine;
import javax.sound.sampled.AudioInputStream;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.transcribestreaming.TranscribeStreamingAsyncClient;
import
 software.amazon.awssdk.services.transcribestreaming.model.TranscribeStreamingException ;
import
 software.amazon.awssdk.services.transcribestreaming.model.StartStreamTranscriptionRequest;
import software.amazon.awssdk.services.transcribestreaming.model.MediaEncoding;
import software.amazon.awssdk.services.transcribestreaming.model.LanguageCode;
import
 software.amazon.awssdk.services.transcribestreaming.model.StartStreamTranscriptionResponseHandler;
import software.amazon.awssdk.services.transcribestreaming.model.TranscriptEvent;

```

## Code

```

 public static void convertAudio(TranscribeStreamingAsyncClient client) throws
 Exception {

 try {

 StartStreamTranscriptionRequest request =
 StartStreamTranscriptionRequest.builder()
 .mediaEncoding(MediaEncoding.PCM)
 .languageCode(LanguageCode.EN_US)
 .mediaSampleRateHertz(16_000).build();

 TargetDataLine mic = Microphone.get();
 mic.start();

 AudioStreamPublisher publisher = new AudioStreamPublisher(new
 AudioInputStream(mic));

 StartStreamTranscriptionResponseHandler response =
 StartStreamTranscriptionResponseHandler.builder().subscriber(e -> {
 TranscriptEvent event = (TranscriptEvent) e;
 event.transcript().results().forEach(r ->
 r.alternatives().forEach(a -> System.out.println(a.transcript())));
 }).build();

 // Keeps Streaming until you end the Java program
 client.startStreamTranscription(request, publisher, response);

```

```
 } catch (TranscribeStreamingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

See the [complete example](#) on GitHub.

## More information

- [How It Works](#) in the Amazon Transcribe Developer Guide.
- [Getting Started With Streaming Audio](#) in the Amazon Transcribe Developer Guide.

# SDK for Java 2.x code examples

The code examples in this topic show you how to use the AWS SDK for Java 2.x with AWS.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Some services contain additional example categories that show how to leverage libraries or functions specific to the service.

## Services

- [ACM examples using SDK for Java 2.x](#)
- [API Gateway examples using SDK for Java 2.x](#)
- [Application Auto Scaling examples using SDK for Java 2.x](#)
- [Application Recovery Controller examples using SDK for Java 2.x](#)
- [Aurora examples using SDK for Java 2.x](#)
- [Auto Scaling examples using SDK for Java 2.x](#)
- [AWS Batch examples using SDK for Java 2.x](#)
- [Amazon Bedrock examples using SDK for Java 2.x](#)
- [Amazon Bedrock Runtime examples using SDK for Java 2.x](#)
- [CloudFront examples using SDK for Java 2.x](#)
- [CloudWatch examples using SDK for Java 2.x](#)
- [CloudWatch Events examples using SDK for Java 2.x](#)
- [CloudWatch Logs examples using SDK for Java 2.x](#)
- [Amazon Cognito Identity examples using SDK for Java 2.x](#)
- [Amazon Cognito Identity Provider examples using SDK for Java 2.x](#)
- [Amazon Comprehend examples using SDK for Java 2.x](#)
- [AWS Control Tower examples using SDK for Java 2.x](#)
- [Firehose examples using SDK for Java 2.x](#)

- [Amazon DocumentDB examples using SDK for Java 2.x](#)
- [DynamoDB examples using SDK for Java 2.x](#)
- [Amazon EC2 examples using SDK for Java 2.x](#)
- [Amazon ECR examples using SDK for Java 2.x](#)
- [Amazon ECS examples using SDK for Java 2.x](#)
- [Elastic Load Balancing - Version 2 examples using SDK for Java 2.x](#)
- [MediaStore examples using SDK for Java 2.x](#)
- [AWS Entity Resolution examples using SDK for Java 2.x](#)
- [OpenSearch Service examples using SDK for Java 2.x](#)
- [EventBridge examples using SDK for Java 2.x](#)
- [EventBridge Scheduler examples using SDK for Java 2.x](#)
- [Forecast examples using SDK for Java 2.x](#)
- [Amazon Glacier examples using SDK for Java 2.x](#)
- [AWS Glue examples using SDK for Java 2.x](#)
- [HealthImaging examples using SDK for Java 2.x](#)
- [IAM examples using SDK for Java 2.x](#)
- [Amazon Inspector examples using SDK for Java 2.x](#)
- [AWS IoT examples using SDK for Java 2.x](#)
- [AWS IoT data examples using SDK for Java 2.x](#)
- [AWS IoT SiteWise examples using SDK for Java 2.x](#)
- [Amazon Keyspaces examples using SDK for Java 2.x](#)
- [Kinesis examples using SDK for Java 2.x](#)
- [AWS KMS examples using SDK for Java 2.x](#)
- [Lambda examples using SDK for Java 2.x](#)
- [Amazon Lex examples using SDK for Java 2.x](#)
- [Amazon Location examples using SDK for Java 2.x](#)
- [Location Service Places examples using SDK for Java 2.x](#)
- [AWS Marketplace Catalog API examples using SDK for Java 2.x](#)
- [AWS Marketplace Agreement API examples using SDK for Java 2.x](#)
- [MediaConvert examples using SDK for Java 2.x](#)

- [Migration Hub examples using SDK for Java 2.x](#)
- [Amazon MSK examples using SDK for Java 2.x](#)
- [Neptune examples using SDK for Java 2.x](#)
- [Partner Central examples using SDK for Java 2.x](#)
- [Amazon Personalize examples using SDK for Java 2.x](#)
- [Amazon Personalize Events examples using SDK for Java 2.x](#)
- [Amazon Personalize Runtime examples using SDK for Java 2.x](#)
- [Amazon Pinpoint examples using SDK for Java 2.x](#)
- [Amazon Pinpoint SMS and Voice API examples using SDK for Java 2.x](#)
- [Amazon Polly examples using SDK for Java 2.x](#)
- [Amazon RDS examples using SDK for Java 2.x](#)
- [Amazon RDS Data Service examples using SDK for Java 2.x](#)
- [Amazon Redshift examples using SDK for Java 2.x](#)
- [Amazon Rekognition examples using SDK for Java 2.x](#)
- [Route 53 domain registration examples using SDK for Java 2.x](#)
- [Amazon S3 examples using SDK for Java 2.x](#)
- [Amazon S3 Control examples using SDK for Java 2.x](#)
- [S3 Directory Buckets examples using SDK for Java 2.x](#)
- [SageMaker AI examples using SDK for Java 2.x](#)
- [Secrets Manager examples using SDK for Java 2.x](#)
- [Amazon SES examples using SDK for Java 2.x](#)
- [Amazon SES API v2 examples using SDK for Java 2.x](#)
- [Amazon SNS examples using SDK for Java 2.x](#)
- [Amazon SQS examples using SDK for Java 2.x](#)
- [Step Functions examples using SDK for Java 2.x](#)
- [AWS STS examples using SDK for Java 2.x](#)
- [Support examples using SDK for Java 2.x](#)
- [Systems Manager examples using SDK for Java 2.x](#)
- [Amazon Textract examples using SDK for Java 2.x](#)
- [Amazon Transcribe examples using SDK for Java 2.x](#)



- [Amazon Transcribe Streaming examples using SDK for Java 2.x](#)
- [Amazon Translate examples using SDK for Java 2.x](#)
- [X-Ray examples using SDK for Java 2.x](#)

## ACM examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with ACM.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### AddTagsToCertificate

The following code example shows how to use AddTagsToCertificate.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
```

```

* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class AddTagsToCertificate {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 "";
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 addTags(certArn);
 }

 /**
 * Adds tags to a certificate in AWS Certificate Manager (ACM).
 *
 * @param certArn the Amazon Resource Name (ARN) of the certificate to add tags
 to
 */
 public static void addTags(String certArn) {
 AcmClient acmClient = AcmClient.create();
 List<Tag> expectedTags =
List.of(Tag.builder().key("key").value("value").build());
 AddTagsToCertificateRequest addTagsToCertificateRequest =
AddTagsToCertificateRequest.builder()
 .certificateArn(certArn)
 .tags(expectedTags)
 .build();

 try {
 acmClient.addTagsToCertificate(addTagsToCertificateRequest);
 System.out.println("Successfully added tags to a certificate");
 } catch (AcmException e) {
 System.out.println(e.getMessage());
 }
 }
}

```

```
}
```

- For API details, see [AddTagsToCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCertificate

The following code example shows how to use DeleteCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteCert {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 "";
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 deleteCertificate(certArn);
 }
}
```

```

 }

 /**
 * Deletes an SSL/TLS certificate from the AWS Certificate Manager (ACM).
 *
 * @param certArn the Amazon Resource Name (ARN) of the certificate to be
 deleted
 */
 public static void deleteCertificate(String certArn) {
 AcmClient acmClient = AcmClient.create();
 DeleteCertificateRequest request = DeleteCertificateRequest.builder()
 .certificateArn(certArn)
 .build();

 try {
 acmClient.deleteCertificate(request);
 System.out.println("The certificate was deleted");

 } catch (AcmException e) {
 System.out.println(e.getMessage());
 }
 }
}

```

- For API details, see [DeleteCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeCertificate

The following code example shows how to use DescribeCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Before running this Java V2 code example, set up your development

```

```

* environment, including your credentials.
* <p>
* For more information, see the following documentation topic:
* <p>
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/

```

```

public class DescribeCert {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 "";
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 describeCertificate(certArn);
 }

 /**
 * Describes the details of an SSL/TLS certificate.
 *
 * @param certArn the Amazon Resource Name (ARN) of the certificate to describe
 * @throws AcmException if an error occurs while describing the certificate
 */
 public static void describeCertificate(String certArn) {
 AcmClient acmClient = AcmClient.create();
 DescribeCertificateRequest req = DescribeCertificateRequest.builder()
 .certificateArn(certArn)
 .build();

 try {
 DescribeCertificateResponse response =
acmClient.describeCertificate(req);

 // Print the certificate details.

```

```

 System.out.println("Certificate ARN: " +
response.certificate().certificateArn());
 System.out.println("Domain Name: " +
response.certificate().domainName());
 System.out.println("Issued By: " + response.certificate().issuer());
 System.out.println("Issued On: " + response.certificate().issuedAt());
 System.out.println("Status: " + response.certificate().status());
 } catch (AcmException e) {
 System.out.println(e.getMessage());
 }
}
}
}

```

- For API details, see [DescribeCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## ExportCertificate

The following code example shows how to use ExportCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ExportCertificate {

 public static void main(String[] args) throws Exception {
 final String usage = ""

```

```

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 """;
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 exportCert(certArn);
}

/**
 * Exports an SSL/TLS certificate and its associated private key and certificate
 * chain from AWS Certificate Manager (ACM).
 *
 * @param certArn The Amazon Resource Name (ARN) of the certificate that you
 * want to export.
 * @throws IOException If an I/O error occurs while reading the private key
 * passphrase file or exporting the certificate.
 */
public static void exportCert(String certArn) throws IOException {
 AcmClient acmClient = AcmClient.create();

 // Initialize a file descriptor for the passphrase file.
 RandomAccessFile filePassphrase = null;
 ByteBuffer bufPassphrase = null;

 // Create a file stream for reading the private key passphrase.
 try {
 filePassphrase = new RandomAccessFile("C:\\AWS\\password.txt", "r");
 } catch (IllegalArgumentException | SecurityException |
FileNotFoundException ex) {
 throw ex;
 }

 // Create a channel to map the file.
 FileChannel channelPassphrase = filePassphrase.getChannel();

 // Map the file to the buffer.
 try {

```

```

 bufPassphrase = channelPassphrase.map(FileChannel.MapMode.READ_ONLY, 0,
channelPassphrase.size());
 channelPassphrase.close();
 filePassphrase.close();
 } catch (IOException ex) {
 throw ex;
 }

 // Create a request object.
 ExportCertificateRequest req = ExportCertificateRequest.builder()
 .certificateArn(certArn)
 .passphrase(SdkBytes.fromByteBuffer(bufPassphrase))
 .build();

 // Export the certificate.
 ExportCertificateResponse result = null;
 try {
 result = acmClient.exportCertificate(req);
 } catch (InvalidArnException | InvalidTagException |
ResourceNotFoundException ex) {
 throw ex;
 }

 // Clear the buffer.
 bufPassphrase.clear();

 // Display the certificate and certificate chain.
 String certificate = result.certificate();
 System.out.println(certificate);

 String certificateChain = result.certificateChain();
 System.out.println(certificateChain);

 // This example retrieves but does not display the private key.
 String privateKey = result.privateKey();
 System.out.println("The example is complete");
}
}

```

- For API details, see [ExportCertificate](#) in *AWS SDK for Java 2.x API Reference*.



## ImportCertificate

The following code example shows how to use ImportCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ImportCert {

 public static void main(String[] args) {
 final String usage = ""
 Usage: <bucketName> <certificateKey> <privateKeyKey>

 Where:
 bucketName - The name of the S3 bucket containing the certificate
and private key.
 certificateKey - The object key for the SSL/TLS certificate file in
S3.
 privateKeyKey - The object key for the private key file in S3.
 "";

 if (args.length != 3) {
 System.out.println(usage);
 return;
 }

 String bucketName = args[0];
 String certificateKey = args[1];
 String privateKeyKey = args[2];
```

```

 String certificateArn = importCertificate(bucketName, certificateKey,
privateKeyKey);
 System.out.println("Certificate imported with ARN: " + certificateArn);
 }

 /**
 * Imports an SSL/TLS certificate and private key from S3 into AWS Certificate
 Manager (ACM).
 *
 * @param bucketName The name of the S3 bucket.
 * @param certificateKey The key for the SSL/TLS certificate file in S3.
 * @param privateKeyKey The key for the private key file in S3.
 * @return The ARN of the imported certificate.
 */
 public static String importCertificate(String bucketName, String certificateKey,
String privateKeyKey) {
 AcmClient acmClient = AcmClient.create();
 S3Client s3Client = S3Client.create();

 try {
 byte[] certificateBytes = downloadFileFromS3(s3Client, bucketName,
certificateKey);
 byte[] privateKeyBytes = downloadFileFromS3(s3Client, bucketName,
privateKeyKey);

 ImportCertificateRequest request = ImportCertificateRequest.builder()

 .certificate(SdkBytes.fromByteBuffer(ByteBuffer.wrap(certificateBytes)))

 .privateKey(SdkBytes.fromByteBuffer(ByteBuffer.wrap(privateKeyBytes)))
 .build();

 ImportCertificateResponse response =
acmClient.importCertificate(request);
 return response.certificateArn();

 } catch (IOException e) {
 System.err.println("Error downloading certificate or private key from
S3: " + e.getMessage());
 } catch (S3Exception e) {
 System.err.println("S3 error: " + e.awsErrorDetails().errorMessage());
 }
 return "";
 }
}

```

```

/**
 * Downloads a file from Amazon S3 and returns its contents as a byte array.
 *
 * @param s3Client The S3 client.
 * @param bucketName The name of the S3 bucket.
 * @param objectKey The key of the object in S3.
 * @return The file contents as a byte array.
 * @throws IOException If an I/O error occurs.
 */
private static byte[] downloadFileFromS3(S3Client s3Client, String bucketName,
String objectKey) throws IOException {
 GetObjectRequest getObjectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 try (ResponseInputStream<GetObjectResponse> s3Object =
s3Client.getObject(getObjectRequest);
 ByteArrayOutputStream byteArrayOutputStream = new
ByteArrayOutputStream()) {
 IoUtils.copy(s3Object, byteArrayOutputStream);
 return byteArrayOutputStream.toByteArray();
 }
}
}

```

- For API details, see [ImportCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## ListCertificates

The following code example shows how to use `ListCertificates`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListCerts {
 public static void main(String[] args) {
 listCertificates();
 }

 /**
 * Lists all the certificates managed by AWS Certificate Manager (ACM) that have
 a status of "ISSUED".
 */
 public static void listCertificates() {
 AcmClient acmClient = AcmClient.create();
 try {
 ListCertificatesRequest listRequest = ListCertificatesRequest.builder()
 .certificateStatuses(CertificateStatus.ISSUED)
 .maxItems(100)
 .build();
 ListCertificatesIterable listResponse =
acmClient.listCertificatesPaginator(listRequest);

 // Print the certificate details using streams
 listResponse.certificateSummaryList().stream()
 .forEach(certificate -> {
 System.out.println("Certificate ARN: " +
certificate.certificateArn());
 System.out.println("Certificate Domain Name: " +
certificate.domainName());
 System.out.println("Certificate Status: " +
certificate.statusAsString());
 System.out.println("---");
 });

 } catch (AcmException e) {
 System.err.println(e.getMessage());
 }
 }
}

```

```
}
```

- For API details, see [ListCertificates](#) in *AWS SDK for Java 2.x API Reference*.

## ListTagsForCertificate

The following code example shows how to use `ListTagsForCertificate`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListCertTags {

 public static void main(String[] args) {

 final String usage = ""

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 "";
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }
 }
}
```

```

 String certArn = args[0];
 listCertTags(certArn);
 }

 /**
 * Lists the tags associated with an AWS Certificate Manager (ACM) certificate.
 *
 * @param certArn the Amazon Resource Name (ARN) of the ACM certificate
 */
 public static void listCertTags(String certArn) {
 AcmClient acmClient = AcmClient.create();

 ListTagsForCertificateRequest request =
 ListTagsForCertificateRequest.builder()
 .certificateArn(certArn)
 .build();

 ListTagsForCertificateResponse response =
 acmClient.listTagsForCertificate(request);
 List<Tag> tagList = response.tags();
 tagList.forEach(tag -> {
 System.out.println("Key: " + tag.key());
 System.out.println("Value: " + tag.value());
 });
 }
}

```

- For API details, see [ListTagsForCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## RemoveTagsFromCertificate

The following code example shows how to use `RemoveTagsFromCertificate`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class RemoveTagsFromCert {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <certArn>

 Where:
 certArn - the ARN of the certificate.
 "";
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 removeTags(certArn);
 }

 /**
 * Removes tags from an AWS Certificate Manager (ACM) certificate.
 *
 * @param certArn the Amazon Resource Name (ARN) of the certificate from which
 to remove tags
 */
 public static void removeTags(String certArn) {
 AcmClient acmClient = AcmClient.create();
 List<Tag> expectedTags =
List.of(Tag.builder().key("key").value("value").build());
 RemoveTagsFromCertificateRequest req =
RemoveTagsFromCertificateRequest.builder()
 .certificateArn(certArn)
 .tags(expectedTags)
 }

```

```

 .build();

 try {
 acmClient.removeTagsFromCertificate(req);
 System.out.println("Successfully removed tags from the certificate");
 } catch (AcmException e) {
 System.err.println(e.getMessage());
 }
}
}

```

- For API details, see [RemoveTagsFromCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## RenewCertificate

The following code example shows how to use `RenewCertificate`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * <p>
 * For more information, see the following documentation topic:
 *
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class RenewCert {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <certArn>

```



```

 Where:
 certArn - the ARN of the certificate.
 """;
 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String certArn = args[0];
 renewCertificate(certArn);
}

/**
 * Renews an existing SSL/TLS certificate in AWS Certificate Manager (ACM).
 *
 * @param certArn The Amazon Resource Name (ARN) of the certificate to be
renewed.
 * @throws AcmException If there is an error renewing the certificate.
 */
public static void renewCertificate(String certArn) {
 AcmClient acmClient = AcmClient.create();

 RenewCertificateRequest certificateRequest =
RenewCertificateRequest.builder()
 .certificateArn(certArn)
 .build();

 try {
 acmClient.renewCertificate(certificateRequest);
 System.out.println("The certificate was renewed");
 } catch (AcmException e) {
 System.out.println(e.getMessage());
 }
}
}

```

- For API details, see [RenewCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## RequestCertificate

The following code example shows how to use RequestCertificate.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class RequestCert {

 public static void main(String[] args) {
 requestCertificate();
 }

 /**
 * Requests a certificate from the AWS Certificate Manager (ACM) service.
 */
 public static void requestCertificate() {
 AcmClient acmClient = AcmClient.create();
 ArrayList<String> san = new ArrayList<>();
 san.add("www.example.com");

 RequestCertificateRequest req = RequestCertificateRequest.builder()
 .domainName("example.com")
 .idempotencyToken("1Aq25pTy")
 .subjectAlternativeNames(san)
 .build();

 try {
 RequestCertificateResponse response = acmClient.requestCertificate(req);
 System.out.println("Cert ARN IS " + response.certificateArn());
 } catch (AcmException e) {
 System.err.println(e.getMessage());
 }
 }
}
```

```
}
 }
}
```

- For API details, see [RequestCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## API Gateway examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with API Gateway.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

*AWS community contributions* are examples that were created and are maintained by multiple teams across AWS. To provide feedback, use the mechanism provided in the linked repositories.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)
- [AWS community contributions](#)

## Actions

### CreateDeployment

The following code example shows how to use CreateDeployment.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createNewDeployment(ApiGatewayClient apiGateway, String
restApiId, String stageName) {

 try {
 CreateDeploymentRequest request = CreateDeploymentRequest.builder()
 .restApiId(restApiId)
 .description("Created using the AWS API Gateway Java API")
 .stageName(stageName)
 .build();

 CreateDeploymentResponse response =
apiGateway.createDeployment(request);
 System.out.println("The id of the deployment is " + response.id());
 return response.id();

 } catch (ApiGatewayException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateDeployment](#) in *AWS SDK for Java 2.x API Reference*.

## CreateRestApi

The following code example shows how to use `CreateRestApi`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createAPI(ApiGatewayClient apiGateway, String restApiId,
String restApiName) {

 try {
 CreateRestApiRequest request = CreateRestApiRequest.builder()
 .cloneFrom(restApiId)
 .description("Created using the Gateway Java API")
 .name(restApiName)
 .build();

 CreateRestApiResponse response = apiGateway.createRestApi(request);
 System.out.println("The id of the new api is " + response.id());
 return response.id();

 } catch (ApiGatewayException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateRestApi](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDeployment

The following code example shows how to use DeleteDeployment.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteSpecificDeployment(ApiGatewayClient apiGateway, String
restApiId, String deploymentId) {

 try {
 DeleteDeploymentRequest request = DeleteDeploymentRequest.builder()
 .restApiId(restApiId)
 .deploymentId(deploymentId)
 .build();

 apiGateway.deleteDeployment(request);
 System.out.println("Deployment was deleted");

 } catch (ApiGatewayException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteDeployment](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteRestApi

The following code example shows how to use DeleteRestApi.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteAPI(ApiGatewayClient apiGateway, String restApiId) {

 try {
 DeleteRestApiRequest request = DeleteRestApiRequest.builder()
 .restApiId(restApiId)
 .build();

 apiGateway.deleteRestApi(request);
 System.out.println("The API was successfully deleted");

 } catch (ApiGatewayException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteRestApi](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

#### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

#### Services used in this example

- API Gateway
- DynamoDB
- Lambda

- Amazon Rekognition
- Amazon S3
- Amazon SNS

## Use API Gateway to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by Amazon API Gateway.

### SDK for Java 2.x

Shows how to create an AWS Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create a Lambda function invoked by Amazon API Gateway that scans an Amazon DynamoDB table for work anniversaries and uses Amazon Simple Notification Service (Amazon SNS) to send a text message to your employees that congratulates them at their one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon SNS

## AWS community contributions

### Build and test a serverless application

The following code example shows how to build and test a serverless application using API Gateway with Lambda and DynamoDB

### SDK for Java 2.x

Shows how to build and test a serverless application that consists of an API Gateway with Lambda and DynamoDB using the Java SDK.



For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda

## Application Auto Scaling examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Application Auto Scaling.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### DeleteScalingPolicy

The following code example shows how to use DeleteScalingPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
```

```

import
 software.amazon.awssdk.services.applicationautoscaling.ApplicationAutoScalingClient;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ApplicationAutoScalingException;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DeleteScalingPolicyRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DeregisterScalableTargetRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalableTargetsRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalableTargetsResponse;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalingPoliciesRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalingPoliciesResponse;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ScalableDimension;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ServiceNamespace;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class DisableDynamoDBAutoscaling {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableId> <policyName>\s

 Where:
 tableId - The table Id value (for example, table/Music).\s
 policyName - The name of the policy (for example, $Music5-scaling-
policy).

 """;
 if (args.length != 2) {

```

```

 System.out.println(usage);
 System.exit(1);
 }

 ApplicationAutoScalingClient appAutoScalingClient =
ApplicationAutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 ServiceNamespace ns = ServiceNamespace.DYNAMODB;
 ScalableDimension tableWCUs =
ScalableDimension.DYNAMODB_TABLE_WRITE_CAPACITY_UNITS;
 String tableId = args[0];
 String policyName = args[1];

 deletePolicy(appAutoScalingClient, policyName, tableWCUs, ns, tableId);
 verifyScalingPolicies(appAutoScalingClient, tableId, ns, tableWCUs);
 deregisterScalableTarget(appAutoScalingClient, tableId, ns, tableWCUs);
 verifyTarget(appAutoScalingClient, tableId, ns, tableWCUs);
}

public static void deletePolicy(ApplicationAutoScalingClient
appAutoScalingClient, String policyName, ScalableDimension tableWCUs,
ServiceNamespace ns, String tableId) {
 try {
 DeleteScalingPolicyRequest delSPRequest =
DeleteScalingPolicyRequest.builder()
 .policyName(policyName)
 .scalableDimension(tableWCUs)
 .serviceNamespace(ns)
 .resourceId(tableId)
 .build();

 appAutoScalingClient.deleteScalingPolicy(delSPRequest);
 System.out.println(policyName + " was deleted successfully.");

 } catch (ApplicationAutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
}

// Verify that the scaling policy was deleted

```

```

 public static void verifyScalingPolicies(ApplicationAutoScalingClient
appAutoScalingClient, String tableId, ServiceNamespace ns, ScalableDimension
tableWCUs) {
 DescribeScalingPoliciesRequest dscRequest =
DescribeScalingPoliciesRequest.builder()
 .scalableDimension(tableWCUs)
 .serviceNamespace(ns)
 .resourceId(tableId)
 .build();

 DescribeScalingPoliciesResponse response =
appAutoScalingClient.describeScalingPolicies(dscRequest);
 System.out.println("DescribeScalableTargets result: ");
 System.out.println(response);
 }

 public static void deregisterScalableTarget(ApplicationAutoScalingClient
appAutoScalingClient, String tableId, ServiceNamespace ns, ScalableDimension
tableWCUs) {
 try {
 DeregisterScalableTargetRequest targetRequest =
DeregisterScalableTargetRequest.builder()
 .scalableDimension(tableWCUs)
 .serviceNamespace(ns)
 .resourceId(tableId)
 .build();

 appAutoScalingClient.deregisterScalableTarget(targetRequest);
 System.out.println("The scalable target was deregistered.");

 } catch (ApplicationAutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 }

 public static void verifyTarget(ApplicationAutoScalingClient
appAutoScalingClient, String tableId, ServiceNamespace ns, ScalableDimension
tableWCUs) {
 DescribeScalableTargetsRequest dscRequest =
DescribeScalableTargetsRequest.builder()
 .scalableDimension(tableWCUs)
 .serviceNamespace(ns)
 .resourceIds(tableId)
 .build();

```

```

 DescribeScalableTargetsResponse response =
appAutoScalingClient.describeScalableTargets(dscRequest);
 System.out.println("DescribeScalableTargets result: ");
 System.out.println(response);
 }
}

```

- For API details, see [DeleteScalingPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## RegisterScalableTarget

The following code example shows how to use RegisterScalableTarget.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.applicationautoscaling.ApplicationAutoScalingClient;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ApplicationAutoScalingException;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalableTargetsRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalableTargetsResponse;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalingPoliciesRequest;
import
 software.amazon.awssdk.services.applicationautoscaling.model.DescribeScalingPoliciesResponse;
import software.amazon.awssdk.services.applicationautoscaling.model.PolicyType;
import
 software.amazon.awssdk.services.applicationautoscaling.model.PredefinedMetricSpecification;
import
 software.amazon.awssdk.services.applicationautoscaling.model.PutScalingPolicyRequest;

```

```

import
 software.amazon.awssdk.services.applicationautoscaling.model.RegisterScalableTargetRequest;
import software.amazon.awssdk.services.applicationautoscaling.model.ScalingPolicy;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ServiceNamespace;
import
 software.amazon.awssdk.services.applicationautoscaling.model.ScalableDimension;
import software.amazon.awssdk.services.applicationautoscaling.model.MetricType;
import
 software.amazon.awssdk.services.applicationautoscaling.model.TargetTrackingScalingPolicyCon
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class EnableDynamoDBAutoscaling {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableId> <roleARN> <policyName>\s

 Where:
 tableId - The table Id value (for example, table/Music).
 roleARN - The ARN of the role that has ApplicationAutoScaling
permissions.
 policyName - The name of the policy to create.

 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 System.out.println("This example registers an Amazon DynamoDB table, which
is the resource to scale.");
 String tableId = args[0];
 String roleARN = args[1];

```

```

 String policyName = args[2];
 ServiceNamespace ns = ServiceNamespace.DYNAMODB;
 ScalableDimension tableWCUs =
 ScalableDimension.DYNAMODB_TABLE_WRITE_CAPACITY_UNITS;
 ApplicationAutoScalingClient appAutoScalingClient =
 ApplicationAutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 registerScalableTarget(appAutoScalingClient, tableId, roleARN, ns,
 tableWCUs);
 verifyTarget(appAutoScalingClient, tableId, ns, tableWCUs);
 configureScalingPolicy(appAutoScalingClient, tableId, ns, tableWCUs,
 policyName);
 }

```

```

 public static void registerScalableTarget(ApplicationAutoScalingClient
 appAutoScalingClient, String tableId, String roleARN, ServiceNamespace ns,
 ScalableDimension tableWCUs) {

```

```

 try {
 RegisterScalableTargetRequest targetRequest =
 RegisterScalableTargetRequest.builder()
 .serviceNamespace(ns)
 .scalableDimension(tableWCUs)
 .resourceId(tableId)
 .roleARN(roleARN)
 .minCapacity(5)
 .maxCapacity(10)
 .build();

 appAutoScalingClient.registerScalableTarget(targetRequest);
 System.out.println("You have registered " + tableId);

```

```

 } catch (ApplicationAutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 }

```

```

 // Verify that the target was created.

```

```

 public static void verifyTarget(ApplicationAutoScalingClient
 appAutoScalingClient, String tableId, ServiceNamespace ns, ScalableDimension
 tableWCUs) {
 DescribeScalableTargetsRequest dscRequest =
 DescribeScalableTargetsRequest.builder()

```

```

 .scalableDimension(tableWCUs)
 .serviceNamespace(ns)
 .resourceIds(tableId)
 .build();

 DescribeScalableTargetsResponse response =
appAutoScalingClient.describeScalableTargets(dscRequest);
 System.out.println("DescribeScalableTargets result: ");
 System.out.println(response);
}

// Configure a scaling policy.
public static void configureScalingPolicy(ApplicationAutoScalingClient
appAutoScalingClient, String tableId, ServiceNamespace ns, ScalableDimension
tableWCUs, String policyName) {
 // Check if the policy exists before creating a new one.
 DescribeScalingPoliciesResponse describeScalingPoliciesResponse =
appAutoScalingClient.describeScalingPolicies(DescribeScalingPoliciesRequest.builder()
 .serviceNamespace(ns)
 .resourceId(tableId)
 .scalableDimension(tableWCUs)
 .build());

 if (!describeScalingPoliciesResponse.scalingPolicies().isEmpty()) {
 // If policies exist, consider updating an existing policy instead of
creating a new one.
 System.out.println("Policy already exists. Consider updating it
instead.");
 List<ScalingPolicy> polList =
describeScalingPoliciesResponse.scalingPolicies();
 for (ScalingPolicy pol : polList) {
 System.out.println("Policy name:" +pol.policyName());
 }
 } else {
 // If no policies exist, proceed with creating a new policy.
 PredefinedMetricSpecification specification =
PredefinedMetricSpecification.builder()

 .predefinedMetricType(MetricType.DYNAMO_DB_WRITE_CAPACITY_UTILIZATION)
 .build();

 TargetTrackingScalingPolicyConfiguration policyConfiguration =
TargetTrackingScalingPolicyConfiguration.builder()
 .predefinedMetricSpecification(specification)

```



```

 .targetValue(50.0)
 .scaleInCooldown(60)
 .scaleOutCooldown(60)
 .build();

 PutScalingPolicyRequest putScalingPolicyRequest =
 PutScalingPolicyRequest.builder()
 .targetTrackingScalingPolicyConfiguration(policyConfiguration)
 .serviceNamespace(ns)
 .scalableDimension(tableWCUs)
 .resourceId(tableId)
 .policyName(policyName)
 .policyType(PolicyType.TARGET_TRACKING_SCALING)
 .build();

 try {
 appAutoScalingClient.putScalingPolicy(putScalingPolicyRequest);
 System.out.println("You have successfully created a scaling policy
for an Application Auto Scaling scalable target");
 } catch (ApplicationAutoScalingException e) {
 System.err.println("Error: " + e.awsErrorDetails().errorMessage());
 }
}
}
}
}

```

- For API details, see [RegisterScalableTarget](#) in *AWS SDK for Java 2.x API Reference*.

## Application Recovery Controller examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Application Recovery Controller.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### GetRoutingControlState

The following code example shows how to use `GetRoutingControlState`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static GetRoutingControlStateResponse
getRoutingControlState(List<ClusterEndpoint> clusterEndpoints,
 String routingControlArn) {
 // As a best practice, we recommend choosing a random cluster endpoint to
 get or
 // set routing control states.
 // For more information, see
 // https://docs.aws.amazon.com/r53recovery/latest/dg/route53-arc-best-
 practices.html#route53-arc-best-practices.regional
 Collections.shuffle(clusterEndpoints);
 for (ClusterEndpoint clusterEndpoint : clusterEndpoints) {
 try {
 System.out.println(clusterEndpoint);
 Route53RecoveryClusterClient client =
Route53RecoveryClusterClient.builder()
 .endpointOverride(URI.create(clusterEndpoint.endpoint()))
 .region(Region.of(clusterEndpoint.region())).build();
 return client.getRoutingControlState(
 GetRoutingControlStateRequest.builder()
 .routingControlArn(routingControlArn).build());
 } catch (Exception exception) {
 System.out.println(exception);
 }
 }
}
```

```
 return null;
 }
}
```

- For API details, see [GetRoutingControlState](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateRoutingControlState

The following code example shows how to use `UpdateRoutingControlState`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static UpdateRoutingControlStateResponse
updateRoutingControlState(List<ClusterEndpoint> clusterEndpoints,
 String routingControlArn,
 String routingControlState) {
 // As a best practice, we recommend choosing a random cluster endpoint to
 get or
 // set routing control states.
 // For more information, see
 // https://docs.aws.amazon.com/r53recovery/latest/dg/route53-arc-best-
 practices.html#route53-arc-best-practices.regional
 Collections.shuffle(clusterEndpoints);
 for (ClusterEndpoint clusterEndpoint : clusterEndpoints) {
 try {
 System.out.println(clusterEndpoint);
 Route53RecoveryClusterClient client =
 Route53RecoveryClusterClient.builder()
 .endpointOverride(URI.create(clusterEndpoint.endpoint()))
 .region(Region.of(clusterEndpoint.region()))
 .build();
 return client.updateRoutingControlState(
 UpdateRoutingControlStateRequest.builder()
 .routingControlArn(routingControlArn).routingControlState(routingControlState).build());
 }
 }
}
```

```
 } catch (Exception exception) {
 System.out.println(exception);
 }
 }
 return null;
}
```

- For API details, see [UpdateRoutingControlState](#) in *AWS SDK for Java 2.x API Reference*.

## Aurora examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Aurora.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Aurora

The following code example shows how to get started using Aurora.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.paginators.DescribeDBClustersIterable;

public class DescribeDbClusters {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 describeClusters(rdsClient);
 rdsClient.close();
 }

 public static void describeClusters(RdsClient rdsClient) {
 DescribeDBClustersIterable clustersIterable =
rdsClient.describeDBClustersPaginator();
 clustersIterable.stream()
 .flatMap(r -> r.dbClusters().stream())
 .forEach(cluster -> System.out
 .println("Database name: " + cluster.databaseName() + " Arn
= " + cluster.dbClusterArn()));
 }
}
```

- For API details, see [DescribeDBClusters](#) in *AWS SDK for Java 2.x API Reference*.

# Basics

## Learn the basics

The following code example shows how to:

- Create a custom Aurora DB cluster parameter group and set parameter values.
- Create a DB cluster that uses the parameter group.
- Create a DB instance that contains a database.
- Take a snapshot of the DB cluster, then clean up resources.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This example requires an AWS Secrets Manager secret that contains the
 * database credentials. If you do not create a
 * secret, this example will not work. For details, see:
 *
 * https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating-how-
 * services-use-secrets_RS.html
 *
 * This Java example performs the following tasks:
 *
 * 1. Gets available engine families for Amazon Aurora MySQL-Compatible Edition
 * by calling the DescribeDbEngineVersions(Engine='aurora-mysql') method.
 * 2. Selects an engine family and creates a custom DB cluster parameter group
 * by invoking the describeDBClusterParameters method.
```

```

* 3. Gets the parameter groups by invoking the describeDBClusterParameterGroups
* method.
* 4. Gets parameters in the group by invoking the describeDBClusterParameters
* method.
* 5. Modifies the auto_increment_offset parameter by invoking the
* modifyDbClusterParameterGroupRequest method.
* 6. Gets and displays the updated parameters.
* 7. Gets a list of allowed engine versions by invoking the
* describeDbEngineVersions method.
* 8. Creates an Aurora DB cluster database cluster that contains a MySQL
* database.
* 9. Waits for DB instance to be ready.
* 10. Gets a list of instance classes available for the selected engine.
* 11. Creates a database instance in the cluster.
* 12. Waits for DB instance to be ready.
* 13. Creates a snapshot.
* 14. Waits for DB snapshot to be ready.
* 15. Deletes the DB cluster.
* 16. Deletes the DB cluster group.
*/
public class AuroraScenario {
 public static long sleepTime = 20;
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws InterruptedException {
 final String usage = "\n" +
 "Usage:\n" +
 " <dbClusterGroupName> <dbParameterGroupFamily>
<dbInstanceClusterIdentifier> <dbInstanceIdIdentifier> <dbName>
<dbSnapshotIdentifier><secretName>"
 +
 "Where:\n" +
 " dbClusterGroupName - The name of the DB cluster parameter
group. \n" +
 " dbParameterGroupFamily - The DB cluster parameter group family
name (for example, aurora-mysql5.7). \n"
 +
 " dbInstanceClusterIdentifier - The instance cluster identifier
value.\n" +
 " dbInstanceIdIdentifier - The database instance identifier.\n" +
 " dbName - The database name.\n" +
 " dbSnapshotIdentifier - The snapshot identifier.\n" +
 " secretName - The name of the AWS Secrets Manager secret that
contains the database credentials\n\n";
 }
}

```

```
;

if (args.length != 7) {
 System.out.println(usage);
 System.exit(1);
}

String dbClusterGroupName = args[0];
String dbParameterGroupFamily = args[1];
String dbInstanceClusterIdentifier = args[2];
String dbInstanceIdentifier = args[3];
String dbName = args[4];
String dbSnapshotIdentifier = args[5];
String secretName = args[6];

// Retrieve the database credentials using AWS Secrets Manager.
Gson gson = new Gson();
User user = gson.fromJson(String.valueOf(getSecretValues(secretName)),
User.class);
String username = user.getUsername();
String userPassword = user.getPassword();

Region region = Region.US_WEST_2;
RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

System.out.println(DASHES);
System.out.println("Welcome to the Amazon Aurora example scenario.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("1. Return a list of the available DB engines");
describeDBEngines(rdsClient);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("2. Create a custom parameter group");
createDBClusterParameterGroup(rdsClient, dbClusterGroupName,
dbParameterGroupFamily);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Get the parameter group");
```



```
describeDbClusterParameterGroups(rdsClient, dbClusterGroupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Get the parameters in the group");
describeDbClusterParameters(rdsClient, dbClusterGroupName, 0);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Modify the auto_increment_offset parameter");
modifyDBClusterParas(rdsClient, dbClusterGroupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Display the updated parameter value");
describeDbClusterParameters(rdsClient, dbClusterGroupName, -1);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Get a list of allowed engine versions");
getAllowedEngines(rdsClient, dbParameterGroupFamily);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("8. Create an Aurora DB cluster database");
String arnClusterVal = createDBCluster(rdsClient, dbClusterGroupName,
dbName, dbInstanceClusterIdentifier,
 username, userPassword);
System.out.println("The ARN of the cluster is " + arnClusterVal);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("9. Wait for DB instance to be ready");
waitForInstanceReady(rdsClient, dbInstanceClusterIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("10. Get a list of instance classes available for the
selected engine");
String instanceClass = getListInstanceClasses(rdsClient);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("11. Create a database instance in the cluster.");
```

```
String clusterDBARN = createDBInstanceCluster(rdsClient,
dbInstanceIdentifier, dbInstanceClusterIdentifier,
 instanceClass);
System.out.println("The ARN of the database is " + clusterDBARN);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("12. Wait for DB instance to be ready");
waitDBInstanceReady(rdsClient, dbInstanceIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("13. Create a snapshot");
createDBClusterSnapshot(rdsClient, dbInstanceClusterIdentifier,
dbSnapshotIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("14. Wait for DB snapshot to be ready");
waitForSnapshotReady(rdsClient, dbSnapshotIdentifier,
dbInstanceClusterIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("14. Delete the DB instance");
deleteDatabaseInstance(rdsClient, dbInstanceIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("14. Delete the DB instance");
deleteDatabaseInstance(rdsClient, dbInstanceIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("15. Delete the DB cluster");
deleteCluster(rdsClient, dbInstanceClusterIdentifier);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("16. Delete the DB cluster group");
deleteDBClusterGroup(rdsClient, dbClusterGroupName, clusterDBARN);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("The Scenario has successfully completed.");
System.out.println(DASHES);
rdsClient.close();
}
```

```

private static SecretsManagerClient getSecretClient() {
 Region region = Region.US_WEST_2;
 return SecretsManagerClient.builder()
 .region(region)

 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();
}

private static String getSecretValues(String secretName) {
 SecretsManagerClient secretClient = getSecretClient();
 GetSecretValueRequest valueRequest = GetSecretValueRequest.builder()
 .secretId(secretName)
 .build();

 GetSecretValueResponse valueResponse =
secretClient.getSecretValue(valueRequest);
 return valueResponse.secretString();
}

public static void deleteDBClusterGroup(RdsClient rdsClient, String
dbClusterGroupName, String clusterDBARN)
 throws InterruptedException {
 try {
 boolean isDataDel = false;
 boolean didFind;
 String instanceARN;

 // Make sure that the database has been deleted.
 while (!isDataDel) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 int listSize = instanceList.size();
 didFind = false;
 int index = 1;
 for (DBInstance instance : instanceList) {
 instanceARN = instance.dbInstanceArn();
 if (instanceARN.compareTo(clusterDBARN) == 0) {
 System.out.println(clusterDBARN + " still exists");
 didFind = true;
 }
 }
 if ((index == listSize) && (!didFind)) {

```

```

 // Went through the entire list and did not find the
 database ARN.
 isDataDel = true;
 }
 Thread.sleep(sleepTime * 1000);
 index++;
}

DeleteDbClusterParameterGroupRequest clusterParameterGroupRequest =
DeleteDbClusterParameterGroupRequest
 .builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .build();

rdsClient.deleteDBClusterParameterGroup(clusterParameterGroupRequest);
System.out.println(dbClusterGroupName + " was deleted.");

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}

}

public static void deleteCluster(RdsClient rdsClient, String
dbInstanceClusterIdentifier) {
 try {
 DeleteDbClusterRequest deleteDbClusterRequest =
DeleteDbClusterRequest.builder()
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .skipFinalSnapshot(true)
 .build();

 rdsClient.deleteDBCluster(deleteDbClusterRequest);
 System.out.println(dbInstanceClusterIdentifier + " was deleted!");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }

}

public static void deleteDatabaseInstance(RdsClient rdsClient, String
dbInstanceIdentifier) {

```

```

 try {
 DeleteDbInstanceRequest deleteDbInstanceRequest =
DeleteDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .deleteAutomatedBackups(true)
 .skipFinalSnapshot(true)
 .build();

 DeleteDbInstanceResponse response =
rdsClient.deleteDBInstance(deleteDbInstanceRequest);
 System.out.println("The status of the database is " +
response.dbInstance().dbInstanceStatus());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 public static void waitForSnapshotReady(RdsClient rdsClient, String
dbSnapshotIdentifier,
 String dbInstanceClusterIdentifier) {
 try {
 boolean snapshotReady = false;
 String snapshotReadyStr;
 System.out.println("Waiting for the snapshot to become available.");

 DescribeDbClusterSnapshotsRequest snapshotsRequest =
DescribeDbClusterSnapshotsRequest.builder()
 .dbClusterSnapshotIdentifier(dbSnapshotIdentifier)
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .build();

 while (!snapshotReady) {
 DescribeDbClusterSnapshotsResponse response =
rdsClient.describeDBClusterSnapshots(snapshotsRequest);
 List<DBClusterSnapshot> snapshotList =
response.dbClusterSnapshots();
 for (DBClusterSnapshot snapshot : snapshotList) {
 snapshotReadyStr = snapshot.status();
 if (snapshotReadyStr.contains("available")) {
 snapshotReady = true;
 } else {
 System.out.println(".");
 }
 }
 }
 }
 }

```

```

 Thread.sleep(sleepTime * 5000);
 }
}

System.out.println("The Snapshot is available!");

} catch (RdsException | InterruptedException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}

}

public static void createDBClusterSnapshot(RdsClient rdsClient, String
dbInstanceClusterIdentifier,
 String dbSnapshotIdentifier) {
 try {
 CreateDbClusterSnapshotRequest snapshotRequest =
CreateDbClusterSnapshotRequest.builder()
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .dbClusterSnapshotIdentifier(dbSnapshotIdentifier)
 .build();

 CreateDbClusterSnapshotResponse response =
rdsClient.createDBClusterSnapshot(snapshotRequest);
 System.out.println("The Snapshot ARN is " +
response.dbClusterSnapshot().dbClusterSnapshotArn());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void waitDBInstanceReady(RdsClient rdsClient, String
dbInstanceIdentifier) {
 boolean instanceReady = false;
 String instanceReadyStr;
 System.out.println("Waiting for instance to become available.");
 try {
 DescribeDbInstancesRequest instanceRequest =
DescribeDbInstancesRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .build();

```

```

 String endpoint = "";
 while (!instanceReady) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances(instanceRequest);
 List<DBInstance> instanceList = response.dbInstances();
 for (DBInstance instance : instanceList) {
 instanceReadyStr = instance.dbInstanceStatus();
 if (instanceReadyStr.contains("available")) {
 endpoint = instance.endpoint().address();
 instanceReady = true;
 } else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }
 System.out.println("Database instance is available! The connection
endpoint is " + endpoint);

 } catch (RdsException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static String createDBInstanceCluster(RdsClient rdsClient,
 String dbInstanceIdentifier,
 String dbInstanceClusterIdentifier,
 String instanceClass) {
 try {
 CreateDbInstanceRequest instanceRequest =
CreateDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .engine("aurora-mysql")
 .dbInstanceClass(instanceClass)
 .build();

 CreateDbInstanceResponse response =
rdsClient.createDBInstance(instanceRequest);
 System.out.print("The status is " +
response.dbInstance().dbInstanceStatus());
 return response.dbInstance().dbInstanceArn();
 }
}

```

```

 } catch (RdsException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}

public static String getListInstanceClasses(RdsClient rdsClient) {
 try {
 DescribeOrderableDbInstanceOptionsRequest optionsRequest =
DescribeOrderableDbInstanceOptionsRequest
 .builder()
 .engine("aurora-mysql")
 .maxRecords(20)
 .build();

 DescribeOrderableDbInstanceOptionsResponse response = rdsClient
 .describeOrderableDBInstanceOptions(optionsRequest);
 List<OrderableDBInstanceOption> instanceOptions =
response.orderableDBInstanceOptions();
 String instanceClass = "";
 for (OrderableDBInstanceOption instanceOption : instanceOptions) {
 instanceClass = instanceOption.dbInstanceClass();
 System.out.println("The instance class is " +
instanceOption.dbInstanceClass());
 System.out.println("The engine version is " +
instanceOption.engineVersion());
 }
 return instanceClass;
 } catch (RdsException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}

// Waits until the database instance is available.
public static void waitForInstanceReady(RdsClient rdsClient, String
dbClusterIdentifier) {
 boolean instanceReady = false;
 String instanceReadyStr;
 System.out.println("Waiting for instance to become available.");

```



```

 try {
 DescribeDbClustersRequest instanceRequest =
DescribeDbClustersRequest.builder()
 .dbClusterIdentifier(dbClusterIdentifier)
 .build();

 while (!instanceReady) {
 DescribeDbClustersResponse response =
rdsClient.describeDBClusters(instanceRequest);
 List<DBCluster> clusterList = response.dbClusters();
 for (DBCluster cluster : clusterList) {
 instanceReadyStr = cluster.status();
 if (instanceReadyStr.contains("available")) {
 instanceReady = true;
 } else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }
 System.out.println("Database cluster is available!");

 } catch (RdsException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 public static String createDBCluster(RdsClient rdsClient, String
dbParameterGroupFamily, String dbName,
 String dbClusterIdentifier, String userName, String password) {
 try {
 CreateDbClusterRequest clusterRequest = CreateDbClusterRequest.builder()
 .databaseName(dbName)
 .dbClusterIdentifier(dbClusterIdentifier)
 .dbClusterParameterGroupName(dbParameterGroupFamily)
 .engine("aurora-mysql")
 .masterUsername(userName)
 .masterUserPassword(password)
 .build();

 CreateDbClusterResponse response =
rdsClient.createDBCluster(clusterRequest);
 return response.dbCluster().dbClusterArn();
 }
 }

```

```

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
 }

 // Get a list of allowed engine versions.
 public static void getAllowedEngines(RdsClient rdsClient, String
dbParameterGroupFamily) {
 try {
 DescribeDbEngineVersionsRequest versionsRequest =
DescribeDbEngineVersionsRequest.builder()
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .engine("aurora-mysql")
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(versionsRequest);
 List<DBEngineVersion> dbEngines = response.dbEngineVersions();
 for (DBEngineVersion dbEngine : dbEngines) {
 System.out.println("The engine version is " +
dbEngine.engineVersion());
 System.out.println("The engine description is " +
dbEngine.dbEngineDescription());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 // Modify the auto_increment_offset parameter.
 public static void modifyDBClusterParas(RdsClient rdsClient, String
dClusterGroupName) {
 try {
 Parameter parameter1 = Parameter.builder()
 .parameterName("auto_increment_offset")
 .applyMethod("immediate")
 .parameterValue("5")
 .build();

```

```

 List<Parameter> paraList = new ArrayList<>();
 paraList.add(parameter1);
 ModifyDbClusterParameterGroupRequest groupRequest =
ModifyDbClusterParameterGroupRequest.builder()
 .dbClusterParameterGroupName(dClusterGroupName)
 .parameters(paraList)
 .build();

 ModifyDbClusterParameterGroupResponse response =
rdsClient.modifyDBClusterParameterGroup(groupRequest);
 System.out.println(
 "The parameter group " + response.dbClusterParameterGroupName()
+ " was successfully modified");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void describeDbClusterParameters(RdsClient rdsClient, String
dbClusterGroupName, int flag) {
 try {
 DescribeDbClusterParametersRequest dbParameterGroupsRequest;
 if (flag == 0) {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .build();
 } else {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .source("user")
 .build();
 }

 DescribeDbClusterParametersResponse response = rdsClient
 .describeDBClusterParameters(dbParameterGroupsRequest);
 List<Parameter> dbParameters = response.parameters();
 String paraName;
 for (Parameter para : dbParameters) {
 // Only print out information about either auto_increment_offset or
 // auto_increment_increment.

```

```

 paraName = para.parameterName();
 if ((paraName.compareTo("auto_increment_offset") == 0)
 || (paraName.compareTo("auto_increment_increment ") == 0)) {
 System.out.println("*** The parameter name is " + paraName);
 System.out.println("*** The parameter value is " +
para.parameterValue());
 System.out.println("*** The parameter data type is " +
para.dataType());
 System.out.println("*** The parameter description is " +
para.description());
 System.out.println("*** The parameter allowed values is " +
para.allowedValues());
 }
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void describeDbClusterParameterGroups(RdsClient rdsClient, String
dbClusterGroupName) {
 try {
 DescribeDbClusterParameterGroupsRequest groupsRequest =
DescribeDbClusterParameterGroupsRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .maxRecords(20)
 .build();

 List<DBClusterParameterGroup> groups =
rdsClient.describeDBClusterParameterGroups(groupsRequest)
 .dbClusterParameterGroups();
 for (DBClusterParameterGroup group : groups) {
 System.out.println("The group name is " +
group.dbClusterParameterGroupName());
 System.out.println("The group ARN is " +
group.dbClusterParameterGroupArn());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```

```

 }

 public static void createDBClusterParameterGroup(RdsClient rdsClient, String
dbClusterGroupName,
 String dbParameterGroupFamily) {
 try {
 CreateDbClusterParameterGroupRequest groupRequest =
CreateDbClusterParameterGroupRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .description("Created by using the AWS SDK for Java")
 .build();

 CreateDbClusterParameterGroupResponse response =
rdsClient.createDBClusterParameterGroup(groupRequest);
 System.out.println("The group name is " +
response.dbClusterParameterGroup().dbClusterParameterGroupName());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 public static void describeDBEngines(RdsClient rdsClient) {
 try {
 DescribeDbEngineVersionsRequest engineVersionsRequest =
DescribeDbEngineVersionsRequest.builder()
 .engine("aurora-mysql")
 .defaultOnly(true)
 .maxRecords(20)
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(engineVersionsRequest);
 List<DBEngineVersion> engines = response.dbEngineVersions();

 // Get all DBEngineVersion objects.
 for (DBEngineVersion engineOb : engines) {
 System.out.println("The name of the DB parameter group family for
the database engine is "
 + engineOb.dbParameterGroupFamily());
 System.out.println("The name of the database engine " +
engineOb.engine());
 }
 }
 }

```

```
 System.out.println("The version number of the database engine " +
engine0b.engineVersion());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateDBCluster](#)
- [CreateDBClusterParameterGroup](#)
- [CreateDBClusterSnapshot](#)
- [CreateDBInstance](#)
- [DeleteDBCluster](#)
- [DeleteDBClusterParameterGroup](#)
- [DeleteDBInstance](#)
- [DescribeDBClusterParameterGroups](#)
- [DescribeDBClusterParameters](#)
- [DescribeDBClusterSnapshots](#)
- [DescribeDBClusters](#)
- [DescribeDBEngineVersions](#)
- [DescribeDBInstances](#)
- [DescribeOrderableDBInstanceOptions](#)
- [ModifyDBClusterParameterGroup](#)

## Actions

### CreateDBCluster

The following code example shows how to use `CreateDBCluster`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createDBCluster(RdsClient rdsClient, String
dbParameterGroupFamily, String dbName,
 String dbClusterIdentifier, String userName, String password) {
 try {
 CreateDbClusterRequest clusterRequest = CreateDbClusterRequest.builder()
 .databaseName(dbName)
 .dbClusterIdentifier(dbClusterIdentifier)
 .dbClusterParameterGroupName(dbParameterGroupFamily)
 .engine("aurora-mysql")
 .masterUsername(userName)
 .masterUserPassword(password)
 .build();

 CreateDbClusterResponse response =
rdsClient.createDBCluster(clusterRequest);
 return response.dbCluster().dbClusterArn();

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBClusterParameterGroup

The following code example shows how to use `CreateDBClusterParameterGroup`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void createDBClusterParameterGroup(RdsClient rdsClient, String
dbClusterGroupName,
 String dbParameterGroupFamily) {
 try {
 CreateDbClusterParameterGroupRequest groupRequest =
CreateDbClusterParameterGroupRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .description("Created by using the AWS SDK for Java")
 .build();

 CreateDbClusterParameterGroupResponse response =
rdsClient.createDBClusterParameterGroup(groupRequest);
 System.out.println("The group name is " +
response.dbClusterParameterGroup().dbClusterParameterGroupName());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateDBClusterParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBClusterSnapshot

The following code example shows how to use `CreateDBClusterSnapshot`.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void createDBClusterSnapshot(RdsClient rdsClient, String
dbInstanceClusterIdentifier,
 String dbSnapshotIdentifier) {
 try {
 CreateDbClusterSnapshotRequest snapshotRequest =
CreateDbClusterSnapshotRequest.builder()
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .dbClusterSnapshotIdentifier(dbSnapshotIdentifier)
 .build();

 CreateDbClusterSnapshotResponse response =
rdsClient.createDBClusterSnapshot(snapshotRequest);
 System.out.println("The Snapshot ARN is " +
response.dbClusterSnapshot().dbClusterSnapshotArn());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateDBClusterSnapshot](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBInstance

The following code example shows how to use `CreateDBInstance`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createDBInstanceCluster(RdsClient rdsClient,
 String dbInstanceIdentifier,
 String dbInstanceClusterIdentifier,
 String instanceClass) {
 try {
 CreateDbInstanceRequest instanceRequest =
 CreateDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .engine("aurora-mysql")
 .dbInstanceClass(instanceClass)
 .build();

 CreateDbInstanceResponse response =
 rdsClient.createDBInstance(instanceRequest);
 System.out.print("The status is " +
 response.dbInstance().dbInstanceStatus());
 return response.dbInstance().dbInstanceArn();

 } catch (RdsException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBCluster

The following code example shows how to use DeleteDBCluster.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteCluster(RdsClient rdsClient, String
dbInstanceClusterIdentifier) {
 try {
 DeleteDbClusterRequest deleteDbClusterRequest =
DeleteDbClusterRequest.builder()
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .skipFinalSnapshot(true)
 .build();

 rdsClient.deleteDBCluster(deleteDbClusterRequest);
 System.out.println(dbInstanceClusterIdentifier + " was deleted!");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBClusterParameterGroup

The following code example shows how to use DeleteDBClusterParameterGroup.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

 public static void deleteDBClusterGroup(RdsClient rdsClient, String
dbClusterGroupName, String clusterDBARN)
 throws InterruptedException {
 try {
 boolean isDataDel = false;
 boolean didFind;
 String instanceARN;

 // Make sure that the database has been deleted.
 while (!isDataDel) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 int listSize = instanceList.size();
 didFind = false;
 int index = 1;
 for (DBInstance instance : instanceList) {
 instanceARN = instance.dbInstanceArn();
 if (instanceARN.compareTo(clusterDBARN) == 0) {
 System.out.println(clusterDBARN + " still exists");
 didFind = true;
 }
 if ((index == listSize) && (!didFind)) {
 // Went through the entire list and did not find the
database ARN.

 isDataDel = true;
 }
 Thread.sleep(sleepTime * 1000);
 index++;
 }
 }

 DeleteDbClusterParameterGroupRequest clusterParameterGroupRequest =
DeleteDbClusterParameterGroupRequest
 .builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .build();

 rdsClient.deleteDBClusterParameterGroup(clusterParameterGroupRequest);
 System.out.println(dbClusterGroupName + " was deleted.");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 }
}

```

```
 System.exit(1);
 }
}
```

- For API details, see [DeleteDBClusterParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBInstance

The following code example shows how to use DeleteDBInstance.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteDatabaseInstance(RdsClient rdsClient, String
dbInstanceIdIdentifier) {
 try {
 DeleteDbInstanceRequest deleteDbInstanceRequest =
DeleteDbInstanceRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .deleteAutomatedBackups(true)
 .skipFinalSnapshot(true)
 .build();

 DeleteDbInstanceResponse response =
rdsClient.deleteDBInstance(deleteDbInstanceRequest);
 System.out.println("The status of the database is " +
response.dbInstance().dbInstanceStatus());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBClusterParameterGroups

The following code example shows how to use `DescribeDBClusterParameterGroups`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeDbClusterParameterGroups(RdsClient rdsClient, String
dbClusterGroupName) {
 try {
 DescribeDbClusterParameterGroupsRequest groupsRequest =
DescribeDbClusterParameterGroupsRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .maxRecords(20)
 .build();

 List<DBClusterParameterGroup> groups =
rdsClient.describeDBClusterParameterGroups(groupsRequest)
 .dbClusterParameterGroups();
 for (DBClusterParameterGroup group : groups) {
 System.out.println("The group name is " +
group.dbClusterParameterGroupName());
 System.out.println("The group ARN is " +
group.dbClusterParameterGroupArn());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeDBClusterParameterGroups](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBClusterParameters

The following code example shows how to use `DescribeDBClusterParameters`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeDbClusterParameters(RdsClient rdsClient, String
dbClusterGroupName, int flag) {
 try {
 DescribeDbClusterParametersRequest dbParameterGroupsRequest;
 if (flag == 0) {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .build();
 } else {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .source("user")
 .build();
 }

 DescribeDbClusterParametersResponse response = rdsClient
 .describeDBClusterParameters(dbParameterGroupsRequest);
 List<Parameter> dbParameters = response.parameters();
 String paraName;
 for (Parameter para : dbParameters) {
 // Only print out information about either auto_increment_offset or
 // auto_increment_increment.
 paraName = para.parameterName();
 if ((paraName.compareTo("auto_increment_offset") == 0)
 || (paraName.compareTo("auto_increment_increment ") == 0)) {
```

```

 System.out.println("*** The parameter name is " + paraName);
 System.out.println("*** The parameter value is " +
para.parameterValue());
 System.out.println("*** The parameter data type is " +
para.dataType());
 System.out.println("*** The parameter description is " +
para.description());
 System.out.println("*** The parameter allowed values is " +
para.allowedValues());
 }
}

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
}

```

- For API details, see [DescribeDBClusterParameters](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBClusterSnapshots

The following code example shows how to use `DescribeDBClusterSnapshots`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void waitForSnapshotReady(RdsClient rdsClient, String
dbSnapshotIdentifier,
 String dbInstanceClusterIdentifier) {
 try {
 boolean snapshotReady = false;
 String snapshotReadyStr;
 System.out.println("Waiting for the snapshot to become available.");
 }
}

```



```

 DescribeDbClusterSnapshotsRequest snapshotsRequest =
DescribeDbClusterSnapshotsRequest.builder()
 .dbClusterSnapshotIdentifier(dbSnapshotIdentifier)
 .dbClusterIdentifier(dbInstanceClusterIdentifier)
 .build();

 while (!snapshotReady) {
 DescribeDbClusterSnapshotsResponse response =
rdsClient.describeDBClusterSnapshots(snapshotsRequest);
 List<DBClusterSnapshot> snapshotList =
response.dbClusterSnapshots();
 for (DBClusterSnapshot snapshot : snapshotList) {
 snapshotReadyStr = snapshot.status();
 if (snapshotReadyStr.contains("available")) {
 snapshotReady = true;
 } else {
 System.out.println(".");
 Thread.sleep(sleepTime * 5000);
 }
 }
 }

 System.out.println("The Snapshot is available!");

 } catch (RdsException | InterruptedException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeDBClusterSnapshots](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBClusters

The following code example shows how to use DescribeDBClusters.

## SDK for Java 2.x

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

 public static void describeDbClusterParameters(RdsClient rdsClient, String
dbClusterGroupName, int flag) {
 try {
 DescribeDbClusterParametersRequest dbParameterGroupsRequest;
 if (flag == 0) {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .build();
 } else {
 dbParameterGroupsRequest =
DescribeDbClusterParametersRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .source("user")
 .build();
 }

 DescribeDbClusterParametersResponse response = rdsClient
 .describeDBClusterParameters(dbParameterGroupsRequest);
 List<Parameter> dbParameters = response.parameters();
 String paraName;
 for (Parameter para : dbParameters) {
 // Only print out information about either auto_increment_offset or
 // auto_increment_increment.
 paraName = para.parameterName();
 if ((paraName.compareTo("auto_increment_offset") == 0)
 || (paraName.compareTo("auto_increment_increment ") == 0)) {
 System.out.println("*** The parameter name is " + paraName);
 System.out.println("*** The parameter value is " +
para.parameterValue());
 System.out.println("*** The parameter data type is " +
para.dataType());
 System.out.println("*** The parameter description is " +
para.description());
 }
 }
 } catch (Exception e) {
 System.out.println("Error: " + e.getMessage());
 }
 }

```

```

 System.out.println("*** The parameter allowed values is " +
para.allowedValues());
 }
}

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
}

```

- For API details, see [DescribeDBClusters](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBEngineVersions

The following code example shows how to use `DescribeDBEngineVersions`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void describeDBEngines(RdsClient rdsClient) {
 try {
 DescribeDbEngineVersionsRequest engineVersionsRequest =
DescribeDbEngineVersionsRequest.builder()
 .engine("aurora-mysql")
 .defaultOnly(true)
 .maxRecords(20)
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(engineVersionsRequest);
 List<DBEngineVersion> engines = response.dbEngineVersions();

 // Get all DBEngineVersion objects.
 for (DBEngineVersion engineOb : engines) {

```

```

 System.out.println("The name of the DB parameter group family for
the database engine is "
 + engine0b.dbParameterGroupFamily());
 System.out.println("The name of the database engine " +
engine0b.engine());
 System.out.println("The version number of the database engine " +
engine0b.engineVersion());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeDBEngineVersions](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBInstances

The following code example shows how to use DescribeDBInstances.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

// Waits until the database instance is available.
public static void waitForInstanceReady(RdsClient rdsClient, String
dbClusterIdentifier) {
 boolean instanceReady = false;
 String instanceReadyStr;
 System.out.println("Waiting for instance to become available.");
 try {
 DescribeDbClustersRequest instanceRequest =
DescribeDbClustersRequest.builder()
 .dbClusterIdentifier(dbClusterIdentifier)
 .build();
 }
}

```

```

 while (!instanceReady) {
 DescribeDbClustersResponse response =
rdsClient.describeDBClusters(instanceRequest);
 List<DBCluster> clusterList = response.dbClusters();
 for (DBCluster cluster : clusterList) {
 instanceReadyStr = cluster.status();
 if (instanceReadyStr.contains("available")) {
 instanceReady = true;
 } else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }
 System.out.println("Database cluster is available!");

 } catch (RdsException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeDBInstances](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeOrderableDBInstanceOptions

The following code example shows how to use `DescribeOrderableDBInstanceOptions`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void describeDBEngines(RdsClient rdsClient) {
 try {
 DescribeDbEngineVersionsRequest engineVersionsRequest =
DescribeDbEngineVersionsRequest.builder()

```

```

 .engine("aurora-mysql")
 .defaultOnly(true)
 .maxRecords(20)
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(engineVersionsRequest);
 List<DBEngineVersion> engines = response.dbEngineVersions();

 // Get all DBEngineVersion objects.
 for (DBEngineVersion engineOb : engines) {
 System.out.println("The name of the DB parameter group family for
the database engine is "
 + engineOb.dbParameterGroupFamily());
 System.out.println("The name of the database engine " +
engineOb.engine());
 System.out.println("The version number of the database engine " +
engineOb.engineVersion());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeOrderableDBInstanceOptions](#) in *AWS SDK for Java 2.x API Reference*.

## ModifyDBClusterParameterGroup

The following code example shows how to use `ModifyDBClusterParameterGroup`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

 public static void describeDbClusterParameterGroups(RdsClient rdsClient, String
dbClusterGroupName) {
 try {
 DescribeDbClusterParameterGroupsRequest groupsRequest =
DescribeDbClusterParameterGroupsRequest.builder()
 .dbClusterParameterGroupName(dbClusterGroupName)
 .maxRecords(20)
 .build();

 List<DBClusterParameterGroup> groups =
rdsClient.describeDBClusterParameterGroups(groupsRequest)
 .dbClusterParameterGroups();
 for (DBClusterParameterGroup group : groups) {
 System.out.println("The group name is " +
group.dbClusterParameterGroupName());
 System.out.println("The group ARN is " +
group.dbClusterParameterGroupArn());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ModifyDBClusterParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create an Aurora Serverless work item tracker

The following code example shows how to create a web application that tracks work items in an Amazon Aurora Serverless database and uses Amazon Simple Email Service (Amazon SES) to send reports.

#### SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon RDS database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Aurora Serverless data and for use by a React application, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run an example that uses the JDBC API, see the full example on [GitHub](#).

### Services used in this example

- Aurora
- Amazon RDS
- Amazon RDS Data Service
- Amazon SES

## Auto Scaling examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Auto Scaling.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)



# Get started

## Hello Auto Scaling

The following code example shows how to get started using Auto Scaling.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.autoscaling.AutoScalingClient;
import software.amazon.awssdk.services.autoscaling.model.AutoScalingGroup;
import
 software.amazon.awssdk.services.autoscaling.model.DescribeAutoScalingGroupsResponse;
import java.util.List;

/**
 * Before running this SDK for Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeAutoScalingGroups {
 public static void main(String[] args) throws InterruptedException {
 AutoScalingClient autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 describeGroups(autoScalingClient);
 }

 public static void describeGroups(AutoScalingClient autoScalingClient) {
 DescribeAutoScalingGroupsResponse response =
 autoScalingClient.describeAutoScalingGroups();
 List<AutoScalingGroup> groups = response.autoScalingGroups();
 }
}
```

```
groups.forEach(group -> {
 System.out.println("Group Name: " + group.autoScalingGroupName());
 System.out.println("Group ARN: " + group.autoScalingGroupARN());
});
}
```

- For API details, see [DescribeAutoScalingGroups](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an Amazon EC2 Auto Scaling group with a launch template and Availability Zones, and get information about running instances.
- Enable Amazon CloudWatch metrics collection.
- Update the group's desired capacity and wait for an instance to start.
- Terminate an instance in the group.
- List scaling activities that occur in response to user requests and capacity changes.
- Get statistics for CloudWatch metrics, then clean up resources.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this SDK for Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
```

```

* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*
* In addition, create a launch template. For more information, see the
* following topic:
*
* https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-launch-templates.html#create-launch-template
*
* This code example performs the following operations:
* 1. Creates an Auto Scaling group using an AutoScalingWaiter.
* 2. Gets a specific Auto Scaling group and returns an instance Id value.
* 3. Describes Auto Scaling with the Id value.
* 4. Enables metrics collection.
* 5. Update an Auto Scaling group.
* 6. Describes Account details.
* 7. Describe account details"
* 8. Updates an Auto Scaling group to use an additional instance.
* 9. Gets the specific Auto Scaling group and gets the number of instances.
* 10. List the scaling activities that have occurred for the group.
* 11. Terminates an instance in the Auto Scaling group.
* 12. Stops the metrics collection.
* 13. Deletes the Auto Scaling group.
*/

public class AutoScalingScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final String ROLES_STACK = "MyCdkAutoScaleStack";
 public static void main(String[] args) throws InterruptedException {
 final String usage = ""

 Usage:
 <groupName>

 Where:
 groupName - The name of the Auto Scaling group.
 """;

 String groupName = "MyAutoScalingGroup2";
 AutoScalingClient autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_WEST_2)
 .build();

 Ec2Client ec2 = Ec2Client.builder()
 .region(Region.US_WEST_2)

```

```

 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon EC2 Auto Scaling example
scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("First, we will create a launch template using a
CloudFormation script");
 CloudFormationHelper.deployCloudFormationStack(ROLES_STACK);
 Map<String, String> stackOutputs =
CloudFormationHelper.getStackOutputsAsync(ROLES_STACK).join();
 String launchTemplateName = stackOutputs.get("LaunchTemplateNameOutput");
 String vpcZoneId = getVPC(ec2);
 updateTemplate(ec2, launchTemplateName);
 System.out.println("The VPC zone id created by the CloudFormation stack
is"+vpcZoneId);

 System.out.println("1. Create an Auto Scaling group named " + groupName);
 createAutoScalingGroup(autoScalingClient, ec2, groupName,
launchTemplateName, vpcZoneId);

 System.out.println(
 "Wait 1 min for the resources, including the instance. Otherwise, an
empty instance Id is returned");
 Thread.sleep(60000);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Get Auto Scale group Id value");
 String instanceId = getSpecificAutoScalingGroups(autoScalingClient,
groupName);
 if (instanceId.compareTo("") == 0) {
 System.out.println("Error - no instance Id value");
 System.exit(1);
 } else {
 System.out.println("The instance Id value is " + instanceId);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Describe Auto Scaling with the Id value " +
instanceId);

```

```
describeAutoScalingInstance(autoScalingClient, instanceId);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Enable metrics collection " + instanceId);
enableMetricsCollection(autoScalingClient, groupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Update an Auto Scaling group to update max size to
3");
updateAutoScalingGroup(autoScalingClient, groupName, launchTemplateName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Describe Auto Scaling groups");
describeAutoScalingGroups(autoScalingClient, groupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Describe account details");
describeAccountLimits(autoScalingClient);
System.out.println(
 "Wait 1 min for the resources, including the instance. Otherwise, an
empty instance Id is returned");
Thread.sleep(60000);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("8. Set desired capacity to 2");
setDesiredCapacity(autoScalingClient, groupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("9. Get the two instance Id values and state");
getSpecificAutoScalingGroups(autoScalingClient, groupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("10. List the scaling activities that have occurred for
the group");
describeScalingActivities(autoScalingClient, groupName);
System.out.println(DASHES);
```

```

 System.out.println(DASHES);
 System.out.println("11. Terminate an instance in the Auto Scaling group");
 terminateInstanceInAutoScalingGroup(autoScalingClient, instanceId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("12. Stop the metrics collection");
 disableMetricsCollection(autoScalingClient, groupName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("13. Delete the Auto Scaling group and cloud formation
resources");
 CloudFormationHelper.destroyCloudFormationStack(ROLES_STACK);
 deleteAutoScalingGroup(autoScalingClient, groupName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The Scenario has successfully completed.");
 System.out.println(DASHES);

 autoScalingClient.close();
 }

 public static String getVPC(Ec2Client ec2) {
 try {
 DescribeVpcsRequest request = DescribeVpcsRequest.builder()
 .filters(f -> f.name("isDefault").values("true"))
 .build();

 DescribeVpcsResponse response = ec2.describeVpcs(request);

 if (!response.vpcs().isEmpty()) {
 Vpc defaultVpc = response.vpcs().get(0);
 System.out.println("Default VPC ID: " + defaultVpc.vpcId());
 return defaultVpc.vpcId();
 } else {
 System.out.println("No default VPC found.");
 return null; // Return null if no default VPC is found
 }
 } catch (Ec2Exception e) {
 System.err.println("EC2 error: " + e.awsErrorDetails().errorMessage());
 return null; // Return null in case of an error
 }
 }

```

```

 }
}

public static void updateTemplate(Ec2Client ec2, String launchTemplateName) {
 // Step 1: Create new launch template version
 String newAmiId = "ami-0025f0db847eb6254";
 RequestLaunchTemplateData launchTemplateData =
RequestLaunchTemplateData.builder()
 .imageId(newAmiId)
 .build();

 CreateLaunchTemplateVersionRequest createVersionRequest =
CreateLaunchTemplateVersionRequest.builder()
 .launchTemplateName(launchTemplateName)
 .versionDescription("Updated with valid AMI")
 .sourceVersion("1")
 .launchTemplateData(launchTemplateData)
 .build();

 CreateLaunchTemplateVersionResponse createResponse =
ec2.createLaunchTemplateVersion(createVersionRequest);
 int newVersionNumber =
createResponse.launchTemplateVersion().versionNumber().intValue();

 // Step 2: Modify default version
 ModifyLaunchTemplateRequest modifyRequest =
ModifyLaunchTemplateRequest.builder()
 .launchTemplateName(launchTemplateName)
 .defaultVersion(String.valueOf(newVersionNumber))
 .build();

 ec2.modifyLaunchTemplate(modifyRequest);
 System.out.println("Updated launch template to version " + newVersionNumber
+ " with AMI " + newAmiId);
}

public static void describeScalingActivities(AutoScalingClient
autoScalingClient, String groupName) {
 try {
 DescribeScalingActivitiesRequest scalingActivitiesRequest =
DescribeScalingActivitiesRequest.builder()
 .autoScalingGroupName(groupName)

```

```

 .maxRecords(10)
 .build();

 DescribeScalingActivitiesResponse response = autoScalingClient
 .describeScalingActivities(scalingActivitiesRequest);
 List<Activity> activities = response.activities();
 for (Activity activity : activities) {
 System.out.println("The activity Id is " + activity.activityId());
 System.out.println("The activity details are " +
activity.details());
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

 public static void setDesiredCapacity(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 SetDesiredCapacityRequest capacityRequest =
SetDesiredCapacityRequest.builder()
 .autoScalingGroupName(groupName)
 .desiredCapacity(2)
 .build();

 autoScalingClient.setDesiredCapacity(capacityRequest);
 System.out.println("You have set the DesiredCapacity to 2");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void createAutoScalingGroup(AutoScalingClient autoScalingClient,
 Ec2Client ec2Client,
 String groupName,
 String launchTemplateName,
 String vpcId) {
 try {
 // Step 1: Get one subnet ID in the given VPC
 DescribeSubnetsRequest subnetRequest = DescribeSubnetsRequest.builder()

```



```

 .filters(Filter.builder().name("vpc-id").values(vpcId).build())
 .build();

 DescribeSubnetsResponse subnetResponse =
ec2Client.describeSubnets(subnetRequest);

 if (subnetResponse.subnets().isEmpty()) {
 throw new RuntimeException("No subnets found in VPC: " + vpcId);
 }

 String subnetId = subnetResponse.subnets().get(0).subnetId(); // Use
first subnet
 System.out.println("Using subnet: " + subnetId);

 // Step 2: Create launch template reference
 LaunchTemplateSpecification templateSpecification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(launchTemplateName)
 .build();

 // Step 3: Create Auto Scaling group
 CreateAutoScalingGroupRequest request =
CreateAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .launchTemplate(templateSpecification)
 .minSize(1)
 .maxSize(1)
 .vpcZoneIdentifier(subnetId) // Correct: subnet ID, not VPC ID
 .build();

 autoScalingClient.createAutoScalingGroup(request);

 // Step 4: Wait until group is created
 AutoScalingWaiter waiter = autoScalingClient.waiter();
 DescribeAutoScalingGroupsRequest groupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 WaiterResponse<DescribeAutoScalingGroupsResponse> waiterResponse =
 waiter.waitUntilGroupExists(groupsRequest);

 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Auto Scaling Group created");

```

```
 } catch (Ec2Exception | AutoScalingException e) {
 System.err.println("Error: " + e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void describeAutoScalingInstance(AutoScalingClient
autoScalingClient, String id) {
 try {
 DescribeAutoScalingInstancesRequest describeAutoScalingInstancesRequest
= DescribeAutoScalingInstancesRequest
 .builder()
 .instanceIds(id)
 .build();

 DescribeAutoScalingInstancesResponse response = autoScalingClient

.describeAutoScalingInstances(describeAutoScalingInstancesRequest);
 List<AutoScalingInstanceDetails> instances =
response.autoScalingInstances();
 for (AutoScalingInstanceDetails instance : instances) {
 System.out.println("The instance lifecycle state is: " +
instance.lifecycleState());
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void describeAutoScalingGroups(AutoScalingClient
autoScalingClient, String groupName) {
 try {
 DescribeAutoScalingGroupsRequest groupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .maxRecords(10)
 .build();

 DescribeAutoScalingGroupsResponse response =
autoScalingClient.describeAutoScalingGroups(groupsRequest);
 List<AutoScalingGroup> groups = response.autoScalingGroups();
```

```

 for (AutoScalingGroup group : groups) {
 System.out.println("*** The service to use for the health checks: "
+ group.healthCheckType());
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static String getSpecificAutoScalingGroups(AutoScalingClient
autoScalingClient, String groupName) {
 try {
 String instanceId = "";
 DescribeAutoScalingGroupsRequest scalingGroupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response = autoScalingClient
 .describeAutoScalingGroups(scalingGroupsRequest);
 List<AutoScalingGroup> groups = response.autoScalingGroups();
 for (AutoScalingGroup group : groups) {
 System.out.println("The group name is " +
group.autoScalingGroupName());
 System.out.println("The group ARN is " +
group.autoScalingGroupARN());
 List<Instance> instances = group.instances();

 for (Instance instance : instances) {
 instanceId = instance.instanceId();
 System.out.println("The instance id is " + instanceId);
 System.out.println("The lifecycle state is " +
instance.lifecycleState());
 }
 }

 return instanceId;
 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

```

```
}

 public static void enableMetricsCollection(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 EnableMetricsCollectionRequest collectionRequest =
EnableMetricsCollectionRequest.builder()
 .autoScalingGroupName(groupName)
 .metrics("GroupMaxSize")
 .granularity("1Minute")
 .build();

 autoScalingClient.enableMetricsCollection(collectionRequest);
 System.out.println("The enable metrics collection operation was
successful");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void disableMetricsCollection(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 DisableMetricsCollectionRequest disableMetricsCollectionRequest =
DisableMetricsCollectionRequest.builder()
 .autoScalingGroupName(groupName)
 .metrics("GroupMaxSize")
 .build();

 autoScalingClient.disableMetricsCollection(disableMetricsCollectionRequest);
 System.out.println("The disable metrics collection operation was
successful");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void describeAccountLimits(AutoScalingClient autoScalingClient) {
 try {
```

```

 DescribeAccountLimitsResponse response =
autoScalingClient.describeAccountLimits();
 System.out.println("The max number of auto scaling groups is " +
response.maxNumberOfAutoScalingGroups());
 System.out.println("The current number of auto scaling groups is " +
response.numberOfWorkAutoScalingGroups());

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void updateAutoScalingGroup(AutoScalingClient autoScalingClient,
String groupName,
 String launchTemplateName) {
 try {
 AutoScalingWaiter waiter = autoScalingClient.waiter();
 LaunchTemplateSpecification templateSpecification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(launchTemplateName)
 .build();

 UpdateAutoScalingGroupRequest groupRequest =
UpdateAutoScalingGroupRequest.builder()
 .maxSize(3)
 .autoScalingGroupName(groupName)
 .launchTemplate(templateSpecification)
 .build();

 autoScalingClient.updateAutoScalingGroup(groupRequest);
 DescribeAutoScalingGroupsRequest groupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 WaiterResponse<DescribeAutoScalingGroupsResponse> waiterResponse =
waiter
 .waitUntilGroupInService(groupsRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("You successfully updated the auto scaling group " +
groupName);

 } catch (AutoScalingException e) {

```

```
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void terminateInstanceInAutoScalingGroup(AutoScalingClient
autoScalingClient, String instanceId) {
 try {
 TerminateInstanceInAutoScalingGroupRequest request =
 TerminateInstanceInAutoScalingGroupRequest.builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 autoScalingClient.terminateInstanceInAutoScalingGroup(request);
 System.out.println("You have terminated instance " + instanceId);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteAutoScalingGroup(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
 DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

 autoScalingClient.deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println("You successfully deleted " + groupName);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateAutoScalingGroup](#)
- [DeleteAutoScalingGroup](#)
- [DescribeAutoScalingGroups](#)
- [DescribeAutoScalingInstances](#)
- [DescribeScalingActivities](#)
- [DisableMetricsCollection](#)
- [EnableMetricsCollection](#)
- [SetDesiredCapacity](#)
- [TerminateInstanceInAutoScalingGroup](#)
- [UpdateAutoScalingGroup](#)

## Actions

### CreateAutoScalingGroup

The following code example shows how to use CreateAutoScalingGroup.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.autoscaling.AutoScalingClient;
import software.amazon.awssdk.services.autoscaling.model.AutoScalingException;
import
 software.amazon.awssdk.services.autoscaling.model.CreateAutoScalingGroupRequest;
import
 software.amazon.awssdk.services.autoscaling.model.DescribeAutoScalingGroupsRequest;
import
 software.amazon.awssdk.services.autoscaling.model.DescribeAutoScalingGroupsResponse;
import
 software.amazon.awssdk.services.autoscaling.model.LaunchTemplateSpecification;
```

```

import software.amazon.awssdk.services.autoscaling.waiters.AutoScalingWaiter;

/**
 * Before running this SDK for Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateAutoScalingGroup {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <groupName> <launchTemplateName> <serviceLinkedRoleARN>
<vpcZoneId>

 Where:
 groupName - The name of the Auto Scaling group.
 launchTemplateName - The name of the launch template.\s
 vpcZoneId - A subnet Id for a virtual private cloud (VPC) where
instances in the Auto Scaling group can be created.
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
 String launchTemplateName = args[1];
 String vpcZoneId = args[2];
 AutoScalingClient autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 createAutoScalingGroup(autoScalingClient, groupName, launchTemplateName,
vpcZoneId);
 autoScalingClient.close();
 }

 public static void createAutoScalingGroup(AutoScalingClient autoScalingClient,
 String groupName,

```



```

 String launchTemplateName,
 String vpcZoneId) {

 try {
 AutoScalingWaiter waiter = autoScalingClient.waiter();
 LaunchTemplateSpecification templateSpecification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(launchTemplateName)
 .build();

 CreateAutoScalingGroupRequest request =
CreateAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .availabilityZones("us-east-1a")
 .launchTemplate(templateSpecification)
 .maxSize(1)
 .minSize(1)
 .vpcZoneIdentifier(vpcZoneId)
 .build();

 autoScalingClient.createAutoScalingGroup(request);
 DescribeAutoScalingGroupsRequest groupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 WaiterResponse<DescribeAutoScalingGroupsResponse> waiterResponse =
waiter
 .waitUntilGroupExists(groupsRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Auto Scaling Group created");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [CreateAutoScalingGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAutoScalingGroup

The following code example shows how to use DeleteAutoScalingGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.autoscaling.AutoScalingClient;
import software.amazon.awssdk.services.autoscaling.model.AutoScalingException;
import
 software.amazon.awssdk.services.autoscaling.model.DeleteAutoScalingGroupRequest;

/**
 * Before running this SDK for Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteAutoScalingGroup {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <groupName>

 Where:
 groupName - The name of the Auto Scaling group.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
```

```

 AutoScalingClient autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 deleteAutoScalingGroup(autoScalingClient, groupName);
 autoScalingClient.close();
 }

 public static void deleteAutoScalingGroup(AutoScalingClient autoScalingClient,
 String groupName) {
 try {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
 DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

 autoScalingClient.deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println("You successfully deleted " + groupName);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DeleteAutoScalingGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAutoScalingGroups

The following code example shows how to use `DescribeAutoScalingGroups`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.autoscaling.AutoScalingClient;
import software.amazon.awssdk.services.autoscaling.model.AutoScalingException;
import software.amazon.awssdk.services.autoscaling.model.AutoScalingGroup;
import
 software.amazon.awssdk.services.autoscaling.model.DescribeAutoScalingGroupsResponse;
import
 software.amazon.awssdk.services.autoscaling.model.DescribeAutoScalingGroupsRequest;
import software.amazon.awssdk.services.autoscaling.model.Instance;
import java.util.List;

/**
 * Before running this SDK for Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeAutoScalingInstances {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <groupName>

 Where:
 groupName - The name of the Auto Scaling group.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
 AutoScalingClient autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String instanceId = getAutoScaling(autoScalingClient, groupName);
 System.out.println(instanceId);
 autoScalingClient.close();
 }
}
```

```

 }

 public static String getAutoScaling(AutoScalingClient autoScalingClient, String
groupName) {
 try {
 String instanceId = "";
 DescribeAutoScalingGroupsRequest scalingGroupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response = autoScalingClient
 .describeAutoScalingGroups(scalingGroupsRequest);
 List<AutoScalingGroup> groups = response.autoScalingGroups();
 for (AutoScalingGroup group : groups) {
 System.out.println("The group name is " +
group.autoScalingGroupName());
 System.out.println("The group ARN is " +
group.autoScalingGroupARN());

 List<Instance> instances = group.instances();
 for (Instance instance : instances) {
 instanceId = instance.instanceId();
 }
 }
 return instanceId;
 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [DescribeAutoScalingGroups](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAutoScalingInstances

The following code example shows how to use `DescribeAutoScalingInstances`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeAutoScalingInstance(AutoScalingClient
autoScalingClient, String id) {
 try {
 DescribeAutoScalingInstancesRequest describeAutoScalingInstancesRequest
= DescribeAutoScalingInstancesRequest
 .builder()
 .instanceIds(id)
 .build();

 DescribeAutoScalingInstancesResponse response = autoScalingClient
 .describeAutoScalingInstances(describeAutoScalingInstancesRequest);
 List<AutoScalingInstanceDetails> instances =
response.autoScalingInstances();
 for (AutoScalingInstanceDetails instance : instances) {
 System.out.println("The instance lifecycle state is: " +
instance.lifecycleState());
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeAutoScalingInstances](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeScalingActivities

The following code example shows how to use `DescribeScalingActivities`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeScalingActivities(AutoScalingClient
autoScalingClient, String groupName) {
 try {
 DescribeScalingActivitiesRequest scalingActivitiesRequest =
DescribeScalingActivitiesRequest.builder()
 .autoScalingGroupName(groupName)
 .maxRecords(10)
 .build();

 DescribeScalingActivitiesResponse response = autoScalingClient
 .describeScalingActivities(scalingActivitiesRequest);
 List<Activity> activities = response.activities();
 for (Activity activity : activities) {
 System.out.println("The activity Id is " + activity.activityId());
 System.out.println("The activity details are " +
activity.details());
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeScalingActivities](#) in *AWS SDK for Java 2.x API Reference*.

## DisableMetricsCollection

The following code example shows how to use `DisableMetricsCollection`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void disableMetricsCollection(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 DisableMetricsCollectionRequest disableMetricsCollectionRequest =
DisableMetricsCollectionRequest.builder()
 .autoScalingGroupName(groupName)
 .metrics("GroupMaxSize")
 .build();

 autoScalingClient.disableMetricsCollection(disableMetricsCollectionRequest);
 System.out.println("The disable metrics collection operation was
successful");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DisableMetricsCollection](#) in *AWS SDK for Java 2.x API Reference*.

## EnableMetricsCollection

The following code example shows how to use `EnableMetricsCollection`.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void enableMetricsCollection(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 EnableMetricsCollectionRequest collectionRequest =
 EnableMetricsCollectionRequest.builder()
 .autoScalingGroupName(groupName)
 .metrics("GroupMaxSize")
 .granularity("1Minute")
 .build();

 autoScalingClient.enableMetricsCollection(collectionRequest);
 System.out.println("The enable metrics collection operation was
successful");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [EnableMetricsCollection](#) in *AWS SDK for Java 2.x API Reference*.

## SetDesiredCapacity

The following code example shows how to use SetDesiredCapacity.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void setDesiredCapacity(AutoScalingClient autoScalingClient,
String groupName) {
 try {
 SetDesiredCapacityRequest capacityRequest =
SetDesiredCapacityRequest.builder()
 .autoScalingGroupName(groupName)
 .desiredCapacity(2)
 .build();

 autoScalingClient.setDesiredCapacity(capacityRequest);
 System.out.println("You have set the DesiredCapacity to 2");

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [SetDesiredCapacity](#) in *AWS SDK for Java 2.x API Reference*.

## TerminateInstanceInAutoScalingGroup

The following code example shows how to use `TerminateInstanceInAutoScalingGroup`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void terminateInstanceInAutoScalingGroup(AutoScalingClient
autoScalingClient, String instanceId) {
 try {
 TerminateInstanceInAutoScalingGroupRequest request =
 TerminateInstanceInAutoScalingGroupRequest.builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 autoScalingClient.terminateInstanceInAutoScalingGroup(request);
 System.out.println("You have terminated instance " + instanceId);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [TerminateInstanceInAutoScalingGroup](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateAutoScalingGroup

The following code example shows how to use UpdateAutoScalingGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void updateAutoScalingGroup(AutoScalingClient autoScalingClient,
String groupName,
 String launchTemplateName) {
 try {
 AutoScalingWaiter waiter = autoScalingClient.waiter();
 LaunchTemplateSpecification templateSpecification =
 LaunchTemplateSpecification.builder()
```

```

 .launchTemplateName(launchTemplateName)
 .build();

 UpdateAutoScalingGroupRequest groupRequest =
UpdateAutoScalingGroupRequest.builder()
 .maxSize(3)
 .autoScalingGroupName(groupName)
 .launchTemplate(templateSpecification)
 .build();

 autoScalingClient.updateAutoScalingGroup(groupRequest);
 DescribeAutoScalingGroupsRequest groupsRequest =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 WaiterResponse<DescribeAutoScalingGroupsResponse> waiterResponse =
waiter
 .waitUntilGroupInService(groupsRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("You successfully updated the auto scaling group " +
groupName);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [UpdateAutoScalingGroup](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build and manage a resilient service

The following code example shows how to create a load-balanced web service that returns book, movie, and song recommendations. The example shows how the service responds to failures, and how to restructure the service for more resilience when failures occur.

- Use an Amazon EC2 Auto Scaling group to create Amazon Elastic Compute Cloud (Amazon EC2) instances based on a launch template and to keep the number of instances in a specified range.

- Handle and distribute HTTP requests with Elastic Load Balancing.
- Monitor the health of instances in an Auto Scaling group and forward requests only to healthy instances.
- Run a Python web server on each EC2 instance to handle HTTP requests. The web server responds with recommendations and health checks.
- Simulate a recommendation service with an Amazon DynamoDB table.
- Control web server response to requests and health checks by updating AWS Systems Manager parameters.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run the interactive scenario at a command prompt.

```
public class Main {

 public static final String fileName = "C:\\\\AWS\\\\resworkflow\\recommenda-
tions.json"; // Modify file location.
 public static final String tableName = "doc-example-recommendation-service";
 public static final String startScript = "C:\\\\AWS\\\\resworkflow\\server_startu-
p_script.sh"; // Modify file location.
 public static final String policyFile = "C:\\\\AWS\\\\resworkflow\\instance_
policy.json"; // Modify file location.
 public static final String ssmJSON = "C:\\\\AWS\\\\resworkflow\\ssm_only_
policy.json"; // Modify file location.
 public static final String failureResponse = "doc-example-resilient-
architecture-failure-response";
 public static final String healthCheck = "doc-example-resilient-architecture-
health-check";
 public static final String templateName = "doc-example-resilience-template";
 public static final String roleName = "doc-example-resilience-role";
 public static final String policyName = "doc-example-resilience-pol";
 public static final String profileName = "doc-example-resilience-prof";
```

```

 public static final String badCredsProfileName = "doc-example-resilience-prof-
bc";

 public static final String targetGroupName = "doc-example-resilience-tg";
 public static final String autoScalingGroupName = "doc-example-resilience-
group";
 public static final String lbName = "doc-example-resilience-lb";
 public static final String protocol = "HTTP";
 public static final int port = 80;

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws IOException, InterruptedException
 {
 Scanner in = new Scanner(System.in);
 Database database = new Database();
 AutoScaler autoScaler = new AutoScaler();
 LoadBalancer loadBalancer = new LoadBalancer();

 System.out.println(DASHES);
 System.out.println("Welcome to the demonstration of How to Build and Manage
a Resilient Service!");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("A - SETUP THE RESOURCES");
 System.out.println("Press Enter when you're ready to start deploying
resources.");
 in.nextLine();
 deploy(loadBalancer);
 System.out.println(DASHES);
 System.out.println(DASHES);
 System.out.println("B - DEMO THE RESILIENCE FUNCTIONALITY");
 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 demo(loadBalancer);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("C - DELETE THE RESOURCES");
 System.out.println(""""
 This concludes the demo of how to build and manage a resilient
service.

```

```

 To keep things tidy and to avoid unwanted charges on your account,
 we can clean up all AWS resources
 that were created for this demo.
 """);

 System.out.println("\n Do you want to delete the resources (y/n)? ");
 String userInput = in.nextLine().trim().toLowerCase(); // Capture user input

 if (userInput.equals("y")) {
 // Delete resources here
 deleteResources(loadBalancer, autoScaler, database);
 System.out.println("Resources deleted.");
 } else {
 System.out.println("""
 Okay, we'll leave the resources intact.
 Don't forget to delete them when you're done with them or you
 might incur unexpected charges.
 """);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The example has completed. ");
 System.out.println("\n Thanks for watching!");
 System.out.println(DASHES);
}

// Deletes the AWS resources used in this example.
private static void deleteResources(LoadBalancer loadBalancer, AutoScaler
autoScaler, Database database)
 throws IOException, InterruptedException {
 loadBalancer.deleteLoadBalancer(lbName);
 System.out.println("*** Wait 30 secs for resource to be deleted");
 TimeUnit.SECONDS.sleep(30);
 loadBalancer.deleteTargetGroup(targetGroupName);
 autoScaler.deleteAutoScaleGroup(autoScalingGroupName);
 autoScaler.deleteRolesPolicies(policyName, roleName, profileName);
 autoScaler.deleteTemplate(templateName);
 database.deleteTable(tableName);
}

private static void deploy(LoadBalancer loadBalancer) throws
InterruptedException, IOException {
 Scanner in = new Scanner(System.in);

```

```

 System.out.println(
 """
 For this demo, we'll use the AWS SDK for Java (v2) to create
several AWS resources
 to set up a load-balanced web service endpoint and explore
some ways to make it resilient
 against various kinds of failures.

 Some of the resources create by this demo are:
 \t* A DynamoDB table that the web service depends on to
provide book, movie, and song recommendations.
 \t* An EC2 launch template that defines EC2 instances that
each contain a Python web server.
 \t* An EC2 Auto Scaling group that manages EC2 instances
across several Availability Zones.
 \t* An Elastic Load Balancing (ELB) load balancer that
targets the Auto Scaling group to distribute requests.
 """);

 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Creating and populating a DynamoDB table named " +
tableName);
 Database database = new Database();
 database.createTable(tableName, fileName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an EC2 launch template that runs '{startup_script}' when an
instance starts.
 This script starts a Python web server defined in the `server.py`
script. The web server
 listens to HTTP requests on port 80 and responds to requests to '/'
and to '/healthcheck'.
 For demo purposes, this server is run as the root user. In
production, the best practice is to
 run a web server, such as Apache, with least-privileged credentials.

 The template also defines an IAM policy that each instance uses to
assume a role that grants

```



```

 permissions to access the DynamoDB recommendation table and Systems
Manager parameters
 that control the flow of the demo.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 templateCreator.createTemplate(policyFile, policyName, profileName,
startScript, templateName, roleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(
 "Creating an EC2 Auto Scaling group that maintains three EC2
instances, each in a different Availability Zone.");
 System.out.println("*** Wait 30 secs for the VPC to be created");
 TimeUnit.SECONDS.sleep(30);
 AutoScaler autoScaler = new AutoScaler();
 String[] zones = autoScaler.createGroup(3, templateName,
autoScalingGroupName);

 System.out.println("""
 At this point, you have EC2 instances created. Once each instance
starts, it listens for
 HTTP requests. You can see these instances in the console or
continue with the demo.
 Press Enter when you're ready to continue.
 """);

 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Creating variables that control the flow of the demo.");
 ParameterHelper paramHelper = new ParameterHelper();
 paramHelper.reset();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an Elastic Load Balancing target group and load balancer.
The target group
 defines how the load balancer connects to instances. The load
balancer provides a

```

```

 single endpoint where clients connect and dispatches requests to
instances in the group.
 """);

 String vpcId = autoScaler.getDefaultVPC();
 List<Subnet> subnets = autoScaler.getSubnets(vpcId, zones);
 System.out.println("You have retrieved a list with " + subnets.size() + "
subnets");
 String targetGroupArn = loadBalancer.createTargetGroup(protocol, port,
vpcId, targetGroupName);
 String elbDnsName = loadBalancer.createLoadBalancer(subnets, targetGroupArn,
lbName, port, protocol);
 autoScaler.attachLoadBalancerTargetGroup(autoScalingGroupName,
targetGroupArn);
 System.out.println("Verifying access to the load balancer endpoint...");
 boolean wasSuccessful = loadBalancer.verifyLoadBalancerEndpoint(elbDnsName);
 if (!wasSuccessful) {
 System.out.println("Couldn't connect to the load balancer, verifying
that the port is open...");
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to "http://checkip.amazonaws.com"
 HttpGet httpGet = new HttpGet("http://checkip.amazonaws.com");
 try {
 // Execute the request and get the response
 HttpResponse response = httpClient.execute(httpGet);

 // Read the response content.
 String ipAddress =
IOUtils.toString(response.getEntity().getContent(), StandardCharsets.UTF_8).trim();

 // Print the public IP address.
 System.out.println("Public IP Address: " + ipAddress);
 GroupInfo groupInfo = autoScaler.verifyInboundPort(vpcId, port,
ipAddress);
 if (!groupInfo.isPortOpen()) {
 System.out.println("""
 For this example to work, the default security group for
your default VPC must
 allow access from this computer. You can either add it
automatically from this
 example or add it yourself using the AWS Management
Console.
 """);
 }
 }
 }
 }
}

```

```

 System.out.println(
 "Do you want to add a rule to security group " +
groupInfo.getGroupName() + " to allow");
 System.out.println("inbound traffic on port " + port + " from
your computer's IP address (y/n) ");
 String ans = in.nextLine();
 if ("y".equalsIgnoreCase(ans)) {
 autoScaler.openInboundPort(groupInfo.getGroupName(),
String.valueOf(port), ipAddress);
 System.out.println("Security group rule added.");
 } else {
 System.out.println("No security group rule added.");
 }
 }

 } catch (AutoScalingException e) {
 e.printStackTrace();
 }
 } else if (wasSuccessful) {
 System.out.println("Your load balancer is ready. You can access it by
browsing to:");
 System.out.println("\t http://" + elbDnsName);
 } else {
 System.out.println("Couldn't get a successful response from the load
balancer endpoint. Troubleshoot by");
 System.out.println("manually verifying that your VPC and security group
are configured correctly and that");
 System.out.println("you can successfully make a GET request to the load
balancer.");
 }

 System.out.println("Press Enter when you're ready to continue with the
demo.");
 in.nextLine();
}

// A method that controls the demo part of the Java program.
public static void demo(LoadBalancer loadBalancer) throws IOException,
InterruptedException {
 ParameterHelper paramHelper = new ParameterHelper();
 System.out.println("Read the ssm_only_policy.json file");
 String ssmOnlyPolicy = readFileAsString(ssmJSON);

```

```

 System.out.println("Resetting parameters to starting values for demo.");
 paramHelper.reset();

 System.out.println(
 """
 This part of the demonstration shows how to toggle
different parts of the system
 to create situations where the web service fails, and shows
how using a resilient
 architecture can keep the web service running in spite of
these failures.

 At the start, the load balancer endpoint returns
recommendations and reports that all targets are healthy.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 """
 The web service running on the EC2 instances gets
recommendations by querying a DynamoDB table.
 The table name is contained in a Systems Manager parameter
named self.param_helper.table.
 To simulate a failure of the recommendation service, let's
set this parameter to name a non-existent table.
 """);
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 System.out.println(
 """
 \nNow, sending a GET request to the load balancer endpoint
returns a failure code. But, the service reports as
 healthy to the load balancer because shallow health checks
don't check for failure of the recommendation service.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 """
 Instead of failing when the recommendation service fails,
the web service can return a static response.
 While this is not a perfect solution, it presents the
customer with a somewhat better experience than failure.
 """);

```

```

 paramHelper.put(paramHelper.failureResponse, "static");

 System.out.println("""
 Now, sending a GET request to the load balancer endpoint returns a
static response.
 The service still reports as healthy because health checks are still
shallow.
 """);
 demoChoices(loadBalancer);

 System.out.println("Let's reinstate the recommendation service.");
 paramHelper.put(paramHelper.tableName, paramHelper.dyntable);

 System.out.println("""
 Let's also substitute bad credentials for one of the instances in
the target group so that it can't
 access the DynamoDB recommendation table. We will get an instance id
value.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 AutoScaler autoScaler = new AutoScaler();

 // Create a new instance profile based on badCredsProfileName.
 templateCreator.createInstanceProfile(policyFile, policyName,
badCredsProfileName, roleName);
 String badInstanceId = autoScaler.getBadInstance(autoScalingGroupName);
 System.out.println("The bad instance id values used for this demo is " +
badInstanceId);

 String profileAssociationId = autoScaler.getInstanceProfile(badInstanceId);
 System.out.println("The association Id value is " + profileAssociationId);
 System.out.println("Replacing the profile for instance " + badInstanceId
 + " with a profile that contains bad credentials");
 autoScaler.replaceInstanceProfile(badInstanceId, badCredsProfileName,
profileAssociationId);

 System.out.println(
 """
 Now, sending a GET request to the load balancer endpoint
returns either a recommendation or a static response,
 depending on which instance is selected by the load
balancer.
 """);

```

```
demoChoices(loadBalancer);

System.out.println("""
 Let's implement a deep health check. For this demo, a deep health
check tests whether
 the web service can access the DynamoDB table that it depends on for
recommendations. Note that
 the deep health check is only for ELB routing and not for Auto
Scaling instance health.
 This kind of deep health check is not recommended for Auto Scaling
instance health, because it
 risks accidental termination of all instances in the Auto Scaling
group when a dependent service fails.
 """);

System.out.println("""
 By implementing deep health checks, the load balancer can detect
when one of the instances is failing
 and take that instance out of rotation.
 """);

paramHelper.put(paramHelper.healthCheck, "deep");

System.out.println("""
 Now, checking target health indicates that the instance with bad
credentials
 is unhealthy. Note that it might take a minute or two for the load
balancer to detect the unhealthy
 instance. Sending a GET request to the load balancer endpoint always
returns a recommendation, because
 the load balancer takes unhealthy instances out of its rotation.
 """);

demoChoices(loadBalancer);

System.out.println(
 """
 Because the instances in this demo are controlled by an auto
scaler, the simplest way to fix an unhealthy
 instance is to terminate it and let the auto scaler start a
new instance to replace it.
 """);
autoScaler.terminateInstance(badInstanceId);
```

```

 System.out.println("""
 Even while the instance is terminating and the new instance is
starting, sending a GET
 request to the web service continues to get a successful
recommendation response because
 the load balancer routes requests to the healthy instances. After
the replacement instance
 starts and reports as healthy, it is included in the load balancing
rotation.

 Note that terminating and replacing an instance typically takes
several minutes, during which time you
 can see the changing health check status until the new instance is
running and healthy.
 """);

 demoChoices(loadBalancer);
 System.out.println(
 "If the recommendation service fails now, deep health checks mean
all instances report as unhealthy.");
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 demoChoices(loadBalancer);
 paramHelper.reset();
 }

 public static void demoChoices(LoadBalancer loadBalancer) throws IOException,
InterruptedException {
 String[] actions = {
 "Send a GET request to the load balancer endpoint.",
 "Check the health of load balancer targets.",
 "Go to the next part of the demo."
 };

 Scanner scanner = new Scanner(System.in);

 while (true) {
 System.out.println("-".repeat(88));
 System.out.println("See the current state of the service by selecting
one of the following choices:");
 for (int i = 0; i < actions.length; i++) {
 System.out.println(i + ": " + actions[i]);
 }

 try {

```

```

 System.out.print("\nWhich action would you like to take? ");
 int choice = scanner.nextInt();
 System.out.println("-".repeat(88));

 switch (choice) {
 case 0 -> {
 System.out.println("Request:\n");
 System.out.println("GET http://" +
loadBalancer.getEndpoint(lbName));
 CloseableHttpClient httpClient =
HttpClient.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" +
loadBalancer.getEndpoint(lbName));

 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);

 // Display the JSON response
 BufferedReader reader = new BufferedReader(
 new
InputStreamReader(response.getEntity().getContent()));
 StringBuilder jsonResponse = new StringBuilder();
 String line;
 while ((line = reader.readLine()) != null) {
 jsonResponse.append(line);
 }
 reader.close();

 // Print the formatted JSON response.
 System.out.println("Full Response:\n");
 System.out.println(jsonResponse.toString());

 // Close the HTTP client.
 httpClient.close();
 }
 case 1 -> {
 System.out.println("\nChecking the health of load balancer
targets:\n");

```



```

 List<TargetHealthDescription> health =
loadBalancer.checkTargetHealth(targetGroupName);
 for (TargetHealthDescription target : health) {
 System.out.printf("\tTarget %s on port %d is %s\n",
target.target().id(),
 target.target().port(),
target.targetHealth().stateAsString());
 }
 System.out.println("""
 Note that it can take a minute or two for the health
check to update
 after changes are made.
 """);
 }
 case 2 -> {
 System.out.println("\nOkay, let's move on.");
 System.out.println("-".repeat(88));
 return; // Exit the method when choice is 2
 }
 default -> System.out.println("You must choose a value between
0-2. Please select again.");
}

 } catch (java.util.InputMismatchException e) {
 System.out.println("Invalid input. Please select again.");
 scanner.nextLine(); // Clear the input buffer.
 }
}

}

public static String readFileAsString(String filePath) throws IOException {
 byte[] bytes = Files.readAllBytes(Paths.get(filePath));
 return new String(bytes);
}
}

```

Create a class that wraps Auto Scaling and Amazon EC2 actions.

```

public class AutoScaler {

 private static Ec2Client ec2Client;
 private static AutoScalingClient autoScalingClient;

```

```
private static IamClient iamClient;

private static SsmClient ssmClient;

private IamClient getIAMClient() {
 if (iamClient == null) {
 iamClient = IamClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return iamClient;
}

private SsmClient getSSMClient() {
 if (ssmClient == null) {
 ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ssmClient;
}

private Ec2Client getEc2Client() {
 if (ec2Client == null) {
 ec2Client = Ec2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ec2Client;
}

private AutoScalingClient getAutoScalingClient() {
 if (autoScalingClient == null) {
 autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return autoScalingClient;
}

/**
 * Terminates and instances in an EC2 Auto Scaling group. After an instance is
 * terminated, it can no longer be accessed.
 */
```

```

 public void terminateInstance(String instanceId) {
 TerminateInstanceInAutoScalingGroupRequest terminateInstanceRequest =
 TerminateInstanceInAutoScalingGroupRequest
 .builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 getAutoScalingClient().terminateInstanceInAutoScalingGroup(terminateInstanceRequest);
 System.out.format("Terminated instance %s.", instanceId);
 }

 /**
 * Replaces the profile associated with a running instance. After the profile is
 * replaced, the instance is rebooted to ensure that it uses the new profile.
 * When
 * the instance is ready, Systems Manager is used to restart the Python web
 * server.
 */
 public void replaceInstanceProfile(String instanceId, String
 newInstanceProfileName, String profileAssociationId)
 throws InterruptedException {
 // Create an IAM instance profile specification.
 software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
 iamInstanceProfile =
 software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
 .builder()
 .name(newInstanceProfileName) // Make sure 'newInstanceProfileName'
 is a valid IAM Instance Profile
 // name.

 .build();

 // Replace the IAM instance profile association for the EC2 instance.
 ReplaceIamInstanceProfileAssociationRequest replaceRequest =
 ReplaceIamInstanceProfileAssociationRequest
 .builder()
 .iamInstanceProfile(iamInstanceProfile)
 .associationId(profileAssociationId) // Make sure
 'profileAssociationId' is a valid association ID.
 .build();

 try {
 getEc2Client().replaceIamInstanceProfileAssociation(replaceRequest);

```

```

 // Handle the response as needed.
 } catch (Ec2Exception e) {
 // Handle exceptions, log, or report the error.
 System.err.println("Error: " + e.getMessage());
 }

 System.out.format("Replaced instance profile for association %s with profile
%s.", profileAssociationId,
 newInstanceProfileName);
 TimeUnit.SECONDS.sleep(15);
 boolean instReady = false;
 int tries = 0;

 // Reboot after 60 seconds
 while (!instReady) {
 if (tries % 6 == 0) {
 getEc2Client().rebootInstances(RebootInstancesRequest.builder()
 .instanceIds(instanceId)
 .build());
 System.out.println("Rebooting instance " + instanceId + " and
waiting for it to be ready.");
 }
 tries++;
 try {
 TimeUnit.SECONDS.sleep(10);
 } catch (InterruptedException e) {
 e.printStackTrace();
 }

 DescribeInstanceInformationResponse informationResponse =
getSSMClient().describeInstanceInformation();
 List<InstanceInformation> instanceInformationList =
informationResponse.getInstanceInformationList();
 for (InstanceInformation info : instanceInformationList) {
 if (info.getInstanceId().equals(instanceId)) {
 instReady = true;
 break;
 }
 }
 }

 SendCommandRequest sendCommandRequest = SendCommandRequest.builder()
 .instanceIds(instanceId)
 .documentName("AWS-RunShellScript")
 .parameters(Collections.singletonMap("commands",

```

```

 Collections.singletonList("cd / && sudo python3 server.py
80"))))
 .build();

 getSSMClient().sendCommand(sendCommandRequest);
 System.out.println("Restarted the Python web server on instance " +
instanceId + ".");
 }

 public void openInboundPort(String secGroupId, String port, String ipAddress) {
 AuthorizeSecurityGroupIngressRequest ingressRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(secGroupId)
 .cidrIp(ipAddress)
 .fromPort(Integer.parseInt(port))
 .build();

 getEc2Client().authorizeSecurityGroupIngress(ingressRequest);
 System.out.format("Authorized ingress to %s on port %s from %s.",
secGroupId, port, ipAddress);
 }

 /**
 * Detaches a role from an instance profile, detaches policies from the role,
 * and deletes all the resources.
 */
 public void deleteInstanceProfile(String roleName, String profileName) {
 try {
 software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
getInstanceProfileRequest =
software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
 .builder()
 .instanceProfileName(profileName)
 .build();

 GetInstanceProfileResponse response =
getIAMClient().getInstanceProfile(getInstanceProfileRequest);
 String name = response.getInstanceProfile().getInstanceProfileName();
 System.out.println(name);

 RemoveRoleFromInstanceProfileRequest profileRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .roleName(roleName)

```

```

 .build();

 getIAMClient().removeRoleFromInstanceProfile(profileRequest);
 DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .build();

 getIAMClient().deleteInstanceProfile(deleteInstanceProfileRequest);
 System.out.println("Deleted instance profile " + profileName);

 DeleteRoleRequest deleteRoleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 // List attached role policies.
 ListAttachedRolePoliciesResponse rolesResponse = getIAMClient()
 .listAttachedRolePolicies(role -> role.roleName(roleName));
 List<AttachedPolicy> attachedPolicies =
rolesResponse.attachedPolicies();
 for (AttachedPolicy attachedPolicy : attachedPolicies) {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(attachedPolicy.policyArn())
 .build();

 getIAMClient().detachRolePolicy(request);
 System.out.println("Detached and deleted policy " +
attachedPolicy.policyName());
 }

 getIAMClient().deleteRole(deleteRoleRequest);
 System.out.println("Instance profile and role deleted.");

 } catch (IamException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public void deleteTemplate(String templateName) {
 getEc2Client().deleteLaunchTemplate(name ->
name.launchTemplateName(templateName));
 System.out.format(templateName + " was deleted.");
}

```

```

 }

 public void deleteAutoScaleGroup(String groupName) {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

getAutoScalingClient().deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println(groupName + " was deleted.");
 }

 /*
 * Verify the default security group of the specified VPC allows ingress from
 * this
 * computer. This can be done by allowing ingress from this computer's IP
 * address. In some situations, such as connecting from a corporate network, you
 * must instead specify a prefix list ID. You can also temporarily open the port
 * to
 * any IP address while running this example. If you do, be sure to remove
 * public
 * access when you're done.
 *
 */
 public GroupInfo verifyInboundPort(String VPC, int port, String ipAddress) {
 boolean portIsOpen = false;
 GroupInfo groupInfo = new GroupInfo();
 try {
 Filter filter = Filter.builder()
 .name("group-name")
 .values("default")
 .build();

 Filter filter1 = Filter.builder()
 .name("vpc-id")
 .values(VPC)
 .build();

 DescribeSecurityGroupsRequest securityGroupsRequest =
DescribeSecurityGroupsRequest.builder()
 .filters(filter, filter1)
 .build();

```

```

 DescribeSecurityGroupsResponse securityGroupsResponse = getEc2Client()
 .describeSecurityGroups(securityGroupsRequest);
 String securityGroup =
securityGroupsResponse.securityGroups().get(0).groupName();
 groupInfo.setGroupName(securityGroup);

 for (SecurityGroup secGroup : securityGroupsResponse.securityGroups()) {
 System.out.println("Found security group: " + secGroup.groupId());

 for (IpPermission ipPermission : secGroup.ipPermissions()) {
 if (ipPermission.fromPort() == port) {
 System.out.println("Found inbound rule: " + ipPermission);
 for (IpRange ipRange : ipPermission.ipRanges()) {
 String cidrIp = ipRange.cidrIp();
 if (cidrIp.startsWith(ipAddress) ||
cidrIp.equals("0.0.0.0/0")) {
 System.out.println(cidrIp + " is applicable");
 portIsOpen = true;
 }
 }

 if (!ipPermission.prefixListIds().isEmpty()) {
 System.out.println("Prefix lList is applicable");
 portIsOpen = true;
 }

 if (!portIsOpen) {
 System.out
 .println("The inbound rule does not appear to be
open to either this computer's IP,"
 + " all IP addresses (0.0.0.0/0), or to
a prefix list ID.");
 } else {
 break;
 }
 }
 }
 }

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
}

```



```

 groupInfo.setPortOpen(portIsOpen);
 return groupInfo;
 }

 /**
 * Attaches an Elastic Load Balancing (ELB) target group to this EC2 Auto
 * Scaling group.
 * The target group specifies how the load balancer forward requests to the
 * instances
 * in the group.
 */
 public void attachLoadBalancerTargetGroup(String asGroupName, String
targetGroupARN) {
 try {
 AttachLoadBalancerTargetGroupsRequest targetGroupsRequest =
AttachLoadBalancerTargetGroupsRequest.builder()
 .autoScalingGroupName(asGroupName)
 .targetGroupARNs(targetGroupARN)
 .build();

 getAutoScalingClient().attachLoadBalancerTargetGroups(targetGroupsRequest);
 System.out.println("Attached load balancer to " + asGroupName);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 // Creates an EC2 Auto Scaling group with the specified size.
 public String[] createGroup(int groupSize, String templateName, String
autoScalingGroupName) {

 // Get availability zones.
 software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
zonesRequest =
software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
 .builder()
 .build();

 DescribeAvailabilityZonesResponse zonesResponse =
getEc2Client().describeAvailabilityZones(zonesRequest);
 }

```

```

 List<String> availabilityZoneNames =
zonesResponse.availabilityZones().stream()

.map(software.amazon.awssdk.services.ec2.model.AvailabilityZone::zoneName)
 .collect(Collectors.toList());

 String availabilityZones = String.join(",", availabilityZoneNames);
 LaunchTemplateSpecification specification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(templateName)
 .version("$Default")
 .build();

 String[] zones = availabilityZones.split(",");
 CreateAutoScalingGroupRequest groupRequest =
CreateAutoScalingGroupRequest.builder()
 .launchTemplate(specification)
 .availabilityZones(zones)
 .maxSize(groupSize)
 .minSize(groupSize)
 .autoScalingGroupName(autoScalingGroupName)
 .build();

 try {
 getAutoScalingClient().createAutoScalingGroup(groupRequest);

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Created an EC2 Auto Scaling group named " +
autoScalingGroupName);
 return zones;
 }

 public String getDefaultVPC() {
 // Define the filter.
 Filter defaultFilter = Filter.builder()
 .name("is-default")
 .values("true")
 .build();

 software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest request =
software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest

```

```

 .builder()
 .filters(defaultFilter)
 .build();

 DescribeVpcsResponse response = getEc2Client().describeVpcs(request);
 return response.vpcs().get(0).vpcId();
}

// Gets the default subnets in a VPC for a specified list of Availability Zones.
public List<Subnet> getSubnets(String vpcId, String[] availabilityZones) {
 List<Subnet> subnets = null;
 Filter vpcFilter = Filter.builder()
 .name("vpc-id")
 .values(vpcId)
 .build();

 Filter azFilter = Filter.builder()
 .name("availability-zone")
 .values(availabilityZones)
 .build();

 Filter defaultForAZ = Filter.builder()
 .name("default-for-az")
 .values("true")
 .build();

 DescribeSubnetsRequest request = DescribeSubnetsRequest.builder()
 .filters(vpcFilter, azFilter, defaultForAZ)
 .build();

 DescribeSubnetsResponse response = getEc2Client().describeSubnets(request);
 subnets = response.subnets();
 return subnets;
}

// Gets data about the instances in the EC2 Auto Scaling group.
public String getBadInstance(String groupName) {
 DescribeAutoScalingGroupsRequest request =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response =
getAutoScalingClient().describeAutoScalingGroups(request);

```

```

 AutoScalingGroup autoScalingGroup = response.autoScalingGroups().get(0);
 List<String> instanceIds = autoScalingGroup.instances().stream()
 .map(instance -> instance.instanceId())
 .collect(Collectors.toList());

 String[] instanceIdArray = instanceIds.toArray(new String[0]);
 for (String instanceId : instanceIdArray) {
 System.out.println("Instance ID: " + instanceId);
 return instanceId;
 }
 return "";
 }

 // Gets data about the profile associated with an instance.
 public String getInstanceProfile(String instanceId) {
 Filter filter = Filter.builder()
 .name("instance-id")
 .values(instanceId)
 .build();

 DescribeIamInstanceProfileAssociationsRequest associationsRequest =
 DescribeIamInstanceProfileAssociationsRequest
 .builder()
 .filters(filter)
 .build();

 DescribeIamInstanceProfileAssociationsResponse response = getEc2Client()
 .describeIamInstanceProfileAssociations(associationsRequest);
 return response.iamInstanceProfileAssociations().get(0).associationId();
 }

 public void deleteRolesPolicies(String policyName, String roleName, String
 InstanceProfile) {
 ListPoliciesRequest listPoliciesRequest =
 ListPoliciesRequest.builder().build();
 ListPoliciesResponse listPoliciesResponse =
 getIAMClient().listPolicies(listPoliciesRequest);
 for (Policy policy : listPoliciesResponse.policies()) {
 if (policy.policyName().equals(policyName)) {
 // List the entities (users, groups, roles) that are attached to the
 policy.

 software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest

```

```

listEntitiesRequest =
software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
 .builder()
 .policyArn(policy.arn())
 .build();
ListEntitiesForPolicyResponse listEntitiesResponse = iamClient
 .listEntitiesForPolicy(listEntitiesRequest);
if (!listEntitiesResponse.policyGroups().isEmpty() || !
listEntitiesResponse.policyUsers().isEmpty()
 || !listEntitiesResponse.policyRoles().isEmpty()) {
 // Detach the policy from any entities it is attached to.
 DetachRolePolicyRequest detachPolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy.arn())
 .roleName(roleName) // Specify the name of the IAM role
 .build();

 getIAMClient().detachRolePolicy(detachPolicyRequest);
 System.out.println("Policy detached from entities.");
}

// Now, you can delete the policy.
DeletePolicyRequest deletePolicyRequest =
DeletePolicyRequest.builder()
 .policyArn(policy.arn())
 .build();

getIAMClient().deletePolicy(deletePolicyRequest);
System.out.println("Policy deleted successfully.");
break;
}
}

// List the roles associated with the instance profile
ListInstanceProfilesForRoleRequest listRolesRequest =
ListInstanceProfilesForRoleRequest.builder()
 .roleName(roleName)
 .build();

// Detach the roles from the instance profile
ListInstanceProfilesForRoleResponse listRolesResponse =
iamClient.listInstanceProfilesForRole(listRolesRequest);
for (software.amazon.awssdk.services.iam.model.InstanceProfile profile :
listRolesResponse.instanceProfiles()) {

```

```

 RemoveRoleFromInstanceProfileRequest removeRoleRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .roleName(roleName) // Remove the extra dot here
 .build();

 getIAMClient().removeRoleFromInstanceProfile(removeRoleRequest);
 System.out.println("Role " + roleName + " removed from instance profile
" + InstanceProfile);
 }

 // Delete the instance profile after removing all roles
 DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .build();

 getIAMClient().deleteInstanceProfile(r ->
r.instanceProfileName(InstanceProfile));
 System.out.println(InstanceProfile + " Deleted");
 System.out.println("All roles and policies are deleted.");
}
}

```

Create a class that wraps Elastic Load Balancing actions.

```

public class LoadBalancer {
 public ElasticLoadBalancingV2Client elasticLoadBalancingV2Client;

 public ElasticLoadBalancingV2Client getLoadBalancerClient() {
 if (elasticLoadBalancingV2Client == null) {
 elasticLoadBalancingV2Client = ElasticLoadBalancingV2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }

 return elasticLoadBalancingV2Client;
 }

 // Checks the health of the instances in the target group.
 public List<TargetHealthDescription> checkTargetHealth(String targetGroupName) {

```

```

 DescribeTargetGroupsRequest targetGroupsRequest =
DescribeTargetGroupsRequest.builder()
 .names(targetGroupName)
 .build();

 DescribeTargetGroupsResponse tgResponse =
getLoadBalancerClient().describeTargetGroups(targetGroupsRequest);

 DescribeTargetHealthRequest healthRequest =
DescribeTargetHealthRequest.builder()
 .targetGroupArn(tgResponse.targetGroups().get(0).targetGroupArn())
 .build();

 DescribeTargetHealthResponse healthResponse =
getLoadBalancerClient().describeTargetHealth(healthRequest);
 return healthResponse.targetHealthDescriptions();
 }

 // Gets the HTTP endpoint of the load balancer.
 public String getEndpoint(String lbName) {
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 return res.loadBalancers().get(0).dnsName();
 }

 // Deletes a load balancer.
 public void deleteLoadBalancer(String lbName) {
 try {
 // Use a waiter to delete the Load Balancer.
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(res.loadBalancers().get(0).loadBalancerArn())
 .build();

 getLoadBalancerClient().deleteLoadBalancer(
 builder ->
builder.loadBalancerArn(res.loadBalancers().get(0).loadBalancerArn()));
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancersDeleted(request);

```

```

 waiterResponse.matched().response().ifPresent(System.out::println);

 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(lbName + " was deleted.");
}

// Deletes the target group.
public void deleteTargetGroup(String targetGroupName) {
 try {
 DescribeTargetGroupsResponse res = getLoadBalancerClient()
 .describeTargetGroups(describe ->
describe.names(targetGroupName));
 getLoadBalancerClient()
 .deleteTargetGroup(builder ->
builder.targetGroupArn(res.targetGroups().get(0).targetGroupArn()));
 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(targetGroupName + " was deleted.");
}

// Verify this computer can successfully send a GET request to the load balancer
// endpoint.
public boolean verifyLoadBalancerEndpoint(String elbDnsName) throws IOException,
InterruptedException {
 boolean success = false;
 int retries = 3;
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" + elbDnsName);
 try {
 while ((!success) && (retries > 0)) {
 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);
 if (statusCode == 200) {
 success = true;
 } else {
 retries--;
 }
 }
 }
}

```



```

 System.out.println("Got connection error from load balancer
endpoint, retrying...");
 TimeUnit.SECONDS.sleep(15);
 }
}

} catch (org.apache.http.conn.HttpHostConnectException e) {
 System.out.println(e.getMessage());
}

System.out.println("Status.." + success);
return success;
}

/*
 * Creates an Elastic Load Balancing target group. The target group specifies
 * how
 * the load balancer forward requests to instances in the group and how instance
 * health is checked.
 */
public String createTargetGroup(String protocol, int port, String vpcId, String
targetGroupName) {
 CreateTargetGroupRequest targetGroupRequest =
CreateTargetGroupRequest.builder()
 .healthCheckPath("/healthcheck")
 .healthCheckTimeoutSeconds(5)
 .port(port)
 .vpcId(vpcId)
 .name(targetGroupName)
 .protocol(protocol)
 .build();

 CreateTargetGroupResponse targetGroupResponse =
getLoadBalancerClient().createTargetGroup(targetGroupRequest);
 String targetGroupArn =
targetGroupResponse.targetGroups().get(0).targetGroupArn();
 String targetGroup =
targetGroupResponse.targetGroups().get(0).targetGroupName();
 System.out.println("The " + targetGroup + " was created with ARN" +
targetGroupArn);
 return targetGroupArn;
}

/*

```

```

 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */
 public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
String lbName, int port,
 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("*** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.

```

```

 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

 .loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}
}

```

Create a class that uses DynamoDB to simulate a recommendation service.

```

public class Database {

 private static DynamoDbClient dynamoDbClient;

 public static DynamoDbClient getDynamoDbClient() {
 if (dynamoDbClient == null) {
 dynamoDbClient = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return dynamoDbClient;
 }
}

```

```

// Checks to see if the Amazon DynamoDB table exists.
private boolean doesTableExist(String tableName) {
 try {
 // Describe the table and catch any exceptions.
 DescribeTableRequest describeTableRequest =
DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 getDynamoDbClient().describeTable(describeTableRequest);
 System.out.println("Table '" + tableName + "' exists.");
 return true;

 } catch (ResourceNotFoundException e) {
 System.out.println("Table '" + tableName + "' does not exist.");
 } catch (DynamoDbException e) {
 System.err.println("Error checking table existence: " + e.getMessage());
 }
 return false;
}

/*
 * Creates a DynamoDB table to use a recommendation service. The table has a
 * hash key named 'MediaType' that defines the type of media recommended, such
 * as
 * Book or Movie, and a range key named 'ItemId' that, combined with the
 * MediaType,
 * forms a unique identifier for the recommended item.
 */
public void createTable(String tableName, String fileName) throws IOException {
 // First check to see if the table exists.
 boolean doesExist = doesTableExist(tableName);
 if (!doesExist) {
 DynamoDbWaiter dbWaiter = getDynamoDbClient().waiter();
 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("MediaType")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("ItemId")

```

```

 .attributeType(ScalarAttributeType.N)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("MediaType")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("ItemId")
 .keyType(KeyType.RANGE)
 .build())
 .provisionedThroughput(
 ProvisionedThroughput.builder()
 .readCapacityUnits(5L)
 .writeCapacityUnits(5L)
 .build())
 .build();

getDynamoDbClient().createTable(createTableRequest);
System.out.println("Creating table " + tableName + "...");

// Wait until the Amazon DynamoDB table is created.
DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
waiterResponse.matched().response().ifPresent(System.out::println);
System.out.println("Table " + tableName + " created.");

// Add records to the table.
populateTable(fileName, tableName);
}
}

public void deleteTable(String tableName) {
 getDynamoDbClient().deleteTable(table -> table.tableName(tableName));
 System.out.println("Table " + tableName + " deleted.");
}

// Populates the table with data located in a JSON file using the DynamoDB
// enhanced client.

```

```

public void populateTable(String fileName, String tableName) throws IOException
{
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(getDynamoDbClient())
 .build();
 ObjectMapper objectMapper = new ObjectMapper();
 File jsonFile = new File(fileName);
 JsonNode rootNode = objectMapper.readTree(jsonFile);

 DynamoDbTable<Recommendation> mappedTable = enhancedClient.table(tableName,
 TableSchema.fromBean(Recommendation.class));
 for (JsonNode currentNode : rootNode) {
 String mediaType = currentNode.path("MediaType").path("S").asText();
 int itemId = currentNode.path("ItemId").path("N").asInt();
 String title = currentNode.path("Title").path("S").asText();
 String creator = currentNode.path("Creator").path("S").asText();

 // Create a Recommendation object and set its properties.
 Recommendation rec = new Recommendation();
 rec.setMediaType(mediaType);
 rec.setItemId(itemId);
 rec.setTitle(title);
 rec.setCreator(creator);

 // Put the item into the DynamoDB table.
 mappedTable.putItem(rec); // Add the Recommendation to the list.
 }
 System.out.println("Added all records to the " + tableName);
}
}

```

Create a class that wraps Systems Manager actions.

```

public class ParameterHelper {

 String tableName = "doc-example-resilient-architecture-table";
 String dyntable = "doc-example-recommendation-service";
 String failureResponse = "doc-example-resilient-architecture-failure-response";
 String healthCheck = "doc-example-resilient-architecture-health-check";

 public void reset() {
 put(dyntable, tableName);
 }
}

```

```
 put(failureResponse, "none");
 put(healthCheck, "shallow");
 }

 public void put(String name, String value) {
 SsmClient ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();

 PutParameterRequest parameterRequest = PutParameterRequest.builder()
 .name(name)
 .value(value)
 .overwrite(true)
 .type("String")
 .build();

 ssmClient.putParameter(parameterRequest);
 System.out.printf("Setting demo parameter %s to '%s'.", name, value);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [AttachLoadBalancerTargetGroups](#)
- [CreateAutoScalingGroup](#)
- [CreateInstanceProfile](#)
- [CreateLaunchTemplate](#)
- [CreateListener](#)
- [CreateLoadBalancer](#)
- [CreateTargetGroup](#)
- [DeleteAutoScalingGroup](#)
- [DeleteInstanceProfile](#)
- [DeleteLaunchTemplate](#)
- [DeleteLoadBalancer](#)
- [DeleteTargetGroup](#)
- [DescribeAutoScalingGroups](#)
- [DescribeAvailabilityZones](#)

- [DescribeIamInstanceProfileAssociations](#)
- [DescribeInstances](#)
- [DescribeLoadBalancers](#)
- [DescribeSubnets](#)
- [DescribeTargetGroups](#)
- [DescribeTargetHealth](#)
- [DescribeVpcs](#)
- [RebootInstances](#)
- [ReplaceIamInstanceProfileAssociation](#)
- [TerminateInstanceInAutoScalingGroup](#)
- [UpdateAutoScalingGroup](#)

## AWS Batch examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Batch.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello AWS Batch

The following code example shows how to get started using AWS Batch.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryPolicy;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.batch.BatchAsyncClient;
import software.amazon.awssdk.services.batch.model.JobStatus;
import software.amazon.awssdk.services.batch.model.JobSummary;
import software.amazon.awssdk.services.batch.model.ListJobsRequest;
import software.amazon.awssdk.services.batch.paginators.ListJobsPublisher;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class HelloBatch {
 private static BatchAsyncClient batchClient;

 public static void main(String[] args) {
 List<JobSummary> jobs = listJobs("my-job-queue");
 jobs.forEach(job ->
 System.out.printf("Job ID: %s, Job Name: %s, Job Status: %s%n",
 job.jobId(), job.jobName(), job.status())
);
 }

 public static List<JobSummary> listJobs(String jobQueue) {
 if (jobQueue == null || jobQueue.isEmpty()) {
 throw new IllegalArgumentException("Job queue cannot be null or empty");
 }

 ListJobsRequest listJobsRequest = ListJobsRequest.builder()
 .jobQueue(jobQueue)
 .jobStatus(JobStatus.SUCCEEDED)
```

```

 .build();

 List<JobSummary> jobSummaries = new ArrayList<>();
 ListJobsPublisher listJobsPaginator =
getAsyncClient().listJobsPaginator(listJobsRequest);
 CompletableFuture<Void> future = listJobsPaginator.subscribe(response -> {
 jobSummaries.addAll(response.jobSummaryList());
 });

 future.join();
 return jobSummaries;
 }

 private static BatchAsyncClient getAsyncClient() {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100) // Increase max concurrency to handle more
simultaneous connections.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.
 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.
 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the individual
call attempt timeout.
 .retryPolicy(RetryPolicy.builder() // Add a retry policy to handle
transient errors.
 .numRetries(3) // Number of retry attempts.
 .build())
 .build();

 if (batchClient == null) {
 batchClient = BatchAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return batchClient;
 }
}

```

```
}
```

- For API details, see [listJobsPaginator](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an AWS Batch compute environment.
- Check the status of the compute environment.
- Set up an AWS Batch job queue and job definition.
- Register a job definition.
- Submit an AWS Batch Job.
- Get a list of jobs applicable to the job queue.
- Check the status of job.
- Delete AWS Batch resources.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating AWS Batch features.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.batch.model.BatchException;
import software.amazon.awssdk.services.batch.model.ClientException;
import software.amazon.awssdk.services.batch.model.CreateComputeEnvironmentResponse;
import software.amazon.awssdk.services.batch.model.JobSummary;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsRequest;
```

```

import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeSubnetsRequest;
import software.amazon.awssdk.services.ec2.model.DescribeSubnetsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest;
import software.amazon.awssdk.services.ec2.model.Filter;
import software.amazon.awssdk.services.ec2.model.SecurityGroup;
import software.amazon.awssdk.services.ec2.model.Subnet;
import software.amazon.awssdk.services.ec2.model.Vpc;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.List;
import java.util.Map;
import java.util.Scanner;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * NOTE
 * This scenario submits a job that pulls a Docker image named echo-text from Amazon
 * ECR to Amazon Fargate.
 *
 * To place this Docker image on Amazon ECR, run the following Basics scenario.
 *
 * https://github.com/awsdocs/aws-doc-sdk-examples/tree/main/javav2/example_code/ecr
 */
public class BatchScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 // Define two stacks used in this Basics Scenario.
 private static final String ROLES_STACK = "RolesStack";
 private static String defaultSubnet;
 private static String defaultSecurityGroup;

 private static final Logger logger =
 LoggerFactory.getLogger(BatchScenario.class);

```

```

public static void main(String[] args) throws InterruptedException {

 BatchActions batchActions = new BatchActions();
 Scanner scanner = new Scanner(System.in);
 String computeEnvironmentName = "my-compute-environment";
 String jobQueueName = "my-job-queue";
 String jobDefinitionName = "my-job-definition";

 // See the NOTE in this Java code example (at start).
 String dockerImage = "dkr.ecr.us-east-1.amazonaws.com/echo-text:echo-text";

 logger.info("""
 AWS Batch is a fully managed batch processing service that dynamically
provisions the required compute
 resources for batch computing workloads. The Java V2 `BatchAsyncClient`
allows
 developers to automate the submission, monitoring, and management of
batch jobs.

 This scenario provides an example of setting up a compute environment,
job queue and job definition,
 and then submitting a job.

 This scenario submits a job that pulls a Docker image named echo-text
from Amazon ECR to Amazon Fargate.

 To place this Docker image on Amazon ECR, run the following Basics
scenario.

 https://github.com/awsdocs/aws-doc-sdk-examples/tree/main/javav2/
example_code/ecr

 Let's get started...

 You have two choices:

 1 - Run the entire program.
 2 - Delete an existing Compute Environment (created from a previous
execution of
 this program that did not complete).
 """);

```

```

 while (true) {
 String input = scanner.nextLine();
 if (input.trim().equalsIgnoreCase("1")) {
 logger.info("Continuing with the program...");
 // logger.info("");
 break;
 } else if (input.trim().equalsIgnoreCase("2")) {
 String jobQueueARN = String.valueOf(batchActions.
describeJobQueueAsync(computeEnvironmentName));
 if (!jobQueueARN.isEmpty()) {
 batchActions.disableJobQueueAsync(jobQueueARN);
 countdown(1);
 batchActions.deleteJobQueueAsync(jobQueueARN);
 }

 try {

batchActions.disableComputeEnvironmentAsync(computeEnvironmentName)
 .exceptionally(ex -> {
 logger.info("Disable compute environment failed: " +
ex.getMessage());
 return null;
 })
 .join();
 } catch (CompletionException ex) {
 logger.info("Failed to disable compute environment: " +
ex.getMessage());
 }
 countdown(2);

batchActions.deleteComputeEnvironmentAsync(computeEnvironmentName).join();
 return;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
 System.out.println(DASHES);

 waitForInputToContinue(scanner);
 // Get an AWS Account id used to retrieve the docker image from Amazon ECR.
 // Create a single-element array to store the `accountId` value.
 String[] accId = new String[1];
 CompletableFuture<String> accountIdFuture = batchActions.getAccountId();

```

```

 accountIdFuture.thenAccept(accountId -> {
 logger.info("Account ID: " + accountId);
 accId[0] = accountId;
 }).join();

 dockerImage = accId[0]+"."+dockerImage;

 // Get a default subnet and default security associated with the default
VPC.
 getSubnetSecurityGroup();

 logger.info("Use AWS CloudFormation to create two IAM roles that are
required for this scenario.");
 CloudFormationHelper.deployCloudFormationStack(ROLES_STACK);

 Map<String, String> stackOutputs =
CloudFormationHelper.getStackOutputs(ROLES_STACK);
 String batchIAMRole = stackOutputs.get("BatchRoleArn");
 String executionRoleARN = stackOutputs.get("EcsRoleArn");

 logger.info("The IAM role needed to interact with AWS Batch is
"+batchIAMRole);
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("1. Create a Batch compute environment");
 logger.info("""
 A compute environment is a resource where you can run your batch jobs.
 After creating a compute environment, you can define job queues and job
definitions to submit jobs for
 execution.

 The benefit of creating a compute environment is it allows you to easily
configure and manage the compute
 resources that will be used to run your Batch jobs. By separating the
compute environment from the job definitions,
 you can easily scale your compute resources up or down as needed,
without having to modify your job definitions.
 This makes it easier to manage your Batch workloads and ensures that
your jobs have the necessary
 compute resources to run efficiently.
 """);

 waitForInputToContinue(scanner);

```

```

 try {
 CompletableFuture<CreateComputeEnvironmentResponse> future =
batchActions.createComputeEnvironmentAsync(computeEnvironmentName, batchIAMRole,
defaultSubnet, defaultSecurityGroup);
 CreateComputeEnvironmentResponse response = future.join();
 logger.info("Compute Environment ARN: " +
response.computeEnvironmentArn());
 } catch (RuntimeException rte) {
 Throwable cause = rte.getCause();
 if (cause instanceof ClientException batchExceptionEx) {
 String myErrorCode =
batchExceptionEx.awsErrorDetails().errorMessage();
 if ("Object already exists".contains(myErrorCode)) {
 logger.info("The compute environment '" + computeEnvironmentName
+ "' already exists. Moving on...");
 } else {
 logger.info("Batch error occurred: {} (Code: {})",
batchExceptionEx.getMessage(), batchExceptionEx.awsErrorDetails().errorCode());
 return;
 }
 } else {
 logger.info("An unexpected error occurred: {}", (cause != null ?
cause.getMessage() : rte.getMessage()));
 }
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("2. Check the status of the "+computeEnvironmentName +" Compute
Environment.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<String> future =
batchActions.checkComputeEnvironmentsStatus(computeEnvironmentName);
 String status = future.join();
 logger.info("Compute Environment Status: " + status);

 } catch (RuntimeException rte) {
 Throwable cause = rte.getCause();
 if (cause instanceof ClientException batchExceptionEx) {
 logger.info("Batch error occurred: {} (Code: {})",
batchExceptionEx.getMessage(), batchExceptionEx.awsErrorDetails().errorCode());
 return;
 }
 }
 }
}

```



```

 } else {
 logger.info("An unexpected error occurred: " + (cause != null ?
cause.getMessage() : rte.getMessage()));
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Create a job queue");
 logger.info("""
 A job queue is an essential component that helps manage the execution
of your batch jobs.
 It acts as a buffer, where jobs are placed and then scheduled for
execution based on their
 priority and the available resources in the compute environment.
 """);
 waitForInputToContinue(scanner);

 String jobQueueArn = null;
 try {
 CompletableFuture<String> jobQueueFuture =
batchActions.createJobQueueAsync(jobQueueName, computeEnvironmentName);
 jobQueueArn = jobQueueFuture.join();
 logger.info("Job Queue ARN: " + jobQueueArn);

 } catch (RuntimeException rte) {
 Throwable cause = rte.getCause();
 if (cause instanceof BatchException batchExceptionEx) {
 String myErrorCode =
batchExceptionEx.awsErrorDetails().errorMessage();
 if ("Object already exists".contains(myErrorCode)) {
 logger.info("The job queue '" + jobQueueName + "' already
exists. Moving on...");
 // Retrieve the ARN of the job queue.
 CompletableFuture<String> jobQueueArnFuture =
batchActions.getJobQueueARN(jobQueueName);
 jobQueueArn = jobQueueArnFuture.join();
 logger.info("Job Queue ARN: " + jobQueueArn);
 } else {
 logger.info("Batch error occurred: {} (Code: {})",
batchExceptionEx.getMessage(), batchExceptionEx.awsErrorDetails().errorCode());
 return;
 }
 }
 }

```

```

 }
 } else {
 logger.info("An unexpected error occurred: " + (cause != null ?
cause.getMessage() : rte.getMessage()));
 return; // End the execution
 }
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("4. Register a Job Definition.");
logger.info("""
 Registering a job in AWS Batch using the Fargate launch type ensures
that all
 necessary parameters, such as the execution role, command to run, and so
on
 are specified and reused across multiple job submissions.

 The job definition pulls a Docker image from Amazon ECR and executes
the Docker image.
 """);

waitForInputToContinue(scanner);
String jobARN;
try {
 String platform = "";
 while (true) {
 logger.info("""
 On which platform/CPU architecture combination did you build the
Docker image?:

 1. Windows X86_64
 2. Mac or Linux ARM64
 3. Mac or Linux X86_64

 Please select 1, 2, or 3.
 """);
 String platAns = scanner.nextLine().trim();
 if (platAns.equals("1")) {
 platform = "X86_64";
 break; // Exit loop since a valid option is selected
 } else if (platAns.equals("2")) {
 platform = "ARM64";
 break; // Exit loop since a valid option is selected
 } else if (platAns.equals("3")) {

```

```

 platform = "X86_64";
 break; // Exit loop since a valid option is selected
 } else {
 System.out.println("Invalid input. Please select either 1 or
2.");
 }
}

jobARN = batchActions.registerJobDefinitionAsync(jobDefinitionName,
executionRoleARN, dockerImage, platform)
 .exceptionally(ex -> {
 System.err.println("Register job definition failed: " +
ex.getMessage());
 return null;
 })
 .join();
if (jobARN != null) {
 logger.info("Job ARN: " + jobARN);
}
} catch (RuntimeException rte) {
 logger.error("A Batch exception occurred while registering the job: {}",
rte.getCause() != null ? rte.getCause().getMessage() : rte.getMessage());
 return;
}
logger.info(DASHES);

logger.info(DASHES);
logger.info("5. Submit an AWS Batch job from a job definition.");
waitForInputToContinue(scanner);
String jobId;
try {
 jobId = batchActions.submitJobAsync(jobDefinitionName, jobQueueName,
jobARN)
 .exceptionally(ex -> {
 System.err.println("Submit job failed: " + ex.getMessage());
 return null;
 })
 .join();

 logger.info("The job id is "+jobId);
 logger.info("Let's wait 2 minutes for the job to complete");
 countdown(2);

} catch (RuntimeException rte) {

```

```
 logger.error("A Batch exception occurred while submitting the job: {}",
rte.getCause() != null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 logger.info(DASHES);
 logger.info("6. Get a list of jobs applicable to the job queue.");

 waitForInputToContinue(scanner);
 try {
 List<JobSummary> jobs = batchActions.listJobsAsync(jobQueueName);
 jobs.forEach(job ->
 logger.info("Job ID: {}, Job Name: {}, Job Status: {}", job.jobId(),
job.jobName(), job.status()));

 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred while submitting the job: {}",
rte.getCause() != null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Check the status of job "+jobId);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<String> future = batchActions.describeJobAsync(jobId);
 String jobStatus = future.join();
 logger.info("Job Status: " + jobStatus);

 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred while submitting the job: {}",
rte.getCause() != null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }

 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 logger.info("8. Delete Batch resources");
```

```

 logger.info(
 ""
 When deleting an AWS Batch compute environment, it does not happen
instantaneously.
 There is typically a delay, similar to some other AWS resources.
 AWS Batch starts the deletion process.
 "");
 logger.info("Would you like to delete the AWS Batch resources such as the
compute environment? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 logger.info("You selected to delete the AWS ECR resources.");
 logger.info("First, we will deregister the Job Definition.");
 waitForInputToContinue(scanner);
 try {
 batchActions.deregisterJobDefinitionAsync(jobARN)
 .exceptionally(ex -> {
 logger.info("Deregister job definition failed: " +
ex.getMessage());
 return null;
 })
 .join();
 logger.info(jobARN + " was deregistered");
 } catch (RuntimeException rte) {
 logger.error("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }

 logger.info("Second, we will disable and then delete the Job Queue.");
 waitForInputToContinue(scanner);
 try {
 batchActions.disableJobQueueAsync(jobQueueArn)
 .exceptionally(ex -> {
 logger.info("Disable job queue failed: " + ex.getMessage());
 return null;
 })
 .join();
 logger.info(jobQueueArn + " was disabled");
 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }
 }
 }

```

```

 batchActions.waitForJobQueueToBeDisabledAsync(jobQueueArn);
 try {
 CompletableFuture<Void> future =
batchActions.waitForJobQueueToBeDisabledAsync(jobQueueArn);
 future.join();
 logger.info("Job queue is now disabled.");
 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }

 waitForInputToContinue(scanner);
 try {
 batchActions.deleteJobQueueAsync(jobQueueArn);
 logger.info(jobQueueArn + " was deleted");
 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }
 logger.info("Let's wait 2 minutes for the job queue to be deleted");
 countdown(2);
 waitForInputToContinue(scanner);

 logger.info("Third, we will delete the Compute Environment.");
 waitForInputToContinue(scanner);
 try {
 batchActions.disableComputeEnvironmentAsync(computeEnvironmentName)
 .exceptionally(ex -> {
 System.err.println("Disable compute environment failed: " +
ex.getMessage());
 return null;
 })
 .join();
 logger.info("Compute environment disabled") ;
 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }
 }

```

```

batchActions.checkComputeEnvironmentsStatus(computeEnvironmentName).thenAccept(state
-> {
 logger.info("Current State: " + state);
}).join();

 logger.info("Lets wait 1 min for the compute environment to be
deleted");
 countdown(1);

 try {

batchActions.deleteComputeEnvironmentAsync(computeEnvironmentName).join();
 logger.info(computeEnvironmentName + " was deleted.");

 } catch (RuntimeException rte) {
 logger.info("A Batch exception occurred: {}", rte.getCause() !=
null ? rte.getCause().getMessage() : rte.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 CloudFormationHelper.destroyCloudFormationStack(ROLES_STACK);
 }

 logger.info(DASHES);
 logger.info("This concludes the AWS Batch SDK scenario");
 logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}

```

```

 }

 public static void countdown(int minutes) throws InterruptedException {
 int seconds = 0;
 for (int i = minutes * 60 + seconds; i >= 0; i--) {
 int displayMinutes = i / 60;
 int displaySeconds = i % 60;
 System.out.print(String.format("\r%02d:%02d", displayMinutes,
displaySeconds));
 Thread.sleep(1000); // Wait for 1 second
 }
 logger.info("Countdown complete!");
 }

 private static void getSubnetSecurityGroup() {
 try (Ec2AsyncClient ec2Client = Ec2AsyncClient.create()) {
 CompletableFuture<Vpc> defaultVpcFuture =
ec2Client.describeVpcs(DescribeVpcsRequest.builder()
 .filters(Filter.builder()
 .name("is-default")
 .values("true")
 .build())
 .build())
 .thenApply(response -> response.vpcs().stream()
 .findFirst()
 .orElseThrow(() -> new RuntimeException("Default VPC not
found"))));

 CompletableFuture<String> defaultSubnetFuture = defaultVpcFuture
 .thenCompose(vpc ->
ec2Client.describeSubnets(DescribeSubnetsRequest.builder()
 .filters(Filter.builder()
 .name("vpc-id")
 .values(vpc.vpcId())
 .build(),
 Filter.builder()
 .name("default-for-az")
 .values("true")
 .build())
 .build())
 .thenApply(DescribeSubnetsResponse::subnets)
 .thenApply(subnets -> subnets.stream()
 .findFirst()
 .map(Subnet::subnetId)

```



```

 .orElseThrow(() -> new RuntimeException("No
default subnet found"))));

 CompletableFuture<String> defaultSecurityGroupFuture = defaultVpcFuture
 .thenCompose(vpc ->
ec2Client.describeSecurityGroups(DescribeSecurityGroupsRequest.builder()
 .filters(Filter.builder()
 .name("group-name")
 .values("default")
 .build(),
 Filter.builder()
 .name("vpc-id")
 .values(vpc.vpcId())
 .build())
 .build())

 .thenApply(DescribeSecurityGroupsResponse::securityGroups)
 .thenApply(securityGroups -> securityGroups.stream()
 .findFirst()
 .map(SecurityGroup::groupId)
 .orElseThrow(() -> new RuntimeException("No
default security group found"))));

 defaultSubnet = defaultSubnetFuture.join();
 defaultSecurityGroup = defaultSecurityGroupFuture.join();
 }
}
}

```

A wrapper class for AWS Batch SDK methods.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryPolicy;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.batch.BatchAsyncClient;
import software.amazon.awssdk.services.batch.BatchClient;
import software.amazon.awssdk.services.batch.model.AssignPublicIp;
import software.amazon.awssdk.services.batch.model.BatchException;

```

```
import software.amazon.awssdk.services.batch.model.CEState;
import software.amazon.awssdk.services.batch.model.CEType;
import software.amazon.awssdk.services.batch.model.CRType;
import software.amazon.awssdk.services.batch.model.ComputeEnvironmentOrder;
import software.amazon.awssdk.services.batch.model.ComputeResource;
import software.amazon.awssdk.services.batch.model.ContainerProperties;
import software.amazon.awssdk.services.batch.model.CreateComputeEnvironmentRequest;
import software.amazon.awssdk.services.batch.model.CreateComputeEnvironmentResponse;
import software.amazon.awssdk.services.batch.model.CreateJobQueueRequest;
import software.amazon.awssdk.services.batch.model.DeleteComputeEnvironmentRequest;
import software.amazon.awssdk.services.batch.model.DeleteComputeEnvironmentResponse;
import software.amazon.awssdk.services.batch.model.DeleteJobQueueRequest;
import software.amazon.awssdk.services.batch.model.DeleteJobQueueResponse;
import software.amazon.awssdk.services.batch.model.DeregisterJobDefinitionRequest;
import software.amazon.awssdk.services.batch.model.DeregisterJobDefinitionResponse;
import
 software.amazon.awssdk.services.batch.model.DescribeComputeEnvironmentsRequest;
import
 software.amazon.awssdk.services.batch.model.DescribeComputeEnvironmentsResponse;
import software.amazon.awssdk.services.batch.model.DescribeJobQueuesRequest;
import software.amazon.awssdk.services.batch.model.DescribeJobQueuesResponse;
import software.amazon.awssdk.services.batch.model.DescribeJobsRequest;
import software.amazon.awssdk.services.batch.model.DescribeJobsResponse;
import software.amazon.awssdk.services.batch.model.JQState;
import software.amazon.awssdk.services.batch.model.JobDefinitionType;
import software.amazon.awssdk.services.batch.model.JobDetail;
import software.amazon.awssdk.services.batch.model.JobQueueDetail;
import software.amazon.awssdk.services.batch.model.JobStatus;
import software.amazon.awssdk.services.batch.model.JobSummary;
import software.amazon.awssdk.services.batch.model.ListJobsRequest;
import software.amazon.awssdk.services.batch.model.RegisterJobDefinitionResponse;
import software.amazon.awssdk.services.batch.model.NetworkConfiguration;
import software.amazon.awssdk.services.batch.model.PlatformCapability;
import software.amazon.awssdk.services.batch.model.RegisterJobDefinitionRequest;
import software.amazon.awssdk.services.batch.model.ResourceRequirement;
import software.amazon.awssdk.services.batch.model.ResourceType;
import software.amazon.awssdk.services.batch.model.RuntimePlatform;
import software.amazon.awssdk.services.batch.model.SubmitJobRequest;
import software.amazon.awssdk.services.batch.model.CreateJobQueueResponse;
import java.time.Duration;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
```

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.atomic.AtomicBoolean;
import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.services.batch.model.SubmitJobResponse;
import software.amazon.awssdk.services.batch.model.UpdateComputeEnvironmentRequest;
import software.amazon.awssdk.services.batch.model.UpdateComputeEnvironmentResponse;
import software.amazon.awssdk.services.batch.model.UpdateJobQueueRequest;
import software.amazon.awssdk.services.batch.model.UpdateJobQueueResponse;
import software.amazon.awssdk.services.batch.paginators.ListJobsPublisher;
import software.amazon.awssdk.services.sts.StsAsyncClient;
import software.amazon.awssdk.services.sts.model.GetCallerIdentityResponse;

public class BatchActions {
 private static BatchAsyncClient batchClient;

 private static final Logger logger =
 LoggerFactory.getLogger(BatchActions.class);

 private static BatchAsyncClient getAsyncClient() {
 if (batchClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 batchClient = BatchAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return batchClient;
 }
}
```

```

 }

 /**
 * Asynchronously creates a new compute environment in AWS Batch.
 *
 * @param computeEnvironmentName the name of the compute environment to create
 * @param batchIAMRole the IAM role to be used by the compute environment
 * @param subnet the subnet ID to be used for the compute environment
 * @param secGroup the security group ID to be used for the compute environment
 * @return a {@link CompletableFuture} representing the asynchronous operation,
 which will complete with the
 * {@link CreateComputeEnvironmentResponse} when the compute environment
 has been created
 * @throws BatchException if there is an error creating the compute environment
 * @throws RuntimeException if there is an unexpected error during the operation
 */
 public CompletableFuture<CreateComputeEnvironmentResponse>
 createComputeEnvironmentAsync(
 String computeEnvironmentName, String batchIAMRole, String subnet, String
 secGroup) {
 CreateComputeEnvironmentRequest environmentRequest =
 CreateComputeEnvironmentRequest.builder()
 .computeEnvironmentName(computeEnvironmentName)
 .type(CETType.MANAGED)
 .state(CESState.ENABLED)
 .computeResources(ComputeResource.builder()
 .type(CRTType.FARGATE)
 .maxvCpus(256)
 .subnets(Collections.singletonList(subnet))
 .securityGroupIds(Collections.singletonList(secGroup))
 .build())
 .serviceRole(batchIAMRole)
 .build();

 CompletableFuture<CreateComputeEnvironmentResponse> response =
 getAsyncClient().createComputeEnvironment(environmentRequest);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
 ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });
 }

```

```

 return response;
 }

 public CompletableFuture<DeleteComputeEnvironmentResponse>
deleteComputeEnvironmentAsync(String computeEnvironmentName) {
 DeleteComputeEnvironmentRequest deleteComputeEnvironment =
DeleteComputeEnvironmentRequest.builder()
 .computeEnvironment(computeEnvironmentName)
 .build();

 return getAsyncClient().deleteComputeEnvironment(deleteComputeEnvironment)
 .whenComplete((response, ex) -> {
 if (ex != null) {
 Throwable cause = ex.getCause();
 if (cause instanceof BatchException) {
 throw new RuntimeException(cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 }
 });
 }

 /**
 * Checks the status of the specified compute environment.
 *
 * @param computeEnvironmentName the name of the compute environment to check
 * @return a CompletableFuture containing the status of the compute environment,
 or "ERROR" if an exception occurs
 */
 public CompletableFuture<String> checkComputeEnvironmentsStatus(String
computeEnvironmentName) {
 if (computeEnvironmentName == null || computeEnvironmentName.isEmpty()) {
 throw new IllegalArgumentException("Compute environment name cannot be
null or empty");
 }

 DescribeComputeEnvironmentsRequest environmentsRequest =
DescribeComputeEnvironmentsRequest.builder()
 .computeEnvironments(computeEnvironmentName)
 .build();
 }

```

```

 CompletableFuture<DescribeComputeEnvironmentsResponse> response =
getAsyncClient().describeComputeEnvironments(environmentsRequest);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });

 return response.thenApply(resp -> resp.computeEnvironments().stream()
 .map(env -> env.statusAsString())
 .findFirst()
 .orElse("UNKNOWN"));
 }

 /**
 * Creates a job queue asynchronously.
 *
 * @param jobQueueName the name of the job queue to create
 * @param computeEnvironmentName the name of the compute environment to
associate with the job queue
 * @return a CompletableFuture that completes with the Amazon Resource Name
(ARN) of the job queue
 */
 public CompletableFuture<String> createJobQueueAsync(String jobQueueName, String
computeEnvironmentName) {
 if (jobQueueName == null || jobQueueName.isEmpty()) {
 throw new IllegalArgumentException("Job queue name cannot be null or
empty");
 }
 if (computeEnvironmentName == null || computeEnvironmentName.isEmpty()) {
 throw new IllegalArgumentException("Compute environment name cannot be
null or empty");
 }

 CreateJobQueueRequest request = CreateJobQueueRequest.builder()
 .jobQueueName(jobQueueName)
 .priority(1)
 .computeEnvironmentOrder(ComputeEnvironmentOrder.builder()
 .computeEnvironment(computeEnvironmentName)
 .order(1)
 .build())
 .build();
 }

```

```

 CompletableFuture<CreateJobQueueResponse> response =
getAsyncClient().createJobQueue(request);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });

 return response.thenApply(CreateJobQueueResponse::jobQueueArn);
 }

/**
 * Asynchronously lists the jobs in the specified job queue with the given job
status.
 *
 * @param jobQueue the name of the job queue to list jobs from
 * @return a List<JobSummary> that contains the jobs that succeeded
 */
public List<JobSummary> listJobsAsync(String jobQueue) {
 if (jobQueue == null || jobQueue.isEmpty()) {
 throw new IllegalArgumentException("Job queue cannot be null or empty");
 }

 ListJobsRequest listJobsRequest = ListJobsRequest.builder()
 .jobQueue(jobQueue)
 .jobStatus(JobStatus.SUCCEEDED) // Filter jobs by status.
 .build();

 List<JobSummary> jobSummaries = new ArrayList<>();
 ListJobsPublisher listJobsPaginator =
getAsyncClient().listJobsPaginator(listJobsRequest);
 CompletableFuture<Void> future = listJobsPaginator.subscribe(response -> {
 jobSummaries.addAll(response.jobSummaryList());
 });
 future.join();
 return jobSummaries;
}

/**
 * Registers a new job definition asynchronously in AWS Batch.
 * <p>

```

```

 * When using Fargate as the compute environment, it is crucial to set the
 * {@link NetworkConfiguration} with {@link AssignPublicIp#ENABLED} to
 * ensure proper networking configuration for the Fargate tasks. This
 * allows the tasks to communicate with external services, access the
 * internet, or communicate within a VPC.
 *
 * @param jobDefinitionName the name of the job definition to be registered
 * @param executionRoleARN the ARN (Amazon Resource Name) of the execution role
 * that provides permissions for the containers in the
job
 * @param cpuArch a value of either X86_64 or ARM64 required for the service
call
 * @return a CompletableFuture that completes with the ARN of the registered
 * job definition upon successful execution, or completes exceptionally
with
 * an error if the registration fails
 */
 public CompletableFuture<String> registerJobDefinitionAsync(String
jobDefinitionName, String executionRoleARN, String image, String cpuArch) {
 NetworkConfiguration networkConfiguration = NetworkConfiguration.builder()
 .assignPublicIp(AssignPublicIp.ENABLED)
 .build();

 ContainerProperties containerProperties = ContainerProperties.builder()
 .image(image)
 .executionRoleArn(executionRoleARN)
 .resourceRequirements(
 Arrays.asList(
 ResourceRequirement.builder()
 .type(ResourceType.VCPU)
 .value("1")
 .build(),
 ResourceRequirement.builder()
 .type(ResourceType.MEMORY)
 .value("2048")
 .build()
)
)
 .networkConfiguration(networkConfiguration)
 .runtimePlatform(b -> b
 .cpuArchitecture(cpuArch)
 .operatingSystemFamily("LINUX"))
 .build();

```



```

 RegisterJobDefinitionRequest request =
RegisterJobDefinitionRequest.builder()
 .jobDefinitionName(jobDefinitionName)
 .type(JobDefinitionType.CONTAINER)
 .containerProperties(containerProperties)
 .platformCapabilities(PlatformCapability.FARGATE)
 .build();

 CompletableFuture<String> future = new CompletableFuture<>();
 getAsyncClient().registerJobDefinition(request)
 .thenApply(RegisterJobDefinitionResponse::jobDefinitionArn)
 .whenComplete((result, ex) -> {
 if (ex != null) {
 future.completeExceptionally(ex);
 } else {
 future.complete(result);
 }
 });
 });

 return future;
}

/**
 * Deregisters a job definition asynchronously.
 *
 * @param jobDefinition the name of the job definition to be deregistered
 * @return a CompletableFuture that completes when the job definition has been
deregistered
 * or an exception has occurred
 */
public CompletableFuture<DeregisterJobDefinitionResponse>
deregisterJobDefinitionAsync(String jobDefinition) {
 DeregisterJobDefinitionRequest jobDefinitionRequest =
DeregisterJobDefinitionRequest.builder()
 .jobDefinition(jobDefinition)
 .build();

 CompletableFuture<DeregisterJobDefinitionResponse> responseFuture =
getAsyncClient().deregisterJobDefinition(jobDefinitionRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);
 }
 });
}

```

```

 });

 return responseFuture;
}

/**
 * Disables the specified job queue asynchronously.
 *
 * @param jobQueueArn the Amazon Resource Name (ARN) of the job queue to be
disabled
 * @return a {@link CompletableFuture} that completes when the job queue update
operation is complete,
 * or completes exceptionally if an error occurs during the operation
 */
public CompletableFuture<Void> disableJobQueueAsync(String jobQueueArn) {
 UpdateJobQueueRequest updateRequest = UpdateJobQueueRequest.builder()
 .jobQueue(jobQueueArn)
 .state(JQState.DISABLED)
 .build();

 CompletableFuture<UpdateJobQueueResponse> responseFuture =
getAsyncClient().updateJobQueue(updateRequest);
 return responseFuture.whenComplete((updateResponse, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to update job queue: " +
ex.getMessage(), ex);
 }
 }).thenApply(updateResponse -> null);
}

/**
 * Deletes a Batch job queue asynchronously.
 *
 * @param jobQueueArn The Amazon Resource Name (ARN) of the job queue to delete.
 * @return A CompletableFuture that represents the asynchronous deletion of the
job queue.
 *
 * The future completes when the job queue has been successfully deleted
or if an error occurs.
 *
 * If successful, the future will be completed with a {@code Void}
value.
 *
 * If an error occurs, the future will be completed exceptionally with
the thrown exception.
 */
public CompletableFuture<Void> deleteJobQueueAsync(String jobQueueArn) {

```

```

 DeleteJobQueueRequest deleteRequest = DeleteJobQueueRequest.builder()
 .jobQueue(jobQueueArn)
 .build();

 CompletableFuture<DeleteJobQueueResponse> responseFuture =
getAsyncClient().deleteJobQueue(deleteRequest);
 return responseFuture.whenComplete((deleteResponse, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to delete job queue: " +
ex.getMessage(), ex);
 }
 }).thenApply(deleteResponse -> null);
 }

 /**
 * Asynchronously describes the job queue associated with the specified compute
environment.
 *
 * @param computeEnvironmentName the name of the compute environment to find the
associated job queue for
 * @return a {@link CompletableFuture} that, when completed, contains the job
queue ARN associated with the specified compute environment
 * @throws RuntimeException if the job queue description fails
 */
 public CompletableFuture<String> describeJobQueueAsync(String
computeEnvironmentName) {
 DescribeJobQueuesRequest describeJobQueuesRequest =
DescribeJobQueuesRequest.builder()
 .build();

 CompletableFuture<DescribeJobQueuesResponse> responseFuture =
getAsyncClient().describeJobQueues(describeJobQueuesRequest);
 return responseFuture.whenComplete((describeJobQueuesResponse, ex) -> {
 if (describeJobQueuesResponse != null) {
 String jobQueueARN;
 for (JobQueueDetail jobQueueDetail :
describeJobQueuesResponse.jobQueues()) {
 for (ComputeEnvironmentOrder computeEnvironmentOrder :
jobQueueDetail.computeEnvironmentOrder()) {
 String computeEnvironment =
computeEnvironmentOrder.computeEnvironment();
 String name = getComputeEnvironmentName(computeEnvironment);
 if (name.equals(computeEnvironmentName)) {
 jobQueueARN = jobQueueDetail.jobQueueArn();

```

```

 logger.info("Job queue ARN associated with the compute
environment: " + jobQueueARN);
 }
}
} else {
 throw new RuntimeException("Failed to describe job queue: " +
ex.getMessage(), ex);
}
}).thenApply(describeJobQueuesResponse -> {
 String jobQueueARN = "";
 for (JobQueueDetail jobQueueDetail :
describeJobQueuesResponse.jobQueues()) {
 for (ComputeEnvironmentOrder computeEnvironmentOrder :
jobQueueDetail.computeEnvironmentOrder()) {
 String computeEnvironment =
computeEnvironmentOrder.computeEnvironment();
 String name = getComputeEnvironmentName(computeEnvironment);
 if (name.equals(computeEnvironmentName)) {
 jobQueueARN = jobQueueDetail.jobQueueArn();
 }
 }
 }
 return jobQueueARN;
});
}

/**
 * Disables the specified compute environment asynchronously.
 *
 * @param computeEnvironmentName the name of the compute environment to disable
 * @return a CompletableFuture that completes when the compute environment is
disabled
 */
public CompletableFuture<UpdateComputeEnvironmentResponse>
disableComputeEnvironmentAsync(String computeEnvironmentName) {
 UpdateComputeEnvironmentRequest updateRequest =
UpdateComputeEnvironmentRequest.builder()
 .computeEnvironment(computeEnvironmentName)
 .state(CEState.DISABLED)
 .build();

 CompletableFuture<UpdateComputeEnvironmentResponse> responseFuture =
getAsyncClient().updateComputeEnvironment(updateRequest);

```

```

 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to disable compute environment: "
+ ex.getMessage(), ex);
 }
 });

 return responseFuture;
 }

 /**
 * Submits a job asynchronously to the AWS Batch service.
 *
 * @param jobDefinitionName the name of the job definition to use
 * @param jobQueueName the name of the job queue to submit the job to
 * @param jobARN the Amazon Resource Name (ARN) of the job definition
 * @return a CompletableFuture that, when completed, contains the job ID of the
 submitted job
 */
 public CompletableFuture<String> submitJobAsync(String jobDefinitionName, String
jobQueueName, String jobARN) {
 SubmitJobRequest jobRequest = SubmitJobRequest.builder()
 .jobDefinition(jobARN)
 .jobName(jobDefinitionName)
 .jobQueue(jobQueueName)
 .build();

 CompletableFuture<SubmitJobResponse> responseFuture =
getAsyncClient().submitJob(jobRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);
 }
 });

 return responseFuture.thenApply(SubmitJobResponse::jobId);
 }

 /**
 * Asynchronously retrieves the status of a specific job.
 *
 * @param jobId the ID of the job to retrieve the status for
 * @return a CompletableFuture that completes with the job status

```

```

 */
 public CompletableFuture<String> describeJobAsync(String jobId) {
 DescribeJobsRequest describeJobsRequest = DescribeJobsRequest.builder()
 .jobs(jobId)
 .build();

 CompletableFuture<DescribeJobsResponse> responseFuture =
getAsyncClient().describeJobs(describeJobsRequest);
 return responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);
 }
 }).thenApply(response -> response.jobs().get(0).status().toString());
 }

 /**
 * Disables the specific job queue using the asynchronous Java client.
 *
 * @param jobQueueArn the Amazon Resource Name (ARN) of the job queue to wait
for
 * @return a {@link CompletableFuture} that completes when the job queue is
disabled
 */
 public CompletableFuture<Void> waitForJobQueueToBeDisabledAsync(String
jobQueueArn) {
 AtomicBoolean isDisabled = new AtomicBoolean(false);
 return CompletableFuture.runAsync(() -> {
 while (!isDisabled.get()) {
 DescribeJobQueuesRequest describeRequest =
DescribeJobQueuesRequest.builder()
 .jobQueues(jobQueueArn)
 .build();

 CompletableFuture<DescribeJobQueuesResponse> responseFuture =
getAsyncClient().describeJobQueues(describeRequest);
 responseFuture.whenComplete((describeResponse, ex) -> {
 if (describeResponse != null) {
 for (JobQueueDetail jobQueue : describeResponse.jobQueues())
{
 if (jobQueue.jobQueueArn().equals(jobQueueArn) &&
jobQueue.state() == JQState.DISABLED) {
 isDisabled.set(true);
 break;

```

```

 }
 }
 } else {
 throw new RuntimeException("Error describing job queues",
ex);
 }
}).join();

 if (!isDisabled.get()) {
 try {
 logger.info("Waiting for job queue to be disabled...");
 Thread.sleep(5000);
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException("Thread interrupted while waiting
for job queue to be disabled", e);
 }
 }
}

}).whenComplete((result, throwable) -> {
 if (throwable != null) {
 throw new RuntimeException("Error while waiting for job queue to be
disabled", throwable);
 }
});
}

public CompletableFuture<String> getJobQueueARN(String jobQueueName) {
 // Describe the job queue asynchronously
 CompletableFuture<DescribeJobQueuesResponse> describeJobQueuesFuture =
batchClient.describeJobQueues(
 DescribeJobQueuesRequest.builder()
 .jobQueues(jobQueueName)
 .build()
);

 // Handle the asynchronous response and return the Job Queue ARN in the
CompletableFuture<String>
 CompletableFuture<String> jobQueueArnFuture = new CompletableFuture<>();
 describeJobQueuesFuture.whenComplete((response, error) -> {
 if (error != null) {
 if (error instanceof BatchException) {
 logger.info("Batch error: " + ((BatchException)
error).awsErrorDetails().errorMessage());

```

```

 } else {
 logger.info("Error describing job queue: " +
error.getMessage());
 }
 jobQueueArnFuture.completeExceptionally(new RuntimeException("Failed
to retrieve Job Queue ARN", error));
 } else {
 if (response.jobQueues().isEmpty()) {
 jobQueueArnFuture.completeExceptionally(new
RuntimeException("Job queue not found: " + jobQueueName));
 } else {
 // Assuming only one job queue is returned for the given name
 String jobQueueArn = response.jobQueues().get(0).jobQueueArn();
 jobQueueArnFuture.complete(jobQueueArn);
 }
 }
 });

 return jobQueueArnFuture;
}

private static String getComputeEnvironmentName(String computeEnvironment) {
 String[] parts = computeEnvironment.split("/");
 if (parts.length == 2) {
 return parts[1];
 }
 return null;
}

public CompletableFuture<String> getAccountId() {
 StsAsyncClient stsAsyncClient = StsAsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 return stsAsyncClient.getCallerIdentity()
 .thenApply(GetCallerIdentityResponse::account);
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.



- [CreateComputeEnvironment](#)
- [CreateJobQueue](#)
- [DeleteComputeEnvironment](#)
- [DeleteJobQueue](#)
- [DeregisterJobDefinition](#)
- [DescribeComputeEnvironments](#)
- [DescribeJobQueues](#)
- [DescribeJobs](#)
- [ListJobsPaginator](#)
- [RegisterJobDefinition](#)
- [SubmitJob](#)
- [UpdateComputeEnvironment](#)
- [UpdateJobQueue](#)

## Actions

### CreateComputeEnvironment

The following code example shows how to use CreateComputeEnvironment.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously creates a new compute environment in AWS Batch.
 *
 * @param computeEnvironmentName the name of the compute environment to create
 * @param batchIAMRole the IAM role to be used by the compute environment
 * @param subnet the subnet ID to be used for the compute environment
 * @param secGroup the security group ID to be used for the compute environment
```

```

 * @return a {@link CompletableFuture} representing the asynchronous operation,
 which will complete with the
 * {@link CreateComputeEnvironmentResponse} when the compute environment
 has been created
 * @throws BatchException if there is an error creating the compute environment
 * @throws RuntimeException if there is an unexpected error during the operation
 */
 public CompletableFuture<CreateComputeEnvironmentResponse>
 createComputeEnvironmentAsync(
 String computeEnvironmentName, String batchIAMRole, String subnet, String
 secGroup) {
 CreateComputeEnvironmentRequest environmentRequest =
 CreateComputeEnvironmentRequest.builder()
 .computeEnvironmentName(computeEnvironmentName)
 .type(CETType.MANAGED)
 .state(CESState.ENABLED)
 .computeResources(ComputeResource.builder()
 .type(CRType.FARGATE)
 .maxvCpus(256)
 .subnets(Collections.singletonList(subnet))
 .securityGroupIds(Collections.singletonList(secGroup))
 .build())
 .serviceRole(batchIAMRole)
 .build();

 CompletableFuture<CreateComputeEnvironmentResponse> response =
 getAsyncClient().createComputeEnvironment(environmentRequest);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
 ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });

 return response;
 }

```

- For API details, see [CreateComputeEnvironment](#) in *AWS SDK for Java 2.x API Reference*.

## CreateJobQueue

The following code example shows how to use CreateJobQueue.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a job queue asynchronously.
 *
 * @param jobQueueName the name of the job queue to create
 * @param computeEnvironmentName the name of the compute environment to
associate with the job queue
 * @return a CompletableFuture that completes with the Amazon Resource Name
(ARN) of the job queue
 */
public CompletableFuture<String> createJobQueueAsync(String jobQueueName, String
computeEnvironmentName) {
 if (jobQueueName == null || jobQueueName.isEmpty()) {
 throw new IllegalArgumentException("Job queue name cannot be null or
empty");
 }
 if (computeEnvironmentName == null || computeEnvironmentName.isEmpty()) {
 throw new IllegalArgumentException("Compute environment name cannot be
null or empty");
 }

 CreateJobQueueRequest request = CreateJobQueueRequest.builder()
 .jobQueueName(jobQueueName)
 .priority(1)
 .computeEnvironmentOrder(ComputeEnvironmentOrder.builder()
 .computeEnvironment(computeEnvironmentName)
 .order(1)
 .build())
 .build();

 CompletableFuture<CreateJobQueueResponse> response =
getAsyncClient().createJobQueue(request);
}
```

```

 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });

 return response.thenApply(CreateJobQueueResponse::jobQueueArn);
 }

```

- For API details, see [CreateJobQueue](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteComputeEnvironment

The following code example shows how to use DeleteComputeEnvironment.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public CompletableFuture<DeleteComputeEnvironmentResponse>
deleteComputeEnvironmentAsync(String computeEnvironmentName) {
 DeleteComputeEnvironmentRequest deleteComputeEnvironment =
DeleteComputeEnvironmentRequest.builder()
 .computeEnvironment(computeEnvironmentName)
 .build();

 return getAsyncClient().deleteComputeEnvironment(deleteComputeEnvironment)
 .whenComplete((response, ex) -> {
 if (ex != null) {
 Throwable cause = ex.getCause();
 if (cause instanceof BatchException) {
 throw new RuntimeException(cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 }
 });
}

```

```

 }
 }
});
}

```

- For API details, see [DeleteComputeEnvironment](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteJobQueue

The following code example shows how to use DeleteJobQueue.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a Batch job queue asynchronously.
 *
 * @param jobQueueArn The Amazon Resource Name (ARN) of the job queue to delete.
 * @return A CompletableFuture that represents the asynchronous deletion of the
 * job queue.
 *
 * The future completes when the job queue has been successfully deleted
 * or if an error occurs.
 *
 * If successful, the future will be completed with a {@code Void}
 * value.
 *
 * If an error occurs, the future will be completed exceptionally with
 * the thrown exception.
 */
public CompletableFuture<Void> deleteJobQueueAsync(String jobQueueArn) {
 DeleteJobQueueRequest deleteRequest = DeleteJobQueueRequest.builder()
 .jobQueue(jobQueueArn)
 .build();

 CompletableFuture<DeleteJobQueueResponse> responseFuture =
getAsyncClient().deleteJobQueue(deleteRequest);
 return responseFuture.whenComplete((deleteResponse, ex) -> {
 if (ex != null) {

```

```

 throw new RuntimeException("Failed to delete job queue: " +
ex.getMessage(), ex);
 }
}).thenApply(deleteResponse -> null);
}

```

- For API details, see [DeleteJobQueue](#) in *AWS SDK for Java 2.x API Reference*.

## DeregisterJobDefinition

The following code example shows how to use DeregisterJobDefinition.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deregisters a job definition asynchronously.
 *
 * @param jobDefinition the name of the job definition to be deregistered
 * @return a CompletableFuture that completes when the job definition has been
deregistered
 * or an exception has occurred
 */
public CompletableFuture<DeregisterJobDefinitionResponse>
deregisterJobDefinitionAsync(String jobDefinition) {
 DeregisterJobDefinitionRequest jobDefinitionRequest =
DeregisterJobDefinitionRequest.builder()
 .jobDefinition(jobDefinition)
 .build();

 CompletableFuture<DeregisterJobDefinitionResponse> responseFuture =
getAsyncClient().deregisterJobDefinition(jobDefinitionRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);

```

```

 }
 });

 return responseFuture;
}

```

- For API details, see [DeregisterJobDefinition](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeComputeEnvironments

The following code example shows how to use `DescribeComputeEnvironments`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Checks the status of the specified compute environment.
 *
 * @param computeEnvironmentName the name of the compute environment to check
 * @return a CompletableFuture containing the status of the compute environment,
 * or "ERROR" if an exception occurs
 */
public CompletableFuture<String> checkComputeEnvironmentsStatus(String
computeEnvironmentName) {
 if (computeEnvironmentName == null || computeEnvironmentName.isEmpty()) {
 throw new IllegalArgumentException("Compute environment name cannot be
null or empty");
 }

 DescribeComputeEnvironmentsRequest environmentsRequest =
DescribeComputeEnvironmentsRequest.builder()
 .computeEnvironments(computeEnvironmentName)
 .build();

 CompletableFuture<DescribeComputeEnvironmentsResponse> response =
getAsyncClient().describeComputeEnvironments(environmentsRequest);

```

```

 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 });

 return response.thenApply(resp -> resp.computeEnvironments().stream()
 .map(env -> env.statusAsString())
 .findFirst()
 .orElse("UNKNOWN"));
 }
}

```

- For API details, see [DescribeComputeEnvironments](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeJobQueues

The following code example shows how to use DescribeJobQueues.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously describes the job queue associated with the specified compute
 * environment.
 *
 * @param computeEnvironmentName the name of the compute environment to find the
 * associated job queue for
 * @return a {@link CompletableFuture} that, when completed, contains the job
 * queue ARN associated with the specified compute environment
 * @throws RuntimeException if the job queue description fails
 */
public CompletableFuture<String> describeJobQueueAsync(String
computeEnvironmentName) {

```



```

 DescribeJobQueuesRequest describeJobQueuesRequest =
DescribeJobQueuesRequest.builder()
 .build();

 CompletableFuture<DescribeJobQueuesResponse> responseFuture =
getAsyncClient().describeJobQueues(describeJobQueuesRequest);
 return responseFuture.whenComplete((describeJobQueuesResponse, ex) -> {
 if (describeJobQueuesResponse != null) {
 String jobQueueARN;
 for (JobQueueDetail jobQueueDetail :
describeJobQueuesResponse.jobQueues()) {
 for (ComputeEnvironmentOrder computeEnvironmentOrder :
jobQueueDetail.computeEnvironmentOrder()) {
 String computeEnvironment =
computeEnvironmentOrder.computeEnvironment();
 String name = getComputeEnvironmentName(computeEnvironment);
 if (name.equals(computeEnvironmentName)) {
 jobQueueARN = jobQueueDetail.jobQueueArn();
 logger.info("Job queue ARN associated with the compute
environment: " + jobQueueARN);
 }
 }
 }
 } else {
 throw new RuntimeException("Failed to describe job queue: " +
ex.getMessage(), ex);
 }
 }).thenApply(describeJobQueuesResponse -> {
 String jobQueueARN = "";
 for (JobQueueDetail jobQueueDetail :
describeJobQueuesResponse.jobQueues()) {
 for (ComputeEnvironmentOrder computeEnvironmentOrder :
jobQueueDetail.computeEnvironmentOrder()) {
 String computeEnvironment =
computeEnvironmentOrder.computeEnvironment();
 String name = getComputeEnvironmentName(computeEnvironment);
 if (name.equals(computeEnvironmentName)) {
 jobQueueARN = jobQueueDetail.jobQueueArn();
 }
 }
 }
 return jobQueueARN;
 });
 }
}

```

- For API details, see [DescribeJobQueues](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeJobs

The following code example shows how to use DescribeJobs.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously retrieves the status of a specific job.
 *
 * @param jobId the ID of the job to retrieve the status for
 * @return a CompletableFuture that completes with the job status
 */
public CompletableFuture<String> describeJobAsync(String jobId) {
 DescribeJobsRequest describeJobsRequest = DescribeJobsRequest.builder()
 .jobs(jobId)
 .build();

 CompletableFuture<DescribeJobsResponse> responseFuture =
getAsyncClient().describeJobs(describeJobsRequest);
 return responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);
 }
 }).thenApply(response -> response.jobs().get(0).status().toString());
}
```

- For API details, see [DescribeJobs](#) in *AWS SDK for Java 2.x API Reference*.

## ListJobsPaginator

The following code example shows how to use ListJobsPaginator.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously lists the jobs in the specified job queue with the given job
 * status.
 *
 * @param jobQueue the name of the job queue to list jobs from
 * @return a List<JobSummary> that contains the jobs that succeeded
 */
public List<JobSummary> listJobsAsync(String jobQueue) {
 if (jobQueue == null || jobQueue.isEmpty()) {
 throw new IllegalArgumentException("Job queue cannot be null or empty");
 }

 ListJobsRequest listJobsRequest = ListJobsRequest.builder()
 .jobQueue(jobQueue)
 .jobStatus(JobStatus.SUCCEEDED) // Filter jobs by status.
 .build();

 List<JobSummary> jobSummaries = new ArrayList<>();
 ListJobsPublisher listJobsPaginator =
getAsyncClient().listJobsPaginator(listJobsRequest);
 CompletableFuture<Void> future = listJobsPaginator.subscribe(response -> {
 jobSummaries.addAll(response.jobSummaryList());
 });
 future.join();
 return jobSummaries;
}
```

- For API details, see [ListJobsPaginator](#) in *AWS SDK for Java 2.x API Reference*.

## RegisterJobDefinition

The following code example shows how to use RegisterJobDefinition.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Registers a new job definition asynchronously in AWS Batch.
 * <p>
 * When using Fargate as the compute environment, it is crucial to set the
 * {@link NetworkConfiguration} with {@link AssignPublicIp#ENABLED} to
 * ensure proper networking configuration for the Fargate tasks. This
 * allows the tasks to communicate with external services, access the
 * internet, or communicate within a VPC.
 *
 * @param jobDefinitionName the name of the job definition to be registered
 * @param executionRoleARN the ARN (Amazon Resource Name) of the execution role
 * that provides permissions for the containers in the
job
 * @param cpuArch a value of either X86_64 or ARM64 required for the service
call
 * @return a CompletableFuture that completes with the ARN of the registered
 * job definition upon successful execution, or completes exceptionally
with
 * an error if the registration fails
 */
public CompletableFuture<String> registerJobDefinitionAsync(String
jobDefinitionName, String executionRoleARN, String image, String cpuArch) {
 NetworkConfiguration networkConfiguration = NetworkConfiguration.builder()
 .assignPublicIp(AssignPublicIp.ENABLED)
 .build();

 ContainerProperties containerProperties = ContainerProperties.builder()
 .image(image)
 .executionRoleArn(executionRoleARN)
 .resourceRequirements(
 Arrays.asList(
```

```

 ResourceRequirement.builder()
 .type(ResourceType.VCPU)
 .value("1")
 .build(),
 ResourceRequirement.builder()
 .type(ResourceType.MEMORY)
 .value("2048")
 .build()
)
)
.networkConfiguration(networkConfiguration)
.runtimePlatform(b -> b
 .cpuArchitecture(cpuArch)
 .operatingSystemFamily("LINUX"))
.build();

RegisterJobDefinitionRequest request =
RegisterJobDefinitionRequest.builder()
 .jobDefinitionName(jobDefinitionName)
 .type(JobDefinitionType.CONTAINER)
 .containerProperties(containerProperties)
 .platformCapabilities(PlatformCapability.FARGATE)
 .build();

CompletableFuture<String> future = new CompletableFuture<>();
getAsyncClient().registerJobDefinition(request)
 .thenApply(RegisterJobDefinitionResponse::jobDefinitionArn)
 .whenComplete((result, ex) -> {
 if (ex != null) {
 future.completeExceptionally(ex);
 } else {
 future.complete(result);
 }
 });

return future;
}

```

- For API details, see [RegisterJobDefinition](#) in *AWS SDK for Java 2.x API Reference*.

## SubmitJob

The following code example shows how to use SubmitJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Submits a job asynchronously to the AWS Batch service.
 *
 * @param jobDefinitionName the name of the job definition to use
 * @param jobQueueName the name of the job queue to submit the job to
 * @param jobARN the Amazon Resource Name (ARN) of the job definition
 * @return a CompletableFuture that, when completed, contains the job ID of the
 * submitted job
 */
public CompletableFuture<String> submitJobAsync(String jobDefinitionName, String
jobQueueName, String jobARN) {
 SubmitJobRequest jobRequest = SubmitJobRequest.builder()
 .jobDefinition(jobARN)
 .jobName(jobDefinitionName)
 .jobQueue(jobQueueName)
 .build();

 CompletableFuture<SubmitJobResponse> responseFuture =
getAsyncClient().submitJob(jobRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Unexpected error occurred: " +
ex.getMessage(), ex);
 }
 });

 return responseFuture.thenApply(SubmitJobResponse::jobId);
}
```

- For API details, see [SubmitJob](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateComputeEnvironment

The following code example shows how to use UpdateComputeEnvironment.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Disables the specified compute environment asynchronously.
 *
 * @param computeEnvironmentName the name of the compute environment to disable
 * @return a CompletableFuture that completes when the compute environment is
 * disabled
 */
public CompletableFuture<UpdateComputeEnvironmentResponse>
disableComputeEnvironmentAsync(String computeEnvironmentName) {
 UpdateComputeEnvironmentRequest updateRequest =
UpdateComputeEnvironmentRequest.builder()
 .computeEnvironment(computeEnvironmentName)
 .state(CESState.DISABLED)
 .build();

 CompletableFuture<UpdateComputeEnvironmentResponse> responseFuture =
getAsyncClient().updateComputeEnvironment(updateRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to disable compute environment: "
+ ex.getMessage(), ex);
 }
 });

 return responseFuture;
}
```

- For API details, see [UpdateComputeEnvironment](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateJobQueue

The following code example shows how to use UpdateJobQueue.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Disables the specified job queue asynchronously.
 *
 * @param jobQueueArn the Amazon Resource Name (ARN) of the job queue to be
disabled
 * @return a {@link CompletableFuture} that completes when the job queue update
operation is complete,
 * or completes exceptionally if an error occurs during the operation
 */
public CompletableFuture<Void> disableJobQueueAsync(String jobQueueArn) {
 UpdateJobQueueRequest updateRequest = UpdateJobQueueRequest.builder()
 .jobQueue(jobQueueArn)
 .state(JQState.DISABLED)
 .build();

 CompletableFuture<UpdateJobQueueResponse> responseFuture =
getAsyncClient().updateJobQueue(updateRequest);
 return responseFuture.whenComplete((updateResponse, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to update job queue: " +
ex.getMessage(), ex);
 }
 }).thenApply(updateResponse -> null);
}
```

- For API details, see [UpdateJobQueue](#) in *AWS SDK for Java 2.x API Reference*.



# Amazon Bedrock examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Bedrock.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### GetFoundationModel

The following code example shows how to use `GetFoundationModel`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get details about a foundation model using the synchronous Amazon Bedrock client.

```
/**
 * Get details about an Amazon Bedrock foundation model.
 *
 * @param bedrockClient The service client for accessing Amazon Bedrock.
 * @param modelIdentifier The model identifier.
 * @return An object containing the foundation model's details.
 */
public static FoundationModelDetails getFoundationModel(BedrockClient
bedrockClient, String modelIdentifier) {
 try {
```

```

 GetFoundationModelResponse response = bedrockClient.getFoundationModel(
 r -> r.modelIdentifier(modelIdentifier)
);

 FoundationModelDetails model = response.modelDetails();

 System.out.println(" Model ID: " + model.modelId());
 System.out.println(" Model ARN: " +
model.modelArn());
 System.out.println(" Model Name: " +
model.modelName());
 System.out.println(" Provider Name: " +
model.providerName());
 System.out.println(" Lifecycle status: " +
model.modelLifecycle().statusAsString());
 System.out.println(" Input modalities: " +
model.inputModalities());
 System.out.println(" Output modalities: " +
model.outputModalities());
 System.out.println(" Supported customizations: " +
model.customizationsSupported());
 System.out.println(" Supported inference types: " +
model.inferenceTypesSupported());
 System.out.println(" Response streaming supported: " +
model.responseStreamingSupported());

 return model;

 } catch (ValidationException e) {
 throw new IllegalArgumentException(e.getMessage());
 } catch (SdkException e) {
 System.err.println(e.getMessage());
 throw new RuntimeException(e);
 }
}

```

Get details about a foundation model using the asynchronous Amazon Bedrock client.

```

/**
 * Get details about an Amazon Bedrock foundation model.
 *
 * @param bedrockClient The async service client for accessing Amazon Bedrock.

```

```

 * @param modelIdentifier The model identifier.
 * @return An object containing the foundation model's details.
 */
 public static FoundationModelDetails getFoundationModel(BedrockAsyncClient
bedrockClient, String modelIdentifier) {
 try {
 CompletableFuture<GetFoundationModelResponse> future =
bedrockClient.getFoundationModel(
 r -> r.modelIdentifier(modelIdentifier)
);

 FoundationModelDetails model = future.get().modelDetails();

 System.out.println(" Model ID: " + model.modelId());
 System.out.println(" Model ARN: " +
model.modelArn());
 System.out.println(" Model Name: " +
model.modelName());
 System.out.println(" Provider Name: " +
model.providerName());
 System.out.println(" Lifecycle status: " +
model.modelLifecycle().statusAsString());
 System.out.println(" Input modalities: " +
model.inputModalities());
 System.out.println(" Output modalities: " +
model.outputModalities());
 System.out.println(" Supported customizations: " +
model.customizationsSupported());
 System.out.println(" Supported inference types: " +
model.inferenceTypesSupported());
 System.out.println(" Response streaming supported: " +
model.responseStreamingSupported());

 return model;

 } catch (ExecutionException e) {
 if (e.getMessage().contains("ValidationException")) {
 throw new IllegalArgumentException(e.getMessage());
 } else {
 System.err.println(e.getMessage());
 throw new RuntimeException(e);
 }
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();

```

```

 System.err.println(e.getMessage());
 throw new RuntimeException(e);
 }
}

```

- For API details, see [GetFoundationModel](#) in *AWS SDK for Java 2.x API Reference*.

## ListFoundationModels

The following code example shows how to use `ListFoundationModels`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List the available Amazon Bedrock foundation models using the synchronous Amazon Bedrock client.

```

/**
 * Lists Amazon Bedrock foundation models that you can use.
 * You can filter the results with the request parameters.
 *
 * @param bedrockClient The service client for accessing Amazon Bedrock.
 * @return A list of objects containing the foundation models' details
 */
public static List<FoundationModelSummary> listFoundationModels(BedrockClient
bedrockClient) {

 try {
 ListFoundationModelsResponse response =
bedrockClient.listFoundationModels(r -> {});

 List<FoundationModelSummary> models = response.modelSummaries();

 if (models.isEmpty()) {
 System.out.println("No available foundation models in " +
region.toString());

```

```

 } else {
 for (FoundationModelSummary model : models) {
 System.out.println("Model ID: " + model.modelId());
 System.out.println("Provider: " + model.providerName());
 System.out.println("Name: " + model.modelName());
 System.out.println();
 }
 }

 return models;
 } catch (SdkClientException e) {
 System.err.println(e.getMessage());
 throw new RuntimeException(e);
 }
}

```

List the available Amazon Bedrock foundation models using the asynchronous Amazon Bedrock client.

```

/**
 * Lists Amazon Bedrock foundation models that you can use.
 * You can filter the results with the request parameters.
 *
 * @param bedrockClient The async service client for accessing Amazon Bedrock.
 * @return A list of objects containing the foundation models' details
 */
public static List<FoundationModelSummary>
listFoundationModels(BedrockAsyncClient bedrockClient) {
 try {
 CompletableFuture<ListFoundationModelsResponse> future =
 bedrockClient.listFoundationModels(r -> {});

 List<FoundationModelSummary> models = future.get().modelSummaries();

 if (models.isEmpty()) {
 System.out.println("No available foundation models in " +
 region.toString());
 } else {
 for (FoundationModelSummary model : models) {
 System.out.println("Model ID: " + model.modelId());
 System.out.println("Provider: " + model.providerName());
 }
 }
 }
}

```

```
 System.out.println("Name: " + model.modelName());
 System.out.println();
 }
}

return models;

} catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 System.err.println(e.getMessage());
 throw new RuntimeException(e);
} catch (ExecutionException e) {
 System.err.println(e.getMessage());
 throw new RuntimeException(e);
}
}
```

- For API details, see [ListFoundationModels](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Bedrock Runtime examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Bedrock Runtime.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Scenarios](#)
- [Amazon Nova](#)
- [Amazon Nova Canvas](#)
- [Amazon Titan Image Generator](#)
- [Amazon Titan Text Embeddings](#)
- [Anthropic Claude](#)

- [Cohere Command](#)
- [Meta Llama](#)
- [Mistral AI](#)
- [Stable Diffusion](#)

## Scenarios

### Create a playground application to interact with Amazon Bedrock foundation models

The following code example shows how to create playgrounds to interact with Amazon Bedrock foundation models through different modalities.

#### SDK for Java 2.x

The Java Foundation Model (FM) Playground is a Spring Boot sample application that showcases how to use Amazon Bedrock with Java. This example shows how Java developers can use Amazon Bedrock to build generative AI-enabled applications. You can test and interact with Amazon Bedrock foundation models by using the following three playgrounds:

- A text playground.
- A chat playground.
- An image playground.

The example also lists and displays the foundation models you have access to, along with their characteristics. For source code and deployment instructions, see the project in [GitHub](#).

#### Services used in this example

- Amazon Bedrock Runtime

### Generate videos from text prompts using Amazon Bedrock

The following code example shows how to a Spring Boot app that generates videos from text prompts using Amazon Bedrock and the Nova-Reel model.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Generate videos from text prompts using Amazon Bedrock and Nova-Reel.

```
import org.springframework.stereotype.Service;
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

import java.util.concurrent.CompletableFuture;

@Service
public class VideoGenerationService {

 public GenerateVideoResponse generateVideo(String prompt) {

 // add S3 bucket you want to store your generated videos
 String s3Bucket = "s3://mygeneratedvidoenovatest";

 //Create json request as an instance of Document class
 Document novaRequest = prepareDocument(prompt);

 // Create request
 StartAsyncInvokeRequest request = StartAsyncInvokeRequest.builder()
 .modelId("amazon.nova-reel-v1:0")
 .modelInput(novaRequest)
 .outputDataConfig(AsyncInvokeOutputDataConfig.builder()

.s3OutputDataConfig(AsyncInvokeS3OutputDataConfig.builder().s3Uri(s3Bucket).build())
 .build())
 .build();

 try (BedrockRuntimeAsyncClient bedrockClient =
getBedrockRuntimeAsyncClient()) {
```



```

 CompletableFuture<StartAsyncInvokeResponse>
startAsyncInvokeResponseCompletableFuture =
bedrockClient.startAsyncInvoke(request);

 //blocking operation to wait for the AWS API response
 StartAsyncInvokeResponse startAsyncInvokeResponse =
startAsyncInvokeResponseCompletableFuture.get();
 System.out.println("invocation ARN: " +
startAsyncInvokeResponse.invocationArn());

 GenerateVideoResponse response = new GenerateVideoResponse();
 response.setStatus("InProgress");
 response.setExecutionArn(startAsyncInvokeResponse.invocationArn());

 return response;
 } catch (Exception e) {
 System.out.println(e);
 throw new RuntimeException(e);
 }
}

public GenerateVideoResponse checkGenerationStatus(String invocationArn) {
 GenerateVideoResponse response = new GenerateVideoResponse();

 try (BedrockRuntimeAsyncClient bedrockClient =
getBedrockRuntimeAsyncClient()) {
 //creating async request to fetch status by invocation Arn
 GetAsyncInvokeRequest asyncRequest =
GetAsyncInvokeRequest.builder().invocationArn(invocationArn).build();

 CompletableFuture<GetAsyncInvokeResponse> asyncInvoke =
bedrockClient.getAsyncInvoke(asyncRequest);

 //blocking operation to wait for the AWS API response
 GetAsyncInvokeResponse asyncInvokeResponse = asyncInvoke.get();
 System.out.println("Invocation status =" +
asyncInvokeResponse.statusAsString());

 response.setExecutionArn(invocationArn);
 response.setStatus(asyncInvokeResponse.statusAsString());
 return response;
 } catch (Exception e) {
 e.printStackTrace();
 }
}

```

```
 throw new RuntimeException(e);
 }

}

private static BedrockRuntimeAsyncClient getBedrockRuntimeAsyncClient() {
 BedrockRuntimeAsyncClient bedrockClient =
BedrockRuntimeAsyncClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();
 return bedrockClient;
}

private static Document prepareDocument(String prompt) {
 Document textToVideoParams = Document.mapBuilder()
 .putString("text", prompt)
 .build();

 Document videoGenerationConfig = Document.mapBuilder()
 .putNumber("durationSeconds", 6)
 .putNumber("fps", 24)
 .putString("dimension", "1280x720")
 .build();

 Document novaRequest = Document.mapBuilder()
 .putString("taskType", "TEXT_VIDEO")
 .putDocument("textToVideoParams", textToVideoParams)
 .putDocument("videoGenerationConfig", videoGenerationConfig)
 .build();
 return novaRequest;
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [GetAsyncInvoke](#)
  - [StartAsyncInvoke](#)

## Tool use with the Converse API

The following code example shows how to build a typical interaction between an application, a generative AI model, and connected tools or APIs to mediate interactions between the AI and the outside world. It uses the example of connecting an external weather API to the AI model so it can provide real-time weather information based on user input.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The primary execution of the scenario flow. This scenario orchestrates the conversation between the user, the Amazon Bedrock Converse API, and a weather tool.

```
/*
This demo illustrates a tool use scenario using Amazon Bedrock's Converse API and a
weather tool.
The program interacts with a foundation model on Amazon Bedrock to provide weather
information based on user
input. It uses the Open-Meteo API (https://open-meteo.com) to retrieve current
weather data for a given location.
*/
public class BedrockScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static String modelId = "amazon.nova-lite-v1:0";
 private static String defaultPrompt = "What is the weather like in Seattle?";
 private static WeatherTool weatherTool = new WeatherTool();

 // The maximum number of recursive calls allowed in the tool use function.
 // This helps prevent infinite loops and potential performance issues.
 private static int maxRecursions = 5;
 static BedrockActions bedrockActions = new BedrockActions();
 public static boolean interactive = true;

 private static final String systemPrompt = ""
 You are a weather assistant that provides current weather data for user-
 specified locations using only
```

the Weather\_Tool, which expects latitude and longitude. Infer the coordinates from the location yourself.

If the user provides coordinates, infer the approximate location and refer to it in your response.

To use the tool, you strictly apply the provided tool specification.

- Explain your step-by-step process, and give brief updates before each step.

- Only use the Weather\_Tool for data. Never guess or make up information.

- Repeat the tool use for subsequent requests if necessary.

- If the tool errors, apologize, explain weather is unavailable, and suggest other options.

- Report temperatures in °C (°F) and wind in km/h (mph). Keep weather reports concise. Sparingly use emojis where appropriate.

- Only respond to weather queries. Remind off-topic users of your purpose.

- Never claim to search online, access external data, or use tools besides Weather\_Tool.

- Complete the entire process until you have all required data before sending the complete response.

```
""";
```

```
public static void main(String[] args) {
 Scanner scanner = new Scanner(System.in);
 System.out.println("""
=====
Welcome to the Amazon Bedrock Tool Use demo!
=====
```

This assistant provides current weather information for user-specified locations.

You can ask for weather details by providing the location name or coordinates.

Example queries:

- What's the weather like in New York?
- Current weather for latitude 40.70, longitude -74.01
- Is it warmer in Rome or Barcelona today?

To exit the program, simply type 'x' and press Enter.

```

 P.S.: You're not limited to single locations, or even to using
English!

 Have fun and experiment with the app!
 "");
 System.out.println(DASHES);

 try {
 runConversation(scanner);

 } catch (Exception ex) {
 System.out.println("There was a problem running the scenario: " +
ex.getMessage());
 }

 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("Amazon Bedrock Converse API with Tool Use Feature
Scenario is complete.");
 System.out.println(DASHES);
}

/**
 * Starts the conversation with the user and handles the interaction with
Bedrock.
 */
private static List<Message> runConversation(Scanner scanner) {
 List<Message> conversation = new ArrayList<>();

 // Get the first user input
 String userInput = getUserInput("Your weather info request:", scanner);
 System.out.println(userInput);

 while (userInput != null) {
 ContentBlock block = ContentBlock.builder()
 .text(userInput)
 .build();

 List<ContentBlock> blockList = new ArrayList<>();
 blockList.add(block);

 Message message = Message.builder()
 .role(ConversationRole.USER)
 .content(blockList)

```

```

 .build();

 conversation.add(message);

 // Send the conversation to Amazon Bedrock.
 ConverseResponse bedrockResponse =
sendConversationToBedrock(conversation);

 // Recursively handle the model's response until the model has returned
its final response or the recursion counter has reached 0.
 processModelResponse(bedrockResponse, conversation, maxRecursions);

 // Repeat the loop until the user decides to exit the application.
 userInput = getUserInput("Your weather info request:", scanner);
 }
 printFooter();
 return conversation;
}

/**
 * Processes the response from the model and updates the conversation
accordingly.
 *
 * @param modelResponse the response from the model
 * @param conversation the ongoing conversation
 * @param maxRecursion the maximum number of recursions allowed
 */
private static void processModelResponse(ConverseResponse modelResponse,
List<Message> conversation, int maxRecursion) {
 if (maxRecursion <= 0) {
 // Stop the process, the number of recursive calls could indicate an
infinite loop
 System.out.println("\tWarning: Maximum number of recursions reached.
Please try again.");
 }

 // Append the model's response to the ongoing conversation
 conversation.add(modelResponse.output().message());

 String modelResponseVal = modelResponse.stopReasonAsString();
 if (modelResponseVal.compareTo("tool_use") == 0) {
 // If the stop reason is "tool_use", forward everything to the tool use
handler
 handleToolUse(modelResponse.output(), conversation, maxRecursion - 1);
 }
}

```

```

 }

 if (modelResponseVal.compareTo("end_turn") == 0) {
 // If the stop reason is "end_turn", print the model's response text,
 and finish the process

PrintModelResponse(modelResponse.output().message().content().get(0).text());
 if (!interactive) {
 defaultPrompt = "x";
 }
 }
}

/**
 * Handles the use of a tool by the model in a conversation.
 *
 * @param modelResponse the response from the model, which may include a tool
use request
 * @param conversation the current conversation, which will be updated with the
tool use results
 * @param maxRecursion the maximum number of recursive calls allowed to handle
the model's response
 */
private static void handleToolUse(ConverseOutput modelResponse, List<Message>
conversation, int maxRecursion) {
 List<ContentBlock> toolResults = new ArrayList<>();

 // The model's response can consist of multiple content blocks
 for (ContentBlock contentBlock : modelResponse.message().content()) {
 if (contentBlock.text() != null && !contentBlock.text().isEmpty()) {
 // If the content block contains text, print it to the console
 PrintModelResponse(contentBlock.text());
 }

 if (contentBlock.toolUse() != null) {
 ToolResponse toolResponse = invokeTool(contentBlock.toolUse());

 // Add the tool use ID and the tool's response to the list of
results

 List<ToolResultContentBlock> contentBlockList = new ArrayList<>();
 ToolResultContentBlock block = ToolResultContentBlock.builder()
 .json(toolResponse.getContent())
 .build();
 contentBlockList.add(block);

```

```

 ToolResultBlock toolResultBlock = ToolResultBlock.builder()
 .toolUseId(toolResponse.getToolUseId())
 .content(contentBlockList)
 .build();

 ContentBlock contentBlock1 = ContentBlock.builder()
 .toolResult(toolResultBlock)
 .build();

 toolResults.add(contentBlock1);
 }
}

// Embed the tool results in a new user message
Message message = Message.builder()
 .role(ConversationRole.USER)
 .content(toolResults)
 .build();

// Append the new message to the ongoing conversation
//conversation.add(message);
conversation.add(message);

// Send the conversation to Amazon Bedrock
var response = sendConversationToBedrock(conversation);

// Recursively handle the model's response until the model has returned its
final response or the recursion counter has reached 0
processModelResponse(response, conversation, maxRecursion);
}

// Invokes the specified tool with the given payload and returns the tool's
response.
// If the requested tool does not exist, an error message is returned.
private static ToolResponse invokeTool(ToolUseBlock payload) {
 String toolName = payload.name();

 if (Objects.equals(toolName, "Weather_Tool")) {
 Map<String, Document> inputData = payload.input().asMap();
 printToolUse(toolName, inputData);

 // Invoke the weather tool with the input data provided
 }
}

```



```

 Document weatherResponse =
weatherTool.fetchWeatherData(inputData.get("latitude").toString(),
inputData.get("longitude").toString());

 ToolResponse toolResponse = new ToolResponse();
 toolResponse.setContent(weatherResponse);
 toolResponse.setToolUseId(payload.toolUseId());
 return toolResponse;
 } else {
 String errorMessage = "The requested tool with name " + toolName + "
does not exist.";
 System.out.println(errorMessage);
 return null;
 }
}

public static void printToolUse(String toolName, Map<String, Document>
inputData) {
 System.out.println("Invoking tool: " + toolName + " with input: " +
inputData.get("latitude").toString() + ", " + inputData.get("longitude").toString()
+ "...");
}

private static void PrintModelResponse(String message) {
 System.out.println("\tThe model's response:\n");
 System.out.println(message);
 System.out.println("");
}

private static ConverseResponse sendConversationToBedrock(List<Message>
conversation) {
 System.out.println("Calling Bedrock...");

 try {
 return bedrockActions.sendConverseRequestAsync(modelId, systemPrompt,
conversation, weatherTool.getToolSpec());
 } catch (ModelNotReadyException ex) {
 System.err.println("Model is not ready. Please try again later: " +
ex.getMessage());
 throw ex;
 } catch (BedrockRuntimeException ex) {
 System.err.println("Bedrock service error: " + ex.getMessage());
 throw ex;
 } catch (RuntimeException ex) {

```

```

 System.err.println("Unexpected error occurred: " + ex.getMessage());
 throw ex;
 }
}

private static ConverseResponse sendConversationToBedrockwithSpec(List<Message>
conversation, ToolSpecification toolSpec) {
 System.out.println("Calling Bedrock...");

 // Send the conversation, system prompt, and tool configuration, and return
the response
 return bedrockActions.sendConverseRequestAsync(modelId, systemPrompt,
conversation, toolSpec);
}

public static String getUserInput(String prompt, Scanner scanner) {
 String userInput = defaultPrompt;
 if (interactive) {
 System.out.println("*".repeat(80));
 System.out.println(prompt + " (x to exit): \n\t");
 userInput = scanner.nextLine();
 }

 if (userInput == null || userInput.trim().isEmpty()) {
 return getUserInput("\tPlease enter your weather info request, e.g., the
name of a city", scanner);
 }

 if (userInput.equalsIgnoreCase("x")) {
 return null;
 }

 return userInput;
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 }
 }
}

```

```

 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
}

public static void printFooter() {
 System.out.println("""
 =====
 Thank you for checking out the Amazon Bedrock Tool Use demo. We hope
you learned something new, or got some inspiration for your own apps
today!

 For more Bedrock examples in different programming languages, have a
look at:
 https://docs.aws.amazon.com/bedrock/latest/userguide/
service_code_examples.html
 =====
 """);
}
}

```

The weather tool used by the demo. This file defines the tool specification and implements the logic to retrieve weather data using from the Open-Meteo API.

```

public class WeatherTool {

 private static final Logger logger = LoggerFactory.getLogger(WeatherTool.class);
 private static java.net.http.HttpClient httpClient = null;

 /**
 * Returns the JSON Schema specification for the Weather tool. The tool
specification
 * defines the input schema and describes the tool's functionality.
 * For more information, see https://json-schema.org/understanding-json-schema/
reference.
 *
 * @return The tool specification for the Weather tool.
 */
}

```

```

public ToolSpecification getToolSpec() {
 Map<String, Document> latitudeMap = new HashMap<>();
 latitudeMap.put("type", Document.fromString("string"));
 latitudeMap.put("description", Document.fromString("Geographical WGS84
latitude of the location."));

 // Create the nested "longitude" object
 Map<String, Document> longitudeMap = new HashMap<>();
 longitudeMap.put("type", Document.fromString("string"));
 longitudeMap.put("description", Document.fromString("Geographical WGS84
longitude of the location."));

 // Create the "properties" object
 Map<String, Document> propertiesMap = new HashMap<>();
 propertiesMap.put("latitude", Document.fromMap(latitudeMap));
 propertiesMap.put("longitude", Document.fromMap(longitudeMap));

 // Create the "required" array
 List<Document> requiredList = new ArrayList<>();
 requiredList.add(Document.fromString("latitude"));
 requiredList.add(Document.fromString("longitude"));

 // Create the root object
 Map<String, Document> rootMap = new HashMap<>();
 rootMap.put("type", Document.fromString("object"));
 rootMap.put("properties", Document.fromMap(propertiesMap));
 rootMap.put("required", Document.fromList(requiredList));

 // Now create the Document representing the JSON schema
 Document document = Document.fromMap(rootMap);

 ToolSpecification specification = ToolSpecification.builder()
 .name("Weather_Tool")
 .description("Get the current weather for a given location, based on its
WGS84 coordinates.")
 .inputSchema(ToolInputSchema.builder()
 .json(document)
 .build())
 .build();

 return specification;
}

/**

```

```

 * Fetches weather data for the given latitude and longitude.
 *
 * @param latitude the latitude coordinate
 * @param longitude the longitude coordinate
 * @return a {@link CompletableFuture} containing the weather data as a JSON
string
 */
 public Document fetchWeatherData(String latitude, String longitude) {
 HttpClient httpClient = HttpClient.newHttpClient();

 // Ensure no extra double quotes
 latitude = latitude.replace("\"", "");
 longitude = longitude.replace("\"", "");

 String endpoint = "https://api.open-meteo.com/v1/forecast";
 String url = String.format("%s?latitude=%s&longitude=%s¤t_weather=True", endpoint, latitude, longitude);

 HttpRequest request = HttpRequest.newBuilder()
 .uri(URI.create(url))
 .build();

 try {
 HttpResponse<String> response = httpClient.send(request,
 HttpResponse.BodyHandlers.ofString());
 if (response.statusCode() == 200) {
 String weatherJson = response.body();
 System.out.println(weatherJson);
 ObjectMapper objectMapper = new ObjectMapper();
 Map<String, Object> rawMap = objectMapper.readValue(weatherJson, new
 TypeReference<Map<String, Object>>() {});
 Map<String, Document> documentMap = convertToDocumentMap(rawMap);

 Document weatherDocument = Document.fromMap(documentMap);
 System.out.println(weatherDocument);
 return weatherDocument;
 } else {
 throw new RuntimeException("Error fetching weather data: " +
 response.statusCode());
 }
 } catch (Exception e) {
 System.out.println("Error fetching weather data: " + e.getMessage());
 throw new RuntimeException("Error fetching weather data", e);
 }
 }

```

```

 }

 }

 private static Map<String, Document> convertToDocumentMap(Map<String, Object>
inputMap) {
 Map<String, Document> result = new HashMap<>();
 for (Map.Entry<String, Object> entry : inputMap.entrySet()) {
 result.put(entry.getKey(), convertToDocument(entry.getValue()));
 }
 return result;
 }

 // Convert different types of Objects to Document
 private static Document convertToDocument(Object value) {
 if (value instanceof Map) {
 return Document.fromMap(convertToDocumentMap((Map<String, Object>)
value));
 } else if (value instanceof Integer) {
 return Document.fromNumber(SdkNumber.fromInteger((Integer) value));
 } else if (value instanceof Double) { //
 return Document.fromNumber(SdkNumber.fromDouble((Double) value));
 } else if (value instanceof Boolean) {
 return Document.fromBoolean((Boolean) value);
 } else if (value instanceof String) {
 return Document.fromString((String) value);
 }
 return Document.fromNull(); // Handle null values safely
 }
}

```

The Converse API action with a tool configuration.

```

/**
 * Sends an asynchronous converse request to the AI model.
 *
 * @param modelId the unique identifier of the AI model to be used for the
converse request
 * @param systemPrompt the system prompt to be included in the converse request
 * @param conversation a list of messages representing the conversation history
 * @param toolSpec the specification of the tool to be used in the converse
request

```

```

 * @return the converse response received from the AI model
 */
 public ConverseResponse sendConverseRequestAsync(String modelId, String
systemPrompt, List<Message> conversation, ToolSpecification toolSpec) {
 List<Tool> toolList = new ArrayList<>();
 Tool tool = Tool.builder()
 .toolSpec(toolSpec)
 .build();

 toolList.add(tool);

 ToolConfiguration configuration = ToolConfiguration.builder()
 .tools(toolList)
 .build();

 SystemContentBlock block = SystemContentBlock.builder()
 .text(systemPrompt)
 .build();

 ConverseRequest request = ConverseRequest.builder()
 .modelId(modelId)
 .system(block)
 .messages(conversation)
 .toolConfig(configuration)
 .build();

 try {
 ConverseResponse response = getClient().converse(request).join();
 return response;
 } catch (ModelNotReadyException ex) {
 throw new RuntimeException("Model is not ready: " + ex.getMessage(),
ex);
 } catch (BedrockRuntimeException ex) {
 throw new RuntimeException("Failed to converse with Bedrock model: " +
ex.getMessage(), ex);
 }
 }
}

```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

# Amazon Nova

## Converse

The following code example shows how to send a text message to Amazon Nova, using Bedrock's Converse API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Amazon Nova using Bedrock's Converse API with the async Java client.

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

import java.util.concurrent.CompletableFuture;

/**
 * This example demonstrates how to use the Amazon Nova foundation models
 * with an asynchronous Amazon Bedrock runtime client to generate text.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Create a message
 * - Configure and send a request
 * - Process the response
 */
public class ConverseAsync {

 public static String converseAsync() {

 // Step 1: Create the Amazon Bedrock runtime client
 // The runtime client handles the communication with AI models on Amazon
 Bedrock
 BedrockRuntimeAsyncClient client = BedrockRuntimeAsyncClient.builder()
```



```

 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

// Step 2: Specify which model to use
// Available Amazon Nova models and their characteristics:
// - Amazon Nova Micro: Text-only model optimized for lowest latency and
cost
// - Amazon Nova Lite: Fast, low-cost multimodal model for image, video,
and text
// - Amazon Nova Pro: Advanced multimodal model balancing accuracy, speed,
and cost
//
// For the latest available models, see:
// https://docs.aws.amazon.com/bedrock/latest/userguide/models-
supported.html
String modelId = "amazon.nova-lite-v1:0";

// Step 3: Create the message
// The message includes the text prompt and specifies that it comes from the
user
var inputText = "Describe the purpose of a 'hello world' program in one
line.";
var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

// Step 4: Configure the request
// Optional parameters to control the model's response:
// - maxTokens: maximum number of tokens to generate
// - temperature: randomness (max: 1.0, default: 0.7)
// OR
// - topP: diversity of word choice (max: 1.0, default: 0.9)
// Note: Use either temperature OR topP, but not both
ConverseRequest request = ConverseRequest.builder()
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(500) // The maximum response length
 .temperature(0.5F) // Using temperature for
randomness control
 .topP(0.9F) // Alternative: use topP instead of
temperature
)
 .build();

```

```

).build();

 // Step 5: Send and process the request asynchronously
 // - Send the request to the model
 // - Extract and return the generated text from the response
 try {
 CompletableFuture<ConverseResponse> asyncResponse =
client.converse(request);
 return asyncResponse.thenApply(
 response -> response.output().message().content().get(0).text()
).get();

 } catch (Exception e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) {
 String response = converseAsync();
 System.out.println(response);
 }
}

```

Send a text message to Amazon Nova, using Bedrock's Converse API.

```

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

/**
 * This example demonstrates how to use the Amazon Nova foundation models
 * with a synchronous Amazon Bedrock runtime client to generate text.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Create a message
 * - Configure and send a request
 * - Process the response

```

```

*/
public class Converse {

 public static String converse() {

 // Step 1: Create the Amazon Bedrock runtime client
 // The runtime client handles the communication with AI models on Amazon
 Bedrock
 BedrockRuntimeClient client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Step 2: Specify which model to use
 // Available Amazon Nova models and their characteristics:
 // - Amazon Nova Micro: Text-only model optimized for lowest latency and
 cost
 // - Amazon Nova Lite: Fast, low-cost multimodal model for image, video,
 and text
 // - Amazon Nova Pro: Advanced multimodal model balancing accuracy, speed,
 and cost
 //
 // For the latest available models, see:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/models-
 supported.html
 String modelId = "amazon.nova-lite-v1:0";

 // Step 3: Create the message
 // The message includes the text prompt and specifies that it comes from the
 user
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Step 4: Configure the request
 // Optional parameters to control the model's response:
 // - maxTokens: maximum number of tokens to generate
 // - temperature: randomness (max: 1.0, default: 0.7)
 // OR
 // - topP: diversity of word choice (max: 1.0, default: 0.9)
 // Note: Use either temperature OR topP, but not both
 }
}

```

```

 ConverseRequest request = ConverseRequest.builder()
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(500) // The maximum response length
 .temperature(0.5F) // Using temperature for
randomness control
 // .topP(0.9F) // Alternative: use topP instead of
temperature
).build();

 // Step 5: Send and process the request
 // - Send the request to the model
 // - Extract and return the generated text from the response
 try {
 ConverseResponse response = client.converse(request);
 return response.output().message().content().get(0).text();

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) {
 String response = converse();
 System.out.println(response);
 }
}

```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## ConverseStream

The following code example shows how to send a text message to Amazon Nova, using Bedrock's Converse API and process the response stream in real-time.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Amazon Nova using Bedrock's Converse API and process the response stream in real-time.

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

import java.util.concurrent.ExecutionException;

/**
 * This example demonstrates how to use the Amazon Nova foundation models with an
 * asynchronous Amazon Bedrock runtime client to generate streaming text responses.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Create a message
 * - Configure a streaming request
 * - Set up a stream handler to process the response chunks
 * - Process the streaming response
 */
public class ConverseStream {

 public static void converseStream() {

 // Step 1: Create the Amazon Bedrock runtime client
 // The runtime client handles the communication with AI models on Amazon
 Bedrock
 BedrockRuntimeAsyncClient client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Step 2: Specify which model to use
```

```

 // Available Amazon Nova models and their characteristics:
 // - Amazon Nova Micro: Text-only model optimized for lowest latency and
cost
 // - Amazon Nova Lite: Fast, low-cost multimodal model for image, video,
and text
 // - Amazon Nova Pro: Advanced multimodal model balancing accuracy, speed,
and cost
 //
 // For the latest available models, see:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/models-
supported.html
 String modelId = "amazon.nova-lite-v1:0";

 // Step 3: Create the message
 // The message includes the text prompt and specifies that it comes from the
user
 var inputText = "Describe the purpose of a 'hello world' program in one
paragraph";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Step 4: Configure the request
 // Optional parameters to control the model's response:
 // - maxTokens: maximum number of tokens to generate
 // - temperature: randomness (max: 1.0, default: 0.7)
 // OR
 // - topP: diversity of word choice (max: 1.0, default: 0.9)
 // Note: Use either temperature OR topP, but not both
 ConverseStreamRequest request = ConverseStreamRequest.builder()
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(500) // The maximum response length
 .temperature(0.5F) // Using temperature for
randomness control
 // .topP(0.9F) // Alternative: use topP instead of
temperature
).build();

 // Step 5: Set up the stream handler
 // The stream handler processes chunks of the response as they arrive
 // - onContentBlockDelta: Processes each text chunk

```

```

 // - onError: Handles any errors during streaming
 var streamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 System.out.print(chunk.delta().text());
 System.out.flush(); // Ensure immediate output of each
chunk
 }).build())
 .onError(err -> System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage()))
 .build();

 // Step 6: Send the streaming request and process the response
 // - Send the request to the model
 // - Attach the handler to process response chunks as they arrive
 // - Handle any errors during streaming
 try {
 client.converseStream(request, streamHandler).get();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 }
 }

 public static void main(String[] args) {
 converseStream();
 }
}

```

- For API details, see [ConverseStream](#) in *AWS SDK for Java 2.x API Reference*.

## Scenario: Tool use with the Converse API

The following code example shows how to build a typical interaction between an application, a generative AI model, and connected tools or APIs to mediate interactions between the AI and the outside world. It uses the example of connecting an external weather API to the AI model so it can provide real-time weather information based on user input.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The primary execution of the scenario flow. This scenario orchestrates the conversation between the user, the Amazon Bedrock Converse API, and a weather tool.

```
/*
This demo illustrates a tool use scenario using Amazon Bedrock's Converse API and a
weather tool.
The program interacts with a foundation model on Amazon Bedrock to provide weather
information based on user
input. It uses the Open-Meteo API (https://open-meteo.com) to retrieve current
weather data for a given location.
*/
public class BedrockScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static String modelId = "amazon.nova-lite-v1:0";
 private static String defaultPrompt = "What is the weather like in Seattle?";
 private static WeatherTool weatherTool = new WeatherTool();

 // The maximum number of recursive calls allowed in the tool use function.
 // This helps prevent infinite loops and potential performance issues.
 private static int maxRecursions = 5;
 static BedrockActions bedrockActions = new BedrockActions();
 public static boolean interactive = true;

 private static final String systemPrompt = ""
 You are a weather assistant that provides current weather data for user-
specified locations using only
 the Weather_Tool, which expects latitude and longitude. Infer the
coordinates from the location yourself.
 If the user provides coordinates, infer the approximate location and
refer to it in your response.
 To use the tool, you strictly apply the provided tool specification.

 - Explain your step-by-step process, and give brief updates before each
step.
```



- Only use the Weather\_Tool for data. Never guess or make up information.
- Repeat the tool use for subsequent requests if necessary.
- If the tool errors, apologize, explain weather is unavailable, and suggest other options.
- Report temperatures in °C (°F) and wind in km/h (mph). Keep weather reports concise. Sparingly use emojis where appropriate.
- Only respond to weather queries. Remind off-topic users of your purpose.
- Never claim to search online, access external data, or use tools besides Weather\_Tool.
- Complete the entire process until you have all required data before sending the complete response.

```
""";
```

```
public static void main(String[] args) {
 Scanner scanner = new Scanner(System.in);
 System.out.println("""
=====
Welcome to the Amazon Bedrock Tool Use demo!
=====
```

This assistant provides current weather information for user-specified locations.

You can ask for weather details by providing the location name or coordinates.

Example queries:

- What's the weather like in New York?
- Current weather for latitude 40.70, longitude -74.01
- Is it warmer in Rome or Barcelona today?

To exit the program, simply type 'x' and press Enter.

P.S.: You're not limited to single locations, or even to using English!

Have fun and experiment with the app!

```
""");
```

```
System.out.println(DASHES);
```

```
try {
 runConversation(scanner);
```

```
 } catch (Exception ex) {
 System.out.println("There was a problem running the scenario: " +
ex.getMessage());
 }

 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("Amazon Bedrock Converse API with Tool Use Feature
Scenario is complete.");
 System.out.println(DASHES);
}

/**
 * Starts the conversation with the user and handles the interaction with
Bedrock.
 */
private static List<Message> runConversation(Scanner scanner) {
 List<Message> conversation = new ArrayList<>();

 // Get the first user input
 String userInput = getUserInput("Your weather info request:", scanner);
 System.out.println(userInput);

 while (userInput != null) {
 ContentBlock block = ContentBlock.builder()
 .text(userInput)
 .build();

 List<ContentBlock> blockList = new ArrayList<>();
 blockList.add(block);

 Message message = Message.builder()
 .role(ConversationRole.USER)
 .content(blockList)
 .build();

 conversation.add(message);

 // Send the conversation to Amazon Bedrock.
 ConverseResponse bedrockResponse =
sendConversationToBedrock(conversation);
```

```

 // Recursively handle the model's response until the model has returned
 its final response or the recursion counter has reached 0.
 processModelResponse(bedrockResponse, conversation, maxRecursions);

 // Repeat the loop until the user decides to exit the application.
 userInput = getUserInput("Your weather info request:", scanner);
 }
 printFooter();
 return conversation;
}

/**
 * Processes the response from the model and updates the conversation
 accordingly.
 *
 * @param modelResponse the response from the model
 * @param conversation the ongoing conversation
 * @param maxRecursion the maximum number of recursions allowed
 */
private static void processModelResponse(ConverseResponse modelResponse,
List<Message> conversation, int maxRecursion) {
 if (maxRecursion <= 0) {
 // Stop the process, the number of recursive calls could indicate an
 infinite loop
 System.out.println("\tWarning: Maximum number of recursions reached.
 Please try again.");
 }

 // Append the model's response to the ongoing conversation
 conversation.add(modelResponse.output().message());

 String modelResponseVal = modelResponse.stopReasonAsString();
 if (modelResponseVal.compareTo("tool_use") == 0) {
 // If the stop reason is "tool_use", forward everything to the tool use
 handler
 handleToolUse(modelResponse.output(), conversation, maxRecursion - 1);
 }

 if (modelResponseVal.compareTo("end_turn") == 0) {
 // If the stop reason is "end_turn", print the model's response text,
 and finish the process
 PrintModelResponse(modelResponse.output().message().content().get(0).text());
 if (!interactive) {

```

```

 defaultPrompt = "x";
 }
}

/**
 * Handles the use of a tool by the model in a conversation.
 *
 * @param modelResponse the response from the model, which may include a tool
use request
 * @param conversation the current conversation, which will be updated with the
tool use results
 * @param maxRecursion the maximum number of recursive calls allowed to handle
the model's response
 */
private static void handleToolUse(ConverseOutput modelResponse, List<Message>
conversation, int maxRecursion) {
 List<ContentBlock> toolResults = new ArrayList<>();

 // The model's response can consist of multiple content blocks
 for (ContentBlock contentBlock : modelResponse.message().content()) {
 if (contentBlock.text() != null && !contentBlock.text().isEmpty()) {
 // If the content block contains text, print it to the console
 PrintModelResponse(contentBlock.text());
 }

 if (contentBlock.toolUse() != null) {
 ToolResponse toolResponse = invokeTool(contentBlock.toolUse());

 // Add the tool use ID and the tool's response to the list of
results
 List<ToolResultContentBlock> contentBlockList = new ArrayList<>();
 ToolResultContentBlock block = ToolResultContentBlock.builder()
 .json(toolResponse.getContent())
 .build();
 contentBlockList.add(block);

 ToolResultBlock toolResultBlock = ToolResultBlock.builder()
 .toolUseId(toolResponse.getToolUseId())
 .content(contentBlockList)
 .build();

 ContentBlock contentBlock1 = ContentBlock.builder()
 .toolResult(toolResultBlock)

```

```

 .build();

 toolResults.add(contentBlock1);
 }
}

// Embed the tool results in a new user message
Message message = Message.builder()
 .role(ConversationRole.USER)
 .content(toolResults)
 .build();

// Append the new message to the ongoing conversation
//conversation.add(message);
conversation.add(message);

// Send the conversation to Amazon Bedrock
var response = sendConversationToBedrock(conversation);

// Recursively handle the model's response until the model has returned its
final response or the recursion counter has reached 0
processModelResponse(response, conversation, maxRecursion);
}

// Invokes the specified tool with the given payload and returns the tool's
response.
// If the requested tool does not exist, an error message is returned.
private static ToolResponse invokeTool(ToolUseBlock payload) {
 String toolName = payload.name();

 if (Objects.equals(toolName, "Weather_Tool")) {
 Map<String, Document> inputData = payload.input().asMap();
 printToolUse(toolName, inputData);

 // Invoke the weather tool with the input data provided
 Document weatherResponse =
weatherTool.fetchWeatherData(inputData.get("latitude").toString(),
inputData.get("longitude").toString());

 ToolResponse toolResponse = new ToolResponse();
 toolResponse.setContent(weatherResponse);
 toolResponse.setToolUseId(payload.toolUseId());
 return toolResponse;
 } else {

```

```

 String errorMessage = "The requested tool with name " + toolName + "
does not exist.";
 System.out.println(errorMessage);
 return null;
 }
}

public static void printToolUse(String toolName, Map<String, Document>
inputData) {
 System.out.println("Invoking tool: " + toolName + " with input: " +
inputData.get("latitude").toString() + ", " + inputData.get("longitude").toString()
+ "...");
}

private static void PrintModelResponse(String message) {
 System.out.println("\tThe model's response:\n");
 System.out.println(message);
 System.out.println("");
}

private static ConverseResponse sendConversationToBedrock(List<Message>
conversation) {
 System.out.println("Calling Bedrock...");

 try {
 return bedrockActions.sendConverseRequestAsync(modelId, systemPrompt,
conversation, weatherTool.getToolSpec());
 } catch (ModelNotReadyException ex) {
 System.err.println("Model is not ready. Please try again later: " +
ex.getMessage());
 throw ex;
 } catch (BedrockRuntimeException ex) {
 System.err.println("Bedrock service error: " + ex.getMessage());
 throw ex;
 } catch (RuntimeException ex) {
 System.err.println("Unexpected error occurred: " + ex.getMessage());
 throw ex;
 }
}

private static ConverseResponse sendConversationToBedrockwithSpec(List<Message>
conversation, ToolSpecification toolSpec) {
 System.out.println("Calling Bedrock...");

```

```

 // Send the conversation, system prompt, and tool configuration, and return
 the response
 return bedrockActions.sendConverseRequestAsync(modelId, systemPrompt,
conversation, toolSpec);
 }

 public static String getUserInput(String prompt, Scanner scanner) {
 String userInput = defaultPrompt;
 if (interactive) {
 System.out.println("*".repeat(80));
 System.out.println(prompt + " (x to exit): \n\t");
 userInput = scanner.nextLine();
 }

 if (userInput == null || userInput.trim().isEmpty()) {
 return getUserInput("\tPlease enter your weather info request, e.g., the
name of a city", scanner);
 }

 if (userInput.equalsIgnoreCase("x")) {
 return null;
 }

 return userInput;
 }

 private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
 }
 }

 public static void printFooter() {

```

```

 System.out.println("""
 =====
 Thank you for checking out the Amazon Bedrock Tool Use demo. We hope
you learned something new, or got some inspiration for your own apps
today! For more Bedrock examples in different programming languages, have a
look at: https://docs.aws.amazon.com/bedrock/latest/userguide/
service_code_examples.html
 =====
 """);
 }
}

```

The weather tool used by the demo. This file defines the tool specification and implements the logic to retrieve weather data using from the Open-Meteo API.

```

public class WeatherTool {

 private static final Logger logger = LoggerFactory.getLogger(WeatherTool.class);
 private static java.net.http.HttpClient httpClient = null;

 /**
 * Returns the JSON Schema specification for the Weather tool. The tool
specification
 * defines the input schema and describes the tool's functionality.
 * For more information, see https://json-schema.org/understanding-json-schema/
reference.
 *
 * @return The tool specification for the Weather tool.
 */
 public ToolSpecification getToolSpec() {
 Map<String, Document> latitudeMap = new HashMap<>();
 latitudeMap.put("type", Document.fromString("string"));
 latitudeMap.put("description", Document.fromString("Geographical WGS84
latitude of the location."));

 // Create the nested "longitude" object
 Map<String, Document> longitudeMap = new HashMap<>();
 longitudeMap.put("type", Document.fromString("string"));
 }
}

```



```

 longitudeMap.put("description", Document.fromString("Geographical WGS84
longitude of the location.));

 // Create the "properties" object
 Map<String, Document> propertiesMap = new HashMap<>();
 propertiesMap.put("latitude", Document.fromMap(latitudeMap));
 propertiesMap.put("longitude", Document.fromMap(longitudeMap));

 // Create the "required" array
 List<Document> requiredList = new ArrayList<>();
 requiredList.add(Document.fromString("latitude"));
 requiredList.add(Document.fromString("longitude"));

 // Create the root object
 Map<String, Document> rootMap = new HashMap<>();
 rootMap.put("type", Document.fromString("object"));
 rootMap.put("properties", Document.fromMap(propertiesMap));
 rootMap.put("required", Document.fromList(requiredList));

 // Now create the Document representing the JSON schema
 Document document = Document.fromMap(rootMap);

 ToolSpecification specification = ToolSpecification.builder()
 .name("Weather_Tool")
 .description("Get the current weather for a given location, based on its
WGS84 coordinates.")
 .inputSchema(ToolInputSchema.builder()
 .json(document)
 .build())
 .build();

 return specification;
 }

 /**
 * Fetches weather data for the given latitude and longitude.
 *
 * @param latitude the latitude coordinate
 * @param longitude the longitude coordinate
 * @return a {@link CompletableFuture} containing the weather data as a JSON
string
 */
 public Document fetchWeatherData(String latitude, String longitude) {
 HttpClient httpClient = HttpClient.newHttpClient();

```

```

 // Ensure no extra double quotes
 latitude = latitude.replace("\"", "");
 longitude = longitude.replace("\"", "");

 String endpoint = "https://api.open-meteo.com/v1/forecast";
 String url = String.format("%s?latitude=%s&longitude=%s¤t_weather=True", endpoint, latitude, longitude);

 HttpRequest request = HttpRequest.newBuilder()
 .uri(URI.create(url))
 .build();

 try {
 HttpResponse<String> response = httpClient.send(request,
 HttpResponse.BodyHandlers.ofString());
 if (response.statusCode() == 200) {
 String weatherJson = response.body();
 System.out.println(weatherJson);
 ObjectMapper objectMapper = new ObjectMapper();
 Map<String, Object> rawMap = objectMapper.readValue(weatherJson, new
 TypeReference<Map<String, Object>>() {});
 Map<String, Document> documentMap = convertToDocumentMap(rawMap);

 Document weatherDocument = Document.fromMap(documentMap);
 System.out.println(weatherDocument);
 return weatherDocument;
 } else {
 throw new RuntimeException("Error fetching weather data: " +
 response.statusCode());
 }
 } catch (Exception e) {
 System.out.println("Error fetching weather data: " + e.getMessage());
 throw new RuntimeException("Error fetching weather data", e);
 }
 }

 private static Map<String, Document> convertToDocumentMap(Map<String, Object>
 inputMap) {
 Map<String, Document> result = new HashMap<>();
 for (Map.Entry<String, Object> entry : inputMap.entrySet()) {
 result.put(entry.getKey(), convertToDocument(entry.getValue()));
 }
 }

```

```

 }
 return result;
}

// Convert different types of Objects to Document
private static Document convertToDocument(Object value) {
 if (value instanceof Map) {
 return Document.fromMap(convertToDocumentMap((Map<String, Object>)
value));
 } else if (value instanceof Integer) {
 return Document.fromNumber(SdkNumber.fromInteger((Integer) value));
 } else if (value instanceof Double) { //
 return Document.fromNumber(SdkNumber.fromDouble((Double) value));
 } else if (value instanceof Boolean) {
 return Document.fromBoolean((Boolean) value);
 } else if (value instanceof String) {
 return Document.fromString((String) value);
 }
 return Document.fromNull(); // Handle null values safely
}
}

```

The Converse API action with a tool configuration.

```

/**
 * Sends an asynchronous converse request to the AI model.
 *
 * @param modelId the unique identifier of the AI model to be used for the
converse request
 * @param systemPrompt the system prompt to be included in the converse request
 * @param conversation a list of messages representing the conversation history
 * @param toolSpec the specification of the tool to be used in the converse
request
 * @return the converse response received from the AI model
 */
public ConverseResponse sendConverseRequestAsync(String modelId, String
systemPrompt, List<Message> conversation, ToolSpecification toolSpec) {
 List<Tool> toolList = new ArrayList<>();
 Tool tool = Tool.builder()
 .toolSpec(toolSpec)
 .build();
}

```

```
toolList.add(tool);

ToolConfiguration configuration = ToolConfiguration.builder()
 .tools(toolList)
 .build();

SystemContentBlock block = SystemContentBlock.builder()
 .text(systemPrompt)
 .build();

ConverseRequest request = ConverseRequest.builder()
 .modelId(modelId)
 .system(block)
 .messages(conversation)
 .toolConfig(configuration)
 .build();

try {
 ConverseResponse response = getClient().converse(request).join();
 return response;
} catch (ModelNotReadyException ex) {
 throw new RuntimeException("Model is not ready: " + ex.getMessage(),
ex);
} catch (BedrockRuntimeException ex) {
 throw new RuntimeException("Failed to converse with Bedrock model: " +
ex.getMessage(), ex);
}
}
```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Nova Canvas

### InvokeModel

The following code example shows how to invoke Amazon Nova Canvas on Amazon Bedrock to generate an image.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create an image with Amazon Nova Canvas.

```
import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.InvokeModelResponse;

import java.security.SecureRandom;
import java.util.Base64;

import static com.example.bedrockruntime.libs.ImageTools.displayImage;

/**
 * This example demonstrates how to use Amazon Nova Canvas to generate images.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Configure the image generation parameters
 * - Send a request to generate an image
 * - Process the response and handle the generated image
 */
public class InvokeModel {

 public static byte[] invokeModel() {

 // Step 1: Create the Amazon Bedrock runtime client
 // The runtime client handles the communication with AI models on Amazon
 Bedrock
 BedrockRuntimeClient client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
```

```

 .build();

// Step 2: Specify which model to use
// For the latest available models, see:
// https://docs.aws.amazon.com/bedrock/latest/userguide/models-
supported.html
String modelId = "amazon.nova-canvas-v1:0";

// Step 3: Configure the generation parameters and create the request
// First, set the main parameters:
// - prompt: Text description of the image to generate
// - seed: Random number for reproducible generation (0 to 858,993,459)
String prompt = "A stylized picture of a cute old steampunk robot";
int seed = new SecureRandom().nextInt(858_993_460);

// Then, create the request using a template with the following structure:
// - taskType: TEXT_IMAGE (specifies text-to-image generation)
// - textToImageParams: Contains the text prompt
// - imageGenerationConfig: Contains optional generation settings (seed,
quality, etc.)
// For a list of available request parameters, see:
// https://docs.aws.amazon.com/nova/latest/userguide/image-gen-req-resp-
structure.html
String request = ""
 {
 "taskType": "TEXT_IMAGE",
 "textToImageParams": {
 "text": "{{prompt}}"
 },
 "imageGenerationConfig": {
 "seed": {{seed}},
 "quality": "standard"
 }
 }""
 .replace("{{prompt}}", prompt)
 .replace("{{seed}}", String.valueOf(seed));

// Step 4: Send and process the request
// - Send the request to the model using InvokeModelResponse
// - Extract the Base64-encoded image from the JSON response
// - Convert the encoded image to a byte array and return it
try {
 InvokeModelResponse response = client.invokeModel(builder -> builder
 .modelId(modelId)

```

```

 .body(SdkBytes.fromUtf8String(request))
);

 JSONObject responseBody = new
 JSONObject(response.body().asUtf8String());
 // Convert the Base64 string to byte array for better handling
 return Base64.getDecoder().decode(
 new JSONPointer("/images/0").queryFrom(responseBody).toString()
);

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s\n", modelId,
e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 System.out.println("Generating image. This may take a few seconds...");
 byte[] imageData = invokeModel();
 displayImage(imageData);
}
}

```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Titan Image Generator

### InvokeModel

The following code example shows how to invoke Amazon Titan Image on Amazon Bedrock to generate an image.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Create an image with the Amazon Titan Image Generator.

```
// Create an image with the Amazon Titan Image Generator.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

import java.math.BigInteger;
import java.security.SecureRandom;

import static com.example.bedrockruntime.libs.ImageTools.displayImage;

public class InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Titan Image G2.
 var modelId = "amazon.titan-image-generator-v2:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 titan-image.html
 var nativeRequestTemplate = ""
 {
 "taskType": "TEXT_IMAGE",
 "textToImageParams": { "text": "{{prompt}}" },
 "imageGenerationConfig": { "seed": {{seed}} }
 }"";
 }
}
```



```

// Define the prompt for the image generation.
var prompt = "A stylized picture of a cute old steampunk robot";

// Get a random 31-bit seed for the image generation (max. 2,147,483,647).
var seed = new BigInteger(31, new SecureRandom());

// Embed the prompt and seed in the model's native request payload.
var nativeRequest = nativeRequestTemplate
 .replace("{{prompt}}", prompt)
 .replace("{{seed}}", seed.toString());

try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

 // Retrieve the generated image data from the model's response.
 var base64ImageData = new JSONObject("/
images/0").queryFrom(responseBody).toString();

 return base64ImageData;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 System.out.println("Generating image. This may take a few seconds...");

 String base64ImageData = invokeModel();

 displayImage(base64ImageData);
}
}

```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Titan Text Embeddings

### InvokeModel

The following code example shows how to:

- Get started creating your first embedding.
- Create embeddings configuring the number of dimensions and normalization (V2 only).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create your first embedding with Titan Text Embeddings V2.

```
// Generate and print an embedding with Amazon Titan Text Embeddings.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

public class InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
```

```

 .build();

// Set the model ID, e.g., Titan Text Embeddings V2.
var modelId = "amazon.titan-embed-text-v2:0";

// The InvokeModel API uses the model's native payload.
// Learn more about the available inference parameters and response fields
at:
// https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
titan-embed-text.html
var nativeRequestTemplate = "{ \"inputText\": \"{{inputText}}\" }";

// The text to convert into an embedding.
var inputText = "Please recommend books with a theme similar to the movie
'Inception'.";

// Embed the prompt in the model's native request payload.
String nativeRequest = nativeRequestTemplate.replace("{{inputText}}",
inputText);

try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

 // Retrieve the generated text from the model's response.
 var text = new JSONPointer("/
embedding").queryFrom(responseBody).toString();
 System.out.println(text);

 return text;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}
}

```

```

 public static void main(String[] args) {
 invokeModel();
 }
}

```

Invoke Titan Text Embeddings V2 configuring the number of dimensions and normalization.

```

/**
 * Invoke Amazon Titan Text Embeddings V2 with additional inference parameters.
 *
 * @param inputText - The text to convert to an embedding.
 * @param dimensions - The number of dimensions the output embeddings should
have.
 *
 * Values accepted by the model: 256, 512, 1024.
 * @param normalize - A flag indicating whether or not to normalize the output
embeddings.
 * @return The {@link JSONObject} representing the model's response.
 */
public static JSONObject invokeModel(String inputText, int dimensions, boolean
normalize) {

 // Create a Bedrock Runtime client in the AWS Region of your choice.
 var client = BedrockRuntimeClient.builder()
 .region(Region.US_WEST_2)
 .build();

 // Set the model ID, e.g., Titan Embed Text v2.0.
 var modelId = "amazon.titan-embed-text-v2:0";

 // Create the request for the model.
 var nativeRequest = ""
 {
 "inputText": "%s",
 "dimensions": %d,
 "normalize": %b
 }
 "".formatted(inputText, dimensions, normalize);

 // Encode and send the request.
 var response = client.invokeModel(request -> {
 request.body(SdkBytes.fromUtf8String(nativeRequest));
 });
}

```

```
 request.modelId(modelId);
 });

 // Decode the model's response.
 var modelResponse = new JSONObject(response.body().asUtf8String());

 // Extract and print the generated embedding and the input text token count.
 var embedding = modelResponse.getJSONArray("embedding");
 var inputTokenCount = modelResponse.getBigInteger("inputTextTokenCount");
 System.out.println("Embedding: " + embedding);
 System.out.println("\nInput token count: " + inputTokenCount);

 // Return the model's native response.
 return modelResponse;
}
```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## Anthropic Claude

### Converse

The following code example shows how to send a text message to Anthropic Claude, using Bedrock's Converse API.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Anthropic Claude, using Bedrock's Converse API.

```
// Use the Converse API to send a text message to Anthropic Claude.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
```

```
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.ConverseResponse;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

public class Converse {

 public static String converse() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Claude 3 Haiku.
 var modelId = "anthropic.claude-3-haiku-20240307-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 try {
 // Send the message with a basic inference configuration.
 ConverseResponse response = client.converse(request -> request
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)));

 // Retrieve the generated text from Bedrock's response object.
 var responseText =
 response.output().message().content().getFirst().text();
 System.out.println(responseText);
 }
```

```

 return responseText;

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 converse();
}
}

```

Send a text message to Anthropic Claude, using Bedrock's Converse API with the async Java client.

```

// Use the Converse API to send a text message to Anthropic Claude
// with the async Java client.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

public class ConverseAsync {

 public static String converseAsync() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();
 }
}

```

```

// Set the model ID, e.g., Claude 3 Haiku.
var modelId = "anthropic.claude-3-haiku-20240307-v1:0";

// Create the input text and embed it in a message object with the user
role.
var inputText = "Describe the purpose of a 'hello world' program in one
line.";
var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

// Send the message with a basic inference configuration.
var request = client.converse(params -> params
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F))
);

// Prepare a future object to handle the asynchronous response.
CompletableFuture<String> future = new CompletableFuture<>();

// Handle the response or error using the future object.
request.whenComplete((response, error) -> {
 if (error == null) {
 // Extract the generated text from Bedrock's response object.
 String responseText =
response.output().message().content().getFirst().text();
 future.complete(responseText);
 } else {
 future.completeExceptionally(error);
 }
});

try {
 // Wait for the future object to complete and retrieve the generated
text.
 String responseText = future.get();
 System.out.println(responseText);
}

```



```

 return responseText;

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 converseAsync();
}
}

```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## ConverseStream

The following code example shows how to send a text message to Anthropic Claude, using Bedrock's Converse API and process the response stream in real-time.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Anthropic Claude, using Bedrock's Converse API and process the response stream in real-time.

```

// Use the Converse API to send a text message to Anthropic Claude
// and print the response stream.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import
 software.amazon.awssdk.services.bedrockruntime.model.ConverseStreamResponseHandler;

```

```

import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.ExecutionException;

public class ConverseStream {

 public static void main(String[] args) {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Claude 3 Haiku.
 var modelId = "anthropic.claude-3-haiku-20240307-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Create a handler to extract and print the response text in real-time.
 var responseStreamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 String responseText = chunk.delta().text();
 System.out.print(responseText);
 }).build())
 .onError(err ->
 System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage()))
 .build();

 try {
 // Send the message with a basic inference configuration and attach the
 handler.
 client.converseStream(request -> request.modelId(modelId)

```

```

 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)
), responseStreamHandler).get();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
 e.getCause().getMessage());
 }
}
}

```

- For API details, see [ConverseStream](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModel

The following code example shows how to send a text message to Anthropic Claude, using the Invoke Model API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message.

```

// Use the native inference API to send a text message to Anthropic Claude.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

```

```

public class InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Claude 3 Haiku.
 var modelId = "anthropic.claude-3-haiku-20240307-v1:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 anthropic-claude-messages.html
 var nativeRequestTemplate = """
 {
 "anthropic_version": "bedrock-2023-05-31",
 "max_tokens": 512,
 "temperature": 0.5,
 "messages": [{
 "role": "user",
 "content": "{{prompt}}"
 }]
 }""";

 // Define the prompt for the model.
 var prompt = "Describe the purpose of a 'hello world' program in one line.";

 // Embed the prompt in the model's native request payload.
 String nativeRequest = nativeRequestTemplate.replace("{{prompt}}", prompt);

 try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);
 }
 }
}

```

```
// Decode the response body.
var responseBody = new JSONObject(response.body().asUtf8String());

// Retrieve the generated text from the model's response.
var text = new JSONPointer("/content/0/
text").queryFrom(responseBody).toString();
System.out.println(text);

return text;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 invokeModel();
}

}
```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModelWithResponseStream

The following code example shows how to send a text message to Anthropic Claude models, using the Invoke Model API, and print the response stream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message and process the response stream in real-time.

```
// Use the native inference API to send a text message to Anthropic Claude
// and print the response stream.
```

```

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamRequest;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

import java.util.Objects;
import java.util.concurrent.ExecutionException;

import static
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

public class InvokeModelWithResponseStream {

 public static String invokeModelWithResponseStream() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Claude 3 Haiku.
 var modelId = "anthropic.claude-3-haiku-20240307-v1:0";

 // The InvokeModelWithResponseStream API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 anthropic-claude-messages.html
 var nativeRequestTemplate = ""
 {
 "anthropic_version": "bedrock-2023-05-31",
 "max_tokens": 512,
 "temperature": 0.5,
 "messages": [{

```

```

 "role": "user",
 "content": "{{prompt}}"
 }}
}""";

// Define the prompt for the model.
var prompt = "Describe the purpose of a 'hello world' program in one line.";

// Embed the prompt in the model's native request payload.
String nativeRequest = nativeRequestTemplate.replace("{{prompt}}", prompt);

// Create a request with the model ID and the model's native request
payload.
var request = InvokeModelWithResponseStreamRequest.builder()
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
 .build();

// Prepare a buffer to accumulate the generated response text.
var completeResponseTextBuffer = new StringBuilder();

// Prepare a handler to extract, accumulate, and print the response text in
real-time.
var responseStreamHandler =
InvokeModelWithResponseStreamResponseHandler.builder()
 .subscriber(Visitor.builder().onChunk(chunk -> {
 var response = new JSONObject(chunk.bytes().asUtf8String());

 // Extract and print the text from the content blocks.
 if (Objects.equals(response.getString("type"),
"content_block_delta")) {
 var text = new JSONPointer("/delta/
text").queryFrom(response);
 System.out.print(text);

 // Append the text to the response text buffer.
 completeResponseTextBuffer.append(text);
 }
 })).build()).build();

try {
 // Send the request and wait for the handler to process the response.
 client.invokeModelWithResponseStream(request,
responseStreamHandler).get();

```

```

 // Return the complete response text.
 return completeResponseTextBuffer.toString();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
 e.getCause().getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) throws ExecutionException,
 InterruptedException {
 invokeModelWithResponseStream();
}
}

```

- For API details, see [InvokeModelWithResponseStream](#) in *AWS SDK for Java 2.x API Reference*.

## Reasoning

The following code example shows how to use Anthropic Claude 3.7 Sonnet's reasoning capability on Amazon Bedrock

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use Anthropic Claude 3.7 Sonnet's reasoning capability with the asynchronous Bedrock runtime client.

```

import com.example.bedrockruntime.models.anthropicClaude.lib.ReasoningResponse;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;

```



```
import software.amazon.awssdk.services.bedrockruntime.model.*;

import java.util.concurrent.CompletableFuture;

/**
 * This example demonstrates how to use Anthropic Claude 3.7 Sonnet's reasoning
 * capability
 * with an asynchronous Amazon Bedrock runtime client.
 * It shows how to:
 * - Set up the Amazon Bedrock async runtime client
 * - Create a message
 * - Configure reasoning parameters
 * - Send an asynchronous request with reasoning enabled
 * - Process both the reasoning output and final response
 */
public class ReasoningAsync {

 public static ReasoningResponse reasoningAsync() {

 // Create the Amazon Bedrock runtime client
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Specify the model ID. For the latest available models, see:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/models-
supported.html
 var modelId = "us.anthropic.claude-sonnet-4-20250514-v1:0";

 // Create the message with the user's prompt
 var prompt = "Describe the purpose of a 'hello world' program in one line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(prompt))
 .role(ConversationRole.USER)
 .build();

 // Configure reasoning parameters with a 2000 token budget
 Document reasoningConfig = Document.mapBuilder()
 .putDocument("thinking", Document.mapBuilder()
 .putString("type", "enabled")
 .putNumber("budget_tokens", 2000)
 .build())
 .build();
 }
}
```

```

 try {
 // Send message and reasoning configuration to the model
 CompletableFuture<ConverseResponse> asyncResponse =
client.converse(request -> request
 .additionalModelRequestFields(reasoningConfig)
 .messages(message)
 .modelId(modelId)
);

 // Process the response asynchronously
 return asyncResponse.thenApply(response -> {

 var content = response.output().message().content();
 ReasoningContentBlock reasoning = null;
 String text = null;

 // Process each content block to find reasoning and response
text
 for (ContentBlock block : content) {
 if (block.reasoningContent() != null) {
 reasoning = block.reasoningContent();
 } else if (block.text() != null) {
 text = block.text();
 }
 }

 return new ReasoningResponse(reasoning, text);
 })
 .get();

 } catch (Exception e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) {
 // Execute the example and display reasoning and final response
 ReasoningResponse response = reasoningAsync();
 System.out.println("\n<thinking>");
 System.out.println(response.reasoning().reasoningText());
 System.out.println("</thinking>\n");
 System.out.println(response.text());
 }

```

```
 }
}
```

Use Anthropic Claude 3.7 Sonnet's reasoning capability with the synchronous Bedrock runtime client.

```
import com.example.bedrockruntime.models.anthropicClaude.lib.ReasoningResponse;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

/**
 * This example demonstrates how to use Anthropic Claude 3.7 Sonnet's reasoning
 * capability
 * with the synchronous Amazon Bedrock runtime client.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Create a message
 * - Configure reasoning parameters
 * - Send a request with reasoning enabled
 * - Process both the reasoning output and final response
 */
public class Reasoning {

 public static ReasoningResponse reasoning() {

 // Create the Amazon Bedrock runtime client
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Specify the model ID. For the latest available models, see:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/models-
 // supported.html
 var modelId = "us.anthropic.claude-sonnet-4-20250514-v1:0";

 // Create the message with the user's prompt
```

```

var prompt = "Describe the purpose of a 'hello world' program in one line.";
var message = Message.builder()
 .content(ContentBlock.fromText(prompt))
 .role(ConversationRole.USER)
 .build();

// Configure reasoning parameters with a 2000 token budget
Document reasoningConfig = Document.mapBuilder()
 .putDocument("thinking", Document.mapBuilder()
 .putString("type", "enabled")
 .putNumber("budget_tokens", 2000)
 .build())
 .build();

try {
 // Send message and reasoning configuration to the model
 ConverseResponse bedrockResponse = client.converse(request -> request
 .additionalModelRequestFields(reasoningConfig)
 .messages(message)
 .modelId(modelId)
);

 // Extract both reasoning and final response
 var content = bedrockResponse.output().message().content();
 ReasoningContentBlock reasoning = null;
 String text = null;

 // Process each content block to find reasoning and response text
 for (ContentBlock block : content) {
 if (block.reasoningContent() != null) {
 reasoning = block.reasoningContent();
 } else if (block.text() != null) {
 text = block.text();
 }
 }

 return new ReasoningResponse(reasoning, text);

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

```

```
}

public static void main(String[] args) {
 // Execute the example and display reasoning and final response
 ReasoningResponse response = reasoning();
 System.out.println("\n<thinking>");
 System.out.println(response.reasoning().reasoningText());
 System.out.println("</thinking>\n");
 System.out.println(response.text());
}
}
```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## Reasoning with a streaming response

The following code example shows how to use Anthropic Claude 3.7 Sonnet's reasoning capability on Amazon Bedrock

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use Anthropic Claude 3.7 Sonnet's reasoning capability to generate streaming text responses.

```
import com.example.bedrockruntime.models.anthropicClaude.lib.ReasoningResponse;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.*;

import java.util.concurrent.ExecutionException;
import java.util.concurrent.atomic.AtomicReference;
```

```
/**
 * This example demonstrates how to use Anthropic Claude 3.7 Sonnet's reasoning
 * capability to generate streaming text responses.
 * It shows how to:
 * - Set up the Amazon Bedrock runtime client
 * - Create a message
 * - Configure a streaming request
 * - Set up a stream handler to process the response chunks
 * - Process the streaming response
 */
public class ReasoningStream {

 public static ReasoningResponse reasoningStream() {

 // Create the Amazon Bedrock runtime client
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Specify the model ID. For the latest available models, see:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/models-
supported.html
 var modelId = "us.anthropic.claude-sonnet-4-20250514-v1:0";

 // Create the message with the user's prompt
 var prompt = "Describe the purpose of a 'hello world' program in one line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(prompt))
 .role(ConversationRole.USER)
 .build();

 // Configure reasoning parameters with a 2000 token budget
 Document reasoningConfig = Document.mapBuilder()
 .putDocument("thinking", Document.mapBuilder()
 .putString("type", "enabled")
 .putNumber("budget_tokens", 2000)
 .build())
 .build();

 // Configure the request with the message, model ID, and reasoning config
 ConverseStreamRequest request = ConverseStreamRequest.builder()
 .additionalModelRequestFields(reasoningConfig)
 .messages(message)
```

```

 .modelId(modelId)
 .build();

 StringBuilder reasoning = new StringBuilder();
 StringBuilder text = new StringBuilder();
 AtomicReference<ReasoningResponse> finalresponse = new AtomicReference<>();

 // Set up the stream handler to processes chunks of the response as they
arrive
 var streamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 ContentBlockDelta delta = chunk.delta();
 if (delta.reasoningContent() != null) {
 if (reasoning.isEmpty()) {
 System.out.println("\n<thinking>");
 }
 if (delta.reasoningContent().text() != null) {

System.out.print(delta.reasoningContent().text());

reasoning.append(delta.reasoningContent().text());
 }
 } else if (delta.text() != null) {
 if (text.isEmpty()) {
 System.out.println("\n</thinking>\n");
 }
 System.out.print(delta.text());
 text.append(delta.text());
 }
 System.out.flush(); // Ensure immediate output of each
chunk

 }).build())
 .onComplete(() -> finalresponse.set(new ReasoningResponse(
 ReasoningContentBlock.fromReasoningText(t ->
t.text(reasoning.toString()),
 text.toString()
)))
 .onError(err -> System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage()))
 .build();

 // Step 6: Send the streaming request and process the response
 // - Send the request to the model

```

```
// - Attach the handler to process response chunks as they arrive
// - Handle any errors during streaming
try {
 client.converseStream(request, streamHandler).get();
 return finalresponse.get();

} catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 throw new RuntimeException(e);
} catch (Exception e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 reasoningStream();
}
}
```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## Cohere Command

### Converse

The following code example shows how to send a text message to Cohere Command, using Bedrock's Converse API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Cohere Command, using Bedrock's Converse API.



```
// Use the Converse API to send a text message to Cohere Command.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.ConverseResponse;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

public class Converse {

 public static String converse() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Command R.
 var modelId = "cohere.command-r-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 try {
 // Send the message with a basic inference configuration.
 ConverseResponse response = client.converse(request -> request
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
```

```

 .temperature(0.5F)
 .topP(0.9F)));

 // Retrieve the generated text from Bedrock's response object.
 var responseText = response.output().message().content().get(0).text();
 System.out.println(responseText);

 return responseText;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 converse();
}
}

```

Send a text message to Cohere Command, using Bedrock's Converse API with the async Java client.

```

// Use the Converse API to send a text message to Cohere Command
// with the async Java client.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

public class ConverseAsync {

 public static String converseAsync() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.

```

```

// Replace the DefaultCredentialsProvider with your preferred credentials
provider.
var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

// Set the model ID, e.g., Command R.
var modelId = "cohere.command-r-v1:0";

// Create the input text and embed it in a message object with the user
role.
var inputText = "Describe the purpose of a 'hello world' program in one
line.";
var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

// Send the message with a basic inference configuration.
var request = client.converse(params -> params
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F))
);

// Prepare a future object to handle the asynchronous response.
CompletableFuture<String> future = new CompletableFuture<>();

// Handle the response or error using the future object.
request.whenComplete((response, error) -> {
 if (error == null) {
 // Extract the generated text from Bedrock's response object.
 String responseText =
response.output().message().content().get(0).text();
 future.complete(responseText);
 } else {
 future.completeExceptionally(error);
 }
});

```

```
 try {
 // Wait for the future object to complete and retrieve the generated
 text.

 String responseText = future.get();
 System.out.println(responseText);

 return responseText;

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) {
 converseAsync();
 }
}
```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## ConverseStream

The following code example shows how to send a text message to Cohere Command, using Bedrock's Converse API and process the response stream in real-time.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Cohere Command, using Bedrock's Converse API and process the response stream in real-time.

```
// Use the Converse API to send a text message to Cohere Command
// and print the response stream.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
```

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import
 software.amazon.awssdk.services.bedrockruntime.model.ConverseStreamResponseHandler;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.ExecutionException;

public class ConverseStream {

 public static void main(String[] args) {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Command R.
 var modelId = "cohere.command-r-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Create a handler to extract and print the response text in real-time.
 var responseStreamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 String responseText = chunk.delta().text();
 System.out.print(responseText);
 }).build()
).onError(err ->
 System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage())
)
}

```

```

).build();

 try {
 // Send the message with a basic inference configuration and attach the
 handler.
 client.converseStream(request -> request.modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)
), responseStreamHandler).get();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
 e.getCause().getMessage());
 }
}
}

```

- For API details, see [ConverseStream](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModel: Command R and R+

The following code example shows how to send a text message to Cohere Command R and R+, using the Invoke Model API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message.

```

// Use the native inference API to send a text message to Cohere Command R.

import org.json.JSONObject;
import org.json.JSONPointer;

```

```

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

public class Command_R_InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Command R.
 var modelId = "cohere.command-r-v1:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 cohere-command-r-plus.html
 var nativeRequestTemplate = "{ \"message\": \"{{prompt}}\" }";

 // Define the prompt for the model.
 var prompt = "Describe the purpose of a 'hello world' program in one line.";

 // Embed the prompt in the model's native request payload.
 String nativeRequest = nativeRequestTemplate.replace("{{prompt}}", prompt);

 try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

```

```

 // Retrieve the generated text from the model's response.
 var text = new JSONPointer("/text").queryFrom(responseBody).toString();
 System.out.println(text);

 return text;

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
 e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 invokeModel();
}
}

```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModelWithResponseStream: Command R and R+

The following code example shows how to send a text message to Cohere Command, using the Invoke Model API with a response stream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message and process the response stream in real-time.

```

// Use the native inference API to send a text message to Cohere Command R
// and print the response stream.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;

```



```

import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamRequest;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

import java.util.concurrent.ExecutionException;

import static
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

public class Command_R_InvokeModelWithResponseStream {

 public static String invokeModelWithResponseStream() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Command R.
 var modelId = "cohere.command-r-v1:0";

 // The InvokeModelWithResponseStream API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 cohere-command-r-plus.html
 var nativeRequestTemplate = "{ \"message\": \"{{prompt}}\" }";

 // Define the prompt for the model.
 var prompt = "Describe the purpose of a 'hello world' program in one line.";

 // Embed the prompt in the model's native request payload.
 String nativeRequest = nativeRequestTemplate.replace("{{prompt}}", prompt);

 // Create a request with the model ID and the model's native request
 payload.
 var request = InvokeModelWithResponseStreamRequest.builder()

```

```

 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
 .build();

// Prepare a buffer to accumulate the generated response text.
var completeResponseTextBuffer = new StringBuilder();

// Prepare a handler to extract, accumulate, and print the response text in
real-time.
var responseStreamHandler =
InvokeModelWithResponseStreamResponseHandler.builder()
 .subscriber(Visitor.builder().onChunk(chunk -> {
 // Extract and print the text from the model's native response.
 var response = new JSONObject(chunk.bytes().asUtf8String());
 var text = new JSONPointer("/text").queryFrom(response);
 System.out.print(text);

 // Append the text to the response text buffer.
 completeResponseTextBuffer.append(text);
 })).build()).build();

try {
 // Send the request and wait for the handler to process the response.
 client.invokeModelWithResponseStream(request,
responseStreamHandler).get();

 // Return the complete response text.
 return completeResponseTextBuffer.toString();

} catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) throws ExecutionException,
InterruptedException {
 invokeModelWithResponseStream();
}
}

```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

# Meta Llama

## Converse

The following code example shows how to send a text message to Meta Llama, using Bedrock's Converse API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Meta Llama, using Bedrock's Converse API.

```
// Use the Converse API to send a text message to Meta Llama.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.ConverseResponse;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

public class Converse {

 public static String converse() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Llama 3 8b Instruct.
 var modelId = "meta.llama3-8b-instruct-v1:0";
```

```

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 try {
 // Send the message with a basic inference configuration.
 ConverseResponse response = client.converse(request -> request
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)));

 // Retrieve the generated text from Bedrock's response object.
 var responseText = response.output().message().content().get(0).text();
 System.out.println(responseText);

 return responseText;

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
 e.getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) {
 converse();
 }
}

```

Send a text message to Meta Llama, using Bedrock's Converse API with the async Java client.

```
// Use the Converse API to send a text message to Meta Llama
```

```
// with the async Java client.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

public class ConverseAsync {

 public static String converseAsync() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Llama 3 8b Instruct.
 var modelId = "meta.llama3-8b-instruct-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Send the message with a basic inference configuration.
 var request = client.converse(params -> params
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F))
```

```

);

 // Prepare a future object to handle the asynchronous response.
 CompletableFuture<String> future = new CompletableFuture<>();

 // Handle the response or error using the future object.
 request.whenComplete((response, error) -> {
 if (error == null) {
 // Extract the generated text from Bedrock's response object.
 String responseText =
response.output().message().content().get(0).text();
 future.complete(responseText);
 } else {
 future.completeExceptionally(error);
 }
 });

 try {
 // Wait for the future object to complete and retrieve the generated
text.
 String responseText = future.get();
 System.out.println(responseText);

 return responseText;
 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 converseAsync();
}
}

```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## ConverseStream

The following code example shows how to send a text message to Meta Llama, using Bedrock's Converse API and process the response stream in real-time.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Meta Llama, using Bedrock's Converse API and process the response stream in real-time.

```
// Use the Converse API to send a text message to Meta Llama
// and print the response stream.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import
 software.amazon.awssdk.services.bedrockruntime.model.ConverseStreamResponseHandler;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.ExecutionException;

public class ConverseStream {

 public static void main(String[] args) {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Llama 3 8b Instruct.
 var modelId = "meta.llama3-8b-instruct-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
```

```

 var inputText = "Describe the purpose of a 'hello world' program in one
line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Create a handler to extract and print the response text in real-time.
 var responseStreamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 String responseText = chunk.delta().text();
 System.out.print(responseText);
 }).build()
).onError(err ->
 System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage())
).build();

 try {
 // Send the message with a basic inference configuration and attach the
handler.
 client.converseStream(request -> request
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)
), responseStreamHandler).get();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 }
 }
}

```

- For API details, see [ConverseStream](#) in *AWS SDK for Java 2.x API Reference*.



## InvokeModel

The following code example shows how to send a text message to Meta Llama, using the Invoke Model API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message.

```
// Use the native inference API to send a text message to Meta Llama 3.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

public class Llama3_InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_WEST_2)
 .build();

 // Set the model ID, e.g., Llama 3 70b Instruct.
 var modelId = "meta.llama3-70b-instruct-v1:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
```

```

// https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
meta.html
var nativeRequestTemplate = "{ \"prompt\": \"{{instruction}}\" }";

// Define the prompt for the model.
var prompt = "Describe the purpose of a 'hello world' program in one line.";

// Embed the prompt in Llama 3's instruction format.
var instruction = (
 "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\\n" +
 "{{prompt}} <|eot_id|>\\n" +
 "<|start_header_id|>assistant<|end_header_id|>\\n"
).replace("{{prompt}}", prompt);

// Embed the instruction in the the native request payload.
var nativeRequest = nativeRequestTemplate.replace("{{instruction}}",
instruction);

try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

 // Retrieve the generated text from the model's response.
 var text = new JSONPointer("/
generation").queryFrom(responseBody).toString();
 System.out.println(text);

 return text;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 invokeModel();
}

```

```
}
}
```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModelWithResponseStream

The following code example shows how to send a text message to Meta Llama, using the Invoke Model API, and print the response stream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message and process the response stream in real-time.

```
// Use the native inference API to send a text message to Meta Llama 3
// and print the response stream.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamRequest;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

import java.util.concurrent.ExecutionException;

import static
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

public class Llama3_InvokeModelWithResponseStream {
```

```

public static String invokeModelWithResponseStream() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_WEST_2)
 .build();

 // Set the model ID, e.g., Llama 3 70b Instruct.
 var modelId = "meta.llama3-70b-instruct-v1:0";

 // The InvokeModelWithResponseStream API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 meta.html
 var nativeRequestTemplate = "{ \"prompt\": \"{{instruction}}\" }";

 // Define the prompt for the model.
 var prompt = "Describe the purpose of a 'hello world' program in one line.";

 // Embed the prompt in Llama 3's instruction format.
 var instruction = (
 "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\\n" +
 "{{prompt}} <|eot_id|>\\n" +
 "<|start_header_id|>assistant<|end_header_id|>\\n"
).replace("{{prompt}}", prompt);

 // Embed the instruction in the the native request payload.
 var nativeRequest = nativeRequestTemplate.replace("{{instruction}}",
 instruction);

 // Create a request with the model ID and the model's native request
 payload.
 var request = InvokeModelWithResponseStreamRequest.builder()
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
 .build();

 // Prepare a buffer to accumulate the generated response text.
 var completeResponseTextBuffer = new StringBuilder();

```

```

 // Prepare a handler to extract, accumulate, and print the response text in
 // real-time.
 var responseStreamHandler =
 InvokeModelWithResponseStreamResponseHandler.builder()
 .subscriber(Visitor.builder().onChunk(chunk -> {
 // Extract and print the text from the model's native response.
 var response = new JSONObject(chunk.bytes().asUtf8String());
 var text = new JSONPointer("/generation").queryFrom(response);
 System.out.print(text);

 // Append the text to the response text buffer.
 completeResponseTextBuffer.append(text);
 }).build()).build();

 try {
 // Send the request and wait for the handler to process the response.
 client.invokeModelWithResponseStream(request,
 responseStreamHandler).get();

 // Return the complete response text.
 return completeResponseTextBuffer.toString();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
 e.getCause().getMessage());
 throw new RuntimeException(e);
 }
 }

 public static void main(String[] args) throws ExecutionException,
 InterruptedException {
 invokeModelWithResponseStream();
 }
}

```

- For API details, see [InvokeModelWithResponseStream](#) in *AWS SDK for Java 2.x API Reference*.

# Mistral AI

## Converse

The following code example shows how to send a text message to Mistral, using Bedrock's Converse API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Mistral, using Bedrock's Converse API.

```
// Use the Converse API to send a text message to Mistral.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.ConverseResponse;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

public class Converse {

 public static String converse() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Mistral Large.
 var modelId = "mistral.mistral-large-2402-v1:0";
```

```

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 try {
 // Send the message with a basic inference configuration.
 ConverseResponse response = client.converse(request -> request
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)));

 // Retrieve the generated text from Bedrock's response object.
 var responseText = response.output().message().content().get(0).text();
 System.out.println(responseText);

 return responseText;

 } catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
 e.getMessage());
 throw new RuntimeException(e);
 }

 }

 public static void main(String[] args) {
 converse();
 }
}

```

Send a text message to Mistral, using Bedrock's Converse API with the async Java client.

```
// Use the Converse API to send a text message to Mistral
// with the async Java client.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

public class ConverseAsync {

 public static String converseAsync() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Mistral Large.
 var modelId = "mistral.mistral-large-2402-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Send the message with a basic inference configuration.
 var request = client.converse(params -> params
 .modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
```



```

 .temperature(0.5F)
 .topP(0.9F))
);

 // Prepare a future object to handle the asynchronous response.
 CompletableFuture<String> future = new CompletableFuture<>();

 // Handle the response or error using the future object.
 request.whenComplete((response, error) -> {
 if (error == null) {
 // Extract the generated text from Bedrock's response object.
 String responseText =
response.output().message().content().get(0).text();
 future.complete(responseText);
 } else {
 future.completeExceptionally(error);
 }
 });

 try {
 // Wait for the future object to complete and retrieve the generated
text.
 String responseText = future.get();
 System.out.println(responseText);

 return responseText;
 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId, e.getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) {
 converseAsync();
}
}

```

- For API details, see [Converse](#) in *AWS SDK for Java 2.x API Reference*.

## ConverseStream

The following code example shows how to send a text message to Mistral, using Bedrock's Converse API and process the response stream in real-time.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send a text message to Mistral, using Bedrock's Converse API and process the response stream in real-time.

```
// Use the Converse API to send a text message to Mistral
// and print the response stream.

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import software.amazon.awssdk.services.bedrockruntime.model.ContentBlock;
import software.amazon.awssdk.services.bedrockruntime.model.ConversationRole;
import
 software.amazon.awssdk.services.bedrockruntime.model.ConverseStreamResponseHandler;
import software.amazon.awssdk.services.bedrockruntime.model.Message;

import java.util.concurrent.ExecutionException;

public class ConverseStream {

 public static void main(String[] args) {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();
```

```

 // Set the model ID, e.g., Mistral Large.
 var modelId = "mistral.mistral-large-2402-v1:0";

 // Create the input text and embed it in a message object with the user
 role.
 var inputText = "Describe the purpose of a 'hello world' program in one
 line.";
 var message = Message.builder()
 .content(ContentBlock.fromText(inputText))
 .role(ConversationRole.USER)
 .build();

 // Create a handler to extract and print the response text in real-time.
 var responseStreamHandler = ConverseStreamResponseHandler.builder()
 .subscriber(ConverseStreamResponseHandler.Visitor.builder()
 .onContentBlockDelta(chunk -> {
 String responseText = chunk.delta().text();
 System.out.print(responseText);
 }).build())
 .onError(err ->
 System.err.printf("Can't invoke '%s': %s", modelId,
err.getMessage()))
 .build();

 try {
 // Send the message with a basic inference configuration and attach the
 handler.
 client.converseStream(request -> request.modelId(modelId)
 .messages(message)
 .inferenceConfig(config -> config
 .maxTokens(512)
 .temperature(0.5F)
 .topP(0.9F)
), responseStreamHandler).get();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 }
 }
}

```

- For API details, see [ConverseStream](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModel

The following code example shows how to send a text message to Mistral models, using the Invoke Model API.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message.

```
// Use the native inference API to send a text message to Mistral.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

public class InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Mistral Large.
 var modelId = "mistral.mistral-large-2402-v1:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
```

```

// https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
mistral-text-completion.html
var nativeRequestTemplate = "{ \"prompt\": \"{{instruction}}\" }";

// Define the prompt for the model.
var prompt = "Describe the purpose of a 'hello world' program in one line.";

// Embed the prompt in Mistral's instruction format.
var instruction = "<s>[INST] {{prompt}} [/INST]\\n".replace("{{prompt}}",
prompt);

// Embed the instruction in the the native request payload.
var nativeRequest = nativeRequestTemplate.replace("{{instruction}}",
instruction);

try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

 // Retrieve the generated text from the model's response.
 var text = new JSONPointer("/outputs/0/
text").queryFrom(responseBody).toString();
 System.out.println(text);

 return text;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {
 invokeModel();
}
}

```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## InvokeModelWithResponseStream

The following code example shows how to send a text message to Mistral AI models, using the Invoke Model API, and print the response stream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the Invoke Model API to send a text message and process the response stream in real-time.

```
// Use the native inference API to send a text message to Mistral
// and print the response stream.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeAsyncClient;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamRequest;
import
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

import java.util.concurrent.ExecutionException;

import static
 software.amazon.awssdk.services.bedrockruntime.model.InvokeModelWithResponseStreamResponse;

public class InvokeModelWithResponseStream {

 public static String invokeModelWithResponseStream() {
```

```

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeAsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Mistral Large.
 var modelId = "mistral.mistral-large-2402-v1:0";

 // The InvokeModelWithResponseStream API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
 // https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
 mistral-text-completion.html
 var nativeRequestTemplate = "{ \"prompt\": \"{{instruction}}\" }";

 // Define the prompt for the model.
 var prompt = "Describe the purpose of a 'hello world' program in one line.";

 // Embed the prompt in Mistral's instruction format.
 var instruction = "<s>[INST] {{prompt}} [/INST]\\n".replace("{{prompt}}",
 prompt);

 // Embed the instruction in the the native request payload.
 var nativeRequest = nativeRequestTemplate.replace("{{instruction}}",
 instruction);

 // Create a request with the model ID and the model's native request
 payload.
 var request = InvokeModelWithResponseStreamRequest.builder()
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
 .build();

 // Prepare a buffer to accumulate the generated response text.
 var completeResponseTextBuffer = new StringBuilder();

 // Prepare a handler to extract, accumulate, and print the response text in
 real-time.
 var responseStreamHandler =
 InvokeModelWithResponseStreamResponseHandler.builder()
 .subscriber(Visitor.builder().onChunk(chunk -> {

```

```

 // Extract and print the text from the model's native response.
 var response = new JSONObject(chunk.bytes().asUtf8String());
 var text = new JSONPointer("/outputs/0/
text").queryFrom(response);
 System.out.print(text);

 // Append the text to the response text buffer.
 completeResponseTextBuffer.append(text);
 }).build()).build();

 try {
 // Send the request and wait for the handler to process the response.
 client.invokeModelWithResponseStream(request,
responseStreamHandler).get();

 // Return the complete response text.
 return completeResponseTextBuffer.toString();

 } catch (ExecutionException | InterruptedException e) {
 System.err.printf("Can't invoke '%s': %s", modelId,
e.getCause().getMessage());
 throw new RuntimeException(e);
 }
}

public static void main(String[] args) throws ExecutionException,
InterruptedException {
 invokeModelWithResponseStream();
}
}

```

- For API details, see [InvokeModelWithResponseStream](#) in *AWS SDK for Java 2.x API Reference*.

## Stable Diffusion

### InvokeModel

The following code example shows how to invoke Stability.ai Stable Diffusion XL on Amazon Bedrock to generate an image.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create an image with Stable Diffusion.

```
// Create an image with Stability AI Stable Image Core.

import org.json.JSONObject;
import org.json.JSONPointer;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.exception.SdkClientException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.bedrockruntime.BedrockRuntimeClient;

import java.math.BigInteger;
import java.security.SecureRandom;

import static com.example.bedrockruntime.libs.ImageTools.displayImage;

public class InvokeModel {

 public static String invokeModel() {

 // Create a Bedrock Runtime client in the AWS Region you want to use.
 // Replace the DefaultCredentialsProvider with your preferred credentials
 provider.
 var client = BedrockRuntimeClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
 .region(Region.US_EAST_1)
 .build();

 // Set the model ID, e.g., Stable Image Core.
 var modelId = "stability.stable-image-core-v1:0";

 // The InvokeModel API uses the model's native payload.
 // Learn more about the available inference parameters and response fields
 at:
```

```

// https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-
diffusion-stable-image-core-text-image-request-response.html
var nativeRequestTemplate = """
 {
 "prompt": "{{prompt}}",
 "seed": {{seed}}
 }""";

// Define the prompt for the image generation.
var prompt = "A stylized picture of a cute old steampunk robot";

// Get a random seed for the image generation (max. 4,294,967,294).
var seed = new BigInteger(31, new SecureRandom());

// Embed the prompt and seed in the model's native request payload.
String nativeRequest = nativeRequestTemplate
 .replace("{{prompt}}", prompt)
 .replace("{{seed}}", seed.toString());

try {
 // Encode and send the request to the Bedrock Runtime.
 var response = client.invokeModel(request -> request
 .body(SdkBytes.fromUtf8String(nativeRequest))
 .modelId(modelId)
);

 // Decode the response body.
 var responseBody = new JSONObject(response.body().asUtf8String());

 // Retrieve the generated image data from the model's response.
 var base64ImageData = new JSONPointer("/images/0")
 .queryFrom(responseBody)
 .toString();

 return base64ImageData;

} catch (SdkClientException e) {
 System.err.printf("ERROR: Can't invoke '%s'. Reason: %s", modelId,
e.getMessage());
 throw new RuntimeException(e);
}

}

public static void main(String[] args) {

```

```
 System.out.println("Generating image. This may take a few seconds...");

 String base64ImageData = invokeModel();

 displayImage(base64ImageData);
 }

}
```

- For API details, see [InvokeModel](#) in *AWS SDK for Java 2.x API Reference*.

## CloudFront examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with CloudFront.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### CreateDistribution

The following code example shows how to use `CreateDistribution`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following example uses an Amazon Simple Storage Service (Amazon S3) bucket as a content origin.

After creating the distribution, the code creates a [CloudFrontWaiter](#) to wait until the distribution is deployed before returning the distribution.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.internal.waiters.ResponseOrException;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.CreateDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.Distribution;
import software.amazon.awssdk.services.cloudfront.model.GetDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.ItemSelection;
import software.amazon.awssdk.services.cloudfront.model.Method;
import software.amazon.awssdk.services.cloudfront.model.ViewerProtocolPolicy;
import software.amazon.awssdk.services.cloudfront.waiters.CloudFrontWaiter;
import software.amazon.awssdk.services.s3.S3Client;

import java.time.Instant;

public class CreateDistribution {

 private static final Logger logger =
 LoggerFactory.getLogger(CreateDistribution.class);

 public static Distribution createDistribution(CloudFrontClient
 cloudFrontClient, S3Client s3Client,
 final String bucketName, final String keyGroupId, final
 String originAccessControlId) {

 final String region = s3Client.headBucket(b ->
 b.bucket(bucketName)).sdkHttpResponse().headers()
```

```

 .get("x-amz-bucket-region").get(0);
 final String originDomain = bucketName + ".s3." + region +
".amazonaws.com";
 String originId = originDomain; // Use the originDomain value for
the originId.

 // The service API requires some deprecated methods, such as
 // DefaultCacheBehavior.Builder#minTTL and #forwardedValue.
 CreateDistributionResponse createDistResponse =
cloudFrontClient.createDistribution(builder -> builder
 .distributionConfig(b1 -> b1
 .origins(b2 -> b2
 .quantity(1)
 .items(b3 -> b3

.domainName(originDomain)

.id(originId)

.s3OriginConfig(builder4 -> builder4

 .originAccessIdentity(

 ""))

 .originAccessControlId(

 originAccessControlId)))

 .defaultCacheBehavior(b2 -> b2

.viewerProtocolPolicy(ViewerProtocolPolicy.ALLOW_ALL)

.targetOriginId(originId)

 .minTTL(200L)
 .forwardedValues(b5

-> b5

.cookies(cp -> cp

 .forward(ItemSelection.NONE))

.queryString(true))

 .trustedKeyGroups(b3

-> b3

```

```

 .quantity(1)

 .items(keyGroupId)

 .enabled(true))

 .allowedMethods(b4 -
> b4

 .quantity(2)

 .items(Method.HEAD, Method.GET)

 .cachedMethods(b5 -> b5

 .quantity(2)

 .items(Method.HEAD,

 Method.GET))))

 .cacheBehaviors(b -> b
 .quantity(1)
 .items(b2 -> b2

 .pathPattern("/index.html")

 .viewerProtocolPolicy(

 ViewerProtocolPolicy.ALLOW_ALL)

 .targetOriginId(originId)

 .trustedKeyGroups(b3 -> b3

 .quantity(1)

 .items(keyGroupId)

 .enabled(true))

 .minTTL(200L)

 .forwardedValues(b4 -> b4

```

```

 .cookies(cp -> cp

 .forward(ItemSelection.NONE))

 .queryString(true))

.allowedMethods(b5 -> b5.quantity(2)

 .items(Method.HEAD,

 Method.GET)

 .cachedMethods(b6 -> b6

 .quantity(2)

 .items(Method.HEAD,

 Method.GET))))))
 .enabled(true)
 .comment("Distribution built with
java")

.callerReference(Instant.now().toString())));

 final Distribution distribution = createDistResponse.distribution();
 logger.info("Distribution created. DomainName: [{}] Id: [{}]",
distribution.domainName(),
 distribution.id());
 logger.info("Waiting for distribution to be deployed ...");
 try (CloudFrontWaiter cfWaiter =
CloudFrontWaiter.builder().client(cloudFrontClient).build()) {
 ResponseOrException<GetDistributionResponse>
responseOrException = cfWaiter

 .waitUntilDistributionDeployed(builder ->
builder.id(distribution.id()))

 .matched();
 responseOrException.response()
 .orElseThrow(() -> new
RuntimeException("Distribution not created"));
 logger.info("Distribution deployed. DomainName: [{}] Id:
[{}]", distribution.domainName(),
 distribution.id());

```

```

 }
 return distribution;
 }
}

```

- For API details, see [CreateDistribution](#) in *AWS SDK for Java 2.x API Reference*.

## CreateFunction

The following code example shows how to use CreateFunction.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.CloudFrontException;
import software.amazon.awssdk.services.cloudfront.model.CreateFunctionRequest;
import software.amazon.awssdk.services.cloudfront.model.CreateFunctionResponse;
import software.amazon.awssdk.services.cloudfront.model.FunctionConfig;
import software.amazon.awssdk.services.cloudfront.model.FunctionRuntime;
import java.io.InputStream;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateFunction {

 public static void main(String[] args) {
 final String usage = ""

```



**Usage:**

<functionName> <filePath>

**Where:**

functionName - The name of the function to create.\s

filePath - The path to a file that contains the application logic for the function.\s  
 """;

```

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String functionName = args[0];
 String filePath = args[1];
 CloudFrontClient cloudFrontClient = CloudFrontClient.builder()
 .region(Region.AWS_GLOBAL)
 .build();

 String funArn = createNewFunction(cloudFrontClient, functionName, filePath);
 System.out.println("The function ARN is " + funArn);
 cloudFrontClient.close();
}

public static String createNewFunction(CloudFrontClient cloudFrontClient, String
functionName, String filePath) {
 try {
 InputStream fileIs =
CreateFunction.class.getClassLoader().getResourceAsStream(filePath);
 SdkBytes functionCode = SdkBytes.fromInputStream(fileIs);

 FunctionConfig config = FunctionConfig.builder()
 .comment("Created by using the CloudFront Java API")
 .runtime(FunctionRuntime.CLOUDFRONT_JS_1_0)
 .build();

 CreateFunctionRequest functionRequest = CreateFunctionRequest.builder()
 .name(functionName)
 .functionCode(functionCode)
 .functionConfig(config)
 .build();

```

```
 CreateFunctionResponse response =
cloudFrontClient.createFunction(functionRequest);
 return response.functionSummary().functionMetadata().functionARN();

 } catch (CloudFrontException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}
}
```

- For API details, see [CreateFunction](#) in *AWS SDK for Java 2.x API Reference*.

## CreateKeyGroup

The following code example shows how to use CreateKeyGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

A key group requires at least one public key that is used to verify signed URLs or cookies.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;

import java.util.UUID;

public class CreateKeyGroup {
 private static final Logger logger =
 LoggerFactory.getLogger(CreateKeyGroup.class);

 public static String createKeyGroup(CloudFrontClient cloudFrontClient, String
 publicKeyId) {
```

```

 String keyGroupId = cloudFrontClient.createKeyGroup(b -> b.keyGroupConfig(c
-> c
 .items(publicKeyId)
 .name("JavaKeyGroup" + UUID.randomUUID()))
 .keyGroup().id());
 logger.info("KeyGroup created with ID: [{}]", keyGroupId);
 return keyGroupId;
 }
}

```

- For API details, see [CreateKeyGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreatePublicKey

The following code example shows how to use `CreatePublicKey`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following code example reads in a public key and uploads it to Amazon CloudFront.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.CreatePublicKeyResponse;
import software.amazon.awssdk.utils.IoUtils;

import java.io.IOException;
import java.io.InputStream;
import java.util.UUID;

public class CreatePublicKey {
 private static final Logger logger =
 LoggerFactory.getLogger(CreatePublicKey.class);
}

```

```

 public static String createPublicKey(CloudFrontClient cloudFrontClient, String
 publicKeyFileName) {
 try (InputStream is =
 CreatePublicKey.class.getClassLoader().getResourceAsStream(publicKeyFileName)) {
 String publicKeyString = IoUtils.toUtf8String(is);
 CreatePublicKeyResponse createPublicKeyResponse = cloudFrontClient
 .createPublicKey(b -> b.publicKeyConfig(c -> c
 .name("JavaCreatedPublicKey" + UUID.randomUUID())
 .encodedKey(publicKeyString)
 .callerReference(UUID.randomUUID().toString())));
 String createdPublicKeyId = createPublicKeyResponse.publicKey().id();
 logger.info("Public key created with id: [{}]", createdPublicKeyId);
 return createdPublicKeyId;

 } catch (IOException e) {
 throw new RuntimeException(e);
 }
 }
}

```

- For API details, see [CreatePublicKey](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDistribution

The following code example shows how to use DeleteDistribution.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following code example updates a distribution to *disabled*, uses a waiter that waits for the change to be deployed, then deletes the distribution.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.internal.waiters.ResponseOrException;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;

```

```

import software.amazon.awssdk.services.cloudfront.model.DeleteDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.DistributionConfig;
import software.amazon.awssdk.services.cloudfront.model.GetDistributionResponse;
import software.amazon.awssdk.services.cloudfront.waiters.CloudFrontWaiter;

public class DeleteDistribution {
 private static final Logger logger =
 LoggerFactory.getLogger(DeleteDistribution.class);

 public static void deleteDistribution(final CloudFrontClient
cloudFrontClient, final String distributionId) {
 // First, disable the distribution by updating it.
 GetDistributionResponse response =
cloudFrontClient.getDistribution(b -> b
 .id(distributionId));
 String etag = response.eTag();
 DistributionConfig distConfig =
response.distribution().distributionConfig();

 cloudFrontClient.updateDistribution(builder -> builder
 .id(distributionId)
 .distributionConfig(builder1 -> builder1

.cacheBehaviors(distConfig.cacheBehaviors())

.defaultCacheBehavior(distConfig.defaultCacheBehavior())
 .enabled(false)
 .origins(distConfig.origins())
 .comment(distConfig.comment())

.callerReference(distConfig.callerReference())

.defaultCacheBehavior(distConfig.defaultCacheBehavior())
 .priceClass(distConfig.priceClass())
 .aliases(distConfig.aliases())
 .logging(distConfig.logging())

.defaultRootObject(distConfig.defaultRootObject())

.customErrorResponses(distConfig.customErrorResponses())

.httpVersion(distConfig.httpVersion())

.isIPV6Enabled(distConfig.isIPV6Enabled())

```

```

 .restrictions(distConfig.restrictions())

 .viewerCertificate(distConfig.viewerCertificate())
 .webACLId(distConfig.webACLId())

 .originGroups(distConfig.originGroups())
 .ifMatch(etag));

 logger.info("Distribution [{}] is DISABLED, waiting for deployment
before deleting ...",
 distributionId);
 GetDistributionResponse distributionResponse;
 try (CloudFrontWaiter cfWaiter =
CloudFrontWaiter.builder().client(cloudFrontClient).build()) {
 ResponseOrException<GetDistributionResponse>
responseOrException = cfWaiter
 .waitUntilDistributionDeployed(builder ->
builder.id(distributionId)).matched();
 distributionResponse = responseOrException.response()
 .orElseThrow(() -> new
RuntimeException("Could not disable distribution"));
 }

 DeleteDistributionResponse deleteDistributionResponse =
cloudFrontClient
 .deleteDistribution(builder -> builder
 .id(distributionId)

 .ifMatch(distributionResponse.eTag()));
 if (deleteDistributionResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Distribution [{}] DELETED", distributionId);
 }
 }
}

```

- For API details, see [DeleteDistribution](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateDistribution

The following code example shows how to use UpdateDistribution.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.GetDistributionRequest;
import software.amazon.awssdk.services.cloudfront.model.GetDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.Distribution;
import software.amazon.awssdk.services.cloudfront.model.DistributionConfig;
import software.amazon.awssdk.services.cloudfront.model.UpdateDistributionRequest;
import software.amazon.awssdk.services.cloudfront.model.CloudFrontException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ModifyDistribution {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <id>\s

 Where:
 id - the id value of the distribution.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String id = args[0];
```

```

 CloudFrontClient cloudFrontClient = CloudFrontClient.builder()
 .region(Region.AWS_GLOBAL)
 .build();

 modDistribution(cloudFrontClient, id);
 cloudFrontClient.close();
 }

 public static void modDistribution(CloudFrontClient cloudFrontClient, String
idVal) {
 try {
 // Get the Distribution to modify.
 GetDistributionRequest disRequest = GetDistributionRequest.builder()
 .id(idVal)
 .build();

 GetDistributionResponse response =
cloudFrontClient.getDistribution(disRequest);
 Distribution disObject = response.distribution();
 DistributionConfig config = disObject.distributionConfig();

 // Create a new DistributionConfig object and add new values to comment
and
 // aliases
 DistributionConfig config1 = DistributionConfig.builder()
 .aliases(config.aliases()) // You can pass in new values here
 .comment("New Comment")
 .cacheBehaviors(config.cacheBehaviors())
 .priceClass(config.priceClass())
 .defaultCacheBehavior(config.defaultCacheBehavior())
 .enabled(config.enabled())
 .callerReference(config.callerReference())
 .logging(config.logging())
 .originGroups(config.originGroups())
 .origins(config.origins())
 .restrictions(config.restrictions())
 .defaultRootObject(config.defaultRootObject())
 .webACLId(config.webACLId())
 .httpVersion(config.httpVersion())
 .viewerCertificate(config.viewerCertificate())
 .customErrorResponses(config.customErrorResponses())
 .build();

```



```

 UpdateDistributionRequest updateDistributionRequest =
UpdateDistributionRequest.builder()
 .distributionConfig(config1)
 .id(disObject.id())
 .ifMatch(response.eTag())
 .build();

 cloudFrontClient.updateDistribution(updateDistributionRequest);

 } catch (CloudFrontException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [UpdateDistribution](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a multi-tenant distribution and distribution tenant

The following code example shows how to create a multi-tenant distribution and distribution tenant with various configurations.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following example demonstrates how to create a multi-tenant distribution with parameters and wildcard certificate.

```

import software.amazon.awssdk.core.internal.waiters.ResponseOrException;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.ConnectionMode;

```

```

import software.amazon.awssdk.services.cloudfront.model.CreateDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.Distribution;
import software.amazon.awssdk.services.cloudfront.model.GetDistributionResponse;
import software.amazon.awssdk.services.cloudfront.model.HttpVersion;
import software.amazon.awssdk.services.cloudfront.model.Method;
import software.amazon.awssdk.services.cloudfront.model.SSLSupportMethod;
import software.amazon.awssdk.services.cloudfront.model.ViewerProtocolPolicy;
import software.amazon.awssdk.services.cloudfront.waiters.CloudFrontWaiter;
import software.amazon.awssdk.services.s3.S3Client;

import java.time.Instant;

public class CreateMultiTenantDistribution {
 public static Distribution
 CreateMultiTenantDistributionWithCert(CloudFrontClient cloudFrontClient,
 S3Client
 s3Client,
 final String
 bucketName,
 final String
 certificateArn) {
 // fetch the origin info if necessary
 final String region = s3Client.headBucket(b ->
 b.bucket(bucketName)).sdkHttpResponse().headers()
 .get("x-amz-bucket-region").get(0);
 final String originDomain = bucketName + ".s3." + region + ".amazonaws.com";
 String originId = originDomain; // Use the originDomain value for the
 originId.

 CreateDistributionResponse createDistResponse =
 cloudFrontClient.createDistribution(builder -> builder
 .distributionConfig(b1 -> b1
 .httpVersion(HttpVersion.HTTP2)
 .enabled(true)
 .comment("Template Distribution with cert built with java")
 .connectionMode(ConnectionMode.TENANT_ONLY)
 .callerReference(Instant.now().toString())
 .viewerCertificate(certBuilder -> certBuilder
 .acmCertificateArn(certificateArn)
 .sslSupportMethod(SSLSupportMethod.SNI_ONLY))
 .origins(b2 -> b2
 .quantity(1)
 .items(b3 -> b3
 .domainName(originDomain)

```

```

 .id(originId)
 .originPath("/{tenantName}")
 .s3OriginConfig(builder4 -> builder4
 .originAccessIdentity(
 "")))
 .tenantConfig(b5 -> b5
 .parameterDefinitions(b6 -> b6
 .name("tenantName")
 .definition(b7 -> b7
 .stringSchema(b8 -> b8
 .comment("tenantName value")
 .defaultValue("root")
 .required(false))))
 .defaultCacheBehavior(b2 -> b2

.viewerProtocolPolicy(ViewerProtocolPolicy.ALLOW_ALL)
 .targetOriginId(originId)
 .cachePolicyId("658327ea-f89d-4fab-
a63d-7e88639e58f6") // CachingOptimized Policy
 .allowedMethods(b4 -> b4
 .quantity(2)
 .items(Method.HEAD, Method.GET)))
));

 final Distribution distribution = createDistResponse.distribution();
 try (CloudFrontWaiter cfWaiter =
CloudFrontWaiter.builder().client(cloudFrontClient).build()) {
 ResponseOrException<GetDistributionResponse> responseOrException =
cfWaiter
 .waitUntilDistributionDeployed(builder ->
builder.id(distribution.id()))
 .matched();
 responseOrException.response()
 .orElseThrow(() -> new RuntimeException("Distribution not
created"));
 }
 return distribution;
}

 public static Distribution CreateMultiTenantDistributionNoCert(CloudFrontClient
cloudFrontClient,

 S3Client s3Client,
 final String
bucketName) {

```

```

 // fetch the origin info if necessary
 final String region = s3Client.headBucket(b ->
b.bucket(bucketName)).sdkHttpResponse().headers()
 .get("x-amz-bucket-region").get(0);
 final String originDomain = bucketName + ".s3." + region + ".amazonaws.com";
 String originId = originDomain; // Use the originDomain value for the
originId.

 CreateDistributionResponse createDistResponse =
cloudFrontClient.createDistribution(builder -> builder
 .distributionConfig(b1 -> b1
 .httpVersion(HttpVersion.HTTP2)
 .enabled(true)
 .comment("Template Distribution with cert built with java")
 .connectionMode(ConnectionMode.TENANT_ONLY)
 .callerReference(Instant.now().toString())
 .origins(b2 -> b2
 .quantity(1)
 .items(b3 -> b3
 .domainName(originDomain)
 .id(originId)
 .originPath("/{tenantName}")
 .s3OriginConfig(builder4 -> builder4
 .originAccessIdentity(
 ""))))
 .tenantConfig(b5 -> b5
 .parameterDefinitions(b6 -> b6
 .name("tenantName")
 .definition(b7 -> b7
 .stringSchema(b8 -> b8
 .comment("tenantName value")
 .defaultValue("root")
 .required(false))))
 .defaultCacheBehavior(b2 -> b2

 .viewerProtocolPolicy(ViewerProtocolPolicy.ALLOW_ALL)
 .targetOriginId(originId)
 .cachePolicyId("658327ea-f89d-4fab-
a63d-7e88639e58f6") // CachingOptimized Policy
 .allowedMethods(b4 -> b4
 .quantity(2)
 .items(Method.HEAD, Method.GET)))
));

```

```

 final Distribution distribution = createDistResponse.distribution();
 try (CloudFrontWaiter cfWaiter =
CloudFrontWaiter.builder().client(cloudFrontClient).build()) {
 ResponseOrException<GetDistributionResponse> responseOrException =
cfWaiter
 .waitUntilDistributionDeployed(builder ->
builder.id(distribution.id()))
 .matched();
 responseOrException.response()
 .orElseThrow(() -> new RuntimeException("Distribution not
created"));
 }
 return distribution;
 }
}

```

The following example demonstrates how to create a distribution tenant associated with that template, including utilizing the parameter we declared above. Note that we don't need to add certificate info here because our domain is already covered by the parent template.

```

import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import
 software.amazon.awssdk.services.cloudfront.model.CreateConnectionGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.CreateDistributionTenantResponse;
import software.amazon.awssdk.services.cloudfront.model.DistributionTenant;
import software.amazon.awssdk.services.cloudfront.model.GetConnectionGroupResponse;
import software.amazon.awssdk.services.cloudfront.model.ValidationTokenHost;
import software.amazon.awssdk.services.route53.Route53Client;
import software.amazon.awssdk.services.route53.model.RRType;

import java.time.Instant;

public class CreateDistributionTenant {

 public static DistributionTenant createDistributionTenantNoCert(CloudFrontClient
cloudFrontClient,
 Route53Client
route53Client,
 String
distributionId,

```

String domain,  
String

```

hostedZoneId) {
 CreateDistributionTenantResponse createResponse =
cloudFrontClient.createDistributionTenant(builder -> builder
 .distributionId(distributionId)
 .domains(b1 -> b1
 .domain(domain))
 .parameters(b2 -> b2
 .name("tenantName")
 .value("myTenant"))
 .enabled(false)
 .name("no-cert-tenant")
);

 final DistributionTenant distributionTenant =
createResponse.distributionTenant();

 // Then update the Route53 hosted zone to point your domain at the
distribution tenant
 // We fetch the RoutingEndpoint to point to via the default connection group
that was created for your tenant
 final GetConnectionGroupResponse fetchedConnectionGroup =
cloudFrontClient.getConnectionGroup(builder -> builder
 .identifier(distributionTenant.connectionGroupId()));

 route53Client.changeResourceRecordSets(builder -> builder
 .hostedZoneId(hostedZoneId)
 .changeBatch(b1 -> b1
 .comment("ChangeBatch comment")
 .changes(b2 -> b2
 .resourceRecordSet(b3 -> b3
 .name(domain)
 .type("CNAME")
 .ttl(300L)
 .resourceRecords(b4 -> b4

.value(fetchedConnectionGroup.connectionGroup().routingEndpoint()))
 .action("CREATE"))
));
 return distributionTenant;
}
}

```

If the viewer certificate was omitted from the parent template, you would need to add certificate info on the tenant(s) associated with it instead. The following example demonstrates how to do so via an ACM certificate arn that covers the necessary domain for the tenant.

```
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import
 software.amazon.awssdk.services.cloudfront.model.CreateConnectionGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.CreateDistributionTenantResponse;
import software.amazon.awssdk.services.cloudfront.model.DistributionTenant;
import software.amazon.awssdk.services.cloudfront.model.GetConnectionGroupResponse;
import software.amazon.awssdk.services.cloudfront.model.ValidationTokenHost;
import software.amazon.awssdk.services.route53.Route53Client;
import software.amazon.awssdk.services.route53.model.RRType;

import java.time.Instant;

public class CreateDistributionTenant {

 public static DistributionTenant
 createDistributionTenantWithCert(CloudFrontClient cloudFrontClient,
 Route53Client
 route53Client,
 String
 distributionId,
 String domain,
 String
 hostedZoneId,
 String
 certificateArn) {
 CreateDistributionTenantResponse createResponse =
 cloudFrontClient.createDistributionTenant(builder -> builder
 .distributionId(distributionId)
 .domains(b1 -> b1
 .domain(domain))
 .enabled(false)
 .name("tenant-with-cert")
 .parameters(b2 -> b2
 .name("tenantName")
 .value("myTenant"))
```

```

 .customizations(b3 -> b3
 .certificate(b4 -> b4
 .arn(certificateArn))) // NOTE: Cert must be in Us-
// East-1 and cover the domain provided in this request

);

 final DistributionTenant distributionTenant =
createResponse.distributionTenant();

 // Then update the Route53 hosted zone to point your domain at the
distribution tenant
 // We fetch the RoutingEndpoint to point to via the default connection group
that was created for your tenant
 final GetConnectionGroupResponse fetchedConnectionGroup =
cloudFrontClient.getConnectionGroup(builder -> builder
 .identifier(distributionTenant.connectionGroupId()));

 route53Client.changeResourceRecordSets(builder -> builder
 .hostedZoneId(hostedZoneId)
 .changeBatch(b1 -> b1
 .comment("ChangeBatch comment")
 .changes(b2 -> b2
 .resourceRecordSet(b3 -> b3
 .name(domain)
 .type("CNAME")
 .ttl(300L)
 .resourceRecords(b4 -> b4

.value(fetchedConnectionGroup.connectionGroup().routingEndpoint()))
 .action("CREATE"))
));
 return distributionTenant;
}
}

```

The following example demonstrates how to do so with a CloudFront-hosted managed certificate request. This is ideal if you don't already have traffic towards your domain. In this case, we create a ConnectionGroup to generate a RoutingEndpoint. Then we use that RoutingEndpoint to create DNS records which verify domain ownership and point to



CloudFront. CloudFront will then automatically serve a token to validate domain ownership and create a managed certificate.

```
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import
 software.amazon.awssdk.services.cloudfront.model.CreateConnectionGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.CreateDistributionTenantResponse;
import software.amazon.awssdk.services.cloudfront.model.DistributionTenant;
import software.amazon.awssdk.services.cloudfront.model.GetConnectionGroupResponse;
import software.amazon.awssdk.services.cloudfront.model.ValidationTokenHost;
import software.amazon.awssdk.services.route53.Route53Client;
import software.amazon.awssdk.services.route53.model.RRType;

import java.time.Instant;

public class CreateDistributionTenant {

 public static DistributionTenant
 createDistributionTenantCfHosted(CloudFrontClient cloudFrontClient,
 Route53Client
 route53Client,
 String
 distributionId,
 String domain,
 String
 hostedZoneId) throws InterruptedException {
 CreateConnectionGroupResponse createConnectionGroupResponse =
 cloudFrontClient.createConnectionGroup(builder -> builder
 .ipv6Enabled(true)
 .name("cf-hosted-connection-group")
 .enabled(true));

 route53Client.changeResourceRecordSets(builder -> builder
 .hostedZoneId(hostedZoneId)
 .changeBatch(b1 -> b1
 .comment("cf-hosted domain validation record")
 .changes(b2 -> b2
 .resourceRecordSet(b3 -> b3
 .name(domain)
 .type(RRType.CNAME)
 .ttl(300L))
)
)
);
 }
}
```

```

 .resourceRecords(b4 -> b4

 .value(createConnectionGroupResponse.connectionGroup().routingEndpoint()))
 .action("CREATE"))
 });

 // Give the R53 record time to propagate, if it isn't being returned by
 // servers yet, the following call will fail
 Thread.sleep(60000);

 CreateDistributionTenantResponse createResponse =
cloudFrontClient.createDistributionTenant(builder -> builder
 .distributionId(distributionId)
 .domains(b1 -> b1
 .domain(domain))

 .connectionGroupId(createConnectionGroupResponse.connectionGroup().id())
 .enabled(false)
 .name("cf-hosted-tenant")
 .parameters(b2 -> b2
 .name("tenantName")
 .value("myTenant"))
 .managedCertificateRequest(b3 -> b3
 .validationTokenHost(ValidationTokenHost.CLOUDFRONT)
)
);

 return createResponse.distributionTenant();
}
}

```

The following example demonstrates how to do so with a self-hosted managed certificate request. This is ideal if you have traffic towards your domain and can't tolerate downtime during a migration. At the end of this example, the Tenant will be created in a state awaiting domain validation and DNS setup. Follow steps [here](https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/managed-cloudfront-certificates.html#complete-domain-ownership) to complete setup when you are ready to migrate traffic.

```
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
```

```

import
 software.amazon.awssdk.services.cloudfront.model.CreateConnectionGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.CreateDistributionTenantResponse;
import software.amazon.awssdk.services.cloudfront.model.DistributionTenant;
import software.amazon.awssdk.services.cloudfront.model.GetConnectionGroupResponse;
import software.amazon.awssdk.services.cloudfront.model.ValidationTokenHost;
import software.amazon.awssdk.services.route53.Route53Client;
import software.amazon.awssdk.services.route53.model.RRType;

import java.time.Instant;

public class CreateDistributionTenant {

 public static DistributionTenant
 createDistributionTenantSelfHosted(CloudFrontClient cloudFrontClient,
 String
 distributionId,
 String
 domain) {
 CreateDistributionTenantResponse createResponse =
 cloudFrontClient.createDistributionTenant(builder -> builder
 .distributionId(distributionId)
 .domains(b1 -> b1
 .domain(domain))
 .parameters(b2 -> b2
 .name("tenantName")
 .value("myTenant"))
 .enabled(false)
 .name("self-hosted-tenant")
 .managedCertificateRequest(b3 -> b3
 .validationTokenHost(ValidationTokenHost.SELF_HOSTED)
 .primaryDomainName(domain)
)
);

 return createResponse.distributionTenant();
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateDistribution](#)
- [CreateDistributionTenant](#)

## Delete signing resources

The following code example shows how to delete resources that are used to gain access to restricted content in an Amazon Simple Storage Service (Amazon S3) bucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.cloudfront.CloudFrontClient;
import software.amazon.awssdk.services.cloudfront.model.DeleteKeyGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.DeleteOriginAccessControlResponse;
import software.amazon.awssdk.services.cloudfront.model.DeletePublicKeyResponse;
import software.amazon.awssdk.services.cloudfront.model.GetKeyGroupResponse;
import
 software.amazon.awssdk.services.cloudfront.model.GetOriginAccessControlResponse;
import software.amazon.awssdk.services.cloudfront.model.GetPublicKeyResponse;

public class DeleteSigningResources {
 private static final Logger logger =
 LoggerFactory.getLogger(DeleteSigningResources.class);

 public static void deleteOriginAccessControl(final CloudFrontClient
cloudFrontClient,
 final String originAccessControlId) {
 GetOriginAccessControlResponse getResponse = cloudFrontClient
 .getOriginAccessControl(b -> b.id(originAccessControlId));
 DeleteOriginAccessControlResponse deleteResponse =
cloudFrontClient.deleteOriginAccessControl(builder -> builder
 .id(originAccessControlId)
 .ifMatch(getResponse.eTag()));
 }
}
```

```

 if (deleteResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Successfully deleted Origin Access Control [{}]",
originAccessControlId);
 }
 }

 public static void deleteKeyGroup(final CloudFrontClient cloudFrontClient, final
String keyGroupId) {

 GetKeyGroupResponse getResponse = cloudFrontClient.getKeyGroup(b ->
b.id(keyGroupId));
 DeleteKeyGroupResponse deleteResponse =
cloudFrontClient.deleteKeyGroup(builder -> builder
 .id(keyGroupId)
 .ifMatch(getResponse.eTag()));
 if (deleteResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Successfully deleted Key Group [{}]", keyGroupId);
 }
 }

 public static void deletePublicKey(final CloudFrontClient cloudFrontClient,
final String publicKeyId) {
 GetPublicKeyResponse getResponse = cloudFrontClient.getPublicKey(b ->
b.id(publicKeyId));

 DeletePublicKeyResponse deleteResponse =
cloudFrontClient.deletePublicKey(builder -> builder
 .id(publicKeyId)
 .ifMatch(getResponse.eTag()));

 if (deleteResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Successfully deleted Public Key [{}]", publicKeyId);
 }
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteKeyGroup](#)
  - [DeleteOriginAccessControl](#)
  - [DeletePublicKey](#)

## Sign URLs and cookies

The following code example shows how to create signed URLs and cookies that allow access to restricted resources.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the [CannedSignerRequest](#) class to sign URLs or cookies with a *canned* policy.

```
import software.amazon.awssdk.services.cloudfront.model.CannedSignerRequest;

import java.net.URL;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.temporal.ChronoUnit;

public class CreateCannedPolicyRequest {

 public static CannedSignerRequest createRequestForCannedPolicy(String
distributionDomainName,
 String fileNameToUpload,
 String privateKeyFullPath, String publicKeyId) throws Exception {
 String protocol = "https";
 String resourcePath = "/" + fileNameToUpload;

 String cloudFrontUrl = new URL(protocol, distributionDomainName,
resourcePath).toString();
 Instant expirationDate = Instant.now().plus(7, ChronoUnit.DAYS);
 Path path = Paths.get(privateKeyFullPath);

 return CannedSignerRequest.builder()
 .resourceUrl(cloudFrontUrl)
 .privateKey(path)
 .keyPairId(publicKeyId)
 .expirationDate(expirationDate)
 .build();
 }
}
```

```
 }
}
```

Use the [CustomSignerRequest](#) class to sign URLs or cookies with a *custom* policy. The `activeDate` and `ipRange` are optional methods.

```
import software.amazon.awssdk.services.cloudfront.model.CustomSignerRequest;

import java.net.URL;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.temporal.ChronoUnit;

public class CreateCustomPolicyRequest {

 public static CustomSignerRequest createRequestForCustomPolicy(String
distributionDomainName,
 String fileNameToUpload,
 String privateKeyFullPath, String publicKeyId) throws Exception {
 String protocol = "https";
 String resourcePath = "/" + fileNameToUpload;

 String cloudFrontUrl = new URL(protocol, distributionDomainName,
resourcePath).toString();
 Instant expireDate = Instant.now().plus(7, ChronoUnit.DAYS);
 // URL will be accessible tomorrow using the signed URL.
 Instant activeDate = Instant.now().plus(1, ChronoUnit.DAYS);
 Path path = Paths.get(privateKeyFullPath);

 return CustomSignerRequest.builder()
 .resourceUrl(cloudFrontUrl)
 // .resourceUrlPattern("https://*.example.com/*") // Optional.
 .privateKey(path)
 .keyPairId(publicKeyId)
 .expirationDate(expireDate)
 .activeDate(activeDate) // Optional.
 // .ipRange("192.168.0.1/24") // Optional.
 .build();
 }
}
```

The following example demonstrates the use of the [CloudFrontUtilities](#) class to produce signed cookies and URLs. [View](#) this code example on GitHub.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.cloudfront.CloudFrontUtilities;
import software.amazon.awssdk.services.cloudfront.cookie.CookiesForCannedPolicy;
import software.amazon.awssdk.services.cloudfront.cookie.CookiesForCustomPolicy;
import software.amazon.awssdk.services.cloudfront.model.CannedSignerRequest;
import software.amazon.awssdk.services.cloudfront.model.CustomSignerRequest;
import software.amazon.awssdk.services.cloudfront.url.SignedUrl;

public class SigningUtilities {
 private static final Logger logger =
 LoggerFactory.getLogger(SigningUtilities.class);
 private static final CloudFrontUtilities cloudFrontUtilities =
 CloudFrontUtilities.create();

 public static SignedUrl signUrlForCannedPolicy(CannedSignerRequest
cannedSignerRequest) {
 SignedUrl signedUrl =
cloudFrontUtilities.getSignedUrlWithCannedPolicy(cannedSignerRequest);
 logger.info("Signed URL: [{}]", signedUrl.url());
 return signedUrl;
 }

 public static SignedUrl signUrlForCustomPolicy(CustomSignerRequest
customSignerRequest) {
 SignedUrl signedUrl =
cloudFrontUtilities.getSignedUrlWithCustomPolicy(customSignerRequest);
 logger.info("Signed URL: [{}]", signedUrl.url());
 return signedUrl;
 }

 public static CookiesForCannedPolicy
getCookiesForCannedPolicy(CannedSignerRequest cannedSignerRequest) {
 CookiesForCannedPolicy cookiesForCannedPolicy = cloudFrontUtilities
 .getCookiesForCannedPolicy(cannedSignerRequest);
 logger.info("Cookie EXPIRES header [{}]",
cookiesForCannedPolicy.expiresHeaderValue());
 logger.info("Cookie KEYPAIR header [{}]",
cookiesForCannedPolicy.keyPairIdHeaderValue());
 }
}
```



```

 logger.info("Cookie SIGNATURE header [{}]",
cookiesForCannedPolicy.signatureHeaderValue());
 return cookiesForCannedPolicy;
 }

 public static CookiesForCustomPolicy
getCookiesForCustomPolicy(CustomSignerRequest customSignerRequest) {
 CookiesForCustomPolicy cookiesForCustomPolicy = cloudFrontUtilities
 .getCookiesForCustomPolicy(customSignerRequest);
 logger.info("Cookie POLICY header [{}]",
cookiesForCustomPolicy.policyHeaderValue());
 logger.info("Cookie KEYPAIR header [{}]",
cookiesForCustomPolicy.keyPairIdHeaderValue());
 logger.info("Cookie SIGNATURE header [{}]",
cookiesForCustomPolicy.signatureHeaderValue());
 return cookiesForCustomPolicy;
 }
}

```

- For API details, see [CloudFrontUtilities](#) in *AWS SDK for Java 2.x API Reference*.

## CloudWatch examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with CloudWatch.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)

- [Actions](#)
- [Scenarios](#)

## Get started

### Hello CloudWatch

The following code example shows how to get started using CloudWatch.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.ListMetricsRequest;
import software.amazon.awssdk.services.cloudwatch.paginators.ListMetricsIterable;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloService {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <namespace>\s

 Where:
 namespace - The namespace to filter against (for example, AWS/
EC2).\s
```

```

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String namespace = args[0];
 Region region = Region.US_EAST_1;
 CloudWatchClient cw = CloudWatchClient.builder()
 .region(region)
 .build();

 listMets(cw, namespace);
 cw.close();
}

public static void listMets(CloudWatchClient cw, String namespace) {
 try {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .build();

 ListMetricsIterable listRes = cw.listMetricsPaginator(request);
 listRes.stream()
 .flatMap(r -> r.metrics().stream())
 .forEach(metrics -> System.out.println(" Retrieved metric is: "
+ metrics.metricName()));

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListMetrics](#) in *AWS SDK for Java 2.x API Reference*.

# Basics

## Learn the basics

The following code example shows how to:

- List CloudWatch namespaces and metrics.
- Get statistics for a metric and for estimated billing.
- Create and update a dashboard.
- Create and add data to a metric.
- Create and trigger an alarm, then view alarm history.
- Add an anomaly detector.
- Get a metric image, then clean up resources.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating CloudWatch features.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import
 software.amazon.awssdk.services.cloudwatch.model.DashboardInvalidInputErrorException;
import software.amazon.awssdk.services.cloudwatch.model.DeleteAlarmsResponse;
import
 software.amazon.awssdk.services.cloudwatch.model.DeleteAnomalyDetectorResponse;
import software.amazon.awssdk.services.cloudwatch.model.DeleteDashboardsResponse;
import software.amazon.awssdk.services.cloudwatch.model.Dimension;
import software.amazon.awssdk.services.cloudwatch.model.GetMetricStatisticsResponse;
import software.amazon.awssdk.services.cloudwatch.model.LimitExceededException;
import software.amazon.awssdk.services.cloudwatch.model.PutDashboardResponse;
import software.amazon.awssdk.services.cloudwatch.model.PutMetricDataResponse;
import java.io.IOException;
```

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To enable billing metrics and statistics for this example, make sure billing
 * alerts are enabled for your account:
 * https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/monitor_estimated_charges_with_cloudwatch.html#turning_on_billing_metrics
 *
 * This Java code example performs the following tasks:
 *
 * 1. List available namespaces from Amazon CloudWatch.
 * 2. List available metrics within the selected Namespace.
 * 3. Get statistics for the selected metric over the last day.
 * 4. Get CloudWatch estimated billing for the last week.
 * 5. Create a new CloudWatch dashboard with metrics.
 * 6. List dashboards using a paginator.
 * 7. Create a new custom metric by adding data for it.
 * 8. Add the custom metric to the dashboard.
 * 9. Create an alarm for the custom metric.
 * 10. Describe current alarms.
 * 11. Get current data for the new custom metric.
 * 12. Push data into the custom metric to trigger the alarm.
 * 13. Check the alarm state using the action DescribeAlarmsForMetric.
 * 14. Get alarm history for the new alarm.
 * 15. Add an anomaly detector for the custom metric.
 * 16. Describe current anomaly detectors.
 * 17. Get a metric image for the custom metric.
 * 18. Clean up the Amazon CloudWatch resources.
 */
public class CloudWatchScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 static CloudWatchActions cwActions = new CloudWatchActions();
```

```

private static final Logger logger =
LoggerFactory.getLogger(CloudWatchScenario.class);
static Scanner scanner = new Scanner(System.in);
public static void main(String[] args) throws Throwable {

 final String usage = ""

 Usage:
 <myDate> <costDateWeek> <dashboardName> <dashboardJson> <dashboardAdd>
<settings> <metricImage> \s

 Where:
 myDate - The start date to use to get metric statistics. (For example,
2023-01-11T18:35:24.00Z.)\s
 costDateWeek - The start date to use to get AWS/Billing statistics.
(For example, 2023-01-11T18:35:24.00Z.)\s
 dashboardName - The name of the dashboard to create.\s
 dashboardJson - The location of a JSON file to use to create a
dashboard. (See jsonWidgets.json in javav2/example_code/cloudwatch.)\s
 dashboardAdd - The location of a JSON file to use to update a
dashboard. (See CloudDashboard.json in javav2/example_code/cloudwatch.)\s
 settings - The location of a JSON file from which various values are
read. (See settings.json in javav2/example_code/cloudwatch.)\s
 metricImage - The location of a BMP file that is used to create a
graph.\s

 """;

 if (args.length != 7) {
 logger.info(usage);
 return;
 }
 String myDate = args[0];
 String costDateWeek = args[1];
 String dashboardName = args[2];
 String dashboardJson = args[3];
 String dashboardAdd = args[4];
 String settings = args[5];
 String metricImage = args[6];

 logger.info(DASHES);
 logger.info("Welcome to the Amazon CloudWatch Basics scenario.");
 logger.info("")
 Amazon CloudWatch is a comprehensive monitoring and observability
service

```

provided by Amazon Web Services (AWS). It is designed to help you monitor your AWS resources, applications, and services, as well as on-premises resources, in real-time.

CloudWatch collects and tracks various types of data, including metrics, logs, and events, from your AWS and on-premises resources. It allows you to set alarms and automatically respond to changes in your environment, enabling you to quickly identify and address issues before they impact your applications or services.

With CloudWatch, you can gain visibility into your entire infrastructure, from the cloud to the edge, and use this information to make informed decisions and optimize your resource utilization.

This scenario guides you through how to perform Amazon CloudWatch tasks by using the

AWS SDK for Java v2. Let's get started...

```

 """);
 waitForInputToContinue(scanner);

 try {
 runScenario(myDate, costDateWeek, dashboardName, dashboardJson,
dashboardAdd, settings, metricImage);
 } catch (RuntimeException e) {
 e.printStackTrace();
 }
 logger.info(DASHES);
}

private static void runScenario(String myDate, String costDateWeek, String
dashboardName, String dashboardJson, String dashboardAdd, String settings, String
metricImage) throws Throwable {
 Double dataPoint = Double.parseDouble("10.0");
 logger.info(DASHES);
 logger.info("
1. List at least five available unique namespaces from Amazon CloudWatch.
Select one from the list.

```

```

 "");
 String selectedNamespace;
 String selectedMetrics;
 int num;
 try {
 CompletableFuture<ArrayList<String>> future =
cwActions.listNameSpacesAsync();
 ArrayList<String> list = future.join();
 for (int z = 0; z < 5; z++) {
 int index = z + 1;
 logger.info(" " + index + ". {}", list.get(z));
 }

 num = Integer.parseInt(scanner.nextLine());
 if (1 <= num && num <= 5) {
 selectedNamespace = list.get(num - 1);
 } else {
 logger.info("You did not select a valid option.");
 return;
 }
 logger.info("You selected {}", selectedNamespace);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("2. List available metrics within the selected namespace.");
 logger.info("
 A metric is a measure of the performance or health of your AWS
resources,
 applications, or custom resources. Metrics are the basic building blocks
of CloudWatch
 and provide data points that represent a specific aspect of your system
or application over time.

```



```

 Select a metric from the list.
 """);

 Dimension myDimension = null;
 try {
 CompletableFuture<ArrayList<String>> future =
cwActions.listMetsAsync(selectedNamespace);
 ArrayList<String> metList = future.join();
 logger.info("Metrics successfully retrieved. Total metrics: {}",
metList.size());
 for (int z = 0; z < 5; z++) {
 int index = z + 1;
 logger.info(" " + index + ". " + metList.get(z));
 }
 num = Integer.parseInt(scanner.nextLine());
 if (1 <= num && num <= 5) {
 selectedMetrics = metList.get(num - 1);
 } else {
 logger.info("You did not select a valid option.");
 return;
 }
 logger.info("You selected {}", selectedMetrics);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 try {
 myDimension = cwActions.getSpecificMetAsync(selectedNamespace).join();
 logger.info("Metric statistics successfully retrieved and displayed.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {

```

```

 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
}

waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("3. Get statistics for the selected metric over the last day.");
logger.info("""
 Statistics refer to the various mathematical calculations that can be
performed on the
 collected metrics to derive meaningful insights. Statistics provide a
way to summarize and
 analyze the data collected for a specific metric over a specified time
period.
 """);
waitForInputToContinue(scanner);
String metricOption = "";
ArrayList<String> statTypes = new ArrayList<>();
statTypes.add("SampleCount");
statTypes.add("Average");
statTypes.add("Sum");
statTypes.add("Minimum");
statTypes.add("Maximum");

for (int t = 0; t < 5; t++) {
 logger.info(" " + (t + 1) + ". {}", statTypes.get(t));
}
logger.info("Select a metric statistic by entering a number from the
preceding list:");
num = Integer.parseInt(scanner.nextLine());
if (1 <= num && num <= 5) {
 metricOption = statTypes.get(num - 1);
} else {
 logger.info("You did not select a valid option.");
 return;
}
logger.info("You selected " + metricOption);
waitForInputToContinue(scanner);
try {

```

```

 CompletableFuture<GetMetricStatisticsResponse> future =
cwActions.getAndDisplayMetricStatisticsAsync(selectedNamespace, selectedMetrics,
metricOption, myDate, myDimension);
 future.join();
 logger.info("Metric statistics retrieved successfully.");

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("4. Get CloudWatch estimated billing for the last week.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<GetMetricStatisticsResponse> future =
cwActions.getMetricStatisticsAsync(costDateWeek);
 future.join();

 logger.info("Metric statistics successfully retrieved and displayed.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("5. Create a new CloudWatch dashboard with metrics.");
 waitForInputToContinue(scanner);

```

```

 try {
 CompletableFuture<PutDashboardResponse> future =
cwActions.createDashboardWithMetricsAsync(dashboardName, dashboardJson);
 future.join();

 } catch (RuntimeException | IOException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof DashboardInvalidInputErrorException cwEx) {
 logger.info("Invalid CloudWatch data. Error message: {}, Error code
{}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("6. List dashboards using a paginator.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future = cwActions.listDashboardsAsync();
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Create a new custom metric by adding data to it.");
 logger.info(""""
 The primary benefit of using a custom metric in Amazon CloudWatch is the
ability to

```

```

 monitor and collect data that is specific to your application or
 infrastructure.
 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<PutMetricDataResponse> future =
 cwActions.createNewCustomMetricAsync(dataPoint);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("8. Add an additional metric to the dashboard.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<PutDashboardResponse> future =
 cwActions.addMetricToDashboardAsync(dashboardAdd, dashboardName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof DashboardInvalidInputErrorException cwEx) {
 logger.info("Invalid CloudWatch data. Error message: {}, Error code
{}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("9. Create an alarm for the custom metric.");

```

```

 waitForInputToContinue(scanner);
 String alarmName = "" ;
 try {
 CompletableFuture<String> future = cwActions.createAlarmAsync(settings);
 alarmName = future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof LimitExceededException cwEx) {
 logger.info("The quota for alarms has been reached: Error message:
{}, Error code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("10. Describe ten current alarms.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future = cwActions.describeAlarmsAsync();
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("11. Get current data for new custom metric.");
 try {
 CompletableFuture<Void> future =
cwActions.getCustomMetricDataAsync(settings);

```

```

 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("12. Push data into the custom metric to trigger the alarm.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<PutMetricDataResponse> future =
cwActions.addMetricDataForAlarmAsync(settings);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("13. Check the alarm state using the action
DescribeAlarmsForMetric.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
cwActions.checkForMetricAlarmAsync(settings);
 future.join();
 }

```

```

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("14. Get alarm history for the new alarm.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
cwActions.getAlarmHistoryAsync(settings, myDate);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("15. Add an anomaly detector for the custom metric.");
 logger.info("""
 An anomaly detector is a feature that automatically detects unusual
patterns or deviations in your
 monitored metrics. It uses machine learning algorithms to analyze the
historical behavior
 of your metrics and establish a baseline.
 """);

```



The anomaly detector then compares the current metric values against this baseline and identifies any anomalies or outliers that may indicate potential issues or unexpected changes in your system's performance or behavior.

```

 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
 cwActions.addAnomalyDetectorAsync(settings);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("16. Describe current anomaly detectors.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
 cwActions.describeAnomalyDetectorsAsync(settings);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 }
}

```

```

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("17. Get a metric image for the custom metric.");
 try {
 CompletableFuture<Void> future =
cwActions.downloadAndSaveMetricImageAsync(metricImage);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("18. Clean up the Amazon CloudWatch resources.");

 try {
 logger.info(". Delete the Dashboard.");
 waitForInputToContinue(scanner);
 CompletableFuture<DeleteDashboardsResponse> future =
cwActions.deleteDashboardAsync(dashboardName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 try {

```

```

 logger.info("Delete the alarm.");
 waitForInputToContinue(scanner);
 CompletableFuture<DeleteAlarmsResponse> future =
cwActions.deleteCWAlarmAsync(alarmName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 try {
 logger.info("Delete the anomaly detector.");
 waitForInputToContinue(scanner);
 CompletableFuture<DeleteAnomalyDetectorResponse> future =
cwActions.deleteAnomalyDetectorAsync(settings);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof CloudWatchException cwEx) {
 logger.info("CloudWatch error occurred: Error message: {}, Error
code {}", cwEx.getMessage(), cwEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("The Amazon CloudWatch example scenario is complete.");
 logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {

```

```

 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();
 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}
}

```

A wrapper class for CloudWatch SDK methods.

```

public class CloudWatchActions {

 private static CloudWatchAsyncClient cloudWatchAsyncClient;

 private static final Logger logger =
 LoggerFactory.getLogger(CloudWatchActions.class);

 /**
 * Retrieves an asynchronous CloudWatch client instance.
 *
 * <p>
 * This method ensures that the CloudWatch client is initialized with the
 following configurations:
 *
 * Maximum concurrency: 100
 * Connection timeout: 60 seconds
 * Read timeout: 60 seconds
 * Write timeout: 60 seconds
 * API call timeout: 2 minutes
 * API call attempt timeout: 90 seconds
 * Retry strategy: STANDARD
 *
 * </p>
 *
 * @return the asynchronous CloudWatch client instance
 */
}

```

```

 */
 private static CloudWatchAsyncClient getAsyncClient() {
 if (cloudWatchAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 cloudWatchAsyncClient = CloudWatchAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return cloudWatchAsyncClient;
 }

 /**
 * Deletes an Anomaly Detector.
 *
 * @param fileName the name of the file containing the Anomaly Detector
 configuration
 * @return a CompletableFuture that represents the asynchronous deletion of the
 Anomaly Detector
 */
 public CompletableFuture<DeleteAnomalyDetectorResponse>
 deleteAnomalyDetectorAsync(String fileName) {
 CompletableFuture<JsonNode> readFileFuture =
 CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
 File(fileName));
 return new ObjectMapper().readTree(parser); // Return the root node
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 })
 }

```

```

 });

 return readFileFuture.thenCompose(rootNode -> {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 SingleMetricAnomalyDetector singleMetricAnomalyDetector =
SingleMetricAnomalyDetector.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .stat("Maximum")
 .build();

 DeleteAnomalyDetectorRequest request =
DeleteAnomalyDetectorRequest.builder()
 .singleMetricAnomalyDetector(singleMetricAnomalyDetector)
 .build();

 return getAsyncClient().deleteAnomalyDetector(request);
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the Anomaly Detector",
exception);
 } else {
 logger.info("Successfully deleted the Anomaly Detector.");
 }
 });
}

/**
 * Deletes a CloudWatch alarm.
 *
 * @param alarmName the name of the alarm to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
to delete the alarm
 * the {@link DeleteAlarmsResponse} is returned when the operation completes
successfully,
 * or a {@link RuntimeException} is thrown if the operation fails
 */
public CompletableFuture<DeleteAlarmsResponse> deleteCWAlarmAsync(String
alarmName) {
 DeleteAlarmsRequest request = DeleteAlarmsRequest.builder()

```

```

 .alarmNames(alarmName)
 .build();

 return getAsyncClient().deleteAlarms(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the alarm:{} " +
alarmName, exception);
 } else {
 logger.info("Successfully deleted alarm {} ", alarmName);
 }
 });
 }

 /**
 * Deletes the specified dashboard.
 *
 * @param dashboardName the name of the dashboard to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
of deleting the dashboard
 * @throws RuntimeException if the dashboard deletion fails
 */
 public CompletableFuture<DeleteDashboardsResponse> deleteDashboardAsync(String
dashboardName) {
 DeleteDashboardsRequest dashboardsRequest =
DeleteDashboardsRequest.builder()
 .dashboardNames(dashboardName)
 .build();

 return getAsyncClient().deleteDashboards(dashboardsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the dashboard: " +
dashboardName, exception);
 } else {
 logger.info("{} was successfully deleted.", dashboardName);
 }
 });
 }

 /**
 * Retrieves and saves a custom metric image to a file.
 *

```

```

 * @param fileName the name of the file to save the metric image to
 * @return a {@link CompletableFuture} that completes when the image has been
 saved to the file
 */
 public CompletableFuture<Void> downloadAndSaveMetricImageAsync(String fileName)
 {
 logger.info("Getting Image data for custom metric.");
 String myJSON = ""
 {
 "title": "Example Metric Graph",
 "view": "timeSeries",
 "stacked ": false,
 "period": 10,
 "width": 1400,
 "height": 600,
 "metrics": [
 [
 "AWS/Billing",
 "EstimatedCharges",
 "Currency",
 "USD"
]
]
 }
 """;

 GetMetricWidgetImageRequest imageRequest =
 GetMetricWidgetImageRequest.builder()
 .metricWidget(myJSON)
 .build();

 return getAsyncClient().getMetricWidgetImage(imageRequest)
 .thenCompose(response -> {
 SdkBytes sdkBytes = response.metricWidgetImage();
 byte[] bytes = sdkBytes.asByteArray();
 return CompletableFuture.runAsync(() -> {
 try {
 File outputFile = new File(fileName);
 try (FileOutputStream outputStream = new
 FileOutputStream(outputFile)) {
 outputStream.write(bytes);
 }
 } catch (IOException e) {

```



```

 throw new RuntimeException("Failed to write image to file",
e);
 }
 });
}
 .whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error getting and saving metric
image", exception);
 } else {
 logger.info("Image data saved successfully to {}", fileName);
 }
 });
}

/**
 * Describes the anomaly detectors based on the specified JSON file.
 *
 * @param fileName the name of the JSON file containing the custom metric
namespace and name
 * @return a {@link CompletableFuture} that completes when the anomaly detectors
have been described
 * @throws RuntimeException if there is a failure during the operation, such as
when reading or parsing the JSON file,
 *
 * or when describing the anomaly detectors
 */
public CompletableFuture<Void> describeAnomalyDetectorsAsync(String fileName) {
 CompletableFuture<JsonNode> readFileFuture =
CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 return new ObjectMapper().readTree(parser);
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 });

 return readFileFuture.thenCompose(rootNode -> {
 try {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();

```

```

 String customMetricName =
rootNode.findValue("customMetricName").asText();

 DescribeAnomalyDetectorsRequest detectorsRequest =
DescribeAnomalyDetectorsRequest.builder()
 .maxResults(10)
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 return
getAsyncClient().describeAnomalyDetectors(detectorsRequest).thenAccept(response ->
{
 List<AnomalyDetector> anomalyDetectorList =
response.anomalyDetectors();
 for (AnomalyDetector detector : anomalyDetectorList) {
 logger.info("Metric name: {} ",
detector.singleMetricAnomalyDetector().metricName());
 logger.info("State: {} ", detector.stateValue());
 }
 });
 } catch (RuntimeException e) {
 throw new RuntimeException("Failed to describe anomaly detectors",
e);
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error describing anomaly detectors",
exception);
 }
 });
}

/**
 * Adds an anomaly detector for the given file.
 *
 * @param fileName the name of the file containing the anomaly detector
configuration
 * @return a {@link CompletableFuture} that completes when the anomaly detector
has been added
 */
public CompletableFuture<Void> addAnomalyDetectorAsync(String fileName) {

```

```

 CompletableFuture<JsonNode> readFileFuture =
CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 return new ObjectMapper().readTree(parser); // Return the root node
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
});

return readFileFuture.thenCompose(rootNode -> {
 try {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 SingleMetricAnomalyDetector singleMetricAnomalyDetector =
SingleMetricAnomalyDetector.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .stat("Maximum")
 .build();

 PutAnomalyDetectorRequest anomalyDetectorRequest =
PutAnomalyDetectorRequest.builder()
 .singleMetricAnomalyDetector(singleMetricAnomalyDetector)
 .build();

 return
getAsyncClient().putAnomalyDetector(anomalyDetectorRequest).thenAccept(response ->
{
 logger.info("Added anomaly detector for metric {}",
customMetricName);
 });
 } catch (Exception e) {
 throw new RuntimeException("Failed to create anomaly detector", e);
 }
}).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error adding anomaly detector",
exception);
 }
}

```

```

 });
}

/**
 * Retrieves the alarm history for a given alarm name and date range.
 *
 * @param fileName the path to the JSON file containing the alarm name
 * @param date the date to start the alarm history search (in the format
 * "yyyy-MM-dd'T'HH:mm:ss'Z'")
 * @return a {@code CompletableFuture<Void>} that completes when the alarm
 * history has been retrieved and processed
 */
public CompletableFuture<Void> getAlarmHistoryAsync(String fileName, String
date) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.findValue("exampleAlarmName").asText(); // Return
alarmName from the JSON file
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 });

 // Use the alarm name to describe alarm history with a paginator.
 return readFileFuture.thenCompose(alarmName -> {
 try {
 Instant start = Instant.parse(date);
 Instant endDate = Instant.now();
 DescribeAlarmHistoryRequest historyRequest =
DescribeAlarmHistoryRequest.builder()
 .startDate(start)
 .endDate(endDate)
 .alarmName(alarmName)
 .historyItemType(HistoryItemType.ACTION)
 .build();

 // Use the paginator to paginate through alarm history pages.

```

```

 DescribeAlarmHistoryPublisher historyPublisher =
getAsyncClient().describeAlarmHistoryPaginator(historyRequest);
 CompletableFuture<Void> future = historyPublisher
 .subscribe(response -> response.alarmHistoryItems().forEach(item
-> {
 logger.info("History summary: {}", item.historySummary());
 logger.info("Timestamp: {}", item.timestamp());
 })))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 logger.error("Error occurred while getting alarm
history: " + exception.getMessage(), exception);
 } else {
 logger.info("Successfully retrieved all alarm
history.");
 }
 });

 // Return the future to the calling code for further handling
 return future;
 } catch (Exception e) {
 throw new RuntimeException("Failed to process alarm history", e);
 }
}).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error completing alarm history
processing", exception);
 }
});
}

/**
 * Checks for a metric alarm in AWS CloudWatch.
 *
 * @param fileName the name of the file containing the JSON configuration for
the custom metric
 * @return a {@link CompletableFuture} that completes when the check for the
metric alarm is complete
 */
public CompletableFuture<Void> checkForMetricAlarmAsync(String fileName) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {

```

```

 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {
 throw new RuntimeException("Failed to read file", e);
 }
 });

 return readFileFuture.thenCompose(jsonContent -> {
 try {
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 DescribeAlarmsForMetricRequest metricRequest =
DescribeAlarmsForMetricRequest.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 return checkForAlarmAsync(metricRequest, customMetricName, 10);

 } catch (IOException e) {
 throw new RuntimeException("Failed to parse JSON content", e);
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error checking metric alarm",
exception);
 }
 });
}

// Recursive method to check for the alarm.

/**
 * Checks for the existence of an alarm asynchronously for the specified metric.

```

```

 *
 * @param metricRequest the request to describe the alarms for the specified
metric
 * @param customMetricName the name of the custom metric to check for an alarm
 * @param retries the number of retries to perform if no alarm is found
 * @return a {@link CompletableFuture} that completes when an alarm is found or
the maximum number of retries has been reached
 */
 private static CompletableFuture<Void>
checkForAlarmAsync(DescribeAlarmsForMetricRequest metricRequest, String
customMetricName, int retries) {
 if (retries == 0) {
 return CompletableFuture.completedFuture(null).thenRun(() ->
 logger.info("No Alarm state found for {} after 10 retries.",
customMetricName)
);
 }

 return
(getAsyncClient().describeAlarmsForMetric(metricRequest).thenCompose(response -> {
 if (response.hasMetricAlarms()) {
 logger.info("Alarm state found for {}", customMetricName);
 return CompletableFuture.completedFuture(null); // Alarm found,
complete the future
 } else {
 return CompletableFuture.runAsync(() -> {
 try {
 Thread.sleep(20000);
 logger.info(".");
 } catch (InterruptedException e) {
 throw new RuntimeException("Interrupted while waiting to
retry", e);
 }
 }).thenCompose(v -> checkForAlarmAsync(metricRequest,
customMetricName, retries - 1)); // Recursive call
 }
 }));
 }

 /**
 * Adds metric data for an alarm asynchronously.
 *
 * @param fileName the name of the JSON file containing the metric data

```

```

 * @return a CompletableFuture that asynchronously returns the
 PutMetricDataResponse
 */
 public CompletableFuture<PutMetricDataResponse>
 addMetricDataForAlarmAsync(String fileName) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {
 throw new RuntimeException("Failed to read file", e);
 }
 });

 return readFileFuture.thenCompose(jsonContent -> {
 try {
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();
 Instant instant = Instant.now();

 // Create MetricDatum objects.
 MetricDatum datum1 = MetricDatum.builder()
 .metricName(customMetricName)
 .unit(StandardUnit.NONE)
 .value(1001.00)
 .timestamp(instant)
 .build();

 MetricDatum datum2 = MetricDatum.builder()
 .metricName(customMetricName)
 .unit(StandardUnit.NONE)
 .value(1002.00)
 .timestamp(instant)
 .build();
 }
 });
 }

```



```

 List<MetricDatum> metricDataList = new ArrayList<>();
 metricDataList.add(datum1);
 metricDataList.add(datum2);

 // Build the PutMetricData request.
 PutMetricDataRequest request = PutMetricDataRequest.builder()
 .namespace(customMetricNamespace)
 .metricData(metricDataList)
 .build();

 // Send the request asynchronously.
 return getAsyncClient().putMetricData(request);

 } catch (IOException e) {
 CompletableFuture<PutMetricDataResponse> failedFuture = new
CompletableFuture<>();
 failedFuture.completeExceptionally(new RuntimeException("Failed to
parse JSON content", e));
 return failedFuture;
 }
}).whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to put metric data: " + exception.getMessage(),
exception);
 } else {
 logger.info("Added metric values for metric.");
 }
});
}

/**
 * Retrieves custom metric data from the AWS CloudWatch service.
 *
 * @param fileName the name of the file containing the custom metric information
 * @return a {@link CompletableFuture} that completes when the metric data has
been retrieved
 */
public CompletableFuture<Void> getCustomMetricDataAsync(String fileName) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 // Read values from the JSON file.

```

```

 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {
 throw new RuntimeException("Failed to read file", e);
 }
});

return readFileFuture.thenCompose(jsonContent -> {
 try {
 // Parse the JSON string to extract relevant values.
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 // Set the current time and date range for metric query.
 Instant nowDate = Instant.now();
 long hours = 1;
 long minutes = 30;
 Instant endTime = nowDate.plus(hours,
ChronoUnit.HOURS).plus(minutes, ChronoUnit.MINUTES);

 Metric met = Metric.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 MetricStat metStat = MetricStat.builder()
 .stat("Maximum")
 .period(60) // Assuming period in seconds
 .metric(met)
 .build();

 MetricDataQuery dataQuery = MetricDataQuery.builder()
 .metricStat(metStat)
 .id("foo2")
 .returnData(true)
 .build();
 }
});

```

```

 List<MetricDataQuery> dq = new ArrayList<>();
 dq.add(dataQuery);

 GetMetricDataRequest getMetricDataRequest =
GetMetricDataRequest.builder()
 .maxDatapoints(10)
 .scanBy(ScanBy.TIMESTAMP_DESCENDING)
 .startTime(nowDate)
 .endTime(endTime)
 .metricDataQueries(dq)
 .build();

 // Call the async method for CloudWatch data retrieval.
 return getAsyncClient().getMetricData(getMetricDataRequest);

 } catch (IOException e) {
 throw new RuntimeException("Failed to parse JSON content", e);
 }
}).thenAccept(response -> {
 List<MetricDataResult> data = response.metricDataResults();
 for (MetricDataResult item : data) {
 logger.info("The label is: {}", item.label());
 logger.info("The status code is: {}", item.statusCode().toString());
 }
}).exceptionally(exception -> {
 throw new RuntimeException("Failed to get metric data", exception);
});
}

/**
 * Describes the CloudWatch alarms of the 'METRIC_ALARM' type.
 *
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of describing the CloudWatch alarms. The future completes when the
 * operation is finished, either successfully or with an error.
 */
public CompletableFuture<Void> describeAlarmsAsync() {
 List<AlarmType> typeList = new ArrayList<>();
 typeList.add(AlarmType.METRIC_ALARM);
 DescribeAlarmsRequest alarmsRequest = DescribeAlarmsRequest.builder()
 .alarmTypes(typeList)

```

```

 .maxRecords(10)
 .build();

return getAsyncClient().describeAlarms(alarmsRequest)
 .thenAccept(response -> {
 List<MetricAlarm> alarmList = response.metricAlarms();
 for (MetricAlarm alarm : alarmList) {
 logger.info("Alarm name: {}", alarm.alarmName());
 logger.info("Alarm description: {} ", alarm.alarmDescription());
 }
 })
 .whenComplete((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to describe alarms: {}", ex.getMessage());
 } else {
 logger.info("Successfully described alarms.");
 }
 });
}

/**
 * Creates an alarm based on the configuration provided in a JSON file.
 *
 * @param fileName the name of the JSON file containing the alarm configuration
 * @return a CompletableFuture that represents the asynchronous operation of
creating the alarm
 * @throws RuntimeException if an exception occurs while reading the JSON file
or creating the alarm
 */
public CompletableFuture<String> createAlarmAsync(String fileName) {
 com.fasterxml.jackson.databind.JsonNode rootNode;
 try {
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 rootNode = new ObjectMapper().readTree(parser);
 } catch (IOException e) {
 throw new RuntimeException("Failed to read the alarm configuration
file", e);
 }

 // Extract values from the JSON node.
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName = rootNode.findValue("customMetricName").asText();
 String alarmName = rootNode.findValue("exampleAlarmName").asText();

```

```

String emailTopic = rootNode.findValue("emailTopic").asText();
String accountId = rootNode.findValue("accountId").asText();
String region = rootNode.findValue("region").asText();

// Create a List for alarm actions.
List<String> alarmActions = new ArrayList<>();
alarmActions.add("arn:aws:sns:" + region + ":" + accountId + ":" +
emailTopic);

PutMetricAlarmRequest alarmRequest = PutMetricAlarmRequest.builder()
 .alarmActions(alarmActions)
 .alarmDescription("Example metric alarm")
 .alarmName(alarmName)

.comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
 .threshold(100.00)
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .evaluationPeriods(1)
 .period(10)
 .statistic("Maximum")
 .datapointsToAlarm(1)
 .treatMissingData("ignore")
 .build();

// Call the putMetricAlarm asynchronously and handle the result.
return getAsyncClient().putMetricAlarm(alarmRequest)
 .handle((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to create alarm: {}", ex.getMessage());
 throw new RuntimeException("Failed to create alarm", ex);
 } else {
 logger.info("{} was successfully created!", alarmName);
 return alarmName;
 }
 });
}

/**
 * Adds a metric to a dashboard asynchronously.
 *
 * @param fileName the name of the file containing the dashboard content
 * @param dashboardName the name of the dashboard to be updated

```

```

 * @return a {@link CompletableFuture} representing the asynchronous operation,
 which will complete with a
 * {@link PutDashboardResponse} when the dashboard is successfully updated
 */
 public CompletableFuture<PutDashboardResponse> addMetricToDashboardAsync(String
fileName, String dashboardName) {
 String dashboardBody;
 try {
 dashboardBody = readFileAsString(fileName);
 } catch (IOException e) {
 throw new RuntimeException("Failed to read the dashboard file", e);
 }

 PutDashboardRequest dashboardRequest = PutDashboardRequest.builder()
 .dashboardName(dashboardName)
 .dashboardBody(dashboardBody)
 .build();

 return getAsyncClient().putDashboard(dashboardRequest)
 .handle((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to update dashboard: {}", ex.getMessage());
 throw new RuntimeException("Error updating dashboard", ex);
 } else {
 logger.info("{} was successfully updated.", dashboardName);
 return response;
 }
 });
 }

 /**
 * Creates a new custom metric.
 *
 * @param dataPoint the data point to be added to the custom metric
 * @return a {@link CompletableFuture} representing the asynchronous operation
 of adding the custom metric
 */
 public CompletableFuture<PutMetricDataResponse>
createNewCustomMetricAsync(Double dataPoint) {
 Dimension dimension = Dimension.builder()
 .name("UNIQUE_PAGES")
 .value("URLS")
 .build();

```

```

 // Set an Instant object for the current time in UTC.
 String time =
ZonedDateTime.now(ZoneOffset.UTC).format(DateTimeFormatter.ISO_INSTANT);
 Instant instant = Instant.parse(time);

 // Create the MetricDatum.
 MetricDatum datum = MetricDatum.builder()
 .metricName("PAGES_VISITED")
 .unit(StandardUnit.NONE)
 .value(dataPoint)
 .timestamp(instant)
 .dimensions(dimension)
 .build();

 PutMetricDataRequest request = PutMetricDataRequest.builder()
 .namespace("SITE/TRAFFIC")
 .metricData(datum)
 .build();

 return getAsyncClient().putMetricData(request)
 .whenComplete((response, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Error adding custom metric", ex);
 } else {
 logger.info("Successfully added metric values for
PAGES_VISITED.");
 }
 });
 }

 /**
 * Lists the available dashboards.
 *
 * @return a {@link CompletableFuture} that completes when the operation is
 finished.
 * The future will complete exceptionally if an error occurs while listing the
 dashboards.
 */
 public CompletableFuture<Void> listDashboardsAsync() {
 ListDashboardsRequest listDashboardsRequest =
ListDashboardsRequest.builder().build();
 ListDashboardsPublisher paginator =
getAsyncClient().listDashboardsPaginator(listDashboardsRequest);
 return paginator.subscribe(response -> {

```

```

 response.dashboardEntries().forEach(entry -> {
 logger.info("Dashboard name is: {} ", entry.dashboardName());
 logger.info("Dashboard ARN is: {} ", entry.dashboardArn());
 });
 }).exceptionally(ex -> {
 logger.info("Failed to list dashboards: {} ", ex.getMessage());
 throw new RuntimeException("Error occurred while listing dashboards",
ex);
 });
}

/**
 * Creates a new dashboard with the specified name and metrics from the given
file.
 *
 * @param dashboardName the name of the dashboard to be created
 * @param fileName the name of the file containing the dashboard body
 * @return a {@link CompletableFuture} representing the asynchronous operation
of creating the dashboard
 * @throws IOException if there is an error reading the dashboard body from the
file
 */
public CompletableFuture<PutDashboardResponse>
createDashboardWithMetricsAsync(String dashboardName, String fileName) throws
IOException {
 String dashboardBody = readFileAsString(fileName);
 PutDashboardRequest dashboardRequest = PutDashboardRequest.builder()
 .dashboardName(dashboardName)
 .dashboardBody(dashboardBody)
 .build();

 return getAsyncClient().putDashboard(dashboardRequest)
 .handle((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to create dashboard: {}", ex.getMessage());
 throw new RuntimeException("Dashboard creation failed", ex);
 } else {
 // Handle the normal response case
 logger.info("{} was successfully created.", dashboardName);
 List<DashboardValidationMessage> messages =
response.dashboardValidationMessages();
 if (messages.isEmpty()) {
 logger.info("There are no messages in the new Dashboard.");

```



```

 } else {
 for (DashboardValidationMessage message : messages) {
 logger.info("Message: {}", message.message());
 }
 }
 return response; // Return the response for further use
 }
}

/**
 * Retrieves the metric statistics for the "EstimatedCharges" metric in the
 * "AWS/Billing" namespace.
 *
 * @param costDateWeek the start date for the metric statistics, in the format
 * of an ISO-8601 date string (e.g., "2023-04-05")
 * @return a {@link CompletableFuture} that, when completed, contains the {@link
 * GetMetricStatisticsResponse} with the retrieved metric statistics
 * @throws RuntimeException if the metric statistics cannot be retrieved
 * successfully
 */
public CompletableFuture<GetMetricStatisticsResponse>
getMetricStatisticsAsync(String costDateWeek) {
 Instant start = Instant.parse(costDateWeek);
 Instant endDate = Instant.now();

 // Define dimension
 Dimension dimension = Dimension.builder()
 .name("Currency")
 .value("USD")
 .build();

 List<Dimension> dimensionList = new ArrayList<>();
 dimensionList.add(dimension);

 GetMetricStatisticsRequest statisticsRequest =
 GetMetricStatisticsRequest.builder()
 .metricName("EstimatedCharges")
 .namespace("AWS/Billing")
 .dimensions(dimensionList)
 .statistics(Statistic.MAXIMUM)
 .startTime(start)
 .endTime(endDate)

```

```

 .period(86400) // One day period
 .build();

 return getAsyncClient().getMetricStatistics(statisticsRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 List<Datapoint> data = response.datapoints();
 if (!data.isEmpty()) {
 for (Datapoint datapoint : data) {
 logger.info("Timestamp: {} Maximum value: {}",
datapoint.timestamp(), datapoint.maximum());
 }
 } else {
 logger.info("The returned data list is empty");
 }
 } else {
 throw new RuntimeException("Failed to get metric statistics: " +
exception.getMessage(), exception);
 }
 });
 }

/**
 * Retrieves and displays metric statistics for the specified parameters.
 *
 * @param namespace the namespace for the metric
 * @param metVal the name of the metric
 * @param metricOption the statistic to retrieve for the metric (e.g.,
"Maximum", "Average")
 * @param date the date for which to retrieve the metric statistics, in
the format "yyyy-MM-dd'T'HH:mm:ss'Z'"
 * @param myDimension the dimension(s) to filter the metric statistics by
 * @return a {@link CompletableFuture} that completes when the metric statistics
have been retrieved and displayed
 */
 public CompletableFuture<GetMetricStatisticsResponse>
getAndDisplayMetricStatisticsAsync(String namespace, String metVal,

 String metricOption, String date, Dimension myDimension) {

 Instant start = Instant.parse(date);
 Instant endDate = Instant.now();

```

```

 // Building the request for metric statistics.
 GetMetricStatisticsRequest statisticsRequest =
 GetMetricStatisticsRequest.builder()
 .endTime(endDate)
 .startTime(start)
 .dimensions(myDimension)
 .metricName(metVal)
 .namespace(nameSpace)
 .period(86400) // 1 day period
 .statistics(Statistic.fromValue(metricOption))
 .build();

 return getAsyncClient().getMetricStatistics(statisticsRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 List<Datapoint> data = response.datapoints();
 if (!data.isEmpty()) {
 for (Datapoint datapoint : data) {
 logger.info("Timestamp: {} Maximum value: {}",
 datapoint.timestamp(), datapoint.maximum());
 }
 } else {
 logger.info("The returned data list is empty");
 }
 } else {
 logger.info("Failed to get metric statistics: {} ",
 exception.getMessage());
 }
 })
 .exceptionally(exception -> {
 throw new RuntimeException("Error while getting metric statistics: "
 + exception.getMessage(), exception);
 });
 }

 /**
 * Retrieves a list of metric names for the specified namespace.
 *
 * @param namespace the namespace for which to retrieve the metric names
 * @return a {@link CompletableFuture} that, when completed, contains an {@link
 * ArrayList} of
 * * the metric names in the specified namespace
 * * @throws RuntimeException if an error occurs while listing the metrics

```

```

 */
 public CompletableFuture<ArrayList<String>> listMetsAsync(String namespace) {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .build();

 ListMetricsPublisher metricsPaginator =
getAsyncClient().listMetricsPaginator(request);
 Set<String> metSet = new HashSet<>();
 CompletableFuture<Void> future = metricsPaginator.subscribe(response -> {
 response.metrics().forEach(metric -> {
 String metricName = metric.metricName();
 metSet.add(metricName);
 });
 });

 return future
 .thenApply(ignored -> new ArrayList<>(metSet))
 .exceptionally(exception -> {
 throw new RuntimeException("Failed to list metrics: " +
exception.getMessage(), exception);
 });
 }

 /**
 * Lists the available namespaces for the current AWS account.
 *
 * @return a {@link CompletableFuture} that, when completed, contains an {@link
ArrayList} of the available namespace names.
 * @throws RuntimeException if an error occurs while listing the namespaces.
 */
 public CompletableFuture<ArrayList<String>> listNameSpacesAsync() {
 ArrayList<String> nameSpaceList = new ArrayList<>();
 ListMetricsRequest request = ListMetricsRequest.builder().build();

 ListMetricsPublisher metricsPaginator =
getAsyncClient().listMetricsPaginator(request);
 CompletableFuture<Void> future = metricsPaginator.subscribe(response -> {
 response.metrics().forEach(metric -> {
 String namespace = metric.namespace();
 if (!nameSpaceList.contains(namespace)) {
 nameSpaceList.add(namespace);
 }
 });
 });
 }

```

```

 });

 return future
 .thenApply(ignored -> namespaceList)
 .exceptionally(exception -> {
 throw new RuntimeException("Failed to list namespaces: " +
exception.getMessage(), exception);
 });
 }
 /**
 * Retrieves the specific metric asynchronously.
 *
 * @param namespace the namespace of the metric to retrieve
 * @return a CompletableFuture that completes with the first dimension of the
first metric found in the specified namespace,
 * or throws a RuntimeException if an error occurs or no metrics or dimensions
are found
 */
 public CompletableFuture<Dimension> getSpecificMetAsync(String namespace) {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .build();

 return getAsyncClient().listMetrics(request).handle((response, exception) ->
{
 if (exception != null) {
 logger.info("Error occurred while listing metrics: {} ",
exception.getMessage());
 throw new RuntimeException("Failed to retrieve specific metric
dimension", exception);
 } else {
 List<Metric> myList = response.metrics();
 if (!myList.isEmpty()) {
 Metric metric = myList.get(0);
 if (!metric.dimensions().isEmpty()) {
 return metric.dimensions().get(0); // Return the first
dimension
 }
 }
 throw new RuntimeException("No metrics or dimensions found");
 }
 });
 }
}

```

```
public static String readFileAsString(String file) throws IOException {
 return new String(Files.readAllBytes(Paths.get(file)));
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteAlarms](#)
  - [DeleteAnomalyDetector](#)
  - [DeleteDashboards](#)
  - [DescribeAlarmHistory](#)
  - [DescribeAlarms](#)
  - [DescribeAlarmsForMetric](#)
  - [DescribeAnomalyDetectors](#)
  - [GetMetricData](#)
  - [GetMetricStatistics](#)
  - [GetMetricWidgetImage](#)
  - [ListMetrics](#)
  - [PutAnomalyDetector](#)
  - [PutDashboard](#)
  - [PutMetricAlarm](#)
  - [PutMetricData](#)

## Actions

### DeleteAlarms

The following code example shows how to use DeleteAlarms.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a CloudWatch alarm.
 *
 * @param alarmName the name of the alarm to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
to delete the alarm
 * the {@link DeleteAlarmsResponse} is returned when the operation completes
successfully,
 * or a {@link RuntimeException} is thrown if the operation fails
 */
public CompletableFuture<DeleteAlarmsResponse> deleteCWAlarmAsync(String
alarmName) {
 DeleteAlarmsRequest request = DeleteAlarmsRequest.builder()
 .alarmNames(alarmName)
 .build();

 return getAsyncClient().deleteAlarms(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the alarm:{} " +
alarmName, exception);
 } else {
 logger.info("Successfully deleted alarm {}", alarmName);
 }
 });
}

```

- For API details, see [DeleteAlarms](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAnomalyDetector

The following code example shows how to use DeleteAnomalyDetector.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an Anomaly Detector.
 *
 * @param fileName the name of the file containing the Anomaly Detector
configuration
 * @return a CompletableFuture that represents the asynchronous deletion of the
Anomaly Detector
 */
public CompletableFuture<DeleteAnomalyDetectorResponse>
deleteAnomalyDetectorAsync(String fileName) {
 CompletableFuture<JsonNode> readFileFuture =
CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 return new ObjectMapper().readTree(parser); // Return the root node
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 });

 return readFileFuture.thenCompose(rootNode -> {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 SingleMetricAnomalyDetector singleMetricAnomalyDetector =
SingleMetricAnomalyDetector.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .stat("Maximum")
 .build();

 DeleteAnomalyDetectorRequest request =
DeleteAnomalyDetectorRequest.builder()
 .singleMetricAnomalyDetector(singleMetricAnomalyDetector)
 .build();

 return getAsyncClient().deleteAnomalyDetector(request);
 }).whenComplete((result, exception) -> {
 if (exception != null) {

```



```

 throw new RuntimeException("Failed to delete the Anomaly Detector",
exception);
 } else {
 logger.info("Successfully deleted the Anomaly Detector.");
 }
});
}

```

- For API details, see [DeleteAnomalyDetector](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDashboards

The following code example shows how to use DeleteDashboards.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes the specified dashboard.
 *
 * @param dashboardName the name of the dashboard to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
of deleting the dashboard
 * @throws RuntimeException if the dashboard deletion fails
 */
public CompletableFuture<DeleteDashboardsResponse> deleteDashboardAsync(String
dashboardName) {
 DeleteDashboardsRequest dashboardsRequest =
DeleteDashboardsRequest.builder()
 .dashboardNames(dashboardName)
 .build();

 return getAsyncClient().deleteDashboards(dashboardsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```

```

 throw new RuntimeException("Failed to delete the dashboard: " +
dashboardName, exception);
 } else {
 logger.info("{} was successfully deleted.", dashboardName);
 }
});
}

```

- For API details, see [DeleteDashboards](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAlarmHistory

The following code example shows how to use DescribeAlarmHistory.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the alarm history for a given alarm name and date range.
 *
 * @param fileName the path to the JSON file containing the alarm name
 * @param date the date to start the alarm history search (in the format
"yyyy-MM-dd'T'HH:mm:ss'Z'")
 * @return a {@code CompletableFuture<Void>} that completes when the alarm
history has been retrieved and processed
 */
public CompletableFuture<Void> getAlarmHistoryAsync(String fileName, String
date) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);

```

```

 return rootNode.findValue("exampleAlarmName").asText(); // Return
alarmName from the JSON file
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
});

// Use the alarm name to describe alarm history with a paginator.
return readFileFuture.thenCompose(alarmName -> {
 try {
 Instant start = Instant.parse(date);
 Instant endDate = Instant.now();
 DescribeAlarmHistoryRequest historyRequest =
DescribeAlarmHistoryRequest.builder()
 .startDate(start)
 .endDate(endDate)
 .alarmName(alarmName)
 .historyItemType(HistoryItemType.ACTION)
 .build();

 // Use the paginator to paginate through alarm history pages.
 DescribeAlarmHistoryPublisher historyPublisher =
getAsyncClient().describeAlarmHistoryPaginator(historyRequest);
 CompletableFuture<Void> future = historyPublisher
 .subscribe(response -> response.alarmHistoryItems().forEach(item
-> {
 logger.info("History summary: {}", item.historySummary());
 logger.info("Timestamp: {}", item.timestamp());
 }))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 logger.error("Error occurred while getting alarm
history: " + exception.getMessage(), exception);
 } else {
 logger.info("Successfully retrieved all alarm
history.");
 }
 });

 // Return the future to the calling code for further handling
 return future;
 } catch (Exception e) {
 throw new RuntimeException("Failed to process alarm history", e);
 }
});

```

```

 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error completing alarm history
processing", exception);
 }
 });
}

```

- For API details, see [DescribeAlarmHistory](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAlarms

The following code example shows how to use DescribeAlarms.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Describes the CloudWatch alarms of the 'METRIC_ALARM' type.
 *
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of describing the CloudWatch alarms. The future completes when the
 * operation is finished, either successfully or with an error.
 */
public CompletableFuture<Void> describeAlarmsAsync() {
 List<AlarmType> typeList = new ArrayList<>();
 typeList.add(AlarmType.METRIC_ALARM);
 DescribeAlarmsRequest alarmsRequest = DescribeAlarmsRequest.builder()
 .alarmTypes(typeList)
 .maxRecords(10)
 .build();

 return getAsyncClient().describeAlarms(alarmsRequest)

```

```

 .thenAccept(response -> {
 List<MetricAlarm> alarmList = response.metricAlarms();
 for (MetricAlarm alarm : alarmList) {
 logger.info("Alarm name: {}", alarm.alarmName());
 logger.info("Alarm description: {} ", alarm.alarmDescription());
 }
 })
 .whenComplete((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to describe alarms: {}", ex.getMessage());
 } else {
 logger.info("Successfully described alarms.");
 }
 });
 }
}

```

- For API details, see [DescribeAlarms](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAlarmsForMetric

The following code example shows how to use `DescribeAlarmsForMetric`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Checks for a metric alarm in AWS CloudWatch.
 *
 * @param fileName the name of the file containing the JSON configuration for
the custom metric
 * @return a {@link CompletableFuture} that completes when the check for the
metric alarm is complete
 */
public CompletableFuture<Void> checkForMetricAlarmAsync(String fileName) {

```

```

 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {
 throw new RuntimeException("Failed to read file", e);
 }
 });

 return readFileFuture.thenCompose(jsonContent -> {
 try {
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 DescribeAlarmsForMetricRequest metricRequest =
DescribeAlarmsForMetricRequest.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 return checkForAlarmAsync(metricRequest, customMetricName, 10);

 } catch (IOException e) {
 throw new RuntimeException("Failed to parse JSON content", e);
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error checking metric alarm",
exception);
 }
 });
 }

 // Recursive method to check for the alarm.

```

```

/**
 * Checks for the existence of an alarm asynchronously for the specified metric.
 *
 * @param metricRequest the request to describe the alarms for the specified
metric
 * @param customMetricName the name of the custom metric to check for an alarm
 * @param retries the number of retries to perform if no alarm is found
 * @return a {@link CompletableFuture} that completes when an alarm is found or
the maximum number of retries has been reached
 */
private static CompletableFuture<Void>
checkForAlarmAsync(DescribeAlarmsForMetricRequest metricRequest, String
customMetricName, int retries) {
 if (retries == 0) {
 return CompletableFuture.completedFuture(null).thenRun(() ->
 logger.info("No Alarm state found for {} after 10 retries.",
customMetricName)
);
 }

 return
(getAsyncClient().describeAlarmsForMetric(metricRequest).thenCompose(response -> {
 if (response.hasMetricAlarms()) {
 logger.info("Alarm state found for {}", customMetricName);
 return CompletableFuture.completedFuture(null); // Alarm found,
complete the future
 } else {
 return CompletableFuture.runAsync(() -> {
 try {
 Thread.sleep(20000);
 logger.info(".");
 } catch (InterruptedException e) {
 throw new RuntimeException("Interrupted while waiting to
retry", e);
 }
 }).thenCompose(v -> checkForAlarmAsync(metricRequest,
customMetricName, retries - 1)); // Recursive call
 }
 }));
}

```

- For API details, see [DescribeAlarmsForMetric](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAnomalyDetectors

The following code example shows how to use DescribeAnomalyDetectors.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Describes the anomaly detectors based on the specified JSON file.
 *
 * @param fileName the name of the JSON file containing the custom metric
namespace and name
 * @return a {@link CompletableFuture} that completes when the anomaly detectors
have been described
 * @throws RuntimeException if there is a failure during the operation, such as
when reading or parsing the JSON file,
 *
 * or when describing the anomaly detectors
 */
public CompletableFuture<Void> describeAnomalyDetectorsAsync(String fileName) {
 CompletableFuture<JsonNode> readFileFuture =
CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 return new ObjectMapper().readTree(parser);
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 });

 return readFileFuture.thenCompose(rootNode -> {
 try {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();
```



```

 DescribeAnomalyDetectorsRequest detectorsRequest =
DescribeAnomalyDetectorsRequest.builder()
 .maxResults(10)
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 return
getAsyncClient().describeAnomalyDetectors(detectorsRequest).thenAccept(response ->
{
 List<AnomalyDetector> anomalyDetectorList =
response.anomalyDetectors();
 for (AnomalyDetector detector : anomalyDetectorList) {
 logger.info("Metric name: {} ",
detector.singleMetricAnomalyDetector().metricName());
 logger.info("State: {} ", detector.stateValue());
 }
 });
 } catch (RuntimeException e) {
 throw new RuntimeException("Failed to describe anomaly detectors",
e);
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error describing anomaly detectors",
exception);
 }
 });
}

```

- For API details, see [DescribeAnomalyDetectors](#) in *AWS SDK for Java 2.x API Reference*.

## DisableAlarmActions

The following code example shows how to use `DisableAlarmActions`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.DisableAlarmActionsRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DisableAlarmActions {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <alarmName>

 Where:
 alarmName - An alarm name to disable (for example, MyAlarm).
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String alarmName = args[0];
 Region region = Region.US_EAST_1;
 CloudWatchClient cw = CloudWatchClient.builder()
 .region(region)
 .build();
```

```
 disableActions(cw, alarmName);
 cw.close();
 }

 public static void disableActions(CloudWatchClient cw, String alarmName) {
 try {
 DisableAlarmActionsRequest request =
 DisableAlarmActionsRequest.builder()
 .alarmNames(alarmName)
 .build();

 cw.disableAlarmActions(request);
 System.out.printf("Successfully disabled actions on alarm %s",
 alarmName);

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DisableAlarmActions](#) in *AWS SDK for Java 2.x API Reference*.

## EnableAlarmActions

The following code example shows how to use `EnableAlarmActions`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.CloudWatchClient;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatch.model.EnableAlarmActionsRequest;
```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class EnableAlarmActions {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <alarmName>

 Where:
 alarmName - An alarm name to enable (for example, MyAlarm).
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String alarm = args[0];
 Region region = Region.US_EAST_1;
 CloudWatchClient cw = CloudWatchClient.builder()
 .region(region)
 .build();

 enableActions(cw, alarm);
 cw.close();
 }

 public static void enableActions(CloudWatchClient cw, String alarm) {
 try {
 EnableAlarmActionsRequest request = EnableAlarmActionsRequest.builder()
 .alarmNames(alarm)
 .build();

 cw.enableAlarmActions(request);
 System.out.printf("Successfully enabled actions on alarm %s", alarm);
 }
 }
}

```

```

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [EnableAlarmActions](#) in *AWS SDK for Java 2.x API Reference*.

## GetMetricData

The following code example shows how to use `GetMetricData`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves custom metric data from the AWS CloudWatch service.
 *
 * @param fileName the name of the file containing the custom metric information
 * @return a {@link CompletableFuture} that completes when the metric data has
 * been retrieved
 */
public CompletableFuture<Void> getCustomMetricDataAsync(String fileName) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 // Read values from the JSON file.
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {

```

```

 throw new RuntimeException("Failed to read file", e);
 }
});

return readFileFuture.thenCompose(jsonContent -> {
 try {
 // Parse the JSON string to extract relevant values.
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 // Set the current time and date range for metric query.
 Instant nowDate = Instant.now();
 long hours = 1;
 long minutes = 30;
 Instant endTime = nowDate.plus(hours,
ChronoUnit.HOURS).plus(minutes, ChronoUnit.MINUTES);

 Metric met = Metric.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .build();

 MetricStat metStat = MetricStat.builder()
 .stat("Maximum")
 .period(60) // Assuming period in seconds
 .metric(met)
 .build();

 MetricDataQuery dataQuery = MetricDataQuery.builder()
 .metricStat(metStat)
 .id("foo2")
 .returnData(true)
 .build();

 List<MetricDataQuery> dq = new ArrayList<>();
 dq.add(dataQuery);

 GetMetricDataRequest getMetricDataRequest =
GetMetricDataRequest.builder()
 .maxDatapoints(10)

```

```

 .scanBy(ScanBy.TIMESTAMP_DESCENDING)
 .startTime(nowDate)
 .endTime(endTime)
 .metricDataQueries(dq)
 .build();

 // Call the async method for CloudWatch data retrieval.
 return getAsyncClient().getMetricData(getMetricDataRequest);

} catch (IOException e) {
 throw new RuntimeException("Failed to parse JSON content", e);
}
}).thenAccept(response -> {
 List<MetricDataResult> data = response.metricDataResults();
 for (MetricDataResult item : data) {
 logger.info("The label is: {}", item.label());
 logger.info("The status code is: {}", item.statusCode().toString());
 }
}).exceptionally(exception -> {
 throw new RuntimeException("Failed to get metric data", exception);
});
}

```

- For API details, see [GetMetricData](#) in *AWS SDK for Java 2.x API Reference*.

## GetMetricStatistics

The following code example shows how to use `GetMetricStatistics`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves and displays metric statistics for the specified parameters.
 *
 * @param nameSpace the namespace for the metric

```

```

 * @param metVal the name of the metric
 * @param metricOption the statistic to retrieve for the metric (e.g.,
 "Maximum", "Average")
 * @param date the date for which to retrieve the metric statistics, in
 the format "yyyy-MM-dd'T'HH:mm:ss'Z'"
 * @param myDimension the dimension(s) to filter the metric statistics by
 * @return a {@link CompletableFuture} that completes when the metric statistics
 have been retrieved and displayed
 */
 public CompletableFuture<GetMetricStatisticsResponse>
 getAndDisplayMetricStatisticsAsync(String nameSpace, String metVal,

 String metricOption, String date, Dimension myDimension) {

 Instant start = Instant.parse(date);
 Instant endDate = Instant.now();

 // Building the request for metric statistics.
 GetMetricStatisticsRequest statisticsRequest =
 GetMetricStatisticsRequest.builder()
 .endTime(endDate)
 .startTime(start)
 .dimensions(myDimension)
 .metricName(metVal)
 .namespace(nameSpace)
 .period(86400) // 1 day period
 .statistics(Statistic.fromValue(metricOption))
 .build();

 return getAsyncClient().getMetricStatistics(statisticsRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 List<Datapoint> data = response.datapoints();
 if (!data.isEmpty()) {
 for (Datapoint datapoint : data) {
 logger.info("Timestamp: {} Maximum value: {}",
 datapoint.timestamp(), datapoint.maximum());
 }
 } else {
 logger.info("The returned data list is empty");
 }
 } else {
 logger.info("Failed to get metric statistics: {} ",
 exception.getMessage());
 }
 });
 }

```



```

 }
 })
 .exceptionally(exception -> {
 throw new RuntimeException("Error while getting metric statistics: "
+ exception.getMessage(), exception);
 });
}

```

- For API details, see [GetMetricStatistics](#) in *AWS SDK for Java 2.x API Reference*.

## GetMetricWidgetImage

The following code example shows how to use `GetMetricWidgetImage`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves and saves a custom metric image to a file.
 *
 * @param fileName the name of the file to save the metric image to
 * @return a {@link CompletableFuture} that completes when the image has been
saved to the file
 */
public CompletableFuture<Void> downloadAndSaveMetricImageAsync(String fileName)
{
 logger.info("Getting Image data for custom metric.");
 String myJSON = ""
 {
 "title": "Example Metric Graph",
 "view": "timeSeries",
 "stacked ": false,
 "period": 10,
 "width": 1400,

```

```

 "height": 600,
 "metrics": [
 [
 "AWS/Billing",
 "EstimatedCharges",
 "Currency",
 "USD"
]
]
 }
 """;

 GetMetricWidgetImageRequest imageRequest =
 GetMetricWidgetImageRequest.builder()
 .metricWidget(myJSON)
 .build();

 return getAsyncClient().getMetricWidgetImage(imageRequest)
 .thenCompose(response -> {
 SdkBytes sdkBytes = response.metricWidgetImage();
 byte[] bytes = sdkBytes.asByteArray();
 return CompletableFuture.runAsync(() -> {
 try {
 File outputFile = new File(fileName);
 try (FileOutputStream outputStream = new
FileOutputStream(outputFile)) {
 outputStream.write(bytes);
 }
 } catch (IOException e) {
 throw new RuntimeException("Failed to write image to file",
e);
 }
 });
 })
 .whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error getting and saving metric
image", exception);
 } else {
 logger.info("Image data saved successfully to {}", fileName);
 }
 });
 }
}

```

- For API details, see [GetMetricWidgetImage](#) in *AWS SDK for Java 2.x API Reference*.

## ListDashboards

The following code example shows how to use ListDashboards.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Lists the available dashboards.
 *
 * @return a {@link CompletableFuture} that completes when the operation is
 * finished.
 * The future will complete exceptionally if an error occurs while listing the
 * dashboards.
 */
public CompletableFuture<Void> listDashboardsAsync() {
 ListDashboardsRequest listDashboardsRequest =
ListDashboardsRequest.builder().build();
 ListDashboardsPublisher paginator =
getAsyncClient().listDashboardsPaginator(listDashboardsRequest);
 return paginator.subscribe(response -> {
 response.dashboardEntries().forEach(entry -> {
 logger.info("Dashboard name is: {}", entry.dashboardName());
 logger.info("Dashboard ARN is: {}", entry.dashboardArn());
 });
 }).exceptionally(ex -> {
 logger.info("Failed to list dashboards: {}", ex.getMessage());
 throw new RuntimeException("Error occurred while listing dashboards",
ex);
 });
}
```

- For API details, see [ListDashboards](#) in *AWS SDK for Java 2.x API Reference*.

## ListMetrics

The following code example shows how to use `ListMetrics`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves a list of metric names for the specified namespace.
 *
 * @param namespace the namespace for which to retrieve the metric names
 * @return a {@link CompletableFuture} that, when completed, contains an {@link
 * ArrayList} of
 * * the metric names in the specified namespace
 * @throws RuntimeException if an error occurs while listing the metrics
 */
public CompletableFuture<ArrayList<String>> listMetsAsync(String namespace) {
 ListMetricsRequest request = ListMetricsRequest.builder()
 .namespace(namespace)
 .build();

 ListMetricsPublisher metricsPaginator =
getAsyncClient().listMetricsPaginator(request);
 Set<String> metSet = new HashSet<>();
 CompletableFuture<Void> future = metricsPaginator.subscribe(response -> {
 response.metrics().forEach(metric -> {
 String metricName = metric.metricName();
 metSet.add(metricName);
 });
 });

 return future
 .thenApply(ignored -> new ArrayList<>(metSet))
 .exceptionally(exception -> {
```

```

 throw new RuntimeException("Failed to list metrics: " +
exception.getMessage(), exception);
 });
}

```

- For API details, see [ListMetrics](#) in *AWS SDK for Java 2.x API Reference*.

## PutAnomalyDetector

The following code example shows how to use PutAnomalyDetector.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Adds an anomaly detector for the given file.
 *
 * @param fileName the name of the file containing the anomaly detector
configuration
 * @return a {@link CompletableFuture} that completes when the anomaly detector
has been added
 */
public CompletableFuture<Void> addAnomalyDetectorAsync(String fileName) {
 CompletableFuture<JsonNode> readFileFuture =
CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 return new ObjectMapper().readTree(parser); // Return the root node
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse the file", e);
 }
 });
}

```

```

 return readFileFuture.thenCompose(rootNode -> {
 try {
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();

 SingleMetricAnomalyDetector singleMetricAnomalyDetector =
SingleMetricAnomalyDetector.builder()
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .stat("Maximum")
 .build();

 PutAnomalyDetectorRequest anomalyDetectorRequest =
PutAnomalyDetectorRequest.builder()
 .singleMetricAnomalyDetector(singleMetricAnomalyDetector)
 .build();

 return
getAsyncClient().putAnomalyDetector(anomalyDetectorRequest).thenAccept(response ->
{
 logger.info("Added anomaly detector for metric {}",
customMetricName);
 });
 } catch (Exception e) {
 throw new RuntimeException("Failed to create anomaly detector", e);
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error adding anomaly detector",
exception);
 }
 });
 }
}

```

- For API details, see [PutAnomalyDetector](#) in *AWS SDK for Java 2.x API Reference*.

## PutDashboard

The following code example shows how to use PutDashboard.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new dashboard with the specified name and metrics from the given
 * file.
 *
 * @param dashboardName the name of the dashboard to be created
 * @param fileName the name of the file containing the dashboard body
 * @return a {@link CompletableFuture} representing the asynchronous operation
 * of creating the dashboard
 * @throws IOException if there is an error reading the dashboard body from the
 * file
 */
public CompletableFuture<PutDashboardResponse>
createDashboardWithMetricsAsync(String dashboardName, String fileName) throws
IOException {
 String dashboardBody = readFileAsString(fileName);
 PutDashboardRequest dashboardRequest = PutDashboardRequest.builder()
 .dashboardName(dashboardName)
 .dashboardBody(dashboardBody)
 .build();

 return getAsyncClient().putDashboard(dashboardRequest)
 .handle((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to create dashboard: {}", ex.getMessage());
 throw new RuntimeException("Dashboard creation failed", ex);
 } else {
 // Handle the normal response case
 logger.info("{} was successfully created.", dashboardName);
 List<DashboardValidationMessage> messages =
response.dashboardValidationMessages();
 if (messages.isEmpty()) {
 logger.info("There are no messages in the new Dashboard.");
 } else {
```

```

 for (DashboardValidationMessage message : messages) {
 logger.info("Message: {}", message.message());
 }
 }
 return response; // Return the response for further use
}
});
}

```

- For API details, see [PutDashboard](#) in *AWS SDK for Java 2.x API Reference*.

## PutMetricAlarm

The following code example shows how to use PutMetricAlarm.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an alarm based on the configuration provided in a JSON file.
 *
 * @param fileName the name of the JSON file containing the alarm configuration
 * @return a CompletableFuture that represents the asynchronous operation of
 * creating the alarm
 * @throws RuntimeException if an exception occurs while reading the JSON file
 * or creating the alarm
 */
public CompletableFuture<String> createAlarmAsync(String fileName) {
 com.fasterxml.jackson.databind.JsonNode rootNode;
 try {
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 rootNode = new ObjectMapper().readTree(parser);
 } catch (IOException e) {
 throw new RuntimeException("Failed to read the alarm configuration
file", e);
 }
}

```



```

 // Extract values from the JSON node.
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName = rootNode.findValue("customMetricName").asText();
 String alarmName = rootNode.findValue("exampleAlarmName").asText();
 String emailTopic = rootNode.findValue("emailTopic").asText();
 String accountId = rootNode.findValue("accountId").asText();
 String region = rootNode.findValue("region").asText();

 // Create a List for alarm actions.
 List<String> alarmActions = new ArrayList<>();
 alarmActions.add("arn:aws:sns:" + region + ":" + accountId + ":" +
emailTopic);

 PutMetricAlarmRequest alarmRequest = PutMetricAlarmRequest.builder()
 .alarmActions(alarmActions)
 .alarmDescription("Example metric alarm")
 .alarmName(alarmName)

.comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
 .threshold(100.00)
 .metricName(customMetricName)
 .namespace(customMetricNamespace)
 .evaluationPeriods(1)
 .period(10)
 .statistic("Maximum")
 .datapointsToAlarm(1)
 .treatMissingData("ignore")
 .build();

 // Call the putMetricAlarm asynchronously and handle the result.
 return getAsyncClient().putMetricAlarm(alarmRequest)
 .handle((response, ex) -> {
 if (ex != null) {
 logger.info("Failed to create alarm: {}", ex.getMessage());
 throw new RuntimeException("Failed to create alarm", ex);
 } else {
 logger.info("{} was successfully created!", alarmName);
 return alarmName;
 }
 });
 }
}

```

- For API details, see [PutMetricAlarm](#) in *AWS SDK for Java 2.x API Reference*.

## PutMetricData

The following code example shows how to use PutMetricData.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Adds metric data for an alarm asynchronously.
 *
 * @param fileName the name of the JSON file containing the metric data
 * @return a CompletableFuture that asynchronously returns the
 * PutMetricDataResponse
 */
public CompletableFuture<PutMetricDataResponse>
addMetricDataForAlarmAsync(String fileName) {
 CompletableFuture<String> readFileFuture = CompletableFuture.supplyAsync(()
-> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 return rootNode.toString(); // Return JSON as a string for further
processing
 } catch (IOException e) {
 throw new RuntimeException("Failed to read file", e);
 }
 });

 return readFileFuture.thenCompose(jsonContent -> {
 try {
```

```

 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(jsonContent);
 String customMetricNamespace =
rootNode.findValue("customMetricNamespace").asText();
 String customMetricName =
rootNode.findValue("customMetricName").asText();
 Instant instant = Instant.now();

 // Create MetricDatum objects.
 MetricDatum datum1 = MetricDatum.builder()
 .metricName(customMetricName)
 .unit(StandardUnit.NONE)
 .value(1001.00)
 .timestamp(instant)
 .build();

 MetricDatum datum2 = MetricDatum.builder()
 .metricName(customMetricName)
 .unit(StandardUnit.NONE)
 .value(1002.00)
 .timestamp(instant)
 .build();

 List<MetricDatum> metricDataList = new ArrayList<>();
 metricDataList.add(datum1);
 metricDataList.add(datum2);

 // Build the PutMetricData request.
 PutMetricDataRequest request = PutMetricDataRequest.builder()
 .namespace(customMetricNamespace)
 .metricData(metricDataList)
 .build();

 // Send the request asynchronously.
 return getAsyncClient().putMetricData(request);

 } catch (IOException e) {
 CompletableFuture<PutMetricDataResponse> failedFuture = new
CompletableFuture<>();
 failedFuture.completeExceptionally(new RuntimeException("Failed to
parse JSON content", e));
 return failedFuture;
 }
}).whenComplete((response, exception) -> {

```

```
 if (exception != null) {
 logger.error("Failed to put metric data: " + exception.getMessage(),
exception);
 } else {
 logger.info("Added metric values for metric.");
 }
 });
}
```

- For API details, see [PutMetricData](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Monitor DynamoDB performance

The following code example shows how to configure an application's use of DynamoDB to monitor performance.

#### SDK for Java 2.x

This example shows how to configure a Java application to monitor the performance of DynamoDB. The application sends metric data to CloudWatch where you can monitor the performance.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- CloudWatch
- DynamoDB

## CloudWatch Events examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with CloudWatch Events.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### PutEvents

The following code example shows how to use PutEvents.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutEventsRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.PutEventsRequestEntry;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PutEvents {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <resourceArn>

 Where:
```

```

 resourceArn - An Amazon Resource Name (ARN) related to the
events.

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String resourceArn = args[0];
 CloudWatchEventsClient cwe = CloudWatchEventsClient.builder()
 .build();

 putCWEvents(cwe, resourceArn);
 cwe.close();
}

public static void putCWEvents(CloudWatchEventsClient cwe, String resourceArn) {
 try {
 final String EVENT_DETAILS = "{ \"key1\": \"value1\", \"key2\": \"value2\" }";

 PutEventsRequestEntry requestEntry = PutEventsRequestEntry.builder()
 .detail(EVENT_DETAILS)
 .detailType("sampleSubmitted")
 .resources(resourceArn)
 .source("aws-sdk-java-cloudwatch-example")
 .build();

 PutEventsRequest request = PutEventsRequest.builder()
 .entries(requestEntry)
 .build();

 cwe.putEvents(request);
 System.out.println("Successfully put CloudWatch event");

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [PutEvents](#) in *AWS SDK for Java 2.x API Reference*.

## PutRule

The following code example shows how to use PutRule.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutRuleRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.PutRuleResponse;
import software.amazon.awssdk.services.cloudwatchevents.model.RuleState;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PutRule {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <ruleName> roleArn\s

 Where:
 ruleName - A rule name (for example, myrule).
 roleArn - A role ARN value (for example,
arn:aws:iam::xxxxxx047983:user/MyUser).
 """;

 if (args.length != 2) {
```

```

 System.out.println(usage);
 System.exit(1);
 }

 String ruleName = args[0];
 String roleArn = args[1];
 CloudWatchEventsClient cwe = CloudWatchEventsClient.builder()
 .build();

 putCWRule(cwe, ruleName, roleArn);
 cwe.close();
}

public static void putCWRule(CloudWatchEventsClient cwe, String ruleName, String
roleArn) {
 try {
 PutRuleRequest request = PutRuleRequest.builder()
 .name(ruleName)
 .roleArn(roleArn)
 .scheduleExpression("rate(5 minutes)")
 .state(RuleState.ENABLED)
 .build();

 PutRuleResponse response = cwe.putRule(request);
 System.out.printf(
 "Successfully created CloudWatch events rule %s with arn %s",
 roleArn, response.ruleArn());

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [PutRule](#) in *AWS SDK for Java 2.x API Reference*.

## PutTargets

The following code example shows how to use PutTargets.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient;
import software.amazon.awssdk.services.cloudwatchevents.model.PutTargetsRequest;
import software.amazon.awssdk.services.cloudwatchevents.model.Target;

/**
 * To run this Java V2 code example, ensure that you have setup your development
 * environment, including your credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PutTargets {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <ruleName> <functionArn> <targetId>\s

 Where:
 ruleName - A rule name (for example, myrule).
 functionArn - An AWS Lambda function ARN (for example,
arn:aws:lambda:us-west-2:xxxxxx047983:function:lamda1).
 targetId - A target id value.
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String ruleName = args[0];
 String functionArn = args[1];
```

```

 String targetId = args[2];
 CloudWatchEventsClient cwe = CloudWatchEventsClient.builder()
 .build();

 putCWTargets(cwe, ruleName, functionArn, targetId);
 cwe.close();
 }

 public static void putCWTargets(CloudWatchEventsClient cwe, String ruleName,
 String functionArn, String targetId) {
 try {
 Target target = Target.builder()
 .arn(functionArn)
 .id(targetId)
 .build();

 PutTargetsRequest request = PutTargetsRequest.builder()
 .targets(target)
 .rule(ruleName)
 .build();

 cwe.putTargets(request);
 System.out.printf(
 "Successfully created CloudWatch events target for rule %s",
 ruleName);

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [PutTargets](#) in *AWS SDK for Java 2.x API Reference*.

## CloudWatch Logs examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with CloudWatch Logs.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### DeleteSubscriptionFilter

The following code example shows how to use DeleteSubscriptionFilter.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchlogs.CloudWatchLogsClient;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.DeleteSubscriptionFilterRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
```

```

public class DeleteSubscriptionFilter {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <filter> <logGroup>

 Where:
 filter - The name of the subscription filter (for example,
MyFilter).
 logGroup - The name of the log group. (for example, testgroup).
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String filter = args[0];
 String logGroup = args[1];
 CloudWatchLogsClient logs = CloudWatchLogsClient.builder()
 .build();

 deleteSubFilter(logs, filter, logGroup);
 logs.close();
 }

 public static void deleteSubFilter(CloudWatchLogsClient logs, String filter,
String logGroup) {
 try {
 DeleteSubscriptionFilterRequest request =
DeleteSubscriptionFilterRequest.builder()
 .filterName(filter)
 .logGroupName(logGroup)
 .build();

 logs.deleteSubscriptionFilter(request);
 System.out.printf("Successfully deleted CloudWatch logs subscription
filter %s", filter);

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

```
}
}
```

- For API details, see [DeleteSubscriptionFilter](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeLogStreams

The following code example shows how to use DescribeLogStreams.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Searches for log streams within a specified log group that match a given prefix.

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CloudWatchLogsSearch {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <logGroupName> <logStreamName>

 Where:
 logGroupName - The name of the log group (for example,
 WeathertopJavaContainerLogs).
 logStreamName - The name of the log stream (for example,
 weathertop-java-stream).
```

```

 pattern - the pattern to use (for example, INFO)

 """;

 if (args.length != 3) {
 System.out.print(usage);
 System.exit(1);
 }

 String logGroupName = args[0] ;
 String logStreamName = args[1] ;
 String pattern = args[2] ;

 CloudWatchLogsClient cwlClient = CloudWatchLogsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 searchLogStreamsAndFilterEvents(cwlClient, logGroupName, logStreamName,
pattern);
 }

 /**
 * Searches for log streams with a specific prefix within a log group and
 filters log events based on a specified pattern.
 *
 * @param cwlClient the CloudWatchLogsClient used to interact with AWS
CloudWatch Logs
 * @param logGroupName the name of the log group to search within
 * @param logStreamPrefix the prefix of the log streams to search for
 * @param pattern the pattern to filter log events by
 */
 public static void searchLogStreamsAndFilterEvents(CloudWatchLogsClient
cwlClient, String logGroupName, String logStreamPrefix, String pattern) {
 DescribeLogStreamsRequest describeLogStreamsRequest =
DescribeLogStreamsRequest.builder()
 .logGroupName(logGroupName)
 .logStreamNamePrefix(logStreamPrefix)
 .build();

 DescribeLogStreamsResponse describeLogStreamsResponse =
cwlClient.describeLogStreams(describeLogStreamsRequest);
 List<LogStream> logStreams = describeLogStreamsResponse.logStreams();

 for (LogStream logStream : logStreams) {

```

```

 String logStreamName = logStream.getLogStreamName();
 System.out.println("Searching in log stream: " + logStreamName);

 FilterLogEventsRequest filterLogEventsRequest =
 FilterLogEventsRequest.builder()
 .logGroupName(logGroupName)
 .logStreamNames(logStreamName)
 .filterPattern(pattern)
 .build();

 FilterLogEventsResponse filterLogEventsResponse =
 cwlClient.filterLogEvents(filterLogEventsRequest);

 for (FilteredLogEvent event : filterLogEventsResponse.events()) {
 System.out.println(event.message());
 }

 System.out.println("-----"); //
 Separator for better readability
 }
}
}

```

Prints metadata about the most recent log stream in a specified log group.

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CloudWatchLogQuery {
 public static void main(final String[] args) {
 final String usage = ""
 Usage:
 <logGroupName>

 Where:
 }
}

```

```

 logGroupName - The name of the log group (for example, /aws/
 lambda/ChatAIHandler).
 """;

 if (args.length != 1) {
 System.out.print(usage);
 System.exit(1);
 }

 String logGroupName = "/aws/lambda/ChatAIHandler" ; //args[0];
 CloudWatchLogsClient logsClient = CloudWatchLogsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 describeMostRecentLogStream(logsClient, logGroupName);
}

/**
 * Describes and prints metadata about the most recent log stream in the
 * specified log group.
 *
 * @param logsClient the CloudWatchLogsClient used to interact with AWS
 * CloudWatch Logs
 * @param logGroupName the name of the log group
 */
public static void describeMostRecentLogStream(CloudWatchLogsClient logsClient,
String logGroupName) {
 DescribeLogStreamsRequest streamsRequest =
DescribeLogStreamsRequest.builder()
 .logGroupName(logGroupName)
 .orderBy(OrderBy.LAST_EVENT_TIME)
 .descending(true)
 .limit(1)
 .build();

 try {
 DescribeLogStreamsResponse streamsResponse =
logsClient.describeLogStreams(streamsRequest);
 List<LogStream> logStreams = streamsResponse.logStreams();

 if (logStreams.isEmpty()) {
 System.out.println("No log streams found for log group: " +
logGroupName);
 return;

```



```

 }

 LogStream stream = logStreams.get(0);
 System.out.println("Most Recent Log Stream:");
 System.out.println(" Name: " + stream.logStreamName());
 System.out.println(" ARN: " + stream.arn());
 System.out.println(" Creation Time: " + stream.creationTime());
 System.out.println(" First Event Time: " +
stream.firstEventTimestamp());
 System.out.println(" Last Event Time: " + stream.lastEventTimestamp());
 System.out.println(" Stored Bytes: " + stream.storedBytes());
 System.out.println(" Upload Sequence Token: " +
stream.uploadSequenceToken());

 } catch (CloudWatchLogsException e) {
 System.err.println("Failed to describe log stream: " +
e.awsErrorDetails().errorMessage());
 }
}
}

```

- For API details, see [DescribeLogStreams](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeSubscriptionFilters

The following code example shows how to use `DescribeSubscriptionFilters`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchlogs.CloudWatchLogsClient;
import
software.amazon.awssdk.services.cloudwatchlogs.model.DescribeSubscriptionFiltersRequest;

```

```
import
 software.amazon.awssdk.services.cloudwatchlogs.model.DescribeSubscriptionFiltersResponse;
import software.amazon.awssdk.services.cloudwatchlogs.model.SubscriptionFilter;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeSubscriptionFilters {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <logGroup>

 Where:
 logGroup - A log group name (for example, myloggroup).
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String logGroup = args[0];
 CloudWatchLogsClient logs = CloudWatchLogsClient.builder()
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 describeFilters(logs, logGroup);
 logs.close();
 }

 public static void describeFilters(CloudWatchLogsClient logs, String logGroup) {
 try {
 boolean done = false;
 String newToken = null;

 while (!done) {
```

```

 DescribeSubscriptionFiltersResponse response;
 if (newToken == null) {
 DescribeSubscriptionFiltersRequest request =
DescribeSubscriptionFiltersRequest.builder()
 .logGroupName(logGroup)
 .limit(1).build();

 response = logs.describeSubscriptionFilters(request);
 } else {
 DescribeSubscriptionFiltersRequest request =
DescribeSubscriptionFiltersRequest.builder()
 .nextToken(newToken)
 .logGroupName(logGroup)
 .limit(1).build();
 response = logs.describeSubscriptionFilters(request);
 }

 for (SubscriptionFilter filter : response.subscriptionFilters()) {
 System.out.printf("Retrieved filter with name %s, " + "pattern
%s " + "and destination arn %s",
 filter.filterName(),
 filter.filterPattern(),
 filter.destinationArn());
 }

 if (response.nextToken() == null) {
 done = true;
 } else {
 newToken = response.nextToken();
 }
 }

} catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
System.out.printf("Done");
}
}

```

- For API details, see [DescribeSubscriptionFilters](#) in *AWS SDK for Java 2.x API Reference*.

## GetLogEvents

The following code example shows how to use GetLogEvents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatch.model.CloudWatchException;
import software.amazon.awssdk.services.cloudwatchlogs.CloudWatchLogsClient;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.DescribeLogStreamsRequest;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.DescribeLogStreamsResponse;
import software.amazon.awssdk.services.cloudwatchlogs.model.GetLogEventsRequest;
import software.amazon.awssdk.services.cloudwatchlogs.model.GetLogEventsResponse;

import java.time.Instant;
import java.time.temporal.ChronoUnit;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetLogEvents {

 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <logGroupName> <logStreamName>

 Where:
```

```

 logGroupName - The name of the log group (for example,
myloggroup).
 logStreamName - The name of the log stream (for example,
mystream).

 "";

// if (args.length != 2) {
// System.out.print(usage);
// System.exit(1);
// }

String logGroupName = "WeathertopJavaContainerLogs" ; //args[0];
String logStreamName = "weathertop-java-stream" ; //args[1];

Region region = Region.US_EAST_1 ;
CloudWatchLogsClient cloudWatchLogsClient = CloudWatchLogsClient.builder()
 .region(region)
 .build();

getCWLogEvents(cloudWatchLogsClient, logGroupName, logStreamName);
cloudWatchLogsClient.close();
}

public static void getCWLogEvents(CloudWatchLogsClient cloudWatchLogsClient,
 String logGroupName,
 String logStreamPrefix) {

 try {
 // First, find the exact log stream name
 DescribeLogStreamsRequest describeRequest =
DescribeLogStreamsRequest.builder()
 .logGroupName(logGroupName)
 .logStreamNamePrefix(logStreamPrefix)
 .limit(1) // get the first matching stream
 .build();

 DescribeLogStreamsResponse describeResponse =
cloudWatchLogsClient.describeLogStreams(describeRequest);

 if (describeResponse.getLogStreams().isEmpty()) {
 System.out.println("No matching log streams found for prefix: " +
logStreamPrefix);
 return;
 }
 }

```

```

 String exactLogStreamName =
describeResponse.logStreams().get(0).logStreamName();
 System.out.println("Using exact log stream: " + exactLogStreamName);

 long startTime = Instant.now().minus(7, ChronoUnit.DAYS).toEpochMilli();
 long endTime = Instant.now().toEpochMilli();

 GetLogEventsRequest getLogEventsRequest = GetLogEventsRequest.builder()
 .logGroupName(logGroupName)
 .logStreamName(exactLogStreamName) // <-- exact name, not prefix
 .startTime(startTime)
 .endTime(endTime)
 .startFromHead(true)
 .build();

 GetLogEventsResponse response =
cloudWatchLogsClient.getLogEvents(getLogEventsRequest);

 if (response.events().isEmpty()) {
 System.out.println("No log events found in the past 7 days.");
 } else {
 response.events().forEach(e -> System.out.println(e.message()));
 }

 } catch (CloudWatchException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [GetLogEvents](#) in *AWS SDK for Java 2.x API Reference*.

## PutSubscriptionFilter

The following code example shows how to use PutSubscriptionFilter.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cloudwatchlogs.CloudWatchLogsClient;
import software.amazon.awssdk.services.cloudwatchlogs.model.CloudWatchLogsException;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.PutSubscriptionFilterRequest;

/**
 * Before running this code example, you need to grant permission to CloudWatch
 * Logs the right to execute your Lambda function.
 * To perform this task, you can use this CLI command:
 *
 * aws lambda add-permission --function-name "lamda1" --statement-id "lamda1"
 * --principal "logs.us-west-2.amazonaws.com" --action "lambda:InvokeFunction"
 * --source-arn "arn:aws:logs:us-west-2:111111111111:log-group:testgroup:*"
 * --source-account "111111111111"
 *
 * Make sure you replace the function name with your function name and replace
 * '111111111111' with your account details.
 * For more information, see "Subscription Filters with AWS Lambda" in the
 * Amazon CloudWatch Logs Guide.
 *
 * Also, before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class PutSubscriptionFilter {
 public static void main(String[] args) {
 final String usage = ""
```

## Usage:

```
<filter> <pattern> <logGroup> <functionArn>\s
```

## Where:

filter - A filter name (for example, myfilter).

pattern - A filter pattern (for example, ERROR).

logGroup - A log group name (testgroup).

functionArn - An AWS Lambda function ARN (for example, arn:aws:lambda:us-west-2:111111111111:function:lambda1) .  
 """;

```
if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
}

String filter = args[0];
String pattern = args[1];
String logGroup = args[2];
String functionArn = args[3];
Region region = Region.US_WEST_2;
CloudWatchLogsClient cwl = CloudWatchLogsClient.builder()
 .region(region)
 .build();

putSubFilters(cwl, filter, pattern, logGroup, functionArn);
cwl.close();
}

public static void putSubFilters(CloudWatchLogsClient cwl,
 String filter,
 String pattern,
 String logGroup,
 String functionArn) {

 try {
 PutSubscriptionFilterRequest request =
 PutSubscriptionFilterRequest.builder()
 .filterName(filter)
 .filterPattern(pattern)
 .logGroupName(logGroup)
 .destinationArn(functionArn)
 .build();
```



```

 cwl.putSubscriptionFilter(request);
 System.out.printf(
 "Successfully created CloudWatch logs subscription filter %s",
 filter);

 } catch (CloudWatchLogsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [PutSubscriptionFilter](#) in *AWS SDK for Java 2.x API Reference*.

## StartLiveTail

The following code example shows how to use StartLiveTail.

### SDK for Java 2.x

Include the required files.

```

import io.reactivex.FlowableSubscriber;
import io.reactivex.annotations.NonNull;
import org.reactivestreams.Subscription;
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.services.cloudwatchlogs.CloudWatchLogsAsyncClient;
import software.amazon.awssdk.services.cloudwatchlogs.model.LiveTailSessionLogEvent;
import software.amazon.awssdk.services.cloudwatchlogs.model.LiveTailSessionStart;
import software.amazon.awssdk.services.cloudwatchlogs.model.LiveTailSessionUpdate;
import software.amazon.awssdk.services.cloudwatchlogs.model.StartLiveTailRequest;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.StartLiveTailResponseHandler;
import software.amazon.awssdk.services.cloudwatchlogs.model.CloudWatchLogsException;
import
 software.amazon.awssdk.services.cloudwatchlogs.model.StartLiveTailResponseStream;

import java.util.Date;
import java.util.List;
import java.util.concurrent.atomic.AtomicReference;

```

Handle the events from the Live Tail session.

```
private static StartLiveTailResponseHandler
getStartLiveTailResponseStreamHandler(
 AtomicReference<Subscription> subscriptionAtomicReference) {
 return StartLiveTailResponseHandler.builder()
 .onResponse(r -> System.out.println("Received initial response"))
 .onError(throwable -> {
 CloudWatchLogsException e = (CloudWatchLogsException)
throwable.getCause();
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 })
 .subscriber(() -> new FlowableSubscriber<>() {
 @Override
 public void onSubscribe(@NonNull Subscription s) {
 subscriptionAtomicReference.set(s);
 s.request(Long.MAX_VALUE);
 }

 @Override
 public void onNext(StartLiveTailResponseStream event) {
 if (event instanceof LiveTailSessionStart) {
 LiveTailSessionStart sessionStart = (LiveTailSessionStart)
event;

 System.out.println(sessionStart);
 } else if (event instanceof LiveTailSessionUpdate) {
 LiveTailSessionUpdate sessionUpdate =
(LiveTailSessionUpdate) event;
 List<LiveTailSessionLogEvent> logEvents =
sessionUpdate.sessionResults();
 logEvents.forEach(e -> {
 long timestamp = e.timestamp();
 Date date = new Date(timestamp);
 System.out.println "[" + date + "] " + e.message());
 });
 } else {
 throw CloudWatchLogsException.builder().message("Unknown
event type").build();
 }
 }
 })
 }
```

```

 @Override
 public void onError(Throwable throwable) {
 System.out.println(throwable.getMessage());
 System.exit(1);
 }

 @Override
 public void onComplete() {
 System.out.println("Completed Streaming Session");
 }
 })
 .build();
}

```

Start the Live Tail session.

```

CloudWatchLogsAsyncClient cloudWatchLogsAsyncClient =
 CloudWatchLogsAsyncClient.builder()
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

StartLiveTailRequest request =
 StartLiveTailRequest.builder()
 .logGroupIdentifiers(logGroupIdentifiers)
 .logStreamNames(logStreamNames)
 .logEventFilterPattern(logEventFilterPattern)
 .build();

/* Create a reference to store the subscription */
final AtomicReference<Subscription> subscriptionAtomicReference = new
AtomicReference<>(null);

cloudWatchLogsAsyncClient.startLiveTail(request,
getStartLiveTailResponseStreamHandler(subscriptionAtomicReference));

```

Stop the Live Tail session after a period of time has elapsed.

```

/* Set a timeout for the session and cancel the subscription. This will:
 * 1). Close the stream
 * 2). Stop the Live Tail session

```

```
 */
 try {
 Thread.sleep(10000);
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 if (subscriptionAtomicReference.get() != null) {
 subscriptionAtomicReference.get().cancel();
 System.out.println("Subscription to stream closed");
 }
}
```

- For API details, see [StartLiveTail](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Use scheduled events to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by an Amazon EventBridge scheduled event.

#### SDK for Java 2.x

Shows how to create an Amazon EventBridge scheduled event that invokes an AWS Lambda function. Configure EventBridge to use a cron expression to schedule when the Lambda function is invoked. In this example, you create a Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create an app that sends a mobile text message to your employees that congratulates them at the one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- CloudWatch Logs
- DynamoDB
- EventBridge
- Lambda
- Amazon SNS

# Amazon Cognito Identity examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Cognito Identity.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### CreateIdentityPool

The following code example shows how to use CreateIdentityPool.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cognitoidentity.CognitoIdentityClient;
import
 software.amazon.awssdk.services.cognitoidentity.model.CreateIdentityPoolRequest;
import
 software.amazon.awssdk.services.cognitoidentity.model.CreateIdentityPoolResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
```

```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreateIdentityPool {
 public static void main(String[] args) {
 final String usage = ""
 Usage:
 <identityPoolName>\s

 Where:
 identityPoolName - The name to give your identity pool.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String identityPoolName = args[0];
 CognitoIdentityClient cognitoClient = CognitoIdentityClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String identityPoolId = createIdPool(cognitoClient, identityPoolName);
 System.out.println("Unity pool ID " + identityPoolId);
 cognitoClient.close();
 }

 public static String createIdPool(CognitoIdentityClient cognitoClient, String
identityPoolName) {
 try {
 CreateIdentityPoolRequest poolRequest =
CreateIdentityPoolRequest.builder()
 .allowUnauthenticatedIdentities(false)
 .identityPoolName(identityPoolName)
 .build();

 CreateIdentityPoolResponse response =
cognitoClient.createIdentityPool(poolRequest);
 return response.identityPoolId();

 } catch (CognitoIdentityProviderException e) {

```

```

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see [CreateIdentityPool](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteIdentityPool

The following code example shows how to use DeleteIdentityPool.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.awscore.exception.AwsServiceException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cognitoidentity.CognitoIdentityClient;
import
 software.amazon.awssdk.services.cognitoidentity.model.DeleteIdentityPoolRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteIdentityPool {

 public static void main(String[] args) {
 final String usage = ""

```

```

 Usage:
 <identityPoolId>\s

 Where:
 identityPoolId - The Id value of your identity pool.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String identityPoolId = args[0];
 CognitoIdentityClient cognitoIdClient = CognitoIdentityClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 deleteIdPool(cognitoIdClient, identityPoolId);
 cognitoIdClient.close();
}

public static void deleteIdPool(CognitoIdentityClient cognitoIdClient, String
identityPoolId) {
 try {

 DeleteIdentityPoolRequest identityPoolRequest =
DeleteIdentityPoolRequest.builder()
 .identityPoolId(identityPoolId)
 .build();

 cognitoIdClient.deleteIdentityPool(identityPoolRequest);
 System.out.println("Done");

 } catch (AwsServiceException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [DeleteIdentityPool](#) in *AWS SDK for Java 2.x API Reference*.



## GetCredentialsForIdentity

The following code example shows how to use `GetCredentialsForIdentity`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cognitoidentity.CognitoIdentityClient;
import
 software.amazon.awssdk.services.cognitoidentity.model.GetCredentialsForIdentityRequest;
import
 software.amazon.awssdk.services.cognitoidentity.model.GetCredentialsForIdentityResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetIdentityCredentials {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <identityId>\s

 Where:
 identityId - The Id of an existing identity in the format
 REGION:GUID.
 """;

 if (args.length != 1) {
```

```

 System.out.println(usage);
 System.exit(1);
 }

 String identityId = args[0];
 CognitoIdentityClient cognitoClient = CognitoIdentityClient.builder()
 .region(Region.US_EAST_1)
 .build();

 getCredsForIdentity(cognitoClient, identityId);
 cognitoClient.close();
}

public static void getCredsForIdentity(CognitoIdentityClient cognitoClient,
String identityId) {
 try {
 GetCredentialsForIdentityRequest getCredentialsForIdentityRequest =
 GetCredentialsForIdentityRequest
 .builder()
 .identityId(identityId)
 .build();

 GetCredentialsForIdentityResponse response = cognitoClient
 .getCredentialsForIdentity(getCredentialsForIdentityRequest);
 System.out.println(
 "Identity ID " + response.identityId() + ", Access key ID " +
response.credentials().accessKeyId());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [GetCredentialsForIdentity](#) in *AWS SDK for Java 2.x API Reference*.

## ListIdentityPools

The following code example shows how to use `ListIdentityPools`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.cognitoidentity.CognitoIdentityClient;
import
 software.amazon.awssdk.services.cognitoidentity.model.ListIdentityPoolsRequest;
import
 software.amazon.awssdk.services.cognitoidentity.model.ListIdentityPoolsResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListIdentityPools {
 public static void main(String[] args) {
 CognitoIdentityClient cognitoClient = CognitoIdentityClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listIdPools(cognitoClient);
 cognitoClient.close();
 }

 public static void listIdPools(CognitoIdentityClient cognitoClient) {
 try {
 ListIdentityPoolsRequest poolsRequest =
 ListIdentityPoolsRequest.builder()
 .maxResults(15)
 .build();
 }
 }
}
```

```
ListIdentityPoolsResponse response =
cognitoClient.listIdentityPools(poolsRequest);
response.identityPools().forEach(pool -> {
 System.out.println("Pool ID: " + pool.identityPoolId());
 System.out.println("Pool name: " + pool.identityPoolName());
});

} catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see [ListIdentityPools](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Cognito Identity Provider examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Cognito Identity Provider.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)

# Get started

## Hello Amazon Cognito

The following code example shows how to get started using Amazon Cognito.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUserPoolsResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUserPoolsRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListUserPools {
 public static void main(String[] args) {
 CognitoIdentityProviderClient cognitoClient =
 CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllUserPools(cognitoClient);
 cognitoClient.close();
 }
}
```

```
public static void listAllUserPools(CognitoIdentityProviderClient cognitoClient)
{
 try {
 ListUserPoolsRequest request = ListUserPoolsRequest.builder()
 .maxResults(10)
 .build();

 ListUserPoolsResponse response = cognitoClient.listUserPools(request);
 response.userPools().forEach(userpool -> {
 System.out.println("User pool " + userpool.name() + ", User ID " +
 userpool.id());
 });

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListUserPools](#) in *AWS SDK for Java 2.x API Reference*.

## Actions

### AdminGetUser

The following code example shows how to use AdminGetUser.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void getAdminUser(CognitoIdentityProviderClient
 identityProviderClient, String userName,
 String poolId) {
```

```
try {
 AdminGetUserRequest userRequest = AdminGetUserRequest.builder()
 .username(userName)
 .userPoolId(poolId)
 .build();

 AdminGetUserResponse response =
identityProviderClient.adminGetUser(userRequest);
 System.out.println("User status " + response.userStatusAsString());

} catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see [AdminGetUser](#) in *AWS SDK for Java 2.x API Reference*.

## AdminInitiateAuth

The following code example shows how to use AdminInitiateAuth.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static AdminInitiateAuthResponse
initiateAuth(CognitoIdentityProviderClient identityProviderClient,
 String clientId, String userName, String password, String userPoolId) {
 try {
 Map<String, String> authParameters = new HashMap<>();
 authParameters.put("USERNAME", userName);
 authParameters.put("PASSWORD", password);

 AdminInitiateAuthRequest authRequest =
AdminInitiateAuthRequest.builder()
```

```

 .clientId(clientId)
 .userPoolId(userPoolId)
 .authParameters(authParameters)
 .authFlow(AuthFlowType.ADMIN_USER_PASSWORD_AUTH)
 .build();

 AdminInitiateAuthResponse response =
identityProviderClient.adminInitiateAuth(authRequest);
 System.out.println("Result Challenge is : " + response.challengeName());
 return response;

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

```

- For API details, see [AdminInitiateAuth](#) in *AWS SDK for Java 2.x API Reference*.

## AdminRespondToAuthChallenge

The following code example shows how to use AdminRespondToAuthChallenge.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

// Respond to an authentication challenge.
public static void adminRespondToAuthChallenge(CognitoIdentityProviderClient
identityProviderClient,
 String userName, String clientId, String mfaCode, String session) {
 System.out.println("SOFTWARE_TOKEN_MFA challenge is generated");
 Map<String, String> challengeResponses = new HashMap<>();

 challengeResponses.put("USERNAME", userName);
}

```



```

 challengeResponses.put("SOFTWARE_TOKEN_MFA_CODE", mfaCode);

 AdminRespondToAuthChallengeRequest respondToAuthChallengeRequest =
AdminRespondToAuthChallengeRequest.builder()
 .challengeName(ChallengeNameType.SOFTWARE_TOKEN_MFA)
 .clientId(clientId)
 .challengeResponses(challengeResponses)
 .session(session)
 .build();

 AdminRespondToAuthChallengeResponse respondToAuthChallengeResult =
identityProviderClient
 .adminRespondToAuthChallenge(respondToAuthChallengeRequest);
 System.out.println("respondToAuthChallengeResult.getAuthenticationResult()"
 + respondToAuthChallengeResult.authenticationResult());
 }

```

- For API details, see [AdminRespondToAuthChallenge](#) in *AWS SDK for Java 2.x API Reference*.

## AssociateSoftwareToken

The following code example shows how to use AssociateSoftwareToken.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String getSecretForAppMFA(CognitoIdentityProviderClient
identityProviderClient, String session) {
 AssociateSoftwareTokenRequest softwareTokenRequest =
AssociateSoftwareTokenRequest.builder()
 .session(session)
 .build();

 AssociateSoftwareTokenResponse tokenResponse = identityProviderClient
 .associateSoftwareToken(softwareTokenRequest);
 String secretCode = tokenResponse.secretCode();
}

```

```
 System.out.println("Enter this token into Google Authenticator");
 System.out.println(secretCode);
 return tokenResponse.session();
 }
```

- For API details, see [AssociateSoftwareToken](#) in *AWS SDK for Java 2.x API Reference*.

## ConfirmSignUp

The following code example shows how to use ConfirmSignUp.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void confirmSignUp(CognitoIdentityProviderClient
identityProviderClient, String clientId, String code,
 String userName) {
 try {
 ConfirmSignUpRequest signUpRequest = ConfirmSignUpRequest.builder()
 .clientId(clientId)
 .confirmationCode(code)
 .username(userName)
 .build();

 identityProviderClient.confirmSignUp(signUpRequest);
 System.out.println(userName + " was confirmed");

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ConfirmSignUp](#) in *AWS SDK for Java 2.x API Reference*.

## CreateUserPool

The following code example shows how to use CreateUserPool.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CreateUserPoolRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CreateUserPoolResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateUserPool {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <userPoolName>\s

 Where:
 userPoolName - The name to give your user pool when it's
created.

 """;
```

```

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String userPoolName = args[0];
 CognitoIdentityProviderClient cognitoClient =
CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String id = createPool(cognitoClient, userPoolName);
 System.out.println("User pool ID: " + id);
 cognitoClient.close();
 }

 public static String createPool(CognitoIdentityProviderClient cognitoClient,
String userPoolName) {
 try {
 CreateUserPoolRequest request = CreateUserPoolRequest.builder()
 .poolName(userPoolName)
 .build();

 CreateUserPoolResponse response = cognitoClient.createUserPool(request);
 return response.userPool().id();

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [CreateUserPool](#) in *AWS SDK for Java 2.x API Reference*.

## CreateUserPoolClient

The following code example shows how to use `CreateUserPoolClient`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CreateUserPoolClientRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CreateUserPoolClientResponse;

/**
 * A user pool client app is an application that authenticates with Amazon
 * Cognito user pools.
 * When you create a user pool, you can configure app clients that allow mobile
 * or web applications
 * to call API operations to authenticate users, manage user attributes and
 * profiles,
 * and implement sign-up and sign-in flows.
 *
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateUserPoolClient {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clientName> <userPoolId>\s

 Where:
 clientName - The name for the user pool client to create.
```

```

 userPoolId - The ID for the user pool.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String clientName = args[0];
 String userPoolId = args[1];
 CognitoIdentityProviderClient cognitoClient =
CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 createPoolClient(cognitoClient, clientName, userPoolId);
 cognitoClient.close();
}

public static void createPoolClient(CognitoIdentityProviderClient cognitoClient,
String clientName,
 String userPoolId) {
 try {
 CreateUserPoolClientRequest request =
CreateUserPoolClientRequest.builder()
 .clientName(clientName)
 .userPoolId(userPoolId)
 .build();

 CreateUserPoolClientResponse response =
cognitoClient.createUserPoolClient(request);
 System.out.println("User pool " + response.userPoolClient().clientName()
+ " created. ID: "
 + response.userPoolClient().clientId());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [CreateUserPoolClient](#) in *AWS SDK for Java 2.x API Reference*.

## ListUserPools

The following code example shows how to use ListUserPools.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUserPoolsResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUserPoolsRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListUserPools {
 public static void main(String[] args) {
 CognitoIdentityProviderClient cognitoClient =
 CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllUserPools(cognitoClient);
 cognitoClient.close();
 }

 public static void listAllUserPools(CognitoIdentityProviderClient cognitoClient)
 {
```

```

 try {
 ListUserPoolsRequest request = ListUserPoolsRequest.builder()
 .maxResults(10)
 .build();

 ListUserPoolsResponse response = cognitoClient.listUserPools(request);
 response.userPools().forEach(userpool -> {
 System.out.println("User pool " + userpool.name() + ", User ID " +
 userpool.id());
 });

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [ListUserPools](#) in *AWS SDK for Java 2.x API Reference*.

## ListUsers

The following code example shows how to use ListUsers.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUsersRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ListUsersResponse;

```



```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListUsers {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <userPoolId>\s

 Where:
 userPoolId - The ID given to your user pool when it's created.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String userPoolId = args[0];
 CognitoIdentityProviderClient cognitoClient =
CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllUsers(cognitoClient, userPoolId);
 listUsersFilter(cognitoClient, userPoolId);
 cognitoClient.close();
 }

 public static void listAllUsers(CognitoIdentityProviderClient cognitoClient,
String userPoolId) {
 try {
 ListUsersRequest usersRequest = ListUsersRequest.builder()
 .userPoolId(userPoolId)
 .build();

```

```

 ListUsersResponse response = cognitoClient.listUsers(usersRequest);
 response.users().forEach(user -> {
 System.out.println("User " + user.username() + " Status " +
 user.userStatus() + " Created "
 + user.userCreateDate());
 });

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Shows how to list users by using a filter.
public static void listUsersFilter(CognitoIdentityProviderClient cognitoClient,
 String userPoolId) {

 try {
 String filter = "email = \"tblue@noserver.com\"";
 ListUsersRequest usersRequest = ListUsersRequest.builder()
 .userPoolId(userPoolId)
 .filter(filter)
 .build();

 ListUsersResponse response = cognitoClient.listUsers(usersRequest);
 response.users().forEach(user -> {
 System.out.println("User with filter applied " + user.username() + "
 Status " + user.userStatus()
 + " Created " + user.userCreateDate());
 });

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListUsers](#) in *AWS SDK for Java 2.x API Reference*.

## ResendConfirmationCode

The following code example shows how to use ResendConfirmationCode.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void resendConfirmationCode(CognitoIdentityProviderClient
identityProviderClient, String clientId,
 String userName) {
 try {
 ResendConfirmationCodeRequest codeRequest =
ResendConfirmationCodeRequest.builder()
 .clientId(clientId)
 .username(userName)
 .build();

 ResendConfirmationCodeResponse response =
identityProviderClient.resendConfirmationCode(codeRequest);
 System.out.println("Method of delivery is " +
response.codeDeliveryDetails().deliveryMediumAsString());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ResendConfirmationCode](#) in *AWS SDK for Java 2.x API Reference*.

## SignUp

The following code example shows how to use SignUp.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void signUp(CognitoIdentityProviderClient identityProviderClient,
String clientId, String userName,
 String password, String email) {
 AttributeType userAttrs = AttributeType.builder()
 .name("email")
 .value(email)
 .build();

 List<AttributeType> userAttrsList = new ArrayList<>();
 userAttrsList.add(userAttrs);
 try {
 SignUpRequest signUpRequest = SignUpRequest.builder()
 .userAttributes(userAttrsList)
 .username(userName)
 .clientId(clientId)
 .password(password)
 .build();

 identityProviderClient.signUp(signUpRequest);
 System.out.println("User has been signed up ");

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [SignUp](#) in *AWS SDK for Java 2.x API Reference*.

## VerifySoftwareToken

The following code example shows how to use `VerifySoftwareToken`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Verify the TOTP and register for MFA.
public static void verifyTOTP(CognitoIdentityProviderClient
identityProviderClient, String session, String code) {
 try {
 VerifySoftwareTokenRequest tokenRequest =
VerifySoftwareTokenRequest.builder()
 .userCode(code)
 .session(session)
 .build();

 VerifySoftwareTokenResponse verifyResponse =
identityProviderClient.verifySoftwareToken(tokenRequest);
 System.out.println("The status of the token is " +
verifyResponse.statusAsString());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [VerifySoftwareToken](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Sign up a user with a user pool that requires MFA

The following code example shows how to:

- Sign up and confirm a user with a username, password, and email address.
- Set up multi-factor authentication by associating an MFA application with the user.

- Sign in by using a password and an MFA code.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.cognitoidentityprovider.CognitoIdentityProviderClient;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminGetUserRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminGetUserResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminInitiateAuthRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminInitiateAuthResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminRespondToAuthChallengeRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AdminRespondToAuthChallengeResponse;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AssociateSoftwareTokenRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.AssociateSoftwareTokenResponse;
import software.amazon.awssdk.services.cognitoidentityprovider.model.AttributeType;
import software.amazon.awssdk.services.cognitoidentityprovider.model.AuthFlowType;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ChallengeNameType;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.CognitoIdentityProviderException;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ConfirmSignUpRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ResendConfirmationCodeRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.ResendConfirmationCodeResponse;
import software.amazon.awssdk.services.cognitoidentityprovider.model.SignUpRequest;
```

```
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.VerifySoftwareTokenRequest;
import
 software.amazon.awssdk.services.cognitoidentityprovider.model.VerifySoftwareTokenResponse;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * TIP: To set up the required user pool, run the AWS Cloud Development Kit (AWS
 * CDK) script provided in this GitHub repo at
 * resources/cdk/cognito_scenario_user_pool_with_mfa.
 *
 * This code example performs the following operations:
 *
 * 1. Invokes the signUp method to sign up a user.
 * 2. Invokes the adminGetUser method to get the user's confirmation status.
 * 3. Invokes the ResendConfirmationCode method if the user requested another
 * code.
 * 4. Invokes the confirmSignUp method.
 * 5. Invokes the AdminInitiateAuth to sign in. This results in being prompted
 * to set up TOTP (time-based one-time password). (The response is
 * "ChallengeName": "MFA_SETUP").
 * 6. Invokes the AssociateSoftwareToken method to generate a TOTP MFA private
 * key. This can be used with Google Authenticator.
 * 7. Invokes the VerifySoftwareToken method to verify the TOTP and register for
 * MFA.
 * 8. Invokes the AdminInitiateAuth to sign in again. This results in being
 * prompted to submit a TOTP (Response: "ChallengeName": "SOFTWARE_TOKEN_MFA").
 * 9. Invokes the AdminRespondToAuthChallenge to get back a token.
 */

public class CognitoMVP {
```

```

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws NoSuchAlgorithmException,
InvalidKeyException {
 final String usage = ""

 Usage:
 <clientId> <poolId>

 Where:
 clientId - The app client Id value that you can get from the AWS
CDK script.
 poolId - The pool Id that you can get from the AWS CDK script.\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String clientId = args[0];
 String poolId = args[1];
 CognitoIdentityProviderClient identityProviderClient =
CognitoIdentityProviderClient.builder()
 .region(Region.US_EAST_1)
 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon Cognito example scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("*** Enter your user name");
 Scanner in = new Scanner(System.in);
 String userName = in.nextLine();

 System.out.println("*** Enter your password");
 String password = in.nextLine();

 System.out.println("*** Enter your email");
 String email = in.nextLine();

 System.out.println("1. Signing up " + userName);
 signUp(identityProviderClient, clientId, userName, password, email);
 }

```



```
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("2. Getting " + userName + " in the user pool");
getAdminUser(identityProviderClient, userName, poolId);

System.out
 .println("*** Conformation code sent to " + userName + ". Would you
like to send a new code? (Yes/No)");
System.out.println(DASHES);

System.out.println(DASHES);
String ans = in.nextLine();

if (ans.compareTo("Yes") == 0) {
 resendConfirmationCode(identityProviderClient, clientId, userName);
 System.out.println("3. Sending a new confirmation code");
}
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Enter confirmation code that was emailed");
String code = in.nextLine();
confirmSignUp(identityProviderClient, clientId, code, userName);
System.out.println("Rechecking the status of " + userName + " in the user
pool");
getAdminUser(identityProviderClient, userName, poolId);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Invokes the initiateAuth to sign in");
AdminInitiateAuthResponse authResponse =
initiateAuth(identityProviderClient, clientId, userName, password,
 poolId);
String mySession = authResponse.session();
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Invokes the AssociateSoftwareToken method to generate
a TOTP key");
String newSession = getSecretForAppMFA(identityProviderClient, mySession);
System.out.println(DASHES);

System.out.println(DASHES);
```

```

 System.out.println("*** Enter the 6-digit code displayed in Google
Authenticator");
 String myCode = in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Verify the TOTP and register for MFA");
 verifyTOTP(identityProviderClient, newSession, myCode);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Re-enter a 6-digit code displayed in Google
Authenticator");
 String mfaCode = in.nextLine();
 AdminInitiateAuthResponse authResponse1 =
initiateAuth(identityProviderClient, clientId, userName, password,
 poolId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. Invokes the AdminRespondToAuthChallenge");
 String session2 = authResponse1.session();
 adminRespondToAuthChallenge(identityProviderClient, userName, clientId,
mfaCode, session2);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("All Amazon Cognito operations were successfully
performed");
 System.out.println(DASHES);
 }

 // Respond to an authentication challenge.
 public static void adminRespondToAuthChallenge(CognitoIdentityProviderClient
identityProviderClient,
 String userName, String clientId, String mfaCode, String session) {
 System.out.println("SOFTWARE_TOKEN_MFA challenge is generated");
 Map<String, String> challengeResponses = new HashMap<>();

 challengeResponses.put("USERNAME", userName);
 challengeResponses.put("SOFTWARE_TOKEN_MFA_CODE", mfaCode);

 AdminRespondToAuthChallengeRequest respondToAuthChallengeRequest =
AdminRespondToAuthChallengeRequest.builder()

```

```

 .challengeName(ChallengeNameType.SOFTWARE_TOKEN_MFA)
 .clientId(clientId)
 .challengeResponses(challengeResponses)
 .session(session)
 .build();

 AdminRespondToAuthChallengeResponse respondToAuthChallengeResult =
identityProviderClient
 .adminRespondToAuthChallenge(respondToAuthChallengeRequest);
 System.out.println("respondToAuthChallengeResult.getAuthenticationResult()"
 + respondToAuthChallengeResult.authenticationResult());
}

// Verify the TOTP and register for MFA.
public static void verifyTOTP(CognitoIdentityProviderClient
identityProviderClient, String session, String code) {
 try {
 VerifySoftwareTokenRequest tokenRequest =
VerifySoftwareTokenRequest.builder()
 .userCode(code)
 .session(session)
 .build();

 VerifySoftwareTokenResponse verifyResponse =
identityProviderClient.verifySoftwareToken(tokenRequest);
 System.out.println("The status of the token is " +
verifyResponse.statusAsString());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static AdminInitiateAuthResponse
initiateAuth(CognitoIdentityProviderClient identityProviderClient,
 String clientId, String userName, String password, String userPoolId) {
 try {
 Map<String, String> authParameters = new HashMap<>();
 authParameters.put("USERNAME", userName);
 authParameters.put("PASSWORD", password);

 AdminInitiateAuthRequest authRequest =
AdminInitiateAuthRequest.builder()

```

```

 .clientId(clientId)
 .userPoolId(userPoolId)
 .authParameters(authParameters)
 .authFlow(AuthFlowType.ADMIN_USER_PASSWORD_AUTH)
 .build();

 AdminInitiateAuthResponse response =
identityProviderClient.adminInitiateAuth(authRequest);
 System.out.println("Result Challenge is : " + response.challengeName());
 return response;

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

 public static String getSecretForAppMFA(CognitoIdentityProviderClient
identityProviderClient, String session) {
 AssociateSoftwareTokenRequest softwareTokenRequest =
AssociateSoftwareTokenRequest.builder()
 .session(session)
 .build();

 AssociateSoftwareTokenResponse tokenResponse = identityProviderClient
 .associateSoftwareToken(softwareTokenRequest);
 String secretCode = tokenResponse.secretCode();
 System.out.println("Enter this token into Google Authenticator");
 System.out.println(secretCode);
 return tokenResponse.session();
 }

 public static void confirmSignUp(CognitoIdentityProviderClient
identityProviderClient, String clientId, String code,
 String userName) {
 try {
 ConfirmSignUpRequest signUpRequest = ConfirmSignUpRequest.builder()
 .clientId(clientId)
 .confirmationCode(code)
 .username(userName)
 .build();

```

```

 identityProviderClient.confirmSignUp(signUpRequest);
 System.out.println(userName + " was confirmed");

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void resendConfirmationCode(CognitoIdentityProviderClient
identityProviderClient, String clientId,
 String userName) {
 try {
 ResendConfirmationCodeRequest codeRequest =
ResendConfirmationCodeRequest.builder()
 .clientId(clientId)
 .username(userName)
 .build();

 ResendConfirmationCodeResponse response =
identityProviderClient.resendConfirmationCode(codeRequest);
 System.out.println("Method of delivery is " +
response.codeDeliveryDetails().deliveryMediumAsString());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void signUp(CognitoIdentityProviderClient identityProviderClient,
String clientId, String userName,
 String password, String email) {
 AttributeType userAttrs = AttributeType.builder()
 .name("email")
 .value(email)
 .build();

 List<AttributeType> userAttrsList = new ArrayList<>();
 userAttrsList.add(userAttrs);
 try {
 SignUpRequest signUpRequest = SignUpRequest.builder()
 .userAttributes(userAttrsList)
 .username(userName)

```

```

 .clientId(clientId)
 .password(password)
 .build();

 identityProviderClient.signUp(signUpRequest);
 System.out.println("User has been signed up ");

} catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

}

public static void getAdminUser(CognitoIdentityProviderClient
identityProviderClient, String userName,
 String poolId) {
 try {
 AdminGetUserRequest userRequest = AdminGetUserRequest.builder()
 .username(userName)
 .userPoolId(poolId)
 .build();

 AdminGetUserResponse response =
identityProviderClient.adminGetUser(userRequest);
 System.out.println("User status " + response.userStatusAsString());

 } catch (CognitoIdentityProviderException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AdminGetUser](#)
  - [AdminInitiateAuth](#)
  - [AdminRespondToAuthChallenge](#)
  - [AssociateSoftwareToken](#)
  - [ConfirmDevice](#)
  - [ConfirmSignUp](#)

- [InitiateAuth](#)
- [ListUsers](#)
- [ResendConfirmationCode](#)
- [RespondToAuthChallenge](#)
- [SignUp](#)
- [VerifySoftwareToken](#)

## Amazon Comprehend examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Comprehend.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### CreateDocumentClassifier

The following code example shows how to use `CreateDocumentClassifier`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import
 software.amazon.awssdk.services.comprehend.model.CreateDocumentClassifierRequest;
import
 software.amazon.awssdk.services.comprehend.model.CreateDocumentClassifierResponse;
import
 software.amazon.awssdk.services.comprehend.model.DocumentClassifierInputDataConfig;

/**
 * Before running this code example, you can setup the necessary resources, such
 * as the CSV file and IAM Roles, by following this document:
 * https://aws.amazon.com/blogs/machine-learning/building-a-custom-classifier-using-amazon-comprehend/
 *
 * Also, set up your development environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DocumentClassifierDemo {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <dataAccessRoleArn> <s3Uri> <documentClassifierName>

 Where:
 dataAccessRoleArn - The ARN value of the role used for this
operation.
 s3Uri - The Amazon S3 bucket that contains the CSV file.
 documentClassifierName - The name of the document classifier.
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String dataAccessRoleArn = args[0];
 String s3Uri = args[1];
 String documentClassifierName = args[2];
 }
}

```



```

 Region region = Region.US_EAST_1;
 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 createDocumentClassifier(comClient, dataAccessRoleArn, s3Uri,
documentClassifierName);
 comClient.close();
 }

 public static void createDocumentClassifier(ComprehendClient comClient, String
dataAccessRoleArn, String s3Uri,
 String documentClassifierName) {
 try {
 DocumentClassifierInputDataConfig config =
DocumentClassifierInputDataConfig.builder()
 .s3Uri(s3Uri)
 .build();

 CreateDocumentClassifierRequest createDocumentClassifierRequest =
CreateDocumentClassifierRequest.builder()
 .documentClassifierName(documentClassifierName)
 .dataAccessRoleArn(dataAccessRoleArn)
 .languageCode("en")
 .inputDataConfig(config)
 .build();

 CreateDocumentClassifierResponse createDocumentClassifierResult =
comClient
 .createDocumentClassifier(createDocumentClassifierRequest);
 String documentClassifierArn =
createDocumentClassifierResult.documentClassifierArn();
 System.out.println("Document Classifier ARN: " + documentClassifierArn);

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [CreateDocumentClassifier](#) in *AWS SDK for Java 2.x API Reference*.

## DetectDominantLanguage

The following code example shows how to use DetectDominantLanguage.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import
 software.amazon.awssdk.services.comprehend.model.DetectDominantLanguageRequest;
import
 software.amazon.awssdk.services.comprehend.model.DetectDominantLanguageResponse;
import software.amazon.awssdk.services.comprehend.model.DominantLanguage;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectLanguage {
 public static void main(String[] args) {
 // Specify French text - "It is raining today in Seattle".
 String text = "Il pleut aujourd'hui à Seattle";
 Region region = Region.US_EAST_1;

 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 System.out.println("Calling DetectDominantLanguage");
 detectTheDominantLanguage(comClient, text);
 comClient.close();
 }
}
```

```

 }

 public static void detectTheDominantLanguage(ComprehendClient comClient, String
text) {
 try {
 DetectDominantLanguageRequest request =
DetectDominantLanguageRequest.builder()
 .text(text)
 .build();

 DetectDominantLanguageResponse resp =
comClient.detectDominantLanguage(request);
 List<DominantLanguage> allLanList = resp.languages();
 for (DominantLanguage lang : allLanList) {
 System.out.println("Language is " + lang.languageCode());
 }

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DetectDominantLanguage](#) in *AWS SDK for Java 2.x API Reference*.

## DetectEntities

The following code example shows how to use DetectEntities.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.DetectEntitiesRequest;

```

```

import software.amazon.awssdk.services.comprehend.model.DetectEntitiesResponse;
import software.amazon.awssdk.services.comprehend.model.Entity;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectEntities {
 public static void main(String[] args) {
 String text = "Amazon.com, Inc. is located in Seattle, WA and was founded
 July 5th, 1994 by Jeff Bezos, allowing customers to buy everything from books to
 blenders. Seattle is north of Portland and south of Vancouver, BC. Other notable
 Seattle - based companies are Starbucks and Boeing.";
 Region region = Region.US_EAST_1;
 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 System.out.println("Calling DetectEntities");
 detectAllEntities(comClient, text);
 comClient.close();
 }

 public static void detectAllEntities(ComprehendClient comClient, String text) {
 try {
 DetectEntitiesRequest detectEntitiesRequest =
 DetectEntitiesRequest.builder()
 .text(text)
 .languageCode("en")
 .build();

 DetectEntitiesResponse detectEntitiesResult =
 comClient.detectEntities(detectEntitiesRequest);
 List<Entity> entList = detectEntitiesResult.entities();
 for (Entity entity : entList) {
 System.out.println("Entity text is " + entity.text());
 }
 }
 }
}

```

```

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DetectEntities](#) in *AWS SDK for Java 2.x API Reference*.

## DetectKeyPhrases

The following code example shows how to use DetectKeyPhrases.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.DetectKeyPhrasesRequest;
import software.amazon.awssdk.services.comprehend.model.DetectKeyPhrasesResponse;
import software.amazon.awssdk.services.comprehend.model.KeyPhrase;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectKeyPhrases {
 public static void main(String[] args) {
 String text = "Amazon.com, Inc. is located in Seattle, WA and was founded
 July 5th, 1994 by Jeff Bezos, allowing customers to buy everything from books to

```

```

blenders. Seattle is north of Portland and south of Vancouver, BC. Other notable
Seattle - based companies are Starbucks and Boeing.";
 Region region = Region.US_EAST_1;
 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 System.out.println("Calling DetectKeyPhrases");
 detectAllKeyPhrases(comClient, text);
 comClient.close();
}

public static void detectAllKeyPhrases(ComprehendClient comClient, String text)
{
 try {
 DetectKeyPhrasesRequest detectKeyPhrasesRequest =
 DetectKeyPhrasesRequest.builder()
 .text(text)
 .languageCode("en")
 .build();

 DetectKeyPhrasesResponse detectKeyPhrasesResult =
 comClient.detectKeyPhrases(detectKeyPhrasesRequest);
 List<KeyPhrase> phraseList = detectKeyPhrasesResult.keyPhrases();
 for (KeyPhrase keyPhrase : phraseList) {
 System.out.println("Key phrase text is " + keyPhrase.text());
 }

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DetectKeyPhrases](#) in *AWS SDK for Java 2.x API Reference*.

## DetectSentiment

The following code example shows how to use DetectSentiment.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import software.amazon.awssdk.services.comprehend.model.DetectSentimentRequest;
import software.amazon.awssdk.services.comprehend.model.DetectSentimentResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectSentiment {
 public static void main(String[] args) {
 String text = "Amazon.com, Inc. is located in Seattle, WA and was founded
 July 5th, 1994 by Jeff Bezos, allowing customers to buy everything from books to
 blenders. Seattle is north of Portland and south of Vancouver, BC. Other notable
 Seattle - based companies are Starbucks and Boeing.";
 Region region = Region.US_EAST_1;
 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 System.out.println("Calling DetectSentiment");
 detectSentiments(comClient, text);
 comClient.close();
 }

 public static void detectSentiments(ComprehendClient comClient, String text) {
 try {
 DetectSentimentRequest detectSentimentRequest =
 DetectSentimentRequest.builder()
```

```

 .text(text)
 .languageCode("en")
 .build();

 DetectSentimentResponse detectSentimentResult =
comClient.detectSentiment(detectSentimentRequest);
 System.out.println("The Neutral value is " +
detectSentimentResult.sentimentScore().neutral());

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DetectSentiment](#) in *AWS SDK for Java 2.x API Reference*.

## DetectSyntax

The following code example shows how to use DetectSyntax.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.comprehend.ComprehendClient;
import software.amazon.awssdk.services.comprehend.model.ComprehendException;
import software.amazon.awssdk.services.comprehend.model.DetectSyntaxRequest;
import software.amazon.awssdk.services.comprehend.model.DetectSyntaxResponse;
import software.amazon.awssdk.services.comprehend.model.SyntaxToken;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.

```



```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class DetectSyntax {
 public static void main(String[] args) {
 String text = "Amazon.com, Inc. is located in Seattle, WA and was founded
 July 5th, 1994 by Jeff Bezos, allowing customers to buy everything from books to
 blenders. Seattle is north of Portland and south of Vancouver, BC. Other notable
 Seattle - based companies are Starbucks and Boeing.";
 Region region = Region.US_EAST_1;
 ComprehendClient comClient = ComprehendClient.builder()
 .region(region)
 .build();

 System.out.println("Calling DetectSyntax");
 detectAllSyntax(comClient, text);
 comClient.close();
 }

 public static void detectAllSyntax(ComprehendClient comClient, String text) {
 try {
 DetectSyntaxRequest detectSyntaxRequest = DetectSyntaxRequest.builder()
 .text(text)
 .languageCode("en")
 .build();

 DetectSyntaxResponse detectSyntaxResult =
 comClient.detectSyntax(detectSyntaxRequest);
 List<SyntaxToken> syntaxTokens = detectSyntaxResult.syntaxTokens();
 for (SyntaxToken token : syntaxTokens) {
 System.out.println("Language is " + token.text());
 System.out.println("Part of speech is " +
 token.partOfSpeech().tagAsString());
 }

 } catch (ComprehendException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DetectSyntax](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Building an Amazon Lex chatbot

The following code example shows how to create a chatbot to engage your website visitors.

#### SDK for Java 2.x

Shows how to use the Amazon Lex API to create a Chatbot within a web application to engage your web site visitors.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Comprehend
- Amazon Lex
- Amazon Translate

### Create a messaging application

The following code example shows how to create a messaging application by using Amazon SQS.

#### SDK for Java 2.x

Shows how to use the Amazon SQS API to develop a Spring REST API that sends and retrieves messages.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Comprehend
- Amazon SQS

## Create an application to analyze customer feedback

The following code example shows how to create an application that analyzes customer comment cards, translates them from their original language, determines their sentiment, and generates an audio file from the translated text.

### SDK for Java 2.x

This example application analyzes and stores customer feedback cards. Specifically, it fulfills the need of a fictitious hotel in New York City. The hotel receives feedback from guests in various languages in the form of physical comment cards. That feedback is uploaded into the app through a web client. After an image of a comment card is uploaded, the following steps occur:

- Text is extracted from the image using Amazon Textract.
- Amazon Comprehend determines the sentiment of the extracted text and its language.
- The extracted text is translated to English using Amazon Translate.
- Amazon Polly synthesizes an audio file from the extracted text.

The full app can be deployed with the AWS CDK. For source code and deployment instructions, see the project in [GitHub](#).

### Services used in this example

- Amazon Comprehend
- Lambda
- Amazon Polly
- Amazon Textract
- Amazon Translate

## AWS Control Tower examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Control Tower.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello AWS Control Tower

The following code example shows how to get started using AWS Control Tower.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class HelloControlTower {

 public static void main(String[] args) {
 try {
 ControlTowerClient controlTowerClient = ControlTowerClient.builder()
 .build() ;
 helloControlTower(controlTowerClient);
 } catch (ControlTowerException e) {
 System.out.println("Control Tower error occurred: " +
e.awsErrorDetails().errorMessage());
 }
 }

 /**
 * Use the AWS SDK for Java (v2) to create an AWS Control Tower client
 * and list all available baselines.
 * This example uses the default settings specified in your shared credentials
 * and config files.
 */
}
```

```

*
* @param controlTowerClient A ControlTowerClient object. This object wraps
* the low-level AWS Control Tower service API.
*/
public static void helloControlTower(ControlTowerClient controlTowerClient) {
 System.out.println("Hello, AWS Control Tower! Let's list available
baselines:\n");

 ListBaselinesIterable paginator = controlTowerClient.listBaselinesPaginator(
 ListBaselinesRequest.builder().build());
 List<String> baselineNames = new ArrayList<>();

 try {
 paginator.stream()
 .flatMap(response -> response.baselines().stream())
 .forEach(baseline -> baselineNames.add(baseline.name()));

 System.out.println(baselineNames.size() + " baseline(s) retrieved.");
 for (String baselineName : baselineNames) {
 System.out.println("\t" + baselineName);
 }

 } catch (ControlTowerException e) {
 if ("AccessDeniedException".equals(e.awsErrorDetails().errorCode())) {
 System.out.println("Access denied. Please ensure you have the
necessary permissions.");
 } else {
 System.out.println("An error occurred: " + e.getMessage());
 }
 }
}
}

```

- For API details, see [ListBaselines](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- List landing zones.

- List, enable, get, reset, and disable baselines.
- List, enable, get, and disable controls.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating AWS Control Tower features.

```
public class ControlTowerScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final Scanner scanner = new Scanner(in);

 private static OrganizationsClient orgClient;
 private static ControlCatalogClient catClient;

 private static String ouId = null;
 private static String ouArn = null;
 private static String landingZoneArn = null;
 private static boolean useLandingZone = false;

 private String stack = null;
 private String accountId = null;

 public static void main(String[] args) {

 System.out.println(DASHES);
 System.out.println("Welcome to the AWS Control Tower basics scenario!");
 System.out.println(DASHES);

 try {
 runScenarioAsync();
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

```

// -----
// Utilities
// -----
private static boolean askYesNo(String msg) {
 System.out.println(msg);
 return scanner.nextLine().trim().toLowerCase().startsWith("y");
}

private static void runScenarioAsync() {
 try {
 ControlTowerActions actions = new ControlTowerActions();

 // -----
 // Step 1: Landing Zones
 // -----
 System.out.println(DASHES);
 System.out.println("""
Some demo operations require the use of a landing zone.
You can use an existing landing zone or opt out of these operations in the
demo.
For instructions on how to set up a landing zone,
see https://docs.aws.amazon.com/controltower/latest/userguide/getting-started-from-console.html
""");

 System.out.println("Step 1: Listing landing zones...");
 waitForInputToContinue(scanner);

 List<LandingZoneSummary> landingZones =
 actions.listLandingZonesAsync().join();

 if (landingZones.isEmpty()) {
 System.out.println("No landing zones found. Landing-zone-dependent
steps will be skipped.");
 useLandingZone = false;
 waitForInputToContinue(scanner);
 } else {
 System.out.println("\nAvailable Landing Zones:");
 for (int i = 0; i < landingZones.size(); i++) {
 System.out.printf("%d) %s\n", i + 1, landingZones.get(i).arn());
 }

 if (askYesNo("Do you want to use the first landing zone in the list
(" +

```

```

 landingZones.get(0).arn() + ")? (y/n): ")) {
 useLandingZone = true;
 landingZoneArn = landingZones.get(0).arn();
 } else if (askYesNo("Do you want to use a different existing Landing
Zone for this demo? (y/n): ")) {
 useLandingZone = true;
 System.out.println("Enter landing zone ARN: ");
 landingZoneArn = scanner.nextLine().trim();
 } else {
 System.out.println("Proceeding without a landing zone.");
 useLandingZone = false;
 waitForInputToContinue(scanner);
 }
 }

 // -----
 // Setup Organization + Sandbox OU
 // -----
 if (useLandingZone) {
 System.out.println("Using landing zone ARN: " + landingZoneArn);

 ControlTowerActions.OrgSetupResult result =
 actions.setupOrganizationAsync().join();

 ouArn = result.sandboxOuArn();
 ouId = result.sandboxOuArn();

 System.out.println("Organization ID: " + result.orgId());
 System.out.println("Using Sandbox OU ARN: " + ouArn);
 }

 // -----
 // Step 2: Baselines
 // -----
 System.out.println(DASHES);
 System.out.println("Step 2: Listing available baselines...");
 System.out.println("

```

In this step, the program lists available AWS Control Tower baselines and may perform baseline-related operations (enable, disable, reset) if requested.

#### NOTE:

AWS Control Tower enforces governance through baselines and mandatory controls (guardrails). Mandatory controls are required for landing zone governance and may



restrict certain operations depending on the account, region, or organizational policy.

For more information, see:

- Types of baselines in AWS Control Tower:  
<https://docs.aws.amazon.com/controltower/latest/userguide/types-of-baselines.html>
- Mandatory controls (guardrails) in AWS Control Tower:  
<https://docs.aws.amazon.com/controltower/latest/controlreference/mandatory-controls.html>
- Baseline API examples:  
<https://docs.aws.amazon.com/controltower/latest/userguide/baseline-api-examples.html>

```
""");
```

```

waitForInputToContinue(scanner);
List<BaselineSummary> baselines =
 actions.listBaselinesAsync().join();

BaselineSummary controlTowerBaseline = null;
for (BaselineSummary b : baselines) {
 System.out.println("Baseline: " + b.name());
 System.out.println(" ARN: " + b.arn());
 if ("AWSControlTowerBaseline".equals(b.name())) {
 controlTowerBaseline = b;
 }
}

waitForInputToContinue(scanner);

if (useLandingZone && controlTowerBaseline != null) {

 System.out.println("\nListing enabled baselines:");
 List<EnabledBaselineSummary> enabledBaselines =
 actions.listEnabledBaselinesAsync().join();

 String enabledBaselineArn = null;
 for (EnabledBaselineSummary eb : enabledBaselines) {
 System.out.println("Checking enabled baseline ARN: " +
eb.arn());
 if (eb.baselineIdentifier().equals(controlTowerBaseline.arn()))
{

```

```

 enabledBaselineArn = eb.arn(); // correct enabled ARN for
this baseline
 break; // stop after finding the matching one
 }
}

if (enabledBaselineArn == null) {
 System.out.println("No enabled baseline found for " +
controlTowerBaseline.arn());
} else {
 System.out.println("Selected enabled baseline ARN for reset/
disable: " + enabledBaselineArn);
}

// Enable the Baseline
if (askYesNo("Do you want to enable the Control Tower Baseline? (y/
n): ")) {
 System.out.println("\nEnabling Control Tower Baseline...");

 String baselineId = controlTowerBaseline.arn();
 String enabledBaselineId =
 actions.enableBaselineAsync(
 ouArn, // targetIdentifier #
the OU or account ARN
 baselineId, // baselineIdentifier #
the Control Tower baseline ARN
 "5.0" // baselineVersion #
version string
).join();

 System.out.println("Enabled baseline operation ID: " +
enabledBaselineId);
 if (enabledBaselineId == null) {
 enabledBaselineId = enabledBaselineArn;
 }

 // Reset the Baseline
 if (askYesNo("Do you want to reset the Control Tower Baseline?
(y/n): ")) {
 String operationId =

 actions.resetEnabledBaselineAsync(enabledBaselineId).join();

```

```

 System.out.println("Reset baseline operation ID: " +
operationId);
 }

 if (askYesNo("Do you want to disable the Control Tower Baseline?
(y/n): ")) {
 String operationId =

actions.disableBaselineAsync(enabledBaselineId).join();
 System.out.println("Disabled baseline operation ID: " +
operationId);

 System.out.println("Now we will re-enable the baseline and
wait 1 minute before making the call...");
 try {
 Thread.sleep(Duration.ofMinutes(1).toMillis());
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 System.out.println("Wait interrupted");
 }
 String reEnabledBaselineId = actions.enableBaselineAsync(
 ouArn,
 baselineId, // reuse baseline definition ARN
 "5.0"
).join();

 System.out.println("Re-enabled baseline operation ID: " +
reEnabledBaselineId);
 }
}

// -----
// Step 3: Controls
// -----
System.out.println(DASHES);
System.out.println("Step 3: Managing Controls:");
waitForInputToContinue(scanner);

List<ControlSummary> controls =
 actions.listControlsAsync().join();

System.out.println("\nListing first 5 available Controls:");
for (int i = 0; i < Math.min(5, controls.size()); i++) {

```

```

 ControlSummary c = controls.get(i);
 System.out.println("%d. %s - %s".formatted(i + 1, c.name(),
c.arn()));
 }

 if (useLandingZone) {
 waitForInputToContinue(scanner);

 List<EnabledControlSummary> enabledControls =
 actions.listEnabledControlsAsync(ouArn).join();

 System.out.println("\nListing enabled controls:");
 for (int i = 0; i < enabledControls.size(); i++) {
 System.out.println("%d. %s".formatted(i + 1,
enabledControls.get(i).controlIdentifier()));
 }

 String controlArnToEnable = null;
 for (ControlSummary control : controls) {
 boolean enabled = enabledControls.stream()
 .anyMatch(ec ->
ec.controlIdentifier().equals(control.arn()));
 if (!enabled) {
 controlArnToEnable = control.arn();
 break;
 }
 }

 waitForInputToContinue(scanner);
 if (controlArnToEnable != null &&
 askYesNo("Do you want to enable the control " +
controlArnToEnable + "? (y/n): ")) {

 String operationId =
 actions.enableControlAsync(controlArnToEnable,
ouArn).join();

 System.out.println("Enabled control with operation ID: " +
operationId);
 }

 waitForInputToContinue(scanner);

 if (controlArnToEnable != null &&

```

```

 askYesNo("Do you want to disable the control? (y/n): ")) {

 String operationId =
 actions.disableControlAsync(controlArnToEnable,
ouArn).join();

 System.out.println("Disable operation ID: " + operationId);
 }
 }
 System.out.println("\nThis concludes the example scenario.");
 System.out.println("Thanks for watching!");
 System.out.println(DASHES);

} catch (CompletionException e) {
 Throwable cause = e.getCause() != null ? e.getCause() : e;
 System.out.println("Scenario failed: " + cause.getMessage());
 throw e; // bubble up for tests / callers
} catch (Exception e) {
 System.out.println("Unexpected error running scenario: " +
e.getMessage());
 throw new RuntimeException(e);
}
}

private static void waitForInputToContinue(Scanner sc) {
 System.out.println("\nEnter 'c' then <ENTER> to continue:");
 while (true) {
 String input = sc.nextLine();
 if ("c".equalsIgnoreCase(input.trim())) {
 System.out.println("Continuing...");
 break;
 }
 }
}

}

public class ControlTowerActions {
 private static ControlCatalogAsyncClient controlCatalogAsyncClient;
 private static ControlTowerAsyncClient controlTowerAsyncClient;
 private static OrganizationsAsyncClient orgAsyncClient;

 private static OrganizationsAsyncClient getAsyncOrgClient() {
 if (orgAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()

```

```

 .maxConcurrency(50)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .build();

 orgAsyncClient = OrganizationsAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
}
return orgAsyncClient;
}

private static ControlCatalogAsyncClient getAsyncCatClient() {
 if (controlCatalogAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 controlCatalogAsyncClient = ControlCatalogAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return controlCatalogAsyncClient;
}

```

```

private static ControlTowerAsyncClient getAsyncClient() {
 if (controlTowerAsyncClient == null) {

 SdkAsyncHttpClient httpClient =
 AwsCrtAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 controlTowerAsyncClient =
 ControlTowerAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }

 return controlTowerAsyncClient;
}

public record OrgSetupResult(String orgId, String sandboxOuArn) {
}

public CompletableFuture<OrgSetupResult> setupOrganizationAsync() {
 System.out.println("Starting organization setup...");

 OrganizationsAsyncClient client = getAsyncOrgClient();

 // Step 1: Describe or create organization
 CompletableFuture<Organization> orgFuture = client.describeOrganization()
 .thenApply(desc -> {
 System.out.println("Organization exists: " +
desc.organization().id());
 return desc.organization();
 })
 .exceptionallyCompose(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;
 if (cause instanceof AwsServiceException awsEx &&

```

```

"AWSOrganizationsNotInUseException".equals(awsEx.awsErrorDetails().errorCode())) {
 System.out.println("No organization found. Creating one...");
 return
client.createOrganization(CreateOrganizationRequest.builder()
 .featureSet(OrganizationFeatureSet.ALL)
 .build())
 .thenApply(createResp -> {
 System.out.println("Created organization: {" +
createResp.organization().id());
 return createResp.organization();
 });
 }
 return CompletableFuture.failedFuture(
 new CompletionException("Failed to describe or create
organization", cause)
);
});

// Step 2: Locate Sandbox OU
return orgFuture.thenCompose(org -> {
 String orgId = org.id();
 System.out.println("Organization ID: {" + orgId);

 return client.listRoots()
 .thenCompose(rootsResp -> {
 if (rootsResp.roots().isEmpty()) {
 return CompletableFuture.failedFuture(
 new RuntimeException("No root found in
organization")
);
 }
 String rootId = rootsResp.roots().get(0).id();

 ListOrganizationalUnitsForParentRequest ouRequest =
 ListOrganizationalUnitsForParentRequest.builder()
 .parentId(rootId)
 .build();

 ListOrganizationalUnitsForParentPublisher paginator =
client.listOrganizationalUnitsForParentPaginator(ouRequest);

```



```

 AtomicReference<String> sandboxOuArnRef = new
AtomicReference<>();
 return paginator.subscribe(page -> {
 for (OrganizationalUnit ou :
page.organizationalUnits()) {
 if ("Sandbox".equals(ou.name())) {
 sandboxOuArnRef.set(ou.arn());
 System.out.println("Found Sandbox OU: "
+ ou.id());
 break;
 }
 }
 })
 .thenApply(v -> {
 String sandboxArn = sandboxOuArnRef.get();
 if (sandboxArn == null) {
 System.out.println("Sandbox OU not found.");
 }
 return new OrgSetupResult(orgId, sandboxArn);
 });
 });
 }).exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;
 System.out.println("Failed to setup organization: {}" +
cause.getMessage());
 throw new CompletionException(cause);
 });
}

/**
 * Lists all landing zones using pagination to retrieve complete results.
 *
 * @return a list of all landing zones
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<LandingZoneSummary>> listLandingZonesAsync() {
 System.out.println("Starting list landing zones paginator...");

 ListLandingZonesRequest request = ListLandingZonesRequest.builder().build();
 ListLandingZonesPublisher paginator =
getAsyncClient().listLandingZonesPaginator(request);
 List<LandingZoneSummary> landingZones = new ArrayList<>();

```

```

 return paginator.subscribe(response -> {
 if (response.landingZones() != null && !
response.landingZones().isEmpty()) {
 response.landingZones().forEach(lz -> {
 System.out.println("Landing zone ARN: " + lz.arn());
 landingZones.add(lz);
 });
 } else {
 System.out.println("Page contained no landing zones.");
 }
 })
 .thenRun(() -> System.out.println("Successfully retrieved "+
landingZones.size() + " landing zones."))
 .thenApply(v -> landingZones)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();
 switch (errorCode) {
 case "AccessDeniedException":
 throw new CompletionException(
 "Access denied when listing landing zones: "
+ e.getMessage(), e);
 default:
 throw new CompletionException(
 "Error listing landing zones: " +
e.getMessage(), e);
 }
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing landing zones: " +
cause.getMessage(), cause);
 }

 throw new CompletionException("Failed to list landing zones",
cause);
 });
 }

 /**

```

```

 * Lists all available baselines using pagination to retrieve complete results.
 *
 * @return a list of all baselines
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<BaselineSummary>> listBaselinesAsync() {
 System.out.println("Starting list baselines paginator...");
 ListBaselinesRequest request = ListBaselinesRequest.builder().build();
 ListBaselinesPublisher paginator =
 getAsyncClient().listBaselinesPaginator(request);

 List<BaselineSummary> baselines = new ArrayList<>();
 return paginator.subscribe(response -> {
 if (response.baselines() != null && !
response.baselines().isEmpty()) {
 response.baselines().forEach(baseline -> {
 baselines.add(baseline);
 });
 } else {
 System.out.println("Page contained no baselines.");
 }
 })
 .thenRun(() ->
 System.out.println("Successfully listed baselines. Total: "
+ baselines.size())
)
 .thenApply(v -> baselines)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("AccessDeniedException".equals(errorCode)) {
 throw new CompletionException(
 "Access denied when listing baselines:
%s".formatted(e.getMessage()),
 e
);
 }
 }

 throw new CompletionException(

```

```

 "Error listing baselines:
%s".formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing baselines:
%s".formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException("Failed to list baselines",
cause);
 });
}

/**
 * Lists all enabled baselines using pagination to retrieve complete results.
 *
 * @return a list of all enabled baselines
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<EnabledBaselineSummary>>
listEnabledBaselinesAsync() {
 System.out.println("Starting list enabled baselines paginator...");

 ListEnabledBaselinesRequest request =
 ListEnabledBaselinesRequest.builder().build();

 ListEnabledBaselinesPublisher paginator =
 getAsyncClient().listEnabledBaselinesPaginator(request);

 List<EnabledBaselineSummary> enabledBaselines = new ArrayList<>();
 return paginator.subscribe(response -> {
 if (response.enabledBaselines() != null
 && !response.enabledBaselines().isEmpty()) {

 response.enabledBaselines().forEach(baseline -> {
 enabledBaselines.add(baseline);
 });
 }
 });
}

```

```

 } else {
 System.out.println("Page contained no enabled baselines.");
 }
 })
 .thenRun(() ->
 System.out.println(
 "Successfully listed enabled baselines. Total: " +
 enabledBaselines.size()
)
)
 .thenApply(v -> enabledBaselines)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("AccessDeniedException".equals(errorCode)) {
 throw new CompletionException(
 "Access denied when listing enabled baselines:
%s".formatted(e.getMessage()), e);
 }

 throw new CompletionException(
 "Error listing enabled baselines: %s"
 .formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing enabled baselines: %s"
 .formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to list enabled baselines",
 cause
);
 });
}

```

```

/**
 * Asynchronously enables a baseline for the specified target if not already
 * enabled.
 *
 * @param targetIdentifier The ARN of the target (OU or account).
 * @param baselineIdentifier The baseline definition ARN to enable.
 * @param baselineVersion The baseline version to enable.
 * @return A CompletableFuture containing the enabled baseline ARN, or null if
 * already enabled.
 */
public CompletableFuture<String> enableBaselineAsync(
 String targetIdentifier,
 String baselineIdentifier,
 String baselineVersion
) {
 EnableBaselineRequest request = EnableBaselineRequest.builder()
 .baselineIdentifier(baselineIdentifier)
 .baselineVersion(baselineVersion)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().enableBaseline(request)
 .handle((resp, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;
 if (cause instanceof ControlTowerException e) {
 String code = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String msg = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

 if ("ValidationException".equals(code) &&
msg.contains("already enabled")) {
 System.out.println("Baseline is already enabled for
this target # fetching ARN...");
 return fetchEnabledBaselineArn(targetIdentifier,
baselineIdentifier)

 .join(); // fetch existing ARN synchronously
 }
 }
 }
 })
}

```

```

 throw new RuntimeException("Error enabling baseline: " +
code + " - " + msg, e);
 }

 throw new RuntimeException("Unexpected error enabling
baseline: " + cause.getMessage(), cause);
}

 return resp;
})
.thenCompose(result -> {
 if (result instanceof EnableBaselineResponse resp) {
 String operationId = resp.operationIdentifier();
 String enabledBaselineArn = resp.arn();
 System.out.println("Baseline enable started. ARN: " +
enabledBaselineArn
 + ", operation ID: " + operationId);

 // Inline polling
 return CompletableFuture.supplyAsync(() -> {
 while (true) {
 GetBaselineOperationRequest opReq =
GetBaselineOperationRequest.builder()
 .operationIdentifier(operationId)
 .build();

 GetBaselineOperationResponse opResp =
getAsyncClient().getBaselineOperation(opReq).join();
 BaselineOperation op = opResp.baselineOperation();
 BaselineOperationStatus status = op.status();
 System.out.println("Operation " + operationId + "
status: " + status);

 if (status == BaselineOperationStatus.SUCCEEDED) {
 return enabledBaselineArn;
 } else if (status == BaselineOperationStatus.FAILED)
{
 String opId = op.operationIdentifier();
 String reason = op.statusMessage() != null ?
op.statusMessage() : "No failure reason provided";
 throw new RuntimeException("Baseline operation
failed (ID: " + opId + "), status: "
 + status + ", reason: " + reason);
 }
 }
 });
 }
});

```

```

 try {
 Thread.sleep(Duration.ofSeconds(15).toMillis());
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException(e);
 }
 }
});
} else if (result instanceof String existingArn) {
 // Already enabled branch
 return CompletableFuture.completedFuture(existingArn);
}

return CompletableFuture.completedFuture(null);
});
}

/**
 * Fetches the ARN of an already-enabled baseline for the target asynchronously.
 */
private CompletableFuture<String> fetchEnabledBaselineArn(String
targetIdentifier, String baselineIdentifier) {
 return
getAsyncClient().listEnabledBaselines(ListEnabledBaselinesRequest.builder().build())
 .thenApply(listResp -> {
 for (EnabledBaselineSummary eb : listResp.enabledBaselines()) {
 if (baselineIdentifier.equals(eb.baselineIdentifier())
 && targetIdentifier.equals(eb.targetIdentifier())) {
 return eb.arn();
 }
 }
 return null; // not yet available
 });
}

/**
 * Disables a baseline for a specified target.
 *
 * @param enabledBaselineIdentifier the identifier of the enabled baseline to
disable
 * @return the operation identifier

```



```

 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
 public CompletableFuture<String> disableBaselineAsync(String
enabledBaselineIdentifier) {

 System.out.println("Starting disable of enabled baseline...");
 System.out.println("This operation will check the status every 15 seconds
until it completes (SUCCEEDED or FAILED).");

 DisableBaselineRequest request = DisableBaselineRequest.builder()
 .enabledBaselineIdentifier(enabledBaselineIdentifier)
 .build();

 return getAsyncClient().disableBaseline(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Disable baseline operation ID: " +
operationId);

 // CompletableFuture that will be completed when operation
finishes
 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 // Polling loop
 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getBaselineOperationAsync(operationId)
 .thenAccept(statusObj -> {
 String status = statusObj.toString(); //
Convert enum/status to string for printing
 System.out.println("Current disable
operation status: " + status + " # waiting for SUCCEEDED or FAILED...");

 if ("SUCCEEDED".equalsIgnoreCase(status) ||
"FAILED".equalsIgnoreCase(status)) {
 System.out.println("Disable operation
finished with status: " + status);
 resultFuture.complete(operationId);
 } else {
 // Schedule next poll in 15 seconds

```

```

 CompletableFuture.delayedExecutor(15,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 System.out.println("Error checking baseline
operation status: " + ex.getMessage());
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 };

 // Start first poll immediately
 poller.run();

 return resultFuture;
 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String errorMessage = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

 System.out.println("ControlTowerException caught while
disabling baseline: Code=" + errorCode + ", Message=" + errorMessage);
 return null;
 }

 if (cause instanceof SdkException sdkEx) {
 System.out.println("SDK exception caught while disabling
baseline: " + sdkEx.getMessage());
 return null;
 }

 System.out.println("Unexpected exception while disabling
baseline: " + cause.getMessage());
 return null;
 });
 }

```

```

/**
 * Gets the status of a baseline operation.
 *
 * @param operationIdentifier the identifier of the operation
 * @return the operation status
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<BaselineOperationStatus> getBaselineOperationAsync(
 String operationIdentifier) {

 GetBaselineOperationRequest request = GetBaselineOperationRequest.builder()
 .operationIdentifier(operationIdentifier)
 .build();

 return getAsyncClient().getBaselineOperation(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null
 ? exception.getCause()
 : exception;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 throw new CompletionException(
 "Baseline operation not found: %s"
 .formatted(e.getMessage()),
 e
);
 }

 throw new CompletionException(
 "Error getting baseline operation status: %s"
 .formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {

```

```

 throw new CompletionException(
 "SDK error getting baseline operation status:
%s"
 .formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to get baseline operation status",
 cause
);
}

}))
.thenApply(response -> {
 BaselineOperationStatus status =
 response.baselineOperation().status();
 return status;
});
}

/**
 * Lists all enabled controls for a specific target using pagination.
 *
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return a list of enabled controls
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<EnabledControlSummary>>
listEnabledControlsAsync(String targetIdentifier) {
 System.out.println("Starting list enabled controls paginator for target " +
targetIdentifier);
 ListEnabledControlsRequest request = ListEnabledControlsRequest.builder()
 .targetIdentifier(targetIdentifier)
 .build();

 ListEnabledControlsPublisher paginator =
getAsyncClient().listEnabledControlsPaginator(request);
 List<EnabledControlSummary> enabledControls = new ArrayList<>();

 // Subscribe to the paginator asynchronously
 return paginator.subscribe(response -> {

```

```

 if (response.enabledControls() != null && !
response.enabledControls().isEmpty()) {
 response.enabledControls().forEach(control -> {
 enabledControls.add(control);
 });
 } else {
 System.out.println("Page contained no enabled controls.");
 }
 })
 .thenRun(() -> System.out.println(
 "Successfully retrieved "+enabledControls.size() +" enabled
controls for target "+targetIdentifier
))
 .thenApply(v -> enabledControls)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 switch (errorCode) {
 case "AccessDeniedException":
 throw new CompletionException(
 "Access denied when listing enabled
controls: %s".formatted(e.getMessage()), e);

 case "ResourceNotFoundException":
 if (e.getMessage() != null &&
e.getMessage().contains("not registered with AWS Control Tower")) {
 throw new CompletionException(
 "Control Tower must be enabled to work
with controls", e);
 }
 throw new CompletionException(
 "Target not found when listing enabled
controls: %s".formatted(e.getMessage()), e);

 default:
 throw new CompletionException(
 "Error listing enabled controls:
%s".formatted(e.getMessage()), e);
 }
 }
 })
}

```

```

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing enabled controls:
%s".formatted(cause.getMessage()), cause);
 }

 throw new CompletionException("Failed to list enabled controls",
cause);
 });
}

/**
 * Enables a control for a specified target.
 *
 * @param controlIdIdentifier the identifier of the control to enable
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<String> enableControlAsync(
 String controlIdIdentifier,
 String targetIdentifier) {

 EnableControlRequest request = EnableControlRequest.builder()
 .controlIdentifier(controlIdentifier)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().enableControl(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Enable control operation started. Operation
ID: " + operationId);

 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getControlOperationAsync(operationId)
 .thenAccept(status -> {

```

```

 System.out.println("Control operation
status: " + status);

 if (status ==
ControlOperationStatus.SUCCEEDED
 || status ==
ControlOperationStatus.FAILED) {
 resultFuture.complete(operationId);
 } else {
 // Poll again after 30 seconds
 CompletableFuture.delayedExecutor(30,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 resultFuture.completeExceptionally(ex);
 return null;
 });
}

};

// Start polling immediately
poller.run();

return resultFuture;
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();
 String message = e.getMessage() != null ? e.getMessage() :
"";

 if ("ValidationException".equals(errorCode)
 && message.contains("already enabled")) {
 System.out.println("Control is already enabled for this
target");

 return null;
 }

 if ("ValidationException".equals(errorCode)
 && message.contains("enforced by default")) {

```

```

 System.out.println("Control is enforced by default and
cannot be enabled manually.");
 return null;
 }

 if ("ResourceNotFoundException".equals(errorCode)
 && message.contains("not registered with AWS Control
Tower")) {

 System.out.println(
 "Control Tower must be enabled to work with
controls.");

 return null;
 }

 throw new CompletionException(
 "Couldn't enable control: %s".formatted(message),
 e
);
}

if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error enabling control: %s"
 .formatted(cause.getMessage()),
 cause
);
}

throw new CompletionException(
 "Failed to enable control",
 cause
);
});
}

/**
 * Disables a control for a specified target.
 *
 * @param controlIdIdentifier the identifier of the control to disable
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs

```



```

 */
 public CompletableFuture<String> disableControlAsync(
 String controlIdentifier,
 String targetIdentifier) {

 DisableControlRequest request = DisableControlRequest.builder()
 .controlIdentifier(controlIdentifier)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().disableControl(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Disable control operation started. Operation
ID: " + operationId);

 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getControlOperationAsync(operationId)
 .thenAccept(status -> {
 System.out.println("Control operation
status: " + status);

 if (status ==
ControlOperationStatus.SUCCEEDED
 || status ==
ControlOperationStatus.FAILED) {
 resultFuture.complete(operationId);
 } else {
 // poll again after 30 seconds
 CompletableFuture.delayedExecutor(30,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 });
 }

```

```

 };

 // start polling immediately
 poller.run();

 return resultFuture;
 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 // SPEC: notify user and continue
 System.out.println("Control not found for disabling: " +
e.getMessage());

 return null;
 }

 throw new CompletionException(
 "Error disabling control: " + e.getMessage(), e);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error disabling control: " +
cause.getMessage(), cause);
 }

 throw new CompletionException(
 "Failed to disable control", cause);
 });
}

/**
 * Gets the status of a control operation.
 *
 * @param operationIdentifier the identifier of the operation
 * @return the operation status
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs

```

```

 */
 public CompletableFuture<ControlOperationStatus> getControlOperationAsync(
 String operationIdentifier) {

 GetControlOperationRequest request = GetControlOperationRequest.builder()
 .operationIdentifier(operationIdentifier)
 .build();

 return getAsyncClient().getControlOperation(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 throw new CompletionException(
 "Control operation not found:
%s".formatted(e.getMessage()),
 e
);
 }

 throw new CompletionException(
 "Error getting control operation status:
%s".formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error getting control operation status:
%s".formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException("Failed to get control
operation status", cause);
 }
 })
 }

```

```

 .thenApply(response -> response.controlOperation().status());
 }

 /**
 * Lists all controls in the Control Tower control catalog.
 *
 * @return a list of controls
 * @throws SdkException if a service-specific error occurs
 */
 public CompletableFuture<List<ControlSummary>> listControlsAsync() {
 System.out.println("Starting list controls paginator...");

 ListControlsRequest request = ListControlsRequest.builder().build();
 ListControlsPublisher paginator =
getAsyncCatClient().listControlsPaginator(request);
 List<ControlSummary> controls = new ArrayList<>();

 return paginator.subscribe(response -> {
 if (response.controls() != null && !
response.controls().isEmpty()) {
 response.controls().forEach(control -> {
 controls.add(control);
 });
 } else {
 System.out.println("Page contained no controls.");
 }
 })
 .thenRun(() -> System.out.println("Successfully retrieved " +
controls.size() + " controls."))
 .thenApply(v -> controls)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof SdkException sdkEx) {
 if (sdkEx.getMessage() != null &&
sdkEx.getMessage().contains("AccessDeniedException")) {
 throw new CompletionException(
 "Access denied when listing controls. Please
ensure you have the necessary permissions.",
 sdkEx
);
 } else {
 throw new CompletionException(

```

```

 "SDK error listing controls:
%s".formatted(sdkEx.getMessage()),
 sdkEx
);
 }
 }

 throw new CompletionException("Failed to list controls", cause);
 });
 }

 /**
 * Resets an enabled baseline for a specific target.
 *
 * @param enabledBaselineIdentifier the identifier of the enabled baseline to
reset
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
 public CompletableFuture<String> resetEnabledBaselineAsync(String
enabledBaselineIdentifier) {

 System.out.println("Starting reset of enabled baseline...");
 System.out.println("This operation will check the status every 15 seconds
until it completes (SUCCEEDED or FAILED).");

 ResetEnabledBaselineRequest request = ResetEnabledBaselineRequest.builder()
 .enabledBaselineIdentifier(enabledBaselineIdentifier)
 .build();

 return getAsyncClient().resetEnabledBaseline(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Reset enabled baseline operation ID: " +
operationId);

 // Polling loop
 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {

```

```

 getBaselineOperationAsync(operationId)
 .thenAccept(statusObj -> {
 String status = statusObj.toString(); //
Convert enum/status to string for printing
 System.out.println("Current baseline
operation status: " + status + " # waiting for SUCCEEDED or FAILED...");

 if ("SUCCEEDED".equalsIgnoreCase(status) ||
"FAILED".equalsIgnoreCase(status)) {
 System.out.println("Baseline operation
finished with status: " + status);

 resultFuture.complete(operationId);
 } else {
 // Schedule next poll in 15 seconds
 CompletableFuture.delayedExecutor(15,
TimeUnit.SECONDS)

 .execute(this);
 }
 })
 .exceptionally(ex -> {
 System.out.println("Error checking baseline
operation status: " + ex.getMessage());
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
};

// Start first poll immediately
poller.run();

return resultFuture;
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String errorMessage = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

 System.out.println("ControlTowerException caught: Code=" +
errorCode + ", Message=" + errorMessage);
 }
});

```

```
 return null;
 }

 if (cause instanceof SdkException sdkEx) {
 System.out.println("SDK exception caught: " +
sdkEx.getMessage());
 return null;
 }

 System.out.println("Unexpected exception resetting baseline: " +
cause.getMessage());
 return null;
});
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateLandingZone](#)
- [DeleteLandingZone](#)
- [DisableBaseline](#)
- [DisableControl](#)
- [EnableBaseline](#)
- [EnableControl](#)
- [GetControlOperation](#)
- [GetLandingZoneOperation](#)
- [ListBaselines](#)
- [ListEnabledBaselines](#)
- [ListEnabledControls](#)
- [ListLandingZones](#)
- [ResetEnabledBaseline](#)

## Actions

### DisableBaseline

The following code example shows how to use `DisableBaseline`.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Disables a baseline for a specified target.
 *
 * @param enabledBaselineIdentifier the identifier of the enabled baseline to
disable
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<String> disableBaselineAsync(String
enabledBaselineIdentifier) {

 System.out.println("Starting disable of enabled baseline...");
 System.out.println("This operation will check the status every 15 seconds
until it completes (SUCCEEDED or FAILED).");

 DisableBaselineRequest request = DisableBaselineRequest.builder()
 .enabledBaselineIdentifier(enabledBaselineIdentifier)
 .build();

 return getAsyncClient().disableBaseline(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Disable baseline operation ID: " +
operationId);

 // CompletableFuture that will be completed when operation
finishes

 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 // Polling loop
 Runnable poller = new Runnable() {
```



```

 @Override
 public void run() {
 getBaselineOperationAsync(operationId)
 .thenAccept(statusObj -> {
 String status = statusObj.toString(); //
Convert enum/status to string for printing
 System.out.println("Current disable
operation status: " + status + " # waiting for SUCCEEDED or FAILED...");

 if ("SUCCEEDED".equalsIgnoreCase(status) ||
"FAILED".equalsIgnoreCase(status)) {
 System.out.println("Disable operation
finished with status: " + status);
 resultFuture.complete(operationId);
 } else {
 // Schedule next poll in 15 seconds
 CompletableFuture.delayedExecutor(15,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 System.out.println("Error checking baseline
operation status: " + ex.getMessage());
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 };

 // Start first poll immediately
 poller.run();

 return resultFuture;
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String errorMessage = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

```

```

 System.out.println("ControlTowerException caught while
disabling baseline: Code=" + errorCode + ", Message=" + errorMessage);
 return null;
 }

 if (cause instanceof SdkException sdkEx) {
 System.out.println("SDK exception caught while disabling
baseline: " + sdkEx.getMessage());
 return null;
 }

 System.out.println("Unexpected exception while disabling
baseline: " + cause.getMessage());
 return null;
});
}

```

- For API details, see [DisableBaseline](#) in *AWS SDK for Java 2.x API Reference*.

## DisableControl

The following code example shows how to use `DisableControl`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Disables a control for a specified target.
 *
 * @param controlIdIdentifier the identifier of the control to disable
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs

```

```

 */
 public CompletableFuture<String> disableControlAsync(
 String controlIdIdentifier,
 String targetIdentifier) {

 DisableControlRequest request = DisableControlRequest.builder()
 .controlIdentifier(controlIdentifier)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().disableControl(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Disable control operation started. Operation
ID: " + operationId);

 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getControlOperationAsync(operationId)
 .thenAccept(status -> {
 System.out.println("Control operation
status: " + status);

 if (status ==
ControlOperationStatus.SUCCEEDED
 || status ==
ControlOperationStatus.FAILED) {
 resultFuture.complete(operationId);
 } else {
 // poll again after 30 seconds
 CompletableFuture.delayedExecutor(30,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 })
 }

```

```

 };

 // start polling immediately
 poller.run();

 return resultFuture;
 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 // SPEC: notify user and continue
 System.out.println("Control not found for disabling: " +
e.getMessage());

 return null;
 }

 throw new CompletionException(
 "Error disabling control: " + e.getMessage(), e);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error disabling control: " +
cause.getMessage(), cause);
 }

 throw new CompletionException(
 "Failed to disable control", cause);
 });
}

```

- For API details, see [DisableControl](#) in *AWS SDK for Java 2.x API Reference*.

## EnableBaseline

The following code example shows how to use EnableBaseline.

**SDK for Java 2.x****Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously enables a baseline for the specified target if not already
 * enabled.
 *
 * @param targetIdentifier The ARN of the target (OU or account).
 * @param baselineIdentifier The baseline definition ARN to enable.
 * @param baselineVersion The baseline version to enable.
 * @return A CompletableFuture containing the enabled baseline ARN, or null if
 * already enabled.
 */
public CompletableFuture<String> enableBaselineAsync(
 String targetIdentifier,
 String baselineIdentifier,
 String baselineVersion
) {
 EnableBaselineRequest request = EnableBaselineRequest.builder()
 .baselineIdentifier(baselineIdentifier)
 .baselineVersion(baselineVersion)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().enableBaseline(request)
 .handle((resp, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;
 if (cause instanceof ControlTowerException e) {
 String code = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String msg = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

```

```

 if ("ValidationException".equals(code) &&
msg.contains("already enabled")) {
 System.out.println("Baseline is already enabled for
this target # fetching ARN...");
 return fetchEnabledBaselineArn(targetIdentifier,
baselineIdentifier)
 .join(); // fetch existing ARN synchronously
 }

 throw new RuntimeException("Error enabling baseline: " +
code + " - " + msg, e);
 }

 throw new RuntimeException("Unexpected error enabling
baseline: " + cause.getMessage(), cause);
}

return resp;
}))
.thenCompose(result -> {
 if (result instanceof EnableBaselineResponse resp) {
 String operationId = resp.operationIdentifier();
 String enabledBaselineArn = resp.arn();
 System.out.println("Baseline enable started. ARN: " +
enabledBaselineArn
 + ", operation ID: " + operationId);

 // Inline polling
 return CompletableFuture.supplyAsync(() -> {
 while (true) {
 GetBaselineOperationRequest opReq =
GetBaselineOperationRequest.builder()
 .operationIdentifier(operationId)
 .build();

 GetBaselineOperationResponse opResp =
getAsyncClient().getBaselineOperation(opReq).join();
 BaselineOperation op = opResp.baselineOperation();
 BaselineOperationStatus status = op.status();
 System.out.println("Operation " + operationId + "
status: " + status);

 if (status == BaselineOperationStatus.SUCCEEDED) {
 return enabledBaselineArn;

```

```

 } else if (status == BaselineOperationStatus.FAILED)
 {
 String opId = op.operationIdentifier();
 String reason = op.statusMessage() != null ?
op.statusMessage() : "No failure reason provided";
 throw new RuntimeException("Baseline operation
failed (ID: " + opId + "), status: "
 + status + ", reason: " + reason);
 }

 try {
 Thread.sleep(Duration.ofSeconds(15).toMillis());
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException(e);
 }
 }
});
} else if (result instanceof String existingArn) {
 // Already enabled branch
 return CompletableFuture.completedFuture(existingArn);
}

return CompletableFuture.completedFuture(null);
});
}

/**
 * Fetches the ARN of an already-enabled baseline for the target asynchronously.
 */
private CompletableFuture<String> fetchEnabledBaselineArn(String
targetIdentifier, String baselineIdentifier) {
 return
getAsyncClient().listEnabledBaselines(ListEnabledBaselinesRequest.builder().build())
 .thenApply(listResp -> {
 for (EnabledBaselineSummary eb : listResp.enabledBaselines()) {
 if (baselineIdentifier.equals(eb.baselineIdentifier())
 && targetIdentifier.equals(eb.targetIdentifier())) {
 return eb.arn();
 }
 }
 return null; // not yet available
 });
}

```

```
}
```

- For API details, see [EnableBaseline](#) in *AWS SDK for Java 2.x API Reference*.

## EnableControl

The following code example shows how to use `EnableControl`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Enables a control for a specified target.
 *
 * @param controlIdIdentifier the identifier of the control to enable
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<String> enableControlAsync(
 String controlIdIdentifier,
 String targetIdentifier) {

 EnableControlRequest request = EnableControlRequest.builder()
 .controlIdentifier(controlIdentifier)
 .targetIdentifier(targetIdentifier)
 .build();

 return getAsyncClient().enableControl(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Enable control operation started. Operation
ID: " + operationId);
```



```

 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getControlOperationAsync(operationId)
 .thenAccept(status -> {
 System.out.println("Control operation
status: " + status);

 if (status ==
ControlOperationStatus.SUCCEEDED
 || status ==
ControlOperationStatus.FAILED) {
 resultFuture.complete(operationId);
 } else {
 // Poll again after 30 seconds
 CompletableFuture.delayedExecutor(30,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 };

 // Start polling immediately
 poller.run();

 return resultFuture;
 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();
 String message = e.getMessage() != null ? e.getMessage() :
"";

 if ("ValidationException".equals(errorCode)

```

```

 && message.contains("already enabled")) {
 System.out.println("Control is already enabled for this
target");
 return null;
 }

 if ("ValidationException".equals(errorCode)
 && message.contains("enforced by default")) {
 System.out.println("Control is enforced by default and
cannot be enabled manually.");
 return null;
 }

 if ("ResourceNotFoundException".equals(errorCode)
 && message.contains("not registered with AWS Control
Tower")) {
 System.out.println(
 "Control Tower must be enabled to work with
controls.");
 return null;
 }

 throw new CompletionException(
 "Couldn't enable control: %s".formatted(message),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error enabling control: %s"
 .formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to enable control",
 cause
);
});
}

```

- For API details, see [EnableControl](#) in *AWS SDK for Java 2.x API Reference*.

## GetBaselineOperation

The following code example shows how to use `GetBaselineOperation`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Gets the status of a baseline operation.
 *
 * @param operationIdentifier the identifier of the operation
 * @return the operation status
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<BaselineOperationStatus> getBaselineOperationAsync(
 String operationIdentifier) {

 GetBaselineOperationRequest request = GetBaselineOperationRequest.builder()
 .operationIdentifier(operationIdentifier)
 .build();

 return getAsyncClient().getBaselineOperation(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null
 ? exception.getCause()
 : exception;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 throw new CompletionException(
```

```

 "Baseline operation not found: %s"
 .formatted(e.getMessage()),
 e
);
}

throw new CompletionException(
 "Error getting baseline operation status: %s"
 .formatted(e.getMessage()),
 e
);
}

if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error getting baseline operation status:
%s"
 .formatted(cause.getMessage()),
 cause
);
}

throw new CompletionException(
 "Failed to get baseline operation status",
 cause
);
}
})
.thenApply(response -> {
 BaselineOperationStatus status =
 response.baselineOperation().status();
 return status;
});
}

```

- For API details, see [GetBaselineOperation](#) in *AWS SDK for Java 2.x API Reference*.

## GetControlOperation

The following code example shows how to use `GetControlOperation`.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Gets the status of a control operation.
 *
 * @param operationIdentifier the identifier of the operation
 * @return the operation status
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<ControlOperationStatus> getControlOperationAsync(
 String operationIdentifier) {

 GetControlOperationRequest request = GetControlOperationRequest.builder()
 .operationIdentifier(operationIdentifier)
 .build();

 return getAsyncClient().getControlOperation(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("ResourceNotFoundException".equals(errorCode)) {
 throw new CompletionException(
 "Control operation not found:
%s".formatted(e.getMessage()),
 e
);
 }
 }

 throw new CompletionException(
```

```

 "Error getting control operation status:
%s".formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error getting control operation status:
%s".formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException("Failed to get control
operation status", cause);
 }
})
 .thenApply(response -> response.controlOperation().status());
}

```

- For API details, see [GetControlOperation](#) in *AWS SDK for Java 2.x API Reference*.

## ListBaselines

The following code example shows how to use ListBaselines.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all available baselines using pagination to retrieve complete results.
 *
 * @return a list of all baselines
 * @throws ControlTowerException if a service-specific error occurs

```

```

 * @throws SdkException if an SDK error occurs
 */
 public CompletableFuture<List<BaselineSummary>> listBaselinesAsync() {
 System.out.println("Starting list baselines paginator...");
 ListBaselinesRequest request = ListBaselinesRequest.builder().build();
 ListBaselinesPublisher paginator =
 getAsyncClient().listBaselinesPaginator(request);

 List<BaselineSummary> baselines = new ArrayList<>();
 return paginator.subscribe(response -> {
 if (response.baselines() != null && !
response.baselines().isEmpty()) {
 response.baselines().forEach(baseline -> {
 baselines.add(baseline);
 });
 } else {
 System.out.println("Page contained no baselines.");
 }
 })
 .thenRun(() ->
 System.out.println("Successfully listed baselines. Total: "
+ baselines.size())
)
 .thenApply(v -> baselines)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("AccessDeniedException".equals(errorCode)) {
 throw new CompletionException(
 "Access denied when listing baselines:
%s".formatted(e.getMessage()),
 e
);
 }

 throw new CompletionException(
 "Error listing baselines:
%s".formatted(e.getMessage()),
 e
);
 }
 })
 }

```

```

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing baselines:
%s".formatted(cause.getMessage()),
 cause
);
 }

 throw new CompletionException("Failed to list baselines",
cause);
 });
}

```

- For API details, see [ListBaselines](#) in *AWS SDK for Java 2.x API Reference*.

## ListEnabledBaselines

The following code example shows how to use `ListEnabledBaselines`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all enabled baselines using pagination to retrieve complete results.
 *
 * @return a list of all enabled baselines
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<EnabledBaselineSummary>>
listEnabledBaselinesAsync() {
 System.out.println("Starting list enabled baselines paginator...");

 ListEnabledBaselinesRequest request =
 ListEnabledBaselinesRequest.builder().build();
}

```



```

ListEnabledBaselinesPublisher paginator =
 getAsyncClient().listEnabledBaselinesPaginator(request);

List<EnabledBaselineSummary> enabledBaselines = new ArrayList<>();
return paginator.subscribe(response -> {
 if (response.enabledBaselines() != null
 && !response.enabledBaselines().isEmpty()) {

 response.enabledBaselines().forEach(baseline -> {
 enabledBaselines.add(baseline);
 });
 } else {
 System.out.println("Page contained no enabled baselines.");
 }
})
.thenRun(() ->
 System.out.println(
 "Successfully listed enabled baselines. Total: " +
 enabledBaselines.size()
)
)
.thenApply(v -> enabledBaselines)
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 if ("AccessDeniedException".equals(errorCode)) {
 throw new CompletionException(
 "Access denied when listing enabled baselines:
%s".formatted(e.getMessage()), e);
 }

 throw new CompletionException(
 "Error listing enabled baselines: %s"
 .formatted(e.getMessage()),
 e
);
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(

```

```

 "SDK error listing enabled baselines: %s"
 .formatted(cause.getMessage()),
 cause
);
}

throw new CompletionException(
 "Failed to list enabled baselines",
 cause
);
});
}

```

- For API details, see [ListEnabledBaselines](#) in *AWS SDK for Java 2.x API Reference*.

## ListEnabledControls

The following code example shows how to use ListEnabledControls.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all enabled controls for a specific target using pagination.
 *
 * @param targetIdentifier the identifier of the target (e.g., OU ARN)
 * @return a list of enabled controls
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<List<EnabledControlSummary>>
listEnabledControlsAsync(String targetIdentifier) {
 System.out.println("Starting list enabled controls paginator for target " +
targetIdentifier);
 ListEnabledControlsRequest request = ListEnabledControlsRequest.builder()

```

```

 .targetIdentifier(targetIdentifier)
 .build();

 ListEnabledControlsPublisher paginator =
getAsyncClient().listEnabledControlsPaginator(request);
 List<EnabledControlSummary> enabledControls = new ArrayList<>();

 // Subscribe to the paginator asynchronously
 return paginator.subscribe(response -> {
 if (response.enabledControls() != null && !
response.enabledControls().isEmpty()) {
 response.enabledControls().forEach(control -> {
 enabledControls.add(control);
 });
 } else {
 System.out.println("Page contained no enabled controls.");
 }
 })
 .thenRun(() -> System.out.println(
 "Successfully retrieved "+enabledControls.size()+" enabled
controls for target "+targetIdentifier
))
 .thenApply(v -> enabledControls)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();

 switch (errorCode) {
 case "AccessDeniedException":
 throw new CompletionException(
 "Access denied when listing enabled
controls: %s".formatted(e.getMessage()), e);

 case "ResourceNotFoundException":
 if (e.getMessage() != null &&
e.getMessage().contains("not registered with AWS Control Tower")) {
 throw new CompletionException(
 "Control Tower must be enabled to work
with controls", e);
 }
 throw new CompletionException(

```

```

 "Target not found when listing enabled
controls: %s".formatted(e.getMessage()), e);

 default:
 throw new CompletionException(
 "Error listing enabled controls:
%s".formatted(e.getMessage()), e);
 }
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(
 "SDK error listing enabled controls:
%s".formatted(cause.getMessage()), cause);
 }

 throw new CompletionException("Failed to list enabled controls",
cause);
 });
 }

```

- For API details, see [ListEnabledControls](#) in *AWS SDK for Java 2.x API Reference*.

## ListLandingZones

The following code example shows how to use ListLandingZones.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all landing zones using pagination to retrieve complete results.
 *
 * @return a list of all landing zones
 * @throws ControlTowerException if a service-specific error occurs

```

```

 * @throws SdkException if an SDK error occurs
 */
 public CompletableFuture<List<LandingZoneSummary>> listLandingZonesAsync() {
 System.out.println("Starting list landing zones paginator...");

 ListLandingZonesRequest request = ListLandingZonesRequest.builder().build();
 ListLandingZonesPublisher paginator =
 getAsyncClient().listLandingZonesPaginator(request);
 List<LandingZoneSummary> landingZones = new ArrayList<>();

 return paginator.subscribe(response -> {
 if (response.landingZones() != null && !
response.landingZones().isEmpty()) {
 response.landingZones().forEach(lz -> {
 System.out.println("Landing zone ARN: " + lz.arn());
 landingZones.add(lz);
 });
 } else {
 System.out.println("Page contained no landing zones.");
 }
 })
 .thenRun(() -> System.out.println("Successfully retrieved " +
landingZones.size() + " landing zones."))
 .thenApply(v -> landingZones)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails().errorCode();
 switch (errorCode) {
 case "AccessDeniedException":
 throw new CompletionException(
 "Access denied when listing landing zones: "
+ e.getMessage(), e);
 default:
 throw new CompletionException(
 "Error listing landing zones: " +
e.getMessage(), e);
 }
 }

 if (cause instanceof SdkException) {
 throw new CompletionException(

```

```

 "SDK error listing landing zones: " +
cause.getMessage(), cause);
 }

 throw new CompletionException("Failed to list landing zones",
cause);
 });
 }

```

- For API details, see [ListLandingZones](#) in *AWS SDK for Java 2.x API Reference*.

## ResetEnabledBaseline

The following code example shows how to use `ResetEnabledBaseline`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Resets an enabled baseline for a specific target.
 *
 * @param enabledBaselineIdentifier the identifier of the enabled baseline to
reset
 * @return the operation identifier
 * @throws ControlTowerException if a service-specific error occurs
 * @throws SdkException if an SDK error occurs
 */
public CompletableFuture<String> resetEnabledBaselineAsync(String
enabledBaselineIdentifier) {

 System.out.println("Starting reset of enabled baseline...");
 System.out.println("This operation will check the status every 15 seconds
until it completes (SUCCEEDED or FAILED).");

 ResetEnabledBaselineRequest request = ResetEnabledBaselineRequest.builder()
 .enabledBaselineIdentifier(enabledBaselineIdentifier)

```

```

 .build();

 return getAsyncClient().resetEnabledBaseline(request)
 .thenCompose(response -> {
 String operationId = response.operationIdentifier();
 System.out.println("Reset enabled baseline operation ID: " +
operationId);

 // Polling loop
 CompletableFuture<String> resultFuture = new
CompletableFuture<>();

 Runnable poller = new Runnable() {
 @Override
 public void run() {
 getBaselineOperationAsync(operationId)
 .thenAccept(statusObj -> {
 String status = statusObj.toString(); //
Convert enum/status to string for printing
 System.out.println("Current baseline
operation status: " + status + " # waiting for SUCCEEDED or FAILED...");

 if ("SUCCEEDED".equalsIgnoreCase(status) ||
"FAILED".equalsIgnoreCase(status)) {
 System.out.println("Baseline operation
finished with status: " + status);
 resultFuture.complete(operationId);
 } else {
 // Schedule next poll in 15 seconds
 CompletableFuture.delayedExecutor(15,
TimeUnit.SECONDS)
 .execute(this);
 }
 })
 .exceptionally(ex -> {
 System.out.println("Error checking baseline
operation status: " + ex.getMessage());
 resultFuture.completeExceptionally(ex);
 return null;
 });
 }
 };

 // Start first poll immediately

```

```

 poller.run();

 return resultFuture;
 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ControlTowerException e) {
 String errorCode = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorCode() : "UNKNOWN";
 String errorMessage = e.awsErrorDetails() != null ?
e.awsErrorDetails().errorMessage() : e.getMessage();

 System.out.println("ControlTowerException caught: Code=" +
errorCode + ", Message=" + errorMessage);
 return null;
 }

 if (cause instanceof SdkException sdkEx) {
 System.out.println("SDK exception caught: " +
sdkEx.getMessage());
 return null;
 }

 System.out.println("Unexpected exception resetting baseline: " +
cause.getMessage());
 return null;
 });
}

```

- For API details, see [ResetEnabledBaseline](#) in *AWS SDK for Java 2.x API Reference*.

## Firehose examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Firehose.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.



*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### PutRecord

The following code example shows how to use PutRecord.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Puts a record to the specified Amazon Kinesis Data Firehose delivery stream.
 *
 * @param record The record to be put to the delivery stream. The record must be
 * a {@link Map} of String keys and Object values.
 * @param deliveryStreamName The name of the Amazon Kinesis Data Firehose
 * delivery stream to which the record should be put.
 * @throws IllegalArgumentException if the input record or delivery stream name
 * is null or empty.
 * @throws RuntimeException if there is an error putting the record to the
 * delivery stream.
 */
public static void putRecord(Map<String, Object> record, String
deliveryStreamName) {
 if (record == null || deliveryStreamName == null ||
deliveryStreamName.isEmpty()) {
```

```

 throw new IllegalArgumentException("Invalid input: record or delivery
stream name cannot be null/empty");
 }
 try {
 String jsonRecord = new ObjectMapper().writeValueAsString(record);
 Record firehoseRecord = Record.builder()

.data(SdkBytes.fromByteArray(jsonRecord.getBytes(StandardCharsets.UTF_8)))
 .build();

 PutRecordRequest putRecordRequest = PutRecordRequest.builder()
 .deliveryStreamName(deliveryStreamName)
 .record(firehoseRecord)
 .build();

 getFirehoseClient().putRecord(putRecordRequest);
 System.out.println("Record sent: " + jsonRecord);
 } catch (Exception e) {
 throw new RuntimeException("Failed to put record: " + e.getMessage(),
e);
 }
}

```

- For API details, see [PutRecord](#) in *AWS SDK for Java 2.x API Reference*.

## PutRecordBatch

The following code example shows how to use PutRecordBatch.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Puts a batch of records to an Amazon Kinesis Data Firehose delivery stream.
 *
 * @param records a list of maps representing the records to be sent

```

```

 * @param batchSize the maximum number of records to include in each
batch
 * @param deliveryStreamName the name of the Kinesis Data Firehose delivery
stream
 * @throws IllegalArgumentException if the input parameters are invalid (null or
empty)
 * @throws RuntimeException if there is an error putting the record
batch
 */
 public static void putRecordBatch(List<Map<String, Object>> records, int
batchSize, String deliveryStreamName) {
 if (records == null || records.isEmpty() || deliveryStreamName == null ||
deliveryStreamName.isEmpty()) {
 throw new IllegalArgumentException("Invalid input: records or delivery
stream name cannot be null/empty");
 }
 ObjectMapper objectMapper = new ObjectMapper();

 try {
 for (int i = 0; i < records.size(); i += batchSize) {
 List<Map<String, Object>> batch = records.subList(i, Math.min(i +
batchSize, records.size()));

 List<Record> batchRecords = batch.stream().map(record -> {
 try {
 String jsonRecord = objectMapper.writeValueAsString(record);
 return Record.builder()

 .data(SdkBytes.fromByteArray(jsonRecord.getBytes(StandardCharsets.UTF_8)))
 .build();
 } catch (Exception e) {
 throw new RuntimeException("Error creating Firehose record",
e);
 }
 }).collect(Collectors.toList());

 PutRecordBatchRequest request = PutRecordBatchRequest.builder()
 .deliveryStreamName(deliveryStreamName)
 .records(batchRecords)
 .build();

 PutRecordBatchResponse response =
getFirehoseClient().putRecordBatch(request);
 }
 }
 }

```

```

 if (response.failedPutCount() > 0) {
 response.requestResponses().stream()
 .filter(r -> r.errorCode() != null)
 .forEach(r -> System.err.println("Failed record: " +
r.errorMessage()));
 }
 System.out.println("Batch sent with size: " + batchRecords.size());
 }
} catch (Exception e) {
 throw new RuntimeException("Failed to put record batch: " +
e.getMessage(), e);
}
}

```

- For API details, see [PutRecordBatch](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Put records to Firehose

The following code example shows how to use Firehose to process individual and batch records.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

This example puts individual and batch records to Firehose.

```

/**
 * Amazon Firehose Scenario example using Java V2 SDK.
 *
 * Demonstrates individual and batch record processing,
 * and monitoring Firehose delivery stream metrics.
 */
public class FirehoseScenario {

 private static FirehoseClient firehoseClient;

```

```
private static CloudWatchClient cloudWatchClient;

public static void main(String[] args) {
 final String usage = ""
 Usage:
 <deliveryStreamName>
 Where:
 deliveryStreamName - The Firehose delivery stream name.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 return;
 }

 String deliveryStreamName = args[0];

 try {
 // Read and parse sample data.
 String jsonContent = readJsonFile("sample_records.json");
 ObjectMapper objectMapper = new ObjectMapper();
 List<Map<String, Object>> sampleData =
objectMapper.readValue(jsonContent, new TypeReference<>() {});

 // Process individual records.
 System.out.println("Processing individual records...");
 sampleData.subList(0, 100).forEach(record -> {
 try {
 putRecord(record, deliveryStreamName);
 } catch (Exception e) {
 System.err.println("Error processing record: " +
e.getMessage());
 }
 });

 // Monitor metrics.
 monitorMetrics(deliveryStreamName);

 // Process batch records.
 System.out.println("Processing batch records...");
 putRecordBatch(sampleData.subList(100, sampleData.size()), 500,
deliveryStreamName);
 monitorMetrics(deliveryStreamName);
 }
```

```

 } catch (Exception e) {
 System.err.println("Scenario failed: " + e.getMessage());
 } finally {
 closeClients();
 }
 }

 private static FirehoseClient getFirehoseClient() {
 if (firehoseClient == null) {
 firehoseClient = FirehoseClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return firehoseClient;
 }

 private static CloudWatchClient getCloudWatchClient() {
 if (cloudWatchClient == null) {
 cloudWatchClient = CloudWatchClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return cloudWatchClient;
 }

 /**
 * Puts a record to the specified Amazon Kinesis Data Firehose delivery stream.
 *
 * @param record The record to be put to the delivery stream. The record must be
 * a {@link Map} of String keys and Object values.
 * @param deliveryStreamName The name of the Amazon Kinesis Data Firehose
 * delivery stream to which the record should be put.
 * @throws IllegalArgumentException if the input record or delivery stream name
 * is null or empty.
 * @throws RuntimeException if there is an error putting the record to the
 * delivery stream.
 */
 public static void putRecord(Map<String, Object> record, String
 deliveryStreamName) {
 if (record == null || deliveryStreamName == null ||
 deliveryStreamName.isEmpty()) {
 throw new IllegalArgumentException("Invalid input: record or delivery
 stream name cannot be null/empty");
 }
 }

```

```

 try {
 String jsonRecord = new ObjectMapper().writeValueAsString(record);
 Record firehoseRecord = Record.builder()

.data(SdkBytes.fromByteArray(jsonRecord.getBytes(StandardCharsets.UTF_8)))
 .build();

 PutRecordRequest putRecordRequest = PutRecordRequest.builder()
 .deliveryStreamName(deliveryStreamName)
 .record(firehoseRecord)
 .build();

 getFirehoseClient().putRecord(putRecordRequest);
 System.out.println("Record sent: " + jsonRecord);
 } catch (Exception e) {
 throw new RuntimeException("Failed to put record: " + e.getMessage(),
e);
 }
 }

 /**
 * Puts a batch of records to an Amazon Kinesis Data Firehose delivery stream.
 *
 * @param records a list of maps representing the records to be sent
 * @param batchSize the maximum number of records to include in each
batch
 * @param deliveryStreamName the name of the Kinesis Data Firehose delivery
stream
 * @throws IllegalArgumentException if the input parameters are invalid (null or
empty)
 * @throws RuntimeException if there is an error putting the record
batch
 */
 public static void putRecordBatch(List<Map<String, Object>> records, int
batchSize, String deliveryStreamName) {
 if (records == null || records.isEmpty() || deliveryStreamName == null ||
deliveryStreamName.isEmpty()) {
 throw new IllegalArgumentException("Invalid input: records or delivery
stream name cannot be null/empty");
 }
 ObjectMapper objectMapper = new ObjectMapper();

 try {

```

```

 for (int i = 0; i < records.size(); i += batchSize) {
 List<Map<String, Object>> batch = records.subList(i, Math.min(i +
batchSize, records.size()));

 List<Record> batchRecords = batch.stream().map(record -> {
 try {
 String jsonRecord = objectMapper.writeValueAsString(record);
 return Record.builder()

.data(SdkBytes.fromByteArray(jsonRecord.getBytes(StandardCharsets.UTF_8)))
 .build();
 } catch (Exception e) {
 throw new RuntimeException("Error creating Firehose record",
e);
 }
 }).collect(Collectors.toList());

 PutRecordBatchRequest request = PutRecordBatchRequest.builder()
 .deliveryStreamName(deliveryStreamName)
 .records(batchRecords)
 .build();

 PutRecordBatchResponse response =
getFirehoseClient().putRecordBatch(request);

 if (response.failedPutCount() > 0) {
 response.requestResponses().stream()
 .filter(r -> r.errorCode() != null)
 .forEach(r -> System.err.println("Failed record: " +
r.errorMessage()));
 }
 System.out.println("Batch sent with size: " + batchRecords.size());
 }
 } catch (Exception e) {
 throw new RuntimeException("Failed to put record batch: " +
e.getMessage(), e);
 }
}

public static void monitorMetrics(String deliveryStreamName) {
 Instant endTime = Instant.now();
 Instant startTime = endTime.minusSeconds(600);

```



```

 List<String> metrics = List.of("IncomingBytes", "IncomingRecords",
"FailedPutCount");
 metrics.forEach(metric -> monitorMetric(metric, startTime, endTime,
deliveryStreamName));
 }

 private static void monitorMetric(String metricName, Instant startTime, Instant
endTime, String deliveryStreamName) {
 try {
 GetMetricStatisticsRequest request =
GetMetricStatisticsRequest.builder()
 .namespace("AWS/Firehose")
 .metricName(metricName)

.dimensions(Dimension.builder().name("DeliveryStreamName").value(deliveryStreamName).build()
 .startTime(startTime)
 .endTime(endTime)
 .period(60)
 .statistics(Statistic.SUM)
 .build());

 GetMetricStatisticsResponse response =
getCloudWatchClient().getMetricStatistics(request);
 double totalSum =
response.datapoints().stream().mapToDouble(Datapoint::sum).sum();
 System.out.println(metricName + ": " + totalSum);
 } catch (Exception e) {
 System.err.println("Failed to monitor metric " + metricName + ": " +
e.getMessage());
 }
 }

 public static String readJsonFile(String fileName) throws IOException {
 try (InputStream inputStream =
FirehoseScenario.class.getResourceAsStream("/" + fileName);
 Scanner scanner = new Scanner(inputStream, StandardCharsets.UTF_8)) {
 return scanner.useDelimiter("\\\\A").next();
 } catch (Exception e) {
 throw new RuntimeException("Error reading file: " + fileName, e);
 }
 }

 private static void closeClients() {
 try {

```

```
 if (firehoseClient != null) firehoseClient.close();
 if (cloudWatchClient != null) cloudWatchClient.close();
 } catch (Exception e) {
 System.err.println("Error closing clients: " + e.getMessage());
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [PutRecord](#)
  - [PutRecordBatch](#)

## Amazon DocumentDB examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon DocumentDB.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Serverless examples](#)

## Serverless examples

### Invoke a Lambda function from a Amazon DocumentDB trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a DocumentDB change stream. The function retrieves the DocumentDB payload and logs the record contents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Consuming a Amazon DocumentDB event with Lambda using Java.

```
import java.util.List;
import java.util.Map;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;

public class Example implements RequestHandler<Map<String, Object>, String> {

 @SuppressWarnings("unchecked")
 @Override
 public String handleRequest(Map<String, Object> event, Context context) {
 List<Map<String, Object>> events = (List<Map<String, Object>>)
event.get("events");
 for (Map<String, Object> record : events) {
 Map<String, Object> eventData = (Map<String, Object>)
record.get("event");
 processEventData(eventData);
 }

 return "OK";
 }

 @SuppressWarnings("unchecked")
 private void processEventData(Map<String, Object> eventData) {
 String operationType = (String) eventData.get("operationType");
 System.out.println("operationType: %s".formatted(operationType));

 Map<String, Object> ns = (Map<String, Object>) eventData.get("ns");

 String db = (String) ns.get("db");
 System.out.println("db: %s".formatted(db));
 String coll = (String) ns.get("coll");
 System.out.println("coll: %s".formatted(coll));

 Map<String, Object> fullDocument = (Map<String, Object>)
eventData.get("fullDocument");
 System.out.println("fullDocument: %s".formatted(fullDocument));
 }
}
```

# DynamoDB examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with DynamoDB.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

*AWS community contributions* are examples that were created and are maintained by multiple teams across AWS. To provide feedback, use the mechanism provided in the linked repositories.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)
- [AWS community contributions](#)

## Get started

### Hello DynamoDB

The following code example shows how to get started using DynamoDB.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;
import software.amazon.awssdk.services.dynamodb.model.ListTablesResponse;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListTables {
 public static void main(String[] args) {
 System.out.println("Listing your Amazon DynamoDB tables:\n");
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();
 listAllTables(ddb);
 ddb.close();
 }

 public static void listAllTables(DynamoDbClient ddb) {
 boolean moreTables = true;
 String lastName = null;

 while (moreTables) {
 try {
 ListTablesResponse response = null;
 if (lastName == null) {
```

```

 ListTablesRequest request = ListTablesRequest.builder().build();
 response = ddb.listTables(request);
 } else {
 ListTablesRequest request = ListTablesRequest.builder()
 .exclusiveStartTableName(lastName).build();
 response = ddb.listTables(request);
 }

 List<String> tableNames = response.tableNames();
 if (tableNames.size() > 0) {
 for (String curName : tableNames) {
 System.out.format("* %s\n", curName);
 }
 } else {
 System.out.println("No tables found!");
 System.exit(0);
 }

 lastName = response.lastEvaluatedTableName();
 if (lastName == null) {
 moreTables = false;
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
System.out.println("\nDone!");
}
}

```

- For API details, see [ListTables](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a table that can hold movie data.

- Put, get, and update a single movie in the table.
- Write movie data to the table from a sample JSON file.
- Query for movies that were released in a given year.
- Scan for movies that were released in a range of years.
- Delete a movie from the table, then delete the table.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

### Create a DynamoDB table.

```
// Create a table with a Sort key.
public static void createTable(DynamoDbClient ddb, String tableName) {
 DynamoDbWaiter dbWaiter = ddb.waiter();
 ArrayList<AttributeDefinition> attributeDefinitions = new ArrayList<>();

 // Define attributes.
 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("year")
 .attributeType("N")
 .build());

 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("title")
 .attributeType("S")
 .build());

 ArrayList<KeySchemaElement> tableKey = new ArrayList<>();
 KeySchemaElement key = KeySchemaElement.builder()
 .attributeName("year")
 .keyType(KeyType.HASH)
 .build();

 KeySchemaElement key2 = KeySchemaElement.builder()
 .attributeName("title")
```

```

 .keyType(KeyType.RANGE)
 .build();

// Add KeySchemaElement objects to the list.
tableKey.add(key);
tableKey.add(key2);

CreateTableRequest request = CreateTableRequest.builder()
 .keySchema(tableKey)
 .billingMode(BillingMode.PAY_PER_REQUEST) // DynamoDB automatically
scales based on traffic.
 .attributeDefinitions(attributeDefinitions)
 .tableName(tableName)
 .build();

try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created.
 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 String newTable = response.tableDescription().tableName();
 System.out.println("The " + newTable + " was successfully created.");

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
}

```

Create a helper function to download and extract the sample JSON file.

```

// Load data into the table.
public static void loadData(DynamoDbClient ddb, String tableName, String
fileName) throws IOException {
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(ddb)
 .build();

```



```

 DynamoDbTable<Movies> mappedTable = enhancedClient.table("Movies",
TableSchema.fromBean(Movies.class));
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 ObjectNode currentNode;
 int t = 0;
 while (iter.hasNext()) {
 // Only add 200 Movies to the table.
 if (t == 200)
 break;
 currentNode = (ObjectNode) iter.next();

 int year = currentNode.path("year").asInt();
 String title = currentNode.path("title").asText();
 String info = currentNode.path("info").toString();

 Movies movies = new Movies();
 movies.setYear(year);
 movies.setTitle(title);
 movies.setInfo(info);

 // Put the data into the Amazon DynamoDB Movie table.
 mappedTable.putItem(movies);
 t++;
 }
 }
}

```

### Get an item from a table.

```

public static void getItem(DynamoDbClient ddb) {

 HashMap<String, AttributeValue> keyToGet = new HashMap<>();
 keyToGet.put("year", AttributeValue.builder()
 .n("1933")
 .build());

 keyToGet.put("title", AttributeValue.builder()
 .s("King Kong")
 .build());
}

```

```

 GetItemRequest request = GetItemRequest.builder()
 .key(keyToGet)
 .tableName("Movies")
 .build();

 try {
 Map<String, AttributeValue> returnedItem = ddb.getItem(request).item();

 if (returnedItem != null) {
 Set<String> keys = returnedItem.keySet();
 System.out.println("Amazon DynamoDB table attributes: \n");

 for (String key1 : keys) {
 System.out.format("%s: %s\n", key1,
returnedItem.get(key1).toString());
 }
 } else {
 System.out.format("No item found with the key %s!\n", "year");
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

### Full example.

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 * <p>
 * This Java example performs these tasks:
 * <p>
 * 1. Creates the Amazon DynamoDB Movie table with partition and sort key.
 * 2. Puts data into the Amazon DynamoDB table from a JSON document using the

```

```

* Enhanced client.
* 3. Gets data from the Movie table.
* 4. Adds a new item.
* 5. Updates an item.
* 6. Uses a Scan to query items using the Enhanced client.
* 7. Queries all items where the year is 2013 using the Enhanced Client.
* 8. Deletes the table.
*/

```

```

public class Scenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws IOException {
 String tableName = "Movies";
 String fileName = "../../resources/sample_files/movies.json";
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon DynamoDB example scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(
 "1. Creating an Amazon DynamoDB table named Movies with a key named year
and a sort key named title.");
 createTable(ddb, tableName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Loading data into the Amazon DynamoDB table.");
 loadData(ddb, tableName, fileName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Getting data from the Movie table.");
 getItem(ddb);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. Putting a record into the Amazon DynamoDB table.");
 putRecord(ddb);
 }
}

```

```
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Updating a record.");
 updateTableItem(ddb, tableName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Scanning the Amazon DynamoDB table.");
 scanMovies(ddb, tableName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Querying the Movies released in 2013.");
 queryTable(ddb);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Deleting the Amazon DynamoDB table.");
 deleteDynamoDBTable(ddb, tableName);
 System.out.println(DASHES);

 ddb.close();
 }

 // Create a table with a Sort key.
 public static void createTable(DynamoDbClient ddb, String tableName) {
 DynamoDbWaiter dbWaiter = ddb.waiter();
 ArrayList<AttributeDefinition> attributeDefinitions = new ArrayList<>();

 // Define attributes.
 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("year")
 .attributeType("N")
 .build());

 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("title")
 .attributeType("S")
 .build());

 ArrayList<KeySchemaElement> tableKey = new ArrayList<>();
 KeySchemaElement key = KeySchemaElement.builder()
 .attributeName("year")
```

```

 .keyType(KeyType.HASH)
 .build();

 KeySchemaElement key2 = KeySchemaElement.builder()
 .attributeName("title")
 .keyType(KeyType.RANGE)
 .build();

 // Add KeySchemaElement objects to the list.
 tableKey.add(key);
 tableKey.add(key2);

 CreateTableRequest request = CreateTableRequest.builder()
 .keySchema(tableKey)
 .billingMode(BillingMode.PAY_PER_REQUEST) // DynamoDB automatically
scales based on traffic.
 .attributeDefinitions(attributeDefinitions)
 .tableName(tableName)
 .build();

 try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created.
 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 String newTable = response.tableDescription().tableName();
 System.out.println("The " + newTable + " was successfully created.");

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

// Query the table.
public static void queryTable(DynamoDbClient ddb) {
 try {
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(ddb)

```

```

 .build();

 DynamoDbTable<Movies> custTable = enhancedClient.table("Movies",
TableSchema.fromBean(Movies.class));
 QueryConditional queryConditional = QueryConditional
 .keyEqualTo(Key.builder()
 .partitionValue(2013)
 .build());

 // Get items in the table and write out the ID value.
 Iterator<Movies> results =
custTable.query(queryConditional).items().iterator();
 String result = "";

 while (results.hasNext()) {
 Movies rec = results.next();
 System.out.println("The title of the movie is " + rec.getTitle());
 System.out.println("The movie information is " + rec.getInfo());
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

// Scan the table.
public static void scanMovies(DynamoDbClient ddb, String tableName) {
 System.out.println("***** Scanning all movies.\n");
 try {
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(ddb)
 .build();

 DynamoDbTable<Movies> custTable = enhancedClient.table("Movies",
TableSchema.fromBean(Movies.class));
 Iterator<Movies> results = custTable.scan().items().iterator();
 while (results.hasNext()) {
 Movies rec = results.next();
 System.out.println("The movie title is " + rec.getTitle());
 System.out.println("The movie year is " + rec.getYear());
 }

 } catch (DynamoDbException e) {

```

```

 System.err.println(e.getMessage());
 System.exit(1);
 }
}

// Load data into the table.
public static void loadData(DynamoDbClient ddb, String tableName, String
fileName) throws IOException {
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(ddb)
 .build();

 DynamoDbTable<Movies> mappedTable = enhancedClient.table("Movies",
TableSchema.fromBean(Movies.class));
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 ObjectNode currentNode;
 int t = 0;
 while (iter.hasNext()) {
 // Only add 200 Movies to the table.
 if (t == 200)
 break;
 currentNode = (ObjectNode) iter.next();

 int year = currentNode.path("year").asInt();
 String title = currentNode.path("title").asText();
 String info = currentNode.path("info").toString();

 Movies movies = new Movies();
 movies.setYear(year);
 movies.setTitle(title);
 movies.setInfo(info);

 // Put the data into the Amazon DynamoDB Movie table.
 mappedTable.putItem(movies);
 t++;
 }
}

// Update the record to include show only directors.
public static void updateTableItem(DynamoDbClient ddb, String tableName) {
 HashMap<String, AttributeValue> itemKey = new HashMap<>();

```

```

 itemKey.put("year", AttributeValue.builder().n("1933").build());
 itemKey.put("title", AttributeValue.builder().s("King Kong").build());

 HashMap<String, AttributeValueUpdate> updatedValues = new HashMap<>();
 updatedValues.put("info", AttributeValueUpdate.builder()
 .value(AttributeValue.builder().s("{\\"directors\\":[\\"Merian C. Cooper\\",
\\"Ernest B. Schoedsack\\"]}").build())
 .action(AttributeAction.PUT)
 .build());

 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(itemKey)
 .attributeUpdates(updatedValues)
 .build();

 try {
 ddb.updateItem(request);
 } catch (ResourceNotFoundException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }

 System.out.println("Item was updated!");
 }

 public static void deleteDynamoDBTable(DynamoDbClient ddb, String tableName) {
 DeleteTableRequest request = DeleteTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 ddb.deleteTable(request);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println(tableName + " was successfully deleted!");
 }
}

```



```
public static void putRecord(DynamoDbClient ddb) {
 try {
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(ddb)
 .build();

 DynamoDbTable<Movies> table = enhancedClient.table("Movies",
TableSchema.fromBean(Movies.class));

 // Populate the Table.
 Movies record = new Movies();
 record.setYear(2020);
 record.setTitle("My Movie2");
 record.setInfo("no info");
 table.putItem(record);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("Added a new movie to the table.");
}

public static void getItem(DynamoDbClient ddb) {

 HashMap<String, AttributeValue> keyToGet = new HashMap<>();
 keyToGet.put("year", AttributeValue.builder()
 .n("1933")
 .build());

 keyToGet.put("title", AttributeValue.builder()
 .s("King Kong")
 .build());

 GetItemRequest request = GetItemRequest.builder()
 .key(keyToGet)
 .tableName("Movies")
 .build();

 try {
 Map<String, AttributeValue> returnedItem = ddb.getItem(request).item();

 if (returnedItem != null) {
```

```
 Set<String> keys = returnedItem.keySet();
 System.out.println("Amazon DynamoDB table attributes: \n");

 for (String key1 : keys) {
 System.out.format("%s: %s\n", key1,
returnedItem.get(key1).toString());
 }
 } else {
 System.out.format("No item found with the key %s!\n", "year");
 }

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [BatchWriteItem](#)
- [CreateTable](#)
- [DeleteItem](#)
- [DeleteTable](#)
- [DescribeTable](#)
- [GetItem](#)
- [PutItem](#)
- [Query](#)
- [Scan](#)
- [UpdateItem](#)

## Actions

### BatchGetItem

The following code example shows how to use BatchGetItem.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Shows how to get batch items using the service client.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.BatchGetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.BatchGetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.KeysAndAttributes;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class BatchReadItems {
 public static void main(String[] args){
 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table (for example, Music).\s
 """;

 String tableName = "Music";
 Region region = Region.US_EAST_1;
 DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
 .region(region)
```

```

 .build();

 getBatchItems(dynamoDbClient, tableName);
}

public static void getBatchItems(DynamoDbClient dynamoDbClient, String
tableName) {
 // Define the primary key values for the items you want to retrieve.
 Map<String, AttributeValue> key1 = new HashMap<>();
 key1.put("Artist", AttributeValue.builder().s("Artist1").build());

 Map<String, AttributeValue> key2 = new HashMap<>();
 key2.put("Artist", AttributeValue.builder().s("Artist2").build());

 // Construct the batchGetItem request.
 Map<String, KeysAndAttributes> requestItems = new HashMap<>();
 requestItems.put(tableName, KeysAndAttributes.builder()
 .keys(List.of(key1, key2))
 .projectionExpression("Artist, SongTitle")
 .build());

 BatchGetItemRequest batchGetItemRequest = BatchGetItemRequest.builder()
 .requestItems(requestItems)
 .build();

 // Make the batchGetItem request.
 BatchGetItemResponse batchGetItemResponse =
dynamoDbClient.batchGetItem(batchGetItemRequest);

 // Extract and print the retrieved items.
 Map<String, List<Map<String, AttributeValue>>> responses =
batchGetItemResponse.responses();
 if (responses.containsKey(tableName)) {
 List<Map<String, AttributeValue>> musicItems = responses.get(tableName);
 for (Map<String, AttributeValue> item : musicItems) {
 System.out.println("Artist: " + item.get("Artist").s() +
 ", SongTitle: " + item.get("SongTitle").s());
 }
 } else {
 System.out.println("No items retrieved.");
 }
}
}
}

```

Shows how to get batch items using the service client and a paginator.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.BatchGetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.KeysAndAttributes;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class BatchGetItemsPaginator {

 public static void main(String[] args){
 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table (for example, Music).\s
 "";

 String tableName = "Music";
 Region region = Region.US_EAST_1;
 DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
 .region(region)
 .build();

 getBatchItemsPaginator(dynamoDbClient, tableName) ;
 }

 public static void getBatchItemsPaginator(DynamoDbClient dynamoDbClient, String
tableName) {
 // Define the primary key values for the items you want to retrieve.
 Map<String, AttributeValue> key1 = new HashMap<>();
 key1.put("Artist", AttributeValue.builder().s("Artist1").build());

 Map<String, AttributeValue> key2 = new HashMap<>();
 key2.put("Artist", AttributeValue.builder().s("Artist2").build());
```

```

// Construct the batchGetItem request.
Map<String, KeysAndAttributes> requestItems = new HashMap<>();
requestItems.put(tableName, KeysAndAttributes.builder()
 .keys(List.of(key1, key2))
 .projectionExpression("Artist, SongTitle")
 .build());

BatchGetItemRequest batchGetItemRequest = BatchGetItemRequest.builder()
 .requestItems(requestItems)
 .build();

// Use batchGetItemPaginator for paginated requests.
dynamoDbClient.batchGetItemPaginator(batchGetItemRequest).stream()
 .flatMap(response -> response.responses().getOrDefault(tableName,
Collections.emptyList()).stream())
 .forEach(item -> {
 System.out.println("Artist: " + item.get("Artist").s() +
 ", SongTitle: " + item.get("SongTitle").s());
 });
}
}

```

- For API details, see [BatchGetItem](#) in *AWS SDK for Java 2.x API Reference*.

## BatchWriteItem

The following code example shows how to use BatchWriteItem.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Inserts many items into a table by using the service client.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;

```

```

import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.BatchWriteItemRequest;
import software.amazon.awssdk.services.dynamodb.model.BatchWriteItemResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.PutRequest;
import software.amazon.awssdk.services.dynamodb.model.WriteRequest;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class BatchWriteItems {
 public static void main(String[] args){
 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table (for example, Music).\s
 """;

 String tableName = "Music";
 Region region = Region.US_EAST_1;
 DynamoDbClient dynamoDbClient = DynamoDbClient.builder()
 .region(region)
 .build();

 addBatchItems(dynamoDbClient, tableName);
 }

 public static void addBatchItems(DynamoDbClient dynamoDbClient, String
tableName) {
 // Specify the updates you want to perform.
 List<WriteRequest> writeRequests = new ArrayList<>();

```

```

 // Set item 1.
 Map<String, AttributeValue> item1Attributes = new HashMap<>();
 item1Attributes.put("Artist",
 AttributeValue.builder().s("Artist1").build());
 item1Attributes.put("Rating", AttributeValue.builder().s("5").build());
 item1Attributes.put("Comments", AttributeValue.builder().s("Great
song!").build());
 item1Attributes.put("SongTitle",
 AttributeValue.builder().s("SongTitle1").build());

writeRequests.add(WriteRequest.builder().putRequest(PutRequest.builder().item(item1Attribut

 // Set item 2.
 Map<String, AttributeValue> item2Attributes = new HashMap<>();
 item2Attributes.put("Artist",
 AttributeValue.builder().s("Artist2").build());
 item2Attributes.put("Rating", AttributeValue.builder().s("4").build());
 item2Attributes.put("Comments", AttributeValue.builder().s("Nice
melody.").build());
 item2Attributes.put("SongTitle",
 AttributeValue.builder().s("SongTitle2").build());

writeRequests.add(WriteRequest.builder().putRequest(PutRequest.builder().item(item2Attribut

 try {
 // Create the BatchWriteItemRequest.
 BatchWriteItemRequest batchWriteItemRequest =
 BatchWriteItemRequest.builder()
 .requestItems(Map.of(tableName, writeRequests))
 .build();

 // Execute the BatchWriteItem operation.
 BatchWriteItemResponse batchWriteItemResponse =
 dynamoDbClient.batchWriteItem(batchWriteItemRequest);

 // Process the response.
 System.out.println("Batch write successful: " + batchWriteItemResponse);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```



Inserts many items into a table by using the enhanced client.

```
import com.example.dynamodb.Customer;
import com.example.dynamodb.Music;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.Key;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
import software.amazon.awssdk.enhanced.dynamodb.model.BatchWriteItemEnhancedRequest;
import software.amazon.awssdk.enhanced.dynamodb.model.WriteBatch;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneOffset;

/*
 * Before running this code example, create an Amazon DynamoDB table named Customer
 * with these columns:
 * - id - the id of the record that is the key
 * - custName - the customer name
 * - email - the email value
 * - registrationDate - an instant value when the item was added to the table
 *
 * Also, ensure that you have set up your development environment, including your
 * credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class EnhancedBatchWriteItems {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();
 DynamoDbEnhancedClient enhancedClient =
 DynamoDbEnhancedClient.builder()
```

```

 .dynamoDbClient(ddb)
 .build();
 putBatchRecords(enhancedClient);
 ddb.close();
}

public static void putBatchRecords(DynamoDbEnhancedClient enhancedClient) {
 try {
 DynamoDbTable<Customer> customerMappedTable =
enhancedClient.table("Customer",
 TableSchema.fromBean(Customer.class));
 DynamoDbTable<Music> musicMappedTable =
enhancedClient.table("Music",
 TableSchema.fromBean(Music.class));
 LocalDate localDate = LocalDate.parse("2020-04-07");
 LocalDateTime localDateTime = localDate.atStartOfDay();
 Instant instant = localDateTime.toInstant(ZoneOffset.UTC);

 Customer record2 = new Customer();
 record2.setCustName("Fred Pink");
 record2.setId("id110");
 record2.setEmail("fredp@noserver.com");
 record2.setRegistrationDate(instant);

 Customer record3 = new Customer();
 record3.setCustName("Susan Pink");
 record3.setId("id120");
 record3.setEmail("spink@noserver.com");
 record3.setRegistrationDate(instant);

 Customer record4 = new Customer();
 record4.setCustName("Jerry orange");
 record4.setId("id101");
 record4.setEmail("jorange@noserver.com");
 record4.setRegistrationDate(instant);

 BatchWriteItemEnhancedRequest batchWriteItemEnhancedRequest
= BatchWriteItemEnhancedRequest
 .builder()
 .writeBatches(
WriteBatch.builder(Customer.class) // add items to the Customer

// table

```

```

 .mappedTableResource(customerMappedTable)

 .addPutItem(builder -> builder.item(record2))

 .addPutItem(builder -> builder.item(record3))

 .addPutItem(builder -> builder.item(record4))

 .build(),

WriteBatch.builder(Music.class) // delete an item from the Music

 // table

 .mappedTableResource(musicMappedTable)

 .addDeleteItem(builder -> builder.key(

 Key.builder().partitionValue(

 "Famous Band")

 .build()))

 .build())

 .build();

 // Add three items to the Customer table and delete one item
from the Music
 // table.

enhancedClient.batchWriteItem(batchWriteItemEnhancedRequest);
 System.out.println("done");

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [BatchWriteItem](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTable

The following code example shows how to use CreateTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.BillingMode;
import software.amazon.awssdk.services.dynamodb.model.CreateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.CreateTableResponse;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.OnDemandThroughput;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.waiters.DynamoDbWaiter;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateTable {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
```

```

 <tableName> <key>

 Where:
 tableName - The Amazon DynamoDB table to create (for example,
Music3).
 key - The key for the Amazon DynamoDB table (for example, Artist).
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String key = args[1];
 System.out.println("Creating an Amazon DynamoDB table " + tableName + " with
a simple primary key: " + key);
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 String result = createTable(ddb, tableName, key);
 System.out.println("New table is " + result);
 ddb.close();
 }

 public static String createTable(DynamoDbClient ddb, String tableName, String
key) {
 DynamoDbWaiter dbWaiter = ddb.waiter();
 CreateTableRequest request = CreateTableRequest.builder()
 .attributeDefinitions(AttributeDefinition.builder()
 .attributeName(key)
 .attributeType(ScalarAttributeType.S)
 .build())
 .keySchema(KeySchemaElement.builder()
 .attributeName(key)
 .keyType(KeyType.HASH)
 .build())
 .billingMode(BillingMode.PAY_PER_REQUEST) // DynamoDB automatically
scales based on traffic.
 .tableName(tableName)
 .build();

```

```

String newTable;
try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created.
 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 newTable = response.tableDescription().tableName();
 return newTable;

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
return "";
}
}

```

- For API details, see [CreateTable](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteItem

The following code example shows how to use DeleteItem.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DeleteItemRequest;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;

```

```

import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteItem {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName> <key> <keyval>

 Where:
 tableName - The Amazon DynamoDB table to delete the item from
(for example, Music3).
 key - The key used in the Amazon DynamoDB table (for example,
Artist).\s
 keyval - The key value that represents the item to delete (for
example, Famous Band).
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String key = args[1];
 String keyVal = args[2];
 System.out.format("Deleting item \"%s\" from %s\n", keyVal, tableName);
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 deleteDynamoDBItem(ddb, tableName, key, keyVal);
 ddb.close();
 }
}

```

```

 public static void deleteDynamoDBItem(DynamoDbClient ddb, String tableName,
String key, String keyVal) {
 HashMap<String, AttributeValue> keyToGet = new HashMap<>();
 keyToGet.put(key, AttributeValue.builder()
 .s(keyVal)
 .build());

 DeleteItemRequest deleteReq = DeleteItemRequest.builder()
 .tableName(tableName)
 .key(keyToGet)
 .build();

 try {
 ddb.deleteItem(deleteReq);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DeleteItem](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteTable

The following code example shows how to use DeleteTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DeleteTableRequest;

/**

```



```

* Before running this Java V2 code example, set up your development
* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/

```

```

public class DeleteTable {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table to delete (for example,
Music3).

 Warning This program will delete the table that you specify!
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 System.out.format("Deleting the Amazon DynamoDB table %s...\n", tableName);
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 deleteDynamoDBTable(ddb, tableName);
 ddb.close();
 }

 public static void deleteDynamoDBTable(DynamoDbClient ddb, String tableName) {
 DeleteTableRequest request = DeleteTableRequest.builder()
 .tableName(tableName)
 .build();

 try {

```

```

 ddb.deleteTable(request);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println(tableName + " was successfully deleted!");
}
}

```

- For API details, see [DeleteTable](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeTable

The following code example shows how to use DescribeTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableRequest;
import
 software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughputDescription;
import software.amazon.awssdk.services.dynamodb.model.TableDescription;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

```

```

*/
public class DescribeTable {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table to get information about
(for example, Music3).
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 System.out.format("Getting description for %s\n\n", tableName);
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 describeDynamoDBTable(ddb, tableName);
 ddb.close();
 }

 public static void describeDynamoDBTable(DynamoDbClient ddb, String tableName) {
 DescribeTableRequest request = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 TableDescription tableInfo = ddb.describeTable(request).table();
 if (tableInfo != null) {
 System.out.format("Table name : %s\n", tableInfo.tableName());
 System.out.format("Table ARN : %s\n", tableInfo.tableArn());
 System.out.format("Status : %s\n", tableInfo.tableStatus());
 System.out.format("Item count : %d\n", tableInfo.itemCount());
 System.out.format("Size (bytes): %d\n", tableInfo.tableSizeBytes());
 }
 }
 }
}

```

```

 ProvisionedThroughputDescription throughputInfo =
tableInfo.provisionedThroughput();
 System.out.println("Throughput");
 System.out.format(" Read Capacity : %d\n",
throughputInfo.readCapacityUnits());
 System.out.format(" Write Capacity: %d\n",
throughputInfo.writeCapacityUnits());

 List<AttributeDefinition> attributes =
tableInfo.attributeDefinitions();
 System.out.println("Attributes");
 for (AttributeDefinition a : attributes) {
 System.out.format(" %s (%s)\n", a.attributeName(),
a.attributeType());
 }
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("\nDone!");
}
}

```

- For API details, see [DescribeTable](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeTimeToLive

The following code example shows how to use `DescribeTimeToLive`.

### SDK for Java 2.x

Describe TTL configuration on an existing DynamoDB table using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DescribeTimeToLiveRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTimeToLiveResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

```

```
import java.util.logging.Level;
import java.util.logging.Logger;

 public DescribeTimeToLiveResponse describeTTL(final String tableName, final
Region region) {
 final DescribeTimeToLiveRequest request =
 DescribeTimeToLiveRequest.builder().tableName(tableName).build();

 try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient
 : DynamoDbClient.builder().region(region).build()) {
 return ddb.describeTimeToLive(request);
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }
}
```

- For API details, see [DescribeTimeToLive](#) in *AWS SDK for Java 2.x API Reference*.

## GetItem

The following code example shows how to use `GetItem`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Gets an item from a table by using the `DynamoDbClient`.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
```

```

import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To get an item from an Amazon DynamoDB table using the AWS SDK for Java V2,
 * its better practice to use the
 * Enhanced Client, see the EnhancedGetItem example.
 */
public class GetItem {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName> <key> <keyVal>

 Where:
 tableName - The Amazon DynamoDB table from which an item is
retrieved (for example, Music3).\s
 key - The key used in the Amazon DynamoDB table (for example,
Artist).\s
 keyval - The key value that represents the item to get (for
example, Famous Band).
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String key = args[1];
 String keyVal = args[2];
 System.out.format("Retrieving item \"%s\" from \"%s\"\\n", keyVal,
tableName);
 Region region = Region.US_EAST_1;

```

```

 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 getDynamoDBItem(ddb, tableName, key, keyVal);
 ddb.close();
 }

 public static void getDynamoDBItem(DynamoDbClient ddb, String tableName, String
key, String keyVal) {
 HashMap<String, AttributeValue> keyToGet = new HashMap<>();
 keyToGet.put(key, AttributeValue.builder()
 .s(keyVal)
 .build());

 GetItemRequest request = GetItemRequest.builder()
 .key(keyToGet)
 .tableName(tableName)
 .build();

 try {
 // If there is no matching item, GetItem does not return any data.
 Map<String, AttributeValue> returnedItem = ddb.getItem(request).item();
 if (returnedItem.isEmpty())
 System.out.format("No item found with the key %s!\n", key);
 else {
 Set<String> keys = returnedItem.keySet();
 System.out.println("Amazon DynamoDB table attributes: \n");
 for (String key1 : keys) {
 System.out.format("%s: %s\n", key1,
returnedItem.get(key1).toString());
 }
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [GetItem](#) in *AWS SDK for Java 2.x API Reference*.

## ListTables

The following code example shows how to use ListTables.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ListTablesRequest;
import software.amazon.awssdk.services.dynamodb.model.ListTablesResponse;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListTables {
 public static void main(String[] args) {
 System.out.println("Listing your Amazon DynamoDB tables:\n");
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();
 listAllTables(ddb);
 ddb.close();
 }

 public static void listAllTables(DynamoDbClient ddb) {
 boolean moreTables = true;
 String lastName = null;

 while (moreTables) {
```



```

 try {
 ListTablesResponse response = null;
 if (lastName == null) {
 ListTablesRequest request = ListTablesRequest.builder().build();
 response = ddb.listTables(request);
 } else {
 ListTablesRequest request = ListTablesRequest.builder()
 .exclusiveStartTableName(lastName).build();
 response = ddb.listTables(request);
 }

 List<String> tableNames = response.tableNames();
 if (tableNames.size() > 0) {
 for (String curName : tableNames) {
 System.out.format("* %s\n", curName);
 }
 } else {
 System.out.println("No tables found!");
 System.exit(0);
 }

 lastName = response.lastEvaluatedTableName();
 if (lastName == null) {
 moreTables = false;
 }

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
 System.out.println("\nDone!");
}

```

- For API details, see [ListTables](#) in *AWS SDK for Java 2.x API Reference*.

## PutItem

The following code example shows how to use PutItem.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Puts an item into a table using [DynamoDbClient](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import software.amazon.awssdk.services.dynamodb.model.PutItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To place items into an Amazon DynamoDB table using the AWS SDK for Java V2,
 * its better practice to use the
 * Enhanced Client. See the EnhancedPutItem example.
 */
public class PutItem {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName> <key> <keyVal> <albumtitle> <albumtitleval> <awards>
<awardsval> <Songtitle> <songtitleval>

 Where:
 tableName - The Amazon DynamoDB table in which an item is placed
(for example, Music3).
```

```

 key - The key used in the Amazon DynamoDB table (for example,
Artist).
 keyval - The key value that represents the item to get (for
example, Famous Band).
 albumTitle - The Album title (for example, AlbumTitle).
 AlbumTitleValue - The name of the album (for example, Songs
About Life).
 Awards - The awards column (for example, Awards).
 AwardVal - The value of the awards (for example, 10).
 SongTitle - The song title (for example, SongTitle).
 SongTitleVal - The value of the song title (for example, Happy
Day).

 Warning This program will place an item that you specify into a
table!

 """;

 if (args.length != 9) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String key = args[1];
 String keyVal = args[2];
 String albumTitle = args[3];
 String albumTitleValue = args[4];
 String awards = args[5];
 String awardVal = args[6];
 String songTitle = args[7];
 String songTitleVal = args[8];

 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 putItemInTable(ddb, tableName, key, keyVal, albumTitle, albumTitleValue,
awards, awardVal, songTitle,
 songTitleVal);
 System.out.println("Done!");
 ddb.close();
}

public static void putItemInTable(DynamoDbClient ddb,

```

```

 String tableName,
 String key,
 String keyVal,
 String albumTitle,
 String albumTitleValue,
 String awards,
 String awardVal,
 String songTitle,
 String songTitleVal) {

 HashMap<String, AttributeValue> itemValues = new HashMap<>();
 itemValues.put(key, AttributeValue.builder().s(keyVal).build());
 itemValues.put(songTitle, AttributeValue.builder().s(songTitleVal).build());
 itemValues.put(albumTitle,
AttributeValue.builder().s(albumTitleValue).build());
 itemValues.put(awards, AttributeValue.builder().s(awardVal).build());

 PutItemRequest request = PutItemRequest.builder()
 .tableName(tableName)
 .item(itemValues)
 .build();

 try {
 PutItemResponse response = ddb.putItem(request);
 System.out.println(tableName + " was successfully updated. The request
id is "
 + response.responseMetadata().requestId());

 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 System.err.println("Be sure that it exists and that you've typed its
name correctly!");
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [PutItem](#) in *AWS SDK for Java 2.x API Reference*.

## Query

The following code example shows how to use Query.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Queries a table by using [DynamoDbClient](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To query items from an Amazon DynamoDB table using the AWS SDK for Java V2,
 * its better practice to use the
 * Enhanced Client. See the EnhancedQueryRecords example.
 */
public class Query {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName> <partitionKeyName> <partitionKeyVal>

 Where:
```

```

 tableName - The Amazon DynamoDB table to put the item in (for
example, Music3).
 partitionKeyName - The partition key name of the Amazon DynamoDB
table (for example, Artist).
 partitionKeyVal - The value of the partition key that should
match (for example, Famous Band).
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String partitionKeyName = args[1];
 String partitionKeyVal = args[2];

 // For more information about an alias, see:
 // https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/
Expressions.ExpressionAttributeNames.html
 String partitionAlias = "#a";

 System.out.format("Querying %s", tableName);
 System.out.println("");
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 int count = queryTable(ddb, tableName, partitionKeyName, partitionKeyVal,
partitionAlias);
 System.out.println("There were " + count + " record(s) returned");
 ddb.close();
}

public static int queryTable(DynamoDbClient ddb, String tableName, String
partitionKeyName, String partitionKeyVal,
 String partitionAlias) {
 // Set up an alias for the partition key name in case it's a reserved word.
 HashMap<String, String> attrNameAlias = new HashMap<String, String>();
 attrNameAlias.put(partitionAlias, partitionKeyName);

 // Set up mapping of the partition name with the value.
 HashMap<String, AttributeValue> attrValues = new HashMap<>();

```

```

 attrValues.put(":" + partitionKeyName, AttributeValue.builder()
 .s(partitionKeyVal)
 .build());

 QueryRequest queryReq = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(partitionAlias + " = :" + partitionKeyName)
 .expressionAttributeNames(attrNameAlias)
 .expressionAttributeValues(attrValues)
 .build();

 try {
 QueryResponse response = ddb.query(queryReq);
 return response.count();
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return -1;
 }
}

```

Queries a table by using `DynamoDbClient` and a secondary index.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import java.util.HashMap;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 */

```

```

* Create the Movies table by running the Scenario example and loading the Movie
* data from the JSON file. Next create a secondary
* index for the Movies table that uses only the year column. Name the index
* **year-index**. For more information, see:
*
* https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html
*/
public class QueryItemsUsingIndex {
 public static void main(String[] args) {
 String tableName = "Movies";
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 queryIndex(ddb, tableName);
 ddb.close();
 }

 public static void queryIndex(DynamoDbClient ddb, String tableName) {
 try {
 Map<String, String> expressionAttributesNames = new HashMap<>();
 expressionAttributesNames.put("#year", "year");
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 expressionAttributeValues.put(":yearValue",
 AttributeValue.builder().n("2013").build());

 QueryRequest request = QueryRequest.builder()
 .tableName(tableName)
 .indexName("year-index")
 .keyConditionExpression("#year = :yearValue")
 .expressionAttributeNames(expressionAttributesNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 System.out.println("=== Movie Titles ===");
 QueryResponse response = ddb.query(request);
 response.items()
 .forEach(movie -> System.out.println(movie.get("title").s()));

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```



```
}
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Scan

The following code example shows how to use Scan.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Scans an Amazon DynamoDB table using [DynamoDbClient](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanResponse;
import java.util.Map;
import java.util.Set;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To scan items from an Amazon DynamoDB table using the AWS SDK for Java V2,
 * its better practice to use the
 * Enhanced Client, See the EnhancedScanRecords example.
 */
```

```
public class DynamoDBScanItems {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <tableName>

 Where:
 tableName - The Amazon DynamoDB table to get information from
(for example, Music3).
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 scanItems(ddb, tableName);
 ddb.close();
 }

 public static void scanItems(DynamoDbClient ddb, String tableName) {
 try {
 ScanRequest scanRequest = ScanRequest.builder()
 .tableName(tableName)
 .build();

 ScanResponse response = ddb.scan(scanRequest);
 for (Map<String, AttributeValue> item : response.items()) {
 Set<String> keys = item.keySet();
 for (String key : keys) {
 System.out.println("The key name is " + key + "\n");
 System.out.println("The value is " + item.get(key).s());
 }
 }
 } catch (DynamoDbException e) {
```

```

 e.printStackTrace();
 System.exit(1);
 }
}

```

- For API details, see [Scan](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateItem

The following code example shows how to use UpdateItem.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Updates an item in a table using [DynamoDbClient](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.AttributeAction;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.AttributeValueUpdate;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To update an Amazon DynamoDB table using the AWS SDK for Java V2, its better
 * practice to use the

```

```

 * Enhanced Client, See the EnhancedModifyItem example.
 */
public class UpdateItem {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <tableName> <key> <keyVal> <name> <updateVal>

 Where:
 tableName - The Amazon DynamoDB table (for example, Music3).
 key - The name of the key in the table (for example, Artist).
 keyVal - The value of the key (for example, Famous Band).
 name - The name of the column where the value is updated (for
example, Awards).
 updateVal - The value used to update an item (for example, 14).
 Example:
 UpdateItem Music3 Artist Famous Band Awards 14
 """;

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String tableName = args[0];
 String key = args[1];
 String keyVal = args[2];
 String name = args[3];
 String updateVal = args[4];

 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();
 updateTableItem(ddb, tableName, key, keyVal, name, updateVal);
 ddb.close();
 }

 public static void updateTableItem(DynamoDbClient ddb,
 String tableName,
 String key,
 String keyVal,
 String name,

```

```

 String updateVal) {

 HashMap<String, AttributeValue> itemKey = new HashMap<>();
 itemKey.put(key, AttributeValue.builder()
 .s(keyVal)
 .build());

 HashMap<String, AttributeValueUpdate> updatedValues = new HashMap<>();
 updatedValues.put(name, AttributeValueUpdate.builder()
 .value(AttributeValue.builder().s(updateVal).build())
 .action(AttributeAction.PUT)
 .build());

 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(itemKey)
 .attributeUpdates(updatedValues)
 .build();

 try {
 ddb.updateItem(request);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("The Amazon DynamoDB table was updated!");
}
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateTimeToLive

The following code example shows how to use `UpdateTimeToLive`.

### SDK for Java 2.x

Enable TTL on an existing DynamoDB table using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;

```

```

import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.TimeToLiveSpecification;
import software.amazon.awssdk.services.dynamodb.model.UpdateTimeToLiveRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateTimeToLiveResponse;

import java.util.logging.Level;
import java.util.logging.Logger;

 public UpdateTimeToLiveResponse enableTTL(final String tableName, final String
attributeName, final Region region) {
 final TimeToLiveSpecification ttlSpec = TimeToLiveSpecification.builder()
 .attributeName(attributeName)
 .enabled(true)
 .build();

 final UpdateTimeToLiveRequest request = UpdateTimeToLiveRequest.builder()
 .tableName(tableName)
 .timeToLiveSpecification(ttlSpec)
 .build();

 try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient
 : DynamoDbClient.builder().region(region).build()) {
 return ddb.updateTimeToLive(request);
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }
 }
}

```

## Disable TTL on an existing DynamoDB table using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.TimeToLiveSpecification;
import software.amazon.awssdk.services.dynamodb.model.UpdateTimeToLiveRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateTimeToLiveResponse;

```

```
import java.util.logging.Level;
import java.util.logging.Logger;

public UpdateTimeToLiveResponse disableTTL(
 final String tableName, final String attributeName, final Region region) {
 final TimeToLiveSpecification ttlSpec = TimeToLiveSpecification.builder()
 .attributeName(attributeName)
 .enabled(false)
 .build();

 final UpdateTimeToLiveRequest request = UpdateTimeToLiveRequest.builder()
 .tableName(tableName)
 .timeToLiveSpecification(ttlSpec)
 .build();

 try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient
 : DynamoDbClient.builder().region(region).build()) {
 return ddb.updateTimeToLive(request);
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }
}
```

- For API details, see [UpdateTimeToLive](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build an app to submit data to a DynamoDB table

The following code example shows how to build an application that submits data to an Amazon DynamoDB table and notifies you when a user updates the table.

## SDK for Java 2.x

Shows how to create a dynamic web application that submits data using the Amazon DynamoDB Java API and sends a text message using the Amazon Simple Notification Service Java API.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Amazon SNS

## Compare multiple values with a single attribute

The following code example shows how to compare multiple values with a single attribute in DynamoDB.

- Use the IN operator to compare multiple values with a single attribute.
- Compare the IN operator with multiple OR conditions.
- Understand the performance and expression complexity benefits of using IN.

## SDK for Java 2.x

Compare multiple values with a single attribute in DynamoDB using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanResponse;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
```



```

/**
 * Queries a table using the IN operator to compare multiple values with a
 * single attribute.
 *
 * <p>This method demonstrates how to use the IN operator in a filter expression
 * to match an attribute against multiple values.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param partitionKeyName The name of the partition key attribute
 * @param partitionKeyValue The value of the partition key to query
 * @param attributeName The name of the attribute to compare
 * @param valuesList List of values to compare against
 * @return The query response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static QueryResponse compareMultipleValues(
 DynamoDbClient dynamoDbClient,
 String tableName,
 String partitionKeyName,
 AttributeValue partitionKeyValue,
 String attributeName,
 List<AttributeValue> valuesList) {

 // Create expression attribute names
 Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#pkName", partitionKeyName);
 expressionAttributeNames.put("#attrName", attributeName);

 // Create expression attribute values
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 expressionAttributeValues.put(":pkValue", partitionKeyValue);

 // Add values for IN operator
 for (int i = 0; i < valuesList.size(); i++) {
 expressionAttributeValues.put(":val" + i, valuesList.get(i));
 }

 // Build the IN clause
 StringBuilder inClause = new StringBuilder();
 for (int i = 0; i < valuesList.size(); i++) {
 if (i > 0) {
 inClause.append(", ");
 }
 }

```

```

 inClause.append(":val").append(i);
 }

 // Define the query parameters
 QueryRequest request = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression("#pkName = :pkValue")
 .filterExpression("#attrName IN (" + inClause.toString() + ")")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 // Perform the query operation
 return dynamoDbClient.query(request);
}

/**
 * Queries a table using multiple OR conditions to compare multiple values with
 * a single attribute.
 *
 * <p>This method demonstrates the alternative approach to using the IN
 * operator,
 * by using multiple OR conditions.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param partitionKeyName The name of the partition key attribute
 * @param partitionKeyValue The value of the partition key to query
 * @param attributeName The name of the attribute to compare
 * @param valuesList List of values to compare against
 * @return The query response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static QueryResponse compareWithOrConditions(
 DynamoDbClient dynamoDbClient,
 String tableName,
 String partitionKeyName,
 AttributeValue partitionKeyValue,
 String attributeName,
 List<AttributeValue> valuesList) {

 // Create expression attribute names
 Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#pkName", partitionKeyName);

```

```

 expressionAttributeNames.put("#attrName", attributeName);

 // Create expression attribute values
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 expressionAttributeValues.put(":pkValue", partitionKeyValue);

 // Add values for OR conditions
 for (int i = 0; i < valuesList.size(); i++) {
 expressionAttributeValues.put(":val" + i, valuesList.get(i));
 }

 // Build the OR conditions
 StringBuilder orConditions = new StringBuilder();
 for (int i = 0; i < valuesList.size(); i++) {
 if (i > 0) {
 orConditions.append(" OR ");
 }
 orConditions.append("#attrName = :val").append(i);
 }

 // Define the query parameters
 QueryRequest request = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression("#pkName = :pkValue")
 .filterExpression(orConditions.toString())
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 // Perform the query operation
 return dynamoDbClient.query(request);
 }

 /**
 * Compares the performance of using the IN operator versus multiple OR
 * conditions.
 *
 * <p>This method demonstrates the performance difference between using the IN
 * operator
 * and using multiple OR conditions.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param partitionKeyName The name of the partition key attribute

```

```

 * @param partitionKeyValue The value of the partition key to query
 * @param attributeName The name of the attribute to compare
 * @param valuesList List of values to compare against
 * @return Map containing the performance comparison results
 */
 public static Map<String, Object> comparePerformance(
 DynamoDbClient dynamoDbClient,
 String tableName,
 String partitionKeyName,
 AttributeValue partitionKeyValue,
 String attributeName,
 List<AttributeValue> valuesList) {

 Map<String, Object> results = new HashMap<>();

 try {
 // Measure performance of IN operator
 long inStartTime = System.nanoTime();
 QueryResponse inResponse = compareMultipleValues(
 dynamoDbClient, tableName, partitionKeyName, partitionKeyValue,
 attributeName, valuesList);
 long inEndTime = System.nanoTime();
 long inDuration = inEndTime - inStartTime;

 // Measure performance of OR conditions
 long orStartTime = System.nanoTime();
 QueryResponse orResponse = compareWithOrConditions(
 dynamoDbClient, tableName, partitionKeyName, partitionKeyValue,
 attributeName, valuesList);
 long orEndTime = System.nanoTime();
 long orDuration = orEndTime - orStartTime;

 // Record results
 results.put("inOperatorDuration", inDuration);
 results.put("orConditionsDuration", orDuration);
 results.put("inOperatorItems", inResponse.count());
 results.put("orConditionsItems", orResponse.count());
 results.put("inOperatorExpression", "IN operator with " +
 valuesList.size() + " values");
 results.put("orConditionsExpression", valuesList.size() + " OR
 conditions");
 results.put("success", true);

 } catch (DynamoDbException e) {

```

```

 results.put("success", false);
 results.put("error", e.getMessage());
 }

 return results;
}

/**
 * Scans a table using the IN operator with a large number of values.
 *
 * <p>This method demonstrates how to use the IN operator with a large number of
values,
 * which can help stay within the 300 operator limit.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param attributeName The name of the attribute to compare
 * @param valuesList List of values to compare against
 * @return The scan response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static ScanResponse scanWithLargeInClause(
 DynamoDbClient dynamoDbClient, String tableName, String attributeName,
 List<AttributeValue> valuesList) {

 // Create expression attribute names
 Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#attrName", attributeName);

 // Create expression attribute values
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();

 // Add values for IN operator
 for (int i = 0; i < valuesList.size(); i++) {
 expressionAttributeValues.put(":val" + i, valuesList.get(i));
 }

 // Build the IN clause
 StringBuilder inClause = new StringBuilder();
 for (int i = 0; i < valuesList.size(); i++) {
 if (i > 0) {
 inClause.append(", ");
 }
 inClause.append(":val").append(i);
 }
}

```

```

 }

 // Define the scan parameters
 ScanRequest request = ScanRequest.builder()
 .tableName(tableName)
 .filterExpression("#attrName IN (" + inClause.toString() + ")")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 // Perform the scan operation
 return dynamoDbClient.scan(request);
}

/**
 * Generates a list of sample values for testing.
 *
 * <p>Helper method to generate a list of sample values for testing.
 *
 * @param valueType The type of values to generate (string, number, or boolean)
 * @param count The number of values to generate
 * @return List of generated attribute values
 */
public static List<AttributeValue> generateSampleValues(String valueType, int
count) {
 List<AttributeValue> values = new ArrayList<>();

 for (int i = 0; i < count; i++) {
 AttributeValue value;

 switch (valueType.toLowerCase(Locale.ROOT)) {
 case "string":
 value = AttributeValue.builder().s("Value" + i).build();
 break;
 case "number":
 value = AttributeValue.builder().n(String.valueOf(i)).build();
 break;
 case "boolean":
 value = AttributeValue.builder().bool(i % 2 == 0).build();
 break;
 default:
 throw new IllegalArgumentException("Unsupported value type: " +
valueType);
 }
 }
}

```

```
 values.add(value);
 }

 return values;
}
```

### Example usage of comparing multiple values with AWS SDK for Java 2.x.

```
public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 System.out.println("Demonstrating how to compare multiple values with a
single attribute in DynamoDB");

 try {
 // Example 1: Using the IN operator
 System.out.println("\nExample 1: Using the IN operator");
 List<AttributeValue> categories = List.of(
 AttributeValue.builder().s("Electronics").build(),
 AttributeValue.builder().s("Computers").build(),
 AttributeValue.builder().s("Accessories").build());

 QueryResponse inResponse = compareMultipleValues(
 dynamoDbClient,
 tableName,
 "Department",
 AttributeValue.builder().s("Retail").build(),
 "Category",
 categories);

 System.out.println("Found " + inResponse.count() + " items using IN
operator");
 System.out.println("Items: " + inResponse.items());

 // Example 2: Using multiple OR conditions
 System.out.println("\nExample 2: Using multiple OR conditions");
 QueryResponse orResponse = compareWithOrConditions(
 dynamoDbClient,
 tableName,
 "Department",
 AttributeValue.builder().s("Retail").build(),
 "Category",
```

```

 categories);

 System.out.println("Found " + orResponse.count() + " items using OR
conditions");
 System.out.println("Items: " + orResponse.items());

 // Example 3: Performance comparison
 System.out.println("\nExample 3: Performance comparison");
 Map<String, Object> perfComparison = comparePerformance(
 dynamoDbClient,
 tableName,
 "Department",
 AttributeValue.builder().s("Retail").build(),
 "Category",
 categories);

 if ((boolean) perfComparison.get("success")) {
 System.out.println("IN operator duration: " +
perfComparison.get("inOperatorDuration") + " ns");
 System.out.println("OR conditions duration: " +
perfComparison.get("orConditionsDuration") + " ns");
 System.out.println("IN operator found " +
perfComparison.get("inOperatorItems") + " items");
 System.out.println("OR conditions found " +
perfComparison.get("orConditionsItems") + " items");
 System.out.println("Expression complexity comparison:");
 System.out.println(" IN operator: " +
perfComparison.get("inOperatorExpression"));
 System.out.println(" OR conditions: " +
perfComparison.get("orConditionsExpression"));
 } else {
 System.out.println("Performance comparison failed: " +
perfComparison.get("error"));
 }

 // Example 4: Using IN with a large number of values
 System.out.println("\nExample 4: Using IN with a large number of
values");
 List<AttributeValue> productIds = generateSampleValues("string", 20);

 ScanResponse largeInResponse = scanWithLargeInClause(dynamoDbClient,
tableName, "ProductId", productIds);

 System.out.println(

```



```

 "Found " + largeInResponse.count() + " items using IN with " +
 productIds.size() + " values");

 // Explain the benefits of using IN
 System.out.println("\nKey points about using the IN operator in
 DynamoDB:");
 System.out.println("1. The IN operator allows comparing a single
 attribute against multiple values");
 System.out.println("2. IN is more concise than using multiple OR
 conditions");
 System.out.println("3. IN counts as only 1 operator regardless of the
 number of values");
 System.out.println("4. Multiple OR conditions count as 1 operator per
 condition plus 1 per OR");
 System.out.println("5. Using IN helps stay within the 300 operator limit
 for complex expressions");
 System.out.println("6. IN can be used in filter expressions and
 condition expressions");
 System.out.println("7. The IN operator supports up to 100 comparison
 values");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [Query](#)
  - [Scan](#)

## Conditionally update an item's TTL

The following code example shows how to conditionally update an item's TTL.

### SDK for Java 2.x

Update TTL on an existing DynamoDB Item in a table, with a condition.

```
package com.amazon.samplelib.ttl;
```

```

import com.amazon.samplelib.CodeSampleUtils;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import
 software.amazon.awssdk.services.dynamodb.model.ConditionalCheckFailedException;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.Map;
import java.util.Optional;

/**
 * Updates an item in a DynamoDB table with TTL attributes using a conditional
 * expression.
 * This class demonstrates how to conditionally update TTL expiration timestamps.
 */
public class UpdateTTLConditional {

 private static final String USAGE =
 """
 Usage:
 <tableName> <primaryKey> <sortKey> <region>
 Where:
 tableName - The Amazon DynamoDB table being queried.
 primaryKey - The name of the primary key. Also known as the hash or
partition key.
 sortKey - The name of the sort key. Also known as the range
attribute.
 region (optional) - The AWS region that the Amazon DynamoDB table is
located in. (Default: us-east-1)
 """;

 private static final int DAYS_TO_EXPIRE = 90;
 private static final int SECONDS_PER_DAY = 24 * 60 * 60;
 private static final String PRIMARY_KEY_ATTR = "primaryKey";
 private static final String SORT_KEY_ATTR = "sortKey";
 private static final String UPDATED_AT_ATTR = "updatedAt";
 private static final String EXPIRE_AT_ATTR = "expireAt";
 private static final String UPDATE_EXPRESSION = "SET " + UPDATED_AT_ATTR + "=:c,
" + EXPIRE_AT_ATTR + "=:e";

```

```

 private static final String CONDITION_EXPRESSION = "attribute_exists(" +
PRIMARY_KEY_ATTR + ")";
 private static final String SUCCESS_MESSAGE = "%s UpdateItem operation with TTL
successful.";
 private static final String CONDITION_FAILED_MESSAGE = "Condition check failed.
Item does not exist.";
 private static final String TABLE_NOT_FOUND_ERROR = "Error: The Amazon DynamoDB
table \"%s\" can't be found.";

 private final DynamoDbClient dynamoDbClient;

 /**
 * Constructs an UpdateTTLConditional with a default DynamoDB client.
 */
 public UpdateTTLConditional() {
 this.dynamoDbClient = null;
 }

 /**
 * Constructs an UpdateTTLConditional with the specified DynamoDB client.
 *
 * @param dynamoDbClient The DynamoDB client to use
 */
 public UpdateTTLConditional(final DynamoDbClient dynamoDbClient) {
 this.dynamoDbClient = dynamoDbClient;
 }

 /**
 * Main method to demonstrate conditionally updating an item with TTL.
 *
 * @param args Command line arguments
 */
 public static void main(final String[] args) {
 try {
 int result = new UpdateTTLConditional().processArgs(args);
 System.exit(result);
 } catch (Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 /**
 * Process command line arguments and conditionally update an item with TTL.

```

```

*
* @param args Command line arguments
* @return 0 if successful, non-zero otherwise
* @throws ResourceNotFoundException If the table doesn't exist
* @throws DynamoDbException If an error occurs during the operation
* @throws IllegalArgumentException If arguments are invalid
*/
public int processArgs(final String[] args) {
 // Argument validation (remove or replace this line when reusing this code)
 CodeSampleUtils.validateArgs(args, new int[] {3, 4}, USAGE);

 final String tableName = args[0];
 final String primaryKey = args[1];
 final String sortKey = args[2];
 final Region region = Optional.ofNullable(args.length > 3 ? args[3] : null)
 .map(Region::of)
 .orElse(Region.US_EAST_1);

 // Get current time in epoch second format
 final long currentTime = System.currentTimeMillis() / 1000;

 // Calculate expiration time 90 days from now in epoch second format
 final long expireDate = currentTime + (DAYS_TO_EXPIRE * SECONDS_PER_DAY);

 // Create the key map for the item to update
 final Map<String, AttributeValue> keyMap = Map.of(
 PRIMARY_KEY_ATTR, AttributeValue.builder().s(primaryKey).build(),
 SORT_KEY_ATTR, AttributeValue.builder().s(sortKey).build());

 // Create the expression attribute values
 final Map<String, AttributeValue> expressionAttributeValues = Map.of(
 ":c", AttributeValue.builder().n(String.valueOf(currentTime)).build(),
 ":e", AttributeValue.builder().n(String.valueOf(expireDate)).build());

 final UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(keyMap)
 .updateExpression(UPDATE_EXPRESSION)
 .conditionExpression(CONDITION_EXPRESSION)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient

```

```

 : DynamoDbClient.builder().region(region).build()) {
 final UpdateItemResponse response = ddb.updateItem(request);
 System.out.println(String.format(SUCCESS_MESSAGE, tableName));
 return 0;
 } catch (ConditionalCheckFailedException e) {
 System.err.println(CONDITION_FAILED_MESSAGE);
 throw e;
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }
}
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Count expression operators

The following code example shows how to count expression operators in DynamoDB.

- Understand DynamoDB's 300 operator limit.
- Count operators in complex expressions.
- Optimize expressions to stay within limits.

## SDK for Java 2.x

Demonstrate expression operator counting using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.Map;
import java.util.regex.Matcher;

```

```

import java.util.regex.Pattern;

/**
 * Creates a complex filter expression with a specified number of conditions.
 *
 * <p>This method demonstrates how to generate a complex expression with
 * a specific number of operators to test the 300 operator limit.
 *
 * @param conditionsCount Number of conditions to include
 * @param useAnd Whether to use AND (true) or OR (false) between conditions
 * @return Map containing the filter expression, attribute values, and operator
count
 */
public static Map<String, Object> createComplexFilterExpression(int
conditionsCount, boolean useAnd) {
 // Initialize the expression parts and attribute values
 StringBuilder filterExpression = new StringBuilder();
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();

 // Generate the specified number of conditions
 for (int i = 0; i < conditionsCount; i++) {
 // Add the operator between conditions (except for the first one)
 if (i > 0) {
 filterExpression.append(useAnd ? " AND " : " OR ");
 }

 // Alternate between different comparison operators for variety
 String valueKey = ":val" + i;

 switch (i % 5) {
 case 0:
 filterExpression.append("attribute").append(i).append(" =
").append(valueKey);
 expressionAttributeValues.put(
 valueKey, AttributeValue.builder().s("value" + i).build());
 break;
 case 1:
 filterExpression.append("attribute").append(i).append(" >
").append(valueKey);
 expressionAttributeValues.put(
 valueKey,
 AttributeValue.builder().n(String.valueOf(i)).build());
 break;
 case 2:

```

```

 filterExpression.append("attribute").append(i).append(" <
").append(valueKey);
 expressionAttributeValues.put(
 valueKey,
 AttributeValue.builder().n(String.valueOf(i * 10)).build());
 break;
 case 3:
 filterExpression
 .append("contains(attribute")
 .append(i)
 .append(", ")
 .append(valueKey)
 .append(")");
 expressionAttributeValues.put(
 valueKey, AttributeValue.builder().s("substring" +
i).build());
 break;
 case 4:
 filterExpression
 .append("attribute_exists(attribute")
 .append(i)
 .append(")");
 break;
 default:
 // This case will never be reached, but added to satisfy
checkstyle
 break;
 }
}

// Calculate the operator count
// Each condition has 1 operator (=, >, <, contains, attribute_exists)
// Each AND or OR between conditions is 1 operator
int operatorCount = conditionsCount + (conditionsCount > 0 ? conditionsCount
- 1 : 0);

// Create the result map
Map<String, Object> result = new HashMap<>();
result.put("filterExpression", filterExpression.toString());
result.put("expressionAttributeValues", expressionAttributeValues);
result.put("operatorCount", operatorCount);

return result;
}

```

```

/**
 * Creates a complex update expression with a specified number of operations.
 *
 * <p>This method demonstrates how to generate a complex update expression with
 * a specific number of operators to test the 300 operator limit.
 *
 * @param operationsCount Number of operations to include
 * @return Map containing the update expression, attribute values, and operator
count
 */
public static Map<String, Object> createComplexUpdateExpression(int
operationsCount) {
 // Initialize the expression parts and attribute values
 StringBuilder updateExpression = new StringBuilder("SET ");
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();

 // Generate the specified number of SET operations
 for (int i = 0; i < operationsCount; i++) {
 // Add comma between operations (except for the first one)
 if (i > 0) {
 updateExpression.append(", ");
 }

 // Alternate between different types of SET operations
 String valueKey = ":val" + i;

 switch (i % 3) {
 case 0:
 // Simple assignment (1 operator: =)
 updateExpression.append("attribute").append(i).append(" =
").append(valueKey);
 expressionAttributeValues.put(
 valueKey, AttributeValue.builder().s("value" + i).build());
 break;
 case 1:
 // Addition (2 operators: = and +)
 updateExpression
 .append("attribute")
 .append(i)
 .append(" = attribute")
 .append(i)
 .append(" + ")
 .append(valueKey);

```



```

 expressionAttributeValues.put(
 valueKey,
 AttributeValue.builder().n(String.valueOf(i)).build());
 break;
 case 2:
 // Conditional assignment with if_not_exists (2 operators: = and
if_not_exists)
 updateExpression
 .append("attribute")
 .append(i)
 .append(" = if_not_exists(attribute")
 .append(i)
 .append(", ")
 .append(valueKey)
 .append(")");
 expressionAttributeValues.put(
 valueKey,
 AttributeValue.builder().n(String.valueOf(i * 10)).build());
 break;
 default:
 // This case will never be reached, but added to satisfy
checkstyle
 break;
 }
}

// Calculate the operator count
// Each operation has 1-2 operators as noted above
int operatorCount = 0;
for (int i = 0; i < operationsCount; i++) {
 operatorCount += (i % 3 == 0) ? 1 : 2;
}

// Create the result map
Map<String, Object> result = new HashMap<>();
result.put("updateExpression", updateExpression.toString());
result.put("expressionAttributeValues", expressionAttributeValues);
result.put("operatorCount", operatorCount);

return result;
}

/**
 * Test the operator limit by attempting an operation with a complex expression.

```

```

*
* <p>This method demonstrates what happens when an expression approaches or
* exceeds the 300 operator limit.
*
* @param dynamoDbClient The DynamoDB client
* @param tableName The name of the DynamoDB table
* @param key The key of the item to update
* @param operatorCount Target number of operators to include
* @return Map containing the result of the operation attempt
*/
public static Map<String, Object> testOperatorLimit(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key, int operatorCount) {

 // Create a complex update expression with the specified operator count
 Map<String, Object> expressionData =
 createComplexUpdateExpression((int) Math.ceil(operatorCount / 1.5)); //
Adjust to get close to target count

 String updateExpression = (String) expressionData.get("updateExpression");
 @SuppressWarnings("unchecked")
 Map<String, AttributeValue> expressionAttributeValues =
 (Map<String, AttributeValue>)
expressionData.get("expressionAttributeValues");
 int actualCount = (int) expressionData.get("operatorCount");

 System.out.println("Generated update expression with approximately " +
actualCount + " operators");

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression(updateExpression)
 .expressionAttributeValues(expressionAttributeValues)
 .returnValues("UPDATED_NEW")
 .build();

 try {
 // Attempt the update operation
 UpdateItemResponse response = dynamoDbClient.updateItem(request);

 Map<String, Object> result = new HashMap<>();
 result.put("success", true);
 }
}

```

```

 result.put("message", "Operation succeeded with " + actualCount + "
operators");
 result.put("data", response);
 return result;

 } catch (DynamoDbException e) {
 // Check if the error is due to exceeding the operator limit
 if (e.getMessage().contains("too many operators")) {
 Map<String, Object> result = new HashMap<>();
 result.put("success", false);
 result.put("message", "Operation failed: " + e.getMessage());
 result.put("operatorCount", actualCount);
 return result;
 }

 // Return other errors
 Map<String, Object> result = new HashMap<>();
 result.put("success", false);
 result.put("message", "Operation failed: " + e.getMessage());
 result.put("error", e);
 return result;
 }
}

/**
 * Break down a complex expression into multiple simpler operations.
 *
 * <p>This method demonstrates how to handle expressions that would exceed
 * the 300 operator limit by breaking them into multiple operations.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param totalOperations Total number of operations to perform
 * @return Map containing the results of the operations
 */
public static Map<String, Object> breakDownComplexExpression(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key, int totalOperations) {

 // Calculate how many operations we can safely include in each batch
 // Using 150 as a conservative limit (well below 300)
 final int operationsPerBatch = 100;

```

```

 final int batchCount = (int) Math.ceil((double) totalOperations /
operationsPerBatch);

 System.out.println("Breaking down " + totalOperations + " operations into "
+ batchCount + " batches");

 Map<String, Object> results = new HashMap<>();
 results.put("totalBatches", batchCount);

 Map<Integer, Map<String, Object>> batchResults = new HashMap<>();

 // Process each batch
 for (int batch = 0; batch < batchCount; batch++) {
 // Calculate the operations for this batch
 int batchStart = batch * operationsPerBatch;
 int batchEnd = Math.min(batchStart + operationsPerBatch,
totalOperations);
 int batchSize = batchEnd - batchStart;

 System.out.println(
 "Processing batch " + (batch + 1) + "/" + batchCount + " with " +
batchSize + " operations");

 // Create an update expression for this batch
 Map<String, Object> expressionData =
createComplexUpdateExpression(batchSize);

 String updateExpression = (String)
expressionData.get("updateExpression");
 @SuppressWarnings("unchecked")
 Map<String, AttributeValue> expressionAttributeValues =
 (Map<String, AttributeValue>)
expressionData.get("expressionAttributeValues");
 int operatorCount = (int) expressionData.get("operatorCount");

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression(updateExpression)
 .expressionAttributeValues(expressionAttributeValues)
 .returnValues("UPDATED_NEW")
 .build();

```

```

 try {
 // Perform the update operation for this batch
 UpdateItemResponse response = dynamoDbClient.updateItem(request);

 Map<String, Object> batchResult = new HashMap<>();
 batchResult.put("batch", batch + 1);
 batchResult.put("success", true);
 batchResult.put("operatorCount", operatorCount);
 batchResult.put("attributes", response.attributes());

 batchResults.put(batch, batchResult);

 } catch (DynamoDbException e) {
 Map<String, Object> batchResult = new HashMap<>();
 batchResult.put("batch", batch + 1);
 batchResult.put("success", false);
 batchResult.put("operatorCount", operatorCount);
 batchResult.put("error", e.getMessage());

 batchResults.put(batch, batchResult);

 // Continue with next batch instead of breaking
 continue;
 }
 }

 results.put("results", batchResults);
 return results;
}

/**
 * Count operators in a DynamoDB expression based on the rules in the
 * documentation.
 *
 * <p>This method demonstrates how operators are counted according to the
 * DynamoDB documentation.
 *
 * @param expression The DynamoDB expression to analyze
 * @return Map containing the breakdown of operator counts
 */
public static Map<String, Integer> countOperatorsInExpression(String expression)
{
 // Initialize counters for different operator types
 Map<String, Integer> counts = new HashMap<>();

```

```

counts.put("comparisonOperators", 0);
counts.put("logicalOperators", 0);
counts.put("functions", 0);
counts.put("arithmeticOperators", 0);
counts.put("specialOperators", 0);
counts.put("total", 0);

// Count comparison operators (=, <>, <, <=, >, >=)
// This is a simplified approach and may not catch all cases
int comparisonCount = 0;
Pattern comparisonPattern = Pattern.compile("(=|<>|<|<=|>|>=)");
Matcher comparisonMatcher = comparisonPattern.matcher(expression);
while (comparisonMatcher.find()) {
 comparisonCount++;
}
counts.put("comparisonOperators", comparisonCount);

// Count logical operators (AND, OR, NOT)
int andCount = countOccurrences(expression, "\\bAND\\b");
int orCount = countOccurrences(expression, "\\bOR\\b");
int notCount = countOccurrences(expression, "\\bNOT\\b");
counts.put("logicalOperators", andCount + orCount + notCount);

// Count functions (attribute_exists, attribute_not_exists, attribute_type,
// begins_with, contains, size)
int functionCount = countOccurrences(
 expression,
 "\\b(attribute_exists|attribute_not_exists|attribute_type|begins_with|
contains|size|if_not_exists)\\b");
counts.put("functions", functionCount);

// Count arithmetic operators (+ and -)
// This is a simplified approach and may not catch all cases
int arithmeticCount = 0;
Pattern arithmeticPattern = Pattern.compile("[a-zA-Z0-9_]\\\\]\\\\s*[\\\\+\\\\-\\\\
\\\\s*[a-zA-Z0-9_:(]");
Matcher arithmeticMatcher = arithmeticPattern.matcher(expression);
while (arithmeticMatcher.find()) {
 arithmeticCount++;
}
counts.put("arithmeticOperators", arithmeticCount);

// Count special operators (BETWEEN, IN)
int betweenCount = countOccurrences(expression, "\\bBETWEEN\\b");

```

```

 int inCount = countOccurrences(expression, "\\bIN\\b");
 counts.put("specialOperators", betweenCount + inCount);

 // Add extra operators for BETWEEN (each BETWEEN includes an AND)
 int currentLogicalOps = counts.getOrDefault("logicalOperators", 0);
 counts.put("logicalOperators", currentLogicalOps + betweenCount);

 // Calculate total
 int total = counts.getOrDefault("comparisonOperators", 0)
 + counts.getOrDefault("logicalOperators", 0)
 + counts.getOrDefault("functions", 0)
 + counts.getOrDefault("arithmeticOperators", 0)
 + counts.getOrDefault("specialOperators", 0);
 counts.put("total", total);

 return counts;
 }

 /**
 * Helper method to count occurrences of a pattern in a string.
 *
 * @param text The text to search in
 * @param regex The regular expression pattern to search for
 * @return The number of occurrences
 */
 private static int countOccurrences(String text, String regex) {
 final Pattern pattern = Pattern.compile(regex);
 final Matcher matcher = pattern.matcher(text);
 int count = 0;
 while (matcher.find()) {
 count++;
 }
 return count;
 }
}

```

### Example usage of expression operator counting with AWS SDK for Java 2.x.

```

public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());
}

```

```

 System.out.println("Demonstrating DynamoDB expression operator counting and
the 300 operator limit");

 try {
 // Example 1: Analyze a simple expression
 System.out.println("\nExample 1: Analyzing a simple expression");
 String simpleExpression = "Price = :price AND Rating > :rating AND
Category IN (:cat1, :cat2, :cat3)";
 Map<String, Integer> simpleCount =
countOperatorsInExpression(simpleExpression);

 System.out.println("Expression: " + simpleExpression);
 System.out.println("Operator count breakdown:");
 System.out.println("- Comparison operators: " +
simpleCount.get("comparisonOperators"));
 System.out.println("- Logical operators: " +
simpleCount.get("logicalOperators"));
 System.out.println("- Functions: " + simpleCount.get("functions"));
 System.out.println("- Arithmetic operators: " +
simpleCount.get("arithmeticOperators"));
 System.out.println("- Special operators: " +
simpleCount.get("specialOperators"));
 System.out.println("- Total operators: " + simpleCount.get("total"));

 // Example 2: Analyze a complex expression
 System.out.println("\nExample 2: Analyzing a complex expression");
 String complexExpression = "(attribute_exists(Category) AND Size
BETWEEN :min AND :max) OR "
 + "(Price > :price AND contains(Description, :keyword) AND "
 + "(Rating >= :minRating OR Reviews > :minReviews))";
 Map<String, Integer> complexCount =
countOperatorsInExpression(complexExpression);

 System.out.println("Expression: " + complexExpression);
 System.out.println("Operator count breakdown:");
 System.out.println("- Comparison operators: " +
complexCount.get("comparisonOperators"));
 System.out.println("- Logical operators: " +
complexCount.get("logicalOperators"));
 System.out.println("- Functions: " + complexCount.get("functions"));
 System.out.println("- Arithmetic operators: " +
complexCount.get("arithmeticOperators"));

```



```

 System.out.println("- Special operators: " +
complexCount.get("specialOperators"));
 System.out.println("- Total operators: " + complexCount.get("total"));

 // Example 3: Test approaching the operator limit
 System.out.println("\nExample 3: Testing an expression approaching the
operator limit");
 Map<String, Object> approachingLimit = testOperatorLimit(dynamoDbClient,
tableName, key, 290);
 System.out.println(approachingLimit.get("message"));

 // Example 4: Test exceeding the operator limit
 System.out.println("\nExample 4: Testing an expression exceeding the
operator limit");
 Map<String, Object> exceedingLimit = testOperatorLimit(dynamoDbClient,
tableName, key, 310);
 System.out.println(exceedingLimit.get("message"));

 // Example 5: Breaking down a complex expression
 System.out.println("\nExample 5: Breaking down a complex expression into
multiple operations");
 Map<String, Object> breakdownResult =
breakDownComplexExpression(dynamoDbClient, tableName, key, 500);
 @SuppressWarnings("unchecked")
 Map<Integer, Map<String, Object>> results =
 (Map<Integer, Map<String, Object>>) breakdownResult.get("results");
 System.out.println(
 "Processed " + results.size() + " of " +
breakdownResult.get("totalBatches") + " batches");

 // Explain the operator counting rules
 System.out.println("\nKey points about DynamoDB expression operator
counting:");
 System.out.println("1. The maximum number of operators in any expression
is 300");
 System.out.println("2. Each comparison operator (=, <>, <, <=, >, >=)
counts as 1 operator");
 System.out.println("3. Each logical operator (AND, OR, NOT) counts as 1
operator");
 System.out.println("4. Each function call (attribute_exists, contains,
etc.) counts as 1 operator");
 System.out.println("5. Each arithmetic operator (+ or -) counts as 1
operator");

```

```
 System.out.println("6. BETWEEN counts as 2 operators (BETWEEN itself and
the AND within it)");
 System.out.println("7. IN counts as 1 operator regardless of the number
of values");
 System.out.println("8. Parentheses for grouping and attribute paths
don't count as operators");
 System.out.println("9. When you exceed the limit, the error always
reports '301 operators'");
 System.out.println("10. For complex operations, break them into multiple
smaller operations");

 } catch (Exception e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
}
```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon Rekognition
- Amazon S3

- Amazon SNS

## Create a table with global secondary index

The following code example shows how to create a table with global secondary index.

### SDK for Java 2.x

Create DynamoDB table with Global Secondary Index using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.CreateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DeleteTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableRequest;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GlobalSecondaryIndex;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.Projection;
import software.amazon.awssdk.services.dynamodb.model.ProjectionType;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.waiters.DynamoDbWaiter;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public void createTable() {
 try {
 // Attribute definitions
 final List<AttributeDefinition> attributeDefinitions = new
 ArrayList<>();
```

```
attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName(ISSUE_ID_ATTR)
 .attributeType(ScalarAttributeType.S)
 .build());
attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName(TITLE_ATTR)
 .attributeType(ScalarAttributeType.S)
 .build());
attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName(CREATE_DATE_ATTR)
 .attributeType(ScalarAttributeType.S)
 .build());
attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName(DUE_DATE_ATTR)
 .attributeType(ScalarAttributeType.S)
 .build());

// Key schema for table
final List<KeySchemaElement> tableKeySchema = new ArrayList<>();
tableKeySchema.add(KeySchemaElement.builder()
 .attributeName(ISSUE_ID_ATTR)
 .keyType(KeyType.HASH)
 .build()); // Partition key
tableKeySchema.add(KeySchemaElement.builder()
 .attributeName(TITLE_ATTR)
 .keyType(KeyType.RANGE)
 .build()); // Sort key

// Initial provisioned throughput settings for the indexes
final ProvisionedThroughput ptIndex = ProvisionedThroughput.builder()
 .readCapacityUnits(1L)
 .writeCapacityUnits(1L)
 .build();

// CreateDateIndex
final List<KeySchemaElement> createDateKeySchema = new ArrayList<>();
createDateKeySchema.add(KeySchemaElement.builder()
 .attributeName(CREATE_DATE_ATTR)
 .keyType(KeyType.HASH)
 .build());
createDateKeySchema.add(KeySchemaElement.builder()
 .attributeName(ISSUE_ID_ATTR)
 .keyType(KeyType.RANGE)
 .build());
```

```

 final Projection createDateProjection = Projection.builder()
 .projectionType(ProjectionType.INCLUDE)
 .nonKeyAttributes(DESCRIPTION_ATTR, STATUS_ATTR)
 .build();

 final GlobalSecondaryIndex createDateIndex =
GlobalSecondaryIndex.builder()
 .indexName(CREATE_DATE_INDEX)
 .keySchema(createDateKeySchema)
 .projection(createDateProjection)
 .provisionedThroughput(ptIndex)
 .build();

// TitleIndex
final List<KeySchemaElement> titleKeySchema = new ArrayList<>();
titleKeySchema.add(KeySchemaElement.builder()
 .attributeName(TITLE_ATTR)
 .keyType(KeyType.HASH)
 .build());
titleKeySchema.add(KeySchemaElement.builder()
 .attributeName(ISSUE_ID_ATTR)
 .keyType(KeyType.RANGE)
 .build());

final Projection titleProjection =
Projection.builder().projectionType(ProjectionType.KEYS_ONLY).build();

final GlobalSecondaryIndex titleIndex = GlobalSecondaryIndex.builder()
 .indexName(TITLE_INDEX)
 .keySchema(titleKeySchema)
 .projection(titleProjection)
 .provisionedThroughput(ptIndex)
 .build();

// DueDateIndex
final List<KeySchemaElement> dueDateKeySchema = new ArrayList<>();
dueDateKeySchema.add(KeySchemaElement.builder()
 .attributeName(DUE_DATE_ATTR)
 .keyType(KeyType.HASH)
 .build());

final Projection dueDateProjection =

```

```

 Projection.builder().projectionType(ProjectionType.ALL).build();

 final GlobalSecondaryIndex dueDateIndex = GlobalSecondaryIndex.builder()
 .indexName(DUE_DATE_INDEX)
 .keySchema(dueDateKeySchema)
 .projection(dueDateProjection)
 .provisionedThroughput(ptIndex)
 .build();

 final CreateTableRequest createTableRequest =
CreateTableRequest.builder()
 .tableName(TABLE_NAME)
 .keySchema(tableKeySchema)
 .attributeDefinitions(attributeDefinitions)
 .globalSecondaryIndexes(createDateIndex, titleIndex, dueDateIndex)
 .provisionedThroughput(ProvisionedThroughput.builder()
 .readCapacityUnits(1L)
 .writeCapacityUnits(1L)
 .build())
 .build();

 System.out.println("Creating table " + TABLE_NAME + "...");
 dynamoDbClient.createTable(createTableRequest);

 // Wait for table to become active
 System.out.println("Waiting for " + TABLE_NAME + " to become
ACTIVE...");
 final DynamoDbWaiter waiter = dynamoDbClient.waiter();
 final DescribeTableRequest describeTableRequest =
 DescribeTableRequest.builder().tableName(TABLE_NAME).build();

 final WaiterResponse<DescribeTableResponse> waiterResponse =
 waiter.waitUntilTableExists(describeTableRequest);
 waiterResponse.matched().response().ifPresent(response ->
System.out.println("Table is now ready for use"));

 } catch (DynamoDbException e) {
 System.err.println("Error creating table: " + e.getMessage());
 e.printStackTrace();
 }
}

```

- For API details, see [CreateTable](#) in *AWS SDK for Java 2.x API Reference*.

## Create a table with warm throughput enabled

The following code example shows how to create a table with warm throughput enabled.

### SDK for Java 2.x

Create DynamoDB table with warm throughput setting using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeDefinition;
import software.amazon.awssdk.services.dynamodb.model.CreateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.CreateTableResponse;
import software.amazon.awssdk.services.dynamodb.model.GlobalSecondaryIndex;
import software.amazon.awssdk.services.dynamodb.model.KeySchemaElement;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.Projection;
import software.amazon.awssdk.services.dynamodb.model.ProvisionedThroughput;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;
import software.amazon.awssdk.services.dynamodb.model.WarmThroughput;

 public static WarmThroughput buildWarmThroughput(final Long readUnitsPerSecond,
final Long writeUnitsPerSecond) {
 return WarmThroughput.builder()
 .readUnitsPerSecond(readUnitsPerSecond)
 .writeUnitsPerSecond(writeUnitsPerSecond)
 .build();
 }

 /**
 * Builds a ProvisionedThroughput object with the specified read and write
 * capacity units.
 *
 * @param readCapacityUnits The read capacity units
 * @param writeCapacityUnits The write capacity units
 * @return A configured ProvisionedThroughput object
 */
 public static ProvisionedThroughput buildProvisionedThroughput(
 final Long readCapacityUnits, final Long writeCapacityUnits) {
 return ProvisionedThroughput.builder()
 .readCapacityUnits(readCapacityUnits)
 .writeCapacityUnits(writeCapacityUnits)
 .build();
 }
```

```

/**
 * Builds an AttributeDefinition with the specified name and type.
 *
 * @param attributeName The attribute name
 * @param scalarAttributeType The attribute type
 * @return A configured AttributeDefinition
 */
private static AttributeDefinition buildAttributeDefinition(
 final String attributeName, final ScalarAttributeType scalarAttributeType) {
 return AttributeDefinition.builder()
 .attributeName(attributeName)
 .attributeType(scalarAttributeType)
 .build();
}

/**
 * Builds a KeySchemaElement with the specified name and key type.
 *
 * @param attributeName The attribute name
 * @param keyType The key type (HASH or RANGE)
 * @return A configured KeySchemaElement
 */
private static KeySchemaElement buildKeySchemaElement(final String
attributeName, final KeyType keyType) {
 return KeySchemaElement.builder()
 .attributeName(attributeName)
 .keyType(keyType)
 .build();
}

/**
 * Creates a DynamoDB table with the specified configuration including warm
throughput settings.
 *
 * @param ddb The DynamoDB client
 * @param tableName The name of the table to create
 * @param partitionKey The partition key attribute name
 * @param sortKey The sort key attribute name
 * @param miscellaneousKeyAttribute Additional key attribute name for GSI
 * @param nonKeyAttribute Non-key attribute to include in GSI projection
 * @param tableReadCapacityUnits Read capacity units for the table
 * @param tableWriteCapacityUnits Write capacity units for the table
 * @param tableWarmReadUnitsPerSecond Warm read units per second for the table
 * @param tableWarmWriteUnitsPerSecond Warm write units per second for the table

```



```

 * @param globalSecondaryIndexName The name of the GSI to create
 * @param globalSecondaryIndexReadCapacityUnits Read capacity units for the GSI
 * @param globalSecondaryIndexWriteCapacityUnits Write capacity units for the
 GSI
 * @param globalSecondaryIndexWarmReadUnitsPerSecond Warm read units per second
 for the GSI
 * @param globalSecondaryIndexWarmWriteUnitsPerSecond Warm write units per
 second for the GSI
 */
 public static void createDynamoDBTable(
 final DynamoDbClient ddb,
 final String tableName,
 final String partitionKey,
 final String sortKey,
 final String miscellaneousKeyAttribute,
 final String nonKeyAttribute,
 final Long tableReadCapacityUnits,
 final Long tableWriteCapacityUnits,
 final Long tableWarmReadUnitsPerSecond,
 final Long tableWarmWriteUnitsPerSecond,
 final String globalSecondaryIndexName,
 final Long globalSecondaryIndexReadCapacityUnits,
 final Long globalSecondaryIndexWriteCapacityUnits,
 final Long globalSecondaryIndexWarmReadUnitsPerSecond,
 final Long globalSecondaryIndexWarmWriteUnitsPerSecond) {

 // Define the table attributes
 final AttributeDefinition partitionKeyAttribute =
 buildAttributeDefinition(partitionKey, ScalarAttributeType.S);
 final AttributeDefinition sortKeyAttribute =
 buildAttributeDefinition(sortKey, ScalarAttributeType.S);
 final AttributeDefinition miscellaneousKeyAttributeDefinition =
 buildAttributeDefinition(miscellaneousKeyAttribute,
 ScalarAttributeType.N);
 final AttributeDefinition[] attributeDefinitions = {
 partitionKeyAttribute, sortKeyAttribute,
 miscellaneousKeyAttributeDefinition
 };

 // Define the table key schema
 final KeySchemaElement partitionKeyElement =
 buildKeySchemaElement(partitionKey, KeyType.HASH);
 final KeySchemaElement sortKeyElement = buildKeySchemaElement(sortKey,
 KeyType.RANGE);

```

```
final KeySchemaElement[] keySchema = {partitionKeyElement, sortKeyElement};

// Define the provisioned throughput for the table
final ProvisionedThroughput provisionedThroughput =
 buildProvisionedThroughput(tableReadCapacityUnits,
tableWriteCapacityUnits);

// Define the Global Secondary Index (GSI)
final KeySchemaElement globalSecondaryIndexPartitionKeyElement =
buildKeySchemaElement(sortKey, KeyType.HASH);
final KeySchemaElement globalSecondaryIndexSortKeyElement =
 buildKeySchemaElement(miscellaneousKeyAttribute, KeyType.RANGE);
final KeySchemaElement[] gsiKeySchema = {
 globalSecondaryIndexPartitionKeyElement,
globalSecondaryIndexSortKeyElement
};

final Projection gsiProjection = Projection.builder()
 .projectionType(PROJECTION_TYPE_INCLUDE)
 .nonKeyAttributes(nonKeyAttribute)
 .build();

final ProvisionedThroughput gsiProvisionedThroughput =
 buildProvisionedThroughput(globalSecondaryIndexReadCapacityUnits,
globalSecondaryIndexWriteCapacityUnits);

// Define the warm throughput for the Global Secondary Index (GSI)
final WarmThroughput gsiWarmThroughput = buildWarmThroughput(
 globalSecondaryIndexWarmReadUnitsPerSecond,
globalSecondaryIndexWarmWriteUnitsPerSecond);

final GlobalSecondaryIndex globalSecondaryIndex =
GlobalSecondaryIndex.builder()
 .indexName(globalSecondaryIndexName)
 .keySchema(gsiKeySchema)
 .projection(gsiProjection)
 .provisionedThroughput(gsiProvisionedThroughput)
 .warmThroughput(gsiWarmThroughput)
 .build();

// Define the warm throughput for the table
final WarmThroughput tableWarmThroughput =
 buildWarmThroughput(tableWarmReadUnitsPerSecond,
tableWarmWriteUnitsPerSecond);
```

```
final CreateTableRequest request = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(attributeDefinitions)
 .keySchema(keySchema)
 .provisionedThroughput(provisionedThroughput)
 .globalSecondaryIndexes(globalSecondaryIndex)
 .warmThroughput(tableWarmThroughput)
 .build();

final CreateTableResponse response = ddb.createTable(request);
System.out.println(response);
}
```

- For API details, see [CreateTable](#) in *AWS SDK for Java 2.x API Reference*.

## Create a web application to track DynamoDB data

The following code example shows how to create a web application that tracks work items in an Amazon DynamoDB table and uses Amazon Simple Email Service (Amazon SES) to send reports.

### SDK for Java 2.x

Shows how to use the Amazon DynamoDB API to create a dynamic web application that tracks DynamoDB work data.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Amazon SES

## Create an item with a TTL

The following code example shows how to create an item with TTL.

### SDK for Java 2.x

```

package com.amazon.samplelib.ttl;

import com.amazon.samplelib.CodeSampleUtils;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import software.amazon.awssdk.services.dynamodb.model.PutItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

/**
 * Creates an item in a DynamoDB table with TTL attributes.
 * This class demonstrates how to add TTL expiration timestamps to DynamoDB items.
 */
public class CreateTTL {

 private static final String USAGE =
 """
 Usage:
 <tableName> <primaryKey> <sortKey> <region>
 Where:
 tableName - The Amazon DynamoDB table being queried.
 primaryKey - The name of the primary key. Also known as the hash or
partition key.
 sortKey - The name of the sort key. Also known as the range
attribute.
 region (optional) - The AWS region that the Amazon DynamoDB table is
located in. (Default: us-east-1)
 """;

 private static final int DAYS_TO_EXPIRE = 90;
 private static final int SECONDS_PER_DAY = 24 * 60 * 60;
 private static final String PRIMARY_KEY_ATTR = "primaryKey";
 private static final String SORT_KEY_ATTR = "sortKey";
 private static final String CREATION_DATE_ATTR = "creationDate";
 private static final String EXPIRE_AT_ATTR = "expireAt";
 private static final String SUCCESS_MESSAGE = "%s PutItem operation with TTL
successful.";
 private static final String TABLE_NOT_FOUND_ERROR = "Error: The Amazon DynamoDB
table \"%s\" can't be found.";

```

```

private final DynamoDbClient dynamoDbClient;

/**
 * Constructs a CreateTTL instance with the specified DynamoDB client.
 *
 * @param dynamoDbClient The DynamoDB client to use
 */
public CreateTTL(final DynamoDbClient dynamoDbClient) {
 this.dynamoDbClient = dynamoDbClient;
}

/**
 * Constructs a CreateTTL with a default DynamoDB client.
 */
public CreateTTL() {
 this.dynamoDbClient = null;
}

/**
 * Main method to demonstrate creating an item with TTL.
 *
 * @param args Command line arguments
 */
public static void main(final String[] args) {
 try {
 int result = new CreateTTL().processArgs(args);
 System.exit(result);
 } catch (Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

/**
 * Process command line arguments and create an item with TTL.
 *
 * @param args Command line arguments
 * @return 0 if successful, non-zero otherwise
 * @throws ResourceNotFoundException If the table doesn't exist
 * @throws DynamoDbException If an error occurs during the operation
 * @throws IllegalArgumentException If arguments are invalid
 */
public int processArgs(final String[] args) {

```

```

// Argument validation (remove or replace this line when reusing this code)
CodeSampleUtils.validateArgs(args, new int[] {3, 4}, USAGE);

final String tableName = args[0];
final String primaryKey = args[1];
final String sortKey = args[2];
final Region region = Optional.ofNullable(args.length > 3 ? args[3] : null)
 .map(Region::of)
 .orElse(Region.US_EAST_1);

try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient
 : DynamoDbClient.builder().region(region).build()) {
 final CreateTTL createTTL = new CreateTTL(ddb);
 createTTL.createItemWithTTL(tableName, primaryKey, sortKey);
 return 0;
} catch (Exception e) {
 throw e;
}
}

/**
 * Creates an item in the specified table with TTL attributes.
 *
 * @param tableName The name of the table
 * @param primaryKeyValue The value for the primary key
 * @param sortKeyValue The value for the sort key
 * @return The response from the PutItem operation
 * @throws ResourceNotFoundException If the table doesn't exist
 * @throws DynamoDbException If an error occurs during the operation
 */
public PutItemResponse createItemWithTTL(
 final String tableName, final String primaryKeyValue, final String
sortKeyValue) {
 // Get current time in epoch second format
 final long createDate = System.currentTimeMillis() / 1000;

 // Calculate expiration time 90 days from now in epoch second format
 final long expireDate = createDate + (DAYS_TO_EXPIRE * SECONDS_PER_DAY);

 final Map<String, AttributeValue> itemMap = new HashMap<>();
 itemMap.put(
 PRIMARY_KEY_ATTR, AttributeValue.builder().s(primaryKeyValue).build());

```

```

 itemMap.put(SORT_KEY_ATTR,
 AttributeValue.builder().s(sortKeyValue).build());
 itemMap.put(
 CREATION_DATE_ATTR,
 AttributeValue.builder().n(String.valueOf(createDate)).build());
 itemMap.put(
 EXPIRE_AT_ATTR,
 AttributeValue.builder().n(String.valueOf(expireDate)).build());

 final PutItemRequest request =
 PutItemRequest.builder().tableName(tableName).item(itemMap).build();

 try {
 final PutItemResponse response = dynamoDbClient.putItem(request);
 System.out.println(String.format(SUCCESS_MESSAGE, tableName));
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }
 }
}

```

- For API details, see [PutItem](#) in *AWS SDK for Java 2.x API Reference*.

## Create and manage MRSC global tables

The following code example shows how to create and manage DynamoDB global tables with Multi-Region Strong Consistency (MRSC).

- Create a table with Multi-Region Strong Consistency.
- Verify MRSC configuration and replica status.
- Test strong consistency across Regions with immediate reads.
- Perform conditional writes with MRSC guarantees.
- Clean up MRSC global table resources.

## SDK for Java 2.x

Create a regional table ready for MRSC conversion using AWS SDK for Java 2.x.

```
public static CreateTableResponse createRegionalTable(final DynamoDbClient
dynamoDbClient, final String tableName) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }

 try {
 LOGGER.info("Creating regional table: " + tableName + " (must be empty
for MRSC)");

 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("Artist")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("SongTitle")
 .attributeType(ScalarAttributeType.S)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("Artist")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("SongTitle")
 .keyType(KeyType.RANGE)
 .build())
 .billingMode(BillingMode.PAY_PER_REQUEST)
 .build();

 CreateTableResponse response =
dynamoDbClient.createTable(createTableRequest);
```



```

 LOGGER.info("Regional table creation initiated. Status: "
 + response.tableDescription().tableStatus());

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to create regional table: " + tableName + " - " +
 e.getMessage());
 throw DynamoDbException.builder()
 .message("Failed to create regional table: " + tableName)
 .cause(e)
 .build();
 }
}

```

Convert a regional table to MRSC with replicas and witness using AWS SDK for Java 2.x.

```

public static UpdateTableResponse convertToMRSCWithWitness(
 final DynamoDbClient dynamoDbClient,
 final String tableName,
 final Region replicaRegion,
 final Region witnessRegion) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (replicaRegion == null) {
 throw new IllegalArgumentException("Replica region cannot be null");
 }
 if (witnessRegion == null) {
 throw new IllegalArgumentException("Witness region cannot be null");
 }

 try {
 LOGGER.info("Converting table to MRSC with replica in " +
 replicaRegion.id() + " and witness in "
 + witnessRegion.id());
 }
}

```

```

 // Create replica update using ReplicationGroupUpdate
 ReplicationGroupUpdate replicaUpdate = ReplicationGroupUpdate.builder()
 .create(CreateReplicationGroupMemberAction.builder()
 .regionName(replicaRegion.id())
 .build())
 .build();

 // Create witness update
 GlobalTableWitnessGroupUpdate witnessUpdate =
 GlobalTableWitnessGroupUpdate.builder()
 .create(CreateGlobalTableWitnessGroupMemberAction.builder()
 .regionName(witnessRegion.id())
 .build())
 .build();

 UpdateTableRequest updateTableRequest = UpdateTableRequest.builder()
 .tableName(tableName)
 .replicaUpdates(List.of(replicaUpdate))
 .globalTableWitnessUpdates(List.of(witnessUpdate))
 .multiRegionConsistency(MultiRegionConsistency.STRONG)
 .build();

 UpdateTableResponse response =
 dynamoDbClient.updateTable(updateTableRequest);
 LOGGER.info("MRSC conversion initiated. Status: "
 + response.tableDescription().tableStatus());
 LOGGER.info("UpdateTableResponse full object: " + response);
 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to convert table to MRSC: " + tableName + " - " +
 e.getMessage());
 throw DynamoDbException.builder()
 .message("Failed to convert table to MRSC: " + tableName)
 .cause(e)
 .build();
 }
}

```

Describe an MRSC global table configuration using AWS SDK for Java 2.x.

```
public static DescribeTableResponse describeMRSCTable(final DynamoDbClient
dynamoDbClient, final String tableName) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }

 try {
 LOGGER.info("Describing MRSC global table: " + tableName);

 DescribeTableRequest request =
 DescribeTableRequest.builder().tableName(tableName).build();

 DescribeTableResponse response = dynamoDbClient.describeTable(request);

 LOGGER.info("Table status: " + response.table().tableStatus());
 LOGGER.info("Multi-region consistency: " +
response.table().multiRegionConsistency());

 if (response.table().replicas() != null
 && !response.table().replicas().isEmpty()) {
 LOGGER.info("Number of replicas: " +
response.table().replicas().size());
 response.table()
 .replicas()
 .forEach(replica -> LOGGER.info(
 "Replica region: " + replica.regionName() + ", Status: " +
replica.replicaStatus()));
 }

 if (response.table().globalTableWitnesses() != null
 && !response.table().globalTableWitnesses().isEmpty()) {
 LOGGER.info("Number of witnesses: "
 + response.table().globalTableWitnesses().size());
 response.table()
 .globalTableWitnesses()
 .forEach(witness -> LOGGER.info(
 "Witness region: " + witness.regionName() + ", Status: " +
witness.witnessStatus()));
 }
 }
}
```

```

 }

 return response;

} catch (ResourceNotFoundException e) {
 LOGGER.severe("Table not found: " + tableName + " - " + e.getMessage());
 throw DynamoDbException.builder()
 .message("Table not found: " + tableName)
 .cause(e)
 .build();
} catch (DynamoDbException e) {
 LOGGER.severe("Failed to describe table: " + tableName + " - " +
e.getMessage());
 throw DynamoDbException.builder()
 .message("Failed to describe table: " + tableName)
 .cause(e)
 .build();
}
}

```

Add test items to verify MRSC strong consistency using AWS SDK for Java 2.x.

```

public static PutItemResponse putTestItem(
 final DynamoDbClient dynamoDbClient,
 final String tableName,
 final String artist,
 final String songTitle,
 final String album,
 final String year) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (artist == null || artist.trim().isEmpty()) {
 throw new IllegalArgumentException("Artist cannot be null or empty");
 }
 if (songTitle == null || songTitle.trim().isEmpty()) {

```

```

 throw new IllegalArgumentException("Song title cannot be null or
empty");
 }

 try {
 LOGGER.info("Adding test item to MRSC global table: " + tableName);

 Map<String, AttributeValue> item = new HashMap<>();
 item.put("Artist", AttributeValue.builder().s(artist).build());
 item.put("SongTitle", AttributeValue.builder().s(songTitle).build());

 if (album != null && !album.trim().isEmpty()) {
 item.put("Album", AttributeValue.builder().s(album).build());
 }
 if (year != null && !year.trim().isEmpty()) {
 item.put("Year", AttributeValue.builder().n(year).build());
 }

 PutItemRequest putItemRequest =
 PutItemRequest.builder().tableName(tableName).item(item).build();

 PutItemResponse response = dynamoDbClient.putItem(putItemRequest);
 LOGGER.info("Test item added successfully with strong consistency");

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to add test item to table: " + tableName + " - " +
e.getMessage());
 throw DynamoDbException.builder()
 .message("Failed to add test item to table: " + tableName)
 .cause(e)
 .build();
 }
}

```

Read items with consistent reads from MRSC replicas using AWS SDK for Java 2.x.

```

public static GetItemResponse getItemWithConsistentRead(
 final DynamoDbClient dynamoDbClient, final String tableName, final String
artist, final String songTitle) {

```

```
 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (artist == null || artist.trim().isEmpty()) {
 throw new IllegalArgumentException("Artist cannot be null or empty");
 }
 if (songTitle == null || songTitle.trim().isEmpty()) {
 throw new IllegalArgumentException("Song title cannot be null or
empty");
 }

 try {
 LOGGER.info("Reading item from MRSC global table with consistent read: "
+ tableName);

 Map<String, AttributeValue> key = new HashMap<>();
 key.put("Artist", AttributeValue.builder().s(artist).build());
 key.put("SongTitle", AttributeValue.builder().s(songTitle).build());

 GetItemRequest getItemRequest = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .consistentRead(true)
 .build();

 GetItemResponse response = dynamoDbClient.getItem(getItemRequest);

 if (response.hasItem()) {
 LOGGER.info("Item found with strong consistency - no wait time
needed");
 } else {
 LOGGER.info("Item not found");
 }

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to read item from table: " + tableName + " - " +
e.getMessage());
 throw DynamoDbException.builder()
```

```

 .message("Failed to read item from table: " + tableName)
 .cause(e)
 .build();
 }
}

```

## Perform conditional updates with MRSC guarantees using AWS SDK for Java 2.x.

```

public static UpdateItemResponse performConditionalUpdate(
 final DynamoDbClient dynamoDbClient,
 final String tableName,
 final String artist,
 final String songTitle,
 final String rating) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (artist == null || artist.trim().isEmpty()) {
 throw new IllegalArgumentException("Artist cannot be null or empty");
 }
 if (songTitle == null || songTitle.trim().isEmpty()) {
 throw new IllegalArgumentException("Song title cannot be null or
empty");
 }
 if (rating == null || rating.trim().isEmpty()) {
 throw new IllegalArgumentException("Rating cannot be null or empty");
 }

 try {
 LOGGER.info("Performing conditional update on MRSC global table: " +
tableName);

 Map<String, AttributeValue> key = new HashMap<>();
 key.put("Artist", AttributeValue.builder().s(artist).build());
 key.put("SongTitle", AttributeValue.builder().s(songTitle).build());

 Map<String, String> expressionAttributeNames = new HashMap<>();

```

```

 expressionAttributeNames.put("#rating", "Rating");

 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 expressionAttributeValues.put(
 ":rating", AttributeValue.builder().n(rating).build());

 UpdateItemRequest updateItemRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #rating = :rating")
 .conditionExpression("attribute_exists(Artist)")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 UpdateItemResponse response =
dynamoDbClient.updateItem(updateItemRequest);
 LOGGER.info("Conditional update successful - demonstrates strong
consistency");

 return response;

 } catch (ConditionalCheckFailedException e) {
 LOGGER.warning("Conditional check failed: " + e.getMessage());
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to perform conditional update: " + tableName + " -
" + e.getMessage());
 throw DynamoDbException.builder()
 .message("Failed to perform conditional update: " + tableName)
 .cause(e)
 .build();
 }
}

```

Wait for MRSC replicas and witnesses to become active using AWS SDK for Java 2.x.

```

public static void waitForMRSCReplicasActive(
 final DynamoDbClient dynamoDbClient, final String tableName, final int
maxWaitTimeSeconds)
 throws InterruptedException {

```



```

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (maxWaitTimeSeconds <= 0) {
 throw new IllegalArgumentException("Max wait time must be positive");
 }

 try {
 LOGGER.info("Waiting for MRSC replicas and witnesses to become active: "
+ tableName);

 final long startTime = System.currentTimeMillis();
 final long maxWaitTimeMillis = maxWaitTimeSeconds * 1000L;
 int backoffSeconds = 5; // Start with 5 second intervals
 final int maxBackoffSeconds = 30; // Cap at 30 seconds

 while (System.currentTimeMillis() - startTime < maxWaitTimeMillis) {
 DescribeTableResponse response = describeMRSCTable(dynamoDbClient,
tableName);

 boolean allActive = true;
 StringBuilder statusReport = new StringBuilder();

 if (response.table().multiRegionConsistency() == null
 || !MultiRegionConsistency.STRONG
 .toString()
.equals(response.table().multiRegionConsistency().toString())) {
 allActive = false;
 statusReport
 .append("MultiRegionConsistency: ")
 .append(response.table().multiRegionConsistency())
 .append(" ");
 }
 if (response.table().replicas() == null
 || response.table().replicas().isEmpty()) {
 allActive = false;
 statusReport.append("No replicas found. ");
 }
 if (response.table().globalTableWitnesses() == null

```

```

 || response.table().globalTableWitnesses().isEmpty()) {
 allActive = false;
 statusReport.append("No witnesses found. ");
 }

 // Check table status
 if (!"ACTIVE".equals(response.table().tableStatus().toString())) {
 allActive = false;
 statusReport
 .append("Table: ")
 .append(response.table().tableStatus())
 .append(" ");
 }

 // Check replica status
 if (response.table().replicas() != null) {
 for (var replica : response.table().replicas()) {
 if (!"ACTIVE".equals(replica.replicaStatus().toString())) {
 allActive = false;
 statusReport
 .append("Replica(")
 .append(replica.regionName())
 .append("): ")
 .append(replica.replicaStatus())
 .append(" ");
 }
 }
 }

 // Check witness status
 if (response.table().globalTableWitnesses() != null) {
 for (var witness : response.table().globalTableWitnesses()) {
 if (!"ACTIVE".equals(witness.witnessStatus().toString())) {
 allActive = false;
 statusReport
 .append("Witness(")
 .append(witness.regionName())
 .append("): ")
 .append(witness.witnessStatus())
 .append(" ");
 }
 }
 }
 }
}

```

```

 if (allActive) {
 LOGGER.info("All MRSC replicas and witnesses are now active: " +
tableName);
 return;
 }

 LOGGER.info("Waiting for MRSC components to become active. Status: "
+ statusReport.toString());
 LOGGER.info("Next check in " + backoffSeconds + " seconds...");

 tempWait(backoffSeconds);

 // Exponential backoff with cap
 backoffSeconds = Math.min(backoffSeconds * 2, maxBackoffSeconds);
 }

 throw DynamoDbException.builder()
 .message("Timeout waiting for MRSC replicas to become active after "
+ maxWaitTimeSeconds + " seconds")
 .build();

 } catch (DynamoDbException | InterruptedException e) {
 LOGGER.severe("Failed to wait for MRSC replicas to become active: " +
tableName + " - " + e.getMessage());
 throw e;
 }
}

```

## Clean up MRSC replicas and witnesses using AWS SDK for Java 2.x.

```

public static UpdateTableResponse cleanupMRSCReplicas(
 final DynamoDbClient dynamoDbClient,
 final String tableName,
 final Region replicaRegion,
 final Region witnessRegion) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
}

```

```

 }
 if (replicaRegion == null) {
 throw new IllegalArgumentException("Replica region cannot be null");
 }
 if (witnessRegion == null) {
 throw new IllegalArgumentException("Witness region cannot be null");
 }

 try {
 LOGGER.info("Cleaning up MRSC replicas and witnesses for table: " +
 tableName);

 // Remove replica using ReplicationGroupUpdate
 ReplicationGroupUpdate replicaUpdate = ReplicationGroupUpdate.builder()
 .delete(DeleteReplicationGroupMemberAction.builder()
 .regionName(replicaRegion.id())
 .build())
 .build();

 // Remove witness
 GlobalTableWitnessGroupUpdate witnessUpdate =
 GlobalTableWitnessGroupUpdate.builder()
 .delete(DeleteGlobalTableWitnessGroupMemberAction.builder()
 .regionName(witnessRegion.id())
 .build())
 .build();

 UpdateTableRequest updateTableRequest = UpdateTableRequest.builder()
 .tableName(tableName)
 .replicaUpdates(List.of(replicaUpdate))
 .globalTableWitnessUpdates(List.of(witnessUpdate))
 .build();

 UpdateTableResponse response =
 dynamoDbClient.updateTable(updateTableRequest);
 LOGGER.info("MRSC cleanup initiated - removing replica and witness.
 Response: " + response);

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to cleanup MRSC replicas: " + tableName + " - " +
 e.getMessage());
 throw DynamoDbException.builder()

```

```

 .message("Failed to cleanup MRSC replicas: " + tableName)
 .cause(e)
 .build();
 }
}

```

## Complete MRSC workflow demonstration using AWS SDK for Java 2.x.

```

public static void demonstrateCompleteMRSCWorkflow(
 final DynamoDbClient primaryClient,
 final DynamoDbClient replicaClient,
 final String tableName,
 final Region replicaRegion,
 final Region witnessRegion)
 throws InterruptedException {

 if (primaryClient == null) {
 throw new IllegalArgumentException("Primary DynamoDB client cannot be
null");
 }
 if (replicaClient == null) {
 throw new IllegalArgumentException("Replica DynamoDB client cannot be
null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (replicaRegion == null) {
 throw new IllegalArgumentException("Replica region cannot be null");
 }
 if (witnessRegion == null) {
 throw new IllegalArgumentException("Witness region cannot be null");
 }

 try {
 LOGGER.info("=== Starting Complete MRSC Workflow Demonstration ===");

 // Step 1: Create an empty single-Region table
 LOGGER.info("Step 1: Creating empty single-Region table");
 createRegionalTable(primaryClient, tableName);
 }
}

```

```
// Use the existing GlobalTableOperations method for basic table waiting
LOGGER.info("Intermediate step: Waiting for table [" + tableName + "] to
become active before continuing");
GlobalTableOperations.waitForTableActive(primaryClient, tableName);

// Step 2: Convert to MRSC with replica and witness
LOGGER.info("Step 2: Converting to MRSC with replica and witness");
convertToMRSCWithWitness(primaryClient, tableName, replicaRegion,
witnessRegion);

// Wait for MRSC conversion to complete using MRSC-specific waiter
LOGGER.info("Waiting for MRSC conversion to complete...");
waitForMRSCReplicasActive(primaryClient, tableName);

LOGGER.info("Intermediate step: Waiting for table [" + tableName + "] to
become active before continuing");
GlobalTableOperations.waitForTableActive(primaryClient, tableName);

// Step 3: Verify MRSC configuration
LOGGER.info("Step 3: Verifying MRSC configuration");
describeMRSCTable(primaryClient, tableName);

// Step 4: Test strong consistency with data operations
LOGGER.info("Step 4: Testing strong consistency with data operations");

// Add test item to primary region
putTestItem(primaryClient, tableName, "The Beatles", "Hey Jude", "The
Beatles 1967-1970", "1968");

// Immediately read from replica region (no wait needed with MRSC)
LOGGER.info("Reading from replica region immediately (strong
consistency):");
GetItemResponse getResponse =
 getItemWithConsistentRead(replicaClient, tableName, "The Beatles",
"Hey Jude");

if (getResponse.hasItem()) {
 LOGGER.info("# Strong consistency verified - item immediately
available in replica region");
} else {
 LOGGER.warning("# Item not found in replica region");
}

// Test conditional update from replica region
```

```

 LOGGER.info("Testing conditional update from replica region:");
 performConditionalUpdate(replicaClient, tableName, "The Beatles", "Hey
Jude", "5");
 LOGGER.info("# Conditional update successful - demonstrates strong
consistency");

 // Step 5: Cleanup
 LOGGER.info("Step 5: Cleaning up resources");
 cleanupMRSCReplicas(primaryClient, tableName, replicaRegion,
witnessRegion);

 // Wait for cleanup to complete using basic table waiter
 LOGGER.info("Waiting for replica cleanup to complete...");
 GlobalTableOperations.waitForTableActive(primaryClient, tableName);

 // "Halt" until replica/witness cleanup is complete
 DescribeTableResponse cleanupVerification =
describeMRSCTable(primaryClient, tableName);
 int backoffSeconds = 5; // Start with 5 second intervals
 while (cleanupVerification.table().multiRegionConsistency() != null) {
 LOGGER.info("Waiting additional time (" + backoffSeconds + "
seconds) for MRSC cleanup to complete...");
 tempWait(backoffSeconds);

 // Exponential backoff with cap
 backoffSeconds = Math.min(backoffSeconds * 2, 30);
 cleanupVerification = describeMRSCTable(primaryClient, tableName);
 }

 // Delete the primary table
 deleteTable(primaryClient, tableName);

 LOGGER.info("=== MRSC Workflow Demonstration Complete ===");
 LOGGER.info("");
 LOGGER.info("Key benefits of Multi-Region Strong Consistency (MRSC):");
 LOGGER.info("- Immediate consistency across all regions (no eventual
consistency delays)");
 LOGGER.info("- Simplified application logic (no need to handle eventual
consistency)");
 LOGGER.info("- Support for conditional writes and transactions across
regions");
 LOGGER.info("- Consistent read operations from any region without
waiting");

```

```
 } catch (DynamoDbException | InterruptedException e) {
 LOGGER.severe("MRSC workflow failed: " + e.getMessage());
 throw e;
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateTable](#)
- [DeleteTable](#)
- [DescribeTable](#)
- [GetItem](#)
- [PutItem](#)
- [UpdateItem](#)
- [UpdateTable](#)

## Create and manage global tables demonstrating MREC

The following code example shows how to create and manage DynamoDB global tables with replicas across multiple Regions.

- Create a table with Global Secondary Index and DynamoDB Streams.
- Add replicas in different Regions to create a global table.
- Remove replicas from a global table.
- Add test items to verify replication across Regions.
- Describe global table configuration and replica status.

## SDK for Java 2.x

Create a table with Global Secondary Index and DynamoDB Streams using AWS SDK for Java 2.x.

```
public static CreateTableResponse createTableWithGSI(
 final DynamoDbClient dynamoDbClient, final String tableName, final String
 indexName) {

 if (dynamoDbClient == null) {
```



```

 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (indexName == null || indexName.trim().isEmpty()) {
 throw new IllegalArgumentException("Index name cannot be null or
empty");
 }

 try {
 LOGGER.info("Creating table: " + tableName + " with GSI: " + indexName);

 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("Artist")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("SongTitle")
 .attributeType(ScalarAttributeType.S)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("Artist")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("SongTitle")
 .keyType(KeyType.RANGE)
 .build())
 .billingMode(BillingMode.PAY_PER_REQUEST)
 .globalSecondaryIndexes(GlobalSecondaryIndex.builder()
 .indexName(indexName)
 .keySchema(KeySchemaElement.builder()
 .attributeName("SongTitle")
 .keyType(KeyType.HASH)
 .build())
 .projection(
 Projection.builder().projectionType(ProjectionType.ALL).build())
)
 }

```

```

 .build())
 .streamSpecification(StreamSpecification.builder()
 .streamEnabled(true)
 .streamViewType(StreamViewType.NEW_AND_OLD_IMAGES)
 .build())
 .build();

 CreateTableResponse response =
dynamoDbClient.createTable(createTableRequest);
 LOGGER.info("Table creation initiated. Status: "
 + response.tableDescription().tableStatus());

 return response;

} catch (DynamoDbException e) {
 LOGGER.severe("Failed to create table: " + tableName + " - " +
e.getMessage());
 throw e;
}
}

```

Wait for a table to become active using AWS SDK for Java 2.x.

```

 public static void waitForTableActive(final DynamoDbClient dynamoDbClient, final
String tableName) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }

 try {
 LOGGER.info("Waiting for table to become active: " + tableName);

 try (DynamoDbWaiter waiter =
 DynamoDbWaiter.builder().client(dynamoDbClient).build()) {
 DescribeTableRequest request =
 DescribeTableRequest.builder().tableName(tableName).build();

```

```

 waiter.waitForTableExists(request);
 LOGGER.info("Table is now active: " + tableName);
 }

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to wait for table to become active: " + tableName
+ " - " + e.getMessage());
 throw e;
 }
}

```

Add a replica to create or extend a global table using AWS SDK for Java 2.x.

```

public static UpdateTableResponse addReplica(
 final DynamoDbClient dynamoDbClient,
 final String tableName,
 final Region replicaRegion,
 final String indexName,
 final Long readCapacity) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (replicaRegion == null) {
 throw new IllegalArgumentException("Replica region cannot be null");
 }
 if (indexName == null || indexName.trim().isEmpty()) {
 throw new IllegalArgumentException("Index name cannot be null or
empty");
 }
 if (readCapacity == null || readCapacity <= 0) {
 throw new IllegalArgumentException("Read capacity must be a positive
number");
 }

 try {
 LOGGER.info("Adding replica in region: " + replicaRegion.id() + " for
table: " + tableName);
 }
}

```

```

 // Create a ReplicationGroupUpdate for adding a replica
 ReplicationGroupUpdate replicationGroupUpdate =
ReplicationGroupUpdate.builder()
 .create(builder -> builder.regionName(replicaRegion.id())
 .globalSecondaryIndexes(ReplicaGlobalSecondaryIndex.builder()
 .indexName(indexName)

.provisionedThroughputOverride(ProvisionedThroughputOverride.builder()
 .readCapacityUnits(readCapacity)
 .build())
 .build())
 .build()
 .build();

 UpdateTableRequest updateTableRequest = UpdateTableRequest.builder()
 .tableName(tableName)
 .replicaUpdates(replicationGroupUpdate)
 .build();

 UpdateTableResponse response =
dynamoDbClient.updateTable(updateTableRequest);
 LOGGER.info("Replica addition initiated in region: " +
replicaRegion.id());

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to add replica in region: " + replicaRegion.id() +
" - " + e.getMessage());
 throw e;
 }
}

```

## Remove a replica from a global table using AWS SDK for Java 2.x.

```

public static UpdateTableResponse removeReplica(
 final DynamoDbClient dynamoDbClient, final String tableName, final Region
replicaRegion) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
}

```

```

 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (replicaRegion == null) {
 throw new IllegalArgumentException("Replica region cannot be null");
 }

 try {
 LOGGER.info("Removing replica in region: " + replicaRegion.id() + " for
table: " + tableName);

 // Create a ReplicationGroupUpdate for removing a replica
 ReplicationGroupUpdate replicationGroupUpdate =
ReplicationGroupUpdate.builder()
 .delete(builder -> builder.regionName(replicaRegion.id()).build())
 .build();

 UpdateTableRequest updateTableRequest = UpdateTableRequest.builder()
 .tableName(tableName)
 .replicaUpdates(replicationGroupUpdate)
 .build();

 UpdateTableResponse response =
dynamoDbClient.updateTable(updateTableRequest);
 LOGGER.info("Replica removal initiated in region: " +
replicaRegion.id());

 return response;

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to remove replica in region: " +
replicaRegion.id() + " - " + e.getMessage());
 throw e;
 }
}

```

Add test items to verify replication using AWS SDK for Java 2.x.

```
public static PutItemResponse putTestItem(
```

```
final DynamoDbClient dynamoDbClient, final String tableName, final String
artist, final String songTitle) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }
 if (artist == null || artist.trim().isEmpty()) {
 throw new IllegalArgumentException("Artist cannot be null or empty");
 }
 if (songTitle == null || songTitle.trim().isEmpty()) {
 throw new IllegalArgumentException("Song title cannot be null or
empty");
 }

 try {
 LOGGER.info("Adding test item to table: " + tableName);

 Map<String,
software.amazon.awssdk.services.dynamodb.model.AttributeValue> item = new
HashMap<>();
 item.put(
 "Artist",

software.amazon.awssdk.services.dynamodb.model.AttributeValue.builder()
 .s(artist)
 .build());
 item.put(
 "SongTitle",

software.amazon.awssdk.services.dynamodb.model.AttributeValue.builder()
 .s(songTitle)
 .build());

 PutItemRequest putItemRequest =
 PutItemRequest.builder().tableName(tableName).item(item).build();

 PutItemResponse response = dynamoDbClient.putItem(putItemRequest);
 LOGGER.info("Test item added successfully");

 return response;
 }
}
```

```

 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to add test item to table: " + tableName + " - " +
e.getMessage());
 throw e;
 }
 }
}

```

Describe global table configuration and replicas using AWS SDK for Java 2.x.

```

public static DescribeTableResponse describeTable(final DynamoDbClient
dynamoDbClient, final String tableName) {

 if (dynamoDbClient == null) {
 throw new IllegalArgumentException("DynamoDB client cannot be null");
 }
 if (tableName == null || tableName.trim().isEmpty()) {
 throw new IllegalArgumentException("Table name cannot be null or
empty");
 }

 try {
 LOGGER.info("Describing table: " + tableName);

 DescribeTableRequest request =
 DescribeTableRequest.builder().tableName(tableName).build();

 DescribeTableResponse response = dynamoDbClient.describeTable(request);

 LOGGER.info("Table status: " + response.table().tableStatus());
 if (response.table().replicas() != null
 && !response.table().replicas().isEmpty()) {
 LOGGER.info("Number of replicas: " +
response.table().replicas().size());
 response.table()
 .replicas()
 .forEach(replica -> LOGGER.info(
 "Replica region: " + replica.regionName() + ", Status: " +
replica.replicaStatus()));
 }

 return response;
 }
}

```

```

 } catch (ResourceNotFoundException e) {
 LOGGER.severe("Table not found: " + tableName + " - " + e.getMessage());
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.severe("Failed to describe table: " + tableName + " - " +
e.getMessage());
 throw e;
 }
}

```

## Complete example of global table operations using AWS SDK for Java 2.x.

```

public static void exampleUsage(final Region sourceRegion, final Region
replicaRegion) {

 String tableName = "Music";
 String indexName = "SongTitleIndex";
 Long readCapacity = 15L;

 // Create DynamoDB client for the source region
 try (DynamoDbClient dynamoDbClient =
 DynamoDbClient.builder().region(sourceRegion).build()) {

 try {
 // Step 1: Create the initial table with GSI and streams
 LOGGER.info("Step 1: Creating table in source region: " +
sourceRegion.id());
 createTableWithGSI(dynamoDbClient, tableName, indexName);

 // Step 2: Wait for table to become active
 LOGGER.info("Step 2: Waiting for table to become active");
 waitForTableActive(dynamoDbClient, tableName);

 // Step 3: Add replica in destination region
 LOGGER.info("Step 3: Adding replica in region: " +
replicaRegion.id());
 addReplica(dynamoDbClient, tableName, replicaRegion, indexName,
readCapacity);

 // Step 4: Wait a moment for replica creation to start
 Thread.sleep(5000);

```



```

 // Step 5: Describe table to view replica information
 LOGGER.info("Step 5: Describing table to view replicas");
 describeTable(dynamoDbClient, tableName);

 // Step 6: Add a test item to verify replication
 LOGGER.info("Step 6: Adding test item to verify replication");
 putTestItem(dynamoDbClient, tableName, "TestArtist", "TestSong");

 LOGGER.info("Global table setup completed successfully!");
 LOGGER.info("You can verify replication by checking the item in
region: " + replicaRegion.id());

 // Step 7: Remove replica and clean up table
 LOGGER.info("Step 7: Removing replica from region: " +
replicaRegion.id());
 removeReplica(dynamoDbClient, tableName, replicaRegion);
 DeleteTableResponse deleteTableResponse =
dynamoDbClient.deleteTable(
 DeleteTableRequest.builder().tableName(tableName).build());
 LOGGER.info("MREC global table demonstration completed
successfully!");

 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException("Thread was interrupted", e);
 } catch (DynamoDbException e) {
 LOGGER.severe("DynamoDB operation failed: " + e.getMessage());
 throw e;
 }
}
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateTable](#)
  - [DescribeTable](#)
  - [PutItem](#)
  - [UpdateTable](#)

## Detect PPE in images

The following code example shows how to build an app that uses Amazon Rekognition to detect Personal Protective Equipment (PPE) in images.

### SDK for Java 2.x

Shows how to create an AWS Lambda function that detects images with Personal Protective Equipment.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB
- Amazon Rekognition
- Amazon S3
- Amazon SES

## Monitor DynamoDB performance

The following code example shows how to configure an application's use of DynamoDB to monitor performance.

### SDK for Java 2.x

This example shows how to configure a Java application to monitor the performance of DynamoDB. The application sends metric data to CloudWatch where you can monitor the performance.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- CloudWatch
- DynamoDB

## Perform advanced query operations

The following code example shows how to perform advanced query operations in DynamoDB.

- Query tables using various filtering and condition techniques.
- Implement pagination for large result sets.
- Use Global Secondary Indexes for alternate access patterns.
- Apply consistency controls based on application requirements.

### SDK for Java 2.x

Query with strongly consistent reads using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithConsistentReads(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final boolean useConsistentRead) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
```

```

 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

// Create the query request
final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .consistentRead(useConsistentRead)
 .build();

try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query successful. Found {0} items",
response.count());
 return response;
} catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
} catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying with consistent reads", e);
 throw e;
}
}

```

## Query using a Global Secondary Index with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;

 public QueryResponse queryTable(

```

```

 final String tableName, final String partitionKeyName, final String
partitionKeyValue) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query on base table successful. Found " +
response.count() + " items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw new DynamoDbQueryException("Table not found: " + tableName, e);
 } catch (DynamoDbException e) {
 System.err.println("Error querying base table: " + e.getMessage());
 throw new DynamoDbQueryException("Failed to execute query on base
table", e);
 }
}

/**
 * Queries a DynamoDB Global Secondary Index (GSI) by partition key.

```

```

*
* @param tableName The name of the DynamoDB table
* @param indexName The name of the GSI
* @param partitionKeyName The name of the GSI partition key attribute
* @param partitionKeyValue The value of the GSI partition key to query
* @return The query response from DynamoDB
* @throws ResourceNotFoundException if the table or index doesn't exist
* @throws DynamoDbException if the query fails
*/
public QueryResponse queryGlobalSecondaryIndex(
 final String tableName, final String indexName, final String
partitionKeyName, final String partitionKeyValue) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Index name", indexName);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_IK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_IK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .indexName(indexName)
 .keyConditionExpression(GSI_KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query on GSI successful. Found " + response.count()
+ " items");
 return response;
 } catch (ResourceNotFoundException e) {

```

```

 System.err.format(
 "Error: The Amazon DynamoDB table \"%s\" or index \"%s\" can't be
found.\n", tableName, indexName);
 throw new DynamoDbQueryException("Table or index not found: " +
tableName + "/" + indexName, e);
 } catch (DynamoDbException e) {
 System.err.println("Error querying GSI: " + e.getMessage());
 throw new DynamoDbQueryException("Failed to execute query on GSI", e);
 }
}

```

## Query with pagination using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

 public List<Map<String, AttributeValue>> queryWithPagination(
 final String tableName, final String partitionKeyName, final String
partitionKeyValue, final int pageSize) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validatePositiveInteger("Page size", pageSize);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();

```

```

 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

// Create the query request
QueryRequest.Builder queryRequestBuilder = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .limit(pageSize);

// List to store all items from all pages
final List<Map<String, AttributeValue>> allItems = new ArrayList<>();

// Map to store the last evaluated key for pagination
Map<String, AttributeValue> lastEvaluatedKey = null;
int pageNumber = 1;

try {
 do {
 // If we have a last evaluated key, use it for the next page
 if (lastEvaluatedKey != null) {
 queryRequestBuilder.exclusiveStartKey(lastEvaluatedKey);
 }

 // Execute the query
 final QueryResponse response =
dynamoDbClient.query(queryRequestBuilder.build());

 // Process the current page of results
 final List<Map<String, AttributeValue>> pageItems =
response.items();
 allItems.addAll(pageItems);

 // Get the last evaluated key for the next page
 lastEvaluatedKey = response.lastEvaluatedKey();
 if (lastEvaluatedKey != null && lastEvaluatedKey.isEmpty()) {
 lastEvaluatedKey = null;
 }

 System.out.println("Page " + pageNumber + ": Retrieved " +
pageItems.size() + " items (Running total: "
 + allItems.size() + ")");
 } while (lastEvaluatedKey != null);
} catch (Exception e) {
 e.printStackTrace();
}

```



```

 pageNumber++;

 } while (lastEvaluatedKey != null);

 System.out.println("Query with pagination complete. Retrieved a total of
" + allItems.size()
 + " items across " + (pageNumber - 1) + " pages");

 return allItems;
} catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw e;
} catch (DynamoDbException e) {
 System.err.println("Error querying with pagination: " + e.getMessage());
 throw e;
}
}

```

## Query with complex filters using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithComplexFilter(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String statusAttrName,
 final String activeStatus,
 final String pendingStatus,

```

```

 final String priceAttrName,
 final double minPrice,
 final double maxPrice,
 final String categoryAttrName) {

 // Validate parameters
 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyKeyValue);
 CodeSampleUtils.validateStringParameter("Status attribute name",
statusAttrName);
 CodeSampleUtils.validateStringParameter("Active status", activeStatus);
 CodeSampleUtils.validateStringParameter("Pending status", pendingStatus);
 CodeSampleUtils.validateStringParameter("Price attribute name",
priceAttrName);
 CodeSampleUtils.validateStringParameter("Category attribute name",
categoryAttrName);
 CodeSampleUtils.validateNumericRange("Minimum price", minPrice, 0.0,
Double.MAX_VALUE);
 CodeSampleUtils.validateNumericRange("Maximum price", maxPrice, minPrice,
Double.MAX_VALUE);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#pk", partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_STATUS,
statusAttrName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PRICE,
priceAttrName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_CATEGORY,
categoryAttrName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 ":pkValue", AttributeValue.builder().s(partitionKeyKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_ACTIVE,
 AttributeValue.builder().s(activeStatus).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PENDING,
 AttributeValue.builder().s(pendingStatus).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_MIN_PRICE,

```

```

 AttributeValue.builder().n(String.valueOf(minPrice)).build());
expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_MAX_PRICE,
 AttributeValue.builder().n(String.valueOf(maxPrice)).build());

// Create the query request
final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(FILTER_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

return dynamoDbClient.query(queryRequest);
}

```

## Query with a dynamically constructed filter expression using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;

public static QueryResponse queryWithDynamicFilter(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final Map<String, Object> filterCriteria,
 final Region region,
 final DynamoDbClient dynamoDbClient) {

 validateParameters(tableName, partitionKeyName, partitionKeyValue,
 filterCriteria);

 DynamoDbClient ddbClient = dynamoDbClient;

```

```

 boolean shouldClose = false;

 try {
 if (ddbClient == null) {
 ddbClient = createClient(region);
 shouldClose = true;
 }

 final QueryWithDynamicFilter queryHelper = new
QueryWithDynamicFilter(ddbClient);
 return queryHelper.queryWithDynamicFilter(tableName, partitionKeyName,
partitionKeyValue, filterCriteria);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println("Failed to execute dynamic filter query: " +
e.getMessage());
 throw e;
 } catch (Exception e) {
 System.err.println("Unexpected error during query: " + e.getMessage());
 throw e;
 } finally {
 if (shouldClose && ddbClient != null) {
 ddbClient.close();
 }
 }
 }

 public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <tableName> <partitionKeyName> <partitionKeyValue>
<filterAttrName> <filterAttrValue> [region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 partitionKeyName - The name of the partition key attribute.
 partitionKeyValue - The value of the partition key to query.
 filterAttrName - The name of the attribute to filter on.
 filterAttrValue - The value to filter by.
 region (optional) - The AWS region where the table exists.
 (Default: us-east-1)
 """;

```

```
 if (args.length < 5) {
 System.out.println(usage);
 System.exit(1);
 }

 final String tableName = args[0];
 final String partitionKeyName = args[1];
 final String partitionKeyValue = args[2];
 final String filterAttrName = args[3];
 final String filterAttrValue = args[4];
 final Region region = args.length > 5 ? Region.of(args[5]) :
Region.US_EAST_1;

 System.out.println("Querying items with dynamic filter: " + filterAttrName +
" = " + filterAttrValue);

 try {
 // Using the builder pattern to create and execute the query
 final QueryResponse response = new DynamicFilterQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withFilterCriterion(filterAttrName, filterAttrValue)
 .withRegion(region)
 .execute();

 // Process the results
 System.out.println("Found " + response.count() + " items:");
 response.items().forEach(item -> System.out.println(item));

 // Demonstrate multiple filter criteria
 System.out.println("\nNow querying with multiple filter criteria:");

 Map<String, Object> multipleFilters = new HashMap<>();
 multipleFilters.put(filterAttrName, filterAttrValue);
 multipleFilters.put("status", "active");

 final QueryResponse multiFilterResponse = new
DynamicFilterQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withFilterCriteria(multipleFilters)
```

```

 .withRegion(region)
 .execute();

 System.out.println("Found " + multiFilterResponse.count() + " items with
multiple filters:");
 multiFilterResponse.items().forEach(item -> System.out.println(item));

 } catch (IllegalArgumentException e) {
 System.err.println("Invalid input: " + e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println("DynamoDB error: " + e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 System.exit(1);
 }
}

```

## Query with a filter expression and limit using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

 public QueryResponse queryWithFilterAndLimit(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String filterAttrName,

```

```
 final String filterAttrValue,
 final int limit) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Filter attribute name",
filterAttrName);
 CodeSampleUtils.validateStringParameter("Filter attribute value",
filterAttrValue);
 CodeSampleUtils.validatePositiveInteger("Limit", limit);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_FILTER,
filterAttrName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_FILTER,
 AttributeValue.builder().s(filterAttrValue).build());

 // Create the filter expression
 final String filterExpression = "#filterAttr = :filterValue";

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(filterExpression)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .limit(limit)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
```

```

 LOGGER.log(Level.INFO, "Query with filter and limit successful. Found
{0} items", response.count());
 LOGGER.log(
 Level.INFO, "ScannedCount: {0} (total items evaluated before
filtering)", response.scannedCount());
 return response;
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying with filter and limit: {0}",
e.getMessage());
 throw e;
 }
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Perform list operations

The following code example shows how to perform list operations in DynamoDB.

- Add elements to a list attribute.
- Remove elements from a list attribute.
- Update specific elements in a list by index.
- Use list append and list index functions.

## SDK for Java 2.x

Demonstrate list operations using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.ArrayList;

```



```

import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Appends items to a list attribute.
 *
 * <p>This method demonstrates how to use the list_append function to add
 * items to the end of a list attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param listAttributeName The name of the list attribute
 * @param itemsToAppend The items to append to the list
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse appendToList(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String listAttributeName,
 List<AttributeValue> itemsToAppend) {

 // Create a list value from the items to append
 AttributeValue listValue =
AttributeValue.builder().l(itemsToAppend).build();

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #attrName =
list_append(if_not_exists(#attrName, :emptyList), :newItems)")
 .expressionAttributeNames(Map.of("#attrName", listAttributeName))
 .expressionAttributeValues(Map.of(
 ":newItems",
 listValue,
 ":emptyList",
 AttributeValue.builder().l(new
ArrayList<AttributeValue>()).build()))
 .returnValues("UPDATED_NEW")
 .build();

```

```

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Prepends items to a list attribute.
 *
 * <p>This method demonstrates how to use the list_append function to add
 * items to the beginning of a list attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param listAttributeName The name of the list attribute
 * @param itemsToPrepend The items to prepend to the list
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse prependToList(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String listAttributeName,
 List<AttributeValue> itemsToPrepend) {

 // Create a list value from the items to prepend
 AttributeValue listValue =
 AttributeValue.builder().l(itemsToPrepend).build();

 // Define the update parameters
 // Note: To prepend, we put the new items first in the list_append function
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #attrName = list_append(:newItems,
if_not_exists(#attrName, :emptyList))")
 .expressionAttributeNames(Map.of("#attrName", listAttributeName))
 .expressionAttributeValues(Map.of(
 ":newItems",
 listValue,
 ":emptyList",
 AttributeValue.builder().l(new
 ArrayList<AttributeValue>()).build()))

```

```

 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Updates a specific element in a list attribute.
 *
 * <p>This method demonstrates how to update a specific element in a list
 * by its index.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param listAttributeName The name of the list attribute
 * @param index The index of the element to update
 * @param newValue The new value for the element
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateListElement(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String listAttributeName,
 int index,
 AttributeValue newValue) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #attrName[" + index + "] = :newValue")
 .expressionAttributeNames(Map.of("#attrName", listAttributeName))
 .expressionAttributeValues(Map.of(":newValue", newValue))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

```

```

/**
 * Removes a specific element from a list attribute.
 *
 * <p>This method demonstrates how to remove a specific element from a list
 * by its index.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param listAttributeName The name of the list attribute
 * @param index The index of the element to remove
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse removeListElement(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String listAttributeName,
 int index) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("REMOVE #attrName[" + index + "]")
 .expressionAttributeNames(Map.of("#attrName", listAttributeName))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Gets the current value of a list attribute.
 *
 * <p>Helper method to retrieve the current value of a list attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to get
 * @param listAttributeName The name of the list attribute
 * @return The list attribute value or null if not found

```

```

 * @throws DynamoDbException if an error occurs during the operation
 */
 public static List<AttributeValue> getListAttribute(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
 key, String listAttributeName) {

 // Define the get parameters
 GetItemRequest request = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .projectionExpression(listAttributeName)
 .build();

 try {
 // Perform the get operation
 GetItemResponse response = dynamoDbClient.getItem(request);

 // Return the list attribute if it exists, otherwise null
 if (response.item() != null &&
 response.item().containsKey(listAttributeName)) {
 return response.item().get(listAttributeName).l();
 }

 return null;
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to get list attribute: " + e.getMessage())
 .cause(e)
 .build();
 }
 }
}

```

### Example usage of list operations with AWS SDK for Java 2.x.

```

public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating list operations in DynamoDB");
}

```

```
try {
 // Example 1: Append items to a list
 System.out.println("\nExample 1: Appending items to a list");
 List<AttributeValue> tagsToAppend = List.of(
 AttributeValue.builder().s("Electronics").build(),
 AttributeValue.builder().s("Gadget").build());

 UpdateItemResponse appendResponse = appendToList(dynamoDbClient,
 tableName, key, "Tags", tagsToAppend);

 System.out.println("Updated list attribute: " +
 appendResponse.attributes());

 // Example 2: Prepend items to a list
 System.out.println("\nExample 2: Prepending items to a list");
 List<AttributeValue> tagsToPrepend = List.of(
 AttributeValue.builder().s("Featured").build(),
 AttributeValue.builder().s("New").build());

 UpdateItemResponse prependResponse = prependToList(dynamoDbClient,
 tableName, key, "Tags", tagsToPrepend);

 System.out.println("Updated list attribute: " +
 prependResponse.attributes());

 // Example 3: Update a specific element in a list
 System.out.println("\nExample 3: Updating a specific element in a
list");
 UpdateItemResponse updateResponse = updateListElement(
 dynamoDbClient,
 tableName,
 key,
 "Tags",
 0,
 AttributeValue.builder().s("BestSeller").build());

 System.out.println("Updated list attribute: " +
 updateResponse.attributes());

 // Example 4: Remove a specific element from a list
 System.out.println("\nExample 4: Removing a specific element from a
list");
 UpdateItemResponse removeResponse = removeListElement(dynamoDbClient,
 tableName, key, "Tags", 1);
```

```

 System.out.println("Updated list attribute: " +
removeResponse.attributes());

 // Example 5: Get the current value of a list attribute
 System.out.println("\nExample 5: Getting the current value of a list
attribute");
 List<AttributeValue> currentList = getListAttribute(dynamoDbClient,
tableName, key, "Tags");

 if (currentList != null) {
 System.out.println("Current list attribute:");
 for (int i = 0; i < currentList.size(); i++) {
 System.out.println(" [" + i + "]: " + currentList.get(i).s());
 }
 } else {
 System.out.println("List attribute not found");
 }

 // Explain list operations
 System.out.println("\nKey points about DynamoDB list operations:");
 System.out.println("1. Lists are ordered collections of attributes");
 System.out.println("2. Use list_append to add items to a list");
 System.out.println("3. To append items, use list_append(existingList,
newItems)");
 System.out.println("4. To prepend items, use list_append(newItems,
existingList)");
 System.out.println("5. Use index notation (list[0]) to access or update
specific elements");
 System.out.println("6. Use REMOVE to delete elements from a list");
 System.out.println("7. List indices are zero-based");
 System.out.println("8. Use if_not_exists to handle the case where the
list doesn't exist yet");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
 }
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Perform map operations

The following code example shows how to perform map operations in DynamoDB.

- Add and update nested attributes in map structures.
- Remove specific fields from maps.
- Work with deeply nested map attributes.

### SDK for Java 2.x

Demonstrate map operations using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Updates a map attribute that may not exist.
 *
 * <p>This method demonstrates how to safely update a map attribute
 * by using if_not_exists to handle the case where the map doesn't exist yet.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param mapName The name of the map attribute
 * @param mapKey The key within the map to update
 * @param value The value to set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateMapAttributeSafe(
 DynamoDbClient dynamoDbClient,
 String tableName,
```



```

 Map<String, AttributeValue> key,
 String mapName,
 String mapKey,
 AttributeValue value) {

 // Create an empty map to use if the map doesn't exist
 Map<String, AttributeValue> emptyMap = new HashMap<>();
 AttributeValue emptyMapValue = AttributeValue.builder().m(emptyMap).build();

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #mapName = if_not_exists(#mapName, :emptyMap),
#mapName.#mapKey = :value")
 .expressionAttributeNames(Map.of(
 "#mapName", mapName,
 "#mapKey", mapKey))
 .expressionAttributeValues(Map.of(
 ":value",
 value,
 ":emptyMap",
 AttributeValue.builder().m(new HashMap<>()).build()))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Adds an attribute to a nested map.
 *
 * <p>This method demonstrates how to update a nested attribute without
 * overwriting the entire map.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param path The path to the nested attribute as a list
 * @param value The value to set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */

```

```

public static UpdateItemResponse addToNestedMap(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 List<String> path,
 AttributeValue value) {

 // Create expression attribute names for each part of the path
 Map<String, String> expressionAttributeNames = new HashMap<>();
 for (int i = 0; i < path.size(); i++) {
 expressionAttributeNames.put("#attr" + i, path.get(i));
 }

 // Build the attribute path using the expression attribute names
 StringBuilder attributePathExpression = new StringBuilder();
 for (int i = 0; i < path.size(); i++) {
 if (i > 0) {
 attributePathExpression.append(".");
 }
 attributePathExpression.append("#attr").append(i);
 }

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET " + attributePathExpression.toString() + "
= :value")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(Map.of(":value", value))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Removes an attribute from a map.
 *
 * <p>This method demonstrates how to remove a specific attribute from a map.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table

```

```

 * @param key The key of the item to update
 * @param mapName The name of the map attribute
 * @param mapKey The key within the map to remove
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse removeMapAttribute(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String mapName,
 String mapKey) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("REMOVE #mapName.#mapKey")
 .expressionAttributeNames(Map.of(
 "#mapName", mapName,
 "#mapKey", mapKey))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Creates a map with multiple attributes in a single operation.
 *
 * <p>This method demonstrates how to create a map with multiple attributes
 * in a single update operation.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param mapName The name of the map attribute
 * @param attributes The attributes to set in the map
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse createMapWithAttributes(
 DynamoDbClient dynamoDbClient,

```

```

 String tableName,
 Map<String, AttributeValue> key,
 String mapName,
 Map<String, AttributeValue> attributes) {

 // Create a map value from the attributes
 AttributeValue mapValue = AttributeValue.builder().m(attributes).build();

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #mapName = :mapValue")
 .expressionAttributeNames(Map.of("#mapName", mapName))
 .expressionAttributeValues(Map.of(":mapValue", mapValue))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Gets the current value of a map attribute.
 *
 * <p>Helper method to retrieve the current value of a map attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to get
 * @param mapName The name of the map attribute
 * @return The map attribute value or null if not found
 * @throws DynamoDbException if an error occurs during the operation
 */
public static Map<String, AttributeValue> getMapAttribute(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
 key, String mapName) {

 // Define the get parameters
 GetItemRequest request = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .projectionExpression(mapName)
 .build();

```

```

 try {
 // Perform the get operation
 GetItemResponse response = dynamoDbClient.getItem(request);

 // Return the map attribute if it exists, otherwise null
 if (response.item() != null && response.item().containsKey(mapName)) {
 return response.item().get(mapName).m();
 }

 return null;
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to get map attribute: " + e.getMessage())
 .cause(e)
 .build();
 }
}

```

### Example usage of map operations with AWS SDK for Java 2.x.

```

public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating map operations in DynamoDB");

 try {
 // Example 1: Create a map with multiple attributes
 System.out.println("\nExample 1: Creating a map with multiple
attributes");
 Map<String, AttributeValue> productDetails = new HashMap<>();
 productDetails.put("Color", AttributeValue.builder().s("Red").build());
 productDetails.put("Weight", AttributeValue.builder().n("2.5").build());
 productDetails.put(
 "Dimensions", AttributeValue.builder().s("10x20x5").build());

 UpdateItemResponse createResponse =
 createMapWithAttributes(dynamoDbClient, tableName, key, "Details",
productDetails);
 }
}

```

```
 System.out.println("Created map attribute: " +
createResponse.attributes());

 // Example 2: Update a specific attribute in a map
 System.out.println("\nExample 2: Updating a specific attribute in a
map");
 UpdateItemResponse updateResponse = updateMapAttributeSafe(
 dynamoDbClient,
 tableName,
 key,
 "Details",
 "Color",
 AttributeValue.builder().s("Blue").build());

 System.out.println("Updated map attribute: " +
updateResponse.attributes());

 // Example 3: Add an attribute to a nested map
 System.out.println("\nExample 3: Adding an attribute to a nested map");
 UpdateItemResponse nestedResponse = addToNestedMap(
 dynamoDbClient,
 tableName,
 key,
 List.of("Specifications", "Technical", "Resolution"),
 AttributeValue.builder().s("1920x1080").build());

 System.out.println("Added to nested map: " +
nestedResponse.attributes());

 // Example 4: Remove an attribute from a map
 System.out.println("\nExample 4: Removing an attribute from a map");
 UpdateItemResponse removeResponse =
 removeMapAttribute(dynamoDbClient, tableName, key, "Details",
"Dimensions");

 System.out.println("Updated map after removal: " +
removeResponse.attributes());

 // Example 5: Get the current value of a map attribute
 System.out.println("\nExample 5: Getting the current value of a map
attribute");
 Map<String, AttributeValue> currentMap = getMapAttribute(dynamoDbClient,
tableName, key, "Details");
```

```

 if (currentMap != null) {
 System.out.println("Current map attribute:");
 for (Map.Entry<String, AttributeValue> entry :
currentMap.entrySet()) {
 System.out.println(" " + entry.getKey() + ": " +
entry.getValue());
 }
 } else {
 System.out.println("Map attribute not found");
 }

 // Explain map operations
 System.out.println("\nKey points about DynamoDB map operations:");
 System.out.println("1. Maps are unordered collections of name-value
pairs");
 System.out.println("2. Use dot notation (map.key) to access or update
specific attributes");
 System.out.println("3. You can update individual attributes without
overwriting the entire map");
 System.out.println("4. Maps can be nested to create complex data
structures");
 System.out.println("5. Use REMOVE to delete attributes from a map");
 System.out.println("6. You can create a map with multiple attributes in
a single operation");
 System.out.println("7. Map keys are case-sensitive");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Perform set operations

The following code example shows how to perform set operations in DynamoDB.

- Add elements to a set attribute.
- Remove elements from a set attribute.

- Use ADD and DELETE operations with sets.

## SDK for Java 2.x

Demonstrate set operations using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ReturnValue;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * Adds values to a string set attribute.
 *
 * <p>This method demonstrates how to use the ADD operation to add values
 * to a string set attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param setAttributeName The name of the set attribute
 * @param valuesToAdd The values to add to the set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse addToStringSet(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String setAttributeName,
 Set<String> valuesToAdd) {

 // Create a string set value from the values to add
 AttributeValue setValue = AttributeValue.builder().ss(valuesToAdd).build();
```



```

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("ADD #setAttr :valuesToAdd")
 .expressionAttributeNames(Map.of("#setAttr", setAttributeName))
 .expressionAttributeValues(Map.of(":valuesToAdd", setValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Adds values to a number set attribute.
 *
 * <p>This method demonstrates how to use the ADD operation to add values
 * to a number set attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param setAttributeName The name of the set attribute
 * @param valuesToAdd The values to add to the set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse addToNumberSet(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String setAttributeName,
 Set<Number> valuesToAdd) {

 // Convert numbers to strings for DynamoDB
 Set<String> stringValueSet = new HashSet<>();
 for (Number value : valuesToAdd) {
 stringValueSet.add(value.toString());
 }

 // Create a number set value from the values to add
 AttributeValue setValue = AttributeValue.builder().ns(stringValueSet).build();
 }

```

```

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("ADD #setAttr :valuesToAdd")
 .expressionAttributeNames(Map.of("#setAttr", setAttributeName))
 .expressionAttributeValues(Map.of(":valuesToAdd", setValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Removes values from a set attribute.
 *
 * <p>This method demonstrates how to use the DELETE operation to remove values
 * from a set attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param setAttributeName The name of the set attribute
 * @param valuesToRemove The values to remove from the set
 * @param isNumberSet Whether the set is a number set (true) or string set
 * (false)
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse removeFromSet(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String setAttributeName,
 Set<?> valuesToRemove,
 boolean isNumberSet) {

 AttributeValue setValue;

 if (isNumberSet) {
 // Convert numbers to strings for DynamoDB
 Set<String> stringValues = new HashSet<>();

```

```

 for (Object value : valuesToRemove) {
 if (value instanceof Number) {
 stringValue.add(value.toString());
 } else {
 throw new IllegalArgumentException("Values must be numbers for a
number set");
 }
 }

 setValue = AttributeValue.builder().ns(stringValue).build();
 } else {
 // Convert objects to strings for DynamoDB
 Set<String> stringValue = new HashSet<>();
 for (Object value : valuesToRemove) {
 stringValue.add(value.toString());
 }

 setValue = AttributeValue.builder().ss(stringValue).build();
 }

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("DELETE #setAttr :valuesToRemove")
 .expressionAttributeNames(Map.of("#setAttr", setAttributeName))
 .expressionAttributeValues(Map.of(":valuesToRemove", setValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Checks if a value exists in a set attribute.
 *
 * <p>This method demonstrates how to use the contains function to check
 * if a value exists in a set attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to check
 * @param setAttributeName The name of the set attribute

```

```

 * @param valueToCheck The value to check for
 * @return Map containing the result of the check
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static Map<String, Object> checkIfValueInSet(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String setAttributeName,
 String valueToCheck) {

 Map<String, Object> result = new HashMap<>();

 try {
 // Define the update parameters with a condition expression
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #tempAttr = :tempVal")
 .conditionExpression("contains(#setAttr, :valueToCheck)")
 .expressionAttributeNames(Map.of("#setAttr", setAttributeName,
"#tempAttr", "TempAttribute"))
 .expressionAttributeValues(Map.of(
 ":valueToCheck",
AttributeValue.builder().s(valueToCheck).build(),
 ":tempVal", AttributeValue.builder().s("TempValue").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Attempt the update operation
 dynamoDbClient.updateItem(request);

 // If we get here, the condition was met
 result.put("exists", true);
 result.put("message", "Value '" + valueToCheck + "' exists in the set");

 // Clean up the temporary attribute
 UpdateItemRequest cleanupRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("REMOVE #tempAttr")
 .expressionAttributeNames(Map.of("#tempAttr", "TempAttribute"))
 .build();

```

```

 dynamoDbClient.updateItem(cleanupRequest);

 } catch (DynamoDbException e) {
 if (e.getMessage().contains("ConditionalCheckFailed")) {
 // The condition was not met
 result.put("exists", false);
 result.put("message", "Value '" + valueToCheck + "' does not exist
in the set");
 } else {
 // Some other error occurred
 result.put("exists", false);
 result.put("message", "Error checking set: " + e.getMessage());
 result.put("error", e.getClass().getSimpleName());
 }
 }

 return result;
}

/**
 * Creates a set with multiple values in a single operation.
 *
 * <p>This method demonstrates how to create a set with multiple values
 * in a single update operation.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param setAttributeName The name of the set attribute
 * @param setValues The values to include in the set
 * @param isNumberSet Whether to create a number set (true) or string set
 * (false)
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse createSetWithValues(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String setAttributeName,
 Set<?> setValues,
 boolean isNumberSet) {

 AttributeValue setValue;

```

```

 if (isNumberSet) {
 // Convert numbers to strings for DynamoDB
 Set<String> stringValueSet = new HashSet<>();
 for (Object value : setValues) {
 if (value instanceof Number) {
 stringValueSet.add(value.toString());
 } else {
 throw new IllegalArgumentException("Values must be numbers for a
number set");
 }
 }

 setValue = AttributeValue.builder().ns(stringValueSet).build();
 } else {
 // Convert objects to strings for DynamoDB
 Set<String> stringValueSet = new HashSet<>();
 for (Object value : setValues) {
 stringValueSet.add(value.toString());
 }

 setValue = AttributeValue.builder().ss(stringValueSet).build();
 }

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #setAttr = :setValue")
 .expressionAttributeNames(Map.of("#setAttr", setAttributeName))
 .expressionAttributeValues(Map.of(":setValue", setValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Gets the current value of a set attribute.
 *
 * <p>Helper method to retrieve the current value of a set attribute.
 *
 * @param dynamoDbClient The DynamoDB client

```

```

 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to get
 * @param setAttributeName The name of the set attribute
 * @return The set attribute value or null if not found
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static AttributeValue getSetAttribute(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
 key, String setAttributeName) {

 // Define the get parameters
 GetItemRequest request = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .projectionExpression(setAttributeName)
 .build();

 try {
 // Perform the get operation
 GetItemResponse response = dynamoDbClient.getItem(request);

 // Return the set attribute if it exists, otherwise null
 if (response.item() != null &&
 response.item().containsKey(setAttributeName)) {
 return response.item().get(setAttributeName);
 }

 return null;
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to get set attribute: " + e.getMessage())
 .cause(e)
 .build();
 }
 }
}

```

### Example usage of set operations with AWS SDK for Java 2.x.

```

 public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
 {
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 }

```

```
key.put("ProductId", AttributeValue.builder().s("P12345").build());

System.out.println("Demonstrating set operations in DynamoDB");

try {
 // Example 1: Create a string set with multiple values
 System.out.println("\nExample 1: Creating a string set with multiple
values");
 Set<String> tags = new HashSet<>();
 tags.add("Electronics");
 tags.add("Gadget");
 tags.add("Smartphone");

 UpdateItemResponse createResponse = createSetWithValues(
 dynamoDbClient, tableName, key, "Tags", tags, false // Not a number
set
);

 System.out.println("Created set attribute: " +
createResponse.attributes());

 // Example 2: Add values to a string set
 System.out.println("\nExample 2: Adding values to a string set");
 Set<String> additionalTags = new HashSet<>();
 additionalTags.add("Mobile");
 additionalTags.add("Wireless");

 UpdateItemResponse addResponse = addToStringSet(dynamoDbClient,
tableName, key, "Tags", additionalTags);

 System.out.println("Updated set attribute: " +
addResponse.attributes());

 // Example 3: Create a number set with multiple values
 System.out.println("\nExample 3: Creating a number set with multiple
values");
 Set<Number> ratings = new HashSet<>();
 ratings.add(4);
 ratings.add(5);
 ratings.add(4.5);

 UpdateItemResponse createNumberSetResponse = createSetWithValues(
 dynamoDbClient, tableName, key, "Ratings", ratings, true // Is a
number set
```



```
);

 System.out.println("Created number set attribute: " +
createNumberSetResponse.attributes());

 // Example 4: Add values to a number set
 System.out.println("\nExample 4: Adding values to a number set");
 Set<Number> additionalRatings = new HashSet<>();
 additionalRatings.add(3.5);
 additionalRatings.add(4.2);

 UpdateItemResponse addNumberResponse =
 addToNumberSet(dynamoDbClient, tableName, key, "Ratings",
additionalRatings);

 System.out.println("Updated number set attribute: " +
addNumberResponse.attributes());

 // Example 5: Remove values from a set
 System.out.println("\nExample 5: Removing values from a set");
 Set<String> tagsToRemove = new HashSet<>();
 tagsToRemove.add("Gadget");

 UpdateItemResponse removeResponse = removeFromSet(
 dynamoDbClient, tableName, key, "Tags", tagsToRemove, false // Not a
number set
);

 System.out.println("Updated set after removal: " +
removeResponse.attributes());

 // Example 6: Check if a value exists in a set
 System.out.println("\nExample 6: Checking if a value exists in a set");
 Map<String, Object> checkResult = checkIfValueInSet(dynamoDbClient,
tableName, key, "Tags", "Electronics");

 System.out.println("Check result: " + checkResult.get("message"));

 // Example 7: Get the current value of a set attribute
 System.out.println("\nExample 7: Getting the current value of a set
attribute");
 AttributeValue currentStringSet = getSetAttribute(dynamoDbClient,
tableName, key, "Tags");
```

```

 if (currentStringSet != null && currentStringSet.ss() != null) {
 System.out.println("Current string set values: " +
currentStringSet.ss());
 } else {
 System.out.println("String set attribute not found");
 }

 AttributeValue currentNumberSet = getSetAttribute(dynamoDbClient,
tableName, key, "Ratings");

 if (currentNumberSet != null && currentNumberSet.ns() != null) {
 System.out.println("Current number set values: " +
currentNumberSet.ns());
 } else {
 System.out.println("Number set attribute not found");
 }

 // Explain set operations
 System.out.println("\nKey points about DynamoDB set operations:");
 System.out.println(
 "1. DynamoDB supports three set types: string sets (SS), number sets
(NS), and binary sets (BS)");
 System.out.println("2. Sets can only contain elements of the same
type");
 System.out.println("3. Use ADD to add elements to a set");
 System.out.println("4. Use DELETE to remove elements from a set");
 System.out.println("5. Sets automatically remove duplicate values");
 System.out.println("6. Sets are unordered collections");
 System.out.println("7. Use the contains function to check if a value
exists in a set");
 System.out.println("8. You can create a set with multiple values in a
single operation");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
 }
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table by using batches of PartiQL statements

The following code example shows how to:

- Get a batch of items by running multiple SELECT statements.
- Add a batch of items by running multiple INSERT statements.
- Update a batch of items by running multiple UPDATE statements.
- Delete a batch of items by running multiple DELETE statements.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class ScenarioPartiQLBatch {
 public static void main(String[] args) throws IOException {
 String tableName = "MoviesPartiQBatch";
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 System.out.println("Creating an Amazon DynamoDB table named " + tableName
 + " with a key named year and a sort key named title.");
 createTable(ddb, tableName);

 System.out.println("Adding multiple records into the " + tableName
 + " table using a batch command.");
 putRecordBatch(ddb);

 // Update multiple movies by using the BatchExecute statement.
 String title1 = "Star Wars";
 int year1 = 1977;
 String title2 = "Wizard of Oz";
 int year2 = 1939;

 System.out.println("Query two movies.");
```

```

 getBatch(ddb, tableName, title1, title2, year1, year2);

 System.out.println("Updating multiple records using a batch command.");
 updateTableItemBatch(ddb);

 System.out.println("Deleting multiple records using a batch command.");
 deleteItemBatch(ddb);

 System.out.println("Deleting the Amazon DynamoDB table.");
 deleteDynamoDBTable(ddb, tableName);
 ddb.close();
 }

 public static boolean getBatch(DynamoDbClient ddb, String tableName, String
title1, String title2, int year1, int year2) {
 String getBatch = "SELECT * FROM " + tableName + " WHERE title = ? AND year
= ?";

 List<BatchStatementRequest> statements = new ArrayList<>();
 statements.add(BatchStatementRequest.builder()
 .statement(getBatch)
 .parameters(AttributeValue.builder().s(title1).build(),
 AttributeValue.builder().n(String.valueOf(year1)).build())
 .build());
 statements.add(BatchStatementRequest.builder()
 .statement(getBatch)
 .parameters(AttributeValue.builder().s(title2).build(),
 AttributeValue.builder().n(String.valueOf(year2)).build())
 .build());

 BatchExecuteStatementRequest batchExecuteStatementRequest =
BatchExecuteStatementRequest.builder()
 .statements(statements)
 .build();

 try {
 BatchExecuteStatementResponse response =
ddb.batchExecuteStatement(batchExecuteStatementRequest);
 if (!response.responses().isEmpty()) {
 response.responses().forEach(r -> {
 System.out.println(r.item().get("title") + "\\t" +
r.item().get("year"));
 });
 }
 return true;
 }
 }

```

```

 } else {
 System.out.println("Couldn't find either " + title1 + " or " +
title2 + ".");
 return false;
 }
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 return false;
 }
}

public static void createTable(DynamoDbClient ddb, String tableName) {
 DynamoDbWaiter dbWaiter = ddb.waiter();
 ArrayList<AttributeDefinition> attributeDefinitions = new ArrayList<>();

 // Define attributes.
 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("year")
 .attributeType("N")
 .build());

 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("title")
 .attributeType("S")
 .build());

 ArrayList<KeySchemaElement> tableKey = new ArrayList<>();
 KeySchemaElement key = KeySchemaElement.builder()
 .attributeName("year")
 .keyType(KeyType.HASH)
 .build();

 KeySchemaElement key2 = KeySchemaElement.builder()
 .attributeName("title")
 .keyType(KeyType.RANGE) // Sort
 .build();

 // Add KeySchemaElement objects to the list.
 tableKey.add(key);
 tableKey.add(key2);

 CreateTableRequest request = CreateTableRequest.builder()
 .keySchema(tableKey)

```

```

 .billingMode(BillingMode.PAY_PER_REQUEST) // DynamoDB automatically
scales based on traffic.
 .attributeDefinitions(attributeDefinitions)
 .tableName(tableName)
 .build();

try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created.
 WaiterResponse<DescribeTableResponse> waiterResponse = dbWaiter
 .waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 String newTable = response.tableDescription().tableName();
 System.out.println("The " + newTable + " was successfully created.");

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

}

public static void putRecordBatch(DynamoDbClient ddb) {
 String sqlStatement = "INSERT INTO MoviesPartiQBatch VALUE {'year':?,
'title' : ?, 'info' : ?}";
 try {
 // Create three movies to add to the Amazon DynamoDB table.
 // Set data for Movie 1.
 List<AttributeValue> parameters = new ArrayList<>();

 AttributeValue att1 = AttributeValue.builder()
 .n("1977")
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s("Star Wars")
 .build();

 AttributeValue att3 = AttributeValue.builder()
 .s("No Information")
 .build();
 }
}

```

```
parameters.add(att1);
parameters.add(att2);
parameters.add(att3);

BatchStatementRequest statementRequestMovie1 =
BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parameters)
 .build();

// Set data for Movie 2.
List<AttributeValue> parametersMovie2 = new ArrayList<>();
AttributeValue attMovie2 = AttributeValue.builder()
 .n("1939")
 .build();

AttributeValue attMovie2A = AttributeValue.builder()
 .s("Wizard of Oz")
 .build();

AttributeValue attMovie2B = AttributeValue.builder()
 .s("No Information")
 .build();

parametersMovie2.add(attMovie2);
parametersMovie2.add(attMovie2A);
parametersMovie2.add(attMovie2B);

BatchStatementRequest statementRequestMovie2 =
BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersMovie2)
 .build();

// Set data for Movie 3.
List<AttributeValue> parametersMovie3 = new ArrayList<>();
AttributeValue attMovie3 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue attMovie3A = AttributeValue.builder()
 .s("My Movie 3")
 .build();
```

```

 AttributeValue attMovie3B = AttributeValue.builder()
 .s("No Information")
 .build();

 parametersMovie3.add(attMovie3);
 parametersMovie3.add(attMovie3A);
 parametersMovie3.add(attMovie3B);

 BatchStatementRequest statementRequestMovie3 =
BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersMovie3)
 .build();

 // Add all three movies to the list.
 List<BatchStatementRequest> myBatchStatementList = new ArrayList<>();
 myBatchStatementList.add(statementRequestMovie1);
 myBatchStatementList.add(statementRequestMovie2);
 myBatchStatementList.add(statementRequestMovie3);

 BatchExecuteStatementRequest batchRequest =
BatchExecuteStatementRequest.builder()
 .statements(myBatchStatementList)
 .build();

 BatchExecuteStatementResponse response =
ddb.batchExecuteStatement(batchRequest);
 System.out.println("ExecuteStatement successful: " +
response.toString());
 System.out.println("Added new movies using a batch command.");

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void updateTableItemBatch(DynamoDbClient ddb) {
 String sqlStatement = "UPDATE MoviesPartiQBatch SET info = 'directors\\":
[\\\"Merian C. Cooper\\\",\\\"Ernest B. Schoedsack' where year=? and title=?";
 List<AttributeValue> parametersRec1 = new ArrayList<>();

 // Update three records.

```



```
AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue att2 = AttributeValue.builder()
 .s("My Movie 1")
 .build();

parametersRec1.add(att1);
parametersRec1.add(att2);

BatchStatementRequest statementRequestRec1 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec1)
 .build();

// Update record 2.
List<AttributeValue> parametersRec2 = new ArrayList<>();
AttributeValue attRec2 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue attRec2a = AttributeValue.builder()
 .s("My Movie 2")
 .build();

parametersRec2.add(attRec2);
parametersRec2.add(attRec2a);
BatchStatementRequest statementRequestRec2 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec2)
 .build();

// Update record 3.
List<AttributeValue> parametersRec3 = new ArrayList<>();
AttributeValue attRec3 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue attRec3a = AttributeValue.builder()
 .s("My Movie 3")
 .build();

parametersRec3.add(attRec3);
```

```

 parametersRec3.add(attRec3a);
 BatchStatementRequest statementRequestRec3 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec3)
 .build();

 // Add all three movies to the list.
 List<BatchStatementRequest> myBatchStatementList = new ArrayList<>();
 myBatchStatementList.add(statementRequestRec1);
 myBatchStatementList.add(statementRequestRec2);
 myBatchStatementList.add(statementRequestRec3);

 BatchExecuteStatementRequest batchRequest =
 BatchExecuteStatementRequest.builder()
 .statements(myBatchStatementList)
 .build();

 try {
 BatchExecuteStatementResponse response =
 ddb.batchExecuteStatement(batchRequest);
 System.out.println("ExecuteStatement successful: " +
 response.toString());
 System.out.println("Updated three movies using a batch command.");

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("Item was updated!");
 }

 public static void deleteItemBatch(DynamoDbClient ddb) {
 String sqlStatement = "DELETE FROM MoviesPartiQBatch WHERE year = ? and
 title=?";
 List<AttributeValue> parametersRec1 = new ArrayList<>();

 // Specify three records to delete.
 AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s("My Movie 1")
 .build();
 }

```

```
parametersRec1.add(att1);
parametersRec1.add(att2);

BatchStatementRequest statementRequestRec1 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec1)
 .build();

// Specify record 2.
List<AttributeValue> parametersRec2 = new ArrayList<>();
AttributeValue attRec2 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue attRec2a = AttributeValue.builder()
 .s("My Movie 2")
 .build();

parametersRec2.add(attRec2);
parametersRec2.add(attRec2a);
BatchStatementRequest statementRequestRec2 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec2)
 .build();

// Specify record 3.
List<AttributeValue> parametersRec3 = new ArrayList<>();
AttributeValue attRec3 = AttributeValue.builder()
 .n(String.valueOf("2022"))
 .build();

AttributeValue attRec3a = AttributeValue.builder()
 .s("My Movie 3")
 .build();

parametersRec3.add(attRec3);
parametersRec3.add(attRec3a);

BatchStatementRequest statementRequestRec3 = BatchStatementRequest.builder()
 .statement(sqlStatement)
 .parameters(parametersRec3)
 .build();
```

```

// Add all three movies to the list.
List<BatchStatementRequest> myBatchStatementList = new ArrayList<>();
myBatchStatementList.add(statementRequestRec1);
myBatchStatementList.add(statementRequestRec2);
myBatchStatementList.add(statementRequestRec3);

BatchExecuteStatementRequest batchRequest =
BatchExecuteStatementRequest.builder()
 .statements(myBatchStatementList)
 .build();

try {
 ddb.batchExecuteStatement(batchRequest);
 System.out.println("Deleted three movies using a batch command.");

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

}

public static void deleteDynamoDBTable(DynamoDbClient ddb, String tableName) {
 DeleteTableRequest request = DeleteTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 ddb.deleteTable(request);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }

 System.out.println(tableName + " was successfully deleted!");
}

private static ExecuteStatementResponse executeStatementRequest(DynamoDbClient
ddb, String statement,

List<AttributeValue> parameters) {
 ExecuteStatementRequest request = ExecuteStatementRequest.builder()
 .statement(statement)
 .parameters(parameters)
 .build();

```

```
 return ddb.executeStatement(request);
 }
}
```

- For API details, see [BatchExecuteStatement](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table using PartiQL

The following code example shows how to:

- Get an item by running a SELECT statement.
- Add an item by running an INSERT statement.
- Update an item by running an UPDATE statement.
- Delete an item by running a DELETE statement.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class ScenarioPartiQ {
 public static void main(String[] args) throws IOException {
 String fileName = "../../resources/sample_files/movies.json";
 String tableName = "MoviesPartiQ";
 Region region = Region.US_EAST_1;
 DynamoDbClient ddb = DynamoDbClient.builder()
 .region(region)
 .build();

 System.out.println(
 "***** Creating an Amazon DynamoDB table named MoviesPartiQ with a key"
 + "named year and a sort key named title.");
 createTable(ddb, tableName);

 System.out.println("Loading data into the MoviesPartiQ table.");
 }
}
```

```

 loadData(ddb, fileName);

 System.out.println("Getting data from the MoviesPartiQ table.");
 getItem(ddb);

 System.out.println("Putting a record into the MoviesPartiQ table.");
 putRecord(ddb);

 System.out.println("Updating a record.");
 updateTableItem(ddb);

 System.out.println("Querying the movies released in 2013.");
 queryTable(ddb);

 System.out.println("Deleting the Amazon DynamoDB table.");
 deleteDynamoDBTable(ddb, tableName);
 ddb.close();
 }

 public static void createTable(DynamoDbClient ddb, String tableName) {
 DynamoDbWaiter dbWaiter = ddb.waiter();
 ArrayList<AttributeDefinition> attributeDefinitions = new ArrayList<>();

 // Define attributes.
 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("year")
 .attributeType("N")
 .build());

 attributeDefinitions.add(AttributeDefinition.builder()
 .attributeName("title")
 .attributeType("S")
 .build());

 ArrayList<KeySchemaElement> tableKey = new ArrayList<>();
 KeySchemaElement key = KeySchemaElement.builder()
 .attributeName("year")
 .keyType(KeyType.HASH)
 .build();

 KeySchemaElement key2 = KeySchemaElement.builder()
 .attributeName("title")
 .keyType(KeyType.RANGE) // Sort
 .build();
 }

```

```

// Add KeySchemaElement objects to the list.
tableKey.add(key);
tableKey.add(key2);

CreateTableRequest request = CreateTableRequest.builder()
 .keySchema(tableKey)
 .billingMode(BillingMode.PAY_PER_REQUEST) //Scales based on traffic.
 .attributeDefinitions(attributeDefinitions)
 .tableName(tableName)
 .build();

try {
 CreateTableResponse response = ddb.createTable(request);
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 // Wait until the Amazon DynamoDB table is created.
 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 String newTable = response.tableDescription().tableName();
 System.out.println("The " + newTable + " was successfully created.");

} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

}

// Load data into the table.
public static void loadData(DynamoDbClient ddb, String fileName) throws
IOException {

 String sqlStatement = "INSERT INTO MoviesPartiQ VALUE {'year':?,
'title' : ?, 'info' : ?}";
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 ObjectNode currentNode;
 int t = 0;
 List<AttributeValue> parameters = new ArrayList<>();

```

```
while (iter.hasNext()) {

 // Add 200 movies to the table.
 if (t == 200)
 break;
 currentNode = (ObjectNode) iter.next();

 int year = currentNode.path("year").asInt();
 String title = currentNode.path("title").asText();
 String info = currentNode.path("info").toString();

 AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf(year))
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s(title)
 .build();

 AttributeValue att3 = AttributeValue.builder()
 .s(info)
 .build();

 parameters.add(att1);
 parameters.add(att2);
 parameters.add(att3);

 // Insert the movie into the Amazon DynamoDB table.
 executeStatementRequest(ddb, sqlStatement, parameters);
 System.out.println("Added Movie " + title);

 parameters.remove(att1);
 parameters.remove(att2);
 parameters.remove(att3);
 t++;
}

}

public static void getItem(DynamoDbClient ddb) {

 String sqlStatement = "SELECT * FROM MoviesPartiQ where year=? and title=?";
 List<AttributeValue> parameters = new ArrayList<>();
 AttributeValue att1 = AttributeValue.builder()
 .n("2012")
```



```
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s("The Perks of Being a Wallflower")
 .build();

 parameters.add(att1);
 parameters.add(att2);

 try {
 ExecuteStatementResponse response = executeStatementRequest(ddb,
 sqlStatement, parameters);
 System.out.println("ExecuteStatement successful: " +
 response.toString());

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 public static void putRecord(DynamoDbClient ddb) {

 String sqlStatement = "INSERT INTO MoviesPartiQ VALUE {'year':?,
 'title' : ?, 'info' : ?}";
 try {
 List<AttributeValue> parameters = new ArrayList<>();

 AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf("2020"))
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s("My Movie")
 .build();

 AttributeValue att3 = AttributeValue.builder()
 .s("No Information")
 .build();

 parameters.add(att1);
 parameters.add(att2);
 parameters.add(att3);
```

```

 executeStatementRequest(ddb, sqlStatement, parameters);
 System.out.println("Added new movie.");

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void updateTableItem(DynamoDbClient ddb) {

 String sqlStatement = "UPDATE MoviesPartiQ SET info = 'directors\":[\"Merian
C. Cooper\", \"Ernest B. Schoedsack' where year=? and title=?";
 List<AttributeValue> parameters = new ArrayList<>();
 AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf("2013"))
 .build();

 AttributeValue att2 = AttributeValue.builder()
 .s("The East")
 .build();

 parameters.add(att1);
 parameters.add(att2);

 try {
 executeStatementRequest(ddb, sqlStatement, parameters);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("Item was updated!");
}

// Query the table where the year is 2013.
public static void queryTable(DynamoDbClient ddb) {
 String sqlStatement = "SELECT * FROM MoviesPartiQ where year = ? ORDER BY
year";
 try {

 List<AttributeValue> parameters = new ArrayList<>();
 AttributeValue att1 = AttributeValue.builder()
 .n(String.valueOf("2013"))

```

```

 .build();
 parameters.add(att1);

 // Get items in the table and write out the ID value.
 ExecuteStatementResponse response = executeStatementRequest(ddb,
sqlStatement, parameters);
 System.out.println("ExecuteStatement successful: " +
response.toString());

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void deleteDynamoDBTable(DynamoDbClient ddb, String tableName) {

 DeleteTableRequest request = DeleteTableRequest.builder()
 .tableName(tableName)
 .build();

 try {
 ddb.deleteTable(request);

 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }

 System.out.println(tableName + " was successfully deleted!");
}

private static ExecuteStatementResponse executeStatementRequest(DynamoDbClient
ddb, String statement,
List<AttributeValue> parameters) {
 ExecuteStatementRequest request = ExecuteStatementRequest.builder()
 .statement(statement)
 .parameters(parameters)
 .build();

 return ddb.executeStatement(request);
}

```

```
private static void processResults(ExecuteStatementResponse
executeStatementResult) {
 System.out.println("ExecuteStatement successful: " +
executeStatementResult.toString());
}
}
```

- For API details, see [ExecuteStatement](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table using a Global Secondary Index

The following code example shows how to query a table using a Global Secondary Index.

- Query a DynamoDB table using its primary key.
- Query a Global Secondary Index (GSI) for alternate access patterns.
- Compare table queries and GSI queries.

## SDK for Java 2.x

Query a DynamoDB table using its primary key and a Global Secondary Index (GSI) with AWS SDK for Java 2.x.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;

public QueryResponse queryTable(
 final String tableName, final String partitionKeyName, final String
partitionKeyValue) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
```

```

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query on base table successful. Found " +
response.count() + " items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw new DynamoDbQueryException("Table not found: " + tableName, e);
 } catch (DynamoDbException e) {
 System.err.println("Error querying base table: " + e.getMessage());
 throw new DynamoDbQueryException("Failed to execute query on base
table", e);
 }
 }

 /**
 * Queries a DynamoDB Global Secondary Index (GSI) by partition key.
 *
 * @param tableName The name of the DynamoDB table
 * @param indexName The name of the GSI
 * @param partitionKeyName The name of the GSI partition key attribute
 * @param partitionKeyValue The value of the GSI partition key to query
 * @return The query response from DynamoDB

```

```

 * @throws ResourceNotFoundException if the table or index doesn't exist
 * @throws DynamoDbException if the query fails
 */
 public QueryResponse queryGlobalSecondaryIndex(
 final String tableName, final String indexName, final String
partitionKeyName, final String partitionKeyValue) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Index name", indexName);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_IK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_IK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .indexName(indexName)
 .keyConditionExpression(GSI_KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query on GSI successful. Found " + response.count()
+ " items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format(
 "Error: The Amazon DynamoDB table \"%s\" or index \"%s\" can't be
found.\n", tableName, indexName);
 throw new DynamoDbQueryException("Table or index not found: " +
tableName + "/" + indexName, e);
 } catch (DynamoDbException e) {

```

```

 System.err.println("Error querying GSI: " + e.getMessage());
 throw new DynamoDbQueryException("Failed to execute query on GSI", e);
 }
}

```

## Compare querying a table directly versus querying a GSI with AWS SDK for Java 2.x.

```

public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <tableName> <basePartitionKeyName> <basePartitionKeyValue>
<gsiName> <gsiPartitionKeyName> <gsiPartitionKeyValue> [region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 basePartitionKeyName - The name of the base table partition key
attribute.
 basePartitionKeyValue - The value of the base table partition
key to query.
 gsiName - The name of the Global Secondary Index.
 gsiPartitionKeyName - The name of the GSI partition key
attribute.
 gsiPartitionKeyValue - The value of the GSI partition key to
query.
 region (optional) - The AWS region where the table exists.
(Default: us-east-1)
 """;

 if (args.length < 6) {
 System.out.println(usage);
 System.exit(1);
 }

 final String tableName = args[0];
 final String basePartitionKeyName = args[1];
 final String basePartitionKeyValue = args[2];
 final String gsiName = args[3];
 final String gsiPartitionKeyName = args[4];
 final String gsiPartitionKeyValue = args[5];
 final Region region = args.length > 6 ? Region.of(args[6]) :
Region.US_EAST_1;

```

```

try (DynamoDbClient ddb = DynamoDbClient.builder().region(region).build()) {
 final QueryTableAndGSI queryHelper = new QueryTableAndGSI(ddb);

 // Query the base table
 System.out.println("Querying base table where " + basePartitionKeyName +
" = " + basePartitionKeyValue);
 final QueryResponse tableResponse =
 queryHelper.queryTable(tableName, basePartitionKeyName,
basePartitionKeyValue);

 System.out.println("Found " + tableResponse.count() + " items in base
table:");
 tableResponse.items().forEach(item -> System.out.println(item));

 // Query the GSI
 System.out.println(
 "\nQuerying GSI '" + gsiName + "' where " + gsiPartitionKeyName + "
= " + gsiPartitionKeyValue);
 final QueryResponse gsiResponse =
 queryHelper.queryGlobalSecondaryIndex(tableName, gsiName,
gsiPartitionKeyName, gsiPartitionKeyValue);

 System.out.println("Found " + gsiResponse.count() + " items in GSI:");
 gsiResponse.items().forEach(item -> System.out.println(item));

 // Explain the differences between querying a table and a GSI
 System.out.println("\nKey differences between querying a table and a
GSI:");
 System.out.println("1. When querying a GSI, you must specify the
indexName parameter");
 System.out.println("2. GSIs may not contain all attributes from the base
table (projection)");
 System.out.println("3. GSIs consume read capacity units from the GSI's
capacity, not the base table's");
 System.out.println("4. GSIs may have eventually consistent data (cannot
use ConsistentRead=true)");

 } catch (IllegalArgumentException e) {
 System.err.println("Invalid input: " + e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table or index not found: " + e.getMessage());
 System.exit(1);
 } catch (DynamoDbException e) {

```



```
 System.err.println("DynamoDB error: " + e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table using a begins\_with condition

The following code example shows how to query a table using a begins\_with condition.

- Use the begins\_with function in a key condition expression.
- Filter items based on a prefix pattern in the sort key.

## SDK for Java 2.x

Query a DynamoDB table using a begins\_with condition on the sort key with AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithBeginsWithCondition(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String sortKeyName,
 final String sortKeyPrefix) {
```

```

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Sort key name", sortKeyName);
 CodeSampleUtils.validateStringParameter("Sort key prefix", sortKeyPrefix);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_SK, sortKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_SK_PREFIX,
 AttributeValue.builder().s(sortKeyPrefix).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query with begins_with condition successful.
Found {0} items", response.count());
 return response;
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying with begins_with condition",
e);
 throw e;
 }
 }
}

```

Demonstrate using `begins_with` with different prefix lengths with AWS SDK for Java 2.x.

```
public static void main(String[] args) {
 try {
 CodeSampleUtils.BeginsWithQueryConfig config =
CodeSampleUtils.BeginsWithQueryConfig.fromArgs(args);
 LOGGER.log(Level.INFO, "Querying items where {0} = {1} and {2} begins
with '{3}'", new Object[] {
 config.getPartitionKeyName(),
 config.getPartitionKeyValue(),
 config.getSortKeyName(),
 config.getSortKeyPrefix()
 });

 // Using the builder pattern to create and execute the query
 final QueryResponse response = new BeginsWithQueryBuilder()
 .withTableName(config.getTableName())
 .withPartitionKeyName(config.getPartitionKeyName())
 .withPartitionKeyValue(config.getPartitionKeyValue())
 .withSortKeyName(config.getSortKeyName())
 .withSortKeyPrefix(config.getSortKeyPrefix())
 .withRegion(config.getRegion())
 .execute();

 // Process the results
 LOGGER.log(Level.INFO, "Found {0} items:", response.count());
 response.items().forEach(item -> LOGGER.info(item.toString()));

 // Demonstrate with a different prefix
 if (!config.getSortKeyPrefix().isEmpty()) {
 String shorterPrefix = config.getSortKeyPrefix()
 .substring(0, Math.max(1, config.getSortKeyPrefix().length() /
2));

 LOGGER.log(Level.INFO, "\nNow querying with a shorter prefix:
'{0}'", shorterPrefix);

 final QueryResponse response2 = new BeginsWithQueryBuilder()
 .withTableName(config.getTableName())
 .withPartitionKeyName(config.getPartitionKeyName())
 .withPartitionKeyValue(config.getPartitionKeyValue())
 .withSortKeyName(config.getSortKeyName())
```

```

 .withSortKeyPrefix(shorterPrefix)
 .withRegion(config.getRegion())
 .execute();

 LOGGER.log(Level.INFO, "Found {0} items with shorter prefix:",
response2.count());
 response2.items().forEach(item -> LOGGER.info(item.toString()));
 }
} catch (IllegalArgumentException e) {
 LOGGER.log(Level.SEVERE, "Invalid input: {0}", e.getMessage());
 printUsage();
} catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found", e);
} catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "DynamoDB error", e);
} catch (Exception e) {
 LOGGER.log(Level.SEVERE, "Unexpected error", e);
}
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table using a date range

The following code example shows how to query a table using a date range in the sort key.

- Query items within a specific date range.
- Use comparison operators on date-formatted sort keys.

## SDK for Java 2.x

Query a DynamoDB table for items within a date range with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

```

```

import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

 public QueryResponse queryWithDateRange(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String dateKeyName,
 final LocalDate startDate,
 final LocalDate endDate) {

 // Focus on query logic, assuming parameters are valid
 if (startDate == null || endDate == null) {
 throw new IllegalArgumentException("Start date and end date cannot be
null");
 }

 if (endDate.isBefore(startDate)) {
 throw new IllegalArgumentException("End date must be after start date");
 }

 // Format dates as ISO strings for DynamoDB (using just the date part)
 final String formattedStartDate = startDate.toString();
 final String formattedEndDate = endDate.toString();

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_SK, dateKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_START_DATE,
 AttributeValue.builder().s(formattedStartDate).build());
 expressionAttributeValues.put(

```

```

 EXPRESSION_ATTRIBUTE_VALUE_END_DATE,
 AttributeValue.builder().s(formattedEndDate).build());

// Create the query request
final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query by date range successful. Found {0}
items", response.count());
 return response;
} catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
} catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying by date range: {0}",
e.getMessage());
 throw e;
}
}

```

Demonstrates how to query a DynamoDB table with date range filtering.

```

public static void main(String[] args) {
 final String usage =
 ""
 Usage:
 <tableName> <partitionKeyName> <partitionKeyValue> <dateKeyName>
<startDate> <endDate> [region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 partitionKeyName - The name of the partition key attribute.
 partitionKeyValue - The value of the partition key to query.
 dateKeyName - The name of the date attribute to filter on.
 startDate - The start date for the range query (YYYY-MM-DD).
 endDate - The end date for the range query (YYYY-MM-DD).

```

```

 region (optional) - The AWS region where the table exists.
(Default: us-east-1)
 """;

 if (args.length < 6) {
 System.out.println(usage);
 System.exit(1);
 }

 try {
 // Parse command line arguments into a config object
 CodeSampleUtils.DateRangeQueryConfig config =
CodeSampleUtils.DateRangeQueryConfig.fromArgs(args);

 LOGGER.log(
 Level.INFO, "Querying items from {0} to {1}", new Object[]
{config.getStartDate(), config.getEndDate()
 });

 // Using the builder pattern to create and execute the query
 final QueryResponse response = new DateRangeQueryBuilder()
 .withTableName(config.getTableName())
 .withPartitionKeyName(config.getPartitionKeyName())
 .withPartitionKeyValue(config.getPartitionKeyValue())
 .withDateKeyName(config.getDateKeyName())
 .withStartDate(config.getStartDate())
 .withEndDate(config.getEndDate())
 .withRegion(config.getRegion())
 .execute();

 // Process the results
 LOGGER.log(Level.INFO, "Found {0} items:", response.count());
 response.items().forEach(item -> {
 LOGGER.info(item.toString());

 // Extract and display the date attribute for clarity
 if (item.containsKey(config.getDateKeyName())) {
 LOGGER.log(
 Level.INFO,
 " Date attribute: {0}",
 item.get(config.getDateKeyName()).s());
 }
 });
 }
}

```

```

 // Demonstrate with a different date range
 LocalDate narrowerStartDate = config.getStartDate().plusDays(1);
 LocalDate narrowerEndDate = config.getEndDate().minusDays(1);

 if (!narrowerStartDate.isAfter(narrowerEndDate)) {
 LOGGER.log(Level.INFO, "\nNow querying with a narrower date range:
{0} to {1}", new Object[] {
 narrowerStartDate, narrowerEndDate
 });

 final QueryResponse response2 = new DateRangeQueryBuilder()
 .withTableName(config.getTableName())
 .withPartitionKeyName(config.getPartitionKeyName())
 .withPartitionKeyValue(config.getPartitionKeyValue())
 .withDateKeyName(config.getDateKeyName())
 .withStartDate(narrowerStartDate)
 .withEndDate(narrowerEndDate)
 .withRegion(config.getRegion())
 .execute();

 LOGGER.log(Level.INFO, "Found {0} items with narrower date range:",
response2.count());
 response2.items().forEach(item -> LOGGER.info(item.toString()));
 }

 LOGGER.info("\nNote: When storing dates in DynamoDB:");
 LOGGER.info("1. Use ISO format (YYYY-MM-DD) for lexicographical
ordering");
 LOGGER.info("2. Use the BETWEEN operator for inclusive date range
queries");
 LOGGER.info("3. Consider using ISO-8601 format for timestamps with time
components");

 } catch (IllegalArgumentException e) {
 LOGGER.log(Level.SEVERE, "Invalid input: {0}", e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", e.getMessage());
 System.exit(1);
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "DynamoDB error: {0}", e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 LOGGER.log(Level.SEVERE, "Unexpected error: {0}", e.getMessage());

```



```
 System.exit(1);
 }
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with a complex filter expression

The following code example shows how to query a table with a complex filter expression.

- Apply complex filter expressions to query results.
- Combine multiple conditions using logical operators.
- Filter items based on non-key attributes.

## SDK for Java 2.x

Query a DynamoDB table with a complex filter expression using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithComplexFilter(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String statusAttrName,
 final String activeStatus,
 final String pendingStatus,
 final String priceAttrName,
 final double minPrice,
```

```

 final double maxPrice,
 final String categoryAttrName) {

 // Validate parameters
 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyKeyValue);
 CodeSampleUtils.validateStringParameter("Status attribute name",
statusAttrName);
 CodeSampleUtils.validateStringParameter("Active status", activeStatus);
 CodeSampleUtils.validateStringParameter("Pending status", pendingStatus);
 CodeSampleUtils.validateStringParameter("Price attribute name",
priceAttrName);
 CodeSampleUtils.validateStringParameter("Category attribute name",
categoryAttrName);
 CodeSampleUtils.validateNumericRange("Minimum price", minPrice, 0.0,
Double.MAX_VALUE);
 CodeSampleUtils.validateNumericRange("Maximum price", maxPrice, minPrice,
Double.MAX_VALUE);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#pk", partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_STATUS,
statusAttrName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PRICE,
priceAttrName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_CATEGORY,
categoryAttrName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 ":pkValue", AttributeValue.builder().s(partitionKeyKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_ACTIVE,
 AttributeValue.builder().s(activeStatus).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PENDING,
 AttributeValue.builder().s(pendingStatus).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_MIN_PRICE,
 AttributeValue.builder().n(String.valueOf(minPrice)).build());
 expressionAttributeValues.put(

```

```
 EXPRESSION_ATTRIBUTE_VALUE_MAX_PRICE,
 AttributeValue.builder().n(String.valueOf(maxPrice)).build());

// Create the query request
final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(FILTER_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

return dynamoDbClient.query(queryRequest);
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with a dynamic filter expression

The following code example shows how to query a table with a dynamic filter expression.

- Build filter expressions dynamically at runtime.
- Construct filter conditions based on user input or application state.
- Add or remove filter criteria conditionally.

## SDK for Java 2.x

Query a DynamoDB table with a dynamically constructed filter expression using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
```

```
public static QueryResponse queryWithDynamicFilter(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final Map<String, Object> filterCriteria,
 final Region region,
 final DynamoDbClient dynamoDbClient) {

 validateParameters(tableName, partitionKeyName, partitionKeyValue,
filterCriteria);

 DynamoDbClient ddbClient = dynamoDbClient;
 boolean shouldClose = false;

 try {
 if (ddbClient == null) {
 ddbClient = createClient(region);
 shouldClose = true;
 }

 final QueryWithDynamicFilter queryHelper = new
QueryWithDynamicFilter(ddbClient);
 return queryHelper.queryWithDynamicFilter(tableName, partitionKeyName,
partitionKeyValue, filterCriteria);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println("Failed to execute dynamic filter query: " +
e.getMessage());
 throw e;
 } catch (Exception e) {
 System.err.println("Unexpected error during query: " + e.getMessage());
 throw e;
 } finally {
 if (shouldClose && ddbClient != null) {
 ddbClient.close();
 }
 }
}
```

## Demonstrates how to use dynamic filter expressions with AWS SDK for Java 2.x.

```

public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <tableName> <partitionKeyName> <partitionKeyValue>
<filterAttrName> <filterAttrValue> [region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 partitionKeyName - The name of the partition key attribute.
 partitionKeyValue - The value of the partition key to query.
 filterAttrName - The name of the attribute to filter on.
 filterAttrValue - The value to filter by.
 region (optional) - The AWS region where the table exists.
(Default: us-east-1)
 """;

 if (args.length < 5) {
 System.out.println(usage);
 System.exit(1);
 }

 final String tableName = args[0];
 final String partitionKeyName = args[1];
 final String partitionKeyValue = args[2];
 final String filterAttrName = args[3];
 final String filterAttrValue = args[4];
 final Region region = args.length > 5 ? Region.of(args[5]) :
Region.US_EAST_1;

 System.out.println("Querying items with dynamic filter: " + filterAttrName +
" = " + filterAttrValue);

 try {
 // Using the builder pattern to create and execute the query
 final QueryResponse response = new DynamicFilterQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withFilterCriterion(filterAttrName, filterAttrValue)
 .withRegion(region)
 .execute();
 }
}

```

```

 // Process the results
 System.out.println("Found " + response.count() + " items:");
 response.items().forEach(item -> System.out.println(item));

 // Demonstrate multiple filter criteria
 System.out.println("\nNow querying with multiple filter criteria:");

 Map<String, Object> multipleFilters = new HashMap<>();
 multipleFilters.put(filterAttrName, filterAttrValue);
 multipleFilters.put("status", "active");

 final QueryResponse multiFilterResponse = new
DynamicFilterQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withFilterCriteria(multipleFilters)
 .withRegion(region)
 .execute();

 System.out.println("Found " + multiFilterResponse.count() + " items with
multiple filters:");
 multiFilterResponse.items().forEach(item -> System.out.println(item));

 } catch (IllegalArgumentException e) {
 System.err.println("Invalid input: " + e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println("DynamoDB error: " + e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with a filter expression and limit

The following code example shows how to query a table with a filter expression and limit.

- Apply filter expressions to query results with a limit on items evaluated.
- Understand how limit affects filtered query results.
- Control the maximum number of items processed in a query.

### SDK for Java 2.x

Query a DynamoDB table with a filter expression and limit using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithFilterAndLimit(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String filterAttrName,
 final String filterAttrValue,
 final int limit) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Filter attribute name",
filterAttrName);
 CodeSampleUtils.validateStringParameter("Filter attribute value",
filterAttrValue);
 CodeSampleUtils.validatePositiveInteger("Limit", limit);

 // Create expression attribute names for the column names
```

```

 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_FILTER,
filterAttrName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_FILTER,
 AttributeValue.builder().s(filterAttrValue).build());

 // Create the filter expression
 final String filterExpression = "#filterAttr = :filterValue";

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(filterExpression)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .limit(limit)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query with filter and limit successful. Found
{0} items", response.count());
 LOGGER.log(
 Level.INFO, "ScannedCount: {0} (total items evaluated before
filtering)", response.scannedCount());
 return response;
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying with filter and limit: {0}",
e.getMessage());
 throw e;
 }
 }

```



```
}
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with nested attributes

The following code example shows how to query a table with nested attributes.

- Access and filter by nested attributes in DynamoDB items.
- Use document path expressions to reference nested elements.

## SDK for Java 2.x

Query a DynamoDB table with nested attributes using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;

public QueryResponse queryWithNestedAttributes(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String nestedPath,
 final String nestedAttr,
 final String nestedValue) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Nested path", nestedPath);
 CodeSampleUtils.validateStringParameter("Nested attribute", nestedAttr);
 CodeSampleUtils.validateStringParameter("Nested value", nestedValue);

```

```

// Split the nested path into components
final String[] pathComponents = nestedPath.split("\\.");

// Create expression attribute names for the column names
final Map<String, String> expressionAttributeNames = new HashMap<>();
expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

// Build the nested attribute reference using document path notation
final StringBuilder nestedAttributeRef = new StringBuilder();
for (int i = 0; i < pathComponents.length; i++) {
 final String aliasName = "#n" + i;
 expressionAttributeNames.put(aliasName, pathComponents[i]);

 if (i > 0) {
 nestedAttributeRef.append(".");
 }
 nestedAttributeRef.append(aliasName);
}

// Create expression attribute values for the column values
final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_NESTED,
 AttributeValue.builder().s(nestedValue).build());

// Create the filter expression using the nested attribute reference
final String filterExpression = nestedAttributeRef + " = :nestedValue";

// Create the query request
final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(filterExpression)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);

```

```

 System.out.println("Query with nested attribute filter successful. Found
" + response.count() + " items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println("Error querying with nested attribute filter: " +
e.getMessage());
 throw e;
 }
}
}

```

Demonstrates how to query a DynamoDB table with nested attributes.

```

public static void main(String[] args) {
 final String usage =
 ""
 Usage:
 <tableName> <partitionKeyName> <partitionKeyValue> <nestedPath>
<nestedAttr> <nestedValue> [region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 partitionKeyName - The name of the partition key attribute.
 partitionKeyValue - The value of the partition key to query.
 nestedPath - The path to the nested map attribute (e.g.,
"address").
 nestedAttr - The name of the nested attribute (e.g., "city").
 nestedValue - The value to filter by (e.g., "Seattle").
 region (optional) - The AWS region where the table exists.
(Default: us-east-1)
 "";

 if (args.length < 6) {
 System.out.println(usage);
 System.exit(1);
 }

 final String tableName = args[0];
 final String partitionKeyName = args[1];
 final String partitionKeyValue = args[2];

```

```

 final String nestedPath = args[3];
 final String nestedAttr = args[4];
 final String nestedValue = args[5];
 final Region region = args.length > 6 ? Region.of(args[6]) :
Region.US_EAST_1;

 System.out.println("Querying items where " + partitionKeyName + " = " +
partitionKeyValue + " and " + nestedPath
 + "." + nestedAttr + " = " + nestedValue);

 try {
 // Using the builder pattern to create and execute the query
 final QueryResponse response = new NestedAttributeQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withNestedPath(nestedPath)
 .withNestedAttribute(nestedAttr)
 .withNestedValue(nestedValue)
 .withRegion(region)
 .execute();

 // Process the results
 System.out.println("Found " + response.count() + " items:");
 response.items().forEach(item -> {
 System.out.println(item);

 // Extract and display the nested attribute for clarity
 if (item.containsKey(nestedPath) && item.get(nestedPath).hasM()) {
 Map<String, AttributeValue> nestedMap =
item.get(nestedPath).m();
 if (nestedMap.containsKey(nestedAttr)) {
 System.out.println(" Nested attribute " + nestedPath + "."
+ nestedAttr + ": "
 + formatAttributeValue(nestedMap.get(nestedAttr)));
 }
 }
 });

 System.out.println("\nNote: When working with nested attributes in
DynamoDB:");
 System.out.println("1. Use dot notation in filter expressions to access
nested attributes");

```

```

 System.out.println("2. Use expression attribute names for each component
of the path");
 System.out.println("3. Check if the nested attribute exists before
accessing it");

 } catch (IllegalArgumentException e) {
 System.err.println("Invalid input: " + e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println("DynamoDB error: " + e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with pagination

The following code example shows how to query a table with pagination.

- Implement pagination for DynamoDB query results.
- Use the `LastEvaluatedKey` to retrieve subsequent pages.
- Control the number of items per page with the `Limit` parameter.

## SDK for Java 2.x

Query a DynamoDB table with pagination using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;

```

```

import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

 public List<Map<String, AttributeValue>> queryWithPagination(
 final String tableName, final String partitionKeyName, final String
partitionKeyValue, final int pageSize) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validatePositiveInteger("Page size", pageSize);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 QueryRequest.Builder queryRequestBuilder = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .limit(pageSize);

 // List to store all items from all pages
 final List<Map<String, AttributeValue>> allItems = new ArrayList<>();

 // Map to store the last evaluated key for pagination
 Map<String, AttributeValue> lastEvaluatedKey = null;
 int pageNumber = 1;

 try {
 do {

```

```

 // If we have a last evaluated key, use it for the next page
 if (lastEvaluatedKey != null) {
 queryRequestBuilder.exclusiveStartKey(lastEvaluatedKey);
 }

 // Execute the query
 final QueryResponse response =
dynamoDbClient.query(queryRequestBuilder.build());

 // Process the current page of results
 final List<Map<String, AttributeValue>> pageItems =
response.items();
 allItems.addAll(pageItems);

 // Get the last evaluated key for the next page
 lastEvaluatedKey = response.lastEvaluatedKey();
 if (lastEvaluatedKey != null && lastEvaluatedKey.isEmpty()) {
 lastEvaluatedKey = null;
 }

 System.out.println("Page " + pageNumber + ": Retrieved " +
pageItems.size() + " items (Running total: "
 + allItems.size() + ")");

 pageNumber++;

 } while (lastEvaluatedKey != null);

 System.out.println("Query with pagination complete. Retrieved a total of
" + allItems.size()
 + " items across " + (pageNumber - 1) + " pages");

 return allItems;
} catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw e;
} catch (DynamoDbException e) {
 System.err.println("Error querying with pagination: " + e.getMessage());
 throw e;
}
}

```

## Demonstrates how to query a DynamoDB table with pagination.

```

public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <tableName> <partitionKeyName> <partitionKeyValue> [pageSize]
[region]
 Where:
 tableName - The Amazon DynamoDB table to query.
 partitionKeyName - The name of the partition key attribute.
 partitionKeyValue - The value of the partition key to query.
 pageSize (optional) - The maximum number of items to return per
page. (Default: 10)
 region (optional) - The AWS region where the table exists.
(Default: us-east-1)
 """;

 if (args.length < 3) {
 System.out.println(usage);
 System.exit(1);
 }

 final String tableName = args[0];
 final String partitionKeyName = args[1];
 final String partitionKeyValue = args[2];
 final int pageSize = args.length > 3 ? Integer.parseInt(args[3]) : 10;
 final Region region = args.length > 4 ? Region.of(args[4]) :
Region.US_EAST_1;

 System.out.println("Querying items with pagination (page size: " + pageSize
+ ")");

 try {
 // Using the builder pattern to create and execute the query
 final List<Map<String, AttributeValue>> allItems = new
PaginationQueryBuilder()
 .withTableName(tableName)
 .withPartitionKeyName(partitionKeyName)
 .withPartitionKeyValue(partitionKeyValue)
 .withPageSize(pageSize)
 .withRegion(region)
 .executeWithPagination();
 }
}

```



```

 // Process the results
 System.out.println("\nSummary: Retrieved a total of " + allItems.size()
+ " items");

 // Display the first few items as a sample
 final int sampleSize = Math.min(5, allItems.size());
 if (sampleSize > 0) {
 System.out.println("\nSample of retrieved items (first " +
sampleSize + "):");
 for (int i = 0; i < sampleSize; i++) {
 System.out.println(allItems.get(i));
 }

 if (allItems.size() > sampleSize) {
 System.out.println("... and " + (allItems.size() - sampleSize) +
" more items");
 }
 }
 } catch (IllegalArgumentException e) {
 System.err.println("Invalid input: " + e.getMessage());
 System.exit(1);
 } catch (ResourceNotFoundException e) {
 System.err.println("Table not found: " + tableName);
 System.exit(1);
 } catch (DynamoDbException e) {
 System.err.println("DynamoDB error: " + e.getMessage());
 System.exit(1);
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query a table with strongly consistent reads

The following code example shows how to query a table with strongly consistent reads.

- Configure the consistency level for DynamoDB queries.
- Use strongly consistent reads to get the most up-to-date data.

- Understand the tradeoffs between eventual consistency and strong consistency.

## SDK for Java 2.x

Query a DynamoDB table with configurable read consistency using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

public QueryResponse queryWithConsistentReads(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final boolean useConsistentRead) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
```

```

 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .consistentRead(useConsistentRead)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query successful. Found {0} items",
response.count());
 return response;
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying with consistent reads", e);
 throw e;
 }
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query for TTL items

The following code example shows how to query for TTL items.

### SDK for Java 2.x

Query Filtered Expression to gather TTL items in a DynamoDB table using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.util.Map;
import java.util.Optional;

 final QueryRequest request = QueryRequest.builder()
 .tableName(tableName)

```

```

 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .filterExpression(FILTER_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

try (DynamoDbClient ddb = dynamoDbClient != null
 ? dynamoDbClient
 : DynamoDbClient.builder().region(region).build()) {
 final QueryResponse response = ddb.query(request);
 System.out.println("Query successful. Found " + response.count() + "
items that have not expired yet.");

 // Print each item
 response.items().forEach(item -> {
 System.out.println("Item: " + item);
 });

 return 0;
} catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
} catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
}

```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Query tables using date and time patterns

The following code example shows how to query tables using date and time patterns.

- Store and query date/time values in DynamoDB.
- Implement date range queries using sort keys.
- Format date strings for effective querying.

## SDK for Java 2.x

Query using date ranges in sort keys with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;
import java.util.logging.Level;
import java.util.logging.Logger;

 public QueryResponse queryWithDateRange(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String dateKeyName,
 final LocalDate startDate,
 final LocalDate endDate) {

 // Focus on query logic, assuming parameters are valid
 if (startDate == null || endDate == null) {
 throw new IllegalArgumentException("Start date and end date cannot be
null");
 }

 if (endDate.isBefore(startDate)) {
 throw new IllegalArgumentException("End date must be after start date");
 }

 // Format dates as ISO strings for DynamoDB (using just the date part)
 final String formattedStartDate = startDate.toString();
 final String formattedEndDate = endDate.toString();

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_SK, dateKeyName);

 // Create expression attribute values for the column values

```

```

 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_START_DATE,
 AttributeValue.builder().s(formattedStartDate).build());
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_END_DATE,
 AttributeValue.builder().s(formattedEndDate).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 LOGGER.log(Level.INFO, "Query by date range successful. Found {0}
items", response.count());
 return response;
 } catch (ResourceNotFoundException e) {
 LOGGER.log(Level.SEVERE, "Table not found: {0}", tableName);
 throw e;
 } catch (DynamoDbException e) {
 LOGGER.log(Level.SEVERE, "Error querying by date range: {0}",
e.getMessage());
 throw e;
 }
 }
}

```

## Query using date-time variables with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;

```

```

import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.util.HashMap;
import java.util.Map;

 public QueryResponse queryWithDateTime(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String dateKeyName,
 final String startDate,
 final String endDate) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateDateRangeParameters(dateKeyName, startDate,
endDate);
 CodeSampleUtils.validateDateFormat("Start date", startDate);
 CodeSampleUtils.validateDateFormat("End date", endDate);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put("#dateKey", dateKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 ":startDate", AttributeValue.builder().s(startDate).build());
 expressionAttributeValues.put(
 ":endDate", AttributeValue.builder().s(endDate).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)

```

```

 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query successful. Found " + response.count() + "
items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println("Error querying with date range: " + e.getMessage());
 throw e;
 }
}

```

## Query within date ranges in Unix epoch timestamps with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.util.HashMap;
import java.util.Map;

public QueryResponse queryWithDateTimeEpoch(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String dateKeyName,
 final long startEpoch,
 final long endEpoch) {

```



```

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Date key name", dateKeyName);
 CodeSampleUtils.validateEpochTimestamp("Start epoch", startEpoch);
 CodeSampleUtils.validateEpochTimestamp("End epoch", endEpoch);

 // Create expression attribute names for the column names
 final Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put(EXPRESSION_ATTRIBUTE_NAME_PK,
partitionKeyName);
 expressionAttributeNames.put("#dateKey", dateKeyName);

 // Create expression attribute values for the column values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 EXPRESSION_ATTRIBUTE_VALUE_PK,
 AttributeValue.builder().s(partitionKeyValue).build());
 expressionAttributeValues.put(
 ":startDate",
 AttributeValue.builder().n(String.valueOf(startEpoch)).build());
 expressionAttributeValues.put(
 ":endDate",
 AttributeValue.builder().n(String.valueOf(endEpoch)).build());

 // Create the query request
 final QueryRequest queryRequest = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression(KEY_CONDITION_EXPRESSION)
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final QueryResponse response = dynamoDbClient.query(queryRequest);
 System.out.println("Query successful. Found " + response.count() + "
items");
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be
found.\n", tableName);
 throw e;
 } catch (DynamoDbException e) {

```

```

 System.err.println("Error querying with epoch timestamps: " +
e.getMessage());
 throw e;
 }
}

```

## Query within date ranges using LocalDateTime objects with AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.util.HashMap;
import java.util.Map;

 public QueryResponse queryWithDateTimeLocalDateTime(
 final String tableName,
 final String partitionKeyName,
 final String partitionKeyValue,
 final String dateKeyName,
 final LocalDateTime startDateTime,
 final LocalDateTime endDateTime) {

 CodeSampleUtils.validateTableParameters(tableName, partitionKeyName,
partitionKeyValue);
 CodeSampleUtils.validateStringParameter("Date key name", dateKeyName);
 if (startDateTime == null || endDateTime == null) {
 throw new IllegalArgumentException("Start and end LocalDateTime must not
be null");
 }

 // Convert LocalDateTime to ISO-8601 strings in UTC with the correct format
 final String startDate =
startDateTime.atZone(ZoneOffset.UTC).format(DATE_TIME_FORMATTER);
 final String endDate =
endDateTime.atZone(ZoneOffset.UTC).format(DATE_TIME_FORMATTER);

```

```
 return queryWithDateTime(tableName, partitionKeyName, partitionKeyValue,
 dateKeyName, startDate, endDate);
 }
}
```

- For API details, see [Query](#) in *AWS SDK for Java 2.x API Reference*.

## Understand update expression order

The following code example shows how to understand update expression order.

- Learn how DynamoDB processes update expressions.
- Understand the order of operations in update expressions.
- Avoid unexpected results by understanding expression evaluation.

## SDK for Java 2.x

Demonstrate update expression order using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ReturnValue;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.Map;

/**
 * Demonstrates the effect of update expression order.
 *
 * <p>This method shows how the order of operations in an update expression
 * affects the result of the update.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 */
```

```

 * @return Map containing the results of different update orders
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static Map<String, Object> demonstrateUpdateOrder(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key) {

 Map<String, Object> results = new HashMap<>();

 try {
 // Initialize the item with a counter
 UpdateItemRequest initRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET Counter = :zero, OldCounter = :zero")
 .expressionAttributeValues(
 Map.of(":zero", AttributeValue.builder().n("0").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 dynamoDbClient.updateItem(initRequest);

 // Example 1: SET first, then ADD
 UpdateItemRequest setFirstRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET Counter = :value ADD OldCounter :increment")
 .expressionAttributeValues(Map.of(
 ":value", AttributeValue.builder().n("10").build(),
 ":increment", AttributeValue.builder().n("5").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 UpdateItemResponse setFirstResponse =
dynamoDbClient.updateItem(setFirstRequest);
 results.put("setFirstResponse", setFirstResponse);

 // Reset the item
 dynamoDbClient.updateItem(initRequest);

 // Example 2: ADD first, then SET
 UpdateItemRequest addFirstRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)

```

```

 .updateExpression("ADD Counter :increment SET OldCounter = :value")
 .expressionAttributeValues(Map.of(
 ":value", AttributeValue.builder().n("10").build(),
 ":increment", AttributeValue.builder().n("5").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 UpdateItemResponse addFirstResponse =
dynamoDbClient.updateItem(addFirstRequest);
 results.put("addFirstResponse", addFirstResponse);

 // Reset the item
 dynamoDbClient.updateItem(initRequest);

 // Example 3: SET with multiple attributes
 UpdateItemRequest multiSetRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET Counter = :value, OldCounter = Counter")
 .expressionAttributeValues(
 Map.of(":value", AttributeValue.builder().n("10").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 UpdateItemResponse multiSetResponse =
dynamoDbClient.updateItem(multiSetRequest);
 results.put("multiSetResponse", multiSetResponse);

 // Reset the item
 dynamoDbClient.updateItem(initRequest);

 // Example 4: SET with expression using the same attribute
 UpdateItemRequest selfReferenceRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET Counter = Counter + :increment, OldCounter =
Counter")
 .expressionAttributeValues(
 Map.of(":increment", AttributeValue.builder().n("5").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 UpdateItemResponse selfReferenceResponse =
dynamoDbClient.updateItem(selfReferenceRequest);

```

```

 results.put("selfReferenceResponse", selfReferenceResponse);

 results.put("success", true);

 } catch (DynamoDbException e) {
 results.put("success", false);
 results.put("error", e.getMessage());
 }

 return results;
}

/**
 * Updates an item with SET first, then REMOVE.
 *
 * <p>This method demonstrates updating an item with SET operation first,
 * followed by a REMOVE operation.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param attributeToSet The attribute to set
 * @param setValue The value to set
 * @param attributeToRemove The attribute to remove
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateWithSetFirst(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String attributeToSet,
 AttributeValue setValue,
 String attributeToRemove) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #setAttr = :setValue REMOVE #removeAttr")
 .expressionAttributeNames(Map.of(
 "#setAttr", attributeToSet,
 "#removeAttr", attributeToRemove))
 .expressionAttributeValues(Map.of(":setValue", setValue))

```

```

 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 try {
 return dynamoDbClient.updateItem(request);
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to update item with SET first: " + e.getMessage())
 .cause(e)
 .build();
 }
}

/**
 * Updates an item with REMOVE first, then SET.
 *
 * <p>This method demonstrates updating an item with REMOVE operation first,
 * followed by a SET operation.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param attributeToSet The attribute to set
 * @param setValue The value to set
 * @param attributeToRemove The attribute to remove
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateWithRemoveFirst(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String attributeToSet,
 AttributeValue setValue,
 String attributeToRemove) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("REMOVE #removeAttr SET #setAttr = :setValue")
 .expressionAttributeNames(Map.of(
 "#setAttr", attributeToSet,

```

```

 "#removeAttr", attributeToRemove))
 .expressionAttributeValues(Map.of(":setValue", setValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 try {
 return dynamoDbClient.updateItem(request);
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to update item with REMOVE first: " +
e.getMessage())
 .cause(e)
 .build();
 }
}

/**
 * Updates an item with all operation types in a specific order.
 *
 * <p>This method demonstrates using all operation types (SET, REMOVE, ADD,
DELETE)
 * in a specific order in a single update expression.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateWithAllOperationTypes(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #stringAttr = :stringVal, #mapAttr.#nestedAttr
= :nestedVal " + "REMOVE #oldAttr "
 + "ADD #counterAttr :increment "
 + "DELETE #stringSetAttr :stringSetVal")
 .expressionAttributeNames(Map.of(
 "#stringAttr", "StringAttribute",

```



```

 "#mapAttr", "MapAttribute",
 "#nestedAttr", "NestedAttribute",
 "#oldAttr", "OldAttribute",
 "#counterAttr", "CounterAttribute",
 "#stringSetAttr", "StringSetAttribute"))
 .expressionAttributeValues(Map.of(
 ":stringValue", AttributeValue.builder().s("New Value").build(),
 ":nestedVal", AttributeValue.builder().s("Nested Value").build(),
 ":increment", AttributeValue.builder().n("1").build(),
 ":stringSetVal", AttributeValue.builder().ss("Value1").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 try {
 return dynamoDbClient.updateItem(request);
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to update item with all operation types: " +
e.getMessage())
 .cause(e)
 .build();
 }
}

/**
 * Gets the current state of an item.
 *
 * <p>Helper method to retrieve the current state of an item.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to get
 * @return The item or null if not found
 * @throws DynamoDbException if an error occurs during the operation
 */
public static Map<String, AttributeValue> getItem(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key) {

 // Define the get parameters
 GetItemRequest request =
 GetItemRequest.builder().tableName(tableName).key(key).build();

```

```

 // Perform the get operation
 try {
 GetItemResponse response = dynamoDbClient.getItem(request);

 // Return the item if it exists, otherwise null
 return response.item();
 } catch (DynamoDbException e) {
 throw DynamoDbException.builder()
 .message("Failed to get item: " + e.getMessage())
 .cause(e)
 .build();
 }
}

```

### Example usage of update expression order with AWS SDK for Java 2.x.

```

public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating update expression order in DynamoDB");

 try {
 // Example 1: Demonstrate update order effects
 System.out.println("\nExample 1: Demonstrating update order effects");
 Map<String, Object> orderResults =
 demonstrateUpdateOrder(dynamoDbClient, tableName, key);

 if ((boolean) orderResults.get("success")) {
 System.out.println("SET first, then ADD:");
 System.out.println(" " + orderResults.get("setFirstResponse"));

 System.out.println("ADD first, then SET:");
 System.out.println(" " + orderResults.get("addFirstResponse"));

 System.out.println("SET with multiple attributes:");
 System.out.println(" " + orderResults.get("multiSetResponse"));

 System.out.println("SET with self-reference:");

```

```
 System.out.println(" " +
orderResults.get("selfReferenceResponse"));
 } else {
 System.out.println("Error: " + orderResults.get("error"));
 }

// Example 2: Update with SET first, then REMOVE
System.out.println("\nExample 2: Update with SET first, then REMOVE");
UpdateItemResponse setFirstResponse = updateWithSetFirst(
 dynamoDbClient,
 tableName,
 key,
 "Status",
 AttributeValue.builder().s("Active").build(),
 "OldStatus");

System.out.println("Updated attributes: " +
setFirstResponse.attributes());

// Example 3: Update with REMOVE first, then SET
System.out.println("\nExample 3: Update with REMOVE first, then SET");
UpdateItemResponse removeFirstResponse = updateWithRemoveFirst(
 dynamoDbClient,
 tableName,
 key,
 "Status",
 AttributeValue.builder().s("Inactive").build(),
 "OldStatus");

System.out.println("Updated attributes: " +
removeFirstResponse.attributes());

// Example 4: Update with all operation types
System.out.println("\nExample 4: Update with all operation types");
UpdateItemResponse allOpsResponse =
updateWithAllOperationTypes(dynamoDbClient, tableName, key);

System.out.println("Updated attributes: " +
allOpsResponse.attributes());

// Example 5: Get the current state of the item
System.out.println("\nExample 5: Current state of the item");
Map<String, AttributeValue> item = getItem(dynamoDbClient, tableName,
key);
```

```

 if (item != null) {
 System.out.println("Item: " + item);
 } else {
 System.out.println("Item not found");
 }

 // Explain update expression order
 System.out.println("\nKey points about update expression order in
DynamoDB:");
 System.out.println("1. Update expressions are processed in this order:
SET, REMOVE, ADD, DELETE");
 System.out.println("2. Within each clause, operations are processed from
left to right");
 System.out.println("3. SET operations use the item state before any
updates in the expression");
 System.out.println("4. When an attribute is referenced multiple times,
the first operation wins");
 System.out.println("5. To reference a new value, split the update into
multiple operations");
 System.out.println("6. The order of clauses in the expression doesn't
change the evaluation order");
 System.out.println("7. For complex updates, consider using multiple
separate update operations");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
 }
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Update a table's warm throughput setting

The following code example shows how to update a table's warm throughput setting.

### SDK for Java 2.x

Update warm throughput setting on an existing DynamoDB table using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
```

```

import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GlobalSecondaryIndexUpdate;
import
 software.amazon.awssdk.services.dynamodb.model.UpdateGlobalSecondaryIndexAction;
import software.amazon.awssdk.services.dynamodb.model.UpdateTableRequest;
import software.amazon.awssdk.services.dynamodb.model.WarmThroughput;

 public static WarmThroughput buildWarmThroughput(final Long readUnitsPerSecond,
final Long writeUnitsPerSecond) {
 return WarmThroughput.builder()
 .readUnitsPerSecond(readUnitsPerSecond)
 .writeUnitsPerSecond(writeUnitsPerSecond)
 .build();
 }

 /**
 * Updates a DynamoDB table with warm throughput settings for both the table and
 a global secondary index.
 *
 * @param ddb The DynamoDB client
 * @param tableName The name of the table to update
 * @param tableReadUnitsPerSecond Read units per second for the table
 * @param tableWriteUnitsPerSecond Write units per second for the table
 * @param globalSecondaryIndexName The name of the global secondary index to
 update
 * @param globalSecondaryIndexReadUnitsPerSecond Read units per second for the
 GSI
 * @param globalSecondaryIndexWriteUnitsPerSecond Write units per second for the
 GSI
 */
 public static void updateDynamoDBTable(
 final DynamoDbClient ddb,
 final String tableName,
 final Long tableReadUnitsPerSecond,
 final Long tableWriteUnitsPerSecond,
 final String globalSecondaryIndexName,
 final Long globalSecondaryIndexReadUnitsPerSecond,
 final Long globalSecondaryIndexWriteUnitsPerSecond) {

 final WarmThroughput tableWarmThroughput =
 buildWarmThroughput(tableReadUnitsPerSecond, tableWriteUnitsPerSecond);
 final WarmThroughput gsiWarmThroughput =
 buildWarmThroughput(globalSecondaryIndexReadUnitsPerSecond,
globalSecondaryIndexWriteUnitsPerSecond);

```

```

 final GlobalSecondaryIndexUpdate globalSecondaryIndexUpdate =
GlobalSecondaryIndexUpdate.builder()
 .update(UpdateGlobalSecondaryIndexAction.builder()
 .indexName(globalSecondaryIndexName)
 .warmThroughput(gsiWarmThroughput)
 .build())
 .build();

 final UpdateTableRequest request = UpdateTableRequest.builder()
 .tableName(tableName)
 .globalSecondaryIndexUpdates(globalSecondaryIndexUpdate)
 .warmThroughput(tableWarmThroughput)
 .build();

 try {
 ddb.updateTable(request);
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 throw e;
 }

 System.out.println(SUCCESS_MESSAGE);
}

```

- For API details, see [UpdateTable](#) in *AWS SDK for Java 2.x API Reference*.

## Update an item's TTL

The following code example shows how to update an item's TTL.

### SDK for Java 2.x

Update TTL on an existing DynamoDB item in a table.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

```

```

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

public UpdateItemResponse updateItemWithTTL(
 final String tableName, final String primaryKeyValue, final String
 sortKeyValue) {
 // Get current time in epoch second format
 final long currentTime = System.currentTimeMillis() / 1000;

 // Calculate expiration time 90 days from now in epoch second format
 final long expireDate = currentTime + (DAYS_TO_EXPIRE * SECONDS_PER_DAY);

 // Create the key map for the item to update
 final Map<String, AttributeValue> keyMap = new HashMap<>();
 keyMap.put(PRIMARY_KEY_ATTR,
AttributeValue.builder().s(primaryKeyValue).build());
 keyMap.put(SORT_KEY_ATTR, AttributeValue.builder().s(sortKeyValue).build());

 // Create the expression attribute values
 final Map<String, AttributeValue> expressionAttributeValues = new
HashMap<>();
 expressionAttributeValues.put(
 ":c", AttributeValue.builder().n(String.valueOf(currentTime)).build());
 expressionAttributeValues.put(
 ":e", AttributeValue.builder().n(String.valueOf(expireDate)).build());

 final UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(keyMap)
 .updateExpression(UPDATE_EXPRESSION)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 try {
 final UpdateItemResponse response = dynamoDbClient.updateItem(request);
 System.out.println(String.format(SUCCESS_MESSAGE, tableName));
 return response;
 } catch (ResourceNotFoundException e) {
 System.err.format(TABLE_NOT_FOUND_ERROR, tableName);
 throw e;
 } catch (DynamoDbException e) {
 System.err.println(e.getMessage());
 }
}

```

```
 throw e;
 }
}
```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Use API Gateway to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by Amazon API Gateway.

### SDK for Java 2.x

Shows how to create an AWS Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create a Lambda function invoked by Amazon API Gateway that scans an Amazon DynamoDB table for work anniversaries and uses Amazon Simple Notification Service (Amazon SNS) to send a text message to your employees that congratulates them at their one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon SNS

## Use Step Functions to invoke Lambda functions

The following code example shows how to create an AWS Step Functions state machine that invokes AWS Lambda functions in sequence.

### SDK for Java 2.x

Shows how to create an AWS serverless workflow by using AWS Step Functions and the AWS SDK for Java 2.x. Each workflow step is implemented using an AWS Lambda function.



For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Lambda
- Amazon SES
- Step Functions

### Use atomic counter operations

The following code example shows how to use atomic counter operations in DynamoDB.

- Increment counters atomically using ADD and SET operations.
- Safely increment counters that might not exist.
- Implement optimistic locking for counter operations.

### SDK for Java 2.x

Demonstrate atomic counter operations using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ReturnValue;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.Map;

/**
 * Increments a counter using the ADD operation.
 *
 * <p>This method demonstrates how to use the ADD operation to atomically
 * increment a counter attribute.
 *
 * @param dynamoDbClient The DynamoDB client
```

```

 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param counterName The name of the counter attribute
 * @param incrementValue The value to increment by
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse incrementCounterWithAdd(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String counterName,
 int incrementValue) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("ADD #counterName :increment")
 .expressionAttributeNames(Map.of("#counterName", counterName))
 .expressionAttributeValues(Map.of(
 ":increment",
 AttributeValue.builder().n(String.valueOf(incrementValue)).build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Increments a counter using the SET operation.
 *
 * <p>This method demonstrates how to use the SET operation with an expression
 * to increment a counter attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param counterName The name of the counter attribute
 * @param incrementValue The value to increment by
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */

```

```

public static UpdateItemResponse incrementCounterWithSet(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String counterName,
 int incrementValue) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #counterName = #counterName + :increment")
 .expressionAttributeNames(Map.of("#counterName", counterName))
 .expressionAttributeValues(Map.of(
 ":increment",
 AttributeValue.builder().n(String.valueOf(incrementValue)).build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Increments a counter safely, handling the case where the counter doesn't
 * exist yet.
 *
 * <p>This method demonstrates how to use if_not_exists to safely increment a
 * counter
 * that may not exist yet.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param counterName The name of the counter attribute
 * @param incrementValue The value to increment by
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse incrementCounterSafely(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String counterName,

```

```

 int incrementValue) {

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #counterName = if_not_exists(#counterName, :zero)
+ :increment")
 .expressionAttributeNames(Map.of("#counterName", counterName))
 .expressionAttributeValues(Map.of(
 ":increment",

AttributeValue.builder().n(String.valueOf(incrementValue)).build(),
 ":zero", AttributeValue.builder().n("0").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

/**
 * Decrements a counter safely, ensuring it doesn't go below zero.
 *
 * <p>This method demonstrates how to use a condition expression to safely
 * decrement a counter without going below zero.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param counterName The name of the counter attribute
 * @param decrementValue The value to decrement by
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation or if the
counter would go below zero
 */
public static UpdateItemResponse decrementCounterSafely(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String counterName,
 int decrementValue) {

 // Define the update parameters

```

```

 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #counterName = #counterName - :decrement")
 .conditionExpression("#counterName >= :decrement")
 .expressionAttributeNames(Map.of("#counterName", counterName))
 .expressionAttributeValues(Map.of(
 ":decrement",
 AttributeValue.builder().n(String.valueOf(decrementValue)).build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Compares the ADD and SET approaches for incrementing counters.
 *
 * <p>This method demonstrates the differences between using ADD and SET
 * for incrementing counters in DynamoDB.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @return Map containing the comparison results
 */
 public static Map<String, Object> compareAddVsSet(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
key) {

 Map<String, Object> results = new HashMap<>();

 try {
 // Reset counters to ensure a fair comparison
 UpdateItemRequest resetRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET AddCounter = :zero, SetCounter = :zero")
 .expressionAttributeValues(
 Map.of(":zero", AttributeValue.builder().n("0").build()))
 .build();

 dynamoDbClient.updateItem(resetRequest);

```

```

 // Increment with ADD
 long addStartTime = System.nanoTime();
 UpdateItemResponse addResponse = incrementCounterWithAdd(dynamoDbClient,
tableName, key, "AddCounter", 1);
 long addEndTime = System.nanoTime();
 long addDuration = addEndTime - addStartTime;

 // Increment with SET
 long setStartTime = System.nanoTime();
 UpdateItemResponse setResponse = incrementCounterWithSet(dynamoDbClient,
tableName, key, "SetCounter", 1);
 long setEndTime = System.nanoTime();
 long setDuration = setEndTime - setStartTime;

 // Record results
 results.put("addResponse", addResponse);
 results.put("setResponse", setResponse);
 results.put("addDuration", addDuration);
 results.put("setDuration", setDuration);
 results.put("success", true);

 } catch (DynamoDbException e) {
 results.put("success", false);
 results.put("error", e.getMessage());
 }

 return results;
}

/**
 * Gets the current value of a counter attribute.
 *
 * <p>Helper method to retrieve the current value of a counter attribute.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to get
 * @param counterName The name of the counter attribute
 * @return The counter value or null if not found
 * @throws DynamoDbException if an error occurs during the operation
 */
public static Integer getCounterValue(

```

```

 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
 key, String counterName) {

 // Define the get parameters
 GetItemRequest request = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .projectionExpression(counterName)
 .build();

 // Perform the get operation
 GetItemResponse response = dynamoDbClient.getItem(request);

 // Return the counter value if it exists, otherwise null
 if (response.item() != null && response.item().containsKey(counterName)) {
 return Integer.parseInt(response.item().get(counterName).n());
 }

 return null;
 }

```

### Example usage of atomic counter operations with AWS SDK for Java 2.x.

```

 public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
 {
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating atomic counter operations in DynamoDB");

 try {
 // Example 1: Increment a counter using ADD
 System.out.println("\nExample 1: Incrementing a counter using ADD");
 UpdateItemResponse addResponse = incrementCounterWithAdd(dynamoDbClient,
 tableName, key, "ViewCount", 1);

 System.out.println("Updated counter: " + addResponse.attributes());

 // Example 2: Increment a counter using SET
 System.out.println("\nExample 2: Incrementing a counter using SET");

```

```

 UpdateItemResponse setResponse = incrementCounterWithSet(dynamoDbClient,
 tableName, key, "LikeCount", 1);

 System.out.println("Updated counter: " + setResponse.attributes());

 // Example 3: Increment a counter safely
 System.out.println("\nExample 3: Incrementing a counter safely");
 UpdateItemResponse safeResponse = incrementCounterSafely(dynamoDbClient,
 tableName, key, "ShareCount", 1);

 System.out.println("Updated counter: " + safeResponse.attributes());

 // Example 4: Decrement a counter safely
 System.out.println("\nExample 4: Decrementing a counter safely");
 try {
 UpdateItemResponse decrementResponse =
 decrementCounterSafely(dynamoDbClient, tableName, key,
"InventoryCount", 1);

 System.out.println("Updated counter: " +
decrementResponse.attributes());
 } catch (DynamoDbException e) {
 if (e.getMessage().contains("ConditionalCheckFailed")) {
 System.out.println("Cannot decrement counter below zero");
 } else {
 throw e;
 }
 }

 // Example 5: Compare ADD vs SET
 System.out.println("\nExample 5: Comparing ADD vs SET");
 Map<String, Object> comparison = compareAddVsSet(dynamoDbClient,
 tableName, key);

 if ((boolean) comparison.get("success")) {
 System.out.println("ADD duration: " + comparison.get("addDuration")
+ " ns");
 System.out.println("SET duration: " + comparison.get("setDuration")
+ " ns");
 System.out.println("ADD response: " +
comparison.get("addResponse"));
 System.out.println("SET response: " +
comparison.get("setResponse"));
 } else {

```



```

 System.out.println("Comparison failed: " + comparison.get("error"));
 }

 // Explain atomic counter operations
 System.out.println("\nKey points about DynamoDB atomic counter
operations:");
 System.out.println("1. Both ADD and SET can be used for atomic
counters");
 System.out.println("2. ADD is more concise for simple increments");
 System.out.println("3. SET with an expression is more flexible for
complex operations");
 System.out.println("4. Use if_not_exists to handle the case where the
counter doesn't exist yet");
 System.out.println("5. Use condition expressions to prevent counters
from going below zero");
 System.out.println("6. Atomic operations are guaranteed to be isolated
from other writes");
 System.out.println("7. ADD can only be used with number and set data
types");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
}

```

- For API details, see [UpdateItem](#) in *AWS SDK for Java 2.x API Reference*.

## Use conditional operations

The following code example shows how to use conditional operations in DynamoDB.

- Implement conditional writes to prevent overwriting data.
- Use condition expressions to enforce business rules.
- Handle conditional check failures gracefully.

## SDK for Java 2.x

Demonstrate conditional operations using AWS SDK for Java 2.x.

```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import
 software.amazon.awssdk.services.dynamodb.model.ConditionalCheckFailedException;
import software.amazon.awssdk.services.dynamodb.model.DeleteItemRequest;
import software.amazon.awssdk.services.dynamodb.model.DeleteItemResponse;
import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;
import software.amazon.awssdk.services.dynamodb.model.ReturnValue;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.HashMap;
import java.util.Map;

/**
 * Performs a conditional update on an item.
 *
 * <p>This method demonstrates how to use a condition expression to update an
item
 * only if a specific condition is met.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param conditionAttribute The attribute to check in the condition
 * @param conditionValue The value to compare against
 * @param updateAttribute The attribute to update
 * @param updateValue The new value to set
 * @return Map containing the operation result and status
 */
public static Map<String, Object> conditionalUpdate(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String conditionAttribute,
 AttributeValue conditionValue,
 String updateAttribute,
 AttributeValue updateValue) {

 Map<String, Object> result = new HashMap<>();

```

```

 try {
 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #updateAttr = :updateVal")
 .conditionExpression("#condAttr = :condVal")
 .expressionAttributeNames(Map.of(
 "#condAttr", conditionAttribute,
 "#updateAttr", updateAttribute))
 .expressionAttributeValues(Map.of(
 ":condVal", conditionValue,
 ":updateVal", updateValue))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 UpdateItemResponse response = dynamoDbClient.updateItem(request);

 // Record success result
 result.put("success", true);
 result.put("message", "Condition was met and update was performed");
 result.put("attributes", response.attributes());

 } catch (ConditionalCheckFailedException e) {
 // Record failure due to condition not being met
 result.put("success", false);
 result.put("message", "Condition was not met, update was not
performed");
 result.put("error", "ConditionalCheckFailedException");

 } catch (DynamoDbException e) {
 // Record failure due to other errors
 result.put("success", false);
 result.put("message", "Error occurred: " + e.getMessage());
 result.put("error", e.getClass().getSimpleName());
 }

 return result;
}

/**
 * Performs a conditional delete on an item.
 */

```

```

 * <p>This method demonstrates how to use a condition expression to delete an
 item
 * only if a specific condition is met.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to delete
 * @param conditionAttribute The attribute to check in the condition
 * @param conditionValue The value to compare against
 * @return Map containing the operation result and status
 */
 public static Map<String, Object> conditionalDelete(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String conditionAttribute,
 AttributeValue conditionValue) {

 Map<String, Object> result = new HashMap<>();

 try {
 // Define the delete parameters
 DeleteItemRequest request = DeleteItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .conditionExpression("#condAttr = :condVal")
 .expressionAttributeNames(Map.of("#condAttr", conditionAttribute))
 .expressionAttributeValues(Map.of(":condVal", conditionValue))
 .returnValues(ReturnValue.ALL_OLD)
 .build();

 // Perform the delete operation
 DeleteItemResponse response = dynamoDbClient.deleteItem(request);

 // Record success result
 result.put("success", true);
 result.put("message", "Condition was met and delete was performed");
 result.put("attributes", response.attributes());

 } catch (ConditionalCheckFailedException e) {
 // Record failure due to condition not being met
 result.put("success", false);
 result.put("message", "Condition was not met, delete was not
performed");

```

```

 result.put("error", "ConditionalCheckFailedException");

 } catch (DynamoDbException e) {
 // Record failure due to other errors
 result.put("success", false);
 result.put("message", "Error occurred: " + e.getMessage());
 result.put("error", e.getClass().getSimpleName());
 }

 return result;
}

/**
 * Demonstrates optimistic locking using a version attribute.
 *
 * <p>This method shows how to implement optimistic locking by using a version
 * attribute that is incremented with each update.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param versionAttribute The name of the version attribute
 * @return Map containing the operation result
 */
public static Map<String, Object> optimisticLockingExample(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
 key, String versionAttribute) {

 Map<String, Object> result = new HashMap<>();

 try {
 // Get the current version of the item
 GetItemRequest getRequest = GetItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .projectionExpression(versionAttribute)
 .build();

 GetItemResponse getResponse = dynamoDbClient.getItem(getRequest);

 // Check if the item exists
 if (getResponse.item() == null || !
 getResponse.item().containsKey(versionAttribute)) {
 // Item doesn't exist or doesn't have a version attribute

```

```

 // Initialize with version 1
 UpdateItemRequest initRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #verAttr = :newVer, #dataAttr = :data")
 .expressionAttributeNames(Map.of("#verAttr", versionAttribute,
"#dataAttr", "Data"))
 .expressionAttributeValues(Map.of(
 ":newVer", AttributeValue.builder().n("1").build(),
 ":data", AttributeValue.builder().s("Initial
data").build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 UpdateItemResponse initResponse =
dynamoDbClient.updateItem(initRequest);

 result.put("operation", "initialize");
 result.put("success", true);
 result.put("attributes", initResponse.attributes());

 return result;
 }

 // Get the current version number
 int currentVersion =
 Integer.parseInt(getResponse.item().get(versionAttribute).n());
 int newVersion = currentVersion + 1;

 // Update the item with a condition on the version
 UpdateItemRequest updateRequest = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #verAttr = :newVer, #dataAttr = :newData")
 .conditionExpression("#verAttr = :curVer")
 .expressionAttributeNames(Map.of("#verAttr", versionAttribute,
"#dataAttr", "Data"))
 .expressionAttributeValues(Map.of(
 ":curVer",
 AttributeValue.builder()
 .n(String.valueOf(currentVersion))
 .build(),
 ":newVer",

```

```

AttributeValue.builder().n(String.valueOf(newVersion)).build(),
 ":newData",
 AttributeValue.builder()
 .s("Updated data at version " + newVersion)
 .build()))
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

UpdateItemResponse updateResponse =
dynamoDbClient.updateItem(updateRequest);

 // Record success result
 result.put("operation", "update");
 result.put("success", true);
 result.put("oldVersion", currentVersion);
 result.put("newVersion", newVersion);
 result.put("attributes", updateResponse.attributes());

} catch (ConditionalCheckFailedException e) {
 // Record failure due to version mismatch
 result.put("operation", "update");
 result.put("success", false);
 result.put("message", "Version mismatch, another process may have
updated the item");
 result.put("error", "ConditionalCheckFailedException");

} catch (DynamoDbException e) {
 // Record failure due to other errors
 result.put("operation", "update");
 result.put("success", false);
 result.put("message", "Error occurred: " + e.getMessage());
 result.put("error", e.getClass().getSimpleName());
}

return result;
}

/**
 * Performs a conditional update with multiple conditions.
 *
 * <p>This method demonstrates how to use multiple conditions in a condition
expression.
 *

```

```

 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param conditions Map of attribute names to values for conditions
 * @param updateAttribute The attribute to update
 * @param updateValue The new value to set
 * @return Map containing the operation result and status
 */
 public static Map<String, Object> conditionalUpdateWithMultipleConditions(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 Map<String, AttributeValue> conditions,
 String updateAttribute,
 AttributeValue updateValue) {

 Map<String, Object> result = new HashMap<>();

 try {
 // Build the condition expression and attribute names/values
 StringBuilder conditionExpression = new StringBuilder();
 Map<String, String> expressionAttributeNames = new HashMap<>();
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();

 // Add update attribute
 expressionAttributeNames.put("#updateAttr", updateAttribute);
 expressionAttributeValues.put(":updateVal", updateValue);

 // Add conditions
 int i = 0;
 for (Map.Entry<String, AttributeValue> condition :
conditions.entrySet()) {
 String attrName = condition.getKey();
 AttributeValue attrValue = condition.getValue();

 String nameKey = "#cond" + i;
 String valueKey = ":val" + i;

 expressionAttributeNames.put(nameKey, attrName);
 expressionAttributeValues.put(valueKey, attrValue);

 // Add AND between conditions (except for the first one)
 if (i > 0) {
 conditionExpression.append(" AND ");

```



```

 }

 conditionExpression.append(nameKey).append(" = ").append(valueKey);
 i++;
 }

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #updateAttr = :updateVal")
 .conditionExpression(conditionExpression.toString())
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .returnValues(ReturnValue.UPDATED_NEW)
 .build();

 // Perform the update operation
 UpdateItemResponse response = dynamoDbClient.updateItem(request);

 // Record success result
 result.put("success", true);
 result.put("message", "All conditions were met and update was
performed");
 result.put("attributes", response.attributes());

 } catch (ConditionalCheckFailedException e) {
 // Record failure due to condition not being met
 result.put("success", false);
 result.put("message", "One or more conditions were not met, update was
not performed");
 result.put("error", "ConditionalCheckFailedException");

 } catch (DynamoDbException e) {
 // Record failure due to other errors
 result.put("success", false);
 result.put("message", "Error occurred: " + e.getMessage());
 result.put("error", e.getClass().getSimpleName());
 }

 return result;
}

```

## Example usage of conditional operations with AWS SDK for Java 2.x.

```
public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating conditional operations in DynamoDB");

 try {
 // Example 1: Conditional update
 System.out.println("\nExample 1: Conditional update");
 Map<String, Object> updateResult = conditionalUpdate(
 dynamoDbClient,
 tableName,
 key,
 "InStock",
 AttributeValue.builder().bool(true).build(),
 "Status",
 AttributeValue.builder().s("Available").build());

 System.out.println("Update result: " + updateResult.get("message"));
 if ((boolean) updateResult.get("success")) {
 System.out.println("Updated attributes: " +
updateResult.get("attributes"));
 }

 // Example 2: Conditional delete
 System.out.println("\nExample 2: Conditional delete");
 Map<String, Object> deleteResult = conditionalDelete(
 dynamoDbClient,
 tableName,
 key,
 "Status",
 AttributeValue.builder().s("Discontinued").build());

 System.out.println("Delete result: " + deleteResult.get("message"));
 if ((boolean) deleteResult.get("success")) {
 System.out.println("Deleted item: " +
deleteResult.get("attributes"));
 }

 // Example 3: Optimistic locking
 }
}
```

```

 System.out.println("\nExample 3: Optimistic locking");
 Map<String, Object> lockingResult =
optimisticLockingExample(dynamoDbClient, tableName, key, "Version");

 System.out.println("Optimistic locking result:");
 System.out.println(" Operation: " + lockingResult.get("operation"));
 System.out.println(" Success: " + lockingResult.get("success"));
 if (lockingResult.get("operation").equals("update") && (boolean)
lockingResult.get("success")) {
 System.out.println(" Old version: " +
lockingResult.get("oldVersion"));
 System.out.println(" New version: " +
lockingResult.get("newVersion"));
 }
 System.out.println(" Attributes: " + lockingResult.get("attributes"));

 // Example 4: Multiple conditions
 System.out.println("\nExample 4: Multiple conditions");
 Map<String, AttributeValue> conditions = new HashMap<>();
 conditions.put("Price", AttributeValue.builder().n("199.99").build());
 conditions.put("Category",
AttributeValue.builder().s("Electronics").build());

 Map<String, Object> multiConditionResult =
conditionalUpdateWithMultipleConditions(
 dynamoDbClient,
 tableName,
 key,
 conditions,
 "OnSale",
 AttributeValue.builder().bool(true).build());

 System.out.println("Multiple conditions result: " +
multiConditionResult.get("message"));
 if ((boolean) multiConditionResult.get("success")) {
 System.out.println("Updated attributes: " +
multiConditionResult.get("attributes"));
 }

 // Explain conditional operations
 System.out.println("\nKey points about DynamoDB conditional
operations:");
 System.out.println("1. Conditional operations only succeed if the
condition is met");

```

```
 System.out.println("2. ConditionalCheckFailedException is thrown when
the condition fails");
 System.out.println("3. No changes are made to the item if the condition
fails");
 System.out.println("4. Conditions can be used with update, delete, and
put operations");
 System.out.println("5. Multiple conditions can be combined with AND and
OR");
 System.out.println("6. Optimistic locking can be implemented using a
version attribute");
 System.out.println(
 "7. Conditional operations consume the same amount of write capacity
whether they succeed or fail");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteItem](#)
  - [PutItem](#)
  - [UpdateItem](#)

## Use expression attribute names

The following code example shows how to use expression attribute names in DynamoDB.

- Work with reserved words in DynamoDB expressions.
- Use expression attribute name placeholders.
- Handle special characters in attribute names.

## SDK for Java 2.x

Demonstrate expression attribute names using AWS SDK for Java 2.x.

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
```

```

import software.amazon.awssdk.services.dynamodb.model.DynamoDbException;
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanResponse;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;
import software.amazon.awssdk.services.dynamodb.model.UpdateItemResponse;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Updates an attribute that is a reserved word in DynamoDB.
 *
 * <p>This method demonstrates how to use expression attribute names to update
 * attributes that are reserved words in DynamoDB.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param reservedWordAttribute The reserved word attribute to update
 * @param value The value to set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static UpdateItemResponse updateReservedWordAttribute(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String reservedWordAttribute,
 AttributeValue value) {

 // Define the update parameters using expression attribute names
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #attr = :value")
 .expressionAttributeNames(Map.of("#attr", reservedWordAttribute))
 .expressionAttributeValues(Map.of(":value", value))
 .returnValues("UPDATED_NEW")
 .build();

```

```

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Updates an attribute that contains special characters.
 *
 * <p>This method demonstrates how to use expression attribute names to update
 * attributes that contain special characters.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param key The key of the item to update
 * @param specialCharAttribute The attribute with special characters to update
 * @param value The value to set
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
 public static UpdateItemResponse updateSpecialCharacterAttribute(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 String specialCharAttribute,
 AttributeValue value) {

 // Define the update parameters using expression attribute names
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET #attr = :value")
 .expressionAttributeNames(Map.of("#attr", specialCharAttribute))
 .expressionAttributeValues(Map.of(":value", value))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
 }

 /**
 * Queries items using an attribute that is a reserved word.
 *
 * <p>This method demonstrates how to use expression attribute names in a query
 * when the attribute is a reserved word.

```

```

*
* @param dynamoDbClient The DynamoDB client
* @param tableName The name of the DynamoDB table
* @param partitionKeyName The name of the partition key attribute
* @param partitionKeyValue The value of the partition key
* @param reservedWordAttribute The reserved word attribute to filter on
* @param value The value to compare against
* @return The response from DynamoDB
* @throws DynamoDbException if an error occurs during the operation
*/
public static QueryResponse queryWithReservedWordAttribute(
 DynamoDbClient dynamoDbClient,
 String tableName,
 String partitionKeyName,
 AttributeValue partitionKeyValue,
 String reservedWordAttribute,
 AttributeValue value) {

 // Define the query parameters using expression attribute names
 Map<String, String> expressionAttributeNames = new HashMap<>();
 expressionAttributeNames.put("#pkName", partitionKeyName);
 expressionAttributeNames.put("#attr", reservedWordAttribute);

 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 expressionAttributeValues.put(":pkValue", partitionKeyValue);
 expressionAttributeValues.put(":value", value);

 QueryRequest request = QueryRequest.builder()
 .tableName(tableName)
 .keyConditionExpression("#pkName = :pkValue")
 .filterExpression("#attr = :value")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

 // Perform the query operation
 return dynamoDbClient.query(request);
}

/**
 * Updates a nested attribute with a path that contains reserved words.
 *
 * <p>This method demonstrates how to use expression attribute names to update
 * nested attributes where the path contains reserved words.

```

```

*
* @param dynamoDbClient The DynamoDB client
* @param tableName The name of the DynamoDB table
* @param key The key of the item to update
* @param attributePath The path to the nested attribute as an array
* @param value The value to set
* @return The response from DynamoDB
* @throws DynamoDbException if an error occurs during the operation
*/
public static UpdateItemResponse updateNestedReservedWordAttribute(
 DynamoDbClient dynamoDbClient,
 String tableName,
 Map<String, AttributeValue> key,
 List<String> attributePath,
 AttributeValue value) {

 // Create expression attribute names for each part of the path
 Map<String, String> expressionAttributeNames = new HashMap<>();
 for (int i = 0; i < attributePath.size(); i++) {
 expressionAttributeNames.put("#attr" + i, attributePath.get(i));
 }

 // Build the attribute path using the expression attribute names
 StringBuilder attributePathExpression = new StringBuilder();
 for (int i = 0; i < attributePath.size(); i++) {
 if (i > 0) {
 attributePathExpression.append(".");
 }
 attributePathExpression.append("#attr").append(i);
 }

 // Define the update parameters
 UpdateItemRequest request = UpdateItemRequest.builder()
 .tableName(tableName)
 .key(key)
 .updateExpression("SET " + attributePathExpression.toString() + "
= :value")
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(Map.of(":value", value))
 .returnValues("UPDATED_NEW")
 .build();

 // Perform the update operation
 return dynamoDbClient.updateItem(request);
}

```



```

}

/**
 * Scans a table with multiple attribute name placeholders.
 *
 * <p>This method demonstrates how to use multiple expression attribute names
 * in a complex filter expression.
 *
 * @param dynamoDbClient The DynamoDB client
 * @param tableName The name of the DynamoDB table
 * @param filters Object mapping attribute names to filter values
 * @return The response from DynamoDB
 * @throws DynamoDbException if an error occurs during the operation
 */
public static ScanResponse scanWithMultipleAttributeNames(
 DynamoDbClient dynamoDbClient, String tableName, Map<String, AttributeValue>
filters) {

 // Create expression attribute names and values
 Map<String, String> expressionAttributeNames = new HashMap<>();
 Map<String, AttributeValue> expressionAttributeValues = new HashMap<>();
 StringBuilder filterExpression = new StringBuilder();

 // Build the filter expression
 int index = 0;
 for (Map.Entry<String, AttributeValue> entry : filters.entrySet()) {
 String attrName = entry.getKey();
 AttributeValue attrValue = entry.getValue();

 String nameKey = "#attr" + index;
 String valueKey = ":val" + index;

 expressionAttributeNames.put(nameKey, attrName);
 expressionAttributeValues.put(valueKey, attrValue);

 // Add AND between conditions (except for the first one)
 if (index > 0) {
 filterExpression.append(" AND ");
 }

 filterExpression.append(nameKey).append(" = ").append(valueKey);
 index++;
 }
}

```

```
// Define the scan parameters
ScanRequest request = ScanRequest.builder()
 .tableName(tableName)
 .filterExpression(filterExpression.toString())
 .expressionAttributeNames(expressionAttributeNames)
 .expressionAttributeValues(expressionAttributeValues)
 .build();

// Perform the scan operation
return dynamoDbClient.scan(request);
}
```

### Example usage of expression attribute names with AWS SDK for Java 2.x.

```
public static void exampleUsage(DynamoDbClient dynamoDbClient, String tableName)
{
 // Example key
 Map<String, AttributeValue> key = new HashMap<>();
 key.put("ProductId", AttributeValue.builder().s("P12345").build());

 System.out.println("Demonstrating expression attribute names in DynamoDB");

 try {
 // Example 1: Update an attribute that is a reserved word
 System.out.println("\nExample 1: Updating an attribute that is a
reserved word");
 UpdateItemResponse response1 = updateReservedWordAttribute(
 dynamoDbClient,
 tableName,
 key,
 "Size", // "SIZE" is a reserved word in DynamoDB
 AttributeValue.builder().s("Large").build());

 System.out.println("Updated attribute: " + response1.attributes());

 // Example 2: Update an attribute with special characters
 System.out.println("\nExample 2: Updating an attribute with special
characters");
 UpdateItemResponse response2 = updateSpecialCharacterAttribute(
 dynamoDbClient,
 tableName,
 key,
```

```

 "Product-Type", // Contains a hyphen, which is a special character
 AttributeValue.builder().s("Electronics").build());

 System.out.println("Updated attribute: " + response2.attributes());

 // Example 3: Query with a reserved word attribute
 System.out.println("\nExample 3: Querying with a reserved word
attribute");
 QueryResponse response3 = queryWithReservedWordAttribute(
 dynamoDbClient,
 tableName,
 "Category",
 AttributeValue.builder().s("Electronics").build(),
 "Count", // "COUNT" is a reserved word in DynamoDB
 AttributeValue.builder().n("10").build());

 System.out.println("Found " + response3.count() + " items");

 // Example 4: Update a nested attribute with reserved words in the path
 System.out.println("\nExample 4: Updating a nested attribute with
reserved words in the path");
 UpdateItemResponse response4 = updateNestedReservedWordAttribute(
 dynamoDbClient,
 tableName,
 key,
 Arrays.asList("Dimensions", "Size", "Height"), // "SIZE" is a
reserved word
 AttributeValue.builder().n("30").build());

 System.out.println("Updated nested attribute: " +
response4.attributes());

 // Example 5: Scan with multiple attribute name placeholders
 System.out.println("\nExample 5: Scanning with multiple attribute name
placeholders");
 Map<String, AttributeValue> filters = new HashMap<>();
 filters.put("Size", AttributeValue.builder().s("Large").build());
 filters.put("Count", AttributeValue.builder().n("10").build());
 filters.put(
 "Product-Type", AttributeValue.builder().s("Electronics").build());

 ScanResponse response5 = scanWithMultipleAttributeNames(dynamoDbClient,
tableName, filters);

```

```

 System.out.println("Found " + response5.count() + " items");

 // Show some common reserved words
 System.out.println("\nSome common DynamoDB reserved words:");
 List<String> commonReservedWords = getDynamoDBReservedWords();
 System.out.println(String.join(", ", commonReservedWords));

 // Explain expression attribute names
 System.out.println("\nKey points about expression attribute names:");
 System.out.println("1. Use expression attribute names (#name) for reserved words");
 System.out.println("2. Use expression attribute names for attributes with special characters");
 System.out.println("3. Special characters include: spaces, hyphens, dots, and other non-alphanumeric characters");
 System.out.println("4. Expression attribute names are required for nested attributes with reserved words");
 System.out.println("5. You can use multiple expression attribute names in a single expression");
 System.out.println("6. Expression attribute names are case-sensitive");
 System.out.println("7. Expression attribute names are only used in expressions, not in the actual data");

 } catch (DynamoDbException e) {
 System.err.println("Error: " + e.getMessage());
 e.printStackTrace();
 }
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [Query](#)
  - [UpdateItem](#)

## Use scheduled events to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by an Amazon EventBridge scheduled event.

## SDK for Java 2.x

Shows how to create an Amazon EventBridge scheduled event that invokes an AWS Lambda function. Configure EventBridge to use a cron expression to schedule when the Lambda function is invoked. In this example, you create a Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create an app that sends a mobile text message to your employees that congratulates them at the one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- CloudWatch Logs
- DynamoDB
- EventBridge
- Lambda
- Amazon SNS

## Serverless examples

### Invoke a Lambda function from a DynamoDB trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a DynamoDB stream. The function retrieves the DynamoDB payload and logs the record contents.

## SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming a DynamoDB event with Lambda using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
```

```
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.DynamodbEvent;
import
 com.amazonaws.services.lambda.runtime.events.DynamodbEvent.DynamodbStreamRecord;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

public class example implements RequestHandler<DynamodbEvent, Void> {

 private static final Gson GSON = new GsonBuilder().setPrettyPrinting().create();

 @Override
 public Void handleRequest(DynamodbEvent event, Context context) {
 System.out.println(GSON.toJson(event));
 event.getRecords().forEach(this::logDynamoDBRecord);
 return null;
 }

 private void logDynamoDBRecord(DynamodbStreamRecord record) {
 System.out.println(record.getEventID());
 System.out.println(record.getEventName());
 System.out.println("DynamoDB Record: " + GSON.toJson(record.getDynamodb()));
 }
}
```

## Reporting batch item failures for Lambda functions with a DynamoDB trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from a DynamoDB stream. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Reporting DynamoDB batch item failures with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.DynamodbEvent;
import com.amazonaws.services.lambda.runtime.events.StreamsEventResponse;
import com.amazonaws.services.lambda.runtime.events.models.dynamodb.StreamRecord;

import java.util.ArrayList;
import java.util.List;

public class ProcessDynamodbRecords implements RequestHandler<DynamodbEvent,
StreamsEventResponse> {

 @Override
 public StreamsEventResponse handleRequest(DynamodbEvent input, Context context)
 {

 List<StreamsEventResponse.BatchItemFailure> batchItemFailures = new
ArrayList<>();
 String curRecordSequenceNumber = "";

 for (DynamodbEvent.DynamodbStreamRecord dynamodbStreamRecord :
input.getRecords()) {
 try {
 //Process your record
 StreamRecord dynamodbRecord = dynamodbStreamRecord.getDynamodb();
 curRecordSequenceNumber = dynamodbRecord.getSequenceNumber();

 } catch (Exception e) {
 /* Since we are working with streams, we can return the failed item
immediately.
 Lambda will immediately begin to retry processing from this
failed item onwards. */
 batchItemFailures.add(new
StreamsEventResponse.BatchItemFailure(curRecordSequenceNumber));
 return new StreamsEventResponse(batchItemFailures);
 }
 }

 return new StreamsEventResponse();
 }
}
```

## AWS community contributions

### Build and test a serverless application

The following code example shows how to build and test a serverless application using API Gateway with Lambda and DynamoDB

#### SDK for Java 2.x

Shows how to build and test a serverless application that consists of an API Gateway with Lambda and DynamoDB using the Java SDK.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- API Gateway
- DynamoDB
- Lambda

## Amazon EC2 examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon EC2.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics



- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Amazon EC2

The following code example shows how to get started using Amazon EC2.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously describes the security groups for the specified group ID.
 *
 * @param groupName the name of the security group to describe
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of describing the security groups. The future will complete with a
 * {@link DescribeSecurityGroupsResponse} object that contains the
 * security group information.
 */
public CompletableFuture<String> describeSecurityGroupArnByNameAsync(String
groupName) {
 DescribeSecurityGroupsRequest request =
DescribeSecurityGroupsRequest.builder()
 .groupNames(groupName)
 .build();

 DescribeSecurityGroupsPublisher paginator =
getAsyncClient().describeSecurityGroupsPaginator(request);
 AtomicReference<String> groupIdRef = new AtomicReference<>();
```

```
return paginator.subscribe(response -> {
 response.securityGroups().stream()
 .filter(securityGroup ->
securityGroup.groupName().equals(groupName))
 .findFirst()
 .ifPresent(securityGroup ->
groupIdRef.set(securityGroup.groupId()));
 }).thenApply(v -> {
 String groupId = groupIdRef.get();
 if (groupId == null) {
 throw new RuntimeException("No security group found with the name: "
+ groupName);
 }
 return groupId;
 }).exceptionally(ex -> {
 logger.info("Failed to describe security group: " + ex.getMessage());
 throw new RuntimeException("Failed to describe security group", ex);
 });
}
```

- For API details, see [DescribeSecurityGroups](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a key pair and security group.
- Select an Amazon Machine Image (AMI) and compatible instance type, then create an instance.
- Stop and restart the instance.
- Associate an Elastic IP address with your instance.
- Connect to your instance with SSH, then clean up resources.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run a scenario at a command prompt.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.ec2.model.CreateKeyPairResponse;
import software.amazon.awssdk.services.ec2.model.DeleteKeyPairResponse;
import software.amazon.awssdk.services.ec2.model.DescribeKeyPairsResponse;
import software.amazon.awssdk.services.ec2.model.DisassociateAddressResponse;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.ReleaseAddressResponse;
import software.amazon.awssdk.services.ssm.model.GetParametersByPathResponse;
import software.amazon.awssdk.services.ssm.model.Parameter;

import java.net.InetAddress;
import java.net.UnknownHostException;
import java.util.List;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java example performs the following tasks:
 *
 * 1. Creates an RSA key pair and saves the private key data as a .pem file.
 * 2. Lists key pairs.
 * 3. Creates a security group for the default VPC.
 * 4. Displays security group information.
 * 5. Gets a list of Amazon Linux 2 AMIs and selects one.
```

```

* 6. Gets additional information about the image.
* 7. Gets a list of instance types that are compatible with the selected AMI's
* architecture.
* 8. Creates an instance with the key pair, security group, AMI, and an
* instance type.
* 9. Displays information about the instance.
* 10. Stops the instance and waits for it to stop.
* 11. Starts the instance and waits for it to start.
* 12. Allocates an Elastic IP address and associates it with the instance.
* 13. Displays SSH connection info for the instance.
* 14. Disassociates and deletes the Elastic IP address.
* 15. Terminates the instance and waits for it to terminate.
* 16. Deletes the security group.
* 17. Deletes the key pair.
*/
public class EC2Scenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final Logger logger = LoggerFactory.getLogger(EC2Scenario.class);
 public static void main(String[] args) throws InterruptedException,
 UnknownHostException {

 logger.info("""
 Usage:
 <keyName> <fileName> <groupName> <groupDesc>

 Where:
 keyName - A key pair name (for example, TestKeyPair).\s
 fileName - A file name where the key information is written to.\s
 groupName - The name of the security group.\s
 groupDesc - The description of the security group.\s
 """);

 Scanner scanner = new Scanner(System.in);
 EC2Actions ec2Actions = new EC2Actions();

 String keyName = "TestKeyPair7" ;
 String fileName = "ec2Key.pem";
 String groupName = "TestSecGroup7" ;
 String groupDesc = "Test Group" ;
 String vpcId = ec2Actions.describeFirstEC2VpcAsync().join().vpcId();
 InetAddress localAddress = InetAddress.getLocalHost();
 String myIpAddress = localAddress.getHostAddress();

```

```
logger.info("""
 Amazon Elastic Compute Cloud (EC2) is a web service that provides
secure, resizable compute
 capacity in the cloud. It allows developers and organizations to easily
launch and manage
 virtual server instances, known as EC2 instances, to run their
applications.
```

```

 EC2 provides a wide range of instance types, each with different
compute, memory,
 and storage capabilities, to meet the diverse needs of various
workloads. Developers
 can choose the appropriate instance type based on their application's
requirements,
 such as high-performance computing, memory-intensive tasks, or GPU-
accelerated workloads.
```

```

 The `Ec2AsyncClient` interface in the AWS SDK for Java 2.x provides a
set of methods to
 programmatically interact with the Amazon EC2 service. This allows
developers to
 automate the provisioning, management, and monitoring of EC2 instances
as part of their
 application deployment pipelines. With EC2, teams can focus on building
and deploying
 their applications without having to worry about the underlying
infrastructure
 required to host and manage physical servers.
```

```

 This scenario walks you through how to perform key operations for this
service.
```

```
 Let's get started...
 """);
```

```

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("1. Create an RSA key pair and save the private key material as
a .pem file.");
 logger.info("""
 An RSA key pair for Amazon EC2 is a security mechanism used to
authenticate and secure
```

```

 access to your EC2 instances. It consists of a public key and a private
key,
 which are generated as a pair.
 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<CreateKeyPairResponse> future =
ec2Actions.createKeyPairAsync(keyName, fileName);
 CreateKeyPairResponse response = future.join();
 logger.info("Key Pair successfully created. Key Fingerprint: " +
response.keyFingerprint());

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 if (ec2Ex.getMessage().contains("already exists")) {
 // Key pair already exists.
 logger.info("The key pair '" + keyName + "' already exists.
Moving on...");
 } else {
 logger.info("EC2 error occurred: Error message: {}, Error code
{}", ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 }
 } else {
 logger.info("An unexpected error occurred: " + (rt.getMessage()));
 return;
 }
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("2. List key pairs.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<DescribeKeyPairsResponse> future =
ec2Actions.describeKeysAsync();
 DescribeKeyPairsResponse keyPairsResponse = future.join();
 keyPairsResponse.keyPairs().forEach(keyPair -> logger.info(
 "Found key pair with name {} and fingerprint {}",
 keyPair.keyName(),
 keyPair.keyFingerprint()));
 }

```

```

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Error message: {}, Error code {}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", (cause != null ?
cause.getMessage() : rt.getMessage()));
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Create a security group.");
 logger.info("""
 An AWS EC2 Security Group is a virtual firewall that controls the
 inbound and outbound traffic to an EC2 instance. It acts as a first
line
 of defense for your EC2 instances, allowing you to specify the rules
that
 govern the network traffic entering and leaving your instances.
 """);
 waitForInputToContinue(scanner);
 String groupId = "";
 try {
 CompletableFuture<String> future =
ec2Actions.createSecurityGroupAsync(groupName, groupDesc, vpcId, myIpAddress);
 future.join();
 logger.info("Created security group");

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 if (ec2Ex.awsErrorDetails().errorMessage().contains("already
exists")) {
 logger.info("The Security Group already exists. Moving on...");
 } else {
 logger.error("An unexpected error occurred: {}",
ec2Ex.awsErrorDetails().errorMessage());
 return;
 }
 }
 }

```

```

 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("4. Display security group information for the new security
group.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<String> future =
ec2Actions.describeSecurityGroupArnByNameAsync(groupName);
 groupId = future.join();
 logger.info("The security group Id is "+groupId);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 String errorCode = ec2Ex.awsErrorDetails().errorCode();
 if ("InvalidGroup.NotFound".equals(errorCode)) {
 logger.info("Security group '{}' does not exist. Error Code:
{}", groupName, errorCode);
 } else {
 logger.info("EC2 error occurred: Message {}, Error Code: {}",
ec2Ex.getMessage(), errorCode);
 }
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("5. Get a list of Amazon Linux 2 AMIs and select one with amzn2
in the name.");
 logger.info(""""
 An Amazon EC2 AMI (Amazon Machine Image) is a pre-configured virtual
machine image that

```



serves as a template for launching EC2 instances. It contains all the necessary software and configurations required to run an application or operating system on an EC2 instance.

```

 """);
 waitForInputToContinue(scanner);
 String instanceAMI="";
 try {
 CompletableFuture<GetParametersByPathResponse> future =
ec2Actions.getParaValuesAsync();
 GetParametersByPathResponse pathResponse = future.join();
 List<Parameter> parameterList = pathResponse.parameters();
 for (Parameter para : parameterList) {
 if (filterName(para.name())) {
 instanceAMI = para.value();
 break;
 }
 }
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 logger.info("The AMI value with amzn2 is: {}", instanceAMI);
 waitForInputToContinue(scanner);
 logger.info(DASHES);

```

```

 logger.info(DASHES);
 logger.info("6. Get the (Amazon Machine Image) AMI value from the amzn2
image.");

```

```

 logger.info("""
 An AMI value represents a specific version of a virtual machine (VM) or
server image.

 It uniquely identifies a particular version of an EC2 instance, including
its operating system,
 pre-installed software, and any custom configurations. This allows you to
consistently deploy the same
 VM image across your infrastructure.
 """);

```

```

 "");
 waitForInputToContinue(scanner);
 String amiValue;
 try {
 CompletableFuture<String> future =
ec2Actions.describeImageAsync(instanceAMI);
 amiValue = future.join();

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof Ec2Exception) {
 Ec2Exception ec2Ex = (Ec2Exception) cause;
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Retrieves an instance type available in the current AWS
region.");
 waitForInputToContinue(scanner);
 String instanceType;
 try {
 CompletableFuture<String> future = ec2Actions.getInstanceTypesAsync();
 instanceType = future.join();
 if (!instanceType.isEmpty()) {
 logger.info("Found instance type: " + instanceType);
 } else {
 logger.info("Desired instance type not found.");
 }
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {

```

```

 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("8. Create an Amazon EC2 instance using the key pair, the
instance type, the security group, and the EC2 AMI value.");
logger.info("Once the EC2 instance is created, it is placed into a running
state.");
waitForInputToContinue(scanner);
String newInstanceId;
try {
 CompletableFuture<String> future =
ec2Actions.runInstanceAsync(instanceType, keyName, groupName, amiValue);
 newInstanceId = future.join();
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception) {
 Ec2Exception ec2Ex = (Ec2Exception) cause;
 switch (ec2Ex.awsErrorDetails().errorCode()) {
 case "InvalidParameterValue":
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 break;
 case "InsufficientInstanceCapacity":
 // Handle insufficient instance capacity.
 logger.info("Insufficient instance capacity: {}, {}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 break;
 case "InvalidGroup.NotFound":
 // Handle security group not found.
 logger.info("Security group not found: {},{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 break;
 default:
 logger.info("EC2 error occurred: {} (Code: {})",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 break;
 }
 }
 return;
} else {

```

```

 logger.info("An unexpected error occurred: {}", (cause != null ?
cause.getMessage() : rt.getMessage()));
 return;
 }
}
logger.info("The instance Id is " + newInstanceId);
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("9. Display information about the running instance. ");

waitForInputToContinue(scanner);
String publicIp;
try {
 CompletableFuture<String> future =
ec2Actions.describeEC2InstancesAsync(newInstanceId);
 publicIp = future.join();
 logger.info("EC2 instance public IP {}", publicIp);
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}

logger.info("You can SSH to the instance using this command:");
logger.info("ssh -i " + fileName + " ec2-user@" + publicIp);
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("10. Stop the instance using a waiter (this may take a few
mins).");
// Remove the 2nd one
waitForInputToContinue(scanner);
try {
 CompletableFuture<Void> future =
ec2Actions.stopInstanceAsync(newInstanceId);

```

```

 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("11. Start the instance using a waiter (this may take a few
mins).");
 try {
 CompletableFuture<Void> future =
ec2Actions.startInstanceAsync(newInstanceId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 // Handle EC2 exceptions.
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("12. Allocate an Elastic IP address and associate it with the
instance.");
 logger.info("""

```

An Elastic IP address is a static public IP address that you can associate with your EC2 instance.

This allows you to have a fixed, predictable IP address that remains the same even if your instance is stopped, terminated, or replaced.

This is particularly useful for applications or services that need to be accessed consistently from a known IP address.

An EC2 Allocation ID (also known as a Reserved Instance Allocation ID) is a unique identifier associated with a Reserved Instance (RI) that you have purchased in AWS.

When you purchase a Reserved Instance, AWS assigns a unique Allocation ID to it.

This Allocation ID is used to track and identify the specific RI you have purchased, and it is important for managing and monitoring your Reserved Instances.

```

 "");

waitForInputToContinue(scanner);
String allocationId;
try {
 CompletableFuture<String> future = ec2Actions.allocateAddressAsync();
 allocationId = future.join();
 logger.info("Successfully allocated address with ID: " +allocationId);
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}
logger.info("The allocation Id value is " + allocationId);
waitForInputToContinue(scanner);
String associationId;
try {
 CompletableFuture<String> future =
ec2Actions.associateAddressAsync(newInstanceId, allocationId);

```

```

 associationId = future.join();
 logger.info("Successfully associated address with ID: " +associationId);
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("13. Describe the instance again. Note that the public IP
address has changed");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<String> future =
ec2Actions.describeEC2InstancesAsync(newInstanceId);
 publicIp = future.join();
 logger.info("EC2 instance public IP: " + publicIp);
 logger.info("You can SSH to the instance using this command:");
 logger.info("ssh -i " + fileName + " ec2-user@" + publicIp);
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("14. Disassociate and release the Elastic IP address.");
 waitForInputToContinue(scanner);

```

```

 try {
 CompletableFuture<DisassociateAddressResponse> future =
ec2Actions.disassociateAddressAsync(associationId);
 future.join();
 logger.info("Address successfully disassociated.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 // Handle EC2 exceptions.
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
 }
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<ReleaseAddressResponse> future =
ec2Actions.releaseEC2AddressAsync(allocationId);
 future.join(); // Wait for the operation to complete
 logger.info("Elastic IP address successfully released.");
 } catch (RuntimeException rte) {
 logger.info("An unexpected error occurred: {}", rte.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("15. Terminate the instance and use a waiter (this may take a
few mins).");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Object> future =
ec2Actions.terminateEC2Async(newInstanceId);
 future.join();
 logger.info("EC2 instance successfully terminated.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 // Handle EC2 exceptions.

```



```

 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}
logger.info(DASHES);

logger.info(DASHES);
logger.info("16. Delete the security group.");
waitForInputToContinue(scanner);
try {
 CompletableFuture<Void> future =
ec2Actions.deleteEC2SecGroupAsync(groupId);
 future.join();
 logger.info("Security group successfully deleted.");
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {
 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("17. Delete the key.");
waitForInputToContinue(scanner);
try {
 CompletableFuture<DeleteKeyPairResponse> future =
ec2Actions.deleteKeysAsync(keyName);
 future.join();
 logger.info("Successfully deleted key pair named " + keyName);
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof Ec2Exception ec2Ex) {

```

```

 logger.info("EC2 error occurred: Message {}, Error Code:{}",
ec2Ex.getMessage(), ec2Ex.awsErrorDetails().errorCode());
 return;
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage());
 return;
 }
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("You successfully completed the Amazon EC2 scenario.");
logger.info(DASHES);
}

public static boolean filterName(String name) {
 String[] parts = name.split("/");
 String myValue = parts[4];
 return myValue.contains("amzn2");
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}
}
}
}

```

Define a class that wraps EC2 actions.

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.AllocateAddressRequest;
import software.amazon.awssdk.services.ec2.model.AllocateAddressResponse;
import software.amazon.awssdk.services.ec2.model.AssociateAddressRequest;
import software.amazon.awssdk.services.ec2.model.AssociateAddressResponse;
import
 software.amazon.awssdk.services.ec2.model.AuthorizeSecurityGroupIngressRequest;
import software.amazon.awssdk.services.ec2.model.CreateKeyPairRequest;
import software.amazon.awssdk.services.ec2.model.CreateKeyPairResponse;
import software.amazon.awssdk.services.ec2.model.CreateSecurityGroupRequest;
import software.amazon.awssdk.services.ec2.model.DeleteKeyPairRequest;
import software.amazon.awssdk.services.ec2.model.DeleteKeyPairResponse;
import software.amazon.awssdk.services.ec2.model.DeleteSecurityGroupRequest;
import software.amazon.awssdk.services.ec2.model.DeleteSecurityGroupResponse;
import software.amazon.awssdk.services.ec2.model.DescribeImagesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstanceTypesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstanceTypesResponse;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeKeyPairsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsRequest;
import software.amazon.awssdk.services.ec2.model.DescribeSecurityGroupsResponse;
import software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest;
import software.amazon.awssdk.services.ec2.model.DisassociateAddressRequest;
import software.amazon.awssdk.services.ec2.model.DisassociateAddressResponse;
import software.amazon.awssdk.services.ec2.model.DomainType;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.Filter;
import software.amazon.awssdk.services.ec2.model.InstanceTypeInfo;
import software.amazon.awssdk.services.ec2.model.IpPermission;
import software.amazon.awssdk.services.ec2.model.IpRange;
import software.amazon.awssdk.services.ec2.model.ReleaseAddressRequest;
import software.amazon.awssdk.services.ec2.model.ReleaseAddressResponse;
import software.amazon.awssdk.services.ec2.model.RunInstancesRequest;
import software.amazon.awssdk.services.ec2.model.RunInstancesResponse;
import software.amazon.awssdk.services.ec2.model.StopInstancesRequest;
import software.amazon.awssdk.services.ec2.model.StartInstancesRequest;
import software.amazon.awssdk.services.ec2.model.TerminateInstancesRequest;
import software.amazon.awssdk.services.ec2.model.Vpc;
import software.amazon.awssdk.services.ec2.paginators.DescribeImagesPublisher;
```

```

import software.amazon.awssdk.services.ec2.paginators.DescribeInstancesPublisher;
import
 software.amazon.awssdk.services.ec2.paginators.DescribeSecurityGroupsPublisher;
import software.amazon.awssdk.services.ec2.paginators.DescribeVpcsPublisher;
import software.amazon.awssdk.services.ec2.waiters.Ec2AsyncWaiter;
import software.amazon.awssdk.services.ssm.SsmAsyncClient;
import software.amazon.awssdk.services.ssm.model.GetParametersByPathRequest;
import software.amazon.awssdk.services.ssm.model.GetParametersByPathResponse;
import software.amazon.awssdk.services.ec2.model.TerminateInstancesResponse;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.concurrent.atomic.AtomicReference;

public class EC2Actions {
 private static final Logger logger = LoggerFactory.getLogger(EC2Actions.class);
 private static Ec2AsyncClient ec2AsyncClient;

 /**
 * Retrieves an asynchronous Amazon Elastic Container Registry (ECR) client.
 *
 * @return the configured ECR asynchronous client.
 */
 private static Ec2AsyncClient getAsyncClient() {
 if (ec2AsyncClient == null) {
 /**
 * The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
 version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
 client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
 communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
 responses.
 */
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
 timeout.
 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.

```

```

 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.
 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .build();

 ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return ec2AsyncClient;
}

/**
 * Deletes a key pair asynchronously.
 *
 * @param keyPair the name of the key pair to delete
 * @return a {@link CompletableFuture} that represents the result of the
asynchronous operation.
 *
 * The {@link CompletableFuture} will complete with a {@link
DeleteKeyPairResponse} object
 *
 * that provides the result of the key pair deletion operation.
 */
public CompletableFuture<DeleteKeyPairResponse> deleteKeysAsync(String keyPair)
{
 DeleteKeyPairRequest request = DeleteKeyPairRequest.builder()
 .keyName(keyPair)
 .build();

 // Initiate the asynchronous request to delete the key pair.
 CompletableFuture<DeleteKeyPairResponse> response =
getAsyncClient().deleteKeyPair(request);
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to delete key pair: " + keyPair,
ex);
 } else if (resp == null) {

```

```

 throw new RuntimeException("No response received for deleting key
pair: " + keyPair);
 }
 });
}

/**
 * Deletes an EC2 security group asynchronously.
 *
 * @param groupId the ID of the security group to delete
 * @return a CompletableFuture that completes when the security group is deleted
 */
public CompletableFuture<Void> deleteEC2SecGroupAsync(String groupId) {
 DeleteSecurityGroupRequest request = DeleteSecurityGroupRequest.builder()
 .groupId(groupId)
 .build();

 CompletableFuture<DeleteSecurityGroupResponse> response =
getAsyncClient().deleteSecurityGroup(request);
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to delete security group with Id
" + groupId, ex);
 } else if (resp == null) {
 throw new RuntimeException("No response received for deleting
security group with Id " + groupId);
 }
 }).thenApply(resp -> null);
}

/**
 * Terminates an EC2 instance asynchronously and waits for it to reach the
terminated state.
 *
 * @param instanceId the ID of the EC2 instance to terminate
 * @return a {@link CompletableFuture} that completes when the instance has been
terminated
 * @throws RuntimeException if there is no response from the AWS SDK or if there
is a failure during the termination process
 */
public CompletableFuture<Object> terminateEC2Async(String instanceId) {
 TerminateInstancesRequest terminateRequest =
TerminateInstancesRequest.builder()
 .instanceIds(instanceId)

```

```

 .build();

 CompletableFuture<TerminateInstancesResponse> responseFuture =
getAsyncClient().terminateInstances(terminateRequest);
 return responseFuture.thenCompose(terminateResponse -> {
 if (terminateResponse == null) {
 throw new RuntimeException("No response received for terminating
instance " + instanceId);
 }
 System.out.println("Going to terminate an EC2 instance and use a waiter
to wait for it to be in terminated state");
 return getAsyncClient().waiter()
 .waitUntilInstanceTerminated(r -> r.instanceIds(instanceId))
 .thenApply(waiterResponse -> null);
 }).exceptionally(throwable -> {
 // Handle any exceptions that occurred during the async call
 throw new RuntimeException("Failed to terminate EC2 instance: " +
throwable.getMessage(), throwable);
 });
 }

 /**
 * Releases an Elastic IP address asynchronously.
 *
 * @param allocId the allocation ID of the Elastic IP address to be released
 * @return a {@link CompletableFuture} representing the asynchronous operation
of releasing the Elastic IP address
 */
 public CompletableFuture<ReleaseAddressResponse> releaseEC2AddressAsync(String
allocId) {
 ReleaseAddressRequest request = ReleaseAddressRequest.builder()
 .allocationId(allocId)
 .build();

 CompletableFuture<ReleaseAddressResponse> response =
getAsyncClient().releaseAddress(request);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to release Elastic IP address",
ex);
 }
 });

 return response;
 }

```

```

 }

 /**
 * Disassociates an Elastic IP address from an instance asynchronously.
 *
 * @param associationId The ID of the association you want to disassociate.
 * @return a {@link CompletableFuture} representing the asynchronous operation
 of disassociating the address. The
 * {@link CompletableFuture} will complete with a {@link
 DisassociateAddressResponse} when the operation is
 * finished.
 * @throws RuntimeException if the disassociation of the address fails.
 */
 public CompletableFuture<DisassociateAddressResponse>
 disassociateAddressAsync(String associationId) {
 Ec2AsyncClient ec2 = getAsyncClient();
 DisassociateAddressRequest addressRequest =
 DisassociateAddressRequest.builder()
 .associationId(associationId)
 .build();

 // Disassociate the address asynchronously.
 CompletableFuture<DisassociateAddressResponse> response =
 ec2.disassociateAddress(addressRequest);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to disassociate address", ex);
 }
 });

 return response;
 }

 /**
 * Associates an Elastic IP address with an EC2 instance asynchronously.
 *
 * @param instanceId the ID of the EC2 instance to associate the Elastic IP
 address with
 * @param allocationId the allocation ID of the Elastic IP address to associate
 * @return a {@link CompletableFuture} that completes with the association ID
 when the operation is successful,
 * or throws a {@link RuntimeException} if the operation fails
 */

```



```

 public CompletableFuture<String> associateAddressAsync(String instanceId, String
allocationId) {
 AssociateAddressRequest associateRequest = AssociateAddressRequest.builder()
 .instanceId(instanceId)
 .allocationId(allocationId)
 .build();

 CompletableFuture<AssociateAddressResponse> responseFuture =
getAsyncClient().associateAddress(associateRequest);
 return responseFuture.thenApply(response -> {
 if (response.associationId() != null) {
 return response.associationId();
 } else {
 throw new RuntimeException("Association ID is null after associating
address.");
 }
 }).whenComplete((result, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to associate address", ex);
 }
 });
 }

 /**
 * Allocates an Elastic IP address asynchronously in the VPC domain.
 *
 * @return a {@link CompletableFuture} containing the allocation ID of the
allocated Elastic IP address
 */
 public CompletableFuture<String> allocateAddressAsync() {
 AllocateAddressRequest allocateRequest = AllocateAddressRequest.builder()
 .domain(DomainType.VPC)
 .build();

 CompletableFuture<AllocateAddressResponse> responseFuture =
getAsyncClient().allocateAddress(allocateRequest);
 return
responseFuture.thenApply(AllocateAddressResponse::allocationId).whenComplete((result,
ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to allocate address", ex);
 }
 });
 }
}

```

```

/**
 * Asynchronously describes the state of an EC2 instance.
 * The paginator helps you iterate over multiple pages of results.
 *
 * @param newInstanceId the ID of the EC2 instance to describe
 * @return a {@link CompletableFuture} that, when completed, contains a string
describing the state of the EC2 instance
 */
public CompletableFuture<String> describeEC2InstancesAsync(String newInstanceId)
{
 DescribeInstancesRequest request = DescribeInstancesRequest.builder()
 .instanceIds(newInstanceId)
 .build();

 DescribeInstancesPublisher paginator =
getAsyncClient().describeInstancesPaginator(request);
 AtomicReference<String> publicIpAddressRef = new AtomicReference<>();
 return paginator.subscribe(response -> {
 response.reservations().stream()
 .flatMap(reservation -> reservation.instances().stream())
 .filter(instance -> instance.instanceId().equals(newInstanceId))
 .findFirst()
 .ifPresent(instance ->
publicIpAddressRef.set(instance.publicIpAddress()));
 }).thenApply(v -> {
 String publicIpAddress = publicIpAddressRef.get();
 if (publicIpAddress == null) {
 throw new RuntimeException("Instance with ID " + newInstanceId + "
not found.");
 }
 return publicIpAddress;
 }).exceptionally(ex -> {
 logger.info("Failed to describe instances: " + ex.getMessage());
 throw new RuntimeException("Failed to describe instances", ex);
 });
}

/**
 * Runs an EC2 instance asynchronously.
 *
 * @param instanceType The instance type to use for the EC2 instance.
 * @param keyName The name of the key pair to associate with the EC2 instance.

```

```

 * @param groupName The name of the security group to associate with the EC2
instance.
 * @param amiId The ID of the Amazon Machine Image (AMI) to use for the EC2
instance.
 * @return A {@link CompletableFuture} that completes with the ID of the started
EC2 instance.
 * @throws RuntimeException If there is an error running the EC2 instance.
 */
 public CompletableFuture<String> runInstanceAsync(String instanceType, String
keyName, String groupName, String amiId) {
 RunInstancesRequest runRequest = RunInstancesRequest.builder()
 .instanceType(instanceType)
 .keyName(keyName)
 .securityGroups(groupName)
 .maxCount(1)
 .minCount(1)
 .imageId(amiId)
 .build();

 CompletableFuture<RunInstancesResponse> responseFuture =
getAsyncClient().runInstances(runRequest);
 return responseFuture.thenCompose(response -> {
 String instanceIdVal = response.instances().get(0).instanceId();
 System.out.println("Going to start an EC2 instance and use a waiter to
wait for it to be in running state");
 return getAsyncClient().waiter()
 .waitUntilInstanceExists(r -> r.instanceIds(instanceIdVal))
 .thenCompose(waitResponse -> getAsyncClient().waiter()
 .waitUntilInstanceRunning(r -> r.instanceIds(instanceIdVal))
 .thenApply(runningResponse -> instanceIdVal));
 }).exceptionally(throwable -> {
 // Handle any exceptions that occurred during the async call
 throw new RuntimeException("Failed to run EC2 instance: " +
throwable.getMessage(), throwable);
 });
 }

 /**
 * Asynchronously retrieves the instance types available in the current AWS
region.
 * <p>
 * This method uses the AWS SDK's asynchronous API to fetch the available
instance types

```

```

 * and then processes the response. It logs the memory information, network
 information,
 * and instance type for each instance type returned. Additionally, it returns a
 * {@link CompletableFuture} that resolves to the instance type string for the
 "t2.2xlarge"
 * instance type, if it is found in the response. If the "t2.2xlarge" instance
 type is not
 * found, an empty string is returned.
 * </p>
 *
 * @return a {@link CompletableFuture} that resolves to the instance type string
 for the
 * "t2.2xlarge" instance type, or an empty string if the instance type is not
 found
 */
 public CompletableFuture<String> getInstanceTypesAsync() {
 DescribeInstanceTypesRequest typesRequest =
DescribeInstanceTypesRequest.builder()
 .maxResults(10)
 .build();

 CompletableFuture<DescribeInstanceTypesResponse> response =
getAsyncClient().describeInstanceTypes(typesRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 List<InstanceTypeInfo> instanceTypes = resp.instanceTypes();
 for (InstanceTypeInfo type : instanceTypes) {
 logger.info("The memory information of this type is " +
type.memoryInfo().sizeInMiB());
 logger.info("Network information is " +
type.networkInfo().toString());
 logger.info("Instance type is " +
type.instanceType().toString());
 }
 } else {
 throw (RuntimeException) ex;
 }
 });

 return response.thenApply(resp -> {
 for (InstanceTypeInfo type : resp.instanceTypes()) {
 String instanceType = type.instanceType().toString();
 if (instanceType.equals("t2.2xlarge")) {
 return instanceType;
 }
 }
 });
 }

```

```

 }
 }
 return "";
});
}

/**
 * Asynchronously describes an AWS EC2 image with the specified image ID.
 *
 * @param imageId the ID of the image to be described
 * @return a {@link CompletableFuture} that, when completed, contains the ID of
the described image
 * @throws RuntimeException if no images are found with the provided image ID,
or if an error occurs during the AWS API call
 */
public CompletableFuture<String> describeImageAsync(String imageId) {
 DescribeImagesRequest imagesRequest = DescribeImagesRequest.builder()
 .imageIds(imageId)
 .build();

 AtomicReference<String> imageIdRef = new AtomicReference<>();
 DescribeImagesPublisher paginator =
getAsyncClient().describeImagesPaginator(imagesRequest);
 return paginator.subscribe(response -> {
 response.images().stream()
 .filter(image -> image.imageId().equals(imageId))
 .findFirst()
 .ifPresent(image -> {
 logger.info("The description of the image is " +
image.description());
 logger.info("The name of the image is " + image.name());
 imageIdRef.set(image.imageId());
 });
 }).thenApply(v -> {
 String id = imageIdRef.get();
 if (id == null) {
 throw new RuntimeException("No images found with the provided image
ID.");
 }
 return id;
 }).exceptionally(ex -> {
 logger.info("Failed to describe image: " + ex.getMessage());
 throw new RuntimeException("Failed to describe image", ex);
 });
}

```

```

 }

 /**
 * Retrieves the parameter values asynchronously using the AWS Systems Manager
 * (SSM) API.
 *
 * @return a {@link CompletableFuture} that holds the response from the SSM API
 * call to get parameters by path
 */
 public CompletableFuture<GetParametersByPathResponse> getParaValuesAsync() {
 SsmAsyncClient ssmClient = SsmAsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 GetParametersByPathRequest parameterRequest =
 GetParametersByPathRequest.builder()
 .path("/aws/service/ami-amazon-linux-latest")
 .build();

 // Create a CompletableFuture to hold the final result.
 CompletableFuture<GetParametersByPathResponse> responseFuture = new
 CompletableFuture<>();
 ssmClient.getParametersByPath(parameterRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 responseFuture.completeExceptionally(new
 RuntimeException("Failed to get parameters by path", exception));
 } else {
 responseFuture.complete(response);
 }
 });

 return responseFuture;
 }

 /**
 * Asynchronously describes the security groups for the specified group ID.
 *
 * @param groupName the name of the security group to describe
 * @return a {@link CompletableFuture} that represents the asynchronous
 * operation
 *
 * of describing the security groups. The future will complete with a
 * {@link DescribeSecurityGroupsResponse} object that contains the

```

```

 * security group information.
 */
 public CompletableFuture<String> describeSecurityGroupArnByNameAsync(String
groupName) {
 DescribeSecurityGroupsRequest request =
DescribeSecurityGroupsRequest.builder()
 .groupNames(groupName)
 .build();

 DescribeSecurityGroupsPublisher paginator =
getAsyncClient().describeSecurityGroupsPaginator(request);
 AtomicReference<String> groupIdRef = new AtomicReference<>();
 return paginator.subscribe(response -> {
 response.securityGroups().stream()
 .filter(securityGroup ->
securityGroup.groupName().equals(groupName))
 .findFirst()
 .ifPresent(securityGroup ->
groupIdRef.set(securityGroup.groupId()));
 }).thenApply(v -> {
 String groupId = groupIdRef.get();
 if (groupId == null) {
 throw new RuntimeException("No security group found with the name: "
+ groupName);
 }
 return groupId;
 }).exceptionally(ex -> {
 logger.info("Failed to describe security group: " + ex.getMessage());
 throw new RuntimeException("Failed to describe security group", ex);
 });
 }

 /**
 * Creates a new security group asynchronously with the specified group name,
description, and VPC ID. It also
 * authorizes inbound traffic on ports 80 and 22 from the specified IP address.
 *
 * @param groupName the name of the security group to create
 * @param groupDesc the description of the security group
 * @param vpcId the ID of the VPC in which to create the security group
 * @param myIpAddress the IP address from which to allow inbound traffic (e.g.,
"192.168.1.1/0" to allow traffic from
 * any IP address in the 192.168.1.0/24 subnet)

```

```

 * @return a CompletableFuture that, when completed, returns the ID of the
 created security group
 * @throws RuntimeException if there was a failure creating the security group
 or authorizing the inbound traffic
 */
 public CompletableFuture<String> createSecurityGroupAsync(String groupName,
String groupDesc, String vpcId, String myIpAddress) {
 CreateSecurityGroupRequest createRequest =
CreateSecurityGroupRequest.builder()
 .groupName(groupName)
 .description(groupDesc)
 .vpcId(vpcId)
 .build();

 return getAsyncClient().createSecurityGroup(createRequest)
 .thenCompose(createResponse -> {
 String groupId = createResponse.groupId();
 IpRange ipRange = IpRange.builder()
 .cidrIp(myIpAddress + "/32")
 .build();

 IpPermission ipPerm = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(80)
 .fromPort(80)
 .ipRanges(ipRange)
 .build();

 IpPermission ipPerm2 = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(22)
 .fromPort(22)
 .ipRanges(ipRange)
 .build();

 AuthorizeSecurityGroupIngressRequest authRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(groupName)
 .ipPermissions(ipPerm, ipPerm2)
 .build();

 return getAsyncClient().authorizeSecurityGroupIngress(authRequest)
 .thenApply(authResponse -> groupId);
 })
 }

```



```

 .whenComplete((result, exception) -> {
 if (exception != null) {
 if (exception instanceof CompletionException &&
exception.getCause() instanceof Ec2Exception) {
 throw (Ec2Exception) exception.getCause();
 } else {
 throw new RuntimeException("Failed to create security group:
" + exception.getMessage(), exception);
 }
 }
 });
 }

 /**
 * Asynchronously describes the key pairs associated with the current AWS
account.
 *
 * @return a {@link CompletableFuture} containing the {@link
DescribeKeyPairsResponse} object, which provides
 * information about the key pairs.
 */
 public CompletableFuture<DescribeKeyPairsResponse> describeKeysAsync() {
 CompletableFuture<DescribeKeyPairsResponse> responseFuture =
getAsyncClient().describeKeyPairs();
 responseFuture.whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to describe key pairs: " +
exception.getMessage(), exception);
 }
 });

 return responseFuture;
 }

 /**
 * Creates a new key pair asynchronously.
 *
 * @param keyName the name of the key pair to create
 * @param fileName the name of the file to write the key material to
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of creating the key pair and writing the key material to a file
 */

```

```

 public CompletableFuture<CreateKeyPairResponse> createKeyPairAsync(String
keyName, String fileName) {
 CreateKeyPairRequest request = CreateKeyPairRequest.builder()
 .keyName(keyName)
 .build();

 CompletableFuture<CreateKeyPairResponse> responseFuture =
getAsyncClient().createKeyPair(request);
 responseFuture.whenComplete((response, exception) -> {
 if (response != null) {
 try {
 BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName));
 writer.write(response.keyMaterial());
 writer.close();
 } catch (IOException e) {
 throw new RuntimeException("Failed to write key material to
file: " + e.getMessage(), e);
 }
 } else {
 throw new RuntimeException("Failed to create key pair: " +
exception.getMessage(), exception);
 }
 });

 return responseFuture;
 }

/**
 * Describes the first default VPC asynchronously and using a paginator.
 *
 * @return a {@link CompletableFuture} that, when completed, contains the first
default VPC found.
 */
 public CompletableFuture<Vpc> describeFirstEC2VpcAsync() {
 Filter myFilter = Filter.builder()
 .name("is-default")
 .values("true")
 .build();

 DescribeVpcsRequest request = DescribeVpcsRequest.builder()
 .filters(myFilter)
 .build();

```

```

 DescribeVpcsPublisher paginator =
getAsyncClient().describeVpcsPaginator(request);
 AtomicReference<Vpc> vpcRef = new AtomicReference<>();
 return paginator.subscribe(response -> {
 response.vpcs().stream()
 .findFirst()
 .ifPresent(vpcRef::set);
 }).thenApply(v -> {
 Vpc vpc = vpcRef.get();
 if (vpc == null) {
 throw new RuntimeException("Default VPC not found");
 }
 return vpc;
 }).exceptionally(ex -> {
 logger.info("Failed to describe VPCs: " + ex.getMessage());
 throw new RuntimeException("Failed to describe VPCs", ex);
 });
 }

 /**
 * Stops the EC2 instance with the specified ID asynchronously and waits for the
 * instance to stop.
 *
 * @param instanceId the ID of the EC2 instance to stop
 * @return a {@link CompletableFuture} that completes when the instance has been
 * stopped, or exceptionally if an error occurs
 */
 public CompletableFuture<Void> stopInstanceAsync(String instanceId) {
 StopInstancesRequest stopRequest = StopInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 DescribeInstancesRequest describeRequest =
DescribeInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 Ec2AsyncWaiter ec2Waiter = Ec2AsyncWaiter.builder()
 .client(getAsyncClient())
 .build();

 CompletableFuture<Void> resultFuture = new CompletableFuture<>();
 logger.info("Stopping instance " + instanceId + " and waiting for it to
stop.");
 }

```

```

 getAsyncClient().stopInstances(stopRequest)
 .thenCompose(response -> {
 if (response.stoppingInstances().isEmpty()) {
 return CompletableFuture.failedFuture(new RuntimeException("No
instances were stopped. Please check the instance ID: " + instanceId));
 }
 return ec2Waiter.waitUntilInstanceStopped(describeRequest);
 })
 .thenAccept(waiterResponse -> {
 logger.info("Successfully stopped instance " + instanceId);
 resultFuture.complete(null);
 })
 .exceptionally(throwable -> {
 logger.error("Failed to stop instance " + instanceId + ": " +
throwable.getMessage(), throwable);
 resultFuture.completeExceptionally(new RuntimeException("Failed to
stop instance: " + throwable.getMessage(), throwable));
 return null;
 });

 return resultFuture;
 }

 /**
 * Starts an Amazon EC2 instance asynchronously and waits until it is in the
"running" state.
 *
 * @param instanceId the ID of the instance to start
 * @return a {@link CompletableFuture} that completes when the instance has been
started and is in the "running" state, or exceptionally if an error occurs
 */
 public CompletableFuture<Void> startInstanceAsync(String instanceId) {
 StartInstancesRequest startRequest = StartInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 Ec2AsyncWaiter ec2Waiter = Ec2AsyncWaiter.builder()
 .client(getAsyncClient())
 .build();

 DescribeInstancesRequest describeRequest =
DescribeInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

```

```
 logger.info("Starting instance " + instanceId + " and waiting for it to
run.");
 CompletableFuture<Void> resultFuture = new CompletableFuture<>();
 return getAsyncClient().startInstances(startRequest)
 .thenCompose(response ->
 ec2Waiter.waitForInstanceRunning(describeRequest)
)
 .thenAccept(waiterResponse -> {
 logger.info("Successfully started instance " + instanceId);
 resultFuture.complete(null);
 })
 .exceptionally(throwable -> {
 resultFuture.completeExceptionally(new RuntimeException("Failed to
start instance: " + throwable.getMessage(), throwable));
 return null;
 });
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [AllocateAddress](#)
- [AssociateAddress](#)
- [AuthorizeSecurityGroupIngress](#)
- [CreateKeyPair](#)
- [CreateSecurityGroup](#)
- [DeleteKeyPair](#)
- [DeleteSecurityGroup](#)
- [DescribeImages](#)
- [DescribeInstanceTypes](#)
- [DescribeInstances](#)
- [DescribeKeyPairs](#)
- [DescribeSecurityGroups](#)
- [DisassociateAddress](#)
- [ReleaseAddress](#)

- [RunInstances](#)
- [StartInstances](#)
- [StopInstances](#)
- [TerminateInstances](#)
- [UnmonitorInstances](#)

## Actions

### AllocateAddress

The following code example shows how to use `AllocateAddress`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Allocates an Elastic IP address asynchronously in the VPC domain.
 *
 * @return a {@link CompletableFuture} containing the allocation ID of the
 * allocated Elastic IP address
 */
public CompletableFuture<String> allocateAddressAsync() {
 AllocateAddressRequest allocateRequest = AllocateAddressRequest.builder()
 .domain(DomainType.VPC)
 .build();

 CompletableFuture<AllocateAddressResponse> responseFuture =
getAsyncClient().allocateAddress(allocateRequest);
 return
responseFuture.thenApply(AllocateAddressResponse::allocationId).whenComplete((result,
ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to allocate address", ex);
 }
 });
}
```

```
}
```

- For API details, see [AllocateAddress](#) in *AWS SDK for Java 2.x API Reference*.

## AssociateAddress

The following code example shows how to use `AssociateAddress`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Associates an Elastic IP address with an EC2 instance asynchronously.
 *
 * @param instanceId the ID of the EC2 instance to associate the Elastic IP
address with
 * @param allocationId the allocation ID of the Elastic IP address to associate
 * @return a {@link CompletableFuture} that completes with the association ID
when the operation is successful,
 * or throws a {@link RuntimeException} if the operation fails
 */
public CompletableFuture<String> associateAddressAsync(String instanceId, String
allocationId) {
 AssociateAddressRequest associateRequest = AssociateAddressRequest.builder()
 .instanceId(instanceId)
 .allocationId(allocationId)
 .build();

 CompletableFuture<AssociateAddressResponse> responseFuture =
getAsyncClient().associateAddress(associateRequest);
 return responseFuture.thenApply(response -> {
 if (response.associationId() != null) {
 return response.associationId();
 } else {
 throw new RuntimeException("Association ID is null after associating
address.");
 }
 });
}
```

```

 }
 }).whenComplete((result, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to associate address", ex);
 }
 });
}

```

- For API details, see [AssociateAddress](#) in *AWS SDK for Java 2.x API Reference*.

## AuthorizeSecurityGroupIngress

The following code example shows how to use `AuthorizeSecurityGroupIngress`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new security group asynchronously with the specified group name,
 * description, and VPC ID. It also
 * * authorizes inbound traffic on ports 80 and 22 from the specified IP address.
 *
 * @param groupName the name of the security group to create
 * @param groupDesc the description of the security group
 * @param vpcId the ID of the VPC in which to create the security group
 * @param myIpAddress the IP address from which to allow inbound traffic (e.g.,
 * "192.168.1.1/0" to allow traffic from
 * * any IP address in the 192.168.1.0/24 subnet)
 * @return a CompletableFuture that, when completed, returns the ID of the
 * created security group
 * @throws RuntimeException if there was a failure creating the security group
 * or authorizing the inbound traffic
 */
public CompletableFuture<String> createSecurityGroupAsync(String groupName,
String groupDesc, String vpcId, String myIpAddress) {

```



```

 CreateSecurityGroupRequest createRequest =
CreateSecurityGroupRequest.builder()
 .groupName(groupName)
 .description(groupDesc)
 .vpcId(vpcId)
 .build();

return getAsyncClient().createSecurityGroup(createRequest)
 .thenCompose(createResponse -> {
 String groupId = createResponse.groupId();
 IpRange ipRange = IpRange.builder()
 .cidrIp(myIpAddress + "/32")
 .build();

 IpPermission ipPerm = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(80)
 .fromPort(80)
 .ipRanges(ipRange)
 .build();

 IpPermission ipPerm2 = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(22)
 .fromPort(22)
 .ipRanges(ipRange)
 .build();

 AuthorizeSecurityGroupIngressRequest authRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(groupName)
 .ipPermissions(ipPerm, ipPerm2)
 .build();

 return getAsyncClient().authorizeSecurityGroupIngress(authRequest)
 .thenApply(authResponse -> groupId);
 })
 .whenComplete((result, exception) -> {
 if (exception != null) {
 if (exception instanceof CompletionException &&
exception.getCause() instanceof Ec2Exception) {
 throw (Ec2Exception) exception.getCause();
 } else {

```

```

 throw new RuntimeException("Failed to create security group:
" + exception.getMessage(), exception);
 }
}
});
}

```

- For API details, see [AuthorizeSecurityGroupIngress](#) in *AWS SDK for Java 2.x API Reference*.

## CreateKeyPair

The following code example shows how to use `CreateKeyPair`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new key pair asynchronously.
 *
 * @param keyName the name of the key pair to create
 * @param fileName the name of the file to write the key material to
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of creating the key pair and writing the key material to a file
 */
public CompletableFuture<CreateKeyPairResponse> createKeyPairAsync(String
keyName, String fileName) {
 CreateKeyPairRequest request = CreateKeyPairRequest.builder()
 .keyName(keyName)
 .build();

 CompletableFuture<CreateKeyPairResponse> responseFuture =
getAsyncClient().createKeyPair(request);
 responseFuture.whenComplete((response, exception) -> {
 if (response != null) {
 try {

```

```

 BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName));
 writer.write(response.keyMaterial());
 writer.close();
 } catch (IOException e) {
 throw new RuntimeException("Failed to write key material to
file: " + e.getMessage(), e);
 }
 } else {
 throw new RuntimeException("Failed to create key pair: " +
exception.getMessage(), exception);
 }
 });

 return responseFuture;
}

```

- For API details, see [CreateKeyPair](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSecurityGroup

The following code example shows how to use CreateSecurityGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new security group asynchronously with the specified group name,
description, and VPC ID. It also
 * authorizes inbound traffic on ports 80 and 22 from the specified IP address.
 *
 * @param groupName the name of the security group to create
 * @param groupDesc the description of the security group
 * @param vpcId the ID of the VPC in which to create the security group
 * @param myIpAddress the IP address from which to allow inbound traffic (e.g.,
"192.168.1.1/0" to allow traffic from

```

```

 * any IP address in the 192.168.1.0/24 subnet)
 * @return a CompletableFuture that, when completed, returns the ID of the
 created security group
 * @throws RuntimeException if there was a failure creating the security group
 or authorizing the inbound traffic
 */
 public CompletableFuture<String> createSecurityGroupAsync(String groupName,
String groupDesc, String vpcId, String myIpAddress) {
 CreateSecurityGroupRequest createRequest =
CreateSecurityGroupRequest.builder()
 .groupName(groupName)
 .description(groupDesc)
 .vpcId(vpcId)
 .build();

 return getAsyncClient().createSecurityGroup(createRequest)
 .thenCompose(createResponse -> {
 String groupId = createResponse.groupId();
 IpRange ipRange = IpRange.builder()
 .cidrIp(myIpAddress + "/32")
 .build();

 IpPermission ipPerm = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(80)
 .fromPort(80)
 .ipRanges(ipRange)
 .build();

 IpPermission ipPerm2 = IpPermission.builder()
 .ipProtocol("tcp")
 .toPort(22)
 .fromPort(22)
 .ipRanges(ipRange)
 .build();

 AuthorizeSecurityGroupIngressRequest authRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(groupName)
 .ipPermissions(ipPerm, ipPerm2)
 .build();

 return getAsyncClient().authorizeSecurityGroupIngress(authRequest)
 .thenApply(authResponse -> groupId);
 });
 }

```

```

 })
 .whenComplete((result, exception) -> {
 if (exception != null) {
 if (exception instanceof CompletionException &&
exception.getCause() instanceof Ec2Exception) {
 throw (Ec2Exception) exception.getCause();
 } else {
 throw new RuntimeException("Failed to create security group:
" + exception.getMessage(), exception);
 }
 }
 });
}

```

- For API details, see [CreateSecurityGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteKeyPair

The following code example shows how to use DeleteKeyPair.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a key pair asynchronously.
 *
 * @param keyPair the name of the key pair to delete
 * @return a {@link CompletableFuture} that represents the result of the
asynchronous operation.
 * The {@link CompletableFuture} will complete with a {@link
DeleteKeyPairResponse} object
 * that provides the result of the key pair deletion operation.
 */
public CompletableFuture<DeleteKeyPairResponse> deleteKeysAsync(String keyPair)
{
 DeleteKeyPairRequest request = DeleteKeyPairRequest.builder()

```

```

 .keyName(keyPair)
 .build();

 // Initiate the asynchronous request to delete the key pair.
 CompletableFuture<DeleteKeyPairResponse> response =
getAsyncClient().deleteKeyPair(request);
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to delete key pair: " + keyPair,
ex);
 } else if (resp == null) {
 throw new RuntimeException("No response received for deleting key
pair: " + keyPair);
 }
 });
 }
}

```

- For API details, see [DeleteKeyPair](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteSecurityGroup

The following code example shows how to use DeleteSecurityGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an EC2 security group asynchronously.
 *
 * @param groupId the ID of the security group to delete
 * @return a CompletableFuture that completes when the security group is deleted
 */
public CompletableFuture<Void> deleteEC2SecGroupAsync(String groupId) {
 DeleteSecurityGroupRequest request = DeleteSecurityGroupRequest.builder()
 .groupId(groupId)
 .build();
}

```

```

 CompletableFuture<DeleteSecurityGroupResponse> response =
getAsyncClient().deleteSecurityGroup(request);
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to delete security group with Id
" + groupId, ex);
 } else if (resp == null) {
 throw new RuntimeException("No response received for deleting
security group with Id " + groupId);
 }
 }).thenApply(resp -> null);
 }

```

- For API details, see [DeleteSecurityGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeInstanceTypes

The following code example shows how to use DescribeInstanceTypes.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously retrieves the instance types available in the current AWS
region.
 * <p>
 * This method uses the AWS SDK's asynchronous API to fetch the available
instance types
 * and then processes the response. It logs the memory information, network
information,
 * and instance type for each instance type returned. Additionally, it returns a
 * {@link CompletableFuture} that resolves to the instance type string for the
"t2.2xlarge"
 * instance type, if it is found in the response. If the "t2.2xlarge" instance
type is not

```

```

 * found, an empty string is returned.
 * </p>
 *
 * @return a {@link CompletableFuture} that resolves to the instance type string
 for the
 * "t2.2xlarge" instance type, or an empty string if the instance type is not
 found
 */
 public CompletableFuture<String> getInstanceTypesAsync() {
 DescribeInstanceTypesRequest typesRequest =
DescribeInstanceTypesRequest.builder()
 .maxResults(10)
 .build();

 CompletableFuture<DescribeInstanceTypesResponse> response =
getAsyncClient().describeInstanceTypes(typesRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 List<InstanceTypeInfo> instanceTypes = resp.instanceTypes();
 for (InstanceTypeInfo type : instanceTypes) {
 logger.info("The memory information of this type is " +
type.memoryInfo().sizeInMiB());
 logger.info("Network information is " +
type.networkInfo().toString());
 logger.info("Instance type is " +
type.instanceType().toString());
 }
 } else {
 throw (RuntimeException) ex;
 }
 });

 return response.thenApply(resp -> {
 for (InstanceTypeInfo type : resp.instanceTypes()) {
 String instanceType = type.instanceType().toString();
 if (instanceType.equals("t2.2xlarge")) {
 return instanceType;
 }
 }
 return "";
 });
 }
}

```



- For API details, see [DescribeInstanceTypes](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeInstances

The following code example shows how to use `DescribeInstances`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously describes the state of an EC2 instance.
 * The paginator helps you iterate over multiple pages of results.
 *
 * @param newInstanceId the ID of the EC2 instance to describe
 * @return a {@link CompletableFuture} that, when completed, contains a string
 * describing the state of the EC2 instance
 */
public CompletableFuture<String> describeEC2InstancesAsync(String newInstanceId)
{
 DescribeInstancesRequest request = DescribeInstancesRequest.builder()
 .instanceIds(newInstanceId)
 .build();

 DescribeInstancesPublisher paginator =
getAsyncClient().describeInstancesPaginator(request);
 AtomicReference<String> publicIpAddressRef = new AtomicReference<>();
 return paginator.subscribe(response -> {
 response.reservations().stream()
 .flatMap(reservation -> reservation.instances().stream())
 .filter(instance -> instance.instanceId().equals(newInstanceId))
 .findFirst()
 .ifPresent(instance ->
publicIpAddressRef.set(instance.publicIpAddress()));
 }).thenApply(v -> {
 String publicIpAddress = publicIpAddressRef.get();
 if (publicIpAddress == null) {
```

```

 throw new RuntimeException("Instance with ID " + newInstanceId + "
not found.");
 }
 return publicIpAddress;
}).exceptionally(ex -> {
 logger.info("Failed to describe instances: " + ex.getMessage());
 throw new RuntimeException("Failed to describe instances", ex);
});
}

```

- For API details, see [DescribeInstances](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeKeyPairs

The following code example shows how to use `DescribeKeyPairs`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously describes the key pairs associated with the current AWS
account.
 *
 * @return a {@link CompletableFuture} containing the {@link
DescribeKeyPairsResponse} object, which provides
 * information about the key pairs.
 */
public CompletableFuture<DescribeKeyPairsResponse> describeKeysAsync() {
 CompletableFuture<DescribeKeyPairsResponse> responseFuture =
getAsyncClient().describeKeyPairs();
 responseFuture.whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to describe key pairs: " +
exception.getMessage(), exception);
 }
 });
}

```

```
 return responseFuture;
 }
}
```

- For API details, see [DescribeKeyPairs](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeSecurityGroups

The following code example shows how to use DescribeSecurityGroups.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously describes the security groups for the specified group ID.
 *
 * @param groupName the name of the security group to describe
 * @return a {@link CompletableFuture} that represents the asynchronous
operation
 * of describing the security groups. The future will complete with a
 * {@link DescribeSecurityGroupsResponse} object that contains the
 * security group information.
 */
public CompletableFuture<String> describeSecurityGroupArnByNameAsync(String
groupName) {
 DescribeSecurityGroupsRequest request =
DescribeSecurityGroupsRequest.builder()
 .groupNames(groupName)
 .build();

 DescribeSecurityGroupsPublisher paginator =
getAsyncClient().describeSecurityGroupsPaginator(request);
 AtomicReference<String> groupIdRef = new AtomicReference<>();
 return paginator.subscribe(response -> {
 response.securityGroups().stream()
```

```

 .filter(securityGroup ->
securityGroup.groupName().equals(groupName))
 .findFirst()
 .ifPresent(securityGroup ->
groupIdRef.set(securityGroup.groupId()));
 }).thenApply(v -> {
 String groupId = groupIdRef.get();
 if (groupId == null) {
 throw new RuntimeException("No security group found with the name: "
+ groupName);
 }
 return groupId;
 }).exceptionally(ex -> {
 logger.info("Failed to describe security group: " + ex.getMessage());
 throw new RuntimeException("Failed to describe security group", ex);
 });
}

```

- For API details, see [DescribeSecurityGroups](#) in *AWS SDK for Java 2.x API Reference*.

## DisassociateAddress

The following code example shows how to use `DisassociateAddress`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Disassociates an Elastic IP address from an instance asynchronously.
 *
 * @param associationId The ID of the association you want to disassociate.
 * @return a {@link CompletableFuture} representing the asynchronous operation
of disassociating the address. The
 * {@link CompletableFuture} will complete with a {@link
DisassociateAddressResponse} when the operation is
 * finished.

```

```

 * @throws RuntimeException if the disassociation of the address fails.
 */
 public CompletableFuture<DisassociateAddressResponse>
 disassociateAddressAsync(String associationId) {
 Ec2AsyncClient ec2 = getAsyncClient();
 DisassociateAddressRequest addressRequest =
 DisassociateAddressRequest.builder()
 .associationId(associationId)
 .build();

 // Disassociate the address asynchronously.
 CompletableFuture<DisassociateAddressResponse> response =
 ec2.disassociateAddress(addressRequest);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to disassociate address", ex);
 }
 });

 return response;
 }

```

- For API details, see [DisassociateAddress](#) in *AWS SDK for Java 2.x API Reference*.

## GetPasswordData

The following code example shows how to use GetPasswordData.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ec2.Ec2AsyncClient;
import software.amazon.awssdk.services.ec2.model.*;
import java.util.concurrent.CompletableFuture;

```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetPasswordData {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <instanceId>

 Where:
 instanceId - An instance id value that you can obtain from the
AWS Management Console.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 String instanceId = args[0];
 Ec2AsyncClient ec2AsyncClient = Ec2AsyncClient.builder()
 .region(Region.US_EAST_1)
 .build();

 try {
 CompletableFuture<Void> future = getPasswordDataAsync(ec2AsyncClient,
instanceId);
 future.join();
 } catch (RuntimeException rte) {
 System.err.println("An exception occurred: " + (rte.getCause() != null ?
rte.getCause().getMessage() : rte.getMessage()));
 }
 }

 /**
 * Fetches the password data for the specified EC2 instance asynchronously.
 *
 * @param ec2AsyncClient the EC2 asynchronous client to use for the request

```

```

 * @param instanceId instanceId the ID of the EC2 instance for which you want to
 fetch the password data
 * @return a {@link CompletableFuture} that completes when the password data has
 been fetched
 * @throws RuntimeException if there was a failure in fetching the password data
 */
 public static CompletableFuture<Void> getPasswordDataAsync(Ec2AsyncClient
ec2AsyncClient, String instanceId) {
 GetPasswordDataRequest getPasswordDataRequest =
GetPasswordDataRequest.builder()
 .instanceId(instanceId)
 .build();

 CompletableFuture<GetPasswordDataResponse> response =
ec2AsyncClient.getPasswordData(getPasswordDataRequest);
 response.whenComplete((getPasswordDataResponse, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to get password data for
instance: " + instanceId, ex);
 } else if (getPasswordDataResponse == null ||
getPasswordDataResponse.getPasswordData().isEmpty()) {
 throw new RuntimeException("No password data found for instance: " +
instanceId);
 } else {
 String encryptedPasswordData =
getPasswordDataResponse.getPasswordData();
 System.out.println("Encrypted Password Data: " +
encryptedPasswordData);
 }
 });

 return response.thenApply(resp -> null);
 }
}

```

- For API details, see [GetPasswordData](#) in *AWS SDK for Java 2.x API Reference*.

## ReleaseAddress

The following code example shows how to use ReleaseAddress.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Releases an Elastic IP address asynchronously.
 *
 * @param allocId the allocation ID of the Elastic IP address to be released
 * @return a {@link CompletableFuture} representing the asynchronous operation
 * of releasing the Elastic IP address
 */
public CompletableFuture<ReleaseAddressResponse> releaseEC2AddressAsync(String
allocId) {
 ReleaseAddressRequest request = ReleaseAddressRequest.builder()
 .allocationId(allocId)
 .build();

 CompletableFuture<ReleaseAddressResponse> response =
getAsyncClient().releaseAddress(request);
 response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to release Elastic IP address",
ex);
 }
 });

 return response;
}
```

- For API details, see [ReleaseAddress](#) in *AWS SDK for Java 2.x API Reference*.

## RunInstances

The following code example shows how to use RunInstances.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Runs an EC2 instance asynchronously.
 *
 * @param instanceType The instance type to use for the EC2 instance.
 * @param keyName The name of the key pair to associate with the EC2 instance.
 * @param groupName The name of the security group to associate with the EC2
instance.
 * @param amiId The ID of the Amazon Machine Image (AMI) to use for the EC2
instance.
 * @return A {@link CompletableFuture} that completes with the ID of the started
EC2 instance.
 * @throws RuntimeException If there is an error running the EC2 instance.
 */
public CompletableFuture<String> runInstanceAsync(String instanceType, String
keyName, String groupName, String amiId) {
 RunInstancesRequest runRequest = RunInstancesRequest.builder()
 .instanceType(instanceType)
 .keyName(keyName)
 .securityGroups(groupName)
 .maxCount(1)
 .minCount(1)
 .imageId(amiId)
 .build();

 CompletableFuture<RunInstancesResponse> responseFuture =
getAsyncClient().runInstances(runRequest);
 return responseFuture.thenCompose(response -> {
 String instanceIdVal = response.instances().get(0).instanceId();
 System.out.println("Going to start an EC2 instance and use a waiter to
wait for it to be in running state");
 return getAsyncClient().waiter()
 .waitUntilInstanceExists(r -> r.instanceIds(instanceIdVal))
 .thenCompose(waitResponse -> getAsyncClient().waiter()
 .waitUntilInstanceRunning(r -> r.instanceIds(instanceIdVal)))
 });
}
```

```

 .thenApply(runningResponse -> instanceIdVal));
 }).exceptionally(throwable -> {
 // Handle any exceptions that occurred during the async call
 throw new RuntimeException("Failed to run EC2 instance: " +
 throwable.getMessage(), throwable);
 });
}

```

- For API details, see [RunInstances](#) in *AWS SDK for Java 2.x API Reference*.

## StartInstances

The following code example shows how to use `StartInstances`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Starts an Amazon EC2 instance asynchronously and waits until it is in the
 * "running" state.
 *
 * @param instanceId the ID of the instance to start
 * @return a {@link CompletableFuture} that completes when the instance has been
 * started and is in the "running" state, or exceptionally if an error occurs
 */
public CompletableFuture<Void> startInstanceAsync(String instanceId) {
 StartInstancesRequest startRequest = StartInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 Ec2AsyncWaiter ec2Waiter = Ec2AsyncWaiter.builder()
 .client(getAsyncClient())
 .build();

 DescribeInstancesRequest describeRequest =
 DescribeInstancesRequest.builder()

```

```

 .instanceIds(instanceId)
 .build();

 logger.info("Starting instance " + instanceId + " and waiting for it to
run.");
 CompletableFuture<Void> resultFuture = new CompletableFuture<>();
 return getAsyncClient().startInstances(startRequest)
 .thenCompose(response ->
 ec2Waiter.waitForInstanceRunning(describeRequest)
)
 .thenAccept(waiterResponse -> {
 logger.info("Successfully started instance " + instanceId);
 resultFuture.complete(null);
 })
 .exceptionally(throwable -> {
 resultFuture.completeExceptionally(new RuntimeException("Failed to
start instance: " + throwable.getMessage(), throwable));
 return null;
 });
}

```

- For API details, see [StartInstances](#) in *AWS SDK for Java 2.x API Reference*.

## StopInstances

The following code example shows how to use StopInstances.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Stops the EC2 instance with the specified ID asynchronously and waits for the
instance to stop.
 *
 * @param instanceId the ID of the EC2 instance to stop

```

```

 * @return a {@link CompletableFuture} that completes when the instance has been
 stopped, or exceptionally if an error occurs
 */
 public CompletableFuture<Void> stopInstanceAsync(String instanceId) {
 StopInstancesRequest stopRequest = StopInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 DescribeInstancesRequest describeRequest =
 DescribeInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 Ec2AsyncWaiter ec2Waiter = Ec2AsyncWaiter.builder()
 .client(getAsyncClient())
 .build();

 CompletableFuture<Void> resultFuture = new CompletableFuture<>();
 logger.info("Stopping instance " + instanceId + " and waiting for it to
stop.");
 getAsyncClient().stopInstances(stopRequest)
 .thenCompose(response -> {
 if (response.stoppingInstances().isEmpty()) {
 return CompletableFuture.failedFuture(new RuntimeException("No
instances were stopped. Please check the instance ID: " + instanceId));
 }
 return ec2Waiter.waitUntilInstanceStopped(describeRequest);
 })
 .thenAccept(waiterResponse -> {
 logger.info("Successfully stopped instance " + instanceId);
 resultFuture.complete(null);
 })
 .exceptionally(throwable -> {
 logger.error("Failed to stop instance " + instanceId + ": " +
throwable.getMessage(), throwable);
 resultFuture.completeExceptionally(new RuntimeException("Failed to
stop instance: " + throwable.getMessage(), throwable));
 return null;
 });

 return resultFuture;
 }

```

- For API details, see [StopInstances](#) in *AWS SDK for Java 2.x API Reference*.

## TerminateInstances

The following code example shows how to use `TerminateInstances`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Terminates an EC2 instance asynchronously and waits for it to reach the
 * terminated state.
 *
 * @param instanceId the ID of the EC2 instance to terminate
 * @return a {@link CompletableFuture} that completes when the instance has been
 * terminated
 * @throws RuntimeException if there is no response from the AWS SDK or if there
 * is a failure during the termination process
 */
public CompletableFuture<Object> terminateEC2Async(String instanceId) {
 TerminateInstancesRequest terminateRequest =
 TerminateInstancesRequest.builder()
 .instanceIds(instanceId)
 .build();

 CompletableFuture<TerminateInstancesResponse> responseFuture =
 getAsyncClient().terminateInstances(terminateRequest);
 return responseFuture.thenCompose(terminateResponse -> {
 if (terminateResponse == null) {
 throw new RuntimeException("No response received for terminating
instance " + instanceId);
 }
 System.out.println("Going to terminate an EC2 instance and use a waiter
to wait for it to be in terminated state");
 return getAsyncClient().waiter()
 .waitUntilInstanceTerminated(r -> r.instanceIds(instanceId))
 .thenApply(waiterResponse -> null);
 });
}
```

```
 }).exceptionally(throwable -> {
 // Handle any exceptions that occurred during the async call
 throw new RuntimeException("Failed to terminate EC2 instance: " +
throwable.getMessage(), throwable);
 });
}
```

- For API details, see [TerminateInstances](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build and manage a resilient service

The following code example shows how to create a load-balanced web service that returns book, movie, and song recommendations. The example shows how the service responds to failures, and how to restructure the service for more resilience when failures occur.

- Use an Amazon EC2 Auto Scaling group to create Amazon Elastic Compute Cloud (Amazon EC2) instances based on a launch template and to keep the number of instances in a specified range.
- Handle and distribute HTTP requests with Elastic Load Balancing.
- Monitor the health of instances in an Auto Scaling group and forward requests only to healthy instances.
- Run a Python web server on each EC2 instance to handle HTTP requests. The web server responds with recommendations and health checks.
- Simulate a recommendation service with an Amazon DynamoDB table.
- Control web server response to requests and health checks by updating AWS Systems Manager parameters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run the interactive scenario at a command prompt.

```
public class Main {

 public static final String fileName = "C:\\\\AWS\\resworkflow\\
\\recommendations.json"; // Modify file location.
 public static final String tableName = "doc-example-recommendation-service";
 public static final String startScript = "C:\\\\AWS\\resworkflow\\
\\server_startup_script.sh"; // Modify file location.
 public static final String policyFile = "C:\\\\AWS\\resworkflow\\
\\instance_policy.json"; // Modify file location.
 public static final String ssmJSON = "C:\\\\AWS\\resworkflow\\
\\ssm_only_policy.json"; // Modify file location.
 public static final String failureResponse = "doc-example-resilient-
architecture-failure-response";
 public static final String healthCheck = "doc-example-resilient-architecture-
health-check";
 public static final String templateName = "doc-example-resilience-template";
 public static final String roleName = "doc-example-resilience-role";
 public static final String policyName = "doc-example-resilience-pol";
 public static final String profileName = "doc-example-resilience-prof";

 public static final String badCredsProfileName = "doc-example-resilience-prof-
bc";

 public static final String targetGroupName = "doc-example-resilience-tg";
 public static final String autoScalingGroupName = "doc-example-resilience-
group";
 public static final String lbName = "doc-example-resilience-lb";
 public static final String protocol = "HTTP";
 public static final int port = 80;

 public static final String DASHES = new String(new char[80]).replace("\\0", "-");

 public static void main(String[] args) throws IOException, InterruptedException
 {
 Scanner in = new Scanner(System.in);
 Database database = new Database();
 AutoScaler autoScaler = new AutoScaler();
 LoadBalancer loadBalancer = new LoadBalancer();

 System.out.println(DASHES);
 System.out.println("Welcome to the demonstration of How to Build and Manage
a Resilient Service!");
 System.out.println(DASHES);
 }
}
```

```

 System.out.println(DASHES);
 System.out.println("A - SETUP THE RESOURCES");
 System.out.println("Press Enter when you're ready to start deploying
resources.");
 in.nextLine();
 deploy(loadBalancer);
 System.out.println(DASHES);
 System.out.println(DASHES);
 System.out.println("B - DEMO THE RESILIENCE FUNCTIONALITY");
 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 demo(loadBalancer);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("C - DELETE THE RESOURCES");
 System.out.println("""
 This concludes the demo of how to build and manage a resilient
service.

 To keep things tidy and to avoid unwanted charges on your account,
we can clean up all AWS resources
 that were created for this demo.
 """);

 System.out.println("\n Do you want to delete the resources (y/n)? ");
 String userInput = in.nextLine().trim().toLowerCase(); // Capture user input

 if (userInput.equals("y")) {
 // Delete resources here
 deleteResources(loadBalancer, autoScaler, database);
 System.out.println("Resources deleted.");
 } else {
 System.out.println("""
 Okay, we'll leave the resources intact.
 Don't forget to delete them when you're done with them or you
might incur unexpected charges.
 """);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The example has completed. ");
 System.out.println("\n Thanks for watching!");

```



```

 System.out.println(DASHES);
 }

 // Deletes the AWS resources used in this example.
 private static void deleteResources(LoadBalancer loadBalancer, AutoScaler
autoScaler, Database database)
 throws IOException, InterruptedException {
 loadBalancer.deleteLoadBalancer(lbName);
 System.out.println("*** Wait 30 secs for resource to be deleted");
 TimeUnit.SECONDS.sleep(30);
 loadBalancer.deleteTargetGroup(targetGroupName);
 autoScaler.deleteAutoScalingGroup(autoScalingGroupName);
 autoScaler.deleteRolesPolicies(policyName, roleName, profileName);
 autoScaler.deleteTemplate(templateName);
 database.deleteTable(tableName);
 }

 private static void deploy(LoadBalancer loadBalancer) throws
InterruptedException, IOException {
 Scanner in = new Scanner(System.in);
 System.out.println(
 """
 For this demo, we'll use the AWS SDK for Java (v2) to create
several AWS resources
 to set up a load-balanced web service endpoint and explore
some ways to make it resilient
 against various kinds of failures.

 Some of the resources create by this demo are:
 \t* A DynamoDB table that the web service depends on to
provide book, movie, and song recommendations.
 \t* An EC2 launch template that defines EC2 instances that
each contain a Python web server.
 \t* An EC2 Auto Scaling group that manages EC2 instances
across several Availability Zones.
 \t* An Elastic Load Balancing (ELB) load balancer that
targets the Auto Scaling group to distribute requests.
 """);

 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);

```

```

 System.out.println("Creating and populating a DynamoDB table named " +
tableName);
 Database database = new Database();
 database.createTable(tableName, fileName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an EC2 launch template that runs '{startup_script}' when an
instance starts.
 This script starts a Python web server defined in the `server.py`
script. The web server
 listens to HTTP requests on port 80 and responds to requests to '/'
and to '/healthcheck'.
 For demo purposes, this server is run as the root user. In
production, the best practice is to
 run a web server, such as Apache, with least-privileged credentials.

 The template also defines an IAM policy that each instance uses to
assume a role that grants
 permissions to access the DynamoDB recommendation table and Systems
Manager parameters
 that control the flow of the demo.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 templateCreator.createTemplate(policyFile, policyName, profileName,
startScript, templateName, roleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(
 "Creating an EC2 Auto Scaling group that maintains three EC2
instances, each in a different Availability Zone.");
 System.out.println("*** Wait 30 secs for the VPC to be created");
 TimeUnit.SECONDS.sleep(30);
 AutoScaler autoScaler = new AutoScaler();
 String[] zones = autoScaler.createGroup(3, templateName,
autoScalingGroupName);

 System.out.println("""
 At this point, you have EC2 instances created. Once each instance
starts, it listens for

```

```

 HTTP requests. You can see these instances in the console or
 continue with the demo.
 Press Enter when you're ready to continue.
 "");

 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Creating variables that control the flow of the demo.");
 ParameterHelper paramHelper = new ParameterHelper();
 paramHelper.reset();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an Elastic Load Balancing target group and load balancer.
The target group
 defines how the load balancer connects to instances. The load
balancer provides a
 single endpoint where clients connect and dispatches requests to
instances in the group.
 """);

 String vpcId = autoScaler.getDefaultVPC();
 List<Subnet> subnets = autoScaler.getSubnets(vpcId, zones);
 System.out.println("You have retrieved a list with " + subnets.size() + "
subnets");
 String targetGroupArn = loadBalancer.createTargetGroup(protocol, port,
vpcId, targetGroupName);
 String elbDnsName = loadBalancer.createLoadBalancer(subnets, targetGroupArn,
lbName, port, protocol);
 autoScaler.attachLoadBalancerTargetGroup(autoScalingGroupName,
targetGroupArn);
 System.out.println("Verifying access to the load balancer endpoint...");
 boolean wasSuccessful = loadBalancer.verifyLoadBalancerEndpoint(elbDnsName);
 if (!wasSuccessful) {
 System.out.println("Couldn't connect to the load balancer, verifying
that the port is open...");
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to "http://checkip.amazonaws.com"
 HttpGet httpGet = new HttpGet("http://checkip.amazonaws.com");
 try {

```

```

 // Execute the request and get the response
 HttpResponse response = httpClient.execute(httpGet);

 // Read the response content.
 String ipAddress =
 IOUtils.toString(response.getEntity().getContent(), StandardCharsets.UTF_8).trim();

 // Print the public IP address.
 System.out.println("Public IP Address: " + ipAddress);
 GroupInfo groupInfo = autoScaler.verifyInboundPort(vpcId, port,
 ipAddress);
 if (!groupInfo.isPortOpen()) {
 System.out.println("""
 For this example to work, the default security group for
 your default VPC must
 allow access from this computer. You can either add it
 automatically from this
 example or add it yourself using the AWS Management
 Console.
 """);

 System.out.println(
 "Do you want to add a rule to security group " +
 groupInfo.getGroupName() + " to allow");
 System.out.println("inbound traffic on port " + port + " from
 your computer's IP address (y/n) ");
 String ans = in.nextLine();
 if ("y".equalsIgnoreCase(ans)) {
 autoScaler.openInboundPort(groupInfo.getGroupName(),
 String.valueOf(port), ipAddress);
 System.out.println("Security group rule added.");
 } else {
 System.out.println("No security group rule added.");
 }
 }

 } catch (AutoScalingException e) {
 e.printStackTrace();
 }
 } else if (wasSuccessful) {
 System.out.println("Your load balancer is ready. You can access it by
 browsing to:");
 System.out.println("\t http://" + elbDnsName);
 } else {

```

```

 System.out.println("Couldn't get a successful response from the load
balancer endpoint. Troubleshoot by");
 System.out.println("manually verifying that your VPC and security group
are configured correctly and that");
 System.out.println("you can successfully make a GET request to the load
balancer.");
 }

 System.out.println("Press Enter when you're ready to continue with the
demo.");
 in.nextLine();
}

// A method that controls the demo part of the Java program.
public static void demo(LoadBalancer loadBalancer) throws IOException,
InterruptedException {
 ParameterHelper paramHelper = new ParameterHelper();
 System.out.println("Read the ssm_only_policy.json file");
 String ssmOnlyPolicy = readFileAsString(ssmJSON);

 System.out.println("Resetting parameters to starting values for demo.");
 paramHelper.reset();

 System.out.println(
 """
 This part of the demonstration shows how to toggle
different parts of the system
 to create situations where the web service fails, and shows
how using a resilient
 architecture can keep the web service running in spite of
these failures.

 At the start, the load balancer endpoint returns
recommendations and reports that all targets are healthy.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 """
 The web service running on the EC2 instances gets
recommendations by querying a DynamoDB table.
 The table name is contained in a Systems Manager parameter
named self.param_helper.table.
 """
);
}

```

```

 To simulate a failure of the recommendation service, let's
 set this parameter to name a non-existent table.
 """);
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 System.out.println(
 ""
 \nNow, sending a GET request to the load balancer endpoint
 returns a failure code. But, the service reports as
 healthy to the load balancer because shallow health checks
 don't check for failure of the recommendation service.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 ""
 Instead of failing when the recommendation service fails,
 the web service can return a static response.
 While this is not a perfect solution, it presents the
 customer with a somewhat better experience than failure.
 """);
 paramHelper.put(paramHelper.failureResponse, "static");

 System.out.println("""
 Now, sending a GET request to the load balancer endpoint returns a
 static response.
 The service still reports as healthy because health checks are still
 shallow.
 """);
 demoChoices(loadBalancer);

 System.out.println("Let's reinstate the recommendation service.");
 paramHelper.put(paramHelper.tableName, paramHelper.dyntable);

 System.out.println("""
 Let's also substitute bad credentials for one of the instances in
 the target group so that it can't
 access the DynamoDB recommendation table. We will get an instance id
 value.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 AutoScaler autoScaler = new AutoScaler();

```

```

 // Create a new instance profile based on badCredsProfileName.
 templateCreator.createInstanceProfile(policyFile, policyName,
badCredsProfileName, roleName);
 String badInstanceId = autoScaler.getBadInstance(autoScalingGroupName);
 System.out.println("The bad instance id values used for this demo is " +
badInstanceId);

 String profileAssociationId = autoScaler.getInstanceProfile(badInstanceId);
 System.out.println("The association Id value is " + profileAssociationId);
 System.out.println("Replacing the profile for instance " + badInstanceId
 + " with a profile that contains bad credentials");
 autoScaler.replaceInstanceProfile(badInstanceId, badCredsProfileName,
profileAssociationId);

 System.out.println(
 ""
 Now, sending a GET request to the load balancer endpoint
returns either a recommendation or a static response,
 depending on which instance is selected by the load
balancer.
 "");

 demoChoices(loadBalancer);

 System.out.println("""
 Let's implement a deep health check. For this demo, a deep health
check tests whether
 the web service can access the DynamoDB table that it depends on for
recommendations. Note that
 the deep health check is only for ELB routing and not for Auto
Scaling instance health.
 This kind of deep health check is not recommended for Auto Scaling
instance health, because it
 risks accidental termination of all instances in the Auto Scaling
group when a dependent service fails.
 """);

 System.out.println("""
 By implementing deep health checks, the load balancer can detect
when one of the instances is failing
 and take that instance out of rotation.
 """);

 paramHelper.put(paramHelper.healthCheck, "deep");

```

```
 System.out.println("""
 Now, checking target health indicates that the instance with bad
credentials
 is unhealthy. Note that it might take a minute or two for the load
balancer to detect the unhealthy
 instance. Sending a GET request to the load balancer endpoint always
returns a recommendation, because
 the load balancer takes unhealthy instances out of its rotation.
 """);

 demoChoices(loadBalancer);

 System.out.println(
 """
 Because the instances in this demo are controlled by an auto
scaler, the simplest way to fix an unhealthy
 instance is to terminate it and let the auto scaler start a
new instance to replace it.
 """);
 autoScaler.terminateInstance(badInstanceId);

 System.out.println("""
 Even while the instance is terminating and the new instance is
starting, sending a GET
 request to the web service continues to get a successful
recommendation response because
 the load balancer routes requests to the healthy instances. After
the replacement instance
 starts and reports as healthy, it is included in the load balancing
rotation.

 Note that terminating and replacing an instance typically takes
several minutes, during which time you
 can see the changing health check status until the new instance is
running and healthy.
 """);

 demoChoices(loadBalancer);
 System.out.println(
 "If the recommendation service fails now, deep health checks mean
all instances report as unhealthy.");
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 demoChoices(loadBalancer);
```



```

 paramHelper.reset();
 }

 public static void demoChoices(LoadBalancer loadBalancer) throws IOException,
 InterruptedException {
 String[] actions = {
 "Send a GET request to the load balancer endpoint.",
 "Check the health of load balancer targets.",
 "Go to the next part of the demo."
 };
 Scanner scanner = new Scanner(System.in);

 while (true) {
 System.out.println("-".repeat(88));
 System.out.println("See the current state of the service by selecting
one of the following choices:");
 for (int i = 0; i < actions.length; i++) {
 System.out.println(i + ": " + actions[i]);
 }

 try {
 System.out.print("\nWhich action would you like to take? ");
 int choice = scanner.nextInt();
 System.out.println("-".repeat(88));

 switch (choice) {
 case 0 -> {
 System.out.println("Request:\n");
 System.out.println("GET http://" +
loadBalancer.getEndpoint(lbName));
 CloseableHttpClient httpClient =
HttpClientBuilder.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" +
loadBalancer.getEndpoint(lbName));

 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);

 // Display the JSON response
 BufferedReader reader = new BufferedReader(

```

```

 new
 InputStreamReader(response.getEntity().getContent()));
 StringBuilder jsonResponse = new StringBuilder();
 String line;
 while ((line = reader.readLine()) != null) {
 jsonResponse.append(line);
 }
 reader.close();

 // Print the formatted JSON response.
 System.out.println("Full Response:\n");
 System.out.println(jsonResponse.toString());

 // Close the HTTP client.
 httpClient.close();
 }
 case 1 -> {
 System.out.println("\nChecking the health of load balancer
targets:\n");
 List<TargetHealthDescription> health =
loadBalancer.checkTargetHealth(targetGroupName);
 for (TargetHealthDescription target : health) {
 System.out.printf("\tTarget %s on port %d is %s\n",
target.target().id(),
 target.target().port(),
target.targetHealth().stateAsString());
 }
 System.out.println("""
 Note that it can take a minute or two for the health
check to update

 after changes are made.
 """);
 }
 case 2 -> {
 System.out.println("\nOkay, let's move on.");
 System.out.println("-".repeat(88));
 return; // Exit the method when choice is 2
 }
 default -> System.out.println("You must choose a value between
0-2. Please select again.");
}

} catch (java.util.InputMismatchException e) {

```

```

 System.out.println("Invalid input. Please select again.");
 scanner.nextLine(); // Clear the input buffer.
 }
}

public static String readFileAsString(String filePath) throws IOException {
 byte[] bytes = Files.readAllBytes(Paths.get(filePath));
 return new String(bytes);
}
}

```

Create a class that wraps Auto Scaling and Amazon EC2 actions.

```

public class AutoScaler {

 private static Ec2Client ec2Client;
 private static AutoScalingClient autoScalingClient;
 private static IamClient iamClient;

 private static SsmClient ssmClient;

 private IamClient getIAMClient() {
 if (iamClient == null) {
 iamClient = IamClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return iamClient;
 }

 private SsmClient getSSMClient() {
 if (ssmClient == null) {
 ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ssmClient;
 }

 private Ec2Client getEc2Client() {
 if (ec2Client == null) {

```

```

 ec2Client = Ec2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ec2Client;
}

private AutoScalingClient getAutoScalingClient() {
 if (autoScalingClient == null) {
 autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return autoScalingClient;
}

/**
 * Terminates instances in an EC2 Auto Scaling group. After an instance is
 * terminated, it can no longer be accessed.
 */
public void terminateInstance(String instanceId) {
 TerminateInstanceInAutoScalingGroupRequest terminateInstanceRequest =
 TerminateInstanceInAutoScalingGroupRequest
 .builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 getAutoScalingClient().terminateInstanceInAutoScalingGroup(terminateInstanceRequest);
 System.out.format("Terminated instance %s.", instanceId);
}

/**
 * Replaces the profile associated with a running instance. After the profile is
 * replaced, the instance is rebooted to ensure that it uses the new profile.
 * When
 * the instance is ready, Systems Manager is used to restart the Python web
 * server.
 */
public void replaceInstanceProfile(String instanceId, String
newInstanceProfileName, String profileAssociationId)
 throws InterruptedException {
 // Create an IAM instance profile specification.

```

```

 software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
iamInstanceProfile =
software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
 .builder()
 .name(newInstanceProfileName) // Make sure 'newInstanceProfileName'
is a valid IAM Instance Profile
 // name.
 .build();

// Replace the IAM instance profile association for the EC2 instance.
ReplaceIamInstanceProfileAssociationRequest replaceRequest =
ReplaceIamInstanceProfileAssociationRequest
 .builder()
 .iamInstanceProfile(iamInstanceProfile)
 .associationId(profileAssociationId) // Make sure
'profileAssociationId' is a valid association ID.
 .build();

try {
 getEc2Client().replaceIamInstanceProfileAssociation(replaceRequest);
 // Handle the response as needed.
} catch (Ec2Exception e) {
 // Handle exceptions, log, or report the error.
 System.err.println("Error: " + e.getMessage());
}

System.out.format("Replaced instance profile for association %s with profile
%s.", profileAssociationId,
 newInstanceProfileName);
TimeUnit.SECONDS.sleep(15);
boolean instReady = false;
int tries = 0;

// Reboot after 60 seconds
while (!instReady) {
 if (tries % 6 == 0) {
 getEc2Client().rebootInstances(RebootInstancesRequest.builder()
 .instanceIds(instanceId)
 .build());
 System.out.println("Rebooting instance " + instanceId + " and
waiting for it to be ready.");
 }
 tries++;
 try {
 TimeUnit.SECONDS.sleep(10);
 }
}

```

```

 } catch (InterruptedException e) {
 e.printStackTrace();
 }

 DescribeInstanceInformationResponse informationResponse =
getSSMClient().describeInstanceInformation();
 List<InstanceInformation> instanceInformationList =
informationResponse.getInstanceInformationList();
 for (InstanceInformation info : instanceInformationList) {
 if (info.getInstanceId().equals(instanceId)) {
 instReady = true;
 break;
 }
 }
}

SendCommandRequest sendCommandRequest = SendCommandRequest.builder()
 .instanceIds(instanceId)
 .documentName("AWS-RunShellScript")
 .parameters(Collections.singletonMap("commands",
 Collections.singletonList("cd / && sudo python3 server.py
80")))
 .build();

getSSMClient().sendCommand(sendCommandRequest);
System.out.println("Restarted the Python web server on instance " +
instanceId + ".");
}

public void openInboundPort(String secGroupId, String port, String ipAddress) {
 AuthorizeSecurityGroupIngressRequest ingressRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(secGroupId)
 .cidrIp(ipAddress)
 .fromPort(Integer.parseInt(port))
 .build();

 getEc2Client().authorizeSecurityGroupIngress(ingressRequest);
 System.out.format("Authorized ingress to %s on port %s from %s.",
secGroupId, port, ipAddress);
}

/**
 * Detaches a role from an instance profile, detaches policies from the role,

```

```

 * and deletes all the resources.
 */
 public void deleteInstanceProfile(String roleName, String profileName) {
 try {
 software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
getInstanceProfileRequest =
software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
 .builder()
 .instanceProfileName(profileName)
 .build();

 GetInstanceProfileResponse response =
getIAMClient().getInstanceProfile(getInstanceProfileRequest);
 String name = response.instanceProfile().instanceProfileName();
 System.out.println(name);

 RemoveRoleFromInstanceProfileRequest profileRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .roleName(roleName)
 .build();

 getIAMClient().removeRoleFromInstanceProfile(profileRequest);
 DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .build();

 getIAMClient().deleteInstanceProfile(deleteInstanceProfileRequest);
 System.out.println("Deleted instance profile " + profileName);

 DeleteRoleRequest deleteRoleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 // List attached role policies.
 ListAttachedRolePoliciesResponse rolesResponse = getIAMClient()
 .listAttachedRolePolicies(role -> role.roleName(roleName));
 List<AttachedPolicy> attachedPolicies =
rolesResponse.attachedPolicies();
 for (AttachedPolicy attachedPolicy : attachedPolicies) {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(attachedPolicy.policyArn())

```

```

 .build();

 getIAMClient().detachRolePolicy(request);
 System.out.println("Detached and deleted policy " +
attachedPolicy.policyName());
 }

 getIAMClient().deleteRole(deleteRoleRequest);
 System.out.println("Instance profile and role deleted.");

 } catch (IamException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public void deleteTemplate(String templateName) {
 getEc2Client().deleteLaunchTemplate(name ->
name.launchTemplateName(templateName));
 System.out.format(templateName + " was deleted.");
}

public void deleteAutoScaleGroup(String groupName) {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

 getAutoScalingClient().deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println(groupName + " was deleted.");
}

/*
 * Verify the default security group of the specified VPC allows ingress from
 * this
 * computer. This can be done by allowing ingress from this computer's IP
 * address. In some situations, such as connecting from a corporate network, you
 * must instead specify a prefix list ID. You can also temporarily open the port
 * to
 * any IP address while running this example. If you do, be sure to remove
 * public
 * access when you're done.

```



```

 *
 */
 public GroupInfo verifyInboundPort(String VPC, int port, String ipAddress) {
 boolean portIsOpen = false;
 GroupInfo groupInfo = new GroupInfo();
 try {
 Filter filter = Filter.builder()
 .name("group-name")
 .values("default")
 .build();

 Filter filter1 = Filter.builder()
 .name("vpc-id")
 .values(VPC)
 .build();

 DescribeSecurityGroupsRequest securityGroupsRequest =
 DescribeSecurityGroupsRequest.builder()
 .filters(filter, filter1)
 .build();

 DescribeSecurityGroupsResponse securityGroupsResponse = getEc2Client()
 .describeSecurityGroups(securityGroupsRequest);
 String securityGroup =
 securityGroupsResponse.securityGroups().get(0).groupName();
 groupInfo.setGroupName(securityGroup);

 for (SecurityGroup secGroup : securityGroupsResponse.securityGroups()) {
 System.out.println("Found security group: " + secGroup.groupId());

 for (IpPermission ipPermission : secGroup.ipPermissions()) {
 if (ipPermission.fromPort() == port) {
 System.out.println("Found inbound rule: " + ipPermission);
 for (IpRange ipRange : ipPermission.ipRanges()) {
 String cidrIp = ipRange.cidrIp();
 if (cidrIp.startsWith(ipAddress) ||
 cidrIp.equals("0.0.0.0/0")) {
 System.out.println(cidrIp + " is applicable");
 portIsOpen = true;
 }
 }
 }

 if (!ipPermission.prefixListIds().isEmpty()) {
 System.out.println("Prefix lList is applicable");
 }
 }
 }
 }
 }

```

```

 portIsOpen = true;
 }

 if (!portIsOpen) {
 System.out
 .println("The inbound rule does not appear to be
open to either this computer's IP,"
 + " all IP addresses (0.0.0.0/0), or to
a prefix list ID.");
 } else {
 break;
 }
}

}

} catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
}

groupInfo.setPortOpen(portIsOpen);
return groupInfo;
}

/*
 * Attaches an Elastic Load Balancing (ELB) target group to this EC2 Auto
 * Scaling group.
 * The target group specifies how the load balancer forward requests to the
 * instances
 * in the group.
 */
public void attachLoadBalancerTargetGroup(String asGroupName, String
targetGroupARN) {
 try {
 AttachLoadBalancerTargetGroupsRequest targetGroupsRequest =
AttachLoadBalancerTargetGroupsRequest.builder()
 .autoScalingGroupName(asGroupName)
 .targetGroupARNs(targetGroupARN)
 .build();

getAutoScalingClient().attachLoadBalancerTargetGroups(targetGroupsRequest);
 System.out.println("Attached load balancer to " + asGroupName);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Creates an EC2 Auto Scaling group with the specified size.
public String[] createGroup(int groupSize, String templateName, String
autoScalingGroupName) {

 // Get availability zones.
 software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
zonesRequest =
software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
 .builder()
 .build();

 DescribeAvailabilityZonesResponse zonesResponse =
getEc2Client().describeAvailabilityZones(zonesRequest);
 List<String> availabilityZoneNames =
zonesResponse.availabilityZones().stream()

.map(software.amazon.awssdk.services.ec2.model.AvailabilityZone::zoneName)
 .collect(Collectors.toList());

 String availabilityZones = String.join(",", availabilityZoneNames);
 LaunchTemplateSpecification specification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(templateName)
 .version("$Default")
 .build();

 String[] zones = availabilityZones.split(",");
 CreateAutoScalingGroupRequest groupRequest =
CreateAutoScalingGroupRequest.builder()
 .launchTemplate(specification)
 .availabilityZones(zones)
 .maxSize(groupSize)
 .minSize(groupSize)
 .autoScalingGroupName(autoScalingGroupName)
 .build();

 try {
 getAutoScalingClient().createAutoScalingGroup(groupRequest);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Created an EC2 Auto Scaling group named " +
autoScalingGroupName);
 return zones;
 }

 public String getDefaultVPC() {
 // Define the filter.
 Filter defaultFilter = Filter.builder()
 .name("is-default")
 .values("true")
 .build();

 software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest request =
software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest
 .builder()
 .filters(defaultFilter)
 .build();

 DescribeVpcsResponse response = getEc2Client().describeVpcs(request);
 return response.vpcs().get(0).vpcId();
 }

 // Gets the default subnets in a VPC for a specified list of Availability Zones.
 public List<Subnet> getSubnets(String vpcId, String[] availabilityZones) {
 List<Subnet> subnets = null;
 Filter vpcFilter = Filter.builder()
 .name("vpc-id")
 .values(vpcId)
 .build();

 Filter azFilter = Filter.builder()
 .name("availability-zone")
 .values(availabilityZones)
 .build();

 Filter defaultForAZ = Filter.builder()
 .name("default-for-az")
 .values("true")
 .build();
 }

```

```

 DescribeSubnetsRequest request = DescribeSubnetsRequest.builder()
 .filters(vpcFilter, azFilter, defaultForAZ)
 .build();

 DescribeSubnetsResponse response = getEc2Client().describeSubnets(request);
 subnets = response.subnets();
 return subnets;
 }

 // Gets data about the instances in the EC2 Auto Scaling group.
 public String getBadInstance(String groupName) {
 DescribeAutoScalingGroupsRequest request =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response =
getAutoScalingClient().describeAutoScalingGroups(request);
 AutoScalingGroup autoScalingGroup = response.autoScalingGroups().get(0);
 List<String> instanceIds = autoScalingGroup.instances().stream()
 .map(instance -> instance.instanceId())
 .collect(Collectors.toList());

 String[] instanceIdArray = instanceIds.toArray(new String[0]);
 for (String instanceId : instanceIdArray) {
 System.out.println("Instance ID: " + instanceId);
 return instanceId;
 }
 return "";
 }

 // Gets data about the profile associated with an instance.
 public String getInstanceProfile(String instanceId) {
 Filter filter = Filter.builder()
 .name("instance-id")
 .values(instanceId)
 .build();

 DescribeIamInstanceProfileAssociationsRequest associationsRequest =
DescribeIamInstanceProfileAssociationsRequest
 .builder()
 .filters(filter)
 .build();
 }

```

```

 DescribeIamInstanceProfileAssociationsResponse response = getEc2Client()
 .describeIamInstanceProfileAssociations(associationsRequest);
 return response.iamInstanceProfileAssociations().get(0).associationId();
 }

 public void deleteRolesPolicies(String policyName, String roleName, String
InstanceProfile) {
 ListPoliciesRequest listPoliciesRequest =
ListPoliciesRequest.builder().build();
 ListPoliciesResponse listPoliciesResponse =
getIAMClient().listPolicies(listPoliciesRequest);
 for (Policy policy : listPoliciesResponse.policies()) {
 if (policy.policyName().equals(policyName)) {
 // List the entities (users, groups, roles) that are attached to the
policy.

software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
listEntitiesRequest =
software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
 .builder()
 .policyArn(policy.arn())
 .build();

 ListEntitiesForPolicyResponse listEntitiesResponse = iamClient
 .listEntitiesForPolicy(listEntitiesRequest);
 if (!listEntitiesResponse.policyGroups().isEmpty() || !
listEntitiesResponse.policyUsers().isEmpty()
 || !listEntitiesResponse.policyRoles().isEmpty()) {
 // Detach the policy from any entities it is attached to.
 DetachRolePolicyRequest detachPolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy.arn())
 .roleName(roleName) // Specify the name of the IAM role
 .build();

 getIAMClient().detachRolePolicy(detachPolicyRequest);
 System.out.println("Policy detached from entities.");
 }

 // Now, you can delete the policy.
 DeletePolicyRequest deletePolicyRequest =
DeletePolicyRequest.builder()
 .policyArn(policy.arn())
 .build();

```

```

 getIAMClient().deletePolicy(deletePolicyRequest);
 System.out.println("Policy deleted successfully.");
 break;
 }
}

// List the roles associated with the instance profile
ListInstanceProfilesForRoleRequest listRolesRequest =
ListInstanceProfilesForRoleRequest.builder()
 .roleName(roleName)
 .build();

// Detach the roles from the instance profile
ListInstanceProfilesForRoleResponse listRolesResponse =
iamClient.listInstanceProfilesForRole(listRolesRequest);
for (software.amazon.awssdk.services.iam.model.InstanceProfile profile :
listRolesResponse.instanceProfiles()) {
 RemoveRoleFromInstanceProfileRequest removeRoleRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .roleName(roleName) // Remove the extra dot here
 .build();

 getIAMClient().removeRoleFromInstanceProfile(removeRoleRequest);
 System.out.println("Role " + roleName + " removed from instance profile
" + InstanceProfile);
}

// Delete the instance profile after removing all roles
DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .build();

getIAMClient().deleteInstanceProfile(r ->
r.instanceProfileName(InstanceProfile));
System.out.println(InstanceProfile + " Deleted");
System.out.println("All roles and policies are deleted.");
}
}

```

## Create a class that wraps Elastic Load Balancing actions.

```
public class LoadBalancer {
 public ElasticLoadBalancingV2Client elasticLoadBalancingV2Client;

 public ElasticLoadBalancingV2Client getLoadBalancerClient() {
 if (elasticLoadBalancingV2Client == null) {
 elasticLoadBalancingV2Client = ElasticLoadBalancingV2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }

 return elasticLoadBalancingV2Client;
 }

 // Checks the health of the instances in the target group.
 public List<TargetHealthDescription> checkTargetHealth(String targetGroupName) {
 DescribeTargetGroupsRequest targetGroupsRequest =
 DescribeTargetGroupsRequest.builder()
 .names(targetGroupName)
 .build();

 DescribeTargetGroupsResponse tgResponse =
 getLoadBalancerClient().describeTargetGroups(targetGroupsRequest);

 DescribeTargetHealthRequest healthRequest =
 DescribeTargetHealthRequest.builder()
 .targetGroupArn(tgResponse.targetGroups().get(0).targetGroupArn())
 .build();

 DescribeTargetHealthResponse healthResponse =
 getLoadBalancerClient().describeTargetHealth(healthRequest);
 return healthResponse.targetHealthDescriptions();
 }

 // Gets the HTTP endpoint of the load balancer.
 public String getEndpoint(String lbName) {
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 return res.loadBalancers().get(0).dnsName();
 }

 // Deletes a load balancer.
 public void deleteLoadBalancer(String lbName) {
```



```

 try {
 // Use a waiter to delete the Load Balancer.
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(res.loadBalancers().get(0).loadBalancerArn())
 .build();

 getLoadBalancerClient().deleteLoadBalancer(
 builder ->
builder.loadBalancerArn(res.loadBalancers().get(0).loadBalancerArn()));
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancersDeleted(request);
 waiterResponse.matched().response().ifPresent(System.out::println);

 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(lbName + " was deleted.");
 }

 // Deletes the target group.
 public void deleteTargetGroup(String targetGroupName) {
 try {
 DescribeTargetGroupsResponse res = getLoadBalancerClient()
 .describeTargetGroups(describe ->
describe.names(targetGroupName));
 getLoadBalancerClient()
 .deleteTargetGroup(builder ->
builder.targetGroupArn(res.targetGroups().get(0).targetGroupArn()));
 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(targetGroupName + " was deleted.");
 }

 // Verify this computer can successfully send a GET request to the load balancer
 // endpoint.
 public boolean verifyLoadBalancerEndpoint(String elbDnsName) throws IOException,
 InterruptedException {

```

```

 boolean success = false;
 int retries = 3;
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" + elbDnsName);
 try {
 while ((!success) && (retries > 0)) {
 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);
 if (statusCode == 200) {
 success = true;
 } else {
 retries--;
 System.out.println("Got connection error from load balancer endpoint, retrying...");
 TimeUnit.SECONDS.sleep(15);
 }
 }

 } catch (org.apache.http.conn.HttpHostConnectException e) {
 System.out.println(e.getMessage());
 }

 System.out.println("Status.." + success);
 return success;
 }

 /**
 * Creates an Elastic Load Balancing target group. The target group specifies
 * how
 * the load balancer forward requests to instances in the group and how instance
 * health is checked.
 */
 public String createTargetGroup(String protocol, int port, String vpcId, String
targetGroupName) {
 CreateTargetGroupRequest targetGroupRequest =
CreateTargetGroupRequest.builder()
 .healthCheckPath("/healthcheck")
 .healthCheckTimeoutSeconds(5)
 .port(port)
 .vpcId(vpcId)

```

```

 .name(targetGroupName)
 .protocol(protocol)
 .build();

 CreateTargetGroupResponse targetGroupResponse =
getLoadBalancerClient().createTargetGroup(targetGroupRequest);
 String targetGroupArn =
targetGroupResponse.targetGroups().get(0).targetGroupArn();
 String targetGroup =
targetGroupResponse.targetGroups().get(0).targetGroupName();
 System.out.println("The " + targetGroup + " was created with ARN" +
targetGroupArn);
 return targetGroupArn;
}

/*
 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */
public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
String lbName, int port,
 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()

```

```

 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("**** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.
 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

.loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}
}

```

Create a class that uses DynamoDB to simulate a recommendation service.

```
public class Database {

 private static DynamoDbClient dynamoDbClient;

 public static DynamoDbClient getDynamoDbClient() {
 if (dynamoDbClient == null) {
 dynamoDbClient = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return dynamoDbClient;
 }

 // Checks to see if the Amazon DynamoDB table exists.
 private boolean doesTableExist(String tableName) {
 try {
 // Describe the table and catch any exceptions.
 DescribeTableRequest describeTableRequest =
 DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 getDynamoDbClient().describeTable(describeTableRequest);
 System.out.println("Table '" + tableName + "' exists.");
 return true;

 } catch (ResourceNotFoundException e) {
 System.out.println("Table '" + tableName + "' does not exist.");
 } catch (DynamoDbException e) {
 System.err.println("Error checking table existence: " + e.getMessage());
 }
 return false;
 }

 /**
 * Creates a DynamoDB table to use a recommendation service. The table has a
 * hash key named 'MediaType' that defines the type of media recommended, such
 * as
 * Book or Movie, and a range key named 'ItemId' that, combined with the

```

```

 * MediaType,
 * forms a unique identifier for the recommended item.
 */
public void createTable(String tableName, String fileName) throws IOException {
 // First check to see if the table exists.
 boolean doesExist = doesTableExist(tableName);
 if (!doesExist) {
 DynamoDbWaiter dbWaiter = getDynamoDbClient().waiter();
 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("MediaType")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("ItemId")
 .attributeType(ScalarAttributeType.N)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("MediaType")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("ItemId")
 .keyType(KeyType.RANGE)
 .build())
 .provisionedThroughput(
 ProvisionedThroughput.builder()
 .readCapacityUnits(5L)
 .writeCapacityUnits(5L)
 .build())
 .build();

 getDynamoDbClient().createTable(createTableRequest);
 System.out.println("Creating table " + tableName + "...");

 // Wait until the Amazon DynamoDB table is created.
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();
 }
}

```

```

 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Table " + tableName + " created.");

 // Add records to the table.
 populateTable(fileName, tableName);
 }
}

public void deleteTable(String tableName) {
 getDynamoDbClient().deleteTable(table -> table.tableName(tableName));
 System.out.println("Table " + tableName + " deleted.");
}

// Populates the table with data located in a JSON file using the DynamoDB
// enhanced client.
public void populateTable(String fileName, String tableName) throws IOException
{
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(getDynamoDbClient())
 .build();
 ObjectMapper objectMapper = new ObjectMapper();
 File jsonFile = new File(fileName);
 JsonNode rootNode = objectMapper.readTree(jsonFile);

 DynamoDbTable<Recommendation> mappedTable = enhancedClient.table(tableName,
 TableSchema.fromBean(Recommendation.class));
 for (JsonNode currentNode : rootNode) {
 String mediaType = currentNode.path("MediaType").path("S").asText();
 int itemId = currentNode.path("ItemId").path("N").asInt();
 String title = currentNode.path("Title").path("S").asText();
 String creator = currentNode.path("Creator").path("S").asText();

 // Create a Recommendation object and set its properties.
 Recommendation rec = new Recommendation();
 rec.setMediaType(mediaType);
 rec.setItemId(itemId);
 rec.setTitle(title);
 rec.setCreator(creator);

 // Put the item into the DynamoDB table.
 mappedTable.putItem(rec); // Add the Recommendation to the list.
 }
}

```

```
 System.out.println("Added all records to the " + tableName);
 }
}
```

Create a class that wraps Systems Manager actions.

```
public class ParameterHelper {

 String tableName = "doc-example-resilient-architecture-table";
 String dyntable = "doc-example-recommendation-service";
 String failureResponse = "doc-example-resilient-architecture-failure-response";
 String healthCheck = "doc-example-resilient-architecture-health-check";

 public void reset() {
 put(dyntable, tableName);
 put(failureResponse, "none");
 put(healthCheck, "shallow");
 }

 public void put(String name, String value) {
 SsmClient ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();

 PutParameterRequest parameterRequest = PutParameterRequest.builder()
 .name(name)
 .value(value)
 .overwrite(true)
 .type("String")
 .build();

 ssmClient.putParameter(parameterRequest);
 System.out.printf("Setting demo parameter %s to '%s'.", name, value);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AttachLoadBalancerTargetGroups](#)
  - [CreateAutoScalingGroup](#)
  - [CreateInstanceProfile](#)



- [CreateLaunchTemplate](#)
- [CreateListener](#)
- [CreateLoadBalancer](#)
- [CreateTargetGroup](#)
- [DeleteAutoScalingGroup](#)
- [DeleteInstanceProfile](#)
- [DeleteLaunchTemplate](#)
- [DeleteLoadBalancer](#)
- [DeleteTargetGroup](#)
- [DescribeAutoScalingGroups](#)
- [DescribeAvailabilityZones](#)
- [DescribeIamInstanceProfileAssociations](#)
- [DescribeInstances](#)
- [DescribeLoadBalancers](#)
- [DescribeSubnets](#)
- [DescribeTargetGroups](#)
- [DescribeTargetHealth](#)
- [DescribeVpcs](#)
- [RebootInstances](#)
- [ReplacesIamInstanceProfileAssociation](#)
- [TerminateInstanceInAutoScalingGroup](#)
- [UpdateAutoScalingGroup](#)

## Amazon ECR examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon ECR.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Amazon ECR

The following code example shows how to get started using Amazon ECR.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecr.EcrClient;
import software.amazon.awssdk.services.ecr.model.EcrException;
import software.amazon.awssdk.services.ecr.model.ListImagesRequest;
import software.amazon.awssdk.services.ecr.paginators.ListImagesIterable;

public class HelloECR {

 public static void main(String[] args) {
 final String usage = ""
 Usage: <repositoryName>

 Where:
 repositoryName - The name of the Amazon ECR repository.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 }
 }
}
```

```
 System.exit(1);
 }

 String repoName = args[0];
 EcrClient ecrClient = EcrClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listImageTags(ecrClient, repoName);
}

public static void listImageTags(EcrClient ecrClient, String repoName){
 ListImagesRequest listImagesPaginator = ListImagesRequest.builder()
 .repositoryName(repoName)
 .build();

 ListImagesIterable imagesIterable =
 ecrClient.listImagesPaginator(listImagesPaginator);
 imagesIterable.stream()
 .flatMap(r -> r.imageIds().stream())
 .forEach(image -> System.out.println("The docker image tag is: "
+image.imageTag()));
}
}
```

- For API details, see [listImages](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an Amazon ECR repository.
- Set repository policies.
- Retrieve repository URIs.
- Get Amazon ECR authorization tokens.
- Set lifecycle policies for Amazon ECR repositories.
- Push a Docker image to an Amazon ECR repository.
- Verify the existence of an image in an Amazon ECR repository.

- List Amazon ECR repositories for your account and get details about them.
- Delete Amazon ECR repositories.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Amazon ECR features.

```
import software.amazon.awssdk.services.ecr.model.EcrException;
import software.amazon.awssdk.services.ecr.model.RepositoryPolicyNotFoundException;

import java.util.Scanner;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java code example requires an IAM Role that has permissions to interact with
 * the Amazon ECR service.
 *
 * To create an IAM role, see:
 *
 * https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_create.html
 *
 * This Java scenario example requires a local docker image named echo-text. Without
 * a local image,
 * this Java program will not successfully run. For more information including how
 * to create the local
 * image, see:
 *
 * /scenarios/basics/ecr/README
 */
```

```

*/
public class ECRScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 public static void main(String[] args) {
 final String usage = ""
 Usage: <iamRoleARN> <accountId>

 Where:
 iamRoleARN - The IAM role ARN that has the necessary permissions to
access and manage the Amazon ECR repository.
 accountId - Your AWS account number.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 return;
 }

 ECRActions ecrActions = new ECRActions();
 String iamRole = args[0];
 String accountId = args[1];
 String localImageName;

 Scanner scanner = new Scanner(System.in);
 System.out.println("")
 The Amazon Elastic Container Registry (ECR) is a fully-managed Docker
container registry
 service provided by AWS. It allows developers and organizations to
securely
 store, manage, and deploy Docker container images.
 ECR provides a simple and scalable way to manage container images
throughout their lifecycle,
 from building and testing to production deployment.\s

 The `EcrAsyncClient` interface in the AWS SDK for Java 2.x provides a
set of methods to
 programmatically interact with the Amazon ECR service. This allows
developers to
 automate the storage, retrieval, and management of container images as
part of their application
 deployment pipelines. With ECR, teams can focus on building and
deploying their
 applications without having to worry about the underlying
infrastructure required to

```

host and manage a container registry.

This scenario walks you through how to perform key operations for this service.

Let's get started...

You have two choices:

- 1 - Run the entire program.
- 2 - Delete an existing Amazon ECR repository named echo-text (created from a previous execution of this program that did not complete).

```

 """);

while (true) {
 String input = scanner.nextLine();
 if (input.trim().equalsIgnoreCase("1")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else if (input.trim().equalsIgnoreCase("2")) {
 String repoName = "echo-text";
 ecrActions.deleteECRRepository(repoName);
 return;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
}

```

```

waitForInputToContinue(scanner);
System.out.println(DASHES);

```

```

System.out.println("""
 1. Create an ECR repository.

```

The first task is to ensure we have a local Docker image named echo-text.

If this image exists, then an Amazon ECR repository is created.

An ECR repository is a private Docker container repository provided by Amazon Web Services (AWS). It is a managed service that makes it easy to store, manage, and deploy Docker container images.\s""");

```

// Ensure that a local docker image named echo-text exists.
boolean doesExist = ecrActions.isEchoTextImagePresent();
String repoName;
if (!doesExist){
 System.out.println("The local image named echo-text does not exist");
 return;
} else {
 localImageName = "echo-text";
 repoName = "echo-text";
}

try {
 String repoArn = ecrActions.createECRRepository(repoName);
 System.out.println("The ARN of the ECR repository is " + repoArn);

} catch (IllegalArgumentException e) {
 System.err.println("Invalid repository name: " + e.getMessage());
 return;
} catch (RuntimeException e) {
 System.err.println("An error occurred while creating the ECR repository:
" + e.getMessage());
 e.printStackTrace();
 return;
}
waitForInputToContinue(scanner);

System.out.println(DASHES);
System.out.println("")
2. Set an ECR repository policy.

Setting an ECR repository policy using the `setRepositoryPolicy` function is
crucial for maintaining
the security and integrity of your container images. The repository policy
allows you to
define specific rules and restrictions for accessing and managing the images
stored within your ECR
repository.
""");
waitForInputToContinue(scanner);
try {
 ecrActions.setRepoPolicy(repoName, iamRole);

} catch (RepositoryPolicyNotFoundException e) {
 System.err.println("Invalid repository name: " + e.getMessage());

```

```

 return;
 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("An error occurred while creating the ECR repository:
" + e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("
3. Display ECR repository policy.

Now we will retrieve the ECR policy to ensure it was successfully set.
");
 waitForInputToContinue(scanner);
 try {
 String policyText = ecrActions.getRepoPolicy(repoName);
 System.out.println("Policy Text:");
 System.out.println(policyText);

 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("An error occurred while creating the ECR repository:
" + e.getMessage());
 return;
 }

 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("
4. Retrieve an ECR authorization token.

```

You need an authorization token to securely access and interact with the Amazon ECR registry.

The `getAuthorizationToken` method of the `EcrAsyncClient` is responsible for securely accessing and interacting with an Amazon ECR repository. This operation is responsible for obtaining a



valid authorization token, which is required to authenticate your requests to the ECR service.

Without a valid authorization token, you would not be able to perform any operations on the ECR repository, such as pushing, pulling, or managing your Docker images.

```

 "");
 waitForInputToContinue(scanner);
 try {
 ecrActions.getAuthToken();

 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("An error occurred while retrieving the authorization
token: " + e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("")
 5. Get the ECR Repository URI.

```

The URI of an Amazon ECR repository is important. When you want to deploy a container image to a container orchestration platform like Amazon Elastic Kubernetes Service (EKS) or Amazon Elastic Container Service (ECS), you need to specify the full image URI, which includes the ECR repository URI. This allows the container runtime to pull the correct container image from the ECR repository.

```

 "");
 waitForInputToContinue(scanner);

 try {
 ecrActions.getRepositoryURI(repoName);

 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;

```

```

 } catch (RuntimeException e) {
 System.err.println("An error occurred while retrieving the URI: " +
e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);

```

```

System.out.println(DASHES);
System.out.println("""
 6. Set an ECR Lifecycle Policy.

```

An ECR Lifecycle Policy is used to manage the lifecycle of Docker images stored in your ECR repositories.

These policies allow you to automatically remove old or unused Docker images from your repositories, freeing up storage space and reducing costs.

This example policy helps to maintain the size and efficiency of the container registry by automatically removing older and potentially unused images, ensuring that the

storage is optimized and the registry remains up-to-date.  
 """);

```

 waitForInputToContinue(scanner);
 try {
 ecrActions.setLifeCyclePolicy(repoName);

 } catch (RuntimeException e) {
 System.err.println("An error occurred while setting the lifecycle
policy: " + e.getMessage());
 e.printStackTrace();
 return;
 }
 waitForInputToContinue(scanner);

```

```

System.out.println(DASHES);
System.out.println("""
 7. Push a docker image to the Amazon ECR Repository.

```

The `pushImageCmd()` method pushes a local Docker image to an Amazon ECR repository.

It sets up the Docker client by connecting to the local Docker host using the default port.

It then retrieves the authorization token for the ECR repository by making a call to the AWS SDK.

The method uses the authorization token to create an `AuthConfig` object, which is used to authenticate

the Docker client when pushing the image. Finally, the method tags the Docker image with the specified

repository name and image tag, and then pushes the image to the ECR repository using the Docker client.

If the push operation is successful, the method prints a message indicating that the image was pushed to ECR.

```

 "");
 waitForInputToContinue(scanner);

 try {
 ecrActions.pushDockerImage(repoName, localImageName);

 } catch (RuntimeException e) {
 System.err.println("An error occurred while pushing a local Docker image
to Amazon ECR: " + e.getMessage());
 e.printStackTrace();
 return;
 }
 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("8. Verify if the image is in the ECR Repository.");
 waitForInputToContinue(scanner);
 try {
 ecrActions.verifyImage(repoName, localImageName);

 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("An error occurred " + e.getMessage());
 e.printStackTrace();
 return;
 }
 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("9. As an optional step, you can interact with the image
in Amazon ECR by using the CLI.");

```

```

 System.out.println("Would you like to view instructions on how to use the
CLI to run the image? (y/n)");
 String ans = scanner.nextLine().trim();
 if (ans.equalsIgnoreCase("y")) {
 String instructions = ""
 1. Authenticate with ECR - Before you can pull the image from Amazon
ECR, you need to authenticate with the registry. You can do this using the AWS CLI:

 aws ecr get-login-password --region us-east-1 | docker login --
username AWS --password-stdin %s.dkr.ecr.us-east-1.amazonaws.com

 2. Describe the image using this command:

 aws ecr describe-images --repository-name %s --image-ids imageTag=%s

 3. Run the Docker container and view the output using this command:

 docker run --rm %s.dkr.ecr.us-east-1.amazonaws.com/%s:%s
 """;

 instructions = String.format(instructions, accountId, repoName,
localImageName, accountId, repoName, localImageName);
 System.out.println(instructions);
 }
 waitForInputToContinue(scanner);

 System.out.println(DASHES);
 System.out.println("10. Delete the ECR Repository.");
 System.out.println(
 ""
 If the repository isn't empty, you must either delete the contents of the
repository
 or use the force option (used in this scenario) to delete the repository and
have Amazon ECR delete all of its contents
 on your behalf.
 "");
 System.out.println("Would you like to delete the Amazon ECR Repository? (y/
n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 System.out.println("You selected to delete the AWS ECR resources.");

 try {
 ecrActions.deleteECRRepository(repoName);

```

```

 } catch (EcrException e) {
 System.err.println("An ECR exception occurred: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("An error occurred while deleting the Docker
image: " + e.getMessage());
 e.printStackTrace();
 return;
 }
 }

 System.out.println(DASHES);
 System.out.println("This concludes the Amazon ECR SDK scenario");
 System.out.println(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
 }
}
}
}

```

A wrapper class for Amazon ECR SDK methods.

```

import com.github.dockerjava.api.DockerClient;
import com.github.dockerjava.api.exception.DockerClientException;
import com.github.dockerjava.api.model.AuthConfig;
import com.github.dockerjava.api.model.Image;
import com.github.dockerjava.core.DockerClientBuilder;

```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecr.EcrAsyncClient;
import software.amazon.awssdk.services.ecr.model.AuthorizationData;
import software.amazon.awssdk.services.ecr.model.CreateRepositoryRequest;
import software.amazon.awssdk.services.ecr.model.CreateRepositoryResponse;
import software.amazon.awssdk.services.ecr.model.DeleteRepositoryRequest;
import software.amazon.awssdk.services.ecr.model.DeleteRepositoryResponse;
import software.amazon.awssdk.services.ecr.model.DescribeImagesRequest;
import software.amazon.awssdk.services.ecr.model.DescribeImagesResponse;
import software.amazon.awssdk.services.ecr.model.DescribeRepositoriesRequest;
import software.amazon.awssdk.services.ecr.model.DescribeRepositoriesResponse;
import software.amazon.awssdk.services.ecr.model.EcrException;
import software.amazon.awssdk.services.ecr.model.GetAuthorizationTokenResponse;
import software.amazon.awssdk.services.ecr.model.GetRepositoryPolicyRequest;
import software.amazon.awssdk.services.ecr.model.GetRepositoryPolicyResponse;
import software.amazon.awssdk.services.ecr.model.ImageIdentifier;
import software.amazon.awssdk.services.ecr.model.Repository;
import software.amazon.awssdk.services.ecr.model.RepositoryPolicyNotFoundException;
import software.amazon.awssdk.services.ecr.model.SetRepositoryPolicyRequest;
import software.amazon.awssdk.services.ecr.model.SetRepositoryPolicyResponse;
import software.amazon.awssdk.services.ecr.model.StartLifecyclePolicyPreviewRequest;
import
 software.amazon.awssdk.services.ecr.model.StartLifecyclePolicyPreviewResponse;
import com.github.dockerjava.api.command.DockerCmdExecFactory;
import com.github.dockerjava.netty.NettyDockerCmdExecFactory;
import java.time.Duration;
import java.util.Base64;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

public class ECRActions {
 private static EcrAsyncClient ecrClient;

 private static DockerClient dockerClient;

 private static Logger logger = LoggerFactory.getLogger(ECRActions.class);

 /**
```

```

 * Creates an Amazon Elastic Container Registry (Amazon ECR) repository.
 *
 * @param repoName the name of the repository to create.
 * @return the Amazon Resource Name (ARN) of the created repository, or an empty
string if the operation failed.
 * @throws IllegalArgumentException If repository name is invalid.
 * @throws RuntimeException if an error occurs while creating the
repository.
 */
 public String createECRRepository(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 CreateRepositoryRequest request = CreateRepositoryRequest.builder()
 .repositoryName(repoName)
 .build();

 CompletableFuture<CreateRepositoryResponse> response =
getAsyncClient().createRepository(request);
 try {
 CreateRepositoryResponse result = response.join();
 if (result != null) {
 System.out.println("The " + repoName + " repository was created
successfully.");
 return result.repository().repositoryArn();
 } else {
 throw new RuntimeException("Unexpected response type");
 }
 } catch (CompletionException e) {
 Throwable cause = e.getCause();
 if (cause instanceof EcrException ex) {
 if
("RepositoryAlreadyExistsException".equals(ex.awsErrorDetails().errorCode())) {
 System.out.println("The Amazon ECR repository already exists,
moving on...");

 DescribeRepositoriesRequest describeRequest =
DescribeRepositoriesRequest.builder()
 .repositoryNames(repoName)
 .build();

 DescribeRepositoriesResponse describeResponse =
getAsyncClient().describeRepositories(describeRequest).join();
 return describeResponse.repositories().get(0).repositoryArn();
 }
 }
 }
 }

```

```

 } else {
 throw new RuntimeException(ex);
 }
 } else {
 throw new RuntimeException(e);
 }
}

/**
 * Deletes an ECR (Elastic Container Registry) repository.
 *
 * @param repoName the name of the repository to delete.
 * @throws IllegalArgumentException if the repository name is null or empty.
 * @throws EcrException if there is an error deleting the repository.
 * @throws RuntimeException if an unexpected error occurs during the deletion
process.
 */
public void deleteECRRepository(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 DeleteRepositoryRequest repositoryRequest =
DeleteRepositoryRequest.builder()
 .force(true)
 .repositoryName(repoName)
 .build();

 CompletableFuture<DeleteRepositoryResponse> response =
getAsyncClient().deleteRepository(repositoryRequest);
 response.whenComplete((deleteRepositoryResponse, ex) -> {
 if (deleteRepositoryResponse != null) {
 System.out.println("You have successfully deleted the " + repoName +
" repository");
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 }
 });
}

```



```

 }
 });

 // Wait for the CompletableFuture to complete
 response.join();
}

private static DockerClient getDockerClient() {
 String osName = System.getProperty("os.name");
 if (osName.startsWith("Windows")) {
 // Make sure Docker Desktop is running.
 String dockerHost = "tcp://localhost:2375"; // Use the Docker Desktop
default port.
 DockerCmdExecFactory dockerCmdExecFactory = new
NettyDockerCmdExecFactory().withReadTimeout(20000).withConnectTimeout(20000);
 dockerClient =
DockerClientBuilder.getInstance(dockerHost).withDockerCmdExecFactory(dockerCmdExecFactory).
 } else {
 dockerClient = DockerClientBuilder.getInstance().build();
 }
 return dockerClient;
}

/**
 * Retrieves an asynchronous Amazon Elastic Container Registry (ECR) client.
 *
 * @return the configured ECR asynchronous client.
 */
private static EcrAsyncClient getAsyncClient() {

 /**
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP client
for interacting with AWS services.
 It uses the Netty framework to handle the underlying network communication
and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.

```

```

 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.
 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.
 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the individual
call attempt timeout.
 .build();

 if (ecrClient == null) {
 ecrClient = EcrAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return ecrClient;
 }

 /**
 * Sets the lifecycle policy for the specified repository.
 *
 * @param repoName the name of the repository for which to set the lifecycle
policy.
 */
 public void setLifecyclePolicy(String repoName) {
 /**
 This policy helps to maintain the size and efficiency of the container
registry
 by automatically removing older and potentially unused images,
 ensuring that the storage is optimized and the registry remains up-to-
date.
 */
 String polText = ""
 {
 "rules": [
 {
 "rulePriority": 1,
 "description": "Expire images older than 14 days",

```

```

 "selection": {
 "tagStatus": "any",
 "countType": "sinceImagePushed",
 "countUnit": "days",
 "countNumber": 14
 },
 "action": {
 "type": "expire"
 }
 }
}
}
}
""";

StartLifecyclePolicyPreviewRequest lifecyclePolicyPreviewRequest =
StartLifecyclePolicyPreviewRequest.builder()
 .lifecyclePolicyText(polText)
 .repositoryName(repoName)
 .build();

CompletableFuture<StartLifecyclePolicyPreviewResponse> response =
getAsyncClient().startLifecyclePolicyPreview(lifecyclePolicyPreviewRequest);
response.whenComplete((lifecyclePolicyPreviewResponse, ex) -> {
 if (lifecyclePolicyPreviewResponse != null) {
 System.out.println("Lifecycle policy preview started
successfully.");
 } else {
 if (ex.getCause() instanceof EcrException) {
 throw (EcrException) ex.getCause();
 } else {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 }
});
// Wait for the CompletableFuture to complete.
response.join();
}

/**
 * Verifies the existence of an image in an Amazon Elastic Container Registry
 (Amazon ECR) repository asynchronously.
 */

```

```

 * @param repositoryName The name of the Amazon ECR repository.
 * @param imageTag The tag of the image to verify.
 * @throws EcrException if there is an error retrieving the image
information from Amazon ECR.
 * @throws CompletionException if the asynchronous operation completes
exceptionally.
 */
 public void verifyImage(String repositoryName, String imageTag) {
 DescribeImagesRequest request = DescribeImagesRequest.builder()
 .repositoryName(repositoryName)
 .imageIds(ImageIdentifier.builder().imageTag(imageTag).build())
 .build();

 CompletableFuture<DescribeImagesResponse> response =
getAsyncClient().describeImages(request);
 response.whenComplete((describeImagesResponse, ex) -> {
 if (ex != null) {
 if (ex instanceof CompletionException) {
 Throwable cause = ex.getCause();
 if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 } else {
 throw new RuntimeException("Unexpected error: " +
ex.getCause());
 }
 } else if (describeImagesResponse != null && !
describeImagesResponse.imageDetails().isEmpty()) {
 System.out.println("Image is present in the repository.");
 } else {
 System.out.println("Image is not present in the repository.");
 }
 });

 // Wait for the CompletableFuture to complete.
 response.join();
 }

 /**
 * Retrieves the repository URI for the specified repository name.
 *

```

```

 * @param repoName the name of the repository to retrieve the URI for.
 * @return the repository URI for the specified repository name.
 * @throws EcrException if there is an error retrieving the repository
information.
 * @throws CompletionException if the asynchronous operation completes
exceptionally.
 */
 public void getRepositoryURI(String repoName) {
 DescribeRepositoriesRequest request = DescribeRepositoriesRequest.builder()
 .repositoryNames(repoName)
 .build();

 CompletableFuture<DescribeRepositoriesResponse> response =
getAsyncClient().describeRepositories(request);
 response.whenComplete((describeRepositoriesResponse, ex) -> {
 if (ex != null) {
 Throwable cause = ex.getCause();
 if (cause instanceof InterruptedException) {
 Thread.currentThread().interrupt();
 String errorMessage = "Thread interrupted while waiting for
asynchronous operation: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 } else if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 String errorMessage = "Unexpected error: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 }
 } else {
 if (describeRepositoriesResponse != null) {
 if (!describeRepositoriesResponse.repositories().isEmpty()) {
 String repositoryUri =
describeRepositoriesResponse.repositories().get(0).repositoryUri();
 System.out.println("Repository URI found: " +
repositoryUri);
 } else {
 System.out.println("No repositories found for the given
name.");
 }
 } else {
 System.err.println("No response received from
describeRepositories.");
 }
 }
 })
 }

```

```

 });
 response.join();
}

/**
 * Retrieves the authorization token for Amazon Elastic Container Registry
 * (ECR).
 * This method makes an asynchronous call to the ECR client to retrieve the
 * authorization token.
 * If the operation is successful, the method prints the token to the console.
 * If an exception occurs, the method handles the exception and prints the error
 * message.
 *
 * @throws EcrException if there is an error retrieving the authorization
 * token from ECR.
 * @throws RuntimeException if there is an unexpected error during the
 * operation.
 */
public void getAuthToken() {
 CompletableFuture<GetAuthorizationTokenResponse> response =
getAsyncClient().getAuthorizationToken();
 response.whenComplete((authorizationTokenResponse, ex) -> {
 if (authorizationTokenResponse != null) {
 AuthorizationData authorizationData =
authorizationTokenResponse.authorizationData().get(0);
 String token = authorizationData.authorizationToken();
 if (!token.isEmpty()) {
 System.out.println("The token was successfully retrieved.");
 }
 } else {
 if (ex.getCause() instanceof EcrException) {
 throw (EcrException) ex.getCause();
 } else {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex); // Rethrow the
exception
 }
 }
 });
 response.join();
}

/**

```

```

 * Gets the repository policy for the specified repository.
 *
 * @param repoName the name of the repository.
 * @throws EcrException if an AWS error occurs while getting the repository
policy.
 */
 public String getRepoPolicy(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 GetRepositoryPolicyRequest getRepositoryPolicyRequest =
GetRepositoryPolicyRequest.builder()
 .repositoryName(repoName)
 .build();

 CompletableFuture<GetRepositoryPolicyResponse> response =
getAsyncClient().getRepositoryPolicy(getRepositoryPolicyRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 System.out.println("Repository policy retrieved successfully.");
 } else {
 if (ex.getCause() instanceof EcrException) {
 throw (EcrException) ex.getCause();
 } else {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 }
 });

 GetRepositoryPolicyResponse result = response.join();
 return result != null ? result.policyText() : null;
 }

 /**
 * Sets the repository policy for the specified ECR repository.
 *
 * @param repoName the name of the ECR repository.
 * @param iamRole the IAM role to be granted access to the repository.
 * @throws RepositoryPolicyNotFoundException if the repository policy does not
exist.

```

```

 * @throws EcrException if there is an unexpected error
 setting the repository policy.
 */
 public void setRepoPolicy(String repoName, String iamRole) {
 /*
 This example policy document grants the specified AWS principal the
 permission to perform the
 `ecr:BatchGetImage` action. This policy is designed to allow the specified
 principal
 to retrieve Docker images from the ECR repository.
 */
 String policyDocumentTemplate = ""
 {
 "Version": "2012-10-17",
 "Statement" : [{
 "Sid" : "new statement",
 "Effect" : "Allow",
 "Principal" : {
 "AWS" : "%s"
 },
 "Action" : "ecr:BatchGetImage"
 }]
 }
 """;

 String policyDocument = String.format(policyDocumentTemplate, iamRole);
 SetRepositoryPolicyRequest setRepositoryPolicyRequest =
 SetRepositoryPolicyRequest.builder()
 .repositoryName(repoName)
 .policyText(policyDocument)
 .build();

 CompletableFuture<SetRepositoryPolicyResponse> response =
 getAsyncClient().setRepositoryPolicy(setRepositoryPolicyRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 System.out.println("Repository policy set successfully.");
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof RepositoryPolicyNotFoundException) {
 throw (RepositoryPolicyNotFoundException) cause;
 } else if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {

```



```

 String errorMessage = "Unexpected error: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 }
}

});
response.join();
}

/**
 * Pushes a Docker image to an Amazon Elastic Container Registry (ECR)
repository.
 *
 * @param repoName the name of the ECR repository to push the image to.
 * @param imageName the name of the Docker image.
 */
public void pushDockerImage(String repoName, String imageName) {
 System.out.println("Pushing " + imageName + " to Amazon ECR will take a few
seconds.");
 CompletableFuture<AuthConfig> authResponseFuture =
getAsyncClient().getAuthorizationToken()
 .thenApply(response -> {
 String token =
response.authorizationData().get(0).authorizationToken();
 String decodedToken = new String(Base64.getDecoder().decode(token));
 String password = decodedToken.substring(4);

 DescribeRepositoriesResponse descrRepoResponse =
getAsyncClient().describeRepositories(b -> b.repositoryNames(repoName)).join();
 Repository repoData =
descrRepoResponse.repositories().stream().filter(r ->
r.repositoryName().equals(repoName)).findFirst().orElse(null);
 assert repoData != null;
 String registryURL = repoData.repositoryUri().split("/")[0];

 AuthConfig authConfig = new AuthConfig()
 .withUsername("AWS")
 .withPassword(password)
 .withRegistryAddress(registryURL);
 return authConfig;
 })
 .thenCompose(authConfig -> {
 DescribeRepositoriesResponse descrRepoResponse =
getAsyncClient().describeRepositories(b -> b.repositoryNames(repoName)).join();

```

```

 Repository repoData =
descrRepoResponse.repositories().stream().filter(r ->
r.repositoryName().equals(repoName)).findFirst().orElse(null);
 getDockerClient().tagImageCmd(imageName + ":latest",
repoData.repositoryUri() + ":latest", imageName).exec();
 try {

getDockerClient().pushImageCmd(repoData.repositoryUri()).withTag("echo-
text").withAuthConfig(authConfig).start().awaitCompletion();
 System.out.println("The " + imageName + " was pushed to ECR");

 } catch (InterruptedException e) {
 throw (RuntimeException) e.getCause();
 }
 return CompletableFuture.completedFuture(authConfig);
 });

 authResponseFuture.join();
}

// Make sure local image echo-text exists.
public boolean isEchoTextImagePresent() {
 try {
 List<Image> images = getDockerClient().listImagesCmd().exec();
 boolean helloWorldFound = false;
 for (Image image : images) {
 String[] repoTags = image.getRepoTags();
 if (repoTags != null) {
 for (String tag : repoTags) {
 if (tag.startsWith("echo-text")) {
 System.out.println(tag);
 helloWorldFound = true;
 }
 }
 }
 }
 if (helloWorldFound) {
 System.out.println("The local image named echo-text exists.");
 return true;
 } else {
 System.out.println("The local image named echo-text does not
exist.");
 return false;
 }
 }
}

```

```
 } catch (DockerClientException ex) {
 logger.error("ERROR: " + ex.getMessage());
 return false;
 }
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateRepository](#)
  - [DeleteRepository](#)
  - [DescribeImages](#)
  - [DescribeRepositories](#)
  - [GetAuthorizationToken](#)
  - [GetRepositoryPolicy](#)
  - [SetRepositoryPolicy](#)
  - [StartLifecyclePolicyPreview](#)

## Actions

### CreateRepository

The following code example shows how to use `CreateRepository`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates an Amazon Elastic Container Registry (Amazon ECR) repository.
 *
 * @param repoName the name of the repository to create.
 * @return the Amazon Resource Name (ARN) of the created repository, or an empty
 * string if the operation failed.
```

```

 * @throws IllegalArgumentException If repository name is invalid.
 * @throws RuntimeException if an error occurs while creating the
repository.
 */
 public String createECRRepository(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 CreateRepositoryRequest request = CreateRepositoryRequest.builder()
 .repositoryName(repoName)
 .build();

 CompletableFuture<CreateRepositoryResponse> response =
getAsyncClient().createRepository(request);
 try {
 CreateRepositoryResponse result = response.join();
 if (result != null) {
 System.out.println("The " + repoName + " repository was created
successfully.");
 return result.repository().repositoryArn();
 } else {
 throw new RuntimeException("Unexpected response type");
 }
 } catch (CompletionException e) {
 Throwable cause = e.getCause();
 if (cause instanceof EcrException ex) {
 if
("RepositoryAlreadyExistsException".equals(ex.awsErrorDetails().errorCode())) {
 System.out.println("The Amazon ECR repository already exists,
moving on...");
 DescribeRepositoriesRequest describeRequest =
DescribeRepositoriesRequest.builder()
 .repositoryNames(repoName)
 .build();
 DescribeRepositoriesResponse describeResponse =
getAsyncClient().describeRepositories(describeRequest).join();
 return describeResponse.repositories().get(0).repositoryArn();
 } else {
 throw new RuntimeException(ex);
 }
 } else {
 throw new RuntimeException(e);
 }
 }
 }

```

```
 }
 }
}
```

- For API details, see [CreateRepository](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteRepository

The following code example shows how to use DeleteRepository.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an ECR (Elastic Container Registry) repository.
 *
 * @param repoName the name of the repository to delete.
 * @throws IllegalArgumentException if the repository name is null or empty.
 * @throws EcrException if there is an error deleting the repository.
 * @throws RuntimeException if an unexpected error occurs during the deletion
 * process.
 */
public void deleteECRRepository(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 DeleteRepositoryRequest repositoryRequest =
DeleteRepositoryRequest.builder()
 .force(true)
 .repositoryName(repoName)
 .build();

 CompletableFuture<DeleteRepositoryResponse> response =
getAsyncClient().deleteRepository(repositoryRequest);
}
```

```

 response.whenComplete((deleteRepositoryResponse, ex) -> {
 if (deleteRepositoryResponse != null) {
 System.out.println("You have successfully deleted the " + repoName +
" repository");
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 }
 });

 // Wait for the CompletableFuture to complete
 response.join();
 }
}

```

- For API details, see [DeleteRepository](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeImages

The following code example shows how to use DescribeImages.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Verifies the existence of an image in an Amazon Elastic Container Registry
(Amazon ECR) repository asynchronously.
 *
 * @param repositoryName The name of the Amazon ECR repository.
 * @param imageTag The tag of the image to verify.
 * @throws EcrException if there is an error retrieving the image
information from Amazon ECR.

```

```

 * @throws CompletionException if the asynchronous operation completes
exceptionally.
 */
 public void verifyImage(String repositoryName, String imageTag) {
 DescribeImagesRequest request = DescribeImagesRequest.builder()
 .repositoryName(repositoryName)
 .imageIds(ImageIdentifier.builder().imageTag(imageTag).build())
 .build();

 CompletableFuture<DescribeImagesResponse> response =
getAsyncClient().describeImages(request);
 response.whenComplete((describeImagesResponse, ex) -> {
 if (ex != null) {
 if (ex instanceof CompletionException) {
 Throwable cause = ex.getCause();
 if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 } else {
 throw new RuntimeException("Unexpected error: " +
ex.getCause());
 }
 } else if (describeImagesResponse != null && !
describeImagesResponse.imageDetails().isEmpty()) {
 System.out.println("Image is present in the repository.");
 } else {
 System.out.println("Image is not present in the repository.");
 }
 });

 // Wait for the CompletableFuture to complete.
 response.join();
 }

```

- For API details, see [DescribeImages](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeRepositories

The following code example shows how to use `DescribeRepositories`.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the repository URI for the specified repository name.
 *
 * @param repoName the name of the repository to retrieve the URI for.
 * @return the repository URI for the specified repository name.
 * @throws EcrException if there is an error retrieving the repository
information.
 * @throws CompletionException if the asynchronous operation completes
exceptionally.
 */
public void getRepositoryURI(String repoName) {
 DescribeRepositoriesRequest request = DescribeRepositoriesRequest.builder()
 .repositoryNames(repoName)
 .build();

 CompletableFuture<DescribeRepositoriesResponse> response =
getAsyncClient().describeRepositories(request);
 response.whenComplete((describeRepositoriesResponse, ex) -> {
 if (ex != null) {
 Throwable cause = ex.getCause();
 if (cause instanceof InterruptedException) {
 Thread.currentThread().interrupt();
 String errorMessage = "Thread interrupted while waiting for
asynchronous operation: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 } else if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 String errorMessage = "Unexpected error: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 }
 } else {
 if (describeRepositoriesResponse != null) {
 if (!describeRepositoriesResponse.repositories().isEmpty()) {
```



```

 String repositoryUri =
describeRepositoriesResponse.repositories().get(0).repositoryUri();
 System.out.println("Repository URI found: " +
repositoryUri);
 } else {
 System.out.println("No repositories found for the given
name.");
 }
} else {
 System.err.println("No response received from
describeRepositories.");
}
}
});
response.join();
}

```

- For API details, see [DescribeRepositories](#) in *AWS SDK for Java 2.x API Reference*.

## GetAuthorizationToken

The following code example shows how to use `GetAuthorizationToken`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the authorization token for Amazon Elastic Container Registry
(ECR).
 * This method makes an asynchronous call to the ECR client to retrieve the
authorization token.
 * If the operation is successful, the method prints the token to the console.
 * If an exception occurs, the method handles the exception and prints the error
message.
 *

```

```

 * @throws EcrException if there is an error retrieving the authorization
token from ECR.
 * @throws RuntimeException if there is an unexpected error during the
operation.
 */
 public void getAuthToken() {
 CompletableFuture<GetAuthorizationTokenResponse> response =
getAsyncClient().getAuthorizationToken();
 response.whenComplete((authorizationTokenResponse, ex) -> {
 if (authorizationTokenResponse != null) {
 AuthorizationData authorizationData =
authorizationTokenResponse.authorizationData().get(0);
 String token = authorizationData.authorizationToken();
 if (!token.isEmpty()) {
 System.out.println("The token was successfully retrieved.");
 }
 } else {
 if (ex.getCause() instanceof EcrException) {
 throw (EcrException) ex.getCause();
 } else {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex); // Rethrow the
exception
 }
 }
 });
 response.join();
 }
}

```

- For API details, see [GetAuthorizationToken](#) in *AWS SDK for Java 2.x API Reference*.

## GetRepositoryPolicy

The following code example shows how to use `GetRepositoryPolicy`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Gets the repository policy for the specified repository.
 *
 * @param repoName the name of the repository.
 * @throws EcrException if an AWS error occurs while getting the repository
 * policy.
 */
public String getRepoPolicy(String repoName) {
 if (repoName == null || repoName.isEmpty()) {
 throw new IllegalArgumentException("Repository name cannot be null or
empty");
 }

 GetRepositoryPolicyRequest getRepositoryPolicyRequest =
GetRepositoryPolicyRequest.builder()
 .repositoryName(repoName)
 .build();

 CompletableFuture<GetRepositoryPolicyResponse> response =
getAsyncClient().getRepositoryPolicy(getRepositoryPolicyRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 System.out.println("Repository policy retrieved successfully.");
 } else {
 if (ex.getCause() instanceof EcrException) {
 throw (EcrException) ex.getCause();
 } else {
 String errorMessage = "Unexpected error occurred: " +
ex.getMessage();
 throw new RuntimeException(errorMessage, ex);
 }
 }
 });
}
```

```

 GetRepositoryPolicyResponse result = response.join();
 return result != null ? result.policyText() : null;
 }

```

- For API details, see [GetRepositoryPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## PushImageCmd

The following code example shows how to use PushImageCmd.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Pushes a Docker image to an Amazon Elastic Container Registry (ECR)
repository.
 *
 * @param repoName the name of the ECR repository to push the image to.
 * @param imageName the name of the Docker image.
 */
public void pushDockerImage(String repoName, String imageName) {
 System.out.println("Pushing " + imageName + " to Amazon ECR will take a few
seconds.");
 CompletableFuture<AuthConfig> authResponseFuture =
getAsyncClient().getAuthorizationToken()
 .thenApply(response -> {
 String token =
response.authorizationData().get(0).authorizationToken();
 String decodedToken = new String(Base64.getDecoder().decode(token));
 String password = decodedToken.substring(4);

 DescribeRepositoriesResponse descrRepoResponse =
getAsyncClient().describeRepositories(b -> b.repositoryNames(repoName)).join();
 Repository repoData =
descrRepoResponse.repositories().stream().filter(r ->
r.repositoryName().equals(repoName)).findFirst().orElse(null);

```

```

 assert repoData != null;
 String registryURL = repoData.repositoryUri().split("/")[0];

 AuthConfig authConfig = new AuthConfig()
 .withUsername("AWS")
 .withPassword(password)
 .withRegistryAddress(registryURL);
 return authConfig;
 })
 .thenCompose(authConfig -> {
 DescribeRepositoriesResponse descrRepoResponse =
getAsyncClient().describeRepositories(b -> b.repositoryNames(repoName)).join();
 Repository repoData =
descrRepoResponse.repositories().stream().filter(r ->
r.repositoryName().equals(repoName)).findFirst().orElse(null);
 getDockerClient().tagImageCmd(imageName + ":latest",
repoData.repositoryUri() + ":latest", imageName).exec();
 try {

getDockerClient().pushImageCmd(repoData.repositoryUri()).withTag("echo-
text").withAuthConfig(authConfig).start().awaitCompletion();
 System.out.println("The " + imageName + " was pushed to ECR");

 } catch (InterruptedException e) {
 throw (RuntimeException) e.getCause();
 }
 return CompletableFuture.completedFuture(authConfig);
 });

 authResponseFuture.join();
}

```

- For API details, see [PushImageCmd](#) in *AWS SDK for Java 2.x API Reference*.

## SetRepositoryPolicy

The following code example shows how to use SetRepositoryPolicy.

## SDK for Java 2.x

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Sets the repository policy for the specified ECR repository.
 *
 * @param repoName the name of the ECR repository.
 * @param iamRole the IAM role to be granted access to the repository.
 * @throws RepositoryPolicyNotFoundException if the repository policy does not
 * exist.
 * @throws EcrException if there is an unexpected error
 * setting the repository policy.
 */
public void setRepoPolicy(String repoName, String iamRole) {
 /**
 * This example policy document grants the specified AWS principal the
 * permission to perform the
 * `ecr:BatchGetImage` action. This policy is designed to allow the specified
 * principal
 * to retrieve Docker images from the ECR repository.
 */
 String policyDocumentTemplate = ""
 {
 "Version": "2012-10-17",
 "Statement" : [{
 "Sid" : "new statement",
 "Effect" : "Allow",
 "Principal" : {
 "AWS" : "%s"
 },
 "Action" : "ecr:BatchGetImage"
 }]
 }
 """;

 String policyDocument = String.format(policyDocumentTemplate, iamRole);
}
```

```

 SetRepositoryPolicyRequest setRepositoryPolicyRequest =
SetRepositoryPolicyRequest.builder()
 .repositoryName(repoName)
 .policyText(policyDocument)
 .build();

 CompletableFuture<SetRepositoryPolicyResponse> response =
getAsyncClient().setRepositoryPolicy(setRepositoryPolicyRequest);
 response.whenComplete((resp, ex) -> {
 if (resp != null) {
 System.out.println("Repository policy set successfully.");
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof RepositoryPolicyNotFoundException) {
 throw (RepositoryPolicyNotFoundException) cause;
 } else if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 String errorMessage = "Unexpected error: " + cause.getMessage();
 throw new RuntimeException(errorMessage, cause);
 }
 }
 });
 response.join();
}

```

- For API details, see [SetRepositoryPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## StartLifecyclePolicyPreview

The following code example shows how to use StartLifecyclePolicyPreview.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Verifies the existence of an image in an Amazon Elastic Container Registry
 (Amazon ECR) repository asynchronously.
 *
 * @param repositoryName The name of the Amazon ECR repository.
 * @param imageTag The tag of the image to verify.
 * @throws EcrException if there is an error retrieving the image
 information from Amazon ECR.
 * @throws CompletionException if the asynchronous operation completes
 exceptionally.
 */
 public void verifyImage(String repositoryName, String imageTag) {
 DescribeImagesRequest request = DescribeImagesRequest.builder()
 .repositoryName(repositoryName)
 .imageIds(ImageIdentifier.builder().imageTag(imageTag).build())
 .build();

 CompletableFuture<DescribeImagesResponse> response =
 getAsyncClient().describeImages(request);
 response.whenComplete((describeImagesResponse, ex) -> {
 if (ex != null) {
 if (ex instanceof CompletionException) {
 Throwable cause = ex.getCause();
 if (cause instanceof EcrException) {
 throw (EcrException) cause;
 } else {
 throw new RuntimeException("Unexpected error: " +
 cause.getMessage(), cause);
 }
 } else {
 throw new RuntimeException("Unexpected error: " +
 ex.getCause());
 }
 } else if (describeImagesResponse != null && !
 describeImagesResponse.imageDetails().isEmpty()) {
 System.out.println("Image is present in the repository.");
 } else {
 System.out.println("Image is not present in the repository.");
 }
 });

 // Wait for the CompletableFuture to complete.
 response.join();
 }

```



- For API details, see [StartLifecyclePolicyPreview](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon ECS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon ECS.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### CreateCluster

The following code example shows how to use `CreateCluster`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.ExecuteCommandConfiguration;
import software.amazon.awssdk.services.ecs.model.ExecuteCommandLogging;
import software.amazon.awssdk.services.ecs.model.ClusterConfiguration;
import software.amazon.awssdk.services.ecs.model.CreateClusterResponse;
import software.amazon.awssdk.services.ecs.model.EcsException;
```

```

import software.amazon.awssdk.services.ecs.model.CreateClusterRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateCluster {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clusterName>\s

 Where:
 clusterName - The name of the ECS cluster to create.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String clusterName = args[0];
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

 String clusterArn = createGivenCluster(ecsClient, clusterName);
 System.out.println("The cluster ARN is " + clusterArn);
 ecsClient.close();
 }

 public static String createGivenCluster(EcsClient ecsClient, String clusterName)
 {
 try {
 ExecuteCommandConfiguration commandConfiguration =
 ExecuteCommandConfiguration.builder()
 .logging(ExecuteCommandLogging.DEFAULT)
 .build();

```

```

 ClusterConfiguration clusterConfiguration =
ClusterConfiguration.builder()
 .executeCommandConfiguration(commandConfiguration)
 .build();

 CreateClusterRequest clusterRequest = CreateClusterRequest.builder()
 .clusterName(clusterName)
 .configuration(clusterConfiguration)
 .build();

 CreateClusterResponse response =
ecsClient.createCluster(clusterRequest);
 return response.cluster().clusterArn();

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see [CreateCluster](#) in *AWS SDK for Java 2.x API Reference*.

## CreateService

The following code example shows how to use CreateService.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.AwsVpcConfiguration;
import software.amazon.awssdk.services.ecs.model.NetworkConfiguration;

```

```

import software.amazon.awssdk.services.ecs.model.CreateServiceRequest;
import software.amazon.awssdk.services.ecs.model.LaunchType;
import software.amazon.awssdk.services.ecs.model.CreateServiceResponse;
import software.amazon.awssdk.services.ecs.model.EcsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateService {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clusterName> <serviceName> <securityGroups>

 <subnets> <taskDefinition>

 Where:
 clusterName - The name of the ECS cluster.
 serviceName - The name of the ECS service to
 create.

 securityGroups - The name of the security group.
 subnets - The name of the subnet.
 taskDefinition - The name of the task definition.
 """;

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String clusterName = args[0];
 String serviceName = args[1];
 String securityGroups = args[2];
 String subnets = args[3];
 String taskDefinition = args[4];
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

```

```

 String serviceArn = createNewService(ecsClient, clusterName,
serviceName, securityGroups, subnets,
 taskDefinition);
 System.out.println("The ARN of the service is " + serviceArn);
 ecsClient.close();
 }

 public static String createNewService(EcsClient ecsClient,
 String clusterName,
 String serviceName,
 String securityGroups,
 String subnets,
 String taskDefinition) {

 try {
 AwsVpcConfiguration vpcConfiguration =
AwsVpcConfiguration.builder()
 .securityGroups(securityGroups)
 .subnets(subnets)
 .build();

 NetworkConfiguration configuration =
NetworkConfiguration.builder()
 .awsvpcConfiguration(vpcConfiguration)
 .build();

 CreateServiceRequest serviceRequest =
CreateServiceRequest.builder()
 .cluster(clusterName)
 .networkConfiguration(configuration)
 .desiredCount(1)
 .launchType(LaunchType.FARGATE)
 .serviceName(serviceName)
 .taskDefinition(taskDefinition)
 .build();

 CreateServiceResponse response =
ecsClient.createService(serviceRequest);
 return response.service().serviceArn();

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

```

```

 }
 return "";
 }
}

```

- For API details, see [CreateService](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteService

The following code example shows how to use DeleteService.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.DeleteServiceRequest;
import software.amazon.awssdk.services.ecs.model.EcsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class DeleteService {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clusterName> <serviceArn>\s

```

```

 Where:
 clusterName - The name of the ECS cluster.
 serviceArn - The ARN of the ECS service.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String clusterName = args[0];
 String serviceArn = args[1];
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

 deleteSpecificService(ecsClient, clusterName, serviceArn);
 ecsClient.close();
}

public static void deleteSpecificService(EcsClient ecsClient, String
clusterName, String serviceArn) {
 try {
 DeleteServiceRequest serviceRequest = DeleteServiceRequest.builder()
 .cluster(clusterName)
 .service(serviceArn)
 .build();

 ecsClient.deleteService(serviceRequest);
 System.out.println("The Service was successfully deleted");

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteService](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeClusters

The following code example shows how to use DescribeClusters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.DescribeClustersRequest;
import software.amazon.awssdk.services.ecs.model.DescribeClustersResponse;
import software.amazon.awssdk.services.ecs.model.Cluster;
import software.amazon.awssdk.services.ecs.model.EcsException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeClusters {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clusterArn> \s

 Where:
 clusterArn - The ARN of the ECS cluster to describe.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```



```
String clusterArn = args[0];
Region region = Region.US_EAST_1;
EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

descCluster(ecsClient, clusterArn);
}

public static void descCluster(EcsClient ecsClient, String clusterArn) {
 try {
 DescribeClustersRequest clustersRequest =
DescribeClustersRequest.builder()
 .clusters(clusterArn)
 .build();

 DescribeClustersResponse response =
ecsClient.describeClusters(clustersRequest);
 List<Cluster> clusters = response.clusters();
 for (Cluster cluster : clusters) {
 System.out.println("The cluster name is " + cluster.clusterName());
 }

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
```

- For API details, see [DescribeClusters](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeTasks

The following code example shows how to use DescribeTasks.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.DescribeTasksRequest;
import software.amazon.awssdk.services.ecs.model.DescribeTasksResponse;
import software.amazon.awssdk.services.ecs.model.EcsException;
import software.amazon.awssdk.services.ecs.model.Task;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListTaskDefinitions {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <clusterArn> <taskId>\s

 Where:
 clusterArn - The ARN of an ECS cluster.
 taskId - The task Id value.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String clusterArn = args[0];
```

```

 String taskId = args[1];
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

 getAllTasks(ecsClient, clusterArn, taskId);
 ecsClient.close();
 }

 public static void getAllTasks(EcsClient ecsClient, String clusterArn, String
taskId) {
 try {
 DescribeTasksRequest tasksRequest = DescribeTasksRequest.builder()
 .cluster(clusterArn)
 .tasks(taskId)
 .build();

 DescribeTasksResponse response = ecsClient.describeTasks(tasksRequest);
 List<Task> tasks = response.tasks();
 for (Task task : tasks) {
 System.out.println("The task ARN is " + task.taskDefinitionArn());
 }

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DescribeTasks](#) in *AWS SDK for Java 2.x API Reference*.

## ListClusters

The following code example shows how to use `ListClusters`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.ListClustersResponse;
import software.amazon.awssdk.services.ecs.model.EcsException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class ListClusters {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

 listAllClusters(ecsClient);
 ecsClient.close();
 }

 public static void listAllClusters(EcsClient ecsClient) {
 try {
 ListClustersResponse response = ecsClient.listClusters();
 List<String> clusters = response.clusterArns();
 for (String cluster : clusters) {
 System.out.println("The cluster arn is " + cluster);
 }
 }
 }
}
```

```

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListClusters](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateService

The following code example shows how to use UpdateService.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ecs.EcsClient;
import software.amazon.awssdk.services.ecs.model.EcsException;
import software.amazon.awssdk.services.ecs.model.UpdateServiceRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class UpdateService {

 public static void main(String[] args) {

 final String usage = ""

```

```

 Usage:
 <clusterName> <serviceArn>\s

 Where:
 clusterName - The cluster name.
 serviceArn - The service ARN value.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String clusterName = args[0];
 String serviceArn = args[1];
 Region region = Region.US_EAST_1;
 EcsClient ecsClient = EcsClient.builder()
 .region(region)
 .build();

 updateSpecificService(ecsClient, clusterName, serviceArn);
 ecsClient.close();
}

public static void updateSpecificService(EcsClient ecsClient, String
clusterName, String serviceArn) {
 try {
 UpdateServiceRequest serviceRequest = UpdateServiceRequest.builder()
 .cluster(clusterName)
 .service(serviceArn)
 .desiredCount(0)
 .build();

 ecsClient.updateService(serviceRequest);
 System.out.println("The service was modified");

 } catch (EcsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [UpdateService](#) in *AWS SDK for Java 2.x API Reference*.

## Elastic Load Balancing - Version 2 examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Elastic Load Balancing - Version 2.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Elastic Load Balancing

The following code example shows how to get started using Elastic Load Balancing.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class HelloLoadBalancer {
```

```

 public static void main(String[] args) {
 ElasticLoadBalancingV2Client loadBalancingV2Client =
 ElasticLoadBalancingV2Client.builder()
 .region(Region.US_EAST_1)
 .build();

 DescribeLoadBalancersResponse loadBalancersResponse =
 loadBalancingV2Client
 .describeLoadBalancers(r -> r.pageSize(10));
 List<LoadBalancer> loadBalancerList =
 loadBalancersResponse.loadBalancers();
 for (LoadBalancer lb : loadBalancerList)
 System.out.println("Load Balancer DNS name = " +
 lb.dnsName());
 }
}

```

- For API details, see [DescribeLoadBalancers](#) in *AWS SDK for Java 2.x API Reference*.

## Actions

### CreateListener

The following code example shows how to use `CreateListener`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/*
 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */
public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
 String lbName, int port,

```



```

 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("*** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.
 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

```

```

 .loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}

```

- For API details, see [CreateListener](#) in *AWS SDK for Java 2.x API Reference*.

## CreateLoadBalancer

The following code example shows how to use `CreateLoadBalancer`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/*
 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */

```

```

 public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
String lbName, int port,
 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("*** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.
 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

```

```

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

 .loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}

```

- For API details, see [CreateLoadBalancer](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTargetGroup

The following code example shows how to use `CreateTargetGroup`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/*
 * Creates an Elastic Load Balancing target group. The target group specifies
 * how

```

```

 * the load balancer forward requests to instances in the group and how instance
 * health is checked.
 */
 public String createTargetGroup(String protocol, int port, String vpcId, String
targetGroupName) {
 CreateTargetGroupRequest targetGroupRequest =
CreateTargetGroupRequest.builder()
 .healthCheckPath("/healthcheck")
 .healthCheckTimeoutSeconds(5)
 .port(port)
 .vpcId(vpcId)
 .name(targetGroupName)
 .protocol(protocol)
 .build();

 CreateTargetGroupResponse targetGroupResponse =
getLoadBalancerClient().createTargetGroup(targetGroupRequest);
 String targetGroupArn =
targetGroupResponse.targetGroups().get(0).targetGroupArn();
 String targetGroup =
targetGroupResponse.targetGroups().get(0).targetGroupName();
 System.out.println("The " + targetGroup + " was created with ARN" +
targetGroupArn);
 return targetGroupArn;
 }

```

- For API details, see [CreateTargetGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteLoadBalancer

The following code example shows how to use DeleteLoadBalancer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Deletes a load balancer.
```

```
public void deleteLoadBalancer(String lbName) {
 try {
 // Use a waiter to delete the Load Balancer.
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
 getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
 DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(res.loadBalancers().get(0).loadBalancerArn())
 .build();

 getLoadBalancerClient().deleteLoadBalancer(
 builder ->
 builder.loadBalancerArn(res.loadBalancers().get(0).loadBalancerArn()));
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
 loadBalancerWaiter
 .waitUntilLoadBalancersDeleted(request);
 waiterResponse.matched().response().ifPresent(System.out::println);

 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(lbName + " was deleted.");
}
```

- For API details, see [DeleteLoadBalancer](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteTargetGroup

The following code example shows how to use DeleteTargetGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Deletes the target group.
```

```
public void deleteTargetGroup(String targetGroupName) {
 try {
 DescribeTargetGroupsResponse res = getLoadBalancerClient()
 .describeTargetGroups(describe ->
describe.names(targetGroupName));
 getLoadBalancerClient()
 .deleteTargetGroup(builder ->
builder.targetGroupArn(res.targetGroups().get(0).targetGroupArn()));
 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(targetGroupName + " was deleted.");
}
```

- For API details, see [DeleteTargetGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeTargetHealth

The following code example shows how to use `DescribeTargetHealth`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Checks the health of the instances in the target group.
public List<TargetHealthDescription> checkTargetHealth(String targetGroupName) {
 DescribeTargetGroupsRequest targetGroupsRequest =
DescribeTargetGroupsRequest.builder()
 .names(targetGroupName)
 .build();

 DescribeTargetGroupsResponse tgResponse =
getLoadBalancerClient().describeTargetGroups(targetGroupsRequest);

 DescribeTargetHealthRequest healthRequest =
DescribeTargetHealthRequest.builder()
```

```
 .targetGroupArn(tgResponse.targetGroups().get(0).targetGroupArn())
 .build();

 DescribeTargetHealthResponse healthResponse =
getLoadBalancerClient().describeTargetHealth(healthRequest);
 return healthResponse.targetHealthDescriptions();
}
```

- For API details, see [DescribeTargetHealth](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build and manage a resilient service

The following code example shows how to create a load-balanced web service that returns book, movie, and song recommendations. The example shows how the service responds to failures, and how to restructure the service for more resilience when failures occur.

- Use an Amazon EC2 Auto Scaling group to create Amazon Elastic Compute Cloud (Amazon EC2) instances based on a launch template and to keep the number of instances in a specified range.
- Handle and distribute HTTP requests with Elastic Load Balancing.
- Monitor the health of instances in an Auto Scaling group and forward requests only to healthy instances.
- Run a Python web server on each EC2 instance to handle HTTP requests. The web server responds with recommendations and health checks.
- Simulate a recommendation service with an Amazon DynamoDB table.
- Control web server response to requests and health checks by updating AWS Systems Manager parameters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).



## Run the interactive scenario at a command prompt.

```

public class Main {

 public static final String fileName = "C:\\AWS\\resworkflow\\
\\recommendations.json"; // Modify file location.
 public static final String tableName = "doc-example-recommendation-service";
 public static final String startScript = "C:\\AWS\\resworkflow\\
\\server_startup_script.sh"; // Modify file location.
 public static final String policyFile = "C:\\AWS\\resworkflow\\
\\instance_policy.json"; // Modify file location.
 public static final String ssmJSON = "C:\\AWS\\resworkflow\\
\\ssm_only_policy.json"; // Modify file location.
 public static final String failureResponse = "doc-example-resilient-
architecture-failure-response";
 public static final String healthCheck = "doc-example-resilient-architecture-
health-check";
 public static final String templateName = "doc-example-resilience-template";
 public static final String roleName = "doc-example-resilience-role";
 public static final String policyName = "doc-example-resilience-pol";
 public static final String profileName = "doc-example-resilience-prof";

 public static final String badCredsProfileName = "doc-example-resilience-prof-
bc";

 public static final String targetGroupName = "doc-example-resilience-tg";
 public static final String autoScalingGroupName = "doc-example-resilience-
group";
 public static final String lbName = "doc-example-resilience-lb";
 public static final String protocol = "HTTP";
 public static final int port = 80;

 public static final String DASHES = new String(new char[80]).replace("\\0", "-");

 public static void main(String[] args) throws IOException, InterruptedException
 {
 Scanner in = new Scanner(System.in);
 Database database = new Database();
 AutoScaler autoScaler = new AutoScaler();
 LoadBalancer loadBalancer = new LoadBalancer();

 System.out.println(DASHES);
 System.out.println("Welcome to the demonstration of How to Build and Manage
a Resilient Service!");
 }
}

```

```

 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("A - SETUP THE RESOURCES");
 System.out.println("Press Enter when you're ready to start deploying
resources.");
 in.nextLine();
 deploy(loadBalancer);
 System.out.println(DASHES);
 System.out.println(DASHES);
 System.out.println("B - DEMO THE RESILIENCE FUNCTIONALITY");
 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 demo(loadBalancer);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("C - DELETE THE RESOURCES");
 System.out.println("""
 This concludes the demo of how to build and manage a resilient
service.

 To keep things tidy and to avoid unwanted charges on your account,
we can clean up all AWS resources
 that were created for this demo.
 """);

 System.out.println("\n Do you want to delete the resources (y/n)? ");
 String userInput = in.nextLine().trim().toLowerCase(); // Capture user input

 if (userInput.equals("y")) {
 // Delete resources here
 deleteResources(loadBalancer, autoScaler, database);
 System.out.println("Resources deleted.");
 } else {
 System.out.println("""
 Okay, we'll leave the resources intact.
 Don't forget to delete them when you're done with them or you
might incur unexpected charges.
 """);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The example has completed. ");

```

```

 System.out.println("\n Thanks for watching!");
 System.out.println(DASHES);
 }

 // Deletes the AWS resources used in this example.
 private static void deleteResources(LoadBalancer loadBalancer, AutoScaler
autoScaler, Database database)
 throws IOException, InterruptedException {
 loadBalancer.deleteLoadBalancer(lbName);
 System.out.println("**** Wait 30 secs for resource to be deleted");
 TimeUnit.SECONDS.sleep(30);
 loadBalancer.deleteTargetGroup(targetGroupName);
 autoScaler.deleteAutoScaleGroup(autoScalingGroupName);
 autoScaler.deleteRolesPolicies(policyName, roleName, profileName);
 autoScaler.deleteTemplate(templateName);
 database.deleteTable(tableName);
 }

 private static void deploy(LoadBalancer loadBalancer) throws
InterruptedException, IOException {
 Scanner in = new Scanner(System.in);
 System.out.println(
 """
 For this demo, we'll use the AWS SDK for Java (v2) to create
several AWS resources
 to set up a load-balanced web service endpoint and explore
some ways to make it resilient
 against various kinds of failures.

 Some of the resources create by this demo are:
 \t* A DynamoDB table that the web service depends on to
provide book, movie, and song recommendations.
 \t* An EC2 launch template that defines EC2 instances that
each contain a Python web server.
 \t* An EC2 Auto Scaling group that manages EC2 instances
across several Availability Zones.
 \t* An Elastic Load Balancing (ELB) load balancer that
targets the Auto Scaling group to distribute requests.
 """);

 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 System.out.println(DASHES);

```

```

 System.out.println(DASHES);
 System.out.println("Creating and populating a DynamoDB table named " +
tableName);
 Database database = new Database();
 database.createTable(tableName, fileName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an EC2 launch template that runs '{startup_script}' when an
instance starts.
 This script starts a Python web server defined in the `server.py`
script. The web server
 listens to HTTP requests on port 80 and responds to requests to '/'
and to '/healthcheck'.
 For demo purposes, this server is run as the root user. In
production, the best practice is to
 run a web server, such as Apache, with least-privileged credentials.

 The template also defines an IAM policy that each instance uses to
assume a role that grants
 permissions to access the DynamoDB recommendation table and Systems
Manager parameters
 that control the flow of the demo.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 templateCreator.createTemplate(policyFile, policyName, profileName,
startScript, templateName, roleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(
 "Creating an EC2 Auto Scaling group that maintains three EC2
instances, each in a different Availability Zone.");
 System.out.println("**** Wait 30 secs for the VPC to be created");
 TimeUnit.SECONDS.sleep(30);
 AutoScaler autoScaler = new AutoScaler();
 String[] zones = autoScaler.createGroup(3, templateName,
autoScalingGroupName);

 System.out.println("""
 At this point, you have EC2 instances created. Once each instance
starts, it listens for

```

HTTP requests. You can see these instances in the console or continue with the demo.

```
 Press Enter when you're ready to continue.
 "");
```

```
in.nextLine();
System.out.println(DASHES);
```

```
System.out.println(DASHES);
System.out.println("Creating variables that control the flow of the demo.");
ParameterHelper paramHelper = new ParameterHelper();
paramHelper.reset();
System.out.println(DASHES);
```

```
System.out.println(DASHES);
System.out.println("""
```

Creating an Elastic Load Balancing target group and load balancer. The target group defines how the load balancer connects to instances. The load balancer provides a single endpoint where clients connect and dispatches requests to instances in the group.

```
 """);
```

```
 String vpcId = autoScaler.getDefaultVPC();
 List<Subnet> subnets = autoScaler.getSubnets(vpcId, zones);
 System.out.println("You have retrieved a list with " + subnets.size() + "
subnets");
 String targetGroupArn = loadBalancer.createTargetGroup(protocol, port,
vpcId, targetGroupName);
 String elbDnsName = loadBalancer.createLoadBalancer(subnets, targetGroupArn,
lbName, port, protocol);
 autoScaler.attachLoadBalancerTargetGroup(autoScalingGroupName,
targetGroupArn);
 System.out.println("Verifying access to the load balancer endpoint...");
 boolean wasSuccessful = loadBalancer.verifyLoadBalancerEndpoint(elbDnsName);
 if (!wasSuccessful) {
 System.out.println("Couldn't connect to the load balancer, verifying
that the port is open...");
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to "http://checkip.amazonaws.com"
 HttpGet httpGet = new HttpGet("http://checkip.amazonaws.com");
 try {
```

```

 // Execute the request and get the response
 HttpResponse response = httpClient.execute(httpGet);

 // Read the response content.
 String ipAddress =
IOUtils.toString(response.getEntity().getContent(), StandardCharsets.UTF_8).trim();

 // Print the public IP address.
 System.out.println("Public IP Address: " + ipAddress);
 GroupInfo groupInfo = autoScaler.verifyInboundPort(vpcId, port,
ipAddress);
 if (!groupInfo.isPortOpen()) {
 System.out.println("""
 For this example to work, the default security group for
your default VPC must
 allow access from this computer. You can either add it
automatically from this
 example or add it yourself using the AWS Management
Console.
 """);

 System.out.println(
 "Do you want to add a rule to security group " +
groupInfo.getGroupName() + " to allow");
 System.out.println("inbound traffic on port " + port + " from
your computer's IP address (y/n) ");
 String ans = in.nextLine();
 if ("y".equalsIgnoreCase(ans)) {
 autoScaler.openInboundPort(groupInfo.getGroupName(),
String.valueOf(port), ipAddress);
 System.out.println("Security group rule added.");
 } else {
 System.out.println("No security group rule added.");
 }
 }

 } catch (AutoScalingException e) {
 e.printStackTrace();
 }
 } else if (wasSuccessful) {
 System.out.println("Your load balancer is ready. You can access it by
browsing to:");
 System.out.println("\t http://" + elbDnsName);
 } else {

```

```

 System.out.println("Couldn't get a successful response from the load
balancer endpoint. Troubleshoot by");
 System.out.println("manually verifying that your VPC and security group
are configured correctly and that");
 System.out.println("you can successfully make a GET request to the load
balancer.");
 }

 System.out.println("Press Enter when you're ready to continue with the
demo.");
 in.nextLine();
}

// A method that controls the demo part of the Java program.
public static void demo(LoadBalancer loadBalancer) throws IOException,
InterruptedException {
 ParameterHelper paramHelper = new ParameterHelper();
 System.out.println("Read the ssm_only_policy.json file");
 String ssmOnlyPolicy = readFileAsString(ssmJSON);

 System.out.println("Resetting parameters to starting values for demo.");
 paramHelper.reset();

 System.out.println(
 """
 This part of the demonstration shows how to toggle
different parts of the system
 to create situations where the web service fails, and shows
how using a resilient
 architecture can keep the web service running in spite of
these failures.

 At the start, the load balancer endpoint returns
recommendations and reports that all targets are healthy.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 """
 The web service running on the EC2 instances gets
recommendations by querying a DynamoDB table.
 The table name is contained in a Systems Manager parameter
named self.param_helper.table.
 """
);
}

```

```

 To simulate a failure of the recommendation service, let's
 set this parameter to name a non-existent table.
 """);
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 System.out.println(
 ""
 \nNow, sending a GET request to the load balancer endpoint
returns a failure code. But, the service reports as
 healthy to the load balancer because shallow health checks
don't check for failure of the recommendation service.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 ""
 Instead of failing when the recommendation service fails,
the web service can return a static response.
 While this is not a perfect solution, it presents the
customer with a somewhat better experience than failure.
 """);
 paramHelper.put(paramHelper.failureResponse, "static");

 System.out.println("""
 Now, sending a GET request to the load balancer endpoint returns a
static response.
 The service still reports as healthy because health checks are still
shallow.
 """);
 demoChoices(loadBalancer);

 System.out.println("Let's reinstate the recommendation service.");
 paramHelper.put(paramHelper.tableName, paramHelper.dyntable);

 System.out.println("""
 Let's also substitute bad credentials for one of the instances in
the target group so that it can't
 access the DynamoDB recommendation table. We will get an instance id
value.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 AutoScaler autoScaler = new AutoScaler();

```



```

 // Create a new instance profile based on badCredsProfileName.
 templateCreator.createInstanceProfile(policyFile, policyName,
 badCredsProfileName, roleName);
 String badInstanceId = autoScaler.getBadInstance(autoScalingGroupName);
 System.out.println("The bad instance id values used for this demo is " +
 badInstanceId);

 String profileAssociationId = autoScaler.getInstanceProfile(badInstanceId);
 System.out.println("The association Id value is " + profileAssociationId);
 System.out.println("Replacing the profile for instance " + badInstanceId
 + " with a profile that contains bad credentials");
 autoScaler.replaceInstanceProfile(badInstanceId, badCredsProfileName,
 profileAssociationId);

 System.out.println(
 """
 Now, sending a GET request to the load balancer endpoint
returns either a recommendation or a static response,
 depending on which instance is selected by the load
balancer.

 """);

 demoChoices(loadBalancer);

 System.out.println("""
 Let's implement a deep health check. For this demo, a deep health
check tests whether
 the web service can access the DynamoDB table that it depends on for
recommendations. Note that
 the deep health check is only for ELB routing and not for Auto
Scaling instance health.
 This kind of deep health check is not recommended for Auto Scaling
instance health, because it
 risks accidental termination of all instances in the Auto Scaling
group when a dependent service fails.
 """);

 System.out.println("""
 By implementing deep health checks, the load balancer can detect
when one of the instances is failing
 and take that instance out of rotation.
 """);

 paramHelper.put(paramHelper.healthCheck, "deep");

```

```
 System.out.println("""
 Now, checking target health indicates that the instance with bad
credentials
 is unhealthy. Note that it might take a minute or two for the load
balancer to detect the unhealthy
 instance. Sending a GET request to the load balancer endpoint always
returns a recommendation, because
 the load balancer takes unhealthy instances out of its rotation.
 """);

 demoChoices(loadBalancer);

 System.out.println(
 """
 Because the instances in this demo are controlled by an auto
scaler, the simplest way to fix an unhealthy
 instance is to terminate it and let the auto scaler start a
new instance to replace it.
 """);
 autoScaler.terminateInstance(badInstanceId);

 System.out.println("""
 Even while the instance is terminating and the new instance is
starting, sending a GET
 request to the web service continues to get a successful
recommendation response because
 the load balancer routes requests to the healthy instances. After
the replacement instance
 starts and reports as healthy, it is included in the load balancing
rotation.

 Note that terminating and replacing an instance typically takes
several minutes, during which time you
 can see the changing health check status until the new instance is
running and healthy.
 """);

 demoChoices(loadBalancer);
 System.out.println(
 "If the recommendation service fails now, deep health checks mean
all instances report as unhealthy.");
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 demoChoices(loadBalancer);
```

```

 paramHelper.reset();
 }

 public static void demoChoices(LoadBalancer loadBalancer) throws IOException,
 InterruptedException {
 String[] actions = {
 "Send a GET request to the load balancer endpoint.",
 "Check the health of load balancer targets.",
 "Go to the next part of the demo."
 };
 Scanner scanner = new Scanner(System.in);

 while (true) {
 System.out.println("-".repeat(88));
 System.out.println("See the current state of the service by selecting
one of the following choices:");
 for (int i = 0; i < actions.length; i++) {
 System.out.println(i + ": " + actions[i]);
 }

 try {
 System.out.print("\nWhich action would you like to take? ");
 int choice = scanner.nextInt();
 System.out.println("-".repeat(88));

 switch (choice) {
 case 0 -> {
 System.out.println("Request:\n");
 System.out.println("GET http://" +
loadBalancer.getEndpoint(lbName));
 CloseableHttpClient httpClient =
HttpClients.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" +
loadBalancer.getEndpoint(lbName));

 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);

 // Display the JSON response
 BufferedReader reader = new BufferedReader(

```

```

 new
 InputStreamReader(response.getEntity().getContent()));
 StringBuilder jsonResponse = new StringBuilder();
 String line;
 while ((line = reader.readLine()) != null) {
 jsonResponse.append(line);
 }
 reader.close();

 // Print the formatted JSON response.
 System.out.println("Full Response:\n");
 System.out.println(jsonResponse.toString());

 // Close the HTTP client.
 httpClient.close();
 }
 case 1 -> {
 System.out.println("\nChecking the health of load balancer
targets:\n");
 List<TargetHealthDescription> health =
loadBalancer.checkTargetHealth(targetGroupName);
 for (TargetHealthDescription target : health) {
 System.out.printf("\tTarget %s on port %d is %s\n",
target.target().id(),
 target.target().port(),
target.targetHealth().stateAsString());
 }
 System.out.println("""
 Note that it can take a minute or two for the health
check to update
 after changes are made.
 """);
 }
 case 2 -> {
 System.out.println("\nOkay, let's move on.");
 System.out.println("-".repeat(88));
 return; // Exit the method when choice is 2
 }
 default -> System.out.println("You must choose a value between
0-2. Please select again.");
}

} catch (java.util.InputMismatchException e) {

```

```

 System.out.println("Invalid input. Please select again.");
 scanner.nextLine(); // Clear the input buffer.
 }
}

public static String readFileAsString(String filePath) throws IOException {
 byte[] bytes = Files.readAllBytes(Paths.get(filePath));
 return new String(bytes);
}
}

```

Create a class that wraps Auto Scaling and Amazon EC2 actions.

```

public class AutoScaler {

 private static Ec2Client ec2Client;
 private static AutoScalingClient autoScalingClient;
 private static IamClient iamClient;

 private static SsmClient ssmClient;

 private IamClient getIAMClient() {
 if (iamClient == null) {
 iamClient = IamClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return iamClient;
 }

 private SsmClient getSSMClient() {
 if (ssmClient == null) {
 ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ssmClient;
 }

 private Ec2Client getEc2Client() {
 if (ec2Client == null) {

```

```

 ec2Client = Ec2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ec2Client;
}

private AutoScalingClient getAutoScalingClient() {
 if (autoScalingClient == null) {
 autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return autoScalingClient;
}

/**
 * Terminates instances in an EC2 Auto Scaling group. After an instance is
 * terminated, it can no longer be accessed.
 */
public void terminateInstance(String instanceId) {
 TerminateInstanceInAutoScalingGroupRequest terminateInstanceRequest =
 TerminateInstanceInAutoScalingGroupRequest
 .builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 getAutoScalingClient().terminateInstanceInAutoScalingGroup(terminateInstanceRequest);
 System.out.format("Terminated instance %s.", instanceId);
}

/**
 * Replaces the profile associated with a running instance. After the profile is
 * replaced, the instance is rebooted to ensure that it uses the new profile.
 * When
 * the instance is ready, Systems Manager is used to restart the Python web
 * server.
 */
public void replaceInstanceProfile(String instanceId, String
newInstanceProfileName, String profileAssociationId)
 throws InterruptedException {
 // Create an IAM instance profile specification.

```

```

 software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
iamInstanceProfile =
software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
 .builder()
 .name(newInstanceProfileName) // Make sure 'newInstanceProfileName'
is a valid IAM Instance Profile
 // name.
 .build();

// Replace the IAM instance profile association for the EC2 instance.
ReplaceIamInstanceProfileAssociationRequest replaceRequest =
ReplaceIamInstanceProfileAssociationRequest
 .builder()
 .iamInstanceProfile(iamInstanceProfile)
 .associationId(profileAssociationId) // Make sure
'profileAssociationId' is a valid association ID.
 .build();

try {
 getEc2Client().replaceIamInstanceProfileAssociation(replaceRequest);
 // Handle the response as needed.
} catch (Ec2Exception e) {
 // Handle exceptions, log, or report the error.
 System.err.println("Error: " + e.getMessage());
}

System.out.format("Replaced instance profile for association %s with profile
%s.", profileAssociationId,
 newInstanceProfileName);
TimeUnit.SECONDS.sleep(15);
boolean instReady = false;
int tries = 0;

// Reboot after 60 seconds
while (!instReady) {
 if (tries % 6 == 0) {
 getEc2Client().rebootInstances(RebootInstancesRequest.builder()
 .instanceIds(instanceId)
 .build());
 System.out.println("Rebooting instance " + instanceId + " and
waiting for it to be ready.");
 }
 tries++;
 try {
 TimeUnit.SECONDS.sleep(10);
 }
}

```

```

 } catch (InterruptedException e) {
 e.printStackTrace();
 }

 DescribeInstanceInformationResponse informationResponse =
getSSMClient().describeInstanceInformation();
 List<InstanceInformation> instanceInformationList =
informationResponse.getInstanceInformationList();
 for (InstanceInformation info : instanceInformationList) {
 if (info.getInstanceId().equals(instanceId)) {
 instReady = true;
 break;
 }
 }
}

SendCommandRequest sendCommandRequest = SendCommandRequest.builder()
 .instanceIds(instanceId)
 .documentName("AWS-RunShellScript")
 .parameters(Collections.singletonMap("commands",
 Collections.singletonList("cd / && sudo python3 server.py
80")))
 .build();

getSSMClient().sendCommand(sendCommandRequest);
System.out.println("Restarted the Python web server on instance " +
instanceId + ".");
}

public void openInboundPort(String secGroupId, String port, String ipAddress) {
 AuthorizeSecurityGroupIngressRequest ingressRequest =
AuthorizeSecurityGroupIngressRequest.builder()
 .groupName(secGroupId)
 .cidrIp(ipAddress)
 .fromPort(Integer.parseInt(port))
 .build();

 getEc2Client().authorizeSecurityGroupIngress(ingressRequest);
 System.out.format("Authorized ingress to %s on port %s from %s.",
secGroupId, port, ipAddress);
}

/**
 * Detaches a role from an instance profile, detaches policies from the role,

```



```

 * and deletes all the resources.
 */
 public void deleteInstanceProfile(String roleName, String profileName) {
 try {
 software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
 getInstanceProfileRequest =
 software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
 .builder()
 .instanceProfileName(profileName)
 .build();

 GetInstanceProfileResponse response =
 getIAMClient().getInstanceProfile(getInstanceProfileRequest);
 String name = response.instanceProfile().instanceProfileName();
 System.out.println(name);

 RemoveRoleFromInstanceProfileRequest profileRequest =
 RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .roleName(roleName)
 .build();

 getIAMClient().removeRoleFromInstanceProfile(profileRequest);
 DeleteInstanceProfileRequest deleteInstanceProfileRequest =
 DeleteInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .build();

 getIAMClient().deleteInstanceProfile(deleteInstanceProfileRequest);
 System.out.println("Deleted instance profile " + profileName);

 DeleteRoleRequest deleteRoleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 // List attached role policies.
 ListAttachedRolePoliciesResponse rolesResponse = getIAMClient()
 .listAttachedRolePolicies(role -> role.roleName(roleName));
 List<AttachedPolicy> attachedPolicies =
 rolesResponse.attachedPolicies();
 for (AttachedPolicy attachedPolicy : attachedPolicies) {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(attachedPolicy.policyArn())

```

```

 .build();

 getIAMClient().detachRolePolicy(request);
 System.out.println("Detached and deleted policy " +
attachedPolicy.policyName());
 }

 getIAMClient().deleteRole(deleteRoleRequest);
 System.out.println("Instance profile and role deleted.");

 } catch (IamException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public void deleteTemplate(String templateName) {
 getEc2Client().deleteLaunchTemplate(name ->
name.launchTemplateName(templateName));
 System.out.format(templateName + " was deleted.");
}

public void deleteAutoScaleGroup(String groupName) {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

 getAutoScalingClient().deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println(groupName + " was deleted.");
}

/*
 * Verify the default security group of the specified VPC allows ingress from
 * this
 * computer. This can be done by allowing ingress from this computer's IP
 * address. In some situations, such as connecting from a corporate network, you
 * must instead specify a prefix list ID. You can also temporarily open the port
 * to
 * any IP address while running this example. If you do, be sure to remove
 * public
 * access when you're done.

```

```

 *
 */
 public GroupInfo verifyInboundPort(String VPC, int port, String ipAddress) {
 boolean portIsOpen = false;
 GroupInfo groupInfo = new GroupInfo();
 try {
 Filter filter = Filter.builder()
 .name("group-name")
 .values("default")
 .build();

 Filter filter1 = Filter.builder()
 .name("vpc-id")
 .values(VPC)
 .build();

 DescribeSecurityGroupsRequest securityGroupsRequest =
 DescribeSecurityGroupsRequest.builder()
 .filters(filter, filter1)
 .build();

 DescribeSecurityGroupsResponse securityGroupsResponse = getEc2Client()
 .describeSecurityGroups(securityGroupsRequest);
 String securityGroup =
 securityGroupsResponse.securityGroups().get(0).groupName();
 groupInfo.setGroupName(securityGroup);

 for (SecurityGroup secGroup : securityGroupsResponse.securityGroups()) {
 System.out.println("Found security group: " + secGroup.groupId());

 for (IpPermission ipPermission : secGroup.ipPermissions()) {
 if (ipPermission.fromPort() == port) {
 System.out.println("Found inbound rule: " + ipPermission);
 for (IpRange ipRange : ipPermission.ipRanges()) {
 String cidrIp = ipRange.cidrIp();
 if (cidrIp.startsWith(ipAddress) ||
 cidrIp.equals("0.0.0.0/0")) {
 System.out.println(cidrIp + " is applicable");
 portIsOpen = true;
 }
 }
 }

 if (!ipPermission.prefixListIds().isEmpty()) {
 System.out.println("Prefix lList is applicable");
 }
 }
 }
 }
 }

```

```

 portIsOpen = true;
 }

 if (!portIsOpen) {
 System.out
 .println("The inbound rule does not appear to be
open to either this computer's IP,"
 + " all IP addresses (0.0.0.0/0), or to
a prefix list ID.");
 } else {
 break;
 }
}

}

} catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
}

groupInfo.setPortOpen(portIsOpen);
return groupInfo;
}

/*
 * Attaches an Elastic Load Balancing (ELB) target group to this EC2 Auto
 * Scaling group.
 * The target group specifies how the load balancer forward requests to the
 * instances
 * in the group.
 */
public void attachLoadBalancerTargetGroup(String asGroupName, String
targetGroupARN) {
 try {
 AttachLoadBalancerTargetGroupsRequest targetGroupsRequest =
AttachLoadBalancerTargetGroupsRequest.builder()
 .autoScalingGroupName(asGroupName)
 .targetGroupARNs(targetGroupARN)
 .build();

getAutoScalingClient().attachLoadBalancerTargetGroups(targetGroupsRequest);
 System.out.println("Attached load balancer to " + asGroupName);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Creates an EC2 Auto Scaling group with the specified size.
public String[] createGroup(int groupSize, String templateName, String
autoScalingGroupName) {

 // Get availability zones.
 software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
zonesRequest =
software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
 .builder()
 .build();

 DescribeAvailabilityZonesResponse zonesResponse =
getEc2Client().describeAvailabilityZones(zonesRequest);
 List<String> availabilityZoneNames =
zonesResponse.availabilityZones().stream()

.map(software.amazon.awssdk.services.ec2.model.AvailabilityZone::zoneName)
 .collect(Collectors.toList());

 String availabilityZones = String.join(",", availabilityZoneNames);
 LaunchTemplateSpecification specification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(templateName)
 .version("$Default")
 .build();

 String[] zones = availabilityZones.split(",");
 CreateAutoScalingGroupRequest groupRequest =
CreateAutoScalingGroupRequest.builder()
 .launchTemplate(specification)
 .availabilityZones(zones)
 .maxSize(groupSize)
 .minSize(groupSize)
 .autoScalingGroupName(autoScalingGroupName)
 .build();

 try {
 getAutoScalingClient().createAutoScalingGroup(groupRequest);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Created an EC2 Auto Scaling group named " +
autoScalingGroupName);
 return zones;
 }

 public String getDefaultVPC() {
 // Define the filter.
 Filter defaultFilter = Filter.builder()
 .name("is-default")
 .values("true")
 .build();

 software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest request =
software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest
 .builder()
 .filters(defaultFilter)
 .build();

 DescribeVpcsResponse response = getEc2Client().describeVpcs(request);
 return response.vpcs().get(0).vpcId();
 }

 // Gets the default subnets in a VPC for a specified list of Availability Zones.
 public List<Subnet> getSubnets(String vpcId, String[] availabilityZones) {
 List<Subnet> subnets = null;
 Filter vpcFilter = Filter.builder()
 .name("vpc-id")
 .values(vpcId)
 .build();

 Filter azFilter = Filter.builder()
 .name("availability-zone")
 .values(availabilityZones)
 .build();

 Filter defaultForAZ = Filter.builder()
 .name("default-for-az")
 .values("true")
 .build();

```

```

 DescribeSubnetsRequest request = DescribeSubnetsRequest.builder()
 .filters(vpcFilter, azFilter, defaultForAZ)
 .build();

 DescribeSubnetsResponse response = getEc2Client().describeSubnets(request);
 subnets = response.subnets();
 return subnets;
 }

 // Gets data about the instances in the EC2 Auto Scaling group.
 public String getBadInstance(String groupName) {
 DescribeAutoScalingGroupsRequest request =
DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response =
getAutoScalingClient().describeAutoScalingGroups(request);
 AutoScalingGroup autoScalingGroup = response.autoScalingGroups().get(0);
 List<String> instanceIds = autoScalingGroup.instances().stream()
 .map(instance -> instance.instanceId())
 .collect(Collectors.toList());

 String[] instanceIdArray = instanceIds.toArray(new String[0]);
 for (String instanceId : instanceIdArray) {
 System.out.println("Instance ID: " + instanceId);
 return instanceId;
 }
 return "";
 }

 // Gets data about the profile associated with an instance.
 public String getInstanceProfile(String instanceId) {
 Filter filter = Filter.builder()
 .name("instance-id")
 .values(instanceId)
 .build();

 DescribeIamInstanceProfileAssociationsRequest associationsRequest =
DescribeIamInstanceProfileAssociationsRequest
 .builder()
 .filters(filter)
 .build();
 }

```

```

 DescribeIamInstanceProfileAssociationsResponse response = getEc2Client()
 .describeIamInstanceProfileAssociations(associationsRequest);
 return response.iamInstanceProfileAssociations().get(0).associationId();
 }

 public void deleteRolesPolicies(String policyName, String roleName, String
InstanceProfile) {
 ListPoliciesRequest listPoliciesRequest =
ListPoliciesRequest.builder().build();
 ListPoliciesResponse listPoliciesResponse =
getIAMClient().listPolicies(listPoliciesRequest);
 for (Policy policy : listPoliciesResponse.policies()) {
 if (policy.policyName().equals(policyName)) {
 // List the entities (users, groups, roles) that are attached to the
policy.

software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
listEntitiesRequest =
software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
 .builder()
 .policyArn(policy.arn())
 .build();
 ListEntitiesForPolicyResponse listEntitiesResponse = iamClient
 .listEntitiesForPolicy(listEntitiesRequest);
 if (!listEntitiesResponse.policyGroups().isEmpty() || !
listEntitiesResponse.policyUsers().isEmpty()
 || !listEntitiesResponse.policyRoles().isEmpty()) {
 // Detach the policy from any entities it is attached to.
 DetachRolePolicyRequest detachPolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy.arn())
 .roleName(roleName) // Specify the name of the IAM role
 .build();

 getIAMClient().detachRolePolicy(detachPolicyRequest);
 System.out.println("Policy detached from entities.");
 }

 // Now, you can delete the policy.
 DeletePolicyRequest deletePolicyRequest =
DeletePolicyRequest.builder()
 .policyArn(policy.arn())
 .build();

```



```

 getIAMClient().deletePolicy(deletePolicyRequest);
 System.out.println("Policy deleted successfully.");
 break;
 }
}

// List the roles associated with the instance profile
ListInstanceProfilesForRoleRequest listRolesRequest =
ListInstanceProfilesForRoleRequest.builder()
 .roleName(roleName)
 .build();

// Detach the roles from the instance profile
ListInstanceProfilesForRoleResponse listRolesResponse =
iamClient.listInstanceProfilesForRole(listRolesRequest);
for (software.amazon.awssdk.services.iam.model.InstanceProfile profile :
listRolesResponse.instanceProfiles()) {
 RemoveRoleFromInstanceProfileRequest removeRoleRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .roleName(roleName) // Remove the extra dot here
 .build();

 getIAMClient().removeRoleFromInstanceProfile(removeRoleRequest);
 System.out.println("Role " + roleName + " removed from instance profile
" + InstanceProfile);
}

// Delete the instance profile after removing all roles
DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .build();

getIAMClient().deleteInstanceProfile(r ->
r.instanceProfileName(InstanceProfile));
System.out.println(InstanceProfile + " Deleted");
System.out.println("All roles and policies are deleted.");
}
}

```

## Create a class that wraps Elastic Load Balancing actions.

```
public class LoadBalancer {
 public ElasticLoadBalancingV2Client elasticLoadBalancingV2Client;

 public ElasticLoadBalancingV2Client getLoadBalancerClient() {
 if (elasticLoadBalancingV2Client == null) {
 elasticLoadBalancingV2Client = ElasticLoadBalancingV2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }

 return elasticLoadBalancingV2Client;
 }

 // Checks the health of the instances in the target group.
 public List<TargetHealthDescription> checkTargetHealth(String targetGroupName) {
 DescribeTargetGroupsRequest targetGroupsRequest =
 DescribeTargetGroupsRequest.builder()
 .names(targetGroupName)
 .build();

 DescribeTargetGroupsResponse tgResponse =
 getLoadBalancerClient().describeTargetGroups(targetGroupsRequest);

 DescribeTargetHealthRequest healthRequest =
 DescribeTargetHealthRequest.builder()
 .targetGroupArn(tgResponse.targetGroups().get(0).targetGroupArn())
 .build();

 DescribeTargetHealthResponse healthResponse =
 getLoadBalancerClient().describeTargetHealth(healthRequest);
 return healthResponse.targetHealthDescriptions();
 }

 // Gets the HTTP endpoint of the load balancer.
 public String getEndpoint(String lbName) {
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 return res.loadBalancers().get(0).dnsName();
 }

 // Deletes a load balancer.
 public void deleteLoadBalancer(String lbName) {
```

```

 try {
 // Use a waiter to delete the Load Balancer.
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(res.loadBalancers().get(0).loadBalancerArn())
 .build();

 getLoadBalancerClient().deleteLoadBalancer(
 builder ->
builder.loadBalancerArn(res.loadBalancers().get(0).loadBalancerArn()));
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancersDeleted(request);
 waiterResponse.matched().response().ifPresent(System.out::println);

 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(lbName + " was deleted.");
 }

 // Deletes the target group.
 public void deleteTargetGroup(String targetGroupName) {
 try {
 DescribeTargetGroupsResponse res = getLoadBalancerClient()
 .describeTargetGroups(describe ->
describe.names(targetGroupName));
 getLoadBalancerClient()
 .deleteTargetGroup(builder ->
builder.targetGroupArn(res.targetGroups().get(0).targetGroupArn()));
 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(targetGroupName + " was deleted.");
 }

 // Verify this computer can successfully send a GET request to the load balancer
 // endpoint.
 public boolean verifyLoadBalancerEndpoint(String elbDnsName) throws IOException,
 InterruptedException {

```

```

 boolean success = false;
 int retries = 3;
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" + elbDnsName);
 try {
 while ((!success) && (retries > 0)) {
 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);
 if (statusCode == 200) {
 success = true;
 } else {
 retries--;
 System.out.println("Got connection error from load balancer endpoint, retrying...");
 TimeUnit.SECONDS.sleep(15);
 }
 }

 } catch (org.apache.http.conn.HttpHostConnectException e) {
 System.out.println(e.getMessage());
 }

 System.out.println("Status.." + success);
 return success;
 }

 /**
 * Creates an Elastic Load Balancing target group. The target group specifies
 * how
 * the load balancer forward requests to instances in the group and how instance
 * health is checked.
 */
 public String createTargetGroup(String protocol, int port, String vpcId, String
targetGroupName) {
 CreateTargetGroupRequest targetGroupRequest =
CreateTargetGroupRequest.builder()
 .healthCheckPath("/healthcheck")
 .healthCheckTimeoutSeconds(5)
 .port(port)
 .vpcId(vpcId)

```

```

 .name(targetGroupName)
 .protocol(protocol)
 .build();

 CreateTargetGroupResponse targetGroupResponse =
getLoadBalancerClient().createTargetGroup(targetGroupRequest);
 String targetGroupArn =
targetGroupResponse.targetGroups().get(0).targetGroupArn();
 String targetGroup =
targetGroupResponse.targetGroups().get(0).targetGroupName();
 System.out.println("The " + targetGroup + " was created with ARN" +
targetGroupArn);
 return targetGroupArn;
}

/*
 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */
public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
String lbName, int port,
 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()

```

```

 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("*** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.
 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

.loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}
}

```

Create a class that uses DynamoDB to simulate a recommendation service.

```
public class Database {

 private static DynamoDbClient dynamoDbClient;

 public static DynamoDbClient getDynamoDbClient() {
 if (dynamoDbClient == null) {
 dynamoDbClient = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return dynamoDbClient;
 }

 // Checks to see if the Amazon DynamoDB table exists.
 private boolean doesTableExist(String tableName) {
 try {
 // Describe the table and catch any exceptions.
 DescribeTableRequest describeTableRequest =
 DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 getDynamoDbClient().describeTable(describeTableRequest);
 System.out.println("Table '" + tableName + "' exists.");
 return true;

 } catch (ResourceNotFoundException e) {
 System.out.println("Table '" + tableName + "' does not exist.");
 } catch (DynamoDbException e) {
 System.err.println("Error checking table existence: " + e.getMessage());
 }
 return false;
 }

 /**
 * Creates a DynamoDB table to use a recommendation service. The table has a
 * hash key named 'MediaType' that defines the type of media recommended, such
 * as
 * Book or Movie, and a range key named 'ItemId' that, combined with the

```

```

 * MediaType,
 * forms a unique identifier for the recommended item.
 */
public void createTable(String tableName, String fileName) throws IOException {
 // First check to see if the table exists.
 boolean doesExist = doesTableExist(tableName);
 if (!doesExist) {
 DynamoDbWaiter dbWaiter = getDynamoDbClient().waiter();
 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("MediaType")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("ItemId")
 .attributeType(ScalarAttributeType.N)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("MediaType")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("ItemId")
 .keyType(KeyType.RANGE)
 .build())
 .provisionedThroughput(
 ProvisionedThroughput.builder()
 .readCapacityUnits(5L)
 .writeCapacityUnits(5L)
 .build())
 .build();

 getDynamoDbClient().createTable(createTableRequest);
 System.out.println("Creating table " + tableName + "...");

 // Wait until the Amazon DynamoDB table is created.
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();
 }
}

```



```

 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Table " + tableName + " created.");

 // Add records to the table.
 populateTable(fileName, tableName);
 }
}

public void deleteTable(String tableName) {
 getDynamoDbClient().deleteTable(table -> table.tableName(tableName));
 System.out.println("Table " + tableName + " deleted.");
}

// Populates the table with data located in a JSON file using the DynamoDB
// enhanced client.
public void populateTable(String fileName, String tableName) throws IOException
{
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(getDynamoDbClient())
 .build();
 ObjectMapper objectMapper = new ObjectMapper();
 File jsonFile = new File(fileName);
 JsonNode rootNode = objectMapper.readTree(jsonFile);

 DynamoDbTable<Recommendation> mappedTable = enhancedClient.table(tableName,
 TableSchema.fromBean(Recommendation.class));
 for (JsonNode currentNode : rootNode) {
 String mediaType = currentNode.path("MediaType").path("S").asText();
 int itemId = currentNode.path("ItemId").path("N").asInt();
 String title = currentNode.path("Title").path("S").asText();
 String creator = currentNode.path("Creator").path("S").asText();

 // Create a Recommendation object and set its properties.
 Recommendation rec = new Recommendation();
 rec.setMediaType(mediaType);
 rec.setItemId(itemId);
 rec.setTitle(title);
 rec.setCreator(creator);

 // Put the item into the DynamoDB table.
 mappedTable.putItem(rec); // Add the Recommendation to the list.
 }
}

```

```
 System.out.println("Added all records to the " + tableName);
 }
}
```

Create a class that wraps Systems Manager actions.

```
public class ParameterHelper {

 String tableName = "doc-example-resilient-architecture-table";
 String dyntable = "doc-example-recommendation-service";
 String failureResponse = "doc-example-resilient-architecture-failure-response";
 String healthCheck = "doc-example-resilient-architecture-health-check";

 public void reset() {
 put(dyntable, tableName);
 put(failureResponse, "none");
 put(healthCheck, "shallow");
 }

 public void put(String name, String value) {
 SsmClient ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();

 PutParameterRequest parameterRequest = PutParameterRequest.builder()
 .name(name)
 .value(value)
 .overwrite(true)
 .type("String")
 .build();

 ssmClient.putParameter(parameterRequest);
 System.out.printf("Setting demo parameter %s to '%s'.", name, value);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AttachLoadBalancerTargetGroups](#)
  - [CreateAutoScalingGroup](#)
  - [CreateInstanceProfile](#)

- [CreateLaunchTemplate](#)
- [CreateListener](#)
- [CreateLoadBalancer](#)
- [CreateTargetGroup](#)
- [DeleteAutoScalingGroup](#)
- [DeleteInstanceProfile](#)
- [DeleteLaunchTemplate](#)
- [DeleteLoadBalancer](#)
- [DeleteTargetGroup](#)
- [DescribeAutoScalingGroups](#)
- [DescribeAvailabilityZones](#)
- [DescribeIamInstanceProfileAssociations](#)
- [DescribeInstances](#)
- [DescribeLoadBalancers](#)
- [DescribeSubnets](#)
- [DescribeTargetGroups](#)
- [DescribeTargetHealth](#)
- [DescribeVpcs](#)
- [RebootInstances](#)
- [ReplaceIamInstanceProfileAssociation](#)
- [TerminateInstanceInAutoScalingGroup](#)
- [UpdateAutoScalingGroup](#)

## MediaStore examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with MediaStore.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### CreateContainer

The following code example shows how to use CreateContainer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.model.CreateContainerRequest;
import software.amazon.awssdk.services.mediastore.model.CreateContainerResponse;
import software.amazon.awssdk.services.mediastore.model.MediaStoreException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateContainer {
 public static long sleepTime = 10;

 public static void main(String[] args) {
 final String usage = ""

 Usage: <containerName>

 Where:
 containerName - The name of the container to create.
```

```

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String containerName = args[0];
 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 createMediaContainer(mediaStoreClient, containerName);
 mediaStoreClient.close();
}

public static void createMediaContainer(MediaStoreClient mediaStoreClient,
String containerName) {
 try {
 CreateContainerRequest containerRequest =
CreateContainerRequest.builder()
 .containerName(containerName)
 .build();

 CreateContainerResponse containerResponse =
mediaStoreClient.createContainer(containerRequest);
 String status = containerResponse.container().status().toString();
 while (!status.equalsIgnoreCase("Active")) {
 status = DescribeContainer.checkContainer(mediaStoreClient,
containerName);
 System.out.println("Status - " + status);
 Thread.sleep(sleepTime * 1000);
 }

 System.out.println("The container ARN value is " +
containerResponse.container().arn());
 System.out.println("Finished ");

 } catch (MediaStoreException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

```
 }
}
```

- For API details, see [CreateContainer](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteContainer

The following code example shows how to use DeleteContainer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.model.CreateContainerRequest;
import software.amazon.awssdk.services.mediastore.model.CreateContainerResponse;
import software.amazon.awssdk.services.mediastore.model.MediaStoreException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateContainer {
 public static long sleepTime = 10;

 public static void main(String[] args) {
 final String usage = ""

 Usage: <containerName>

 Where:
 containerName - The name of the container to create.
```

```

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String containerName = args[0];
 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 createMediaContainer(mediaStoreClient, containerName);
 mediaStoreClient.close();
}

public static void createMediaContainer(MediaStoreClient mediaStoreClient,
String containerName) {
 try {
 CreateContainerRequest containerRequest =
CreateContainerRequest.builder()
 .containerName(containerName)
 .build();

 CreateContainerResponse containerResponse =
mediaStoreClient.createContainer(containerRequest);
 String status = containerResponse.container().status().toString();
 while (!status.equalsIgnoreCase("Active")) {
 status = DescribeContainer.checkContainer(mediaStoreClient,
containerName);
 System.out.println("Status - " + status);
 Thread.sleep(sleepTime * 1000);
 }

 System.out.println("The container ARN value is " +
containerResponse.container().arn());
 System.out.println("Finished ");

 } catch (MediaStoreException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

```
}
}
```

- For API details, see [DeleteContainer](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteObject

The following code example shows how to use DeleteObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerRequest;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerResponse;
import software.amazon.awssdk.services.mediastoredata.MediaStoreDataClient;
import software.amazon.awssdk.services.mediastoredata.model.DeleteObjectRequest;
import software.amazon.awssdk.services.mediastoredata.model.MediaStoreDataException;
import java.net.URI;
import java.net.URISyntaxException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteObject {
 public static void main(String[] args) throws URISyntaxException {
 final String usage = ""

 Usage: <completePath> <containerName>
```



```

 Where:
 completePath - The path (including the container) of the item to
delete.
 containerName - The name of the container.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String completePath = args[0];
 String containerName = args[1];
 Region region = Region.US_EAST_1;
 URI uri = new URI(getEndpoint(containerName));

 MediaStoreDataClient mediaStoreData = MediaStoreDataClient.builder()
 .endpointOverride(uri)
 .region(region)
 .build();

 deleteMediaObject(mediaStoreData, completePath);
 mediaStoreData.close();
}

public static void deleteMediaObject(MediaStoreDataClient mediaStoreData, String
completePath) {
 try {
 DeleteObjectRequest deleteObjectRequest = DeleteObjectRequest.builder()
 .path(completePath)
 .build();

 mediaStoreData.deleteObject(deleteObjectRequest);

 } catch (MediaStoreDataException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

private static String getEndpoint(String containerName) {
 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)

```

```

 .build();

 DescribeContainerRequest containerRequest =
 DescribeContainerRequest.builder()
 .containerName(containerName)
 .build();

 DescribeContainerResponse response =
 mediaStoreClient.describeContainer(containerRequest);
 mediaStoreClient.close();
 return response.container().endpoint();
 }
}

```

- For API details, see [DeleteObject](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeContainer

The following code example shows how to use DescribeContainer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerRequest;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerResponse;
import software.amazon.awssdk.services.mediastore.model.MediaStoreException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

```

```
*/
public class DescribeContainer {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <containerName>

 Where:
 containerName - The name of the container to describe.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String containerName = args[0];
 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 System.out.println("Status is " + checkContainer(mediaStoreClient,
 containerName));
 mediaStoreClient.close();
 }

 public static String checkContainer(MediaStoreClient mediaStoreClient, String
 containerName) {
 try {
 DescribeContainerRequest describeContainerRequest =
 DescribeContainerRequest.builder()
 .containerName(containerName)
 .build();

 DescribeContainerResponse containerResponse =
 mediaStoreClient.describeContainer(describeContainerRequest);
 System.out.println("The container name is " +
 containerResponse.container().name());
 System.out.println("The container ARN is " +
 containerResponse.container().arn());
 return containerResponse.container().status().toString();
 }
 }
}
```

```

 } catch (MediaStoreException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [DescribeContainer](#) in *AWS SDK for Java 2.x API Reference*.

## GetObject

The following code example shows how to use `GetObject`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.ResponseInputStream;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerRequest;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerResponse;
import software.amazon.awssdk.services.mediastoredata.MediaStoreDataClient;
import software.amazon.awssdk.services.mediastoredata.model.GetObjectRequest;
import software.amazon.awssdk.services.mediastoredata.model.GetObjectResponse;
import software.amazon.awssdk.services.mediastoredata.model.MediaStoreDataException;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.net.URI;
import java.net.URISyntaxException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.

```

```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class GetObject {
 public static void main(String[] args) throws URISyntaxException {
 final String usage = ""

 Usage: <completePath> <containerName> <savePath>

 Where:
 completePath - The path of the object in the container (for
example, Videos5/sampleVideo.mp4).
 containerName - The name of the container.
 savePath - The path on the local drive where the file is saved,
including the file name (for example, C:/AWS/myvid.mp4).
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String completePath = args[0];
 String containerName = args[1];
 String savePath = args[2];

 Region region = Region.US_EAST_1;
 URI uri = new URI(getEndpoint(containerName));
 MediaStoreDataClient mediaStoreData = MediaStoreDataClient.builder()
 .endpointOverride(uri)
 .region(region)
 .build();

 getMediaObject(mediaStoreData, completePath, savePath);
 mediaStoreData.close();
 }

 public static void getMediaObject(MediaStoreDataClient mediaStoreData, String
completePath, String savePath) {

 try {
 GetObjectRequest objectRequest = GetObjectRequest.builder()

```

```

 .path(completePath)
 .build();

 // Write out the data to a file.
 ResponseInputStream<GetObjectResponse> data =
mediaStoreData.getObject(objectRequest);
 byte[] buffer = new byte[data.available()];
 data.read(buffer);

 File targetFile = new File(savePath);
 OutputStream outputStream = new FileOutputStream(targetFile);
 outputStream.write(buffer);
 System.out.println("The data was written to " + savePath);

 } catch (MediaStoreDataException | IOException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

private static String getEndpoint(String containerName) {
 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 DescribeContainerRequest containerRequest =
DescribeContainerRequest.builder()
 .containerName(containerName)
 .build();

 DescribeContainerResponse response =
mediaStoreClient.describeContainer(containerRequest);
 return response.container().endpoint();
}
}

```

- For API details, see [GetObject](#) in *AWS SDK for Java 2.x API Reference*.

## ListContainers

The following code example shows how to use `ListContainers`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.services.mediastore.model.Container;
import software.amazon.awssdk.services.mediastore.model.ListContainersResponse;
import software.amazon.awssdk.services.mediastore.model.MediaStoreException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListContainers {

 public static void main(String[] args) {

 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 listAllContainers(mediaStoreClient);
 mediaStoreClient.close();
 }

 public static void listAllContainers(MediaStoreClient mediaStoreClient) {
 try {
 ListContainersResponse containersResponse =
mediaStoreClient.listContainers();
 List<Container> containers = containersResponse.containers();

```

```

 for (Container container : containers) {
 System.out.println("Container name is " + container.name());
 }

 } catch (MediaStoreException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListContainers](#) in *AWS SDK for Java 2.x API Reference*.

## PutObject

The following code example shows how to use PutObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediastore.MediaStoreClient;
import software.amazon.awssdk.services.mediastoredata.MediaStoreDataClient;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.mediastoredata.model.PutObjectRequest;
import software.amazon.awssdk.services.mediastoredata.model.MediaStoreDataException;
import software.amazon.awssdk.services.mediastoredata.model.PutObjectResponse;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerRequest;
import software.amazon.awssdk.services.mediastore.model.DescribeContainerResponse;
import java.io.File;
import java.net.URI;
import java.net.URISyntaxException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.

```



```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class PutObject {
 public static void main(String[] args) throws URISyntaxException {
 final String USAGE = ""

 To run this example, supply the name of a container, a file location
 to use, and path in the container\s

 Ex: <containerName> <filePath> <completePath>
 """;

 if (args.length < 3) {
 System.out.println(USAGE);
 System.exit(1);
 }

 String containerName = args[0];
 String filePath = args[1];
 String completePath = args[2];

 Region region = Region.US_EAST_1;
 URI uri = new URI(getEndpoint(containerName));
 MediaStoreDataClient mediaStoreData = MediaStoreDataClient.builder()
 .endpointOverride(uri)
 .region(region)
 .build();

 putMediaObject(mediaStoreData, filePath, completePath);
 mediaStoreData.close();
 }

 public static void putMediaObject(MediaStoreDataClient mediaStoreData, String
filePath, String completePath) {
 try {
 File myFile = new File(filePath);
 RequestBody requestBody = RequestBody.fromFile(myFile);

 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .path(completePath)
 .contentType("video/mp4")

```

```

 .build();

 PutObjectResponse response = mediaStoreData.putObject(objectRequest,
requestBody);
 System.out.println("The saved object is " +
response.storageClass().toString());

 } catch (MediaStoreDataException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static String getEndpoint(String containerName) {

 Region region = Region.US_EAST_1;
 MediaStoreClient mediaStoreClient = MediaStoreClient.builder()
 .region(region)
 .build();

 DescribeContainerRequest containerRequest =
DescribeContainerRequest.builder()
 .containerName(containerName)
 .build();

 DescribeContainerResponse response =
mediaStoreClient.describeContainer(containerRequest);
 return response.container().endpoint();
}
}

```

- For API details, see [PutObject](#) in *AWS SDK for Java 2.x API Reference*.

## AWS Entity Resolution examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Entity Resolution.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello AWS Entity Resolution

The following code example shows how to get started using AWS Entity Resolution.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloEntityResoultion {

 private static final Logger logger =
 LoggerFactory.getLogger(HelloEntityResoultion.class);

 private static EntityResolutionAsyncClient entityResolutionAsyncClient;
 public static void main(String[] args) {
```

```

 listMatchingWorkflows();
 }

 public static EntityResolutionAsyncClient getResolutionAsyncClient() {
 if (entityResolutionAsyncClient == null) {
 /*
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */

 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.

 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.

 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .retryStrategy(RetryMode.STANDARD)
 .build();

 entityResolutionAsyncClient = EntityResolutionAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return entityResolutionAsyncClient;
 }

 /**
 * Lists all matching workflows using an asynchronous paginator.
 * <p>

```

```
* This method requests a paginated list of matching workflows from the
* AWS Entity Resolution service and logs the names of the retrieved workflows.
* It uses an asynchronous approach with a paginator and waits for the operation
* to complete using {@code CompletableFuture#join()}.
* </p>
*/
public static void listMatchingWorkflows() {
 ListMatchingWorkflowsRequest request =
 ListMatchingWorkflowsRequest.builder().build();

 ListMatchingWorkflowsPublisher paginator =
 getResolutionAsyncClient().listMatchingWorkflowsPaginator(request);

 // Iterate through the paginated results asynchronously
 CompletableFuture<Void> future = paginator.subscribe(response -> {
 response.workflowSummaries().forEach(workflow ->
 logger.info("Matching Workflow Name: " + workflow.workflowName())
);
 });

 // Wait for the asynchronous operation to complete
 future.join();
}
```

- For API details, see [ListMatchingWorkflows](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create Schema Mapping.
- Create an AWS Entity Resolution workflow.
- Start the matching job for the workflow.
- Get details for the matching job.
- Get Schema Mapping.
- List all Schema Mappings.

- Tag the Schema Mapping resource.
- Delete the AWS Entity Resolution Assets.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating AWS Entity Resolution features.

```
public class EntityResScenario {
 private static final Logger logger =
 LoggerFactory.getLogger(EntityResScenario.class);
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final String STACK_NAME = "EntityResolutionCdkStack2";
 private static final String ENTITY_RESOLUTION_ROLE_ARN_KEY =
 "EntityResolutionRoleArn";
 private static final String GLUE_DATA_BUCKET_NAME_KEY = "GlueDataBucketName";
 private static final String JSON_GLUE_TABLE_ARN_KEY = "JsonErGlueTableArn";
 private static final String CSV_GLUE_TABLE_ARN_KEY = "CsvErGlueTableArn";
 private static String glueBucketName;
 private static String workflowName = "workflow-" + UUID.randomUUID();

 private static String jsonSchemaMappingName = "jsonschema-" + UUID.randomUUID();
 private static String jsonSchemaMappingArn = null;
 private static String csvSchemaMappingName = "csv-" + UUID.randomUUID();
 private static String roleARN;
 private static String csvGlueTableArn;
 private static String jsonGlueTableArn;
 private static Scanner scanner = new Scanner(System.in);

 private static EntityResActions actions = new EntityResActions();

 public static void main(String[] args) throws InterruptedException {

 logger.info("Welcome to the AWS Entity Resolution Scenario.");
 logger.info(""
```

AWS Entity Resolution is a fully-managed machine learning service provided by

Amazon Web Services (AWS) that helps organizations extract, link, and organize information from multiple data sources. It leverages natural language processing and deep learning models to identify and resolve entities, such as people, places, organizations, and products, across structured and unstructured data.

With Entity Resolution, customers can build robust data integration pipelines to combine and reconcile data from multiple systems, databases,

and documents. The service can handle ambiguous, incomplete, or conflicting information, and provide a unified view of entities and their relationships.

This can be particularly valuable in applications such as customer 360, fraud detection, supply chain management, and knowledge management, where

accurate entity identification is crucial.

The `EntityResolutionAsyncClient` interface in the AWS SDK for Java 2.x provides a set of methods to programmatically interact with the AWS

Entity Resolution service. This allows developers to automate the entity extraction, linking, and deduplication process as part of their data processing workflows.

With Entity Resolution, organizations can unlock the value of their data, improve decision-making, and enhance customer experiences by having a reliable, comprehensive view of their key entities.

```
""");
```

```
waitForInputToContinue(scanner);
logger.info(DASHES);
```

```
logger.info(DASHES);
logger.info("""
```

To prepare the AWS resources needed for this scenario application, the next step uploads

a CloudFormation template whose resulting stack creates the following resources:

- An AWS Glue Data Catalog table

- An AWS IAM role
- An AWS S3 bucket
- An AWS Entity Resolution Schema

It can take a couple minutes for the Stack to finish creating the resources.

```

 """);
 waitForInputToContinue(scanner);
 logger.info("Generating resources...");
 CloudFormationHelper.deployCloudFormationStack(STACK_NAME);
 Map<String, String> outputsMap =
CloudFormationHelper.getStackOutputsAsync(STACK_NAME).join();
 roleARN = outputsMap.get(ENTITY_RESOLUTION_ROLE_ARN_KEY);
 glueBucketName = outputsMap.get(GLUE_DATA_BUCKET_NAME_KEY);
 csvGlueTableArn = outputsMap.get(CSV_GLUE_TABLE_ARN_KEY);
 jsonGlueTableArn = outputsMap.get(JSON_GLUE_TABLE_ARN_KEY);
 logger.info(DASHES);
 waitForInputToContinue(scanner);

 try {
 runScenario();

 } catch (Exception ce) {
 Throwable cause = ce.getCause();
 logger.error("An exception happened: " + (cause != null ?
cause.getMessage() : ce.getMessage()));
 }
 }

 private static void runScenario() throws InterruptedException {
 /*
 This JSON is a valid input for the AWS Entity Resolution service.
 The JSON represents an array of three objects, each containing an "id",
"name", and "email"
 property. This format aligns with the expected input structure for the
 Entity Resolution service.
 */
 String json = ""
 {"id":"1","name":"Jane Doe","email":"jane.doe@example.com"}
 {"id":"2","name":"John Doe","email":"john.doe@example.com"}
 {"id":"3","name":"Jorge Souza","email":"jorge_souza@example.com"}
 "";
 logger.info("Upload the following JSON objects to the {} S3 bucket.",
glueBucketName);

```



```

 logger.info(json);
 String csv = ""
 id,name,email,phone
 1,Jane B.,Doe,jane.doe@example.com,555-876-9846
 2,John Doe Jr.,john.doe@example.com,555-654-3210
 3,María García,maría_garcia@company.com,555-567-1234
 4,Mary Major,mary_major@company.com,555-222-3333
 "";
 logger.info("Upload the following CSV data to the {} S3 bucket.",
glueBucketName);
 logger.info(csv);
 waitForInputToContinue(scanner);
 try {
 actions.uploadInputData(glueBucketName, json, csv);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();

 if (cause == null) {
 logger.error("Failed to upload input data: {}", ce.getMessage(),
ce);
 }

 if (cause instanceof ResourceNotFoundException) {
 logger.error("Failed to upload input data as the resource was not
found: {}", cause.getMessage(), cause);
 }
 return;
 }
 logger.info("The JSON and CSV objects have been uploaded to the S3
bucket.");
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("1. Create Schema Mapping");
 logger.info("")
 Entity Resolution schema mapping aligns and integrates data from
 multiple sources by identifying and matching corresponding entities
 like customers or products. It unifies schemas, resolves conflicts,
 and uses machine learning to link related entities, enabling a
 consolidated, accurate view for improved data quality and decision-
 making.

```

```

 In this example, the schema mapping lines up with the fields in the JSON
 and CSV objects. That is,
 it contains these fields: id, name, and email.
 "");
 try {
 CreateSchemaMappingResponse response =
actions.createSchemaMappingAsync(jsonSchemaMappingName).join();
 jsonSchemaMappingName = response.schemaName();
 logger.info("The JSON schema mapping name is " + jsonSchemaMappingName);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();

 if (cause == null) {
 logger.error("Failed to create JSON schema mapping: {}",
ce.getMessage(), ce);
 }

 if (cause instanceof ConflictException) {
 logger.error("Schema mapping conflict detected: {}",
cause.getMessage(), cause);
 } else {
 logger.error("Unexpected error while creating schema mapping: {}",
cause.getMessage(), cause);
 }
 return;
 }

 try {
 CreateSchemaMappingResponse response =
actions.createSchemaMappingAsync(csvSchemaMappingName).join();
 csvSchemaMappingName = response.schemaName();
 logger.info("The CSV schema mapping name is " + csvSchemaMappingName);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause == null) {
 logger.error("Failed to create CSV schema mapping: {}",
ce.getMessage(), ce);
 }

 if (cause instanceof ConflictException) {
 logger.error("Schema mapping conflict detected: {}",
cause.getMessage(), cause);
 } else {

```

```

 logger.error("Unexpected error while creating CSV schema mapping:
{}", cause.getMessage(), cause);
 }
 return;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("2. Create an AWS Entity Resolution Workflow. ");
logger.info("""
 An Entity Resolution matching workflow identifies and links records
 across datasets that represent the same real-world entity, such as
 customers or products. Using techniques like schema mapping,
 data profiling, and machine learning algorithms,
 it evaluates attributes like names or emails to detect duplicates
 or relationships, even with variations or inconsistencies.
 The workflow outputs consolidated, de-duplicated data.

 We will use the machine learning-based matching technique.
 """);
waitForInputToContinue(scanner);
try {
 String workflowArn = actions.createMatchingWorkflowAsync(
 roleARN, workflowName, glueBucketName, jsonGlueTableArn,
 jsonSchemaMappingName, csvGlueTableArn,
 csvSchemaMappingName).join();

 logger.info("The workflow ARN is: " + workflowArn);
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();

 if (cause == null) {
 logger.error("An unexpected error occurred: {}", ce.getMessage(),
ce);
 }

 if (cause instanceof ValidationException) {
 logger.error("Validation error: {}", cause.getMessage(), cause);
 } else if (cause instanceof ConflictException) {
 logger.error("Workflow conflict detected: {}", cause.getMessage(),
cause);
 } else {
 logger.error("Unexpected error: {}", cause.getMessage(), cause);
 }
}

```

```

 }
 return;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);
 logger.info("3. Start the matching job of the " + workflowName + "
workflow.");
 waitForInputToContinue(scanner);
 String jobId = null;
 try {
 jobId = actions.startMatchingJobAsync(workflowName).join();
 logger.info("The matching job was successfully started.");
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ConflictException) {
 logger.error("Job conflict detected: {}", cause.getMessage(),
cause);
 } else {
 logger.error("Unexpected error while starting the job: {}",
ce.getMessage(), ce);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("4. While the matching job is running, let's look at other API
methods. First, let's get details for job " + jobId);
 waitForInputToContinue(scanner);
 try {
 actions.getMatchingJobAsync(jobId, workflowName).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The matching job not found: {}", cause.getMessage(),
cause);
 } else {
 logger.error("Failed to start matching job: " + (cause != null ?
cause.getMessage() : ce.getMessage()));
 }
 return;
 }
}

```

```

 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("5. Get the schema mapping for the JSON data.");
 waitForInputToContinue(scanner);
 try {
 GetSchemaMappingResponse response =
actions.getSchemaMappingAsync(jsonSchemaMappingName).join();
 jsonSchemaMappingArn = response.schemaArn();
 logger.info("Schema mapping ARN is " + jsonSchemaMappingArn);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("Schema mapping not found: {}", cause.getMessage(),
cause);
 } else {
 logger.error("Error retrieving the specific schema mapping: " +
ce.getCause().getMessage());
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("6. List Schema Mappings.");
 try {
 actions.ListSchemaMappings();
 } catch (CompletionException ce) {
 logger.error("Error retrieving schema mappings: " +
ce.getCause().getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Tag the {} resource.", jsonSchemaMappingName);
 logger.info("""
 Tags can help you organize and categorize your Entity Resolution
resources.
 You can also use them to scope user permissions by granting a user
permission
 to access or change only resources with certain tag values.

```

In Entity Resolution, SchemaMapping and MatchingWorkflow can be tagged.  
 For this example,  
 the SchemaMapping is tagged.  
 """);

```
try {
 actions.tagEntityResource(jsonSchemaMappingArn).join();
} catch (CompletionException ce) {
 logger.error("Error tagging the resource: " +
ce.getCause().getMessage());
 return;
}
```

```
waitForInputToContinue(scanner);
logger.info(DASHES);
```

```
logger.info(DASHES);
logger.info("8. View the results of the AWS Entity Resolution Workflow.");
logger.info("""
```

You cannot view the result of the workflow that is in a running state.  
 In order to view the results, you need to wait for the workflow that we  
 started in step 3 to complete.

If you choose not to wait, you cannot view the results. You can perform  
 this task manually in the AWS Management Console.

This can take up to 30 mins (y/n).  
 """);

```
String viewAns = scanner.nextLine().trim();
boolean isComplete = false;
if (viewAns.equalsIgnoreCase("y")) {
 logger.info("You selected to view the Entity Resolution Workflow
results.");
 countdownWithWorkflowCheck(actions, 1800, jobId, workflowName);
 isComplete = true;
 try {
 JobMetrics metrics = actions.getJobInfo(workflowName, jobId).join();
 logger.info("Number of input records: {}", metrics.inputRecords());
 logger.info("Number of match ids: {}", metrics.matchIDs());
 logger.info("Number of records not processed: {}",
metrics.recordsNotProcessed());
 logger.info("Number of total records processed: {}",
metrics.totalRecordsProcessed());
```

```

 logger.info("The following represents the output data generated by
the Entity Resolution workflow based on the JSON and CSV input data. The output
data is stored in the {} bucket.", glueBucketName);
 actions.printData(glueBucketName);

 logger.info("""

 Note that each of the last 2 records are considered a match even
though the 'name' differs between the records;
 For example 'John Doe Jr.' compared to 'John Doe'.
 The confidence level is a value between 0 and 1, where 1
indicates a perfect match.

 """);

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The job not found: {}", cause.getMessage(),
cause);
 } else {
 logger.error("Error retrieving job information: " +
ce.getCause().getMessage());
 }
 return;
 }
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("9. Do you want to delete the resources, including the workflow?
(y/n)");
 logger.info("""
 You cannot delete the workflow that is in a running state.
 In order to delete the workflow, you need to wait for the workflow to
complete.

 You can delete the workflow manually in the AWS Management Console at a
later time.

 If you already waited for the workflow to complete in the previous
step,

```

```

 the workflow is completed and you can delete it.

 If the workflow is not completed, this can take up to 30 mins (y/n).
 """);
String delAns = scanner.nextLine().trim();
if (delAns.equalsIgnoreCase("y")) {
 try {
 if (!isComplete) {
 countdownWithWorkflowCheck(actions, 1800, jobId, workflowName);
 }
 actions.deleteMatchingWorkflowAsync(workflowName).join();
 logger.info("Workflow deleted successfully!");
 } catch (CompletionException ce) {
 logger.info("Error deleting the workflow: {} ", ce.getMessage());
 return;
 }

 try {
 // Delete both schema mappings.
 actions.deleteSchemaMappingAsync(jsonSchemaMappingName).join();
 actions.deleteSchemaMappingAsync(csvSchemaMappingName).join();
 logger.info("Both schema mappings were deleted successfully!");
 } catch (CompletionException ce) {
 logger.error("Error deleting schema mapping: {}", ce.getMessage());
 return;
 }
}

waitForInputToContinue(scanner);
logger.info(DASHES);
logger.info("""
 Now we delete the CloudFormation stack, which deletes
 the resources that were created at the beginning of this scenario.
 """);
waitForInputToContinue(scanner);
logger.info(DASHES);
try {
 deleteCloudFormationStack();
} catch (RuntimeException e) {
 logger.error("Failed to delete the stack: {}", e.getMessage());
 return;
}

} else {

```



```
 logger.info("You can delete the AWS resources in the AWS Management
Console.");
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("This concludes the AWS Entity Resolution scenario.");
 logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}

public static void countdownWithWorkflowCheck(EntityResActions actions, int
totalSeconds, String jobId, String workflowName) throws InterruptedException {
 int secondsElapsed = 0;

 while (true) {
 // Calculate display minutes and seconds.
 int remainingTime = totalSeconds - secondsElapsed;
 int displayMinutes = remainingTime / 60;
 int displaySeconds = remainingTime % 60;

 // Print the countdown.
 System.out.printf("\r%02d:%02d", displayMinutes, displaySeconds);
 Thread.sleep(1000); // Wait for 1 second
 secondsElapsed++;
 }
}
```

```

 // Check workflow status every 60 seconds.
 if (secondsElapsed % 60 == 0 || remainingTime <= 0) {
 GetMatchingJobResponse response =
actions.checkWorkflowStatusCompleteAsync(jobId, workflowName).join();
 if (response != null &&
"SUCCEEDED".equalsIgnoreCase(String.valueOf(response.status()))) {
 logger.info(""); // Move to the next line after countdown.
 logger.info("Countdown complete: Workflow is in Completed
state!");
 break; // Break out of the loop if the status is "SUCCEEDED"
 }
 }

 // If countdown reaches zero, reset it for continuous countdown.
 if (remainingTime <= 0) {
 secondsElapsed = 0;
 }
 }
}

private static void deleteCloudFormationStack() {
 try {
 CloudFormationHelper.emptyS3Bucket(glueBucketName);
 CloudFormationHelper.destroyCloudFormationStack(STACK_NAME);
 logger.info("Resources deleted successfully!");
 } catch (CloudFormationException e) {
 throw new RuntimeException("Failed to delete CloudFormation stack: " +
e.getMessage(), e);
 } catch (S3Exception e) {
 throw new RuntimeException("Failed to empty S3 bucket: " +
e.getMessage(), e);
 }
}
}

```

A wrapper class for AWS Entity Resolution SDK methods.

```

public class EntityResActions {

 private static final String PREFIX = "eroutput/";
 private static final Logger logger =
LoggerFactory.getLogger(EntityResActions.class);

```

```

private static EntityResolutionAsyncClient entityResolutionAsyncClient;

private static S3AsyncClient s3AsyncClient;

public static EntityResolutionAsyncClient getResolutionAsyncClient() {
 if (entityResolutionAsyncClient == null) {
 /*
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */

 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.

 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.

 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .retryStrategy(RetryMode.STANDARD)
 .build();

 entityResolutionAsyncClient = EntityResolutionAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return entityResolutionAsyncClient;
}

public static S3AsyncClient getS3AsyncClient() {

```

```

 if (s3AsyncClient == null) {
 /*
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */

 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.

 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.

 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .retryStrategy(RetryMode.STANDARD)
 .build();

 s3AsyncClient = S3AsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return s3AsyncClient;
 }

 /**
 * Deletes the schema mapping asynchronously.
 *
 * @param schemaName the name of the schema to delete
 * @return a {@link CompletableFuture} that completes when the schema mapping is
deleted successfully,
 * or throws a {@link RuntimeException} if the deletion fails

```

```

 */
 public CompletableFuture<DeleteSchemaMappingResponse>
deleteSchemaMappingAsync(String schemaName) {
 DeleteSchemaMappingRequest request = DeleteSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .build();

 return getResolutionAsyncClient().deleteSchemaMapping(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully deleted the schema mapping, log the success
message.
 logger.info("Schema mapping '{}' deleted successfully.",
schemaName);
 } else {
 // Ensure exception is not null before accessing its cause.
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while deleting the schema mapping.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The schema mapping was not
found to delete: " + schemaName, cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to delete schema mapping:
" + schemaName, exception);
 }
 });
 }

 /**
 * Lists the schema mappings associated with the current AWS account. This
method uses an asynchronous paginator to
 * retrieve the schema mappings, and prints the name of each schema mapping to
the console.
 */
 public void ListSchemaMappings() {
 ListSchemaMappingsRequest mappingsRequest =
ListSchemaMappingsRequest.builder()
 .build();
 }

```

```

 ListSchemaMappingsPublisher paginator =
getResolutionAsyncClient().listSchemaMappingsPaginator(mappingsRequest);

 // Iterate through the pages of results
 CompletableFuture<Void> future = paginator.subscribe(response -> {
 response.schemaList().forEach(schemaMapping ->
 logger.info("Schema Mapping Name: " + schemaMapping.schemaName())
);
 });

 // Wait for the asynchronous operation to complete
 future.join();
 }

 /**
 * Asynchronously deletes a workflow with the specified name.
 *
 * @param workflowName the name of the workflow to be deleted
 * @return a {@link CompletableFuture} that completes when the workflow has been
deleted
 * @throws RuntimeException if the deletion of the workflow fails
 */
 public CompletableFuture<DeleteMatchingWorkflowResponse>
deleteMatchingWorkflowAsync(String workflowName) {
 DeleteMatchingWorkflowRequest request =
DeleteMatchingWorkflowRequest.builder()
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().deleteMatchingWorkflow(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("{} was deleted", workflowName);
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while deleting the workflow.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The workflow to delete was
not found.", cause);
 }
 }
 });
 }

```

```

 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to delete workflow: " +
exception.getMessage(), exception);
 }
});
}

/**
 * Creates a schema mapping asynchronously.
 *
 * @param schemaName the name of the schema to create
 * @return a {@link CompletableFuture} that represents the asynchronous creation
of the schema mapping
 */
public CompletableFuture<CreateSchemaMappingResponse>
createSchemaMappingAsync(String schemaName) {
 List<SchemaInputAttribute> schemaAttributes = null;
 if (schemaName.startsWith("json")) {
 schemaAttributes = List.of(

SchemaInputAttribute.builder().matchKey("id").fieldName("id").type(SchemaAttributeType.UNIQ

SchemaInputAttribute.builder().matchKey("name").fieldName("name").type(SchemaAttributeType.

SchemaInputAttribute.builder().matchKey("email").fieldName("email").type(SchemaAttributeType

);
 } else {
 schemaAttributes = List.of(

SchemaInputAttribute.builder().matchKey("id").fieldName("id").type(SchemaAttributeType.UNIQ

SchemaInputAttribute.builder().matchKey("name").fieldName("name").type(SchemaAttributeType.

SchemaInputAttribute.builder().matchKey("email").fieldName("email").type(SchemaAttributeType

SchemaInputAttribute.builder().fieldName("phone").type(SchemaAttributeType.PROVIDER_ID).sub

);
 }

 CreateSchemaMappingRequest request = CreateSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .mappedInputFields(schemaAttributes)

```

```

 .build();

 return getResolutionAsyncClient().createSchemaMapping(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("[{}] schema mapping Created Successfully!",
schemaName);
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while creating the schema mapping.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflicting schema mapping
already exists. Resolve conflicts before proceeding.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to create schema mapping:
" + exception.getMessage(), exception);
 }
 });
 }

 /**
 * Retrieves the schema mapping asynchronously.
 *
 * @param schemaName the name of the schema to retrieve the mapping for
 * @return a {@link CompletableFuture} that completes with the {@link
GetSchemaMappingResponse} when the operation
 * is complete
 * @throws RuntimeException if the schema mapping retrieval fails
 */
 public CompletableFuture<GetSchemaMappingResponse> getSchemaMappingAsync(String
schemaName) {
 GetSchemaMappingRequest mappingRequest = GetSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .build();

 return getResolutionAsyncClient().getSchemaMapping(mappingRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {

```



```

 response.mappedInputFields().forEach(attribute ->
 logger.info("Attribute Name: " + attribute.fieldName() +
 ", Attribute Type: " + attribute.type().toString()));
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while getting schema mapping.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested schema mapping
was not found.", cause);
 }

 // Wrap other exceptions in a CompletionException with the
message.
 throw new CompletionException("Failed to get schema mapping: " +
exception.getMessage(), exception);
 }
});
}

/**
 * Asynchronously retrieves a matching job based on the provided job ID and
workflow name.
 *
 * @param jobId the ID of the job to retrieve
 * @param workflowName the name of the workflow associated with the job
 * @return a {@link CompletableFuture} that completes when the job information
is available or an exception occurs
 */
public CompletableFuture<GetMatchingJobResponse> getMatchingJobAsync(String
jobId, String workflowName) {
 GetMatchingJobRequest request = GetMatchingJobRequest.builder()
 .jobId(jobId)
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().getMatchingJob(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully fetched the matching job details, log the job
status.

```

```

 logger.info("Job status: " + response.status());
 logger.info("Job details: " + response.toString());
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while fetching the matching job.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested job could not
be found.", cause);
 }

 // Wrap other exceptions in a CompletionException with the
message.
 throw new CompletionException("Error fetching matching job: " +
exception.getMessage(), exception);
 }
 });
}

/**
 * Starts a matching job asynchronously for the specified workflow name.
 *
 * @param workflowName the name of the workflow for which to start the matching
job
 * @return a {@link CompletableFuture} that completes with the job ID of the
started matching job, or an empty
 * string if the operation fails
 */
public CompletableFuture<String> startMatchingJobAsync(String workflowName) {
 StartMatchingJobRequest jobRequest = StartMatchingJobRequest.builder()
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().startMatchingJob(jobRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 String jobId = response.jobId();
 logger.info("Job ID: " + jobId);
 } else {
 if (exception == null) {

```

```

 throw new CompletionException("An unknown error occurred
while starting the job.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("The job is already running.
Resolve conflicts before starting a new job.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to start the job: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response != null ? response.jobId() : "");
}

/**
 * Checks the status of a workflow asynchronously.
 *
 * @param jobId the ID of the job to check
 * @param workflowName the name of the workflow to check
 * @return a CompletableFuture that resolves to a boolean value indicating
whether the workflow has completed
 * successfully
 */
public CompletableFuture<GetMatchingJobResponse>
checkWorkflowStatusCompleteAsync(String jobId, String workflowName) {
 GetMatchingJobRequest request = GetMatchingJobRequest.builder()
 .jobId(jobId)
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().getMatchingJob(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Process the response and log the job status.
 logger.info("Job status: " + response.status());
 } else {
 // Ensure exception is not null before accessing its cause.
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while checking job status.", null);

```

```

 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested resource was
not found while checking the job status.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to check job status: " +
exception.getMessage(), exception);
 }
 });
}

/**
 * Creates an asynchronous CompletableFuture to manage the creation of a
 * matching workflow.
 *
 * @param roleARN the AWS IAM role ARN to be used for the
 * workflow execution
 * @param workflowName the name of the workflow to be created
 * @param outputBucket the S3 bucket path where the workflow output
 * will be stored
 * @param jsonGlueTableArn the ARN of the Glue Data Catalog table to be
 * used as the input source
 * @param jsonErSchemaMappingName the name of the schema to be used for the
 * input source
 * @return a CompletableFuture that, when completed, will return the ARN of the
 * created workflow
 */
public CompletableFuture<String> createMatchingWorkflowAsync(
 String roleARN
 , String workflowName
 , String outputBucket
 , String jsonGlueTableArn
 , String jsonErSchemaMappingName
 , String csvGlueTableArn
 , String csvErSchemaMappingName) {

 InputSource jsonInputSource = InputSource.builder()
 .inputSourceARN(jsonGlueTableArn)
 .schemaName(jsonErSchemaMappingName)
 .applyNormalization(false)

```

```
.build();

InputSource csvInputSource = InputSource.builder()
 .inputSourceARN(csvGlueTableArn)
 .schemaName(csvErSchemaMappingName)
 .applyNormalization(false)
 .build();

OutputAttribute idOutputAttribute = OutputAttribute.builder()
 .name("id")
 .build();

OutputAttribute nameOutputAttribute = OutputAttribute.builder()
 .name("name")
 .build();

OutputAttribute emailOutputAttribute = OutputAttribute.builder()
 .name("email")
 .build();

OutputAttribute phoneOutputAttribute = OutputAttribute.builder()
 .name("phone")
 .build();

OutputSource outputSource = OutputSource.builder()
 .outputS3Path("s3://" + outputBucket + "/eroutput")
 .output(idOutputAttribute, nameOutputAttribute, emailOutputAttribute,
phoneOutputAttribute)
 .applyNormalization(false)
 .build();

ResolutionTechniques resolutionType = ResolutionTechniques.builder()
 .resolutionType(ResolutionType.ML_MATCHING)
 .build();

CreateMatchingWorkflowRequest workflowRequest =
CreateMatchingWorkflowRequest.builder()
 .roleArn(roleARN)
 .description("Created by using the AWS SDK for Java")
 .workflowName(workflowName)
 .inputSourceConfig(List.of(jsonInputSource, csvInputSource))
 .outputSourceConfig(List.of(outputSource))
 .resolutionTechniques(resolutionType)
 .build();
```

```

 return getResolutionAsyncClient().createMatchingWorkflow(workflowRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Workflow created successfully.");
 } else {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException("Invalid request: Please check
input parameters.", cause);
 }

 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflicting workflow
already exists. Resolve conflicts before proceeding.", cause);
 }
 throw new CompletionException("Failed to create workflow: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(CreateMatchingWorkflowResponse::workflowArn);
 }

 /**
 * Tags the specified schema mapping ARN.
 *
 * @param schemaMappingARN the ARN of the schema mapping to tag
 */
 public CompletableFuture<TagResourceResponse> tagEntityResource(String
schemaMappingARN) {
 Map<String, String> tags = new HashMap<>();
 tags.put("tag1", "tag1Value");
 tags.put("tag2", "tag2Value");

 TagResourceRequest request = TagResourceRequest.builder()
 .resourceArn(schemaMappingARN)
 .tags(tags)
 .build();

 return getResolutionAsyncClient().tagResource(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully tagged the resource, log the success message.
 logger.info("Successfully tagged the resource.");
 }
 })
 }

```

```

 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while tagging the resource.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource to tag was not
found.", cause);
 }
 throw new CompletionException("Failed to tag the resource: " +
exception.getMessage(), exception);
 }
 });
}

public CompletableFuture<JobMetrics> getJobInfo(String workflowName, String
jobId) {
 return getResolutionAsyncClient().getMatchingJob(b -> b
 .workflowName(workflowName)
 .jobId(jobId))
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Job metrics fetched successfully for jobId: " +
jobId);
 } else {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("Invalid request: Job id was
not found.", cause);
 }
 throw new CompletionException("Failed to fetch job info: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response.metrics()); // Extract job metrics
}

/**
 * Uploads data to an Amazon S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to upload the data to
 * @param jsonData the JSON data to be uploaded

```

```

 * @param csvData the CSV data to be uploaded
 * @return a {@link CompletableFuture} representing both asynchronous operation
of uploading the data
 * @throws RuntimeException if an error occurs during the file upload
 */

 public void uploadInputData(String bucketName, String jsonData, String csvData)
 {
 // Upload JSON data.
 String jsonKey = "jsonData/data.json";
 PutObjectRequest jsonUploadRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(jsonKey)
 .contentType("application/json")
 .build();

 CompletableFuture<PutObjectResponse> jsonUploadResponse =
getS3AsyncClient().putObject(jsonUploadRequest,
AsyncRequestBody.fromString(jsonData));

 // Upload CSV data.
 String csvKey = "csvData/data.csv";
 PutObjectRequest csvUploadRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(csvKey)
 .contentType("text/csv")
 .build();

 CompletableFuture<PutObjectResponse> csvUploadResponse =
getS3AsyncClient().putObject(csvUploadRequest,
AsyncRequestBody.fromString(csvData));

 CompletableFuture.allOf(jsonUploadResponse, csvUploadResponse)
 .whenComplete((result, ex) -> {
 if (ex != null) {
 // Wrap an AWS exception.
 throw new CompletionException("Failed to upload files", ex);
 }
 }).join();
 }

 /**
 * Finds the latest file in the S3 bucket that starts with "run-" in any depth
of subfolders

```



```

 */
 private CompletableFuture<String> findLatestMatchingFile(String bucketName) {
 ListObjectsV2Request request = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .prefix(PREFIX) // Searches within the given folder
 .build();

 return getS3AsyncClient().listObjectsV2(request)
 .thenApply(response -> response.contents().stream()
 .map(S3Object::key)
 .filter(key -> key.matches(".*?/run-[0-9a-zA-Z\\-]+")) // Matches
files like run-XXXXX in any subfolder
 .max(String::compareTo) // Gets the latest file
 .orElse(null))
 .whenComplete((result, exception) -> {
 if (exception == null) {
 if (result != null) {
 logger.info("Latest matching file found: " + result);
 } else {
 logger.info("No matching files found.");
 }
 } else {
 throw new CompletionException("Failed to find latest matching
file: " + exception.getMessage(), exception);
 }
 });
 }

 /**
 * Prints the data located in the file in the S3 bucket that starts with "run-"
in any depth of subfolders
 */
 public void printData(String bucketName) {
 try {
 // Find the latest file with "run-" prefix in any depth of subfolders.
 String s3Key = findLatestMatchingFile(bucketName).join();
 if (s3Key == null) {
 logger.error("No matching files found in S3.");
 return;
 }

 logger.info("Downloading file: " + s3Key);

 // Read CSV file as String.

```

```

 String csvContent = readCSVFromS3Async(bucketName, s3Key).join();
 if (csvContent.isEmpty()) {
 logger.error("File is empty.");
 return;
 }

 // Process CSV content.
 List<String[]> records = parseCSV(csvContent);
 printTable(records);

 } catch (RuntimeException | IOException | CsvException e) {
 logger.error("Error processing CSV file from S3: " + e.getMessage());
 e.printStackTrace();
 }
}

/**
 * Reads a CSV file from S3 and returns it as a String.
 */
private static CompletableFuture<String> readCSVFromS3Async(String bucketName,
String s3Key) {
 GetObjectRequest getObjectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(s3Key)
 .build();

 // Initiating the asynchronous request to get the file as bytes
 return getS3AsyncClient().getObject(getObjectRequest,
AsyncResponseTransformer.toBytes())
 .thenApply(responseBytes -> responseBytes.asUtf8String()) // Convert
bytes to UTF-8 string
 .whenComplete((result, exception) -> {
 if (exception != null) {
 throw new CompletionException("Failed to read CSV from S3: " +
exception.getMessage(), exception);
 } else {
 logger.info("Successfully fetched CSV file content from S3.");
 }
 });
}

/**
 * Parses CSV content from a String into a list of records.
 */

```

```

 private static List<String[]> parseCSV(String csvContent) throws IOException,
CsvException {
 try (CSVReader csvReader = new CSVReader(new StringReader(csvContent))) {
 return csvReader.readAll();
 }
 }

 /**
 * Prints the given CSV data in a formatted table
 */
 private static void printTable(List<String[]> records) {
 if (records.isEmpty()) {
 System.out.println("No records found.");
 return;
 }

 String[] headers = records.get(0);
 List<String[]> rows = records.subList(1, records.size());

 // Determine column widths dynamically based on longest content
 int[] columnWidths = new int[headers.length];
 for (int i = 0; i < headers.length; i++) {
 final int columnIndex = i;
 int maxWidth = Math.max(headers[i].length(), rows.stream()
 .map(row -> row.length > columnIndex ? row[columnIndex].length() :
0)
 .max(Integer::compareTo)
 .orElse(0));
 columnWidths[i] = Math.min(maxWidth, 25); // Limit max width for better
readability
 }

 // Enable ANSI Console for colored output
 AnsiConsole.systemInstall();

 // Print table header
 System.out.println(ansi().fgYellow().a("=== CSV Data from S3 ===").reset());
 printRow(headers, columnWidths, true);

 // Print rows
 rows.forEach(row -> printRow(row, columnWidths, false));

 // Restore console to normal
 AnsiConsole.systemUninstall();
 }

```

```

 }

 private static void printRow(String[] row, int[] columnWidths, boolean isHeader)
 {
 String border = IntStream.range(0, columnWidths.length)
 .mapToObj(i -> "-".repeat(columnWidths[i] + 2))
 .collect(Collectors.joining("+", "+", "+"));

 if (isHeader) {
 System.out.println(border);
 }

 System.out.print("|");
 for (int i = 0; i < columnWidths.length; i++) {
 String cell = (i < row.length && row[i] != null) ? row[i] : "";
 System.out.printf(" %-" + columnWidths[i] + "s |", isHeader ?
ansi().fgBrightBlue().a(cell).reset() : cell);
 }
 System.out.println();

 if (isHeader) {
 System.out.println(border);
 }
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateMatchingWorkflow](#)
- [CreateSchemaMapping](#)
- [DeleteMatchingWorkflow](#)
- [DeleteSchemaMapping](#)
- [GetMatchingJob](#)
- [GetSchemaMapping](#)
- [ListMatchingWorkflows](#)
- [ListSchemaMappings](#)
- [StartMatchingJob](#)
- [TagResource](#)

# Actions

## CreateMatchingWorkflow

The following code example shows how to use CreateMatchingWorkflow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates an asynchronous CompletableFuture to manage the creation of a
 * matching workflow.
 *
 * @param roleARN the AWS IAM role ARN to be used for the
 * workflow execution
 * @param workflowName the name of the workflow to be created
 * @param outputBucket the S3 bucket path where the workflow output
 * will be stored
 * @param jsonGlueTableArn the ARN of the Glue Data Catalog table to be
 * used as the input source
 * @param jsonErSchemaMappingName the name of the schema to be used for the
 * input source
 * @return a CompletableFuture that, when completed, will return the ARN of the
 * created workflow
 */
public CompletableFuture<String> createMatchingWorkflowAsync(
 String roleARN
 , String workflowName
 , String outputBucket
 , String jsonGlueTableArn
 , String jsonErSchemaMappingName
 , String csvGlueTableArn
 , String csvErSchemaMappingName) {

 InputSource jsonInputSource = InputSource.builder()
 .inputSourceARN(jsonGlueTableArn)
 .schemaName(jsonErSchemaMappingName)
```

```
 .applyNormalization(false)
 .build();

 InputSource csvInputSource = InputSource.builder()
 .inputSourceARN(csvGlueTableArn)
 .schemaName(csvErSchemaMappingName)
 .applyNormalization(false)
 .build();

 OutputAttribute idOutputAttribute = OutputAttribute.builder()
 .name("id")
 .build();

 OutputAttribute nameOutputAttribute = OutputAttribute.builder()
 .name("name")
 .build();

 OutputAttribute emailOutputAttribute = OutputAttribute.builder()
 .name("email")
 .build();

 OutputAttribute phoneOutputAttribute = OutputAttribute.builder()
 .name("phone")
 .build();

 OutputSource outputSource = OutputSource.builder()
 .outputS3Path("s3://" + outputBucket + "/eroutput")
 .output(idOutputAttribute, nameOutputAttribute, emailOutputAttribute,
phoneOutputAttribute)
 .applyNormalization(false)
 .build();

 ResolutionTechniques resolutionType = ResolutionTechniques.builder()
 .resolutionType(ResolutionType.ML_MATCHING)
 .build();

 CreateMatchingWorkflowRequest workflowRequest =
CreateMatchingWorkflowRequest.builder()
 .roleArn(roleARN)
 .description("Created by using the AWS SDK for Java")
 .workflowName(workflowName)
 .inputSourceConfig(List.of(jsonInputSource, csvInputSource))
 .outputSourceConfig(List.of(outputSource))
 .resolutionTechniques(resolutionType)
```

```

 .build();

 return getResolutionAsyncClient().createMatchingWorkflow(workflowRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Workflow created successfully.");
 } else {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException("Invalid request: Please check
input parameters.", cause);
 }

 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflicting workflow
already exists. Resolve conflicts before proceeding.", cause);
 }
 throw new CompletionException("Failed to create workflow: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(CreateMatchingWorkflowResponse::workflowArn);
}

```

- For API details, see [CreateMatchingWorkflow](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSchemaMapping

The following code example shows how to use `CreateSchemaMapping`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a schema mapping asynchronously.
 *

```

```

 * @param schemaName the name of the schema to create
 * @return a {@link CompletableFuture} that represents the asynchronous creation
of the schema mapping
 */
 public CompletableFuture<CreateSchemaMappingResponse>
createSchemaMappingAsync(String schemaName) {
 List<SchemaInputAttribute> schemaAttributes = null;
 if (schemaName.startsWith("json")) {
 schemaAttributes = List.of(

SchemaInputAttribute.builder().matchKey("id").fieldName("id").type(SchemaAttributeType.UNIQ

SchemaInputAttribute.builder().matchKey("name").fieldName("name").type(SchemaAttributeType.

SchemaInputAttribute.builder().matchKey("email").fieldName("email").type(SchemaAttributeType

);
 } else {
 schemaAttributes = List.of(

SchemaInputAttribute.builder().matchKey("id").fieldName("id").type(SchemaAttributeType.UNIQ

SchemaInputAttribute.builder().matchKey("name").fieldName("name").type(SchemaAttributeType.

SchemaInputAttribute.builder().matchKey("email").fieldName("email").type(SchemaAttributeType

SchemaInputAttribute.builder().fieldName("phone").type(SchemaAttributeType.PROVIDER_ID).sub

);
 }

 CreateSchemaMappingRequest request = CreateSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .mappedInputFields(schemaAttributes)
 .build();

 return getResolutionAsyncClient().createSchemaMapping(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("[{}] schema mapping Created Successfully!",
schemaName);
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while creating the schema mapping.", null);
 }
 }
 })
 }

```



```

 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflicting schema mapping
already exists. Resolve conflicts before proceeding.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to create schema mapping:
" + exception.getMessage(), exception);
 }
 });
}

```

- For API details, see [CreateSchemaMapping](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteMatchingWorkflow

The following code example shows how to use DeleteMatchingWorkflow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously deletes a workflow with the specified name.
 *
 * @param workflowName the name of the workflow to be deleted
 * @return a {@link CompletableFuture} that completes when the workflow has been
deleted
 * @throws RuntimeException if the deletion of the workflow fails
 */
public CompletableFuture<DeleteMatchingWorkflowResponse>
deleteMatchingWorkflowAsync(String workflowName) {
 DeleteMatchingWorkflowRequest request =
DeleteMatchingWorkflowRequest.builder()
 .workflowName(workflowName)

```

```

 .build();

 return getResolutionAsyncClient().deleteMatchingWorkflow(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("{} was deleted", workflowName);
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while deleting the workflow.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The workflow to delete was
not found.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to delete workflow: " +
exception.getMessage(), exception);
 }
 });
}

```

- For API details, see [DeleteMatchingWorkflow](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteSchemaMapping

The following code example shows how to use DeleteSchemaMapping.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Deletes the schema mapping asynchronously.
 *
 * @param schemaName the name of the schema to delete
 * @return a {@link CompletableFuture} that completes when the schema mapping is
 deleted successfully,
 * or throws a {@link RuntimeException} if the deletion fails
 */
 public CompletableFuture<DeleteSchemaMappingResponse>
deleteSchemaMappingAsync(String schemaName) {
 DeleteSchemaMappingRequest request = DeleteSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .build();

 return getResolutionAsyncClient().deleteSchemaMapping(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully deleted the schema mapping, log the success
message.
 logger.info("Schema mapping '{}' deleted successfully.",
schemaName);
 } else {
 // Ensure exception is not null before accessing its cause.
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while deleting the schema mapping.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The schema mapping was not
found to delete: " + schemaName, cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to delete schema mapping:
" + schemaName, exception);
 }
 });
 }

```

- For API details, see [DeleteSchemaMapping](#) in *AWS SDK for Java 2.x API Reference*.

## GetMatchingJob

The following code example shows how to use `GetMatchingJob`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously retrieves a matching job based on the provided job ID and
 * workflow name.
 *
 * @param jobId the ID of the job to retrieve
 * @param workflowName the name of the workflow associated with the job
 * @return a {@link CompletableFuture} that completes when the job information
 * is available or an exception occurs
 */
public CompletableFuture<GetMatchingJobResponse> getMatchingJobAsync(String
jobId, String workflowName) {
 GetMatchingJobRequest request = GetMatchingJobRequest.builder()
 .jobId(jobId)
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().getMatchingJob(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully fetched the matching job details, log the job
 status.

 logger.info("Job status: " + response.status());
 logger.info("Job details: " + response.toString());
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while fetching the matching job.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
```

```

 throw new CompletionException("The requested job could not
be found.", cause);
 }

 // Wrap other exceptions in a CompletionException with the
message.
 throw new CompletionException("Error fetching matching job: " +
exception.getMessage(), exception);
 }
});
}

```

- For API details, see [GetMatchingJob](#) in *AWS SDK for Java 2.x API Reference*.

## GetSchemaMapping

The following code example shows how to use GetSchemaMapping.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the schema mapping asynchronously.
 *
 * @param schemaName the name of the schema to retrieve the mapping for
 * @return a {@link CompletableFuture} that completes with the {@link
GetSchemaMappingResponse} when the operation
 * is complete
 * @throws RuntimeException if the schema mapping retrieval fails
 */
public CompletableFuture<GetSchemaMappingResponse> getSchemaMappingAsync(String
schemaName) {
 GetSchemaMappingRequest mappingRequest = GetSchemaMappingRequest.builder()
 .schemaName(schemaName)
 .build();
}

```

```

 return getResolutionAsyncClient().getSchemaMapping(mappingRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 response.mappedInputFields().forEach(attribute ->
 logger.info("Attribute Name: " + attribute.fieldName() +
 ", Attribute Type: " + attribute.type().toString()));
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while getting schema mapping.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested schema mapping
was not found.", cause);
 }

 // Wrap other exceptions in a CompletionException with the
message.
 throw new CompletionException("Failed to get schema mapping: " +
exception.getMessage(), exception);
 }
 });
 }
}

```

- For API details, see [GetSchemaMapping](#) in *AWS SDK for Java 2.x API Reference*.

## ListSchemaMappings

The following code example shows how to use `ListSchemaMappings`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Lists the schema mappings associated with the current AWS account. This
 method uses an asynchronous paginator to
 * retrieve the schema mappings, and prints the name of each schema mapping to
 the console.
 */
 public void ListSchemaMappings() {
 ListSchemaMappingsRequest mappingsRequest =
 ListSchemaMappingsRequest.builder()
 .build();

 ListSchemaMappingsPublisher paginator =
 getResolutionAsyncClient().listSchemaMappingsPaginator(mappingsRequest);

 // Iterate through the pages of results
 CompletableFuture<Void> future = paginator.subscribe(response -> {
 response.schemaList().forEach(schemaMapping ->
 logger.info("Schema Mapping Name: " + schemaMapping.schemaName())
);
 });

 // Wait for the asynchronous operation to complete
 future.join();
 }

```

- For API details, see [ListSchemaMappings](#) in *AWS SDK for Java 2.x API Reference*.

## StartMatchingJob

The following code example shows how to use StartMatchingJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Starts a matching job asynchronously for the specified workflow name.

```

```

*
* @param workflowName the name of the workflow for which to start the matching
job
* @return a {@link CompletableFuture} that completes with the job ID of the
started matching job, or an empty
* string if the operation fails
*/
public CompletableFuture<String> startMatchingJobAsync(String workflowName) {
 StartMatchingJobRequest jobRequest = StartMatchingJobRequest.builder()
 .workflowName(workflowName)
 .build();

 return getResolutionAsyncClient().startMatchingJob(jobRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 String jobId = response.jobId();
 logger.info("Job ID: " + jobId);
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while starting the job.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("The job is already running.
Resolve conflicts before starting a new job.", cause);
 }

 // Wrap other AWS exceptions in a CompletionException.
 throw new CompletionException("Failed to start the job: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response != null ? response.jobId() : "");
}

```

- For API details, see [StartMatchingJob](#) in *AWS SDK for Java 2.x API Reference*.

## TagResource

The following code example shows how to use TagResource.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Tags the specified schema mapping ARN.
 *
 * @param schemaMappingARN the ARN of the schema mapping to tag
 */
public CompletableFuture<TagResourceResponse> tagEntityResource(String
schemaMappingARN) {
 Map<String, String> tags = new HashMap<>();
 tags.put("tag1", "tag1Value");
 tags.put("tag2", "tag2Value");

 TagResourceRequest request = TagResourceRequest.builder()
 .resourceArn(schemaMappingARN)
 .tags(tags)
 .build();

 return getResolutionAsyncClient().tagResource(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 // Successfully tagged the resource, log the success message.
 logger.info("Successfully tagged the resource.");
 } else {
 if (exception == null) {
 throw new CompletionException("An unknown error occurred
while tagging the resource.", null);
 }

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource to tag was not
found.", cause);
 }
 throw new CompletionException("Failed to tag the resource: " +
exception.getMessage(), exception);
 }
 });
}
```

```
 }
 });
}
```

- For API details, see [TagResource](#) in *AWS SDK for Java 2.x API Reference*.

## OpenSearch Service examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with OpenSearch Service.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello OpenSearch Service

The following code example shows how to get started using OpenSearch Service.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.opensearch.OpenSearchAsyncClient;
import software.amazon.awssdk.services.opensearch.model.ListVersionsRequest;
import java.util.List;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloOpenSearch {
 public static void main(String[] args) {
 try {
 CompletableFuture<Void> future = listVersionsAsync();
 future.join();
 System.out.println("Versions listed successfully.");
 } catch (RuntimeException e) {
 System.err.println("Error occurred while listing versions: " +
e.getMessage());
 }
 }

 private static OpenSearchAsyncClient getAsyncClient() {
 return OpenSearchAsyncClient.builder().build();
 }

 public static CompletableFuture<Void> listVersionsAsync() {
 ListVersionsRequest request = ListVersionsRequest.builder()
 .maxResults(10)
 .build();

 return getAsyncClient().listVersions(request).thenAccept(response -> {
 List<String> versionList = response.versions();
 for (String version : versionList) {
 System.out.println("Version info: " + version);
 }
 }).exceptionally(ex -> {
 // Handle the exception, or propagate it as a RuntimeException
 throw new RuntimeException("Failed to list versions", ex);
 });
 }
}
```

```
}
}
```

- For API details, see [ListVersions](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn OpenSearch Service core operations

The following code example shows how to:

- Create an OpenSearch Service domain.
- Provides detailed information about a specific OpenSearch Service domain.
- Lists all the OpenSearch Service domains owned by the account.
- Waits until the OpenSearch Service domain's change status reaches a completed state.
- Modifies the configuration of an existing OpenSearch Service domain.
- Add a tag to the OpenSearch Service domain.
- Lists the tags associated with an OpenSearch Service domain.
- Removes tags from an OpenSearch Service domain.
- Deletes the OpenSearch Service domain.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating OpenSearch Service features.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.opensearch.model.*;
import java.util.List;
import java.util.Scanner;
```

```
import java.util.concurrent.CompletableFuture;

public class OpenSearchScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 private static final Logger logger =
 LoggerFactory.getLogger(OpenSearchScenario.class);
 static Scanner scanner = new Scanner(System.in);

 static OpenSearchActions openSearchActions = new OpenSearchActions();

 public static void main(String[] args) throws Throwable {
 logger.info("""
 Welcome to the Amazon OpenSearch Service Basics Scenario.

 Use the Amazon OpenSearch Service API to create, configure, and manage
 OpenSearch Service domains.

 The operations exposed by the AWS OpenSearch Service client are focused
 on managing the OpenSearch Service domains
 and their configurations, not the data within the domains (such as
 indexing or querying documents).

 For document management, you typically interact directly with the
 OpenSearch REST API or use other libraries,
 such as the OpenSearch Java client (https://opensearch.org/docs/latest/clients/java/).

 Let's get started...
 """);
 waitForInputToContinue(scanner);
 try {
 runScenario();
 } catch (RuntimeException e) {
 e.printStackTrace();
 }
 }

 private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();
 }
 }
}
```

```

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 logger.info("Invalid input. Please try again.");
 }
 }
}

private static void runScenario() throws Throwable {
 String currentTimestamp = String.valueOf(System.currentTimeMillis());
 String domainName = "test-domain-" + currentTimestamp;

 logger.info(DASHES);
 logger.info("1. Create an Amazon OpenSearch domain");
 logger.info("""
 An Amazon OpenSearch domain is a managed instance of the OpenSearch
engine,
 which is an open-source search and analytics engine derived from
Elasticsearch.
 An OpenSearch domain is essentially a cluster of compute resources and
storage that hosts
 one or more OpenSearch indexes, enabling you to perform full-text
searches, data analysis, and
 visualizations.

 In this step, we'll initiate the creation of the domain. We'll check on
the progress in a later step.
 """);
 waitForInputToContinue(scanner);

 try {
 CompletableFuture<String> future =
openSearchActions.createNewDomainAsync(domainName);
 String domainId = future.join();
 logger.info("Domain successfully created with ID: {}", domainId);
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause != null) {
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.error("OpenSearch error occurred: Error message:
{}, Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());

```

```

 } else {
 logger.error("An unexpected error occurred: " +
cause.getMessage(), cause);
 }
 } else {
 logger.error("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("2. Describe the Amazon OpenSearch domain");
logger.info("In this step, we get back the Domain ARN which is used in an
upcoming step.");
waitForInputToContinue(scanner);

String arn = "";
try {
 CompletableFuture<String> future =
openSearchActions.describeDomainAsync(domainName);
 arn = future.join();
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}, Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("3. List the domains in your account");
waitForInputToContinue(scanner);

try {
 CompletableFuture<List<DomainInfo>> future =
openSearchActions.listAllDomainsAsync();
 List<DomainInfo> domainInfoList = future.join();
 for (DomainInfo domain : domainInfoList) {

```

```

 logger.info("Domain name is: " + domain.domainName());
 }
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 while (cause.getCause() != null && !(cause instanceof
OpenSearchException)) {
 cause = cause.getCause();
 }
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}", Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
}

waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("4. Wait until the domain's change status reaches a completed
state");
logger.info("""
 In this step, we check on the change status of the domain that we
initiated in Step 1.
 Until we reach a COMPLETED state, we stay in a loop by sending a
DescribeDomainChangeProgressRequest.

 The time it takes for a change to an OpenSearch domain to reach a
completed state can range
 from a few minutes to several hours. In this case the change is creating
a new domain that we initiated in Step 1.
 The time varies depending on the complexity of the change and the
current load on
 the OpenSearch service. In general, simple changes, such as scaling the
number of data nodes or
 updating the OpenSearch version, may take 10-30 minutes.
""");

waitForInputToContinue(scanner);

try {

```



```

 CompletableFuture<Void> future =
openSearchActions.domainChangeProgressAsync(domainName);
 future.join();
 logger.info("Domain change progress completed successfully.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 while (cause.getCause() != null && !(cause instanceof
ResourceNotFoundException)) {
 cause = cause.getCause();
 }
 if (cause instanceof ResourceNotFoundException
resourceNotFoundException) {
 logger.info("The specific AWS resource was not found: Error message:
{}", Error code {}", resourceNotFoundException.awsErrorDetails().errorMessage(),
resourceNotFoundException.awsErrorDetails().errorCode());

 if (cause instanceof OpenSearchException ex) {
 logger.info("An OpenSearch error occurred: Error message: " +
ex.getMessage());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("5. Modify the domain");
 logger.info("""
 You can change your OpenSearch domain's settings, like the number of
instances, without starting over from scratch.
 This makes it easy to adjust your domain as your needs change, allowing
you to scale up or
 down quickly without recreating everything.

 We modify the domain in this step by changing the number of instances.
 """);

 waitForInputToContinue(scanner);

 try {
 CompletableFuture<UpdateDomainConfigResponse> future =
openSearchActions.updateSpecificDomainAsync(domainName);

```

```

 UpdateDomainConfigResponse updateResponse = future.join();
 logger.info("Domain update status: " +
updateResponse.domainConfig().changeProgressDetails().configChangeStatusAsString());
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}, Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("6. Wait until the domain's change status reaches a completed
state");
 logger.info(""""
 In this step, we poll the status until the domain's change status
reaches a completed state.
 """);

 waitForInputToContinue(scanner);

 try {
 CompletableFuture<Void> future =
openSearchActions.domainChangeProgressAsync(domainName);
 future.join();
 logger.info("Domain change progress completed successfully.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof OpenSearchException ex) {
 logger.info("EC2 error occurred: Error message: " +ex.getMessage());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("7. Tag the Domain");

```

```

 logger.info("""
 Tags let you assign arbitrary information to an Amazon OpenSearch
 Service domain so you can
 categorize and filter on that information. A tag is a key-value pair
 that you define and
 associate with an OpenSearch Service domain. You can use these tags to
 track costs by grouping
 expenses for similarly tagged resources.

 In this scenario, we create tags with keys "service" and "instances".
 """);

 waitForInputToContinue(scanner);

 try {
 CompletableFuture<AddTagsResponse> future =
openSearchActions.addDomainTagsAsync(arn);
 future.join();
 logger.info("Domain tags added successfully.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 while (cause.getCause() != null && !(cause instanceof
OpenSearchException)) {
 cause = cause.getCause();
 }
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}, Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 if (cause != null) {
 if (cause instanceof OpenSearchException) {
 logger.error("OpenSearch error occurred: Error message: " +
cause.getMessage(), cause);
 } else {
 logger.error("An unexpected error occurred: " +
cause.getMessage(), cause);
 }
 } else {
 logger.error("An unexpected error occurred: " + rt.getMessage(),
rt);
 }
 throw cause;
 }
 }
 }
}

```

```

 }
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("8. List Domain tags");
waitForInputToContinue(scanner);

try {
 CompletableFuture<ListTagsResponse> future =
openSearchActions.listDomainTagsAsync(arn);
 ListTagsResponse listTagsResponse = future.join();
 listTagsResponse.tagList().forEach(tag -> logger.info("Tag Key: " +
tag.key() + ", Tag Value: " + tag.value()));
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 while (cause.getCause() != null && !(cause instanceof
OpenSearchException)) {
 cause = cause.getCause();
 }
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}", Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
}

waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("9. Delete the domain");
logger.info("""
 In this step, we'll delete the Amazon OpenSearch domain that we created
in Step 1.
 Deleting a domain will remove all data and configuration for that
domain.
 """);

waitForInputToContinue(scanner);

```

```

 try {
 CompletableFuture<DeleteDomainResponse> future =
openSearchActions.deleteSpecificDomainAsync(domainName);
 future.join();
 logger.info("Domain successfully deleted.");
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 while (cause.getCause() != null && !(cause instanceof
OpenSearchException)) {
 cause = cause.getCause();
 }
 if (cause instanceof OpenSearchException openSearchEx) {
 logger.info("OpenSearch error occurred: Error message:
{}, Error code {}", openSearchEx.awsErrorDetails().errorMessage(),
openSearchEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("Scenario complete!");
 }
}

```

A wrapper class for OpenSearch Service SDK methods.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryPolicy;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.opensearch.OpenSearchAsyncClient;
import software.amazon.awssdk.services.opensearch.model.AddTagsRequest;
import software.amazon.awssdk.services.opensearch.model.AddTagsResponse;
import software.amazon.awssdk.services.opensearch.model.ClusterConfig;
import software.amazon.awssdk.services.opensearch.model.CreateDomainRequest;

```

```
import software.amazon.awssdk.services.opensearch.model.DeleteDomainRequest;
import software.amazon.awssdk.services.opensearch.model.DeleteDomainResponse;
import
 software.amazon.awssdk.services.opensearch.model.DescribeDomainChangeProgressRequest;
import
 software.amazon.awssdk.services.opensearch.model.DescribeDomainChangeProgressResponse;
import software.amazon.awssdk.services.opensearch.model.DescribeDomainRequest;
import software.amazon.awssdk.services.opensearch.model.DomainInfo;
import software.amazon.awssdk.services.opensearch.model.DomainStatus;
import software.amazon.awssdk.services.opensearch.model.EBSOptions;
import software.amazon.awssdk.services.opensearch.model.ListDomainNamesRequest;
import software.amazon.awssdk.services.opensearch.model.ListTagsRequest;
import software.amazon.awssdk.services.opensearch.model.ListTagsResponse;
import software.amazon.awssdk.services.opensearch.model.NodeToNodeEncryptionOptions;
import software.amazon.awssdk.services.opensearch.model.Tag;
import software.amazon.awssdk.services.opensearch.model.UpdateDomainConfigRequest;
import software.amazon.awssdk.services.opensearch.model.UpdateDomainConfigResponse;
import software.amazon.awssdk.services.opensearch.model.VolumeType;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class OpenSearchActions {
 private static final Logger logger =
 LoggerFactory.getLogger(OpenSearchActions.class);
 private static OpenSearchAsyncClient openSearchClientAsyncClient;
 private static OpenSearchAsyncClient getAsyncClient() {
 if (openSearchClientAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();
 }
 }
```

```

 openSearchClientAsyncClient = OpenSearchAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return openSearchClientAsyncClient;
}

/**
 * Creates a new OpenSearch domain asynchronously.
 * @param domainName the name of the new OpenSearch domain to create
 * @return a {@link CompletableFuture} containing the domain ID of the newly
created domain
 */
public CompletableFuture<String> createNewDomainAsync(String domainName) {
 ClusterConfig clusterConfig = ClusterConfig.builder()
 .dedicatedMasterEnabled(true)
 .dedicatedMasterCount(3)
 .dedicatedMasterType("t2.small.search")
 .instanceType("t2.small.search")
 .instanceCount(5)
 .build();

 EBSOptions ebsOptions = EBSOptions.builder()
 .ebsEnabled(true)
 .volumeSize(10)
 .volumeType(VolumeType.GP2)
 .build();

 NodeToNodeEncryptionOptions encryptionOptions =
NodeToNodeEncryptionOptions.builder()
 .enabled(true)
 .build();

 CreateDomainRequest domainRequest = CreateDomainRequest.builder()
 .domainName(domainName)
 .engineVersion("OpenSearch_1.0")
 .clusterConfig(clusterConfig)
 .ebsOptions(ebsOptions)
 .nodeToNodeEncryptionOptions(encryptionOptions)
 .build();
 logger.info("Sending domain creation request...");
}

```

```

 return getAsyncClient().createDomain(domainRequest)
 .handle((createResponse, throwable) -> {
 if (createResponse != null) {
 logger.info("Domain status is {}",
createResponse.domainStatus().changeProgressDetails().configChangeStatusAsString());
 logger.info("Domain Id is {}",
createResponse.domainStatus().domainId());
 return createResponse.domainStatus().domainId();
 }
 throw new RuntimeException("Failed to create domain",
throwable);
 });
 }

 /**
 * Deletes a specific domain asynchronously.
 * @param domainName the name of the domain to be deleted
 * @return a {@link CompletableFuture} that completes when the domain has been
deleted
 * or throws a {@link RuntimeException} if the deletion fails
 */
 public CompletableFuture<DeleteDomainResponse> deleteSpecificDomainAsync(String
domainName) {
 DeleteDomainRequest domainRequest = DeleteDomainRequest.builder()
 .domainName(domainName)
 .build();

 // Delete domain asynchronously
 return getAsyncClient().deleteDomain(domainRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the domain: " +
domainName, exception);
 }
 });
 }

 /**
 * Describes the specified domain asynchronously.
 *
 * @param domainName the name of the domain to describe
 * @return a {@link CompletableFuture} that completes with the ARN of the domain
 * @throws RuntimeException if the domain description fails
 */

```



```

 public CompletableFuture<String> describeDomainAsync(String domainName) {
 DescribeDomainRequest request = DescribeDomainRequest.builder()
 .domainName(domainName)
 .build();

 return getAsyncClient().describeDomain(request)
 .handle((response, exception) -> { // Handle both response and
exception
 if (exception != null) {
 throw new RuntimeException("Failed to describe domain",
exception);
 }
 DomainStatus domainStatus = response.domainStatus();
 String endpoint = domainStatus.endpoint();
 String arn = domainStatus.arn();
 String engineVersion = domainStatus.engineVersion();
 logger.info("Domain endpoint is: " + endpoint);
 logger.info("ARN: " + arn);
 System.out.println("Engine version: " + engineVersion);

 return arn; // Return ARN when successful
 });
 }

 /**
 * Asynchronously lists all the domains in the current AWS account.
 * @return a {@link CompletableFuture} that, when completed, contains a list of
{@link DomainInfo} objects representing
 * the domains in the account.
 * @throws RuntimeException if there was a failure while listing the domains.
 */
 public CompletableFuture<List<DomainInfo>> listAllDomainsAsync() {
 ListDomainNamesRequest namesRequest = ListDomainNamesRequest.builder()
 .engineType("OpenSearch")
 .build();

 return getAsyncClient().listDomainNames(namesRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to list all domains",
exception);
 }
 return response.domainNames(); // Return the list of domain names
on success
 });
 }

```

```

 });
 }

 /**
 * Updates the configuration of a specific domain asynchronously.
 * @param domainName the name of the domain to update
 * @return a {@link CompletableFuture} that represents the asynchronous
 operation of updating the domain configuration
 */
 public CompletableFuture<UpdateDomainConfigResponse>
updateSpecificDomainAsync(String domainName) {
 ClusterConfig clusterConfig = ClusterConfig.builder()
 .instanceCount(3)
 .build();

 UpdateDomainConfigRequest updateDomainConfigRequest =
UpdateDomainConfigRequest.builder()
 .domainName(domainName)
 .clusterConfig(clusterConfig)
 .build();

 return getAsyncClient().updateDomainConfig(updateDomainConfigRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to update the domain
configuration", exception);
 }
 // Handle success if needed (e.g., logging or additional actions)
 });
 }

 /**
 * Asynchronously checks the progress of a domain change operation in Amazon
 OpenSearch Service.
 * @param domainName the name of the OpenSearch domain to check the progress for
 * @return a {@link CompletableFuture} that completes when the domain change
 operation is completed
 */
 public CompletableFuture<Void> domainChangeProgressAsync(String domainName) {
 DescribeDomainChangeProgressRequest request =
DescribeDomainChangeProgressRequest.builder()
 .domainName(domainName)
 .build();
 }

```

```

 return CompletableFuture.runAsync(() -> {
 boolean isCompleted = false;
 long startTime = System.currentTimeMillis();

 while (!isCompleted) {
 try {
 // Handle the async client call using `join` to block
 // synchronously for the result
 DescribeDomainChangeProgressResponse response = getAsyncClient()
 .describeDomainChangeProgress(request)
 .handle((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to check domain
progress", ex);
 }
 return resp;
 }).join();

 String state = response.changeProgressStatus().statusAsString();
 // Get the status as string

 if ("COMPLETED".equals(state)) {
 logger.info("\nOpenSearch domain status: Completed");
 isCompleted = true;
 } else {
 for (int i = 0; i < 5; i++) {
 long elapsedTimeInSeconds = (System.currentTimeMillis()
- startTime) / 1000;

 String formattedTime = String.format("%02d:%02d",
elapsedTimeInSeconds / 60, elapsedTimeInSeconds % 60);
 System.out.print("\rOpenSearch domain state: " + state +
" | Time Elapsed: " + formattedTime + " ");
 System.out.flush();
 Thread.sleep(1_000);
 }
 }
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException("Thread was interrupted", e);
 }
 }
 });
}

```

```

/**
 * Asynchronously adds tags to an Amazon OpenSearch Service domain.
 * @param domainARN the Amazon Resource Name (ARN) of the Amazon OpenSearch
Service domain to add tags to
 * @return a {@link CompletableFuture} that completes when the tags have been
successfully added to the domain,
 * or throws a {@link RuntimeException} if the operation fails
 */
public CompletableFuture<AddTagsResponse> addDomainTagsAsync(String domainARN) {
 Tag tag1 = Tag.builder()
 .key("service")
 .value("OpenSearch")
 .build();

 Tag tag2 = Tag.builder()
 .key("instances")
 .value("m3.2xlarge")
 .build();

 List<Tag> tagList = new ArrayList<>();
 tagList.add(tag1);
 tagList.add(tag2);

 AddTagsRequest addTagsRequest = AddTagsRequest.builder()
 .arn(domainARN)
 .tagList(tagList)
 .build();

 return getAsyncClient().addTags(addTagsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to add tags to the domain: "
+ domainARN, exception);
 } else {
 logger.info("Added Tags");
 }
 });
}

/**
 * Asynchronously lists the tags associated with the specified Amazon Resource
Name (ARN).

```

```

 * @param arn the Amazon Resource Name (ARN) of the resource for which to list
 the tags
 * @return a {@link CompletableFuture} that, when completed, will contain a list
 of the tags associated with the
 * specified ARN
 * @throws RuntimeException if there is an error listing the tags
 */
 public CompletableFuture<ListTagsResponse> listDomainTagsAsync(String arn) {
 ListTagsRequest tagsRequest = ListTagsRequest.builder()
 .arn(arn)
 .build();

 return getAsyncClient().listTags(tagsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to list domain tags",
exception);
 }

 List<Tag> tagList = response.tagList();
 for (Tag tag : tagList) {
 logger.info("Tag key is " + tag.key());
 logger.info("Tag value is " + tag.value());
 }
 });
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AddTags](#)
  - [CreateDomain](#)
  - [DeleteDomain](#)
  - [DescribeDomain](#)
  - [DescribeDomainChangeProgress](#)
  - [ListDomainNames](#)
  - [ListTags](#)
  - [UpdateDomainConfig](#)

# Actions

## AddTags

The following code example shows how to use AddTags.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously adds tags to an Amazon OpenSearch Service domain.
 * @param domainARN the Amazon Resource Name (ARN) of the Amazon OpenSearch
Service domain to add tags to
 * @return a {@link CompletableFuture} that completes when the tags have been
successfully added to the domain,
 * or throws a {@link RuntimeException} if the operation fails
 */
public CompletableFuture<AddTagsResponse> addDomainTagsAsync(String domainARN) {
 Tag tag1 = Tag.builder()
 .key("service")
 .value("OpenSearch")
 .build();

 Tag tag2 = Tag.builder()
 .key("instances")
 .value("m3.2xlarge")
 .build();

 List<Tag> tagList = new ArrayList<>();
 tagList.add(tag1);
 tagList.add(tag2);

 AddTagsRequest addTagsRequest = AddTagsRequest.builder()
 .arn(domainARN)
 .tagList(tagList)
 .build();
```

```

 return getAsyncClient().addTags(addTagsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to add tags to the domain: "
+ domainARN, exception);
 } else {
 logger.info("Added Tags");
 }
 });
 }
}

```

- For API details, see [AddTags](#) in *AWS SDK for Java 2.x API Reference*.

## ChangeProgress

The following code example shows how to use ChangeProgress.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously checks the progress of a domain change operation in Amazon
 * OpenSearch Service.
 * @param domainName the name of the OpenSearch domain to check the progress for
 * @return a {@link CompletableFuture} that completes when the domain change
 * operation is completed
 */
public CompletableFuture<Void> domainChangeProgressAsync(String domainName) {
 DescribeDomainChangeProgressRequest request =
DescribeDomainChangeProgressRequest.builder()
 .domainName(domainName)
 .build();

 return CompletableFuture.runAsync(() -> {
 boolean isCompleted = false;

```

```

 long startTime = System.currentTimeMillis();

 while (!isCompleted) {
 try {
 // Handle the async client call using `join` to block
 // synchronously for the result
 DescribeDomainChangeProgressResponse response = getAsyncClient()
 .describeDomainChangeProgress(request)
 .handle((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to check domain
progress", ex);
 }
 return resp;
 }).join();

 String state = response.changeProgressStatus().statusAsString();
 // Get the status as string

 if ("COMPLETED".equals(state)) {
 logger.info("\nOpenSearch domain status: Completed");
 isCompleted = true;
 } else {
 for (int i = 0; i < 5; i++) {
 long elapsedTimeInSeconds = (System.currentTimeMillis()
- startTime) / 1000;
 String formattedTime = String.format("%02d:%02d",
elapsedTimeInSeconds / 60, elapsedTimeInSeconds % 60);
 System.out.print("\rOpenSearch domain state: " + state +
" | Time Elapsed: " + formattedTime + " ");
 System.out.flush();
 Thread.sleep(1_000);
 }
 }
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException("Thread was interrupted", e);
 }
 }
 });
}

```

- For API details, see [ChangeProgress](#) in *AWS SDK for Java 2.x API Reference*.



## CreateDomain

The following code example shows how to use CreateDomain.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new OpenSearch domain asynchronously.
 * @param domainName the name of the new OpenSearch domain to create
 * @return a {@link CompletableFuture} containing the domain ID of the newly
created domain
 */
public CompletableFuture<String> createNewDomainAsync(String domainName) {
 ClusterConfig clusterConfig = ClusterConfig.builder()
 .dedicatedMasterEnabled(true)
 .dedicatedMasterCount(3)
 .dedicatedMasterType("t2.small.search")
 .instanceType("t2.small.search")
 .instanceCount(5)
 .build();

 EBSOptions ebsOptions = EBSOptions.builder()
 .ebsEnabled(true)
 .volumeSize(10)
 .volumeType(VolumeType.GP2)
 .build();

 NodeToNodeEncryptionOptions encryptionOptions =
NodeToNodeEncryptionOptions.builder()
 .enabled(true)
 .build();

 CreateDomainRequest domainRequest = CreateDomainRequest.builder()
 .domainName(domainName)
 .engineVersion("OpenSearch_1.0")
 .clusterConfig(clusterConfig)
 .ebsOptions(ebsOptions)
```

```

 .nodeToNodeEncryptionOptions(encryptionOptions)
 .build();
 logger.info("Sending domain creation request...");
 return getAsyncClient().createDomain(domainRequest)
 .handle((createResponse, throwable) -> {
 if (createResponse != null) {
 logger.info("Domain status is {}",
 createResponse.domainStatus().changeProgressDetails().configChangeStatusAsString());
 logger.info("Domain Id is {}",
 createResponse.domainStatus().domainId());
 return createResponse.domainStatus().domainId();
 }
 throw new RuntimeException("Failed to create domain",
 throwable);
 });
}

```

- For API details, see [CreateDomain](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDomain

The following code example shows how to use DeleteDomain.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a specific domain asynchronously.
 * @param domainName the name of the domain to be deleted
 * @return a {@link CompletableFuture} that completes when the domain has been
 * deleted
 * or throws a {@link RuntimeException} if the deletion fails
 */
public CompletableFuture<DeleteDomainResponse> deleteSpecificDomainAsync(String
domainName) {
 DeleteDomainRequest domainRequest = DeleteDomainRequest.builder()

```

```

 .domainName(domainName)
 .build();

 // Delete domain asynchronously
 return getAsyncClient().deleteDomain(domainRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to delete the domain: " +
domainName, exception);
 }
 });
}

```

- For API details, see [DeleteDomain](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDomain

The following code example shows how to use DescribeDomain.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Describes the specified domain asynchronously.
 *
 * @param domainName the name of the domain to describe
 * @return a {@link CompletableFuture} that completes with the ARN of the domain
 * @throws RuntimeException if the domain description fails
 */
public CompletableFuture<String> describeDomainAsync(String domainName) {
 DescribeDomainRequest request = DescribeDomainRequest.builder()
 .domainName(domainName)
 .build();

 return getAsyncClient().describeDomain(request)

```

```

 .handle((response, exception) -> { // Handle both response and
exception
 if (exception != null) {
 throw new RuntimeException("Failed to describe domain",
exception);
 }
 DomainStatus domainStatus = response.domainStatus();
 String endpoint = domainStatus.endpoint();
 String arn = domainStatus.arn();
 String engineVersion = domainStatus.engineVersion();
 logger.info("Domain endpoint is: " + endpoint);
 logger.info("ARN: " + arn);
 System.out.println("Engine version: " + engineVersion);

 return arn; // Return ARN when successful
 });
 }
}

```

- For API details, see [DescribeDomain](#) in *AWS SDK for Java 2.x API Reference*.

## ListDomainNames

The following code example shows how to use `ListDomainNames`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously lists all the domains in the current AWS account.
 * @return a {@link CompletableFuture} that, when completed, contains a list of
{@link DomainInfo} objects representing
 * the domains in the account.
 * @throws RuntimeException if there was a failure while listing the domains.
 */
public CompletableFuture<List<DomainInfo>> listAllDomainsAsync() {
 ListDomainNamesRequest namesRequest = ListDomainNamesRequest.builder()

```

```

 .engineType("OpenSearch")
 .build();

 return getAsyncClient().listDomainNames(namesRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to list all domains",
exception);
 }
 return response.domainNames(); // Return the list of domain names
on success
 });
 }

```

- For API details, see [ListDomainNames](#) in *AWS SDK for Java 2.x API Reference*.

## ListTags

The following code example shows how to use ListTags.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously adds tags to an Amazon OpenSearch Service domain.
 * @param domainARN the Amazon Resource Name (ARN) of the Amazon OpenSearch
Service domain to add tags to
 * @return a {@link CompletableFuture} that completes when the tags have been
successfully added to the domain,
 * or throws a {@link RuntimeException} if the operation fails
 */
public CompletableFuture<AddTagsResponse> addDomainTagsAsync(String domainARN) {
 Tag tag1 = Tag.builder()
 .key("service")
 .value("OpenSearch")

```

```

 .build();

 Tag tag2 = Tag.builder()
 .key("instances")
 .value("m3.2xlarge")
 .build();

 List<Tag> tagList = new ArrayList<>();
 tagList.add(tag1);
 tagList.add(tag2);

 AddTagsRequest addTagsRequest = AddTagsRequest.builder()
 .arn(domainARN)
 .tagList(tagList)
 .build();

 return getAsyncClient().addTags(addTagsRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to add tags to the domain: "
+ domainARN, exception);
 } else {
 logger.info("Added Tags");
 }
 });
}

```

- For API details, see [ListTags](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateDomainConfig

The following code example shows how to use UpdateDomainConfig.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Updates the configuration of a specific domain asynchronously.
 * @param domainName the name of the domain to update
 * @return a {@link CompletableFuture} that represents the asynchronous
operation of updating the domain configuration
 */
public CompletableFuture<UpdateDomainConfigResponse>
updateSpecificDomainAsync(String domainName) {
 ClusterConfig clusterConfig = ClusterConfig.builder()
 .instanceCount(3)
 .build();

 UpdateDomainConfigRequest updateDomainConfigRequest =
UpdateDomainConfigRequest.builder()
 .domainName(domainName)
 .clusterConfig(clusterConfig)
 .build();

 return getAsyncClient().updateDomainConfig(updateDomainConfigRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Failed to update the domain
configuration", exception);
 }
 // Handle success if needed (e.g., logging or additional actions)
 });
}

```

- For API details, see [UpdateDomainConfig](#) in *AWS SDK for Java 2.x API Reference*.

## EventBridge examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with EventBridge.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello EventBridge

The following code example shows how to get started using EventBridge.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloEventBridge {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 EventBridgeClient eventBrClient = EventBridgeClient.builder()
```



```
 .region(region)
 .build();

 listBuses(eventBrClient);
 eventBrClient.close();
}

public static void listBuses(EventBridgeClient eventBrClient) {
 try {
 ListEventBusesRequest busesRequest = ListEventBusesRequest.builder()
 .limit(10)
 .build();

 ListEventBusesResponse response =
eventBrClient.listEventBuses(busesRequest);
 List<EventBus> buses = response.eventBuses();
 for (EventBus bus : buses) {
 System.out.println("The name of the event bus is: " + bus.name());
 System.out.println("The ARN of the event bus is: " + bus.arn());
 }

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
```

- For API details, see [ListEventBuses](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a rule and add a target to it.
- Enable and disable rules.
- List and update rules and targets.
- Send events, then clean up resources.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java code example performs the following tasks:
 *
 * This Java V2 example performs the following tasks with Amazon EventBridge:
 *
 * 1. Creates an AWS Identity and Access Management (IAM) role to use with
 * Amazon EventBridge.
 * 2. Amazon Simple Storage Service (Amazon S3) bucket with EventBridge events
 * enabled.
 * 3. Creates a rule that triggers when an object is uploaded to Amazon S3.
 * 4. Lists rules on the event bus.
 * 5. Creates a new Amazon Simple Notification Service (Amazon SNS) topic and
 * lets the user subscribe to it.
 * 6. Adds a target to the rule that sends an email to the specified topic.
 * 7. Creates an EventBridge event that sends an email when an Amazon S3 object
 * is created.
 * 8. Lists Targets.
 * 9. Lists the rules for the same target.
 * 10. Triggers the rule by uploading a file to the Amazon S3 bucket.
 * 11. Disables a specific rule.
 * 12. Checks and print the state of the rule.
 * 13. Adds a transform to the rule to change the text of the email.
 * 14. Enables a specific rule.
 * 15. Triggers the updated rule by uploading a file to the Amazon S3 bucket.
 * 16. Updates the rule to be a custom rule pattern.
 * 17. Sending an event to trigger the rule.
 * 18. Cleans up resources.
```

```

*
*/
public class EventbridgeMVP {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws InterruptedException, IOException
 {
 final String usage = ""

 Usage:
 <roleName> <bucketName> <topicName> <eventRuleName>

 Where:
 roleName - The name of the role to create.
 bucketName - The Amazon Simple Storage Service (Amazon S3)
bucket name to create.
 topicName - The name of the Amazon Simple Notification Service
(Amazon SNS) topic to create.
 eventRuleName - The Amazon EventBridge rule name to create.
 "";

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String polJSON = "{" +
 "\"Version\": \"2012-10-17\", " +
 "\"Statement\": [{" +
 "\"Effect\": \"Allow\", " +
 "\"Principal\": { " +
 "\"Service\": \"events.amazonaws.com\" " +
 "}, " +
 "\"Action\": \"sts:AssumeRole\" " +
 "}] " +
 "}";

 Scanner sc = new Scanner(System.in);
 String roleName = args[0];
 String bucketName = args[1];
 String topicName = args[2];
 String eventRuleName = args[3];

 Region region = Region.US_EAST_1;
 }
}

```

```
EventBridgeClient eventBrClient = EventBridgeClient.builder()
 .region(region)
 .build();

S3Client s3Client = S3Client.builder()
 .region(region)
 .build();

Region regionGl = Region.AWS_GLOBAL;
IamClient iam = IamClient.builder()
 .region(regionGl)
 .build();

SnsClient snsClient = SnsClient.builder()
 .region(region)
 .build();

System.out.println(DASHES);
System.out.println("Welcome to the Amazon EventBridge example scenario.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out
 .println("1. Create an AWS Identity and Access Management (IAM) role
to use with Amazon EventBridge.");
String roleArn = createIAMRole(iam, roleName, polJSON);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("2. Create an S3 bucket with EventBridge events
enabled.");
if (checkBucket(s3Client, bucketName)) {
 System.out.println("Bucket " + bucketName + " already exists. Ending
this scenario.");
 System.exit(1);
}

createBucket(s3Client, bucketName);
Thread.sleep(3000);
setBucketNotification(s3Client, bucketName);
System.out.println(DASHES);

System.out.println(DASHES);
```

```
 System.out.println("3. Create a rule that triggers when an object is
uploaded to Amazon S3.");
 Thread.sleep(10000);
 addEventRule(eventBrClient, roleArn, bucketName, eventRuleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. List rules on the event bus.");
 listRules(eventBrClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Create a new SNS topic for testing and let the user
subscribe to the topic.");
 String topicArn = createSnsTopic(snsClient, topicName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Add a target to the rule that sends an email to the
specified topic.");
 System.out.println("Enter your email to subscribe to the Amazon SNS
topic:");
 String email = sc.nextLine();
 subEmail(snsClient, topicArn, email);
 System.out.println(
 "Use the link in the email you received to confirm your
subscription. Then, press Enter to continue.");
 sc.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Create an EventBridge event that sends an email when
an Amazon S3 object is created.");
 addSnsEventRule(eventBrClient, eventRuleName, topicArn, topicName,
eventRuleName, bucketName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(" 8. List Targets.");
 listTargets(eventBrClient, eventRuleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(" 9. List the rules for the same target.");
```

```
listTargetRules(eventBrClient, topicArn);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println(" 10. Trigger the rule by uploading a file to the S3
bucket.");
System.out.println("Press Enter to continue.");
sc.nextLine();
uploadTextFiletoS3(s3Client, bucketName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("11. Disable a specific rule.");
changeRuleState(eventBrClient, eventRuleName, false);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("12. Check and print the state of the rule.");
checkRule(eventBrClient, eventRuleName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("13. Add a transform to the rule to change the text of
the email.");
updateSnsEventRule(eventBrClient, topicArn, eventRuleName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("14. Enable a specific rule.");
changeRuleState(eventBrClient, eventRuleName, true);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println(" 15. Trigger the updated rule by uploading a file to the
S3 bucket.");
System.out.println("Press Enter to continue.");
sc.nextLine();
uploadTextFiletoS3(s3Client, bucketName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println(" 16. Update the rule to be a custom rule pattern.");
updateToCustomRule(eventBrClient, eventRuleName);
```

```

 System.out.println("Updated event rule " + eventRuleName + " to use a custom
pattern.");
 updateCustomRuleTargetWithTransform(eventBrClient, topicArn, eventRuleName);
 System.out.println("Updated event target " + topicArn + ".");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("17. Sending an event to trigger the rule. This will
trigger a subscription email.");
 triggerCustomRule(eventBrClient, email);
 System.out.println("Events have been sent. Press Enter to continue.");
 sc.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("18. Clean up resources.");
 System.out.println("Do you want to clean up resources (y/n)");
 String ans = sc.nextLine();
 if (ans.compareTo("y") == 0) {
 cleanupResources(eventBrClient, snsClient, s3Client, iam, topicArn,
eventRuleName, bucketName, roleName);
 } else {
 System.out.println("The resources will not be cleaned up. ");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The Amazon EventBridge example scenario has successfully
completed.");
 System.out.println(DASHES);
 }

 public static void cleanupResources(EventBridgeClient eventBrClient, SnsClient
snsClient, S3Client s3Client,
 IamClient iam, String topicArn, String eventRuleName, String bucketName,
String roleName) {
 System.out.println("Removing all targets from the event rule.");
 deleteTargetsFromRule(eventBrClient, eventRuleName);
 deleteRuleByName(eventBrClient, eventRuleName);
 deleteSNSTopic(snsClient, topicArn);
 deleteS3Bucket(s3Client, bucketName);
 deleteRole(iam, roleName);
 }

```

```
public static void deleteRole(IamClient iam, String roleName) {
 String policyArn = "arn:aws:iam::aws:policy/AmazonEventBridgeFullAccess";
 DetachRolePolicyRequest policyRequest = DetachRolePolicyRequest.builder()
 .policyArn(policyArn)
 .roleName(roleName)
 .build();

 iam.detachRolePolicy(policyRequest);
 System.out.println("Successfully detached policy " + policyArn + " from role " + roleName);

 // Delete the role.
 DeleteRoleRequest roleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 iam.deleteRole(roleRequest);
 System.out.println("**** Successfully deleted " + roleName);
}

public static void deleteS3Bucket(S3Client s3Client, String bucketName) {
 // Remove all the objects from the S3 bucket.
 ListObjectsRequest listObjects = ListObjectsRequest.builder()
 .bucket(bucketName)
 .build();

 ListObjectsResponse res = s3Client.listObjects(listObjects);
 List<S3Object> objects = res.contents();
 ArrayList<ObjectIdentifier> toDelete = new ArrayList<>();

 for (S3Object myValue : objects) {
 toDelete.add(ObjectIdentifier.builder()
 .key(myValue.key())
 .build());
 }

 DeleteObjectsRequest dor = DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(Delete.builder()
 .objects(toDelete).build())
 .build();

 s3Client.deleteObjects(dor);
}
```



```
// Delete the S3 bucket.
DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();

s3Client.deleteBucket(deleteBucketRequest);
System.out.println("You have deleted the bucket and the objects");
}

// Delete the SNS topic.
public static void deleteSNSTopic(SnsClient snsClient, String topicArn) {
 try {
 DeleteTopicRequest request = DeleteTopicRequest.builder()
 .topicArn(topicArn)
 .build();

 DeleteTopicResponse result = snsClient.deleteTopic(request);
 System.out.println("\n\nStatus was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteRuleByName(EventBridgeClient eventBrClient, String
ruleName) {
 DeleteRuleRequest ruleRequest = DeleteRuleRequest.builder()
 .name(ruleName)
 .build();

 eventBrClient.deleteRule(ruleRequest);
 System.out.println("Successfully deleted the rule");
}

public static void deleteTargetsFromRule(EventBridgeClient eventBrClient, String
eventRuleName) {
 // First, get all targets that will be deleted.
 ListTargetsByRuleRequest request = ListTargetsByRuleRequest.builder()
 .rule(eventRuleName)
 .build();
```

```

 ListTargetsByRuleResponse response =
eventBrClient.listTargetsByRule(request);
 List<Target> allTargets = response.targets();

 // Get all targets and delete them.
 for (Target myTarget : allTargets) {
 RemoveTargetsRequest removeTargetsRequest =
RemoveTargetsRequest.builder()
 .rule(eventRuleName)
 .ids(myTarget.id())
 .build();

 eventBrClient.removeTargets(removeTargetsRequest);
 System.out.println("Successfully removed the target");
 }
 }

 public static void triggerCustomRule(EventBridgeClient eventBrClient, String
email) {
 String json = "{" +
 "\"UserEmail\": \"" + email + "\", " +
 "\"Message\": \"This event was generated by example code.\", " +
 "\"UtcTime\": \"Now.\" " +
 "}";

 PutEventsRequestEntry entry = PutEventsRequestEntry.builder()
 .source("ExampleSource")
 .detail(json)
 .detailType("ExampleType")
 .build();

 PutEventsRequest eventsRequest = PutEventsRequest.builder()
 .entries(entry)
 .build();

 eventBrClient.putEvents(eventsRequest);
 }

 public static void updateCustomRuleTargetWithTransform(EventBridgeClient
eventBrClient, String topicArn,
 String ruleName) {
 String targetId = java.util.UUID.randomUUID().toString();
 InputTransformer inputTransformer = InputTransformer.builder()
 .inputTemplate("\"Notification: sample event was received.\"")

```

```

 .build();

 Target target = Target.builder()
 .id(targetId)
 .arn(topicArn)
 .inputTransformer(inputTransformer)
 .build();

 try {
 PutTargetsRequest targetsRequest = PutTargetsRequest.builder()
 .rule(ruleName)
 .targets(target)
 .eventBusName(null)
 .build();

 eventBrClient.putTargets(targetsRequest);
 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void updateToCustomRule(EventBridgeClient eventBrClient, String
ruleName) {
 String customEventsPattern = "{" +
 "\"source\": [\"ExampleSource\"],\" +
 "\"detail-type\": [\"ExampleType\"]\" +
 "}";

 PutRuleRequest request = PutRuleRequest.builder()
 .name(ruleName)
 .description("Custom test rule")
 .eventPattern(customEventsPattern)
 .build();

 eventBrClient.putRule(request);
}

// Update an Amazon S3 object created rule with a transform on the target.
public static void updateSnsEventRule(EventBridgeClient eventBrClient, String
topicArn, String ruleName) {
 String targetId = java.util.UUID.randomUUID().toString();
 Map<String, String> myMap = new HashMap<>();
 myMap.put("bucket", "$.detail.bucket.name");

```

```

 myMap.put("time", "$.time");

 InputTransformer inputTransformer = InputTransformer.builder()
 .inputTemplate("\Notification: an object was uploaded to bucket
<bucket> at <time>.\")
 .inputPathsMap(myMap)
 .build();

 Target target = Target.builder()
 .id(targetId)
 .arn(topicArn)
 .inputTransformer(inputTransformer)
 .build();

 try {
 PutTargetsRequest targetsRequest = PutTargetsRequest.builder()
 .rule(ruleName)
 .targets(target)
 .eventBusName(null)
 .build();

 eventBrClient.putTargets(targetsRequest);

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void checkRule(EventBridgeClient eventBrClient, String
eventRuleName) {
 try {
 DescribeRuleRequest ruleRequest = DescribeRuleRequest.builder()
 .name(eventRuleName)
 .build();

 DescribeRuleResponse response = eventBrClient.describeRule(ruleRequest);
 System.out.println("The state of the rule is " +
response.stateAsString());

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

```

```

 }

 public static void changeRuleState(EventBridgeClient eventBrClient, String
eventRuleName, Boolean isEnabled) {
 try {
 if (!isEnabled) {
 System.out.println("Disabling the rule: " + eventRuleName);
 DisableRuleRequest ruleRequest = DisableRuleRequest.builder()
 .name(eventRuleName)
 .build();

 eventBrClient.disableRule(ruleRequest);
 } else {
 System.out.println("Enabling the rule: " + eventRuleName);
 EnableRuleRequest ruleRequest = EnableRuleRequest.builder()
 .name(eventRuleName)
 .build();
 eventBrClient.enableRule(ruleRequest);
 }
 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 // Create and upload a file to an S3 bucket to trigger an event.
 public static void uploadTextFiletoS3(S3Client s3Client, String bucketName)
throws IOException {
 // Create a unique file name.
 String fileSuffix = new SimpleDateFormat("yyyyMMddHHmmss").format(new
Date());
 String fileName = "TextFile" + fileSuffix + ".txt";

 File myFile = new File(fileName);
 FileWriter fw = new FileWriter(myFile.getAbsolutePath());
 BufferedWriter bw = new BufferedWriter(fw);
 bw.write("This is a sample file for testing uploads.");
 bw.close();

 try {
 PutObjectRequest putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(fileName)

```

```

 .build();

 s3Client.putObject(putOb, RequestBody.fromFile(myFile));

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void listTargetRules(EventBridgeClient eventBrClient, String
topicArn) {
 ListRuleNamesByTargetRequest ruleNamesByTargetRequest =
ListRuleNamesByTargetRequest.builder()
 .targetArn(topicArn)
 .build();

 ListRuleNamesByTargetResponse response =
eventBrClient.listRuleNamesByTarget(ruleNamesByTargetRequest);
 List<String> rules = response.ruleNames();
 for (String rule : rules) {
 System.out.println("The rule name is " + rule);
 }
}

public static void listTargets(EventBridgeClient eventBrClient, String ruleName)
{
 ListTargetsByRuleRequest ruleRequest = ListTargetsByRuleRequest.builder()
 .rule(ruleName)
 .build();

 ListTargetsByRuleResponse res =
eventBrClient.listTargetsByRule(ruleRequest);
 List<Target> targetsList = res.targets();
 for (Target target: targetsList) {
 System.out.println("Target ARN: "+target.arn());
 }
}

// Add a rule which triggers an SNS target when a file is uploaded to an S3
// bucket.
public static void addSnsEventRule(EventBridgeClient eventBrClient, String
ruleName, String topicArn,
 String topicName, String eventRuleName, String bucketName) {

```

```

 String targetID = java.util.UUID.randomUUID().toString();
 Target myTarget = Target.builder()
 .id(targetID)
 .arn(topicArn)
 .build();

 List<Target> targets = new ArrayList<>();
 targets.add(myTarget);
 PutTargetsRequest request = PutTargetsRequest.builder()
 .eventBusName(null)
 .targets(targets)
 .rule(ruleName)
 .build();

 eventBrClient.putTargets(request);
 System.out.println("Added event rule " + eventRuleName + " with Amazon SNS
target " + topicName + " for bucket "
 + bucketName + ".");
 }

 public static void subEmail(SnsClient snsClient, String topicArn, String email)
 {
 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("email")
 .endpoint(email)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("Subscription ARN: " + result.subscriptionArn() + "\n
\n Status is "
 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void listRules(EventBridgeClient eventBrClient) {
 try {
 ListRulesRequest rulesRequest = ListRulesRequest.builder()

```

```

 .eventBusName("default")
 .limit(10)
 .build();

ListRulesResponse response = eventBrClient.listRules(rulesRequest);
List<Rule> rules = response.rules();
for (Rule rule : rules) {
 System.out.println("The rule name is : " + rule.name());
 System.out.println("The rule description is : " +
rule.description());
 System.out.println("The rule state is : " + rule.stateAsString());
}

} catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}

public static String createSnsTopic(SnsClient snsClient, String topicName) {
 String topicPolicy = "{" +
 "\"Version\": \"2012-10-17\", \" +
 "\"Statement\": [{" +
 "\"Sid\": \"EventBridgePublishTopic\", \" +
 "\"Effect\": \"Allow\", \" +
 "\"Principal\": {\" +
 "\"Service\": \"events.amazonaws.com\"\" +
 \"}, \" +
 "\"Resource\": \"*\", \" +
 "\"Action\": \"sns:Publish\"\" +
 \"}]\" +
 "}";

 Map<String, String> topicAttributes = new HashMap<>();
 topicAttributes.put("Policy", topicPolicy);
 CreateTopicRequest topicRequest = CreateTopicRequest.builder()
 .name(topicName)
 .attributes(topicAttributes)
 .build();

 CreateTopicResponse response = snsClient.createTopic(topicRequest);
 System.out.println("Added topic " + topicName + " for email
subscriptions.");
 return response.topicArn();
}

```



```

 }

 // Create a new event rule that triggers when an Amazon S3 object is created in
 // a bucket.
 public static void addEventRule(EventBridgeClient eventBrClient, String roleArn,
 String bucketName,
 String eventRuleName) {
 String pattern = "{\n" +
 " \"source\": [\"aws.s3\"],\n" +
 " \"detail-type\": [\"Object Created\"],\n" +
 " \"detail\": {\n" +
 " \"bucket\": {\n" +
 " \"name\": [\"" + bucketName + "\"] \n" +
 " }\n" +
 " }\n" +
 "}";

 try {
 PutRuleRequest ruleRequest = PutRuleRequest.builder()
 .description("Created by using the AWS SDK for Java v2")
 .name(eventRuleName)
 .eventPattern(pattern)
 .roleArn(roleArn)
 .build();

 PutRuleResponse ruleResponse = eventBrClient.putRule(ruleRequest);
 System.out.println("The ARN of the new rule is " +
ruleResponse.ruleArn());

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 // Determine if the S3 bucket exists.
 public static Boolean checkBucket(S3Client s3Client, String bucketName) {
 try {
 HeadBucketRequest headBucketRequest = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 s3Client.headBucket(headBucketRequest);
 return true;
 }
 }
 }

```

```
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 return false;
}

// Set the S3 bucket notification configuration.
public static void setBucketNotification(S3Client s3Client, String bucketName) {
 try {
 EventBridgeConfiguration eventBridgeConfiguration =
EventBridgeConfiguration.builder()
 .build();

 NotificationConfiguration configuration =
NotificationConfiguration.builder()
 .eventBridgeConfiguration(eventBridgeConfiguration)
 .build();

 PutBucketNotificationConfigurationRequest configurationRequest =
PutBucketNotificationConfigurationRequest
 .builder()
 .bucket(bucketName)
 .notificationConfiguration(configuration)
 .skipDestinationValidation(true)
 .build();

 s3Client.putBucketNotificationConfiguration(configurationRequest);
 System.out.println("Added bucket " + bucketName + " with EventBridge
events enabled.");
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void createBucket(S3Client s3Client, String bucketName) {
 try {
 S3Waiter s3Waiter = s3Client.waiter();
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 s3Client.createBucket(bucketRequest);
 }
```

```

 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 // Wait until the bucket is created and print out the response.
 WaiterResponse<HeadBucketResponse> waiterResponse =
s3Waiter.waitUntilBucketExists(bucketRequestWait);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println(bucketName + " is ready");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static String createIAMRole(IamClient iam, String rolename, String
polJSON) {
 try {
 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(rolename)
 .assumeRolePolicyDocument(polJSON)
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse response = iam.createRole(request);
 AttachRolePolicyRequest rolePolicyRequest =
AttachRolePolicyRequest.builder()
 .roleName(rolename)
 .policyArn("arn:aws:iam::aws:policy/
AmazonEventBridgeFullAccess")
 .build();

 iam.attachRolePolicy(rolePolicyRequest);
 return response.role().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteRule](#)
  - [DescribeRule](#)
  - [DisableRule](#)
  - [EnableRule](#)
  - [ListRuleNamesByTarget](#)
  - [ListRules](#)
  - [ListTargetsByRule](#)
  - [PutEvents](#)
  - [PutRule](#)
  - [PutTargets](#)

## Actions

### DeleteRule

The following code example shows how to use DeleteRule.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteRuleByName(EventBridgeClient eventBrClient, String
ruleName) {
 DeleteRuleRequest ruleRequest = DeleteRuleRequest.builder()
 .name(ruleName)
 .build();

 eventBrClient.deleteRule(ruleRequest);
 System.out.println("Successfully deleted the rule");
}
```

```
}
```

- For API details, see [DeleteRule](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeRule

The following code example shows how to use `DescribeRule`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void checkRule(EventBridgeClient eventBrClient, String
eventRuleName) {
 try {
 DescribeRuleRequest ruleRequest = DescribeRuleRequest.builder()
 .name(eventRuleName)
 .build();

 DescribeRuleResponse response = eventBrClient.describeRule(ruleRequest);
 System.out.println("The state of the rule is " +
response.stateAsString());

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeRule](#) in *AWS SDK for Java 2.x API Reference*.

## DisableRule

The following code example shows how to use `DisableRule`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Disable a rule by using its rule name.

```
public static void changeRuleState(EventBridgeClient eventBrClient, String
eventRuleName, Boolean isEnabled) {
 try {
 if (!isEnabled) {
 System.out.println("Disabling the rule: " + eventRuleName);
 DisableRuleRequest ruleRequest = DisableRuleRequest.builder()
 .name(eventRuleName)
 .build();

 eventBrClient.disableRule(ruleRequest);
 } else {
 System.out.println("Enabling the rule: " + eventRuleName);
 EnableRuleRequest ruleRequest = EnableRuleRequest.builder()
 .name(eventRuleName)
 .build();
 eventBrClient.enableRule(ruleRequest);
 }
 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DisableRule](#) in *AWS SDK for Java 2.x API Reference*.

## EnableRule

The following code example shows how to use `EnableRule`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Enable a rule by using its rule name.

```
public static void changeRuleState(EventBridgeClient eventBrClient, String
eventRuleName, Boolean isEnabled) {
 try {
 if (!isEnabled) {
 System.out.println("Disabling the rule: " + eventRuleName);
 DisableRuleRequest ruleRequest = DisableRuleRequest.builder()
 .name(eventRuleName)
 .build();

 eventBrClient.disableRule(ruleRequest);
 } else {
 System.out.println("Enabling the rule: " + eventRuleName);
 EnableRuleRequest ruleRequest = EnableRuleRequest.builder()
 .name(eventRuleName)
 .build();
 eventBrClient.enableRule(ruleRequest);
 }
 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [EnableRule](#) in *AWS SDK for Java 2.x API Reference*.

## ListRuleNamesByTarget

The following code example shows how to use `ListRuleNamesByTarget`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List all of the rule names by using the target.

```
public static void listTargetRules(EventBridgeClient eventBrClient, String
topicArn) {
 ListRuleNamesByTargetRequest ruleNamesByTargetRequest =
ListRuleNamesByTargetRequest.builder()
 .targetArn(topicArn)
 .build();

 ListRuleNamesByTargetResponse response =
eventBrClient.listRuleNamesByTarget(ruleNamesByTargetRequest);
 List<String> rules = response.ruleNames();
 for (String rule : rules) {
 System.out.println("The rule name is " + rule);
 }
}
```

- For API details, see [ListRuleNamesByTarget](#) in *AWS SDK for Java 2.x API Reference*.

## ListRules

The following code example shows how to use ListRules.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Enable a rule by using its rule name.



```
public static void listRules(EventBridgeClient eventBrClient) {
 try {
 ListRulesRequest rulesRequest = ListRulesRequest.builder()
 .eventBusName("default")
 .limit(10)
 .build();

 ListRulesResponse response = eventBrClient.listRules(rulesRequest);
 List<Rule> rules = response.rules();
 for (Rule rule : rules) {
 System.out.println("The rule name is : " + rule.name());
 System.out.println("The rule description is : " +
rule.description());
 System.out.println("The rule state is : " + rule.stateAsString());
 }

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListRules](#) in *AWS SDK for Java 2.x API Reference*.

## ListTargetsByRule

The following code example shows how to use `ListTargetsByRule`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List all of the targets for a rule by using the rule name.

```
public static void listTargets(EventBridgeClient eventBrClient, String ruleName)
{
```

```

 ListTargetsByRuleRequest ruleRequest = ListTargetsByRuleRequest.builder()
 .rule(ruleName)
 .build();

 ListTargetsByRuleResponse res =
eventBrClient.listTargetsByRule(ruleRequest);
 List<Target> targetsList = res.targets();
 for (Target target: targetsList) {
 System.out.println("Target ARN: "+target.arn());
 }
 }
}

```

- For API details, see [ListTargetsByRule](#) in *AWS SDK for Java 2.x API Reference*.

## PutEvents

The following code example shows how to use PutEvents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void triggerCustomRule(EventBridgeClient eventBrClient, String
email) {
 String json = "{" +
 "\"UserEmail\": \"" + email + "\",\" +
 "\"Message\": \"This event was generated by example code.\",\" +
 "\"UtcTime\": \"Now.\"\" +
 "}";

 PutEventsRequestEntry entry = PutEventsRequestEntry.builder()
 .source("ExampleSource")
 .detail(json)
 .detailType("ExampleType")
 .build();
}

```

```

 PutEventsRequest eventsRequest = PutEventsRequest.builder()
 .entries(entry)
 .build();

 eventBrClient.putEvents(eventsRequest);
 }

```

- For API details, see [PutEvents](#) in *AWS SDK for Java 2.x API Reference*.

## PutRule

The following code example shows how to use PutRule.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create a scheduled rule.

```

public static void createEBRule(EventBridgeClient eventBrClient, String
ruleName, String cronExpression) {
 try {
 PutRuleRequest ruleRequest = PutRuleRequest.builder()
 .name(ruleName)
 .eventBusName("default")
 .scheduleExpression(cronExpression)
 .state("ENABLED")
 .description("A test rule that runs on a schedule created by the
Java API")
 .build();

 PutRuleResponse ruleResponse = eventBrClient.putRule(ruleRequest);
 System.out.println("The ARN of the new rule is " +
ruleResponse.ruleArn());

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
}

```

```

 System.exit(1);
 }
}

```

Create a rule that triggers when an object is added to an Amazon Simple Storage Service bucket.

```

// Create a new event rule that triggers when an Amazon S3 object is created in
// a bucket.
public static void addEventRule(EventBridgeClient eventBrClient, String roleArn,
String bucketName,
 String eventRuleName) {
 String pattern = "{\n" +
 " \"source\": [\"aws.s3\"],\n" +
 " \"detail-type\": [\"Object Created\"],\n" +
 " \"detail\": {\n" +
 " \"bucket\": {\n" +
 " \"name\": [\"" + bucketName + "\"] \n" +
 " }\n" +
 " }\n" +
 "}";

 try {
 PutRuleRequest ruleRequest = PutRuleRequest.builder()
 .description("Created by using the AWS SDK for Java v2")
 .name(eventRuleName)
 .eventPattern(pattern)
 .roleArn(roleArn)
 .build();

 PutRuleResponse ruleResponse = eventBrClient.putRule(ruleRequest);
 System.out.println("The ARN of the new rule is " +
ruleResponse.ruleArn());

 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [PutRule](#) in *AWS SDK for Java 2.x API Reference*.

## PutTargets

The following code example shows how to use PutTargets.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Add an Amazon SNS topic as a target for a rule.

```
// Add a rule which triggers an SNS target when a file is uploaded to an S3
// bucket.
public static void addSnsEventRule(EventBridgeClient eventBrClient, String
ruleName, String topicArn,
 String topicName, String eventRuleName, String bucketName) {
 String targetID = java.util.UUID.randomUUID().toString();
 Target myTarget = Target.builder()
 .id(targetID)
 .arn(topicArn)
 .build();

 List<Target> targets = new ArrayList<>();
 targets.add(myTarget);
 PutTargetsRequest request = PutTargetsRequest.builder()
 .eventBusName(null)
 .targets(targets)
 .rule(ruleName)
 .build();

 eventBrClient.putTargets(request);
 System.out.println("Added event rule " + eventRuleName + " with Amazon SNS
target " + topicName + " for bucket "
 + bucketName + ".");
}
```

Add an input transformer to a target for a rule.

```
public static void updateCustomRuleTargetWithTransform(EventBridgeClient
eventBrClient, String topicArn,
 String ruleName) {
 String targetId = java.util.UUID.randomUUID().toString();
 InputTransformer inputTransformer = InputTransformer.builder()
 .inputTemplate("\Notification: sample event was received.\")
 .build();

 Target target = Target.builder()
 .id(targetId)
 .arn(topicArn)
 .inputTransformer(inputTransformer)
 .build();

 try {
 PutTargetsRequest targetsRequest = PutTargetsRequest.builder()
 .rule(ruleName)
 .targets(target)
 .eventBusName(null)
 .build();

 eventBrClient.putTargets(targetsRequest);
 } catch (EventBridgeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [PutTargets](#) in *AWS SDK for Java 2.x API Reference*.

## RemoveTargets

The following code example shows how to use RemoveTargets.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Remove all of the targets for a rule by using the rule name.

```
public static void deleteTargetsFromRule(EventBridgeClient eventBrClient, String
eventRuleName) {
 // First, get all targets that will be deleted.
 ListTargetsByRuleRequest request = ListTargetsByRuleRequest.builder()
 .rule(eventRuleName)
 .build();

 ListTargetsByRuleResponse response =
eventBrClient.listTargetsByRule(request);
 List<Target> allTargets = response.targets();

 // Get all targets and delete them.
 for (Target myTarget : allTargets) {
 RemoveTargetsRequest removeTargetsRequest =
RemoveTargetsRequest.builder()
 .rule(eventRuleName)
 .ids(myTarget.id())
 .build();

 eventBrClient.removeTargets(removeTargetsRequest);
 System.out.println("Successfully removed the target");
 }
}
```

- For API details, see [RemoveTargets](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Send event notifications to EventBridge

The following code example shows how to enable a bucket to send S3 event notifications to EventBridge and route notifications to an Amazon SNS topic and Amazon SQS queue.

## SDK for Java 2.x

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/** This method configures a bucket to send events to AWS EventBridge and
creates a rule
 * to route the S3 object created events to a topic and a queue.
 *
 * @param bucketName Name of existing bucket
 * @param topicArn ARN of existing topic to receive S3 event notifications
 * @param queueArn ARN of existing queue to receive S3 event notifications
 *
 * An AWS CloudFormation stack sets up the bucket, queue, topic before the
method runs.
 */
public static String setBucketNotificationToEventBridge(String bucketName,
String topicArn, String queueArn) {
 try {
 // Enable bucket to emit S3 Event notifications to EventBridge.
 s3Client.putBucketNotificationConfiguration(b -> b
 .bucket(bucketName)
 .notificationConfiguration(b1 -> b1
 .eventBridgeConfiguration(
 SdkBuilder::build)
).build()).join();

 // Create an EventBridge rule to route Object Created notifications.
 PutRuleRequest putRuleRequest = PutRuleRequest.builder()
 .name(RULE_NAME)
 .eventPattern("""
 {
 "source": ["aws.s3"],
 "detail-type": ["Object Created"],
 "detail": {
 "bucket": {
 "name": ["%s"]
 }
 }
 }
 """)
 }
}

```



```

 }
 """.formatted(bucketName))
 .build();

 // Add the rule to the default event bus.
 PutRuleResponse putRuleResponse =
eventBridgeClient.putRule(putRuleRequest)
 .whenComplete((r, t) -> {
 if (t != null) {
 logger.error("Error creating event bus rule: " +
t.getMessage(), t);
 throw new RuntimeException(t.getCause().getMessage(),
t);
 }
 logger.info("Event bus rule creation request sent
successfully. ARN is: {}", r.ruleArn());
 }).join();

 // Add the existing SNS topic and SQS queue as targets to the rule.
eventBridgeClient.putTargets(b -> b
 .eventBusName("default")
 .rule(RULE_NAME)
 .targets(List.of (
 Target.builder()
 .arn(queueArn)
 .id("Queue")
 .build(),
 Target.builder()
 .arn(topicArn)
 .id("Topic")
 .build()
)
).join();
 return putRuleResponse.ruleArn();
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [PutBucketNotificationConfiguration](#)
- [PutRule](#)
- [PutTargets](#)

## Use scheduled events to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by an Amazon EventBridge scheduled event.

### SDK for Java 2.x

Shows how to create an Amazon EventBridge scheduled event that invokes an AWS Lambda function. Configure EventBridge to use a cron expression to schedule when the Lambda function is invoked. In this example, you create a Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create an app that sends a mobile text message to your employees that congratulates them at the one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- CloudWatch Logs
- DynamoDB
- EventBridge
- Lambda
- Amazon SNS

## EventBridge Scheduler examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with EventBridge Scheduler.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello EventBridge Scheduler

The following code example shows how to get started using EventBridge Scheduler.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.scheduler.SchedulerAsyncClient;
import software.amazon.awssdk.services.scheduler.model.ListSchedulesRequest;
import software.amazon.awssdk.services.scheduler.model.ScheduleSummary;
import software.amazon.awssdk.services.scheduler.paginators.ListSchedulesPublisher;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class HelloScheduler {

 public static void main(String [] args) {
 listSchedulesAsync();
 }
}
```

```

/**
 * Lists all the schedules available.
 * <p>
 * This method uses the {@link SchedulerAsyncClient} to make an asynchronous
request to
 * list all the schedules available. The method uses the {@link
ListSchedulesPublisher}
 * to fetch the schedules in a paginated manner, and then processes the
responses
 * asynchronously.
 */
public static void listSchedulesAsync() {
 SchedulerAsyncClient schedulerAsyncClient = SchedulerAsyncClient.create();

 // Build the request to list schedules
 ListSchedulesRequest listSchedulesRequest =
ListSchedulesRequest.builder().build();

 // Use the paginator to fetch all schedules asynchronously.
 ListSchedulesPublisher paginator =
schedulerAsyncClient.listSchedulesPaginator(listSchedulesRequest);
 List<ScheduleSummary> results = new ArrayList<>();

 // Subscribe to the paginator to process the response asynchronously
 CompletableFuture<Void> future = paginator.subscribe(response -> {
 response.schedules().forEach(schedule -> {
 results.add(schedule);
 System.out.printf("Schedule: %s%n", schedule.name());
 });
 });

 // Wait for the asynchronous operation to complete.
 future.join();

 // After all schedules are fetched, print the total count.
 System.out.printf("Total of %d schedule(s) available.%n", results.size());
}
}

```

- For API details, see [ListSchedules](#) in *AWS SDK for Java 2.x API Reference*.

# Actions

## CreateSchedule

The following code example shows how to use CreateSchedule.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new schedule for a target task.
 *
 * @param name the name of the schedule
 * @param scheduleExpression The schedule expression that defines when the
schedule should run.
 * @param scheduleGroupName the name of the schedule group to which the
schedule belongs
 * @param targetArn the Amazon Resource Name (ARN) of the target
task
 * @param roleArn the ARN of the IAM role to be used for the
schedule
 * @param input the input data for the target task
 * @param deleteAfterCompletion whether to delete the schedule after it's
executed
 * @param useFlexibleTimeWindow whether to use a flexible time window for the
schedule execution
 * @return true if the schedule was successfully created, false otherwise
 */
public CompletableFuture<Boolean> createScheduleAsync(
 String name,
 String scheduleExpression,
 String scheduleGroupName,
 String targetArn,
 String roleArn,
 String input,
 boolean deleteAfterCompletion,
```

```

 boolean useFlexibleTimeWindow) {

 int hoursToRun = 1;
 int flexibleTimeWindowMinutes = 10;

 Target target = Target.builder()
 .arn(targetArn)
 .roleArn(roleArn)
 .input(input)
 .build();

 FlexibleTimeWindow flexibleTimeWindow = FlexibleTimeWindow.builder()
 .mode(useFlexibleTimeWindow
 ? FlexibleTimeWindowMode.FLEXIBLE
 : FlexibleTimeWindowMode.OFF)
 .maximumWindowInMinutes(useFlexibleTimeWindow
 ? flexibleTimeWindowMinutes
 : null)
 .build();

 Instant startDate = Instant.now();
 Instant endDate = startDate.plus(Duration.ofHours(hoursToRun));

 CreateScheduleRequest request = CreateScheduleRequest.builder()
 .name(name)
 .scheduleExpression(scheduleExpression)
 .groupName(scheduleGroupName)
 .target(target)
 .actionAfterCompletion(deleteAfterCompletion
 ? ActionAfterCompletion.DELETE
 : ActionAfterCompletion.NONE)
 .startDate(startDate)
 .endDate(endDate)
 .flexibleTimeWindow(flexibleTimeWindow)
 .build();

 return getAsyncClient().createSchedule(request)
 .thenApply(response -> {
 logger.info("Successfully created schedule {} in schedule group {}",
 "The ARN is {}", name, scheduleGroupName, response.scheduleArn());
 return true;
 })
 .whenComplete((result, ex) -> {
 if (ex != null) {

```

```

 if (ex instanceof ConflictException) {
 // Handle ConflictException
 logger.error("A conflict exception occurred while creating
the schedule: {}", ex.getMessage());
 throw new CompletionException("A conflict exception occurred
while creating the schedule: " + ex.getMessage(), ex);
 } else {
 throw new CompletionException("Error creating schedule: " +
ex.getMessage(), ex);
 }
 }
});
}

```

- For API details, see [CreateSchedule](#) in *AWS SDK for Java 2.x API Reference*.

## CreateScheduleGroup

The following code example shows how to use CreateScheduleGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new schedule group.
 *
 * @param name the name of the schedule group to be created
 * @return a {@link CompletableFuture} representing the asynchronous operation
of creating the schedule group
 */
public CompletableFuture<CreateScheduleGroupResponse> createScheduleGroup(String
name) {
 CreateScheduleGroupRequest request = CreateScheduleGroupRequest.builder()
 .name(name)
 .build();
}

```

```

 logger.info("Initiating createScheduleGroup call for group: {}", name);
 CompletableFuture<CreateScheduleGroupResponse> futureResponse =
getAsyncClient().createScheduleGroup(request);
 futureResponse.whenComplete((response, ex) -> {
 if (ex != null) {
 if (ex instanceof CompletionException && ex.getCause() instanceof
ConflictException) {
 // Rethrow the ConflictException
 throw (ConflictException) ex.getCause();
 } else {
 throw new CompletionException("Failed to create schedule group:
" + name, ex);
 }
 } else if (response == null) {
 throw new RuntimeException("Failed to create schedule group:
response was null");
 } else {
 logger.info("Successfully created schedule group '{}': {}", name,
response.scheduleGroupArn());
 }
 });

 return futureResponse;
 }

```

- For API details, see [CreateScheduleGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteSchedule

The following code example shows how to use DeleteSchedule.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).



```

/**
 * Deletes a schedule with the specified name and group name.
 *
 * @param name the name of the schedule to be deleted
 * @param groupName the group name of the schedule to be deleted
 * @return a {@link CompletableFuture} that, when completed, indicates whether
the schedule was successfully deleted
 * @throws CompletionException if an error occurs while deleting the schedule,
except for the case where the schedule is not found
 */
public CompletableFuture<Boolean> deleteScheduleAsync(String name, String
groupName) {
 DeleteScheduleRequest request = DeleteScheduleRequest.builder()
 .name(name)
 .groupName(groupName)
 .build();

 CompletableFuture<DeleteScheduleResponse> response =
getAsyncClient().deleteSchedule(request);
 return response.handle((result, ex) -> {
 if (ex != null) {
 if (ex instanceof ResourceNotFoundException) {
 throw new CompletionException("Resource not found while deleting
schedule with ID: " + name, ex);
 } else {
 throw new CompletionException("Failed to delete schedule.", ex);
 }
 }
 logger.info("Successfully deleted schedule with name {}. ", name);
 return true;
 });
}

```

- For API details, see [DeleteSchedule](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteScheduleGroup

The following code example shows how to use DeleteScheduleGroup.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes the specified schedule group.
 *
 * @param name the name of the schedule group to delete
 * @return a {@link CompletableFuture} that completes when the schedule group
has been deleted
 * @throws CompletionException if an error occurs while deleting the schedule
group
 */
public CompletableFuture<Void> deleteScheduleGroupAsync(String name) {
 DeleteScheduleGroupRequest request = DeleteScheduleGroupRequest.builder()
 .name(name)
 .build();

 return getAsyncClient().deleteScheduleGroup(request)
 .thenRun(() -> {
 logger.info("Successfully deleted schedule group {}", name);
 })
 .whenComplete((result, ex) -> {
 if (ex != null) {
 if (ex instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource was not found: "
+ ex.getMessage(), ex);
 } else {
 throw new CompletionException("Error deleting schedule
group: " + ex.getMessage(), ex);
 }
 }
 });
}
```

- For API details, see [DeleteScheduleGroup](#) in *AWS SDK for Java 2.x API Reference*.

# Scenarios

## Scheduled Events

The following code example shows how to:

- Deploy a CloudFormation stack with required resources.
- Create a EventBridge Scheduler schedule group.
- Create a one-time EventBridge Scheduler schedule with a flexible time window.
- Create a recurring EventBridge Scheduler schedule with a specified rate.
- Delete EventBridge Scheduler the schedule and schedule group.
- Clean up resources and delete the stack.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run the scenario.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.scheduler.model.SchedulerException;
import javax.mail.internet.AddressException;
import javax.mail.internet.InternetAddress;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Map;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

/**
 * This Java code example performs the following tasks for the Amazon EventBridge
 * Scheduler workflow:
 * <p>
 * 1. Prepare the Application:
```

```

* - Prompt the user for an email address to use for the subscription for the SNS
topic subscription.
* - Deploy the Cloud Formation template in resources/cfn_template.yaml for resource
creation.
* - Store the outputs of the stack into variables for use in the workflow.
* - Create a schedule group for all workflow schedules.
* <p>
* 2. Create one-time Schedule:
* - Create a one-time schedule to send an initial event.
* - Use a Flexible Time Window and set the schedule to delete after completion.
* - Wait for the user to receive the event email from SNS.
* <p>
* 3. Create a time-based schedule:
* - Prompt the user for how many X times per Y hours a recurring event should be
scheduled.
* - Create the scheduled event for X times per hour for Y hours.
* - Wait for the user to receive the event email from SNS.
* - Delete the schedule when the user is finished.
* <p>
* 4. Clean up:
* - Prompt the user for y/n answer if they want to destroy the stack and clean up
all resources.
* - Delete the schedule group.
* - Destroy the Cloud Formation stack and wait until the stack has been removed.
*/

```

```

public class EventbridgeSchedulerScenario {

 private static final Logger logger =
LoggerFactory.getLogger(EventbridgeSchedulerScenario.class);
 private static final Scanner scanner = new Scanner(System.in);
 private static String STACK_NAME = "workflow-stack-name";
 private static final String scheduleGroupName = "schedules-group";

 private static String recurringScheduleName = "";

 private static String oneTimeScheduleName = "";

 private static final EventbridgeSchedulerActions eventbridgeActions = new
EventbridgeSchedulerActions();

 private static final String DASHES = new String(new char[80]).replace("\0", "-");

 private static String roleArn = "";

```

```

public static String snsTopicArn = "";

public static void main(String[] args) {
 logger.info(DASHES);
 logger.info("Welcome to the Amazon EventBridge Scheduler Workflow.");
 logger.info("""
 Amazon EventBridge Scheduler is a fully managed service that helps you
schedule and execute
 a wide range of tasks and events in the cloud. It's designed to simplify
the process of
 scheduling and managing recurring or one-time events, making it easier
for developers and
 businesses to automate various workflows and processes.

 One of the key features of Amazon EventBridge Scheduler is its ability
to schedule events
 based on a variety of triggers, including time-based schedules, custom
event patterns, or
 even integration with other AWS services. For example, you can use
EventBridge Scheduler
 to schedule a report generation task to run every weekday at 9 AM, or to
trigger a
 Lambda function when a specific Amazon S3 object is created.

 This flexibility allows you to build complex and dynamic event-driven
architectures
 that adapt to your business needs.

 Lets get started...
 """);
 waitForInputToContinue();
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("1. Prepare the application.");
 waitForInputToContinue();
 try {
 boolean prepareSuccess = prepareApplication();
 logger.info(DASHES);

 if (prepareSuccess) {
 logger.info("2. Create one-time schedule.");
 logger.info("""

```

A one-time schedule in Amazon EventBridge Scheduler is an event trigger that allows you to schedule a one-time event to run at a specific date and time. This is useful for executing a specific task or workflow at a predetermined time, without the need for recurring or complex scheduling.

```

 "");
 waitForInputToContinue();
 createOneTimeSchedule();
 logger.info("Do you want to delete the schedule {} (y/n) ?",
oneTimeScheduleName);
 String ans = scanner.nextLine().trim();
 if (ans.equalsIgnoreCase("y")) {

eventbridgeActions.deleteScheduleAsync(oneTimeScheduleName,scheduleGroupName);
 }
 logger.info(DASHES);

```

logger.info("3. Create a recurring schedule.");  
logger.info(""

A recurring schedule is a feature that allows you to schedule and manage the execution of your serverless applications or workloads on a recurring basis. For example, with EventBridge Scheduler, you can create custom schedules for your AWS Lambda functions, AWS Step Functions, and other supported event sources, enabling you to automate tasks and workflows without the need for complex infrastructure management.

```

 "");
 waitForInputToContinue();
 createRecurringSchedule();
 logger.info("Do you want to delete the schedule {} (y/n) ?",
oneTimeScheduleName);
 String ans2 = scanner.nextLine().trim();
 if (ans2.equalsIgnoreCase("y")) {

eventbridgeActions.deleteScheduleAsync(recurringScheduleName,scheduleGroupName);
 }
 logger.info(DASHES);
 }
} catch (Exception ex) {

```

```

 logger.info("There was a problem with the workflow {}, initiating
cleanup...", ex.getMessage());
 cleanUp();
 }

 logger.info(DASHES);
 logger.info("4. Clean up the resources.");
 logger.info("Do you want to delete these AWS resources (y/n) ?");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 cleanUp();
 } else {
 logger.info("The AWS resources will not be deleted.");
 }
 logger.info("Amazon EventBridge Scheduler workflow completed.");
 logger.info(DASHES);
}

/**
 * Cleans up the resources associated with the EventBridge scheduler.
 * If any errors occur during the cleanup process, the corresponding error
messages are logged.
 */
public static void cleanUp() {
 logger.info("First, delete the schedule group.");
 logger.info("When the schedule group is deleted, schedules that are part of
that group are deleted.");
 waitForInputToContinue();
 try {
 eventbridgeActions.deleteScheduleGroupAsync(scheduleGroupName).join();

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof SchedulerException schedulerException) {
 logger.error("Scheduler error occurred: Error message: {}, Error
code {}",
 schedulerException.getMessage(),
schedulerException.awsErrorDetails().errorCode(), schedulerException);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 }
 return;
 }
}

```

```

 logger.info("Destroy the CloudFormation stack");
 waitForInputToContinue();
 CloudFormationHelper.destroyCloudFormationStack(STACK_NAME);
 }

 /**
 * Prepares the application by creating resources in a CloudFormation stack,
 including an SNS topic
 * that will be subscribed to the EventBridge Scheduler events. The user will
 need to confirm the subscription
 * in order to receive event emails.
 *
 * @return true if the application preparation was successful, false otherwise
 */
 public static boolean prepareApplication() {
 logger.info("""
 This example creates resources in a CloudFormation stack, including an
SNS topic
 that will be subscribed to the EventBridge Scheduler events.
 You will need to confirm the subscription in order to receive event
emails.
 """);

 String emailAddress = promptUserForEmail();
 logger.info("You entered {}", emailAddress);

 logger.info("Do you want to use a custom Stack name (y/n) ?");
 String ans = scanner.nextLine().trim();
 if (ans.equalsIgnoreCase("y")) {
 String newStackName = scanner.nextLine();
 logger.info("You entered {} for the new stack name", newStackName);
 waitForInputToContinue();
 STACK_NAME = newStackName;
 }

 logger.info("Get the roleArn and snsTopicArn values using a Cloudformation
template.");
 waitForInputToContinue();
 CloudFormationHelper.deployCloudFormationStack(STACK_NAME, emailAddress);
 Map<String, String> stackOutputs =
CloudFormationHelper.getStackOutputs(STACK_NAME);
 roleArn = stackOutputs.get("RoleARN");
 snsTopicArn = stackOutputs.get("SNSTopicARN");
 }

```



```

 logger.info("The roleARN is {}", roleArn);
 logger.info("The snsTopicArn is {}", snsTopicArn);

 try {
 eventbridgeActions.createScheduleGroup(scheduleGroupName).join();
 logger.info("createScheduleGroupAsync completed successfully.");

 } catch (RuntimeException e) {
 logger.error("Error occurred: {} ", e.getMessage());
 return false;
 }
 logger.info("Application preparation complete.");
 return true;
 }

 /**
 * Waits for the user to enter 'c' followed by <ENTER> to continue the program.
 * This method is used to pause the program execution and wait for user input
 before
 * proceeding.
 */
 private static void waitForInputToContinue() {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
 }

 /**
 * Prompts the user to enter an email address and validates the input.
 * If the provided email address is invalid, the method will prompt the user to
 try again.
 */

```

```

 * @return the valid email address entered by the user
 */
private static String promptUserForEmail() {
 logger.info("Enter an email address to use for event subscriptions: ");
 String email = scanner.nextLine();
 if (!isValidEmail(email)) {
 logger.info("Invalid email address. Please try again.");
 return promptUserForEmail();
 }
 return email;
}

/**
 * Checks if the given email address is valid.
 *
 * @param email the email address to be validated
 * @return {@code true} if the email address is valid, {@code false} otherwise
 */
private static boolean isValidEmail(String email) {
 try {
 InternetAddress emailAddress = new InternetAddress(email);
 emailAddress.validate();
 return true;

 } catch (AddressException e) {
 return false;
 }
}

/**
 * Creates a one-time schedule to send an initial event in 1 minute with a
flexible time window.
 *
 * @return {@code true} if the schedule was created successfully, {@code false}
otherwise
 */
public static Boolean createOneTimeSchedule() {
 oneTimeScheduleName = promptUserForResourceName("Enter a name for the one-
time schedule:");
 logger.info("Creating a one-time schedule named {} to send an initial event
in 1 minute with a flexible time window...", oneTimeScheduleName);
 LocalDateTime scheduledTime = LocalDateTime.now();
 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-
dd'T'HH:mm:ss");

```

```

 String scheduleExpression = "at(" + scheduledTime.format(formatter) + ")";
 return eventbridgeActions.createScheduleAsync(
 oneTimeScheduleName,
 scheduleExpression,
 scheduleGroupName,
 snsTopicArn,
 roleArn,
 "One time scheduled event test from schedule",
 true,
 true).join();
 }

 /**
 * Creates a recurring schedule to send events based on a specific time.
 *
 * @return A {@link CompletableFuture} that completes with a boolean value
 indicating the success or failure of the operation.
 */
 public static Boolean createRecurringSchedule() {
 logger.info("Creating a recurring schedule to send events for one hour...");
 recurringScheduleName = promptUserForResourceName("Enter a name for the
recurring schedule:");

 // Prompt the user for the schedule rate (in minutes).
 int scheduleRateInMinutes = promptUserForInteger("Enter the desired schedule
rate (in minutes): ");
 String scheduleExpression = "rate(" + scheduleRateInMinutes + " minutes)";
 return eventbridgeActions.createScheduleAsync(
 recurringScheduleName,
 scheduleExpression,
 scheduleGroupName,
 snsTopicArn,
 roleArn,
 "Recurrent event test from schedule " + recurringScheduleName,
 true,
 true).join();
 }

 /**
 * Prompts the user for a resource name and validates the input.
 *

```

```

 * @param prompt the message to display to the user when prompting for the
resource name
 * @return the valid resource name entered by the user
 */
 private static String promptUserForResourceName(String prompt) {
 logger.info(prompt);
 String resourceName = scanner.nextLine();
 String regex = "[0-9a-zA-Z-_.]+";
 if (!resourceName.matches(regex)) {
 logger.info("Invalid resource name. Please use a name that matches the
pattern " + regex + ".");
 return promptUserForResourceName(prompt);
 }
 return resourceName;
 }

 /**
 * Prompts the user for an integer input and returns the integer value.
 *
 * @param prompt the message to be displayed to the user when prompting for
input
 * @return the integer value entered by the user
 */
 private static int promptUserForInteger(String prompt) {
 logger.info(prompt);
 String stringResponse = scanner.nextLine();
 if (stringResponse == null || stringResponse.trim().isEmpty() || !
isInteger(stringResponse)) {
 logger.info("Invalid integer.");
 return promptUserForInteger(prompt);
 }
 return Integer.parseInt(stringResponse);
 }

 /**
 * Checks if the given string represents a valid integer.
 *
 * @param str the string to be checked
 * @return {@code true} if the string represents a valid integer, {@code false}
otherwise
 */
 private static boolean isInteger(String str) {
 try {
 Integer.parseInt(str);

```

```

 return true;
 } catch (NumberFormatException e) {
 return false;
 }
}
}

```

## Wrapper for service operations.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryMode;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.scheduler.SchedulerAsyncClient;
import software.amazon.awssdk.services.scheduler.model.ActionAfterCompletion;
import software.amazon.awssdk.services.scheduler.model.ConflictException;
import software.amazon.awssdk.services.scheduler.model.CreateScheduleGroupRequest;
import software.amazon.awssdk.services.scheduler.model.CreateScheduleGroupResponse;
import software.amazon.awssdk.services.scheduler.model.CreateScheduleRequest;
import software.amazon.awssdk.services.scheduler.model.DeleteScheduleGroupRequest;
import software.amazon.awssdk.services.scheduler.model.DeleteScheduleRequest;
import software.amazon.awssdk.services.scheduler.model.DeleteScheduleResponse;
import software.amazon.awssdk.services.scheduler.model.FlexibleTimeWindow;
import software.amazon.awssdk.services.scheduler.model.FlexibleTimeWindowMode;
import software.amazon.awssdk.services.scheduler.model.ResourceNotFoundException;
import software.amazon.awssdk.services.scheduler.model.Target;

import java.time.Instant;
import java.util.concurrent.CompletableFuture;
import java.time.Duration;
import java.util.concurrent.CompletionException;

public class EventbridgeSchedulerActions {

 private static SchedulerAsyncClient schedulerClient;
 private static final Logger logger =
 LoggerFactory.getLogger(EventbridgeSchedulerActions.class);

```

```

 public static SchedulerAsyncClient getAsyncClient() {
 if (schedulerClient == null) {
 /*
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */

 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.
 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.
 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .retryStrategy(RetryMode.STANDARD)
 .build();

 schedulerClient = SchedulerAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return schedulerClient;
 }

 /**
 * Creates a new schedule group.
 *
 * @param name the name of the schedule group to be created

```

```

 * @return a {@link CompletableFuture} representing the asynchronous operation
 of creating the schedule group
 */
 public CompletableFuture<CreateScheduleGroupResponse> createScheduleGroup(String
name) {
 CreateScheduleGroupRequest request = CreateScheduleGroupRequest.builder()
 .name(name)
 .build();

 logger.info("Initiating createScheduleGroup call for group: {}", name);
 CompletableFuture<CreateScheduleGroupResponse> futureResponse =
getAsyncClient().createScheduleGroup(request);
 futureResponse.whenComplete((response, ex) -> {
 if (ex != null) {
 if (ex instanceof CompletionException && ex.getCause() instanceof
ConflictException) {
 // Rethrow the ConflictException
 throw (ConflictException) ex.getCause();
 } else {
 throw new CompletionException("Failed to create schedule group:
" + name, ex);
 }
 } else if (response == null) {
 throw new RuntimeException("Failed to create schedule group:
response was null");
 } else {
 logger.info("Successfully created schedule group '{}': {}", name,
response.scheduleGroupArn());
 }
 });

 return futureResponse;
 }

 /**
 * Creates a new schedule for a target task.
 *
 * @param name the name of the schedule
 * @param scheduleExpression The schedule expression that defines when the
schedule should run.
 * @param scheduleGroupName the name of the schedule group to which the
schedule belongs

```

```

 * @param targetArn the Amazon Resource Name (ARN) of the target
task
 * @param roleArn the ARN of the IAM role to be used for the
schedule
 * @param input the input data for the target task
 * @param deleteAfterCompletion whether to delete the schedule after it's
executed
 * @param useFlexibleTimeWindow whether to use a flexible time window for the
schedule execution
 * @return true if the schedule was successfully created, false otherwise
 */
 public CompletableFuture<Boolean> createScheduleAsync(
 String name,
 String scheduleExpression,
 String scheduleGroupName,
 String targetArn,
 String roleArn,
 String input,
 boolean deleteAfterCompletion,
 boolean useFlexibleTimeWindow) {

 int hoursToRun = 1;
 int flexibleTimeWindowMinutes = 10;

 Target target = Target.builder()
 .arn(targetArn)
 .roleArn(roleArn)
 .input(input)
 .build();

 FlexibleTimeWindow flexibleTimeWindow = FlexibleTimeWindow.builder()
 .mode(useFlexibleTimeWindow
 ? FlexibleTimeWindowMode.FLEXIBLE
 : FlexibleTimeWindowMode.OFF)
 .maximumWindowInMinutes(useFlexibleTimeWindow
 ? flexibleTimeWindowMinutes
 : null)
 .build();

 Instant startDate = Instant.now();
 Instant endDate = startDate.plus(Duration.ofHours(hoursToRun));

 CreateScheduleRequest request = CreateScheduleRequest.builder()
 .name(name)

```



```

 .scheduleExpression(scheduleExpression)
 .groupName(scheduleGroupName)
 .target(target)
 .actionAfterCompletion(deleteAfterCompletion
 ? ActionAfterCompletion.DELETE
 : ActionAfterCompletion.NONE)
 .startDate(startDate)
 .endDate(endDate)
 .flexibleTimeWindow(flexibleTimeWindow)
 .build();

 return getAsyncClient().createSchedule(request)
 .thenApply(response -> {
 logger.info("Successfully created schedule {} in schedule group {},
The ARN is {} ", name, scheduleGroupName, response.scheduleArn());
 return true;
 })
 .whenComplete((result, ex) -> {
 if (ex != null) {
 if (ex instanceof ConflictException) {
 // Handle ConflictException
 logger.error("A conflict exception occurred while creating
the schedule: {}", ex.getMessage());
 throw new CompletionException("A conflict exception occurred
while creating the schedule: " + ex.getMessage(), ex);
 } else {
 throw new CompletionException("Error creating schedule: " +
ex.getMessage(), ex);
 }
 }
 });
 }

/**
 * Deletes the specified schedule group.
 *
 * @param name the name of the schedule group to delete
 * @return a {@link CompletableFuture} that completes when the schedule group
has been deleted
 * @throws CompletionException if an error occurs while deleting the schedule
group
 */
 public CompletableFuture<Void> deleteScheduleGroupAsync(String name) {

```

```

 DeleteScheduleGroupRequest request = DeleteScheduleGroupRequest.builder()
 .name(name)
 .build();

 return getAsyncClient().deleteScheduleGroup(request)
 .thenRun(() -> {
 logger.info("Successfully deleted schedule group {}", name);
 })
 .whenComplete((result, ex) -> {
 if (ex != null) {
 if (ex instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource was not found: "
+ ex.getMessage(), ex);
 } else {
 throw new CompletionException("Error deleting schedule
group: " + ex.getMessage(), ex);
 }
 }
 });
 }

 /**
 * Deletes a schedule with the specified name and group name.
 *
 * @param name the name of the schedule to be deleted
 * @param groupName the group name of the schedule to be deleted
 * @return a {@link CompletableFuture} that, when completed, indicates whether
the schedule was successfully deleted
 * @throws CompletionException if an error occurs while deleting the schedule,
except for the case where the schedule is not found
 */
 public CompletableFuture<Boolean> deleteScheduleAsync(String name, String
groupName) {
 DeleteScheduleRequest request = DeleteScheduleRequest.builder()
 .name(name)
 .groupName(groupName)
 .build();

 CompletableFuture<DeleteScheduleResponse> response =
getAsyncClient().deleteSchedule(request);
 return response.handle((result, ex) -> {
 if (ex != null) {
 if (ex instanceof ResourceNotFoundException) {

```

```
 throw new CompletionException("Resource not found while deleting
schedule with ID: " + name, ex);
 } else {
 throw new CompletionException("Failed to delete schedule.", ex);
 }
}
logger.info("Successfully deleted schedule with name {}. ", name);
return true;
});
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateSchedule](#)
  - [CreateScheduleGroup](#)
  - [DeleteSchedule](#)
  - [DeleteScheduleGroups](#)

## Forecast examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Forecast.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### CreateDataset

The following code example shows how to use CreateDataset.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.CreateDatasetRequest;
import software.amazon.awssdk.services.forecast.model.Schema;
import software.amazon.awssdk.services.forecast.model.SchemaAttribute;
import software.amazon.awssdk.services.forecast.model.CreateDatasetResponse;
import software.amazon.awssdk.services.forecast.model.ForecastException;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateDataSet {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <name>\s

 Where:
 name - The name of the data set.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```

 String name = args[0];
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 String myDataSetARN = createForecastDataSet(forecast, name);
 System.out.println("The ARN of the new data set is " + myDataSetARN);
 forecast.close();
 }

 public static String createForecastDataSet(ForecastClient forecast, String name)
 {
 try {
 Schema schema = Schema.builder()
 .attributes(getSchema())
 .build();

 CreateDatasetRequest datasetRequest = CreateDatasetRequest.builder()
 .datasetName(name)
 .domain("CUSTOM")
 .datasetType("RELATED_TIME_SERIES")
 .dataFrequency("D")
 .schema(schema)
 .build();

 CreateDatasetResponse response = forecast.createDataset(datasetRequest);
 return response.datasetArn();

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return "";
 }

 // Create a SchemaAttribute list required to create a data set.
 private static List<SchemaAttribute> getSchema() {

 List<SchemaAttribute> schemaList = new ArrayList<>();
 SchemaAttribute att1 = SchemaAttribute.builder()
 .attributeName("item_id")
 .attributeType("string")

```

```

 .build();

 SchemaAttribute att2 = SchemaAttribute.builder()
 .attributeName("timestamp")
 .attributeType("timestamp")
 .build();

 SchemaAttribute att3 = SchemaAttribute.builder()
 .attributeName("target_value")
 .attributeType("float")
 .build();

 // Push the SchemaAttribute objects to the List.
 schemaList.add(att1);
 schemaList.add(att2);
 schemaList.add(att3);
 return schemaList;
 }
}

```

- For API details, see [CreateDataset](#) in *AWS SDK for Java 2.x API Reference*.

## CreateForecast

The following code example shows how to use CreateForecast.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.CreateForecastRequest;
import software.amazon.awssdk.services.forecast.model.CreateForecastResponse;
import software.amazon.awssdk.services.forecast.model.ForecastException;

/**

```

```

* Before running this Java V2 code example, set up your development
* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreateForecast {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <name> <predictorArn>\s

 Where:
 name - The name of the forecast.\s
 predictorArn - The arn of the predictor to use.\s

 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String name = args[0];
 String predictorArn = args[1];
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 String forecastArn = createNewForecast(forecast, name, predictorArn);
 System.out.println("The ARN of the new forecast is " + forecastArn);
 forecast.close();
 }

 public static String createNewForecast(ForecastClient forecast, String name,
 String predictorArn) {
 try {
 CreateForecastRequest forecastRequest = CreateForecastRequest.builder()
 .forecastName(name)
 .predictorArn(predictorArn)
 .build();

```

```

 CreateForecastResponse response =
forecast.createForecast(forecastRequest);
 return response.forecastArn();

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see [CreateForecast](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDataset

The following code example shows how to use DeleteDataset.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.DeleteDatasetRequest;
import software.amazon.awssdk.services.forecast.model.ForecastException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteDataset {

```



```

public static void main(String[] args) {
 final String usage = ""

 Usage:
 <datasetARN>\s

 Where:
 datasetARN - The ARN of the data set to delete.\s
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String datasetARN = args[0];
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 deleteForecastDataSet(forecast, datasetARN);
 forecast.close();
}

public static void deleteForecastDataSet(ForecastClient forecast, String
myDataSetARN) {
 try {
 DeleteDatasetRequest deleteRequest = DeleteDatasetRequest.builder()
 .datasetArn(myDataSetARN)
 .build();

 forecast.deleteDataset(deleteRequest);
 System.out.println("The Data Set was deleted");

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteDataset](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteForecast

The following code example shows how to use DeleteForecast.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.DeleteDatasetRequest;
import software.amazon.awssdk.services.forecast.model.ForecastException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteDataset {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <datasetARN>\s

 Where:
 datasetARN - The ARN of the data set to delete.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```

 }

 String datasetARN = args[0];
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 deleteForecastDataSet(forecast, datasetARN);
 forecast.close();
}

public static void deleteForecastDataSet(ForecastClient forecast, String
myDataSetARN) {
 try {
 DeleteDatasetRequest deleteRequest = DeleteDatasetRequest.builder()
 .datasetArn(myDataSetARN)
 .build();

 forecast.deleteDataset(deleteRequest);
 System.out.println("The Data Set was deleted");

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [DeleteForecast](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeForecast

The following code example shows how to use DescribeForecast.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.DescribeForecastRequest;
import software.amazon.awssdk.services.forecast.model.DescribeForecastResponse;
import software.amazon.awssdk.services.forecast.model.ForecastException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeForecast {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <forecastarn>\s

 Where:
 forecastarn - The arn of the forecast (for example,
 "arn:aws:forecast:us-west-2:xxxxx322:forecast/my_forecast")
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String forecastarn = args[0];
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 describe(forecast, forecastarn);
 forecast.close();
 }

 public static void describe(ForecastClient forecast, String forecastarn) {
 try {

```

```

 DescribeForecastRequest request = DescribeForecastRequest.builder()
 .forecastArn(forecastarn)
 .build();

 DescribeForecastResponse response = forecast.describeForecast(request);
 System.out.println("The name of the forecast is " +
 response.forecastName());

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeForecast](#) in *AWS SDK for Java 2.x API Reference*.

## ListDatasetGroups

The following code example shows how to use `ListDatasetGroups`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.DatasetGroupSummary;
import software.amazon.awssdk.services.forecast.model.ListDatasetGroupsRequest;
import software.amazon.awssdk.services.forecast.model.ListDatasetGroupsResponse;
import software.amazon.awssdk.services.forecast.model.ForecastException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.

```

```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class ListDataSetGroups {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 listDataGroups(forecast);
 forecast.close();
 }

 public static void listDataGroups(ForecastClient forecast) {
 try {
 ListDatasetGroupsRequest group = ListDatasetGroupsRequest.builder()
 .maxResults(10)
 .build();

 ListDatasetGroupsResponse response = forecast.listDatasetGroups(group);
 List<DatasetGroupSummary> groups = response.datasetGroups();
 for (DatasetGroupSummary myGroup : groups) {
 System.out.println("The Data Set name is " +
myGroup.datasetGroupName());
 }

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListDatasetGroups](#) in *AWS SDK for Java 2.x API Reference*.

## ListForecasts

The following code example shows how to use ListForecasts.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.forecast.ForecastClient;
import software.amazon.awssdk.services.forecast.model.ListForecastsResponse;
import software.amazon.awssdk.services.forecast.model.ListForecastsRequest;
import software.amazon.awssdk.services.forecast.model.ForecastSummary;
import software.amazon.awssdk.services.forecast.model.ForecastException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListForecasts {

 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 ForecastClient forecast = ForecastClient.builder()
 .region(region)
 .build();

 listAllForecasts(forecast);
 forecast.close();
 }

 public static void listAllForecasts(ForecastClient forecast) {
 try {
 ListForecastsRequest request = ListForecastsRequest.builder()
 .maxResults(10)
 .build();
```

```
 ListForecastsResponse response = forecast.listForecasts(request);
 List<ForecastSummary> forecasts = response.forecasts();
 for (ForecastSummary forecastSummary : forecasts) {
 System.out.println("The name of the forecast is " +
forecastSummary.forecastName());
 }

 } catch (ForecastException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListForecasts](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Glacier examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Glacier.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### CreateVault

The following code example shows how to use CreateVault.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.CreateVaultRequest;
import software.amazon.awssdk.services.glacier.model.CreateVaultResponse;
import software.amazon.awssdk.services.glacier.model.GlacierException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateVault {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <vaultName>

 Where:
 vaultName - The name of the vault to create.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String vaultName = args[0];
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();
```

```
 createGlacierVault(glacier, vaultName);
 glacier.close();
 }

 public static void createGlacierVault(GlacierClient glacier, String vaultName) {
 try {
 CreateVaultRequest vaultRequest = CreateVaultRequest.builder()
 .vaultName(vaultName)
 .build();

 CreateVaultResponse createVaultResult =
glacier.createVault(vaultRequest);
 System.out.println("The URI of the new vault is " +
createVaultResult.location());

 } catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [CreateVault](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteArchive

The following code example shows how to use DeleteArchive.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.DeleteArchiveRequest;
import software.amazon.awssdk.services.glacier.model.GlacierException;
```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteArchive {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <vaultName> <accountId> <archiveId>

 Where:
 vaultName - The name of the vault that contains the archive to
delete.
 accountId - The account ID value.
 archiveId - The archive ID value.
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String vaultName = args[0];
 String accountId = args[1];
 String archiveId = args[2];
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();

 deleteGlacierArchive(glacier, vaultName, accountId, archiveId);
 glacier.close();
 }

 public static void deleteGlacierArchive(GlacierClient glacier, String vaultName,
String accountId,
 String archiveId) {
 try {
 DeleteArchiveRequest delArcRequest = DeleteArchiveRequest.builder()
 .vaultName(vaultName)

```

```

 .accountId(accountId)
 .archiveId/archiveId)
 .build();

 glacier.deleteArchive(delArcRequest);
 System.out.println("The archive was deleted.");

 } catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteArchive](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteVault

The following code example shows how to use DeleteVault.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.DeleteVaultRequest;
import software.amazon.awssdk.services.glacier.model.GlacierException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

```

```

 */
 public class DeleteVault {
 public static void main(String[] args) {

 final String usage = ""

 Usage: <vaultName>

 Where:
 vaultName - The name of the vault to delete.\s
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String vaultName = args[0];
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();

 deleteGlacierVault(glacier, vaultName);
 glacier.close();
 }

 public static void deleteGlacierVault(GlacierClient glacier, String vaultName) {
 try {
 DeleteVaultRequest delVaultRequest = DeleteVaultRequest.builder()
 .vaultName(vaultName)
 .build();

 glacier.deleteVault(delVaultRequest);
 System.out.println("The vault was deleted!");

 } catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
 }
}

```

- For API details, see [DeleteVault](#) in *AWS SDK for Java 2.x API Reference*.

## InitiateJob

The following code example shows how to use `InitiateJob`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Retrieve a vault inventory.

```
import software.amazon.awssdk.core.ResponseBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.JobParameters;
import software.amazon.awssdk.services.glacier.model.InitiateJobResponse;
import software.amazon.awssdk.services.glacier.model.GlacierException;
import software.amazon.awssdk.services.glacier.model.InitiateJobRequest;
import software.amazon.awssdk.services.glacier.model.DescribeJobRequest;
import software.amazon.awssdk.services.glacier.model.DescribeJobResponse;
import software.amazon.awssdk.services.glacier.model.GetJobOutputRequest;
import software.amazon.awssdk.services.glacier.model.GetJobOutputResponse;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStream;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ArchiveDownload {
 public static void main(String[] args) {

 final String usage = ""
```

```

 Usage: <vaultName> <accountId> <path>

 Where:
 vaultName - The name of the vault.
 accountId - The account ID value.
 path - The path where the file is written to.
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String vaultName = args[0];
 String accountId = args[1];
 String path = args[2];
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String jobNum = createJob(glacier, vaultName, accountId);
 checkJob(glacier, jobNum, vaultName, accountId, path);
 glacier.close();
}

public static String createJob(GlacierClient glacier, String vaultName, String
accountId) {
 try {
 JobParameters job = JobParameters.builder()
 .type("inventory-retrieval")
 .build();

 InitiateJobRequest initJob = InitiateJobRequest.builder()
 .jobParameters(job)
 .accountId(accountId)
 .vaultName(vaultName)
 .build();

 InitiateJobResponse response = glacier.initiateJob(initJob);
 System.out.println("The job ID is: " + response.jobId());
 System.out.println("The relative URI path of the job is: " +
response.location());
 return response.jobId();
 }
}

```

```
 } catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

// Poll S3 Glacier = Polling a Job may take 4-6 hours according to the
// Documentation.
public static void checkJob(GlacierClient glacier, String jobId, String name,
String account, String path) {
 try {
 boolean finished = false;
 String jobStatus;
 int yy = 0;

 while (!finished) {
 DescribeJobRequest jobRequest = DescribeJobRequest.builder()
 .jobId(jobId)
 .accountId(account)
 .vaultName(name)
 .build();

 DescribeJobResponse response = glacier.describeJob(jobRequest);
 jobStatus = response.statusCodeAsString();

 if (jobStatus.compareTo("Succeeded") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + jobStatus);
 Thread.sleep(1000);
 }
 yy++;
 }

 System.out.println("Job has Succeeded");
 GetJobOutputRequest jobOutputRequest = GetJobOutputRequest.builder()
 .jobId(jobId)
 .vaultName(name)
 .accountId(account)
 .build();
```



```

 ResponseBytes<GetJobOutputResponse> objectBytes =
glacier.getJobOutputAsBytes(jobOutputRequest);
 // Write the data to a local file.
 byte[] data = objectBytes.asByteArray();
 File myFile = new File(path);
 OutputStream os = new FileOutputStream(myFile);
 os.write(data);
 System.out.println("Successfully obtained bytes from a Glacier vault");
 os.close();

 } catch (GlacierException | InterruptedException | IOException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [InitiateJob](#) in *AWS SDK for Java 2.x API Reference*.

## ListVaults

The following code example shows how to use `ListVaults`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.model.ListVaultsRequest;
import software.amazon.awssdk.services.glacier.model.ListVaultsResponse;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.DescribeVaultOutput;
import software.amazon.awssdk.services.glacier.model.GlacierException;
import java.util.List;

/**

```

```
* Before running this Java V2 code example, set up your development
* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class ListVaults {
 public static void main(String[] args) {
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllVault(glacier);
 glacier.close();
 }

 public static void listAllVault(GlacierClient glacier) {
 boolean listComplete = false;
 String newMarker = null;
 int totalVaults = 0;
 System.out.println("Your Amazon Glacier vaults:");
 try {
 while (!listComplete) {
 ListVaultsResponse response = null;
 if (newMarker != null) {
 ListVaultsRequest request = ListVaultsRequest.builder()
 .marker(newMarker)
 .build();

 response = glacier.listVaults(request);
 } else {
 ListVaultsRequest request = ListVaultsRequest.builder()
 .build();
 response = glacier.listVaults(request);
 }

 List<DescribeVaultOutput> vaultList = response.vaultList();
 for (DescribeVaultOutput v : vaultList) {
 totalVaults += 1;
 System.out.println("* " + v.vaultName());
 }

 // Check for further results.
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

```
 newMarker = response.marker();
 if (newMarker == null) {
 listComplete = true;
 }
 }

 if (totalVaults == 0) {
 System.out.println("No vaults found.");
 }

} catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see [ListVaults](#) in *AWS SDK for Java 2.x API Reference*.

## UploadArchive

The following code example shows how to use UploadArchive.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glacier.GlacierClient;
import software.amazon.awssdk.services.glacier.model.UploadArchiveRequest;
import software.amazon.awssdk.services.glacier.model.UploadArchiveResponse;
import software.amazon.awssdk.services.glacier.model.GlacierException;
import java.io.File;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.io.FileInputStream;
import java.io.IOException;
```

```

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class UploadArchive {

 static final int ONE_MB = 1024 * 1024;

 public static void main(String[] args) {
 final String usage = ""

 Usage: <strPath> <vaultName>\s

 Where:
 strPath - The path to the archive to upload (for example, C:\\AWS
\\test.pdf).
 vaultName - The name of the vault.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String strPath = args[0];
 String vaultName = args[1];
 File myFile = new File(strPath);
 Path path = Paths.get(strPath);
 GlacierClient glacier = GlacierClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String archiveId = uploadContent(glacier, path, vaultName, myFile);
 System.out.println("The ID of the archived item is " + archiveId);
 glacier.close();
 }
}

```

```

 public static String uploadContent(GlacierClient glacier, Path path, String
vaultName, File myFile) {
 // Get an SHA-256 tree hash value.
 String checkVal = computeSHA256(myFile);
 try {
 UploadArchiveRequest uploadRequest = UploadArchiveRequest.builder()
 .vaultName(vaultName)
 .checksum(checkVal)
 .build();

 UploadArchiveResponse res = glacier.uploadArchive(uploadRequest, path);
 return res.archiveId();

 } catch (GlacierException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }

 private static String computeSHA256(File inputFile) {
 try {
 byte[] treeHash = computeSHA256TreeHash(inputFile);
 System.out.printf("SHA-256 tree hash = %s\n", toHex(treeHash));
 return toHex(treeHash);

 } catch (IOException ioe) {
 System.err.format("Exception when reading from file %s: %s", inputFile,
ioe.getMessage());
 System.exit(-1);

 } catch (NoSuchAlgorithmException nsae) {
 System.err.format("Cannot locate MessageDigest algorithm for SHA-256:
%s", nsae.getMessage());
 System.exit(-1);
 }
 return "";
 }

 public static byte[] computeSHA256TreeHash(File inputFile) throws IOException,
NoSuchAlgorithmException {

 byte[][] chunkSHA256Hashes = getChunkSHA256Hashes(inputFile);
 return computeSHA256TreeHash(chunkSHA256Hashes);
 }

```

```

 }

 /**
 * Computes an SHA256 checksum for each 1 MB chunk of the input file. This
 * includes the checksum for the last chunk, even if it's smaller than 1 MB.
 */
 public static byte[][] getChunkSHA256Hashes(File file) throws IOException,
 NoSuchAlgorithmException {

 MessageDigest md = MessageDigest.getInstance("SHA-256");
 long numChunks = file.length() / ONE_MB;
 if (file.length() % ONE_MB > 0) {
 numChunks++;
 }

 if (numChunks == 0) {
 return new byte[][] { md.digest() };
 }

 byte[][] chunkSHA256Hashes = new byte[(int) numChunks][];
 FileInputStream fileStream = null;

 try {
 fileStream = new FileInputStream(file);
 byte[] buff = new byte[ONE_MB];

 int bytesRead;
 int idx = 0;

 while ((bytesRead = fileStream.read(buff, 0, ONE_MB)) > 0) {
 md.reset();
 md.update(buff, 0, bytesRead);
 chunkSHA256Hashes[idx++] = md.digest();
 }

 return chunkSHA256Hashes;

 } finally {
 if (fileStream != null) {
 try {
 fileStream.close();
 } catch (IOException ioe) {
 System.err.printf("Exception while closing %s.\n %s",
file.getName(),

```

```

 ioe.getMessage());
 }
}

/**
 * Computes the SHA-256 tree hash for the passed array of 1 MB chunk
 * checksums.
 */
public static byte[] computeSHA256TreeHash(byte[][] chunkSHA256Hashes)
 throws NoSuchAlgorithmException {

 MessageDigest md = MessageDigest.getInstance("SHA-256");
 byte[][] prevLvlHashes = chunkSHA256Hashes;
 while (prevLvlHashes.length > 1) {
 int len = prevLvlHashes.length / 2;
 if (prevLvlHashes.length % 2 != 0) {
 len++;
 }

 byte[][] currLvlHashes = new byte[len][];
 int j = 0;
 for (int i = 0; i < prevLvlHashes.length; i = i + 2, j++) {

 // If there are at least two elements remaining.
 if (prevLvlHashes.length - i > 1) {

 // Calculate a digest of the concatenated nodes.
 md.reset();
 md.update(prevLvlHashes[i]);
 md.update(prevLvlHashes[i + 1]);
 currLvlHashes[j] = md.digest();

 } else { // Take care of the remaining odd chunk
 currLvlHashes[j] = prevLvlHashes[i];
 }
 }

 prevLvlHashes = currLvlHashes;
 }

 return prevLvlHashes[0];
}

```

```
/**
 * Returns the hexadecimal representation of the input byte array
 */
public static String toHex(byte[] data) {
 StringBuilder sb = new StringBuilder(data.length * 2);
 for (byte datum : data) {
 String hex = Integer.toHexString(datum & 0xFF);

 if (hex.length() == 1) {
 // Append leading zero.
 sb.append("0");
 }
 sb.append(hex);
 }
 return sb.toString().toLowerCase();
}
```

- For API details, see [UploadArchive](#) in *AWS SDK for Java 2.x API Reference*.

## AWS Glue examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Glue.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)



# Get started

## Hello AWS Glue

The following code example shows how to get started using AWS Glue.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.glue;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.glue.GlueClient;
import software.amazon.awssdk.services.glue.model.ListJobsRequest;
import software.amazon.awssdk.services.glue.model.ListJobsResponse;
import java.util.List;

public class HelloGlue {
 public static void main(String[] args) {
 GlueClient glueClient = GlueClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listJobs(glueClient);
 }

 public static void listJobs(GlueClient glueClient) {
 ListJobsRequest request = ListJobsRequest.builder()
 .maxResults(10)
 .build();
 ListJobsResponse response = glueClient.listJobs(request);
 List<String> jobList = response.jobNames();
 jobList.forEach(job -> {
 System.out.println("Job Name: " + job);
 });
 }
}
```

- For API details, see [ListJobs](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a crawler that crawls a public Amazon S3 bucket and generates a database of CSV-formatted metadata.
- List information about databases and tables in your AWS Glue Data Catalog.
- Create a job to extract CSV data from the S3 bucket, transform the data, and load JSON-formatted output into another S3 bucket.
- List information about job runs, view transformed data, and clean up resources.

For more information, see [Tutorial: Getting started with AWS Glue Studio](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * <p>
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * To set up the resources, see this documentation topic:
 *
 * https://docs.aws.amazon.com/glue/latest/ug/tutorial-add-crawler.html
```

```

*
* This example performs the following tasks:
*
* 1. Create a database.
* 2. Create a crawler.
* 3. Get a crawler.
* 4. Start a crawler.
* 5. Get a database.
* 6. Get tables.
* 7. Create a job.
* 8. Start a job run.
* 9. List all jobs.
* 10. Get job runs.
* 11. Delete a job.
* 12. Delete a database.
* 13. Delete a crawler.
*/

public class GlueScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws InterruptedException {
 final String usage = ""

 Usage:
 <iam> <s3Path> <cron> <dbName> <crawlerName> <jobName>
<scriptLocation> <locationUri> <bucketNameSc>\s

 Where:
 iam - The ARN of the IAM role that has AWS Glue and S3 permissions.
\s
 s3Path - The Amazon Simple Storage Service (Amazon S3) target that
contains data (for example, s3://<bucket name>/read).
 cron - A cron expression used to specify the schedule (i.e.,
cron(15 12 * * ? *).
 dbName - The database name.\s
 crawlerName - The name of the crawler.\s
 jobName - The name you assign to this job definition.
 scriptLocation - The Amazon S3 path to a script that runs a job.
 locationUri - The location of the database (you can find this file
in resources folder).
 bucketNameSc - The Amazon S3 bucket name used when creating a job
""";

```

```

 if (args.length != 9) {
 System.out.println(usage);
 return;
 }
 Scanner scanner = new Scanner(System.in);
 String iam = args[0];
 String s3Path = args[1];
 String cron = args[2];
 String dbName = args[3];
 String crawlerName = args[4];
 String jobName = args[5];
 String scriptLocation = args[6];
 String locationUri = args[7];
 String bucketNameSc = args[8];

 Region region = Region.US_EAST_1;
 GlueClient glueClient = GlueClient.builder()
 .region(region)
 .build();
 System.out.println(DASHES);
 System.out.println("Welcome to the AWS Glue scenario.");
 System.out.println("""
 AWS Glue is a fully managed extract, transform, and load (ETL) service
provided by Amazon
 Web Services (AWS). It is designed to simplify the process of building,
running, and maintaining
 ETL pipelines, which are essential for data integration and data
warehousing tasks.

 One of the key features of AWS Glue is its ability to automatically
discover and catalog data
 stored in various sources, such as Amazon S3, Amazon RDS, Amazon
Redshift, and other databases.
 This cataloging process creates a central metadata repository, known as
the AWS Glue Data Catalog,
 which provides a unified view of an organization's data assets. This
metadata can then be used to
 create ETL jobs, which can be scheduled and run on-demand or on a
regular basis.

 Lets get started.

 """);
 waitForInputToContinue(scanner);

```

```
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("1. Create a database.");
try {
 createDatabase(glueClient, dbName, locationUri);
} catch (GlueException e) {
 if (e.awsErrorDetails().errorMessage().equals("Database already
exists.")) {
 System.out.println("Database " + dbName + " already exists. Skipping
creation.");
 } else {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
}

waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("2. Create a crawler.");
try {
 createGlueCrawler(glueClient, iam, s3Path, cron, dbName, crawlerName);
} catch (GlueException e) {
 if (e.awsErrorDetails().errorMessage().contains("already exists")) {
 System.out.println("Crawler " + crawlerName + " already exists.
Skipping creation.");
 } else {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Get a crawler.");
try {
 getSpecificCrawler(glueClient, crawlerName);
} catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
}
```

```
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Start a crawler.");
try {
 startSpecificCrawler(glueClient, crawlerName);
} catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
}
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Get a database.");
try {
 getSpecificDatabase(glueClient, dbName);
} catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
}
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("*** Wait 5 min for the tables to become available");
TimeUnit.MINUTES.sleep(5);
System.out.println("6. Get tables.");
String myTableName;
try {
 myTableName = getGlueTables(glueClient, dbName);
} catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
}
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Create a job.");
try {
 createJob(glueClient, jobName, iam, scriptLocation);
} catch (GlueException e) {
```

```
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Start a Job run.");
 try {
 startJob(glueClient, jobName, dbName, myTableName, bucketNameSc);
 } catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. List all jobs.");
 try {
 getAllJobs(glueClient);
 } catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Get job runs.");
 try {
 getJobRuns(glueClient, jobName);
 } catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("11. Delete a job.");
 try {
 deleteJob(glueClient, jobName);
 } catch (GlueException e) {
```

```

 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 System.out.println("*** Wait 5 MIN for the " + crawlerName + " to stop");
 TimeUnit.MINUTES.sleep(5);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("12. Delete a database.");
 try {
 deleteDatabase(glueClient, dbName);
 } catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Delete a crawler.");
 try {
 deleteSpecificCrawler(glueClient, crawlerName);
 } catch (GlueException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Successfully completed the AWS Glue Scenario");
 System.out.println(DASHES);
}

/**
 * Creates a Glue database with the specified name and location URI.
 *
 * @param glueClient The Glue client to use for the database creation.
 * @param dbName The name of the database to create.
 * @param locationUri The location URI for the database.
 */

```



```

 public static void createDatabase(GlueClient glueClient, String dbName, String
locationUri) {
 try {
 DatabaseInput input = DatabaseInput.builder()
 .description("Built with the AWS SDK for Java V2")
 .name(dbName)
 .locationUri(locationUri)
 .build();

 CreateDatabaseRequest request = CreateDatabaseRequest.builder()
 .databaseInput(input)
 .build();

 glueClient.createDatabase(request);
 System.out.println(dbName + " was successfully created");

 } catch (GlueException e) {
 throw e;
 }
 }

 /**
 * Creates a new AWS Glue crawler using the AWS Glue Java API.
 *
 * @param glueClient the AWS Glue client used to interact with the AWS Glue
service
 * @param iam the IAM role that the crawler will use to access the data
source
 * @param s3Path the S3 path that the crawler will scan for data
 * @param cron the cron expression that defines the crawler's schedule
 * @param dbName the name of the AWS Glue database where the crawler will
store the metadata
 * @param crawlerName the name of the crawler to be created
 */
 public static void createGlueCrawler(GlueClient glueClient,
 String iam,
 String s3Path,
 String cron,
 String dbName,
 String crawlerName) {

 try {
 S3Target s3Target = S3Target.builder()

```

```

 .path(s3Path)
 .build();

List<S3Target> targetList = new ArrayList<>();
targetList.add(s3Target);
CrawlerTargets targets = CrawlerTargets.builder()
 .s3Targets(targetList)
 .build();

CreateCrawlerRequest crawlerRequest = CreateCrawlerRequest.builder()
 .databaseName(dbName)
 .name(crawlerName)
 .description("Created by the AWS Glue Java API")
 .targets(targets)
 .role(iam)
 .schedule(cron)
 .build();

glueClient.createCrawler(crawlerRequest);
System.out.println(crawlerName + " was successfully created");

 } catch (GlueException e) {
 throw e;
 }
}

/**
 * Retrieves a specific crawler from the AWS Glue service and waits for it to be
in the "READY" state.
 *
 * @param glueClient the AWS Glue client used to interact with the Glue service
 * @param crawlerName the name of the crawler to be retrieved
 */
public static void getSpecificCrawler(GlueClient glueClient, String crawlerName)
throws InterruptedException {
 try {
 GetCrawlerRequest crawlerRequest = GetCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 boolean ready = false;
 while (!ready) {
 GetCrawlerResponse response = glueClient.getCrawler(crawlerRequest);
 String status = response.crawler().stateAsString();

```

```

 if (status.compareTo("READY") == 0) {
 ready = true;
 }
 Thread.sleep(3000);
 }

 System.out.println("The crawler is now ready");

} catch (GlueException | InterruptedException e) {
 throw e;
}
}

/**
 * Starts a specific AWS Glue crawler.
 *
 * @param glueClient the AWS Glue client to use for the crawler operation
 * @param crawlerName the name of the crawler to start
 * @throws GlueException if there is an error starting the crawler
 */
public static void startSpecificCrawler(GlueClient glueClient, String
crawlerName) {
 try {
 StartCrawlerRequest crawlerRequest = StartCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 glueClient.startCrawler(crawlerRequest);
 System.out.println(crawlerName + " was successfully started!");

 } catch (GlueException e) {
 throw e;
 }
}

/**
 * Retrieves the specific database from the AWS Glue service.
 *
 * @param glueClient an instance of the AWS Glue client used to interact with
the service
 * @param databaseName the name of the database to retrieve
 * @throws GlueException if there is an error retrieving the database from the
AWS Glue service
 */

```

```

 public static void getSpecificDatabase(GlueClient glueClient, String
databaseName) {
 try {
 GetDatabaseRequest databasesRequest = GetDatabaseRequest.builder()
 .name(databaseName)
 .build();

 GetDatabaseResponse response = glueClient.getDatabase(databasesRequest);
 Instant createDate = response.database().createTime();

 // Convert the Instant to readable date.
 DateTimeFormatter formatter =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT)
 .withLocale(Locale.US)
 .withZone(ZoneId.systemDefault());

 formatter.format(createDate);
 System.out.println("The create date of the database is " + createDate);

 } catch (GlueException e) {
 throw e;
 }
 }

 /**
 * Retrieves the names of the tables in the specified Glue database.
 *
 * @param glueClient the Glue client to use for the operation
 * @param dbName the name of the Glue database to retrieve the table names
from
 * @return the name of the first table retrieved, or an empty string if no
tables were found
 */
 public static String getGlueTables(GlueClient glueClient, String dbName) {
 String myTableName = "";
 try {
 GetTablesRequest tableRequest = GetTablesRequest.builder()
 .databaseName(dbName)
 .build();

 GetTablesResponse response = glueClient.getTables(tableRequest);
 List<Table> tables = response.tableList();
 if (tables.isEmpty()) {

```

```

 System.out.println("No tables were returned");
 } else {
 for (Table table : tables) {
 myTableName = table.name();
 System.out.println("Table name is: " + myTableName);
 }
 }

 } catch (GlueException e) {
 throw e;
 }
 return myTableName;
}

/**
 * Starts a job run in AWS Glue.
 *
 * @param glueClient the AWS Glue client to use for the job run
 * @param jobName the name of the Glue job to run
 * @param inputDatabase the name of the input database
 * @param inputTable the name of the input table
 * @param outBucket the URL of the output S3 bucket
 * @throws GlueException if there is an error starting the job run
 */
public static void startJob(GlueClient glueClient, String jobName, String
inputDatabase, String inputTable,
 String outBucket) {
 try {
 Map<String, String> myMap = new HashMap<>();
 myMap.put("--input_database", inputDatabase);
 myMap.put("--input_table", inputTable);
 myMap.put("--output_bucket_url", outBucket);

 StartJobRunRequest runRequest = StartJobRunRequest.builder()
 .workerType(WorkerType.G_1_X)
 .numberOfWorkers(10)
 .arguments(myMap)
 .jobName(jobName)
 .build();

 StartJobRunResponse response = glueClient.startJobRun(runRequest);
 System.out.println("The request Id of the job is " +
response.responseMetadata().requestId());
 }
}

```

```
 } catch (GlueException e) {
 throw e;
 }
 }

 /**
 * Creates a new AWS Glue job.
 *
 * @param glueClient the AWS Glue client to use for the operation
 * @param jobName the name of the job to create
 * @param iam the IAM role to associate with the job
 * @param scriptLocation the location of the script to be used by the job
 * @throws GlueException if there is an error creating the job
 */
 public static void createJob(GlueClient glueClient, String jobName, String iam,
String scriptLocation) {
 try {
 JobCommand command = JobCommand.builder()
 .pythonVersion("3")
 .name("glueetl")
 .scriptLocation(scriptLocation)
 .build();

 CreateJobRequest jobRequest = CreateJobRequest.builder()
 .description("A Job created by using the AWS SDK for Java V2")
 .glueVersion("2.0")
 .workerType(WorkerType.G_1_X)
 .numberOfWorkers(10)
 .name(jobName)
 .role(iam)
 .command(command)
 .build();

 glueClient.createJob(jobRequest);
 System.out.println(jobName + " was successfully created.");

 } catch (GlueException e) {
 throw e;
 }
 }
}
```

```

/**
 * Retrieves and prints information about all the jobs in the Glue data catalog.
 *
 * @param glueClient the Glue client used to interact with the AWS Glue service
 */
public static void getAllJobs(GlueClient glueClient) {
 try {
 GetJobsRequest jobsRequest = GetJobsRequest.builder()
 .maxResults(10)
 .build();

 GetJobsResponse jobsResponse = glueClient.getJobs(jobsRequest);
 List<Job> jobs = jobsResponse.jobs();
 for (Job job : jobs) {
 System.out.println("Job name is : " + job.name());
 System.out.println("The job worker type is : " +
job.workerType().name());
 }

 } catch (GlueException e) {
 throw e;
 }
}

/**
 * Retrieves the job runs for a given Glue job and prints the status of the job
runs.
 *
 * @param glueClient the Glue client used to make API calls
 * @param jobName the name of the Glue job to retrieve the job runs for
 */
public static void getJobRuns(GlueClient glueClient, String jobName) {
 try {
 GetJobRunsRequest runsRequest = GetJobRunsRequest.builder()
 .jobName(jobName)
 .maxResults(20)
 .build();

 boolean jobDone = false;
 while (!jobDone) {
 GetJobRunsResponse response = glueClient.getJobRuns(runsRequest);
 List<JobRun> jobRuns = response.jobRuns();
 for (JobRun jobRun : jobRuns) {
 String jobState = jobRun.jobRunState().name();

```

```

 if (jobState.compareTo("SUCCEEDED") == 0) {
 System.out.println(jobName + " has succeeded");
 jobDone = true;

 } else if (jobState.compareTo("STOPPED") == 0) {
 System.out.println("Job run has stopped");
 jobDone = true;

 } else if (jobState.compareTo("FAILED") == 0) {
 System.out.println("Job run has failed");
 jobDone = true;

 } else if (jobState.compareTo("TIMEOUT") == 0) {
 System.out.println("Job run has timed out");
 jobDone = true;

 } else {
 System.out.println("*** Job run state is " +
jobRun.jobRunState().name());
 System.out.println("Job run Id is " + jobRun.id());
 System.out.println("The Glue version is " +
jobRun.glueVersion());
 }
 TimeUnit.SECONDS.sleep(5);
 }

 } catch (GlueException e) {
 throw e;
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
}

/**
 * Deletes a Glue job.
 *
 * @param glueClient the Glue client to use for the operation
 * @param jobName the name of the job to be deleted
 * @throws GlueException if there is an error deleting the job
 */
public static void deleteJob(GlueClient glueClient, String jobName) {
 try {

```



```

 DeleteJobRequest jobRequest = DeleteJobRequest.builder()
 .jobName(jobName)
 .build();

 glueClient.deleteJob(jobRequest);
 System.out.println(jobName + " was successfully deleted");

 } catch (GlueException e) {
 throw e;
 }
}

/**
 * Deletes a AWS Glue Database.
 *
 * @param glueClient An instance of the AWS Glue client used to interact with
the AWS Glue service.
 * @param databaseName The name of the database to be deleted.
 * @throws GlueException If an error occurs while deleting the database.
 */
public static void deleteDatabase(GlueClient glueClient, String databaseName) {
 try {
 DeleteDatabaseRequest request = DeleteDatabaseRequest.builder()
 .name(databaseName)
 .build();

 glueClient.deleteDatabase(request);
 System.out.println(databaseName + " was successfully deleted");

 } catch (GlueException e) {
 throw e;
 }
}

/**
 * Deletes a specific AWS Glue crawler.
 *
 * @param glueClient the AWS Glue client object
 * @param crawlerName the name of the crawler to be deleted
 * @throws GlueException if an error occurs during the deletion process
 */
public static void deleteSpecificCrawler(GlueClient glueClient, String
crawlerName) {

```

```
 try {
 DeleteCrawlerRequest deleteCrawlerRequest =
DeleteCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 glueClient.deleteCrawler(deleteCrawlerRequest);
 System.out.println(crawlerName + " was deleted");

 } catch (GlueException e) {
 throw e;
 }
 }

 private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
 }
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateCrawler](#)
  - [CreateJob](#)
  - [DeleteCrawler](#)
  - [DeleteDatabase](#)
  - [DeleteJob](#)
  - [DeleteTable](#)

- [GetCrawler](#)
- [GetDatabase](#)
- [GetDatabases](#)
- [GetJob](#)
- [GetJobRun](#)
- [GetJobRuns](#)
- [GetTables](#)
- [ListJobs](#)
- [StartCrawler](#)
- [StartJobRun](#)

## Actions

### CreateCrawler

The following code example shows how to use CreateCrawler.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new AWS Glue crawler using the AWS Glue Java API.
 *
 * @param glueClient the AWS Glue client used to interact with the AWS Glue
service
 * @param iam the IAM role that the crawler will use to access the data
source
 * @param s3Path the S3 path that the crawler will scan for data
 * @param cron the cron expression that defines the crawler's schedule
 * @param dbName the name of the AWS Glue database where the crawler will
store the metadata
```

```

 * @param crawlerName the name of the crawler to be created
 */
 public static void createGlueCrawler(GlueClient glueClient,
 String iam,
 String s3Path,
 String cron,
 String dbName,
 String crawlerName) {

 try {
 S3Target s3Target = S3Target.builder()
 .path(s3Path)
 .build();

 List<S3Target> targetList = new ArrayList<>();
 targetList.add(s3Target);
 CrawlerTargets targets = CrawlerTargets.builder()
 .s3Targets(targetList)
 .build();

 CreateCrawlerRequest crawlerRequest = CreateCrawlerRequest.builder()
 .databaseName(dbName)
 .name(crawlerName)
 .description("Created by the AWS Glue Java API")
 .targets(targets)
 .role(iam)
 .schedule(cron)
 .build();

 glueClient.createCrawler(crawlerRequest);
 System.out.println(crawlerName + " was successfully created");

 } catch (GlueException e) {
 throw e;
 }
 }
}

```

- For API details, see [CreateCrawler](#) in *AWS SDK for Java 2.x API Reference*.

## CreateJob

The following code example shows how to use CreateJob.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new AWS Glue job.
 *
 * @param glueClient the AWS Glue client to use for the operation
 * @param jobName the name of the job to create
 * @param iam the IAM role to associate with the job
 * @param scriptLocation the location of the script to be used by the job
 * @throws GlueException if there is an error creating the job
 */
public static void createJob(GlueClient glueClient, String jobName, String iam,
String scriptLocation) {
 try {
 JobCommand command = JobCommand.builder()
 .pythonVersion("3")
 .name("glueetl")
 .scriptLocation(scriptLocation)
 .build();

 CreateJobRequest jobRequest = CreateJobRequest.builder()
 .description("A Job created by using the AWS SDK for Java V2")
 .glueVersion("2.0")
 .workerType(WorkerType.G_1_X)
 .numberOfWorkers(10)
 .name(jobName)
 .role(iam)
 .command(command)
 .build();

 glueClient.createJob(jobRequest);
 System.out.println(jobName + " was successfully created.");

 } catch (GlueException e) {
 throw e;
 }
}
```

```
}
}
```

- For API details, see [CreateJob](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCrawler

The following code example shows how to use DeleteCrawler.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a specific AWS Glue crawler.
 *
 * @param glueClient the AWS Glue client object
 * @param crawlerName the name of the crawler to be deleted
 * @throws GlueException if an error occurs during the deletion process
 */
public static void deleteSpecificCrawler(GlueClient glueClient, String
crawlerName) {
 try {
 DeleteCrawlerRequest deleteCrawlerRequest =
DeleteCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 glueClient.deleteCrawler(deleteCrawlerRequest);
 System.out.println(crawlerName + " was deleted");

 } catch (GlueException e) {
 throw e;
 }
}
```

- For API details, see [DeleteCrawler](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDatabase

The following code example shows how to use DeleteDatabase.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a AWS Glue Database.
 *
 * @param glueClient An instance of the AWS Glue client used to interact with
the AWS Glue service.
 * @param databaseName The name of the database to be deleted.
 * @throws GlueException If an error occurs while deleting the database.
 */
public static void deleteDatabase(GlueClient glueClient, String databaseName) {
 try {
 DeleteDatabaseRequest request = DeleteDatabaseRequest.builder()
 .name(databaseName)
 .build();

 glueClient.deleteDatabase(request);
 System.out.println(databaseName + " was successfully deleted");

 } catch (GlueException e) {
 throw e;
 }
}
```

- For API details, see [DeleteDatabase](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteJob

The following code example shows how to use DeleteJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a Glue job.
 *
 * @param glueClient the Glue client to use for the operation
 * @param jobName the name of the job to be deleted
 * @throws GlueException if there is an error deleting the job
 */
public static void deleteJob(GlueClient glueClient, String jobName) {
 try {
 DeleteJobRequest jobRequest = DeleteJobRequest.builder()
 .jobName(jobName)
 .build();

 glueClient.deleteJob(jobRequest);
 System.out.println(jobName + " was successfully deleted");

 } catch (GlueException e) {
 throw e;
 }
}
```

- For API details, see [DeleteJob](#) in *AWS SDK for Java 2.x API Reference*.

## GetCrawler

The following code example shows how to use GetCrawler.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves a specific crawler from the AWS Glue service and waits for it to be
 * in the "READY" state.
 *
 * @param glueClient the AWS Glue client used to interact with the Glue service
 * @param crawlerName the name of the crawler to be retrieved
 */
public static void getSpecificCrawler(GlueClient glueClient, String crawlerName)
throws InterruptedException {
 try {
 GetCrawlerRequest crawlerRequest = GetCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 boolean ready = false;
 while (!ready) {
 GetCrawlerResponse response = glueClient.getCrawler(crawlerRequest);
 String status = response.crawler().stateAsString();
 if (status.compareTo("READY") == 0) {
 ready = true;
 }
 Thread.sleep(3000);
 }

 System.out.println("The crawler is now ready");

 } catch (GlueException | InterruptedException e) {
 throw e;
 }
}
```

- For API details, see [GetCrawler](#) in *AWS SDK for Java 2.x API Reference*.

## GetDatabase

The following code example shows how to use GetDatabase.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the specific database from the AWS Glue service.
 *
 * @param glueClient an instance of the AWS Glue client used to interact with
the service
 * @param databaseName the name of the database to retrieve
 * @throws GlueException if there is an error retrieving the database from the
AWS Glue service
 */
public static void getSpecificDatabase(GlueClient glueClient, String
databaseName) {
 try {
 GetDatabaseRequest databasesRequest = GetDatabaseRequest.builder()
 .name(databaseName)
 .build();

 GetDatabaseResponse response = glueClient.getDatabase(databasesRequest);
 Instant createDate = response.database().createTime();

 // Convert the Instant to readable date.
 DateTimeFormatter formatter =
 DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT)
 .withLocale(Locale.US)
 .withZone(ZoneId.systemDefault());

 formatter.format(createDate);
 System.out.println("The create date of the database is " + createDate);

 } catch (GlueException e) {
 throw e;
 }
}
```

```
}
```

- For API details, see [GetDatabase](#) in *AWS SDK for Java 2.x API Reference*.

## GetJobRuns

The following code example shows how to use GetJobRuns.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the job runs for a given Glue job and prints the status of the job
 * runs.
 *
 * @param glueClient the Glue client used to make API calls
 * @param jobName the name of the Glue job to retrieve the job runs for
 */
public static void getJobRuns(GlueClient glueClient, String jobName) {
 try {
 GetJobRunsRequest runsRequest = GetJobRunsRequest.builder()
 .jobName(jobName)
 .maxResults(20)
 .build();

 boolean jobDone = false;
 while (!jobDone) {
 GetJobRunsResponse response = glueClient.getJobRuns(runsRequest);
 List<JobRun> jobRuns = response.jobRuns();
 for (JobRun jobRun : jobRuns) {
 String jobState = jobRun.jobRunState().name();
 if (jobState.compareTo("SUCCEEDED") == 0) {
 System.out.println(jobName + " has succeeded");
 jobDone = true;
 } else if (jobState.compareTo("STOPPED") == 0) {
```

```

 System.out.println("Job run has stopped");
 jobDone = true;

 } else if (jobState.compareTo("FAILED") == 0) {
 System.out.println("Job run has failed");
 jobDone = true;

 } else if (jobState.compareTo("TIMEOUT") == 0) {
 System.out.println("Job run has timed out");
 jobDone = true;

 } else {
 System.out.println("*** Job run state is " +
jobRun.jobRunState().name());
 System.out.println("Job run Id is " + jobRun.id());
 System.out.println("The Glue version is " +
jobRun.glueVersion());
 }
 TimeUnit.SECONDS.sleep(5);
}

} catch (GlueException e) {
 throw e;
} catch (InterruptedException e) {
 throw new RuntimeException(e);
}
}

```

- For API details, see [GetJobRuns](#) in *AWS SDK for Java 2.x API Reference*.

## GetTables

The following code example shows how to use GetTables.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the names of the tables in the specified Glue database.
 *
 * @param glueClient the Glue client to use for the operation
 * @param dbName the name of the Glue database to retrieve the table names
from
 * @return the name of the first table retrieved, or an empty string if no
tables were found
 */
public static String getGlueTables(GlueClient glueClient, String dbName) {
 String myTableName = "";
 try {
 GetTablesRequest tableRequest = GetTablesRequest.builder()
 .databaseName(dbName)
 .build();

 GetTablesResponse response = glueClient.getTables(tableRequest);
 List<Table> tables = response.tableList();
 if (tables.isEmpty()) {
 System.out.println("No tables were returned");
 } else {
 for (Table table : tables) {
 myTableName = table.name();
 System.out.println("Table name is: " + myTableName);
 }
 }

 } catch (GlueException e) {
 throw e;
 }
 return myTableName;
}

```

- For API details, see [GetTables](#) in *AWS SDK for Java 2.x API Reference*.

## StartCrawler

The following code example shows how to use StartCrawler.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Starts a specific AWS Glue crawler.
 *
 * @param glueClient the AWS Glue client to use for the crawler operation
 * @param crawlerName the name of the crawler to start
 * @throws GlueException if there is an error starting the crawler
 */
public static void startSpecificCrawler(GlueClient glueClient, String
crawlerName) {
 try {
 StartCrawlerRequest crawlerRequest = StartCrawlerRequest.builder()
 .name(crawlerName)
 .build();

 glueClient.startCrawler(crawlerRequest);
 System.out.println(crawlerName + " was successfully started!");

 } catch (GlueException e) {
 throw e;
 }
}
```

- For API details, see [StartCrawler](#) in *AWS SDK for Java 2.x API Reference*.

## StartJobRun

The following code example shows how to use StartJobRun.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Starts a job run in AWS Glue.
 *
 * @param glueClient the AWS Glue client to use for the job run
 * @param jobName the name of the Glue job to run
 * @param inputDatabase the name of the input database
 * @param inputTable the name of the input table
 * @param outBucket the URL of the output S3 bucket
 * @throws GlueException if there is an error starting the job run
 */
public static void startJob(GlueClient glueClient, String jobName, String
inputDatabase, String inputTable,
 String outBucket) {
 try {
 Map<String, String> myMap = new HashMap<>();
 myMap.put("--input_database", inputDatabase);
 myMap.put("--input_table", inputTable);
 myMap.put("--output_bucket_url", outBucket);

 StartJobRunRequest runRequest = StartJobRunRequest.builder()
 .workerType(WorkerType.G_1_X)
 .numberOfWorkers(10)
 .arguments(myMap)
 .jobName(jobName)
 .build();

 StartJobRunResponse response = glueClient.startJobRun(runRequest);
 System.out.println("The request Id of the job is " +
response.responseMetadata().requestId());

 } catch (GlueException e) {
 throw e;
 }
}
```

```
}
```

- For API details, see [StartJobRun](#) in *AWS SDK for Java 2.x API Reference*.

## HealthImaging examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with HealthImaging.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### CopyImageSet

The following code example shows how to use CopyImageSet.

#### SDK for Java 2.x

```
/**
 * Copy an AWS HealthImaging image set.
 *
 * @param medicalImagingClient - The AWS HealthImaging client object.
 * @param datastoreId - The datastore ID.
 * @param imageSetId - The image set ID.
 * @param latestVersionId - The version ID.
```



```

 * @param destinationImageSetId - The optional destination image set ID, ignored
 if null.
 * @param destinationVersionId - The optional destination version ID, ignored
 if null.
 * @param force - The force flag.
 * @param subsets - The optional subsets to copy, ignored if null.
 * @return - The image set ID of the copy.
 * @throws MedicalImagingException - Base exception for all service exceptions
 thrown by AWS HealthImaging.
 */
 public static String copyMedicalImageSet(MedicalImagingClient
 medicalImagingClient,

 String datastoreId,
 String imageSetId,
 String latestVersionId,
 String destinationImageSetId,
 String destinationVersionId,
 boolean force,
 Vector<String> subsets) {

 try {
 CopySourceImageSetInformation.Builder copySourceImageSetInformation =
 CopySourceImageSetInformation.builder()
 .latestVersionId(latestVersionId);

 // Optionally copy a subset of image instances.
 if (subsets != null) {
 String subsetInstanceToCopy = getCopiableAttributesJSON(imageSetId,
 subsets);
 copySourceImageSetInformation.dicomCopies(MetadataCopies.builder()
 .copiableAttributes(subsetInstanceToCopy)
 .build());
 }

 CopyImageSetInformation.Builder copyImageSetBuilder =
 CopyImageSetInformation.builder()
 .sourceImageSet(copySourceImageSetInformation.build());

 // Optionally designate a destination image set.
 if (destinationImageSetId != null) {
 copyImageSetBuilder =
 copyImageSetBuilder.destinationImageSet(CopyDestinationImageSet.builder()
 .imageSetId(destinationImageSetId)
 .latestVersionId(destinationVersionId)

```

```

 .build());
 }

 CopyImageSetRequest copyImageSetRequest = CopyImageSetRequest.builder()
 .datastoreId(datastoreId)
 .sourceImageSetId(imageSetId)
 .copyImageSetInformation(copyImageSetBuilder.build())
 .force(force)
 .build();

 CopyImageSetResponse response =
medicalImagingClient.copyImageSet(copyImageSetRequest);

 return response.destinationImageSetProperties().imageSetId();
} catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 throw e;
}
}

```

Utility function to create copiable attributes.

```

/**
 * Create a JSON string of copiable image instances.
 *
 * @param imageSetId - The image set ID.
 * @param subsets - The subsets to copy.
 * @return A JSON string of copiable image instances.
 */
private static String getCopiableAttributesJSON(String imageSetId,
Vector<String> subsets) {
 StringBuilder subsetInstanceToCopy = new StringBuilder(
 ""
 {
 "SchemaVersion": 1.1,
 "Study": {
 "Series": {
 ""
 }
 }
 }
);
}

```

```

subsetInstanceToCopy.append(imageSetId);

subsetInstanceToCopy.append(
 """
 ": {
 "Instances": {
 """
);

for (String subset : subsets) {
 subsetInstanceToCopy.append("'" + subset + "\"": {},");
}
subsetInstanceToCopy.deleteCharAt(subsetInstanceToCopy.length() - 1);
subsetInstanceToCopy.append("""
 }
 }
 }
 }
 """);
return subsetInstanceToCopy.toString();
}

```

- For API details, see [CopyImageSet](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## CreateDatastore

The following code example shows how to use CreateDatastore.

### SDK for Java 2.x

```

public static String createMedicalImageDatastore(MedicalImagingClient
medicalImagingClient,
 String datastoreName) {
 try {

```

```
 CreateDatastoreRequest datastoreRequest =
CreateDatastoreRequest.builder()
 .datastoreName(datastoreName)
 .build();
 CreateDatastoreResponse response =
medicalImagingClient.createDatastore(datastoreRequest);
 return response.datastoreId();
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return "";
}
```

- For API details, see [CreateDatastore](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## DeleteDatastore

The following code example shows how to use DeleteDatastore.

### SDK for Java 2.x

```
public static void deleteMedicalImagingDatastore(MedicalImagingClient
medicalImagingClient,
 String datastoreID) {
 try {
 DeleteDatastoreRequest datastoreRequest =
DeleteDatastoreRequest.builder()
 .datastoreId(datastoreID)
 .build();
 medicalImagingClient.deleteDatastore(datastoreRequest);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

```
 }
}
```

- For API details, see [DeleteDatastore](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## DeleteImageSet

The following code example shows how to use DeleteImageSet.

### SDK for Java 2.x

```
public static void deleteMedicalImageSet(MedicalImagingClient
medicalImagingClient,
 String datastoreId,
 String imagesetId) {
 try {
 DeleteImageSetRequest deleteImageSetRequest =
DeleteImageSetRequest.builder()
 .datastoreId(datastoreId)
 .imageSetId(imagesetId)
 .build();

 medicalImagingClient.deleteImageSet(deleteImageSetRequest);

 System.out.println("The image set was deleted.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteImageSet](#) in *AWS SDK for Java 2.x API Reference*.

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## GetDICOMImportJob

The following code example shows how to use `GetDICOMImportJob`.

### SDK for Java 2.x

```
public static DICOMImportJobProperties getDicomImportJob(MedicalImagingClient
medicalImagingClient,
 String datastoreId,
 String jobId) {

 try {
 GetDicomImportJobRequest getDicomImportJobRequest =
 GetDicomImportJobRequest.builder()
 .datastoreId(datastoreId)
 .jobId(jobId)
 .build();
 GetDicomImportJobResponse response =
 medicalImagingClient.getDICOMImportJob(getDicomImportJobRequest);
 return response.jobProperties();
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
```

- For API details, see [GetDICOMImportJob](#) in *AWS SDK for Java 2.x API Reference*.

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## GetDatastore

The following code example shows how to use GetDatastore.

### SDK for Java 2.x

```
public static DatastoreProperties getMedicalImageDatastore(MedicalImagingClient
medicalImagingClient,
 String datastoreID) {
 try {
 GetDatastoreRequest datastoreRequest = GetDatastoreRequest.builder()
 .datastoreId(datastoreID)
 .build();
 GetDatastoreResponse response =
medicalImagingClient.getDatastore(datastoreRequest);
 return response.datastoreProperties();
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
```

- For API details, see [GetDatastore](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## GetImageFrame

The following code example shows how to use GetImageFrame.

### SDK for Java 2.x

```
public static void getMedicalImageSetFrame(MedicalImagingClient
medicalImagingClient,
 String destinationPath,
```

```
 String datastoreId,
 String imagesetId,
 String imageFrameId) {

 try {
 GetImageFrameRequest getImageSetMetadataRequest =
 GetImageFrameRequest.builder()
 .datastoreId(datastoreId)
 .imageSetId(imagesetId)

 .imageFrameInformation(ImageFrameInformation.builder()
 .imageFrameId(imageFrameId)
 .build())
 .build();

 medicalImagingClient.getImageFrame(getImageSetMetadataRequest,
 FileSystems.getDefault().getPath(destinationPath));

 System.out.println("Image frame downloaded to " +
 destinationPath);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetImageFrame](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## GetImageSet

The following code example shows how to use `GetImageSet`.



## SDK for Java 2.x

```
public static GetImageSetResponse getMedicalImageSet(MedicalImagingClient
medicalImagingClient,
 String datastoreId,
 String imagesetId,
 String versionId) {
 try {
 GetImageSetRequest.Builder getImageSetRequestBuilder =
 GetImageSetRequest.builder()
 .datastoreId(datastoreId)
 .imageSetId(imagesetId);

 if (versionId != null) {
 getImageSetRequestBuilder =
 getImageSetRequestBuilder.versionId(versionId);
 }

 return
 medicalImagingClient.getImageSet(getImageSetRequestBuilder.build());
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
```

- For API details, see [GetImageSet](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## GetImageSetMetadata

The following code example shows how to use `GetImageSetMetadata`.

## SDK for Java 2.x

```
public static void getMedicalImageSetMetadata(MedicalImagingClient
medicalImagingClient,
 String destinationPath,
 String datastoreId,
 String imagesetId,
 String versionId) {

 try {
 GetImageSetMetadataRequest.Builder getImageSetMetadataRequestBuilder =
 GetImageSetMetadataRequest.builder()
 .datastoreId(datastoreId)
 .imageSetId(imagesetId);

 if (versionId != null) {
 getImageSetMetadataRequestBuilder =
 getImageSetMetadataRequestBuilder.versionId(versionId);
 }

 medicalImagingClient.getImageSetMetadata(getImageSetMetadataRequestBuilder.build(),
 FileSystems.getDefault().getPath(destinationPath));

 System.out.println("Metadata downloaded to " + destinationPath);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetImageSetMetadata](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## ListDICOMImportJobs

The following code example shows how to use ListDICOMImportJobs.

### SDK for Java 2.x

```
public static List<DICOMImportJobSummary>
listDicomImportJobs(MedicalImagingClient medicalImagingClient,
 String datastoreId) {

 try {
 ListDicomImportJobsRequest listDicomImportJobsRequest =
 ListDicomImportJobsRequest.builder()
 .datastoreId(datastoreId)
 .build();
 ListDicomImportJobsResponse response =
 medicalImagingClient.listDICOMImportJobs(listDicomImportJobsRequest);
 return response.jobSummaries();
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return new ArrayList<>();
}
```

- For API details, see [ListDICOMImportJobs](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## ListDatastores

The following code example shows how to use ListDatastores.

## SDK for Java 2.x

```
public static List<DatastoreSummary>
listMedicalImagingDatastores(MedicalImagingClient medicalImagingClient) {
 try {
 ListDatastoresRequest datastoreRequest = ListDatastoresRequest.builder()
 .build();
 ListDatastoresIterable responses =
 medicalImagingClient.listDatastoresPaginator(datastoreRequest);
 List<DatastoreSummary> datastoreSummaries = new ArrayList<>();

 responses.stream().forEach(response ->
 datastoreSummaries.addAll(response.datastoreSummaries()));

 return datastoreSummaries;
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
```

- For API details, see [ListDatastores](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## ListImageSetVersions

The following code example shows how to use `ListImageSetVersions`.

## SDK for Java 2.x

```
public static List<ImageSetProperties>
listMedicalImageSetVersions(MedicalImagingClient medicalImagingClient,
 String datastoreId,
 String imagesetId) {
```

```

 try {
 ListImageSetVersionsRequest getImageSetRequest =
ListImageSetVersionsRequest.builder()
 .datastoreId(datastoreId)
 .imageSetId(imagesetId)
 .build();

 ListImageSetVersionsIterable responses = medicalImagingClient
 .listImageSetVersionsPaginator(getImageSetRequest);
 List<ImageSetProperties> imageSetProperties = new ArrayList<>();
 responses.stream().forEach(response ->
imageSetProperties.addAll(response.imageSetPropertiesList()));

 return imageSetProperties;
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

```

- For API details, see [ListImageSetVersions](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## ListTagsForResource

The following code example shows how to use `ListTagsForResource`.

### SDK for Java 2.x

```

public static ListTagsForResourceResponse
listMedicalImagingResourceTags(MedicalImagingClient medicalImagingClient,
 String resourceArn) {
 try {
 ListTagsForResourceRequest listTagsForResourceRequest =
ListTagsForResourceRequest.builder()

```

```

 .resourceArn(resourceArn)
 .build();

 return
medicalImagingClient.listTagsForResource(listTagsForResourceRequest);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

```

- For API details, see [ListTagsForResource](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## SearchImageSets

The following code example shows how to use SearchImageSets.

### SDK for Java 2.x

The utility function for searching image sets.

```

public static List<ImageSetsMetadataSummary> searchMedicalImagingImageSets(
 MedicalImagingClient medicalImagingClient,
 String datastoreId, SearchCriteria searchCriteria) {
 try {
 SearchImageSetsRequest datastoreRequest =
SearchImageSetsRequest.builder()
 .datastoreId(datastoreId)
 .searchCriteria(searchCriteria)
 .build();
 SearchImageSetsIterable responses = medicalImagingClient
 .searchImageSetsPaginator(datastoreRequest);
 List<ImageSetsMetadataSummary> imageSetsMetadataSummaries = new
ArrayList<>();
 }
}

```

```

 responses.stream().forEach(response -> imageSetsMetadataSummaries
 .addAll(response.imageSetsMetadataSummaries()));

 return imageSetsMetadataSummaries;
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

```

### Use case #1: EQUAL operator.

```

List<SearchFilter> searchFilters =
Collections.singletonList(SearchFilter.builder()
 .operator(Operator.EQUAL)
 .values(SearchByAttributeValue.builder()
 .dicomPatientId(patientId)
 .build())
 .build());

SearchCriteria searchCriteria = SearchCriteria.builder()
 .filters(searchFilters)
 .build();

List<ImageSetsMetadataSummary> imageSetsMetadataSummaries =
searchMedicalImagingImageSets(
 medicalImagingClient,
 datastoreId, searchCriteria);
if (imageSetsMetadataSummaries != null) {
 System.out.println("The image sets for patient " + patientId + " are:\n"
 + imageSetsMetadataSummaries);
 System.out.println();
}

```

### Use case #2: BETWEEN operator using DICOMStudyDate and DICOMStudyTime.

```

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyyMMdd");
searchFilters = Collections.singletonList(SearchFilter.builder()

```

```

 .operator(Operator.BETWEEN)
 .values(SearchByAttributeValue.builder()

 .dicomStudyDateAndTime(DICOMStudyDateAndTime.builder()
 .dicomStudyDate("19990101")
 .dicomStudyTime("000000.000")
 .build())
 .build(),
 SearchByAttributeValue.builder()

 .dicomStudyDateAndTime(DICOMStudyDateAndTime.builder()
 .dicomStudyDate((LocalDate.now()
 .format(formatter)))
 .dicomStudyTime("000000.000")
 .build())
 .build())
 .build());

searchCriteria = SearchCriteria.builder()
 .filters(searchFilters)
 .build();

imageSetsMetadataSummaries =
searchMedicalImagingImageSets(medicalImagingClient,
 datastoreId, searchCriteria);
if (imageSetsMetadataSummaries != null) {
 System.out.println(
 "The image sets searched with BETWEEN operator using
DICOMStudyDate and DICOMStudyTime are:\n"
 +
 imageSetsMetadataSummaries);
 System.out.println();
}

```

Use case #3: BETWEEN operator using createdAt. Time studies were previously persisted.

```

searchFilters = Collections.singletonList(SearchFilter.builder()
 .operator(Operator.BETWEEN)
 .values(SearchByAttributeValue.builder()
 .createdAt(Instant.parse("1985-04-12T23:20:50.52Z"))
 .build(),
 SearchByAttributeValue.builder()

```



```

 .createdAt(Instant.now())
 .build())
 .build());

searchCriteria = SearchCriteria.builder()
 .filters(searchFilters)
 .build();
imageSetsMetadataSummaries =
searchMedicalImagingImageSets(medicalImagingClient,
 datastoreId, searchCriteria);
if (imageSetsMetadataSummaries != null) {
 System.out.println("The image sets searched with BETWEEN operator using
createdAt are:\n "
 + imageSetsMetadataSummaries);
 System.out.println();
}

```

Use case #4: EQUAL operator on DICOMSeriesInstanceUID and BETWEEN on updatedAt and sort response in ASC order on updatedAt field.

```

Instant startDate = Instant.parse("1985-04-12T23:20:50.52Z");
Instant endDate = Instant.now();

searchFilters = Arrays.asList(
 SearchFilter.builder()
 .operator(Operator.EQUAL)
 .values(SearchByAttributeValue.builder()
 .dicomSeriesInstanceUID(seriesInstanceUID)
 .build())
 .build(),
 SearchFilter.builder()
 .operator(Operator.BETWEEN)
 .values(
 SearchByAttributeValue.builder().updatedAt(startDate).build(),
 SearchByAttributeValue.builder().updatedAt(endDate).build()
).build());

Sort sort =
Sort.builder().sortOrder(SortOrder.ASC).sortField(SortField.UPDATED_AT).build();

```

```

searchCriteria = SearchCriteria.builder()
 .filters(searchFilters)
 .sort(sort)
 .build();

imageSetsMetadataSummaries =
searchMedicalImagingImageSets(medicalImagingClient,
 datastoreId, searchCriteria);
if (imageSetsMetadataSummaries != null) {
 System.out.println("The image sets searched with EQUAL operator on
DICOMSeriesInstanceUID and BETWEEN on updatedAt and sort response\n" +
 "in ASC order on updatedAt field are:\n "
 + imageSetsMetadataSummaries);
 System.out.println();
}

```

- For API details, see [SearchImageSets](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## StartDICOMImportJob

The following code example shows how to use StartDICOMImportJob.

### SDK for Java 2.x

```

public static String startDicomImportJob(MedicalImagingClient
medicalImagingClient,
 String jobName,
 String datastoreId,
 String dataAccessRoleArn,
 String inputS3Uri,
 String outputS3Uri) {

 try {
 StartDicomImportJobRequest startDicomImportJobRequest =
StartDicomImportJobRequest.builder()

```

```

 .jobName(jobName)
 .datastoreId(datastoreId)
 .dataAccessRoleArn(dataAccessRoleArn)
 .inputS3Uri(inputS3Uri)
 .outputS3Uri(outputS3Uri)
 .build();
 StartDicomImportJobResponse response =
medicalImagingClient.startDICOMImportJob(startDicomImportJobRequest);
 return response.jobId();
} catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

return "";
}

```

- For API details, see [StartDICOMImportJob](#) in *AWS SDK for Java 2.x API Reference*.

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## TagResource

The following code example shows how to use TagResource.

### SDK for Java 2.x

```

public static void tagMedicalImagingResource(MedicalImagingClient
medicalImagingClient,
 String resourceArn,
 Map<String, String> tags) {
 try {
 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tags(tags)
 .build();
 }
}

```

```
 medicalImagingClient.tagResource(tagResourceRequest);

 System.out.println("Tags have been added to the resource.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [TagResource](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## UntagResource

The following code example shows how to use UntagResource.

### SDK for Java 2.x

```
public static void untagMedicalImagingResource(MedicalImagingClient
medicalImagingClient,
 String resourceArn,
 Collection<String> tagKeys) {
 try {
 UntagResourceRequest untagResourceRequest =
UntagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tagKeys(tagKeys)
 .build();

 medicalImagingClient.untagResource(untagResourceRequest);

 System.out.println("Tags have been removed from the resource.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [UntagResource](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## UpdateImageSetMetadata

The following code example shows how to use `UpdateImageSetMetadata`.

### SDK for Java 2.x

```
/**
 * Update the metadata of an AWS HealthImaging image set.
 *
 * @param medicalImagingClient - The AWS HealthImaging client object.
 * @param datastoreId - The datastore ID.
 * @param imageSetId - The image set ID.
 * @param versionId - The version ID.
 * @param metadataUpdates - A MetadataUpdates object containing the
updates.
 * @param force - The force flag.
 * @throws MedicalImagingException - Base exception for all service exceptions
thrown by AWS HealthImaging.
 */
public static void updateMedicalImageSetMetadata(MedicalImagingClient
medicalImagingClient,
 String datastoreId,
 String imageSetId,
 String versionId,
 MetadataUpdates
metadataUpdates,
 boolean force) {
 try {
 UpdateImageSetMetadataRequest updateImageSetMetadataRequest =
UpdateImageSetMetadataRequest
 .builder()
 .datastoreId(datastoreId)
```

```

 .imageSetId(imageSetId)
 .latestVersionId(versionId)
 .updateImageSetMetadataUpdates(metadataUpdates)
 .force(force)
 .build();

 UpdateImageSetMetadataResponse response =
medicalImagingClient.updateImageSetMetadata(updateImageSetMetadataRequest);

 System.out.println("The image set metadata was updated" + response);
} catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 throw e;
}
}

```

### Use case #1: Insert or update an attribute.

```

 final String insertAttributes = ""
 {
 "SchemaVersion": 1.1,
 "Study": {
 "DICOM": {
 "StudyDescription": "CT CHEST"
 }
 }
 }
 """;

 MetadataUpdates metadataInsertUpdates = MetadataUpdates.builder()
 .dicomUpdates(DICOMUpdates.builder()
 .updatableAttributes(SdkBytes.fromByteBuffer(
 ByteBuffer.wrap(insertAttributes
 .getBytes(StandardCharsets.UTF_8))))
 .build())
 .build();

 updateMedicalImageSetMetadata(medicalImagingClient, datastoreId,
imagesetId,
 versionid, metadataInsertUpdates, force);

```

### Use case #2: Remove an attribute.

```

 final String removeAttributes = ""
 {
 "SchemaVersion": 1.1,
 "Study": {
 "DICOM": {
 "StudyDescription": "CT CHEST"
 }
 }
 }
 """;
 MetadataUpdates metadataRemoveUpdates = MetadataUpdates.builder()
 .dicomUpdates(DICOMUpdates.builder()
 .removableAttributes(SdkBytes.fromByteBuffer(
 ByteBuffer.wrap(removeAttributes
 .getBytes(StandardCharsets.UTF_8))))
 .build())
 .build();

 updateMedicalImageSetMetadata(medicalImagingClient, datastoreId,
 imagesetId,
 versionid, metadataRemoveUpdates, force);

```

### Use case #3: Remove an instance.

```

 final String removeInstance = ""
 {
 "SchemaVersion": 1.1,
 "Study": {
 "Series": {
 "1.1.1.1.1.1.12345.123456789012.123.12345678901234.1":
{
 "Instances": {

"1.1.1.1.1.1.12345.123456789012.123.12345678901234.1": {}
 }
 }
 }
 }
 """;
 MetadataUpdates metadataRemoveUpdates = MetadataUpdates.builder()
 .dicomUpdates(DICOMUpdates.builder()

```

```

 .removableAttributes(SdkBytes.fromByteBuffer(
 ByteBuffer.wrap(removeInstance
 .getBytes(StandardCharsets.UTF_8))))
 .build())
 .build();

 updateMedicalImageSetMetadata(medicalImagingClient, datastoreId,
imagesetId,
 versionid, metadataRemoveUpdates, force);

```

Use case #4: Revert to a previous version.

```

 // In this case, revert to previous version.
 String revertVersionId =
Integer.toString(Integer.parseInt(versionid) - 1);
 MetadataUpdates metadataRemoveUpdates = MetadataUpdates.builder()
 .revertToVersionId(revertVersionId)
 .build();
 updateMedicalImageSetMetadata(medicalImagingClient, datastoreId,
imagesetId,
 versionid, metadataRemoveUpdates, force);

```

- For API details, see [UpdateImageSetMetadata](#) in *AWS SDK for Java 2.x API Reference*.

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Scenarios

### Tagging a data store

The following code example shows how to tag a HealthImaging data store.

#### SDK for Java 2.x

To tag a data store.



```

 final String datastoreArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012";

 TagResource.tagMedicalImagingResource(medicalImagingClient,
 datastoreArn,

 ImmutableMap.of("Deployment", "Development"));

```

The utility function for tagging a resource.

```

 public static void tagMedicalImagingResource(MedicalImagingClient
medicalImagingClient,
 String resourceArn,
 Map<String, String> tags) {
 try {
 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tags(tags)
 .build();

 medicalImagingClient.tagResource(tagResourceRequest);

 System.out.println("Tags have been added to the resource.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

To list tags for a data store.

```

 final String datastoreArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012";

 ListTagsForResourceResponse result =
ListTagsForResource.listMedicalImagingResourceTags(
 medicalImagingClient,
 datastoreArn);
 if (result != null) {
 System.out.println("Tags for resource: " + result.tags());
 }
 }
}

```

The utility function for listing a resource's tags.

```
public static ListTagsForResourceResponse
listMedicalImagingResourceTags(MedicalImagingClient medicalImagingClient,
 String resourceArn) {
 try {
 ListTagsForResourceRequest listTagsForResourceRequest =
 ListTagsForResourceRequest.builder()
 .resourceArn(resourceArn)
 .build();

 return
 medicalImagingClient.listTagsForResource(listTagsForResourceRequest);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
```

To untag a data store.

```
final String datastoreArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012";

UntagResource.untagMedicalImagingResource(medicalImagingClient,
 datastoreArn,
 Collections.singletonList("Deployment"));
```

The utility function for untagging a resource.

```
public static void untagMedicalImagingResource(MedicalImagingClient
 medicalImagingClient,
 String resourceArn,
 Collection<String> tagKeys) {
 try {
```

```
UntagResourceRequest untagResourceRequest =
UntagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tagKeys(tagKeys)
 .build();

medicalImagingClient.untagResource(untagResourceRequest);

System.out.println("Tags have been removed from the resource.");
} catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [ListTagsForResource](#)
  - [TagResource](#)
  - [UntagResource](#)

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Tagging an image set

The following code example shows how to tag a HealthImaging image set.

### SDK for Java 2.x

To tag an image set.

```
final String imageSetArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012/
imageset/12345678901234567890123456789012";

TagResource.tagMedicalImagingResource(medicalImagingClient,
 imageSetArn,
```

```
ImmutableMap.of("Deployment", "Development"));
```

The utility function for tagging a resource.

```
public static void tagMedicalImagingResource(MedicalImagingClient
medicalImagingClient,
 String resourceArn,
 Map<String, String> tags) {
 try {
 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tags(tags)
 .build();

 medicalImagingClient.tagResource(tagResourceRequest);

 System.out.println("Tags have been added to the resource.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

To list tags for an image set.

```
final String imageSetArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012/
imageset/12345678901234567890123456789012";

ListTagsForResourceResponse result =
ListTagsForResource.listMedicalImagingResourceTags(
 medicalImagingClient,
 imageSetArn);
if (result != null) {
 System.out.println("Tags for resource: " + result.tags());
}
```

The utility function for listing a resource's tags.

```

 public static ListTagsForResourceResponse
listMedicalImagingResourceTags(MedicalImagingClient medicalImagingClient,
 String resourceArn) {
 try {
 ListTagsForResourceRequest listTagsForResourceRequest =
ListTagsForResourceRequest.builder()
 .resourceArn(resourceArn)
 .build();

 return
medicalImagingClient.listTagsForResource(listTagsForResourceRequest);
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}

```

To untag an image set.

```

 final String imageSetArn = "arn:aws:medical-imaging:us-
east-1:123456789012:datastore/12345678901234567890123456789012/
imageset/12345678901234567890123456789012";

 UntagResource.untagMedicalImagingResource(medicalImagingClient,
imageSetArn,
 Collections.singletonList("Deployment"));

```

The utility function for untagging a resource.

```

 public static void untagMedicalImagingResource(MedicalImagingClient
medicalImagingClient,
 String resourceArn,
 Collection<String> tagKeys) {
 try {
 UntagResourceRequest untagResourceRequest =
UntagResourceRequest.builder()
 .resourceArn(resourceArn)
 .tagKeys(tagKeys)

```

```
 .build();

 medicalImagingClient.untagResource(untagResourceRequest);

 System.out.println("Tags have been removed from the resource.");
 } catch (MedicalImagingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [ListTagsForResource](#)
  - [TagResource](#)
  - [UntagResource](#)

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## IAM examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with IAM.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello IAM

The following code example shows how to get started using IAM.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.ListPoliciesResponse;
import software.amazon.awssdk.services.iam.model.Policy;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloIAM {
 public static void main(String[] args) {
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();
```

```
 listPolicies(iam);
 }

 public static void listPolicies(IamClient iam) {
 ListPoliciesResponse response = iam.listPolicies();
 List<Policy> polList = response.policies();
 polList.forEach(policy -> {
 System.out.println("Policy Name: " + policy.policyName());
 });
 }
}
```

- For API details, see [ListPolicies](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to create a user and assume a role.

#### Warning

To avoid security risks, don't use IAM users for authentication when developing purpose-built software or working with real data. Instead, use federation with an identity provider such as [AWS IAM Identity Center](#).

- Create a user with no permissions.
- Create a role that grants permission to list Amazon S3 buckets for the account.
- Add a policy to let the user assume the role.
- Assume the role and list S3 buckets using temporary credentials, then clean up resources.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create functions that wrap IAM user actions.

```
/*
 To run this Java V2 code example, set up your development environment, including
 your credentials.

 For information, see this documentation topic:

 https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

 This example performs these operations:

 1. Creates a user that has no permissions.
 2. Creates a role and policy that grants Amazon S3 permissions.
 3. Creates a role.
 4. Grants the user permissions.
 5. Gets temporary credentials by assuming the role. Creates an Amazon S3 Service
 client object with the temporary credentials.
 6. Deletes the resources.
*/

public class IAMScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 public static final String PolicyDocument = "{" +
 " \"Version\": \"2012-10-17\", \" +
 " \"Statement\": [\" +
 " {\" +
 " \"Effect\": \"Allow\", \" +
 " \"Action\": [\" +
 " \"s3:*\" \" +
 "], \" +
 " \"Resource\": \"*\" \" +
 " }\" +
 "]\" +
 "}";
```

```

public static String userArn;

public static void main(String[] args) throws Exception {

 final String usage = ""

 Usage:
 <username> <policyName> <roleName> <roleSessionName>
<bucketName>\s

 Where:
 username - The name of the IAM user to create.\s
 policyName - The name of the policy to create.\s
 roleName - The name of the role to create.\s
 roleSessionName - The name of the session required for the
assumeRole operation.\s
 bucketName - The name of the Amazon S3 bucket from which objects
are read.\s

 """;

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String userName = args[0];
 String policyName = args[1];
 String roleName = args[2];
 String roleSessionName = args[3];
 String bucketName = args[4];

 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the AWS IAM example scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(" 1. Create the IAM user.");
 User createUser = createIAMUser(iam, userName);

```

```
System.out.println(DASHES);
userArn = createUser.arn();

AccessKey myKey = createIAMAccessKey(iam, userName);
String accessKey = myKey.accessKeyId();
String secretKey = myKey.secretAccessKey();
String assumeRolePolicyDocument = "{" +
 "\"Version\": \"2012-10-17\"," +
 "\"Statement\": [{" +
 "\"Effect\": \"Allow\"," +
 "\"Principal\": {" +
 " \"AWS\": \"" + userArn + "\"" +
 "}," +
 "\"Action\": \"sts:AssumeRole\"" +
 "}]}" +
 "}";

System.out.println(assumeRolePolicyDocument);
System.out.println(userName + " was successfully created.");
System.out.println(DASHES);
System.out.println("2. Creates a policy.");
String polArn = createIAMPolicy(iam, policyName);
System.out.println("The policy " + polArn + " was successfully created.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Creates a role.");
TimeUnit.SECONDS.sleep(30);
String roleArn = createIAMRole(iam, roleName, assumeRolePolicyDocument);
System.out.println(roleArn + " was successfully created.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Grants the user permissions.");
attachIAMRolePolicy(iam, roleName, polArn);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("**** Wait for 30 secs so the resource is available");
TimeUnit.SECONDS.sleep(30);
System.out.println("5. Gets temporary credentials by assuming the role.");
System.out.println("Perform an Amazon S3 Service operation using the
temporary credentials.");
```

```

 assumeRole(roleArn, roleSessionName, bucketName, accessKey, secretKey);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6 Getting ready to delete the AWS resources");
 deleteKey(iam, userName, accessKey);
 deleteRole(iam, roleName, polArn);
 deleteIAMUser(iam, userName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("This IAM Scenario has successfully completed");
 System.out.println(DASHES);
 }

 public static AccessKey createIAMAccessKey(IamClient iam, String user) {
 try {
 CreateAccessKeyRequest request = CreateAccessKeyRequest.builder()
 .userName(user)
 .build();

 CreateAccessKeyResponse response = iam.createAccessKey(request);
 return response.accessKey();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
 }

 public static User createIAMUser(IamClient iam, String username) {
 try {
 // Create an IamWaiter object
 IamWaiter iamWaiter = iam.waiter();
 CreateUserRequest request = CreateUserRequest.builder()
 .userName(username)
 .build();

 // Wait until the user is created.
 CreateUserResponse response = iam.createUser(request);
 GetUserRequest userRequest = GetUserRequest.builder()
 .userName(response.user().userName())
 .build();

```

```

 WaiterResponse<GetUserResponse> waitUntilUserExists =
iamWaiter.waitUntilUserExists(userRequest);
 waitUntilUserExists.matched().response().ifPresent(System.out::println);
 return response.user();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

public static String createIAMRole(IamClient iam, String rolename, String json)
{

 try {
 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(rolename)
 .assumeRolePolicyDocument(json)
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse response = iam.createRole(request);
 System.out.println("The ARN of the role is " + response.role().arn());
 return response.role().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createIAMPolicy(IamClient iam, String policyName) {
 try {
 // Create an IamWaiter object.
 IamWaiter iamWaiter = iam.waiter();
 CreatePolicyRequest request = CreatePolicyRequest.builder()
 .policyName(policyName)
 .policyDocument(PolicyDocument).build();

 CreatePolicyResponse response = iam.createPolicy(request);
 GetPolicyRequest polRequest = GetPolicyRequest.builder()

```

```

 .policyArn(response.policy().arn())
 .build();

 WaiterResponse<GetPolicyResponse> waitUntilPolicyExists =
iamWaiter.waitUntilPolicyExists(polRequest);

waitUntilPolicyExists.matched().response().ifPresent(System.out::println);
 return response.policy().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static void attachIAMRolePolicy(IamClient iam, String roleName, String
policyArn) {
 try {
 ListAttachedRolePoliciesRequest request =
ListAttachedRolePoliciesRequest.builder()
 .roleName(roleName)
 .build();

 ListAttachedRolePoliciesResponse response =
iam.listAttachedRolePolicies(request);
 List<AttachedPolicy> attachedPolicies = response.attachedPolicies();
 String polArn;
 for (AttachedPolicy policy : attachedPolicies) {
 polArn = policy.policyArn();
 if (polArn.compareTo(policyArn) == 0) {
 System.out.println(roleName + " policy is already attached to
this role.");
 return;
 }
 }

 AttachRolePolicyRequest attachRequest =
AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();

 iam.attachRolePolicy(attachRequest);
 }
}

```

```

 System.out.println("Successfully attached policy " + policyArn + " to
role " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Invoke an Amazon S3 operation using the Assumed Role.
public static void assumeRole(String roleArn, String roleSessionName, String
bucketName, String keyVal,
 String keySecret) {

 // Use the creds of the new IAM user that was created in this code example.
 AwsBasicCredentials credentials = AwsBasicCredentials.create(keyVal,
keySecret);

 StsClient stsClient = StsClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(StaticCredentialsProvider.create(credentials))
 .build();

 try {
 AssumeRoleRequest roleRequest = AssumeRoleRequest.builder()
 .roleArn(roleArn)
 .roleSessionName(roleSessionName)
 .build();

 AssumeRoleResponse roleResponse = stsClient.assumeRole(roleRequest);
 Credentials myCreds = roleResponse.credentials();
 String key = myCreds.accessKeyId();
 String secKey = myCreds.secretAccessKey();
 String secToken = myCreds.sessionToken();

 // List all objects in an Amazon S3 bucket using the temp creds
retrieved by
 // invoking assumeRole.
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .credentialsProvider(
StaticCredentialsProvider.create(AwsSessionCredentials.create(key, secKey,
secToken)))
 .region(region)

```

```

 .build();

 System.out.println("Created a S3Client using temp credentials.");
 System.out.println("Listing objects in " + bucketName);
 ListObjectsRequest listObjects = ListObjectsRequest.builder()
 .bucket(bucketName)
 .build();

 ListObjectsResponse res = s3.listObjects(listObjects);
 List<S3Object> objects = res.contents();
 for (S3Object myValue : objects) {
 System.out.println("The name of the key is " + myValue.key());
 System.out.println("The owner is " + myValue.owner());
 }

 } catch (StsException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void deleteRole(IamClient iam, String roleName, String polArn) {

 try {
 // First the policy needs to be detached.
 DetachRolePolicyRequest rolePolicyRequest =
 DetachRolePolicyRequest.builder()
 .policyArn(polArn)
 .roleName(roleName)
 .build();

 iam.detachRolePolicy(rolePolicyRequest);

 // Delete the policy.
 DeletePolicyRequest request = DeletePolicyRequest.builder()
 .policyArn(polArn)
 .build();

 iam.deletePolicy(request);
 System.out.println("*** Successfully deleted " + polArn);

 // Delete the role.
 DeleteRoleRequest roleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)

```



```
 .build();

 iam.deleteRole(roleRequest);
 System.out.println("*** Successfully deleted " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteKey(IamClient iam, String username, String accessKey) {
 try {
 DeleteAccessKeyRequest request = DeleteAccessKeyRequest.builder()
 .accessKeyId(accessKey)
 .userName(username)
 .build();

 iam.deleteAccessKey(request);
 System.out.println("Successfully deleted access key " + accessKey +
 " from user " + username);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteIAMUser(IamClient iam, String userName) {
 try {
 DeleteUserRequest request = DeleteUserRequest.builder()
 .userName(userName)
 .build();

 iam.deleteUser(request);
 System.out.println("*** Successfully deleted " + userName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AttachRolePolicy](#)
  - [CreateAccessKey](#)
  - [CreatePolicy](#)
  - [CreateRole](#)
  - [CreateUser](#)
  - [DeleteAccessKey](#)
  - [DeletePolicy](#)
  - [DeleteRole](#)
  - [DeleteUser](#)
  - [DeleteUserPolicy](#)
  - [DetachRolePolicy](#)
  - [PutUserPolicy](#)

## Actions

### AttachRolePolicy

The following code example shows how to use `AttachRolePolicy`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.AttachRolePolicyRequest;
import software.amazon.awssdk.services.iam.model.AttachedPolicy;
import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesRequest;
```

```

import software.amazon.awssdk.services.iam.model.ListAttachedRolePoliciesResponse;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class AttachRolePolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <roleName> <policyArn>\s

 Where:
 roleName - A role name that you can obtain from the AWS
Management Console.\s
 policyArn - A policy ARN that you can obtain from the AWS
Management Console.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String roleName = args[0];
 String policyArn = args[1];

 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 attachIAMRolePolicy(iam, roleName, policyArn);
 iam.close();
 }

 public static void attachIAMRolePolicy(IamClient iam, String roleName, String
policyArn) {

```

```

 try {
 ListAttachedRolePoliciesRequest request =
ListAttachedRolePoliciesRequest.builder()
 .roleName(roleName)
 .build();

 ListAttachedRolePoliciesResponse response =
iam.listAttachedRolePolicies(request);
 List<AttachedPolicy> attachedPolicies = response.attachedPolicies();

 // Ensure that the policy is not attached to this role
 String polArn = "";
 for (AttachedPolicy policy : attachedPolicies) {
 polArn = policy.policyArn();
 if (polArn.compareTo(policyArn) == 0) {
 System.out.println(roleName + " policy is already attached to
this role.");
 return;
 }
 }

 AttachRolePolicyRequest attachRequest =
AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();

 iam.attachRolePolicy(attachRequest);

 System.out.println("Successfully attached policy " + policyArn +
 " to role " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
 }
}

```

- For API details, see [AttachRolePolicy](#) in *AWS SDK for Java 2.x API Reference*.

## CreateAccessKey

The following code example shows how to use CreateAccessKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.iam.model.CreateAccessKeyRequest;
import software.amazon.awssdk.services.iam.model.CreateAccessKeyResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateAccessKey {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <user>\s

 Where:
 user - An AWS IAM user that you can obtain from the AWS
Management Console.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```
String user = args[0];
Region region = Region.AWS_GLOBAL;
IamClient iam = IamClient.builder()
 .region(region)
 .build();

String keyId = createIAMAccessKey(iam, user);
System.out.println("The Key Id is " + keyId);
iam.close();
}

public static String createIAMAccessKey(IamClient iam, String user) {
 try {
 CreateAccessKeyRequest request = CreateAccessKeyRequest.builder()
 .userName(user)
 .build();

 CreateAccessKeyResponse response = iam.createAccessKey(request);
 return response.accessKey().accessKeyId();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateAccessKey](#) in *AWS SDK for Java 2.x API Reference*.

## CreateAccountAlias

The following code example shows how to use `CreateAccountAlias`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.iam.model.CreateAccountAliasRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateAccountAlias {
 public static void main(String[] args) {
 final String usage = ""
 Usage:
 <alias>\s

 Where:
 alias - The account alias to create (for example, myawsaccount).
\s

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String alias = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 createIAMAccountAlias(iam, alias);
 iam.close();
 System.out.println("Done");
 }

 public static void createIAMAccountAlias(IamClient iam, String alias) {
 try {
 CreateAccountAliasRequest request = CreateAccountAliasRequest.builder()

```

```

 .accountAlias(alias)
 .build();

 iam.createAccountAlias(request);
 System.out.println("Successfully created account alias: " + alias);

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
}

```

- For API details, see [CreateAccountAlias](#) in *AWS SDK for Java 2.x API Reference*.

## CreatePolicy

The following code example shows how to use CreatePolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.services.iam.model.CreatePolicyRequest;
import software.amazon.awssdk.services.iam.model.CreatePolicyResponse;
import software.amazon.awssdk.services.iam.model.GetPolicyRequest;
import software.amazon.awssdk.services.iam.model.GetPolicyResponse;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.waiters.IamWaiter;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 */

```



```

* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreatePolicy {

 public static final String PolicyDocument = "{" +
 " \"Version\": \"2012-10-17\", \" +
 " \"Statement\": [\" +
 " {\" +
 " \"Effect\": \"Allow\", \" +
 " \"Action\": [\" +
 " \"dynamodb:DeleteItem\", \" +
 " \"dynamodb:GetItem\", \" +
 " \"dynamodb:PutItem\", \" +
 " \"dynamodb:Scan\", \" +
 " \"dynamodb:UpdateItem\"" +
 "], \" +
 " \"Resource\": \"*\"" +
 " }\" +
 "]" +
 "}";

 public static void main(String[] args) {

 final String usage = ""
 Usage:
 CreatePolicy <policyName>\s

 Where:
 policyName - A unique policy name.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String policyName = args[0];
 Region region = Region.AWS_GLOBAL;
 IAMClient iam = IAMClient.builder()
 .region(region)
 .build();
 }
}

```

```

 String result = createIAMPolicy(iam, policyName);
 System.out.println("Successfully created a policy with this ARN value: " +
result);
 iam.close();
 }

 public static String createIAMPolicy(IamClient iam, String policyName) {
 try {
 // Create an IamWaiter object.
 IamWaiter iamWaiter = iam.waiter();

 CreatePolicyRequest request = CreatePolicyRequest.builder()
 .policyName(policyName)
 .policyDocument(PolicyDocument)
 .build();

 CreatePolicyResponse response = iam.createPolicy(request);

 // Wait until the policy is created.
 GetPolicyRequest polRequest = GetPolicyRequest.builder()
 .policyArn(response.policy().arn())
 .build();

 WaiterResponse<GetPolicyResponse> waitUntilPolicyExists =
iamWaiter.waitUntilPolicyExists(polRequest);

 waitUntilPolicyExists.matched().response().ifPresent(System.out::println);
 return response.policy().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [CreatePolicy](#) in *AWS SDK for Java 2.x API Reference*.

## CreateRole

The following code example shows how to use CreateRole.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import software.amazon.awssdk.services.iam.model.CreateRoleRequest;
import software.amazon.awssdk.services.iam.model.CreateRoleResponse;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import java.io.FileReader;

/*
 * This example requires a trust policy document. For more information, see:
 * https://aws.amazon.com/blogs/security/how-to-use-trust-policies-with-iam-roles/
 *
 * In addition, set up your development environment, including your credentials.
 *
 * For information, see this documentation topic:
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class CreateRole {
 public static void main(String[] args) throws Exception {
 final String usage = ""
 Usage:
 <rolename> <fileLocation>\s

 Where:
 rolename - The name of the role to create.\s
 fileLocation - The location of the JSON document that represents
the trust policy.\s
 """;

 if (args.length != 2) {
```

```

 System.out.println(usage);
 System.exit(1);
 }

 String rolename = args[0];
 String fileLocation = args[1];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 String result = createIAMRole(iam, rolename, fileLocation);
 System.out.println("Successfully created user: " + result);
 iam.close();
}

public static String createIAMRole(IamClient iam, String rolename, String
fileLocation) throws Exception {
 try {
 JSONObject jsonObject = (JSONObject) readJsonSimpleDemo(fileLocation);
 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(rolename)
 .assumeRolePolicyDocument(jsonObject.toJSONString())
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse response = iam.createRole(request);
 System.out.println("The ARN of the role is " + response.role().arn());

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static Object readJsonSimpleDemo(String filename) throws Exception {
 FileReader reader = new FileReader(filename);
 JSONParser jsonParser = new JSONParser();
 return jsonParser.parse(reader);
}
}

```

- For API details, see [CreateRole](#) in *AWS SDK for Java 2.x API Reference*.

## CreateUser

The following code example shows how to use CreateUser.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.waiters.WaiterResponse;
import software.amazon.awssdk.services.iam.model.CreateUserRequest;
import software.amazon.awssdk.services.iam.model.CreateUserResponse;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.waiters.IamWaiter;
import software.amazon.awssdk.services.iam.model.GetUserRequest;
import software.amazon.awssdk.services.iam.model.GetUserResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateUser {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <username>\s

 Where:
 username - The name of the user to create.\s
 "";
```

```
 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String username = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 String result = createIAMUser(iam, username);
 System.out.println("Successfully created user: " + result);
 iam.close();
}

public static String createIAMUser(IamClient iam, String username) {
 try {
 // Create an IamWaiter object.
 IamWaiter iamWaiter = iam.waiter();

 CreateUserRequest request = CreateUserRequest.builder()
 .userName(username)
 .build();

 CreateUserResponse response = iam.createUser(request);

 // Wait until the user is created.
 GetUserRequest userRequest = GetUserRequest.builder()
 .userName(response.user().userName())
 .build();

 WaiterResponse<GetUserResponse> waitUntilUserExists =
iamWaiter.waitUntilUserExists(userRequest);
 waitUntilUserExists.matched().response().ifPresent(System.out::println);
 return response.user().userName();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

```
}
```

- For API details, see [CreateUser](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAccessKey

The following code example shows how to use DeleteAccessKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.DeleteAccessKeyRequest;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteAccessKey {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <username> <accessKey>\s

 Where:
 username - The name of the user.\s
 accessKey - The access key ID for the secret access key you want
to delete.\s
```

```

 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String username = args[0];
 String accessKey = args[1];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();
 deleteKey(iam, username, accessKey);
 iam.close();
}

public static void deleteKey(IamClient iam, String username, String accessKey) {
 try {
 DeleteAccessKeyRequest request = DeleteAccessKeyRequest.builder()
 .accessKeyId(accessKey)
 .userName(username)
 .build();

 iam.deleteAccessKey(request);
 System.out.println("Successfully deleted access key " + accessKey +
 " from user " + username);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteAccessKey](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAccountAlias

The following code example shows how to use DeleteAccountAlias.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.iam.model.DeleteAccountAliasRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteAccountAlias {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <alias>\s

 Where:
 alias - The account alias to delete.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String alias = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();
```

```
 deleteIAMAccountAlias(iam, alias);
 iam.close();
 }

 public static void deleteIAMAccountAlias(IamClient iam, String alias) {
 try {
 DeleteAccountAliasRequest request = DeleteAccountAliasRequest.builder()
 .accountAlias(alias)
 .build();

 iam.deleteAccountAlias(request);
 System.out.println("Successfully deleted account alias " + alias);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
 }
}
```

- For API details, see [DeleteAccountAlias](#) in *AWS SDK for Java 2.x API Reference*.

## DeletePolicy

The following code example shows how to use DeletePolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.iam.model.DeletePolicyRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
```

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeletePolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <policyARN>\s

 Where:
 policyARN - A policy ARN value to delete.\s
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String policyARN = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 deleteIAMPolicy(iam, policyARN);
 iam.close();
 }

 public static void deleteIAMPolicy(IamClient iam, String policyARN) {
 try {
 DeletePolicyRequest request = DeletePolicyRequest.builder()
 .policyArn(policyARN)
 .build();

 iam.deletePolicy(request);
 System.out.println("Successfully deleted the policy");
 } catch (IamException e) {
```

```

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
}
}

```

- For API details, see [DeletePolicy](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteUser

The following code example shows how to use DeleteUser.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.DeleteUserRequest;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteUser {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <userName>\s

```

```

 Where:
 userName - The name of the user to delete.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String userName = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 deleteIAMUser(iam, userName);
 System.out.println("Done");
 iam.close();
}

public static void deleteIAMUser(IamClient iam, String userName) {
 try {
 DeleteUserRequest request = DeleteUserRequest.builder()
 .userName(userName)
 .build();

 iam.deleteUser(request);
 System.out.println("Successfully deleted IAM user " + userName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteUser](#) in *AWS SDK for Java 2.x API Reference*.

## DetachRolePolicy

The following code example shows how to use DetachRolePolicy.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.iam.model.DetachRolePolicyRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetachRolePolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <roleName> <policyArn>\s

 Where:
 roleName - A role name that you can obtain from the AWS
Management Console.\s
 policyArn - A policy ARN that you can obtain from the AWS
Management Console.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String roleName = args[0];
 String policyArn = args[1];
```

```
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();
 detachPolicy(iam, roleName, policyArn);
 System.out.println("Done");
 iam.close();
 }

 public static void detachPolicy(IamClient iam, String roleName, String
policyArn) {
 try {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policyArn)
 .build();

 iam.detachRolePolicy(request);
 System.out.println("Successfully detached policy " + policyArn +
 " from role " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DetachRolePolicy](#) in *AWS SDK for Java 2.x API Reference*.

## ListAccessKeys

The following code example shows how to use ListAccessKeys.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.iam.model.AccessKeyMetadata;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListAccessKeysRequest;
import software.amazon.awssdk.services.iam.model.ListAccessKeysResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListAccessKeys {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <userName>\s

 Where:
 userName - The name of the user for which access keys are
retrieved.\s

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String userName = args[0];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 listKeys(iam, userName);
 System.out.println("Done");
 iam.close();
 }
}

```



```
public static void listKeys(IamClient iam, String userName) {
 try {
 boolean done = false;
 String newMarker = null;

 while (!done) {
 ListAccessKeysResponse response;

 if (newMarker == null) {
 ListAccessKeysRequest request = ListAccessKeysRequest.builder()
 .userName(userName)
 .build();

 response = iam.listAccessKeys(request);
 } else {
 ListAccessKeysRequest request = ListAccessKeysRequest.builder()
 .userName(userName)
 .marker(newMarker)
 .build();

 response = iam.listAccessKeys(request);
 }

 for (AccessKeyMetadata metadata : response.accessKeyMetadata()) {
 System.out.format("Retrieved access key %s",
metadata.accessKeyId());
 }

 if (!response.isTruncated()) {
 done = true;
 } else {
 newMarker = response.marker();
 }
 }

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListAccessKeys](#) in *AWS SDK for Java 2.x API Reference*.

## ListAccountAliases

The following code example shows how to use ListAccountAliases.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListAccountAliasesResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListAccountAliases {
 public static void main(String[] args) {
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 listAliases(iam);
 System.out.println("Done");
 iam.close();
 }

 public static void listAliases(IamClient iam) {
 try {
 ListAccountAliasesResponse response = iam.listAccountAliases();
 }
 }
}
```

```

 for (String alias : response.accountAliases()) {
 System.out.printf("Retrieved account alias %s", alias);
 }

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [ListAccountAliases](#) in *AWS SDK for Java 2.x API Reference*.

## ListUsers

The following code example shows how to use ListUsers.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.iam.model.AttachedPermissionsBoundary;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.ListUsersRequest;
import software.amazon.awssdk.services.iam.model.ListUsersResponse;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.User;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

```

```
public class ListUsers {
 public static void main(String[] args) {
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 listAllUsers(iam);
 System.out.println("Done");
 iam.close();
 }

 public static void listAllUsers(IamClient iam) {
 try {
 boolean done = false;
 String newMarker = null;
 while (!done) {
 ListUsersResponse response;
 if (newMarker == null) {
 ListUsersRequest request = ListUsersRequest.builder().build();
 response = iam.listUsers(request);
 } else {
 ListUsersRequest request = ListUsersRequest.builder()
 .marker(newMarker)
 .build();

 response = iam.listUsers(request);
 }

 for (User user : response.users()) {
 System.out.format("\n Retrieved user %s", user.userName());
 AttachedPermissionsBoundary permissionsBoundary =
user.permissionsBoundary();
 if (permissionsBoundary != null)
 System.out.format("\n Permissions boundary details %s",
permissionsBoundary.permissionsBoundaryTypeAsString());
 }

 if (!response.isTruncated()) {
 done = true;
 } else {
 newMarker = response.marker();
 }
 }
 }
 }
}
```

```

 }

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListUsers](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateAccessKey

The following code example shows how to use UpdateAccessKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.StatusType;
import software.amazon.awssdk.services.iam.model.UpdateAccessKeyRequest;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class UpdateAccessKey {

 private static StatusType statusType;

```

```

public static void main(String[] args) {
 final String usage = ""

 Usage:
 <username> <accessId> <status>\s

 Where:
 username - The name of the user whose key you want to update.\s
 accessId - The access key ID of the secret access key you want
to update.\s
 status - The status you want to assign to the secret access key.
\s

 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String username = args[0];
 String accessId = args[1];
 String status = args[2];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 updateKey(iam, username, accessId, status);
 System.out.println("Done");
 iam.close();
}

public static void updateKey(IamClient iam, String username, String accessId,
String status) {
 try {
 if (status.toLowerCase().equalsIgnoreCase("active")) {
 statusType = StatusType.ACTIVE;
 } else if (status.toLowerCase().equalsIgnoreCase("inactive")) {
 statusType = StatusType.INACTIVE;
 } else {
 statusType = StatusType.UNKNOWN_TO_SDK_VERSION;
 }

 UpdateAccessKeyRequest request = UpdateAccessKeyRequest.builder()

```

```

 .accessKeyId(accessId)
 .userName(username)
 .status(statusType)
 .build();

 iam.updateAccessKey(request);
 System.out.printf("Successfully updated the status of access key %s to"
+
 "status %s for user %s", accessId, status, username);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [UpdateAccessKey](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateUser

The following code example shows how to use UpdateUser.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.IamException;
import software.amazon.awssdk.services.iam.model.UpdateUserRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:

```

```

*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class UpdateUser {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <curName> <newName>\s

 Where:
 curName - The current user name.\s
 newName - An updated user name.\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String curName = args[0];
 String newName = args[1];
 Region region = Region.AWS_GLOBAL;
 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 updateIAMUser(iam, curName, newName);
 System.out.println("Done");
 iam.close();
 }

 public static void updateIAMUser(IamClient iam, String curName, String newName)
 {
 try {
 UpdateUserRequest request = UpdateUserRequest.builder()
 .userName(curName)
 .newUserName(newName)
 .build();

 iam.updateUser(request);
 System.out.printf("Successfully updated user to username %s", newName);

 } catch (IamException e) {

```



```
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [UpdateUser](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build and manage a resilient service

The following code example shows how to create a load-balanced web service that returns book, movie, and song recommendations. The example shows how the service responds to failures, and how to restructure the service for more resilience when failures occur.

- Use an Amazon EC2 Auto Scaling group to create Amazon Elastic Compute Cloud (Amazon EC2) instances based on a launch template and to keep the number of instances in a specified range.
- Handle and distribute HTTP requests with Elastic Load Balancing.
- Monitor the health of instances in an Auto Scaling group and forward requests only to healthy instances.
- Run a Python web server on each EC2 instance to handle HTTP requests. The web server responds with recommendations and health checks.
- Simulate a recommendation service with an Amazon DynamoDB table.
- Control web server response to requests and health checks by updating AWS Systems Manager parameters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run the interactive scenario at a command prompt.

```
public class Main {

 public static final String fileName = "C:\\\\AWS\\resworkflow\\
\\recommendations.json"; // Modify file location.
 public static final String tableName = "doc-example-recommendation-service";
 public static final String startScript = "C:\\\\AWS\\resworkflow\\
\\server_startup_script.sh"; // Modify file location.
 public static final String policyFile = "C:\\\\AWS\\resworkflow\\
\\instance_policy.json"; // Modify file location.
 public static final String ssmJSON = "C:\\\\AWS\\resworkflow\\
\\ssm_only_policy.json"; // Modify file location.
 public static final String failureResponse = "doc-example-resilient-
architecture-failure-response";
 public static final String healthCheck = "doc-example-resilient-architecture-
health-check";
 public static final String templateName = "doc-example-resilience-template";
 public static final String roleName = "doc-example-resilience-role";
 public static final String policyName = "doc-example-resilience-pol";
 public static final String profileName = "doc-example-resilience-prof";

 public static final String badCredsProfileName = "doc-example-resilience-prof-
bc";

 public static final String targetGroupName = "doc-example-resilience-tg";
 public static final String autoScalingGroupName = "doc-example-resilience-
group";
 public static final String lbName = "doc-example-resilience-lb";
 public static final String protocol = "HTTP";
 public static final int port = 80;

 public static final String DASHES = new String(new char[80]).replace("\\0", "-");

 public static void main(String[] args) throws IOException, InterruptedException
 {
 Scanner in = new Scanner(System.in);
 Database database = new Database();
 AutoScaler autoScaler = new AutoScaler();
 LoadBalancer loadBalancer = new LoadBalancer();

 System.out.println(DASHES);
 System.out.println("Welcome to the demonstration of How to Build and Manage
a Resilient Service!");
 System.out.println(DASHES);
 }
}
```

```

 System.out.println(DASHES);
 System.out.println("A - SETUP THE RESOURCES");
 System.out.println("Press Enter when you're ready to start deploying
resources.");
 in.nextLine();
 deploy(loadBalancer);
 System.out.println(DASHES);
 System.out.println(DASHES);
 System.out.println("B - DEMO THE RESILIENCE FUNCTIONALITY");
 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 demo(loadBalancer);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("C - DELETE THE RESOURCES");
 System.out.println("""
 This concludes the demo of how to build and manage a resilient
service.

 To keep things tidy and to avoid unwanted charges on your account,
we can clean up all AWS resources
 that were created for this demo.
 """);

 System.out.println("\n Do you want to delete the resources (y/n)? ");
 String userInput = in.nextLine().trim().toLowerCase(); // Capture user input

 if (userInput.equals("y")) {
 // Delete resources here
 deleteResources(loadBalancer, autoScaler, database);
 System.out.println("Resources deleted.");
 } else {
 System.out.println("""
 Okay, we'll leave the resources intact.
 Don't forget to delete them when you're done with them or you
might incur unexpected charges.
 """);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The example has completed. ");
 System.out.println("\n Thanks for watching!");

```

```

 System.out.println(DASHES);
 }

 // Deletes the AWS resources used in this example.
 private static void deleteResources(LoadBalancer loadBalancer, AutoScaler
autoScaler, Database database)
 throws IOException, InterruptedException {
 loadBalancer.deleteLoadBalancer(lbName);
 System.out.println("*** Wait 30 secs for resource to be deleted");
 TimeUnit.SECONDS.sleep(30);
 loadBalancer.deleteTargetGroup(targetGroupName);
 autoScaler.deleteAutoScalingGroup(autoScalingGroupName);
 autoScaler.deleteRolesPolicies(policyName, roleName, profileName);
 autoScaler.deleteTemplate(templateName);
 database.deleteTable(tableName);
 }

 private static void deploy(LoadBalancer loadBalancer) throws
InterruptedException, IOException {
 Scanner in = new Scanner(System.in);
 System.out.println(
 """
 For this demo, we'll use the AWS SDK for Java (v2) to create
several AWS resources
 to set up a load-balanced web service endpoint and explore
some ways to make it resilient
 against various kinds of failures.

 Some of the resources create by this demo are:
 \t* A DynamoDB table that the web service depends on to
provide book, movie, and song recommendations.
 \t* An EC2 launch template that defines EC2 instances that
each contain a Python web server.
 \t* An EC2 Auto Scaling group that manages EC2 instances
across several Availability Zones.
 \t* An Elastic Load Balancing (ELB) load balancer that
targets the Auto Scaling group to distribute requests.
 """);

 System.out.println("Press Enter when you're ready.");
 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);

```

```

 System.out.println("Creating and populating a DynamoDB table named " +
tableName);
 Database database = new Database();
 database.createTable(tableName, fileName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an EC2 launch template that runs '{startup_script}' when an
instance starts.
 This script starts a Python web server defined in the `server.py`
script. The web server
 listens to HTTP requests on port 80 and responds to requests to '/'
and to '/healthcheck'.
 For demo purposes, this server is run as the root user. In
production, the best practice is to
 run a web server, such as Apache, with least-privileged credentials.

 The template also defines an IAM policy that each instance uses to
assume a role that grants
 permissions to access the DynamoDB recommendation table and Systems
Manager parameters
 that control the flow of the demo.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 templateCreator.createTemplate(policyFile, policyName, profileName,
startScript, templateName, roleName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println(
 "Creating an EC2 Auto Scaling group that maintains three EC2
instances, each in a different Availability Zone.");
 System.out.println("*** Wait 30 secs for the VPC to be created");
 TimeUnit.SECONDS.sleep(30);
 AutoScaler autoScaler = new AutoScaler();
 String[] zones = autoScaler.createGroup(3, templateName,
autoScalingGroupName);

 System.out.println("""
 At this point, you have EC2 instances created. Once each instance
starts, it listens for

```

```

 HTTP requests. You can see these instances in the console or
 continue with the demo.
 Press Enter when you're ready to continue.
 "");

 in.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Creating variables that control the flow of the demo.");
 ParameterHelper paramHelper = new ParameterHelper();
 paramHelper.reset();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("""
 Creating an Elastic Load Balancing target group and load balancer.
 The target group
 defines how the load balancer connects to instances. The load
 balancer provides a
 single endpoint where clients connect and dispatches requests to
 instances in the group.
 """);

 String vpcId = autoScaler.getDefaultVPC();
 List<Subnet> subnets = autoScaler.getSubnets(vpcId, zones);
 System.out.println("You have retrieved a list with " + subnets.size() + "
subnets");
 String targetGroupArn = loadBalancer.createTargetGroup(protocol, port,
vpcId, targetGroupName);
 String elbDnsName = loadBalancer.createLoadBalancer(subnets, targetGroupArn,
lbName, port, protocol);
 autoScaler.attachLoadBalancerTargetGroup(autoScalingGroupName,
targetGroupArn);
 System.out.println("Verifying access to the load balancer endpoint...");
 boolean wasSuccessful = loadBalancer.verifyLoadBalancerEndpoint(elbDnsName);
 if (!wasSuccessful) {
 System.out.println("Couldn't connect to the load balancer, verifying
that the port is open...");
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to "http://checkip.amazonaws.com"
 HttpGet httpGet = new HttpGet("http://checkip.amazonaws.com");
 try {

```

```

 // Execute the request and get the response
 HttpResponse response = httpClient.execute(httpGet);

 // Read the response content.
 String ipAddress =
IOUtils.toString(response.getEntity().getContent(), StandardCharsets.UTF_8).trim();

 // Print the public IP address.
 System.out.println("Public IP Address: " + ipAddress);
 GroupInfo groupInfo = autoScaler.verifyInboundPort(vpcId, port,
ipAddress);
 if (!groupInfo.isPortOpen()) {
 System.out.println("""
 For this example to work, the default security group for
your default VPC must
 allow access from this computer. You can either add it
automatically from this
 example or add it yourself using the AWS Management
Console.
 """);

 System.out.println(
 "Do you want to add a rule to security group " +
groupInfo.getGroupName() + " to allow");
 System.out.println("inbound traffic on port " + port + " from
your computer's IP address (y/n) ");
 String ans = in.nextLine();
 if ("y".equalsIgnoreCase(ans)) {
 autoScaler.openInboundPort(groupInfo.getGroupName(),
String.valueOf(port), ipAddress);
 System.out.println("Security group rule added.");
 } else {
 System.out.println("No security group rule added.");
 }
 }

 } catch (AutoScalingException e) {
 e.printStackTrace();
 }
 } else if (wasSuccessful) {
 System.out.println("Your load balancer is ready. You can access it by
browsing to:");
 System.out.println("\t http://" + elbDnsName);
 } else {

```

```

 System.out.println("Couldn't get a successful response from the load
balancer endpoint. Troubleshoot by");
 System.out.println("manually verifying that your VPC and security group
are configured correctly and that");
 System.out.println("you can successfully make a GET request to the load
balancer.");
 }

 System.out.println("Press Enter when you're ready to continue with the
demo.");
 in.nextLine();
}

// A method that controls the demo part of the Java program.
public static void demo(LoadBalancer loadBalancer) throws IOException,
InterruptedException {
 ParameterHelper paramHelper = new ParameterHelper();
 System.out.println("Read the ssm_only_policy.json file");
 String ssmOnlyPolicy = readFileAsString(ssmJSON);

 System.out.println("Resetting parameters to starting values for demo.");
 paramHelper.reset();

 System.out.println(
 """
 This part of the demonstration shows how to toggle
different parts of the system
 to create situations where the web service fails, and shows
how using a resilient
 architecture can keep the web service running in spite of
these failures.

 At the start, the load balancer endpoint returns
recommendations and reports that all targets are healthy.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 """
 The web service running on the EC2 instances gets
recommendations by querying a DynamoDB table.
 The table name is contained in a Systems Manager parameter
named self.param_helper.table.
 """
);
}

```



```

 To simulate a failure of the recommendation service, let's
 set this parameter to name a non-existent table.
 """);
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 System.out.println(
 ""
 \nNow, sending a GET request to the load balancer endpoint
 returns a failure code. But, the service reports as
 healthy to the load balancer because shallow health checks
 don't check for failure of the recommendation service.
 """);
 demoChoices(loadBalancer);

 System.out.println(
 ""
 Instead of failing when the recommendation service fails,
 the web service can return a static response.
 While this is not a perfect solution, it presents the
 customer with a somewhat better experience than failure.
 """);
 paramHelper.put(paramHelper.failureResponse, "static");

 System.out.println("""
 Now, sending a GET request to the load balancer endpoint returns a
 static response.
 The service still reports as healthy because health checks are still
 shallow.
 """);
 demoChoices(loadBalancer);

 System.out.println("Let's reinstate the recommendation service.");
 paramHelper.put(paramHelper.tableName, paramHelper.dyntable);

 System.out.println("""
 Let's also substitute bad credentials for one of the instances in
 the target group so that it can't
 access the DynamoDB recommendation table. We will get an instance id
 value.
 """);

 LaunchTemplateCreator templateCreator = new LaunchTemplateCreator();
 AutoScaler autoScaler = new AutoScaler();

```

```

 // Create a new instance profile based on badCredsProfileName.
 templateCreator.createInstanceProfile(policyFile, policyName,
badCredsProfileName, roleName);
 String badInstanceId = autoScaler.getBadInstance(autoScalingGroupName);
 System.out.println("The bad instance id values used for this demo is " +
badInstanceId);

 String profileAssociationId = autoScaler.getInstanceProfile(badInstanceId);
 System.out.println("The association Id value is " + profileAssociationId);
 System.out.println("Replacing the profile for instance " + badInstanceId
 + " with a profile that contains bad credentials");
 autoScaler.replaceInstanceProfile(badInstanceId, badCredsProfileName,
profileAssociationId);

 System.out.println(
 ""
 Now, sending a GET request to the load balancer endpoint
returns either a recommendation or a static response,
 depending on which instance is selected by the load
balancer.
 "");

 demoChoices(loadBalancer);

 System.out.println("""
 Let's implement a deep health check. For this demo, a deep health
check tests whether
 the web service can access the DynamoDB table that it depends on for
recommendations. Note that
 the deep health check is only for ELB routing and not for Auto
Scaling instance health.
 This kind of deep health check is not recommended for Auto Scaling
instance health, because it
 risks accidental termination of all instances in the Auto Scaling
group when a dependent service fails.
 """);

 System.out.println("""
 By implementing deep health checks, the load balancer can detect
when one of the instances is failing
 and take that instance out of rotation.
 """);

 paramHelper.put(paramHelper.healthCheck, "deep");

```

```
 System.out.println("""
 Now, checking target health indicates that the instance with bad
credentials
 is unhealthy. Note that it might take a minute or two for the load
balancer to detect the unhealthy
 instance. Sending a GET request to the load balancer endpoint always
returns a recommendation, because
 the load balancer takes unhealthy instances out of its rotation.
 """);

 demoChoices(loadBalancer);

 System.out.println(
 """
 Because the instances in this demo are controlled by an auto
scaler, the simplest way to fix an unhealthy
 instance is to terminate it and let the auto scaler start a
new instance to replace it.
 """);
 autoScaler.terminateInstance(badInstanceId);

 System.out.println("""
 Even while the instance is terminating and the new instance is
starting, sending a GET
 request to the web service continues to get a successful
recommendation response because
 the load balancer routes requests to the healthy instances. After
the replacement instance
 starts and reports as healthy, it is included in the load balancing
rotation.

 Note that terminating and replacing an instance typically takes
several minutes, during which time you
 can see the changing health check status until the new instance is
running and healthy.
 """);

 demoChoices(loadBalancer);
 System.out.println(
 "If the recommendation service fails now, deep health checks mean
all instances report as unhealthy.");
 paramHelper.put(paramHelper.tableName, "this-is-not-a-table");

 demoChoices(loadBalancer);
```

```

 paramHelper.reset();
 }

 public static void demoChoices(LoadBalancer loadBalancer) throws IOException,
 InterruptedException {
 String[] actions = {
 "Send a GET request to the load balancer endpoint.",
 "Check the health of load balancer targets.",
 "Go to the next part of the demo."
 };
 Scanner scanner = new Scanner(System.in);

 while (true) {
 System.out.println("-".repeat(88));
 System.out.println("See the current state of the service by selecting
one of the following choices:");
 for (int i = 0; i < actions.length; i++) {
 System.out.println(i + ": " + actions[i]);
 }

 try {
 System.out.print("\nWhich action would you like to take? ");
 int choice = scanner.nextInt();
 System.out.println("-".repeat(88));

 switch (choice) {
 case 0 -> {
 System.out.println("Request:\n");
 System.out.println("GET http://" +
loadBalancer.getEndpoint(lbName));
 CloseableHttpClient httpClient =
HttpClientBuilder.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" +
loadBalancer.getEndpoint(lbName));

 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);

 // Display the JSON response
 BufferedReader reader = new BufferedReader(

```

```

 new
 InputStreamReader(response.getEntity().getContent()));
 StringBuilder jsonResponse = new StringBuilder();
 String line;
 while ((line = reader.readLine()) != null) {
 jsonResponse.append(line);
 }
 reader.close();

 // Print the formatted JSON response.
 System.out.println("Full Response:\n");
 System.out.println(jsonResponse.toString());

 // Close the HTTP client.
 httpClient.close();
 }
 case 1 -> {
 System.out.println("\nChecking the health of load balancer
targets:\n");
 List<TargetHealthDescription> health =
loadBalancer.checkTargetHealth(targetGroupName);
 for (TargetHealthDescription target : health) {
 System.out.printf("\tTarget %s on port %d is %s\n",
target.target().id(),
 target.target().port(),
target.targetHealth().stateAsString());
 }
 System.out.println("""
 Note that it can take a minute or two for the health
check to update

 after changes are made.
 """);
 }
 case 2 -> {
 System.out.println("\nOkay, let's move on.");
 System.out.println("-".repeat(88));
 return; // Exit the method when choice is 2
 }
 default -> System.out.println("You must choose a value between
0-2. Please select again.");
}

} catch (java.util.InputMismatchException e) {

```

```

 System.out.println("Invalid input. Please select again.");
 scanner.nextLine(); // Clear the input buffer.
 }
}

public static String readFileAsString(String filePath) throws IOException {
 byte[] bytes = Files.readAllBytes(Paths.get(filePath));
 return new String(bytes);
}
}

```

Create a class that wraps Auto Scaling and Amazon EC2 actions.

```

public class AutoScaler {

 private static Ec2Client ec2Client;
 private static AutoScalingClient autoScalingClient;
 private static IamClient iamClient;

 private static SsmClient ssmClient;

 private IamClient getIAMClient() {
 if (iamClient == null) {
 iamClient = IamClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return iamClient;
 }

 private SsmClient getSSMClient() {
 if (ssmClient == null) {
 ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ssmClient;
 }

 private Ec2Client getEc2Client() {
 if (ec2Client == null) {

```

```

 ec2Client = Ec2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return ec2Client;
}

private AutoScalingClient getAutoScalingClient() {
 if (autoScalingClient == null) {
 autoScalingClient = AutoScalingClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return autoScalingClient;
}

/**
 * Terminates instances in an EC2 Auto Scaling group. After an instance is
 * terminated, it can no longer be accessed.
 */
public void terminateInstance(String instanceId) {
 TerminateInstanceInAutoScalingGroupRequest terminateInstanceRequest =
 TerminateInstanceInAutoScalingGroupRequest
 .builder()
 .instanceId(instanceId)
 .shouldDecrementDesiredCapacity(false)
 .build();

 getAutoScalingClient().terminateInstanceInAutoScalingGroup(terminateInstanceRequest);
 System.out.format("Terminated instance %s.", instanceId);
}

/**
 * Replaces the profile associated with a running instance. After the profile is
 * replaced, the instance is rebooted to ensure that it uses the new profile.
 * When
 * the instance is ready, Systems Manager is used to restart the Python web
 * server.
 */
public void replaceInstanceProfile(String instanceId, String
newInstanceProfileName, String profileAssociationId)
 throws InterruptedException {
 // Create an IAM instance profile specification.

```

```

 software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
iamInstanceProfile =
software.amazon.awssdk.services.ec2.model.IamInstanceProfileSpecification
 .builder()
 .name(newInstanceProfileName) // Make sure 'newInstanceProfileName'
is a valid IAM Instance Profile
 // name.
 .build();

// Replace the IAM instance profile association for the EC2 instance.
ReplaceIamInstanceProfileAssociationRequest replaceRequest =
ReplaceIamInstanceProfileAssociationRequest
 .builder()
 .iamInstanceProfile(iamInstanceProfile)
 .associationId(profileAssociationId) // Make sure
'profileAssociationId' is a valid association ID.
 .build();

try {
 getEc2Client().replaceIamInstanceProfileAssociation(replaceRequest);
 // Handle the response as needed.
} catch (Ec2Exception e) {
 // Handle exceptions, log, or report the error.
 System.err.println("Error: " + e.getMessage());
}

System.out.format("Replaced instance profile for association %s with profile
%s.", profileAssociationId,
 newInstanceProfileName);
TimeUnit.SECONDS.sleep(15);
boolean instReady = false;
int tries = 0;

// Reboot after 60 seconds
while (!instReady) {
 if (tries % 6 == 0) {
 getEc2Client().rebootInstances(RebootInstancesRequest.builder()
 .instanceIds(instanceId)
 .build());
 System.out.println("Rebooting instance " + instanceId + " and
waiting for it to be ready.");
 }
 tries++;
 try {
 TimeUnit.SECONDS.sleep(10);
 }
}

```





```

 * and deletes all the resources.
 */
 public void deleteInstanceProfile(String roleName, String profileName) {
 try {
 software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
getInstanceProfileRequest =
software.amazon.awssdk.services.iam.model.GetInstanceProfileRequest
 .builder()
 .instanceProfileName(profileName)
 .build();

 GetInstanceProfileResponse response =
getIAMClient().getInstanceProfile(getInstanceProfileRequest);
 String name = response.instanceProfile().instanceProfileName();
 System.out.println(name);

 RemoveRoleFromInstanceProfileRequest profileRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .roleName(roleName)
 .build();

 getIAMClient().removeRoleFromInstanceProfile(profileRequest);
 DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(profileName)
 .build();

 getIAMClient().deleteInstanceProfile(deleteInstanceProfileRequest);
 System.out.println("Deleted instance profile " + profileName);

 DeleteRoleRequest deleteRoleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 // List attached role policies.
 ListAttachedRolePoliciesResponse rolesResponse = getIAMClient()
 .listAttachedRolePolicies(role -> role.roleName(roleName));
 List<AttachedPolicy> attachedPolicies =
rolesResponse.attachedPolicies();
 for (AttachedPolicy attachedPolicy : attachedPolicies) {
 DetachRolePolicyRequest request = DetachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(attachedPolicy.policyArn())

```

```

 .build();

 getIAMClient().detachRolePolicy(request);
 System.out.println("Detached and deleted policy " +
attachedPolicy.policyName());
 }

 getIAMClient().deleteRole(deleteRoleRequest);
 System.out.println("Instance profile and role deleted.");

} catch (IamException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}

}

public void deleteTemplate(String templateName) {
 getEc2Client().deleteLaunchTemplate(name ->
name.launchTemplateName(templateName));
 System.out.format(templateName + " was deleted.");
}

public void deleteAutoScaleGroup(String groupName) {
 DeleteAutoScalingGroupRequest deleteAutoScalingGroupRequest =
DeleteAutoScalingGroupRequest.builder()
 .autoScalingGroupName(groupName)
 .forceDelete(true)
 .build();

 getAutoScalingClient().deleteAutoScalingGroup(deleteAutoScalingGroupRequest);
 System.out.println(groupName + " was deleted.");
}

/*
 * Verify the default security group of the specified VPC allows ingress from
 * this
 * computer. This can be done by allowing ingress from this computer's IP
 * address. In some situations, such as connecting from a corporate network, you
 * must instead specify a prefix list ID. You can also temporarily open the port
 * to
 * any IP address while running this example. If you do, be sure to remove
 * public
 * access when you're done.

```

```

 *
 */
 public GroupInfo verifyInboundPort(String VPC, int port, String ipAddress) {
 boolean portIsOpen = false;
 GroupInfo groupInfo = new GroupInfo();
 try {
 Filter filter = Filter.builder()
 .name("group-name")
 .values("default")
 .build();

 Filter filter1 = Filter.builder()
 .name("vpc-id")
 .values(VPC)
 .build();

 DescribeSecurityGroupsRequest securityGroupsRequest =
 DescribeSecurityGroupsRequest.builder()
 .filters(filter, filter1)
 .build();

 DescribeSecurityGroupsResponse securityGroupsResponse = getEc2Client()
 .describeSecurityGroups(securityGroupsRequest);
 String securityGroup =
 securityGroupsResponse.securityGroups().get(0).groupName();
 groupInfo.setGroupName(securityGroup);

 for (SecurityGroup secGroup : securityGroupsResponse.securityGroups()) {
 System.out.println("Found security group: " + secGroup.groupId());

 for (IpPermission ipPermission : secGroup.ipPermissions()) {
 if (ipPermission.fromPort() == port) {
 System.out.println("Found inbound rule: " + ipPermission);
 for (IpRange ipRange : ipPermission.ipRanges()) {
 String cidrIp = ipRange.cidrIp();
 if (cidrIp.startsWith(ipAddress) ||
 cidrIp.equals("0.0.0.0/0")) {
 System.out.println(cidrIp + " is applicable");
 portIsOpen = true;
 }
 }
 }

 if (!ipPermission.prefixListIds().isEmpty()) {
 System.out.println("Prefix lList is applicable");
 }
 }
 }
 }
 }

```

```

 portIsOpen = true;
 }

 if (!portIsOpen) {
 System.out
 .println("The inbound rule does not appear to be
open to either this computer's IP,"
 + " all IP addresses (0.0.0.0/0), or to
a prefix list ID.");
 } else {
 break;
 }
}

}

} catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
}

groupInfo.setPortOpen(portIsOpen);
return groupInfo;
}

/*
 * Attaches an Elastic Load Balancing (ELB) target group to this EC2 Auto
 * Scaling group.
 * The target group specifies how the load balancer forward requests to the
 * instances
 * in the group.
 */
public void attachLoadBalancerTargetGroup(String asGroupName, String
targetGroupARN) {
 try {
 AttachLoadBalancerTargetGroupsRequest targetGroupsRequest =
AttachLoadBalancerTargetGroupsRequest.builder()
 .autoScalingGroupName(asGroupName)
 .targetGroupARNs(targetGroupARN)
 .build();

getAutoScalingClient().attachLoadBalancerTargetGroups(targetGroupsRequest);
 System.out.println("Attached load balancer to " + asGroupName);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Creates an EC2 Auto Scaling group with the specified size.
public String[] createGroup(int groupSize, String templateName, String
autoScalingGroupName) {

 // Get availability zones.
 software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
zonesRequest =
software.amazon.awssdk.services.ec2.model.DescribeAvailabilityZonesRequest
 .builder()
 .build();

 DescribeAvailabilityZonesResponse zonesResponse =
getEc2Client().describeAvailabilityZones(zonesRequest);
 List<String> availabilityZoneNames =
zonesResponse.availabilityZones().stream()

.map(software.amazon.awssdk.services.ec2.model.AvailabilityZone::zoneName)
 .collect(Collectors.toList());

 String availabilityZones = String.join(",", availabilityZoneNames);
 LaunchTemplateSpecification specification =
LaunchTemplateSpecification.builder()
 .launchTemplateName(templateName)
 .version("$Default")
 .build();

 String[] zones = availabilityZones.split(",");
 CreateAutoScalingGroupRequest groupRequest =
CreateAutoScalingGroupRequest.builder()
 .launchTemplate(specification)
 .availabilityZones(zones)
 .maxSize(groupSize)
 .minSize(groupSize)
 .autoScalingGroupName(autoScalingGroupName)
 .build();

 try {
 getAutoScalingClient().createAutoScalingGroup(groupRequest);
 }
}

```

```

 } catch (AutoScalingException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Created an EC2 Auto Scaling group named " +
autoScalingGroupName);
 return zones;
 }

 public String getDefaultVPC() {
 // Define the filter.
 Filter defaultFilter = Filter.builder()
 .name("is-default")
 .values("true")
 .build();

 software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest request =
software.amazon.awssdk.services.ec2.model.DescribeVpcsRequest
 .builder()
 .filters(defaultFilter)
 .build();

 DescribeVpcsResponse response = getEc2Client().describeVpcs(request);
 return response.vpcs().get(0).vpcId();
 }

 // Gets the default subnets in a VPC for a specified list of Availability Zones.
 public List<Subnet> getSubnets(String vpcId, String[] availabilityZones) {
 List<Subnet> subnets = null;
 Filter vpcFilter = Filter.builder()
 .name("vpc-id")
 .values(vpcId)
 .build();

 Filter azFilter = Filter.builder()
 .name("availability-zone")
 .values(availabilityZones)
 .build();

 Filter defaultForAZ = Filter.builder()
 .name("default-for-az")
 .values("true")
 .build();
 }

```

```

 DescribeSubnetsRequest request = DescribeSubnetsRequest.builder()
 .filters(vpcFilter, azFilter, defaultForAZ)
 .build();

 DescribeSubnetsResponse response = getEc2Client().describeSubnets(request);
 subnets = response.subnets();
 return subnets;
 }

 // Gets data about the instances in the EC2 Auto Scaling group.
 public String getBadInstance(String groupName) {
 DescribeAutoScalingGroupsRequest request =
 DescribeAutoScalingGroupsRequest.builder()
 .autoScalingGroupNames(groupName)
 .build();

 DescribeAutoScalingGroupsResponse response =
 getAutoScalingClient().describeAutoScalingGroups(request);
 AutoScalingGroup autoScalingGroup = response.autoScalingGroups().get(0);
 List<String> instanceIds = autoScalingGroup.instances().stream()
 .map(instance -> instance.instanceId())
 .collect(Collectors.toList());

 String[] instanceIdArray = instanceIds.toArray(new String[0]);
 for (String instanceId : instanceIdArray) {
 System.out.println("Instance ID: " + instanceId);
 return instanceId;
 }
 return "";
 }

 // Gets data about the profile associated with an instance.
 public String getInstanceProfile(String instanceId) {
 Filter filter = Filter.builder()
 .name("instance-id")
 .values(instanceId)
 .build();

 DescribeIamInstanceProfileAssociationsRequest associationsRequest =
 DescribeIamInstanceProfileAssociationsRequest
 .builder()
 .filters(filter)
 .build();
 }

```



```

 DescribeIamInstanceProfileAssociationsResponse response = getEc2Client()
 .describeIamInstanceProfileAssociations(associationsRequest);
 return response.iamInstanceProfileAssociations().get(0).associationId();
 }

 public void deleteRolesPolicies(String policyName, String roleName, String
InstanceProfile) {
 ListPoliciesRequest listPoliciesRequest =
ListPoliciesRequest.builder().build();
 ListPoliciesResponse listPoliciesResponse =
getIAMClient().listPolicies(listPoliciesRequest);
 for (Policy policy : listPoliciesResponse.policies()) {
 if (policy.policyName().equals(policyName)) {
 // List the entities (users, groups, roles) that are attached to the
policy.

software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
listEntitiesRequest =
software.amazon.awssdk.services.iam.model.ListEntitiesForPolicyRequest
 .builder()
 .policyArn(policy.arn())
 .build();
 ListEntitiesForPolicyResponse listEntitiesResponse = iamClient
 .listEntitiesForPolicy(listEntitiesRequest);
 if (!listEntitiesResponse.policyGroups().isEmpty() || !
listEntitiesResponse.policyUsers().isEmpty()
 || !listEntitiesResponse.policyRoles().isEmpty()) {
 // Detach the policy from any entities it is attached to.
 DetachRolePolicyRequest detachPolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy.arn())
 .roleName(roleName) // Specify the name of the IAM role
 .build();

 getIAMClient().detachRolePolicy(detachPolicyRequest);
 System.out.println("Policy detached from entities.");
 }

 // Now, you can delete the policy.
 DeletePolicyRequest deletePolicyRequest =
DeletePolicyRequest.builder()
 .policyArn(policy.arn())
 .build();

```

```

 getIAMClient().deletePolicy(deletePolicyRequest);
 System.out.println("Policy deleted successfully.");
 break;
 }
}

// List the roles associated with the instance profile
ListInstanceProfilesForRoleRequest listRolesRequest =
ListInstanceProfilesForRoleRequest.builder()
 .roleName(roleName)
 .build();

// Detach the roles from the instance profile
ListInstanceProfilesForRoleResponse listRolesResponse =
iamClient.listInstanceProfilesForRole(listRolesRequest);
for (software.amazon.awssdk.services.iam.model.InstanceProfile profile :
listRolesResponse.instanceProfiles()) {
 RemoveRoleFromInstanceProfileRequest removeRoleRequest =
RemoveRoleFromInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .roleName(roleName) // Remove the extra dot here
 .build();

 getIAMClient().removeRoleFromInstanceProfile(removeRoleRequest);
 System.out.println("Role " + roleName + " removed from instance profile
" + InstanceProfile);
}

// Delete the instance profile after removing all roles
DeleteInstanceProfileRequest deleteInstanceProfileRequest =
DeleteInstanceProfileRequest.builder()
 .instanceProfileName(InstanceProfile)
 .build();

getIAMClient().deleteInstanceProfile(r ->
r.instanceProfileName(InstanceProfile));
System.out.println(InstanceProfile + " Deleted");
System.out.println("All roles and policies are deleted.");
}
}

```

## Create a class that wraps Elastic Load Balancing actions.

```
public class LoadBalancer {
 public ElasticLoadBalancingV2Client elasticLoadBalancingV2Client;

 public ElasticLoadBalancingV2Client getLoadBalancerClient() {
 if (elasticLoadBalancingV2Client == null) {
 elasticLoadBalancingV2Client = ElasticLoadBalancingV2Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }

 return elasticLoadBalancingV2Client;
 }

 // Checks the health of the instances in the target group.
 public List<TargetHealthDescription> checkTargetHealth(String targetGroupName) {
 DescribeTargetGroupsRequest targetGroupsRequest =
 DescribeTargetGroupsRequest.builder()
 .names(targetGroupName)
 .build();

 DescribeTargetGroupsResponse tgResponse =
 getLoadBalancerClient().describeTargetGroups(targetGroupsRequest);

 DescribeTargetHealthRequest healthRequest =
 DescribeTargetHealthRequest.builder()
 .targetGroupArn(tgResponse.targetGroups().get(0).targetGroupArn())
 .build();

 DescribeTargetHealthResponse healthResponse =
 getLoadBalancerClient().describeTargetHealth(healthRequest);
 return healthResponse.targetHealthDescriptions();
 }

 // Gets the HTTP endpoint of the load balancer.
 public String getEndpoint(String lbName) {
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 return res.loadBalancers().get(0).dnsName();
 }

 // Deletes a load balancer.
 public void deleteLoadBalancer(String lbName) {
```

```

 try {
 // Use a waiter to delete the Load Balancer.
 DescribeLoadBalancersResponse res = getLoadBalancerClient()
 .describeLoadBalancers(describe -> describe.names(lbName));
 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()
 .loadBalancerArns(res.loadBalancers().get(0).loadBalancerArn())
 .build();

 getLoadBalancerClient().deleteLoadBalancer(
 builder ->
builder.loadBalancerArn(res.loadBalancers().get(0).loadBalancerArn()));
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancersDeleted(request);
 waiterResponse.matched().response().ifPresent(System.out::println);

 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(lbName + " was deleted.");
 }

 // Deletes the target group.
 public void deleteTargetGroup(String targetGroupName) {
 try {
 DescribeTargetGroupsResponse res = getLoadBalancerClient()
 .describeTargetGroups(describe ->
describe.names(targetGroupName));
 getLoadBalancerClient()
 .deleteTargetGroup(builder ->
builder.targetGroupArn(res.targetGroups().get(0).targetGroupArn()));
 } catch (ElasticLoadBalancingV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println(targetGroupName + " was deleted.");
 }

 // Verify this computer can successfully send a GET request to the load balancer
 // endpoint.
 public boolean verifyLoadBalancerEndpoint(String elbDnsName) throws IOException,
 InterruptedException {

```

```

 boolean success = false;
 int retries = 3;
 CloseableHttpClient httpClient = HttpClients.createDefault();

 // Create an HTTP GET request to the ELB.
 HttpGet httpGet = new HttpGet("http://" + elbDnsName);
 try {
 while ((!success) && (retries > 0)) {
 // Execute the request and get the response.
 HttpResponse response = httpClient.execute(httpGet);
 int statusCode = response.getStatusLine().getStatusCode();
 System.out.println("HTTP Status Code: " + statusCode);
 if (statusCode == 200) {
 success = true;
 } else {
 retries--;
 System.out.println("Got connection error from load balancer endpoint, retrying...");
 TimeUnit.SECONDS.sleep(15);
 }
 }

 } catch (org.apache.http.conn.HttpHostConnectException e) {
 System.out.println(e.getMessage());
 }

 System.out.println("Status.." + success);
 return success;
 }

 /**
 * Creates an Elastic Load Balancing target group. The target group specifies
 * how
 * the load balancer forward requests to instances in the group and how instance
 * health is checked.
 */
 public String createTargetGroup(String protocol, int port, String vpcId, String
targetGroupName) {
 CreateTargetGroupRequest targetGroupRequest =
CreateTargetGroupRequest.builder()
 .healthCheckPath("/healthcheck")
 .healthCheckTimeoutSeconds(5)
 .port(port)
 .vpcId(vpcId)

```

```

 .name(targetGroupName)
 .protocol(protocol)
 .build();

 CreateTargetGroupResponse targetGroupResponse =
getLoadBalancerClient().createTargetGroup(targetGroupRequest);
 String targetGroupArn =
targetGroupResponse.targetGroups().get(0).targetGroupArn();
 String targetGroup =
targetGroupResponse.targetGroups().get(0).targetGroupName();
 System.out.println("The " + targetGroup + " was created with ARN" +
targetGroupArn);
 return targetGroupArn;
}

/*
 * Creates an Elastic Load Balancing load balancer that uses the specified
 * subnets
 * and forwards requests to the specified target group.
 */
public String createLoadBalancer(List<Subnet> subnetIds, String targetGroupARN,
String lbName, int port,
 String protocol) {
 try {
 List<String> subnetIdStrings = subnetIds.stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());

 CreateLoadBalancerRequest balancerRequest =
CreateLoadBalancerRequest.builder()
 .subnets(subnetIdStrings)
 .name(lbName)
 .scheme("internet-facing")
 .build();

 // Create and wait for the load balancer to become available.
 CreateLoadBalancerResponse lsResponse =
getLoadBalancerClient().createLoadBalancer(balancerRequest);
 String lbARN = lsResponse.loadBalancers().get(0).loadBalancerArn();

 ElasticLoadBalancingV2Waiter loadBalancerWaiter =
getLoadBalancerClient().waiter();
 DescribeLoadBalancersRequest request =
DescribeLoadBalancersRequest.builder()

```

```

 .loadBalancerArns(lbARN)
 .build();

 System.out.println("Waiting for Load Balancer " + lbName + " to become
available.");
 WaiterResponse<DescribeLoadBalancersResponse> waiterResponse =
loadBalancerWaiter
 .waitUntilLoadBalancerAvailable(request);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Load Balancer " + lbName + " is available.");

 // Get the DNS name (endpoint) of the load balancer.
 String lbDNSName = lsResponse.loadBalancers().get(0).dnsName();
 System.out.println("*** Load Balancer DNS Name: " + lbDNSName);

 // Create a listener for the load balance.
 Action action = Action.builder()
 .targetGroupArn(targetGroupARN)
 .type("forward")
 .build();

 CreateListenerRequest listenerRequest = CreateListenerRequest.builder()

.loadBalancerArn(lsResponse.loadBalancers().get(0).loadBalancerArn())
 .defaultActions(action)
 .port(port)
 .protocol(protocol)
 .build();

 getLoadBalancerClient().createListener(listenerRequest);
 System.out.println("Created listener to forward traffic from load
balancer " + lbName + " to target group "
 + targetGroupARN);

 // Return the load balancer DNS name.
 return lbDNSName;

 } catch (ElasticLoadBalancingV2Exception e) {
 e.printStackTrace();
 }
 return "";
}
}

```

Create a class that uses DynamoDB to simulate a recommendation service.

```
public class Database {

 private static DynamoDbClient dynamoDbClient;

 public static DynamoDbClient getDynamoDbClient() {
 if (dynamoDbClient == null) {
 dynamoDbClient = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .build();
 }
 return dynamoDbClient;
 }

 // Checks to see if the Amazon DynamoDB table exists.
 private boolean doesTableExist(String tableName) {
 try {
 // Describe the table and catch any exceptions.
 DescribeTableRequest describeTableRequest =
 DescribeTableRequest.builder()
 .tableName(tableName)
 .build();

 getDynamoDbClient().describeTable(describeTableRequest);
 System.out.println("Table '" + tableName + "' exists.");
 return true;

 } catch (ResourceNotFoundException e) {
 System.out.println("Table '" + tableName + "' does not exist.");
 } catch (DynamoDbException e) {
 System.err.println("Error checking table existence: " + e.getMessage());
 }
 return false;
 }

 /**
 * Creates a DynamoDB table to use a recommendation service. The table has a
 * hash key named 'MediaType' that defines the type of media recommended, such
 * as
 * Book or Movie, and a range key named 'ItemId' that, combined with the

```



```

 * MediaType,
 * forms a unique identifier for the recommended item.
 */
public void createTable(String tableName, String fileName) throws IOException {
 // First check to see if the table exists.
 boolean doesExist = doesTableExist(tableName);
 if (!doesExist) {
 DynamoDbWaiter dbWaiter = getDynamoDbClient().waiter();
 CreateTableRequest createTableRequest = CreateTableRequest.builder()
 .tableName(tableName)
 .attributeDefinitions(
 AttributeDefinition.builder()
 .attributeName("MediaType")
 .attributeType(ScalarAttributeType.S)
 .build(),
 AttributeDefinition.builder()
 .attributeName("ItemId")
 .attributeType(ScalarAttributeType.N)
 .build())
 .keySchema(
 KeySchemaElement.builder()
 .attributeName("MediaType")
 .keyType(KeyType.HASH)
 .build(),
 KeySchemaElement.builder()
 .attributeName("ItemId")
 .keyType(KeyType.RANGE)
 .build())
 .provisionedThroughput(
 ProvisionedThroughput.builder()
 .readCapacityUnits(5L)
 .writeCapacityUnits(5L)
 .build())
 .build();

 getDynamoDbClient().createTable(createTableRequest);
 System.out.println("Creating table " + tableName + "...");

 // Wait until the Amazon DynamoDB table is created.
 DescribeTableRequest tableRequest = DescribeTableRequest.builder()
 .tableName(tableName)
 .build();
 }
}

```

```

 WaiterResponse<DescribeTableResponse> waiterResponse =
dbWaiter.waitUntilTableExists(tableRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("Table " + tableName + " created.");

 // Add records to the table.
 populateTable(fileName, tableName);
 }
}

public void deleteTable(String tableName) {
 getDynamoDbClient().deleteTable(table -> table.tableName(tableName));
 System.out.println("Table " + tableName + " deleted.");
}

// Populates the table with data located in a JSON file using the DynamoDB
// enhanced client.
public void populateTable(String fileName, String tableName) throws IOException
{
 DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()
 .dynamoDbClient(getDynamoDbClient())
 .build();
 ObjectMapper objectMapper = new ObjectMapper();
 File jsonFile = new File(fileName);
 JsonNode rootNode = objectMapper.readTree(jsonFile);

 DynamoDbTable<Recommendation> mappedTable = enhancedClient.table(tableName,
 TableSchema.fromBean(Recommendation.class));
 for (JsonNode currentNode : rootNode) {
 String mediaType = currentNode.path("MediaType").path("S").asText();
 int itemId = currentNode.path("ItemId").path("N").asInt();
 String title = currentNode.path("Title").path("S").asText();
 String creator = currentNode.path("Creator").path("S").asText();

 // Create a Recommendation object and set its properties.
 Recommendation rec = new Recommendation();
 rec.setMediaType(mediaType);
 rec.setItemId(itemId);
 rec.setTitle(title);
 rec.setCreator(creator);

 // Put the item into the DynamoDB table.
 mappedTable.putItem(rec); // Add the Recommendation to the list.
 }
}

```

```

 System.out.println("Added all records to the " + tableName);
 }
}

```

Create a class that wraps Systems Manager actions.

```

public class ParameterHelper {

 String tableName = "doc-example-resilient-architecture-table";
 String dyntable = "doc-example-recommendation-service";
 String failureResponse = "doc-example-resilient-architecture-failure-response";
 String healthCheck = "doc-example-resilient-architecture-health-check";

 public void reset() {
 put(dyntable, tableName);
 put(failureResponse, "none");
 put(healthCheck, "shallow");
 }

 public void put(String name, String value) {
 SsmClient ssmClient = SsmClient.builder()
 .region(Region.US_EAST_1)
 .build();

 PutParameterRequest parameterRequest = PutParameterRequest.builder()
 .name(name)
 .value(value)
 .overwrite(true)
 .type("String")
 .build();

 ssmClient.putParameter(parameterRequest);
 System.out.printf("Setting demo parameter %s to '%s'.", name, value);
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AttachLoadBalancerTargetGroups](#)
  - [CreateAutoScalingGroup](#)
  - [CreateInstanceProfile](#)

- [CreateLaunchTemplate](#)
- [CreateListener](#)
- [CreateLoadBalancer](#)
- [CreateTargetGroup](#)
- [DeleteAutoScalingGroup](#)
- [DeleteInstanceProfile](#)
- [DeleteLaunchTemplate](#)
- [DeleteLoadBalancer](#)
- [DeleteTargetGroup](#)
- [DescribeAutoScalingGroups](#)
- [DescribeAvailabilityZones](#)
- [DescribeIamInstanceProfileAssociations](#)
- [DescribeInstances](#)
- [DescribeLoadBalancers](#)
- [DescribeSubnets](#)
- [DescribeTargetGroups](#)
- [DescribeTargetHealth](#)
- [DescribeVpcs](#)
- [RebootInstances](#)
- [ReplaceIamInstanceProfileAssociation](#)
- [TerminateInstanceInAutoScalingGroup](#)
- [UpdateAutoScalingGroup](#)

## Work with the IAM Policy Builder API

The following code example shows how to:

- Create IAM policies by using the object-oriented API.
- Use the IAM Policy Builder API with the IAM service.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The examples use the following imports.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.policybuilder.iam.IamConditionOperator;
import software.amazon.awssdk.policybuilder.iam.IamEffect;
import software.amazon.awssdk.policybuilder.iam.IamPolicy;
import software.amazon.awssdk.policybuilder.iam.IamPolicyWriter;
import software.amazon.awssdk.policybuilder.iam.IamPrincipal;
import software.amazon.awssdk.policybuilder.iam.IamPrincipalType;
import software.amazon.awssdk.policybuilder.iam.IamResource;
import software.amazon.awssdk.policybuilder.iam.IamStatement;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iam.IamClient;
import software.amazon.awssdk.services.iam.model.GetPolicyResponse;
import software.amazon.awssdk.services.iam.model.GetPolicyVersionResponse;
import software.amazon.awssdk.services.sts.StsClient;

import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.List;
```

Create a time-based policy.

```
public String timeBasedPolicyExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:GetItem")
 .addResource(IamResource.ALL)
 .addCondition(b1 -> b1
```

```

 .operator(IamConditionOperator.DATE_GREATER_THAN)

 .key("aws:CurrentTime")

 .value("2020-04-01T00:00:00Z"))
 .addCondition(b1 -> b1

 .operator(IamConditionOperator.DATE_LESS_THAN)

 .key("aws:CurrentTime")

 .value("2020-06-30T23:59:59Z"))
 .build();

 // Use an IamPolicyWriter to write out the JSON string to a more
readable
 // format.
 return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true)
 .build());
 }

```

Create a policy with multiple conditions.

```

 public String multipleConditionsExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:GetItem")
 .addAction("dynamodb:BatchGetItem")
 .addAction("dynamodb:Query")
 .addAction("dynamodb:PutItem")
 .addAction("dynamodb:UpdateItem")
 .addAction("dynamodb>DeleteItem")

 .addAction("dynamodb:BatchWriteItem")

 .addResource("arn:aws:dynamodb:*:*:table/table-name")

 .addConditions(IamConditionOperator.STRING_EQUALS

```

```

 .addPrefix("ForAllValues:"),

 "dynamodb:Attributes",

 List.of("column-
name1", "column-name2", "column-name3"))

 .addCondition(b1 -> b1

 .operator(IamConditionOperator.STRING_EQUALS

 .addSuffix("IfExists"))

 .key("dynamodb:Select")

 .value("SPECIFIC_ATTRIBUTES"))))

 .build();

 return policy.toJson(IamPolicyWriter.builder()

 .prettyPrint(true).build());

 }

```

## Use principals in a policy.

```

 public String specifyPrincipalsExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.DENY)
 .addAction("s3:*")
 .addPrincipal(IamPrincipal.ALL)
 .addResource("arn:aws:s3:::amzn-s3-
demo-bucket/*")

 .addResource("arn:aws:s3:::amzn-s3-
demo-bucket")

 .addCondition(b1 -> b1

 .operator(IamConditionOperator.ARN_NOT_EQUALS)

 .key("aws:PrincipalArn")

 .value("arn:aws:iam::444455556666:user/user-name"))))

 .build();

 return policy.toJson(IamPolicyWriter.builder()

```

```
 .prettyPrint(true).build());
 }
```

### Allow cross-account access.

```
public String allowCrossAccountAccessExample() {
 IamPolicy policy = IamPolicy.builder()
 .addStatement(b -> b
 .effect(IamEffect.ALLOW)
 .addPrincipal(IamPrincipalType.AWS,
 "111122223333")
 .addAction("s3:PutObject")
 .addResource("arn:aws:s3:::amzn-s3-
demo-bucket/*")
 .addCondition(b1 -> b1
 .operator(IamConditionOperator.STRING_EQUALS)
 .key("s3:x-amz-acl")
 .value("bucket-
owner-full-control"))))
 .build();
 return policy.toJson(IamPolicyWriter.builder()
 .prettyPrint(true).build());
}
```

### Build and upload an IamPolicy.

```
public String createAndUploadPolicyExample(IamClient iam, String accountID,
String policyName) {
 // Build the policy.
 IamPolicy policy = IamPolicy.builder() // 'version' defaults to
 "2012-10-17".
 .addStatement(IamStatement.builder()
 .effect(IamEffect.ALLOW)
 .addAction("dynamodb:PutItem")
 .addResource("arn:aws:dynamodb:us-
east-1:" + accountID
 + ":table/
exampleTableName")
 .build())
 .build();
}
```



```

 // Upload the policy.
 iam.createPolicy(r ->
r.policyName(policyName).policyDocument(policy.toJson()));
 return
policy.toJson(IamPolicyWriter.builder().prettyPrint(true).build());
 }

```

## Download and work with an IamPolicy.

```

 public String createNewBasedOnExistingPolicyExample(IamClient iam, String
accountID, String policyName,
 String newPolicyName) {

 String policyArn = "arn:aws:iam::" + accountID + ":policy/" +
policyName;
 GetPolicyResponse getPolicyResponse = iam.getPolicy(r ->
r.policyArn(policyArn));

 String policyVersion =
getPolicyResponse.policy().defaultVersionId();
 GetPolicyVersionResponse getPolicyVersionResponse = iam
 .getPolicyVersion(r ->
r.policyArn(policyArn).versionId(policyVersion));

 // Create an IamPolicy instance from the JSON string returned from
IAM.
 String decodedPolicy =
URLDecoder.decode(getPolicyVersionResponse.policyVersion().document(),
 StandardCharsets.UTF_8);
 IamPolicy policy = IamPolicy.fromJson(decodedPolicy);

 /*
 * All IamPolicy components are immutable, so use the copy method
that creates a
 * new instance that
 * can be altered in the same method call.
 *
 * Add the ability to get an item from DynamoDB as an additional
action.
 */
 IamStatement newStatement = policy.statements().get(0).copy(s ->
s.addAction("dynamodb:GetItem"));
 }

```

```
// Create a new statement that replaces the original statement.
IamPolicy newPolicy = policy.copy(p ->
p.statements(Arrays.asList(newStatement)));

// Upload the new policy. IAM now has both policies.
iam.createPolicy(r -> r.policyName(newPolicyName)
 .policyDocument(newPolicy.toJson()));

return
newPolicy.toJson(IamPolicyWriter.builder().prettyPrint(true).build());
}
```

- For more information, see [AWS SDK for Java 2.x Developer Guide](#).
- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreatePolicy](#)
  - [GetPolicy](#)
  - [GetPolicyVersion](#)

## Amazon Inspector examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Inspector.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

# Get started

## Hello

The following code example shows how to get started using .

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloInspector {
 private static final Logger logger =
 LoggerFactory.getLogger(HelloInspector.class);

 public static void main(String[] args) {
 logger.info("Hello Amazon Inspector!");

 try (Inspector2Client inspectorClient = Inspector2Client.builder().build())
 {

 logger.info("Listing member accounts for this Inspector administrator
account...");
 listMembers(inspectorClient);

 logger.info("The Hello Inspector example completed successfully.");

 } catch (Inspector2Exception e) {
 logger.error("Error: {}", e.getMessage());
 logger.info("Troubleshooting:");
 logger.info("1. Verify AWS credentials are configured");
 }
 }
}
```

```

 logger.info("2. Check IAM permissions for Inspector2");
 logger.info("3. Ensure Inspector2 is enabled in your account");
 logger.info("4. Verify you're using a supported region");
 }
}

/**
 * Lists all member accounts associated with the current Inspector administrator
 * account.
 *
 * @param inspectorClient The Inspector2Client used to interact with AWS
 * Inspector.
 */
public static void listMembers(Inspector2Client inspectorClient) {
 try {
 ListMembersRequest request = ListMembersRequest.builder()
 .maxResults(50) // optional: limit results
 .build();

 ListMembersResponse response = inspectorClient.listMembers(request);
 List<Member> members = response.members();

 if (members == null || members.isEmpty()) {
 logger.info("No member accounts found for this Inspector
administrator account.");
 return;
 }

 logger.info("Found {} member account(s):", members.size());
 for (Member member : members) {
 logger.info(" - Account ID: {}, Status: {}",
 member.accountId(),
 member.relationshipStatusAsString());
 }

 } catch (Inspector2Exception e) {
 logger.error("Failed to list members: {}",
 e.awsErrorDetails().errorMessage());
 }
}
}

```

- For API details, see [ListMembers](#) in *AWS SDK for Java 2.x API Reference*.

# Basics

## Learn the basics

The following code example shows how to:

- Check Inspector account status.
- Ensure Inspector is enabled.
- Analyze security findings.
- Check scan coverage.
- Create a findings filter.
- List existing filters.
- Check usage and costs.
- Get coverage statistics.
- Delete a filter.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating features.

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class InspectorScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");
```

```
private static final Logger logger =
LoggerFactory.getLogger(InspectorScenario.class);
private static final Scanner scanner = new Scanner(System.in);

public static void main(String[] args) {
 InspectorActions inspectorActions = new InspectorActions();
 logger.info("Amazon Inspector Basics Scenario");

 logger.info("""
 Amazon Inspector is a security assessment service provided by
 Amazon Web Services (AWS) that helps
 improve the security and compliance of applications deployed on
 AWS. It automatically assesses
 applications for vulnerabilities or deviations from best
 practices. By leveraging Amazon Inspector,
 users can gain insights into the overall security state of their
 application and identify potential
 security risks.

 This service operates by conducting both network and host-based
 assessments, allowing it to detect a
 wide range of security issues, including those related to
 operating systems, network configurations,
 and application dependencies.
 """);

 waitForInputToContinue();

 try {
 runScenario(inspectorActions);

 logger.info("");
 logger.info("Scenario completed successfully!");
 logger.info("");
 logger.info("What you learned:");
 logger.info(" - How to check Inspector account status");
 logger.info(" - How to enable Inspector");
 logger.info(" - How to list and analyze findings");
 logger.info(" - How to check coverage information");
 logger.info(" - How to create and manage filters");
 logger.info(" - How to track usage and costs");
 logger.info(" - How to clean up resources");
 logger.info("");
 }
```

```

 } catch (Exception ex) {
 logger.error("Scenario failed due to unexpected error: {}",
ex.getMessage(), ex);

 } finally {
 scanner.close();
 logger.info("Exiting...");
 }
 }

 /**
 * Runs the Inspector scenario in a step-by-step sequence.
 *
 * All InspectorActions methods are asynchronous and return CompletableFutures.
 * Each step ends with .join(). Any async exception thrown during .join() will
bubble up
 *
 */
 public static void runScenario(InspectorActions actions) {
 String filterArn = null;
 boolean inspectorEnabled = false;

 try {
 // Step 1
 logger.info(DASHES);
 logger.info("Step 1: Checking Inspector account status...");
 String status = actions.getAccountStatusAsync().join();
 logger.info(status);
 waitForInputToContinue();

 // Step 2
 logger.info(DASHES);
 logger.info("Step 2: Enabling Inspector...");
 String message = actions.enableInspectorAsync(null).join();
 logger.info(message);
 inspectorEnabled = true; // track that Inspector was enabled
 waitForInputToContinue();

 // Step 3
 logger.info(DASHES);
 logger.info("Step 3: Listing LOW severity findings...");

 // Call the service method

```

```
List<String> allFindings =
actions.listLowSeverityFindingsAsync().join();

if (!allFindings.isEmpty()) {
 // Only proceed if there are findings
 String lastArn = allFindings.get(allFindings.size() - 1);
 logger.info("Look up details on: {}", lastArn);
 waitForInputToContinue();
 String details = actions.getFindingDetailsAsync(lastArn).join();
 logger.info(details);
} else {
 logger.info("No LOW severity findings found.");
}

waitForInputToContinue();

// Step 4
logger.info(DASHES);
logger.info("Step 4: Listing coverage...");
String coverage = actions.listCoverageAsync(5).join();
logger.info(coverage);
waitForInputToContinue();

// Step 5
logger.info(DASHES);
logger.info("Step 5: Creating filter...");
String filterName = "suppress-low-" + System.currentTimeMillis();
filterArn = actions.createLowSeverityFilterAsync(filterName, "Suppress
low severity findings").join();
logger.info("Created filter: {}", filterArn);
waitForInputToContinue();

// Step 6
logger.info(DASHES);
logger.info("Step 6: Listing filters...");
String filters = actions.listFiltersAsync(10).join();
logger.info(filters);
waitForInputToContinue();

// Step 7
logger.info(DASHES);
logger.info("Step 7: Usage totals...");
String usage = actions.listUsageTotalsAsync(null, 10).join();
logger.info(usage);
```



```

 waitForInputToContinue();

 // Step 8
 logger.info(DASHES);
 logger.info("Step 8: Coverage statistics...");
 String stats = actions.listCoverageStatisticsAsync().join();
 logger.info(stats);
 waitForInputToContinue();

 // Step 9
 logger.info(DASHES);
 logger.info("Step 9: Delete filter?");
 logger.info("Filter ARN: {}", filterArn);
 logger.info("Delete the filter and disable Inspector? (y/n)");

 if (scanner.nextLine().trim().equalsIgnoreCase("y")) {
 actions.deleteFilterAsync(filterArn).join();
 logger.info("Filter deleted.");
 String disableMsg = actions.disableInspectorAsync(null).join();
 logger.info(disableMsg);
 inspectorEnabled = false; // track that Inspector was disabled
 }

 waitForInputToContinue();

 } catch (Exception ex) {
 logger.error("Scenario encountered an error: {}", ex.getMessage(), ex);
 // Rethrow the exception
 throw ex;
 } finally {
 // Cleanup in case of an exception
 if (filterArn != null) {
 try {
 actions.deleteFilterAsync(filterArn).join();
 logger.info("Cleanup: Filter deleted.");
 } catch (Exception e) {
 logger.warn("Failed to delete filter during cleanup: {}",
e.getMessage(), e);
 }
 }

 if (inspectorEnabled) {
 try {

```

```

 actions.disableInspectorAsync(null).join();
 logger.info("Cleanup: Inspector disabled.");
 } catch (Exception e) {
 logger.warn("Failed to disable Inspector during cleanup: {}",
e.getMessage(), e);
 }
}

// Utility Method
private static void waitForInputToContinue() {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' to continue:");
 String input = scanner.nextLine().trim();
 if (input.equalsIgnoreCase("c")) break;
 logger.info("Invalid input, try again.");
 }
}
}

```

A wrapper class for SDK methods.

```

public class InspectorActions {
 private static Inspector2AsyncClient inspectorAsyncClient;
 private static final Logger logger =
LoggerFactory.getLogger(InspectorActions.class);

 private static Inspector2AsyncClient getAsyncClient() {
 if (inspectorAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))

```

```

 .retryStrategy(RetryMode.STANDARD)
 .build();

 inspectorAsyncClient = Inspector2AsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return inspectorAsyncClient;
}

/**
 * Enables AWS Inspector for the provided account(s) and default resource types.
 *
 * @param accountIds Optional list of AWS account IDs.
 */
public CompletableFuture<String> enableInspectorAsync(List<String> accountIds) {

 // The resource types to enable.
 List<ResourceScanType> resourceTypes = List.of(
 ResourceScanType.EC2,
 ResourceScanType.ECR,
 ResourceScanType.LAMBDA,
 ResourceScanType.LAMBDA_CODE
);

 // Build the request.
 EnableRequest.Builder requestBuilder = EnableRequest.builder()
 .resourceTypes(resourceTypes);

 if (accountIds != null && !accountIds.isEmpty()) {
 requestBuilder.accountIds(accountIds);
 }

 EnableRequest request = requestBuilder.build();
 return getAsyncClient().enable(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Inspector may already be enabled for this
account: %s".formatted(cause.getMessage()),

```

```

 cause
);

 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;
 throw new CompletionException(
 "AWS Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to enable Inspector:
%s".formatted(exception.getMessage()),
 exception
);
}

}))
.thenApply(response -> {
 StringBuilder summary = new StringBuilder("Enable results:\n");

 if (response.accounts() == null ||
response.accounts().isEmpty()) {
 summary.append("Inspector may already be enabled for all
target accounts.");
 return summary.toString();
 }

 for (Account account : response.accounts()) {
 String accountId = account.accountId() != null ?
account.accountId() : "Unknown";
 String status = account.status() != null ?
account.statusAsString() : "Unknown";
 summary.append(" • Account: ").append(accountId)
 .append(" # Status: ").append(status).append("\n");
 }

 return summary.toString();
});
}

```

```

/**
 * Retrieves and prints the coverage statistics using a paginator.
 */
public CompletableFuture<String> listCoverageStatisticsAsync() {
 ListCoverageStatisticsRequest request =
ListCoverageStatisticsRequest.builder()
 .build();

 return getAsyncClient().listCoverageStatistics(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();

 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Validation error listing coverage statistics:
%s".formatted(cause.getMessage()),
 cause
);
 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;

 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 e
);
 }

 throw new CompletionException(
 "Unexpected error listing coverage statistics:
%s".formatted(exception.getMessage()),
 exception
);
 }
 })
 .thenApply(response -> {
 List<Counts> countsList = response.countsByGroup();
 StringBuilder sb = new StringBuilder();

 if (countsList == null || countsList.isEmpty()) {

```

```

 sb.append("No coverage statistics available.\n");
 return sb.toString();
 }

 sb.append("Coverage Statistics:\n");

 for (Counts c : countsList) {
 sb.append(" Group: ").append(c.groupKey()).append("\n")
 .append(" Total Count:
").append(c.count()).append("\n\n");
 }

 return sb.toString();
 });
}

/**
 * Asynchronously lists Inspector2 usage totals using a paginator.
 *
 * @param accountIds optional list of account IDs
 * @param maxResults maximum results per page
 * @return CompletableFuture completed with formatted summary text
 */
public CompletableFuture<String> listUsageTotalsAsync(
 List<String> accountIds,
 int maxResults) {

 logger.info("Starting usage totals paginator...");

 ListUsageTotalsRequest.Builder builder = ListUsageTotalsRequest.builder()
 .maxResults(maxResults);

 if (accountIds != null && !accountIds.isEmpty()) {
 builder.accountIds(accountIds);
 }

 ListUsageTotalsRequest request = builder.build();
 ListUsageTotalsPublisher paginator =
getAsyncClient().listUsageTotalsPaginator(request);
 StringBuilder summaryBuilder = new StringBuilder();

 return paginator.subscribe(response -> {
 if (response.totals() != null && !response.totals().isEmpty()) {

```

```

 response.totals().forEach(total -> {
 if (total.usage() != null) {
 total.usage().forEach(usage -> {
 logger.info("Usage: {} = {}",
usage.typeAsString(), usage.total());
 summaryBuilder.append(usage.typeAsString())
 .append(": ")
 .append(usage.total())
 .append("\n");
 });
 }
 });
 } else {
 logger.info("Page contained no usage totals.");
 }
}).thenRun(() -> logger.info("Successfully listed usage totals.")).thenApply(v -> {
 String summary = summaryBuilder.toString();
 return summary.isEmpty() ? "No usage totals found." : summary;
}).exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing usage totals:
%s".formatted(ve.getMessage()),
 ve
);
 }

 throw new CompletionException("Failed to list usage totals",
cause);
});
}

/**
 * Retrieves the account status using the Inspector2Client.
 */
public CompletableFuture<String> getAccountStatusAsync() {
 BatchGetAccountStatusRequest request =
BatchGetAccountStatusRequest.builder()
 .accountIds(Collections.emptyList())

```

```

 .build();

 return getAsyncClient().batchGetAccountStatus(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof AccessDeniedException) {
 throw new CompletionException(
 "You do not have sufficient access:
%s".formatted(cause.getMessage()),
 cause
);
 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;

 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 e
);
 }

 throw new CompletionException(
 "Unexpected error getting account status:
%s".formatted(exception.getMessage()),
 exception
);
 }
 })
 .thenApply(response -> {

 StringBuilder sb = new StringBuilder();
 List<AccountState> accounts = response.accounts();

 if (accounts == null || accounts.isEmpty()) {
 sb.append("No account status returned.\n");
 return sb.toString();
 }

 sb.append("Inspector Account Status:\n");
 for (AccountState account : accounts) {

```



```

 String accountId = account.accountId() != null
 ? account.accountId()
 : "Unknown";

 sb.append(" Account ID: ").append(accountId).append("\n");

 // Overall account state
 if (account.state() != null && account.state().status() !=
null) {
 sb.append(" Overall State: ")
 .append(account.state().status())
 .append("\n");
 } else {
 sb.append(" Overall State: Unknown\n");
 }

 // Resource state (only status available)
 ResourceState resources = account.resourceState();
 if (resources != null) {
 sb.append(" Resource Status: available\n");
 }

 sb.append("\n");
 }

 return sb.toString();
});
}

/**
 * Asynchronously lists Inspector2 filters using a paginator.
 *
 * @param maxResults maximum filters per page (nullable)
 * @return CompletableFuture completed with summary text
 */
public CompletableFuture<String> listFiltersAsync(Integer maxResults) {
 logger.info("Starting async filters paginator...");

 ListFiltersRequest.Builder builder = ListFiltersRequest.builder();
 if (maxResults != null) {
 builder.maxResults(maxResults);
 }
}

```

```

 ListFiltersRequest request = builder.build();

 // Paginator from SDK
 ListFiltersPublisher paginator =
getAsyncClient().listFiltersPaginator(request);
 StringBuilder collectedFilterIds = new StringBuilder();

 return paginator.subscribe(response -> {
 response.filters().forEach(filter -> {
 logger.info("Filter: " + filter.arn());
 collectedFilterIds.append(filter.arn()).append("\n");
 });
 }).thenApply(v -> {
 String result = collectedFilterIds.toString();
 logger.info("Successfully listed all filters.");
 return result.isEmpty() ? "No filters found." : result;
 }).exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing filters:
%s".formatted(ve.getMessage()),
 ve
);
 }

 throw new RuntimeException("Failed to list filters", ex);
 });
 }

 /**
 * Creates a new LOW severity filter in AWS Inspector2 to suppress findings.
 *
 * @param filterName the name of the filter to create
 * @param description a descriptive string explaining the purpose of the filter
 * @return a CompletableFuture that completes with the ARN of the created filter
 * @throws CompletionException wraps any validation, Inspector2 service, or
 unexpected errors
 */
 public CompletableFuture<String> createLowSeverityFilterAsync(
 String filterName,

```

```

 String description) {

 // Define a filter to match LOW severity findings.
 StringFilter severityFilter = StringFilter.builder()
 .value(Severity.LOW.toString())
 .comparison(StringComparison.EQUALS)
 .build();

 // Create filter criteria.
 FilterCriteria filterCriteria = FilterCriteria.builder()
 .severity(Collections.singletonList(severityFilter))
 .build();

 // Build the filter creation request.
 CreateFilterRequest request = CreateFilterRequest.builder()
 .name(filterName)
 .filterCriteria(filterCriteria)
 .action(FilterAction.SUPPRESS)
 .description(description)
 .build();

 return getAsyncClient().createFilter(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error creating filter:
%s".formatted(ve.getMessage()),
 ve
);
 }

 if (cause instanceof Inspector2Exception e) {
 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 e
);
 }

 // Unexpected async error

```

```

 throw new CompletionException(
 "Unexpected error creating filter:
%s".formatted(exception.getMessage()),
 exception
);
 }
})
// Extract and return the ARN of the created filter.
.thenApply(CreateFilterResponse::arn);
}

/**
 * Lists all AWS Inspector findings of LOW severity asynchronously.
 *
 * @return CompletableFuture containing a List of finding ARNs.
 * Returns an empty list if no LOW severity findings are found.
 */
public CompletableFuture<ArrayList<String>> listLowSeverityFindingsAsync() {
 logger.info("Starting async LOW severity findings paginator...");

 // Build a filter criteria for LOW severity.
 StringFilter severityFilter = StringFilter.builder()
 .value(Severity.LOW.toString())
 .comparison(StringComparison.EQUALS)
 .build();

 FilterCriteria filterCriteria = FilterCriteria.builder()
 .severity(Collections.singletonList(severityFilter))
 .build();

 // Build the request.
 ListFindingsRequest request = ListFindingsRequest.builder()
 .filterCriteria(filterCriteria)
 .build();

 ListFindingsPublisher paginator =
getAsyncClient().listFindingsPaginator(request);
 List<String> allArns = Collections.synchronizedList(new ArrayList<>());

 return paginator.subscribe(response -> {
 if (response.findings() != null && !
response.findings().isEmpty()) {
 response.findings().forEach(finding -> {

```

```

 logger.info("Finding ARN: {}", finding.findingArn());
 allArns.add(finding.findingArn());
 });
 } else {
 logger.info("Page contained no findings.");
 }
})
.thenRun(() -> logger.info("Successfully listed all LOW severity
findings."))
.thenApply(v -> new ArrayList<>(allArns)) // Return list instead of
a formatted string
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;
 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing LOW severity findings:
%s".formatted(ve.getMessage()),
 ve
);
 }
 throw new RuntimeException("Failed to list LOW severity
findings", ex);
});
}

/**
 * Lists AWS Inspector2 coverage details for scanned resources using a
paginator.
 *
 * @param maxResults Maximum number of resources to return.
 */
public CompletableFuture<String> listCoverageAsync(int maxResults) {
 ListCoverageRequest initialRequest = ListCoverageRequest.builder()
 .maxResults(maxResults)
 .build();

 ListCoveragePublisher paginator =
getAsyncClient().listCoveragePaginator(initialRequest);
 StringBuilder summary = new StringBuilder();

 return paginator.subscribe(response -> {
 List<CoveredResource> coveredResources = response.coveredResources();

```

```

 if (coveredResources == null || coveredResources.isEmpty()) {
 summary.append("No coverage information available for this page.
\n");
 return;
 }

 Map<String, List<CoveredResource>> byType = coveredResources.stream()

.collect(Collectors.groupingBy(CoveredResource::resourceTypeAsString));

 byType.forEach((type, list) ->
 summary.append(" ").append(type)
 .append(": ").append(list.size())
 .append(" resource(s)\n")
);

 // Include up to 3 sample resources per page
 for (int i = 0; i < Math.min(coveredResources.size(), 3); i++) {
 CoveredResource r = coveredResources.get(i);
 summary.append(" - ").append(r.resourceTypeAsString())
 .append(": ").append(r.resourceId()).append("\n");
 summary.append(" Scan Type:
").append(r.scanTypeAsString()).append("\n");
 if (r.scanStatus() != null) {
 summary.append(" Status:
").append(r.scanStatus().statusCodeAsString()).append("\n");
 }
 if (r.accountId() != null) {
 summary.append(" Account ID:
").append(r.accountId()).append("\n");
 }
 summary.append("\n");
 }

 }).thenApply(v -> {
 if (summary.length() == 0) {
 return "No coverage information found across all pages.";
 } else {
 return "Coverage Information:\n" + summary.toString();
 }
 }).exceptionally(ex -> {
 Throwable cause = ex.getCause();
 if (cause instanceof ValidationException) {

```

```

 throw new CompletionException(
 "Validation error listing coverage: " + cause.getMessage(),
cause);
 } else if (cause instanceof Inspector2Exception e) {
 throw new CompletionException(
 "Inspector2 service error: " +
e.awsErrorDetails().errorMessage(), e);
 }
 throw new CompletionException("Unexpected error listing coverage: " +
ex.getMessage(), ex);
 });
}

/**
 * Deletes an AWS Inspector2 filter.
 *
 * @param filterARN The ARN of the filter to delete.
 */
public CompletableFuture<Void> deleteFilterAsync(String filterARN) {
 return getAsyncClient().deleteFilter(
 DeleteFilterRequest.builder()
 .arn(filterARN)
 .build()
)
 .handle((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ResourceNotFoundException rnfe) {
 String msg = "Filter not found for ARN:
%s".formatted(filterARN);
 logger.warn(msg, rnfe);
 throw new CompletionException(msg, rnfe);
 }

 throw new RuntimeException("Failed to delete the filter: " +
cause, cause);
 }
 return null;
 });
}

```

```

/**
 * Retrieves detailed information about a specific AWS Inspector2 finding
 * asynchronously.
 *
 * @param findingArn The ARN of the finding to look up.
 * @return A {@link CompletableFuture} that, when completed, provides a
 * formatted string
 * containing all available details for the finding.
 * @throws RuntimeException if the async call to Inspector2 fails.
 */
public CompletableFuture<String> getFindingDetailsAsync(String findingArn) {
 BatchGetFindingDetailsRequest request =
BatchGetFindingDetailsRequest.builder()
 .findingArns(findingArn)
 .build();

 return getAsyncClient().batchGetFindingDetails(request)
 .thenApply(response -> {
 if (response.findingDetails() == null ||
response.findingDetails().isEmpty()) {
 return String.format("No details found for ARN: ",
findingArn);
 }

 StringBuilder sb = new StringBuilder();
 response.findingDetails().forEach(detail -> {
 sb.append("Finding ARN:
").append(detail.findingArn()).append("\n")
 .append("Risk Score:
").append(detail.riskScore()).append("\n");

 // ExploitObserved timings
 if (detail.exploitObserved() != null) {
 sb.append("Exploit First Seen:
").append(detail.exploitObserved().firstSeen()).append("\n")
 .append("Exploit Last Seen:
").append(detail.exploitObserved().lastSeen()).append("\n");
 }

 // Reference URLs
 if (detail.hasReferenceUrls()) {
 sb.append("Reference URLs:\n");
 detail.referenceUrls().forEach(url -> sb.append(" •
").append(url).append("\n"));
 }
 });
 });
}

```



```

 }

 // Tools
 if (detail.hasTools()) {
 sb.append("Tools:\n");
 detail.tools().forEach(tool -> sb.append(" •
").append(tool).append("\n"));
 }

 // TTPs
 if (detail.hasTtps()) {
 sb.append("TTPs:\n");
 detail.ttps().forEach(ttp -> sb.append(" •
").append(ttp).append("\n"));
 }

 // CWEs
 if (detail.hasCwes()) {
 sb.append("CWEs:\n");
 detail.cwes().forEach(cwe -> sb.append(" •
").append(cwe).append("\n"));
 }

 // Evidence
 if (detail.hasEvidences()) {
 sb.append("Evidence:\n");
 detail.evidences().forEach(ev -> {
 sb.append(" - Severity:
").append(ev.severity()).append("\n");

 });
 }

 sb.append("\n");
 });

 return sb.toString();
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ResourceNotFoundException rnfe) {
 return "Finding not found: %s".formatted(findingArn);
 }
}

```

```

 // Fallback for other exceptions
 throw new RuntimeException("Failed to get finding details for
ARN: " + findingArn, cause);
 });
}

/**
 * Asynchronously disables AWS Inspector for the specified accounts and resource
types.
 *
 * @param accountIds a {@link List} of AWS account IDs for which to disable
Inspector;
 *
 * may be {@code null} or empty to target the current account
 * @return a {@link CompletableFuture} that, when completed, returns a {@link
String}
 *
 * summarizing the disable results for each account
 * @throws CompletionException if the disable operation fails due to validation
errors,
 *
 * service errors, or other exceptions
 * @see <a href="https://docs.aws.amazon.com/inspector/latest/APIReference/
API_Disable.html">
 * AWS Inspector2 Disable API
 */
public CompletableFuture<String> disableInspectorAsync(List<String> accountIds)
{
 // The resource types to disable.
 List<ResourceScanType> resourceTypes = List.of(
 ResourceScanType.EC2,
 ResourceScanType.ECR,
 ResourceScanType.LAMBDA,
 ResourceScanType.LAMBDA_CODE
);

 // Build the request.
 DisableRequest.Builder requestBuilder = DisableRequest.builder()
 .resourceTypes(resourceTypes);

 if (accountIds != null && !accountIds.isEmpty()) {
 requestBuilder.accountIds(accountIds);
 }
}

```

```

 DisableRequest request = requestBuilder.build();

 return getAsyncClient().disable(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Inspector may already be disabled for this
account: %s".formatted(cause.getMessage()),
 cause
);
 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;
 throw new CompletionException(
 "AWS Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to disable Inspector:
%s".formatted(exception.getMessage()),
 exception
);
 }
 })
 .thenApply(response -> {
 StringBuilder summary = new StringBuilder("Disable results:\n");

 if (response.accounts() == null ||
response.accounts().isEmpty()) {
 summary.append("Inspector may already be disabled for all
target accounts.");
 return summary.toString();
 }

 for (Account account : response.accounts()) {
 String accountId = account.accountId() != null ?
account.accountId() : "Unknown";

```

```
 String status = account.status() != null ?
account.statusAsString() : "Unknown";
 summary.append(" • Account: ").append(accountId)
 .append(" # Status: ").append(status).append("\n");
 }

 return summary.toString();
});
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [BatchGetAccountStatus](#)
- [BatchGetFindingDetails](#)
- [CreateFilter](#)
- [DeleteFilter](#)
- [Enable](#)
- [ListCoverage](#)
- [ListCoverageStatistics](#)
- [ListFilters](#)
- [ListFindings](#)
- [ListUsageTotals](#)

## Actions

### BatchGetAccountStatus

The following code example shows how to use BatchGetAccountStatus.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the account status using the Inspector2Client.
 */
public CompletableFuture<String> getAccountStatusAsync() {
 BatchGetAccountStatusRequest request =
BatchGetAccountStatusRequest.builder()
 .accountIds(Collections.emptyList())
 .build();

 return getAsyncClient().batchGetAccountStatus(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof AccessDeniedException) {
 throw new CompletionException(
 "You do not have sufficient access:
%s".formatted(cause.getMessage()),
 cause
);
 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;

 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 e
);
 }

 throw new CompletionException(
 "Unexpected error getting account status:
%s".formatted(exception.getMessage()),
 exception
);
 }
 })
 .thenApply(response -> {

 StringBuilder sb = new StringBuilder();

```

```

 List<AccountState> accounts = response.accounts();

 if (accounts == null || accounts.isEmpty()) {
 sb.append("No account status returned.\n");
 return sb.toString();
 }

 sb.append("Inspector Account Status:\n");
 for (AccountState account : accounts) {

 String accountId = account.accountId() != null
 ? account.accountId()
 : "Unknown";

 sb.append(" Account ID: ").append(accountId).append("\n");

 // Overall account state
 if (account.state() != null && account.state().status() !=
null) {

 sb.append(" Overall State: ")
 .append(account.state().status())
 .append("\n");
 } else {
 sb.append(" Overall State: Unknown\n");
 }

 // Resource state (only status available)
 ResourceState resources = account.resourceState();
 if (resources != null) {
 sb.append(" Resource Status: available\n");
 }

 sb.append("\n");
 }

 return sb.toString();
 });
}

```

- For API details, see [BatchGetAccountStatus](#) in *AWS SDK for Java 2.x API Reference*.

## BatchGetFindingDetails

The following code example shows how to use BatchGetFindingDetails.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves detailed information about a specific AWS Inspector2 finding
 * asynchronously.
 *
 * @param findingArn The ARN of the finding to look up.
 * @return A {@link CompletableFuture} that, when completed, provides a
 * formatted string
 * containing all available details for the finding.
 * @throws RuntimeException if the async call to Inspector2 fails.
 */
public CompletableFuture<String> getFindingDetailsAsync(String findingArn) {
 BatchGetFindingDetailsRequest request =
BatchGetFindingDetailsRequest.builder()
 .findingArns(findingArn)
 .build();

 return getAsyncClient().batchGetFindingDetails(request)
 .thenApply(response -> {
 if (response.findingDetails() == null ||
response.findingDetails().isEmpty()) {
 return String.format("No details found for ARN: ",
findingArn);
 }

 StringBuilder sb = new StringBuilder();
 response.findingDetails().forEach(detail -> {
 sb.append("Finding ARN:
").append(detail.findingArn()).append("\n")
 .append("Risk Score:
").append(detail.riskScore()).append("\n");
 });
 });
}
```

```

 // ExploitObserved timings
 if (detail.exploitObserved() != null) {
 sb.append("Exploit First Seen:
").append(detail.exploitObserved().firstSeen()).append("\n")
 .append("Exploit Last Seen:
").append(detail.exploitObserved().lastSeen()).append("\n");
 }

 // Reference URLs
 if (detail.hasReferenceUrls()) {
 sb.append("Reference URLs:\n");
 detail.referenceUrls().forEach(url -> sb.append(" •
").append(url).append("\n"));
 }

 // Tools
 if (detail.hasTools()) {
 sb.append("Tools:\n");
 detail.tools().forEach(tool -> sb.append(" •
").append(tool).append("\n"));
 }

 // TTPs
 if (detail.hasTtps()) {
 sb.append("TTPs:\n");
 detail.ttps().forEach(ttp -> sb.append(" •
").append(ttp).append("\n"));
 }

 // CWEs
 if (detail.hasCwes()) {
 sb.append("CWEs:\n");
 detail.cwes().forEach(cwe -> sb.append(" •
").append(cwe).append("\n"));
 }

 // Evidence
 if (detail.hasEvidences()) {
 sb.append("Evidence:\n");
 detail.evidences().forEach(ev -> {
 sb.append(" - Severity:
").append(ev.severity()).append("\n");
 });
 }
 });

```



```

 }

 sb.append("\n");
 });

 return sb.toString();
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ResourceNotFoundException rnfe) {
 return "Finding not found: %s".formatted(findingArn);
 }

 // Fallback for other exceptions
 throw new RuntimeException("Failed to get finding details for
ARN: " + findingArn, cause);
});
}

```

- For API details, see [BatchGetFindingDetails](#) in *AWS SDK for Java 2.x API Reference*.

## CreateFilter

The following code example shows how to use `CreateFilter`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new LOW severity filter in AWS Inspector2 to suppress findings.
 *
 * @param filterName the name of the filter to create
 * @param description a descriptive string explaining the purpose of the filter
 * @return a CompletableFuture that completes with the ARN of the created filter

```

```

 * @throws CompletionException wraps any validation, Inspector2 service, or
 unexpected errors
 */
 public CompletableFuture<String> createLowSeverityFilterAsync(
 String filterName,
 String description) {

 // Define a filter to match LOW severity findings.
 StringFilter severityFilter = StringFilter.builder()
 .value(Severity.LOW.toString())
 .comparison(StringComparison.EQUALS)
 .build();

 // Create filter criteria.
 FilterCriteria filterCriteria = FilterCriteria.builder()
 .severity(Collections.singletonList(severityFilter))
 .build();

 // Build the filter creation request.
 CreateFilterRequest request = CreateFilterRequest.builder()
 .name(filterName)
 .filterCriteria(filterCriteria)
 .action(FilterAction.SUPPRESS)
 .description(description)
 .build();

 return getAsyncClient().createFilter(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error creating filter:
%s".formatted(ve.getMessage()),
 ve
);
 }

 if (cause instanceof Inspector2Exception e) {
 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),

```

```

 e
);
 }

 // Unexpected async error
 throw new CompletionException(
 "Unexpected error creating filter:
%s".formatted(exception.getMessage()),
 exception
);
 }
})
// Extract and return the ARN of the created filter.
.thenApply(CreateFilterResponse::arn);
}

```

- For API details, see [CreateFilter](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteFilter

The following code example shows how to use DeleteFilter.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an AWS Inspector2 filter.
 *
 * @param filterARN The ARN of the filter to delete.
 */
public CompletableFuture<Void> deleteFilterAsync(String filterARN) {
 return getAsyncClient().deleteFilter(
 DeleteFilterRequest.builder()
 .arn(filterARN)
 .build()
);
}

```

```

)
 .handle((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;

 if (cause instanceof ResourceNotFoundException rnfe) {
 String msg = "Filter not found for ARN:
%s".formatted(filterARN);
 logger.warn(msg, rnfe);
 throw new CompletionException(msg, rnfe);
 }

 throw new RuntimeException("Failed to delete the filter: " +
cause, cause);
 }
 return null;
 });
 }
}

```

- For API details, see [DeleteFilter](#) in *AWS SDK for Java 2.x API Reference*.

## Disable

The following code example shows how to use `Disable`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously disables AWS Inspector for the specified accounts and resource
types.
 *
 * @param accountIds a {@link List} of AWS account IDs for which to disable
Inspector;
 *
 * may be {@code null} or empty to target the current account

```

```

 * @return a {@link CompletableFuture} that, when completed, returns a {@link
String}
 * summarizing the disable results for each account
 * @throws CompletionException if the disable operation fails due to validation
errors,
 * service errors, or other exceptions
 * @see <a href="https://docs.aws.amazon.com/inspector/latest/APIReference/
API_Disable.html">
 * AWS Inspector2 Disable API
 */
 public CompletableFuture<String> disableInspectorAsync(List<String> accountIds)
 {

 // The resource types to disable.
 List<ResourceScanType> resourceTypes = List.of(
 ResourceScanType.EC2,
 ResourceScanType.ECR,
 ResourceScanType.LAMBDA,
 ResourceScanType.LAMBDA_CODE
);

 // Build the request.
 DisableRequest.Builder requestBuilder = DisableRequest.builder()
 .resourceTypes(resourceTypes);

 if (accountIds != null && !accountIds.isEmpty()) {
 requestBuilder.accountIds(accountIds);
 }

 DisableRequest request = requestBuilder.build();

 return getAsyncClient().disable(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Inspector may already be disabled for this
account: %s".formatted(cause.getMessage()),
 cause
);
 }

 if (cause instanceof Inspector2Exception) {

```

```

 Inspector2Exception e = (Inspector2Exception) cause;
 throw new CompletionException(
 "AWS Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to disable Inspector:
%s".formatted(exception.getMessage()),
 exception
);
}
}))
.thenApply(response -> {
 StringBuilder summary = new StringBuilder("Disable results:\n");

 if (response.accounts() == null ||
response.accounts().isEmpty()) {
 summary.append("Inspector may already be disabled for all
target accounts.");
 return summary.toString();
 }

 for (Account account : response.accounts()) {
 String accountId = account.accountId() != null ?
account.accountId() : "Unknown";
 String status = account.status() != null ?
account.statusAsString() : "Unknown";
 summary.append(" • Account: ").append(accountId)
 .append(" # Status: ").append(status).append("\n");
 }

 return summary.toString();
});
}

```

- For API details, see [Disable](#) in *AWS SDK for Java 2.x API Reference*.

## Enable

The following code example shows how to use Enable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Enables AWS Inspector for the provided account(s) and default resource types.
 *
 * @param accountIds Optional list of AWS account IDs.
 */
public CompletableFuture<String> enableInspectorAsync(List<String> accountIds) {

 // The resource types to enable.
 List<ResourceScanType> resourceTypes = List.of(
 ResourceScanType.EC2,
 ResourceScanType.ECR,
 ResourceScanType.LAMBDA,
 ResourceScanType.LAMBDA_CODE
);

 // Build the request.
 EnableRequest.Builder requestBuilder = EnableRequest.builder()
 .resourceTypes(resourceTypes);

 if (accountIds != null && !accountIds.isEmpty()) {
 requestBuilder.accountIds(accountIds);
 }

 EnableRequest request = requestBuilder.build();
 return getAsyncClient().enable(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException(
```

```

 "Inspector may already be enabled for this
account: %s".formatted(cause.getMessage()),
 cause
);

 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;
 throw new CompletionException(
 "AWS Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 cause
);
 }

 throw new CompletionException(
 "Failed to enable Inspector:
%s".formatted(exception.getMessage()),
 exception
);
}
}))
.thenApply(response -> {
 StringBuilder summary = new StringBuilder("Enable results:\n");

 if (response.accounts() == null ||
response.accounts().isEmpty()) {
 summary.append("Inspector may already be enabled for all
target accounts.");
 return summary.toString();
 }

 for (Account account : response.accounts()) {
 String accountId = account.accountId() != null ?
account.accountId() : "Unknown";
 String status = account.status() != null ?
account.statusAsString() : "Unknown";
 summary.append(" • Account: ").append(accountId)
 .append(" # Status: ").append(status).append("\n");
 }

 return summary.toString();
});

```



```
}
```

- For API details, see [Enable](#) in *AWS SDK for Java 2.x API Reference*.

## ListCoverage

The following code example shows how to use ListCoverage.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Lists AWS Inspector2 coverage details for scanned resources using a
 * paginator.
 *
 * @param maxResults Maximum number of resources to return.
 */
public CompletableFuture<String> listCoverageAsync(int maxResults) {
 ListCoverageRequest initialRequest = ListCoverageRequest.builder()
 .maxResults(maxResults)
 .build();

 ListCoveragePublisher paginator =
 getAsyncClient().listCoveragePaginator(initialRequest);
 StringBuilder summary = new StringBuilder();

 return paginator.subscribe(response -> {
 List<CoveredResource> coveredResources = response.coveredResources();

 if (coveredResources == null || coveredResources.isEmpty()) {
 summary.append("No coverage information available for this page.
\n");
 }

 return;
 });
}
```

```

 Map<String, List<CoveredResource>> byType = coveredResources.stream()

.collect(Collectors.groupingBy(CoveredResource::resourceTypeAsString));

 byType.forEach((type, list) ->
 summary.append(" ").append(type)
 .append(": ").append(list.size())
 .append(" resource(s)\n")
);

 // Include up to 3 sample resources per page
 for (int i = 0; i < Math.min(coveredResources.size(), 3); i++) {
 CoveredResource r = coveredResources.get(i);
 summary.append(" - ").append(r.resourceTypeAsString())
 .append(": ").append(r.resourceId()).append("\n");
 summary.append(" Scan Type:
").append(r.scanTypeAsString()).append("\n");
 if (r.scanStatus() != null) {
 summary.append(" Status:
").append(r.scanStatus().statusCodeAsString()).append("\n");
 }
 if (r.accountId() != null) {
 summary.append(" Account ID:
").append(r.accountId()).append("\n");
 }
 summary.append("\n");
 }

 }).thenApply(v -> {
 if (summary.length() == 0) {
 return "No coverage information found across all pages.";
 } else {
 return "Coverage Information:\n" + summary.toString();
 }
 }).exceptionally(ex -> {
 Throwable cause = ex.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Validation error listing coverage: " + cause.getMessage(),
 cause);
 } else if (cause instanceof Inspector2Exception e) {
 throw new CompletionException(
 "Inspector2 service error: " +
 e.awsErrorDetails().errorMessage(), e);
 }
 });

```

```

 }
 throw new CompletionException("Unexpected error listing coverage: " +
ex.getMessage(), ex);
 });
}

```

- For API details, see [ListCoverage](#) in *AWS SDK for Java 2.x API Reference*.

## ListCoverageStatistics

The following code example shows how to use `ListCoverageStatistics`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves and prints the coverage statistics using a paginator.
 */
public CompletableFuture<String> listCoverageStatisticsAsync() {
 ListCoverageStatisticsRequest request =
ListCoverageStatisticsRequest.builder()
 .build();

 return getAsyncClient().listCoverageStatistics(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();

 if (cause instanceof ValidationException) {
 throw new CompletionException(
 "Validation error listing coverage statistics:
%s".formatted(cause.getMessage()),
 cause
);
 }
 }
 });
}

```

```

 }

 if (cause instanceof Inspector2Exception) {
 Inspector2Exception e = (Inspector2Exception) cause;

 throw new CompletionException(
 "Inspector2 service error:
%s".formatted(e.awsErrorDetails().errorMessage()),
 e
);
 }

 throw new CompletionException(
 "Unexpected error listing coverage statistics:
%s".formatted(exception.getMessage()),
 exception
);
 }
})
.thenApply(response -> {
 List<Counts> countsList = response.countsByGroup();
 StringBuilder sb = new StringBuilder();

 if (countsList == null || countsList.isEmpty()) {
 sb.append("No coverage statistics available.\n");
 return sb.toString();
 }

 sb.append("Coverage Statistics:\n");

 for (Counts c : countsList) {
 sb.append(" Group: ").append(c.groupKey()).append("\n")
 .append(" Total Count:
").append(c.count()).append("\n\n");
 }

 return sb.toString();
});
}

```

- For API details, see [ListCoverageStatistics](#) in *AWS SDK for Java 2.x API Reference*.

## ListFilters

The following code example shows how to use ListFilters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously lists Inspector2 filters using a paginator.
 *
 * @param maxResults maximum filters per page (nullable)
 * @return CompletableFuture completed with summary text
 */
public CompletableFuture<String> listFiltersAsync(Integer maxResults) {
 logger.info("Starting async filters paginator...");

 ListFiltersRequest.Builder builder = ListFiltersRequest.builder();
 if (maxResults != null) {
 builder.maxResults(maxResults);
 }

 ListFiltersRequest request = builder.build();

 // Paginator from SDK
 ListFiltersPublisher paginator =
getAsyncClient().listFiltersPaginator(request);
 StringBuilder collectedFilterIds = new StringBuilder();

 return paginator.subscribe(response -> {
 response.filters().forEach(filter -> {
 logger.info("Filter: " + filter.arn());
 collectedFilterIds.append(filter.arn()).append("\n");
 });
 }).thenApply(v -> {
 String result = collectedFilterIds.toString();
 logger.info("Successfully listed all filters.");
 return result.isEmpty() ? "No filters found." : result;
 });
}
```

```

 }).exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing filters:
%s".formatted(ve.getMessage()),
 ve
);
 }

 throw new RuntimeException("Failed to list filters", ex);
 });
}

```

- For API details, see [ListFilters](#) in *AWS SDK for Java 2.x API Reference*.

## ListFindings

The following code example shows how to use ListFindings.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all AWS Inspector findings of LOW severity asynchronously.
 *
 * @return CompletableFuture containing a List of finding ARNs.
 * Returns an empty list if no LOW severity findings are found.
 */
public CompletableFuture<ArrayList<String>> listLowSeverityFindingsAsync() {
 logger.info("Starting async LOW severity findings paginator...");

 // Build a filter criteria for LOW severity.
 StringFilter severityFilter = StringFilter.builder()

```

```

 .value(Severity.LOW.toString())
 .comparison(StringComparison.EQUALS)
 .build();

 FilterCriteria filterCriteria = FilterCriteria.builder()
 .severity(Collections.singletonList(severityFilter))
 .build();

 // Build the request.
 ListFindingsRequest request = ListFindingsRequest.builder()
 .filterCriteria(filterCriteria)
 .build();

 ListFindingsPublisher paginator =
 getAsyncClient().listFindingsPaginator(request);
 List<String> allArns = Collections.synchronizedList(new ArrayList<>());

 return paginator.subscribe(response -> {
 if (response.findings() != null && !
response.findings().isEmpty()) {
 response.findings().forEach(finding -> {
 logger.info("Finding ARN: {}", finding.findingArn());
 allArns.add(finding.findingArn());
 });
 } else {
 logger.info("Page contained no findings.");
 }
 })
 .thenRun(() -> logger.info("Successfully listed all LOW severity
findings."))
 .thenApply(v -> new ArrayList<>(allArns)) // Return list instead of
a formatted string
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;
 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing LOW severity findings:
%s".formatted(ve.getMessage()),
 ve
);
 }
 throw new RuntimeException("Failed to list LOW severity
findings", ex);
 });

```

```
}
```

- For API details, see [ListFindings](#) in *AWS SDK for Java 2.x API Reference*.

## ListUsageTotals

The following code example shows how to use `ListUsageTotals`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously lists Inspector2 usage totals using a paginator.
 *
 * @param accountIds optional list of account IDs
 * @param maxResults maximum results per page
 * @return CompletableFuture completed with formatted summary text
 */
public CompletableFuture<String> listUsageTotalsAsync(
 List<String> accountIds,
 int maxResults) {

 logger.info("Starting usage totals paginator...");

 ListUsageTotalsRequest.Builder builder = ListUsageTotalsRequest.builder()
 .maxResults(maxResults);

 if (accountIds != null && !accountIds.isEmpty()) {
 builder.accountIds(accountIds);
 }

 ListUsageTotalsRequest request = builder.build();
 ListUsageTotalsPublisher paginator =
 getAsyncClient().listUsageTotalsPaginator(request);
```



```

 StringBuilder summaryBuilder = new StringBuilder();

 return paginator.subscribe(response -> {
 if (response.totals() != null && !response.totals().isEmpty()) {
 response.totals().forEach(total -> {
 if (total.usage() != null) {
 total.usage().forEach(usage -> {
 logger.info("Usage: {} = {}",
usage.typeAsString(), usage.total());
 summaryBuilder.append(usage.typeAsString())
 .append(": ")
 .append(usage.total())
 .append("\n");
 });
 }
 });
 } else {
 logger.info("Page contained no usage totals.");
 }
 }).thenRun(() -> logger.info("Successfully listed usage totals.")).thenApply(v -> {
 String summary = summaryBuilder.toString();
 return summary.isEmpty() ? "No usage totals found." : summary;
 }).exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ValidationException ve) {
 throw new CompletionException(
 "Validation error listing usage totals:
%s".formatted(ve.getMessage()),
 ve
);
 }

 throw new CompletionException("Failed to list usage totals",
cause);
 });
 }

```

- For API details, see [ListUsageTotals](#) in *AWS SDK for Java 2.x API Reference*.

# AWS IoT examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS IoT.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello AWS IoT

The following code example shows how to get started using AWS IoT.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iot.IotClient;
import software.amazon.awssdk.services.iot.model.ListThingsRequest;
import software.amazon.awssdk.services.iot.model.ListThingsResponse;
import software.amazon.awssdk.services.iot.model.ThingAttribute;
import software.amazon.awssdk.services.iot.paginators.ListThingsIterable;

import java.util.List;
```

```
public class HelloIoT {
 public static void main(String[] args) {
 System.out.println("Hello AWS IoT. Here is a listing of your AWS IoT
Things:");
 IotClient iotClient = IotClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllThings(iotClient);
 }

 public static void listAllThings(IotClient iotClient) {
 iotClient.listThingsPaginator(ListThingsRequest.builder()
 .maxResults(10)
 .build())
 .stream()
 .flatMap(response -> response.things().stream())
 .forEach(attribute -> {
 System.out.println("Thing name: " + attribute.thingName());
 System.out.println("Thing ARN: " + attribute.thingArn());
 });
 }
}
```

- For API details, see [listThings](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an AWS IoT Thing.
- Generate a device certificate.
- Update an AWS IoT Thing with Attributes.
- Return a unique endpoint.
- List your AWS IoT certificates.
- Update an AWS IoT shadow.

- Write out state information.
- Creates a rule.
- List your rules.
- Search things using the Thing name.
- Delete an AWS IoT Thing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating AWS IoT features.

```
import java.util.Scanner;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java example performs these tasks:
 *
 * 1. Creates an AWS IoT Thing.
 * 2. Generate and attach a device certificate.
 * 3. Update an AWS IoT Thing with Attributes.
 * 4. Get an AWS IoT Endpoint.
 * 5. List your certificates.
 * 6. Updates the shadow for the specified thing..
 * 7. Write out the state information, in JSON format
 * 8. Creates a rule
 * 9. List rules
 * 10. Search things
 * 11. Detach and delete the certificate.
 * 12. Delete Thing.
```

```

*/
public class IotScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <roleARN> <snsAction>

 Where:
 roleARN - The ARN of an IAM role that has permission to work
with AWS IoT.
 snsAction - An ARN of an SNS topic.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 IotActions iotActions = new IotActions();
 String thingName;
 String ruleName;
 String roleARN = args[0];
 String snsAction = args[1];
 Scanner scanner = new Scanner(System.in);

 System.out.println(DASHES);
 System.out.println("Welcome to the AWS IoT basics scenario.");
 System.out.println("""
 This example program demonstrates various interactions with the AWS
Internet of Things (IoT) Core service. The program guides you through a series of
steps,
 including creating an IoT Thing, generating a device certificate,
updating the Thing with attributes, and so on.
 It utilizes the AWS SDK for Java V2 and incorporates functionality for
creating and managing IoT Things, certificates, rules,
 shadows, and performing searches. The program aims to showcase AWS IoT
capabilities and provides a comprehensive example for
 developers working with AWS IoT in a Java environment.

 Let's get started...
 """);
 }
}

```

```

 "");
 System.out.println(DASHES);

 System.out.println("1. Create an AWS IoT Thing.");
 System.out.println("""
 An AWS IoT Thing represents a virtual entity in the AWS IoT service that
 can be associated with
 a physical device.
 """);
 // Prompt the user for input.
 System.out.print("Enter Thing name: ");
 thingName = scanner.nextLine();
 iotActions.createIoTThing(thingName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Generate a device certificate.");
 System.out.println("""
 A device certificate performs a role in securing the communication
 between devices (Things)
 and the AWS IoT platform.
 """);

 System.out.print("Do you want to create a certificate for " +thingName +"?
(y/n)");
 String certAns = scanner.nextLine();
 String certificateArn="" ;
 if (certAns != null && certAns.trim().equalsIgnoreCase("y")) {
 certificateArn = iotActions.createCertificate();
 System.out.println("Attach the certificate to the AWS IoT Thing.");
 iotActions.attachCertificateToThing(thingName, certificateArn);
 } else {
 System.out.println("A device certificate was not created.");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Update an AWS IoT Thing with Attributes.");
 System.out.println("""
 IoT Thing attributes, represented as key-value pairs, offer a pivotal
 advantage in facilitating efficient data
 management and retrieval within the AWS IoT ecosystem.
 """);
 waitForInputToContinue(scanner);

```

```

 iotActions.updateShadowThing(thingName);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. Return a unique endpoint specific to the Amazon Web
Services account.");
 System.out.println("""
 An IoT Endpoint refers to a specific URL or Uniform Resource Locator
that serves as the entry point for communication between IoT devices and the AWS
IoT service.
 """);
 waitForInputToContinue(scanner);
 String endpointUrl = iotActions.describeEndpoint();
 System.out.println("The endpoint is "+endpointUrl);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. List your AWS IoT certificates");
 waitForInputToContinue(scanner);
 if (certificateArn.length() > 0) {
 iotActions.listCertificates();
 } else {
 System.out.println("You did not create a certificates. Skipping this
step.");
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Create an IoT shadow that refers to a digital
representation or virtual twin of a physical IoT device");
 System.out.println("""
 A Thing Shadow refers to a feature that enables you to create a virtual
representation, or "shadow,"
 of a physical device or thing. The Thing Shadow allows you to
synchronize and control the state of a device between
 the cloud and the device itself. and the AWS IoT service. For example,
you can write and retrieve JSON data from a Thing Shadow.
 """);
 waitForInputToContinue(scanner);
 iotActions.updateShadowThing(thingName);
 waitForInputToContinue(scanner);

```

```

 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Write out the state information, in JSON format.");
 waitForInputToContinue(scanner);
 iotActions.getPayload(thingName);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Creates a rule");
 System.out.println("""
Creates a rule that is an administrator-level action.
Any user who has permission to create rules will be able to access data
processed by the rule.
""");
 System.out.print("Enter Rule name: ");
 ruleName = scanner.nextLine();
 iotActions.createIoTRule(roleARN, ruleName, snsAction);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. List your rules.");
 waitForInputToContinue(scanner);
 iotActions.listIoTRules();
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Search things using the Thing name.");
 waitForInputToContinue(scanner);
 String queryString = "thingName:"+thingName ;
 iotActions.searchThings(queryString);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 if (certificateArn.length() > 0) {
 System.out.print("Do you want to detach and delete the certificate for "
+thingName +"? (y/n)");
 String delAns = scanner.nextLine();
 if (delAns != null && delAns.trim().equalsIgnoreCase("y")) {

```



```

 System.out.println("11. You selected to detach and delete the
certificate.");
 waitForInputToContinue(scanner);
 iotActions.detachThingPrincipal(thingName, certificateArn);
 iotActions.deleteCertificate(certificateArn);
 waitForInputToContinue(scanner);
 } else {
 System.out.println("11. You selected not to delete the
certificate.");
 }
 } else {
 System.out.println("11. You did not create a certificate so there is
nothing to delete.");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("12. Delete the AWS IoT Thing.");
 System.out.print("Do you want to delete the IoT Thing? (y/n)");
 String delAns = scanner.nextLine();
 if (delAns != null && delAns.trim().equalsIgnoreCase("y")) {
 iotActions.deleteIoTThing(thingName);
 } else {
 System.out.println("The IoT Thing was not deleted.");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The AWS IoT workflow has successfully completed.");
 System.out.println(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {

```

```

 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
}
}
}

```

A wrapper class for AWS IoT SDK methods.

```

import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryPolicy;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.iot.IotAsyncClient;
import software.amazon.awssdk.services.iot.model.Action;
import software.amazon.awssdk.services.iot.model.AttachThingPrincipalRequest;
import software.amazon.awssdk.services.iot.model.AttachThingPrincipalResponse;
import software.amazon.awssdk.services.iot.model.Certificate;
import software.amazon.awssdk.services.iot.model.CreateKeysAndCertificateResponse;
import software.amazon.awssdk.services.iot.model.CreateThingRequest;
import software.amazon.awssdk.services.iot.model.CreateThingResponse;
import software.amazon.awssdk.services.iot.model.CreateTopicRuleRequest;
import software.amazon.awssdk.services.iot.model.CreateTopicRuleResponse;
import software.amazon.awssdk.services.iot.model.DeleteCertificateRequest;
import software.amazon.awssdk.services.iot.model.DeleteCertificateResponse;
import software.amazon.awssdk.services.iot.model.DeleteThingRequest;
import software.amazon.awssdk.services.iot.model.DeleteThingResponse;
import software.amazon.awssdk.services.iot.model.DescribeEndpointRequest;
import software.amazon.awssdk.services.iot.model.DescribeEndpointResponse;
import software.amazon.awssdk.services.iot.model.DescribeThingRequest;
import software.amazon.awssdk.services.iot.model.DescribeThingResponse;
import software.amazon.awssdk.services.iot.model.DetachThingPrincipalRequest;
import software.amazon.awssdk.services.iot.model.DetachThingPrincipalResponse;
import software.amazon.awssdk.services.iot.model.IotException;
import software.amazon.awssdk.services.iot.model.ListCertificatesResponse;
import software.amazon.awssdk.services.iot.model.ListTopicRulesRequest;
import software.amazon.awssdk.services.iot.model.ListTopicRulesResponse;
import software.amazon.awssdk.services.iot.model.SearchIndexRequest;

```

```
import software.amazon.awssdk.services.iot.model.SearchIndexResponse;
import software.amazon.awssdk.services.iot.model.TopicRuleListItem;
import software.amazon.awssdk.services.iot.model.SnsAction;
import software.amazon.awssdk.services.iot.model.TopicRulePayload;
import software.amazon.awssdk.services.iotdataplane.IotDataPlaneAsyncClient;
import software.amazon.awssdk.services.iotdataplane.model.GetThingShadowRequest;
import software.amazon.awssdk.services.iotdataplane.model.GetThingShadowResponse;
import software.amazon.awssdk.services.iotdataplane.model.UpdateThingShadowRequest;
import software.amazon.awssdk.services.iotdataplane.model.UpdateThingShadowResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class IotActions {

 private static IotAsyncClient iotAsyncClient;

 private static IotDataPlaneAsyncClient iotAsyncDataPlaneClient;

 private static final String TOPIC = "your-iot-topic";

 private static IotDataPlaneAsyncClient getAsyncDataPlaneClient() {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 if (iotAsyncDataPlaneClient == null) {
 iotAsyncDataPlaneClient = IotDataPlaneAsyncClient.builder()
```

```

 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return iotAsyncDataPlaneClient;
}

private static IotAsyncClient getAsyncClient() {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 if (iotAsyncClient == null) {
 iotAsyncClient = IotAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return iotAsyncClient;
}

/**
 * Creates an IoT certificate asynchronously.
 *
 * @return The ARN of the created certificate.
 * <p>
 * This method initiates an asynchronous request to create an IoT certificate.
 * If the request is successful, it prints the certificate details and returns
the certificate ARN.

```

```

 * If an exception occurs, it prints the error message.
 */
 public String createCertificate() {
 CompletableFuture<CreateKeysAndCertificateResponse> future =
getAsyncClient().createKeysAndCertificate();
 final String[] certificateArn = {null};
 future.whenComplete((response, ex) -> {
 if (response != null) {
 String certificatePem = response.certificatePem();
 certificateArn[0] = response.certificateArn();

 // Print the details.
 System.out.println("\nCertificate:");
 System.out.println(certificatePem);
 System.out.println("\nCertificate ARN:");
 System.out.println(certificateArn[0]);

 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
 });

 future.join();
 return certificateArn[0];
 }

 /**
 * Creates an IoT Thing with the specified name asynchronously.
 *
 * @param thingName The name of the IoT Thing to create.
 *
 * This method initiates an asynchronous request to create an IoT Thing with the
specified name.
 * If the request is successful, it prints the name of the thing and its ARN
value.
 * If an exception occurs, it prints the error message.
 */

```

```

 public void createIoTThing(String thingName) {
 CreateThingRequest createThingRequest = CreateThingRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<CreateThingResponse> future =
getAsyncClient().createThing(createThingRequest);
 future.whenComplete((createThingResponse, ex) -> {
 if (createThingResponse != null) {
 System.out.println(thingName + " was successfully created. The ARN
value is " + createThingResponse.thingArn());
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
 });

 future.join();
 }

/**
 * Attaches a certificate to an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 * @param certificateArn The ARN of the certificate to attach.
 *
 * This method initiates an asynchronous request to attach a certificate to an
IoT Thing.
 * If the request is successful, it prints a confirmation message and additional
information about the Thing.
 * If an exception occurs, it prints the error message.
 */
 public void attachCertificateToThing(String thingName, String certificateArn) {
 AttachThingPrincipalRequest principalRequest =
AttachThingPrincipalRequest.builder()
 .thingName(thingName)
 .principal(certificateArn)
 .build();
 }

```

```

 CompletableFuture<AttachThingPrincipalResponse> future =
getAsyncClient().attachThingPrincipal(principalRequest);
 future.whenComplete((attachResponse, ex) -> {
 if (attachResponse != null &&
attachResponse.sdkHttpResponse().isSuccessful()) {
 System.out.println("Certificate attached to Thing successfully.");

 // Print additional information about the Thing.
 describeThing(thingName);
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to attach certificate to Thing. HTTP
Status Code: " +
 attachResponse.sdkHttpResponse().statusCode());
 }
 }
 });

 future.join();
 }

/**
 * Describes an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to describe an IoT Thing.
 * If the request is successful, it prints the Thing details.
 * If an exception occurs, it prints the error message.
 */
private void describeThing(String thingName) {
 DescribeThingRequest thingRequest = DescribeThingRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<DescribeThingResponse> future =
getAsyncClient().describeThing(thingRequest);
 future.whenComplete((describeResponse, ex) -> {

```

```

 if (describeResponse != null) {
 System.out.println("Thing Details:");
 System.out.println("Thing Name: " + describeResponse.thingName());
 System.out.println("Thing ARN: " + describeResponse.thingArn());
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to describe Thing.");
 }
 }
 });

 future.join();
}

/**
 * Updates the shadow of an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to update the shadow of an IoT
Thing.
 * If the request is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void updateShadowThing(String thingName) {
 // Create Thing Shadow State Document.
 String stateDocument = "{\"state\":{\"reported\":{\"temperature\":25,
\"humidity\":50}}}";
 SdkBytes data = SdkBytes.fromString(stateDocument, StandardCharsets.UTF_8);
 UpdateThingShadowRequest updateThingShadowRequest =
UpdateThingShadowRequest.builder()
 .thingName(thingName)
 .payload(data)
 .build();

 CompletableFuture<UpdateThingShadowResponse> future =
getAsyncDataPlaneClient().updateThingShadow(updateThingShadowRequest);
 future.whenComplete((updateResponse, ex) -> {

```



```

 if (updateResponse != null) {
 System.out.println("Thing Shadow updated successfully.");
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to update Thing Shadow.");
 }
 }
 });

 future.join();
}

/**
 * Describes the endpoint of the IoT service asynchronously.
 *
 * @return A CompletableFuture containing the full endpoint URL.
 *
 * This method initiates an asynchronous request to describe the endpoint of the
IoT service.
 * If the request is successful, it prints and returns the full endpoint URL.
 * If an exception occurs, it prints the error message.
 */
public String describeEndpoint() {
 CompletableFuture<DescribeEndpointResponse> future =
getAsyncClient().describeEndpoint(DescribeEndpointRequest.builder().endpointType("iot:Data-
ATS").build());
 final String[] result = {null};

 future.whenComplete((endpointResponse, ex) -> {
 if (endpointResponse != null) {
 String endpointUrl = endpointResponse.endpointAddress();
 String exString = getValue(endpointUrl);
 String fullEndpoint = "https://" + exString + "-ats.iot.us-
east-1.amazonaws.com";

 System.out.println("Full Endpoint URL: " + fullEndpoint);
 result[0] = fullEndpoint;
 } else {

```

```

 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
});

 future.join();
 return result[0];
}

/**
 * Extracts a specific value from the endpoint URL.
 *
 * @param input The endpoint URL to process.
 * @return The extracted value from the endpoint URL.
 */
private static String getValue(String input) {
 // Define a regular expression pattern for extracting the subdomain.
 Pattern pattern = Pattern.compile("^(.*)\\.\\.iot\\.\\.us-east-1\\.\\.amazonaws\\.
\\.com");

 // Match the pattern against the input string.
 Matcher matcher = pattern.matcher(input);

 // Check if a match is found.
 if (matcher.find()) {
 // Extract the subdomain from the first capturing group.
 String subdomain = matcher.group(1);
 System.out.println("Extracted subdomain: " + subdomain);
 return subdomain ;
 } else {
 System.out.println("No match found");
 }
 return "" ;
}

/**
 * Lists all certificates asynchronously.
 *

```

```

 * This method initiates an asynchronous request to list all certificates.
 * If the request is successful, it prints the certificate IDs and ARNs.
 * If an exception occurs, it prints the error message.
 */
 public void listCertificates() {
 CompletableFuture<ListCertificatesResponse> future =
getAsyncClient().listCertificates();
 future.whenComplete((response, ex) -> {
 if (response != null) {
 List<Certificate> certList = response.certificates();
 for (Certificate cert : certList) {
 System.out.println("Cert id: " + cert.certificateId());
 System.out.println("Cert Arn: " + cert.certificateArn());
 }
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to list certificates.");
 }
 }
 });

 future.join();
 }

 /**
 * Retrieves the payload of a Thing's shadow asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to get the payload of a Thing's
 shadow.
 * If the request is successful, it prints the shadow data.
 * If an exception occurs, it prints the error message.
 */
 public void getPayload(String thingName) {
 GetThingShadowRequest getThingShadowRequest =
GetThingShadowRequest.builder()
 .thingName(thingName)

```

```

 .build();

 CompletableFuture<GetThingShadowResponse> future =
getAsyncDataPlaneClient().getThingShadow(getThingShadowRequest);
 future.whenComplete((getThingShadowResponse, ex) -> {
 if (getThingShadowResponse != null) {
 // Extracting payload from response.
 SdkBytes payload = getThingShadowResponse.payload();
 String payloadString = payload.asUtf8String();
 System.out.println("Received Shadow Data: " + payloadString);
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to get Thing Shadow payload.");
 }
 }
 });

 future.join();
 }

/**
 * Creates an IoT rule asynchronously.
 *
 * @param roleARN The ARN of the IAM role that grants access to the rule's
actions.
 * @param ruleName The name of the IoT rule.
 * @param action The ARN of the action to perform when the rule is triggered.
 *
 * This method initiates an asynchronous request to create an IoT rule.
 * If the request is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
 public void createIoTRule(String roleARN, String ruleName, String action) {
 String sql = "SELECT * FROM '" + TOPIC + "'";
 SnsAction action1 = SnsAction.builder()
 .targetArn(action)
 .roleArn(roleARN)
 .build();
 }

```

```

// Create the action.
Action myAction = Action.builder()
 .sns(action1)
 .build();

// Create the topic rule payload.
TopicRulePayload topicRulePayload = TopicRulePayload.builder()
 .sql(sql)
 .actions(myAction)
 .build();

// Create the topic rule request.
CreateTopicRuleRequest topicRuleRequest = CreateTopicRuleRequest.builder()
 .ruleName(ruleName)
 .topicRulePayload(topicRulePayload)
 .build();

CompletableFuture<CreateTopicRuleResponse> future =
getAsyncClient().createTopicRule(topicRuleRequest);
future.whenComplete((response, ex) -> {
 if (response != null) {
 System.out.println("IoT Rule created successfully.");
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to create IoT Rule.");
 }
 }
});

future.join();
}

/**
 * Lists IoT rules asynchronously.
 *
 * This method initiates an asynchronous request to list IoT rules.
 * If the request is successful, it prints the names and ARNs of the rules.

```

```

 * If an exception occurs, it prints the error message.
 */
 public void listIoTRules() {
 ListTopicRulesRequest listTopicRulesRequest =
ListTopicRulesRequest.builder().build();
 CompletableFuture<ListTopicRulesResponse> future =
getAsyncClient().listTopicRules(listTopicRulesRequest);
 future.whenComplete((listTopicRulesResponse, ex) -> {
 if (listTopicRulesResponse != null) {
 System.out.println("List of IoT Rules:");
 List<TopicRuleListItem> ruleList = listTopicRulesResponse.rules();
 for (TopicRuleListItem rule : ruleList) {
 System.out.println("Rule Name: " + rule.ruleName());
 System.out.println("Rule ARN: " + rule.ruleArn());
 System.out.println("-----");
 }
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to list IoT Rules.");
 }
 }
 });

 future.join();
 }

 /**
 * Searches for IoT Things asynchronously based on a query string.
 *
 * @param queryString The query string to search for Things.
 *
 * This method initiates an asynchronous request to search for IoT Things.
 * If the request is successful and Things are found, it prints their IDs.
 * If no Things are found, it prints a message indicating so.
 * If an exception occurs, it prints the error message.
 */
 public void searchThings(String queryString) {
 SearchIndexRequest searchIndexRequest = SearchIndexRequest.builder()

```

```

 .queryString(queryString)
 .build();

 CompletableFuture<SearchIndexResponse> future =
getAsyncClient().searchIndex(searchIndexRequest);
 future.whenComplete((searchIndexResponse, ex) -> {
 if (searchIndexResponse != null) {
 // Process the result.
 if (searchIndexResponse.things().isEmpty()) {
 System.out.println("No things found.");
 } else {
 searchIndexResponse.things().forEach(thing ->
System.out.println("Thing id found using search is " + thing.thingId()));
 }
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to search for IoT Things.");
 }
 }
 });

 future.join();
 }

/**
 * Detaches a principal (certificate) from an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 * @param certificateArn The ARN of the certificate to detach.
 *
 * This method initiates an asynchronous request to detach a certificate from an
IoT Thing.
 * If the detachment is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
 public void detachThingPrincipal(String thingName, String certificateArn) {
 DetachThingPrincipalRequest thingPrincipalRequest =
DetachThingPrincipalRequest.builder()

```

```

 .principal(certificateArn)
 .thingName(thingName)
 .build();

 CompletableFuture<DetachThingPrincipalResponse> future =
getAsyncClient().detachThingPrincipal(thingPrincipalRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println(certificateArn + " was successfully removed from
" + thingName);
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });

 future.join();
}

/**
 * Deletes a certificate asynchronously.
 *
 * @param certificateArn The ARN of the certificate to delete.
 *
 * This method initiates an asynchronous request to delete a certificate.
 * If the deletion is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void deleteCertificate(String certificateArn) {
 DeleteCertificateRequest certificateProviderRequest =
DeleteCertificateRequest.builder()
 .certificateId(extractCertificateId(certificateArn))
 .build();

 CompletableFuture<DeleteCertificateResponse> future =
getAsyncClient().deleteCertificate(certificateProviderRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println(certificateArn + " was successfully deleted.");

```



```

 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });

 future.join();
}

/**
 * Deletes an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing to delete.
 *
 * This method initiates an asynchronous request to delete an IoT Thing.
 * If the deletion is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void deleteIoTThing(String thingName) {
 DeleteThingRequest deleteThingRequest = DeleteThingRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<DeleteThingResponse> future =
getAsyncClient().deleteThing(deleteThingRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println("Deleted Thing " + thingName);
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });
}

```

```
 future.join();
 }

 // Get the cert Id from the Cert ARN value.
 private String extractCertificateId(String certificateArn) {
 // Example ARN: arn:aws:iot:region:account-id:cert/certificate-id.
 String[] arnParts = certificateArn.split(":");
 String certificateIdPart = arnParts[arnParts.length - 1];
 return certificateIdPart.substring(certificateIdPart.lastIndexOf("/") + 1);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AttachThingPrincipal](#)
  - [CreateKeysAndCertificate](#)
  - [CreateThing](#)
  - [CreateTopicRule](#)
  - [DeleteCertificate](#)
  - [DeleteThing](#)
  - [DeleteTopicRule](#)
  - [DescribeEndpoint](#)
  - [DescribeThing](#)
  - [DetachThingPrincipal](#)
  - [ListCertificates](#)
  - [ListThings](#)
  - [SearchIndex](#)
  - [UpdateIndexingConfiguration](#)
  - [UpdateThing](#)

## Actions

### AttachThingPrincipal

The following code example shows how to use `AttachThingPrincipal`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Attaches a certificate to an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 * @param certificateArn The ARN of the certificate to attach.
 *
 * This method initiates an asynchronous request to attach a certificate to an
IoT Thing.
 * If the request is successful, it prints a confirmation message and additional
information about the Thing.
 * If an exception occurs, it prints the error message.
 */
public void attachCertificateToThing(String thingName, String certificateArn) {
 AttachThingPrincipalRequest principalRequest =
AttachThingPrincipalRequest.builder()
 .thingName(thingName)
 .principal(certificateArn)
 .build();

 CompletableFuture<AttachThingPrincipalResponse> future =
getAsyncClient().attachThingPrincipal(principalRequest);
 future.whenComplete((attachResponse, ex) -> {
 if (attachResponse != null &&
attachResponse.sdkHttpResponse().isSuccessful()) {
 System.out.println("Certificate attached to Thing successfully.");

 // Print additional information about the Thing.
 describeThing(thingName);
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
```

```

 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to attach certificate to Thing. HTTP
Status Code: " +
 attachResponse.sdkHttpResponse().statusCode());
 }
}
});

future.join();
}

```

- For API details, see [AttachThingPrincipal](#) in *AWS SDK for Java 2.x API Reference*.

## CreateKeysAndCertificate

The following code example shows how to use CreateKeysAndCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an IoT certificate asynchronously.
 *
 * @return The ARN of the created certificate.
 * <p>
 * This method initiates an asynchronous request to create an IoT certificate.
 * If the request is successful, it prints the certificate details and returns
the certificate ARN.
 * If an exception occurs, it prints the error message.
 */
public String createCertificate() {
 CompletableFuture<CreateKeysAndCertificateResponse> future =
getAsyncClient().createKeysAndCertificate();
 final String[] certificateArn = {null};
 future.whenComplete((response, ex) -> {

```

```

 if (response != null) {
 String certificatePem = response.certificatePem();
 certificateArn[0] = response.certificateArn();

 // Print the details.
 System.out.println("\nCertificate:");
 System.out.println(certificatePem);
 System.out.println("\nCertificate ARN:");
 System.out.println(certificateArn[0]);

 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
 });

 future.join();
 return certificateArn[0];
}

```

- For API details, see [CreateKeysAndCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## CreateThing

The following code example shows how to use CreateThing.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Creates an IoT Thing with the specified name asynchronously.
 *
 * @param thingName The name of the IoT Thing to create.
 *
 * This method initiates an asynchronous request to create an IoT Thing with the
 specified name.
 * If the request is successful, it prints the name of the thing and its ARN
 value.
 * If an exception occurs, it prints the error message.
 */
 public void createIoTThing(String thingName) {
 CreateThingRequest createThingRequest = CreateThingRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<CreateThingResponse> future =
 getAsyncClient().createThing(createThingRequest);
 future.whenComplete((createThingResponse, ex) -> {
 if (createThingResponse != null) {
 System.out.println(thingName + " was successfully created. The ARN
value is " + createThingResponse.thingArn());
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
 });

 future.join();
 }

```

- For API details, see [CreateThing](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTopicRule

The following code example shows how to use `CreateTopicRule`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates an IoT rule asynchronously.
 *
 * @param roleARN The ARN of the IAM role that grants access to the rule's
actions.
 * @param ruleName The name of the IoT rule.
 * @param action The ARN of the action to perform when the rule is triggered.
 *
 * This method initiates an asynchronous request to create an IoT rule.
 * If the request is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void createIoTRule(String roleARN, String ruleName, String action) {
 String sql = "SELECT * FROM '" + TOPIC + "'";
 SnsAction action1 = SnsAction.builder()
 .targetArn(action)
 .roleArn(roleARN)
 .build();

 // Create the action.
 Action myAction = Action.builder()
 .sns(action1)
 .build();

 // Create the topic rule payload.
 TopicRulePayload topicRulePayload = TopicRulePayload.builder()
 .sql(sql)
 .actions(myAction)
 .build();

 // Create the topic rule request.
 CreateTopicRuleRequest topicRuleRequest = CreateTopicRuleRequest.builder()
 .ruleName(ruleName)
 .topicRulePayload(topicRulePayload)
```

```

 .build();

 CompletableFuture<CreateTopicRuleResponse> future =
getAsyncClient().createTopicRule(topicRuleRequest);
 future.whenComplete((response, ex) -> {
 if (response != null) {
 System.out.println("IoT Rule created successfully.");
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to create IoT Rule.");
 }
 }
 });

 future.join();
 }

```

- For API details, see [CreateTopicRule](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCertificate

The following code example shows how to use DeleteCertificate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a certificate asynchronously.
 *

```



```

 * @param certificateArn The ARN of the certificate to delete.
 *
 * This method initiates an asynchronous request to delete a certificate.
 * If the deletion is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
 public void deleteCertificate(String certificateArn) {
 DeleteCertificateRequest certificateProviderRequest =
DeleteCertificateRequest.builder()
 .certificateId(extractCertificateId(certificateArn))
 .build();

 CompletableFuture<DeleteCertificateResponse> future =
getAsyncClient().deleteCertificate(certificateProviderRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println(certificateArn + " was successfully deleted.");
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });

 future.join();
 }

```

- For API details, see [DeleteCertificate](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteThing

The following code example shows how to use DeleteThing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing to delete.
 *
 * This method initiates an asynchronous request to delete an IoT Thing.
 * If the deletion is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void deleteIoTThing(String thingName) {
 DeleteThingRequest deleteThingRequest = DeleteThingRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<DeleteThingResponse> future =
getAsyncClient().deleteThing(deleteThingRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println("Deleted Thing " + thingName);
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });

 future.join();
}
```

- For API details, see [DeleteThing](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeEndpoint

The following code example shows how to use DescribeEndpoint.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Describes the endpoint of the IoT service asynchronously.
 *
 * @return A CompletableFuture containing the full endpoint URL.
 *
 * This method initiates an asynchronous request to describe the endpoint of the
 IoT service.
 * If the request is successful, it prints and returns the full endpoint URL.
 * If an exception occurs, it prints the error message.
 */
public String describeEndpoint() {
 CompletableFuture<DescribeEndpointResponse> future =
getAsyncClient().describeEndpoint(DescribeEndpointRequest.builder().endpointType("iot:Data-
ATS").build());
 final String[] result = {null};

 future.whenComplete((endpointResponse, ex) -> {
 if (endpointResponse != null) {
 String endpointUrl = endpointResponse.endpointAddress();
 String exString = getValue(endpointUrl);
 String fullEndpoint = "https://" + exString + "-ats.iot.us-
east-1.amazonaws.com";

 System.out.println("Full Endpoint URL: " + fullEndpoint);
 result[0] = fullEndpoint;
 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
```

```

 if (cause instanceof IoTException) {
 System.err.println(((IoTException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + cause.getMessage());
 }
 }
});

future.join();
return result[0];
}

```

- For API details, see [DescribeEndpoint](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeThing

The following code example shows how to use DescribeThing.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Describes an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to describe an IoT Thing.
 * If the request is successful, it prints the Thing details.
 * If an exception occurs, it prints the error message.
 */
private void describeThing(String thingName) {
 DescribeThingRequest thingRequest = DescribeThingRequest.builder()
 .thingName(thingName)
 .build();
}

```

```

 CompletableFuture<DescribeThingResponse> future =
getAsyncClient().describeThing(thingRequest);
 future.whenComplete((describeResponse, ex) -> {
 if (describeResponse != null) {
 System.out.println("Thing Details:");
 System.out.println("Thing Name: " + describeResponse.thingName());
 System.out.println("Thing ARN: " + describeResponse.thingArn());
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to describe Thing.");
 }
 }
 });

 future.join();
 }

```

- For API details, see [DescribeThing](#) in *AWS SDK for Java 2.x API Reference*.

## DetachThingPrincipal

The following code example shows how to use `DetachThingPrincipal`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Detaches a principal (certificate) from an IoT Thing asynchronously.

```

```

*
* @param thingName The name of the IoT Thing.
* @param certificateArn The ARN of the certificate to detach.
*
* This method initiates an asynchronous request to detach a certificate from an
IoT Thing.
* If the detachment is successful, it prints a confirmation message.
* If an exception occurs, it prints the error message.
*/
public void detachThingPrincipal(String thingName, String certificateArn) {
 DetachThingPrincipalRequest thingPrincipalRequest =
DetachThingPrincipalRequest.builder()
 .principal(certificateArn)
 .thingName(thingName)
 .build();

 CompletableFuture<DetachThingPrincipalResponse> future =
getAsyncClient().detachThingPrincipal(thingPrincipalRequest);
 future.whenComplete((voidResult, ex) -> {
 if (ex == null) {
 System.out.println(certificateArn + " was successfully removed from
" + thingName);
 } else {
 Throwable cause = ex.getCause();
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else {
 System.err.println("Unexpected error: " + ex.getMessage());
 }
 }
 });

 future.join();
}

```

- For API details, see [DetachThingPrincipal](#) in *AWS SDK for Java 2.x API Reference*.

## ListCertificates

The following code example shows how to use `ListCertificates`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Lists all certificates asynchronously.
 *
 * This method initiates an asynchronous request to list all certificates.
 * If the request is successful, it prints the certificate IDs and ARNs.
 * If an exception occurs, it prints the error message.
 */
public void listCertificates() {
 CompletableFuture<ListCertificatesResponse> future =
getAsyncClient().listCertificates();
 future.whenComplete((response, ex) -> {
 if (response != null) {
 List<Certificate> certList = response.certificates();
 for (Certificate cert : certList) {
 System.out.println("Cert id: " + cert.certificateId());
 System.out.println("Cert Arn: " + cert.certificateArn());
 }
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to list certificates.");
 }
 }
 });

 future.join();
}
```

- For API details, see [ListCertificates](#) in *AWS SDK for Java 2.x API Reference*.

## SearchIndex

The following code example shows how to use SearchIndex.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Searches for IoT Things asynchronously based on a query string.
 *
 * @param queryString The query string to search for Things.
 *
 * This method initiates an asynchronous request to search for IoT Things.
 * If the request is successful and Things are found, it prints their IDs.
 * If no Things are found, it prints a message indicating so.
 * If an exception occurs, it prints the error message.
 */
public void searchThings(String queryString) {
 SearchIndexRequest searchIndexRequest = SearchIndexRequest.builder()
 .queryString(queryString)
 .build();

 CompletableFuture<SearchIndexResponse> future =
getAsyncClient().searchIndex(searchIndexRequest);
 future.whenComplete((searchIndexResponse, ex) -> {
 if (searchIndexResponse != null) {
 // Process the result.
 if (searchIndexResponse.things().isEmpty()) {
 System.out.println("No things found.");
 } else {
 searchIndexResponse.things().forEach(thing ->
System.out.println("Thing id found using search is " + thing.thingId()));
 }
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
```



```
 if (cause instanceof IoTException) {
 System.err.println(((IoTException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to search for IoT Things.");
 }
 }
});

future.join();
}
```

- For API details, see [SearchIndex](#) in *AWS SDK for Java 2.x API Reference*.

## AWS IoT data examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS IoT data.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### GetThingShadow

The following code example shows how to use `GetThingShadow`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the payload of a Thing's shadow asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to get the payload of a Thing's
shadow.
 * If the request is successful, it prints the shadow data.
 * If an exception occurs, it prints the error message.
 */
public void getPayload(String thingName) {
 GetThingShadowRequest getThingShadowRequest =
GetThingShadowRequest.builder()
 .thingName(thingName)
 .build();

 CompletableFuture<GetThingShadowResponse> future =
getAsyncDataPlaneClient().getThingShadow(getThingShadowRequest);
 future.whenComplete((getThingShadowResponse, ex) -> {
 if (getThingShadowResponse != null) {
 // Extracting payload from response.
 SdkBytes payload = getThingShadowResponse.payload();
 String payloadString = payload.asUtf8String();
 System.out.println("Received Shadow Data: " + payloadString);
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to get Thing Shadow payload.");
 }
 }
 });
}
```

```

 }
 });

 future.join();
}

```

- For API details, see [GetThingShadow](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateThingShadow

The following code example shows how to use UpdateThingShadow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Updates the shadow of an IoT Thing asynchronously.
 *
 * @param thingName The name of the IoT Thing.
 *
 * This method initiates an asynchronous request to update the shadow of an IoT
Thing.
 * If the request is successful, it prints a confirmation message.
 * If an exception occurs, it prints the error message.
 */
public void updateShadowThing(String thingName) {
 // Create Thing Shadow State Document.
 String stateDocument = "{\"state\":{\"reported\":{\"temperature\":25,
 \"humidity\":50}}}\"";
 SdkBytes data = SdkBytes.fromString(stateDocument, StandardCharsets.UTF_8);
 UpdateThingShadowRequest updateThingShadowRequest =
UpdateThingShadowRequest.builder()
 .thingName(thingName)
 .payload(data)
 .build();

```

```

 CompletableFuture<UpdateThingShadowResponse> future =
getAsyncDataPlaneClient().updateThingShadow(updateThingShadowRequest);
 future.whenComplete((updateResponse, ex) -> {
 if (updateResponse != null) {
 System.out.println("Thing Shadow updated successfully.");
 } else {
 Throwable cause = ex != null ? ex.getCause() : null;
 if (cause instanceof IotException) {
 System.err.println(((IotException)
cause).awsErrorDetails().errorMessage());
 } else if (cause != null) {
 System.err.println("Unexpected error: " + cause.getMessage());
 } else {
 System.err.println("Failed to update Thing Shadow.");
 }
 }
 });

 future.join();
 }

```

- For API details, see [UpdateThingShadow](#) in *AWS SDK for Java 2.x API Reference*.

## AWS IoT SiteWise examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS IoT SiteWise.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

# Get started

## Hello AWS IoT SiteWise

The following code example shows how to get started using AWS IoT SiteWise.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class HelloSitewise {
 private static final Logger logger =
 LoggerFactory.getLogger(HelloSitewise.class);
 public static void main(String[] args) {
 fetchAssetModels();
 }

 /**
 * Fetches asset models using the provided {@link IoTSiteWiseAsyncClient}.
 */
 public static void fetchAssetModels() {
 IoTSiteWiseAsyncClient siteWiseAsyncClient =
 IoTSiteWiseAsyncClient.create();
 ListAssetModelsRequest assetModelsRequest = ListAssetModelsRequest.builder()
 .assetModelTypes(AssetModelType.ASSET_MODEL)
 .build();

 // Asynchronous paginator - process paginated results.
 ListAssetModelsPublisher listModelsPaginator =
 siteWiseAsyncClient.listAssetModelsPaginator(assetModelsRequest);
 CompletableFuture<Void> future = listModelsPaginator.subscribe(response -> {
 response.assetModelSummaries().forEach(assetSummary ->
 logger.info("Asset Model Name: {} ", assetSummary.name())
);
 });

 // Wait for the asynchronous operation to complete
 future.join();
 }
}
```

```
}
}
```

- For API details, see [ListAssetModels](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an AWS IoT SiteWise Asset Model.
- Create an AWS IoT SiteWise Asset.
- Retrieve the property ID values.
- Send data to an AWS IoT SiteWise Asset.
- Retrieve the value of the AWS IoT SiteWise Asset property.
- Create an AWS IoT SiteWise Portal.
- Create an AWS IoT SiteWise Gateway.
- Describe the AWS IoT SiteWise Gateway.
- Delete the AWS IoT SiteWise Assets.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating AWS IoT SiteWise features.

```
public class SitewiseScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");
```

```

private static final Logger logger =
LoggerFactory.getLogger(SitewiseScenario.class);
static Scanner scanner = new Scanner(System.in);

private static final String ROLES_STACK = "RoleSitewise";

static SitewiseActions sitewiseActions = new SitewiseActions();

public static void main(String[] args) throws Throwable {
 Scanner scanner = new Scanner(System.in);
 String contactEmail = "user@mydomain.com"; // Change email address.
 String assetModelName = "MyAssetModel1";
 String assetName = "MyAsset1" ;
 String portalName = "MyPortal1" ;
 String gatewayName = "MyGateway1" ;
 String myThing = "MyThing1" ;

 logger.info("""
 AWS IoT SiteWise is a fully managed software-as-a-service (SaaS) that
 makes it easy to collect, store, organize, and monitor data from
industrial equipment and processes.
 It is designed to help industrial and manufacturing organizations
collect data from their equipment and
 processes, and use that data to make informed decisions about their
operations.

 One of the key features of AWS IoT SiteWise is its ability to connect to
a wide range of industrial
 equipment and systems, including programmable logic controllers (PLCs),
sensors, and other
 industrial devices. It can collect data from these devices and organize
it into a unified data model,
 making it easier to analyze and gain insights from the data. AWS IoT
SiteWise also provides tools for
 visualizing the data, setting up alarms and alerts, and generating
reports.

 Another key feature of AWS IoT SiteWise is its ability to scale to
handle large volumes of data.
 It can collect and store data from thousands of devices and process
millions of data points per second,
 making it suitable for large-scale industrial operations. Additionally,
AWS IoT SiteWise is designed

```

to be secure and compliant, with features like role-based access controls, data encryption, and integration with other AWS services for additional security and compliance features.

Let's get started...  
 """);

```
waitForInputToContinue(scanner);
logger.info(DASHES);
```

```
try {
 runScenario(assetModelName, assetName, portalName, contactEmail,
gatewayName, myThing);
} catch (RuntimeException e) {
 logger.info(e.getMessage());
}
}
```

```
public static void runScenario(String assetModelName, String assetName, String
portalName, String contactEmail, String gatewayName, String myThing) throws
Throwable {
```

```
 logger.info("Use AWS CloudFormation to create an IAM role that is required
for this scenario.");
```

```
 CloudFormationHelper.deployCloudFormationStack(ROLES_STACK);
```

```
 Map<String, String> stackOutputs =
```

```
CloudFormationHelper.getStackOutputsAsync(ROLES_STACK).join();
```

```
 String iamRole = stackOutputs.get("SitewiseRoleArn");
```

```
 logger.info("The ARN of the IAM role is {}", iamRole);
```

```
 logger.info(DASHES);
```

```
 logger.info(DASHES);
```

```
 logger.info("1. Create an AWS SiteWise Asset Model");
```

```
 logger.info("""
```

An AWS IoT SiteWise Asset Model is a way to represent the physical assets, such as equipment,

processes, and systems, that exist in an industrial environment. This model provides a structured and

hierarchical representation of these assets, allowing users to define the relationships and properties of each asset.

This scenario creates two asset model properties: temperature and humidity.



```

 """);
 waitForInputToContinue(scanner);
 String assetModelId = null;
 try {
 CreateAssetModelResponse response =
sitewiseActions.createAssetModelAsync(assetModelName).join();
 assetModelId = response.assetModelId();
 logger.info("Asset Model successfully created. Asset Model ID: {}. ",
assetModelId);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceAlreadyExistsException) {
 try {
 assetModelId =
sitewiseActions.getAssetModelIdAsync(assetModelName).join();
 logger.info("The Asset Model {} already exists. The id of the
existing model is {}. Moving on...", assetModelName, assetModelId);
 } catch (CompletionException cex) {
 logger.error("Exception thrown acquiring the asset model id:
{}", cex.getCause().getCause(), cex);
 return;
 }
 } else {
 logger.info("An unexpected error occurred: " + cause.getMessage(),
cause);
 return;
 }
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("2. Create an AWS IoT SiteWise Asset");
 logger.info("""
 The IoT SiteWise model that we just created defines the structure and
 metadata for your physical assets.
 Now we create an asset from the asset model.

 """);
 logger.info("Let's wait 30 seconds for the asset to be ready.");
 countdown(30);
 waitForInputToContinue(scanner);
 String assetId;
 try {

```

```

 CreateAssetResponse response =
sitewiseActions.createAssetAsync(assetName, assetModelId).join();
 assetId = response.assetId();
 logger.info("Asset created with ID: {}", assetId);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The asset model id was not found: {}",
cause.getMessage(), cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Retrieve the property ID values");
 logger.info(""""
 To send data to an asset, we need to get the property ID values. In
this scenario, we access the
 temperature and humidity property ID values.
 """);
 waitForInputToContinue(scanner);
 Map<String, String> propertyIds = null;
 try {
 propertyIds = sitewiseActions.getPropertyIds(assetModelId).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof IoTSiteWiseException) {
 logger.error("IoTSiteWiseException occurred: {}",
cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 return;
 }
 String humPropId = propertyIds.get("Humidity");
 logger.info("The Humidity property Id is {}", humPropId);
 String tempPropId = propertyIds.get("Temperature");
 logger.info("The Temperature property Id is {}", tempPropId);

```

```

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("4. Send data to an AWS IoT SiteWise Asset");
 logger.info("""
 By sending data to an IoT SiteWise Asset, you can aggregate data from
 multiple sources, normalize the data into a standard format, and store
it in a
 centralized location. This makes it easier to analyze and gain insights
from the data.

```

In this example, we generate sample temperature and humidity data and send it to the AWS IoT SiteWise asset.

```

 """);
 waitForInputToContinue(scanner);
 try {
 sitewiseActions.sendDataToSiteWiseAsync(assetId, tempPropId,
humPropId).join();
 logger.info("Data sent successfully.");
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The AWS resource was not found: {}",
cause.getMessage(), cause);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("5. Retrieve the value of the IoT SiteWise Asset property");
 logger.info("""
 IoT SiteWise is an AWS service that allows you to collect, process, and
analyze industrial data
 from connected equipment and sensors. One of the key benefits of reading
an IoT SiteWise property
 is the ability to gain valuable insights from your industrial data.

```

```

 """);
 waitForInputToContinue(scanner);
 try {
 Double assetVal = sitewiseActions.getAssetPropValueAsync(tempPropId,
assetId).join();
 logger.info("The property name is: {}", "Temperature");
 logger.info("The value of this property is: {}", assetVal);

 waitForInputToContinue(scanner);

 assetVal = sitewiseActions.getAssetPropValueAsync(humPropId,
assetId).join();
 logger.info("The property name is: {}", "Humidity");
 logger.info("The value of this property is: {}", assetVal);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The AWS resource was not found: {}",
cause.getMessage(), cause);
 } else {
 logger.info("An unexpected error occurred: {}",
cause.getMessage(), cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("6. Create an IoT SiteWise Gateway");
 logger.info(
 """"
 IoT SiteWise Gateway serves as the bridge between industrial
equipment, sensors, and the
 cloud-based IoT SiteWise service. It is responsible for securely
collecting, processing, and
 transmitting data from various industrial assets to the IoT SiteWise
platform,
 enabling real-time monitoring, analysis, and optimization of
industrial operations.

 """);
 waitForInputToContinue(scanner);

```

```

 String gatewayId = "";
 try {
 gatewayId = sitewiseActions.createGatewayAsync(gatewayName,
myThing).join();
 logger.info("Gateway creation completed successfully. id is {}",
gatewayId);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof IoTSiteWiseException siteWiseEx) {
 logger.error("IoT SiteWise error occurred: Error message: {}", Error
code {}",
 siteWiseEx.getMessage(),
siteWiseEx.awsErrorDetails().errorCode(), siteWiseEx);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 }
 return;
 }
 logger.info(DASHES);
 logger.info(DASHES);

 logger.info("7. Describe the IoT SiteWise Gateway");
 waitForInputToContinue(scanner);
 try {
 sitewiseActions.describeGatewayAsync(gatewayId)
 .thenAccept(response -> {
 logger.info("Gateway Name: {}", response.gatewayName());
 logger.info("Gateway ARN: {}", response.gatewayArn());
 logger.info("Gateway Platform: {}", response.gatewayPlatform());
 logger.info("Gateway Creation Date: {}",
response.creationDate());
 }).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException notFoundException) {
 logger.error("A ResourceNotFoundException occurred: Error message:
{}", Error code {}",
 notFoundException.getMessage(),
notFoundException.awsErrorDetails().errorCode(), notFoundException);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 }
 }

```

```

 return;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("8. Delete the AWS IoT SiteWise Assets");
 logger.info(
 ""
 Before you can delete the Asset Model, you must delete the assets.

 "");
 logger.info("Would you like to delete the IoT SiteWise Assets? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 logger.info("You selected to delete the SiteWise assets.");

 try {
 sitewiseActions.deleteGatewayAsync(gatewayId).join();
 logger.info("Gateway {} was deleted successfully.", gatewayId);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException notFoundException) {
 logger.error("A ResourceNotFoundException occurred: Error
message: {}, Error code {}",
 notFoundException.getMessage(),
notFoundException.awsErrorDetails().errorCode(), notFoundException);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 }
 }

 try {
 sitewiseActions.deleteAssetAsync(assetId).join();
 logger.info("Request to delete asset {} sent successfully",
assetId);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException notFoundException) {
 logger.error("A ResourceNotFoundException occurred: Error
message: {}, Error code {}",
 notFoundException.getMessage(),
notFoundException.awsErrorDetails().errorCode(), notFoundException);
 } else {

```

```

 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 }
}
logger.info("Let's wait 1 minute for the asset to be deleted.");
countdown(60);
waitForInputToContinue(scanner);
logger.info("Delete the AWS IoT SiteWise Asset Model");
try {
 sitewiseActions.deleteAssetModelAsync(assetModelId).join();
 logger.info("Asset model deleted successfully.");
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException notFoundException) {
 logger.error("A ResourceNotFoundException occurred: Error
message: {}, Error code {}",
 notFoundException.getMessage(),
notFoundException.awsErrorDetails().errorCode(), notFoundException);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage());
 }
}
waitForInputToContinue(scanner);

} else {
 logger.info("The resources will not be deleted.");
}
logger.info(DASHES);

logger.info(DASHES);
CloudFormationHelper.destroyCloudFormationStack(ROLES_STACK);
logger.info("This concludes the AWS IoT SiteWise Scenario");
logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");

```

```

 logger.info("");
 break;
 } else {
 logger.info("Invalid input. Please try again.");
 }
}

}

public static void countdown(int totalSeconds) throws InterruptedException {
 for (int i = totalSeconds; i >= 0; i--) {
 int displayMinutes = i / 60;
 int displaySeconds = i % 60;
 System.out.printf("\r%02d:%02d", displayMinutes, displaySeconds);
 Thread.sleep(1000); // Wait for 1 second
 }
 System.out.println(); // Move to the next line after countdown
 logger.info("Countdown complete!");
}
}

```

A wrapper class for AWS IoT SiteWise SDK methods.

```

public class SitewiseActions {

 private static final Logger logger =
 LoggerFactory.getLogger(SitewiseActions.class);

 private static IoTSiteWiseAsyncClient ioTSiteWiseAsyncClient;

 private static IoTSiteWiseAsyncClient getAsyncClient() {
 if (ioTSiteWiseAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))

```



```

 .retryStrategy(RetryMode.STANDARD)
 .build();

 ioTSiteWiseAsyncClient = IoTSiteWiseAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return ioTSiteWiseAsyncClient;
}

/**
 * Creates an asset model.
 *
 * @param name the name of the asset model to create.
 * @return a {@link CompletableFuture} that represents a {@link
CreateAssetModelResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it
 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<CreateAssetModelResponse> createAssetModelAsync(String
name) {
 PropertyType humidity = PropertyType.builder()
 .measurement(Measurement.builder().build())
 .build();

 PropertyType temperaturePropertyType = PropertyType.builder()
 .measurement(Measurement.builder().build())
 .build();

 AssetModelPropertyDefinition temperatureProperty =
AssetModelPropertyDefinition.builder()
 .name("Temperature")
 .dataType(PropertyDataType.DOUBLE)
 .type(temperaturePropertyType)

```

```

 .build();

 AssetModelPropertyDefinition humidityProperty =
AssetModelPropertyDefinition.builder()
 .name("Humidity")
 .dataType(PropertyDataType.DOUBLE)
 .type(humidity)
 .build();

 CreateAssetModelRequest createAssetModelRequest =
CreateAssetModelRequest.builder()
 .assetModelName(name)
 .assetModelDescription("This is my asset model")
 .assetModelProperties(temperatureProperty, humidityProperty)
 .build();

 return getAsyncClient().createAssetModel(createAssetModelRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to create asset model: {} ",
exception.getCause().getMessage());
 }
 });
 }

}

/**
 * Creates an asset with the specified name and asset model Id.
 *
 * @param assetName the name of the asset to create.
 * @param assetModelId the Id of the asset model to associate with the asset.
 * @return a {@link CompletableFuture} that represents a {@link
CreateAssetResponse} result. The calling code can
 * attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it
 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */

```

```

 public CompletableFuture<CreateAssetResponse> createAssetAsync(String assetName,
String assetModelId) {
 CreateAssetRequest createAssetRequest = CreateAssetRequest.builder()
 .assetModelId(assetModelId)
 .assetDescription("Created using the AWS SDK for Java")
 .assetName(assetName)
 .build();

 return getAsyncClient().createAsset(createAssetRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to create asset: {}",
exception.getCause().getMessage());
 }
 });
 }

 /**
 * Sends data to the SiteWise service.
 *
 * @param assetId the ID of the asset to which the data will be sent.
 * @param tempPropertyId the ID of the temperature property.
 * @param humidityPropId the ID of the humidity property.
 * @return a {@link CompletableFuture} that represents a {@link
BatchPutAssetPropertyValueResponse} result. The
 * calling code can attach callbacks, then handle the result or
exception by calling
 * {@link CompletableFuture#join()} or {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it
 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<BatchPutAssetPropertyValueResponse>
sendDataToSiteWiseAsync(String assetId, String tempPropertyId, String
humidityPropId) {
 Map<String, Double> sampleData = generateSampleData();
 long timestamp = Instant.now().toEpochMilli();

 TimeInNanos time = TimeInNanos.builder()
 .timeInSeconds(timestamp / 1000)

```

```

 .offsetInNanos((int) ((timestamp % 1000) * 1000000))
 .build();

 BatchPutAssetPropertyValueRequest request =
 BatchPutAssetPropertyValueRequest.builder()
 .entries(Arrays.asList(
 PutAssetPropertyValueEntry.builder()
 .entryId("entry-3")
 .assetId(assetId)
 .propertyId(tempPropertyId)
 .propertyValues(Arrays.asList(
 AssetPropertyValue.builder()
 .value(Variant.builder()
 .doubleValue(sampleData.get("Temperature"))
 .build())
 .timestamp(time)
 .build()
))
 .build(),
 PutAssetPropertyValueEntry.builder()
 .entryId("entry-4")
 .assetId(assetId)
 .propertyId(humidityPropId)
 .propertyValues(Arrays.asList(
 AssetPropertyValue.builder()
 .value(Variant.builder()
 .doubleValue(sampleData.get("Humidity"))
 .build())
 .timestamp(time)
 .build()
))
 .build()
))
 .build();

 return getAsyncClient().batchPutAssetPropertyValue(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An exception occurred: {}",
exception.getCause().getMessage());
 }
 });
}

```

```

/**
 * Fetches the value of an asset property.
 *
 * @param propId the ID of the asset property to fetch.
 * @param assetId the ID of the asset to fetch the property value for.
 * @return a {@link CompletableFuture} that represents a {@link Double} result.
The calling code can attach
 * callbacks, then handle the result or exception by calling {@link
CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<Double> getAssetPropValueAsync(String propId, String
assetId) {
 GetAssetPropertyValueRequest assetPropertyValueRequest =
GetAssetPropertyValueRequest.builder()
 .propertyId(propId)
 .assetId(assetId)
 .build();

 return getAsyncClient().getAssetPropertyValue(assetPropertyValueRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.error("Error occurred while fetching property value:
{}.", exception.getCause().getMessage());
 throw (CompletionException) exception;
 }
 return response.propertyValue().value().doubleValue();
 });
}

/**
 * Retrieves the property IDs associated with a specific asset model.
 *
 * @param assetModelId the ID of the asset model that defines the properties.
 * @return a {@link CompletableFuture} that represents a {@link Map} result that
associates the property name to the

```

```

 * propert ID. The calling code can attach callbacks, then handle the
result or exception by calling
 * {@link CompletableFuture#join()} or {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<Map<String, String>> getPropertyIds(String
assetModelId) {
 ListAssetModelPropertiesRequest modelPropertiesRequest =
ListAssetModelPropertiesRequest.builder().assetModelId(assetModelId).build();
 return getAsyncClient().listAssetModelProperties(modelPropertiesRequest)
 .handle((response, throwable) -> {
 if (response != null) {
 return response.assetModelPropertySummaries().stream()
 .collect(Collectors
 .toMap(AssetModelPropertySummary::name,
AssetModelPropertySummary::id));
 } else {
 logger.error("Error occurred while fetching property IDs: {}.",
throwable.getCause().getMessage());
 throw (CompletionException) throwable;
 }
 });
 }

/**
 * Deletes an asset.
 *
 * @param assetId the ID of the asset to be deleted.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteAssetResponse} result. The calling code can
 * attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling

```

```

 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<DeleteAssetResponse> deleteAssetAsync(String assetId) {
 DeleteAssetRequest deleteAssetRequest = DeleteAssetRequest.builder()
 .assetId(assetId)
 .build();

 return getAsyncClient().deleteAsset(deleteAssetRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An error occurred deleting asset with id: {}",
assetId);
 }
 });
 }

 /**
 * Deletes an Asset Model with the specified ID.
 *
 * @param assetModelId the ID of the Asset Model to delete.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteAssetModelResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<DeleteAssetModelResponse> deleteAssetModelAsync(String
assetModelId) {
 DeleteAssetModelRequest deleteAssetModelRequest =
DeleteAssetModelRequest.builder()
 .assetModelId(assetModelId)
 .build();

 return getAsyncClient().deleteAssetModel(deleteAssetModelRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```

```

 logger.error("Failed to delete asset model with ID:{}",
exception.getMessage());
 }
 });
}

/**
 * Retrieves the asset model ID for the given asset model name.
 *
 * @param assetModelName the name of the asset model for the ID.
 * @return a {@link CompletableFuture} that represents a {@link String} result
of the asset model ID or null if the
 * asset model cannot be found. The calling code can attach callbacks,
then handle the result or exception
 * by calling {@link CompletableFuture#join()} or {@link
CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<String> getAssetModelIdAsync(String assetModelName) {
 ListAssetModelsRequest listAssetModelsRequest =
ListAssetModelsRequest.builder().build();
 return getAsyncClient().listAssetModels(listAssetModelsRequest)
 .handle((listAssetModelsResponse, exception) -> {
 if (exception != null) {
 logger.error("Failed to retrieve Asset Model ID: {}",
exception.getCause().getMessage());
 throw (CompletionException) exception;
 }
 for (AssetModelSummary assetModelSummary :
listAssetModelsResponse.assetModelSummaries()) {
 if (assetModelSummary.name().equals(assetModelName)) {
 return assetModelSummary.id();
 }
 }
 return null;
 });
}

```



```

/**
 * Creates a new IoT Sitewise gateway.
 *
 * @param gatewayName The name of the gateway to create.
 * @param myThing The name of the core device thing to associate with the
gateway.
 * @return a {@link CompletableFuture} that represents a {@link String} result
of the gateways ID. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<String> createGatewayAsync(String gatewayName, String
myThing) {
 GreengrassV2 gg = GreengrassV2.builder()
 .coreDeviceThingName(myThing)
 .build();

 GatewayPlatform platform = GatewayPlatform.builder()
 .greengrassV2(gg)
 .build();

 Map<String, String> tag = new HashMap<>();
 tag.put("Environment", "Production");

 CreateGatewayRequest createGatewayRequest = CreateGatewayRequest.builder()
 .gatewayName(gatewayName)
 .gatewayPlatform(platform)
 .tags(tag)
 .build();

 return getAsyncClient().createGateway(createGatewayRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.error("Error creating the gateway.");
 throw (CompletionException) exception;
 }
 });
}

```

```

 }
 logger.info("The ARN of the gateway is {}" ,
response.gatewayArn());
 return response.gatewayId();
 });
}

/**
 * Deletes the specified gateway.
 *
 * @param gatewayId the ID of the gateway to delete.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteGatewayResponse} result.. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<DeleteGatewayResponse> deleteGatewayAsync(String
gatewayId) {
 DeleteGatewayRequest deleteGatewayRequest = DeleteGatewayRequest.builder()
 .gatewayId(gatewayId)
 .build();

 return getAsyncClient().deleteGateway(deleteGatewayRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to delete gateway: {}",
exception.getCause().getMessage());
 }
 });
}

/**
 * Describes the specified gateway.
 *
 * @param gatewayId the ID of the gateway to describe.

```

```

 * @return a {@link CompletableFuture} that represents a {@link
DescribeGatewayResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<DescribeGatewayResponse> describeGatewayAsync(String
gatewayId) {
 DescribeGatewayRequest request = DescribeGatewayRequest.builder()
 .gatewayId(gatewayId)
 .build();

 return getAsyncClient().describeGateway(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An error occurred during the describeGateway
method: {}", exception.getCause().getMessage());
 }
 });
 }

 private static Map<String, Double> generateSampleData() {
 Map<String, Double> data = new HashMap<>();
 data.put("Temperature", 23.5);
 data.put("Humidity", 65.0);
 return data;
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [BatchPutAssetPropertyValue](#)
  - [CreateAsset](#)
  - [CreateAssetModel](#)
  - [CreateGateway](#)

- [DeleteAsset](#)
- [DeleteAssetModel](#)
- [DeleteGateway](#)
- [DescribeAssetModel](#)
- [DescribeGateway](#)
- [GetAssetPropertyValue](#)
- [ListAssetModelProperties](#)
- [ListAssetModels](#)

## Actions

### BatchPutAssetPropertyValue

The following code example shows how to use `BatchPutAssetPropertyValue`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Sends data to the SiteWise service.
 *
 * @param assetId the ID of the asset to which the data will be sent.
 * @param tempPropertyId the ID of the temperature property.
 * @param humidityPropId the ID of the humidity property.
 * @return a {@link CompletableFuture} that represents a {@link
BatchPutAssetPropertyValueResponse} result. The
 * calling code can attach callbacks, then handle the result or
exception by calling
 * {@link CompletableFuture#join()} or {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it
```

```

 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<BatchPutAssetPropertyValueResponse>
sendDataToSiteWiseAsync(String assetId, String tempPropertyId, String
humidityPropId) {
 Map<String, Double> sampleData = generateSampleData();
 long timestamp = Instant.now().toEpochMilli();

 TimeInNanos time = TimeInNanos.builder()
 .timeInSeconds(timestamp / 1000)
 .offsetInNanos((int) ((timestamp % 1000) * 1000000))
 .build();

 BatchPutAssetPropertyValueRequest request =
BatchPutAssetPropertyValueRequest.builder()
 .entries(Arrays.asList(
 PutAssetPropertyValueEntry.builder()
 .entryId("entry-3")
 .assetId(assetId)
 .propertyId(tempPropertyId)
 .propertyValues(Arrays.asList(
 AssetPropertyValue.builder()
 .value(Variant.builder()
 .doubleValue(sampleData.get("Temperature"))
 .build())
 .timestamp(time)
 .build()
))
 .build(),
 PutAssetPropertyValueEntry.builder()
 .entryId("entry-4")
 .assetId(assetId)
 .propertyId(humidityPropId)
 .propertyValues(Arrays.asList(
 AssetPropertyValue.builder()
 .value(Variant.builder()
 .doubleValue(sampleData.get("Humidity"))
 .build())
 .timestamp(time)
 .build()
))
 .build()
))
 }

```

```

 .build()
))
 .build();

 return getAsyncClient().batchPutAssetPropertyValue(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An exception occurred: {}",
exception.getCause().getMessage());
 }
 });
}

```

- For API details, see [BatchPutAssetPropertyValue](#) in *AWS SDK for Java 2.x API Reference*.

## CreateAsset

The following code example shows how to use CreateAsset.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an asset with the specified name and asset model Id.
 *
 * @param assetName the name of the asset to create.
 * @param assetModelId the Id of the asset model to associate with the asset.
 * @return a {@link CompletableFuture} that represents a {@link
CreateAssetResponse} result. The calling code can
 * attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it

```

```

 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<CreateAssetResponse> createAssetAsync(String assetName,
String assetModelId) {
 CreateAssetRequest createAssetRequest = CreateAssetRequest.builder()
 .assetModelId(assetModelId)
 .assetDescription("Created using the AWS SDK for Java")
 .assetName(assetName)
 .build();

 return getAsyncClient().createAsset(createAssetRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to create asset: {}",
exception.getCause().getMessage());
 }
 });
 }
}

```

- For API details, see [CreateAsset](#) in *AWS SDK for Java 2.x API Reference*.

## CreateAssetModel

The following code example shows how to use `CreateAssetModel`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an asset model.
 *
 * @param name the name of the asset model to create.

```

```

 * @return a {@link CompletableFuture} that represents a {@link
CreateAssetModelResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps it
 * available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<CreateAssetModelResponse> createAssetModelAsync(String
name) {
 PropertyType humidity = PropertyType.builder()
 .measurement(Measurement.builder().build())
 .build();

 PropertyType temperaturePropertyType = PropertyType.builder()
 .measurement(Measurement.builder().build())
 .build();

 AssetModelPropertyDefinition temperatureProperty =
AssetModelPropertyDefinition.builder()
 .name("Temperature")
 .dataType(PropertyDataType.DOUBLE)
 .type(temperaturePropertyType)
 .build();

 AssetModelPropertyDefinition humidityProperty =
AssetModelPropertyDefinition.builder()
 .name("Humidity")
 .dataType(PropertyDataType.DOUBLE)
 .type(humidity)
 .build();

 CreateAssetModelRequest createAssetModelRequest =
CreateAssetModelRequest.builder()
 .assetModelName(name)
 .assetModelDescription("This is my asset model")
 .assetModelProperties(temperatureProperty, humidityProperty)
 .build();

```



```

 return getAsyncClient().createAssetModel(createAssetModelRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to create asset model: {} ",
exception.getCause().getMessage());
 }
 });
 }
}

```

- For API details, see [CreateAssetModel](#) in *AWS SDK for Java 2.x API Reference*.

## CreateGateway

The following code example shows how to use CreateGateway.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new IoT Sitewise gateway.
 *
 * @param gatewayName The name of the gateway to create.
 * @param myThing The name of the core device thing to associate with the
gateway.
 * @return a {@link CompletableFuture} that represents a {@link String} result
of the gateways ID. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling

```

```

 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<String> createGatewayAsync(String gatewayName, String
myThing) {
 GreengrassV2 gg = GreengrassV2.builder()
 .coreDeviceThingName(myThing)
 .build();

 GatewayPlatform platform = GatewayPlatform.builder()
 .greengrassV2(gg)
 .build();

 Map<String, String> tag = new HashMap<>();
 tag.put("Environment", "Production");

 CreateGatewayRequest createGatewayRequest = CreateGatewayRequest.builder()
 .gatewayName(gatewayName)
 .gatewayPlatform(platform)
 .tags(tag)
 .build();

 return getAsyncClient().createGateway(createGatewayRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.error("Error creating the gateway.");
 throw (CompletionException) exception;
 }
 logger.info("The ARN of the gateway is {}" ,
response.gatewayArn());
 return response.gatewayId();
 });
 }

```

- For API details, see [CreateGateway](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAsset

The following code example shows how to use DeleteAsset.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an asset.
 *
 * @param assetId the ID of the asset to be deleted.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteAssetResponse} result. The calling code can
 * attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<DeleteAssetResponse> deleteAssetAsync(String assetId) {
 DeleteAssetRequest deleteAssetRequest = DeleteAssetRequest.builder()
 .assetId(assetId)
 .build();

 return getAsyncClient().deleteAsset(deleteAssetRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An error occurred deleting asset with id: {}",
assetId);
 }
 });
}
```

- For API details, see [DeleteAsset](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAssetModel

The following code example shows how to use DeleteAssetModel.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an Asset Model with the specified ID.
 *
 * @param assetModelId the ID of the Asset Model to delete.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteAssetModelResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
 * {@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 *
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<DeleteAssetModelResponse> deleteAssetModelAsync(String
assetModelId) {
 DeleteAssetModelRequest deleteAssetModelRequest =
DeleteAssetModelRequest.builder()
 .assetModelId(assetModelId)
 .build();

 return getAsyncClient().deleteAssetModel(deleteAssetModelRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("Failed to delete asset model with ID:{}",
exception.getMessage());
 }
 });
}
```

```
}
```

- For API details, see [DeleteAssetModel](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteGateway

The following code example shows how to use DeleteGateway.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes the specified gateway.
 *
 * @param gatewayId the ID of the gateway to delete.
 * @return a {@link CompletableFuture} that represents a {@link
DeleteGatewayResponse} result.. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 *
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<DeleteGatewayResponse> deleteGatewayAsync(String
gatewayId) {
 DeleteGatewayRequest deleteGatewayRequest = DeleteGatewayRequest.builder()
 .gatewayId(gatewayId)
 .build();

 return getAsyncClient().deleteGateway(deleteGatewayRequest)
 .whenComplete((response, exception) -> {
```

```

 if (exception != null) {
 logger.error("Failed to delete gateway: {}",
exception.getCause().getMessage());
 }
 });
}

```

- For API details, see [DeleteGateway](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAssetModel

The following code example shows how to use `DescribeAssetModel`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the property IDs associated with a specific asset model.
 *
 * @param assetModelId the ID of the asset model that defines the properties.
 * @return a {@link CompletableFuture} that represents a {@link Map} result that
associates the property name to the
 * propert ID. The calling code can attach callbacks, then handle the
result or exception by calling
 * {@link CompletableFuture#join()} or {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<Map<String, String>> getPropertyIds(String
assetModelId) {

```

```

 ListAssetModelPropertiesRequest modelPropertiesRequest =
ListAssetModelPropertiesRequest.builder().assetModelId(assetModelId).build();
 return getAsyncClient().listAssetModelProperties(modelPropertiesRequest)
 .handle((response, throwable) -> {
 if (response != null) {
 return response.assetModelPropertySummaries().stream()
 .collect(Collectors
 .toMap(AssetModelPropertySummary::name,
AssetModelPropertySummary::id));
 } else {
 logger.error("Error occurred while fetching property IDs: {}.",
throwable.getCause().getMessage());
 throw (CompletionException) throwable;
 }
 });
 }
}

```

- For API details, see [DescribeAssetModel](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeGateway

The following code example shows how to use DescribeGateway.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Describes the specified gateway.
 *
 * @param gatewayId the ID of the gateway to describe.
 * @return a {@link CompletableFuture} that represents a {@link
DescribeGatewayResponse} result. The calling code
 * can attach callbacks, then handle the result or exception by calling
{@link CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.

```

```

 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<DescribeGatewayResponse> describeGatewayAsync(String
gatewayId) {
 DescribeGatewayRequest request = DescribeGatewayRequest.builder()
 .gatewayId(gatewayId)
 .build();

 return getAsyncClient().describeGateway(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 logger.error("An error occurred during the describeGateway
method: {}", exception.getCause().getMessage());
 }
 });
 }
}

```

- For API details, see [DescribeGateway](#) in *AWS SDK for Java 2.x API Reference*.

## GetAssetPropertyValue

The following code example shows how to use `GetAssetPropertyValue`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Fetches the value of an asset property.
 *

```



```

 * @param propId the ID of the asset property to fetch.
 * @param assetId the ID of the asset to fetch the property value for.
 * @return a {@link CompletableFuture} that represents a {@link Double} result.
The calling code can attach
 * callbacks, then handle the result or exception by calling {@link
CompletableFuture#join()} or
 * {@link CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
 public CompletableFuture<Double> getAssetPropValueAsync(String propId, String
assetId) {
 GetAssetPropertyValueRequest assetPropertyValueRequest =
GetAssetPropertyValueRequest.builder()
 .propertyId(propId)
 .assetId(assetId)
 .build();

 return getAsyncClient().getAssetPropertyValue(assetPropertyValueRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.error("Error occurred while fetching property value:
{}.", exception.getCause().getMessage());
 throw (CompletionException) exception;
 }
 return response.propertyValue().value().doubleValue();
 });
 }

```

- For API details, see [GetAssetPropertyValue](#) in *AWS SDK for Java 2.x API Reference*.

## ListAssetModels

The following code example shows how to use `ListAssetModels`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the asset model ID for the given asset model name.
 *
 * @param assetModelName the name of the asset model for the ID.
 * @return a {@link CompletableFuture} that represents a {@link String} result
of the asset model ID or null if the
 * asset model cannot be found. The calling code can attach callbacks,
then handle the result or exception
 * by calling {@link CompletableFuture#join()} or {@link
CompletableFuture#get()}.
 * <p>
 * If any completion stage in this method throws an exception, the
method logs the exception cause and keeps
 * it available to the calling code as a {@link CompletionException}. By
calling
 * {@link CompletionException#getCause()}, the calling code can access
the original exception.
 */
public CompletableFuture<String> getAssetModelIdAsync(String assetModelName) {
 ListAssetModelsRequest listAssetModelsRequest =
ListAssetModelsRequest.builder().build();
 return getAsyncClient().listAssetModels(listAssetModelsRequest)
 .handle((listAssetModelsResponse, exception) -> {
 if (exception != null) {
 logger.error("Failed to retrieve Asset Model ID: {}",
exception.getCause().getMessage());
 throw (CompletionException) exception;
 }
 for (AssetModelSummary assetModelSummary :
listAssetModelsResponse.assetModelSummaries()) {
 if (assetModelSummary.name().equals(assetModelName)) {
 return assetModelSummary.id();
 }
 }
 })
}
```

```
 return null;
 });
}
```

- For API details, see [ListAssetModels](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Keyspaces examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Keyspaces.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Amazon Keyspaces

The following code example shows how to get started using Amazon Keyspaces.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.keyspaces.KeyspacesClient;
import software.amazon.awssdk.services.keyspaces.model.KeyspaceSummary;
import software.amazon.awssdk.services.keyspaces.model.KeyspacesException;
import software.amazon.awssdk.services.keyspaces.model.ListKeyspacesRequest;
import software.amazon.awssdk.services.keyspaces.model.ListKeyspacesResponse;
import java.util.List;

/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloKeyspaces {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 KeyspacesClient keyClient = KeyspacesClient.builder()
 .region(region)
 .build();

 listKeyspaces(keyClient);
 }

 public static void listKeyspaces(KeyspacesClient keyClient) {
 try {
 ListKeyspacesRequest keyspacesRequest = ListKeyspacesRequest.builder()
 .maxResults(10)
 .build();

 ListKeyspacesResponse response =
keyClient.listKeyspaces(keyspacesRequest);
 List<KeyspaceSummary> keyspaces = response.keyspaces();
 for (KeyspaceSummary keyspace : keyspaces) {
 System.out.println("The name of the keyspace is " +
keyspace.keyspaceName());
 }

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

```
 }
 }
}
```

- For API details, see [ListKeyspaces](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a keyspace and table. The table schema holds movie data and has point-in-time recovery enabled.
- Connect to the keyspace using a secure TLS connection with SigV4 authentication.
- Query the table. Add, retrieve, and update movie data.
- Update the table. Add a column to track watched movies.
- Restore the table to its previous state and clean up resources.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * Before running this Java code example, you must create a
 * Java keystore (JKS) file and place it in your project's resources folder.
 */
```

```

* This file is a secure file format used to hold certificate information for
* Java applications. This is required to make a connection to Amazon Keyspaces.
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/keyspaces/latest/devguide/using_java_driver.html
*
* This Java example performs the following tasks:
*
* 1. Create a keyspace.
* 2. Check for keyspace existence.
* 3. List keyspaces using a paginator.
* 4. Create a table with a simple movie data schema and enable point-in-time
* recovery.
* 5. Check for the table to be in an Active state.
* 6. List all tables in the keyspace.
* 7. Use a Cassandra driver to insert some records into the Movie table.
* 8. Get all records from the Movie table.
* 9. Get a specific Movie.
* 10. Get a UTC timestamp for the current time.
* 11. Update the table schema to add a 'watched' Boolean column.
* 12. Update an item as watched.
* 13. Query for items with watched = True.
* 14. Restore the table back to the previous state using the timestamp.
* 15. Check for completion of the restore action.
* 16. Delete the table.
* 17. Confirm that both tables are deleted.
* 18. Delete the keyspace.
*/

public class ScenarioKeyspaces {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 /*
 * Usage:
 * fileName - The name of the JSON file that contains movie data. (Get this file
 * from the GitHub repo at resources/sample_file.)
 * keyspaceName - The name of the keyspace to create.
 */
 public static void main(String[] args) throws InterruptedException, IOException
 {
 String fileName = "<Replace with the JSON file that contains movie data>";
 String keyspaceName = "<Replace with the name of the keyspace to create>";
 String titleUpdate = "The Family";
 int yearUpdate = 2013;
 }
}

```

```
String tableName = "Movie";
String tableNameRestore = "MovieRestore";
Region region = Region.US_EAST_1;
KeyspacesClient keyClient = KeyspacesClient.builder()
 .region(region)
 .build();

DriverConfigLoader loader =
DriverConfigLoader.fromClasspath("application.conf");
CqlSession session = CqlSession.builder()
 .withConfigLoader(loader)
 .build();

System.out.println(DASHES);
System.out.println("Welcome to the Amazon Keyspaces example scenario.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("1. Create a keyspace.");
createKeySpace(keyClient, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
Thread.sleep(5000);
System.out.println("2. Check for keyspace existence.");
checkKeyspaceExistence(keyClient, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. List keyspaces using a paginator.");
listKeyspacesPaginator(keyClient);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Create a table with a simple movie data schema and
enable point-in-time recovery.");
createTable(keyClient, keyspaceName, tableName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Check for the table to be in an Active state.");
Thread.sleep(6000);
checkTable(keyClient, keyspaceName, tableName);
System.out.println(DASHES);
```

```
System.out.println(DASHES);
System.out.println("6. List all tables in the keyspace.");
listTables(keyClient, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Use a Cassandra driver to insert some records into
the Movie table.");
Thread.sleep(6000);
loadData(session, fileName, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("8. Get all records from the Movie table.");
getMovieData(session, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("9. Get a specific Movie.");
getSpecificMovie(session, keyspaceName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("10. Get a UTC timestamp for the current time.");
ZonedDateTime utc = ZonedDateTime.now(ZoneOffset.UTC);
System.out.println("DATETIME = " + Date.from(utc.toInstant()));
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("11. Update the table schema to add a watched Boolean
column.");
updateTable(keyClient, keyspaceName, tableName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("12. Update an item as watched.");
Thread.sleep(10000); // Wait 10 secs for the update.
updateRecord(session, keyspaceName, titleUpdate, yearUpdate);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("13. Query for items with watched = True.");
getWatchedData(session, keyspaceName);
```



```

 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("14. Restore the table back to the previous state using
the timestamp.");
 System.out.println("Note that the restore operation can take up to 20
minutes.");
 restoreTable(keyClient, keyspaceName, utc);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("15. Check for completion of the restore action.");
 Thread.sleep(5000);
 checkRestoredTable(keyClient, keyspaceName, "MovieRestore");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("16. Delete both tables.");
 deleteTable(keyClient, keyspaceName, tableName);
 deleteTable(keyClient, keyspaceName, tableNameRestore);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("17. Confirm that both tables are deleted.");
 checkTableDelete(keyClient, keyspaceName, tableName);
 checkTableDelete(keyClient, keyspaceName, tableNameRestore);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("18. Delete the keyspace.");
 deleteKeyspace(keyClient, keyspaceName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The scenario has completed successfully.");
 System.out.println(DASHES);
 }

 public static void deleteKeyspace(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 DeleteKeyspaceRequest deleteKeyspaceRequest =
DeleteKeyspaceRequest.builder()
 .keyspaceName(keyspaceName)

```

```
 .build();

 keyClient.deleteKeyspace(deleteKeyspaceRequest);

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void checkTableDelete(KeyspacesClient keyClient, String
keyspaceName, String tableName)
 throws InterruptedException {
 try {
 String status;
 GetTableResponse response;
 GetTableRequest tableRequest = GetTableRequest.builder()
 .keyspaceName(keyspaceName)
 .tableName(tableName)
 .build();

 // Keep looping until table cannot be found and a
ResourceNotFoundException is
 // thrown.
 while (true) {
 response = keyClient.getTable(tableRequest);
 status = response.statusAsString();
 System.out.println(". The table status is " + status);
 Thread.sleep(500);
 }

 } catch (ResourceNotFoundException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 System.out.println("The table is deleted");
}

public static void deleteTable(KeyspacesClient keyClient, String keyspaceName,
String tableName) {
 try {
 DeleteTableRequest tableRequest = DeleteTableRequest.builder()
 .keyspaceName(keyspaceName)
 .tableName(tableName)
 .build();
```

```
 keyClient.deleteTable(tableRequest);

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void checkRestoredTable(KeyspacesClient keyClient, String
keyspaceName, String tableName)
 throws InterruptedException {
 try {
 boolean tableStatus = false;
 String status;
 GetTableResponse response = null;
 GetTableRequest tableRequest = GetTableRequest.builder()
 .keyspaceName(keyspaceName)
 .tableName(tableName)
 .build();

 while (!tableStatus) {
 response = keyClient.getTable(tableRequest);
 status = response.statusAsString();
 System.out.println("The table status is " + status);

 if (status.compareTo("ACTIVE") == 0) {
 tableStatus = true;
 }
 Thread.sleep(500);
 }

 List<ColumnDefinition> cols = response.schemaDefinition().allColumns();
 for (ColumnDefinition def : cols) {
 System.out.println("The column name is " + def.name());
 System.out.println("The column type is " + def.type());
 }

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

```

 public static void restoreTable(KeyspacesClient keyClient, String keyspaceName,
 ZonedDateTime utc) {
 try {
 Instant myTime = utc.toInstant();
 RestoreTableRequest restoreTableRequest = RestoreTableRequest.builder()
 .restoreTimestamp(myTime)
 .sourceTableName("Movie")
 .targetKeyspaceName(keyspaceName)
 .targetTableName("MovieRestore")
 .sourceKeyspaceName(keyspaceName)
 .build();

 RestoreTableResponse response =
keyClient.restoreTable(restoreTableRequest);
 System.out.println("The ARN of the restored table is " +
response.restoredTableARN());

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void getWatchedData(CqlSession session, String keyspaceName) {
 ResultSet resultSet = session
 .execute("SELECT * FROM \"" + keyspaceName + "\".\"Movie\" WHERE
watched = true ALLOW FILTERING;");
 resultSet.forEach(item -> {
 System.out.println("The Movie title is " + item.getString("title"));
 System.out.println("The Movie year is " + item.getInt("year"));
 System.out.println("The plot is " + item.getString("plot"));
 });
 }

 public static void updateRecord(CqlSession session, String keySpace, String
titleUpdate, int yearUpdate) {
 String sqlStatement = "UPDATE \"" + keySpace
 + "\".\"Movie\" SET watched=true WHERE title = :k0 AND year = :k1;";
 BatchStatementBuilder builder =
BatchStatement.builder(DefaultBatchType.UNLOGGED);
 builder.setConsistencyLevel(ConsistencyLevel.LOCAL_QUORUM);
 PreparedStatement preparedStatement = session.prepare(sqlStatement);
 builder.addStatement(preparedStatement.boundStatementBuilder()
 .setString("k0", titleUpdate)

```

```

 .setInt("k1", yearUpdate)
 .build());

 BatchStatement batchStatement = builder.build();
 session.execute(batchStatement);
}

public static void updateTable(KeyspacesClient keyClient, String keySpace,
String tableName) {
 try {
 ColumnDefinition def = ColumnDefinition.builder()
 .name("watched")
 .type("boolean")
 .build();

 UpdateTableRequest tableRequest = UpdateTableRequest.builder()
 .keyspaceName(keySpace)
 .tableName(tableName)
 .addColumnns(def)
 .build();

 keyClient.updateTable(tableRequest);

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void getSpecificMovie(CqlSession session, String keyspaceName) {
 ResultSet resultSet = session.execute(
 "SELECT * FROM \"" + keyspaceName + "\".\"Movie\" WHERE title = 'The
Family' ALLOW FILTERING ;");
 resultSet.forEach(item -> {
 System.out.println("The Movie title is " + item.getString("title"));
 System.out.println("The Movie year is " + item.getInt("year"));
 System.out.println("The plot is " + item.getString("plot"));
 });
}

// Get records from the Movie table.
public static void getMovieData(CqlSession session, String keyspaceName) {
 ResultSet resultSet = session.execute("SELECT * FROM \"" + keyspaceName +
 "\".\"Movie\";");

```

```

 resultSet.forEach(item -> {
 System.out.println("The Movie title is " + item.getString("title"));
 System.out.println("The Movie year is " + item.getInt("year"));
 System.out.println("The plot is " + item.getString("plot"));
 });
 }

 // Load data into the table.
 public static void loadData(CqlSession session, String fileName, String
keySpace) throws IOException {
 String sqlStatement = "INSERT INTO \"" + keySpace + "\".\"Movie\" (title,
year, plot) values (:k0, :k1, :k2)";
 JsonParser parser = new JsonFactory().createParser(new File(fileName));
 com.fasterxml.jackson.databind.JsonNode rootNode = new
ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 ObjectNode currentNode;
 int t = 0;
 while (iter.hasNext()) {

 // Add 20 movies to the table.
 if (t == 20)
 break;
 currentNode = (ObjectNode) iter.next();

 int year = currentNode.path("year").asInt();
 String title = currentNode.path("title").asText();
 String plot = currentNode.path("info").path("plot").toString();

 // Insert the data into the Amazon Keyspaces table.
 BatchStatementBuilder builder =
BatchStatement.builder(DefaultBatchType.UNLOGGED);
 builder.setConsistencyLevel(ConsistencyLevel.LOCAL_QUORUM);
 PreparedStatement preparedStatement = session.prepare(sqlStatement);
 builder.addStatement(preparedStatement.boundStatementBuilder()
 .setString("k0", title)
 .setInt("k1", year)
 .setString("k2", plot)
 .build());

 BatchStatement batchStatement = builder.build();
 session.execute(batchStatement);
 t++;
 }
 }

```

```

 System.out.println("You have added " + t + " records successfully!");
 }

 public static void listTables(KeyspacesClient keyClient, String keyspaceName) {
 try {
 ListTablesRequest tablesRequest = ListTablesRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 ListTablesIterable listRes =
keyClient.listTablesPaginator(tablesRequest);
 listRes.stream()
 .flatMap(r -> r.tables().stream())
 .forEach(content -> System.out.println(" ARN: " +
content.resourceArn() +
 " Table name: " + content.tableName()));

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void checkTable(KeyspacesClient keyClient, String keyspaceName,
String tableName)
 throws InterruptedException {
 try {
 boolean tableStatus = false;
 String status;
 GetTableResponse response = null;
 GetTableRequest tableRequest = GetTableRequest.builder()
 .keyspaceName(keyspaceName)
 .tableName(tableName)
 .build();

 while (!tableStatus) {
 response = keyClient.getTable(tableRequest);
 status = response.statusAsString();
 System.out.println(". The table status is " + status);

 if (status.compareTo("ACTIVE") == 0) {
 tableStatus = true;
 }
 }
 }
 }

```

```
 Thread.sleep(500);
 }

 List<ColumnDefinition> cols = response.schemaDefinition().allColumns();
 for (ColumnDefinition def : cols) {
 System.out.println("The column name is " + def.name());
 System.out.println("The column type is " + def.type());
 }

} catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

}

public static void createTable(KeyspacesClient keyClient, String keySpace,
String tableName) {
 try {
 // Set the columns.
 ColumnDefinition defTitle = ColumnDefinition.builder()
 .name("title")
 .type("text")
 .build();

 ColumnDefinition defYear = ColumnDefinition.builder()
 .name("year")
 .type("int")
 .build();

 ColumnDefinition defReleaseDate = ColumnDefinition.builder()
 .name("release_date")
 .type("timestamp")
 .build();

 ColumnDefinition defPlot = ColumnDefinition.builder()
 .name("plot")
 .type("text")
 .build();

 List<ColumnDefinition> collist = new ArrayList<>();
 collist.add(defTitle);
 collist.add(defYear);
 collist.add(defReleaseDate);
 collist.add(defPlot);
 }
}
```



```
// Set the keys.
PartitionKey yearKey = PartitionKey.builder()
 .name("year")
 .build();

PartitionKey titleKey = PartitionKey.builder()
 .name("title")
 .build();

List<PartitionKey> keyList = new ArrayList<>();
keyList.add(yearKey);
keyList.add(titleKey);

SchemaDefinition schemaDefinition = SchemaDefinition.builder()
 .partitionKeys(keyList)
 .allColumns(colList)
 .build();

PointInTimeRecovery timeRecovery = PointInTimeRecovery.builder()
 .status(PointInTimeRecoveryStatus.ENABLED)
 .build();

CreateTableRequest tableRequest = CreateTableRequest.builder()
 .keyspaceName(keySpace)
 .tableName(tableName)
 .schemaDefinition(schemaDefinition)
 .pointInTimeRecovery(timeRecovery)
 .build();

CreateTableResponse response = keyClient.createTable(tableRequest);
System.out.println("The table ARN is " + response.resourceArn());

} catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

}

public static void listKeyspacesPaginator(KeyspacesClient keyClient) {
 try {
 ListKeyspacesRequest keyspacesRequest = ListKeyspacesRequest.builder()
 .maxResults(10)
 .build();
```

```
 ListKeyspacesIterable listRes =
keyClient.listKeyspacesPaginator(keyspacesRequest);
 listRes.stream()
 .flatMap(r -> r.keyspaces().stream())
 .forEach(content -> System.out.println(" Name: " +
content.keyspaceName()));

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void checkKeyspaceExistence(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 GetKeyspaceRequest keyspaceRequest = GetKeyspaceRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 GetKeyspaceResponse response = keyClient.getKeyspace(keyspaceRequest);
 String name = response.keyspaceName();
 System.out.println("The " + name + " KeySpace is ready");

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void createKeySpace(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 CreateKeyspaceRequest keyspaceRequest = CreateKeyspaceRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 CreateKeyspaceResponse response =
keyClient.createKeyspace(keyspaceRequest);
 System.out.println("The ARN of the KeySpace is " +
response.resourceArn());

 } catch (KeyspacesException e) {
```

```
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateKeyspace](#)
- [CreateTable](#)
- [DeleteKeyspace](#)
- [DeleteTable](#)
- [GetKeyspace](#)
- [GetTable](#)
- [ListKeyspaces](#)
- [ListTables](#)
- [RestoreTable](#)
- [UpdateTable](#)

## Actions

### CreateKeyspace

The following code example shows how to use CreateKeyspace.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void createKeySpace(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 CreateKeyspaceRequest keyspaceRequest = CreateKeyspaceRequest.builder()
```

```
 .keyspaceName(keyspaceName)
 .build();

 CreateKeyspaceResponse response =
keyClient.createKeyspace(keyspaceRequest);
 System.out.println("The ARN of the KeySpace is " +
response.resourceArn());

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateKeyspace](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTable

The following code example shows how to use CreateTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void createTable(KeyspacesClient keyClient, String keySpace,
String tableName) {
 try {
 // Set the columns.
 ColumnDefinition defTitle = ColumnDefinition.builder()
 .name("title")
 .type("text")
 .build();

 ColumnDefinition defYear = ColumnDefinition.builder()
 .name("year")
 .type("int")
 .build();
```

```
ColumnDefinition defReleaseDate = ColumnDefinition.builder()
 .name("release_date")
 .type("timestamp")
 .build();

ColumnDefinition defPlot = ColumnDefinition.builder()
 .name("plot")
 .type("text")
 .build();

List<ColumnDefinition> collList = new ArrayList<>();
collList.add(defTitle);
collList.add(defYear);
collList.add(defReleaseDate);
collList.add(defPlot);

// Set the keys.
PartitionKey yearKey = PartitionKey.builder()
 .name("year")
 .build();

PartitionKey titleKey = PartitionKey.builder()
 .name("title")
 .build();

List<PartitionKey> keyList = new ArrayList<>();
keyList.add(yearKey);
keyList.add(titleKey);

SchemaDefinition schemaDefinition = SchemaDefinition.builder()
 .partitionKeys(keyList)
 .allColumns(collList)
 .build();

PointInTimeRecovery timeRecovery = PointInTimeRecovery.builder()
 .status(PointInTimeRecoveryStatus.ENABLED)
 .build();

CreateTableRequest tableRequest = CreateTableRequest.builder()
 .keyspaceName(keySpace)
 .tableName(tableName)
 .schemaDefinition(schemaDefinition)
 .pointInTimeRecovery(timeRecovery)
```

```
 .build();

 CreateTableResponse response = keyClient.createTable(tableRequest);
 System.out.println("The table ARN is " + response.resourceArn());

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateTable](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteKeyspace

The following code example shows how to use DeleteKeyspace.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteKeyspace(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 DeleteKeyspaceRequest deleteKeyspaceRequest =
DeleteKeyspaceRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 keyClient.deleteKeyspace(deleteKeyspaceRequest);

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteKeyspace](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteTable

The following code example shows how to use DeleteTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteTable(KeyspacesClient keyClient, String keyspaceName,
String tableName) {
 try {
 DeleteTableRequest tableRequest = DeleteTableRequest.builder()
 .keyspaceName(keyspaceName)
 .tableName(tableName)
 .build();

 keyClient.deleteTable(tableRequest);

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteTable](#) in *AWS SDK for Java 2.x API Reference*.

## GetKeyspace

The following code example shows how to use GetKeyspace.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void checkKeyspaceExistence(KeyspacesClient keyClient, String
keyspaceName) {
 try {
 GetKeyspaceRequest keyspaceRequest = GetKeyspaceRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 GetKeyspaceResponse response = keyClient.getKeyspace(keyspaceRequest);
 String name = response.keyspaceName();
 System.out.println("The " + name + " KeySpace is ready");

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetKeyspace](#) in *AWS SDK for Java 2.x API Reference*.

## GetTable

The following code example shows how to use GetTable.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).



```

 public static void checkTable(KeyspacesClient keyClient, String keySpaceName,
String tableName)
 throws InterruptedException {
 try {
 boolean tableStatus = false;
 String status;
 GetTableResponse response = null;
 GetTableRequest tableRequest = GetTableRequest.builder()
 .keySpaceName(keySpaceName)
 .tableName(tableName)
 .build();

 while (!tableStatus) {
 response = keyClient.getTable(tableRequest);
 status = response.statusAsString();
 System.out.println(". The table status is " + status);

 if (status.compareTo("ACTIVE") == 0) {
 tableStatus = true;
 }
 Thread.sleep(500);
 }

 List<ColumnDefinition> cols = response.schemaDefinition().allColumns();
 for (ColumnDefinition def : cols) {
 System.out.println("The column name is " + def.name());
 System.out.println("The column type is " + def.type());
 }

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [GetTable](#) in *AWS SDK for Java 2.x API Reference*.

## ListKeyspaces

The following code example shows how to use `ListKeyspaces`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listKeyspacesPaginator(KeyspacesClient keyClient) {
 try {
 ListKeyspacesRequest keyspacesRequest = ListKeyspacesRequest.builder()
 .maxResults(10)
 .build();

 ListKeyspacesIterable listRes =
 keyClient.listKeyspacesPaginator(keyspacesRequest);
 listRes.stream()
 .flatMap(r -> r.keyspaces().stream())
 .forEach(content -> System.out.println(" Name: " +
 content.keyspaceName()));
 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListKeyspaces](#) in *AWS SDK for Java 2.x API Reference*.

## ListTables

The following code example shows how to use `ListTables`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listTables(KeyspacesClient keyClient, String keyspaceName) {
 try {
 ListTablesRequest tablesRequest = ListTablesRequest.builder()
 .keyspaceName(keyspaceName)
 .build();

 ListTablesIterable listRes =
keyClient.listTablesPaginator(tablesRequest);
 listRes.stream()
 .flatMap(r -> r.tables().stream())
 .forEach(content -> System.out.println(" ARN: " +
content.resourceArn() +
 " Table name: " + content.tableName()));

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListTables](#) in *AWS SDK for Java 2.x API Reference*.

## RestoreTable

The following code example shows how to use `RestoreTable`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void restoreTable(KeyspacesClient keyClient, String keyspaceName,
 ZonedDateTime utc) {
 try {
 Instant myTime = utc.toInstant();
 RestoreTableRequest restoreTableRequest = RestoreTableRequest.builder()
 .restoreTimestamp(myTime)
```

```
 .sourceTableName("Movie")
 .targetKeyspaceName(keyspaceName)
 .targetTableName("MovieRestore")
 .sourceKeyspaceName(keyspaceName)
 .build();

 RestoreTableResponse response =
keyClient.restoreTable(restoreTableRequest);
 System.out.println("The ARN of the restored table is " +
response.restoredTableARN());

 } catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [RestoreTable](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateTable

The following code example shows how to use UpdateTable.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void updateTable(KeyspacesClient keyClient, String keySpace,
String tableName) {
 try {
 ColumnDefinition def = ColumnDefinition.builder()
 .name("watched")
 .type("boolean")
 .build();

 UpdateTableRequest tableRequest = UpdateTableRequest.builder()
```

```
 .keyspaceName(keySpace)
 .tableName(tableName)
 .addColumn(def)
 .build();

 keyClient.updateTable(tableRequest);

} catch (KeyspacesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

- For API details, see [UpdateTable](#) in *AWS SDK for Java 2.x API Reference*.

## Kinesis examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Kinesis.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Serverless examples](#)

## Actions

### CreateStream

The following code example shows how to use `CreateStream`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.CreateStreamRequest;
import software.amazon.awssdk.services.kinesis.model.KinesisException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateDataStream {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <streamName>

 Where:
 streamName - The Amazon Kinesis data stream (for example,
 StockTradeStream).
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String streamName = args[0];
 Region region = Region.US_EAST_1;
 KinesisClient kinesisClient = KinesisClient.builder()
```

```
 .region(region)
 .build();
 createStream(kinesisClient, streamName);
 System.out.println("Done");
 kinesisClient.close();
}

public static void createStream(KinesisClient kinesisClient, String streamName)
{
 try {
 CreateStreamRequest streamReq = CreateStreamRequest.builder()
 .streamName(streamName)
 .shardCount(1)
 .build();

 kinesisClient.createStream(streamReq);

 } catch (KinesisException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateStream](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteStream

The following code example shows how to use DeleteStream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.DeleteStreamRequest;
```

```
import software.amazon.awssdk.services.kinesis.model.KinesisException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteDataStream {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <streamName>

 Where:
 streamName - The Amazon Kinesis data stream (for example,
StockTradeStream)
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String streamName = args[0];
 Region region = Region.US_EAST_1;
 KinesisClient kinesisClient = KinesisClient.builder()
 .region(region)
 .build();

 deleteStream(kinesisClient, streamName);
 kinesisClient.close();
 System.out.println("Done");
 }

 public static void deleteStream(KinesisClient kinesisClient, String streamName)
 {
 try {
 DeleteStreamRequest delStream = DeleteStreamRequest.builder()
 .streamName(streamName)
```



```

 .build();

 kinesisClient.deleteStream(delStream);

 } catch (KinesisException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteStream](#) in *AWS SDK for Java 2.x API Reference*.

## GetRecords

The following code example shows how to use GetRecords.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamResponse;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamRequest;
import software.amazon.awssdk.services.kinesis.model.Shard;
import software.amazon.awssdk.services.kinesis.model.GetShardIteratorRequest;
import software.amazon.awssdk.services.kinesis.model.GetShardIteratorResponse;
import software.amazon.awssdk.services.kinesis.model.Record;
import software.amazon.awssdk.services.kinesis.model.GetRecordsRequest;
import software.amazon.awssdk.services.kinesis.model.GetRecordsResponse;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development

```

```

 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetRecords {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <streamName>

 Where:
 streamName - The Amazon Kinesis data stream to read from (for
example, StockTradeStream).
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String streamName = args[0];
 Region region = Region.US_EAST_1;
 KinesisClient kinesisClient = KinesisClient.builder()
 .region(region)
 .build();

 getStockTrades(kinesisClient, streamName);
 kinesisClient.close();
 }

 public static void getStockTrades(KinesisClient kinesisClient, String
streamName) {
 String shardIterator;
 String lastShardId = null;
 DescribeStreamRequest describeStreamRequest =
DescribeStreamRequest.builder()
 .streamName(streamName)
 .build();

 List<Shard> shards = new ArrayList<>();
 DescribeStreamResponse streamRes;

```

```

do {
 streamRes = kinesisClient.describeStream(describeStreamRequest);
 shards.addAll(streamRes.streamDescription().shards());

 if (shards.size() > 0) {
 lastShardId = shards.get(shards.size() - 1).shardId();
 }
} while (streamRes.streamDescription().hasMoreShards());

GetShardIteratorRequest itReq = GetShardIteratorRequest.builder()
 .streamName(streamName)
 .shardIteratorType("TRIM_HORIZON")
 .shardId(lastShardId)
 .build();

GetShardIteratorResponse shardIteratorResult =
kinesisClient.getShardIterator(itReq);
shardIterator = shardIteratorResult.shardIterator();

// Continuously read data records from shard.
List<Record> records;

// Create new GetRecordsRequest with existing shardIterator.
// Set maximum records to return to 1000.
GetRecordsRequest recordsRequest = GetRecordsRequest.builder()
 .shardIterator(shardIterator)
 .limit(1000)
 .build();

GetRecordsResponse result = kinesisClient.getRecords(recordsRequest);

// Put result into record list. Result may be empty.
records = result.records();

// Print records
for (Record record : records) {
 SdkBytes byteBuffer = record.data();
 System.out.printf("Seq No: %s - %s%n", record.sequenceNumber(), new
String(byteBuffer.asByteArray()));
}
}
}

```

- For API details, see [GetRecords](#) in *AWS SDK for Java 2.x API Reference*.

## PutRecord

The following code example shows how to use PutRecord.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kinesis.KinesisClient;
import software.amazon.awssdk.services.kinesis.model.PutRecordRequest;
import software.amazon.awssdk.services.kinesis.model.KinesisException;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamRequest;
import software.amazon.awssdk.services.kinesis.model.DescribeStreamResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class StockTradesWriter {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <streamName>

 Where:
 streamName - The Amazon Kinesis data stream to which records are
written (for example, StockTradeStream)
 """;
```

```

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String streamName = args[0];
 Region region = Region.US_EAST_1;
 KinesisClient kinesisClient = KinesisClient.builder()
 .region(region)
 .build();

 // Ensure that the Kinesis Stream is valid.
 validateStream(kinesisClient, streamName);
 setStockData(kinesisClient, streamName);
 kinesisClient.close();
 }

 public static void setStockData(KinesisClient kinesisClient, String streamName)
 {
 try {
 // Repeatedly send stock trades with a 100 milliseconds wait in between.
 StockTradeGenerator stockTradeGenerator = new StockTradeGenerator();

 // Put in 50 Records for this example.
 int index = 50;
 for (int x = 0; x < index; x++) {
 StockTrade trade = stockTradeGenerator.getRandomTrade();
 sendStockTrade(trade, kinesisClient, streamName);
 Thread.sleep(100);
 }

 } catch (KinesisException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 System.out.println("Done");
 }

 private static void sendStockTrade(StockTrade trade, KinesisClient
kinesisClient,
 String streamName) {
 byte[] bytes = trade.toJsonAsBytes();

```

```

 // The bytes could be null if there is an issue with the JSON serialization
by
 // the Jackson JSON library.
 if (bytes == null) {
 System.out.println("Could not get JSON bytes for stock trade");
 return;
 }

 System.out.println("Putting trade: " + trade);
 PutRecordRequest request = PutRecordRequest.builder()
 .partitionKey(trade.getTickerSymbol()) // We use the ticker symbol
as the partition key, explained in
 // the Supplemental
Information section below.
 .streamName(streamName)
 .data(SdkBytes.fromByteArray(bytes))
 .build();

 try {
 kinesisClient.putRecord(request);
 } catch (KinesisException e) {
 System.err.println(e.getMessage());
 }
 }

 private static void validateStream(KinesisClient kinesisClient, String
streamName) {
 try {
 DescribeStreamRequest describeStreamRequest =
DescribeStreamRequest.builder()
 .streamName(streamName)
 .build();

 DescribeStreamResponse describeStreamResponse =
kinesisClient.describeStream(describeStreamRequest);

 if (!
describeStreamResponse.streamDescription().streamStatus().toString().equals("ACTIVE"))
 {
 System.err.println("Stream " + streamName + " is not active. Please
wait a few moments and try again.");
 System.exit(1);
 }
 }

```

```

 } catch (KinesisException e) {
 System.err.println("Error found while describing the stream " +
streamName);
 System.err.println(e);
 System.exit(1);
 }
 }
}

```

- For API details, see [PutRecord](#) in *AWS SDK for Java 2.x API Reference*.

## Serverless examples

### Invoke a Lambda function from a Kinesis trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a Kinesis stream. The function retrieves the Kinesis payload, decodes from Base64, and logs the record contents.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming a Kinesis event with Lambda using Java.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KinesisEvent;

public class Handler implements RequestHandler<KinesisEvent, Void> {
 @Override

```

```
public Void handleRequest(final KinesisEvent event, final Context context) {
 LambdaLogger logger = context.getLogger();
 if (event.getRecords().isEmpty()) {
 logger.log("Empty Kinesis Event received");
 return null;
 }
 for (KinesisEvent.KinesisEventRecord record : event.getRecords()) {
 try {
 logger.log("Processed Event with EventId: "+record.getEventID());
 String data = new String(record.getKinesis().getData().array());
 logger.log("Data:"+ data);
 // TODO: Do interesting work based on the new data
 }
 catch (Exception ex) {
 logger.log("An error occurred:"+ex.getMessage());
 throw ex;
 }
 }
 logger.log("Successfully processed:"+event.getRecords().size()+" records");
 return null;
}
```

## Reporting batch item failures for Lambda functions with a Kinesis trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from a Kinesis stream. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Reporting Kinesis batch item failures with Lambda using Java.



```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KinesisEvent;
import com.amazonaws.services.lambda.runtime.events.StreamsEventResponse;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class ProcessKinesisRecords implements RequestHandler<KinesisEvent,
StreamsEventResponse> {

 @Override
 public StreamsEventResponse handleRequest(KinesisEvent input, Context context) {

 List<StreamsEventResponse.BatchItemFailure> batchItemFailures = new
ArrayList<>();
 String curRecordSequenceNumber = "";

 for (KinesisEvent.KinesisEventRecord kinesisEventRecord :
input.getRecords()) {
 try {
 //Process your record
 KinesisEvent.Record kinesisRecord = kinesisEventRecord.getKinesis();
 curRecordSequenceNumber = kinesisRecord.getSequenceNumber();

 } catch (Exception e) {
 /* Since we are working with streams, we can return the failed item
immediately.
 Lambda will immediately begin to retry processing from this
failed item onwards. */
 batchItemFailures.add(new
StreamsEventResponse.BatchItemFailure(curRecordSequenceNumber));
 return new StreamsEventResponse(batchItemFailures);
 }
 }

 return new StreamsEventResponse(batchItemFailures);
 }
}
```

# AWS KMS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS KMS.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello AWS KMS

The following code example shows how to get started using AWS Key Management Service.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.kms.KmsAsyncClient;
import software.amazon.awssdk.services.kms.model.ListKeysRequest;
import software.amazon.awssdk.services.kms.paginators.ListKeysPublisher;
import java.util.concurrent.CompletableFuture;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
```

```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class HelloKMS {
 public static void main(String[] args) {
 listAllKeys();
 }

 public static void listAllKeys() {
 KmsAsyncClient kmsAsyncClient = KmsAsyncClient.builder()
 .build();
 ListKeysRequest listKeysRequest = ListKeysRequest.builder()
 .limit(15)
 .build();

 /*
 * The `subscribe` method is required when using paginator methods in the
 AWS SDK
 * because paginator methods return an instance of a `ListKeysPublisher`,
 which is
 * based on a reactive stream. This allows asynchronous retrieval of
 paginated
 * results as they become available. By subscribing to the stream, we can
 process
 * each page of results as they are emitted.
 */
 ListKeysPublisher keysPublisher =
kmsAsyncClient.listKeysPaginator(listKeysRequest);
 CompletableFuture<Void> future = keysPublisher
 .subscribe(r -> r.keys().forEach(key ->
 System.out.println("The key ARN is: " + key.keyArn() + ". The key Id
is: " + key.keyId()))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 System.err.println("Error occurred: " + exception.getMessage());
 } else {
 System.out.println("Successfully listed all keys.");
 }
 });

 try {
 future.join();

```

```
 } catch (Exception e) {
 System.err.println("Failed to list keys: " + e.getMessage());
 }
 }
}
```

- For API details, see [ListKeys](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a KMS key.
- List KMS keys for your account and get details about them.
- Enable and disable KMS keys.
- Generate a symmetric data key that can be used for client-side encryption.
- Generate an asymmetric key used to digitally sign data.
- Tag keys.
- Delete KMS keys.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run a scenario at a command prompt.

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.kms.model.AlreadyExistsException;
```

```

import software.amazon.awssdk.services.kms.model.DisabledException;
import software.amazon.awssdk.services.kms.model.EnableKeyRotationResponse;
import software.amazon.awssdk.services.kms.model.KmsException;
import software.amazon.awssdk.services.kms.model.NotFoundException;
import software.amazon.awssdk.services.kms.model.RevokeGrantResponse;
import java.util.List;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class KMSScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static String accountId = "";

 private static final Logger logger = LoggerFactory.getLogger(KMSScenario.class);

 static KMSActions kmsActions = new KMSActions();

 static Scanner scanner = new Scanner(System.in);

 static String aliasName = "alias/dev-encryption-key";

 public static void main(String[] args) {
 final String usage = ""
 Usage: <granteePrincipal>

 Where:
 granteePrincipal - The principal (user, service account, or group) to
whom the grant or permission is being given.
 """;

 if (args.length != 1) {
 logger.info(usage);
 return;
 }
 }

```

```

String granteePrincipal = args[0];
String policyName = "default";

accountId = kmsActions.getAccountId();
String keyDesc = "Created by the AWS KMS API";

logger.info(DASHES);
logger.info("""
 Welcome to the AWS Key Management SDK Basics scenario.

 This program demonstrates how to interact with AWS Key Management using
the AWS SDK for Java (v2).
 The AWS Key Management Service (KMS) is a secure and highly available
service that allows you to create
 and manage AWS KMS keys and control their use across a wide range of AWS
services and applications.
 KMS provides a centralized and unified approach to managing encryption
keys, making it easier to meet your
 data protection and regulatory compliance requirements.

 This Basics scenario creates two key types:

 - A symmetric encryption key is used to encrypt and decrypt data.
 - An asymmetric key used to digitally sign data.

 Let's get started...
""");
waitForInputToContinue(scanner);

try {
 // Run the methods that belong to this scenario.
 String targetKeyId = runScenario(granteePrincipal, keyDesc, policyName);
 requestDeleteResources(aliasName, targetKeyId);

} catch (Throwable rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
}
}

```

```

 private static String runScenario(String granteePrincipal, String keyDesc,
String policyName) throws Throwable {
 logger.info(DASHES);
 logger.info("1. Create a symmetric KMS key\n");
 logger.info("First, the program will creates a symmetric KMS key that you
can used to encrypt and decrypt data.");
 waitForInputToContinue(scanner);
 String targetKeyId;
 try {
 CompletableFuture<String> futureKeyId =
kmsActions.createKeyAsync(keyDesc);
 targetKeyId = futureKeyId.join();
 logger.info("A symmetric key was successfully created " + targetKeyId);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("""
 2. Enable a KMS key

 By default, when the SDK creates an AWS key, it is enabled. The next bit
of code checks to
 determine if the key is enabled.
 """);
 waitForInputToContinue(scanner);
 boolean isEnabled;
 try {
 CompletableFuture<Boolean> futureIsKeyEnabled =
kmsActions.isKeyEnabledAsync(targetKeyId);
 isEnabled = futureIsKeyEnabled.join();
 logger.info("Is the key enabled? {}", isEnabled);

 } catch (RuntimeException rt) {

```

```

 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }

 if (!isEnabled)
 try {
 CompletableFuture<Void> future =
kmsActions.enableKeyAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code
{}", kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("3. Encrypt data using the symmetric KMS key");
 String plaintext = "Hello, AWS KMS!";
 logger.info(""""
 One of the main uses of symmetric keys is to encrypt and decrypt data.
 Next, the code encrypts the string {} with the SYMMETRIC_DEFAULT
encryption algorithm.
 """, plaintext);
 waitForInputToContinue(scanner);
 SdkBytes encryptedData;
 try {
 CompletableFuture<SdkBytes> future =
kmsActions.encryptDataAsync(targetKeyId, plaintext);
 encryptedData = future.join();

 } catch (RuntimeException rt) {

```



```

 Throwable cause = rt.getCause();
 if (cause instanceof DisabledException kmsDisabledEx) {
 logger.info("KMS error occurred due to a disabled
key: Error message: {}, Error code {}", kmsDisabledEx.getMessage(),
kmsDisabledEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("4. Create an alias");
 logger.info("

 The alias name should be prefixed with 'alias/'.
 The default, 'alias/dev-encryption-key'.
 ");
 waitForInputToContinue(scanner);

 try {
 CompletableFuture<Void> future =
kmsActions.createCustomAliasAsync(targetKeyId, aliasName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof AlreadyExistsException kmsExistsEx) {
 if (kmsExistsEx.getMessage().contains("already exists")) {
 logger.info("The alias '" + aliasName + "' already exists.
Moving on...");
 }
 } else {
 logger.error("An unexpected error occurred: " + rt.getMessage(),
rt);

 deleteKey(targetKeyId);
 throw cause;
 }
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);

```

```

 logger.info("5. List all of your aliases");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Object> future = kmsActions.listAllAliasesAsync();
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("6. Enable automatic rotation of the KMS key");
 logger.info("

 By default, when the SDK enables automatic rotation of a KMS key,
 KMS rotates the key material of the KMS key one year (approximately 365
 days) from the enable date and every year
 thereafter.
 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<EnableKeyRotationResponse> future =
kmsActions.enableKeyRotationAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);

```

```

 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("""
 7. Create a grant

 A grant is a policy instrument that allows Amazon Web Services
principals to use KMS keys.
 It also can allow them to view a KMS key (DescribeKey) and create and
manage grants.
 When authorizing access to a KMS key, grants are considered along with
key policies and IAM policies.
 """);

 waitForInputToContinue(scanner);
 String grantId = null;
 try {
 CompletableFuture<String> futureGrantId =
kmsActions.grantKeyAsync(targetKeyId, granteePrincipal);
 grantId = futureGrantId.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("8. List grants for the KMS key");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Object> future =
kmsActions.displayGrantIdsAsync(targetKeyId);

```

```

 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("9. Revoke the grant");
 logger.info("""
 The revocation of a grant immediately removes the permissions and access
that the grant had provided.
 This means that any principal (user, role, or service) that was granted
access to perform specific
 KMS operations on a KMS key will no longer be able to perform those
operations.
 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<RevokeGrantResponse> future =
kmsActions.revokeKeyGrantAsync(targetKeyId, grantId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 if (kmsEx.getMessage().contains("Grant does not exist")) {
 logger.info("The grant ID '" + grantId + "' does not exist.
Moving on...");
 } else {
 logger.info("KMS error occurred: Error message: {}, Error code
{}", kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 throw cause;
 }
 } else {

```

```

 logger.info("An unexpected error occurred: " + rt.getMessage());
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
 }
}
waitForInputToContinue(scanner);

logger.info(DASHES);
logger.info("10. Decrypt the data\n");
logger.info("""
 Lets decrypt the data that was encrypted in an early step.
 The code uses the same key to decrypt the string that we encrypted
earlier in the program.
 """);
waitForInputToContinue(scanner);
String decryptedData = "";
try {
 CompletableFuture<String> future =
kmsActions.decryptDataAsync(encryptedData, targetKeyId);
 decryptedData = future.join();
 logger.info("Decrypted data: " + decryptedData);

} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
}
logger.info("Decrypted text is: " + decryptedData);
waitForInputToContinue(scanner);

logger.info(DASHES);
logger.info("11. Replace a key policy\n");
logger.info("""
 A key policy is a resource policy for a KMS key. Key policies are the
primary way to control

```

access to KMS keys. Every KMS key must have exactly one key policy. The statements in the key policy determine who has permission to use the KMS key and how they can use it.

You can also use IAM policies and grants to control access to the KMS key, but every KMS key must have a key policy.

By default, when you create a key by using the SDK, a policy is created that gives the AWS account that owns the KMS key full access to the KMS key.

Let's try to replace the automatically created policy with the following policy.

```

 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": {"AWS": "arn:aws:iam::000000000000:root"},
 "Action": "kms:*",
 "Resource": "*"
 }]
 """);

 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Boolean> future =
 kmsActions.replacePolicyAsync(targetKeyId, policyName, accountId);
 boolean success = future.join();
 if (success) {
 logger.info("Key policy replacement succeeded.");
 } else {
 logger.error("Key policy replacement failed.");
 }
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
 kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }
 deleteAliasName(aliasName);

```

```

 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("12. Get the key policy\n");
 logger.info("The next bit of code that runs gets the key policy to make sure
it exists.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<String> future =
kmsActions.getKeyPolicyAsync(targetKeyId, policyName);
 String policy = future.join();
 if (!policy.isEmpty()) {
 logger.info("Retrieved policy: " + policy);
 }

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("13. Create an asymmetric KMS key and sign your data\n");
 logger.info("""
 Signing your data with an AWS key can provide several benefits that
make it an attractive option
 for your data signing needs. By using an AWS KMS key, you can leverage
the
 security controls and compliance features provided by AWS,
 which can help you meet various regulatory requirements and enhance the
overall security posture
 of your organization.
 """);

```

```

 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Boolean> future = kmsActions.signVerifyDataAsync();
 boolean success = future.join();
 if (success) {
 logger.info("Sign and verify data operation succeeded.");
 } else {
 logger.error("Sign and verify data operation failed.");
 }
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
 kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
 }
 waitForInputToContinue(scanner);

 logger.info(DASHES);
 logger.info("14. Tag your symmetric KMS Key\n");
 logger.info("""
 By using tags, you can improve the overall management, security, and
 governance of your
 KMS keys, making it easier to organize, track, and control access to
 your encrypted data within
 your AWS environment
 """);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future = kmsActions.tagKMSKeyAsync(targetKeyId);
 future.join();
 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
 kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {

```



```

 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 deleteAliasName(aliasName);
 deleteKey(targetKeyId);
 throw cause;
}
waitForInputToContinue(scanner);
return targetKeyId;
}

// Deletes KMS resources with user input.
private static void requestDeleteResources(String aliasName, String targetKeyId)
{
 logger.info(DASHES);
 logger.info("15. Schedule the deletion of the KMS key\n");
 logger.info("""
 By default, KMS applies a waiting period of 30 days,
 but you can specify a waiting period of 7-30 days. When this operation
is successful,
 the key state of the KMS key changes to PendingDeletion and the key
can't be used in any
 cryptographic operations. It remains in this state for the duration of
the waiting period.

 Deleting a KMS key is a destructive and potentially dangerous operation.
When a KMS key is deleted,
 all data that was encrypted under the KMS key is unrecoverable.
 """);
 logger.info("Would you like to delete the Key Management resources? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 logger.info("You selected to delete the AWS KMS resources.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
kmsActions.deleteSpecificAliasAsync(aliasName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code
{}", kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {

```

```

 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
}
waitForInputToContinue(scanner);
try {
 CompletableFuture<Void> future =
kmsActions.disableKeyAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code
{}", kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }

 try {
 CompletableFuture<Void> future =
kmsActions.deleteKeyAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code
{}", kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }

 } else {
 logger.info("The Key Management resources will not be deleted");
 }

 logger.info(DASHES);
 logger.info("This concludes the AWS Key Management SDK scenario");
 logger.info(DASHES);
}

// This method is invoked from Exceptions to clean up the resources.

```

```

private static void deleteKey(String targetKeyId) {
 try {
 CompletableFuture<Void> future =
kmsActions.disableKeyAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }

 try {
 CompletableFuture<Void> future = kmsActions.deleteKeyAsync(targetKeyId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }
}

// This method is invoked from Exceptions to clean up the resources.
private static void deleteAliasName(String aliasName) {
 try {
 CompletableFuture<Void> future =
kmsActions.deleteSpecificAliasAsync(aliasName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof KmsException kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {

```

```

 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}
}

```

Define a class that wraps KMS actions.

```

public class KMSActions {
 private static final Logger logger = LoggerFactory.getLogger(KMSActions.class);
 private static KmsAsyncClient kmsAsyncClient;

 /**
 * Retrieves an asynchronous AWS Key Management Service (KMS) client.
 * <p>
 * This method creates and returns a singleton instance of the KMS async client,
 with the following configurations:
 *
 * Max concurrency: 100
 * Connection timeout: 60 seconds
 * Read timeout: 60 seconds
 * Write timeout: 60 seconds
 * API call timeout: 2 minutes
 * API call attempt timeout: 90 seconds
 * Retry policy: up to 3 retries
 */
}

```

```

 * Credentials provider: environment variable credentials provider
 *
 * <p>
 * If the client instance has already been created, it is returned instead of
 creating a new one.
 *
 * @return the KMS async client instance
 */
 private static KmsAsyncClient getAsyncClient() {
 if (kmsAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 kmsAsyncClient = KmsAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return kmsAsyncClient;
 }

 /**
 * Creates a new symmetric encryption key asynchronously.
 *
 * @param keyDesc the description of the key to be created
 * @return a {@link CompletableFuture} that completes with the ID of the newly
 created key
 * @throws RuntimeException if an error occurs while creating the key
 */
 public CompletableFuture<String> createKeyAsync(String keyDesc) {
 CreateKeyRequest keyRequest = CreateKeyRequest.builder()

```

```

 .description(keyDesc)
 .keySpec(KeySpec.SYMMETRIC_DEFAULT)
 .keyUsage(KeyUsageType.ENCRYPT_DECRYPT)
 .build();

 return getAsyncClient().createKey(keyRequest)
 .thenApply(resp -> resp.keyMetadata().keyId())
 .exceptionally(ex -> {
 throw new RuntimeException("An error occurred while creating the
key: " + ex.getMessage(), ex);
 });
}

/**
 * Asynchronously checks if a specified key is enabled.
 *
 * @param keyId the ID of the key to check
 * @return a {@link CompletableFuture} that, when completed, indicates whether
the key is enabled or not
 *
 * @throws RuntimeException if an exception occurs while checking the key state
 */
public CompletableFuture<Boolean> isKeyEnabledAsync(String keyId) {
 DescribeKeyRequest keyRequest = DescribeKeyRequest.builder()
 .keyId(keyId)
 .build();

 CompletableFuture<DescribeKeyResponse> responseFuture =
getAsyncClient().describeKey(keyRequest);
 return responseFuture.whenComplete((resp, ex) -> {
 if (resp != null) {
 KeyState keyState = resp.keyMetadata().keyState();
 if (keyState == KeyState.ENABLED) {
 logger.info("The key is enabled.");
 } else {
 logger.info("The key is not enabled. Key state: {}", keyState);
 }
 } else {
 throw new RuntimeException(ex);
 }
 }).thenApply(resp -> resp.keyMetadata().keyState() == KeyState.ENABLED);
}

/**

```

```

 * Asynchronously enables the specified key.
 *
 * @param keyId the ID of the key to enable
 * @return a {@link CompletableFuture} that completes when the key has been
enabled
 */
 public CompletableFuture<Void> enableKeyAsync(String keyId) {
 EnableKeyRequest enableKeyRequest = EnableKeyRequest.builder()
 .keyId(keyId)
 .build();

 CompletableFuture<EnableKeyResponse> responseFuture =
getAsyncClient().enableKey(enableKeyRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("Key with ID [{}] has been enabled.", keyId);
 } else {
 if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while enabling
key: " + kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred while
enabling key: " + exception.getMessage(), exception);
 }
 }
 });

 return responseFuture.thenApply(response -> null);
 }

 /**
 * Encrypts the given text asynchronously using the specified KMS client and key
ID.
 *
 * @param keyId the ID of the KMS key to use for encryption
 * @param text the text to encrypt
 * @return a CompletableFuture that completes with the encrypted data as an
SdkBytes object
 */
 public CompletableFuture<SdkBytes> encryptDataAsync(String keyId, String text) {
 SdkBytes myBytes = SdkBytes.fromUtf8String(text);
 EncryptRequest encryptRequest = EncryptRequest.builder()
 .keyId(keyId)
 .plaintext(myBytes)

```

```

 .build();

 CompletableFuture<EncryptResponse> responseFuture =
getAsyncClient().encrypt(encryptRequest).toCompletableFuture();
 return responseFuture.whenComplete((response, ex) -> {
 if (response != null) {
 String algorithm = response.encryptionAlgorithm().toString();
 logger.info("The string was encrypted with algorithm {}.\"",
algorithm);
 } else {
 throw new RuntimeException(ex);
 }
 }).thenApply(EncryptResponse::ciphertextBlob);
 }

 /**
 * Creates a custom alias for the specified target key asynchronously.
 *
 * @param targetKeyId the ID of the target key for the alias
 * @param aliasName the name of the alias to create
 * @return a {@link CompletableFuture} that completes when the alias creation
operation is finished
 */
 public CompletableFuture<Void> createCustomAliasAsync(String targetKeyId, String
aliasName) {
 CreateAliasRequest aliasRequest = CreateAliasRequest.builder()
 .aliasName(aliasName)
 .targetKeyId(targetKeyId)
 .build();

 CompletableFuture<CreateAliasResponse> responseFuture =
getAsyncClient().createAlias(aliasRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("{} was successfully created.", aliasName);
 } else {
 if (exception instanceof ResourceExistsException) {
 logger.info("Alias [{}] already exists. Moving on...",
aliasName);
 } else if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while creating
alias: " + kmsEx.getMessage(), kmsEx);
 } else {

```



```

 throw new RuntimeException("An unexpected error occurred while
creating alias: " + exception.getMessage(), exception);
 }
}

});

return responseFuture.thenApply(response -> null);
}

/**
 * Asynchronously lists all the aliases in the current AWS account.
 *
 * @return a {@link CompletableFuture} that completes when the list of aliases
has been processed
 */
public CompletableFuture<Object> listAllAliasesAsync() {
 ListAliasesRequest aliasesRequest = ListAliasesRequest.builder()
 .limit(15)
 .build();

 ListAliasesPublisher paginator =
getAsyncClient().listAliasesPaginator(aliasesRequest);
 return paginator.subscribe(response -> {
 response.aliases().forEach(alias ->
 logger.info("The alias name is: " + alias.aliasName())
);
 })
 .thenApply(v -> null)
 .exceptionally(ex -> {
 if (ex.getCause() instanceof KmsException) {
 KmsException e = (KmsException) ex.getCause();
 throw new RuntimeException("A KMS exception occurred: " +
e.getMessage());
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
ex.getMessage());
 }
 });
}

/**
 * Enables key rotation asynchronously for the specified key ID.
 *
 * @param keyId the ID of the key for which to enable key rotation

```

```

 * @return a CompletableFuture that represents the asynchronous operation of
 enabling key rotation
 * @throws RuntimeException if there was an error enabling key rotation, either
 due to a KMS exception or an unexpected error
 */
 public CompletableFuture<EnableKeyRotationResponse>
 enableKeyRotationAsync(String keyId) {
 EnableKeyRotationRequest enableKeyRotationRequest =
 EnableKeyRotationRequest.builder()
 .keyId(keyId)
 .build();

 CompletableFuture<EnableKeyRotationResponse> responseFuture =
 getAsyncClient().enableKeyRotation(enableKeyRotationRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("Key rotation has been enabled for key with id [{}]",
 keyId);
 } else {
 if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("Failed to enable key rotation: " +
 kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
 exception.getMessage(), exception);
 }
 }
 });

 return responseFuture;
 }

 /**
 * Grants permissions to a specified principal on a customer master key (CMK)
 asynchronously.
 *
 * @param keyId The unique identifier for the customer master key
 (CMK) that the grant applies to.
 * @param granteePrincipal The principal that is given permission to perform
 the operations that the grant permits on the CMK.
 * @return A {@link CompletableFuture} that, when completed, contains the ID of
 the created grant.
 * @throws RuntimeException If an error occurs during the grant creation
 process.

```

```

 */
 public CompletableFuture<String> grantKeyAsync(String keyId, String
granteePrincipal) {
 List<GrantOperation> grantPermissions = List.of(
 GrantOperation.ENCRYPT,
 GrantOperation.DECRYPT,
 GrantOperation.DESCRIBE_KEY
);

 CreateGrantRequest grantRequest = CreateGrantRequest.builder()
 .keyId(keyId)
 .name("grant1")
 .granteePrincipal(granteePrincipal)
 .operations(grantPermissions)
 .build();

 CompletableFuture<CreateGrantResponse> responseFuture =
getAsyncClient().createGrant(grantRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex == null) {
 logger.info("Grant created successfully with ID: " +
response.grantId());
 } else {
 if (ex instanceof KmsException kmsEx) {
 throw new RuntimeException("Failed to create grant: " +
kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
ex.getMessage(), ex);
 }
 }
 });

 return responseFuture.thenApply(CreateGrantResponse::grantId);
 }

 /**
 * Asynchronously displays the grant IDs for the specified key ID.
 *
 * @param keyId the ID of the AWS KMS key for which to list the grants
 * @return a {@link CompletableFuture} that, when completed, will be null if
the operation succeeded, or will throw a {@link RuntimeException} if the operation
failed

```

```

 * @throws RuntimeException if there was an error listing the grants, either due
 to an {@link KmsException} or an unexpected error
 */
 public CompletableFuture<Object> displayGrantIdsAsync(String keyId) {
 ListGrantsRequest grantsRequest = ListGrantsRequest.builder()
 .keyId(keyId)
 .limit(15)
 .build();

 ListGrantsPublisher paginator =
getAsyncClient().listGrantsPaginator(grantsRequest);
 return paginator.subscribe(response -> {
 response.grants().forEach(grant -> {
 logger.info("The grant Id is: " + grant.grantId());
 });
 })
 .thenApply(v -> null)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause();
 if (cause instanceof KmsException) {
 throw new RuntimeException("Failed to list grants: " +
cause.getMessage(), cause);
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
cause.getMessage(), cause);
 }
 });
 }

 /**
 * Revokes a grant for the specified AWS KMS key asynchronously.
 *
 * @param keyId The ID or key ARN of the AWS KMS key.
 * @param grantId The identifier of the grant to be revoked.
 * @return A {@link CompletableFuture} representing the asynchronous operation
 of revoking the grant.
 *
 * The {@link CompletableFuture} will complete with a {@link
 RevokeGrantResponse} object
 *
 * if the operation is successful, or with a {@code null} value if an
 error occurs.
 */
 public CompletableFuture<RevokeGrantResponse> revokeKeyGrantAsync(String keyId,
String grantId) {
 RevokeGrantRequest grantRequest = RevokeGrantRequest.builder()

```

```

 .keyId(keyId)
 .grantId(grantId)
 .build();

 CompletableFuture<RevokeGrantResponse> responseFuture =
getAsyncClient().revokeGrant(grantRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("Grant ID: [" + grantId + "] was successfully
revoked!");
 } else {
 if (exception instanceof KmsException kmsEx) {
 if (kmsEx.getMessage().contains("Grant does not exist")) {
 logger.info("The grant ID '" + grantId + "' does not exist.
Moving on...");
 } else {
 throw new RuntimeException("KMS error occurred: " +
kmsEx.getMessage(), kmsEx);
 }
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
exception.getMessage(), exception);
 }
 }
 });

 return responseFuture;
}

/**
 * Asynchronously decrypts the given encrypted data using the specified key ID.
 *
 * @param encryptedData The encrypted data to be decrypted.
 * @param keyId The ID of the key to be used for decryption.
 * @return A CompletableFuture that, when completed, will contain the decrypted
data as a String.
 *
 * If an error occurs during the decryption process, the
CompletableFuture will complete
 *
 * exceptionally with the error, and the method will return an empty
String.
 */
public CompletableFuture<String> decryptDataAsync(SdkBytes encryptedData, String
keyId) {

```

```

 DecryptRequest decryptRequest = DecryptRequest.builder()
 .ciphertextBlob(encryptedData)
 .keyId(keyId)
 .build();

 CompletableFuture<DecryptResponse> responseFuture =
getAsyncClient().decrypt(decryptRequest);
 responseFuture.whenComplete((decryptResponse, exception) -> {
 if (exception == null) {
 logger.info("Data decrypted successfully for key ID: " + keyId);
 } else {
 if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while decrypting
data: " + kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred while
decrypting data: " + exception.getMessage(), exception);
 }
 }
 });

 return responseFuture.thenApply(decryptResponse ->
decryptResponse.plaintext().asString(StandardCharsets.UTF_8));
 }

/**
 * Asynchronously replaces the policy for the specified KMS key.
 *
 * @param keyId the ID of the KMS key to replace the policy for
 * @param policyName the name of the policy to be replaced
 * @param accountId the AWS account ID to be used in the policy
 * @return a {@link CompletableFuture} that completes with a boolean indicating
 * whether the policy replacement was successful or not
 */
 public CompletableFuture<Boolean> replacePolicyAsync(String keyId, String
policyName, String accountId) {
 String policy = ""
 {
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": {"AWS": "arn:aws:iam::%s:root"},
 "Action": "kms:*",
 "Resource": "*"
 }
]
 }
 }

```

```

 }}
}
"".formatted(accountId);

PutKeyPolicyRequest keyPolicyRequest = PutKeyPolicyRequest.builder()
 .keyId(keyId)
 .policyName(policyName)
 .policy(policy)
 .build();

// First, get the current policy to check if it exists
return getAsyncClient().getKeyPolicy(r ->
r.keyId(keyId).policyName(policyName))
 .thenCompose(response -> {
 logger.info("Current policy exists. Replacing it...");
 return getAsyncClient().putKeyPolicy(keyPolicyRequest);
 })
 .thenApply(putPolicyResponse -> {
 logger.info("The key policy has been replaced.");
 return true;
 })
 .exceptionally(throwable -> {
 if (throwable.getCause() instanceof LimitExceededException) {
 logger.error("Cannot replace policy, as only one policy is
allowed per key.");
 return false;
 }
 throw new RuntimeException("Error replacing policy", throwable);
 });
}

/**
 * Asynchronously retrieves the key policy for the specified key ID and policy
name.
 *
 * @param keyId the ID of the AWS KMS key for which to retrieve the policy
 * @param policyName the name of the key policy to retrieve
 * @return a {@link CompletableFuture} that, when completed, contains the key
policy as a {@link String}
 */
public CompletableFuture<String> getKeyPolicyAsync(String keyId, String
policyName) {
 GetKeyPolicyRequest policyRequest = GetKeyPolicyRequest.builder()

```

```

 .keyId(keyId)
 .policyName(policyName)
 .build();

return getAsyncClient().getKeyPolicy(policyRequest)
 .thenApply(response -> {
 String policy = response.policy();
 logger.info("The response is: " + policy);
 return policy;
 })
 .exceptionally(ex -> {
 throw new RuntimeException("Failed to get key policy", ex);
 });
}

/**
 * Asynchronously signs and verifies data using AWS KMS.
 *
 * <p>The method performs the following steps:
 *
 * Creates an AWS KMS key with the specified key spec, key usage, and
origin.
 * Signs the provided message using the created KMS key and the RSASSA-
PSS-SHA-256 algorithm.
 * Verifies the signature of the message using the created KMS key and
the RSASSA-PSS-SHA-256 algorithm.
 *
 *
 * @return a {@link CompletableFuture} that completes with the result of the
signature verification,
 * {@code true} if the signature is valid, {@code false} otherwise.
 * @throws KmsException if any error occurs during the KMS operations.
 * @throws RuntimeException if an unexpected error occurs.
 */
public CompletableFuture<Boolean> signVerifyDataAsync() {
 String signMessage = "Here is the message that will be digitally signed";

 // Create an AWS KMS key used to digitally sign data.
 CreateKeyRequest createKeyRequest = CreateKeyRequest.builder()
 .keySpec(KeySpec.RSA_2048)
 .keyUsage(KeyUsageType.SIGN_VERIFY)
 .origin(OriginType.AWS_KMS)
 .build();

```



```

 return getAsyncClient().createKey(createKeyRequest)
 .thenCompose(createKeyResponse -> {
 String keyId = createKeyResponse.keyMetadata().keyId();

 SdkBytes messageBytes = SdkBytes.fromString(signMessage,
Charset.defaultCharset());
 SignRequest signRequest = SignRequest.builder()
 .keyId(keyId)
 .message(messageBytes)
 .signingAlgorithm(SigningAlgorithmSpec.RSASSA_PSS_SHA_256)
 .build();

 return getAsyncClient().sign(signRequest)
 .thenCompose(signResponse -> {
 byte[] signedBytes = signResponse.signature().asByteArray();

 VerifyRequest verifyRequest = VerifyRequest.builder()
 .keyId(keyId)

 .message(SdkBytes.fromByteArray(signMessage.getBytes(Charset.defaultCharset()))))
 .signature(SdkBytes.fromByteBuffer(ByteBuffer.wrap(signedBytes)))

 .signingAlgorithm(SigningAlgorithmSpec.RSASSA_PSS_SHA_256)
 .build();

 return getAsyncClient().verify(verifyRequest)
 .thenApply(verifyResponse -> {
 return (boolean) verifyResponse.signatureValid();
 });
 });
 });
 }

 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to sign or verify data",
throwable);
 });
}

/**
 * Asynchronously tags a KMS key with a specific tag.
 *
 * @param keyId the ID of the KMS key to be tagged
 * @return a {@link CompletableFuture} that completes when the tagging operation
is finished

```

```

 */
 public CompletableFuture<Void> tagKMSKeyAsync(String keyId) {
 Tag tag = Tag.builder()
 .tagKey("Environment")
 .tagValue("Production")
 .build();

 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .keyId(keyId)
 .tags(tag)
 .build();

 return getAsyncClient().tagResource(tagResourceRequest)
 .thenRun(() -> {
 logger.info("{} key was tagged", keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to tag the KMS key", throwable);
 });
 }

 /**
 * Deletes a specific KMS alias asynchronously.
 *
 * @param aliasName the name of the alias to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
 of deleting the specified alias
 */
 public CompletableFuture<Void> deleteSpecificAliasAsync(String aliasName) {
 DeleteAliasRequest deleteAliasRequest = DeleteAliasRequest.builder()
 .aliasName(aliasName)
 .build();

 return getAsyncClient().deleteAlias(deleteAliasRequest)
 .thenRun(() -> {
 logger.info("Alias {} has been deleted successfully", aliasName);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to delete alias: " + aliasName,
throwable);
 });
 }

 /**

```

```

 * Asynchronously disables the specified AWS Key Management Service (KMS) key.
 *
 * @param keyId the ID or Amazon Resource Name (ARN) of the KMS key to be
disabled
 * @return a CompletableFuture that, when completed, indicates that the key has
been disabled successfully
 */
 public CompletableFuture<Void> disableKeyAsync(String keyId) {
 DisableKeyRequest keyRequest = DisableKeyRequest.builder()
 .keyId(keyId)
 .build();

 return getAsyncClient().disableKey(keyRequest)
 .thenRun(() -> {
 logger.info("Key {} has been disabled successfully",keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to disable key: " + keyId,
throwable);
 });
 }

 /**
 * Deletes a KMS key asynchronously.
 *
 * <p>Warning: Deleting a KMS key is a destructive and
potentially dangerous operation.
 * When a KMS key is deleted, all data that was encrypted under the KMS key
becomes unrecoverable.
 * This means that any files, databases, or other data that were encrypted using
the deleted KMS key
 * will become permanently inaccessible. Exercise extreme caution when deleting
KMS keys.</p>
 *
 * @param keyId the ID of the KMS key to delete
 * @return a {@link CompletableFuture} that completes when the key deletion is
scheduled
 */
 public CompletableFuture<Void> deleteKeyAsync(String keyId) {
 ScheduleKeyDeletionRequest deletionRequest =
ScheduleKeyDeletionRequest.builder()
 .keyId(keyId)
 .pendingWindowInDays(7)
 .build();

```

```
 return getAsyncClient().scheduleKeyDeletion(deletionRequest)
 .thenRun(() -> {
 logger.info("Key {} will be deleted in 7 days", keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to schedule key deletion for key
ID: " + keyId, throwable);
 });
 }

 public String getAccountId(){
 try (StsClient stsClient = StsClient.create()){
 GetCallerIdentityResponse callerIdentity =
stsClient.getCallerIdentity();
 return callerIdentity.account();
 }
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateAlias](#)
- [CreateGrant](#)
- [CreateKey](#)
- [Decrypt](#)
- [DescribeKey](#)
- [DisableKey](#)
- [EnableKey](#)
- [Encrypt](#)
- [GetKeyPolicy](#)
- [ListAliases](#)
- [ListGrants](#)
- [ListKeys](#)
- [RevokeGrant](#)
- [ScheduleKeyDeletion](#)

- [Sign](#)
- [TagResource](#)

## Actions

### CreateAlias

The following code example shows how to use `CreateAlias`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a custom alias for the specified target key asynchronously.
 *
 * @param targetKeyId the ID of the target key for the alias
 * @param aliasName the name of the alias to create
 * @return a {@link CompletableFuture} that completes when the alias creation
 * operation is finished
 */
public CompletableFuture<Void> createCustomAliasAsync(String targetKeyId, String
aliasName) {
 CreateAliasRequest aliasRequest = CreateAliasRequest.builder()
 .aliasName(aliasName)
 .targetKeyId(targetKeyId)
 .build();

 CompletableFuture<CreateAliasResponse> responseFuture =
getAsyncClient().createAlias(aliasRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("{} was successfully created.", aliasName);
 } else {
 if (exception instanceof ResourceExistsException) {
 logger.info("Alias [{}] already exists. Moving on...",
aliasName);
 }
 }
 });
}
```

```

 } else if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while creating
alias: " + kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred while
creating alias: " + exception.getMessage(), exception);
 }
 }
});

return responseFuture.thenApply(response -> null);
}

```

- For API details, see [CreateAlias](#) in *AWS SDK for Java 2.x API Reference*.

## CreateGrant

The following code example shows how to use CreateGrant.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Grants permissions to a specified principal on a customer master key (CMK)
asynchronously.
 *
 * @param keyId The unique identifier for the customer master key
(CMK) that the grant applies to.
 * @param granteePrincipal The principal that is given permission to perform
the operations that the grant permits on the CMK.
 * @return A {@link CompletableFuture} that, when completed, contains the ID of
the created grant.
 * @throws RuntimeException If an error occurs during the grant creation
process.
 */

```

```

 public CompletableFuture<String> grantKeyAsync(String keyId, String
granteePrincipal) {
 List<GrantOperation> grantPermissions = List.of(
 GrantOperation.ENCRYPT,
 GrantOperation.DECRYPT,
 GrantOperation.DESCRIBE_KEY
);

 CreateGrantRequest grantRequest = CreateGrantRequest.builder()
 .keyId(keyId)
 .name("grant1")
 .granteePrincipal(granteePrincipal)
 .operations(grantPermissions)
 .build();

 CompletableFuture<CreateGrantResponse> responseFuture =
getAsyncClient().createGrant(grantRequest);
 responseFuture.whenComplete((response, ex) -> {
 if (ex == null) {
 logger.info("Grant created successfully with ID: " +
response.grantId());
 } else {
 if (ex instanceof KmsException kmsEx) {
 throw new RuntimeException("Failed to create grant: " +
kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
ex.getMessage(), ex);
 }
 }
 });

 return responseFuture.thenApply(CreateGrantResponse::grantId);
 }

```

- For API details, see [CreateGrant](#) in *AWS SDK for Java 2.x API Reference*.

## CreateKey

The following code example shows how to use `CreateKey`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new symmetric encryption key asynchronously.
 *
 * @param keyDesc the description of the key to be created
 * @return a {@link CompletableFuture} that completes with the ID of the newly
created key
 * @throws RuntimeException if an error occurs while creating the key
 */
public CompletableFuture<String> createKeyAsync(String keyDesc) {
 CreateKeyRequest keyRequest = CreateKeyRequest.builder()
 .description(keyDesc)
 .keySpec(KeySpec.SYMMETRIC_DEFAULT)
 .keyUsage(KeyUsageType.ENCRYPT_DECRYPT)
 .build();

 return getAsyncClient().createKey(keyRequest)
 .thenApply(resp -> resp.keyMetadata().keyId())
 .exceptionally(ex -> {
 throw new RuntimeException("An error occurred while creating the
key: " + ex.getMessage(), ex);
 });
}
```

- For API details, see [CreateKey](#) in *AWS SDK for Java 2.x API Reference*.

## Decrypt

The following code example shows how to use Decrypt.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously decrypts the given encrypted data using the specified key ID.
 *
 * @param encryptedData The encrypted data to be decrypted.
 * @param keyId The ID of the key to be used for decryption.
 * @return A CompletableFuture that, when completed, will contain the decrypted
 * data as a String.
 *
 * If an error occurs during the decryption process, the
 * CompletableFuture will complete
 *
 * exceptionally with the error, and the method will return an empty
 * String.
 */
public CompletableFuture<String> decryptDataAsync(SdkBytes encryptedData, String
keyId) {
 DecryptRequest decryptRequest = DecryptRequest.builder()
 .ciphertextBlob(encryptedData)
 .keyId(keyId)
 .build();

 CompletableFuture<DecryptResponse> responseFuture =
getAsyncClient().decrypt(decryptRequest);
 responseFuture.whenComplete((decryptResponse, exception) -> {
 if (exception == null) {
 logger.info("Data decrypted successfully for key ID: " + keyId);
 } else {
 if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while decrypting
data: " + kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred while
decrypting data: " + exception.getMessage(), exception);
 }
 }
 });
}
```

```

 return responseFuture.thenApply(decryptResponse ->
 decryptResponse.plaintext().asString(StandardCharsets.UTF_8));
 }

```

- For API details, see [Decrypt](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteAlias

The following code example shows how to use DeleteAlias.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a specific KMS alias asynchronously.
 *
 * @param aliasName the name of the alias to be deleted
 * @return a {@link CompletableFuture} representing the asynchronous operation
of deleting the specified alias
 */
public CompletableFuture<Void> deleteSpecificAliasAsync(String aliasName) {
 DeleteAliasRequest deleteAliasRequest = DeleteAliasRequest.builder()
 .aliasName(aliasName)
 .build();

 return getAsyncClient().deleteAlias(deleteAliasRequest)
 .thenRun(() -> {
 logger.info("Alias {} has been deleted successfully", aliasName);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to delete alias: " + aliasName,
throwable);
 });
}

```

- For API details, see [DeleteAlias](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeKey

The following code example shows how to use DescribeKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously checks if a specified key is enabled.
 *
 * @param keyId the ID of the key to check
 * @return a {@link CompletableFuture} that, when completed, indicates whether
the key is enabled or not
 *
 * @throws RuntimeException if an exception occurs while checking the key state
 */
public CompletableFuture<Boolean> isKeyEnabledAsync(String keyId) {
 DescribeKeyRequest keyRequest = DescribeKeyRequest.builder()
 .keyId(keyId)
 .build();

 CompletableFuture<DescribeKeyResponse> responseFuture =
getAsyncClient().describeKey(keyRequest);
 return responseFuture.whenComplete((resp, ex) -> {
 if (resp != null) {
 KeyState keyState = resp.keyMetadata().keyState();
 if (keyState == KeyState.ENABLED) {
 logger.info("The key is enabled.");
 } else {
 logger.info("The key is not enabled. Key state: {}", keyState);
 }
 }
 }) else {
```

```

 throw new RuntimeException(ex);
 }
}).thenApply(resp -> resp.keyMetadata().keyState() == KeyState.ENABLED);
}

```

- For API details, see [DescribeKey](#) in *AWS SDK for Java 2.x API Reference*.

## DisableKey

The following code example shows how to use `DisableKey`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously disables the specified AWS Key Management Service (KMS) key.
 *
 * @param keyId the ID or Amazon Resource Name (ARN) of the KMS key to be
disabled
 * @return a CompletableFuture that, when completed, indicates that the key has
been disabled successfully
 */
public CompletableFuture<Void> disableKeyAsync(String keyId) {
 DisableKeyRequest keyRequest = DisableKeyRequest.builder()
 .keyId(keyId)
 .build();

 return getAsyncClient().disableKey(keyRequest)
 .thenRun(() -> {
 logger.info("Key {} has been disabled successfully",keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to disable key: " + keyId,
throwable);
 });
}

```

```
}
```

- For API details, see [DisableKey](#) in *AWS SDK for Java 2.x API Reference*.

## EnableKey

The following code example shows how to use EnableKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously enables the specified key.
 *
 * @param keyId the ID of the key to enable
 * @return a {@link CompletableFuture} that completes when the key has been
 * enabled
 */
public CompletableFuture<Void> enableKeyAsync(String keyId) {
 EnableKeyRequest enableKeyRequest = EnableKeyRequest.builder()
 .keyId(keyId)
 .build();

 CompletableFuture<EnableKeyResponse> responseFuture =
getAsyncClient().enableKey(enableKeyRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("Key with ID [{}] has been enabled.", keyId);
 } else {
 if (exception instanceof KmsException kmsEx) {
 throw new RuntimeException("KMS error occurred while enabling
key: " + kmsEx.getMessage(), kmsEx);
 } else {
 throw new RuntimeException("An unexpected error occurred while
enabling key: " + exception.getMessage(), exception);
 }
 }
 });
}
```

```

 }
 });

 return responseFuture.thenApply(response -> null);
}

```

- For API details, see [EnableKey](#) in *AWS SDK for Java 2.x API Reference*.

## Encrypt

The following code example shows how to use Encrypt.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Encrypts the given text asynchronously using the specified KMS client and key
 * ID.
 *
 * @param keyId the ID of the KMS key to use for encryption
 * @param text the text to encrypt
 * @return a CompletableFuture that completes with the encrypted data as an
 * SdkBytes object
 */
public CompletableFuture<SdkBytes> encryptDataAsync(String keyId, String text) {
 SdkBytes myBytes = SdkBytes.fromUtf8String(text);
 EncryptRequest encryptRequest = EncryptRequest.builder()
 .keyId(keyId)
 .plaintext(myBytes)
 .build();

 CompletableFuture<EncryptResponse> responseFuture =
 getAsyncClient().encrypt(encryptRequest).toCompletableFuture();
 return responseFuture.whenComplete((response, ex) -> {
 if (response != null) {
 String algorithm = response.encryptionAlgorithm().toString();

```

```

 logger.info("The string was encrypted with algorithm {}",
algorithm);
 } else {
 throw new RuntimeException(ex);
 }
}).thenApply(EncryptResponse::ciphertextBlob);
}

```

- For API details, see [Encrypt](#) in *AWS SDK for Java 2.x API Reference*.

## ListAliases

The following code example shows how to use `ListAliases`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously lists all the aliases in the current AWS account.
 *
 * @return a {@link CompletableFuture} that completes when the list of aliases
has been processed
 */
public CompletableFuture<Object> listAllAliasesAsync() {
 ListAliasesRequest aliasesRequest = ListAliasesRequest.builder()
 .limit(15)
 .build();

 ListAliasesPublisher paginator =
getAsyncClient().listAliasesPaginator(aliasesRequest);
 return paginator.subscribe(response -> {
 response.aliases().forEach(alias ->
 logger.info("The alias name is: " + alias.aliasName())
);
 })
 .thenApply(v -> null)

```

```

 .exceptionally(ex -> {
 if (ex.getCause() instanceof KmsException) {
 KmsException e = (KmsException) ex.getCause();
 throw new RuntimeException("A KMS exception occurred: " +
e.getMessage());
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
ex.getMessage());
 }
 });
 }
}

```

- For API details, see [ListAliases](#) in *AWS SDK for Java 2.x API Reference*.

## ListGrants

The following code example shows how to use ListGrants.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously displays the grant IDs for the specified key ID.
 *
 * @param keyId the ID of the AWS KMS key for which to list the grants
 * @return a {@link CompletableFuture} that, when completed, will be null if
the operation succeeded, or will throw a {@link RuntimeException} if the operation
failed
 * @throws RuntimeException if there was an error listing the grants, either due
to an {@link KmsException} or an unexpected error
 */
public CompletableFuture<Object> displayGrantIdsAsync(String keyId) {
 ListGrantsRequest grantsRequest = ListGrantsRequest.builder()
 .keyId(keyId)
 .limit(15)
 .build();
}

```



```

 ListGrantsPublisher paginator =
getAsyncClient().listGrantsPaginator(grantsRequest);
 return paginator.subscribe(response -> {
 response.grants().forEach(grant -> {
 logger.info("The grant Id is: " + grant.grantId());
 });
 })
 .thenApply(v -> null)
 .exceptionally(ex -> {
 Throwable cause = ex.getCause();
 if (cause instanceof KmsException) {
 throw new RuntimeException("Failed to list grants: " +
cause.getMessage(), cause);
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
cause.getMessage(), cause);
 }
 });
 }
}

```

- For API details, see [ListGrants](#) in *AWS SDK for Java 2.x API Reference*.

## ListKeyPolicies

The following code example shows how to use ListKeyPolicies.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously retrieves the key policy for the specified key ID and policy
name.
 *
 * @param keyId the ID of the AWS KMS key for which to retrieve the policy
 * @param policyName the name of the key policy to retrieve

```

```

 * @return a {@link CompletableFuture} that, when completed, contains the key
 policy as a {@link String}
 */
 public CompletableFuture<String> getKeyPolicyAsync(String keyId, String
 policyName) {
 GetKeyPolicyRequest policyRequest = GetKeyPolicyRequest.builder()
 .keyId(keyId)
 .policyName(policyName)
 .build();

 return getAsyncClient().getKeyPolicy(policyRequest)
 .thenApply(response -> {
 String policy = response.policy();
 logger.info("The response is: " + policy);
 return policy;
 })
 .exceptionally(ex -> {
 throw new RuntimeException("Failed to get key policy", ex);
 });
 }
}

```

- For API details, see [ListKeyPolicies](#) in *AWS SDK for Java 2.x API Reference*.

## ListKeys

The following code example shows how to use ListKeys.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.kms.KmsAsyncClient;
import software.amazon.awssdk.services.kms.model.ListKeysRequest;
import software.amazon.awssdk.services.kms.paginators.ListKeysPublisher;
import java.util.concurrent.CompletableFuture;

/**

```

```

* Before running this Java V2 code example, set up your development
* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class HelloKMS {
 public static void main(String[] args) {
 listAllKeys();
 }

 public static void listAllKeys() {
 KmsAsyncClient kmsAsyncClient = KmsAsyncClient.builder()
 .build();
 ListKeysRequest listKeysRequest = ListKeysRequest.builder()
 .limit(15)
 .build();

 /*
 * The `subscribe` method is required when using paginator methods in the
 * AWS SDK
 * because paginator methods return an instance of a `ListKeysPublisher`,
 * which is
 * based on a reactive stream. This allows asynchronous retrieval of
 * paginated
 * results as they become available. By subscribing to the stream, we can
 * process
 * each page of results as they are emitted.
 */
 ListKeysPublisher keysPublisher =
 kmsAsyncClient.listKeysPaginator(listKeysRequest);
 CompletableFuture<Void> future = keysPublisher
 .subscribe(r -> r.keys().forEach(key ->
 System.out.println("The key ARN is: " + key.keyArn() + ". The key Id
is: " + key.keyId()))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 System.err.println("Error occurred: " + exception.getMessage());
 } else {
 System.out.println("Successfully listed all keys.");
 }
 });
 }
}

```

```

 try {
 future.join();
 } catch (Exception e) {
 System.err.println("Failed to list keys: " + e.getMessage());
 }
 }
}

```

- For API details, see [ListKeys](#) in *AWS SDK for Java 2.x API Reference*.

## RevokeGrant

The following code example shows how to use `RevokeGrant`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Revokes a grant for the specified AWS KMS key asynchronously.
 *
 * @param keyId The ID or key ARN of the AWS KMS key.
 * @param grantId The identifier of the grant to be revoked.
 * @return A {@link CompletableFuture} representing the asynchronous operation
of revoking the grant.
 * The {@link CompletableFuture} will complete with a {@link
RevokeGrantResponse} object
 * if the operation is successful, or with a {@code null} value if an
error occurs.
 */
public CompletableFuture<RevokeGrantResponse> revokeKeyGrantAsync(String keyId,
String grantId) {
 RevokeGrantRequest grantRequest = RevokeGrantRequest.builder()
 .keyId(keyId)
 .grantId(grantId)
 .build();

```

```

 CompletableFuture<RevokeGrantResponse> responseFuture =
getAsyncClient().revokeGrant(grantRequest);
 responseFuture.whenComplete((response, exception) -> {
 if (exception == null) {
 logger.info("Grant ID: [" + grantId + "] was successfully
revoked!");
 } else {
 if (exception instanceof KmsException kmsEx) {
 if (kmsEx.getMessage().contains("Grant does not exist")) {
 logger.info("The grant ID '" + grantId + "' does not exist.
Moving on...");
 } else {
 throw new RuntimeException("KMS error occurred: " +
kmsEx.getMessage(), kmsEx);
 }
 } else {
 throw new RuntimeException("An unexpected error occurred: " +
exception.getMessage(), exception);
 }
 }
 });

 return responseFuture;
 }

```

- For API details, see [RevokeGrant](#) in *AWS SDK for Java 2.x API Reference*.

## ScheduleKeyDeletion

The following code example shows how to use `ScheduleKeyDeletion`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Deletes a KMS key asynchronously.
 *
 * <p>Warning: Deleting a KMS key is a destructive and
 potentially dangerous operation.
 * When a KMS key is deleted, all data that was encrypted under the KMS key
 becomes unrecoverable.
 * This means that any files, databases, or other data that were encrypted using
 the deleted KMS key
 * will become permanently inaccessible. Exercise extreme caution when deleting
 KMS keys.</p>
 *
 * @param keyId the ID of the KMS key to delete
 * @return a {@link CompletableFuture} that completes when the key deletion is
 scheduled
 */
 public CompletableFuture<Void> deleteKeyAsync(String keyId) {
 ScheduleKeyDeletionRequest deletionRequest =
 ScheduleKeyDeletionRequest.builder()
 .keyId(keyId)
 .pendingWindowInDays(7)
 .build();

 return getAsyncClient().scheduleKeyDeletion(deletionRequest)
 .thenRun(() -> {
 logger.info("Key {} will be deleted in 7 days", keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to schedule key deletion for key
ID: " + keyId, throwable);
 });
 }

```

- For API details, see [ScheduleKeyDeletion](#) in *AWS SDK for Java 2.x API Reference*.

## Sign

The following code example shows how to use Sign.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously signs and verifies data using AWS KMS.
 *
 * <p>The method performs the following steps:
 *
 * Creates an AWS KMS key with the specified key spec, key usage, and
origin.
 * Signs the provided message using the created KMS key and the RSASSA-
PSS-SHA-256 algorithm.
 * Verifies the signature of the message using the created KMS key and
the RSASSA-PSS-SHA-256 algorithm.
 *
 *
 * @return a {@link CompletableFuture} that completes with the result of the
signature verification,
 * {@code true} if the signature is valid, {@code false} otherwise.
 * @throws KmsException if any error occurs during the KMS operations.
 * @throws RuntimeException if an unexpected error occurs.
 */
public CompletableFuture<Boolean> signVerifyDataAsync() {
 String signMessage = "Here is the message that will be digitally signed";

 // Create an AWS KMS key used to digitally sign data.
 CreateKeyRequest createKeyRequest = CreateKeyRequest.builder()
 .keySpec(KeySpec.RSA_2048)
 .keyUsage(KeyUsageType.SIGN_VERIFY)
 .origin(OriginType.AWS_KMS)
 .build();

 return getAsyncClient().createKey(createKeyRequest)
 .thenCompose(createKeyResponse -> {
 String keyId = createKeyResponse.keyMetadata().keyId();
```

```

 SdkBytes messageBytes = SdkBytes.fromString(signMessage,
Charset.defaultCharset());
 SignRequest signRequest = SignRequest.builder()
 .keyId(keyId)
 .message(messageBytes)
 .signingAlgorithm(SigningAlgorithmSpec.RSASSA_PSS_SHA_256)
 .build();

 return getAsyncClient().sign(signRequest)
 .thenCompose(signResponse -> {
 byte[] signedBytes = signResponse.signature().asByteArray();

 VerifyRequest verifyRequest = VerifyRequest.builder()
 .keyId(keyId)

 .message(SdkBytes.fromByteArray(signMessage.getBytes(Charset.defaultCharset())))

 .signature(SdkBytes.fromByteBuffer(ByteBuffer.wrap(signedBytes)))

 .signingAlgorithm(SigningAlgorithmSpec.RSASSA_PSS_SHA_256)
 .build();

 return getAsyncClient().verify(verifyRequest)
 .thenApply(verifyResponse -> {
 return (boolean) verifyResponse.signatureValid();
 });
 });
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to sign or verify data",
throwable);
 });
}

```

- For API details, see [Sign](#) in *AWS SDK for Java 2.x API Reference*.

## TagResource

The following code example shows how to use TagResource.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously tags a KMS key with a specific tag.
 *
 * @param keyId the ID of the KMS key to be tagged
 * @return a {@link CompletableFuture} that completes when the tagging operation
is finished
 */
public CompletableFuture<Void> tagKMSKeyAsync(String keyId) {
 Tag tag = Tag.builder()
 .tagKey("Environment")
 .tagValue("Production")
 .build();

 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .keyId(keyId)
 .tags(tag)
 .build();

 return getAsyncClient().tagResource(tagResourceRequest)
 .thenRun(() -> {
 logger.info("{} key was tagged", keyId);
 })
 .exceptionally(throwable -> {
 throw new RuntimeException("Failed to tag the KMS key", throwable);
 });
}
```

- For API details, see [TagResource](#) in *AWS SDK for Java 2.x API Reference*.

# Lambda examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Lambda.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

*AWS community contributions* are examples that were created and are maintained by multiple teams across AWS. To provide feedback, use the mechanism provided in the linked repositories.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)
- [AWS community contributions](#)

## Get started

### Hello Lambda

The following code example shows how to get started using Lambda.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Lists the AWS Lambda functions associated with the current AWS account.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is used
to interact with the AWS Lambda service
 *
 * @throws LambdaException if an error occurs while interacting with the AWS
Lambda service
 */
public static void listFunctions(LambdaClient awsLambda) {
 try {
 ListFunctionsResponse functionResult = awsLambda.listFunctions();
 List<FunctionConfiguration> list = functionResult.functions();
 for (FunctionConfiguration config : list) {
 System.out.println("The function name is " + config.functionName());
 }

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListFunctions](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an IAM role and Lambda function, then upload handler code.

- Invoke the function with a single parameter and get results.
- Update the function code and configure with an environment variable.
- Invoke the function with new parameters and get results. Display the returned execution log.
- List the functions for your account, then clean up resources.

For more information, see [Create a Lambda function with the console](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/*
 * Lambda function names appear as:
 *
 * arn:aws:lambda:us-west-2:335556666777:function:HelloFunction
 *
 * To find this value, look at the function in the AWS Management Console.
 *
 * Before running this Java code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This example performs the following tasks:
 *
 * 1. Creates an AWS Lambda function.
 * 2. Gets a specific AWS Lambda function.
 * 3. Lists all Lambda functions.
 * 4. Invokes a Lambda function.
 * 5. Updates the Lambda function code and invokes it again.
 * 6. Updates a Lambda function's configuration value.
 * 7. Deletes a Lambda function.
 */
```

```

public class LambdaScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws InterruptedException {
 final String usage = ""

 Usage:
 <functionName> <role> <handler> <bucketName> <key>\s

 Where:
 functionName - The name of the Lambda function.\s
 role - The AWS Identity and Access Management (IAM) service role
that has Lambda permissions.\s
 handler - The fully qualified method name (for example,
example.Handler::handleRequest).\s
 bucketName - The Amazon Simple Storage Service (Amazon S3) bucket
name that contains the .zip or .jar used to update the Lambda function's code.\s
 key - The Amazon S3 key name that represents the .zip or .jar (for
example, LambdaHello-1.0-SNAPSHOT.jar).
 """;

 if (args.length != 5) {
 System.out.println(usage);
 return;
 }

 String functionName = args[0];
 String role = args[1];
 String handler = args[2];
 String bucketName = args[3];
 String key = args[4];
 LambdaClient awsLambda = LambdaClient.builder()
 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the AWS Lambda Basics scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("1. Create an AWS Lambda function.");
 String funArn = createLambdaFunction(awsLambda, functionName, key,
bucketName, role, handler);
 System.out.println("The AWS Lambda ARN is " + funArn);
 System.out.println(DASHES);
 }
}

```

```
System.out.println(DASHES);
System.out.println("2. Get the " + functionName + " AWS Lambda function.");
getFunction(awsLambda, functionName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. List all AWS Lambda functions.");
listFunctions(awsLambda);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Invoke the Lambda function.");
System.out.println("*** Sleep for 1 min to get Lambda function ready.");
Thread.sleep(60000);
invokeFunction(awsLambda, functionName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Update the Lambda function code and invoke it
again.");
updateFunctionCode(awsLambda, functionName, bucketName, key);
System.out.println("*** Sleep for 1 min to get Lambda function ready.");
Thread.sleep(60000);
invokeFunction(awsLambda, functionName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Update a Lambda function's configuration value.");
updateFunctionConfiguration(awsLambda, functionName, handler);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Delete the AWS Lambda function.");
LambdaScenario.deleteLambdaFunction(awsLambda, functionName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("The AWS Lambda scenario completed successfully");
System.out.println(DASHES);
awsLambda.close();
}

/**
```

```

 * Creates a new Lambda function in AWS using the AWS Lambda Java API.
 *
 * @param awsLambda the AWS Lambda client used to interact with the AWS
Lambda service
 * @param functionName the name of the Lambda function to create
 * @param key the S3 key of the function code
 * @param bucketName the name of the S3 bucket containing the function code
 * @param role the IAM role to assign to the Lambda function
 * @param handler the fully qualified class name of the function handler
 * @return the Amazon Resource Name (ARN) of the created Lambda function
 */
 public static String createLambdaFunction(LambdaClient awsLambda,
 String functionName,
 String key,
 String bucketName,
 String role,
 String handler) {

 try {
 LambdaWaiter waiter = awsLambda.waiter();
 FunctionCode code = FunctionCode.builder()
 .s3Key(key)
 .s3Bucket(bucketName)
 .build();

 CreateFunctionRequest functionRequest = CreateFunctionRequest.builder()
 .functionName(functionName)
 .description("Created by the Lambda Java API")
 .code(code)
 .handler(handler)
 .runtime(Runtime.JAVA17)
 .role(role)
 .build();

 // Create a Lambda function using a waiter
 CreateFunctionResponse functionResponse =
awsLambda.createFunction(functionRequest);
 GetFunctionRequest getFunctionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();

 WaiterResponse<GetFunctionResponse> waiterResponse =
waiter.waitUntilFunctionExists(getFunctionRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 return functionResponse.functionArn();
 }
 }

```

```

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
 }

 /**
 * Retrieves information about an AWS Lambda function.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is
 * used to interact with the AWS Lambda service
 * @param functionName the name of the AWS Lambda function to retrieve
 * information about
 */
 public static void getFunction(LambdaClient awsLambda, String functionName) {
 try {
 GetFunctionRequest functionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();

 GetFunctionResponse response = awsLambda.getFunction(functionRequest);
 System.out.println("The runtime of this Lambda function is " +
 response.configuration().runtime());

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 /**
 * Lists the AWS Lambda functions associated with the current AWS account.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is used
 * to interact with the AWS Lambda service
 *
 * @throws LambdaException if an error occurs while interacting with the AWS
 * Lambda service
 */
 public static void listFunctions(LambdaClient awsLambda) {
 try {
 ListFunctionsResponse functionResult = awsLambda.listFunctions();

```



```

 List<FunctionConfiguration> list = functionResult.functions();
 for (FunctionConfiguration config : list) {
 System.out.println("The function name is " + config.functionName());
 }

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

/**
 * Invokes a specific AWS Lambda function.
 *
 * @param awsLambda an instance of {@link LambdaClient} to interact with the
AWS Lambda service
 * @param functionName the name of the AWS Lambda function to be invoked
 */
public static void invokeFunction(LambdaClient awsLambda, String functionName) {
 InvokeResponse res;
 try {
 // Need a SdkBytes instance for the payload.
 JSONObject jsonObj = new JSONObject();
 jsonObj.put("inputValue", "2000");
 String json = jsonObj.toString();
 SdkBytes payload = SdkBytes.fromUtf8String(json);

 InvokeRequest request = InvokeRequest.builder()
 .functionName(functionName)
 .payload(payload)
 .build();

 res = awsLambda.invoke(request);
 String value = res.payload().asUtf8String();
 System.out.println(value);

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

/**
 * Updates the code for an AWS Lambda function.

```

```

 *
 * @param awsLambda the AWS Lambda client
 * @param functionName the name of the Lambda function to update
 * @param bucketName the name of the S3 bucket where the function code is
 located
 * @param key the key (file name) of the function code in the S3 bucket
 * @throws LambdaException if there is an error updating the function code
 */
 public static void updateFunctionCode(LambdaClient awsLambda, String
 functionName, String bucketName, String key) {
 try {
 LambdaWaiter waiter = awsLambda.waiter();
 UpdateFunctionCodeRequest functionCodeRequest =
 UpdateFunctionCodeRequest.builder()
 .functionName(functionName)
 .publish(true)
 .s3Bucket(bucketName)
 .s3Key(key)
 .build();

 UpdateFunctionCodeResponse response =
 awsLambda.updateFunctionCode(functionCodeRequest);
 GetFunctionConfigurationRequest getFunctionConfigRequest =
 GetFunctionConfigurationRequest.builder()
 .functionName(functionName)
 .build();

 WaiterResponse<GetFunctionConfigurationResponse> waiterResponse = waiter
 .waitUntilFunctionUpdated(getFunctionConfigRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("The last modified value is " +
 response.lastModified());

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 /**
 * Updates the configuration of an AWS Lambda function.
 *
 * @param awsLambda the {@link LambdaClient} instance to use for the AWS
 Lambda operation

```

```

 * @param functionName the name of the AWS Lambda function to update
 * @param handler the new handler for the AWS Lambda function
 *
 * @throws LambdaException if there is an error while updating the function
configuration
 */
 public static void updateFunctionConfiguration(LambdaClient awsLambda, String
functionName, String handler) {
 try {
 UpdateFunctionConfigurationRequest configurationRequest =
UpdateFunctionConfigurationRequest.builder()
 .functionName(functionName)
 .handler(handler)
 .runtime(Runtime.JAVA17)
 .build();

 awsLambda.updateFunctionConfiguration(configurationRequest);

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 /**
 * Deletes an AWS Lambda function.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is
used to interact with the AWS Lambda service
 * @param functionName the name of the Lambda function to be deleted
 *
 * @throws LambdaException if an error occurs while deleting the Lambda function
 */
 public static void deleteLambdaFunction(LambdaClient awsLambda, String
functionName) {
 try {
 DeleteFunctionRequest request = DeleteFunctionRequest.builder()
 .functionName(functionName)
 .build();

 awsLambda.deleteFunction(request);
 System.out.println("The " + functionName + " function was deleted");

 } catch (LambdaException e) {

```

```

 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateFunction](#)
  - [DeleteFunction](#)
  - [GetFunction](#)
  - [Invoke](#)
  - [ListFunctions](#)
  - [UpdateFunctionCode](#)
  - [UpdateFunctionConfiguration](#)

## Actions

### CreateFunction

The following code example shows how to use `CreateFunction`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new Lambda function in AWS using the AWS Lambda Java API.
 *
 * @param awsLambda the AWS Lambda client used to interact with the AWS
Lambda service
 * @param functionName the name of the Lambda function to create
 * @param key the S3 key of the function code
 * @param bucketName the name of the S3 bucket containing the function code

```

```

 * @param role the IAM role to assign to the Lambda function
 * @param handler the fully qualified class name of the function handler
 * @return the Amazon Resource Name (ARN) of the created Lambda function
 */
 public static String createLambdaFunction(LambdaClient awsLambda,
 String functionName,
 String key,
 String bucketName,
 String role,
 String handler) {

 try {
 LambdaWaiter waiter = awsLambda.waiter();
 FunctionCode code = FunctionCode.builder()
 .s3Key(key)
 .s3Bucket(bucketName)
 .build();

 CreateFunctionRequest functionRequest = CreateFunctionRequest.builder()
 .functionName(functionName)
 .description("Created by the Lambda Java API")
 .code(code)
 .handler(handler)
 .runtime(Runtime.JAVA17)
 .role(role)
 .build();

 // Create a Lambda function using a waiter
 CreateFunctionResponse functionResponse =
awsLambda.createFunction(functionRequest);
 GetFunctionRequest getFunctionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();

 WaiterResponse<GetFunctionResponse> waiterResponse =
waiter.waitUntilFunctionExists(getFunctionRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 return functionResponse.functionArn();

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [CreateFunction](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteFunction

The following code example shows how to use DeleteFunction.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an AWS Lambda function.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is
 * used to interact with the AWS Lambda service
 * @param functionName the name of the Lambda function to be deleted
 *
 * @throws LambdaException if an error occurs while deleting the Lambda function
 */
public static void deleteLambdaFunction(LambdaClient awsLambda, String
functionName) {
 try {
 DeleteFunctionRequest request = DeleteFunctionRequest.builder()
 .functionName(functionName)
 .build();

 awsLambda.deleteFunction(request);
 System.out.println("The " + functionName + " function was deleted");

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteFunction](#) in *AWS SDK for Java 2.x API Reference*.

## GetFunction

The following code example shows how to use `GetFunction`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves information about an AWS Lambda function.
 *
 * @param awsLambda an instance of the {@link LambdaClient} class, which is
 * used to interact with the AWS Lambda service
 * @param functionName the name of the AWS Lambda function to retrieve
 * information about
 */
public static void getFunction(LambdaClient awsLambda, String functionName) {
 try {
 GetFunctionRequest functionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();

 GetFunctionResponse response = awsLambda.getFunction(functionRequest);
 System.out.println("The runtime of this Lambda function is " +
 response.configuration().runtime());

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetFunction](#) in *AWS SDK for Java 2.x API Reference*.

## Invoke

The following code example shows how to use Invoke.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Invokes a specific AWS Lambda function.
 *
 * @param awsLambda an instance of {@link LambdaClient} to interact with the
AWS Lambda service
 * @param functionName the name of the AWS Lambda function to be invoked
 */
public static void invokeFunction(LambdaClient awsLambda, String functionName) {
 InvokeResponse res;
 try {
 // Need a SdkBytes instance for the payload.
 JSONObject jsonObj = new JSONObject();
 jsonObj.put("inputValue", "2000");
 String json = jsonObj.toString();
 SdkBytes payload = SdkBytes.fromUtf8String(json);

 InvokeRequest request = InvokeRequest.builder()
 .functionName(functionName)
 .payload(payload)
 .build();

 res = awsLambda.invoke(request);
 String value = res.payload().asUtf8String();
 System.out.println(value);

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```



- For API details, see [Invoke](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateFunctionCode

The following code example shows how to use `UpdateFunctionCode`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Updates the code for an AWS Lambda function.
 *
 * @param awsLambda the AWS Lambda client
 * @param functionName the name of the Lambda function to update
 * @param bucketName the name of the S3 bucket where the function code is
 * located
 * @param key the key (file name) of the function code in the S3 bucket
 * @throws LambdaException if there is an error updating the function code
 */
public static void updateFunctionCode(LambdaClient awsLambda, String
functionName, String bucketName, String key) {
 try {
 LambdaWaiter waiter = awsLambda.waiter();
 UpdateFunctionCodeRequest functionCodeRequest =
UpdateFunctionCodeRequest.builder()
 .functionName(functionName)
 .publish(true)
 .s3Bucket(bucketName)
 .s3Key(key)
 .build();

 UpdateFunctionCodeResponse response =
awsLambda.updateFunctionCode(functionCodeRequest);
 GetFunctionConfigurationRequest getFunctionConfigRequest =
GetFunctionConfigurationRequest.builder()
```

```

 .functionName(functionName)
 .build();

 WaiterResponse<GetFunctionConfigurationResponse> waiterResponse = waiter
 .waitUntilFunctionUpdated(getFunctionConfigRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("The last modified value is " +
response.lastModified());

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [UpdateFunctionCode](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateFunctionConfiguration

The following code example shows how to use UpdateFunctionConfiguration.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Updates the configuration of an AWS Lambda function.
 *
 * @param awsLambda the {@link LambdaClient} instance to use for the AWS
Lambda operation
 * @param functionName the name of the AWS Lambda function to update
 * @param handler the new handler for the AWS Lambda function
 *
 * @throws LambdaException if there is an error while updating the function
configuration
 */

```

```
public static void updateFunctionConfiguration(LambdaClient awsLambda, String
functionName, String handler) {
 try {
 UpdateFunctionConfigurationRequest configurationRequest =
UpdateFunctionConfigurationRequest.builder()
 .functionName(functionName)
 .handler(handler)
 .runtime(Runtime.JAVA17)
 .build();

 awsLambda.updateFunctionConfiguration(configurationRequest);

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [UpdateFunctionConfiguration](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

#### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

#### Services used in this example

- API Gateway
- DynamoDB

- Lambda
- Amazon Rekognition
- Amazon S3
- Amazon SNS

## Create an application to analyze customer feedback

The following code example shows how to create an application that analyzes customer comment cards, translates them from their original language, determines their sentiment, and generates an audio file from the translated text.

### SDK for Java 2.x

This example application analyzes and stores customer feedback cards. Specifically, it fulfills the need of a fictitious hotel in New York City. The hotel receives feedback from guests in various languages in the form of physical comment cards. That feedback is uploaded into the app through a web client. After an image of a comment card is uploaded, the following steps occur:

- Text is extracted from the image using Amazon Textract.
- Amazon Comprehend determines the sentiment of the extracted text and its language.
- The extracted text is translated to English using Amazon Translate.
- Amazon Polly synthesizes an audio file from the extracted text.

The full app can be deployed with the AWS CDK. For source code and deployment instructions, see the project in [GitHub](#).

### Services used in this example

- Amazon Comprehend
- Lambda
- Amazon Polly
- Amazon Textract
- Amazon Translate

## Use API Gateway to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by Amazon API Gateway.

## SDK for Java 2.x

Shows how to create an AWS Lambda function by using the Lambda Java runtime API.

This example invokes different AWS services to perform a specific use case. This example demonstrates how to create a Lambda function invoked by Amazon API Gateway that scans an Amazon DynamoDB table for work anniversaries and uses Amazon Simple Notification Service (Amazon SNS) to send a text message to your employees that congratulates them at their one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon SNS

## Use Step Functions to invoke Lambda functions

The following code example shows how to create an AWS Step Functions state machine that invokes AWS Lambda functions in sequence.

## SDK for Java 2.x

Shows how to create an AWS serverless workflow by using AWS Step Functions and the AWS SDK for Java 2.x. Each workflow step is implemented using an AWS Lambda function.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Lambda
- Amazon SES
- Step Functions

## Use scheduled events to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by an Amazon EventBridge scheduled event.

### SDK for Java 2.x

Shows how to create an Amazon EventBridge scheduled event that invokes an AWS Lambda function. Configure EventBridge to use a cron expression to schedule when the Lambda function is invoked. In this example, you create a Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create an app that sends a mobile text message to your employees that congratulates them at the one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- CloudWatch Logs
- DynamoDB
- EventBridge
- Lambda
- Amazon SNS

## Use the Neptune API to query graph data

The following code example shows how to use the Neptune API to query graph data.

### SDK for Java 2.x

Shows how to use Amazon Neptune Java API to create a Lambda function that queries graph data within the VPC.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Lambda
- Neptune

## Serverless examples

### Connecting to an Amazon RDS database in a Lambda function

The following code example shows how to implement a Lambda function that connects to an RDS database. The function makes a simple database request and returns the result.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Connecting to an Amazon RDS database in a Lambda function using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyRequestEvent;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyResponseEvent;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rdsdata.RdsDataClient;
import software.amazon.awssdk.services.rdsdata.model.ExecuteStatementRequest;
import software.amazon.awssdk.services.rdsdata.model.ExecuteStatementResponse;
import software.amazon.awssdk.services.rdsdata.model.Field;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class RdsLambdaHandler implements RequestHandler<APIGatewayProxyRequestEvent,
 APIGatewayProxyResponseEvent> {

 @Override
 public APIGatewayProxyResponseEvent handleRequest(APIGatewayProxyRequestEvent
 event, Context context) {
 APIGatewayProxyResponseEvent response = new APIGatewayProxyResponseEvent();

 try {
```

```

 // Obtain auth token
 String token = createAuthToken();

 // Define connection configuration
 String connectionString = String.format("jdbc:mysql://%s:%s/%s?
useSSL=true&requireSSL=true",
 System.getenv("ProxyHostName"),
 System.getenv("Port"),
 System.getenv("DBName"));

 // Establish a connection to the database
 try (Connection connection =
DriverManager.getConnection(connectionString, System.getenv("DBUserName"), token);
 PreparedStatement statement = connection.prepareStatement("SELECT ?
+ ? AS sum")) {

 statement.setInt(1, 3);
 statement.setInt(2, 2);

 try (ResultSet resultSet = statement.executeQuery()) {
 if (resultSet.next()) {
 int sum = resultSet.getInt("sum");
 response.setStatusCode(200);
 response.setBody("The selected sum is: " + sum);
 }
 }
 }

 } catch (Exception e) {
 response.setStatusCode(500);
 response.setBody("Error: " + e.getMessage());
 }

 return response;
 }

 private String createAuthToken() {
 // Create RDS Data Service client
 RdsDataClient rdsDataClient = RdsDataClient.builder()
 .region(Region.of(System.getenv("AWS_REGION")))
 .credentialsProvider(DefaultCredentialsProvider.create())
 .build();

 // Define authentication request

```



```

 ExecuteStatementRequest request = ExecuteStatementRequest.builder()
 .resourceArn(System.getenv("ProxyHostName"))
 .secretArn(System.getenv("DBUserName"))
 .database(System.getenv("DBName"))
 .sql("SELECT 'RDS IAM Authentication'")
 .build();

 // Execute request and obtain authentication token
 ExecuteStatementResponse response = rdsDataClient.executeStatement(request);
 Field tokenField = response.records().get(0).get(0);

 return tokenField.stringValue();
 }
}

```

## Invoke a Lambda function from a Kinesis trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a Kinesis stream. The function retrieves the Kinesis payload, decodes from Base64, and logs the record contents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Consuming a Kinesis event with Lambda using Java.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KinesisEvent;

```

```
public class Handler implements RequestHandler<KinesisEvent, Void> {
 @Override
 public Void handleRequest(final KinesisEvent event, final Context context) {
 LambdaLogger logger = context.getLogger();
 if (event.getRecords().isEmpty()) {
 logger.log("Empty Kinesis Event received");
 return null;
 }
 for (KinesisEvent.KinesisEventRecord record : event.getRecords()) {
 try {
 logger.log("Processed Event with EventId: "+record.getEventID());
 String data = new String(record.getKinesis().getData().array());
 logger.log("Data:"+ data);
 // TODO: Do interesting work based on the new data
 }
 catch (Exception ex) {
 logger.log("An error occurred:"+ex.getMessage());
 throw ex;
 }
 }
 logger.log("Successfully processed:"+event.getRecords().size()+" records");
 return null;
 }
}
```

## Invoke a Lambda function from a DynamoDB trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a DynamoDB stream. The function retrieves the DynamoDB payload and logs the record contents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Consuming a DynamoDB event with Lambda using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.DynamodbEvent;
import
 com.amazonaws.services.lambda.runtime.events.DynamodbEvent.DynamodbStreamRecord;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

public class example implements RequestHandler<DynamodbEvent, Void> {

 private static final Gson GSON = new GsonBuilder().setPrettyPrinting().create();

 @Override
 public Void handleRequest(DynamodbEvent event, Context context) {
 System.out.println(GSON.toJson(event));
 event.getRecords().forEach(this::logDynamoDBRecord);
 return null;
 }

 private void logDynamoDBRecord(DynamodbStreamRecord record) {
 System.out.println(record.getEventID());
 System.out.println(record.getEventName());
 System.out.println("DynamoDB Record: " + GSON.toJson(record.getDynamodb()));
 }
}
```

## Invoke a Lambda function from a Amazon DocumentDB trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from a DocumentDB change stream. The function retrieves the DocumentDB payload and logs the record contents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Consuming a Amazon DocumentDB event with Lambda using Java.

```
import java.util.List;
import java.util.Map;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;

public class Example implements RequestHandler<Map<String, Object>, String> {

 @SuppressWarnings("unchecked")
 @Override
 public String handleRequest(Map<String, Object> event, Context context) {
 List<Map<String, Object>> events = (List<Map<String, Object>>)
event.get("events");
 for (Map<String, Object> record : events) {
 Map<String, Object> eventData = (Map<String, Object>)
record.get("event");
 processEventData(eventData);
 }

 return "OK";
 }

 @SuppressWarnings("unchecked")
 private void processEventData(Map<String, Object> eventData) {
 String operationType = (String) eventData.get("operationType");
 System.out.println("operationType: %s".formatted(operationType));

 Map<String, Object> ns = (Map<String, Object>) eventData.get("ns");

 String db = (String) ns.get("db");
 System.out.println("db: %s".formatted(db));
 String coll = (String) ns.get("coll");
 System.out.println("coll: %s".formatted(coll));

 Map<String, Object> fullDocument = (Map<String, Object>)
eventData.get("fullDocument");
 System.out.println("fullDocument: %s".formatted(fullDocument));
 }
}
```

## Invoke a Lambda function from an Amazon MSK trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from an Amazon MSK cluster. The function retrieves the MSK payload and logs the record contents.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming an Amazon MSK event with Lambda using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KafkaEvent;
import com.amazonaws.services.lambda.runtime.events.KafkaEvent.KafkaEventRecord;

import java.util.Base64;
import java.util.Map;

public class Example implements RequestHandler<KafkaEvent, Void> {

 @Override
 public Void handleRequest(KafkaEvent event, Context context) {
 for (Map.Entry<String, java.util.List<KafkaEventRecord>> entry :
event.getRecords().entrySet()) {
 String key = entry.getKey();
 System.out.println("Key: " + key);

 for (KafkaEventRecord record : entry.getValue()) {
 System.out.println("Record: " + record);

 byte[] value = Base64.getDecoder().decode(record.getValue());
 String message = new String(value);
 System.out.println("Message: " + message);
 }
 }
 }
}
```

```
 return null;
 }
}
```

## Invoke a Lambda function from an Amazon S3 trigger

The following code example shows how to implement a Lambda function that receives an event triggered by uploading an object to an S3 bucket. The function retrieves the S3 bucket name and object key from the event parameter and calls the Amazon S3 API to retrieve and log the content type of the object.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Consuming an S3 event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.S3Client;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.S3Event;
import
 com.amazonaws.services.lambda.runtime.events.models.s3.S3EventNotification.S3EventNotificat

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Handler implements RequestHandler<S3Event, String> {
```

```

private static final Logger logger = LoggerFactory.getLogger(Handler.class);
@Override
public String handleRequest(S3Event s3event, Context context) {
 try {
 S3EventNotificationRecord record = s3event.getRecords().get(0);
 String srcBucket = record.getS3().getBucket().getName();
 String srcKey = record.getS3().getObject().getUrlDecodedKey();

 S3Client s3Client = S3Client.builder().build();
 HeadObjectResponse headObject = getHeadObject(s3Client, srcBucket,
srcKey);

 logger.info("Successfully retrieved " + srcBucket + "/" + srcKey + " of
type " + headObject.contentType());

 return "Ok";
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}

private HeadObjectResponse getHeadObject(S3Client s3Client, String bucket,
String key) {
 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(bucket)
 .key(key)
 .build();
 return s3Client.headObject(headObjectRequest);
}
}

```

## Invoke a Lambda function from an Amazon SNS trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving messages from an SNS topic. The function retrieves the messages from the event parameter and logs the content of each message.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

### Consuming an SNS event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SNSEvent;
import com.amazonaws.services.lambda.runtime.events.SNSEvent.SNSRecord;

import java.util.Iterator;
import java.util.List;

public class SNSEventHandler implements RequestHandler<SNSEvent, Boolean> {
 LambdaLogger logger;

 @Override
 public Boolean handleRequest(SNSEvent event, Context context) {
 logger = context.getLogger();
 List<SNSRecord> records = event.getRecords();
 if (!records.isEmpty()) {
 Iterator<SNSRecord> recordsIter = records.iterator();
 while (recordsIter.hasNext()) {
 processRecord(recordsIter.next());
 }
 }
 return Boolean.TRUE;
 }

 public void processRecord(SNSRecord record) {
 try {
 String message = record.getSNS().getMessage();
 }
 }
}
```



```
 logger.log("message: " + message);
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}
}
```

## Invoke a Lambda function from an Amazon SQS trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving messages from an SQS queue. The function retrieves the messages from the event parameter and logs the content of each message.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming an SQS event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SQSEvent;
import com.amazonaws.services.lambda.runtime.events.SQSEvent.SQSMessage;

public class Function implements RequestHandler<SQSEvent, Void> {
 @Override
 public Void handleRequest(SQSEvent sqsEvent, Context context) {
 for (SQSMessage msg : sqsEvent.getRecords()) {
 processMessage(msg, context);
 }
 }
}
```

```
 context.getLogger().log("done");
 return null;
 }

 private void processMessage(SQSMessage msg, Context context) {
 try {
 context.getLogger().log("Processed message " + msg.getBody());

 // TODO: Do interesting work based on the new message

 } catch (Exception e) {
 context.getLogger().log("An error occurred");
 throw e;
 }
 }
}
```

## Reporting batch item failures for Lambda functions with a Kinesis trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from a Kinesis stream. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Reporting Kinesis batch item failures with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KinesisEvent;
import com.amazonaws.services.lambda.runtime.events.StreamsEventResponse;
```

```

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class ProcessKinesisRecords implements RequestHandler<KinesisEvent,
StreamsEventResponse> {

 @Override
 public StreamsEventResponse handleRequest(KinesisEvent input, Context context) {

 List<StreamsEventResponse.BatchItemFailure> batchItemFailures = new
ArrayList<>();
 String curRecordSequenceNumber = "";

 for (KinesisEvent.KinesisEventRecord kinesisEventRecord :
input.getRecords()) {
 try {
 //Process your record
 KinesisEvent.Record kinesisRecord = kinesisEventRecord.getKinesis();
 curRecordSequenceNumber = kinesisRecord.getSequenceNumber();

 } catch (Exception e) {
 /* Since we are working with streams, we can return the failed item
immediately.
 Lambda will immediately begin to retry processing from this
failed item onwards. */
 batchItemFailures.add(new
StreamsEventResponse.BatchItemFailure(curRecordSequenceNumber));
 return new StreamsEventResponse(batchItemFailures);
 }
 }

 return new StreamsEventResponse(batchItemFailures);
 }
}

```

## Reporting batch item failures for Lambda functions with a DynamoDB trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from a DynamoDB stream. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

### Reporting DynamoDB batch item failures with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.DynamodbEvent;
import com.amazonaws.services.lambda.runtime.events.StreamsEventResponse;
import com.amazonaws.services.lambda.runtime.events.models.dynamodb.StreamRecord;

import java.util.ArrayList;
import java.util.List;

public class ProcessDynamodbRecords implements RequestHandler<DynamodbEvent,
 StreamsEventResponse> {

 @Override
 public StreamsEventResponse handleRequest(DynamodbEvent input, Context context)
 {

 List<StreamsEventResponse.BatchItemFailure> batchItemFailures = new
 ArrayList<>();
 String curRecordSequenceNumber = "";

 for (DynamodbEvent.DynamodbStreamRecord dynamodbStreamRecord :
 input.getRecords()) {
 try {
 //Process your record
 StreamRecord dynamodbRecord = dynamodbStreamRecord.getDynamodb();
 curRecordSequenceNumber = dynamodbRecord.getSequenceNumber();

 } catch (Exception e) {
 /* Since we are working with streams, we can return the failed item
 immediately.
```

```

 Lambda will immediately begin to retry processing from this
 failed item onwards. */
 batchItemFailures.add(new
StreamsEventResponse.BatchItemFailure(curRecordSequenceNumber));
 return new StreamsEventResponse(batchItemFailures);
 }
}

return new StreamsEventResponse();
}
}

```

## Reporting batch item failures for Lambda functions with an Amazon SQS trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from an SQS queue. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

## Reporting SQS batch item failures with Lambda using Java.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SQSEvent;
import com.amazonaws.services.lambda.runtime.events.SQSBatchResponse;

import java.util.ArrayList;
import java.util.List;

public class ProcessSQSMessageBatch implements RequestHandler<SQSEvent,
 SQSBatchResponse> {

```

```
@Override
public SQSBatchResponse handleRequest(SQSEvent sqsEvent, Context context) {
 List<SQSBatchResponse.BatchItemFailure> batchItemFailures = new
 ArrayList<SQSBatchResponse.BatchItemFailure>();

 for (SQSEvent.SQSMessage message : sqsEvent.getRecords()) {
 try {
 //process your message
 } catch (Exception e) {
 //Add failed message identifier to the batchItemFailures list
 batchItemFailures.add(new
 SQSBatchResponse.BatchItemFailure(message.getMessageId()));
 }
 }
 return new SQSBatchResponse(batchItemFailures);
}
```

## AWS community contributions

### Build and test a serverless application

The following code example shows how to build and test a serverless application using API Gateway with Lambda and DynamoDB

#### SDK for Java 2.x

Shows how to build and test a serverless application that consists of an API Gateway with Lambda and DynamoDB using the Java SDK.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- API Gateway
- DynamoDB
- Lambda

## Amazon Lex examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Lex.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Scenarios](#)

## Scenarios

### Building an Amazon Lex chatbot

The following code example shows how to create a chatbot to engage your website visitors.

#### SDK for Java 2.x

Shows how to use the Amazon Lex API to create a Chatbot within a web application to engage your web site visitors.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Comprehend
- Amazon Lex
- Amazon Translate

## Amazon Location examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Location.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Amazon Location

The following code example shows how to get started using Amazon Location Service.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * In addition, you need to create a collection using the AWS Management
 * console. For information, see the following documentation.
 *
 * https://docs.aws.amazon.com/location/latest/developerguide/geofence-gs.html
 */
public class HelloLocation {
```



```

private static LocationAsyncClient locationAsyncClient;
private static final Logger logger =
LoggerFactory.getLogger>HelloLocation.class);

// This Singleton pattern ensures that only one `LocationClient`
// instance.
private static LocationAsyncClient getClient() {
 if (locationAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 locationAsyncClient = LocationAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return locationAsyncClient;
}

public static void main(String[] args) {
 final String usage = ""

 Usage:
 <collectionName>

 Where:
 collectionName - The Amazon location collection name.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
}

```

```

 }

 String collectionName = args[0];
 listGeofences(collectionName);
}

/**
 * Lists geofences from a specified geofence collection asynchronously.
 *
 * @param collectionName The name of the geofence collection to list geofences
 * from.
 * @return A {@link CompletableFuture} representing the result of the
 * asynchronous operation.
 *
 * The future completes when all geofences have been processed and
 * logged.
 */
public static CompletableFuture<Void> listGeofences(String collectionName) {
 ListGeofencesRequest geofencesRequest = ListGeofencesRequest.builder()
 .collectionName(collectionName)
 .build();

 ListGeofencesPublisher paginator =
getClient().listGeofencesPaginator(geofencesRequest);
 CompletableFuture<Void> future = paginator.subscribe(response -> {
 if (response.entries().isEmpty()) {
 logger.info("No Geofences were found in the collection.");
 } else {
 response.entries().forEach(geofence ->
 logger.info("Geofence ID: " + geofence.geofenceId())
);
 }
 });
 return future;
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [ListGeofenceCollections](#)
  - [ListGeofences](#)

# Basics

## Learn the basics

The following code example shows how to:

- Create an Amazon Location map.
- Create an Amazon Location API key.
- Display Map URL.
- Create a geofence collection.
- Store a geofence geometry.
- Create a tracker resource.
- Update the position of a device.
- Retrieve the most recent position update for a specified device.
- Create a route calculator.
- Determine the distance between Seattle and Vancouver.
- Use Amazon Location higher level APIs.
- Delete the Amazon Location Assets.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Amazon Location Service features.

```
/*
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
```

```

*/
public class LocationScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 private static final Logger logger =
LoggerFactory.getLogger(LocationScenario.class);
 static Scanner scanner = new Scanner(System.in);

 static LocationActions locationActions = new LocationActions();

 public static void main(String[] args) {
 final String usage = ""

 Usage: <mapName> <keyName> <collectionName> <geoId> <trackerName>
<calculatorName> <deviceId>

 Where:
 mapName - The name of the map to be create (e.g., "AWSMap").
 keyName - The name of the API key to create (e.g., "AWSApiKey").
 collectionName - The name of the geofence collection (e.g.,
"AWSLocationCollection").
 geoId - The geographic identifier used for the geofence or map (e.g.,
"geoId").
 trackerName - The name of the tracker (e.g., "geoTracker").
 calculatorName - The name of the route calculator (e.g.,
"AWSRouteCalc").
 deviceId - The ID of the device (e.g., "iPhone-112356").
 "";

 if (args.length != 7) {
 logger.info(usage);
 return;
 }

 String mapName = args[0];
 String keyName = args[1];
 String collectionName = args[2];
 String geoId = args[3];
 String trackerName = args[4];
 String calculatorName = args[5];
 String deviceId = args[6];

 logger.info("")

```

AWS Location Service is a fully managed service offered by Amazon Web Services (AWS) that provides location-based services for developers. This service simplifies the integration of location-based features into applications, making it easier to build and deploy location-aware applications.

The AWS Location Service offers a range of location-based services, including:

**Maps:** The service provides access to high-quality maps, satellite imagery, and geospatial data from various providers, allowing developers to easily embed maps into their applications.

**Tracking:** The Location Service enables real-time tracking of mobile devices, assets, or other entities, allowing developers to build applications that can monitor the location of people, vehicles, or other objects.

**Geocoding:** The service provides the ability to convert addresses or location names into geographic coordinates (latitude and longitude) and vice versa, enabling developers to integrate location-based search and routing functionality into their applications.

```

 "");
 waitForInputToContinue(scanner);
 try {
 runScenario(mapName, keyName, collectionName, geoId, trackerName,
calculatorName, deviceId);
 } catch (RuntimeException e) {
 // Clean up AWS Resources.
 cleanUp(mapName, keyName, collectionName, trackerName, calculatorName);
 logger.info(e.getMessage());
 }
}

public static void runScenario(String mapName, String keyName, String
collectionName, String geoId, String trackerName, String calculatorName, String
deviceId) {
 logger.info(DASHES);
 logger.info("1. Create a map");
 logger.info("
 An AWS Location map can enhance the user experience of your
 application by providing accurate and personalized location-based
 features. For example, you could use the geocoding capabilities to

```

```

 allow users to search for and locate businesses, landmarks, or
 other points of interest within a specific region.
 """);

waitForInputToContinue(scanner);
String mapArn;
try {
 mapArn = locationActions.createMap(mapName).join();
 logger.info("The Map ARN is: {}", mapArn); // Log success in calling
code
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 logger.error("The request exceeded the maximum quota: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred while creating the map.",
cause);
 }
 return;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("2. Create an AWS Location API key");
logger.info("""
 When you embed a map in a web app or website, the API key is
 included in the map tile URL to authenticate requests. You can
 restrict API keys to specific AWS Location operations (e.g., only
 maps, not geocoding). API keys can expire, ensuring temporary
 access control.
 """);

try {
 String keyArn = locationActions.createKey(keyName, mapArn).join();
 logger.info("The API key was successfully created: {}", keyArn);
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof AccessDeniedException) {
 logger.error("Request was denied: {}", cause.getMessage());
 } else {
 logger.error("An unexpected error occurred while creating the API
key.", cause);

```

```

 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Display Map URL");
 logger.info("""
 In order to get the MAP URL, you need to get the API Key value.
 You can get the key value using the AWS Management Console under
 Location Services. This operation cannot be completed using the
 AWS SDK. For more information about getting the key value, see
 the AWS Location Documentation.
 """);
 String mapUrl = "https://maps.geo.aws.amazon.com/maps/v0/maps/"+mapName+"/
tiles/{z}/{x}/{y}?key={KeyValue}";
 logger.info("Embed this URL in your Web app: " + mapUrl);
 logger.info("");
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("4. Create a geofence collection, which manages and stores
geofences.");
 waitForInputToContinue(scanner);
 try {
 String collectionArn =
locationActions.createGeofenceCollection(collectionName).join();
 logger.info("The geofence collection was successfully created: {}",
collectionArn);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ConflictException) {
 logger.error("A conflict occurred: {}", cause.getMessage());
 } else {
 logger.error("An unexpected error occurred while creating the
geofence collection.", cause);
 }
 return;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

```

```

logger.info(DASHES);
logger.info("5. Store a geofence geometry in a given geofence collection.");
logger.info("""
 An AWS Location geofence is a virtual boundary that defines a geographic
area
 on a map. It is a useful feature for tracking the location of
 assets or monitoring the movement of objects within a specific region.

 To define a geofence, you need to specify the coordinates of a
 polygon that represents the area of interest. The polygon must be
 defined in a counter-clockwise direction, meaning that the points of
 the polygon must be listed in a counter-clockwise order.

 This is a requirement for the AWS Location service to correctly
 interpret the geofence and ensure that the location data is
 accurately processed within the defined area.
 """);

waitForInputToContinue(scanner);
try {
 locationActions.putGeofence(collectionName, geoId).join();
 logger.info("Successfully created geofence: {}", geoId);
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ValidationException) {
 logger.error("A validation error occurred while creating geofence:
{}", cause.getMessage());
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info("6. Create a tracker resource which lets you retrieve current
and historical location of devices..");
waitForInputToContinue(scanner);
try {
 String trackerArn = locationActions.createTracker(trackerName).join();
 logger.info("Successfully created tracker. ARN: {}", trackerArn); //
Log success

```



```

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ConflictException) {
 logger.error("A conflict occurred while creating the tracker: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Update the position of a device in the location tracking
system.");
 logger.info("""
 The AWS location service does not enforce a strict format for deviceId,
but it must:
 - Be a string (case-sensitive).
 - Be 1-100 characters long.
 - Contain only:
 - Alphanumeric characters (A-Z, a-z, 0-9)
 - Underscores (_)
 - Hyphens (-)
 - Be the same ID used when sending and retrieving positions.
 """);

 waitForInputToContinue(scanner);
 try {
 CompletableFuture<BatchUpdateDevicePositionResponse> future =
locationActions.updateDevicePosition(trackerName, deviceId);
 future.join();
 logger.info(deviceId + " was successfully updated in the location
tracking system.");
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The resource was not found: {}", cause.getMessage(),
cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 }

```

```

 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("8. Retrieve the most recent position update for a specified
device..");
 waitForInputToContinue(scanner);
 try {
 GetDevicePositionResponse response =
locationActions.getDevicePosition(trackerName, deviceId).join();
 logger.info("Successfully fetched device position: {}",
response.position());
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The resource was not found: {}", cause.getMessage(),
cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("9. Create a route calculator.");
 waitForInputToContinue(scanner);
 try {
 CreateRouteCalculatorResponse response =
locationActions.createRouteCalculator(calculatorName).join();
 logger.info("Route calculator created successfully: {}",
response.calculatorArn());
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ConflictException) {
 logger.info("A conflict occurred: {}", cause.getMessage(), cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 }

```

```

 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("10. Determine the distance between Seattle and Vancouver using
the route calculator.");
 waitForInputToContinue(scanner);
 try {
 CalculateRouteResponse response =
locationActions.calcDistanceAsync(calculatorName).join();
 logger.info("Successfully calculated route. The distance in kilometers
is {}", response.summary().distance());
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The resource was not found: {}", cause.getMessage(),
cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info("11. Use the GeoPlacesAsyncClient to perform additional
operations.");
 logger.info("""
 This scenario will show use of the GeoPlacesClient that enables
 location search and geocoding capabilities for your applications.\s

 We are going to use this client to perform these AWS Location tasks:
 - Reverse Geocoding (reverseGeocode): Converts geographic coordinates
into addresses.
 - Place Search (searchText): Finds places based on search queries.
 - Nearby Search (searchNearby): Finds places near a specific location.
 """);

 logger.info("First we will perform a Reverse Geocoding operation");
 waitForInputToContinue(scanner);
 try {
 locationActions.reverseGeocode().join();
 }

```

```

 logger.info("Now we are going to perform a text search using coffee
shop.");
 waitForInputToContinue(scanner);
 locationActions.searchText("coffee shop").join();
 waitForInputToContinue(scanner);

 logger.info("Now we are going to perform a nearby Search.");
 //waitForInputToContinue(scanner);
 locationActions.searchNearBy().join();
 waitForInputToContinue(scanner);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 logger.error("A validation error occurred: {}", cause.getMessage(),
cause);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 logger.info(DASHES);

 logger.info("12. Delete the AWS Location Services resources.");
 logger.info("Would you like to delete the AWS Location Services resources?
(y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 cleanUp(mapName, keyName, collectionName, trackerName, calculatorName);
 } else {
 logger.info("The AWS resources will not be deleted.");
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info(" This concludes the AWS Location Service scenario.");
 logger.info(DASHES);
}

/**
 * Cleans up resources by deleting the specified map, key, geofence collection,
tracker, and route calculator.

```

```

 *
 * @param mapName The name of the map to delete.
 * @param keyName The name of the key to delete.
 * @param collectionName The name of the geofence collection to delete.
 * @param trackerName The name of the tracker to delete.
 * @param calculatorName The name of the route calculator to delete.
 */
 private static void cleanUp(String mapName, String keyName, String
collectionName, String trackerName, String calculatorName) {
 try {
 locationActions.deleteMap(mapName).join();
 locationActions.deleteKey(keyName).join();
 locationActions.deleteGeofenceCollectionAsync(collectionName).join();
 locationActions.deleteTracker(trackerName).join();
 locationActions.deleteRouteCalculator(calculatorName).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.info("The resource was not found: {}", cause.getMessage(),
cause);
 } else {
 logger.info("An unexpected error occurred: {}", cause.getMessage(),
cause);
 }
 return;
 }
 }

 private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 logger.info("Invalid input. Please try again.");
 }
 }
 }
}

```

A wrapper class for Amazon Location Service SDK methods.

```
public class LocationActions {

 private static LocationAsyncClient locationAsyncClient;

 private static GeoPlacesAsyncClient geoPlacesAsyncClient;
 private static final Logger logger =
 LoggerFactory.getLogger(LocationActions.class);

 // This Singleton pattern ensures that only one `LocationClient`
 // instance is used throughout the application.
 private LocationAsyncClient getClient() {
 if (locationAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
 ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 locationAsyncClient = LocationAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return locationAsyncClient;
 }

 private static GeoPlacesAsyncClient getGeoPlacesClient() {
 if (geoPlacesAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
```

```

 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 geoPlacesAsyncClient = GeoPlacesAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
}
return geoPlacesAsyncClient;
}

/**
 * Performs a nearby places search based on the provided geographic coordinates
 * (latitude and longitude).
 * The method sends an asynchronous request to search for places within a 1-
 * kilometer radius of the specified location.
 * The results are processed and printed once the search completes successfully.
 */
public CompletableFuture<SearchNearbyResponse> searchNearBy() {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 List<Double> queryPosition = List.of(longitude, latitude);

 // Set up the request for searching nearby places.
 SearchNearbyRequest request = SearchNearbyRequest.builder()
 .queryPosition(queryPosition) // Set the position
 .queryRadius(1000L) // Radius in meters (1000 meters = 1 km).
 .build();

 return getGeoPlacesClient().searchNearby(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {

```

```

 throw new CompletionException("A validation error occurred:
" + cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing place search",
exception);
}

// Process the response and print the results.
response.resultItems().forEach(result -> {
 logger.info("Place Name: " + result.placeType().name());
 logger.info("Address: " + result.address().label());
 logger.info("Distance: " + result.distance() + " meters");
 logger.info("-----");
});
});
}

/**
 * Searches for a place using the provided search query and prints the detailed
information of the first result.
 *
 * @param searchQuery the search query to be used for the place search (ex,
coffee shop)
 */
public CompletableFuture<Void> searchText(String searchQuery) {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 List<Double> queryPosition = List.of(longitude, latitude);

 SearchTextRequest request = SearchTextRequest.builder()
 .queryText(searchQuery)
 .biasPosition(queryPosition)
 .build();

 return getGeoPlacesClient().searchText(request)
 .thenCompose(response -> {
 if (response.resultItems().isEmpty()) {
 logger.info("No places found.");
 return CompletableFuture.completedFuture(null);
 }

 // Get the first place ID
 String placeId = response.resultItems().get(0).placeId();

```



```

 logger.info("Found Place with id: " + placeId);

 // Fetch detailed info using getPlace
 GetPlaceRequest getPlaceRequest = GetPlaceRequest.builder()
 .placeId(placeId)
 .build();

 return getGeoPlacesClient().getPlace(getPlaceRequest)
 .thenAccept(placeResponse -> {
 logger.info("Detailed Place Information:");
 logger.info("Name: " +
placeResponse.placeType().name());
 logger.info("Address: " +
placeResponse.address().label());

 if (placeResponse.foodTypes() != null && !
placeResponse.foodTypes().isEmpty()) {
 logger.info("Food Types:");
 placeResponse.foodTypes().forEach(foodType -> {
 logger.info(" - " + foodType);
 });
 } else {
 logger.info("No food types available.");
 }
 logger.info("-----");
 });
 })
 .exceptionally(exception -> {
 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 throw new CompletionException("A validation error occurred:
" + cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing place search",
exception);
 });
}

/**
 * Performs reverse geocoding using the AWS Geo Places API.

```

```

 * Reverse geocoding is the process of converting geographic coordinates
 (latitude and longitude) to a human-readable address.
 * This method uses the latitude and longitude of San Francisco as the input,
 and prints the resulting address.
 */
 public CompletableFuture<ReverseGeocodeResponse> reverseGeocode() {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 logger.info("Use latitude 37.7749 and longitude -122.4194");

 // AWS expects [longitude, latitude].
 List<Double> queryPosition = List.of(longitude, latitude);
 ReverseGeocodeRequest request = ReverseGeocodeRequest.builder()
 .queryPosition(queryPosition)
 .build();
 CompletableFuture<ReverseGeocodeResponse> futureResponse =
 getGeoPlacesClient().reverseGeocode(request);

 return futureResponse.whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 throw new CompletionException("A validation error occurred: " +
cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing reverse geocoding",
exception);
 }

 response.resultItems().forEach(result ->
 logger.info("The address is: " + result.address().label())
);
 });
 }

 /**
 * Calculates the distance between two locations asynchronously.
 *
 * @param routeCalcName the name of the route calculator to use
 * @return a {@link CompletableFuture} that will complete with a {@link
CalculateRouteResponse} containing the distance and estimated duration of the route
 */

```

```

 public CompletableFuture<CalculateRouteResponse> calcDistanceAsync(String
routeCalcName) {
 // Define coordinates for Seattle, WA and Vancouver, BC.
 List<Double> departurePosition = Arrays.asList(-122.3321, 47.6062);
 List<Double> arrivePosition = Arrays.asList(-123.1216, 49.2827);

 CalculateRouteRequest request = CalculateRouteRequest.builder()
 .calculatorName(routeCalcName)
 .departurePosition(departurePosition)
 .destinationPosition(arrivePosition)
 .travelMode("Car") // Options: Car, Truck, Walking, Bicycle
 .distanceUnit("Kilometers") // Options: Meters, Kilometers, Miles
 .build();

 return getClient().calculateRoute(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The AWS resource was not
found: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Failed to calculate route: " +
exception.getMessage(), exception);
 }
 });
 }

 /**
 * Creates a new route calculator with the specified name and data source.
 *
 * @param routeCalcName the name of the route calculator to be created
 */
 public CompletableFuture<CreateRouteCalculatorResponse>
createRouteCalculator(String routeCalcName) {
 String dataSource = "Esri"; // or "Here"
 CreateRouteCalculatorRequest request =
CreateRouteCalculatorRequest.builder()
 .calculatorName(routeCalcName)
 .dataSource(dataSource)
 .build();

 return getClient().createRouteCalculator(request)

```

```

 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflict error occurred: "
+ cause.getMessage(), cause);
 }
 throw new CompletionException("Failed to create route
calculator: " + exception.getMessage(), exception);
 }
 });
 }

 /**
 * Retrieves the position of a device using the provided LocationClient.
 *
 * @param trackerName The name of the tracker associated with the device.
 * @param deviceId The ID of the device to retrieve the position for.
 * @throws RuntimeException If there is an error fetching the device position.
 */
 public CompletableFuture<GetDevicePositionResponse> getDevicePosition(String
trackerName, String deviceId) {
 GetDevicePositionRequest request = GetDevicePositionRequest.builder()
 .trackerName(trackerName)
 .deviceId(deviceId)
 .build();

 return getClient().getDevicePosition(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The AWS resource was not
found: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error fetching device position: "
+ exception.getMessage(), exception);
 }
 });
 }

 /**

```

```

 * Updates the position of a device in the location tracking system.
 *
 * @param trackerName the name of the tracker associated with the device
 * @param deviceId the unique identifier of the device
 * @throws RuntimeException if an error occurs while updating the device
 position
 */
 public CompletableFuture<BatchUpdateDevicePositionResponse>
 updateDevicePosition(String trackerName, String deviceId) {
 double latitude = 37.7749; // Example: San Francisco
 double longitude = -122.4194;

 DevicePositionUpdate positionUpdate = DevicePositionUpdate.builder()
 .deviceId(deviceId)
 .sampleTime(Instant.now()) // Timestamp of position update.
 .position(Arrays.asList(longitude, latitude)) // AWS requires
[longitude, latitude]
 .build();

 BatchUpdateDevicePositionRequest request =
 BatchUpdateDevicePositionRequest.builder()
 .trackerName(trackerName)
 .updates(positionUpdate)
 .build();

 CompletableFuture<BatchUpdateDevicePositionResponse> futureResponse =
 getClient().batchUpdateDevicePosition(request);
 return futureResponse.whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource was not found: " +
cause.getMessage(), cause);
 } else {
 throw new CompletionException("Error updating device position: "
+ exception.getMessage(), exception);
 }
 }
 });
 }

 /**

```

```

 * Creates a new tracker resource in your AWS account, which you can use to
 track the location of devices.
 *
 * @param trackerName the name of the tracker to be created
 * @return a {@link CompletableFuture} that, when completed, will contain the
 Amazon Resource Name (ARN) of the created tracker
 */
 public CompletableFuture<String> createTracker(String trackerName) {
 CreateTrackerRequest trackerRequest = CreateTrackerRequest.builder()
 .description("Created using the Java V2 SDK")
 .trackerName(trackerName)
 .positionFiltering("TimeBased") // Options: TimeBased, DistanceBased,
AccuracyBased
 .build();

 return getClient().createTracker(trackerRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("Conflict occurred while
creating tracker: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error creating tracker: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(CreateTrackerResponse::trackerArn); // Return only the
tracker ARN
 }

 /**
 * Adds a new geofence to the specified collection.
 *
 * @param collectionName the name of the geofence collection to add the geofence
 to
 * @param geoId the unique identifier for the geofence
 */
 public CompletableFuture<PutGeofenceResponse> putGeofence(String collectionName,
String geoId) {
 // Define the geofence geometry (polygon).
 GeofenceGeometry geofenceGeometry = GeofenceGeometry.builder()

```

```

 .polygon(List.of(
 List.of(
 List.of(-122.3381, 47.6101), // First point
 List.of(-122.3281, 47.6101),
 List.of(-122.3281, 47.6201),
 List.of(-122.3381, 47.6201),
 List.of(-122.3381, 47.6101) // Closing the polygon
)
))
 .build();

 PutGeofenceRequest geofenceRequest = PutGeofenceRequest.builder()
 .collectionName(collectionName) // Specify the collection.
 .geofenceId(geoId) // Unique ID for the geofence.
 .geometry(geofenceGeometry)
 .build();

 return getClient().putGeofence(geofenceRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException("Validation error while
creating geofence: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error creating geofence: " +
exception.getMessage(), exception);
 }
 });
 }

 /**
 * Creates a new geofence collection.
 *
 * @param collectionName the name of the geofence collection to be created
 */
 public CompletableFuture<String> createGeofenceCollection(String collectionName)
 {
 CreateGeofenceCollectionRequest collectionRequest =
CreateGeofenceCollectionRequest.builder()
 .collectionName(collectionName)
 .description("Created by using the AWS SDK for Java")
 .build();
 }

```

```

 return getClient().createGeofenceCollection(collectionRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("The geofence collection was
not created due to ConflictException.", cause);
 }
 throw new CompletionException("Failed to create geofence
collection: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response.collectionArn()); // Return only the ARN
 }

 /**
 * Creates a new API key with the specified name and restrictions.
 *
 * @param keyName the name of the API key to be created
 * @param mapArn the Amazon Resource Name (ARN) of the map resource to which
the API key will be associated
 * @return a {@link CompletableFuture} that completes with the Amazon Resource
Name (ARN) of the created API key,
 * or {@code null} if the operation failed
 */
 public CompletableFuture<String> createKey(String keyName, String mapArn) {
 ApiKeyRestrictions keyRestrictions = ApiKeyRestrictions.builder()
 .allowActions("geo:GetMap*")
 .allowResources(mapArn)
 .build();

 CreateKeyRequest request = CreateKeyRequest.builder()
 .keyName(keyName)
 .restrictions(keyRestrictions)
 .noExpiry(true)
 .build();

 return getClient().createKey(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```



```

 Throwable cause = exception.getCause();
 if (cause instanceof AccessDeniedException) {
 throw new CompletionException("The request was denied
because of insufficient access or permissions.", cause);
 }
 throw new CompletionException("Failed to create API key: " +
exception.getMessage(), exception);
 }
})
 .thenApply(response -> response.keyArn()); // This will never return
null if the response reaches here
}

/**
 * Creates a new map with the specified name and configuration.
 *
 * @param mapName the name of the map to be created
 * @return a {@link CompletableFuture} that, when completed, will contain the
Amazon Resource Name (ARN) of the created map
 * @throws CompletionException if an error occurs while creating the map, such
as exceeding the service quota
 */
public CompletableFuture<String> createMap(String mapName) {
 MapConfiguration configuration = MapConfiguration.builder()
 .style("VectorEsriNavigation")
 .build();

 CreateMapRequest mapRequest = CreateMapRequest.builder()
 .mapName(mapName)
 .configuration(configuration)
 .description("A map created using the Java V2 API")
 .build();

 return getClient().createMap(mapRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 }
 })
}

```

```

 throw new CompletionException("Failed to create map: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response.mapArn()); // Return the map ARN
}

/**
 * Deletes a geofence collection asynchronously.
 *
 * @param collectionName the name of the geofence collection to be deleted
 * @return a {@link CompletableFuture} that completes when the geofence
collection has been deleted
 */
public CompletableFuture<Void> deleteGeofenceCollectionAsync(String
collectionName) {
 DeleteGeofenceCollectionRequest collectionRequest =
DeleteGeofenceCollectionRequest.builder()
 .collectionName(collectionName)
 .build();

 return getClient().deleteGeofenceCollection(collectionRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested geofence
collection was not found.", cause);
 }
 throw new CompletionException("Failed to delete geofence
collection: " + exception.getMessage(), exception);
 }
 logger.info("The geofence collection {} was deleted.",
collectionName);
 })
 .thenApply(response -> null);
}

/**
 * Deletes the specified key from the key-value store.

```

```

 *
 * @param keyName the name of the key to be deleted
 * @return a {@link CompletableFuture} that completes when the key has been
deleted
 * @throws CompletionException if the key was not found or if an error occurred
during the deletion process
 */
 public CompletableFuture<Void> deleteKey(String keyName) {
 DeleteKeyRequest keyRequest = DeleteKeyRequest.builder()
 .keyName(keyName)
 .build();

 return getClient().deleteKey(keyRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The key was not found.",
cause);
 }
 throw new CompletionException("Failed to delete key: " +
exception.getMessage(), exception);
 }
 logger.info("The key {} was deleted.", keyName);
 })
 .thenApply(response -> null);
 }

 /**
 * Deletes a map with the specified name.
 *
 * @param mapName the name of the map to be deleted
 * @return a {@link CompletableFuture} that completes when the map deletion is
successful, or throws a {@link CompletionException} if an error occurs
 */
 public CompletableFuture<Void> deleteMap(String mapName) {
 DeleteMapRequest mapRequest = DeleteMapRequest.builder()
 .mapName(mapName)
 .build();

 return getClient().deleteMap(mapRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```

```

 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The map was not found.",
cause);
 }
 throw new CompletionException("Failed to delete map: " +
exception.getMessage(), exception);
 }
 logger.info("The map {} was deleted.", mapName);
})
.thenApply(response -> null);
}

/**
 * Deletes a tracker with the specified name.
 *
 * @param trackerName the name of the tracker to be deleted
 * @return a {@link CompletableFuture} that completes when the tracker has been
deleted
 * @throws CompletionException if an error occurs while deleting the tracker
 * - if the tracker was not found, a {@link
ResourceNotFoundException} is thrown wrapped in the CompletionException
 * - if any other error occurs, a generic
CompletionException is thrown with the error message
 */
public CompletableFuture<Void> deleteTracker(String trackerName) {
 DeleteTrackerRequest trackerRequest = DeleteTrackerRequest.builder()
 .trackerName(trackerName)
 .build();

 return getClient().deleteTracker(trackerRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The tracker was not found.",
cause);
 }
 throw new CompletionException("Failed to delete the tracker: " +
exception.getMessage(), exception);
 }
 logger.info("The tracker {} was deleted.", trackerName);
 })
}

```

```

 .thenApply(response -> null); // Ensures CompletableFuture<Void>
 }

 /**
 * Deletes a route calculator from the system.
 *
 * @param calcName the name of the route calculator to delete
 * @return a {@link CompletableFuture} that completes when the route calculator
 * has been deleted
 * @throws CompletionException if an error occurs while deleting the route
 * calculator
 * - If the route calculator was not found, a {@link
 * ResourceNotFoundException} will be thrown
 * - If any other error occurs, a generic {@link
 * CompletionException} will be thrown
 */
 public CompletableFuture<Void> deleteRouteCalculator(String calcName) {
 DeleteRouteCalculatorRequest calculatorRequest =
 DeleteRouteCalculatorRequest.builder()
 .calculatorName(calcName)
 .build();

 return getClient().deleteRouteCalculator(calculatorRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The route calculator was not
 found.", cause);
 }
 throw new CompletionException("Failed to delete the route
 calculator: " + exception.getMessage(), exception);
 }
 logger.info("The route calculator {} was deleted.", calcName);
 })
 .thenApply(response -> null);
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [BatchUpdateDevicePosition](#)

- [CalculateRoute](#)
- [CreateGeofenceCollection](#)
- [CreateKey](#)
- [CreateMap](#)
- [CreateRouteCalculator](#)
- [CreateTracker](#)
- [DeleteGeofenceCollection](#)
- [DeleteKey](#)
- [DeleteMap](#)
- [DeleteRouteCalculator](#)
- [DeleteTracker](#)
- [GetDevicePosition](#)
- [PutGeofence](#)

## Actions

### BatchUpdateDevicePosition

The following code example shows how to use BatchUpdateDevicePosition.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Updates the position of a device in the location tracking system.
 *
 * @param trackerName the name of the tracker associated with the device
 * @param deviceId the unique identifier of the device
 * @throws RuntimeException if an error occurs while updating the device
 * position
```

```

 */
 public CompletableFuture<BatchUpdateDevicePositionResponse>
updateDevicePosition(String trackerName, String deviceId) {
 double latitude = 37.7749; // Example: San Francisco
 double longitude = -122.4194;

 DevicePositionUpdate positionUpdate = DevicePositionUpdate.builder()
 .deviceId(deviceId)
 .sampleTime(Instant.now()) // Timestamp of position update.
 .position(Arrays.asList(longitude, latitude)) // AWS requires
[longitude, latitude]
 .build();

 BatchUpdateDevicePositionRequest request =
BatchUpdateDevicePositionRequest.builder()
 .trackerName(trackerName)
 .updates(positionUpdate)
 .build();

 CompletableFuture<BatchUpdateDevicePositionResponse> futureResponse =
getClient().batchUpdateDevicePosition(request);
 return futureResponse.whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The resource was not found: " +
cause.getMessage(), cause);
 } else {
 throw new CompletionException("Error updating device position: "
+ exception.getMessage(), exception);
 }
 }
 });
}

```

- For API details, see [BatchUpdateDevicePosition](#) in *AWS SDK for Java 2.x API Reference*.

## CalculateRoute

The following code example shows how to use CalculateRoute.

## SDK for Java 2.x

**Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Calculates the distance between two locations asynchronously.
 *
 * @param routeCalcName the name of the route calculator to use
 * @return a {@link CompletableFuture} that will complete with a {@link
 * CalculateRouteResponse} containing the distance and estimated duration of the route
 */
public CompletableFuture<CalculateRouteResponse> calcDistanceAsync(String
routeCalcName) {
 // Define coordinates for Seattle, WA and Vancouver, BC.
 List<Double> departurePosition = Arrays.asList(-122.3321, 47.6062);
 List<Double> arrivePosition = Arrays.asList(-123.1216, 49.2827);

 CalculateRouteRequest request = CalculateRouteRequest.builder()
 .calculatorName(routeCalcName)
 .departurePosition(departurePosition)
 .destinationPosition(arrivePosition)
 .travelMode("Car") // Options: Car, Truck, Walking, Bicycle
 .distanceUnit("Kilometers") // Options: Meters, Kilometers, Miles
 .build();

 return getClient().calculateRoute(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The AWS resource was not
found: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Failed to calculate route: " +
exception.getMessage(), exception);
 }
 });
}
```



```
}
```

- For API details, see [CalculateRoute](#) in *AWS SDK for Java 2.x API Reference*.

## CreateGeofenceCollection

The following code example shows how to use `CreateGeofenceCollection`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new geofence collection.
 *
 * @param collectionName the name of the geofence collection to be created
 */
public CompletableFuture<String> createGeofenceCollection(String collectionName)
{
 CreateGeofenceCollectionRequest collectionRequest =
CreateGeofenceCollectionRequest.builder()
 .collectionName(collectionName)
 .description("Created by using the AWS SDK for Java")
 .build();

 return getClient().createGeofenceCollection(collectionRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("The geofence collection was
not created due to ConflictException.", cause);
 }
 throw new CompletionException("Failed to create geofence
collection: " + exception.getMessage(), exception);
 }
 });
}
```

```

 }
 })
 .thenApply(response -> response.collectionArn()); // Return only the ARN
}

```

- For API details, see [CreateGeofenceCollection](#) in *AWS SDK for Java 2.x API Reference*.

## CreateKey

The following code example shows how to use CreateKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new API key with the specified name and restrictions.
 *
 * @param keyName the name of the API key to be created
 * @param mapArn the Amazon Resource Name (ARN) of the map resource to which
the API key will be associated
 * @return a {@link CompletableFuture} that completes with the Amazon Resource
Name (ARN) of the created API key,
 * or {@code null} if the operation failed
 */
public CompletableFuture<String> createKey(String keyName, String mapArn) {
 ApiKeyRestrictions keyRestrictions = ApiKeyRestrictions.builder()
 .allowActions("geo:GetMap*")
 .allowResources(mapArn)
 .build();

 CreateKeyRequest request = CreateKeyRequest.builder()
 .keyName(keyName)

```

```

 .restrictions(keyRestrictions)
 .noExpiry(true)
 .build();

return getClient().createKey(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof AccessDeniedException) {
 throw new CompletionException("The request was denied
because of insufficient access or permissions.", cause);
 }
 throw new CompletionException("Failed to create API key: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response.keyArn()); // This will never return
null if the response reaches here
}

```

- For API details, see [CreateKey](#) in *AWS SDK for Java 2.x API Reference*.

## CreateMap

The following code example shows how to use CreateMap.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new map with the specified name and configuration.
 *
 * @param mapName the name of the map to be created

```

```

 * @return a {@link CompletableFuture} that, when completed, will contain the
 Amazon Resource Name (ARN) of the created map
 * @throws CompletionException if an error occurs while creating the map, such
 as exceeding the service quota
 */
 public CompletableFuture<String> createMap(String mapName) {
 MapConfiguration configuration = MapConfiguration.builder()
 .style("VectorEsriNavigation")
 .build();

 CreateMapRequest mapRequest = CreateMapRequest.builder()
 .mapName(mapName)
 .configuration(configuration)
 .description("A map created using the Java V2 API")
 .build();

 return getClient().createMap(mapRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create map: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(response -> response.mapArn()); // Return the map ARN
 }

```

- For API details, see [CreateMap](#) in *AWS SDK for Java 2.x API Reference*.

## CreateRouteCalculator

The following code example shows how to use CreateRouteCalculator.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new route calculator with the specified name and data source.
 *
 * @param routeCalcName the name of the route calculator to be created
 */
public CompletableFuture<CreateRouteCalculatorResponse>
createRouteCalculator(String routeCalcName) {
 String dataSource = "Esri"; // or "Here"
 CreateRouteCalculatorRequest request =
CreateRouteCalculatorRequest.builder()
 .calculatorName(routeCalcName)
 .dataSource(dataSource)
 .build();

 return getClient().createRouteCalculator(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("A conflict error occurred: "
+ cause.getMessage(), cause);
 }
 throw new CompletionException("Failed to create route
calculator: " + exception.getMessage(), exception);
 }
 });
}
```

- For API details, see [CreateRouteCalculator](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTracker

The following code example shows how to use CreateTracker.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new tracker resource in your AWS account, which you can use to
 * track the location of devices.
 *
 * @param trackerName the name of the tracker to be created
 * @return a {@link CompletableFuture} that, when completed, will contain the
 * Amazon Resource Name (ARN) of the created tracker
 */
public CompletableFuture<String> createTracker(String trackerName) {
 CreateTrackerRequest trackerRequest = CreateTrackerRequest.builder()
 .description("Created using the Java V2 SDK")
 .trackerName(trackerName)
 .positionFiltering("TimeBased") // Options: TimeBased, DistanceBased,
AccuracyBased
 .build();

 return getClient().createTracker(trackerRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ConflictException) {
 throw new CompletionException("Conflict occurred while
creating tracker: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error creating tracker: " +
exception.getMessage(), exception);
 }
 })
 .thenApply(CreateTrackerResponse::trackerArn); // Return only the
tracker ARN
}
```

```
}
```

- For API details, see [CreateTracker](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteGeofenceCollection

The following code example shows how to use DeleteGeofenceCollection.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a geofence collection asynchronously.
 *
 * @param collectionName the name of the geofence collection to be deleted
 * @return a {@link CompletableFuture} that completes when the geofence
collection has been deleted
 */
public CompletableFuture<Void> deleteGeofenceCollectionAsync(String
collectionName) {
 DeleteGeofenceCollectionRequest collectionRequest =
DeleteGeofenceCollectionRequest.builder()
 .collectionName(collectionName)
 .build();

 return getClient().deleteGeofenceCollection(collectionRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The requested geofence
collection was not found.", cause);
 }
 }
 })
}
```

```

 throw new CompletionException("Failed to delete geofence
collection: " + exception.getMessage(), exception);
 }
 logger.info("The geofence collection {} was deleted.",
collectionName);
 })
 .thenApply(response -> null);
}

```

- For API details, see [DeleteGeofenceCollection](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteKey

The following code example shows how to use DeleteKey.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes the specified key from the key-value store.
 *
 * @param keyName the name of the key to be deleted
 * @return a {@link CompletableFuture} that completes when the key has been
deleted
 * @throws CompletionException if the key was not found or if an error occurred
during the deletion process
 */
public CompletableFuture<Void> deleteKey(String keyName) {
 DeleteKeyRequest keyRequest = DeleteKeyRequest.builder()
 .keyName(keyName)
 .build();

 return getClient().deleteKey(keyRequest)
 .whenComplete((response, exception) -> {

```



```

 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The key was not found.",
cause);
 }
 throw new CompletionException("Failed to delete key: " +
exception.getMessage(), exception);
 }
 logger.info("The key {} was deleted.", keyName);
 })
 .thenApply(response -> null);
}

```

- For API details, see [DeleteKey](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteMap

The following code example shows how to use DeleteMap.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a map with the specified name.
 *
 * @param mapName the name of the map to be deleted
 * @return a {@link CompletableFuture} that completes when the map deletion is
successful, or throws a {@link CompletionException} if an error occurs
 */
public CompletableFuture<Void> deleteMap(String mapName) {
 DeleteMapRequest mapRequest = DeleteMapRequest.builder()
 .mapName(mapName)
 .build();
}

```

```

 return getClient().deleteMap(mapRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The map was not found.",
cause);
 }
 throw new CompletionException("Failed to delete map: " +
exception.getMessage(), exception);
 }
 logger.info("The map {} was deleted.", mapName);
 })
 .thenApply(response -> null);
 }

```

- For API details, see [DeleteMap](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteRouteCalculator

The following code example shows how to use DeleteRouteCalculator.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a route calculator from the system.
 *
 * @param calcName the name of the route calculator to delete
 * @return a {@link CompletableFuture} that completes when the route calculator
has been deleted
 * @throws CompletionException if an error occurs while deleting the route
calculator
 *
 * - If the route calculator was not found, a {@link
ResourceNotFoundException} will be thrown

```

```

 *
 * - If any other error occurs, a generic {@link
CompletionException} will be thrown
 */
 public CompletableFuture<Void> deleteRouteCalculator(String calcName) {
 DeleteRouteCalculatorRequest calculatorRequest =
DeleteRouteCalculatorRequest.builder()
 .calculatorName(calcName)
 .build();

 return getClient().deleteRouteCalculator(calculatorRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The route calculator was not
found.", cause);
 }
 throw new CompletionException("Failed to delete the route
calculator: " + exception.getMessage(), exception);
 }
 logger.info("The route calculator {} was deleted.", calcName);
 })
 .thenApply(response -> null);
 }
 }

```

- For API details, see [DeleteRouteCalculator](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteTracker

The following code example shows how to use DeleteTracker.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a tracker with the specified name.
 *
 * @param trackerName the name of the tracker to be deleted
 * @return a {@link CompletableFuture} that completes when the tracker has been
deleted
 * @throws CompletionException if an error occurs while deleting the tracker
 *
 * - if the tracker was not found, a {@link
ResourceNotFoundException} is thrown wrapped in the CompletionException
 *
 * - if any other error occurs, a generic
CompletionException is thrown with the error message
 */
public CompletableFuture<Void> deleteTracker(String trackerName) {
 DeleteTrackerRequest trackerRequest = DeleteTrackerRequest.builder()
 .trackerName(trackerName)
 .build();

 return getClient().deleteTracker(trackerRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The tracker was not found.",
cause);
 }
 throw new CompletionException("Failed to delete the tracker: " +
exception.getMessage(), exception);
 }
 logger.info("The tracker {} was deleted.", trackerName);
 })
 .thenApply(response -> null); // Ensures CompletableFuture<Void>
}

```

- For API details, see [DeleteTracker](#) in *AWS SDK for Java 2.x API Reference*.

## GetDevicePosition

The following code example shows how to use `GetDevicePosition`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Retrieves the position of a device using the provided LocationClient.
 *
 * @param trackerName The name of the tracker associated with the device.
 * @param deviceId The ID of the device to retrieve the position for.
 * @throws RuntimeException If there is an error fetching the device position.
 */
public CompletableFuture<GetDevicePositionResponse> getDevicePosition(String
trackerName, String deviceId) {
 GetDevicePositionRequest request = GetDevicePositionRequest.builder()
 .trackerName(trackerName)
 .deviceId(deviceId)
 .build();

 return getClient().getDevicePosition(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ResourceNotFoundException) {
 throw new CompletionException("The AWS resource was not
found: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error fetching device position: "
+ exception.getMessage(), exception);
 }
 });
}
```

- For API details, see [GetDevicePosition](#) in *AWS SDK for Java 2.x API Reference*.

## PutGeofence

The following code example shows how to use PutGeofence.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Adds a new geofence to the specified collection.
 *
 * @param collectionName the name of the geofence collection to add the geofence
 * to
 * @param geoId the unique identifier for the geofence
 */
public CompletableFuture<PutGeofenceResponse> putGeofence(String collectionName,
String geoId) {
 // Define the geofence geometry (polygon).
 GeofenceGeometry geofenceGeometry = GeofenceGeometry.builder()
 .polygon(List.of(
 List.of(
 List.of(-122.3381, 47.6101), // First point
 List.of(-122.3281, 47.6101),
 List.of(-122.3281, 47.6201),
 List.of(-122.3381, 47.6201),
 List.of(-122.3381, 47.6101) // Closing the polygon
)
))
 .build();

 PutGeofenceRequest geofenceRequest = PutGeofenceRequest.builder()
 .collectionName(collectionName) // Specify the collection.
 .geofenceId(geoId) // Unique ID for the geofence.
 .geometry(geofenceGeometry)
 .build();

 return getClient().putGeofence(geofenceRequest)
 .whenComplete((response, exception) -> {
```

```
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ValidationException) {
 throw new CompletionException("Validation error while
creating geofence: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Error creating geofence: " +
exception.getMessage(), exception);
 }
 });
}
```

- For API details, see [PutGeofence](#) in *AWS SDK for Java 2.x API Reference*.

## Location Service Places examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Location Service Places.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### ReverseGeocode

The following code example shows how to use ReverseGeocode.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Performs reverse geocoding using the AWS Geo Places API.
 * Reverse geocoding is the process of converting geographic coordinates
 (latitude and longitude) to a human-readable address.
 * This method uses the latitude and longitude of San Francisco as the input,
 and prints the resulting address.
 */
public CompletableFuture<ReverseGeocodeResponse> reverseGeocode() {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 logger.info("Use latitude 37.7749 and longitude -122.4194");

 // AWS expects [longitude, latitude].
 List<Double> queryPosition = List.of(longitude, latitude);
 ReverseGeocodeRequest request = ReverseGeocodeRequest.builder()
 .queryPosition(queryPosition)
 .build();
 CompletableFuture<ReverseGeocodeResponse> futureResponse =
 getGeoPlacesClient().reverseGeocode(request);

 return futureResponse.whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 throw new CompletionException("A validation error occurred: " +
cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing reverse geocoding",
exception);
 }

 response.resultItems().forEach(result ->
```



```

 logger.info("The address is: " + result.address().label())
);
});
}

```

- For API details, see [ReverseGeocode](#) in *AWS SDK for Java 2.x API Reference*.

## SearchNearby

The following code example shows how to use SearchNearby.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Performs a nearby places search based on the provided geographic coordinates
 * (latitude and longitude).
 * The method sends an asynchronous request to search for places within a 1-
 * kilometer radius of the specified location.
 * The results are processed and printed once the search completes successfully.
 */
public CompletableFuture<SearchNearbyResponse> searchNearby() {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 List<Double> queryPosition = List.of(longitude, latitude);

 // Set up the request for searching nearby places.
 SearchNearbyRequest request = SearchNearbyRequest.builder()
 .queryPosition(queryPosition) // Set the position
 .queryRadius(1000L) // Radius in meters (1000 meters = 1 km).
 .build();

 return getGeoPlacesClient().searchNearby(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```

```

 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 throw new CompletionException("A validation error occurred:
" + cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing place search",
exception);
 }

 // Process the response and print the results.
 response.resultItems().forEach(result -> {
 logger.info("Place Name: " + result.placeType().name());
 logger.info("Address: " + result.address().label());
 logger.info("Distance: " + result.distance() + " meters");
 logger.info("-----");
 });
});
}

```

- For API details, see [SearchNearby](#) in *AWS SDK for Java 2.x API Reference*.

## SearchText

The following code example shows how to use SearchText.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Searches for a place using the provided search query and prints the detailed
information of the first result.
 *
 * @param searchQuery the search query to be used for the place search (ex,
coffee shop)

```

```

 */
 public CompletableFuture<Void> searchText(String searchQuery) {
 double latitude = 37.7749; // San Francisco
 double longitude = -122.4194;
 List<Double> queryPosition = List.of(longitude, latitude);

 SearchTextRequest request = SearchTextRequest.builder()
 .queryText(searchQuery)
 .biasPosition(queryPosition)
 .build();

 return getGeoPlacesClient().searchText(request)
 .thenCompose(response -> {
 if (response.resultItems().isEmpty()) {
 logger.info("No places found.");
 return CompletableFuture.completedFuture(null);
 }

 // Get the first place ID
 String placeId = response.resultItems().get(0).placeId();
 logger.info("Found Place with id: " + placeId);

 // Fetch detailed info using getPlace
 GetPlaceRequest getPlaceRequest = GetPlaceRequest.builder()
 .placeId(placeId)
 .build();

 return getGeoPlacesClient().getPlace(getPlaceRequest)
 .thenAccept(placeResponse -> {
 logger.info("Detailed Place Information:");
 logger.info("Name: " +
placeResponse.placeType().name());
 logger.info("Address: " +
placeResponse.address().label());

 if (placeResponse.foodTypes() != null && !
placeResponse.foodTypes().isEmpty()) {
 logger.info("Food Types:");
 placeResponse.foodTypes().forEach(foodType -> {
 logger.info(" - " + foodType);
 });
 } else {
 logger.info("No food types available.");
 }
 })
 })
 }

```

```
 logger.info("-----");
 });
 })
 .exceptionally(exception -> {
 Throwable cause = exception.getCause();
 if (cause instanceof
software.amazon.awssdk.services.geoplaces.model.ValidationException) {
 throw new CompletionException("A validation error occurred:
" + cause.getMessage(), cause);
 }
 throw new CompletionException("Error performing place search",
exception);
 });
 }
```

- For API details, see [SearchText](#) in *AWS SDK for Java 2.x API Reference*.

## AWS Marketplace Catalog API examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Marketplace Catalog API.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [AMI products](#)
- [Channel partner offers](#)
- [Container products](#)
- [Entities](#)
- [Offers](#)
- [Products](#)
- [Resale authorization](#)
- [SaaS products](#)
- [Utilities](#)

## AMI products

### Add a dimension to an existing AMI product and update the offer pricing terms

The following code example shows how to add a dimension to an existing AMI product and update the offer pricing terms.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Identifier": "prod-111111111111",
 "Type": "AmiProduct@1.0"
 },
 "DetailsDocument": [
 {
 "Key": "m7g.8xlarge",
 "Description": "m7g.8xlarge",
 "Name": "m7g.8xlarge",
 "Types": [
 "Metered"
],
 "Unit": "Hrs"
 }
]
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
```

```

 "Type": "Offer@1.0",
 "Identifier": "offer-111111111111"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "m5.large",
 "Price": "0.15"
 },
 {
 "DimensionKey": "m7g.4xlarge",
 "Price": "0.45"
 },
 {
 "DimensionKey": "m7g.2xlarge",
 "Price": "0.45"
 },
 {
 "DimensionKey": "m7g.8xlarge",
 "Price": "0.55"
 }
]
 }
]
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Add a region where an AMI product is deployed

The following code example shows how to add a region where an AMI product is deployed.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "AddRegions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": {
 "Regions": [
 "us-east-2",
 "us-west-2"
]
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited AMI product and a public offer with hourly annual pricing

The following code example shows how to create a public or limited AMI product and a public offer with hourly annual pricing. This example creates either a standard or custom EULA.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "AmiProduct@1.0"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Operating Systems"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
```



```

 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
},
{
 "ChangeType": "AddRegions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Regions": [
 "us-east-1"
]
 }
},
{
 "ChangeType": "AddInstanceTypes",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "InstanceTypes": [
 "t2.micro"
]
 }
},
{
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Version": {
 "VersionTitle": "Test AMI Version1.0",
 "ReleaseNotes": "Test AMI Version"
 },
 "DeliveryOptions": [
 {
 "Details": {

```

```

 "AmiDeliveryOptionDetails": {
 "AmiSource": {
 "AmiId": "ami-1111111111111111",
 "AccessRoleArn":
"arn:aws:iam::111111111111:role/AWSMarketplaceAmiIngestion",
 "UserName": "ec2-user",
 "OperatingSystemName": "AMAZONLINUX",
 "OperatingSystemVersion": "10.0.14393",
 "ScanningPort": 22
 },
 "UsageInstructions": "Test AMI Version",
 "RecommendedInstanceType": "t2.micro",
 "SecurityGroups": [
 {
 "IpProtocol": "tcp",
 "IpRanges": [
 "0.0.0.0/0"
],
 "FromPort": 10,
 "ToPort": 22
 }
]
 }
],
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "t2.micro",
 "Description": "t2.micro",
 "Name": "t2.micro",
 "Types": [
 "Metered"
],
 "Unit": "Hrs"
 }
]
 }
]

```

```

]
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public offer for AmiProduct using AWS Marketplace API
Reference Code",

```

```

 "Description": "Test public offer with hourly-annual pricing for
AmiProduct using AWS Marketplace API Reference Code"
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "0.15"
 }
]
 }
]
 },
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P365D"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",

```

```

 "QuantityConfiguration": "Allowed"
 }
}
]
}
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
},
{

```

```

 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited AMI product and public offer with hourly monthly pricing

The following code example shows how to create a public or limited AMI product and public offer with hourly monthly pricing. This example creates either a standard or custom EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "AmiProduct@1.0"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {

```

```

 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Operating Systems"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
},
{
 "ChangeType": "AddRegions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Regions": [
 "us-east-1"
]
 }
},
{
 "ChangeType": "AddInstanceTypes",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "InstanceTypes": [
 "t2.micro"
]
 }
}

```

```

]
 },
 {
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Version": {
 "VersionTitle": "Test AMI Version1.0",
 "ReleaseNotes": "Test AMI Version"
 },
 "DeliveryOptions": [
 {
 "Details": {
 "AmiDeliveryOptionDetails": {
 "AmiSource": {
 "AmiId": "ami-1111111111111111",
 "AccessRoleArn":
"arn:aws:iam::111111111111:role/AWSMarketplaceAmiIngestion",
 "UserName": "ec2-user",
 "OperatingSystemName": "AMAZONLINUX",
 "OperatingSystemVersion": "10.0.14393",
 "ScanningPort": 22
 },
 "UsageInstructions": "Test AMI Version",
 "RecommendedInstanceType": "t2.micro",
 "SecurityGroups": [
 {
 "IpProtocol": "tcp",
 "IpRanges": [
 "0.0.0.0/0"
],
 "FromPort": 10,
 "ToPort": 22
 }
]
 }
 }
 }
]
 }
 }
]

```



```

 },
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "t2.micro",
 "Description": "t2.micro",
 "Name": "t2.micro",
 "Types": [
 "Metered"
],
 "Unit": "Hrs"
 }
]
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "CreateOffer",

```

```

 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public offer for AmiProduct using AWS Marketplace API
Reference Code",
 "Description": "Test public offer with hourly-monthly pricing for
AmiProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "0.15"
 }
]
 }
]
 }
]
 }
 },

```

```

 {
 "Type": "RecurringPaymentTerm",
 "CurrencyCode": "USD",
 "BillingPeriod": "Monthly",
 "Price": "15.0"
 }
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
},

```

```

 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited AMI product and public offer with hourly pricing

The following code example shows how to create a public or limited AMI product and public offer with hourly pricing. This example creates either a standard or custom EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "AmiProduct@1.0"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",

```

```

 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Operating Systems"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
 },
 {
 "ChangeType": "AddRegions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Regions": [
 "us-east-1"
]
 }
 },
 {
 "ChangeType": "AddInstanceTypes",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "InstanceTypes": [

```

```

 "t2.micro"
]
}
},
{
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Version": {
 "VersionTitle": "Test AMI Version1.0",
 "ReleaseNotes": "Test AMI Version"
 },
 "DeliveryOptions": [
 {
 "Details": {
 "AmiDeliveryOptionDetails": {
 "AmiSource": {
 "AmiId": "ami-11111111111111111",
 "AccessRoleArn":
"arn:aws:iam::111111111111:role/AWSMarketplaceAmiIngestion",
 "UserName": "ec2-user",
 "OperatingSystemName": "AMAZONLINUX",
 "OperatingSystemVersion": "10.0.14393",
 "ScanningPort": 22
 },
 "UsageInstructions": "Test AMI Version",
 "RecommendedInstanceType": "t2.micro",
 "SecurityGroups": [
 {
 "IpProtocol": "tcp",
 "IpRanges": [
 "0.0.0.0/0"
],
 "FromPort": 10,
 "ToPort": 22
 }
]
 }
 }
 }
]
 }
}
]

```

```

 }
 },
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "t2.micro",
 "Description": "t2.micro",
 "Name": "t2.micro",
 "Types": [
 "Metered"
],
 "Unit": "Hrs"
 }
]
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {

```

```

 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public offer for AmiProduct using AWS Marketplace API
Reference Code",
 "Description": "Test public offer with hourly pricing for AmiProduct
using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "0.15"
 }
]
 }
]
 }
]
 }
 }
]

```



```

 }
]
}
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 }
}

```

```

 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create an draft AMI product with a draft public offer

The following code example shows how to create an draft AMI product with a draft public offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "AmiProduct@1.0"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product"
 }
 },
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 }
 }
]
}

```

```

 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier",
 "Name": "Test Offer"
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Restrict a region where an AMI product is deployed

The following code example shows how to restrict a region where an AMI product is deployed.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "RestrictRegions",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": {
 "Regions": [
 "us-west-2"
]
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Restrict product visibility

The following code example shows how to restrict product visibility.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateVisibility",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": {
 "TargetVisibility": "Restricted"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Specify whether AMI assets are deployed in new regions

The following code example shows how to specify whether AMI assets are deployed in new regions built by AWS to support future regions.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateFutureRegionSupport",
 "Entity": {
 "Type": "AmiProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": {
 "FutureRegionSupport": {
 "SupportedRegions": [
 "All"
]
 }
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Channel partner offers

### Create a draft CPPO for any product type

The following code example shows how to create a draft CPPO for any product type so you can review them internally before publishing to buyers.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOfferUsingResaleAuthorization",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ResaleAuthorizationId": "11111111-1111-1111-1111-111111111111",
 "Name": "Test Offer",
 "Description": "Test product"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

### Create a resale authorization replacement private offer with contract pricing

The following code example shows how to create a resale authorization replacement private offer from an existing agreement with contract pricing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateReplacementOfferUsingResaleAuthorization",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateReplacementOfferResaleAuth",
 "DetailsDocument": {
 "AgreementId": "agmt-11111111111111111111111111111111",
 "ResaleAuthorizationId": "resaleauthz-1111111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test replacement offer for SaaSProduct using AWS
Marketplace API Reference Codes",
 "Description": "Test private resale replacement offer with contract
pricing for SaaSProduct"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
```

```

 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "FixedUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "Price": "0.0",
 "Duration": "P12M",
 "Grants": [
 {
 "DimensionKey": "BasicService",
 "MaxQuantity": 2
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementEndDate": "2024-01-30"
 }
]
 }
},
{
 "ChangeType": "UpdatePaymentScheduleTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {

```



```

 "Type": "PaymentScheduleTerm",
 "CurrencyCode": "USD",
 "Schedule": [
 {
 "ChargeDate": "2024-01-01",
 "ChargeAmount": "0"
 }
]
 }
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
},
{
 "ChangeType": "ReleaseOffer",

```

```

 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOfferResaleAuth.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## List all CPPOs created by a channel partner

The following code example shows how to list all CPPOs created by a channel partner.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.EntitySummary;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;

```

```
import
software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;

public class ListAllCppoOffers {

 /*
 * List all CPPOs created by a channel partner
 */
 public static void main(String[] args) {

 List<String> cppoOfferIds = getAllCppoOfferIds();

 ReferenceCodesUtils.formatOutput(cppoOfferIds);
 }

 public static List<String> getAllCppoOfferIds() {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 // get all offer entity ids
 List<String> entityIdList = new ArrayList<String>();

 ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .nextToken(null)
 .build();

 ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 for (EntitySummary entitySummary : listEntitiesResponse.entitySummaryList()) {
 entityIdList.add(entitySummary.entityId());
 }

 while (listEntitiesResponse.nextToken() != null) {
 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
```

```

 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 for (EntitySummary entitySummary : listEntitiesResponse.entitySummaryList()) {
 entityIdList.add(entitySummary.entityId());
 }
}

// filter for CPP0 offers: ResaleAuthorizationId exists in Details

List<String> cppoOfferIds = new ArrayList<String>();

for (String entityId : entityIdList) {
 DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(entityId)
 .build();
 DescribeEntityResponse describeEntityResponse =
marketplaceCatalogClient.describeEntity(describeEntityRequest);

 Document resaleAuthorizationDocument =
describeEntityResponse.detailsDocument().asMap().get(ATTRIBUTE_RESALE_AUTHORIZATION_ID);
 String resaleAuthorizationId = resaleAuthorizationDocument != null ?
resaleAuthorizationDocument.asString() : "";

 if (!resaleAuthorizationId.isEmpty()) {
 cppoOfferIds.add(resaleAuthorizationId);
 }
}
return cppoOfferIds;
}
}

```

- For API details, see [ListEntities](#) in *AWS SDK for Java 2.x API Reference*.

## List all shared resale authorizations available to a channel partner

The following code example shows how to list all shared resale authorizations available to a channel partner.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;

public class ListAllSharedResaleAuthorizations {

 /*
 * list all resale authorizations shared to an account
 */
 public static void main(String[] args) {

 List<ListEntitiesResponse> responseList = getListEntityResponseList();
 ReferenceCodesUtils.formatOutput(responseList);
 }

 public static List<ListEntitiesResponse> getListEntityResponseList() {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
```

```
.httpClient(ApacheHttpClient.builder().build())
.credentialsProvider(ProfileCredentialsProvider.create())
.build();

List<ListEntitiesResponse> responseList = new ArrayList<ListEntitiesResponse>();

ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_RESALE_AUTHORIZATION)
 .maxResults(10)
 .ownershipType(OWNERSHIP_TYPE_SHARED)
 .nextToken(null)
 .build();

ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

responseList.add(listEntitiesResponse);

while (listEntitiesResponse.nextToken() != null) {
 listEntitiesRequest = ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_RESALE_AUTHORIZATION)
 .maxResults(10)
 .ownershipType(OWNERSHIP_TYPE_SHARED)
 .nextToken(listEntitiesResponse.nextToken())
 .build();

 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 responseList.add(listEntitiesResponse);
}
return responseList;
}
```

- For API details, see [ListEntities](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a CPPO and append a buyer EULA

The following code example shows how to publish a CPPO and append a buyer EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType" : "CreateOfferUsingResaleAuthorization",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateCPP0offer",
 "DetailsDocument": {
 "ResaleAuthorizationId": "resaleauthz-111111111111",
 "Name": "Test Offer",
 "Description": "Test product"
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
```

```

 "Url": "https://s3.amazonaws.com/sample-bucket/custom-eula.pdf"
 }
]
}
},
{
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": ["222222222222"]
 }
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-07-31"
 }
},
{
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P450D"
 }
]
 }
},

```



```

 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a CPPO using one-time resale authorization and update price markup

The following code example shows how to publish a CPPO using one-time resale authorization on AMI, SaaS, or Container products and update price markup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOfferUsingResaleAuthorization",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateCPP0offer",
 "DetailsDocument": {
 "ResaleAuthorizationId": "resaleauthz-111111111111",
 "Name": "Test Offer",
 "Description": "Test product"
 }
 }
]
}

```

```

 }
 },
 {
 "ChangeType": "UpdateMarkup",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Percentage" : "5.0"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": ["222222222222"]
 }
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-07-31"
 }
 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",

```

```

 "AgreementDuration": "P450D"
 }
}
],
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a draft CPPO and update price markup

The following code example shows how to publish a draft CPPO and update price markup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOfferUsingResaleAuthorization",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateCPP0offer",

```

```

 "DetailsDocument": {
 "ResaleAuthorizationId": "resaleauthz-111111111111",
 "Name": "Test Offer",
 "Description": "Test product"
 }
 },
 {
 "ChangeType": "UpdateMarkup",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Percentage" : "5.0"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": ["222222222222"]
 }
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-07-31"
 }
 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 }
 },

```

```

 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P450D"
 }
]
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateCPP0offer.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update the expiration date of a CPPO

The following code example shows how to update the expiration date of a CPPO to give buyers more time to evaluate and accept the offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {

```

```

 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-111111111111"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2025-07-31"
 }
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Container products

### Create a draft container product with a draft public offer

The following code example shows how to create a draft container product with a draft public offer.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "changeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "ContainerProduct@1.0"
 },

```

```

 "DetailsDocument": {
 "ProductTitle": "Sample product"
 },
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier",
 "Name": "Test Offer"
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a limited container product with a public offer and contract pricing

The following code example shows how to create a limited container product with a public offer, contract pricing, and standard EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "Entity": {

```

```

 "Type": "ContainerProduct@1.0"
 },
 "DetailsDocument": {},
 "ChangeName": "CreateProductChange"
},
{
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "ContainerProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "Categories": [
 "Streaming solutions"
],
 "ProductTitle": "ContainerProduct",
 "AdditionalResources": [],
 "LongDescription": "Long description goes here",
 "SearchKeywords": [
 "container streaming"
],
 "ShortDescription": "Description1",
 "Highlights": [
 "Highlight 1",
 "Highlight 2"
],
 "SupportDescription": "No support available",
 "VideoUrls": []
 }
},
{
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "ContainerProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "Cores",
 "Description": "Cores per cluster",
 "Name": "Cores",
 "Types": [
 "Entitled"
]
 }
]
}

```



```

],
 "Unit": "Units"
 }
]
},
{
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "ContainerProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111"
]
 }
 }
},
{
 "ChangeType": "AddRepositories",
 "Entity": {
 "Type": "ContainerProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Repositories": [
 {
 "RepositoryName": "uniquerepositoryname",
 "RepositoryType": "ECR"
 }
]
 }
},
{
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "ContainerProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "CreateOffer",

```

```

 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 },
 "ChangeName": "CreateOfferChange"
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "Constraints": {
 "MultipleDimensionSelection": "Disallowed",
 "QuantityConfiguration": "Disallowed"
 },
 "RateCard": [
 {
 "DimensionKey": "Cores",
 "Price": "0.25"
 }
]
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateLegalTerms",

```

```

 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "No refunds"
 }
]
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Some container offer Name",
 "Description": "Some interesting container offer description"
 }
 }
},

```

```

 {
 "ChangeType": "UpdateRenewalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "RenewalTerm"
 }
]
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Entities

### Describe all entities in a single call

The following code example shows how to describe all entities in a single call.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.catalogapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.BatchDescribeEntitiesRequest;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.BatchDescribeEntitiesResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityDetail;
import
 software.amazon.awssdk.services.marketplacecatalog.model.BatchDescribeErrorDetail;

import java.util.Arrays;
import java.util.Map;

public class BatchDescribeEntities {

 /*
 * BatchDescribe my entities in a single call and
 * check if it contains all the information I need to know about the entities.
 */
 public static void main(String[] args) {

 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 BatchDescribeEntitiesRequest batchDescribeEntitiesRequest =
 BatchDescribeEntitiesRequest.builder()
 .entityRequestList(Arrays.asList(
 EntityRequest.builder()
 .catalog(AWS_MP_CATALOG).entityId(OFFER_ID)
 .build(),
 EntityRequest.builder()
```

```

.catalog(AWS_MP_CATALOG).entityId(PRODUCT_ID)
 .build()))
 .build();

BatchDescribeEntitiesResponse batchDescribeEntitiesResponse =
marketplaceCatalogClient.batchDescribeEntities(batchDescribeEntitiesRequest);

// Reading the successful entities response
Map<String, EntityDetail> entityDetailsMap =
batchDescribeEntitiesResponse.entityDetails();
for (Map.Entry<String, EntityDetail> entry : entityDetailsMap.entrySet()) {
 System.out.println("EntityId: " + entry.getKey());
 ReferenceCodesUtils.formatOutput(entry.getValue());
}

// Logging the failed entities error details
Map<String, BatchDescribeErrorDetail> entityErrorsMap =
batchDescribeEntitiesResponse.errors();
for (Map.Entry<String, BatchDescribeErrorDetail> entry :
entityErrorsMap.entrySet()) {
 System.out.println(String.format("EntityId: %s, ErrorCode: %s,
ErrorMessage: %s", entry.getKey(),
 entry.getValue().errorCode(), entry.getValue().errorMessage()));
}
}
}

```

- For API details, see [BatchDescribeEntities](#) in *AWS SDK for Java 2.x API Reference*.

## List and describe all offers associated with a product

The following code example shows how to list and describe all offers associated with a product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.EntitySummary;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityTypeFilters;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferFilters;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferProductIdFilter;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferTargetingFilter;

public class ListProductPrivateOffers {

 private static MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();
 /*
 * retrieve all private offer information related to a single product
 */
 public static void main(String[] args) {

 List<EntitySummary> entitySummaryList = getEntitySummaryList();

 // for each offer id, output the offer detail using DescribeEntity API

 for (EntitySummary entitySummary : entitySummaryList) {

```

```

DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(entitySummary.entityId())
 .build();
DescribeEntityResponse describeEntityResponse =
marketplaceCatalogClient.describeEntity(describeEntityRequest);
 ReferenceCodesUtils.formatOutput(describeEntityResponse);
}
}
public static List<EntitySummary> getEntitySummaryList() {
 // define list entities filters

 EntityTypeFilters entityTypeFilters =
 EntityTypeFilters.builder()
 .offerFilters(OfferFilters.builder()
 .targeting(OfferTargetingFilter.builder()
 .valueListWithStrings(OFFER_TARGETING_BUYERACCOUNTS)
 .build())
 .productId(OfferProductIdFilter.builder()
 .valueList(PRODUCT_ID)
 .build())
 .build())
 .build();

 ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER).maxResults(50)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(null)
 .build();

 ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 // save all entitySummary of the results into entitySummaryList

 List<EntitySummary> entitySummaryList = new ArrayList<EntitySummary>();

 entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());

 while (listEntitiesResponse.nextToken() != null &&
listEntitiesResponse.nextToken().length() > 0) {

```



```
listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER).maxResults(50)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());
}
return entitySummaryList;
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DescribeEntity](#)
  - [ListEntities](#)

## Offers

### Create a custom dimension for a SaaS product and create a private offer

The following code example shows how to create a custom dimension for a SaaS product and create a private offer.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
```

```

"Catalog": "AWSMarketplace",
"ChangeSet": [
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": [
 {
 "Types": [
 "Entitled"
],
 "Description": "Custom Pricing 4 w/ terms and coverage to be
defined in Private Offer",
 "Unit": "Units",
 "Key": "Custom4",
 "Name": "Custom Pricing 4"
 }
]
 },
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 },
 "ChangeName": "CreateOfferChange"
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Private Test Offer - SaaS Contract Product",
 "Description": "Private Test Offer - SaaS Contract Product"
 }
 },
 {
 "ChangeType": "UpdateTargeting",

```

```

 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111"
]
 }
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",

```

```

 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 },
 "RateCard": [
 {
 "DimensionKey": "Custom4",
 "Price": "300.0"
 }
],
 "Selector": {
 "Type": "Duration",
 "Value": "P36M"
 }
 }
]
 }
]
 },
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
],
"ChangeSetName": "PrivateOfferWithCustomDimension"
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a draft private offer for an AMI or SaaS product

The following code example shows how to create a draft private offer for an AMI or SaaS product so you can review it internally before publishing to buyers.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "Test Private Offer"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with contract and Pay-As-You-Go pricing for a SaaS product

The following code example shows how to create a private offer with contract and Pay-As-You-Go pricing for a SaaS product.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for SaaSProduct using AWS Marketplace API Reference Code",
 "Description": "Test private offer with subscription pricing for SaaSProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 }
 }
]
}
```

```

 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "WorkloadSmall",
 "Price": "0.15"
 },
 {
 "DimensionKey": "WorkloadMedium",
 "Price": "0.25"
 }
]
 }
]
 },
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",

```

```

 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "150"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "300"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
},
{

```



```

 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with contract pricing and a flexible payment schedule for a SaaS product

The following code example shows how to create a private offer with contract pricing and a flexible payment schedule for a SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {

```

```

 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for SaaSProduct using AWS Marketplace
API Reference Code",
 "Description": "Test private offer with subscription pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {

```

```

 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "FixedUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "Price": "0.0",
 "Grants": [
 {
 "DimensionKey": "BasicService",
 "MaxQuantity": 1
 },
 {
 "DimensionKey": "PremiumService",
 "MaxQuantity": 1
 }
]
 }
]
 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P12M"
 }
]
 }
 },
 {
 "ChangeType": "UpdatePaymentScheduleTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {

```

```

 "Type": "PaymentScheduleTerm",
 "CurrencyCode": "USD",
 "Schedule": [
 {
 "ChargeDate": "2024-01-01",
 "ChargeAmount": "200.00"
 },
 {
 "ChargeDate": "2024-02-01",
 "ChargeAmount": "170.00"
 }
]
 }
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {

```

```

 "AvailabilityEndDate": "2023-12-31"
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with contract pricing for a Container product

The following code example shows how to create a private offer with contract pricing for a Container product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {

```

```

 "ProductId": "prod-111111111111"
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for Container product using AWS
Marketplace API Reference Code",
 "Description": "Test private offer for Container product with
contract pricing using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111"
]
 }
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [

```

```

 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "Constraints": {
 "MultipleDimensionSelection": "Disallowed",
 "QuantityConfiguration": "Disallowed"
 },
 "RateCard": [
 {
 "DimensionKey": "ReqPerHour",
 "Price": "0.25"
 }
]
 }
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",

```

```

 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with contract pricing for an AMI product

The following code example shows how to create a private offer with contract pricing for an AMI product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {

```



```

 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
},
{
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for AmiProduct using AWS Marketplace API
Reference Code",
 "Description": "Test private offer with hourly annual pricing for
AmiProduct using AWS Marketplace API Reference Code"
 }
},
{
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",

```

```

 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "ReadOnlyUsers",
 "Price": "220.00"
 }
]
 }
]
 }
]
 }
 }
]

```

```

],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
}
]
}
]
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with hourly annual pricing and a flexible payment schedule for an AMI product

The following code example shows how to create a private offer with hourly annual pricing and a flexible payment schedule for an AMI product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for AmiProduct using AWS Marketplace API Reference Code",
 "Description": "Test private offer with hourly annual pricing for AmiProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",

```

```

 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [

```

```

 {
 "DimensionKey": "t2.micro",
 "Price": "0.17"
 }
]
}
],
{
 "Type": "FixedUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "Price": "0.0",
 "Duration": "P365D",
 "Grants": [
 {
 "DimensionKey": "t2.micro",
 "MaxQuantity": 1
 }
]
}
]
}
},
{
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P650D"
 }
]
 }
},
{
 "ChangeType": "UpdatePaymentScheduleTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
}

```

```

 "DetailsDocument": {
 "Terms": [
 {
 "Type": "PaymentScheduleTerm",
 "CurrencyCode": "USD",
 "Schedule": [
 {
 "ChargeDate": "2024-01-01",
 "ChargeAmount": "200.00"
 },
 {
 "ChargeDate": "2024-02-01",
 "ChargeAmount": "170.00"
 }
]
 }
]
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with hourly annual pricing for an AMI product

The following code example shows how to create a private offer with hourly annual pricing for an AMI product.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for AmiProduct using AWS Marketplace API Reference Code",
 "Description": "Test private offer with hourly annual pricing for AmiProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 }
 }
]
}
```



```

 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {

```

```

 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "0.17"
 }
]
 }
]
 },
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P365D"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "220.00"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 }
}

```

```

 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P650D"
 }
]
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with hourly pricing for an AMI product

The following code example shows how to create a private offer with hourly pricing for an AMI product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for AmiProduct using AWS Marketplace API Reference Code",
 "Description": "Test private offer with hourly pricing for AmiProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 }
]
}
```

```

 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2025-01-01"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [

```

```

 {
 "RateCard": [
 {
 "DimensionKey": "t2.micro",
 "Price": "0.15"
 }
]
 }
],
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P30D"
 }
]
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with subscription pricing for a SaaS product

The following code example shows how to create a private offer with subscription pricing for a SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for SaaSProduct using AWS Marketplace
API Reference Code",
 "Description": "Test private offer with subscription pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
 }
],
}
```

```

{
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "WorkloadSmall",
 "Price": "0.13"
 },
 {
 "DimensionKey": "WorkloadMedium",
 "Price": "0.22"
 }
]
 }
]
 }
]
 }
}

```



```

 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementDuration": "P30D"
 }
]
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {

```

```

 "AvailabilityEndDate": "2023-12-31"
 },
],
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a private offer with tiered contract pricing for a SaaS product

The following code example shows how to create a private offer with tiered contract pricing for a SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {

```

```

 "ProductId": "prod-111111111111"
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test private offer for SaaSProduct using AWS Marketplace
API Reference Code",
 "Description": "Test private offer with subscription pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [

```

```

 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "120.00"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "200.00"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Disallowed",
 "QuantityConfiguration": "Disallowed"
 }
 }
],
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 }
]

```

```

 }
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public free trial offer with subscription pricing for a SaaS product

The following code example shows how to create a public free trial offer with subscription pricing for a SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
```

```

"Catalog": "AWSMarketplace",
"ChangeSet": [
 {
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "prod-111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public free trial offer for SaaSProduct using AWS
Marketplace API Reference Code",
 "Description": "Test public free trial offer with subscription
pricing for SaaSProduct using AWS Marketplace API Reference Code"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Free",
 "Terms": [
 {
 "Type": "FreeTrialPricingTerm",
 "Duration": "P20D",
 "Grants": [
 {
 "DimensionKey": "WorkloadSmall"
 },
 {
 "DimensionKey": "WorkloadMedium"
 }
]
 }
]
 }
 }
]

```

```

]
 }
}
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a replacement private offer with contract pricing

The following code example shows how to create a replacement private offer from an existing agreement with contract pricing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType" : "CreateReplacementOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateReplacementOffer",
 "DetailsDocument": {
 "AgreementId": "agmt-11111111111111111111111111111111"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test replacement offer for SaaSProduct using AWS
Marketplace API Reference Codes",
 "Description": "Test private replacement offer with contract pricing
for SaaSProduct"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 }
 }
]
}
```



```

 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "FixedUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "Price": "0.0",
 "Grants": [
 {
 "DimensionKey": "BasicService",
 "MaxQuantity": 2
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateValidityTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ValidityTerm",
 "AgreementEndDate": "2024-01-30"
 }
]
 }
 },
 {
 "ChangeType": "UpdatePaymentScheduleTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "PaymentScheduleTerm",
 "CurrencyCode": "USD",

```

```

 "Schedule": [
 {
 "ChargeDate": "2024-01-01",
 "ChargeAmount": "0"
 }
]
 }
],
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-12-31"
 }
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",

```

```

 "Identifier": "$CreateReplacementOffer.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Describe a public offer

The following code example shows how to describe a public offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.catalogapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;

public class DescribeEntity {

 /*
 * Describe my AMI or SaaS or Container product and check if it contains all the
 information I need to know about the product

```

```

 */
 public static void main(String[] args) {

 String offerId = args.length > 0 ? args[0] : OFFER_ID;

 DescribeEntityResponse describeEntityResponse =
 getDescribeEntityResponse(offerId);

 ReferenceCodesUtils.formatOutput(describeEntityResponse);
 }

 public static DescribeEntityResponse getDescribeEntityResponse(String offerId) {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(offerId)
 .build();

 DescribeEntityResponse describeEntityResponse =
 marketplaceCatalogClient.describeEntity(describeEntityRequest);
 return describeEntityResponse;
 }
}

```

- For API details, see [DescribeEntity](#) in *AWS SDK for Java 2.x API Reference*.

## Expire a draft private offer

The following code example shows how to set the expiration date of a private offer to a date in the past so that buyers no longer see the offer.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-01-01"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## List all private offers

The following code example shows how to list all private offers.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.EntitySummary;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityTypeFilters;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferAvailabilityEndDateFilter;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferAvailabilityEndDateFilterDateRange;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferBuyerAccountsFilter;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferFilters;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferReleaseDateFilter;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferReleaseDateFilterDateRange;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferTargetingFilter;

public class ListAllPrivateOffers {

 /*
 * List all my private offers and sort or filter them by Offer Publish Date, Offer
 * Expiry Date and Buyer IDs
 *
 * OfferTargetingFilter = BuyerAccounts (private offer);
 * OfferBuyerAccountsFilter: Buyer IDs filter
 * OfferAvailabilityEndDateFilter : Offer Expiry Date filter
 */

```

```

* OfferReleaseDateFilter : Offer Publish Date filter
*/

private static MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

public static void main(String[] args) {

 String offerReleaseDateAfterValue = "2023-01-01T23:59:59Z";
 String offerAvailableEndDateAfterValue = "2040-12-24T23:59:59Z";

 List<EntitySummary> entitySummaryList =
 getEntitySummaryList(offerReleaseDateAfterValue, offerAvailableEndDateAfterValue);

 // for each offer id, output the offer detail using DescribeEntity API

 for (EntitySummary entitySummary : entitySummaryList) {
 DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(entitySummary.entityId())
 .build();
 DescribeEntityResponse describeEntityResponse =
 marketplaceCatalogClient.describeEntity(describeEntityRequest);
 ReferenceCodesUtils.formatOutput(describeEntityResponse);
 }
}

public static List<EntitySummary> getEntitySummaryList (String
offerReleaseDateAfterValue, String offerAvailableEndDateAfterValue) {

 EntityTypeFilters entityTypeFilters =
 EntityTypeFilters.builder()
 .offerFilters(OfferFilters.builder()
 .targeting(OfferTargetingFilter.builder()
 .valueListWithStrings(OFFER_TARGETING_BUYERACCOUNTS)
 .build())
 .buyerAccounts(OfferBuyerAccountsFilter.builder()
 .wildCardValue(BUYER_ACCOUNT_ID)
 .build())
 .build()

```

```

 .availabilityEndDate(OfferAvailabilityEndDateFilter.builder()
 .dateRange(OfferAvailabilityEndDateFilterDateRange.builder()
 .afterValue(offerAvailableEndDateAfterValue).build())
 .build())
 .releaseDate(OfferReleaseDateFilter.builder()
 .dateRange(OfferReleaseDateFilterDateRange.builder()
 .afterValue(offerReleaseDateAfterValue)
 .build())
 .build())
 .build()
 }.build();

 ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER).maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(null)
 .build();

 ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
 List<EntitySummary> entitySummaryList = new ArrayList<EntitySummary>();

 entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());

 while (listEntitiesResponse.nextToken() != null &&
listEntitiesResponse.nextToken().length() > 0) {
 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
 entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());
 }

 return entitySummaryList;
}

```



```
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## List released public and private offers for a specific product ID

The following code example shows how to list released public and private offers for a specific product ID.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import software.amazon.awssdk.services.marketplacecatalog.model.EntitySummary;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityTypeFilters;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferFilters;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferProductIdFilter;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferStateFilter;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferTargetingFilter;

public class ListProductPublicOrPrivateReleasedOffers {
```

```

/*
 * List released Public/Private offers for a specific product id.
 * Example below is to list released public offers.
 * To change to released private offers, change OFFER_TARGETING_NONE (None) to
OFFER_TARGETING_BUYERACCOUNTS(BuyerAccounts)
 */
public static void main(String[] args) {

 List<EntitySummary> entitySummaryList = getEntitySummaryList();
 ReferenceCodesUtils.formatOutput(entitySummaryList);
}

public static List<EntitySummary> getEntitySummaryList() {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 // define list entities filters

 EntityTypeFilters entityTypeFilters =
 EntityTypeFilters.builder()
 .offerFilters(OfferFilters.builder()
 .targeting(OfferTargetingFilter.builder()
 .valueListWithStrings(OFFER_TARGETING_NONE)
 .build())
 .state(OfferStateFilter.builder()
 .valueListWithStrings(OFFER_STATE_RELEASED)
 .build())
 .productId(OfferProductIdFilter.builder()
 .valueList(PRODUCT_ID)
 .build())
 .build())
 .build();

 ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(null)

```

```

 .build();

 ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 // save all entitySummary of the results into entitySummaryList

 List<EntitySummary> entitySummaryList = new ArrayList<EntitySummary>();

 entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());

 while (listEntitiesResponse.nextToken() != null &&
listEntitiesResponse.nextToken().length() > 0) {
 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
 entitySummaryList.addAll(listEntitiesResponse.entitySummaryList());
 }
 return entitySummaryList;
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update an offer to apply a contract with Pay-As-You-Go pricing

The following code example shows how to update an offer to apply a contract with Pay-As-You-Go pricing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "WorkloadSmall",
 "Price": "0.15"
 },
 {
 "DimensionKey": "WorkloadMedium",
 "Price": "0.25"
 }
]
 }
]
 }
]
 }
 }
]
}
```

```

 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "150"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "300"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update an offer to apply hourly annual pricing

The following code example shows how to update an offer to apply hourly annual pricing.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "m5.large",
 "Price": "0.13"
 }
]
 }
]
 },
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
```

```

 "Selector": {
 "Type": "Duration",
 "Value": "P365D"
 },
 "RateCard": [
 {
 "DimensionKey": "m5.large",
 "Price": "20.03"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
}
]
}
]
}
]
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update an offer to apply targeting to specific geographic regions

The following code example shows how to update an offer to apply targeting to specific geographic regions.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
```

```

 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "CountryCodes": [
 "US",
 "ES",
 "FR",
 "AU"
]
 }
 }
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update name and description of a public offer

The following code example shows how to update name and description of a public offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [

```



```

{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-111111111111"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update the EULA of an offer

The following code example shows how to update the EULA of an offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
```

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "Name": "New offer name",
 "Description": "New offer description"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update the expiration date of a private offer to a future date

The following code example shows how to update the expiration date of a private offer to a date in the future to give buyers more time to evaluate and accept the offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 }
 }
]
}
```

```

 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2026-01-01"
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update the free trial duration of a public free trial offer for a SaaS product

The following code example shows how to update the free trial duration of a public free trial offer for a SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "FreeTrialPricingTerm",
 "Duration": "P21D",
 "Grants": [

```

```

 {
 "DimensionKey": "WorkloadSmall"
 },
 {
 "DimensionKey": "WorkloadMedium"
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update the refund policy of an offer

The following code example shows how to update the refund policy of an offer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "offer-11111111111111"
 },
 "DetailsDocument": {
 "Terms": [

```

```

 {
 "Type": "SupportTerm",
 "RefundPolicy": "Updated refund policy description"
 }
]
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Products

### Describe an AMI, SaaS, or Container product

The following code example shows how to describe an AMI, SaaS, or Container product and check if it contains all the information you want to know about the product.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.catalogapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;

```

```

public class DescribeEntity {

 /**
 * Describe my AMI or SaaS or Container product and check if it contains all the
 information I need to know about the product
 */
 public static void main(String[] args) {

 String offerId = args.length > 0 ? args[0] : OFFER_ID;

 DescribeEntityResponse describeEntityResponse =
 getDescribeEntityResponse(offerId);

 ReferenceCodesUtils.formatOutput(describeEntityResponse);
 }

 public static DescribeEntityResponse getDescribeEntityResponse(String offerId) {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(offerId)
 .build();

 DescribeEntityResponse describeEntityResponse =
 marketplaceCatalogClient.describeEntity(describeEntityRequest);
 return describeEntityResponse;
 }
}

```

- For API details, see [DescribeEntity](#) in *AWS SDK for Java 2.x API Reference*.

## List all AMI, SaaS, or Container products and associated public offers

The following code example shows how to list all AMI, SaaS, or Container products and associated public offers.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
package com.example.awsmarketplace.catalogapi;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import software.amazon.awssdk.services.marketplacecatalog.model.EntitySummary;
import software.amazon.awssdk.services.marketplacecatalog.model.EntityTypeFilters;
import software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.ListEntitiesResponse;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferFilters;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferProductIdFilter;
import software.amazon.awssdk.services.marketplacecatalog.model.OfferStateFilter;
import
 software.amazon.awssdk.services.marketplacecatalog.model.OfferTargetingFilter;

public class ListEntities {

 /*
 * List all my AMI or SaaS or Container products and associated public offers
 */
 public static void main(String[] args) {

 Map<String, List<EntitySummary>> allProductsWithOffers =
 getAllProductsWithOffers();
 }
}
```

```
ReferenceCodesUtils.formatOutput(allProductsWithOffers);
}

public static Map<String, List<EntitySummary>> getAllProductsWithOffers() {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 Map<String, List<EntitySummary>> allProductsWithOffers = new HashMap<String,
List<EntitySummary>> ();

 // get all product entities
 List<EntitySummary> productEntityList = new ArrayList<EntitySummary>();

 ListEntitiesRequest listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(PRODUCT_TYPE_AMI)
 .maxResults(10)
 .nextToken(null)
 .build();

 ListEntitiesResponse listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 productEntityList.addAll(listEntitiesResponse.entitySummaryList());

 while (listEntitiesResponse.nextToken() != null) {
 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(PRODUCT_TYPE_AMI)
 .maxResults(10)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
 productEntityList.addAll(listEntitiesResponse.entitySummaryList());
 }
}
```



```
// loop through each product entity and get the public released offers associated
using product id filter
```

```
for (EntitySummary productEntitySummary : productEntityList) {
 EntityTypeFilters entityTypeFilters =
 EntityTypeFilters.builder()
 .offerFilters(OfferFilters.builder()
 .targeting(OfferTargetingFilter.builder()
 .valueListWithStrings(OFFER_TARGETING_NONE)
 .build())
 .state(OfferStateFilter.builder()
 .valueListWithStrings(OFFER_STATE_RELEASED)
 .build())
 .productId(OfferProductIdFilter.builder()
 .valueList(productEntitySummary.entityId())
 .build())
 .build())
 .build();

 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
 .maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(null)
 .build();

 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);

 // save all entitySummary of the results into entitySummaryList

 List<EntitySummary> offerEntitySummaryList = new ArrayList<EntitySummary>();

 offerEntitySummaryList.addAll(listEntitiesResponse.entitySummaryList());

 while (listEntitiesResponse.nextToken() != null &&
listEntitiesResponse.nextToken().length() > 0) {
 listEntitiesRequest =
 ListEntitiesRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityType(ENTITY_TYPE_OFFER)
```

```
 .maxResults(10)
 .entityTypeFilters(entityTypeFilters)
 .nextToken(listEntitiesResponse.nextToken())
 .build();
 listEntitiesResponse =
marketplaceCatalogClient.listEntities(listEntitiesRequest);
 offerEntitySummaryList.addAll(listEntitiesResponse.entitySummaryList());
}

// save final results into map; key = product id; value = offer entity summary
list

 allProductsWithOffers.put(productEntitySummary.entityId(),
offerEntitySummaryList);
}
return allProductsWithOffers;
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DescribeEntity](#)
  - [ListEntities](#)

## Resale authorization

### Create draft resale authorization

The following code example shows how to create draft resale authorization for any product type so you can review them internally before publishing to a Channel Partner.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Describe a resale authorization

The following code example shows how to describe a resale authorization.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.catalogapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
```

```
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;

public class DescribeEntity {

 /**
 * Describe my AMI or SaaS or Container product and check if it contains all the
 * information I need to know about the product
 */
 public static void main(String[] args) {

 String offerId = args.length > 0 ? args[0] : OFFER_ID;

 DescribeEntityResponse describeEntityResponse =
 getDescribeEntityResponse(offerId);

 ReferenceCodesUtils.formatOutput(describeEntityResponse);
 }

 public static DescribeEntityResponse getDescribeEntityResponse(String offerId) {
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(offerId)
 .build();

 DescribeEntityResponse describeEntityResponse =
 marketplaceCatalogClient.describeEntity(describeEntityRequest);
 return describeEntityResponse;
 }
}
```

- For API details, see [DescribeEntity](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a one-time resale authorization with a private offer

The following code example shows how to publish a one-time resale authorization with a private offer so a Channel Partner can use it to create a Channel Partner Private Offer (CPPO).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 }
 }
]
}
```

```

 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [

```

```

 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
],
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "OffersMaxQuantity": 1
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization with an expiration date

The following code example shows how to publish multi-use resale authorization with an expiration date for an AMI product with hourly annual pricing so a Channel Partner can use it to create a CPPO.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
```



```

 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 },
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-05-31"
 }
},
{
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {

```

```

 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization with an expiration date and a EULA

The following code example shows how to publish multi-use resale authorization with an expiration date for any product type and add a custom EULA to be sent to the buyer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
],
}

```

```

{
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-05-31"
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
]
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",

```

```

 "QuantityConfiguration": "Allowed"
 }
}
]
}
],
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization with an expiration date and reseller contract documentation

The following code example shows how to publish multi-use resale authorization with an expiration date for any product type and add reseller contract documentation between the ISV and Channel Partner.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
```

```

 "AvailabilityEndDate": "2023-05-31"
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 },
 {
 "Type": "ResaleLegalTerm",
 "Documents": [
 {
 "Type": "CustomResellerContract",
 "Url": "https://s3.amazonaws.com/aws-mp-standard-
contracts/Standard-Contact-for-AWS-Marketplace-2022-07-14.pdf"
 }
]
 }
]
 },
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",

```

```

 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization with expiration and add a specific buyer account

The following code example shows how to publish multi-use resale authorization with an expiration date for any product type and add a specific buyer account for the resale.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-05-31"
 }
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 },
]
}
```



```

 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 },
 {
 "ChangeType": "UpdateBuyerTargetingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerTargetingTerm",
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111"
]
 }
 }
]
 }
 }
]
}

```

```

]
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization without an expiration date

The following code example shows how to publish multi-use resale authorization without an expiration date for an AMI product with hourly annual pricing so a CP can use that to create a CPPO.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
```

```

 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization without an expiration date and a EULA

The following code example shows how to publish multi-use resale authorization without an expiration date for any product type and add a custom EULA to be sent to the buyer.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
```

```

 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 },
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {

```

```

 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization without an expiration date and reseller contract documentation

The following code example shows how to publish multi-use resale authorization without an expiration date for any product type and add reseller contract documentation between the ISV and Channel Partner.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 },
],
}

```

```

 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 },
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 }
 }
}

```



```

]
 }
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 },
 {
 "Type": "ResaleLegalTerm",
 "Documents": [
 {
 "Type": "CustomResellerContract",
 "Url": "https://s3.amazonaws.com/aws-mp-standard-
contracts/Standard-Contact-for-AWS-Marketplace-2022-07-14.pdf"
 }
]
 }
]
 }
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish multi-use resale authorization without expiration and add a specific buyer account

The following code example shows how to publish multi-use resale authorization without an expiration date for any product type and add a specific buyer account for the resale.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
```

```

 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 },
 },
 {
 "ChangeType": "UpdateBuyerTargetingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerTargetingTerm",
 "PositiveTargeting": {
 "BuyerAccounts": [

```

```

 "111111111111"
]
 }
 }
],
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish one-time resale authorization and add Flexible payment schedule

The following code example shows how to publish one-time resale authorization for any product type and add Flexible payment schedule.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
```

```

 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleFixedUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "Price": "0.00",
 "Duration": "P12M",
 "Grants": [
 {
 "DimensionKey": "Users",
 "MaxQuantity": 10
 }
]
 }
]
 },
 {
 "ChangeType": "UpdatePaymentScheduleTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "ResalePaymentScheduleTerm",
 "CurrencyCode": "USD",
 "Schedule": [
 {
 "ChargeDate": "2023-09-01",
 "ChargeAmount": "200.00"
 },
 {
 "ChargeDate": "2023-12-01",
 "ChargeAmount": "250.00"
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateAvailability",

```

```

 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "AvailabilityEndDate": "2023-06-30",
 "OffersMaxQuantity": 1
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish one-time resale authorization and add a EULA

The following code example shows how to publish one-time resale authorization for any product type and add a custom EULA to be sent to the buyer.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
```



```

 "OffersMaxQuantity": 1
 },
 {
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
]
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {

```

```

 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish one-time resale authorization and add a specific buyer account

The following code example shows how to publish one-time resale authorization for any product type and add a specific buyer account for the resale.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {

```

```

 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
},
{
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
]
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
 }
}

```

```

 }
 }
]
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "OffersMaxQuantity": "1"
 }
},
{
 "ChangeType": "UpdateBuyerTargetingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },

```

```

 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerTargetingTerm",
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111"
]
 }
 }
]
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish one-time resale authorization and add reseller contract documentation

The following code example shows how to publish one-time resale authorization for any product type and add reseller contract documentation between the ISV and Channel Partner.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 }
 }
]
}

```

```

 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
},
{
 "ChangeType": "ReleaseResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "OffersMaxQuantity": 1
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ResaleConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 }
 }
]
 }
]
 }
},

```

```

 "RateCard": [
 {
 "DimensionKey": "t2.small",
 "Price": "150"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
}

},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerLegalTerm",
 "Documents": [
 {
 "Type": "CustomEula",
 "Url": "https://s3.amazonaws.com/sample-bucket/
custom-eula.pdf"
 }
]
 }
]
 }
}
]
}
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish one-time resale authorization and add whether it is a renewal

The following code example shows how to publish one-time resale authorization for any product type and add whether it is a renewal.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateResaleAuthorization",
 "ChangeName": "ResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0"
 },
 "DetailsDocument": {
 "ProductId": "prod-111111111111",
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "ResellerAccountId": "111111111111"
 }
 },
 {
 "ChangeType": "UpdateBuyerTargetingTerms",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "BuyerTargetingTerm",
```



```

 "PositiveTargeting": {
 "BuyerAccounts": [
 "222222222222"
]
 }
],
 {
 "ChangeType": "UpdateAvailability",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "OffersMaxQuantity": 1
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "$ResaleAuthorization.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product",
 "PreExistingBuyerAgreement": {
 "AcquisitionChannel": "AwsMarketplace",
 "PricingModel": "Contract"
 }
 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Restrict resale authorization

The following code example shows how to restrict resale authorization.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "RestrictResaleAuthorization",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "resaleauthz-111111111111"
 },
 "DetailsDocument": {}
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update name and description of one-time or multi-use resale authorization

The following code example shows how to update name and description of one-time or multi-use resale authorization before publishing for any product type.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "ResaleAuthorization@1.0",
 "Identifier": "resaleauthz-111111111111"
 },
 "DetailsDocument": {
 "Name": "TestResaleAuthorization",
 "Description": "Worldwide ResaleAuthorization for Test Product"
 }
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## SaaS products

### Create a draft SaaS product with a draft public offer

The following code example shows how to create a draft SaaS product with a draft public offer.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
```

```

 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "SaaSProduct@1.0"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product"
 }
 },
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier",
 "Name": "Test Offer"
 }
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited SaaS product and public offer with contract pricing

The following code example shows how to create a public or limited SaaS product and public offer with contract pricing. This example creates either a standard or custom EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0"
 },
 "ChangeName": "CreateProductChange",
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Data Catalogs"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 },
],
}

```

```

 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 },
 },
 {
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "DeliveryOptions": [
 {
 "Details": {
 "SaaSUrlDeliveryOptionDetails": {
 "FulfillmentUrl": "https://sample.amazonaws.com/sample-saas-fulfillment-url"
 }
 }
 }
]
 }
 },
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "BasicService",
 "Description": "Basic Service",
 "Name": "Basic Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 }
]
 }
]

```

```

 {
 "Key": "PremiumService",
 "Description": "Premium Service",
 "Name": "Premium Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 }
],
},
{
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
},
{
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public offer for SaaSProduct using AWS Marketplace API
Reference Code",
 "Description": "Test public offer with contract pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
},
{
 "ChangeType": "UpdatePricingTerms",

```

```

"Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
},
"DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P1M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "20"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "25"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 },
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "150"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "300"
 }
]
 }
]
 }
]
}

```



```

 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
}
]
}
]
}
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
}
}

```

```

]
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
 }
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited SaaS product and public offer with contract with Pay-As-You-Go pricing

The following code example shows how to create a public or limited SaaS product and public offer with a contract with Pay-As-You-Go pricing. This example creates either a standard or custom EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0"
 }
 }
]
}

```

```

 },
 "ChangeName": "CreateProductChange",
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Data Catalogs"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [
 "111111111111",
 "222222222222"
]
 }
 }
 }
}

```

```

 },
 {
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "DeliveryOptions": [
 {
 "Details": {
 "SaaSUrlDeliveryOptionDetails": {
 "FulfillmentUrl": "https://sample.amazonaws.com/
sample-saas-fulfillment-url"
 }
 }
 }
]
 }
 },
 {
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "BasicService",
 "Description": "Basic Service",
 "Name": "Basic Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 },
 {
 "Key": "PremiumService",
 "Description": "Premium Service",
 "Name": "Premium Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 }
]
 }
]
}

```

```

 },
 {
 "Key": "WorkloadSmall",
 "Description": "Workload: Per medium instance",
 "Name": "Workload: Per medium instance",
 "Types": [
 "ExternallyMetered"
],
 "Unit": "Units"
 },
 {
 "Key": "WorkloadMedium",
 "Description": "Workload: Per large instance",
 "Name": "Workload: Per large instance",
 "Types": [
 "ExternallyMetered"
],
 "Unit": "Units"
 }
]
},
{
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
},
{
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 }
}

```

```

 },
 "DetailsDocument": {
 "Name": "Test public offer for SaaSProduct using AWS Marketplace API
Reference Code",
 "Description": "Test public offer with contract pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "WorkloadSmall",
 "Price": "0.15"
 },
 {
 "DimensionKey": "WorkloadMedium",
 "Price": "0.25"
 }
]
 }
]
 }
],
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 }
 }
]
 }
 }
}

```

```

 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "150"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "300"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
}

},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
}

},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",

```

```

 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
},
{
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
}
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Create a public or limited SaaS product and public offer with subscription pricing

The following code example shows how to create a public or limited SaaS product and public offer with subscription pricing. This examples creates either a standard or custom EULA.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",

```



```

"ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0"
 },
 "ChangeName": "CreateProductChange",
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Data Catalogs"
],
 "LogoUrl": "https://s3.amazonaws.com/logos/sample.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
 },
 {
 "ChangeType": "UpdateTargeting",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PositiveTargeting": {
 "BuyerAccounts": [

```

```

 "111111111111",
 "222222222222"
]
}
},
{
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "DeliveryOptions": [
 {
 "Details": {
 "SaaSUrlDeliveryOptionDetails": {
 "FulfillmentUrl": "https://sample.amazonaws.com/
sample-saas-fulfillment-url"
 }
 }
 }
]
 }
},
{
 "ChangeType": "AddDimensions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "WorkloadSmall",
 "Description": "Workload: Per medium instance",
 "Name": "Workload: Per medium instance",
 "Types": [
 "ExternallyMetered"
],
 "Unit": "Units"
 },
 {
 "Key": "WorkloadMedium",
 "Description": "Workload: Per large instance",

```

```

 "Name": "Workload: Per large instance",
 "Types": [
 "ExternallyMetered"
],
 "Unit": "Units"
 }
]
},
{
 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
},
{
 "ChangeType": "CreateOffer",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "ChangeName": "CreateOfferChange",
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
},
{
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "Test public offer for SaaSProduct using AWS Marketplace API
Reference Code",
 "Description": "Test public offer with contract pricing for
SaaSProduct using AWS Marketplace API Reference Code"
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 }
}

```

```

 },
 "DetailsDocument": {
 "PricingModel": "Usage",
 "Terms": [
 {
 "Type": "UsageBasedPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "RateCard": [
 {
 "DimensionKey": "WorkloadSmall",
 "Price": "0.15"
 },
 {
 "DimensionKey": "WorkloadMedium",
 "Price": "0.25"
 }
]
 }
]
 }
]
 }
],
},
{
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
}

```

```

 },
 {
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Absolutely no refund, period."
 }
]
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a SaaS product and associated public offer

The following code example shows how to publish a SaaS product and associated public offer. The product will be in a limited state by default.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```
{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "CreateProduct",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "SaaSProduct@1.0"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "ProductTitle": "Sample product",
 "ShortDescription": "Brief description",
 "LongDescription": "Detailed description",
 "Highlights": [
 "Sample highlight"
],
 "SearchKeywords": [
 "Sample keyword"
],
 "Categories": [
 "Data Catalogs"
],
 "LogoUrl": "https://bucketname.s3.amazonaws.com/logo.png",
 "VideoUrls": [
 "https://sample.amazonaws.com/awsmvp-video-1"
],
 "AdditionalResources": []
 }
 },
 {
 "ChangeType": "AddDimensions",
 "Entity": {
```

```

 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": [
 {
 "Key": "BasicService",
 "Description": "Basic Service",
 "Name": "Basic Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 },
 {
 "Key": "PremiumService",
 "Description": "Premium Service",
 "Name": "Premium Service",
 "Types": [
 "Entitled"
],
 "Unit": "Units"
 }
],
 {
 "ChangeType": "AddDeliveryOptions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "DeliveryOptions": [
 {
 "Details": {
 "SaaSUrlDeliveryOptionDetails": {
 "FulfillmentUrl": "https://www.aws.amazon.com/
marketplace/management"
 }
 }
 }
]
 }
 },
 {

```

```

 "ChangeType": "ReleaseProduct",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "$CreateProductChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 },
 {
 "ChangeType": "CreateOffer",
 "ChangeName": "CreateOfferChange",
 "Entity": {
 "Type": "Offer@1.0"
 },
 "DetailsDocument": {
 "ProductId": "$CreateProductChange.Entity.Identifier"
 }
 },
 {
 "ChangeType": "UpdateInformation",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Name": "New Test Offer",
 "Description": "New offer description"
 }
 },
 {
 "ChangeType": "UpdateLegalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "LegalTerm",
 "Documents": [
 {
 "Type": "StandardEula",
 "Version": "2022-07-14"
 }
]
 }
]
 }
 }
]

```



```

 }
]
}
},
{
 "ChangeType": "UpdateSupportTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "SupportTerm",
 "RefundPolicy": "Updated refund policy description"
 }
]
 }
},
{
 "ChangeType": "UpdatePricingTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "PricingModel": "Contract",
 "Terms": [
 {
 "Type": "ConfigurableUpfrontPricingTerm",
 "CurrencyCode": "USD",
 "RateCards": [
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P1M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "20"
 },
 {
 "DimensionKey": "PremiumService",

```

```

 "Price": "25"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 },
 {
 "Selector": {
 "Type": "Duration",
 "Value": "P12M"
 },
 "RateCard": [
 {
 "DimensionKey": "BasicService",
 "Price": "150"
 },
 {
 "DimensionKey": "PremiumService",
 "Price": "300"
 }
],
 "Constraints": {
 "MultipleDimensionSelection": "Allowed",
 "QuantityConfiguration": "Allowed"
 }
 }
]
}
]
},
{
 "ChangeType": "UpdateRenewalTerms",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {
 "Terms": [
 {
 "Type": "RenewalTerm"
 }
]
 }
}

```

```

]
 }
 },
 {
 "ChangeType": "ReleaseOffer",
 "Entity": {
 "Type": "Offer@1.0",
 "Identifier": "$CreateOfferChange.Entity.Identifier"
 },
 "DetailsDocument": {}
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Publish a SaaS product and associated public offer from an existing draft

The following code example shows how to publish a SaaS product and associated public offer from an existing draft. The product will be in a limited state by default.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateVisibility",
 "ChangeName": "CreateProductChange",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": {
 "TargetVisibility": "Public"
 }
 }
]
}

```

```

 }
 }
]
}

```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Update dimensions on an AMI or SaaS product

The following code example shows how to update dimensions on an AMI or SaaS product.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

To run this example, pass the following JSON changeset to `RunChangesets` in *Utilities to start a changeset* from the **Utilities** section.

```

{
 "Catalog": "AWSMarketplace",
 "ChangeSet": [
 {
 "ChangeType": "UpdateDimensions",
 "Entity": {
 "Type": "SaaSProduct@1.0",
 "Identifier": "prod-111111111111"
 },
 "DetailsDocument": [
 {
 "Key": "BasicService",
 "Types": [
 "Entitled"
],
 "Name": "Some new name",
 "Description": "Some new description"
 }
]
 }
]
}

```

```
 }
]
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## Utilities

### Utilities to start a changeset

The following code example shows how to define utilities to start a changeset.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

Utility to load a changeset from a JSON file and start processing it.

```
package com.example.awsmarketplace.catalogapi;

import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.List;

import org.apache.commons.io.IOUtils;
import org.apache.commons.lang3.StringUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.core.document.Document;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.protocols.json.internal.unmarshall.document.DocumentUnmarshaller;
import software.amazon.awssdk.protocols.jsoncore.JsonNodeParser;
import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import software.amazon.awssdk.services.marketplacecatalog.model.Change;
```

```

import software.amazon.awssdk.services.marketplacecatalog.model.Entity;
import
 software.amazon.awssdk.services.marketplacecatalog.model.StartChangeSetRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.StartChangeSetResponse;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.ToNumberPolicy;
import com.example.awsmarketplace.catalogapi.Entity.ChangeSet;
import com.example.awsmarketplace.catalogapi.Entity.ChangeSetEntity;
import com.example.awsmarketplace.catalogapi.Entity.Root;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;
import com.example.awsmarketplace.utils.StringSerializer;

/**
 * Before running this Java V2 code example, convert all Details attribute to
 * DetailsDocument if any
 */

public class RunChangesets {

 private static final Gson GSON = new GsonBuilder()
 .setObjectToNumberStrategy(ToNumberPolicy.LAZILY_PARSED_NUMBER)
 .registerTypeAdapter(String.class, new StringSerializer())
 .create();

 public static void main(String[] args) {

 // input json can be specified here or passed from input parameter
 String inputChangeSetFile = "changeSets/offers/
CreateReplacementOfferFromAGWithContractPricingDetailDocument.json";

 if (args.length > 0)
 inputChangeSetFile = args[0];

 // parse the input changeset file to string for process
 String changeSetsInput = readChangeSetToString(inputChangeSetFile);

 // process the changeset request
 try {
 StartChangeSetResponse result = getChangeSetRequestResult(changeSetsInput);
 ReferenceCodesUtils.formatOutput(result);
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}

```

```
}
}

public static StartChangeSetResponse getChangeSetRequestResult(String
changeSetsInput) throws IOException {

 //set up AWS credentials
 MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 //changeset list to save all the changesets in the changesets file
 List<Change> changeSetLists = new ArrayList<Change>();

 // read all changesets into object
 Root root = GSON.fromJson(changeSetsInput, Root.class);

 // process each changeset and add each changeset request to changesets list
 for (ChangeSet cs : root.changeSet) {

 ChangeSetEntity entity = cs.Entity;
 String entityType = entity.Type;
 String entityIdIdentifier = StringUtils.defaultIfBlank(entity.Identifier, null);
 Document detailsDocument = getDocumentFromObject(cs.DetailsDocument);

 Entity awsEntity =
 Entity.builder()
 .type(entityType)
 .identifier(entityIdIdentifier)
 .build();

 Change inputChangeRequest =
 Change.builder()
 .changeType(cs.ChangeType)
 .changeName(cs.ChangeName)
 .entity(awsEntity)
 .detailsDocument(detailsDocument)
 .build();

 changeSetLists.add(inputChangeRequest);
 }
}
```

```
// process all changeset requests
StartChangeSetRequest startChangeSetRequest =
 StartChangeSetRequest.builder()
 .catalog(root.catalog)
 .changeSet(changeSetLists)
 .build();

StartChangeSetResponse result =
marketplaceCatalogClient.startChangeSet(startChangeSetRequest);

return result;
}

public static Document getDocumentFromObject(Object detailsObject) {

 String detailsString = "{}";
 try {
 detailsString = IOUtils.toString(new
ByteArrayInputStream(GSON.toJson(detailsObject).getBytes()), "UTF-8");
 } catch (IOException e) {
 e.printStackTrace();
 }

 JsonNodeParser jsonNodeParser = JsonNodeParser.create();
 Document doc = jsonNodeParser.parse(detailsString).visit(new
DocumentUnmarshaller());
 return doc;
}

public static String readChangeSetToString (String inputChangeSetFile) {

 InputStream changesetInputStream =
RunChangesets.class.getClassLoader().getResourceAsStream(inputChangeSetFile);

 String changeSetsInput = null;

 try {
 changeSetsInput = IOUtils.toString(changesetInputStream, "UTF-8");
 } catch (IOException e) {
 e.printStackTrace();
 }

 return changeSetsInput;
}
```



```
}
}
```

- For API details, see [StartChangeSet](#) in *AWS SDK for Java 2.x API Reference*.

## AWS Marketplace Agreement API examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS Marketplace Agreement API.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Agreements](#)

## Agreements

### Get all agreement IDs

The following code example shows how to get all agreement IDs.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
```

```
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAllAgreementsIds {

 /*
 * Get all purchase agreements ids with party type = proposer;
 * Depend on the number of agreements in your account, this code may take some time
 * to finish.
 */
 public static void main(String[] args) {

 List<String> agreementIds = getAllAgreementIds();

 ReferenceCodesUtils.formatOutput(agreementIds);

 }

 public static List<String> getAllAgreementIds() {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 // get all filters
 Filter partyType = Filter.builder().name(PARTY_TYPE_FILTER_NAME)
 .values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build();

 Filter agreementType = Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME)
```

```
.values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASEAGREEMENT).build());

List<Filter> searchFilters = new ArrayList<Filter>();

searchFilters.addAll(Arrays.asList(partyType, agreementType));

// Save all results in a list array
List<AgreementViewSummary> agreementSummaryList = new
ArrayList<AgreementViewSummary>();

SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(searchFilters)
 .build();

SearchAgreementsResponse searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());

while (searchAgreementsResponse.nextToken() != null &&
searchAgreementsResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .nextToken(searchAgreementsResponse.nextToken())
 .filters(searchFilters)
 .build();
 searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());
}

List<String> agreementIds = new ArrayList<String>();
for (AgreementViewSummary summary : agreementSummaryList) {
 agreementIds.add(summary.agreementId());
}
return agreementIds;
}
}
```

- For API details, see [SearchAgreements](#) in *AWS SDK for Java 2.x API Reference*.

## Get all agreements

The following code example shows how to get all agreements.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;

import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAllAgreements {

 /*
 * Get all purchase agreements with party type = proposer;
 */
}
```

```

 * Depend on the number of agreements in your account, this code may take some time
 to finish.
 */
 public static void main(String[] args) {

 List<AgreementViewSummary> agreementSummaryList = getAllAgreements();

 ReferenceCodesUtils.formatOutput(agreementSummaryList);
 }

 public static List<AgreementViewSummary> getAllAgreements() {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 // get all filters

 Filter partyType = Filter.builder().name(PARTY_TYPE_FILTER_NAME)
 .values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build();

 Filter agreementType = Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME)
 .values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASEAGREEMENT).build();

 List<Filter> searchFilters = new ArrayList<Filter>();

 searchFilters.addAll(Arrays.asList(partyType, agreementType));

 // Save all results in a list array

 List<AgreementViewSummary> agreementSummaryList = new
 ArrayList<AgreementViewSummary>();

 SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(searchFilters)
 .build();

 SearchAgreementsResponse searchAgreementsResponse =
 marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());
 }

```

```

 while (searchAgreementsResponse.nextToken() != null &&
searchAgreementsResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .nextToken(searchAgreementsResponse.nextToken())
 .filters(searchFilters).build();
 searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());
 }
 return agreementSummaryList;
}
}

```

- For API details, see [SearchAgreements](#) in *AWS SDK for Java 2.x API Reference*.

## Get customer ID from an agreement

The following code example shows how to get customer ID from an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```

// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;

```

```
import
software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;

public class GetAgreementCustomerInfo {

 /*
 * Obtain metadata about the customer who created the agreement, such as the
 customer's AWS Account ID
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 DescribeAgreementResponse describeAgreementResponse =
getDescribeAgreementResponse(agreementId);

 System.out.println("Customer's AWS Account ID is " +
describeAgreementResponse.acceptor().accountId());

 }

 public static DescribeAgreementResponse getDescribeAgreementResponse(String
agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

 DescribeAgreementResponse describeAgreementResponse =
marketplaceAgreementClient.describeAgreement(describeAgreementRequest);
 return describeAgreementResponse;
 }
}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get financial details from an agreement

The following code example shows how to get financial details from an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;

public class GetAgreementFinancialDetails {

 /**
 * Obtain financial details, such as Total Contract Value of the agreement from a
 * given agreement
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 String totalContractValue = getTotalContractValue(agreementId);

 System.out.println("Total Contract Value is " + totalContractValue);
 }
}
```



```
}

public static String getTotalContractValue(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

 DescribeAgreementResponse describeAgreementResponse =
 marketplaceAgreementClient.describeAgreement(describeAgreementRequest);

 String totalContractValue = "N/A";

 if (describeAgreementResponse.estimatedCharges() != null) {
 totalContractValue =
 describeAgreementResponse.estimatedCharges().agreementValue()
 + " "
 + describeAgreementResponse.estimatedCharges().currencyCode();
 }
 return totalContractValue;
}
}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get free trial details from an agreement

The following code example shows how to get free trial details from an agreement.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.FreeTrialPricingTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;

import java.util.ArrayList;
import java.util.List;

import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsFreeTrialDetails {

 /**
 * Obtain the details from an agreement of a free trial I have provided to the
 * customer
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;
```

```

 List<FreeTrialPricingTerm> freeTrialPricingTerms =
getFreeTrialPricingTerms(agreementId);

 ReferenceCodesUtils.formatOutput(freeTrialPricingTerms);
}

public static List<FreeTrialPricingTerm> getFreeTrialPricingTerms(String
agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 List<FreeTrialPricingTerm> freeTrialPricingTerms = new
ArrayList<FreeTrialPricingTerm>();

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 if (acceptedTerm.freeTrialPricingTerm() != null) {
 freeTrialPricingTerms.add(acceptedTerm.freeTrialPricingTerm());
 }
 }
 return freeTrialPricingTerms;
}
}

```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get information about an agreement

The following code example shows how to get information about an agreement.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;

public class DescribeAgreement {

 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 DescribeAgreementResponse describeAgreementResponse = getResponse(agreementId);

 ReferenceCodesUtils.formatOutput(describeAgreementResponse);

 }

 public static DescribeAgreementResponse getResponse(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();
```

```
DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

DescribeAgreementResponse describeAgreementResponse =
 marketplaceAgreementClient.describeAgreement(describeAgreementRequest);
return describeAgreementResponse;
}

}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get product and offer details from an agreement

The following code example shows how to get product and offer details from an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;
import software.amazon.awssdk.services.marketplaceagreement.model.Resource;
```

```
import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;

public class GetProductAndOfferDetailFromAgreement {

 public static void main(String[] args) {

 // call Agreement API to get offer and product information for the agreement

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<DescribeEntityResponse> entityResponseList = getEntities(agreementId);

 for (DescribeEntityResponse response : entityResponseList) {
 ReferenceCodesUtils.formatOutput(response);
 }
 }

 public static List<DescribeEntityResponse> getEntities(String agreementId) {
 List<DescribeEntityResponse> entityResponseList = new
 ArrayList<DescribeEntityResponse> ();

 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

 DescribeAgreementResponse describeAgreementResponse =
 marketplaceAgreementClient.describeAgreement(describeAgreementRequest);
 }
}
```

```
// get offer id for the given agreement

String offerId = describeAgreementResponse.proposalSummary().offerId();

// get all the product ids for this agreement

List<String> productIds = new ArrayList<String>();
for (Resource resource : describeAgreementResponse.proposalSummary().resources())
{
 productIds.add(resource.id());
}

// call Catalog API to get the details of the offer and products

MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

DescribeEntityRequest describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(offerId).build();

DescribeEntityResponse describeEntityResponse =
marketplaceCatalogClient.describeEntity(describeEntityRequest);

entityResponseList.add(describeEntityResponse);

for (String productId : productIds) {
 describeEntityRequest =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(productId).build();
 describeEntityResponse =
marketplaceCatalogClient.describeEntity(describeEntityRequest);
 System.out.println("Print details for product " + productId);
 entityResponseList.add(describeEntityResponse);
}
return entityResponseList;
}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get the EULA of an agreement

The following code example shows how to get the EULA of an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.DocumentItem;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsEula {

 /*
 * Obtain the EULA I have entered into with my customer via the agreement
 */
}
```



```

public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<DocumentItem> legalEulaArray = getLegalEula(agreementId);

 ReferenceCodesUtils.formatOutput(legalEulaArray);
}

public static List<DocumentItem> getLegalEula(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 List<DocumentItem> legalEulaArray = new ArrayList<>();

 getAgreementTermsResponse.acceptedTerms().stream()
 .filter(acceptedTerm -> acceptedTerm.legalTerm() != null &&
 acceptedTerm.legalTerm().hasDocuments())
 .flatMap(acceptedTerm -> acceptedTerm.legalTerm().documents().stream())
 .filter(docItem -> docItem.type() != null)
 .forEach(legalEulaArray::add);
 return legalEulaArray;
}
}

```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the auto renewal terms of an agreement

The following code example shows how to get the auto renewal terms of an agreement.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

public class GetAgreementAutoRenewal {

 /*
 * Obtain the auto-renewal status of the agreement
 */

 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 String autoRenewal = getAutoRenewal(agreementId);

 System.out.println("Auto-Renewal status is " + autoRenewal);
 }

 public static String getAutoRenewal(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
```

```

 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder()
 .agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 String autoRenewal = "No Auto Renewal";

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 if (acceptedTerm.renewalTerm() != null &&
 acceptedTerm.renewalTerm().configuration() != null
 && acceptedTerm.renewalTerm().configuration().enableAutoRenew() != null) {
 autoRenewal =
 String.valueOf(acceptedTerm.renewalTerm().configuration().enableAutoRenew().booleanValue());
 break;
 }
 }
 return autoRenewal;
}
}

```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the dimensions purchased in an agreement

The following code example shows how to get the dimensions purchased in an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import software.amazon.awssdk.services.marketplaceagreement.model.Dimension;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsDimensionPurchased {

 /*
 * Obtain the dimensions the buyer has purchased from me via the agreement
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<String> dimensionKeys = getDimensionKeys(agreementId);

 ReferenceCodesUtils.formatOutput(dimensionKeys);
 }

 public static List<String> getDimensionKeys(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
```

```

 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 List<String> dimensionKeys = new ArrayList<String>();
 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 if (acceptedTerm.configurableUpfrontPricingTerm() != null) {
 if
(agreementTerm.configurableUpfrontPricingTerm().configuration().selectorValue() !=
null) {
 List<Dimension> dimensions =
acceptedTerm.configurableUpfrontPricingTerm().configuration().dimensions();
 for (Dimension dimension : dimensions) {
 dimensionKeys.add(dimension.dimensionKey());
 }
 }
 }
 }
 return dimensionKeys;
}
}

```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the instances of each dimension purchased in an agreement

The following code example shows how to get the instances of each dimension purchased in an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
```

```
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import software.amazon.awssdk.services.marketplaceagreement.model.Dimension;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsDimensionInstances {

 /**
 * get instances of each dimension that buyer has purchased in the agreement
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 Map<String, List<Dimension>> dimensionMap = getDimensions(agreementId);

 ReferenceCodesUtils.formatOutput(dimensionMap);
 }

 public static Map<String, List<Dimension>> getDimensions(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
```

```

 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 Map<String, List<Dimension>> dimensionMap = new HashMap<String,
List<Dimension>>();

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 List<Dimension> dimensionsList = new ArrayList<Dimension>();
 if (acceptedTerm.configurableUpfrontPricingTerm() != null) {
 String selectorValue = "";
 if (acceptedTerm.configurableUpfrontPricingTerm().configuration() != null) {
 if
 (acceptedTerm.configurableUpfrontPricingTerm().configuration().selectorValue() !=
 null) {
 selectorValue =
acceptedTerm.configurableUpfrontPricingTerm().configuration().selectorValue();
 }
 if
 (acceptedTerm.configurableUpfrontPricingTerm().configuration().hasDimensions()) {
 dimensionsList =
acceptedTerm.configurableUpfrontPricingTerm().configuration().dimensions();
 }
 }
 if (selectorValue.length() > 0) {
 dimensionMap.put(selectorValue, dimensionsList);
 }
 }
 }
 return dimensionMap;
}
}

```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the payment schedule of an agreement

The following code example shows how to get the payment schedule of an agreement.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;
import
 software.amazon.awssdk.services.marketplaceagreement.model.PaymentScheduleTerm;
import software.amazon.awssdk.services.marketplaceagreement.model.ScheduleItem;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsPaymentSchedule {

 /*
 * Obtain the payment schedule I have agreed to with the agreement, including the
 * invoice date and invoice amount
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;
```



```

List<Map<String, Object>> paymentScheduleArray = getPaymentSchedules(agreementId);

ReferenceCodesUtils.formatOutput(paymentScheduleArray);
}

public static List<Map<String, Object>> getPaymentSchedules(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);
 List<Map<String, Object>> paymentScheduleArray = new ArrayList<>();

 String currencyCode = "";

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 if (acceptedTerm.paymentScheduleTerm() != null) {
 PaymentScheduleTerm paymentScheduleTerm = acceptedTerm.paymentScheduleTerm();
 if (paymentScheduleTerm.currencyCode() != null) {
 currencyCode = paymentScheduleTerm.currencyCode();
 }
 if (paymentScheduleTerm.hasSchedule()) {
 for (ScheduleItem schedule : paymentScheduleTerm.schedule()) {
 if (schedule.chargeDate() != null) {
 String chargeDate = schedule.chargeDate().toString();
 String chargeAmount = schedule.chargeAmount();
 Map<String, Object> scheduleMap = new HashMap<>();
 scheduleMap.put(ATTRIBUTE_CURRENCY_CODE, currencyCode);
 scheduleMap.put(ATTRIBUTE_CHARGE_DATE, chargeDate);
 scheduleMap.put(ATTRIBUTE_CHARGE_AMOUNT, chargeAmount);
 paymentScheduleArray.add(scheduleMap);
 }
 }
 }
 }
 }
}

```

```
 return paymentScheduleArray;
 }
}
```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the pricing per dimension in an agreement

The following code example shows how to get the pricing per dimension in an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsPricingEachDimension {

 /*
```

```

 * Obtain pricing per each dimension in the agreement
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<Object> dimensions = getDimensions(agreementId);

 ReferenceCodesUtils.formatOutput(dimensions);
 }

 public static List<Object> getDimensions(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 List<Object> dimensions = new ArrayList<Object>();

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 List<Object> rateInfo = new ArrayList<Object>();
 if (acceptedTerm.configurableUpfrontPricingTerm() != null) {
 if (acceptedTerm.configurableUpfrontPricingTerm().type() != null) {
 rateInfo.add(acceptedTerm.configurableUpfrontPricingTerm().type());
 }
 if (acceptedTerm.configurableUpfrontPricingTerm().currencyCode() != null) {
 rateInfo.add(acceptedTerm.configurableUpfrontPricingTerm().currencyCode());
 }
 if (acceptedTerm.configurableUpfrontPricingTerm().hasRateCards()) {
 rateInfo.add(acceptedTerm.configurableUpfrontPricingTerm().rateCards());
 }
 dimensions.add(rateInfo);
 }
 }
 return dimensions;
 }
}

```

```
}
```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the pricing type of an agreement

The following code example shows how to get the pricing type of an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import com.fasterxml.jackson.annotation.JsonAutoDetect.Visibility;

import java.util.ArrayList;
import java.util.Arrays;
```

```

import java.util.HashMap;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Set;

import org.apache.commons.lang3.tuple.Triple;

import software.amazon.awssdk.services.marketplacecatalog.MarketplaceCatalogClient;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityRequest;
import
 software.amazon.awssdk.services.marketplacecatalog.model.DescribeEntityResponse;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.fasterxml.jackson.annotation.PropertyAccessor;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.ObjectWriter;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;

/*
 * Obtain the pricing type of the agreement (contract, FPS, metered, free etc.)
 */
public class GetAgreementPricingType {

 private static final String FILTER_NAME = "OfferId";

 private static final String FILTER_VALUE = OFFER_ID;

 // Product types
 private static final String SAAS_PRODUCT = "SaaSProduct";
 private static final String AMI_PRODUCT = "AmiProduct";
 private static final String ML_PRODUCT = "MachineLearningProduct";
 private static final String CONTAINER_PRODUCT = "ContainerProduct";
 private static final String DATA_PRODUCT = "DataProduct";
 private static final String PROSERVICE_PRODUCT = "ProfessionalServicesProduct";
 private static final String AIQ_PRODUCT = "AiqProduct";

 // Pricing types
 private static final String CCP = "CCP";

```

```

private static final String ANNUAL = "Annual";
private static final String CONTRACT = "Contract";
private static final String SFT = "SaaS Free Trial";
private static final String HMA = "Hourly and Monthly Agreements";
private static final String HOURLY = "Hourly";
private static final String MONTHLY = "Monthly";
private static final String AFPS = "Annual FPS";
private static final String CFPS = "Contract FPS";
private static final String CCPFPS = "CCP with FPS";
private static final String BYOL = "BYOL";
private static final String FREE = "Free";
private static final String FTH = "Free Trials and Hourly";

// Agreement term pricing types
private static final Set<String> LEGAL = Set.of("LegalTerm");
private static final Set<String> CONFIGURABLE_UPFRONT =
Set.of("ConfigurableUpfrontPricingTerm");
private static final Set<String> USAGE_BASED = Set.of("UsageBasedPricingTerm");
private static final Set<String> CONFIGURABLE_UPFRONT_AND_USAGE_BASED =
Set.of("ConfigurableUpfrontPricingTerm", "UsageBasedPricingTerm");
private static final Set<String> FREE_TRIAL = Set.of("FreeTrialPricingTerm");
private static final Set<String> RECURRING_PAYMENT =
Set.of("RecurringPaymentTerm");
private static final Set<String> USAGE_BASED_AND_RECURRING_PAYMENT =
Set.of("UsageBasedPricingTerm", "RecurringPaymentTerm");
private static final Set<String> FIXED_UPFRONT_AND_PAYMENT_SCHEDULE =
Set.of("FixedUpfrontPricingTerm", "PaymentScheduleTerm");
private static final Set<String> FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED
= Set.of("FixedUpfrontPricingTerm", "PaymentScheduleTerm",
"UsageBasedPricingTerm");
private static final Set<String> BYOL_PRICING = Set.of("ByolPricingTerm");
private static final Set<String> FREE_TRIAL_AND_USAGE_BASED =
Set.of("FreeTrialPricingTerm", "UsageBasedPricingTerm");

private static final List<Set<String>> ALL_AGREEMENT_TERM_TYPES_COMBINATION
= Arrays.asList(LEGAL, CONFIGURABLE_UPFRONT, USAGE_BASED,
CONFIGURABLE_UPFRONT_AND_USAGE_BASED,
 FREE_TRIAL, RECURRING_PAYMENT, USAGE_BASED_AND_RECURRING_PAYMENT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED, BYOL_PRICING,
FREE_TRIAL_AND_USAGE_BASED);

private static MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()

```

```

 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

private static MarketplaceCatalogClient marketplaceCatalogClient =
 MarketplaceCatalogClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 /*
 * Get agreement Pricing Type given product type, agreement term types and offer
 types if needed
 */
public static String getPricingType(String productType, Set<String>
agreementTermType, Set<String> offerType) {
 Map<Triple<String, Set<String>, Set<String>>, String> pricingTypes = new
HashMap<>();

 pricingTypes.put(Triple.of(SAAS_PRODUCT, CONFIGURABLE_UPFRONT_AND_USAGE_BASED, new
HashSet<>()), CCP);
 pricingTypes.put(Triple.of(DATA_PRODUCT, CONFIGURABLE_UPFRONT_AND_USAGE_BASED, new
HashSet<>()), CCP);
 pricingTypes.put(Triple.of(CONTAINER_PRODUCT, CONFIGURABLE_UPFRONT,
CONFIGURABLE_UPFRONT_AND_USAGE_BASED), ANNUAL);
 pricingTypes.put(Triple.of(AMI_PRODUCT, CONFIGURABLE_UPFRONT,
CONFIGURABLE_UPFRONT_AND_USAGE_BASED), ANNUAL);
 pricingTypes.put(Triple.of(ML_PRODUCT, CONFIGURABLE_UPFRONT,
CONFIGURABLE_UPFRONT_AND_USAGE_BASED), ANNUAL);
 pricingTypes.put(Triple.of(CONTAINER_PRODUCT, CONFIGURABLE_UPFRONT,
CONFIGURABLE_UPFRONT), CONTRACT);
 pricingTypes.put(Triple.of(AMI_PRODUCT, CONFIGURABLE_UPFRONT,
CONFIGURABLE_UPFRONT), CONTRACT);
 pricingTypes.put(Triple.of(SAAS_PRODUCT, CONFIGURABLE_UPFRONT, new HashSet<>()),
CONTRACT);
 pricingTypes.put(Triple.of(DATA_PRODUCT, CONFIGURABLE_UPFRONT, new HashSet<>()),
CONTRACT);
 pricingTypes.put(Triple.of(AIQ_PRODUCT, CONFIGURABLE_UPFRONT, new HashSet<>()),
CONTRACT);
 pricingTypes.put(Triple.of(PROSERVICE_PRODUCT, CONFIGURABLE_UPFRONT, new
HashSet<>()), CONTRACT);
 pricingTypes.put(Triple.of(SAAS_PRODUCT, FREE_TRIAL, new HashSet<>()), SFT);
 pricingTypes.put(Triple.of(AMI_PRODUCT, USAGE_BASED_AND_RECURRING_PAYMENT, new
HashSet<>()), HMA);

```

```

pricingTypes.put(Triple.of(SAAS_PRODUCT, USAGE_BASED, new HashSet<>()), HOURLY);
pricingTypes.put(Triple.of(AMI_PRODUCT, USAGE_BASED, new HashSet<>()), HOURLY);
pricingTypes.put(Triple.of(ML_PRODUCT, USAGE_BASED, new HashSet<>()), HOURLY);
pricingTypes.put(Triple.of(CONTAINER_PRODUCT, RECURRING_PAYMENT, new HashSet<>()),
MONTHLY);
pricingTypes.put(Triple.of(AMI_PRODUCT, RECURRING_PAYMENT, new HashSet<>()),
MONTHLY);
pricingTypes.put(Triple.of(CONTAINER_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED), AFPS);
pricingTypes.put(Triple.of(AMI_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED), AFPS);
pricingTypes.put(Triple.of(ML_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE, new
HashSet<>()), AFPS);
pricingTypes.put(Triple.of(CONTAINER_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
new HashSet<>()), CFPS);
pricingTypes.put(Triple.of(AMI_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE), CFPS);
pricingTypes.put(Triple.of(SAAS_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE, new
HashSet<>()), CFPS);
pricingTypes.put(Triple.of(DATA_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE, new
HashSet<>()), CFPS);
pricingTypes.put(Triple.of(AIQ_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE, new
HashSet<>()), CFPS);
pricingTypes.put(Triple.of(PROSERVICE_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE,
new HashSet<>()), CFPS);
pricingTypes.put(Triple.of(SAAS_PRODUCT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED, new HashSet<>()), CCPFPS);
pricingTypes.put(Triple.of(DATA_PRODUCT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED, new HashSet<>()), CCPFPS);
pricingTypes.put(Triple.of(AIQ_PRODUCT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED, new HashSet<>()), CCPFPS);
pricingTypes.put(Triple.of(PROSERVICE_PRODUCT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE_AND_USAGE_BASED, new HashSet<>()), CCPFPS);
pricingTypes.put(Triple.of(AMI_PRODUCT, BYOL_PRICING, new HashSet<>()), BYOL);
pricingTypes.put(Triple.of(SAAS_PRODUCT, BYOL_PRICING, new HashSet<>()), BYOL);
pricingTypes.put(Triple.of(PROSERVICE_PRODUCT, BYOL_PRICING, new HashSet<>()),
BYOL);
pricingTypes.put(Triple.of(AIQ_PRODUCT, BYOL_PRICING, new HashSet<>()), BYOL);
pricingTypes.put(Triple.of(ML_PRODUCT, BYOL_PRICING, new HashSet<>()), BYOL);
pricingTypes.put(Triple.of(CONTAINER_PRODUCT, BYOL_PRICING, new HashSet<>()),
BYOL);
pricingTypes.put(Triple.of(DATA_PRODUCT, BYOL_PRICING, new HashSet<>()), BYOL);
pricingTypes.put(Triple.of(CONTAINER_PRODUCT, LEGAL, new HashSet<>()), FREE);

```



```

 pricingTypes.put(Triple.of(AMI_PRODUCT, FREE_TRIAL_AND_USAGE_BASED, new
HashSet<>()), FTH);
 pricingTypes.put(Triple.of(CONTAINER_PRODUCT, FREE_TRIAL_AND_USAGE_BASED, new
HashSet<>()), FTH);
 pricingTypes.put(Triple.of(ML_PRODUCT, FREE_TRIAL_AND_USAGE_BASED, new
HashSet<>()), FTH);

 Triple<String, Set<String>, Set<String>> key = Triple.of(productType,
agreementTermType, offerType);

 if (pricingTypes.containsKey(key)) {
 return pricingTypes.get(key);
 } else {
 return "Unknown";
 }
}

/*
 * Given product type and agreement term types, some combinations need to check
offer term types as well.
 */
public static String needToCheckOfferTermsType(String productType, Set<String>
agreementTermTypes) {
 Map<KeyPair, String> offerTermTypes = new HashMap<>();
 offerTermTypes.put(new KeyPair(CONTAINER_PRODUCT, CONFIGURABLE_UPFRONT), "Y");
 offerTermTypes.put(new KeyPair(AMI_PRODUCT, CONFIGURABLE_UPFRONT), "Y");
 offerTermTypes.put(new KeyPair(CONTAINER_PRODUCT,
FIXED_UPFRONT_AND_PAYMENT_SCHEDULE), "Y");
 offerTermTypes.put(new KeyPair(AMI_PRODUCT, FIXED_UPFRONT_AND_PAYMENT_SCHEDULE),
"Y");

 KeyPair key = new KeyPair(productType, agreementTermTypes);
 if (offerTermTypes.containsKey(key)) {
 return offerTermTypes.get(key);
 } else {
 return null;
 }
}

public static List<AgreementViewSummary> getAgreementsById() {

 List<AgreementViewSummary> agreementSummaryList = new
ArrayList<AgreementViewSummary>();

```

```

 Filter partyType =
 Filter.builder().name(PARTY_TYPE_FILTER_NAME).values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build()

 Filter agreementType =
 Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME).values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASER).build()

 Filter customizeFilter =
 Filter.builder().name(FILTER_NAME).values(FILTER_VALUE).build();

 SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(partyType, agreementType, customizeFilter).build();

 SearchAgreementsResponse searchResultResponse =
 marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

 agreementSummaryList.addAll(searchResultResponse.agreementViewSummaries());

 while (searchResultResponse.nextToken() != null &&
searchResultResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder().catalog(AWS_MP_CATALOG)
 .filters(partyType,
agreementType).nextToken(searchResultResponse.nextToken()).build();
 searchResultResponse =
 marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchResultResponse.agreementViewSummaries());
 }
 return agreementSummaryList;
}

static class KeyPair {
 private final String first;
 private final Set<String> second;

 public KeyPair(String productType, Set<String> second) {
 this.first = productType;
 this.second = second;
 }

 @Override
 public int hashCode() {

```

```

 return Objects.hash(first, second);
}

@Override
public boolean equals(Object obj) {
 if (this == obj)
 return true;
 if (obj == null || getClass() != obj.getClass())
 return false;
 KeyPair other = (KeyPair) obj;
 return Objects.equals(first, other.first) && Objects.equals(second,
other.second);
}
}

/*
 * Get all the term types for the offer
 */
public static Set<String> getOfferTermTypes(String offerId) {

 Set<String> offerTermTypes = new HashSet<String>();

 DescribeEntityRequest request =
 DescribeEntityRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .entityId(offerId)
 .build();

 DescribeEntityResponse result = marketplaceCatalogClient.describeEntity(request);

 String details = result.details();

 try {
 ObjectMapper objectMapper = new ObjectMapper();
 JsonNode rootNode = objectMapper.readTree(details);
 JsonNode termsNode = rootNode.get(ATTRIBUTE_TERMS);

 for (JsonNode termNode : termsNode) {
 if (termNode.get(ATTRIBUTE_TYPE_ENTITY) != null) {
 offerTermTypes.add(termNode.get(ATTRIBUTE_TYPE_ENTITY).asText());
 }
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
}

```

```

 }

 return offerTermTypes;

}

/*
 * Get all the agreement term types
 */
public static Set<String> getAgreementTermTypes(GetAgreementTermsResponse
agreementTerm) {
 Set<String> agreementTermTypes = new HashSet<String>();
 try {
 for (AcceptedTerm term : agreementTerm.acceptedTerms()) {
 ObjectMapper objectMapper = new ObjectMapper();
 JsonNode termNode = objectMapper.readTree(getJson(term));
 Iterator<Map.Entry<String, JsonNode>> fieldsIterator = termNode.fields();
 while (fieldsIterator.hasNext()) {
 Map.Entry<String, JsonNode> entry = fieldsIterator.next();
 JsonNode value = entry.getValue();
 if (value.isObject() && value.has(ATTRIBUTE_TYPE_AGREEMENT)) {
 agreementTermTypes.add(value.get(ATTRIBUTE_TYPE_AGREEMENT).asText());
 }
 }
 }
 } catch (Exception e) {
 e.printStackTrace();
 }
 return agreementTermTypes;
}

/*
 * make sure all elements in array2 exist in array1
 */
public static boolean allElementsExist(Set<String> array1, Set<String> array2) {
 for (String element : array2) {
 boolean found = false;
 for (String str : array1) {
 if (element.equals(str)) {
 found = true;
 break;
 }
 }
 }
}

```

```

 if (!found) {
 return false;
 }
}
return true;
}

/*
 * Find the combinations of the agreement term types for the agreement
 */
public static Set<String> getMatchedTermTypesCombination(Set<String>
agreementTermTypes) {
 Set<String> matchedCombination = new HashSet<String>();
 for (Set<String> element : ALL_AGREEMENT_TERM_TYPES_COMBINATION) {
 if (allElementsExist(agreementTermTypes, element)) {
 matchedCombination = element;
 }
 }
 return matchedCombination;
}

public static void main(String[] args) {

 List<AgreementViewSummary> agreements = getAgreementsById();

 for (AgreementViewSummary summary : agreements) {
 String pricingType = "";
 String agreementId = summary.agreementId();
 System.out.println(agreementId);
 String offerId = summary.proposalSummary().offerId();

 //get all pricing term types for the offer in the agreement
 Set<String> offerTermTypes = getOfferTermTypes(offerId);
 String productType = summary.proposalSummary().resources().get(0).type();

 //get all pricing term types for the agreement
 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();
 GetAgreementTermsResponse getAgreementTermsResponse =
marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);
 Set<String> agreementTermTypes =
getAgreementTermTypes(getAgreementTermsResponse);
 }
}

```

```

 //get matched pricing term type combination set
 Set<String> agreementMatchedTermType =
 getMatchedTermTypesCombination(agreementTermTypes);

 //check to see if this agreement pricing term combination needs additional check
 on offer pricing terms
 String needToCheckOfferType = needToCheckOfferTermsType(productType,
 agreementMatchedTermType);

 // get the pricing type for the agreement based on the product type, agreement
 term types and offer term types if needed
 if (needToCheckOfferType != null) {
 Set<String> offerMatchedTermType =
 getMatchedTermTypesCombination(offerTermTypes);
 pricingType = getPricingType(productType, agreementMatchedTermType,
 offerMatchedTermType);
 } else if (agreementMatchedTermType == LEGAL) {
 pricingType = FREE;
 } else {
 pricingType = getPricingType(productType, agreementMatchedTermType, new
 HashSet());
 }
 System.out.println("Pricing type is " + pricingType);
}

private static String getJson(Object result) {
 String json = "";

 try {
 ObjectMapper om = new ObjectMapper();
 om.setVisibility(PropertyAccessor.FIELD, Visibility.ANY);
 om.registerModule(new JavaTimeModule());
 ObjectWriter ow = om.writer().withDefaultPrettyPrinter();

 json = ow.writeValueAsString(result);
 } catch (JsonProcessingException e) {
 e.printStackTrace();
 }
 return json;
}
}

```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get the product type of an agreement

The following code example shows how to get the product type of an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;
import software.amazon.awssdk.services.marketplaceagreement.model.Resource;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;

import java.util.ArrayList;
import java.util.List;

import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementProductType {

 /*
 * Obtain the Product Type of the product the agreement was created on
 */
}
```

```
*/
public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<String> productIds = getProducts(agreementId);

 ReferenceCodesUtils.formatOutput(productIds);
}

public static List<String> getProducts(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

 DescribeAgreementResponse describeAgreementResponse =
 marketplaceAgreementClient.describeAgreement(describeAgreementRequest);

 List<String> productIds = new ArrayList<String>();
 for (Resource resource : describeAgreementResponse.proposalSummary().resources())
 {
 productIds.add(resource.id() + ":" + resource.type());
 }
 return productIds;
}
}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get the status of an agreement

The following code example shows how to get the status of an agreement.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.DescribeAgreementResponse;

public class GetAgreementStatus {

 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 DescribeAgreementResponse describeAgreementResponse =
 getDescribeAgreementResponse(agreementId);

 System.out.println("Agreement status is " + describeAgreementResponse.status());

 }

 public static DescribeAgreementResponse getDescribeAgreementResponse(String
agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
```

```
.build();

DescribeAgreementRequest describeAgreementRequest =
 DescribeAgreementRequest.builder()
 .agreementId(agreementId)
 .build();

DescribeAgreementResponse describeAgreementResponse =
 marketplaceAgreementClient.describeAgreement(describeAgreementRequest);
return describeAgreementResponse;
}

}
```

- For API details, see [DescribeAgreement](#) in *AWS SDK for Java 2.x API Reference*.

## Get the support terms of an agreement

The following code example shows how to get the support terms of an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import software.amazon.awssdk.services.marketplaceagreement.model.AcceptedTerm;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;
```

```
import software.amazon.awssdk.services.marketplaceagreement.model.SupportTerm;

import java.util.ArrayList;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.AGREEMENT_ID;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class GetAgreementTermsSupportTerm {

 /*
 * Obtain the support and refund policy I have provided to the customer
 */
 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;

 List<SupportTerm> supportTerms = getSupportTerms(agreementId);

 ReferenceCodesUtils.formatOutput(supportTerms);
 }

 public static List<SupportTerm> getSupportTerms(String agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder().agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);

 List<SupportTerm> supportTerms = new ArrayList<>();

 for (AcceptedTerm acceptedTerm : getAgreementTermsResponse.acceptedTerms()) {
 if (acceptedTerm.supportTerm() != null) {
 supportTerms.add(acceptedTerm.supportTerm());
 }
 }

 return supportTerms;
 }
}
```

```
}

}
```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Get the terms of an agreement

The following code example shows how to get the terms of an agreement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.GetAgreementTermsResponse;

public class GetAgreementTerms {

 public static void main(String[] args) {

 String agreementId = args.length > 0 ? args[0] : AGREEMENT_ID;
```

```
GetAgreementTermsResponse getAgreementTermsResponse =
getAgreementTermsResponse(agreementId);

ReferenceCodesUtils.formatOutput(getAgreementTermsResponse);

}

public static GetAgreementTermsResponse getAgreementTermsResponse(String
agreementId) {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 GetAgreementTermsRequest getAgreementTermsRequest =
 GetAgreementTermsRequest.builder()
 .agreementId(agreementId)
 .build();

 GetAgreementTermsResponse getAgreementTermsResponse =
 marketplaceAgreementClient.getAgreementTerms(getAgreementTermsRequest);
 return getAgreementTermsResponse;
}

}
```

- For API details, see [GetAgreementTerms](#) in *AWS SDK for Java 2.x API Reference*.

## Search for agreements by end date

The following code example shows how to search for agreements by end date.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

public class SearchAgreementsByEndDate {

 static String beforeOrAfterEndtimeFilterName =
 BeforeOrAfterEndTimeFilterName.BeforeEndTime.name();

 static String cutoffDate = "2050-11-18T00:00:00Z";

 static String partyTypeFilterValue = PARTY_TYPE_FILTER_VALUE_PROPOSER;

 public static void main(String[] args) {

 List<AgreementViewSummary> agreementSummaryList = getAgreements();

 ReferenceCodesUtils.formatOutput(agreementSummaryList);
 }

 public static List<AgreementViewSummary> getAgreements() {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
```

```

 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

// set up filters

Filter partyTypeFilter = Filter.builder().name(PARTY_TYPE_FILTER_NAME)
 .values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build();

Filter agreementTypeFilter = Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME)
 .values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASEAGREEMENT).build();

Filter customizeFilter =
Filter.builder().name(beforeOrAfterEndtimeFilterName).values(cutoffDate).build();

List<Filter> filters = new ArrayList<Filter>();

filters.addAll(Arrays.asList(partyTypeFilter, agreementTypeFilter,
customizeFilter));

// search agreement with filters

SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .build();

SearchAgreementsResponse searchAgreementResponse=
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

List<AgreementViewSummary> agreementSummaryList = new
ArrayList<AgreementViewSummary>();

agreementSummaryList.addAll(searchAgreementResponse.agreementViewSummaries());

while (searchAgreementResponse.nextToken() != null &&
searchAgreementResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .nextToken(searchAgreementResponse.nextToken())
 .build();

```

```
searchAgreementResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchAgreementResponse.agreementViewSummaries());
}
return agreementSummaryList;
}
}
```

- For API details, see [SearchAgreements](#) in *AWS SDK for Java 2.x API Reference*.

## Search for agreements with one custom filter

The following code example shows how to search for agreements with one custom filter.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
```



```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import com.example.awsmarketplace.utils.ReferenceCodesUtils;

/**
 * To search by
 * offer id: OfferId;
 * product id: ResourceIdentifier;
 * customer AWS account id: AcceptorAccountId
 * product type: ResourceType (i.e. SaaSProduct)
 * status: Status. status values can be: ACTIVE, CANCELED,
 * EXPIRED, RENEWED, REPLACED, ROLLED_BACK, SUPERSEDED, TERMINATED
 */

public class SearchAgreementsByOneFilter {

 private static final String FILTER_NAME = "ResourceType";

 private static final String FILTER_VALUE = "SaaSProduct";

 /**
 * search agreements by one customize filter
 */
 public static void main(String[] args) {

 List<AgreementViewSummary> agreementSummaryList = getAgreements();

 ReferenceCodesUtils.formatOutput(agreementSummaryList);
 }

 public static List<AgreementViewSummary> getAgreements() {
 MarketplaceAgreementClient marketplaceAgreementClient =
 MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 Filter partyTypeFilter = Filter.builder().name(PARTY_TYPE_FILTER_NAME)
 .values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build();

 Filter agreementTypeFilter = Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME)
 .values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASEAGREEMENT).build();
 }

```

```

 Filter customizeFilter =
Filter.builder().name(FILTER_NAME).values(FILTER_VALUE).build();

 List<Filter> filters = new ArrayList<Filter>();

 filters.addAll(Arrays.asList(partyTypeFilter, agreementTypeFilter,
customizeFilter));

 SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .build();
 SearchAgreementsResponse searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

 List<AgreementViewSummary> agreementSummaryList = new
ArrayList<AgreementViewSummary>();

 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());

 while (searchAgreementsResponse.nextToken() != null &&
searchAgreementsResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .nextToken(searchAgreementsResponse.nextToken())
 .build();
 searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());
 }
 return agreementSummaryList;
}
}

```

- For API details, see [SearchAgreements](#) in *AWS SDK for Java 2.x API Reference*.

## Search for agreements with two custom filters

The following code example shows how to search for agreements with two custom filters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Marketplace API Reference Code Library](#) repository.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package com.example.awsmarketplace.agreementapi;

import software.amazon.awssdk.auth.credentials.AwsCredentialsProvider;
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import
 software.amazon.awssdk.services.marketplaceagreement.MarketplaceAgreementClient;
import
 software.amazon.awssdk.services.marketplaceagreement.model.AgreementViewSummary;
import software.amazon.awssdk.services.marketplaceagreement.model.Filter;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsRequest;
import
 software.amazon.awssdk.services.marketplaceagreement.model.SearchAgreementsResponse;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import static com.example.awsmarketplace.utils.ReferenceCodesConstants.*;
import com.example.awsmarketplace.utils.ReferenceCodesUtils;

/**
 * Party Type = Proposer AND Acceptor:
 * AfterEndTime
 * BeforeEndTime
 * ResourceIdentifier + BeforeEndTime
 * ResourceIdentifier + AfterEndTime
 */
```

```

* ResourceType + BeforeEndTime
* ResourceType + AfterEndTime
*
* Party Type = Proposer
* ResourceIdentifier
* OfferId
* AcceptorAccountId
* Status (ACTIVE)
* Status (ACTIVE) + ResourceIdentifier
* Status (ACTIVE) + AcceptorAccountId
* Status (ACTIVE) + OfferId
* Status (ACTIVE) + ResourceType
* AcceptorAccountId + BeforeEndTime
* AcceptorAccountId + AfterEndTime
* AcceptorAccountId + AfterEndTime
* OfferId + BeforeEndTime
*
* Status values can be: ACTIVE, CANCELLED, EXPIRED, RENEWED, REPLACED, ROLLED_BACK,
SUPERSEDED, TERMINATED
*/

public class SearchAgreementsByTwoFilters {

 public static final String FILTER_1_NAME = "ResourceType";

 public static final String FILTER_1_VALUE = "SaaSProduct";

 public static final String FILTER_2_NAME = "Status";

 public static final String FILTER_2_VALUE = "ACTIVE";

 /*
 * search agreements by two customize filter
 */
 public static void main(String[] args) {

 List<AgreementViewSummary> agreementSummaryList = getAgreements();

 ReferenceCodesUtils.formatOutput(agreementSummaryList);

 }

 public static List<AgreementViewSummary> getAgreements() {
 MarketplaceAgreementClient marketplaceAgreementClient =

```

```
MarketplaceAgreementClient.builder()
 .httpClient(ApacheHttpClient.builder().build())
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

Filter partyTypeFilter = Filter.builder().name(PARTY_TYPE_FILTER_NAME)
 .values(PARTY_TYPE_FILTER_VALUE_PROPOSER).build();

Filter agreementTypeFilter = Filter.builder().name(AGREEMENT_TYPE_FILTER_NAME)
 .values(AGREEMENT_TYPE_FILTER_VALUE_PURCHASEAGREEMENT).build();

Filter customizeFilter1 =
Filter.builder().name(FILTER_1_NAME).values(FILTER_1_VALUE).build();

Filter customizeFilter2 =
Filter.builder().name(FILTER_2_NAME).values(FILTER_2_VALUE).build();

List<Filter> filters = new ArrayList<Filter>();

filters.addAll(Arrays.asList(partyTypeFilter, agreementTypeFilter,
customizeFilter1, customizeFilter2));

SearchAgreementsRequest searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .build();

SearchAgreementsResponse searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);

List<AgreementViewSummary> agreementSummaryList = new
ArrayList<AgreementViewSummary>();

agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());

while (searchAgreementsResponse.nextToken() != null &&
searchAgreementsResponse.nextToken().length() > 0) {
 searchAgreementsRequest =
 SearchAgreementsRequest.builder()
 .catalog(AWS_MP_CATALOG)
 .filters(filters)
 .nextToken(searchAgreementsResponse.nextToken())
```

```
 .build();
 searchAgreementsResponse =
marketplaceAgreementClient.searchAgreements(searchAgreementsRequest);
 agreementSummaryList.addAll(searchAgreementsResponse.agreementViewSummaries());
 }
 return agreementSummaryList;
}
}
```

- For API details, see [SearchAgreements](#) in *AWS SDK for Java 2.x API Reference*.

## MediaConvert examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with MediaConvert.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### CreateJob

The following code example shows how to use CreateJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.mediaconvert;

import java.net.URI;
import java.util.HashMap;
import java.util.Map;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediaconvert.MediaConvertClient;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsResponse;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsRequest;
import software.amazon.awssdk.services.mediaconvert.model.Output;
import software.amazon.awssdk.services.mediaconvert.model.MediaConvertException;
import software.amazon.awssdk.services.mediaconvert.model.OutputGroup;
import software.amazon.awssdk.services.mediaconvert.model.OutputGroupSettings;
import software.amazon.awssdk.services.mediaconvert.model.HlsGroupSettings;
import software.amazon.awssdk.services.mediaconvert.model.OutputGroupType;
import software.amazon.awssdk.services.mediaconvert.model.HlsDirectoryStructure;
import software.amazon.awssdk.services.mediaconvert.model.HlsManifestDurationFormat;
import software.amazon.awssdk.services.mediaconvert.model.HlsStreamInfResolution;
import software.amazon.awssdk.services.mediaconvert.model.HlsClientCache;
import software.amazon.awssdk.services.mediaconvert.model.HlsCaptionLanguageSetting;
import software.amazon.awssdk.services.mediaconvert.model.HlsManifestCompression;
import software.amazon.awssdk.services.mediaconvert.model.HlsCodecSpecification;
import software.amazon.awssdk.services.mediaconvert.model.HlsOutputSelection;
import software.amazon.awssdk.services.mediaconvert.model.HlsProgramDateTime;
import software.amazon.awssdk.services.mediaconvert.model.HlsTimedMetadataId3Frame;
import software.amazon.awssdk.services.mediaconvert.model.HlsSegmentControl;
import software.amazon.awssdk.services.mediaconvert.model.FileGroupSettings;
import software.amazon.awssdk.services.mediaconvert.model.ContainerSettings;
import software.amazon.awssdk.services.mediaconvert.model.VideoDescription;
import software.amazon.awssdk.services.mediaconvert.model.ContainerType;
import software.amazon.awssdk.services.mediaconvert.model.ScalingBehavior;
import software.amazon.awssdk.services.mediaconvert.model.VideoTimecodeInsertion;
import software.amazon.awssdk.services.mediaconvert.model.ColorMetadata;
import software.amazon.awssdk.services.mediaconvert.model.RespondToAfd;
import software.amazon.awssdk.services.mediaconvert.model.AfdSignaling;
import software.amazon.awssdk.services.mediaconvert.model.DropFrameTimecode;
import software.amazon.awssdk.services.mediaconvert.model.VideoCodecSettings;
import software.amazon.awssdk.services.mediaconvert.model.H264Settings;
import software.amazon.awssdk.services.mediaconvert.model.VideoCodec;
import software.amazon.awssdk.services.mediaconvert.model.CreateJobRequest;
import software.amazon.awssdk.services.mediaconvert.model.H264RateControlMode;
import software.amazon.awssdk.services.mediaconvert.model.H264QualityTuningLevel;
import software.amazon.awssdk.services.mediaconvert.model.H264SceneChangeDetect;
```

```
import
 software.amazon.awssdk.services.mediaconvert.model.AacAudioDescriptionBroadcasterMix;
import software.amazon.awssdk.services.mediaconvert.model.H264ParControl;
import software.amazon.awssdk.services.mediaconvert.model.AacRawFormat;
import software.amazon.awssdk.services.mediaconvert.model.H264QvbrSettings;
import
 software.amazon.awssdk.services.mediaconvert.model.H264FramerateConversionAlgorithm;
import software.amazon.awssdk.services.mediaconvert.model.H264CodecLevel;
import software.amazon.awssdk.services.mediaconvert.model.H264FramerateControl;
import software.amazon.awssdk.services.mediaconvert.model.AacCodingMode;
import software.amazon.awssdk.services.mediaconvert.model.H264Telecine;
import
 software.amazon.awssdk.services.mediaconvert.model.H264FlickerAdaptiveQuantization;
import software.amazon.awssdk.services.mediaconvert.model.H264GopSizeUnits;
import software.amazon.awssdk.services.mediaconvert.model.H264CodecProfile;
import software.amazon.awssdk.services.mediaconvert.model.H264GopBReference;
import software.amazon.awssdk.services.mediaconvert.model.AudioTypeControl;
import software.amazon.awssdk.services.mediaconvert.model.AntiAlias;
import software.amazon.awssdk.services.mediaconvert.model.H264SlowPal;
import
 software.amazon.awssdk.services.mediaconvert.model.H264SpatialAdaptiveQuantization;
import software.amazon.awssdk.services.mediaconvert.model.H264Syntax;
import software.amazon.awssdk.services.mediaconvert.model.M3u8Settings;
import software.amazon.awssdk.services.mediaconvert.model.InputDenoiseFilter;
import
 software.amazon.awssdk.services.mediaconvert.model.H264TemporalAdaptiveQuantization;
import software.amazon.awssdk.services.mediaconvert.model.CreateJobResponse;
import
 software.amazon.awssdk.services.mediaconvert.model.H264UnregisteredSeiTimecode;
import software.amazon.awssdk.services.mediaconvert.model.H264EntropyEncoding;
import software.amazon.awssdk.services.mediaconvert.model.InputPsiControl;
import software.amazon.awssdk.services.mediaconvert.model.ColorSpace;
import software.amazon.awssdk.services.mediaconvert.model.H264RepeatPps;
import software.amazon.awssdk.services.mediaconvert.model.H264FieldEncoding;
import software.amazon.awssdk.services.mediaconvert.model.M3u8NielsenId3;
import software.amazon.awssdk.services.mediaconvert.model.InputDeblockFilter;
import software.amazon.awssdk.services.mediaconvert.model.InputRotate;
import software.amazon.awssdk.services.mediaconvert.model.H264DynamicSubGop;
import software.amazon.awssdk.services.mediaconvert.model.TimedMetadata;
import software.amazon.awssdk.services.mediaconvert.model.JobSettings;
import software.amazon.awssdk.services.mediaconvert.model.AudioDefaultSelection;
import software.amazon.awssdk.services.mediaconvert.model.VideoSelector;
import software.amazon.awssdk.services.mediaconvert.model.AacSpecification;
import software.amazon.awssdk.services.mediaconvert.model.Input;
```



```

import software.amazon.awssdk.services.mediaconvert.model.OutputSettings;
import software.amazon.awssdk.services.mediaconvert.model.H264AdaptiveQuantization;
import software.amazon.awssdk.services.mediaconvert.model.AudioLanguageCodeControl;
import software.amazon.awssdk.services.mediaconvert.model.InputFilterEnable;
import software.amazon.awssdk.services.mediaconvert.model.AudioDescription;
import software.amazon.awssdk.services.mediaconvert.model.H264InterlaceMode;
import software.amazon.awssdk.services.mediaconvert.model.AudioCodecSettings;
import software.amazon.awssdk.services.mediaconvert.model.AacSettings;
import software.amazon.awssdk.services.mediaconvert.model.AudioCodec;
import software.amazon.awssdk.services.mediaconvert.model.AacRateControlMode;
import software.amazon.awssdk.services.mediaconvert.model.AacCodecProfile;
import software.amazon.awssdk.services.mediaconvert.model.HlsIFrameOnlyManifest;
import software.amazon.awssdk.services.mediaconvert.model.FrameCaptureSettings;
import software.amazon.awssdk.services.mediaconvert.model.AudioSelector;
import software.amazon.awssdk.services.mediaconvert.model.M3u8PcrControl;
import software.amazon.awssdk.services.mediaconvert.model.InputTimecodeSource;
import software.amazon.awssdk.services.mediaconvert.model.HlsSettings;
import software.amazon.awssdk.services.mediaconvert.model.M3u8Scte35Source;

/**
 * Create a MediaConvert job. Must supply MediaConvert access role Amazon
 * Resource Name (ARN), and a
 * valid video input file via Amazon S3 URL.
 *
 * Also, set up your development environment, including your credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 */
public class CreateJob {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <mcRoleARN> <fileInput>\s

 Where:
 mcRoleARN - The MediaConvert Role ARN.\s
 fileInput - The URL of an Amazon S3 bucket
 where the input file is located.\s
 """;

```

```

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String mcRoleARN = args[0];
 String fileInput = args[1];
 Region region = Region.US_WEST_2;
 MediaConvertClient mc = MediaConvertClient.builder()
 .region(region)
 .build();

 String id = createMediaJob(mc, mcRoleARN, fileInput);
 System.out.println("MediaConvert job created. Job Id = " + id);
 mc.close();
 }

 public static String createMediaJob(MediaConvertClient mc, String mcRoleARN,
 String fileInput) {

 String s3path = fileInput.substring(0, fileInput.lastIndexOf('/') +
1) + "javasdk/out/";
 String fileOutput = s3path + "index";
 String thumbsOutput = s3path + "thumbs/";
 String mp4Output = s3path + "mp4/";

 try {
 System.out.println("MediaConvert role arn: " + mcRoleARN);
 System.out.println("MediaConvert input file: " + fileInput);
 System.out.println("MediaConvert output path: " + s3path);

 // output group Preset HLS low profile
 Output hlsLow = createOutput("hls_low", "_low", "_dt",
750000, 7, 1920, 1080, 640);
 // output group Preset HLS media profile
 Output hlsMedium = createOutput("hls_medium", "_medium", "_
dt", 1200000, 7, 1920, 1080, 1280);
 // output group Preset HLS high profole
 Output hlsHigh = createOutput("hls_high", "_high", "_dt",
3500000, 8, 1920, 1080, 1920);

 OutputGroup appleHLS = OutputGroup.builder().name("Apple
HLS").customName("Example")

```

```
.outputGroupSettings(OutputGroupSettings.builder()

.type(OutputGroupType.HLS_GROUP_SETTINGS)

.hlsGroupSettings(HlsGroupSettings.builder()

.directoryStructure(

 HlsDirectoryStructure.SINGLE_DIRECTORY)

.manifestDurationFormat(

 HlsManifestDurationFormat.INTEGER)

.streamInfResolution(

 HlsStreamInfResolution.INCLUDE)

.clientCache(HlsClientCache.ENABLED)

.captionLanguageSetting(

 HlsCaptionLanguageSetting.OMIT)

.manifestCompression(

 HlsManifestCompression.NONE)

.codecSpecification(

 HlsCodecSpecification.RFC_4281)

.outputSelection(

 HlsOutputSelection.MANIFESTS_AND_SEGMENTS)

.programDateTime(HlsProgramDateTime.EXCLUDE)

.programDateTimePeriod(600)

.timedMetadataId3Frame(

 HlsTimedMetadataId3Frame.PRIV)
```

```

.timedMetadataId3Period(10)

.destination(fileOutput)

.segmentControl(HlsSegmentControl.SEGMENTED_FILES)

.minFinalSegmentLength((double) 0)

.segmentLength(4).minSegmentLength(0).build()
 .build()
 .outputs(hlsLow, hlsMedium,
hlsHigh).build();

 OutputGroup fileMp4 = OutputGroup.builder().name("File
Group").customName("mp4")

.outputGroupSettings(OutputGroupSettings.builder()

.type(OutputGroupType.FILE_GROUP_SETTINGS)

.fileGroupSettings(FileGroupSettings.builder()

.destination(mp4Output).build()
 .build()
 .outputs(Output.builder().extension("mp4")

.containerSettings(ContainerSettings.builder()

.container(ContainerType.MP4).build())

.videoDescription(VideoDescription.builder().width(1280)
 .height(720)

.scalingBehavior(ScalingBehavior.DEFAULT)

.sharpness(50).antiAlias(AntiAlias.ENABLED)

.timecodeInsertion(
 VideoTimecodeInsertion.DISABLED)

.colorMetadata(ColorMetadata.INSERT)

```

```
.respondToAfd(RespondToAfd.NONE)

.afdSignaling(AfdSignaling.NONE)

.dropFrameTimecode(DropFrameTimecode.ENABLED)

.codecSettings(VideoCodecSettings.builder()

 .codec(VideoCodec.H_264)

 .h264Settings(H264Settings

 .builder()

 .rateControlMode(

 H264RateControlMode.QVBR)

 .parControl(H264ParControl.INITIALIZE_FROM_SOURCE)

 .qualityTuningLevel(

 H264QualityTuningLevel.SINGLE_PASS)

 .qvbrSettings(

 H264QvbrSettings.builder()

 .qvbrQualityLevel(

 8)

 .build())

 .codecLevel(H264CodecLevel.AUTO)

 .codecProfile(H264CodecProfile.MAIN)

 .maxBitrate(2400000)

 .framerateControl(

 H264FramerateControl.INITIALIZE_FROM_SOURCE)
```

```
.gopSize(2.0)

.gopSizeUnits(H264GopSizeUnits.SECONDS)

.numberBFramesBetweenReferenceFrames(

 2)

.gopClosedCadence(

 1)

.gopBReference(H264GopBReference.DISABLED)

.slowPal(H264SlowPal.DISABLED)

.syntax(H264Syntax.DEFAULT)

.numberReferenceFrames(

 3)

.dynamicSubGop(H264DynamicSubGop.STATIC)

.fieldEncoding(H264FieldEncoding.PAFF)

.sceneChangeDetect(

 H264SceneChangeDetect.ENABLED)

.minIInterval(0)

.telecine(H264Telecine.NONE)

.framerateConversionAlgorithm(

 H264FramerateConversionAlgorithm.DUPLICATE_DROP)

.entropyEncoding(

 H264EntropyEncoding.CABAC)

.slices(1)
```

```

 .unregisteredSeiTimecode(
 H264UnregisteredSeiTimecode.DISABLED)

 .repeatPps(H264RepeatPps.DISABLED)

 .adaptiveQuantization(
 H264AdaptiveQuantization.HIGH)

 .spatialAdaptiveQuantization(
 H264SpatialAdaptiveQuantization.ENABLED)

 .temporalAdaptiveQuantization(
 H264TemporalAdaptiveQuantization.ENABLED)

 .flickerAdaptiveQuantization(
 H264FlickerAdaptiveQuantization.DISABLED)

 .softness(0)

 .interlaceMode(H264InterlaceMode.PROGRESSIVE)

 .build())

 .build())

 .audioDescriptions(AudioDescription.builder()

 .audioTypeControl(AudioTypeControl.FOLLOW_INPUT)

 .languageCodeControl(
 AudioLanguageCodeControl.FOLLOW_INPUT)

 .codecSettings(AudioCodecSettings.builder()

 .codec(AudioCodec.AAC)

```

```

 .aacSettings(AacSettings
 .builder()
 .codecProfile(AacCodecProfile.LC)
 .rateControlMode(
 AacRateControlMode.CBR)
 .codingMode(AacCodingMode.CODING_MODE_2_0)
 .sampleRate(44100)
 .bitrate(160000)
 .rawFormat(AacRawFormat.NONE)
 .specification(AacSpecification.MPEG4)
 .audioDescriptionBroadcasterMix(
 AacAudioDescriptionBroadcasterMix.NORMAL)
 .build())
 .build())
 .build())
 .build())
 .build();
 OutputGroup thumbs = OutputGroup.builder().name("File
Group").customName("thumbs")
 .outputGroupSettings(OutputGroupSettings.builder()
 .type(OutputGroupType.FILE_GROUP_SETTINGS)
 .fileGroupSettings(FileGroupSettings.builder()
 .destination(thumbsOutput).build())
 .build())
 .outputs(Output.builder().extension("jpg")

```



```
.containerSettings(ContainerSettings.builder()

.container(ContainerType.RAW).build())

.videoDescription(VideoDescription.builder()

.scalingBehavior(ScalingBehavior.DEFAULT)

.sharpness(50).antiAlias(AntiAlias.ENABLED)

.timecodeInsertion(

 VideoTimecodeInsertion.DISABLED)

.colorMetadata(ColorMetadata.INSERT)

.dropFrameTimecode(DropFrameTimecode.ENABLED)

.codecSettings(VideoCodecSettings.builder()

 .codec(VideoCodec.FRAME_CAPTURE)

 .frameCaptureSettings(

 FrameCaptureSettings

 .builder()

 .framerateNumerator(

 1)

 .framerateDenominator(

 1)

 .maxCaptures(10000000)

 .quality(80)

 .build())

 .build())

.build())
```

```

 .build())
 .build())
 .build();

 Map<String, AudioSelector> audioSelectors = new HashMap<>();
 audioSelectors.put("Audio Selector 1",

AudioSelector.builder().defaultSelection(AudioDefaultSelection.DEFAULT)
 .offset(0).build());

 JobSettings jobSettings =
JobSettings.builder().inputs(Input.builder()
 .audioSelectors(audioSelectors)
 .videoSelector(

VideoSelector.builder().colorSpace(ColorSpace.FOLLOW)

.rotate(InputRotate.DEGREE_0).build())

.filterEnable(InputFilterEnable.AUTO).filterStrength(0)
 .deblockFilter(InputDeblockFilter.DISABLED)

.denoiseFilter(InputDenoiseFilter.DISABLED).psiControl(InputPsiControl.USE_PSI)

.timecodeSource(InputTimecodeSource.EMBEDDED).fileInput(fileInput).build())
 .outputGroups(appleHLS, thumbs,
fileMp4).build();

 CreateJobRequest createJobRequest =
CreateJobRequest.builder().role(mcRoleARN)
 .settings(jobSettings)
 .build();

 CreateJobResponse createJobResponse =
mc.createJob(createJobRequest);
 return createJobResponse.job().id();

 } catch (MediaConvertException e) {
 System.out.println(e.toString());
 System.exit(0);
 }
 return "";
}

```

```

 private final static Output createOutput(String customName,
 String nameModifier,
 String segmentModifier,
 int qvbrMaxBitrate,
 int qvbrQualityLevel,
 int originWidth,
 int originHeight,
 int targetWidth) {

 int targetHeight = Math.round(originHeight * targetWidth /
originWidth)
 - (Math.round(originHeight * targetWidth /
originWidth) % 4);
 Output output = null;
 try {
 output =
Output.builder().nameModifier(nameModifier).outputSettings(OutputSettings.builder()

.hlsSettings(HlsSettings.builder().segmentModifier(segmentModifier)

.audioGroupId("program_audio")

.iFrameOnlyManifest(HlsIFrameOnlyManifest.EXCLUDE).build())
 .build())

.containerSettings(ContainerSettings.builder().container(ContainerType.M3_U8)

.m3u8Settings(M3u8Settings.builder().audioFramesPerPes(4)

.pcrControl(M3u8PcrControl.PCR_EVERY_PES_PACKET)

.pmtPid(480).privateMetadataPid(503)

.programNumber(1).patInterval(0).pmtInterval(0)

.scte35Source(M3u8Scte35Source.NONE)

.scte35Pid(500).nielsenId3(M3u8NielsenId3.NONE)

.timedMetadata(TimedMetadata.NONE)

.timedMetadataPid(502).videoPid(481)

.audioPids(482, 483, 484, 485, 486, 487, 488,

```

```

 489, 490, 491, 492)

 .build()

 .build()

 .videoDescription(

VideoDescription.builder().width(targetWidth)

.height(targetHeight)

.scalingBehavior(ScalingBehavior.DEFAULT)

.sharpness(50).antiAlias(AntiAlias.ENABLED)

.timecodeInsertion(

 VideoTimecodeInsertion.DISABLED)

.colorMetadata(ColorMetadata.INSERT)

.respondToAfd(RespondToAfd.NONE)

.afdSignaling(AfdSignaling.NONE)

.dropFrameTimecode(DropFrameTimecode.ENABLED)

.codecSettings(VideoCodecSettings.builder()

 .codec(VideoCodec.H_264)

 .h264Settings(H264Settings

 .builder()

 .rateControlMode(

 H264RateControlMode.QVBR)

 .parControl(H264ParControl.INITIALIZE_FROM_SOURCE)

 .qualityTuningLevel(

 H264QualityTuningLevel.SINGLE_PASS)

```

```
.qvbrSettings(H264QvbrSettings
 .builder()
 .qvbrQualityLevel(
 qvbrQualityLevel)
 .build())
.codecLevel(H264CodecLevel.AUTO)
.codecProfile((targetHeight > 720
 && targetWidth > 1280)
 ? H264CodecProfile.HIGH
 : H264CodecProfile.MAIN)
.maxBitrate(qvbrMaxBitrate)
.framerateControl(
 H264FramerateControl.INITIALIZE_FROM_SOURCE)
.gopSize(2.0)
.gopSizeUnits(H264GopSizeUnits.SECONDS)
.numberBFramesBetweenReferenceFrames(
 2)
.gopClosedCadence(
 1)
.gopBReference(H264GopBReference.DISABLED)
.slowPal(H264SlowPal.DISABLED)
.syntax(H264Syntax.DEFAULT)
```

```
.numberReferenceFrames(
 3)

.dynamicSubGop(H264DynamicSubGop.STATIC)

.fieldEncoding(H264FieldEncoding.PAFF)

.sceneChangeDetect(
 H264SceneChangeDetect.ENABLED)

.minIInterval(0)

.telecine(H264Telecine.NONE)

.framerateConversionAlgorithm(
 H264FramerateConversionAlgorithm.DUPLICATE_DROP)

.entropyEncoding(
 H264EntropyEncoding.CABAC)

.slices(1)

.unregisteredSeiTimecode(
 H264UnregisteredSeiTimecode.DISABLED)

.repeatPps(H264RepeatPps.DISABLED)

.adaptiveQuantization(
 H264AdaptiveQuantization.HIGH)

.spatialAdaptiveQuantization(
 H264SpatialAdaptiveQuantization.ENABLED)

.temporalAdaptiveQuantization(
 H264TemporalAdaptiveQuantization.ENABLED)
```

```

 .flickerAdaptiveQuantization(
 H264FlickerAdaptiveQuantization.DISABLED)

 .softness(0)

 .interlaceMode(H264InterlaceMode.PROGRESSIVE)

 .build())

 .build()

 .build()

 .audioDescriptions(AudioDescription.builder()

 .audioTypeControl(AudioTypeControl.FOLLOW_INPUT)

 .languageCodeControl(AudioLanguageCodeControl.FOLLOW_INPUT)

 .codecSettings(AudioCodecSettings.builder()

 .codec(AudioCodec.AAC).aacSettings(AacSettings

 .builder()

 .codecProfile(AacCodecProfile.LC)

 .rateControlMode(

 AacRateControlMode.CBR)

 .codingMode(AacCodingMode.CODING_MODE_2_0)

 .sampleRate(44100)

 .bitrate(96000)

 .rawFormat(AacRawFormat.NONE)

 .specification(AacSpecification.MPEG4)

 .audioDescriptionBroadcasterMix(

```

```

 AACAudioDescriptionBroadcasterMix.NORMAL)

 .build())

 .build())

 .build();
 } catch (MediaConvertException e) {
 e.printStackTrace();
 System.exit(0);
 }
 return output;
}
}

```

- For API details, see [CreateJob](#) in *AWS SDK for Java 2.x API Reference*.

## GetJob

The following code example shows how to use GetJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsResponse;
import software.amazon.awssdk.services.mediaconvert.model.GetJobRequest;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsRequest;
import software.amazon.awssdk.services.mediaconvert.model.GetJobResponse;
import software.amazon.awssdk.services.mediaconvert.model.MediaConvertException;
import software.amazon.awssdk.services.mediaconvert.MediaConvertClient;
import java.net.URI;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.

```



```
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class GetJob {

 public static void main(String[] args) {

 final String usage = "\n" +
 " <jobId> \n\n" +
 "Where:\n" +
 " jobId - The job id value.\n\n";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String jobId = args[0];
 Region region = Region.US_WEST_2;
 MediaConvertClient mc = MediaConvertClient.builder()
 .region(region)
 .build();

 getSpecificJob(mc, jobId);
 mc.close();
 }

 public static void getSpecificJob(MediaConvertClient mc, String jobId) {
 try {
 GetJobRequest jobRequest = GetJobRequest.builder()
 .id(jobId)
 .build();

 GetJobResponse response = mc.getJob(jobRequest);
 System.out.println("The ARN of the job is " + response.job().arn());

 } catch (MediaConvertException e) {
 System.out.println(e.toString());
 System.exit(0);
 }
 }
}
```

- For API details, see [GetJob](#) in *AWS SDK for Java 2.x API Reference*.

## ListJobs

The following code example shows how to use `ListJobs`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.mediaconvert.MediaConvertClient;
import software.amazon.awssdk.services.mediaconvert.model.ListJobsRequest;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsResponse;
import software.amazon.awssdk.services.mediaconvert.model.DescribeEndpointsRequest;
import software.amazon.awssdk.services.mediaconvert.model.ListJobsResponse;
import software.amazon.awssdk.services.mediaconvert.model.Job;
import software.amazon.awssdk.services.mediaconvert.model.MediaConvertException;
import java.net.URI;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListJobs {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 MediaConvertClient mc = MediaConvertClient.builder()
 .region(region)
 .build();
```

```
 listCompleteJobs(mc);
 mc.close();
 }

 public static void listCompleteJobs(MediaConvertClient mc) {
 try {
 // Create the ListJobsRequest
 ListJobsRequest jobsRequest = ListJobsRequest.builder()
 .maxResults(10)
 .status("COMPLETE")
 .build();

 // Call the listJobs operation
 ListJobsResponse jobsResponse = mc.listJobs(jobsRequest);
 List<Job> jobs = jobsResponse.jobs();
 for (Job job : jobs) {
 System.out.println("The JOB ARN is : " + job.arn());
 }

 } catch (MediaConvertException e) {
 System.out.println(e.toString());
 System.exit(0);
 }
 }
}
```

- For API details, see [ListJobs](#) in *AWS SDK for Java 2.x API Reference*.

## Migration Hub examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Migration Hub.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

# Actions

## DeleteProgressUpdateStream

The following code example shows how to use DeleteProgressUpdateStream.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import
 software.amazon.awssdk.services.migrationhub.model.DeleteProgressUpdateStreamRequest;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteProgressStream {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <progressStream>\s

 Where:
 progressStream - the name of a progress stream to delete.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```

 }

 String progressStream = args[0];
 Region region = Region.US_WEST_2;
 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 deleteStream(migrationClient, progressStream);
 migrationClient.close();
}

public static void deleteStream(MigrationHubClient migrationClient, String
streamName) {
 try {
 DeleteProgressUpdateStreamRequest deleteProgressUpdateStreamRequest =
DeleteProgressUpdateStreamRequest
 .builder()
 .progressUpdateStreamName(streamName)
 .build();

 migrationClient.deleteProgressUpdateStream(deleteProgressUpdateStreamRequest);
 System.out.println(streamName + " is deleted");

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [DeleteProgressUpdateStream](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeApplicationState

The following code example shows how to use DescribeApplicationState.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import
 software.amazon.awssdk.services.migrationhub.model.DescribeApplicationStateRequest;
import
 software.amazon.awssdk.services.migrationhub.model.DescribeApplicationStateResponse;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeAppState {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 DescribeAppState <appId>\s

 Where:
 appId - the application id value.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 Region region = Region.US_WEST_2;
```

```

 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 describeApplicationState(migrationClient, appId);
 migrationClient.close();
 }

 public static void describeApplicationState(MigrationHubClient migrationClient,
 String appId) {
 try {
 DescribeApplicationStateRequest applicationStateRequest =
 DescribeApplicationStateRequest.builder()
 .applicationId(appId)
 .build();

 DescribeApplicationStateResponse applicationStateResponse =
 migrationClient
 .describeApplicationState(applicationStateRequest);
 System.out.println("The application status is " +
 applicationStateResponse.applicationStatusAsString());

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DescribeApplicationState](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeMigrationTask

The following code example shows how to use `DescribeMigrationTask`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import
 software.amazon.awssdk.services.migrationhub.model.DescribeMigrationTaskRequest;
import
 software.amazon.awssdk.services.migrationhub.model.DescribeMigrationTaskResponse;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeMigrationTask {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 DescribeMigrationTask <migrationTask> <progressStream>\s

 Where:
 migrationTask - the name of a migration task.\s
 progressStream - the name of a progress stream.\s
 """;

 if (args.length < 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String migrationTask = args[0];
 String progressStream = args[1];
 Region region = Region.US_WEST_2;
 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 describeMigTask(migrationClient, migrationTask, progressStream);
 migrationClient.close();
 }

```



```

 }

 public static void describeMigTask(MigrationHubClient migrationClient, String
migrationTask,
 String progressStream) {
 try {
 DescribeMigrationTaskRequest migrationTaskRequestRequest =
DescribeMigrationTaskRequest.builder()
 .progressUpdateStream(progressStream)
 .migrationTaskName(migrationTask)
 .build();

 DescribeMigrationTaskResponse migrationTaskResponse = migrationClient
 .describeMigrationTask(migrationTaskRequestRequest);
 System.out.println("The name is " +
migrationTaskResponse.migrationTask().migrationTaskName());

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DescribeMigrationTask](#) in *AWS SDK for Java 2.x API Reference*.

## ImportMigrationTask

The following code example shows how to use `ImportMigrationTask`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;

```

```

import
 software.amazon.awssdk.services.migrationhub.model.CreateProgressUpdateStreamRequest;
import
 software.amazon.awssdk.services.migrationhub.model.ImportMigrationTaskRequest;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ImportMigrationTask {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <migrationTask> <progressStream>\s

 Where:
 migrationTask - the name of a migration task.\s
 progressStream - the name of a progress stream.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String migrationTask = args[0];
 String progressStream = args[1];
 Region region = Region.US_WEST_2;
 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 importMigrTask(migrationClient, migrationTask, progressStream);
 migrationClient.close();
 }

 public static void importMigrTask(MigrationHubClient migrationClient, String
migrationTask, String progressStream) {

```

```

 try {
 CreateProgressUpdateStreamRequest progressUpdateStreamRequest =
 CreateProgressUpdateStreamRequest.builder()
 .progressUpdateStreamName(progressStream)
 .dryRun(false)
 .build();

 migrationClient.createProgressUpdateStream(progressUpdateStreamRequest);
 ImportMigrationTaskRequest migrationTaskRequest =
 ImportMigrationTaskRequest.builder()
 .migrationTaskName(migrationTask)
 .progressUpdateStream(progressStream)
 .dryRun(false)
 .build();

 migrationClient.importMigrationTask(migrationTaskRequest);

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [ImportMigrationTask](#) in *AWS SDK for Java 2.x API Reference*.

## ListApplications

The following code example shows how to use `ListApplications`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import software.amazon.awssdk.services.migrationhub.model.ApplicationState;

```

```
import
 software.amazon.awssdk.services.migrationhub.model.ListApplicationStatesRequest;
import
 software.amazon.awssdk.services.migrationhub.model.ListApplicationStatesResponse;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListApplications {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 listApps(migrationClient);
 migrationClient.close();
 }

 public static void listApps(MigrationHubClient migrationClient) {
 try {
 ListApplicationStatesRequest applicationStatesRequest =
 ListApplicationStatesRequest.builder()
 .maxResults(10)
 .build();

 ListApplicationStatesResponse response =
 migrationClient.listApplicationStates(applicationStatesRequest);
 List<ApplicationState> apps = response.applicationStateList();
 for (ApplicationState appState : apps) {
 System.out.println("App Id is " + appState.applicationId());
 System.out.println("The status is " +
 appState.applicationStatus().toString());
 }

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 }
 }
}
```

```

 System.exit(1);
 }
}

```

- For API details, see [ListApplications](#) in *AWS SDK for Java 2.x API Reference*.

## ListCreatedArtifacts

The following code example shows how to use `ListCreatedArtifacts`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import software.amazon.awssdk.services.migrationhub.model.CreatedArtifact;
import
 software.amazon.awssdk.services.migrationhub.model.ListCreatedArtifactsRequest;
import
 software.amazon.awssdk.services.migrationhub.model.ListCreatedArtifactsResponse;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;
import java.util.List;

/**
 * To run this Java V2 code example, ensure that you have setup your development
 * environment, including your credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListCreatedArtifacts {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;

```

```

 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 listArtifacts(migrationClient);
 migrationClient.close();
 }

 public static void listArtifacts(MigrationHubClient migrationClient) {
 try {
 ListCreatedArtifactsRequest listCreatedArtifactsRequest =
 ListCreatedArtifactsRequest.builder()
 .maxResults(10)
 .migrationTaskName("SampleApp5")
 .progressUpdateStream("ProgressSteamB")
 .build();

 ListCreatedArtifactsResponse response =
 migrationClient.listCreatedArtifacts(listCreatedArtifactsRequest);
 List<CreatedArtifact> apps = response.createdArtifactList();
 for (CreatedArtifact artifact : apps) {
 System.out.println("APp Id is " + artifact.description());
 System.out.println("The name is " + artifact.name());
 }

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListCreatedArtifacts](#) in *AWS SDK for Java 2.x API Reference*.

## ListMigrationTasks

The following code example shows how to use ListMigrationTasks.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.migrationhub.MigrationHubClient;
import software.amazon.awssdk.services.migrationhub.model.ListMigrationTasksRequest;
import
 software.amazon.awssdk.services.migrationhub.model.ListMigrationTasksResponse;
import software.amazon.awssdk.services.migrationhub.model.MigrationTaskSummary;
import software.amazon.awssdk.services.migrationhub.model.MigrationHubException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListMigrationTasks {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 MigrationHubClient migrationClient = MigrationHubClient.builder()
 .region(region)
 .build();

 listMigrTasks(migrationClient);
 migrationClient.close();
 }

 public static void listMigrTasks(MigrationHubClient migrationClient) {
 try {
 ListMigrationTasksRequest listMigrationTasksRequest =
 ListMigrationTasksRequest.builder()
 .maxResults(10)
 .build();
```

```
 ListMigrationTasksResponse response =
migrationClient.listMigrationTasks(listMigrationTasksRequest);
 List<MigrationTaskSummary> migrationList =
response.migrationTaskSummaryList();
 for (MigrationTaskSummary migration : migrationList) {
 System.out.println("Migration task name is " +
migration.migrationTaskName());
 System.out.println("The Progress update stream is " +
migration.progressUpdateStream());
 }

 } catch (MigrationHubException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListMigrationTasks](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon MSK examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon MSK.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Serverless examples](#)

## Serverless examples

### Invoke a Lambda function from an Amazon MSK trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving records from an Amazon MSK cluster. The function retrieves the MSK payload and logs the record contents.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming an Amazon MSK event with Lambda using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.KafkaEvent;
import com.amazonaws.services.lambda.runtime.events.KafkaEvent.KafkaEventRecord;

import java.util.Base64;
import java.util.Map;

public class Example implements RequestHandler<KafkaEvent, Void> {

 @Override
 public Void handleRequest(KafkaEvent event, Context context) {
 for (Map.Entry<String, java.util.List<KafkaEventRecord>> entry :
event.getRecords().entrySet()) {
 String key = entry.getKey();
 System.out.println("Key: " + key);

 for (KafkaEventRecord record : entry.getValue()) {
 System.out.println("Record: " + record);

 byte[] value = Base64.getDecoder().decode(record.getValue());
 String message = new String(value);
 System.out.println("Message: " + message);
 }
 }

 return null;
 }
}
```

# Neptune examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Neptune.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Neptune

The following code example shows how to get started using Neptune.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
```

```

*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class HelloNeptune {
 public static void main(String[] args) {
 NeptuneAsyncClient neptuneClient = NeptuneAsyncClient.create();
 describeDbCluster(neptuneClient).join(); // This ensures the async code runs
to completion
 }

 /**
 * Describes the Amazon Neptune DB clusters.
 *
 * @param neptuneClient the Neptune asynchronous client used to make the request
 * @return a {@link CompletableFuture} that completes when the operation is
finished
 */
 public static CompletableFuture<Void> describeDbCluster(NeptuneAsyncClient
neptuneClient) {
 DescribeDbClustersRequest request = DescribeDbClustersRequest.builder()
 .maxRecords(20)
 .build();

 SdkPublisher<DescribeDbClustersResponse> paginator =
neptuneClient.describeDBClustersPaginator(request);
 CompletableFuture<Void> future = new CompletableFuture<>();

 paginator.subscribe(new Subscriber<DescribeDbClustersResponse>() {
 private Subscription subscription;

 @Override
 public void onSubscribe(Subscription s) {
 this.subscription = s;
 s.request(Long.MAX_VALUE); // request all items
 }

 @Override
 public void onNext(DescribeDbClustersResponse response) {
 response.dbClusters().forEach(cluster -> {
 System.out.println("Cluster Identifier: " +
cluster.dbClusterIdentifier());
 System.out.println("Status: " + cluster.status());

```

```
 });
 }

 @Override
 public void onError(Throwable t) {
 future.completeExceptionally(t);
 }

 @Override
 public void onComplete() {
 future.complete(null);
 }
});

return future.whenComplete((result, throwable) -> {
 neptuneClient.close();
 if (throwable != null) {
 System.err.println("Error describing DB clusters: " +
throwable.getMessage());
 }
});
}
```

- For API details, see [DescribeDBClustersPaginator](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an Amazon Neptune Subnet Group.
- Create an Neptune Cluster.
- Create an Neptune Instance.
- Check the status of the Neptune Instance.
- Show Neptune cluster details.
- Stop the Neptune cluster.
- Start the Neptune cluster.
- Delete the Neptune Assets.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Neptune features.

```
public class NeptuneScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final Logger logger =
 LoggerFactory.getLogger(NeptuneScenario.class);
 static Scanner scanner = new Scanner(System.in);
 static NeptuneActions neptuneActions = new NeptuneActions();

 public static void main(String[] args) {
 final String usage =
 """
 Usage:
 <subnetGroupName> <clusterName> <dbInstanceId>

 Where:
 subnetGroupName - The name of an existing Neptune DB subnet
group that includes subnets in at least two Availability Zones.
 clusterName - The unique identifier for the Neptune DB
cluster.
 dbInstanceId - The identifier for a specific Neptune DB
instance within the cluster.
 """;
 String subnetGroupName = "neptuneSubnetGroup65";
 String clusterName = "neptuneCluster65";
 String dbInstanceId = "neptuneDB65";

 logger.info("""
 Amazon Neptune is a fully managed graph
 database service by AWS, designed specifically
 for handling complex relationships and connected
 datasets at scale. It supports two popular graph models:
 property graphs (via openCypher and Gremlin) and RDF
 graphs (via SPARQL). This makes Neptune ideal for
 use cases such as knowledge graphs, fraud detection,
```

social networking, recommendation engines, and network management, where relationships between entities are central to the data.

Being fully managed, Neptune handles database provisioning, patching, backups, and replication, while also offering high availability and durability within AWS's infrastructure.

For developers, programming with Neptune allows for building intelligent, relationship-aware applications that go beyond traditional tabular databases. Developers can use the AWS SDK for Java to automate infrastructure operations (via `NeptuneClient`).

Let's get started...

```
 """);
 waitForInputToContinue(scanner);
 runScenario(subnetGroupName, dbInstanceId, clusterName);
}

public static void runScenario(String subnetGroupName, String dbInstanceId,
String clusterName) {
 logger.info(DASHES);
 logger.info("1. Create a Neptune DB Subnet Group");
 logger.info("The Neptune DB subnet group is used when launching a Neptune
cluster");
 waitForInputToContinue(scanner);
 try {
 neptuneActions.createSubnetGroupAsync(subnetGroupName).join();

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 logger.error("The request failed due to service quota exceeded: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);
}
```

```
 logger.info(DASHES);
 logger.info("2. Create a Neptune Cluster");
 logger.info("A Neptune Cluster allows you to store and query highly
connected datasets with low latency.");
 waitForInputToContinue(scanner);
 String dbClusterId;
 try {
 dbClusterId = neptuneActions.createDBClusterAsync(clusterName).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 logger.error("The request failed due to service quota exceeded: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 }
 return;
 }

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Create a Neptune DB Instance");
 logger.info("In this step, we add a new database instance to the Neptune
cluster");
 waitForInputToContinue(scanner);
 try {
 neptuneActions.createDBInstanceAsync(dbInstanceId, dbClusterId).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 logger.error("The request failed due to service quota exceeded: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 }
 return;
}

waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("4. Check the status of the Neptune DB Instance");
```

```
logger.info("""
 In this step, we will wait until the DB instance
 becomes available. This may take around 10 minutes.
 """);
waitForInputToContinue(scanner);
try {
 neptuneActions.checkInstanceStatus(dbInstanceId, "available").join();
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 logger.error("An unexpected error occurred.", cause);
 return;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("5.Show Neptune Cluster details");
waitForInputToContinue(scanner);
try {
 neptuneActions.describeDBClustersAsync(clusterName).join();
} catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The request failed due to the resource not found: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 return;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("6. Stop the Amazon Neptune cluster");
logger.info("""
 Once stopped, this step polls the status
 until the cluster is in a stopped state.
 """);
waitForInputToContinue(scanner);
try {
 neptuneActions.stopDBClusterAsync(dbClusterId);
 neptuneActions.waitForClusterStatus(dbClusterId, "stopped");
} catch (CompletionException ce) {
```



```

 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The request failed due to the resource not found: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 return;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("7. Start the Amazon Neptune cluster");
 logger.info("""
 Once started, this step polls the clusters
 status until it's in an available state.
 We will also poll the instance status.
 """);
 waitForInputToContinue(scanner);
 try {
 neptuneActions.startDBClusterAsync(dbClusterId);
 neptuneActions.waitForClusterStatus(dbClusterId, "available");
 neptuneActions.checkInstanceStatus(dbInstanceId, "available").join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The request failed due to the resource not found: {}",
cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 return;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("8. Delete the Neptune Assets");
 logger.info("Would you like to delete the Neptune Assets? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 logger.info("You selected to delete the Neptune assets.");
 try {

```

```

 neptuneActions.deleteNeptuneResourcesAsync(dbInstanceId,
clusterName, subnetGroupName);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof ResourceNotFoundException) {
 logger.error("The request failed due to the resource not found:
{}", cause.getMessage());
 } else {
 logger.error("An unexpected error occurred.", cause);
 }
 return;
 }
} else {
 logger.info("You selected not to delete Neptune assets.");
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info(
 """
 Thank you for checking out the Amazon Neptune Service Use demo. We
hope you
 learned something new, or got some inspiration for your own apps
today.
 For more AWS code examples, have a look at:
 https://docs.aws.amazon.com/code-library/latest/ug/what-is-code-
library.html
 """);
logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 logger.info("Invalid input. Please try again.");

```

```

 }
 }
}

```

## A wrapper class for Neptune SDK methods.

```

public class NeptuneActions {
 private CompletableFuture<Void> instanceCheckFuture;
 private static NeptuneAsyncClient neptuneAsyncClient;
 private final Region region = Region.US_EAST_1;
 private static final Logger logger =
 LoggerFactory.getLogger(NeptuneActions.class);
 private final NeptuneClient neptuneClient =
 NeptuneClient.builder().region(region).build();

 /**
 * Retrieves an instance of the NeptuneAsyncClient.
 * <p>
 * This method initializes and returns a singleton instance of the
 NeptuneAsyncClient. The client
 * is configured with the following settings:
 *
 * Maximum concurrency: 100
 * Connection timeout: 60 seconds
 * Read timeout: 60 seconds
 * Write timeout: 60 seconds
 * API call timeout: 2 minutes
 * API call attempt timeout: 90 seconds
 * Retry strategy: STANDARD
 *
 * The client is built using the NettyNioAsyncHttpClient.
 *
 * @return the singleton instance of the NeptuneAsyncClient
 */
 private static NeptuneAsyncClient getAsyncClient() {
 if (neptuneAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))

```

```

 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 neptuneAsyncClient = NeptuneAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return neptuneAsyncClient;
}

/**
 * Asynchronously deletes a set of Amazon Neptune resources in a defined order.
 * <p>
 * The method performs the following operations in sequence:
 *
 * Deletes the Neptune DB instance identified by {@code dbInstanceId}.
li>
 * Waits until the DB instance is fully deleted.
 * Deletes the Neptune DB cluster identified by {@code dbClusterId}.
li>
 * Deletes the Neptune DB subnet group identified by {@code
subnetGroupName}.
 *
 * <p>
 * If any step fails, the subsequent operations are not performed, and the
exception
 * is logged. This method blocks the calling thread until all operations
complete.
 *
 * @param dbInstanceId the ID of the Neptune DB instance to delete
 * @param dbClusterId the ID of the Neptune DB cluster to delete
 * @param subnetGroupName the name of the Neptune DB subnet group to delete
 */
public void deleteNeptuneResourcesAsync(String dbInstanceId, String dbClusterId,
String subnetGroupName) {
 deleteDBInstanceAsync(dbInstanceId)
 .thenCompose(v -> waitUntilInstanceDeletedAsync(dbInstanceId))

```

```

 .thenCompose(v -> deleteDBClusterAsync(dbClusterId))
 .thenCompose(v -> deleteDBSubnetGroupAsync(subnetGroupName))
 .whenComplete((v, ex) -> {
 if (ex != null) {
 logger.info("Failed to delete Neptune resources: " +
ex.getMessage());
 } else {
 logger.info("Neptune resources deleted successfully.");
 }
 })
 .join(); // Waits for the entire async chain to complete
 }

 /**
 * Deletes a subnet group.
 *
 * @param subnetGroupName the identifier of the subnet group to delete
 * @return a {@link CompletableFuture} that completes when the cluster has been
deleted
 */
 public CompletableFuture<Void> deleteDBSubnetGroupAsync(String subnetGroupName)
 {
 DeleteDbSubnetGroupRequest request = DeleteDbSubnetGroupRequest.builder()
 .dbSubnetGroupName(subnetGroupName)
 .build();

 return getAsyncClient().deleteDBSubnetGroup(request)
 .thenAccept(response -> logger.info("## Deleting Subnet Group: " +
subnetGroupName));
 }

 /**
 * Deletes a DB instance asynchronously.
 *
 * @param clusterId the identifier of the cluster to delete
 * @return a {@link CompletableFuture} that completes when the cluster has been
deleted
 */
 public CompletableFuture<Void> deleteDBClusterAsync(String clusterId) {
 DeleteDbClusterRequest request = DeleteDbClusterRequest.builder()
 .dbClusterIdentifier(clusterId)
 .skipFinalSnapshot(true)
 .build();
 }

```

```

 return getAsyncClient().deleteDBCluster(request)
 .thenAccept(response -> System.out.println("## Deleting DB Cluster:
" + clusterId));
 }

 public CompletableFuture<Void> waitUntilInstanceDeletedAsync(String instanceId)
 {
 CompletableFuture<Void> future = new CompletableFuture<>();
 long startTime = System.currentTimeMillis();
 checkInstanceDeletedRecursive(instanceId, startTime, future);
 return future;
 }

 /**
 * Deletes a DB instance asynchronously.
 *
 * @param instanceId the identifier of the DB instance to be deleted
 * @return a {@link CompletableFuture} that completes when the DB instance has
 * been deleted
 */
 public CompletableFuture<Void> deleteDBInstanceAsync(String instanceId) {
 DeleteDbInstanceRequest request = DeleteDbInstanceRequest.builder()
 .dbInstanceIdentifier(instanceId)
 .skipFinalSnapshot(true)
 .build();

 return getAsyncClient().deleteDBInstance(request)
 .thenAccept(response -> System.out.println("## Deleting DB Instance:
" + instanceId));
 }

 private void checkInstanceDeletedRecursive(String instanceId, long startTime,
 CompletableFuture<Void> future) {
 DescribeDbInstancesRequest request = DescribeDbInstancesRequest.builder()
 .dbInstanceIdentifier(instanceId)
 .build();

 getAsyncClient().describeDBInstances(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof NeptuneException &&

```

```

 ((NeptuneException)
cause).awsErrorDetails().errorCode().equals("DBInstanceNotFound")) {
 long elapsed = (System.currentTimeMillis() -
startTime) / 1000;
 logger.info("\r Instance %s deleted after %ds%n",
instanceId, elapsed);
 future.complete(null);
 return;
 }
 future.completeExceptionally(new CompletionException("Error
polling DB instance", cause));
 return;
 }

 String status =
response.dbInstances().get(0).dbInstanceStatus();
 long elapsed = (System.currentTimeMillis() - startTime) / 1000;
 System.out.printf("\r Waiting: Instance %s status: %-10s (%ds
elapsed)", instanceId, status, elapsed);
 System.out.flush();

 CompletableFuture.delayedExecutor(20, TimeUnit.SECONDS)
 .execute(() -> checkInstanceDeletedRecursive(instanceId,
startTime, future));
 });
}

public void waitForClusterStatus(String clusterId, String desiredStatus) {
 System.out.printf("Waiting for cluster '%s' to reach status '%s'...\n",
clusterId, desiredStatus);
 CompletableFuture<Void> future = new CompletableFuture<>();
 checkClusterStatusRecursive(clusterId, desiredStatus,
System.currentTimeMillis(), future);
 future.join();
}

private void checkClusterStatusRecursive(String clusterId, String desiredStatus,
long startTime, CompletableFuture<Void> future) {
 DescribeDbClustersRequest request = DescribeDbClustersRequest.builder()
 .dbClusterIdentifier(clusterId)
 .build();

 getAsyncClient().describeDBClusters(request)

```

```

 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 future.completeExceptionally(
 new CompletionException("Error checking Neptune
cluster status", cause)
);
 return;
 }

 List<DBCluster> clusters = response.dbClusters();
 if (clusters.isEmpty()) {
 future.completeExceptionally(new RuntimeException("Cluster
not found: " + clusterId));
 return;
 }

 String currentStatus = clusters.get(0).status();
 long elapsedSeconds = (System.currentTimeMillis() - startTime) /
1000;

 System.out.printf("\r Elapsed: %-20s Cluster status: %-20s",
formatElapsedTime((int) elapsedSeconds), currentStatus);
 System.out.flush();

 if (desiredStatus.equalsIgnoreCase(currentStatus)) {
 System.out.printf("\r Neptune cluster reached desired status
's' after %s.\n", desiredStatus, formatElapsedTime((int) elapsedSeconds));
 future.complete(null);
 } else {
 CompletableFuture.delayedExecutor(20, TimeUnit.SECONDS)
 .execute(() ->
checkClusterStatusRecursive(clusterId, desiredStatus, startTime, future));
 }
 });
 }

 /**
 * Starts an Amazon Neptune DB cluster.
 *
 * @param clusterIdentifier the unique identifier of the DB cluster to be
stopped
 */

```



```

 public CompletableFuture<StartDbClusterResponse> startDBClusterAsync(String
clusterIdentifier) {
 StartDbClusterRequest clusterRequest = StartDbClusterRequest.builder()
 .dbClusterIdentifier(clusterIdentifier)
 .build();

 return getAsyncClient().startDBCluster(clusterRequest)
 .whenComplete((response, error) -> {
 if (error != null) {
 Throwable cause = error.getCause() != null ?
error.getCause() : error;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }

 throw new RuntimeException("Failed to start DB cluster: " +
cause.getMessage(), cause);
 } else {
 logger.info("DB Cluster starting: " + clusterIdentifier);
 }
 });
 }

 /**
 * Stops an Amazon Neptune DB cluster.
 *
 * @param clusterIdentifier the unique identifier of the DB cluster to be
stopped
 */
 public CompletableFuture<StopDbClusterResponse> stopDBClusterAsync(String
clusterIdentifier) {
 StopDbClusterRequest clusterRequest = StopDbClusterRequest.builder()
 .dbClusterIdentifier(clusterIdentifier)
 .build();

 return getAsyncClient().stopDBCluster(clusterRequest)
 .whenComplete((response, error) -> {
 if (error != null) {
 Throwable cause = error.getCause() != null ?
error.getCause() : error;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }
 }
 });
 }

```

```

 }

 throw new RuntimeException("Failed to stop DB cluster: " +
cause.getMessage(), cause);
 } else {
 logger.info("DB Cluster stopped: " + clusterIdentifier);
 }
 });
}

/**
 * Asynchronously describes the specified Amazon RDS DB cluster.
 *
 * @param clusterId the identifier of the DB cluster to describe
 * @return a {@link CompletableFuture} that completes when the operation is
done, or throws a {@link RuntimeException}
 * if an error occurs
 */
public CompletableFuture<Void> describeDBClustersAsync(String clusterId) {
 DescribeDbClustersRequest request = DescribeDbClustersRequest.builder()
 .dbClusterIdentifier(clusterId)
 .build();

 return getAsyncClient().describeDBClusters(request)
 .thenAccept(response -> {
 for (DBCluster cluster : response.dbClusters()) {
 logger.info("Cluster Identifier: " +
cluster.dbClusterIdentifier());
 logger.info("Status: " + cluster.status());
 logger.info("Engine: " + cluster.engine());
 logger.info("Engine Version: " + cluster.engineVersion());
 logger.info("Endpoint: " + cluster.endpoint());
 logger.info("Reader Endpoint: " + cluster.readerEndpoint());
 logger.info("Availability Zones: " +
cluster.availabilityZones());
 logger.info("Subnet Group: " + cluster.dbSubnetGroup());
 logger.info("VPC Security Groups:");
 cluster.vpcSecurityGroups().forEach(vpcGroup ->
 logger.info(" - " +
vpcGroup.vpcSecurityGroupId()));
 logger.info("Storage Encrypted: " +
cluster.storageEncrypted());
 }
 });
}

```

```

 logger.info("IAM DB Auth Enabled: " +
cluster.iamDatabaseAuthenticationEnabled());
 logger.info("Backup Retention Period: " +
cluster.backupRetentionPeriod() + " days");
 logger.info("Preferred Backup Window: " +
cluster.preferredBackupWindow());
 logger.info("Preferred Maintenance Window: " +
cluster.preferredMaintenanceWindow());
 logger.info("-----");
 }
})
.exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }

 throw new RuntimeException("Failed to describe the DB cluster: "
+ cause.getMessage(), cause);
});
}

 public CompletableFuture<Void> checkInstanceStatus(String instanceId, String
desiredStatus) {
 CompletableFuture<Void> future = new CompletableFuture<>();
 long startTime = System.currentTimeMillis();
 checkStatusRecursive(instanceId, desiredStatus.toLowerCase(), startTime,
future);
 return future;
 }

 /**
 * Checks the status of a Neptune instance recursively until the desired status
 is reached or a timeout occurs.
 *
 * @param instanceId the ID of the Neptune instance to check
 * @param desiredStatus the desired status of the Neptune instance
 * @param startTime the start time of the operation, used to calculate the
 elapsed time
 * @param future a {@link CompletableFuture} that will be completed when
 the desired status is reached
 */

```

```

private void checkStatusRecursive(String instanceId, String desiredStatus, long
startTime, CompletableFuture<Void> future) {
 DescribeDbInstancesRequest request = DescribeDbInstancesRequest.builder()
 .dbInstanceIdentifier(instanceId)
 .build();

 getAsyncClient().describeDBInstances(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 future.completeExceptionally(
 new CompletionException("Error checking Neptune
instance status", cause)
);
 return;
 }

 List<DBInstance> instances = response.dbInstances();
 if (instances.isEmpty()) {
 future.completeExceptionally(new RuntimeException("Instance
not found: " + instanceId));
 return;
 }

 String currentStatus = instances.get(0).dbInstanceStatus();
 long elapsedSeconds = (System.currentTimeMillis() - startTime) /
1000;

 System.out.printf("\r Elapsed: %-20s Status: %-20s",
formatElapsedTime((int) elapsedSeconds), currentStatus);
 System.out.flush();

 if (desiredStatus.equalsIgnoreCase(currentStatus)) {
 System.out.printf("\r Neptune instance reached desired
status '%s' after %s.\n", desiredStatus, formatElapsedTime((int) elapsedSeconds));
 future.complete(null);
 } else {
 CompletableFuture.delayedExecutor(20, TimeUnit.SECONDS)
 .execute(() -> checkStatusRecursive(instanceId,
desiredStatus, startTime, future));
 }
 });
}

```

```

private String formatElapsedTime(int seconds) {
 int minutes = seconds / 60;
 int remainingSeconds = seconds % 60;

 if (minutes > 0) {
 return minutes + (minutes == 1 ? " min" : " mins") + ", " +
 remainingSeconds + (remainingSeconds == 1 ? " sec" : " secs");
 } else {
 return remainingSeconds + (remainingSeconds == 1 ? " sec" : " secs");
 }
}

/**
 * Creates a new Amazon Neptune DB instance asynchronously.
 *
 * @param dbInstanceId the identifier for the new DB instance
 * @param dbClusterId the identifier for the DB cluster that the new instance
will be a part of
 * @return a {@link CompletableFuture} that completes with the identifier of the
newly created DB instance
 * @throws CompletionException if the operation fails, with a cause of either:
 *
 * - {@link ServiceQuotaExceededException} if the
request would exceed the maximum quota, or
 *
 * - a general exception with the failure message
 */
public CompletableFuture<String> createDBInstanceAsync(String dbInstanceId,
String dbClusterId) {
 CreateDbInstanceRequest request = CreateDbInstanceRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceId)
 .dbInstanceClass("db.r5.large")
 .engine("neptune")
 .dbClusterIdentifier(dbClusterId)
 .build();

 return getAsyncClient().createDBInstance(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 }
 })
}

```

```

 throw new CompletionException("Failed to create Neptune DB
instance: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> {
 String instanceId =
response.dbInstance().dbInstanceIdIdentifier();
 logger.info("Created Neptune DB Instance: " + instanceId);
 return instanceId;
 });
}

/**
 * Creates a new Amazon Neptune DB cluster asynchronously.
 *
 * @param dbName the name of the DB cluster to be created
 * @return a CompletableFuture that, when completed, provides the ID of the
created DB cluster
 * @throws CompletionException if the operation fails for any reason, including
if the request would exceed the maximum quota
 */
public CompletableFuture<String> createDBClusterAsync(String dbName) {
 CreateDbClusterRequest request = CreateDbClusterRequest.builder()
 .dbClusterIdentifier(dbName)
 .engine("neptune")
 .deletionProtection(false)
 .backupRetentionPeriod(1)
 .build();

 return getAsyncClient().createDBCluster(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create Neptune DB
cluster: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> {
 String clusterId = response.dbCluster().dbClusterIdentifier();

```

```

 logger.info("DB Cluster created: " + clusterId);
 return clusterId;
 });
}

/**
 * Creates a new DB subnet group asynchronously.
 *
 * @param groupName the name of the subnet group to create
 * @return a CompletableFuture that, when completed, returns the Amazon Resource
 * Name (ARN) of the created subnet group
 * @throws CompletionException if the operation fails, with a cause that may be
 * a ServiceQuotaExceededException if the request would exceed the maximum quota
 */
public CompletableFuture<String> createSubnetGroupAsync(String groupName) {

 // Get the Amazon Virtual Private Cloud (VPC) where the Neptune cluster and
 resources will be created
 String vpcId = getDefaultVpcId();
 logger.info("VPC is : " + vpcId);

 List<String> subnetList = getSubnetIds(vpcId);
 for (String subnetId : subnetList) {
 System.out.println("Subnet group:" + subnetId);
 }

 CreateDbSubnetGroupRequest request = CreateDbSubnetGroupRequest.builder()
 .dbSubnetGroupName(groupName)
 .dbSubnetGroupDescription("Subnet group for Neptune cluster")
 .subnetIds(subnetList)
 .build();

 return getAsyncClient().createDBSubnetGroup(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create subnet
group: " + exception.getMessage(), exception);
 }
 })
}

```

```

 })
 .thenApply(response -> {
 String name = response.dbSubnetGroup().dbSubnetGroupName();
 String arn = response.dbSubnetGroup().dbSubnetGroupArn();
 logger.info("Subnet group created: " + name);
 return arn;
 });
 }

 private List<String> getSubnetIds(String vpcId) {
 try (Ec2Client ec2 = Ec2Client.builder().region(region).build()) {
 DescribeSubnetsRequest request = DescribeSubnetsRequest.builder()
 .filters(builder -> builder.name("vpc-id").values(vpcId))
 .build();

 DescribeSubnetsResponse response = ec2.describeSubnets(request);
 return response.subnets().stream()
 .map(Subnet::subnetId)
 .collect(Collectors.toList());
 }
 }

 public static String getDefaultVpcId() {
 Ec2Client ec2 = Ec2Client.builder()
 .region(Region.US_EAST_1)
 .build();

 Filter myFilter = Filter.builder()
 .name("isDefault")
 .values("true")
 .build();

 List<Filter> filterList = new ArrayList<>();
 filterList.add(myFilter);

 DescribeVpcsRequest request = DescribeVpcsRequest.builder()
 .filters(filterList)
 .build();

 DescribeVpcsResponse response = ec2.describeVpcs(request);
 if (!response.vpcs().isEmpty()) {
 Vpc defaultVpc = response.vpcs().get(0);
 return defaultVpc.vpcId();
 }
 }

```



```
 } else {
 throw new RuntimeException("No default VPC found in this region.");
 }
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateDBCluster](#)
- [CreateDBInstance](#)
- [CreateDBSubnetGroup](#)
- [CreateGraph](#)
- [DeleteDBCluster](#)
- [DeleteDBInstance](#)
- [DeleteDBSubnetGroup](#)
- [DescribeDBClusters](#)
- [DescribeDBInstances](#)
- [ExecuteGremlinProfileQuery](#)
- [ExecuteGremlinQuery](#)
- [ExecuteOpenCypherExplainQuery](#)
- [ExecuteQuery](#)
- [StartDBCluster](#)
- [StopDBCluster](#)

## Actions

### CreateDBCluster

The following code example shows how to use CreateDBCluster.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new Amazon Neptune DB cluster asynchronously.
 *
 * @param dbName the name of the DB cluster to be created
 * @return a CompletableFuture that, when completed, provides the ID of the
 * created DB cluster
 * @throws CompletionException if the operation fails for any reason, including
 * if the request would exceed the maximum quota
 */
public CompletableFuture<String> createDBClusterAsync(String dbName) {
 CreateDbClusterRequest request = CreateDbClusterRequest.builder()
 .dbClusterIdentifier(dbName)
 .engine("neptune")
 .deletionProtection(false)
 .backupRetentionPeriod(1)
 .build();

 return getAsyncClient().createDBCluster(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create Neptune DB
cluster: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> {
 String clusterId = response.dbCluster().dbClusterIdentifier();
 logger.info("DB Cluster created: " + clusterId);
 return clusterId;
 });
}
```

```
 });
}
```

- For API details, see [CreateDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBInstance

The following code example shows how to use CreateDBInstance.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates a new Amazon Neptune DB instance asynchronously.
 *
 * @param dbInstanceId the identifier for the new DB instance
 * @param dbClusterId the identifier for the DB cluster that the new instance
will be a part of
 * @return a {@link CompletableFuture} that completes with the identifier of the
newly created DB instance
 * @throws CompletionException if the operation fails, with a cause of either:
 *
 * - {@link ServiceQuotaExceededException} if the
request would exceed the maximum quota, or
 *
 * - a general exception with the failure message
 */
public CompletableFuture<String> createDBInstanceAsync(String dbInstanceId,
String dbClusterId) {
 CreateDbInstanceRequest request = CreateDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceId)
 .dbInstanceClass("db.r5.large")
 .engine("neptune")
 .dbClusterIdentifier(dbClusterId)
 .build();

 return getAsyncClient().createDBInstance(request)
```

```

 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create Neptune DB
instance: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> {
 String instanceId =
response.dbInstance().dbInstanceIdentifier();
 logger.info("Created Neptune DB Instance: " + instanceId);
 return instanceId;
 });
 }
}

```

- For API details, see [CreateDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBSubnetGroup

The following code example shows how to use CreateDBSubnetGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates a new DB subnet group asynchronously.
 *
 * @param groupName the name of the subnet group to create
 * @return a CompletableFuture that, when completed, returns the Amazon Resource
Name (ARN) of the created subnet group

```

```

 * @throws CompletionException if the operation fails, with a cause that may be
 * a ServiceQuotaExceededException if the request would exceed the maximum quota
 */
 public CompletableFuture<String> createSubnetGroupAsync(String groupName) {

 // Get the Amazon Virtual Private Cloud (VPC) where the Neptune cluster and
 // resources will be created
 String vpcId = getDefaultVpcId();
 logger.info("VPC is : " + vpcId);

 List<String> subnetList = getSubnetIds(vpcId);
 for (String subnetId : subnetList) {
 System.out.println("Subnet group:" + subnetId);
 }

 CreateDbSubnetGroupRequest request = CreateDbSubnetGroupRequest.builder()
 .dbSubnetGroupName(groupName)
 .dbSubnetGroupDescription("Subnet group for Neptune cluster")
 .subnetIds(subnetList)
 .build();

 return getAsyncClient().createDBSubnetGroup(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof ServiceQuotaExceededException) {
 throw new CompletionException("The operation was denied
because the request would exceed the maximum quota.", cause);
 }
 throw new CompletionException("Failed to create subnet
group: " + exception.getMessage(), exception);
 }
 })
 .thenApply(response -> {
 String name = response.dbSubnetGroup().dbSubnetGroupName();
 String arn = response.dbSubnetGroup().dbSubnetGroupArn();
 logger.info("Subnet group created: " + name);
 return arn;
 });
 }
}

```

- For API details, see [CreateDBSubnetGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreateGraph

The following code example shows how to use CreateGraph.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Executes the process of creating a new Neptune graph.
 *
 * @param client the Neptune graph client used to interact with the
Neptune service
 * @param graphName the name of the graph to be created
 * @throws NeptuneGraphException if an error occurs while creating the graph
 */
public static void executeCreateGraph(NeptuneGraphClient client, String
graphName) {
 try {
 // Create the graph request
 CreateGraphRequest request = CreateGraphRequest.builder()
 .graphName(graphName)
 .provisionedMemory(16)
 .build();

 // Create the graph
 CreateGraphResponse response = client.createGraph(request);

 // Extract the graph name and ARN
 String createdGraphName = response.name();
 String graphArn = response.arn();
 String graphEndpoint = response.endpoint();

 System.out.println("Graph created successfully!");
 System.out.println("Graph Name: " + createdGraphName);
 System.out.println("Graph ARN: " + graphArn);
 System.out.println("Graph Endpoint: " +graphEndpoint);

 } catch (NeptuneGraphException e) {
```

```

 System.err.println("Failed to create graph: " +
e.awsErrorDetails().errorMessage());
 } finally {
 client.close();
 }
}

```

- For API details, see [CreateGraph](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBCluster

The following code example shows how to use DeleteDBCluster.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes a DB instance asynchronously.
 *
 * @param clusterId the identifier of the cluster to delete
 * @return a {@link CompletableFuture} that completes when the cluster has been
deleted
 */
public CompletableFuture<Void> deleteDBClusterAsync(String clusterId) {
 DeleteDbClusterRequest request = DeleteDbClusterRequest.builder()
 .dbClusterIdentifier(clusterId)
 .skipFinalSnapshot(true)
 .build();

 return getAsyncClient().deleteDBCluster(request)
 .thenAccept(response -> System.out.println("## Deleting DB Cluster:
" + clusterId));
}

```

- For API details, see [DeleteDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBInstance

The following code example shows how to use DeleteDBInstance.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a DB instance asynchronously.
 *
 * @param instanceId the identifier of the DB instance to be deleted
 * @return a {@link CompletableFuture} that completes when the DB instance has
 * been deleted
 */
public CompletableFuture<Void> deleteDBInstanceAsync(String instanceId) {
 DeleteDbInstanceRequest request = DeleteDbInstanceRequest.builder()
 .dbInstanceIdentifier(instanceId)
 .skipFinalSnapshot(true)
 .build();

 return getAsyncClient().deleteDBInstance(request)
 .thenAccept(response -> System.out.println("## Deleting DB Instance:
" + instanceId));
}
```

- For API details, see [DeleteDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBSubnetGroup

The following code example shows how to use DeleteDBSubnetGroup.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes a subnet group.
 *
 * @param subnetGroupName the identifier of the subnet group to delete
 * @return a {@link CompletableFuture} that completes when the cluster has been
deleted
 */
public CompletableFuture<Void> deleteDBSubnetGroupAsync(String subnetGroupName)
{
 DeleteDbSubnetGroupRequest request = DeleteDbSubnetGroupRequest.builder()
 .dbSubnetGroupName(subnetGroupName)
 .build();

 return getAsyncClient().deleteDBSubnetGroup(request)
 .thenAccept(response -> logger.info("## Deleting Subnet Group: " +
subnetGroupName));
}
```

- For API details, see [DeleteDBSubnetGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBClusters

The following code example shows how to use DescribeDBClusters.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously describes the specified Amazon RDS DB cluster.
 *
 * @param clusterId the identifier of the DB cluster to describe
 * @return a {@link CompletableFuture} that completes when the operation is
 * done, or throws a {@link RuntimeException}
 * if an error occurs
 */
public CompletableFuture<Void> describeDBClustersAsync(String clusterId) {
 DescribeDbClustersRequest request = DescribeDbClustersRequest.builder()
 .dbClusterIdentifier(clusterId)
 .build();

 return getAsyncClient().describeDBClusters(request)
 .thenAccept(response -> {
 for (DBCluster cluster : response.dbClusters()) {
 logger.info("Cluster Identifier: " +
cluster.dbClusterIdentifier());
 logger.info("Status: " + cluster.status());
 logger.info("Engine: " + cluster.engine());
 logger.info("Engine Version: " + cluster.engineVersion());
 logger.info("Endpoint: " + cluster.endpoint());
 logger.info("Reader Endpoint: " + cluster.readerEndpoint());
 logger.info("Availability Zones: " +
cluster.availabilityZones());
 logger.info("Subnet Group: " + cluster.dbSubnetGroup());
 logger.info("VPC Security Groups:");
 cluster.vpcSecurityGroups().forEach(vpcGroup ->
 logger.info(" - " +
vpcGroup.vpcSecurityGroupId()));
 logger.info("Storage Encrypted: " +
cluster.storageEncrypted());
 logger.info("IAM DB Auth Enabled: " +
cluster.iamDatabaseAuthenticationEnabled());
 logger.info("Backup Retention Period: " +
cluster.backupRetentionPeriod() + " days");
 logger.info("Preferred Backup Window: " +
cluster.preferredBackupWindow());
 logger.info("Preferred Maintenance Window: " +
cluster.preferredMaintenanceWindow());
 logger.info("-----");
 }
 })
}

```

```

 })
 .exceptionally(ex -> {
 Throwable cause = ex.getCause() != null ? ex.getCause() : ex;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }

 throw new RuntimeException("Failed to describe the DB cluster: "
+ cause.getMessage(), cause);
 });
 }

```

- For API details, see [DescribeDBClusters](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBInstances

The following code example shows how to use DescribeDBInstances.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Checks the status of a Neptune instance recursively until the desired status
 * is reached or a timeout occurs.
 *
 * @param instanceId the ID of the Neptune instance to check
 * @param desiredStatus the desired status of the Neptune instance
 * @param startTime the start time of the operation, used to calculate the
 * elapsed time
 * @param future a {@link CompletableFuture} that will be completed when
 * the desired status is reached
 */
private void checkStatusRecursive(String instanceId, String desiredStatus, long
startTime, CompletableFuture<Void> future) {

```

```

DescribeDbInstancesRequest request = DescribeDbInstancesRequest.builder()
 .dbInstanceIdentifier(instanceId)
 .build();

getAsyncClient().describeDBInstances(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 future.completeExceptionally(
 new CompletionException("Error checking Neptune
instance status", cause)
);
 return;
 }

 List<DBInstance> instances = response.dbInstances();
 if (instances.isEmpty()) {
 future.completeExceptionally(new RuntimeException("Instance
not found: " + instanceId));
 return;
 }

 String currentStatus = instances.get(0).dbInstanceStatus();
 long elapsedSeconds = (System.currentTimeMillis() - startTime) /
1000;

 System.out.printf("\r Elapsed: %-20s Status: %-20s",
formatElapsedTime((int) elapsedSeconds), currentStatus);
 System.out.flush();

 if (desiredStatus.equalsIgnoreCase(currentStatus)) {
 System.out.printf("\r Neptune instance reached desired
status '%s' after %s.\n", desiredStatus, formatElapsedTime((int) elapsedSeconds));
 future.complete(null);
 } else {
 CompletableFuture.delayedExecutor(20, TimeUnit.SECONDS)
 .execute(() -> checkStatusRecursive(instanceId,
desiredStatus, startTime, future));
 }
 });
}

```

- For API details, see [DescribeDBInstances](#) in *AWS SDK for Java 2.x API Reference*.

## ExecuteGremlinProfileQuery

The following code example shows how to use `ExecuteGremlinProfileQuery`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Executes a Gremlin query against an Amazon Neptune database using the
 * provided {@link NeptuneDataClient}.
 *
 * @param client the {@link NeptuneDataClient} instance to use for executing the
 * Gremlin query
 */
public static void executeGremlinQuery(NeptuneDataClient client) {
 try {
 System.out.println("Querying Neptune...");
 ExecuteGremlinQueryRequest request =
 ExecuteGremlinQueryRequest.builder()
 .gremlinQuery("g.V().has('code', 'ANC')")
 .build();

 ExecuteGremlinQueryResponse response =
 client.executeGremlinQuery(request);

 System.out.println("Full Response:");
 System.out.println(response);

 // Retrieve and print the result
 if (response.result() != null) {
 System.out.println("Query Result:");
 System.out.println(response.result().toString());
 } else {
 System.out.println("No result returned from the query.");
 }
 } catch (NeptuneDataException e) {
 System.err.println("Error calling Neptune: " +
 e.awsErrorDetails().errorMessage());
 }
}
```

```

 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 } finally {
 client.close();
 }
}

```

- For API details, see [ExecuteGremlinProfileQuery](#) in *AWS SDK for Java 2.x API Reference*.

## ExecuteGremlinQuery

The following code example shows how to use `ExecuteGremlinQuery`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Executes a Gremlin PROFILE query using the provided NeptuneDataClient.
 *
 * @param client The NeptuneDataClient instance to be used for executing the
 * Gremlin PROFILE query.
 */
private static void executeGremlinProfileQuery(NeptuneDataClient client) {
 System.out.println("Executing Gremlin PROFILE query...");

 ExecuteGremlinProfileQueryRequest request =
 ExecuteGremlinProfileQueryRequest.builder()
 .gremlinQuery("g.V().has('code', 'ANC')")
 .build();

 ExecuteGremlinProfileQueryResponse response =
 client.executeGremlinProfileQuery(request);
 if (response.output() != null) {
 System.out.println("Query Profile Output:");
 System.out.println(response.output());
 } else {

```

```

 System.out.println("No output returned from the profile query.");
 }
}

```

- For API details, see [ExecuteGremlinQuery](#) in *AWS SDK for Java 2.x API Reference*.

## ExecuteOpenCypherExplainQuery

The following code example shows how to use `ExecuteOpenCypherExplainQuery`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Executes an OpenCypher EXPLAIN query using the provided Neptune data client.
 *
 * @param client The Neptune data client to use for the query execution.
 */
public static void executeGremlinQuery(NeptunedataClient client) {
 try {
 System.out.println("Executing OpenCypher EXPLAIN query...");
 ExecuteOpenCypherExplainQueryRequest request =
 ExecuteOpenCypherExplainQueryRequest.builder()
 .openCypherQuery("MATCH (n {code: 'ANC'}) RETURN n")
 .explainMode("debug")
 .build();

 ExecuteOpenCypherExplainQueryResponse response =
 client.executeOpenCypherExplainQuery(request);

 if (response.results() != null) {
 System.out.println("Explain Results:");
 System.out.println(response.results().asUtf8String());
 } else {
 System.out.println("No explain results returned.");
 }
 }
}

```

```

 } catch (NeptunedataException e) {
 System.err.println("Neptune error: " +
e.awsErrorDetails().errorMessage());
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 } finally {
 client.close();
 }
 }
}

```

- For API details, see [ExecuteOpenCypherExplainQuery](#) in *AWS SDK for Java 2.x API Reference*.

## ExecuteQuery

The following code example shows how to use `ExecuteQuery`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Executes a Gremlin profile query on the Neptune Analytics graph.
 *
 * @param client the {@link NeptuneGraphClient} instance to use for the
query
 * @param graphId the identifier of the graph to execute the query on
 *
 * @throws NeptuneGraphException if an error occurs while executing the query on
the Neptune Graph
 * @throws Exception if an unexpected error occurs
 */
public static void executeGremlinProfileQuery(NeptuneGraphClient client, String
graphId) {

 try {
 System.out.println("Running openCypher query on Neptune Analytics...");
 }
}

```



```

 ExecuteQueryRequest request = ExecuteQueryRequest.builder()
 .graphIdentifier(graphId)
 .queryString("MATCH (n {code: 'ANC'}) RETURN n")
 .language("OPEN_CYPHER")
 .build();

 ResponseInputStream<ExecuteQueryResponse> response =
client.executeQuery(request);
 try (BufferedReader reader = new BufferedReader(new
InputStreamReader(response, StandardCharsets.UTF_8))) {
 String result = reader.lines().collect(Collectors.joining("\n"));
 System.out.println("Query Result:");
 System.out.println(result);
 } catch (Exception e) {
 System.err.println("Error reading response: " + e.getMessage());
 }

 } catch (NeptuneGraphException e) {
 System.err.println("NeptuneGraph error: " +
e.awsErrorDetails().errorMessage());
 } catch (Exception e) {
 System.err.println("Unexpected error: " + e.getMessage());
 } finally {
 client.close();
 }
 }
}

```

- For API details, see [ExecuteQuery](#) in *AWS SDK for Java 2.x API Reference*.

## StartDBCluster

The following code example shows how to use StartDBCluster.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Starts an Amazon Neptune DB cluster.
 *
 * @param clusterIdentifier the unique identifier of the DB cluster to be
stopped
 */
public CompletableFuture<StartDbClusterResponse> startDBClusterAsync(String
clusterIdentifier) {
 StartDbClusterRequest clusterRequest = StartDbClusterRequest.builder()
 .dbClusterIdentifier(clusterIdentifier)
 .build();

 return getAsyncClient().startDBCluster(clusterRequest)
 .whenComplete((response, error) -> {
 if (error != null) {
 Throwable cause = error.getCause() != null ?
error.getCause() : error;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }

 throw new RuntimeException("Failed to start DB cluster: " +
cause.getMessage(), cause);
 } else {
 logger.info("DB Cluster starting: " + clusterIdentifier);
 }
 });
}

```

- For API details, see [StartDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## StopDBCluster

The following code example shows how to use StopDBCluster.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Stops an Amazon Neptune DB cluster.
 *
 * @param clusterIdentifier the unique identifier of the DB cluster to be
 * stopped
 */
public CompletableFuture<StopDbClusterResponse> stopDBClusterAsync(String
clusterIdentifier) {
 StopDbClusterRequest clusterRequest = StopDbClusterRequest.builder()
 .dbClusterIdentifier(clusterIdentifier)
 .build();

 return getAsyncClient().stopDBCluster(clusterRequest)
 .whenComplete((response, error) -> {
 if (error != null) {
 Throwable cause = error.getCause() != null ?
error.getCause() : error;

 if (cause instanceof ResourceNotFoundException) {
 throw (ResourceNotFoundException) cause;
 }

 throw new RuntimeException("Failed to stop DB cluster: " +
cause.getMessage(), cause);
 } else {
 logger.info("DB Cluster stopped: " + clusterIdentifier);
 }
 });
}
```

- For API details, see [StopDBCluster](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Use the Neptune API to query graph data

The following code example shows how to use the Neptune API to query graph data.

#### SDK for Java 2.x

Shows how to use Amazon Neptune Java API to create a Lambda function that queries graph data within the VPC.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Lambda
- Neptune

## Partner Central examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Partner Central.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

# Actions

## AssignOpportunity

The following code example shows how to use AssignOpportunity.

### SDK for Java 2.x

Reassign an existing Opportunity to another user.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssignOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssignOpportunityResponse;
import software.amazon.awssdk.services.partnercentralselling.model.AssigneeContact;

/*
Purpose
PC-API-07 Assigning a new owner
*/

public class AssignOpportunity {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {
```

```
String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

String assigneeFirstName = "John";

String assigneeLastName = "Doe";

String assigneeEmail = "test@test.com";

String businessTitle = "PartnerAccountManager";

AssignOpportunityResponse response = getResponse(opportunityId,
assigneeFirstName, assigneeLastName, assigneeEmail, businessTitle);

ReferenceCodesUtils.formatOutput(response);
}

static AssignOpportunityResponse getResponse(String opportunityId, String
assigneeFirstName, String assigneeLastName, String assigneeEmail, String
businessTitle) {

 AssignOpportunityRequest assignOpportunityRequest =
AssignOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(opportunityId)
 .assignee(AssigneeContact.builder()
 .firstName(assigneeFirstName)
 .lastName(assigneeLastName)
 .email(assigneeEmail)
 .businessTitle(businessTitle)
 .build())
 .build();

 AssignOpportunityResponse response =
client.assignOpportunity(assignOpportunityRequest);

 return response;
}
}
```

- For API details, see [AssignOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## AssociateOpportunity

The following code example shows how to use AssociateOpportunity.

### SDK for Java 2.x

Create a formal association between an Opportunity and various related entities.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssociateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssociateOpportunityResponse;

/*
Purpose
PC-API -11 Associating a product
PC-API -12 Associating a solution
PC-API -13 Associating an offer
entity_type = Solutions | AWSProducts | AWSMarketplaceOffers
*/

public class AssociateOpportunity {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;
```

```
String entityType = "Solutions";

String entityIdentifier = "S-00000000";

AssociateOpportunityResponse response = getResponse(opportunityId, entityType,
entityIdentifier);

ReferenceCodesUtils.formatOutput(response);
}

static AssociateOpportunityResponse getResponse(String opportunityId, String
entityType, String entityIdentifier) {

 AssociateOpportunityRequest associateOpportunityRequest =
AssociateOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .opportunityIdentifier(opportunityId)
 .relatedEntityType(entityType)
 .relatedEntityIdentifier(entityIdentifier)
 .build();

 AssociateOpportunityResponse response =
client.associateOpportunity(associateOpportunityRequest);

 return response;
}
}
```

- For API details, see [AssociateOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## CreateOpportunity

The following code example shows how to use CreateOpportunity.

### SDK for Java 2.x

Create an opportunity.

```
package org.example;

import java.time.Instant;
```



```
import java.util.ArrayList;
import java.util.List;

import static org.example.utils.Constants.*;

import org.example.entity.Root;
import org.example.utils.ReferenceCodesUtils;
import org.example.utils.StringSerializer;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import software.amazon.awssdk.services.partnercentralselling.model.Account;
import software.amazon.awssdk.services.partnercentralselling.model.Address;
import software.amazon.awssdk.services.partnercentralselling.model.Contact;
import
 software.amazon.awssdk.services.partnercentralselling.model.CreateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.CreateOpportunityResponse;
import software.amazon.awssdk.services.partnercentralselling.model.Customer;
import
 software.amazon.awssdk.services.partnercentralselling.model.ExpectedCustomerSpend;
import software.amazon.awssdk.services.partnercentralselling.model.LifeCycle;
import software.amazon.awssdk.services.partnercentralselling.model.Marketing;
import software.amazon.awssdk.services.partnercentralselling.model.MonetaryValue;
import software.amazon.awssdk.services.partnercentralselling.model.NextStepsHistory;
import software.amazon.awssdk.services.partnercentralselling.model.Project;
import software.amazon.awssdk.services.partnercentralselling.model.SoftwareRevenue;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.ToNumberPolicy;

public class CreateOpportunity {

 static final Gson GSON = new GsonBuilder()
 .setObjectToNumberStrategy(ToNumberPolicy.LAZILY_PARSED_NUMBER)
 .registerTypeAdapter(String.class, new StringSerializer())
 .create();

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
```

```

 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

public static void main(String[] args) {

 String inputFile = "CreateOpportunity2.json";

 if (args.length > 0)
 inputFile = args[0];

 CreateOpportunityResponse response = createOpportunity(inputFile);

 client.close();
}

static CreateOpportunityResponse createOpportunity(String inputFile) {

 String inputString = ReferenceCodesUtils.readInputFileToString(inputFile);

 Root root = GSON.fromJson(inputString, Root.class);

 List<NextStepsHistory> nextStepsHistories = new ArrayList<NextStepsHistory>();
 if (root.lifeCycle != null && root.lifeCycle.nextStepsHistories != null) {
 for (org.example.entity.NextStepsHistory nextStepsHistoryJson :
root.lifeCycle.nextStepsHistories) {
 NextStepsHistory nextStepsHistory = NextStepsHistory.builder()
 .time(Instant.parse(nextStepsHistoryJson.time))
 .value(nextStepsHistoryJson.value)
 .build();
 nextStepsHistories.add(nextStepsHistory);
 }
 }

 Lifecycle lifeCycle = null;
 if (root.lifeCycle != null) {
 lifeCycle = Lifecycle.builder()
 .closedLostReason(root.lifeCycle.closedLostReason)
 .nextSteps(root.lifeCycle.nextSteps)
 .nextStepsHistory(nextStepsHistories)
 .reviewComments(root.lifeCycle.reviewComments)
 .reviewStatus(root.lifeCycle.reviewStatus)
 .reviewStatusReason(root.lifeCycle.reviewStatusReason)
 .stage(root.lifeCycle.stage)

```

```

 .targetCloseDate(root.lifeCycle.targetCloseDate)
 .build();
 }

 Marketing marketing = null;
 if (root.marketing != null) {
 marketing = Marketing.builder()
 .awsFundingUsed(root.marketing.awsFundingUsed)
 .campaignName(root.marketing.campaignName)
 .channels(root.marketing.channels)
 .source(root.marketing.source)
 .useCases(root.marketing.useCases)
 .build();
 }

 Address address = null;
 if (root.customer != null && root.customer.account != null &&
 root.customer.account.address != null) {
 address = Address.builder()
 .city(root.customer.account.address.city)
 .postalCode(root.customer.account.address.postalCode)
 .stateOrRegion(root.customer.account.address.stateOrRegion)
 .countryCode(root.customer.account.address.countryCode)
 .streetAddress(root.customer.account.address.streetAddress)
 .build();
 }

 Account account = null;
 if (root.customer != null && root.customer.account != null) {
 account = Account.builder()
 .address(address)
 .awsAccountId(root.customer.account.awsAccountId)
 .duns(root.customer.account.duns)
 .industry(root.customer.account.industry)
 .otherIndustry(root.customer.account.otherIndustry)
 .companyName(root.customer.account.companyName)
 .websiteUrl(root.customer.account.websiteUrl)
 .build();
 }

 List<Contact> contacts = new ArrayList<Contact>();
 if (root.customer != null && root.customer.contacts != null) {
 for (org.example.entity.Contact jsonContact : root.customer.contacts) {

```

```

 Contact contact = Contact.builder()
 .email(jsonContact.email)
 .firstName(jsonContact.firstName)
 .lastName(jsonContact.lastName)
 .phone(jsonContact.phone)
 .businessTitle(jsonContact.businessTitle)
 .build();
 contacts.add(contact);
 }
}

Customer customer = Customer.builder()
 .account(account)
 .contacts(contacts)
 .build();

Contact opportunityTeamContact = null;
if (root.opportunityTeam != null && root.opportunityTeam.get(0) != null) {
 opportunityTeamContact = Contact.builder()
 .firstName(root.opportunityTeam.get(0).firstName)
 .lastName(root.opportunityTeam.get(0).lastName)
 .email(root.opportunityTeam.get(0).email)
 .phone(root.opportunityTeam.get(0).phone)
 .businessTitle(root.opportunityTeam.get(0).businessTitle)
 .build();
}

List<ExpectedCustomerSpend> expectedCustomerSpend = new
ArrayList<ExpectedCustomerSpend>();
if (root.project != null && root.project.expectedCustomerSpend != null) {
 for (org.example.entity.ExpectedCustomerSpend expectedCustomerSpendJson :
root.project.expectedCustomerSpend) {
 ExpectedCustomerSpend expectedCustomerSpend = null;
 expectedCustomerSpend = ExpectedCustomerSpend.builder()
 .amount(expectedCustomerSpendJson.amount)
 .currencyCode(expectedCustomerSpendJson.currencyCode)
 .frequency(expectedCustomerSpendJson.frequency)
 .targetCompany(expectedCustomerSpendJson.targetCompany)
 .build();
 expectedCustomerSpend.add(expectedCustomerSpend);
 }
}

Project project = null;

```

```

 if (root.project != null) {
 project = Project.builder()
 .title(root.project.title)
 .customerBusinessProblem(root.project.customerBusinessProblem)
 .customerUseCase(root.project.customerUseCase)
 .deliveryModels(root.project.deliveryModels)
 .expectedCustomerSpend(expectedCustomerSpends)
 .salesActivities(root.project.salesActivities)
 .competitorName(root.project.competitorName)
 .otherSolutionDescription(root.project.otherSolutionDescription)
 .build();
 }

 SoftwareRevenue softwareRevenue = null;
 if (root.softwareRevenue != null) {
 MonetaryValue monetaryValue = null;
 if (root.softwareRevenue.value != null) {
 monetaryValue = MonetaryValue.builder()
 .amount(root.softwareRevenue.value.amount)
 .currencyCode(root.softwareRevenue.value.currencyCode)
 .build();
 }
 softwareRevenue = SoftwareRevenue.builder()
 .deliveryModel(root.softwareRevenue.deliveryModel)
 .effectiveDate(root.softwareRevenue.effectiveDate)
 .expirationDate(root.softwareRevenue.expirationDate)
 .value(monetaryValue)
 .build();
 }

 // Building the Actual CreateOpportunity Request
 CreateOpportunityRequest createOpportunityRequest =
 CreateOpportunityRequest.builder()
 .catalog(CATALOG_TO_USE)
 .clientToken(root.clientToken)
 .primaryNeedsFromAwsWithStrings(root.primaryNeedsFromAws)
 .opportunityType(root.opportunityType)
 .lifeCycle(lifeCycle)
 .marketing(marketing)
 .nationalSecurity(root.nationalSecurity)
 .origin(root.origin)
 .customer(customer)
 .project(project)
 .partnerOpportunityIdentifier(root.partnerOpportunityIdentifier)

```

```

 .opportunityTeam(opportunityTeamContact)
 .softwareRevenue(softwareRevenue)
 .build();

 CreateOpportunityResponse response =
client.createOpportunity(createOpportunityRequest);
 System.out.println("Successfully created: " + response);

 return response;
}
}

```

- For API details, see [CreateOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## DisassociateOpportunity

The following code example shows how to use DisassociateOpportunity.

### SDK for Java 2.x

Remove an existing association between an Opportunity and related entities.

```

package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.DisassociateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.DisassociateOpportunityResponse;

/*
Purpose

```

```

PC-API -14 Removing a Solution
PC-API -15 Removing an offer
PC-API -16 Removing a product
entity_type = Solutions | AWSProducts | AWSMarketplaceOffers
*/

public class DisassociateOpportunity {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 String entityType = "Solutions";

 String entityIdentifier = "S-00000000";

 DisassociateOpportunityResponse response = getResponse(opportunityId,
entityType, entityIdentifier);

 ReferenceCodesUtils.formatOutput(response);
 }

 static DisassociateOpportunityResponse getResponse(String opportunityId, String
entityType, String entityIdentifier) {

 DisassociateOpportunityRequest disassociateOpportunityRequest =
DisassociateOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .opportunityIdentifier(opportunityId)
 .relatedEntityType(entityType)
 .relatedEntityIdentifier(entityIdentifier)
 .build();

 DisassociateOpportunityResponse response =
client.disassociateOpportunity(disassociateOpportunityRequest);

 return response;
 }
}

```

```
}
```

- For API details, see [DisassociateOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## GetAwsOpportunitySummary

The following code example shows how to use `GetAwsOpportunitySummary`.

### SDK for Java 2.x

Retrieves a summary of an AWS Opportunity.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetAwsOpportunitySummaryRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetAwsOpportunitySummaryResponse;

/**
 * Purpose
 * PC-API-25 Retrieves a summary of an AWS Opportunity.
 */

public class GetAwsOpportunitySummary {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();
```



```
public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 GetAwsOpportunitySummaryResponse response = getResponse(opportunityId);

 ReferenceCodesUtils.formatOutput(response);
}

public static GetAwsOpportunitySummaryResponse getResponse(String opportunityId) {

 GetAwsOpportunitySummaryRequest getOpportunityRequest =
 GetAwsOpportunitySummaryRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .relatedOpportunityIdentifier(opportunityId)
 .build();

 GetAwsOpportunitySummaryResponse response =
 client.getAwsOpportunitySummary(getOpportunityRequest);

 return response;
}
}
```

- For API details, see [GetAwsOpportunitySummary](#) in *AWS SDK for Java 2.x API Reference*.

## GetEngagementInvitation

The following code example shows how to use GetEngagementInvitation.

### SDK for Java 2.x

Retrieves the details of an engagement invitation shared by AWS with a partner.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
```

```

import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetEngagementInvitationRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetEngagementInvitationResponse;

/*
 * Purpose
 * PC-API-22 Get engagement invitation opportunity
 */

public class GetEngagementInvitation {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 GetEngagementInvitationResponse response = getResponse(opportunityId);

 ReferenceCodesUtils.formatOutput(response);
 }

 static GetEngagementInvitationResponse getResponse(String opportunityId) {

 GetEngagementInvitationRequest getOpportunityRequest =
 GetEngagementInvitationRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(opportunityId)
 .build();

 GetEngagementInvitationResponse response =
 client.getEngagementInvitation(getOpportunityRequest);

 return response;
 }
}

```

```
}
```

- For API details, see [GetEngagementInvitation](#) in *AWS SDK for Java 2.x API Reference*.

## GetOpportunity

The following code example shows how to use GetOpportunity.

### SDK for Java 2.x

Get an opportunity.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetOpportunityResponse;

/*
 * Purpose
 * PC-API-08 Get updated Opportunity
 */

public class GetOpportunity {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();
```

```
public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 GetOpportunityResponse response = getResponse(opportunityId);

 ReferenceCodesUtils.formatOutput(response);
}

public static GetOpportunityResponse getResponse(String opportunityId) {

 GetOpportunityRequest getOpportunityRequest =
 GetOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(opportunityId)
 .build();

 GetOpportunityResponse response =
 client.getOpportunity(getOpportunityRequest);

 return response;
}
}
```

- For API details, see [GetOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## ListEngagementInvitations

The following code example shows how to use ListEngagementInvitations.

### SDK for Java 2.x

Retrieves a list of engagement invitations sent to the partner.

```
package org.example;

import java.util.ArrayList;
import java.util.List;

import org.example.utils.ReferenceCodesUtils;
import static org.example.utils.Constants.*;
```

```

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListEngagementInvitationsRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListEngagementInvitationsResponse;
import software.amazon.awssdk.services.partnercentralselling.model.ParticipantType;
import
 software.amazon.awssdk.services.partnercentralselling.model.EngagementInvitationSummary;

public class ListEngagementInvitations {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 List<EngagementInvitationSummary> opportunitySummaries = getResponse();
 ReferenceCodesUtils.formatOutput(opportunitySummaries);
 }

 static List<EngagementInvitationSummary> getResponse() {

 List<EngagementInvitationSummary> opportunitySummaries = new
 ArrayList<EngagementInvitationSummary>();

 ListEngagementInvitationsRequest listOpportunityRequest =
 ListEngagementInvitationsRequest.builder()
 .catalog(CATALOG_TO_USE)
 .participantType(ParticipantType.RECEIVER)
 .maxResults(5)
 .build();

 ListEngagementInvitationsResponse response =
 client.listEngagementInvitations(listOpportunityRequest);

 opportunitySummaries.addAll(response.engagementInvitationSummaries());
 }
}

```

```
 client.close();

 return opportunitySummaries;
 }
}
```

- For API details, see [ListEngagementInvitations](#) in *AWS SDK for Java 2.x API Reference*.

## ListOpportunities

The following code example shows how to use ListOpportunities.

### SDK for Java 2.x

List opportunities.

```
package org.example;

import java.util.ArrayList;
import java.util.List;

import org.example.utils.ReferenceCodesUtils;
import static org.example.utils.Constants.*;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListOpportunitiesRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListOpportunitiesResponse;
import
 software.amazon.awssdk.services.partnercentralselling.model.OpportunitySummary;

/*
 * Purpose
 * PC-API-18 Getting list of Opportunities
 */

public class ListOpportunititesPaging {
```

```
static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

public static void main(String[] args) {
 List<OpportunitySummary> opportunitySummaries = getResponse();
 ReferenceCodesUtils.formatOutput(opportunitySummaries);
}

private static List<OpportunitySummary> getResponse() {
 List<OpportunitySummary> opportunitySummaries = new
ArrayList<OpportunitySummary>();

 ListOpportunitiesRequest listOpportunityRequest =
ListOpportunitiesRequest.builder()
 .catalog(CATALOG_T0_USE)
 .maxResults(5)
 .build();

 ListOpportunitiesResponse response =
client.listOpportunities(listOpportunityRequest);

 opportunitySummaries.addAll(response.opportunitySummaries());

 while (response.nextToken() != null && response.nextToken().length() > 0) {
 listOpportunityRequest = ListOpportunitiesRequest.builder()
 .catalog(CATALOG_T0_USE)
 .maxResults(5)
 .nextToken(response.nextToken())
 .build();
 response = client.listOpportunities(listOpportunityRequest);
 opportunitySummaries.addAll(response.opportunitySummaries());
 }

 client.close();

 return opportunitySummaries;
}
}
```

- For API details, see [ListOpportunities](#) in *AWS SDK for Java 2.x API Reference*.

## ListSolutions

The following code example shows how to use ListSolutions.

### SDK for Java 2.x

Retrieves a list of Partner Solutions that the partner registered on Partner Central.

```
package org.example;

import java.util.ArrayList;
import java.util.List;

import static org.example.utils.Constants.*;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListSolutionsRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.ListSolutionsResponse;
import software.amazon.awssdk.services.partnercentralselling.model.SolutionBase;

/*
 * Purpose
 * PC-API-10 Getting list of solutions
 */

public class ListSolutions {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {
```



```

 List<SolutionBase> solutionSummaries = getResponse();
 ReferenceCodesUtils.formatOutput(solutionSummaries);
 }

 static List<SolutionBase> getResponse() {
 List<SolutionBase> solutionSummaries = new ArrayList<SolutionBase>();

 ListSolutionsRequest listSolutionsRequest = ListSolutionsRequest.builder()
 .catalog(CATALOG_TO_USE)
 .maxResults(5)
 .build();

 ListSolutionsResponse response = client.listSolutions(listSolutionsRequest);

 solutionSummaries.addAll(response.solutionSummaries());

 return solutionSummaries;
 }
}

```

- For API details, see [ListSolutions](#) in *AWS SDK for Java 2.x API Reference*.

## RejectEngagementInvitation

The following code example shows how to use RejectEngagementInvitation.

### SDK for Java 2.x

Rejects an EngagementInvitation that AWS shared.

```

package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;

```

```

import
 software.amazon.awssdk.services.partnercentralselling.model.RejectEngagementInvitationRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.RejectEngagementInvitationResponse;

/**
 * Purpose
 * PC-API-05 AWS Originated(A0) rejection
 */

public class RejectEngagementInvitation {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 RejectEngagementInvitationResponse response = getResponse(opportunityId);

 ReferenceCodesUtils.formatOutput(response);
 }

 static RejectEngagementInvitationResponse getResponse(String invitationId) {

 RejectEngagementInvitationRequest rejectOpportunityRequest =
 RejectEngagementInvitationRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(invitationId)
 .rejectionReason("Unable to support")
 .build();

 RejectEngagementInvitationResponse response =
 client.rejectEngagementInvitation(rejectOpportunityRequest);

 return response;
 }
}

```

- For API details, see [RejectEngagementInvitation](#) in *AWS SDK for Java 2.x API Reference*.

## StartEngagementByAcceptingInvitationTask

The following code example shows how to use `StartEngagementByAcceptingInvitationTask`.

### SDK for Java 2.x

Starts the engagement by accepting an `EngagementInvitation`.

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.StartEngagementByAcceptingInvit
import
 software.amazon.awssdk.services.partnercentralselling.model.StartEngagementByAcceptingInvit
import
 software.amazon.awssdk.services.partnercentralselling.model.GetEngagementInvitationRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetEngagementInvitationResponse
import software.amazon.awssdk.services.partnercentralselling.model.InvitationStatus;

/*
Purpose
PC-API-04: Start Engagement By Accepting InvitationTask for AWS Originated(A0)
opportunity
*/

public class StartEngagementByAcceptingInvitationTask {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
```

```
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

static String clientToken = "test-a30d161";

public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 StartEngagementByAcceptingInvitationTaskResponse response =
getResponse(opportunityId);

 if (response == null) {
 System.out.println("Opportunity is not AWS Originated.");
 } else {
 ReferenceCodesUtils.formatOutput(response);
 }
}

private static GetEngagementInvitationResponse getInvitation(String
invitationId) {

 GetEngagementInvitationRequest getRequest =
GetEngagementInvitationRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(invitationId)
 .build();

 GetEngagementInvitationResponse response =
client.getEngagementInvitation(getRequest);

 return response;
}

static StartEngagementByAcceptingInvitationTaskResponse getResponse(String
invitationId) {

 if (getInvitation(invitationId).status().equals(InvitationStatus.PENDING)) {
 StartEngagementByAcceptingInvitationTaskRequest acceptOpportunityRequest =
 StartEngagementByAcceptingInvitationTaskRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(invitationId)
 .clientToken(clientToken)
```

```

 .build();

 StartEngagementByAcceptingInvitationTaskResponse response =
 client.startEngagementByAcceptingInvitationTask(acceptOpportunityRequest);
 return response;
 }
 return null;
}
}

```

- For API details, see [StartEngagementByAcceptingInvitationTask](#) in *AWS SDK for Java 2.x API Reference*.

## StartEngagementFromOpportunityTask

The following code example shows how to use `StartEngagementFromOpportunityTask`.

### SDK for Java 2.x

Initiates the engagement process from an existing opportunity by accepting the engagement invitation and creating a corresponding opportunity in the partner's system.

```

package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import software.amazon.awssdk.services.partnercentralselling.model.AwsSubmission;
import
 software.amazon.awssdk.services.partnercentralselling.model.SalesInvolvementType;
import
 software.amazon.awssdk.services.partnercentralselling.model.StartEngagementFromOpportunityT
import
 software.amazon.awssdk.services.partnercentralselling.model.StartEngagementFromOpportunityT
import software.amazon.awssdk.services.partnercentralselling.model.Visibility;

```

```

/*
 * Purpose
 * PC-API-01 Partner Originated (PO) opp submission(Start Engagement From
 * Opportunity Task for AO Originated Opportunity)
 */

public class StartEngagementFromOpportunityTask {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 StartEngagementFromOpportunityTaskResponse response =
 getResponse(opportunityId);

 ReferenceCodesUtils.formatOutput(response);
 }

 static StartEngagementFromOpportunityTaskResponse getResponse(String opportunityId)
 {

 StartEngagementFromOpportunityTaskRequest submitOpportunityRequest =
 StartEngagementFromOpportunityTaskRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(opportunityId)
 .clientToken("test-annjqwesdsd99")

 .awsSubmission(AwsSubmission.builder().involvementType(SalesInvolvementType.CO_SELL).visibi
 .build());

 StartEngagementFromOpportunityTaskResponse response =
 client.startEngagementFromOpportunityTask(submitOpportunityRequest);

 return response;
 }
}

```

- For API details, see [StartEngagementFromOpportunityTask](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateOpportunity

The following code example shows how to use UpdateOpportunity.

### SDK for Java 2.x

Update an opportunity.

```
package org.example;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

import static org.example.utils.Constants.*;

import org.example.entity.Root;
import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;
import org.example.utils.StringSerializer;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import software.amazon.awssdk.services.partnercentralselling.model.Account;
import software.amazon.awssdk.services.partnercentralselling.model.Address;
import software.amazon.awssdk.services.partnercentralselling.model.Contact;
import software.amazon.awssdk.services.partnercentralselling.model.Customer;
import
 software.amazon.awssdk.services.partnercentralselling.model.ExpectedCustomerSpend;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.GetOpportunityResponse;
import software.amazon.awssdk.services.partnercentralselling.model.LifeCycle;
import software.amazon.awssdk.services.partnercentralselling.model.Marketing;
```

```

import software.amazon.awssdk.services.partnercentralselling.model.NextStepsHistory;
import software.amazon.awssdk.services.partnercentralselling.model.Project;
import software.amazon.awssdk.services.partnercentralselling.model.ReviewStatus;
import
 software.amazon.awssdk.services.partnercentralselling.model.UpdateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.UpdateOpportunityResponse;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.ToNumberPolicy;

/*
 * Purpose
 * PC-API-02/06 Update opportunity when LifeCycle.ReviewStatus is not Submitted or
 * In-Review
 */

public class UpdateOpportunity {

 static final Gson GSON = new GsonBuilder()
 .setObjectToNumberStrategy(ToNumberPolicy.LAZILY_PARSED_NUMBER)
 .registerTypeAdapter(String.class, new StringSerializer())
 .create();

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

 static String OPPORTUNITY_ORIGIN = ORIGIN_PARTNER_ORIGINATED;

 public static void main(String[] args) {

 String inputFile = "updateOpportunity.json";

 if (args.length > 0)
 inputFile = args[0];

 UpdateOpportunityResponse response = updateOpportunity(inputFile);

 client.close();
 }
}

```



```

public static GetOpportunityResponse getResponse(String opportunityId) {

 GetOpportunityRequest getOpportunityRequest =
GetOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .identifier(opportunityId)
 .build();

 GetOpportunityResponse response =
client.getOpportunity(getOpportunityRequest);
 System.out.println(opportunityId + ":" + response);
 return response;
}

public static UpdateOpportunityResponse updateOpportunity(String inputFile) {

 String inputString = ReferenceCodesUtils.readInputFileToString(inputFile);

 Root root = GSON.fromJson(inputString, Root.class);
 GetOpportunityResponse response = getResponse(root.identifier);

 if (response != null
 && response.lifeCycle() != null
 && response.lifeCycle().reviewStatus() != null
 && response.lifeCycle().reviewStatus() != ReviewStatus.SUBMITTED
 && response.lifeCycle().reviewStatus() != ReviewStatus.IN_REVIEW) {

 List<NextStepsHistory> nextStepsHistories = new ArrayList<NextStepsHistory>();
 if (root.lifeCycle != null && root.lifeCycle.nextStepsHistories != null) {
 for (org.example.entity.NextStepsHistory nextStepsHistoryJson :
root.lifeCycle.nextStepsHistories) {
 NextStepsHistory nextStepsHistory = NextStepsHistory.builder()
 .time(Instant.parse(nextStepsHistoryJson.time))
 .value(nextStepsHistoryJson.value)
 .build();
 nextStepsHistories.add(nextStepsHistory);
 }
 }

 LifeCycle lifeCycle = null;
 if (root.lifeCycle != null) {
 lifeCycle = LifeCycle.builder()
 .closedLostReason(root.lifeCycle.closedLostReason)

```

```

 .nextSteps(root.lifeCycle.nextSteps)
 .nextStepsHistory(nextStepsHistories)
 .reviewComments(root.lifeCycle.reviewComments)
 .reviewStatus(root.lifeCycle.reviewStatus)
 .reviewStatusReason(root.lifeCycle.reviewStatusReason)
 .stage(root.lifeCycle.stage)
 .targetCloseDate(root.lifeCycle.targetCloseDate)
 .build();
 }

 Marketing marketing = null;
 if (root.marketing != null) {
 marketing = Marketing.builder()
 .awsFundingUsed(root.marketing.awsFundingUsed)
 .campaignName(root.marketing.campaignName)
 .channels(root.marketing.channels)
 .source(root.marketing.source)
 .useCases(root.marketing.useCases)
 .build();
 }

 Address address = null;
 if (root.customer != null && root.customer.account != null &&
root.customer.account.address != null) {
 address = Address.builder().postalCode(root.customer.account.address.postalCode)
 .stateOrRegion(root.customer.account.address.stateOrRegion)
 .countryCode(root.customer.account.address.countryCode).build();
 }

 Account account = null;
 if (root.customer != null && root.customer.account != null) {
 account = Account.builder().address(address).duns(root.customer.account.duns)

.industry(root.customer.account.industry).companyName(root.customer.account.companyName)
 .websiteUrl(root.customer.account.websiteUrl).build();
 }

 List<Contact> contacts = new ArrayList<Contact>();
 if (root.customer != null && root.customer.contacts != null) {
 for (org.example.entity.Contact jsonContact : root.customer.contacts) {
 Contact contact = Contact.builder()
 .email(jsonContact.email)
 .firstName(jsonContact.firstName)

```

```

 .lastName(jsonContact.lastName)
 .phone(jsonContact.phone)
 .businessTitle(jsonContact.businessTitle)
 .build();
 contacts.add(contact);
}
}

Customer customer =
Customer.builder().account(account).contacts(contacts).build();

List<ExpectedCustomerSpend> expectedCustomerSpend = new
ArrayList<ExpectedCustomerSpend>();
if (root.project != null && root.project.expectedCustomerSpend != null) {
 for (org.example.entity.ExpectedCustomerSpend expectedCustomerSpendJson :
root.project.expectedCustomerSpend) {
 ExpectedCustomerSpend expectedCustomerSpend = null;
 expectedCustomerSpend = ExpectedCustomerSpend.builder()
 .amount(expectedCustomerSpendJson.amount)
 .currencyCode(expectedCustomerSpendJson.currencyCode)
 .frequency(expectedCustomerSpendJson.frequency)
 .targetCompany(expectedCustomerSpendJson.targetCompany)
 .build();
 expectedCustomerSpend.add(expectedCustomerSpend);
 }
}

Project project = null;
if (root.project != null) {
 project = Project.builder().title(root.project.title)
 .customerBusinessProblem(root.project.customerBusinessProblem)

.customerUseCase(root.project.customerUseCase).deliveryModels(root.project.deliveryModels)
 .expectedCustomerSpend(expectedCustomerSpend)

.salesActivities(root.project.salesActivities).competitorName(root.project.competitorName)
 .otherSolutionDescription(root.project.otherSolutionDescription).build();
}

// Building the Actual CreateOpportunity Request
UpdateOpportunityRequest updateOpportunityRequest =
UpdateOpportunityRequest.builder().catalog(root.catalog)

.identifier(root.identifier).lastModifiedDate(Instant.parse(root.lastModifiedDate))

```

```
.primaryNeedsFromAwsWithStrings(root.primaryNeedsFromAws).opportunityType(root.opportunityType)
 .lifeCycle(lifeCycle)
 .customer(customer)
 .project(project)
 .partnerOpportunityIdentifier(root.partnerOpportunityIdentifier)
 .marketing(marketing)
 .nationalSecurity(root.nationalSecurity)
 .opportunityType(root.opportunityType)
 .build();

UpdateOpportunityResponse updateResponse =
client.updateOpportunity(updateOpportunityRequest);
System.out.println("Successfully updated opportunity: " + updateResponse);

return updateResponse;
} else {
 System.out.println("Opportunity cannot be updated.");
 return null;
}
}
```

- For API details, see [UpdateOpportunity](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Update associated entity of an opportunity

The following code example shows how to:

- Disassociate an old entity.
- Associate a new entity.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Scenarios](#) repository.

### Update associated entity of an opportunity

```
package org.example;

import static org.example.utils.Constants.*;

import org.example.utils.Constants;
import org.example.utils.ReferenceCodesUtils;

import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.regions.Region;
import
 software.amazon.awssdk.services.partnercentralselling.PartnerCentralSellingClient;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssociateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.AssociateOpportunityResponse;
import
 software.amazon.awssdk.services.partnercentralselling.model.DisassociateOpportunityRequest;
import
 software.amazon.awssdk.services.partnercentralselling.model.DisassociateOpportunityResponse;

/*
Purpose
PC-API -17 Replacing a solution
*/

public class ReplaceSolution {

 static PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();
```

```

public static void main(String[] args) {

 String opportunityId = args.length > 0 ? args[0] : OPPORTUNITY_ID;

 String entityType = "Solutions";
 String originalEntityIdentifier = "S-00000000";
 String newEntityIdentifier = "S-00111111";

 disassociateOppornitityResponse(opportunityId, entityType,
originalEntityIdentifier);
 AssociateOpportunityResponse associateOpportunityResponse =
associateOpportunityResponse(opportunityId, entityType, newEntityIdentifier);

 ReferenceCodesUtils.formatOutput(associateOpportunityResponse);
}

private static AssociateOpportunityResponse associateOpportunityResponse(String
opportunityId, String entityType, String entityIdentifier) {

 AssociateOpportunityRequest associateOpportunityRequest =
AssociateOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .opportunityIdentifier(opportunityId)
 .relatedEntityType(entityType)
 .relatedEntityIdentifier(entityIdentifier)
 .build();

 AssociateOpportunityResponse response =
client.associateOpportunity(associateOpportunityRequest);

 return response;
}

private static DisassociateOpportunityResponse
disassociateOppornitityResponse(String opportunityId, String entityType, String
entityIdentifier) {
 PartnerCentralSellingClient client = PartnerCentralSellingClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(DefaultCredentialsProvider.create())
 .httpClient(ApacheHttpClient.builder().build())
 .build();

```

```
 DisassociateOpportunityRequest disassociateOpportunityRequest =
DisassociateOpportunityRequest.builder()
 .catalog(Constants.CATALOG_TO_USE)
 .opportunityIdentifier(opportunityId)
 .relatedEntityType(entityType)
 .relatedEntityIdentifier(entityIdentifier)
 .build();

 DisassociateOpportunityResponse response =
client.disassociateOpportunity(disassociateOpportunityRequest);

 return response;
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AssociateOpportunity](#)
  - [DisassociateOpportunity](#)

## Amazon Personalize examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Personalize.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### CreateBatchInferenceJob

The following code example shows how to use CreateBatchInferenceJob.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createPersonalizeBatchInferenceJob(PersonalizeClient
personalizeClient,
 String solutionVersionArn,
 String jobName,
 String s3InputDataSourcePath,
 String s3DataDestinationPath,
 String roleArn,
 String explorationWeight,
 String explorationItemAgeCutOff) {

 long waitInMilliseconds = 60 * 1000;
 String status;
 String batchInferenceJobArn;

 try {

 // Set up data input and output parameters.
 S3DataConfig inputSource = S3DataConfig.builder()
 .path(s3InputDataSourcePath)
 .build();

 S3DataConfig outputDestination = S3DataConfig.builder()
 .path(s3DataDestinationPath)
 .build();

 BatchInferenceJobInput jobInput =
 BatchInferenceJobInput.builder()
 .s3DataSource(inputSource)
 .build();

 BatchInferenceJobOutput jobOutputLocation =
 BatchInferenceJobOutput.builder()
 .s3DataDestination(outputDestination)
 .build();
```



```

// Optional code to build the User-Personalization specific
item exploration
// config.
HashMap<String, String> explorationConfig = new HashMap<>();

explorationConfig.put("explorationWeight",
explorationWeight);
explorationConfig.put("explorationItemAgeCutOff",
explorationItemAgeCutOff);

BatchInferenceJobConfig jobConfig =
BatchInferenceJobConfig.builder()
 .itemExplorationConfig(explorationConfig)
 .build();

// End optional User-Personalization recipe specific code.

CreateBatchInferenceJobRequest
createBatchInferenceJobRequest = CreateBatchInferenceJobRequest
 .builder()
 .solutionVersionArn(solutionVersionArn)
 .jobInput(jobInput)
 .jobOutput(jobOutputLocation)
 .jobName(jobName)
 .roleArn(roleArn)
 .batchInferenceJobConfig(jobConfig) //
Optional
 .build();

batchInferenceJobArn =
personalizeClient.createBatchInferenceJob(createBatchInferenceJobRequest)
 .batchInferenceJobArn();

DescribeBatchInferenceJobRequest
describeBatchInferenceJobRequest = DescribeBatchInferenceJobRequest
 .builder()
 .batchInferenceJobArn(batchInferenceJobArn)
 .build();

long maxTime = Instant.now().getEpochSecond() + 3 * 60 * 60;
while (Instant.now().getEpochSecond() < maxTime) {

```

```

 BatchInferenceJob batchInferenceJob =
personalizeClient

 .describeBatchInferenceJob(describeBatchInferenceJobRequest)
 .batchInferenceJob();

 status = batchInferenceJob.status();
 System.out.println("Batch inference job status: " +
status);

 if (status.equals("ACTIVE") || status.equals("CREATE
FAILED")) {

 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }
 return batchInferenceJobArn;

} catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
}
return "";
}

```

- For API details, see [CreateBatchInferenceJob](#) in *AWS SDK for Java 2.x API Reference*.

## CreateCampaign

The following code example shows how to use CreateCampaign.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createPersonalCampaign(PersonalizeClient personalizeClient,
String solutionVersionArn,
 String name) {

 try {
 CreateCampaignRequest createCampaignRequest =
CreateCampaignRequest.builder()
 .minProvisionedTPS(1)
 .solutionVersionArn(solutionVersionArn)
 .name(name)
 .build();

 CreateCampaignResponse campaignResponse =
personalizeClient.createCampaign(createCampaignRequest);
 System.out.println("The campaign ARN is " +
campaignResponse.campaignArn());
 return campaignResponse.campaignArn();
 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}
```

- For API details, see [CreateCampaign](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDataset

The following code example shows how to use CreateDataset.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createDataset(PersonalizeClient personalizeClient,
 String datasetName,
```

```
 String datasetGroupArn,
 String datasetType,
 String schemaArn) {
 try {
 CreateDatasetRequest request = CreateDatasetRequest.builder()
 .name(datasetName)
 .datasetGroupArn(datasetGroupArn)
 .datasetType(datasetType)
 .schemaArn(schemaArn)
 .build();

 String datasetArn = personalizeClient.createDataset(request)
 .datasetArn();
 System.out.println("Dataset " + datasetName + " created.");
 return datasetArn;

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateDataset](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDatasetExportJob

The following code example shows how to use `CreateDatasetExportJob`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createDatasetExportJob(PersonalizeClient personalizeClient,
 String jobName,
 String datasetArn,
 IngestionMode ingestionMode,
```

```
 String roleArn,
 String s3BucketPath,
 String kmsKeyArn) {

 long waitInMilliseconds = 30 * 1000; // 30 seconds
 String status = null;

 try {

 S3DataConfig exportS3DataConfig =
 S3DataConfig.builder().path(s3BucketPath).kmsKeyArn(kmsKeyArn).build();
 DatasetExportJobOutput jobOutput =
 DatasetExportJobOutput.builder().s3DataDestination(exportS3DataConfig)
 .build();

 CreateDatasetExportJobRequest createRequest =
 CreateDatasetExportJobRequest.builder()
 .jobName(jobName)
 .datasetArn(datasetArn)
 .ingestionMode(ingestionMode)
 .jobOutput(jobOutput)
 .roleArn(roleArn)
 .build();

 String datasetExportJobArn =
 personalizeClient.createDatasetExportJob(createRequest).datasetExportJobArn();

 DescribeDatasetExportJobRequest describeDatasetExportJobRequest =
 DescribeDatasetExportJobRequest.builder()
 .datasetExportJobArn(datasetExportJobArn)
 .build();

 long maxTime = Instant.now().getEpochSecond() + 3 * 60 * 60;

 while (Instant.now().getEpochSecond() < maxTime) {

 DatasetExportJob datasetExportJob = personalizeClient
 .describeDatasetExportJob(describeDatasetExportJobRequest)
 .datasetExportJob();

 status = datasetExportJob.status();
 System.out.println("Export job status: " + status);

 if (status.equals("ACTIVE") || status.equals("CREATE FAILED")) {
```

```

 return status;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
}
} catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
}
return "";
}

```

- For API details, see [CreateDatasetExportJob](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDatasetGroup

The following code example shows how to use CreateDatasetGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String createDatasetGroup(PersonalizeClient personalizeClient,
String datasetGroupName) {

 try {
 CreateDatasetGroupRequest createDatasetGroupRequest =
CreateDatasetGroupRequest.builder()
 .name(datasetGroupName)
 .build();

 return
personalizeClient.createDatasetGroup(createDatasetGroupRequest).datasetGroupArn();
 } catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
 }
}

```

```
 }
 return "";
}
```

Create a domain dataset group.

```
public static String createDomainDatasetGroup(PersonalizeClient
personalizeClient,
 String datasetGroupName,
 String domain) {

 try {
 CreateDatasetGroupRequest createDatasetGroupRequest =
CreateDatasetGroupRequest.builder()
 .name(datasetGroupName)
 .domain(domain)
 .build();
 return
personalizeClient.createDatasetGroup(createDatasetGroupRequest).datasetGroupArn();
 } catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
 }
 return "";
}
```

- For API details, see [CreateDatasetGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDatasetImportJob

The following code example shows how to use `CreateDatasetImportJob`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createPersonalizeDatasetImportJob(PersonalizeClient
personalizeClient,
 String jobName,
 String datasetArn,
 String s3BucketPath,
 String roleArn) {

 long waitInMilliseconds = 60 * 1000;
 String status;
 String datasetImportJobArn;

 try {
 DataSource importDataSource = DataSource.builder()
 .dataLocation(s3BucketPath)
 .build();

 CreateDatasetImportJobRequest createDatasetImportJobRequest =
CreateDatasetImportJobRequest.builder()
 .datasetArn(datasetArn)
 .dataSource(importDataSource)
 .jobName(jobName)
 .roleArn(roleArn)
 .build();

 datasetImportJobArn =
personalizeClient.createDatasetImportJob(createDatasetImportJobRequest)
 .datasetImportJobArn();

 DescribeDatasetImportJobRequest describeDatasetImportJobRequest =
DescribeDatasetImportJobRequest.builder()
 .datasetImportJobArn(datasetImportJobArn)
 .build();

 long maxTime = Instant.now().getEpochSecond() + 3 * 60 * 60;

 while (Instant.now().getEpochSecond() < maxTime) {

 DatasetImportJob datasetImportJob = personalizeClient
 .describeDatasetImportJob(describeDatasetImportJobRequest)
 .datasetImportJob();

 status = datasetImportJob.status();
 System.out.println("Dataset import job status: " + status);
 }
 }
}
```



```

 if (status.equals("ACTIVE") || status.equals("CREATE FAILED")) {
 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }
 return datasetImportJobArn;

} catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
}
return "";
}

```

- For API details, see [CreateDatasetImportJob](#) in *AWS SDK for Java 2.x API Reference*.

## CreateEventTracker

The following code example shows how to use CreateEventTracker.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String createEventTracker(PersonalizeClient personalizeClient,
 String eventTrackerName,
 String datasetGroupArn) {

 String eventTrackerId = "";
 String eventTrackerArn;
 long maxTime = 3 * 60 * 60; // 3 hours
 long waitInMilliseconds = 20 * 1000; // 20 seconds
 String status;

```

```
try {

 CreateEventTrackerRequest createEventTrackerRequest =
CreateEventTrackerRequest.builder()
 .name(eventTrackerName)
 .datasetGroupArn(datasetGroupArn)
 .build();

 CreateEventTrackerResponse createEventTrackerResponse =
personalizeClient
 .createEventTracker(createEventTrackerRequest);

 eventTrackerArn = createEventTrackerResponse.eventTrackerArn();
 eventTrackerId = createEventTrackerResponse.trackingId();
 System.out.println("Event tracker ARN: " + eventTrackerArn);
 System.out.println("Event tracker ID: " + eventTrackerId);

 maxTime = Instant.now().getEpochSecond() + maxTime;

 DescribeEventTrackerRequest describeRequest =
DescribeEventTrackerRequest.builder()
 .eventTrackerArn(eventTrackerArn)
 .build();

 while (Instant.now().getEpochSecond() < maxTime) {

 status =
personalizeClient.describeEventTracker(describeRequest).eventTracker().status();
 System.out.println("EventTracker status: " + status);

 if (status.equals("ACTIVE") || status.equals("CREATE FAILED")) {
 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }
 return eventTrackerId;
} catch (PersonalizeException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

```
 return eventTrackerId;
 }
```

- For API details, see [CreateEventTracker](#) in *AWS SDK for Java 2.x API Reference*.

## CreateFilter

The following code example shows how to use `CreateFilter`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createFilter(PersonalizeClient personalizeClient,
 String filterName,
 String datasetGroupArn,
 String filterExpression) {
 try {
 CreateFilterRequest request = CreateFilterRequest.builder()
 .name(filterName)
 .datasetGroupArn(datasetGroupArn)
 .filterExpression(filterExpression)
 .build();

 return personalizeClient.createFilter(request).filterArn();
 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateFilter](#) in *AWS SDK for Java 2.x API Reference*.

## CreateRecommender

The following code example shows how to use CreateRecommender.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createRecommender(PersonalizeClient personalizeClient,
 String name,
 String datasetGroupArn,
 String recipeArn) {

 long maxTime = 0;
 long waitInMilliseconds = 30 * 1000; // 30 seconds
 String recommenderStatus = "";

 try {
 CreateRecommenderRequest createRecommenderRequest =
 CreateRecommenderRequest.builder()
 .datasetGroupArn(datasetGroupArn)
 .name(name)
 .recipeArn(recipeArn)
 .build();

 CreateRecommenderResponse recommenderResponse = personalizeClient
 .createRecommender(createRecommenderRequest);
 String recommenderArn = recommenderResponse.recommenderArn();
 System.out.println("The recommender ARN is " + recommenderArn);

 DescribeRecommenderRequest describeRecommenderRequest =
 DescribeRecommenderRequest.builder()
 .recommenderArn(recommenderArn)
 .build();

 maxTime = Instant.now().getEpochSecond() + 3 * 60 * 60;

 while (Instant.now().getEpochSecond() < maxTime) {
```

```

 recommenderStatus =
personalizeClient.describeRecommender(describeRecommenderRequest).recommender()
 .status();
 System.out.println("Recommender status: " + recommenderStatus);

 if (recommenderStatus.equals("ACTIVE") ||
recommenderStatus.equals("CREATE FAILED")) {
 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }
 return recommenderArn;

} catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "";
}

```

- For API details, see [CreateRecommender](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSchema

The following code example shows how to use CreateSchema.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String createSchema(PersonalizeClient personalizeClient, String
schemaName, String filePath) {

```

```

 String schema = null;
 try {
 schema = new String(Files.readAllBytes(Paths.get(filePath)));
 } catch (IOException e) {
 System.out.println(e.getMessage());
 }

 try {
 CreateSchemaRequest createSchemaRequest = CreateSchemaRequest.builder()
 .name(schemaName)
 .schema(schema)
 .build();

 String schemaArn =
personalizeClient.createSchema(createSchemaRequest).schemaArn();

 System.out.println("Schema arn: " + schemaArn);

 return schemaArn;

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

Create a schema with a domain.

```

 public static String createDomainSchema(PersonalizeClient personalizeClient,
String schemaName, String domain,
 String filePath) {

 String schema = null;
 try {
 schema = new String(Files.readAllBytes(Paths.get(filePath)));
 } catch (IOException e) {
 System.out.println(e.getMessage());
 }

 try {
 CreateSchemaRequest createSchemaRequest = CreateSchemaRequest.builder()

```

```
 .name(schemaName)
 .domain(domain)
 .schema(schema)
 .build();

 String schemaArn =
personalizeClient.createSchema(createSchemaRequest).schemaArn();

 System.out.println("Schema arn: " + schemaArn);

 return schemaArn;

} catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "";
}
```

- For API details, see [CreateSchema](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSolution

The following code example shows how to use CreateSolution.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createPersonalizeSolution(PersonalizeClient
personalizeClient,
 String datasetGroupArn,
 String solutionName,
 String recipeArn) {

 try {
 CreateSolutionRequest solutionRequest = CreateSolutionRequest.builder()
```

```

 .name(solutionName)
 .datasetGroupArn(datasetGroupArn)
 .recipeArn(recipeArn)
 .build();

 CreateSolutionResponse solutionResponse =
personalizeClient.createSolution(solutionRequest);
 return solutionResponse.solutionArn();

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

```

- For API details, see [CreateSolution](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSolutionVersion

The following code example shows how to use `CreateSolutionVersion`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String createPersonalizeSolutionVersion(PersonalizeClient
personalizeClient, String solutionArn) {
 long maxTime = 0;
 long waitInMilliseconds = 30 * 1000; // 30 seconds
 String solutionStatus = "";
 String solutionVersionStatus = "";
 String solutionVersionArn = "";

 try {
 DescribeSolutionRequest describeSolutionRequest =
DescribeSolutionRequest.builder()

```



```

 .solutionArn(solutionArn)
 .build();

 maxTime = Instant.now().getEpochSecond() + 3 * 60 * 60;

 // Wait until solution is active.
 while (Instant.now().getEpochSecond() < maxTime) {

 solutionStatus =
personalizeClient.describeSolution(describeSolutionRequest).solution().status();
 System.out.println("Solution status: " + solutionStatus);

 if (solutionStatus.equals("ACTIVE") || solutionStatus.equals("CREATE
FAILED")) {
 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }

 if (solutionStatus.equals("ACTIVE")) {

 CreateSolutionVersionRequest createSolutionVersionRequest =
CreateSolutionVersionRequest.builder()
 .solutionArn(solutionArn)
 .build();

 CreateSolutionVersionResponse createSolutionVersionResponse =
personalizeClient
 .createSolutionVersion(createSolutionVersionRequest);
 solutionVersionArn =
createSolutionVersionResponse.solutionVersionArn();

 System.out.println("Solution version ARN: " + solutionVersionArn);

 DescribeSolutionVersionRequest describeSolutionVersionRequest =
DescribeSolutionVersionRequest.builder()
 .solutionVersionArn(solutionVersionArn)
 .build();

 while (Instant.now().getEpochSecond() < maxTime) {

```

```

 solutionVersionStatus =
personalizeClient.describeSolutionVersion(describeSolutionVersionRequest)
 .solutionVersion().status();
 System.out.println("Solution version status: " +
solutionVersionStatus);

 if (solutionVersionStatus.equals("ACTIVE") ||
solutionVersionStatus.equals("CREATE_FAILED")) {
 break;
 }
 try {
 Thread.sleep(waitInMilliseconds);
 } catch (InterruptedException e) {
 System.out.println(e.getMessage());
 }
 }
 return solutionVersionArn;
}
} catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "";
}

```

- For API details, see [CreateSolutionVersion](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCampaign

The following code example shows how to use DeleteCampaign.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteSpecificCampaign(PersonalizeClient personalizeClient,
String campaignArn) {
 try {
 DeleteCampaignRequest campaignRequest = DeleteCampaignRequest.builder()
 .campaignArn(campaignArn)
 .build();

 personalizeClient.deleteCampaign(campaignRequest);
 System.out.println("Delete request sent successfully.");
 } catch (PersonalizeException e) {
 System.err.println("Error deleting campaign: " +
e.awsErrorDetails().errorMessage());
 throw new RuntimeException(e);
 }
}
```

- For API details, see [DeleteCampaign](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteEventTracker

The following code example shows how to use DeleteEventTracker.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteEventTracker(PersonalizeClient personalizeClient,
String eventTrackerArn) {
 try {
 DeleteEventTrackerRequest deleteEventTrackerRequest =
DeleteEventTrackerRequest.builder()
 .eventTrackerArn(eventTrackerArn)
 .build();
```

```
 int status =
personalizeClient.deleteEventTracker(deleteEventTrackerRequest).sdkHttpResponse().statusCode();

 System.out.println("Status code:" + status);

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteEventTracker](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteSolution

The following code example shows how to use DeleteSolution.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteGivenSolution(PersonalizeClient personalizeClient,
String solutionArn) {

 try {
 DeleteSolutionRequest solutionRequest = DeleteSolutionRequest.builder()
 .solutionArn(solutionArn)
 .build();

 personalizeClient.deleteSolution(solutionRequest);
 System.out.println("Done");

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

```
}
```

- For API details, see [DeleteSolution](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeCampaign

The following code example shows how to use `DescribeCampaign`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeSpecificCampaign(PersonalizeClient personalizeClient,
String campaignArn) {

 try {
 DescribeCampaignRequest campaignRequest =
DescribeCampaignRequest.builder()
 .campaignArn(campaignArn)
 .build();

 DescribeCampaignResponse campaignResponse =
personalizeClient.describeCampaign(campaignRequest);
 Campaign myCampaign = campaignResponse.campaign();
 System.out.println("The Campaign name is " + myCampaign.name());
 System.out.println("The Campaign status is " + myCampaign.status());

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeCampaign](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeRecipe

The following code example shows how to use DescribeRecipe.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeSpecificRecipe(PersonalizeClient personalizeClient,
String recipeArn) {

 try {
 DescribeRecipeRequest recipeRequest = DescribeRecipeRequest.builder()
 .recipeArn(recipeArn)
 .build();

 DescribeRecipeResponse recipeResponse =
personalizeClient.describeRecipe(recipeRequest);
 System.out.println("The recipe name is " +
recipeResponse.recipe().name());

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeRecipe](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeSolution

The following code example shows how to use DescribeSolution.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeSpecificSolution(PersonalizeClient personalizeClient,
String solutionArn) {

 try {
 DescribeSolutionRequest solutionRequest =
DescribeSolutionRequest.builder()
 .solutionArn(solutionArn)
 .build();

 DescribeSolutionResponse response =
personalizeClient.describeSolution(solutionRequest);
 System.out.println("The Solution name is " +
response.solution().name());

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeSolution](#) in *AWS SDK for Java 2.x API Reference*.

## ListCampaigns

The following code example shows how to use ListCampaigns.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listAllCampaigns(PersonalizeClient personalizeClient, String
solutionArn) {

 try {
 ListCampaignsRequest campaignsRequest = ListCampaignsRequest.builder()
 .maxResults(10)
 .solutionArn(solutionArn)
 .build();

 ListCampaignsResponse response =
personalizeClient.listCampaigns(campaignsRequest);
 List<CampaignSummary> campaigns = response.campaigns();
 for (CampaignSummary campaign : campaigns) {
 System.out.println("Campaign name is : " + campaign.name());
 System.out.println("Campaign ARN is : " + campaign.campaignArn());
 }

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListCampaigns](#) in *AWS SDK for Java 2.x API Reference*.

## ListDatasetGroups

The following code example shows how to use `ListDatasetGroups`.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listDSGroups(PersonalizeClient personalizeClient) {

 try {
 ListDatasetGroupsRequest groupsRequest =
ListDatasetGroupsRequest.builder()
 .maxResults(15)
 .build();

 ListDatasetGroupsResponse groupsResponse =
personalizeClient.listDatasetGroups(groupsRequest);
 List<DatasetGroupSummary> groups = groupsResponse.datasetGroups();
 for (DatasetGroupSummary group : groups) {
 System.out.println("The DataSet name is : " + group.name());
 System.out.println("The DataSet ARN is : " +
group.datasetGroupArn());
 }

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListDatasetGroups](#) in *AWS SDK for Java 2.x API Reference*.

## ListRecipes

The following code example shows how to use `ListRecipes`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listAllRecipes(PersonalizeClient personalizeClient) {

 try {
 ListRecipesRequest recipesRequest = ListRecipesRequest.builder()
 .maxResults(15)
 .build();

 ListRecipesResponse response =
personalizeClient.listRecipes(recipesRequest);
 List<RecipeSummary> recipes = response.recipes();
 for (RecipeSummary recipe : recipes) {
 System.out.println("The recipe ARN is: " + recipe.recipeArn());
 System.out.println("The recipe name is: " + recipe.name());
 }

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListRecipes](#) in *AWS SDK for Java 2.x API Reference*.

## ListSolutions

The following code example shows how to use `ListSolutions`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listAllSolutions(PersonalizeClient personalizeClient, String
datasetGroupArn) {

 try {
 ListSolutionsRequest solutionsRequest = ListSolutionsRequest.builder()
 .maxResults(10)
 .datasetGroupArn(datasetGroupArn)
 .build();

 ListSolutionsResponse response =
personalizeClient.listSolutions(solutionsRequest);
 List<SolutionSummary> solutions = response.solutions();
 for (SolutionSummary solution : solutions) {
 System.out.println("The solution ARN is: " +
solution.solutionArn());
 System.out.println("The solution name is: " + solution.name());
 }

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListSolutions](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateCampaign

The following code example shows how to use UpdateCampaign.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String updateCampaign(PersonalizeClient personalizeClient,
 String campaignArn,
 String solutionVersionArn,
 Integer minProvisionedTPS) {

 try {
 // build the updateCampaignRequest
 UpdateCampaignRequest updateCampaignRequest =
UpdateCampaignRequest.builder()
 .campaignArn(campaignArn)
 .solutionVersionArn(solutionVersionArn)
 .minProvisionedTPS(minProvisionedTPS)
 .build();

 // update the campaign
 personalizeClient.updateCampaign(updateCampaignRequest);

 DescribeCampaignRequest campaignRequest =
DescribeCampaignRequest.builder()
 .campaignArn(campaignArn)
 .build();

 DescribeCampaignResponse campaignResponse =
personalizeClient.describeCampaign(campaignRequest);
 Campaign updatedCampaign = campaignResponse.campaign();

 System.out.println("The Campaign status is " +
updatedCampaign.status());
 return updatedCampaign.status();

 } catch (PersonalizeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

```
 return "";
 }
```

- For API details, see [UpdateCampaign](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Personalize Events examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Personalize Events.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### PutEvents

The following code example shows how to use PutEvents.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static int putItems(PersonalizeEventsClient personalizeEventsClient,
 String datasetArn,
 String item1Id,
 String item1PropertyName,
 String item1PropertyValue,
```

```

 String item2Id,
 String item2PropertyName,
 String item2PropertyValue) {

 int responseCode = 0;
 ArrayList<Item> items = new ArrayList<>();

 try {
 Item item1 = Item.builder()
 .itemId(item1Id)
 .properties(String.format("{\ \"%1$s\ ": \"%2$s
\}"),
 item1PropertyName,
 item1PropertyValue))
 .build();

 items.add(item1);

 Item item2 = Item.builder()
 .itemId(item2Id)
 .properties(String.format("{\ \"%1$s\ ": \"%2$s
\}"),
 item2PropertyName,
 item2PropertyValue))
 .build();

 items.add(item2);

 PutItemsRequest putItemsRequest = PutItemsRequest.builder()
 .datasetArn(datasetArn)
 .items(items)
 .build();

 responseCode =
personalizeEventsClient.putItems(putItemsRequest).sdkHttpResponse().statusCode();
 System.out.println("Response code: " + responseCode);
 return responseCode;

 } catch (PersonalizeEventsException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
 }
 return responseCode;
}

```

- For API details, see [PutEvents](#) in *AWS SDK for Java 2.x API Reference*.

## PutUsers

The following code example shows how to use PutUsers.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static int putUsers(PersonalizeEventsClient personalizeEventsClient,
 String datasetArn,
 String user1Id,
 String user1PropertyName,
 String user1PropertyValue,
 String user2Id,
 String user2PropertyName,
 String user2PropertyValue) {

 int responseCode = 0;
 ArrayList<User> users = new ArrayList<>();

 try {
 User user1 = User.builder()
 .userId(user1Id)
 .properties(String.format("{\\"%1$s\\": \\"%2$s\\",
 user1PropertyName,
 user1PropertyValue))
 .build();

 users.add(user1);

 User user2 = User.builder()
 .userId(user2Id)
```

```

 .properties(String.format("{\\\"%1$s\\\": \\\"%2$s\\\"}",
 user2PropertyName,
 user2PropertyValue))
 .build();

 users.add(user2);

 PutUsersRequest putUsersRequest = PutUsersRequest.builder()
 .datasetArn(datasetArn)
 .users(users)
 .build();

 responseCode =
personalizeEventsClient.putUsers(putUsersRequest).sdkHttpResponse().statusCode();
 System.out.println("Response code: " + responseCode);
 return responseCode;

 } catch (PersonalizeEventsException e) {
 System.out.println(e.awsErrorDetails().errorMessage());
 }
 return responseCode;
}

```

- For API details, see [PutUsers](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Personalize Runtime examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Personalize Runtime.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)



# Actions

## GetPersonalizedRanking

The following code example shows how to use GetPersonalizedRanking.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static List<PredictedItem> getRankedRecs(PersonalizeRuntimeClient
personalizeRuntimeClient,
 String campaignArn,
 String userId,
 ArrayList<String> items) {

 try {
 GetPersonalizedRankingRequest rankingRecommendationsRequest =
 GetPersonalizedRankingRequest.builder()
 .campaignArn(campaignArn)
 .userId(userId)
 .inputList(items)
 .build();

 GetPersonalizedRankingResponse recommendationsResponse =
 personalizeRuntimeClient
 .getPersonalizedRanking(rankingRecommendationsRequest);
 List<PredictedItem> rankedItems =
 recommendationsResponse.personalizedRanking();
 int rank = 1;
 for (PredictedItem item : rankedItems) {
 System.out.println("Item ranked at position " + rank + " details");
 System.out.println("Item Id is : " + item.itemId());
 System.out.println("Item score is : " + item.score());
 System.out.println("-----");
 rank++;
 }
 return rankedItems;
 }
```

```
 } catch (PersonalizeRuntimeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}
```

- For API details, see [GetPersonalizedRanking](#) in *AWS SDK for Java 2.x API Reference*.

## GetRecommendations

The following code example shows how to use `GetRecommendations`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get a list of recommended items.

```
public static void getRecs(PersonalizeRuntimeClient personalizeRuntimeClient,
String campaignArn, String userId) {

 try {
 GetRecommendationsRequest recommendationsRequest =
 GetRecommendationsRequest.builder()
 .campaignArn(campaignArn)
 .numResults(20)
 .userId(userId)
 .build();

 GetRecommendationsResponse recommendationsResponse =
 personalizeRuntimeClient
 .getRecommendations(recommendationsRequest);
 List<PredictedItem> items = recommendationsResponse.itemList();
 for (PredictedItem item : items) {
 System.out.println("Item Id is : " + item.itemId());
 System.out.println("Item score is : " + item.score());
 }
 }
}
```

```

 }

 } catch (AwsServiceException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

Get a list of recommended items from a recommender created in a domain dataset group.

```

 public static void getRecs(PersonalizeRuntimeClient personalizeRuntimeClient,
 String recommenderArn,
 String userId) {

 try {
 GetRecommendationsRequest recommendationsRequest =
 GetRecommendationsRequest.builder()
 .recommenderArn(recommenderArn)
 .numResults(20)
 .userId(userId)
 .build();

 GetRecommendationsResponse recommendationsResponse =
 personalizeRuntimeClient
 .getRecommendations(recommendationsRequest);
 List<PredictedItem> items = recommendationsResponse.itemList();

 for (PredictedItem item : items) {
 System.out.println("Item Id is : " + item.itemId());
 System.out.println("Item score is : " + item.score());
 }
 } catch (AwsServiceException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Use a filter when requesting recommendations.

```

 public static void getFilteredRecs(PersonalizeRuntimeClient
 personalizeRuntimeClient,

```

```

 String campaignArn,
 String userId,
 String filterArn,
 String parameter1Name,
 String parameter1Value1,
 String parameter1Value2,
 String parameter2Name,
 String parameter2Value) {

 try {

 Map<String, String> filterValues = new HashMap<>();

 filterValues.put(parameter1Name, String.format("\"%1$s\", \"%2$s\"",
 parameter1Value1, parameter1Value2));
 filterValues.put(parameter2Name, String.format("\"%1$s\"",
 parameter2Value));

 GetRecommendationsRequest recommendationsRequest =
 GetRecommendationsRequest.builder()
 .campaignArn(campaignArn)
 .numResults(20)
 .userId(userId)
 .filterArn(filterArn)
 .filterValues(filterValues)
 .build();

 GetRecommendationsResponse recommendationsResponse =
 personalizeRuntimeClient
 .getRecommendations(recommendationsRequest);
 List<PredictedItem> items = recommendationsResponse.itemList();

 for (PredictedItem item : items) {
 System.out.println("Item Id is : " + item.itemId());
 System.out.println("Item score is : " + item.score());
 }
 } catch (PersonalizeRuntimeException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [GetRecommendations](#) in *AWS SDK for Java 2.x API Reference*.

# Amazon Pinpoint examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Pinpoint.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### CreateApp

The following code example shows how to use CreateApp.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.CreateAppRequest;
import software.amazon.awssdk.services.pinpoint.model.CreateAppResponse;
import software.amazon.awssdk.services.pinpoint.model.CreateApplicationRequest;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
```

```
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreateApp {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appName>

 Where:
 appName - The name of the application to create.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 String appName = args[0];
 System.out.println("Creating an application with name: " + appName);

 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String appID = createApplication(pinpoint, appName);
 System.out.println("App ID is: " + appID);
 pinpoint.close();
 }

 public static String createApplication(PinpointClient pinpoint, String appName)
 {
 try {
 CreateApplicationRequest appRequest = CreateApplicationRequest.builder()
 .name(appName)
 .build();

 CreateAppRequest request = CreateAppRequest.builder()
 .createApplicationRequest(appRequest)
 .build();

 CreateAppResponse result = pinpoint.createApp(request);
 return result.applicationResponse().id();
 }
 }
}
```

```

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [CreateApp](#) in *AWS SDK for Java 2.x API Reference*.

## CreateCampaign

The following code example shows how to use CreateCampaign.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create a campaign.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.CampaignResponse;
import software.amazon.awssdk.services.pinpoint.model.Message;
import software.amazon.awssdk.services.pinpoint.model.Schedule;
import software.amazon.awssdk.services.pinpoint.model.Action;
import software.amazon.awssdk.services.pinpoint.model.MessageConfiguration;
import software.amazon.awssdk.services.pinpoint.model.WriteCampaignRequest;
import software.amazon.awssdk.services.pinpoint.model.CreateCampaignResponse;
import software.amazon.awssdk.services.pinpoint.model.CreateCampaignRequest;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:

```

```
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreateCampaign {
 public static void main(String[] args) {

 final String usage = ""

 Usage: <appId> <segmentId>

 Where:
 appId - The ID of the application to create the campaign in.
 segmentId - The ID of the segment to create the campaign from.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 String segmentId = args[1];
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 createPinCampaign(pinpoint, appId, segmentId);
 pinpoint.close();
 }

 public static void createPinCampaign(PinpointClient pinpoint, String appId,
String segmentId) {
 CampaignResponse result = createCampaign(pinpoint, appId, segmentId);
 System.out.println("Campaign " + result.name() + " created.");
 System.out.println(result.description());
 }

 public static CampaignResponse createCampaign(PinpointClient client, String
appId, String segmentID) {

 try {
 Schedule schedule = Schedule.builder()
 .startTime("IMMEDIATE")
 .build();
```



```

 Message defaultMessage = Message.builder()
 .action(Action.OPEN_APP)
 .body("My message body.")
 .title("My message title.")
 .build();

 MessageConfiguration messageConfiguration =
MessageConfiguration.builder()
 .defaultMessage(defaultMessage)
 .build();

 WriteCampaignRequest request = WriteCampaignRequest.builder()
 .description("My description")
 .schedule(schedule)
 .name("MyCampaign")
 .segmentId(segmentID)
 .messageConfiguration(messageConfiguration)
 .build();

 CreateCampaignResponse result =
client.createCampaign(CreateCampaignRequest.builder()
 .applicationId(appID)
 .writeCampaignRequest(request).build());

 System.out.println("Campaign ID: " + result.campaignResponse().id());
 return result.campaignResponse();

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return null;
}
}

```

- For API details, see [CreateCampaign](#) in *AWS SDK for Java 2.x API Reference*.

## CreateExportJob

The following code example shows how to use `CreateExportJob`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

### Export an endpoint.

```
import software.amazon.awssdk.core.ResponseBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.ExportJobRequest;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpoint.model.CreateExportJobRequest;
import software.amazon.awssdk.services.pinpoint.model.CreateExportJobResponse;
import software.amazon.awssdk.services.pinpoint.model.GetExportJobResponse;
import software.amazon.awssdk.services.pinpoint.model.GetExportJobRequest;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Response;
import software.amazon.awssdk.services.s3.model.S3Object;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.stream.Collectors;

/**
 * To run this code example, you need to create an AWS Identity and Access
 * Management (IAM) role with the correct policy as described in this
 * documentation:
 * https://docs.aws.amazon.com/pinpoint/latest/developerguide/audience-data-
 * export.html
```

```

*
* Also, set up your development environment, including your credentials.
*
* For information, see this documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/

```

```

public class ExportEndpoints {
 public static void main(String[] args) {
 final String usage = ""

```

This program performs the following steps:

1. Exports the endpoints to an Amazon S3 bucket.
2. Downloads the exported endpoints files from Amazon S3.
3. Parses the endpoints files to obtain the endpoint IDs and prints

them.

Usage: ExportEndpoints <applicationId> <s3BucketName>  
<iamExportRoleArn> <path>

Where:

applicationId - The ID of the Amazon Pinpoint application that has the endpoint.

s3BucketName - The name of the Amazon S3 bucket to export the JSON file to.\s

iamExportRoleArn - The ARN of an IAM role that grants Amazon Pinpoint write permissions to the S3 bucket. path - The path where the files downloaded from the Amazon S3 bucket are written (for example, C:/AWS/).

"";

```

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

```

```

 String applicationId = args[0];
 String s3BucketName = args[1];
 String iamExportRoleArn = args[2];
 String path = args[3];
 System.out.println("Deleting an application with ID: " + applicationId);

```

```

 Region region = Region.US_EAST_1;
 PinpointClient pinpoint = PinpointClient.builder()

```

```

 .region(region)
 .build();

 S3Client s3Client = S3Client.builder()
 .region(region)
 .build();

 exportAllEndpoints(pinpoint, s3Client, applicationId, s3BucketName, path,
iamExportRoleArn);
 pinpoint.close();
 s3Client.close();
}

public static void exportAllEndpoints(PinpointClient pinpoint,
 S3Client s3Client,
 String applicationId,
 String s3BucketName,
 String path,
 String iamExportRoleArn) {

 try {
 List<String> objectKeys = exportEndpointsToS3(pinpoint, s3Client,
s3BucketName, iamExportRoleArn,
 applicationId);
 List<String> endpointFileKeys = objectKeys.stream().filter(o ->
o.endsWith(".gz"))
 .collect(Collectors.toList());
 downloadFromS3(s3Client, path, s3BucketName, endpointFileKeys);

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static List<String> exportEndpointsToS3(PinpointClient pinpoint, S3Client
s3Client, String s3BucketName,
 String iamExportRoleArn, String applicationId) {

 SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd-
HH_mm:ss.SSS_z");
 String endpointsKeyPrefix = "exports/" + applicationId + "_" +
dateFormat.format(new Date());

```

```

String s3UrlPrefix = "s3://" + s3BucketName + "/" + endpointsKeyPrefix +
"/";

List<String> objectKeys = new ArrayList<>();
String key;

try {
 // Defines the export job that Amazon Pinpoint runs.
 ExportJobRequest jobRequest = ExportJobRequest.builder()
 .roleArn(iamExportRoleArn)
 .s3UrlPrefix(s3UrlPrefix)
 .build();

 CreateExportJobRequest exportJobRequest =
CreateExportJobRequest.builder()
 .applicationId(applicationId)
 .exportJobRequest(jobRequest)
 .build();

 System.out.format("Exporting endpoints from Amazon Pinpoint application
%s to Amazon S3 " +
 "bucket %s . . .\n", applicationId, s3BucketName);

 CreateExportJobResponse exportResult =
pinpoint.createExportJob(exportJobRequest);
 String jobId = exportResult.exportJobResponse().id();
 System.out.println(jobId);
 printExportJobStatus(pinpoint, applicationId, jobId);

 ListObjectsV2Request v2Request = ListObjectsV2Request.builder()
 .bucket(s3BucketName)
 .prefix(endpointsKeyPrefix)
 .build();

 // Create a list of object keys.
 ListObjectsV2Response v2Response = s3Client.listObjectsV2(v2Request);
 List<S3Object> objects = v2Response.contents();
 for (S3Object object : objects) {
 key = object.key();
 objectKeys.add(key);
 }

 return objectKeys;

} catch (PinpointException e) {

```

```

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

private static void printExportJobStatus(PinpointClient pinpointClient,
 String applicationId,
 String jobId) {

 GetExportJobResponse getExportJobResult;
 String status;

 try {
 // Checks the job status until the job completes or fails.
 GetExportJobRequest exportJobRequest = GetExportJobRequest.builder()
 .jobId(jobId)
 .applicationId(applicationId)
 .build();

 do {
 getExportJobResult = pinpointClient.getExportJob(exportJobRequest);
 status =
getExportJobResult.exportJobResponse().jobStatus().toString().toUpperCase();
 System.out.format("Export job %s . . .\n", status);
 TimeUnit.SECONDS.sleep(3);

 } while (!status.equals("COMPLETED") && !status.equals("FAILED"));

 if (status.equals("COMPLETED")) {
 System.out.println("Finished exporting endpoints.");
 } else {
 System.err.println("Failed to export endpoints.");
 System.exit(1);
 }
 } catch (PinpointException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

// Download files from an Amazon S3 bucket and write them to the path location.

```

```

 public static void downloadFromS3(S3Client s3Client, String path, String
s3BucketName, List<String> objectKeys) {

 String newPath;
 try {
 for (String key : objectKeys) {
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .bucket(s3BucketName)
 .key(key)
 .build();

 ResponseBytes<GetObjectResponse> objectBytes =
s3Client.getObjectAsBytes(objectRequest);
 byte[] data = objectBytes.asByteArray();

 // Write the data to a local file.
 String fileSuffix = new
SimpleDateFormat("yyyyMMddHHmmss").format(new Date());
 newPath = path + fileSuffix + ".gz";
 File myFile = new File(newPath);
 OutputStream os = new FileOutputStream(myFile);
 os.write(data);
 }
 System.out.println("Download finished.");

 } catch (S3Exception | NullPointerException | IOException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [CreateExportJob](#) in *AWS SDK for Java 2.x API Reference*.

## CreateImportJob

The following code example shows how to use `CreateImportJob`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

### Import a segment.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.CreateImportJobRequest;
import software.amazon.awssdk.services.pinpoint.model.ImportJobResponse;
import software.amazon.awssdk.services.pinpoint.model.ImportJobRequest;
import software.amazon.awssdk.services.pinpoint.model.Format;
import software.amazon.awssdk.services.pinpoint.model.CreateImportJobResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ImportSegment {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId> <bucket> <key> <roleArn>\s

 Where:
 appId - The application ID to create a segment for.
 bucket - The name of the Amazon S3 bucket that contains the
segment definitons.
 key - The key of the S3 object.
 roleArn - ARN of the role that allows Amazon Pinpoint to
access S3. You need to set trust management for this to work. See https://
docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements_principal.html
""";
```



```
 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 String bucket = args[1];
 String key = args[2];
 String roleArn = args[3];

 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 ImportJobResponse response = createImportSegment(pinpoint, appId, bucket,
key, roleArn);
 System.out.println("Import job for " + bucket + " submitted.");
 System.out.println("See application " + response.applicationId() + " for
import job status.");
 System.out.println("See application " + response.jobStatus() + " for import
job status.");
 pinpoint.close();
 }

 public static ImportJobResponse createImportSegment(PinpointClient client,
 String appId,
 String bucket,
 String key,
 String roleArn) {

 try {
 ImportJobRequest importRequest = ImportJobRequest.builder()
 .defineSegment(true)
 .registerEndpoints(true)
 .roleArn(roleArn)
 .format(Format.JSON)
 .s3Url("s3://" + bucket + "/" + key)
 .build();

 CreateImportJobRequest jobRequest = CreateImportJobRequest.builder()
 .importJobRequest(importRequest)
 .applicationId(appId)
 .build();
```

```
 CreateImportJobResponse jobResponse =
client.createImportJob(jobRequest);
 return jobResponse.importJobResponse();

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}
}
```

- For API details, see [CreateImportJob](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSegment

The following code example shows how to use CreateSegment.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.AttributeDimension;
import software.amazon.awssdk.services.pinpoint.model.SegmentResponse;
import software.amazon.awssdk.services.pinpoint.model.AttributeType;
import software.amazon.awssdk.services.pinpoint.model.RecencyDimension;
import software.amazon.awssdk.services.pinpoint.model.SegmentBehaviors;
import software.amazon.awssdk.services.pinpoint.model.SegmentDemographics;
import software.amazon.awssdk.services.pinpoint.model.SegmentLocation;
import software.amazon.awssdk.services.pinpoint.model.SegmentDimensions;
import software.amazon.awssdk.services.pinpoint.model.WriteSegmentRequest;
import software.amazon.awssdk.services.pinpoint.model.CreateSegmentRequest;
import software.amazon.awssdk.services.pinpoint.model.CreateSegmentResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import java.util.HashMap;
```

```

import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateSegment {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId>

 Where:
 appId - The application ID to create a segment
for.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 SegmentResponse result = createSegment(pinpoint, appId);
 System.out.println("Segment " + result.name() + " created.");
 System.out.println(result.segmentType());
 pinpoint.close();
 }

 public static SegmentResponse createSegment(PinpointClient client, String
appId) {
 try {
 Map<String, AttributeDimension> segmentAttributes = new
HashMap<>();
 segmentAttributes.put("Team", AttributeDimension.builder()

```

```
 .attributeType(AttributeType.INCLUSIVE)
 .values("Lakers")
 .build());

 RecencyDimension recencyDimension =
RecencyDimension.builder()

 .duration("DAY_30")
 .recencyType("ACTIVE")
 .build();

 SegmentBehaviors segmentBehaviors =
SegmentBehaviors.builder()

 .recency(recencyDimension)
 .build();

 SegmentDemographics segmentDemographics =
SegmentDemographics

 .builder()
 .build();

 SegmentLocation segmentLocation = SegmentLocation
 .builder()
 .build();

 SegmentDimensions dimensions = SegmentDimensions
 .builder()
 .attributes(segmentAttributes)
 .behavior(segmentBehaviors)
 .demographic(segmentDemographics)
 .location(segmentLocation)
 .build();

 WriteSegmentRequest writeSegmentRequest =
WriteSegmentRequest.builder()

 .name("MySegment")
 .dimensions(dimensions)
 .build();

 CreateSegmentRequest createSegmentRequest =
CreateSegmentRequest.builder()

 .applicationId(appId)
 .writeSegmentRequest(writeSegmentRequest)
 .build();
```

```

 CreateSegmentResponse createSegmentResult =
client.createSegment(createSegmentRequest);
 System.out.println("Segment ID: " +
createSegmentResult.segmentResponse().id());
 System.out.println("Done");
 return createSegmentResult.segmentResponse();

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}
}

```

- For API details, see [CreateSegment](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteApp

The following code example shows how to use DeleteApp.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete an application.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.DeleteAppRequest;
import software.amazon.awssdk.services.pinpoint.model.DeleteAppResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 */

```

```

* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class DeleteApp {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId>

 Where:
 appId - The ID of the application to delete.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 System.out.println("Deleting an application with ID: " + appId);
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 deletePinApp(pinpoint, appId);
 System.out.println("Done");
 pinpoint.close();
 }

 public static void deletePinApp(PinpointClient pinpoint, String appId) {
 try {
 DeleteAppRequest appRequest = DeleteAppRequest.builder()
 .applicationId(appId)
 .build();

 DeleteAppResponse result = pinpoint.deleteApp(appRequest);
 String appName = result.applicationResponse().name();
 System.out.println("Application " + appName + " has been deleted.");

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

```
 }
 }
}
```

- For API details, see [DeleteApp](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteEndpoint

The following code example shows how to use DeleteEndpoint.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete an endpoint.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.DeleteEndpointRequest;
import software.amazon.awssdk.services.pinpoint.model.DeleteEndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteEndpoint {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appName> <endpointId >
```

```

 Where:
 appId - The id of the application to delete.
 endpointId - The id of the endpoint to delete.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 String endpointId = args[1];
 System.out.println("Deleting an endpoint with id: " + endpointId);
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 deletePinEncpoint(pinpoint, appId, endpointId);
 pinpoint.close();
}

public static void deletePinEncpoint(PinpointClient pinpoint, String appId,
String endpointId) {
 try {
 DeleteEndpointRequest appRequest = DeleteEndpointRequest.builder()
 .applicationId(appId)
 .endpointId(endpointId)
 .build();

 DeleteEndpointResponse result = pinpoint.deleteEndpoint(appRequest);
 String id = result.endpointResponse().id();
 System.out.println("The deleted endpoint id " + id);

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
}
}

```

- For API details, see [DeleteEndpoint](#) in *AWS SDK for Java 2.x API Reference*.



## GetEndpoint

The following code example shows how to use GetEndpoint.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import com.google.gson.FieldNamingPolicy;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.EndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.GetEndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpoint.model.GetEndpointRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class LookUpEndpoint {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId> <endpoint>

 Where:
 appId - The ID of the application to delete.
 endpoint - The ID of the endpoint.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 }
 }
}
```

```

 System.exit(1);
 }

 String appId = args[0];
 String endpoint = args[1];
 System.out.println("Looking up an endpoint point with ID: " + endpoint);
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 lookupPinpointEndpoint(pinpoint, appId, endpoint);
 pinpoint.close();
}

public static void lookupPinpointEndpoint(PinpointClient pinpoint, String appId,
String endpoint) {
 try {
 GetEndpointRequest appRequest = GetEndpointRequest.builder()
 .applicationId(appId)
 .endpointId(endpoint)
 .build();

 GetEndpointResponse result = pinpoint.getEndpoint(appRequest);
 EndpointResponse endResponse = result.endpointResponse();

 // Uses the Google Gson library to pretty print the endpoint JSON.
 Gson gson = new GsonBuilder()
 .setFieldNamingPolicy(FieldNamingPolicy.UPPER_CAMEL_CASE)
 .setPrettyPrinting()
 .create();

 String endpointJson = gson.toJson(endResponse);
 System.out.println(endpointJson);

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Done");
}
}

```

- For API details, see [GetEndpoint](#) in *AWS SDK for Java 2.x API Reference*.

## GetSegments

The following code example shows how to use GetSegments.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List segments.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.GetSegmentsRequest;
import software.amazon.awssdk.services.pinpoint.model.GetSegmentsResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpoint.model.SegmentResponse;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListSegments {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId>

 Where:
 appId - The ID of the application that contains a segment.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 }
 }
}
```

```

 System.exit(1);
 }

 String appId = args[0];
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listSegs(pinpoint, appId);
 pinpoint.close();
}

public static void listSegs(PinpointClient pinpoint, String appId) {
 try {
 GetSegmentsRequest request = GetSegmentsRequest.builder()
 .applicationId(appId)
 .build();

 GetSegmentsResponse response = pinpoint.getSegments(request);
 List<SegmentResponse> segments = response.segmentsResponse().item();
 for (SegmentResponse segment : segments) {
 System.out
 .println("Segment " + segment.id() + " " + segment.name() +
 " " + segment.lastModifiedDate());
 }

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [GetSegments](#) in *AWS SDK for Java 2.x API Reference*.

## GetSmsChannel

The following code example shows how to use `GetSmsChannel`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.SMSChannelResponse;
import software.amazon.awssdk.services.pinpoint.model.GetSmsChannelRequest;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpoint.model.SMSChannelRequest;
import software.amazon.awssdk.services.pinpoint.model.UpdateSmsChannelRequest;
import software.amazon.awssdk.services.pinpoint.model.UpdateSmsChannelResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class UpdateChannel {
 public static void main(String[] args) {
 final String usage = ""

 Usage: CreateChannel <appId>

 Where:
 appId - The name of the application whose channel is updated.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
```

```
PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

SMSChannelResponse getResponse = getSMSChannel(pinpoint, appId);
toggleSmsChannel(pinpoint, appId, getResponse);
pinpoint.close();
}

private static SMSChannelResponse getSMSChannel(PinpointClient client, String
appId) {
 try {
 GetSmsChannelRequest request = GetSmsChannelRequest.builder()
 .applicationId(appId)
 .build();

 SMSChannelResponse response =
client.getSmsChannel(request).smsChannelResponse();
 System.out.println("Channel state is " + response.enabled());
 return response;

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

private static void toggleSmsChannel(PinpointClient client, String appId,
SMSChannelResponse getResponse) {
 boolean enabled = !getResponse.enabled();
 try {
 SMSChannelRequest request = SMSChannelRequest.builder()
 .enabled(enabled)
 .build();

 UpdateSmsChannelRequest updateRequest =
UpdateSmsChannelRequest.builder()
 .smsChannelRequest(request)
 .applicationId(appId)
 .build();

 UpdateSmsChannelResponse result =
client.updateSmsChannel(updateRequest);
 }
```

```

 System.out.println("Channel state: " +
result.smsChannelResponse().enabled());

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [GetSmsChannel](#) in *AWS SDK for Java 2.x API Reference*.

## GetUserEndpoints

The following code example shows how to use GetUserEndpoints.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.EndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.GetUserEndpointsRequest;
import software.amazon.awssdk.services.pinpoint.model.GetUserEndpointsResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListEndpointIds {

```

```

public static void main(String[] args) {
 final String usage = ""

 Usage: <applicationId> <userId>

 Where:
 applicationId - The ID of the Amazon Pinpoint application that
has the endpoint.
 userId - The user id applicable to the endpoints""";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String applicationId = args[0];
 String userId = args[1];
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listAllEndpoints(pinpoint, applicationId, userId);
 pinpoint.close();
}

public static void listAllEndpoints(PinpointClient pinpoint,
 String applicationId,
 String userId) {

 try {
 GetUserEndpointsRequest endpointsRequest =
GetUserEndpointsRequest.builder()
 .userId(userId)
 .applicationId(applicationId)
 .build();

 GetUserEndpointsResponse response =
pinpoint.getUserEndpoints(endpointsRequest);
 List<EndpointResponse> endpoints = response.endpointsResponse().item();

 // Display the results.
 for (EndpointResponse endpoint : endpoints) {
 System.out.println("The channel type is: " +
endpoint.channelType());
 }
 }
}

```



```

 System.out.println("The address is " + endpoint.address());
 }

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [GetUserEndpoints](#) in *AWS SDK for Java 2.x API Reference*.

## SendMessages

The following code example shows how to use SendMessages.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Send an email message.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.AddressConfiguration;
import software.amazon.awssdk.services.pinpoint.model.ChannelType;
import software.amazon.awssdk.services.pinpoint.model.SimpleEmailPart;
import software.amazon.awssdk.services.pinpoint.model.SimpleEmail;
import software.amazon.awssdk.services.pinpoint.model.EmailMessage;
import software.amazon.awssdk.services.pinpoint.model.DirectMessageConfiguration;
import software.amazon.awssdk.services.pinpoint.model.MessageRequest;
import software.amazon.awssdk.services.pinpoint.model.SendMessagesRequest;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpointemail.PinpointEmailClient;
import software.amazon.awssdk.services.pinpointemail.model.Body;
import software.amazon.awssdk.services.pinpointemail.model.Content;
import software.amazon.awssdk.services.pinpointemail.model.Destination;

```

```

import software.amazon.awssdk.services.pinpointemail.model.EmailContent;
import software.amazon.awssdk.services.pinpointemail.model.Message;
import software.amazon.awssdk.services.pinpointemail.model.SendEmailRequest;

import java.util.HashMap;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendEmailMessage {

 // The character encoding the you want to use for the subject line and
 // message body of the email.
 public static String charset = "UTF-8";

 // The body of the email for recipients whose email clients support HTML
 content.
 static final String body = ""
 Amazon Pinpoint test (AWS SDK for Java 2.x)

 This email was sent through the Amazon Pinpoint Email API using the AWS SDK
 for Java 2.x

 "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <subject> <appId> <senderAddress>
<toAddress>

 Where:
 subject - The email subject to use.
 senderAddress - The from address. This address has to be verified in
Amazon Pinpoint in the region you're using to send email\s
 toAddress - The to address. This address has to be verified in Amazon
Pinpoint in the region you're using to send email\s
 "";

```

```
 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String subject = args[0];
 String senderAddress = args[1];
 String toAddress = args[2];
 System.out.println("Sending a message");
 PinpointEmailClient pinpoint = PinpointEmailClient.builder()
 .region(Region.US_EAST_1)
 .build();

 sendEmail(pinpoint, subject, senderAddress, toAddress);
 System.out.println("Email was sent");
 pinpoint.close();
 }

 public static void sendEmail(PinpointEmailClient pinpointEmailClient, String
subject, String senderAddress, String toAddress) {
 try {
 Content content = Content.builder()
 .data(body)
 .build();

 Body messageBody = Body.builder()
 .text(content)
 .build();

 Message message = Message.builder()
 .body(messageBody)
 .subject(Content.builder().data(subject).build())
 .build();

 Destination destination = Destination.builder()
 .toAddresses(toAddress)
 .build();

 EmailContent emailContent = EmailContent.builder()
 .simple(message)
 .build();

 SendEmailRequest sendEmailRequest = SendEmailRequest.builder()
```

```

 .fromEmailAddress(senderAddress)
 .destination(destination)
 .content(emailContent)
 .build();

 pinpointEmailClient.sendEmail(sendEmailRequest);
 System.out.println("Message Sent");

} catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
}

```

### Send an email message with CC values.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpointemail.PinpointEmailClient;
import software.amazon.awssdk.services.pinpointemail.model.Body;
import software.amazon.awssdk.services.pinpointemail.model.Content;
import software.amazon.awssdk.services.pinpointemail.model.Destination;
import software.amazon.awssdk.services.pinpointemail.model.EmailContent;
import software.amazon.awssdk.services.pinpointemail.model.Message;
import software.amazon.awssdk.services.pinpointemail.model.SendEmailRequest;
import java.util.ArrayList;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendEmailMessageCC {

 // The body of the email.
 static final String body = ""
 Amazon Pinpoint test (AWS SDK for Java 2.x)
 ;
}

```

This email was sent through the Amazon Pinpoint Email API using the AWS SDK for Java 2.x

```

 """;
 public static void main(String[] args) {
 final String usage = ""

 Usage: <subject> <senderAddress> <toAddress> <ccAddress>

 Where:
 subject - The email subject to use.
 senderAddress - The from address. This address has to be verified in
Amazon Pinpoint in the region you're using to send email\s
 toAddress - The to address. This address has to be verified in Amazon
Pinpoint in the region you're using to send email\s
 ccAddress - The CC address.
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String subject = args[0];
 String senderAddress = args[1];
 String toAddress = args[2];
 String ccAddress = args[3];

 System.out.println("Sending a message");
 PinpointEmailClient pinpoint = PinpointEmailClient.builder()
 .region(Region.US_EAST_1)
 .build();

 ArrayList<String> ccList = new ArrayList<>();
 ccList.add(ccAddress);
 sendEmail(pinpoint, subject, senderAddress, toAddress, ccList);
 pinpoint.close();
 }

 public static void sendEmail(PinpointEmailClient pinpointEmailClient, String
subject, String senderAddress, String toAddress, ArrayList<String> ccAddresses) {
 try {
 Content content = Content.builder()
 .data(body)

```

```

 .build();

 Body messageBody = Body.builder()
 .text(content)
 .build();

 Message message = Message.builder()
 .body(messageBody)
 .subject(Content.builder().data(subject).build())
 .build();

 Destination destination = Destination.builder()
 .toAddresses(toAddress)
 .ccAddresses(ccAddresses)
 .build();

 EmailContent emailContent = EmailContent.builder()
 .simple(message)
 .build();

 SendEmailRequest sendEmailRequest = SendEmailRequest.builder()
 .fromEmailAddress(senderAddress)
 .destination(destination)
 .content(emailContent)
 .build();

 pinpointEmailClient.sendEmail(sendEmailRequest);
 System.out.println("Message Sent");

 } catch (PinpointException e) {
 // Handle exception
 e.printStackTrace();
 }
}

```

## Send an SMS message.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.DirectMessageConfiguration;
import software.amazon.awssdk.services.pinpoint.model.SMSMessage;

```

```

import software.amazon.awssdk.services.pinpoint.model.AddressConfiguration;
import software.amazon.awssdk.services.pinpoint.model.ChannelType;
import software.amazon.awssdk.services.pinpoint.model.MessageRequest;
import software.amazon.awssdk.services.pinpoint.model.SendMessagesRequest;
import software.amazon.awssdk.services.pinpoint.model.SendMessagesResponse;
import software.amazon.awssdk.services.pinpoint.model.MessageResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import java.util.HashMap;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendMessage {

 // The type of SMS message that you want to send. If you plan to send
 // time-sensitive content, specify TRANSACTIONAL. If you plan to send
 // marketing-related content, specify PROMOTIONAL.
 public static String messageType = "TRANSACTIONAL";

 // The registered keyword associated with the originating short code.
 public static String registeredKeyword = "myKeyword";

 // The sender ID to use when sending the message. Support for sender ID
 // varies by country or region. For more information, see
 // https://docs.aws.amazon.com/pinpoint/latest/userguide/channels-sms-countries.html
 public static String senderId = "MySenderId";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <message> <appId> <originationNumber>
 <destinationNumber>\s

 Where:
 message - The body of the message to send.
 appId - The Amazon Pinpoint project/application ID
to use when you send this message.

```

originationNumber - The phone number or short code that you specify has to be associated with your Amazon Pinpoint account. For best results, specify long codes in E.164 format (for example, +1-555-555-5654).

destinationNumber - The recipient's phone number. For best results, you should specify the phone number in E.164 format (for example, +1-555-555-5654).\s

```
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String message = args[0];
 String appId = args[1];
 String originationNumber = args[2];
 String destinationNumber = args[3];
 System.out.println("Sending a message");
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 sendSMSMessage(pinpoint, message, appId, originationNumber,
 destinationNumber);
 pinpoint.close();
 }

 public static void sendSMSMessage(PinpointClient pinpoint, String message,
 String appId,
 String originationNumber,
 String destinationNumber) {
 try {
 Map<String, AddressConfiguration> addressMap = new
 HashMap<String, AddressConfiguration>();
 AddressConfiguration addConfig =
 AddressConfiguration.builder()
 .channelType(ChannelType.SMS)
 .build();

 addressMap.put(destinationNumber, addConfig);
 SMSMessage smsMessage = SMSMessage.builder()
 .body(message)
 .messageType(messageType)
 .originationNumber(originationNumber)
```



```

 .senderId(senderId)
 .keyword(registeredKeyword)
 .build();

 // Create a DirectMessageConfiguration object.
 DirectMessageConfiguration direct =
DirectMessageConfiguration.builder()

 .smsMessage(smsMessage)
 .build();

 MessageRequest msgReq = MessageRequest.builder()
 .addresses(addressMap)
 .messageConfiguration(direct)
 .build();

 // create a SendMessagesRequest object
 SendMessagesRequest request = SendMessagesRequest.builder()
 .applicationId(appId)
 .messageRequest(msgReq)
 .build();

 SendMessagesResponse response =
pinpoint.sendMessages(request);
 MessageResponse msg1 = response.messageResponse();
 Map map1 = msg1.result();

 // Write out the result of sendMessage.
 map1.forEach((k, v) -> System.out.println((k + ":" + v)));

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

## Send batch SMS messages.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.DirectMessageConfiguration;

```

```

import software.amazon.awssdk.services.pinpoint.model.SMSMessage;
import software.amazon.awssdk.services.pinpoint.model.AddressConfiguration;
import software.amazon.awssdk.services.pinpoint.model.ChannelType;
import software.amazon.awssdk.services.pinpoint.model.MessageRequest;
import software.amazon.awssdk.services.pinpoint.model.SendMessagesRequest;
import software.amazon.awssdk.services.pinpoint.model.SendMessagesResponse;
import software.amazon.awssdk.services.pinpoint.model.MessageResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;

import java.util.HashMap;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendMessageBatch {

 // The type of SMS message that you want to send. If you plan to send
 // time-sensitive content, specify TRANSACTIONAL. If you plan to send
 // marketing-related content, specify PROMOTIONAL.
 public static String messageType = "TRANSACTIONAL";

 // The registered keyword associated with the originating short code.
 public static String registeredKeyword = "myKeyword";

 // The sender ID to use when sending the message. Support for sender ID
 // varies by country or region. For more information, see
 // https://docs.aws.amazon.com/pinpoint/latest/userguide/channels-sms-countries.html
 public static String senderId = "MySenderId";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <message> <appId> <originationNumber> <destinationNumber>
<destinationNumber1>\s

 Where:
 message - The body of the message to send.

```

appId - The Amazon Pinpoint project/application ID to use when you send this message.

originationNumber - The phone number or short code that you specify has to be associated with your Amazon Pinpoint account. For best results, specify long codes in E.164 format (for example, +1-555-555-5654).

destinationNumber - The recipient's phone number. For best results, you should specify the phone number in E.164 format (for example, +1-555-555-5654).

destinationNumber1 - The second recipient's phone number. For best results, you should specify the phone number in E.164 format (for example, +1-555-555-5654).\s

```
""";
```

```
if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
}
```

```
String message = args[0];
String appId = args[1];
String originationNumber = args[2];
String destinationNumber = args[3];
String destinationNumber1 = args[4];
System.out.println("Sending a message");
PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();
```

```
sendSMSMessage(pinpoint, message, appId, originationNumber,
destinationNumber, destinationNumber1);
pinpoint.close();
}
```

```
public static void sendSMSMessage(PinpointClient pinpoint, String message,
String appId,
 String originationNumber,
 String destinationNumber, String
destinationNumber1) {
 try {
 Map<String, AddressConfiguration> addressMap = new HashMap<String,
AddressConfiguration>();
 AddressConfiguration addConfig = AddressConfiguration.builder()
 .channelType(ChannelType.SMS)
 .build();
```

```

// Add an entry to the Map object for each number to whom you want to
send a
// message.
addressMap.put(destinationNumber, addConfig);
addressMap.put(destinationNumber1, addConfig);
SMSMessage smsMessage = SMSMessage.builder()
 .body(message)
 .messageType(messageType)
 .originationNumber(originationNumber)
 .senderId(senderId)
 .keyword(registeredKeyword)
 .build();

// Create a DirectMessageConfiguration object.
DirectMessageConfiguration direct = DirectMessageConfiguration.builder()
 .smsMessage(smsMessage)
 .build();

MessageRequest msgReq = MessageRequest.builder()
 .addresses(addressMap)
 .messageConfiguration(direct)
 .build();

// Create a SendMessagesRequest object.
SendMessagesRequest request = SendMessagesRequest.builder()
 .applicationId(appId)
 .messageRequest(msgReq)
 .build();

SendMessagesResponse response = pinpoint.sendMessages(request);
MessageResponse msg1 = response.messageResponse();
Map map1 = msg1.result();

// Write out the result of sendMessage.
map1.forEach((k, v) -> System.out.println((k + ":" + v)));

} catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
}

```

- For API details, see [SendMessages](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateEndpoint

The following code example shows how to use UpdateEndpoint.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.pinpoint.PinpointClient;
import software.amazon.awssdk.services.pinpoint.model.EndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.EndpointRequest;
import software.amazon.awssdk.services.pinpoint.model.UpdateEndpointRequest;
import software.amazon.awssdk.services.pinpoint.model.UpdateEndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.GetEndpointRequest;
import software.amazon.awssdk.services.pinpoint.model.GetEndpointResponse;
import software.amazon.awssdk.services.pinpoint.model.PinpointException;
import software.amazon.awssdk.services.pinpoint.model.EndpointDemographic;
import software.amazon.awssdk.services.pinpoint.model.EndpointLocation;
import software.amazon.awssdk.services.pinpoint.model.EndpointUser;
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.List;
import java.util.UUID;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
import java.util.Date;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
```

```

*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class UpdateEndpoint {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <appId>

 Where:
 appId - The ID of the application to create an endpoint for.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String appId = args[0];
 PinpointClient pinpoint = PinpointClient.builder()
 .region(Region.US_EAST_1)
 .build();

 EndpointResponse response = createEndpoint(pinpoint, appId);
 System.out.println("Got Endpoint: " + response.id());
 pinpoint.close();
 }

 public static EndpointResponse createEndpoint(PinpointClient client, String
appId) {
 String endpointId = UUID.randomUUID().toString();
 System.out.println("Endpoint ID: " + endpointId);

 try {
 EndpointRequest endpointRequest = createEndpointRequestData();
 UpdateEndpointRequest updateEndpointRequest =
UpdateEndpointRequest.builder()
 .applicationId(appId)
 .endpointId(endpointId)
 .endpointRequest(endpointRequest)
 .build();

```

```

 UpdateEndpointResponse updateEndpointResponse =
client.updateEndpoint(updateEndpointRequest);
 System.out.println("Update Endpoint Response: " +
updateEndpointResponse.messageBody());

 GetEndpointRequest getEndpointRequest = GetEndpointRequest.builder()
 .applicationId(appId)
 .endpointId(endpointId)
 .build();

 GetEndpointResponse getEndpointResponse =
client.getEndpoint(getEndpointRequest);
 System.out.println(getEndpointResponse.endpointResponse().address());

System.out.println(getEndpointResponse.endpointResponse().channelType());

System.out.println(getEndpointResponse.endpointResponse().applicationId());

System.out.println(getEndpointResponse.endpointResponse().endpointStatus());
 System.out.println(getEndpointResponse.endpointResponse().requestId());
 System.out.println(getEndpointResponse.endpointResponse().user());

 return getEndpointResponse.endpointResponse();

 } catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

private static EndpointRequest createEndpointRequestData() {
 try {
 List<String> favoriteTeams = new ArrayList<>();
 favoriteTeams.add("Lakers");
 favoriteTeams.add("Warriors");
 HashMap<String, List<String>> customAttributes = new HashMap<>();
 customAttributes.put("team", favoriteTeams);

 EndpointDemographic demographic = EndpointDemographic.builder()
 .appVersion("1.0")
 .make("apple")
 .model("iPhone")
 .modelVersion("7")

```

```

 .platform("ios")
 .platformVersion("10.1.1")
 .timezone("America/Los_Angeles")
 .build();

 EndpointLocation location = EndpointLocation.builder()
 .city("Los Angeles")
 .country("US")
 .latitude(34.0)
 .longitude(-118.2)
 .postalCode("90068")
 .region("CA")
 .build();

 Map<String, Double> metrics = new HashMap<>();
 metrics.put("health", 100.00);
 metrics.put("luck", 75.00);

 EndpointUser user = EndpointUser.builder()
 .userId(UUID.randomUUID().toString())
 .build();

 DateFormat df = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm'Z'"); // Quoted
 "Z" to indicate UTC, no timezone // offset

 String nowAsISO = df.format(new Date());

 return EndpointRequest.builder()
 .address(UUID.randomUUID().toString())
 .attributes(customAttributes)
 .channelType("APNS")
 .demographic(demographic)
 .effectiveDate(nowAsISO)
 .location(location)
 .metrics(metrics)
 .optOut("NONE")
 .requestId(UUID.randomUUID().toString())
 .user(user)
 .build();

} catch (PinpointException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

```



```
 return null;
 }
}
```

- For API details, see [UpdateEndpoint](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Pinpoint SMS and Voice API examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Pinpoint SMS and Voice API.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### SendVoiceMessage

The following code example shows how to use `SendVoiceMessage`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.regions.Region;
```

```

import software.amazon.awssdk.services.pinpointsmsvoice.PinpointSmsVoiceClient;
import software.amazon.awssdk.services.pinpointsmsvoice.model.SSMLMessageType;
import software.amazon.awssdk.services.pinpointsmsvoice.model.VoiceMessageContent;
import
 software.amazon.awssdk.services.pinpointsmsvoice.model.SendVoiceMessageRequest;
import
 software.amazon.awssdk.services.pinpointsmsvoice.model.PinpointSmsVoiceException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendVoiceMessage {

 // The Amazon Polly voice that you want to use to send the message. For a list
 // of voices, see https://docs.aws.amazon.com/polly/latest/dg/voicelist.html
 static final String voiceName = "Matthew";

 // The language to use when sending the message. For a list of supported
 // languages, see
 // https://docs.aws.amazon.com/polly/latest/dg/SupportedLanguage.html
 static final String languageCode = "en-US";

 // The content of the message. This example uses SSML to customize and control
 // certain aspects of the message, such as by adding pauses and changing
 // phonation. The message can't contain any line breaks.
 static final String ssmlMessage = "<speack>This is a test message sent from "
 + "<emphasis>Amazon Pinpoint</emphasis> "
 + "using the <break strength='weak'/>AWS "
 + "SDK for Java. "
 + "<amazon:effect phonation='soft'>Thank "
 + "you for listening.</amazon:effect></speack>";

 public static void main(String[] args) {

```

```

final String usage = ""
 Usage: <originationNumber> <destinationNumber>\s

 Where:
 originationNumber - The phone number or short code that you
specify has to be associated with your Amazon Pinpoint account. For best results,
specify long codes in E.164 format (for example, +1-555-555-5654).
 destinationNumber - The recipient's phone number. For best
results, you should specify the phone number in E.164 format (for example,
+1-555-555-5654).\s
 "";

if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
}
String originationNumber = args[0];
String destinationNumber = args[1];
System.out.println("Sending a voice message");

// Set the content type to application/json.
List<String> listVal = new ArrayList<>();
listVal.add("application/json");
Map<String, List<String>> values = new HashMap<>();
values.put("Content-Type", listVal);

ClientOverrideConfiguration config2 = ClientOverrideConfiguration.builder()
 .headers(values)
 .build();

PinpointSmsVoiceClient client = PinpointSmsVoiceClient.builder()
 .overrideConfiguration(config2)
 .region(Region.US_EAST_1)
 .build();

sendVoiceMsg(client, originationNumber, destinationNumber);
client.close();
}

public static void sendVoiceMsg(PinpointSmsVoiceClient client, String
originationNumber,
 String destinationNumber) {
 try {
 SSMLMessageType ssmlMessageType = SSMLMessageType.builder()

```

```
 .languageCode(languageCode)
 .text(ssmlMessage)
 .voiceId(voiceName)
 .build();

 VoiceMessageContent content = VoiceMessageContent.builder()
 .ssmlMessage(ssmlMessageType)
 .build();

 SendVoiceMessageRequest voiceMessageRequest =
 SendVoiceMessageRequest.builder()
 .destinationPhoneNumber(destinationNumber)
 .originationPhoneNumber(originationNumber)
 .content(content)
 .build();

 client.sendVoiceMessage(voiceMessageRequest);
 System.out.println("The message was sent successfully.");

 } catch (PinpointSmsVoiceException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [SendVoiceMessage](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Polly examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Polly.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### DescribeVoices

The following code example shows how to use DescribeVoices.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.polly.PollyClient;
import software.amazon.awssdk.services.polly.model.DescribeVoicesRequest;
import software.amazon.awssdk.services.polly.model.DescribeVoicesResponse;
import software.amazon.awssdk.services.polly.model.PollyException;
import software.amazon.awssdk.services.polly.model.Voice;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeVoicesSample {
 public static void main(String args[]) {
 PollyClient polly = PollyClient.builder()
 .region(Region.US_WEST_2)
 .build();

 describeVoice(polly);
 }
}
```

```
 polly.close();
 }

 public static void describeVoice(PollyClient polly) {
 try {
 DescribeVoicesRequest voicesRequest = DescribeVoicesRequest.builder()
 .languageCode("en-US")
 .build();

 DescribeVoicesResponse enUsVoicesResult =
polly.describeVoices(vocesRequest);
 List<Voice> voices = enUsVoicesResult.voices();
 for (Voice myVoice : voices) {
 System.out.println("The ID of the voice is " + myVoice.id());
 System.out.println("The gender of the voice is " +
myVoice.gender());
 }

 } catch (PollyException e) {
 System.err.println("Exception caught: " + e);
 System.exit(1);
 }
 }
}
```

- For API details, see [DescribeVoices](#) in *AWS SDK for Java 2.x API Reference*.

## ListLexicons

The following code example shows how to use `ListLexicons`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.polly.PollyClient;
```

```
import software.amazon.awssdk.services.polly.model.ListLexiconsResponse;
import software.amazon.awssdk.services.polly.model.ListLexiconsRequest;
import software.amazon.awssdk.services.polly.model.LexiconDescription;
import software.amazon.awssdk.services.polly.model.PollyException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListLexicons {
 public static void main(String args[]) {
 PollyClient polly = PollyClient.builder()
 .region(Region.US_WEST_2)
 .build();

 listLexicons(polly);
 polly.close();
 }

 public static void listLexicons(PollyClient client) {
 try {
 ListLexiconsRequest listLexiconsRequest = ListLexiconsRequest.builder()
 .build();

 ListLexiconsResponse listLexiconsResult =
client.listLexicons(listLexiconsRequest);
 List<LexiconDescription> lexiconDescription =
listLexiconsResult.lexicons();
 for (LexiconDescription lexDescription : lexiconDescription) {
 System.out.println("The name of the Lexicon is " +
lexDescription.name());
 }

 } catch (PollyException e) {
 System.err.println("Exception caught: " + e);
 System.exit(1);
 }
 }
}
```

- For API details, see [ListLexicons](#) in *AWS SDK for Java 2.x API Reference*.

## SynthesizeSpeech

The following code example shows how to use SynthesizeSpeech.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import javazoom.jl.decoder.JavaLayerException;
import software.amazon.awssdk.core.ResponseInputStream;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.polly.PollyClient;
import software.amazon.awssdk.services.polly.model.DescribeVoicesRequest;
import software.amazon.awssdk.services.polly.model.Voice;
import software.amazon.awssdk.services.polly.model.DescribeVoicesResponse;
import software.amazon.awssdk.services.polly.model.OutputFormat;
import software.amazon.awssdk.services.polly.model.PollyException;
import software.amazon.awssdk.services.polly.model.SynthesizeSpeechRequest;
import software.amazon.awssdk.services.polly.model.SynthesizeSpeechResponse;
import java.io.IOException;
import java.io.InputStream;
import javazoom.jl.player.advanced.AdvancedPlayer;
import javazoom.jl.player.advanced.PlaybackEvent;
import javazoom.jl.player.advanced.PlaybackListener;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PollyDemo {
```



```

 private static final String SAMPLE = "Congratulations. You have successfully
built this working demo " +
 " of Amazon Polly in Java Version 2. Have fun building voice enabled
apps with Amazon Polly (that's me!), and always "
 +
 " look at the AWS website for tips and tricks on using Amazon Polly and
other great services from AWS";

 public static void main(String args[]) {
 PollyClient polly = PollyClient.builder()
 .region(Region.US_WEST_2)
 .build();

 talkPolly(polly);
 polly.close();
 }

 public static void talkPolly(PollyClient polly) {
 try {
 DescribeVoicesRequest describeVoiceRequest =
DescribeVoicesRequest.builder()
 .engine("standard")
 .build();

 DescribeVoicesResponse describeVoicesResult =
polly.describeVoices(describeVoiceRequest);
 Voice voice = describeVoicesResult.voices().stream()
 .filter(v -> v.name().equals("Joanna"))
 .findFirst()
 .orElseThrow(() -> new RuntimeException("Voice not found"));
 InputStream stream = synthesize(polly, SAMPLE, voice, OutputFormat.MP3);
 AdvancedPlayer player = new AdvancedPlayer(stream,

javazoom.jl.player.FactoryRegistry.systemRegistry().createAudioDevice());
 player.setPlayBackListener(new PlaybackListener() {
 public void playbackStarted(PlaybackEvent evt) {
 System.out.println("Playback started");
 System.out.println(SAMPLE);
 }

 public void playbackFinished(PlaybackEvent evt) {
 System.out.println("Playback finished");
 }
 });
 }
 }

```

```

 // play it!
 player.play();

 } catch (PollyException | JavaLayerException | IOException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static InputStream synthesize(PollyClient polly, String text, Voice
voice, OutputFormat format)
 throws IOException {
 SynthesizeSpeechRequest synthReq = SynthesizeSpeechRequest.builder()
 .text(text)
 .voiceId(voice.id())
 .outputFormat(format)
 .build();

 ResponseInputStream<SynthesizeSpeechResponse> synthRes =
polly.synthesizeSpeech(synthReq);
 return synthRes;
}
}

```

- For API details, see [SynthesizeSpeech](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create an application to analyze customer feedback

The following code example shows how to create an application that analyzes customer comment cards, translates them from their original language, determines their sentiment, and generates an audio file from the translated text.

#### SDK for Java 2.x

This example application analyzes and stores customer feedback cards. Specifically, it fulfills the need of a fictitious hotel in New York City. The hotel receives feedback from guests in various languages in the form of physical comment cards. That feedback is uploaded into the app through a web client. After an image of a comment card is uploaded, the following steps occur:

- Text is extracted from the image using Amazon Textract.
- Amazon Comprehend determines the sentiment of the extracted text and its language.
- The extracted text is translated to English using Amazon Translate.
- Amazon Polly synthesizes an audio file from the extracted text.

The full app can be deployed with the AWS CDK. For source code and deployment instructions, see the project in [GitHub](#).

### Services used in this example

- Amazon Comprehend
- Lambda
- Amazon Polly
- Amazon Textract
- Amazon Translate

## Amazon RDS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon RDS.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)

# Get started

## Hello Amazon RDS

The following code example shows how to get started using Amazon RDS.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesResponse;
import software.amazon.awssdk.services.rds.model.DBInstance;
import software.amazon.awssdk.services.rds.model.RdsException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeDBInstances {

 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 describeInstances(rdsClient);
 rdsClient.close();
 }

 public static void describeInstances(RdsClient rdsClient) {
```

```
 try {
 DescribeDbInstancesResponse response = rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 for (DBInstance instance : instanceList) {
 System.out.println("Instance ARN is: " + instance.dbInstanceArn());
 System.out.println("The Engine is " + instance.engine());
 System.out.println("Connection endpoint is" +
instance.endpoint().address());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DescribeDBInstances](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a custom DB parameter group and set parameter values.
- Create a DB instance that's configured to use the parameter group. The DB instance also contains a database.
- Take a snapshot of the instance.
- Delete the instance and parameter group.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Run multiple operations.

```
import com.google.gson.Gson;
import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.CreateDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.CreateDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.CreateDbParameterGroupResponse;
import software.amazon.awssdk.services.rds.model.CreateDbSnapshotRequest;
import software.amazon.awssdk.services.rds.model.CreateDbSnapshotResponse;
import software.amazon.awssdk.services.rds.model.DBEngineVersion;
import software.amazon.awssdk.services.rds.model.DBInstance;
import software.amazon.awssdk.services.rds.model.DBParameterGroup;
import software.amazon.awssdk.services.rds.model.DBSnapshot;
import software.amazon.awssdk.services.rds.model.DeleteDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.DeleteDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.DescribeDbEngineVersionsRequest;
import software.amazon.awssdk.services.rds.model.DescribeDbEngineVersionsResponse;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesRequest;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesResponse;
import software.amazon.awssdk.services.rds.model.DescribeDbParameterGroupsResponse;
import software.amazon.awssdk.services.rds.model.DescribeDbParametersResponse;
import software.amazon.awssdk.services.rds.model.DescribeDbSnapshotsRequest;
import software.amazon.awssdk.services.rds.model.DescribeDbSnapshotsResponse;
import
 software.amazon.awssdk.services.rds.model.DescribeOrderableDbInstanceOptionsResponse;
import software.amazon.awssdk.services.rds.model.ModifyDbParameterGroupResponse;
import software.amazon.awssdk.services.rds.model.OrderableDBInstanceOption;
import software.amazon.awssdk.services.rds.model.Parameter;
import software.amazon.awssdk.services.rds.model.RdsException;
import software.amazon.awssdk.services.rds.model.CreateDbParameterGroupRequest;
import software.amazon.awssdk.services.rds.model.DescribeDbParameterGroupsRequest;
import software.amazon.awssdk.services.rds.model.DescribeDbParametersRequest;
import software.amazon.awssdk.services.rds.model.ModifyDbParameterGroupRequest;
import
 software.amazon.awssdk.services.rds.model.DescribeOrderableDbInstanceOptionsRequest;
import software.amazon.awssdk.services.rds.model.DeleteDbParameterGroupRequest;
import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;
import java.util.ArrayList;
import java.util.List;
```

```

/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This example requires an AWS Secrets Manager secret that contains the
 * database credentials. If you do not create a
 * secret, this example will not work. For details, see:
 *
 * https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_how-services-use-secrets_RS.html
 *
 * This Java example performs these tasks:
 *
 * 1. Returns a list of the available DB engines.
 * 2. Selects an engine family and create a custom DB parameter group.
 * 3. Gets the parameter groups.
 * 4. Gets parameters in the group.
 * 5. Modifies the auto_increment_offset parameter.
 * 6. Gets and displays the updated parameters.
 * 7. Gets a list of allowed engine versions.
 * 8. Gets a list of micro instance classes available for the selected engine.
 * 9. Creates an RDS database instance that contains a MySQL database and uses
 * the parameter group.
 * 10. Waits for the DB instance to be ready and prints out the connection
 * endpoint value.
 * 11. Creates a snapshot of the DB instance.
 * 12. Waits for an RDS DB snapshot to be ready.
 * 13. Deletes the RDS DB instance.
 * 14. Deletes the parameter group.
 */
public class RDSScenario {
 public static long sleepTime = 20;
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws InterruptedException {
 final String usage = ""

 Usage:

```

```

 <dbGroupName> <dbParameterGroupFamily> <dbInstanceIdentifier>
<dbName> <dbSnapshotIdentifier> <secretName>

 Where:
 dbGroupName - The database group name.\s
 dbParameterGroupFamily - The database parameter group name (for
example, mysql8.0).
 dbInstanceIdentifier - The database instance identifier\s
 dbName - The database name.\s
 dbSnapshotIdentifier - The snapshot identifier.\s
 secretName - The name of the AWS Secrets Manager secret that
contains the database credentials"
 """;

 if (args.length != 6) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbGroupName = args[0];
 String dbParameterGroupFamily = args[1];
 String dbInstanceIdentifier = args[2];
 String dbName = args[3];
 String dbSnapshotIdentifier = args[4];
 String secretName = args[5];

 Gson gson = new Gson();
 User user = gson.fromJson(String.valueOf(getSecretValues(secretName)),
User.class);
 String masterUsername = user.getUsername();
 String masterUserPassword = user.getPassword();

 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();
 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon RDS example scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("1. Return a list of the available DB engines");
 describeDBEngines(rdsClient);
 System.out.println(DASHES);

```



```
System.out.println(DASHES);
System.out.println("2. Create a custom parameter group");
createDBParameterGroup(rdsClient, dbGroupName, dbParameterGroupFamily);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Get the parameter group");
describeDbParameterGroups(rdsClient, dbGroupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Get the parameters in the group");
describeDbParameters(rdsClient, dbGroupName, 0);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Modify the auto_increment_offset parameter");
modifyDBParas(rdsClient, dbGroupName);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Display the updated value");
describeDbParameters(rdsClient, dbGroupName, -1);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Get a list of allowed engine versions");
getAllowedEngines(rdsClient, dbParameterGroupFamily);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("8. Get a list of micro instance classes available for
the selected engine");
getMicroInstances(rdsClient);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println(
 "9. Create an RDS database instance that contains a MySQL database
and uses the parameter group");
String dbARN = createDatabaseInstance(rdsClient, dbGroupName,
dbInstanceIdentifier, dbName, masterUsername,
 masterUserPassword);
```

```
 System.out.println("The ARN of the new database is " + dbARN);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Wait for DB instance to be ready");
 waitForInstanceReady(rdsClient, dbInstanceIdentifier);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("11. Create a snapshot of the DB instance");
 createSnapshot(rdsClient, dbInstanceIdentifier, dbSnapshotIdentifier);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("12. Wait for DB snapshot to be ready");
 waitForSnapshotReady(rdsClient, dbInstanceIdentifier, dbSnapshotIdentifier);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("13. Delete the DB instance");
 deleteDatabaseInstance(rdsClient, dbInstanceIdentifier);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("14. Delete the parameter group");
 deleteParaGroup(rdsClient, dbGroupName, dbARN);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The Scenario has successfully completed.");
 System.out.println(DASHES);

 rdsClient.close();
 }

 private static SecretsManagerClient getSecretClient() {
 Region region = Region.US_WEST_2;
 return SecretsManagerClient.builder()
 .region(region)
 .build();
 }

 public static String getSecretValues(String secretName) {
 SecretsManagerClient secretClient = getSecretClient();
```

```

 GetSecretValueRequest valueRequest = GetSecretValueRequest.builder()
 .secretId(secretName)
 .build();

 GetSecretValueResponse valueResponse =
secretClient.getSecretValue(valueRequest);
 return valueResponse.secretString();
 }

 // Delete the parameter group after database has been deleted.
 // An exception is thrown if you attempt to delete the para group while database
 // exists.
 public static void deleteParaGroup(RdsClient rdsClient, String dbGroupName,
String dbARN)
 throws InterruptedException {
 try {
 boolean isDataDel = false;
 boolean didFind;
 String instanceARN;

 // Make sure that the database has been deleted.
 while (!isDataDel) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 int listSize = instanceList.size();
 didFind = false;
 int index = 1;
 for (DBInstance instance : instanceList) {
 instanceARN = instance.dbInstanceArn();
 if (instanceARN.compareTo(dbARN) == 0) {
 System.out.println(dbARN + " still exists");
 didFind = true;
 }
 if ((index == listSize) && (!didFind)) {
 // Went through the entire list and did not find the
database ARN.

 isDataDel = true;
 }
 Thread.sleep(sleepTime * 1000);
 index++;
 }
 }
 }
 }

```

```
// Delete the para group.
DeleteDbParameterGroupRequest parameterGroupRequest =
DeleteDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .build();

rdsClient.deleteDBParameterGroup(parameterGroupRequest);
System.out.println(dbGroupName + " was deleted.");

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}

}

// Delete the DB instance.
public static void deleteDatabaseInstance(RdsClient rdsClient, String
dbInstanceIdIdentifier) {
 try {
 DeleteDbInstanceRequest deleteDbInstanceRequest =
DeleteDbInstanceRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .deleteAutomatedBackups(true)
 .skipFinalSnapshot(true)
 .build();

 DeleteDbInstanceResponse response =
rdsClient.deleteDBInstance(deleteDbInstanceRequest);
 System.out.print("The status of the database is " +
response.dbInstance().dbInstanceStatus());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

// Waits until the snapshot instance is available.
public static void waitForSnapshotReady(RdsClient rdsClient, String
dbInstanceIdIdentifier,
 String dbSnapshotIdentifier) {
 try {
 boolean snapshotReady = false;
 String snapshotReadyStr;
```

```

 System.out.println("Waiting for the snapshot to become available.");

 DescribeDbSnapshotsRequest snapshotsRequest =
DescribeDbSnapshotsRequest.builder()
 .dbSnapshotIdentifier(dbSnapshotIdentifier)
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .build();

 while (!snapshotReady) {
 DescribeDbSnapshotsResponse response =
rdsClient.describeDBSnapshots(snapshotsRequest);
 List<DBSnapshot> snapshotList = response.dbSnapshots();
 for (DBSnapshot snapshot : snapshotList) {
 snapshotReadyStr = snapshot.status();
 if (snapshotReadyStr.contains("available")) {
 snapshotReady = true;
 } else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }

 System.out.println("The Snapshot is available!");
 } catch (RdsException | InterruptedException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

// Create an Amazon RDS snapshot.
public static void createSnapshot(RdsClient rdsClient, String
dbInstanceIdentifier, String dbSnapshotIdentifier) {
 try {
 CreateDbSnapshotRequest snapshotRequest =
CreateDbSnapshotRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .dbSnapshotIdentifier(dbSnapshotIdentifier)
 .build();

 CreateDbSnapshotResponse response =
rdsClient.createDBSnapshot(snapshotRequest);
 System.out.println("The Snapshot id is " +
response.dbSnapshot().dbiResourceId());
 }
}

```

```

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 // Waits until the database instance is available.
 public static void waitForInstanceReady(RdsClient rdsClient, String
dbInstanceIdIdentifier) {
 boolean instanceReady = false;
 String instanceReadyStr;
 System.out.println("Waiting for instance to become available.");
 try {
 DescribeDbInstancesRequest instanceRequest =
DescribeDbInstancesRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .build();

 String endpoint = "";
 while (!instanceReady) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances(instanceRequest);
 List<DBInstance> instanceList = response.dbInstances();
 for (DBInstance instance : instanceList) {
 instanceReadyStr = instance.dbInstanceStatus();
 if (instanceReadyStr.contains("available")) {
 endpoint = instance.endpoint().address();
 instanceReady = true;
 } else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }
 System.out.println("Database instance is available! The connection
endpoint is " + endpoint);

 } catch (RdsException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

```

// Create a database instance and return the ARN of the database.
public static String createDatabaseInstance(RdsClient rdsClient,
 String dbGroupName,
 String dbInstanceIdentifier,
 String dbName,
 String userName,
 String userPassword) {

 try {
 CreateDbInstanceRequest instanceRequest =
CreateDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .allocatedStorage(100)
 .dbName(dbName)
 .engine("mysql")
 .dbInstanceClass("db.t3.medium") // Updated to a supported class
 .engineVersion("8.0.32") // Updated to a supported version
 .storageType("gp2") // Changed to General Purpose SSD
(gp2)
 .masterUsername(userName)
 .masterUserPassword(userPassword)
 .build();

 CreateDbInstanceResponse response =
rdsClient.createDBInstance(instanceRequest);
 System.out.print("The status is " +
response.dbInstance().dbInstanceStatus());
 return response.dbInstance().dbInstanceArn();

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }

 return "";
}

// Get a list of micro instances.
public static void getMicroInstances(RdsClient rdsClient) {
 try {
 DescribeOrderableDbInstanceOptionsRequest dbInstanceOptionsRequest =
DescribeOrderableDbInstanceOptionsRequest
 .builder()
 .engine("mysql")

```

```

 .build();

 DescribeOrderableDbInstanceOptionsResponse response = rdsClient
 .describeOrderableDBInstanceOptions(dbInstanceOptionsRequest);
 List<OrderableDBInstanceOption> orderableDBInstances =
response.orderableDBInstanceOptions();
 for (OrderableDBInstanceOption dbInstanceOption : orderableDBInstances)
 {
 System.out.println("The engine version is " +
dbInstanceOption.engineVersion());
 System.out.println("The engine description is " +
dbInstanceOption.engine());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

// Get a list of allowed engine versions.
public static void getAllowedEngines(RdsClient rdsClient, String
dbParameterGroupFamily) {
 try {
 DescribeDbEngineVersionsRequest versionsRequest =
DescribeDbEngineVersionsRequest.builder()
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .engine("mysql")
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(versionsRequest);
 List<DBEngineVersion> dbEngines = response.dbEngineVersions();
 for (DBEngineVersion dbEngine : dbEngines) {
 System.out.println("The engine version is " +
dbEngine.engineVersion());
 System.out.println("The engine description is " +
dbEngine.dbEngineDescription());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```



```

 }

 // Modify auto_increment_offset and auto_increment_increment parameters.
 public static void modifyDBParas(RdsClient rdsClient, String dbGroupName) {
 try {
 Parameter parameter1 = Parameter.builder()
 .parameterName("auto_increment_offset")
 .applyMethod("immediate")
 .parameterValue("5")
 .build();

 List<Parameter> paraList = new ArrayList<>();
 paraList.add(parameter1);
 ModifyDbParameterGroupRequest groupRequest =
 ModifyDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .parameters(paraList)
 .build();

 ModifyDbParameterGroupResponse response =
 rdsClient.modifyDBParameterGroup(groupRequest);
 System.out.println("The parameter group " +
 response.dbParameterGroupName() + " was successfully modified");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 // Retrieve parameters in the group.
 public static void describeDbParameters(RdsClient rdsClient, String dbGroupName,
 int flag) {
 try {
 DescribeDbParametersRequest dbParameterGroupsRequest;
 if (flag == 0) {
 dbParameterGroupsRequest = DescribeDbParametersRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .build();
 } else {
 dbParameterGroupsRequest = DescribeDbParametersRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .source("user")
 .build();
 }
 }
 }
 }

```

```

 }

 DescribeDbParametersResponse response =
rdsClient.describeDBParameters(dbParameterGroupsRequest);
 List<Parameter> dbParameters = response.parameters();
 String paraName;
 for (Parameter para : dbParameters) {
 // Only print out information about either auto_increment_offset or
 // auto_increment_increment.
 paraName = para.parameterName();
 if ((paraName.compareTo("auto_increment_offset") == 0)
 || (paraName.compareTo("auto_increment_increment ") == 0)) {
 System.out.println("*** The parameter name is " + paraName);
 System.out.println("*** The parameter value is " +
para.parameterValue());
 System.out.println("*** The parameter data type is " +
para.dataType());
 System.out.println("*** The parameter description is " +
para.description());
 System.out.println("*** The parameter allowed values is " +
para.allowedValues());
 }
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void describeDbParameterGroups(RdsClient rdsClient, String
dbGroupName) {
 try {
 DescribeDbParameterGroupsRequest groupsRequest =
DescribeDbParameterGroupsRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .maxRecords(20)
 .build();

 DescribeDbParameterGroupsResponse response =
rdsClient.describeDBParameterGroups(groupsRequest);
 List<DBParameterGroup> groups = response.dbParameterGroups();
 for (DBParameterGroup group : groups) {

```

```

 System.out.println("The group name is " +
group.dbParameterGroupName());
 System.out.println("The group description is " +
group.description());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void createDBParameterGroup(RdsClient rdsClient, String
dbGroupName, String dbParameterGroupFamily) {
 try {
 CreateDbParameterGroupRequest groupRequest =
CreateDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .description("Created by using the AWS SDK for Java")
 .build();

 CreateDbParameterGroupResponse response =
rdsClient.createDBParameterGroup(groupRequest);
 System.out.println("The group name is " +
response.dbParameterGroup().dbParameterGroupName());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void describeDBEngines(RdsClient rdsClient) {
 try {
 DescribeDbEngineVersionsRequest engineVersionsRequest =
DescribeDbEngineVersionsRequest.builder()
 .defaultOnly(true)
 .engine("mysql")
 .maxRecords(20)
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(engineVersionsRequest);

```

```
List<DBEngineVersion> engines = response.dbEngineVersions();

// Get all DBEngineVersion objects.
for (DBEngineVersion engineOb : engines) {
 System.out.println("The name of the DB parameter group family for
the database engine is "
 + engineOb.dbParameterGroupFamily());
 System.out.println("The name of the database engine " +
engineOb.engine());
 System.out.println("The version number of the database engine " +
engineOb.engineVersion());
}

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateDBInstance](#)
- [CreateDBParameterGroup](#)
- [CreateDBSnapshot](#)
- [DeleteDBInstance](#)
- [DeleteDBParameterGroup](#)
- [DescribeDBEngineVersions](#)
- [DescribeDBInstances](#)
- [DescribeDBParameterGroups](#)
- [DescribeDBParameters](#)
- [DescribeDBSnapshots](#)
- [DescribeOrderableDBInstanceOptions](#)
- [ModifyDBParameterGroup](#)

# Actions

## CreateDBInstance

The following code example shows how to use CreateDBInstance.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import com.google.gson.Gson;
import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesRequest;
import software.amazon.awssdk.services.rds.model.CreateDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.CreateDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.RdsException;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesResponse;
import software.amazon.awssdk.services.rds.model.DBInstance;
import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;

import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This example requires an AWS Secrets Manager secret that contains the
 * database credentials. If you do not create a
 * secret, this example will not work. For more details, see:
```

```

*
* https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_how-
services-use-secrets_RS.html
*
*
*/

public class CreateDBInstance {
 public static long sleepTime = 20;

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <dbInstanceId> <dbName> <secretName>

 Where:
 dbInstanceId - The database instance identifier.\s
 dbName - The database name.\s
 secretName - The name of the AWS Secrets Manager secret that
contains the database credentials."
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbInstanceId = args[0];
 String dbName = args[1];
 String secretName = args[2];
 Gson gson = new Gson();
 User user = gson.fromJson(String.valueOf(getSecretValues(secretName)),
User.class);
 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 createDatabaseInstance(rdsClient, dbInstanceId, dbName,
user.getUsername(), user.getPassword());
 waitForInstanceReady(rdsClient, dbInstanceId);
 rdsClient.close();
 }
}

```

```

private static SecretsManagerClient getSecretClient() {
 Region region = Region.US_WEST_2;
 return SecretsManagerClient.builder()
 .region(region)

.credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();
}

private static String getSecretValues(String secretName) {
 SecretsManagerClient secretClient = getSecretClient();
 GetSecretValueRequest valueRequest = GetSecretValueRequest.builder()
 .secretId(secretName)
 .build();

 GetSecretValueResponse valueResponse =
secretClient.getSecretValue(valueRequest);
 return valueResponse.secretString();
}

public static void createDatabaseInstance(RdsClient rdsClient,
 String dbInstanceIdentifier,
 String dbName,
 String userName,
 String userPassword) {

 try {
 CreateDbInstanceRequest instanceRequest =
CreateDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .allocatedStorage(100)
 .dbName(dbName)
 .engine("mysql")
 .dbInstanceClass("db.t3.medium") // Updated to a supported class
 .engineVersion("8.0.32") // Updated to a supported version
 .storageType("gp2") // Changed to General Purpose SSD
(gp2)

 .masterUsername(userName)
 .masterUserPassword(userPassword)
 .build();

 CreateDbInstanceResponse response =
rdsClient.createDBInstance(instanceRequest);

```

```

 System.out.print("The status is " +
response.dbInstance().dbInstanceStatus());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

// Waits until the database instance is available.
public static void waitForInstanceReady(RdsClient rdsClient, String
dbInstanceIdIdentifier) {
 boolean instanceReady = false;
 String instanceReadyStr;
 System.out.println("Waiting for instance to become available.");
 try {
 DescribeDbInstancesRequest instanceRequest =
DescribeDbInstancesRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .build();

 // Loop until the cluster is ready.
 while (!instanceReady) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances(instanceRequest);
 List<DBInstance> instanceList = response.dbInstances();
 for (DBInstance instance : instanceList) {
 instanceReadyStr = instance.dbInstanceStatus();
 if (instanceReadyStr.contains("available"))
 instanceReady = true;
 else {
 System.out.print(".");
 Thread.sleep(sleepTime * 1000);
 }
 }
 }
 System.out.println("Database instance is available!");

 } catch (RdsException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```



- For API details, see [CreateDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBParameterGroup

The following code example shows how to use CreateDBParameterGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void createDBParameterGroup(RdsClient rdsClient, String
dbGroupName, String dbParameterGroupFamily) {
 try {
 CreateDbParameterGroupRequest groupRequest =
 CreateDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .dbParameterGroupFamily(dbParameterGroupFamily)
 .description("Created by using the AWS SDK for Java")
 .build();

 CreateDbParameterGroupResponse response =
 rdsClient.createDBParameterGroup(groupRequest);
 System.out.println("The group name is " +
 response.dbParameterGroup().dbParameterGroupName());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateDBParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreateDBSnapshot

The following code example shows how to use CreateDBSnapshot.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Create an Amazon RDS snapshot.
public static void createSnapshot(RdsClient rdsClient, String
dbInstanceIdIdentifier, String dbSnapshotIdentifier) {
 try {
 CreateDbSnapshotRequest snapshotRequest =
CreateDbSnapshotRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .dbSnapshotIdentifier(dbSnapshotIdentifier)
 .build();

 CreateDbSnapshotResponse response =
rdsClient.createDBSnapshot(snapshotRequest);
 System.out.println("The Snapshot id is " +
response.dbSnapshot().dbiResourceId());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [CreateDBSnapshot](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBInstance

The following code example shows how to use DeleteDBInstance.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.DeleteDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.DeleteDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.RdsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteDBInstance {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <dbInstanceIdentifier>\s

 Where:
 dbInstanceIdentifier - The database instance identifier\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbInstanceIdentifier = args[0];
 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
```

```

 .build();

 deleteDatabaseInstance(rdsClient, dbInstanceIdentifier);
 rdsClient.close();
 }

 public static void deleteDatabaseInstance(RdsClient rdsClient, String
dbInstanceIdentifier) {
 try {
 DeleteDbInstanceRequest deleteDbInstanceRequest =
DeleteDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .deleteAutomatedBackups(true)
 .skipFinalSnapshot(true)
 .build();

 DeleteDbInstanceResponse response =
rdsClient.deleteDBInstance(deleteDbInstanceRequest);
 System.out.print("The status of the database is " +
response.dbInstance().dbInstanceStatus());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DeleteDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDBParameterGroup

The following code example shows how to use DeleteDBParameterGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

// Delete the parameter group after database has been deleted.
// An exception is thrown if you attempt to delete the para group while database
// exists.
public static void deleteParaGroup(RdsClient rdsClient, String dbGroupName,
String dbARN)
 throws InterruptedException {
 try {
 boolean isDataDel = false;
 boolean didFind;
 String instanceARN;

 // Make sure that the database has been deleted.
 while (!isDataDel) {
 DescribeDbInstancesResponse response =
rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 int listSize = instanceList.size();
 didFind = false;
 int index = 1;
 for (DBInstance instance : instanceList) {
 instanceARN = instance.dbInstanceArn();
 if (instanceARN.compareTo(dbARN) == 0) {
 System.out.println(dbARN + " still exists");
 didFind = true;
 }
 if ((index == listSize) && (!didFind)) {
 // Went through the entire list and did not find the
database ARN.

 isDataDel = true;
 }
 Thread.sleep(sleepTime * 1000);
 index++;
 }
 }

 // Delete the para group.
 DeleteDbParameterGroupRequest parameterGroupRequest =
DeleteDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .build();

 rdsClient.deleteDBParameterGroup(parameterGroupRequest);
 System.out.println(dbGroupName + " was deleted.");
 }
}

```

```
 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteDBParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAccountAttributes

The following code example shows how to use `DescribeAccountAttributes`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.AccountQuota;
import software.amazon.awssdk.services.rds.model.RdsException;
import software.amazon.awssdk.services.rds.model.DescribeAccountAttributesResponse;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeAccountAttributes {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
```

```

 .build();

 getAccountAttributes(rdsClient);
 rdsClient.close();
 }

 public static void getAccountAttributes(RdsClient rdsClient) {
 try {
 DescribeAccountAttributesResponse response =
rdsClient.describeAccountAttributes();
 List<AccountQuota> quotasList = response.accountQuotas();
 for (AccountQuota quotas : quotasList) {
 System.out.println("Name is: " + quotas.accountQuotaName());
 System.out.println("Max value is " + quotas.max());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DescribeAccountAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBEngineVersions

The following code example shows how to use DescribeDBEngineVersions.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void describeDBEngines(RdsClient rdsClient) {
 try {
 DescribeDbEngineVersionsRequest engineVersionsRequest =
DescribeDbEngineVersionsRequest.builder()

```

```

 .defaultOnly(true)
 .engine("mysql")
 .maxRecords(20)
 .build();

 DescribeDbEngineVersionsResponse response =
rdsClient.describeDBEngineVersions(engineVersionsRequest);
 List<DBEngineVersion> engines = response.dbEngineVersions();

 // Get all DBEngineVersion objects.
 for (DBEngineVersion engineOb : engines) {
 System.out.println("The name of the DB parameter group family for
the database engine is "
 + engineOb.dbParameterGroupFamily());
 System.out.println("The name of the database engine " +
engineOb.engine());
 System.out.println("The version number of the database engine " +
engineOb.engineVersion());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

```

- For API details, see [DescribeDBEngineVersions](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBInstances

The following code example shows how to use DescribeDBInstances.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
```



```
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.DescribeDbInstancesResponse;
import software.amazon.awssdk.services.rds.model.DBInstance;
import software.amazon.awssdk.services.rds.model.RdsException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeDBInstances {

 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 describeInstances(rdsClient);
 rdsClient.close();
 }

 public static void describeInstances(RdsClient rdsClient) {
 try {
 DescribeDbInstancesResponse response = rdsClient.describeDBInstances();
 List<DBInstance> instanceList = response.dbInstances();
 for (DBInstance instance : instanceList) {
 System.out.println("Instance ARN is: " + instance.dbInstanceArn());
 System.out.println("The Engine is " + instance.engine());
 System.out.println("Connection endpoint is" +
instance.endpoint().address());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DescribeDBInstances](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBParameterGroups

The following code example shows how to use DescribeDBParameterGroups.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeDbParameterGroups(RdsClient rdsClient, String
dbGroupName) {
 try {
 DescribeDbParameterGroupsRequest groupsRequest =
DescribeDbParameterGroupsRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .maxRecords(20)
 .build();

 DescribeDbParameterGroupsResponse response =
rdsClient.describeDBParameterGroups(groupsRequest);
 List<DBParameterGroup> groups = response.dbParameterGroups();
 for (DBParameterGroup group : groups) {
 System.out.println("The group name is " +
group.dbParameterGroupName());
 System.out.println("The group description is " +
group.description());
 }

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeDBParameterGroups](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeDBParameters

The following code example shows how to use DescribeDBParameters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Retrieve parameters in the group.
public static void describeDbParameters(RdsClient rdsClient, String dbGroupName,
int flag) {
 try {
 DescribeDbParametersRequest dbParameterGroupsRequest;
 if (flag == 0) {
 dbParameterGroupsRequest = DescribeDbParametersRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .build();
 } else {
 dbParameterGroupsRequest = DescribeDbParametersRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .source("user")
 .build();
 }

 DescribeDbParametersResponse response =
rdsClient.describeDBParameters(dbParameterGroupsRequest);
 List<Parameter> dbParameters = response.parameters();
 String paraName;
 for (Parameter para : dbParameters) {
 // Only print out information about either auto_increment_offset or
 // auto_increment_increment.
 paraName = para.parameterName();
 if ((paraName.compareTo("auto_increment_offset") == 0)
 || (paraName.compareTo("auto_increment_increment ") == 0)) {
 System.out.println("**** The parameter name is " + paraName);
 }
 }
 }
}
```

```

 System.out.println("*** The parameter value is " +
para.parameterValue());
 System.out.println("*** The parameter data type is " +
para.dataType());
 System.out.println("*** The parameter description is " +
para.description());
 System.out.println("*** The parameter allowed values is " +
para.allowedValues());
 }
}

} catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
}

```

- For API details, see [DescribeDBParameters](#) in *AWS SDK for Java 2.x API Reference*.

## GenerateRDSAuthToken

The following code example shows how to use `GenerateRDSAuthToken`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the [RdsUtilities](#) class to generate an authentication token.

```

public class GenerateRDSAuthToken {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <dbInstanceIdentifier> <masterUsername>

 Where:

```

```

 dbInstanceIdIdentifier - The database instance identifier.\s
 masterUsername - The master user name.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbInstanceIdIdentifier = args[0];
 String masterUsername = args[1];
 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 String token = getAuthToken(rdsClient, dbInstanceIdIdentifier,
masterUsername);
 System.out.println("The token response is " + token);
}

public static String getAuthToken(RdsClient rdsClient, String
dbInstanceIdIdentifier, String masterUsername) {

 RdsUtilities utilities = rdsClient.utilities();
 try {
 GenerateAuthenticationTokenRequest tokenRequest =
GenerateAuthenticationTokenRequest.builder()
 .credentialsProvider(ProfileCredentialsProvider.create())
 .username(masterUsername)
 .port(3306)
 .hostname(dbInstanceIdIdentifier)
 .build();

 return utilities.generateAuthenticationToken(tokenRequest);

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see [GenerateRDSToken](#) in *AWS SDK for Java 2.x API Reference*.

## ModifyDBInstance

The following code example shows how to use `ModifyDBInstance`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.ModifyDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.ModifyDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.RdsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ModifyDBInstance {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <dbInstanceIdentifier> <dbSnapshotIdentifier>\s
 Where:
 dbInstanceIdentifier - The database instance identifier.\s
 masterUserPassword - The updated password that corresponds to
the master user name.\s
 "";
```

```

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbInstanceIdentifier = args[0];
 String masterUserPassword = args[1];
 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 updateIntance(rdsClient, dbInstanceIdentifier, masterUserPassword);
 rdsClient.close();
 }

 public static void updateIntance(RdsClient rdsClient, String
dbInstanceIdentifier, String masterUserPassword) {
 try {
 // For a demo - modify the DB instance by modifying the master password.
 ModifyDbInstanceRequest modifyDbInstanceRequest =
ModifyDbInstanceRequest.builder()
 .dbInstanceIdentifier(dbInstanceIdentifier)
 .publiclyAccessible(true)
 .masterUserPassword(masterUserPassword)
 .build();

 ModifyDbInstanceResponse instanceResponse =
rdsClient.modifyDBInstance(modifyDbInstanceRequest);
 System.out.print("The ARN of the modified database is: " +
instanceResponse.dbInstance().dbInstanceArn());

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ModifyDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## ModifyDBParameterGroup

The following code example shows how to use `ModifyDBParameterGroup`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Modify auto_increment_offset and auto_increment_increment parameters.
public static void modifyDBParas(RdsClient rdsClient, String dbGroupName) {
 try {
 Parameter parameter1 = Parameter.builder()
 .parameterName("auto_increment_offset")
 .applyMethod("immediate")
 .parameterValue("5")
 .build();

 List<Parameter> paraList = new ArrayList<>();
 paraList.add(parameter1);
 ModifyDbParameterGroupRequest groupRequest =
 ModifyDbParameterGroupRequest.builder()
 .dbParameterGroupName(dbGroupName)
 .parameters(paraList)
 .build();

 ModifyDbParameterGroupResponse response =
 rdsClient.modifyDBParameterGroup(groupRequest);
 System.out.println("The parameter group " +
 response.dbParameterGroupName() + " was successfully modified");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [ModifyDBParameterGroup](#) in *AWS SDK for Java 2.x API Reference*.



## RebootDBInstance

The following code example shows how to use RebootDBInstance.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rds.RdsClient;
import software.amazon.awssdk.services.rds.model.RebootDbInstanceRequest;
import software.amazon.awssdk.services.rds.model.RebootDbInstanceResponse;
import software.amazon.awssdk.services.rds.model.RdsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class RebootDBInstance {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <dbInstanceIdentifier>\s

 Where:
 dbInstanceIdentifier - The database instance identifier\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String dbInstanceIdentifier = args[0];
```

```

 Region region = Region.US_WEST_2;
 RdsClient rdsClient = RdsClient.builder()
 .region(region)
 .build();

 rebootInstance(rdsClient, dbInstanceIdIdentifier);
 rdsClient.close();
 }

 public static void rebootInstance(RdsClient rdsClient, String
dbInstanceIdIdentifier) {
 try {
 RebootDbInstanceRequest rebootDbInstanceRequest =
RebootDbInstanceRequest.builder()
 .dbInstanceIdIdentifier(dbInstanceIdIdentifier)
 .build();

 RebootDbInstanceResponse instanceResponse =
rdsClient.rebootDBInstance(rebootDbInstanceRequest);
 System.out.print("The database " +
instanceResponse.dbInstance().dbInstanceArn() + " was rebooted");

 } catch (RdsException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [RebootDBInstance](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create an Aurora Serverless work item tracker

The following code example shows how to create a web application that tracks work items in an Amazon Aurora Serverless database and uses Amazon Simple Email Service (Amazon SES) to send reports.

## SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon RDS database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Aurora Serverless data and for use by a React application, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run an example that uses the JDBC API, see the full example on [GitHub](#).

### Services used in this example

- Aurora
- Amazon RDS
- Amazon RDS Data Service
- Amazon SES

## Serverless examples

### Connecting to an Amazon RDS database in a Lambda function

The following code example shows how to implement a Lambda function that connects to an RDS database. The function makes a simple database request and returns the result.

## SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Connecting to an Amazon RDS database in a Lambda function using Java.

```
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyRequestEvent;
import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyResponseEvent;
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
```

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rdsdata.RdsDataClient;
import software.amazon.awssdk.services.rdsdata.model.ExecuteStatementRequest;
import software.amazon.awssdk.services.rdsdata.model.ExecuteStatementResponse;
import software.amazon.awssdk.services.rdsdata.model.Field;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class RdsLambdaHandler implements RequestHandler<APIGatewayProxyRequestEvent,
 APIGatewayProxyResponseEvent> {

 @Override
 public APIGatewayProxyResponseEvent handleRequest(APIGatewayProxyRequestEvent
 event, Context context) {
 APIGatewayProxyResponseEvent response = new APIGatewayProxyResponseEvent();

 try {
 // Obtain auth token
 String token = createAuthToken();

 // Define connection configuration
 String connectionString = String.format("jdbc:mysql://%s:%s/%s?
useSSL=true&requireSSL=true",
 System.getenv("ProxyHostName"),
 System.getenv("Port"),
 System.getenv("DBName"));

 // Establish a connection to the database
 try (Connection connection =
 DriverManager.getConnection(connectionString, System.getenv("DBUserName"), token);
 PreparedStatement statement = connection.prepareStatement("SELECT ?
+ ? AS sum")) {

 statement.setInt(1, 3);
 statement.setInt(2, 2);

 try (ResultSet resultSet = statement.executeQuery()) {
 if (resultSet.next()) {
 int sum = resultSet.getInt("sum");
 response.setStatusCode(200);
 response.setBody("The selected sum is: " + sum);
 }
 }
 }
 } catch (Exception e) {
 response.setStatusCode(500);
 response.setBody(e.getMessage());
 }
 }
}

```

```

 }
 }
}

} catch (Exception e) {
 response.setStatusCode(500);
 response.setBody("Error: " + e.getMessage());
}

return response;
}

private String createAuthToken() {
 // Create RDS Data Service client
 RdsDataClient rdsDataClient = RdsDataClient.builder()
 .region(Region.of(System.getenv("AWS_REGION")))
 .credentialsProvider(DefaultCredentialsProvider.create())
 .build();

 // Define authentication request
 ExecuteStatementRequest request = ExecuteStatementRequest.builder()
 .resourceArn(System.getenv("ProxyHostName"))
 .secretArn(System.getenv("DBUserName"))
 .database(System.getenv("DBName"))
 .sql("SELECT 'RDS IAM Authentication'")
 .build();

 // Execute request and obtain authentication token
 ExecuteStatementResponse response = rdsDataClient.executeStatement(request);
 Field tokenField = response.records().get(0).get(0);

 return tokenField.stringValue();
}
}

```

## Amazon RDS Data Service examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon RDS Data Service.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Scenarios](#)

## Scenarios

### Create an Aurora Serverless work item tracker

The following code example shows how to create a web application that tracks work items in an Amazon Aurora Serverless database and uses Amazon Simple Email Service (Amazon SES) to send reports.

#### SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon RDS database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Aurora Serverless data and for use by a React application, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run an example that uses the JDBC API, see the full example on [GitHub](#).

#### Services used in this example

- Aurora
- Amazon RDS
- Amazon RDS Data Service
- Amazon SES

# Amazon Redshift examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Redshift.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Amazon Redshift

The following code example shows how to get started using Amazon Redshift.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.redshift.RedshiftClient;
import software.amazon.awssdk.services.redshift.paginators.DescribeClustersIterable;
```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloRedshift {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 RedshiftClient redshiftClient = RedshiftClient.builder()
 .region(region)
 .build();

 listClustersPaginator(redshiftClient);
 }

 public static void listClustersPaginator(RedshiftClient redshiftClient) {
 DescribeClustersIterable clustersIterable =
redshiftClient.describeClustersPaginator();
 clustersIterable.stream()
 .flatMap(r -> r.clusters().stream())
 .forEach(cluster -> System.out
 .println(" Cluster identifier: " + cluster.clusterIdentifier() + "
status = " + cluster.clusterStatus()));
 }
}

```

- For API details, see [DescribeClusters](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a Redshift cluster.
- List databases in the cluster.
- Create a table named Movies.



- Populate the Movies table.
- Query the Movies table by year.
- Modify the Redshift cluster.
- Delete the Amazon Redshift cluster.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Amazon Redshift features.

```
import com.example.redshift.User;
import com.google.gson.Gson;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.redshift.model.ClusterAlreadyExistsException;
import software.amazon.awssdk.services.redshift.model.CreateClusterResponse;
import software.amazon.awssdk.services.redshift.model.DeleteClusterResponse;
import software.amazon.awssdk.services.redshift.model.ModifyClusterResponse;
import software.amazon.awssdk.services.redshift.model.RedshiftException;
import software.amazon.awssdk.services.redshiftdata.model.ExecuteStatementResponse;
import software.amazon.awssdk.services.redshiftdata.model.RedshiftDataException;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;
import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
```

```

*
* This example requires an AWS Secrets Manager secret that contains the
* database credentials. If you do not create a
* secret that specifies user name and password, this example will not work. For
details, see:
*
* https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_how-
services-use-secrets_RS.html
*
This Java example performs these tasks:
*
* 1. Prompts the user for a unique cluster ID or use the default value.
* 2. Creates a Redshift cluster with the specified or default cluster Id value.
* 3. Waits until the Redshift cluster is available for use.
* 4. Lists all databases using a pagination API call.
* 5. Creates a table named "Movies" with fields ID, title, and year.
* 6. Inserts a specified number of records into the "Movies" table by reading the
Movies JSON file.
* 7. Prompts the user for a movie release year.
* 8. Runs a SQL query to retrieve movies released in the specified year.
* 9. Modifies the Redshift cluster.
* 10. Prompts the user for confirmation to delete the Redshift cluster.
* 11. If confirmed, deletes the specified Redshift cluster.
*/

public class RedshiftScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final Logger logger =
LoggerFactory.getLogger(RedshiftScenario.class);

 static RedshiftActions redshiftActions = new RedshiftActions();
 public static void main(String[] args) throws Exception {
 final String usage = ""

 Usage:
 <jsonFilePath> <secretName>\s

 Where:
 jsonFilePath - The path to the Movies JSON file (you can locate that
file in ../../../../resources/sample_files/movies.json)
 secretName - The name of the secret that belongs to Secret Manager
that stores the user name and password used in this scenario.
 """;

```

```

 if (args.length != 2) {
 logger.info(usage);
 return;
 }

```

```

 String jsonFilePath = args[0];
 String secretName = args[1];
 Scanner scanner = new Scanner(System.in);
 logger.info(DASHES);
 logger.info("Welcome to the Amazon Redshift SDK Basics scenario.");
 logger.info("

```

        This Java program demonstrates how to interact with Amazon Redshift by using the AWS SDK for Java (v2).\s

        Amazon Redshift is a fully managed, petabyte-scale data warehouse service hosted in the cloud.

        The program's primary functionalities include cluster creation, verification of cluster readiness,\s

        list databases, table creation, data population within the table, and execution of SQL statements.

        Furthermore, it demonstrates the process of querying data from the Movie table.\s

        Upon completion of the program, all AWS resources are cleaned up.  
 """);

```

 logger.info("Lets get started...");
 logger.info("

```

        First, we will retrieve the user name and password from Secrets Manager.

        Using Amazon Secrets Manager to store Redshift credentials provides several security benefits.

        It allows you to securely store and manage sensitive information, such as passwords, API keys, and database credentials, without embedding them directly in your application code.

        More information can be found here:

[https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating\\_how-services-use-secrets\\_RS.html](https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_how-services-use-secrets_RS.html)

        """);

```

 Gson gson = new Gson();

```

```

 User user = gson.fromJson(String.valueOf(getSecretValues(secretName)),
User.class);
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 try {
 runScenario(user, scanner, jsonFilePath);
 } catch (RuntimeException e) {
 e.printStackTrace();
 } catch (Throwable e) {
 throw new RuntimeException(e);
 }
 }

 private static void runScenario(User user, Scanner scanner, String
jsonFilePath) throws Throwable {
 String databaseName = "dev";
 System.out.println(DASHES);
 logger.info("Create a Redshift Cluster");
 logger.info("A Redshift cluster refers to the collection of computing
resources and storage that work together to process and analyze large volumes of
data.");
 logger.info("Enter a cluster id value or accept the default by hitting Enter
(default is redshift-cluster-movies): ");
 String userClusterId = scanner.nextLine();
 String clusterId = userClusterId.isEmpty() ? "redshift-cluster-movies" :
userClusterId;
 try {
 CompletableFuture<CreateClusterResponse> future =
redshiftActions.createClusterAsync(clusterId, user.getUserName(),
user.getUserPassword());
 CreateClusterResponse response = future.join();
 logger.info("Cluster successfully created. Cluster Identifier {} ",
response.cluster().clusterIdentifier());

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof ClusterAlreadyExistsException) {
 logger.info("The Cluster {} already exists. Moving on...",
clusterId);
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }
 }

```

```

 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("Wait until {} is available.", clusterId);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
redshiftActions.waitForClusterReadyAsync(clusterId);
 future.join();
 logger.info("Cluster is ready!");

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftException redshiftEx) {
 logger.info("Redshift error occurred: Error message: {}, Error code
{}", redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 String databaseInfo = ""
 When you created $clusteridD, the dev database is created by default and
 used in this scenario.\s

 To create a custom database, you need to have a CREATEDB privilege.\s
 For more information, see the documentation here: https://
docs.aws.amazon.com/redshift/latest/dg/r_CREATE_DATABASE.html.
 """.replace("$clusteridD", clusterId);

 logger.info(databaseInfo);
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("List databases in {} ",clusterId);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
redshiftActions.listAllDatabasesAsync(clusterId, user.getUserName(), "dev");
 future.join();

```

```

 logger.info("Databases listed successfully.");

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.error("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.error("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("Now you will create a table named Movies.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<ExecuteStatementResponse> future =
redshiftActions.createTableAsync(clusterId, databaseName, user.getUserName());
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("Populate the Movies table using the Movies.json file.");
 logger.info("Specify the number of records you would like to add to the
Movies Table.");
 logger.info("Please enter a value between 50 and 200.");
 int numRecords;
 do {
 logger.info("Enter a value: ");
 while (!scanner.hasNextInt()) {

```

```

 logger.info("Invalid input. Please enter a value between 50 and
200.");
 logger.info("Enter a year: ");
 scanner.next();
 }
 numRecords = scanner.nextInt();
} while (numRecords < 50 || numRecords > 200);
try {
 redshiftActions.popTableAsync(clusterId, databaseName,
user.getUserName(), jsonFilePath, numRecords).join(); // Wait for the operation to
complete
} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("Query the Movies table by year. Enter a value between
2012-2014.");
int movieYear;
do {
 logger.info("Enter a year: ");
 while (!scanner.hasNextInt()) {
 logger.info("Invalid input. Please enter a valid year between 2012
and 2014.");
 logger.info("Enter a year: ");
 scanner.next();
 }
 movieYear = scanner.nextInt();
 scanner.nextLine();
} while (movieYear < 2012 || movieYear > 2014);

String id;
try {

```

```

 CompletableFuture<String> future =
redshiftActions.queryMoviesByYearAsync(databaseName, user.getUserName(), movieYear,
clusterId);
 id = future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 logger.info("The identifier of the statement is " + id);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
redshiftActions.checkStatementAsync(id);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
 }

 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future = redshiftActions.getResultsAsync(id);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {

```



```

 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("Now you will modify the Redshift cluster.");
waitForInputToContinue(scanner);
try {
 CompletableFuture<ModifyClusterResponse> future =
redshiftActions.modifyClusterAsync(clusterId);
 future.join();

} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: {}", rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("Would you like to delete the Amazon Redshift cluster? (y/n)");
String delAns = scanner.nextLine().trim();
if (delAns.equalsIgnoreCase("y")) {
 logger.info("You selected to delete {} ", clusterId);
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<DeleteClusterResponse> future =
redshiftActions.deleteRedshiftClusterAsync(clusterId);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof RedshiftDataException redshiftEx) {
 logger.info("Redshift Data error occurred: {} Error code: {}",
redshiftEx.getMessage(), redshiftEx.awsErrorDetails().errorCode());

```

```

 } else {
 logger.info("An unexpected error occurred: {}",
rt.getMessage());
 }
 throw cause;
 }
} else {
 logger.info("The {} was not deleted", clusterId);
}
logger.info(DASHES);

logger.info(DASHES);
logger.info("This concludes the Amazon Redshift SDK Basics scenario.");
logger.info(DASHES);
}

private static SecretsManagerClient getSecretClient() {
 Region region = Region.US_EAST_1;
 return SecretsManagerClient.builder()
 .region(region)
 .build();
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
 }
}

// Get the Amazon Redshift credentials from AWS Secrets Manager.
private static String getSecretValues(String secretName) {
 SecretsManagerClient secretClient = getSecretClient();
 GetSecretValueRequest valueRequest = GetSecretValueRequest.builder()

```

```
 .secretId(secretName)
 .build();

 GetSecretValueResponse valueResponse =
secretClient.getSecretValue(valueRequest);
 return valueResponse.secretString();
}
}
```

A wrapper class for Amazon Redshift SDK methods.

```
public class RedshiftActions {

 private static final Logger logger =
LoggerFactory.getLogger(RedshiftActions.class);
 private static RedshiftDataAsyncClient redshiftDataAsyncClient;

 private static RedshiftAsyncClient redshiftAsyncClient;

 private static RedshiftAsyncClient getAsyncClient() {
 if (redshiftAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 redshiftAsyncClient = RedshiftAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return redshiftAsyncClient;
 }
}
```

```

private static RedshiftDataAsyncClient getAsyncDataClient() {
 if (redshiftDataAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 redshiftDataAsyncClient = RedshiftDataAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return redshiftDataAsyncClient;
}

/**
 * Creates a new Amazon Redshift cluster asynchronously.
 * @param clusterId the unique identifier for the cluster
 * @param username the username for the administrative user
 * @param userPassword the password for the administrative user
 * @return a CompletableFuture that represents the asynchronous operation of
creating the cluster
 * @throws RuntimeException if the cluster creation fails
 */
public CompletableFuture<CreateClusterResponse> createClusterAsync(String
clusterId, String username, String userPassword) {
 CreateClusterRequest clusterRequest = CreateClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .masterUsername(username)
 .masterUserPassword(userPassword)
 .nodeType("ra3.4xlarge")
 .publiclyAccessible(true)
 .numberOfNodes(2)
 .build();

```

```

 return getAsyncClient().createCluster(clusterRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Created cluster ");
 } else {
 throw new RuntimeException("Failed to create cluster: " +
exception.getMessage(), exception);
 }
 });
 }

 /**
 * Waits asynchronously for the specified cluster to become available.
 * @param clusterId the identifier of the cluster to wait for
 * @return a {@link CompletableFuture} that completes when the cluster is ready
 */
 public CompletableFuture<Void> waitForClusterReadyAsync(String clusterId) {
 DescribeClustersRequest clustersRequest = DescribeClustersRequest.builder()
 .clusterIdentifier(clusterId)
 .build();

 logger.info("Waiting for cluster to become available. This may take a few
minutes.");
 long startTime = System.currentTimeMillis();

 // Recursive method to poll the cluster status.
 return checkClusterStatusAsync(clustersRequest, startTime);
 }

 private CompletableFuture<Void> checkClusterStatusAsync(DescribeClustersRequest
clustersRequest, long startTime) {
 return getAsyncClient().describeClusters(clustersRequest)
 .thenCompose(clusterResponse -> {
 List<Cluster> clusterList = clusterResponse.clusters();
 boolean clusterReady = false;
 for (Cluster cluster : clusterList) {
 if ("available".equals(cluster.clusterStatus())) {
 clusterReady = true;
 break;
 }
 }
 })

 if (clusterReady) {

```

```

 logger.info(String.format("Cluster is available!"));
 return CompletableFuture.completedFuture(null);
 } else {
 long elapsedTimeMillis = System.currentTimeMillis() - startTime;
 long elapsedSeconds = elapsedTimeMillis / 1000;
 long minutes = elapsedSeconds / 60;
 long seconds = elapsedSeconds % 60;
 System.out.printf("\rElapsed Time: %02d:%02d - Waiting for
cluster...", minutes, seconds);
 System.out.flush();

 // Wait 1 second before the next status check
 return CompletableFuture.runAsync(() -> {
 try {
 TimeUnit.SECONDS.sleep(1);
 } catch (InterruptedException e) {
 throw new RuntimeException("Error during sleep: " +
e.getMessage(), e);
 }
 }).thenCompose(ignored ->
checkClusterStatusAsync(clustersRequest, startTime));
 }
 }).exceptionally(exception -> {
 throw new RuntimeException("Failed to get cluster status: " +
exception.getMessage(), exception);
 });
}

/**
 * Lists all databases asynchronously for the specified cluster, database user,
and database.
 * @param clusterId the identifier of the cluster to list databases for
 * @param dbUser the database user to use for the list databases request
 * @param database the database to list databases for
 * @return a {@link CompletableFuture} that completes when the database listing
is complete, or throws a {@link RuntimeException} if there was an error
 */
public CompletableFuture<Void> listAllDatabasesAsync(String clusterId, String
dbUser, String database) {
 ListDatabasesRequest databasesRequest = ListDatabasesRequest.builder()
 .clusterIdentifier(clusterId)
 .dbUser(dbUser)
 .database(database)
 .build();

```

```

 // Asynchronous paginator for listing databases.
 ListDatabasesPublisher databasesPaginator =
getAsyncDataClient().listDatabasesPaginator(databasesRequest);
 CompletableFuture<Void> future = databasesPaginator.subscribe(response -> {
 response.databases().forEach(db -> {
 logger.info("The database name is {} ", db);
 });
 });

 // Return the future for asynchronous handling.
 return future.exceptionally(exception -> {
 throw new RuntimeException("Failed to list databases: " +
exception.getMessage(), exception);
 });
 }

 /**
 * Creates an asynchronous task to execute a SQL statement for creating a new
table.
 *
 * @param clusterId the identifier of the Amazon Redshift cluster
 * @param databaseName the name of the database to create the table in
 * @param userName the username to use for the database connection
 * @return a {@link CompletableFuture} that completes with the result of the SQL
statement execution
 * @throws RuntimeException if there is an error creating the table
 */
 public CompletableFuture<ExecuteStatementResponse> createTableAsync(String
clusterId, String databaseName, String userName) {
 ExecuteStatementRequest createTableRequest =
ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .dbUser(userName)
 .database(databaseName)
 .sql("CREATE TABLE Movies (" +
 "id INT PRIMARY KEY, " +
 "title VARCHAR(100), " +
 "year INT)")
 .build();

 return getAsyncDataClient().executeStatement(createTableRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {

```

```

 throw new RuntimeException("Error creating table: " +
exception.getMessage(), exception);
 } else {
 logger.info("Table created: Movies");
 }
 });
}

/**
 * Asynchronously pops a table from a JSON file.
 *
 * @param clusterId the ID of the cluster
 * @param databaseName the name of the database
 * @param userName the username
 * @param fileName the name of the JSON file
 * @param number the number of records to process
 * @return a CompletableFuture that completes with the number of records added
to the Movies table
 */
public CompletableFuture<Integer> popTableAsync(String clusterId, String
databaseName, String userName, String fileName, int number) {
 return CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 JsonNode rootNode = new ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 return iter;
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse JSON file: "
+ e.getMessage(), e);
 }
 }).thenCompose(iter -> processNodesAsync(clusterId, databaseName,
userName, iter, number))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 logger.info("Error {} ", exception.getMessage());
 } else {
 logger.info("{} records were added to the Movies table." ,
result);
 }
 });
}

```



```

private CompletableFuture<Integer> processNodesAsync(String clusterId, String
databaseName, String userName, Iterator<JsonNode> iter, int number) {
 return CompletableFuture.supplyAsync(() -> {
 int t = 0;
 try {
 while (iter.hasNext()) {
 if (t == number)
 break;
 JsonNode currentNode = iter.next();
 int year = currentNode.get("year").asInt();
 String title = currentNode.get("title").asText();

 // Use SqlParameter to avoid SQL injection.
 List<SqlParameter> parameterList = new ArrayList<>();
 String sqlStatement = "INSERT INTO Movies
VALUES(:id , :title, :year);";
 SqlParameter idParam = SqlParameter.builder()
 .name("id")
 .value(String.valueOf(t))
 .build();

 SqlParameter titleParam = SqlParameter.builder()
 .name("title")
 .value(title)
 .build();

 SqlParameter yearParam = SqlParameter.builder()
 .name("year")
 .value(String.valueOf(year))
 .build();
 parameterList.add(idParam);
 parameterList.add(titleParam);
 parameterList.add(yearParam);

 ExecuteStatementRequest insertStatementRequest =
ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .sql(sqlStatement)
 .database(databaseName)
 .dbUser(userName)
 .parameters(parameterList)
 .build();

 getAsyncDataClient().executeStatement(insertStatementRequest);
 }
 } catch (Exception e) {
 // Handle exception
 }
 });
}

```

```

 logger.info("Inserted: " + title + " (" + year + ")");
 t++;
 }
 } catch (RedshiftDataException e) {
 throw new RuntimeException("Error inserting data: " +
e.getMessage(), e);
 }
 return t;
});
}

/**
 * Checks the status of an SQL statement asynchronously and handles the
completion of the statement.
 *
 * @param sqlId the ID of the SQL statement to check
 * @return a {@link CompletableFuture} that completes when the SQL statement's
status is either "FINISHED" or "FAILED"
 */
public CompletableFuture<Void> checkStatementAsync(String sqlId) {
 DescribeStatementRequest statementRequest =
DescribeStatementRequest.builder()
 .id(sqlId)
 .build();

 return getAsyncDataClient().describeStatement(statementRequest)
 .thenCompose(response -> {
 String status = response.statusAsString();
 logger.info("... Status: {} ", status);

 if ("FAILED".equals(status)) {
 throw new RuntimeException("The Query Failed. Ending program");
 } else if ("FINISHED".equals(status)) {
 return CompletableFuture.completedFuture(null);
 } else {
 // Sleep for 1 second and recheck status
 return CompletableFuture.runAsync(() -> {
 try {
 TimeUnit.SECONDS.sleep(1);
 } catch (InterruptedException e) {
 throw new RuntimeException("Error during sleep: " +
e.getMessage(), e);
 }
 })
 }
 })
 }
}

```

```

 }).thenCompose(ignore -> checkStatementAsync(sqlId)); //
Recursively call until status is FINISHED or FAILED
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 // Handle exceptions
 logger.info("Error: {} ", exception.getMessage());
 } else {
 logger.info("The statement is finished!");
 }
 });
}

/**
 * Asynchronously retrieves the results of a statement execution.
 *
 * @param statementId the ID of the statement for which to retrieve the results
 * @return a {@link CompletableFuture} that completes when the statement result
has been processed
 */
public CompletableFuture<Void> getResultsAsync(String statementId) {
 GetStatementResultRequest resultRequest =
GetStatementResultRequest.builder()
 .id(statementId)
 .build();

 return getAsyncDataClient().getStatementResult(resultRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.info("Error getting statement result {} ",
exception.getMessage());
 throw new RuntimeException("Error getting statement result: " +
exception.getMessage(), exception);
 }

 // Extract and print the field values using streams if the response
is valid.
 response.records().stream()
 .flatMap(List::stream)
 .map(Field::stringValue)
 .filter(value -> value != null)
 .forEach(value -> System.out.println("The Movie title field is "
+ value));

```

```

 return response;
 }).thenAccept(response -> {
 // Optionally add more logic here if needed after handling the
response
 });
}

/**
 * Asynchronously queries movies by a given year from a Redshift database.
 *
 * @param database the name of the database to query
 * @param dbUser the user to connect to the database with
 * @param year the year to filter the movies by
 * @param clusterId the identifier of the Redshift cluster to connect to
 * @return a {@link CompletableFuture} containing the response ID of the
executed SQL statement
 */
public CompletableFuture<String> queryMoviesByYearAsync(String database,
 String dbUser,
 int year,
 String clusterId)
{
 String sqlStatement = "SELECT * FROM Movies WHERE year = :year";
 SqlParameter yearParam = SqlParameter.builder()
 .name("year")
 .value(String.valueOf(year))
 .build();

 ExecuteStatementRequest statementRequest = ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .database(database)
 .dbUser(dbUser)
 .parameters(yearParam)
 .sql(sqlStatement)
 .build();

 return CompletableFuture.supplyAsync(() -> {
 try {
 ExecuteStatementResponse response =
getAsyncDataClient().executeStatement(statementRequest).join(); // Use join() to
wait for the result
 return response.id();
 }
 });
}

```

```

 } catch (RedshiftDataException e) {
 throw new RuntimeException("Error executing statement: " +
e.getMessage(), e);
 }
 }).exceptionally(exception -> {
 logger.info("Error: {}", exception.getMessage());
 return "";
 });
}

/**
 * Modifies an Amazon Redshift cluster asynchronously.
 *
 * @param clusterId the identifier of the cluster to be modified
 * @return a {@link CompletableFuture} that completes when the cluster
modification is complete
 */
public CompletableFuture<ModifyClusterResponse> modifyClusterAsync(String
clusterId) {
 ModifyClusterRequest modifyClusterRequest = ModifyClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .preferredMaintenanceWindow("wed:07:30-wed:08:00")
 .build();

 return getAsyncClient().modifyCluster(modifyClusterRequest)
 .whenComplete((clusterResponse, exception) -> {
 if (exception != null) {
 if (exception.getCause() instanceof RedshiftException) {
 logger.info("Error: {} ", exception.getMessage());
 } else {
 logger.info("Unexpected error: {} ",
exception.getMessage());
 }
 } else {
 logger.info("The modified cluster was successfully modified and
has "
 + clusterResponse.cluster().preferredMaintenanceWindow() + "
as the maintenance window");
 }
 });
}

/**
 * Deletes a Redshift cluster asynchronously.

```

```

 *
 * @param clusterId the identifier of the Redshift cluster to be deleted
 * @return a {@link CompletableFuture} that represents the asynchronous
 operation of deleting the Redshift cluster
 */
 public CompletableFuture<DeleteClusterResponse>
deleteRedshiftClusterAsync(String clusterId) {
 DeleteClusterRequest deleteClusterRequest = DeleteClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .skipFinalClusterSnapshot(true)
 .build();

 return getAsyncClient().deleteCluster(deleteClusterRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 // Handle exceptions
 if (exception.getCause() instanceof RedshiftException) {
 logger.info("Error: {}", exception.getMessage());
 } else {
 logger.info("Unexpected error: {}", exception.getMessage());
 }
 } else {
 // Handle successful response
 logger.info("The status is {}",
response.cluster().clusterStatus());
 }
 });
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateCluster](#)
  - [DescribeClusters](#)
  - [DescribeStatement](#)
  - [ExecuteStatement](#)
  - [GetStatementResult](#)
  - [ListDatabasesPaginator](#)
  - [ModifyCluster](#)

# Actions

## CreateCluster

The following code example shows how to use CreateCluster.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create the cluster.

```
/**
 * Creates a new Amazon Redshift cluster asynchronously.
 * @param clusterId the unique identifier for the cluster
 * @param username the username for the administrative user
 * @param userPassword the password for the administrative user
 * @return a CompletableFuture that represents the asynchronous operation of
creating the cluster
 * @throws RuntimeException if the cluster creation fails
 */
public CompletableFuture<CreateClusterResponse> createClusterAsync(String
clusterId, String username, String userPassword) {
 CreateClusterRequest clusterRequest = CreateClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .masterUsername(username)
 .masterUserPassword(userPassword)
 .nodeType("ra3.4xlarge")
 .publiclyAccessible(true)
 .numberOfNodes(2)
 .build();

 return getAsyncClient().createCluster(clusterRequest)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Created cluster ");
 } else {
 throw new RuntimeException("Failed to create cluster: " +
exception.getMessage(), exception);
 }
 });
}
```

```

 }
 });
}

```

- For API details, see [CreateCluster](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCluster

The following code example shows how to use DeleteCluster.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete the cluster.

```

/**
 * Deletes a Redshift cluster asynchronously.
 *
 * @param clusterId the identifier of the Redshift cluster to be deleted
 * @return a {@link CompletableFuture} that represents the asynchronous
 * operation of deleting the Redshift cluster
 */
public CompletableFuture<DeleteClusterResponse>
deleteRedshiftClusterAsync(String clusterId) {
 DeleteClusterRequest deleteClusterRequest = DeleteClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .skipFinalClusterSnapshot(true)
 .build();

 return getAsyncClient().deleteCluster(deleteClusterRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 // Handle exceptions
 if (exception.getCause() instanceof RedshiftException) {
 logger.info("Error: {}", exception.getMessage());
 } else {

```



```

 logger.info("Unexpected error: {}", exception.getMessage());
 }
 } else {
 // Handle successful response
 logger.info("The status is {}",
response.cluster().clusterStatus());
 }
 });
}

```

- For API details, see [DeleteCluster](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeClusters

The following code example shows how to use DescribeClusters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Describe the cluster.

```

/**
 * Waits asynchronously for the specified cluster to become available.
 * @param clusterId the identifier of the cluster to wait for
 * @return a {@link CompletableFuture} that completes when the cluster is ready
 */
public CompletableFuture<Void> waitForClusterReadyAsync(String clusterId) {
 DescribeClustersRequest clustersRequest = DescribeClustersRequest.builder()
 .clusterIdentifier(clusterId)
 .build();

 logger.info("Waiting for cluster to become available. This may take a few
minutes.");
 long startTime = System.currentTimeMillis();

 // Recursive method to poll the cluster status.

```

```

 return checkClusterStatusAsync(clustersRequest, startTime);
 }

 private CompletableFuture<Void> checkClusterStatusAsync(DescribeClustersRequest
clustersRequest, long startTime) {
 return getAsyncClient().describeClusters(clustersRequest)
 .thenCompose(clusterResponse -> {
 List<Cluster> clusterList = clusterResponse.clusters();
 boolean clusterReady = false;
 for (Cluster cluster : clusterList) {
 if ("available".equals(cluster.clusterStatus())) {
 clusterReady = true;
 break;
 }
 }

 if (clusterReady) {
 logger.info(String.format("Cluster is available!"));
 return CompletableFuture.completedFuture(null);
 } else {
 long elapsedTimeMillis = System.currentTimeMillis() - startTime;
 long elapsedSeconds = elapsedTimeMillis / 1000;
 long minutes = elapsedSeconds / 60;
 long seconds = elapsedSeconds % 60;
 System.out.printf("\rElapsed Time: %02d:%02d - Waiting for
cluster...", minutes, seconds);
 System.out.flush();

 // Wait 1 second before the next status check
 return CompletableFuture.runAsync(() -> {
 try {
 TimeUnit.SECONDS.sleep(1);
 } catch (InterruptedException e) {
 throw new RuntimeException("Error during sleep: " +
e.getMessage(), e);
 }
 }).thenCompose(ignored ->
checkClusterStatusAsync(clustersRequest, startTime));
 }
 }).exceptionally(exception -> {
 throw new RuntimeException("Failed to get cluster status: " +
exception.getMessage(), exception);
 });
 }
}

```

- For API details, see [DescribeClusters](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeStatement

The following code example shows how to use DescribeStatement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Checks the status of an SQL statement asynchronously and handles the
 * completion of the statement.
 *
 * @param sqlId the ID of the SQL statement to check
 * @return a {@link CompletableFuture} that completes when the SQL statement's
 * status is either "FINISHED" or "FAILED"
 */
public CompletableFuture<Void> checkStatementAsync(String sqlId) {
 DescribeStatementRequest statementRequest =
DescribeStatementRequest.builder()
 .id(sqlId)
 .build();

 return getAsyncDataClient().describeStatement(statementRequest)
 .thenCompose(response -> {
 String status = response.statusAsString();
 logger.info("... Status: {} ", status);

 if ("FAILED".equals(status)) {
 throw new RuntimeException("The Query Failed. Ending program");
 } else if ("FINISHED".equals(status)) {
 return CompletableFuture.completedFuture(null);
 } else {
 // Sleep for 1 second and recheck status
 return CompletableFuture.runAsync(() -> {
```

```

 try {
 TimeUnit.SECONDS.sleep(1);
 } catch (InterruptedException e) {
 throw new RuntimeException("Error during sleep: " +
e.getMessage(), e);
 }
 }).thenCompose(ignore -> checkStatementAsync(sqlId)); //
 Recursively call until status is FINISHED or FAILED
 }
 }).whenComplete((result, exception) -> {
 if (exception != null) {
 // Handle exceptions
 logger.info("Error: {} ", exception.getMessage());
 } else {
 logger.info("The statement is finished!");
 }
 });
}

```

- For API details, see [DescribeStatement](#) in *AWS SDK for Java 2.x API Reference*.

## ExecuteStatement

The following code example shows how to use ExecuteStatement.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Executes a SQL statement to create a database table.

```

/**
 * Creates an asynchronous task to execute a SQL statement for creating a new
table.
 *
 * @param clusterId the identifier of the Amazon Redshift cluster
 * @param databaseName the name of the database to create the table in

```

```

 * @param userName the username to use for the database connection
 * @return a {@link CompletableFuture} that completes with the result of the SQL
statement execution
 * @throws RuntimeException if there is an error creating the table
 */
 public CompletableFuture<ExecuteStatementResponse> createTableAsync(String
clusterId, String databaseName, String userName) {
 ExecuteStatementRequest createTableRequest =
ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .dbUser(userName)
 .database(databaseName)
 .sql("CREATE TABLE Movies (" +
 "id INT PRIMARY KEY, " +
 "title VARCHAR(100), " +
 "year INT)")
 .build();

 return getAsyncDataClient().executeStatement(createTableRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 throw new RuntimeException("Error creating table: " +
exception.getMessage(), exception);
 } else {
 logger.info("Table created: Movies");
 }
 });
 }
}

```

Executes a SQL statement to insert data into a database table.

```

/**
 * Asynchronously pops a table from a JSON file.
 *
 * @param clusterId the ID of the cluster
 * @param databaseName the name of the database
 * @param userName the username
 * @param fileName the name of the JSON file
 * @param number the number of records to process
 * @return a CompletableFuture that completes with the number of records added
to the Movies table
 */

```

```

 public CompletableFuture<Integer> popTableAsync(String clusterId, String
databaseName, String userName, String fileName, int number) {
 return CompletableFuture.supplyAsync(() -> {
 try {
 JsonParser parser = new JsonFactory().createParser(new
File(fileName));
 JsonNode rootNode = new ObjectMapper().readTree(parser);
 Iterator<JsonNode> iter = rootNode.iterator();
 return iter;
 } catch (IOException e) {
 throw new RuntimeException("Failed to read or parse JSON file: "
+ e.getMessage(), e);
 }
 }).thenCompose(iter -> processNodesAsync(clusterId, databaseName,
userName, iter, number))
 .whenComplete((result, exception) -> {
 if (exception != null) {
 logger.info("Error {} ", exception.getMessage());
 } else {
 logger.info("{} records were added to the Movies table." ,
result);
 }
 });
 }

 private CompletableFuture<Integer> processNodesAsync(String clusterId, String
databaseName, String userName, Iterator<JsonNode> iter, int number) {
 return CompletableFuture.supplyAsync(() -> {
 int t = 0;
 try {
 while (iter.hasNext()) {
 if (t == number)
 break;
 JsonNode currentNode = iter.next();
 int year = currentNode.get("year").asInt();
 String title = currentNode.get("title").asText();

 // Use SqlParameter to avoid SQL injection.
 List<SqlParameter> parameterList = new ArrayList<>();
 String sqlStatement = "INSERT INTO Movies
VALUES(:id , :title, :year);";
 SqlParameter idParam = SqlParameter.builder()
 .name("id")
 .value(String.valueOf(t))

```

```

 .build();

 SqlParameter titleParam = SqlParameter.builder()
 .name("title")
 .value(title)
 .build();

 SqlParameter yearParam = SqlParameter.builder()
 .name("year")
 .value(String.valueOf(year))
 .build();
 parameterList.add(idParam);
 parameterList.add(titleParam);
 parameterList.add(yearParam);

 ExecuteStatementRequest insertStatementRequest =
ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .sql(sqlStatement)
 .database(databaseName)
 .dbUser(userName)
 .parameters(parameterList)
 .build();

 getAsyncDataClient().executeStatement(insertStatementRequest);
 logger.info("Inserted: " + title + " (" + year + ")");
 t++;
 }
 } catch (RedshiftDataException e) {
 throw new RuntimeException("Error inserting data: " +
e.getMessage(), e);
 }
 return t;
});
}

```

Executes a SQL statement to query a database table.

```

/**
 * Asynchronously queries movies by a given year from a Redshift database.
 *
 * @param database the name of the database to query

```

```

 * @param dbUser the user to connect to the database with
 * @param year the year to filter the movies by
 * @param clusterId the identifier of the Redshift cluster to connect to
 * @return a {@link CompletableFuture} containing the response ID of the
 executed SQL statement
 */
 public CompletableFuture<String> queryMoviesByYearAsync(String database,
 String dbUser,
 int year,
 String clusterId)
 {

 String sqlStatement = "SELECT * FROM Movies WHERE year = :year";
 SqlParameter yearParam = SqlParameter.builder()
 .name("year")
 .value(String.valueOf(year))
 .build();

 ExecuteStatementRequest statementRequest = ExecuteStatementRequest.builder()
 .clusterIdentifier(clusterId)
 .database(database)
 .dbUser(dbUser)
 .parameters(yearParam)
 .sql(sqlStatement)
 .build();

 return CompletableFuture.supplyAsync(() -> {
 try {
 ExecuteStatementResponse response =
getAsyncDataClient().executeStatement(statementRequest).join(); // Use join() to
wait for the result
 return response.id();
 } catch (RedshiftDataException e) {
 throw new RuntimeException("Error executing statement: " +
e.getMessage(), e);
 }
 }).exceptionally(exception -> {
 logger.info("Error: {}", exception.getMessage());
 return "";
 });
 }
}

```

- For API details, see [ExecuteStatement](#) in *AWS SDK for Java 2.x API Reference*.



## GetStatementResult

The following code example shows how to use `GetStatementResult`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Check the statement result.

```
/**
 * Asynchronously retrieves the results of a statement execution.
 *
 * @param statementId the ID of the statement for which to retrieve the results
 * @return a {@link CompletableFuture} that completes when the statement result
has been processed
 */
public CompletableFuture<Void> getResultsAsync(String statementId) {
 GetStatementResultRequest resultRequest =
GetStatementResultRequest.builder()
 .id(statementId)
 .build();

 return getAsyncDataClient().getStatementResult(resultRequest)
 .handle((response, exception) -> {
 if (exception != null) {
 logger.info("Error getting statement result {}",
exception.getMessage());
 throw new RuntimeException("Error getting statement result: " +
exception.getMessage(), exception);
 }

 // Extract and print the field values using streams if the response
is valid.
 response.records().stream()
 .flatMap(List::stream)
 .map(Field::stringValue)
 .filter(value -> value != null)
```

```

 .forEach(value -> System.out.println("The Movie title field is "
+ value));

 return response;
 }).thenAccept(response -> {
 // Optionally add more logic here if needed after handling the
response
 });
}

```

- For API details, see [GetStatementResult](#) in *AWS SDK for Java 2.x API Reference*.

## ListDatabases

The following code example shows how to use `ListDatabases`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Lists all databases asynchronously for the specified cluster, database user,
and database.
 * @param clusterId the identifier of the cluster to list databases for
 * @param dbUser the database user to use for the list databases request
 * @param database the database to list databases for
 * @return a {@link CompletableFuture} that completes when the database listing
is complete, or throws a {@link RuntimeException} if there was an error
 */
public CompletableFuture<Void> listAllDatabasesAsync(String clusterId, String
dbUser, String database) {
 ListDatabasesRequest databasesRequest = ListDatabasesRequest.builder()
 .clusterIdentifier(clusterId)
 .dbUser(dbUser)
 .database(database)
 .build();

```

```

 // Asynchronous paginator for listing databases.
 ListDatabasesPublisher databasesPaginator =
getAsyncDataClient().listDatabasesPaginator(databasesRequest);
 CompletableFuture<Void> future = databasesPaginator.subscribe(response -> {
 response.databases().forEach(db -> {
 logger.info("The database name is {} ", db);
 });
 });

 // Return the future for asynchronous handling.
 return future.exceptionally(exception -> {
 throw new RuntimeException("Failed to list databases: " +
exception.getMessage(), exception);
 });
 }

```

- For API details, see [ListDatabases](#) in *AWS SDK for Java 2.x API Reference*.

## ModifyCluster

The following code example shows how to use `ModifyCluster`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Modify a cluster.

```

/**
 * Modifies an Amazon Redshift cluster asynchronously.
 *
 * @param clusterId the identifier of the cluster to be modified
 * @return a {@link CompletableFuture} that completes when the cluster
modification is complete
 */

```

```

 public CompletableFuture<ModifyClusterResponse> modifyClusterAsync(String
clusterId) {
 ModifyClusterRequest modifyClusterRequest = ModifyClusterRequest.builder()
 .clusterIdentifier(clusterId)
 .preferredMaintenanceWindow("wed:07:30-wed:08:00")
 .build();

 return getAsyncClient().modifyCluster(modifyClusterRequest)
 .whenComplete((clusterResponse, exception) -> {
 if (exception != null) {
 if (exception.getCause() instanceof RedshiftException) {
 logger.info("Error: {} ", exception.getMessage());
 } else {
 logger.info("Unexpected error: {} ",
exception.getMessage());
 }
 } else {
 logger.info("The modified cluster was successfully modified and
has "
 + clusterResponse.cluster().preferredMaintenanceWindow() + "
as the maintenance window");
 }
 });
 }

```

- For API details, see [ModifyCluster](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a web application to track Amazon Redshift data

The following code example shows how to create a web application that tracks and reports on work items using an Amazon Redshift database.

#### SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon Redshift database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Redshift data and for use by a React application, see the full example on [GitHub](#).

### Services used in this example

- Amazon Redshift
- Amazon SES

## Amazon Rekognition examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Rekognition.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### CompareFaces

The following code example shows how to use CompareFaces.

For more information, see [Comparing faces in images](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;
import software.amazon.awssdk.core.SdkBytes;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CompareFaces {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <bucketName> <sourceKey> <targetKey>

 Where:
 bucketName - The name of the S3 bucket where the images are stored.
 sourceKey - The S3 key (file name) for the source image.
 targetKey - The S3 key (file name) for the target image.
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String sourceKey = args[1];
 String targetKey = args[2];

 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();
```

```

 compareTwoFaces(rekClient, bucketName, sourceKey, targetKey);
 }

 /**
 * Compares two faces from images stored in an Amazon S3 bucket using AWS
 * Rekognition.
 *
 * <p>This method takes two image keys from an S3 bucket and compares the faces
 * within them.
 *
 * It prints out the confidence level of matched faces and reports the number of
 * unmatched faces.</p>
 *
 * @param rekClient The {@link RekognitionClient} used to call AWS
 * Rekognition.
 * @param bucketName The name of the S3 bucket containing the images.
 * @param sourceKey The object key (file path) for the source image in the S3
 * bucket.
 * @param targetKey The object key (file path) for the target image in the S3
 * bucket.
 * @throws RuntimeException If the Rekognition service returns an error.
 */
 public static void compareTwoFaces(RekognitionClient rekClient, String
 bucketName, String sourceKey, String targetKey) {
 try {
 Float similarityThreshold = 70F;
 S3Object s3objectSource = S3Object.builder()
 .bucket(bucketName)
 .name(sourceKey)
 .build();

 Image sourceImage = Image.builder()
 .s3object(s3objectSource)
 .build();

 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(targetKey)
 .build();

 Image targetImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 CompareFacesRequest facesRequest = CompareFacesRequest.builder()

```

```

 .sourceImage(sourceImage)
 .targetImage(targetImage)
 .similarityThreshold(similarityThreshold)
 .build();

 // Compare the two images.
 CompareFacesResponse compareFacesResult =
rekClient.compareFaces(facesRequest);
 List<CompareFacesMatch> faceDetails = compareFacesResult.faceMatches();

 for (CompareFacesMatch match : faceDetails) {
 ComparedFace face = match.face();
 BoundingBox position = face.boundingBox();
 System.out.println("Face at " + position.left().toString()
 + " " + position.top()
 + " matches with " + face.confidence().toString()
 + "% confidence.");
 }

 List<ComparedFace> unmatchedFaces = compareFacesResult.unmatchedFaces();
 System.out.println("There were " + unmatchedFaces.size() + " face(s)
that did not match.");

 } catch (RekognitionException e) {
 System.err.println("Error comparing faces: " +
e.awsErrorDetails().errorMessage());
 throw new RuntimeException(e);
 }
}
}

```

- For API details, see [CompareFaces](#) in *AWS SDK for Java 2.x API Reference*.

## CreateCollection

The following code example shows how to use `CreateCollection`.

For more information, see [Creating a collection](#).



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.CreateCollectionResponse;
import software.amazon.awssdk.services.rekognition.model.CreateCollectionRequest;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateCollection {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <collectionName>\s

 Where:
 collectionName - The name of the collection.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();
```

```

 System.out.println("Creating collection: " + collectionId);
 createMyCollection(rekClient, collectionId);
 rekClient.close();
 }

 /**
 * Creates a new Amazon Rekognition collection.
 *
 * @param rekClient the Amazon Rekognition client used to interact with the
 * Rekognition service
 * @param collectionId the unique identifier for the collection to be created
 */
 public static void createMyCollection(RekognitionClient rekClient, String
collectionId) {
 try {
 CreateCollectionRequest collectionRequest =
CreateCollectionRequest.builder()
 .collectionId(collectionId)
 .build();

 CreateCollectionResponse collectionResponse =
rekClient.createCollection(collectionRequest);
 System.out.println("CollectionArn: " +
collectionResponse.collectionArn());
 System.out.println("Status code: " +
collectionResponse.statusCode().toString());

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [CreateCollection](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteCollection

The following code example shows how to use DeleteCollection.

For more information, see [Deleting a collection](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.DeleteCollectionRequest;
import software.amazon.awssdk.services.rekognition.model.DeleteCollectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteCollection {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <collectionId>\s

 Where:
 collectionId - The id of the collection to delete.\s
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();
```

```

 System.out.println("Deleting collection: " + collectionId);
 deleteMyCollection(rekClient, collectionId);
 rekClient.close();
 }

 /**
 * Deletes an Amazon Rekognition collection.
 *
 * @param rekClient An instance of the {@link RekognitionClient} class,
 * which is used to interact with the Amazon Rekognition service.
 * @param collectionId The ID of the collection to be deleted.
 */
 public static void deleteMyCollection(RekognitionClient rekClient, String
collectionId) {
 try {
 DeleteCollectionRequest deleteCollectionRequest =
DeleteCollectionRequest.builder()
 .collectionId(collectionId)
 .build();

 DeleteCollectionResponse deleteCollectionResponse =
rekClient.deleteCollection(deleteCollectionRequest);
 System.out.println(collectionId + ": " +
deleteCollectionResponse.statusCode().toString());

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DeleteCollection](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteFaces

The following code example shows how to use DeleteFaces.

For more information, see [Deleting faces from a collection](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.DeleteFacesRequest;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteFacesFromCollection {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <collectionId> <faceId>\s

 Where:
 collectionId - The id of the collection from which faces are
deleted.\s
 faceId - The id of the face to delete.\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 String faceId = args[1];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
```

```

 .build();

 System.out.println("Deleting collection: " + collectionId);
 deleteFacesCollection(rekClient, collectionId, faceId);
 rekClient.close();
 }

 /**
 * Deletes a face from the specified Amazon Rekognition collection.
 *
 * @param rekClient an instance of the Amazon Rekognition client
 * @param collectionId the ID of the collection from which the face should be
deleted
 * @param faceId the ID of the face to be deleted
 * @throws RekognitionException if an error occurs while deleting the face
 */
 public static void deleteFacesCollection(RekognitionClient rekClient,
 String collectionId,
 String faceId) {

 try {
 DeleteFacesRequest deleteFacesRequest = DeleteFacesRequest.builder()
 .collectionId(collectionId)
 .faceIds(faceId)
 .build();

 rekClient.deleteFaces(deleteFacesRequest);
 System.out.println("The face was deleted from the collection.");

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DeleteFaces](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeCollection

The following code example shows how to use DescribeCollection.

For more information, see [Describing a collection](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.DescribeCollectionRequest;
import software.amazon.awssdk.services.rekognition.model.DescribeCollectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DescribeCollection {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <collectionName>

 Where:
 collectionName - The name of the Amazon Rekognition collection.\s
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionName = args[0];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
```

```

 .build();

 describeColl(rekClient, collectionName);
 rekClient.close();
 }

 /**
 * Describes an Amazon Rekognition collection.
 *
 * @param rekClient The Amazon Rekognition client used to make the
 request.
 * @param collectionName The name of the collection to describe.
 *
 * @throws RekognitionException If an error occurs while describing the
 collection.
 */
 public static void describeColl(RekognitionClient rekClient, String
 collectionName) {
 try {
 DescribeCollectionRequest describeCollectionRequest =
 DescribeCollectionRequest.builder()
 .collectionId(collectionName)
 .build();

 DescribeCollectionResponse describeCollectionResponse = rekClient
 .describeCollection(describeCollectionRequest);
 System.out.println("Collection Arn : " +
 describeCollectionResponse.collectionARN());
 System.out.println("Created : " +
 describeCollectionResponse.creationTimestamp().toString());

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DescribeCollection](#) in *AWS SDK for Java 2.x API Reference*.



## DetectFaces

The following code example shows how to use DetectFaces.

For more information, see [Detecting faces in an image](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;

import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectFaces {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucketName> <sourceImage>

 Where:
 bucketName = The name of the Amazon S3 bucket where the source image
is stored.
 sourceImage - The name of the source image file in the Amazon S3
bucket. (for example, pic1.png).\s
 """;

 if (args.length != 2) {
```

```
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String sourceImage = args[1];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 detectFacesinImage(rekClient, bucketName, sourceImage);
 rekClient.close();
}

/**
 * Detects faces in an image stored in an Amazon S3 bucket using the Amazon
 * Rekognition service.
 *
 * @param rekClient The Amazon Rekognition client used to interact with the
 * Rekognition service.
 * @param bucketName The name of the Amazon S3 bucket where the source image
 * is stored.
 * @param sourceImage The name of the source image file in the Amazon S3
 * bucket.
 */
public static void detectFacesinImage(RekognitionClient rekClient, String
bucketName, String sourceImage) {
 try {
 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image targetImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 DetectFacesRequest facesRequest = DetectFacesRequest.builder()
 .attributes(Attribute.ALL)
 .image(targetImage)
 .build();

 DetectFacesResponse facesResponse = rekClient.detectFaces(facesRequest);
 }
}
```

```

 List<FaceDetail> faceDetails = facesResponse.faceDetails();
 for (FaceDetail face : faceDetails) {
 AgeRange ageRange = face.ageRange();
 System.out.println("The detected face is estimated to be between "
 + ageRange.low().toString() + " and " +
ageRange.high().toString()
 + " years old.");

 System.out.println("There is a smile : " +
face.smile().value().toString());
 }

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DetectFaces](#) in *AWS SDK for Java 2.x API Reference*.

## DetectLabels

The following code example shows how to use DetectLabels.

For more information, see [Detecting labels in an image](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;

import java.io.FileInputStream;

```

```
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectLabels {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <bucketName> <sourceImage>

 Where:
 bucketName - The name of the Amazon S3 bucket where the image is
stored
 sourceImage - The name of the image file (for example, pic1.png).\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0] ;
 String sourceImage = args[1] ;
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 detectImageLabels(rekClient, bucketName, sourceImage);
 rekClient.close();
 }

 /**
 * Detects the labels in an image stored in an Amazon S3 bucket using the Amazon
 Rekognition service.
 *

```

```

 * @param rekClient the Amazon Rekognition client used to make the detection
request
 * @param bucketName the name of the Amazon S3 bucket where the image is
stored
 * @param sourceImage the name of the image file to be analyzed
 */
 public static void detectImageLabels(RekognitionClient rekClient, String
bucketName, String sourceImage) {
 try {
 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image souImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 DetectLabelsRequest detectLabelsRequest = DetectLabelsRequest.builder()
 .image(souImage)
 .maxLabels(10)
 .build();

 DetectLabelsResponse labelsResponse =
rekClient.detectLabels(detectLabelsRequest);
 List<Label> labels = labelsResponse.labels();
 System.out.println("Detected labels for the given photo");
 for (Label label : labels) {
 System.out.println(label.name() + ": " +
label.confidence().toString());
 }

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DetectLabels](#) in *AWS SDK for Java 2.x API Reference*.

## DetectModerationLabels

The following code example shows how to use DetectModerationLabels.

For more information, see [Detecting inappropriate images](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectModerationLabels {

 public static void main(String[] args) {
 final String usage = ""
 Usage: <bucketName> <sourceImage>

 Where:
 bucketName - The name of the S3 bucket where the images are stored.
 sourceImage - The name of the image (for example, pic1.png).\s
 """;

 if (args.length != 2) {
```

```

 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String sourceImage = args[1];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 detectModLabels(rekClient, bucketName, sourceImage);
 rekClient.close();
}

/**
 * Detects moderation labels in an image stored in an Amazon S3 bucket.
 *
 * @param rekClient the Amazon Rekognition client to use for the detection
 * @param bucketName the name of the Amazon S3 bucket where the image is
stored
 * @param sourceImage the name of the image file to be analyzed
 *
 * @throws RekognitionException if there is an error during the image detection
process
 */
public static void detectModLabels(RekognitionClient rekClient, String
bucketName, String sourceImage) {
 try {
 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image targetImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 DetectModerationLabelsRequest moderationLabelsRequest =
DetectModerationLabelsRequest.builder()
 .image(targetImage)
 .minConfidence(60F)
 .build();
 }
}

```

```

 DetectModerationLabelsResponse moderationLabelsResponse = rekClient
 .detectModerationLabels(moderationLabelsRequest);
 List<ModerationLabel> labels =
 moderationLabelsResponse.moderationLabels();
 System.out.println("Detected labels for image");
 for (ModerationLabel label : labels) {
 System.out.println("Label: " + label.name()
 + "\n Confidence: " + label.confidence().toString() + "%"
 + "\n Parent:" + label.parentName());
 }
 } catch (RekognitionException e) {
 e.printStackTrace();
 System.exit(1);
 }
}
}

```

- For API details, see [DetectModerationLabels](#) in *AWS SDK for Java 2.x API Reference*.

## DetectText

The following code example shows how to use DetectText.

For more information, see [Detecting text in an image](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;

import java.io.FileInputStream;
import java.io.FileNotFoundException;

```



```

import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectText {
 public static void main(String[] args) {
 final String usage = "\n" +
 "Usage: <bucketName> <sourceImage>\n" +
 "\n" +
 "Where:\n" +
 " bucketName - The name of the S3 bucket where the image is stored\n"
+
 " sourceImage - The path to the image that contains text (for example,
pic1.png). \n";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String sourceImage = args[1];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 detectTextLabels(rekClient, bucketName, sourceImage);
 rekClient.close();
 }

 /**
 * Detects text labels in an image stored in an S3 bucket using Amazon
 * Rekognition.
 *
 * @param rekClient an instance of the Amazon Rekognition client
 * @param bucketName the name of the S3 bucket where the image is stored

```

```

 * @param sourceImage the name of the image file in the S3 bucket
 * @throws RekognitionException if an error occurs while calling the Amazon
 Rekognition API
 */
 public static void detectTextLabels(RekognitionClient rekClient, String
 bucketName, String sourceImage) {
 try {
 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image souImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 DetectTextRequest textRequest = DetectTextRequest.builder()
 .image(souImage)
 .build();

 DetectTextResponse textResponse = rekClient.detectText(textRequest);
 List<TextDetection> textCollection = textResponse.textDetections();
 System.out.println("Detected lines and words");
 for (TextDetection text : textCollection) {
 System.out.println("Detected: " + text.detectedText());
 System.out.println("Confidence: " + text.confidence().toString());
 System.out.println("Id : " + text.id());
 System.out.println("Parent Id: " + text.parentId());
 System.out.println("Type: " + text.type());
 System.out.println();
 }

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [DetectText](#) in *AWS SDK for Java 2.x API Reference*.

## IndexFaces

The following code example shows how to use IndexFaces.

For more information, see [Adding faces to a collection](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.*;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class AddFacesToCollection {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <collectionId> <sourceImage> <bucketName>

 Where:
 collectionName - The name of the collection.
 sourceImage - The name of the image (for example, pic1.png).
 bucketName - The name of the S3 bucket.
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```

 String collectionId = args[0];
 String sourceImage = args[1];
 String bucketName = args[2];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 addToCollection(rekClient, collectionId, bucketName, sourceImage);
 rekClient.close();
 }

 /**
 * Adds a face from an image to an Amazon Rekognition collection.
 *
 * @param rekClient the Amazon Rekognition client
 * @param collectionId the ID of the collection to add the face to
 * @param bucketName the name of the Amazon S3 bucket containing the image
 * @param sourceImage the name of the image file to add to the collection
 * @throws RekognitionException if there is an error while interacting with the
 Amazon Rekognition service
 */
 public static void addToCollection(RekognitionClient rekClient, String
collectionId, String bucketName, String sourceImage) {
 try {
 S3Object s3objectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image targetImage = Image.builder()
 .s3object(s3objectTarget)
 .build();

 IndexFacesRequest facesRequest = IndexFacesRequest.builder()
 .collectionId(collectionId)
 .image(targetImage)
 .maxFaces(1)
 .qualityFilter(QualityFilter.AUTO)
 .detectionAttributes(Attribute.DEFAULT)
 .build();

 IndexFacesResponse facesResponse = rekClient.indexFaces(facesRequest);
 System.out.println("Results for the image");
 }
 }

```

```

 System.out.println("\n Faces indexed:");
 List<FaceRecord> faceRecords = facesResponse.faceRecords();
 for (FaceRecord faceRecord : faceRecords) {
 System.out.println(" Face ID: " + faceRecord.face().faceId());
 System.out.println(" Location:" +
faceRecord.faceDetail().boundingBox().toString());
 }

 List<UnindexedFace> unindexedFaces = facesResponse.unindexedFaces();
 System.out.println("Faces not indexed:");
 for (UnindexedFace unindexedFace : unindexedFaces) {
 System.out.println(" Location:" +
unindexedFace.faceDetail().boundingBox().toString());
 System.out.println(" Reasons:");
 for (Reason reason : unindexedFace.reasons()) {
 System.out.println("Reason: " + reason);
 }
 }

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [IndexFaces](#) in *AWS SDK for Java 2.x API Reference*.

## ListCollections

The following code example shows how to use `ListCollections`.

For more information, see [Listing collections](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.ListCollectionsRequest;
import software.amazon.awssdk.services.rekognition.model.ListCollectionsResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListCollections {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 System.out.println("Listing collections");
 listAllCollections(rekClient);
 rekClient.close();
 }

 public static void listAllCollections(RekognitionClient rekClient) {
 try {
 ListCollectionsRequest listCollectionsRequest =
 ListCollectionsRequest.builder()
 .maxResults(10)
 .build();

 ListCollectionsResponse response =
 rekClient.listCollections(listCollectionsRequest);
 List<String> collectionIds = response.collectionIds();
 for (String resultId : collectionIds) {
 System.out.println(resultId);
 }
 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 }
 }
}
```

```

 System.exit(1);
 }
}

```

- For API details, see [ListCollections](#) in *AWS SDK for Java 2.x API Reference*.

## ListFaces

The following code example shows how to use ListFaces.

For more information, see [Listing faces in a collection](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.Face;
import software.amazon.awssdk.services.rekognition.model.ListFacesRequest;
import software.amazon.awssdk.services.rekognition.model.ListFacesResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListFacesInCollection {
 public static void main(String[] args) {
 final String usage = ""

```

```

 Usage: <collectionId>

 Where:
 collectionId - The name of the collection.\s
 """;

 if (args.length < 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 System.out.println("Faces in collection " + collectionId);
 listFacesCollection(rekClient, collectionId);
 rekClient.close();
}

public static void listFacesCollection(RekognitionClient rekClient, String
collectionId) {
 try {
 ListFacesRequest facesRequest = ListFacesRequest.builder()
 .collectionId(collectionId)
 .maxResults(10)
 .build();

 ListFacesResponse facesResponse = rekClient.listFaces(facesRequest);
 List<Face> faces = facesResponse.faces();
 for (Face face : faces) {
 System.out.println("Confidence level there is a face: " +
face.confidence());
 System.out.println("The face Id value is " + face.faceId());
 }

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}
}

```



- For API details, see [ListFaces](#) in *AWS SDK for Java 2.x API Reference*.

## RecognizeCelebrities

The following code example shows how to use `RecognizeCelebrities`.

For more information, see [Recognizing celebrities in an image](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.core.SdkBytes;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

import software.amazon.awssdk.services.rekognition.model.*;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class RecognizeCelebrities {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <bucketName> <sourceImage>

 Where:
```

```

 bucketName - The name of the S3 bucket where the images are
stored.

 sourceImage - The path to the image (for example, C:\\AWS\\
\\pic1.png).\\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];;
 String sourceImage = args[1];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 System.out.println("Locating celebrities in " + sourceImage);
 recognizeAllCelebrities(rekClient, bucketName, sourceImage);
 rekClient.close();
}

/**
 * Recognizes all celebrities in an image stored in an Amazon S3 bucket.
 *
 * @param rekClient the Amazon Rekognition client used to perform the
celebrity recognition operation
 * @param bucketName the name of the Amazon S3 bucket where the source image
is stored
 * @param sourceImage the name of the source image file stored in the Amazon S3
bucket
 */
public static void recognizeAllCelebrities(RekognitionClient rekClient, String
bucketName, String sourceImage) {
 try {
 S3Object s3ObjectTarget = S3Object.builder()
 .bucket(bucketName)
 .name(sourceImage)
 .build();

 Image souImage = Image.builder()
 .s3Object(s3ObjectTarget)
 .build();
 }
}

```

```

 RecognizeCelebritiesRequest request =
RecognizeCelebritiesRequest.builder()
 .image(souImage)
 .build();

 RecognizeCelebritiesResponse result =
rekClient.recognizeCelebrities(request);
 List<Celebrity> celebs = result.celebrityFaces();
 System.out.println(celebs.size() + " celebrity(s) were recognized.\n");
 for (Celebrity celebrity : celebs) {
 System.out.println("Celebrity recognized: " + celebrity.name());
 System.out.println("Celebrity ID: " + celebrity.id());

 System.out.println("Further information (if available):");
 for (String url : celebrity.urls()) {
 System.out.println(url);
 }
 System.out.println();
 }
 System.out.println(result.unrecognizedFaces().size() + " face(s) were
unrecognized.");

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [RecognizeCelebrities](#) in *AWS SDK for Java 2.x API Reference*.

## SearchFaces

The following code example shows how to use SearchFaces.

For more information, see [Searching for a face \(face ID\)](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.SearchFacesByImageRequest;
import software.amazon.awssdk.services.rekognition.model.Image;
import software.amazon.awssdk.services.rekognition.model.SearchFacesByImageResponse;
import software.amazon.awssdk.services.rekognition.model.FaceMatch;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SearchFaceMatchingImageCollection {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <collectionId> <sourceImage>

 Where:
 collectionId - The id of the collection. \s
 sourceImage - The path to the image (for example, C:\\AWS\\
\\pic1.png).\s

 """;
```

```

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 String sourceImage = args[1];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 System.out.println("Searching for a face in a collections");
 searchFaceInCollection(rekClient, collectionId, sourceImage);
 rekClient.close();
 }

 public static void searchFaceInCollection(RekognitionClient rekClient, String
collectionId, String sourceImage) {
 try {
 InputStream sourceStream = new FileInputStream(new File(sourceImage));
 SdkBytes sourceBytes = SdkBytes.fromInputStream(sourceStream);
 Image souImage = Image.builder()
 .bytes(sourceBytes)
 .build();

 SearchFacesByImageRequest facesByImageRequest =
SearchFacesByImageRequest.builder()
 .image(souImage)
 .maxFaces(10)
 .faceMatchThreshold(70F)
 .collectionId(collectionId)
 .build();

 SearchFacesByImageResponse imageResponse =
rekClient.searchFacesByImage(facesByImageRequest);
 System.out.println("Faces matching in the collection");
 List<FaceMatch> faceImageMatches = imageResponse.faceMatches();
 for (FaceMatch face : faceImageMatches) {
 System.out.println("The similarity level is " + face.similarity());
 System.out.println();
 }
 }
 }

```

```

 } catch (RekognitionException | FileNotFoundException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [SearchFaces](#) in *AWS SDK for Java 2.x API Reference*.

## SearchFacesByImage

The following code example shows how to use SearchFacesByImage.

For more information, see [Searching for a face \(image\)](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.SearchFacesRequest;
import software.amazon.awssdk.services.rekognition.model.SearchFacesResponse;
import software.amazon.awssdk.services.rekognition.model.FaceMatch;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SearchFaceMatchingIdCollection {
 public static void main(String[] args) {

```

```

 final String usage = ""

 Usage: <collectionId> <sourceImage>

 Where:
 collectionId - The id of the collection. \s
 sourceImage - The path to the image (for example, C:\\AWS\\
\\pic1.png)\\.\\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String collectionId = args[0];
 String faceId = args[1];
 Region region = Region.US_WEST_2;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 System.out.println("Searching for a face in a collections");
 searchFaceById(rekClient, collectionId, faceId);
 rekClient.close();
 }

 public static void searchFaceById(RekognitionClient rekClient, String
collectionId, String faceId) {
 try {
 SearchFacesRequest searchFacesRequest = SearchFacesRequest.builder()
 .collectionId(collectionId)
 .faceId(faceId)
 .faceMatchThreshold(70F)
 .maxFaces(2)
 .build();

 SearchFacesResponse imageResponse =
rekClient.searchFaces(searchFacesRequest);
 System.out.println("Faces matching in the collection");
 List<FaceMatch> faceImageMatches = imageResponse.faceMatches();
 for (FaceMatch face : faceImageMatches) {
 System.out.println("The similarity level is " + face.similarity());
 System.out.println();
 }
 }
 }

```

```
 }
 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}
```

- For API details, see [SearchFacesByImage](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

#### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

#### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon Rekognition
- Amazon S3
- Amazon SNS



## Detect PPE in images

The following code example shows how to build an app that uses Amazon Rekognition to detect Personal Protective Equipment (PPE) in images.

### SDK for Java 2.x

Shows how to create an AWS Lambda function that detects images with Personal Protective Equipment.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB
- Amazon Rekognition
- Amazon S3
- Amazon SES

## Detect information in videos

The following code example shows how to:

- Start Amazon Rekognition jobs to detect elements like people, objects, and text in videos.
- Check job status until jobs finish.
- Output the list of elements detected by each job.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get celebrity results from a video located in an Amazon S3 bucket.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
```

```

import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.Video;
import
 software.amazon.awssdk.services.rekognition.model.StartCelebrityRecognitionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.CelebrityRecognitionSortBy;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.CelebrityRecognition;
import software.amazon.awssdk.services.rekognition.model.CelebrityDetail;
import
 software.amazon.awssdk.services.rekognition.model.StartCelebrityRecognitionRequest;
import
 software.amazon.awssdk.services.rekognition.model.GetCelebrityRecognitionRequest;
import
 software.amazon.awssdk.services.rekognition.model.GetCelebrityRecognitionResponse;
import java.util.List;

/**
 * To run this code example, ensure that you perform the Prerequisites as stated
 * in the Amazon Rekognition Guide:
 * https://docs.aws.amazon.com/rekognition/latest/dg/video-analyzing-with-sqs.html
 *
 * Also, ensure that set up your development environment, including your
 * credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class VideoCelebrityDetection {
 private static String startJobId = "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucket> <video> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
 (for example, (for example, myBucket).\s
 video - The name of video (for example, people.mp4).\s

```

```

 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String topicArn = args[2];
 String roleArn = args[3];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startCelebrityDetection(rekClient, channel, bucket, video);
 getCelebrityDetectionResults(rekClient);
 System.out.println("This example is done!");
 rekClient.close();
}

public static void startCelebrityDetection(RekognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {
 try {
 S3Object s3Obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vidObj = Video.builder()
 .s3Object(s3Obj)
 .build();
 }
}

```

```

 StartCelebrityRecognitionRequest recognitionRequest =
StartCelebrityRecognitionRequest.builder()
 .jobTag("Celebrities")
 .notificationChannel(channel)
 .video(vid0b)
 .build();

 StartCelebrityRecognitionResponse startCelebrityRecognitionResult =
rekClient
 .startCelebrityRecognition(recognitionRequest);
 startJobId = startCelebrityRecognitionResult.jobId();

 } catch (RekognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}

public static void getCelebrityDetectionResults(RekognitionClient rekClient) {
 try {
 String paginationToken = null;
 GetCelebrityRecognitionResponse recognitionResponse = null;
 boolean finished = false;
 String status;
 int yy = 0;

 do {
 if (recognitionResponse != null)
 paginationToken = recognitionResponse.nextToken();

 GetCelebrityRecognitionRequest recognitionRequest =
GetCelebrityRecognitionRequest.builder()
 .jobId(startJobId)
 .nextToken(paginationToken)
 .sortBy(CelebrityRecognitionSortBy.TIMESTAMP)
 .maxResults(10)
 .build();

 // Wait until the job succeeds
 while (!finished) {
 recognitionResponse =
rekClient.getCelebrityRecognition(recognitionRequest);
 status = recognitionResponse.jobStatusAsString();
 }
 } while (true);
 }
}

```

```

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + status);
 Thread.sleep(1000);
 }
 yy++;
 }

 finished = false;

 // Proceed when the job is done - otherwise VideoMetadata is null.
 VideoMetadata videoMetaData = recognitionResponse.videoMetadata();
 System.out.println("Format: " + videoMetaData.format());
 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());
 System.out.println("Job");

 List<CelebrityRecognition> celebs =
recognitionResponse.celebrities();
 for (CelebrityRecognition celeb : celebs) {
 long seconds = celeb.timestamp() / 1000;
 System.out.print("Sec: " + seconds + " ");
 CelebrityDetail details = celeb.celebrity();
 System.out.println("Name: " + details.name());
 System.out.println("Id: " + details.id());
 System.out.println();
 }

 } while (recognitionResponse.nextToken() != null);

 } catch (RekognitionException | InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

Detect labels in a video by a label detection operation.

```

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonMappingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import
 software.amazon.awssdk.services.rekognition.model.StartLabelDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.Video;
import software.amazon.awssdk.services.rekognition.model.StartLabelDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.GetLabelDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.GetLabelDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.LabelDetectionSortBy;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.LabelDetection;
import software.amazon.awssdk.services.rekognition.model.Label;
import software.amazon.awssdk.services.rekognition.model.Instance;
import software.amazon.awssdk.services.rekognition.model.Parent;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoDetect {
 private static String startJobId = "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucket> <video> <queueUrl> <topicArn> <roleArn>

```

```

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of the video (for example, people.mp4).\s
 queueUrl- The URL of a SQS queue.\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String queueUrl = args[2];
 String topicArn = args[3];
 String roleArn = args[4];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 SqsClient sqs = SqsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startLabels(rekClient, channel, bucket, video);
 getLabelJob(rekClient, sqs, queueUrl);
 System.out.println("This example is done!");
 sqs.close();
 rekClient.close();
 }

 public static void startLabels(RekognitionClient rekClient,
 NotificationChannel channel,

```

```
 String bucket,
 String video) {
 try {
 S3Object s3obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vidObj = Video.builder()
 .s3Object(s3obj)
 .build();

 StartLabelDetectionRequest labelDetectionRequest =
 StartLabelDetectionRequest.builder()
 .jobTag("DetectingLabels")
 .notificationChannel(channel)
 .video(vidObj)
 .minConfidence(50F)
 .build();

 StartLabelDetectionResponse labelDetectionResponse =
 rekClient.startLabelDetection(labelDetectionRequest);
 startJobId = labelDetectionResponse.jobId();

 boolean ans = true;
 String status = "";
 int yy = 0;
 while (ans) {

 GetLabelDetectionRequest detectionRequest =
 GetLabelDetectionRequest.builder()
 .jobId(startJobId)
 .maxResults(10)
 .build();

 GetLabelDetectionResponse result =
 rekClient.getLabelDetection(detectionRequest);
 status = result.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 ans = false;
 else
 System.out.println(yy + " status is: " + status);
```



```

 Thread.sleep(1000);
 yy++;
 }

 System.out.println(startJobId + " status is: " + status);

} catch (RekognitionException | InterruptedException e) {
 e.getMessage();
 System.exit(1);
}
}

public static void getLabelJob(RekognitionClient rekClient, SqsClient sqs,
String queueUrl) {
 List<Message> messages;
 ReceiveMessageRequest messageRequest = ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .build();

 try {
 messages = sqs.receiveMessage(messageRequest).messages();

 if (!messages.isEmpty()) {
 for (Message message : messages) {
 String notification = message.body();

 // Get the status and job id from the notification
 ObjectMapper mapper = new ObjectMapper();
 JsonNode jsonMessageTree = mapper.readTree(notification);
 JsonNode messageBodyText = jsonMessageTree.get("Message");
 ObjectMapper operationResultMapper = new ObjectMapper();
 JsonNode jsonResultTree =
operationResultMapper.readTree(messageBodyText.textValue());
 JsonNode operationJobId = jsonResultTree.get("JobId");
 JsonNode operationStatus = jsonResultTree.get("Status");
 System.out.println("Job found in JSON is " + operationJobId);

 DeleteMessageRequest deleteMessageRequest =
DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .build();

 String jobId = operationJobId.textValue();
 if (startJobId.compareTo(jobId) == 0) {

```

```

 System.out.println("Job id: " + operationJobId);
 System.out.println("Status : " +
operationStatus.toString());

 if (operationStatus.asText().equals("SUCCEEDED"))
 getResultsLabels(rekClient);
 else
 System.out.println("Video analysis failed");

 sqs.deleteMessage(deleteMessageRequest);
 } else {
 System.out.println("Job received was not job " +
startJobId);
 sqs.deleteMessage(deleteMessageRequest);
 }
}

} catch (RekognitionException e) {
 e.getMessage();
 System.exit(1);
} catch (JsonMappingException e) {
 e.printStackTrace();
} catch (JsonProcessingException e) {
 e.printStackTrace();
}
}

// Gets the job results by calling GetLabelDetection
private static void getResultsLabels(RekognitionClient rekClient) {

 int maxResults = 10;
 String paginationToken = null;
 GetLabelDetectionResponse labelDetectionResult = null;

 try {
 do {
 if (labelDetectionResult != null)
 paginationToken = labelDetectionResult.nextToken();

 GetLabelDetectionRequest labelDetectionRequest =
GetLabelDetectionRequest.builder()
 .jobId(startJobId)
 .sortBy(LabelDetectionSortBy.TIMESTAMP)

```

```

 .maxResults(maxResults)
 .nextToken(paginationToken)
 .build();

 LabelDetectionResult labelDetectionResult =
rekClient.getLabelDetection(labelDetectionRequest);
 VideoMetadata videoMetaData = labelDetectionResult.videoMetadata();
 System.out.println("Format: " + videoMetaData.format());
 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());

 List<LabelDetection> detectedLabels = labelDetectionResult.labels();
 for (LabelDetection detectedLabel : detectedLabels) {
 long seconds = detectedLabel.timestamp();
 Label label = detectedLabel.label();
 System.out.println("Millisecond: " + seconds + " ");

 System.out.println(" Label:" + label.name());
 System.out.println(" Confidence:" +
detectedLabel.label().confidence().toString());

 List<Instance> instances = label.instances();
 System.out.println(" Instances of " + label.name());

 if (instances.isEmpty()) {
 System.out.println(" " + "None");
 } else {
 for (Instance instance : instances) {
 System.out.println(" Confidence: " +
instance.confidence().toString());
 System.out.println(" Bounding box: " +
instance.boundingBox().toString());
 }
 }
 System.out.println(" Parent labels for " + label.name() +
":");

 List<Parent> parents = label.parents();

 if (parents.isEmpty()) {
 System.out.println(" None");
 } else {
 for (Parent parent : parents) {
 System.out.println(" " + parent.name());
 }
 }
 }
}

```

```

 }
 }
 System.out.println();
}
} while (labelDetectionResult != null &&
labelDetectionResult.nextToken() != null);

 } catch (RekognitionException e) {
 e.getMessage();
 System.exit(1);
 }
}
}

```

Detect faces in a video stored in an Amazon S3 bucket.

```

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonMappingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import
 software.amazon.awssdk.services.rekognition.model.StartLabelDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.Video;
import software.amazon.awssdk.services.rekognition.model.StartLabelDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.GetLabelDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.GetLabelDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.LabelDetectionSortBy;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.LabelDetection;
import software.amazon.awssdk.services.rekognition.model.Label;
import software.amazon.awssdk.services.rekognition.model.Instance;
import software.amazon.awssdk.services.rekognition.model.Parent;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import java.util.List;

```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoDetect {
 private static String startJobId = "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucket> <video> <queueUrl> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of the video (for example, people.mp4).\s
 queueUrl- The URL of a SQS queue.\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 5) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String queueUrl = args[2];
 String topicArn = args[3];
 String roleArn = args[4];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 SqsClient sqs = SqsClient.builder()

```

```

 .region(Region.US_EAST_1)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startLabels(rekClient, channel, bucket, video);
 getLabelJob(rekClient, sqs, queueUrl);
 System.out.println("This example is done!");
 sqs.close();
 rekClient.close();
}

public static void startLabels(RekognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {
 try {
 S3Object s3obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vid0b = Video.builder()
 .s3Object(s3obj)
 .build();

 StartLabelDetectionRequest labelDetectionRequest =
 StartLabelDetectionRequest.builder()
 .jobTag("DetectingLabels")
 .notificationChannel(channel)
 .video(vid0b)
 .minConfidence(50F)
 .build();

 StartLabelDetectionResponse labelDetectionResponse =
 rekClient.startLabelDetection(labelDetectionRequest);
 startJobId = labelDetectionResponse.jobId();

 boolean ans = true;
 String status = "";
 int yy = 0;
 }
}

```

```

 while (ans) {

 GetLabelDetectionRequest detectionRequest =
GetLabelDetectionRequest.builder()
 .jobId(startJobId)
 .maxResults(10)
 .build();

 GetLabelDetectionResponse result =
rekClient.getLabelDetection(detectionRequest);
 status = result.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 ans = false;
 else
 System.out.println(yy + " status is: " + status);

 Thread.sleep(1000);
 yy++;
 }

 System.out.println(startJobId + " status is: " + status);

 } catch (RecognitionException | InterruptedException e) {
 e.getMessage();
 System.exit(1);
 }
}

public static void getLabelJob(RecognitionClient rekClient, SqsClient sqs,
String queueUrl) {
 List<Message> messages;
 ReceiveMessageRequest messageRequest = ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .build();

 try {
 messages = sqs.receiveMessage(messageRequest).messages();

 if (!messages.isEmpty()) {
 for (Message message : messages) {
 String notification = message.body();

 // Get the status and job id from the notification

```

```

 ObjectMapper mapper = new ObjectMapper();
 JsonNode jsonMessageTree = mapper.readTree(notification);
 JsonNode messageBodyText = jsonMessageTree.get("Message");
 ObjectMapper operationResultMapper = new ObjectMapper();
 JsonNode jsonResultTree =
operationResultMapper.readTree(messageBodyText.textValue());
 JsonNode operationJobId = jsonResultTree.get("JobId");
 JsonNode operationStatus = jsonResultTree.get("Status");
 System.out.println("Job found in JSON is " + operationJobId);

 DeleteMessageRequest deleteMessageRequest =
DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .build();

 String jobId = operationJobId.textValue();
 if (startJobId.compareTo(jobId) == 0) {
 System.out.println("Job id: " + operationJobId);
 System.out.println("Status : " +
operationStatus.toString());

 if (operationStatus.asText().equals("SUCCEEDED"))
 getResultsLabels(rekClient);
 else
 System.out.println("Video analysis failed");

 sqs.deleteMessage(deleteMessageRequest);
 } else {
 System.out.println("Job received was not job " +
startJobId);
 sqs.deleteMessage(deleteMessageRequest);
 }
 }
}

} catch (RekognitionException e) {
 e.getMessage();
 System.exit(1);
} catch (JsonMappingException e) {
 e.printStackTrace();
} catch (JsonProcessingException e) {
 e.printStackTrace();
}
}

```



```
// Gets the job results by calling GetLabelDetection
private static void getResultsLabels(RecognitionClient rekClient) {

 int maxResults = 10;
 String paginationToken = null;
 GetLabelDetectionResponse labelDetectionResult = null;

 try {
 do {
 if (labelDetectionResult != null)
 paginationToken = labelDetectionResult.nextToken();

 GetLabelDetectionRequest labelDetectionRequest =
GetLabelDetectionRequest.builder()
 .jobId(startJobId)
 .sortBy(LabelDetectionSortBy.TIMESTAMP)
 .maxResults(maxResults)
 .nextToken(paginationToken)
 .build();

 labelDetectionResult =
rekClient.getLabelDetection(labelDetectionRequest);
 VideoMetadata videoMetaData = labelDetectionResult.videoMetadata();
 System.out.println("Format: " + videoMetaData.format());
 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());

 List<LabelDetection> detectedLabels = labelDetectionResult.labels();
 for (LabelDetection detectedLabel : detectedLabels) {
 long seconds = detectedLabel.timestamp();
 Label label = detectedLabel.label();
 System.out.println("Millisecond: " + seconds + " ");

 System.out.println(" Label:" + label.name());
 System.out.println(" Confidence:" +
detectedLabel.label().confidence().toString());

 List<Instance> instances = label.instances();
 System.out.println(" Instances of " + label.name());

 if (instances.isEmpty()) {
 System.out.println(" " + "None");
 }
 }
 } while (labelDetectionResult.nextToken() != null);
 } catch (Exception e) {
 System.out.println("Error: " + e.getMessage());
 }
}
```

```

 } else {
 for (Instance instance : instances) {
 System.out.println(" Confidence: " +
instance.confidence().toString());
 System.out.println(" Bounding box: " +
instance.boundingBox().toString());
 }
 }
 System.out.println(" Parent labels for " + label.name() +
":");
 List<Parent> parents = label.parents();

 if (parents.isEmpty()) {
 System.out.println(" None");
 } else {
 for (Parent parent : parents) {
 System.out.println(" " + parent.name());
 }
 }
 System.out.println();
 }
 } while (labelDetectionResult != null &&
labelDetectionResult.nextToken() != null);

 } catch (RekognitionException e) {
 e.getMessage();
 System.exit(1);
 }
}
}

```

Detect inappropriate or offensive content in a video stored in an Amazon S3 bucket.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.Video;
import
 software.amazon.awssdk.services.rekognition.model.StartContentModerationRequest;
import
 software.amazon.awssdk.services.rekognition.model.StartContentModerationResponse;

```

```

import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import
 software.amazon.awssdk.services.rekognition.model.GetContentModerationResponse;
import
 software.amazon.awssdk.services.rekognition.model.GetContentModerationRequest;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.ContentModerationDetection;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoDetectInappropriate {
 private static String startJobId = "";

 public static void main(String[] args) {

 final String usage = ""

 Usage: <bucket> <video> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of video (for example, people.mp4).\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String topicArn = args[2];

```

```

 String roleArn = args[3];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startModerationDetection(rekClient, channel, bucket, video);
 getModResults(rekClient);
 System.out.println("This example is done!");
 rekClient.close();
 }

 public static void startModerationDetection(RekognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {

 try {
 S3Object s3Obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vidObj = Video.builder()
 .s3Object(s3Obj)
 .build();

 StartContentModerationRequest modDetectionRequest =
StartContentModerationRequest.builder()
 .jobTag("Moderation")
 .notificationChannel(channel)
 .video(vidObj)
 .build();

 StartContentModerationResponse startModDetectionResult = rekClient
 .startContentModeration(modDetectionRequest);
 startJobId = startModDetectionResult.jobId();

 } catch (RekognitionException e) {

```

```

 System.out.println(e.getMessage());
 System.exit(1);
 }
}

public static void getModResults(RekognitionClient rekClient) {
 try {
 String paginationToken = null;
 GetContentModerationResponse modDetectionResponse = null;
 boolean finished = false;
 String status;
 int yy = 0;

 do {
 if (modDetectionResponse != null)
 paginationToken = modDetectionResponse.nextToken();

 GetContentModerationRequest modRequest =
GetContentModerationRequest.builder()
 .jobId(startJobId)
 .nextToken(paginationToken)
 .maxResults(10)
 .build();

 // Wait until the job succeeds.
 while (!finished) {
 modDetectionResponse =
rekClient.getContentModeration(modRequest);
 status = modDetectionResponse.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + status);
 Thread.sleep(1000);
 }
 yy++;
 }

 finished = false;

 // Proceed when the job is done - otherwise VideoMetadata is null.
 VideoMetadata videoMetaData = modDetectionResponse.videoMetadata();
 System.out.println("Format: " + videoMetaData.format());

```

```

 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());
 System.out.println("Job");

 List<ContentModerationDetection> mods =
modDetectionResponse.moderationLabels();
 for (ContentModerationDetection mod : mods) {
 long seconds = mod.timestamp() / 1000;
 System.out.print("Mod label: " + seconds + " ");
 System.out.println(mod.moderationLabel().toString());
 System.out.println();
 }

 } while (modDetectionResponse != null &&
modDetectionResponse.nextToken() != null);

 } catch (RekognitionException | InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

Detect technical cue segments and shot detection segments in a video stored in an Amazon S3 bucket.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.Video;
import software.amazon.awssdk.services.rekognition.model.StartShotDetectionFilter;
import
 software.amazon.awssdk.services.rekognition.model.StartTechnicalCueDetectionFilter;
import
 software.amazon.awssdk.services.rekognition.model.StartSegmentDetectionFilters;
import
 software.amazon.awssdk.services.rekognition.model.StartSegmentDetectionRequest;
import
 software.amazon.awssdk.services.rekognition.model.StartSegmentDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;

```

```

import
 software.amazon.awssdk.services.rekognition.model.GetSegmentDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.GetSegmentDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.SegmentDetection;
import software.amazon.awssdk.services.rekognition.model.TechnicalCueSegment;
import software.amazon.awssdk.services.rekognition.model.ShotSegment;
import software.amazon.awssdk.services.rekognition.model.SegmentType;
import software.amazon.awssdk.services.sqs.SqsClient;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoDetectSegment {
 private static String startJobId = "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucket> <video> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of video (for example, people.mp4).\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];

```

```

 String topicArn = args[2];
 String roleArn = args[3];

 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 SqsClient sqs = SqsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startSegmentDetection(rekClient, channel, bucket, video);
 getSegmentResults(rekClient);
 System.out.println("This example is done!");
 sqs.close();
 rekClient.close();
}

public static void startSegmentDetection(RekognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {
 try {
 S3Object s3obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vidObj = Video.builder()
 .s3Object(s3obj)
 .build();

 StartShotDetectionFilter cueDetectionFilter =
StartShotDetectionFilter.builder()
 .minSegmentConfidence(60F)
 .build();

```



```

 StartTechnicalCueDetectionFilter technicalCueDetectionFilter =
StartTechnicalCueDetectionFilter.builder()
 .minSegmentConfidence(60F)
 .build();

 StartSegmentDetectionFilters filters =
StartSegmentDetectionFilters.builder()
 .shotFilter(cueDetectionFilter)
 .technicalCueFilter(technicalCueDetectionFilter)
 .build();

 StartSegmentDetectionRequest segDetectionRequest =
StartSegmentDetectionRequest.builder()
 .jobTag("DetectingLabels")
 .notificationChannel(channel)
 .segmentTypes(SegmentType.TECHNICAL_CUE, SegmentType.SHOT)
 .video(vid0b)
 .filters(filters)
 .build();

 StartSegmentDetectionResponse segDetectionResponse =
rekClient.startSegmentDetection(segDetectionRequest);
 startJobId = segDetectionResponse.jobId();

 } catch (RekognitionException e) {
 e.getMessage();
 System.exit(1);
 }
}

public static void getSegmentResults(RekognitionClient rekClient) {
 try {
 String paginationToken = null;
 GetSegmentDetectionResponse segDetectionResponse = null;
 boolean finished = false;
 String status;
 int yy = 0;

 do {
 if (segDetectionResponse != null)
 paginationToken = segDetectionResponse.nextToken();

 GetSegmentDetectionRequest recognitionRequest =
GetSegmentDetectionRequest.builder()

```

```

 .jobId(startJobId)
 .nextToken(paginationToken)
 .maxResults(10)
 .build();

 // Wait until the job succeeds.
 while (!finished) {
 segDetectionResponse =
rekClient.getSegmentDetection(recognitionRequest);
 status = segDetectionResponse.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + status);
 Thread.sleep(1000);
 }
 yy++;
 }
 finished = false;

 // Proceed when the job is done - otherwise VideoMetadata is null.
 List<VideoMetadata> videoMetaData =
segDetectionResponse.videoMetadata();
 for (VideoMetadata metaData : videoMetaData) {
 System.out.println("Format: " + metaData.format());
 System.out.println("Codec: " + metaData.codec());
 System.out.println("Duration: " + metaData.durationMillis());
 System.out.println("FrameRate: " + metaData.frameRate());
 System.out.println("Job");
 }

 List<SegmentDetection> detectedSegments =
segDetectionResponse.segments();
 for (SegmentDetection detectedSegment : detectedSegments) {
 String type = detectedSegment.type().toString();
 if (type.contains(SegmentType.technicalCue.toString())) {
 System.out.println("Technical Cue");
 TechnicalCueSegment segmentCue =
detectedSegment.technicalCueSegment();
 System.out.println("\tType: " + segmentCue.type());
 System.out.println("\tConfidence: " +
segmentCue.confidence().toString());
 }
 }

```

```

 if (type.contains(SegmentType.SHOT.toString())) {
 System.out.println("Shot");
 ShotSegment segmentShot = detectedSegment.shotSegment();
 System.out.println("\tIndex " + segmentShot.index());
 System.out.println("\tConfidence: " +
segmentShot.confidence().toString());
 }

 long seconds = detectedSegment.durationMillis();
 System.out.println("\tDuration : " + seconds + " milliseconds");
 System.out.println("\tStart time code: " +
detectedSegment.startTimecodeSMPTE());
 System.out.println("\tEnd time code: " +
detectedSegment.endTimecodeSMPTE());
 System.out.println("\tDuration time code: " +
detectedSegment.durationSMPTE());
 System.out.println();
 }

 } while (segDetectionResponse != null &&
segDetectionResponse.nextToken() != null);

 } catch (RekognitionException | InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

Detect text in a video stored in a video stored in an Amazon S3 bucket.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.Video;
import software.amazon.awssdk.services.rekognition.model.StartTextDetectionRequest;
import software.amazon.awssdk.services.rekognition.model.StartTextDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.GetTextDetectionResponse;
import software.amazon.awssdk.services.rekognition.model.GetTextDetectionRequest;

```

```

import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.TextDetectionResult;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoDetectText {
 private static String startJobId = "";

 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucket> <video> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of video (for example, people.mp4).\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String topicArn = args[2];
 String roleArn = args[3];

 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();
 }
}

```

```

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
 .build();

 startTextLabels(rekClient, channel, bucket, video);
 getTextResults(rekClient);
 System.out.println("This example is done!");
 rekClient.close();
 }

 public static void startTextLabels(RecognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {
 try {
 S3Object s3obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vidObj = Video.builder()
 .s3Object(s3obj)
 .build();

 StartTextDetectionRequest labelDetectionRequest =
 StartTextDetectionRequest.builder()
 .jobTag("DetectingLabels")
 .notificationChannel(channel)
 .video(vidObj)
 .build();

 StartTextDetectionResponse labelDetectionResponse =
 rekClient.startTextDetection(labelDetectionRequest);
 startJobId = labelDetectionResponse.jobId();

 } catch (RecognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }

 public static void getTextResults(RecognitionClient rekClient) {

```

```
try {
 String paginationToken = null;
 GetTextDetectionResponse textDetectionResponse = null;
 boolean finished = false;
 String status;
 int yy = 0;

 do {
 if (textDetectionResponse != null)
 paginationToken = textDetectionResponse.nextToken();

 GetTextDetectionRequest recognitionRequest =
GetTextDetectionRequest.builder()
 .jobId(startJobId)
 .nextToken(paginationToken)
 .maxResults(10)
 .build();

 // Wait until the job succeeds.
 while (!finished) {
 textDetectionResponse =
rekClient.getTextDetection(recognitionRequest);
 status = textDetectionResponse.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + status);
 Thread.sleep(1000);
 }
 yy++;
 }

 finished = false;

 // Proceed when the job is done - otherwise VideoMetadata is null.
 VideoMetadata videoMetaData = textDetectionResponse.videoMetadata();
 System.out.println("Format: " + videoMetaData.format());
 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());
 System.out.println("Job");
 }
```

```

 List<TextDetectionResult> labels =
textDetectionResponse.textDetections();
 for (TextDetectionResult detectedText : labels) {
 System.out.println("Confidence: " +
detectedText.textDetection().confidence().toString());
 System.out.println("Id : " + detectedText.textDetection().id());
 System.out.println("Parent Id: " +
detectedText.textDetection().parentId());
 System.out.println("Type: " +
detectedText.textDetection().type());
 System.out.println("Text: " +
detectedText.textDetection().detectedText());
 System.out.println();
 }

 } while (textDetectionResponse != null &&
textDetectionResponse.nextToken() != null);

 } catch (RekognitionException | InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}

```

Detect people in a video stored in a video stored in an Amazon S3 bucket.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.rekognition.RekognitionClient;
import software.amazon.awssdk.services.rekognition.model.S3Object;
import software.amazon.awssdk.services.rekognition.model.NotificationChannel;
import software.amazon.awssdk.services.rekognition.model.StartPersonTrackingRequest;
import software.amazon.awssdk.services.rekognition.model.Video;
import
 software.amazon.awssdk.services.rekognition.model.StartPersonTrackingResponse;
import software.amazon.awssdk.services.rekognition.model.RekognitionException;
import software.amazon.awssdk.services.rekognition.model.GetPersonTrackingResponse;
import software.amazon.awssdk.services.rekognition.model.GetPersonTrackingRequest;
import software.amazon.awssdk.services.rekognition.model.VideoMetadata;
import software.amazon.awssdk.services.rekognition.model.PersonDetection;
import java.util.List;

```

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class VideoPersonDetection {
 private static String startJobId = "";

 public static void main(String[] args) {

 final String usage = ""

 Usage: <bucket> <video> <topicArn> <roleArn>

 Where:
 bucket - The name of the bucket in which the video is located
(for example, (for example, myBucket).\s
 video - The name of video (for example, people.mp4).\s
 topicArn - The ARN of the Amazon Simple Notification Service
(Amazon SNS) topic.\s
 roleArn - The ARN of the AWS Identity and Access Management (IAM)
role to use.\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucket = args[0];
 String video = args[1];
 String topicArn = args[2];
 String roleArn = args[3];
 Region region = Region.US_EAST_1;
 RekognitionClient rekClient = RekognitionClient.builder()
 .region(region)
 .build();

 NotificationChannel channel = NotificationChannel.builder()
 .snsTopicArn(topicArn)
 .roleArn(roleArn)
```



```

 .build();

 startPersonLabels(rekClient, channel, bucket, video);
 getPersonDetectionResults(rekClient);
 System.out.println("This example is done!");
 rekClient.close();
 }

 public static void startPersonLabels(RecognitionClient rekClient,
 NotificationChannel channel,
 String bucket,
 String video) {
 try {
 S3Object s3obj = S3Object.builder()
 .bucket(bucket)
 .name(video)
 .build();

 Video vid0b = Video.builder()
 .s3Object(s3obj)
 .build();

 StartPersonTrackingRequest personTrackingRequest =
 StartPersonTrackingRequest.builder()
 .jobTag("DetectingLabels")
 .video(vid0b)
 .notificationChannel(channel)
 .build();

 StartPersonTrackingResponse labelDetectionResponse =
 rekClient.startPersonTracking(personTrackingRequest);
 startJobId = labelDetectionResponse.jobId();

 } catch (RecognitionException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
 }

 public static void getPersonDetectionResults(RecognitionClient rekClient) {
 try {
 String paginationToken = null;
 GetPersonTrackingResponse personTrackingResult = null;
 boolean finished = false;

```

```

String status;
int yy = 0;

do {
 if (personTrackingResult != null)
 paginationToken = personTrackingResult.nextToken();

 GetPersonTrackingRequest recognitionRequest =
GetPersonTrackingRequest.builder()
 .jobId(startJobId)
 .nextToken(paginationToken)
 .maxResults(10)
 .build();

 // Wait until the job succeeds
 while (!finished) {

 personTrackingResult =
rekClient.getPersonTracking(recognitionRequest);
 status = personTrackingResult.jobStatusAsString();

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(yy + " status is: " + status);
 Thread.sleep(1000);
 }
 yy++;
 }

 finished = false;

 // Proceed when the job is done - otherwise VideoMetadata is null.
 VideoMetadata videoMetaData = personTrackingResult.videoMetadata();

 System.out.println("Format: " + videoMetaData.format());
 System.out.println("Codec: " + videoMetaData.codec());
 System.out.println("Duration: " + videoMetaData.durationMillis());
 System.out.println("FrameRate: " + videoMetaData.frameRate());
 System.out.println("Job");

 List<PersonDetection> detectedPersons =
personTrackingResult.persons();
 for (PersonDetection detectedPerson : detectedPersons) {

```

```
 long seconds = detectedPerson.timestamp() / 1000;
 System.out.print("Sec: " + seconds + " ");
 System.out.println("Person Identifier: " +
detectedPerson.person().index());
 System.out.println();
 }

 } while (personTrackingResult != null &&
personTrackingResult.nextToken() != null);

 } catch (RekognitionException | InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
 }
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [GetCelebrityRecognition](#)
- [GetContentModeration](#)
- [GetLabelDetection](#)
- [GetPersonTracking](#)
- [GetSegmentDetection](#)
- [GetTextDetection](#)
- [StartCelebrityRecognition](#)
- [StartContentModeration](#)
- [StartLabelDetection](#)
- [StartPersonTracking](#)
- [StartSegmentDetection](#)
- [StartTextDetection](#)

## Detect objects in images

The following code example shows how to build an app that uses Amazon Rekognition to detect objects by category in images.

## SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app that uses Amazon Rekognition to identify objects by category in images located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- Amazon Rekognition
- Amazon S3
- Amazon SES

## Detect people and objects in a video

The following code example shows how to detect people and objects in a video with Amazon Rekognition.

## SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app to detect faces and objects in videos located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- Amazon Rekognition
- Amazon S3
- Amazon SES
- Amazon SNS
- Amazon SQS

# Route 53 domain registration examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Route 53 domain registration.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Route 53 domain registration

The following code example shows how to get started using Route 53 domain registration.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.route53domains.Route53DomainsClient;
import software.amazon.awssdk.services.route53.model.Route53Exception;
import software.amazon.awssdk.services.route53domains.model.DomainPrice;
import software.amazon.awssdk.services.route53domains.model.ListPricesRequest;
import software.amazon.awssdk.services.route53domains.model.ListPricesResponse;
import java.util.List;
```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java code examples performs the following operation:
 *
 * 1. Invokes ListPrices for at least one domain type, such as the "com" type
 * and displays the prices for Registration and Renewal.
 */
public class HelloRoute53 {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage = "\n" +
 "Usage:\n" +
 " <hostedZoneId> \n\n" +
 "Where:\n" +
 " hostedZoneId - The id value of an existing hosted zone. \n";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String domainType = args[0];
 Region region = Region.US_EAST_1;
 Route53DomainsClient route53DomainsClient = Route53DomainsClient.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 System.out.println("Invokes ListPrices for at least one domain type.");
 listPrices(route53DomainsClient, domainType);
 System.out.println(DASHES);
 }

 public static void listPrices(Route53DomainsClient route53DomainsClient, String
domainType) {
 try {

```

```

 ListPricesRequest pricesRequest = ListPricesRequest.builder()
 .maxItems(10)
 .tld(domainType)
 .build();

 ListPricesResponse response =
route53DomainsClient.listPrices(pricesRequest);
 List<DomainPrice> prices = response.prices();
 for (DomainPrice pr : prices) {
 System.out.println("Name: " + pr.name());
 System.out.println(
 "Registration: " + pr.registrationPrice().price() + " " +
pr.registrationPrice().currency());
 System.out.println("Renewal: " + pr.renewalPrice().price() + " " +
pr.renewalPrice().currency());
 System.out.println("Transfer: " + pr.transferPrice().price() + " " +
pr.transferPrice().currency());
 System.out.println("Transfer: " + pr.transferPrice().price() + " " +
pr.transferPrice().currency());
 System.out.println("Change Ownership: " +
pr.changeOwnershipPrice().price() + " "
 + pr.changeOwnershipPrice().currency());
 System.out.println(
 "Restoration: " + pr.restorationPrice().price() + " " +
pr.restorationPrice().currency());
 System.out.println(" ");
 }

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListPrices](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- List current domains, and list operations in the past year.
- View billing for the past year, and view prices for domain types.
- Get domain suggestions.
- Check domain availability and transferability.
- Optionally, request a domain registration.
- Get an operation detail.
- Optionally, get a domain detail.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This example uses pagination methods where applicable. For example, to list
 * domains, the
 * listDomainsPaginator method is used. For more information about pagination,
 * see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/pagination.html
 *
 * This Java code example performs the following operations:
 *
 * 1. List current domains.
 * 2. List operations in the past year.
 * 3. View billing for the account in the past year.
 * 4. View prices for domain types.
 * 5. Get domain suggestions.
 * 6. Check domain availability.
```



```

* 7. Check domain transferability.
* 8. Request a domain registration.
* 9. Get operation details.
* 10. Optionally, get domain details.
*/

public class Route53Scenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <domainType> <phoneNumber> <email> <domainSuggestion>
<firstName> <lastName> <city>

 Where:
 domainType - The domain type (for example, com).\s
 phoneNumber - The phone number to use (for example,
+91.9966564xxx) email - The email address to use. domainSuggestion - The
domain suggestion (for example, findmy.accountants).\s
 firstName - The first name to use to register a domain.\s
 lastName - The last name to use to register a domain.\s
 city - the city to use to register a domain.\s
 """;

 if (args.length != 7) {
 System.out.println(usage);
 System.exit(1);
 }

 String domainType = args[0];
 String phoneNumber = args[1];
 String email = args[2];
 String domainSuggestion = args[3];
 String firstName = args[4];
 String lastName = args[5];
 String city = args[6];
 Region region = Region.US_EAST_1;
 Route53DomainsClient route53DomainsClient = Route53DomainsClient.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 }
}

```

```
 System.out.println("Welcome to the Amazon Route 53 domains example
scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("1. List current domains.");
 listDomains(route53DomainsClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. List operations in the past year.");
 listOperations(route53DomainsClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. View billing for the account in the past year.");
 listBillingRecords(route53DomainsClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. View prices for domain types.");
 listPrices(route53DomainsClient, domainType);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Get domain suggestions.");
 listDomainSuggestions(route53DomainsClient, domainSuggestion);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Check domain availability.");
 checkDomainAvailability(route53DomainsClient, domainSuggestion);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Check domain transferability.");
 checkDomainTransferability(route53DomainsClient, domainSuggestion);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Request a domain registration.");
 String opId = requestDomainRegistration(route53DomainsClient,
 domainSuggestion, phoneNumber, email, firstName,
 lastName, city);
```

```

 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. Get operation details.");
 getOperationalDetail(route53DomainsClient, opId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Get domain details.");
 System.out.println("Note: You must have a registered domain to get
details.");
 System.out.println("Otherwise, an exception is thrown that states ");
 System.out.println("Domain xxxxxxxx not found in xxxxxxxx account.");
 getDomainDetails(route53DomainsClient, domainSuggestion);
 System.out.println(DASHES);
 }

 public static void getDomainDetails(Route53DomainsClient route53DomainsClient,
String domainSuggestion) {
 try {
 GetDomainDetailRequest detailRequest = GetDomainDetailRequest.builder()
 .domainName(domainSuggestion)
 .build();

 GetDomainDetailResponse response =
route53DomainsClient.getDomainDetail(detailRequest);
 System.out.println("The contact first name is " +
response.registrantContact().firstName());
 System.out.println("The contact last name is " +
response.registrantContact().lastName());
 System.out.println("The contact org name is " +
response.registrantContact().organizationName());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 public static void getOperationalDetail(Route53DomainsClient
route53DomainsClient, String operationId) {
 try {
 GetOperationDetailRequest detailRequest =
GetOperationDetailRequest.builder()

```

```

 .operationId(operationId)
 .build();

 GetOperationDetailResponse response =
route53DomainsClient.getOperationDetail(detailRequest);
 System.out.println("Operation detail message is " + response.message());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static String requestDomainRegistration(Route53DomainsClient
route53DomainsClient,
 String domainSuggestion,
 String phoneNumber,
 String email,
 String firstName,
 String lastName,
 String city) {

 try {
 ContactDetail contactDetail = ContactDetail.builder()
 .contactType(ContactType.COMPANY)
 .state("LA")
 .countryCode(CountryCode.IN)
 .email(email)
 .firstName(firstName)
 .lastName(lastName)
 .city(city)
 .phoneNumber(phoneNumber)
 .organizationName("My Org")
 .addressLine1("My Address")
 .zipCode("123 123")
 .build();

 RegisterDomainRequest domainRequest = RegisterDomainRequest.builder()
 .adminContact(contactDetail)
 .registrantContact(contactDetail)
 .techContact(contactDetail)
 .domainName(domainSuggestion)
 .autoRenew(true)
 .durationInYears(1)

```

```
 .build();

 RegisterDomainResponse response =
route53DomainsClient.registerDomain(domainRequest);
 System.out.println("Registration requested. Operation Id: " +
response.operationId());
 return response.operationId();

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}

public static void checkDomainTransferability(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 CheckDomainTransferabilityRequest transferabilityRequest =
CheckDomainTransferabilityRequest.builder()
 .domainName(domainSuggestion)
 .build();

 CheckDomainTransferabilityResponse response = route53DomainsClient
 .checkDomainTransferability(transferabilityRequest);
 System.out.println("Transferability: " +
response.transferability().transferable().toString());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void checkDomainAvailability(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 CheckDomainAvailabilityRequest availabilityRequest =
CheckDomainAvailabilityRequest.builder()
 .domainName(domainSuggestion)
 .build();

 CheckDomainAvailabilityResponse response = route53DomainsClient
 .checkDomainAvailability(availabilityRequest);
```

```

 System.out.println(domainSuggestion + " is " +
response.availability().toString());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void listDomainSuggestions(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 GetDomainSuggestionsRequest suggestionsRequest =
GetDomainSuggestionsRequest.builder()
 .domainName(domainSuggestion)
 .suggestionCount(5)
 .onlyAvailable(true)
 .build();

 GetDomainSuggestionsResponse response =
route53DomainsClient.getDomainSuggestions(suggestionsRequest);
 List<DomainSuggestion> suggestions = response.suggestionsList();
 for (DomainSuggestion suggestion : suggestions) {
 System.out.println("Suggestion Name: " + suggestion.domainName());
 System.out.println("Availability: " + suggestion.availability());
 System.out.println(" ");
 }

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void listPrices(Route53DomainsClient route53DomainsClient, String
domainType) {
 try {
 ListPricesRequest pricesRequest = ListPricesRequest.builder()
 .tld(domainType)
 .build();

 ListPricesIterable listRes =
route53DomainsClient.listPricesPaginator(pricesRequest);
 listRes.stream()

```

```

 .flatMap(r -> r.prices().stream())
 .forEach(content -> System.out.println(" Name: " +
content.name() +
 " Registration: " + content.registrationPrice().price()
+ " "
 + content.registrationPrice().currency() +
 " Renewal: " + content.renewalPrice().price() + " " +
content.renewalPrice().currency()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void listBillingRecords(Route53DomainsClient route53DomainsClient)
{
 try {
 Date currentDate = new Date();
 LocalDateTime localDateTime =
currentDate.toInstant().atZone(ZoneId.systemDefault()).toLocalDateTime();
 ZoneOffset zoneOffset = ZoneOffset.of("+01:00");
 LocalDateTime localDateTime2 = localDateTime.minusYears(1);
 Instant myStartTime = localDateTime2.toInstant(zoneOffset);
 Instant myEndTime = localDateTime.toInstant(zoneOffset);

 ViewBillingRequest viewBillingRequest = ViewBillingRequest.builder()
 .start(myStartTime)
 .end(myEndTime)
 .build();

 ViewBillingIterable listRes =
route53DomainsClient.viewBillingPaginator(viewBillingRequest);
 listRes.stream()
 .flatMap(r -> r.billingRecords().stream())
 .forEach(content -> System.out.println(" Bill Date:: " +
content.billDate() +
 " Operation: " + content.operationAsString() +
 " Price: " + content.price()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

```

 }

 public static void listOperations(Route53DomainsClient route53DomainsClient) {
 try {
 Date currentDate = new Date();
 LocalDateTime localDateTime =
currentDate.toInstant().atZone(ZoneId.systemDefault()).toLocalDateTime();
 ZoneOffset zoneOffset = ZoneOffset.of("+01:00");
 localDateTime = localDateTime.minusYears(1);
 Instant myTime = localDateTime.toInstant(zoneOffset);

 ListOperationsRequest operationsRequest =
ListOperationsRequest.builder()
 .submittedSince(myTime)
 .build();

 ListOperationsIterable listRes =
route53DomainsClient.listOperationsPaginator(operationsRequest);
 listRes.stream()
 .flatMap(r -> r.operations().stream())
 .forEach(content -> System.out.println(" Operation Id: " +
content.operationId() +
 " Status: " + content.statusAsString() +
 " Date: " + content.submittedDate()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

 public static void listDomains(Route53DomainsClient route53DomainsClient) {
 try {
 ListDomainsIterable listRes =
route53DomainsClient.listDomainsPaginator();
 listRes.stream()
 .flatMap(r -> r.domains().stream())
 .forEach(content -> System.out.println("The domain name is " +
content.domainName()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }

```



```
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CheckDomainAvailability](#)
  - [CheckDomainTransferability](#)
  - [GetDomainDetail](#)
  - [GetDomainSuggestions](#)
  - [GetOperationDetail](#)
  - [ListDomains](#)
  - [ListOperations](#)
  - [ListPrices](#)
  - [RegisterDomain](#)
  - [ViewBilling](#)

## Actions

### CheckDomainAvailability

The following code example shows how to use CheckDomainAvailability.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void checkDomainAvailability(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 CheckDomainAvailabilityRequest availabilityRequest =
 CheckDomainAvailabilityRequest.builder()
 .domainName(domainSuggestion)
```

```
 .build();

 CheckDomainAvailabilityResponse response = route53DomainsClient
 .checkDomainAvailability(availabilityRequest);
 System.out.println(domainSuggestion + " is " +
response.availability().toString());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [CheckDomainAvailability](#) in *AWS SDK for Java 2.x API Reference*.

## CheckDomainTransferability

The following code example shows how to use CheckDomainTransferability.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void checkDomainTransferability(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 CheckDomainTransferabilityRequest transferabilityRequest =
CheckDomainTransferabilityRequest.builder()
 .domainName(domainSuggestion)
 .build();

 CheckDomainTransferabilityResponse response = route53DomainsClient
 .checkDomainTransferability(transferabilityRequest);
 System.out.println("Transferability: " +
response.transferability().transferable().toString());
 }
```

```
 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [CheckDomainTransferability](#) in *AWS SDK for Java 2.x API Reference*.

## GetDomainDetail

The following code example shows how to use `GetDomainDetail`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void getDomainDetails(Route53DomainsClient route53DomainsClient,
String domainSuggestion) {
 try {
 GetDomainDetailRequest detailRequest = GetDomainDetailRequest.builder()
 .domainName(domainSuggestion)
 .build();

 GetDomainDetailResponse response =
route53DomainsClient.getDomainDetail(detailRequest);
 System.out.println("The contact first name is " +
response.registrantContact().firstName());
 System.out.println("The contact last name is " +
response.registrantContact().lastName());
 System.out.println("The contact org name is " +
response.registrantContact().organizationName());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

```
}
```

- For API details, see [GetDomainDetail](#) in *AWS SDK for Java 2.x API Reference*.

## GetDomainSuggestions

The following code example shows how to use `GetDomainSuggestions`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listDomainSuggestions(Route53DomainsClient
route53DomainsClient, String domainSuggestion) {
 try {
 GetDomainSuggestionsRequest suggestionsRequest =
 GetDomainSuggestionsRequest.builder()
 .domainName(domainSuggestion)
 .suggestionCount(5)
 .onlyAvailable(true)
 .build();

 GetDomainSuggestionsResponse response =
 route53DomainsClient.getDomainSuggestions(suggestionsRequest);
 List<DomainSuggestion> suggestions = response.suggestionsList();
 for (DomainSuggestion suggestion : suggestions) {
 System.out.println("Suggestion Name: " + suggestion.domainName());
 System.out.println("Availability: " + suggestion.availability());
 System.out.println(" ");
 }

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetDomainSuggestions](#) in *AWS SDK for Java 2.x API Reference*.

## GetOperationDetail

The following code example shows how to use `GetOperationDetail`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void getOperationalDetail(Route53DomainsClient
route53DomainsClient, String operationId) {
 try {
 GetOperationDetailRequest detailRequest =
 GetOperationDetailRequest.builder()
 .operationId(operationId)
 .build();

 GetOperationDetailResponse response =
 route53DomainsClient.getOperationDetail(detailRequest);
 System.out.println("Operation detail message is " + response.message());

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetOperationDetail](#) in *AWS SDK for Java 2.x API Reference*.

## ListDomains

The following code example shows how to use `ListDomains`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listDomains(Route53DomainsClient route53DomainsClient) {
 try {
 ListDomainsIterable listRes =
route53DomainsClient.listDomainsPaginator();
 listRes.stream()
 .flatMap(r -> r.domains().stream())
 .forEach(content -> System.out.println("The domain name is " +
content.domainName()));
 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListDomains](#) in *AWS SDK for Java 2.x API Reference*.

## ListOperations

The following code example shows how to use ListOperations.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listOperations(Route53DomainsClient route53DomainsClient) {
```

```

 try {
 Date currentDate = new Date();
 LocalDateTime localDateTime =
currentDate.toInstant().atZone(ZoneId.systemDefault()).toLocalDateTime();
 ZoneOffset zoneOffset = ZoneOffset.of("+01:00");
 localDateTime = localDateTime.minusYears(1);
 Instant myTime = localDateTime.toInstant(zoneOffset);

 ListOperationsRequest operationsRequest =
ListOperationsRequest.builder()
 .submittedSince(myTime)
 .build();

 ListOperationsIterable listRes =
route53DomainsClient.listOperationsPaginator(operationsRequest);
 listRes.stream()
 .flatMap(r -> r.operations().stream())
 .forEach(content -> System.out.println(" Operation Id: " +
content.operationId() +
 " Status: " + content.statusAsString() +
 " Date: " + content.submittedDate()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

```

- For API details, see [ListOperations](#) in *AWS SDK for Java 2.x API Reference*.

## ListPrices

The following code example shows how to use ListPrices.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listPrices(Route53DomainsClient route53DomainsClient, String
domainType) {
 try {
 ListPricesRequest pricesRequest = ListPricesRequest.builder()
 .tld(domainType)
 .build();

 ListPricesIterable listRes =
route53DomainsClient.listPricesPaginator(pricesRequest);
 listRes.stream()
 .flatMap(r -> r.prices().stream())
 .forEach(content -> System.out.println(" Name: " +
content.name() +
 " Registration: " + content.registrationPrice().price()
+ " "
 + content.registrationPrice().currency() +
 " Renewal: " + content.renewalPrice().price() + " " +
content.renewalPrice().currency()));

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListPrices](#) in *AWS SDK for Java 2.x API Reference*.

## RegisterDomain

The following code example shows how to use RegisterDomain.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).



```
public static String requestDomainRegistration(Route53DomainsClient
route53DomainsClient,
 String domainSuggestion,
 String phoneNumber,
 String email,
 String firstName,
 String lastName,
 String city) {

 try {
 ContactDetail contactDetail = ContactDetail.builder()
 .contactType(ContactType.COMPANY)
 .state("LA")
 .countryCode(CountryCode.IN)
 .email(email)
 .firstName(firstName)
 .lastName(lastName)
 .city(city)
 .phoneNumber(phoneNumber)
 .organizationName("My Org")
 .addressLine1("My Address")
 .zipCode("123 123")
 .build();

 RegisterDomainRequest domainRequest = RegisterDomainRequest.builder()
 .adminContact(contactDetail)
 .registrantContact(contactDetail)
 .techContact(contactDetail)
 .domainName(domainSuggestion)
 .autoRenew(true)
 .durationInYears(1)
 .build();

 RegisterDomainResponse response =
route53DomainsClient.registerDomain(domainRequest);
 System.out.println("Registration requested. Operation Id: " +
response.operationId());
 return response.operationId();

 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

```
 return "";
 }
}
```

- For API details, see [RegisterDomain](#) in *AWS SDK for Java 2.x API Reference*.

## ViewBilling

The following code example shows how to use ViewBilling.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void listBillingRecords(Route53DomainsClient route53DomainsClient)
{
 try {
 Date currentDate = new Date();
 LocalDateTime localDateTime =
currentDate.toInstant().atZone(ZoneId.systemDefault()).toLocalDateTime();
 ZoneOffset zoneOffset = ZoneOffset.of("+01:00");
 LocalDateTime localDateTime2 = localDateTime.minusYears(1);
 Instant myStartTime = localDateTime2.toInstant(zoneOffset);
 Instant myEndTime = localDateTime.toInstant(zoneOffset);

 ViewBillingRequest viewBillingRequest = ViewBillingRequest.builder()
 .start(myStartTime)
 .end(myEndTime)
 .build();

 ViewBillingIterable listRes =
route53DomainsClient.viewBillingPaginator(viewBillingRequest);
 listRes.stream()
 .flatMap(r -> r.billingRecords().stream())
 .forEach(content -> System.out.println(" Bill Date:: " +
content.billDate() +
 " Operation: " + content.operationAsString() +
 " Price: " + content.price()));
 }
}
```

```
 } catch (Route53Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
```

- For API details, see [ViewBilling](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon S3 examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon S3.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)

## Get started

### Hello Amazon S3

The following code example shows how to get started using Amazon S3.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.Bucket;
import software.amazon.awssdk.services.s3.model.ListBucketsResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloS3 {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 listBuckets(s3);
 }

 /**
 * Lists all the S3 buckets associated with the provided AWS S3 client.
 *
 * @param s3 the S3Client instance used to interact with the AWS S3 service
 */
 public static void listBuckets(S3Client s3) {
 try {
 ListBucketsResponse response = s3.listBuckets();
 }
 }
}
```

```
 List<Bucket> bucketList = response.buckets();
 bucketList.forEach(bucket -> {
 System.out.println("Bucket Name: " + bucket.name());
 });

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [ListBuckets](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a bucket and upload a file to it.
- Download an object from a bucket.
- Copy an object to a subfolder in a bucket.
- List the objects in a bucket.
- Delete the bucket objects and the bucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

A scenario example.

```
import java.io.IOException;
```

```

import java.util.Scanner;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * This Java code example performs the following tasks:
 *
 * 1. Creates an Amazon S3 bucket.
 * 2. Uploads an object to the bucket.
 * 3. Downloads the object to another local file.
 * 4. Uploads an object using multipart upload.
 * 5. List all objects located in the Amazon S3 bucket.
 * 6. Copies the object to another Amazon S3 bucket.
 * 7. Copy the object to another Amazon S3 bucket using multi copy.
 * 8. Deletes the object from the Amazon S3 bucket.
 * 9. Deletes the Amazon S3 bucket.
 */

public class S3Scenario {

 public static Scanner scanner = new Scanner(System.in);
 static S3Actions s3Actions = new S3Actions();
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final Logger logger = LoggerFactory.getLogger(S3Scenario.class);
 public static void main(String[] args) throws IOException {
 final String usage = ""
 Usage:
 <bucketName> <key> <objectPath> <savePath> <toBucket>

 Where:
 bucketName - The name of the S3 bucket.
 key - The unique identifier for the object stored in the S3 bucket.

```

objectPath - The full file path of the object within the S3 bucket (e.g., "documents/reports/annual\_report.pdf").

savePath - The local file path where the object will be downloaded and saved (e.g., "C:/Users/username/Downloads/annual\_report.pdf").

toBucket - The name of the S3 bucket to which the object will be copied.

```

 """;

 if (args.length != 5) {
 logger.info(usage);
 return;
 }

 String bucketName = args[0];
 String key = args[1];
 String objectPath = args[2];
 String savePath = args[3];
 String toBucket = args[4];

 logger.info(DASHES);
 logger.info("Welcome to the Amazon Simple Storage Service (S3) example scenario.");
 logger.info("""
 Amazon S3 is a highly scalable and durable object storage
 service provided by Amazon Web Services (AWS). It is designed to store
 and retrieve
 any amount of data, from anywhere on the web, at any time.

 The `S3AsyncClient` interface in the AWS SDK for Java 2.x provides a set
 of methods to
 programmatically interact with the Amazon S3 (Simple Storage Service)
 service. This allows
 developers to automate the management and manipulation of S3 buckets and
 objects as
 part of their application deployment pipelines. With S3, teams can focus
 on building
 and deploying their applications without having to worry about the
 underlying storage
 infrastructure required to host and manage large amounts of data.

 This scenario walks you through how to perform key operations for this
 service.

 Let's get started...
 """);

```

```

 waitForInputToContinue(scanner);
 logger.info(DASHES);

 try {
 // Run the methods that belong to this scenario.
 runScenario(bucketName, key, objectPath, savePath, toBucket);

 } catch (Throwable rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception kmsEx) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
 kmsEx.getMessage(), kmsEx.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 }
 }

 private static void runScenario(String bucketName, String key, String
objectPath, String savePath, String toBucket) throws Throwable {
 logger.info(DASHES);
 logger.info("1. Create an Amazon S3 bucket.");
 try {
 CompletableFuture<Void> future =
s3Actions.createBucketAsync(bucketName);
 future.join();
 waitForInputToContinue(scanner);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
 s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }

 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("2. Upload a local file to the Amazon S3 bucket.");
 waitForInputToContinue(scanner);

```



```

 try {
 CompletableFuture<PutObjectResponse> future =
s3Actions.uploadLocalFileAsync(bucketName, key, objectPath);
 future.join();
 logger.info("File uploaded successfully to {}/{}", bucketName, key);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("3. Download the object to another local file.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
s3Actions.getObjectBytesAsync(bucketName, key, savePath);
 future.join();
 logger.info("Successfully obtained bytes from S3 object and wrote to
file {}", savePath);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);

```

```
logger.info("4. Perform a multipart upload.");
waitForInputToContinue(scanner);
String multipartKey = "multiPartKey";
try {
 // Call the multipartUpload method
 CompletableFuture<Void> future = s3Actions.multipartUpload(bucketName,
multipartKey);
 future.join();
 logger.info("Multipart upload completed successfully for bucket '{}' and
key '{}'", bucketName, multipartKey);

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("5. List all objects located in the Amazon S3 bucket.");
waitForInputToContinue(scanner);
try {
 CompletableFuture<Void> future =
s3Actions.listAllObjectsAsync(bucketName);
 future.join();
 logger.info("Object listing completed successfully.");

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
waitForInputToContinue(scanner);
```

```
logger.info(DASHES);

logger.info(DASHES);
logger.info("6. Copy the object to another Amazon S3 bucket.");
waitForInputToContinue(scanner);
try {
 CompletableFuture<String> future =
s3Actions.copyBucketObjectAsync(bucketName, key, toBucket);
 String result = future.join();
 logger.info("Copy operation result: {}", result);

} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
}
waitForInputToContinue(scanner);
logger.info(DASHES);

logger.info(DASHES);
logger.info("7. Copy the object to another Amazon S3 bucket using multi
copy.");
waitForInputToContinue(scanner);

try {
 CompletableFuture<String> future = s3Actions.performMultiCopy(toBucket,
bucketName, key);
 String result = future.join();
 logger.info("Copy operation result: {}", result);

} catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("KMS error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
}
}
```

```
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("8. Delete objects from the Amazon S3 bucket.");
 waitForInputToContinue(scanner);
 try {
 CompletableFuture<Void> future =
s3Actions.deleteObjectFromBucketAsync(bucketName, key);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 try {
 CompletableFuture<Void> future =
s3Actions.deleteObjectFromBucketAsync(bucketName, "multiPartKey");
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("9. Delete the Amazon S3 bucket.");
 waitForInputToContinue(scanner);
 try {
```

```

 CompletableFuture<Void> future =
s3Actions.deleteBucketAsync(bucketName);
 future.join();

 } catch (RuntimeException rt) {
 Throwable cause = rt.getCause();
 if (cause instanceof S3Exception s3Ex) {
 logger.info("S3 error occurred: Error message: {}, Error code {}",
s3Ex.getMessage(), s3Ex.awsErrorDetails().errorCode());
 } else {
 logger.info("An unexpected error occurred: " + rt.getMessage());
 }
 throw cause;
 }
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 logger.info(DASHES);
 logger.info("You successfully completed the Amazon S3 scenario.");
 logger.info(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 // Handle invalid input.
 logger.info("Invalid input. Please try again.");
 }
 }
}
}
}
}

```

A wrapper class that contains the operations.

```

public class S3Actions {

 private static final Logger logger = LoggerFactory.getLogger(S3Actions.class);
 private static S3AsyncClient s3AsyncClient;

 public static S3AsyncClient getAsyncClient() {
 if (s3AsyncClient == null) {
 /*
 The `NettyNioAsyncHttpClient` class is part of the AWS SDK for Java,
version 2,
 and it is designed to provide a high-performance, asynchronous HTTP
client for interacting with AWS services.
 It uses the Netty framework to handle the underlying network
communication and the Java NIO API to
 provide a non-blocking, event-driven approach to HTTP requests and
responses.
 */

 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(50) // Adjust as needed.
 .connectionTimeout(Duration.ofSeconds(60)) // Set the connection
timeout.
 .readTimeout(Duration.ofSeconds(60)) // Set the read timeout.
 .writeTimeout(Duration.ofSeconds(60)) // Set the write timeout.
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2)) // Set the overall API call
timeout.
 .apiCallAttemptTimeout(Duration.ofSeconds(90)) // Set the
individual call attempt timeout.
 .retryStrategy(RetryMode.STANDARD)
 .build();

 s3AsyncClient = S3AsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return s3AsyncClient;
 }
}

```

```

/**
 * Creates an S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to create
 * @return a {@link CompletableFuture} that completes when the bucket is created
and ready
 * @throws RuntimeException if there is a failure while creating the bucket
 */
public CompletableFuture<Void> createBucketAsync(String bucketName) {
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 CompletableFuture<CreateBucketResponse> response =
getAsyncClient().createBucket(bucketRequest);
 return response.thenCompose(resp -> {
 S3AsyncWaiter s3Waiter = getAsyncClient().waiter();
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 CompletableFuture<WaiterResponse<HeadBucketResponse>>
waiterResponseFuture =
 s3Waiter.waitUntilBucketExists(bucketRequestWait);
 return waiterResponseFuture.thenAccept(waiterResponse -> {
 waiterResponse.matched().response().ifPresent(headBucketResponse ->
{
 logger.info(bucketName + " is ready");
 });
 });
 }).whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to create bucket", ex);
 }
 });
}

/**
 * Uploads a local file to an AWS S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to upload the file to

```

```

 * @param key the key (object name) to use for the uploaded file
 * @param objectPath the local file path of the file to be uploaded
 * @return a {@link CompletableFuture} that completes with the {@link
 PutObjectResponse} when the upload is successful, or throws a {@link
 RuntimeException} if the upload fails
 */
 public CompletableFuture<PutObjectResponse> uploadLocalFileAsync(String
 bucketName, String key, String objectPath) {
 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 CompletableFuture<PutObjectResponse> response =
 getAsyncClient().putObject(objectRequest,
 AsyncRequestBody.fromFile(Paths.get(objectPath)));
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to upload file", ex);
 }
 });
 }

 /**
 * Asynchronously retrieves the bytes of an object from an Amazon S3 bucket and
 writes them to a local file.
 *
 * @param bucketName the name of the S3 bucket containing the object
 * @param keyName the key (or name) of the S3 object to retrieve
 * @param path the local file path where the object's bytes will be
 written
 * @return a {@link CompletableFuture} that completes when the object bytes have
 been written to the local file
 */
 public CompletableFuture<Void> getObjectBytesAsync(String bucketName, String
 keyName, String path) {
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 CompletableFuture<ResponseBytes<GetObjectResponse>> response =
 getAsyncClient().getObject(objectRequest, AsyncResponseTransformer.toBytes());

```



```

 return response.thenAccept(objectBytes -> {
 try {
 byte[] data = objectBytes.asByteArray();
 Path filePath = Paths.get(path);
 Files.write(filePath, data);
 logger.info("Successfully obtained bytes from an S3 object");
 } catch (IOException ex) {
 throw new RuntimeException("Failed to write data to file", ex);
 }
 }).whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to get object bytes from S3",
ex);
 }
 });
 }

 /**
 * Asynchronously lists all objects in the specified S3 bucket.
 *
 * @param bucketName the name of the S3 bucket to list objects for
 * @return a {@link CompletableFuture} that completes when all objects have been
listed
 */
 public CompletableFuture<Void> listAllObjectsAsync(String bucketName) {
 ListObjectsV2Request initialRequest = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .maxKeys(1)
 .build();

 ListObjectsV2Publisher paginator =
getAsyncClient().listObjectsV2Paginator(initialRequest);
 return paginator.subscribe(response -> {
 response.contents().forEach(s3Object -> {
 logger.info("Object key: " + s3Object.key());
 });
 }).thenRun(() -> {
 logger.info("Successfully listed all objects in the bucket: " +
bucketName);
 }).exceptionally(ex -> {
 throw new RuntimeException("Failed to list objects", ex);
 });
 }
}

```

```

/**
 * Asynchronously copies an object from one S3 bucket to another.
 *
 * @param fromBucket the name of the source S3 bucket
 * @param objectKey the key (name) of the object to be copied
 * @param toBucket the name of the destination S3 bucket
 * @return a {@link CompletableFuture} that completes with the copy result as a
 * {@link String}
 * @throws RuntimeException if the URL could not be encoded or an S3 exception
 * occurred during the copy
 */
public CompletableFuture<String> copyBucketObjectAsync(String fromBucket, String
objectKey, String toBucket) {
 CopyObjectRequest copyReq = CopyObjectRequest.builder()
 .sourceBucket(fromBucket)
 .sourceKey(objectKey)
 .destinationBucket(toBucket)
 .destinationKey(objectKey)
 .build();

 CompletableFuture<CopyObjectResponse> response =
getAsyncClient().copyObject(copyReq);
 response.whenComplete((copyRes, ex) -> {
 if (copyRes != null) {
 logger.info("The " + objectKey + " was copied to " + toBucket);
 } else {
 throw new RuntimeException("An S3 exception occurred during copy",
ex);
 }
 });

 return response.thenApply(CopyObjectResponse::copyObjectResult)
 .thenApply(Object::toString);
}

/**
 * Performs a multipart upload to an Amazon S3 bucket.
 *
 * @param bucketName the name of the S3 bucket to upload the file to
 * @param key the key (name) of the file to be uploaded
 * @return a {@link CompletableFuture} that completes when the multipart upload
 * is successful

```

```

 */
 public CompletableFuture<Void> multipartUpload(String bucketName, String key) {
 int mB = 1024 * 1024;

 CreateMultipartUploadRequest createMultipartUploadRequest =
 CreateMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 return getAsyncClient().createMultipartUpload(createMultipartUploadRequest)
 .thenCompose(createResponse -> {
 String uploadId = createResponse.uploadId();
 System.out.println("Upload ID: " + uploadId);

 // Upload part 1.
 UploadPartRequest uploadPartRequest1 = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .partNumber(1)
 .contentLength((long) (5 * mB)) // Specify the content length
 .build();

 CompletableFuture<CompletedPart> part1Future =
 getAsyncClient().uploadPart(uploadPartRequest1,
 AsyncRequestBody.fromByteBuffer(getRandomByteBuffer(5 *
mB)))

 .thenApply(uploadPartResponse -> CompletedPart.builder()
 .partNumber(1)
 .eTag(uploadPartResponse.eTag())
 .build());

 // Upload part 2.
 UploadPartRequest uploadPartRequest2 = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .partNumber(2)
 .contentLength((long) (3 * mB))
 .build();

 CompletableFuture<CompletedPart> part2Future =
 getAsyncClient().uploadPart(uploadPartRequest2,

```

```

 AsyncRequestBody.fromByteBuffer(getRandomByteBuffer(3 *
mB)))

 .thenApply(uploadPartResponse -> CompletedPart.builder()
 .partNumber(2)
 .eTag(uploadPartResponse.eTag())
 .build());

 // Combine the results of both parts.
 return CompletableFuture.allOf(part1Future, part2Future)
 .thenCompose(v -> {
 CompletedPart part1 = part1Future.join();
 CompletedPart part2 = part2Future.join();

 CompletedMultipartUpload completedMultipartUpload =
CompletedMultipartUpload.builder()
 .parts(part1, part2)
 .build();

 CompleteMultipartUploadRequest
completeMultipartUploadRequest = CompleteMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .multipartUpload(completedMultipartUpload)
 .build();

 // Complete the multipart upload
 return
getAsyncClient().completeMultipartUpload(completeMultipartUploadRequest);
 });

 })
 .thenAccept(response -> System.out.println("Multipart upload completed
successfully"))
 .exceptionally(ex -> {
 System.err.println("Failed to complete multipart upload: " +
ex.getMessage());
 throw new RuntimeException(ex);
 });
}

/**
 * Deletes an object from an S3 bucket asynchronously.
 *

```

```

 * @param bucketName the name of the S3 bucket
 * @param key the key (file name) of the object to be deleted
 * @return a {@link CompletableFuture} that completes when the object has been
deleted
 */
 public CompletableFuture<Void> deleteObjectFromBucketAsync(String bucketName,
String key) {
 DeleteObjectRequest deleteObjectRequest = DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 CompletableFuture<DeleteObjectResponse> response =
getAsyncClient().deleteObject(deleteObjectRequest);
 response.whenComplete((deleteRes, ex) -> {
 if (deleteRes != null) {
 logger.info(key + " was deleted");
 } else {
 throw new RuntimeException("An S3 exception occurred during delete",
ex);
 }
 });

 return response.thenApply(r -> null);
 }

 /**
 * Deletes an S3 bucket asynchronously.
 *
 * @param bucket the name of the bucket to be deleted
 * @return a {@link CompletableFuture} that completes when the bucket deletion
is successful, or throws a {@link RuntimeException}
 * if an error occurs during the deletion process
 */
 public CompletableFuture<Void> deleteBucketAsync(String bucket) {
 DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()
 .bucket(bucket)
 .build();

 CompletableFuture<DeleteBucketResponse> response =
getAsyncClient().deleteBucket(deleteBucketRequest);
 response.whenComplete((deleteRes, ex) -> {
 if (deleteRes != null) {

```

```

 logger.info(bucket + " was deleted.");
 } else {
 throw new RuntimeException("An S3 exception occurred during bucket
deletion", ex);
 }
});
return response.thenApply(r -> null);
}

public CompletableFuture<String> performMultiCopy(String toBucket, String
bucketName, String key) {
 CreateMultipartUploadRequest createMultipartUploadRequest =
CreateMultipartUploadRequest.builder()
 .bucket(toBucket)
 .key(key)
 .build();

 getAsyncClient().createMultipartUpload(createMultipartUploadRequest)
 .thenApply(createMultipartUploadResponse -> {
 String uploadId = createMultipartUploadResponse.uploadId();
 System.out.println("Upload ID: " + uploadId);

 UploadPartCopyRequest uploadPartCopyRequest =
UploadPartCopyRequest.builder()
 .sourceBucket(bucketName)
 .destinationBucket(toBucket)
 .sourceKey(key)
 .destinationKey(key)
 .uploadId(uploadId) // Use the valid uploadId.
 .partNumber(1) // Ensure the part number is correct.
 .copySourceRange("bytes=0-1023") // Adjust range as needed
 .build();

 return getAsyncClient().uploadPartCopy(uploadPartCopyRequest);
 })
 .thenCompose(uploadPartCopyFuture -> uploadPartCopyFuture)
 .whenComplete((uploadPartCopyResponse, exception) -> {
 if (exception != null) {
 // Handle any exceptions.
 logger.error("Error during upload part copy: " +
exception.getMessage());
 } else {
 // Successfully completed the upload part copy.

```

```
 System.out.println("Upload Part Copy completed successfully.
ETag: " + uploadPartCopyResponse.copyPartResult().eTag());
 }
 });
 return null;
}

private static ByteBuffer getRandomByteBuffer(int size) {
 ByteBuffer buffer = ByteBuffer.allocate(size);
 for (int i = 0; i < size; i++) {
 buffer.put((byte) (Math.random() * 256));
 }
 buffer.flip();
 return buffer;
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CopyObject](#)
  - [CreateBucket](#)
  - [DeleteBucket](#)
  - [DeleteObjects](#)
  - [GetObject](#)
  - [ListObjectsV2](#)
  - [PutObject](#)

## Actions

### AbortMultipartUpload

The following code example shows how to use `AbortMultipartUpload`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.AbortMultipartUploadRequest;
import software.amazon.awssdk.services.s3.model.AbortMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.CompletedMultipartUpload;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.LifecycleRule;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsRequest;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsResponse;
import software.amazon.awssdk.services.s3.model.MultipartUpload;
import
 software.amazon.awssdk.services.s3.model.PutBucketLifecycleConfigurationResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.UploadPartRequest;
import software.amazon.awssdk.services.s3.model.UploadPartResponse;
import software.amazon.awssdk.services.s3.waiters.S3Waiter;
import software.amazon.awssdk.services.sts.StsClient;
import software.amazon.awssdk.utils.builder.SdkBuilder;

import java.io.IOException;
import java.io.RandomAccessFile;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.ByteBuffer;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
```



```

import java.util.Objects;
import java.util.UUID;

import static software.amazon.awssdk.transfer.s3.SizeConstant.KB;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class AbortMultipartUploadExamples {
 static final S3Client s3Client = S3Client.create();
 static final String classPathFilePath = "/multipartUploadFiles/s3-
userguide.pdf";
 static final String filePath = getFullFilePath(classPathFilePath);
 private static final Logger logger =
LoggerFactory.getLogger(AbortMultipartUploadExamples.class);
 private static final String accountId = getAccountId();

 public static void main(String[] args) {
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 String key = UUID.randomUUID().toString();

 createBucket(bucketName);
 try {
 initiateAndInterruptMultiPartUpload(bucketName, key, "uploadThread");
 abortIncompleteMultipartUploadsFromList(bucketName);
 abortMultipartUploadUsingUploadId(bucketName, key);
 abortMultipartUploadsUsingLifecycleConfig(bucketName);
 } catch (S3Exception e) {
 logger.error(e.getMessage());
 } finally {
 deleteResources(bucketName, key);
 }
 }

 /**
 * Aborts all incomplete multipart uploads from the specified S3 bucket.
 *
 * @param bucketName the name of the S3 bucket

```

```

 */
 public static void abortIncompleteMultipartUploadsFromList(String bucketName) {
 ListMultipartUploadsRequest listMultipartUploadsRequest =
ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 ListMultipartUploadsResponse response =
s3Client.listMultipartUploads(listMultipartUploadsRequest);
 List<MultipartUpload> uploads = response.uploads();

 AbortMultipartUploadRequest abortMultipartUploadRequest;
 for (MultipartUpload upload : uploads) {
 abortMultipartUploadRequest = AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(upload.key())
 .expectedBucketOwner(accountId)
 .uploadId(upload.uploadId())
 .build();

 AbortMultipartUploadResponse abortMultipartUploadResponse =
s3Client.abortMultipartUpload(abortMultipartUploadRequest);
 if (abortMultipartUploadResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully aborted.",
upload.uploadId(), bucketName);
 }
 }
 }

 /**
 * Aborts incomplete multipart uploads older than the specified point in time.
 *
 * @param bucketName the name of the S3 bucket
 * @param pointInTime the cutoff time; uploads initiated before this are aborted
 */
 static void abortIncompleteMultipartUploadsOlderThan(String bucketName, Instant
pointInTime) {
 ListMultipartUploadsRequest listMultipartUploadsRequest =
ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 ListMultipartUploadsResponse response =
s3Client.listMultipartUploads(listMultipartUploadsRequest);

```

```

 List<MultipartUpload> uploads = response.uploads();

 AbortMultipartUploadRequest abortMultipartUploadRequest;
 for (MultipartUpload upload : uploads) {
 logger.info("Found multipartUpload with upload ID [{}], initiated [{}]",
upload.uploadId(), upload.initiated());
 if (upload.initiated().isBefore(pointInTime)) {
 abortMultipartUploadRequest = AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(upload.key())
 .expectedBucketOwner(accountId)
 .uploadId(upload.uploadId())
 .build();

 AbortMultipartUploadResponse abortMultipartUploadResponse =
s3Client.abortMultipartUpload(abortMultipartUploadRequest);
 if (abortMultipartUploadResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully
aborted.", upload.uploadId(), bucketName);
 }
 }
 }
 }

 /**
 * Aborts a multipart upload using the upload ID.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the object key
 */
 static void abortMultipartUploadUsingUploadId(String bucketName, String key) {
 String uploadId = startUploadReturningUploadId(bucketName, key);
 AbortMultipartUploadResponse response = s3Client.abortMultipartUpload(b -> b
 .uploadId(uploadId)
 .bucket(bucketName)
 .key(key));

 if (response.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully aborted.",
uploadId, bucketName);
 }
 }

 /**

```

```

 * Configures a lifecycle rule to abort incomplete multipart uploads after 7
 days.
 *
 * @param bucketName the name of the S3 bucket
 */
 static void abortMultipartUploadsUsingLifecycleConfig(String bucketName) {
 Collection<LifecycleRule> lifeCycleRules = List.of(LifecycleRule.builder()
 .abortIncompleteMultipartUpload(b -> b.
 daysAfterInitiation(7))
 .status("Enabled")
 .filter(SdkBuilder::build) // Filter element is required.
 .build());

 PutBucketLifecycleConfigurationResponse response =
s3Client.putBucketLifecycleConfiguration(b -> b
 .bucket(bucketName)
 .lifecycleConfiguration(b1 -> b1.rules(lifeCycleRules)));

 if (response.sdkHttpResponse().isSuccessful()) {
 logger.info("Rule to abort incomplete multipart uploads added to
bucket.");
 } else {
 logger.error("Unsuccessfully applied rule. HTTP status code is [{}]",
response.sdkHttpResponse().statusCode());
 }
 }

 /*****
 Multipart upload methods
 *****/

 static void initiateAndInterruptMultiPartUpload(String bucketName, String key,
String threadName) {
 Runnable upload = () -> {
 try {
 doMultipartUpload(bucketName, key);
 } catch (SdkException e) {
 logger.error(e.getMessage());
 }
 };
 Thread uploadThread = new Thread(upload, threadName);
 uploadThread.start();
 try {
 Thread.sleep(Duration.ofSeconds(1).toMillis());

```

```

 } catch (InterruptedException e) {
 logger.error(e.getMessage());
 }
 uploadThread.interrupt();
 }

 static Instant initiateAndInterruptTwoUploads(String bucketName, String key) {
 initiateAndInterruptMultiPartUpload(bucketName, key, "uploadThread1");
 try {
 Thread.sleep(Duration.ofSeconds(5).toMillis());
 } catch (InterruptedException e) {
 logger.error(e.getMessage());
 }
 Instant secondUploadInstant = Instant.now();
 initiateAndInterruptMultiPartUpload(bucketName, key, "uploadThread2");
 return secondUploadInstant;
 }

 static void doMultipartUpload(String bucketName, String key) {
 String uploadId = step1CreateMultipartUpload(bucketName, key);
 List<CompletedPart> completedParts = step2UploadParts(bucketName, key,
uploadId);
 step3CompleteMultipartUpload(bucketName, key, uploadId, completedParts);
 }

 static String step1CreateMultipartUpload(String bucketName, String key) {
 CreateMultipartUploadResponse createMultipartUploadResponse =
s3Client.createMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key));
 return createMultipartUploadResponse.uploadId();
 }

 static List<CompletedPart> step2UploadParts(String bucketName, String key,
String uploadId) {
 int partNumber = 1;
 List<CompletedPart> completedParts = new ArrayList<>();
 ByteBuffer bb = ByteBuffer.allocate(Long.valueOf(1024 * KB).intValue());

 try (RandomAccessFile file = new RandomAccessFile(filePath, "r")) {
 long fileSize = file.length();
 long position = 0;
 while (position < fileSize) {
 file.seek(position);

```

```

 long read = file.getChannel().read(bb);

 bb.flip();
 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .partNumber(partNumber)
 .build();

 UploadPartResponse partResponse = s3Client.uploadPart(
 uploadPartRequest,
 RequestBody.fromByteBuffer(bb));

 CompletedPart part = CompletedPart.builder()
 .partNumber(partNumber)
 .eTag(partResponse.eTag())
 .build();
 completedParts.add(part);
 logger.info("Part {} upload", partNumber);

 bb.clear();
 position += read;
 partNumber++;
 }
} catch (IOException | S3Exception e) {
 logger.error(e.getMessage());
 return null;
}
return completedParts;
}

static void step3CompleteMultipartUpload(String bucketName, String key, String
uploadId, List<CompletedPart> completedParts) {
 s3Client.completeMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)

 .multipartUpload(CompletedMultipartUpload.builder().parts(completedParts).build()));
}

static String startUploadReturningUploadId(String bucketName, String key) {
 String uploadId = step1CreateMultipartUpload(bucketName, key);

```

```

 doMultipartUploadWithUploadId(bucketName, key, uploadId);
 return uploadId;
 }

 static void doMultipartUploadWithUploadId(String bucketName, String key, String
uploadId) {
 new Thread(() -> {
 try {
 List<CompletedPart> completedParts = step2UploadParts(bucketName,
key, uploadId);
 step3CompleteMultipartUpload(bucketName, key, uploadId,
completedParts);
 } catch (SdkException e) {
 logger.error(e.getMessage());
 }
 }, "upload thread").start();
 try {
 Thread.sleep(Duration.ofSeconds(2L).toMillis());
 } catch (InterruptedException e) {
 logger.error(e.getMessage());
 System.exit(1);
 }
 }

 /*****
 Resource handling methods
 *****/

 static void createBucket(String bucketName) {
 logger.info("Creating bucket: [{}]", bucketName);
 s3Client.createBucket(b -> b.bucket(bucketName));
 try (S3Waiter s3Waiter = s3Client.waiter()) {
 s3Waiter.waitUntilBucketExists(b -> b.bucket(bucketName));
 }
 logger.info("Bucket created.");
 }

 static void deleteResources(String bucketName, String key) {
 logger.info("Deleting resources ...");
 s3Client.deleteObject(b -> b.bucket(bucketName).key(key));
 s3Client.deleteBucket(b -> b.bucket(bucketName));
 try (S3Waiter s3Waiter = s3Client.waiter()) {
 s3Waiter.waitUntilBucketNotExists(b -> b.bucket(bucketName));
 }
 }

```

```

 logger.info("Resources deleted.");
 }

 private static String getAccountId() {
 try (StsClient stsClient = StsClient.create()) {
 return stsClient.getCallerIdentity().account();
 }
 }

 static String getFullFilePath(String filePath) {
 URL uploadDirectoryURL = PerformMultiPartUpload.class.getResource(filePath);
 String fullFilePath;
 try {
 fullFilePath =
Objects.requireNonNull(uploadDirectoryURL).toURI().getPath();
 } catch (URISyntaxException e) {
 throw new RuntimeException(e);
 }
 return fullFilePath;
 }
}

```

- For API details, see [AbortMultipartUpload](#) in *AWS SDK for Java 2.x API Reference*.

## CopyObject

The following code example shows how to use CopyObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Copy an object using an [S3Client](#).

```

/**
 * Asynchronously copies an object from one S3 bucket to another.

```



```

 *
 * @param fromBucket the name of the source S3 bucket
 * @param objectKey the key (name) of the object to be copied
 * @param toBucket the name of the destination S3 bucket
 * @return a {@link CompletableFuture} that completes with the copy result as a
 * {@link String}
 * @throws RuntimeException if the URL could not be encoded or an S3 exception
 * occurred during the copy
 */
 public CompletableFuture<String> copyBucketObjectAsync(String fromBucket, String
 objectKey, String toBucket) {
 CopyObjectRequest copyReq = CopyObjectRequest.builder()
 .sourceBucket(fromBucket)
 .sourceKey(objectKey)
 .destinationBucket(toBucket)
 .destinationKey(objectKey)
 .build();

 CompletableFuture<CopyObjectResponse> response =
 getAsyncClient().copyObject(copyReq);
 response.whenComplete((copyRes, ex) -> {
 if (copyRes != null) {
 logger.info("The " + objectKey + " was copied to " + toBucket);
 } else {
 throw new RuntimeException("An S3 exception occurred during copy",
 ex);
 }
 });

 return response.thenApply(CopyObjectResponse::copyObjectResult)
 .thenApply(Object::toString);
 }

```

Use an [S3TransferManager](#) to [copy an object](#) from one bucket to another. View the [complete file](#) and [test](#).

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.model.CopyObjectRequest;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedCopy;

```

```
import software.amazon.awssdk.transfer.s3.model.Copy;
import software.amazon.awssdk.transfer.s3.model.CopyRequest;

import java.util.UUID;

public String copyObject(S3TransferManager transferManager, String bucketName,
 String key, String destinationBucket, String destinationKey) {
 CopyObjectRequest copyObjectRequest = CopyObjectRequest.builder()
 .sourceBucket(bucketName)
 .sourceKey(key)
 .destinationBucket(destinationBucket)
 .destinationKey(destinationKey)
 .build();

 CopyRequest copyRequest = CopyRequest.builder()
 .copyObjectRequest(copyObjectRequest)
 .build();

 Copy copy = transferManager.copy(copyRequest);

 CompletedCopy completedCopy = copy.completionFuture().join();
 return completedCopy.response().copyObjectResult().eTag();
}
```

- For API details, see [CopyObject](#) in *AWS SDK for Java 2.x API Reference*.

## CreateBucket

The following code example shows how to use CreateBucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create a bucket.

```

/**
 * Creates an S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to create
 * @return a {@link CompletableFuture} that completes when the bucket is created
and ready
 * @throws RuntimeException if there is a failure while creating the bucket
 */
public CompletableFuture<Void> createBucketAsync(String bucketName) {
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 CompletableFuture<CreateBucketResponse> response =
getAsyncClient().createBucket(bucketRequest);
 return response.thenCompose(resp -> {
 S3AsyncWaiter s3Waiter = getAsyncClient().waiter();
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 CompletableFuture<WaiterResponse<HeadBucketResponse>>
waiterResponseFuture =
 s3Waiter.waitUntilBucketExists(bucketRequestWait);
 return waiterResponseFuture.thenAccept(waiterResponse -> {
 waiterResponse.matched().response().ifPresent(headBucketResponse ->
{
 logger.info(bucketName + " is ready");
 });
 });
 }).whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to create bucket", ex);
 }
 });
}

```

Create a bucket with object lock enabled.

```

// Create a new Amazon S3 bucket with object lock options.
public void createBucketWithLockOptions(boolean enableObjectLock, String
bucketName) {

```

```

S3Waiter s3Waiter = getClient().waiter();
CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .objectLockEnabledForBucket(enableObjectLock)
 .build();

getClient().createBucket(bucketRequest);
HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

// Wait until the bucket is created and print out the response.
s3Waiter.waitUntilBucketExists(bucketRequestWait);
System.out.println(bucketName + " is ready");
}

```

- For API details, see [CreateBucket](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucket

The following code example shows how to use DeleteBucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an S3 bucket asynchronously.
 *
 * @param bucket the name of the bucket to be deleted
 * @return a {@link CompletableFuture} that completes when the bucket deletion
 * is successful, or throws a {@link RuntimeException}
 * if an error occurs during the deletion process
 */
public CompletableFuture<Void> deleteBucketAsync(String bucket) {
 DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()

```

```

 .bucket(bucket)
 .build();

 CompletableFuture<DeleteBucketResponse> response =
getAsyncClient().deleteBucket(deleteBucketRequest);
 response.whenComplete((deleteRes, ex) -> {
 if (deleteRes != null) {
 logger.info(bucket + " was deleted.");
 } else {
 throw new RuntimeException("An S3 exception occurred during bucket
deletion", ex);
 }
 });
 return response.thenApply(r -> null);
}

```

- For API details, see [DeleteBucket](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucketPolicy

The following code example shows how to use DeleteBucketPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteBucketPolicyRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 */

```

```
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
```

```
public class DeleteBucketPolicy {
 public static void main(String[] args) {

 final String usage = ""

 Usage:
 <bucketName>

 Where:
 bucketName - The Amazon S3 bucket to delete the policy from (for
example, bucket1)."";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 System.out.format("Deleting policy from bucket: \"%s\"\n\n", bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 deleteS3BucketPolicy(s3, bucketName);
 s3.close();
 }

 /**
 * Deletes the S3 bucket policy for the specified bucket.
 *
 * @param s3 the {@link S3Client} instance to use for the operation
 * @param bucketName the name of the S3 bucket for which the policy should be
deleted
 *
 * @throws S3Exception if there is an error deleting the bucket policy
 */
 public static void deleteS3BucketPolicy(S3Client s3, String bucketName) {
 DeleteBucketPolicyRequest delReq = DeleteBucketPolicyRequest.builder()
 .bucket(bucketName)
 .build();
 }
}
```

```
 try {
 s3.deleteBucketPolicy(delReq);
 System.out.println("Done!");
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucketWebsite

The following code example shows how to use DeleteBucketWebsite.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteBucketWebsiteRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
```

```

public class DeleteWebsiteConfiguration {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucketName>

 Where:
 bucketName - The Amazon S3 bucket to delete the website
configuration from.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 System.out.format("Deleting website configuration for Amazon S3 bucket: %s
\n", bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 deleteBucketWebsiteConfig(s3, bucketName);
 System.out.println("Done!");
 s3.close();
 }

 /**
 * Deletes the website configuration for an Amazon S3 bucket.
 *
 * @param s3 The {@link S3Client} instance used to interact with Amazon S3.
 * @param bucketName The name of the S3 bucket for which the website
configuration should be deleted.
 * @throws S3Exception If an error occurs while deleting the website
configuration.
 */
 public static void deleteBucketWebsiteConfig(S3Client s3, String bucketName) {
 DeleteBucketWebsiteRequest delReq = DeleteBucketWebsiteRequest.builder()
 .bucket(bucketName)
 .build();

 try {

```



```

 s3.deleteBucketWebsite(delReq);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.out.println("Failed to delete website configuration!");
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteBucketWebsite](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteObject

The following code example shows how to use DeleteObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an object from an S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the key (file name) of the object to be deleted
 * @return a {@link CompletableFuture} that completes when the object has been
 * deleted
 */
public CompletableFuture<Void> deleteObjectFromBucketAsync(String bucketName,
String key) {
 DeleteObjectRequest deleteObjectRequest = DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();
}

```

```

 CompletableFuture<DeleteObjectResponse> response =
getAsyncClient().deleteObject(deleteObjectRequest);
 response.whenComplete((deleteRes, ex) -> {
 if (deleteRes != null) {
 logger.info(key + " was deleted");
 } else {
 throw new RuntimeException("An S3 exception occurred during delete",
ex);
 }
 });

 return response.thenApply(r -> null);
 }

```

- For API details, see [DeleteObject](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteObjects

The following code example shows how to use DeleteObjects.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.services.s3.model.Delete;
import software.amazon.awssdk.services.s3.model.DeleteObjectsRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.util.ArrayList;

/**

```

```

* Before running this Java V2 code example, set up your development
* environment, including your credentials.
* <p>
* For more information, see the following documentation topic:
* <p>
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/

```

```

public class DeleteMultiObjects {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucketName>

 Where:
 bucketName - the Amazon S3 bucket name.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 deleteBucketObjects(s3, bucketName);
 s3.close();
 }

 /**
 * Deletes multiple objects from an Amazon S3 bucket.
 *
 * @param s3 An Amazon S3 client object.
 * @param bucketName The name of the Amazon S3 bucket to delete objects from.
 */
 public static void deleteBucketObjects(S3Client s3, String bucketName) {
 // Upload three sample objects to the specified Amazon S3 bucket.
 ArrayList<ObjectIdentifier> keys = new ArrayList<>();
 PutObjectRequest putOb;
 ObjectIdentifier objectId;
 }

```

```
for (int i = 0; i < 3; i++) {
 String keyName = "delete object example " + i;
 objectId = ObjectIdentifier.builder()
 .key(keyName)
 .build();

 putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();

 s3.putObject(putOb, RequestBody.fromString(keyName));
 keys.add(objectId);
}

System.out.println(keys.size() + " objects successfully created.");

// Delete multiple objects in one request.
Delete del = Delete.builder()
 .objects(keys)
 .build();

try {
 DeleteObjectsRequest multiObjectDeleteRequest =
DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(del)
 .build();

 s3.deleteObjects(multiObjectDeleteRequest);
 System.out.println("Multiple objects are deleted!");

} catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see [DeleteObjects](#) in *AWS SDK for Java 2.x API Reference*.

## GetBucketAcl

The following code example shows how to use `GetBucketAcl`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectAclRequest;
import software.amazon.awssdk.services.s3.model.GetObjectAclResponse;
import software.amazon.awssdk.services.s3.model.Grant;

import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class GetAcl {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <objectKey>

 Where:
 bucketName - The Amazon S3 bucket to get the access control list (ACL)
for.
 objectKey - The object to get the ACL for.\s
 """;
```

```

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String objectKey = args[1];
 System.out.println("Retrieving ACL for object: " + objectKey);
 System.out.println("in bucket: " + bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 getBucketACL(s3, objectKey, bucketName);
 s3.close();
 System.out.println("Done!");
 }

 /**
 * Retrieves the Access Control List (ACL) for an object in an Amazon S3 bucket.
 *
 * @param s3 The S3Client object used to interact with the Amazon S3 service.
 * @param objectKey The key of the object for which the ACL is to be retrieved.
 * @param bucketName The name of the bucket containing the object.
 * @return The ID of the grantee who has permission on the object, or an empty
 string if an error occurs.
 */
 public static String getBucketACL(S3Client s3, String objectKey, String
bucketName) {
 try {
 GetObjectAclRequest aclReq = GetObjectAclRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 GetObjectAclResponse aclRes = s3.getObjectAcl(aclReq);
 List<Grant> grants = aclRes.grants();
 String grantee = "";
 for (Grant grant : grants) {
 System.out.format(" %s: %s\n", grant.grantee().id(),
grant.permission());
 grantee = grant.grantee().id();
 }
 }
 }

```

```

 }

 return grantee;
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return "";
}
}

```

- For API details, see [GetBucketAcl](#) in *AWS SDK for Java 2.x API Reference*.

## GetBucketPolicy

The following code example shows how to use GetBucketPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetBucketPolicyRequest;
import software.amazon.awssdk.services.s3.model.GetBucketPolicyResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

```

```

public class GetBucketPolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName>

 Where:
 bucketName - The Amazon S3 bucket to get the policy from.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 System.out.format("Getting policy for bucket: \"%s\"\n\n", bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 String polText = getPolicy(s3, bucketName);
 System.out.println("Policy Text: " + polText);
 s3.close();
 }

 /**
 * Retrieves the policy for the specified Amazon S3 bucket.
 *
 * @param s3 the {@link S3Client} instance to use for making the request
 * @param bucketName the name of the S3 bucket for which to retrieve the policy
 * @return the policy text for the specified bucket, or an empty string if an
 error occurs
 */
 public static String getPolicy(S3Client s3, String bucketName) {
 String policyText;
 System.out.format("Getting policy for bucket: \"%s\"\n\n", bucketName);
 GetBucketPolicyRequest policyReq = GetBucketPolicyRequest.builder()
 .bucket(bucketName)
 .build();
 }

```



```

 try {
 GetBucketPolicyResponse policyRes = s3.getBucketPolicy(policyReq);
 policyText = policyRes.policy();
 return policyText;

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 return "";
}
}

```

- For API details, see [GetBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## GetBucketReplication

The following code example shows how to use `GetBucketReplication`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Retrieves the replication details for the specified S3 bucket.
 *
 * @param s3Client the S3 client used to interact with the S3 service
 * @param sourceBucketName the name of the S3 bucket to retrieve the
replication details for
 *
 * @throws S3Exception if there is an error retrieving the replication details
 */
public static void getReplicationDetails(S3Client s3Client, String
sourceBucketName) {
 GetBucketReplicationRequest getRequest =
GetBucketReplicationRequest.builder()

```

```

 .bucket(sourceBucketName)
 .build();

 try {
 ReplicationConfiguration replicationConfig =
s3Client.getBucketReplication(getRequest).replicationConfiguration();
 ReplicationRule rule = replicationConfig.rules().get(0);
 System.out.println("Retrieved destination bucket: " +
rule.destination().bucket());
 System.out.println("Retrieved priority: " + rule.priority());
 System.out.println("Retrieved source-bucket replication rule status: " +
rule.status());

 } catch (S3Exception e) {
 System.err.println("Failed to retrieve replication details: " +
e.awsErrorDetails().errorMessage());
 }
 }
}

```

- For API details, see [GetBucketReplication](#) in *AWS SDK for Java 2.x API Reference*.

## GetObject

The following code example shows how to use `GetObject`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Read data as a byte array using an [S3Client](#).

```

/**
 * Asynchronously retrieves the bytes of an object from an Amazon S3 bucket and
 * writes them to a local file.
 *
 * @param bucketName the name of the S3 bucket containing the object

```

```

 * @param keyName the key (or name) of the S3 object to retrieve
 * @param path the local file path where the object's bytes will be
written
 * @return a {@link CompletableFuture} that completes when the object bytes have
been written to the local file
 */
 public CompletableFuture<Void> getObjectBytesAsync(String bucketName, String
keyName, String path) {
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 CompletableFuture<ResponseBytes<GetObjectResponse>> response =
getAsyncClient().getObject(objectRequest, AsyncResponseTransformer.toBytes());
 return response.thenAccept(objectBytes -> {
 try {
 byte[] data = objectBytes.asByteArray();
 Path filePath = Paths.get(path);
 Files.write(filePath, data);
 logger.info("Successfully obtained bytes from an S3 object");
 } catch (IOException ex) {
 throw new RuntimeException("Failed to write data to file", ex);
 }
 }).whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to get object bytes from S3",
ex);
 }
 });
 }
}

```

Use an [S3TransferManager](#) to [download an object](#) in an S3 bucket to a local file. View the [complete file](#) and [test](#).

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedFileDownload;

```

```
import software.amazon.awssdk.transfer.s3.model.DownloadFileRequest;
import software.amazon.awssdk.transfer.s3.model.FileDownload;
import software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener;

import java.io.IOException;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.UUID;

 public Long downloadFile(S3TransferManager transferManager, String bucketName,
 String key, String downloadedFileWithPath) {
 DownloadFileRequest downloadFileRequest = DownloadFileRequest.builder()
 .getObjectRequest(b -> b.bucket(bucketName).key(key))
 .destination(Paths.get(downloadedFileWithPath))
 .build();

 FileDownload downloadFile =
transferManager.downloadFile(downloadFileRequest);

 CompletedFileDownload downloadResult =
downloadFile.completionFuture().join();
 logger.info("Content length [{ }]",
downloadResult.response().contentLength());
 return downloadResult.response().contentLength();
 }
```

Read tags that belong to an object using an [S3Client](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectTaggingRequest;
import software.amazon.awssdk.services.s3.model.GetObjectTaggingResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.Tag;

import java.util.List;
```

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class GetObjectTags {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName>\s

 Where:
 bucketName - The Amazon S3 bucket name.\s
 keyName - A key name that represents the object.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String keyName = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 listTags(s3, bucketName, keyName);
 s3.close();
 }

 /**
 * Lists the tags associated with an Amazon S3 object.
 *
 * @param s3 the S3Client object used to interact with the Amazon S3 service
 * @param bucketName the name of the S3 bucket that contains the object
 * @param keyName the key (name) of the S3 object
 */
}
```

```

 public static void listTags(S3Client s3, String bucketName, String keyName) {
 try {
 GetObjectTaggingRequest getTaggingRequest = GetObjectTaggingRequest
 .builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 GetObjectTaggingResponse tags = s3.getObjectTagging(getTaggingRequest);
 List<Tag> tagSet = tags.tagSet();
 for (Tag tag : tagSet) {
 System.out.println(tag.key());
 System.out.println(tag.value());
 }

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Get a URL for an object using an [S3Client](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetUrlRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.net.URL;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class GetObjectUrl {

```

```

public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName>\s

 Where:
 bucketName - The Amazon S3 bucket name.
 keyName - A key name that represents the object.\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String keyName = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 getURL(s3, bucketName, keyName);
 s3.close();
}

/**
 * Retrieves the URL for a specific object in an Amazon S3 bucket.
 *
 * @param s3 the S3Client object used to interact with the Amazon S3 service
 * @param bucketName the name of the S3 bucket where the object is stored
 * @param keyName the name of the object for which the URL should be retrieved
 * @throws S3Exception if there is an error retrieving the URL for the specified
object
 */
public static void getURL(S3Client s3, String bucketName, String keyName) {
 try {
 GetUrlRequest request = GetUrlRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();

 URL url = s3.utilities().getUrl(request);
 }
}

```

```

 System.out.println("The URL for " + keyName + " is " + url);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

Get an object by using the S3Presigner client object using an [S3Client](#).

```

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.time.Duration;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.model.GetObjectPresignRequest;
import software.amazon.awssdk.services.s3.presigner.model.PresignedGetObjectRequest;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.utils.IoUtils;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetObjectPresignedUrl {
 public static void main(String[] args) {
 final String USAGE = ""

 Usage:
 <bucketName> <keyName>\s

 Where:

```



```

 bucketName - The Amazon S3 bucket name.\s
 keyName - A key name that represents a text file.\s
 """;

 if (args.length != 2) {
 System.out.println(USAGE);
 System.exit(1);
 }

 String bucketName = args[0];
 String keyName = args[1];
 Region region = Region.US_EAST_1;
 S3Presigner presigner = S3Presigner.builder()
 .region(region)
 .build();

 getPresignedUrl(presigner, bucketName, keyName);
 presigner.close();
}

/**
 * Generates a pre-signed URL for an Amazon S3 object.
 *
 * @param presigner The {@link S3Presigner} instance to use for generating the
pre-signed URL.
 * @param bucketName The name of the Amazon S3 bucket where the object is
stored.
 * @param keyName The key name (file name) of the object in the Amazon S3
bucket.
 *
 * @throws S3Exception If there is an error interacting with the Amazon S3
service.
 * @throws IOException If there is an error opening the HTTP connection or
reading/writing the request/response.
 */
public static void getPresignedUrl(S3Presigner presigner, String bucketName,
String keyName) {
 try {
 GetObjectRequest getObjectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();
 }

```

```

 GetObjectPresignRequest getObjectPresignRequest =
GetObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(60))
 .getObjectRequest(getObjectRequest)
 .build();

 PresignedGetObjectRequest presignedGetObjectRequest =
presigner.presignGetObject(getObjectPresignRequest);
 String theUrl = presignedGetObjectRequest.url().toString();
 System.out.println("Presigned URL: " + theUrl);
 HttpURLConnection connection = (HttpURLConnection)
presignedGetObjectRequest.url().openConnection();
 presignedGetObjectRequest.httpRequest().headers().forEach((header,
values) -> {
 values.forEach(value -> {
 connection.addRequestProperty(header, value);
 });
 });

 // Send any request payload that the service needs (not needed when
 // isBrowserExecutable is true).
 if (presignedGetObjectRequest.signedPayload().isPresent()) {
 connection.setDoOutput(true);

 try (InputStream signedPayload =
presignedGetObjectRequest.signedPayload().get().asInputStream();
 OutputStream httpOutputStream = connection.getOutputStream()) {
 IoUtils.copy(signedPayload, httpOutputStream);
 }
 }

 // Download the result of executing the request.
 try (InputStream content = connection.getInputStream()) {
 System.out.println("Service returned response: ");
 IoUtils.copy(content, System.out);
 }

 } catch (S3Exception | IOException e) {
 e.printStackTrace();
 }
}
}

```

## Get an object by using a ResponseTransformer object and [S3Client](#).

```
import software.amazon.awssdk.core.ResponseBytes;
import software.amazon.awssdk.core.sync.ResponseTransformer;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;

import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.OutputStream;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class GetObjectData {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName> <path>

 Where:
 bucketName - The Amazon S3 bucket name.\s
 keyName - The key name.\s
 path - The path where the file is written to.\s
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
```

```

 String keyName = args[1];
 String path = args[2];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 getObjectBytes(s3, bucketName, keyName, path);
 s3.close();
 }

 /**
 * Retrieves the bytes of an object stored in an Amazon S3 bucket and saves them
 * to a local file.
 *
 * @param s3 The S3Client instance used to interact with the Amazon S3 service.
 * @param bucketName The name of the S3 bucket where the object is stored.
 * @param keyName The key (or name) of the S3 object.
 * @param path The local file path where the object's bytes will be saved.
 * @throws IOException If an I/O error occurs while writing the bytes to the
 * local file.
 * @throws S3Exception If an error occurs while retrieving the object from the
 * S3 bucket.
 */
 public static void getObjectBytes(S3Client s3, String bucketName, String
 keyName, String path) {
 try {
 GetObjectRequest objectRequest = GetObjectRequest
 .builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 ResponseBytes<GetObjectResponse> objectBytes =
 s3.getObject(objectRequest, ResponseTransformer.toBytes());
 byte[] data = objectBytes.asByteArray();

 // Write the data to a local file.
 File myFile = new File(path);
 OutputStream os = new FileOutputStream(myFile);
 os.write(data);
 System.out.println("Successfully obtained bytes from an S3 object");
 os.close();
 }
 }

```

```

 } catch (IOException ex) {
 ex.printStackTrace();
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [GetObject](#) in *AWS SDK for Java 2.x API Reference*.

## GetObjectLegalHold

The following code example shows how to use `GetObjectLegalHold`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

// Get the legal hold details for an S3 object.
public ObjectLockLegalHold getObjectLegalHold(String bucketName, String
objectKey) {
 try {
 GetObjectLegalHoldRequest legalHoldRequest =
GetObjectLegalHoldRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 GetObjectLegalHoldResponse response =
getClient().getObjectLegalHold(legalHoldRequest);
 System.out.println("Object legal hold for " + objectKey + " in " +
bucketName +
 ":\n\tStatus: " + response.legalHold().status());
 return response.legalHold();
 }
}

```

```
 } catch (S3Exception ex) {
 System.out.println("\tUnable to fetch legal hold: '" + ex.getMessage() +
 "'");
 }

 return null;
}
```

- For API details, see [GetObjectLegalHold](#) in *AWS SDK for Java 2.x API Reference*.

## GetObjectLockConfiguration

The following code example shows how to use `GetObjectLockConfiguration`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Get the object lock configuration details for an S3 bucket.
public void getBucketObjectLockConfiguration(String bucketName) {
 GetObjectLockConfigurationRequest objectLockConfigurationRequest =
 GetObjectLockConfigurationRequest.builder()
 .bucket(bucketName)
 .build();

 GetObjectLockConfigurationResponse response =
 getClient().getObjectLockConfiguration(objectLockConfigurationRequest);
 System.out.println("Bucket object lock config for "+bucketName+": ");
 System.out.println("\tEnabled:
 "+response.getObjectLockConfiguration().objectLockEnabled());
 System.out.println("\tRule: "+
 response.getObjectLockConfiguration().rule().defaultRetention());
}
```

- For API details, see [GetObjectLockConfiguration](#) in *AWS SDK for Java 2.x API Reference*.

## GetObjectRetention

The following code example shows how to use `GetObjectRetention`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Get the retention period for an S3 object.
public ObjectLockRetention getObjectRetention(String bucketName, String key){
 try {
 GetObjectRetentionRequest retentionRequest =
 GetObjectRetentionRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 GetObjectRetentionResponse response =
 getClient().getObjectRetention(retentionRequest);
 System.out.println("Object retention for "+key+" in "+ bucketName+":
 "+ response.retention().mode()+" until "+ response.retention().retainUntilDate()
 +".");
 return response.retention();

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return null;
 }
}
```

- For API details, see [GetObjectRetention](#) in *AWS SDK for Java 2.x API Reference*.

## HeadObject

The following code example shows how to use `HeadObject`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Determine the content type of an object.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetObjectContentType {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName>

 Where:
 bucketName - The Amazon S3 bucket name.\s
 keyName - The key name.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
```



```

 String keyName = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 getContentType(s3, bucketName, keyName);
 s3.close();
 }

 /**
 * Retrieves the content type of an object stored in an Amazon S3 bucket.
 *
 * @param s3 an instance of the {@link S3Client} class, which is used to
 * interact with the Amazon S3 service
 * @param bucketName the name of the S3 bucket where the object is stored
 * @param keyName the key (file name) of the object in the S3 bucket
 */
 public static void getContentType(S3Client s3, String bucketName, String
keyName) {
 try {
 HeadObjectRequest objectRequest = HeadObjectRequest.builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 HeadObjectResponse objectHead = s3.headObject(objectRequest);
 String type = objectHead.contentType();
 System.out.println("The object content type is " + type);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Get the restore status of an object.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;

```

```

import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

public class GetObjectRestoreStatus {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName>\s

 Where:
 bucketName - The Amazon S3 bucket name.\s
 keyName - A key name that represents the object.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String keyName = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 checkStatus(s3, bucketName, keyName);
 s3.close();
 }

 /**
 * Checks the restoration status of an Amazon S3 object.
 *
 * @param s3 an instance of the {@link S3Client} class used to interact
 * with the Amazon S3 service
 * @param bucketName the name of the Amazon S3 bucket where the object is stored
 * @param keyName the name of the Amazon S3 object to be checked
 * @throws S3Exception if an error occurs while interacting with the Amazon S3
 * service
 */
 public static void checkStatus(S3Client s3, String bucketName, String keyName) {
 try {

```

```

 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();

 HeadObjectResponse response = s3.headObject(headObjectRequest);
 System.out.println("The Amazon S3 object restoration status is " +
 response.restore());

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [HeadObject](#) in *AWS SDK for Java 2.x API Reference*.

## ListBuckets

The following code example shows how to use `ListBuckets`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.paginators.ListBucketsIterable;
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

```

```
public class ListBuckets {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 listAllBuckets(s3);
 }

 /**
 * Lists all the S3 buckets available in the current AWS account.
 *
 * @param s3 The {@link S3Client} instance to use for interacting with the
 Amazon S3 service.
 */
 public static void listAllBuckets(S3Client s3) {
 ListBucketsIterable response = s3.listBucketsPaginator();
 response.buckets().forEach(bucket ->
 System.out.println("Bucket Name: " + bucket.name()));
 }
}
```

- For API details, see [ListBuckets](#) in *AWS SDK for Java 2.x API Reference*.

## ListMultipartUploads

The following code example shows how to use ListMultipartUploads.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsRequest;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsResponse;
```

```

import software.amazon.awssdk.services.s3.model.MultipartUpload;
import software.amazon.awssdk.services.s3.model.S3Exception;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class ListMultipartUploads {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName>\s

 Where:
 bucketName - The name of the Amazon S3 bucket where an in-
progress multipart upload is occurring.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();
 listUploads(s3, bucketName);
 s3.close();
 }

 /**
 * Lists the multipart uploads currently in progress in the specified Amazon S3
 bucket.
 *
 * @param s3 the S3Client object used to interact with Amazon S3

```

```

 * @param bucketName the name of the Amazon S3 bucket to list the multipart
 uploads for
 */
 public static void listUploads(S3Client s3, String bucketName) {
 try {
 ListMultipartUploadsRequest listMultipartUploadsRequest =
 ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 ListMultipartUploadsResponse response =
 s3.listMultipartUploads(listMultipartUploadsRequest);
 List<MultipartUpload> uploads = response.uploads();
 for (MultipartUpload upload : uploads) {
 System.out.println("Upload in progress: Key = \"" + upload.key() +
 "\", id = " + upload.uploadId());
 }

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListMultipartUploads](#) in *AWS SDK for Java 2.x API Reference*.

## ListObjectsV2

The following code example shows how to use ListObjectsV2.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
```

```

 * Asynchronously lists all objects in the specified S3 bucket.
 *
 * @param bucketName the name of the S3 bucket to list objects for
 * @return a {@link CompletableFuture} that completes when all objects have been
listed
 */
 public CompletableFuture<Void> listAllObjectsAsync(String bucketName) {
 ListObjectsV2Request initialRequest = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .maxKeys(1)
 .build();

 ListObjectsV2Publisher paginator =
getAsyncClient().listObjectsV2Paginator(initialRequest);
 return paginator.subscribe(response -> {
 response.contents().forEach(s3Object -> {
 logger.info("Object key: " + s3Object.key());
 });
 }).thenRun(() -> {
 logger.info("Successfully listed all objects in the bucket: " +
bucketName);
 }).exceptionally(ex -> {
 throw new RuntimeException("Failed to list objects", ex);
 });
 }

```

## List objects using pagination.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.paginators.ListObjectsV2Iterable;

public class ListObjectsPaginated {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName>\s

```

```

 Where:
 bucketName - The Amazon S3 bucket from which objects are read.\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 listBucketObjects(s3, bucketName);
 s3.close();
}

/**
 * Lists the objects in the specified S3 bucket.
 *
 * @param s3 the S3Client instance used to interact with Amazon S3
 * @param bucketName the name of the S3 bucket to list the objects from
 */
public static void listBucketObjects(S3Client s3, String bucketName) {
 try {
 ListObjectsV2Request listReq = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .maxKeys(1)
 .build();

 ListObjectsV2Iterable listRes = s3.listObjectsV2Paginator(listReq);
 listRes.stream()
 .flatMap(r -> r.contents().stream())
 .forEach(content -> System.out.println(" Key: " + content.key() + "
size = " + content.size()));

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```



- For API details, see [ListObjectsV2](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketAcl

The following code example shows how to use PutBucketAcl.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.AccessControlPolicy;
import software.amazon.awssdk.services.s3.model.Grant;
import software.amazon.awssdk.services.s3.model.Permission;
import software.amazon.awssdk.services.s3.model.PutBucketAclRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.Type;

import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SetAcl {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
```

```

 <bucketName> <id>\s

 Where:
 bucketName - The Amazon S3 bucket to grant permissions on.\s
 id - The ID of the owner of this bucket (you can get this value from
the AWS Management Console).
 """;

 if (args.length != 2) {
 System.out.println(usage);
 return;
 }

 String bucketName = args[0];
 String id = args[1];
 System.out.format("Setting access \n");
 System.out.println(" in bucket: " + bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setBucketAcl(s3, bucketName, id);
 System.out.println("Done!");
 s3.close();
}

/**
 * Sets the Access Control List (ACL) for an Amazon S3 bucket.
 *
 * @param s3 the S3Client instance to be used for the operation
 * @param bucketName the name of the S3 bucket to set the ACL for
 * @param id the ID of the AWS user or account that will be granted full control
of the bucket
 * @throws S3Exception if an error occurs while setting the bucket ACL
 */
public static void setBucketAcl(S3Client s3, String bucketName, String id) {
 try {
 Grant ownerGrant = Grant.builder()
 .grantee(builder -> builder.id(id)
 .type(Type.CANONICAL_USER))
 .permission(Permission.FULL_CONTROL)
 .build();
 }

```

```

 List<Grant> grantList2 = new ArrayList<>();
 grantList2.add(ownerGrant);

 AccessControlPolicy acl = AccessControlPolicy.builder()
 .owner(builder -> builder.id(id))
 .grants(grantList2)
 .build();

 PutBucketAclRequest putAclReq = PutBucketAclRequest.builder()
 .bucket(bucketName)
 .accessControlPolicy(acl)
 .build();

 s3.putBucketAcl(putAclReq);

 } catch (S3Exception e) {
 e.printStackTrace();
 System.exit(1);
 }
}

```

- For API details, see [PutBucketAcl](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketCors

The following code example shows how to use PutBucketCors.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;

import java.util.ArrayList;

```

```
import java.util.List;

import software.amazon.awssdk.services.s3.model.GetBucketCorsRequest;
import software.amazon.awssdk.services.s3.model.GetBucketCorsResponse;
import software.amazon.awssdk.services.s3.model.DeleteBucketCorsRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.CORSRule;
import software.amazon.awssdk.services.s3.model.CORSConfiguration;
import software.amazon.awssdk.services.s3.model.PutBucketCorsRequest;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class S3Cors {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <accountId>\s

 Where:
 bucketName - The Amazon S3 bucket to upload an object into.
 accountId - The id of the account that owns the Amazon S3 bucket.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String accountId = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setCorsInformation(s3, bucketName, accountId);
 getBucketCorsInformation(s3, bucketName, accountId);
 }
}
```

```

 deleteBucketCorsInformation(s3, bucketName, accountId);
 s3.close();
 }

 /**
 * Deletes the CORS (Cross-Origin Resource Sharing) configuration for an Amazon
 S3 bucket.
 *
 * @param s3 the {@link S3Client} instance used to interact with the
 Amazon S3 service
 * @param bucketName the name of the Amazon S3 bucket for which the CORS
 configuration should be deleted
 * @param accountId the expected AWS account ID of the bucket owner
 *
 * @throws S3Exception if an error occurs while deleting the CORS configuration
 for the bucket
 */
 public static void deleteBucketCorsInformation(S3Client s3, String bucketName,
String accountId) {
 try {
 DeleteBucketCorsRequest bucketCorsRequest =
DeleteBucketCorsRequest.builder()
 .bucket(bucketName)
 .expectedBucketOwner(accountId)
 .build();

 s3.deleteBucketCors(bucketCorsRequest);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 /**
 * Retrieves the CORS (Cross-Origin Resource Sharing) configuration for the
 specified S3 bucket.
 *
 * @param s3 the S3Client instance to use for the operation
 * @param bucketName the name of the S3 bucket to retrieve the CORS
 configuration for
 * @param accountId the expected bucket owner's account ID
 *
 * @throws S3Exception if there is an error retrieving the CORS configuration

```

```

 */
 public static void getBucketCorsInformation(S3Client s3, String bucketName,
String accountId) {
 try {
 GetBucketCorsRequest bucketCorsRequest = GetBucketCorsRequest.builder()
 .bucket(bucketName)
 .expectedBucketOwner(accountId)
 .build();

 GetBucketCorsResponse corsResponse =
s3.getBucketCors(bucketCorsRequest);
 List<CORSRule> corsRules = corsResponse.corsRules();
 for (CORSRule rule : corsRules) {
 System.out.println("allowOrigins: " + rule.allowedOrigins());
 System.out.println("AllowedMethod: " + rule.allowedMethods());
 }

 } catch (S3Exception e) {

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 /**
 * Sets the Cross-Origin Resource Sharing (CORS) rules for an Amazon S3 bucket.
 *
 * @param s3 The S3Client object used to interact with the Amazon S3 service.
 * @param bucketName The name of the S3 bucket to set the CORS rules for.
 * @param accountId The AWS account ID of the bucket owner.
 */
 public static void setCorsInformation(S3Client s3, String bucketName, String
accountId) {
 List<String> allowMethods = new ArrayList<>();
 allowMethods.add("PUT");
 allowMethods.add("POST");
 allowMethods.add("DELETE");

 List<String> allowOrigins = new ArrayList<>();
 allowOrigins.add("http://example.com");
 try {
 // Define CORS rules.
 CORSRule corsRule = CORSRule.builder()
 .allowedMethods(allowMethods)

```

```

 .allowedOrigins(allowOrigins)
 .build();

List<CORSRule> corsRules = new ArrayList<>();
corsRules.add(corsRule);
CORSConfiguration configuration = CORSConfiguration.builder()
 .corsRules(corsRules)
 .build();

PutBucketCorsRequest putBucketCorsRequest =
PutBucketCorsRequest.builder()
 .bucket(bucketName)
 .corsConfiguration(configuration)
 .expectedBucketOwner(accountId)
 .build();

s3.putBucketCors(putBucketCorsRequest);

} catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
}

```

- For API details, see [PutBucketCors](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketLifecycleConfiguration

The following code example shows how to use `PutBucketLifecycleConfiguration`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
```

```

import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.LifecycleRuleFilter;
import software.amazon.awssdk.services.s3.model.Transition;
import
 software.amazon.awssdk.services.s3.model.GetBucketLifecycleConfigurationRequest;
import
 software.amazon.awssdk.services.s3.model.GetBucketLifecycleConfigurationResponse;
import software.amazon.awssdk.services.s3.model.DeleteBucketLifecycleRequest;
import software.amazon.awssdk.services.s3.model.TransitionStorageClass;
import software.amazon.awssdk.services.s3.model.LifecycleRule;
import software.amazon.awssdk.services.s3.model.ExpirationStatus;
import software.amazon.awssdk.services.s3.model.BucketLifecycleConfiguration;
import
 software.amazon.awssdk.services.s3.model.PutBucketLifecycleConfigurationRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class LifecycleConfiguration {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <accountId>\s

 Where:
 bucketName - The Amazon Simple Storage Service (Amazon S3) bucket to
upload an object into.
 accountId - The id of the account that owns the Amazon S3 bucket.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}

```



```

 String bucketName = args[0];
 String accountId = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setLifecycleConfig(s3, bucketName, accountId);
 getLifecycleConfig(s3, bucketName, accountId);
 deleteLifecycleConfig(s3, bucketName, accountId);
 System.out.println("You have successfully created, updated, and deleted a
Lifecycle configuration");
 s3.close();
 }

 /**
 * Sets the lifecycle configuration for an Amazon S3 bucket.
 *
 * @param s3 The Amazon S3 client to use for the operation.
 * @param bucketName The name of the Amazon S3 bucket.
 * @param accountId The expected owner of the Amazon S3 bucket.
 *
 * @throws S3Exception if there is an error setting the lifecycle configuration.
 */
 public static void setLifecycleConfig(S3Client s3, String bucketName, String
accountId) {
 try {
 // Create a rule to archive objects with the "glacierobjects/" prefix to
the
 // S3 Glacier Flexible Retrieval storage class immediately.
 LifecycleRuleFilter ruleFilter = LifecycleRuleFilter.builder()
 .prefix("glacierobjects/")
 .build();

 Transition transition = Transition.builder()
 .storageClass(TransitionStorageClass.GLACIER)
 .days(0)
 .build();

 LifecycleRule rule1 = LifecycleRule.builder()
 .id("Archive immediately rule")
 .filter(ruleFilter)
 .transitions(transition)

```

```
 .status(ExpirationStatus.ENABLED)
 .build();

// Create a second rule.
Transition transition2 = Transition.builder()
 .storageClass(TransitionStorageClass.GLACIER)
 .days(0)
 .build();

List<Transition> transitionList = new ArrayList<>();
transitionList.add(transition2);

LifecycleRuleFilter ruleFilter2 = LifecycleRuleFilter.builder()
 .prefix("glacierobjects/")
 .build();

LifecycleRule rule2 = LifecycleRule.builder()
 .id("Archive and then delete rule")
 .filter(ruleFilter2)
 .transitions(transitionList)
 .status(ExpirationStatus.ENABLED)
 .build();

// Add the LifecycleRule objects to an ArrayList.
ArrayList<LifecycleRule> ruleList = new ArrayList<>();
ruleList.add(rule1);
ruleList.add(rule2);

BucketLifecycleConfiguration lifecycleConfiguration =
BucketLifecycleConfiguration.builder()
 .rules(ruleList)
 .build();

PutBucketLifecycleConfigurationRequest
putBucketLifecycleConfigurationRequest = PutBucketLifecycleConfigurationRequest
 .builder()
 .bucket(bucketName)
 .lifecycleConfiguration(lifecycleConfiguration)
 .expectedBucketOwner(accountId)
 .build();

s3.putBucketLifecycleConfiguration(putBucketLifecycleConfigurationRequest);
```

```

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

/**
 * Retrieves the lifecycle configuration for an Amazon S3 bucket and adds a new
 * lifecycle rule to it.
 *
 * @param s3 the S3Client instance used to interact with Amazon S3
 * @param bucketName the name of the Amazon S3 bucket
 * @param accountId the expected owner of the Amazon S3 bucket
 */
public static void getLifecycleConfig(S3Client s3, String bucketName, String
accountId) {
 try {
 GetBucketLifecycleConfigurationRequest
getBucketLifecycleConfigurationRequest = GetBucketLifecycleConfigurationRequest
 .builder()
 .bucket(bucketName)
 .expectedBucketOwner(accountId)
 .build();

 GetBucketLifecycleConfigurationResponse response = s3

.getBucketLifecycleConfiguration(getBucketLifecycleConfigurationRequest);
 List<LifecycleRule> newList = new ArrayList<>();
 List<LifecycleRule> rules = response.rules();
 for (LifecycleRule rule : rules) {
 newList.add(rule);
 }

 // Add a new rule with both a prefix predicate and a tag predicate.
 LifecycleRuleFilter ruleFilter = LifecycleRuleFilter.builder()
 .prefix("YearlyDocuments/")
 .build();

 Transition transition = Transition.builder()
 .storageClass(TransitionStorageClass.GLACIER)
 .days(3650)
 .build();

 LifecycleRule rule1 = LifecycleRule.builder()

```

```

 .id("NewRule")
 .filter(ruleFilter)
 .transitions(transition)
 .status(ExpirationStatus.ENABLED)
 .build();

 // Add the new rule to the list.
 newList.add(rule1);
 BucketLifecycleConfiguration lifecycleConfiguration =
BucketLifecycleConfiguration.builder()
 .rules(newList)
 .build();

 PutBucketLifecycleConfigurationRequest
putBucketLifecycleConfigurationRequest = PutBucketLifecycleConfigurationRequest
 .builder()
 .bucket(bucketName)
 .lifecycleConfiguration(lifecycleConfiguration)
 .expectedBucketOwner(accountId)
 .build();

s3.putBucketLifecycleConfiguration(putBucketLifecycleConfigurationRequest);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

/**
 * Deletes the lifecycle configuration for an Amazon S3 bucket.
 *
 * @param s3 the {@link S3Client} to use for the operation
 * @param bucketName the name of the S3 bucket
 * @param accountId the expected account owner of the S3 bucket
 *
 * @throws S3Exception if an error occurs while deleting the lifecycle
configuration
 */
public static void deleteLifecycleConfig(S3Client s3, String bucketName, String
accountId) {
 try {

```

```

 DeleteBucketLifecycleRequest deleteBucketLifecycleRequest =
DeleteBucketLifecycleRequest
 .builder()
 .bucket(bucketName)
 .expectedBucketOwner(accountId)
 .build();

 s3.deleteBucketLifecycle(deleteBucketLifecycleRequest);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [PutBucketLifecycleConfiguration](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketPolicy

The following code example shows how to use PutBucketPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutBucketPolicyRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.regions.Region;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;

```

```

import com.fasterxml.jackson.core.JsonParser;
import com.fasterxml.jackson.databind.ObjectMapper;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SetBucketPolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <polFile>

 Where:
 bucketName - The Amazon S3 bucket to set the policy on.
 polFile - A JSON file containing the policy (see the Amazon S3
Readme for an example).\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String polFile = args[1];
 String policyText = getBucketPolicyFromFile(polFile);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setPolicy(s3, bucketName, policyText);
 s3.close();
 }

 /**
 * Sets the policy for an Amazon S3 bucket.

```

```

*
* @param s3 the {@link S3Client} object used to interact with the
Amazon S3 service
* @param bucketName the name of the Amazon S3 bucket
* @param policyText the text of the policy to be set on the bucket
* @throws S3Exception if there is an error setting the bucket policy
*/
public static void setPolicy(S3Client s3, String bucketName, String policyText)
{
 System.out.println("Setting policy:");
 System.out.println("----");
 System.out.println(policyText);
 System.out.println("----");
 System.out.format("On Amazon S3 bucket: \"%s\"\n", bucketName);

 try {
 PutBucketPolicyRequest policyReq = PutBucketPolicyRequest.builder()
 .bucket(bucketName)
 .policy(policyText)
 .build();

 s3.putBucketPolicy(policyReq);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 System.out.println("Done!");
}

/**
* Retrieves the bucket policy from a specified file.
*
* @param policyFile the path to the file containing the bucket policy
* @return the content of the bucket policy file as a string
*/
public static String getBucketPolicyFromFile(String policyFile) {
 StringBuilder fileText = new StringBuilder();
 try {
 List<String> lines = Files.readAllLines(Paths.get(policyFile),
StandardCharsets.UTF_8);
 for (String line : lines) {
 fileText.append(line);

```

```

 }

 } catch (IOException e) {
 System.out.format("Problem reading file: \"%s\"", policyFile);
 System.out.println(e.getMessage());
 }

 try {
 final JsonParser parser = new
ObjectMapper().getFactory().createParser(fileText.toString());
 while (parser.nextToken() != null) {
 }

 } catch (IOException jpe) {
 jpe.printStackTrace();
 }
 return fileText.toString();
 }
}

```

- For API details, see [PutBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketReplication

The following code example shows how to use PutBucketReplication.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Sets the replication configuration for an Amazon S3 bucket.
 *
 * @param s3Client the S3Client instance to use for the operation
 * @param sourceBucketName the name of the source bucket
 * @param destBucketName the name of the destination bucket

```



```

 * @param destinationBucketARN the Amazon Resource Name (ARN) of the destination
 bucket
 * @param roleARN the ARN of the IAM role to use for the
 replication configuration
 */
 public static void setReplication(S3Client s3Client, String sourceBucketName,
 String destBucketName, String destinationBucketARN, String roleARN) {
 try {
 Destination destination = Destination.builder()
 .bucket(destinationBucketARN)
 .storageClass(StorageClass.STANDARD)
 .build();

 // Define a prefix filter for replication.
 ReplicationRuleFilter ruleFilter = ReplicationRuleFilter.builder()
 .prefix("documents/")
 .build();

 // Define delete marker replication setting.
 DeleteMarkerReplication deleteMarkerReplication =
 DeleteMarkerReplication.builder()
 .status(DeleteMarkerReplicationStatus.DISABLED)
 .build();

 // Create the replication rule.
 ReplicationRule replicationRule = ReplicationRule.builder()
 .priority(1)
 .filter(ruleFilter)
 .status(ReplicationRuleStatus.ENABLED)
 .deleteMarkerReplication(deleteMarkerReplication)
 .destination(destination)
 .build();

 List<ReplicationRule> replicationRuleList = new ArrayList<>();
 replicationRuleList.add(replicationRule);

 // Define the replication configuration with IAM role.
 ReplicationConfiguration configuration =
 ReplicationConfiguration.builder()
 .role(roleARN)
 .rules(replicationRuleList)
 .build();

 // Apply the replication configuration to the source bucket.

```

```
PutBucketReplicationRequest replicationRequest =
PutBucketReplicationRequest.builder()
 .bucket(sourceBucketName)
 .replicationConfiguration(configuration)
 .build();

s3Client.putBucketReplication(replicationRequest);
System.out.println("Replication configuration set successfully.");

} catch (IllegalArgumentException e) {
 System.err.println("Configuration error: " + e.getMessage());
} catch (S3Exception e) {
 System.err.println("S3 Exception: " +
e.awsErrorDetails().errorMessage());
 System.err.println("Status Code: " + e.statusCode());
 System.err.println("Error Code: " + e.awsErrorDetails().errorCode());

} catch (SdkException e) {
 System.err.println("SDK Exception: " + e.getMessage());
}
}
```

- For API details, see [PutBucketReplication](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketVersioning

The following code example shows how to use PutBucketVersioning.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Enables bucket versioning for the specified S3 bucket.
 *
 * @param s3Client the S3 client to use for the operation
```

```

 * @param bucketName the name of the S3 bucket to enable versioning for
 */
 public static void enableBucketVersioning(S3Client s3Client, String bucketName){
 VersioningConfiguration versioningConfiguration =
VersioningConfiguration.builder()
 .status(BucketVersioningStatus.ENABLED)
 .build();

 PutBucketVersioningRequest versioningRequest =
PutBucketVersioningRequest.builder()
 .bucket(bucketName)
 .versioningConfiguration(versioningConfiguration)
 .build();

 s3Client.putBucketVersioning(versioningRequest);
 System.out.println("Bucket versioning has been enabled for "+bucketName);
 }

```

- For API details, see [PutBucketVersioning](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketWebsite

The following code example shows how to use PutBucketWebsite.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.IndexDocument;
import software.amazon.awssdk.services.s3.model.PutBucketWebsiteRequest;
import software.amazon.awssdk.services.s3.model.WebsiteConfiguration;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.regions.Region;

/**

```

```

* Before running this Java V2 code example, set up your development
* environment, including your credentials.
* <p>
* For more information, see the following documentation topic:
* <p>
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/

```

```

public class SetWebsiteConfiguration {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <bucketName> [indexdoc]\s

 Where:
 bucketName - The Amazon S3 bucket to set the website configuration
on.\s
 indexdoc - The index document, ex. 'index.html'
 If not specified, 'index.html' will be set.

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String indexDoc = "index.html";
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setWebsiteConfig(s3, bucketName, indexDoc);
 s3.close();
 }

 /**
 * Sets the website configuration for an Amazon S3 bucket.
 *
 * @param s3 The {@link S3Client} instance to use for the AWS SDK operations.
 * @param bucketName The name of the S3 bucket to configure.
 * @param indexDoc The name of the index document to use for the website
configuration.

```

```
 */
 public static void setWebsiteConfig(S3Client s3, String bucketName, String
indexDoc) {
 try {
 WebsiteConfiguration websiteConfig = WebsiteConfiguration.builder()
 .indexDocument(IndexDocument.builder().suffix(indexDoc).build())
 .build();

 PutBucketWebsiteRequest pubWebsiteReq =
PutBucketWebsiteRequest.builder()
 .bucket(bucketName)
 .websiteConfiguration(websiteConfig)
 .build();

 s3.putBucketWebsite(pubWebsiteReq);
 System.out.println("The call was successful");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [PutBucketWebsite](#) in *AWS SDK for Java 2.x API Reference*.

## PutObject

The following code example shows how to use `PutObject`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Upload a file to a bucket using an [S3Client](#).

```

/**
 * Uploads a local file to an AWS S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to upload the file to
 * @param key the key (object name) to use for the uploaded file
 * @param objectPath the local file path of the file to be uploaded
 * @return a {@link CompletableFuture} that completes with the {@link
 * PutObjectResponse} when the upload is successful, or throws a {@link
 * RuntimeException} if the upload fails
 */
public CompletableFuture<PutObjectResponse> uploadLocalFileAsync(String
bucketName, String key, String objectPath) {
 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 CompletableFuture<PutObjectResponse> response =
getAsyncClient().putObject(objectRequest,
AsyncRequestBody.fromFile(Paths.get(objectPath)));
 return response.whenComplete((resp, ex) -> {
 if (ex != null) {
 throw new RuntimeException("Failed to upload file", ex);
 }
 });
}

```

Use an [S3TransferManager](#) to [upload a file](#) to a bucket. View the [complete file](#) and [test](#).

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedFileUpload;
import software.amazon.awssdk.transfer.s3.model.FileUpload;
import software.amazon.awssdk.transfer.s3.model.UploadFileRequest;
import software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener;
import java.net.URI;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Paths;
import java.util.UUID;

```

```

public String uploadFile(S3TransferManager transferManager, String bucketName,
 String key, URI filePathURI) {
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b.bucket(bucketName).key(key))
 .source(Paths.get(filePathURI))
 .build();

 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);

 CompletedFileUpload uploadResult = fileUpload.completionFuture().join();
 return uploadResult.response().eTag();
}

```

Upload an object to a bucket and set tags using an [S3Client](#).

```

/**
 * Puts tags on an Amazon S3 object.
 *
 * @param s3 An {@link S3Client} object that represents the Amazon S3 client.
 * @param bucketName The name of the Amazon S3 bucket.
 * @param objectKey The key of the Amazon S3 object.
 * @param objectPath The file path of the object to be uploaded.
 */
public static void putS3ObjectTags(S3Client s3, String bucketName, String
objectKey, String objectPath) {
 try {
 Tag tag1 = Tag.builder()
 .key("Tag 1")
 .value("This is tag 1")
 .build();

 Tag tag2 = Tag.builder()
 .key("Tag 2")
 .value("This is tag 2")
 .build();

 List<Tag> tags = new ArrayList<>();
 tags.add(tag1);
 tags.add(tag2);

 Tagging allTags = Tagging.builder()
 .tagSet(tags)

```

```

 .build();

 PutObjectRequest putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .tagging(allTags)
 .build();

 s3.putObject(putOb, RequestBody.fromBytes(getObjectFile(objectPath)));

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

/**
 * Updates the tags associated with an object in an Amazon S3 bucket.
 *
 * @param s3 an instance of the S3Client class, which is used to interact with
the Amazon S3 service
 * @param bucketName the name of the S3 bucket containing the object
 * @param objectKey the key (or name) of the object in the S3 bucket
 * @throws S3Exception if there is an error updating the object's tags
 */
public static void updateObjectTags(S3Client s3, String bucketName, String
objectKey) {
 try {
 GetObjectTaggingRequest taggingRequest =
GetObjectTaggingRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 GetObjectTaggingResponse getTaggingRes =
s3.getObjectTagging(taggingRequest);
 List<Tag> obTags = getTaggingRes.tagSet();
 for (Tag sinTag : obTags) {
 System.out.println("The tag key is: " + sinTag.key());
 System.out.println("The tag value is: " + sinTag.value());
 }

 // Replace the object's tags with two new tags.
 Tag tag3 = Tag.builder()

```



```

 .key("Tag 3")
 .value("This is tag 3")
 .build();

 Tag tag4 = Tag.builder()
 .key("Tag 4")
 .value("This is tag 4")
 .build();

 List<Tag> tags = new ArrayList<>();
 tags.add(tag3);
 tags.add(tag4);

 Tagging updatedTags = Tagging.builder()
 .tagSet(tags)
 .build();

 PutObjectTaggingRequest taggingRequest1 =
 PutObjectTaggingRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .tagging(updatedTags)
 .build();

 s3.putObjectTagging(taggingRequest1);
 GetObjectTaggingResponse getTaggingRes2 =
 s3.getObjectTagging(taggingRequest);
 List<Tag> modTags = getTaggingRes2.tagSet();
 for (Tag sinTag : modTags) {
 System.out.println("The tag key is: " + sinTag.key());
 System.out.println("The tag value is: " + sinTag.value());
 }

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

/**
 * Retrieves the contents of a file as a byte array.
 *
 * @param filePath the path of the file to be read

```

```
 * @return a byte array containing the contents of the file, or null if an error
 occurs
 */
 private static byte[] getObjectFile(String filePath) {
 FileInputStream fileInputStream = null;
 byte[] byteArray = null;

 try {
 File file = new File(filePath);
 byteArray = new byte[(int) file.length()];
 fileInputStream = new FileInputStream(file);
 fileInputStream.read(byteArray);

 } catch (IOException e) {
 e.printStackTrace();
 } finally {
 if (fileInputStream != null) {
 try {
 fileInputStream.close();
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
 }

 return byteArray;
 }
}
```

Upload an object to a bucket and set metadata using an [S3Client](#).

```
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.io.File;
import java.util.HashMap;
import java.util.Map;
```

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PutObjectMetadata {
 public static void main(String[] args) {
 final String USAGE = ""

 Usage:
 <bucketName> <objectKey> <objectPath>\s

 Where:
 bucketName - The Amazon S3 bucket to upload an object into.
 objectKey - The object to upload (for example, book.pdf).
 objectPath - The path where the file is located (for example, C:/AWS/
book2.pdf).\s
 """;

 if (args.length != 3) {
 System.out.println(USAGE);
 System.exit(1);
 }

 String bucketName = args[0];
 String objectKey = args[1];
 String objectPath = args[2];
 System.out.println("Putting object " + objectKey + " into bucket " +
bucketName);
 System.out.println(" in bucket: " + bucketName);
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 putS3Object(s3, bucketName, objectKey, objectPath);
 s3.close();
 }

 /**
 * Uploads an object to an Amazon S3 bucket with metadata.
```

```

 *
 * @param s3 the S3Client object used to interact with the Amazon S3 service
 * @param bucketName the name of the S3 bucket to upload the object to
 * @param objectKey the name of the object to be uploaded
 * @param objectPath the local file path of the object to be uploaded
 */
 public static void putS3Object(S3Client s3, String bucketName, String objectKey,
 String objectPath) {
 try {
 Map<String, String> metadata = new HashMap<>();
 metadata.put("author", "Mary Doe");
 metadata.put("version", "1.0.0.0");

 PutObjectRequest putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .metadata(metadata)
 .build();

 s3.putObject(putOb, RequestBody.fromFile(new File(objectPath)));
 System.out.println("Successfully placed " + objectKey + " into bucket "
+ bucketName);

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

Upload an object to a bucket and set an object retention value using an [S3Client](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRetentionRequest;
import software.amazon.awssdk.services.s3.model.ObjectLockRetention;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;

```

```

import java.time.ZoneOffset;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class PutObjectRetention {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <key> <bucketName>\s

 Where:
 key - The name of the object (for example, book.pdf).\s
 bucketName - The Amazon S3 bucket name that contains the object (for
example, bucket1).\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String key = args[0];
 String bucketName = args[1];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 setRentionPeriod(s3, key, bucketName);
 s3.close();
 }

 /**
 * Sets the retention period for an object in an Amazon S3 bucket.
 *
 * @param s3 the S3Client object used to interact with the Amazon S3 service

```

```

 * @param key the key (name) of the object in the S3 bucket
 * @param bucket the name of the S3 bucket where the object is stored
 *
 * @throws S3Exception if an error occurs while setting the object retention
period
 */
 public static void setRetentionPeriod(S3Client s3, String key, String bucket) {
 try {
 LocalDate localDate = LocalDate.parse("2020-07-17");
 LocalDateTime localDateTime = localDate.atStartOfDay();
 Instant instant = localDateTime.toInstant(ZoneOffset.UTC);

 ObjectLockRetention lockRetention = ObjectLockRetention.builder()
 .mode("COMPLIANCE")
 .retainUntilDate(instant)
 .build();

 PutObjectRetentionRequest retentionRequest =
PutObjectRetentionRequest.builder()
 .bucket(bucket)
 .key(key)
 .bypassGovernanceRetention(true)
 .retention(lockRetention)
 .build();

 // To set Retention on an object, the Amazon S3 bucket must support
object
 // locking, otherwise an exception is thrown.
 s3.putObjectRetention(retentionRequest);
 System.out.print("An object retention configuration was successfully
placed on the object");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
 }
}

```

- For API details, see [PutObject](#) in *AWS SDK for Java 2.x API Reference*.

## PutObjectLegalHold

The following code example shows how to use PutObjectLegalHold.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Set or modify a legal hold on an object in an S3 bucket.
public void modifyObjectLegalHold(String bucketName, String objectKey, boolean
legalHoldOn) {
 ObjectLockLegalHold legalHold ;
 if (legalHoldOn) {
 legalHold = ObjectLockLegalHold.builder()
 .status(ObjectLockLegalHoldStatus.ON)
 .build();
 } else {
 legalHold = ObjectLockLegalHold.builder()
 .status(ObjectLockLegalHoldStatus.OFF)
 .build();
 }

 PutObjectLegalHoldRequest legalHoldRequest =
PutObjectLegalHoldRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .legalHold(legalHold)
 .build();

 getClient().putObjectLegalHold(legalHoldRequest) ;
 System.out.println("Modified legal hold for "+ objectKey +" in "+bucketName
+".");
}
```

- For API details, see [PutObjectLegalHold](#) in *AWS SDK for Java 2.x API Reference*.

## PutObjectLockConfiguration

The following code example shows how to use PutObjectLockConfiguration.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Set the object lock configuration of a bucket.

```
// Enable object lock on an existing bucket.
public void enableObjectLockOnBucket(String bucketName) {
 try {
 VersioningConfiguration versioningConfiguration =
VersioningConfiguration.builder()
 .status(BucketVersioningStatus.ENABLED)
 .build();

 PutBucketVersioningRequest putBucketVersioningRequest =
PutBucketVersioningRequest.builder()
 .bucket(bucketName)
 .versioningConfiguration(versioningConfiguration)
 .build();

 // Enable versioning on the bucket.
 getClient().putBucketVersioning(putBucketVersioningRequest);
 PutObjectLockConfigurationRequest request =
PutObjectLockConfigurationRequest.builder()
 .bucket(bucketName)
 .objectLockConfiguration(ObjectLockConfiguration.builder()
 .objectLockEnabled(ObjectLockEnabled.ENABLED)
 .build())
 .build();

 getClient().putObjectLockConfiguration(request);
 System.out.println("Successfully enabled object lock on "+bucketName);

 } catch (S3Exception ex) {
```



```

 System.out.println("Error modifying object lock: '" + ex.getMessage() +
 """);
 }
}

```

Set the default retention period of a bucket.

```

// Set or modify a retention period on an S3 bucket.
public void modifyBucketDefaultRetention(String bucketName) {
 VersioningConfiguration versioningConfiguration =
VersioningConfiguration.builder()
 .mfaDelete(MFADelete.DISABLED)
 .status(BucketVersioningStatus.ENABLED)
 .build();

 PutBucketVersioningRequest versioningRequest =
PutBucketVersioningRequest.builder()
 .bucket(bucketName)
 .versioningConfiguration(versioningConfiguration)
 .build();

 getClient().putBucketVersioning(versioningRequest);
 DefaultRetention rention = DefaultRetention.builder()
 .days(1)
 .mode(ObjectLockRetentionMode.GOVERNANCE)
 .build();

 ObjectLockRule lockRule = ObjectLockRule.builder()
 .defaultRetention(rention)
 .build();

 ObjectLockConfiguration objectLockConfiguration =
ObjectLockConfiguration.builder()
 .objectLockEnabled(ObjectLockEnabled.ENABLED)
 .rule(lockRule)
 .build();

 PutObjectLockConfigurationRequest putObjectLockConfigurationRequest =
PutObjectLockConfigurationRequest.builder()
 .bucket(bucketName)
 .objectLockConfiguration(objectLockConfiguration)
 .build();
}

```

```
getClient().putObjectLockConfiguration(putObjectLockConfigurationRequest) ;
System.out.println("Added a default retention to bucket "+bucketName +".");
}
```

- For API details, see [PutObjectLockConfiguration](#) in *AWS SDK for Java 2.x API Reference*.

## PutObjectRetention

The following code example shows how to use PutObjectRetention.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Set or modify a retention period on an object in an S3 bucket.
public void modifyObjectRetentionPeriod(String bucketName, String objectKey) {
 // Calculate the instant one day from now.
 Instant futureInstant = Instant.now().plus(1, ChronoUnit.DAYS);

 // Convert the Instant to a ZonedDateTime object with a specific time zone.
 ZonedDateTime zonedDateTime = futureInstant.atZone(ZoneId.systemDefault());

 // Define a formatter for human-readable output.
 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");

 // Format the ZonedDateTime object to a human-readable date string.
 String humanReadableDate = formatter.format(zonedDateTime);

 // Print the formatted date string.
 System.out.println("Formatted Date: " + humanReadableDate);
 ObjectLockRetention retention = ObjectLockRetention.builder()
 .mode(ObjectLockRetentionMode.GOVERNANCE)
 .retainUntilDate(futureInstant)
 .build();
```

```

 PutObjectRetentionRequest retentionRequest =
PutObjectRetentionRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .retention(retention)
 .build();

 getClient().putObjectRetention(retentionRequest);
 System.out.println("Set retention for "+objectKey+" in "+bucketName+"
until "+humanReadableDate+".");
 }

```

- For API details, see [PutObjectRetention](#) in *AWS SDK for Java 2.x API Reference*.

## RestoreObject

The following code example shows how to use `RestoreObject`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.RestoreRequest;
import software.amazon.awssdk.services.s3.model.GlacierJobParameters;
import software.amazon.awssdk.services.s3.model.RestoreObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.Tier;

/*
 * For more information about restoring an object, see "Restoring an archived
 * object" at
 * https://docs.aws.amazon.com/AmazonS3/latest/userguide/restoring-objects.html
 */

```

```

* Before running this Java V2 code example, set up your development environment,
including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class RestoreObject {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <keyName> <expectedBucketOwner>

 Where:
 bucketName - The Amazon S3 bucket name.\s
 keyName - The key name of an object with a Storage class value of
Glacier.\s
 expectedBucketOwner - The account that owns the bucket (you can
obtain this value from the AWS Management Console).\s
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String keyName = args[1];
 String expectedBucketOwner = args[2];
 Region region = Region.US_EAST_1;
 S3Client s3 = S3Client.builder()
 .region(region)
 .build();

 restoreS3Object(s3, bucketName, keyName, expectedBucketOwner);
 s3.close();
 }

 /**
 * Restores an S3 object from the Glacier storage class.
 *
 * @param s3 an instance of the {@link S3Client} to be used
 for interacting with Amazon S3

```

```

 * @param bucketName the name of the S3 bucket where the object is
stored
 * @param keyName the key (object name) of the S3 object to be
restored
 * @param expectedBucketOwner the AWS account ID of the expected bucket owner
 */
 public static void restoreS3Object(S3Client s3, String bucketName, String
keyName, String expectedBucketOwner) {
 try {
 RestoreRequest restoreRequest = RestoreRequest.builder()
 .days(10)

.glacierJobParameters(GlacierJobParameters.builder().tier(Tier.STANDARD).build())
 .build();

 RestoreObjectRequest objectRequest = RestoreObjectRequest.builder()
 .expectedBucketOwner(expectedBucketOwner)
 .bucket(bucketName)
 .key(keyName)
 .restoreRequest(restoreRequest)
 .build();

 s3.restoreObject(objectRequest);

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [RestoreObject](#) in *AWS SDK for Java 2.x API Reference*.

## SelectObjectContent

The following code example shows how to use `SelectObjectContent`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following example shows a query using a JSON object. The [complete example](#) also shows the use of a CSV object.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.core.async.BlockingInputStreamAsyncRequestBody;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.model.CSVInput;
import software.amazon.awssdk.services.s3.model.CSVOutput;
import software.amazon.awssdk.services.s3.model.CompressionType;
import software.amazon.awssdk.services.s3.model.ExpressionType;
import software.amazon.awssdk.services.s3.model.FileHeaderInfo;
import software.amazon.awssdk.services.s3.model.InputSerialization;
import software.amazon.awssdk.services.s3.model.JSONInput;
import software.amazon.awssdk.services.s3.model.JSONOutput;
import software.amazon.awssdk.services.s3.model.JSONType;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.services.s3.model.OutputSerialization;
import software.amazon.awssdk.services.s3.model.Progress;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
import software.amazon.awssdk.services.s3.model.SelectObjectContentRequest;
import software.amazon.awssdk.services.s3.model.SelectObjectContentResponseHandler;
import software.amazon.awssdk.services.s3.model.Stats;

import java.io.IOException;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
```

```

public class SelectObjectContentExample {
 static final Logger logger =
 LoggerFactory.getLogger(SelectObjectContentExample.class);
 static String BUCKET_NAME = "amzn-s3-demo-bucket"; // Replace with your bucket
 name.
 static final S3AsyncClient s3AsyncClient = S3AsyncClient.create();
 static String FILE_CSV = "csv";
 static String FILE_JSON = "json";
 static String URL_CSV = "https://raw.githubusercontent.com/mledoze/countries/
 master/dist/countries.csv";
 static String URL_JSON = "https://raw.githubusercontent.com/mledoze/countries/
 master/dist/countries.json";

 public static void main(String[] args) {
 SelectObjectContentExample selectObjectContentExample = new
 SelectObjectContentExample();
 try {
 SelectObjectContentExample.setUp();
 selectObjectContentExample.runSelectObjectContentMethodForJSON();
 selectObjectContentExample.runSelectObjectContentMethodForCSV();
 } catch (SdkException e) {
 logger.error(e.getMessage(), e);
 System.exit(1);
 } finally {
 SelectObjectContentExample.tearDown();
 }
 }

 EventStreamInfo runSelectObjectContentMethodForJSON() {
 // Set up request parameters.
 final String queryExpression = "select * from s3object[*][*] c where c.area
 < 350000";
 final String fileType = FILE_JSON;

 InputSerialization inputSerialization = InputSerialization.builder()
 .json(JSONInput.builder().type(JSONType.DOCUMENT).build())
 .compressionType(CompressionType.NONE)
 .build();

 OutputSerialization outputSerialization = OutputSerialization.builder()
 .json(JSONOutput.builder().recordDelimiter(null).build())
 .build();

 // Build the SelectObjectContentRequest.

```

```

 SelectObjectContentRequest select = SelectObjectContentRequest.builder()
 .bucket(BUCKET_NAME)
 .key(FILE_JSON)
 .expression(queryExpression)
 .expressionType(ExpressionType.SQL)
 .inputSerialization(inputSerialization)
 .outputSerialization(outputSerialization)
 .build();

 EventStreamInfo eventStreamInfo = new EventStreamInfo();
 // Call the selectObjectContent method with the request and a response
 handler.
 // Supply an EventStreamInfo object to the response handler to gather
 records and information from the response.
 s3AsyncClient.selectObjectContent(select,
 buildResponseHandler(eventStreamInfo)).join();

 // Log out information gathered while processing the response stream.
 long recordCount = eventStreamInfo.getRecords().stream().mapToInt(record ->
 record.split("\n").length
).sum();
 logger.info("Total records {}: {}", fileType, recordCount);
 logger.info("Visitor onRecords for fileType {} called {} times", fileType,
 eventStreamInfo.getCountOnRecordsCalled());
 logger.info("Visitor onStats for fileType {}, {}", fileType,
 eventStreamInfo.getStats());
 logger.info("Visitor onContinuations for fileType {}, {}", fileType,
 eventStreamInfo.getCountContinuationEvents());
 return eventStreamInfo;
 }

 static SelectObjectContentResponseHandler buildResponseHandler(EventStreamInfo
 eventStreamInfo) {
 // Use a Visitor to process the response stream. This visitor logs
 information and gathers details while processing.
 final SelectObjectContentResponseHandler.Visitor visitor =
 SelectObjectContentResponseHandler.Visitor.builder()
 .onRecords(r -> {
 logger.info("Record event received.");
 eventStreamInfo.addRecord(r.payload().asUtf8String());
 eventStreamInfo.incrementOnRecordsCalled();
 })
 .onCont(ce -> {
 logger.info("Continuation event received.");
 })
 }

```



```

 eventStreamInfo.incrementContinuationEvents();
 })
 .onProgress(pe -> {
 Progress progress = pe.details();
 logger.info("Progress event received:\n bytesScanned:
{} \n bytesProcessed: {} \n bytesReturned: {}",
 progress.bytesScanned(),
 progress.bytesProcessed(),
 progress.bytesReturned());
 })
 .onEnd(ee -> logger.info("End event received."))
 .onStats(se -> {
 logger.info("Stats event received.");
 eventStreamInfo.addStats(se.details());
 })
 .build();

 // Build the SelectObjectContentResponseHandler with the visitor that
 // processes the stream.
 return SelectObjectContentResponseHandler.builder()
 .subscriber(visitor).build();
}

// The EventStreamInfo class is used to store information gathered while
// processing the response stream.
static class EventStreamInfo {
 private final List<String> records = new ArrayList<>();
 private Integer countOnRecordsCalled = 0;
 private Integer countContinuationEvents = 0;
 private Stats stats;

 void incrementOnRecordsCalled() {
 countOnRecordsCalled++;
 }

 void incrementContinuationEvents() {
 countContinuationEvents++;
 }

 void addRecord(String record) {
 records.add(record);
 }

 void addStats(Stats stats) {

```

```
 this.stats = stats;
 }

 public List<String> getRecords() {
 return records;
 }

 public Integer getCountOnRecordsCalled() {
 return countOnRecordsCalled;
 }

 public Integer getCountContinuationEvents() {
 return countContinuationEvents;
 }

 public Stats getStats() {
 return stats;
 }
}
```

- For API details, see [SelectObjectContent](#) in *AWS SDK for Java 2.x API Reference*.

## UploadPartCopy

The following code example shows how to use UploadPartCopy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public CompletableFuture<String> performMultiCopy(String toBucket, String
bucketName, String key) {
 CreateMultipartUploadRequest createMultipartUploadRequest =
CreateMultipartUploadRequest.builder()
 .bucket(toBucket)
 .key(key)
 .build();
```

```

 getAsyncClient().createMultipartUpload(createMultipartUploadRequest)
 .thenApply(createMultipartUploadResponse -> {
 String uploadId = createMultipartUploadResponse.uploadId();
 System.out.println("Upload ID: " + uploadId);

 UploadPartCopyRequest uploadPartCopyRequest =
UploadPartCopyRequest.builder()
 .sourceBucket(bucketName)
 .destinationBucket(toBucket)
 .sourceKey(key)
 .destinationKey(key)
 .uploadId(uploadId) // Use the valid uploadId.
 .partNumber(1) // Ensure the part number is correct.
 .copySourceRange("bytes=0-1023") // Adjust range as needed
 .build();

 return getAsyncClient().uploadPartCopy(uploadPartCopyRequest);
 })
 .thenCompose(uploadPartCopyFuture -> uploadPartCopyFuture)
 .whenComplete((uploadPartCopyResponse, exception) -> {
 if (exception != null) {
 // Handle any exceptions.
 logger.error("Error during upload part copy: " +
exception.getMessage());
 } else {
 // Successfully completed the upload part copy.
 System.out.println("Upload Part Copy completed successfully.
ETag: " + uploadPartCopyResponse.copyPartResult().eTag());
 }
 });
 return null;
 }

```

- For API details, see [UploadPartCopy](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Check if a bucket exists

The following code example shows how to check if a bucket exists.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

You can use the following `doesBucketExists` method as a replacement for the the SDK for Java V1 [AmazonS3Client#doesBucketExistV2\(String\)](#) method.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.awscore.exception.AwsServiceException;
import software.amazon.awssdk.http.HttpStatusCode;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.utils.Validate;

public class DoesBucketExist {
 private static final Logger logger =
 LoggerFactory.getLogger(DoesBucketExist.class);

 public static void main(String[] args) {
 DoesBucketExist doesBucketExist = new DoesBucketExist();

 final S3Client s3SyncClient = S3Client.builder().build();
 final String bucketName = "amzn-s3-demo-bucket"; // Replace with the bucket
 name that you want to check.

 boolean exists = doesBucketExist.doesBucketExist(bucketName, s3SyncClient);
 logger.info("Bucket exists: {}", exists);
 }

 /**
 * Checks if the specified bucket exists. Amazon S3 buckets are named in a
 * global namespace; use this method to
 * determine if a specified bucket name already exists, and therefore can't be
 * used to create a new bucket.
 * <p>
 * Internally this method uses the <a
```

```

 * href="https://sdk.amazonaws.com/java/api/latest/software/amazon/awssdk/
services/s3/
S3Client.html#getBucketAcl(java.util.function.Consumer)">S3Client.getBucketAcl(String)</
a>
 * operation to determine whether the bucket exists.
 * <p>
 * This method is equivalent to the AWS SDK for Java V1's <a
 * href="https://docs.aws.amazon.com/AWSJavaSDK/latest/javadoc/
com/amazonaws/services/s3/AmazonS3Client.html#doesBucketExistV2-
java.lang.String-">AmazonS3Client#doesBucketExistV2(String).
 *
 * @param bucketName The name of the bucket to check.
 * @param s3SyncClient An <code>S3Client</code> instance. The method checks for
the bucket in the AWS Region
 *
 * configured on the instance.
 * @return The value true if the specified bucket exists in Amazon S3; the value
false if there is no bucket in
 *
 * Amazon S3 with that name.
 */
 public boolean doesBucketExist(String bucketName, S3Client s3SyncClient) {
 try {
 Validate.notEmpty(bucketName, "The bucket name must not be null or an
empty string.", "");
 s3SyncClient.getBucketAcl(r -> r.bucket(bucketName));
 return true;
 } catch (AwsServiceException ase) {
 // A redirect error or an AccessDenied exception means the bucket exists
but it's not in this region
 // or we don't have permissions to it.
 if ((ase.statusCode() == HttpStatus.MOVED_PERMANENTLY) ||
"AccessDenied".equals(ase.awsErrorDetails().errorCode())) {
 return true;
 }
 if (ase.statusCode() == HttpStatus.NOT_FOUND) {
 return false;
 }
 throw ase;
 }
 }
}

```

- For API details, see [GetBucketAcl](#) in *AWS SDK for Java 2.x API Reference*.

## Create a presigned URL

The following code example shows how to create a presigned URL for Amazon S3 and upload an object.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following shows three example of how to create presigned URLs and use the URLs with HTTP client libraries:

- An HTTP GET request that uses the URL with three HTTP client libraries
- An HTTP PUT request with metadata in headers that uses the URL with three HTTP client libraries
- An HTTP PUT request with query parameters that uses the URL with one HTTP client library

Generate a pre-signed URL for an object, then download it (GET request).

Imports.

```
import com.example.s3.util.PresignUrlUtils;
import org.slf4j.Logger;
import software.amazon.awssdk.http.HttpExecuteRequest;
import software.amazon.awssdk.http.HttpExecuteResponse;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.SdkHttpMethod;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.GetObjectPresignRequest;
import software.amazon.awssdk.services.s3.presigner.model.PresignedGetObjectRequest;
import software.amazon.awssdk.utils.IoUtils;
```

```
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.URISyntaxException;
import java.net.URL;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.file.Paths;
import java.time.Duration;
import java.util.UUID;
```

## Generate the URL.

```
/* Create a pre-signed URL to download an object in a subsequent GET request. */
public String createPresignedGetUrl(String bucketName, String keyName) {
 try (S3Presigner presigner = S3Presigner.create()) {

 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build();

 GetObjectPresignRequest presignRequest =
 GetObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // The URL will
 expire in 10 minutes.
 .getObjectRequest(objectRequest)
 .build();

 PresignedGetObjectRequest presignedRequest =
 presigner.presignGetObject(presignRequest);
 logger.info("Presigned URL: [{}]", presignedRequest.url().toString());
 logger.info("HTTP method: [{}]",
 presignedRequest.httpRequest().method());

 return presignedRequest.url().toExternalForm();
 }
}
```

Download the object by using any one of the following three approaches.

Use JDK `URLConnection` (since v1.1) class to do the download.

```
/* Use the JDK HttpURLConnection (since v1.1) class to do the download. */
public byte[] useHttpURLConnectionToGet(String presignedUrlString) {
 ByteArrayOutputStream byteArrayOutputStream = new
 ByteArrayOutputStream(); // Capture the response body to a byte array.

 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpURLConnection connection = (HttpURLConnection)
presignedUrl.openConnection();
 connection.setRequestMethod("GET");
 // Download the result of executing the request.
 try (InputStream content = connection.getInputStream()) {
 IoUtils.copy(content, byteArrayOutputStream);
 }
 logger.info("HTTP response code is " + connection.getResponseCode());

 } catch (S3Exception | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
}
```

Use JDK `HttpClient` (since v1.1) class to do the download.

```
/* Use the JDK HttpClient (since v1.1) class to do the download. */
public byte[] useHttpClientToGet(String presignedUrlString) {
 ByteArrayOutputStream byteArrayOutputStream = new
 ByteArrayOutputStream(); // Capture the response body to a byte array.

 HttpRequest.Builder requestBuilder = HttpRequest.newBuilder();
 HttpClient httpClient = HttpClient.newHttpClient();
 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpResponse<InputStream> response = httpClient.send(requestBuilder
 .uri(presignedUrl.toURI())
 .GET()
 .build(),
 HttpResponse.BodyHandlers.ofInputStream());
 }
```



```

 IoUtils.copy(response.body(), byteArrayOutputStream);

 logger.info("HTTP response code is " + response.statusCode());

 } catch (URISyntaxException | InterruptedException | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
}

```

Use the AWS SDK for Java `SdkHttpClient` class to do the download.

```

/* Use the AWS SDK for Java SdkHttpClient class to do the download. */
public byte[] useSdkHttpClientToGet(String presignedUrlString) {

 ByteArrayOutputStream byteArrayOutputStream = new
ByteArrayOutputStream(); // Capture the response body to a byte array.
 try {
 URL presignedUrl = new URL(presignedUrlString);
 SdkHttpRequest request = SdkHttpRequest.builder()
 .method(SdkHttpMethod.GET)
 .uri(presignedUrl.toURI())
 .build();

 HttpExecuteRequest executeRequest = HttpExecuteRequest.builder()
 .request(request)
 .build();

 try (SdkHttpClient sdkHttpClient = ApacheHttpClient.create()) {
 HttpExecuteResponse response =
sdkHttpClient.prepareRequest(executeRequest).call();
 response.responseBody().ifPresentOrElse(
 abortableInputStream -> {
 try {
 IoUtils.copy(abortableInputStream,
byteArrayOutputStream);
 } catch (IOException e) {
 throw new RuntimeException(e);
 }
 },
 () -> logger.error("No response body."));
 }
 }
}

```

```
 logger.info("HTTP Response code is {}",
response.httpResponse().statusCode());
 }
 } catch (URISyntaxException | IOException e) {
 logger.error(e.getMessage(), e);
 }
 return byteArrayOutputStream.toByteArray();
}
```

Generate a pre-signed URL with metadata in headers for an upload, then upload a file (PUT request).

### Imports.

```
import com.example.s3.util.PresignUrlUtils;
import org.slf4j.Logger;
import software.amazon.awssdk.core.internal.sync.FileContentStreamProvider;
import software.amazon.awssdk.http.HttpExecuteRequest;
import software.amazon.awssdk.http.HttpExecuteResponse;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.SdkHttpMethod;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.PresignedPutObjectRequest;
import software.amazon.awssdk.services.s3.presigner.model.PutObjectPresignRequest;

import java.io.File;
import java.io.IOException;
import java.io.OutputStream;
import java.io.RandomAccessFile;
import java.net.HttpURLConnection;
import java.net.URISyntaxException;
import java.net.URL;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.ByteBuffer;
```

```
import java.nio.channels.FileChannel;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Duration;
import java.util.Map;
import java.util.UUID;
```

## Generate the URL.

```
/* Create a presigned URL to use in a subsequent PUT request */
public String createPresignedUrl(String bucketName, String keyName, Map<String,
String> metadata) {
 try (S3Presigner presigner = S3Presigner.create()) {

 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .metadata(metadata)
 .build();

 PutObjectPresignRequest presignRequest =
PutObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // The URL expires
in 10 minutes.
 .putObjectRequest(objectRequest)
 .build();

 PresignedPutObjectRequest presignedRequest =
presigner.presignPutObject(presignRequest);
 String myURL = presignedRequest.url().toString();
 logger.info("Presigned URL to upload a file to: [{}]", myURL);
 logger.info("HTTP method: [{}]",
presignedRequest.httpRequest().method());

 return presignedRequest.url().toExternalForm();
 }
}
```

Upload a file object by using any one of the following three approaches.

Use the JDK `URLConnection` (since v1.1) class to do the upload.

```

/* Use the JDK HttpURLConnection (since v1.1) class to do the upload. */
public void useHttpURLConnectionToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());
 try {
 URL presignedUrl = new URL(presignedUrlString);
 HttpURLConnection connection = (HttpURLConnection)
presignedUrl.openConnection();
 connection.setDoOutput(true);
 metadata.forEach((k, v) -> connection.setRequestProperty("x-amz-meta-" +
k, v));
 connection.setRequestMethod("PUT");
 OutputStream out = connection.getOutputStream();

 try (RandomAccessFile file = new RandomAccessFile(fileToPut, "r");
 FileChannel inChannel = file.getChannel()) {
 ByteBuffer buffer = ByteBuffer.allocate(8192); //Buffer size is 8k

 while (inChannel.read(buffer) > 0) {
 buffer.flip();
 for (int i = 0; i < buffer.limit(); i++) {
 out.write(buffer.get());
 }
 buffer.clear();
 }
 } catch (IOException e) {
 logger.error(e.getMessage(), e);
 }

 out.close();
 connection.getResponseCode();
 logger.info("HTTP response code is " + connection.getResponseCode());

 } catch (S3Exception | IOException e) {
 logger.error(e.getMessage(), e);
 }
}

```

Use the JDK `HttpClient` (since v11) class to do the upload.

```

/* Use the JDK HttpClient (since v11) class to do the upload. */
public void useHttpClientToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());

 HttpRequest.Builder requestBuilder = HttpRequest.newBuilder();
 metadata.forEach((k, v) -> requestBuilder.header("x-amz-meta-" + k, v));

 HttpClient httpClient = HttpClient.newHttpClient();
 try {
 final HttpResponse<Void> response = httpClient.send(requestBuilder
 .uri(new URL(presignedUrlString).toURI())

 .PUT(HttpRequest.BodyPublishers.ofFile(Path.of(fileToPut.toURI()))
 .build(),
 HttpResponse.BodyHandlers.discarding());

 logger.info("HTTP response code is " + response.statusCode());

 } catch (URISyntaxException | InterruptedException | IOException e) {
 logger.error(e.getMessage(), e);
 }
}

```

Use the AWS for Java V2 SdkHttpClient class to do the upload.

```

/* Use the AWS SDK for Java V2 SdkHttpClient class to do the upload. */
public void useSdkHttpClientToPut(String presignedUrlString, File fileToPut,
Map<String, String> metadata) {
 logger.info("Begin [{}] upload", fileToPut.toString());

 try {
 URL presignedUrl = new URL(presignedUrlString);

 SdkHttpRequest.Builder requestBuilder = SdkHttpRequest.builder()
 .method(SdkHttpMethod.PUT)
 .uri(presignedUrl.toURI());
 // Add headers
 metadata.forEach((k, v) -> requestBuilder.putHeader("x-amz-meta-" + k,
v));

 // Finish building the request.
 SdkHttpRequest request = requestBuilder.build();
 }
}

```

```

 HttpExecuteRequest executeRequest = HttpExecuteRequest.builder()
 .request(request)
 .contentStreamProvider(new
FileContentStreamProvider(fileToPut.toPath()))
 .build();

 try (SdkHttpClient sdkHttpClient = ApacheHttpClient.create()) {
 HttpExecuteResponse response =
sdkHttpClient.prepareRequest(executeRequest).call();
 logger.info("Response code: {}",
response.httpResponse().statusCode());
 }
 } catch (URISyntaxException | IOException e) {
 logger.error(e.getMessage(), e);
 }
}

```

Generate a pre-signed URL with query parameters for an upload, then upload a file (PUT request).

### Imports.

```

import com.example.s3.util.PresignUrlUtils;
import org.slf4j.Logger;
import software.amazon.awssdk.awscore.AWSRequestOverrideConfiguration;
import software.amazon.awssdk.core.internal.sync.FileContentStreamProvider;
import software.amazon.awssdk.http.HttpExecuteRequest;
import software.amazon.awssdk.http.HttpExecuteResponse;
import software.amazon.awssdk.http.SdkHttpClient;
import software.amazon.awssdk.http.SdkHttpMethod;
import software.amazon.awssdk.http.SdkHttpRequest;
import software.amazon.awssdk.http.apache.ApacheHttpClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.PresignedPutObjectRequest;
import software.amazon.awssdk.services.s3.presigner.model.PutObjectPresignRequest;

import java.io.File;
import java.io.IOException;
import java.net.URISyntaxException;

```

```
import java.net.URL;
import java.nio.file.Paths;
import java.time.Duration;
import java.util.Map;
import java.util.UUID;
```

## Generate the URL.

```
/**
 * Creates a presigned URL to use in a subsequent HTTP PUT request. The code
 * adds query parameters
 * to the request instead of using headers. By using query parameters, you do
 * not need to add the
 * the parameters as headers when the PUT request is eventually sent.
 *
 * @param bucketName Bucket name where the object will be uploaded.
 * @param keyName Key name of the object that will be uploaded.
 * @param queryParams Query string parameters to be added to the presigned URL.
 * @return
 */
public String createPresignedUrl(String bucketName, String keyName, Map<String,
String> queryParams) {
 try (S3Presigner presigner = S3Presigner.create()) {
 // Create an override configuration to store the query parameters.
 AwsRequestOverrideConfiguration.Builder overrideConfigurationBuilder =
 AwsRequestOverrideConfiguration.builder();

 queryParams.forEach(overrideConfigurationBuilder::putRawQueryParameter);

 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .overrideConfiguration(overrideConfigurationBuilder.build()) //
 Add the override configuration.
 .build();

 PutObjectPresignRequest presignRequest =
 PutObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // The URL expires
 in 10 minutes.
 .putObjectRequest(objectRequest)
 .build();
```

```

 PresignedPutObjectRequest presignedRequest =
presigner.presignPutObject(presignRequest);
 String myURL = presignedRequest.url().toString();
 logger.info("Presigned URL to upload a file to: [{}]", myURL);
 logger.info("HTTP method: [{}]",
presignedRequest.httpRequest().method());

 return presignedRequest.url().toExternalForm();
 }
}

```

Use the AWS for Java V2 `SdkHttpClient` class to do the upload.

```

/**
 * Use the AWS SDK for Java V2 SdkHttpClient class to execute the PUT request.
Since the
 * URL contains the query parameters, no headers are needed for metadata, SSE
settings, or ACL settings.
 *
 * @param presignedUrlString The URL for the PUT request.
 * @param fileToPut File to uplaod
 */
public void useSdkHttpClientToPut(String presignedUrlString, File fileToPut) {
 logger.info("Begin [{}] upload", fileToPut.toString());

 try {
 URL presignedUrl = new URL(presignedUrlString);

 SdkHttpRequest.Builder requestBuilder = SdkHttpRequest.builder()
 .method(SdkHttpMethod.PUT)
 .uri(presignedUrl.toURI());

 SdkHttpRequest request = requestBuilder.build();

 HttpExecuteRequest executeRequest = HttpExecuteRequest.builder()
 .request(request)
 .contentStreamProvider(new
FileContentStreamProvider(fileToPut.toPath()))
 .build();
 }
}

```



```
 try (SdkHttpClient sdkHttpClient = ApacheHttpClient.create()) {
 HttpExecuteResponse response =
 sdkHttpClient.prepareRequest(executeRequest).call();
 logger.info("Response code: {}",
 response.httpResponse().statusCode());
 }
 } catch (URISyntaxException | IOException e) {
 logger.error(e.getMessage(), e);
 }
}
```

## Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon Rekognition
- Amazon S3
- Amazon SNS

## Delete incomplete multipart uploads

The following code example shows how to delete or stop incomplete Amazon S3 multipart uploads.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

To stop multipart uploads that are in-progress or incomplete for any reason, you can get a list uploads and then delete them as shown in the following example.

```
/**
 * Aborts all incomplete multipart uploads from the specified S3 bucket.
 *
 * @param bucketName the name of the S3 bucket
 */
public static void abortIncompleteMultipartUploadsFromList(String bucketName) {
 ListMultipartUploadsRequest listMultipartUploadsRequest =
ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 ListMultipartUploadsResponse response =
s3Client.listMultipartUploads(listMultipartUploadsRequest);
 List<MultipartUpload> uploads = response.uploads();

 AbortMultipartUploadRequest abortMultipartUploadRequest;
 for (MultipartUpload upload : uploads) {
 abortMultipartUploadRequest = AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(upload.key())
 .expectedBucketOwner(accountId)
 .uploadId(upload.uploadId())
 .build();

 AbortMultipartUploadResponse abortMultipartUploadResponse =
s3Client.abortMultipartUpload(abortMultipartUploadRequest);
 if (abortMultipartUploadResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully aborted.",
upload.uploadId(), bucketName);
 }
 }
}
```

```
}
```

To delete incomplete multipart uploads that were initiated before or after a date, you can selectively delete multipart uploads based on a point in time as shown in the following example.

```
/**
 * Aborts incomplete multipart uploads older than the specified point in time.
 *
 * @param bucketName the name of the S3 bucket
 * @param pointInTime the cutoff time; uploads initiated before this are aborted
 */
static void abortIncompleteMultipartUploadsOlderThan(String bucketName, Instant
pointInTime) {
 ListMultipartUploadsRequest listMultipartUploadsRequest =
ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 ListMultipartUploadsResponse response =
s3Client.listMultipartUploads(listMultipartUploadsRequest);
 List<MultipartUpload> uploads = response.uploads();

 AbortMultipartUploadRequest abortMultipartUploadRequest;
 for (MultipartUpload upload : uploads) {
 logger.info("Found multipartUpload with upload ID [{}], initiated [{}]",
upload.uploadId(), upload.initiated());
 if (upload.initiated().isBefore(pointInTime)) {
 abortMultipartUploadRequest = AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(upload.key())
 .expectedBucketOwner(accountId)
 .uploadId(upload.uploadId())
 .build();

 AbortMultipartUploadResponse abortMultipartUploadResponse =
s3Client.abortMultipartUpload(abortMultipartUploadRequest);
 if (abortMultipartUploadResponse.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully
aborted.", upload.uploadId(), bucketName);
 }
 }
 }
}
```

```
 }
}
```

If you have access to the upload ID after you begin a multipart upload, you can delete the in-progress upload by using the ID.

```
/**
 * Aborts a multipart upload using the upload ID.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the object key
 */
static void abortMultipartUploadUsingUploadId(String bucketName, String key) {
 String uploadId = startUploadReturningUploadId(bucketName, key);
 AbortMultipartUploadResponse response = s3Client.abortMultipartUpload(b -> b
 .uploadId(uploadId)
 .bucket(bucketName)
 .key(key));

 if (response.sdkHttpResponse().isSuccessful()) {
 logger.info("Upload ID [{}] to bucket [{}] successfully aborted.",
uploadId, bucketName);
 }
}
```

To consistently delete incomplete multipart uploads older than a certain number of days, set up a bucket lifecycle configuration for the bucket. The following example shows how to create a rule to delete incomplete uploads older than 7 days.

```
/**
 * Configures a lifecycle rule to abort incomplete multipart uploads after 7
days.
 *
 * @param bucketName the name of the S3 bucket
 */
static void abortMultipartUploadsUsingLifecycleConfig(String bucketName) {
 Collection<LifecycleRule> lifeCycleRules = List.of(LifecycleRule.builder()
 .abortIncompleteMultipartUpload(b -> b.
 daysAfterInitiation(7))
 .status("Enabled")
);
}
```

```

 .filter(SdkBuilder::build) // Filter element is required.
 .build());

 PutBucketLifecycleConfigurationResponse response =
s3Client.putBucketLifecycleConfiguration(b -> b
 .bucket(bucketName)
 .lifecycleConfiguration(b1 -> b1.rules(lifeCycleRules)));

 if (response.sdkHttpResponse().isSuccessful()) {
 logger.info("Rule to abort incomplete multipart uploads added to
bucket.");
 } else {
 logger.error("Unsuccessfully applied rule. HTTP status code is [{}]",
response.sdkHttpResponse().statusCode());
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AbortMultipartUpload](#)
  - [ListMultipartUploads](#)
  - [PutBucketLifecycleConfiguration](#)

## Detect PPE in images

The following code example shows how to build an app that uses Amazon Rekognition to detect Personal Protective Equipment (PPE) in images.

### SDK for Java 2.x

Shows how to create an AWS Lambda function that detects images with Personal Protective Equipment.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Amazon Rekognition
- Amazon S3

- Amazon SES

## Detect objects in images

The following code example shows how to build an app that uses Amazon Rekognition to detect objects by category in images.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app that uses Amazon Rekognition to identify objects by category in images located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Rekognition
- Amazon S3
- Amazon SES

## Detect people and objects in a video

The following code example shows how to detect people and objects in a video with Amazon Rekognition.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app to detect faces and objects in videos located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Rekognition
- Amazon S3

- Amazon SES
- Amazon SNS
- Amazon SQS

## Download S3 'directories'

The following code example shows how to download and filter the contents of Amazon S3 bucket 'directories'.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

This example show how to use the [S3TransferManager](#) in the AWS SDK for Java 2.x to download 'directories' from an Amazon S3 bucket. It also demonstrates how to use [DownloadFilters](#) in the request.

```
/**
 * For standard buckets, S3 provides the illusion of a directory structure
 * through the use of keys. When you upload
 * an object to an S3 bucket, you specify a key, which is essentially the "path"
 * to the object. The key can contain
 * forward slashes ("/") to make it appear as if the object is stored in a
 * directory structure, but this is just a
 * logical representation, not an actual directory.
 * <p><pre>
 * In this example, our S3 bucket contains the following objects:
 *
 * folder1/file1.txt
 * folder1/file2.txt
 * folder1/file3.txt
 * folder2/file1.txt
 * folder2/file2.txt
 * folder2/file3.txt
 * folder3/file1.txt
 * folder3/file2.txt
 * folder3/file3.txt
```

```

*
* When method `downloadS3Directories` is invoked with
* `destinationPathURI` set to `/test`, the downloaded
* directory looks like:
*
* |- test
* |- folder1
* |- file1.txt
* |- file2.txt
* |- file3.txt
* |- folder3
* |- file1.txt
* |- file2.txt
* |- file3.txt
* </pre>
*
* @param transferManager An S3TransferManager instance.
* @param destinationPathURI local directory to hold the downloaded S3
'directories' and files.
* @param bucketName The S3 bucket that contains the 'directories' to
download.
* @return The number of objects (files, in this case) that were downloaded.
*/
public Integer downloadS3Directories(S3TransferManager transferManager,
 URI destinationPathURI, String bucketName)
{
 // Define the filters for which 'directories' we want to download.
 DownloadFilter folder1Filter = (S3Object s3Object) ->
s3Object.key().startsWith("folder1/");
 DownloadFilter folder3Filter = (S3Object s3Object) ->
s3Object.key().startsWith("folder3/");
 DownloadFilter folderFilter = s3Object ->
folder1Filter.or(folder3Filter).test(s3Object);

 DirectoryDownload directoryDownload =
transferManager.downloadDirectory(DownloadDirectoryRequest.builder()
 .destination(Paths.get(destinationPathURI))
 .bucket(bucketName)
 .filter(folderFilter)
 .build());

 CompletedDirectoryDownload completedDirectoryDownload =
directoryDownload.completionFuture().join();
}

```



```

 Integer numFilesInFolder1 =
Paths.get(destinationPathURI).resolve("folder1").toFile().list().length;
 Integer numFilesInFolder3 =
Paths.get(destinationPathURI).resolve("folder3").toFile().list().length;

 try {
 assert numFilesInFolder1 == 3;
 assert numFilesInFolder3 == 3;
 assert !
Paths.get(destinationPathURI).resolve("folder2").toFile().exists(); // `folder2` was
not downloaded.
 } catch (AssertionError e) {
 logger.error("An assertion failed.");
 }

 completedDirectoryDownload.failedTransfers()
 .forEach(fail -> logger.warn("Object failed to transfer [{}]",
fail.exception().getMessage()));
 return numFilesInFolder1 + numFilesInFolder3;
 }

```

## Download objects to a local directory

The following code example shows how to download all objects in an Amazon Simple Storage Service (Amazon S3) bucket to a local directory.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use an [S3TransferManager](#) to [download all S3 objects](#) in the same S3 bucket. View the [complete file](#) and [test](#).

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.sync.RequestBody;

```

```

import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedDirectoryDownload;
import software.amazon.awssdk.transfer.s3.model.DirectoryDownload;
import software.amazon.awssdk.transfer.s3.model.DownloadDirectoryRequest;
import java.io.IOException;
import java.net.URI;
import java.net.URISyntaxException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.HashSet;
import java.util.Set;
import java.util.UUID;
import java.util.stream.Collectors;

 public Integer downloadObjectsToDirectory(S3TransferManager transferManager,
 URI destinationPathURI, String bucketName) {
 DirectoryDownload directoryDownload =
transferManager.downloadDirectory(DownloadDirectoryRequest.builder()
 .destination(Paths.get(destinationPathURI))
 .bucket(bucketName)
 .build());
 CompletedDirectoryDownload completedDirectoryDownload =
directoryDownload.completionFuture().join();

 completedDirectoryDownload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryDownload.failedTransfers().size();
 }

```

- For API details, see [DownloadDirectory](#) in *AWS SDK for Java 2.x API Reference*.

## Lock Amazon S3 objects

The following code example shows how to work with S3 object lock features.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Amazon S3 object lock features.

```
import software.amazon.awssdk.services.s3.model.ObjectLockLegalHold;
import software.amazon.awssdk.services.s3.model.ObjectLockRetention;
import java.io.BufferedWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

/*
Before running this Java V2 code example, set up your development
environment, including your credentials.

For more information, see the following documentation topic:
https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/setup.html

This Java example performs the following tasks:
 1. Create test Amazon Simple Storage Service (S3) buckets with different lock
 policies.
 2. Upload sample objects to each bucket.
 3. Set some Legal Hold and Retention Periods on objects and buckets.
 4. Investigate lock policies by viewing settings or attempting to delete or
 overwrite objects.
 5. Clean up objects and buckets.
*/
public class S3ObjectLockWorkflow {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 static String bucketName;
 static S3LockActions s3LockActions;
 private static final List<String> bucketNames = new ArrayList<>();
 private static final List<String> fileNames = new ArrayList<>();
```

```
public static void main(String[] args) {
 final String usage = ""
 Usage:
 <bucketName> \s

 Where:
 bucketName - The Amazon S3 bucket name.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 s3LockActions = new S3LockActions();
 bucketName = args[0];
 Scanner scanner = new Scanner(System.in);

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon Simple Storage Service (S3) Object
Locking Feature Scenario.");
 System.out.println("Press Enter to continue...");
 scanner.nextLine();
 configurationSetup();
 System.out.println(DASHES);

 System.out.println(DASHES);
 setup();
 System.out.println("Setup is complete. Press Enter to continue...");
 scanner.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Lets present the user with choices.");
 System.out.println("Press Enter to continue...");
 scanner.nextLine();
 demoActionChoices() ;
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Would you like to clean up the resources? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 cleanup();
 System.out.println("Clean up is complete.");
 }
}
```

```

 }

 System.out.println("Press Enter to continue...");
 scanner.nextLine();
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Amazon S3 Object Locking Workflow is complete.");
 System.out.println(DASHES);
}

// Present the user with the demo action choices.
public static void demoActionChoices() {
 String[] choices = {
 "List all files in buckets.",
 "Attempt to delete a file.",
 "Attempt to delete a file with retention period bypass.",
 "Attempt to overwrite a file.",
 "View the object and bucket retention settings for a file.",
 "View the legal hold settings for a file.",
 "Finish the workflow."
 };

 int choice = 0;
 while (true) {
 System.out.println(DASHES);
 choice = getChoiceResponse("Explore the S3 locking features by selecting
one of the following choices:", choices);
 System.out.println(DASHES);
 System.out.println("You selected "+choices[choice]);
 switch (choice) {
 case 0 -> {
 s3LockActions.listBucketsAndObjects(bucketNames, true);
 }

 case 1 -> {
 System.out.println("Enter the number of the object to delete:");
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, true);
 List<String> fileKeys = allFiles.stream().map(f ->
f.getKeyName()).collect(Collectors.toList());
 String[] fileKeysArray = fileKeys.toArray(new String[0]);
 int fileChoice = getChoiceResponse(null, fileKeysArray);
 String objectKey = fileKeys.get(fileChoice);

```

```

 String bucketName = allFiles.get(fileChoice).getBucketName();
 String version = allFiles.get(fileChoice).getVersion();
 s3LockActions.deleteObjectFromBucket(bucketName, objectKey,
false, version);
 }

 case 2 -> {
 System.out.println("Enter the number of the object to delete:");
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, true);
 List<String> fileKeys = allFiles.stream().map(f ->
f.getKeyName()).collect(Collectors.toList());
 String[] fileKeysArray = fileKeys.toArray(new String[0]);
 int fileChoice = getChoiceResponse(null, fileKeysArray);
 String objectKey = fileKeys.get(fileChoice);
 String bucketName = allFiles.get(fileChoice).getBucketName();
 String version = allFiles.get(fileChoice).getVersion();
 s3LockActions.deleteObjectFromBucket(bucketName, objectKey,
true, version);
 }

 case 3 -> {
 System.out.println("Enter the number of the object to
overwrite:");
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, true);
 List<String> fileKeys = allFiles.stream().map(f ->
f.getKeyName()).collect(Collectors.toList());
 String[] fileKeysArray = fileKeys.toArray(new String[0]);
 int fileChoice = getChoiceResponse(null, fileKeysArray);
 String objectKey = fileKeys.get(fileChoice);
 String bucketName = allFiles.get(fileChoice).getBucketName();

 // Attempt to overwrite the file.
 try (BufferedWriter writer = new BufferedWriter(new
java.io.FileWriter(objectKey))) {
 writer.write("This is a modified text.");

 } catch (IOException e) {
 e.printStackTrace();
 }
 s3LockActions.uploadFile(bucketName, objectKey, objectKey);
 }
}

```

```

 case 4 -> {
 System.out.println("Enter the number of the object to
overwrite:");
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, true);
 List<String> fileKeys = allFiles.stream().map(f ->
f.getKeyName()).collect(Collectors.toList());
 String[] fileKeysArray = fileKeys.toArray(new String[0]);
 int fileChoice = getChoiceResponse(null, fileKeysArray);
 String objectKey = fileKeys.get(fileChoice);
 String bucketName = allFiles.get(fileChoice).getBucketName();
 s3LockActions.getObjectRetention(bucketName, objectKey);
 }

 case 5 -> {
 System.out.println("Enter the number of the object to view:");
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, true);
 List<String> fileKeys = allFiles.stream().map(f ->
f.getKeyName()).collect(Collectors.toList());
 String[] fileKeysArray = fileKeys.toArray(new String[0]);
 int fileChoice = getChoiceResponse(null, fileKeysArray);
 String objectKey = fileKeys.get(fileChoice);
 String bucketName = allFiles.get(fileChoice).getBucketName();
 s3LockActions.getObjectLegalHold(bucketName, objectKey);
 s3LockActions.getBucketObjectLockConfiguration(bucketName);
 }

 case 6 -> {
 System.out.println("Exiting the workflow...");
 return;
 }

 default -> {
 System.out.println("Invalid choice. Please select again.");
 }
 }
}

// Clean up the resources from the scenario.
private static void cleanup() {
 List<S3InfoObject> allFiles =
s3LockActions.listBucketsAndObjects(bucketNames, false);

```

```

 for (S3InfoObject fileInfo : allFiles) {
 String bucketName = fileInfo.getBucketName();
 String key = fileInfo.getKeyName();
 String version = fileInfo.getVersion();
 if (bucketName.contains("lock-enabled") ||
(bucketName.contains("retention-after-creation")))) {
 ObjectLockLegalHold legalHold =
s3LockActions.getObjectLegalHold(bucketName, key);
 if (legalHold != null) {
 String holdStatus = legalHold.status().name();
 System.out.println(holdStatus);
 if (holdStatus.compareTo("ON") == 0) {
 s3LockActions.modifyObjectLegalHold(bucketName, key, false);
 }
 }
 // Check for a retention period.
 ObjectLockRetention retention =
s3LockActions.getObjectRetention(bucketName, key);
 boolean hasRetentionPeriod ;
 hasRetentionPeriod = retention != null;
 s3LockActions.deleteObjectFromBucket(bucketName,
key,hasRetentionPeriod, version);

 } else {
 System.out.println(bucketName +" objects do not have a legal lock");
 s3LockActions.deleteObjectFromBucket(bucketName, key,false,
version);
 }
 }

 // Delete the buckets.
 System.out.println("Delete "+bucketName);
 for (String bucket : bucketNames){
 s3LockActions.deleteBucketByName(bucket);
 }
 }

 private static void setup() {
 Scanner scanner = new Scanner(System.in);
 System.out.println("""
 For this workflow, we will use the AWS SDK for Java to create
several S3
 buckets and files to demonstrate working with S3 locking features.
 """);
 }

```



```

 System.out.println("S3 buckets can be created either with or without object
lock enabled.");
 System.out.println("Press Enter to continue...");
 scanner.nextLine();

 // Create three S3 buckets.
 s3LockActions.createBucketWithLockOptions(false, bucketNames.get(0));
 s3LockActions.createBucketWithLockOptions(true, bucketNames.get(1));
 s3LockActions.createBucketWithLockOptions(false, bucketNames.get(2));
 System.out.println("Press Enter to continue.");
 scanner.nextLine();

 System.out.println("Bucket "+bucketNames.get(2) +" will be configured to use
object locking with a default retention period.");
 s3LockActions.modifyBucketDefaultRetention(bucketNames.get(2));
 System.out.println("Press Enter to continue.");
 scanner.nextLine();

 System.out.println("Object lock policies can also be added to existing
buckets. For this example, we will use "+bucketNames.get(1));
 s3LockActions.enableObjectLockOnBucket(bucketNames.get(1));
 System.out.println("Press Enter to continue.");
 scanner.nextLine();

 // Upload some files to the buckets.
 System.out.println("Now let's add some test files:");
 String fileName = "exampleFile.txt";
 int fileCount = 2;
 try (BufferedWriter writer = new BufferedWriter(new
java.io.FileWriter(fileName))) {
 writer.write("This is a sample file for uploading to a bucket.");

 } catch (IOException e) {
 e.printStackTrace();
 }

 for (String bucketName : bucketNames){
 for (int i = 0; i < fileCount; i++) {
 // Get the file name without extension.
 String fileNameWithoutExtension =
java.nio.file.Paths.get(fileName).getFileName().toString();
 int extensionIndex = fileNameWithoutExtension.lastIndexOf('.');
 if (extensionIndex > 0) {

```

```

 fileNameWithoutExtension = fileNameWithoutExtension.substring(0,
extensionIndex);
 }

 // Create the numbered file names.
 String numberedFileName = fileNameWithoutExtension + i +
getFileExtension(fileName);
 fileNames.add(numberedFileName);
 s3LockActions.uploadFile(bucketName, numberedFileName, fileName);
}
}

String question = null;
System.out.print("Press Enter to continue...");
scanner.nextLine();
System.out.println("Now we can set some object lock policies on individual
files:");
for (String bucketName : bucketNames) {
 for (int i = 0; i < fileNames.size(); i++){

 // No modifications to the objects in the first bucket.
 if (!bucketName.equals(bucketNames.get(0))) {
 String exampleFileName = fileNames.get(i);
 switch (i) {
 case 0 -> {
 question = "Would you like to add a legal hold to " +
exampleFileName + " in " + bucketName + " (y/n)?";
 System.out.println(question);
 String ans = scanner.nextLine().trim();
 if (ans.equalsIgnoreCase("y")) {
 System.out.println("**** You have selected to put a
legal hold " + exampleFileName);

 // Set a legal hold.
 s3LockActions.modifyObjectLegalHold(bucketName,
exampleFileName, true);
 }
 }
 case 1 -> {
 ""
 Would you like to add a 1 day Governance retention
period to %s in %s (y/n)?

```

Reminder: Only a user with the `s3:BypassGovernanceRetention` permission will be able to delete this file or its bucket until the retention period has expired.

```

 ""formatted(exampleFileName, bucketName);
 System.out.println(question);
 String ans2 = scanner.nextLine().trim();
 if (ans2.equalsIgnoreCase("y")) {

s3LockActions.modifyObjectRetentionPeriod(bucketName, exampleFileName);
 }
 }
}

// Get file extension.
private static String getFileExtension(String fileName) {
 int dotIndex = fileName.lastIndexOf('.');
 if (dotIndex > 0) {
 return fileName.substring(dotIndex);
 }
 return "";
}

public static void configurationSetup() {
 String noLockBucketName = bucketName + "-no-lock";
 String lockEnabledBucketName = bucketName + "-lock-enabled";
 String retentionAfterCreationBucketName = bucketName + "-retention-after-
creation";
 bucketNames.add(noLockBucketName);
 bucketNames.add(lockEnabledBucketName);
 bucketNames.add(retentionAfterCreationBucketName);
}

public static int getChoiceResponse(String question, String[] choices) {
 Scanner scanner = new Scanner(System.in);
 if (question != null) {
 System.out.println(question);
 for (int i = 0; i < choices.length; i++) {
 System.out.println("\t" + (i + 1) + ". " + choices[i]);
 }
 }
}

```

```
int choiceNumber = 0;
while (choiceNumber < 1 || choiceNumber > choices.length) {
 String choice = scanner.nextLine();
 try {
 choiceNumber = Integer.parseInt(choice);
 } catch (NumberFormatException e) {
 System.out.println("Invalid choice. Please enter a valid number.");
 }
}

return choiceNumber - 1;
}
}
```

A wrapper class for S3 functions.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.BucketVersioningStatus;
import software.amazon.awssdk.services.s3.model.ChecksumAlgorithm;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.DefaultRetention;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.GetObjectLegalHoldRequest;
import software.amazon.awssdk.services.s3.model.GetObjectLegalHoldResponse;
import software.amazon.awssdk.services.s3.model.GetObjectLockConfigurationRequest;
import software.amazon.awssdk.services.s3.model.GetObjectLockConfigurationResponse;
import software.amazon.awssdk.services.s3.model.GetObjectRetentionRequest;
import software.amazon.awssdk.services.s3.model.GetObjectRetentionResponse;
import software.amazon.awssdk.services.s3.model.HeadBucketRequest;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsRequest;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsResponse;
import software.amazon.awssdk.services.s3.model.MFADelete;
import software.amazon.awssdk.services.s3.model.ObjectLockConfiguration;
import software.amazon.awssdk.services.s3.model.ObjectLockEnabled;
import software.amazon.awssdk.services.s3.model.ObjectLockLegalHold;
import software.amazon.awssdk.services.s3.model.ObjectLockLegalHoldStatus;
import software.amazon.awssdk.services.s3.model.ObjectLockRetention;
import software.amazon.awssdk.services.s3.model.ObjectLockRetentionMode;
import software.amazon.awssdk.services.s3.model.ObjectLockRule;
```

```
import software.amazon.awssdk.services.s3.model.PutBucketVersioningRequest;
import software.amazon.awssdk.services.s3.model.PutObjectLegalHoldRequest;
import software.amazon.awssdk.services.s3.model.PutObjectLockConfigurationRequest;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
import software.amazon.awssdk.services.s3.model.PutObjectRetentionRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.VersioningConfiguration;
import software.amazon.awssdk.services.s3.waiters.S3Waiter;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Collectors;

// Contains application logic for the Amazon S3 operations used in this workflow.
public class S3LockActions {

 private static S3Client getClient() {
 return S3Client.builder()
 .region(Region.US_EAST_1)
 .build();
 }

 // Set or modify a retention period on an object in an S3 bucket.
 public void modifyObjectRetentionPeriod(String bucketName, String objectKey) {
 // Calculate the instant one day from now.
 Instant futureInstant = Instant.now().plus(1, ChronoUnit.DAYS);

 // Convert the Instant to a ZonedDateTime object with a specific time zone.
 ZonedDateTime zonedDateTime = futureInstant.atZone(ZoneId.systemDefault());

 // Define a formatter for human-readable output.
 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");

 // Format the ZonedDateTime object to a human-readable date string.
 String humanReadableDate = formatter.format(zonedDateTime);
 }
}
```

```

 // Print the formatted date string.
 System.out.println("Formatted Date: " + humanReadableDate);
 ObjectLockRetention retention = ObjectLockRetention.builder()
 .mode(ObjectLockRetentionMode.GOVERNANCE)
 .retainUntilDate(futureInstant)
 .build();

 PutObjectRetentionRequest retentionRequest =
 PutObjectRetentionRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .retention(retention)
 .build();

 getClient().putObjectRetention(retentionRequest);
 System.out.println("Set retention for "+objectKey+" in "+bucketName+"
until "+ humanReadableDate +".");
 }

 // Get the legal hold details for an S3 object.
 public ObjectLockLegalHold getObjectLegalHold(String bucketName, String
objectKey) {
 try {
 GetObjectLegalHoldRequest legalHoldRequest =
 GetObjectLegalHoldRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 GetObjectLegalHoldResponse response =
 getClient().getObjectLegalHold(legalHoldRequest);
 System.out.println("Object legal hold for " + objectKey + " in " +
bucketName +
 ":\n\tStatus: " + response.legalHold().status());
 return response.legalHold();

 } catch (S3Exception ex) {
 System.out.println("\tUnable to fetch legal hold: '" + ex.getMessage() +
 "'");
 }

 return null;
 }

```

```

// Create a new Amazon S3 bucket with object lock options.
public void createBucketWithLockOptions(boolean enableObjectLock, String
bucketName) {
 S3Waiter s3Waiter = getClient().waiter();
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .objectLockEnabledForBucket(enableObjectLock)
 .build();

 getClient().createBucket(bucketRequest);
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 // Wait until the bucket is created and print out the response.
 s3Waiter.waitUntilBucketExists(bucketRequestWait);
 System.out.println(bucketName + " is ready");
}

public List<S3InfoObject> listBucketsAndObjects(List<String> bucketNames,
Boolean interactive) {
 AtomicInteger counter = new AtomicInteger(0); // Initialize counter.
 return bucketNames.stream()
 .flatMap(bucketName ->
listBucketObjectsAndVersions(bucketName).versions().stream()
 .map(version -> {
 S3InfoObject s3InfoObject = new S3InfoObject();
 s3InfoObject.setBucketName(bucketName);
 s3InfoObject.setVersion(version.versionId());
 s3InfoObject.setKeyName(version.key());
 return s3InfoObject;
 })))
 .peek(s3InfoObject -> {
 int i = counter.incrementAndGet(); // Increment and get the updated
value.

 if (interactive) {
 System.out.println(i + ": " + s3InfoObject.getKeyName());
 System.out.printf("%5s Bucket name: %s\n", "",
s3InfoObject.getBucketName());
 System.out.printf("%5s Version: %s\n", "",
s3InfoObject.getVersion());
 }
 })
 .collect(Collectors.toList());
}

```

```
}

 public ListObjectVersionsResponse listBucketObjectsAndVersions(String
bucketName) {
 ListObjectVersionsRequest versionsRequest =
ListObjectVersionsRequest.builder()
 .bucket(bucketName)
 .build();

 return getClient().listObjectVersions(versionsRequest);
 }

 // Set or modify a retention period on an S3 bucket.
 public void modifyBucketDefaultRetention(String bucketName) {
 VersioningConfiguration versioningConfiguration =
VersioningConfiguration.builder()
 .mfaDelete(MFADelete.DISABLED)
 .status(BucketVersioningStatus.ENABLED)
 .build();

 PutBucketVersioningRequest versioningRequest =
PutBucketVersioningRequest.builder()
 .bucket(bucketName)
 .versioningConfiguration(versioningConfiguration)
 .build();

 getClient().putBucketVersioning(versioningRequest);
 DefaultRetention rention = DefaultRetention.builder()
 .days(1)
 .mode(ObjectLockRetentionMode.GOVERNANCE)
 .build();

 ObjectLockRule lockRule = ObjectLockRule.builder()
 .defaultRetention(rention)
 .build();

 ObjectLockConfiguration objectLockConfiguration =
ObjectLockConfiguration.builder()
 .objectLockEnabled(ObjectLockEnabled.ENABLED)
 .rule(lockRule)
 .build();

 PutObjectLockConfigurationRequest putObjectLockConfigurationRequest =
PutObjectLockConfigurationRequest.builder()
```



```

 .bucket(bucketName)
 .objectLockConfiguration(objectLockConfiguration)
 .build();

 getClient().putObjectLockConfiguration(putObjectLockConfigurationRequest) ;
 System.out.println("Added a default retention to bucket "+bucketName +".");
 }

 // Enable object lock on an existing bucket.
 public void enableObjectLockOnBucket(String bucketName) {
 try {
 VersioningConfiguration versioningConfiguration =
VersioningConfiguration.builder()
 .status(BucketVersioningStatus.ENABLED)
 .build();

 PutBucketVersioningRequest putBucketVersioningRequest =
PutBucketVersioningRequest.builder()
 .bucket(bucketName)
 .versioningConfiguration(versioningConfiguration)
 .build();

 // Enable versioning on the bucket.
 getClient().putBucketVersioning(putBucketVersioningRequest);
 PutObjectLockConfigurationRequest request =
PutObjectLockConfigurationRequest.builder()
 .bucket(bucketName)
 .objectLockConfiguration(ObjectLockConfiguration.builder()
 .objectLockEnabled(ObjectLockEnabled.ENABLED)
 .build())
 .build();

 getClient().putObjectLockConfiguration(request);
 System.out.println("Successfully enabled object lock on "+bucketName);

 } catch (S3Exception ex) {
 System.out.println("Error modifying object lock: '" + ex.getMessage() +
""");
 }
 }

 public void uploadFile(String bucketName, String objectName, String filePath) {
 Path file = Paths.get(filePath);
 PutObjectRequest request = PutObjectRequest.builder()

```

```

 .bucket(bucketName)
 .key(objectName)
 .checksumAlgorithm(ChecksumAlgorithm.SHA256)
 .build();

 PutObjectResponse response = getClient().putObject(request, file);
 if (response != null) {
 System.out.println("\tSuccessfully uploaded " + objectName + " to " +
bucketName + ".");
 } else {
 System.out.println("\tCould not upload " + objectName + " to " +
bucketName + ".");
 }
}

// Set or modify a legal hold on an object in an S3 bucket.
public void modifyObjectLegalHold(String bucketName, String objectKey, boolean
legalHoldOn) {
 ObjectLockLegalHold legalHold ;
 if (legalHoldOn) {
 legalHold = ObjectLockLegalHold.builder()
 .status(ObjectLockLegalHoldStatus.ON)
 .build();
 } else {
 legalHold = ObjectLockLegalHold.builder()
 .status(ObjectLockLegalHoldStatus.OFF)
 .build();
 }

 PutObjectLegalHoldRequest legalHoldRequest =
PutObjectLegalHoldRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .legalHold(legalHold)
 .build();

 getClient().putObjectLegalHold(legalHoldRequest) ;
 System.out.println("Modified legal hold for "+ objectKey +" in "+bucketName
+".");
}

// Delete an object from a specific bucket.
public void deleteObjectFromBucket(String bucketName, String objectKey, boolean
hasRetention, String versionId) {

```

```

 try {
 DeleteObjectRequest objectRequest;
 if (hasRetention) {
 objectRequest = DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .versionId(versionId)
 .bypassGovernanceRetention(true)
 .build();
 } else {
 objectRequest = DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .versionId(versionId)
 .build();
 }

 getClient().deleteObject(objectRequest) ;
 System.out.println("The object was successfully deleted");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 }

 // Get the retention period for an S3 object.
 public ObjectLockRetention getObjectRetention(String bucketName, String key){
 try {
 GetObjectRetentionRequest retentionRequest =
 GetObjectRetentionRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 GetObjectRetentionResponse response =
 getClient().getObjectRetention(retentionRequest);
 System.out.println("tObject retention for "+key+" in "+ bucketName+":
 " + response.retention().mode() +" until "+ response.retention().retainUntilDate()
 +".");

 return response.retention();

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 return null;
 }
 }

```

```

 }
}

public void deleteBucketByName(String bucketName) {
 try {
 DeleteBucketRequest request = DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();

 getClient().deleteBucket(request);
 System.out.println(bucketName + " was deleted.");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
}

// Get the object lock configuration details for an S3 bucket.
public void getBucketObjectLockConfiguration(String bucketName) {
 GetObjectLockConfigurationRequest objectLockConfigurationRequest =
GetObjectLockConfigurationRequest.builder()
 .bucket(bucketName)
 .build();

 GetObjectLockConfigurationResponse response =
getClient().getObjectLockConfiguration(objectLockConfigurationRequest);
 System.out.println("Bucket object lock config for "+bucketName +": ");
 System.out.println("\tEnabled:
"+response.getObjectLockConfiguration().getObjectLockEnabled());
 System.out.println("\tRule: "+
response.getObjectLockConfiguration().rule().defaultRetention());
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [GetObjectLegalHold](#)
- [GetObjectLockConfiguration](#)
- [GetObjectRetention](#)
- [PutObjectLegalHold](#)
- [PutObjectLockConfiguration](#)

- [PutObjectRetention](#)

## Manage large messages using S3

The following code example shows how to use the Amazon SQS Extended Client Library to work with large Amazon SQS messages.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import com.amazon.sqs.javamessaging.AmazonSQSExtendedClient;
import com.amazon.sqs.javamessaging.ExtendedClientConfiguration;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.joda.time.DateTime;
import org.joda.time.format.DateTimeFormat;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.BucketLifecycleConfiguration;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.ExpirationStatus;
import software.amazon.awssdk.services.s3.model.LifecycleExpiration;
import software.amazon.awssdk.services.s3.model.LifecycleRule;
import software.amazon.awssdk.services.s3.model.LifecycleRuleFilter;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsRequest;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsResponse;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Response;
import
 software.amazon.awssdk.services.s3.model.PutBucketLifecycleConfigurationRequest;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.CreateQueueResponse;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
```

```
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageResponse;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;

import java.util.Arrays;
import java.util.List;
import java.util.UUID;

/**
 * Example of using Amazon SQS Extended Client Library for Java 2.x.
 */
public class SqsExtendedClientExample {
 private static final Logger logger =
 LoggerFactory.getLogger(SqsExtendedClientExample.class);

 private String s3BucketName;
 private String queueUrl;
 private final String queueName;
 private final S3Client s3Client;
 private final SqsClient sqsExtendedClient;
 private final int messageSize;

 /**
 * Constructor with default clients and message size.
 */
 public SqsExtendedClientExample() {
 this(S3Client.create(), 300000);
 }

 /**
 * Constructor with custom S3 client and message size.
 *
 * @param s3Client The S3 client to use
 * @param messageSize The size of the test message to create
 */
 public SqsExtendedClientExample(S3Client s3Client, int messageSize) {
 this.s3Client = s3Client;
 this.messageSize = messageSize;

 // Generate a unique bucket name.
 this.s3BucketName = UUID.randomUUID() + "-" +
 DateTimeFormat.forPattern("yyMMdd-hhmmss").print(new DateTime());
 }
}
```

```

 // Generate a unique queue name.
 this.queueName = "MyQueue-" + UUID.randomUUID();

 // Configure the SQS extended client.
 final ExtendedClientConfiguration extendedClientConfig = new
ExtendedClientConfiguration()
 .withPayloadSupportEnabled(s3Client, s3BucketName);

 this.sqsExtendedClient = new
AmazonSQSExtendedClient(SqsClient.builder().build(), extendedClientConfig);
 }

 public static void main(String[] args) {
 SqsExtendedClientExample example = new SqsExtendedClientExample();
 try {
 example.setup();
 example.sendAndReceiveMessage();
 } finally {
 example.cleanup();
 }
 }

 /**
 * Send a large message and receive it back.
 *
 * @return The received message
 */
 public Message sendAndReceiveMessage() {
 try {
 // Create a large message.
 char[] chars = new char[messageSize];
 Arrays.fill(chars, 'x');
 String largeMessage = new String(chars);

 // Send the message.
 final SendMessageRequest sendMessageRequest =
SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody(largeMessage)
 .build();

 sqsExtendedClient.sendMessage(sendMessageRequest);
 logger.info("Sent message of size: {}", largeMessage.length());
 }
 }

```

```
// Receive and return the message.
final ReceiveMessageResponse receiveMessageResponse =
 sqsExtendedClient.receiveMessage(
 ReceiveMessageRequest.builder().queueUrl(queueUrl).build());

List<Message> messages = receiveMessageResponse.messages();
if (messages.isEmpty()) {
 throw new RuntimeException("No messages received");
}

Message message = messages.getFirst();
logger.info("\nMessage received.");
logger.info(" ID: {}", message.messageId());
logger.info(" Receipt handle: {}", message.receiptHandle());
logger.info(" Message body size: {}", message.body().length());
logger.info(" Message body (first 5 characters): {}",
 message.body().substring(0, 5));

return message;
} catch (RuntimeException e) {
 logger.error("Error during message processing: {}", e.getMessage(), e);
 throw e;
}
}
```

- For more information, see [AWS SDK for Java 2.x Developer Guide](#).
- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateBucket](#)
  - [PutBucketLifecycleConfiguration](#)
  - [ReceiveMessage](#)
  - [SendMessage](#)

## Parse URIs

The following code example shows how to parse Amazon S3 URIs to extract important components like the bucket name and object key.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Parse an Amazon S3 URI by using the [S3Uri](#) class.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.S3Uri;
import software.amazon.awssdk.services.s3.S3Utilities;

import java.net.URI;
import java.util.List;
import java.util.Map;

/**
 *
 * @param s3Client - An S3Client through which you acquire an S3Uri instance.
 * @param s3objectUrl - A complex URL (String) that is used to demonstrate S3Uri
 * capabilities.
 */
public static void parseS3UriExample(S3Client s3Client, String s3objectUrl) {
 logger.info(s3objectUrl);
 // Console output:
 // 'https://s3.us-west-1.amazonaws.com/myBucket/resources/doc.txt?
versionId=abc123&partNumber=77&partNumber=88'.

 // Create an S3Utilities object using the configuration of the s3Client.
 S3Utilities s3Utilities = s3Client.utilities();

 // From a String URL create a URI object to pass to the parseUri() method.
 URI uri = URI.create(s3objectUrl);
 S3Uri s3Uri = s3Utilities.parseUri(uri);

 // If the URI contains no value for the Region, bucket or key, the SDK
returns
 // an empty Optional.
```

```
// The SDK returns decoded URI values.

Region region = s3Uri.region().orElse(null);
log("region", region);
// Console output: 'region: us-west-1'.

String bucket = s3Uri.bucket().orElse(null);
log("bucket", bucket);
// Console output: 'bucket: myBucket'.

String key = s3Uri.key().orElse(null);
log("key", key);
// Console output: 'key: resources/doc.txt'.

Boolean isPathStyle = s3Uri.isPathStyle();
log("isPathStyle", isPathStyle);
// Console output: 'isPathStyle: true'.

// If the URI contains no query parameters, the SDK returns an empty map.
Map<String, List<String>> queryParams = s3Uri.rawQueryParameters();
log("rawQueryParameters", queryParams);
// Console output: 'rawQueryParameters: {versionId=[abc123], partNumber=[77,
// 88]}'.

// Retrieve the first or all values for a query parameter as shown in the
// following code.
String versionId =
s3Uri.firstMatchingRawQueryParameter("versionId").orElse(null);
log("firstMatchingRawQueryParameter-versionId", versionId);
// Console output: 'firstMatchingRawQueryParameter-versionId: abc123'.

String partNumber =
s3Uri.firstMatchingRawQueryParameter("partNumber").orElse(null);
log("firstMatchingRawQueryParameter-partNumber", partNumber);
// Console output: 'firstMatchingRawQueryParameter-partNumber: 77'.

List<String> partNumbers =
s3Uri.firstMatchingRawQueryParameters("partNumber");
log("firstMatchingRawQueryParameter", partNumbers);
// Console output: 'firstMatchingRawQueryParameter: [77, 88]'.

/*
 * Object keys and query parameters with reserved or unsafe characters, must
be
```

```

 * URL-encoded.
 * For example replace whitespace " " with "%20".
 * Valid:
 * "https://s3.us-west-1.amazonaws.com/myBucket/object%20key?query=
%5Bbrackets%5D"
 * Invalid:
 * "https://s3.us-west-1.amazonaws.com/myBucket/object key?query=[brackets]"
 *
 * Virtual-hosted-style URIs with bucket names that contain a dot, ".", the
dot
 * must not be URL-encoded.
 * Valid: "https://my.Bucket.s3.us-west-1.amazonaws.com/key"
 * Invalid: "https://my%2EBucket.s3.us-west-1.amazonaws.com/key"
 */
 }

 private static void log(String s3UriElement, Object element) {
 if (element == null) {
 logger.info("{}: {}", s3UriElement, "null");
 } else {
 logger.info("{}: {}", s3UriElement, element);
 }
 }
}

```

## Process S3 event notifications

The following code example shows how to work with S3 event notifications in an object-oriented way.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

This example show how to process S3 notification event by using Amazon SQS.

```

/**
 * This method receives S3 event notifications by using an SqsAsyncClient.

```

```

 * After the client receives the messages it deserializes the JSON payload and
 logs them. It uses
 * the S3EventNotification class (part of the S3 event notification API for
 Java) to deserialize
 * the JSON payload and access the messages in an object-oriented way.
 *
 * @param queueUrl The URL of the AWS SQS queue that receives the S3 event
 notifications.
 * @see <a href="https://sdk.amazonaws.com/java/api/latest/software/amazon/
 awssdk/eventnotifications/s3/model/package-summary.html">S3EventNotification API</
 a>.
 * <p>
 * To use S3 event notification serialization/deserialization to objects, add
 the following
 * dependency to your Maven pom.xml file.
 * <dependency>
 * <groupId>software.amazon.awssdk</groupId>
 * <artifactId>s3-event-notifications</artifactId>
 * <version><LATEST></version>
 * </dependency>
 * <p>
 * The S3 event notification API became available with version 2.25.11 of the
 Java SDK.
 * <p>
 * This example shows the use of the API with AWS SQS, but it can be used to
 process S3 event notifications
 * in AWS SNS or AWS Lambda as well.
 * <p>
 * Note: The S3EventNotification class does not work with messages routed
 through AWS EventBridge.
 */
 static void processS3Events(String bucketName, String queueUrl, String queueArn)
 {
 try {
 // Configure the bucket to send Object Created and Object Tagging
 notifications to an existing SQS queue.
 s3Client.putBucketNotificationConfiguration(b -> b
 .notificationConfiguration(ncb -> ncb
 .queueConfigurations(qcb -> qcb
 .events(Event.S3_OBJECT_CREATED,
 Event.S3_OBJECT_TAGGING)
 .queueArn(queueArn)))
 .bucket(bucketName)
).join();
 }
 }

```

```

 triggerS3EventNotifications(bucketName);
 // Wait for event notifications to propagate.
 Thread.sleep(Duration.ofSeconds(5).toMillis());

 boolean didReceiveMessages = true;
 while (didReceiveMessages) {
 // Display the number of messages that are available in the queue.
 sqsClient.getQueueAttributes(b -> b
 .queueUrl(queueUrl)

 .attributeNames(QueueAttributeName.APPROXIMATE_NUMBER_OF_MESSAGES)
).thenAccept(attributeResponse ->
 logger.info("Approximate number of messages in the
queue: {}",

attributeResponse.attributes().get(QueueAttributeName.APPROXIMATE_NUMBER_OF_MESSAGES)))
 .join();

 // Receive the messages.
 ReceiveMessageResponse response = sqsClient.receiveMessage(b -> b
 .queueUrl(queueUrl)
).get();
 logger.info("Count of received messages: {}",
response.messages().size());
 didReceiveMessages = !response.messages().isEmpty();

 // Create a collection to hold the received message for deletion
 // after we log the messages.
 HashSet<DeleteMessageBatchRequestEntry> messagesToDelete = new
HashSet<>();

 // Process each message.
 response.messages().forEach(message -> {
 logger.info("Message id: {}", message.messageId());
 // Deserialize JSON message body to a S3EventNotification object
 // to access messages in an object-oriented way.
 S3EventNotification event =
S3EventNotification.fromJson(message.body());

 // Log the S3 event notification record details.
 if (event.getRecords() != null) {
 event.getRecords().forEach(record -> {
 String eventName = record.getEventName();
 String key = record.getS3().getObject().getKey();

```

```

 logger.info(record.toString());
 logger.info("Event name is {} and key is {}", eventName,
key);
 });
}
// Add logged messages to collection for batch deletion.
messagesToDelete.add(DeleteMessageBatchRequestEntry.builder()
 .id(message.messageId())
 .receiptHandle(message.receiptHandle())
 .build());
});
// Delete messages.
if (!messagesToDelete.isEmpty()) {
 sqsClient.deleteMessageBatch(DeleteMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(messagesToDelete)
 .build()
).join();
}
} // End of while block.
} catch (InterruptedException | ExecutionException e) {
 throw new RuntimeException(e);
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteMessageBatch](#)
  - [GetQueueAttributes](#)
  - [PutBucketNotificationConfiguration](#)
  - [ReceiveMessage](#)

## Send event notifications to EventBridge

The following code example shows how to enable a bucket to send S3 event notifications to EventBridge and route notifications to an Amazon SNS topic and Amazon SQS queue.

## SDK for Java 2.x

 **Note**

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/** This method configures a bucket to send events to AWS EventBridge and
creates a rule
 * to route the S3 object created events to a topic and a queue.
 *
 * @param bucketName Name of existing bucket
 * @param topicArn ARN of existing topic to receive S3 event notifications
 * @param queueArn ARN of existing queue to receive S3 event notifications
 *
 * An AWS CloudFormation stack sets up the bucket, queue, topic before the
method runs.
 */
public static String setBucketNotificationToEventBridge(String bucketName,
String topicArn, String queueArn) {
 try {
 // Enable bucket to emit S3 Event notifications to EventBridge.
 s3Client.putBucketNotificationConfiguration(b -> b
 .bucket(bucketName)
 .notificationConfiguration(b1 -> b1
 .eventBridgeConfiguration(
 SdkBuilder::build)
).build()).join();

 // Create an EventBridge rule to route Object Created notifications.
 PutRuleRequest putRuleRequest = PutRuleRequest.builder()
 .name(RULE_NAME)
 .eventPattern(""""
 {
 "source": ["aws.s3"],
 "detail-type": ["Object Created"],
 "detail": {
 "bucket": {
 "name": ["%s"]
 }
 }
 }
 """)
 }
}

```

```

 }
 """).formatted(bucketName))
 .build();

 // Add the rule to the default event bus.
 PutRuleResponse putRuleResponse =
eventBridgeClient.putRule(putRuleRequest)
 .whenComplete((r, t) -> {
 if (t != null) {
 logger.error("Error creating event bus rule: " +
t.getMessage(), t);
 throw new RuntimeException(t.getCause().getMessage(),
t);
 }
 logger.info("Event bus rule creation request sent
successfully. ARN is: {}", r.ruleArn());
 }).join();

 // Add the existing SNS topic and SQS queue as targets to the rule.
 eventBridgeClient.putTargets(b -> b
 .eventBusName("default")
 .rule(RULE_NAME)
 .targets(List.of (
 Target.builder()
 .arn(queueArn)
 .id("Queue")
 .build(),
 Target.builder()
 .arn(topicArn)
 .id("Topic")
 .build()
)
).join();
 return putRuleResponse.ruleArn();
 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.



- [PutBucketNotificationConfiguration](#)
- [PutRule](#)
- [PutTargets](#)

## Track uploads and downloads

The following code example shows how to track an Amazon S3 object upload or download.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Track the progress of a file upload.

```
public void trackUploadFile(S3TransferManager transferManager, String
bucketName,
 String key, URI filePathURI) {
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b.bucket(bucketName).key(key))
 .addTransferListener(LoggingTransferListener.create()) // Add
listener.
 .source(Paths.get(filePathURI))
 .build();

 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);

 fileUpload.completionFuture().join();
 /*
 The SDK provides a LoggingTransferListener implementation of the
 TransferListener interface.
 You can also implement the interface to provide your own logic.

 Configure log4J2 with settings such as the following.
 <Configuration status="WARN">
 <Appenders>
 <Console name="AlignedConsoleAppender" target="SYSTEM_OUT">
 <PatternLayout pattern="%m%n"/>

```

```

 </Console>
 </Appenders>

 <Loggers>
 <logger
name="software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener"
level="INFO" additivity="false">
 <AppenderRef ref="AlignedConsoleAppender"/>
 </logger>
 </Loggers>
</Configuration>

```

Log4J2 logs the progress. The following is example output for a 21.3 MB file upload.

```

 Transfer initiated...
 | | 0.0%
 |==== | 21.1%
 |===== | 60.5%
 |===== | 100.0%
 Transfer complete!
*/
}

```

## Track the progress of a file download.

```

 public void trackDownloadFile(S3TransferManager transferManager, String
bucketName,

 String key, String downloadedFilePath) {
 DownloadFileRequest downloadFileRequest = DownloadFileRequest.builder()
 .getObjectRequest(b -> b.bucket(bucketName).key(key))
 .addTransferListener(LoggingTransferListener.create()) // Add
listener.

 .destination(Paths.get(downloadedFilePath))
 .build();

 FileDownload downloadFile =
transferManager.downloadFile(downloadFileRequest);

 CompletedFileDownload downloadResult =
downloadFile.completionFuture().join();
 /*

```

The SDK provides a `LoggingTransferListener` implementation of the `TransferListener` interface.

You can also implement the interface to provide your own logic.

Configure log4j2 with settings such as the following.

```
<Configuration status="WARN">
 <Appenders>
 <Console name="AlignedConsoleAppender" target="SYSTEM_OUT">
 <PatternLayout pattern="%m%n"/>
 </Console>
 </Appenders>

 <Loggers>
 <logger
name="software.amazon.awssdk.transfer.s3.progress.LoggingTransferListener"
level="INFO" additivity="false">
 <AppenderRef ref="AlignedConsoleAppender"/>
 </logger>
 </Loggers>
</Configuration>
```

Log4j2 logs the progress. The following is example output for a 21.3 MB file download.

```
Transfer initiated...
|=====| 39.4%
|=====| 78.8%
|=====| 100.0%
Transfer complete!

 */
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [GetObject](#)
  - [PutObject](#)

## Upload directory to a bucket

The following code example shows how to upload a local directory recursively to an Amazon Simple Storage Service (Amazon S3) bucket.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use an [S3TransferManager](#) to [upload a local directory](#). View the [complete file](#) and [test](#).

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedDirectoryUpload;
import software.amazon.awssdk.transfer.s3.model.DirectoryUpload;
import software.amazon.awssdk.transfer.s3.model.UploadDirectoryRequest;

import java.net.URI;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.file.Paths;

 public Integer uploadDirectory(S3TransferManager transferManager,
 URI sourceDirectory, String bucketName) {
 DirectoryUpload directoryUpload =
transferManager.uploadDirectory(UploadDirectoryRequest.builder()
 .source(Paths.get(sourceDirectory))
 .bucket(bucketName)
 .build());

 CompletedDirectoryUpload completedDirectoryUpload =
directoryUpload.completionFuture().join();
 completedDirectoryUpload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryUpload.failedTransfers().size();
 }
```

- For API details, see [UploadDirectory](#) in *AWS SDK for Java 2.x API Reference*.

## Upload or download large files

The following code example shows how to upload or download large files to and from Amazon S3.

For more information, see [Uploading an object using multipart upload](#).

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Call functions that transfer files to and from an S3 bucket using the `S3TransferManager`.

```
public Integer downloadObjectsToDirectory(S3TransferManager transferManager,
 URI destinationPathURI, String bucketName) {
 DirectoryDownload directoryDownload =
transferManager.downloadDirectory(DownloadDirectoryRequest.builder()
 .destination(Paths.get(destinationPathURI))
 .bucket(bucketName)
 .build());
 CompletedDirectoryDownload completedDirectoryDownload =
directoryDownload.completionFuture().join();

 completedDirectoryDownload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryDownload.failedTransfers().size();
}
```

Upload an entire local directory.

```
public Integer uploadDirectory(S3TransferManager transferManager,
 URI sourceDirectory, String bucketName) {
 DirectoryUpload directoryUpload =
transferManager.uploadDirectory(UploadDirectoryRequest.builder()
 .source(Paths.get(sourceDirectory))
 .bucket(bucketName)
 .build());
}
```

```

 CompletedDirectoryUpload completedDirectoryUpload =
directoryUpload.completionFuture().join();
 completedDirectoryUpload.failedTransfers()
 .forEach(fail -> logger.warn("Object [{}] failed to transfer",
fail.toString()));
 return completedDirectoryUpload.failedTransfers().size();
 }

```

## Upload a single file.

```

public String uploadFile(S3TransferManager transferManager, String bucketName,
 String key, URI filePathURI) {
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b.bucket(bucketName).key(key))
 .source(Paths.get(filePathURI))
 .build();

 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);

 CompletedFileUpload uploadResult = fileUpload.completionFuture().join();
 return uploadResult.response().eTag();
}

```

## The code examples use the following imports.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CompletedMultipartUpload;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;
import software.amazon.awssdk.services.s3.model.UploadPartRequest;
import software.amazon.awssdk.services.s3.model.UploadPartResponse;
import software.amazon.awssdk.services.s3.waiters.S3Waiter;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.FileUpload;

```

```
import software.amazon.awssdk.transfer.s3.model.UploadFileRequest;

import java.io.IOException;
import java.io.RandomAccessFile;
import java.net.URISyntaxException;
import java.net.URL;
import java.nio.ByteBuffer;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
```

Use the [S3 Transfer Manager](#) on top of the [AWS CRT-based S3 client](#) to transparently perform a multipart upload when the size of the content exceeds a threshold. The default threshold size is 8 MB.

```
/**
 * Uploads a file to an Amazon S3 bucket using the S3TransferManager.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the object key
 * @param filePath the file path of the file to be uploaded
 */
public void multipartUploadWithTransferManager(String bucketName, String key,
String filePath) {
 S3TransferManager transferManager = S3TransferManager.create();
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b
 .bucket(bucketName)
 .key(key))
 .source(Paths.get(filePath))
 .build();
 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);
 fileUpload.completionFuture().join();
 transferManager.close();
}
```

Use the [S3Client API](#) to perform a multipart upload.

```

/**
 * Performs a multipart upload to Amazon S3 using the provided S3 client.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the object key
 * @param filePath the path to the file to be uploaded
 */
public void multipartUploadWithS3Client(String bucketName, String key, String
filePath) {

 // Initiate the multipart upload.
 CreateMultipartUploadResponse createMultipartUploadResponse =
s3Client.createMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key));
 String uploadId = createMultipartUploadResponse.uploadId();

 // Upload the parts of the file.
 int partNumber = 1;
 List<CompletedPart> completedParts = new ArrayList<>();
 ByteBuffer bb = ByteBuffer.allocate(1024 * 1024 * 5); // 5 MB byte buffer

 try (RandomAccessFile file = new RandomAccessFile(filePath, "r")) {
 long fileSize = file.length();
 long position = 0;
 while (position < fileSize) {
 file.seek(position);
 long read = file.getChannel().read(bb);

 bb.flip(); // Swap position and limit before reading from the
buffer.

 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .partNumber(partNumber)
 .build();

 UploadPartResponse partResponse = s3Client.uploadPart(
 uploadPartRequest,
 RequestBody.fromByteBuffer(bb));

 CompletedPart part = CompletedPart.builder()

```



```

 .partNumber(partNumber)
 .eTag(partResponse.eTag())
 .build();
 completedParts.add(part);

 bb.clear();
 position += read;
 partNumber++;
 }
} catch (IOException e) {
 logger.error(e.getMessage());
}

// Complete the multipart upload.
s3Client.completeMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)

.multipartUpload(CompletedMultipartUpload.builder().parts(completedParts).build()));
}

```

Use the [S3AsyncClient API](#) with multipart support enabled to perform a multipart upload.

```

/**
 * Uploads a file to an S3 bucket using the S3AsyncClient and enabling multipart
 * support.
 *
 * @param bucketName the name of the S3 bucket
 * @param key the object key
 * @param filePath the local file path of the file to be uploaded
 */
public void multipartUploadWithS3AsyncClient(String bucketName, String key,
String filePath) {
 // Enable multipart support.
 S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
 .multipartEnabled(true)
 .build();

 CompletableFuture<PutObjectResponse> response = s3AsyncClient.putObject(b ->
b

```

```
 .bucket(bucketName)
 .key(key),
 Paths.get(filePath));

 response.join();
 logger.info("File uploaded in multiple 8 MiB parts using S3AsyncClient.");
}
```

## Upload stream of unknown size

The following code example shows how to upload a stream of unknown size to an Amazon S3 object.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Use the [AWS CRT-based S3 Client](#).

```
import com.example.s3.util.AsyncExampleUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;

import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PutObjectFromStreamAsync {
```

```

 private static final Logger logger =
 LoggerFactory.getLogger(PutObjectFromStreamAsync.class);

 public static void main(String[] args) {
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 String key = UUID.randomUUID().toString();

 AsyncExampleUtils.createBucket(bucketName);
 try {
 PutObjectFromStreamAsync example = new PutObjectFromStreamAsync();
 S3AsyncClient s3AsyncClientCrt = S3AsyncClient.crtCreate();
 PutObjectResponse putObjectResponse =
example.putObjectFromStreamCrt(s3AsyncClientCrt, bucketName, key);
 logger.info("Object {} etag: {}", key, putObjectResponse.eTag());
 logger.info("Object {} uploaded to bucket {}. ", key, bucketName);
 } catch (SdkException e) {
 logger.error(e.getMessage(), e);
 } finally {
 AsyncExampleUtils.deleteObject(bucketName, key);
 AsyncExampleUtils.deleteBucket(bucketName);
 }
 }

 /**
 * @param s3CrtAsyncClient - To upload content from a stream of unknown size,
 use can the AWS CRT-based S3 client.
 * @param bucketName - The name of the bucket.
 * @param key - The name of the object.
 * @return software.amazon.awssdk.services.s3.model.PutObjectResponse - Returns
 metadata pertaining to the put object operation.
 */
 public PutObjectResponse putObjectFromStreamCrt(S3AsyncClient s3CrtAsyncClient,
 String bucketName, String key) {

 // AsyncExampleUtils.randomString() returns a random string up to 100
 characters.
 String randomString = AsyncExampleUtils.randomString();
 logger.info("random string to upload: {}: length={}", randomString,
 randomString.length());
 InputStream inputStream = new ByteArrayInputStream(randomString.getBytes());

 // Executor required to handle reading from the InputStream on a separate
 thread so the main upload is not blocked.
 ExecutorService executor = Executors.newSingleThreadExecutor();

```

```

 // Specify `null` for the content length when you don't know the content
length.
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
executor);

 CompletableFuture<PutObjectResponse> responseFuture =
 s3CrtAsyncClient.putObject(r -> r.bucket(bucketName).key(key),
body);

 PutObjectResponse response = responseFuture.join(); // Wait for the
response.
 logger.info("Object {} uploaded to bucket {}. ", key, bucketName);
 executor.shutdown();
 return response;
 }
}

```

Use the standard [asynchronous S3 client with multipart upload enabled](#).

```

import com.example.s3.util.AsyncExampleUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.services.s3.S3AsyncClient;
import software.amazon.awssdk.services.s3.model.PutObjectResponse;

import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PutObjectFromStreamAsyncMp {
 private static final Logger logger =
 LoggerFactory.getLogger(PutObjectFromStreamAsyncMp.class);

 public static void main(String[] args) {
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 String key = UUID.randomUUID().toString();
 }
}

```

```

 AsyncExampleUtils.createBucket(bucketName);
 try {
 PutObjectFromStreamAsyncMp example = new PutObjectFromStreamAsyncMp();
 S3AsyncClient s3AsyncClientMp =
S3AsyncClient.builder().multipartEnabled(true).build();
 PutObjectResponse putObjectResponse =
example.putObjectFromStreamMp(s3AsyncClientMp, bucketName, key);
 logger.info("Object {} etag: {}", key, putObjectResponse.eTag());
 logger.info("Object {} uploaded to bucket {}. ", key, bucketName);
 } catch (SdkException e) {
 logger.error(e.getMessage(), e);
 } finally {
 AsyncExampleUtils.deleteObject(bucketName, key);
 AsyncExampleUtils.deleteBucket(bucketName);
 }
}

/**
 * @param s3AsyncClientMp - To upload content from a stream of unknown size, use
can the S3 asynchronous client with multipart enabled.
 * @param bucketName - The name of the bucket.
 * @param key - The name of the object.
 * @return software.amazon.awssdk.services.s3.model.PutObjectResponse - Returns
metadata pertaining to the put object operation.
 */
public PutObjectResponse putObjectFromStreamMp(S3AsyncClient s3AsyncClientMp,
String bucketName, String key) {

 // AsyncExampleUtils.randomString() returns a random string up to 100
characters.
 String randomString = AsyncExampleUtils.randomString();
 logger.info("random string to upload: {}: length={}", randomString,
randomString.length());
 InputStream inputStream = new ByteArrayInputStream(randomString.getBytes());

 // Executor required to handle reading from the InputStream on a separate
thread so the main upload is not blocked.
 ExecutorService executor = Executors.newSingleThreadExecutor();
 // Specify `null` for the content length when you don't know the content
length.
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
executor);

```

```

 CompletableFuture<PutObjectResponse> responseFuture =
 s3AsyncClientMp.putObject(r -> r.bucket(bucketName).key(key), body);

 PutObjectResponse response = responseFuture.join(); // Wait for the
response.
 logger.info("Object {} uploaded to bucket {}.\"", key, bucketName);
 executor.shutdown();
 return response;
 }
}

```

Use the [Amazon S3 Transfer Manager](#).

```

import com.example.s3.util.AsyncExampleUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.async.AsyncRequestBody;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.CompletedUpload;
import software.amazon.awssdk.transfer.s3.model.Upload;

import java.io.ByteArrayInputStream;
import java.io.InputStream;
import java.util.UUID;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class UploadStream {
 private static final Logger logger =
 LoggerFactory.getLogger(UploadStream.class);

 public static void main(String[] args) {
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 String key = UUID.randomUUID().toString();

 AsyncExampleUtils.createBucket(bucketName);
 try {
 UploadStream example = new UploadStream();
 CompletedUpload completedUpload =
 example.uploadStream(S3TransferManager.create(), bucketName, key);

```

```

 logger.info("Object {} etag: {}", key,
completedUpload.response().eTag());
 logger.info("Object {} uploaded to bucket {}. ", key, bucketName);
 } catch (SdkException e) {
 logger.error(e.getMessage(), e);
 } finally {
 AsyncExampleUtils.deleteObject(bucketName, key);
 AsyncExampleUtils.deleteBucket(bucketName);
 }
}

/**
 * @param transferManager - To upload content from a stream of unknown size, you
can use the S3TransferManager based on the AWS CRT-based S3 client.
 * @param bucketName - The name of the bucket.
 * @param key - The name of the object.
 * @return - software.amazon.awssdk.transfer.s3.model.CompletedUpload - The
result of the completed upload.
 */
public CompletedUpload uploadStream(S3TransferManager transferManager, String
bucketName, String key) {

 // AsyncExampleUtils.randomString() returns a random string up to 100
characters.
 String randomString = AsyncExampleUtils.randomString();
 logger.info("random string to upload: {}: length={}", randomString,
randomString.length());
 InputStream inputStream = new ByteArrayInputStream(randomString.getBytes());

 // Executor required to handle reading from the InputStream on a separate
thread so the main upload is not blocked.
 ExecutorService executor = Executors.newSingleThreadExecutor();
 // Specify `null` for the content length when you don't know the content
length.
 AsyncRequestBody body = AsyncRequestBody.fromInputStream(inputStream, null,
executor);

 Upload upload = transferManager.upload(builder -> builder
 .requestBody(body)
 .putObjectRequest(req -> req.bucket(bucketName).key(key))
 .build());

 CompletedUpload completedUpload = upload.completionFuture().join();
 executor.shutdown();
}

```

```
 return completedUpload;
 }
}
```

## Use checksums

The following code example shows how to use checksums to work with an Amazon S3 object.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The code examples use a subset of the following imports.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ChecksumAlgorithm;
import software.amazon.awssdk.services.s3.model.ChecksumMode;
import software.amazon.awssdk.services.s3.model.CompletedMultipartUpload;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import software.amazon.awssdk.services.s3.model.UploadPartRequest;
import software.amazon.awssdk.services.s3.model.UploadPartResponse;
import software.amazon.awssdk.services.s3.waiters.S3Waiter;
import software.amazon.awssdk.transfer.s3.S3TransferManager;
import software.amazon.awssdk.transfer.s3.model.FileUpload;
import software.amazon.awssdk.transfer.s3.model.UploadFileRequest;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.RandomAccessFile;
import java.net.URISyntaxException;
```



```
import java.net.URL;
import java.nio.ByteBuffer;
import java.nio.file.Paths;
import java.security.DigestInputStream;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.ArrayList;
import java.util.Base64;
import java.util.List;
import java.util.Objects;
import java.util.UUID;
```

Specify a checksum algorithm for the `putObject` method when you [build the PutObjectRequest](#).

```
public void putObjectWithChecksum(String bucketName, String key) {
 s3Client.putObject(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(ChecksumAlgorithm.CRC32),
 RequestBody.fromString("This is a test"));
}
```

Verify the checksum for the `getObject` method when you [build the GetObjectRequest](#).

```
public GetObjectResponse getObjectWithChecksum(String bucketName, String key) {
 return s3Client.getObject(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumMode(ChecksumMode.ENABLED))
 .response();
}
```

Pre-calculate a checksum for the `putObject` method when you [build the PutObjectRequest](#).

```
public void putObjectWithPrecalculatedChecksum(String bucketName, String key,
 String filePath) {
 String checksum = calculateChecksum(filePath, "SHA-256");
```

```
s3Client.putObject((b -> b
 .bucket(bucketName)
 .key(key)
 .checksumSHA256(checksum)),
 RequestBody.fromFile(Paths.get(filePath)));
}
```

Use the [S3 Transfer Manager](#) on top of the [AWS CRT-based S3 client](#) to transparently perform a multipart upload when the size of the content exceeds a threshold. The default threshold size is 8 MB.

You can specify a checksum algorithm for the SDK to use. By default, the SDK uses the CRC32 algorithm.

```
public void multipartUploadWithChecksumTm(String bucketName, String key, String
filePath) {
 S3TransferManager transferManager = S3TransferManager.create();
 UploadFileRequest uploadFileRequest = UploadFileRequest.builder()
 .putObjectRequest(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(ChecksumAlgorithm.SHA1))
 .source(Paths.get(filePath))
 .build();
 FileUpload fileUpload = transferManager.uploadFile(uploadFileRequest);
 fileUpload.completionFuture().join();
 transferManager.close();
}
```

Use the [S3Client API](#) or (S3AsyncClient API) to perform a multipart upload. If you specify an additional checksum, you must specify the algorithm to use on the initiation of the upload. You must also specify the algorithm for each part request and provide the checksum calculated for each part after it is uploaded.

```
public void multipartUploadWithChecksumS3Client(String bucketName, String key,
String filePath) {
 ChecksumAlgorithm algorithm = ChecksumAlgorithm.CRC32;

 // Initiate the multipart upload.
```

```

 CreateMultipartUploadResponse createMultipartUploadResponse =
s3Client.createMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .checksumAlgorithm(algorithm)); // Checksum specified on initiation.
String uploadId = createMultipartUploadResponse.uploadId();

// Upload the parts of the file.
int partNumber = 1;
List<CompletedPart> completedParts = new ArrayList<>();
ByteBuffer bb = ByteBuffer.allocate(1024 * 1024 * 5); // 5 MB byte buffer

try (RandomAccessFile file = new RandomAccessFile(filePath, "r")) {
 long fileSize = file.length();
 long position = 0;
 while (position < fileSize) {
 file.seek(position);
 long read = file.getChannel().read(bb);

 bb.flip(); // Swap position and limit before reading from the
buffer.

 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)
 .checksumAlgorithm(algorithm) // Checksum specified on each
part.

 .partNumber(partNumber)
 .build();

 UploadPartResponse partResponse = s3Client.uploadPart(
 uploadPartRequest,
 RequestBody.fromByteBuffer(bb));

 CompletedPart part = CompletedPart.builder()
 .partNumber(partNumber)
 .checksumCRC32(partResponse.checksumCRC32()) // Provide the
calculated checksum.
 .eTag(partResponse.eTag())
 .build();
 completedParts.add(part);

 bb.clear();
 position += read;
 }
}

```

```
 partNumber++;
 }
} catch (IOException e) {
 System.err.println(e.getMessage());
}

// Complete the multipart upload.
s3Client.completeMultipartUpload(b -> b
 .bucket(bucketName)
 .key(key)
 .uploadId(uploadId)

.multipartUpload(CompletedMultipartUpload.builder().parts(completedParts).build()));
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CompleteMultipartUpload](#)
  - [CreateMultipartUpload](#)
  - [UploadPart](#)

## Serverless examples

### Invoke a Lambda function from an Amazon S3 trigger

The following code example shows how to implement a Lambda function that receives an event triggered by uploading an object to an S3 bucket. The function retrieves the S3 bucket name and object key from the event parameter and calls the Amazon S3 API to retrieve and log the content type of the object.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming an S3 event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.S3Client;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.S3Event;
import
 com.amazonaws.services.lambda.runtime.events.models.s3.S3EventNotification.S3EventNotificationRecord;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Handler implements RequestHandler<S3Event, String> {
 private static final Logger logger = LoggerFactory.getLogger(Handler.class);
 @Override
 public String handleRequest(S3Event s3event, Context context) {
 try {
 S3EventNotificationRecord record = s3event.getRecords().get(0);
 String srcBucket = record.getS3().getBucket().getName();
 String srcKey = record.getS3().getObject().getUrlDecodedKey();

 S3Client s3Client = S3Client.builder().build();
 HeadObjectResponse headObject = getHeadObject(s3Client, srcBucket,
srcKey);

 logger.info("Successfully retrieved " + srcBucket + "/" + srcKey + " of
type " + headObject.contentType());

 return "Ok";
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
 }

 private HeadObjectResponse getHeadObject(S3Client s3Client, String bucket,
String key) {
 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(bucket)

```

```
 .key(key)
 .build();
 return s3Client.headObject(headObjectRequest);
}
}
```

## Amazon S3 Control examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon S3 Control.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Amazon S3 Control

The following code example shows how to get started using Amazon S3 Control.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.core.retry.RetryMode;
import software.amazon.awssdk.core.retry.RetryPolicy;
import software.amazon.awssdk.http.async.SdkAsyncHttpClient;
import software.amazon.awssdk.http.nio.netty.NettyNioAsyncHttpClient;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3control.S3ControlAsyncClient;
import software.amazon.awssdk.services.s3control.model.JobListDescriptor;
import software.amazon.awssdk.services.s3control.model.JobStatus;
import software.amazon.awssdk.services.s3control.model.ListJobsRequest;
import software.amazon.awssdk.services.s3control.paginators.ListJobsPublisher;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;

/**
 * Before running this example:
 * <p/>
 * The SDK must be able to authenticate AWS requests on your behalf. If you have not
 * configured
 * authentication for SDKs and tools, see https://docs.aws.amazon.com/sdkref/latest/guide/access.html in the AWS SDKs and Tools Reference Guide.
 * <p/>
 * You must have a runtime environment configured with the Java SDK.
 * See https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/setup.html in
 * the Developer Guide if this is not set up.
 */
public class HelloS3Batch {
 private static S3ControlAsyncClient asyncClient;

 public static void main(String[] args) {
 S3BatchActions actions = new S3BatchActions();
 String accountId = actions.getAccountId();
 try {
 listBatchJobsAsync(accountId)
 .exceptionally(ex -> {
 System.err.println("List batch jobs failed: " +
ex.getMessage());
 return null;
 })
 }
 }
}

```

```

 .join();

 } catch (CompletionException ex) {
 System.err.println("Failed to list batch jobs: " + ex.getMessage());
 }
}

/**
 * Retrieves the asynchronous S3 Control client instance.
 * <p>
 * This method creates and returns a singleton instance of the {@link
S3ControlAsyncClient}. If the instance
 * has not been created yet, it will be initialized with the following
configuration:
 *
 * Maximum concurrency: 100
 * Connection timeout: 60 seconds
 * Read timeout: 60 seconds
 * Write timeout: 60 seconds
 * API call timeout: 2 minutes
 * API call attempt timeout: 90 seconds
 * Retry policy: 3 retries
 * Region: US_EAST_1
 * Credentials provider: {@link EnvironmentVariableCredentialsProvider}</
li>
 *
 *
 * @return the asynchronous S3 Control client instance
 */
private static S3ControlAsyncClient getAsyncClient() {
 if (asyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();
 }
}

```



```

 asyncClient = S3ControlAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return asyncClient;
}

/**
 * Asynchronously lists batch jobs that have completed for the specified
 * account.
 *
 * @param accountId the ID of the account to list jobs for
 * @return a CompletableFuture that completes when the job listing operation is
 * finished
 */
public static CompletableFuture<Void> listBatchJobsAsync(String accountId) {
 ListJobsRequest jobsRequest = ListJobsRequest.builder()
 .jobStatuses(JobStatus.COMPLETE)
 .accountId(accountId)
 .maxResults(10)
 .build();

 ListJobsPublisher publisher =
getAsyncClient().listJobsPaginator(jobsRequest);
 return publisher.subscribe(response -> {
 List<JobListDescriptor> jobs = response.jobs();
 for (JobListDescriptor job : jobs) {
 System.out.println("The job id is " + job.jobId());
 System.out.println("The job priority is " + job.priority());
 }
 }).thenAccept(response -> {
 System.out.println("Listing batch jobs completed");
 }).exceptionally(ex -> {
 System.err.println("Failed to list batch jobs: " + ex.getMessage());
 throw new RuntimeException(ex);
 });
}

```

- For API details, see [ListJobs](#) in *AWS SDK for Java 2.x API Reference*.

# Basics

## Learn the basics

The following code example shows how to learn core operations for Amazon S3 Control.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Learn core operations.

```
package com.example.s3.batch;

import software.amazon.awssdk.services.s3.model.S3Exception;
import java.io.IOException;
import java.util.Map;
import java.util.Scanner;
import java.util.UUID;
import java.util.concurrent.CompletionException;

public class S3BatchScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final String STACK_NAME = "MyS3Stack";
 public static void main(String[] args) throws IOException {
 S3BatchActions actions = new S3BatchActions();
 String accountId = actions.getAccountId();
 String uuid = java.util.UUID.randomUUID().toString();
 Scanner scanner = new Scanner(System.in);

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon S3 Batch basics scenario.");
 System.out.println("""
 S3 Batch operations enables efficient and cost-effective processing of
 large-scale
 data stored in Amazon S3. It automatically scales resources to handle
 varying workloads
 without the need for manual intervention.
 """);
 }
}
```

One of the key features of S3 Batch is its ability to perform tagging operations on objects stored in S3 buckets. Users can leverage S3 Batch to apply, update, or remove tags on thousands or millions of objects in a single operation, streamlining the management and organization of their data.

This can be particularly useful for tasks such as cost allocation, lifecycle management, or metadata-driven workflows, where consistent and accurate tagging is essential.

S3 Batch's scalability and serverless nature make it an ideal solution for organizations with growing data volumes and complex data management requirements.

This Java program walks you through Amazon S3 Batch operations.

Let's get started...

```

 "");
 waitForInputToContinue(scanner);
 // Use CloudFormation to stand up the resource required for this scenario.
 System.out.println("Use CloudFormation to stand up the resource required for
this scenario.");
 CloudFormationHelper.deployCloudFormationStack(STACK_NAME);

 Map<String, String> stackOutputs =
CloudFormationHelper.getStackOutputs(STACK_NAME);
 String iamRoleArn = stackOutputs.get("S3BatchRoleArn");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Setup the required bucket for this scenario.");
 waitForInputToContinue(scanner);
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 actions.createBucket(bucketName);
 String reportBucketName = "arn:aws:s3:::"+bucketName;
 String manifestLocation = "arn:aws:s3:::"+bucketName+"/job-manifest.csv";
 System.out.println("Populate the bucket with the required files.");
 String[] fileNames = {"job-manifest.csv", "object-key-1.txt", "object-
key-2.txt", "object-key-3.txt", "object-key-4.txt"};
 actions.uploadFilesToBucket(bucketName, fileNames, actions);
 waitForInputToContinue(scanner);

```

```

 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("1. Create a S3 Batch Job");
 System.out.println("This job tags all objects listed in the manifest file
with tags");
 waitForInputToContinue(scanner);
 String jobId ;
 try {
 jobId = actions.createS3JobAsync(accountId, iamRoleArn,
manifestLocation, reportBucketName, uuid).join();
 System.out.println("The Job id is " + jobId);

 } catch (S3Exception e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }

 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Update an existing S3 Batch Operations job's
priority");
 System.out.println("""
 In this step, we modify the job priority value. The higher the number,
the higher the priority.
 So, a job with a priority of `30` would have a higher priority than a
job with
 a priority of `20`. This is a common way to represent the priority of a
task
 or job, with higher numbers indicating a higher priority.

 Ensure that the job status allows for priority updates. Jobs in
certain
 states (e.g., Cancelled, Failed, or Completed) cannot have their
priorities
 updated. Only jobs in the Active or Suspended state typically allow
priority
 updates.
 """);

```

```
try {
 actions.updateJobPriorityAsync(jobId, accountId)
 .exceptionally(ex -> {
 System.err.println("Update job priority failed: " +
ex.getMessage());
 return null;
 })
 .join();
} catch (CompletionException ex) {
 System.err.println("Failed to update job priority: " + ex.getMessage());
}
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Cancel the S3 Batch job");
System.out.print("Do you want to cancel the Batch job? (y/n): ");
String cancelAns = scanner.nextLine();
if (cancelAns != null && cancelAns.trim().equalsIgnoreCase("y")) {
 try {
 actions.cancelJobAsync(jobId, accountId)
 .exceptionally(ex -> {
 System.err.println("Cancel job failed: " + ex.getMessage());
 return null;
 })
 .join();
 } catch (CompletionException ex) {
 System.err.println("Failed to cancel job: " + ex.getMessage());
 }
} else {
 System.out.println("Job " + jobId + " was not canceled.");
}
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Describe the job that was just created");
waitForInputToContinue(scanner);
try {
 actions.describeJobAsync(jobId, accountId)
 .exceptionally(ex -> {
 System.err.println("Describe job failed: " + ex.getMessage());
 return null;
 })
 .join();
}
```

```

 .join();
 } catch (CompletionException ex) {
 System.err.println("Failed to describe job: " + ex.getMessage());
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Describe the tags associated with the job");
 waitForInputToContinue(scanner);
 try {
 actions.getJobTagsAsync(jobId, accountId)
 .exceptionally(ex -> {
 System.err.println("Get job tags failed: " + ex.getMessage());
 return null;
 })
 .join();
 } catch (CompletionException ex) {
 System.err.println("Failed to get job tags: " + ex.getMessage());
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Update Batch Job Tags");
 waitForInputToContinue(scanner);
 try {
 actions.putJobTaggingAsync(jobId, accountId)
 .exceptionally(ex -> {
 System.err.println("Put job tagging failed: " +
ex.getMessage());
 return null;
 })
 .join();
 } catch (CompletionException ex) {
 System.err.println("Failed to put job tagging: " + ex.getMessage());
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Delete the Amazon S3 Batch job tagging.");
 System.out.print("Do you want to delete Batch job tagging? (y/n)");
 String delAns = scanner.nextLine();
 if (delAns != null && delAns.trim().equalsIgnoreCase("y")) {
 try {
 actions.deleteBatchJobTagsAsync(jobId, accountId)

```

```

 .exceptionally(ex -> {
 System.err.println("Delete batch job tags failed: " +
ex.getMessage());
 return null;
 })
 .join();
 } catch (CompletionException ex) {
 System.err.println("Failed to delete batch job tags: " +
ex.getMessage());
 }
 } else {
 System.out.println("Tagging was not deleted.");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.print("Do you want to delete the AWS resources used in this
scenario? (y/n)");
 String delResAns = scanner.nextLine();
 if (delResAns != null && delResAns.trim().equalsIgnoreCase("y")) {
 actions.deleteFilesFromBucket(bucketName, fileNames, actions);
 actions.deleteBucketFolderAsync(bucketName);
 actions.deleteBucket(bucketName)
 .thenRun(() -> System.out.println("Bucket deletion completed"))
 .exceptionally(ex -> {
 System.err.println("Error occurred: " + ex.getMessage());
 return null;
 });
 CloudFormationHelper.destroyCloudFormationStack(STACK_NAME);
 } else {
 System.out.println("The AWS resources were not deleted.");
 }
 System.out.println("The Amazon S3 Batch scenario has successfully
completed.");
 System.out.println(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println();
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {

```

```

 System.out.println("Continuing with the program...");
 System.out.println();
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
}
}
}
}
}

```

An action class that wraps operations.

```

public class S3BatchActions {

 private static S3ControlAsyncClient asyncClient;

 private static S3AsyncClient s3AsyncClient ;

 /**
 * Retrieves the asynchronous S3 Control client instance.
 * <p>
 * This method creates and returns a singleton instance of the {@link
 S3ControlAsyncClient}. If the instance
 * has not been created yet, it will be initialized with the following
 configuration:
 *
 * Maximum concurrency: 100
 * Connection timeout: 60 seconds
 * Read timeout: 60 seconds
 * Write timeout: 60 seconds
 * API call timeout: 2 minutes
 * API call attempt timeout: 90 seconds
 * Retry policy: 3 retries
 * Region: US_EAST_1
 * Credentials provider: {@link EnvironmentVariableCredentialsProvider}</
 li>
 *
 *
 * @return the asynchronous S3 Control client instance
 */
}

```



```
private static S3ControlAsyncClient getAsyncClient() {
 if (asyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 asyncClient = S3ControlAsyncClient.builder()
 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return asyncClient;
}

private static S3AsyncClient getS3AsyncClient() {
 if (asyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 s3AsyncClient = S3AsyncClient.builder()
```

```

 .region(Region.US_EAST_1)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return s3AsyncClient;
}

/**
 * Cancels a job asynchronously.
 *
 * @param jobId The ID of the job to be canceled.
 * @param accountId The ID of the account associated with the job.
 * @return A {@link CompletableFuture} that completes when the job status has
 * been updated to "CANCELLED".
 *
 * If an error occurs during the update, the returned future will
 * complete exceptionally.
 */
public CompletableFuture<Void> cancelJobAsync(String jobId, String accountId) {
 UpdateJobStatusRequest updateJobStatusRequest =
UpdateJobStatusRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .requestedJobStatus(String.valueOf(JobStatus.CANCELLED))
 .build();

 return asyncClient.updateJobStatus(updateJobStatusRequest)
 .thenAccept(updateJobStatusResponse -> {
 System.out.println("Job status updated to: " +
updateJobStatusResponse.status());
 })
 .exceptionally(ex -> {
 System.err.println("Failed to cancel job: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
}

/**
 * Updates the priority of a job asynchronously.
 *
 * @param jobId the ID of the job to update
 * @param accountId the ID of the account associated with the job

```

```

 * @return a {@link CompletableFuture} that represents the asynchronous
 operation, which completes when the job priority has been updated or an error has
 occurred
 */
 public CompletableFuture<Void> updateJobPriorityAsync(String jobId, String
accountId) {
 UpdateJobPriorityRequest priorityRequest =
UpdateJobPriorityRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .priority(60)
 .build();

 CompletableFuture<Void> future = new CompletableFuture<>();
 getAsyncClient().updateJobPriority(priorityRequest)
 .thenAccept(response -> {
 System.out.println("The job priority was updated");
 future.complete(null); // Complete the CompletableFuture on
successful execution
 })
 .exceptionally(ex -> {
 System.err.println("Failed to update job priority: " +
ex.getMessage());
 future.completeExceptionally(ex); // Complete the CompletableFuture
exceptionally on error
 return null; // Return null to handle the exception
 });

 return future;
 }

 /**
 * Asynchronously retrieves the tags associated with a specific job in an AWS
account.
 *
 * @param jobId the ID of the job for which to retrieve the tags
 * @param accountId the ID of the AWS account associated with the job
 * @return a {@link CompletableFuture} that completes when the job tags have
been retrieved, or with an exception if the operation fails
 * @throws RuntimeException if an error occurs while retrieving the job tags
 */
 public CompletableFuture<Void> getJobTagsAsync(String jobId, String accountId) {
 GetJobTaggingRequest request = GetJobTaggingRequest.builder()
 .jobId(jobId)

```

```

 .accountId(accountId)
 .build();

return asyncClient.getJobTagging(request)
 .thenAccept(response -> {
 List<S3Tag> tags = response.tags();
 if (tags.isEmpty()) {
 System.out.println("No tags found for job ID: " + jobId);
 } else {
 for (S3Tag tag : tags) {
 System.out.println("Tag key is: " + tag.key());
 System.out.println("Tag value is: " + tag.value());
 }
 }
 })
 .exceptionally(ex -> {
 System.err.println("Failed to get job tags: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
}

/**
 * Asynchronously deletes the tags associated with a specific batch job.
 *
 * @param jobId The ID of the batch job whose tags should be deleted.
 * @param accountId The ID of the account associated with the batch job.
 * @return A CompletableFuture that completes when the job tags have been
 * successfully deleted, or an exception is thrown if the deletion fails.
 */
public CompletableFuture<Void> deleteBatchJobTagsAsync(String jobId, String
accountId) {
 DeleteJobTaggingRequest jobTaggingRequest =
DeleteJobTaggingRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .build();

 return asyncClient.deleteJobTagging(jobTaggingRequest)
 .thenAccept(response -> {
 System.out.println("You have successfully deleted " + jobId + "
tagging.");
 })
 .exceptionally(ex -> {
 System.err.println("Failed to delete job tags: " + ex.getMessage());

```

```

 throw new RuntimeException(ex);
 });
}

/**
 * Asynchronously describes the specified job.
 *
 * @param jobId the ID of the job to describe
 * @param accountId the ID of the AWS account associated with the job
 * @return a {@link CompletableFuture} that completes when the job description
is available
 * @throws RuntimeException if an error occurs while describing the job
 */
public CompletableFuture<Void> describeJobAsync(String jobId, String accountId)
{
 DescribeJobRequest jobRequest = DescribeJobRequest.builder()
 .jobId(jobId)
 .accountId(accountId)
 .build();

 return getAsyncClient().describeJob(jobRequest)
 .thenAccept(response -> {
 System.out.println("Job ID: " + response.job().jobId());
 System.out.println("Description: " + response.job().description());
 System.out.println("Status: " + response.job().statusAsString());
 System.out.println("Role ARN: " + response.job().roleArn());
 System.out.println("Priority: " + response.job().priority());
 System.out.println("Progress Summary: " +
response.job().progressSummary());

 // Print out details about the job manifest.
 JobManifest manifest = response.job().manifest();
 System.out.println("Manifest Location: " +
manifest.location().objectArn());
 System.out.println("Manifest ETag: " + manifest.location().eTag());

 // Print out details about the job operation.
 JobOperation operation = response.job().operation();
 if (operation.s3PutObjectTagging() != null) {
 System.out.println("Operation: S3 Put Object Tagging");
 System.out.println("Tag Set: " +
operation.s3PutObjectTagging().tagSet());
 }
 });
}

```

```

 // Print out details about the job report.
 JobReport report = response.job().report();
 System.out.println("Report Bucket: " + report.bucket());
 System.out.println("Report Prefix: " + report.prefix());
 System.out.println("Report Format: " + report.format());
 System.out.println("Report Enabled: " + report.enabled());
 System.out.println("Report Scope: " + report.reportScopeAsString());
 })
 .exceptionally(ex -> {
 System.err.println("Failed to describe job: " + ex.getMessage());
 throw new RuntimeException(ex);
 });
}

/**
 * Creates an asynchronous S3 job using the AWS Java SDK.
 *
 * @param accountId the AWS account ID associated with the job
 * @param iamRoleArn the ARN of the IAM role to be used for the job
 * @param manifestLocation the location of the job manifest file in S3
 * @param reportBucketName the name of the S3 bucket to store the job report
 * @param uuid a unique identifier for the job
 * @return a CompletableFuture that represents the asynchronous creation of the
S3 job.
 * The CompletableFuture will return the job ID if the job is created
successfully,
 * or throw an exception if there is an error.
 */
public CompletableFuture<String> createS3JobAsync(String accountId, String
iamRoleArn,
 String manifestLocation,
String reportBucketName, String uuid) {

 String[] bucketName = new String[]{"");
 String[] parts = reportBucketName.split(":::");
 if (parts.length > 1) {
 bucketName[0] = parts[1];
 } else {
 System.out.println("The input string does not contain the expected
format.");
 }

 return CompletableFuture.supplyAsync(() -> getETag(bucketName[0], "job-
manifest.csv"))

```

```
.thenCompose(eTag -> {
 ArrayList<S3Tag> tagSet = new ArrayList<>();
 S3Tag s3Tag = S3Tag.builder()
 .key("keyOne")
 .value("ValueOne")
 .build();
 S3Tag s3Tag2 = S3Tag.builder()
 .key("keyTwo")
 .value("ValueTwo")
 .build();
 tagSet.add(s3Tag);
 tagSet.add(s3Tag2);

 S3SetObjectTaggingOperation objectTaggingOperation =
S3SetObjectTaggingOperation.builder()
 .tagSet(tagSet)
 .build();

 JobOperation jobOperation = JobOperation.builder()
 .s3PutObjectTagging(objectTaggingOperation)
 .build();

 JobManifestLocation jobManifestLocation =
JobManifestLocation.builder()
 .objectArn(manifestLocation)
 .eTag(eTag)
 .build();

 JobManifestSpec manifestSpec = JobManifestSpec.builder()
 .fieldsWithStrings("Bucket", "Key")
 .format("S3BatchOperations_CSV_20180820")
 .build();

 JobManifest jobManifest = JobManifest.builder()
 .spec(manifestSpec)
 .location(jobManifestLocation)
 .build();

 JobReport jobReport = JobReport.builder()
 .bucket(reportBucketName)
 .prefix("reports")
 .format("Report_CSV_20180820")
 .enabled(true)
 .reportScope("AllTasks")
```

```

 .build();

 CreateJobRequest jobRequest = CreateJobRequest.builder()
 .accountId(accountId)
 .description("Job created using the AWS Java SDK")
 .manifest(jobManifest)
 .operation(jobOperation)
 .report(jobReport)
 .priority(42)
 .roleArn(iamRoleArn)
 .clientRequestToken(uuid)
 .confirmationRequired(false)
 .build();

 // Create the job asynchronously.
 return getAsyncClient().createJob(jobRequest)
 .thenApply(CreateJobResponse::jobId);
 })
 .handle((jobId, ex) -> {
 if (ex != null) {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof S3ControlException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 return jobId;
 });
}

/**
 * Retrieves the ETag (Entity Tag) for an object stored in an Amazon S3 bucket.
 *
 * @param bucketName the name of the Amazon S3 bucket where the object is stored
 * @param key the key (file name) of the object in the Amazon S3 bucket
 * @return the ETag of the object
 */
public String getETag(String bucketName, String key) {
 S3Client s3Client = S3Client.builder()
 .region(Region.US_EAST_1)
 .build();

```



```

 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();

 HeadObjectResponse headObjectResponse =
s3Client.headObject(headObjectRequest);
 return headObjectResponse.eTag();
 }

 /**
 * Asynchronously adds tags to a job in the system.
 *
 * @param jobId the ID of the job to add tags to
 * @param accountId the account ID associated with the job
 * @return a CompletableFuture that completes when the tagging operation is
finished
 */
 public CompletableFuture<Void> putJobTaggingAsync(String jobId, String
accountId) {
 S3Tag departmentTag = S3Tag.builder()
 .key("department")
 .value("Marketing")
 .build();

 S3Tag fiscalYearTag = S3Tag.builder()
 .key("FiscalYear")
 .value("2020")
 .build();

 PutJobTaggingRequest putJobTaggingRequest = PutJobTaggingRequest.builder()
 .jobId(jobId)
 .accountId(accountId)
 .tags(departmentTag, fiscalYearTag)
 .build();

 return asyncClient.putJobTagging(putJobTaggingRequest)
 .thenRun(() -> {
 System.out.println("Additional Tags were added to job " + jobId);
 })
 .exceptionally(ex -> {
 System.err.println("Failed to add tags to job: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
 }

```

```

 }

 // Setup the S3 bucket required for this scenario.
 /**
 * Creates an Amazon S3 bucket with the specified name.
 *
 * @param bucketName the name of the S3 bucket to create
 * @throws S3Exception if there is an error creating the bucket
 */
 public void createBucket(String bucketName) {
 try {
 S3Client s3Client = S3Client.builder()
 .region(Region.US_EAST_1)
 .build();

 S3Waiter s3Waiter = s3Client.waiter();
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 s3Client.createBucket(bucketRequest);
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 // Wait until the bucket is created and print out the response.
 WaiterResponse<HeadBucketResponse> waiterResponse =
s3Waiter.waitUntilBucketExists(bucketRequestWait);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println(bucketName + " is ready");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 /**
 * Uploads a file to an Amazon S3 bucket asynchronously.
 *
 * @param bucketName the name of the S3 bucket to upload the file to
 * @param fileName the name of the file to be uploaded
 * @throws RuntimeException if an error occurs during the file upload

```

```
 */
 public void populateBucket(String bucketName, String fileName) {
 // Define the path to the directory.
 Path filePath = Paths.get("src/main/resources/batch/",
 fileName).toAbsolutePath();
 PutObjectRequest putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(fileName)
 .build();

 CompletableFuture<PutObjectResponse> future =
 getS3AsyncClient().putObject(putOb, AsyncRequestBody.fromFile(filePath));
 future.whenComplete((result, ex) -> {
 if (ex != null) {
 System.err.println("Error uploading file: " + ex.getMessage());
 } else {
 System.out.println("Successfully placed " + fileName + " into bucket
 " + bucketName);
 }
 }).join();
 }

 // Update the bucketName in CSV.
 public void updateCSV(String newValue) {
 Path csvFilePath = Paths.get("src/main/resources/batch/job-
 manifest.csv").toAbsolutePath();
 try {
 // Read all lines from the CSV file.
 List<String> lines = Files.readAllLines(csvFilePath);

 // Update the first value in each line.
 List<String> updatedLines = lines.stream()
 .map(line -> {
 String[] parts = line.split(",");
 parts[0] = newValue;
 return String.join(",", parts);
 })
 .collect(Collectors.toList());

 // Write the updated lines back to the CSV file
 Files.write(csvFilePath, updatedLines);
 System.out.println("CSV file updated successfully.");
 } catch (Exception e) {
```

```

 e.printStackTrace();
 }
}

/**
 * Deletes an object from an Amazon S3 bucket asynchronously.
 *
 * @param bucketName The name of the S3 bucket where the object is stored.
 * @param objectName The name of the object to be deleted.
 * @return A {@link CompletableFuture} that completes when the object has been
deleted,
 * or throws a {@link RuntimeException} if an error occurs during the
deletion.
 */
public CompletableFuture<Void> deleteBucketObjects(String bucketName, String
objectName) {
 ArrayList<ObjectIdentifier> toDelete = new ArrayList<>();
 toDelete.add(ObjectIdentifier.builder()
 .key(objectName)
 .build());

 DeleteObjectsRequest dor = DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(Delete.builder()
 .objects(toDelete).build())
 .build();

 return getS3AsyncClient().deleteObjects(dor)
 .thenAccept(result -> {
 System.out.println("The object was deleted!");
 })
 .exceptionally(ex -> {
 throw new RuntimeException("Error deleting object: " +
ex.getMessage(), ex);
 });
}

/**
 * Deletes a folder and all its contents asynchronously from an Amazon S3
bucket.
 *
 * @param bucketName the name of the S3 bucket containing the folder to be
deleted

```

```

 * @return a {@link CompletableFuture} that completes when the folder and its
 contents have been deleted
 * @throws RuntimeException if any error occurs during the deletion process
 */
 public void deleteBucketFolderAsync(String bucketName) {
 String folderName = "reports/";
 ListObjectsV2Request request = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .prefix(folderName)
 .build();

 CompletableFuture<ListObjectsV2Response> listObjectsFuture =
 getS3AsyncClient().listObjectsV2(request);
 listObjectsFuture.thenCompose(response -> {
 List<CompletableFuture<DeleteObjectResponse>> deleteFutures =
 response.contents().stream()
 .map(obj -> {
 DeleteObjectRequest deleteRequest =
 DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(obj.key())
 .build();
 return getS3AsyncClient().deleteObject(deleteRequest)
 .thenApply(deleteResponse -> {
 System.out.println("Deleted object: " + obj.key());
 return deleteResponse;
 });
 })
 .collect(Collectors.toList());

 return CompletableFuture.allOf(deleteFutures.toArray(new
 CompletableFuture[0]))
 .thenCompose(v -> {
 // Delete the folder.
 DeleteObjectRequest deleteRequest =
 DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(folderName)
 .build();
 return getS3AsyncClient().deleteObject(deleteRequest)
 .thenApply(deleteResponse -> {
 System.out.println("Deleted folder: " + folderName);
 return deleteResponse;
 });
 });
 });
 }

```

```

 });
 }).join();
}

/**
 * Deletes an Amazon S3 bucket.
 *
 * @param bucketName the name of the bucket to delete
 * @return a {@link CompletableFuture} that completes when the bucket has been
 * deleted, or exceptionally if there is an error
 * @throws RuntimeException if there is an error deleting the bucket
 */
public CompletableFuture<Void> deleteBucket(String bucketName) {
 S3AsyncClient s3Client = getS3AsyncClient();
 return s3Client.deleteBucket(DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build())
 .thenAccept(deleteBucketResponse -> {
 System.out.println(bucketName + " was deleted");
 })
 .exceptionally(ex -> {
 // Handle the exception or rethrow it.
 throw new RuntimeException("Failed to delete bucket: " + bucketName,
ex);
 });
}

/**
 * Uploads a set of files to an Amazon S3 bucket.
 *
 * @param bucketName the name of the S3 bucket to upload the files to
 * @param fileNames an array of file names to be uploaded
 * @param actions an instance of {@link S3BatchActions} that provides the
 * implementation for the necessary S3 operations
 * @throws IOException if there's an error creating the text files or uploading
 * the files to the S3 bucket
 */
public static void uploadFilesToBucket(String bucketName, String[] fileNames,
S3BatchActions actions) throws IOException {
 actions.updateCSV(bucketName);
 createTextFiles(fileNames);
 for (String fileName : fileNames) {
 actions.populateBucket(bucketName, fileName);
 }
}

```

```

 System.out.println("All files are placed in the S3 bucket " + bucketName);
 }

 /**
 * Deletes the specified files from the given S3 bucket.
 *
 * @param bucketName the name of the S3 bucket
 * @param fileNames an array of file names to be deleted from the bucket
 * @param actions the S3BatchActions instance to be used for the file deletion
 * @throws IOException if an I/O error occurs during the file deletion
 */
 public void deleteFilesFromBucket(String bucketName, String[] fileNames,
 S3BatchActions actions) throws IOException {
 for (String fileName : fileNames) {
 actions.deleteBucketObjects(bucketName, fileName)
 .thenRun(() -> System.out.println("Object deletion completed"))
 .exceptionally(ex -> {
 System.err.println("Error occurred: " + ex.getMessage());
 return null;
 });
 }
 System.out.println("All files have been deleted from the bucket " +
 bucketName);
 }

 public static void createTextFiles(String[] fileNames) {
 String currentDirectory = System.getProperty("user.dir");
 String directoryPath = currentDirectory + "\\src\\main\\resources\\batch";
 Path path = Paths.get(directoryPath);

 try {
 // Create the directory if it doesn't exist.
 if (Files.notExists(path)) {
 Files.createDirectories(path);
 System.out.println("Created directory: " + path.toString());
 } else {
 System.out.println("Directory already exists: " + path.toString());
 }
 }

 for (String fileName : fileNames) {
 // Check if the file is a .txt file.
 if (fileName.endsWith(".txt")) {
 // Define the path for the new file.
 Path filePath = path.resolve(fileName);
 }
 }
 }

```

```

 System.out.println("Attempting to create file: " +
filePath.toString());

 // Create and write content to the new file.
 Files.write(filePath, "This is a test".getBytes());

 // Verify the file was created.
 if (Files.exists(filePath)) {
 System.out.println("Successfully created file: " +
filePath.toString());
 } else {
 System.out.println("Failed to create file: " +
filePath.toString());
 }
 }

} catch (IOException e) {
 System.err.println("An error occurred: " + e.getMessage());
 e.printStackTrace();
}

}

public String getAccountId() {
 StsClient stsClient = StsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 GetCallerIdentityResponse callerIdentityResponse =
stsClient.getCallerIdentity();
 return callerIdentityResponse.account();
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateJob](#)
- [DeleteJobTagging](#)
- [DescribeJob](#)
- [GetJobTagging](#)
- [ListJobs](#)



- [PutJobTagging](#)
- [UpdateJobPriority](#)
- [UpdateJobStatus](#)

## Actions

### CreateJob

The following code example shows how to use CreateJob.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create an asynchronous S3 job.

```
/**
 * Creates an asynchronous S3 job using the AWS Java SDK.
 *
 * @param accountId the AWS account ID associated with the job
 * @param iamRoleArn the ARN of the IAM role to be used for the job
 * @param manifestLocation the location of the job manifest file in S3
 * @param reportBucketName the name of the S3 bucket to store the job report
 * @param uuid a unique identifier for the job
 * @return a CompletableFuture that represents the asynchronous creation of the
 * S3 job.
 * The CompletableFuture will return the job ID if the job is created
 * successfully,
 * or throw an exception if there is an error.
 */
public CompletableFuture<String> createS3JobAsync(String accountId, String
iamRoleArn,
 String manifestLocation,
String reportBucketName, String uuid) {

 String[] bucketName = new String[]{" "};
 String[] parts = reportBucketName.split("::");
```

```

 if (parts.length > 1) {
 bucketName[0] = parts[1];
 } else {
 System.out.println("The input string does not contain the expected
format.");
 }

 return CompletableFuture.supplyAsync(() -> getETag(bucketName[0], "job-
manifest.csv"))
 .thenCompose(eTag -> {
 ArrayList<S3Tag> tagSet = new ArrayList<>();
 S3Tag s3Tag = S3Tag.builder()
 .key("keyOne")
 .value("ValueOne")
 .build();
 S3Tag s3Tag2 = S3Tag.builder()
 .key("keyTwo")
 .value("ValueTwo")
 .build();
 tagSet.add(s3Tag);
 tagSet.add(s3Tag2);

 S3SetObjectTaggingOperation objectTaggingOperation =
S3SetObjectTaggingOperation.builder()
 .tagSet(tagSet)
 .build();

 JobOperation jobOperation = JobOperation.builder()
 .s3PutObjectTagging(objectTaggingOperation)
 .build();

 JobManifestLocation jobManifestLocation =
JobManifestLocation.builder()
 .objectArn(manifestLocation)
 .eTag(eTag)
 .build();

 JobManifestSpec manifestSpec = JobManifestSpec.builder()
 .fieldsWithStrings("Bucket", "Key")
 .format("S3BatchOperations_CSV_20180820")
 .build();

 JobManifest jobManifest = JobManifest.builder()
 .spec(manifestSpec)

```

```

 .location(jobManifestLocation)
 .build();

 JobReport jobReport = JobReport.builder()
 .bucket(reportBucketName)
 .prefix("reports")
 .format("Report_CSV_20180820")
 .enabled(true)
 .reportScope("AllTasks")
 .build();

 CreateJobRequest jobRequest = CreateJobRequest.builder()
 .accountId(accountId)
 .description("Job created using the AWS Java SDK")
 .manifest(jobManifest)
 .operation(jobOperation)
 .report(jobReport)
 .priority(42)
 .roleArn(iamRoleArn)
 .clientRequestToken(uuid)
 .confirmationRequired(false)
 .build();

 // Create the job asynchronously.
 return getAsyncClient().createJob(jobRequest)
 .thenApply(CreateJobResponse::jobId);
 })
 .handle((jobId, ex) -> {
 if (ex != null) {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof S3ControlException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 return jobId;
 });
}

```

Create a compliance retention job.

```

/**
 * Creates a compliance retention job in Amazon S3 Control.
 * <p>
 * A compliance retention job in Amazon S3 Control is a feature that allows you
to
 * set a retention period for objects stored in an S3 bucket.
 * This feature is particularly useful for organizations that need to comply
with
 * regulatory requirements or internal policies that mandate the retention of
data for
 * a specific duration.
 *
 * @param s3ControlClient The S3ControlClient instance to use for the API call.
 * @return The job ID of the created compliance retention job.
 */
public static String createComplianceRetentionJob(final S3ControlClient
s3ControlClient, String roleArn, String bucketName, String accountId) {
 final String manifestObjectArn = "arn:aws:s3:::amzn-s3-demo-manifest-bucket/
compliance-objects-manifest.csv";
 final String manifestObjectVersionId = "your-object-version-Id";

 Instant jan2025 = Instant.parse("2025-01-01T00:00:00Z");
 JobOperation jobOperation = JobOperation.builder()
 .s3PutObjectRetention(S3SetObjectRetentionOperation.builder()
 .retention(S3Retention.builder()
 .mode(S3ObjectLockRetentionMode.COMPLIANCE)
 .retainUntilDate(jan2025)
 .build())
 .build())
 .build();

 JobManifestLocation manifestLocation = JobManifestLocation.builder()
 .objectArn(manifestObjectArn)
 .eTag(manifestObjectVersionId)
 .build();

 JobManifestSpec manifestSpec = JobManifestSpec.builder()
 .fieldsWithStrings("Bucket", "Key")
 .format("S3BatchOperations_CSV_20180820")
 .build();

 JobManifest manifestToPublicApi = JobManifest.builder()

```

```

 .location(manifestLocation)
 .spec(manifestSpec)
 .build();

// Report details.
final String jobReportBucketArn = "arn:aws:s3:::" + bucketName;
final String jobReportPrefix = "reports/compliance-objects-bops";

JobReport jobReport = JobReport.builder()
 .enabled(true)
 .reportScope(JobReportScope.ALL_TASKS)
 .bucket(jobReportBucketArn)
 .prefix(jobReportPrefix)
 .format(JobReportFormat.REPORT_CSV_20180820)
 .build();

final Boolean requiresConfirmation = true;
final int priority = 10;
CreateJobRequest request = CreateJobRequest.builder()
 .accountId(accountId)
 .description("Set compliance retain-until to 1 Jan 2025")
 .manifest(manifestToPublicApi)
 .operation(jobOperation)
 .priority(priority)
 .roleArn(roleArn)
 .report(jobReport)
 .confirmationRequired(requiresConfirmation)
 .build();

// Create the job and get the result.
CreateJobResponse result = s3ControlClient.createJob(request);
return result.jobId();
}

```

### Create a legal hold off job.

```

/**
 * Creates a compliance retention job in Amazon S3 Control.
 * <p>
 * A compliance retention job in Amazon S3 Control is a feature that allows you
to

```

```

 * set a retention period for objects stored in an S3 bucket.
 * This feature is particularly useful for organizations that need to comply
with
 * regulatory requirements or internal policies that mandate the retention of
data for
 * a specific duration.
 *
 * @param s3ControlClient The S3ControlClient instance to use for the API call.
 * @return The job ID of the created compliance retention job.
 */
 public static String createComplianceRetentionJob(final S3ControlClient
s3ControlClient, String roleArn, String bucketName, String accountId) {
 final String manifestObjectArn = "arn:aws:s3:::amzn-s3-demo-manifest-bucket/
compliance-objects-manifest.csv";
 final String manifestObjectVersionId = "your-object-version-Id";

 Instant jan2025 = Instant.parse("2025-01-01T00:00:00Z");
 JobOperation jobOperation = JobOperation.builder()
 .s3PutObjectRetention(S3SetObjectRetentionOperation.builder()
 .retention(S3Retention.builder()
 .mode(S3ObjectLockRetentionMode.COMPLIANCE)
 .retainUntilDate(jan2025)
 .build())
 .build())
 .build();

 JobManifestLocation manifestLocation = JobManifestLocation.builder()
 .objectArn(manifestObjectArn)
 .eTag(manifestObjectVersionId)
 .build();

 JobManifestSpec manifestSpec = JobManifestSpec.builder()
 .fieldsWithStrings("Bucket", "Key")
 .format("S3BatchOperations_CSV_20180820")
 .build();

 JobManifest manifestToPublicApi = JobManifest.builder()
 .location(manifestLocation)
 .spec(manifestSpec)
 .build();

 // Report details.
 final String jobReportBucketArn = "arn:aws:s3:::" + bucketName;
 final String jobReportPrefix = "reports/compliance-objects-bops";

```

```

 JobReport jobReport = JobReport.builder()
 .enabled(true)
 .reportScope(JobReportScope.ALL_TASKS)
 .bucket(jobReportBucketArn)
 .prefix(jobReportPrefix)
 .format(JobReportFormat.REPORT_CSV_20180820)
 .build();

 final Boolean requiresConfirmation = true;
 final int priority = 10;
 CreateJobRequest request = CreateJobRequest.builder()
 .accountId(accountId)
 .description("Set compliance retain-until to 1 Jan 2025")
 .manifest(manifestToPublicApi)
 .operation(jobOperation)
 .priority(priority)
 .roleArn(roleArn)
 .report(jobReport)
 .confirmationRequired(requiresConfirmation)
 .build();

 // Create the job and get the result.
 CreateJobResponse result = s3ControlClient.createJob(request);
 return result.jobId();
 }

```

Create a new governance retention job.

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateGovernanceRetentionJob {

 public static void main(String[]args) throws ParseException {
 final String usage = ""

```

**Usage:**

```
<manifestObjectArn> <jobReportBucketArn> <roleArn> <accountId>
<manifestObjectVersionId>
```

**Where:**

manifestObjectArn - The Amazon Resource Name (ARN) of the S3 object that contains the manifest file for the governance objects.\s

bucketName - The ARN of the S3 bucket where the job report will be stored.

roleArn - The ARN of the IAM role that will be used to perform the governance retention operation.

accountId - Your AWS account Id.

manifestObjectVersionId = A unique value that is used as the `eTag` property of the `JobManifestLocation` object.

```
"";
```

```
if (args.length != 4) {
 System.out.println(usage);
 return;
}
```

```
String manifestObjectArn = args[0];
String jobReportBucketArn = args[1];
String roleArn = args[2];
String accountId = args[3];
String manifestObjectVersionId = args[4];
```

```
S3ControlClient s3ControlClient = S3ControlClient.create();
createGovernanceRetentionJob(s3ControlClient, manifestObjectArn,
jobReportBucketArn, roleArn, accountId, manifestObjectVersionId);
}
```

```
public static String createGovernanceRetentionJob(final S3ControlClient
s3ControlClient, String manifestObjectArn, String jobReportBucketArn, String
roleArn, String accountId, String manifestObjectVersionId) throws ParseException {
 final JobManifestLocation manifestLocation = JobManifestLocation.builder()
 .objectArn(manifestObjectArn)
 .eTag(manifestObjectVersionId)
 .build();

 final JobManifestSpec manifestSpec = JobManifestSpec.builder()
 .format(JobManifestFormat.S3_BATCH_OPERATIONS_CSV_20180820)
 .fields(Arrays.asList(JobManifestFieldName.BUCKET,
JobManifestFieldName.KEY))
```



```
 .build();

 final JobManifest manifestToPublicApi = JobManifest.builder()
 .location(manifestLocation)
 .spec(manifestSpec)
 .build();

 final String jobReportPrefix = "reports/governance-objects";
 final JobReport jobReport = JobReport.builder()
 .enabled(true)
 .reportScope(JobReportScope.ALL_TASKS)
 .bucket(jobReportBucketArn)
 .prefix(jobReportPrefix)
 .format(JobReportFormat.REPORT_CSV_20180820)
 .build();

 final SimpleDateFormat format = new SimpleDateFormat("dd/MM/yyyy");
 final Date jan30th = format.parse("30/01/2025");

 final S3SetObjectRetentionOperation s3SetObjectRetentionOperation =
 S3SetObjectRetentionOperation.builder()
 .retention(S3Retention.builder()
 .mode(S3ObjectLockRetentionMode.GOVERNANCE)
 .retainUntilDate(jan30th.toInstant())
 .build())
 .build();

 final JobOperation jobOperation = JobOperation.builder()
 .s3PutObjectRetention(s3SetObjectRetentionOperation)
 .build();

 final Boolean requiresConfirmation = true;
 final int priority = 10;

 final CreateJobRequest request = CreateJobRequest.builder()
 .accountId(accountId)
 .description("Put governance retention")
 .manifest(manifestToPublicApi)
 .operation(jobOperation)
 .priority(priority)
 .roleArn(roleArn)
 .report(jobReport)
 .confirmationRequired(requiresConfirmation)
 .build();
```

```

 final CreateJobResponse result = s3ControlClient.createJob(request);
 return result.jobId();
 }
}

```

- For API details, see [CreateJob](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteJobTagging

The following code example shows how to use DeleteJobTagging.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously deletes the tags associated with a specific batch job.
 *
 * @param jobId The ID of the batch job whose tags should be deleted.
 * @param accountId The ID of the account associated with the batch job.
 * @return A CompletableFuture that completes when the job tags have been
 * successfully deleted, or an exception is thrown if the deletion fails.
 */
public CompletableFuture<Void> deleteBatchJobTagsAsync(String jobId, String
accountId) {
 DeleteJobTaggingRequest jobTaggingRequest =
DeleteJobTaggingRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .build();

 return asyncClient.deleteJobTagging(jobTaggingRequest)
 .thenAccept(response -> {
 System.out.println("You have successfully deleted " + jobId + "
tagging.");

```

```

 })
 .exceptionally(ex -> {
 System.err.println("Failed to delete job tags: " + ex.getMessage());
 throw new RuntimeException(ex);
 });
}

```

- For API details, see [DeleteJobTagging](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeJob

The following code example shows how to use DescribeJob.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Asynchronously describes the specified job.
 *
 * @param jobId the ID of the job to describe
 * @param accountId the ID of the AWS account associated with the job
 * @return a {@link CompletableFuture} that completes when the job description
is available
 * @throws RuntimeException if an error occurs while describing the job
 */
public CompletableFuture<Void> describeJobAsync(String jobId, String accountId)
{
 DescribeJobRequest jobRequest = DescribeJobRequest.builder()
 .jobId(jobId)
 .accountId(accountId)
 .build();

 return getAsyncClient().describeJob(jobRequest)
 .thenAccept(response -> {
 System.out.println("Job ID: " + response.job().jobId());
 });
}

```

```

 System.out.println("Description: " + response.job().description());
 System.out.println("Status: " + response.job().statusAsString());
 System.out.println("Role ARN: " + response.job().roleArn());
 System.out.println("Priority: " + response.job().priority());
 System.out.println("Progress Summary: " +
response.job().progressSummary());

 // Print out details about the job manifest.
 JobManifest manifest = response.job().manifest();
 System.out.println("Manifest Location: " +
manifest.location().objectArn());
 System.out.println("Manifest ETag: " + manifest.location().eTag());

 // Print out details about the job operation.
 JobOperation operation = response.job().operation();
 if (operation.s3PutObjectTagging() != null) {
 System.out.println("Operation: S3 Put Object Tagging");
 System.out.println("Tag Set: " +
operation.s3PutObjectTagging().tagSet());
 }

 // Print out details about the job report.
 JobReport report = response.job().report();
 System.out.println("Report Bucket: " + report.bucket());
 System.out.println("Report Prefix: " + report.prefix());
 System.out.println("Report Format: " + report.format());
 System.out.println("Report Enabled: " + report.enabled());
 System.out.println("Report Scope: " + report.reportScopeAsString());
 })
 .exceptionally(ex -> {
 System.err.println("Failed to describe job: " + ex.getMessage());
 throw new RuntimeException(ex);
 });
}

```

- For API details, see [DescribeJob](#) in *AWS SDK for Java 2.x API Reference*.

## GetJobTagging

The following code example shows how to use GetJobTagging.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously retrieves the tags associated with a specific job in an AWS
 * account.
 *
 * @param jobId the ID of the job for which to retrieve the tags
 * @param accountId the ID of the AWS account associated with the job
 * @return a {@link CompletableFuture} that completes when the job tags have
 * been retrieved, or with an exception if the operation fails
 * @throws RuntimeException if an error occurs while retrieving the job tags
 */
public CompletableFuture<Void> getJobTagsAsync(String jobId, String accountId) {
 GetJobTaggingRequest request = GetJobTaggingRequest.builder()
 .jobId(jobId)
 .accountId(accountId)
 .build();

 return asyncClient.getJobTagging(request)
 .thenAccept(response -> {
 List<S3Tag> tags = response.tags();
 if (tags.isEmpty()) {
 System.out.println("No tags found for job ID: " + jobId);
 } else {
 for (S3Tag tag : tags) {
 System.out.println("Tag key is: " + tag.key());
 System.out.println("Tag value is: " + tag.value());
 }
 }
 })
 .exceptionally(ex -> {
 System.err.println("Failed to get job tags: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
}
```

- For API details, see [GetJobTagging](#) in *AWS SDK for Java 2.x API Reference*.

## PutJobTagging

The following code example shows how to use PutJobTagging.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Asynchronously adds tags to a job in the system.
 *
 * @param jobId the ID of the job to add tags to
 * @param accountId the account ID associated with the job
 * @return a CompletableFuture that completes when the tagging operation is
 * finished
 */
public CompletableFuture<Void> putJobTaggingAsync(String jobId, String
accountId) {
 S3Tag departmentTag = S3Tag.builder()
 .key("department")
 .value("Marketing")
 .build();

 S3Tag fiscalYearTag = S3Tag.builder()
 .key("FiscalYear")
 .value("2020")
 .build();

 PutJobTaggingRequest putJobTaggingRequest = PutJobTaggingRequest.builder()
 .jobId(jobId)
 .accountId(accountId)
 .tags(departmentTag, fiscalYearTag)
 .build();

 return asyncClient.putJobTagging(putJobTaggingRequest)
 .thenRun(() -> {
```

```

 System.out.println("Additional Tags were added to job " + jobId);
 })
 .exceptionally(ex -> {
 System.err.println("Failed to add tags to job: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
}

```

- For API details, see [PutJobTagging](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateJobPriority

The following code example shows how to use UpdateJobPriority.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Updates the priority of a job asynchronously.
 *
 * @param jobId the ID of the job to update
 * @param accountId the ID of the account associated with the job
 * @return a {@link CompletableFuture} that represents the asynchronous
 * operation, which completes when the job priority has been updated or an error has
 * occurred
 */
public CompletableFuture<Void> updateJobPriorityAsync(String jobId, String
accountId) {
 UpdateJobPriorityRequest priorityRequest =
UpdateJobPriorityRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .priority(60)
 .build();

 CompletableFuture<Void> future = new CompletableFuture<>();

```

```

 getAsyncClient().updateJobPriority(priorityRequest)
 .thenAccept(response -> {
 System.out.println("The job priority was updated");
 future.complete(null); // Complete the CompletableFuture on
successful execution
 })
 .exceptionally(ex -> {
 System.err.println("Failed to update job priority: " +
ex.getMessage());
 future.completeExceptionally(ex); // Complete the CompletableFuture
exceptionally on error
 return null; // Return null to handle the exception
 });

 return future;
 }

```

- For API details, see [UpdateJobPriority](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateJobStatus

The following code example shows how to use UpdateJobStatus.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Cancels a job asynchronously.
 *
 * @param jobId The ID of the job to be canceled.
 * @param accountId The ID of the account associated with the job.
 * @return A {@link CompletableFuture} that completes when the job status has
been updated to "CANCELLED".
 * If an error occurs during the update, the returned future will
complete exceptionally.
 */

```



```
public CompletableFuture<Void> cancelJobAsync(String jobId, String accountId) {
 UpdateJobStatusRequest updateJobStatusRequest =
UpdateJobStatusRequest.builder()
 .accountId(accountId)
 .jobId(jobId)
 .requestedJobStatus(String.valueOf(JobStatus.CANCELLED))
 .build();

 return asyncClient.updateJobStatus(updateJobStatusRequest)
 .thenAccept(updateJobStatusResponse -> {
 System.out.println("Job status updated to: " +
updateJobStatusResponse.status());
 })
 .exceptionally(ex -> {
 System.err.println("Failed to cancel job: " + ex.getMessage());
 throw new RuntimeException(ex); // Propagate the exception
 });
}
```

- For API details, see [UpdateJobStatus](#) in *AWS SDK for Java 2.x API Reference*.

## S3 Directory Buckets examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with S3 Directory Buckets.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)

- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Amazon S3 directory buckets

The following code example shows how to get started using Amazon S3 directory buckets.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.s3.directorybucket;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.Bucket;
import software.amazon.awssdk.services.s3.model.BucketInfo;
import software.amazon.awssdk.services.s3.model.BucketType;
import software.amazon.awssdk.services.s3.model.CreateBucketConfiguration;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.CreateBucketResponse;
import software.amazon.awssdk.services.s3.model.DataRedundancy;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.ListDirectoryBucketsRequest;
import software.amazon.awssdk.services.s3.model.ListDirectoryBucketsResponse;
import software.amazon.awssdk.services.s3.model.LocationInfo;
import software.amazon.awssdk.services.s3.model.LocationType;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.util.List;
import java.util.stream.Collectors;
```

```
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;

/**
 * Before running this example:
 * <p>
 * The SDK must be able to authenticate AWS requests on your behalf. If you have
 * not configured
 * authentication for SDKs and tools, see
 * https://docs.aws.amazon.com/sdkref/latest/guide/access.html in the AWS SDKs
 * and Tools Reference Guide.
 * <p>
 * You must have a runtime environment configured with the Java SDK.
 * See
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/setup.html in
 * the Developer Guide if this is not set up.
 * <p>
 * To use S3 directory buckets, configure a gateway VPC endpoint. This is the
 * recommended method to enable directory bucket traffic without
 * requiring an internet gateway or NAT device. For more information on
 * configuring VPC gateway endpoints, visit
 * https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-express-networking.html#s3-express-networking-vpc-gateway.
 * <p>
 * Directory buckets are available in specific AWS Regions and Zones. For
 * details on Regions and Zones supporting directory buckets, see
 * https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-express-networking.html#s3-express-endpoints.
 */

public class HelloS3DirectoryBuckets {
 private static final Logger logger =
 LoggerFactory.getLogger(HelloS3DirectoryBuckets.class);

 public static void main(String[] args) {
 String bucketName = "amzn-s3-demo-bucket"; // Replace with your bucket name.
 Region region = Region.US_WEST_2;
 String zone = "usw2-az1";
 S3Client s3Client = createS3Client(region);

 try {
 // Create the directory bucket
 createDirectoryBucket(s3Client, bucketName, zone);
 logger.info("Created bucket: {}", bucketName);
 }
 }
}
```

```

 // List all directory buckets
 List<String> bucketNames = listDirectoryBuckets(s3Client);
 bucketNames.forEach(name -> logger.info("Bucket Name: {}", name));
 } catch (S3Exception e) {
 logger.error("An error occurred during S3 operations: {} - Error code:
{}",
 e.awsErrorDetails().errorMessage(),
e.awsErrorDetails().errorCode(), e);
 } finally {
 try {
 // Delete the created bucket
 deleteDirectoryBucket(s3Client, bucketName);
 logger.info("Deleted bucket: {}", bucketName);
 } catch (S3Exception e) {
 logger.error("Failed to delete the bucket due to S3 error: {} -
Error code: {}",
 e.awsErrorDetails().errorMessage(),
e.awsErrorDetails().errorCode(), e);
 } catch (RuntimeException e) {
 logger.error("Failed to delete the bucket due to unexpected error:
{}", e.getMessage(), e);
 } finally {
 s3Client.close();
 }
 }
}

/**
 * Creates a new S3 directory bucket in a specified Zone (For example, a
 * specified Availability Zone in this code example).
 *
 * @param s3Client The S3 client used to create the bucket
 * @param bucketName The name of the bucket to be created
 * @param zone The region where the bucket will be created
 * @throws S3Exception if there's an error creating the bucket
 */
public static void createDirectoryBucket(S3Client s3Client, String bucketName,
String zone) throws S3Exception {
 logger.info("Creating bucket: {}", bucketName);

 CreateBucketConfiguration bucketConfiguration =
CreateBucketConfiguration.builder()
 .location(LocationInfo.builder()

```

```

 .type(LocationType.AVAILABILITY_ZONE)
 .name(zone).build())
 .bucket(BucketInfo.builder()
 .type(BucketType.DIRECTORY)
 .dataRedundancy(DataRedundancy.SINGLE_AVAILABILITY_ZONE)
 .build())
 .build();
 try {
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .createBucketConfiguration(bucketConfiguration).build();
 CreateBucketResponse response = s3Client.createBucket(bucketRequest);
 logger.info("Bucket created successfully with location: {}",
response.location());
 } catch (S3Exception e) {
 logger.error("Error creating bucket: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

/**
 * Lists all S3 directory buckets.
 *
 * @param s3Client The S3 client used to interact with S3
 * @return A list of bucket names
 */
public static List<String> listDirectoryBuckets(S3Client s3Client) {
 logger.info("Listing all directory buckets");

 try {
 // Create a ListBucketsRequest
 ListDirectoryBucketsRequest listBucketsRequest =
ListDirectoryBucketsRequest.builder().build();

 // Retrieve the list of buckets
 ListDirectoryBucketsResponse response =
s3Client.listDirectoryBuckets(listBucketsRequest);

 // Extract bucket names
 List<String> bucketNames = response.buckets().stream()
 .map(Bucket::name)
 .collect(Collectors.toList());
 }
}

```

```

 return bucketNames;
 } catch (S3Exception e) {
 logger.error("Failed to list buckets: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

/**
 * Deletes the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the bucket to delete
 */
public static void deleteDirectoryBucket(S3Client s3Client, String bucketName) {
 try {
 DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();
 s3Client.deleteBucket(deleteBucketRequest);
 } catch (S3Exception e) {
 logger.error("Failed to delete bucket: " + bucketName + " - Error code: "
 + e.awsErrorDetails().errorCode(),
 e);
 throw e;
 }
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateBucket](#)
  - [ListDirectoryBuckets](#)

## Basics

### Learn the basics

The following code example shows how to:

- Set up a VPC and VPC Endpoint.
- Set up the Policies, Roles, and User to work with S3 directory buckets and the S3 Express One Zone storage class.
- Create two S3 Clients.
- Create two buckets.
- Create an object and copy it over.
- Demonstrate performance difference.
- Populate the buckets to show the lexicographical difference.
- Prompt the user to see if they want to clean up the resources.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run an interactive scenario demonstrating Amazon S3 features.

```
public class S3DirectoriesScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 private static final Logger logger =
 LoggerFactory.getLogger(S3DirectoriesScenario.class);
 static Scanner scanner = new Scanner(System.in);

 private static S3AsyncClient mS3RegularClient;
 private static S3AsyncClient mS3ExpressClient;

 private static String mdirectoryBucketName;
 private static String mregularBucketName;

 private static String stackName = "cfn-stack-s3-express-basics--" +
 UUID.randomUUID();

 private static String regularUser = "";
 private static String vpcId = "";
```

```
private static String expressUser = "";

private static String vpcEndpointId = "";

private static final S3DirectoriesActions s3DirectoriesActions = new
S3DirectoriesActions();

public static void main(String[] args) {
 try {
 s3ExpressScenario();
 } catch (RuntimeException e) {
 logger.info(e.getMessage());
 }
}

// Runs the scenario.
private static void s3ExpressScenario() {
 logger.info(DASHES);
 logger.info("Welcome to the Amazon S3 Express Basics demo using AWS SDK for
Java V2.");
 logger.info(""""
 Let's get started! First, please note that S3 Express One Zone works
best when working within the AWS infrastructure,
 specifically when working in the same Availability Zone (AZ). To see the
best results in this example and when you implement
 directory buckets into your infrastructure, it is best to put your
compute resources in the same AZ as your directory
 bucket.
 """);
 waitForInputToContinue(scanner);
 logger.info(DASHES);

 // Create an optional VPC and create 2 IAM users.
 UserNames userNames = createVpcUsers();
 String expressUserName = userNames.getExpressUserName();
 String regularUserName = userNames.getRegularUserName();

 // Set up two S3 clients, one regular and one express,
 // and two buckets, one regular and one directory.
 setupClientsAndBuckets(expressUserName, regularUserName);

 // Create an S3 session for the express S3 client and add objects to the
buckets.
```



```

 logger.info("Now let's add some objects to our buckets and demonstrate how
to work with S3 Sessions.");
 waitForInputToContinue(scanner);
 String bucketObject = createSessionAddObjects();

 // Demonstrate performance differences between regular and directory
buckets.
 demonstratePerformance(bucketObject);

 // Populate the buckets to show the lexicographical difference between
// regular and express buckets.
 showLexicographicalDifferences(bucketObject);

 logger.info(DASHES);
 logger.info("That's it for our tour of the basic operations for S3 Express
One Zone.");
 logger.info("Would you like to cleanUp the AWS resources? (y/n): ");
 String response = scanner.next().trim().toLowerCase();
 if (response.equals("y")) {
 cleanUp(stackName);
 }
 }

 /**
 Delete resources created by this scenario.
 */
 public static void cleanUp(String stackName) {
 try {
 if (mdirectoryBucketName != null) {
 s3DirectoriesActions.deleteBucketAndObjectsAsync(mS3ExpressClient,
mdirectoryBucketName).join();
 }
 logger.info("Deleted directory bucket " + mdirectoryBucketName);
 mdirectoryBucketName = null;
 if (mregularBucketName != null) {
 s3DirectoriesActions.deleteBucketAndObjectsAsync(mS3RegularClient,
mregularBucketName).join();
 }
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof S3Exception) {
 logger.error("S3Exception occurred: {}", cause.getMessage(), ce);
 } else {

```

```

 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
}

logger.info("Deleted regular bucket " + mregularBucketName);
mregularBucketName = null;
CloudFormationHelper.destroyCloudFormationStack(stackName);
}

private static void showLexicographicalDifferences(String bucketObject) {
 logger.info(DASHES);
 logger.info("7. Populate the buckets to show the lexicographical (alphabetical)
difference
store
name
 when object names are listed. Now let's explore how directory buckets
objects in a different manner to regular buckets. The key is in the
name
 \"Directory\". Where regular buckets store their key/value pairs in a
flat manner, directory buckets use actual directories/folders.
This allows for more rapid indexing, traversing, and therefore
retrieval times!

 The more segmented your bucket is, with lots of
directories, sub-directories, and objects, the more efficient it
becomes.

 This structural difference also causes `ListObject` operations to
behave
differently, which can cause unexpected results. Let's add a few more
objects in sub-directories to see how the output of
ListObjects changes.
 \"\");

 waitForInputToContinue(scanner);

 // Populate a few more files in each bucket so that we can use
 // ListObjects and show the difference.
 String otherObject = "other/" + bucketObject;
 String altObject = "alt/" + bucketObject;
 String otherAltObject = "other/alt/" + bucketObject;

 try {

```

```

 s3DirectoriesActions.putObjectAsync(mS3RegularClient,
mregularBucketName, otherObject, "").join();
 s3DirectoriesActions.putObjectAsync(mS3ExpressClient,
mdirectoryBucketName, otherObject, "").join();
 s3DirectoriesActions.putObjectAsync(mS3RegularClient,
mregularBucketName, altObject, "").join();
 s3DirectoriesActions.putObjectAsync(mS3ExpressClient,
mdirectoryBucketName, altObject, "").join();
 s3DirectoriesActions.putObjectAsync(mS3RegularClient,
mregularBucketName, otherAltObject, "").join();
 s3DirectoriesActions.putObjectAsync(mS3ExpressClient,
mdirectoryBucketName, otherAltObject, "").join();

 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof NoSuchBucketException) {
 logger.error("S3Exception occurred: {}", cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 return;
 }

 try {
 // List objects in both S3 buckets.
 List<String> dirBucketObjects =
s3DirectoriesActions.listObjectsAsync(mS3ExpressClient,
mdirectoryBucketName).join();
 List<String> regBucketObjects =
s3DirectoriesActions.listObjectsAsync(mS3RegularClient, mregularBucketName).join();

 logger.info("Directory bucket content");
 for (String obj : dirBucketObjects) {
 logger.info(obj);
 }

 logger.info("Regular bucket content");
 for (String obj : regBucketObjects) {
 logger.info(obj);
 }
 } catch (CompletionException e) {
 logger.error("Async operation failed: {} ", e.getCause().getMessage());
 return;
 }

```

```

 }

 logger.info("""
 Notice how the regular bucket lists objects in lexicographical order,
while the directory bucket does not. This is
 because the regular bucket considers the whole "key" to be the object
identifier, while the directory bucket actually
 creates directories and uses the object "key" as a path to the object.
 """);
 waitForInputToContinue(scanner);
}

/**
 * Demonstrates the performance difference between downloading an object from a
directory bucket and a regular bucket.
 *
 * <p>This method:
 *
 * Prompts the user to choose the number of downloads (default is
1,000).
 * Downloads the specified object from the directory bucket and measures
the total time.
 * Downloads the same object from the regular bucket and measures the
total time.
 * Compares the time differences and prints the results.
 *
 *
 * <p>Note: The performance difference will be more pronounced if this example
is run on an EC2 instance
 * in the same Availability Zone as the buckets.
 *
 * @param bucketObject the name of the object to download
 */
private static void demonstratePerformance(String bucketObject) {
 logger.info(DASHES);
 logger.info("6. Demonstrate the performance difference.");
 logger.info("""
 Now, let's do a performance test. We'll download the same object from
each
 bucket repeatedly and compare the total time needed.

 Note: the performance difference will be much more pronounced if this
example is run in an EC2 instance in the same Availability Zone as
 the bucket.
 """);
}

```

```

 "");
 waitForInputToContinue(scanner);

 int downloads = 1000; // Default value.
 logger.info("The default number of downloads of the same object for this
example is set at " + downloads + ".");

 // Ask if the user wants to download a different number.
 logger.info("Would you like to download the file a different number of
times? (y/n): ");
 String response = scanner.next().trim().toLowerCase();
 if (response.equals("y")) {
 int maxDownloads = 1_000_000;

 // Ask for a valid number of downloads.
 while (true) {
 logger.info("Enter a number between 1 and " + maxDownloads + " for
the number of downloads: ");
 if (scanner.hasNextInt()) {
 downloads = scanner.nextInt();
 if (downloads >= 1 && downloads <= maxDownloads) {
 break;
 } else {
 logger.info("Please enter a number between 1 and " +
maxDownloads + ".");
 }
 } else {
 logger.info("Invalid input. Please enter a valid integer.");
 scanner.next();
 }
 }

 logger.info("You have chosen to download {} items.", downloads);
 } else {
 logger.info("No changes made. Using default downloads: {}", downloads);
 }
 // Simulating the download process for the directory bucket.
 logger.info("Downloading from the directory bucket.");
 long directoryTimeStart = System.nanoTime();
 for (int index = 0; index < downloads; index++) {
 if (index % 50 == 0) {
 logger.info("Download " + index + " of " + downloads);
 }
 }
 }
}

```

```
 try {
 // Get the object from the directory bucket.
 s3DirectoriesActions.getObjectAsync(mS3ExpressClient,
mdirectoryBucketName, bucketObject).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof NoSuchKeyException) {
 logger.error("S3Exception occurred: {}", cause.getMessage(),
ce);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage(), ce);
 }
 return;
 }
 }
 long directoryTimeDifference = System.nanoTime() - directoryTimeStart;

 // Download from the regular bucket.
 logger.info("Downloading from the regular bucket.");
 long normalTimeStart = System.nanoTime();
 for (int index = 0; index < downloads; index++) {
 if (index % 50 == 0) {
 logger.info("Download " + index + " of " + downloads);
 }

 try {
 s3DirectoriesActions.getObjectAsync(mS3RegularClient,
mregularBucketName, bucketObject).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof NoSuchKeyException) {
 logger.error("S3Exception occurred: {}", cause.getMessage(),
ce);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage(), ce);
 }
 return;
 }
 }

 long normalTimeDifference = System.nanoTime() - normalTimeStart;
```

```

 logger.info("The directory bucket took " + directoryTimeDifference
+ " nanoseconds, while the regular bucket took " + normalTimeDifference + "
nanoseconds.");
 long difference = normalTimeDifference - directoryTimeDifference;
 logger.info("That's a difference of " + difference + " nanoseconds, or");
 logger.info(difference / 1_000_000_000.0 + " seconds.");

 if (difference < 0) {
 logger.info("The directory buckets were slower. This can happen if you
are not running on the cloud within a VPC.");
 }
 waitForInputToContinue(scanner);
 }

 private static String createSessionAddObjects() {
 logger.info(DASHES);
 logger.info("""
 5. Create an object and copy it.
 We'll create an object consisting of some text and upload it to the
 regular bucket.
 """);
 waitForInputToContinue(scanner);

 String bucketObject = "basic-text-object.txt";
 try {
 s3DirectoriesActions.putObjectAsync(mS3RegularClient,
mregularBucketName, bucketObject, "Look Ma, I'm a bucket!").join();
 s3DirectoriesActions.createSessionAsync(mS3ExpressClient,
mdirectoryBucketName).join();

 // Copy the object to the destination S3 bucket.
 s3DirectoriesActions.copyObjectAsync(mS3ExpressClient,
mregularBucketName, bucketObject, mdirectoryBucketName, bucketObject).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof S3Exception) {
 logger.error("S3Exception occurred: {}", cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 }
 logger.info("""

```

It worked! This is because the S3Client that performed the copy operation is the expressClient using the credentials for the user with permission to work with directory buckets.

It's important to remember the user permissions when interacting with directory buckets. Instead of validating permissions on every call as regular buckets do, directory buckets utilize the user credentials and session token to validate. This allows for much faster connection speeds on every call.

For single calls, this is low, but for many concurrent calls this adds up to a lot of time saved.

```

 """);
 waitForInputToContinue(scanner);
 return bucketObject;
 }

 /**
 * Creates VPC users for the S3 Express One Zone scenario.
 * <p>
 * This method performs the following steps:
 *
 * Optionally creates a new VPC and VPC Endpoint if the application
is running in an EC2 instance in the same Availability Zone as the directory
buckets.
 * Creates two IAM users: one with S3 Express One Zone permissions and
one without.
 *
 *
 * @return a {@link UserNames} object containing the names of the created IAM
users
 */
 public static UserNames createVpcUsers() {
 /**
 * Optionally create a VPC.
 * Create two IAM users, one with S3 Express One Zone permissions and one
without.
 */
 logger.info(DASHES);
 logger.info("""
 1. First, we'll set up a new VPC and VPC Endpoint if this program is
running in an EC2 instance in the same AZ as your\s

```



```

 directory buckets will be. Are you running this in an EC2 instance
 located in the same AZ as your intended directory buckets?
 """);

 logger.info("Do you want to setup a VPC Endpoint? (y/n)");
 String endpointAns = scanner.nextLine().trim();
 if (endpointAns.equalsIgnoreCase("y")) {
 logger.info("""
 Great! Let's set up a VPC, retrieve the Route Table from it, and
 create a VPC Endpoint to connect the S3 Client to.
 """);
 try {
 s3DirectoriesActions.setupVPCAsync().join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof Ec2Exception) {
 logger.error("IamException occurred: {}", cause.getMessage(),
ce);
 } else {
 logger.error("An unexpected error occurred: {}",
cause.getMessage(), ce);
 }
 }
 waitForInputToContinue(scanner);
 } else {
 logger.info("Skipping the VPC setup. Don't forget to use this in
production!");
 }
 logger.info(DASHES);
 logger.info("""
 2. Create a RegularUser and ExpressUser by using the AWS CDK.
 One IAM User, named RegularUser, will have permissions to work only
 with regular buckets and one IAM user, named ExpressUser, will have
 permissions to work only with directory buckets.
 """);
 waitForInputToContinue(scanner);

 // Create two users required for this scenario.
 Map<String, String> stackOutputs = createUsersUsingCDK(stackName);
 regularUser = stackOutputs.get("RegularUser");
 expressUser = stackOutputs.get("ExpressUser");

 UserNames names = new UserNames();
 names.setRegularUserName(regularUser);

```

```

 names.setExpressUserName(expressUser);
 return names;
 }

 /**
 * Creates users using AWS CloudFormation.
 *
 * @return a {@link Map} of String keys and String values representing the stack
 outputs,
 * which may include user-related information such as user names and IDs.
 */
 public static Map<String, String> createUsersUsingCDK(String stackName) {
 logger.info("We'll use an AWS CloudFormation template to create the IAM
 users and policies.");
 CloudFormationHelper.deployCloudFormationStack(stackName);
 return CloudFormationHelper.getStackOutputsAsync(stackName).join();
 }

 /**
 * Sets up the necessary clients and buckets for the S3 Express service.
 *
 * @param expressUserName the username for the user with S3 Express permissions
 * @param regularUserName the username for the user with regular S3 permissions
 */
 public static void setupClientsAndBuckets(String expressUserName, String
 regularUserName) {
 Scanner locscanner = new Scanner(System.in);
 String accessKeyIdforRegUser;
 String secretAccessforRegUser;
 try {
 CreateAccessKeyResponse keyResponse =
s3DirectoriesActions.createAccessKeyAsync(regularUserName).join();
 accessKeyIdforRegUser = keyResponse.accessKey().accessKeyId();
 secretAccessforRegUser = keyResponse.accessKey().secretAccessKey();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof IamException) {
 logger.error("IamException occurred: {}", cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 }
 return;
 }
}

```

```

 String accessKeyIdforExpressUser;
 String secretAccessforExpressUser;
 try {
 CreateAccessKeyResponse keyResponseExpress =
s3DirectoriesActions.createAccessKeyAsync(expressUserName).join();
 accessKeyIdforExpressUser =
keyResponseExpress.accessKey().accessKeyId();
 secretAccessforExpressUser =
keyResponseExpress.accessKey().secretAccessKey();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof IamException) {
 logger.error("IamException occurred: {}", cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 return;
 }

 logger.info(DASHES);
 logger.info("""
 3. Create two S3Clients; one uses the ExpressUser's credentials and one
uses the RegularUser's credentials.
 The 2 S3Clients will use different credentials.
 """);
 waitForInputToContinue(locscanner);
 try {
 mS3RegularClient =
createS3ClientWithAccessKeyAsync(accessKeyIdforRegUser,
secretAccessforRegUser).join();
 mS3ExpressClient =
createS3ClientWithAccessKeyAsync(accessKeyIdforExpressUser,
secretAccessforExpressUser).join();
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof IllegalArgumentException) {
 logger.error("An invalid argument exception occurred: {}",
cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 }
 }

```

```

 return;
 }

 logger.info("""
 We can now use the ExpressUser client to make calls to S3 Express
operations.
 """);
 waitForInputToContinue(locscanner);
 logger.info(DASHES);
 logger.info("""
 4. Create two buckets.
 Now we will create a directory bucket which is the linchpin of the S3
Express One Zone service. Directory buckets
 behave differently from regular S3 buckets which we will explore here.
We'll also create a regular bucket, put
 an object into the regular bucket, and copy it to the directory bucket.
 """);

 logger.info("""
 Now, let's choose an availability zone (AZ) for the directory bucket.
 We'll choose one that is supported.
 """);
 String zoneId;
 String regularBucketName;
 try {
 zoneId = s3DirectoriesActions.selectAvailabilityZoneIdAsync().join();
 regularBucketName = "amzn-s3-demo-bucket"; // Replace with your bucket
name.
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof Ec2Exception) {
 logger.error("EC2Exception occurred: {}", cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 }
 }
 return;
}
 logger.info("""
 Now, let's create the actual directory bucket, as well as a regular
bucket."
 """);

```

```

 String directoryBucketName = "amzn-s3-demo-bucket--" + zoneId + "--x-s3"; //
 Replace with your directory bucket name.
 try {
 s3DirectoriesActions.createDirectoryBucketAsync(mS3ExpressClient,
directoryBucketName, zoneId).join();
 logger.info("Created directory bucket {}", directoryBucketName);
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof BucketAlreadyExistsException) {
 logger.error("The bucket already exists. Moving on: {}",
cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 return;
 }
 }

 // Assign to the data member.
 mdirectoryBucketName = directoryBucketName;
 try {
 s3DirectoriesActions.createBucketAsync(mS3RegularClient,
regularBucketName).join();
 logger.info("Created regular bucket {} ", regularBucketName);
 mregularBucketName = regularBucketName;
 } catch (CompletionException ce) {
 Throwable cause = ce.getCause();
 if (cause instanceof BucketAlreadyExistsException) {
 logger.error("The bucket already exists. Moving on: {}",
cause.getMessage(), ce);
 } else {
 logger.error("An unexpected error occurred: {}", cause.getMessage(),
ce);
 return;
 }
 }
 logger.info("Great! Both buckets were created.");
 waitForInputToContinue(locscanner);
 }

 /**
 * Creates an asynchronous S3 client with the specified access key and secret
 * access key.
 *

```

```

 * @param accessKeyId the AWS access key ID
 * @param secretAccessKey the AWS secret access key
 * @return a {@link CompletableFuture} that asynchronously creates the S3 client
 * @throws IllegalArgumentException if the access key ID or secret access key is
 null
 */
 public static CompletableFuture<S3AsyncClient>
createS3ClientWithAccessKeyAsync(String accessKeyId, String secretAccessKey) {
 return CompletableFuture.supplyAsync(() -> {
 // Validate input parameters
 if (accessKeyId == null || accessKeyId.isBlank() || secretAccessKey ==
null || secretAccessKey.isBlank()) {
 throw new IllegalArgumentException("Access Key ID and Secret Access
Key must not be null or empty");
 }

 AwsBasicCredentials awsCredentials =
AwsBasicCredentials.create(accessKeyId, secretAccessKey);
 return S3AsyncClient.builder()

.credentialsProvider(StaticCredentialsProvider.create(awsCredentials))
 .region(Region.US_WEST_2)
 .build();
 });
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 logger.info("");
 logger.info("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 logger.info("Continuing with the program...");
 logger.info("");
 break;
 } else {
 logger.info("Invalid input. Please try again.");
 }
 }
}
}
}

```

## A wrapper class for Amazon S3 SDK methods.

```
public class S3DirectoriesActions {

 private static IamAsyncClient iamAsyncClient;

 private static Ec2AsyncClient ec2AsyncClient;
 private static final Logger logger =
LoggerFactory.getLogger(S3DirectoriesActions.class);

 private static IamAsyncClient getIAMAsyncClient() {
 if (iamAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 iamAsyncClient = IamAsyncClient.builder()
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return iamAsyncClient;
 }

 private static Ec2AsyncClient getEc2AsyncClient() {
 if (ec2AsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))
 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();
 }
 }
}
```

```

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryStrategy(RetryMode.STANDARD)
 .build();

 ec2AsyncClient = Ec2AsyncClient.builder()
 .httpClient(httpClient)
 .region(Region.US_WEST_2)
 .overrideConfiguration(overrideConfig)
 .build();
 }
 return ec2AsyncClient;
}

/**
 * Deletes the specified S3 bucket and all the objects within it asynchronously.
 *
 * @param s3AsyncClient the S3 asynchronous client to use for the operations
 * @param bucketName the name of the S3 bucket to be deleted
 * @return a {@link CompletableFuture} that completes with a {@link
WaiterResponse} containing the
 * {@link HeadBucketResponse} when the bucket has been successfully
deleted
 * @throws CompletionException if there was an error deleting the bucket or its
objects
 */
public CompletableFuture<WaiterResponse<HeadBucketResponse>>
deleteBucketAndObjectsAsync(S3AsyncClient s3AsyncClient, String bucketName) {
 ListObjectsV2Request listRequest = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .build();

 return s3AsyncClient.listObjectsV2(listRequest)
 .thenCompose(listResponse -> {
 if (!listResponse.contents().isEmpty()) {
 List<ObjectIdentifier> objectIdentifiers =
listResponse.contents().stream()
 .map(s3Object ->
ObjectIdentifier.builder().key(s3Object.key()).build())
 .collect(Collectors.toList());

```



```

 DeleteObjectsRequest deleteRequest =
DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(Delete.builder().objects(objectIdentifiers).build())
 .build();

 return s3AsyncClient.deleteObjects(deleteRequest)
 .thenAccept(deleteResponse -> {
 if (!deleteResponse.errors().isEmpty()) {
 deleteResponse.errors().forEach(error ->
 logger.error("Couldn't delete object " +
error.key() + ". Reason: " + error.message()));
 }
 });
 }
 return CompletableFuture.completedFuture(null);
})
 .thenCompose(ignored -> {
 DeleteBucketRequest deleteBucketRequest =
DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();
 return s3AsyncClient.deleteBucket(deleteBucketRequest);
 })
 .thenCompose(ignored -> {
 S3AsyncWaiter waiter = s3AsyncClient.waiter();
 HeadBucketRequest headBucketRequest =
HeadBucketRequest.builder().bucket(bucketName).build();
 return waiter.waitUntilBucketNotExists(headBucketRequest);
 })
 .whenComplete((ignored, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof S3Exception) {
 throw new CompletionException("Error deleting bucket: " +
bucketName, cause);
 }
 throw new CompletionException("Failed to delete bucket and
objects: " + bucketName, exception);
 }
 logger.info("Bucket deleted successfully: " + bucketName);
 });
}

```

```

/**
 * Lists the objects in an S3 bucket asynchronously.
 *
 * @param s3Client the S3 async client to use for the operation
 * @param bucketName the name of the S3 bucket containing the objects to list
 * @return a {@link CompletableFuture} that contains the list of object keys in
the specified bucket
 */
public CompletableFuture<List<String>> listObjectsAsync(S3AsyncClient s3Client,
String bucketName) {
 ListObjectsV2Request request = ListObjectsV2Request.builder()
 .bucket(bucketName)
 .build();

 return s3Client.listObjectsV2(request)
 .thenApply(response -> response.contents().stream()
 .map(S3Object::key)
 .toList())
 .whenComplete((result, exception) -> {
 if (exception != null) {
 throw new CompletionException("Couldn't list objects in bucket:
" + bucketName, exception);
 }
 });
}

/**
 * Retrieves an object from an Amazon S3 bucket asynchronously.
 *
 * @param s3Client the S3 async client to use for the operation
 * @param bucketName the name of the S3 bucket containing the object
 * @param keyName the unique identifier (key) of the object to retrieve
 * @return a {@link CompletableFuture} that, when completed, contains the
object's content as a {@link ResponseBytes} of {@link GetObjectResponse}
 */
public CompletableFuture<ResponseBytes<GetObjectResponse>>
getObjectAsync(S3AsyncClient s3Client, String bucketName, String keyName) {
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .key(keyName)
 .bucket(bucketName)
 .build();

 // Get the object asynchronously and transform it into a byte array
 return s3Client.getObject(objectRequest, AsyncResponseTransformer.toBytes())

```

```

 .exceptionally(exception -> {
 Throwable cause = exception.getCause();
 if (cause instanceof NoSuchKeyException) {
 throw new CompletionException("Failed to get the object. Reason:
" + ((S3Exception) cause).awsErrorDetails().errorMessage(), cause);
 }
 throw new CompletionException("Failed to get the object",
exception);
 });
 }

 /**
 * Asynchronously copies an object from one S3 bucket to another.
 *
 * @param s3Client the S3 async client to use for the copy operation
 * @param sourceBucket the name of the source bucket
 * @param sourceKey the key of the object to be copied in the source
bucket
 * @param destinationBucket the name of the destination bucket
 * @param destinationKey the key of the copied object in the destination
bucket
 * @return a {@link CompletableFuture} that completes when the copy operation is
finished
 */
 public CompletableFuture<Void> copyObjectAsync(S3AsyncClient s3Client, String
sourceBucket, String sourceKey, String destinationBucket, String destinationKey) {
 CopyObjectRequest copyRequest = CopyObjectRequest.builder()
 .sourceBucket(sourceBucket)
 .sourceKey(sourceKey)
 .destinationBucket(destinationBucket)
 .destinationKey(destinationKey)
 .build();

 return s3Client.copyObject(copyRequest)
 .thenRun(() -> logger.info("Copied object '" + sourceKey + "' from
bucket '" + sourceBucket + "' to bucket '" + destinationBucket + "'"))
 .whenComplete((ignored, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof S3Exception) {
 throw new CompletionException("Couldn't copy object '" +
sourceKey + "' from bucket '" + sourceBucket + "' to bucket '" + destinationBucket
+ "'. Reason: " + ((S3Exception) cause).awsErrorDetails().errorMessage(), cause);
 }
 }
 });
 }

```

```

 throw new CompletionException("Failed to copy object",
exception);
 }
 });
}

/**
 * Asynchronously creates a session for the specified S3 bucket.
 *
 * @param s3Client the S3 asynchronous client to use for creating the session
 * @param bucketName the name of the S3 bucket for which to create the session
 * @return a {@link CompletableFuture} that completes when the session is
created, or throws a {@link CompletionException} if an error occurs
 */
public CompletableFuture<CreateSessionResponse> createSessionAsync(S3AsyncClient
s3Client, String bucketName) {
 CreateSessionRequest request = CreateSessionRequest.builder()
 .bucket(bucketName)
 .build();

 return s3Client.createSession(request)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof S3Exception) {
 throw new CompletionException("Couldn't create the session.
Reason: " + ((S3Exception) cause).awsErrorDetails().errorMessage(), cause);
 }
 throw new CompletionException("Unexpected error occurred while
creating session", exception);
 }
 logger.info("Created session for bucket: " + bucketName);
 });
}

/**
 * Creates a new S3 directory bucket in a specified Zone (For example, a
 * specified Availability Zone in this code example).
 *
 * @param s3Client The asynchronous S3 client used to create the bucket
 * @param bucketName The name of the bucket to be created
 * @param zone The Availability Zone where the bucket will be created
 * @throws CompletionException if there's an error creating the bucket

```

```

 */
 public CompletableFuture<CreateBucketResponse>
createDirectoryBucketAsync(S3AsyncClient s3Client, String bucketName, String zone)
{
 logger.info("Creating bucket: " + bucketName);

 CreateBucketConfiguration bucketConfiguration =
CreateBucketConfiguration.builder()
 .location(LocationInfo.builder()
 .type(LocationType.AVAILABILITY_ZONE)
 .name(zone)
 .build())
 .bucket(BucketInfo.builder()
 .type(BucketType.DIRECTORY)
 .dataRedundancy(DataRedundancy.SINGLE_AVAILABILITY_ZONE)
 .build())
 .build();

 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .createBucketConfiguration(bucketConfiguration)
 .build();

 return s3Client.createBucket(bucketRequest)
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof BucketAlreadyExistsException) {
 throw new CompletionException("The bucket already exists: "
+ ((S3Exception) cause).awsErrorDetails().errorMessage(), cause);
 }
 throw new CompletionException("Unexpected error occurred while
creating bucket", exception);
 }
 logger.info("Bucket created successfully with location: " +
response.location());
 });
}

/**
 * Creates an S3 bucket asynchronously.
 *
 * @param s3Client the S3 async client to use for the bucket creation
 * @param bucketName the name of the S3 bucket to create

```

```

 * @return a {@link CompletableFuture} that completes with the {@link
 WaiterResponse} containing the {@link HeadBucketResponse}
 * when the bucket is successfully created
 * @throws CompletionException if there's an error creating the bucket
 */
 public CompletableFuture<WaiterResponse<HeadBucketResponse>>
 createBucketAsync(S3AsyncClient s3Client, String bucketName) {
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 return s3Client.createBucket(bucketRequest)
 .thenCompose(response -> {
 S3AsyncWaiter s3Waiter = s3Client.waiter();
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();
 return s3Waiter.waitUntilBucketExists(bucketRequestWait);
 })
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof BucketAlreadyExistsException) {
 throw new CompletionException("The S3 bucket exists: " +
cause.getMessage(), cause);
 } else {
 throw new CompletionException("Failed to create access key:
" + exception.getMessage(), exception);
 }
 }
 logger.info(bucketName + " is ready");
 });
 }

 /**
 * Uploads an object to an Amazon S3 bucket asynchronously.
 *
 * @param s3Client the S3 async client to use for the upload
 * @param bucketName the destination S3 bucket name
 * @param bucketObject the name of the object to be uploaded
 * @param text the content to be uploaded as the object
 */
 public CompletableFuture<PutObjectResponse> putObjectAsync(S3AsyncClient
s3Client, String bucketName, String bucketObject, String text) {

```

```

 PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(bucketObject)
 .build();

 return s3Client.putObject(objectRequest, AsyncRequestBody.fromString(text))
 .whenComplete((response, exception) -> {
 if (exception != null) {
 Throwable cause = exception.getCause();
 if (cause instanceof NoSuchBucketException) {
 throw new CompletionException("The S3 bucket does not exist:
" + cause.getMessage(), cause);
 } else {
 throw new CompletionException("Failed to create access key:
" + exception.getMessage(), exception);
 }
 }
 });
 }

 /**
 * Creates an AWS IAM access key asynchronously for the specified user name.
 *
 * @param userName the name of the IAM user for whom to create the access key
 * @return a {@link CompletableFuture} that completes with the {@link
CreateAccessKeyResponse} containing the created access key
 */
 public CompletableFuture<CreateAccessKeyResponse> createAccessKeyAsync(String
userName) {
 CreateAccessKeyRequest request = CreateAccessKeyRequest.builder()
 .userName(userName)
 .build();

 return getIAMAsyncClient().createAccessKey(request)
 .whenComplete((response, exception) -> {
 if (response != null) {
 logger.info("Access Key Created.");
 } else {
 if (exception == null) {
 Throwable cause = exception.getCause();
 if (cause instanceof IamException) {
 throw new CompletionException("IAM error while creating
access key: " + cause.getMessage(), cause);
 } else {

```

```

 throw new CompletionException("Failed to create access
key: " + exception.getMessage(), exception);
 }
}

});
}

/**
 * Asynchronously selects an Availability Zone ID from the available EC2 zones.
 *
 * @return A {@link CompletableFuture} that resolves to the selected
Availability Zone ID.
 * @throws CompletionException if an error occurs during the request or
processing.
 */
public CompletableFuture<String> selectAvailabilityZoneIdAsync() {
 DescribeAvailabilityZonesRequest zonesRequest =
DescribeAvailabilityZonesRequest.builder()
 .build();

 return getEc2AsyncClient().describeAvailabilityZones(zonesRequest)
 .thenCompose(response -> {
 List<AvailabilityZone> zonesList = response.availabilityZones();
 if (zonesList.isEmpty()) {
 logger.info("No availability zones found.");
 return CompletableFuture.completedFuture(null); // Return null
if no zones are found
 }

 List<String> zoneIds = zonesList.stream()
 .map(AvailabilityZone::zoneId) // Get the zoneId (e.g., "usw2-
az1")

 .toList();

 return CompletableFuture.supplyAsync(() ->
promptUserForZoneSelection(zonesList, zoneIds))
 .thenApply(selectedZone -> {
 // Return only the selected Zone ID (e.g., "usw2-az1").
 return selectedZone.zoneId();
 });
 })
 .whenComplete((result, exception) -> {
 if (exception == null) {

```



```

 if (result != null) {
 logger.info("Selected Availability Zone ID: " + result);
 } else {
 logger.info("No availability zone selected.");
 }
 } else {
 Throwable cause = exception.getCause();
 if (cause instanceof Ec2Exception) {
 throw new CompletionException("EC2 error while selecting
availability zone: " + cause.getMessage(), cause);
 }
 throw new CompletionException("Failed to select availability
zone: " + exception.getMessage(), exception);
 }
});
}

/**
 * Prompts the user to select an Availability Zone from the given list.
 *
 * @param zonesList the list of Availability Zones
 * @param zoneIds the list of zone IDs
 * @return the selected Availability Zone
 */
private static AvailabilityZone
promptUserForZoneSelection(List<AvailabilityZone> zonesList, List<String> zoneIds)
{
 Scanner scanner = new Scanner(System.in);
 int index = -1;

 while (index < 0 || index >= zoneIds.size()) {
 logger.info("Select an availability zone:");
 IntStream.range(0, zoneIds.size()).forEach(i ->
 logger.info(i + ": " + zoneIds.get(i))
);

 logger.info("Enter the number corresponding to your choice: ");
 if (scanner.hasNextInt()) {
 index = scanner.nextInt();
 } else {
 scanner.next();
 }
 }
}

```

```

 AvailabilityZone selectedZone = zonesList.get(index);
 logger.info("You selected: " + selectedZone.zoneId());
 return selectedZone;
 }

 /**
 * Asynchronously sets up a new VPC, including creating the VPC, finding the
 associated route table, and
 * creating a VPC endpoint for the S3 service.
 *
 * @return a {@link CompletableFuture} that, when completed, contains a
 AbstractMap with the
 * VPC ID and VPC endpoint ID.
 */
 public CompletableFuture<AbstractMap.SimpleEntry<String, String>>
 setupVPCAsync() {
 String cidr = "10.0.0.0/16";
 CreateVpcRequest vpcRequest = CreateVpcRequest.builder()
 .cidrBlock(cidr)
 .build();

 return getEc2AsyncClient().createVpc(vpcRequest)
 .thenCompose(vpcResponse -> {
 String vpcId = vpcResponse.vpc().vpcId();
 logger.info("VPC Created: {}", vpcId);

 Ec2AsyncWaiter waiter = getEc2AsyncClient().waiter();
 DescribeVpcsRequest request = DescribeVpcsRequest.builder()
 .vpcIds(vpcId)
 .build();

 return waiter.waitUntilVpcAvailable(request)
 .thenApply(waiterResponse -> vpcId);
 })
 .thenCompose(vpcId -> {
 Filter filter = Filter.builder()
 .name("vpc-id")
 .values(vpcId)
 .build();

 DescribeRouteTablesRequest describeRouteTablesRequest =
 DescribeRouteTablesRequest.builder()
 .filters(filter)
 .build();

```

```

 return
getEc2AsyncClient().describeRouteTables(describeRouteTablesRequest)
 .thenApply(routeTablesResponse -> {
 if (routeTablesResponse.routeTables().isEmpty()) {
 throw new CompletionException("No route tables found for
VPC: " + vpcId, null);
 }
 String routeTableId =
routeTablesResponse.routeTables().get(0).routeTableId();
 logger.info("Route table found: {}", routeTableId);
 return new AbstractMap.SimpleEntry<>(vpcId, routeTableId);
 });
 })
 .thenCompose(vpcAndRouteTable -> {
 String vpcId = vpcAndRouteTable.getKey();
 String routeTableId = vpcAndRouteTable.getValue();
 Region region =
getEc2AsyncClient().serviceClientConfiguration().region();
 String serviceName = String.format("com.amazonaws.%s.s3express",
region.id());

 CreateVpcEndpointRequest endpointRequest =
CreateVpcEndpointRequest.builder()
 .vpcId(vpcId)
 .routeTableIds(routeTableId)
 .serviceName(serviceName)
 .build();

 return getEc2AsyncClient().createVpcEndpoint(endpointRequest)
 .thenApply(vpcEndpointResponse -> {
 String vpcEndpointId =
vpcEndpointResponse.vpcEndpoint().vpcEndpointId();
 logger.info("VPC Endpoint created: {}", vpcEndpointId);
 return new AbstractMap.SimpleEntry<>(vpcId, vpcEndpointId);
 });
 })
 .exceptionally(exception -> {
 Throwable cause = exception.getCause() != null ?
exception.getCause() : exception;
 if (cause instanceof Ec2Exception) {
 logger.error("EC2 error during VPC setup: {}",
cause.getMessage(), cause);

```

```
 throw new CompletionException("EC2 error during VPC setup: " +
 cause.getMessage(), cause);
 }

 logger.error("VPC setup failed: {}", cause.getMessage(), cause);
 throw new CompletionException("VPC setup failed: " +
 cause.getMessage(), cause);
 });
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CopyObject](#)
  - [CreateBucket](#)
  - [DeleteBucket](#)
  - [DeleteObject](#)
  - [GetObject](#)
  - [ListObjects](#)
  - [PutObject](#)

## Actions

### AbortMultipartUpload

The following code example shows how to use AbortMultipartUpload.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Abort a multipart upload in a directory bucket.

```
import org.slf4j.Logger;
```

```

import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.AbortMultipartUploadRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Aborts a specific multipart upload for the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be uploaded
 * @param uploadId The upload ID of the multipart upload to abort
 * @return True if the multipart upload is successfully aborted, false otherwise
 */
public static boolean abortDirectoryBucketMultipartUpload(S3Client s3Client,
String bucketName,
 String objectKey, String uploadId) {
 logger.info("Aborting multipart upload: {} for bucket: {}", uploadId,
bucketName);
 try {
 // Abort the multipart upload
 AbortMultipartUploadRequest abortMultipartUploadRequest =
AbortMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .uploadId(uploadId)
 .build();

 s3Client.abortMultipartUpload(abortMultipartUploadRequest);
 logger.info("Aborted multipart upload: {} for object: {}", uploadId,
objectKey);
 return true;
 } catch (S3Exception e) {
 logger.error("Failed to abort multipart upload: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 }
}

```

```
 return false;
 }
}
```

- For API details, see [AbortMultipartUpload](#) in *AWS SDK for Java 2.x API Reference*.

## CompleteMultipartUpload

The following code example shows how to use CompleteMultipartUpload.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Complete a multipart upload in a directory bucket.

```
import com.example.s3.util.S3DirectoryBucketUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CompleteMultipartUploadRequest;
import software.amazon.awssdk.services.s3.model.CompleteMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.CompletedMultipartUpload;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.io.IOException;
import java.nio.file.Path;
import java.util.List;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
```

```

import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static
 com.example.s3.util.S3DirectoryBucketUtils.multipartUploadForDirectoryBucket;

/**
 * This method completes the multipart upload request by collating all the
 * upload parts.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be uploaded
 * @param uploadId The upload ID used to track the multipart upload
 * @param uploadParts The list of completed parts
 * @return True if the multipart upload is successfully completed, false
 * otherwise
 */
public static boolean completeDirectoryBucketMultipartUpload(S3Client s3Client,
 String bucketName, String objectKey,
 String uploadId, List<CompletedPart> uploadParts) {
 try {
 CompletedMultipartUpload completedMultipartUpload =
 CompletedMultipartUpload.builder()
 .parts(uploadParts)
 .build();

 CompleteMultipartUploadRequest completeMultipartUploadRequest =
 CompleteMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .uploadId(uploadId)
 .multipartUpload(completedMultipartUpload)
 .build();

 CompleteMultipartUploadResponse response =
 s3Client.completeMultipartUpload(completeMultipartUploadRequest);
 logger.info("Multipart upload completed. ETag: {}", response.eTag());
 return true;
 } catch (S3Exception e) {
 logger.error("Failed to complete multipart upload: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 }
}

```

```
 return false;
 }
}
```

- For API details, see [CompleteMultipartUpload](#) in *AWS SDK for Java 2.x API Reference*.

## CopyObject

The following code example shows how to use CopyObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Copy an object from a directory bucket to a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CopyObjectRequest;
import software.amazon.awssdk.services.s3.model.CopyObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Copies an object from one S3 general purpose bucket to one S3 directory
```



```

 * bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param sourceBucket The name of the source bucket
 * @param objectKey The key (name) of the object to be copied
 * @param targetBucket The name of the target bucket
 */
 public static void copyDirectoryBucketObject(S3Client s3Client, String
sourceBucket, String objectKey,
 String targetBucket) {
 logger.info("Copying object: {} from bucket: {} to bucket: {}", objectKey,
sourceBucket, targetBucket);

 try {
 // Create a CopyObjectRequest
 CopyObjectRequest copyReq = CopyObjectRequest.builder()
 .sourceBucket(sourceBucket)
 .sourceKey(objectKey)
 .destinationBucket(targetBucket)
 .destinationKey(objectKey)
 .build();

 // Copy the object
 CopyObjectResponse copyRes = s3Client.copyObject(copyReq);
 logger.info("Successfully copied {} from bucket {} into bucket {}.
CopyObjectResponse: {}",
 objectKey, sourceBucket, targetBucket,
copyRes.copyObjectResult().toString());

 } catch (S3Exception e) {
 logger.error("Failed to copy object: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
 }
}

```

- For API details, see [CopyObject](#) in *AWS SDK for Java 2.x API Reference*.

## CreateBucket

The following code example shows how to use CreateBucket.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create an S3 directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.BucketInfo;
import software.amazon.awssdk.services.s3.model.BucketType;
import software.amazon.awssdk.services.s3.model.CreateBucketConfiguration;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.CreateBucketResponse;
import software.amazon.awssdk.services.s3.model.DataRedundancy;
import software.amazon.awssdk.services.s3.model.LocationInfo;
import software.amazon.awssdk.services.s3.model.LocationType;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Creates a new S3 directory bucket in a specified Zone (For example, a
 * specified Availability Zone in this code example).
 *
 * @param s3Client The S3 client used to create the bucket
 * @param bucketName The name of the bucket to be created
 * @param zone The region where the bucket will be created
 * @throws S3Exception if there's an error creating the bucket
 */
public static void createDirectoryBucket(S3Client s3Client, String bucketName,
String zone) throws S3Exception {
 logger.info("Creating bucket: {}", bucketName);

 CreateBucketConfiguration bucketConfiguration =
CreateBucketConfiguration.builder()
```

```

 .location(LocationInfo.builder()
 .type(LocationType.AVAILABILITY_ZONE)
 .name(zone).build())
 .bucket(BucketInfo.builder()
 .type(BucketType.DIRECTORY)
 .dataRedundancy(DataRedundancy.SINGLE_AVAILABILITY_ZONE)
 .build())
 .build();
 try {
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .createBucketConfiguration(bucketConfiguration).build();
 CreateBucketResponse response = s3Client.createBucket(bucketRequest);
 logger.info("Bucket created successfully with location: {}",
response.location());
 } catch (S3Exception e) {
 logger.error("Error creating bucket: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [CreateBucket](#) in *AWS SDK for Java 2.x API Reference*.

## CreateMultipartUpload

The following code example shows how to use `CreateMultipartUpload`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create a multipart upload in a directory bucket.

```

import com.example.s3.util.S3DirectoryBucketUtils;
import org.slf4j.Logger;

```

```

import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadRequest;
import software.amazon.awssdk.services.s3.model.CreateMultipartUploadResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * This method creates a multipart upload request that generates a unique upload
 * ID used to track
 * all the upload parts.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be uploaded
 * @return The upload ID used to track the multipart upload
 */
public static String createDirectoryBucketMultipartUpload(S3Client s3Client,
String bucketName, String objectKey) {
 logger.info("Creating multipart upload for object: {} in bucket: {}",
objectKey, bucketName);

 try {
 // Create a CreateMultipartUploadRequest
 CreateMultipartUploadRequest createMultipartUploadRequest =
CreateMultipartUploadRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 // Initiate the multipart upload
 CreateMultipartUploadResponse response =
s3Client.createMultipartUpload(createMultipartUploadRequest);
 String uploadId = response.uploadId();
 logger.info("Multipart upload initiated. Upload ID: {}", uploadId);
 return uploadId;

 } catch (S3Exception e) {
 logger.error("Failed to create multipart upload: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),

```

```

 e.awsErrorDetails().errorCode(), e);
 throw e;
}
}

```

- For API details, see [CreateMultipartUpload](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucket

The following code example shows how to use DeleteBucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete an S3 directory bucket.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;

/**
 * Deletes the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket to delete
 */
public static void deleteDirectoryBucket(S3Client s3Client, String bucketName) {
 logger.info("Deleting bucket: {}", bucketName);
}

```

```
try {
 // Create a DeleteBucketRequest
 DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();

 // Delete the bucket
 s3Client.deleteBucket(deleteBucketRequest);
 logger.info("Successfully deleted bucket: {}", bucketName);

} catch (S3Exception e) {
 logger.error("Failed to delete bucket: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
}
```

- For API details, see [DeleteBucket](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucketEncryption

The following code example shows how to use DeleteBucketEncryption.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete the encryption configuration for a directory bucket.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteBucketEncryptionRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Deletes the encryption configuration from an S3 bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 */
public static void deleteDirectoryBucketEncryption(S3Client s3Client, String
bucketName) {
 DeleteBucketEncryptionRequest deleteRequest =
DeleteBucketEncryptionRequest.builder()
 .bucket(bucketName)
 .build();

 try {
 s3Client.deleteBucketEncryption(deleteRequest);
 logger.info("Bucket encryption deleted for bucket: {}", bucketName);
 } catch (S3Exception e) {
 logger.error("Failed to delete bucket encryption: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [DeleteBucketEncryption](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteBucketPolicy

The following code example shows how to use DeleteBucketPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Delete a bucket policy for a directory bucket.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteBucketPolicyRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getAwsAccountId;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketPolicy;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

/**
 * Deletes the bucket policy for the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 */
public static void deleteDirectoryBucketPolicy(S3Client s3Client, String
bucketName) {
 logger.info("Deleting policy for bucket: {}", bucketName);

 try {
 // Create a DeleteBucketPolicyRequest
 DeleteBucketPolicyRequest deletePolicyReq =
DeleteBucketPolicyRequest.builder()
 .bucket(bucketName)
 .build();

 // Delete the bucket policy
 s3Client.deleteBucketPolicy(deletePolicyReq);
 logger.info("Successfully deleted bucket policy");

 } catch (S3Exception e) {
 logger.error("Failed to delete bucket policy: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}
```



```
}
}
```

- For API details, see [DeleteBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteObject

The following code example shows how to use DeleteObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Delete an object in a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Deletes an object from the specified S3 directory bucket. */
```

```

*
* @param s3Client The S3 client used to interact with S3
* @param bucketName The name of the directory bucket
* @param objectKey The key (name) of the object to be deleted
*/
public static void deleteDirectoryBucketObject(S3Client s3Client, String
bucketName, String objectKey) {
 logger.info("Deleting object: {} from bucket: {}", objectKey, bucketName);

 try {
 // Create a DeleteObjectRequest
 DeleteObjectRequest deleteObjectRequest = DeleteObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 // Delete the object
 s3Client.deleteObject(deleteObjectRequest);
 logger.info("Object {} has been deleted", objectKey);

 } catch (S3Exception e) {
 logger.error("Failed to delete object: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [DeleteObject](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteObjects

The following code example shows how to use DeleteObjects.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Delete multiple objects in a directory bucket.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.Delete;
import software.amazon.awssdk.services.s3.model.DeleteObjectsRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectsResponse;
import software.amazon.awssdk.services.s3.model.ObjectIdentifier;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.net.URISyntaxException;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Deletes multiple objects from the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKeys The list of keys (names) of the objects to be deleted
 */
public static void deleteDirectoryBucketObjects(S3Client s3Client, String
bucketName, List<String> objectKeys) {
 logger.info("Deleting objects from bucket: {}", bucketName);

 try {
 // Create a list of ObjectIdentifier.
 List<ObjectIdentifier> identifiers = objectKeys.stream()
 .map(key -> ObjectIdentifier.builder().key(key).build())
 .toList();

 Delete delete = Delete.builder()
```

```

 .objects(identifiers)
 .build();

 DeleteObjectsRequest deleteObjectsRequest =
DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(delete)
 .build();

 DeleteObjectsResponse deleteObjectsResponse =
s3Client.deleteObjects(deleteObjectsRequest);
 deleteObjectsResponse.deleted().forEach(deleted -> logger.info("Deleted
object: {}", deleted.key()));

 } catch (S3Exception e) {
 logger.error("Failed to delete objects: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [DeleteObjects](#) in *AWS SDK for Java 2.x API Reference*.

## GetBucketEncryption

The following code example shows how to use GetBucketEncryption.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get the encryption configuration of a directory bucket.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

```

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetBucketEncryptionRequest;
import software.amazon.awssdk.services.s3.model.GetBucketEncryptionResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.ServerSideEncryptionRule;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Retrieves the encryption configuration for an S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @return The type of server-side encryption applied to the bucket (e.g.,
 * AES256, aws:kms)
 */
public static String getDirectoryBucketEncryption(S3Client s3Client, String
bucketName) {
 try {
 // Create a GetBucketEncryptionRequest
 GetBucketEncryptionRequest getRequest =
GetBucketEncryptionRequest.builder()
 .bucket(bucketName)
 .build();

 // Retrieve the bucket encryption configuration
 GetBucketEncryptionResponse response =
s3Client.getBucketEncryption(getRequest);
 ServerSideEncryptionRule rule =
response.getServerSideEncryptionConfiguration().rules().get(0);

 String encryptionType =
rule.applyServerSideEncryptionByDefault().sseAlgorithmAsString();
 logger.info("Bucket encryption algorithm: {}", encryptionType);
 logger.info("KMS Customer Managed Key ID: {}",
rule.applyServerSideEncryptionByDefault().kmsMasterKeyID());
 logger.info("Bucket Key Enabled: {}", rule.bucketKeyEnabled());

 return encryptionType;
 } catch (S3Exception e) {

```

```
 logger.error("Failed to get bucket encryption: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}
```

- For API details, see [GetBucketEncryption](#) in *AWS SDK for Java 2.x API Reference*.

## GetBucketPolicy

The following code example shows how to use `GetBucketPolicy`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get the policy of a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetBucketPolicyRequest;
import software.amazon.awssdk.services.s3.model.GetBucketPolicyResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getAwsAccountId;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketPolicy;

/**
 * Retrieves the bucket policy for the specified S3 directory bucket.
```

```

*
* @param s3Client The S3 client used to interact with S3
* @param bucketName The name of the directory bucket
* @return The bucket policy text
*/
public static String getDirectoryBucketPolicy(S3Client s3Client, String
bucketName) {
 logger.info("Getting policy for bucket: {}", bucketName);

 try {
 // Create a GetBucketPolicyRequest
 GetBucketPolicyRequest policyReq = GetBucketPolicyRequest.builder()
 .bucket(bucketName)
 .build();

 // Retrieve the bucket policy
 GetBucketPolicyResponse response = s3Client.getBucketPolicy(policyReq);

 // Print and return the policy text
 String policyText = response.policy();
 logger.info("Bucket policy: {}", policyText);
 return policyText;
 } catch (S3Exception e) {
 logger.error("Failed to get bucket policy: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [GetBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## GetObject

The following code example shows how to use `GetObject`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get an object from a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.ResponseBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.nio.charset.StandardCharsets;
import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Retrieves an object from the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be retrieved
 * @return The retrieved object as a ResponseInputStream
 */
public static boolean getDirectoryBucketObject(S3Client s3Client, String
bucketName, String objectKey) {
 logger.info("Retrieving object: {} from bucket: {}", objectKey, bucketName);
```



```
try {
 // Create a GetObjectRequest
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .key(objectKey)
 .bucket(bucketName)
 .build();

 // Retrieve the object as bytes
 ResponseBytes<GetObjectResponse> objectBytes =
s3Client.getObjectAsBytes(objectRequest);
 byte[] data = objectBytes.asByteArray();

 // Print object contents to console
 String objectContent = new String(data, StandardCharsets.UTF_8);
 logger.info("Object contents: \n{}", objectContent);

 return true;

} catch (S3Exception e) {
 logger.error("Failed to retrieve object: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 return false;
}
}
```

- For API details, see [GetObject](#) in *AWS SDK for Java 2.x API Reference*.

## GetObjectAttributes

The following code example shows how to use `GetObjectAttributes`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get an object attributes from a directory bucket.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectAttributesRequest;
import software.amazon.awssdk.services.s3.model.GetObjectAttributesResponse;
import software.amazon.awssdk.services.s3.model.ObjectAttributes;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Retrieves attributes for an object in the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to retrieve attributes for
 * @return True if the object attributes are successfully retrieved, false
 * otherwise
 */
public static boolean getDirectoryBucketObjectAttributes(S3Client s3Client,
String bucketName, String objectKey) {
 logger.info("Retrieving attributes for object: {} from bucket: {}",
objectKey, bucketName);

 try {
 // Create a GetObjectAttributesRequest
 GetObjectAttributesRequest getObjectAttributesRequest =
GetObjectAttributesRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .objectAttributes(ObjectAttributes.E_TAG,
ObjectAttributes.STORAGE_CLASS,
ObjectAttributes.OBJECT_SIZE)

```

```

 .build();

 // Retrieve the object attributes
 GetObjectAttributesResponse response =
s3Client.getObjectAttributes(getObjectAttributesRequest);
 logger.info("Attributes for object {}: ", objectKey);
 logger.info("ETag: {}", response.eTag());
 logger.info("Storage Class: {}", response.storageClass());
 logger.info("Object Size: {}", response.objectSize());
 return true;

 } catch (S3Exception e) {
 logger.error("Failed to retrieve object attributes: {} - Error code:
{}",
 e.awsErrorDetails().errorMessage(),
e.awsErrorDetails().errorCode(), e);
 return false;
 }
}

```

- For API details, see [GetObjectAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## HeadBucket

The following code example shows how to use HeadBucket.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Checks if the specified S3 directory bucket exists and is accessible.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;

```

```

import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.HeadBucketRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Checks if the specified S3 directory bucket exists and is accessible.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket to check
 * @return True if the bucket exists and is accessible, false otherwise
 */
public static boolean headDirectoryBucket(S3Client s3Client, String bucketName)
{
 logger.info("Checking if bucket exists: {}", bucketName);

 try {
 // Create a HeadBucketRequest
 HeadBucketRequest headBucketRequest = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();
 // If the bucket doesn't exist, the following statement throws
 NoSuchBucketException,
 // which is a subclass of S3Exception.
 s3Client.headBucket(headBucketRequest);
 logger.info("Amazon S3 directory bucket: \"{}\" found.", bucketName);
 return true;

 } catch (S3Exception e) {
 logger.error("Failed to access bucket: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [HeadBucket](#) in *AWS SDK for Java 2.x API Reference*.

## HeadObject

The following code example shows how to use HeadObject.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Get metadata of an object in a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Retrieves metadata for an object in the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to retrieve metadata for
 * @return True if the object exists, false otherwise
 */
public static boolean headDirectoryBucketObject(S3Client s3Client, String
bucketName, String objectKey) {
```

```
logger.info("Retrieving metadata for object: {} from bucket: {}", objectKey,
bucketName);

try {
 // Create a HeadObjectRequest
 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 // Retrieve the object metadata
 HeadObjectResponse response = s3Client.headObject(headObjectRequest);
 logger.info("Amazon S3 object: \"{}\" found in bucket: \"{}\" with ETag:
\"{}\"", objectKey, bucketName,
 response.eTag());
 logger.info("Content-Type: {}", response.contentType());
 logger.info("Content-Length: {}", response.contentLength());
 logger.info("Last Modified: {}", response.lastModified());
 return true;

} catch (S3Exception e) {
 logger.error("Failed to retrieve object metadata: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 return false;
}
}
```

- For API details, see [HeadObject](#) in *AWS SDK for Java 2.x API Reference*.

## ListDirectoryBuckets

The following code example shows how to use ListDirectoryBuckets.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## List all directory buckets.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.Bucket;
import software.amazon.awssdk.services.s3.model.ListDirectoryBucketsRequest;
import software.amazon.awssdk.services.s3.model.ListDirectoryBucketsResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;

/**
 * Lists all S3 directory buckets and no general purpose buckets.
 *
 * @param s3Client The S3 client used to interact with S3
 * @return A list of bucket names
 */
public static List<String> listDirectoryBuckets(S3Client s3Client) {
 logger.info("Listing all directory buckets");

 try {
 // Create a ListBucketsRequest
 ListDirectoryBucketsRequest listDirectoryBucketsRequest =
 ListDirectoryBucketsRequest.builder().build();

 // Retrieve the list of buckets
 ListDirectoryBucketsResponse response =
 s3Client.listDirectoryBuckets(listDirectoryBucketsRequest);

 // Extract bucket names
 List<String> bucketNames = response.buckets().stream()
 .map(Bucket::name)
 .collect(Collectors.toList());

 return bucketNames;
 }
}
```

```
 } catch (S3Exception e) {
 logger.error("Failed to list buckets: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode());
 throw e;
 }
}
```

- For API details, see [ListDirectoryBuckets](#) in *AWS SDK for Java 2.x API Reference*.

## ListMultipartUploads

The following code example shows how to use `ListMultipartUploads`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List multipart uploads in a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsRequest;
import software.amazon.awssdk.services.s3.model.ListMultipartUploadsResponse;
import software.amazon.awssdk.services.s3.model.MultipartUpload;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.io.IOException;
import java.nio.file.Path;
import java.util.List;

import static
 com.example.s3.util.S3DirectoryBucketUtils.abortDirectoryBucketMultipartUploads;
import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
```



```

import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static
 com.example.s3.util.S3DirectoryBucketUtils.multipartUploadForDirectoryBucket;

/**
 * Lists multipart uploads for the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @return A list of MultipartUpload objects representing the multipart uploads
 */
public static List<MultipartUpload> listDirectoryBucketMultipartUploads(S3Client
s3Client, String bucketName) {
 logger.info("Listing in-progress multipart uploads for bucket: {}",
bucketName);

 try {
 // Create a ListMultipartUploadsRequest
 ListMultipartUploadsRequest listMultipartUploadsRequest =
ListMultipartUploadsRequest.builder()
 .bucket(bucketName)
 .build();

 // List the multipart uploads
 ListMultipartUploadsResponse response =
s3Client.listMultipartUploads(listMultipartUploadsRequest);
 List<MultipartUpload> uploads = response.uploads();
 for (MultipartUpload upload : uploads) {
 logger.info("In-progress multipart upload: Upload ID: {}, Key: {},
Initiated: {}", upload.uploadId(),
 upload.key(), upload.initiated());
 }
 return uploads;
 } catch (S3Exception e) {
 logger.error("Failed to list multipart uploads: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode());
 return List.of(); // Return an empty list if an exception is thrown
 }
}

```

```
}
}
```

- For API details, see [ListMultipartUploads](#) in *AWS SDK for Java 2.x API Reference*.

## ListObjectsV2

The following code example shows how to use ListObjectsV2.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List objects in a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Response;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.S3Object;

import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;
```

```

/**
 * Lists objects in the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @return A list of object keys in the bucket
 */
public static List<String> listDirectoryBucketObjectsV2(S3Client s3Client,
String bucketName) {
 logger.info("Listing objects in bucket: {}", bucketName);

 try {
 // Create a ListObjectsV2Request
 ListObjectsV2Request listObjectsV2Request =
ListObjectsV2Request.builder()
 .bucket(bucketName)
 .build();

 // Retrieve the list of objects
 ListObjectsV2Response response =
s3Client.listObjectsV2(listObjectsV2Request);

 // Extract and return the object keys
 return response.contents().stream()
 .map(S3Object::key)
 .collect(Collectors.toList());

 } catch (S3Exception e) {
 logger.error("Failed to list objects: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
e.awsErrorDetails().errorCode());
 throw e;
 }
}

```

- For API details, see [ListObjectsV2](#) in *AWS SDK for Java 2.x API Reference*.

## ListParts

The following code example shows how to use ListParts.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

List parts of a multipart upload in a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.ListPartsRequest;
import software.amazon.awssdk.services.s3.model.ListPartsResponse;
import software.amazon.awssdk.services.s3.model.Part;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.io.IOException;
import java.nio.file.Path;
import java.util.List;

import static
 com.example.s3.util.S3DirectoryBucketUtils.abortDirectoryBucketMultipartUploads;
import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static
 com.example.s3.util.S3DirectoryBucketUtils.multipartUploadForDirectoryBucket;

/**
 * Lists the parts of a multipart upload for the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object being uploaded
 * @param uploadId The upload ID used to track the multipart upload
 * @return A list of Part representing the parts of the multipart upload
```

```

 */
 public static List<Part> listDirectoryBucketMultipartUploadParts(S3Client
s3Client, String bucketName,
 String objectKey, String uploadId) {
 logger.info("Listing parts for object: {} in bucket: {}", objectKey,
bucketName);

 try {
 // Create a ListPartsRequest
 ListPartsRequest listPartsRequest = ListPartsRequest.builder()
 .bucket(bucketName)
 .uploadId(uploadId)
 .key(objectKey)
 .build();

 // List the parts of the multipart upload
 ListPartsResponse response = s3Client.listParts(listPartsRequest);
 List<Part> parts = response.parts();
 for (Part part : parts) {
 logger.info("Uploaded part: Part number = \"{}\", etag = {}",
part.partNumber(), part.eTag());
 }
 return parts;

 } catch (S3Exception e) {
 logger.error("Failed to list parts: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode());
 return List.of(); // Return an empty list if an exception is thrown
 }
 }
}

```

- For API details, see [ListParts](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketEncryption

The following code example shows how to use PutBucketEncryption.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Set bucket encryption to a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.kms.KmsClient;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutBucketEncryptionRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.ServerSideEncryption;
import software.amazon.awssdk.services.s3.model.ServerSideEncryptionByDefault;
import software.amazon.awssdk.services.s3.model.ServerSideEncryptionConfiguration;
import software.amazon.awssdk.services.s3.model.ServerSideEncryptionRule;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createKmsClient;
import static com.example.s3.util.S3DirectoryBucketUtils.createKmsKey;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.scheduleKeyDeletion;

/**
 * Sets the default encryption configuration for an S3 bucket as SSE-KMS.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param kmsKeyId The ID of the customer-managed KMS key
 */
public static void putDirectoryBucketEncryption(S3Client s3Client, String
bucketName, String kmsKeyId) {
 // Define the default encryption configuration to use SSE-KMS. For directory
 // buckets, AWS managed KMS keys aren't supported. Only customer-managed
keys
 // are supported.
```

```

 ServerSideEncryptionByDefault encryptionByDefault =
ServerSideEncryptionByDefault.builder()
 .sseAlgorithm(ServerSideEncryption.AWS_KMS)
 .kmsMasterKeyId(kmsKeyId)
 .build();

 // Create a server-side encryption rule to apply the default encryption
 // configuration. For directory buckets, the bucketKeyEnabled field is
enforced
 // to be true.
 ServerSideEncryptionRule rule = ServerSideEncryptionRule.builder()
 .bucketKeyEnabled(true)
 .applyServerSideEncryptionByDefault(encryptionByDefault)
 .build();

 // Create the server-side encryption configuration for the bucket
 ServerSideEncryptionConfiguration encryptionConfiguration =
ServerSideEncryptionConfiguration.builder()
 .rules(rule)
 .build();

 // Create the PutBucketEncryption request
 PutBucketEncryptionRequest putRequest = PutBucketEncryptionRequest.builder()
 .bucket(bucketName)
 .serverSideEncryptionConfiguration(encryptionConfiguration)
 .build();

 // Set the bucket encryption
 try {
 s3Client.putBucketEncryption(putRequest);
 logger.info("SSE-KMS Bucket encryption configuration set for the
directory bucket: {}", bucketName);
 } catch (S3Exception e) {
 logger.error("Failed to set bucket encryption: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode());
 throw e;
 }
}

```

- For API details, see [PutBucketEncryption](#) in *AWS SDK for Java 2.x API Reference*.

## PutBucketPolicy

The following code example shows how to use PutBucketPolicy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Apply a bucket policy to a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutBucketPolicyRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getAwsAccountId;

/**
 * Sets the following bucket policy for the specified S3 directory bucket.
 * <pre>
 * {
 * "Version": "2012-10-17",
 * "Statement": [
 * {
 * "Sid": "AdminPolicy",
 * "Effect": "Allow",
 * "Principal": {
 * "AWS": "arn:aws:iam::<ACCOUNT_ID>:root"
 * },
 * "Action": "s3express:*",
 * "Resource": "arn:aws:s3express:us-west-2:<ACCOUNT_ID>:bucket/
 * <DIR_BUCKET_NAME>
 * }
 *]
 * }
```



```

*]
* }
* </pre>
* This policy grants all S3 directory bucket actions to identities in the same
account as the bucket.
*
* @param s3Client The S3 client used to interact with S3
* @param bucketName The name of the directory bucket
* @param policyText The policy text to be applied
*/
public static void putDirectoryBucketPolicy(S3Client s3Client, String
bucketName, String policyText) {
 logger.info("Setting policy on bucket: {}", bucketName);
 logger.info("Policy: {}", policyText);

 try {
 PutBucketPolicyRequest policyReq = PutBucketPolicyRequest.builder()
 .bucket(bucketName)
 .policy(policyText)
 .build();

 s3Client.putBucketPolicy(policyReq);
 logger.info("Bucket policy set successfully!");

 } catch (S3Exception e) {
 logger.error("Failed to set bucket policy: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}

```

- For API details, see [PutBucketPolicy](#) in *AWS SDK for Java 2.x API Reference*.

## PutObject

The following code example shows how to use PutObject.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Put an object into a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.awscore.exception.AwsErrorDetails;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;

import java.io.UncheckedIOException;
import java.nio.file.Path;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;

/**
 * Puts an object into the specified S3 directory bucket.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be placed in the bucket
 * @param filePath The path of the file to be uploaded
 */
public static void putDirectoryBucketObject(S3Client s3Client, String
bucketName, String objectKey, Path filePath) {
 logger.info("Putting object: {} into bucket: {}", objectKey, bucketName);

 try {
 // Create a PutObjectRequest
```

```

 PutObjectRequest putObj = PutObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 // Upload the object
 s3Client.putObject(putObj, filePath);
 logger.info("Successfully placed {} into bucket {}", objectKey,
bucketName);

 } catch (UncheckedIOException e) {
 throw S3Exception.builder().message("Failed to read the file: " +
e.getMessage()).cause(e)
 .awsErrorDetails(AwsErrorDetails.builder()
 .errorCode("ClientSideException:FailedToReadFile")
 .errorMessage(e.getMessage())
 .build())
 .build();
 } catch (S3Exception e) {
 logger.error("Failed to put object: {}", e.getMessage(), e);
 throw e;
 }
}

```

- For API details, see [PutObject](#) in *AWS SDK for Java 2.x API Reference*.

## UploadPart

The following code example shows how to use UploadPart.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Upload part of a multipart upload for a directory bucket.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.UploadPartRequest;
import software.amazon.awssdk.services.s3.model.UploadPartResponse;

import java.io.IOException;
import java.io.RandomAccessFile;
import java.nio.ByteBuffer;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

import static
 com.example.s3.util.S3DirectoryBucketUtils.abortDirectoryBucketMultipartUploads;
import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;

/**
 * This method creates part requests and uploads individual parts to S3.
 * While it uses the UploadPart API to upload a single part, it does so
 * sequentially to handle multiple parts of a file, returning all the completed
 * parts.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to be uploaded
 * @param uploadId The upload ID used to track the multipart upload
 * @param filePath The path to the file to be uploaded
 * @return A list of uploaded parts
 * @throws IOException if an I/O error occurs
 */
public static List<CompletedPart> multipartUploadForDirectoryBucket(S3Client
s3Client, String bucketName,

```

```
String objectKey, String uploadId, Path filePath) throws IOException {
 logger.info("Uploading parts for object: {} in bucket: {}", objectKey,
bucketName);

 int partNumber = 1;
 List<CompletedPart> uploadedParts = new ArrayList<>();
 ByteBuffer bb = ByteBuffer.allocate(1024 * 1024 * 5); // 5 MB byte buffer

 // Read the local file, break down into chunks and process
 try (RandomAccessFile file = new RandomAccessFile(filePath.toFile(), "r")) {
 long fileSize = file.length();
 int position = 0;

 // Sequentially upload parts of the file
 while (position < fileSize) {
 file.seek(position);
 int read = file.getChannel().read(bb);

 bb.flip(); // Swap position and limit before reading from the buffer
 UploadPartRequest uploadPartRequest = UploadPartRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .uploadId(uploadId)
 .partNumber(partNumber)
 .build();

 UploadPartResponse partResponse = s3Client.uploadPart(
 uploadPartRequest,
 RequestBody.fromByteBuffer(bb));

 // Build the uploaded part
 CompletedPart uploadedPart = CompletedPart.builder()
 .partNumber(partNumber)
 .eTag(partResponse.eTag())
 .build();

 // Add the uploaded part to the list
 uploadedParts.add(uploadedPart);

 // Log to indicate the part upload is done
 logger.info("Uploaded part number: {} with ETag: {}", partNumber,
partResponse.eTag());

 bb.clear();
 }
 }
}
```

```
 position += read;
 partNumber++;
 }
} catch (S3Exception e) {
 logger.error("Failed to list parts: {} - Error code: {}",
e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode());
 throw e;
}
return uploadedParts;
}
```

- For API details, see [UploadPart](#) in *AWS SDK for Java 2.x API Reference*.

## UploadPartCopy

The following code example shows how to use UploadPartCopy.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create copy parts based on source object size and copy over individual parts to a directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CompletedPart;
import software.amazon.awssdk.services.s3.model.HeadObjectRequest;
import software.amazon.awssdk.services.s3.model.HeadObjectResponse;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.model.UploadPartCopyRequest;
import software.amazon.awssdk.services.s3.model.UploadPartCopyResponse;
```

```

import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

import static
 com.example.s3.util.S3DirectoryBucketUtils.abortDirectoryBucketMultipartUploads;
import static
 com.example.s3.util.S3DirectoryBucketUtils.completeDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static
 com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucketMultipartUpload;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static
 com.example.s3.util.S3DirectoryBucketUtils.multipartUploadForDirectoryBucket;

/**
 * Creates copy parts based on source object size and copies over individual
 * parts.
 *
 * @param s3Client The S3 client used to interact with S3
 * @param sourceBucket The name of the source bucket
 * @param sourceKey The key (name) of the source object
 * @param destinationBucket The name of the destination bucket
 * @param destinationKey The key (name) of the destination object
 * @param uploadId The upload ID used to track the multipart upload
 * @return A list of completed parts
 */
public static List<CompletedPart> multipartUploadCopyForDirectoryBucket(S3Client
s3Client, String sourceBucket,
 String sourceKey, String destinationBucket, String destinationKey,
String uploadId) {
 // Get the object size to track the end of the copy operation
 HeadObjectRequest headObjectRequest = HeadObjectRequest.builder()
 .bucket(sourceBucket)
 .key(sourceKey)
 .build();
 HeadObjectResponse headObjectResponse =
s3Client.headObject(headObjectRequest);
 long objectSize = headObjectResponse.contentLength();

```

```

 logger.info("Source Object size: {}", objectSize);

 // Copy the object using 20 MB parts
 long partSize = 20 * 1024 * 1024; // 20 MB
 long bytePosition = 0;
 int partNum = 1;
 List<CompletedPart> uploadedParts = new ArrayList<>();

 while (bytePosition < objectSize) {
 long lastByte = Math.min(bytePosition + partSize - 1, objectSize - 1);
 logger.info("Part Number: {}, Byte Position: {}, Last Byte: {}",
partNum, bytePosition, lastByte);

 try {
 UploadPartCopyRequest uploadPartCopyRequest =
UploadPartCopyRequest.builder()
 .sourceBucket(sourceBucket)
 .sourceKey(sourceKey)
 .destinationBucket(destinationBucket)
 .destinationKey(destinationKey)
 .uploadId(uploadId)
 .copySourceRange("bytes=" + bytePosition + "-" + lastByte)
 .partNumber(partNum)
 .build();
 UploadPartCopyResponse uploadPartCopyResponse =
s3Client.uploadPartCopy(uploadPartCopyRequest);

 CompletedPart part = CompletedPart.builder()
 .partNumber(partNum)
 .eTag(uploadPartCopyResponse.copyPartResult().eTag())
 .build();
 uploadedParts.add(part);

 bytePosition += partSize;
 partNum++;
 } catch (S3Exception e) {
 logger.error("Failed to copy part number {}: {} - Error code: {}",
partNum,
 e.awsErrorDetails().errorMessage(),
e.awsErrorDetails().errorCode());
 throw e;
 }
 }
 }
}

```



```
 return uploadedParts;
 }
```

- For API details, see [UploadPartCopy](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a presigned URL to get an object

The following code example shows how to create a presigned URL for S3 directory buckets and get an object.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Generate a presigned GET URL for accessing an object in an S3 directory bucket.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.S3Exception;
import software.amazon.awssdk.services.s3.presigner.S3Presigner;
import software.amazon.awssdk.services.s3.presigner.model.GetObjectPresignRequest;
import software.amazon.awssdk.services.s3.presigner.model.PresignedGetObjectRequest;

import java.nio.file.Path;
import java.time.Duration;

import static com.example.s3.util.S3DirectoryBucketUtils.createDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Client;
import static com.example.s3.util.S3DirectoryBucketUtils.createS3Presigner;
```

```

import static
 com.example.s3.util.S3DirectoryBucketUtils.deleteAllObjectsInDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.deleteDirectoryBucket;
import static com.example.s3.util.S3DirectoryBucketUtils.getFilePath;
import static com.example.s3.util.S3DirectoryBucketUtils.putDirectoryBucketObject;

/**
 * Generates a presigned URL for accessing an object in the specified S3
 * directory bucket.
 *
 * @param s3Presigner The S3 presigner client used to generate the presigned URL
 * @param bucketName The name of the directory bucket
 * @param objectKey The key (name) of the object to access
 * @return A presigned URL for accessing the specified object
 */
public static String generatePresignedGetURLForDirectoryBucket(S3Presigner
s3Presigner, String bucketName,
 String objectKey) {
 logger.info("Generating presigned URL for object: {} in bucket: {}",
objectKey, bucketName);

 try {
 // Create a GetObjectRequest
 GetObjectRequest getObjectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(objectKey)
 .build();

 // Create a GetObjectPresignRequest
 GetObjectPresignRequest getObjectPresignRequest =
GetObjectPresignRequest.builder()
 .signatureDuration(Duration.ofMinutes(10)) // Presigned URL
valid for 10 minutes
 .getObjectRequest(getObjectRequest)
 .build();

 // Generate the presigned URL
 PresignedGetObjectRequest presignedGetObjectRequest =
s3Presigner.presignGetObject(getObjectPresignRequest);

 // Get the presigned URL
 String presignedURL = presignedGetObjectRequest.url().toString();
 logger.info("Presigned URL: {}", presignedURL);
 }
}

```

```
 return presignedURL;

 } catch (S3Exception e) {
 logger.error("Failed to generate presigned URL: {} - Error code: {}",
 e.awsErrorDetails().errorMessage(),
 e.awsErrorDetails().errorCode(), e);
 throw e;
 }
}
```

- For API details, see [GetObject](#) in *AWS SDK for Java 2.x API Reference*.

## SageMaker AI examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with SageMaker AI.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello SageMaker AI

The following code example shows how to get started using SageMaker AI.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloSageMaker {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 SageMakerClient sageMakerClient = SageMakerClient.builder()
 .region(region)
 .build();

 listBooks(sageMakerClient);
 sageMakerClient.close();
 }

 public static void listBooks(SageMakerClient sageMakerClient) {
 try {
 ListNotebookInstancesResponse notebookInstancesResponse =
sageMakerClient.listNotebookInstances();
 List<NotebookInstanceSummary> items =
notebookInstancesResponse.notebookInstances();
 for (NotebookInstanceSummary item : items) {
 System.out.println("The notebook name is: " +
item.notebookInstanceName());
 }
 } catch (SageMakerException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

```
}
}
```

- For API details, see [ListNotebookInstances](#) in *AWS SDK for Java 2.x API Reference*.

## Actions

### CreatePipeline

The following code example shows how to use `CreatePipeline`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Create a pipeline from the example pipeline JSON.
public static void setupPipeline(SageMakerClient sageMakerClient, String
filePath, String roleArn,
 String functionArn, String pipelineName) {
 System.out.println("Setting up the pipeline.");
 JSONParser parser = new JSONParser();

 // Read JSON and get pipeline definition.
 try (FileReader reader = new FileReader(filePath)) {
 Object obj = parser.parse(reader);
 JSONObject jsonObject = (JSONObject) obj;
 JSONArray stepsArray = (JSONArray) jsonObject.get("Steps");
 for (Object stepObj : stepsArray) {
 JSONObject step = (JSONObject) stepObj;
 if (step.containsKey("FunctionArn")) {
 step.put("FunctionArn", functionArn);
 }
 }
 System.out.println(jsonObject);

 // Create the pipeline.
```

```

 CreatePipelineRequest pipelineRequest = CreatePipelineRequest.builder()
 .pipelineDescription("Java SDK example pipeline")
 .roleArn(roleArn)
 .pipelineName(pipelineName)
 .pipelineDefinition(jsonObject.toString())
 .build();

 sageMakerClient.createPipeline(pipelineRequest);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 } catch (IOException | ParseException e) {
 throw new RuntimeException(e);
 }
}

```

- For API details, see [CreatePipeline](#) in *AWS SDK for Java 2.x API Reference*.

## DeletePipeline

The following code example shows how to use DeletePipeline.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

// Delete a SageMaker pipeline by name.
public static void deletePipeline(SageMakerClient sageMakerClient, String
pipelineName) {
 DeletePipelineRequest pipelineRequest = DeletePipelineRequest.builder()
 .pipelineName(pipelineName)
 .build();

 sageMakerClient.deletePipeline(pipelineRequest);
 System.out.println("*** Successfully deleted " + pipelineName);
}

```

```
}
```

- For API details, see [DeletePipeline](#) in *AWS SDK for Java 2.x API Reference*.

## DescribePipelineExecution

The following code example shows how to use `DescribePipelineExecution`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Check the status of a pipeline execution.
public static void waitForPipelineExecution(SageMakerClient sageMakerClient,
String executionArn)
 throws InterruptedException {
 String status;
 int index = 0;
 do {
 DescribePipelineExecutionRequest pipelineExecutionRequest =
DescribePipelineExecutionRequest.builder()
 .pipelineExecutionArn(executionArn)
 .build();

 DescribePipelineExecutionResponse response = sageMakerClient
 .describePipelineExecution(pipelineExecutionRequest);
 status = response.pipelineExecutionStatusAsString();
 System.out.println(index + ". The Status of the pipeline is " + status);
 TimeUnit.SECONDS.sleep(4);
 index++;
 } while ("Executing".equals(status));
 System.out.println("Pipeline finished with status " + status);
}
```

- For API details, see [DescribePipelineExecution](#) in *AWS SDK for Java 2.x API Reference*.

## StartPipelineExecution

The following code example shows how to use StartPipelineExecution.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Start a pipeline run with job configurations.
public static String executePipeline(SageMakerClient sageMakerClient, String
bucketName, String queueUrl,
 String roleArn, String pipelineName) {
 System.out.println("Starting pipeline execution.");
 String inputBucketLocation = "s3://" + bucketName + "/samplefiles/
latlongtest.csv";
 String output = "s3://" + bucketName + "/outputfiles/";
 Gson gson = new GsonBuilder()
 .setFieldNamingPolicy(FieldNamingPolicy.UPPER_CAMEL_CASE)
 .setPrettyPrinting().create();

 // Set up all parameters required to start the pipeline.
 List<Parameter> parameters = new ArrayList<>();
 Parameter para1 = Parameter.builder()
 .name("parameter_execution_role")
 .value(roleArn)
 .build();

 Parameter para2 = Parameter.builder()
 .name("parameter_queue_url")
 .value(queueUrl)
 .build();

 String inputJSON = "{\n" +
 " \"DataSourceConfig\": {\n" +
 " \"S3Data\": {\n" +
 " \"S3Uri\": \"s3://" + bucketName + "/samplefiles/
latlongtest.csv\"\n" +
 " },\n" +
 " \"Type\": \"S3_DATA\"\n" +
```



```

 " },\n" +
 " \"DocumentType\": \"CSV\"\n" +
 "}";

System.out.println(inputJSON);

Parameter para3 = Parameter.builder()
 .name("parameter_vej_input_config")
 .value(inputJSON)
 .build();

// Create an ExportVectorEnrichmentJobOutputConfig object.
VectorEnrichmentJobS3Data jobS3Data = VectorEnrichmentJobS3Data.builder()
 .s3Uri(output)
 .build();

ExportVectorEnrichmentJobOutputConfig outputConfig =
ExportVectorEnrichmentJobOutputConfig.builder()
 .s3Data(jobS3Data)
 .build();

String gson4 = gson.toJson(outputConfig);
Parameter para4 = Parameter.builder()
 .name("parameter_vej_export_config")
 .value(gson4)
 .build();
System.out.println("parameter_vej_export_config:" +
gson.toJson(outputConfig));

// Create a VectorEnrichmentJobConfig object.
ReverseGeocodingConfig reverseGeocodingConfig =
ReverseGeocodingConfig.builder()
 .xAttributeName("Longitude")
 .yAttributeName("Latitude")
 .build();

VectorEnrichmentJobConfig jobConfig = VectorEnrichmentJobConfig.builder()
 .reverseGeocodingConfig(reverseGeocodingConfig)
 .build();

String para5JSON = "{\"MapMatchingConfig\":null,\"ReverseGeocodingConfig\":
{\"XAttributeName\":\"Longitude\",\"YAttributeName\":\"Latitude\"}}";
Parameter para5 = Parameter.builder()
 .name("parameter_step_1_vej_config")

```

```
 .value(para5JSON)
 .build();

 System.out.println("parameter_step_1_vej_config:" + gson.toJson(jobConfig));
 parameters.add(para1);
 parameters.add(para2);
 parameters.add(para3);
 parameters.add(para4);
 parameters.add(para5);

 StartPipelineExecutionRequest pipelineExecutionRequest =
 StartPipelineExecutionRequest.builder()
 .pipelineExecutionDescription("Created using Java SDK")
 .pipelineExecutionDisplayName(pipelineName + "-example-execution")
 .pipelineParameters(parameters)
 .pipelineName(pipelineName)
 .build();

 StartPipelineExecutionResponse response =
 sageMakerClient.startPipelineExecution(pipelineExecutionRequest);
 return response.pipelineExecutionArn();
 }
}
```

- For API details, see [StartPipelineExecution](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Get started with geospatial jobs and pipelines

The following code example shows how to:

- Set up resources for a pipeline.
- Set up a pipeline that executes a geospatial job.
- Start a pipeline execution.
- Monitor the status of the execution.
- View the output of the pipeline.
- Clean up resources.

For more information, see [Create and run SageMaker pipelines using AWS SDKs on Community.aws](#).

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class SagemakerWorkflow {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static String eventSourceMapping = "";

 public static void main(String[] args) throws InterruptedException {
 final String usage = "\n" +
 "Usage:\n" +
 " <sageMakerRoleName> <lambdaRoleName> <functionFileLocation>
<functionName> <queueName> <bucketName> <lnglatData> <spatialPipelinePath>
<pipelineName>\n\n"
 +
 "Where:\n" +
 " sageMakerRoleName - The name of the Amazon SageMaker role.\n\n"
 +
 " lambdaRoleName - The name of the AWS Lambda role.\n\n" +
 " functionFileLocation - The file location where the JAR file
that represents the AWS Lambda function is located.\n\n"
 +
 " functionName - The name of the AWS Lambda function (for
example, SageMakerExampleFunction).\n\n" +
 " queueName - The name of the Amazon Simple Queue Service (Amazon
SQS) queue.\n\n" +
 " bucketName - The name of the Amazon Simple Storage Service
(Amazon S3) bucket.\n\n" +
 " lnglatData - The file location of the latlongtest.csv file
required for this use case.\n\n" +
 " spatialPipelinePath - The file location of the
GeoSpatialPipeline.json file required for this use case.\n\n"
 +
 " pipelineName - The name of the pipeline to create (for example,
sagemaker-sdk-example-pipeline).\n\n";
```

```
 if (args.length != 9) {
 System.out.println(usage);
 System.exit(1);
 }

 String sagemakerRoleName = args[0];
 String lambdaRoleName = args[1];
 String functionFileLocation = args[2];
 String functionName = args[3];
 String queueName = args[4];
 String bucketName = args[5];
 String lnglatData = args[6];
 String spatialPipelinePath = args[7];
 String pipelineName = args[8];
 String handlerName = "org.example.SageMakerLambdaFunction::handleRequest";

 Region region = Region.US_WEST_2;
 SageMakerClient sagemakerClient = SageMakerClient.builder()
 .region(region)
 .build();

 IamClient iam = IamClient.builder()
 .region(region)
 .build();

 LambdaClient lambdaClient = LambdaClient.builder()
 .region(region)
 .build();

 SqsClient sqsClient = SqsClient.builder()
 .region(region)
 .build();

 S3Client s3Client = S3Client.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 System.out.println("Welcome to the Amazon SageMaker pipeline example
scenario.");
 System.out.println(
 "\nThis example workflow will guide you through setting up and
running an" +
```

```

 "\nAmazon SageMaker pipeline. The pipeline uses an AWS
Lambda function and an" +
 "\nAmazon SQS Queue. It runs a vector enrichment reverse
geocode job to" +
 "\nreverse geocode addresses in an input file and store the
results in an export file.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("First, we will set up the roles, functions, and queue
needed by the SageMaker pipeline.");
 String lambdaRoleArn = checkLambdaRole(iam, lambdaRoleName);
 String sageMakerRoleArn = checkSageMakerRole(iam, sageMakerRoleName);

 String functionArn = checkFunction(lambdaClient, functionName,
functionFileLocation, lambdaRoleArn,
 handlerName);
 String queueUrl = checkQueue(sqsClient, lambdaClient, queueName,
functionName);
 System.out.println("The queue URL is " + queueUrl);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Setting up bucket " + bucketName);
 if (!checkBucket(s3Client, bucketName)) {
 setupBucket(s3Client, bucketName);
 System.out.println("Put " + lnglatData + " into " + bucketName);
 putS3Object(s3Client, bucketName, "latlongtest.csv", lnglatData);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Now we can create and run our pipeline.");
 setupPipeline(sageMakerClient, spatialPipelinePath, sageMakerRoleArn,
functionArn, pipelineName);
 String pipelineExecutionARN = executePipeline(sageMakerClient, bucketName,
queueUrl, sageMakerRoleArn,
 pipelineName);
 System.out.println("The pipeline execution ARN value is " +
pipelineExecutionARN);
 waitForPipelineExecution(sageMakerClient, pipelineExecutionARN);
 System.out.println("Getting output results " + bucketName);
 getOutputResults(s3Client, bucketName);
 System.out.println(DASHES);

```

```

 System.out.println(DASHES);
 System.out.println("The pipeline has completed. To view the pipeline and
runs " +
 "in SageMaker Studio, follow these instructions:" +
 "\nhttps://docs.aws.amazon.com/sagemaker/latest/dg/pipelines-
studio.html");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("Do you want to delete the AWS resources used in this
Workflow? (y/n)");
 Scanner in = new Scanner(System.in);
 String delResources = in.nextLine();
 if (delResources.compareTo("y") == 0) {
 System.out.println("Lets clean up the AWS resources. Wait 30 seconds");
 TimeUnit.SECONDS.sleep(30);
 deleteEventSourceMapping(lambdaClient);
 deleteSQSQueue(sqsClient, queueName);
 listBucketObjects(s3Client, bucketName);
 deleteBucket(s3Client, bucketName);
 deleteLambdaFunction(lambdaClient, functionName);
 deleteLambdaRole(iam, lambdaRoleName);
 deleteSagemakerRole(iam, sageMakerRoleName);
 deletePipeline(sageMakerClient, pipelineName);
 } else {
 System.out.println("The AWS Resources were not deleted!");
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("SageMaker pipeline scenario is complete.");
 System.out.println(DASHES);
 }

 private static void readObject(S3Client s3Client, String bucketName, String key)
 {
 System.out.println("Output file contents: \n");
 GetObjectRequest objectRequest = GetObjectRequest.builder()
 .bucket(bucketName)
 .key(key)
 .build();
 }

```

```

 ResponseBytes<GetObjectResponse> objectBytes =
s3Client.getObjectAsBytes(objectRequest);
 byte[] byteArray = objectBytes.asByteArray();
 String text = new String(byteArray, StandardCharsets.UTF_8);
 System.out.println("Text output: " + text);
 }

 // Display some results from the output directory.
 public static void getOutputResults(S3Client s3Client, String bucketName) {
 System.out.println("Getting output results {bucketName}.");
 ListObjectsRequest listObjectsRequest = ListObjectsRequest.builder()
 .bucket(bucketName)
 .prefix("outputfiles/")
 .build();

 ListObjectsResponse response = s3Client.listObjects(listObjectsRequest);
 List<S3Object> s3Objects = response.contents();
 for (S3Object object : s3Objects) {
 readObject(s3Client, bucketName, object.key());
 }
 }

 // Check the status of a pipeline execution.
 public static void waitForPipelineExecution(SageMakerClient sageMakerClient,
String executionArn)
 throws InterruptedException {
 String status;
 int index = 0;
 do {
 DescribePipelineExecutionRequest pipelineExecutionRequest =
DescribePipelineExecutionRequest.builder()
 .pipelineExecutionArn(executionArn)
 .build();

 DescribePipelineExecutionResponse response = sageMakerClient
 .describePipelineExecution(pipelineExecutionRequest);
 status = response.pipelineExecutionStatusAsString();
 System.out.println(index + ". The Status of the pipeline is " + status);
 TimeUnit.SECONDS.sleep(4);
 index++;
 } while ("Executing".equals(status));
 System.out.println("Pipeline finished with status " + status);
 }

```

```
// Delete a SageMaker pipeline by name.
public static void deletePipeline(SageMakerClient sageMakerClient, String
pipelineName) {
 DeletePipelineRequest pipelineRequest = DeletePipelineRequest.builder()
 .pipelineName(pipelineName)
 .build();

 sageMakerClient.deletePipeline(pipelineRequest);
 System.out.println("*** Successfully deleted " + pipelineName);
}

// Create a pipeline from the example pipeline JSON.
public static void setupPipeline(SageMakerClient sageMakerClient, String
filePath, String roleArn,
 String functionArn, String pipelineName) {
 System.out.println("Setting up the pipeline.");
 JSONParser parser = new JSONParser();

 // Read JSON and get pipeline definition.
 try (FileReader reader = new FileReader(filePath)) {
 Object obj = parser.parse(reader);
 JSONObject jsonObject = (JSONObject) obj;
 JSONArray stepsArray = (JSONArray) jsonObject.get("Steps");
 for (Object stepObj : stepsArray) {
 JSONObject step = (JSONObject) stepObj;
 if (step.containsKey("FunctionArn")) {
 step.put("FunctionArn", functionArn);
 }
 }
 }
 System.out.println(jsonObject);

 // Create the pipeline.
 CreatePipelineRequest pipelineRequest = CreatePipelineRequest.builder()
 .pipelineDescription("Java SDK example pipeline")
 .roleArn(roleArn)
 .pipelineName(pipelineName)
 .pipelineDefinition(jsonObject.toString())
 .build();

 sageMakerClient.createPipeline(pipelineRequest);

} catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```



```

 } catch (IOException | ParseException e) {
 throw new RuntimeException(e);
 }
}

// Start a pipeline run with job configurations.
public static String executePipeline(SageMakerClient sageMakerClient, String
bucketName, String queueUrl,
 String roleArn, String pipelineName) {
 System.out.println("Starting pipeline execution.");
 String inputBucketLocation = "s3://" + bucketName + "/samplefiles/
latlongtest.csv";
 String output = "s3://" + bucketName + "/outputfiles/";
 Gson gson = new GsonBuilder()
 .setFieldNamingPolicy(FieldNamingPolicy.UPPER_CAMEL_CASE)
 .setPrettyPrinting().create();

 // Set up all parameters required to start the pipeline.
 List<Parameter> parameters = new ArrayList<>();
 Parameter para1 = Parameter.builder()
 .name("parameter_execution_role")
 .value(roleArn)
 .build();

 Parameter para2 = Parameter.builder()
 .name("parameter_queue_url")
 .value(queueUrl)
 .build();

 String inputJSON = "{\n" +
 " \"DataSourceConfig\": {\n" +
 " \"S3Data\": {\n" +
 " \"S3Uri\": \"s3://\" + bucketName + \"/samplefiles/
latlongtest.csv\"\n" +
 " },\n" +
 " \"Type\": \"S3_DATA\"\n" +
 " },\n" +
 " \"DocumentType\": \"CSV\"\n" +
 "}";

 System.out.println(inputJSON);

 Parameter para3 = Parameter.builder()
 .name("parameter_vej_input_config")

```

```

 .value(inputJSON)
 .build();

 // Create an ExportVectorEnrichmentJobOutputConfig object.
 VectorEnrichmentJobS3Data jobS3Data = VectorEnrichmentJobS3Data.builder()
 .s3Uri(output)
 .build();

 ExportVectorEnrichmentJobOutputConfig outputConfig =
 ExportVectorEnrichmentJobOutputConfig.builder()
 .s3Data(jobS3Data)
 .build();

 String gson4 = gson.toJson(outputConfig);
 Parameter para4 = Parameter.builder()
 .name("parameter_vej_export_config")
 .value(gson4)
 .build();
 System.out.println("parameter_vej_export_config:" +
 gson.toJson(outputConfig));

 // Create a VectorEnrichmentJobConfig object.
 ReverseGeocodingConfig reverseGeocodingConfig =
 ReverseGeocodingConfig.builder()
 .xAttributeName("Longitude")
 .yAttributeName("Latitude")
 .build();

 VectorEnrichmentJobConfig jobConfig = VectorEnrichmentJobConfig.builder()
 .reverseGeocodingConfig(reverseGeocodingConfig)
 .build();

 String para5JSON = "{\"MapMatchingConfig\":null,\"ReverseGeocodingConfig\":
 {\"XAttributeName\": \"Longitude\", \"YAttributeName\": \"Latitude\"}}";
 Parameter para5 = Parameter.builder()
 .name("parameter_step_1_vej_config")
 .value(para5JSON)
 .build();

 System.out.println("parameter_step_1_vej_config:" + gson.toJson(jobConfig));
 parameters.add(para1);
 parameters.add(para2);
 parameters.add(para3);
 parameters.add(para4);

```

```

 parameters.add(para5);

 StartPipelineExecutionRequest pipelineExecutionRequest =
StartPipelineExecutionRequest.builder()
 .pipelineExecutionDescription("Created using Java SDK")
 .pipelineExecutionDisplayName(pipelineName + "-example-execution")
 .pipelineParameters(parameters)
 .pipelineName(pipelineName)
 .build();

 StartPipelineExecutionResponse response =
sageMakerClient.startPipelineExecution(pipelineExecutionRequest);
 return response.pipelineExecutionArn();
 }

 public static void deleteEventSourceMapping(LambdaClient lambdaClient) {
 DeleteEventSourceMappingRequest eventSourceMappingRequest =
DeleteEventSourceMappingRequest.builder()
 .uuid(eventSourceMapping)
 .build();

 lambdaClient.deleteEventSourceMapping(eventSourceMappingRequest);
 }

 public static void deleteSagemakerRole(IamClient iam, String roleName) {
 String[] sageMakerRolePolicies = getSageMakerRolePolicies();
 try {
 for (String policy : sageMakerRolePolicies) {
 // First the policy needs to be detached.
 DetachRolePolicyRequest rolePolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy)
 .roleName(roleName)
 .build();

 iam.detachRolePolicy(rolePolicyRequest);
 }

 // Delete the role.
 DeleteRoleRequest roleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 iam.deleteRole(roleRequest);
 }
 }

```

```

 System.out.println("*** Successfully deleted " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteLambdaRole(IamClient iam, String roleName) {
 String[] lambdaRolePolicies = getLambdaRolePolicies();
 try {
 for (String policy : lambdaRolePolicies) {
 // First the policy needs to be detached.
 DetachRolePolicyRequest rolePolicyRequest =
DetachRolePolicyRequest.builder()
 .policyArn(policy)
 .roleName(roleName)
 .build();

 iam.detachRolePolicy(rolePolicyRequest);
 }

 // Delete the role.
 DeleteRoleRequest roleRequest = DeleteRoleRequest.builder()
 .roleName(roleName)
 .build();

 iam.deleteRole(roleRequest);
 System.out.println("*** Successfully deleted " + roleName);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Delete the specific AWS Lambda function.
public static void deleteLambdaFunction(LambdaClient awsLambda, String
functionName) {
 try {
 DeleteFunctionRequest request = DeleteFunctionRequest.builder()
 .functionName(functionName)
 .build();
 }
}

```

```

 awsLambda.deleteFunction(request);
 System.out.println("**** " + functionName + " was deleted");

 } catch (LambdaException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

// Delete the specific S3 bucket.
public static void deleteBucket(S3Client s3Client, String bucketName) {
 DeleteBucketRequest deleteBucketRequest = DeleteBucketRequest.builder()
 .bucket(bucketName)
 .build();
 s3Client.deleteBucket(deleteBucketRequest);
 System.out.println("**** " + bucketName + " was deleted.");
}

public static void listBucketObjects(S3Client s3, String bucketName) {
 try {
 ListObjectsRequest listObjects = ListObjectsRequest
 .builder()
 .bucket(bucketName)
 .build();

 ListObjectsResponse res = s3.listObjects(listObjects);
 List<S3Object> objects = res.contents();
 for (S3Object myValue : objects) {
 System.out.print("\n The name of the key is " + myValue.key());
 deleteBucketObjects(s3, bucketName, myValue.key());
 }

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteBucketObjects(S3Client s3, String bucketName, String
objectName) {
 ArrayList<ObjectIdentifier> toDelete = new ArrayList<>();
 toDelete.add(ObjectIdentifier.builder()
 .key(objectName)
 .build());
}

```

```
 try {
 DeleteObjectsRequest dor = DeleteObjectsRequest.builder()
 .bucket(bucketName)
 .delete(Delete.builder()
 .objects(toDelete).build())
 .build();

 s3.deleteObjects(dor);
 System.out.println("*** " + bucketName + " objects were deleted.");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 // Delete the specific Amazon SQS queue.
 public static void deleteSQSQueue(SqsClient sqsClient, String queueName) {
 try {
 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();
 DeleteQueueRequest deleteQueueRequest = DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();

 sqsClient.deleteQueue(deleteQueueRequest);

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void putS3Object(S3Client s3, String bucketName, String objectKey,
 String objectPath) {
 try {
 Map<String, String> metadata = new HashMap<>();
 metadata.put("x-amz-meta-myVal", "test");
 PutObjectRequest putOb = PutObjectRequest.builder()
 .bucket(bucketName)
 .key("samplefiles/" + objectKey)
```

```

 .metadata(metadata)
 .build();

 s3.putObject(putOb, RequestBody.fromFile(new File(objectPath)));
 System.out.println("Successfully placed " + objectKey + " into bucket "
+ bucketName);

 } catch (S3Exception e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void setupBucket(S3Client s3Client, String bucketName) {
 try {
 S3Waiter s3Waiter = s3Client.waiter();
 CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
 .bucket(bucketName)
 .build();

 s3Client.createBucket(bucketRequest);
 HeadBucketRequest bucketRequestWait = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 // Wait until the bucket is created and print out the response.
 WaiterResponse<HeadBucketResponse> waiterResponse =
s3Waiter.waitUntilBucketExists(bucketRequestWait);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println(bucketName + " is ready");

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Set up the SQS queue to use with the pipeline.
public static String setupQueue(SqsClient sqsClient, LambdaClient lambdaClient,
String queueName,
 String lambdaName) {
 System.out.println("Setting up queue named " + queueName);
 try {
 Map<QueueAttributeName, String> queueAtt = new HashMap<>();

```

```

 queueAtt.put(QueueAttributeName.DELAY_SECONDS, "5");
 queueAtt.put(QueueAttributeName.RECEIVE_MESSAGE_WAIT_TIME_SECONDS, "5");
 queueAtt.put(QueueAttributeName.VISIBILITY_TIMEOUT, "300");
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .attributes(queueAtt)
 .build();

 sqsClient.createQueue(createQueueRequest);
 System.out.println("\nGet queue url");
 GetQueueUrlResponse getQueueUrlResponse = sqsClient

.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 TimeUnit.SECONDS.sleep(15);

 connectLambda(sqsClient, lambdaClient, getQueueUrlResponse.queueUrl(),
lambdaName);
 System.out.println("Queue ready with Url " +
getQueueUrlResponse.queueUrl());
 return getQueueUrlResponse.queueUrl();

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 return "";
}

// Connect the queue to the Lambda function as an event source.
public static void connectLambda(SqsClient sqsClient, LambdaClient lambdaClient,
String queueUrl,
 String lambdaName) {
 System.out.println("Connecting the Lambda function and queue for the
pipeline.");
 String queueArn = "";

 // Specify the attributes to retrieve.
 List<QueueAttributeName> atts = new ArrayList<>();
 atts.add(QueueAttributeName.QUEUE_ARN);
 GetQueueAttributesRequest attributesRequest =
GetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)

```



```

 .attributeNames(atts)
 .build();

 GetQueueAttributesResponse response =
sqscClient.getQueueAttributes(attributesRequest);
 Map<String, String> queueAtts = response.attributesAsStrings();
 for (Map.Entry<String, String> queueAtt : queueAtts.entrySet()) {
 System.out.println("Key = " + queueAtt.getKey() + ", Value = " +
queueAtt.getValue());
 queueArn = queueAtt.getValue();
 }

 CreateEventSourceMappingRequest eventSourceMappingRequest =
CreateEventSourceMappingRequest.builder()
 .eventSourceArn(queueArn)
 .functionName(lambdaName)
 .build();

 CreateEventSourceMappingResponse response1 =
lambdaClient.createEventSourceMapping(eventSourceMappingRequest);
 eventSourceMapping = response1.uuid();
 System.out.println("The mapping between the event source and Lambda function
was successful");
}

// Create an AWS Lambda function.
public static String createLambdaFunction(LambdaClient awsLambda, String
functionName, String filePath, String role,
String handler) {
 try {
 LambdaWaiter waiter = awsLambda.waiter();
 InputStream is = new FileInputStream(filePath);
 SdkBytes fileToUpload = SdkBytes.fromInputStream(is);
 FunctionCode code = FunctionCode.builder()
 .zipFile(fileToUpload)
 .build();

 CreateFunctionRequest functionRequest = CreateFunctionRequest.builder()
 .functionName(functionName)
 .description("SageMaker example function.")
 .code(code)
 .handler(handler)
 .runtime(Runtime.JAVA11)
 .timeout(200)

```

```

 .memorySize(1024)
 .role(role)
 .build();

 // Create a Lambda function using a waiter.
 CreateFunctionResponse functionResponse =
awsLambda.createFunction(functionRequest);
 GetFunctionRequest getFunctionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();
 WaiterResponse<GetFunctionResponse> waiterResponse =
waiter.waitUntilFunctionExists(getFunctionRequest);
 waiterResponse.matched().response().ifPresent(System.out::println);
 System.out.println("The function ARN is " +
functionResponse.functionArn());
 return functionResponse.functionArn();

 } catch (LambdaException | FileNotFoundException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
}

public static String createSageMakerRole(IamClient iam, String roleName) {
 String[] sageMakerRolePolicies = getSageMakerRolePolicies();
 System.out.println("Creating a role to use with SageMaker.");
 String assumeRolePolicy = "{" +
 "\"Version\": \"2012-10-17\", " +
 "\"Statement\": [{" +
 "\"Effect\": \"Allow\", " +
 "\"Principal\": { " +
 "\"Service\": [" +
 "\"sagemaker.amazonaws.com\", " +
 "\"sagemaker-geospatial.amazonaws.com\", " +
 "\"lambda.amazonaws.com\", " +
 "\"s3.amazonaws.com\" " +
 "]" +
 "}, " +
 "\"Action\": \"sts:AssumeRole\" " +
 "}] " +
 "}";

 try {

```

```

 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(roleName)
 .assumeRolePolicyDocument(assumeRolePolicy)
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse roleResult = iam.createRole(request);

 // Attach the policies to the role.
 for (String policy : sageMakerRolePolicies) {
 AttachRolePolicyRequest attachRequest =
AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policy)
 .build();

 iam.attachRolePolicy(attachRequest);
 }

 // Allow time for the role to be ready.
 TimeUnit.SECONDS.sleep(15);
 System.out.println("Role ready with ARN " + roleResult.role().arn());
 return roleResult.role().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 return "";
}

private static String createLambdaRole(IamClient iam, String roleName) {
 String[] lambdaRolePolicies = getLambdaRolePolicies();
 String assumeRolePolicy = "{" +
 "\"Version\": \"2012-10-17\", " +
 "\"Statement\": [{" +
 "\"Effect\": \"Allow\", " +
 "\"Principal\": { " +
 "\"Service\": [" +
 "\"sagemaker.amazonaws.com\", " +
 "\"sagemaker-geospatial.amazonaws.com\", " +
 "\"lambda.amazonaws.com\", " +

```

```

 "\"s3.amazonaws.com\" +
 "]" +
 "}," +
 "\"Action\": \"sts:AssumeRole\" +
 "]" +
 "};

 try {
 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(roleName)
 .assumeRolePolicyDocument(assumeRolePolicy)
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse roleResult = iam.createRole(request);

 // Attach the policies to the role.
 for (String policy : lambdaRolePolicies) {
 AttachRolePolicyRequest attachRequest =
AttachRolePolicyRequest.builder()
 .roleName(roleName)
 .policyArn(policy)
 .build();

 iam.attachRolePolicy(attachRequest);
 }

 // Allow time for the role to be ready.
 TimeUnit.SECONDS.sleep(15);
 System.out.println("Role ready with ARN " + roleResult.role().arn());
 return roleResult.role().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());

 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 return "";
}

 public static String checkFunction(LambdaClient lambdaClient, String
functionName, String filePath, String role,
 String handler) {

```

```

 System.out.println("Create an AWS Lambda function used in this workflow.");
 String functionArn;
 try {
 // Does this function already exist.
 GetFunctionRequest functionRequest = GetFunctionRequest.builder()
 .functionName(functionName)
 .build();

 GetFunctionResponse response =
lambdaClient.getFunction(functionRequest);
 functionArn = response.configuration().functionArn();

 } catch (LambdaException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 functionArn = createLambdaFunction(lambdaClient, functionName, filePath,
role, handler);
 }
 return functionArn;
 }

 // Check to see if the specific S3 bucket exists. If the S3 bucket exists, this
 // method returns true.
 public static boolean checkBucket(S3Client s3, String bucketName) {
 try {
 HeadBucketRequest headBucketRequest = HeadBucketRequest.builder()
 .bucket(bucketName)
 .build();

 s3.headBucket(headBucketRequest);
 System.out.println(bucketName + " exists");
 return true;

 } catch (S3Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 return false;
 }

 // Checks to see if the Amazon SQS queue exists. If not, this method creates a
 // new queue
 // and returns the ARN value.
 public static String checkQueue(SqsClient sqsClient, LambdaClient lambdaClient,
String queueName,
 String lambdaName) {

```

```
 System.out.println("Creating a queue for this use case.");
 String queueUrl;
 try {
 GetQueueUrlRequest request = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 GetQueueUrlResponse response = sqsClient.getQueueUrl(request);
 queueUrl = response.queueUrl();
 System.out.println(queueUrl);

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 queueUrl = setupQueue(sqsClient, lambdaClient, queueName, lambdaName);
 }
 return queueUrl;
 }

 // Checks to see if the Lambda role exists. If not, this method creates it.
 public static String checkLambdaRole(IamClient iam, String roleName) {
 System.out.println("Creating a role to for AWS Lambda to use.");
 String roleArn;
 try {
 GetRoleRequest roleRequest = GetRoleRequest.builder()
 .roleName(roleName)
 .build();

 GetRoleResponse response = iam.getRole(roleRequest);
 roleArn = response.role().arn();
 System.out.println(roleArn);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 roleArn = createLambdaRole(iam, roleName);
 }
 return roleArn;
 }

 // Checks to see if the SageMaker role exists. If not, this method creates it.
 public static String checkSageMakerRole(IamClient iam, String roleName) {
 System.out.println("Creating a role to for AWS SageMaker to use.");
 String roleArn;
 try {
 GetRoleRequest roleRequest = GetRoleRequest.builder()
```

```

 .roleName(roleName)
 .build();

 GetRoleResponse response = iam.getRole(roleRequest);
 roleArn = response.role().arn();
 System.out.println(roleArn);

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 roleArn = createSageMakerRole(iam, roleName);
 }
 return roleArn;
}

private static String[] getSageMakerRolePolicies() {
 String[] sageMakerRolePolicies = new String[3];
 sageMakerRolePolicies[0] = "arn:aws:iam::aws:policy/AmazonSageMakerFullAccess";
 sageMakerRolePolicies[1] = "arn:aws:iam::aws:policy/" +
 "AmazonSageMakerGeospatialFullAccess";
 sageMakerRolePolicies[2] = "arn:aws:iam::aws:policy/AmazonSQSFullAccess";
 return sageMakerRolePolicies;
}

private static String[] getLambdaRolePolicies() {
 String[] lambdaRolePolicies = new String[5];
 lambdaRolePolicies[0] = "arn:aws:iam::aws:policy/AmazonSageMakerFullAccess";
 lambdaRolePolicies[1] = "arn:aws:iam::aws:policy/AmazonSQSFullAccess";
 lambdaRolePolicies[2] = "arn:aws:iam::aws:policy/service-role/" +
 "AmazonSageMakerGeospatialFullAccess";
 lambdaRolePolicies[3] = "arn:aws:iam::aws:policy/service-role/"
 + "AmazonSageMakerServiceCatalogProductsLambdaServiceRolePolicy";
 lambdaRolePolicies[4] = "arn:aws:iam::aws:policy/service-role/" +
 "AWSLambdaSQSQueueExecutionRole";
 return lambdaRolePolicies;
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreatePipeline](#)
  - [DeletePipeline](#)

- [DescribePipelineExecution](#)
- [StartPipelineExecution](#)
- [UpdatePipeline](#)

## Secrets Manager examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Secrets Manager.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### GetSecretValue

The following code example shows how to use `GetSecretValue`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;
import software.amazon.awssdk.services.secretsmanager.model.SecretsManagerException;
```



```
/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * We recommend that you cache your secret values by using client-side caching.
 *
 * Caching secrets improves speed and reduces your costs. For more information,
 * see the following documentation topic:
 *
 * https://docs.aws.amazon.com/secretsmanager/latest/userguide/retrieving-secrets.html
 */
public class GetSecretValue {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <secretName>\s

 Where:
 secretName - The name of the secret (for example, tutorials/
MyFirstSecret).\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String secretName = args[0];
 Region region = Region.US_EAST_1;
 SecretsManagerClient secretsClient = SecretsManagerClient.builder()
 .region(region)
 .build();

 getValue(secretsClient, secretName);
 secretsClient.close();
 }
}
```

```
public static void getValue(SecretsManagerClient secretsClient, String
secretName) {
 try {
 GetSecretValueRequest valueRequest = GetSecretValueRequest.builder()
 .secretId(secretName)
 .build();

 GetSecretValueResponse valueResponse =
secretsClient.getSecretValue(valueRequest);
 String secret = valueResponse.secretString();
 System.out.println(secret);

 } catch (SecretsManagerException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetSecretValue](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon SES examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon SES.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

# Actions

## ListIdentities

The following code example shows how to use `ListIdentities`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ses.SesClient;
import software.amazon.awssdk.services.ses.model.ListIdentitiesResponse;
import software.amazon.awssdk.services.ses.model.SesException;
import java.io.IOException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListIdentities {

 public static void main(String[] args) throws IOException {
 Region region = Region.US_WEST_2;
 SesClient client = SesClient.builder()
 .region(region)
 .build();

 listSESIIdentities(client);
 }

 public static void listSESIIdentities(SesClient client) {
 try {
```

```

 ListIdentitiesResponse identitiesResponse = client.listIdentities();
 List<String> identities = identitiesResponse.getIdentities();
 for (String identity : identities) {
 System.out.println("The identity is " + identity);
 }

 } catch (SesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListIdentities](#) in *AWS SDK for Java 2.x API Reference*.

## ListTemplates

The following code example shows how to use ListTemplates.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sesv2.SesV2Client;
import software.amazon.awssdk.services.sesv2.model.ListEmailTemplatesRequest;
import software.amazon.awssdk.services.sesv2.model.ListEmailTemplatesResponse;
import software.amazon.awssdk.services.sesv2.model.SesV2Exception;

public class ListTemplates {

 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 SesV2Client sesv2Client = SesV2Client.builder()
 .region(region)
 .build();
 }
}

```

```

 listAllTemplates(sesv2Client);
 }

 public static void listAllTemplates(SesV2Client sesv2Client) {
 try {
 ListEmailTemplatesRequest templatesRequest =
ListEmailTemplatesRequest.builder()
 .pageSize(1)
 .build();

 ListEmailTemplatesResponse response =
sesv2Client.listEmailTemplates(templatesRequest);
 response.templatesMetadata()
 .forEach(template -> System.out.println("Template name: " +
template.templateName()));

 } catch (SesV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListTemplates](#) in *AWS SDK for Java 2.x API Reference*.

## SendEmail

The following code example shows how to use SendEmail.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ses.SesClient;
import software.amazon.awssdk.services.ses.model.Content;
import software.amazon.awssdk.services.ses.model.Destination;

```

```

import software.amazon.awssdk.services.ses.model.Message;
import software.amazon.awssdk.services.ses.model.Body;
import software.amazon.awssdk.services.ses.model.SendEmailRequest;
import software.amazon.awssdk.services.ses.model.SesException;

import javax.mail.MessagingException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendMessageEmailRequest {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <sender> <recipient> <subject>\s

 Where:
 sender - An email address that represents the sender.\s
 recipient - An email address that represents the recipient.\s
 subject - The subject line.\s
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String sender = args[0];
 String recipient = args[1];
 String subject = args[2];

 Region region = Region.US_EAST_1;
 SesClient client = SesClient.builder()
 .region(region)
 .build();

 // The HTML body of the email.
 String bodyHTML = "<html>" + "<head></head>" + "<body>" + "<h1>Hello!</h1>"

```

```
 + "<p> See the list of customers.</p>" + "</body>" + "</html>";

 try {
 send(client, sender, recipient, subject, bodyHTML);
 client.close();
 System.out.println("Done");

 } catch (MessagingException e) {
 e.printStackTrace();
 }
}

public static void send(SesClient client,
 String sender,
 String recipient,
 String subject,
 String bodyHTML) throws MessagingException {

 Destination destination = Destination.builder()
 .toAddresses(recipient)
 .build();

 Content content = Content.builder()
 .data(bodyHTML)
 .build();

 Content sub = Content.builder()
 .data(subject)
 .build();

 Body body = Body.builder()
 .html(content)
 .build();

 Message msg = Message.builder()
 .subject(sub)
 .body(body)
 .build();

 SendEmailRequest emailRequest = SendEmailRequest.builder()
 .destination(destination)
 .message(msg)
 .source(sender)
 .build();
```

```

 try {
 System.out.println("Attempting to send an email through Amazon SES " +
"using the AWS SDK for Java...");
 client.sendEmail(emailRequest);

 } catch (SesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ses.SesClient;
import javax.activation.DataHandler;
import javax.activation.DataSource;
import javax.mail.Message;
import javax.mail.MessagingException;
import javax.mail.Session;
import javax.mail.internet.AddressException;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;
import javax.mail.internet.MimeMultipart;
import javax.mail.internet.MimeBodyPart;
import javax.mail.util.ByteArrayDataSource;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.file.Files;
import java.util.Properties;
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.services.ses.model.SendRawEmailRequest;
import software.amazon.awssdk.services.ses.model.RawMessage;
import software.amazon.awssdk.services.ses.model.SesException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

```



```

public class SendMessageAttachment {
 public static void main(String[] args) throws IOException {
 final String usage = ""

 Usage:
 <sender> <recipient> <subject> <fileLocation>\s

 Where:
 sender - An email address that represents the sender.\s
 recipient - An email address that represents the recipient.\s
 subject - The subject line.\s
 fileLocation - The location of a Microsoft Excel file to use as
an attachment (C:/AWS/customers.xls).\s
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String sender = args[0];
 String recipient = args[1];
 String subject = args[2];
 String fileLocation = args[3];

 // The email body for recipients with non-HTML email clients.
 String bodyText = "Hello,\r\n" + "Please see the attached file for a list "
 + "of customers to contact.";

 // The HTML body of the email.
 String bodyHTML = "<html>" + "<head></head>" + "<body>" + "<h1>Hello!</h1>"
 + "<p>Please see the attached file for a " + "list of customers to
contact.</p>" + "</body>"
 + "</html>";

 Region region = Region.US_WEST_2;
 SesClient client = SesClient.builder()
 .region(region)
 .build();

 try {
 sendemailAttachment(client, sender, recipient, subject, bodyText,
bodyHTML, fileLocation);

```

```
 client.close();
 System.out.println("Done");

 } catch (IOException | MessagingException e) {
 e.printStackTrace();
 }
}

public static void sendemailAttachment(SesClient client,
 String sender,
 String recipient,
 String subject,
 String bodyText,
 String bodyHTML,
 String fileLocation) throws AddressException, MessagingException,
IOException {

 java.io.File theFile = new java.io.File(fileLocation);
 byte[] fileContent = Files.readAllBytes(theFile.toPath());

 Session session = Session.getDefaultInstance(new Properties());

 // Create a new MimeMessage object.
 MimeMessage message = new MimeMessage(session);

 // Add subject, from and to lines.
 message.setSubject(subject, "UTF-8");
 message.setFrom(new InternetAddress(sender));
 message.setRecipients(Message.RecipientType.TO,
 InternetAddress.parse(recipient));

 // Create a multipart/alternative child container.
 MimeMultipart msgBody = new MimeMultipart("alternative");

 // Create a wrapper for the HTML and text parts.
 MimeBodyPart wrap = new MimeBodyPart();

 // Define the text part.
 MimeBodyPart textPart = new MimeBodyPart();
 textPart.setContent(bodyText, "text/plain; charset=UTF-8");

 // Define the HTML part.
 MimeBodyPart htmlPart = new MimeBodyPart();
 htmlPart.setContent(bodyHTML, "text/html; charset=UTF-8");
```

```
// Add the text and HTML parts to the child container.
msgBody.addBodyPart(textPart);
msgBody.addBodyPart(htmlPart);

// Add the child container to the wrapper object.
wrap.setContent(msgBody);

// Create a multipart/mixed parent container.
MimeMultipart msg = new MimeMultipart("mixed");

// Add the parent container to the message.
message.setContent(msg);
msg.addBodyPart(wrap);

// Define the attachment.
MimeBodyPart att = new MimeBodyPart();
DataSource fds = new ByteArrayDataSource(fileContent,
 "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet");
att.setDataHandler(new DataHandler(fds));

String reportName = "WorkReport.xls";
att.setFileName(reportName);

// Add the attachment to the message.
msg.addBodyPart(att);

try {
 System.out.println("Attempting to send an email through Amazon SES " +
 "using the AWS SDK for Java...");

 ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
 message.writeTo(outputStream);

 ByteBuffer buf = ByteBuffer.wrap(outputStream.toByteArray());

 byte[] arr = new byte[buf.remaining()];
 buf.get(arr);

 SdkBytes data = SdkBytes.fromByteArray(arr);
 RawMessage rawMessage = RawMessage.builder()
 .data(data)
 .build();
```

```

 SendRawEmailRequest rawEmailRequest = SendRawEmailRequest.builder()
 .rawMessage(rawMessage)
 .build();

 client.sendRawEmail(rawEmailRequest);

 } catch (SesException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 System.out.println("Email sent using SesClient with attachment");
}
}

```

- For API details, see [SendEmail](#) in *AWS SDK for Java 2.x API Reference*.

## SendTemplatedEmail

The following code example shows how to use `SendTemplatedEmail`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sesv2.model.Destination;
import software.amazon.awssdk.services.sesv2.model.EmailContent;
import software.amazon.awssdk.services.sesv2.model.SendEmailRequest;
import software.amazon.awssdk.services.sesv2.model.SesV2Exception;
import software.amazon.awssdk.services.sesv2.SesV2Client;
import software.amazon.awssdk.services.sesv2.model.Template;

/**
 * Before running this AWS SDK for Java (v2) example, set up your development
 * environment, including your credentials.
 */

```

```

* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*
* Also, make sure that you create a template. See the following documentation
* topic:
*
* https://docs.aws.amazon.com/ses/latest/dg/send-personalized-email-api.html
*/

```

```

public class SendEmailTemplate {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <template> <sender> <recipient>\s

 Where:
 template - The name of the email template.
 sender - An email address that represents the sender.\s
 recipient - An email address that represents the recipient.\s
 "";

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String templateName = args[0];
 String sender = args[1];
 String recipient = args[2];
 Region region = Region.US_EAST_1;
 SesV2Client sesv2Client = SesV2Client.builder()
 .region(region)
 .build();

 send(sesv2Client, sender, recipient, templateName);
 }

 public static void send(SesV2Client client, String sender, String recipient,
 String templateName) {
 Destination destination = Destination.builder()
 .toAddresses(recipient)
 .build();
 }

```

```

 /**
 * Specify both name and favorite animal (favoriteanimal) in your code when
 * defining the Template object.
 * If you don't specify all the variables in the template, Amazon SES
doesn't
 * send the email.
 */
 Template myTemplate = Template.builder()
 .templateName(templateName)
 .templateData("{\n" +
 " \"name\": \"Jason\"\n," +
 " \"favoriteanimal\": \"Cat\"\n" +
 "}")
 .build();

 EmailContent emailContent = EmailContent.builder()
 .template(myTemplate)
 .build();

 SendEmailRequest emailRequest = SendEmailRequest.builder()
 .destination(destination)
 .content(emailContent)
 .fromEmailAddress(sender)
 .build();

 try {
 System.out.println("Attempting to send an email based on a template
using the AWS SDK for Java (v2)...");
 client.sendEmail(emailRequest);
 System.out.println("email based on a template was sent");

 } catch (SesV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [SendTemplatedEmail](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a web application to track DynamoDB data

The following code example shows how to create a web application that tracks work items in an Amazon DynamoDB table and uses Amazon Simple Email Service (Amazon SES) to send reports.

#### SDK for Java 2.x

Shows how to use the Amazon DynamoDB API to create a dynamic web application that tracks DynamoDB work data.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB
- Amazon SES

### Create a web application to track Amazon Redshift data

The following code example shows how to create a web application that tracks and reports on work items using an Amazon Redshift database.

#### SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon Redshift database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Redshift data and for use by a React application, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Redshift
- Amazon SES

## Create an Aurora Serverless work item tracker

The following code example shows how to create a web application that tracks work items in an Amazon Aurora Serverless database and uses Amazon Simple Email Service (Amazon SES) to send reports.

### SDK for Java 2.x

Shows how to create a web application that tracks and reports on work items stored in an Amazon RDS database.

For complete source code and instructions on how to set up a Spring REST API that queries Amazon Aurora Serverless data and for use by a React application, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run an example that uses the JDBC API, see the full example on [GitHub](#).

#### Services used in this example

- Aurora
- Amazon RDS
- Amazon RDS Data Service
- Amazon SES

## Detect PPE in images

The following code example shows how to build an app that uses Amazon Rekognition to detect Personal Protective Equipment (PPE) in images.

### SDK for Java 2.x

Shows how to create an AWS Lambda function that detects images with Personal Protective Equipment.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB
- Amazon Rekognition



- Amazon S3
- Amazon SES

## Detect objects in images

The following code example shows how to build an app that uses Amazon Rekognition to detect objects by category in images.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app that uses Amazon Rekognition to identify objects by category in images located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Rekognition
- Amazon S3
- Amazon SES

## Detect people and objects in a video

The following code example shows how to detect people and objects in a video with Amazon Rekognition.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app to detect faces and objects in videos located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Rekognition

- Amazon S3
- Amazon SES
- Amazon SNS
- Amazon SQS

## Use Step Functions to invoke Lambda functions

The following code example shows how to create an AWS Step Functions state machine that invokes AWS Lambda functions in sequence.

### SDK for Java 2.x

Shows how to create an AWS serverless workflow by using AWS Step Functions and the AWS SDK for Java 2.x. Each workflow step is implemented using an AWS Lambda function.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- DynamoDB
- Lambda
- Amazon SES
- Step Functions

## Amazon SES API v2 examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon SES API v2.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### CreateContact

The following code example shows how to use CreateContact.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 // Create a new contact with the provided email address in the
 CreateContactRequest contactRequest = CreateContactRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .emailAddress(emailAddress)
 .build();

 sesClient.createContact(contactRequest);
 contacts.add(emailAddress);

 System.out.println("Contact created: " + emailAddress);

 // Send a welcome email to the new contact
 String welcomeHtml = Files.readString(Paths.get("resources/
coupon_newsletter/welcome.html"));
 String welcomeText = Files.readString(Paths.get("resources/
coupon_newsletter/welcome.txt"));

 SendEmailRequest welcomeEmailRequest = SendEmailRequest.builder()
 .fromEmailAddress(this.verifiedEmail)
 .destination(Destination.builder().toAddresses(emailAddress).build())
 .content(EmailContent.builder()
 .simple(
```

```

 Message.builder()
 .subject(Content.builder().data("Welcome to the Weekly
Coupons Newsletter").build())
 .body(Body.builder()
 .text(Content.builder().data(welcomeText).build())
 .html(Content.builder().data(welcomeHtml).build())
 .build())
 .build()
 .build();
 SendEmailResponse welcomeEmailResponse =
sesClient.sendEmail(welcomeEmailRequest);
 System.out.println("Welcome email sent: " +
welcomeEmailResponse.messageId());
 } catch (AlreadyExistsException e) {
 // If the contact already exists, skip this step for that contact and
 proceed
 // with the next contact
 System.out.println("Contact already exists, skipping creation...");
 } catch (Exception e) {
 System.err.println("Error occurred while processing email address " +
emailAddress + ": " + e.getMessage());
 throw e;
 }
}

```

- For API details, see [CreateContact](#) in *AWS SDK for Java 2.x API Reference*.

## CreateContactList

The following code example shows how to use CreateContactList.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
```

```
// 2. Create a contact list
String contactListName = CONTACT_LIST_NAME;
CreateContactListRequest createContactListRequest =
CreateContactListRequest.builder()
 .contactListName(contactListName)
 .build();
sesClient.createContactList(createContactListRequest);
System.out.println("Contact list created: " + contactListName);
} catch (AlreadyExistsException e) {
 System.out.println("Contact list already exists, skipping creation: weekly-
coupons-newsletter");
} catch (LimitExceededException e) {
 System.err.println("Limit for contact lists has been exceeded.");
 throw e;
} catch (SesV2Exception e) {
 System.err.println("Error creating contact list: " + e.getMessage());
 throw e;
}
```

- For API details, see [CreateContactList](#) in *AWS SDK for Java 2.x API Reference*.

## CreateEmailIdentity

The following code example shows how to use CreateEmailIdentity.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 CreateEmailIdentityRequest createEmailIdentityRequest =
CreateEmailIdentityRequest.builder()
 .emailIdentity(verifiedEmail)
 .build();
 sesClient.createEmailIdentity(createEmailIdentityRequest);
 System.out.println("Email identity created: " + verifiedEmail);
} catch (AlreadyExistsException e) {
```

```

 System.out.println("Email identity already exists, skipping creation: " +
verifiedEmail);
 } catch (NotFoundException e) {
 System.err.println("The provided email address is not verified: " +
verifiedEmail);
 throw e;
 } catch (LimitExceededException e) {
 System.err
 .println("You have reached the limit for email identities. Please remove
some identities and try again.");
 throw e;
 } catch (SesV2Exception e) {
 System.err.println("Error creating email identity: " + e.getMessage());
 throw e;
 }
}

```

- For API details, see [CreateEmailIdentity](#) in *AWS SDK for Java 2.x API Reference*.

## CreateEmailTemplate

The following code example shows how to use CreateEmailTemplate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

try {
 // Create an email template named "weekly-coupons"
 String newsletterHtml = loadFile("resources/coupon_newsletter/coupon-
newsletter.html");
 String newsletterText = loadFile("resources/coupon_newsletter/coupon-
newsletter.txt");

 CreateEmailTemplateRequest templateRequest =
CreateEmailTemplateRequest.builder()
 .templateName(TEMPLATE_NAME)
 .templateContent(EmailTemplateContent.builder()

```

```

 .subject("Weekly Coupons Newsletter")
 .html(newsletterHtml)
 .text(newsletterText)
 .build())
 .build();

 sesClient.createEmailTemplate(templateRequest);

 System.out.println("Email template created: " + TEMPLATE_NAME);
} catch (AlreadyExistsException e) {
 // If the template already exists, skip this step and proceed with the next
 // operation
 System.out.println("Email template already exists, skipping creation...");
} catch (LimitExceededException e) {
 // If the limit for email templates is exceeded, fail the workflow and inform
 // the user
 System.err.println("You have reached the limit for email templates. Please
remove some templates and try again.");
 throw e;
} catch (Exception e) {
 System.err.println("Error occurred while creating email template: " +
e.getMessage());
 throw e;
}

```

- For API details, see [CreateEmailTemplate](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteContactList

The following code example shows how to use DeleteContactList.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

try {
 // Delete the contact list

```

```
DeleteContactListRequest deleteContactListRequest =
DeleteContactListRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build();

sesClient.deleteContactList(deleteContactListRequest);

System.out.println("Contact list deleted: " + CONTACT_LIST_NAME);
} catch (NotFoundException e) {
 // If the contact list does not exist, log the error and proceed
 System.out.println("Contact list not found. Skipping deletion...");
} catch (Exception e) {
 System.err.println("Error occurred while deleting the contact list: " +
e.getMessage());
 e.printStackTrace();
}
```

- For API details, see [DeleteContactList](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteEmailIdentity

The following code example shows how to use DeleteEmailIdentity.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 // Delete the email identity
 DeleteEmailIdentityRequest deleteIdentityRequest =
DeleteEmailIdentityRequest.builder()
 .emailIdentity(this.verifiedEmail)
 .build();

 sesClient.deleteEmailIdentity(deleteIdentityRequest);
}
```



```
 System.out.println("Email identity deleted: " + this.verifiedEmail);
 } catch (NotFoundException e) {
 // If the email identity does not exist, log the error and proceed
 System.out.println("Email identity not found. Skipping deletion...");
 } catch (Exception e) {
 System.err.println("Error occurred while deleting the email identity: " +
 e.getMessage());
 e.printStackTrace();
 }
} else {
 System.out.println("Skipping email identity deletion.");
}
```

- For API details, see [DeleteEmailIdentity](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteEmailTemplate

The following code example shows how to use DeleteEmailTemplate.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 // Delete the template
 DeleteEmailTemplateRequest deleteTemplateRequest =
DeleteEmailTemplateRequest.builder()
 .templateName(TEMPLATE_NAME)
 .build();

 sesClient.deleteEmailTemplate(deleteTemplateRequest);

 System.out.println("Email template deleted: " + TEMPLATE_NAME);
} catch (NotFoundException e) {
 // If the email template does not exist, log the error and proceed
 System.out.println("Email template not found. Skipping deletion...");
}
```

```
 } catch (Exception e) {
 System.err.println("Error occurred while deleting the email template: " +
e.getMessage());
 e.printStackTrace();
 }
```

- For API details, see [DeleteEmailTemplate](#) in *AWS SDK for Java 2.x API Reference*.

## ListContacts

The following code example shows how to use `ListContacts`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
ListContactsRequest contactListRequest = ListContactsRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build();

List<String> contactEmails;
try {
 ListContactsResponse contactListResponse =
sesClient.listContacts(contactListRequest);

 contactEmails = contactListResponse.contacts().stream()
 .map(Contact::emailAddress)
 .toList();
} catch (Exception e) {
 // TODO: Remove when listContacts's GET body issue is resolved.
 contactEmails = this.contacts;
}
```

- For API details, see [ListContacts](#) in *AWS SDK for Java 2.x API Reference*.

## SendEmail

The following code example shows how to use `SendEmail`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Sends a message.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sesv2.model.Body;
import software.amazon.awssdk.services.sesv2.model.Content;
import software.amazon.awssdk.services.sesv2.model.Destination;
import software.amazon.awssdk.services.sesv2.model.EmailContent;
import software.amazon.awssdk.services.sesv2.model.Message;
import software.amazon.awssdk.services.sesv2.model.SendEmailRequest;
import software.amazon.awssdk.services.sesv2.model.SesV2Exception;
import software.amazon.awssdk.services.sesv2.SesV2Client;

/**
 * Before running this AWS SDK for Java (v2) example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class SendEmail {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <sender> <recipient> <subject>\s

 Where:
```

```

 sender - An email address that represents the
sender.\s

 recipient - An email address that represents the
recipient.\s

 subject - The subject line.\s
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String sender = args[0];
 String recipient = args[1];
 String subject = args[2];

 Region region = Region.US_EAST_1;
 SesV2Client sesv2Client = SesV2Client.builder()
 .region(region)
 .build();

 // The HTML body of the email.
 String bodyHTML = "<html>" + "<head></head>" + "<body>" + "<h1>Hello!</h1>"
 + "<p> See the list of customers.</p>" + "</body>" + "</html>";

 send(sesv2Client, sender, recipient, subject, bodyHTML);
}

public static void send(SesV2Client client,
 String sender,
 String recipient,
 String subject,
 String bodyHTML) {

 Destination destination = Destination.builder()
 .toAddresses(recipient)
 .build();

 Content content = Content.builder()
 .data(bodyHTML)
 .build();

 Content sub = Content.builder()
 .data(subject)

```

```

 .build();

 Body body = Body.builder()
 .html(content)
 .build();

 Message msg = Message.builder()
 .subject(sub)
 .body(body)
 .build();

 EmailContent emailContent = EmailContent.builder()
 .simple(msg)
 .build();

 SendEmailRequest emailRequest = SendEmailRequest.builder()
 .destination(destination)
 .content(emailContent)
 .fromEmailAddress(sender)
 .build();

 try {
 System.out.println("Attempting to send an email through Amazon SES "
 + "using the AWS SDK for Java...");
 client.sendEmail(emailRequest);
 System.out.println("email was sent");

 } catch (SesV2Exception e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

Sends a message using a template.

```

 String coupons = Files.readString(Paths.get("resources/coupon_newsletter/
sample_coupons.json"));
 for (String emailAddress : contactEmails) {
 SendEmailRequest newsletterRequest = SendEmailRequest.builder()
 .destination(Destination.builder().toAddresses(emailAddress).build())
 .content(EmailContent.builder()

```

```

 .template(Template.builder()
 .templateName(TEMPLATE_NAME)
 .templateData(coupons)
 .build())
 .build()
 .fromEmailAddress(this.verifiedEmail)
 .listManagementOptions(ListManagementOptions.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build())
 .build();
 SendEmailResponse newsletterResponse =
sesClient.sendEmail(newsletterRequest);
 System.out.println("Newsletter sent to " + emailAddress + ": " +
newsletterResponse.messageId());
}

```

Sends a message with header information.

```

public class SendwithHeader {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <sender> <recipient> <subject>\s

 Where:
 sender - An email address that represents the sender.\s
 recipient - An email address that represents the recipient.\s
 subject - The subject line.\s
 """;

 if (args.length != 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String sender = args[0];
 String recipient = args[1];
 String subject = args[2];
 Region region = Region.US_EAST_1;
 SesV2Client sesv2Client = SesV2Client.builder()

```

```

 .region(region)
 .build();

String bodyHTML = ""
 <html>
 <head></head>
 <body>
 <h1>Hello!</h1>
 <p>See the list of customers.</p>
 </body>
 </html>
 "";

 sendWithHeader(sesv2Client, sender, recipient, subject, bodyHTML);
 sesv2Client.close();
}

/**
 * Sends an email using the AWS SES V2 client.
 *
 * @param sesv2Client the SES V2 client to use for sending the email
 * @param sender the email address of the sender
 * @param recipient the email address of the recipient
 * @param subject the subject of the email
 * @param bodyHTML the HTML content of the email body
 */
public static void sendWithHeader(SesV2Client sesv2Client,
 String sender,
 String recipient,
 String subject,
 String bodyHTML) {
 EmailContent emailContent = EmailContent.builder()
 .simple(Message.builder()
 .body(b -> b.html(c ->
c.charset(UTF_8.name()).data(bodyHTML))
 .text(c -> c.charset(UTF_8.name()).data(bodyHTML)))
 .subject(c -> c.charset(UTF_8.name()).data(subject))
 .headers(List.of(
 MessageHeader.builder()
 .name("List-Unsubscribe")
 .value("<https://nutrition.co/?
address=x&topic=x>, <mailto:unsubscribe@nutrition.co?subject=TopicUnsubscribe>")
 .build(),
 MessageHeader.builder()

```

```

 .name("List-Unsubscribe-Post")
 .value("List-Unsubscribe=One-Click")
 .build()))
 .build())
 .build();

 SendEmailRequest request = SendEmailRequest.builder()
 .fromEmailAddress(sender)
 .destination(d -> d.toAddresses(recipient))
 .content(emailContent)
 .build();

 try {
 SendEmailResponse response = sesv2Client.sendEmail(request);
 System.out.println("Email sent! Message ID: " + response.messageId());
 } catch (SesV2Exception e) {
 System.err.println("Failed to send email: " +
 e.awsErrorDetails().errorMessage());
 throw new RuntimeException(e);
 }
}
}
}

```

- For API details, see [SendEmail](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Newsletter scenario

The following code example shows how to run the Amazon SES API v2 newsletter scenario.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
```



```

 // 2. Create a contact list
 String contactListName = CONTACT_LIST_NAME;
 CreateContactListRequest createContactListRequest =
 CreateContactListRequest.builder()
 .contactListName(contactListName)
 .build();
 sesClient.createContactList(createContactListRequest);
 System.out.println("Contact list created: " + contactListName);
 } catch (AlreadyExistsException e) {
 System.out.println("Contact list already exists, skipping creation: weekly-
coupons-newsletter");
 } catch (LimitExceededException e) {
 System.err.println("Limit for contact lists has been exceeded.");
 throw e;
 } catch (SesV2Exception e) {
 System.err.println("Error creating contact list: " + e.getMessage());
 throw e;
 }

 try {
 // Create a new contact with the provided email address in the
 CreateContactRequest contactRequest = CreateContactRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .emailAddress(emailAddress)
 .build();

 sesClient.createContact(contactRequest);
 contacts.add(emailAddress);

 System.out.println("Contact created: " + emailAddress);

 // Send a welcome email to the new contact
 String welcomeHtml = Files.readString(Paths.get("resources/
coupon_newsletter/welcome.html"));
 String welcomeText = Files.readString(Paths.get("resources/
coupon_newsletter/welcome.txt"));

 SendEmailRequest welcomeEmailRequest = SendEmailRequest.builder()
 .fromEmailAddress(this.verifiedEmail)
 .destination(Destination.builder().toAddresses(emailAddress).build())
 .content(EmailContent.builder()
 .simple(
 Message.builder()

```

```

 .subject(Content.builder().data("Welcome to the Weekly
Coupons Newsletter").build())
 .body(Body.builder()
 .text(Content.builder().data(welcomeText).build())
 .html(Content.builder().data(welcomeHtml).build())
 .build())
 .build())
 .build();

 SendEmailResponse welcomeEmailResponse =
sesClient.sendEmail(welcomeEmailRequest);
 System.out.println("Welcome email sent: " +
welcomeEmailResponse.messageId());
 } catch (AlreadyExistsException e) {
 // If the contact already exists, skip this step for that contact and
 proceed
 // with the next contact
 System.out.println("Contact already exists, skipping creation...");
 } catch (Exception e) {
 System.err.println("Error occurred while processing email address " +
emailAddress + ": " + e.getMessage());
 throw e;
 }
}

ListContactsRequest contactListRequest = ListContactsRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build();

List<String> contactEmails;
try {
 ListContactsResponse contactListResponse =
sesClient.listContacts(contactListRequest);

 contactEmails = contactListResponse.contacts().stream()
 .map(Contact::emailAddress)
 .toList();
} catch (Exception e) {
 // TODO: Remove when listContacts's GET body issue is resolved.
 contactEmails = this.contacts;
}

```

```

 String coupons = Files.readString(Paths.get("resources/coupon_newsletter/
sample_coupons.json"));
 for (String emailAddress : contactEmails) {
 SendEmailRequest newsletterRequest = SendEmailRequest.builder()
 .destination(Destination.builder().toAddresses(emailAddress).build())
 .content(EmailContent.builder()
 .template(Template.builder()
 .templateName(TEMPLATE_NAME)
 .templateData(coupons)
 .build())
 .build())
 .fromEmailAddress(this.verifiedEmail)
 .listManagementOptions(ListManagementOptions.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build())
 .build();
 SendEmailResponse newsletterResponse =
sesClient.sendEmail(newsletterRequest);
 System.out.println("Newsletter sent to " + emailAddress + ": " +
newsletterResponse.messageId());
 }

 try {
 CreateEmailIdentityRequest createEmailIdentityRequest =
CreateEmailIdentityRequest.builder()
 .emailIdentity(verifiedEmail)
 .build();
 sesClient.createEmailIdentity(createEmailIdentityRequest);
 System.out.println("Email identity created: " + verifiedEmail);
 } catch (AlreadyExistsException e) {
 System.out.println("Email identity already exists, skipping creation: " +
verifiedEmail);
 } catch (NotFoundException e) {
 System.err.println("The provided email address is not verified: " +
verifiedEmail);
 throw e;
 } catch (LimitExceededException e) {
 System.err
 .println("You have reached the limit for email identities. Please remove
some identities and try again.");
 throw e;
 } catch (SesV2Exception e) {
 System.err.println("Error creating email identity: " + e.getMessage());
 throw e;
 }
 }
}

```

```

 }

 try {
 // Create an email template named "weekly-coupons"
 String newsletterHtml = loadFile("resources/coupon_newsletter/coupon-
newsletter.html");
 String newsletterText = loadFile("resources/coupon_newsletter/coupon-
newsletter.txt");

 CreateEmailTemplateRequest templateRequest =
CreateEmailTemplateRequest.builder()
 .templateName(TEMPLATE_NAME)
 .templateContent(EmailTemplateContent.builder()
 .subject("Weekly Coupons Newsletter")
 .html(newsletterHtml)
 .text(newsletterText)
 .build())
 .build();

 sesClient.createEmailTemplate(templateRequest);

 System.out.println("Email template created: " + TEMPLATE_NAME);
 } catch (AlreadyExistsException e) {
 // If the template already exists, skip this step and proceed with the next
 // operation
 System.out.println("Email template already exists, skipping creation...");
 } catch (LimitExceededException e) {
 // If the limit for email templates is exceeded, fail the workflow and inform
 // the user
 System.err.println("You have reached the limit for email templates. Please
remove some templates and try again.");
 throw e;
 } catch (Exception e) {
 System.err.println("Error occurred while creating email template: " +
e.getMessage());
 throw e;
 }

 try {
 // Delete the contact list
 DeleteContactListRequest deleteContactListRequest =
DeleteContactListRequest.builder()
 .contactListName(CONTACT_LIST_NAME)
 .build();

```

```
sesClient.deleteContactList(deleteContactListRequest);

System.out.println("Contact list deleted: " + CONTACT_LIST_NAME);
} catch (NotFoundException e) {
 // If the contact list does not exist, log the error and proceed
 System.out.println("Contact list not found. Skipping deletion...");
} catch (Exception e) {
 System.err.println("Error occurred while deleting the contact list: " +
e.getMessage());
 e.printStackTrace();
}

try {
 // Delete the email identity
 DeleteEmailIdentityRequest deleteIdentityRequest =
DeleteEmailIdentityRequest.builder()
 .emailIdentity(this.verifiedEmail)
 .build();

 sesClient.deleteEmailIdentity(deleteIdentityRequest);

 System.out.println("Email identity deleted: " + this.verifiedEmail);
} catch (NotFoundException e) {
 // If the email identity does not exist, log the error and proceed
 System.out.println("Email identity not found. Skipping deletion...");
} catch (Exception e) {
 System.err.println("Error occurred while deleting the email identity: " +
e.getMessage());
 e.printStackTrace();
}
} else {
 System.out.println("Skipping email identity deletion.");
}

try {
 // Delete the template
 DeleteEmailTemplateRequest deleteTemplateRequest =
DeleteEmailTemplateRequest.builder()
 .templateName(TEMPLATE_NAME)
 .build();

 sesClient.deleteEmailTemplate(deleteTemplateRequest);
```

```
 System.out.println("Email template deleted: " + TEMPLATE_NAME);
 } catch (NotFoundException e) {
 // If the email template does not exist, log the error and proceed
 System.out.println("Email template not found. Skipping deletion...");
 } catch (Exception e) {
 System.err.println("Error occurred while deleting the email template: " +
 e.getMessage());
 e.printStackTrace();
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateContact](#)
  - [CreateContactList](#)
  - [CreateEmailIdentity](#)
  - [CreateEmailTemplate](#)
  - [DeleteContactList](#)
  - [DeleteEmailIdentity](#)
  - [DeleteEmailTemplate](#)
  - [ListContacts](#)
  - [SendEmail.simple](#)
  - [SendEmail.template](#)

## Amazon SNS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon SNS.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)

## Get started

### Hello Amazon SNS

The following code example shows how to get started using Amazon SNS.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.sns;

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.paginators.ListTopicsIterable;

public class HelloSNS {
 public static void main(String[] args) {
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listSNSTopics(snsClient);
 snsClient.close();
 }

 public static void listSNSTopics(SnsClient snsClient) {
 try {
 ListTopicsIterable listTopics = snsClient.listTopicsPaginator();
 listTopics.stream()
```

```

 .flatMap(r -> r.topics().stream())
 .forEach(content -> System.out.println(" Topic ARN: " +
content.topicArn()));

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListTopics](#) in *AWS SDK for Java 2.x API Reference*.

## Actions

### CheckIfPhoneNumberIsOptedOut

The following code example shows how to use `CheckIfPhoneNumberIsOptedOut`.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import
 software.amazon.awssdk.services.sns.model.CheckIfPhoneNumberIsOptedOutRequest;
import
 software.amazon.awssdk.services.sns.model.CheckIfPhoneNumberIsOptedOutResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:

```



```

*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CheckOptOut {
 public static void main(String[] args) {

 final String usage = ""

 Usage: <phoneNumber>

 Where:
 phoneNumber - The mobile phone number to look up (for example,
+1XXX5550100).

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String phoneNumber = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 checkPhone(snsClient, phoneNumber);
 snsClient.close();
 }

 public static void checkPhone(SnsClient snsClient, String phoneNumber) {
 try {
 CheckIfPhoneNumberIsOptedOutRequest request =
CheckIfPhoneNumberIsOptedOutRequest.builder()
 .phoneNumber(phoneNumber)
 .build();

 CheckIfPhoneNumberIsOptedOutResponse result =
snsClient.checkIfPhoneNumberIsOptedOut(request);
 System.out.println(
 result.isOptedOut() + "Phone Number " + phoneNumber + " has
Opted Out of receiving sns messages." +
 "\n\nStatus was " +
result.sdkHttpResponse().statusCode());

```

```

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [CheckIfPhoneNumbersOptedOut](#) in *AWS SDK for Java 2.x API Reference*.

## ConfirmSubscription

The following code example shows how to use ConfirmSubscription.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.ConfirmSubscriptionRequest;
import software.amazon.awssdk.services.sns.model.ConfirmSubscriptionResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ConfirmSubscription {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <subscriptionToken> <topicArn>

```

```

 Where:
 subscriptionToken - A short-lived token sent to an endpoint
during the Subscribe action.
 topicArn - The ARN of the topic.\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String subscriptionToken = args[0];
 String topicArn = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 confirmSub(snsClient, subscriptionToken, topicArn);
 snsClient.close();
}

 public static void confirmSub(SnsClient snsClient, String subscriptionToken,
String topicArn) {
 try {
 ConfirmSubscriptionRequest request =
ConfirmSubscriptionRequest.builder()
 .token(subscriptionToken)
 .topicArn(topicArn)
 .build();

 ConfirmSubscriptionResponse result =
snsClient.confirmSubscription(request);
 System.out.println("\n\nStatus was " +
result.sdkHttpResponse().statusCode() + "\n\nSubscription Arn: \n\n"
 + result.subscriptionArn());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [ConfirmSubscription](#) in *AWS SDK for Java 2.x API Reference*.

## CreateTopic

The following code example shows how to use CreateTopic.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreateTopicRequest;
import software.amazon.awssdk.services.sns.model.CreateTopicResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateTopic {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicName>

 Where:
 topicName - The name of the topic to create (for example,
mytopic).

 """;
```

```

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicName = args[0];
 System.out.println("Creating a topic with name: " + topicName);
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String arnVal = createSNSTopic(snsClient, topicName);
 System.out.println("The topic ARN is" + arnVal);
 snsClient.close();
 }

 public static String createSNSTopic(SnsClient snsClient, String topicName) {
 CreateTopicResponse result;
 try {
 CreateTopicRequest request = CreateTopicRequest.builder()
 .name(topicName)
 .build();

 result = snsClient.createTopic(request);
 return result.topicArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
 }
}

```

- For API details, see [CreateTopic](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteTopic

The following code example shows how to use DeleteTopic.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.DeleteTopicRequest;
import software.amazon.awssdk.services.sns.model.DeleteTopicResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteTopic {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn>

 Where:
 topicArn - The ARN of the topic to delete.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
```

```
 System.out.println("Deleting a topic with name: " + topicArn);
 deleteSNSTopic(snsClient, topicArn);
 snsClient.close();
 }

 public static void deleteSNSTopic(SnsClient snsClient, String topicArn) {
 try {
 DeleteTopicRequest request = DeleteTopicRequest.builder()
 .topicArn(topicArn)
 .build();

 DeleteTopicResponse result = snsClient.deleteTopic(request);
 System.out.println("\n\nStatus was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DeleteTopic](#) in *AWS SDK for Java 2.x API Reference*.

## GetSMSAttributes

The following code example shows how to use `GetSMSAttributes`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.GetSubscriptionAttributesRequest;
import software.amazon.awssdk.services.sns.model.GetSubscriptionAttributesResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
```

```

import java.util.Iterator;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetSMSAttributes {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn>

 Where:
 topicArn - The ARN of the topic from which to retrieve
attributes.

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 getSNSAttributes(snsClient, topicArn);
 snsClient.close();
 }

 public static void getSNSAttributes(SnsClient snsClient, String topicArn) {
 try {
 GetSubscriptionAttributesRequest request =
GetSubscriptionAttributesRequest.builder()
 .subscriptionArn(topicArn)
 .build();

 // Get the Subscription attributes

```



```

 GetSubscriptionAttributesResponse res =
snsClient.getSubscriptionAttributes(request);
 Map<String, String> map = res.attributes();

 // Iterate through the map
 Iterator iter = map.entrySet().iterator();
 while (iter.hasNext()) {
 Map.Entry entry = (Map.Entry) iter.next();
 System.out.println("[Key] : " + entry.getKey() + " [Value] : " +
entry.getValue());
 }

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }

 System.out.println("\n\nStatus was good");
}
}

```

- For API details, see [GetSMSAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## GetTopicAttributes

The following code example shows how to use `GetTopicAttributes`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.GetTopicAttributesRequest;
import software.amazon.awssdk.services.sns.model.GetTopicAttributesResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetTopicAttributes {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn>

 Where:
 topicArn - The ARN of the topic to look up.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 System.out.println("Getting attributes for a topic with name: " + topicArn);
 getSNSTopicAttributes(snsClient, topicArn);
 snsClient.close();
 }

 public static void getSNSTopicAttributes(SnsClient snsClient, String topicArn) {
 try {
 GetTopicAttributesRequest request = GetTopicAttributesRequest.builder()
 .topicArn(topicArn)
 .build();

 GetTopicAttributesResponse result =
snsClient.getTopicAttributes(request);
 System.out.println("\n\nStatus is " +
result.sdkHttpResponse().statusCode() + "\n\nAttributes: \n\n"
 + result.attributes());
 }
 }
}

```

```

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [GetTopicAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## ListPhoneNumbersOptedOut

The following code example shows how to use `ListPhoneNumbersOptedOut`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.ListPhoneNumbersOptedOutRequest;
import software.amazon.awssdk.services.sns.model.ListPhoneNumbersOptedOutResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListOptOut {
 public static void main(String[] args) {
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)

```

```

 .build();

 listOpts(snsClient);
 snsClient.close();
 }

 public static void listOpts(SnsClient snsClient) {
 try {
 ListPhoneNumbersOptedOutRequest request =
 ListPhoneNumbersOptedOutRequest.builder().build();
 ListPhoneNumbersOptedOutResponse result =
 snsClient.listPhoneNumbersOptedOut(request);
 System.out.println("Status is " + result.sdkHttpResponse().statusCode())
 + "\n\nPhone Numbers: \n\n"
 + result.phoneNumbers());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListPhoneNumbersOptedOut](#) in *AWS SDK for Java 2.x API Reference*.

## ListSubscriptions

The following code example shows how to use `ListSubscriptions`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.ListSubscriptionsRequest;

```

```

import software.amazon.awssdk.services.sns.model.ListSubscriptionsResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListSubscriptions {
 public static void main(String[] args) {
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listSNSSubscriptions(snsClient);
 snsClient.close();
 }

 public static void listSNSSubscriptions(SnsClient snsClient) {
 try {
 ListSubscriptionsRequest request = ListSubscriptionsRequest.builder()
 .build();

 ListSubscriptionsResponse result = snsClient.listSubscriptions(request);
 System.out.println(result.subscriptions());

 } catch (SnsException e) {

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);

 }
 }
}

```

- For API details, see [ListSubscriptions](#) in *AWS SDK for Java 2.x API Reference*.

## ListTopics

The following code example shows how to use ListTopics.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.ListTopicsRequest;
import software.amazon.awssdk.services.sns.model.ListTopicsResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListTopics {
 public static void main(String[] args) {
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listSNSTopics(snsClient);
 snsClient.close();
 }

 public static void listSNSTopics(SnsClient snsClient) {
 try {
 ListTopicsRequest request = ListTopicsRequest.builder()
 .build();

 ListTopicsResponse result = snsClient.listTopics(request);
 System.out.println(
 "Status was " + result.sdkHttpResponse().statusCode() + "\n"
 + "\nTopics\n\n" + result.topics());
 } catch (SnsException e) {
 System.out.println("Error: " + e.getMessage());
 }
 }
}
```

```

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListTopics](#) in *AWS SDK for Java 2.x API Reference*.

## Publish

The following code example shows how to use Publish.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PublishTopic {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <message> <topicArn>

```

```

 Where:
 message - The message text to send.
 topicArn - The ARN of the topic to publish.
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String message = args[0];
 String topicArn = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
 pubTopic(snsClient, message, topicArn);
 snsClient.close();
}

public static void pubTopic(SnsClient snsClient, String message, String
topicArn) {
 try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .topicArn(topicArn)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status is " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
}

```

- For API details, see [Publish](#) in *AWS SDK for Java 2.x API Reference*.



## SetSMSAttributes

The following code example shows how to use SetSMSAttributes.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SetSmsAttributesRequest;
import software.amazon.awssdk.services.sns.model.SetSmsAttributesResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SetSMSAttributes {
 public static void main(String[] args) {
 HashMap<String, String> attributes = new HashMap<>(1);
 attributes.put("DefaultSMSType", "Transactional");
 attributes.put("UsageReportS3Bucket", "janbucket");

 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
 setSNSAttributes(snsClient, attributes);
 snsClient.close();
 }

 public static void setSNSAttributes(SnsClient snsClient, HashMap<String, String>
attributes) {
 try {
```

```

 SetSmsAttributesRequest request = SetSmsAttributesRequest.builder()
 .attributes(attributes)
 .build();

 SetSmsAttributesResponse result = snsClient.setSMSAttributes(request);
 System.out.println("Set default Attributes to " + attributes + ". Status
was "
 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [SetSMSAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## SetSubscriptionAttributes

The following code example shows how to use SetSubscriptionAttributes.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import java.util.ArrayList;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 */

```

```
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class UseMessageFilterPolicy {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <subscriptionArn>

 Where:
 subscriptionArn - The ARN of a subscription.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String subscriptionArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 usePolicy(snsClient, subscriptionArn);
 snsClient.close();
 }

 public static void usePolicy(SnsClient snsClient, String subscriptionArn) {
 try {
 SNSMessageFilterPolicy fp = new SNSMessageFilterPolicy();
 // Add a filter policy attribute with a single value
 fp.addAttribute("store", "example_corp");
 fp.addAttribute("event", "order_placed");

 // Add a prefix attribute
 fp.addAttributePrefix("customer_interests", "bas");

 // Add an anything-but attribute
 fp.addAttributeAnythingBut("customer_interests", "baseball");

 // Add a filter policy attribute with a list of values
 ArrayList<String> attributeValues = new ArrayList<>();
 attributeValues.add("rugby");
 attributeValues.add("soccer");
```

```

 attributeValues.add("hockey");
 fp.addAttribute("customer_interests", attributeValues);

 // Add a numeric attribute
 fp.addAttribute("price_usd", "=", 0);

 // Add a numeric attribute with a range
 fp.addAttributeRange("price_usd", ">", 0, "<=", 100);

 // Apply the filter policy attributes to an Amazon SNS subscription
 fp.apply(snsClient, subscriptionArn);

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [SetSubscriptionAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## SetTopicAttributes

The following code example shows how to use SetTopicAttributes.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SetTopicAttributesRequest;
import software.amazon.awssdk.services.sns.model.SetTopicAttributesResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development

```

```

* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class SetTopicAttributes {

 public static void main(String[] args) {
 final String usage = ""

 Usage: <attribute> <topicArn> <value>

 Where:
 attribute - The attribute action to use. Valid parameters are:
Policy | DisplayName | DeliveryPolicy .
 topicArn - The ARN of the topic.\s
 value - The value for the attribute.
 "";

 if (args.length < 3) {
 System.out.println(usage);
 System.exit(1);
 }

 String attribute = args[0];
 String topicArn = args[1];
 String value = args[2];

 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 setTopAttr(snsClient, attribute, topicArn, value);
 snsClient.close();
 }

 public static void setTopAttr(SnsClient snsClient, String attribute, String
topicArn, String value) {
 try {
 SetTopicAttributesRequest request = SetTopicAttributesRequest.builder()
 .attributeName(attribute)
 .attributeValue(value)
 .topicArn(topicArn)

```

```

 .build();

 SetTopicAttributesResponse result =
snsClient.setTopicAttributes(request);
 System.out.println(
 "\n\nStatus was " + result.sdkHttpResponse().statusCode() + "\n
\nTopic " + request.topicArn()
 + " updated " + request.attributeName() + " to " +
request.attributeValue());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [SetTopicAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## Subscribe

The following code example shows how to use `Subscribe`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Subscribe an email address to a topic.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;

/**
 * Before running this Java V2 code example, set up your development

```

```

* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class SubscribeEmail {
 public static void main(String[] args) {
 final String usage = ""
 Usage: <topicArn> <email>

 Where:
 topicArn - The ARN of the topic to subscribe.
 email - The email address to use.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 String email = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 subEmail(snsClient, topicArn, email);
 snsClient.close();
 }

 public static void subEmail(SnsClient snsClient, String topicArn, String email)
 {
 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("email")
 .endpoint(email)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("Subscription ARN: " + result.subscriptionArn() + "\n
\n Status is "

```

```

 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

## Subscribe an HTTP endpoint to a topic.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SubscribeHTTPS {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn> <url>

 Where:
 topicArn - The ARN of the topic to subscribe.
 url - The HTTPS endpoint that you want to receive notifications.
 "";

 if (args.length < 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];

```



```

 String url = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 subHTTPS(snsClient, topicArn, url);
 snsClient.close();
 }

 public static void subHTTPS(SnsClient snsClient, String topicArn, String url) {
 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("https")
 .endpoint(url)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("Subscription ARN is " + result.subscriptionArn() +
 "\n\n Status is "
 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Subscribe a Lambda function to a topic.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 */

```

```
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class SubscribeLambda {

 public static void main(String[] args) {

 final String usage = ""

 Usage: <topicArn> <lambdaArn>

 Where:
 topicArn - The ARN of the topic to subscribe.
 lambdaArn - The ARN of an AWS Lambda function.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 String lambdaArn = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String arnValue = subLambda(snsClient, topicArn, lambdaArn);
 System.out.println("Subscription ARN: " + arnValue);
 snsClient.close();
 }

 public static String subLambda(SnsClient snsClient, String topicArn, String
lambdaArn) {
 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("lambda")
 .endpoint(lambdaArn)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
```

```

 return result.subscriptionArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```

- For API details, see [Subscribe](#) in *AWS SDK for Java 2.x API Reference*.

## TagResource

The following code example shows how to use TagResource.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.Tag;
import software.amazon.awssdk.services.sns.model.TagResourceRequest;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class AddTags {

```

```
public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn>

 Where:
 topicArn - The ARN of the topic to which tags are added.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 addTopicTags(snsClient, topicArn);
 snsClient.close();
}

public static void addTopicTags(SnsClient snsClient, String topicArn) {
 try {
 Tag tag = Tag.builder()
 .key("Team")
 .value("Development")
 .build();

 Tag tag2 = Tag.builder()
 .key("Environment")
 .value("Gamma")
 .build();

 List<Tag> tagList = new ArrayList<>();
 tagList.add(tag);
 tagList.add(tag2);

 TagResourceRequest tagResourceRequest = TagResourceRequest.builder()
 .resourceArn(topicArn)
 .tags(tagList)
 .build();
 }
}
```

```

 snsClient.tagResource(tagResourceRequest);
 System.out.println("Tags have been added to " + topicArn);

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [TagResource](#) in *AWS SDK for Java 2.x API Reference*.

## Unsubscribe

The following code example shows how to use Unsubscribe.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.UnsubscribeRequest;
import software.amazon.awssdk.services.sns.model.UnsubscribeResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class Unsubscribe {

```

```

public static void main(String[] args) {
 final String usage = ""

 Usage: <subscriptionArn>

 Where:
 subscriptionArn - The ARN of the subscription to delete.
 "";

 if (args.length < 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String subscriptionArn = args[0];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 unSub(snsClient, subscriptionArn);
 snsClient.close();
}

public static void unSub(SnsClient snsClient, String subscriptionArn) {
 try {
 UnsubscribeRequest request = UnsubscribeRequest.builder()
 .subscriptionArn(subscriptionArn)
 .build();

 UnsubscribeResponse result = snsClient.unsubscribe(request);
 System.out.println("\n\nStatus was " +
result.sdkHttpResponse().statusCode()
 + "\n\nSubscription was removed for " +
request.subscriptionArn());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [Unsubscribe](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Build an app to submit data to a DynamoDB table

The following code example shows how to build an application that submits data to an Amazon DynamoDB table and notifies you when a user updates the table.

#### SDK for Java 2.x

Shows how to create a dynamic web application that submits data using the Amazon DynamoDB Java API and sends a text message using the Amazon Simple Notification Service Java API.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB
- Amazon SNS

### Building an Amazon SNS application

The following code example shows how to create an application that has subscription and publish functionality and translates messages.

#### SDK for Java 2.x

Shows how to use the Amazon Simple Notification Service Java API to create a web application that has subscription and publish functionality. In addition, this example application also translates messages.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run the example that uses the Java Async API, see the full example on [GitHub](#).

#### Services used in this example

- Amazon SNS
- Amazon Translate

## Create a platform endpoint for push notifications

The following code example shows how to create a platform endpoint for Amazon SNS push notifications.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreatePlatformEndpointRequest;
import software.amazon.awssdk.services.sns.model.CreatePlatformEndpointResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * In addition, create a platform application using the AWS Management Console.
 * See this doc topic:
 *
 * https://docs.aws.amazon.com/sns/latest/dg/mobile-push-send-register.html
 *
 * Without the values created by following the previous link, this code examples
 * does not work.
 */

public class RegistrationExample {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <token> <platformApplicationArn>
```



Where:

token - The device token or registration ID of the mobile device.

This is a unique

identifier provided by the device platform (e.g., Apple Push Notification Service (APNS) for iOS devices, Firebase Cloud Messaging (FCM) for Android devices) when the mobile app is registered to receive push notifications.

platformApplicationArn - The ARN value of platform application. You can get this value from the AWS Management Console.\s

```

 """;

 if (args.length != 2) {
 System.out.println(usage);
 return;
 }

 String token = args[0];
 String platformApplicationArn = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 createEndpoint(snsClient, token, platformApplicationArn);
 }

 public static void createEndpoint(SnsClient snsClient, String token, String
platformApplicationArn) {
 System.out.println("Creating platform endpoint with token " + token);
 try {
 CreatePlatformEndpointRequest endpointRequest =
CreatePlatformEndpointRequest.builder()
 .token(token)
 .platformApplicationArn(platformApplicationArn)
 .build();

 CreatePlatformEndpointResponse response =
snsClient.createPlatformEndpoint(endpointRequest);
 System.out.println("The ARN of the endpoint is " +
response.endpointArn());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
 }

```

```
}
}
```

## Create a serverless application to manage photos

The following code example shows how to create a serverless application that lets users manage photos using labels.

### SDK for Java 2.x

Shows how to develop a photo asset management application that detects labels in images using Amazon Rekognition and stores them for later retrieval.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For a deep dive into the origin of this example see the post on [AWS Community](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon Rekognition
- Amazon S3
- Amazon SNS

## Create and publish to a FIFO topic

The following code example shows how to create and publish to a FIFO Amazon SNS topic.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## This example

- creates an Amazon SNS FIFO topic, two Amazon SQS FIFO queues, and one Standard queue.
- subscribes the queues to the topic and publishes a message to the topic.

The [test](#) verifies the receipt of the message to each queue. The [complete example](#) also shows the addition of access policies and deletes the resources at the end.

```
public class PriceUpdateExample {
 public final static SnsClient snsClient = SnsClient.create();
 public final static SqsClient sqsClient = SqsClient.create();

 public static void main(String[] args) {

 final String usage = "\n" +
 "Usage: " +
 " <topicName> <wholesaleQueueFifoName> <retailQueueFifoName>
<analyticsQueueName>\n\n" +
 "Where:\n" +
 " fifoTopicName - The name of the FIFO topic that you want to create.
\n\n" +
 " wholesaleQueueARN - The name of a SQS FIFO queue that will be
created for the wholesale consumer. \n\n"
 +
 " retailQueueARN - The name of a SQS FIFO queue that will be created for
the retail consumer. \n\n" +
 " analyticsQueueARN - The name of a SQS standard queue that will be
created for the analytics consumer. \n\n";
 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 final String fifoTopicName = args[0];
 final String wholeSaleQueueName = args[1];
 final String retailQueueName = args[2];
 final String analyticsQueueName = args[3];

 // For convenience, the QueueData class holds metadata about a queue: ARN,
 URL,
 // name and type.
 List<QueueData> queues = List.of(
 new QueueData(wholeSaleQueueName, QueueType.FIFO),
 new QueueData(retailQueueName, QueueType.FIFO),
```

```

 new QueueData(analyticsQueueName, QueueType.Standard));

 // Create queues.
 createQueues(queues);

 // Create a topic.
 String topicARN = createFIFOTopic(fifoTopicName);

 // Subscribe each queue to the topic.
 subscribeQueues(queues, topicARN);

 // Allow the newly created topic to send messages to the queues.
 addAccessPolicyToQueuesFINAL(queues, topicARN);

 // Publish a sample price update message with payload.
 publishPriceUpdate(topicARN, "{\"product\": 214, \"price\": 79.99}\",
 "Consumables");

 // Clean up resources.
 deleteSubscriptions(queues);
 deleteQueues(queues);
 deleteTopic(topicARN);
}

public static String createFIFOTopic(String topicName) {
 try {
 // Create a FIFO topic by using the SNS service client.
 Map<String, String> topicAttributes = Map.of(
 "FifoTopic", "true",
 "ContentBasedDeduplication", "false",
 "FifoThroughputScope", "MessageGroup");

 CreateTopicRequest topicRequest = CreateTopicRequest.builder()
 .name(topicName)
 .attributes(topicAttributes)
 .build();

 CreateTopicResponse response = snsClient.createTopic(topicRequest);
 String topicArn = response.topicArn();
 System.out.println("The topic ARN is" + topicArn);

 return topicArn;
 } catch (SnsException e) {

```

```

 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static void subscribeQueues(List<QueueData> queues, String topicARN) {
 queues.forEach(queue -> {
 SubscribeRequest subscribeRequest = SubscribeRequest.builder()
 .topicArn(topicARN)
 .endpoint(queue.queueARN)
 .protocol("sqs")
 .build();

 // Subscribe to the endpoint by using the SNS service client.
 // Only Amazon SQS queues can receive notifications from an Amazon SNS
 FIFO
 // topic.
 SubscribeResponse subscribeResponse =
snsClient.subscribe(subscribeRequest);
 System.out.println("The queue [" + queue.queueARN + "] subscribed to the
topic [" + topicARN + "]);
 queue.subscriptionARN = subscribeResponse.subscriptionArn();
 });
}

public static void publishPriceUpdate(String topicArn, String payload, String
groupId) {

 try {
 // Create and publish a message that updates the wholesale price.
 String subject = "Price Update";
 String dedupId = UUID.randomUUID().toString();
 String attributeName = "business";
 String attributeValue = "wholesale";

 MessageAttributeValue msgAttValue = MessageAttributeValue.builder()
 .dataType("String")
 .stringValue(attributeValue)
 .build();

 Map<String, MessageAttributeValue> attributes = new HashMap<>();
 attributes.put(attributeName, msgAttValue);
 PublishRequest pubRequest = PublishRequest.builder()

```

```
 .topicArn(topicArn)
 .subject(subject)
 .message(payload)
 .messageGroupId(groupId)
 .messageDeduplicationId(dedupId)
 .messageAttributes(attributes)
 .build();

 final PublishResponse response = snsClient.publish(pubRequest);
 System.out.println(response.messageId());
 System.out.println(response.sequenceNumber());
 System.out.println("Message was published to " + topicArn);

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateTopic](#)
  - [Publish](#)
  - [Subscribe](#)

## Detect people and objects in a video

The following code example shows how to detect people and objects in a video with Amazon Rekognition.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app to detect faces and objects in videos located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- Amazon Rekognition

- Amazon S3
- Amazon SES
- Amazon SNS
- Amazon SQS

## Publish SMS messages to a topic

The following code example shows how to:

- Create an Amazon SNS topic.
- Subscribe phone numbers to the topic.
- Publish SMS messages to the topic so that all subscribed phone numbers receive the message at once.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Create a topic and return its ARN.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreateTopicRequest;
import software.amazon.awssdk.services.sns.model.CreateTopicResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateTopic {
```

```

public static void main(String[] args) {
 final String usage = ""

 Usage: <topicName>

 Where:
 topicName - The name of the topic to create (for example,
mytopic).

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicName = args[0];
 System.out.println("Creating a topic with name: " + topicName);
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 String arnVal = createSNSTopic(snsClient, topicName);
 System.out.println("The topic ARN is" + arnVal);
 snsClient.close();
}

public static String createSNSTopic(SnsClient snsClient, String topicName) {
 CreateTopicResponse result;
 try {
 CreateTopicRequest request = CreateTopicRequest.builder()
 .name(topicName)
 .build();

 result = snsClient.createTopic(request);
 return result.topicArn();
 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}

```



## Subscribe an endpoint to a topic.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SubscribeTextSMS {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <topicArn> <phoneNumber>

 Where:
 topicArn - The ARN of the topic to subscribe.
 phoneNumber - A mobile phone number that receives notifications
(for example, +1XXX5550100).
 "";

 if (args.length < 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String topicArn = args[0];
 String phoneNumber = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();

 subTextSNS(snsClient, topicArn, phoneNumber);
 snsClient.close();
 }
}
```

```

 }

 public static void subTextSNS(SnsClient snsClient, String topicArn, String
phoneNumber) {
 try {
 SubscribeRequest request = SubscribeRequest.builder()
 .protocol("sms")
 .endpoint(phoneNumber)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("Subscription ARN: " + result.subscriptionArn() + "\n
\n Status is "
 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Set attributes on the message, such as the ID of the sender, the maximum price, and its type. Message attributes are optional.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.SetSmsAttributesRequest;
import software.amazon.awssdk.services.sns.model.SetSmsAttributesResponse;
import software.amazon.awssdk.services.sns.model.SnsException;
import java.util.HashMap;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

```

```

public class SetSMSAttributes {
 public static void main(String[] args) {
 HashMap<String, String> attributes = new HashMap<>(1);
 attributes.put("DefaultSMSType", "Transactional");
 attributes.put("UsageReportS3Bucket", "janbucket");

 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
 setSNSAttributes(snsClient, attributes);
 snsClient.close();
 }

 public static void setSNSAttributes(SnsClient snsClient, HashMap<String, String>
attributes) {
 try {
 SetSmsAttributesRequest request = SetSmsAttributesRequest.builder()
 .attributes(attributes)
 .build();

 SetSmsAttributesResponse result = snsClient.setSMSAttributes(request);
 System.out.println("Set default Attributes to " + attributes + ". Status
was "
 + result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Publish a message to a topic. The message is sent to every subscriber.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development

```

```

* environment, including your credentials.
*
* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class PublishTextSMS {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <message> <phoneNumber>

 Where:
 message - The message text to send.
 phoneNumber - The mobile phone number to which a message is sent
(for example, +1XXX5550100).\s
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String message = args[0];
 String phoneNumber = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
 pubTextSMS(snsClient, message, phoneNumber);
 snsClient.close();
 }

 public static void pubTextSMS(SnsClient snsClient, String message, String
phoneNumber) {
 try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .phoneNumber(phoneNumber)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status was " +
result.sdkHttpResponse().statusCode());
 }
 }
}

```

```

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

## Publish an SMS text message

The following code example shows how to publish SMS messages using Amazon SNS.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SnsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class PublishTextSMS {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <message> <phoneNumber>

```

```

 Where:
 message - The message text to send.
 phoneNumber - The mobile phone number to which a message is sent
(for example, +1XXX5550100).\s
 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String message = args[0];
 String phoneNumber = args[1];
 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .build();
 pubTextSMS(snsClient, message, phoneNumber);
 snsClient.close();
 }

 public static void pubTextSMS(SnsClient snsClient, String message, String
phoneNumber) {
 try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .phoneNumber(phoneNumber)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [Publish](#) in *AWS SDK for Java 2.x API Reference*.

## Publish messages to queues

The following code example shows how to:

- Create topic (FIFO or non-FIFO).
- Subscribe several queues to the topic with an option to apply a filter.
- Publish messages to the topic.
- Poll the queues for messages received.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.sns;

import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreateTopicRequest;
import software.amazon.awssdk.services.sns.model.CreateTopicResponse;
import software.amazon.awssdk.services.sns.model.DeleteTopicRequest;
import software.amazon.awssdk.services.sns.model.DeleteTopicResponse;
import software.amazon.awssdk.services.sns.model.MessageAttributeValue;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SetSubscriptionAttributesRequest;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;
import software.amazon.awssdk.services.sns.model.UnsubscribeRequest;
import software.amazon.awssdk.services.sns.model.UnsubscribeResponse;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequestEntry;
```

```
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueAttributesRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueAttributesResponse;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlResponse;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.QueueAttributeName;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.SetQueueAttributesRequest;
import software.amazon.awssdk.services.sqs.model.SqsException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import com.google.gson.JsonPrimitive;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 * <p>
 * This Java example performs these tasks:
 * <p>
 * 1. Gives the user three options to choose from.
 * 2. Creates an Amazon Simple Notification Service (Amazon SNS) topic.
 * 3. Creates an Amazon Simple Queue Service (Amazon SQS) queue.
 * 4. Gets the SQS queue Amazon Resource Name (ARN) attribute.
 * 5. Attaches an AWS Identity and Access Management (IAM) policy to the queue.
 * 6. Subscribes to the SQS queue.
 * 7. Publishes a message to the topic.
 * 8. Displays the messages.
 * 9. Deletes the received message.
 * 10. Unsubscribes from the topic.
 * 11. Deletes the SNS topic.
 */
```



```
public class SNSWorkflow {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage = "\n" +
 "Usage:\n" +
 " <fifoQueueARN>\n\n" +
 "Where:\n" +
 " accountId - Your AWS account Id value.";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();

 SqsClient sqsClient = SqsClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();

 Scanner in = new Scanner(System.in);
 String accountId = args[0];
 String useFIFO;
 String duplication = "n";
 String topicName;
 String deduplicationID = null;
 String groupId = null;

 String topicArn;
 String sqsQueueName;
 String sqsQueueUrl;
 String sqsQueueArn;
 String subscriptionArn;
 boolean selectFIFO = false;

 String message;
 List<Message> messageList;
 List<String> filterList = new ArrayList<>();
 String msgAttValue = "";
```

```

 System.out.println(DASHES);
 System.out.println("Welcome to messaging with topics and queues.");
 System.out.println("In this scenario, you will create an SNS topic and
subscribe an SQS queue to the topic.\n" +
 "You can select from several options for configuring the topic and the
subscriptions for the queue.\n" +
 "You can then post to the topic and see the results in the queue.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("SNS topics can be configured as FIFO (First-In-First-
Out).\n" +
 "FIFO topics deliver messages in order and support deduplication and
message filtering.\n" +
 "Would you like to work with FIFO topics? (y/n)");
 useFIFO = in.nextLine();
 if (useFIFO.compareTo("y") == 0) {
 selectFIFO = true;
 System.out.println("You have selected FIFO");
 System.out.println(" Because you have chosen a FIFO topic, deduplication
is supported.\n" +
 " Deduplication IDs are either set in the message or
automatically generated from content using a hash function.\n"
 +
 " If a message is successfully published to an SNS FIFO
topic, any message published and determined to have the same deduplication ID,\n"
 +
 " within the five-minute deduplication interval, is accepted
but not delivered.\n" +
 " For more information about deduplication, see https://
docs.aws.amazon.com/sns/latest/dg/fifo-message-dedup.html.");

 System.out.println(
 "Would you like to use content-based deduplication instead of
entering a deduplication ID? (y/n)");
 duplication = in.nextLine();
 if (duplication.compareTo("y") == 0) {
 System.out.println("Please enter a group id value");
 groupId = in.nextLine();
 } else {
 System.out.println("Please enter deduplication Id value");
 deduplicationID = in.nextLine();
 System.out.println("Please enter a group id value");
 }
 }
 }
}

```

```
 groupId = in.nextLine();
 }
}
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("2. Create a topic.");
System.out.println("Enter a name for your SNS topic.");
topicName = in.nextLine();
if (selectFIFO) {
 System.out.println("Because you have selected a FIFO topic, '.fifo' must
be appended to the topic name.");
 topicName = topicName + ".fifo";
 System.out.println("The name of the topic is " + topicName);
 topicArn = createFIFO(snsClient, topicName, duplication);
 System.out.println("The ARN of the FIFO topic is " + topicArn);

} else {
 System.out.println("The name of the topic is " + topicName);
 topicArn = createSNSTopic(snsClient, topicName);
 System.out.println("The ARN of the non-FIFO topic is " + topicArn);

}
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("3. Create an SQS queue.");
System.out.println("Enter a name for your SQS queue.");
sqsQueueName = in.nextLine();
if (selectFIFO) {
 sqsQueueName = sqsQueueName + ".fifo";
}
sqsQueueUrl = createQueue(sqsClient, sqsQueueName, selectFIFO);
System.out.println("The queue URL is " + sqsQueueUrl);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("4. Get the SQS queue ARN attribute.");
sqsQueueArn = getSqsQueueAttrs(sqsClient, sqsQueueUrl);
System.out.println("The ARN of the new queue is " + sqsQueueArn);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Attach an IAM policy to the queue.");
```

```

// Define the policy to use. Make sure that you change the REGION if you are
// running this code
// in a different region.
String policy = ""
{
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": {
 "Service": "sns.amazonaws.com"
 },
 "Action": "sqs:SendMessage",
 "Resource": "arn:aws:sqs:us-east-1:%s:%s",
 "Condition": {
 "ArnEquals": {
 "aws:SourceArn": "arn:aws:sns:us-east-1:%s:%s"
 }
 }
 }
]
}
"".formatted(accountId, sqsQueueName, accountId, topicName);

setQueueAttr(sqsClient, sqsQueueUrl, policy);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Subscribe to the SQS queue.");
if (selectFIFO) {
 System.out.println(
 "If you add a filter to this subscription, then only the filtered
messages will be received in the queue.\n"
 +
 "For information about message filtering, see https://
docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html\n"
 +
 "For this example, you can filter messages by a \"tone\"
attribute.");
 System.out.println("Would you like to filter messages for " +
sqsQueueName + "'s subscription to the topic "
 + topicName + "? (y/n)");
 String filterAns = in.nextLine();
 if (filterAns.compareTo("y") == 0) {

```

```

 boolean moreAns = false;
 System.out.println("You can filter messages by one or more of the
following \"tone\" attributes.");
 System.out.println("1. cheerful");
 System.out.println("2. funny");
 System.out.println("3. serious");
 System.out.println("4. sincere");
 while (!moreAns) {
 System.out.println("Select a number or choose 0 to end.");
 String ans = in.nextLine();
 switch (ans) {
 case "1":
 filterList.add("cheerful");
 break;
 case "2":
 filterList.add("funny");
 break;
 case "3":
 filterList.add("serious");
 break;
 case "4":
 filterList.add("sincere");
 break;
 default:
 moreAns = true;
 break;
 }
 }
 }
}

subscriptionArn = subQueue(snsClient, topicArn, sqsQueueArn, filterList);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Publish a message to the topic.");
if (selectFIFO) {
 System.out.println("Would you like to add an attribute to this message?
(y/n)");
 String msgAns = in.nextLine();
 if (msgAns.compareTo("y") == 0) {
 System.out.println("You can filter messages by one or more of the
following \"tone\" attributes.");
 System.out.println("1. cheerful");
 System.out.println("2. funny");

```

```
 System.out.println("3. serious");
 System.out.println("4. sincere");
 System.out.println("Select a number or choose 0 to end.");
 String ans = in.nextLine();
 switch (ans) {
 case "1":
 msgAttValue = "cheerful";
 break;
 case "2":
 msgAttValue = "funny";
 break;
 case "3":
 msgAttValue = "serious";
 break;
 default:
 msgAttValue = "sincere";
 break;
 }

 System.out.println("Selected value is " + msgAttValue);
 }
 System.out.println("Enter a message.");
 message = in.nextLine();
 pubMessageFIFO(snsClient, message, topicArn, msgAttValue, duplication,
groupId, deduplicationID);

} else {
 System.out.println("Enter a message.");
 message = in.nextLine();
 pubMessage(snsClient, message, topicArn);
}
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("8. Display the message. Press any key to continue.");
in.nextLine();
messageList = receiveMessages(sqsClient, sqsQueueUrl, msgAttValue);
for (Message mes : messageList) {
 System.out.println("Message Id: " + mes.messageId());
 System.out.println("Full Message: " + mes.body());
}
System.out.println(DASHES);

System.out.println(DASHES);
```

```

 System.out.println("9. Delete the received message. Press any key to
continue.");
 in.nextLine();
 deleteMessages(sqsClient, sqsQueueUrl, messageList);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Unsubscribe from the topic and delete the queue.
Press any key to continue.");
 in.nextLine();
 unSub(snsClient, subscriptionArn);
 deleteSQSQueue(sqsClient, sqsQueueName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("11. Delete the topic. Press any key to continue.");
 in.nextLine();
 deleteSNSTopic(snsClient, topicArn);

 System.out.println(DASHES);
 System.out.println("The SNS/SQS workflow has completed successfully.");
 System.out.println(DASHES);
 }

 public static void deleteSNSTopic(SnsClient snsClient, String topicArn) {
 try {
 DeleteTopicRequest request = DeleteTopicRequest.builder()
 .topicArn(topicArn)
 .build();

 DeleteTopicResponse result = snsClient.deleteTopic(request);
 System.out.println("Status was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void deleteSQSQueue(SqsClient sqsClient, String queueName) {
 try {
 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)

```

```

 .build();

 String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();
 DeleteQueueRequest deleteQueueRequest = DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();

 sqsClient.deleteQueue(deleteQueueRequest);
 System.out.println(queueName + " was successfully deleted.");

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void unSub(SnsClient snsClient, String subscriptionArn) {
 try {
 UnsubscribeRequest request = UnsubscribeRequest.builder()
 .subscriptionArn(subscriptionArn)
 .build();

 UnsubscribeResponse result = snsClient.unsubscribe(request);
 System.out.println("Status was " + result.sdkHttpResponse().statusCode()
 + "\nSubscription was removed for " + request.subscriptionArn());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteMessages(SqsClient sqsClient, String queueUrl,
 List<Message> messages) {
 try {
 List<DeleteMessageBatchRequestEntry> entries = new ArrayList<>();
 for (Message msg : messages) {
 DeleteMessageBatchRequestEntry entry =
DeleteMessageBatchRequestEntry.builder()
 .id(msg.messageId())
 .build();

 entries.add(entry);
 }
 }
}

```



```

 DeleteMessageBatchRequest deleteMessageBatchRequest =
DeleteMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(entries)
 .build();

 sqsClient.deleteMessageBatch(deleteMessageBatchRequest);
 System.out.println("The batch delete of messages was successful");

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static List<Message> receiveMessages(SqsClient sqsClient, String
queueUrl, String msgAttValue) {
 try {
 if (msgAttValue.isEmpty()) {
 ReceiveMessageRequest receiveMessageRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .maxNumberOfMessages(5)
 .build();
 return sqsClient.receiveMessage(receiveMessageRequest).messages();
 } else {
 // We know there are filters on the message.
 ReceiveMessageRequest receiveRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageAttributeName(msgAttValue) // Include other message
attributes if needed.
 .maxNumberOfMessages(5)
 .build();

 return sqsClient.receiveMessage(receiveRequest).messages();
 }
 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

```

```

 }

 public static void pubMessage(SnsClient snsClient, String message, String
topicArn) {
 try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .topicArn(topicArn)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status is " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void pubMessageFIFO(SnsClient snsClient,
 String message,
 String topicArn,
 String msgAttValue,
 String duplication,
 String groupId,
 String deduplicationID) {

 try {
 PublishRequest request;
 // Means the user did not choose to use a message attribute.
 if (msgAttValue.isEmpty()) {
 if (duplication.compareTo("y") == 0) {
 request = PublishRequest.builder()
 .message(message)
 .messageGroupId(groupId)
 .topicArn(topicArn)
 .build();
 } else {
 request = PublishRequest.builder()
 .message(message)
 .messageDeduplicationId(deduplicationID)
 .messageGroupId(groupId)

```

```

 .topicArn(topicArn)
 .build();
 }

 } else {
 Map<String, MessageAttributeValue> messageAttributes = new
HashMap<>();
 messageAttributes.put(msgAttValue, MessageAttributeValue.builder()
 .dataType("String")
 .stringValue("true")
 .build());

 if (duplication.compareTo("y") == 0) {
 request = PublishRequest.builder()
 .message(message)
 .messageGroupId(groupId)
 .topicArn(topicArn)
 .build();
 } else {
 // Create a publish request with the message and attributes.
 request = PublishRequest.builder()
 .topicArn(topicArn)
 .message(message)
 .messageDeduplicationId(deduplicationID)
 .messageGroupId(groupId)
 .messageAttributes(messageAttributes)
 .build();
 }
 }

 // Publish the message to the topic.
 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

// Subscribe to the SQS queue.

```

```

 public static String subQueue(SnsClient snsClient, String topicArn, String
queueArn, List<String> filterList) {
 try {
 SubscribeRequest request;
 if (filterList.isEmpty()) {
 // No filter subscription is added.
 request = SubscribeRequest.builder()
 .protocol("sqs")
 .endpoint(queueArn)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("The queue " + queueArn + " has been subscribed
to the topic " + topicArn + "\n" +
 "with the subscription ARN " + result.subscriptionArn());
 return result.subscriptionArn();
 } else {
 request = SubscribeRequest.builder()
 .protocol("sqs")
 .endpoint(queueArn)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("The queue " + queueArn + " has been subscribed
to the topic " + topicArn + "\n" +
 "with the subscription ARN " + result.subscriptionArn());

 String attributeName = "FilterPolicy";
 Gson gson = new Gson();
 String jsonString = "{\"tone\": []}";
 JsonObject jsonObject = gson.fromJson(jsonString, JsonObject.class);
 JsonArray toneArray = jsonObject.getAsJsonArray("tone");
 for (String value : filterList) {
 toneArray.add(new JsonPrimitive(value));
 }

 String updatedJsonString = gson.toJson(jsonObject);
 System.out.println(updatedJsonString);
 SetSubscriptionAttributesRequest attRequest =
SetSubscriptionAttributesRequest.builder()

```

```

 .subscriptionArn(result.subscriptionArn())
 .attributeName(attributeName)
 .attributeValue(updatedJsonString)
 .build();

 snsClient.setSubscriptionAttributes(attRequest);
 return result.subscriptionArn();
}

} catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return "";
}

// Attach a policy to the queue.
public static void setQueueAttr(SqsClient sqsClient, String queueUrl, String
policy) {
 try {
 Map<software.amazon.awssdk.services.sqs.model.QueueAttributeName,
String> attrMap = new HashMap<>();
 attrMap.put(QueueAttributeName.POLICY, policy);

 SetQueueAttributesRequest attributesRequest =
SetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)
 .attributes(attrMap)
 .build();

 sqsClient.setQueueAttributes(attributesRequest);
 System.out.println("The policy has been successfully attached.");

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static String getSQSQueueAttrs(SqsClient sqsClient, String queueUrl) {
 // Specify the attributes to retrieve.
 List<QueueAttributeName> atts = new ArrayList<>();
 atts.add(QueueAttributeName.QUEUE_ARN);

```

```

 GetQueueAttributesRequest attributesRequest =
GetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)
 .attributeNames(atts)
 .build();

 GetQueueAttributesResponse response =
sqsClient.getQueueAttributes(attributesRequest);
 Map<String, String> queueAtts = response.attributesAsStrings();
 for (Map.Entry<String, String> queueAtt : queueAtts.entrySet())
 return queueAtt.getValue();

 return "";
 }

 public static String createQueue(SqsClient sqsClient, String queueName, Boolean
selectFIFO) {
 try {
 System.out.println("\nCreate Queue");
 if (selectFIFO) {
 Map<QueueAttributeName, String> attrs = new HashMap<>();
 attrs.put(QueueAttributeName.FIFO_QUEUE, "true");
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .attributes(attrs)
 .build();

 sqsClient.createQueue(createQueueRequest);
 System.out.println("\nGet queue url");
 GetQueueUrlResponse getQueueUrlResponse = sqsClient

.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 return getQueueUrlResponse.queueUrl();
 } else {
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();

 sqsClient.createQueue(createQueueRequest);
 System.out.println("\nGet queue url");
 GetQueueUrlResponse getQueueUrlResponse = sqsClient

.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 return getQueueUrlResponse.queueUrl();
 }
 }
 }

```

```

 }

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createSNSTopic(SnsClient snsClient, String topicName) {
 CreateTopicResponse result;
 try {
 CreateTopicRequest request = CreateTopicRequest.builder()
 .name(topicName)
 .build();

 result = snsClient.createTopic(request);
 return result.topicArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createFIFO(SnsClient snsClient, String topicName, String
duplication) {
 try {
 // Create a FIFO topic by using the SNS service client.
 Map<String, String> topicAttributes = new HashMap<>();
 if (duplication.compareTo("n") == 0) {
 topicAttributes.put("FifoTopic", "true");
 topicAttributes.put("ContentBasedDeduplication", "false");
 } else {
 topicAttributes.put("FifoTopic", "true");
 topicAttributes.put("ContentBasedDeduplication", "true");
 }

 CreateTopicRequest topicRequest = CreateTopicRequest.builder()
 .name(topicName)
 .attributes(topicAttributes)
 .build();
 }

```

```
 CreateTopicResponse response = snsClient.createTopic(topicRequest);
 return response.topicArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateQueue](#)
- [CreateTopic](#)
- [DeleteMessageBatch](#)
- [DeleteQueue](#)
- [DeleteTopic](#)
- [GetQueueAttributes](#)
- [Publish](#)
- [ReceiveMessage](#)
- [SetQueueAttributes](#)
- [Subscribe](#)
- [Unsubscribe](#)

## Use API Gateway to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by Amazon API Gateway.

### SDK for Java 2.x

Shows how to create an AWS Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create a Lambda function invoked by Amazon API Gateway that scans an Amazon DynamoDB table for work anniversaries and uses Amazon Simple Notification Service



(Amazon SNS) to send a text message to your employees that congratulates them at their one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- API Gateway
- DynamoDB
- Lambda
- Amazon SNS

### Use scheduled events to invoke a Lambda function

The following code example shows how to create an AWS Lambda function invoked by an Amazon EventBridge scheduled event.

#### SDK for Java 2.x

Shows how to create an Amazon EventBridge scheduled event that invokes an AWS Lambda function. Configure EventBridge to use a cron expression to schedule when the Lambda function is invoked. In this example, you create a Lambda function by using the Lambda Java runtime API. This example invokes different AWS services to perform a specific use case. This example demonstrates how to create an app that sends a mobile text message to your employees that congratulates them at the one year anniversary date.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- CloudWatch Logs
- DynamoDB
- EventBridge
- Lambda
- Amazon SNS

## Serverless examples

### Invoke a Lambda function from an Amazon SNS trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving messages from an SNS topic. The function retrieves the messages from the event parameter and logs the content of each message.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

Consuming an SNS event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
package example;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SNSEvent;
import com.amazonaws.services.lambda.runtime.events.SNSEvent.SNSRecord;

import java.util.Iterator;
import java.util.List;

public class SNSEventHandler implements RequestHandler<SNSEvent, Boolean> {
 LambdaLogger logger;

 @Override
 public Boolean handleRequest(SNSEvent event, Context context) {
 logger = context.getLogger();
 List<SNSRecord> records = event.getRecords();
 if (!records.isEmpty()) {
 Iterator<SNSRecord> recordsIter = records.iterator();
 while (recordsIter.hasNext()) {
```

```
 processRecord(recordsIter.next());
 }
}
return Boolean.TRUE;
}

public void processRecord(SNSRecord record) {
 try {
 String message = record.getSNS().getMessage();
 logger.log("message: " + message);
 } catch (Exception e) {
 throw new RuntimeException(e);
 }
}
}
```

## Amazon SQS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon SQS.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Actions](#)
- [Scenarios](#)
- [Serverless examples](#)

# Get started

## Hello Amazon SQS

The following code example shows how to get started using Amazon SQS.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.SqsException;
import software.amazon.awssdk.services.sqs.paginators.ListQueuesIterable;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class HelloSQS {
 public static void main(String[] args) {
 SqsClient sqsClient = SqsClient.builder()
 .region(Region.US_WEST_2)
 .build();

 listQueues(sqsClient);
 sqsClient.close();
 }

 public static void listQueues(SqsClient sqsClient) {
 try {
 ListQueuesIterable listQueues = sqsClient.listQueuesPaginator();
 listQueues.stream()
 .flatMap(r -> r.queueUrls().stream())
 }
 }
}
```

```

 .forEach(content -> System.out.println(" Queue URL: " +
content.toLowerCase()));

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [ListQueues](#) in *AWS SDK for Java 2.x API Reference*.

## Actions

### CreateQueue

The following code example shows how to use CreateQueue.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.ChangeMessageVisibilityRequest;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlResponse;
import software.amazon.awssdk.services.sqs.model.ListQueuesRequest;
import software.amazon.awssdk.services.sqs.model.ListQueuesResponse;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.SendMessageBatchRequest;
import software.amazon.awssdk.services.sqs.model.SendMessageBatchRequestEntry;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;

```

```
import software.amazon.awssdk.services.sqs.model.SqsException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SQSExample {
 public static void main(String[] args) {
 String queueName = "queue" + System.currentTimeMillis();
 SqsClient sqsClient = SqsClient.builder()
 .region(Region.US_WEST_2)
 .build();

 // Perform various tasks on the Amazon SQS queue.
 String queueUrl = createQueue(sqsClient, queueName);
 listQueues(sqsClient);
 listQueuesFilter(sqsClient, queueUrl);
 List<Message> messages = receiveMessages(sqsClient, queueUrl);
 sendBatchMessages(sqsClient, queueUrl);
 changeMessages(sqsClient, queueUrl, messages);
 deleteMessages(sqsClient, queueUrl, messages);
 sqsClient.close();
 }

 public static String createQueue(SqsClient sqsClient, String queueName) {
 try {
 System.out.println("\nCreate Queue");

 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();

 sqsClient.createQueue(createQueueRequest);

 System.out.println("\nGet queue url");

 GetQueueUrlResponse getQueueUrlResponse = sqsClient

.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
```

```
 return getQueueUrlResponse.queueUrl();

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static void listQueues(SqsClient sqsClient) {

 System.out.println("\nList Queues");
 String prefix = "que";

 try {
 ListQueuesRequest listQueuesRequest =
ListQueuesRequest.builder().queueNamePrefix(prefix).build();
 ListQueuesResponse listQueuesResponse =
sqsClient.listQueues(listQueuesRequest);
 for (String url : listQueuesResponse.queueUrls()) {
 System.out.println(url);
 }

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void listQueuesFilter(SqsClient sqsClient, String queueUrl) {
 // List queues with filters
 String namePrefix = "queue";
 ListQueuesRequest filterListRequest = ListQueuesRequest.builder()
 .queueNamePrefix(namePrefix)
 .build();

 ListQueuesResponse listQueuesFilteredResponse =
sqsClient.listQueues(filterListRequest);
 System.out.println("Queue URLs with prefix: " + namePrefix);
 for (String url : listQueuesFilteredResponse.queueUrls()) {
 System.out.println(url);
 }

 System.out.println("\nSend message");
}
```

```

 try {
 sqsClient.sendMessage(SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody("Hello world!")
 .delaySeconds(10)
 .build());

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static void sendBatchMessages(SqsClient sqsClient, String queueUrl) {

 System.out.println("\nSend multiple messages");
 try {
 SendMessageBatchRequest sendMessageBatchRequest =
SendMessageBatchRequest.builder()
 .queueUrl(queueUrl)

 .entries(SendMessageBatchRequestEntry.builder().id("id1").messageBody("Hello from
msg 1").build(),

SendMessageBatchRequestEntry.builder().id("id2").messageBody("msg
2").delaySeconds(10)

 .build())

 .build();
 sqsClient.sendMessageBatch(sendMessageBatchRequest);

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }

 public static List<Message> receiveMessages(SqsClient sqsClient, String
queueUrl) {

 System.out.println("\nReceive messages");
 try {
 ReceiveMessageRequest receiveMessageRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)

```



```

 .maxNumberOfMessages(5)
 .build();
 return sqsClient.receiveMessage(receiveMessageRequest).messages();

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return null;
}

public static void changeMessages(SqsClient sqsClient, String queueUrl,
List<Message> messages) {

 System.out.println("\nChange Message Visibility");
 try {

 for (Message message : messages) {
 ChangeMessageVisibilityRequest req =
ChangeMessageVisibilityRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(message.receiptHandle())
 .visibilityTimeout(100)
 .build();
 sqsClient.changeMessageVisibility(req);
 }

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteMessages(SqsClient sqsClient, String queueUrl,
List<Message> messages) {
 System.out.println("\nDelete Messages");

 try {
 for (Message message : messages) {
 DeleteMessageRequest deleteMessageRequest =
DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(message.receiptHandle())
 .build();

```

```
 sqsClient.deleteMessage(deleteMessageRequest);
 }
} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}
```

- For API details, see [CreateQueue](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteMessage

The following code example shows how to use DeleteMessage.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 for (Message message : messages) {
 DeleteMessageRequest deleteMessageRequest =
DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(message.receiptHandle())
 .build();
 sqsClient.deleteMessage(deleteMessageRequest);
 }
} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

- For API details, see [DeleteMessage](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteQueue

The following code example shows how to use DeleteQueue.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
import software.amazon.awssdk.services.sqs.model.SqsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteQueue {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <queueName>

 Where:
 queueName - The name of the Amazon SQS queue to delete.

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String queueName = args[0];
```

```
SqsClient sqs = SqsClient.builder()
 .region(Region.US_WEST_2)
 .build();

deleteSQSQueue(sqs, queueName);
sqs.close();
}

public static void deleteSQSQueue(SqsClient sqsClient, String queueName) {
 try {
 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();
 DeleteQueueRequest deleteQueueRequest = DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();

 sqsClient.deleteQueue(deleteQueueRequest);

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteQueue](#) in *AWS SDK for Java 2.x API Reference*.

## GetQueueUrl

The following code example shows how to use `GetQueueUrl`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
GetQueueUrlResponse getQueueUrlResponse = sqsClient
 .getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 return getQueueUrlResponse.queueUrl();
```

- For API details, see [GetQueueUrl](#) in *AWS SDK for Java 2.x API Reference*.

## ListQueues

The following code example shows how to use `ListQueues`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
String prefix = "que";

try {
 ListQueuesRequest listQueuesRequest =
ListQueuesRequest.builder().queueNamePrefix(prefix).build();
 ListQueuesResponse listQueuesResponse =
sqsClient.listQueues(listQueuesRequest);
 for (String url : listQueuesResponse.queueUrls()) {
 System.out.println(url);
 }

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
```

- For API details, see [ListQueues](#) in *AWS SDK for Java 2.x API Reference*.

## ReceiveMessage

The following code example shows how to use `ReceiveMessage`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
try {
 ReceiveMessageRequest receiveMessageRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .numberOfMessages(5)
 .build();
 return sqsClient.receiveMessage(receiveMessageRequest).messages();

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
return null;
```

- For API details, see [ReceiveMessage](#) in *AWS SDK for Java 2.x API Reference*.

## SendMessage

The following code example shows how to use `SendMessage`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Two examples of the SendMessage operation follow:

- Send a message with a body and a delay
- Send a message with a body and message attributes

Send a message with a body and a delay.

```
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;
import software.amazon.awssdk.services.sqs.model.SqsException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class SendMessages {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <queueName> <message>

 Where:
 queueName - The name of the queue.
 message - The message to send.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String queueName = args[0];
 String message = args[1];
 SqsClient sqsClient = SqsClient.builder()
 .region(Region.US_WEST_2)
 .build();
```

```

 sendMessage(sqsClient, queueName, message);
 sqsClient.close();
 }

 public static void sendMessage(SqsClient sqsClient, String queueName, String
message) {
 try {
 CreateQueueRequest request = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();
 sqsClient.createQueue(request);

 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();
 SendMessageRequest sendMsgRequest = SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody(message)
 .delaySeconds(5)
 .build();

 sqsClient.sendMessage(sendMsgRequest);

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

Send a message with a body and message attributes.

```

/**
 * <p>This method demonstrates how to add message attributes to a message.
 * Each attribute must specify a name, value, and data type. You use a Java Map
to supply the attributes. The map's
 * key is the attribute name, and you specify the map's entry value using a
builder that includes the attribute
 * value and data type.</p>
 *

```



```

 * <p>The data type must start with one of "String", "Number" or "Binary". You
 can optionally
 * define a custom extension by using a "." and your extension.</p>
 *
 * <p>The SQS Developer Guide provides more information on @see <a
 * href="https://docs.aws.amazon.com/AWSSimpleQueueService/latest/
 SQSDeveloperGuide/sqs-message-metadata.html#sqs-message-attributes">message
 * attributes.</p>
 *
 * @param thumbnailPath Filesystem path of the image.
 * @param queueUrl URL of the SQS queue.
 */
 static void sendMessageWithAttributes(Path thumbnailPath, String queueUrl) {
 Map<String, MessageAttributeValue> messageAttributeMap;
 try {
 messageAttributeMap = Map.of(
 "Name", MessageAttributeValue.builder()
 .stringValue("Jane Doe")
 .dataType("String").build(),
 "Age", MessageAttributeValue.builder()
 .stringValue("42")
 .dataType("Number.int").build(),
 "Image", MessageAttributeValue.builder()
 .binaryValue(SdkBytes.fromByteArray(Files.readAllBytes(thumbnailPath)))
 .dataType("Binary.jpg").build()
);
 } catch (IOException e) {
 LOGGER.error("An I/O exception occurred reading thumbnail image: {}",
 e.getMessage(), e);
 throw new RuntimeException(e);
 }

 SendMessageRequest request = SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody("Hello SQS")
 .messageAttributes(messageAttributeMap)
 .build();

 try {
 SendMessageResponse sendMessageResponse =
 SQS_CLIENT.sendMessage(request);
 LOGGER.info("Message ID: {}", sendMessageResponse.messageId());
 } catch (SqsException e) {

```

```
 LOGGER.error("Exception occurred sending message: {}", e.getMessage(),
e);
 throw new RuntimeException(e);
 }
}
```

- For API details, see [SendMessage](#) in *AWS SDK for Java 2.x API Reference*.

## SendMessageBatch

The following code example shows how to use `SendMessageBatch`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
SendMessageBatchRequest sendMessageBatchRequest =
SendMessageBatchRequest.builder()
 .queueUrl(queueUrl)

 .entries(SendMessageBatchRequestEntry.builder().id("id1").messageBody("Hello from
msg 1").build(),

SendMessageBatchRequestEntry.builder().id("id2").messageBody("msg
2").delaySeconds(10)

 .build())
 .build();
sqsClient.sendMessageBatch(sendMessageBatchRequest);
```

- For API details, see [SendMessageBatch](#) in *AWS SDK for Java 2.x API Reference*.

## SetQueueAttributes

The following code example shows how to use `SetQueueAttributes`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Configure an Amazon SQS to use server-side encryption (SSE) using a custom KMS key.

```
public static void addEncryption(String queueName, String kmsMasterKeyAlias) {
 SqsClient sqsClient = SqsClient.create();

 GetQueueUrlRequest urlRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 GetQueueUrlResponse getQueueUrlResponse;
 try {
 getQueueUrlResponse = sqsClient.getQueueUrl(urlRequest);
 } catch (QueueDoesNotExistException e) {
 LOGGER.error(e.getMessage(), e);
 throw new RuntimeException(e);
 }
 String queueUrl = getQueueUrlResponse.queueUrl();

 Map<QueueAttributeName, String> attributes = Map.of(
 QueueAttributeName.KMS_MASTER_KEY_ID, kmsMasterKeyAlias,
 QueueAttributeName.KMS_DATA_KEY_REUSE_PERIOD_SECONDS, "140" // Set
the data key reuse period to 140 seconds.
);
 // This
is how long SQS can reuse the data key before requesting a new one from KMS.

 SetQueueAttributesRequest attRequest = SetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)
 .attributes(attributes)
 .build();

 try {
 sqsClient.setQueueAttributes(attRequest);
 LOGGER.info("The attributes have been applied to {}", queueName);
 } catch (InvalidAttributeNameException | InvalidAttributeValueException e) {
 LOGGER.error(e.getMessage(), e);
 }
}
```

```
 throw new RuntimeException(e);
 } finally {
 sqsClient.close();
 }
}
```

- For API details, see [SetQueueAttributes](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create a messaging application

The following code example shows how to create a messaging application by using Amazon SQS.

#### SDK for Java 2.x

Shows how to use the Amazon SQS API to develop a Spring REST API that sends and retrieves messages.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Comprehend
- Amazon SQS

### Create and publish to a FIFO topic

The following code example shows how to create and publish to a FIFO Amazon SNS topic.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

This example

- creates an Amazon SNS FIFO topic, two Amazon SQS FIFO queues, and one Standard queue.
- subscribes the queues to the topic and publishes a message to the topic.

The [test](#) verifies the receipt of the message to each queue. The [complete example](#) also shows the addition of access policies and deletes the resources at the end.

```
public class PriceUpdateExample {
 public final static SnsClient snsClient = SnsClient.create();
 public final static SqsClient sqsClient = SqsClient.create();

 public static void main(String[] args) {

 final String usage = "\n" +
 "Usage: " +
 " <topicName> <wholesaleQueueFifoName> <retailQueueFifoName>
<analyticsQueueName>\n\n" +
 "Where:\n" +
 " fifoTopicName - The name of the FIFO topic that you want to create.
\n\n" +
 " wholesaleQueueARN - The name of a SQS FIFO queue that will be
created for the wholesale consumer. \n\n"
 +
 " retailQueueARN - The name of a SQS FIFO queue that will created for
the retail consumer. \n\n" +
 " analyticsQueueARN - The name of a SQS standard queue that will be
created for the analytics consumer. \n\n";
 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 final String fifoTopicName = args[0];
 final String wholeSaleQueueName = args[1];
 final String retailQueueName = args[2];
 final String analyticsQueueName = args[3];

 // For convenience, the QueueData class holds metadata about a queue: ARN,
URL,
 // name and type.
 List<QueueData> queues = List.of(
 new QueueData(wholeSaleQueueName, QueueType.FIFO),
 new QueueData(retailQueueName, QueueType.FIFO),
 new QueueData(analyticsQueueName, QueueType.Standard));
 }
}
```

```
// Create queues.
createQueues(queues);

// Create a topic.
String topicARN = createFIFOTopic(fifoTopicName);

// Subscribe each queue to the topic.
subscribeQueues(queues, topicARN);

// Allow the newly created topic to send messages to the queues.
addAccessPolicyToQueuesFINAL(queues, topicARN);

// Publish a sample price update message with payload.
publishPriceUpdate(topicARN, "{\"product\": 214, \"price\": 79.99}\",
"Consumables");

// Clean up resources.
deleteSubscriptions(queues);
deleteQueues(queues);
deleteTopic(topicARN);
}

public static String createFIFOTopic(String topicName) {
 try {
 // Create a FIFO topic by using the SNS service client.
 Map<String, String> topicAttributes = Map.of(
 "FifoTopic", "true",
 "ContentBasedDeduplication", "false",
 "FifoThroughputScope", "MessageGroup");

 CreateTopicRequest topicRequest = CreateTopicRequest.builder()
 .name(topicName)
 .attributes(topicAttributes)
 .build();

 CreateTopicResponse response = snsClient.createTopic(topicRequest);
 String topicArn = response.topicArn();
 System.out.println("The topic ARN is" + topicArn);

 return topicArn;
 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 }
}
```

```

 System.exit(1);
 }
 return "";
}

public static void subscribeQueues(List<QueueData> queues, String topicARN) {
 queues.forEach(queue -> {
 SubscribeRequest subscribeRequest = SubscribeRequest.builder()
 .topicArn(topicARN)
 .endpoint(queue.queueARN)
 .protocol("sqs")
 .build();

 // Subscribe to the endpoint by using the SNS service client.
 // Only Amazon SQS queues can receive notifications from an Amazon SNS
 FIFO
 // topic.
 SubscribeResponse subscribeResponse =
snsClient.subscribe(subscribeRequest);
 System.out.println("The queue [" + queue.queueARN + "] subscribed to the
topic [" + topicARN + "]);
 queue.subscriptionARN = subscribeResponse.subscriptionArn();
 });
}

public static void publishPriceUpdate(String topicArn, String payload, String
groupId) {
 try {
 // Create and publish a message that updates the wholesale price.
 String subject = "Price Update";
 String dedupId = UUID.randomUUID().toString();
 String attributeName = "business";
 String attributeValue = "wholesale";

 MessageAttributeValue msgAttValue = MessageAttributeValue.builder()
 .dataType("String")
 .stringValue(attributeValue)
 .build();

 Map<String, MessageAttributeValue> attributes = new HashMap<>();
 attributes.put(attributeName, msgAttValue);
 PublishRequest pubRequest = PublishRequest.builder()
 .topicArn(topicArn)

```

```
 .subject(subject)
 .message(payload)
 .messageGroupId(groupId)
 .messageDeduplicationId(dedupId)
 .messageAttributes(attributes)
 .build();

 final PublishResponse response = snsClient.publish(pubRequest);
 System.out.println(response.messageId());
 System.out.println(response.sequenceNumber());
 System.out.println("Message was published to " + topicArn);

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateTopic](#)
  - [Publish](#)
  - [Subscribe](#)

## Detect people and objects in a video

The following code example shows how to detect people and objects in a video with Amazon Rekognition.

### SDK for Java 2.x

Shows how to use Amazon Rekognition Java API to create an app to detect faces and objects in videos located in an Amazon Simple Storage Service (Amazon S3) bucket. The app sends the admin an email notification with the results using Amazon Simple Email Service (Amazon SES).

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

### Services used in this example

- Amazon Rekognition



- Amazon S3
- Amazon SES
- Amazon SNS
- Amazon SQS

## Manage large messages using S3

The following code example shows how to use the Amazon SQS Extended Client Library to work with large Amazon SQS messages.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import com.amazon.sqs.javamessaging.AmazonSQSExtendedClient;
import com.amazon.sqs.javamessaging.ExtendedClientConfiguration;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.joda.time.DateTime;
import org.joda.time.format.DateTimeFormat;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.BucketLifecycleConfiguration;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteBucketRequest;
import software.amazon.awssdk.services.s3.model.DeleteObjectRequest;
import software.amazon.awssdk.services.s3.model.ExpirationStatus;
import software.amazon.awssdk.services.s3.model.LifecycleExpiration;
import software.amazon.awssdk.services.s3.model.LifecycleRule;
import software.amazon.awssdk.services.s3.model.LifecycleRuleFilter;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsRequest;
import software.amazon.awssdk.services.s3.model.ListObjectVersionsResponse;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Request;
import software.amazon.awssdk.services.s3.model.ListObjectsV2Response;
import
 software.amazon.awssdk.services.s3.model.PutBucketLifecycleConfigurationRequest;
import software.amazon.awssdk.services.sqs.SqsClient;
```

```
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.CreateQueueResponse;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageResponse;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;

import java.util.Arrays;
import java.util.List;
import java.util.UUID;

/**
 * Example of using Amazon SQS Extended Client Library for Java 2.x.
 */
public class SqsExtendedClientExample {
 private static final Logger logger =
 LoggerFactory.getLogger(SqsExtendedClientExample.class);

 private String s3BucketName;
 private String queueUrl;
 private final String queueName;
 private final S3Client s3Client;
 private final SqsClient sqsExtendedClient;
 private final int messageSize;

 /**
 * Constructor with default clients and message size.
 */
 public SqsExtendedClientExample() {
 this(S3Client.create(), 300000);
 }

 /**
 * Constructor with custom S3 client and message size.
 *
 * @param s3Client The S3 client to use
 * @param messageSize The size of the test message to create
 */
 public SqsExtendedClientExample(S3Client s3Client, int messageSize) {
 this.s3Client = s3Client;
 this.messageSize = messageSize;
 }
}
```

```

 // Generate a unique bucket name.
 this.s3BucketName = UUID.randomUUID() + "-" +
 DateTimeFormat.forPattern("yyMMdd-hhmmss").print(new DateTime());

 // Generate a unique queue name.
 this.queueName = "MyQueue-" + UUID.randomUUID();

 // Configure the SQS extended client.
 final ExtendedClientConfiguration extendedClientConfig = new
ExtendedClientConfiguration()
 .withPayloadSupportEnabled(s3Client, s3BucketName);

 this.sqsExtendedClient = new
AmazonSQSExtendedClient(SqsClient.builder().build(), extendedClientConfig);
 }

 public static void main(String[] args) {
 SqsExtendedClientExample example = new SqsExtendedClientExample();
 try {
 example.setup();
 example.sendAndReceiveMessage();
 } finally {
 example.cleanup();
 }
 }

 /**
 * Send a large message and receive it back.
 *
 * @return The received message
 */
 public Message sendAndReceiveMessage() {
 try {
 // Create a large message.
 char[] chars = new char[messageSize];
 Arrays.fill(chars, 'x');
 String largeMessage = new String(chars);

 // Send the message.
 final SendMessageRequest sendMessageRequest =
SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody(largeMessage)
 .build();

```

```

 sqsExtendedClient.sendMessage(sendMessageRequest);
 logger.info("Sent message of size: {}", largeMessage.length());

 // Receive and return the message.
 final ReceiveMessageResponse receiveMessageResponse =
sqsExtendedClient.receiveMessage(
 ReceiveMessageRequest.builder().queueUrl(queueUrl).build());

 List<Message> messages = receiveMessageResponse.messages();
 if (messages.isEmpty()) {
 throw new RuntimeException("No messages received");
 }

 Message message = messages.getFirst();
 logger.info("\nMessage received.");
 logger.info(" ID: {}", message.messageId());
 logger.info(" Receipt handle: {}", message.receiptHandle());
 logger.info(" Message body size: {}", message.body().length());
 logger.info(" Message body (first 5 characters): {}",
message.body().substring(0, 5));

 return message;
 } catch (RuntimeException e) {
 logger.error("Error during message processing: {}", e.getMessage(), e);
 throw e;
 }
}

```

- For more information, see [AWS SDK for Java 2.x Developer Guide](#).
- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateBucket](#)
  - [PutBucketLifecycleConfiguration](#)
  - [ReceiveMessage](#)
  - [SendMessage](#)

## Process S3 event notifications

The following code example shows how to work with S3 event notifications in an object-oriented way.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

This example show how to process S3 notification event by using Amazon SQS.

```
/**
 * This method receives S3 event notifications by using an SqsAsyncClient.
 * After the client receives the messages it deserializes the JSON payload and
 * logs them. It uses
 * the S3EventNotification class (part of the S3 event notification API for
 * Java) to deserialize
 * the JSON payload and access the messages in an object-oriented way.
 *
 * @param queueUrl The URL of the AWS SQS queue that receives the S3 event
 * notifications.
 * @see <a href="https://sdk.amazonaws.com/java/api/latest/software/amazon/
 * awssdk/eventnotifications/s3/model/package-summary.html">S3EventNotification API</
 * a>.
 *
 * <p>
 * To use S3 event notification serialization/deserialization to objects, add
 * the following
 * dependency to your Maven pom.xml file.
 * <dependency>
 * <groupId>software.amazon.awssdk</groupId>
 * <artifactId>s3-event-notifications</artifactId>
 * <version><LATEST></version>
 * </dependency>
 * <p>
 * The S3 event notification API became available with version 2.25.11 of the
 * Java SDK.
 * <p>
 * This example shows the use of the API with AWS SQS, but it can be used to
 * process S3 event notifications
 * in AWS SNS or AWS Lambda as well.
 * <p>
 * Note: The S3EventNotification class does not work with messages routed
 * through AWS EventBridge.
 */
```

```

static void processS3Events(String bucketName, String queueUrl, String queueArn)
{
 try {
 // Configure the bucket to send Object Created and Object Tagging
 notifications to an existing SQS queue.
 s3Client.putBucketNotificationConfiguration(b -> b
 .notificationConfiguration(ncb -> ncb
 .queueConfigurations(qcb -> qcb
 .events(Event.S3_OBJECT_CREATED,
Event.S3_OBJECT_TAGGING)
 .queueArn(queueArn)))
 .bucket(bucketName)
).join();

 triggerS3EventNotifications(bucketName);
 // Wait for event notifications to propagate.
 Thread.sleep(Duration.ofSeconds(5).toMillis());

 boolean didReceiveMessages = true;
 while (didReceiveMessages) {
 // Display the number of messages that are available in the queue.
 sqsClient.getQueueAttributes(b -> b
 .queueUrl(queueUrl)

 .attributeNames(QueueAttributeName.APPROXIMATE_NUMBER_OF_MESSAGES)
).thenAccept(attributeResponse ->
 logger.info("Approximate number of messages in the
queue: {}",
attributeResponse.attributes().get(QueueAttributeName.APPROXIMATE_NUMBER_OF_MESSAGES)))
 .join();

 // Receive the messages.
 ReceiveMessageResponse response = sqsClient.receiveMessage(b -> b
 .queueUrl(queueUrl)
).get();
 logger.info("Count of received messages: {}",
response.messages().size());
 didReceiveMessages = !response.messages().isEmpty();

 // Create a collection to hold the received message for deletion
 // after we log the messages.
 HashSet<DeleteMessageBatchRequestEntry> messagesToDelete = new
HashSet<>();

```

```

 // Process each message.
 response.messages().forEach(message -> {
 logger.info("Message id: {}", message.messageId());
 // Deserialize JSON message body to a S3EventNotification object
 // to access messages in an object-oriented way.
 S3EventNotification event =
S3EventNotification.fromJson(message.body());

 // Log the S3 event notification record details.
 if (event.getRecords() != null) {
 event.getRecords().forEach(record -> {
 String eventName = record.getEventName();
 String key = record.getS3().getObject().getKey();
 logger.info(record.toString());
 logger.info("Event name is {} and key is {}", eventName,
key);

 });
 }
 // Add logged messages to collection for batch deletion.
 messagesToDelete.add(DeleteMessageBatchRequestEntry.builder()
 .id(message.messageId())
 .receiptHandle(message.receiptHandle())
 .build());
 });
 // Delete messages.
 if (!messagesToDelete.isEmpty()) {
 sqsClient.deleteMessageBatch(DeleteMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(messagesToDelete)
 .build()
).join();
 }
 } // End of while block.
} catch (InterruptedException | ExecutionException e) {
 throw new RuntimeException(e);
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [DeleteMessageBatch](#)
  - [GetQueueAttributes](#)

- [PutBucketNotificationConfiguration](#)
- [ReceiveMessage](#)

## Publish messages to queues

The following code example shows how to:

- Create topic (FIFO or non-FIFO).
- Subscribe several queues to the topic with an option to apply a filter.
- Publish messages to the topic.
- Poll the queues for messages received.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
package com.example.sns;

import
 software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sns.SnsClient;
import software.amazon.awssdk.services.sns.model.CreateTopicRequest;
import software.amazon.awssdk.services.sns.model.CreateTopicResponse;
import software.amazon.awssdk.services.sns.model.DeleteTopicRequest;
import software.amazon.awssdk.services.sns.model.DeleteTopicResponse;
import software.amazon.awssdk.services.sns.model.MessageAttributeValue;
import software.amazon.awssdk.services.sns.model.PublishRequest;
import software.amazon.awssdk.services.sns.model.PublishResponse;
import software.amazon.awssdk.services.sns.model.SetSubscriptionAttributesRequest;
import software.amazon.awssdk.services.sns.model.SnsException;
import software.amazon.awssdk.services.sns.model.SubscribeRequest;
import software.amazon.awssdk.services.sns.model.SubscribeResponse;
import software.amazon.awssdk.services.sns.model.UnsubscribeRequest;
import software.amazon.awssdk.services.sns.model.UnsubscribeResponse;
```



```
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequestEntry;
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueAttributesRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueAttributesResponse;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlResponse;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.QueueAttributeName;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.SetQueueAttributesRequest;
import software.amazon.awssdk.services.sqs.model.SqsException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import com.google.gson.JsonPrimitive;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 * <p>
 * For more information, see the following documentation topic:
 * <p>
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 * <p>
 * This Java example performs these tasks:
 * <p>
 * 1. Gives the user three options to choose from.
 * 2. Creates an Amazon Simple Notification Service (Amazon SNS) topic.
 * 3. Creates an Amazon Simple Queue Service (Amazon SQS) queue.
 * 4. Gets the SQS queue Amazon Resource Name (ARN) attribute.
 * 5. Attaches an AWS Identity and Access Management (IAM) policy to the queue.
 * 6. Subscribes to the SQS queue.
 * 7. Publishes a message to the topic.
 * 8. Displays the messages.
```

```

* 9. Deletes the received message.
* 10. Unsubscribes from the topic.
* 11. Deletes the SNS topic.
*/
public class SNSWorkflow {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage = "\n" +
 "Usage:\n" +
 " <fifoQueueARN>\n\n" +
 "Where:\n" +
 " accountId - Your AWS account Id value.";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 SnsClient snsClient = SnsClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();

 SqsClient sqsClient = SqsClient.builder()
 .region(Region.US_EAST_1)
 .credentialsProvider(EnvironmentVariableCredentialsProvider.create())
 .build();

 Scanner in = new Scanner(System.in);
 String accountId = args[0];
 String useFIFO;
 String duplication = "n";
 String topicName;
 String deduplicationID = null;
 String groupId = null;

 String topicArn;
 String sqsQueueName;
 String sqsQueueUrl;
 String sqsQueueArn;
 String subscriptionArn;
 boolean selectFIFO = false;

```

```
String message;
List<Message> messageList;
List<String> filterList = new ArrayList<>();
String msgAttValue = "";

System.out.println(DASHES);
System.out.println("Welcome to messaging with topics and queues.");
System.out.println("In this scenario, you will create an SNS topic and
subscribe an SQS queue to the topic.\n" +
 "You can select from several options for configuring the topic and the
subscriptions for the queue.\n" +
 "You can then post to the topic and see the results in the queue.");
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("SNS topics can be configured as FIFO (First-In-First-
Out).\n" +
 "FIFO topics deliver messages in order and support deduplication and
message filtering.\n" +
 "Would you like to work with FIFO topics? (y/n)");
useFIFO = in.nextLine();
if (useFIFO.compareTo("y") == 0) {
 selectFIFO = true;
 System.out.println("You have selected FIFO");
 System.out.println(" Because you have chosen a FIFO topic, deduplication
is supported.\n" +
 " Deduplication IDs are either set in the message or
automatically generated from content using a hash function.\n"
 +
 " If a message is successfully published to an SNS FIFO
topic, any message published and determined to have the same deduplication ID,\n"
 +
 " within the five-minute deduplication interval, is accepted
but not delivered.\n" +
 " For more information about deduplication, see https://
docs.aws.amazon.com/sns/latest/dg/fifo-message-dedup.html.");

 System.out.println(
 "Would you like to use content-based deduplication instead of
entering a deduplication ID? (y/n)");
 duplication = in.nextLine();
 if (duplication.compareTo("y") == 0) {
 System.out.println("Please enter a group id value");
 groupId = in.nextLine();
```

```
 } else {
 System.out.println("Please enter deduplication Id value");
 deduplicationID = in.nextLine();
 System.out.println("Please enter a group id value");
 groupId = in.nextLine();
 }
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Create a topic.");
 System.out.println("Enter a name for your SNS topic.");
 topicName = in.nextLine();
 if (selectFIFO) {
 System.out.println("Because you have selected a FIFO topic, '.fifo' must
be appended to the topic name.");
 topicName = topicName + ".fifo";
 System.out.println("The name of the topic is " + topicName);
 topicArn = createFIFO(snsClient, topicName, duplication);
 System.out.println("The ARN of the FIFO topic is " + topicArn);

 } else {
 System.out.println("The name of the topic is " + topicName);
 topicArn = createSNSTopic(snsClient, topicName);
 System.out.println("The ARN of the non-FIFO topic is " + topicArn);

 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Create an SQS queue.");
 System.out.println("Enter a name for your SQS queue.");
 sqsQueueName = in.nextLine();
 if (selectFIFO) {
 sqsQueueName = sqsQueueName + ".fifo";
 }
 sqsQueueUrl = createQueue(sqsClient, sqsQueueName, selectFIFO);
 System.out.println("The queue URL is " + sqsQueueUrl);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. Get the SQS queue ARN attribute.");
 sqsQueueArn = getSQSQueueAttrs(sqsClient, sqsQueueUrl);
 System.out.println("The ARN of the new queue is " + sqsQueueArn);
```

```

System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("5. Attach an IAM policy to the queue.");

// Define the policy to use. Make sure that you change the REGION if you are
// running this code
// in a different region.
String policy = ""
{
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": {
 "Service": "sns.amazonaws.com"
 },
 "Action": "sqs:SendMessage",
 "Resource": "arn:aws:sqs:us-east-1:%s:%s",
 "Condition": {
 "ArnEquals": {
 "aws:SourceArn": "arn:aws:sns:us-east-1:%s:%s"
 }
 }
 }
]
}
"".formatted(accountId, sqsQueueName, accountId, topicName);

setQueueAttr(sqsClient, sqsQueueUrl, policy);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("6. Subscribe to the SQS queue.");
if (selectFIFO) {
 System.out.println(
 "If you add a filter to this subscription, then only the filtered
messages will be received in the queue.\n"
 +
 "For information about message filtering, see https://
docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html\n"
 +
 "For this example, you can filter messages by a \"tone\"
attribute.");
}

```

```

 System.out.println("Would you like to filter messages for " +
sqsQueueName + "'s subscription to the topic "
 + topicName + "? (y/n)");
 String filterAns = in.nextLine();
 if (filterAns.compareTo("y") == 0) {
 boolean moreAns = false;
 System.out.println("You can filter messages by one or more of the
following \"tone\" attributes.");
 System.out.println("1. cheerful");
 System.out.println("2. funny");
 System.out.println("3. serious");
 System.out.println("4. sincere");
 while (!moreAns) {
 System.out.println("Select a number or choose 0 to end.");
 String ans = in.nextLine();
 switch (ans) {
 case "1":
 filterList.add("cheerful");
 break;
 case "2":
 filterList.add("funny");
 break;
 case "3":
 filterList.add("serious");
 break;
 case "4":
 filterList.add("sincere");
 break;
 default:
 moreAns = true;
 break;
 }
 }
 }
 }
 subscriptionArn = subQueue(snsClient, topicArn, sqsQueueArn, filterList);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Publish a message to the topic.");
 if (selectFIFO) {
 System.out.println("Would you like to add an attribute to this message?
(y/n)");
 String msgAns = in.nextLine();
 }

```

```

 if (msgAns.compareTo("y") == 0) {
 System.out.println("You can filter messages by one or more of the
following \"tone\" attributes.");
 System.out.println("1. cheerful");
 System.out.println("2. funny");
 System.out.println("3. serious");
 System.out.println("4. sincere");
 System.out.println("Select a number or choose 0 to end.");
 String ans = in.nextLine();
 switch (ans) {
 case "1":
 msgAttValue = "cheerful";
 break;
 case "2":
 msgAttValue = "funny";
 break;
 case "3":
 msgAttValue = "serious";
 break;
 default:
 msgAttValue = "sincere";
 break;
 }

 System.out.println("Selected value is " + msgAttValue);
 }
 System.out.println("Enter a message.");
 message = in.nextLine();
 pubMessageFIFO(snsClient, message, topicArn, msgAttValue, duplication,
groupId, deduplicationID);

 } else {
 System.out.println("Enter a message.");
 message = in.nextLine();
 pubMessage(snsClient, message, topicArn);
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Display the message. Press any key to continue.");
 in.nextLine();
 messageList = receiveMessages(sqsClient, sqsQueueUrl, msgAttValue);
 for (Message mes : messageList) {
 System.out.println("Message Id: " + mes.messageId());
 }
}

```

```

 System.out.println("Full Message: " + mes.body());
 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. Delete the received message. Press any key to
continue.");
 in.nextLine();
 deleteMessages(sqsClient, sqsQueueUrl, messageList);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Unsubscribe from the topic and delete the queue.
Press any key to continue.");
 in.nextLine();
 unSub(snsClient, subscriptionArn);
 deleteSQSQueue(sqsClient, sqsQueueName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("11. Delete the topic. Press any key to continue.");
 in.nextLine();
 deleteSNSTopic(snsClient, topicArn);

 System.out.println(DASHES);
 System.out.println("The SNS/SQS workflow has completed successfully.");
 System.out.println(DASHES);
}

public static void deleteSNSTopic(SnsClient snsClient, String topicArn) {
 try {
 DeleteTopicRequest request = DeleteTopicRequest.builder()
 .topicArn(topicArn)
 .build();

 DeleteTopicResponse result = snsClient.deleteTopic(request);
 System.out.println("Status was " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}

```



```

public static void deleteSQSQueue(SqsClient sqsClient, String queueName) {
 try {
 GetQueueUrlRequest getQueueRequest = GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build();

 String queueUrl = sqsClient.getQueueUrl(getQueueRequest).queueUrl();
 DeleteQueueRequest deleteQueueRequest = DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();

 sqsClient.deleteQueue(deleteQueueRequest);
 System.out.println(queueName + " was successfully deleted.");

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void unSub(SnsClient snsClient, String subscriptionArn) {
 try {
 UnsubscribeRequest request = UnsubscribeRequest.builder()
 .subscriptionArn(subscriptionArn)
 .build();

 UnsubscribeResponse result = snsClient.unsubscribe(request);
 System.out.println("Status was " + result.sdkHttpResponse().statusCode()
 + "\nSubscription was removed for " + request.subscriptionArn());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void deleteMessages(SqsClient sqsClient, String queueUrl,
 List<Message> messages) {
 try {
 List<DeleteMessageBatchRequestEntry> entries = new ArrayList<>();
 for (Message msg : messages) {
 DeleteMessageBatchRequestEntry entry =
 DeleteMessageBatchRequestEntry.builder()

```

```

 .id(msg.messageId())
 .build();

 entries.add(entry);
}

DeleteMessageBatchRequest deleteMessageBatchRequest =
DeleteMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(entries)
 .build();

sqsClient.deleteMessageBatch(deleteMessageBatchRequest);
System.out.println("The batch delete of messages was successful");

} catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}
}

public static List<Message> receiveMessages(SqsClient sqsClient, String
queueUrl, String msgAttValue) {
 try {
 if (msgAttValue.isEmpty()) {
 ReceiveMessageRequest receiveMessageRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .numberOfMessages(5)
 .build();
 return sqsClient.receiveMessage(receiveMessageRequest).messages();
 } else {
 // We know there are filters on the message.
 ReceiveMessageRequest receiveRequest =
ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageAttributeNames(msgAttValue) // Include other message
attributes if needed.
 .numberOfMessages(5)
 .build();

 return sqsClient.receiveMessage(receiveRequest).messages();
 }
 }
}

```

```

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return null;
}

public static void pubMessage(SnsClient snsClient, String message, String
topicArn) {
 try {
 PublishRequest request = PublishRequest.builder()
 .message(message)
 .topicArn(topicArn)
 .build();

 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status is " +
result.sdkHttpResponse().statusCode());

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void pubMessageFIFO(SnsClient snsClient,
 String message,
 String topicArn,
 String msgAttValue,
 String duplication,
 String groupId,
 String deduplicationID) {

 try {
 PublishRequest request;
 // Means the user did not choose to use a message attribute.
 if (msgAttValue.isEmpty()) {
 if (duplication.compareTo("y") == 0) {
 request = PublishRequest.builder()
 .message(message)
 .messageGroupId(groupId)
 .topicArn(topicArn)
 .build();
 }
 }
 }
}

```

```

 } else {
 request = PublishRequest.builder()
 .message(message)
 .messageDeduplicationId(deduplicationID)
 .messageGroupId(groupId)
 .topicArn(topicArn)
 .build();
 }

 } else {
 Map<String, MessageAttributeValue> messageAttributes = new
HashMap<>();
 messageAttributes.put(msgAttValue, MessageAttributeValue.builder()
 .dataType("String")
 .stringValue("true")
 .build());

 if (duplication.compareTo("y") == 0) {
 request = PublishRequest.builder()
 .message(message)
 .messageGroupId(groupId)
 .topicArn(topicArn)
 .build();
 } else {
 // Create a publish request with the message and attributes.
 request = PublishRequest.builder()
 .topicArn(topicArn)
 .message(message)
 .messageDeduplicationId(deduplicationID)
 .messageGroupId(groupId)
 .messageAttributes(messageAttributes)
 .build();
 }
 }

 // Publish the message to the topic.
 PublishResponse result = snsClient.publish(request);
 System.out
 .println(result.messageId() + " Message sent. Status was " +
result.sdkHttpResponse().statusCode());

} catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
}

```

```

 }
}

// Subscribe to the SQS queue.
public static String subQueue(SnsClient snsClient, String topicArn, String
queueArn, List<String> filterList) {
 try {
 SubscribeRequest request;
 if (filterList.isEmpty()) {
 // No filter subscription is added.
 request = SubscribeRequest.builder()
 .protocol("sqs")
 .endpoint(queueArn)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("The queue " + queueArn + " has been subscribed
to the topic " + topicArn + "\n" +
 "with the subscription ARN " + result.subscriptionArn());
 return result.subscriptionArn();
 } else {
 request = SubscribeRequest.builder()
 .protocol("sqs")
 .endpoint(queueArn)
 .returnSubscriptionArn(true)
 .topicArn(topicArn)
 .build();

 SubscribeResponse result = snsClient.subscribe(request);
 System.out.println("The queue " + queueArn + " has been subscribed
to the topic " + topicArn + "\n" +
 "with the subscription ARN " + result.subscriptionArn());

 String attributeName = "FilterPolicy";
 Gson gson = new Gson();
 String jsonString = "{\"tone\": []}";
 JsonObject jsonObject = gson.fromJson(jsonString, JsonObject.class);
 JsonArray toneArray = jsonObject.getAsJsonArray("tone");
 for (String value : filterList) {
 toneArray.add(new JsonPrimitive(value));
 }
 }
 }
}

```

```

 String updatedJsonString = gson.toJson(jsonObject);
 System.out.println(updatedJsonString);
 SetSubscriptionAttributesRequest attRequest =
SetSubscriptionAttributesRequest.builder()
 .subscriptionArn(result.subscriptionArn())
 .attributeName(attributeName)
 .attributeValue(updatedJsonString)
 .build();

 snsClient.setSubscriptionAttributes(attRequest);
 return result.subscriptionArn();
 }

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

// Attach a policy to the queue.
public static void setQueueAttr(SqsClient sqsClient, String queueUrl, String
policy) {
 try {
 Map<software.amazon.awssdk.services.sqs.model.QueueAttributeName,
String> attrMap = new HashMap<>();
 attrMap.put(QueueAttributeName.POLICY, policy);

 SetQueueAttributesRequest attributesRequest =
SetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)
 .attributes(attrMap)
 .build();

 sqsClient.setQueueAttributes(attributesRequest);
 System.out.println("The policy has been successfully attached.");

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static String getSQSQueueAttrs(SqsClient sqsClient, String queueUrl) {

```

```

 // Specify the attributes to retrieve.
 List<QueueAttributeName> atts = new ArrayList<>();
 atts.add(QueueAttributeName.QUEUE_ARN);

 GetQueueAttributesRequest attributesRequest =
 GetQueueAttributesRequest.builder()
 .queueUrl(queueUrl)
 .attributeNames(atts)
 .build();

 GetQueueAttributesResponse response =
 sqsClient.getQueueAttributes(attributesRequest);
 Map<String, String> queueAtts = response.attributesAsStrings();
 for (Map.Entry<String, String> queueAtt : queueAtts.entrySet())
 return queueAtt.getValue();

 return "";
 }

 public static String createQueue(SqsClient sqsClient, String queueName, Boolean
selectFIFO) {
 try {
 System.out.println("\nCreate Queue");
 if (selectFIFO) {
 Map<QueueAttributeName, String> attrs = new HashMap<>();
 attrs.put(QueueAttributeName.FIFO_QUEUE, "true");
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .attributes(attrs)
 .build();

 sqsClient.createQueue(createQueueRequest);
 System.out.println("\nGet queue url");
 GetQueueUrlResponse getQueueUrlResponse = sqsClient

.getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 return getQueueUrlResponse.queueUrl();
 } else {
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();

 sqsClient.createQueue(createQueueRequest);
 System.out.println("\nGet queue url");
 }
 }
 }

```

```

 GetQueueUrlResponse getQueueUrlResponse = sqsClient
 .getQueueUrl(GetQueueUrlRequest.builder().queueName(queueName).build());
 return getQueueUrlResponse.queueUrl();
 }

 } catch (SqsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createSNSTopic(SnsClient snsClient, String topicName) {
 CreateTopicResponse result;
 try {
 CreateTopicRequest request = CreateTopicRequest.builder()
 .name(topicName)
 .build();

 result = snsClient.createTopic(request);
 return result.topicArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createFIFO(SnsClient snsClient, String topicName, String
duplication) {
 try {
 // Create a FIFO topic by using the SNS service client.
 Map<String, String> topicAttributes = new HashMap<>();
 if (duplication.compareTo("n") == 0) {
 topicAttributes.put("FifoTopic", "true");
 topicAttributes.put("ContentBasedDeduplication", "false");
 } else {
 topicAttributes.put("FifoTopic", "true");
 topicAttributes.put("ContentBasedDeduplication", "true");
 }

 CreateTopicRequest topicRequest = CreateTopicRequest.builder()

```



```
 .name(topicName)
 .attributes(topicAttributes)
 .build();

 CreateTopicResponse response = snsClient.createTopic(topicRequest);
 return response.topicArn();

 } catch (SnsException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateQueue](#)
- [CreateTopic](#)
- [DeleteMessageBatch](#)
- [DeleteQueue](#)
- [DeleteTopic](#)
- [GetQueueAttributes](#)
- [Publish](#)
- [ReceiveMessage](#)
- [SetQueueAttributes](#)
- [Subscribe](#)
- [Unsubscribe](#)

## Send and receive batches of messages

The following code example shows how to:

- Create an Amazon SQS queue.
- Send batches of messages to the queue.
- Receive batches of messages from the queue.
- Delete batches of messages from the queue.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

As shown in the following examples, you can handle batch message operations with Amazon SQS using two different approaches with the AWS SDK for Java 2.x:

**SendRecvBatch.java** uses explicit batch operations. You manually create message batches and call `sendMessageBatch()` and `deleteMessageBatch()` directly. You also handle batch responses, including any failed messages. This approach gives you full control over batch sizing and error handling. However, it requires more code to manage the batching logic.

**SimpleProducerConsumer.java** uses the high-level `SqsAsyncBatchManager` library for automatic request batching. You make individual `sendMessage()` and `deleteMessage()` calls with the same method signatures as the standard client. The SDK automatically buffers these calls and sends them as batch operations. This approach requires minimal code changes while providing batching performance benefits.

Use explicit batching when you need fine-grained control over batch composition and error handling. Use automatic batching when you want to optimize performance with minimal code changes.

**SendRecvBatch.java** - Uses explicit batch operations with messages.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.BatchResultErrorEntry;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchRequestEntry;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchResponse;
import software.amazon.awssdk.services.sqs.model.DeleteMessageBatchResultEntry;
import software.amazon.awssdk.services.sqs.model.DeleteQueueRequest;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.MessageAttributeValue;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
```

```
import software.amazon.awssdk.services.sqs.model.SendMessageBatchRequest;
import software.amazon.awssdk.services.sqs.model.SendMessageBatchRequestEntry;
import software.amazon.awssdk.services.sqs.model.SendMessageBatchResponse;
import software.amazon.awssdk.services.sqs.model.SendMessageBatchResultEntry;
import software.amazon.awssdk.services.sqs.model.SqsException;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

/**
 * This code demonstrates basic message operations in Amazon Simple Queue Service
 * (Amazon SQS).
 */

public class SendRecvBatch {
 private static final Logger LOGGER =
 LoggerFactory.getLogger(SendRecvBatch.class);
 private static final SqsClient sqsClient = SqsClient.create();

 public static void main(String[] args) {
 usageDemo();
 }

 /**
 * Send a batch of messages in a single request to an SQS queue.
 * This request may return overall success even when some messages were not
 sent.
 * The caller must inspect the Successful and Failed lists in the response and

```

```

 * resend any failed messages.
 *
 * @param queueUrl The URL of the queue to receive the messages.
 * @param messages The messages to send to the queue. Each message contains a
 body and attributes.
 * @return The response from SQS that contains the list of successful and failed
 messages.
 */
 public static SendMessageBatchResponse sendMessages(
 String queueUrl, List<MessageEntry> messages) {

 try {
 List<SendMessageBatchRequestEntry> entries = new ArrayList<>();

 for (int i = 0; i < messages.size(); i++) {
 MessageEntry message = messages.get(i);
 entries.add(SendMessageBatchRequestEntry.builder()
 .id(String.valueOf(i))
 .messageBody(message.getBody())
 .messageAttributes(message.getAttributes())
 .build());
 }

 SendMessageBatchRequest sendBatchRequest =
 SendMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(entries)
 .build();

 SendMessageBatchResponse response =
 sqsClient.sendMessageBatch(sendBatchRequest);

 if (!response.successful().isEmpty()) {
 for (SendMessageBatchResultEntry resultEntry :
 response.successful()) {
 LOGGER.info("Message sent: {}: {}", resultEntry.messageId(),
 messages.get(Integer.parseInt(resultEntry.id())).getBody());
 }
 }

 if (!response.failed().isEmpty()) {
 for (BatchResultErrorEntry errorEntry : response.failed()) {
 LOGGER.warn("Failed to send: {}: {}", errorEntry.id(),

```

```

messages.get(Integer.parseInt(errorEntry.id())).getBody());
 }
 }

 return response;

} catch (SqsException e) {
 LOGGER.error("Send messages failed to queue: {}", queueUrl, e);
 throw e;
}
}

/**
 * Receive a batch of messages in a single request from an SQS queue.
 *
 * @param queueUrl The URL of the queue from which to receive messages.
 * @param maxNumber The maximum number of messages to receive (capped at 10 by
SQS).
 *
 * The actual number of messages received might be less.
 * @param waitTime The maximum time to wait (in seconds) before returning.
When
 *
 * this number is greater than zero, long polling is used.
This
 *
 * can result in reduced costs and fewer false empty
responses.
 * @return The list of Message objects received. These each contain the body
 *
 * of the message and metadata and custom attributes.
 */
public static List<Message> receiveMessages(String queueUrl, int maxNumber, int
waitTime) {
 try {
 ReceiveMessageRequest receiveRequest = ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .maxNumberOfMessages(maxNumber)
 .waitTimeSeconds(waitTime)
 .messageAttributeNames("All")
 .build();

 List<Message> messages =
sqsClient.receiveMessage(receiveRequest).messages();

 for (Message message : messages) {

```

```

 LOGGER.info("Received message: {}: {}", message.messageId(),
message.body());
 }

 return messages;

} catch (SqsException e) {
 LOGGER.error("Couldn't receive messages from queue: {}", queueUrl, e);
 throw e;
}
}

/**
 * Delete a batch of messages from a queue in a single request.
 *
 * @param queueUrl The URL of the queue from which to delete the messages.
 * @param messages The list of messages to delete.
 * @return The response from SQS that contains the list of successful and failed
 * message deletions.
 */
public static DeleteMessageBatchResponse deleteMessages(String queueUrl,
List<Message> messages) {
 try {
 List<DeleteMessageBatchRequestEntry> entries = new ArrayList<>();

 for (int i = 0; i < messages.size(); i++) {
 entries.add(DeleteMessageBatchRequestEntry.builder()
 .id(String.valueOf(i))
 .receiptHandle(messages.get(i).receiptHandle())
 .build());
 }

 DeleteMessageBatchRequest deleteRequest =
DeleteMessageBatchRequest.builder()
 .queueUrl(queueUrl)
 .entries(entries)
 .build();

 DeleteMessageBatchResponse response =
sqsClient.deleteMessageBatch(deleteRequest);

 if (!response.successful().isEmpty()) {
 for (DeleteMessageBatchResultEntry resultEntry :
response.successful()) {

```

```

 LOGGER.info("Deleted {}",
messages.get(Integer.parseInt(resultEntry.id())).receiptHandle());
 }
}

 if (!response.failed().isEmpty()) {
 for (BatchResultErrorEntry errorEntry : response.failed()) {
 LOGGER.warn("Could not delete {}",
messages.get(Integer.parseInt(errorEntry.id())).receiptHandle());
 }
 }

 return response;

} catch (SqsException e) {
 LOGGER.error("Couldn't delete messages from queue {}", queueUrl, e);
 throw e;
}
}

/**
 * Helper class to represent a message with body and attributes.
 */
public static class MessageEntry {
 private final String body;
 private final Map<String, MessageAttributeValue> attributes;

 public MessageEntry(String body, Map<String, MessageAttributeValue>
attributes) {
 this.body = body;
 this.attributes = attributes != null ? attributes : new HashMap<>();
 }

 public String getBody() {
 return body;
 }

 public Map<String, MessageAttributeValue> getAttributes() {
 return attributes;
 }
}

/**
 * Shows how to:

```

```

 * * Read the lines from a file and send the lines in
 * batches of 10 as messages to a queue.
 * * Receive the messages in batches until the queue is empty.
 * * Reassemble the lines of the file and verify they match the original file.
 */
 public static void usageDemo() {
 LOGGER.info("-".repeat(88));
 LOGGER.info("Welcome to the Amazon Simple Queue Service (Amazon SQS)
demo!");
 LOGGER.info("-".repeat(88));

 String queueUrl = null;
 try {
 // Create a queue for the demo.
 String queueName = "sqs-usage-demo-message-wrapper-" +
System.currentTimeMillis();
 CreateQueueRequest createRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .build();
 queueUrl = sqsClient.createQueue(createRequest).queueUrl();
 LOGGER.info("Created queue: {}", queueUrl);

 try (InputStream inputStream =
SendRecvBatch.class.getResourceAsStream("/log4j2.xml");
 BufferedReader reader = new BufferedReader(new
InputStreamReader(inputStream))) {

 List<String> lines = reader.lines().toList();

 // Send file lines in batches.
 int batchSize = 10;
 LOGGER.info("Sending file lines in batches of {} as messages.",
batchSize);

 for (int i = 0; i < lines.size(); i += batchSize) {
 List<MessageEntry> messageBatch = new ArrayList<>();

 for (int j = i; j < Math.min(i + batchSize, lines.size()); j++)
{
 String line = lines.get(j);
 if (line == null || line.trim().isEmpty()) {
 continue; // Skip empty lines.
 }

```



```

 Map<String, MessageAttributeValue> attributes = new
HashMap<>();
 attributes.put("line", MessageAttributeValue.builder()
 .dataType("String")
 .stringValue(String.valueOf(j))
 .build());

 messageBatch.add(new MessageEntry(lines.get(j),
attributes));
 }

 sendMessages(queueUrl, messageBatch);
 System.out.print(".");
 System.out.flush();
}

LOGGER.info("\nDone. Sent {} messages.", lines.size());

// Receive and process messages.
LOGGER.info("Receiving, handling, and deleting messages in batches
of {}. ", batchSize);
String[] receivedLines = new String[lines.size()];
boolean moreMessages = true;

while (moreMessages) {
 List<Message> receivedMessages = receiveMessages(queueUrl,
batchSize, 5);

 for (Message message : receivedMessages) {
 int lineNumber =
Integer.parseInt(message.messageAttributes().get("line").stringValue());
 receivedLines[lineNumber] = message.body();
 }

 if (!receivedMessages.isEmpty()) {
 deleteMessages(queueUrl, receivedMessages);
 } else {
 moreMessages = false;
 }
}

LOGGER.info("\nDone.");

// Verify that all lines were received correctly.

```

```

 boolean allLinesMatch = true;
 for (int i = 0; i < lines.size(); i++) {
 String originalLine = lines.get(i);
 String receivedLine = receivedLines[i] == null ? "" :
receivedLines[i];

 if (!originalLine.equals(receivedLine)) {
 allLinesMatch = false;
 break;
 }
 }

 if (allLinesMatch) {
 LOGGER.info("Successfully reassembled all file lines!");
 } else {
 LOGGER.info("Uh oh, some lines were missed!");
 }
 }
} catch (SqsException e) {
 LOGGER.error("SQS operation failed", e);
} catch (RuntimeException | IOException e) {
 LOGGER.error("Unexpected runtime error during demo", e);
} finally {
 // Clean up by deleting the queue if it was created.
 if (queueUrl != null) {
 try {
 DeleteQueueRequest deleteQueueRequest =
DeleteQueueRequest.builder()
 .queueUrl(queueUrl)
 .build();
 sqsClient.deleteQueue(deleteQueueRequest);
 LOGGER.info("Deleted queue: {}", queueUrl);
 } catch (SqsException e) {
 LOGGER.error("Failed to delete queue: {}", queueUrl, e);
 }
 }
}

LOGGER.info("Thanks for watching!");
LOGGER.info("-".repeat(88));
}
}

```

## SimpleProducerConsumer.java - Uses automatic batching of messages.

```
package com.example.sqs;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.sqs.SqsAsyncClient;
import software.amazon.awssdk.services.sqs.batchmanager.SqsAsyncBatchManager;
import software.amazon.awssdk.services.sqs.model.DeleteMessageRequest;
import software.amazon.awssdk.services.sqs.model.DeleteMessageResponse;
import software.amazon.awssdk.services.sqs.model.GetQueueUrlRequest;
import software.amazon.awssdk.services.sqs.model.Message;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageRequest;
import software.amazon.awssdk.services.sqs.model.ReceiveMessageResponse;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;
import software.amazon.awssdk.services.sqs.model.SendMessageResponse;
import software.amazon.awssdk.core.exception.SdkException;

import java.math.BigInteger;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Demonstrates the AWS SDK for Java 2.x Automatic Request Batching API for Amazon
 * SQS.
 *
 * This example showcases the high-level SqsAsyncBatchManager library that provides
 * efficient batching and buffering for SQS operations. The batch manager offers
 * methods that directly mirror SqsAsyncClient methods—sendMessage,
 * changeMessageVisibility,
 * deleteMessage, and receiveMessage—making it a drop-in replacement with minimal
 * code changes.
 *
 * Key features of the SqsAsyncBatchManager:
 * - Automatic batching: The SDK automatically buffers individual requests and sends
 * them
 * as batches when maxBatchSize (default: 10) or sendRequestFrequency (default:
 * 200ms)
 * thresholds are reached

```

```

* - Familiar API: Method signatures match SqsAsyncClient exactly, requiring no
learning curve
* - Background optimization: The batch manager maintains internal buffers and
handles
* batching logic transparently
* - Asynchronous operations: All methods return CompletableFuture for non-blocking
execution
*
* Performance benefits demonstrated:
* - Reduced API calls: Multiple individual requests are consolidated into single
batch operations
* - Lower costs: Fewer API calls result in reduced SQS charges
* - Higher throughput: Batch operations process more messages per second
* - Efficient resource utilization: Fewer network round trips and better connection
reuse
*
* This example compares:
* 1. Single-message operations using SqsAsyncClient directly
* 2. Batch operations using SqsAsyncBatchManager with identical method calls
*
* Usage patterns:
* - Set batch size to 1 to use SqsAsyncClient for baseline performance measurement
* - Set batch size > 1 to use SqsAsyncBatchManager for optimized batch processing
* - Monitor real-time throughput metrics to observe performance improvements
*
* Prerequisites:
* - AWS SDK for Java 2.x version 2.28.0 or later
* - An existing SQS queue
* - Valid AWS credentials configured
*
* The program displays real-time metrics showing the dramatic performance
difference
* between individual operations and automatic batching.
*/
public class SimpleProducerConsumer {

 // The maximum runtime of the program.
 private final static int MAX_RUNTIME_MINUTES = 60;
 private final static Logger log =
LoggerFactory.getLogger(SimpleProducerConsumer.class);

 /**
 * Runs the SQS batching demonstration with user-configured parameters.
 */

```

```

 * Prompts for queue name, thread counts, batch size, message size, and runtime.
 * Creates producer and consumer threads to demonstrate batching performance.
 *
 * @param args command line arguments (not used)
 * @throws InterruptedException if thread operations are interrupted
 */
 public static void main(String[] args) throws InterruptedException {

 final Scanner input = new Scanner(System.in);

 System.out.print("Enter the queue name: ");
 final String queueName = input.nextLine();

 System.out.print("Enter the number of producers: ");
 final int producerCount = input.nextInt();

 System.out.print("Enter the number of consumers: ");
 final int consumerCount = input.nextInt();

 System.out.print("Enter the number of messages per batch: ");
 final int batchSize = input.nextInt();

 System.out.print("Enter the message size in bytes: ");
 final int messageSizeByte = input.nextInt();

 System.out.print("Enter the run time in minutes: ");
 final int runTimeMinutes = input.nextInt();

 // Create SQS async client and batch manager for all operations.
 // The SqsAsyncBatchManager is created from the SqsAsyncClient using the
 // batchManager() factory method, which provides default batching
configuration.
 // This high-level library automatically handles request buffering and
batching
 // while maintaining the same method signatures as SqsAsyncClient.
 final SqsAsyncClient sqsAsyncClient = SqsAsyncClient.create();
 final SqsAsyncBatchManager batchManager = sqsAsyncClient.batchManager();

 final String queueUrl =
sqsAsyncClient.getQueueUrl(GetQueueUrlRequest.builder()
 .queueName(queueName)
 .build()).join().queueUrl();

 // The flag used to stop producer, consumer, and monitor threads.

```

```
final AtomicBoolean stop = new AtomicBoolean(false);

// Start the producers.
final AtomicInteger producedCount = new AtomicInteger();
final Thread[] producers = new Thread[producerCount];
for (int i = 0; i < producerCount; i++) {
 if (batchSize == 1) {
 producers[i] = new Producer(sqsAsyncClient, queueUrl,
messageSizeByte,
 producedCount, stop);
 } else {
 producers[i] = new BatchProducer(batchManager, queueUrl, batchSize,
messageSizeByte, producedCount, stop);
 }
 producers[i].start();
}

// Start the consumers.
final AtomicInteger consumedCount = new AtomicInteger();
final Thread[] consumers = new Thread[consumerCount];
for (int i = 0; i < consumerCount; i++) {
 if (batchSize == 1) {
 consumers[i] = new Consumer(sqsAsyncClient, queueUrl, consumedCount,
stop);
 } else {
 consumers[i] = new BatchConsumer(batchManager, queueUrl, batchSize,
consumedCount, stop);
 }
 consumers[i].start();
}

// Start the monitor thread.
final Thread monitor = new Monitor(producedCount, consumedCount, stop);
monitor.start();

// Wait for the specified amount of time then stop.
Thread.sleep(TimeUnit.MINUTES.toMillis(Math.min(runTimeMinutes,
MAX_RUNTIME_MINUTES)));
stop.set(true);

// Join all threads.
for (int i = 0; i < producerCount; i++) {
 producers[i].join();
}
```

```

 for (int i = 0; i < consumerCount; i++) {
 consumers[i].join();
 }

 monitor.interrupt();
 monitor.join();

 // Close resources
 batchManager.close();
 sqsAsyncClient.close();
 }

 /**
 * Creates a random string of approximately the specified size in bytes.
 *
 * @param sizeByte the target size in bytes for the generated string
 * @return a random string encoded in base-32
 */
 private static String makeRandomString(int sizeByte) {
 final byte[] bs = new byte[(int) Math.ceil(sizeByte * 5 / 8)];
 new Random().nextBytes(bs);
 bs[0] = (byte) ((bs[0] | 64) & 127);
 return new BigInteger(bs).toString(32);
 }

 /**
 * Sends messages individually using SqsAsyncClient for baseline performance
 measurement.
 *
 * This producer demonstrates traditional single-message operations without
 batching.
 * Each sendMessage() call results in a separate API request to SQS, providing
 * a performance baseline for comparison with the batch operations.
 *
 * The sendMessage() method signature is identical to
 SqsAsyncBatchManager.sendMessage(),
 * showing how the high-level batching library maintains API compatibility while
 * adding automatic optimization behind the scenes.
 */
 private static class Producer extends Thread {
 final SqsAsyncClient sqsAsyncClient;
 final String queueUrl;
 final AtomicInteger producedCount;
 }

```

```

final AtomicBoolean stop;
final String theMessage;

/**
 * Creates a producer thread for single-message operations.
 *
 * @param sqsAsyncClient the SQS client for sending messages
 * @param queueUrl the URL of the target queue
 * @param messageSizeByte the size of messages to generate
 * @param producedCount shared counter for tracking sent messages
 * @param stop shared flag to signal thread termination
 */
Producer(SqsAsyncClient sqsAsyncClient, String queueUrl, int
messageSizeByte,
 AtomicInteger producedCount, AtomicBoolean stop) {
 this.sqsAsyncClient = sqsAsyncClient;
 this.queueUrl = queueUrl;
 this.producedCount = producedCount;
 this.stop = stop;
 this.theMessage = makeRandomString(messageSizeByte);
}

/**
 * Continuously sends messages until the stop flag is set.
 *
 * Uses SqsAsyncClient.sendMessage() directly, resulting in one API call per
message.
 * This approach provides baseline performance metrics for comparison with
batching.
 * Each call blocks until the individual message is sent, demonstrating
traditional
 * one-request-per-operation behavior.
 */
public void run() {
 try {
 while (!stop.get()) {
 sqsAsyncClient.sendMessage(SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody(theMessage)
 .build()).join();
 producedCount.incrementAndGet();
 }
 } catch (SdkException | java.util.concurrent.CompletionException e) {

```



```

 // Handle both SdkException and CompletionException from async
operations.
 // If this unlikely condition occurs, stop.
 log.error("Producer: " + e.getMessage());
 System.exit(1);
 }
}

/**
 * Sends messages using SqsAsyncBatchManager for automatic request batching and
optimization.
 *
 * This producer demonstrates the AWS SDK for Java 2.x high-level batching
library.
 * The SqsAsyncBatchManager automatically buffers individual sendMessage() calls
and
 * sends them as batches when thresholds are reached:
 * - maxBatchSize: Maximum 10 messages per batch (default)
 * - sendRequestFrequency: 200ms timeout before sending partial batches
(default)
 *
 * Key advantages of the batching approach:
 * - Identical API: batchManager.sendMessage() has the same signature as
sqsAsyncClient.sendMessage()
 * - Automatic optimization: No code changes needed to benefit from batching
 * - Transparent buffering: The SDK handles batching logic internally
 * - Reduced API calls: Multiple messages sent in single batch requests
 * - Lower costs: Fewer API calls result in reduced SQS charges
 * - Higher throughput: Batch operations process significantly more messages per
second
 */
private static class BatchProducer extends Thread {
 final SqsAsyncBatchManager batchManager;
 final String queueUrl;
 final int batchSize;
 final AtomicInteger producedCount;
 final AtomicBoolean stop;
 final String theMessage;

 /**
 * Creates a producer thread for batch operations.
 *
 * @param batchManager the batch manager for efficient message sending

```

```

 * @param queueUrl the URL of the target queue
 * @param batchSize the number of messages to send per batch
 * @param messageSizeByte the size of messages to generate
 * @param producedCount shared counter for tracking sent messages
 * @param stop shared flag to signal thread termination
 */
 BatchProducer(SqsAsyncBatchManager batchManager, String queueUrl, int
batchSize,
 int messageSizeByte, AtomicInteger producedCount,
 AtomicBoolean stop) {
 this.batchManager = batchManager;
 this.queueUrl = queueUrl;
 this.batchSize = batchSize;
 this.producedCount = producedCount;
 this.stop = stop;
 this.theMessage = makeRandomString(messageSizeByte);
 }

 /**
 * Continuously sends batches of messages using the high-level batching
library.
 *
 * Notice how batchManager.sendMessage() uses the exact same method
signature
 * and request builder pattern as SqsAsyncClient.sendMessage(). This
demonstrates
 * the drop-in replacement capability of the SqsAsyncBatchManager.
 *
 * The SDK automatically:
 * - Buffers individual sendMessage() calls internally
 * - Groups them into batch requests when thresholds are met
 * - Sends SendMessageBatchRequest operations to SQS
 * - Returns individual CompletableFuture responses for each message
 *
 * This transparent batching provides significant performance improvements
 * without requiring changes to application logic or error handling
patterns.
 */
 public void run() {
 try {
 while (!stop.get()) {
 // Send multiple messages using the high-level batch manager.
 // Each batchManager.sendMessage() call uses identical syntax to

```

```

 // sqsAsyncClient.sendMessage(), demonstrating API
compatibility.
 // The SDK automatically buffers these calls and sends them as
 // batch operations when batchSize (10) or
sendRequestFrequency (200ms)
 // thresholds are reached, significantly improving throughput.
 for (int i = 0; i < batchSize; i++) {
 CompletableFuture<SendMessageResponse> future =
batchManager.sendMessage(
 SendMessageRequest.builder()
 .queueUrl(queueUrl)
 .messageBody(theMessage)
 .build());

 // Handle the response asynchronously
 future.whenComplete((response, throwable) -> {
 if (throwable == null) {
 producedCount.incrementAndGet();
 } else if (!(throwable instanceof
java.util.concurrent.CancellationException) &&
 !(throwable.getMessage() != null &&
throwable.getMessage().contains("executor not accepting a task"))) {
 log.error("BatchProducer: Failed to send message",
throwable);
 }
 // Ignore CancellationException and executor shutdown
errors - expected during shutdown
 });
 }

 // Small delay to allow batching to occur
 Thread.sleep(10);
 }
} catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 log.error("BatchProducer interrupted: " + e.getMessage());
} catch (SdkException | java.util.concurrent.CompletionException e) {
 log.error("BatchProducer: " + e.getMessage());
 System.exit(1);
}
}
}

/**

```

```

 * Receives and deletes messages individually using SqsAsyncClient for baseline
 measurement.
 *
 * This consumer demonstrates traditional single-message operations without
 batching.
 * Each receiveMessage() and deleteMessage() call results in separate API
 requests,
 * providing a performance baseline for comparison with batch operations.
 *
 * The method signatures are identical to SqsAsyncBatchManager methods:
 * - receiveMessage() matches batchManager.receiveMessage()
 * - deleteMessage() matches batchManager.deleteMessage()
 *
 * This API consistency allows easy migration to the high-level batching
 library.
 */
 private static class Consumer extends Thread {
 final SqsAsyncClient sqsAsyncClient;
 final String queueUrl;
 final AtomicInteger consumedCount;
 final AtomicBoolean stop;

 /**
 * Creates a consumer thread for single-message operations.
 *
 * @param sqsAsyncClient the SQS client for receiving messages
 * @param queueUrl the URL of the source queue
 * @param consumedCount shared counter for tracking processed messages
 * @param stop shared flag to signal thread termination
 */
 Consumer(SqsAsyncClient sqsAsyncClient, String queueUrl, AtomicInteger
consumedCount,
 AtomicBoolean stop) {
 this.sqsAsyncClient = sqsAsyncClient;
 this.queueUrl = queueUrl;
 this.consumedCount = consumedCount;
 this.stop = stop;
 }

 /**
 * Continuously receives and deletes messages using traditional single-
 request operations.
 *
 * Uses SqsAsyncClient methods directly:

```

```

 * - receiveMessage(): One API call per receive operation
 * - deleteMessage(): One API call per delete operation
 *
 * This approach demonstrates the baseline performance without batching
optimization.
 * Compare these method calls with the identical signatures used in
BatchConsumer
 * to see how the high-level batching library maintains API compatibility.
 */
public void run() {
 try {
 while (!stop.get()) {
 try {
 final ReceiveMessageResponse result =
sqsAsyncClient.receiveMessage(
 ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .build()).join();

 if (!result.messages().isEmpty()) {
 final Message m = result.messages().get(0);
 // Note: deleteMessage() signature identical to
batchManager.deleteMessage()

sqsAsyncClient.deleteMessage(DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(m.receiptHandle())
 .build()).join();
 consumedCount.incrementAndGet();
 }
 } catch (SdkException | java.util.concurrent.CompletionException
e) {
 log.error(e.getMessage());
 }
 }
 } catch (SdkException | java.util.concurrent.CompletionException e) {
 // Handle both SdkException and CompletionException from async
operations.

 // If this unlikely condition occurs, stop.
 log.error("Consumer: " + e.getMessage());
 System.exit(1);
 }
}
}

```

```

/**
 * Receives and deletes messages using SqsAsyncBatchManager for automatic
optimization.
 *
 * This consumer demonstrates the AWS SDK for Java 2.x high-level batching
library
 * for message consumption. The SqsAsyncBatchManager provides two key
optimizations:
 *
 * 1. Receive optimization: Maintains an internal buffer of messages fetched in
the
 * background, so receiveMessage() calls return immediately from the buffer
 * 2. Delete batching: Automatically buffers deleteMessage() calls and sends
them
 * as DeleteMessageBatchRequest operations when thresholds are reached
 *
 * Key features:
 * - Identical API: receiveMessage() and deleteMessage() have the same
signatures
 * as SqsAsyncClient methods, making this a true drop-in replacement
 * - Background fetching: The batch manager continuously fetches messages to
keep
 * the internal buffer populated, reducing receive latency
 * - Automatic delete batching: Individual deleteMessage() calls are buffered
and
 * sent as batch operations (up to 10 per batch, 200ms frequency)
 * - Transparent optimization: No application logic changes needed to benefit
 *
 * Performance benefits:
 * - Reduced API calls through automatic batching of delete operations
 * - Lower latency for receives due to background message buffering
 * - Higher overall throughput with fewer network round trips
 */
private static class BatchConsumer extends Thread {
 final SqsAsyncBatchManager batchManager;
 final String queueUrl;
 final int batchSize;
 final AtomicInteger consumedCount;
 final AtomicBoolean stop;

 /**
 * Creates a consumer thread for batch operations.
 */

```

```

 * @param batchManager the batch manager for efficient message processing
 * @param queueUrl the URL of the source queue
 * @param batchSize the maximum number of messages to receive per batch
 * @param consumedCount shared counter for tracking processed messages
 * @param stop shared flag to signal thread termination
 */
 BatchConsumer(SqsAsyncBatchManager batchManager, String queueUrl, int
batchSize,
 AtomicInteger consumedCount, AtomicBoolean stop) {
 this.batchManager = batchManager;
 this.queueUrl = queueUrl;
 this.batchSize = batchSize;
 this.consumedCount = consumedCount;
 this.stop = stop;
 }

 /**
 * Continuously receives and deletes messages using the high-level batching
library.
 *
 * Demonstrates the key advantage of SqsAsyncBatchManager: identical method
signatures
 * with automatic optimization. Notice how:
 *
 * - batchManager.receiveMessage() uses the same syntax as
sqsAsyncClient.receiveMessage()
 * - batchManager.deleteMessage() uses the same syntax as
sqsAsyncClient.deleteMessage()
 *
 * Behind the scenes, the batch manager:
 * 1. Maintains an internal message buffer populated by background fetching
 * 2. Returns messages immediately from the buffer (reduced latency)
 * 3. Automatically batches deleteMessage() calls into
DeleteMessageBatchRequest operations
 * 4. Sends batch deletes when maxBatchSize (10) or sendRequestFrequency
(200ms) is reached
 *
 * This provides significant performance improvements with zero code changes
 * compared to traditional SqsAsyncClient usage patterns.
 */
 public void run() {
 try {
 while (!stop.get()) {
 // Receive messages using the high-level batch manager.

```

```

 // This call uses identical syntax to
sqsAsyncClient.receiveMessage()
 // but benefits from internal message buffering for improved
performance.

 final ReceiveMessageResponse result =
batchManager.receiveMessage(
 ReceiveMessageRequest.builder()
 .queueUrl(queueUrl)
 .numberOfMessages(Math.min(batchSize, 10))
 .build()).join();

 if (!result.messages().isEmpty()) {
 final List<Message> messages = result.messages();

 // Delete messages using the batch manager.
 // Each deleteMessage() call uses identical syntax to
SqsAsyncClient
 // but the SDK automatically buffers these calls and sends
them
 // as DeleteMessageBatchRequest operations for optimal
performance.

 for (Message message : messages) {
 CompletableFuture<DeleteMessageResponse> future =
batchManager.deleteMessage(
 DeleteMessageRequest.builder()
 .queueUrl(queueUrl)
 .receiptHandle(message.receiptHandle())
 .build());

 future.whenComplete((response, throwable) -> {
 if (throwable == null) {
 consumedCount.incrementAndGet();
 } else if (!(throwable instanceof
java.util.concurrent.CancellationException) &&
 !(throwable.getMessage() != null &&
throwable.getMessage().contains("executor not accepting a task")) {
 log.error("BatchConsumer: Failed to delete
message", throwable);
 }
 // Ignore CancellationException and executor
shutdown errors - expected during shutdown
 });
 }
 }
 }
}

```



```

 // Small delay to prevent tight polling
 Thread.sleep(10);
 }
} catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 log.error("BatchConsumer interrupted: " + e.getMessage());
} catch (SdkException | java.util.concurrent.CompletionException e) {
 // Handle both SdkException and CompletionException from async
operations.
 // If this unlikely condition occurs, stop.
 log.error("BatchConsumer: " + e.getMessage());
 System.exit(1);
}
}
}

/**
 * Displays real-time throughput statistics every second.
 *
 * This thread logs the current count of produced and consumed messages
 * to help you monitor the performance comparison.
 */
private static class Monitor extends Thread {
 private final AtomicInteger producedCount;
 private final AtomicInteger consumedCount;
 private final AtomicBoolean stop;

 /**
 * Creates a monitoring thread that displays throughput statistics.
 *
 * @param producedCount shared counter for messages sent
 * @param consumedCount shared counter for messages processed
 * @param stop shared flag to signal thread termination
 */
 Monitor(AtomicInteger producedCount, AtomicInteger consumedCount,
 AtomicBoolean stop) {
 this.producedCount = producedCount;
 this.consumedCount = consumedCount;
 this.stop = stop;
 }

 /**
 * Logs throughput statistics every second until stopped.

```

```
 *
 * Displays the current count of produced and consumed messages
 * to help monitor the performance comparison between batching strategies.
 */
 public void run() {
 try {
 while (!stop.get()) {
 Thread.sleep(1000);
 log.info("produced messages = " + producedCount.get()
 + ", consumed messages = " + consumedCount.get());
 }
 } catch (InterruptedException e) {
 // Allow the thread to exit.
 }
 }
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateQueue](#)
  - [DeleteMessage](#)
  - [DeleteMessageBatch](#)
  - [DeleteQueue](#)
  - [ReceiveMessage](#)
  - [SendMessage](#)
  - [SendMessageBatch](#)

## Use the Amazon SQS Java Messaging Library to work with the JMS interface

The following code example shows how to use the Amazon SQS Java Messaging Library to work with the JMS interface.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following examples work with standard Amazon SQS queues and include:

- Sending a text message.
- Receiving messages synchronously.
- Receiving messages asynchronously.
- Receiving messages using CLIENT\_ACKNOWLEDGE mode.
- Receiving messages using the UNORDERED\_ACKNOWLEDGE mode.
- Using Spring to inject dependencies.
- A utility class that provides common methods used by the other examples.

For more information on using JMS with Amazon SQS, see the [Amazon SQS Developer Guide](#).

Sending a text message.

```
/**
 * This method establishes a connection to a standard Amazon SQS queue using the
 * Amazon SQS
 * Java Messaging Library and sends text messages to it. It uses JMS (Java
 * Message Service) API
 * with automatic acknowledgment mode to ensure reliable message delivery, and
 * automatically
 * manages all messaging resources.
 *
 * @throws JMSEException If there is a problem connecting to or sending messages
 * to the queue
 */
public static void doSendTextMessage() throws JMSEException {
 // Create a connection factory.
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);
}
```

```

 // Create the connection in a try-with-resources statement so that it's
 closed automatically.
 try (SQSConnection connection = connectionFactory.createConnection()) {

 // Create the queue if needed.
 SqsJmsExampleUtils.ensureQueueExists(connection, QUEUE_NAME,
 SqsJmsExampleUtils.QUEUE_VISIBILITY_TIMEOUT);

 // Create a session that uses the JMS auto-acknowledge mode.
 Session session = connection.createSession(false,
 Session.AUTO_ACKNOWLEDGE);
 MessageProducer producer =
 session.createProducer(session.createQueue(QUEUE_NAME));

 createAndSendMessages(session, producer);
 } // The connection closes automatically. This also closes the session.
 LOGGER.info("Connection closed");
 }

 /**
 * This method reads text input from the keyboard and sends each line as a
 separate message
 * to a standard Amazon SQS queue using the Amazon SQS Java Messaging Library.
 It continues
 * to accept input until the user enters an empty line, using JMS (Java Message
 Service) API to
 * handle the message delivery.
 *
 * @param session The JMS session used to create messages
 * @param producer The JMS message producer used to send messages to the queue
 */
 private static void createAndSendMessages(Session session, MessageProducer
 producer) {
 BufferedReader inputReader = new BufferedReader(
 new InputStreamReader(System.in, Charset.defaultCharset()));

 try {
 String input;
 while (true) {
 LOGGER.info("Enter message to send (leave empty to exit): ");
 input = inputReader.readLine();
 if (input == null || input.isEmpty()) break;

 TextMessage message = session.createTextMessage(input);

```

```

 producer.send(message);
 LOGGER.info("Send message {}", message.getJMSMessageID());
 }
} catch (EOFException e) {
 // Just return on EOF
} catch (IOException e) {
 LOGGER.error("Failed reading input: {}", e.getMessage(), e);
} catch (JMSEException e) {
 LOGGER.error("Failed sending message: {}", e.getMessage(), e);
}
}

```

## Receiving messages synchronously.

```

/**
 * This method receives messages from a standard Amazon SQS queue using the
 * Amazon SQS Java
 * Messaging Library. It creates a connection to the queue using JMS (Java
 * Message Service),
 * * waits for messages to arrive, and processes them one at a time. The method
 * handles all
 * * necessary setup and cleanup of messaging resources.
 *
 * @throws JMSEException If there is a problem connecting to or receiving
 * messages from the queue
 */
public static void doReceiveMessageSync() throws JMSEException {
 // Create a connection factory.
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);

 // Create a connection.
 try (SQSConnection connection = connectionFactory.createConnection()) {

 // Create the queue if needed.
 SqsJmsExampleUtils.ensureQueueExists(connection, QUEUE_NAME,
 SqsJmsExampleUtils.QUEUE_VISIBILITY_TIMEOUT);

 // Create a session.

```

```

 Session session = connection.createSession(false,
Session.CLIENT_ACKNOWLEDGE);
 MessageConsumer consumer =
session.createConsumer(session.createQueue(QueueName));

 connection.start();

 receiveMessages(consumer);
 } // The connection closes automatically. This also closes the session.
 LOGGER.info("Connection closed");
}

/**
 * This method continuously checks for new messages from a standard Amazon SQS
queue using
 * the Amazon SQS Java Messaging Library. It waits up to 20 seconds for each
message, processes
 * it using JMS (Java Message Service), and confirms receipt. The method stops
checking for
 * messages after 20 seconds of no activity.
 *
 * @param consumer The JMS message consumer that receives messages from the
queue
 */
private static void receiveMessages(MessageConsumer consumer) {
 try {
 while (true) {
 LOGGER.info("Waiting for messages...");
 // Wait 1 minute for a message
 Message message =
consumer.receive(Duration.ofSeconds(20).toMillis());
 if (message == null) {
 LOGGER.info("Shutting down after 20 seconds of silence.");
 break;
 }
 SqsJmsExampleUtils.handleMessage(message);
 message.acknowledge();
 LOGGER.info("Acknowledged message {}", message.getJMSMessageID());
 }
 } catch (JMSException e) {
 LOGGER.error("Error receiving from SQS: {}", e.getMessage(), e);
 }
}

```

## Receiving messages asynchronously.

```

/**
 * This method sets up automatic message handling for a standard Amazon SQS
queue using the
 * Amazon SQS Java Messaging Library. It creates a listener that processes
messages as soon
 * as they arrive using JMS (Java Message Service), runs for 5 seconds, then
cleans up all
 * messaging resources.
 *
 * @throws JMSEException If there is a problem connecting to or receiving
messages from the queue
 */
public static void doReceiveMessageAsync() throws JMSEException {
 // Create a connection factory.
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);

 // Create a connection.
 try (SQSConnection connection = connectionFactory.createConnection()) {

 // Create the queue if needed.
 SqsJmsExampleUtils.ensureQueueExists(connection, QUEUE_NAME,
SqsJmsExampleUtils.QUEUE_VISIBILITY_TIMEOUT);

 // Create a session.
 Session session = connection.createSession(false,
Session.CLIENT_ACKNOWLEDGE);

 try {
 // Create a consumer for the queue.
 MessageConsumer consumer =
session.createConsumer(session.createQueue(QUEUE_NAME));
 // Provide an implementation of the MessageListener interface, which
has a single 'onMessage' method.
 // We use a lambda expression for the implementation.
 consumer.setMessageListener(message -> {
 try {

```

```

 SqsJmsExampleUtils.handleMessage(message);
 message.acknowledge();
 } catch (JMSEException e) {
 LOGGER.error("Error processing message: {}",
e.getMessage());
 }
});
// Start receiving incoming messages.
connection.start();
LOGGER.info("Waiting for messages...");
} catch (JMSEException e) {
 throw new RuntimeException(e);
}
try {
 Thread.sleep(5000);
} catch (InterruptedException e) {
 throw new RuntimeException(e);
}
} // The connection closes automatically. This also closes the session.
LOGGER.info("Connection closed");
}

```

## Receiving messages using CLIENT\_ACKNOWLEDGE mode.

```

/**
 * This method demonstrates how message acknowledgment affects message
processing in a standard
 * Amazon SQS queue using the Amazon SQS Java Messaging Library. It sends
messages to the queue,
 * then shows how JMS (Java Message Service) client acknowledgment mode handles
both explicit
 * and implicit message confirmations, including how acknowledging one message
can automatically
 * acknowledge previous messages.
 *
 * @throws JMSEException If there is a problem with the messaging operations
 */
public static void doReceiveMessagesSyncClientAcknowledge() throws JMSEException
{
 // Create a connection factory.
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),

```



```
 SqsClient.create()
);

 // Create the connection in a try-with-resources statement so that it's
 closed automatically.
 try (SQSConnection connection = connectionFactory.createConnection()) {

 // Create the queue if needed.
 SqsJmsExampleUtils.ensureQueueExists(connection, QUEUE_NAME,
 TIME_OUT_SECONDS);

 // Create a session with client acknowledge mode.
 Session session = connection.createSession(false,
 Session.CLIENT_ACKNOWLEDGE);

 // Create a producer and consumer.
 MessageProducer producer =
 session.createProducer(session.createQueue(QUEUE_NAME));
 MessageConsumer consumer =
 session.createConsumer(session.createQueue(QUEUE_NAME));

 // Open the connection.
 connection.start();

 // Send two text messages.
 sendMessage(producer, session, "Message 1");
 sendMessage(producer, session, "Message 2");

 // Receive a message and don't acknowledge it.
 receiveMessage(consumer, false);

 // Receive another message and acknowledge it.
 receiveMessage(consumer, true);

 // Wait for the visibility time out, so that unacknowledged messages
 reappear in the queue,
 LOGGER.info("Waiting for visibility timeout...");
 try {
 Thread.sleep(TIME_OUT_MILLIS);
 } catch (InterruptedException e) {
 LOGGER.error("Interrupted while waiting for visibility timeout", e);
 Thread.currentThread().interrupt();
 throw new RuntimeException("Processing interrupted", e);
 }
 }
```

```

 /* We will attempt to receive another message, but none will be
available. This is because in
 CLIENT_ACKNOWLEDGE mode, when we acknowledged the second message,
all previous messages were
 automatically acknowledged as well. Therefore, although we never
directly acknowledged the first
 message, it was implicitly acknowledged when we confirmed the second
one. */
 receiveMessage(consumer, true);
 } // The connection closes automatically. This also closes the session.
 LOGGER.info("Connection closed.");

}

/**
 * Sends a text message using the specified JMS MessageProducer and Session.
 *
 * @param producer The JMS MessageProducer used to send the message
 * @param session The JMS Session used to create the text message
 * @param messageText The text content to be sent in the message
 * @throws JMSEException If there is an error creating or sending the message
 */
private static void sendMessage(MessageProducer producer, Session session,
String messageText) throws JMSEException {
 // Create a text message and send it.
 producer.send(session.createTextMessage(messageText));
}

/**
 * Receives and processes a message from a JMS queue using the specified
consumer.
 *
 * The method waits for a message until the configured timeout period is
reached.
 *
 * If a message is received, it is logged and optionally acknowledged based on
the
 *
 * acknowledge parameter.
 *
 * @param consumer The JMS MessageConsumer used to receive messages from the
queue
 * @param acknowledge Boolean flag indicating whether to acknowledge the
message.
 *
 * If true, the message will be acknowledged after processing

```

```

 * @throws JMSEException If there is an error receiving, processing, or
 acknowledging the message
 */
 private static void receiveMessage(MessageConsumer consumer, boolean
 acknowledge) throws JMSEException {
 // Receive a message.
 Message message = consumer.receive(TIME_OUT_MILLIS);

 if (message == null) {
 LOGGER.info("Queue is empty!");
 } else {
 // Since this queue has only text messages, cast the message object and
 print the text.
 LOGGER.info("Received: {} Acknowledged: {}", ((TextMessage)
 message).getText(), acknowledge);

 // Acknowledge the message if asked.
 if (acknowledge) message.acknowledge();
 }
 }
}

```

## Receiving messages using the UNORDERED\_ACKNOWLEDGE mode.

```

/**
 * Demonstrates message acknowledgment behavior in UNORDERED_ACKNOWLEDGE mode
 with Amazon SQS JMS.
 * In this mode, each message must be explicitly acknowledged regardless of
 receive order.
 * Unacknowledged messages return to the queue after the visibility timeout
 expires,
 * unlike CLIENT_ACKNOWLEDGE mode where acknowledging one message acknowledges
 all previous messages.
 *
 * @throws JMSEException If a JMS-related error occurs during message
 operations
 */
public static void doReceiveMessagesUnorderedAcknowledge() throws JMSEException {
 // Create a connection factory.
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);
}

```

```
// Create the connection in a try-with-resources statement so that it's
closed automatically.
try(SQSConnection connection = connectionFactory.createConnection()) {

 // Create the queue if needed.
 SqsJmsExampleUtils.ensureQueueExists(connection, QUEUE_NAME,
TIME_OUT_SECONDS);

 // Create a session with unordered acknowledge mode.
 Session session = connection.createSession(false,
SQSSession.UNORDERED_ACKNOWLEDGE);

 // Create the producer and consumer.
 MessageProducer producer =
session.createProducer(session.createQueue(QUEUE_NAME));
 MessageConsumer consumer =
session.createConsumer(session.createQueue(QUEUE_NAME));

 // Open a connection.
 connection.start();

 // Send two text messages.
 sendMessage(producer, session, "Message 1");
 sendMessage(producer, session, "Message 2");

 // Receive a message and don't acknowledge it.
 receiveMessage(consumer, false);

 // Receive another message and acknowledge it.
 receiveMessage(consumer, true);

 // Wait for the visibility time out, so that unacknowledged messages
reappear in the queue.
 LOGGER.info("Waiting for visibility timeout...");
 try {
 Thread.sleep(TIME_OUT_MILLIS);
 } catch (InterruptedException e) {
 LOGGER.error("Interrupted while waiting for visibility timeout", e);
 Thread.currentThread().interrupt();
 throw new RuntimeException("Processing interrupted", e);
 }
}
```

```

 /* We will attempt to receive another message, and we'll get the first
 message again. This occurs
 because in UNORDERED_ACKNOWLEDGE mode, each message requires its own
 separate acknowledgment.
 Since we only acknowledged the second message, the first message
 remains in the queue for
 redelivery. */
 receiveMessage(consumer, true);

 LOGGER.info("Connection closed.");
 } // The connection closes automatically. This also closes the session.
}

/**
 * Sends a text message to an Amazon SQS queue using JMS.
 *
 * @param producer The JMS MessageProducer for the queue
 * @param session The JMS Session for message creation
 * @param messageText The message content
 * @throws JMSEException If message creation or sending fails
 */
private static void sendMessage(MessageProducer producer, Session session,
String messageText) throws JMSEException {
 // Create a text message and send it.
 producer.send(session.createTextMessage(messageText));
}

/**
 * Synchronously receives a message from an Amazon SQS queue using the JMS API
 * with an acknowledgment parameter.
 *
 * @param consumer The JMS MessageConsumer for the queue
 * @param acknowledge If true, acknowledges the message after receipt
 * @throws JMSEException If message reception or acknowledgment fails
 */
private static void receiveMessage(MessageConsumer consumer, boolean
acknowledge) throws JMSEException {
 // Receive a message.
 Message message = consumer.receive(TIME_OUT_MILLIS);

 if (message == null) {
 LOGGER.info("Queue is empty!");
 } else {
 // Since this queue has only text messages, cast the message object and
 print the text.

```

```

 LOGGER.info("Received: {} Acknowledged: {}", ((TextMessage)
message).getText(), acknowledge);

 // Acknowledge the message if asked.
 if (acknowledge) message.acknowledge();
 }
}

```

## Using Spring to inject dependencies.

```

package com.example.sqs.jms.spring;

import com.amazon.sqs.javamessaging.SQSConnection;
import com.example.sqs.jms.SqsJmsExampleUtils;
import jakarta.jms.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.NoSuchBeanDefinitionException;
import org.springframework.context.support.FileSystemXmlApplicationContext;

import java.io.File;
import java.net.URL;
import java.util.concurrent.TimeUnit;

/**
 * Demonstrates how to send and receive messages using the Amazon SQS Java Messaging
 * Library
 * with Spring Framework integration. This example connects to a standard Amazon SQS
 * message
 * queue using Spring's dependency injection to configure the connection and
 * messaging components.
 * The application uses the JMS (Java Message Service) API to handle message
 * operations.
 */
public class SpringExample {
 private static final Integer POLLING_SECONDS = 15;
 private static final String SPRING_XML_CONFIG_FILE =
"SpringExampleConfiguration.xml.txt";
 private static final Logger LOGGER =
LoggerFactory.getLogger(SpringExample.class);

 /**

```

```

 * Demonstrates sending and receiving messages through a standard Amazon SQS
message queue
 * using Spring Framework configuration. This method loads connection settings
from an XML file,
 * establishes a messaging session using the Amazon SQS Java Messaging Library,
and processes
 * messages using JMS (Java Message Service) operations. If the queue doesn't
exist, it will
 * be created automatically.
 *
 * @param args Command line arguments (not used)
 */
public static void main(String[] args) {

 URL resource =
SpringExample.class.getClassLoader().getResource(RESOURCE_XML_CONFIG_FILE);
 File springFile = new File(resource.getFile());
 if (!springFile.exists() || !springFile.canRead()) {
 LOGGER.error("File " + RESOURCE_XML_CONFIG_FILE + " doesn't exist or isn't
readable.");
 System.exit(1);
 }

 try (FileSystemXmlApplicationContext context =
 new FileSystemXmlApplicationContext("file://" +
springFile.getAbsolutePath())) {

 Connection connection;
 try {
 connection = context.getBean(Connection.class);
 } catch (NoSuchBeanDefinitionException e) {
 LOGGER.error("Can't find the JMS connection to use: " +
e.getMessage(), e);
 System.exit(2);
 return;
 }

 String queueName;
 try {
 queueName = context.getBean("queueName", String.class);
 } catch (NoSuchBeanDefinitionException e) {
 LOGGER.error("Can't find the name of the queue to use: " +
e.getMessage(), e);
 System.exit(3);

```

```

 return;
 }
 try {
 if (connection instanceof SQSConnection) {
 SqsJmsExampleUtils.ensureQueueExists((SQSConnection) connection,
queueName, SqsJmsExampleUtils.QUEUE_VISIBILITY_TIMEOUT);
 }
 // Create the JMS session.
 Session session = connection.createSession(false,
Session.CLIENT_ACKNOWLEDGE);

 SqsJmsExampleUtils.sendTextMessage(session, queueName);
 MessageConsumer consumer =
session.createConsumer(session.createQueue(queueName));

 receiveMessages(consumer);
 } catch (JMSEException e) {
 LOGGER.error(e.getMessage(), e);
 throw new RuntimeException(e);
 }
} // Spring context autocloses. Managed Spring beans that implement
AutoClosable, such as the
// 'connection' bean, are also closed.
LOGGER.info("Context closed");
}

/**
 * Continuously checks for and processes messages from a standard Amazon SQS
message queue
 * using the Amazon SQS Java Messaging Library underlying the JMS API. This
method waits for incoming messages,
 * processes them when they arrive, and acknowledges their receipt using JMS
(Java Message
 * Service) operations. The method will stop checking for messages after 15
seconds of
 * inactivity.
 *
 * @param consumer The JMS message consumer used to receive messages from the
queue
 */
private static void receiveMessages(MessageConsumer consumer) {
 try {
 while (true) {
 LOGGER.info("Waiting for messages...");

```



```

 // Wait 15 seconds for a message.
 Message message =
consumer.receive(TimeUnit.SECONDS.toMillis(POLLING_SECONDS));
 if (message == null) {
 LOGGER.info("Shutting down after {} seconds of silence.",
POLLING_SECONDS);
 break;
 }
 SqsJmsExampleUtils.handleMessage(message);
 message.acknowledge();
 LOGGER.info("Message acknowledged.");
 }
} catch (JMSEException e) {
 LOGGER.error("Error receiving from SQS.", e);
}
}
}

```

## Spring bean definitions.

```

<?xml version="1.0" encoding="UTF-8"?>
<beans
 xmlns="http://www.springframework.org/schema/beans"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.org/schema/
beans/spring-beans-3.0.xsd
 ">
 <!-- Define the AWS Region -->
 <bean id="region" class="software.amazon.awssdk.regions.Region" factory-
method="of">
 <constructor-arg value="us-east-1"/>
 </bean>

 <bean id="credentialsProviderBean"
class="software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider"
 factory-method="create"/>

 <bean id="clientBuilder" class="software.amazon.awssdk.services.sqs.SqsClient"
factory-method="builder"/>

```

```

 <bean id="regionSetClientBuilder" factory-bean="clientBuilder" factory-
method="region">
 <constructor-arg ref="region"/>
 </bean>

 <!-- Configure the Builder with Credentials Provider -->
 <bean id="sqsClient" factory-bean="regionSetClientBuilder" factory-
method="credentialsProvider">
 <constructor-arg ref="credentialsProviderBean"/>
 </bean>

 <bean id="providerConfiguration"
class="com.amazon.sqs.javamessaging.ProviderConfiguration">
 <property name="numberOfMessagesToPrefetch" value="5"/>
 </bean>

 <bean id="connectionFactory"
class="com.amazon.sqs.javamessaging.SQSConnectionFactory">
 <constructor-arg ref="providerConfiguration"/>
 <constructor-arg ref="clientBuilder"/>
 </bean>

 <bean id="connection"
 factory-bean="connectionFactory"
 factory-method="createConnection"
 init-method="start"
 destroy-method="close"/>

 <bean id="queueName" class="java.lang.String">
 <constructor-arg value="SQSJMSClientExampleQueue"/>
 </bean>
</beans>

```

A utility class that provides common methods used by the other examples.

```

package com.example.sqs.jms;

import com.amazon.sqs.javamessaging.AmazonSQSMessagingClientWrapper;
import com.amazon.sqs.javamessaging.ProviderConfiguration;
import com.amazon.sqs.javamessaging.SQSConnection;
import com.amazon.sqs.javamessaging.SQSConnectionFactory;

```

```

import jakarta.jms.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.core.exception.SdkException;
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.CreateQueueRequest;
import software.amazon.awssdk.services.sqs.model.QueueAttributeName;

import java.time.Duration;
import java.util.Base64;
import java.util.Map;

/**
 * This utility class provides helper methods for working with Amazon Simple Queue
 * Service (Amazon SQS)
 * through the Java Message Service (JMS) interface. It contains common operations
 * for managing message
 * queues and handling message delivery.
 */
public class SqsJmsExampleUtils {
 private static final Logger LOGGER =
 LoggerFactory.getLogger(SqsJmsExampleUtils.class);
 public static final Long QUEUE_VISIBILITY_TIMEOUT = 5L;

 /**
 * This method verifies that a message queue exists and creates it if necessary.
 The method checks for
 * an existing queue first to optimize performance.
 *
 * @param connection The active connection to the messaging service
 * @param queueName The name of the queue to verify or create
 * @param visibilityTimeout The duration in seconds that messages will be hidden
 after being received
 * @throws JMSEException If there is an error accessing or creating the queue
 */
 public static void ensureQueueExists(SQSConnection connection, String queueName,
 Long visibilityTimeout) throws JMSEException {
 AmazonSQSMessagingClientWrapper client =
 connection.getWrappedAmazonSQSClient();

 /* In most cases, you can do this with just a 'createQueue' call, but
 'getQueueUrl'
 (called by 'queueExists') is a faster operation for the common case where the
 queue

```

```

 already exists. Also, many users and roles have permission to call
 'getQueueUrl'
 but don't have permission to call 'createQueue'.
 */
 if(!client.queueExists(queueName)) {
 CreateQueueRequest createQueueRequest = CreateQueueRequest.builder()
 .queueName(queueName)
 .attributes(Map.of(QueueAttributeName.VISIBILITY_TIMEOUT,
String.valueOf(visibilityTimeout)))
 .build();
 client.createQueue(createQueueRequest);
 }
}

/**
 * This method sends a simple text message to a specified message queue. It
handles all necessary
 * setup for the message delivery process.
 *
 * @param session The active messaging session used to create and send the
message
 * @param queueName The name of the queue where the message will be sent
 */
public static void sendTextMessage(Session session, String queueName) {
 // Rest of implementation...

 try {
 MessageProducer producer =
session.createProducer(session.createQueue(queueName));
 Message message = session.createTextMessage("Hello world!");
 producer.send(message);
 } catch (JMSEException e) {
 LOGGER.error("Error receiving from SQS", e);
 }
}

/**
 * This method processes incoming messages and logs their content based on the
message type.
 * It supports text messages, binary data, and Java objects.
 *
 * @param message The message to be processed and logged
 * @throws JMSEException If there is an error reading the message content
 */

```

```

 public static void handleMessage(Message message) throws JMSEException {
 // Rest of implementation...
 LOGGER.info("Got message {}", message.getJMSMessageID());
 LOGGER.info("Content: ");
 if(message instanceof TextMessage txtMessage) {
 LOGGER.info("\t{}", txtMessage.getText());
 } else if(message instanceof BytesMessage byteMessage){
 // Assume the length fits in an int - SQS only supports sizes up to 256k
so that
 // should be true
 byte[] bytes = new byte[(int)byteMessage.getBodyLength()];
 byteMessage.readBytes(bytes);
 LOGGER.info("\t{}", Base64.getEncoder().encodeToString(bytes));
 } else if(message instanceof ObjectMessage) {
 ObjectMessage objMessage = (ObjectMessage) message;
 LOGGER.info("\t{}", objMessage.getObject());
 }
 }

 /**
 * This method sets up automatic message processing for a specified queue. It
 creates a listener
 * that will receive and handle incoming messages without blocking the main
 program.
 *
 * @param session The active messaging session
 * @param queueName The name of the queue to monitor
 * @param connection The active connection to the messaging service
 */
 public static void receiveMessagesAsync(Session session, String queueName,
 Connection connection) {
 // Rest of implementation...
 try {
 // Create a consumer for the queue.
 MessageConsumer consumer =
session.createConsumer(session.createQueue(queueName));
 // Provide an implementation of the MessageListener interface, which has
a single 'onMessage' method.
 // We use a lambda expression for the implementation.
 consumer.setMessageListener(message -> {
 try {
 SqsJmsExampleUtils.handleMessage(message);
 message.acknowledge();
 } catch (JMSEException e) {

```

```

 LOGGER.error("Error processing message: {}", e.getMessage());
 }
});
// Start receiving incoming messages.
connection.start();
} catch (JMSEException e) {
 throw new RuntimeException(e);
}
try {
 Thread.sleep(2000);
} catch (InterruptedException e) {
 throw new RuntimeException(e);
}
}

/**
 * This method performs cleanup operations after message processing is complete.
It receives
 * any messages in the specified queue, removes the message queue and closes all
 * active connections to prevent resource leaks.
 *
 * @param queueName The name of the queue to be removed
 * @param visibilityTimeout The duration in seconds that messages are hidden
after being received
 * @throws JMSEException If there is an error during the cleanup process
 */
public static void cleanUpExample(String queueName, Long visibilityTimeout)
throws JMSEException {
 LOGGER.info("Performing cleanup.");

 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);

 try (SQSConnection connection = connectionFactory.createConnection()) {
 ensureQueueExists(connection, queueName, visibilityTimeout);
 Session session = connection.createSession(false,
Session.AUTO_ACKNOWLEDGE);

 receiveMessagesAsync(session, queueName, connection);
 }
}

```

```

 SqsClient sqsClient =
connection.getWrappedAmazonSQSClient().getAmazonSQSClient();
 try {
 String queueUrl = sqsClient.getQueueUrl(b ->
b.queueName(queueName)).queueUrl();
 sqsClient.deleteQueue(b -> b.queueUrl(queueUrl));
 LOGGER.info("Queue deleted: {}", queueUrl);
 } catch (SdkException e) {
 LOGGER.error("Error during SQS operations: ", e);
 }
 }
 LOGGER.info("Clean up: Connection closed");
}

/**
 * This method creates a background task that sends multiple messages to a
specified queue
 * after waiting for a set time period. The task operates independently to
ensure efficient
 * message processing without interrupting other operations.
 *
 * @param queueName The name of the queue where messages will be sent
 * @param secondsToWait The number of seconds to wait before sending messages
 * @param numMessages The number of messages to send
 * @param visibilityTimeout The duration in seconds that messages remain hidden
after being received
 * @return A task that can be executed to send the messages
 */
public static Runnable sendAMessageAsync(String queueName, Long secondsToWait,
Integer numMessages, Long visibilityTimeout) {
 return () -> {
 try {
 Thread.sleep(Duration.ofSeconds(secondsToWait).toMillis());
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 throw new RuntimeException(e);
 }
 try {
 SQSConnectionFactory connectionFactory = new SQSConnectionFactory(
 new ProviderConfiguration(),
 SqsClient.create()
);
 try (SQSConnection connection =
connectionFactory.createConnection()) {

```

```

 ensureQueueExists(connection, queueName, visibilityTimeout);
 Session session = connection.createSession(false,
Session.CLIENT_ACKNOWLEDGE);
 for (int i = 1; i <= numMessages; i++) {
 MessageProducer producer =
session.createProducer(session.createQueue(queueName));
 producer.send(session.createTextMessage("Hello World " + i +
"!"));
 }
 }
} catch (JMSEException e) {
 LOGGER.error(e.getMessage(), e);
 throw new RuntimeException(e);
}
};
}
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateQueue](#)
  - [DeleteQueue](#)

## Work with queue tags

The following code example shows how to perform tagging operation with Amazon SQS.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

The following example creates tags for a queue, lists tags, and removes a tag.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

```



```

import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.ListQueueTagsResponse;
import software.amazon.awssdk.services.sqs.model.QueueDoesNotExistException;
import software.amazon.awssdk.services.sqs.model.SqsException;

import java.util.Map;
import java.util.UUID;

/**
 * Before running this Java V2 code example, set up your development environment,
 * including your credentials. For more
 * information, see the <a href="https://docs.aws.amazon.com/sdk-for-java/latest/
developer-guide/get-started.html">AWS
 * SDK for Java Developer Guide.
 */
public class TagExamples {
 static final SqsClient sqsClient = SqsClient.create();
 static final String queueName = "TagExamples-queue-" +
 UUID.randomUUID().toString().replace("-", "").substring(0, 20);
 private static final Logger LOGGER = LoggerFactory.getLogger(TagExamples.class);

 public static void main(String[] args) {
 final String queueUrl;
 try {
 queueUrl = sqsClient.createQueue(b ->
b.queueName(queueName)).queueUrl();
 LOGGER.info("Queue created. The URL is: {}", queueUrl);
 } catch (RuntimeException e) {
 LOGGER.error("Program ending because queue was not created.");
 throw new RuntimeException(e);
 }
 try {
 addTags(queueUrl);
 listTags(queueUrl);
 removeTags(queueUrl);
 } catch (RuntimeException e) {
 LOGGER.error("Program ending because of an error in a method.");
 } finally {
 try {
 sqsClient.deleteQueue(b -> b.queueUrl(queueUrl));
 LOGGER.info("Queue successfully deleted. Program ending.");
 sqsClient.close();
 } catch (RuntimeException e) {
 LOGGER.error("Program ending.");
 }
 }
 }
}

```

```

 } finally {
 sqsClient.close();
 }
 }
}

/** This method demonstrates how to use a Java Map to tag a queue.
 * @param queueUrl The URL of the queue to tag.
 */
public static void addTags(String queueUrl) {
 // Build a map of the tags.
 final Map<String, String> tagsToAdd = Map.of(
 "Team", "Development",
 "Priority", "Beta",
 "Accounting ID", "456def");

 try {
 // Add tags to the queue using a Consumer<TagQueueRequest.Builder>
parameter.
 sqsClient.tagQueue(b -> b
 .queueUrl(queueUrl)
 .tags(tagsToAdd)
);
 } catch (QueueDoesNotExistException e) {
 LOGGER.error("Queue does not exist: {}", e.getMessage(), e);
 throw new RuntimeException(e);
 }
}

/** This method demonstrates how to view the tags for a queue.
 * @param queueUrl The URL of the queue whose tags you want to list.
 */
public static void listTags(String queueUrl) {
 ListQueueTagsResponse response;
 try {
 // Call the listQueueTags method with a
Consumer<ListQueueTagsRequest.Builder> parameter that creates a
ListQueueTagsRequest.
 response = sqsClient.listQueueTags(b -> b
 .queueUrl(queueUrl));
 } catch (SqsException e) {
 LOGGER.error("Exception thrown: {}", e.getMessage(), e);
 throw new RuntimeException(e);
 }
}

```

```

 // Log the tags.
 response.tags()
 .forEach((k, v) ->
 LOGGER.info("Key: {} -> Value: {}", k, v));
 }

 /**
 * This method demonstrates how to remove tags from a queue.
 * @param queueUrl The URL of the queue whose tags you want to remove.
 */
 public static void removeTags(String queueUrl) {
 try {
 // Call the untagQueue method with a Consumer<UntagQueueRequest.Builder>
 // parameter.
 sqsClient.untagQueue(b -> b
 .queueUrl(queueUrl)
 .tagKeys("Accounting ID") // Remove a single tag.
);
 } catch (SqsException e) {
 LOGGER.error("Exception thrown: {}", e.getMessage(), e);
 throw new RuntimeException(e);
 }
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [ListQueueTags](#)
  - [TagQueue](#)
  - [UntagQueue](#)

## Serverless examples

### Invoke a Lambda function from an Amazon SQS trigger

The following code example shows how to implement a Lambda function that receives an event triggered by receiving messages from an SQS queue. The function retrieves the messages from the event parameter and logs the content of each message.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

### Consuming an SQS event with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SQSEvent;
import com.amazonaws.services.lambda.runtime.events.SQSEvent.SQSMessage;

public class Function implements RequestHandler<SQSEvent, Void> {
 @Override
 public Void handleRequest(SQSEvent sqsEvent, Context context) {
 for (SQSMessage msg : sqsEvent.getRecords()) {
 processMessage(msg, context);
 }
 context.getLogger().log("done");
 return null;
 }

 private void processMessage(SQSMessage msg, Context context) {
 try {
 context.getLogger().log("Processed message " + msg.getBody());

 // TODO: Do interesting work based on the new message

 } catch (Exception e) {
 context.getLogger().log("An error occurred");
 throw e;
 }
 }
}
```

## Reporting batch item failures for Lambda functions with an Amazon SQS trigger

The following code example shows how to implement partial batch response for Lambda functions that receive events from an SQS queue. The function reports the batch item failures in the response, signaling to Lambda to retry those messages later.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [Serverless examples](#) repository.

### Reporting SQS batch item failures with Lambda using Java.

```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SQSEvent;
import com.amazonaws.services.lambda.runtime.events.SQSBatchResponse;

import java.util.ArrayList;
import java.util.List;

public class ProcessSQSMessageBatch implements RequestHandler<SQSEvent,
 SQSBatchResponse> {
 @Override
 public SQSBatchResponse handleRequest(SQSEvent sqsEvent, Context context) {
 List<SQSBatchResponse.BatchItemFailure> batchItemFailures = new
 ArrayList<SQSBatchResponse.BatchItemFailure>();

 for (SQSEvent.SQSMessage message : sqsEvent.getRecords()) {
 try {
 //process your message
 } catch (Exception e) {
 //Add failed message identifier to the batchItemFailures list
 batchItemFailures.add(new
 SQSBatchResponse.BatchItemFailure(message.getMessageId()));
 }
 }
 }
}
```

```
 return new SQSBatchResponse(batchItemFailures);
 }
}
```

## Step Functions examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Step Functions.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)
- [Scenarios](#)

## Get started

### Hello Step Functions

The following code example shows how to get started using Step Functions.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

## Java version of Hello.

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sfn.SfnClient;
import software.amazon.awssdk.services.sfn.model.ListStateMachinesResponse;
import software.amazon.awssdk.services.sfn.model.SfnException;
import software.amazon.awssdk.services.sfn.model.StateMachineListItem;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListStateMachines {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 SfnClient sfnClient = SfnClient.builder()
 .region(region)
 .build();

 listMachines(sfnClient);
 sfnClient.close();
 }

 public static void listMachines(SfnClient sfnClient) {
 try {
 ListStateMachinesResponse response = sfnClient.listStateMachines();
 List<StateMachineListItem> machines = response.stateMachines();
 for (StateMachineListItem machine : machines) {
 System.out.println("The name of the state machine is: " +
machine.name());
 System.out.println("The ARN value is : " +
machine.stateMachineArn());
 }

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}
```

```
}
```

- For API details, see [ListStateMachines](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create an activity.
- Create a state machine from an Amazon States Language definition that contains the previously created activity as a step.
- Run the state machine and respond to the activity with user input.
- Get the final status and output after the run completes, then clean up resources.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * You can obtain the JSON file to create a state machine in the following
 * GitHub location.
 * <p>
 * https://github.com/awsdocs/aws-doc-sdk-examples/tree/main/resources/sample_files
 * <p>
 * To run this code example, place the chat_sfn_state_machine.json file into
 * your project's resources folder.
 * <p>
 * Also, set up your development environment, including your credentials.
 * <p>
 * For information, see this documentation topic:
 * <p>
```



```

* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
* <p>
* This Java code example performs the following tasks:
* <p>
* 1. Creates an activity.
* 2. Creates a state machine.
* 3. Describes the state machine.
* 4. Starts execution of the state machine and interacts with it.
* 5. Describes the execution.
* 6. Delete the activity.
* 7. Deletes the state machine.
*/
public class StepFunctionsScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) throws Exception {
 final String usage = ""

 Usage:
 <roleARN> <activityName> <stateMachineName>

 Where:
 roleName - The name of the IAM role to create for this state
machine.
 activityName - The name of an activity to create.
 stateMachineName - The name of the state machine to create.
 jsonFile - The location of the chat_sfn_state_machine.json file. You
can located it in resources/sample_files.
 """;

 if (args.length != 4) {
 System.out.println(usage);
 System.exit(1);
 }

 String roleName = args[0];
 String activityName = args[1];
 String stateMachineName = args[2];
 String jsonFile = args[3];
 String polJSON = ""
 {
 "Version": "2012-10-17",
 "Statement": [
 {

```

```

 "Sid": "",
 "Effect": "Allow",
 "Principal": {
 "Service": "states.amazonaws.com"
 },
 "Action": "sts:AssumeRole"
 }
]
}
"";

```

```

Scanner sc = new Scanner(System.in);
boolean action = false;

```

```

Region region = Region.US_EAST_1;
SfnClient sfnClient = SfnClient.builder()
 .region(region)
 .build();

```

```

Region regionGl = Region.AWS_GLOBAL;
IamClient iam = IamClient.builder()
 .region(regionGl)
 .build();

```

```

System.out.println(DASHES);
System.out.println("Welcome to the AWS Step Functions example scenario.");
System.out.println(DASHES);

```

```

System.out.println(DASHES);
System.out.println("1. Create an activity.");
String activityArn = createActivity(sfnClient, activityName);
System.out.println("The ARN of the activity is " + activityArn);
System.out.println(DASHES);

```

```

// Read the file using FileInputStream
FileInputStream inputStream = new FileInputStream(jsonFile);
ObjectMapper mapper = new ObjectMapper();
JsonNode jsonNode = mapper.readValue(inputStream, JsonNode.class);
String jsonString = mapper.writeValueAsString(jsonNode);

```

```

// Modify the Resource node.
ObjectMapper objectMapper = new ObjectMapper();
JsonNode root = objectMapper.readTree(jsonString);

```

```

 ((ObjectNode) root.path("States").path("GetInput")).put("Resource",
activityArn);

 // Convert the modified Java object back to a JSON string.
 String stateDefinition = objectMapper.writeValueAsString(root);
 System.out.println(stateDefinition);

 System.out.println(DASHES);
 System.out.println("2. Create a state machine.");
 String roleARN = createIAMRole(iam, roleName, polJSON);
 String stateMachineArn = createMachine(sfnClient, roleARN, stateMachineName,
stateDefinition);
 System.out.println("The ARN of the state machine is " + stateMachineArn);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Describe the state machine.");
 describeStateMachine(sfnClient, stateMachineArn);
 System.out.println("What should ChatSFN call you?");
 String userName = sc.nextLine();
 System.out.println("Hello " + userName);
 System.out.println(DASHES);

 System.out.println(DASHES);
 // The JSON to pass to the StartExecution call.
 String executionJson = "{ \"name\" : \"" + userName + "\" }";
 System.out.println(executionJson);
 System.out.println("4. Start execution of the state machine and interact
with it.");
 String runArn = startWorkflow(sfnClient, stateMachineArn, executionJson);
 System.out.println("The ARN of the state machine execution is " + runArn);
 List<String> myList;
 while (!action) {
 myList = getActivityTask(sfnClient, activityArn);
 System.out.println("ChatSFN: " + myList.get(1));
 System.out.println(userName + " please specify a value.");
 String myAction = sc.nextLine();
 if (myAction.compareTo("done") == 0)
 action = true;

 System.out.println("You have selected " + myAction);
 String taskJson = "{ \"action\" : \"" + myAction + "\" }";
 System.out.println(taskJson);
 sendTaskSuccess(sfnClient, myList.get(0), taskJson);

```

```

 }
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Describe the execution.");
 describeExe(sfnClient, runArn);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Delete the activity.");
 deleteActivity(sfnClient, activityArn);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. Delete the state machines.");
 deleteMachine(sfnClient, stateMachineArn);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("The AWS Step Functions example scenario is complete.");
 System.out.println(DASHES);
}

public static String createIAMRole(IamClient iam, String rolename, String
polJSON) {
 try {
 CreateRoleRequest request = CreateRoleRequest.builder()
 .roleName(rolename)
 .assumeRolePolicyDocument(polJSON)
 .description("Created using the AWS SDK for Java")
 .build();

 CreateRoleResponse response = iam.createRole(request);
 return response.role().arn();

 } catch (IamException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static void describeExe(SfnClient sfnClient, String executionArn) {
 try {

```

```

 DescribeExecutionRequest executionRequest =
DescribeExecutionRequest.builder()
 .executionArn(executionArn)
 .build();

 String status = "";
 boolean hasSucceeded = false;
 while (!hasSucceeded) {
 DescribeExecutionResponse response =
sfnClient.describeExecution(executionRequest);
 status = response.statusAsString();
 if (status.compareTo("RUNNING") == 0) {
 System.out.println("The state machine is still running, let's
wait for it to finish.");
 Thread.sleep(2000);
 } else if (status.compareTo("SUCCEEDED") == 0) {
 System.out.println("The Step Function workflow has succeeded");
 hasSucceeded = true;
 } else {
 System.out.println("The Status is neither running or
succeeded");
 }
 }
 System.out.println("The Status is " + status);

 } catch (SfnException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}

public static void sendTaskSuccess(SfnClient sfnClient, String token, String
json) {
 try {
 SendTaskSuccessRequest successRequest = SendTaskSuccessRequest.builder()
 .taskToken(token)
 .output(json)
 .build();

 sfnClient.sendTaskSuccess(successRequest);

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

```
 }
}

public static List<String> getActivityTask(SfnClient sfnClient, String actArn) {
 List<String> myList = new ArrayList<>();
 GetActivityTaskRequest getActivityTaskRequest =
GetActivityTaskRequest.builder()
 .activityArn(actArn)
 .build();

 GetActivityTaskResponse response =
sfnClient.getActivityTask(getActivityTaskRequest);
 myList.add(response.taskToken());
 myList.add(response.input());
 return myList;
}

public static void deleteActivity(SfnClient sfnClient, String actArn) {
 try {
 DeleteActivityRequest activityRequest = DeleteActivityRequest.builder()
 .activityArn(actArn)
 .build();

 sfnClient.deleteActivity(activityRequest);
 System.out.println("You have deleted " + actArn);

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

public static void describeStateMachine(SfnClient sfnClient, String
stateMachineArn) {
 try {
 DescribeStateMachineRequest stateMachineRequest =
DescribeStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 DescribeStateMachineResponse response =
sfnClient.describeStateMachine(stateMachineRequest);
 System.out.println("The name of the State machine is " +
response.name());
 }
}
```

```

 System.out.println("The status of the State machine is " +
response.status());
 System.out.println("The ARN value of the State machine is " +
response.stateMachineArn());
 System.out.println("The role ARN value is " + response.roleArn());

 } catch (SfnException e) {
 System.err.println(e.getMessage());
 }
}

public static void deleteMachine(SfnClient sfnClient, String stateMachineArn) {
 try {
 DeleteStateMachineRequest deleteStateMachineRequest =
DeleteStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 sfnClient.deleteStateMachine(deleteStateMachineRequest);
 DescribeStateMachineRequest describeStateMachine =
DescribeStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 while (true) {
 DescribeStateMachineResponse response =
sfnClient.describeStateMachine(describeStateMachine);
 System.out.println("The state machine is not deleted yet. The status
is " + response.status());
 Thread.sleep(3000);
 }

 } catch (SfnException | InterruptedException e) {
 System.err.println(e.getMessage());
 }
 System.out.println(stateMachineArn + " was successfully deleted.");
}

public static String startWorkflow(SfnClient sfnClient, String stateMachineArn,
String jsonEx) {
 UUID uuid = UUID.randomUUID();
 String uuidValue = uuid.toString();
 try {
 StartExecutionRequest executionRequest = StartExecutionRequest.builder()

```

```
 .input(jsonEx)
 .stateMachineArn(stateMachineArn)
 .name(uuidValue)
 .build();

 StartExecutionResponse response =
sfnClient.startExecution(executionRequest);
 return response.executionArn();

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createMachine(SfnClient sfnClient, String roleARN, String
stateMachineName, String json) {
 try {
 CreateStateMachineRequest machineRequest =
CreateStateMachineRequest.builder()
 .definition(json)
 .name(stateMachineName)
 .roleArn(roleARN)
 .type(StateMachineType.STANDARD)
 .build();

 CreateStateMachineResponse response =
sfnClient.createStateMachine(machineRequest);
 return response.stateMachineArn();

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}

public static String createActivity(SfnClient sfnClient, String activityName) {
 try {
 CreateActivityRequest activityRequest = CreateActivityRequest.builder()
 .name(activityName)
 .build();
```



```
 CreateActivityResponse response =
 sfmClient.createActivity(activityRequest);
 return response.activityArn();

 } catch (SfmException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.

- [CreateActivity](#)
- [CreateStateMachine](#)
- [DeleteActivity](#)
- [DeleteStateMachine](#)
- [DescribeExecution](#)
- [DescribeStateMachine](#)
- [GetActivityTask](#)
- [ListActivities](#)
- [ListStateMachines](#)
- [SendTaskSuccess](#)
- [StartExecution](#)
- [StopExecution](#)

## Actions

### CreateActivity

The following code example shows how to use `CreateActivity`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createActivity(SfnClient sfnClient, String activityName) {
 try {
 CreateActivityRequest activityRequest = CreateActivityRequest.builder()
 .name(activityName)
 .build();

 CreateActivityResponse response =
sfnClient.createActivity(activityRequest);
 return response.activityArn();

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateActivity](#) in *AWS SDK for Java 2.x API Reference*.

## CreateStateMachine

The following code example shows how to use CreateStateMachine.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createMachine(SfnClient sfnClient, String roleARN, String
stateMachineName, String json) {
 try {
 CreateStateMachineRequest machineRequest =
 CreateStateMachineRequest.builder()
 .definition(json)
 .name(stateMachineName)
 .roleArn(roleARN)
 .type(StateMachineType.STANDARD)
 .build();

 CreateStateMachineResponse response =
 sfnClient.createStateMachine(machineRequest);
 return response.stateMachineArn();

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateStateMachine](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteActivity

The following code example shows how to use DeleteActivity.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteActivity(SfnClient sfnClient, String actArn) {
 try {
 DeleteActivityRequest activityRequest = DeleteActivityRequest.builder()
 .activityArn(actArn)
```

```
 .build();

 sfnClient.deleteActivity(activityRequest);
 System.out.println("You have deleted " + actArn);

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [DeleteActivity](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteStateMachine

The following code example shows how to use DeleteStateMachine.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void deleteMachine(SfnClient sfnClient, String stateMachineArn) {
 try {
 DeleteStateMachineRequest deleteStateMachineRequest =
DeleteStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 sfnClient.deleteStateMachine(deleteStateMachineRequest);
 DescribeStateMachineRequest describeStateMachine =
DescribeStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 while (true) {
 DescribeStateMachineResponse response =
sfnClient.describeStateMachine(describeStateMachine);
```

```

 System.out.println("The state machine is not deleted yet. The status
is " + response.status());
 Thread.sleep(3000);
 }

 } catch (SfnException | InterruptedException e) {
 System.err.println(e.getMessage());
 }
 System.out.println(stateMachineArn + " was successfully deleted.");
}

```

- For API details, see [DeleteStateMachine](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeExecution

The following code example shows how to use `DescribeExecution`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void describeExe(SfnClient sfnClient, String executionArn) {
 try {
 DescribeExecutionRequest executionRequest =
DescribeExecutionRequest.builder()
 .executionArn(executionArn)
 .build();

 String status = "";
 boolean hasSucceeded = false;
 while (!hasSucceeded) {
 DescribeExecutionResponse response =
sfnClient.describeExecution(executionRequest);
 status = response.statusAsString();
 if (status.compareTo("RUNNING") == 0) {
 System.out.println("The state machine is still running, let's
wait for it to finish.");
 }
 }
 }
}

```

```

 Thread.sleep(2000);
 } else if (status.compareTo("SUCCEEDED") == 0) {
 System.out.println("The Step Function workflow has succeeded");
 hasSucceeded = true;
 } else {
 System.out.println("The Status is neither running or
succeeded");
 }
}
System.out.println("The Status is " + status);

} catch (SfnException | InterruptedException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
}

```

- For API details, see [DescribeExecution](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeStateMachine

The following code example shows how to use DescribeStateMachine.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void describeStateMachine(SfnClient sfnClient, String
stateMachineArn) {
 try {
 DescribeStateMachineRequest stateMachineRequest =
DescribeStateMachineRequest.builder()
 .stateMachineArn(stateMachineArn)
 .build();

 DescribeStateMachineResponse response =
sfnClient.describeStateMachine(stateMachineRequest);
 }
}

```

```

 System.out.println("The name of the State machine is " +
response.name());
 System.out.println("The status of the State machine is " +
response.status());
 System.out.println("The ARN value of the State machine is " +
response.stateMachineArn());
 System.out.println("The role ARN value is " + response.roleArn());

 } catch (SfnException e) {
 System.err.println(e.getMessage());
 }
}

```

- For API details, see [DescribeStateMachine](#) in *AWS SDK for Java 2.x API Reference*.

## GetActivityTask

The following code example shows how to use `GetActivityTask`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static List<String> getActivityTask(SfnClient sfnClient, String actArn) {
 List<String> myList = new ArrayList<>();
 GetActivityTaskRequest getActivityTaskRequest =
GetActivityTaskRequest.builder()
 .activityArn(actArn)
 .build();

 GetActivityTaskResponse response =
sfnClient.getActivityTask(getActivityTaskRequest);
 myList.add(response.taskToken());
 myList.add(response.input());
 return myList;
}

```

- For API details, see [GetActivityTask](#) in *AWS SDK for Java 2.x API Reference*.

## ListActivities

The following code example shows how to use ListActivities.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sfn.SfnClient;
import software.amazon.awssdk.services.sfn.model.ListActivitiesRequest;
import software.amazon.awssdk.services.sfn.model.ListActivitiesResponse;
import software.amazon.awssdk.services.sfn.model.SfnException;
import software.amazon.awssdk.services.sfn.model.ActivityListItem;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListActivities {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 SfnClient sfnClient = SfnClient.builder()
 .region(region)
 .build();

 listAllActivites(sfnClient);
 sfnClient.close();
 }
}
```



```
}

public static void listAllActivites(SfnClient sfnClient) {
 try {
 ListActivitiesRequest activitiesRequest =
ListActivitiesRequest.builder()
 .maxResults(10)
 .build();

 ListActivitiesResponse response =
sfnClient.listActivities(activitiesRequest);
 List<ActivityListItem> items = response.activities();
 for (ActivityListItem item : items) {
 System.out.println("The activity ARN is " + item.activityArn());
 System.out.println("The activity name is " + item.name());
 }

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
}
```

- For API details, see [ListActivities](#) in *AWS SDK for Java 2.x API Reference*.

## ListExecutions

The following code example shows how to use ListExecutions.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void getExeHistory(SfnClient sfnClient, String exeARN) {
 try {
```

```

 GetExecutionHistoryRequest historyRequest =
 GetExecutionHistoryRequest.builder()
 .executionArn(exeARN)
 .maxResults(10)
 .build();

 GetExecutionHistoryResponse historyResponse =
 sfncClient.getExecutionHistory(historyRequest);
 List<HistoryEvent> events = historyResponse.events();
 for (HistoryEvent event : events) {
 System.out.println("The event type is " + event.type().toString());
 }

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}

```

- For API details, see [ListExecutions](#) in *AWS SDK for Java 2.x API Reference*.

## ListStateMachines

The following code example shows how to use `ListStateMachines`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sfn.SfnClient;
import software.amazon.awssdk.services.sfn.model.ListStateMachinesResponse;
import software.amazon.awssdk.services.sfn.model.SfnException;
import software.amazon.awssdk.services.sfn.model.StateMachineListItem;
import java.util.List;

```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class ListStateMachines {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 SfnClient sfnClient = SfnClient.builder()
 .region(region)
 .build();

 listMachines(sfnClient);
 sfnClient.close();
 }

 public static void listMachines(SfnClient sfnClient) {
 try {
 ListStateMachinesResponse response = sfnClient.listStateMachines();
 List<StateMachineListItem> machines = response.stateMachines();
 for (StateMachineListItem machine : machines) {
 System.out.println("The name of the state machine is: " +
machine.name());
 System.out.println("The ARN value is : " +
machine.stateMachineArn());
 }

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [ListStateMachines](#) in *AWS SDK for Java 2.x API Reference*.

## SendTaskSuccess

The following code example shows how to use SendTaskSuccess.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void sendTaskSuccess(SfnClient sfnClient, String token, String
json) {
 try {
 SendTaskSuccessRequest successRequest = SendTaskSuccessRequest.builder()
 .taskToken(token)
 .output(json)
 .build();

 sfnClient.sendTaskSuccess(successRequest);

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
}
```

- For API details, see [SendTaskSuccess](#) in *AWS SDK for Java 2.x API Reference*.

## StartExecution

The following code example shows how to use StartExecution.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String startWorkflow(SfnClient sfnClient, String stateMachineArn,
String jsonEx) {
 UUID uuid = UUID.randomUUID();
 String uuidValue = uuid.toString();
 try {
 StartExecutionRequest executionRequest = StartExecutionRequest.builder()
 .input(jsonEx)
 .stateMachineArn(stateMachineArn)
 .name(uuidValue)
 .build();

 StartExecutionResponse response =
sfnClient.startExecution(executionRequest);
 return response.executionArn();

 } catch (SfnException e) {
 System.err.println(e.awsErrorDetails().errorMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [StartExecution](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Use Step Functions to invoke Lambda functions

The following code example shows how to create an AWS Step Functions state machine that invokes AWS Lambda functions in sequence.

#### SDK for Java 2.x

Shows how to create an AWS serverless workflow by using AWS Step Functions and the AWS SDK for Java 2.x. Each workflow step is implemented using an AWS Lambda function.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- DynamoDB

- Lambda
- Amazon SES
- Step Functions

## AWS STS examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with AWS STS.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

## Actions

### AssumeRole

The following code example shows how to use AssumeRole.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.sts.StsClient;
import software.amazon.awssdk.services.sts.model.AssumeRoleRequest;
import software.amazon.awssdk.services.sts.model.StsException;
import software.amazon.awssdk.services.sts.model.AssumeRoleResponse;
import software.amazon.awssdk.services.sts.model.Credentials;
import java.time.Instant;
```

```

import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

/**
 * To make this code example work, create a Role that you want to assume.
 * Then define a Trust Relationship in the AWS Console. You can use this as an
 * example:
 *
 * {
 * "Version": "2012-10-17",
 * "Statement": [
 * {
 * "Effect": "Allow",
 * "Principal": {
 * "AWS": "<Specify the ARN of your IAM user you are using in this code example>"
 * },
 * "Action": "sts:AssumeRole"
 * }
 *]
 * }
 *
 * For more information, see "Editing the Trust Relationship for an Existing
 * Role" in the AWS Directory Service guide.
 *
 * Also, set up your development environment, including your credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class AssumeRole {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <roleArn> <roleSessionName>\s

 Where:
 roleArn - The Amazon Resource Name (ARN) of the role to assume
 (for example, arn:aws:iam::000008047983:role/s3role).\s
 roleSessionName - An identifier for the assumed role session
 (for example, mysession).\s

```

```

 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String roleArn = args[0];
 String roleSessionName = args[1];
 Region region = Region.US_EAST_1;
 StsClient stsClient = StsClient.builder()
 .region(region)
 .build();

 assumeGivenRole(stsClient, roleArn, roleSessionName);
 stsClient.close();
}

public static void assumeGivenRole(StsClient stsClient, String roleArn, String
roleSessionName) {
 try {
 AssumeRoleRequest roleRequest = AssumeRoleRequest.builder()
 .roleArn(roleArn)
 .roleSessionName(roleSessionName)
 .build();

 AssumeRoleResponse roleResponse = stsClient.assumeRole(roleRequest);
 Credentials myCreds = roleResponse.credentials();

 // Display the time when the temp creds expire.
 Instant exTime = myCreds.expiration();
 String tokenInfo = myCreds.sessionToken();

 // Convert the Instant to readable date.
 DateTimeFormatter formatter =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT)
 .withLocale(Locale.US)
 .withZone(ZoneId.systemDefault());

 formatter.format(exTime);
 System.out.println("The token " + tokenInfo + " expires on " + exTime);

 } catch (StsException e) {
 System.err.println(e.getMessage());
 }
}

```



```
 System.exit(1);
 }
}
```

- For API details, see [AssumeRole](#) in *AWS SDK for Java 2.x API Reference*.

## Support examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Support.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Support

The following code example shows how to get started using Support.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.support.SupportClient;
import software.amazon.awssdk.services.support.model.Category;
import software.amazon.awssdk.services.support.model.DescribeServicesRequest;
import software.amazon.awssdk.services.support.model.DescribeServicesResponse;
import software.amazon.awssdk.services.support.model.Service;
import software.amazon.awssdk.services.support.model.SupportException;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * In addition, you must have the AWS Business Support Plan to use the AWS
 * Support Java API. For more information, see:
 *
 * https://aws.amazon.com/premiumsupport/plans/
 *
 * This Java example performs the following task:
 *
 * 1. Gets and displays available services.
 *
 * NOTE: To see multiple operations, see SupportScenario.
 */

public class HelloSupport {
 public static void main(String[] args) {
 Region region = Region.US_WEST_2;
 SupportClient supportClient = SupportClient.builder()
 .region(region)
 .build();

 System.out.println("***** Step 1. Get and display available services.");
 displayServices(supportClient);
 }

 // Return a List that contains a Service name and Category name.
```

```
public static void displayServices(SupportClient supportClient) {
 try {
 DescribeServicesRequest servicesRequest =
 DescribeServicesRequest.builder()
 .language("en")
 .build();

 DescribeServicesResponse response =
 supportClient.describeServices(servicesRequest);
 List<Service> services = response.services();

 System.out.println("Get the first 10 services");
 int index = 1;
 for (Service service : services) {
 if (index == 11)
 break;

 System.out.println("The Service name is: " + service.name());

 // Display the Categories for this service.
 List<Category> categories = service.categories();
 for (Category cat : categories) {
 System.out.println("The category name is: " + cat.name());
 }
 index++;
 }

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeServices](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Get and display available services and severity levels for cases.
- Create a support case using a selected service, category, and severity level.
- Get and display a list of open cases for the current day.
- Add an attachment set and a communication to the new case.
- Describe the new attachment and communication for the case.
- Resolve the case.
- Get and display a list of resolved cases for the current day.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Run various Support operations.

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.support.SupportClient;
import software.amazon.awssdk.services.support.model.AddAttachmentsToSetResponse;
import software.amazon.awssdk.services.support.model.AddCommunicationToCaseRequest;
import software.amazon.awssdk.services.support.model.AddCommunicationToCaseResponse;
import software.amazon.awssdk.services.support.model.Attachment;
import software.amazon.awssdk.services.support.model.AttachmentDetails;
import software.amazon.awssdk.services.support.model.CaseDetails;
import software.amazon.awssdk.services.support.model.Category;
import software.amazon.awssdk.services.support.model.Communication;
import software.amazon.awssdk.services.support.model.CreateCaseRequest;
import software.amazon.awssdk.services.support.model.CreateCaseResponse;
import software.amazon.awssdk.services.support.model.DescribeAttachmentRequest;
import software.amazon.awssdk.services.support.model.DescribeAttachmentResponse;
import software.amazon.awssdk.services.support.model.DescribeCasesRequest;
import software.amazon.awssdk.services.support.model.DescribeCasesResponse;
import software.amazon.awssdk.services.support.model.DescribeCommunicationsRequest;
import software.amazon.awssdk.services.support.model.DescribeCommunicationsResponse;
import software.amazon.awssdk.services.support.model.DescribeServicesRequest;
import software.amazon.awssdk.services.support.model.DescribeServicesResponse;
```

```
import software.amazon.awssdk.services.support.model.DescribeSeverityLevelsRequest;
import software.amazon.awssdk.services.support.model.DescribeSeverityLevelsResponse;
import software.amazon.awssdk.services.support.model.ResolveCaseRequest;
import software.amazon.awssdk.services.support.model.ResolveCaseResponse;
import software.amazon.awssdk.services.support.model.Service;
import software.amazon.awssdk.services.support.model.SeverityLevel;
import software.amazon.awssdk.services.support.model.SupportException;
import software.amazon.awssdk.services.support.model.AddAttachmentsToSetRequest;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java (v2) code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 *
 * In addition, you must have the AWS Business Support Plan to use the AWS
 * Support Java API. For more information, see:
 *
 * https://aws.amazon.com/premiumsupport/plans/
 *
 * This Java example performs the following tasks:
 *
 * 1. Gets and displays available services.
 * 2. Gets and displays severity levels.
 * 3. Creates a support case by using the selected service, category, and
 * severity level.
 * 4. Gets a list of open cases for the current day.
 * 5. Creates an attachment set with a generated file.
 * 6. Adds a communication with the attachment to the support case.
 * 7. Lists the communications of the support case.
 * 8. Describes the attachment set included with the communication.
 * 9. Resolves the support case.
 * 10. Gets a list of resolved cases for the current day.
 */
```

```

public class SupportScenario {

 public static final String DASHES = new String(new char[80]).replace("\0", "-");

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <fileAttachment>Where:
 fileAttachment - The file can be a simple saved .txt file to use
as an email attachment.\s
 """;

 // if (args.length != 1) {
 // System.out.println(usage);
 // System.exit(1);
 // }

 String fileAttachment = "C:\\\\AWS\\test.txt" ; //args[0];
 Region region = Region.US_WEST_2;
 SupportClient supportClient = SupportClient.builder()
 .region(region)
 .build();

 System.out.println(DASHES);
 System.out.println("***** Welcome to the AWS Support case example
scenario.");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("1. Get and display available services.");
 List<String> sevCatList = displayServices(supportClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("2. Get and display Support severity levels.");
 String sevLevel = displaySevLevels(supportClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("3. Create a support case using the selected service,
category, and severity level.");
 String caseId = createSupportCase(supportClient, sevCatList, sevLevel);
 if (caseId.compareTo("") == 0) {

```

```
 System.out.println("A support case was not successfully created!");
 System.exit(1);
 } else
 System.out.println("Support case " + caseId + " was successfully
created!");
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("4. Get open support cases.");
 getOpenCase(supportClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("5. Create an attachment set with a generated file to add
to the case.");
 String attachmentSetId = addAttachment(supportClient, fileAttachment);
 System.out.println("The Attachment Set id value is" + attachmentSetId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("6. Add communication with the attachment to the support
case.");
 addAttachSupportCase(supportClient, caseId, attachmentSetId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("7. List the communications of the support case.");
 String attachId = listCommunications(supportClient, caseId);
 System.out.println("The Attachment id value is" + attachId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("8. Describe the attachment set included with the
communication.");
 describeAttachment(supportClient, attachId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("9. Resolve the support case.");
 resolveSupportCase(supportClient, caseId);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("10. Get a list of resolved cases for the current day.");
```

```

 getResolvedCase(supportClient);
 System.out.println(DASHES);

 System.out.println(DASHES);
 System.out.println("***** This Scenario has successfully completed");
 System.out.println(DASHES);
 }

 public static void getResolvedCase(SupportClient supportClient) {
 try {
 // Specify the start and end time.
 Instant now = Instant.now();
 java.time.LocalDate.now();
 Instant yesterday = now.minus(1, ChronoUnit.DAYS);

 DescribeCasesRequest describeCasesRequest =
DescribeCasesRequest.builder()
 .maxResults(30)
 .afterTime(yesterday.toString())
 .beforeTime(now.toString())
 .includeResolvedCases(true)
 .build();

 DescribeCasesResponse response =
supportClient.describeCases(describeCasesRequest);
 List<CaseDetails> cases = response.cases();
 for (CaseDetails sinCase : cases) {
 if (sinCase.status().compareTo("resolved") == 0)
 System.out.println("The case status is " + sinCase.status());
 }

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 }

 public static void resolveSupportCase(SupportClient supportClient, String
caseId) {
 try {
 ResolveCaseRequest caseRequest = ResolveCaseRequest.builder()
 .caseId(caseId)
 .build();

```



```

 ResolveCaseResponse response = supportClient.resolveCase(caseRequest);
 System.out.println("The status of case " + caseId + " is " +
response.finalCaseStatus());

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static void describeAttachment(SupportClient supportClient, String
attachId) {
 try {
 DescribeAttachmentRequest attachmentRequest =
DescribeAttachmentRequest.builder()
 .attachmentId(attachId)
 .build();

 DescribeAttachmentResponse response =
supportClient.describeAttachment(attachmentRequest);
 System.out.println("The name of the file is " +
response.attachment().fileName());

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static String listCommunications(SupportClient supportClient, String
caseId) {
 try {
 String attachId = null;
 DescribeCommunicationsRequest communicationsRequest =
DescribeCommunicationsRequest.builder()
 .caseId(caseId)
 .maxResults(10)
 .build();

 DescribeCommunicationsResponse response =
supportClient.describeCommunications(communicationsRequest);
 List<Communication> communications = response.communications();
 for (Communication comm : communications) {
 System.out.println("the body is: " + comm.body());

```

```

 // Get the attachment id value.
 List<AttachmentDetails> attachments = comm.attachmentSet();
 for (AttachmentDetails detail : attachments) {
 attachId = detail.attachmentId();
 }
 }
 return attachId;

} catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
return "";
}

public static void addAttachSupportCase(SupportClient supportClient, String
caseId, String attachmentSetId) {
 try {
 AddCommunicationToCaseRequest caseRequest =
AddCommunicationToCaseRequest.builder()
 .caseId(caseId)
 .attachmentSetId(attachmentSetId)
 .communicationBody("Please refer to attachment for details.")
 .build();

 AddCommunicationToCaseResponse response =
supportClient.addCommunicationToCase(caseRequest);
 if (response.result())
 System.out.println("You have successfully added a communication to
an AWS Support case");
 else
 System.out.println("There was an error adding the communication to
an AWS Support case");

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static String addAttachment(SupportClient supportClient, String
fileAttachment) {
 try {

```

```

 File myFile = new File(fileAttachment);
 InputStream sourceStream = new FileInputStream(myFile);
 SdkBytes sourceBytes = SdkBytes.fromInputStream(sourceStream);

 Attachment attachment = Attachment.builder()
 .fileName(myFile.getName())
 .data(sourceBytes)
 .build();

 AddAttachmentsToSetRequest setRequest =
AddAttachmentsToSetRequest.builder()
 .attachments(attachment)
 .build();

 AddAttachmentsToSetResponse response =
supportClient.addAttachmentsToSet(setRequest);
 return response.attachmentSetId();

 } catch (SupportException | FileNotFoundException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}

public static void getOpenCase(SupportClient supportClient) {
 try {
 // Specify the start and end time.
 Instant now = Instant.now();
 java.time.LocalDate.now();
 Instant yesterday = now.minus(1, ChronoUnit.DAYS);

 DescribeCasesRequest describeCasesRequest =
DescribeCasesRequest.builder()
 .maxResults(20)
 .afterTime(yesterday.toString())
 .beforeTime(now.toString())
 .build();

 DescribeCasesResponse response =
supportClient.describeCases(describeCasesRequest);
 List<CaseDetails> cases = response.cases();
 for (CaseDetails sinCase : cases) {
 System.out.println("The case status is " + sinCase.status());
 }
 }
}

```

```

 System.out.println("The case Id is " + sinCase.caseId());
 System.out.println("The case subject is " + sinCase.subject());
 }

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}

public static String createSupportCase(SupportClient supportClient, List<String>
sevCatList, String sevLevel) {
 try {
 String serviceCode = sevCatList.get(0);
 String caseCat = sevCatList.get(1);
 CreateCaseRequest caseRequest = CreateCaseRequest.builder()
 .categoryCode(caseCat.toLowerCase())
 .serviceCode(serviceCode.toLowerCase())
 .severityCode(sevLevel.toLowerCase())
 .communicationBody("Test issue with " +
serviceCode.toLowerCase())
 .subject("Test case, please ignore")
 .language("en")
 .issueType("technical")
 .build();

 CreateCaseResponse response = supportClient.createCase(caseRequest);
 return response.caseId();

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}

public static String displaySevLevels(SupportClient supportClient) {
 try {
 DescribeSeverityLevelsRequest severityLevelsRequest =
DescribeSeverityLevelsRequest.builder()
 .language("en")
 .build();
 }

```

```

 DescribeSeverityLevelsResponse response =
supportClient.describeSeverityLevels(severityLevelsRequest);
 List<SeverityLevel> severityLevels = response.severityLevels();
 String levelName = null;
 for (SeverityLevel sevLevel : severityLevels) {
 System.out.println("The severity level name is: " +
sevLevel.name());
 if (sevLevel.name().compareTo("High") == 0)
 levelName = sevLevel.name();
 }
 return levelName;

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}

// Return a List that contains a Service name and Category name.
public static List<String> displayServices(SupportClient supportClient) {
 try {
 DescribeServicesRequest servicesRequest =
DescribeServicesRequest.builder()
 .language("en")
 .build();

 DescribeServicesResponse response =
supportClient.describeServices(servicesRequest);
 String serviceCode = null;
 String catName = null;
 List<String> sevCatList = new ArrayList<>();
 List<Service> services = response.services();

 System.out.println("Get the first 10 services");
 int index = 1;
 for (Service service : services) {
 if (index == 11)
 break;

 System.out.println("The Service name is: " + service.name());
 if (service.name().compareTo("Account") == 0)
 serviceCode = service.code();
 }
 }
}

```

```
 // Get the Categories for this service.
 List<Category> categories = service.categories();
 for (Category cat : categories) {
 System.out.println("The category name is: " + cat.name());
 if (cat.name().compareTo("Security") == 0)
 catName = cat.name();
 }
 index++;
 }

 // Push the two values to the list.
 sevCatList.add(serviceCode);
 sevCatList.add(catName);
 return sevCatList;

} catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
return null;
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [AddAttachmentsToSet](#)
  - [AddCommunicationToCase](#)
  - [CreateCase](#)
  - [DescribeAttachment](#)
  - [DescribeCases](#)
  - [DescribeCommunications](#)
  - [DescribeServices](#)
  - [DescribeSeverityLevels](#)
  - [ResolveCase](#)

# Actions

## AddAttachmentsToSet

The following code example shows how to use `AddAttachmentsToSet`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String addAttachment(SupportClient supportClient, String
fileAttachment) {
 try {
 File myFile = new File(fileAttachment);
 InputStream sourceStream = new FileInputStream(myFile);
 SdkBytes sourceBytes = SdkBytes.fromInputStream(sourceStream);

 Attachment attachment = Attachment.builder()
 .fileName(myFile.getName())
 .data(sourceBytes)
 .build();

 AddAttachmentsToSetRequest setRequest =
 AddAttachmentsToSetRequest.builder()
 .attachments(attachment)
 .build();

 AddAttachmentsToSetResponse response =
 supportClient.addAttachmentsToSet(setRequest);
 return response.attachmentSetId();

 } catch (SupportException | FileNotFoundException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [AddAttachmentsToSet](#) in *AWS SDK for Java 2.x API Reference*.

## AddCommunicationToCase

The following code example shows how to use `AddCommunicationToCase`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void addAttachSupportCase(SupportClient supportClient, String
caseId, String attachmentSetId) {
 try {
 AddCommunicationToCaseRequest caseRequest =
AddCommunicationToCaseRequest.builder()
 .caseId(caseId)
 .attachmentSetId(attachmentSetId)
 .communicationBody("Please refer to attachment for details.")
 .build();

 AddCommunicationToCaseResponse response =
supportClient.addCommunicationToCase(caseRequest);
 if (response.result())
 System.out.println("You have successfully added a communication to
an AWS Support case");
 else
 System.out.println("There was an error adding the communication to
an AWS Support case");

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [AddCommunicationToCase](#) in *AWS SDK for Java 2.x API Reference*.



## CreateCase

The following code example shows how to use CreateCase.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String createSupportCase(SupportClient supportClient, List<String>
sevCatList, String sevLevel) {
 try {
 String serviceCode = sevCatList.get(0);
 String caseCat = sevCatList.get(1);
 CreateCaseRequest caseRequest = CreateCaseRequest.builder()
 .categoryCode(caseCat.toLowerCase())
 .serviceCode(serviceCode.toLowerCase())
 .severityCode(sevLevel.toLowerCase())
 .communicationBody("Test issue with " +
serviceCode.toLowerCase())
 .subject("Test case, please ignore")
 .language("en")
 .issueType("technical")
 .build();

 CreateCaseResponse response = supportClient.createCase(caseRequest);
 return response.caseId();

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [CreateCase](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeAttachment

The following code example shows how to use DescribeAttachment.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void describeAttachment(SupportClient supportClient, String
attachId) {
 try {
 DescribeAttachmentRequest attachmentRequest =
DescribeAttachmentRequest.builder()
 .attachmentId(attachId)
 .build();

 DescribeAttachmentResponse response =
supportClient.describeAttachment(attachmentRequest);
 System.out.println("The name of the file is " +
response.attachment().fileName());

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeAttachment](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeCases

The following code example shows how to use DescribeCases.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static void getOpenCase(SupportClient supportClient) {
 try {
 // Specify the start and end time.
 Instant now = Instant.now();
 java.time.LocalDate.now();
 Instant yesterday = now.minus(1, ChronoUnit.DAYS);

 DescribeCasesRequest describeCasesRequest =
 DescribeCasesRequest.builder()
 .maxResults(20)
 .afterTime(yesterday.toString())
 .beforeTime(now.toString())
 .build();

 DescribeCasesResponse response =
 supportClient.describeCases(describeCasesRequest);
 List<CaseDetails> cases = response.cases();
 for (CaseDetails sinCase : cases) {
 System.out.println("The case status is " + sinCase.status());
 System.out.println("The case Id is " + sinCase.caseId());
 System.out.println("The case subject is " + sinCase.subject());
 }

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [DescribeCases](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeCommunications

The following code example shows how to use DescribeCommunications.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public static String listCommunications(SupportClient supportClient, String
caseId) {
 try {
 String attachId = null;
 DescribeCommunicationsRequest communicationsRequest =
DescribeCommunicationsRequest.builder()
 .caseId(caseId)
 .maxResults(10)
 .build();

 DescribeCommunicationsResponse response =
supportClient.describeCommunications(communicationsRequest);
 List<Communication> communications = response.communications();
 for (Communication comm : communications) {
 System.out.println("the body is: " + comm.body());

 // Get the attachment id value.
 List<AttachmentDetails> attachments = comm.attachmentSet();
 for (AttachmentDetails detail : attachments) {
 attachId = detail.attachmentId();
 }
 }
 return attachId;

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
 return "";
}
```

- For API details, see [DescribeCommunications](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeServices

The following code example shows how to use DescribeServices.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
// Return a List that contains a Service name and Category name.
public static List<String> displayServices(SupportClient supportClient) {
 try {
 DescribeServicesRequest servicesRequest =
DescribeServicesRequest.builder()
 .language("en")
 .build();

 DescribeServicesResponse response =
supportClient.describeServices(servicesRequest);
 String serviceCode = null;
 String catName = null;
 List<String> sevCatList = new ArrayList<>();
 List<Service> services = response.services();

 System.out.println("Get the first 10 services");
 int index = 1;
 for (Service service : services) {
 if (index == 11)
 break;

 System.out.println("The Service name is: " + service.name());
 if (service.name().compareTo("Account") == 0)
 serviceCode = service.code();
 }
 }
}
```

```

 // Get the Categories for this service.
 List<Category> categories = service.categories();
 for (Category cat : categories) {
 System.out.println("The category name is: " + cat.name());
 if (cat.name().compareTo("Security") == 0)
 catName = cat.name();
 }
 index++;
 }

 // Push the two values to the list.
 sevCatList.add(serviceCode);
 sevCatList.add(catName);
 return sevCatList;

} catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
return null;
}

```

- For API details, see [DescribeServices](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeSeverityLevels

The following code example shows how to use `DescribeSeverityLevels`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static String displaySevLevels(SupportClient supportClient) {
 try {
 DescribeSeverityLevelsRequest severityLevelsRequest =
 DescribeSeverityLevelsRequest.builder()

```

```

 .language("en")
 .build();

 DescribeSeverityLevelsResponse response =
supportClient.describeSeverityLevels(severityLevelsRequest);
 List<SeverityLevel> severityLevels = response.severityLevels();
 String levelName = null;
 for (SeverityLevel sevLevel : severityLevels) {
 System.out.println("The severity level name is: " +
sevLevel.name());
 if (sevLevel.name().compareTo("High") == 0)
 levelName = sevLevel.name();
 }
 return levelName;

} catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
}
return "";
}

```

- For API details, see [DescribeSeverityLevels](#) in *AWS SDK for Java 2.x API Reference*.

## ResolveCase

The following code example shows how to use `ResolveCase`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public static void resolveSupportCase(SupportClient supportClient, String
caseId) {
 try {
 ResolveCaseRequest caseRequest = ResolveCaseRequest.builder()

```

```
 .caseId(caseId)
 .build();

 ResolveCaseResponse response = supportClient.resolveCase(caseRequest);
 System.out.println("The status of case " + caseId + " is " +
response.finalCaseStatus());

 } catch (SupportException e) {
 System.out.println(e.getLocalizedMessage());
 System.exit(1);
 }
}
```

- For API details, see [ResolveCase](#) in *AWS SDK for Java 2.x API Reference*.

## Systems Manager examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Systems Manager.

*Basics* are code examples that show you how to perform the essential operations within a service.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Get started](#)
- [Basics](#)
- [Actions](#)

## Get started

### Hello Systems Manager

The following code example shows how to get started using Systems Manager.



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ssm.SsmClient;
import software.amazon.awssdk.services.ssm.model.DocumentFilter;
import software.amazon.awssdk.services.ssm.model.ListDocumentsRequest;
import software.amazon.awssdk.services.ssm.model.ListDocumentsResponse;

public class HelloSSM {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <awsAccount>

 Where:
 awsAccount - Your AWS Account number.
 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String awsAccount = args[0] ;
 Region region = Region.US_EAST_1;
 SsmClient ssmClient = SsmClient.builder()
 .region(region)
 .build();

 listDocuments(ssmClient, awsAccount);
 }

 /*
```

```
This code automatically fetches the next set of results using the `nextToken`
and
stops once the desired maxResults (20 in this case) have been reached.
*/
public static void listDocuments(SsmClient ssmClient, String awsAccount) {
 String nextToken = null;
 int totalDocumentsReturned = 0;
 int maxResults = 20;
 do {
 ListDocumentsRequest request = ListDocumentsRequest.builder()
 .documentFilterList(
 DocumentFilter.builder()
 .key("Owner")
 .value(awsAccount)
 .build()
)
 .maxResults(maxResults)
 .nextToken(nextToken)
 .build();

 ListDocumentsResponse response = ssmClient.listDocuments(request);
 response.documentIdentifiers().forEach(identifier ->
System.out.println("Document Name: " + identifier.name()));
 nextToken = response.nextToken();
 totalDocumentsReturned += response.documentIdentifiers().size();
 } while (nextToken != null && totalDocumentsReturned < maxResults);
}
}
```

- For API details, see [ListDocuments](#) in *AWS SDK for Java 2.x API Reference*.

## Basics

### Learn the basics

The following code example shows how to:

- Create a maintenance window.
- Modify the maintenance window schedule.
- Create a document.

- Send a command to a specified EC2 instance.
- Create an OpsItem.
- Update and resolve the OpsItem.
- Delete the maintenance window, OpsItem, and document.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.services.ssm.model.DocumentAlreadyExistsException;
import software.amazon.awssdk.services.ssm.model.SsmException;

import java.util.Map;
import java.util.Scanner;
public class SSMScenario {
 public static final String DASHES = new String(new char[80]).replace("\0", "-");
 private static final String ROLES_STACK = "SsmStack3`1";

 public static void main(String[] args) {
 String usage = ""
 Usage:
 <title> <source> <category> <severity>

 Where:
 title - The title of the parameter (default is Disk Space Alert).
 source - The source of the parameter (default is EC2).
 category - The category of the parameter. Valid values are
'Availability', 'Cost', 'Performance', 'Recovery', 'Security' (default is
Performance).
 severity - The severity of the parameter. Severity should be a
number from 1 to 4 (default is 2).
 """;

 Scanner scanner = new Scanner(System.in);
 SSMActions actions = new SSMActions();
 String documentName;
```

```

 String windowName;

 System.out.println("Use AWS CloudFormation to create the EC2 instance that
is required for this scenario.");
 CloudFormationHelper.deployCloudFormationStack(ROLES_STACK);
 Map<String, String> stackOutputs =
CloudFormationHelper.getStackOutputsAsync(ROLES_STACK).join();
 String instanceId = stackOutputs.get("InstanceId");
 System.out.println("The Instance ID: " + instanceId + " was created.");
 String title = "Disk Space Alert" ;
 String source = "EC2" ;
 String category = "Availability" ;
 String severity = "2" ;

 System.out.println(DASHES);
 System.out.println("""
 Welcome to the AWS Systems Manager SDK Basics scenario.
 This Java program demonstrates how to interact with AWS Systems
Manager using the AWS SDK for Java (v2).
 AWS Systems Manager is the operations hub for your AWS applications
and resources and a secure end-to-end management solution.
 The program's primary functionalities include creating a maintenance
window, creating a document, sending a command to a document,
 listing documents, listing commands, creating an OpsItem, modifying
an OpsItem, and deleting AWS SSM resources.
 Upon completion of the program, all AWS resources are cleaned up.
 Let's get started...

 """);
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println("1. Create an SSM maintenance window.");
 System.out.println("Please enter the maintenance window name (default is
ssm-maintenance-window):");
 String win = scanner.nextLine();
 windowName = win.isEmpty() ? "ssm-maintenance-window" : win;
 String winId = null;
 try {
 winId = actions.createMaintenanceWindow(windowName);
 waitForInputToContinue(scanner);
 System.out.println("The maintenance window ID is: " + winId);
 } catch (DocumentAlreadyExistsException e) {

```

```
 System.err.println("The SSM maintenance window already exists.
Retrieving existing window ID...");
 String existingWinId = actions.createMaintenanceWindow(windowName);
 System.out.println("Existing window ID: " + existingWinId);
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println("2. Modify the maintenance window by changing the
schedule");
 waitForInputToContinue(scanner);
 try {
 actions.updateSSMMaintenanceWindow(winId, windowName);
 waitForInputToContinue(scanner);
 System.out.println("The SSM maintenance window was successfully
updated");
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println("3. Create an SSM document that defines the actions that
Systems Manager performs on your managed nodes.");
 System.out.println("Please enter the document name (default is
ssmdocument):");
 String doc = scanner.nextLine();
 documentName = doc.isEmpty() ? "ssmdocument" : doc;
 try {
 actions.createSSMDoc(documentName);
 waitForInputToContinue(scanner);
 System.out.println("The SSM document was successfully created");
 } catch (DocumentAlreadyExistsException e) {
 System.err.println("The SSM document already exists. Moving on");
```

```
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println("4. Now we are going to run a command on an EC2
instance");
 waitForInputToContinue(scanner);
 String commandId="";
 try {
 commandId = actions.sendSSMCommand(documentName, instanceId);
 waitForInputToContinue(scanner);
 System.out.println("The command was successfully sent. Command ID: " +
commandId);
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 } catch (InterruptedException e) {
 System.err.println("Thread was interrupted: " + e.getMessage());
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);

 System.out.println("5. Lets get the time when the specific command was sent
to the specific managed node");
 waitForInputToContinue(scanner);
 try {
 actions.displayCommands(commandId);
 System.out.println("The command invocations were successfully
displayed.");
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }
 waitForInputToContinue(scanner);
 System.out.println(DASHES);
```

```

System.out.println(DASHES);
System.out.println("""
 6. Now we will create an SSM OpsItem.
 A SSM OpsItem is a feature provided by Amazon's Systems Manager (SSM)
service.
 It is a type of operational data item that allows you to manage and
track various operational issues,
 events, or tasks within your AWS environment.

 You can create OpsItems to track and manage operational issues as they
arise.
 For example, you could create an OpsItem whenever your application
detects a critical error
 or an anomaly in your infrastructure.
 """);

waitForInputToContinue(scanner);
String opsItemId;
try {
 opsItemId = actions.createSSMOpsItem(title, source, category, severity);
 System.out.println(opsItemId + " was created");
} catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
} catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
}
waitForInputToContinue(scanner);
System.out.println(DASHES);

System.out.println(DASHES);
System.out.println("7. Now we will update the SSM OpsItem "+opsItemId);
waitForInputToContinue(scanner);
String description = "An update to "+opsItemId ;
try {
 actions.updateOpsItem(opsItemId, title, description);
} catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
} catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
}

```

```
 }

 System.out.println(DASHES);
 System.out.println("8. Now we will get the status of the SSM OpsItem
"+opsItemId);
 waitForInputToContinue(scanner);
 try {
 actions.describeOpsItems(opsItemId);
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }

 System.out.println(DASHES);
 System.out.println("9. Now we will resolve the SSM OpsItem "+opsItemId);
 waitForInputToContinue(scanner);
 try {
 actions.resolveOpsItem(opsItemId);
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 return;
 }

 System.out.println(DASHES);
 System.out.println("10. Would you like to delete the AWS Systems Manager
resources? (y/n)");
 String delAns = scanner.nextLine().trim();
 if (delAns.equalsIgnoreCase("y")) {
 System.out.println("You selected to delete the resources.");
 waitForInputToContinue(scanner);
 try {
 actions.deleteMaintenanceWindow(winId);
 actions.deleteDoc(documentName);
 } catch (SsmException e) {
 System.err.println("SSM error: " + e.getMessage());
 return;
 } catch (RuntimeException e) {
 System.err.println("Unexpected error: " + e.getMessage());
 }
 }
}
```



```

 return;
 }
} else {
 System.out.println("The AWS Systems Manager resources will not be
deleted");
}
System.out.println(DASHES);
CloudFormationHelper.destroyCloudFormationStack(ROLES_STACK);
System.out.println("This concludes the AWS Systems Manager SDK Basics
scenario.");
System.out.println(DASHES);
}

private static void waitForInputToContinue(Scanner scanner) {
 while (true) {
 System.out.println("");
 System.out.println("Enter 'c' followed by <ENTER> to continue:");
 String input = scanner.nextLine();

 if (input.trim().equalsIgnoreCase("c")) {
 System.out.println("Continuing with the program...");
 System.out.println("");
 break;
 } else {
 // Handle invalid input.
 System.out.println("Invalid input. Please try again.");
 }
 }
}
}
}

```

A wrapper class for Systems Manager SDK methods.

```

public class SSMActions {

 private static SsmAsyncClient ssmAsyncClient;

 private static SsmAsyncClient getAsyncClient() {
 if (ssmAsyncClient == null) {
 SdkAsyncHttpClient httpClient = NettyNioAsyncHttpClient.builder()
 .maxConcurrency(100)
 .connectionTimeout(Duration.ofSeconds(60))

```

```

 .readTimeout(Duration.ofSeconds(60))
 .writeTimeout(Duration.ofSeconds(60))
 .build();

 ClientOverrideConfiguration overrideConfig =
ClientOverrideConfiguration.builder()
 .apiCallTimeout(Duration.ofMinutes(2))
 .apiCallAttemptTimeout(Duration.ofSeconds(90))
 .retryPolicy(RetryPolicy.builder()
 .numRetries(3)
 .build())
 .build();

 ssmAsyncClient = SsmAsyncClient.builder()
 .region(Region.US_WEST_2)
 .httpClient(httpClient)
 .overrideConfiguration(overrideConfig)
 .build();
}
return ssmAsyncClient;
}

/**
 * Deletes an AWS SSM document asynchronously.
 *
 * @param documentName The name of the document to delete.
 * <p>
 * This method initiates an asynchronous request to delete an SSM document.
 * If an exception occurs, it handles the error appropriately.
 */
public void deleteDoc(String documentName) {
 DeleteDocumentRequest documentRequest = DeleteDocumentRequest.builder()
 .name(documentName)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().deleteDocument(documentRequest)
 .thenAccept(response -> {
 System.out.println("The SSM document was successfully
deleted.");
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 });
}

```

```

 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

/**
 * Deletes an AWS SSM Maintenance Window asynchronously.
 *
 * @param winId The ID of the Maintenance Window to delete.
 * <p>
 * This method initiates an asynchronous request to delete an SSM Maintenance
Window.
 * If an exception occurs, it handles the error appropriately.
 */
public void deleteMaintenanceWindow(String winId) {
 DeleteMaintenanceWindowRequest windowRequest =
DeleteMaintenanceWindowRequest.builder()
 .windowId(winId)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().deleteMaintenanceWindow(windowRequest)
 .thenAccept(response -> {
 System.out.println("The maintenance window was successfully
deleted.");
 })
 }).exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
}

```

```

 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

/**
 * Resolves an AWS SSM OpsItem asynchronously.
 *
 * @param opsID The ID of the OpsItem to resolve.
 * <p>
 * This method initiates an asynchronous request to resolve an SSM OpsItem.
 * If an exception occurs, it handles the error appropriately.
 */
public void resolveOpsItem(String opsID) {
 UpdateOpsItemRequest opsItemRequest = UpdateOpsItemRequest.builder()
 .opsItemId(opsID)
 .status(OpsItemStatus.RESOLVED)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().updateOpsItem(opsItemRequest)
 .thenAccept(response -> {
 System.out.println("OpsItem resolved successfully.");
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 }).exceptionally(ex -> {

```

```

 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

/**
 * Describes AWS SSM OpsItems asynchronously.
 *
 * @param key The key to filter OpsItems by (e.g., OPS_ITEM_ID).
 *
 * This method initiates an asynchronous request to describe SSM OpsItems.
 * If the request is successful, it prints the title and status of each OpsItem.
 * If an exception occurs, it handles the error appropriately.
 */
public void describeOpsItems(String key) {
 OpsItemFilter filter = OpsItemFilter.builder()
 .key(OpsItemFilterKey.OPS_ITEM_ID)
 .values(key)
 .operator(OpsItemFilterOperator.EQUAL)
 .build();

 DescribeOpsItemsRequest itemsRequest = DescribeOpsItemsRequest.builder()
 .maxResults(10)
 .opsItemFilters(filter)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().describeOpsItems(itemsRequest)
 .thenAccept(itemsResponse -> {

```

```

 List<OpsItemSummary> items =
itemsResponse.opsItemSummaries();
 for (OpsItemSummary item : items) {
 System.out.println("The item title is " + item.title() +
" and the status is " + item.status().toString());
 }
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
}).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
});

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

/**
 * Updates the AWS SSM OpsItem asynchronously.
 *
 * @param opsItemId The ID of the OpsItem to update.
 * @param title The new title of the OpsItem.
 * @param description The new description of the OpsItem.
 * <p>
 * This method initiates an asynchronous request to update an SSM OpsItem.
 * If the request is successful, it completes without returning a value.
 * If an exception occurs, it handles the error appropriately.
 */
public void updateOpsItem(String opsItemId, String title, String description) {
 Map<String, OpsItemDataValue> operationalData = new HashMap<>();

```

```

 operationalData.put("key1",
OpsItemDataValue.builder().value("value1").build());
 operationalData.put("key2",
OpsItemDataValue.builder().value("value2").build());

 CompletableFuture<Void> future = getOpsItem(opsItemId).thenCompose(opsItem -
> {
 UpdateOpsItemRequest request = UpdateOpsItemRequest.builder()
 .opsItemId(opsItemId)
 .title(title)
 .operationalData(operationalData)
 .status(opsItem.statusAsString())
 .description(description)
 .build();

 return getAsyncClient().updateOpsItem(request).thenAccept(response -> {
 System.out.println(opsItemId + " updated successfully.");
 }).exceptionally(ex -> {
 throw new CompletionException(ex);
 });
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
 }

 private static CompletableFuture<OpsItem> getOpsItem(String opsItemId) {
 GetOpsItemRequest request =
GetOpsItemRequest.builder().opsItemId(opsItemId).build();

```

```

 return
getAsyncClient().getOpsItem(request).thenApply(GetOpsItemResponse::opsItem);
 }

 /**
 * Creates an SSM OpsItem asynchronously.
 *
 * @param title The title of the OpsItem.
 * @param source The source of the OpsItem.
 * @param category The category of the OpsItem.
 * @param severity The severity of the OpsItem.
 * @return The ID of the created OpsItem.
 * <p>
 * This method initiates an asynchronous request to create an SSM OpsItem.
 * If the request is successful, it returns the OpsItem ID.
 * If an exception occurs, it handles the error appropriately.
 */
 public String createSSMOpsItem(String title, String source, String category,
String severity) {
 CreateOpsItemRequest opsItemRequest = CreateOpsItemRequest.builder()
 .description("Created by the SSM Java API")
 .title(title)
 .source(source)
 .category(category)
 .severity(severity)
 .build();

 CompletableFuture<CreateOpsItemResponse> future =
getAsyncClient().createOpsItem(opsItemRequest);

 try {
 CreateOpsItemResponse response = future.join();
 return response.opsItemId();
 } catch (CompletionException e) {
 Throwable cause = e.getCause();
 if (cause instanceof SsmException) {
 throw (SsmException) cause;
 } else {
 throw new RuntimeException(cause);
 }
 }
 }

 /**

```



```

 * Displays the date and time when the specific command was invoked.
 *
 * @param commandId The ID of the command to describe.
 * <p>
 * This method initiates an asynchronous request to list command invocations and
 prints the date and time of each command invocation.
 * If an exception occurs, it handles the error appropriately.
 */
 public void displayCommands(String commandId) {
 ListCommandInvocationsRequest commandInvocationsRequest =
ListCommandInvocationsRequest.builder()
 .commandId(commandId)
 .build();

 CompletableFuture<ListCommandInvocationsResponse> future =
getAsyncClient().listCommandInvocations(commandInvocationsRequest);
 future.thenAccept(response -> {
 List<CommandInvocation> commandList = response.commandInvocations();
 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss").withZone(ZoneId.systemDefault());
 for (CommandInvocation invocation : commandList) {
 System.out.println("The time of the command invocation is " +
formatter.format(invocation.requestedDateTime()));
 }
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof SsmException) {
 throw (SsmException) cause;
 } else {
 throw new RuntimeException(cause);
 }
 }).join();
 }

 /**
 * Sends a SSM command to a managed node asynchronously.
 *
 * @param documentName The name of the document to use.
 * @param instanceId The ID of the instance to send the command to.
 * @return The command ID.
 * <p>
 * This method initiates asynchronous requests to send a SSM command to a
 managed node.

```

```

 * It waits until the document is active, sends the command, and checks the
 command execution status.
 */
 public String sendSSMCommand(String documentName, String instanceId) throws
 InterruptedException, SsmException {
 // Before we use Document to send a command - make sure it is active.
 CompletableFuture<Void> documentActiveFuture = CompletableFuture.runAsync(()
-> {
 boolean isDocumentActive = false;
 DescribeDocumentRequest request = DescribeDocumentRequest.builder()
 .name(documentName)
 .build();

 while (!isDocumentActive) {
 CompletableFuture<DescribeDocumentResponse> response =
getAsyncClient().describeDocument(request);
 String documentStatus = response.join().document().statusAsString();
 if (documentStatus.equals("Active")) {
 System.out.println("The SSM document is active and ready to
use.");
 isDocumentActive = true;
 } else {
 System.out.println("The SSM document is not active. Status: " +
documentStatus);
 try {
 Thread.sleep(5000);
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 }
 }
 }));

 documentActiveFuture.join();

 // Create the SendCommandRequest.
 SendCommandRequest commandRequest = SendCommandRequest.builder()
 .documentName(documentName)
 .instanceIds(instanceId)
 .build();

 // Send the command.
 CompletableFuture<SendCommandResponse> commandFuture =
getAsyncClient().sendCommand(commandRequest);

```

```

final String[] commandId = {null};

commandFuture.whenComplete((commandResponse, ex) -> {
 if (commandResponse != null) {
 commandId[0] = commandResponse.command().commandId();
 System.out.println("Command ID: " + commandId[0]);

 // Wait for the command execution to complete.
 GetCommandInvocationRequest invocationRequest =
GetCommandInvocationRequest.builder()
 .commandId(commandId[0])
 .instanceId(instanceId)
 .build();

 try {
 System.out.println("Wait 5 secs");
 TimeUnit.SECONDS.sleep(5);

 // Retrieve the command execution details.
 CompletableFuture<GetCommandInvocationResponse> invocationFuture
= getAsyncClient().getCommandInvocation(invocationRequest);
 invocationFuture.whenComplete((commandInvocationResponse,
invocationEx) -> {
 if (commandInvocationResponse != null) {
 // Check the status of the command execution.
 CommandInvocationStatus status =
commandInvocationResponse.status();
 if (status == CommandInvocationStatus.SUCCESS) {
 System.out.println("Command execution successful");
 } else {
 System.out.println("Command execution failed.
Status: " + status);
 }
 } else {
 Throwable invocationCause = (invocationEx instanceof
CompletionException) ? invocationEx.getCause() : invocationEx;
 throw new CompletionException(invocationCause);
 }
 }).join();
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 } else {

```

```

 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof SsmException) {
 throw (SsmException) cause;
 } else {
 throw new RuntimeException(cause);
 }
 }
}).join();

return commandId[0];
}

/**
 * Creates an AWS SSM document asynchronously.
 *
 * @param docName The name of the document to create.
 * <p>
 * This method initiates an asynchronous request to create an SSM document.
 * If the request is successful, it prints the document status.
 * If an exception occurs, it handles the error appropriately.
 */
public void createSSMDoc(String docName) throws SsmException {
 String jsonData = ""
 {
 "schemaVersion": "2.2",
 "description": "Run a simple shell command",
 "mainSteps": [
 {
 "action": "aws:runShellScript",
 "name": "runEchoCommand",
 "inputs": {
 "runCommand": [
 "echo 'Hello, world!'"
]
 }
 }
]
 }
 """;

 CreateDocumentRequest request = CreateDocumentRequest.builder()
 .content(jsonData)
 .name(docName)

```

```

 .documentType(DocumentType.COMMAND)
 .build();

 CompletableFuture<CreateDocumentResponse> future =
getAsyncClient().createDocument(request);
 future.thenAccept(response -> {
 System.out.println("The status of the SSM document is " +
response.documentDescription().status());
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof DocumentAlreadyExistsException) {
 throw new CompletionException(cause);
 } else if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }).join();
}

/**
 * Updates an SSM maintenance window asynchronously.
 *
 * @param id The ID of the maintenance window to update.
 * @param name The new name for the maintenance window.
 * <p>
 * This method initiates an asynchronous request to update an SSM maintenance
window.
 * If the request is successful, it prints a success message.
 * If an exception occurs, it handles the error appropriately.
 */
public void updateSSMMaintenanceWindow(String id, String name) throws
SsmException {
 UpdateMaintenanceWindowRequest updateRequest =
UpdateMaintenanceWindowRequest.builder()
 .windowId(id)
 .allowUnassociatedTargets(true)
 .duration(24)
 .enabled(true)
 .name(name)
 .schedule("cron(0 0 ? * MON *)")
 .build();

```

```

 CompletableFuture<UpdateMaintenanceWindowResponse> future =
getAsyncClient().updateMaintenanceWindow(updateRequest);
 future.whenComplete((response, ex) -> {
 if (response != null) {
 System.out.println("The SSM maintenance window was successfully
updated");
 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 }).join();
 }

/**
 * Creates an SSM maintenance window asynchronously.
 *
 * @param winName The name of the maintenance window.
 * @return The ID of the created or existing maintenance window.
 * <p>
 * This method initiates an asynchronous request to create an SSM maintenance
window.
 * If the request is successful, it prints the maintenance window ID.
 * If an exception occurs, it handles the error appropriately.
 */
 public String createMaintenanceWindow(String winName) throws SsmException,
DocumentAlreadyExistsException {
 CreateMaintenanceWindowRequest request =
CreateMaintenanceWindowRequest.builder()
 .name(winName)
 .description("This is my maintenance window")
 .allowUnassociatedTargets(true)
 .duration(2)
 .cutoff(1)
 .schedule("cron(0 10 ? * MON-FRI *)")
 .build();

 CompletableFuture<CreateMaintenanceWindowResponse> future =
getAsyncClient().createMaintenanceWindow(request);
 final String[] windowId = {null};

```

```

 future.whenComplete((response, ex) -> {
 if (response != null) {
 String maintenanceWindowId = response.windowId();
 System.out.println("The maintenance window id is " +
maintenanceWindowId);
 windowId[0] = maintenanceWindowId;
 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof DocumentAlreadyExistsException) {
 throw new CompletionException(cause);
 } else if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 }).join();

 if (windowId[0] == null) {
 MaintenanceWindowFilter filter = MaintenanceWindowFilter.builder()
 .key("name")
 .values(winName)
 .build();

 DescribeMaintenanceWindowsRequest winRequest =
DescribeMaintenanceWindowsRequest.builder()
 .filters(filter)
 .build();

 CompletableFuture<DescribeMaintenanceWindowsResponse> describeFuture =
getAsyncClient().describeMaintenanceWindows(winRequest);
 describeFuture.whenComplete((describeResponse, describeEx) -> {
 if (describeResponse != null) {
 List<MaintenanceWindowIdentity> windows =
describeResponse.windowIdentities();
 if (!windows.isEmpty()) {
 windowId[0] = windows.get(0).windowId();
 System.out.println("Window ID: " + windowId[0]);
 } else {
 System.out.println("Window not found.");
 windowId[0] = "";
 }
 }
 })
 } else {

```

```
 Throwable describeCause = (describeEx instanceof
CompletionException) ? describeEx.getCause() : describeEx;
 throw new RuntimeException("Error describing maintenance
windows: " + describeCause.getMessage(), describeCause);
 }
 }).join();
}

return windowId[0];
}
}
```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [CreateDocument](#)
  - [CreateMaintenanceWindow](#)
  - [CreateOpsItem](#)
  - [DeleteMaintenanceWindow](#)
  - [ListCommandInvocations](#)
  - [SendCommand](#)
  - [UpdateOpsItem](#)

## Actions

### CreateDocument

The following code example shows how to use CreateDocument.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Creates an AWS SSM document asynchronously.
```



```

*
* @param docName The name of the document to create.
* <p>
* This method initiates an asynchronous request to create an SSM document.
* If the request is successful, it prints the document status.
* If an exception occurs, it handles the error appropriately.
*/
public void createSSMDoc(String docName) throws SsmException {
 String jsonData = ""
 {
 "schemaVersion": "2.2",
 "description": "Run a simple shell command",
 "mainSteps": [
 {
 "action": "aws:runShellScript",
 "name": "runEchoCommand",
 "inputs": {
 "runCommand": [
 "echo 'Hello, world!'"
]
 }
 }
]
 }
 """;

 CreateDocumentRequest request = CreateDocumentRequest.builder()
 .content(jsonData)
 .name(docName)
 .documentType(DocumentType.COMMAND)
 .build();

 CompletableFuture<CreateDocumentResponse> future =
getAsyncClient().createDocument(request);
 future.thenAccept(response -> {
 System.out.println("The status of the SSM document is " +
response.documentDescription().status());
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof DocumentAlreadyExistsException) {
 throw new CompletionException(cause);
 } else if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 }
 });
}

```

```

 } else {
 throw new RuntimeException(cause);
 }
 }).join();
}

```

- For API details, see [CreateDocument](#) in *AWS SDK for Java 2.x API Reference*.

## CreateMaintenanceWindow

The following code example shows how to use CreateMaintenanceWindow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an SSM maintenance window asynchronously.
 *
 * @param winName The name of the maintenance window.
 * @return The ID of the created or existing maintenance window.
 * <p>
 * This method initiates an asynchronous request to create an SSM maintenance
window.
 * If the request is successful, it prints the maintenance window ID.
 * If an exception occurs, it handles the error appropriately.
 */
public String createMaintenanceWindow(String winName) throws SsmException,
DocumentAlreadyExistsException {
 CreateMaintenanceWindowRequest request =
CreateMaintenanceWindowRequest.builder()
 .name(winName)
 .description("This is my maintenance window")
 .allowUnassociatedTargets(true)
 .duration(2)
 .cutoff(1)
 .schedule("cron(0 10 ? * MON-FRI *)")

```

```

 .build();

 CompletableFuture<CreateMaintenanceWindowResponse> future =
getAsyncClient().createMaintenanceWindow(request);
 final String[] windowId = {null};
 future.whenComplete((response, ex) -> {
 if (response != null) {
 String maintenanceWindowId = response.windowId();
 System.out.println("The maintenance window id is " +
maintenanceWindowId);
 windowId[0] = maintenanceWindowId;
 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof DocumentAlreadyExistsException) {
 throw new CompletionException(cause);
 } else if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 }).join();

 if (windowId[0] == null) {
 MaintenanceWindowFilter filter = MaintenanceWindowFilter.builder()
 .key("name")
 .values(winName)
 .build();

 DescribeMaintenanceWindowsRequest winRequest =
DescribeMaintenanceWindowsRequest.builder()
 .filters(filter)
 .build();

 CompletableFuture<DescribeMaintenanceWindowsResponse> describeFuture =
getAsyncClient().describeMaintenanceWindows(winRequest);
 describeFuture.whenComplete((describeResponse, describeEx) -> {
 if (describeResponse != null) {
 List<MaintenanceWindowIdentity> windows =
describeResponse.windowIdentities();
 if (!windows.isEmpty()) {
 windowId[0] = windows.get(0).windowId();
 System.out.println("Window ID: " + windowId[0]);
 }
 }
 });
 }
 }
}

```

```

 } else {
 System.out.println("Window not found.");
 windowId[0] = "";
 }
 } else {
 Throwable describeCause = (describeEx instanceof
CompletionException) ? describeEx.getCause() : describeEx;
 throw new RuntimeException("Error describing maintenance
windows: " + describeCause.getMessage(), describeCause);
 }
}).join();
}

return windowId[0];
}

```

- For API details, see [CreateMaintenanceWindow](#) in *AWS SDK for Java 2.x API Reference*.

## CreateOpsItem

The following code example shows how to use CreateOpsItem.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Creates an SSM OpsItem asynchronously.
 *
 * @param title The title of the OpsItem.
 * @param source The source of the OpsItem.
 * @param category The category of the OpsItem.
 * @param severity The severity of the OpsItem.
 * @return The ID of the created OpsItem.
 * <p>
 * This method initiates an asynchronous request to create an SSM OpsItem.
 * If the request is successful, it returns the OpsItem ID.

```

```

 * If an exception occurs, it handles the error appropriately.
 */
 public String createSSMOpsItem(String title, String source, String category,
String severity) {
 CreateOpsItemRequest opsItemRequest = CreateOpsItemRequest.builder()
 .description("Created by the SSM Java API")
 .title(title)
 .source(source)
 .category(category)
 .severity(severity)
 .build();

 CompletableFuture<CreateOpsItemResponse> future =
getAsyncClient().createOpsItem(opsItemRequest);

 try {
 CreateOpsItemResponse response = future.join();
 return response.opsItemId();
 } catch (CompletionException e) {
 Throwable cause = e.getCause();
 if (cause instanceof SsmException) {
 throw (SsmException) cause;
 } else {
 throw new RuntimeException(cause);
 }
 }
 }
}

```

- For API details, see [CreateOpsItem](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteDocument

The following code example shows how to use DeleteDocument.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Deletes an AWS SSM document asynchronously.
 *
 * @param documentName The name of the document to delete.
 * <p>
 * This method initiates an asynchronous request to delete an SSM document.
 * If an exception occurs, it handles the error appropriately.
 */
public void deleteDoc(String documentName) {
 DeleteDocumentRequest documentRequest = DeleteDocumentRequest.builder()
 .name(documentName)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().deleteDocument(documentRequest)
 .thenAccept(response -> {
 System.out.println("The SSM document was successfully
deleted.");
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

```

- For API details, see [DeleteDocument](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteMaintenanceWindow

The following code example shows how to use DeleteMaintenanceWindow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Deletes an AWS SSM Maintenance Window asynchronously.
 *
 * @param winId The ID of the Maintenance Window to delete.
 * <p>
 * This method initiates an asynchronous request to delete an SSM Maintenance
Window.
 * If an exception occurs, it handles the error appropriately.
 */
public void deleteMaintenanceWindow(String winId) {
 DeleteMaintenanceWindowRequest windowRequest =
DeleteMaintenanceWindowRequest.builder()
 .windowId(winId)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().deleteMaintenanceWindow(windowRequest)
 .thenAccept(response -> {
 System.out.println("The maintenance window was successfully
deleted.");
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;

 if (cause instanceof SsmException) {
```

```

 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
});

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

```

- For API details, see [DeleteMaintenanceWindow](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeOpsItems

The following code example shows how to use DescribeOpsItems.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Describes AWS SSM OpsItems asynchronously.
 *
 * @param key The key to filter OpsItems by (e.g., OPS_ITEM_ID).
 *
 * This method initiates an asynchronous request to describe SSM OpsItems.
 * If the request is successful, it prints the title and status of each OpsItem.
 * If an exception occurs, it handles the error appropriately.
 */
public void describeOpsItems(String key) {
 OpsItemFilter filter = OpsItemFilter.builder()

```



```

 .key(OpsItemFilterKey.OPS_ITEM_ID)
 .values(key)
 .operator(OpsItemFilterOperator.EQUAL)
 .build();

DescribeOpsItemsRequest itemsRequest = DescribeOpsItemsRequest.builder()
 .maxResults(10)
 .opsItemFilters(filter)
 .build();

CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().describeOpsItems(itemsRequest)
 .thenAccept(itemsResponse -> {
 List<OpsItemSummary> items =
itemsResponse.opsItemSummaries();
 for (OpsItemSummary item : items) {
 System.out.println("The item title is " + item.title() +
" and the status is " + item.status().toString());
 }
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

```

- For API details, see [DescribeOpsItems](#) in *AWS SDK for Java 2.x API Reference*.

## DescribeParameters

The following code example shows how to use DescribeParameters.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ssm.SsmClient;
import software.amazon.awssdk.services.ssm.model.GetParameterRequest;
import software.amazon.awssdk.services.ssm.model.GetParameterResponse;
import software.amazon.awssdk.services.ssm.model.SsmException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetParameter {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <paraName>

 Where:
 paraName - The name of the parameter.
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }
 }
}
```

```

 }

 String paraName = args[0];
 Region region = Region.US_EAST_1;
 SsmClient ssmClient = SsmClient.builder()
 .region(region)
 .build();

 getParaValue(ssmClient, paraName);
 ssmClient.close();
}

public static void getParaValue(SsmClient ssmClient, String paraName) {
 try {
 GetParameterRequest parameterRequest = GetParameterRequest.builder()
 .name(paraName)
 .build();

 GetParameterResponse parameterResponse =
 ssmClient.getParameter(parameterRequest);
 System.out.println("The parameter value is " +
 parameterResponse.parameter().value());

 } catch (SsmException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DescribeParameters](#) in *AWS SDK for Java 2.x API Reference*.

## PutParameter

The following code example shows how to use PutParameter.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.ssm.SsmClient;
import software.amazon.awssdk.services.ssm.model.ParameterType;
import software.amazon.awssdk.services.ssm.model.PutParameterRequest;
import software.amazon.awssdk.services.ssm.model.SsmException;

public class PutParameter {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <paraName>

 Where:
 paraName - The name of the parameter.
 paraValue - The value of the parameter.
 "";

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String paraName = args[0];
 String paraValue = args[1];
 Region region = Region.US_EAST_1;
 SsmClient ssmClient = SsmClient.builder()
 .region(region)
 .build();

 putParaValue(ssmClient, paraName, paraValue);
 ssmClient.close();
 }
}
```

```
public static void putParaValue(SsmClient ssmClient, String paraName, String
value) {
 try {
 PutParameterRequest parameterRequest = PutParameterRequest.builder()
 .name(paraName)
 .type(ParameterType.STRING)
 .value(value)
 .build();

 ssmClient.putParameter(parameterRequest);
 System.out.println("The parameter was successfully added.");

 } catch (SsmException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [PutParameter](#) in *AWS SDK for Java 2.x API Reference*.

## SendCommand

The following code example shows how to use SendCommand.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/**
 * Sends a SSM command to a managed node asynchronously.
 *
 * @param documentName The name of the document to use.
 * @param instanceId The ID of the instance to send the command to.
 * @return The command ID.
 * <p>
```

```

 * This method initiates asynchronous requests to send a SSM command to a
 managed node.
 * It waits until the document is active, sends the command, and checks the
 command execution status.
 */
 public String sendSSMCommand(String documentName, String instanceId) throws
 InterruptedException, SsmException {
 // Before we use Document to send a command - make sure it is active.
 CompletableFuture<Void> documentActiveFuture = CompletableFuture.runAsync(()
-> {
 boolean isDocumentActive = false;
 DescribeDocumentRequest request = DescribeDocumentRequest.builder()
 .name(documentName)
 .build();

 while (!isDocumentActive) {
 CompletableFuture<DescribeDocumentResponse> response =
getAsyncClient().describeDocument(request);
 String documentStatus = response.join().document().statusAsString();
 if (documentStatus.equals("Active")) {
 System.out.println("The SSM document is active and ready to
use.");
 isDocumentActive = true;
 } else {
 System.out.println("The SSM document is not active. Status: " +
documentStatus);
 try {
 Thread.sleep(5000);
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 }
 }
 });

 documentActiveFuture.join();

 // Create the SendCommandRequest.
 SendCommandRequest commandRequest = SendCommandRequest.builder()
 .documentName(documentName)
 .instanceIds(instanceId)
 .build();

 // Send the command.

```

```

 CompletableFuture<SendCommandResponse> commandFuture =
getAsyncClient().sendCommand(commandRequest);
 final String[] commandId = {null};

 commandFuture.whenComplete((commandResponse, ex) -> {
 if (commandResponse != null) {
 commandId[0] = commandResponse.command().commandId();
 System.out.println("Command ID: " + commandId[0]);

 // Wait for the command execution to complete.
 GetCommandInvocationRequest invocationRequest =
GetCommandInvocationRequest.builder()
 .commandId(commandId[0])
 .instanceId(instanceId)
 .build();

 try {
 System.out.println("Wait 5 secs");
 TimeUnit.SECONDS.sleep(5);

 // Retrieve the command execution details.
 CompletableFuture<GetCommandInvocationResponse> invocationFuture
= getAsyncClient().getCommandInvocation(invocationRequest);
 invocationFuture.whenComplete((commandInvocationResponse,
invocationEx) -> {
 if (commandInvocationResponse != null) {
 // Check the status of the command execution.
 CommandInvocationStatus status =
commandInvocationResponse.status();
 if (status == CommandInvocationStatus.SUCCESS) {
 System.out.println("Command execution successful");
 } else {
 System.out.println("Command execution failed.
Status: " + status);
 }
 } else {
 Throwable invocationCause = (invocationEx instanceof
CompletionException) ? invocationEx.getCause() : invocationEx;
 throw new CompletionException(invocationCause);
 }
 }).join();
 } catch (InterruptedException e) {
 throw new RuntimeException(e);
 }
 }
 });
 }

```

```

 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof SsmException) {
 throw (SsmException) cause;
 } else {
 throw new RuntimeException(cause);
 }
 }
 }).join();

 return commandId[0];
}

```

- For API details, see [SendCommand](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateMaintenanceWindow

The following code example shows how to use UpdateMaintenanceWindow.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Updates an SSM maintenance window asynchronously.
 *
 * @param id The ID of the maintenance window to update.
 * @param name The new name for the maintenance window.
 * <p>
 * This method initiates an asynchronous request to update an SSM maintenance
window.
 * If the request is successful, it prints a success message.
 * If an exception occurs, it handles the error appropriately.
 */
public void updateSSMMaintenanceWindow(String id, String name) throws
SsmException {

```



```

 UpdateMaintenanceWindowRequest updateRequest =
UpdateMaintenanceWindowRequest.builder()
 .windowId(id)
 .allowUnassociatedTargets(true)
 .duration(24)
 .enabled(true)
 .name(name)
 .schedule("cron(0 0 ? * MON *)")
 .build();

 CompletableFuture<UpdateMaintenanceWindowResponse> future =
getAsyncClient().updateMaintenanceWindow(updateRequest);
 future.whenComplete((response, ex) -> {
 if (response != null) {
 System.out.println("The SSM maintenance window was successfully
updated");
 } else {
 Throwable cause = (ex instanceof CompletionException) ?
ex.getCause() : ex;
 if (cause instanceof SsmException) {
 throw new CompletionException(cause);
 } else {
 throw new RuntimeException(cause);
 }
 }
 }).join();
 }

```

- For API details, see [UpdateMaintenanceWindow](#) in *AWS SDK for Java 2.x API Reference*.

## UpdateOpsItem

The following code example shows how to use UpdateOpsItem.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * Resolves an AWS SSM OpsItem asynchronously.
 *
 * @param opsID The ID of the OpsItem to resolve.
 * <p>
 * This method initiates an asynchronous request to resolve an SSM OpsItem.
 * If an exception occurs, it handles the error appropriately.
 */
public void resolveOpsItem(String opsID) {
 UpdateOpsItemRequest opsItemRequest = UpdateOpsItemRequest.builder()
 .opsItemId(opsID)
 .status(OpsItemStatus.RESOLVED)
 .build();

 CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
 getAsyncClient().updateOpsItem(opsItemRequest)
 .thenAccept(response -> {
 System.out.println("OpsItem resolved successfully.");
 })
 .exceptionally(ex -> {
 throw new CompletionException(ex);
 }).join();
 }).exceptionally(ex -> {
 Throwable cause = (ex instanceof CompletionException) ? ex.getCause() :
ex;
 if (cause instanceof SsmException) {
 throw new RuntimeException("SSM error: " + cause.getMessage(),
cause);
 } else {
 throw new RuntimeException("Unexpected error: " +
cause.getMessage(), cause);
 }
 });

 try {
 future.join();
 } catch (CompletionException ex) {
 throw ex.getCause() instanceof RuntimeException ? (RuntimeException)
ex.getCause() : ex;
 }
}

```

- For API details, see [UpdateOpsItem](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Textract examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Textract.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### AnalyzeDocument

The following code example shows how to use `AnalyzeDocument`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.textract.TextractClient;
import software.amazon.awssdk.services.textract.model.AnalyzeDocumentRequest;
import software.amazon.awssdk.services.textract.model.Document;
```

```
import software.amazon.awssdk.services.textract.model.FeatureType;
import software.amazon.awssdk.services.textract.model.AnalyzeDocumentResponse;
import software.amazon.awssdk.services.textract.model.Block;
import software.amazon.awssdk.services.textract.model.TexttractException;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class AnalyzeDocument {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <sourceDoc>\s

 Where:
 sourceDoc - The path where the document is located (must be an
image, for example, C:/AWS/book.png).\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String sourceDoc = args[0];
 Region region = Region.US_EAST_2;
 TextractClient textractClient = TextractClient.builder()
 .region(region)
 .build();

 analyzeDoc(textractClient, sourceDoc);
 }
}
```

```

 textractClient.close();
 }

 public static void analyzeDoc(TextractClient textractClient, String sourceDoc) {
 try {
 InputStream sourceStream = new FileInputStream(new File(sourceDoc));
 SdkBytes sourceBytes = SdkBytes.fromInputStream(sourceStream);

 // Get the input Document object as bytes
 Document myDoc = Document.builder()
 .bytes(sourceBytes)
 .build();

 List<FeatureType> featureTypes = new ArrayList<FeatureType>();
 featureTypes.add(FeatureType.FORMS);
 featureTypes.add(FeatureType.TABLES);

 AnalyzeDocumentRequest analyzeDocumentRequest =
 AnalyzeDocumentRequest.builder()
 .featureTypes(featureTypes)
 .document(myDoc)
 .build();

 AnalyzeDocumentResponse analyzeDocument =
 textractClient.analyzeDocument(analyzeDocumentRequest);
 List<Block> docInfo = analyzeDocument.blocks();
 Iterator<Block> blockIterator = docInfo.iterator();

 while (blockIterator.hasNext()) {
 Block block = blockIterator.next();
 System.out.println("The block type is " +
 block.blockType().toString());
 }

 } catch (TextractException | FileNotFoundException e) {

 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
 }
}

```

- For API details, see [AnalyzeDocument](#) in *AWS SDK for Java 2.x API Reference*.

## DetectDocumentText

The following code example shows how to use DetectDocumentText.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

Detect text from an input document.

```
import software.amazon.awssdk.core.SdkBytes;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.textract.TextractClient;
import software.amazon.awssdk.services.textract.model.Document;
import software.amazon.awssdk.services.textract.model.DetectDocumentTextRequest;
import software.amazon.awssdk.services.textract.model.DetectDocumentTextResponse;
import software.amazon.awssdk.services.textract.model.Block;
import software.amazon.awssdk.services.textract.model.DocumentMetadata;
import software.amazon.awssdk.services.textract.model.TextractException;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectDocumentText {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <sourceDoc>\s
```

```

 Where:
 sourceDoc - The path where the document is located (must be an
image, for example, C:/AWS/book.png).\s
 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String sourceDoc = args[0];
 Region region = Region.US_EAST_2;
 TextractClient textractClient = TextractClient.builder()
 .region(region)
 .build();

 detectDocText(textractClient, sourceDoc);
 textractClient.close();
}

public static void detectDocText(TextractClient textractClient, String
sourceDoc) {
 try {
 InputStream sourceStream = new FileInputStream(new File(sourceDoc));
 SdkBytes sourceBytes = SdkBytes.fromInputStream(sourceStream);

 // Get the input Document object as bytes.
 Document myDoc = Document.builder()
 .bytes(sourceBytes)
 .build();

 DetectDocumentTextRequest detectDocumentTextRequest =
DetectDocumentTextRequest.builder()
 .document(myDoc)
 .build();

 // Invoke the Detect operation.
 DetectDocumentTextResponse textResponse =
textractClient.detectDocumentText(detectDocumentTextRequest);
 List<Block> docInfo = textResponse.blocks();
 for (Block block : docInfo) {
 System.out.println("The block type is " +
block.blockType().toString());

```

```

 }

 DocumentMetadata documentMetadata = textResponse.documentMetadata();
 System.out.println("The number of pages in the document is " +
documentMetadata.pages());

 } catch (TextractException | FileNotFoundException e) {

 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}
}

```

Detect text from a document located in an Amazon S3 bucket.

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.textract.model.S3Object;
import software.amazon.awssdk.services.textract.TextractClient;
import software.amazon.awssdk.services.textract.model.Document;
import software.amazon.awssdk.services.textract.model.DetectDocumentTextRequest;
import software.amazon.awssdk.services.textract.model.DetectDocumentTextResponse;
import software.amazon.awssdk.services.textract.model.Block;
import software.amazon.awssdk.services.textract.model.DocumentMetadata;
import software.amazon.awssdk.services.textract.model.TextractException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DetectDocumentTextS3 {

 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <docName>\s

```



```

 Where:
 bucketName - The name of the Amazon S3 bucket that contains the
document.\s

 docName - The document name (must be an image, i.e., book.png).
\s

 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String docName = args[1];
 Region region = Region.US_WEST_2;
 TextractClient textractClient = TextractClient.builder()
 .region(region)
 .build();

 detectDocTextS3(textractClient, bucketName, docName);
 textractClient.close();
 }

 public static void detectDocTextS3(TextractClient textractClient, String
bucketName, String docName) {
 try {
 S3Object s3Object = S3Object.builder()
 .bucket(bucketName)
 .name(docName)
 .build();

 // Create a Document object and reference the s3Object instance.
 Document myDoc = Document.builder()
 .s3Object(s3Object)
 .build();

 DetectDocumentTextRequest detectDocumentTextRequest =
DetectDocumentTextRequest.builder()
 .document(myDoc)
 .build();

 DetectDocumentTextResponse textResponse =
textractClient.detectDocumentText(detectDocumentTextRequest);

```

```

 for (Block block : textResponse.blocks()) {
 System.out.println("The block type is " +
 block.blockType().toString());
 }

 DocumentMetadata documentMetadata = textResponse.documentMetadata();
 System.out.println("The number of pages in the document is " +
 documentMetadata.pages());

 } catch (TextractException e) {

 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DetectDocumentText](#) in *AWS SDK for Java 2.x API Reference*.

## StartDocumentAnalysis

The following code example shows how to use StartDocumentAnalysis.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.textract.model.S3Object;
import software.amazon.awssdk.services.textract.TextractClient;
import software.amazon.awssdk.services.textract.model.StartDocumentAnalysisRequest;
import software.amazon.awssdk.services.textract.model.DocumentLocation;
import software.amazon.awssdk.services.textract.model.TextractException;
import software.amazon.awssdk.services.textract.model.StartDocumentAnalysisResponse;
import software.amazon.awssdk.services.textract.model.GetDocumentAnalysisRequest;
import software.amazon.awssdk.services.textract.model.GetDocumentAnalysisResponse;
import software.amazon.awssdk.services.textract.model.FeatureType;

```

```

import java.util.ArrayList;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class StartDocumentAnalysis {
 public static void main(String[] args) {
 final String usage = ""

 Usage:
 <bucketName> <docName>\s

 Where:
 bucketName - The name of the Amazon S3 bucket that contains the
document.\s
 docName - The document name (must be an image, for example,
book.png).\s

 """;

 if (args.length != 2) {
 System.out.println(usage);
 System.exit(1);
 }

 String bucketName = args[0];
 String docName = args[1];
 Region region = Region.US_WEST_2;
 TextractClient textractClient = TextractClient.builder()
 .region(region)
 .build();

 String jobId = startDocAnalysisS3(textractClient, bucketName, docName);
 System.out.println("Getting results for job " + jobId);
 String status = getJobResults(textractClient, jobId);
 System.out.println("The job status is " + status);
 textractClient.close();
 }
}

```

```

 public static String startDocAnalysisS3(TexttractClient texttractClient, String
bucketName, String docName) {
 try {
 List<FeatureType> myList = new ArrayList<>();
 myList.add(FeatureType.TABLES);
 myList.add(FeatureType.FORMS);

 S3Object s3Object = S3Object.builder()
 .bucket(bucketName)
 .name(docName)
 .build();

 DocumentLocation location = DocumentLocation.builder()
 .s3Object(s3Object)
 .build();

 StartDocumentAnalysisRequest documentAnalysisRequest =
StartDocumentAnalysisRequest.builder()
 .documentLocation(location)
 .featureTypes(myList)
 .build();

 StartDocumentAnalysisResponse response =
texttractClient.startDocumentAnalysis(documentAnalysisRequest);

 // Get the job ID
 String jobId = response.jobId();
 return jobId;

 } catch (TexttractException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 return "";
 }

 private static String getJobResults(TexttractClient texttractClient, String jobId)
{
 boolean finished = false;
 int index = 0;
 String status = "";

 try {
 while (!finished) {

```

```

 GetDocumentAnalysisRequest analysisRequest =
GetDocumentAnalysisRequest.builder()
 .jobId(jobId)
 .maxResults(1000)
 .build();

 GetDocumentAnalysisResponse response =
textractClient.getDocumentAnalysis(analysisRequest);
 status = response.jobStatus().toString();

 if (status.compareTo("SUCCEEDED") == 0)
 finished = true;
 else {
 System.out.println(index + " status is: " + status);
 Thread.sleep(1000);
 }
 index++;
 }

 return status;

} catch (InterruptedException e) {
 System.out.println(e.getMessage());
 System.exit(1);
}
return "";
}
}

```

- For API details, see [StartDocumentAnalysis](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Create an application to analyze customer feedback

The following code example shows how to create an application that analyzes customer comment cards, translates them from their original language, determines their sentiment, and generates an audio file from the translated text.

## SDK for Java 2.x

This example application analyzes and stores customer feedback cards. Specifically, it fulfills the need of a fictitious hotel in New York City. The hotel receives feedback from guests in various languages in the form of physical comment cards. That feedback is uploaded into the app through a web client. After an image of a comment card is uploaded, the following steps occur:

- Text is extracted from the image using Amazon Textract.
- Amazon Comprehend determines the sentiment of the extracted text and its language.
- The extracted text is translated to English using Amazon Translate.
- Amazon Polly synthesizes an audio file from the extracted text.

The full app can be deployed with the AWS CDK. For source code and deployment instructions, see the project in [GitHub](#).

### Services used in this example

- Amazon Comprehend
- Lambda
- Amazon Polly
- Amazon Textract
- Amazon Translate

## Amazon Transcribe examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Transcribe.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

### Topics

- [Actions](#)

- [Scenarios](#)

## Actions

### ListTranscriptionJobs

The following code example shows how to use ListTranscriptionJobs.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class ListTranscriptionJobs {
 public static void main(String[] args) {
 TranscribeClient transcribeClient = TranscribeClient.builder()
 .region(Region.US_EAST_1)
 .build();

 listTranscriptionJobs(transcribeClient);
 }

 public static void listTranscriptionJobs(TranscribeClient transcribeClient)
 {
 ListTranscriptionJobsRequest listJobsRequest =
 ListTranscriptionJobsRequest.builder()
 .build();

 transcribeClient.listTranscriptionJobsPaginator(listJobsRequest).stream()
 .flatMap(response -> response.transcriptionJobSummaries().stream())
 .forEach(jobSummary -> {
 System.out.println("Job Name: " +
 jobSummary.transcriptionJobName());
 System.out.println("Job Status: " +
 jobSummary.transcriptionJobStatus());
 System.out.println("Output Location: " +
 jobSummary.outputLocationType());
 });
 }
}
```

```
 // Add more information as needed

 // Retrieve additional details for the job if necessary
 GetTranscriptionJobResponse jobDetails =
transcribeClient.getTranscriptionJob(
 GetTranscriptionJobRequest.builder()
 .transcriptionJobName(jobSummary.transcriptionJobName())
 .build());

 // Display additional details
 System.out.println("Language Code: " +
jobDetails.transcriptionJob().languageCode());
 System.out.println("Media Format: " +
jobDetails.transcriptionJob().mediaFormat());
 // Add more details as needed

 System.out.println("-----");
 });
}
```

- For API details, see [ListTranscriptionJobs](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Transcribe audio and get job data

The following code example shows how to:

- Start a transcription job with Amazon Transcribe.
- Wait for the job to complete.
- Get the URI where the transcript is stored.

For more information, see [Getting started with Amazon Transcribe](#).



## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](https://github.com/aws-samples/aws-code-examples).

Transcribes a PCM file.

```
/**
 * To run this AWS code example, ensure that you have set up your development
 * environment, including your AWS credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class TranscribeStreamingDemoFile {
 private static final Region REGION = Region.US_EAST_1;
 private static TranscribeStreamingAsyncClient client;

 public static void main(String args[]) throws ExecutionException,
 InterruptedException {

 final String USAGE = "\n" +
 "Usage:\n" +
 " <file> \n\n" +
 "Where:\n" +
 " file - the location of a PCM file to transcribe. In this
example, ensure the PCM file is 16 hertz (Hz). \n";

 if (args.length != 1) {
 System.out.println(USAGE);
 System.exit(1);
 }

 String file = args[0];
 client = TranscribeStreamingAsyncClient.builder()
 .region(REGION)
 .build();
 }
}
```

```

 CompletableFuture<Void> result =
client.startStreamTranscription(getRequest(16_000),
 new AudioStreamPublisher(getStreamFromFile(file)),
 getResponseHandler());

 result.get();
 client.close();
 }

 private static InputStream getStreamFromFile(String file) {
 try {
 File inputFile = new File(file);
 InputStream audioStream = new FileInputStream(inputFile);
 return audioStream;

 } catch (FileNotFoundException e) {
 throw new RuntimeException(e);
 }
 }

 private static StartStreamTranscriptionRequest getRequest(Integer
mediaSampleRateHertz) {
 return StartStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US)
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .build();
 }

 private static StartStreamTranscriptionResponseHandler getResponseHandler() {
 return StartStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 System.out.println("Error Occurred: " + sw.toString());
 })
 .onComplete(() -> {
 System.out.println("=== All records stream successfully ===");
 })
 .subscriber(event -> {

```

```

 List<Result> results = ((TranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

System.out.println(results.get(0).alternatives().get(0).transcript());

 }
 }
 })
 .build();
}

private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private final InputStream inputStream;
 private static Subscription currentSubscription;

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (this.currentSubscription == null) {
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 this.currentSubscription.cancel();
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
}

public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024 * 1;
 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;
 private ExecutorService executor = Executors.newFixedThreadPool(1);
 private AtomicLong demand = new AtomicLong(0);

 SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
{
 this.subscriber = s;

```

```

 this.inputStream = inputStream;
 }

 @Override
 public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (Exception e) {
 subscriber.onError(e);
 }
 });
 }

 @Override
 public void cancel() {
 executor.shutdown();
 }

 private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;
 try {
 len = inputStream.read(audioBytes);

```

```

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
}

private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
}
}
}

```

Transcribes streaming audio from your computer's microphone.

```

public class TranscribeStreamingDemoApp {
 private static final Region REGION = Region.US_EAST_1;
 private static TranscribeStreamingAsyncClient client;

 public static void main(String[] args)
 throws URISyntaxException, ExecutionException, InterruptedException,
 LineUnavailableException {

 client = TranscribeStreamingAsyncClient.builder()
 .credentialsProvider(getCredentials())
 .region(REGION)
 .build();

 CompletableFuture<Void> result =
client.startStreamTranscription(getRequest(16_000),
 new AudioStreamPublisher(getStreamFromMic()),
 getResponseHandler());

 result.get();
 client.close();
 }
}

```

```

 }

 private static InputStream getStreamFromMic() throws LineUnavailableException {

 // Signed PCM AudioFormat with 16kHz, 16 bit sample size, mono
 int sampleRate = 16000;
 AudioFormat format = new AudioFormat(sampleRate, 16, 1, true, false);
 DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);

 if (!AudioSystem.isLineSupported(info)) {
 System.out.println("Line not supported");
 System.exit(0);
 }

 TargetDataLine line = (TargetDataLine) AudioSystem.getLine(info);
 line.open(format);
 line.start();

 InputStream audioStream = new AudioInputStream(line);
 return audioStream;
 }

 private static AwsCredentialsProvider getCredentials() {
 return DefaultCredentialsProvider.create();
 }

 private static StartStreamTranscriptionRequest getRequest(Integer
mediaSampleRateHertz) {
 return StartStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US.toString())
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .build();
 }

 private static StartStreamTranscriptionResponseHandler getResponseHandler() {
 return StartStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 })
 }

```

```

 System.out.println("Error Occurred: " + sw);
 })
 .onComplete(() -> {
 System.out.println("=== All records stream successfully ===");
 })
 .subscriber(event -> {
 List<Result> results = ((TranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

System.out.println(results.get(0).alternatives().get(0).transcript());

 }
 }
 })
 .build();
}

private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private static Subscription currentSubscription;
 private final InputStream inputStream;

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (currentSubscription == null) {
 currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 currentSubscription.cancel();
 currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
}

public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024;
 private final Subscriber<? super AudioStream> subscriber;

```

```

private final InputStream inputStream;
private final ExecutorService executor = Executors.newFixedThreadPool(1);
private final AtomicLong demand = new AtomicLong(0);

SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
{
 this.subscriber = s;
 this.inputStream = inputStream;
}

@Override
public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (Exception e) {
 subscriber.onError(e);
 }
 });
}

@Override
public void cancel() {
 executor.shutdown();
}

private ByteBuffer getNextEvent() {

```



```

 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;
 try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
 }

 private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
 }
}

```

- For API details, see the following topics in *AWS SDK for Java 2.x API Reference*.
  - [GetTranscriptionJob](#)
  - [StartTranscriptionJob](#)

## Amazon Transcribe Streaming examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Transcribe Streaming.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)
- [Scenarios](#)

## Actions

### StartMedicalStreamTranscription

The following code example shows how to use `StartMedicalStreamTranscription`.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
/*
To run this AWS code example, ensure that you have set up your development
environment, including your AWS credentials.

For information, see this documentation topic:

https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html

This code demonstrates the process of starting a medical transcription job using the
AWS Transcribe
Streaming service, including setting up the audio input stream, configuring the
transcription request,
and handling the transcription response.
*/

public class TranscribeMedicalStreamingDemoApp {
```

```
private static TranscribeStreamingAsyncClient client;

public static void main(String args[])
 throws ExecutionException, InterruptedException, LineUnavailableException {

 client = TranscribeStreamingAsyncClient.builder()
 .credentialsProvider(getCredentials())
 .build();

 CompletableFuture<Void> result =
client.startMedicalStreamTranscription(getMedicalRequest(16_000),
 new AudioStreamPublisher(getStreamFromMic()),
 getMedicalResponseHandler());

 result.get();
 client.close();
}

private static InputStream getStreamFromMic() throws LineUnavailableException {

 // Signed PCM AudioFormat with 16kHz, 16 bit sample size, mono
 int sampleRate = 16000;
 AudioFormat format = new AudioFormat(sampleRate, 16, 1, true, false);
 DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);

 if (!AudioSystem.isLineSupported(info)) {
 System.out.println("Line not supported");
 throw new LineUnavailableException("The audio system microphone line is
not supported.");
 }

 TargetDataLine line = (TargetDataLine) AudioSystem.getLine(info);
 line.open(format);
 line.start();

 InputStream audioStream = new AudioInputStream(line);
 return audioStream;
}

private static AwsCredentialsProvider getCredentials() {
 return DefaultCredentialsProvider.create();
}
```

```

 private static StartMedicalStreamTranscriptionRequest getMedicalRequest(Integer
mediaSampleRateHertz) {
 return StartMedicalStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US.toString()) // For medical
transcription, EN_US is typically used.
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .specialty(Specialty.PRIMARYCARE) // Specify the medical specialty.
 .type(Type.CONVERSATION) // Set the type as CONVERSATION or DICTATION.
 .build();
 }

 private static StartMedicalStreamTranscriptionResponseHandler
getMedicalResponseHandler() {
 return StartMedicalStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 System.out.println("Error Occurred: " + sw.toString());
 })
 .onComplete(() -> {
 System.out.println("=== All records streamed successfully ===");
 })
 .subscriber(event -> {
 List<MedicalResult> results = ((MedicalTranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

 System.out.println(results.get(0).alternatives().get(0).transcript());
 }
 }
 })
 .build();
 }

 private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private final InputStream inputStream;
 private static Subscription currentSubscription;
 }

```

```

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (this.currentSubscription == null) {
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 this.currentSubscription.cancel();
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
}

public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024 * 1;
 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;
 private ExecutorService executor = Executors.newFixedThreadPool(1);
 private AtomicLong demand = new AtomicLong(0);

 SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
 {
 this.subscriber = s;
 this.inputStream = inputStream;
 }

 @Override
 public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);
 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {

```

```

 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
} catch (Exception e) {
 subscriber.onError(e);
}
});
}

@Override
public void cancel() {
 executor.shutdown();
}

private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;
 try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
}

private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
}

```

```
}
}
```

- For API details, see [StartMedicalStreamTranscription](#) in *AWS SDK for Java 2.x API Reference*.

## StartStreamTranscription

The following code example shows how to use `StartStreamTranscription`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
public class TranscribeStreamingDemoApp {
 private static final Region REGION = Region.US_EAST_1;
 private static TranscribeStreamingAsyncClient client;

 public static void main(String[] args)
 throws URISyntaxException, ExecutionException, InterruptedException,
 LineUnavailableException {

 client = TranscribeStreamingAsyncClient.builder()
 .credentialsProvider(getCredentials())
 .region(REGION)
 .build();

 CompletableFuture<Void> result =
client.startStreamTranscription(getRequest(16_000),
 new AudioStreamPublisher(getStreamFromMic()),
 getResponseHandler());

 result.get();
 client.close();
 }

 private static InputStream getStreamFromMic() throws LineUnavailableException {
```

```

 // Signed PCM AudioFormat with 16kHz, 16 bit sample size, mono
 int sampleRate = 16000;
 AudioFormat format = new AudioFormat(sampleRate, 16, 1, true, false);
 DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);

 if (!AudioSystem.isLineSupported(info)) {
 System.out.println("Line not supported");
 System.exit(0);
 }

 TargetDataLine line = (TargetDataLine) AudioSystem.getLine(info);
 line.open(format);
 line.start();

 InputStream audioStream = new AudioInputStream(line);
 return audioStream;
 }

 private static AwsCredentialsProvider getCredentials() {
 return DefaultCredentialsProvider.create();
 }

 private static StartStreamTranscriptionRequest getRequest(Integer
mediaSampleRateHertz) {
 return StartStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US.toString())
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .build();
 }

 private static StartStreamTranscriptionResponseHandler getResponseHandler() {
 return StartStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 System.out.println("Error Occurred: " + sw);
 })
 .onComplete(() -> {
 System.out.println("=== All records stream successfully ===");
 })
 }

```



```

 })
 .subscriber(event -> {
 List<Result> results = ((TranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

System.out.println(results.get(0).alternatives().get(0).transcript());

 }
 }
 })
 .build();
 }

private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private static Subscription currentSubscription;
 private final InputStream inputStream;

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (currentSubscription == null) {
 currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 currentSubscription.cancel();
 currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
}

public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024;
 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;
 private final ExecutorService executor = Executors.newFixedThreadPool(1);
 private final AtomicLong demand = new AtomicLong(0);

```

```

SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
{
 this.subscriber = s;
 this.inputStream = inputStream;
}

@Override
public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (Exception e) {
 subscriber.onError(e);
 }
 });
}

@Override
public void cancel() {
 executor.shutdown();
}

private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;

```

```
 try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
 }

 private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
 }
}
```

- For API details, see [StartStreamTranscription](#) in *AWS SDK for Java 2.x API Reference*.

## Scenarios

### Transcribe an audio file

The following code example shows how to generate a transcription of a source audio file using Amazon Transcribe streaming.

#### SDK for Java 2.x

##### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

/**
 * To run this AWS code example, ensure that you have set up your development
 * environment, including your AWS credentials.
 *
 * For information, see this documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */

public class TranscribeStreamingDemoFile {
 private static final Region REGION = Region.US_EAST_1;
 private static TranscribeStreamingAsyncClient client;

 public static void main(String args[]) throws ExecutionException,
 InterruptedException {

 final String USAGE = "\n" +
 "Usage:\n" +
 " <file> \n\n" +
 "Where:\n" +
 " file - the location of a PCM file to transcribe. In this
example, ensure the PCM file is 16 hertz (Hz). \n";

 if (args.length != 1) {
 System.out.println(USAGE);
 System.exit(1);
 }

 String file = args[0];
 client = TranscribeStreamingAsyncClient.builder()
 .region(REGION)
 .build();

 CompletableFuture<Void> result =
client.startStreamTranscription(getRequest(16_000),
 new AudioStreamPublisher(getStreamFromFile(file)),
 getResponseHandler());

 result.get();
 client.close();
 }

 private static InputStream getStreamFromFile(String file) {

```

```

 try {
 File inputFile = new File(file);
 InputStream audioStream = new FileInputStream(inputFile);
 return audioStream;

 } catch (FileNotFoundException e) {
 throw new RuntimeException(e);
 }
 }

 private static StartStreamTranscriptionRequest getRequest(Integer
mediaSampleRateHertz) {
 return StartStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US)
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .build();
 }

 private static StartStreamTranscriptionResponseHandler getResponseHandler() {
 return StartStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 System.out.println("Error Occurred: " + sw.toString());
 })
 .onComplete(() -> {
 System.out.println("=== All records stream successfully ===");
 })
 .subscriber(event -> {
 List<Result> results = ((TranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

 System.out.println(results.get(0).alternatives().get(0).transcript());
 }
 }
 })
 }

```

```

 .build();
 }

 private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private final InputStream inputStream;
 private static Subscription currentSubscription;

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (this.currentSubscription == null) {
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 this.currentSubscription.cancel();
 this.currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
 }

 public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024 * 1;
 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;
 private ExecutorService executor = Executors.newFixedThreadPool(1);
 private AtomicLong demand = new AtomicLong(0);

 SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
 {
 this.subscriber = s;
 this.inputStream = inputStream;
 }

 @Override
 public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }
 }
 }

```

```

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (Exception e) {
 subscriber.onError(e);
 }
 });
 }

 @Override
 public void cancel() {
 executor.shutdown();
 }

 private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;
 try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
 }

```

```

 }

 private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
 }
}

```

- For API details, see [StartStreamTranscription](#) in *AWS SDK for Java 2.x API Reference*.

## Transcribe audio from a microphone

The following code example shows how to generate a transcription from a microphone using Amazon Transcribe streaming.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

public class TranscribeStreamingDemoApp {
 private static final Region REGION = Region.US_EAST_1;
 private static TranscribeStreamingAsyncClient client;

 public static void main(String[] args)
 throws URISyntaxException, ExecutionException, InterruptedException,
 LineUnavailableException {

 client = TranscribeStreamingAsyncClient.builder()
 .credentialsProvider(getCredentials())
 .region(REGION)
 .build();

 CompletableFuture<Void> result =
 client.startStreamTranscription(getRequest(16_000),
 new AudioStreamPublisher(getStreamFromMic()),

```



```

 getResponseHandler());

 result.get();
 client.close();
}

private static InputStream getStreamFromMic() throws LineUnavailableException {

 // Signed PCM AudioFormat with 16kHz, 16 bit sample size, mono
 int sampleRate = 16000;
 AudioFormat format = new AudioFormat(sampleRate, 16, 1, true, false);
 DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);

 if (!AudioSystem.isLineSupported(info)) {
 System.out.println("Line not supported");
 System.exit(0);
 }

 TargetDataLine line = (TargetDataLine) AudioSystem.getLine(info);
 line.open(format);
 line.start();

 InputStream audioStream = new AudioInputStream(line);
 return audioStream;
}

private static AwsCredentialsProvider getCredentials() {
 return DefaultCredentialsProvider.create();
}

private static StartStreamTranscriptionRequest getRequest(Integer
mediaSampleRateHertz) {
 return StartStreamTranscriptionRequest.builder()
 .languageCode(LanguageCode.EN_US.toString())
 .mediaEncoding(MediaEncoding.PCM)
 .mediaSampleRateHertz(mediaSampleRateHertz)
 .build();
}

private static StartStreamTranscriptionResponseHandler getResponseHandler() {
 return StartStreamTranscriptionResponseHandler.builder()
 .onResponse(r -> {
 System.out.println("Received Initial response");
 })
}

```

```

 .onError(e -> {
 System.out.println(e.getMessage());
 StringWriter sw = new StringWriter();
 e.printStackTrace(new PrintWriter(sw));
 System.out.println("Error Occurred: " + sw);
 })
 .onComplete(() -> {
 System.out.println("=== All records stream successfully ===");
 })
 .subscriber(event -> {
 List<Result> results = ((TranscriptEvent)
event).transcript().results();
 if (results.size() > 0) {
 if (!
results.get(0).alternatives().get(0).transcript().isEmpty()) {

System.out.println(results.get(0).alternatives().get(0).transcript());

 }
 }
 })
 .build();
 }

 private static class AudioStreamPublisher implements Publisher<AudioStream> {
 private static Subscription currentSubscription;
 private final InputStream inputStream;

 private AudioStreamPublisher(InputStream inputStream) {
 this.inputStream = inputStream;
 }

 @Override
 public void subscribe(Subscriber<? super AudioStream> s) {

 if (currentSubscription == null) {
 currentSubscription = new SubscriptionImpl(s, inputStream);
 } else {
 currentSubscription.cancel();
 currentSubscription = new SubscriptionImpl(s, inputStream);
 }
 s.onSubscribe(currentSubscription);
 }
 }
}

```

```

public static class SubscriptionImpl implements Subscription {
 private static final int CHUNK_SIZE_IN_BYTES = 1024;
 private final Subscriber<? super AudioStream> subscriber;
 private final InputStream inputStream;
 private final ExecutorService executor = Executors.newFixedThreadPool(1);
 private final AtomicLong demand = new AtomicLong(0);

 SubscriptionImpl(Subscriber<? super AudioStream> s, InputStream inputStream)
 {
 this.subscriber = s;
 this.inputStream = inputStream;
 }

 @Override
 public void request(long n) {
 if (n <= 0) {
 subscriber.onError(new IllegalArgumentException("Demand must be
positive"));
 }

 demand.getAndAdd(n);

 executor.submit(() -> {
 try {
 do {
 ByteBuffer audioBuffer = getNextEvent();
 if (audioBuffer.remaining() > 0) {
 AudioEvent audioEvent =
audioEventFromBuffer(audioBuffer);
 subscriber.onNext(audioEvent);
 } else {
 subscriber.onComplete();
 break;
 }
 } while (demand.decrementAndGet() > 0);
 } catch (Exception e) {
 subscriber.onError(e);
 }
 });
 }

 @Override
 public void cancel() {

```

```

 executor.shutdown();
 }

 private ByteBuffer getNextEvent() {
 ByteBuffer audioBuffer = null;
 byte[] audioBytes = new byte[CHUNK_SIZE_IN_BYTES];

 int len = 0;
 try {
 len = inputStream.read(audioBytes);

 if (len <= 0) {
 audioBuffer = ByteBuffer.allocate(0);
 } else {
 audioBuffer = ByteBuffer.wrap(audioBytes, 0, len);
 }
 } catch (IOException e) {
 throw new UncheckedIOException(e);
 }

 return audioBuffer;
 }

 private AudioEvent audioEventFromBuffer(ByteBuffer bb) {
 return AudioEvent.builder()
 .audioChunk(SdkBytes.fromByteBuffer(bb))
 .build();
 }
}

```

- For API details, see [StartStreamTranscription](#) in *AWS SDK for Java 2.x API Reference*.

## Amazon Translate examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with Amazon Translate.

*Scenarios* are code examples that show you how to accomplish specific tasks by calling multiple functions within a service or combined with other AWS services.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Scenarios](#)

## Scenarios

### Building an Amazon Lex chatbot

The following code example shows how to create a chatbot to engage your website visitors.

#### SDK for Java 2.x

Shows how to use the Amazon Lex API to create a Chatbot within a web application to engage your web site visitors.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

#### Services used in this example

- Amazon Comprehend
- Amazon Lex
- Amazon Translate

### Building an Amazon SNS application

The following code example shows how to create an application that has subscription and publish functionality and translates messages.

#### SDK for Java 2.x

Shows how to use the Amazon Simple Notification Service Java API to create a web application that has subscription and publish functionality. In addition, this example application also translates messages.

For complete source code and instructions on how to set up and run, see the full example on [GitHub](#).

For complete source code and instructions on how to set up and run the example that uses the Java Async API, see the full example on [GitHub](#).

### Services used in this example

- Amazon SNS
- Amazon Translate

## Create an application to analyze customer feedback

The following code example shows how to create an application that analyzes customer comment cards, translates them from their original language, determines their sentiment, and generates an audio file from the translated text.

### SDK for Java 2.x

This example application analyzes and stores customer feedback cards. Specifically, it fulfills the need of a fictitious hotel in New York City. The hotel receives feedback from guests in various languages in the form of physical comment cards. That feedback is uploaded into the app through a web client. After an image of a comment card is uploaded, the following steps occur:

- Text is extracted from the image using Amazon Textract.
- Amazon Comprehend determines the sentiment of the extracted text and its language.
- The extracted text is translated to English using Amazon Translate.
- Amazon Polly synthesizes an audio file from the extracted text.

The full app can be deployed with the AWS CDK. For source code and deployment instructions, see the project in [GitHub](#).

### Services used in this example

- Amazon Comprehend
- Lambda
- Amazon Polly
- Amazon Textract
- Amazon Translate

# X-Ray examples using SDK for Java 2.x

The following code examples show you how to perform actions and implement common scenarios by using the AWS SDK for Java 2.x with X-Ray.

*Actions* are code excerpts from larger programs and must be run in context. While actions show you how to call individual service functions, you can see actions in context in their related scenarios.

Each example includes a link to the complete source code, where you can find instructions on how to set up and run the code in context.

## Topics

- [Actions](#)

## Actions

### CreateGroup

The following code example shows how to use CreateGroup.

#### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.CreateGroupRequest;
import software.amazon.awssdk.services.xray.model.CreateGroupResponse;
import software.amazon.awssdk.services.xray.model.XRayException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 */
```

```

* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class CreateGroup {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <groupName>

 Where:
 groupName - The name of the group to create\s

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();

 createNewGroup(xRayClient, groupName);
 }

 public static void createNewGroup(XRayClient xRayClient, String groupName) {
 try {
 CreateGroupRequest groupRequest = CreateGroupRequest.builder()
 .filterExpression("fault = true AND http.url CONTAINS \"example/
game\" AND responsetime >= 5")
 .groupName(groupName)
 .build();

 CreateGroupResponse groupResponse =
xRayClient.createGroup(groupRequest);
 System.out.println("The Group ARN is " +
groupResponse.group().groupARN());

 } catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```



```
}
}
```

- For API details, see [CreateGroup](#) in *AWS SDK for Java 2.x API Reference*.

## CreateSamplingRule

The following code example shows how to use `CreateSamplingRule`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.CreateSamplingRuleRequest;
import software.amazon.awssdk.services.xray.model.SamplingRule;
import software.amazon.awssdk.services.xray.model.XRayException;
import software.amazon.awssdk.services.xray.model.CreateSamplingRuleResponse;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class CreateSamplingRule {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <ruleName>

 Where:
 ruleName - The name of the rule\s
```

```

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String ruleName = args[0];
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();

 createRule(xRayClient, ruleName);
}

public static void createRule(XRayClient xRayClient, String ruleName) {
 try {

 SamplingRule rule = SamplingRule.builder()
 .ruleName(ruleName)
 .priority(1)
 .httpMethod("")
 .serviceType("")
 .serviceName("")
 .urlPath("")
 .version(1)
 .host("")
 .resourceARN("")
 .build();

 CreateSamplingRuleRequest ruleRequest =
 CreateSamplingRuleRequest.builder()
 .samplingRule(rule)
 .build();

 CreateSamplingRuleResponse ruleResponse =
 xRayClient.createSamplingRule(ruleRequest);
 System.out.println(
 "The ARN of the new rule is " +
 ruleResponse.samplingRuleRecord().samplingRule().ruleARN());

 } catch (XRayException e) {
 System.err.println(e.getMessage());
 }
}

```

```

 System.exit(1);
 }
}
}

```

- For API details, see [CreateSamplingRule](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteGroup

The following code example shows how to use DeleteGroup.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.DeleteGroupRequest;
import software.amazon.awssdk.services.xray.model.XRayException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteGroup {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <groupName>

 Where:
 groupName - The name of the group to delete\s

```

```

 """;

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();

 deleteSpecificGroup(xRayClient, groupName);
}

public static void deleteSpecificGroup(XRayClient xRayClient, String groupName)
{
 try {
 DeleteGroupRequest groupRequest = DeleteGroupRequest.builder()
 .groupName(groupName)
 .build();

 xRayClient.deleteGroup(groupRequest);
 System.out.println(groupName + " was deleted!");

 } catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
}

```

- For API details, see [DeleteGroup](#) in *AWS SDK for Java 2.x API Reference*.

## DeleteSamplingRule

The following code example shows how to use DeleteSamplingRule.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.DeleteSamplingRuleRequest;
import software.amazon.awssdk.services.xray.model.XRayException;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class DeleteSamplingRule {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <ruleName>

 Where:
 ruleName - The name of the rule to delete\s

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String ruleName = args[0];
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();
```

```
 deleteRule(xRayClient, ruleName);
 }

 public static void deleteRule(XRayClient xRayClient, String ruleName) {
 try {
 DeleteSamplingRuleRequest ruleRequest =
DeleteSamplingRuleRequest.builder()
 .ruleName(ruleName)
 .build();

 xRayClient.deleteSamplingRule(ruleRequest);
 } catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}
```

- For API details, see [DeleteSamplingRule](#) in *AWS SDK for Java 2.x API Reference*.

## GetGroups

The following code example shows how to use `GetGroups`.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.GetGroupsResponse;
import software.amazon.awssdk.services.xray.model.GroupSummary;
import software.amazon.awssdk.services.xray.model.XRayException;
import java.util.List;
```

```

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetGroups {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();

 getAllGroups(xRayClient);
 }

 public static void getAllGroups(XRayClient xRayClient) {
 try {
 GetGroupsResponse groupsResponse = xRayClient.getGroups();
 List<GroupSummary> groups = groupsResponse.groups();
 for (GroupSummary group : groups) {
 System.out.println("The AWS XRay group name is " +
group.groupName());
 }

 } catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
 }
}

```

- For API details, see [GetGroups](#) in *AWS SDK for Java 2.x API Reference*.

## GetSamplingRules

The following code example shows how to use `GetSamplingRules`.

## SDK for Java 2.x

### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```
import software.amazon.awssdk.auth.credentials.ProfileCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.GetSamplingRulesResponse;
import software.amazon.awssdk.services.xray.model.SamplingRuleRecord;
import software.amazon.awssdk.services.xray.model.XRayException;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 *
 * For more information, see the following documentation topic:
 *
 * https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
 */
public class GetSamplingRules {
 public static void main(String[] args) {
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .credentialsProvider(ProfileCredentialsProvider.create())
 .build();

 getRules(xRayClient);
 }

 public static void getRules(XRayClient xRayClient) {
 try {
 GetSamplingRulesResponse response = xRayClient.getSamplingRules(r ->
r.build());
 List<SamplingRuleRecord> records = response.samplingRuleRecords();

 for (SamplingRuleRecord record : records) {
```



```

 System.out.println("The rule name is: " +
record.samplingRule().ruleName());
 System.out.println("The related service is: " +
record.samplingRule().serviceName());
 }
} catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
}
}
}

```

- For API details, see [GetSamplingRules](#) in *AWS SDK for Java 2.x API Reference*.

## GetServiceGraph

The following code example shows how to use GetServiceGraph.

### SDK for Java 2.x

#### Note

There's more on GitHub. Find the complete example and learn how to set up and run in the [AWS Code Examples Repository](#).

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.xray.XRayClient;
import software.amazon.awssdk.services.xray.model.GetServiceGraphRequest;
import software.amazon.awssdk.services.xray.model.GetServiceGraphResponse;
import software.amazon.awssdk.services.xray.model.Service;
import software.amazon.awssdk.services.xray.model.XRayException;
import java.time.LocalDateTime;
import java.time.Instant;
import java.time.ZoneId;
import java.util.List;

/**
 * Before running this Java V2 code example, set up your development
 * environment, including your credentials.
 */

```

```

* For more information, see the following documentation topic:
*
* https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/get-started.html
*/
public class GetServiceGraph {
 public static void main(String[] args) {
 final String usage = ""

 Usage: <groupName>

 Where:
 groupName - The name of a group based on which you want to
generate a graph.

 "";

 if (args.length != 1) {
 System.out.println(usage);
 System.exit(1);
 }

 String groupName = args[0];
 Region region = Region.US_EAST_1;
 XRayClient xRayClient = XRayClient.builder()
 .region(region)
 .build();
 getGraph(xRayClient, groupName);
 }

 public static void getGraph(XRayClient xRayClient, String groupName) {
 try {
 // The Instant values have to be 6 hours apart.
 LocalDateTime localDateTime =
LocalDateTime.parse("2022-03-09T06:00:00");
 Instant start = localDateTime.atZone(ZoneId.of("America/
New_York")).toInstant();

 LocalDateTime localDateTime2 =
LocalDateTime.parse("2022-03-09T12:00:00");
 Instant end = localDateTime2.atZone(ZoneId.of("America/
New_York")).toInstant();

 GetServiceGraphRequest getServiceGraphRequest =
GetServiceGraphRequest.builder()

```

```
 .groupName(groupName)
 .startTime(start)
 .endTime(end)
 .build();

 GetServiceGraphResponse graphResponse =
xRayClient.getServiceGraph(getServiceGraphRequest);
 List<Service> services = graphResponse.services();
 for (Service service : services) {
 System.out.println("The name of the service is " + service.name());
 }

 } catch (XRayException e) {
 System.err.println(e.getMessage());
 System.exit(1);
 }
}
```

- For API details, see [GetServiceGraph](#) in *AWS SDK for Java 2.x API Reference*.

# Migrate from version 1.x to 2.x of the AWS SDK for Java

The AWS SDK for Java 2.x is a major rewrite of the 1.x code base built on top of Java 8+. It includes many updates, such as improved consistency, ease of use, and strongly enforced immutability. This section describes the major features that are new in version 2.x, and provides guidance on how to migrate your code to version 2.x from 1.x.

## Topics

- [What's new in version 2](#)
- [Find applications using AWS SDK for Java 1.x clients](#)
- [How to migrate your code from AWS SDK for Java 1.x to 2.x](#)
- [What's different between the AWS SDK for Java 1.x and 2.x](#)
- [Use the SDK for Java 1.x and 2.x side-by-side](#)

## What's new in version 2

- You can configure your own HTTP clients. See [HTTP transport configuration](#).
- Async clients feature non-blocking I/O support and return `CompletableFuture` objects. See [Asynchronous programming](#).
- Operations that return multiple pages have autopaginated responses. This way, you can focus your code on what to do with the response, without the need to check for and get subsequent pages. See [Pagination](#).
- SDK start time performance for AWS Lambda functions is improved. See [SDK start time performance improvements](#).
- Version 2.x supports a new shorthand method for creating requests.

### Example

```
dynamoDbClient.putItem(request -> request.tableName(TABLE))
```

For more details about the new features and to see specific code examples, refer to the other sections of this guide.

- [Quick Start](#)

- [Setting up](#)
- [Code examples for the AWS SDK for Java 2.x](#)
- [Use the SDK](#)
- [Security for the AWS SDK for Java](#)

## Find applications using AWS SDK for Java 1.x clients

Before migrating to the AWS SDK for Java 2.x, you need to identify which applications in your environment use SDK for Java 1.x clients. You can use CloudTrail logs to trace SDK usage, search application logs for deprecation warnings, inspect your source code and build configurations, or examine your deployable Java artifacts. Use whichever methods are available in your environment.

### Use CloudWatch Logs Insights to find applications with 1.x clients

Amazon CloudWatch Logs Insights lets you query CloudTrail events delivered to a CloudWatch Logs log group. Use this method to identify SDK versions from the *userAgent* field in CloudTrail records. You need a CloudTrail trail with **CloudWatch Logs** enabled. If you don't have one, see [Creating a trail with the CloudTrail console](#) and [Sending events to CloudWatch Logs](#).

In the CloudWatch console, navigate to **Logs Insights** under **Logs**. Select your CloudTrail log group and set the time range to cover the period you want to analyze. For details on using Logs Insights, see [Analyzing log data with CloudWatch Logs Insights](#).

The following sample queries find all requests from user applications and third-party tools made with SDK for Java 1.x. Logs Insights supports both Logs Insights QL and SQL. The QL query uses the log group selected in the console. The SQL query requires you to specify the log group name in the FROM clause.

Logs Insights QL query:

```
fields userIdentity.type, userIdentity.arn, eventSource, awsRegion,
 eventName, eventType, eventTime, userAgent, sourceIPAddress
| filter userAgent like /aws-sdk-java\1\./
| filter userAgent not like /aws-internal\//
| sort eventTime desc
```

SQL query:

```
SELECT `userIdentity.type`,
 `userIdentity.arn`,
 eventSource,
 awsRegion,
 eventName,
 eventType,
 eventTime,
 userAgent,
 sourceIPAddress
FROM `sample-cloudtrail-log-group`
WHERE userAgent LIKE '%aws-sdk-java/1.%'
 AND userAgent NOT LIKE '%aws-internal/%'
ORDER BY eventTime DESC
```

Replace *sample-cloudtrail-log-group* with the name of your CloudTrail log group.

To export the results, choose **Export results**. The following example shows a single event in JSON format:

```
{
 "userIdentity.type": "AssumedRole",
 "userIdentity.arn": "arn:aws:sts::123456789012:assumed-role/Admin/Alice",
 "eventSource": "dynamodb.amazonaws.com",
 "awsRegion": "us-east-1",
 "eventName": "ListTables",
 "eventType": "AwsApiCall",
 "eventTime": "2025-07-01T22:02:23Z",
 "userAgent": "aws-sdk-java/1.12.746 Linux/5.10.240 OpenJDK/11.0.25+9-LTS ...",
 "sourceIPAddress": "12.345.6.78"
}
```

In this example, a DynamoDB ListTables request was made at 2025-07-01T22:02:23 (UTC) from IP address 12.345.6.78 with the credentials of the assumed role Admin/Alice. The *userAgent* field shows that the request was made using SDK for Java version 1.12.746 on a Linux system with JDK 11.

For a description of the fields in the AWS CloudTrail event record, see [CloudTrail record contents](#).

If CloudTrail is not enabled in your account, contact your organization's AWS account administrator to enable it, or use one of the alternative methods described in the following sections.

CloudWatch Logs Insights charges per query based on the amount of data scanned. If you created a trail specifically for this investigation, consider stopping or deleting it to avoid ongoing charges. For current pricing, see [Amazon CloudWatch Pricing](#).

## Search application warning-level logs for SDK deprecation

Starting with version 1.12.767 (released on July 30, 2024), the AWS SDK for Java 1.x emits a deprecation warning at application startup. You can search your application logs for this warning to identify which applications and hosts are using SDK for Java 1.x.

The exact wording of the warning depends on the SDK version:

- Versions 1.12.767 through 1.12.796:

```
WARNING: The AWS SDK for Java 1.x entered maintenance mode starting July
31, 2024 and will reach end of support on December 31, 2025...
```

- Versions 1.12.797 and higher:

```
WARNING: The AWS SDK for Java 1.x reached end of support on December 31,
2025...
```

The trailing . . . indicates that the warning message continues with additional text. You can search for the common prefix `The AWS SDK for Java 1.x` to find either version of the warning.

The following example demonstrates searching for this warning using `grep`:

```
grep -r "The AWS SDK for Java 1.x" /path/to/your/application/logs/
```

If the warning is found, the `grep` command prints the matching log lines. If no warning is found, either your application is not using SDK for Java 1.x, or it uses a version earlier than 1.12.767. In that case, use one of the other methods described in this document.

## Search source code and dependencies

You can search your codebase and build configuration files for references to the AWS SDK for Java 1.x. The key identifier is the `com.amazonaws` group ID, which is used by all SDK for Java 1.x artifacts.

The following examples demonstrate using `grep` to search for `com.amazonaws` references across common Java project files.

**Example: Search Java source files for SDK for Java 1.x imports (run from project root directory)**

```
grep -r "import com.amazonaws" --include="*.java" .
```

Example output:

```
src/main/java/com/example/App.java:import com.amazonaws.services.s3.AmazonS3;
```

**Note**

The `com.amazonaws` package is also used by libraries that are not part of the SDK for Java 1.x, such as `aws-lambda-java-core`. To confirm an import is from SDK for Java 1.x, check that the corresponding artifact ID in your `pom.xml`, `build.gradle`, or dependency management configuration starts with `aws-java-sdk-`.

**Example: Search Maven `pom.xml` files for SDK for Java 1.x dependencies (run from project root directory)**

```
grep -r "com.amazonaws" --include="pom.xml" .
```

Example output:

```
pom.xml: <groupId>com.amazonaws</groupId>
```

**Example: Search Gradle build files for SDK for Java 1.x dependencies (run from project root directory)**

```
grep -r "com.amazonaws:aws-java-sdk" --include="*.gradle" .
```

Example output:

```
build.gradle: implementation 'com.amazonaws:aws-java-sdk-s3:1.12.xxx'
```

The preceding `grep` commands identify SDK for Java 1.x references declared directly in your source and build files. However, your application might also depend on SDK for Java 1.x transitively — through a third-party library that itself depends on the SDK. Use your build tool's dependency tree



to find both direct and transitive SDK for Java 1.x dependencies. Choose the example that matches your build system.

### Example: Use Maven to find all transitive SDK for Java 1.x dependencies (run from project root directory)

```
mvn dependency:tree -Dincludes=com.amazonaws
```

Example output:

```
[INFO] com.example:my-application:jar:1.0-SNAPSHOT
[INFO] +- com.amazonaws:aws-java-sdk-s3:jar:1.12.746:compile
[INFO] | \- com.amazonaws:aws-java-sdk-core:jar:1.12.746:compile
[INFO] \- some.thirdparty:library:jar:2.3.1:compile
[INFO] \- com.amazonaws:aws-java-sdk-dynamodb:jar:1.12.600:compile
```

The `-Dincludes=com.amazonaws` flag filters the tree to show only SDK for Java 1.x artifacts. In this example, `aws-java-sdk-s3` is a direct dependency, but `aws-java-sdk-dynamodb` is a transitive dependency brought in by `some.thirdparty:library`.

### Example: Use Gradle to find all transitive SDK for Java 1.x dependencies (run from project root directory)

```
gradle dependencies --configuration runtimeClasspath | grep "com.amazonaws"
```

Example output:

```
+--- com.amazonaws:aws-java-sdk-s3:1.12.746
| \--- com.amazonaws:aws-java-sdk-core:1.12.746
\--- com.amazonaws:aws-java-sdk-dynamodb:1.12.600
```

Gradle does not have a built-in dependency filter equivalent to Maven's `-Dincludes`, so piping through `grep` is the simplest approach.

## Inspect deployable Java artifacts

You can inspect your deployable Java artifacts (JARs, WARs, or EARs) to confirm whether the AWS SDK for Java 1.x is packaged with your application. Java archive files are ZIP-format files. To determine if SDK for Java 1.x is present, look for the file `com/amazonaws/sdk/`

`versionInfo.properties` inside the archive. This file is included in the `aws-java-sdk-core` module and contains the SDK version number.

## Quick check with the `jar` command

For uber-jars where all dependency classes are merged at the top level, list the archive contents and search for the version file:

In the following examples, replace `myapp.jar` with the path to your application's JAR file.

```
jar -tf myapp.jar | grep 'versionInfo.properties'
```

If the SDK is present, the output is:

```
com/amazonaws/sdk/versionInfo.properties
```

If the `jar` command is not available in your environment (for example, JRE-only or minimal container images), you can use `unzip -l` instead:

```
unzip -l myapp.jar | grep 'versionInfo.properties'
```

To print the version:

```
unzip -p myapp.jar com/amazonaws/sdk/versionInfo.properties
```

Example output:

```
platform=java
version=1.12.xxx
```

### Note

The preceding commands only search top-level entries in uber-jars. They will not find SDK classes in thin JARs (where dependencies are external) or inside nested JARs (such as those in WARs, EARs, or Lambda packages under `lib/` or `WEB-INF/lib/`). For thin JARs, check your build configuration (`pom.xml`, `build.gradle`) or dependency tree instead. For nested JARs, search the bundled JARs using a tool that can read ZIP archives recursively without extracting to disk.

## Use CloudTrail Lake to find applications with 1.x clients

### Important

AWS CloudTrail Lake is no longer open to new customers starting May 31, 2026. Existing customers can continue to use the service. For more information, see [CloudTrail Lake availability change](#). For an alternative approach, see [the section called "Use CloudWatch Logs Insights to find applications with 1.x clients"](#).

AWS CloudTrail Lake lets you query events recorded by CloudTrail. Follow these steps to create a data lake that identifies SDK versions used by your applications:

1. Create a CloudTrail data lake. Refer to the [User guide](#) to create an event data store.
2. After you create the data store, examine the record contents. The record body contains fields that determine the requested action, timing, and location. For details, refer to the [User guide for CloudTrail record contents](#).
3. Run queries against your data. Follow the [User Guide to query and save query results](#).

The *userAgent* field in each record contains the SDK version that made the request. Use this field to identify applications using SDK for Java 1.x.

The following sample query finds all requests from user applications and third-party tools made with SDK for Java 1.x starting from June 17, 2025, for an EventDatastoreID sample-Data-Store-Id:

```
select userIdentity, eventSource, awsRegion,
 eventName, eventType, eventTime, userAgent,
 requestParameters, sourceIPAddress
from sample-Data-Store-Id
where eventTime > '2025-06-17 00:00:00'
and userAgent like '%aws-sdk-java/1.%'
and userAgent not like '%aws-internal/%'
order by eventTime desc
```

An example of event content in the query result looks like this:

```
{
```

```
"userIdentity": "{
 "type": "IAMUser",
 "principalId": "AIDAJ45Q7YFFAREXAMPLE",
 "arn": "arn:aws:iam::123456789012:user/Alice",
 "accountId": "123456789012",
 "accessKeyId": "",
 "userName": "Alice"
}",
"eventSource": "dynamodb.amazonaws.com",
"awsRegion": "us-west-2",
"eventName": "ListTables",
"eventType": "AwsApiCall",
"eventTime": "2025-07-01 02:23:52.000",
"userAgent": "aws-sdk-java/1.12.746 Linux/5.10.240 OpenJDK/11.0.25+9-LTS ...",
"requestParameters": "",
"sourceIPAddress": "12.345.6.78"
}
```

You can use this information to help determine when and where the request was made.

In the example, a DynamoDB `ListTables` request was made at 2025-07-01 02:23:52 (UTC) from the IP Address 12.345.6.78 with the credentials of the IAM user named Alice. The *userAgent* field's value shows that the request was made using the AWS SDK for Java version 1.12.746 from a Linux system with JDK 11.

For a description of the fields in the AWS CloudTrail event record, see [CloudTrail record contents for management, data, and network activity events](#).

If CloudTrail is not enabled in your account, contact your organization's AWS account administrator to enable it, or use one of the alternative methods described in the preceding sections.

CloudTrail Lake charges for data ingested and data scanned per query. To minimize costs, filter queries to specific time ranges and regions. For current pricing, see [AWS CloudTrail Pricing](#).

## How to migrate your code from AWS SDK for Java 1.x to 2.x

You can migrate your existing SDK for Java 1.x applications in a couple ways.

1. Automated approach by using the [migration tool](#).
2. [Manual approach](#) by incrementally replacing 1.x imports with 2.x imports.

We recommend that you start by using the migration tool. It automates much of the routine, replacement work from 1.x to 2.x code.

Since the tool [doesn't migrate all features](#), you'll need to search for remaining v1 code after running the tool. When you find code that the tool didn't migrate, follow the [step-by-step instructions](#) (manual approach) and use the [migration guide articles](#) to finish the migration.

## Topics

- [AWS SDK for Java migration tool](#)
- [Migration step-by-step instructions with example](#)

## AWS SDK for Java migration tool

The AWS SDK for Java provides a migration tool that helps automate the migration of SDK for Java 1.x (V1) code to 2.x (V2). The tool uses [OpenRewrite](#)—an open source, source code refactoring tool—to perform the migration. OpenRewrite uses code modification rules (called "recipes") to automatically update your source code from V1 to V2 syntax and patterns.

The tool supports code modification rules for SDK service clients and the [S3 Transfer Manager](#) high-level library. Code modification rules for other high-level APIs such as V1's [DynamoDBMapper](#) to V2's [DynamoDB Enhanced Client API](#) are not supported.

For more details on limitations, see the [end of this page](#). For detailed examples of common unsupported code patterns with manual migration steps, see [Unsupported code patterns](#).

## Use the migration tool

### Migrate a Maven project

Follow the instructions below to migrate your SDK for Java 1.x Maven-based project by using the [OpenRewrite Maven plugin](#) tool.

1. Navigate to your Maven project's root directory

Open a terminal (command line) window and navigate to the root directory of your Maven-based application.

2. Run the plugin's `rewrite-maven-plugin` command

You can choose from two modes (Maven goals): `dryRun` and `run`.

## dryRun mode

In the `dryRun` mode, the plugin generates diff logs in the console output and a patch file named `rewrite.patch` in the `target/rewrite` folder. This mode allows you to preview the changes that would be made, since no changes are made to source code files.

The following example show how to invoke the plugin in `dryRun` mode.

```
mvn org.openrewrite.maven:rewrite-maven-plugin:<rewrite-plugin-version>*:dryRun \
 -Drewrite.recipeArtifactCoordinates=software.amazon.awssdk:v2-
migration:<sdkversion>** \
 -Drewrite.activeRecipes=software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
```

\*Replace `<rewrite-plugin-version>` with the `rewriteMavenPluginVersion` value that you see in this [test file](#).

\*\*Replace `<sdkversion>` with a 2.x SDK version. Visit [Maven Central](#) to check for the latest version.

### Important

Be sure to use the version of the `rewrite-maven-plugin` shown in the [test file](#) because other versions may not work.

Your console output from the `dryRun` mode should resemble the following output.

```
[WARNING] These recipes would make changes to project/src/test/resources/maven/
before/pom.xml:
[WARNING] software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
[WARNING] software.amazon.awssdk.v2migration.UpgradeSdkDependencies
[WARNING] org.openrewrite.java.dependencies.AddDependency:
{groupId=software.amazon.awssdk, artifactId=apache-client, version=2.27.0,
onlyIfUsing=com.amazonaws.ClientConfiguration}
[WARNING] org.openrewrite.java.dependencies.AddDependency:
{groupId=software.amazon.awssdk, artifactId=netty-nio-client, version=2.27.0,
onlyIfUsing=com.amazonaws.ClientConfiguration}
[WARNING] org.openrewrite.java.dependencies.ChangeDependency:
{oldGroupId=com.amazonaws, oldArtifactId=aws-java-sdk-bom,
newGroupId=software.amazon.awssdk, newArtifactId=bom, newVersion=2.27.0}
```

```

[WARNING] org.openrewrite.java.dependencies.ChangeDependency:
{oldGroupId=com.amazonaws, oldArtifactId=aws-java-sdk-s3,
newGroupId=software.amazon.awssdk, newArtifactId=s3, newVersion=2.27.0}
[WARNING] org.openrewrite.java.dependencies.ChangeDependency:
{oldGroupId=com.amazonaws, oldArtifactId=aws-java-sdk-sqs,
newGroupId=software.amazon.awssdk, newArtifactId=sqs, newVersion=2.27.0}
[WARNING] These recipes would make changes to project/src/test/resources/maven/
before/src/main/java/foo/bar/Application.java:
[WARNING] software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
[WARNING] software.amazon.awssdk.v2migration.S3GetObjectConstructorToFluent
[WARNING] software.amazon.awssdk.v2migration.ConstructorToFluent
[WARNING] software.amazon.awssdk.v2migration.S3StreamingResponseToV2
[WARNING] software.amazon.awssdk.v2migration.ChangeSdkType
[WARNING] software.amazon.awssdk.v2migration.ChangeSdkCoreTypes
[WARNING] software.amazon.awssdk.v2migration.ChangeExceptionTypes
[WARNING] org.openrewrite.java.ChangeType:
{oldFullyQualifiedTypeName=com.amazonaws.AmazonClientException,
newFullyQualifiedTypeName=software.amazon.awssdk.core.exception.SdkException}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getId(),
newMethodName=requestId}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getErrorCode(),
newMethodName=awsErrorDetails().errorCode}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getServiceName(),
newMethodName=awsErrorDetails().serviceName}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getErrorMessage(),
newMethodName=awsErrorDetails().errorMessage}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getRawResponse(),
newMethodName=awsErrorDetails().rawResponse().asByteArray}
[WARNING] org.openrewrite.java.ChangeMethodName:
{methodPattern=com.amazonaws.AmazonServiceException getRawResponseContent(),
newMethodName=awsErrorDetails().rawResponse().asUtf8String}
[WARNING] org.openrewrite.java.ChangeType:
{oldFullyQualifiedTypeName=com.amazonaws.AmazonServiceException,
newFullyQualifiedTypeName=software.amazon.awssdk.awscore.exception.AwsServiceException}
[WARNING] software.amazon.awssdk.v2migration.NewClassToBuilderPattern
[WARNING] software.amazon.awssdk.v2migration.NewClassToBuilder
[WARNING] software.amazon.awssdk.v2migration.V1SetterToV2
[WARNING] software.amazon.awssdk.v2migration.V1GetterToV2
...

```

```
[WARNING] software.amazon.awssdk.v2migration.V1BuilderVariationsToV2Builder
[WARNING] software.amazon.awssdk.v2migration.NewClassToBuilderPattern
[WARNING] software.amazon.awssdk.v2migration.NewClassToBuilder
[WARNING] software.amazon.awssdk.v2migration.V1SetterToV2
[WARNING] software.amazon.awssdk.v2migration.HttpSettingsToHttpClient
[WARNING] software.amazon.awssdk.v2migration.WrapSdkClientBuilderRegionStr
[WARNING] Patch file available:
[WARNING] project/src/test/resources/maven/before/target/rewrite/rewrite.patch
[WARNING] Estimate time saved: 20m
[WARNING] Run 'mvn rewrite:run' to apply the recipes.
```

## run mode

When you run the plugin in run mode, it modifies the source code on disk to apply the changes. Make sure you have a backup of the source code before running the command.

The following example show how to invoke the plugin in run mode.

```
mvn org.openrewrite.maven:rewrite-maven-plugin:<rewrite-plugin-version>*:run \
 -Drewrite.recipeArtifactCoordinates=software.amazon.awssdk:v2-
migration:<sdkversion>** \
 -Drewrite.activeRecipes=software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
```

\*Replace `<rewrite-plugin-version>` with the `rewriteMavenPluginVersion` value that you see in this [test file](#).

\*\*Replace `<sdkversion>` with a 2.x SDK version. Visit [Maven Central](#) to check for the latest version.

After you run the command, compile your application and run tests to verify the changes.

## Migrate a Gradle project

Follow the instructions below to migrate your SDK for Java 1.x Gradle-based project by using the [OpenRewrite Gradle plugin](#) tool.

### 1. Navigate to your Gradle project's root directory

Open a terminal (command line) window and navigate to the root directory of your Gradle-based application.



## 2. Create a Gradle init script

Create a `init.gradle` file with the following content in the directory.

```
initscript {
 repositories {
 maven { url "https://plugins.gradle.org/m2" }
 }
 dependencies {
 classpath("org.openrewrite:plugin:<rewrite-plugin-version>*")
 }
}

rootProject {
 plugins.apply(org.openrewrite.gradle.RewritePlugin)
 dependencies {
 rewrite("software.amazon.awssdk:v2-migration:latest.release")
 }

 afterEvaluate {
 if (repositories.isEmpty()) {
 repositories {
 mavenCentral()
 }
 }
 }
}
```

\*Replace `<rewrite-plugin-version>` with the version that you see in this [test file](#).

## 3. Run the rewrite command

As with the Maven plugin, you can run the Gradle plugin in `dryRun` or `run` mode.

### **dryRun mode**

The following example show how to invoke the plugin in `dryRun` mode.

```
gradle rewriteDryRun --init-script init.gradle \
-Drewrite.activeRecipes=software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
```

### **run mode**

The following example show how to invoke the plugin in run mode.

```
gradle rewriteRun --init-script init.gradle \
-Drewrite.activeRecipes=software.amazon.awssdk.v2migration.AwsSdkJavaV1ToV2
```

## Current limitations

While the migration supports most V1 code through code modification rules that update to the V2 equivalent, some classes and methods aren't covered. For these classes and methods, follow the [step-by-step instructions](#) to manually migrate your code.

For some unsupported code modification rules, the migration tool may add a comment that begins with:

```
/*AWS SDK for Java v2 migration: Transform for ...
```

Following the comment, the tool outputs a generic stub of the V2 version of the method or class. For example, in the following output, the migration tool attempted to migrate the V1 S3 client's `setBucketLifecycleConfiguration` method:

```
/*AWS SDK for Java v2 migration: Transform for setBucketLifecycleConfiguration method
not supported.
Please manually migrate your code by using builder pattern, update from
BucketLifecycleConfiguration.Rule
to LifecycleRule, StorageClass to TransitionStorageClass, and adjust imports and
names.*/
s3.putBucketLifecycleConfiguration(
 PutBucketLifecycleConfigurationRequest.builder()
 .bucket(bucketName)
 .lifecycleConfiguration(BucketLifecycleConfiguration.builder()
 .build())
 .build());
```

The links in the list below, take you to migration information to help you manually migrate the code.

- [the section called “S3 client differences”](#)
- [S3 Transfer Manager](#) (TransferManager)

- [DynamoDB object mapping](#) (DynamoDBMapper)
- [EC2 metadata utility](#) (EC2MetadataUtils)
- [Waiters](#) (AmazonDynamoDBWaiters)
- [IAM Policy Builder](#) (Policy)
- [CloudFront presigning](#) (CloudFrontUrlSigner, CloudFrontCookieSigner)
- [S3 Event Notifications](#) (S3EventNotification)
- SDK metric publishing ([1.x documentation](#), [2.x documentation](#))
- [Unsupported code patterns](#)—Detailed examples of common code patterns that require manual migration

## Unsupported code patterns of the migration tool

The migration tool automatically converts most v1 code to v2. However, some code patterns require manual migration. This topic provides detailed examples of the most common unsupported patterns and shows you how to manually convert them.

The following list of patterns is not exhaustive. If your code doesn't compile after running the migration tool, follow the [step-by-step migration instructions](#) to manually migrate the remaining v1 code.

### Request object constructors with parameters

For request POJOs (excluding Amazon S3), the migration tool transforms only setter methods. The tool does not support constructors with parameters.

#### Supported pattern: Request object using setters (no constructor parameters)

Before (original v1 code):

```
import com.amazonaws.services.sqs.model.SendMessageRequest;

SendMessageRequest request = new SendMessageRequest().withMessageBody("Hello World");
request.setQueueUrl("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue");
```

After (migration tool result):

```
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;
```

```
SendMessageRequest request = SendMessageRequest.builder()
 .messageBody("Hello World").build();
request = request.toBuilder()
 .queueUrl("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue").build();
```

## Unsupported pattern: Request object constructor with parameters

The migration tool cannot convert constructors with parameters:

Before manual migration, but after migration tool:

```
import software.amazon.awssdk.services.sqs.model.SendMessageRequest; // Import updated
to v2.

// This pattern requires manual migration.
SendMessageRequest request = new SendMessageRequest(
 "https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue",
 "Hello World");
```

The migration tool transforms the import to v2, but the constructor code remains unchanged and requires manual updates to use the builder pattern.

After manual migration:

```
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;

SendMessageRequest request = SendMessageRequest.builder()
 .messageBody("Hello World")
 .queueUrl("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue")
 .build();
```

## Service client methods with individual parameters

The migration tool cannot convert service client methods that take individual parameters instead of request objects (excluding Amazon S3).

Before (v1 code):

```
import com.amazonaws.services.sqs.AmazonSQS;
import com.amazonaws.services.sqs.AmazonSQSClient;

AmazonSQS sqs = new AmazonSQSClient();
```

```
// The following v1 method takes individual parameters.
sqs.sendMessage("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue", "Hello
World");
```

After (migration tool result–does not compile):

```
import software.amazon.awssdk.services.sqs.SqsClient; // Import updated to v2.
// No import statement for the v2 request POJO.

SqsClient sqs = SqsClient.builder().build();

// Does not compile–v2 methods only accept request POJOs.
sqs.sendMessage("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue", "Hello
World");
```

You must manually update the method arguments to use a request object:

```
import software.amazon.awssdk.services.sqs.SqsClient;
import software.amazon.awssdk.services.sqs.model.SendMessageRequest; // Add manually.

SqsClient sqs = SqsClient.builder().build();

// Corrected v2 code.
SendMessageRequest request = SendMessageRequest.builder()
 .queueUrl("https://sqs.us-west-2.amazonaws.com/0123456789012/demo-queue")
 .messageBody("Hello World")
 .build();
sqs.sendMessage(request);
```

## Request timeout methods

The migration tool does not convert methods that set timeouts on request objects.

Before (v1 code):

```
import com.amazonaws.services.sqs.model.SendMessageRequest;

SendMessageRequest request = new SendMessageRequest();
request.setSdkRequestTimeout(7);
```

After (migration tool result–does not compile):

```
import software.amazon.awssdk.services.sqs.model.SendMessageRequest; // Import updated
to v2.

SendMessageRequest request = SendMessageRequest.builder().build();

// Does not compile.
request.setSdkRequestTimeout(7);
```

You must manually migrate to use v2's `overrideConfiguration` method:

```
import software.amazon.awssdk.services.sqs.model.SendMessageRequest;
import java.time.Duration;

SendMessageRequest request = SendMessageRequest.builder().build();

// Corrected v2 code.
request = request.toBuilder()
 .overrideConfiguration(o -> o.apiCallTimeout(Duration.ofSeconds(7)))
 .build();
```

## Service client constructors with parameters

The migration tool converts empty service client constructors but cannot convert constructors that accept parameters like credentials or configuration.

Before (v1 code):

```
import com.amazonaws.auth.AWSCredentials;
import com.amazonaws.auth.BasicAWSCredentials;
import com.amazonaws.services.sqs.AmazonSQS;
import com.amazonaws.services.sqs.AmazonSQSClient;

AWSCredentials awsCredentials = new BasicAWSCredentials("akid", "skid");
AmazonSQS sqs = new AmazonSQSClient(awsCredentials);
```

After (migration tool result—does not compile):

```
import software.amazon.awssdk.auth.credentials.AwsCredentials;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.services.sqs.SqsClient; // Import updated to v2.
```

```
AwsCredentials awsCredentials = AwsBasicCredentials.create("akid", "skid");

// Does not compile.
SqsClient sqs = new SqsClient(awsCredentials);
```

You must manually update the service client constructor to use the builder pattern:

```
import software.amazon.awssdk.auth.credentials.AwsCredentials;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider; // Add manually.
import software.amazon.awssdk.services.sqs.SqsClient;

AwsCredentials awsCredentials = AwsBasicCredentials.create("akid", "skid");

// Corrected v2 code.
SqsClient sqs = SqsClient.builder()
 .credentialsProvider(StaticCredentialsProvider.create(awsCredentials))
 .build();
```

## Migration step-by-step instructions with example

This section provides a step-by-step guide to migrate your application that currently uses the SDK for Java v1.x to the SDK for Java 2.x. The first part presents an overview of the steps followed by a detailed example of a migration.

The steps that are covered here describe a migration of a normal use case, where the application calls AWS services using model-driven service clients. If you need to migrate code that uses higher level APIs such as [S3 Transfer Manager](#) or [CloudFront presigning](#), refer to the section under [the section called “What's different between 1.x and 2.x”](#) table of contents.

The approach described here is a suggestion. You may use other techniques and leverage your IDE's code editing features to reach the same result.

## Overview of steps

### 1. Begin by adding the SDK for Java 2.x BOM

By adding the Maven BOM (Bill of Materials) element for the SDK for Java 2.x to your POM file, you ensure that all of the v2 dependency you need are from the same version. Your POM can contain

both v1 and v2 dependencies. This allows you to incrementally migrate your code rather than change it all at once.

## SDK for Java 2.x BOM

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.27.21</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>
```

You can find the [latest version](#) on the Maven Central Repository.

## 2. Search files for v1 class import statements

By scanning the files in your application for SERVICE\_IDs used in v1 imports, you'll find the unique SERVICE\_IDs used. A SERVICE\_ID is a short, unique name for an AWS service. For example `cognitoidentity` is the SERVICE\_ID for Amazon Cognito Identity.

## 3. Determine the v2 Maven dependencies from the v1 import statements

After you find all unique v1 SERVICE\_IDs, you can determine the corresponding Maven artifact for the v2 dependency by referring to [the section called "Package name to artifactId mappings"](#).

## 4. Add v2 dependency elements to the POM file

Update the Maven POM file with dependency elements determined in step 3.

## 5. In the Java files, incrementally change over the v1 classes to v2 classes

As you replace v1 classes with v2 classes, make the necessary changes to support the v2 API such as using builders instead of constructors and using fluent getters and setters.

## 6. Remove v1 Maven dependencies from the POM and v1 imports from files

After you migrate your code to use v2 classes, remove any leftover v1 imports from files and all dependencies from your build file.



## 7. Refactor the code to use v2 API enhancements

After the code successfully compiles and passes tests, you can take advantage of v2 enhancements such as using a different HTTP client or paginators to simplify code. This is an optional step.

### Example migration

In this example, we migrate an application that uses the SDK for Java v1 and accesses several AWS services. We work through the following v1 method in detail in step 5. This is one method in a class that contains eight methods and there are 32 classes in the application.

#### v1 method to migrate

Only the v1 SDK imports are listed below from the Java file.

```
import com.amazonaws.ClientConfiguration;
import com.amazonaws.regions.Region;
import com.amazonaws.regions.RegionUtils;
import com.amazonaws.services.ec2.AmazonEC2Client;
import com.amazonaws.services.ec2.model.AmazonEC2Exception;
import com.amazonaws.services.ec2.model.CreateTagsRequest;
import com.amazonaws.services.ec2.model.DescribeInstancesRequest;
import com.amazonaws.services.ec2.model.DescribeInstancesResult;
import com.amazonaws.services.ec2.model.Instance;
import com.amazonaws.services.ec2.model.InstanceStateName;
import com.amazonaws.services.ec2.model.Reservation;
import com.amazonaws.services.ec2.model.Tag;
import com.amazonaws.services.ec2.model.TerminateInstancesRequest;
...
private static List<Instance> getRunningInstances(AmazonEC2Client ec2, List<String>
instanceIds) {
 List<Instance> runningInstances = new ArrayList<>();
 try {
 DescribeInstancesRequest request = new DescribeInstancesRequest()
 .withInstanceIds(instanceIds);
 DescribeInstancesResult result;
 do {
 // DescribeInstancesResponse is a paginated response, so use tokens with
multiple requests.
 result = ec2.describeInstances(request);
 request.setNextToken(result.getNextToken()); // Prepare request for next
page.

 for (final Reservation r : result.getReservations()) {
```

```

 for (final Instance instance : r.getInstances()) {
 LOGGER.info("Examining instanceId: " + instance.getInstanceId());
 // if instance is in a running state, add it to runningInstances
list.
 if (RUNNING_STATES.contains(instance.getState().getName())) {
 runningInstances.add(instance);
 }
 }
 } while (result.getNextToken() != null);
} catch (final AmazonEC2Exception exception) {
 // if instance isn't found, assume its terminated and continue.
 if (exception.getErrorCode().equals(NOT_FOUND_ERROR_CODE)) {
 LOGGER.info("Instance probably terminated; moving on.");
 } else {
 throw exception;
 }
}
return runningInstances;
}

```

## 1. Add v2 Maven BOM

Add the Maven BOM for the SDK for Java 2.x to the POM along side any other dependencies in the `dependencyManagement` section. If your POM file has the BOM for v1 of the SDK, leave it for now. It will be removed at a later step.

### POM Dependency management at outset

```

<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>org.example</groupId> <!--Existing dependency in POM. -->
 <artifactId>bom</artifactId>
 <version>1.3.4</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 ...
 <dependency>
 <groupId>com.amazonaws</groupId>
 <artifactId>aws-java-sdk-bom</artifactId> <!--Existing v1 BOM dependency. -->
 <version>1.11.1000</version>
 </dependency>
 </dependencies>
</dependencyManagement>

```

```
<type>pom</type>
<scope>import</scope>
</dependency>
...
<dependency>
 <groupId>software.amazon.awssdk</groupId> <!--Add v2 BOM dependency. -->
 <artifactId>bom</artifactId>
 <version>2.27.21</version>
 <type>pom</type>
 <scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

## 2. Search files for v1 class import statements

Search the application's code for unique occurrences of `import com.amazonaws.services`. This helps us determine the v1 dependencies used by the project. If your application has a Maven POM file with v1 dependencies listed, you can use this information instead.

For this example we use the [ripgrep \(rg\)](#) command to search the code base.

From the root of your code base, execute the following `ripgrep` command. After `ripgrep` finds the import statements, they are piped to the `cut`, `sort`, and `uniq` commands to isolate the `SERVICE_IDS`.

```
rg --no-filename 'import\s+com\.amazonaws\.services' | cut -d '.' -f 4 | sort | uniq
```

For this application, the following `SERVICE_IDS` are logged to the console.

```
autoscaling
cloudformation
ec2
identitymanagement
```

This indicates that there was at least one occurrence of each of the following package names used in `import` statements. For our purposes, the individual class names don't matter. We just need to find the `SERVICE_IDS` that are used.

```
com.amazonaws.services.autoscaling.*
com.amazonaws.services.cloudformation.*
```

```
com.amazonaws.services.ec2.*
com.amazonaws.services.identitymanagement.*
```

3. Determine the v2 Maven dependencies from the v1 import statements

The SERVICE\_IDs for v1 that we isolated from Step 2—for example autoscaling and cloudformation—can be mapped to the same v2 SERVICE\_ID for the most part. Since the v2 Maven artifactId matches the SERVICE\_ID in most cases, you have the information you need to add dependency blocks to your POM file.

The following table shows how we can determine the v2 dependencies.

v1 SERVICE_ID maps to ... package name	v2 SERVICE_ID maps to ... package name	v2 Maven dependency
ec2  com.amazonaws.serv ices. <b>ec2.*</b>	ec2  software.amazon.aw ssdk.services. <b>ec2.*</b>	<pre>&lt;dependency&gt;   &lt;groupId&gt;software. amazon.awssdk&lt;/gro upId&gt;   &lt;artifactId&gt; <b>ec2</b>&lt;/ artifactId&gt; &lt;/dependency&gt;</pre>
autoscaling  com.amazonaws.serv ices. <b>autoscaling .*</b>	autoscaling  software.amazon.aw ssdk.serv ices. <b>autoscaling .*</b>	<pre>&lt;dependency&gt;   &lt;groupId&gt;software. amazon.awssdk&lt;/gro upId&gt;   &lt;artifactId&gt; <b>autoscali ng</b> &lt;/artifactId&gt; &lt;/dependency&gt;</pre>
cloudformation  com.amazonaws.serv ices. <b>cloudform ation .*</b>	cloudformation  software.amazon.aw ssdk. <b>cloudform ation .*</b>	<pre>&lt;dependency&gt;   &lt;groupId&gt;software. amazon.awssdk&lt;/gro upId&gt;   &lt;artifactId&gt; <b>cloudform ation</b> &lt;/artifactId&gt; &lt;/dependency&gt;</pre>

v1 SERVICE_ID maps to ... package name	v2 SERVICE_ID maps to ... package name	v2 Maven dependency
<b>identitymanagement*</b>  com.amazonaws.serv ices. <b>identitym</b> <b>anagement</b> .*	<b>iam*</b>  software.amazon.aw ssdk. <b>iam.*</b>	<pre>&lt;dependency&gt;   &lt;groupId&gt;software. amazon.awssdk&lt;/gro upId&gt;   &lt;artifactId&gt; iam&lt;/ artifactId&gt; &lt;/dependency&gt;</pre>

\* The identitymanagement to iam mapping is an exception where the SERVICE\_ID differs between versions. Refer to the [the section called “Package name to artifactId mappings”](#) for exceptions if Maven or Gradle cannot resolve the v2 dependency.

#### 4. Add v2 dependency elements to the POM file

In step 3, we determined the four dependency blocks that need to be added to the POM file. We don't need to add a version because we have specified the BOM in step 1. After the imports are added, our POM file has the following dependency elements.

```
...
<dependencies>
...
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>autoscaling</artifactId>
</dependency>
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>iam</artifactId>
</dependency>
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>cloudformation</artifactId>
</dependency>
<dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>ec2</artifactId>
</dependency>
```

```
...
</dependencies>
...
```

## 5. In the Java files, incrementally change over the v1 classes to v2 classes

In the method that we are migrating, we see

- An EC2 service client from `com.amazonaws.services.ec2.AmazonEC2Client`.
- Several EC2 model classes used. For example `DescribeInstancesRequest` and `DescribeInstancesResult`.

```
import com.amazonaws.ClientConfiguration;
import com.amazonaws.regions.Region;
import com.amazonaws.regions.RegionUtils;
import com.amazonaws.services.ec2.AmazonEC2Client;
import com.amazonaws.services.ec2.model.AmazonEC2Exception;
import com.amazonaws.services.ec2.model.CreateTagsRequest;
import com.amazonaws.services.ec2.model.DescribeInstancesRequest;
import com.amazonaws.services.ec2.model.DescribeInstancesResult;
import com.amazonaws.services.ec2.model.Instance;
import com.amazonaws.services.ec2.model.InstanceStateName;
import com.amazonaws.services.ec2.model.Reservation;
import com.amazonaws.services.ec2.model.Tag;
import com.amazonaws.services.ec2.model.TerminateInstancesRequest;
...
private static List<Instance> getRunningInstances(AmazonEC2Client ec2, List<String>
instanceIds)
 List<Instance> runningInstances = new ArrayList<>();
 try {
 DescribeInstancesRequest request = new DescribeInstancesRequest()
 .withInstanceIds(instanceIds);
 DescribeInstancesResult result;
 do {
 // DescribeInstancesResponse is a paginated response, so use tokens with
multiple re
 result = ec2.describeInstances(request);
 request.setNextToken(result.getNextToken()); // Prepare request for next
page.
 for (final Reservation r : result.getReservations()) {
 for (final Instance instance : r.getInstances()) {
 LOGGER.info("Examining instanceId: " + instance.getInstanceId());
```

```
 // if instance is in a running state, add it to runningInstances
list.
 if (RUNNING_STATES.contains(instance.getState().getName())) {
 runningInstances.add(instance);
 }
 }
} while (result.getNextToken() != null);
} catch (final AmazonEC2Exception exception) {
 // if instance isn't found, assume its terminated and continue.
 if (exception.getErrorCode().equals(NOT_FOUND_ERROR_CODE)) {
 LOGGER.info("Instance probably terminated; moving on.");
 } else {
 throw exception;
 }
}
return runningInstances;
}
...
```

Our goal is to replace all v1 imports with v2 imports. We proceed one class at a time.

#### a. Replace import statement or class name

We see that the first parameter to the `describeRunningInstances` method is a v1 `AmazonEC2Client` instance. Do one of the following:

- Replace the import for `com.amazonaws.services.ec2.AmazonEC2Client` with `software.amazon.awssdk.services.ec2.Ec2Client` and change `AmazonEC2Client` to `Ec2Client`.
- Change the parameter type to `Ec2Client` and let the IDE prompt us for the correct import. Our IDE will prompt us to import the v2 class because the client names differ—`AmazonEC2Client` and `Ec2Client`. This approach does not work if the class name is the same in both versions.

#### b. Replace v1 model classes with v2 equivalents

After the change to the v2 `Ec2Client`, if we use an IDE, we see compilation errors in the following statement.

```
result = ec2.describeInstances(request);
```

The compilation error results from using an instance of v1's `DescribeInstancesRequest` as a parameter to the v2 `Ec2Client` `describeInstances` method. To fix, make the following replacement or import statements.

replace	with
<pre>import com.amazonaws.services.ec2. model.DescribeInstancesRequest</pre>	<pre>import software.amazon.awssdk.serv ices.ec2.model.DescribeInst ancesRequest</pre>

### c. Change v1 constructors to v2 builders.

We still see compilation errors because there are [no constructors on v2 classes](#). To fix, make the following change.

change	to
<pre>final DescribeInstancesRequest request = new DescribeInstancesR equest()         .withInstanceIds(instanceId sCopy);</pre>	<pre>final DescribeInstancesRequest request = DescribeInstancesR equest.builder()         .instanceIds(instanceIdsCop y)         .build();</pre>

### d. Replace v1 `*Result` response objects with v2 `*Response` equivalents

A consistent difference between v1 and v2 is that all [response objects in v2 end with `\*Response` instead of `\*Result`](#). Replace the v1 `DescribeInstancesResult` import to the v2 import, `DescribeInstancesResponse`.

### d. Make API changes

The following statement needs a few changes.

```
request.setNextToken(result.getNextToken());
```



In v2, [setter methods](#) do not use the set or with prefix. Getter methods prefixed with get are also gone in the SDK for Java 2.x

Model classes, such as the request instance, are immutable in v2, so we need to create a new `DescribeInstancesRequest` with a builder.

In v2, the statement becomes the following.

```
request = DescribeInstancesRequest.builder()
 .nextToken(result.nextToken())
 .build();
```

#### d. Repeat until method compiles with v2 classes

Continue with the rest of the code. Replace v1 imports with v2 imports and fix the compilation errors. Refer to the [v2 API Reference](#) and [What's different reference](#) as needed.

After we migrate this single method, we have the following v2 code.

```
import com.amazonaws.ClientConfiguration;
import com.amazonaws.regions.Region;
import com.amazonaws.regions.RegionUtils;
import com.amazonaws.services.ec2.AmazonEC2Client;
import com.amazonaws.services.ec2.model.AmazonEC2Exception;
import com.amazonaws.services.ec2.model.CreateTagsRequest;
import com.amazonaws.services.ec2.model.InstanceStateName;
import com.amazonaws.services.ec2.model.Tag;
import com.amazonaws.services.ec2.model.TerminateInstancesRequest;

import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesResponse;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.Instance;
import software.amazon.awssdk.services.ec2.model.Reservation;
...
private static List<Instance> getRunningInstances(Ec2Client ec2, List<String>
instanceIds) {
 List<Instance> runningInstances = new ArrayList<>();
 try {
 DescribeInstancesRequest request = DescribeInstancesRequest.builder()
 .instanceIds(instanceIds)
 .build();
```

```

 DescribeInstancesResponse result;
 do {
 // DescribeInstancesResponse is a paginated response, so use tokens
with multiple re
 result = ec2.describeInstances(request);
 request = DescribeInstancesRequest.builder() // Prepare request for
next page.
 .nextToken(result.nextToken())
 .build();
 for (final Reservation r : result.reservations()) {
 for (final Instance instance : r.instances()) {
 // if instance is in a running state, add it to
runningInstances list.
 if (RUNNING_STATES.contains(instance.state().nameAsString())) {
 runningInstances.add(instance);
 }
 }
 }
 } while (result.nextToken() != null);
 } catch (final Ec2Exception exception) {
 // if instance isn't found, assume its terminated and continue.
 if (exception.awsErrorDetails().errorCode().equals(NOT_FOUND_ERROR_CODE)) {
 LOGGER.info("Instance probably terminated; moving on.");
 } else {
 throw exception;
 }
 }
 return runningInstances;
}
...

```

Because we are migrating a single method in a Java file with eight methods, we have a mix of v1 and v2 imports as we work through the file. We added the last six import statements as we performed the steps.

After we migrate all the code, there will be no more v1 import statements.

## 6. Remove v1 Maven dependencies from the POM and v1 imports from files

After we migrate all v1 code in the file, we have the following v2 SDK import statements.

```

import software.amazon.awssdk.core.client.config.ClientOverrideConfiguration;
import software.amazon.awssdk.regions.Region;

```

```
import software.amazon.awssdk.regions.ServiceMetadata;
import software.amazon.awssdk.services.ec2.Ec2Client;
import software.amazon.awssdk.services.ec2.model.CreateTagsRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesRequest;
import software.amazon.awssdk.services.ec2.model.DescribeInstancesResponse;
import software.amazon.awssdk.services.ec2.model.Ec2Exception;
import software.amazon.awssdk.services.ec2.model.Instance;
import software.amazon.awssdk.services.ec2.model.InstanceStateName;
import software.amazon.awssdk.services.ec2.model.Reservation;
import software.amazon.awssdk.services.ec2.model.Tag;
import software.amazon.awssdk.services.ec2.model.TerminateInstancesRequest;
```

After we migrate *all* files in our application, we no longer need the v1 dependencies in our POM file. Remove the v1 BOM from the dependencyManagement section, if using, and all v1 dependency blocks.

## 7. Refactor the code to use v2 API enhancements

For the snippet we have been migrating, we can optionally use a v2 paginator and let the SDK manage the token-based requests for more data.

We can replace the entire `do` clause with the following.

```
DescribeInstancesIterable responses =
ec2.describeInstancesPaginator(request);

responses.reservations().stream()
 .forEach(reservation -> reservation.instances()
 .forEach(instance -> {
 if
(RUNNING_STATES.contains(instance.state().nameAsString())) {
 runningInstances.put(instance.instanceId(),
instance);
 }
 }));
```

## Package name to Maven artifactId mappings

When you migrate your Maven or Gradle project from v1 of the SDK for Java to v2, you need to figure out which dependencies to add to your build file. The approach described in the [the section called “Step-by-step instructions”](#) (step 3) uses the package names in import statements as a starting point to determine the dependencies (as artifactIds) to add to your build file.

You can use the information in this topic to map the v1 package names to v2 artifactIds.

## Common naming convention used in package names and Maven artifactIds

The following table shows the common naming convention that the SDKs use for a given `SERVICE_ID`. A `SERVICE_ID` is a unique identifier for an AWS service. For example, the `SERVICE_ID` for the Amazon S3 service is `s3` and `cognitoidentity` is the `SERVICE_ID` for Amazon Cognito Identity.

v1 package name (import statement)	v1 artifactId	v2 artifactId	v2 package name (import statement)
<code>com.amazonaws.services.SERVICE_ID</code>	<code>aws-java-sdk-SERVICE_ID</code>	<code>SERVICE_ID</code>	<code>software.amazon.awssdk.services.SERVICE_ID</code>

### *Example for Amazon Cognito Identity (`SERVICE_ID: cognitoidentity`)*

<code>com.amazonaws.services.<b>cognitoidentity</b></code>	<code>aws-java-sdk-<b>cognitoidentity</b></code>	<b><i>cognitoidentity</i></b>	<code>software.amazon.awssdk.services.<b>cognitoidentity</b></code>
------------------------------------------------------------	--------------------------------------------------	-------------------------------	---------------------------------------------------------------------

## SERVICE\_ID differences

### Within v1

In some cases the `SERVICE_ID` differs between the package name and in the artifactId for the same service. For example, the CloudWatch Metrics row of the following table shows that `metrics` is the `SERVICE_ID` in the package name but `cloudwatchmetrics` is the artifactId's `SERVICE_ID`.

### Within v2

There are no differences in the `SERVICE_ID` used in package names and artifactIds.

### Between v1 and v2

For the majority of services, the `SERVICE_ID` in v2 is the same as v1's `SERVICE_ID` in both package names and artifactIds. An example of this is the `cognitoedentity` `SERVICE_ID` as shown in the

previous table. However, some SERVICE\_IDs differ between the SDKs as shown in the following table.

A **boldface SERVICE\_ID** in either of the v1 columns indicates that it's different from the SERVICE\_ID used in v2.

Service name	v1 package name	v1 artifactId	v2 artifactId	v2 package name
	All package names begin with <code>com.amazonaws.services</code> as shown in the first row.	All artifactIds are enclosed in tags as shown in the first row.	All artifactIds are enclosed in tags as shown in the first row.	All package names begin with <code>software.amazon.awssdk</code> as shown in the first row.
API Gateway	<code>com.amazonaws.services.apigateway</code>	<code>&lt;artifactId&gt;aws-java-sdk-<b>api-gateway</b>&lt;/artifactId&gt;</code>	<code>&lt;artifactId&gt;apigateway&lt;/artifactId&gt;</code>	<code>software.amazon.awssdk.services.apigateway</code>
App Registry	<b>appregistry</b>	<b>appregistry</b>	<code>servicecatalogappregistry</code>	<code>servicecatalogappregistry</code>
Application Discovery	<code>applicationdiscovery</code>	<b>discovery</b>	<code>applicationdiscovery</code>	<code>applicationdiscovery</code>
Augmented AI Runtime	<b>augmentedairuntime</b>	<b>augmentedairuntime</b>	<code>sagemaker-a2iruntime</code>	<code>sagemaker-a2iruntime</code>
Certificate Manager	<b>certificatemanager</b>	<code>acm</code>	<code>acm</code>	<code>acm</code>
CloudControl API	<b>cloudcontrolapi</b>	<code>cloudcontrolapi</code>	<code>cloudcontrol</code>	<code>cloudcontrol</code>
CloudSearch	<b>cloudsearchv2</b>	<code>cloudsearch</code>	<code>cloudsearch</code>	<code>cloudsearch</code>

Service name	v1 package name	v1 artifactId	v2 artifactId	v2 package name
CloudSearch Domain	cloudsear chdomain	<b>cloudsearch</b>	cloudsear chdomain	cloudsear chdomain
CloudWatch Events	cloudwatc hevents	<b>events</b>	cloudwatc hevents	cloudwatc hevents
CloudWatch Evidently	<b>cloudwatc hevidently</b>	<b>cloudwatc hevidently</b>	evidently	evidently
CloudWatch Logs	<b>logs</b>	<b>logs</b>	cloudwatchlogs	cloudwatchlogs
CloudWatch Metrics	<b>metrics</b>	<b>cloudwatc hmetrics</b>	cloudwatch	cloudwatch
CloudWatch Rum	<b>cloudwatchrum</b>	<b>cloudwatchrum</b>	rum	rum
Cognito Identity Provider	<b>cognitoidp</b>	<b>cognitoidp</b>	cognitoid entityprovider	cognitoid entityprovider
Connect Campaign	<b>connectca mpaign</b>	<b>connectca mpaign</b>	connectca mpaigns	connectca mpaigns
Connect Wisdom	<b>connectwisdom</b>	<b>connectwisdom</b>	wisdom	wisdom
Database Migration Service	<b>databasem igrationservice</b>	<b>dms</b>	databasem igration	databasem igration
DataZone	datazone	<b>datazonee xternal</b>	datazone	datazone
DynamoDB	<b>dynamodbv2</b>	dynamodb	dynamodb	dynamodb
Elastic File System	<b>elasticfi lesystem</b>	efs	efs	efs

Service name	v1 package name	v1 artifactId	v2 artifactId	v2 package name
Elastic Map Reduce	<b>elasticmapreduce</b>	emr	emr	emr
Glue DataBrew	<b>gluedatabrew</b>	<b>gluedatabrew</b>	databrew	databrew
IAM Roles Anywhere	<b>iamrolesanywhere</b>	<b>iamrolesanywhere</b>	rolesanywhere	rolesanywhere
Identity Management	<b>identitymanagement</b>	iam	iam	iam
IoT Data	<b>iotdata</b>	<b>iot</b>	iotdataplane	iotdataplane
Kinesis Analytics	kinesisanalytics	<b>kinesis</b>	kinesisanalytics	kinesisanalytics
Kinesis Firehose	<b>kinesisfirehose</b>	<b>kinesis</b>	firehose	firehose
Kinesis Video Signaling Channels	<b>kinesisvideosignalingchannels</b>	<b>kinesisvideosignalingchannels</b>	kinesisvideosignaling	kinesisvideosignaling
Lex	lexruntime	<b>lex</b>	lexruntime	lexruntime
Lookout For Vision	<b>lookoutforvision</b>	<b>lookoutforvision</b>	lookoutvision	lookoutvision
Mainframe Modernization	<b>mainframemodernization</b>	<b>mainframemodernization</b>	m2	m2
Marketplace Metering	marketplacemetering	<b>marketplacemeteringservice</b>	marketplacemetering	marketplacemetering
Managed Grafana	<b>managedgrafana</b>	<b>managedgrafana</b>	grafana	grafana

Service name	v1 package name	v1 artifactId	v2 artifactId	v2 package name
Mechanical Turk	mturk	mechanicalturkrequester	mturk	mturk
Migration Hub Strategy Recommendations	migrationhubstrategyrecommendations	migrationhubstrategyrecommendations	migrationhubstrategy	migrationhubstrategy
Nimble Studio	nimblestudio	nimblestudio	nimble	nimble
Private 5G	private5g	private5g	privatenetworks	privatenetworks
Prometheus	prometheus	prometheus	amp	amp
Recycle Bin	recyclebin	recyclebin	rbin	rbin
Redshift Data API	redshiftdataapi	redshiftdataapi	redshiftdata	redshiftdata
Route 53	route53domains	route53	route53domains	route53domains
Sage Maker Edge Manager	sagemakeredgemanager	sagemakeredgemanager	sagemakeredge	sagemakeredge
Security Token	securitytoken	sts	sts	sts
Server Migration	servermigration	servermigration	sms	sms
Simple Email	simpleemail	ses	ses	ses
Simple Email V2	simpleemailv2	sesv2	sesv2	sesv2
Simple Systems Management	simplesystemsmangement	ssm	ssm	ssm



Service name	v1 package name	v1 artifactId	v2 artifactId	v2 package name
Simple Workflow	<b>simpleworkflow</b>	<b>simpleworkflow</b>	swf	swf
Step Functions	<b>stepfunctions</b>	<b>stepfunctions</b>	sfn	sfn

## What's different between the AWS SDK for Java 1.x and 2.x

This section describes the main changes to be aware of when converting an application from using the AWS SDK for Java version 1.x to version 2.x.

### Package name change

A noticeable change from the SDK for Java 1.x to the SDK for Java 2.x is the package name change. Package names begin with `software.amazon.awssdk` in SDK 2.x, whereas the SDK 1.x uses `com.amazonaws`.

These same names differentiate Maven artifacts from SDK 1.x to SDK 2.x. Maven artifacts for the SDK 2.x use the `software.amazon.awssdk` groupId, whereas the SDK 1.x uses the `com.amazonaws` groupId.

There are a few times when your code requires a `com.amazonaws` dependency for a project that otherwise uses only SDK 2.x artifacts. One example of this is when you work with server-side AWS Lambda. This was shown in the [Set up an Apache Maven project](#) section earlier in this guide.

#### Note

Several package names in the SDK 1.x contain v2. The use of v2 in this case usually means that code in the package is targeted to work with version 2 of the service.

Since the full package name begins with `com.amazonaws`, these are SDK 1.x components. Examples of these package names in the SDK 1.x are:

- `com.amazonaws.services.dynamodbv2`
- `com.amazonaws.retry.v2`
- `com.amazonaws.services.apigatewayv2`

- `com.amazonaws.services.simpleemailv2`

## Adding version 2.x to your project

Maven is the recommended way to manage dependencies when using the AWS SDK for Java 2.x. To add version 2.x components to your project, update your `pom.xml` file with a dependency on the SDK.

### Example

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.27.21</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>

<dependencies>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>dynamodb</artifactId>
 </dependency>
</dependencies>
```

You can also [use version 1.x and 2.x side-by-side](#) as you migrate your project to version 2.x.

## Immutable POJOs

Clients and operation request and response objects are now immutable and cannot be changed after creation. To reuse a request or response variable, you must build a new object to assign to it.

### Example of updating a request object in 1.x

```
DescribeAlarmsRequest request = new DescribeAlarmsRequest();
```

```
DescribeAlarmsResult response = cw.describeAlarms(request);

request.setNextToken(response.getNextToken());
```

## Example of updating a request object in 2.x

```
DescribeAlarmsRequest request = DescribeAlarmsRequest.builder().build();
DescribeAlarmsResponse response = cw.describeAlarms(request);

request = DescribeAlarmsRequest.builder()
 .nextToken(response.getNextToken())
 .build();
```

## Setter and getter methods

In the AWS SDK for Java 2.x, setter method names don't include the `set` or `with` prefix. For example, `*.withEndpoint()` is now `*.endpoint()`.

Getter method names do not use the `get` prefix.

## Example of using setter methods in 1.x

```
AmazonDynamoDB client = AmazonDynamoDBClientBuilder.standard()
 .withRegion("us-east-1")
 .build();
```

## Example of using setter methods in 2.x

```
DynamoDbClient client = DynamoDbClient.builder()
 .region(Region.US_EAST_1)
 .build();
```

## Example of using getter methods in 1.x

```
String token = request.getNextToken();
```

## Example of using getter methods in 2.x

```
String token = request.nextToken();
```

## Model class names

Model class names that represent service responses end with `Response` in v2 instead of `Result` that v1 uses.

### Example of class names that represent a response in v1

```
CreateApiKeyResult
AllocateAddressResult
```

### Example of class names that represent a response in v2

```
CreateApiKeyResponse
AllocateAddressResponse
```

## Migration status of libraries and utilities

### SDK for Java libraries and utilities

The following table lists the migration status of libraries and utilities for the SDK for Java.

Version 1.12.x name	Version 2.x name	Since version in 2.x
DynamoDBMapper	<a href="#">DynamoDbEnhancedClient</a>	2.12.0
Writers	<a href="#">Writers</a>	2.15.0
CloudFrontUrlSigner, CloudFrontCookieSigner	<a href="#">CloudFrontUtilities</a>	2.18.33
TransferManager	<a href="#">S3TransferManager</a>	2.19.0
EC2 Metadata client	<a href="#">EC2 Metadata client</a>	2.19.29
S3 URI parser	<a href="#">S3 URI parser</a>	2.20.41
IAM Policy Builder	<a href="#">IAM Policy Builder</a>	2.20.126
S3 Event Notifications	<a href="#">S3 Event Notifications</a>	2.25.11

Version 1.12.x name	Version 2.x name	Since version in 2.x
Amazon SQS Client-side Buffering	<a href="#">Automatic Request Batching API for Amazon SQS</a>	2.28.0
Progress Listeners	Progress Listeners	<a href="#">not yet released</a>

## Related libraries

The following table lists libraries that are released separately but work with the SDK for Java 2.x.

Name used with version 2.x of the SDK for Java	Since version
<a href="#">Amazon S3 Encryption Client</a>	3.0.0 <sup>1</sup>
<a href="#">AWS Database Encryption SDK for DynamoDB</a>	3.0.0 <sup>2</sup>

### <sup>1</sup>Amazon S3 Encryption Client

The encryption client for Amazon S3 is available by using the following Maven dependency.

```
<dependency>
 <groupId>software.amazon.encryption.s3</groupId>
 <artifactId>amazon-s3-encryption-client-java</artifactId>
 <version>3.x</version>
</dependency>
```

### <sup>2</sup>AWS Database Encryption SDK for DynamoDB

The AWS Database Encryption SDK for DynamoDB is available for V2 by using the following Maven dependency.

```
<dependency>
 <groupId>software.amazon.cryptography</groupId>
 <artifactId>aws-database-encryption-sdk-dynamodb</artifactId>
 <version>3.x</version>
</dependency>
```

Information on the encryption library for DynamoDB that works with v1 of the Java SDK is available in the [AWS Database Encryption SDK Developer Guide](#) (named *Amazon DynamoDB Encryption Client for Java*) and in [GitHub](#).

For more information on the DynamoDB encryption library that is compatible with V2 of the Java SDK, see the [AWS Database Encryption SDK Developer Guide](#) and the [GitHub source](#).

Migration information about the encryption library is available in the [AWS Database Encryption SDK Developer Guide](#).

## Migration details for libraries and utilities

- [Transfer Manager](#)
- [EC2 metadata utility](#)
- [CloudFront presigning](#)
- [S3 URI parsing](#)
- [DynamoDB mapping/document APIs](#)
- [IAM Policy Builder](#)
- [S3 Event Notifications](#)
- SDK metric publishing ([1.x documentation](#), [2.x documentation](#))

## Client changes

### Client builders

You must create all clients using the client builder method. Constructors are no longer available.

#### Example of creating a client in version 1.x

```
AmazonDynamoDB ddbClient = AmazonDynamoDBClientBuilder.defaultClient();
AmazonDynamoDBClient ddbClient = new AmazonDynamoDBClient();
```

#### Example of creating a client in version 2.x

```
DynamoDbClient ddbClient = DynamoDbClient.create();
DynamoDbClient ddbClient = DynamoDbClient.builder().build();
```

## Client class names

All client class names are now fully camel cased and no longer prefixed by Amazon. These changes are aligned with names used in the AWS CLI.

### Example of class names in 1.x

```
AmazonDynamoDB
AWSACMPCAAsyncClient
```

### Example of class names in 2.x

```
DynamoDbClient
AcmAsyncClient
```

### Client class name changes

1.x Client	2.x Client
com.amazonaws.services.acmpca.AWSACMPCAAsyncClient	software.amazon.awssdk.services.acm.AcmAsyncClient
com.amazonaws.services.acmpca.AWSACMPCAClient	software.amazon.awssdk.services.acm.AcmClient
com.amazonaws.services.alexaforbusiness.AmazonAlexaForBusinessAsyncClient	software.amazon.awssdk.services.alexaforbusiness.AlexaForBusinessAsyncClient
com.amazonaws.services.alexaforbusiness.AmazonAlexaForBusinessClient	software.amazon.awssdk.services.alexaforbusiness.AlexaForBusinessClient
com.amazonaws.services.apigateway.AmazonApiGatewayAsyncClient	software.amazon.awssdk.services.apigateway.ApiGatewayAsyncClient
com.amazonaws.services.apigateway.AmazonApiGatewayClient	software.amazon.awssdk.services.apigateway.ApiGatewayClient

1.x Client	2.x Client
<code>com.amazonaws.services.applicationautoscaling.AWSApplicationAutoScalingAsyncClient</code>	<code>software.amazon.awssdk.services.applicationautoscaling.ApplicationAutoScalingAsyncClient</code>
<code>com.amazonaws.services.applicationautoscaling.AWSApplicationAutoScalingClient</code>	<code>software.amazon.awssdk.services.applicationautoscaling.ApplicationAutoScalingClient</code>
<code>com.amazonaws.services.applicationdiscovery.AWSApplicationDiscoveryAsyncClient</code>	<code>software.amazon.awssdk.services.applicationdiscovery.ApplicationDiscoveryAsyncClient</code>
<code>com.amazonaws.services.applicationdiscovery.AWSApplicationDiscoveryClient</code>	<code>software.amazon.awssdk.services.applicationdiscovery.ApplicationDiscoveryClient</code>
<code>com.amazonaws.services.appstream.AmazonAppStreamAsyncClient</code>	<code>software.amazon.awssdk.services.appstream.AppStreamAsyncClient</code>
<code>com.amazonaws.services.appstream.AmazonAppStreamClient</code>	<code>software.amazon.awssdk.services.appstream.AppStreamClient</code>
<code>com.amazonaws.services.appsync.AWSAppSyncAsyncClient</code>	<code>software.amazon.awssdk.services.appsync.AppSyncAsyncClient</code>
<code>com.amazonaws.services.appsync.AWSAppSyncClient</code>	<code>software.amazon.awssdk.services.appsync.AppSyncClient</code>
<code>com.amazonaws.services.athena.AmazonAthenaAsyncClient</code>	<code>software.amazon.awssdk.services.athena.AthenaAsyncClient</code>
<code>com.amazonaws.services.athena.AmazonAthenaClient</code>	<code>software.amazon.awssdk.services.athena.AthenaClient</code>



1.x Client	2.x Client
<code>com.amazonaws.services.autoscaling.AmazonAutoScalingAsyncClient</code>	<code>software.amazon.awssdk.services.autoscaling.AutoScalingAsyncClient</code>
<code>com.amazonaws.services.autoscaling.AmazonAutoScalingClient</code>	<code>software.amazon.awssdk.services.autoscaling.AutoScalingClient</code>
<code>com.amazonaws.services.autoscalingplans.AWSAutoScalingPlansAsyncClient</code>	<code>software.amazon.awssdk.services.autoscalingplans.AutoScalingPlansAsyncClient</code>
<code>com.amazonaws.services.autoscalingplans.AWSAutoScalingPlansClient</code>	<code>software.amazon.awssdk.services.autoscalingplans.AutoScalingPlansClient</code>
<code>com.amazonaws.services.batch.AWSBatchAsyncClient</code>	<code>software.amazon.awssdk.services.batch.BatchAsyncClient</code>
<code>com.amazonaws.services.batch.AWSBatchClient</code>	<code>software.amazon.awssdk.services.batch.BatchClient</code>
<code>com.amazonaws.services.budgets.AWSBudgetsAsyncClient</code>	<code>software.amazon.awssdk.services.budgets.BudgetsAsyncClient</code>
<code>com.amazonaws.services.budgets.AWSBudgetsClient</code>	<code>software.amazon.awssdk.services.budgets.BudgetsClient</code>
<code>com.amazonaws.services.certificatemanager.AWSCertificateManagerAsyncClient</code>	<code>software.amazon.awssdk.services.acm.AcmAsyncClient</code>
<code>com.amazonaws.services.certificatemanager.AWSCertificateManagerClient</code>	<code>software.amazon.awssdk.services.acm.AcmClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.cloud9.AWSCloud9AsyncClient</code>	<code>software.amazon.awssdk.services.cloud9.Cloud9AsyncClient</code>
<code>com.amazonaws.services.cloud9.AWSCloud9Client</code>	<code>software.amazon.awssdk.services.cloud9.Cloud9Client</code>
<code>com.amazonaws.services.clouddirectory.AmazonCloudDirectoryAsyncClient</code>	<code>software.amazon.awssdk.services.clouddirectory.CloudDirectoryAsyncClient</code>
<code>com.amazonaws.services.clouddirectory.AmazonCloudDirectoryClient</code>	<code>software.amazon.awssdk.services.clouddirectory.CloudDirectoryClient</code>
<code>com.amazonaws.services.cloudformation.AmazonCloudFormationAsyncClient</code>	<code>software.amazon.awssdk.services.cloudformation.CloudFormationAsyncClient</code>
<code>com.amazonaws.services.cloudformation.AmazonCloudFormationClient</code>	<code>software.amazon.awssdk.services.cloudformation.CloudFormationClient</code>
<code>com.amazonaws.services.cloudfront.AmazonCloudFrontAsyncClient</code>	<code>software.amazon.awssdk.services.cloudfront.CloudFrontAsyncClient</code>
<code>com.amazonaws.services.cloudfront.AmazonCloudFrontClient</code>	<code>software.amazon.awssdk.services.cloudfront.CloudFrontClient</code>
<code>com.amazonaws.services.cloudhsm.AWSCloudHSMAsyncClient</code>	<code>software.amazon.awssdk.services.cloudhsm.CloudHsmAsyncClient</code>
<code>com.amazonaws.services.cloudhsm.AWSCloudHSMClient</code>	<code>software.amazon.awssdk.services.cloudhsm.CloudHsmClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.cloudhsmv2.AWSCloudHSMV2AsyncClient</code>	<code>software.amazon.awssdk.services.cloudhsmv2.CloudHsmV2AsyncClient</code>
<code>com.amazonaws.services.cloudhsmv2.AWSCloudHSMV2Client</code>	<code>software.amazon.awssdk.services.cloudhsmv2.CloudHsmV2Client</code>
<code>com.amazonaws.services.cloudsearchdomain.AmazonCloudSearchDomainAsyncClient</code>	<code>software.amazon.awssdk.services.cloudsearchdomain.CloudSearchDomainAsyncClient</code>
<code>com.amazonaws.services.cloudsearchdomain.AmazonCloudSearchDomainClient</code>	<code>software.amazon.awssdk.services.cloudsearchdomain.CloudSearchDomainClient</code>
<code>com.amazonaws.services.cloudsearchv2.AmazonCloudSearchAsyncClient</code>	<code>software.amazon.awssdk.services.cloudsearch.CloudSearchAsyncClient</code>
<code>com.amazonaws.services.cloudsearchv2.AmazonCloudSearchClient</code>	<code>software.amazon.awssdk.services.cloudsearch.CloudSearchClient</code>
<code>com.amazonaws.services.cloudtrail.AWSCloudTrailAsyncClient</code>	<code>software.amazon.awssdk.services.cloudtrail.CloudTrailAsyncClient</code>
<code>com.amazonaws.services.cloudtrail.AWSCloudTrailClient</code>	<code>software.amazon.awssdk.services.cloudtrail.CloudTrailClient</code>
<code>com.amazonaws.services.cloudwatch.AmazonCloudWatchAsyncClient</code>	<code>software.amazon.awssdk.services.cloudwatch.CloudWatchAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.cloudwatch.AmazonCloudWatchClient</code>	<code>software.amazon.awssdk.services.cloudwatch.CloudWatchClient</code>
<code>com.amazonaws.services.cloudwatchevents.AmazonCloudWatchEventsAsyncClient</code>	<code>software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsAsyncClient</code>
<code>com.amazonaws.services.cloudwatchevents.AmazonCloudWatchEventsClient</code>	<code>software.amazon.awssdk.services.cloudwatchevents.CloudWatchEventsClient</code>
<code>com.amazonaws.services.codebuild.AWSCodeBuildAsyncClient</code>	<code>software.amazon.awssdk.services.codebuild.CodeBuildAsyncClient</code>
<code>com.amazonaws.services.codebuild.AWSCodeBuildClient</code>	<code>software.amazon.awssdk.services.codebuild.CodeBuildClient</code>
<code>com.amazonaws.services.codecommit.AWSCodeCommitAsyncClient</code>	<code>software.amazon.awssdk.services.codecommit.CodeCommitAsyncClient</code>
<code>com.amazonaws.services.codecommit.AWSCodeCommitClient</code>	<code>software.amazon.awssdk.services.codecommit.CodeCommitClient</code>
<code>com.amazonaws.services.codedeploy.AmazonCodeDeployAsyncClient</code>	<code>software.amazon.awssdk.services.codedeploy.CodeDeployAsyncClient</code>
<code>com.amazonaws.services.codedeploy.AmazonCodeDeployClient</code>	<code>software.amazon.awssdk.services.codedeploy.CodeDeployClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.codepipeline.AWSCodePipelineAsyncClient</code>	<code>software.amazon.awssdk.services.codepipeline.CodePipelineAsyncClient</code>
<code>com.amazonaws.services.codepipeline.AWSCodePipelineClient</code>	<code>software.amazon.awssdk.services.codepipeline.CodePipelineClient</code>
<code>com.amazonaws.services.codestar.AWSCodeStarAsyncClient</code>	<code>software.amazon.awssdk.services.codestar.CodeStarAsyncClient</code>
<code>com.amazonaws.services.codestar.AWSCodeStarClient</code>	<code>software.amazon.awssdk.services.codestar.CodeStarClient</code>
<code>com.amazonaws.services.cognitoidentity.AmazonCognitoIdentityAsyncClient</code>	<code>software.amazon.awssdk.services.cognitoidentity.CognitoIdentityAsyncClient</code>
<code>com.amazonaws.services.cognitoidentity.AmazonCognitoIdentityClient</code>	<code>software.amazon.awssdk.services.cognitoidentity.CognitoIdentityClient</code>
<code>com.amazonaws.services.cognito.CognitoIdentityProviderAsyncClient</code>	<code>software.amazon.awssdk.services.cognito.CognitoIdentityProviderAsyncClient</code>
<code>com.amazonaws.services.cognito.CognitoIdentityProviderClient</code>	<code>software.amazon.awssdk.services.cognito.CognitoIdentityProviderClient</code>
<code>com.amazonaws.services.cognitosync.AmazonCognitoSyncAsyncClient</code>	<code>software.amazon.awssdk.services.cognitosync.CognitoSyncAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.cognitosync.AmazonCognitoSyncClient</code>	<code>software.amazon.awssdk.services.cognitosync.CognitoSyncClient</code>
<code>com.amazonaws.services.comprehend.AmazonComprehendAsyncClient</code>	<code>software.amazon.awssdk.services.comprehend.ComprehendAsyncClient</code>
<code>com.amazonaws.services.comprehend.AmazonComprehendClient</code>	<code>software.amazon.awssdk.services.comprehend.ComprehendClient</code>
<code>com.amazonaws.services.config.AmazonConfigAsyncClient</code>	<code>software.amazon.awssdk.services.config.ConfigAsyncClient</code>
<code>com.amazonaws.services.config.AmazonConfigClient</code>	<code>software.amazon.awssdk.services.config.ConfigClient</code>
<code>com.amazonaws.services.connect.AmazonConnectAsyncClient</code>	<code>software.amazon.awssdk.services.connect.ConnectAsyncClient</code>
<code>com.amazonaws.services.connect.AmazonConnectClient</code>	<code>software.amazon.awssdk.services.connect.ConnectClient</code>
<code>com.amazonaws.services.costandusagereport.AWSCostAndUsageReportAsyncClient</code>	<code>software.amazon.awssdk.services.costandusagereport.CostAndUsageReportAsyncClient</code>
<code>com.amazonaws.services.costandusagereport.AWSCostAndUsageReportClient</code>	<code>software.amazon.awssdk.services.costandusagereport.CostAndUsageReportClient</code>
<code>com.amazonaws.services.costexplorer.AWSCostExplorerAsyncClient</code>	<code>software.amazon.awssdk.services.costexplorer.CostExplorerAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.costexplorer.AWSCostExplorerClient</code>	<code>software.amazon.awssdk.services.costexplorer.CostExplorerClient</code>
<code>com.amazonaws.services.databasemigrationservice.AWSDatabaseMigrationServiceAsyncClient</code>	<code>software.amazon.awssdk.services.databasemigration.DatabaseMigrationAsyncClient</code>
<code>com.amazonaws.services.databasemigrationservice.AWSDatabaseMigrationServiceClient</code>	<code>software.amazon.awssdk.services.databasemigration.DatabaseMigrationClient</code>
<code>com.amazonaws.services.datapipeline.DataPipelineAsyncClient</code>	<code>software.amazon.awssdk.services.datapipeline.DataPipelineAsyncClient</code>
<code>com.amazonaws.services.datapipeline.DataPipelineClient</code>	<code>software.amazon.awssdk.services.datapipeline.DataPipelineAsyncClient</code>
<code>com.amazonaws.services.dax.AmazonDaxAsyncClient</code>	<code>software.amazon.awssdk.services.dax.DaxAsyncClient</code>
<code>com.amazonaws.services.dax.AmazonDaxClient</code>	<code>software.amazon.awssdk.services.dax.DaxClient</code>
<code>com.amazonaws.services.devicefarm.AWSDeviceFarmAsyncClient</code>	<code>software.amazon.awssdk.services.devicefarm.DeviceFarmAsyncClient</code>
<code>com.amazonaws.services.devicefarm.AWSDeviceFarmClient</code>	<code>software.amazon.awssdk.services.devicefarm.DeviceFarmClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.directconnect.AmazonDirectConnectAsyncClient</code>	<code>software.amazon.awssdk.services.directconnect.DirectConnectAsyncClient</code>
<code>com.amazonaws.services.directconnect.AmazonDirectConnectClient</code>	<code>software.amazon.awssdk.services.directconnect.DirectConnectClient</code>
<code>com.amazonaws.services.directory.AWSDirectoryServiceAsyncClient</code>	<code>software.amazon.awssdk.services.directory.DirectoryAsyncClient</code>
<code>com.amazonaws.services.directory.AWSDirectoryServiceClient</code>	<code>software.amazon.awssdk.services.directory.DirectoryClient</code>
<code>com.amazonaws.services.dlm.AmazonDLMAAsyncClient</code>	<code>software.amazon.awssdk.services.dlm.DlmAsyncClient</code>
<code>com.amazonaws.services.dlm.AmazonDLMClient</code>	<code>software.amazon.awssdk.services.dlm.DlmClient</code>
<code>com.amazonaws.services.dynamodbv2.AmazonDynamoDBAsyncClient</code>	<code>software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient</code>
<code>com.amazonaws.services.dynamodbv2.AmazonDynamoDBClient</code>	<code>software.amazon.awssdk.services.dynamodb.DynamoDbClient</code>
<code>com.amazonaws.services.dynamodbv2.AmazonDynamoDBStreamsAsyncClient</code>	<code>software.amazon.awssdk.services.dynamodb.streams.DynamoDbStreamsAsyncClient</code>
<code>com.amazonaws.services.dynamodbv2.AmazonDynamoDBStreamsClient</code>	<code>software.amazon.awssdk.services.dynamodb.streams.DynamoDbStreamsClient</code>



1.x Client	2.x Client
<code>com.amazonaws.services.ec2. AmazonEC2AsyncClient</code>	<code>software.amazon.awssdk.serv ices.ec2.Ec2AsyncClient</code>
<code>com.amazonaws.services.ec2. AmazonEC2Client</code>	<code>software.amazon.awssdk.serv ices.ec2.Ec2Client</code>
<code>com.amazonaws.services.ecr. AmazonECRAsyncClient</code>	<code>software.amazon.awssdk.serv ices.ecr.EcrAsyncClient</code>
<code>com.amazonaws.services.ecr. AmazonECRClient</code>	<code>software.amazon.awssdk.serv ices.ecr.EcrClient</code>
<code>com.amazonaws.services.ecs. AmazonECSAsyncClient</code>	<code>software.amazon.awssdk.serv ices.ecs.EcsAsyncClient</code>
<code>com.amazonaws.services.ecs. AmazonECSClient</code>	<code>software.amazon.awssdk.serv ices.ecs.EcsClient</code>
<code>com.amazonaws.services.eks. AmazonEKSAsyncClient</code>	<code>software.amazon.awssdk.serv ices.eks.EksAsyncClient</code>
<code>com.amazonaws.services.eks. AmazonEKSClient</code>	<code>software.amazon.awssdk.serv ices.eks.EksClient</code>
<code>com.amazonaws.services.elas ticache.AmazonElastiCacheAs yncClient</code>	<code>software.amazon.awssdk.serv ices.elasticache.ElastiCach eAsyncClient</code>
<code>com.amazonaws.services.elas ticache.AmazonElastiCacheClient</code>	<code>software.amazon.awssdk.serv ices.elasticache.ElastiCach eClient</code>
<code>com.amazonaws.services.elas ticbeanstalk.AWSElasticBean stalkAsyncClient</code>	<code>software.amazon.awssdk.serv ices.elasticbeanstalk.Elast icBeanstalkAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.elasticbeanstalk.AWSElasticBeanstalkClient</code>	<code>software.amazon.awssdk.services.elasticbeanstalk.ElasticBeanstalkClient</code>
<code>com.amazonaws.services.elasticfilesystem.AmazonElasticFileSystemAsyncClient</code>	<code>software.amazon.awssdk.services.efs.EfsAsyncClient</code>
<code>com.amazonaws.services.elasticfilesystem.AmazonElasticFileSystemClient</code>	<code>software.amazon.awssdk.services.efs.EfsClient</code>
<code>com.amazonaws.services.elasticloadbalancing.AmazonElasticLoadBalancingAsyncClient</code>	<code>software.amazon.awssdk.services.elasticloadbalancing.ElasticLoadBalancingAsyncClient</code>
<code>com.amazonaws.services.elasticloadbalancing.AmazonElasticLoadBalancingClient</code>	<code>software.amazon.awssdk.services.elasticloadbalancing.ElasticLoadBalancingClient</code>
<code>com.amazonaws.services.elasticloadbalancingv2.AmazonElasticLoadBalancingAsyncClient</code>	<code>software.amazon.awssdk.services.elasticloadbalancingv2.ElasticLoadBalancingV2AsyncClient</code>
<code>com.amazonaws.services.elasticloadbalancingv2.AmazonElasticLoadBalancingClient</code>	<code>software.amazon.awssdk.services.elasticloadbalancingv2.ElasticLoadBalancingV2Client</code>
<code>com.amazonaws.services.elasticmapreduce.AmazonElasticMapReduceAsyncClient</code>	<code>software.amazon.awssdk.services.emr.EmrAsyncClient</code>
<code>com.amazonaws.services.elasticmapreduce.AmazonElasticMapReduceClient</code>	<code>software.amazon.awssdk.services.emr.EmrClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.elasticsearch.AWSElasticsearchAsyncClient</code>	<code>software.amazon.awssdk.services.elasticsearch.ElasticsearchAsyncClient</code>
<code>com.amazonaws.services.elasticsearch.AWSElasticsearchClient</code>	<code>software.amazon.awssdk.services.elasticsearch.ElasticsearchClient</code>
<code>com.amazonaws.services.elastictranscoder.AmazonElasticTranscoderAsyncClient</code>	<code>software.amazon.awssdk.services.elastictranscoder.ElasticTranscoderAsyncClient</code>
<code>com.amazonaws.services.elastictranscoder.AmazonElasticTranscoderClient</code>	<code>software.amazon.awssdk.services.elastictranscoder.ElasticTranscoderClient</code>
<code>com.amazonaws.services.fms.AWSFMSAsyncClient</code>	<code>software.amazon.awssdk.services.fms.FmsAsyncClient</code>
<code>com.amazonaws.services.fms.AWSFMSClient</code>	<code>software.amazon.awssdk.services.fms.FmsClient</code>
<code>com.amazonaws.services.gamelift.AmazonGameLiftAsyncClient</code>	<code>software.amazon.awssdk.services.gamelift.GameLiftAsyncClient</code>
<code>com.amazonaws.services.gamelift.AmazonGameLiftClient</code>	<code>software.amazon.awssdk.services.gamelift.GameLiftClient</code>
<code>com.amazonaws.services.glacier.AmazonGlacierAsyncClient</code>	<code>software.amazon.awssdk.services.glacier.GlacierAsyncClient</code>
<code>com.amazonaws.services.glacier.AmazonGlacierClient</code>	<code>software.amazon.awssdk.services.glacier.GlacierClient</code>
<code>com.amazonaws.services.glue.AWSGlueAsyncClient</code>	<code>software.amazon.awssdk.services.glue.GlueAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.glue.AWSGlueClient</code>	<code>software.amazon.awssdk.services.glue.GlueClient</code>
<code>com.amazonaws.services.greengrass.AWSGreengrassAsyncClient</code>	<code>software.amazon.awssdk.services.greengrass.GreengrassAsyncClient</code>
<code>com.amazonaws.services.greengrass.AWSGreengrassClient</code>	<code>software.amazon.awssdk.services.greengrass.GreengrassClient</code>
<code>com.amazonaws.services.guardduty.AmazonGuardDutyAsyncClient</code>	<code>software.amazon.awssdk.services.guardduty.GuardDutyAsyncClient</code>
<code>com.amazonaws.services.guardduty.AmazonGuardDutyClient</code>	<code>software.amazon.awssdk.services.guardduty.GuardDutyClient</code>
<code>com.amazonaws.services.health.AWSHealthAsyncClient</code>	<code>software.amazon.awssdk.services.health.HealthAsyncClient</code>
<code>com.amazonaws.services.health.AWSHealthClient</code>	<code>software.amazon.awssdk.services.health.HealthClient</code>
<code>com.amazonaws.services.identitymanagement.AmazonIdentityManagementAsyncClient</code>	<code>software.amazon.awssdk.services.iam.IamAsyncClient</code>
<code>com.amazonaws.services.identitymanagement.AmazonIdentityManagementClient</code>	<code>software.amazon.awssdk.services.iam.IamClient</code>
<code>com.amazonaws.services.importexport.AmazonImportExportAsyncClient</code>	<i>Deprecated</i>

1.x Client	2.x Client
<code>com.amazonaws.services.importexport.AmazonImportExportClient</code>	<i>Deprecated</i>
<code>com.amazonaws.services.inspector.AmazonInspectorAsyncClient</code>	<code>software.amazon.awssdk.services.inspector.InspectorAsyncClient</code>
<code>com.amazonaws.services.inspector.AmazonInspectorClient</code>	<code>software.amazon.awssdk.services.inspector.InspectorClient</code>
<code>com.amazonaws.services.iot.AWSIoTAsyncClient</code>	<code>software.amazon.awssdk.services.iot.IotAsyncClient</code>
<code>com.amazonaws.services.iot.AWSIoTClient</code>	<code>software.amazon.awssdk.services.iot.IotClient</code>
<code>com.amazonaws.services.iot1clickdevices.AWSIoT1ClickDevicesAsyncClient</code>	<code>software.amazon.awssdk.services.iot1clickdevices.Iot1ClickDevicesAsyncClient</code>
<code>com.amazonaws.services.iot1clickdevices.AWSIoT1ClickDevicesClient</code>	<code>software.amazon.awssdk.services.iot1clickdevices.Iot1ClickDevicesClient</code>
<code>com.amazonaws.services.iot1clickprojects.AWSIoT1ClickProjectsAsyncClient</code>	<code>software.amazon.awssdk.services.iot1clickprojects.Iot1ClickProjectsAsyncClient</code>
<code>com.amazonaws.services.iot1clickprojects.AWSIoT1ClickProjectsClient</code>	<code>software.amazon.awssdk.services.iot1clickprojects.Iot1ClickProjectsClient</code>
<code>com.amazonaws.services.iotanalytics.AWSIoTAnalyticsAsyncClient</code>	<code>software.amazon.awssdk.services.iotanalytics.IotAnalyticsAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.iotanalytics.AWSIoTAnalyticsClient</code>	<code>software.amazon.awssdk.services.iotanalytics.IotAnalyticsClient</code>
<code>com.amazonaws.services.iotdata.AWSIoTDataAsyncClient</code>	<code>software.amazon.awssdk.services.iotdata.IotDataAsyncClient</code>
<code>com.amazonaws.services.iotdata.AWSIoTDataClient</code>	<code>software.amazon.awssdk.services.iotdata.IotDataClient</code>
<code>com.amazonaws.services.iotjobsdataplane.AWSIoTJobsDataPlaneAsyncClient</code>	<code>software.amazon.awssdk.services.iotdataplane.IotDataPlaneAsyncClient</code>
<code>com.amazonaws.services.iotjobsdataplane.AWSIoTJobsDataPlaneClient</code>	<code>software.amazon.awssdk.services.iotdataplane.IotDataPlaneClient</code>
<code>com.amazonaws.services.kinesis.AmazonKinesisAsyncClient</code>	<code>software.amazon.awssdk.services.kinesis.KinesisAsyncClient</code>
<code>com.amazonaws.services.kinesis.AmazonKinesisClient</code>	<code>software.amazon.awssdk.services.kinesis.KinesisClient</code>
<code>com.amazonaws.services.kinesisanalytics.AmazonKinesisAnalyticsAsyncClient</code>	<code>software.amazon.awssdk.services.kinesisanalytics.KinesisAnalyticsAsyncClient</code>
<code>com.amazonaws.services.kinesisanalytics.AmazonKinesisAnalyticsClient</code>	<code>software.amazon.awssdk.services.kinesisanalytics.KinesisAnalyticsClient</code>
<code>com.amazonaws.services.kinesisfirehose.AmazonKinesisFirehoseAsyncClient</code>	<code>software.amazon.awssdk.services.firehose.FirehoseAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.kinesisfirehose.AmazonKinesisFirehoseClient</code>	<code>software.amazon.awssdk.services.firehose.FirehoseClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoArchivedMediaAsyncClient</code>	<code>software.amazon.awssdk.services.kinesisvideoarchivedmedia.KinesisVideoArchivedMediaAsyncClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoArchivedMediaClient</code>	<code>software.amazon.awssdk.services.kinesisvideoarchivedmedia.KinesisVideoArchivedMediaClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoAsyncClient</code>	<code>software.amazon.awssdk.services.kinesisvideo.KinesisVideoAsyncClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoClient</code>	<code>software.amazon.awssdk.services.kinesisvideo.KinesisVideoClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoMediaAsyncClient</code>	<code>software.amazon.awssdk.services.kinesisvideomedia.KinesisVideoMediaAsyncClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoMediaClient</code>	<code>software.amazon.awssdk.services.kinesisvideomedia.KinesisVideoMediaClient</code>
<code>com.amazonaws.services.kinesisvideo.AmazonKinesisVideoPutMediaClient</code>	Not Supported
<code>com.amazonaws.services.kms.AWSKMSAsyncClient</code>	<code>software.amazon.awssdk.services.kms.KmsAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.kms.AWSKMSClient</code>	<code>software.amazon.awssdk.services.kms.KmsClient</code>
<code>com.amazonaws.services.lambda.AWSLambdaAsyncClient</code>	<code>software.amazon.awssdk.services.lambda.LambdaAsyncClient</code>
<code>com.amazonaws.services.lambda.AWSLambdaClient</code>	<code>software.amazon.awssdk.services.lambda.LambdaClient</code>
<code>com.amazonaws.services.lexmodelbuilding.AmazonLexModelBuildingAsyncClient</code>	<code>software.amazon.awssdk.services.lexmodelbuilding.LexModelBuildingAsyncClient</code>
<code>com.amazonaws.services.lexmodelbuilding.AmazonLexModelBuildingClient</code>	<code>software.amazon.awssdk.services.lexmodelbuilding.LexModelBuildingClient</code>
<code>com.amazonaws.services.lexruntime.AmazonLexRuntimeAsyncClient</code>	<code>software.amazon.awssdk.services.lexruntime.LexRuntimeAsyncClient</code>
<code>com.amazonaws.services.lexruntime.AmazonLexRuntimeClient</code>	<code>software.amazon.awssdk.services.lexruntime.LexRuntimeClient</code>
<code>com.amazonaws.services.lightsail.AmazonLightsailAsyncClient</code>	<code>software.amazon.awssdk.services.lightsail.LightsailAsyncClient</code>
<code>com.amazonaws.services.lightsail.AmazonLightsailClient</code>	<code>software.amazon.awssdk.services.lightsail.LightsailClient</code>
<code>com.amazonaws.services.logs.AWSLogsAsyncClient</code>	<code>software.amazon.awssdk.services.logs.LogsAsyncClient</code>
<code>com.amazonaws.services.logs.AWSLogsClient</code>	<code>software.amazon.awssdk.services.logs.LogsClient</code>



1.x Client	2.x Client
<code>com.amazonaws.services.machinelearning.AmazonMachineLearningAsyncClient</code>	<code>software.amazon.awssdk.services.machinelearning.MachineLearningAsyncClient</code>
<code>com.amazonaws.services.machinelearning.AmazonMachineLearningClient</code>	<code>software.amazon.awssdk.services.machinelearning.MachineLearningClient</code>
<code>com.amazonaws.services.macie.AmazonMacieAsyncClient</code>	<code>software.amazon.awssdk.services.macie.MacieAsyncClient</code>
<code>com.amazonaws.services.macie.AmazonMacieClient</code>	<code>software.amazon.awssdk.services.macie.MacieClient</code>
<code>com.amazonaws.services.marketplacecommerceanalytics.AWSMarketplaceCommerceAnalyticsAsyncClient</code>	<code>software.amazon.awssdk.services.marketplacecommerceanalytics.MarketplaceCommerceAnalyticsAsyncClient</code>
<code>com.amazonaws.services.marketplacecommerceanalytics.AWSMarketplaceCommerceAnalyticsClient</code>	<code>software.amazon.awssdk.services.marketplacecommerceanalytics.MarketplaceCommerceAnalyticsClient</code>
<code>com.amazonaws.services.marketplaceentitlement.AWSMarketplaceEntitlementAsyncClient</code>	<code>software.amazon.awssdk.services.marketplaceentitlement.MarketplaceEntitlementAsyncClient</code>
<code>com.amazonaws.services.marketplaceentitlement.AWSMarketplaceEntitlementClient</code>	<code>software.amazon.awssdk.services.marketplaceentitlement.MarketplaceEntitlementClient</code>
<code>com.amazonaws.services.marketplacemetering.AWSMarketplaceMeteringAsyncClient</code>	<code>software.amazon.awssdk.services.marketplacemetering.MarketplaceMeteringAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.marketplacemetering.AWSMarketplaceMeteringClient</code>	<code>software.amazon.awssdk.services.marketplacemetering.MarketplaceMeteringClient</code>
<code>com.amazonaws.services.mediaconvert.AWSMediaConvertAsyncClient</code>	<code>software.amazon.awssdk.services.mediaconvert.MediaConvertAsyncClient</code>
<code>com.amazonaws.services.mediaconvert.AWSMediaConvertClient</code>	<code>software.amazon.awssdk.services.mediaconvert.MediaConvertClient</code>
<code>com.amazonaws.services.medialive.AWSMediaLiveAsyncClient</code>	<code>software.amazon.awssdk.services.medialive.MediaLiveAsyncClient</code>
<code>com.amazonaws.services.medialive.AWSMediaLiveClient</code>	<code>software.amazon.awssdk.services.medialive.MediaLiveClient</code>
<code>com.amazonaws.services.mediapackage.AWSMediaPackageAsyncClient</code>	<code>software.amazon.awssdk.services.mediapackage.MediaPackageAsyncClient</code>
<code>com.amazonaws.services.mediapackage.AWSMediaPackageClient</code>	<code>software.amazon.awssdk.services.mediapackage.MediaPackageClient</code>
<code>com.amazonaws.services.mediastore.AWSMediaStoreAsyncClient</code>	<code>software.amazon.awssdk.services.mediastore.MediaStoreAsyncClient</code>
<code>com.amazonaws.services.mediastore.AWSMediaStoreClient</code>	<code>software.amazon.awssdk.services.mediastore.MediaStoreClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.mediastoredata.AWSMediaStoreDataAsyncClient</code>	<code>software.amazon.awssdk.services.mediastoredata.MediaStoreDataAsyncClient</code>
<code>com.amazonaws.services.mediastoredata.AWSMediaStoreDataClient</code>	<code>software.amazon.awssdk.services.mediastoredata.MediaStoreDataClient</code>
<code>com.amazonaws.services.mediatailor.AWSMediaTailorAsyncClient</code>	<code>software.amazon.awssdk.services.mediatailor.MediaTailorAsyncClient</code>
<code>com.amazonaws.services.mediatailor.AWSMediaTailorClient</code>	<code>software.amazon.awssdk.services.mediatailor.MediaTailorClient</code>
<code>com.amazonaws.services.migrationhub.AWSMigrationHubAsyncClient</code>	<code>software.amazon.awssdk.services.migrationhub.MigrationHubAsyncClient</code>
<code>com.amazonaws.services.migrationhub.AWSMigrationHubClient</code>	<code>software.amazon.awssdk.services.migrationhub.MigrationHubClient</code>
<code>com.amazonaws.services.mobile.AWSMobileAsyncClient</code>	<code>software.amazon.awssdk.services.mobile.MobileAsyncClient</code>
<code>com.amazonaws.services.mobile.AWSMobileClient</code>	<code>software.amazon.awssdk.services.mobile.MobileClient</code>
<code>com.amazonaws.services.mq.AmazonMQAsyncClient</code>	<code>software.amazon.awssdk.services.mq.MqAsyncClient</code>
<code>com.amazonaws.services.mq.AmazonMQClient</code>	<code>software.amazon.awssdk.services.mq.MqClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.mturk.AmazonMTurkAsyncClient</code>	<code>software.amazon.awssdk.services.mturk.MTurkAsyncClient</code>
<code>com.amazonaws.services.mturk.AmazonMTurkClient</code>	<code>software.amazon.awssdk.services.mturk.MTurkClient</code>
<code>com.amazonaws.services.neptune.AmazonNeptuneAsyncClient</code>	<code>software.amazon.awssdk.services.neptune.NeptuneAsyncClient</code>
<code>com.amazonaws.services.neptune.AmazonNeptuneClient</code>	<code>software.amazon.awssdk.services.neptune.NeptuneClient</code>
<code>com.amazonaws.services.opsworks.AWSOpsWorksAsyncClient</code>	<code>software.amazon.awssdk.services.opsworks.OpsWorksAsyncClient</code>
<code>com.amazonaws.services.opsworks.AWSOpsWorksClient</code>	<code>software.amazon.awssdk.services.opsworks.OpsWorksClient</code>
<code>com.amazonaws.services.opsworkscm.AWSOpsWorksCMAsyncClient</code>	<code>software.amazon.awssdk.services.opsworkscm.OpsWorksCmAsyncClient</code>
<code>com.amazonaws.services.opsworkscm.AWSOpsWorksCMClient</code>	<code>software.amazon.awssdk.services.opsworkscm.OpsWorksCmClient</code>
<code>com.amazonaws.services.organizations.AWSOrganizationsAsyncClient</code>	<code>software.amazon.awssdk.services.organizations.OrganizationsAsyncClient</code>
<code>com.amazonaws.services.organizations.AWSOrganizationsClient</code>	<code>software.amazon.awssdk.services.organizations.OrganizationsClient</code>
<code>com.amazonaws.services.pi.AWSPIAsyncClient</code>	<code>software.amazon.awssdk.services.pi.PiAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.pi.AWSPIClient</code>	<code>software.amazon.awssdk.services.pi.PiClient</code>
<code>com.amazonaws.services.pinpoint.AmazonPinpointAsyncClient</code>	<code>software.amazon.awssdk.services.pinpoint.PinpointAsyncClient</code>
<code>com.amazonaws.services.pinpoint.AmazonPinpointClient</code>	<code>software.amazon.awssdk.services.pinpoint.PinpointClient</code>
<code>com.amazonaws.services.polly.AmazonPollyAsyncClient</code>	<code>software.amazon.awssdk.services.polly.PollyAsyncClient</code>
<code>com.amazonaws.services.polly.AmazonPollyClient</code>	<code>software.amazon.awssdk.services.polly.PollyClient</code>
<code>com.amazonaws.services.pricing.AWS PricingAsyncClient</code>	<code>software.amazon.awssdk.services.pricing.PricingAsyncClient</code>
<code>com.amazonaws.services.pricing.AWS PricingClient</code>	<code>software.amazon.awssdk.services.pricing.PricingClient</code>
<code>com.amazonaws.services.rds.AmazonRDSAsyncClient</code>	<code>software.amazon.awssdk.services.rds.RdsAsyncClient</code>
<code>com.amazonaws.services.rds.AmazonRDSClient</code>	<code>software.amazon.awssdk.services.rds.RdsClient</code>
<code>com.amazonaws.services.redshift.AmazonRedshiftAsyncClient</code>	<code>software.amazon.awssdk.services.redshift.RedshiftAsyncClient</code>
<code>com.amazonaws.services.redshift.AmazonRedshiftClient</code>	<code>software.amazon.awssdk.services.redshift.RedshiftClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.rekognition.AmazonRekognitionAsyncClient</code>	<code>software.amazon.awssdk.services.rekognition.RekognitionAsyncClient</code>
<code>com.amazonaws.services.rekognition.AmazonRekognitionClient</code>	<code>software.amazon.awssdk.services.rekognition.RekognitionClient</code>
<code>com.amazonaws.services.resourcegroups.AWSResourceGroupsAsyncClient</code>	<code>software.amazon.awssdk.services.resourcegroups.ResourceGroupsAsyncClient</code>
<code>com.amazonaws.services.resourcegroups.AWSResourceGroupsClient</code>	<code>software.amazon.awssdk.services.resourcegroups.ResourceGroupsClient</code>
<code>com.amazonaws.services.resourcegroupstaggingapi.AWSResourceGroupsTaggingAPIAsyncClient</code>	<code>software.amazon.awssdk.services.resourcegroupstaggingapi.ResourceGroupsTaggingApiAsyncClient</code>
<code>com.amazonaws.services.resourcegroupstaggingapi.AWSResourceGroupsTaggingAPIClient</code>	<code>software.amazon.awssdk.services.resourcegroupstaggingapi.ResourceGroupsTaggingApiClient</code>
<code>com.amazonaws.services.route53.AmazonRoute53AsyncClient</code>	<code>software.amazon.awssdk.services.route53.Route53AsyncClient</code>
<code>com.amazonaws.services.route53.AmazonRoute53Client</code>	<code>software.amazon.awssdk.services.route53.Route53Client</code>
<code>com.amazonaws.services.route53domains.AmazonRoute53DomainsAsyncClient</code>	<code>software.amazon.awssdk.services.route53domains.Route53DomainsAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.route53domains.AmazonRoute53DomainsClient</code>	<code>software.amazon.awssdk.services.route53domains.Route53DomainsClient</code>
<code>com.amazonaws.services.s3.AmazonS3Client</code>	<code>software.amazon.awssdk.services.s3.S3Client</code>
<code>com.amazonaws.services.sagemaker.AmazonSageMakerAsyncClient</code>	<code>software.amazon.awssdk.services.sagemaker.SageMakerAsyncClient</code>
<code>com.amazonaws.services.sagemaker.AmazonSageMakerClient</code>	<code>software.amazon.awssdk.services.sagemaker.SageMakerClient</code>
<code>com.amazonaws.services.sagemakeruntime.AmazonSageMakerRuntimeAsyncClient</code>	<code>software.amazon.awssdk.services.sagemakeruntime.SageMakerRuntimeAsyncClient</code>
<code>com.amazonaws.services.sagemakeruntime.AmazonSageMakerRuntimeClient</code>	<code>software.amazon.awssdk.services.sagemakeruntime.SageMakerRuntimeClient</code>
<code>com.amazonaws.services.secretsmanager.AWSSecretsManagerAsyncClient</code>	<code>software.amazon.awssdk.services.secretsmanager.SecretsManagerAsyncClient</code>
<code>com.amazonaws.services.secretsmanager.AWSSecretsManagerClient</code>	<code>software.amazon.awssdk.services.secretsmanager.SecretsManagerClient</code>
<code>com.amazonaws.services.securitytoken.AWSSecurityTokenServiceAsyncClient</code>	<code>software.amazon.awssdk.services.sts.StsAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.securitytoken.AWSSecurityTokenServiceClient</code>	<code>software.amazon.awssdk.services.sts.StsClient</code>
<code>com.amazonaws.services.serverlessapplicationrepository.AWSServerlessApplicationRepositoryAsyncClient</code>	<code>software.amazon.awssdk.services.serverlessapplicationrepository.ServerlessApplicationRepositoryAsyncClient</code>
<code>com.amazonaws.services.serverlessapplicationrepository.AWSServerlessApplicationRepositoryClient</code>	<code>software.amazon.awssdk.services.serverlessapplicationrepository.ServerlessApplicationRepositoryClient</code>
<code>com.amazonaws.services.servermigration.AWSServerMigrationAsyncClient</code>	<code>software.amazon.awssdk.services.sms.SmsAsyncClient</code>
<code>com.amazonaws.services.servermigration.AWSServerMigrationClient</code>	<code>software.amazon.awssdk.services.sms.SmsClient</code>
<code>com.amazonaws.services.servicecatalog.AWSServiceCatalogAsyncClient</code>	<code>software.amazon.awssdk.services.servicecatalog.ServiceCatalogAsyncClient</code>
<code>com.amazonaws.services.servicecatalog.AWSServiceCatalogClient</code>	<code>software.amazon.awssdk.services.servicecatalog.ServiceCatalogClient</code>
<code>com.amazonaws.services.servicediscovery.AWSServiceDiscoveryAsyncClient</code>	<code>software.amazon.awssdk.services.servicediscovery.ServiceDiscoveryAsyncClient</code>



1.x Client	2.x Client
<code>com.amazonaws.services.servicediscovery.AWSServiceDiscoveryClient</code>	<code>software.amazon.awssdk.services.servicediscovery.ServiceDiscoveryClient</code>
<code>com.amazonaws.services.shield.AWSShieldAsyncClient</code>	<code>software.amazon.awssdk.services.shield.ShieldAsyncClient</code>
<code>com.amazonaws.services.shield.AWSShieldClient</code>	<code>software.amazon.awssdk.services.shield.ShieldClient</code>
<code>com.amazonaws.services.simplesdb.AmazonSimpleDBAsyncClient</code>	<code>software.amazon.awssdk.services.simplesdb.SimpleDbAsyncClient</code>
<code>com.amazonaws.services.simplesdb.AmazonSimpleDBClient</code>	<code>software.amazon.awssdk.services.simplesdb.SimpleDbClient</code>
<code>com.amazonaws.services.simpleemail.AmazonSimpleEmailServiceAsyncClient</code>	<code>software.amazon.awssdk.services.ses.SesAsyncClient</code>
<code>com.amazonaws.services.simpleemail.AmazonSimpleEmailServiceClient</code>	<code>software.amazon.awssdk.services.ses.SesClient</code>
<code>com.amazonaws.services.simplesystemsmanagement.AWSSimpleSystemsManagementAsyncClient</code>	<code>software.amazon.awssdk.services.ssm.SsmAsyncClient</code>
<code>com.amazonaws.services.simplesystemsmanagement.AWSSimpleSystemsManagementClient</code>	<code>software.amazon.awssdk.services.ssm.SsmClient</code>
<code>com.amazonaws.services.simpleswf.AmazonSimpleWorkflowAsyncClient</code>	<code>software.amazon.awssdk.services.swf.SwfAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.simpleworkflow.AmazonSimpleWorkflowClient</code>	<code>software.amazon.awssdk.services.swf.SwfClient</code>
<code>com.amazonaws.services.snowball.AmazonSnowballAsyncClient</code>	<code>software.amazon.awssdk.services.snowball.SnowballAsyncClient</code>
<code>com.amazonaws.services.snowball.AmazonSnowballClient</code>	<code>software.amazon.awssdk.services.snowball.SnowballClient</code>
<code>com.amazonaws.services.sns.AmazonSNSAsyncClient</code>	<code>software.amazon.awssdk.services.sns.SnsAsyncClient</code>
<code>com.amazonaws.services.sns.AmazonSNSClient</code>	<code>software.amazon.awssdk.services.sns.SnsClient</code>
<code>com.amazonaws.services.sqs.AmazonSQSAsyncClient</code>	<code>software.amazon.awssdk.services.sqs.SqsAsyncClient</code>
<code>com.amazonaws.services.sqs.AmazonSQSClient</code>	<code>software.amazon.awssdk.services.sqs.SqsClient</code>
<code>com.amazonaws.services.stepfunctions.AWSStepFunctionsAsyncClient</code>	<code>software.amazon.awssdk.services.sfn.SfnAsyncClient</code>
<code>com.amazonaws.services.stepfunctions.AWSStepFunctionsClient</code>	<code>software.amazon.awssdk.services.sfn.SfnClient</code>
<code>com.amazonaws.services.storagegateway.AWSStorageGatewayAsyncClient</code>	<code>software.amazon.awssdk.services.storagegateway.StorageGatewayAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.storagegateway.AWSStorageGatewayClient</code>	<code>software.amazon.awssdk.services.storagegateway.StorageGatewayClient</code>
<code>com.amazonaws.services.support.AWSSupportAsyncClient</code>	<code>software.amazon.awssdk.services.support.SupportAsyncClient</code>
<code>com.amazonaws.services.support.AWSSupportClient</code>	<code>software.amazon.awssdk.services.support.SupportClient</code>
<code>com.amazonaws.services.transcribe.AmazonTranscribeAsyncClient</code>	<code>software.amazon.awssdk.services.transcribe.TranscribeAsyncClient</code>
<code>com.amazonaws.services.transcribe.AmazonTranscribeClient</code>	<code>software.amazon.awssdk.services.transcribe.TranscribeClient</code>
<code>com.amazonaws.services.translate.AmazonTranslateAsyncClient</code>	<code>software.amazon.awssdk.services.translate.TranslateAsyncClient</code>
<code>com.amazonaws.services.translate.AmazonTranslateClient</code>	<code>software.amazon.awssdk.services.translate.TranslateClient</code>
<code>com.amazonaws.services.waf.AWSWAFAsyncClient</code>	<code>software.amazon.awssdk.services.waf.WafAsyncClient</code>
<code>com.amazonaws.services.waf.AWSWAFClient</code>	<code>software.amazon.awssdk.services.waf.WafClient</code>
<code>com.amazonaws.services.waf.AWSWAFRegionalAsyncClient</code>	<code>software.amazon.awssdk.services.waf.regional.WafRegionalAsyncClient</code>

1.x Client	2.x Client
<code>com.amazonaws.services.waf.AWSWAFRegionalClient</code>	<code>software.amazon.awssdk.services.waf.regional.WafRegionalClient</code>
<code>com.amazonaws.services.workdocs.AmazonWorkDocsAsyncClient</code>	<code>software.amazon.awssdk.services.workdocs.WorkDocsAsyncClient</code>
<code>com.amazonaws.services.workdocs.AmazonWorkDocsClient</code>	<code>software.amazon.awssdk.services.workdocs.WorkDocsClient</code>
<code>com.amazonaws.services.workmail.AmazonWorkMailAsyncClient</code>	<code>software.amazon.awssdk.services.workmail.WorkMailAsyncClient</code>
<code>com.amazonaws.services.workmail.AmazonWorkMailClient</code>	<code>software.amazon.awssdk.services.workmail.WorkMailClient</code>
<code>com.amazonaws.services.workspaces.AmazonWorkspacesAsyncClient</code>	<code>software.amazon.awssdk.services.workspaces.WorkSpacesAsyncClient</code>
<code>com.amazonaws.services.workspaces.AmazonWorkspacesClient</code>	<code>software.amazon.awssdk.services.workspaces.WorkSpacesClient</code>
<code>com.amazonaws.services.xray.AWSXRayAsyncClient</code>	<code>software.amazon.awssdk.services.xray.XRayAsyncClient</code>
<code>com.amazonaws.services.xray.AWSXRayClient</code>	<code>software.amazon.awssdk.services.xray.XRayClient</code>

## Client creation defaults

In version 2.x, the following changes have been made to the default client creation logic.

- The default credential provider chain for S3 no longer includes anonymous credentials. You must manually specify anonymous access to S3 by using the `AnonymousCredentialsProvider`.
- The following environment variables related to default client creation are different.

1.x	2.x
<code>AWS_CBOR_DISABLED</code>	<code>CBOR_ENABLED</code>
<code>AWS_ION_BINARY_DISABLE</code>	<code>BINARY_ION_ENABLED</code>

- The following system properties related to default client creation are different.

1.x	2.x
<code>com.amazonaws.sdk.disableEc2Metadata</code>	<code>aws.disableEc2Metadata</code>
<code>com.amazonaws.sdk.ec2MetadataServiceEndpointOverride</code>	<code>aws.ec2MetadataServiceEndpoint</code>
<code>com.amazonaws.sdk.disableCbor</code>	<code>aws.cborEnabled</code>
<code>com.amazonaws.sdk.disableIonBinary</code>	<code>aws.binaryIonEnabled</code>

- Version 2.x does not support the following system properties.

1.x
<code>com.amazonaws.sdk.disableCertChecking</code>
<code>com.amazonaws.sdk.enableDefaultMetrics</code>
<code>com.amazonaws.sdk.enableThrottledRetry</code>
<code>com.amazonaws.regions.RegionUtils.fileOverride</code>
<code>com.amazonaws.regions.RegionUtils.disableRemote</code>
<code>com.amazonaws.services.s3.disableImplicitGlobalClients</code>

## 1.x

```
com.amazonaws.sdk.enableInRegionOptimizedMode
```

- Loading Region configuration from a custom `endpoints.json` file is no longer supported.

## Client configuration

In 1.x, SDK client configuration was modified by setting a `ClientConfiguration` instance on the client or client builder. In version 2.x, the client configuration is split into separate configuration classes. With the separate configuration classes, you can configure different HTTP clients for async versus synchronous clients but still use the same `ClientOverrideConfiguration` class.

### Example of client configuration in version 1.x

```
AmazonDynamoDBClientBuilder.standard()
.withClientConfiguration(clientConfiguration)
.build()
```

### Example of synchronous client configuration in version 2.x

```
ProxyConfiguration.Builder proxyConfig = ProxyConfiguration.builder();

ApacheHttpClient.Builder httpClientBuilder =
 ApacheHttpClient.builder()
 .proxyConfiguration(proxyConfig.build());

ClientOverrideConfiguration.Builder overrideConfig =
 ClientOverrideConfiguration.builder();

DynamoDbClient client =
 DynamoDbClient.builder()
 .httpClientBuilder(httpClientBuilder)
 .overrideConfiguration(overrideConfig.build())
 .build();
```

### Example of asynchronous client configuration in version 2.x

```
NettyNioAsyncHttpClient.Builder httpClientBuilder =
 NettyNioAsyncHttpClient.builder();
```

```
ClientOverrideConfiguration.Builder overrideConfig =
 ClientOverrideConfiguration.builder();

ClientAsyncConfiguration.Builder asyncConfig =
 ClientAsyncConfiguration.builder();

DynamoDbAsyncClient client =
 DynamoDbAsyncClient.builder()
 .httpClientBuilder(httpClientBuilder)
 .overrideConfiguration(overrideConfig.build())
 .asyncConfiguration(asyncConfig.build())
 .build();
```

HTTP clients

Notable changes

- In version 2.x, you can change which HTTP client to use at runtime by specifying an implementation using `clientBuilder.httpClientBuilder`.
- When you pass an HTTP client by using `clientBuilder.httpClient` to a service client builder, the HTTP client is not closed by default if the service client closes. This allows you to share HTTP clients between service clients.
- Asynchronous HTTP clients now use non-blocking IO.
- Some operations now use HTTP/2 for improved performance.

Settings changes

Setting	1.x	2.x Sync, Apache	2.x Async, Netty
	ClientCon figuration clientConfig = new ClientCon figuration()	ApacheHtt pClient.B uilder httpClien tBuilder = ApacheHtt pClient.b uilder()	NettyNioA syncHttpC lient.Builder httpClient tBuilder = NettyNioA syncHttpC lient.builder()

Setting	1.x	2.x Sync, Apache	2.x Async, Netty
Max connections	<pre>clientConfig.setMaxConnections(...) clientConfig.withMaxConnections(...)</pre>	<pre>httpClientBuilder.maxConnections(...)</pre>	<pre>httpClientBuilder.maxConcurrency(...)</pre>
Connection timeout	<pre>clientConfig.setConnectionTimeout(...) clientConfig.withConnectionTimeout(...)</pre>	<pre>httpClientBuilder.connectionTimeout(...) httpClientBuilder.connectionAcquisitionTimeout(...)</pre>	<pre>httpClientBuilder.connectionTimeout(...)</pre>
Socket timeout	<pre>clientConfig.setSocketTimeout(...) clientConfig.withSocketTimeout(...)</pre>	<pre>httpClientBuilder.socketTimeout(...)</pre>	<pre>httpClientBuilder.writeTimeout(...) httpClientBuilder.readTimeout(...)</pre>
Connection TTL	<pre>clientConfig.setConnectionTTL(...) clientConfig.withConnectionTTL(...)</pre>	<pre>httpClientBuilder.connectionTimeToLive(...)</pre>	<pre>httpClientBuilder.connectionTimeToLive(...)</pre>



Setting	1.x	2.x Sync, Apache	2.x Async, Netty
Connection max idle	<pre>clientCon fig.setCo nnectionM axIdleMil lis(...) clientCon fig.withC onnection MaxIdleMi llis(...)</pre>	<pre>httpClien tBuilder. connectio nMaxIdleT ime(...)</pre>	<pre>httpClien tBuilder. connectio nMaxIdleT ime(...)</pre>
Validate after inactivity	<pre>clientCon fig.setVa lidaateAft erInactiv ityMillis(...) clientConfig .withVali dateAfter Inactivit yMillis(...)</pre>	Not supported ( <a href="#">Request Feature</a> )	Not supported ( <a href="#">Request Feature</a> )
Local address	<pre>clientCon fig.setLo calAddress(...) clientConfi g.withLoc alAddress(...)</pre>	<pre>httpClien tBuilder. localAddr ess(...)</pre>	<a href="#">Not supported</a>
Expect-continue enabled	<pre>clientCon fig.setUs eExpectCo ntinue(...) clientConfig.wi thUseExpe ctContinue(...)</pre>	<pre>httpClien tBuilder. expectCon tinueEnab led(...)</pre>	Not supported ( <a href="#">Request Feature</a> )

Setting	1.x	2.x Sync, Apache	2.x Async, Netty
Connection reaper	<pre>clientConfig.setUseReaper(...) clientConfig.withReaper(...)</pre>	<pre>httpClientBuilder.useIdleConnectionReaper(...)</pre>	<pre>httpClientBuilder.useIdleConnectionReaper(...)</pre>
	<pre>AmazonDynamoDBClientBuilder     .standard()     .withClientConfiguration(         clientConfiguration)     .build()</pre>	<pre>DynamoDbClient.builder()     .httpClientBuilder(         httpClientBuilder)     .build()</pre>	<pre>DynamoDbAsyncClient.builder()     .httpClientBuilder(httpClientBuilder)     .build()</pre>

## HTTP client proxies

Settings	1.x	2.x Sync, Apache	2.x Async, Netty
	<pre>ClientConfiguration     clientConfig =         new ClientConfiguration()</pre>	<pre>ProxyConfiguration     .Builder     proxyConfig =         ProxyConfiguration     .builder()</pre>	<pre>ProxyConfiguration     .Builder     proxyConfig =         ProxyConfiguration     .builder()</pre>
Proxy host	<pre>clientConfig.setProxyHost(...) clientConfig.withProxyHost(...)</pre>	<pre>proxyConfig.endpoint(...)</pre>	<pre>proxyConfig.host(...)</pre>

Settings	1.x	2.x Sync, Apache	2.x Async, Netty
Proxy port	<pre>clientConfig.setProxyPort(...) clientConfig.withProxyPort(...)</pre>	<pre>proxyConfig.endpoint(...)</pre>  <a href="#">Proxy port</a> is embedded in endpoint	<pre>proxyConfig.port(...)</pre>
Proxy username	<pre>clientConfig.setProxyUsername(...) clientConfig.withProxyUsername(...)</pre>	<pre>proxyConfig.username(...)</pre>	<pre>proxyConfig.username(...)</pre>
Proxy password	<pre>clientConfig.setProxyPassword(...) clientConfig.withProxyPassword(...)</pre>	<pre>proxyConfig.password(...)</pre>	<pre>proxyConfig.password(...)</pre>
Proxy domain	<pre>clientConfig.setProxyDomain(...) clientConfig.withProxyDomain(...)</pre>	<pre>proxyConfig.ntlmDomain(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )

Settings	1.x	2.x Sync, Apache	2.x Async, Netty
Proxy workstation	<pre>clientConfig.setProxyWorkstation(...) clientConfig.withProxyWorkstation(...)</pre>	<pre>proxyConfig.ntlmWorkstation(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
Proxy authentication methods	<pre>clientConfig.setProxyAuthenticationMethods(...) clientConfig.withProxyAuthenticationMethods(...)</pre>	<a href="#">Not Supported</a>	Not Supported ( <a href="#">Request Feature</a> )
Preemptive basic proxy authentication	<pre>clientConfig.setPreemptiveBasicProxyAuth(...) clientConfig.withPreemptiveBasicProxyAuth(...)</pre>	<pre>proxyConfig.preemptiveBasicAuthenticationEnabled(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )

Settings	1.x	2.x Sync, Apache	2.x Async, Netty
Non-proxy hosts	<pre>clientConfig.setNonProxyHosts(...) clientConfig.withNonProxyHosts(...)</pre>	<pre>proxyConfiguration.nonProxyHosts(...)</pre>	<pre>proxyConfiguration.nonProxyHosts(...)</pre>
Disable socket proxy	<pre>clientConfig.setDisableSocketProxy(...) clientConfig.withDisableSocketProxy(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )	Not Supported ( <a href="#">Request Feature</a> )
	<pre>AmazonDynamoDBClientBuilder.standard()     .withClientConfiguration(clientConfiguration)     .build()</pre>	<pre>httpClientBuilder.proxyConfiguration(proxyConfiguration).build()</pre>	<pre>httpClientBuilder.proxyConfiguration(proxyConfiguration).build()</pre>

## Client overrides

Setting	1.x	2.x
	<pre>ClientConfiguration clientConfig =</pre>	<pre>ClientOverrideConfiguration.Builder overrideConfig =</pre>

Setting	1.x	2.x
	<code>new ClientConfiguration()</code>	<code>ClientOverrideConfiguration.builder()</code>
User agent prefix	<code>clientConfig.setUserAgentPrefix(...)</code> <code>clientConfig.withUserAgentPrefix(...)</code>	<code>overrideConfig.advancedOption(SdkAdvancedClientOption.USER_AGENT_PREFIX, ...)</code>
User agent suffix	<code>clientConfig.setUserAgentSuffix(...)</code> <code>clientConfig.withUserAgentSuffix(...)</code>	<code>overrideConfig.advancedOption(SdkAdvancedClientOption.USER_AGENT_SUFFIX, ...)</code>
Signer	<code>clientConfig.setSignerOverride(...)</code> <code>clientConfig.withSignerOverride(...)</code>	<code>overrideConfig.advancedOption(SdkAdvancedClientOption.SIGNER, ...)</code>
Additional headers	<code>clientConfig.addHeader(...)</code> <code>clientConfig.withHeader(...)</code>	<code>overrideConfig.putHeader(...)</code>
Request timeout	<code>clientConfig.setRequestTimeout(...)</code> <code>clientConfig.withRequestTimeout(...)</code>	<code>overrideConfig.apiCallAttemptTimeout(...)</code>

Setting	1.x	2.x
Client execution timeout	<pre>clientConfig.setClientExecutionTimeout(...) clientConfig.withClientExecutionTimeout(...)</pre>	<pre>overrideConfig.apiCallTimeout(...)</pre>
Use Gzip	<pre>clientConfig.setUseGzip(...) clientConfig.withGzip(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
Socket buffer size hint	<pre>clientConfig.setSocketBufferSizeHints(...) clientConfig.withSocketBufferSizeHints(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
Cache response metadata	<pre>clientConfig.setCacheResponseMetadata(...) clientConfig.withCacheResponseMetadata(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
Response metadata cache size	<pre>clientConfig.setResponseMetadataCacheSize(...) clientConfig.withResponseMetadataCacheSize(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )

Setting	1.x	2.x
DNS resolver	<pre>clientConfig.setDnsResolver(...) clientConfig.withDnsResolver(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
TCP keepalive	<pre>clientConfig.setUseTcpKeepAlive(...) clientConfig.withTcpKeepAlive(...)</pre>	This option is now in the HTTP Client configuration       - <code>ApacheHttpClient.builder().tcpKeepAlive(true)</code>     - <code>NettyNioAsyncHttpClient.builder().tcpKeepAlive(true)</code>
Secure random	<pre>clientConfig.setSecureRandom(...) clientConfig.withSecureRandom(...)</pre>	Not Supported ( <a href="#">Request Feature</a> )
	<pre>AmazonDynamoDBClientBuilder.standard()     .withClientConfiguration(clientConfiguration)     .build()</pre>	<pre>DynamoDbClient.builder()     .overrideConfiguration(overrideConfiguration.build())     .build()</pre>

## Client override retries

Setting	1.x	2.x
	<pre>ClientConfiguration clientConfig =     new ClientConfiguration()</pre>	<pre>ClientOverrideConfiguration.Builder overrideConfigBuilder =</pre>



Setting	1.x	2.x
		<code>ClientOverrideConfiguration.builder();</code>
Max error retry	<pre>clientConfig.setMaxErrorRetry(...) clientConfig.withMaxErrorRetry(...)</pre>	<pre>// Configure the default // retry strategy. builder.retryStrategy(b -&gt; b.maxAttempts(...)) );</pre>
Use throttled retries	<pre>clientConfig.setUseThrottleRetries(...) clientConfig.withUseThrottleRetries(...)</pre>	<a href="#">Not supported</a>
Max consecutive retries before throttling	<pre>clientConfig.setMaxConsecutiveRetriesBeforeThrottling(...) clientConfig.withMaxConsecutiveRetriesBeforeThrottling(...)</pre>	<a href="#">Not supported</a>

Setting	1.x	2.x
	<pre>AmazonDynamoDBClientBuilder.standard()     .withClientConfiguration(clientConfiguration)     .build()</pre>	<pre>DynamoDbClient.builder()     .overrideConfiguration(overrideConfigBuilder.build())     .build();  // You also have the option to use a lambda expression to configure and // build the 'ClientOverrideConfiguration.Builder'. DynamoDbClient client = DynamoDbClient.builder()     .overrideConfiguration(o -&gt; o.retryStrategy(b -&gt; b.maxAttempts(5)))     .build();</pre>

## Asynchronous clients

Setting	1.x	2.x
		<pre>ClientAsyncConfiguration.Builder     asyncConfig = ClientAsyncConfiguration.builder()</pre>
Executor	<pre>AmazonDynamoDBAsyncClientBuilder.standard()</pre>	<pre>asyncConfig.advancedOption()</pre>

Setting	1.x	2.x
	<pre> .withExecutorFacto ry(...) .build() </pre>	<pre> SdkAdvancedAsyncCl ientOption.FUTURE_ COMPLETION_EXECUTO R, ...) </pre>
		<pre> DynamoDbAsyncClie nt.builder()     .asyncConfiguratio n(asyncConfig)     .build() </pre>

## Other client changes

The following `ClientConfiguration` option from 1.x has changed in 2.x of the SDK and doesn't have a direct equivalent.

Setting	1.x	2.x equivalent
Protocol	<pre> clientConfig.setPr otocol(Protocol.HTTP) clientConfig.w ithProtocol(Protoc ol.HTTP) </pre>	The protocol setting is HTTPS by default. To modify the setting, specify the protocol setting an HTTP endpoint on the client builder:  <pre> clientBuilder.endp ointOverride(     URI.create("http:/ /...")) </pre>

## Credentials provider changes

This section provides a mapping of the name changes of credentials provider classes and methods between versions 1.x and 2.x of the AWS SDK for Java.

## Notable differences

- The default credentials provider loads system properties before environment variables in version 2.x. For more information, see [Using credentials](#).
- The constructor method is replaced with the create or builder methods.

### Example

```
DefaultCredentialsProvider.create();
```

- Asynchronous refresh is no longer set by default. You must specify it with the builder of the credentials provider.

### Example

```
ContainerCredentialsProvider provider = ContainerCredentialsProvider.builder()
 .asyncCredentialUpdateEnabled(true)
 .build();
```

- You can specify a path to a custom profile file using the `ProfileCredentialsProvider.builder()`.

### Example

```
ProfileCredentialsProvider profile = ProfileCredentialsProvider.builder()

 .profileFile(ProfileFile.builder().content(Paths.get("myProfileFile.file")).build())
 .build();
```

- Profile file format has changed to more closely match the AWS CLI. For details, see [Configuring the AWS CLI](#) in the *AWS Command Line Interface User Guide*.

## Credentials provider changes mapped between versions 1.x and 2.x

### AWSCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.AWSCredentialsProvider</code>	<code>software.amazon.awssdk.auth.credentials.AwsCredentialsProvider</code>
Method name	<code>getCredentials</code>	<code>resolveCredentials</code>
Unsupported method	<code>refresh</code>	Not supported

### DefaultAWSCredentialsProviderChain

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.DefaultAWSCredentialsProviderChain</code>	<code>software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider</code>
Creation	<code>new DefaultAWSCredentialsProviderChain</code>	<code>DefaultCredentialsProvider.create</code>
Unsupported method	<code>getInstance</code>	Not supported
Priority order of external settings	Environment variables before system properties	System properties before environment variables

**AWSStaticCredentialsProvider**

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.AWSStaticCredentialsProvider</code>	<code>software.amazon.awssdk.auth.credentials.StaticCredentialsProvider</code>
Creation	<code>new AWSStaticCredentialsProvider</code>	<code>StaticCredentialsProvider.create</code>

**EnvironmentVariableCredentialsProvider**

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.EnvironmentVariableCredentialsProvider</code>	<code>software.amazon.awssdk.auth.credentials.EnvironmentVariableCredentialsProvider</code>
Creation	<code>new EnvironmentVariableCredentialsProvider</code>	<code>EnvironmentVariableCredentialsProvider.create</code>
Environment variable name	<code>AWS_ACCESS_KEY</code>	<code>AWS_ACCESS_KEY_ID</code>
	<code>AWS_SECRET_KEY</code>	<code>AWS_SECRET_ACCESS_KEY</code>

## SystemPropertiesCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth. SystemPropertiesC redentialsProvider</code>	<code>software.amazon.aw ssdk.auth.credenti als.SystemProperty CredentialsProvider</code>
Creation	<code>new SystemPro pertiesCredentials Provider</code>	<code>SystemPropertiesCr edentialsProvider. create</code>
System property name	<code>aws.secretKey</code>	<code>aws.secretAccessKey</code>

## ProfileCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth .profile.ProfileCr edentialsProvider</code>	<code>software.amazon.aw ssdk.auth.credenti als.ProfileCredent ialsProvider</code>
Creation	<code>new ProfileCr edentialsProvider</code>	<code>ProfileCredentials Provider.create</code>
Location of custom profile	<code>AWS_CREDENTIAL_PRO FILES_FILE</code> environme nt variable     <code>new ProfileCr edentialsProvider</code>	<code>AWS_SHARED_CREDENT IALS_FILE</code> environment variable     <code>ProfileCredentials Provider.builder</code>

## ContainerCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth. ContainerCredenti alsProvider</code>	<code>software.amazon.aw ssdk.auth.credenti als.ContainerCrede ntialsProvider</code>
Creation	<code>new Container CredentialsProvider</code>	<code>ContainerCredentia lsProvider.create</code>
Specify asynchronous refresh	Not supported	Default behavior

## InstanceProfileCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth .InstanceProfileCr edentialsProvider</code>	<code>software.amazon.aw ssdk.auth.credenti als.InstanceProfil eCredentialsProvid er</code>
Creation	<code>new InstanceP rofileCredentialsP rovider</code>	<code>InstanceProfileCre dentialsProvider.c reate</code>
Specify asynchronous refresh	<code>new InstanceP rofileCredentialsP rovider(true)</code>	<code>InstanceProfileCre dentialProvider.bu ilder().asyncCrede ntialUpdateEnabled (true).build()</code>
System property name	<code>com.amazonaws.sdk. disableEc2Metadata</code>	<code>aws.disableEc2Meta data</code>



Change category	1.x	2.x
	<code>com.amazonaws.sdk. ec2MetadataService EndpointOverride</code>	<code>aws.ec2MetadataSer viceEndpoint</code>

## STSAssumeRoleSessionCredentialsProvider

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth .STSAssumeRoleSess ionCredentialsProv ider</code>	<code>software.amazon.aw ssdk.services.sts. auth.StsAssumeRole CredentialsProvider</code>
Creation	<code>new STSAssumeRoleSessionCredentialsProvider</code>     <code>new STSAssumeRoleSessionCredentialsProvider.Builder</code>	<code>StsAssumeRoleCreden tialsProvider.bui lder</code>
Asynchronous refresh	Default behavior	Default behavior
Configuration	<code>new STSAssumeRoleSessionCredentialsProvider.Builder</code>	Configure a <code>StsClient</code> and <code>AssumeRoleRequest</code> request

**STSSessionCredentialsProvider**

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.STSSessionCredentialsProvider</code>	<code>software.amazon.awssdk.services.sts.auth.StsGetSessionTokenCredentialsProvider</code>
Creation	<code>new STSSessionCredentialsProvider</code>	<code>StsGetSessionTokenCredentialsProvider.builder</code>
Asynchronous refresh	Default behavior	<code>StsGetSessionTokenCredentialsProvider.builder</code>
Configuration	Constructor parameters	Configure an <code>StsClient</code> and <code>GetSessionTokenRequest</code> request in a builder

**WebIdentityFederationSessionCredentialsProvider**

Change category	1.x	2.x
Package/class name	<code>com.amazonaws.auth.WebIdentityFederationSessionCredentialsProvider</code>	<code>software.amazon.awssdk.services.sts.auth.StsAssumeRoleWithWebIdentityCredentialsProvider</code>
Creation	<code>new WebIdentityFederationSessionCredentialsProvider</code>	<code>StsAssumeRoleWithWebIdentityCredentialsProvider.builder</code>

Change category	1.x	2.x
Asynchronous refresh	Default behavior	<code>StsAssumeRoleWithWebIdentityCredentialsProvider.builder</code>
Configuration	Constructor parameters	Configure an <code>StsClient</code> and <code>AssumeRoleWithWebIdentityRequest</code> request in a builder

## Classes replaced

1.x class	2.x replacement classes
<code>com.amazonaws.auth.EC2ContainerCredentialsProviderWrapper</code>	<code>software.amazon.awssdk.auth.credentials.ContainerCredentialsProvider</code> and <code>software.amazon.awssdk.auth.credentials.InstanceProfileCredentialsProvider</code>
<code>com.amazonaws.services.s3.S3CredentialsProviderChain</code>	<code>software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider</code> and <code>software.amazon.awssdk.auth.credentials.AnonymousCredentialsProvider</code>

## Classes removed

1.x class
<code>com.amazonaws.auth.ClasspathPropertiesFileCredentialsProvider</code>
<code>com.amazonaws.auth.PropertiesFileCredentialsProvider</code>

## Region changes

This section describes the changes implemented in the AWS SDK for Java 2.x for using the Region and Regions classes.

### Region configuration

- Some AWS services don't have Region specific endpoints. When using those services, you must set the Region as `Region.AWS_GLOBAL` or `Region.AWS_CN_GLOBAL`.

#### Example

```
Region region = Region.AWS_GLOBAL;
```

- `com.amazonaws.regions.Regions` and `com.amazonaws.regions.Region` classes are now combined into one class, `software.amazon.awssdk.regions.Region`.

### Method and class name mappings

The following tables map Region related classes between versions 1.x and 2.x of the AWS SDK for Java. You can create an instance of these classes using the `of()` method.

#### Example

```
RegionMetadata regionMetadata = RegionMetadata.of(Region.US_EAST_1);
```

#### 1.x Regions class method changes

1.x	2.x
<code>Regions.fromName</code>	<code>Region.of</code>
<code>Regions.getName</code>	<code>Region.id</code>
<code>Regions.getDescription</code>	<code>Region.metadata().description()</code>
<code>Regions.getCurrentRegion</code>	Not Supported
<code>Regions.DEFAULT_REGION</code>	Not Supported

1.x	2.x
<code>Regions.name</code>	<code>Region.id</code>

### 1.x Region class method changes

1.x	2.x
<code>Region.getName</code>	<code>Region.id</code>
<code>Region.hasHttpsEndpoint</code>	Not Supported
<code>Region.hasHttpEndpoint</code>	Not Supported
<code>Region.getAvailableEndpoints</code>	Not Supported
<code>Region.createClient</code>	Not Supported

### RegionMetadata class method changes

1.x	2.x
<code>RegionMetadata.getName</code>	<code>RegionMetadata.name</code>
<code>RegionMetadata.getDomain</code>	<code>RegionMetadata.domain</code>
<code>RegionMetadata.getPartition</code>	<code>RegionMetadata.partition</code>

### ServiceMetadata class method changes

1.x	2.x
<code>Region.getServiceEndpoint</code>	<code>ServiceMetadata.endpointFor(Region)</code>
<code>Region.isServiceSupported</code>	<code>ServiceMetadata.regions().contains(Region)</code>

## Operations, requests and responses changes

In version 2 of the SDK for Java, requests are passed to a client operation. For example `DynamoDbClient`'s `PutItemRequest` is passed to `DynamoDbClient.putItem` operation. These operations return a response from the AWS service, such as a `PutItemResponse`.

Version 2 of the SDK for Java has the following changes from version 1.

- Operations with multiple response pages now have a `Paginator` method for automatically iterating over all items in the response.
- You cannot mutate requests and responses.
- You must create requests and responses with a static builder method instead of a constructor. For example, version 1's new `PutItemRequest().withTableName(...)` is now `PutItemRequest.builder().tableName(...).build()`.
- Operations support a short-hand way to create requests: `dynamoDbClient.putItem(request -> request.tableName(...))`.

The following sections describe specific changes between version 1 and version 2. Some parameter type changes can be converted automatically using the [migration tool](#), while other changes require manual updates to your code.

### Date parameter changes

In version 1, many operations accepted `java.util.Date` objects for time-based parameters. In version 2, these operations use `java.time.Instant` objects instead.

You can convert `Date` parameters automatically using the [migration tool](#), or you can convert them manually by calling the `toInstant()` method on your `Date` object.

#### Example- Generate a presigned URL with an expiration date in version 1

```
// Generate a presigned URL that expires at a specific date
Date expiration = new Date(System.currentTimeMillis() + 3600000); // 1 hour from now
URL presignedUrl = s3Client.generatePresignedUrl(bucketName, keyName, expiration);
```

#### Example- Generate a presigned URL with an expiration instant in version 2

```
// Generate a presigned URL that expires at a specific instant
```

```
Date expiration = new Date(System.currentTimeMillis() + 3600000); // 1 hour from now
PresignedGetObjectRequest presignedRequest = presigner.presignGetObject(
 GetObjectPresignRequest.builder()
 .getObjectRequest(GetObjectRequest.builder()
 .bucket(bucketName)
 .key(keyName)
 .build())
 .signatureDuration(Duration.between(Instant.now(), expiration.toInstant()))
 .build());
```

## Binary data handling changes

In version 1, binary data was handled using `ByteBuffer` objects directly. In version 2, the SDK uses `SdkBytes` objects that provide a more convenient and type-safe way to work with binary data.

You can convert `SdkBytes` to `ByteBuffer` automatically using the [migration tool](#), or you can convert them manually by calling `asByteBuffer()` on the returned `SdkBytes` object.

### Example- Get binary data from a message attribute in version 1

```
// Get binary data from a message attribute
MessageAttributeValue messageAttributeValue = new MessageAttributeValue();
ByteBuffer binaryValue = messageAttributeValue.getBinaryValue();
String binaryString = new String(messageAttributeValue.getBinaryValue().array());
```

### Example- Get binary data from a message attribute in version 2

```
// Get binary data from a message attribute
MessageAttributeValue messageAttributeValue = MessageAttributeValue.builder().build();
ByteBuffer binaryValue = messageAttributeValue.binaryValue().asByteBuffer();
String binaryString = new
 String(messageAttributeValue.binaryValue().asByteBuffer().array());
```

## Timeout parameter changes

In version 1, timeout values were specified as integer values representing milliseconds. In version 2, timeout parameters use `java.time.Duration` objects for better type safety and clarity.

You can convert numeric timeout values automatically using the [migration tool](#), or you can convert them manually by wrapping your numeric values with the appropriate `Duration` factory method.

## Example- Set a request timeout in version 1

```
// Set request timeout in milliseconds
ClientConfiguration clientConfiguration = new ClientConfiguration();
clientConfiguration.setRequestTimeout(5000); // 5 seconds
```

## Example- Set a request timeout in version 2

```
// Set request timeout using Duration
ClientConfiguration clientConfiguration = new ClientConfiguration();
clientConfiguration.setRequestTimeout(Duration.ofMillis(5000)); // 5 seconds

// Or more clearly:
clientConfiguration.setRequestTimeout(Duration.ofSeconds(5)); // 5 seconds
```

You can use the following `Duration` factory methods for timeout values:

- `Duration.ofMillis(long millis)` - For millisecond values.
- `Duration.ofSeconds(long seconds)` - For second values.
- `Duration.ofMinutes(long minutes)` - For minute values.

## Streaming operation differences between 1.x and 2.x of the AWS SDK for Java

Streaming operations, such as Amazon S3 `getObject` and `putObject` methods, support non-blocking I/O in version 2.x of the SDK. As a result, the request and response model objects no longer take an `InputStream` as a parameter. Instead, for synchronous requests the request object accepts `RequestBody`, which is a stream of bytes. The asynchronous equivalent accepts an `AsyncRequestBody`.

### Example of Amazon S3 `putObject` operation in 1.x

```
s3client.putObject(BUCKET, KEY, new File(file_path));
```

### Example of Amazon S3 `putObject` operation in 2.x

```
s3client.putObject(PutObjectRequest.builder()
 .bucket(BUCKET)
 .key(KEY)
 .build(),
```



```
RequestBody.of(Paths.get("myfile.in"))));
```

A streaming response object accepts a `ResponseTransformer` for synchronous clients and a `AsyncResponseTransformer` for asynchronous clients in V2.

### Example of Amazon S3 getObject operation in 1.x

```
S3Object o = s3.getObject(bucket, key);
S3ObjectInputStream s3is = o.getObjectContent();
FileOutputStream fos = new FileOutputStream(new File(key));
```

### Example of Amazon S3 getObject operation in 2.x

```
s3client.getObject(GetObjectRequest.builder().bucket(bucket).key(key).build(),
 ResponseTransformer.toFile(Paths.get("key")));
```

In the SDK for Java 2.x, streaming response operations have an `AsBytes` method to load the response into memory and simplify common type conversions in-memory.

## Serialization differences between 1.x and 2.x of the AWS SDK for Java

### List objects to request parameters difference

The SDK for Java v1.x and v2.x differ in how they serialize List objects to request parameters.

The SDK for Java 1.x does not serialize an empty list, whereas the SDK for Java 2.x serializes an empty list as an empty parameter.

For example, consider a service with a `SampleOperation` that takes a `SampleRequest`. The `SampleRequest` accepts two parameters—a String type `str1` and List type `listParam`—as shown in the following examples.

### Example of SampleOperation in 1.x

```
SampleRequest v1Request = new SampleRequest()
 .withStr1("TestName");

sampleServiceV1Client.sampleOperation(v1Request);
```

Wire-level logging shows that the `listParam` parameter is not serialized.

```
Action=SampleOperation&Version=2011-01-01&str1=TestName
```

## Example of SampleOperation in 2.x

```
sampleServiceV2Client.sampleOperation(b -> b
 .str1("TestName"));
```

Wire-level logging shows that the `listParam` parameter is serialized with no value.

```
Action=SampleOperation&Version=2011-01-01&str1=TestName&listParam=
```

## POJOs in V1 compared to builders in V2

Because the V1 SDK for Java uses mutable POJO classes, serialization and deserialization libraries—such as [Jackson](#)—can use model objects directly.

The V2 SDK for Java, by contrast, uses immutable model objects. You must use an intermediate builder to perform de/serialization.

The following example shows the differences between de/serializing a `headBucket` API call with V1 and V2 using a Jackson `ObjectMapper`.

```
public void sendRequest() throws IOException {
 final String bucketName = "amzn-s3-demo-bucket";
 final ObjectMapper mapper = new ObjectMapper();

 // V1 uses POJOs to serialize and deserialize.
 final AmazonS3 v1S3Client = AmazonS3ClientBuilder.defaultClient();
 HeadBucketResult resultV1 = v1S3Client.headBucket(
 new HeadBucketRequest(bucketName));

 String v1Serialized = mapper.writeValueAsString(resultV1);

 HeadBucketResult deserializedV1 = mapper.readValue(v1Serialized,
 HeadBucketResult.class);

 // V2 uses builders to serialize and deserialize.
 S3Client v2S3Client = S3Client.create();
 HeadBucketResponse v2Response = v2S3Client.headBucket(
 b -> b.bucket(bucketName));
```

```
String v2Serialized = mapper.writeValueAsString(
 v2Response.toBuilder());

HeadBucketResponse v2Deserialized = mapper.readValue(
 v2Serialized, HeadBucketResponse.serializableBuilderClass()
 .build());
}
```

## Deserialization differences between 1.x and 2.x of the AWS SDK for Java

### Empty collections in V2 compared to nulls in V1

The SDK for Java v1.x and v2.x differ in how they deserialize JSON responses with lists and maps that are empty.

When the SDK receives a response with a missing property that is modeled as a list or map, V1 deserializes the missing property to `null`, whereas V2 deserializes the property to an immutable empty collection object.

For example, consider the response returned for the `describeTable` method from a DynamoDB client. The following example method contains a V2 DynamoDB client and a V1 DynamoDB client that both execute the `describeTable` method on a table that has no global secondary indexes

### Example of the deserialization of a property modeled as a list that is missing in the response

```
public void deserializationDiffs(){

 DescribeTableResponse v2Response = dynamoDbClientV2.describeTable(builder ->
builder.tableName(TABLE_NAME));
 // V2 provides has* methods on model objects for list/map members. No null check
 needed.
 LOGGER.info(String.valueOf(v2Response.table().hasGlobalSecondaryIndexes()));

 LOGGER.info(String.valueOf(v2Response.table().globalSecondaryIndexes().isEmpty()));
 // V2 deserialize to an empty collection.
 LOGGER.info(v2Response.table().globalSecondaryIndexes().toString());

 // V1 deserialize to null.
 DescribeTableResult v1Result = dynamoDbClientV1.describeTable(new
DescribeTableRequest(TABLE_NAME));
 if (v1Result.getTable().getGlobalSecondaryIndexes() != null){
 LOGGER.info(v1Result.getTable().getGlobalSecondaryIndexes().toString());
 } else {
```

```
 LOGGER.info("The list of global secondary indexes returned by the V1 call is
<null>");
 }
}
```

The following shows the logged output:

```
INFO org.example.DeserializationDifferences:45 - false
INFO org.example.DeserializationDifferences:46 - true
INFO org.example.DeserializationDifferences:48 - []
INFO org.example.DeserializationDifferences:55 - The list of global secondary indexes
returned by the V1 call is <null>
```

The Java SDK 2.x takes an idiomatic approach by deserializing empty lists and maps to immutable empty collections instead of returning `null`, promoting safer and more concise code. With V2, you can check if a service returned an attribute modeled as a list or map with the `has*` method, such as the `hasGlobalSecondaryIndexes` shown in the previous example.

This approach avoids the need for explicit `null` checks and aligns with modern Java best practices for handling absent or empty data structures.

## Complete example

```
package org.example;

import com.amazonaws.regions.Regions;
import com.amazonaws.services.dynamodbv2.AmazonDynamoDB;
import com.amazonaws.services.dynamodbv2.AmazonDynamoDBClientBuilder;
import com.amazonaws.services.dynamodbv2.model.DescribeTableRequest;
import com.amazonaws.services.dynamodbv2.model.DescribeTableResult;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.BillingMode;
import software.amazon.awssdk.services.dynamodb.model.DescribeTableResponse;
import software.amazon.awssdk.services.dynamodb.model.KeyType;
import software.amazon.awssdk.services.dynamodb.model.ScalarAttributeType;

import java.util.UUID;

public class DeserializationDifferences {
```

```

 private static final Logger LOGGER =
 LoggerFactory.getLogger(DeserializationDifferences.class);
 private static final String TABLE_NAME = "DeserializationTable-" +
 UUID.randomUUID();
 DynamoDbClient dynamoDbClientV2 = DynamoDbClient.create();
 AmazonDynamoDB dynamoDbClientV1 =
 AmazonDynamoDBClientBuilder.standard().withRegion(Regions.US_EAST_1).build();

 public static void main(String[] args) {

 DeserializationDifferences difference = new DeserializationDifferences();
 difference.createTable();
 difference.deserializationDiffs();
 difference.deleteTable();
 }

 public void createTable(){
 dynamoDbClientV2.createTable(b -> b
 .billingMode(BillingMode.PAY_PER_REQUEST)
 .tableName(TABLE_NAME)
 .keySchema(b1 -> b1.attributeName("Id").keyType(KeyType.HASH))
 .attributeDefinitions(b2 ->
b2.attributeName("Id").attributeType(ScalarAttributeType.S)));
 dynamoDbClientV2.waiter().waitUntilTableExists(b -> b.tableName(TABLE_NAME));
 }

 public void deserializationDiffs(){

 DescribeTableResponse v2Response = dynamoDbClientV2.describeTable(builder ->
builder.tableName(TABLE_NAME));
 // V2 provides has* methods on model objects for list/map members. No null
 check needed.
 LOGGER.info(String.valueOf(v2Response.table().hasGlobalSecondaryIndexes()));

 LOGGER.info(String.valueOf(v2Response.table().globalSecondaryIndexes().isEmpty()));
 // V2 deserialize to an empty collection.
 LOGGER.info(v2Response.table().globalSecondaryIndexes().toString());

 // V1 deserialize to null.
 DescribeTableResult v1Result = dynamoDbClientV1.describeTable(new
DescribeTableRequest(TABLE_NAME));
 if (v1Result.getTable().getGlobalSecondaryIndexes() != null){
 LOGGER.info(v1Result.getTable().getGlobalSecondaryIndexes().toString());
 } else {

```

```

 LOGGER.info("The list of global secondary indexes returned by the V1 call
is <null>");
 }
}

public void deleteTable(){
 dynamoDbClientV2.deleteTable(b -> b.tableName(TABLE_NAME));
 dynamoDbClientV2.waiter().waitUntilTableNotExists(b ->
b.tableName(TABLE_NAME));
}
}

```

The JSON response for the `describeTable` method from the V1 and V2 client contains no `GlobalSecondaryIndexes` attribute:

```

{
 "Table": {
 "AttributeDefinitions": [
 {
 "AttributeName": "Id",
 "AttributeType": "S"
 }
],
 "BillingModeSummary": {
 "BillingMode": "PAY_PER_REQUEST",
 "LastUpdateToPayPerRequestDateTime": ...
 },
 "CreationDateTime": ...,
 "DeletionProtectionEnabled": false,
 "ItemCount": 0,
 "KeySchema": [
 {
 "AttributeName": "Id",
 "KeyType": "HASH"
 }
],
 "ProvisionedThroughput": {
 "NumberOfDecreasesToday": 0,
 "ReadCapacityUnits": 0,
 "WriteCapacityUnits": 0
 },
 "TableArn": "arn:aws:dynamodb:us-east-1:111111111111:table/
DeserializationTable-...",

```

```
"TableId": "...",
"TableName": "DeserializationTable-...",
"TableSizeBytes": 0,
"TableStatus": "ACTIVE",
"TableThroughputModeSummary": {
 "LastUpdateToPayPerRequestDateTime": ...,
 "TableThroughputMode": "PAY_PER_REQUEST"
},
"WarmThroughput": {
 "ReadUnitsPerSecond": 12000,
 "Status": "ACTIVE",
 "WriteUnitsPerSecond": 4000
}
}
```

## Exception changes

Exception class names, their structures, and their relationships have changed. `software.amazon.awssdk.core.exception.SdkException` is the new base Exception class that all the other exceptions extend.

This table maps the exception class name changes.

1.x	2.x
<code>com.amazonaws.SdkBaseException</code> <code>com.amazonaws.AmazonClientException</code>	<code>software.amazon.awssdk.core.exception.SdkException</code>
<code>com.amazonaws.SdkClientException</code>	<code>software.amazon.awssdk.core.exception.SdkClientException</code>
<code>com.amazonaws.AmazonServiceException</code>	<code>software.amazon.awssdk.awscore.exception.AwsServiceException</code>

The following table maps the methods on exception classes between version 1.x and 2.x.

1.x	2.x
<code>AmazonServiceException.getRequestId</code>	<code>SdkServiceException.requestId</code>
<code>AmazonServiceException.getServiceName</code>	<code>AwsServiceException.awsErrorDetails().serviceName</code>
<code>AmazonServiceException.getErrorCode</code>	<code>AwsServiceException.awsErrorDetails().errorCode</code>
<code>AmazonServiceException.getErrorMessage</code>	<code>AwsServiceException.awsErrorDetails().errorMessage</code>
<code>AmazonServiceException.getStatusCode</code>	<code>AwsServiceException.awsErrorDetails().sdkHttpResponse().statusCode</code>
<code>AmazonServiceException.getHttpHeaders</code>	<code>AwsServiceException.awsErrorDetails().sdkHttpResponse().headers</code>
<code>AmazonServiceException.getRawResponse</code>	<code>AwsServiceException.awsErrorDetails().rawResponse</code>

## Service-specific changes

### Amazon S3 changes

SDK for Java 2.x disables anonymous access by default. As a result, you must enable anonymous access by using the `AnonymousCredentialsProvider`.

### Operation name changes

Many of the operation names for the Amazon S3 client have changed in the AWS SDK for Java 2.x. In version 1.x, the Amazon S3 client is not generated directly from the service API. This results in inconsistency between the SDK operations and the service API. In version 2.x, we now generate the Amazon S3 client to be more consistent with the service API.



The following table shows the operation names in the two versions.

### Amazon S3 Operation names

1.x	2.x
abortMultipartUpload	abortMultipartUpload
changeObjectStorageClass	copyObject
completeMultipartUpload	completeMultipartUpload
copyObject	copyObject
copyPart	uploadPartCopy
createBucket	createBucket
deleteBucket	deleteBucket
deleteBucketAnalyticsConfiguration	deleteBucketAnalyticsConfiguration
deleteBucketCrossOriginConfiguration	deleteBucketCors
deleteBucketEncryption	deleteBucketEncryption
deleteBucketInventoryConfiguration	deleteBucketInventoryConfiguration
deleteBucketLifecycleConfiguration	deleteBucketLifecycle
deleteBucketMetricsConfiguration	deleteBucketMetricsConfiguration
deleteBucketPolicy	deleteBucketPolicy
deleteBucketReplicationConfiguration	deleteBucketReplication
deleteBucketTaggingConfiguration	deleteBucketTagging

1.x	2.x
deleteBucketWebsiteConfiguration	deleteBucketWebsite
deleteObject	deleteObject
deleteObjectTagging	deleteObjectTagging
deleteObjects	deleteObjects
deleteVersion	deleteObject
disableRequesterPays	putBucketRequestPayment
doesBucketExist	headBucket
doesBucketExistV2	headBucket
doesObjectExist	headObject
enableRequesterPays	putBucketRequestPayment
generatePresignedUrl	<a href="#">S3Presigner</a>
getBucketAccelerateConfiguration	getBucketAccelerateConfiguration
getBucketAcl	getBucketAcl
getBucketAnalyticsConfiguration	getBucketAnalyticsConfiguration
getBucketCrossOriginConfiguration	getBucketCors
getBucketEncryption	getBucketEncryption
getBucketInventoryConfiguration	getBucketInventoryConfiguration
getBucketLifecycleConfiguration	getBucketLifecycle or getBucketLifecycleConfiguration
getBucketLocation	getBucketLocation

1.x	2.x
<code>getBucketLoggingConfiguration</code>	<code>getBucketLogging</code>
<code>getBucketMetricsConfiguration</code>	<code>getBucketMetricsConfiguration</code>
<code>getBucketNotificationConfiguration</code>	<code>getBucketNotification</code> or <code>getBucketNotificationConfiguration</code>
<code>getBucketPolicy</code>	<code>getBucketPolicy</code>
<code>getBucketReplicationConfiguration</code>	<code>getBucketReplication</code>
<code>getBucketTaggingConfiguration</code>	<code>getBucketTagging</code>
<code>getBucketVersioningConfiguration</code>	<code>getBucketVersioning</code>
<code>getBucketWebsiteConfiguration</code>	<code>getBucketWebsite</code>
<code>getObject</code>	<code>getObject</code>
<code>getObjectAcl</code>	<code>getObjectAcl</code>
<code>getObjectAsString</code>	<code>getObjectAsBytes().asUtf8String</code>
<code>getObjectMetadata</code>	<code>headObject</code>
<code>getObjectTagging</code>	<code>getObjectTagging</code>
<code>getResourceUrl</code>	<a href="#">S3Utilities#getUrl</a>
<code>getS3AccountOwner</code>	<code>listBuckets</code>
<code>getUrl</code>	<a href="#">S3Utilities#getUrl</a>
<code>headBucket</code>	<code>headBucket</code>
<code>initiateMultipartUpload</code>	<code>createMultipartUpload</code>
<code>isRequesterPaysEnabled</code>	<code>getBucketRequestPayment</code>

1.x	2.x
<code>listBucketAnalyticsConfigurations</code>	<code>listBucketAnalyticsConfigurations</code>
<code>listBucketInventoryConfigurations</code>	<code>listBucketInventoryConfigurations</code>
<code>listBucketMetricsConfigurations</code>	<code>listBucketMetricsConfigurations</code>
<code>listBuckets</code>	<code>listBuckets</code>
<code>listMultipartUploads</code>	<code>listMultipartUploads</code>
<code>listNextBatchOfObjects</code>	<code>listObjectsV2Paginator</code>
<code>listNextBatchOfVersions</code>	<code>listObjectVersionsPaginator</code>
<code>listObjects</code>	<code>listObjects</code>
<code>listObjectsV2</code>	<code>listObjectsV2</code>
<code>listParts</code>	<code>listParts</code>
<code>listVersions</code>	<code>listObjectVersions</code>
<code>putObject</code>	<code>putObject</code>
<code>restoreObject</code>	<code>restoreObject</code>
<code>restoreObjectV2</code>	<code>restoreObject</code>
<code>selectObjectContent</code>	<code>selectObjectContent</code>
<code>setBucketAccelerateConfiguration</code>	<code>putBucketAccelerateConfiguration</code>
<code>setBucketAcl</code>	<code>putBucketAcl</code>
<code>setBucketAnalyticsConfiguration</code>	<code>putBucketAnalyticsConfiguration</code>

1.x	2.x
setBucketCrossOriginConfiguration	putBucketCors
setBucketEncryption	putBucketEncryption
setBucketInventoryConfiguration	putBucketInventoryConfiguration
setBucketLifecycleConfiguration	putBucketLifecycle or putBucketLifecycleConfiguration
setBucketLoggingConfiguration	putBucketLogging
setBucketMetricsConfiguration	putBucketMetricsConfiguration
setBucketNotificationConfiguration	putBucketNotification or putBucketNotificationConfiguration
setBucketPolicy	putBucketPolicy
setBucketReplicationConfiguration	putBucketReplication
setBucketTaggingConfiguration	putBucketTagging
setBucketVersioningConfiguration	putBucketVersioning
setBucketWebsiteConfiguration	putBucketWebsite
setObjectAcl	putObjectAcl
setObjectRedirectLocation	copyObject
setObjectTagging	putObjectTagging
uploadPart	uploadPart

## Amazon SNS changes

An SNS client can no longer access SNS topics in Regions other than the Region that it is configured to access.

## Amazon SQS changes

An SQS client can no longer access SQS queues in Regions other than the Region that it is configured to access.

## Amazon RDS changes

The SDK for Java 2.x uses `RdsUtilities#generateAuthenticationToken` in place of the class `RdsIamAuthTokenGenerator` in 1.x.

## Changes in working with Amazon S3 from version 1 to version 2 of the AWS SDK for Java

The AWS SDK for Java 2.x introduces significant changes to the S3 client, including a new package structure, updated class names, and revised method signatures. While many methods can be automatically migrated from V1 to V2 using the [migration tool](#), some require manual migration, such as `listNextBatchOfObjects` and `selectObjectContent`. Additionally, V2 replaces certain V1 classes like `AccessControlList` and `CannedAccessControlList` with new implementations.

### Topics

- [S3 client differences between version 1 and version 2 of the AWS SDK for Java](#)
- [Migrate the Transfer Manager from version 1 to version 2 of the AWS SDK for Java](#)
- [Changes in parsing Amazon S3 URLs from version 1 to version 2](#)
- [Changes in the S3 Event Notifications API from version 1 to version 2](#)

## S3 client differences between version 1 and version 2 of the AWS SDK for Java

In this topic, the differences between the S3 clients in version 1 and version 2 of SDK for Java are organized by how the [migration tool](#) can automate the migration. The tool supports migration of most methods from V1 to V2, but some methods require manual migration. In addition to S3 client methods, some S3 V1 classes do not have a direct V2 equivalent requiring you to manually migrate them.

## Contents

- [Example V1 methods supported by the migration tool](#)
  - [putObject](#)
  - [getObject](#)
  - [getObject to a file](#)
- [V1 methods that require manual migration](#)
  - [V1's getObject using V1's S3ObjectId to V2's GetObjectRequest.builder\(\)](#)
  - [V1's getETag\(\) using V1's ObjectMetadata to V2's GetObjectResponse.eTag\(\)](#)
  - [V1's listNextBatchOfObjects to V2's listObjectsV2Paginator](#)
  - [V1's listNextBatchOfVersions to V2's listObjectVersionsPaginator](#)
  - [selectObjectContent](#)
  - [V1's setBucketAcl to V2's acl method on PutBucketAclRequest.builder\(\)](#)
  - [V1's setObjectAcl to V2's acl method on PutObjectAclRequest.builder\(\)](#)
  - [V1's initiateMultipartUpload to V2's createMultipartUpload](#)
    - [Example migration](#)
    - [Implementation differences](#)
  - [V1's setRegion to V2's region method on client builder](#)
  - [V1's setS3ClientOptions\(S3ClientOptions clientOptions\)](#)
  - [V1's setBucketLoggingConfiguration to V2's putBucketLogging](#)
  - [V1's setBucketLifecycleConfiguration to V2's putBucketLifecycleConfiguration](#)
  - [V1's setBucketTaggingConfiguration to V2's putBucketTagging](#)
- [V1 classes that require manual migration](#)
  - [V1's AccessControlList to V2's AccessControlPolicy](#)
  - [V1's CannedAccessControlList enum to V2's BucketCannedACL and ObjectCannedACL enums](#)
  - [V1's BucketNotificationConfiguration to V2's NotificationConfiguration](#)
  - [V1's MultiFactorAuthentication to V2's mfa method on a request builder](#)

## Example V1 methods supported by the migration tool

The migration tool automatically migrates most methods from V1 to V2. The `putObject` and `getObject` methods are two examples.

## putObject

```
// SDK V1
s3Client.putObject("amzn-s3-demo-bucket", "my-key", "Hello World!");

// SDK V2
s3Client.putObject(req -> req
 .bucket("amzn-s3-demo-bucket")
 .key("my-key"),
 RequestBody.fromString("Hello World!"));
```

## getObject

```
// SDK V1
S3Object object = s3Client.getObject("amzn-s3-demo-bucket", "my-key");
InputStream content = object.getObjectContent();

// SDK V2
ResponseInputStream<GetObjectResponse> response = s3Client.getObject(req -> req
 .bucket("amzn-s3-demo-bucket")
 .key("my-key"));
```

## getObject to a file

```
// SDK V1
// V1 automatically creates parent directories if they don't exist.
ObjectMetadata metadata = s3Client.getObject(
 new GetObjectRequest("amzn-s3-demo-bucket", "my-key"),
 new File("/path/to/parent/dir/my-file.txt"));

// SDK V2
// V2 does NOT auto-create parent directories. Ensure they exist before calling
// getObject.
Files.createDirectories(Paths.get("/path/to/parent/dir"));
s3Client.getObject(
 req -> req.bucket("amzn-s3-demo-bucket").key("my-key"),
 ResponseTransformerToFile(Paths.get("/path/to/parent/dir/my-file.txt")));
```



**⚠ Important**

Unlike SDK V1, SDK V2's `ResponseTransformer.toFile()` does not automatically create parent directories. If the parent directory does not exist, the SDK throws a `NoSuchFileException`. Call `Files.createDirectories()` before downloading if the parent directories might not exist.

**V1 methods that require manual migration**

You need to manually migrate the following V1 S3 client methods. When you use the migration tool, you see a comment in the V2 output Java file that directs you to this topic.

**V1's `getObject` using V1's `S3ObjectId` to V2's `GetObjectRequest.builder()`**

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

S3ObjectId s3objectId = new S3ObjectId(
 "amzn-s3-demo-bucket",
 "object-key",
 "abc123version"
);

GetObjectRequest getRequest = new GetObjectRequest(s3objectId);
S3Object s3objectVersioned = s3Client.getObject(getRequest);

// SDK V2
// V2 does not include a 'S3ObjectId' class. V2 uses the request builder pattern
// to supply the bucket, key, and version parameters.
S3Client s3Client = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

GetObjectRequest getRequest = GetObjectRequest.builder()
 .bucket("amzn-s3-demo-bucket")
 .key("object-key")
 .versionId("abc123version")
 .build();
```

```
ResponseInputStream<GetObjectResponse> response = s3Client.getObject(getRequest);
```

## V1's `getETag()` using V1's `ObjectMetadata` to V2's `GetObjectResponse.eTag()`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

S3Object object = s3ClientV1.getObject("amzn-s3-demo-bucket", "my-key");

// Double quotes are removed by the S3 client.
System.out.println(object.getObjectMetadata().getETag());

// SDK V2
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

ResponseInputStream<GetObjectResponse> response = s3ClientV2.getObject(
 req -> req.bucket("amzn-s3-demo-bucket").key("my-key"));

// Double quotes are *NOT* removed. This is the unchanged ETag value as S3 sent it.
System.out.println(response.response().eTag());
```

## V1's `listNextBatchOfObjects` to V2's `listObjectsV2Paginator`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

ObjectListing objectListing = s3ClientV1.listObjects("bucket-name");
while (objectListing.isTruncated()) {
 objectListing = s3ClientV1.listNextBatchOfObjects(objectListing);
 for (S3ObjectSummary summary : objectListing.getObjectSummaries()) {
 System.out.println(summary.getKey());
 }
}

// SDK V2
```

```
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

ListObjectsV2Request request = ListObjectsV2Request.builder()
 .bucket("bucket-name")
 .build();

// V2 returns a paginator.
ListObjectsV2Iterable responses = s3Client.listObjectsV2Paginator(request);

for (ListObjectsV2Response page : responses) {
 page.contents().forEach(content -> {
 System.out.println(content.key());
 });
}
```

## V1's `listNextBatchOfVersions` to V2's `listObjectVersionsPaginator`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

VersionListing versionListing = s3ClientV1.listVersions("bucket-name", "prefix");
while (versionListing.isTruncated()) {
 versionListing = s3ClientV1.listNextBatchOfVersions(versionListing);
 for (S3VersionSummary version : versionListing.getVersionSummaries()) {
 System.out.println(version.getKey() + " " + version.getVersionId());
 }
}

// SDK V2
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

ListObjectVersionsRequest request = ListObjectVersionsRequest.builder()
 .bucket("bucket-name")
 .prefix("prefix")
 .build();
```

```
// V2 returns a paginator.
ListObjectVersionsIterable responses = s3ClientV2.listObjectVersionsPaginator(request);

for (ListObjectVersionsResponse page : responses) {
 page.versions().forEach(version -> {
 System.out.println(version.key() + " " + version.versionId());
 });
}
```

## selectObjectContent

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

SelectObjectContentRequest request = new SelectObjectContentRequest()
 .withBucket("bucket-name")
 .withKey("object-key")
 .withExpression("select * from S3object")
 .withExpressionType(ExpressionType.SQL)

SelectObjectContentResult result = s3ClientV1.selectObjectContent(request);
InputStream resultInputStream = result.getPayload().getRecordsInputStream();

// SDK V2
// In V2, 'selectObjectContent()' is available only on the S3AsyncClient.
// V2 handles responses using an event-based 'SelectObjectContentEventStream'.
S3AsyncClient s3ClientV2 = S3AsyncClient.builder()
 .region(Region.US_WEST_2)
 .build();

SelectObjectContentRequest request = SelectObjectContentRequest.builder()
 .bucket("bucket-name")
 .key("object-key")
 .expression("select * from S3object")
 .expressionType(ExpressionType.SQL)
 .build();

SelectObjectContentResponseHandler handler = new SelectObjectContentResponseHandler() {
 // Implement the required abstract methods such as 'onEventStream()'.
```

```
};

CompletableFuture<Void> future = s3ClientV2.selectObjectContent(request, handler);
// The 'SelectObjectContentResponseHandler' implementation processes the results.
```

## V1's `setBucketAcl` to V2's `acl` method on `PutBucketAclRequest.builder()`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

AccessControlList acl = new AccessControlList();
acl.grantPermission(GroupGrantee.AllUsers, Permission.Read);
s3ClientV1.setBucketAcl("bucket-name", acl);

// SDK V2
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

PutBucketAclRequest request = PutBucketAclRequest.builder()
 .bucket("bucket-name")
 .acl(BucketCannedACL.PRIVATE)
 .build();

s3ClientV2.putBucketAcl(request);
```

## V1's `setObjectAcl` to V2's `acl` method on `PutObjectAclRequest.builder()`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

AccessControlList acl = new AccessControlList();
acl.grantPermission(GroupGrantee.AllUsers, Permission.Read);
s3ClientV1.setObjectAcl("bucket-name", "object-key", acl);

// SDK V2
S3Client s3ClientV2 = S3Client.builder()
```

```
.region(Region.US_WEST_2)
.build();

PutObjectAclRequest request = PutObjectAclRequest.builder()
 .bucket("bucket-name")
 .key("object-key")
 .acl(ObjectCannedACL.PRIVATE)
 .build();

s3ClientV2.putObjectAcl(request);
```

## V1's `initiateMultipartUpload` to V2's `createMultipartUpload`

### Example migration

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

ObjectMetadata metadata = new ObjectMetadata();
metadata.setContentType("application/zip");
metadata.addUserMetadata("mykey", "myvalue");

InitiateMultipartUploadRequest initRequest = new InitiateMultipartUploadRequest(
 "bucket-name",
 "object-key",
 metadata
);

InitiateMultipartUploadResult initResponse =
 s3ClientV1.initiateMultipartUpload(initRequest);
String uploadId = initResponse.getUploadId();

// SDK V2
// V1 uses ObjectMetadata methods, whereas V2 uses a simple Map.
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

CreateMultipartUploadRequest createMultipartRequest =
 CreateMultipartUploadRequest.builder()
 .bucket("amzn-s3-demo-bucket")
```

```

 .key("object-key")
 .contentType("application/zip")
 .metadata(Collections.singletonMap("mykey", "myvalue"))
 .build();

```

```

CreateMultipartUploadResponse response =
 s3ClientV2.createMultipartUpload(createMultipartRequest);
String uploadId = response.uploadId();

```

## Implementation differences

The default Content-Type header value for the following methods differ as shown in the following table.

SDK version	Method	Default Content-Type value
version 1	<a href="#">initiateMultipartUpload</a>	application/octet-stream
version 2	<a href="#">createMultipartUpload</a>	binary/octet-stream

## V1's setRegion to V2's region method on client builder

```

// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard().build();
s3ClientV1.setRegion(Region.getRegion(Regions.US_WEST_2));

// SDK V2
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

```

## V1's setS3ClientOptions(S3ClientOptions clientOptions)

Instead of using a single S3ClientOptions object with the setS3ClientOptions method, V2 provides methods on the client builder to set options.

## V1's `setBucketLoggingConfiguration` to V2's `putBucketLogging`

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
 .withRegion(Regions.US_WEST_2)
 .build();

BucketLoggingConfiguration loggingConfig = new BucketLoggingConfiguration();
loggingConfig.setDestinationBucketName("log-bucket");

SetBucketLoggingConfigurationRequest request = new
 SetBucketLoggingConfigurationRequest(
 "amzn-s3-demo-source-bucket",
 loggingConfig
);

s3ClientV1.setBucketLoggingConfiguration(request);

// SDK V2
// In V2, V1's 'BucketLoggingConfiguration' is replaced by 'BucketLoggingStatus'
// and 'LoggingEnabled'.
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

LoggingEnabled loggingEnabled = LoggingEnabled.builder()
 .targetBucket("log-bucket")
 .build();

BucketLoggingStatus loggingStatus = BucketLoggingStatus.builder()
 .loggingEnabled(loggingEnabled)
 .build();

PutBucketLoggingRequest request = PutBucketLoggingRequest.builder()
 .bucket("amzn-s3-demo-source-bucket")
 .bucketLoggingStatus(loggingStatus)
 .build();

s3ClientV2.putBucketLogging(request);
```



## V1's `setBucketLifecycleConfiguration` to V2's `putBucketLifecycleConfiguration`

```
// SDK V1
BucketLifecycleConfiguration.Rule rule = new BucketLifecycleConfiguration.Rule()
 .withId("Archive and Delete Rule")
 .withPrefix("documents/")
 .withStatus(BucketLifecycleConfiguration.ENABLED)
 .withTransitions(Arrays.asList(
 new Transition()
 .withDays(30)
 .withStorageClass(StorageClass.StandardInfrequentAccess.toString()),
 new Transition()
 .withDays(90)
 .withStorageClass(StorageClass.Glacier.toString())
))
 .withExpirationInDays(365);

BucketLifecycleConfiguration configuration = new BucketLifecycleConfiguration()
 .withRules(Arrays.asList(rule));

s3ClientV1.setBucketLifecycleConfiguration("amzn-s3-demo-bucket", configuration);
```

```
// SDK V2
LifecycleRule rule = LifecycleRule.builder()
 .id("Archive and Delete Rule")
 .filter(LifecycleRuleFilter.builder()
 .prefix("documents/")
 .build())
 .status(ExpirationStatus.ENABLED)
 .transitions(Arrays.asList(
 Transition.builder()
 .days(30)
 .storageClass(TransitionStorageClass.STANDARD_IA)
 .build(),
 Transition.builder()
 .days(90)
 .storageClass(TransitionStorageClass.GLACIER)
 .build()
))
 .expiration(LifecycleExpiration.builder()
 .days(365)
 .build())
 .build();
```

```
PutBucketLifecycleConfigurationRequest request =
 PutBucketLifecycleConfigurationRequest.builder()
 .bucket("amzn-s3-demo-bucket")
 .lifecycleConfiguration(BucketLifecycleConfiguration.builder()
 .rules(rule)
 .build())
 .build();

s3ClientV2.putBucketLifecycleConfiguration(request);
```

## V1's `setBucketTaggingConfiguration` to V2's `putBucketTagging`

```
// SDK V1
List<TagSet> tagsets = new ArrayList<>();
TagSet tagSet = new TagSet();
tagSet.setTag("key1", "value1");
tagSet.setTag("key2", "value2");
tagsets.add(tagSet);

BucketTaggingConfiguration bucketTaggingConfiguration = new
 BucketTaggingConfiguration();
bucketTaggingConfiguration.setTagSets(tagsets);

SetBucketTaggingConfigurationRequest request = new
 SetBucketTaggingConfigurationRequest(
 "amzn-s3-demo-bucket",
 bucketTaggingConfiguration
);

s3ClientV1.setBucketTaggingConfiguration(request);

// SDK V2
Tagging tagging = Tagging.builder()
 .tagSet(Arrays.asList(
 Tag.builder()
 .key("key1")
 .value("value1")
 .build(),
 Tag.builder()
 .key("key2")
 .value("value2")
```

```

 .build()
))
 .build();

PutBucketTaggingRequest request = PutBucketTaggingRequest.builder()
 .bucket("amzn-s3-demo-bucket")
 .tagging(tagging)
 .build();

s3ClientV2.putBucketTagging(request);

```

## V1 classes that require manual migration

### V1's `AccessControlList` to V2's `AccessControlPolicy`

```

// SDK V1
AccessControlList aclV1 = new AccessControlList();
aclV1.setOwner(new Owner("owner-id", "owner-name"));
aclV1.grantPermission(GroupGrantee.AllUsers, Permission.Read);

// SDK V2
// To migrate from V1 to V2, replace direct 'AccessControlList' modifications with an
// 'AccessControlPolicy.builder()' that contains both owner information and grants.
// Note that V2's approach requires building the complete permission set upfront rather
// than modifying permissions incrementally.
AccessControlPolicy acpV2 = AccessControlPolicy.builder()
 .owner(Owner.builder()
 .id("owner-id")
 .displayName("owner-name")
 .build())
 .grants(Arrays.asList(
 Grant.builder()
 .grantee(Grantee.builder()
 .type(Type.GROUP)
 .uri("http://acs.amazonaws.com/groups/global/AllUsers")
 .build())
 .permission(Permission.READ)
 .build())
))
 .build();

```

## V1's CannedAccessControlList enum to V2's BucketCannedACL and ObjectCannedACL enums

```
// SDK V1
// In V1, 'CannedAccessControlList' is an enumeration of predefined ACLs.
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard().build();

// Creating a bucket.
s3ClientV1.setBucketAcl("bucket-name", CannedAccessControlList.PublicRead);

// Creating an object.
PutObjectRequest putObjectRequest = new PutObjectRequest("bucket-name", "object-key",
 file)
 .withCannedAcl(CannedAccessControlList.PublicRead);
s3ClientV1.putObject(putObjectRequest);

// SDK V2
// V2 replaces V1's 'CannedAccessControlList' with 'BucketCannedACL' for buckets and
// 'ObjectCannedACL' for objects.
S3Client s3ClientV2 = S3Client.builder().build();

// Creating a bucket.
PutBucketAclRequest bucketRequest = PutBucketAclRequest.builder()
 .bucket("bucket-name")
 .acl(BucketCannedACL.PRIVATE)
 .build();
s3ClientV2.putBucketAcl(bucketRequest);

// Creating an object.
PutObjectRequest objectRequest = PutObjectRequest.builder()
 .bucket("bucket-name")
 .key("object-key")
 .acl(ObjectCannedACL.PUBLIC_READ)
 .build();
s3ClientV2.putObject(objectRequest, RequestBody.fromFile(file));
```

## V1's BucketNotificationConfiguration to V2's NotificationConfiguration

```
//SDK V1
BucketNotificationConfiguration notificationConfig = new
 BucketNotificationConfiguration();
```

```
// Adding configurations by name
notificationConfig.addConfiguration("lambdaConfig",
 new LambdaConfiguration("arn:aws:lambda:function", "s3:ObjectCreated:"));

notificationConfig.addConfiguration("topicConfig",
 new TopicConfiguration("arn:aws:sns:topic", "s3:ObjectRemoved:"));

notificationConfig.addConfiguration("queueConfig",
 new QueueConfiguration("arn:aws:sqs:queue", "s3:ObjectRestore:*"));

s3Client.setBucketNotificationConfiguration("bucket", notificationConfig);

// SDK V2
// In V2, V1's BucketNotificationConfiguration is renamed to
// NotificationConfiguration.
// V2 contains no common abstract class for LambdaFunction/Topic/Queue configurations.
NotificationConfiguration notificationConfig = NotificationConfiguration.builder()
 .lambdaFunctionConfigurations(
 LambdaFunctionConfiguration.builder()
 .lambdaFunctionArn("arn:aws:lambda:function")
 .events(Event.valueOf("s3:ObjectCreated:"))
 .build())
 .topicConfigurations(
 TopicConfiguration.builder()
 .topicArn("arn:aws:sns:topic")
 .events(Event.valueOf("s3:ObjectRemoved:"))
 .build())
 .queueConfigurations(
 QueueConfiguration.builder()
 .queueArn("arn:aws:sqs:queue")
 .events(Event.valueOf("s3:ObjectRestore:*"))
 .build())
 .build();

s3Client.putBucketNotificationConfiguration(req -> req
 .bucket("bucket")
 .notificationConfiguration(notificationConfig));
```

## V1's MultiFactorAuthentication to V2's mfa method on a request builder

```
// SDK V1
AmazonS3 s3ClientV1 = AmazonS3ClientBuilder.standard()
```

```
.withRegion(Regions.US_WEST_2)
.build();

BucketVersioningConfiguration versioningConfig = new BucketVersioningConfiguration()
 .withStatus(BucketVersioningConfiguration.ENABLED);

// Create an MFA configuration object.
MultiFactorAuthentication mfa = new MultiFactorAuthentication(
 "arn:aws:iam::1234567890:mfa/user",
 "123456"
);

// Create the request object.
SetBucketVersioningConfigurationRequest request = new
 SetBucketVersioningConfigurationRequest(
 "bucket-name",
 versioningConfig,
 mfa
);

// Send the request.
s3ClientV1.setBucketVersioningConfiguration(request);

// SDK V2
// V2 replaces V1's MultiFactorAuthentication POJO with parameters you set on the
// request builder.
S3Client s3ClientV2 = S3Client.builder()
 .region(Region.US_WEST_2)
 .build();

PutBucketVersioningRequest request = PutBucketVersioningRequest.builder()
 .bucket("bucket-name")
 .versioningConfiguration(VersioningConfiguration.builder()
 .status(BucketVersioningStatus.ENABLED)
 .build())
 .mfa("arn:aws:iam::1234567890:mfa/user 123456") // Single method takes both MFA
 .build();

s3ClientV2.putBucketVersioning(request);
```

## Migrate the Transfer Manager from version 1 to version 2 of the AWS SDK for Java

This migration guide covers the key differences between Transfer Manager v1 and S3 Transfer Manager v2, including constructor changes, method mappings, and code examples for common operations. After reviewing these differences, you can successfully migrate your existing Transfer Manager code to take advantage of improved performance and asynchronous operations in v2.

### About the AWS SDK migration tool

The AWS SDK for Java provides an automated [migration tool](#) that can migrate much of the v1 Transfer Manager API to v2. However, the migration tool doesn't support several v1 Transfer Manager features. For these cases, you need to manually migrate Transfer Manager code using the guidance in this topic.

Throughout this guide, the **Migration Status** shows whether the migration tool can automatically migrate a constructor, method, or feature:

- **Supported:** The migration tool can automatically transform this code
- **Not Supported:** You need to manually migrate code

Even for items marked as "Supported," review the migration results and test thoroughly. Transfer Manager migration involves significant architectural changes from synchronous to asynchronous operations.

## Overview

S3 Transfer Manager v2 introduces significant changes to the Transfer Manager API. S3 Transfer Manager v2 is built on asynchronous operations and provides better performance, especially when you use the AWS CRT-based Amazon S3 client.

## Key differences

- **Package:** `com.amazonaws.services.s3.transfer` → `software.amazon.awssdk.transfer.s3`
- **Class name:** `TransferManager` → `S3TransferManager`
- **Client dependency:** Synchronous Amazon S3 client → Asynchronous Amazon S3 client (`S3AsyncClient`)
- **Architecture:** Synchronous operations → Asynchronous operations with `CompletableFuture`

- **Performance:** Enhanced with AWS CRT-based client support

High-level changes

Aspect	V1	V2
Maven dependency	aws-java-sdk-s3	s3-transfer-manager
Package	com.amazonaws.services.s3.transfer	software.amazon.awssdk.transfer.s3
Main class	TransferManager	S3TransferManager
Amazon S3 client	AmazonS3 (sync)	S3AsyncClient (async)
Return types	Blocking operations	CompletableFuture<T>

Maven dependencies

V1	V2
<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt;com.amazonaws&lt;/groupId&gt;       &lt;artifactId&gt;aws-java-sdk-bom&lt;/artifactId&gt;       &lt;version&gt;&gt; 1.12.787&lt;/version&gt;       &lt;type&gt;pom&lt;/type&gt;       &lt;scope&gt;import&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManagement&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt;com.amazonaws&lt;/groupId&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt;software.amazon.awssdk&lt;/groupId&gt;       &lt;artifactId&gt;bom&lt;/artifactId&gt;       &lt;version&gt;&gt; 2.31.68&lt;/version&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManagement&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt;software.amazon.awssdk&lt;/groupId&gt;</pre>



V1	V2
<pre>&lt;artifactId&gt;aws-java-sdk-s3&lt;/artifactId&gt; &lt;/dependency&gt; &lt;/dependencies&gt;</pre>	<pre>&lt;artifactId&gt;s3-transfer-man ager&lt;/artifactId&gt; &lt;/dependency&gt; &lt;!-- Optional: For enhanced performance with AWS CRT --&gt; &lt;dependency&gt; &lt;groupId&gt;software.amazon.aw ssdk.crt&lt;/groupId&gt; &lt;artifactId&gt;aws-crt&lt;/artifa ctId&gt; &lt;version&gt; 0.38.5&lt;/version&gt; &lt;/dependency&gt; &lt;/dependencies&gt;</pre>

<sup>1</sup> [Latest version.](#)   <sup>2</sup> [Latest version.](#)   <sup>3</sup> [Latest version.](#)

Client constructor migration

Supported constructors (automatic migration)

V1 constructor	V2 equivalent	Migration status
new TransferManager()	S3TransferManager. create()	Supported
TransferManagerBuilder. defaultTransferManager()	S3TransferManager. create()	Supported
TransferManagerBuilder. standard().build()	S3TransferManager. builder().build()	Supported
new TransferManager(AW SCredentials)	S3TransferManager. builder() .s3Clie nt(S3AsyncClient.bu ilder() .credentia lsProvider(...).bu ild()) .build()	Supported

V1 constructor	V2 equivalent	Migration status
<code>new TransferManager( AWSCredentialsProvider)</code>	<code>S3TransferManager.builder().s3Client(S3AsyncClient.builder().credentialsProvider(...).build()).build()</code>	Supported

### Unsupported constructors (manual migration required)

V1 constructor	V2 equivalent	Migration notes
<code>new TransferManager(AmazonS3)</code>	Manual migration required	Create an <code>S3AsyncClient</code> separately
<code>new TransferManager(AmazonS3, ExecutorService)</code>	Manual migration required	Create an <code>S3AsyncClient</code> and configure executor
<code>new TransferManager(AmazonS3, ExecutorService, boolean)</code>	Manual migration required	<code>shutdownThreadPools</code> parameter not supported

### Manual migration examples

#### V1 code:

```
AmazonS3 s3Client = AmazonS3ClientBuilder.defaultClient();
TransferManager transferManager = new TransferManager(s3Client);
```

#### V2 code:

```
// Create an `S3AsyncClient` with similar configuration
S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
 .credentialsProvider(DefaultCredentialsProvider.create())
```

```

 .build();

// Provide the configured `S3AsyncClient` to the S3 transfer manager builder.
S3TransferManager transferManager = S3TransferManager.builder()
 .s3Client(s3AsyncClient)
 .build();

```

## Client method migration

Currently, the migration tool supports basic copy, download, upload, uploadDirectory, downloadDirectory, resumeDownload, and resumeUpload methods.

### Core transfer methods

V1 method	V2 method	Return type change	Migration status
upload(String, String, File)	uploadFile(UploadFileRequest)	Upload → FileUpload	Supported
upload(PutObjectRequest)	upload(UploadRequest)	Upload → Upload	Supported
download(String, String, File)	downloadFile(DownloadFileRequest)	Download → FileDownload	Supported
download(GetObjectRequest, File)	downloadFile(DownloadFileRequest)	Download → FileDownload	Supported
copy(String, String, String, String)	copy(CopyRequest)	Copy → Copy	Supported
copy(CopyObjectRequest)	copy(CopyRequest)	Copy → Copy	Supported
uploadDirectory(String, String, File, boolean)	uploadDirectory(UploadDirectoryRequest)	MultipleFileUpload → DirectoryUpload	Supported

V1 method	V2 method	Return type change	Migration status
<code>downloadDirectory(String, String, File)</code>	<code>downloadDirectory(DownloadDirectoryRequest)</code>	<code>MultipleFileDownload</code> → <code>DirectoryDownload</code>	Supported

## Resumable transfer methods

V1 method	V2 method	Migration status
<code>resumeUpload(PersistentUpload)</code>	<code>resumeUploadFile(ResumableFileUpload)</code>	Supported
<code>resumeDownload(PersistentDownload)</code>	<code>resumeDownloadFile(ResumableFileDownload)</code>	Supported

## Lifecycle methods

V1 method	V2 method	Migration status
<code>shutdownNow()</code>	<code>close()</code>	Supported
<code>shutdownNow(boolean)</code>	Manually adjust code using the <code>close()</code> method	Not Supported

## Unsupported V1 client methods

V1 method	V2 alternative	Notes
<code>abortMultipartUploads(String, Date)</code>	Use the low-level Amazon S3 client	Not Supported

V1 method	V2 alternative	Notes
<code>getAmazonS3Client()</code>	Save a reference separately	Not Supported; no getter in v2
<code>getConfiguration()</code>	Save a reference separately	Not Supported; no getter in v2
<code>uploadFileList(...)</code>	Make multiple <code>uploadFile()</code> calls	Not Supported
copy methods with a <code>TransferStateChangeListener</code> parameter	Use <code>TransferListener</code>	<a href="#">See manual migration example</a>
download methods with an <code>S3ProgressListener</code> parameter	Use <a href="#">TransferListener</a>	<a href="#">See manual migration example</a>
downloadDirectory methods with 4 or more parameters		<a href="#">See manual migration example</a>
upload method with an <code>ObjectMetadataProvider</code> parameter	Set metadata in request	<a href="#">See manual migration example</a>
uploadDirectory methods with <code>*Provider</code> parameter	Set tags in request	<a href="#">See manual migration example</a>

### copy methods with a `TransferStateChangeListener` parameter

- `copy(CopyObjectRequest copyObjectRequest, AmazonS3 srcS3, TransferStateChangeListener stateChangeListener)`
- `copy(CopyObjectRequest copyObjectRequest, TransferStateChangeListener stateChangeListener)`

```
// V1

// Initialize source S3 client
AmazonS3 s3client = AmazonS3ClientBuilder.standard()
 .withRegion("us-west-2")
 .build();

// Initialize Transfer Manager
TransferManager tm = TransferManagerBuilder.standard()
 .withS3Client(srcS3)
 .build();

CopyObjectRequest copyObjectRequest = new CopyObjectRequest(
 "amzn-s3-demo-source-bucket",
 "source-key",
 "amzn-s3-demo-destination-bucket",
 "destination-key"
);

TransferStateChangeListener stateChangeListener = new TransferStateChangeListener() {
 @Override
 public void transferStateChanged(Transfer transfer, TransferState state) {
 //Implementation of the TransferStateChangeListener
 }
};

Copy copy = tm.copy(copyObjectRequest, srcS3, stateChangeListener);

// V2

S3AsyncClient s3AsyncClient = S3AsyncClient.builder()
 .region(Region.US_WEST_2)
 .build();

S3TransferManager transferManager = S3TransferManager.builder()
 .s3Client(s3AsyncClient)
 .build();

// Create transfer listener (equivalent to TransferStateChangeListener in v1)

TransferListener transferListener = new TransferListener() {
 @Override
```

```

 public void transferInitiated(Context.TransferInitiated context) {
 //Implementation
 System.out.println("Transfer initiated");
 }

 @Override
 public void bytesTransferred(Context.BytesTransferred context) {
 //Implementation
 System.out.println("Bytes transferred");
 }

 @Override
 public void transferComplete(Context.TransferComplete context) {
 //Implementation
 System.out.println("Transfer completed!");
 }

 @Override
 public void transferFailed(Context.TransferFailed context) {
 //Implementation
 System.out.println("Transfer failed");
 }
 };

```

```

CopyRequest copyRequest = CopyRequest.builder()
 .copyObjectRequest(req -> req
 .sourceBucket("amzn-s3-demo-source-bucket")
 .sourceKey("source-key")
 .destinationBucket("amzn-s3-demo-destination-bucket")
 .destinationKey("destination-key")
)
 .addTransferListener(transferListener) // Configure the
transferListener into the request
 .build();

Copy copy = transferManager.copy(copyRequest);

```

## download methods with an S3ProgressListener parameter

- `download(GetObjectRequest getObjectRequest, File file, S3ProgressListener progressListener)`
- `download(GetObjectRequest getObjectRequest, File file, S3ProgressListener progressListener, long timeoutMillis)`

- `download(GetObjectRequest getObjectRequest, File file, S3ProgressListener progressListener, long timeoutMillis, boolean resumeOnRetry)`

```
// V1

S3ProgressListener progressListener = new S3ProgressListener() {
 @Override
 public void progressChanged(com.amazonaws.event.ProgressEvent progressEvent) {
 long bytes = progressEvent.getBytesTransferred();
 ProgressEventType eventType = progressEvent.getEventType();
 // Use bytes and eventType as needed
 }

 @Override
 public void onPersistableTransfer(PersistableTransfer persistableTransfer) {

 }
};

Download download1 = tm.download(getObjectRequest, file, progressListener);
Download download2 = tm.download(getObjectRequest, file, progressListener,
 timeoutMillis)
Download download3 = tm.download(getObjectRequest, file, progressListener,
 timeoutMillis, true)

// V2

TransferListener transferListener = new TransferListener() {
 @Override
 public void transferInitiated(Context.InitializedContext context) {
 // Equivalent to ProgressEventType.TRANSFER_STARTED_EVENT
 System.out.println("Transfer initiated");
 }

 @Override
 public void bytesTransferred(Context.BytesTransferred context) {
 // Equivalent to ProgressEventType.REQUEST_BYTE_TRANSFER_EVENT
 long bytes = context.bytesTransferred();
 System.out.println("Bytes transferred: " + bytes);
 }

 @Override
```



```

 public void transferComplete(Context.TransferComplete context) {
 // Equivalent to ProgressEventType.TRANSFER_COMPLETED_EVENT
 System.out.println("Transfer completed");
 }

 @Override
 public void transferFailed(Context.TransferFailed context) {
 // Equivalent to ProgressEventType.TRANSFER_FAILED_EVENT
 System.out.println("Transfer failed: " + context.exception().getMessage());
 }
};

DownloadFileRequest downloadFileRequest =
 DownloadFileRequest.builder()
 .getObjectRequest(getObjectRequest)
 .destination(file.toPath())
 .addTransferListener(transferListener)
 .build();

// For download1
FileDownload download = transferManager.downloadFile(downloadFileRequest);

// For download2
CompletedFileDownload completedFileDownload = download.completionFuture()
 .get(timeoutMillis,
 TimeUnit.MILLISECONDS);

// For download3, the v2 SDK does not have a direct equivalent to the `resumeOnRetry`
// method of v1.
// If a download is interrupted, you need to start a new download request.

```

## downloadDirectory methods with 4 or more parameters

- `downloadDirectory(String bucketName, String keyPrefix, File destinationDirectory, boolean resumeOnRetry)`
- `downloadDirectory(String bucketName, String keyPrefix, File destinationDirectory, boolean resumeOnRetry, KeyFilter filter)`
- `downloadDirectory(String bucketName, String keyPrefix, File destinationDirectory, KeyFilter filter)`

```
// V1
```

```

KeyFilter filter = new KeyFilter() {
 @Override
 public boolean shouldInclude(S3ObjectSummary objectSummary) {
 //Filter implementation
 }
};

MultipleFileDownload multipleFileDownload = tm.downloadDirectory(bucketName, keyPrefix,
 destinationDirectory, filter);

// V2

// The v2 SDK does not have a direct equivalent to the `resumeOnRetry` method of v1.
// If a download is interrupted, you need to start a new download request.
DownloadFilter filter = new DownloadFilter() {
 @Override
 public boolean test(S3Object s3Object) {
 // Filter implementation.
 }
};

DownloadDirectoryRequest downloadDirectoryRequest =
 DownloadDirectoryRequest.builder()
 .bucket(bucketName)
 .filter(filter)
 .listObjectsV2RequestTransformer(builder ->
builder.prefix(keyPrefix))
 .destination(destinationDirectory.toPath())
 .build();

DirectoryDownload directoryDownload =
 transferManager.downloadDirectory(downloadDirectoryRequest);

```

## upload method with ObjectMetadata parameter

- `upload(String bucketName, String key, InputStream input, ObjectMetadata objectMetadata)`

```

// V1

ObjectMetadata metadata = new ObjectMetadata();
ObjectMetadata metadata = new ObjectMetadata();

```

```

metadata.setContentType("text/plain"); // System-defined metadata
metadata.setContentLength(22L); // System-defined metadata
metadata.addUserMetadata("myKey", "myValue"); // User-defined metadata

PutObjectRequest putObjectRequest = new PutObjectRequest(bucketName, key, inputStream,
 metadata);

Upload upload = transferManager.upload("amzn-s3-demo-bucket", "my-key", inputStream,
 metadata);

// V2

/* When you use an InputStream to upload in V2, you should specify the content length
 and use `RequestBody.fromInputStream()`.
 If you don't provide the content length, the entire stream will be buffered in
 memory.
 If you can't determine the content length, we recommend using the CRT-based S3
 client.
*/
Map<String, String> userMetadata = new HashMap<>();
userMetadata.put("x-amz-meta-myKey", "myValue");

PutObjectRequest putObjectRequest =
 PutObjectRequest.builder()
 .bucket("amzn-s3-demo-bucket1")
 .key("k")
 .contentType("text/plain") //System-defined metadata
 usually has separate methods in the builder.
 .contentLength(22L)
 .metadata(userMetadata) //metadata() is only for user-
defined metadata.
 .build();

UploadRequest uploadRequest =
 UploadRequest.builder()
 .putObjectRequest(putObjectRequest)
 .requestBody(AsyncRequestBody.fromInputStream(stream, 22L,
 executor))
 .build();

transferManager.upload(uploadRequest).completionFuture().join();

```

## uploadDirectory with ObjectMetadataProvider parameter

- `uploadDirectory(String bucketName, String virtualDirectoryKeyPrefix, File directory, boolean includeSubdirectories, ObjectMetadataProvider metadataProvider)`
- `uploadDirectory(String bucketName, String virtualDirectoryKeyPrefix, File directory, boolean includeSubdirectories, ObjectMetadataProvider metadataProvider, ObjectTaggingProvider taggingProvider)`
- `uploadDirectory(String bucketName, String virtualDirectoryKeyPrefix, File directory, boolean includeSubdirectories, ObjectMetadataProvider metadataProvider, ObjectTaggingProvider taggingProvider, ObjectCannedAclProvider cannedAclProvider)`

```
// V1

tm.uploadDirectory(bucketName, virtualDirectoryKeyPrefix, directory,
 includeSubdirectories, metadataProvider)
tm.uploadDirectory(bucketName, virtualDirectoryKeyPrefix, directory,
 includeSubdirectories, metadataProvider, taggingProvider)
tm.uploadDirectory(bucketName, virtualDirectoryKeyPrefix, directory,
 includeSubdirectories, metadataProvider, taggingProvider, cannedAclProvider)

// V2

UploadDirectoryRequest request = UploadDirectoryRequest.builder()
 .bucket(bucketName)
 .s3Prefix(virtualDirectoryKeyPrefix)
 .source(directory.toPath())
 .maxDepth(includeSubdirectories ? Integer.MAX_VALUE :
1)
 .uploadFileRequestTransformer(builder -> {
 // 1.Replace `ObjectMetadataProvider`,
`ObjectTaggingProvider`, and `ObjectCannedAclProvider` with an
 // `UploadFileRequestTransformer` that can
combine the functionality of all three *Provider implementations.
 // 2. Convert your v1 `ObjectMetadata` to v2
`PutObjectRequest` parameters.
 // 3. Convert your v1 `ObjectTagging` to v2
`Tagging`.
```

```

// 4. Convert your v1 `CannedAccessControlList`
to v2 `ObjectCannedACL`.
))
 .build();

DirectoryUpload directoryUpload = transferManager.uploadDirectory(request);

```

## Model object migration

In AWS SDK for Java 2.x, many of the `TransferManager` model objects have been redesigned, and several getter and setter methods available in v1's model objects are no longer supported.

In v2, you can use the `CompletableFuture<T>` class to perform actions when the transfer completes—either successfully or with an exception. You can use the `join()` method to wait for completion if needed.

## Core transfer objects

V1 class	V2 class	Migration status
<code>TransferManager</code>	<code>S3TransferManager</code>	Supported
<code>TransferManagerBuilder</code>	<code>S3TransferManager.Builder</code>	Supported
<code>Transfer</code>	<code>Transfer</code>	Supported
<code>AbortableTransfer</code>	<code>Transfer</code>	Supported (no separate class)
<code>Copy</code>	<code>Copy</code>	Supported
<code>Download</code>	<code>FileDownload</code>	Supported
<code>Upload</code>	<code>Upload / FileUpload</code>	Supported
<code>MultipleFileDownload</code>	<code>DirectoryDownload</code>	Supported
<code>MultipleFileUpload</code>	<code>DirectoryUpload</code>	Supported

**Persistence objects**

V1 class	V2 class	Migration status
PersistableDownload	ResumableFileDownload	Supported
PersistableUpload	ResumableFileUpload	Supported
PersistableTransfer	ResumableTransfer	Supported
PauseResult<T>	Direct resumable object	Not Supported

**Result objects**

V1 class	V2 class	Migration status
CopyResult	CompletedCopy	Supported
UploadResult	CompletedUpload	Supported

**Configuration objects**

V1 class	V2 class	Migration status
TransferManagerConfiguration	MultipartConfiguration (on Amazon S3 client)	Supported
TransferProgress	TransferProgress + TransferProgressSnapshot	Supported
KeyFilter	DownloadFilter	Supported

## Unsupported objects

V1 class	V2 alternative	Migration status
PauseStatus	Not supported	Not Supported
UploadContext	Not supported	Not Supported
ObjectCannedAclProvider	PutObjectRequest.builder().acl()	Not Supported
ObjectMetadataProvider	PutObjectRequest.builder().metadata()	Not Supported
ObjectTaggingProvider	PutObjectRequest.builder().tagging()	Not Supported
PresignedUrlDownload	Not supported	Not Supported

## TransferManagerBuilder configuration migration

### Configuration changes

The configuration changes that you need to set for the v2 transfer manager depend on which S3 client that you use. You have the choice of the AWS CRT-based S3 client or the standard Java-based S3 async client. For information about the differences, see the [the section called “S3 clients in the SDK”](#) topic.

Use the AWS CRT-based S3 client

Setting	v1	v2 - Transfer Manager using AWS CRT-based S3 client
(get a builder)	<pre>TransferManagerBuilder tmBuilder =     TransferManagerBuilder.standard();</pre>	<pre>S3TransferManager.Builder tmBuilder =     S3TransferManager.builder();</pre>

Setting	v1	v2 - Transfer Manager using AWS CRT-based S3 client
S3 client	<pre>tmBuilder.withS3Client(...); tmBuilder.setS3Client(...);</pre>	<pre>tmBuilder.s3Client(...);</pre>
Executor	<pre>tmBuilder.withExecutorFactory(...); tmBuilder.setExecutorFactory(...);</pre>	<pre>tmBuilder.executor(...);</pre>
Shutdown thread pools	<pre>tmBuilder.withShutdownThreadPools(...); tmBuilder.setShutdownThreadPools(...);</pre>	Not supported. The provided executor will not be shut down when the <code>S3TransferManager</code> is closed
Minimum upload part size	<pre>tmBuilder.withMinimumUploadPartSize(...); tmBuilder.setMinimumUploadPartSize(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.crtBuilder().         minimumPartSizeInBytes(...).         build();  tmBuilder.s3Client(s3);</pre>



Setting	v1	v2 - Transfer Manager using AWS CRT-based S3 client
Multipart upload threshold	<pre>tmBuilder.withMultipartUploadThreshold(...); tmBuilder.setMultipartUploadThreshold(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.crtBuilder().         thresholdInBytes(...).build();  tmBuilder.s3Client(s3);</pre>
Minimum copy part size	<pre>tmBuilder.withMultipartCopyPartSize(...); tmBuilder.setMultipartCopyPartSize(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.crtBuilder().         minimumPartSizeInBytes(...).build();  tmBuilder.s3Client(s3);</pre>
Multipart copy threshold	<pre>tmBuilder.withMultipartCopyThreshold(...); tmBuilder.setMultipartCopyThreshold(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.crtBuilder().         thresholdInBytes(...).build();  tmBuilder.s3Client(s3);</pre>

Setting	v1	v2 - Transfer Manager using AWS CRT-based S3 client
Disable parallel downloads	<pre>tmBuilder.withDisableParallelDownloads(...); tmBuilder.setDisableParallelDownloads(...);</pre>	Disable parallel downloads by passing a standard Java-based S3 client <b>with multipart disabled (default)</b> to the transfer manager.  <pre>S3AsyncClient s3 =     S3AsyncClient.builder().build();  tmBuilder.s3Client(s3);</pre>
Always calculate multipart md5	<pre>tmBuilder.withAlwaysCalculateMultipartMd5(...); tmBuilder.setAlwaysCalculateMultipartMd5(...);</pre>	Not supported.

### Use Java-based S3 async client

Setting	v1	v2 - Transfer Manager using Java-based S3 async client
(get a builder)	<pre>TransferManagerBuilder tmBuilder =     TransferManagerBuilder.standard();</pre>	<pre>S3TransferManagerBuilder tmBuilder =     S3TransferManager.builder();</pre>
S3 client	<pre>tmBuilder.withS3Client(...);</pre>	<pre>tmBuilder.s3Client(...);</pre>

Setting	v1	v2 - Transfer Manager using Java-based S3 async client
	<code>tmBuilder.setS3Client(...);</code>	
Executor	<code>tmBuilder.withExecutorFactory(...);</code> <code>tmBuilder.setExecutorFactory(...);</code>	<code>tmBuilder.executor(...);</code>
Shutdown thread pools	<code>tmBuilder.withShutdownThreadPools(...);</code> <code>tmBuilder.setShutdownThreadPools(...);</code>	Not supported. The provided executor will not be shut down when the <code>S3TransferManager</code> is closed
Minimum upload part size	<code>tmBuilder.withMinimumUploadPartSize(...);</code> <code>tmBuilder.setMinimumUploadPartSize(...);</code>	<code>S3AsyncClient s3 = S3AsyncClient.builder().multipartConfiguration(cfg -&gt; cfg.minimumPartSizeInBytes(...)).build();</code>  <code>tmBuilder.s3Client(s3);</code>

Setting	v1	v2 - Transfer Manager using Java-based S3 async client
Multipart upload threshold	<pre>tmBuilder.withMultipartUploadThreshold(...); tmBuilder.setMultipartUploadThreshold(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.builder()         .multipartConfiguration(cfg -&gt;             cfg.thresholdInBytes(...)).             build();  tmBuilder.s3Client(s3);</pre>
Minimum copy part size	<pre>tmBuilder.withMultipartCopyPartSize(...); tmBuilder.setMultipartCopyPartSize(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.builder()         .multipartConfiguration(cfg -&gt;             cfg.minimumPartSizeInBytes(...)).             build();  tmBuilder.s3Client(s3);</pre>
Multipart copy threshold	<pre>tmBuilder.withMultipartCopyThreshold(...); tmBuilder.setMultipartCopyThreshold(...);</pre>	<pre>S3AsyncClient s3 =     S3AsyncClient.builder()         .multipartConfiguration(cfg -&gt;             cfg.thresholdInBytes(...)).             build();  tmBuilder.s3Client(s3);</pre>

Setting	v1	v2 - Transfer Manager using Java-based S3 async client
Disable parallel downloads	<pre>tmBuilder.withDisableParallelDownloads(...); tmBuilder.setDisableParallelDownloads(...);</pre>	Disable parallel downloads by passing a standard Java-based S3 client <b>with multipart disabled (default)</b> to the transfer manager.  <pre>S3AsyncClient s3 =     S3AsyncClient.builder().build();  tmBuilder.s3Client(s3);</pre>
Always calculate multipart md5	<pre>tmBuilder.withAlwaysCalculateMultipartMd5(...); tmBuilder.setAlwaysCalculateMultipartMd5(...);</pre>	Not supported.

## Behavior changes

### Asynchronous operations

#### V1 (blocking):

```
Upload upload = transferManager.upload("amzn-s3-demo-bucket", "key", file);
upload.waitForCompletion(); // Blocks until complete
```

#### V2 (asynchronous):

```
FileUpload upload = transferManager.uploadFile(UploadFileRequest.builder()
 .putObjectRequest(PutObjectRequest.builder()
 .bucket("amzn-s3-demo-bucket")
 .key("key")
 .build())
 .build())
```

```
.source(file)
.build());
```

```
CompletedFileUpload result = upload.completionFuture().join(); // Blocks until complete
// Or handle asynchronously:
upload.completionFuture().thenAccept(result -> {
 System.out.println("Upload completed: " + result.response().eTag());
});
```

## Error handling

**V1:** Directory transfers fail completely if any sub-request fails.

**V2:** Directory transfers complete successfully even if some sub-requests fail. Check for errors explicitly:

```
DirectoryUpload directoryUpload = transferManager.uploadDirectory(request);
CompletedDirectoryUpload result = directoryUpload.completionFuture().join();

// Check for failed transfers
if (!result.failedTransfers().isEmpty()) {
 System.out.println("Some uploads failed:");
 result.failedTransfers().forEach(failed ->
 System.out.println("Failed: " + failed.exception().getMessage()));
}
```

## Parallel download via byte-range fetches

When the automatic parallel transfer feature is enabled in the v2 SDK, the S3 Transfer Manager uses [byte-range fetches](#) to retrieve specific portions of the object in parallel (multipart download). The way an object is downloaded with v2 does not depend on how the object was originally uploaded. All downloads can benefit from high throughput and concurrency.

In contrast, with v1's Transfer Manager, it does matter how the object was originally uploaded. The v1 Transfer Manager retrieves the parts of the object the same way that the parts were uploaded. If an object was originally uploaded as a single object, the v1 Transfer Manager is not able to accelerate the downloading process by using sub-requests.

## Changes in parsing Amazon S3 URIs from version 1 to version 2

This topic details the changes in parsing Amazon S3 URIs from version 1 (v1) to version 2 (v2.).

## High-level changes

To begin parsing an S3 URI in v1, you instantiate an `AmazonS3URI` by using a constructor. In v2 you call `parseUri()` on an instance of `S3Utilities`, to return an `S3URI`.

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.587&lt;sup&gt;1&lt;/sup&gt;&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;     &lt;artifact Id&gt;s3&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.27.21&lt;sup&gt;2&lt;/sup&gt;&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifact Id&gt;s3&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>
Package name	com.amazonaws.serv ices.s3	software.amazon.aw ssdk.services.s3
Class names	<a href="#">AmazonS3URI</a>	<a href="#">S3URI</a>

<sup>1</sup> [Latest version.](#) <sup>2</sup> [Latest version.](#)

## API changes

Behavior	v1	v2
Parse an S3 URI.	<pre>URI uri = URI.create( "https://s3.amazonaws.com");  AmazonS3Uri s3Uri =     new AmazonS3URI(uri, false);</pre>	<pre>S3Client s3Client =     S3Client.create(); S3Utilities s3Utilities =     s3Client.utilities();  S3Uri s3Uri =     s3Utilities.parseUri(uri);</pre>
Retrieve the bucket name from an S3 URI.	<pre>String bucket =     s3Uri.getBucket();</pre>	<pre>Optional&lt;String&gt; bucket =     s3Uri.bucket();</pre>
Retrieve the key.	<pre>String key = s3Uri.getKey();</pre>	<pre>Optional&lt;String&gt; key =     s3Uri.key();</pre>
Retrieve the region.	<pre>String region =     s3Uri.getRegion();</pre>	<pre>Optional&lt;Region&gt; region =     s3Uri.region();  String region; if (s3Uri.region().isPresent()) {     region = s3Uri.region().get().id(); }</pre>
Retrieve whether the S3 URI is path style.	<pre>boolean isPathStyle =     s3Uri.isPathStyle();</pre>	<pre>boolean isPathStyle =     s3Uri.isPathStyle();</pre>
Retrieve the version ID.	<pre>String versionId =     s3Uri.getVersionId();</pre>	<pre>Optional&lt;String&gt; versionId =</pre>



Behavior	v1	v2
		<pre>s3Uri.firstMatchingRawQueryParameter("versionId");</pre>
Retrieve the query parameter <code>s.</code>	N/A	<pre>Map&lt;String, List&lt;String&gt;&gt; queryParams =     s3Uri.rawQueryParameters();</pre>

## Behavior changes

### URL encoding

v1 provides the option to pass in a flag to specify whether the URI should be URL encoded. The default value is `true`.

In v2, URL encoding is not supported. If you work with object keys or query parameters that have reserved or unsafe characters, you must URL encode them. For example you need to replace a whitespace " " with `%20`.

## Changes in the S3 Event Notifications API from version 1 to version 2

This topic details the changes in the S3 Event Notifications API from version 1.x (v1) to version 2.x (v2) of the AWS SDK for Java.

### High-level changes

#### Structural changes

V1 uses static inner classes for `EventNotificationRecord` types and their attributes, whereas v2 uses separate public classes for `EventNotificationRecord` types.

#### Naming convention changes

In v1, attribute class names include the suffix *Entity*, whereas v2 omits this suffix for simpler naming: for example, *eventData* instead of *eventDataEntity*.

## Changes in dependencies, packages and class names

In v1, S3 Event Notification API classes are transitively imported with along with the S3 module (artifactId `aws-java-sdk-s3`). However, in v2, you need to add a dependency on the `s3-event-notifications` artifact.

Change	v1	v2
Maven dependencies	<pre> &lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.X.X&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;     &lt;artifact Id&gt;aws-java-sdk-s3&lt;/ artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; </pre>	<pre> &lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.X.X<sup>1</sup>&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifactId&gt;s3- event-notifications&lt;/ artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; </pre>
Package name	<code>com.amazonaws.services.s3.event</code>	<code>software.amazon.awssdk.eventnotifications.s3.model</code>

Change	v1	v2
Class names	<a href="#"><u>S3EventNotification</u></a>	<a href="#"><u>S3EventNotification</u></a>
	<a href="#"><u>S3EventNotification.S3EventNotificationRecord</u></a>	<a href="#"><u>S3EventNotificationRecord</u></a>
	<a href="#"><u>S3EventNotification.GlacierEventDataEntity</u></a>	<a href="#"><u>GlacierEventData</u></a>
	<a href="#"><u>S3EventNotification.IntelligentTieringEventDataEntity</u></a>	<a href="#"><u>IntelligentTieringEventData</u></a>
	<a href="#"><u>S3EventNotification.LifecycleEventDataEntity</u></a>	<a href="#"><u>LifecycleEventData</u></a>
	<a href="#"><u>S3EventNotification.ReplicationEventDataEntity</u></a>	<a href="#"><u>ReplicationEventData</u></a>
	<a href="#"><u>S3EventNotification.RequestParametersEntity</u></a>	<a href="#"><u>RequestParameters</u></a>
	<a href="#"><u>S3EventNotification.ResponseElementsEntity</u></a>	<a href="#"><u>ResponseElements</u></a>
	<a href="#"><u>S3EventNotification.RestoreEventDataEntity</u></a>	<a href="#"><u>RestoreEventData</u></a>
	<a href="#"><u>S3EventNotification.S3BucketEntity</u></a>	<a href="#"><u>S3Bucket</u></a>
	<a href="#"><u>S3EventNotification.S3Entity</u></a>	<a href="#"><u>S3</u></a>
	<a href="#"><u>S3EventNotification.S3ObjectEntity</u></a>	<a href="#"><u>S3Object</u></a>
	<a href="#"><u>S3EventNotification.TransitionEventDataEntity</u></a>	<a href="#"><u>TransitionEventData</u></a>
		<a href="#"><u>UserIdentity</u></a>

Change	v1	v2
	<a href="#">S3EventNotification.UserIdentityEntity</a>	

<sup>1</sup> [Latest version.](#)

## API changes

### JSON to S3EventNotification and reverse

Use case	v1	v2
Create S3EventNotification from JSON String	<pre>S3EventNotification notification =     S3EventNotification.parseJson(message.body());</pre>	<pre>S3EventNotification notification =     S3EventNotification.fromJson(message.body());</pre>
Convert S3EventNotification to JSON String	<pre>String json = notification.toJson();</pre>	<pre>String json = notification.toJson();</pre>

### Access attributes of S3EventNotification

Use case	v1	v2
Retrieve records from a notification	<pre>List&lt;S3EventNotification.S3EventNotificationRecord&gt; records =     notification.getRecords();</pre>	<pre>List&lt;S3EventNotificationRecord&gt; records =     notification.getRecords();</pre>
Retrieve a record from a list of records	<pre>S3EventNotification.S3EventNotificationRecord record =</pre>	<pre>S3EventNotificationRecord record =</pre>

Use case	v1	v2
	<pre>records.stream().findAny().get();</pre>	<pre>records.stream().findAny().get();</pre>
Retrieve Glacier event data	<pre>S3EventNotification.GlacierEventDataEntity glacierEventData = record.getGlacierEventData();</pre>	<pre>GlacierEventData glacierEventData = record.getGlacierEventData();</pre>
Retrieve restore event data from a Glacier event	<pre>S3EventNotification.RestoreEventDataEntity restoreEventData = glacierEventData.getRestoreEventDataEntity();</pre>	<pre>RestoreEventData restoreEventData = glacierEventData.getRestoreEventData();</pre>
Retrieve request parameters	<pre>S3EventNotification.RequestParametersEntity requestParameters = record.getRequestParameters();</pre>	<pre>RequestParameters requestParameters = record.getRequestParameters();</pre>
Retrieve Intelligent Tiering event data	<pre>S3EventNotification.IntelligentTieringEventDataEntity tieringEventData = record.getIntelligentTieringEventData();</pre>	<pre>IntelligentTieringEventData intelligentTieringEventData = record.getIntelligentTieringEventData();</pre>
Retrieve lifecycle event data	<pre>S3EventNotification.LifecycleEventDataEntity lifecycleEventData = record.getLifecycleEventData();</pre>	<pre>LifecycleEventData lifecycleEventData = record.getLifecycleEventData();</pre>

Use case	v1	v2
Retrieve event name as enum	<pre>S3Event eventNameAsEnum = record.getEventNameAsEnum();</pre>	<pre>//getEventNameAsEnum does not exist; use 'getEventName()' String eventName =     record.getEventName();</pre>
Retrieve replication event data	<pre>S3EventNotification.ReplicationEventDataEntity replicationEntity =     record.getReplicationEventDataEntity();</pre>	<pre>ReplicationEventData replicationEventData =     record.getReplicationEventData();</pre>
Retrieve S3 bucket and object information	<pre>S3EventNotification.S3Entity s3 =     record.getS3();</pre>	<pre>S3 s3 = record.getS3();</pre>
Retrieve user identity information	<pre>S3EventNotification.UserIdentityEntity userIdentity =     record.getUserIdentity();</pre>	<pre>UserIdentity userIdentity =     record.getUserIdentity();</pre>
Retrieve response elements	<pre>S3EventNotification.ResponseElementsEntity responseElements =     record.getResponseElements();</pre>	<pre>ResponseElements responseElements =     record.getResponseElements();</pre>

## Migrate the aws-lambda-java-events library version

If you use [aws-lambda-java-events](#) to work with S3 notification events within a Lambda function, we recommend that you upgrade to the latest 3.x.x version. Recent versions eliminate all dependencies on AWS SDK for Java 1.x from the S3 event notification API.

For more information about the differences in handling S3 event notifications between the `aws-lambda-java-events` library and the SDK for Java 2.x, see [the section called “S3 event notification handling options”](#).

## Profile file changes

The AWS SDK for Java 2.x parses the profile definitions in `~/.aws/config` and `~/.aws/credentials` to more closely emulate the way the AWS CLI parses the files.

The SDK for Java 2.x:

- Resolves a `~/` or `~` followed by the file system's default path separator at the start of the path by checking, in order, `$HOME`, `$USERPROFILE` (Windows only), `$HOMEDRIVE`, `$HOMEPATH` (Windows only), and then the `user.home` system property.
- Looks for the `AWS_SHARED_CREDENTIALS_FILE` environment variable instead of `AWS_CREDENTIAL_PROFILES_FILE`.
- Silently drops profile definitions in configuration files without the word `profile` at the beginning of the profile name.
- Silently drops profile definitions that do not consist of alphanumeric, underscore or dash characters (after the leading `profile` word has been removed for configuration files).
- Merges settings of profile definitions duplicated within the same file.
- Merges settings of profile definitions duplicated in both the configuration and credentials files.
- Does NOT merge settings if both `[profile foo]` and `[foo]` are found in the same file.
- Uses settings in `[profile foo]` if both `[profile foo]` and `[foo]` are found in the configuration file.
- Uses the value of the last duplicated setting in the same file and profile.
- Recognizes both `;` and `#` for defining a comment.
- Recognizes `;` and `#` in profile definitions to define a comment, even if the characters are adjacent to the closing bracket.
- Recognizes `;` and `#` to define a comment only in setting values only if they are preceded by whitespace.
- Recognizes `;` and `#` and all following content in setting values if they are not preceded by whitespace.
- Considers role-based credentials the highest-priority credentials. The 2.x SDK always uses role-based credentials if the user specifies the `role_arn` property.

- Considers session-based credentials the second-highest-priority credentials. The 2.x SDK always uses session-based credentials if role-based credentials were not used and the user specifies the `aws_access_key_id` and `aws_session_token` properties.
- Uses basic credentials if role-based and session-based credentials are not used and the user specified the `aws_access_key_id` property.

## Environment variables and system properties changes

1.x Environment Variable	1.x System Property	2.x Environment Variable	2.x System Property
<code>AWS_ACCESS_KEY_ID</code> <code>AWS_ACCESS_KEY</code>	<code>aws.accessKeyId</code>	<code>AWS_ACCESS_KEY_ID</code>	<code>aws.accessKeyId</code>
<code>AWS_SECRET_KEY</code> <code>AWS_SECRET_ACCESS_KEY</code>	<code>aws.secretKey</code>	<code>AWS_SECRET_ACCESS_KEY</code>	<code>aws.secretAccessKey</code>
<code>AWS_SESSION_TOKEN</code>	<code>aws.sessionToken</code>	<code>AWS_SESSION_TOKEN</code>	<code>aws.sessionToken</code>
<code>AWS_REGION</code>	<code>aws.region</code>	<code>AWS_REGION</code>	<code>aws.region</code>
<code>AWS_CONFIG_FILE</code>		<code>AWS_CONFIG_FILE</code>	<code>aws.configFile</code>
<code>AWS_CREDENTIALS_PROFILE_FILE</code>		<code>AWS_SHARED_CREDENTIALS_FILE</code>	<code>aws.sharedCredentialsFile</code>
<code>AWS_PROFILE</code>	<code>aws.profile</code>	<code>AWS_PROFILE</code>	<code>aws.profile</code>
<code>AWS_EC2_METADATA_DISABLED</code>	<code>com.amazonaws.sdk.disableEc2Metadata</code>	<code>AWS_EC2_METADATA_DISABLED</code>	<code>aws.disableEc2Metadata</code>



1.x Environment Variable	1.x System Property	2.x Environment Variable	2.x System Property
	<code>com.amazonaws.sdk.ec2MetadataServiceEndpointOverride</code>	<code>AWS_EC2_METADATA_SERVICE_ENDPOINT</code>	<code>aws.ec2MetadataServiceEndpoint</code>
<code>AWS_CONTAINER_CREDENTIALS_RELATIVE_URI</code>		<code>AWS_CONTAINER_CREDENTIALS_RELATIVE_URI</code>	<code>aws.containerCredentialsPath</code>
<code>AWS_CONTAINER_CREDENTIALS_FULL_URI</code>		<code>AWS_CONTAINER_CREDENTIALS_FULL_URI</code>	<code>aws.containerCredentialsFullUri</code>
<code>AWS_CONTAINER_AUTHORIZATION_TOKEN</code>		<code>AWS_CONTAINER_AUTHORIZATION_TOKEN</code>	<code>aws.containerAuthorizationToken</code>
<code>AWS_CBOR_DISABLED</code>	<code>com.amazonaws.sdk.disableCbor</code>	<code>CBOR_ENABLED</code>	<code>aws.cborEnabled</code>
<code>AWS_ION_BINARY_DISABLE</code>	<code>com.amazonaws.sdk.disableIonBinary</code>	<code>BINARY_ION_ENABLED</code>	<code>aws.binaryIonEnabled</code>
<code>AWS_EXECUTION_ENV</code>		<code>AWS_EXECUTION_ENV</code>	<code>aws.executionEnvironment</code>

1.x Environment Variable	1.x System Property	2.x Environment Variable	2.x System Property
	<code>com.amazonaws.sdk.disableCertificateChecking</code>	Not supported ( <a href="#">Request feature</a> )	Not supported ( <a href="#">Request feature</a> )
	<code>com.amazonaws.sdk.enableDefaultMetrics</code>	<a href="#">Not supported</a>	<a href="#">Not supported</a>
	<code>com.amazonaws.sdk.enableThrottledRetry</code>	<a href="#">Not supported</a>	<a href="#">Not supported</a>
	<code>com.amazonaws.regions.RegionUtils.fileOverride</code>	Not supported ( <a href="#">Request feature</a> )	Not supported ( <a href="#">Request feature</a> )
	<code>com.amazonaws.regions.RegionUtils.disableRemote</code>	Not supported ( <a href="#">Request feature</a> )	Not supported ( <a href="#">Request feature</a> )
	<code>com.amazonaws.services.s3.disableImplicitGlobalClients</code>	Not supported ( <a href="#">Request feature</a> )	Not supported ( <a href="#">Request feature</a> )

1.x Environment Variable	1.x System Property	2.x Environment Variable	2.x System Property
	com.amazonaws.sdk.enableInRegionOptimizedMode	Not supported ( <a href="#">Request feature</a> )	Not supported ( <a href="#">Request feature</a> )

## Changes in Waiters from version 1 to version 2

This topic details the changes in the functionality of Waiters from version 1 (v1) to version 2 (v2).

The following tables demonstrate the difference for DynamoDB waiters specifically. Waiters for other services follow the same pattern.

### High-level changes

Waiters classes are in the same Maven artifact as the service.

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.680<sup>1</sup>&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.27.10<sup>2</sup>&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>

Change	v1	v2
	<pre> &lt;/dependencyManagement&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/groupId&gt;     &lt;artifactId&gt;dynamodb&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; </pre>	<pre> &lt;/dependencyManagement&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.awssdk&lt;/groupId&gt;     &lt;artifactId&gt;dynamodb&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; </pre>
Package name	com.amazonaws.services.dynamodbv2.waiters	software.amazon.awssdk.services.dynamodb.waiters
Class names	<a href="#">AmazonDynamoDBWaiters</a>	- Synchronous: <a href="#">DynamoDbWaiter</a>    - Asynchronous: <a href="#">DynamoDbAsyncWaiter</a>

<sup>1</sup> [Latest version.](#) <sup>2</sup> [Latest version.](#)

## API changes

Change	v1	v2
Create a waiter	<pre> AmazonDynamoDB client = AmazonDynamoDBClientBuilder      .standard().build(); AmazonDynamoDBWaiters waiter = client.waiters(); </pre>	Synchronous:  <pre> DynamoDbClient client = DynamoDbClient.create(); DynamoDbWaiter waiter = client.waiter(); </pre>  Asynchronous:

Change	v1	v2
		DynamoDbAsyncClient asyncClient =     DynamoDbA syncClient.create(); DynamoDbAsyncWaiter waiter = asyncClie nt.waiter();

Change	v1	v2
Wait until a table exists	<b>Synchronous:</b>  <pre>waiter.tableExists()     .run(new WaiterParameters&lt;&gt;() {         new DescribeTableRequest(tableName));</pre>  <b>Asynchronous:</b>  <pre>waiter.tableExists()     .runAsync(new WaiterParameters()         .withRequest(new DescribeTableRequest(tableName)),         new WaiterHandler() {             @Override             public void onSuccess(                 AmazonWebServiceRequest amazonWebServiceRequest) {                 System.out.println("Table creation succeeded");             }             @Override             public void onFailure(Exception e) {                 e.printStackTrace();             }         })</pre>	<b>Synchronous:</b>  <pre>WaiterResponse&lt;DescribeTableResponse&gt;     waiterResponse =         waiter.waitForTableExists(             r -&gt; r.tableName("myTable")); waiterResponse.matched().response()     .ifPresent(System.out::println);</pre>  <b>Asynchronous:</b>  <pre>waiter.waitForTableExists(r -&gt;     r.tableName(tableName))     .whenComplete((r, t) -&gt; {         if (t != null) {             t.printStackTrace();         } else {             System.out.println("Table creation succeeded");         }     }).join();</pre>

Change	v1	v2
	<code>}).get();</code>	

Configuration changes

Change	v1	v2
Polling Strategy (max attempts and fixed delay)	<pre>MaxAttemptsRetryStrategy maxAttemptsRetryStrategy =     new MaxAttemptsRetryStrategy(10);  FixedDelayStrategy fixedDelayStrategy =     new FixedDelayStrategy(3);  PollingStrategy pollingStrategy =     new PollingStrategy(maxAttemptsRetryStrategy,         fixedDelayStrategy);  waiter.tableExists().run(     new WaiterParameters&lt;&gt;(         new DescribeTableRequest(tableName),         pollingStrategy);</pre>	<pre>FixedDelayBackoffStrategy fixedDelayBackoffStrategy =     FixedDelayBackoffStrategy.create(         Duration.ofSeconds(3));  waiter.waitForTableExists(r -&gt; r.tableName(),     c -&gt; c.maxAttempts(10)         .backoffStrategy(fixedDelayBackoffStrategy));</pre>

## Changes in the EC2 metadata utility from version 1 to version 2

This topic details the changes in the SDK for Java Amazon Elastic Compute Cloud (EC2) metadata utility from version 1 (v1) to version 2 (v2).

### High-level changes

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.587&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;     &lt;artifact Id&gt;aws-java-sdk-co re&lt;/artifactId&gt;   &lt;/dependency&gt; &lt;/dependencies&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.27.21&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifact Id&gt;imds&lt;/artifactId&gt;   &lt;/dependency&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifact Id&gt;apache-client&lt;sup&gt;3&lt;/sup&gt;&lt;/ artifactId&gt;</pre>



Change	v1	v2
		<pre>&lt;/dependency&gt; &lt;/dependencies&gt;</pre>
Package name	<code>com.amazonaws.util</code>	<code>software.amazon.awssdk.imds</code>
Instantiation approach	Use static utility methods; no instantiation:  <pre>String localHostName =     EC2MetadataUtils.getLocalHostName();</pre>	Use a static factory method:  <pre>Ec2MetadataClient client = Ec2MetadataClient.create();</pre>  Or use a builder approach:  <pre>Ec2MetadataClient client = Ec2MetadataClient.builder()     .endpointMode(EndpointMode.IPV6)     .build();</pre>
Types of clients	Synchronous only utility methods: <code>EC2MetadataUtils</code>	Synchronous: <code>Ec2MetadataClient</code>   Asynchronous: <code>Ec2MetadataAsyncClient</code>

<sup>1</sup> [Latest version.](#) <sup>2</sup> [Latest version.](#)

<sup>3</sup> Notice the declaration of the `apache-client` module for v2. V2 of the EC2 metadata utility requires an implementation of the `SdkHttpClient` interface for the synchronous metadata client, or the `SdkAsyncHttpClient` interface for the asynchronous metadata client. The [??? section](#) shows the list of HTTP clients that you can use.

## Requesting metadata

In v1, you use static methods that accept no parameters to request metadata for an EC2 resource. In contrast, you need to specify the path to the EC2 resource as a parameter in v2. The following table shows the different approaches.

v1	v2
<pre>String userMetaDatum = EC2MetadataUtils.getUserData();</pre>	<pre>Ec2MetadataClient client = Ec2MetadataClient.create(); Ec2MetadataResponse response =     client.get("/latest/user-data"); String userMetaDatum =     response.asString();</pre>

Refer to the [instance metadata categories](#) to find the path you need to supply to request a piece of metadata.

 **Note**

When you use an instance metadata client in v2, you should aim to use the same client for all request to retrieve metadata.

## Behavior changes

### JSON data

On EC2, the locally running Instance Metadata Service (IMDS) returns some metadata as JSON formatted strings. One such example is the dynamic metadata of an [instance identity document](#).

The v1 API contains separate methods for each piece of instance identity metadata, whereas the v2 API directly returns the JSON string. To work with the JSON string, you can use the [Document API](#) to parse the response and navigate the JSON structure.

The following table compares how you retrieve metadata of an instance identity document in v1 and v2.

Use case	v1	v2
Retrieve the Region	<pre> InstanceInfo instanceInfo =     EC2MetadataUtils.getInstanceInfo(); String region =     instanceInfo.getRegion(); </pre>	<pre> Ec2MetadataResponse response =     client.get("/latest/dynamic/instance-identity/document"); Document instanceInfo = response.asDocument(); String region =     instanceInfo.asMap().get("region").asString(); </pre>
Retrieve the instance id	<pre> InstanceInfo instanceInfo =     EC2MetadataUtils.getInstanceInfo(); String instanceId =     instanceInfo.getInstanceId(); </pre>	<pre> Ec2MetadataResponse response =     client.get("/latest/dynamic/instance-identity/document"); Document instanceInfo = response.asDocument(); String instanceId =     instanceInfo.asMap().get("instanceId").asString(); </pre>
Retrieve the instance type	<pre> InstanceInfo instanceInfo =     EC2MetadataUtils.getInstanceInfo(); String instanceType =     instanceInfo.getInstanceType(); </pre>	<pre> Ec2MetadataResponse response =     client.get("/latest/dynamic/instance-identity/document"); Document instanceInfo = response.asDocument(); String instanceType =     instanceInfo.asMap().get("instanceType").asString(); </pre>

## Endpoint resolution differences

The following table shows the locations that the SDK checks to resolve the endpoint to IMDS. The locations are listed in descending priority.

v1	v2
System property: <code>com.amazonaws.sdk.ec2MetadataServiceEndpointOverride</code>	Client builder configuration method: <code>endpoint(...)</code>
Environment variable: <code>AWS_EC2_METADATA_SERVICE_ENDPOINT</code>	System property: <code>aws.ec2MetadataServiceEndpoint</code>
Default Value: <code>http://169.254.169.254</code>	Config file: <code>~/.aws/config</code> with the <code>ec2_metadata_service_endpoint</code> setting
	Value associated with resolved endpoint-mode
	Default value: <code>http://169.254.169.254</code>

## Endpoint resolution in v2

When you explicitly set an endpoint by using the builder, that endpoint value takes priority over all other settings. When the following code executes, the `aws.ec2MetadataServiceEndpoint` system property and config file `ec2_metadata_service_endpoint` setting are ignored if they exist.

```
Ec2MetadataClient client = Ec2MetadataClient
 .builder()
 .endpoint(URI.create("endpoint.to.use"))
 .build();
```

## Endpoint-mode

With v2, you can specify an endpoint-mode to configure the metadata client to use the default endpoint values for IPv4 or IPv6. Endpoint-mode is not available for v1. The default value used for IPv4 is `http://169.254.169.254` and `http://[fd00:ec2::254]` for IPv6.

The following table shows the different ways that you can set the endpoint mode in order of descending priority.

		Possible values
Client builder configuration method: <code>endpointMode(...)</code>	<pre>Ec2MetadataClient client = Ec2MetadataClient .builder() .endpointMode(EndpointMode.IPV4) .build();</pre>	<code>EndpointMode.IPV4</code> , <code>EndpointMode.IPV6</code>
System property	<code>aws.ec2MetadataServiceEndpointMode</code>	IPv4, IPv6 (case does not matter)
Config file: <code>~/.aws/config</code>	<code>ec2_metadata_service_endpoint_mode</code> setting	IPv4, IPv6 (case does not matter)
Not specified in the previous ways	IPv4 is used	

## How the SDK resolves endpoint or endpoint-mode in v2

1. The SDK uses the value that you set in code on the client builder and ignores any external settings. Because the SDK throws an exception if both `endpoint` and `endpointMode` are called on the client builder, the SDK uses the endpoint value from whichever method you use.
2. If you do not set a value in code, the SDK looks to external configuration—first for system properties and then for a setting in the config file.
  - a. The SDK first checks for an endpoint value. If a value is found, it is used.
  - b. If the SDK still hasn't found a value, the SDK looks for endpoint mode settings.

3. Finally, if the SDK finds no external settings and you have not configured the metadata client in code, the SDK uses the IPv4 value of `http://169.254.169.254`.

## IMDSv2

Amazon EC2 defines two approaches to access instance metadata:

- Instance Metadata Service Version 1 (IMDSv1) – Request/response approach
- Instance Metadata Service Version 2 (IMDSv2) – Session-oriented approach

The following table compares how the Java SDKs work with IMDS.

v1	v2
IMDSv2 is used by default	Always uses IMDSv2
Attempts to fetch a session token for each request and falls back to IMDSv1 if it fails to fetch a session token	Keeps a session token in an internal cache that is reused for multiple requests

The SDK for Java 2.x supports only IMDSv2 and does not fall back to IMDSv1.

## Configuration differences

The following table lists the differing configuration options.

Configuration	v1	v2
Retries	Configuration not available	Configurable through builder method <code>retryPolicy(...)</code>
HTTP	Connection timeout configurable through the <code>AWS_METADATA_SERVICE_TIMEOUT</code> environment variable. The default is 1 second.	Configuration available by passing an HTTP client to the builder method <code>httpClient(...)</code> . The default

Configuration	v1	v2
		connection timeout for HTTP clients is 2 seconds.

Example v2 HTTP configuration

The following example shows how you can configure the metadata client. This example configures the connection timeout and uses the Apache HTTP client.

```
SdkHttpClient httpClient = ApacheHttpClient.builder()
 .connectionTimeout(Duration.ofSeconds(1))
 .build();

Ec2MetadataClient imdsClient = Ec2MetadataClient.builder()
 .httpClient(httpClient)
 .build();
```

Changes in Amazon CloudFront presigning from version 1 to version 2

This topic details the changes in the Amazon CloudFront from version 1 (v1) to version 2 (v2).

High-level changes

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.587&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.27.21&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;</pre>

Change	v1	v2
	<pre>         &lt;scope&gt;im port&lt;/scope&gt;         &lt;/dependency&gt;     &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;     &lt;dependency&gt;         &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;         &lt;artifact Id&gt;cloudfront&lt;/art ifactId&gt;         &lt;/dependency&gt;     &lt;/dependencies&gt; </pre>	<pre>         &lt;/dependency&gt;     &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;     &lt;dependency&gt;         &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;         &lt;artifact Id&gt;cloudfront&lt;/art ifactId&gt;         &lt;/dependency&gt;     &lt;/dependencies&gt; </pre>
Package name	com.amazonaws.serv ices.cloudfront	software.amazon.aw ssdk.services.clou dfront
Class names	<a href="#">CloudFrontUrlSigner</a>  <a href="#">CloudFrontCookieSigner</a>	<a href="#">CloudFrontUtilities</a>  <a href="#">SignedUrl</a>  <a href="#">CannedSignerRequest</a>  <a href="#">CustomSignerRequest</a>

<sup>1</sup> [Latest version.](#)   <sup>2</sup> [Latest version.](#)

## API changes

Behavior	v1	v2
Build a canned request	Arguments are passed directly to the API.	<pre> CannedSignerRequest cannedRequest =     CannedSig nerRequest.builder() </pre>



Behavior	v1	v2
		<pre>         .resourceUrl(resourceUrl)          .privateKey(privateKey)          .keyPairId(keyPairId)          .expirationDate(expirationDate)          .build(); </pre>
Build a custom request	Arguments are passed directly to the API.	<pre> CustomSignerRequest customRequest =     CustomSignerRequest.builder()          .resourceUrl(resourceUrl)          .resourceUrlPattern(resourceUrlPattern)          .privateKey(keyFile)          .keyPairId(keyPairId)          .expirationDate(expirationDate)          .activeDate(activeDate)          .ipRange(ipRange)          .build(); </pre>

Behavior	v1	v2
Generate a signed URL (canned)	<pre>String signedUrl =     CloudFrontUrlSigner.getSignedURLWith     CannedPolicy(         resourceUrl,         keyPairId, privateKe         y, expirationDate);</pre>	<pre>CloudFrontUtilities     cloudFrontUtilities =         CloudFrontUtilitie         s.create();  SignedUrl signedUrl =          cloudFrontUtilitie         s.getSignedUrlWith         CannedPolicy(canne         dRequest);  String url = signedUrl         .url();</pre>
Generate a signed cookie (custom)	<pre>CookiesForCustomPolicy     cookies =         CloudFrontCookieSi         gner.getCookiesFor         CustomPolicy(             resourceUrl,             privateKey, keyPairId             , expirationDate,                 activeDate,             ipRange);</pre>	<pre>CloudFrontUtilities     cloudFrontUtilities =         CloudFrontUtilitie         s.create();  CookiesForCustomPolicy     cookies =         cloudFrontUtilitie         s.getCookiesForCus         tomPolicy(customRe         quest);</pre>

## Refactored cookie headers in v2

In Java v1, the Java SDK delivers cookie headers as a `Map.Entry<String, String>`.

```
Map.Entry<String, String> signatureMap = cookies.getSignature();
String signatureKey = signatureMap.getKey(); // "CloudFront-Signature"
String signatureValue = signatureMap.getValue(); // "[SIGNATURE_VALUE]"
```

The Java v2 SDK delivers the entire header as a single `String`.

```
String signatureHeaderValue = cookies.signatureHeaderValue(); // "CloudFront-
Signature=[SIGNATURE_VALUE]"
```

## Changes in the IAM Policy Builder API from version 1 to version 2

This topic details the changes in the IAM Policy Builder API from version 1 (v1) to version 2 (v2).

### High-level changes

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.587&lt;sup&gt;1&lt;/sup&gt;&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;     &lt;artifact Id&gt;aws-java-sdk-co re&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.27.21&lt;sup&gt;2&lt;/sup&gt;&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifact Id&gt;iam-policy-buil der&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>

Change	v1	v2
Package name	<code>com.amazonaws.auth.policy</code>	<code>software.amazon.awssdk.policybuilder.iam</code>
Class names	<a href="#">Policy</a> <a href="#">Statement</a>      • <a href="#">Statement.Effect</a>     • <a href="#">IdentityManagementActions</a>     • <a href="#">Resource</a>     • <a href="#">Principal</a>     • <a href="#">Condition</a>	<a href="#">IamPolicy</a> <a href="#">IamStatement</a>      • <a href="#">IamEffect</a>     • <a href="#">IamAction</a>     • <a href="#">IamResource</a>     • <a href="#">IamPrincipal</a>     • <a href="#">IamCondition</a>     • <a href="#">IamConditionOperator</a>     • <a href="#">IamConditionKey</a>

<sup>1</sup> [Latest version.](#) <sup>2</sup> [Latest version.](#)

## API changes

Setting	v1	v2
Instantiate a policy	<pre>Policy policy = new Policy();</pre>	<pre>IamPolicy.Builder policyBuilder = IamPolicy.builder(); ... IamPolicy policy = policyBuilder.buil d();</pre>
Set id	<pre>policy.withtId(...); policy.setId(...);</pre>	<pre>policyBuilder.id(...);</pre>

Setting	v1	v2
Set version	N/A - uses default version of 2012-10-17	<pre>policyBuilder.version(...);</pre>
Create statement	<pre>Statement statement =     new Statement       (Effect.Allow)         .withActions(...)         .withConditions(...)         .withId(...)         .withPrincipals(...)         .withResources(...);</pre>	<pre>IamStatement statement =     IamStatement.builder()       .effect(IamEffect.ALLOW)       .actions(...)       .notActions(...)       .conditions(...)       .sid(...)       .principals(...)       .notPrincipals(...)       .resources(...)       .notResources(...)       .build();</pre>
Set statement	<pre>policy.withStatements(statement); policy.setStatements(statement);</pre>	<pre>policyBuilder.addStatement(statement);</pre>

## Differences in building a statement

### Actions

#### v1

The v1 SDK has [enum types](#) for service actions that represent [Action](#) elements in a policy statement. The following enum types are some examples.

- [IdentityManagementActions](#)
- [DynamoDBv2Actions](#)
- [SQSActions](#)

The following example shows the `SendMessage` constant for `SQSActions`.

```
Action action = SQSActions.SendMessage;
```

You cannot specify a [NotAction](#) element to a statement in v1.

#### v2

In v2, the [IamAction](#) interface represents all actions. To specify a [service-specific action](#) element, pass a string to the `create` method as shown in the following code.

```
IamAction action = IamAction.create("sqs:SendMessage");
```

You can specify a [NotAction](#) for a statement with v2 as shown in the following code.

```
IamAction action = IamAction.create("sqs:SendMessage");
IamStatement.builder().addNotAction(action);
```

### Conditions

#### v1

To represent statement conditions, the v1 SDK uses subclasses of [Condition](#).

- [ArnCondition](#)

- [BooleanCondition](#)
- [DateCondition](#)
- [IpAddressCondition](#)
- [NumericCondition](#)
- [StringCondition](#)

Each Condition subclass defines a comparison enum type to help define the condition. For example, the following shows a *not like* [string comparison](#) for a condition.

```
Condition condition = new StringCondition(StringComparisonType.StringNotLike, "key", "value");
```

## v2

In v2, you build a condition for a policy statement by using [IamCondition](#) and provide an [IamConditionOperator](#), which contains enums for all types.

```
IamCondition condition = IamCondition.create(IamConditionOperator.STRING_NOT_LIKE, "key", "value");
```

## Resources

### v1

A policy statement's [Resource](#) element is represented by the SDK's [Resource](#) class. You supply the ARN as a string in the constructor. The following subclasses provide convenience constructors.

- [S3BucketResource](#)
- [S3ObjectResource](#)
- [SQSQueueResource](#)

In v1, you can specify a [NotResource](#) element for a [Resource](#) by calling the `withIsNotType` method as shown in the following statement.

```
Resource resource = new Resource("arn:aws:s3:::amzn-s3-demo-bucket").withIsNotType(true);
```

## v2

In v2, you create a [Resource](#) element by passing an ARN to the `IamResource.create` method.

```
IamResource resource = IamResource.create("arn:aws:s3:::amzn-s3-demo-bucket");
```

An [IamResource](#) can be set as [NotResource](#) element as shown in the following snippet.

```
IamResource resource = IamResource.create("arn:aws:s3:::amzn-s3-demo-bucket");
IamStatement.builder().addNotResource(resource);
```

`IamResource.ALL` represents all resources.

## Principals

### v1

The v1 SDK offers the following [Principal](#) classes to represent types of principals that include all members:

- `AllUsers`
- `AllServices`
- `AllWebProviders`
- `All`

You cannot add a [NotPrincipal](#) element to a statement.

### v2

In v2, `IamPrincipal.ALL` represents all principals:

To represent all members in other types of principals, use the [IamPrincipalType](#) classes when you create a `IamPrincipal`.

- `IamPrincipal.create(IamPrincipalType.AWS, "*")` for all users.
- `IamPrincipal.create(IamPrincipalType.SERVICE, "*")` for all services.
- `IamPrincipal.create(IamPrincipalType.FEDERATED, "*")` for all web providers.



- `IamPrincipal.create(IamPrincipalType.CANONICAL_USER, "*")` for all canonical users.

You can use the `addNotPrincipal` method to represent a [NotPrincipal](#) element when you create a policy statement as shown in the following statement.

```
IamPrincipal principal = IamPrincipal.create(IamPrincipalType.AWS,
 "arn:aws:iam::444455556666:root");
IamStatement.builder().addNotPrincipal(principal);
```

## Changes in working with DynamoDB from version 1 to version 2 of the AWS SDK for Java

### Topics

- [DynamoDB mapping API differences between version 1 and version 2 of the AWS SDK for Java](#)
- [Document API differences between version 1 and version 2 of the AWS SDK for Java](#)
- [Encryption library migration](#)

## DynamoDB mapping API differences between version 1 and version 2 of the AWS SDK for Java

The DynamoDB mapping APIs changed significantly between version 1 and version 2 of the AWS SDK for Java. In version 1, you use the `DynamoDBMapper` to work with Java POJOs. In version 2, you use the `DynamoDbEnhancedClient` with updated method names, enhanced schema definition options, and improved type safety.

Key differences include:

- New method names (such as `getItem` instead of `load`)
- Explicit table schema creation
- Built-in support for both synchronous and asynchronous operations
- Changes in how empty strings and configuration are handled

This section covers the mapping API changes, annotation differences, configuration updates, and migration guidance to help you transition from the v1 `DynamoDBMapper` to the v2 `DynamoDbEnhancedClient`.

## Contents

- [High-level changes in mapping libraries from version 1 to version 2 of the SDK for Java](#)
  - [Import dependency differences](#)
- [Changes in the DynamoDB mapping APIs between version 1 and version 2 of the SDK for Java](#)
  - [Create a client](#)
  - [Establish mapping to DynamoDB table/index](#)
  - [Table operations](#)
  - [Map classes and properties](#)
    - [Bean annotations](#)
    - [V2 additional annotations](#)
  - [Configuration](#)
    - [Per-operation configuration](#)
  - [Conditionals](#)
  - [Type conversion](#)
    - [Default converters](#)
    - [Set a custom converter for an attribute](#)
    - [Add a type converter factory or provider](#)
- [String handling differences between version 1 and version 2 of the SDK for Java](#)
- [Optimistic locking differences between version 1 and version 2 of the SDK for Java](#)
- [Fluent setters differences between version 1 and version 2 of the SDK for Java](#)

## High-level changes in mapping libraries from version 1 to version 2 of the SDK for Java

The names of the mapping client in each library differ in V1 and V2:

- V1 - DynamoDBMapper
- V2 - DynamoDB Enhanced Client

You interact with the two libraries in much the same way: you instantiate a mapper/client and then supply a Java POJO to APIs that read and write these items to DynamoDB tables. Both libraries also offer annotations for the class of the POJO to direct how the client handles the POJO.

Notable differences when you move to V2 include:

- V2 and V1 use different method names for the low-level DynamoDB operations. For example:

V1	V2
load	getItem
save	putItem
batchLoad	batchGetItem

- V2 offers multiple ways to define table schemas and map POJOs to tables. You can choose from the use of annotations or a schema generated from code using a builder. V2 also offers mutable and immutable versions of schemas.
- With V2, you specifically create the table schema as one of the first steps, whereas in V1, the table schema is inferred from the annotated class as needed.
- V2 includes the [Document API client](#) in the enhanced client API, whereas V1 uses a [separate API](#).
- All APIs are available in synchronous and asynchronous versions in V2.

See the [DynamoDB mapping section](#) in this guide for more detailed information on the V2 enhanced client.

## Import dependency differences

V1	V2
<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt;com.amazonaws&lt;/gro upId&gt;       &lt;artifactId&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.X.X&lt;/version&gt;       &lt;type&gt;pom&lt;/type&gt;       &lt;scope&gt;import&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManagement&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt;software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifactId&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.X.X* &lt;/version&gt;       &lt;type&gt;pom&lt;/type&gt;       &lt;scope&gt;import&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManagement&gt;  &lt;dependencies&gt;</pre>

V1	V2
<pre> &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt;com.amazonaws&lt;/groupId&gt;     &lt;artifactId&gt;aws-java-sdk-dynamodb&lt;/artifactId&gt;   &lt;/dependency&gt; &lt;/dependencies&gt; </pre>	<pre> &lt;dependency&gt;   &lt;groupId&gt;software.amazon.awssdk&lt;/groupId&gt;   &lt;artifactId&gt;dynamodb-enhanced&lt;/artifactId&gt; &lt;/dependency&gt; &lt;/dependencies&gt; </pre>

\* [Latest version.](#)

In V1, a single dependency includes both the low-level DynamoDB API and the mapping/document API, whereas in V2, you use the dynamodb-enhanced artifact dependency to access the mapping/document API. The dynamodb-enhanced module contains a transitive dependency on the low-level dynamodb module.

## Changes in the DynamoDB mapping APIs between version 1 and version 2 of the SDK for Java

### Create a client

Use case	V1	V2
Normal instantiation	<pre> AmazonDynamoDB standardClient = AmazonDynamoDBClientBuilder.standard()     .withCredentials(credentialsProvider)     .withRegion(Regions.US_EAST_1)     .build(); DynamoDBMapper mapper =     new DynamoDBMapper(standardClient); </pre>	<pre> DynamoDbClient standardClient = DynamoDbClient.builder()     .credentialsProvider(ProfileCredentialsProvider.create())     .region(Region.US_EAST_1)     .build(); DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder()     .dynamoDbClient(standardClient)     .build(); </pre>

Use case	V1	V2
Minimal instantiation	<pre>AmazonDynamoDB standardClient = AmazonDynamoDBClientBuilder.standard(); DynamoDBMapper mapper = new DynamoDBMapper(standardClient);</pre>	<pre>DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.create();</pre>
With attribute transformer <sup>*</sup>	<pre>DynamoDBMapper mapper = new DynamoDBMapper(standardClient,  attributeTransformerInstance);</pre>	<pre>DynamoDbEnhancedClient enhancedClient = DynamoDbEnhancedClient.builder() .dynamoDbClient(standardClient) .extensions(extensionAInstance, extensionBInstance) .build();</pre>

<sup>\*</sup>Extensions in V2 correspond roughly to attribute transformers in V1. The [the section called “Customize operations with extensions”](#) section contains more information on extensions in V2.

## Establish mapping to DynamoDB table/index

In V1, you specify a DynamoDB table name through a bean annotation. In V2, a factory method, `table()`, produces an instance of `DynamoDbTable` that represents the remote DynamoDB table. The first parameter of the `table()` method is the DynamoDB table name.

Use case	V1	V2
Map the Java POJO class to the DynamoDB table	<pre>@DynamoDBTable(tableName = "Customer") public class Customer {     ... }</pre>	<pre>DynamoDbTable&lt;Customer&gt; customerTable = enhancedClient.table("Customer", TableSchema.fromBean(Customer.class));</pre>

Use case	V1	V2
Map to a DynamoDB secondary index	1. Define a POJO class that represents the index.      • Annotate class with the <code>@DynamoDBTable</code> supplying the name of the table that has the index.     • Annotate properties with <code>@DynamoDBIndexHashKey</code> and optionally <code>@DynamoDBIndexRangeKey</code>.           2. Create a query expression.     3. Query using reference to the POJO class that represents the index. For example   <pre>mapper.query(IdEmailIndex.class,     queryExpression)</pre>    where <code>IdEmailIndex</code> is the mapping class for the index.   The section in the DynamoDB Developer Guide that discusses <a href="#">the V1 query method</a> shows a complete example.	1. Annotate attributes of a POJO class with <code>@DynamoDBSecondaryPartitionKey</code> (for a <a href="#">GSI</a>) and <code>@DynamoDBSecondarySortKey</code> (for and GSI or <a href="#">LSI</a>). For example,   <pre>@DynamoDBSecondary SortKey(indexNames =     "IdEmailIndex") public String getEmail() { return     this.email; }</pre>        2. Retrieve a reference to the index. For example,   <pre>DynamoDBIndex&lt;Customer&gt; customerIndex     = customerTable.index("IdEmailIndex");</pre>        3. Query the index.       The <a href="#">??? section</a> in this guide provides more information.

## Table operations

This section describes operation APIs that differ between V1 and V2 for most standard use cases.

In V2, all operations that involve a single table are called on the `DynamoDbTable` instance, not on the enhanced client. The enhanced client contains methods that can target multiple tables.

In the table named *Table operations* below, a POJO instance is referred to as `item` or as a specific type such as `customer1`. For the V2 examples the instances named, `table` is the result of previously calling `enhancedClient.table()` that returns a reference to the `DynamoDbTable` instance.

Note that most V2 operations can be called with a fluent consumer pattern even when not shown. For example,

```
Customer customer = table.getItem(r # r.key(key));
or
Customer customer = table.getItem(r # r.key(k ->
 k.partitionValue("id").sortValue("email")))
```

For V1 operations, *Table operations* (below) contains some of the commonly used forms and not all overloaded forms. For example, the `load()` method has the following overloads:

```
mapper.load(Customer.class, hashKey)
mapper.load(Customer.class, hashKey, rangeKey)
mapper.load(Customer.class, hashKey, config)
mapper.load(Customer.class, hashKey, rangeKey, config)
mapper.load(item)
mapper.load(item, config)
```

*Table operations* (below) shows the commonly used forms:

```
mapper.load(item)
mapper.load(item, config)
```

## Table operations

Use case	V1	V2
Write a Java POJO to a DynamoDB table   <b>DynamoDB operation:</b> PutItem, UpdateItem	<pre>mapper.save(item) mapper.save(item, config) mapper.save(item, saveExpression, config)</pre>  In V1, DynamoDBMapperConfig.SaveBehavior and annotations determines which low-level DynamoDB method will be called. In general, UpdateItem is called except when using SaveBehavior.CLOBBER and SaveBehavior.PUT. Auto-generated keys are a special use case, and occasionally both PutItem and UpdateItem are used.	<pre>table.putItem(putItemRequest) table.putItem(item) table.putItemWithResponse(item) // Returns metadata.  updateItem(updateItemRequest) table.updateItem(item) table.updateItemWithResponse(item) // Returns metadata.</pre>
Read an item from a DynamoDB table to a Java POJO   <b>DynamoDB operation:</b> GetItem	<pre>mapper.load(item) mapper.load(item, config)</pre>	<pre>table.getItem(getItemRequest) table.getItem(item) table.getItem(key) table.getItemWithResponse(key) // Returns POJO with metadata.</pre>



Use case	V1	V2
Delete an item from a Dynamo table   <b>Dynam</b> <b>operati</b> : Delete m	<pre>mapper.delete(item, deleteExpression, config)</pre>	<pre>table.deleteItem(deleteItemRequest) table.deleteItem(item) table.deleteItem(key)</pre>
Query a Dynamo table or secondary index and return a paginated list   <b>Dynam</b> <b>operati</b> : Query	<pre>mapper.query(Customer.class, queryExpression) mapper.query(Custom er.class, queryExpression, mapperConfig)</pre>	<pre>table.query(queryRequest) table.query(queryConditional)</pre>  Use the returned <code>PageIterable.stream()</code> (lazy loading) for sync responses and <code>PagePublisher.subscribe()</code> for async responses

Use case	V1	V2
Query a Dynamo table or second index and return a list   <b>Dynam operati :</b> Query	<pre>mapper.queryPage(Customer.class, queryExpression) mapper.queryPage(Customer.class, queryExpression,     mapperConfig)</pre>	<pre>table.query(queryRequest) table.query(queryConditional)</pre>  Use the returned <code>PageIterable.items()</code> (lazy loading) for sync responses and <code>PagePublisher.items.subscribe()</code> for async responses
Scan a Dynamo table or second index and return a paginated list   <b>Dynam operati :</b> Scan	<pre>mapper.scan(Customer.class, scanExpression) mapper.scan(Customer.class, scanExpression,     mapperConfig)</pre>	<pre>table.scan() table.scan(scanRequest)</pre>  Use the returned <code>PageIterable.stream()</code> (lazy loading) for sync responses and <code>PagePublisher.subscribe()</code> for async responses

Use case	V1	V2
Scan a DynamoDB table or secondary index and return a list  DynamoDB operation: Scan	<pre>mapper.scanPage(Customer.class, scanExpression) mapper.scanPage(Customer.class, scanExpression,                 mapperConfig)</pre>	<pre>table.scan() table.scan(scanRequest)</pre>  Use the returned <code>PageIterable.items()</code> (lazy loading) for sync responses and <code>PagePublisher.items.subscribe()</code> for async responses

Use case	V1	V2
Read multiple items from multiple tables in a batch   <b>Dynam</b> <b>operati</b> : BatchC tem	<pre>mapper.batchLoad(Arrays.asList(customer1,                                 customer2,                                 book1)) mapper.batchLoad(itemsToGet) // itemsToGet: Map&lt;Class&lt;?&gt;, List&lt;KeyPair&gt;&gt;</pre>	<pre>enhancedClient.batchGetItem(batchGetItemRequest)  enhancedClient.batchGetItem(r -&gt;     r.readBatches(         ReadBatch.builder(Record1.class)             .mappedTableResource(mappedTable1)             .addGetItem(i -&gt; i.key(k -&gt; k.partitionValue(0)))             .build(),         ReadBatch.builder(Record2.class)             .mappedTableResource(mappedTable2)             .addGetItem(i -&gt; i.key(k -&gt; k.partitionValue(0)))             .build()))  // Iterate over pages with lazy loading or over all items from the same table.</pre>
Write multiple items to multiple tables in a batch   <b>Dynam</b> <b>operati</b> : BatchW eItem	<pre>mapper.batchSave(Arrays.asList(customer1,                                 customer2,                                 book1))</pre>	<pre>enhancedClient.batchWriteItem(batchWriteItemRequest)  enhancedClient.batchWriteItem(r -&gt;     r.writeBatches(         WriteBatch.builder(Record1.class)             .mappedTableResource(mappedTable1)             .addPutItem(item1)             .build(),         WriteBatch.builder(Record2.class)             .mappedTableResource(mappedTable2)             .addPutItem(item2)             .build()))</pre>

Use case	V1	V2
Delete multiple items from multiple tables in a batch   Dynam operati : BatchV eItem	<pre>mapper.batchDelete(Arrays.asList(customer1, customer2, book1))</pre>	<pre>enhancedClient.batchWriteItem(r -&gt; r.writeBatches(WriteBatch.builder(Record1.class).mappedTableResource(mappedTable1).addDeleteItem(item1key).build(),WriteBatch.builder(Record2.class).mappedTableResource(mappedTable2).addDeleteItem(item2key).build()))</pre>
Write/delete multiple items in a batch   Dynam operati : BatchV eItem	<pre>mapper.batchWrite(Arrays.asList(customer1, book1), Arrays.asList(customer2))</pre>	<pre>enhancedClient.batchWriteItem(r -&gt; r.writeBatches(WriteBatch.builder(Record1.class).mappedTableResource(mappedTable1).addPutItem(item1).build(),WriteBatch.builder(Record2.class).mappedTableResource(mappedTable2).addDeleteItem(item2key).build()))</pre>

Use case	V1	V2
Carry out a transactional write   <b>Dynamic operation:</b> TransactionalWriteItems	<code>mapper.transactionWrite(transactionWriteRequest)</code>	<code>enhancedClient.transactWriteItems(transactWriteItemsRequest)</code>
Carry out a transactional read   <b>Dynamic operation:</b> TransactionalGetItems	<code>mapper.transactionLoad(transactionLoadRequest)</code>	<code>enhancedClient.transactGetItems(transactGetItemsRequest)</code>

Use case	V1	V2
Get a count of matching items of a query   <b>Dynam</b>   <b>operati</b>   :   Query with Select   UNT	<pre>mapper.count(Customer.class, queryExpression)</pre>	<pre>// Get the count from query results. PageIterable&lt;Customer&gt; pageIterable =     customerTable.query(QueryEnhancedRequest.builder()         .queryConditional(queryConditional)         .select(Select.COUNT)         .build()); Iterator&lt;Page&lt;Customer&gt;&gt; iterator =     pageIterable.iterator(); Page&lt;Customer&gt; page = iterator.next(); int count = page.count();  // For a more concise approach, you can // chain the method calls: int count = customerTable.query(QueryEnhancedRequest.builder()     .queryConditional(queryConditional)     .select(Select.COUNT)     .build())     .iterator().next().count();</pre>

Use case	V1	V2
Get a count of matching items of a scan  Dynam operati :Scan with Select UNT	<pre>mapper.count(Customer.class, scanExpression)</pre>	<pre>// Get the count from scan results. PageIterable&lt;Customer&gt; pageIterable =     customerTable.scan(ScanEnhancedRequest.builder()         .filterExpression(filterExpression)         .select(Select.COUNT)         .build()); Iterator&lt;Page&lt;Customer&gt;&gt; iterator =     pageIterable.iterator(); Page&lt;Customer&gt; page = iterator.next(); int count = page.count();  // For a more concise approach, you can // chain the method calls: int count = customerTable.scan(ScanEnhancedRequest.builder()     .filterExpression(filterExpression)     .select(Select.COUNT)     .build())     .iterator().next().count();</pre>



Use case	V1	V2
Create a table in DynamoDB corresponding to the POJO class  DynamoDB operation: CreateTable	<pre>mapper.generateCreateTableRequest(Customer.class)</pre>   The previous statement generates a low-level create table request; users must call <code>createTable</code> on the DynamoDB client.	<pre>table.createTable(createTableRequest)  table.createTable(r -&gt; r.provisionedThroughput(defaultThroughput())     .globalSecondaryIndices(         EnhancedGlobalSecondaryIndex.builder()             .indexName("gsi_1")             .projection(p -&gt; p.projectionType(ProjectionType.ALL))             .provisionedThroughput(defaultThroughput())             .build()));</pre>

Use case	V1	V2
Perform a parallel scan in DynamoDB  Dynam operati : Scan with Segmer and TotalS ents parame s	<pre>mapper.parallelScan(Custome r.class,  scanExpre ssion,  numTotalS egments)</pre>	Users are required to handle the worker threads and call scan for each segment:  <pre>table.scan(r -&gt; r.segment(0).total Segments(5))</pre>
Integra Amazon S3 with Dynam to store intellig nt S3 links	<pre>mapper.createS3Link(bucket, key) mapper.getS3ClientCache()</pre>	Not supported because it couples Amazon S3 and DynamoDB.

Map classes and properties

In both V1 and V2, you map classes to tables using bean-style annotations. V2 also offers [other ways to define schemas](#) for specific use cases, such as working with immutable classes.

Bean annotations

The following table shows the equivalent bean annotations for a specific use case that are used in V1 and V2. A Customer class scenario is used to illustrate parameters.

Annotations—as well as classes and enumerations—in V2 follow camel case convention and use 'DynamoDb', not 'DynamoDB'.

Use case	V1	V2
Map class to table	@DynamoDBTable (tableName = "CustomerTable")	@DynamoDbBean @DynamoDbBean(converterProviders = { ... })   The table name is defined when calling the <code>DynamoDbEnhancedClient#table()</code> method.
Designate a class member as a table attribute	@DynamoDBAttribute (attributeName = "customerName")	@DynamoDbAttribute("customerName")
Designate a class member is a hash/partition key	@DynamoDBHashKey	@DynamoDbPartitionKey
Designate a class member is a range/sort key	@DynamoDBRangeKey	@DynamoDbSortKey
Designate a class member is	@DynamoDBIndexHashKey	@DynamoDbSecondaryPartitionKey

Use case	V1	V2
a secondary index hash/partition key		
Designate a class member is a secondary index range/sort key	<code>@DynamoDBIndexRangeKey</code>	<code>@DynamoDbSecondarySortKey</code>
Ignore this class member when mapping to a table	<code>@DynamoDBIgnore</code>	<code>@DynamoDbIgnore</code>
Designate a class member as an auto-generated UUID key attribute	<code>@DynamoDBAutoGeneratedKey</code>	<code>@DynamoDbAutoGeneratedUuid</code>  The extension that provides this is not loaded by default; you must add the extension to client builder.
Designate a class member as an auto-generated timestamp attribute	<code>@DynamoDBAutoGeneratedTimestamp</code>	<code>@DynamoDbAutoGeneratedTimestampAttribute</code>  The extension that provides this is not loaded by default; you must add the extension to client builder.

Use case	V1	V2
Designate a class member as an auto-incremented version attribute	<code>@DynamoDBVersionAttribute</code>	<code>@DynamoDbVersionAttribute</code>  The extension that provides this is auto-loaded.
Designate a class member as requiring a custom conversion	<code>@DynamoDBTypeConverted</code>	<code>@DynamoDbConvertedBy</code>
Designate a class member to be stored as a different attribute type	<code>@DynamoDBTyped(&lt;DynamoDBAttributeType&gt;)</code>	Use an <code>AttributeConverter</code> implementation. V2 provides many built-in converters for common Java types. You can also implement your own custom <code>AttributeConverter</code> or <code>AttributeConverterProvider</code> . See <a href="#">the section called “Control attribute conversion”</a> in this guide.
Designate a class that can be serialized to a DynamoDB document (JSON-style document) or sub-document	<code>@DynamoDBDocument</code>	Use the Enhanced Document API. See the following resources:     <a href="#">the section called “Work with JSON documents”</a> in this guide    <a href="#">Blog post</a> published when the Enhanced Document API was released

## V2 additional annotations

Use case	V1	V2
Designate a class member not to be stored as a NULL attribute if the Java value is null	N/A	<code>@DynamoDbIgnoreNulls</code>
Designate a class member to be an empty object if all attributes are null	N/A	<code>@DynamoDbPreserveEmptyObject</code>
Designate special update action for a class member	N/A	<code>@DynamoDbUpdateBehavior</code>
Designate an immutable class	N/A	<code>@DynamoDbImmutable</code>
Designate a class member as an auto-incremented counter attribute	N/A	<code>@DynamoDbAtomicCounter</code>  The extension that provides this functionality is auto-loaded.

## Configuration

In V1, you generally control specific behaviors by using an instance of `DynamoDBMapperConfig`. You can supply the configuration object either when you create the mapper or when you make a request. In V2, configuration is specific to the request object for the operation.

Use case	V1	Default in V1	V2
	<code>DynamoDBMapperConfig.builder()</code>		

Use case	V1	Default in V1	V2
Batch load/write retry strategy	<pre>.withBatchLoadRetryStrategy(loadRetryStrategy)</pre> <pre>.withBatchWriteRetryStrategy(writeRetryStrategy)</pre>	retry failed items	Configure the retry strategy on the underlying <code>DynamoDBClient</code> . See <a href="#">the section called “Retries”</a> in this guide.
Consistent reads	<pre>.withConsistentReads(CONSISTENT)</pre>	EVENTUALLY	By default, consistent reads is false for read operations. Override with <code>.consistentRead(true)</code> on the request object.
Conversion schema with sets of marshallers/unmarshallers	<pre>.withConversionSchema(conversionSchema)</pre>  Static implementations provide backwards compatibility with older versions.	V2_COMPATIBLE	Not applicable. This is a legacy feature that refers to how the earliest versions of DynamoDB (V1) stored data types, and this behavior will not be preserved in the enhanced client. An example of behavior in DynamoDB V1 is storing booleans as Number instead of as Boolean.

Use case	V1	Default in V1	V2
Table names	<pre> .withObjectTableNameResolver() .withTableNameOverride() .withTableNameResolver() </pre>  Static implementations provide backwards compatibility with older versions	use annotation or guess from class	The table name is defined when calling the <code>DynamoDbEnhancedClient#table()</code> method.
Pagination load strategy	<pre> .withPaginationLoadingStrategy(strategy) </pre>  Options are: LAZY_LOADING, EAGER_LOADING , or ITERATION_ONLY	LAZY_LOADING	- Iteration only is the default. The other V1 options are not supported.    - You can implement the equivalent of eager loading in V2 with the following: <pre> List&lt;Customer&gt; allItems =     customerTable.scan().items()         .stream().collect(Collectors.toList()); </pre>     - For lazy loading, you must implement your own caching logic for accessed items.
Request metric collection	<pre> .withRequestMetricCollector(collector) </pre>	null	Use <code>metricPublisher()</code> in <code>ClientOverrideConfiguration</code> when building the standard DynamoDB client.



Use case	V1	Default in V1	V2
Save behavior	<pre>.withSaveBehavior(SaveBehavior.CLOBBER)</pre>  Options are UPDATE, CLOBBER, PUT, APPEND_SET , or UPDATE_SKIP_NULL_ATTRIBUTES .	UPDATE	In V2, you call <code>putItem()</code> or <code>updateItem()</code> explicitly.   CLOBBER or PUT: Corresponding action in v2 is calling <code>putItem()</code> . There is no specific CLOBBER configuration.   UPDATE: Corresponds to <code>updateItem()</code>   UPDATE_SKIP_NULL_ATTRIBUTES : Corresponds to <code>updateItem()</code> . Control update behavior with the request setting <code>ignoreNulls</code> and the annotation/tag <code>DynamoDbUpdateBehavior</code> .   APPEND_SET : Not supported
Type converter factory	<pre>.withTypeConverterFactory(typeConverterFactory)</pre>	standard type converters	Set on the bean by using  <pre>@DynamoDbBean(converterProviders = {ConverterProvider.class, DefaultAttributeConverterProvider.class})</pre>

## Per-operation configuration

In V1, some operations, such as `query()`, are highly configurable through an “expression” object submitted to the operation. For example:

```
DynamoDBQueryExpression<Customer> emailBwQueryExpr = new
DynamoDBQueryExpression<Customer>()
 .withRangeKeyCondition("Email",
 new Condition()
 .withComparisonOperator(ComparisonOperator.BEGINS_WITH)
 .withAttributeValueList(
 new AttributeValue().withS("my")));
```

```
mapper.query(Customer.class, emailBwQueryExpr);
```

In V2, instead of using a configuration object, you set parameters on the request object by using a builder. For example:

```
QueryEnhancedRequest emailBw = QueryEnhancedRequest.builder()
 .queryConditional(QueryConditional
 .sortBeginsWith(kb -> kb
 .sortValue("my")))
 .build();

customerTable.query(emailBw);
```

Conditionals

In V2, conditional and filtering expressions are expressed using an Expression object, which encapsulates the condition and the mapping of names and filters.

Use case	Operations	V1	V2
Expected attribute conditionals	save(), delete(), query(), scan()	<pre>new DynamoDBSaveExpression()     .withExpected(Collections.singletonMap(         "otherAttribute", new ExpectedAttributeValue(false)))     .withConditionalOperator(ConditionalOperator.AND);</pre>	Deprecated; use ConditionExpression instead.
Conditional expression	delete()	<pre>deleteExpression.setConditionExpression("zipcode = :zipcode")</pre>	<pre>Expression conditionExpression =     Expression.builder()         .expression("#key = :value OR #key1 = :value1")         .putExpressionName("#key", "attribute")</pre>

Use case	Operations	V1	V2
		<pre>deleteExpression .setExpressionAttributeValues(...)</pre>	<pre>        .putExpressionName         ("#key1", "attribute3")         .putExpressionValue         (":value", AttributeValues.stringValue("wrong"))         .putExpressionValue         (":value1", AttributeValues.stringValue("three"))         .build();  DeleteItemEnhancedRequest request = DeleteItemEnhancedRequest.builder()     .conditionExpression(conditionExpression).build();</pre>
Filter expression	<code>query()</code> , <code>scan()</code>	<pre>scanExpression     .withFilterExpression("#statename = :state")     .withExpressionAttributeValues(attributeValueMapBuilder.build())     .withExpressionAttributeNames(attributeNameMapBuilder.build())</pre>	<pre>Map&lt;String, AttributeValue&gt; values = singletonMap(":key", stringValue("value")); Expression filterExpression =     Expression.builder()         .expression("name = :key")         .expressionValues(values)         .build();  QueryEnhancedRequest request =     QueryEnhancedRequest.builder()         .filterExpression(filterExpression).build();</pre>

Use case	Operations	V1	V2
Conditional expression for query	query()	<pre>queryExpression.withKeyConditionExpression()</pre>	<pre>QueryConditional keyEqual =     QueryConditional.keyEqualTo(b -&gt;         b             .partitionValue("movie01"));  QueryEnhancedRequest tableQuery =     QueryEnhancedRequest.builder()         .queryConditional(keyEqual)         .build();</pre>

## Type conversion

### Default converters

In V2, the SDK provides a set of default converters for all common types. You can change type converters both at the overall provider level as well as for a single attribute. You can find a list of the available converters in the [AttributeConverter](#) API reference.

### Set a custom converter for an attribute

In V1, you can annotate a getter method with `@DynamoDBTypeConverted` to specify the class that converts between the Java attribute type and a DynamoDB attribute type. For instance, a `CurrencyFormatConverter` that converts between a Java Currency type and DynamoDB String can be applied as shown in the following snippet.

```
@DynamoDBTypeConverted(converter = CurrencyFormatConverter.class)
public Currency getCurrency() { return currency; }
```

The V2 equivalent of the previous snippet is shown below.

```
@DynamoDbConvertedBy(CurrencyFormatConverter.class)
public Currency getCurrency() { return currency; }
```

**Note**

In V1, you can apply the annotation to the attribute itself, a type or a user-defined annotation, V2 supports applying the annotation it only to the getter.

**Add a type converter factory or provider**

In V1, you can provide your own set of type converters, or override the types you care about by adding a type converter factory to the configuration. The type converter factory extends `DynamoDBTypeConverterFactory`, and overrides are done by getting a reference to the default set and extending it. The following snippet demonstrates how to do this.

```
DynamoDBTypeConverterFactory typeConverterFactory =
 DynamoDBTypeConverterFactory.standard().override()
 .with(String.class, CustomBoolean.class, new DynamoDBTypeConverter<String,
CustomBoolean>() {
 @Override
 public String convert(CustomBoolean bool) {
 return String.valueOf(bool.getValue());
 }
 @Override
 public CustomBoolean unconvert(String string) {
 return new CustomBoolean(Boolean.valueOf(string));
 }
 }).build();
DynamoDBMapperConfig config =
 DynamoDBMapperConfig.builder()
 .withTypeConverterFactory(typeConverterFactory)
 .build();
DynamoDBMapper mapperWithTypeConverterFactory = new DynamoDBMapper(dynamo, config);
```

V2 provides similar functionality through the `@DynamoDbBean` annotation. You can provide a single `AttributeConverterProvider` or a chain of ordered `AttributeConverterProviders`. Note that if you supply your own chain of attribute converter providers, you will override the default converter provider and must include it in the chain to use its attribute converters.

```
@DynamoDbBean(converterProviders = {
 ConverterProvider1.class,
 ConverterProvider2.class,
 DefaultAttributeConverterProvider.class})
public class Customer {
```

```
...
}
```

The section on [attribute conversion](#) in this guide contains a complete example for V2.

## String handling differences between version 1 and version 2 of the SDK for Java

V1 and V2 handle empty strings differently when sending data to DynamoDB:

- **V1:** Converts empty strings to null values before sending to DynamoDB (resulting in no attribute)
- **V2:** Sends empty strings as actual empty string values to DynamoDB

### Important

After migrating to V2, if you don't want empty strings stored in DynamoDB, you must implement custom converters. Without custom converters, V2 stores empty strings as actual empty string attributes in your DynamoDB items, which differs from V1's behavior of omitting these attributes entirely.

## Example custom converter for V2 that converts an empty string attribute to null

```
/**
 * Custom converter that maintains V1 behavior by converting empty strings to null
 * values
 * when writing to DynamoDB, ensuring compatibility with existing data. No attribute
 * will be saved to DynamoDB.
 */
public class NullifyEmptyStringConverter implements AttributeConverter<String> {
 @Override
 public AttributeValue transformFrom(String value) {
 if (value == null || value.isEmpty()) {
 return AttributeValue.builder().nul(true).build();
 }
 return AttributeValue.builder().s(value).build();
 }

 @Override
 public String transformTo(AttributeValue attributeValue) {
 if (attributeValue.nul()) {
 return null;
 }
 }
}
```

```
 }
 return attributeValue.s();
 }

 @Override
 public EnhancedType<String> type() {
 return EnhancedType.of(String.class);
 }

 @Override
 public AttributeValueType attributeValueType() {
 return AttributeValueType.S;
 }
}

// V2 usage:
@DynamoDbBean
public class Customer {
 private String name;

 @DynamoDbConvertedBy(NullifyEmptyStringConverter.class)
 public String getName() {
 return name;
 }
}
```

Optimistic locking differences between version 1 and version 2 of the SDK for Java

Both V1 and V2 implement optimistic locking with an attribute annotation that marks one property on your bean class to store the version number.

Differences in optimistic locking behavior

	V1	V2
Bean class annotation	@DynamoDBVersionAttribute	@DynamoDbVersionAttribute (note that V2 uses a lowercase "b")
Initial save	Version number	The starting value for the version attribute set with @DynamoDbVersionAttribute(startAt = <span style="color: red;">X</span> ). Default value is 0.

	V1	V2
	attribute set to 1.	
Update	The version number attribute is incremented by 1 if the conditional check verifies that the version number of the object being updated matches the number in the database.	The version number attribute is incremented if the conditional check verifies that the version number of the object being updated matches the number in the database.   The version number attribute incremented by the <code>incrementBy</code> option set with <code>@DynamoDbVersionAttribute(incrementBy = X)</code>. Default value is 1.



	V1	V2
Delete	DynamoDBMapper adds a conditional check that the version number of the object being deleted matches the version number in the database.	V2 does not automatically add conditions for the delete operations. You must add condition expressions manually if you want to control the delete behavior.   In the following example <code>recordVersion</code> is the bean's version attribute.  <pre>// 1. Read the item and get its current version. Customer item = customerTable.getItem(Key.builder().     partitionValue("someId").build()); AttributeValue currentVersion = item.getRecordVersion();  // 2. Create conditional delete with the `currentVersion` value. DeleteItemEnhancedRequest deleteItemRequest =     DeleteItemEnhancedRequest.builder()         .key(KEY)         .conditionExpression(Expression.builder()             .expression("recordVersion = :current_version_value")             .putExpressionValue(":current_version_value", currentVersion)             .build()).build();  customerTable.deleteItem(deleteItemRequest);</pre>
Transactional Write with a Condition Check	You <i>cannot</i> use a bean class that is annotated with <code>@DynamoDBVersionAttribute</code> in an <code>addConditionCheck</code> method.	You <i>can</i> use a bean class with the <code>@DynamoDBVersionAttribute</code> annotation in an <code>addConditionCheck</code> builder method for a <code>transactWriteItems</code> request.

	V1	V2
Disable	Disable optimistic locking by changing the DynamoDBMapperConfig.SaveBehavior enumeration value from UPDATE to CLOBBER.	Do not use the <code>@DynamoDbVersionAttribute</code> annotation.

## Fluent setters differences between version 1 and version 2 of the SDK for Java

You can use POJOs with fluent setters in the DynamoDB mapping API for V1 and with V2 since version 2.30.29.

For example, the following POJO returns a `Customer` instance from the `setName` method:

```
// V1

@DynamoDBTable(tableName = "Customer")
public class Customer{
 private String name;
 // Other attributes and methods not shown.
 public Customer setName(String name){
 this.name = name;
 return this;
 }
}
```

However, if you use a version of V2 prior to 2.30.29, `setName` returns a `Customer` instance with a `name` value of `null`.

```
// V2 prior to version 2.30.29.
```

```
@DynamoDbBean
public class Customer{
 private String name;
 // Other attributes and methods not shown.
 public Customer setName(String name){
 this.name = name;
 return this; // Bug: returns this instance with a `name` value of `null`.
 }
}
```

```
// Available in V2 since version 2.30.29.
```

```
@DynamoDbBean
public class Customer{
 private String name;
 // Other attributes and methods not shown.
 public Customer setName(String name){
 this.name = name;
 return this; // Returns this instance for method chaining with the `name` value
set.
 }
}
```

## Document API differences between version 1 and version 2 of the AWS SDK for Java

The Document API supports working with JSON-style documents as single items in a DynamoDB table. The V1 Document API has a corresponding API in V2, but instead of using a separate client for the document API as in V1, V2 incorporates document API features in the DynamoDB enhanced client.

In V1, the [Item](#) class represents an unstructured record from a DynamoDB table. In V2, an unstructured record is represented by an instance of the [EnhancedDocument](#) class. Note that primary keys are defined in the table schema for V2, and on the item itself in V1.

The table below compares the differences between the Document APIs in V1 and V2.

**Use case****V1****V2****Create a document client**

```
AmazonDynamoDB client
= ... //Create a client.
DynamoDB documentClient
= new DynamoDB(client);
```

```
// The V2 Document API
uses the same DynamoDbEnhancedClient
// that is used for
mapping POJOs.
DynamoDbClient
standardClient
= ... //Create a
standard client.
DynamoDbEnhancedClient enhancedClient
= ... // Create an
enhanced client.
```

**Reference a table**

```
Table documentTable
= docClient.documentClient("Person");
```

```
DynamoDbTable<EnhancedDocument>
documentTable =
enhancedClient.table("Person",
 TableSchema.documentSchemaBuilder()
 .addIndexPartitionKey(TableMetadata.primaryIndexName(), "id",
 AttributeValueType.S)
 .attributeConverterProviders(AttributeConverterProvider.defaultProvider())
 .build())
;
```

**Work with semi-structured data****Put item**

```
Item item = new Item()
```

```
EnhancedDocument
personDocument =
```

**Use case****V1**

```

 .withPrimaryKey("id", 50)
 .withString("firstName", "Shirley");
PutItemOutcome outcome
 = documentTable.putItem(item);

```

**V2**

```

EnhancedDocument.builder()
 .putNumber("id", 50)
 .putString("firstName", "Shirley")
 .build();
documentTable.putItem(personDocument);

```

**Get item**

```

GetItemOutcome outcome
 = documentTable.getItemOutcome("id", 50);
Item personDocFromDb = outcome.getItem();
String firstName = personDocFromDb.getString("firstName");

```

```

EnhancedDocument personDocFromDb =
 documentTable.getItem(
 Key.builder()
 .partitionValue(50)
 .build());
String firstName = personDocFromDb.getString("firstName");

```

**Work with JSON items****Convert a JSON structure to use it with the Document API**

```

// The 'jsonPerson' identifier is a JSON string.
Item item = new Item().fromJSON(jsonPerson);

```

```

// The 'jsonPerson' identifier is a JSON string.
EnhancedDocument document = EnhancedDocument.builder()
 .json(jsonPerson).build();

```

**Put JSON**

```

documentTable.putItem(item);

```

```

documentTable.putItem(document);

```

Use case	V1	V2
Read JSON	<pre>getItemOutcome outcome = //Get item. String jsonPerson = outcome.getItem().toJSON();</pre>	<pre>String jsonPerson = documentTable.getItem(Key.builder().partitionValue(50).build()).fromJson();</pre>

API reference and guides for document APIs

	V1	V2
API reference	<a href="#">API reference</a>	<a href="#">API reference</a>
Documentation guide	<a href="#">Amazon DynamoDB Developer Guide</a>	<a href="#">Enhanced Document API</a> (this guide)

V1 Xspec API to V2 Expressions API

The Expression Specification (Xspec) API available in V1 that helps create expressions to work with document-oriented data is not available in V2. V2 uses the Expression API, which works with both document-oriented data and object-to-item mapped data.

	V1	V2
API name	Expression Specification (Xspec) API	Expression API
Works with	Methods of the Document API <a href="#">Table</a> class such as <a href="#">updateItem</a> and <a href="#">scan</a>	Both APIs of DynamoDB Enhanced Client:  1. methods of the object-to-item mapping API 2. methods of the Enhanced Document API for working with document-oriented data (JSON)

	V1	V2
		For both types of APIs, after you have acquired a <a href="#">DynamoDbTable</a> instance:       • From a <a href="#">BeanTableSchema</a> , for instance, for the <a href="#">object-to-item mapping API</a>     • From a <a href="#">DocumentTableSchema</a> for the <a href="#">document-oriented API</a>       you use expressions in <code>DynamoDbTable</code> methods when you create request objects. For example in the <code>filterExpression</code> method of the <a href="#">QueryEnhancedRequest.Builder</a>
Resources	• <a href="#">Xpec initial release blog post</a>     • <a href="#">API reference</a>	<a href="#">Expressions information</a> in this Java Developer Guide

## Encryption library migration

For information about migrating the encryption library for DynamoDB to work with V2 of the Java SDK, see the [Amazon DynamoDB Encryption Client Developer Guide](#).

## Changes in automatic Amazon SQS request batching from version 1 to version 2

This topic details the changes in automatic request batching for Amazon SQS between version 1 and version 2 of the AWS SDK for Java.

### High-level changes

The AWS SDK for Java 1.x performs client-side buffering using a separate [AmazonSQSBufferedAsyncClient](#) class that requires explicit initialization for request batching.

The AWS SDK for Java 2.x simplifies and enhances buffering functionality with the [SqsAsyncBatchManager](#). The implementation of this interface provides automatic request batching capabilities directly integrated with the standard [SqsAsyncClient](#). To learn about v2's

SqsAsyncBatchManager, see the [the section called “Use automatic request batching”](#) topic in this guide.

Change	v1	v2
Maven dependencies	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;       &lt;artifact Id&gt;aws-java-sdk-bom&lt;/ artifactId&gt;       &lt;version&gt; 1.12.782<sup>1</sup>&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; com.amazonaws&lt;/gro upId&gt;     &lt;artifact Id&gt;aws-java-sdk-sqs&lt;/ artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>	<pre>&lt;dependencyManagement&gt;   &lt;dependencies&gt;     &lt;dependency&gt;       &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;       &lt;artifact Id&gt;bom&lt;/artifactId&gt;       &lt;version&gt; 2.31.15<sup>2</sup>&lt;/version&gt;       &lt;type&gt;pom&lt;/ type&gt;       &lt;scope&gt;im port&lt;/scope&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt; &lt;/dependencyManageme nt&gt; &lt;dependencies&gt;   &lt;dependency&gt;     &lt;groupId&gt; software.amazon.aw ssdk&lt;/groupId&gt;     &lt;artifact Id&gt;sqs&lt;/artifactId&gt;     &lt;/dependency&gt;   &lt;/dependencies&gt;</pre>
Package names	com.amazonaws.serv ices.sqs.buffered	software.amazon.aw ssdk.services.sqs. batchmanager
Class names	<a href="#">AmazonSQSBufferedA syncClient</a>	<a href="#">SqsAsyncBatchManager</a>



<sup>1</sup> [Latest version.](#)

<sup>2</sup> [Latest version.](#)

## Using automatic SQS request batching

Change	v1	v2
Create a batch manager	<pre>AmazonSQSAsync sqsAsync = new AmazonSQS AsyncClient(); AmazonSQSAsync bufferedSqs = new AmazonSQS BufferedAsyncClien t(sqsAsync);</pre>	<pre>SqsAsyncClient asyncClient = SqsAsyncClient.cre ate(); SqsAsyncBatchManager sqsAsyncBatchManager = asyncClie nt.batchManager();</pre>
Create a batch manager with custom configuration	<pre>AmazonSQSAsync sqsAsync = new AmazonSQS AsyncClient();  QueueBufferConfig queueBufferConfig = new QueueBufferConfig( ) .withMaxB atchOpenMs(200) .withMaxB atchSize(10) .withMinR eceiveWaitTimeMs(1 000) .withVisi bilityTimeoutSecon ds(20) .withRece iveMessageAttribut eNames(messageAttr ibuteValues);  AmazonSQSAsync bufferedSqs =</pre>	<pre>BatchOverrideConfi guration batchOver rideConfiguration = BatchOverrideConfi guration.builder() .sendRequ estFrequency(Durat ion.ofMillis(200)) .maxBatch Size(10) .receiveM essageMinWaitDurat ion(Duration.ofMil lis(1000)) .receiveM essageVisibilityTi meout(Duration.ofS econds(20)) .receiveM essageSystemAttrib uteNames(messageSy stemAttributeNames) .receiveM essageAttributeNam es(messageAttribut eValues)</pre>

Change	v1	v2
	<pre>new AmazonSQS BufferedAsyncClient(sqsAsync, queueBufferConfig);</pre>	<pre>.build();  SqsAsyncBatchManager sqsAsyncBatchManager = SqsAsyncBatchManager.builder() .override Configuration(batchOverrideConfiguration) .client(SqsAsyncClient.create()) .schedule dExecutor(Executors.newScheduledThreadPool(8)) .build();</pre>
Send messages	<pre>Future&lt;SendMessageResult&gt; sendResultFuture =     bufferedSqs.sendMessageAsync(new SendMessageRequest()  .withQueueUrl(queueUrl)  .withMessageBody(body));</pre>	<pre>CompletableFuture&lt;SendMessageResponse&gt; sendCompletableFuture =     sqsAsyncBatchManager.sendMessage(  SendMessageRequest.builder()  .queueUrl(queueUrl)  .messageBody(body)  .build());</pre>

Change	v1	v2
Delete messages	<pre>Future&lt;DeleteMessageResult&gt; deleteResultFuture =     bufferedSqs.deleteMessageAsync(new DeleteMessageRequest()          .withQueueUrl(queueUrl));</pre>	<pre>CompletableFuture&lt;DeleteMessageResponse&gt; deleteResultCompletableFuture     = sqsAsyncBatchManager.deleteMessage(          DeleteMessageRequest.builder()              .queueUrl(queueUrl)              .build());</pre>
Change visibility of messages	<pre>Future&lt;ChangeMessageVisibilityResult&gt; changeVisibilityResultFuture =     bufferedSqs.changeMessageVisibilityAsync         (new ChangeMessageVisibilityRequest()              .withQueueUrl(queueUrl)              .withVisibilityTimeout(20));</pre>	<pre>CompletableFuture&lt;ChangeMessageVisibilityResponse&gt;     changeResponseCompletableFuture         = sqsAsyncBatchManager.changeMessageVisibility(          ChangeMessageVisibilityRequest.builder()              .queueUrl(queueUrl)              .visibilityTimeout(20)              .build());</pre>

Change	v1	v2
Receive messages	<pre> ReceiveMessageResult receiveResult =     bufferedS qs.receiveMessage(         new     ReceiveMessageRequ est()      .withQueueUrl(queue eUrl)); </pre>	<pre> CompletableFuture&lt; ReceiveMessageResp onse&gt;     responseC ompletableFuture =     sqsAsyncBatchManag er.receiveMessage(      ReceiveMessageRequ est.builder()      .queueUrl(queueUrl)      .build()); </pre>

## Asynchronous return type differences

Change	v1	v2
Return type	Future<ResultType>	CompletableFuture<ResponseType>
Callback mechanism	Requires an AsyncHandler with separate onSuccess and onError methods	Uses CompletableFuture APIs provided by the JDK, such as whenComplete(), thenCompose(), thenApply()
Exception handling	Uses AsyncHandler#onError() method	Uses CompletableFuture APIs provided by the JDK, such as exceptionally(), handle(), or whenComplete()
Cancellation	Basic support through Future.cancel()	Cancelling a parent CompletableFuture

Change	v1	v2
		automatically cancels all dependent futures in the chain

## Asynchronous completion handling differences

Change	v1	v2
Response handler implementation	<pre> Future&lt;ReceiveMessageResult&gt; future =     bufferedSqs.receiveMessageAsync(         receiveRequest,         new AsyncHandler&lt;ReceiveMessageRequest, ReceiveMessageResult&gt;() {             @Override             public void             onSuccess(ReceiveMessageRequest             request,                  ReceiveMessageResult result) {                  List&lt;Message&gt; messages                 = result.getMessages                 ();                  System.out.println                 ("Received " +                 messages.size() + "                 messages");                  for                 (Message message :                 messages) {                  System.out.println </pre>	<pre> CompletableFuture&lt; ReceiveMessageResponse&gt; completableFuture = sqsAsyncBatchManager     .receiveMessage(ReceiveMessageRequest.builder()         .queueUrl         (queueUrl).build())         .whenComplete((receiveMessageResponse, throwable)         ) -&gt; {             if (throwable             != null) {                  System.err.println                 ("Error receiving                 messages: " + throwable                 .getMessage());                  throwable.printStackTrace();             } else {                  List&lt;Message&gt; messages                 = receiveMessageResponse.messages();                  System.out.println </pre>

Change	v1	v2
	<pre> ("Message ID: " +   message.getMessage   Id());    System.out.println   ("Body: " + message.g   etBody());         }       }        @Override       public void       onError(Exception e) {          System.err.println         ("Error receiving         messages: " + e.getMess         age());          e.printStackTrace();       }     }   ); </pre>	<pre> ("Received " +   messages.size() + "   messages");        for       (Message message :       messages) {          System.out.println         ("Message ID: " +         message.messageId());          System.out.println         ("Body: " + message.b         ody());       }     }   }); </pre>

## Key configuration parameters

Parameter	v1	v2
Maximum batch size	maxBatchSize (default 10 requests per batch)	maxBatchSize (default 10 requests per batch)
Batch wait time	maxBatchOpenMs (default 200 ms)	sendRequestFrequency (default 200 ms)
Visibility timeout	visibilityTimeoutSeconds (-1 for queue default)	receiveMessageVisibilityTimeout (queue default)

Parameter	v1	v2
Minimum wait time	<code>longPollWaitTimeOutSeconds</code> (20s when <code>longPoll</code> is true)	<code>receiveMessageMinWaitDuration</code> (default 50 ms)
Message attributes	Set using <code>ReceiveMessageRequest</code>	<code>receiveMessageAttributeNames</code> (none by default)
System attributes	Set using <code>ReceiveMessageRequest</code>	<code>receiveMessageSystemAttributeNames</code> (none by default)
Long polling	<code>longPoll</code> (default is true)	Not supported to avoid open connections waiting until the server sends the messages
Maximum wait time for long polling	<code>longPollWaitTimeOutSeconds</code> (default 20s)	Not supported to avoid open connections waiting until the server sends the messages
Maximum number of prefetched receive batches stored client-side	<code>maxDoneReceiveBatches</code> (10 batches)	Not supported because it is handled internally
Maximum number of active outbound batches processed simultaneously	<code>maxInflightOutboundBatches</code> (default 5 batches)	Not supported because it is handled internally
Maximum number of active receive batches processed simultaneously	<code>maxInflightReceiveBatches</code> (default 10 batches)	Not supported because it is handled internally

## Use the SDK for Java 1.x and 2.x side-by-side

You can use both versions of the AWS SDK for Java in your projects.

The following shows an example of the `pom.xml` file for a project that uses Amazon S3 from version 1.x and DynamoDB from version 2.27.21.

## Example of POM

This example shows a `pom.xml` file entry for a project that uses both 1.x and 2.x versions of the SDK.

```
<dependencyManagement>
 <dependencies>
 <dependency>
 <groupId>com.amazonaws</groupId>
 <artifactId>aws-java-sdk-bom</artifactId>
 <version>1.12.1</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>bom</artifactId>
 <version>2.27.21</version>
 <type>pom</type>
 <scope>import</scope>
 </dependency>
 </dependencies>
</dependencyManagement>

<dependencies>
 <dependency>
 <groupId>com.amazonaws</groupId>
 <artifactId>aws-java-sdk-s3</artifactId>
 </dependency>
 <dependency>
 <groupId>software.amazon.awssdk</groupId>
 <artifactId>dynamodb</artifactId>
 </dependency>
</dependencies>
```



# Security for the AWS SDK for Java

Cloud security at Amazon Web Services (AWS) is the highest priority. As an AWS customer, you benefit from a data center and network architecture that is built to meet the requirements of the most security-sensitive organizations. Security is a shared responsibility between AWS and you. The [Shared Responsibility Model](#) describes this as Security of the Cloud and Security in the Cloud.

**Security of the Cloud** – AWS is responsible for protecting the infrastructure that runs all of the services offered in the AWS Cloud and providing you with services that you can use securely. Our security responsibility is the highest priority at AWS, and the effectiveness of our security is regularly tested and verified by third-party auditors as part of the [AWS Compliance Programs](#).

**Security in the Cloud** – Your responsibility is determined by the AWS service you are using, and other factors including the sensitivity of your data, your organization's requirements, and applicable laws and regulations.

This AWS product or service follows the [shared responsibility model](#) through the specific Amazon Web Services (AWS) services it supports. For AWS service security information, see the [AWS service security documentation page](#) and [AWS services that are in scope of AWS compliance efforts by compliance program](#).

## Topics

- [Data protection in AWS SDK for Java 2.x](#)
- [Working with TLS in the SDK for Java](#)
- [Identity and Access Management](#)
- [Compliance Validation for this AWS Product or Service](#)
- [Resilience for this AWS Product or Service](#)
- [Infrastructure Security for this AWS Product or Service](#)

## Data protection in AWS SDK for Java 2.x

The AWS [shared responsibility model](#) applies to data protection in AWS SDK for Java. As described in this model, AWS is responsible for protecting the global infrastructure that runs all of the AWS Cloud. You are responsible for maintaining control over your content that is hosted on this infrastructure. You are also responsible for the security configuration and management tasks for

the AWS services that you use. For more information about data privacy, see [Data Privacy FAQ](#). For information about data protection in Europe, see the [General Data Protection Regulation \(GDPR\) Center](#).

For data protection purposes, we recommend that you protect AWS account credentials and set up individual users with AWS IAM Identity Center or AWS Identity and Access Management (IAM). That way, each user is given only the permissions necessary to fulfill their job duties. We also recommend that you secure your data in the following ways:

- Use multi-factor authentication (MFA) with each account.
- Use SSL/TLS to communicate with AWS resources. We require TLS 1.2 and recommend TLS 1.3.
- Set up API and user activity logging with AWS CloudTrail. For information about using CloudTrail trails to capture AWS activities, see [Working with CloudTrail trails](#) in the *AWS CloudTrail User Guide*.
- Use AWS encryption solutions, along with all default security controls within AWS services.
- Use advanced managed security services such as Amazon Macie, which assists in discovering and securing sensitive data that is stored in Amazon S3.
- If you require FIPS 140-3 validated cryptographic modules when accessing AWS through a command line interface or an API, use a FIPS endpoint. For more information about the available FIPS endpoints, see [Federal Information Processing Standard \(FIPS\) 140-3](#).

We strongly recommend that you never put confidential or sensitive information, such as your customers' email addresses, into tags or free-form text fields such as a **Name** field. This includes when you work with SDK for Java or other AWS services using the console, API, AWS CLI, or AWS SDKs. Any data that you enter into tags or free-form text fields used for names may be used for billing or diagnostic logs. If you provide a URL to an external server, we strongly recommend that you do not include credentials information in the URL to validate your request to that server.

## Working with TLS in the SDK for Java

The AWS SDK for Java uses the TLS capabilities of its underlying Java platform. In this topic, we show examples using the OpenJDK implementation used by [Amazon Corretto 17](#).

To work with AWS services, the underlying JDK must support a minimum version of TLS 1.2, but TLS 1.3 is recommended.

Users should consult the documentation of the the Java platform they are using with the SDK to find out which TLS versions are enabled by default as well as how to enable and disable specific TLS versions.

## How to check TLS version information

Using OpenJDK, the following code shows the use of [SSLContext](#) to print which TLS/SSL versions are supported.

```
System.out.println(Arrays.toString(SSLContext.getDefault().getSupportedSSLParameters().getProtocols()));
```

For example, Amazon Corretto 17 (OpenJDK) produces the following output.

```
[TLSv1.3, TLSv1.2, TLSv1.1, TLSv1, SSLv3, SSLv2Hello]
```

To see the SSL handshake in action and what version of TLS is used, you can use the system property **javax.net.debug**.

For example, run a Java applications that uses TLS.

```
java app.jar -Djavax.net.debug=ssl:handshake
```

The application logs the SSL handshake similar to the following.

```
...
javax.net.ssl|DEBUG|10|main|2022-12-23 13:53:12.221 EST|ClientHello.java:641|Produced
ClientHello handshake message (
"ClientHello": {
 "client version" : "TLSv1.2",
 ...
javax.net.ssl|DEBUG|10|main|2022-12-23 13:53:12.295 EST|ServerHello.java:888|Consuming
ServerHello handshake message (
"ServerHello": {
 "server version" : "TLSv1.2",
 ...
```

## Enforce a minimum TLS version

The SDK for Java always prefers the latest TLS version supported by the platform and service. If you wish to enforce a specific minimum TLS version, consult your Java platform's documentation.

For OpenJDK-based JVMs, you can use the system property `jdk.tls.client.protocols`.

For example, if you want SDK service clients in your application to use TLS 1.2, even though TLS 1.3 is available, provide the following system property.

```
java app.jar -Djdk.tls.client.protocols=TLSv1.2
```

## AWS API endpoints upgrade to TLS 1.2

See this [blog post](#) for information about AWS API endpoints moving to TLS 1.2 for the minimum version.

## Identity and Access Management

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be *authenticated* (signed in) and *authorized* (have permissions) to use AWS resources. IAM is an AWS service that you can use with no additional charge.

### Topics

- [Audience](#)
- [Authenticating with identities](#)
- [Managing access using policies](#)
- [How AWS services work with IAM](#)
- [Troubleshooting AWS identity and access](#)

## Audience

How you use AWS Identity and Access Management (IAM) differs, depending on the work that you do in AWS.

**Service user** – If you use AWS services to do your job, then your administrator provides you with the credentials and permissions that you need. As you use more AWS features to do your work, you might need additional permissions. Understanding how access is managed can help you request the right permissions from your administrator. If you cannot access a feature in AWS, see [Troubleshooting AWS identity and access](#) or the user guide of the AWS service you are using.

**Service administrator** – If you're in charge of AWS resources at your company, you probably have full access to AWS. It's your job to determine which AWS features and resources your service users should access. You must then submit requests to your IAM administrator to change the permissions of your service users. Review the information on this page to understand the basic concepts of IAM. To learn more about how your company can use IAM with AWS, see the user guide of the AWS service you are using.

**IAM administrator** – If you're an IAM administrator, you might want to learn details about how you can write policies to manage access to AWS. To view example AWS identity-based policies that you can use in IAM, see the user guide of the AWS service you are using.

## Authenticating with identities

Authentication is how you sign in to AWS using your identity credentials. You must be authenticated as the AWS account root user, an IAM user, or by assuming an IAM role.

You can sign in as a federated identity using credentials from an identity source like AWS IAM Identity Center (IAM Identity Center), single sign-on authentication, or Google/Facebook credentials. For more information about signing in, see [How to sign in to your AWS account](#) in the *AWS Sign-In User Guide*.

For programmatic access, AWS provides an SDK and CLI to cryptographically sign requests. For more information, see [AWS Signature Version 4 for API requests](#) in the *IAM User Guide*.

### AWS account root user

When you create an AWS account, you begin with one sign-in identity called the AWS account *root user* that has complete access to all AWS services and resources. We strongly recommend that you don't use the root user for everyday tasks. For tasks that require root user credentials, see [Tasks that require root user credentials](#) in the *IAM User Guide*.

### Federated identity

As a best practice, require human users to use federation with an identity provider to access AWS services using temporary credentials.

A *federated identity* is a user from your enterprise directory, web identity provider, or Directory Service that accesses AWS services using credentials from an identity source. Federated identities assume roles that provide temporary credentials.

For centralized access management, we recommend AWS IAM Identity Center. For more information, see [What is IAM Identity Center?](#) in the *AWS IAM Identity Center User Guide*.

## IAM users and groups

An [IAM user](#) is an identity with specific permissions for a single person or application. We recommend using temporary credentials instead of IAM users with long-term credentials. For more information, see [Require human users to use federation with an identity provider to access AWS using temporary credentials](#) in the *IAM User Guide*.

An [IAM group](#) specifies a collection of IAM users and makes permissions easier to manage for large sets of users. For more information, see [Use cases for IAM users](#) in the *IAM User Guide*.

## IAM roles

An [IAM role](#) is an identity with specific permissions that provides temporary credentials. You can assume a role by [switching from a user to an IAM role \(console\)](#) or by calling an AWS CLI or AWS API operation. For more information, see [Methods to assume a role](#) in the *IAM User Guide*.

IAM roles are useful for federated user access, temporary IAM user permissions, cross-account access, cross-service access, and applications running on Amazon EC2. For more information, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

## Managing access using policies

You control access in AWS by creating policies and attaching them to AWS identities or resources. A policy defines permissions when associated with an identity or resource. AWS evaluates these policies when a principal makes a request. Most policies are stored in AWS as JSON documents. For more information about JSON policy documents, see [Overview of JSON policies](#) in the *IAM User Guide*.

Using policies, administrators specify who has access to what by defining which **principal** can perform **actions** on what **resources**, and under what **conditions**.

By default, users and roles have no permissions. An IAM administrator creates IAM policies and adds them to roles, which users can then assume. IAM policies define permissions regardless of the method used to perform the operation.

## Identity-based policies

Identity-based policies are JSON permissions policy documents that you attach to an identity (user, group, or role). These policies control what actions identities can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see [Define custom IAM permissions with customer managed policies](#) in the *IAM User Guide*.

Identity-based policies can be *inline policies* (embedded directly into a single identity) or *managed policies* (standalone policies attached to multiple identities). To learn how to choose between managed and inline policies, see [Choose between managed policies and inline policies](#) in the *IAM User Guide*.

## Resource-based policies

Resource-based policies are JSON policy documents that you attach to a resource. Examples include IAM *role trust policies* and Amazon S3 *bucket policies*. In services that support resource-based policies, service administrators can use them to control access to a specific resource. You must [specify a principal](#) in a resource-based policy.

Resource-based policies are inline policies that are located in that service. You can't use AWS managed policies from IAM in a resource-based policy.

## Access control lists (ACLs)

Access control lists (ACLs) control which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format.

Amazon S3, AWS WAF, and Amazon VPC are examples of services that support ACLs. To learn more about ACLs, see [Access control list \(ACL\) overview](#) in the *Amazon Simple Storage Service Developer Guide*.

## Other policy types

AWS supports additional policy types that can set the maximum permissions granted by more common policy types:

- **Permissions boundaries** – Set the maximum permissions that an identity-based policy can grant to an IAM entity. For more information, see [Permissions boundaries for IAM entities](#) in the *IAM User Guide*.

- **Service control policies (SCPs)** – Specify the maximum permissions for an organization or organizational unit in AWS Organizations. For more information, see [Service control policies](#) in the *AWS Organizations User Guide*.
- **Resource control policies (RCPs)** – Set the maximum available permissions for resources in your accounts. For more information, see [Resource control policies \(RCPs\)](#) in the *AWS Organizations User Guide*.
- **Session policies** – Advanced policies passed as a parameter when creating a temporary session for a role or federated user. For more information, see [Session policies](#) in the *IAM User Guide*.

## Multiple policy types

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see [Policy evaluation logic](#) in the *IAM User Guide*.

## How AWS services work with IAM

To get a high-level view of how AWS services work with most IAM features, see [AWS services that work with IAM](#) in the *IAM User Guide*.

To learn how to use a specific AWS service with IAM, see the security section of the relevant service's User Guide.

## Troubleshooting AWS identity and access

Use the following information to help you diagnose and fix common issues that you might encounter when working with AWS and IAM.

### Topics

- [I am not authorized to perform an action in AWS](#)
- [I am not authorized to perform iam:PassRole](#)
- [I want to allow people outside of my AWS account to access my AWS resources](#)

### I am not authorized to perform an action in AWS

If you receive an error that you're not authorized to perform an action, your policies must be updated to allow you to perform the action.



The following example error occurs when the `mateojackson` IAM user tries to use the console to view details about a fictional `my-example-widget` resource but doesn't have the fictional `awes:GetWidget` permissions.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
awes:GetWidget on resource: my-example-widget
```

In this case, the policy for the `mateojackson` user must be updated to allow access to the `my-example-widget` resource by using the `awes:GetWidget` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

## I am not authorized to perform iam:PassRole

If you receive an error that you're not authorized to perform the `iam:PassRole` action, your policies must be updated to allow you to pass a role to AWS.

Some AWS services allow you to pass an existing role to that service instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in AWS. However, the action requires the service to have permissions that are granted by a service role. Mary does not have permissions to pass the role to the service.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

In this case, Mary's policies must be updated to allow her to perform the `iam:PassRole` action.

If you need help, contact your AWS administrator. Your administrator is the person who provided you with your sign-in credentials.

## I want to allow people outside of my AWS account to access my AWS resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether AWS supports these features, see [How AWS services work with IAM](#).
- To learn how to provide access to your resources across AWS accounts that you own, see [Providing access to an IAM user in another AWS account that you own](#) in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see [Providing access to AWS accounts owned by third parties](#) in the *IAM User Guide*.
- To learn how to provide access through identity federation, see [Providing access to externally authenticated users \(identity federation\)](#) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see [Cross account resource access in IAM](#) in the *IAM User Guide*.

## Compliance Validation for this AWS Product or Service

To learn whether an AWS service is within the scope of specific compliance programs, see [AWS services in Scope by Compliance Program](#) and choose the compliance program that you are interested in. For general information, see [AWS Compliance Programs](#).

You can download third-party audit reports using AWS Artifact. For more information, see [Downloading Reports in AWS Artifact](#).

Your compliance responsibility when using AWS services is determined by the sensitivity of your data, your company's compliance objectives, and applicable laws and regulations. For more information about your compliance responsibility when using AWS services, see [AWS Security Documentation](#).

This AWS product or service follows the [shared responsibility model](#) through the specific Amazon Web Services (AWS) services it supports. For AWS service security information, see the [AWS service security documentation page](#) and [AWS services that are in scope of AWS compliance efforts by compliance program](#).

## Resilience for this AWS Product or Service

The AWS global infrastructure is built around AWS Regions and Availability Zones.

AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected with low-latency, high-throughput, and highly redundant networking.

With Availability Zones, you can design and operate applications and databases that automatically fail over between zones without interruption. Availability Zones are more highly available, fault tolerant, and scalable than traditional single or multiple data center infrastructures.

For more information about AWS Regions and Availability Zones, see [AWS Global Infrastructure](#).

This AWS product or service follows the [shared responsibility model](#) through the specific Amazon Web Services (AWS) services it supports. For AWS service security information, see the [AWS service security documentation page](#) and [AWS services that are in scope of AWS compliance efforts by compliance program](#).

## Infrastructure Security for this AWS Product or Service

This AWS product or service uses managed services, and therefore is protected by the AWS global network security. For information about AWS security services and how AWS protects infrastructure, see [AWS Cloud Security](#). To design your AWS environment using the best practices for infrastructure security, see [Infrastructure Protection](#) in *Security Pillar AWS Well-Architected Framework*.

You use AWS published API calls to access this AWS Product or Service through the network. Clients must support the following:

- Transport Layer Security (TLS). We require TLS 1.2 and recommend TLS 1.3.
- Cipher suites with perfect forward secrecy (PFS) such as DHE (Ephemeral Diffie-Hellman) or ECDHE (Elliptic Curve Ephemeral Diffie-Hellman). Most modern systems such as Java 7 and later support these modes.

Additionally, requests must be signed by using an access key ID and a secret access key that is associated with an IAM principal. Or you can use the [AWS Security Token Service](#) (AWS STS) to generate temporary security credentials to sign requests.

This AWS product or service follows the [shared responsibility model](#) through the specific Amazon Web Services (AWS) services it supports. For AWS service security information, see the [AWS service security documentation page](#) and [AWS services that are in scope of AWS compliance efforts by compliance program](#).

# OpenPGP key for the AWS SDK for Java

All publicly available Maven artifacts for the AWS SDK for Java are signed using the OpenPGP standard. The public key that you need to verify the signature of an artifact is available in the following section.

## Current key

The following table shows OpenPGP key information for the current releases of the SDK for Java 1x and SDK for Java 2.x.

Key ID	0xAC107B386692DADD
Type	RSA
Size	4096/4096
Created	2016-06-30
Expires	2026-09-27
User ID	AWS SDKs and Tools <aws-dr-tools@amazon.com>
Key fingerprint	FEB9 209F 2F2F 3F46 6484 1E55 AC10 7B38 6692 DADD

To copy the following OpenPGP public key for the SDK for Java to the clipboard, select the "Copy" icon in the upper right corner.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

```
Comment: Hostname:
```

```
Version: Hockeypuck 2.2
```

```
xsFNBfd1gAUBEACqbmFbxdJgz1lD7wrlskQA1LLuSAC4p8ny9u/D2zLR8Ynk3Yz
mzJuQ+Kfjne2t+xTDex6MPJlMYp0viSwsX2psgvdmeYUpW9ap0lrThNYkc+W5fRc
buFehfbi9LSATZGJi8RG0sCCr5FsYVz0gEk85M2+PeM24cXhQIOZtQUjswX/pdk/
KduGtZASqNAYLKR0mRODzUuaokLPo24pfm9bnr1RnRtw5ktPAA5bM9ZZaGKriej
kT2lPffBbjp8F5AZvmGLtNm2Cmg4FKBvI04SQjy2jjrQ3wBzi5Lc9HTxDuHK/rtV
```

u6PewUe2WP1nx1XenhMZU1UK4YoSB9E9StQ2VxQiySLHSdxR7Ma4WgYdVLn9b0ie  
nj3QxLuQ1ZUKF79ES6JaM4t0z1gGcQeU1+UklgjFLuKwmzWRdEIFfxMyvH6qgKnd  
U+DioH5mcUwhwffAAsuIJyAdMIEUYh7IfzJJXQf+ff+XfOC16byOJFWrIGQkAzMu  
CEvaCfwtHC2Lpzo33/WRFeMAuzzd0QJ4uz4xFFvaS0SZHMLHWI9YV/+Pea3X99Ms  
0Nlek/LolAJh67MynHeVB0HKrq+fluorWepQivctzN6Y1N0kx5naTPGGaKWK7G2q  
TbcY5SMnkIWfLFSougj0Fvmjczq8iZRwYxWA+i+LQvsR9WEXEiQffIWRoQARAQAB  
zSxBV1MgU0RLcyBhbmQgVG9vbHMgPGF3cy1kci10b29sc0BhbWF6b24uY29tPsLB  
1AQTAQoAPgIbAwULCQgHAwUVCgkICwUWAgMBAAIEaQIXgBYhBP65IJ8vLz9GZIQe  
VawQezhmktrdBQJo12ZrBQkTQxnmAAoJEKwQezhmktrdi18P/A3De83MBx8bdcWJ  
Fot71Vk1TyBQFEgrtscySU0czEHx3tGbZgQLbMlyzjir0T03usxEk0eqTVK+RU+  
5uFXNZYQLwMj1HJ6S8tnfLe/ExM5WQ2KPwIUPfZs1GDDRQB2dIKSc+qYrP101vf4  
04iPgflHMW2bFh3zjxccaHCJyqc7Cau33eZFBAsRni1j0Uo7MeyX0h1XfW8pd48Q  
wZ11QVZ/6KmDiFWA0CZ+2svJ5cL0tgPoh10Qjoz0nHpNfuDILMrZ+e7tx2VTlkGH  
UGeNSydnrK8v9ztFn34KtU/k7NEWoVSyEi+5ICZL18FBwPqTwDVWxWxrqZCKiIpr  
8ZdJWdz2sJfgDFNCC6rKgCQ6FmaD9G76dYwkQ4AbZqAB1UzU3q36W1K0r3i0Ab5  
G4td0t4yqXHTe1x+ZUNaeW7gaCmtXAxLw00feJrcq/44b/SQP+qJ8sS0v76Yg2oF  
BsF5DW0VUFghbTyokHAoVR0yhBR4dUUisY39AqLSL8+Lp9Pr3wNuG19GLrMD5701  
piUb88B3Gwe1EiKV1gaKrvZ3mECDUiSMV00Z5iG8E4QDpNmVbJbV1uT821ubvt0v  
2Ko10Fa0uwCYGssdRGqEXNy6jz/Er8LAC3+nmGINDJQzrF+loYoSSkI2Nu71hMuL  
7iWwUPF70hDXoVSAAn4X3x6q2rGK0wsGUBBMBCgA+AhsDBQsJCACDBRUKCQgLBRYC  
AwEAAh4BAheAFiEE/rkgnY8vP0ZkhB5VrBB70GaS2t0FamjXZTsFCRNDGLYACgkQ  
rBB70GaS2t0/0w//YIv51vHtD+kwMmIvk3zpizDHY0zW2d0ezAo+C/DsSyC7wDl1  
Dixw34EQ1yLXH5xLR8CH1zup13JmmEp1ucdQggoeFbidxD18F1d7tJ0D1y3GGnTD  
0jA12ZC+W650h+wS1mD1FlaKjMGgkvJf0dA7RtU2T8dv3vt8dsxg76FMFS3+fqlC  
FN0AsNTn9zWR1SqbIfkMJK83aq6s/rcEV9VrAYgDgqex58fygB5EuTf842/IF7WZ  
Q9gd6fupB0mMZP5Ywd2uj/vsBTYakG+mgQwDxZuKPeEzAqnqqS7biSQ0U06Wozlq  
Yy4fSczE9GkBAvg0pGmbko+zHvpnjvX/h1CUpC6odvFy0AhZp6zyhs0QWz9thfqV  
lU8WlbgJ2atFDn5GUSxF/fe0Yzovlbb56sbYXuvMG9RiE0uJ1mBbZR3aIdZ1U6Do  
BHc/vjc5mWcV7JQSP7i4W/8W7X3UAuN9LdxB+IvF3Cwrgtlw2BWvA5A1co5Tnz8t  
P/CIVmBjk+sLme8W4kfLK3IWEbwC10dNnErI/MHRm65A2Y5EMIhwjr0i07SU1Pxa  
nPg30YJCdvjzdB8QE3/DBiMf014dISfKDVEWnfK8mZaYd/BeRm2gUAa9UrqSFCG  
B1A7Lg+eLI3US0FvWwJ4j5bBJqgLu+y7crIkiU0PAQuLk3l0+5uYU/I3DuLCwZQE  
EwEKAD4CGwMFCwkIBwMFFQoJCAsFFgIDAQACHgECF4AWIQT+uSCfLy8/RmSEH1Ws  
EHs4ZpLa3QUcZwAXCAUJEWvKgWAKCRCsEHs4ZpLa3ZdTEACMBLg2q9zk8ZH02nDz  
Sg5zc8Wlqq8WdxU0Pj8qx4U0rrMca7wyiUvrgoxPW51h1RVNUeMkDRfu9pSXc0VI  
V9LvmYE/WnwKR0ubgGbsC4T7M/LqV0/AulXil4d7IXc0614toa8LTNwTD5b0DgrN  
gvay1AzCU8kq1Qw1cKZ2gAfva3Ba7PWyLeUN4HTlGrXcw73G+0CofY1L8wqWxHCJ  
29XqQzeTEc6MDEeIlN1VdUcy8Qr5uwkEs134H9AxS5F1opJ4TqvXiDZsrSRRv57R  
XYmRZDWeYT+9PZaMsHXza5qgej7BfATxhYfICsNaY6MK3x6b+nDSKkoZg0+i09zh  
1YjpahhQe6G336v/3mRj0dKGCQRQ6znQ9ghUaB5z9zfvgh5A0EkTe3l8MqM+j5A6P  
VjSBBJAHKejxr7+wKJKIA6P+DqpsYAunzftwUzrLVqb+BZQ+DcTmVrE70PcMYJD5  
QglX/Le+WmWZHI154NXgpWU0UgZUBUge4DKrT+zCJ9iecPLKTW70cULyX0+rjb8  
8BGGrD5GP1HB3d0UXXT1MKCqg3qy1Bu2KnZTQiaEEEdZgSIGQbrW0JTMmmXJkKjokd  
JMA4vYeg5en51G9nRQjScPngx77IxxByNyFWTJdG1ENpJpsK9TtmENcpyUJtJZTJ

ZS0IRVPP5RzR5vInuXWq6VV0BMLB1AQTAQoAPgIbAwULCQgHAwUVCgkICwUWAgMB  
AAIeAQIXgBYhBP65IJ8vLz9GZIQeVawQezhmktrdBQJlJEoiBQkPj/2dAAoJEKwQ  
ezhmktrdx1YP/0vvym3jgX/pwnR7K1rafZMb1iKQBr0ISG8cdbaf4pqX5vuUZnyj  
w9C1/o0Nn7jJjnQx0IIzuBoxne2WN28ftM2w0nVxm85mAmz2fwQz/fdKDyonXc0h  
pFD2iMqn7gESjhEgRE7wMDYMDuLdqHI70KWGVfgrh7xEmKapLh45h7cnumo2VjL9  
uDYY1a0BHz993T7oE41y43rhk+6kKbGFd2uu07h5j1ZF8Lj6sYfcEzX0U10hR1D0  
nyBjDy9MYWu0YNouc70WgMceGx6hjvCAM/5fxP7SZFecZ7ePeB0GpvVA24hSNENE  
0r3tUeku0f1I0FunMnMnbh7Z09rPYqWvWDNIpU3S4CjFhY82L+IeKnmLy8N6ASRk  
HsPiNCOHSK8C/0ynrd9xLhX8Jsk/TGiQYaleoHhWkNL1ZsL86QHL8SKEqkqZCQf5  
AEqghDP6NEGS71n0enA7JjIrA9KL1T7fnNWZ0wFi5X+o/CymE2ytEMS0Yf3nmY4U  
n9x56Wgn6J2zqB5nq0Xf6NxDGAIg0Bm098YEnKCIFzk+yhoD1prVpHcnd2b5f60q  
uh8KY0EbKgpMJ3zZuWSL5kwGF1nNoYiAkonMaz9H3p0Qn0MVYCUeUTDRsi0/prrd  
UhNlry4TASBMpeXnFhdLVM3vFQZVpByadG0JNmnaN/Wavw2a00UGBFa4wsF9BBMB  
CgAnAhsDBQsJCACDBRUKCQgLBRYCAwEAAh4BAheABQJhMqGaBQkLn1UVAaoJEKwQ  
ezhmktrd2sQP/3YHM+U+Bb0y1nSEAFykZ71+uCM2hkHMLdxQYWB/rBWkmg/pbu+d  
r4t45RsTASrNjRcZ0nt1PMQRIq973ymHfpmes+noFwvTGH7zDv1BRBR9wPrd1XUz  
iSuEUHGi/fqxUVXQ5mbonzfThX8tuXeuIQmeToqoB00FY1Zm6xsNnEHcjV166mC4  
IPoJLWnZJs4r0CeoRf5XvDTgX6xt5/kLYRZf79qaWGFvaZpsc1CH+rQJUdVa/D4T  
7pI7hX6zy0S91z4iuC5HZUi0TF+y5auEZHGtdTWNS1kv0vfCCTi0XK/GkGL82SZu  
7X2VGnpCeUnFyViRG1k+KaDG1sVyDY+1cBPg6ilr45M6MQV0iHS50F04QNXSKt5+  
UnzJH71ldgNsR6ibRMyNV3k5v3fyUcSBvIYyLORTTBiVEjQDSbk1QNqbrQ1X9CWz  
+EJWn16BFTmMFvxBSWPm640GncHP5J3/0MbMw3Cm90x7k8UfNANIemcrJrSxIDwm  
g9cVAg3a+D+wxjrVe8jGg0ejvECpm+0yswigj5x6Lqj09A4UgdjEauN+/pn0nhBo  
Gv7DzMXtM/LoDtgP6wn93qZVN2TsuHnkEk4UyntB6eWJbBdXHWUr47exiWh0dvQN  
tpwCWPT6I7ZTPtA5K/zx+q9m6797BLgAkTYc6gloQL3vs1Z1S3m/hZNawsF9BBMB  
CgAnAhsDBQsJCACDBRUKCQgLBRYCAwEAAh4BAheABQJgmrz2BQkLBnBxAAoJEKwQ  
ezhmktrd36oP/2rB2EkWSOCKC4m0heWSfDWi60BKOebBDtFtc6/HwqBW8SPsiK1q  
zV0e3qBY/LVju04+ktJEK+EGXLnC3iC36MegrQ8zt391kEx/Zv9LIuV0CX90QIAX  
dL8MVUkkjRLCFFH8pTgRy1cJYwk1X4dYdXWYc29fCwNVartNdNBhsb2ht3VJeKDE  
kUivBHmkjuISDPEnI1coY7Lj0ZtY5cHdRF2eZpB0RkTBpsIt18rCYyHkERZrhmb  
j3r0yPyv0a+1/dQs8/hv5pEmbKx8cy8RdJkmbUHYatPBsjHkJSWr707G9VFW4GoN  
9CRAI4KkbDSEDjCL5dv2pq0Sew1MkLuWJGULAMgiIU1Wc0s5SZZGFSksNQrtSFV9  
Z/wGocecMGkGQNXQ06JV/Fry/TvyphBlmylEqL+NLqEcEjnlz90IVu+ZA+M09J96  
ULH07V5GvBgM+QK/q/dJeMHPWrNlo1gA6Nw1/HBdM0DqzdZ2jEPvsQSABvZrPMty  
+BAqEar4wqY1AH4X5ccEj07nJQoBQSDRSki1fkBsc1nx44N/m0kHdIa0Z/Y+Mw4v  
WiZhRek0ospG1I41Ba3CNTVAhSs9msGsYfkqvFJGHL7sZY8XSv82GBBvA0nUNrsJ  
bLBwo2FaQG9eoatRAGkqp4b/0tNtBuGeiQoNwFGbfUZTAaStj5/zZj0sWSF9BBMB  
CgAnAhsDBQsJCACDBRUKCQgLBRYCAwEAAh4BAheABQJe+9bwBQkJZ4p1AAoJEKwQ  
ezhmktrd+ScP/RoaUKriVVAglH0Gs+/mnfKtnfTlClzi5dsdI9/6H0vLpmSWK/Cl  
2cT6gary45VMgAeVK+H1lQXafYj+FY++I5kYoe2GrSvIXhpjaFAJyNf/dK1eTsqr  
Tm371i8b3FDYs5kvy2CnTbmHB8Ms0Gxck8/YHd1x+g8Wp02IgF89yYCSF3CAdx3  
6bHbs6Z3C3lcm/3SoWF+Yie2P8XeBMPCGp/BcjQzUcHF6G06TwDDYhixucUi6vEY  
EH5Jt0wVVQ7bubT80Fe0oJwVx1zYz4UoqxjKDWymarTzu03AUIT0PXPece94bJAK  
mSh68ItQe3H8tSPMubERWz2tEV31VlChDGXcC7BYQmxHseolxz/qzCtJ0iX9BvZR

dnizNeNJ/Cu8M2pDp47zdNFXzf/Q/sQ9pQ1ws22G2g119rWDneBku9n1vTP80/er  
SB+VLTBjDiAr1CY5y9+BG8wbscExJySoQxkB9j/n1MzPY5rgk0SyxsNj9GbqH+hr  
EjS3/uacNwSLxGcOT2E9Teot5pfTE06fQVq+35QhfA1P8c8jze01W/+u+wXu1Ui9  
azRSzYtCHanGyyet6U1mlBpAkqkZzH6t3CA5czc9i6FbzjvFVZnbRUZIRzfISYew  
1F5WqgTn2iYVdxagPRvLF5kjD696brGW9d5HwirCVGaK04VsXW1Ab1B9wsF9BBMB  
CgAnBQJXdYAFaHsDBQkHhh+ABQsJCACDBRUKCQgLBRYCAwEAAh4BAheAAAoJEKwQ  
ezhmktrdWigP/3QW17a081BUWyby4HEhN4SdAoWGY/FLq04mCtup1cnMgRUCSiL9  
12BSCTMctUcdSwTtYw0gSchN2mMsd1U2FNR5HvNunYR/pFdqjfQurf1ZmKVeG5/4  
uuKa0xMw9e8pK5uYAFs+07gr8gu/f6/Drp7NZk3/yVKpf4WCY9oX9TA1q90/11nN  
cwS45U/d7YP+N1YM9cBXa1DnDcdfm0BlykzouAF0qd1Lwi/tmLENvybD3+2c2WsE  
r1FZGSa5Zaf00tTIWxh5k6wh5FdRRycrnSyRK3B9N9+yaXfMQ0Xp0ypa8dqQEnCi  
IsngDCJPxtTrhMWKhBFRUMzK/WZTDboTQSQDK+YVRrE4K8MtoZSKwZLV2r903TpX  
kpbKsPVYmexerfdMeZfjZMF1bC7BmEs7jciH6JjbqAoAPnHzN0481aeNarINSViX  
PQWr2mp9qShei2/RavLtx2ZNrmGW72ZKpF8E3WWUDpBJqFVeGNRv0m3aZj8o/H1  
ewtNjcT4ouJfq1fKiULv+g7ANEMDLQTFDTg5twRdvmZ1B7oTBSavf+LwxPIXhH32  
IR7TX7VeicMMxmZnmZK2ANT/QBi3laf+ojVHvB+f6D74eLNq0Zqjfi/3UFNysYjg  
E+YgCqEUBpHb161n0HwG0SsQwfap2uKK1zukD/KxH5SPBC3DYGBI+KCbzsFNBfd1  
gAUBEAC8zNArPwb3dPMThL2xAY+fS60vXdB1Sk0tYJpDWPfGvo0d+VQ+hV6XulGA  
HAS6xG1WHysPT9KejIRSgLG+e9CaM5yhsxNa1WFGUM4Q9ESo3t+a75Go7xHIxgFj  
C046/06Vh3g9N/PREeuG8zkZ3H2v5fmD+ejyPgk4W9sFL00zjRiZD0FKVYR/j9ue  
nEC/2NBcLuFy3q6CdFmCoDE0062kXMnaGz3knzEK/X1SkcsxRDq7zaQ1Q1Kou+3  
dICwy4x5SJQ8j1+eeeEvF2C2/dXmDohb57tqUwioohMUQkmCtvZgEHjypUwgp0MT  
o25gWxkvJ1SJKU0b6b1786WnySIzF2gxqlkkEmB14RAssQkeXjrSmGwsMDyHNqyJ  
eYFus18sPaSpo+V2n0z+2B070Uq+wmf1S5A5FpegH0PZzzoNZo8I6QxaZje9YSZU  
ijGmZIdEB1eRVt3Svhi8MY1nasd4bW2RK1sr7plkBf8QRe6biiQRF3KD0Sn5CbmX  
pAchJ1ZHzRRdkXZDNQC6vCJxsy1300TrhJtAV1Yq347uyUbVi291ISVgroUVtprs  
mHoEk5Go0THbg9SCSt+xi/FiJQC+ubWmIGXoFKMR3UmhDnnzobKcbnbs/Hd981Fd  
VghYYvq//gTakJk0WxfGq030wtXRndPOA0T+qhP3TE+LtGRJ+wARAQABwsF8BBgB  
CgAmAhsMFIEE/rkgnY8vP0ZkhB5VrBB70GaS2t0FamjXZm4FCRNDGegACgkQrBB7  
0GaS2t3y5g/7BFXp/fdanzuQPTToJTPen7AVwhLloKaiYhG3GjdXfMPLvu6UtaaGm  
qynLo1UNNoobptFqc1G9BKoAghQrta7CsDhtsQF2xyc3Mfu0gmpL/7X5a7sFIeJ  
j08UjfweHx4DSG4LEZgNaAoWFjZ1tp4+8cqijKAHxt+r+1ayQG4VVH0WyXXqmSH4  
9HqtbPcPyRzxdovLeshZC9jmhHhhKqw/LwGyipWSOUKQDjWarBwdyhNmWCaLvXH1  
ndMp4tq8DPGC3G4T9tYAbANrn7nKfZgHebMSzMw9kSp0L6QvwWTDjJyIWz85WyeH  
WHeBysDaB0it3XD1ehUew27y7N6a9hQSYjnXuwwre5mjdIOqJon/31R6ui2Z1y9P  
a+bC11hbLXXh9tLCXRuo0t6thh9Cq5X1a76PPpEv30o3bpsb612hbrut10KezwvK  
17txito/jfMiWfsZHA904SoM+8GnmVingHtZ805n1T4RddJvT/vaqplfI6zf7jmf  
a691ALP420riF0QcwntNUM5tVmFUZsnFp2YRd4Ls7MiXVjtABahLSbb9415WSVc0  
jr0LDf94edvzk4R8i20b8CfVZNqEsTR6bHz8dT7Q+xQzEdjUujyyZY1UU1157Qeb  
0sHjhCtuZYCI04X9hZ37nKnZXSxR1RDCnt5BEiyFu2WD1RscUe6PcVDCwXwEGA EK  
ACYCGwwWIQT+uSCfLy8/RmSEH1WsEHs4ZpLa3QUCaNd1PQUJE0MYuAAKCRCsEHs4  
ZpLa3XCpD/42DrcveE+q2ulrAIYDPD1U1HiwIMEjqBDRm6zmr1KSAeb4E6/MFcP4s  
rXSSscMlrqG6NVynjNCXjD2YzWii68EwoXLJkgoD3r2ifzkV62EX2MIEeNZAVwuy  
KNxorzmy6bhuWltRYNK/hITS2AG5or0k9ADEJ8PixKymrWlhesPaWX6Yhp9/tWaC



RHOSRiLbRVaJ+7sqT88urLmkV9Hqx949Zxv4+cgBVUGL6WXXsfWhHjbDMNJnozWB  
SZaIJznLAp0M8z+1DNrQYyfrR8SkF4IOVmg6HDzoyuseJJ8JvMA1kvT6F9VBq/iE  
yeDYdEEQxwHwozKrEx5Ybx15mntbqwCXy6kHSx2+/3RZWpZQ8K29YP9QEk0KeGF8  
9Vap3jjNrx4u3cuRNQpeblQc4uFn3Nzaj+cVV4YzcRw94NifecXpujSvk8XU2ytJ  
/JgMBxPIBKglN4eEMet9b4FRB5XeBdPam19/LXyb4I1IipGNXlgNz/HCuBzidzHT  
QQdqfA9rZVx1hwFr7AJCVqWaXVsx1oEAhKqpTtsLMyj594DvnRuwKw5Vse+1eydW  
MIHYdbxmJccsTGIIt/hsOp8zfzfm+QYk5752jshh0KEBy+Ey3QZI1Wb0547N0b2Hwr  
Pgt7fw2NCKMPElSu98zmneFPhqNHf7L5urBe5gADj81E8lm6t/oVxcLBfAQYAQoA  
JgIbDBYhBP65IJ8vLz9GZIQeVawQezhmktrdBQJnABCLBQkRa8qFAAoJEKwQezhm  
ktrde3MP/13CLWp99XvRR0rzD/bW0fwjAenT2PE/tYd0Y9YcTQFbnIUhaVUDWAo3  
pibR3D4u9L1Y4olpGfJ7BTIHFa9myfpaVvmrNjueYI4omli24JQ/CKqNdY8Qzxxz+  
/QyiNK7Aw5cEBWIu84WGB1SsefWWT3rZe9YBb77gNcWHZ15pXTXrcgUxGY4808MC  
I9YFWq8EA0iHawtFnmB3UFfC1Wt37Hy3PKvr1is3uG60+ULI8RQz3/+ZwSG8U+xt  
b+I7H9+gITc1eFCb+tIwp5xWflyxcFXyk6Uz0L7y3Fg2tIEuSNtIHUC9NDVobf6c  
I0KAzZcMvKiPQiuBnV0jgDLmCZM5H6axj9x+gi4oVh6ea3HLqMzyjm5JkeCGgKWv  
H0gD3yGEZDvcbavkQ0le5T+4JefndKzCPrLuX0iyx+oQii0L8WieSSkSB6BsZcUN  
SeuGJwM79Y70qlD/YVrQNBZj5Vz+m3nZ+0EWDDMI0hRgMpSEIc+dnTC0u103Z+Rc  
c2IJq8INmU653sUcfCZE12ParW4rF7ib6kViYrABT8f4e2TP0a0yP5kp51ied9qL  
azaBA6tt/C9X1V2EJZK4srXtmcZ02Im45RAiVXyfpBAmmiF3eZWcbKe7qBC4rDRh  
LZG4RQW/S86Da0BID7gQz9IFSkaG504MsDhvnA7iAqaHUHUpCsiwsF8BBgBCgAm  
AhsMFiEE/rkgny8vP0ZkhB5VrBB70GaS2t0FamUkSiQFCQ+P/Z8ACgkQrBB70GaS  
2t3AwA/9GkXKUgVjKGCxwE4SdDt7c2jw6to2TTP9iFJ3Xbk3+5BURT3gkZCuu9D7  
gt+97aVo/B4EM7Xz8DQKyY7Ic9VAwDRra/Hwi1V0hw1zyIWQ/gAnX3baU6qLRWHR  
vVR5meV8r35C+rg9DaWFYmvS7PIv9LfxESwBPUjbmX8k4/5EJpHUwf12bzkTnot5  
7q5lHxKQa6IvqQak+Hp9ZM2KPdsgK02HWJJIIvYcI5byW9zBKV007YR8gtRAJKp9  
IbtsXx0WT6cqH0FVc5SSzdcaMt0gLF17BTnJyvKK2l9GABGBmzYDjeCyF2J+Ippf  
oqxqfTe6Eo0suEMc2PbLTs9SswjyCC2VG1X8+uUH9SoKwL0VQ6LFsP6fhkVKqi/a  
rB6UuPR/iZnrKIuxMNQ4U+t2Q6UdMlMxsAXTndkwzoK9oJRokIrH0ZV1KtH4sJJ  
A tCic+tOddq+GQLiKe2WpJfx1A0uESCB0TxjAwQmfn1H+dUhPeL1bNimH1H0/hXPd  
ifuNGozzADIRseQDyzjl8xGL1qRZLD3cfmda6RyZ+S3dQRuaRrcFCDccpY/p0+F8  
jbx64zyqqNs+KV+SkQG0cKFhWTZGCfQ/zMDtDmQKjb3eTAkv1zdE0Mw9zEjJmS0q  
8FNl+2w03VnvXwvBbtDdVCIAIq+jVcsy5XtnnV+bJ19Q9yue/XvCwWUEGAEKAA8C  
GwwFamEyoZoFCQueVRUACgkQrBB70GaS2t1uHBAAh0YVvrtchRmzCvdNER1DtkIs  
bgQPJ90xbyfvmvoD06qxH7PrycLZKbt7yYpAUU/CMc86GwaEe0I5Nm1CTs6NvDIv  
g3e7EPIs859tyQf1bM56N1wbsopCuoCJYknuroIf/M6dW6vJKNXLMmnL/AtalUBw  
X+5pblmGUUJep49oT0xQEnvnuqyvaGjXgFXix5PVFJD2ed5NnQeFpvcCpc/ioN0j  
z70R082j1ht5nWqPraXX5AYhQFM/kwR1cK4LV7gVDd/q+dfGYHzpxQ/HtyX/Lasi  
N6I52QqA95SM1ZZLPFLaNH6EvnB7uC9pLCYS8nvi1X7/cez5PFFf1e1gXCOT0jv3  
mJ2exLmXV0BbFKgjjcFCxhRdRLtukfiDfJkySy1zdsncpfng8wJ3xKRv43cUTz7M  
Z240YNMqK26aJZVXEQUYjCwsBylY/F5wjYAWgwZ8yF5Rfix28P/K8JsIHb3QrAJK  
sNWQAb03ZWis3N3spR5M9Mw3VuDZ3WUXq7mxB5M3kpVoZ3vETU5cwTbADYNP4Sw  
BDK2uIVtxabezSBtz0FcyYoF+0W8q7r4WvoyC9/+3GfnozZLJcEIVDk4W2pMW4A  
UhG/6drKTm3HkSDWIDu7d1sHWMffLEYfUhtN5DKkDkGoPfHvZvu9teR5yLfUrPTf  
ktihPn/JMrmwa9pwi8LCwWUEGAEKAA8CGwwFamCavPcFCQsGcHIACgkQrBB70GaS



```

2t0uaA//UWRaRiHEAKeRqBG/T2ak+XZJNu7QHfNgoUEAub9Zru8oPPXx2AJLcHEN
KWmeFLLxADdW0Zs4Bm9o0ew3VQnR/dBqjnXfob9Rc+eYUjA3rXazM/QrqcU8Syi3
MjNGUmjdL5aQF+IppAMg0BLG1TEnM7C5/PvrGJuYpGEnkKEwMK/GYhqg2V60pHEV
Pvs66mefJpCzbZSy56qtknSt6yBNWc14XgDX6VTn2kW4CV/3vVJUuvjvYs9SPyY8
mKEXa6QvUd3PcXv6RiWk4lGYuT1+jh2VkcFQ+JnUwv9TbKFB9b5jq1bvW9+LMDEl
YXux7pBP5Rpk+OLpyiExIRFWhi3x7aMW0zQ+I9yuNTEYkTHiEAQRUhs/1Fh4oLgI
v9QZgC0mRSN3zm8plQdivs1Z1AosAqqkA9BQwqsgosQe7P92irYIJqay0si9wGCD
wSMsmeXdIF6wW3/UMJZL66aarPeiZApGX0QdTzwjMh/QK/8gTKyeZulKmNkNfwWq
0170irWqLKssVHTg3VUM8EIdh+oNqDDXSeWtYUmpPpWp+yWZ0x1MFFZhuQHQTGu
TIj4A92LQzbrfj/jXRvWm2SrJMivUoiDUn+qxKIpVwFli5gVb+uyTFhw89Pckphr
JwRi052RLouY9d6Ek46UH4XfZZWzuzY+zzB7oqG0NphLgi/h3DCwWUEGAEKAA8C
GwwFAL771b8FCQlniTUACgkQrBB70GaS2t2/MxAAjoEGPdZavhs0lXdPCrd1D5QJ
r8T/NSEV2z1cp8ZvdrkjNF09TBP4qsBnKJiuvY1Iw70GX9W2okvXxgJizE45v9MH
WEMz4hmIjmAfRwcqENgp0c1IY/T0/+kkCW8dB6d30J1kT0n2PCRzN9L5vPqZXGTG
mLvd9M0jH1256w4uxLb+e1HMDTCqEN1ppq9G+EAR/29q8JZWs1marbZZWxSWcg/E
1YYbNafzk1gjq4CLh/j8AEWSvLr39zRy9uvQ/yqAKZ4K4aZfh/SPupGDvsD6ZK54
EPHxErQ7aiXTbUHTvwhxWLOP6WmxFA3Shr6L6YUb6jq+0PVliFC517g3mxFHJtw
yXGNiKhzmzr0190lsHafuLj/9QPfK3Ce32SkPhW/11MYA8HzduMv5Arp7cBczXSP
EUTmNIVKv3gTjSQrzRhwhHmMuqyDZ/rXQ01j12sxIDj04MUMvVjYKF+OCNm42gVs
8ca3/wN9ZNU6hyFWeKQDuCAqPPbT5G0/DKseFEwB+07wwyH1RXbyl0v4fneg605X
S71qhNtw2p1hDL0HYHDiV+aPZ+LoOmX6+dmnqE6bQJaIlVb922KWml1i07F3DkqP7
0jF1hoE1gfiXWkxP4Gy8w0obNfEMgvz02djKGQy+oQqNdIcZFZgzPTGKB/nVgpt
9CcRDWjPltFCd2e1FBbCwWUEGAEKAA8FALd1gAUCGwwFCQeGH4AACgkQrBB70GaS
2t1PIQ//Qc5VYfBCxpaMysaPQ44wXPEZSjxIGZhhMGzb1UzzAEY0w+RgKN5nNTXq
L2Ko0k0rGnKqZ0KByMdXwIPH/rGwwEsbbIpopnibf5ic5B/+xCTIK+qLIwX2ZLuk
NhbL6Y+E+7DxMMh+KqBWH0NKKgwVY+rFW0foops839ABKvc9/Ry4/qqkcb40AzpD
11iQJ5vK/DMuaDWxWeKXqJLI13WMGPcPfheuBZL1u7LEEHYKMgzvpbF81WIn3MBo
8jvxf2/o+kMafSSDqgv0u6yu8G0hmScpCbRjN7jV/HrG+tM+zy48TN6/MkGWSR7q
TD34pqBjyatVfVl6dGD6xj/i/Emt5hZB6qXruCDH7AWMoNx+FkDubs4sc4PKysZU
Itya6KdQFo2UeYsNwZhdn6QwKhd85um4JUHJCY0mARvjsQgWXH/5MR40cow77bbE
vVq0XNd+QRVlyT42CEtnIUOFLeDVuZrum5Tuvvna6ImMDoi/z6QcNeL79XsY2m6I
QVRiHr1BDdb/8JLkfnWiwL8GRv169Kf8unx0y5u1YBpcMYkyDD2+pnnk3TY0rR+8X
8goecaS8fbyu/Q48K85ZMD8wKW/bzLQ+tK9y8xed24u2QERftMhIw9b6f45Nrrf/
PhgV8RnuwUusSbdDe8kw3eYtmLdzD4kZc9K7Sd02CqT+hm//9JI=
=uGHC
-----END PGP PUBLIC KEY BLOCK-----

```

## Previous keys

### Important

New keys are created before the previous ones expire. As a result, at any given moment in time, more than one key may be valid. Keys are used to sign artifacts starting the day they are created, so use the more recently-issued key when the validities of keys overlap.

### Expiration date: 2025-10-04

Key ID	0xAC107B386692DADD
Type	RSA
Size	4096/4096
Created	2016-06-30
Expiration date	2025-10-04
User ID	AWS SDKs and Tools <aws-dr-tools@amazon.com>
Key fingerprint	FEB9 209F 2F2F 3F46 6484 1E55 AC10 7B38 6692 DADD

To copy the following OpenPGP public key for the SDK for Java to the clipboard, select the "Copy" icon in the upper right corner.

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

```
Comment: Hostname:
```

```
Version: Hockeypuck 2.2
```

```
xsFNBFd1gAUBEACqbmFbxdJgz1lD7wrlskQA1LLuSAC4p8ny9u/D2zLR8Ynk3Yz
mzJuQ+Kfjne2t+xTDex6MPJlMYp0viSwsX2psgvdmeYUpW9ap0lrThNYkc+W5fRc
buFehfbi9LSATZGJi8RG0sCCr5FsYVz0gEk85M2+PeM24cXhQIOZtQUjswX/pdk/
KduGtZASqNAYLKROmRODzUuaokLPo24pfm9bnr1RnRtw5ktPAA5bM9ZZaGKriej
kT2lPffBbjp8F5AZvmGLtNm2Cmg4FKBvI04SQjy2jjrQ3wBzi5Lc9HTxDuHK/rtV
```

u6PewUe2WP1nx1XenhMZU1UK4YoSB9E9StQ2VxQiySLHSdxR7Ma4WgYdVLn9b0ie  
nj3QxLuQ1ZUKF79ES6JaM4t0z1gGcQeU1+UklgjFLuKwmzWRdEIFfxMyvH6qgKnd  
U+DioH5mcUwhwffAAsuIJyAdMIEUYh7IfzJJXQf+fF+XfOC16byOJFWrIGQkAzMu  
CEvaCfwtHC2Lpzo33/WRFeMAuzzd0QJ4uz4xFFvaS0SZHMLHWI9YV/+Pea3X99Ms  
0Nlek/LolAJh67MynHeVB0HKrq+fLuoRwepQivctzN6Y1N0kx5naTPGGaKWK7G2q  
TbcY5SMnkIWfLFSougj0Fvmjczq8iZRwYxWA+i+LQvsR9WEXEiQffIWRoQARAQAB  
zSxBV1MgU0RLcyBhbmQgVG9vbHMgPGF3cy1kci10b29sc0BhbWF6b24uY29tPsLB  
1AQTAQoAPgIbAwULCQgHAwUVCgkICwUWAgMBAAIEAQIXgBYhBP65IJ8vLz9GZIQe  
VawQezhmktrdBQJnABcIBQkRa8qDAAoJEKwQezhmktrdl1MQAIwEuDar30TxkfTa  
cPNKDnNzxaWqrxZ3FTQ+PyrHhQ6usxxrvDKJS+uCjE9bmWHVFU1R4yQNF+721Jdw  
5UhX0u+ZgT9afApE65uAZuwLhPsz8upXT8C6VeKXh3shdw7qXi2hrwtM1a0Pls40  
Cs2C9rLUDMJTySrVDDVwpnaAB+8DcFrs9bIt5Q3gd0UatdzDvcB7QKh9jUvzCpbE  
cInb1epDN5MRzowMR4iU2VV1RzLxCvm7CQSyXfgf0DFLkXWiknh0q9eINmytJFG/  
ntFdiZFkNZ5hP709loywdfNrmqB6PsF8BPGFh8gKw1pjowrfHpv6cNIqShmA76LT  
30HVi0lqGFB7obffq//eZGPR0oYJFDrOdD2CFRoHnP3N++AfKA4SRN7eXwyoZ6Pk  
Do9WNIEEkAcp6PGvv7AokogDo/40qmxgC6fN+3BT0stWpv4F1D4Nx0ZWsTs49wxg  
kPlCCVf8t75aZzkCjXng1eClZZQ5SB1RtSB7gMqtP7MIn2J5w8spNbs5xQvJc76u  
NvzwEasPkY+UcHd05Rdd0UwoKqDerLUG7Yqd1NCJoQR1mBIgZButbQ1MyaZcmQq0  
iR0kwDi9h6D16fnUb2dFCNJw+eDHvsjG8HI3IVZM10bUQ2kmmwr102YQ1ynJQm01  
lMl1I4hFU8/lHNHm8ie5darpVXQEwsGUBBMBCgA+AhsDBQsJCACDBRUKCQgLBRYC  
AwEAAh4BAheAFiEE/rkgnY8vP0ZkhB5VrBB70GaS2t0FAMUkSiIFCQ+P/Z0ACgkQ  
rBB70GaS2t3HVg//S+/Kbe0Bf+nCdHsrWtp9kxvWIpAGvQhIbxx1tp/impfm+5Rm  
fKPD0KX+g42fuMmOdDE4gj04GjGd7ZY3bx+0zbDSdVebzmYCbPZ/BDP990oPKidd  
w6G18PaIyqfuARK0ESBETvAwNgw04t2ocjs4pYZV+CuHvESYpquHjmHtye6ajZW  
Mv24NhjVo4EfP33dPugTjXLjeuGT7qQpsYV3a66juHmPVkXwuPqxh9wTnc5TU6FG  
UPSfIGMPL0xha7Rg2i5zvRaAxx4bHqG08IAz/l/E/tJkV5xnt494HQam9UDbiFI0  
Q0TSve1R6S45/UjQW6cycyduHtk72s9ipa9YM0ilTdLgKMWFjzYv4h4qeYvLw3oB  
JGQew+I0I4dIrwL/TKet33EuFfwmyT9MaJBhQV6geFaQ0uVmwvzpAcvxIoSqSpkJ  
B/kASqCEM/o0QZLuWc56cDsmMisD0ouVPt+c1Zk7AWLlf6j8LKYTbK0QxLRh/eeZ  
jhSf3HnpaCfonb0oHmeo5d/o3EZ0AiA4GbT3xgScoIgX0T7KGg0WmtWkdYd3Zv1/  
o6q6Hwpg4RsQckwnfNm5ZiVmTAYXWc2hiICSicxrP0fek5Cc4xVgJR5RMNGyI7+m  
ut1SE2WvLhMCwEy15ecWF0tUze8VB1WkHJp0Y4k2ado39Zq/DZrTRQYEVrjCwX0E  
EwEKACcCGwMFCwkIBwMFFQoJCAsFFgIDAQACHgECF4AFAMeyoZoFCQueVRUACgkQ  
rBB70GaS2t3axA//dgcZ5T4Fs7LWdIQB/KRnvX64IzaGQcwt3FBhYH+sFaSaD+lu  
752vi3jlGxMBKs2NFxk6e2U8xBEir3vfKYd+mZ5L6egXC9MYfvM0/UFEFH3A+t3V  
dT0JK4RQCaL9+rFRVdDmZuifN90Ffy25d66JCZ50iqgHTQViVmbrGw2cQdyNWxRq  
YLgg+gktadkmzis4J6hF/le8N0BfrG3n+QthF1/v2ppYYW9pmmxzUIf6tAlR1Vr8  
PhPukjuFfrPLRL3XPiK4Lkd1SI5MX7Llq4RkcZN1NY1LWS8699wJ0LRcr8aQYvzZ  
Jm7tfZUaekJ5ScXJWJEaWT4poMbWxXINj6VwE+DqKWvjKzoxBXSIIdLk4XThA1dIq  
3n5SfMkfuWV2A2xHqJtEzI1XeTm/d/JRxiG8hjIs5FNMGJUSNANJuTVA2putCVf0  
JbP4Q1afXoEV0YwW/EFJY+brjQadwc/knf/QxsZDcKb3THuTxR80A0h6ZysmtLEg  
PCaD1xUCDDr4P7DG0tV7yMaDR608QKmb7TKzCKCPnHouqPT0DhSB2MRq437+mfSe  
EGga/sPMxe0z8ug02CnrCf3ep1U3Z0y4eeQSThTKe0Hp5Y1sF1cdZSvjT7GJaHR2  
9A22nAJY9Pojt1M+0Dkr/PH6r2brv3sEuACRNhzqCWhAve+zVnVLeb+Fk1rCwX0E

EwEKACcCGwMFCwkIBwMFFQoJCAsFFgIDAQACHgECF4AFAMcavPYFCQsGcHEACgkQ  
rBB70GaS2t3fqg//asHYSTBI4IoLibSF5ZJ8NaLo4EqgRts00W1zr8fCoFbxI+yI  
qWIrNXR7eoFj8tW07Tj6S0kQr4QZcucLeILfox6CtDz03f3WQTH9m/0si5U4Jf3RA  
gBd0vwXVSSSNEsIUUfy10BHLVwlhaTVfh1h1dZhzb18LA1Vqu0100GGxvaG3dU14  
oMSRSK8EeaS04hIM8ScjVyhjsuPRm1j1wd1EXZ5mkHRGRMGmwi3XysJjIeQRFmuG  
a9uPes7I/K85r7X91BLz+G/mkSZsrHxzLxF0mSZtQdhq08GyMeQlJavs7sb1UVbg  
ag30JEAjgqRsNIQ0MIv12/amrRJ7DUyQu5YkZQsAyCIhSVZw6z1J1kYVKSw1Cu1I  
VX1n/Aahx5wwaQZA1dDTolX8WvL90/KmEGWbKUSov40uoRwS0eXP3QhW75kD4zT0  
n3pSUc7tXka8GAz5Ar+r9014wc9as2WjWADo3CX8cF0zQ0rN1naMQ++xBIAG9ms8  
y3L4ECoRqvjCpjUAfhflxwSM7uc1CgFBINFKSLV+QGxzWfHjg3+bSQd0hrRn9j4z  
Di9aJmFESQ6iykbUjiUFrcI1NUCFKz2awaxh+Sq8UkYcvux1jxdK/zYYEG8DSdQ2  
uwlssHCjYVpAb16hq1EAaSqnhv86020G4Z6JCg3AUZt9R1MBpK2Pn/NmPSzCwX0E  
EwEKACcCGwMFCwkIBwMFFQoJCAsFFgIDAQACHgECF4AFA1771vAFCQ1nimUACgkQ  
rBB70GaS2t35Jw/9GhpQquJVUCAsc4az7+ad8q2d90UKX0L12x0j3/ofS8umZJYr  
8KXZxPqBqvLj1UyAB5Ur4fWVBdp9iP4Vj74jmRih7YatK8heGmNoUAnI1/90qV50  
ypF0bfvWLxvcUNizmS/LYKdNuYcHwyw4bFyTz9gd3XH6Dxak7YiAXz3JgJIXcIB3  
ELfpsduzpncLeVwz/dKHxY5iJ7Y/xd4Ew8Ian8FyNDNRwcXobTpPAMNiGLG5xSLq  
8RgQfkm07BVVDtu5tPw4V46gnBXGXNjPhSirGMonbKZqtP07TcBQhPQ9c95x73hs  
kAqZKHrwi1B7cfy1I8y5sRFbPa0RXeVWQKEMZdwLsFhCbEex6iXHP+rMK0nSJf0G  
91F2eJk140n8K7wzak0njvN00VfN/9D+xD21CXCzbYbaDXX2tY0d4GS72fW9M/zT  
96tIH5UtMGM0ICuUJjnL34EbzbuxwTENjKhDGQH2P+eUzM9jmuCTRLLGw2P0Zuof  
6GsSNLf+5pw3BIVeZw5PYT1N6i3m19MQ7p9BWr7f1CF8CU/xzyPN7TVb/677Be7V  
SL1rNfLNi0IdqcbLJ63pTwaUGkCSqRnMfqc3ID1zNz2LoVv008VVmdtFRkhHN8hJ  
h7CUX1laqB0faJhV3FqA9G8sXmSN3r3pusZb13kfCKsJUzorThWxdaUBuUH3CwX0E  
EwEKACcFAlD1gAUCGwMFCQeGH4AFCwkIBwMFFQoJCAsFFgIDAQACHgECF4AACgkQ  
rBB70GaS2t1aKA//dBaXto7zUFRBjvLgcSE3hJ0ChYZj8Uuo7iYK26mVycyBFQJK  
Iv2XYFIJMwK1Rx1Ja1jA6BIKE3aYyx2LVTYU1Hke826dhH+kV2qN9C6t/VmYpV4b  
n/i64po7EzD17ykrm5gB+z47uCVyC79/r80uns1mTf/JUq1/hYJj2hf1MDWr07/X  
Wc1zBLj1T93tg/43Vgz1wFdrU0cNx1+bQGxKT0i4AXSp3UvCL+2YsQ2/JspF7ZzZ  
awSuUVkZJrllp87S1MhZeHmTrCHkV1FHJyudLJErcH0337Jpd8xDRek7K1rx2pAS  
cKIiyeAMik/G10uExYqEEVFQzMr9Z1MnuhNBjAMr5hVGsTgrwy2h1IrBktXav07d  
0leSlsqW9ViZ7F6t90x5l+NkwXVsLsGYSzuNyIfomNuoCgA+cfM3TjzVp41qsg1J  
WJc9Bavaan2pKF6Lb9Fq8u3HZk2u+YZbvZkqkXwTdZZQ0kEmoVV4Y1G86bdpmPyj  
8eV7C02NxPii4l+qV8qJQu/6DsA0QwMtBMUN0Dm3BF2+ZmUHuhMGxq9/4vDE8heE  
ffYhHtNftV6JwwzGZmeZkrYA1P9AGLeVp/6iNUe8H5/oPvh4s2rRmqN+L/dQU1ix  
i0AT5iAKoRQGkduXrWc4fAY5KxDB9qna4oqX06QP8rEf1I8ELcNgYEj4oJv0wU0E  
V3WABQEALzM0Cs9Zvd08x0EvEBj59LrS9d0HVkQ61gmKakWC+jR35VD6FXpe6  
UYAcBLrEbVYfKw9P0p6MhFKAsb570JoznKGzE1rVYUZQzhD0RKje35rvkajvEcjG  
AWMLTjr87pWHeD0389ER64bz0Rncfa/1+YP56PI+CThb2wUvTTONGJkPQUpVhH+P  
256cQL/Y0Fwu4XLerpwN+YKgMQ47raRcydobPeSfMQr9fVKRy0zFE0rvNpCVDUqi  
77d0gLDLjH1I1Dy0X5554S8XYLb91eY0iFvnu2pTCKiiExRCSYK29mAQePK1TCCn  
Qx0jbmBbGS8mVIkpQ5vpvXvzpY3JIjMXaDGqWSQSYGXhECyxCR5e0tKYbCwwPIc2  
rI15gW6yXyw9pKmj5XafTP7YHTvRSr7CZ/VLkDkW16AfQ9nP0g1mjwjPpDFpmN71h  
JlSKMaZkh0QGV5FW3dK+GLwxiWdqx3htbZErvWyvumWQF/xBF7puKJBEXcoM5KfkJ

uZekBwcnVkfNFF2RdkM1ALq8InGzLXc7R0uEm0BXVirfju7JRtWLB3UhJWCuhRW2  
muyYegSTkag5MduD1IJK37GL8WILAL65taYgZegUoxHdSaE0ef0hspxuduz8d33z  
UV1WCFhi+r/+BMCQmTRbF8ao7fTC1dGd084DRP6qE/dMT4u0ZEn7ABEBAAHCwXwE  
GAEKACYCGwwWIQT+uScfLy8/RmSEHlWsEHs4ZpLa3QUCZwAXCwUJEWvKhQAKCRCs  
EHs4ZpLa3XtzD/9dwi1qffv70UTq8w/21jn1owHp09jxP7WHTmPWHE0BW5yFIW1V  
A1gKN6Ym0dw+LvS5W0KJaRnyewUyBxWvZsn6Wlb5qzY7nmCOKJpYtuCUPwiqjXWP  
EM8c/v0MojsuWMOXBAViLvOfhgdUrHn1lk962XvWAw++4DXFh2deaV0163IFMRm0  
PNPDAiPWBVqvBANiH2sLRZ5gd1BXwpVrd+x8tzyr69YrN7hutP1CyPEUM9//mcEh  
vFPSbw/i0x/foCE3NXhQm/rSMKecVn5csXBV2J01Mzi+8txYNrSBLkjbSB1AvTQ1  
aG3+nCNCgM2XDLyoj0IrgZ1To4Ay5gmTOR+msY/cfoIuKFYenmtxy6jM8o5uSZHg  
hoClrx9IA98hhGQ73G2r5EDpXuU/uCXn53Sswj65bl9IssfqEIoji/FonkkpEgeg  
bGXFDUnrhicD0/W0zqpXf2Fa0DQWY+Vc/pt52ftBFgwzCNIUYDKUhCHPNZ0wtLtd  
N2fkXHNiCavCDZ10ud7FHHwmRNdj2q1uKxe4m+pFYmKwAU/H+Htkz9Gjsj+ZKedY  
nnfai2s2gQ0rbfwvV9VdhCWSuLK17ZnGTtiJu0UQI1V8n6QQJpohd3mVgmynu6gQ  
uKw0YS2RuEUfV0vOg2tASA+4EM/SBUpGhud0DLA4b5w04gKmh1B1HqQrIsLBfAQY  
AQoAJgIbDBYhBP65IJ8vLz9GZIQeVawQezhmktrdBQJ1JEokBQkPj/2fAAoJEKwQ  
ezhmktrdwMAP/RpFy1IL4yhgscB0EnQ7e3No80raNk0z/YhSd125N/uQVEU94JGQ  
rrvQ+4Lfve2laPweBD018/A0Csm0yHPVQMA0a2vx8ItVdIcNc8iFkP4AJ192210q  
i0Vh0b1UeZnlfK9+Qvq4PQ21hWJr0uzyL/S38REsAT1I25sfJOP+RCaR1MH9dm85  
E56Lee6uZR8SkGuiL6kGpPh6fWTNij3bICjth1iSSCL2HCOW8lvcwSldDu2EfILU  
QCSqfSG7bF8dFk+nKhzhVX0Uks3XGjLdICxZewU5ycryitpfRgARgZs2A43gshdi  
fiKaX6Ksan03uhKDrLhDHNj2y07PUrFo8gggtlRpV/Pr1B/UqCsC9FU0ixbD+n4ZF  
Sqov2qwe1Lj0f4mZ6yiLsTDOUFPPrdk01HTJZ17AF0zXZMM6CvaCUaJCKx9GVdSrR  
+LI4wLQonPrTnXavhkC4intlqSX8ZQNlHeggdE8YwMEJn59R/nVIT3i5WzYph5R9  
P4Vz3Yn7jRqM8wAyEbHkA8s45fMRi9akWsw93H5nWukcmfkt3UEbmka3BQg3HKWP  
6TvhfI28euM8qqjbPilfkpEBjnChYVvk2Rgn0P8zA7Q5kCo293kwJL9c3RDjMPcxI  
45ktKvBTZftsDt1Z718LwW7Q3VQiGiKvo1XLMuV7Z51fmydfUPCcrnv17wsF1BBgB  
CgAPAhSMBQJhMqGaBQkLn1UVAaOJEKwQezhmktrdbhwQAITmFb67XIUZswr3TRED  
Q7ZCLG4EDyFTsw8n75r6A90qsR+z68nC2Sm7e8mKQFFPwjHP0hsGhHtCOTZtQk70  
jbwyL4N3uxDyEv0fbckH5Wz0ejZcG7KKQrqAiWJJ7q6CH/z0nVurySjVyzJpy/wL  
WpVAcF/uaW5Zh1FCXqePaEzsUBJ757qsr2ho14BV4seT1RSQ9nneTZ0Hhab3wqXP  
4qDTo8+zKtVnNo9YbeZ1qj6211+QGIUBTP5MedXCuClE4FQ3f6vnXxmB86cUPx7c1  
/y2rIjei0dkKgPeUjNwWSzxS2jYehL5we7gvaSwmEvJ74pV+/3Hs+TxX39XtYFwj  
k9I795idnsS511dAW3yoI3HBQsYa3US7bpH4g3yZMkstc3bHJ6X54PMcd8Skb+N3  
FE8+zGduDmDTKitumiWVvxEFGIwsLAcWPxecI2AMIMGfMheURYsdvD/yvCbCB29  
0KwCSrDVkAG9N2VorNzd7KUeTPTMN1bg2d11F6u5sQeTN5KVaGd7xE10XME2wA2D  
T3+EsAQytriFbcWm3s8Ugbc9BXMmKBfjlVku6+Fr6Mgvf/txn56M2SyXBCFQ50Ft  
qTFuAFIRv+nayk5tx5Eg1iA7u3dbB1jH3yxGH1B7TeQypA5BqD3x72b7vbXkeci3  
1Kz035LYoT5/yTK5sGvacIvCwsF1BBgBCgAPAhSMBQJgmrz3BQkLbnByAAoJEKwQ  
ezhmktrdLmgP/1FkKwYhxAcnkagRv09mpP12STbu0B3zYKFBALm/Wa7vKDz18dgC  
S3BxDs1pnhZS8QA3Vjmb0AZvaDnsN1UJ0f3Qao5136G/UXPnmFIwN612szP0K6nF  
PEsotzIzRlJo3S+WkBfiKaQDIDgSxtUxJz0wufz76xibmKRhJ5ChMDCvxmIaoNle  
tKRxFT770upnnyaQs22UsueqrZJ0resgTVnNeF4A1+1U59pFuA1f971SVLr472LP  
Uj8mPJihF2ukL1Hdz3F7+kYlp0JRmLk9fo4d1ZHBUPiZ1ML/U2yhQfW+Y6tW71vf

```
izAxJWF7se6QT+UT5Pji6cohMSERVoYt8e2jFjs0PiPcrjU3mJEx4hAEEVIbP9RY
eKC4CL/UGYAtJkUjd85vKZUHYr7NWZQKLAKqpAPQUMKrIKLEHuz/doq2CCamstLI
vcBgg8EjLJn13SBesFt/1DCWZeummqz3omQKR19EHU2cIzIf0Cv/IEysnmbpSpjZ
DX8Fqjtezoq1qiyrLFR7YN1VDPBCHYfqDagw10nlrWFJqT6VqfslmdMdTBRWYVEB
0GUxrkyI+APdi0M2634/410b1ptkqyTir1KIg1J/qSsiKVcBZS0YFW/rskxYcPPT
wpKYaycEYt0dkS6FPcnehJ001B+F32WVq2bs2Ps8we6KhjjaYS4Iv4dwwsF1BBgB
CgAPAhSMBQJe+9W/BQkJZ4k1AAoJEKwQezhmktrdvzMQAi6BBj3c2r4bDpV3TwkX
dQ+UCa/E/zUhFds9XKfGb3a5IzRdPUwT+KrAZyiYrr2NSM0zh1/VtqJL18YCYsx0
0b/TB1hDM+IZiI5gH0cCHKhDYKTnNSGP09P/pJA1vHQend9CdZE9J9jwkcZfS+bz6
mVxkxpi73fTDox9dues0LsS2/ntRzA0wqhDdaaavRvhAEf9vavCWVrNZmq22WVsU
lnIPxNWGGzWn85JYI6uAi4f4/ABFkry69/c0cvbr0P8qgCmeCuGmX4f0j7qRg77A
+mSueBDx8RK002o1021B7b8IcVizj+lpsRQN0oa+i+mFG+o6vtD1ZYhQude4N5sR
RybcLclxjSCoZs5q9JfTpbB2n7pSf/UD3ytwnt9kpD4Vv9dTGAPB83bjL+QK6e3A
XM10jxFE5jSFSr94E40kK80YcIR5jLqsg2f610ENY5drMSA4zuDFDL1Y2ChfjgjZ
uNoFbPHGt/8DfWTV0ochVnikA7ggKjz20+RjvwyrHhRMAft08MMh9UV28pdL+H53
o0t0V0u5aoTbcNqdYQy9B2Bw4lfmj2fi6Dpl+vnZp6h0m0CWiJVW/dtilppYjuxd
w5Kj+9IxZYaBNYH4l1pMT+BsvMDqGzXxDIL89NnY5BkMvqEKnjXSHGRWYMz0xigf
51YKbfQnEQ1oz5bRQndntRQWwsF1BBgBCgAPBQJXdYAFaHsMBQkHhh+AAAoJEKwQ
ezhmktrdTyEP/0H0VWHwQsaWjMrGj000MFzxGUo8SBmYYTBs29VM8wBGDsPkYCje
ZzU16i9iqDpDqxpymTigcjhV8CDx/6xsMBLG2yKaKZ4m3+Yn0Qf/sQkyCvqiyMF
9mS7pDYWy+mPhPuw8TDIfiqgVhzjSpIMFWPqxVjn6KKbPN/QASr3Pf0cuP6qpHG+
NAM6Q5dYkCebyvwzLmg1sVnil6iSyJd1jBj3D34XrgWS9buyxBB2CjIM76WxfNVi
J9zAaPI78X9v6PpDGn0kg6oLzrusrvBjoZknKQm0SZ+41fx6xvrTPs8uPEzevzJB
lkke6kw9+KagY8mrVX1ZenRg+sY/4vxJreYWQeq167ggx+wFjKDcfhZA7m70LH0D
ysrGVCLcmuinUBaN1HmLDcGYXZ+kMCoXf0bpuCVByQmNJgEb47EIF1x/+TEeNHKM
0+22xL1atFzXfkEVZck+NghLZyFDhS3g1bma7puU7r752uiJjA6Iv8+kHDXi+/V7
GNpuiefUYh69QQ2//CS5H51osC/Bkb9evSn/Lp8dMubtWAaXDGJMgw9vqZ55N02N
K0fvF/IKHnGkvH28rv0PCv0WTA/MClv28y0PrSvcmXnduLtkBEX7TISMPW+n+0
Ta63/z4YFfEZ7sFLrEm3Q3vJMN3mE5i3cw+JGXPSu0nTtgqk/oZv//SS
=bboB
-----END PGP PUBLIC KEY BLOCK-----
```

Expiration date: 2024-10-08

Key ID	0xAC107B386692DADD
Type	RSA
Size	4096/4096
Created	2016-06-30



Expiration date	2024-10-08
User ID	AWS SDKs and Tools <aws-dr-tools@amazon.com>
Key fingerprint	FEB9 209F 2F2F 3F46 6484 1E55 AC10 7B38 6692 DADD

To copy the following OpenPGP public key for the SDK for Java to the clipboard, select the "Copy" icon in the upper right corner.

-----BEGIN PGP PUBLIC KEY BLOCK-----

```
xsFNBFd1gAUBEACqbmmbFbxdJgz1lD7wrlskQA1LLuSAC4p8ny9u/D2zLR8Ynk3Yz
mzJuQ+Kfjne2t+xDex6MPJlMYp0viSWsX2psgvdmeyUpW9ap0l1rThNYkc+W5fRc
buFehfbi9LSATZGJi8RG0sCCr5FsYVz0gEk85M2+PeM24cXhQIOZtQUjswX/pdk/
KduGtZASqNAYLKR0mRODzUuaokLPo24pfm9bnr1RnRtw5ktPAA5bM9ZZaGKriej
kT2lPffBbJp8F5AZvmGLtNm2Cmg4FKBvI04SQjy2jjrQ3wBzi5Lc9HTxDuHK/rtV
u6PewUe2WP1nx1XenHMZU1UK4YoSB9E9StQ2VxQiySLHSdxR7Ma4WgYdVLn9b0ie
nj3QxLuQ1ZUKF79ES6JaM4t0z1gGcQeU1+UklgjFLuKwmzWRdEIfxMyvH6qgKnd
U+DioH5mcUwhwffAAsuIJyAdMIEUYh7IfzJJXQf+fF+Xf0C16by0JFWrIGQkAzMu
CEvaCfwtHC2Lpzo33/WRFEmauzzd0QJ4uz4xFFvaS0SZHMLHWI9YV/+Pea3X99Ms
0Nlek/LolAJh67MynHeVB0HKIrq+fLuoRwepQivctzN6Y1N0kx5naTPGgaKWK7G2q
TbcY5SMnkIWfLFSougj0Fvmjczq8iZRwYxWA+i+LQvsR9WExEiQffIWroQARAQAB
zsFNBFd1gAUBEAC8zNAiPwb3dPMThL2xAY+fS60vXdb1Sk0tYJpDwPfgvo0d+VQ+
hV6Xu1GAHAS6xG1WHysPT9KejIRSGLG+e9CaM5yhsxNa1WFGUM4Q9ES03t+a75Go
7xHIxgFjC046/06Vh3g9N/PREeuG8zkZ3H2v5fmD+ejyPgk4W9sFL00zjRiZD0FK
VYR/j9uenEC/2NBcLuFy3q6cDfmCoDE0062kXMnaGz3knzEK/X1SkcjsxRDq7zaQ
lQ1Kou+3dICwy4x5SJQ8jl+eeeEvF2C2/dXmDohb57tqUwioohMUQkmCtvZgEHjy
pUwgp0MTo25gWxkvJlSJkU0b6b1786WnySIzF2gxqlkkEmB14RAssQkeXjrSmGws
MDyHNqyJeYFus18sPaSpo+V2n0z+2B070Uq+wmf1S5A5FpegH0PZzzoNZo8I6Qxa
Zje9YSZUijGmZIdEBleRVt3Svhi8MYlnasd4bW2RK1sr7plkBf8QRe6biiQRF3KD
OSn5CbmXpAchJ1ZHRRdkXZDNQC6vCJxsy1300TrhJtAV1Yq347uyUbVi291ISVg
roUVtprsmHoEk5Go0THbg9SCSt+xi/FiJQC+ubWmIGXoFKMR3UmhDnnzobKcbtnbs
/Hd981FdVghYYvq//gTAKJk0WxfGq030wtXRndPOA0T+qhP3TE+LtGRJ+wARAQAB
wsF1BBgBCgAPBQJXdaYAFahsMBQkHhh+AAAoJEKwQezhmktrdTyEP/0H0VWHwQsaW
jMrGj00MFzxGuo8SBmYYTBs29VM8wBGDsPkYCjeZzU16i9iqDpDqxyqmTigcjh
V8CDx/6xsMBLG2yKaKZ4m3+Yn0Qf/sQkyCvqiyMF9mS7pDYWy+mPhPuw8TDIfiqg
VhzjSpIMFWPqxVjn6KKbPN/QASr3Pf0cuP6qpHG+NAM6Q5dYkCebyvwzLmg1sVni
16iSyJd1jBj3D34XrgWS9buyxBB2CjIM76WxfNViJ9zAaPI78X9v6PpDgN0kg6oL
zrusrvBjoZknKQm0SZ+41fx6xvrTPs8uPEzevzJB1kke6kw9+KagY8mrVX1ZenRg
```

```
+sY/4vxJreYWQeq167ggx+wFjKDcfhZA7m70LH0DysrGVCLcmuinUBaNIHmLDcGY
XZ+kMCoXf0bpuCVByQmNJgEb47EIFlx/+TEeNHKM0+22xL1atFzXfkEVZck+NghL
ZyFDhS3g1bma7puU7r752uiJjA6Iv8+kHDXi+/V7GNpuiEFUYh69QQ2//CS5H51o
sC/Bkb9evSn/Lp8dMubtWAaXDGJMgw9vqZ55N02NK0fvF/IKHnGkvH28rv00PCv0
WTA/MClv28y0PrSvkvMXnduLtkBEX7TISMPW+n+0Ta63/z4YFfEZ7sFLrEm3Q3vJ
MN3mE5i3cw+JGXPSu0nTtgqk/oZv//SS
=Z9u3
-----END PGP PUBLIC KEY BLOCK-----
```



# Document history

This topic describes important changes to the AWS SDK for Java Developer Guide over the course of its history.

Change	Description	Date
<a href="#">the section called “Current key”</a>	Add new PGP key that expires on 2026-09-27.	October 1, 2025
<a href="#">Unsupported code patterns</a>	Document the unsupported code patterns of the migration tool.	September 23, 2025
<a href="#">Migration tool</a>	Updates for GA release.	September 19, 2025
<a href="#">Metrics reference</a>	Add more details about metrics collected by the SDK.	September 3, 2025
<a href="#">the section called “Implement optimistic locking with the VersionedRecordExtension ”</a>	Add details about DynamoDB Enhanced Client's VersionRecordExtension .	September 2, 2025
<a href="#">Find applications using 1.x clients</a>	Add instructions to help identify applications using AWS SDK for Java 1.x clients by querying AWS CloudTrail events before migrating to version 2.	August 20, 2025
<a href="#">Transfer Manager</a>	Add comprehensive migration tables for client constructors, methods, model objects, and behavior changes. Included detailed code examples	July 1, 2025

Change	Description	Date
	for unsupported methods requiring manual migration.	
<a href="#">Metrics</a>	Add comprehensive <a href="#">documentation for LoggingMetricPublisher</a> . Restructured the metrics topic with improved getting-started guidance.	June 20, 2025
<a href="#">the section called “High-level changes”</a>	Add information about empty string handling differences between DynamoDBMapper (v1) and DynamoDB Enhanced Client (v2) in the migration guide.	June 18, 2025
Table of contents reorganization	Add <a href="#">??? chapter</a> assembled from other sections of the guide.	June 16, 2025
<a href="#">???</a>	Add information about using the LegacyMd5Plugin for backward compatibility with systems that require MD5 checksums.	May 19, 2025
<a href="#">the section called “S3 client differences”</a>	Add S3 client differences between v1 and v2 of the AWS SDK for Java and provide migration examples if the migration tool cannot automatically migrate the V1 code.	April 24, 2025

Change	Description	Date
<a href="#">Deserialization differences</a>	Add deserialization difference between v1 and v2 of the SDK for Java.	April 10, 2025
<a href="#">the section called "SQS automatic request batching"</a>	Add migration content for SQS automatic request batching from v1 to v2 of the SDK for Java.	April 8, 2025
<a href="#">???</a>	Update information about automatic checksum calculations	April 3, 2025
<a href="#">the section called "Configure ALPN"</a>	Show configuration of ALPN protocol negotiation with the Netty-based HTTP client.	February 21, 2025
<a href="#">the section called "Lambda functions"</a>	Add information about using the EMF logging metric publisher with AWS Lambda to capture SDK metrics.	February 6, 2025
<a href="#">Implement ContentStreamProvider</a>	Add topic on how to implement a ContentStreamProvider .	January 29, 2025
<a href="#">the section called "Data integrity protection with checksums"</a>	Content updated with details about automatic checksum calculation.	January 16, 2025
<a href="#">the section called "Working with Amazon S3"</a>	Add migration content for working with Amazon S3.	January 8, 2025

Change	Description	Date
<a href="#">the section called “Access the AWS CRT-based HTTP clients”</a>	Add information about how to use a platform-specific jar with AWS CRT-based components.	November 14, 2024
<a href="#">the section called “Use IAM Roles Anywhere for authentication”</a>	Add information about how to use IAM Roles Anywhere for authentication.	November 6, 2024
<a href="#">the section called “Caching credentials configuration example”</a>	Add an example that configures a credentials provider by using the <code>asyncCredentialUpdateEnabled</code> property.	November 4, 2024
<a href="#">the section called “Use automatic request batching”</a>	Add a new topic that documents the Automatic Request Batching API for Amazon SQS.	October 23, 2024
<a href="#">OpenPGP key</a>	Update current OpenPGP key information.	October 10, 2024
<a href="#">the section called “Use complex types in expressions”</a> and <a href="#">the section called “Update items that contain complex types”</a>	Add content for how to work with complex types in expressions and updates.	October 10, 2024
Update Amazon S3 bucket names	Update Amazon S3 bucket names.	September 30, 2024
<a href="#">the section called “Optimize performance with account-based endpoints”</a>	Add information about AWS account-based endpoints for DynamoDB.	September 24, 2024

Change	Description	Date
<a href="#">the section called “Work with attributes that are beans, maps, lists and sets”</a>	Update section for DynamoDB Enhance Client that discusses working with attributes that are complex types.	September 6, 2024
<a href="#">the section called “Configure service clients to shortcut lookups”</a>	Clarify use of the <code>EnvironmentVariableCredentialsProvider</code> when Lambda SnapStart for Java is used.	August 19, 2024
<a href="#">the section called “Configure parallel transfer support”</a>	Add page with information on how to enable parallel transfer support	August 15, 2024
<a href="#">the section called “Generate a UUID with the AutoGeneratedUuidExtension”</a>	Add information about the DynamoDB Enhanced Client <code>AutoGeneratedUuidExtension</code>	August 14, 2024
<a href="#">???</a>	Add a section about the migration tool (preview release)	August 9, 2024
<a href="#">the section called “S3 Event Notifications”</a>	Add section that discusses how to work with the S3 Event Notifications API	July 21, 2024
<a href="#">the section called “Working with DynamoDB”</a>	Add v1 to v2 migration information for the DynamoDB mapping/document APIs	July 21, 2024

Change	Description	Date
<a href="#">the section called "S3 Event Notifications"</a>	Add v1 to b2 migration information for the S3 Event Notifications API	July 21, 2024
<a href="#">the section called "Retries"</a>	Add retry strategy topic	June 18, 2024
<a href="#">How to set the JVM TTL</a>	Remove instructions to set <code>networkaddress.cache.ttl</code> security property by using a java command-line system property.	May 21, 2024
<a href="#">the section called "Reduce SDK startup time for AWS Lambda"</a>	Update HTTP client recommendation to reduce startup time for AWS Lambda	May 14, 2024
<a href="#">the section called "Metrics reference"</a>	Reorganize metrics table items	May 1, 2024
<a href="#">the section called "Troubleshooting"</a>	Add troubleshooting topic.	April 26, 2024
<a href="#">the section called "Metrics collected with each request"</a>	Add new metrics reported by the SDK.	April 26, 2024
<a href="#">the section called "Set the JVM TTL for DNS name lookups"</a>	Change recommended DNS lookup TTL to 5 seconds.	April 23, 2024
<a href="#">the section called "Package name to artifactId mappings"</a>	Add package name to Maven artifactId mapping topic.	April 17, 2024
<a href="#">the section called "Metrics"</a>	Add configuration details to the metrics section.	April 12, 2024
<a href="#">the section called "IAM Policy Builder API"</a>	Add IAM Policy Builder API migration information.	April 11, 2024

Change	Description	Date
<a href="#">???</a>	Update HTTP proxy information.	April 3, 2024
<a href="#">the section called "Securely"</a>	Add instructions to disable IMDSv1.	March 14, 2024
<a href="#">the section called "Step-by-step instructions"</a>	Add step-by-step migration instructions.	March 8, 2024
<a href="#">Migrate to version 2</a>	Update migration topic.	February, 14, 2024
<a href="#">the section called "Configure AWS CRT-based HTTP clients"</a>	Add information about the synchronous AWS CRT-based HTTP client.	January 5, 2024
<a href="#">the section called "Amazon Cognito Identity" and the section called "Amazon Cognito Identity Provider"</a>	Amazon Cognito examples moved to Code Examples section.	December 28, 2023
<a href="#">OpenPGP key</a>	Provide current OpenPGP key.	December 6, 2023
<a href="#">the section called "Serialization differences"</a>	Describe serialization differences between v1 and v2 of the SDK for Java.	December 5, 2023
<a href="#">the section called "Transfer Manager"</a>	Add a section that details the changes in the S3 Transfer Manager from version 1 to version 2.	November 13, 2023
<a href="#">the section called "Annotation reference"</a>	Add a listing of data class annotations that can be used with the DynamoDB Enhanced Client.	October 30, 2023

Change	Description	Date
<a href="#">???</a>	Add information on the migration status of libraries and utilities from SDK for Java v1.x to v2.x	October 17, 2023
<a href="#">???</a>	Update the Gradle setup topic	October 17, 2023
<a href="#">the section called "Ignore null attributes of nested objects"</a>	Add information about the DynamoDB Enhanced Client <code>@DynamoDbIgnoreNulls</code> annotation.	September 22, 2023
<a href="#">the section called "Cross-Region access"</a>	Add information about cross-Region access to Amazon S3 buckets.	August 31, 2023
<a href="#">the section called "Preserve empty objects"</a>	Add section that discusses the <code>@DynamoDbPreserveEmptyObject</code> annotation.	August 25, 2023
<a href="#">???</a>	Update service client section.	August 15, 2023
<a href="#">the section called "Client recommendations"</a>	Since version 0.23, AWS CRT supports musl-based OS such as Alpine Linux. HTTP client recommendations now reflect the musl support.	August 11, 2023
<a href="#">the section called "Create IAM policies"</a>	Add IAM Policy Builder API section	July 31, 2023
<a href="#">the section called "Get started"</a>	Correct several snippets in the Get Started section of the DynamoDB Enhanced Client topic.	July 24, 2023



Change	Description	Date
<a href="#">the section called "Configure HTTP proxies"</a>	Add HTTP proxy support information and examples for each HTTP client.	June 2, 2023
Reorganize the table of contents	Promote <a href="#">Code examples</a> section and <a href="#">Calling AWS services</a> to top-level TOC entries.	May 24, 2023
<a href="#">the section called "Add logging dependency"</a>	Show Gradle dependencies in logging section.	May 23, 2023
<a href="#">the section called "Pagination"</a>	Update pagination topic.	May 18, 2023
<a href="#">the section called "Set up a Gradle project"</a>	Update Gradle project setup.	May 3, 2023
<a href="#">DynamoDB Enhanced Client API</a>	Rewritten DynamoDB Enhanced Client API topic released.	April 28, 2023
<a href="#">Update get started tutorial instructions</a>	Maven archetype modified to include option for credentialsProvider; instructions modified accordingly.	April 11, 2023
<a href="#">the section called "Client recommendations"</a>	Add HTTP client decision guidance	March 30, 2023
IAM best practices updates	Updated guide to align with the IAM best practices . For more information, see <a href="#">Security best practices in IAM</a> .	March 14, 2023
<a href="#">the section called "Reload profile credentials"</a>	Add section on reloading profile credentials.	February 9, 2023

Change	Description	Date
<a href="#">the section called "Configure AWS CRT-based HTTP clients"</a>	Update topic for GA release.	February 8, 2023
<a href="#">the section called "Work with Amazon EC2 instance metadata"</a>	Add guided example for Java SDK client for Amazon S3 instance metadata service.	February 1, 2023
<a href="#">the section called "Use a performant S3 client"</a>	Add section for the AWS CRT-based S3 Client.	December 19, 2022
<a href="#">the section called "Transfer files and directories"</a>	Update Amazon S3 Transfer Manager examples for GA release.	December 19, 2022
<a href="#">the section called "Best practices"</a>	Added best practices section.	November 18, 2022
<a href="#">the section called "Use an external process"</a>	Added section on loading credentials from an external process.	November 15, 2022
<a href="#">the section called "Metrics reference"</a>	Updated metric listing with HTTP client usage requirement.	November 9, 2022
<a href="#">the section called "Transfer files and directories"</a>	Example code corrected.	November 2, 2022
<a href="#">the section called "Reduce SDK startup time for AWS Lambda"</a>	Updated section with additional options to reduce Lambda startup time.	November 1, 2022
<a href="#">the section called "Configure HTTP clients"</a>	Added configuration information to cover all HTTP clients in the SDK.	October 26, 2022

Change	Description	Date
<a href="#">the section called "Logging"</a>	Updated logging topic to include wire logging details for all HTTP clients.	October 4, 2022
<a href="#">the section called "AWS database services"</a>	Added overview section of AWS database services and the SDK for Java 2.x.	September 13, 2022
<a href="#">EC2-Classic Networking is Retiring</a>	EC2-Classic is retiring on August 15, 2022.	July 28, 2022
<a href="#">the section called "Additional authentication options"</a>	Update to dependency required for single sign-on authentication.	July 18, 2022
<a href="#">the section called "Transport Layer Security (TLS)"</a>	Update TLS security information.	April 8, 2022
<a href="#">the section called "Additional authentication options"</a>	Added more information about setting up and using credentials.	February 22, 2021
<a href="#">the section called "Set up a GraalVM Native Image project"</a>	New topic for setting up a GraalVM Native Image project.	February 18, 2021
<a href="#">the section called "Waiters"</a>	Waiters released; added topic for the new feature.	September 30, 2020
<a href="#">the section called "Metrics"</a>	Metrics released; added topic for the new feature.	August 17, 2020
<a href="#">the section called "Amazon SNS"</a>	Added example topics for Amazon SNS.	May 30, 2020

Change	Description	Date
<a href="#">the section called “Reduce SDK startup time for AWS Lambda”</a>	Added AWS Lambda function performance topic.	May 29, 2020
<a href="#">the section called “Set the JVM TTL for DNS name lookups”</a>	Added JVM TTL DNS caching topic.	April 27, 2020
<a href="#">the section called “Set up an Apache Maven project”, the section called “Set up a Gradle project”</a>	New Maven and Gradle set up topics.	April 21, 2020
<a href="#">the section called “Transport Layer Security (TLS)”</a>	Added TLS 1.2 to security section.	March 19, 2020
<a href="#">the section called “Subscribe to Amazon Kinesis Data Streams”</a>	Added Kinesis stream examples.	August 2, 2018
<a href="#">the section called “Pagination”</a>	Added auto pagination topic.	April 5, 2018
<a href="#">???</a>	Added example topics for IAM, Amazon EC2, CloudWatch and DynamoDB.	December 29, 2017
<a href="#">the section called “Amazon S3”</a>	Added getobjects example for Amazon S3.	August 7, 2017
<a href="#">the section called “Asynchronous programming”</a>	Added async topic.	August 4, 2017
GA release of the <a href="#">AWS SDK for Java 2.x</a>	AWS SDK for Java version 2 (v2) released.	June 28, 2017